Module: Free Semester One Shot Exam Course

Notes

Free Semester Exam Course

Contains 13 PDF lessons

Generated: 6 April 2026

KnowledgeGate

Module

Free Semester One Shot Exam Course

Topic

Discrete Mathematics

Subtopic

DM One Shot

Lesson

Discrete Maths_Notes

SEMESTER EXAM SERIES

DISCRETE MATHS

IN 6 HOURS + NOTES

KNOWLEDGE GATE

Video chapters

- Chapter-1 (Set Theory):
- Chapter-2 (Relations):
- Chapter-3 (POSET & Lattices):
- Chapter-4 (Functions):
- Chapter-5 (Theory of Logics):
- Chapter-6 (Algebraic Structures):
- Chapter-7 (Graphs):
- Chapter-8 (Combinatorics):

http://www.knowledgegate.in/gate

KNOWLEDGEGATE

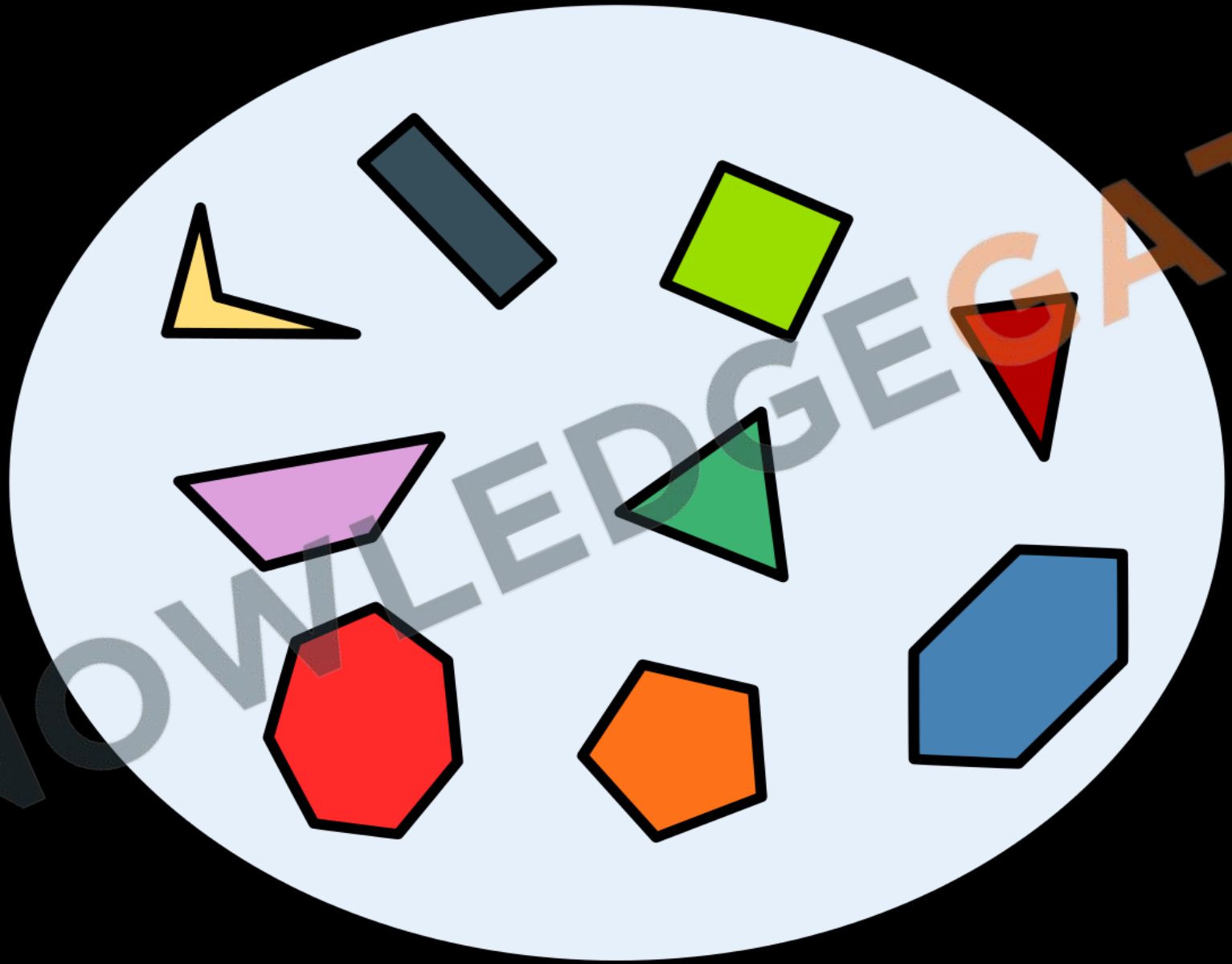
Chapter-1 (Set Theory)

KNOWLEDGEGATE

What is a SET

- Set are the fundamental discrete structures on which all the discrete structures are built. Sets are used to group objects together, formally speaking
- "An *unordered ,well-defined*, collection of *distinct* objects (Called elements or members of a set) of same type". Here the type is defined by the one who is defining the set. For e.g.
- A = {0,2,4,6, ---}
- B = {1,3,5, ---}
- C= {x| x ∈ Natural number}

KNOWLEDGEGATE



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- A Set is generally denoted usually by capital letter. The objects of a set called the elements, or members of the set.
- A set is said to contain its elements.
- Lower case letters are generally used to denote the elements of the set.

KNOWLEDGEGATE

- $x \in A$, means element x is a member of A
- $x \notin A$ means x is not a member of A

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Cardinality of a set** — It is the number of elements present in a Set, denoted like |A|.

- For e.g. A = {0,2,4,6}, |A| = 4

KNOWLEDGEGATE

Representation of set

- **Tabular/Roster representation of set** - here a set is defined by actually listing its members. E.g.

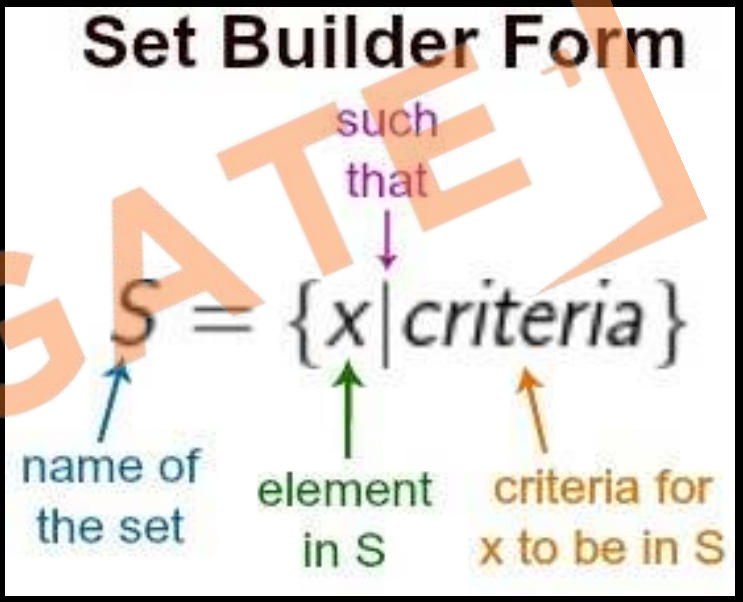
- A = {a, e, i, o, u}

- B = {1, 2, 3, 4}

- C = {---- -4, -2, 0, 2, 4, -----}.

KNOWLEDGEGATE

- **<u>Set Builder representations of set</u>**- here we specify the property which the elements of the set must satisfy. E.g.



- A = {x| x is an odd positive number less than 10},
- A = {x | x ϵ English alphabet && x is vowel}
- B = {x | x ϵ N && x <5}
- C = {x | x ϵ Z && x%2 = 0 }

KNOWLEDGEGATE

- **Finite set** - If there are exactly '*n*' elements in *S* where '*n*' is a nonnegative integer, we say that *S* is a *finite set.*
- *i.e. if a* set contain specific or finite number of elements is called is called finite set. For e.g. A = {1,2,3,4}

KNOWLEDGEGATE

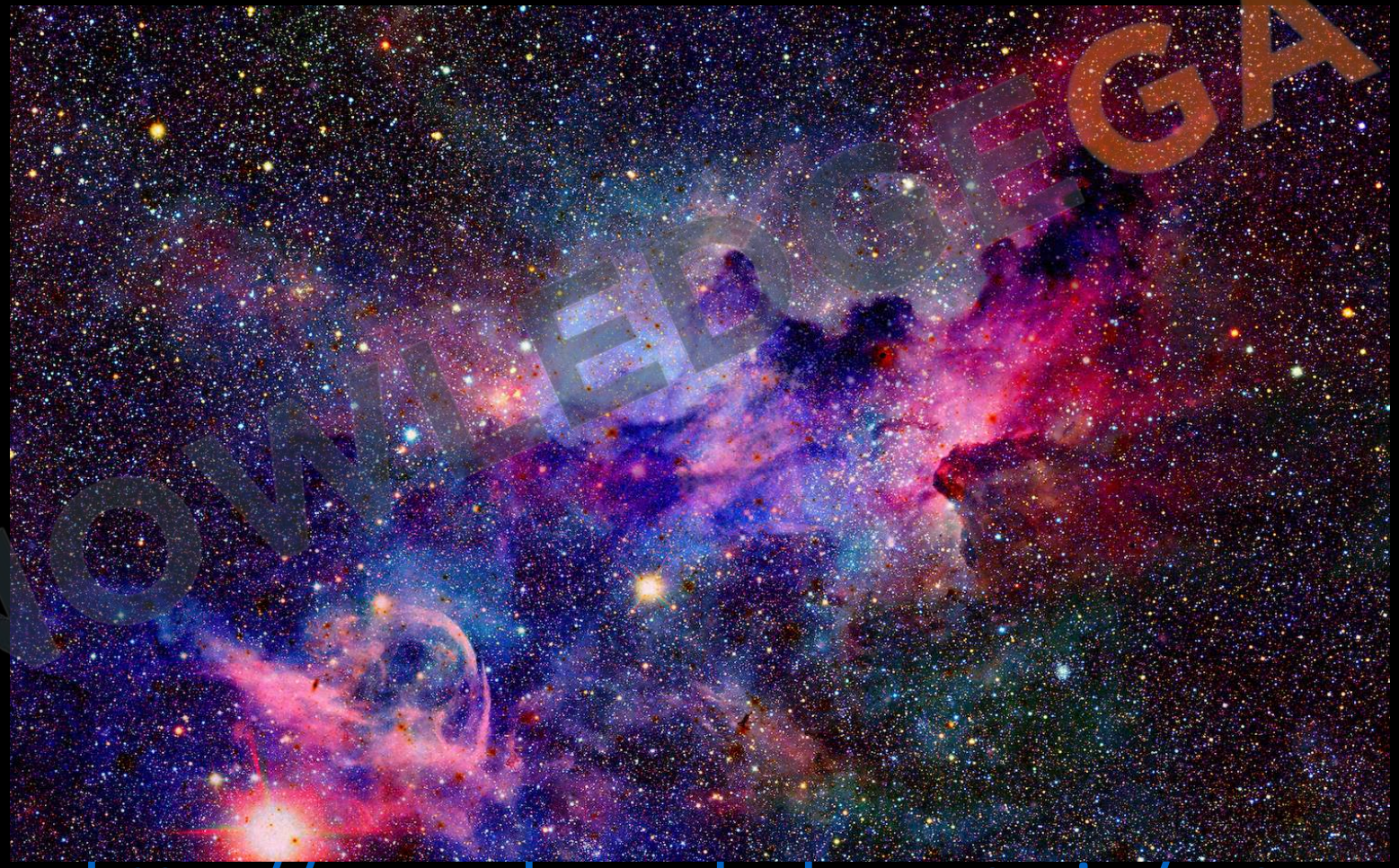
- **Infinite set** – A set contain infinite number of elements is called infinite set, if the counting of different elements of the set does not come to an end. For e.g. a set of natural numbers.

KNOWLEDGEGATE

- **Null set / empty set** - Is the unique set having no elements. its size or cardinality is zero i.e. $|\phi| = 0$. It is denoted by a symbol ϕ or {}. A set with one element is called singleton set.

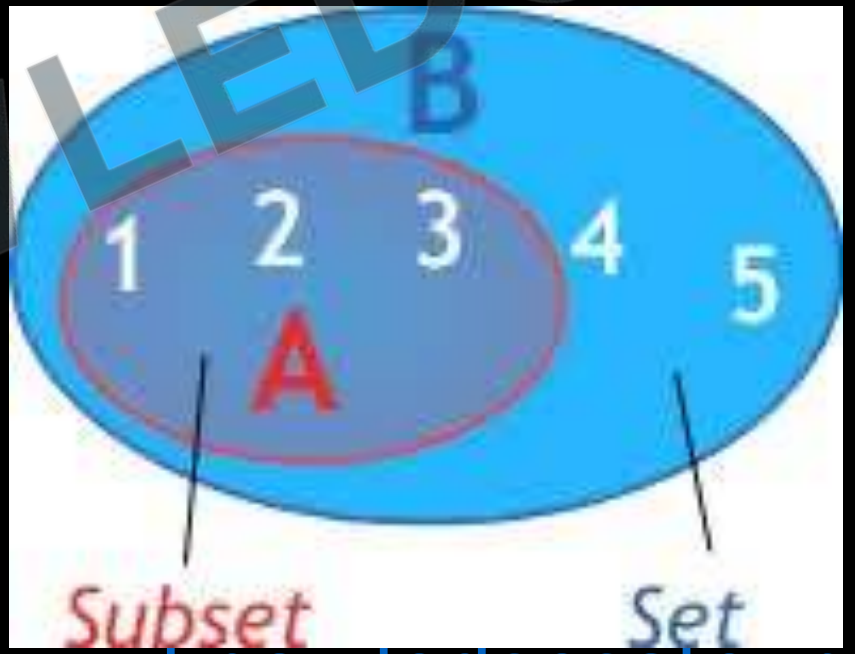
KNOWLEDGEGATE

- **Universal set** – if all the sets under investigation are subsets of a fixed set, i.e. the set containing all objects under investigation, in Venn diagram it is represented by a rectangle, and it is denoted by U.



KNOWLEDGE GATE
http://www.knowledgegate.in/gate

- **Subset of a set** – If every element of set A is also an element of set B i.e.
- $\forall x(x \in A \rightarrow x \in B)$, then A is called subset of B and is written as $A \sqsubseteq B$. B is called the superset of A.
- E.g. A = {1,2,3} B = {1,2,3,4,5}
- Note that to show that *A* is not a subset of *B* we need only find one element $x \in A$ with $x \notin B$. To show that $A \sqsubseteq B$, show that if $x \in A$, then $x \in B$.

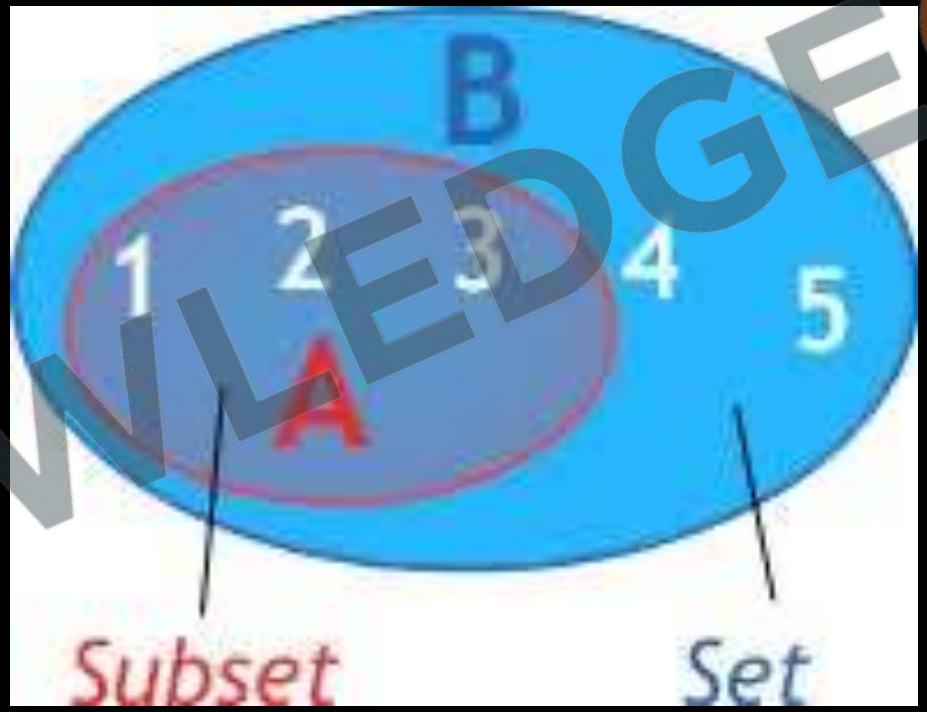


KNOWLEDGEGATE

- $\phi \sqsubseteq A$, Empty Set ϕ is a subset for every set
- $A \sqsubseteq U$, Every Set is a subset of Universal set U
- $A \sqsubseteq A$, Every Set is a subset of itself.

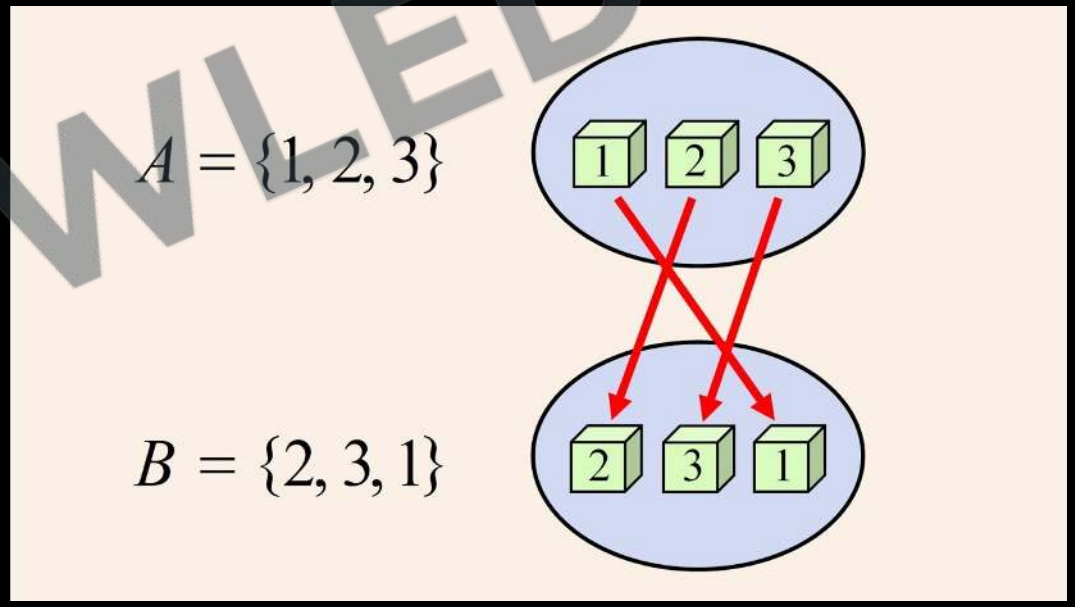
KNOWLEDGEGATE

- **Proper subset** – if A is a subset of B and A ≠ B, then A is said to be a proper subset of B, i.e. there is at least one element in B which is not in A. denoted as A ⊂ B.



KNOWLEDGEGATE

- **Equality of sets** – If two sets A and B have the same element and therefore every element of A also belong to B and every element of B also belong to A, then the set A and B are said to be equal and written as A=B.
- if $A \sqsubseteq B$ and $B \sqsubseteq A$, then A=B
- $\forall x(x \in A \leftrightarrow x \in B)$



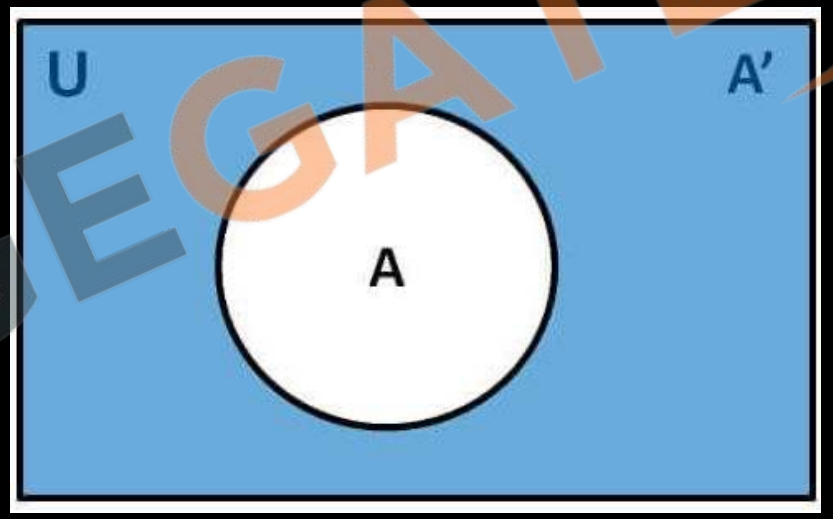
KNOWLEDGEGATE

- **Power set** – let A be any set, then the set of all subsets of A is called power set of A and it is denoted by P(A) or 2^A.
- If A= {1,2,3}, then P(A) = {ϕ, {1}, {2}, {3}, {1,2}, {2,3}, {1,3}, {1,2,3}}
- Cardinality of the power set of A is n, $|P(A)| = 2^n$

KNOWLEDGEGATE

Operation on sets

- **Complement of set** – Set of all x such that $x \notin A$, but $x \in U$.
- $A^c = \{x \mid x \notin A \ \& \ x \in U\}$



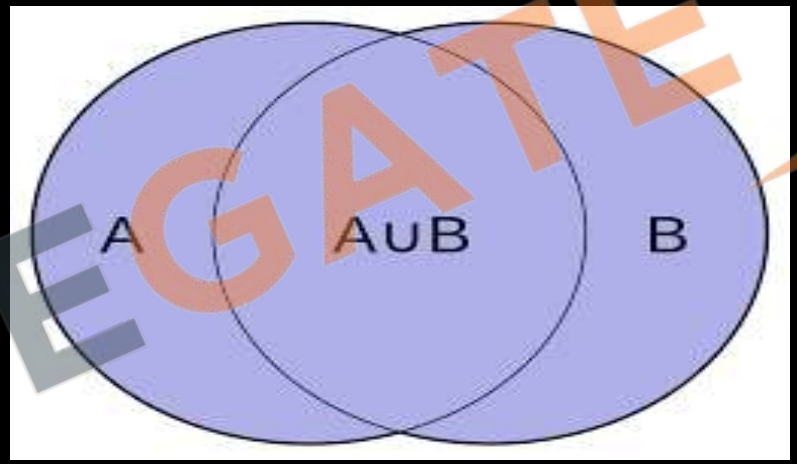
U = {1, 2, 3, 4, 5, 6}

A = {2, 3, 6}

A^c = { }

KNOWLEDGEGATE

- **Union of sets** – Union of two sets A and B is a set of all those elements which either belong to A or B or both, it is denoted by A U B.
- A U B = {x| x ϵ A or x ϵ B}



A = {1, 2, 3, 4}

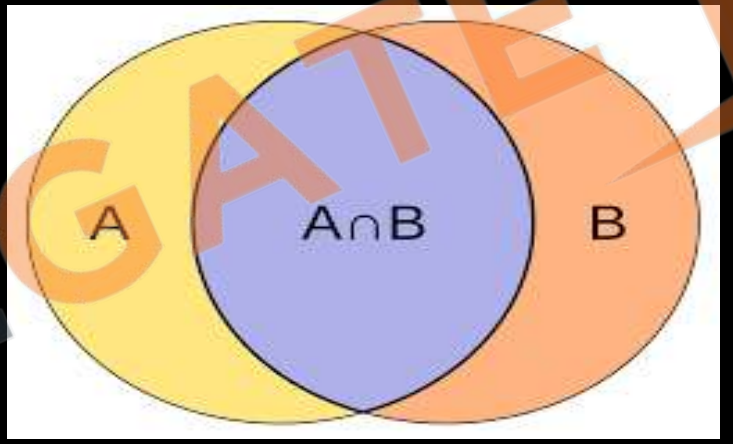
B = {3, 4, 5, 6}

A U B = { }

|A|+|B|=| A U B | ?

KNOWLEDGEGATE

- **Intersection of sets** - Intersection of two sets A and B is a set of all those elements which belong to both A and B, and is denoted by A ∩ B.
- A ∩ B = {x| x ∈ A and x ∈ B}



A = {1, 2, 3, 4}

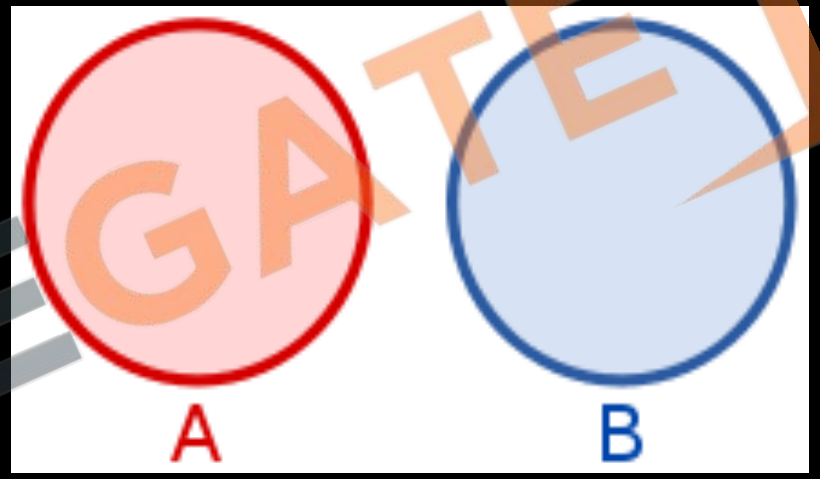
B = {3, 4, 5, 6}

A ∩ B = { }

KNOWLEDGE GATE

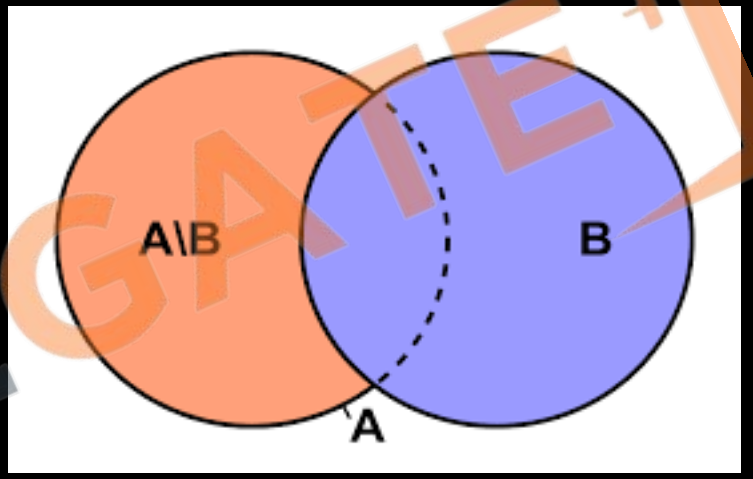
- **Disjoint sets** -- Two sets are said to be disjoint if they do not have a common element, i.e. no element in A is in B and no element in B is in A.

- $A \cap B = \phi$



KNOWLEDGE GATE

- **Set difference** – the set difference of two sets A and B, is the set of all the elements which belongs to A but do not belong to B.
- $A - B = \{x | \ x \in A \text{ and } x \notin B\}$



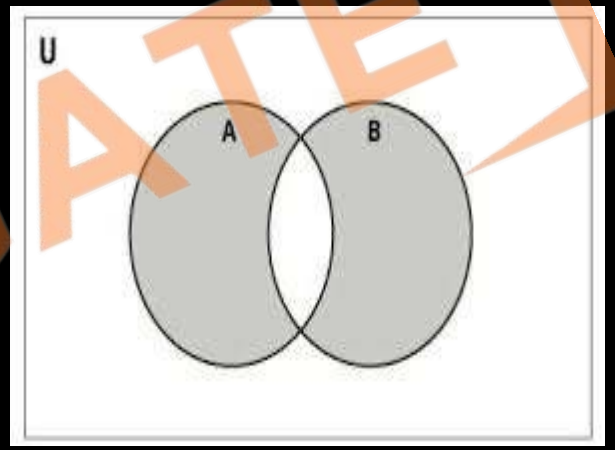
A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

A - B = { }

KNOWLEDGE GATE

Symmetric difference – the symmetric difference of two sets A and B is the set of all the elements that are in A or in B but not in both, denoted as. $A \oplus B = (A \cup B) - (A \cap B)$



$A \oplus B = \{x \mid (x \in A \text{ and } x \notin B) \text{ or } (x \in B \text{ and } x \notin A)\}$

$A \oplus B = (A - B) \cup (B - A)$

$A = \{1, 2, 3, 4\}$

$B = \{3, 4, 5, 6\}$

$A \oplus B = \{\quad\}$

KNOWLEDGEGATE

Idempotent law

- A U A = A
- A ∩ A = A

Associative law

- (A U B) U C = A U (B U C)
- (A ∩ B) ∩ C= A ∩ (B ∩ C)

Commutative law

- A U B = B U A
- A ∩ B = B ∩ A

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Distributive law

- $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
- $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$

De Morgan's law

- $(A \cup B)^C = A^C \cap B^C$
- $(A \cap B)^C = A^C \cup B^C$

Identity law

- $A \cup \phi = A$
- $A \cap \phi = \phi$
- $A \cup U = U$
- $A \cap U = A$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Complement law

- $A \cup A^C = U$
- $A \cap A^C = \phi$
- $U^C = \phi$
- $\phi^C = U$

Involution law

- $((A)^C)^C = A$

KNOWLEDGEGATE

Chapter-2 (Relations)

KNOWLEDGEGATE

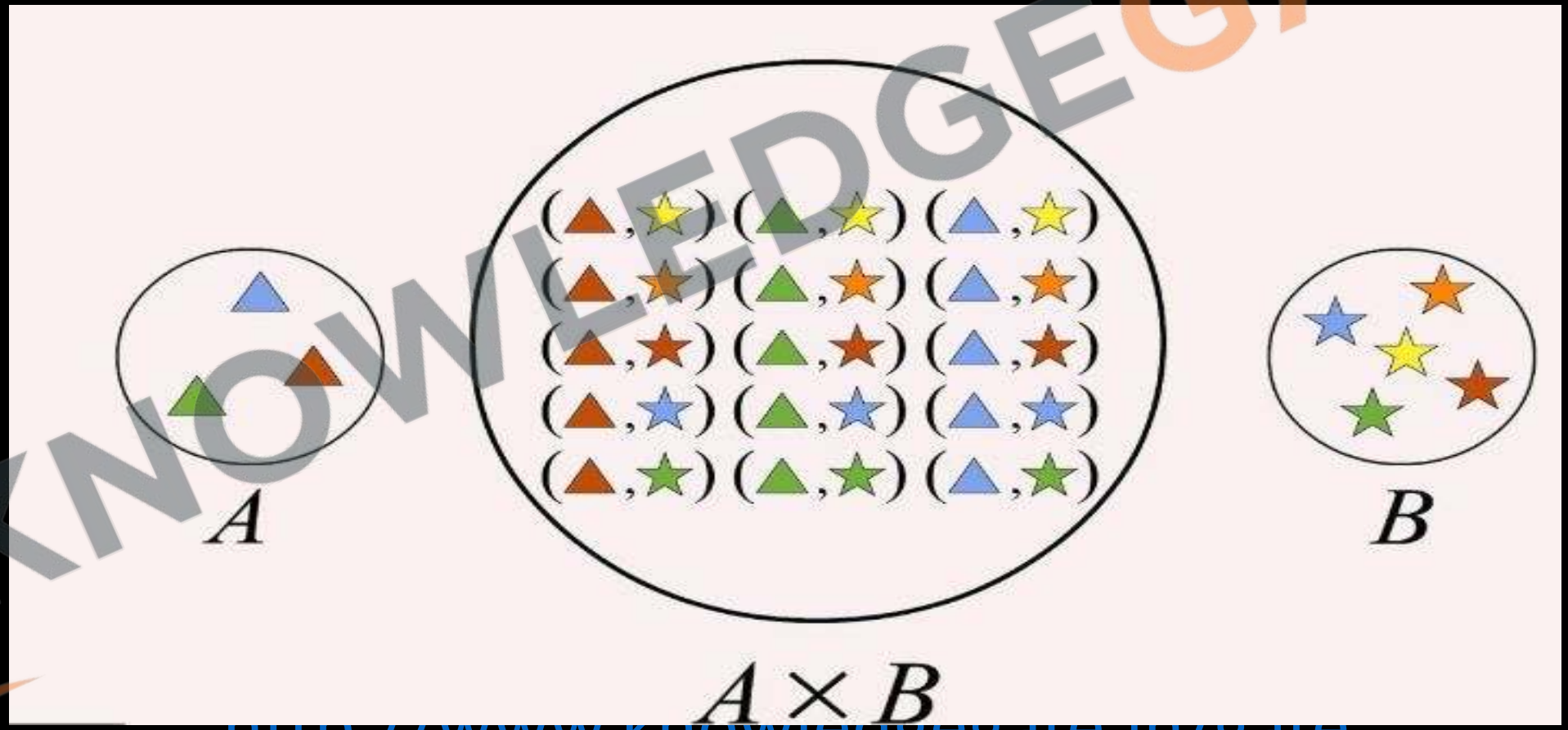
http://www.knowledgegate.in/gate

Cartesian Product

1. Cartesian Product of two sets A and B in the set of all ordered pairs, whose first member belongs to the first set and second member belongs to the second set, denoted by A × B.
2. For E.g. if A = {a, b}, B = {1, 2, 3}
3. A×B = { }

KNOWLEDGEGATE

- It is a kind of maximum relation possible, where every member of the first set belong to every member of the second set.
- $A \times B = \{(a, b) \mid a \in A \text{ and } b \in B\}$



KNOWLEDGEGATE
http://www.knowledgegate.in/gate

1. In general, commutative law does not hold good A× B != B×A

2. If |A| = m and |B| = n then |A×B| =

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Relation

- **Relation**: - Let A and B are sets then every possible subset of 'A×B' is called a relation from A to B.
- If |A| =m and |B| = n then total no of element(pair) will be m*n, every element will have two choice weather to present or not present in the subset(relation), therefore the total number of relation possible is _____

KNOWLEDGEGATE

For E.g. if A = {a, b}, B = {1, 2}, A×B = {(a, 1), (a, 2), (b, 1), (b, 2)}

(a , 1)	(a , 2)	(b , 1)	(b , 2)	
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

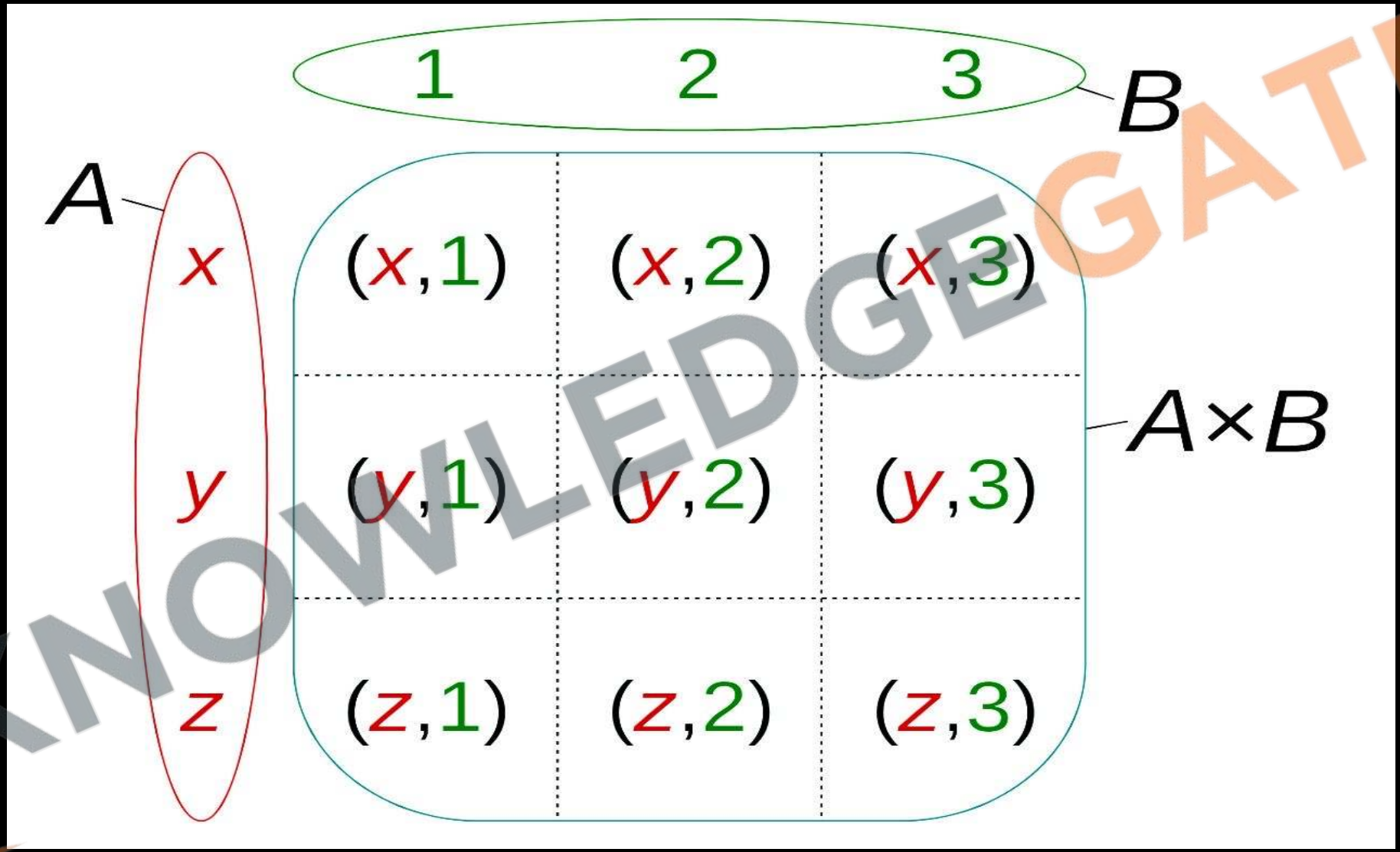
KNOWLEDGEGATE
http://www.knowledgegate.in/gate

For E.g. if A = {a, b}, B = {1, 2}

(a , 1)	(a , 2)	(b , 1)	(b , 2)	
0	0	0	0	{ }
0	0	0	1	{(b, 2)}
0	0	1	0	{(b, 1)}
0	0	1	1	{(b, 1), (b, 2)}
0	1	0	0	{(a, 2)}
0	1	0	1	{(a, 2), (b, 2)}
0	1	1	0	{(a, 2), (b, 1)}
0	1	1	1	{(a, 2), (b, 1), (b, 2)}
1	0	0	0	{(a, 1)}
1	0	0	1	{(a, 1), (b, 2)}
1	0	1	0	{(a, 1), (b, 1)}
1	0	1	1	{(a, 1), (b, 1), (b, 2)}
1	1	0	0	{(a, 1), (a, 2)}
1	1	0	1	{(a, 1), (a, 2), (b, 2)}
1	1	1	0	{(a, 1), (a, 2), (b, 1)}
1	1	1	1	{(a, 1), (a, 2), (b, 1), (b, 2)}

KNOWLEDGE GATE
http://www.knowledgegate.in/gate

Matrix Representation



KNOWLEDGEGATE

- **Complement of a relation**: - Let R be a relation from A to B, then the complement of relation will be denoted by R', R^C or ~~R~~.

- $R' = \{(a, b) | (a, b) \in A \times B, (a, b) \notin R\}$

- $R' = (A \times B) - R$

KNOWLEDGEGATE

- For E.g. if A×B = {(a, 1), (a, 2), (a, 3), (b, 1), (b, 2), (b, 3)}
- R= {(a, 1), (a, 3), (b, 2)}
- R’ = { }

KNOWLEDGEGATE

$R \cup R' =$

$R \cap R' =$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Inverse of a relation**: - Let R be a relation from A to B, then the inverse of relation will be a relation from B to A, denoted by R^{-1}.

- $R^{-1} = \{(b, a) \mid (a, b) \in R\}$

- $A \times B = \{(a, 1), (a, 2), (a, 3), (b, 1), (b, 2), (b, 3)\}$

- $R = \{(a, 1), (a, 3), (b, 2)\}$

- $R^{-1} = \{$ $\}$

- $|R| \square | R^{-1} |$

KNOWLEDGEGATE

- **Diagonal relation**: - A relation R on a set A is said to be diagonal relation if, R is a set of all ordered pair (x, x), for every $\forall x \in A$, sometimes it is also denoted by $\blacktriangle_A$

- $R = \{(x, x) \mid \forall x \in A\}$

	1	2	3
1	11		
2		22	
3			33

KNOWLEDGEGATE

Types of a Relation

- To further study types of relations, we consider a set A with n elements, then a cartesian product A×A will have n^2 elements(pairs). Therefore, total number of relation possible is 2^{n*n}

KNOWLEDGEGATE

- **Reflexive relation**: - A relation R on a set A is said to be reflexive,
- If $\forall x \in A$
  - $(x, x) \in R$

	1	2	3
1	11		
2		22	
3			33

KNOWLEDGEGATE

Q consider a set A = {1,2,3}, find which of the following relations are reflexive and Irreflexive?

	Relation	Reflexive	Irreflexive
1	A×A		
2	ϕ		
3	{(1,1), (2,2), (3,3)}		
4	{(1,2), (2,3), (1,3)}		
5	{(1,1), (1,2), (2,1), (2,2)}		
6	{(1,1), (2,2), (3,3), (1,3), (2,1)}		
7	{(1,3), (2,1), (2,3), (3,2)}		

KNOWLEDGEGATE

For E.g. if A = {a, b}, A×A = {(a, a), (a, b), (b, a), (b, b)}

(a , a)	(a , b)	(b , a)	(b , b)	
0	0	0	0	{ }
0	0	0	1	{(b, b)}
0	0	1	0	{(b, a)}
0	0	1	1	{(b, a), (b, b)}
0	1	0	0	{(a, b)}
0	1	0	1	{(a, b), (b, b)}
0	1	1	0	{(a, b), (b, a)}
0	1	1	1	{(a, b), (b, a), (b, b)}
1	0	0	0	{(a, a)}
1	0	0	1	{(a, a), (b, b)}
1	0	1	0	{(a, a), (b, a)}
1	0	1	1	{(a, a), (b, a), (b, b)}
1	1	0	0	{(a, a), (a, b)}
1	1	0	1	{(a, a), (a, b), (b, b)}
1	1	1	0	{(a, a), (a, b), (b, a)}
1	1	1	1	{(a, a), (a, b), (b, a), (b, b)}

KNOWLEDGE GATE
http://www.knowledgegate.in/gate

For E.g. if A = {a, b}, A×A = {(a, a), (a, b), (b, a), (b, b)}

(a , a)	(a , b)	(b , a)	(b , b)	
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	{(a, a), (b, b)}
1	0	1	0	
1	0	1	1	{(a, a), (b, a), (b, b)}
1	1	0	0	
1	1	0	1	{(a, a), (a, b), (b, b)}
1	1	1	0	
1	1	1	1	{(a, a), (a, b), (b, a), (b, b)}

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Irreflexive relation**: - A relation R on a set A is said to be Irreflexive,
  1. If $\forall x \in A$
  2. $(x, x) \notin R$

KNOWLEDGEGATE

Q consider a set A = {1,2,3}, find which of the following relations are reflexive and Irreflexive?

	Relation	Reflexive	Irreflexive
1	A×A	Y	
2	ϕ	N	
3	{(1,1), (2,2), (3,3)}	Y	
4	{(1,2), (2,3), (1,3)}	N	
5	{(1,1), (1,2), (2,1), (2,2)}	N	
6	{(1,1), (2,2), (3,3), (1,3), (2,1)}	Y	
7	{(1,3), (2,1), (2,3), (3,2)}	N	

KNOWLEDGEGATE

For E.g. if A = {a, b}, A×A = {(a, a), (a, b), (b, a), (b, b)}

(a , a)	(a , b)	(b , a)	(b , b)	
0	0	0	0	{ }
0	0	0	1	{(b, b)}
0	0	1	0	{(b, a)}
0	0	1	1	{(b, a), (b, b)}
0	1	0	0	{(a, b)}
0	1	0	1	{(a, b), (b, b)}
0	1	1	0	{(a, b), (b, a)}
0	1	1	1	{(a, b), (b, a), (b, b)}
1	0	0	0	{(a, a)}
1	0	0	1	{(a, a), (b, b)}
1	0	1	0	{(a, a), (b, a)}
1	0	1	1	{(a, a), (b, a), (b, b)}
1	1	0	0	{(a, a), (a, b)}
1	1	0	1	{(a, a), (a, b), (b, b)}
1	1	1	0	{(a, a), (a, b), (b, a)}
1	1	1	1	{(a, a), (a, b), (b, a), (b, b)}

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

For E.g. if A = {a, b}, A×A = {(a, a), (a, b), (b, a), (b, b)}

(a , a)	(a , b)	(b , a)	(b , b)	
0	0	0	0	{ }
0	0	0	1	
0	0	1	0	{(b, a)}
0	0	1	1	
0	1	0	0	{(a, b)}
0	1	0	1	
0	1	1	0	{(a, b), (b, a)}
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Symmetric relation**: - A relation R on a set A is said to be Symmetric,

If $\forall a, b \in A$

$(a, b) \in R$

.....

then $(b, a) \in R$

.....

KNOWLEDGEGATE

Q consider a set A = {1,2,3}, find which of the following relations are Symmetric, Anti-Symmetric and Asymmetric?

	Relation	Symmetric	Anti-Symmetric	Asymmetric
1	A×A			
2	ϕ			
3	{(1,1), (2,2), (3,3)}			
4	{(1,2), (2,3), (1,3)}			
5	{(1,1), (1,2), (2,1), (2,2)}			
6	{(1,1), (2,2), (3,3), (1,3), (2,1)}			
7	{(1,3), (2,1), (2,3), (3,2)}			

KNOWLEDGEGATE

For E.g. if A = {a, b}, A×A = {(a, a), (a, b), (b, a), (b, b)}

(a , a)	(a , b)	(b , a)	(b , b)	
0	0	0	0	{ }
0	0	0	1	{(b, b)}
0	0	1	0	{(b, a)}
0	0	1	1	{(b, a), (b, b)}
0	1	0	0	{(a, b)}
0	1	0	1	{(a, b), (b, b)}
0	1	1	0	{(a, b), (b, a)}
0	1	1	1	{(a, b), (b, a), (b, b)}
1	0	0	0	{(a, a)}
1	0	0	1	{(a, a), (b, b)}
1	0	1	0	{(a, a), (b, a)}
1	0	1	1	{(a, a), (b, a), (b, b)}
1	1	0	0	{(a, a), (a, b)}
1	1	0	1	{(a, a), (a, b), (b, b)}
1	1	1	0	{(a, a), (a, b), (b, a)}
1	1	1	1	{(a, a), (a, b), (b, a), (b, b)}

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

For E.g. if A = {a, b}, A×A = {(a, a), (a, b), (b, a), (b, b)}

(a , a)	(a , b)	(b , a)	(b , b)	
0	0	0	0	{ }
0	0	0	1	{(b, b)}
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	{(a, b), (b, a)}
0	1	1	1	{(a, b), (b, a), (b, b)}
1	0	0	0	{(a, a)}
1	0	0	1	{(a, a), (b, b)}
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	{(a, a), (a, b), (b, a)}
1	1	1	1	{(a, a), (a, b), (b, a), (b, b)}

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Anti-Symmetric relation**: - A relation R on a set A with cartesian product A×A is said to be Anti-Symmetric,

If $\forall a, b \in A$

$(a, b) \in R$

$(b, a) \in R$

……

$a = b$

……

Conclusion: Symmetry is not allowed but diagonal pairs are allowed

KNOWLEDGEGATE

	Relation	Symmetric	Anti-Symmetric	Asymmetric
1	A×A	Y		
2	ϕ	Y		
3	{(1,1), (2,2), (3,3)}	Y		
4	{(1,2), (2,3), (1,3)}	N		
5	{(1,1), (1,2), (2,1), (2,2)}	Y		
6	{(1,1), (2,2), (3,3), (1,3), (2,1)}	N		
7	{(1,3), (2,1), (2,3), (3,2)}	N		

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

For E.g. if A = {a, b}, A×A = {(a, a), (a, b), (b, a), (b, b)}

(a , a)	(a , b)	(b , a)	(b , b)	
0	0	0	0	{ }
0	0	0	1	{(b, b)}
0	0	1	0	{(b, a)}
0	0	1	1	{(b, a), (b, b)}
0	1	0	0	{(a, b)}
0	1	0	1	{(a, b), (b, b)}
0	1	1	0	{(a, b), (b, a)}
0	1	1	1	{(a, b), (b, a), (b, b)}
1	0	0	0	{(a, a)}
1	0	0	1	{(a, a), (b, b)}
1	0	1	0	{(a, a), (b, a)}
1	0	1	1	{(a, a), (b, a), (b, b)}
1	1	0	0	{(a, a), (a, b)}
1	1	0	1	{(a, a), (a, b), (b, b)}
1	1	1	0	{(a, a), (a, b), (b, a)}
1	1	1	1	{(a, a), (a, b), (b, a), (b, b)}

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

For E.g. if A = {a, b}, A×A = {(a, a), (a, b), (b, a), (b, b)}

(a , a)	(a , b)	(b , a)	(b , b)	
0	0	0	0	{ }
0	0	0	1	{(b, b)}
0	0	1	0	{(b, a)}
0	0	1	1	{(b, a), (b, b)}
0	1	0	0	{(a, b)}
0	1	0	1	{(a, b), (b, b)}
0	1	1	0	
0	1	1	1	
1	0	0	0	{(a, a)}
1	0	0	1	{(a, a), (b, b)}
1	0	1	0	{(a, a), (b, a)}
1	0	1	1	{(a, a), (b, a), (b, b)}
1	1	0	0	{(a, a), (a, b)}
1	1	0	1	{(a, a), (a, b), (b, b)}
1	1	1	0	
1	1	1	1	

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Asymmetric relation**: - A relation R on a set A is said to be Asymmetric,

If $\forall a, b \in A$

$(a, b) \in R$

……

$(b, a) \notin R$

……

Conclusion: Symmetry is not allowed; even diagonal pairs are not allowed

KNOWLEDGEGATE

	Relation	Symmetric	Anti-Symmetric	Asymmetric
1	A×A	Y	N	
2	ϕ	Y	Y	
3	{(1,1), (2,2), (3,3)}	Y	Y	
4	{(1,2), (2,3), (1,3)}	N	Y	
5	{(1,1), (1,2), (2,1), (2,2)}	Y	N	
6	{(1,1), (2,2), (3,3), (1,3), (2,1)}	N	Y	
7	{(1,3), (2,1), (2,3), (3,2)}	N	N	

KNOWLEDGEGATE

For E.g. if A = {a, b}, A×A = {(a, a), (a, b), (b, a), (b, b)}

(a , a)	(a , b)	(b , a)	(b , b)	
0	0	0	0	{ }
0	0	0	1	{(b, b)}
0	0	1	0	{(b, a)}
0	0	1	1	{(b, a), (b, b)}
0	1	0	0	{(a, b)}
0	1	0	1	{(a, b), (b, b)}
0	1	1	0	{(a, b), (b, a)}
0	1	1	1	{(a, b), (b, a), (b, b)}
1	0	0	0	{(a, a)}
1	0	0	1	{(a, a), (b, b)}
1	0	1	0	{(a, a), (b, a)}
1	0	1	1	{(a, a), (b, a), (b, b)}
1	1	0	0	{(a, a), (a, b)}
1	1	0	1	{(a, a), (a, b), (b, b)}
1	1	1	0	{(a, a), (a, b), (b, a)}
1	1	1	1	{(a, a), (a, b), (b, a), (b, b)}

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

For E.g. if A = {a, b}, A×A = {(a, a), (a, b), (b, a), (b, b)}

(a , a)	(a , b)	(b , a)	(b , b)	
0	0	0	0	{ }
0	0	0	1	
0	0	1	0	{(b, a)}
0	0	1	1	
0	1	0	0	{(a, b)}
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

KNOWLEDGE GATE
http://www.knowledgegate.in/gate

- **Transitive relation**: - A relation R on a set A is said to be Transitive,

If $\forall$ a, b, c $\in$ A

(a, b) $\in$ R

(b, c) $\in$ R

.....

(a, c) $\in$ R

.....

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

	Relation	Transitive
1	A×A	
2	ϕ	
3	{(1,1), (2,2), (3,3)}	
4	{(1,2), (2,3), (1,3)}	
5	{(1,1), (1,2), (2,1), (2,2)}	
6	{(1,1), (2,2), (3,3), (1,3), (2,1)}	
7	{(1,3), (2,1), (2,3), (3,2)}	
8	{(1,2)}	
9	{(1,3), (2,3)}	
10	{(1,2), (1,3)}	
11	{(2,3), (1,2)}	

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

For E.g. if A = {a, b}, A×A = {(a, a), (a, b), (b, a), (b, b)}

(a , a)	(a , b)	(b , a)	(b , b)	
0	0	0	0	{ }
0	0	0	1	{(b, b)}
0	0	1	0	{(b, a)}
0	0	1	1	{(b, a), (b, b)}
0	1	0	0	{(a, b)}
0	1	0	1	{(a, b), (b, b)}
0	1	1	0	{(a, b), (b, a)}
0	1	1	1	{(a, b), (b, a), (b, b)}
1	0	0	0	{(a, a)}
1	0	0	1	{(a, a), (b, b)}
1	0	1	0	{(a, a), (b, a)}
1	0	1	1	{(a, a), (b, a), (b, b)}
1	1	0	0	{(a, a), (a, b)}
1	1	0	1	{(a, a), (a, b), (b, b)}
1	1	1	0	{(a, a), (a, b), (b, a)}
1	1	1	1	{(a, a), (a, b), (b, a), (b, b)}

KNOWLEDGE GATE
http://www.knowledgegate.in/gate

For E.g. if A = {a, b}, A×A = {(a, a), (a, b), (b, a), (b, b)}

(a , a)	(a , b)	(b , a)	(b , b)	
0	0	0	0	{ }
0	0	0	1	{(b, b)}
0	0	1	0	{(b, a)}
0	0	1	1	{(b, a), (b, b)}
0	1	0	0	{(a, b)}
0	1	0	1	{(a, b), (b, b)}
0	1	1	0	
0	1	1	1	
1	0	0	0	{(a, a)}
1	0	0	1	{(a, a), (b, b)}
1	0	1	0	{(a, a), (b, a)}
1	0	1	1	{(a, a), (b, a), (b, b)}
1	1	0	0	{(a, a), (a, b)}
1	1	0	1	{(a, a), (a, b), (b, b)}
1	1	1	0	
1	1	1	1	{(a, a), (a, b), (b, a), (b, b)}

KNOWLEDGE GATE
http://www.knowledgegate.in/gate

- **Equivalence Relation**: - A relation R on a set A with cartesian product A×A is said to be Equivalence, if it is

1. **Reflexive**
2. **Symmetric**
3. **Transitive**

KNOWLEDGEGATE

- If two relations R_1 and R_2 are Equivalence then their union need not to be equivalence but intersection will also be Equivalence.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

R_1 : (a, b) iff (a + b) is even over the set of integers

KNOWLEDGEGATE

R_2 : (a, b) iff (a + b) is odd over the set of integers

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

R_3 : (a, b) iff a x b > 0 over the set of non-zero rational numbers

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

R_4 : (a, b) iff $|a - b| \leq 2$ over the set of natural numbers

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Partial Order Relation**: - A relation R on a set A with cartesian product A×A is said to be partial order, if it is

1. **Reflexive**
2. **Anti - Symmetric**
3. **Transitive**

KNOWLEDGEGATE

- **Partial ordering set (Poset):** - a set A with partial ordering relation R defined on A is called a POSET and is denoted by [A, R]

- For e.g. [A, /], [A, <=], [P(S), ⊑]

KNOWLEDGEGATE

Q let R_1 be a relation from A = {1,3,5,7} to B = {2,4,6,8} and R_2 be another relation from B to C = {1,2,3,4} as defined below

(i) an element x in A is related to an element y in B if x + y is divisible by 3

(ii) an element x in B is related to an element y in C if x + y is even but not divisible by 3. Which is the composite relation R_1R_2 from A to C?

a) {(1,2), (1,4), (3,3), (5,4), (7,3)}

b) {(1,2), (1,3), (3,2), (5,2), (7,3)}

c) {(1,2), (3,2), (3,4), (5,4), (7,2)}

d) {(3,2), (3,4), (5,1), (5,3), (7,1)}

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Chapter-3 (POSET & Lattices)

KNOWLEDGEGATE

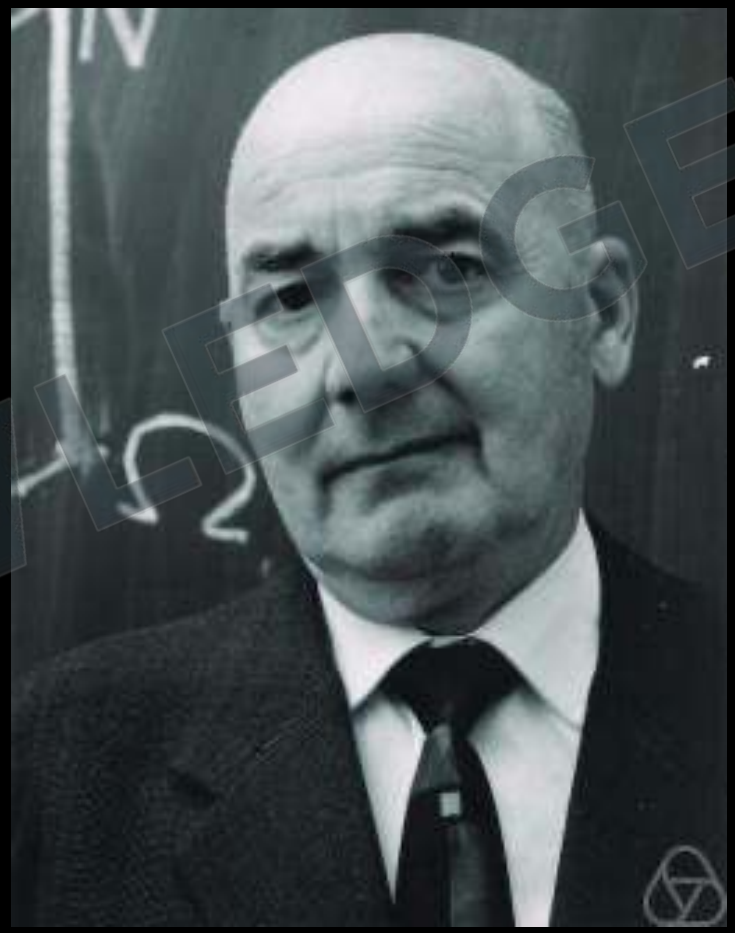
http://www.knowledgegate.in/gate

Conversion of POSET into a Hasse Diagram

- If we want to study Partial order relation further then it will be better to convert it into more convenient notation so that it can be studied easily.
- This graphical representation is called Hasse Diagram

KNOWLEDGEGATE

- In order theory, a Hasse diagram is a type of mathematical diagram used to represent a finite partially ordered set, in the form of a drawing of its transitive reduction. The diagrams are named after Helmut Hasse (1898–1979)



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Steps to convert partial order relation into hasse diagram

1- Draw a vertex for each element in the Set

2- If (a, b) ϵ R then draw an edge from a to b

3- Remove all Reflexive and Transitive edges

4- Remove the direction of edges and arrange them in the increasing order of heights.

Q Consider a Partial order relation and convert it into hasse diagram?

R = {(1,1), (1,2), (1,4), (1,8), (2,2), (2,4), (2,8), (4,4), (4,8), (8,8)}

KNOWLEDGEGATE

Q Consider a Partial order relation and convert it into hasse diagram?

R = {(1,1), (1,2), (1,3), (1,6), (2,2), (2,6), (3,3), (3,6), (6,6)}

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Let A = (1, 2, 3, 4, 6, 8, 9, 12, 18, 24) be ordered set with relation "x divides y". Draw its Hasse diagram?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Draw the Hasse diagram of (A, ≤), where A = (3, 4, 12, 24, 48, 72) and relation ≤ be such that a ≤ b if a divides b.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

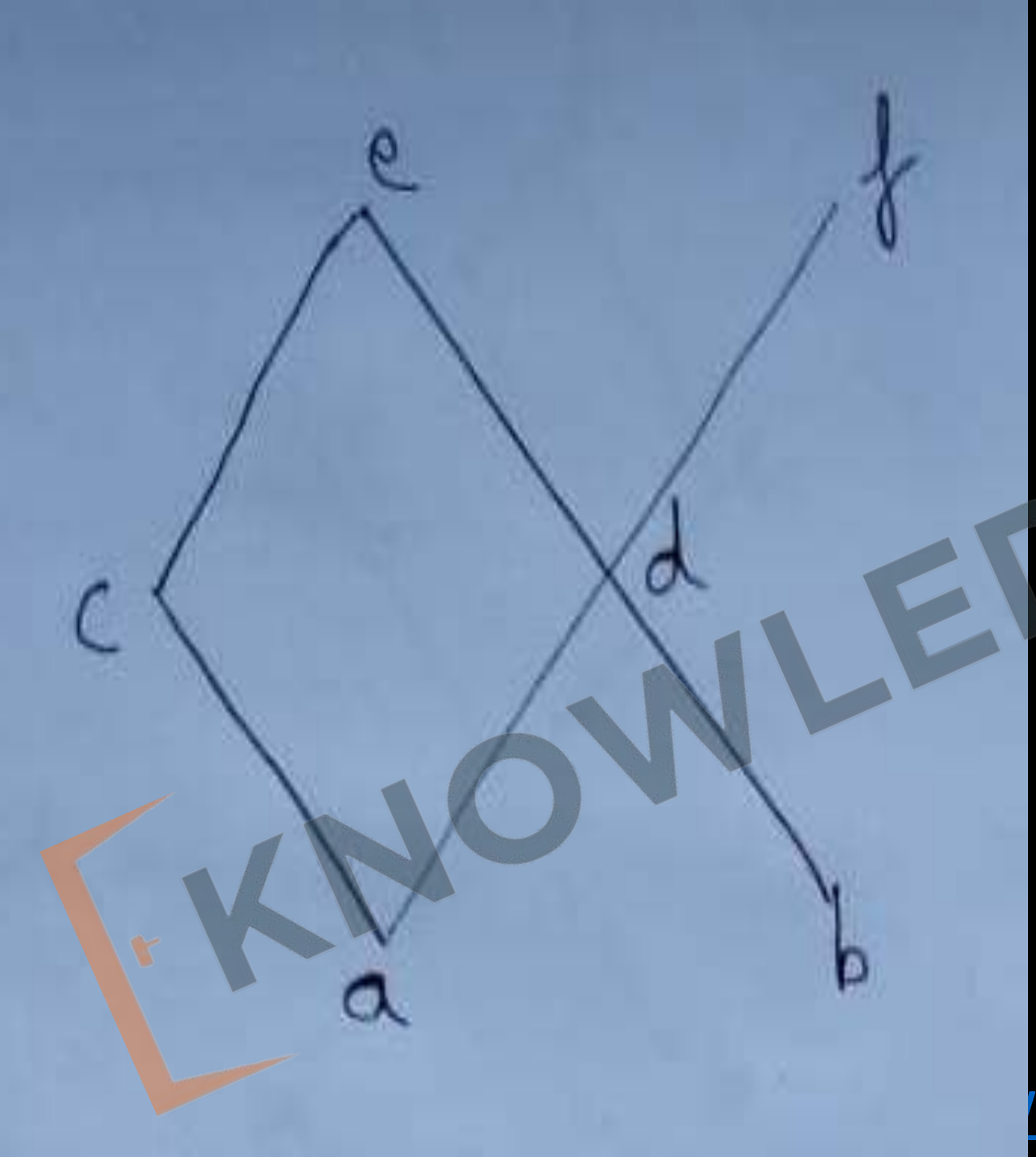
Q Let A = (2, 3, 6, 12, 24, 36) and relation '≤' be such that x≤y if x divides y. Draw the Hasse diagram of (A, ≤)?

KNOWLEDGEGATE

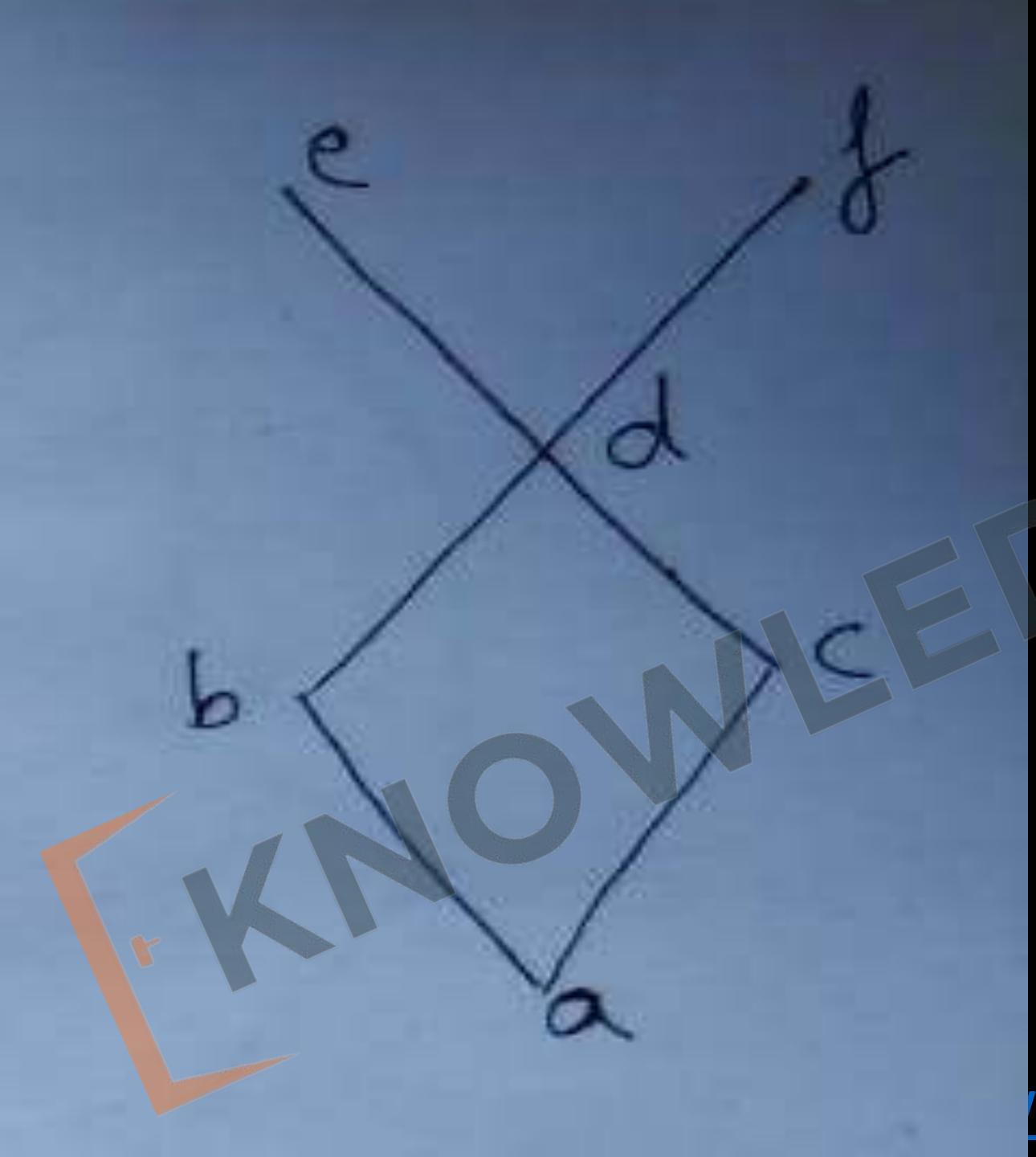
http://www.knowledgegate.in/gate

Lattice :- A hasse diagram/Partial order relation is called Lattice if their exist a Join and Meet for every pair of element. Or A hasse diagram/Partial order relation is called Lattice if it is both Join Semi Lattice and Meet Semi Lattice.

KNOWLEDGEGATE

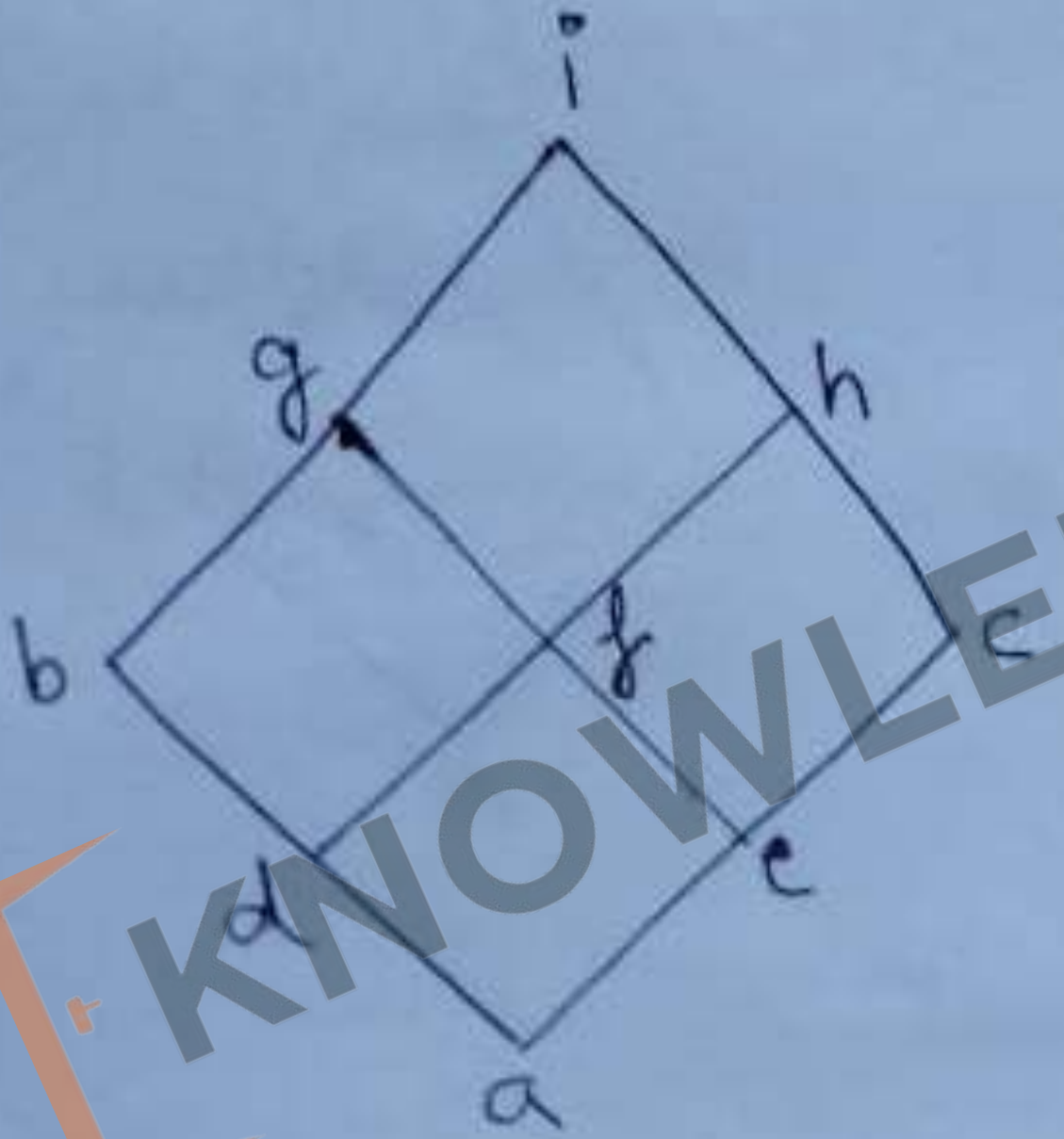


KNOWLEDGEGATE
wledgegate.in/gate

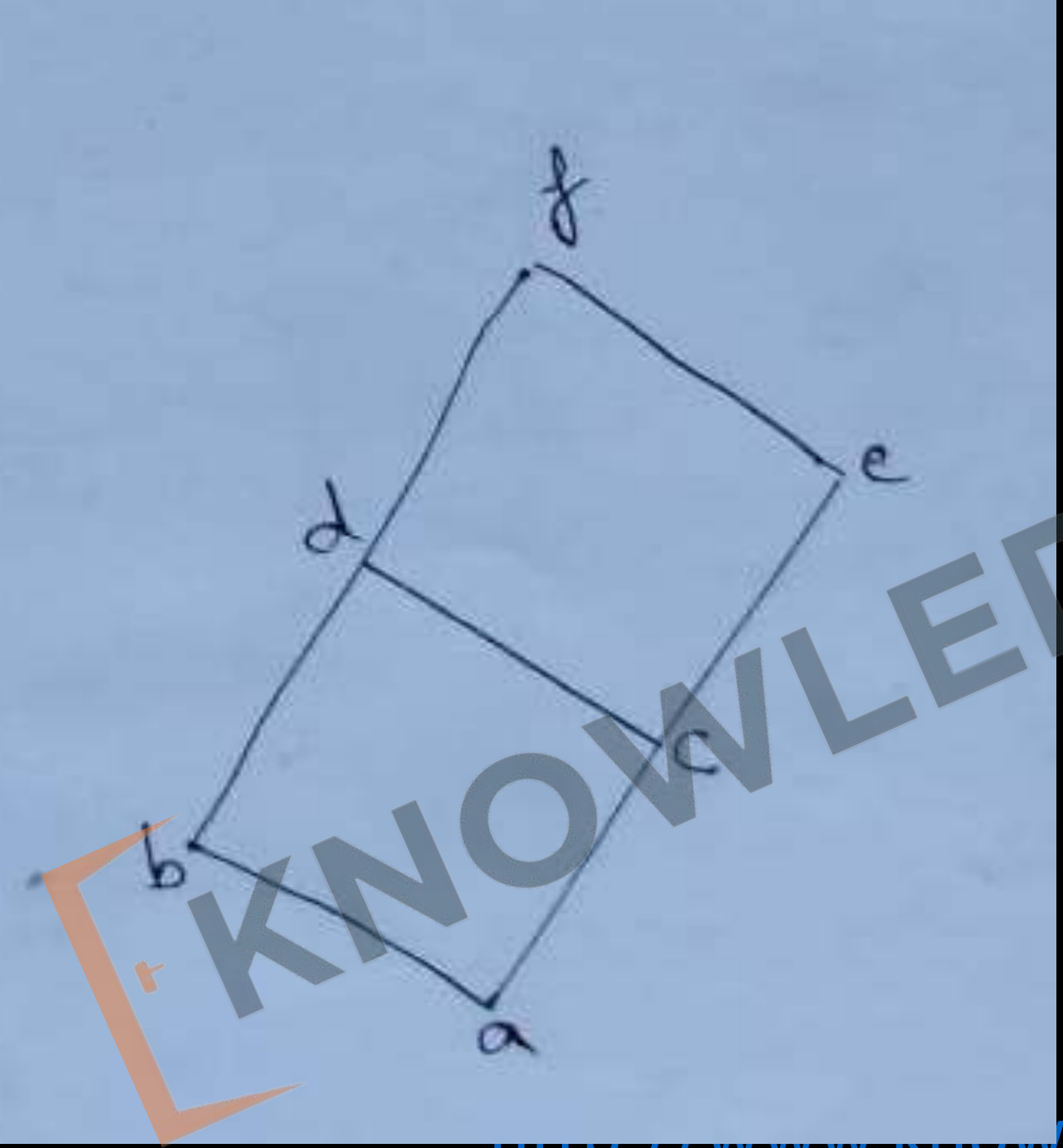


KNOWLEDGEGATE

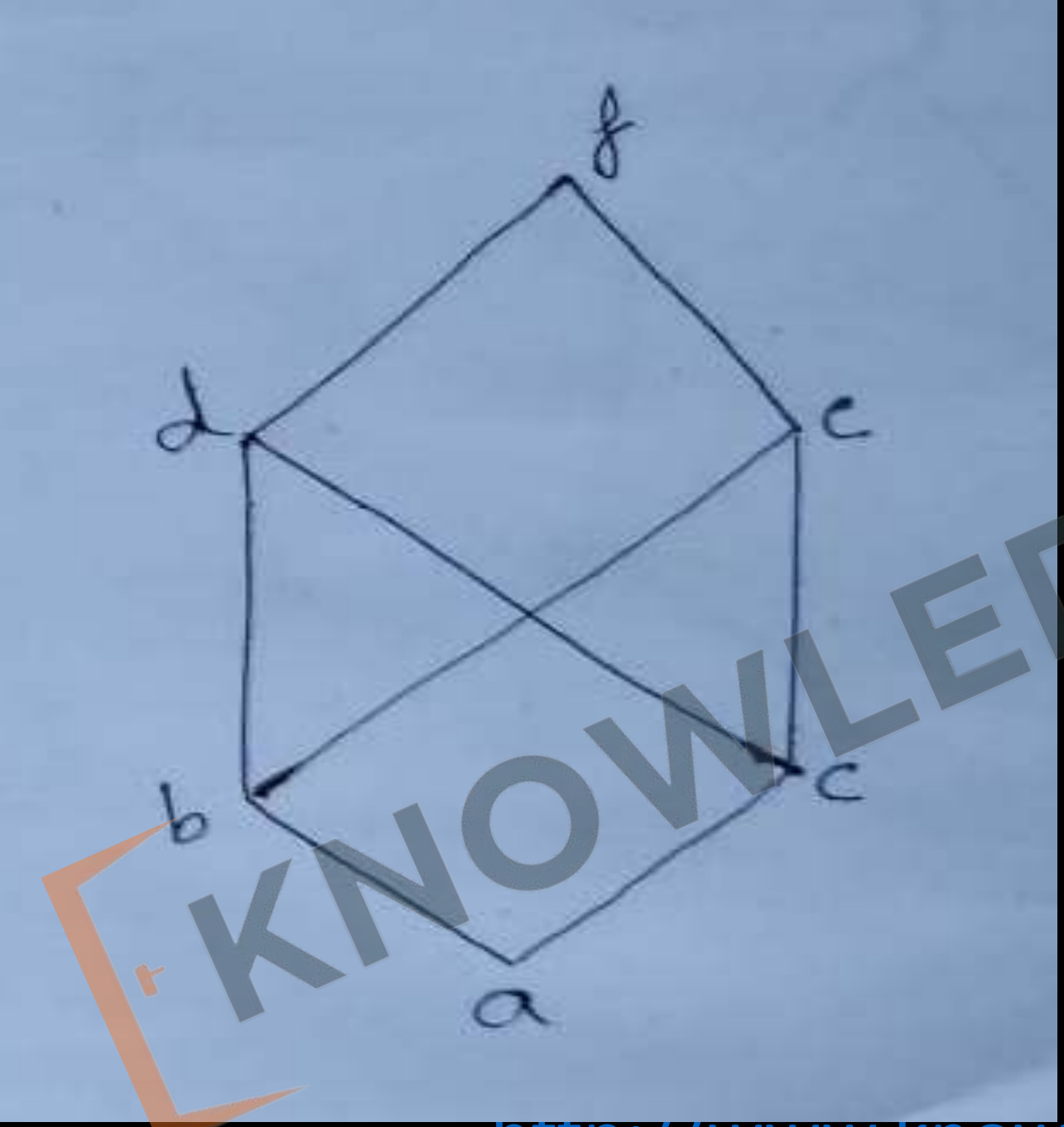
ledgegate.in/gate



KNOWLEDGEGATE
wledgegate.in/gate

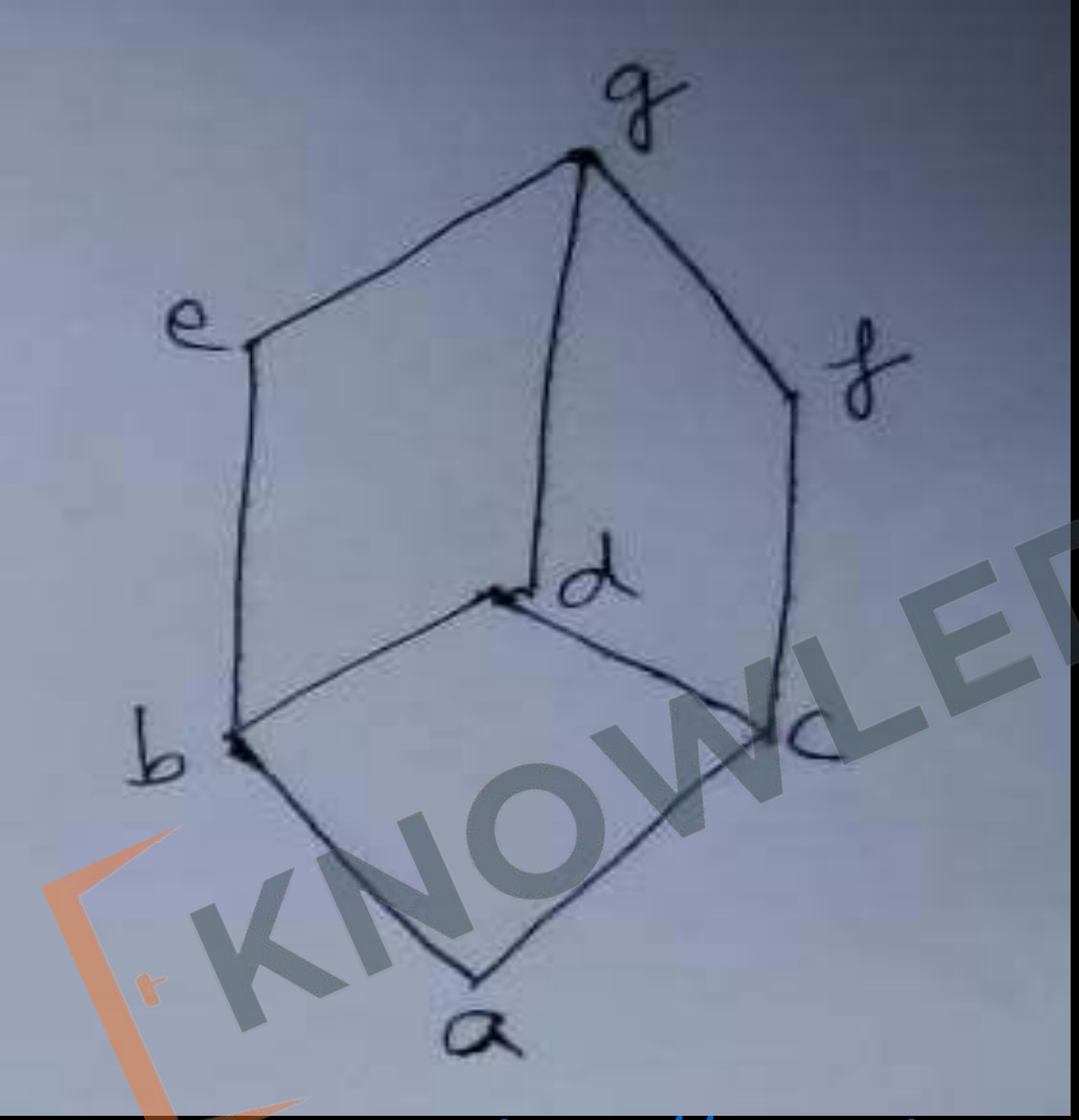


KNOWLEDGEGATE
http://www.knowledgegate.in/gate



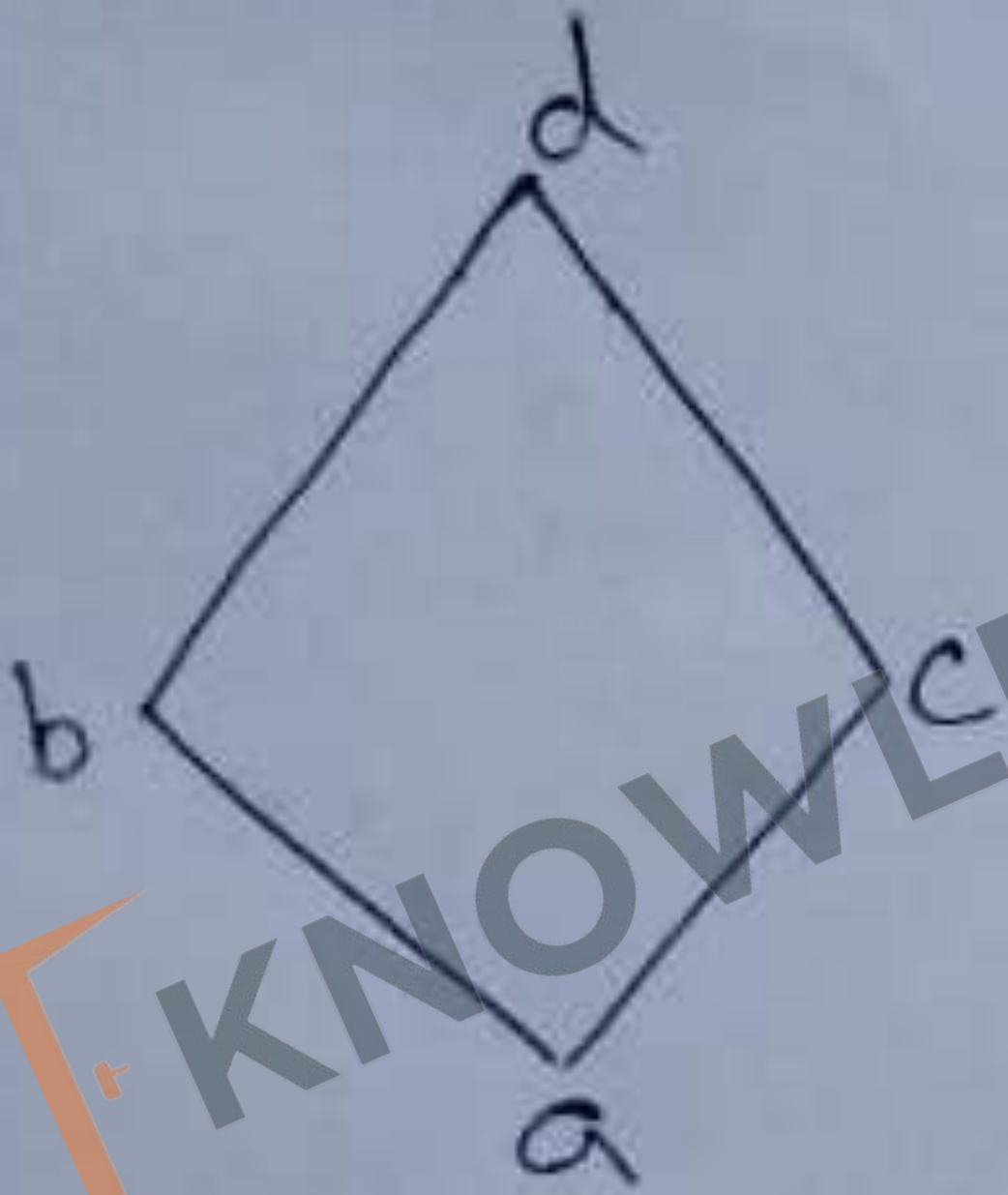
KNOWLEDGEGATE

http://www.knowledgegate.in/gate



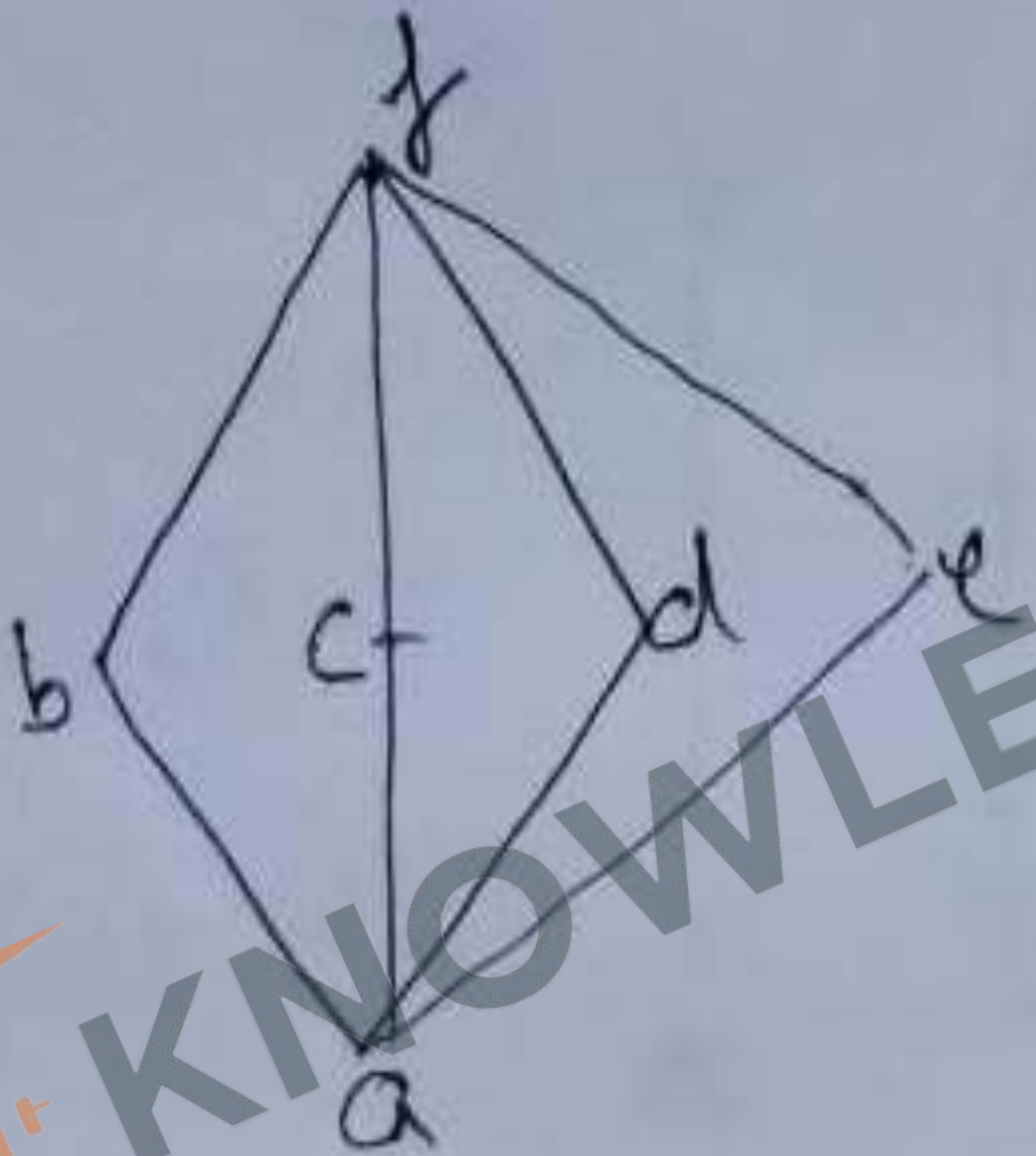
KNOWLEDGEGATE

http://www.knowledgegate.in/gate



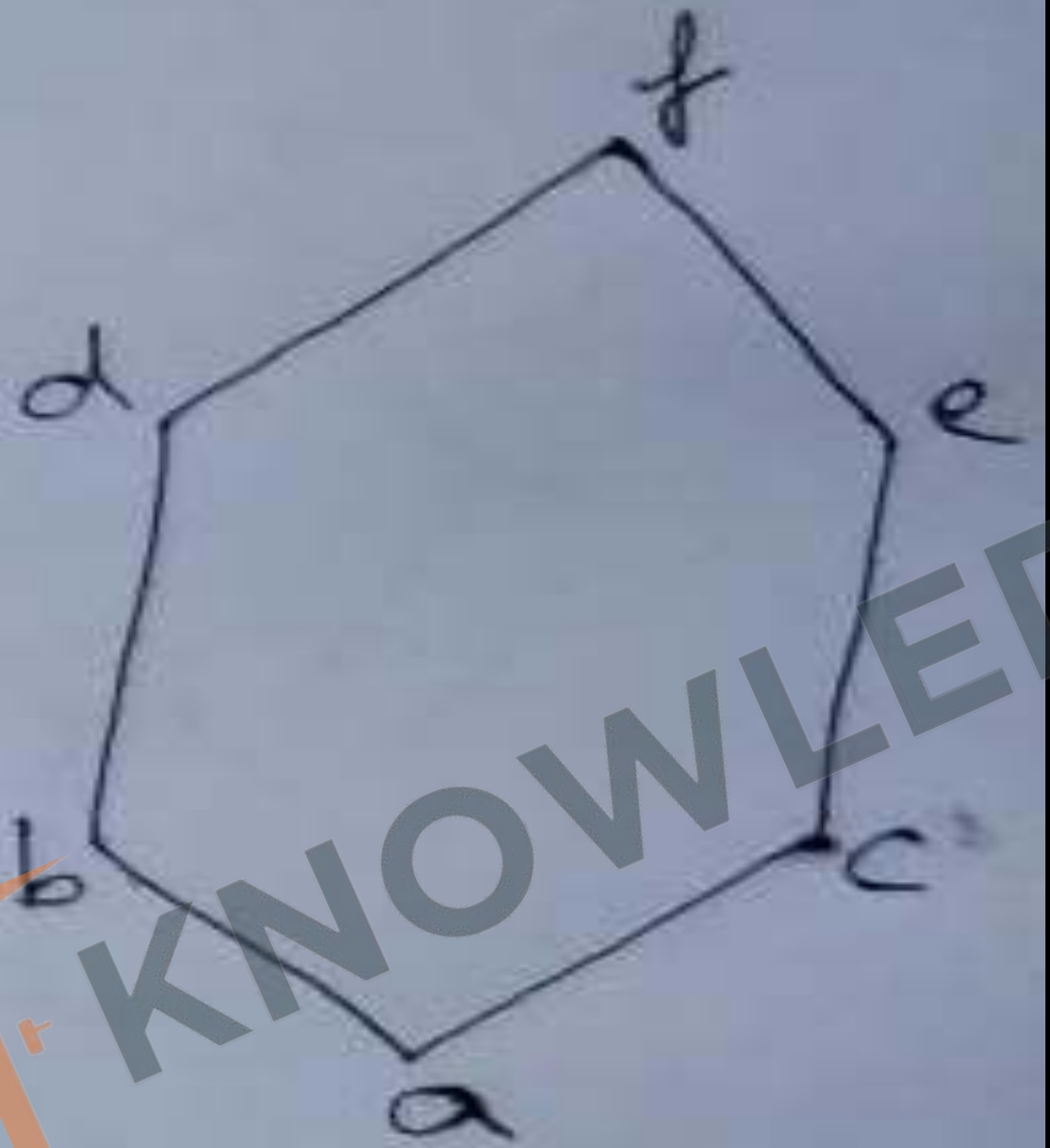
KNOWLEDGEGATE

wledgegate.in/gate



KNOWLEDGEGATE

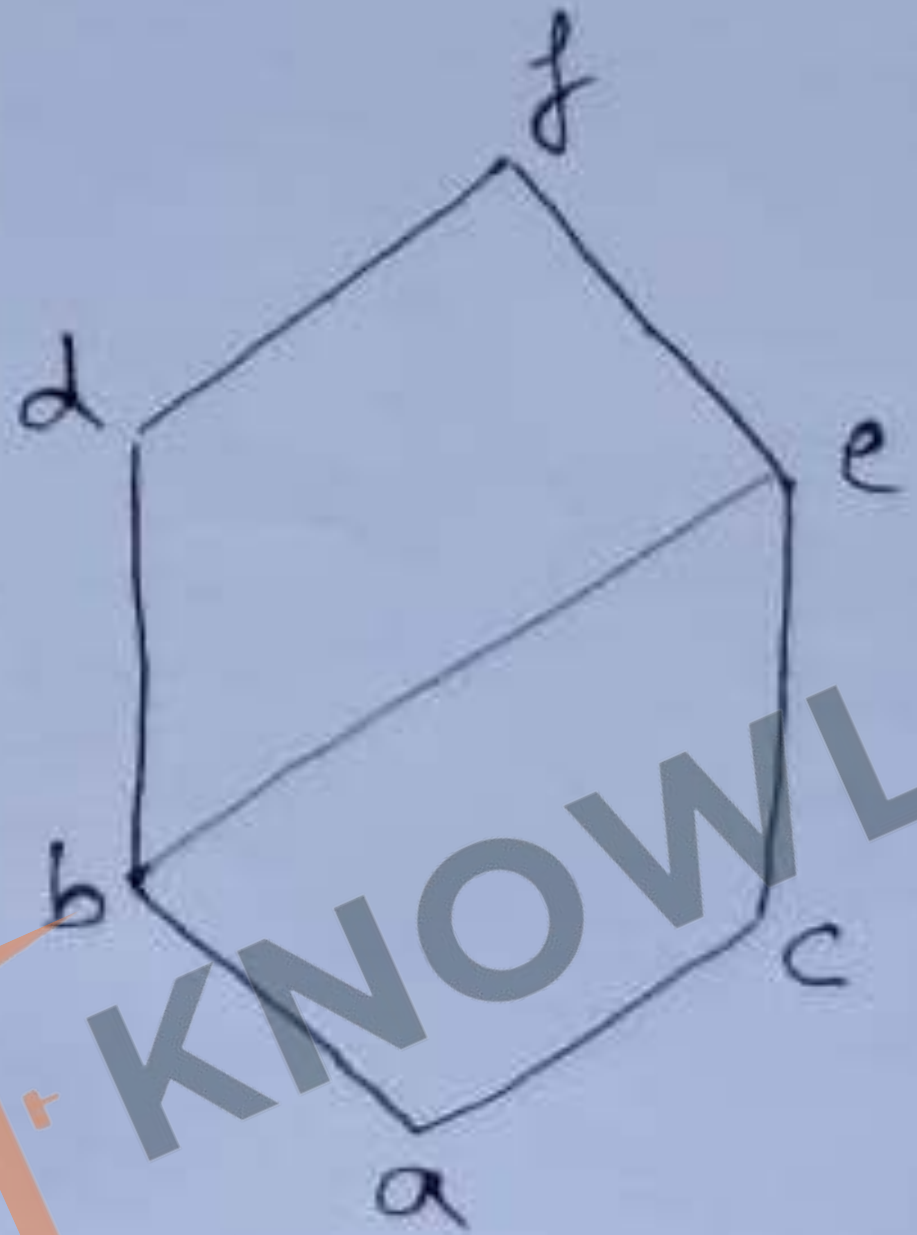
wledgegate.in/gate



KNOWLEDGEGATE

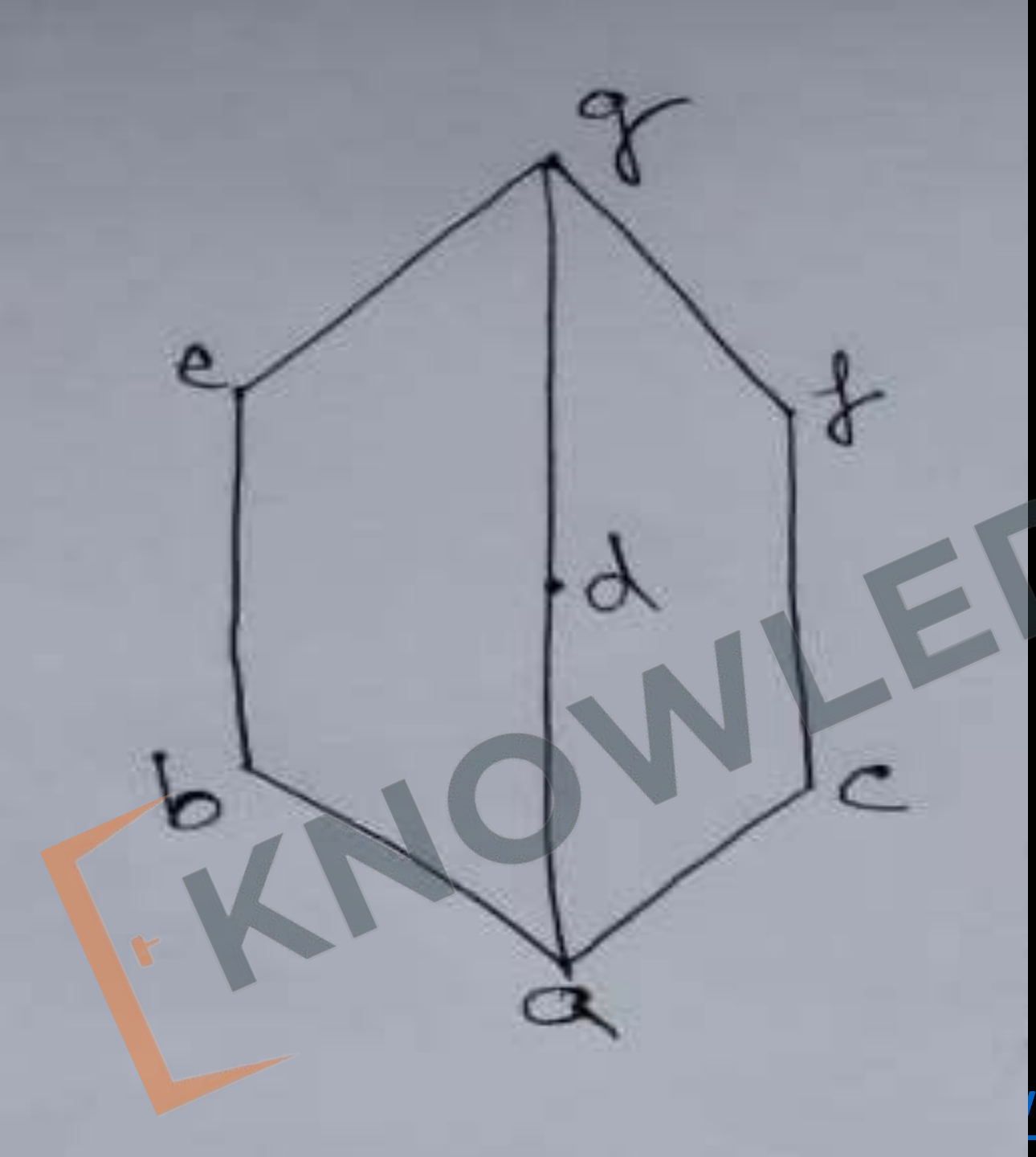
vledgegate.in/gate

KNOWLEDGEGATE
wledgegate.in/gate



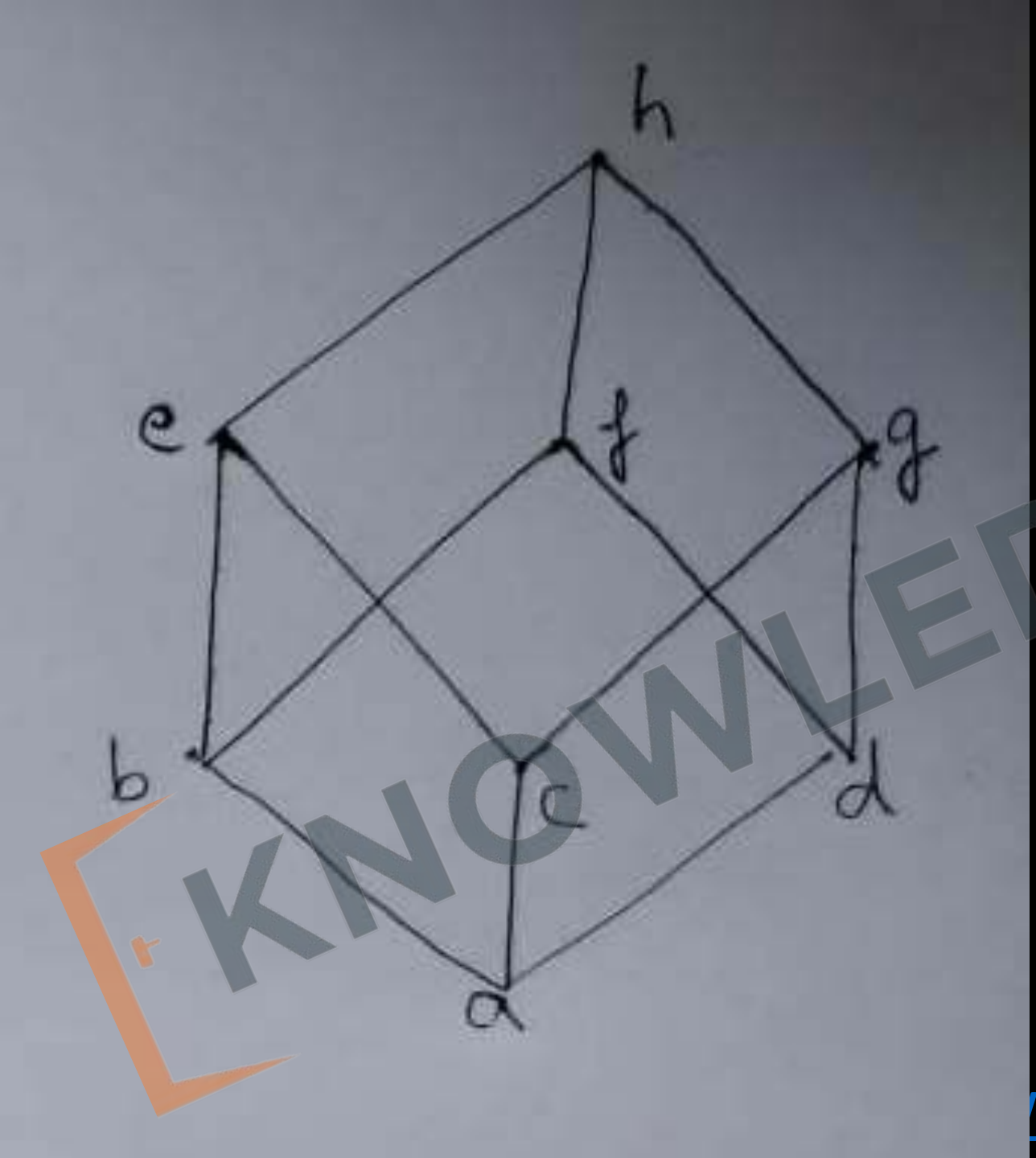
KNOWLEDGEGATE

wledgegate.in/gate



KNOWLEDGEGATE

wledgegate.in/gate



KNOWLEDGEGATE

wledgegate.in/gate

Idempotent law

- $A \vee A = A$
- $A \wedge A = A$

Associative law

- $(A \vee B) \vee C = A \vee (B \vee C)$
- $(A \wedge B) \wedge C = A \wedge (B \wedge C)$

Commutative law

- $A \vee B = B \vee A$
- $A \wedge B = B \wedge A$

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Distributive law

- $A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$
- $A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$

De Morgan's law

- $(A \vee B)^C = A^C \wedge B^C$
- $(A \wedge B)^C = A^C \vee B^C$

Identity law

- $A \vee \phi = A$
- $A \wedge \phi = \phi$
- $A \vee U = U$
- $A \wedge U = A$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Complement law

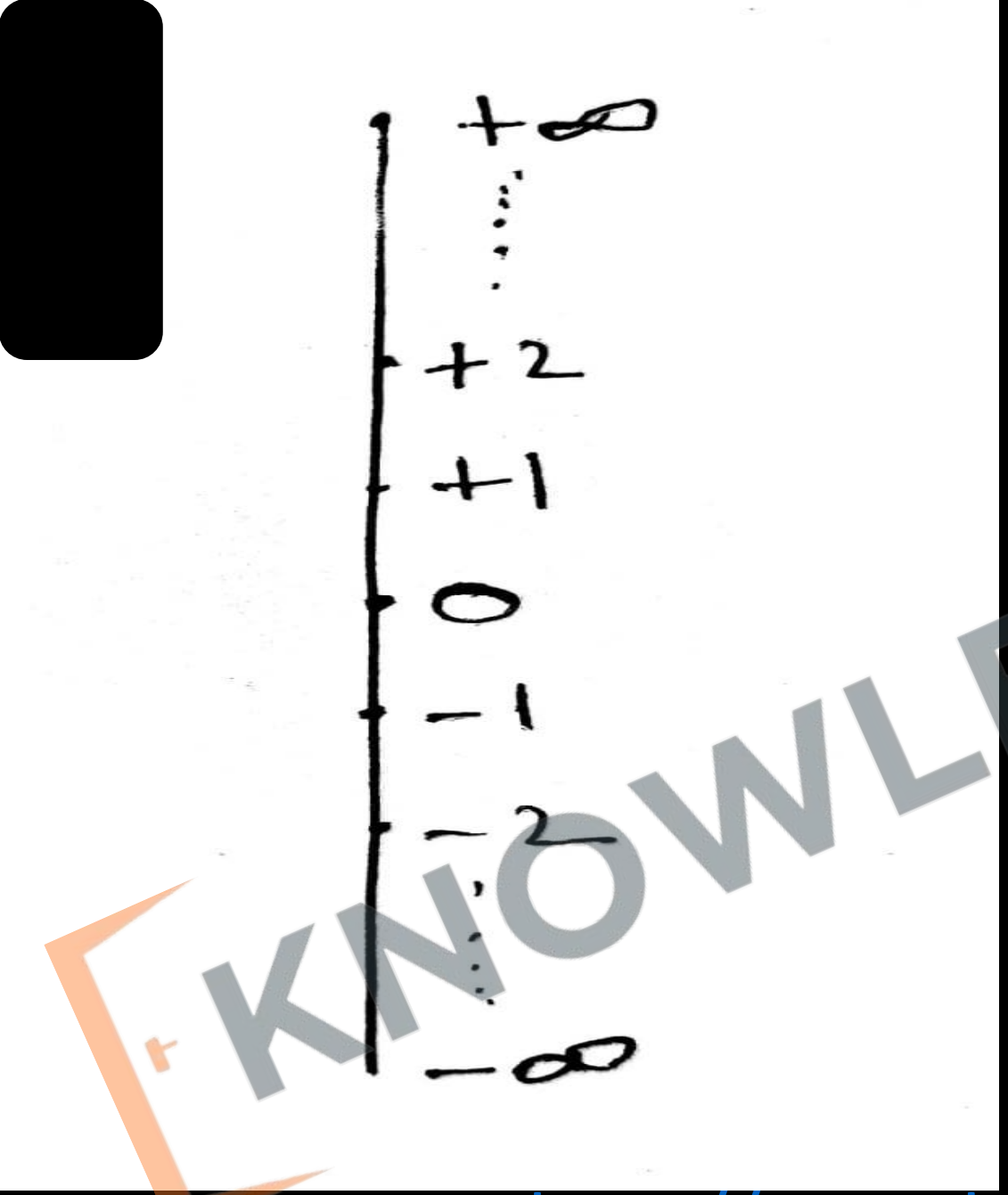
- $A \vee A^C = U$
- $A \wedge A^C = \phi$
- $U^C = \phi$
- $\phi^C = U$

Involution law

- $((A)^C)^C = A$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate



Boolean algebra

- **Unbounded Lattice :-** If a lattice has infinite of elements then it is called Unbounded Lattice.

KNOWLEDGEGATE

- **Bounded Lattice** :- If a lattice has finite number of elements then it is called Bounded lattice, there will be upper and lower bound in lattice.

KNOWLEDGEGATE

- **<u>Complement of an element in a Lattice</u> :-** If two elements a and a^c, are complement of each other, then the following equations must always holds good.
  $a \vee a^c$ = Upper bound of lattice
  $a \wedge a^c$ = Lower bound of lattice

KNOWLEDGEGATE

- **Distributive Lattice** :- A lattice is said to be distributed lattice. if for every element their exist at most one completemt(zero or one).

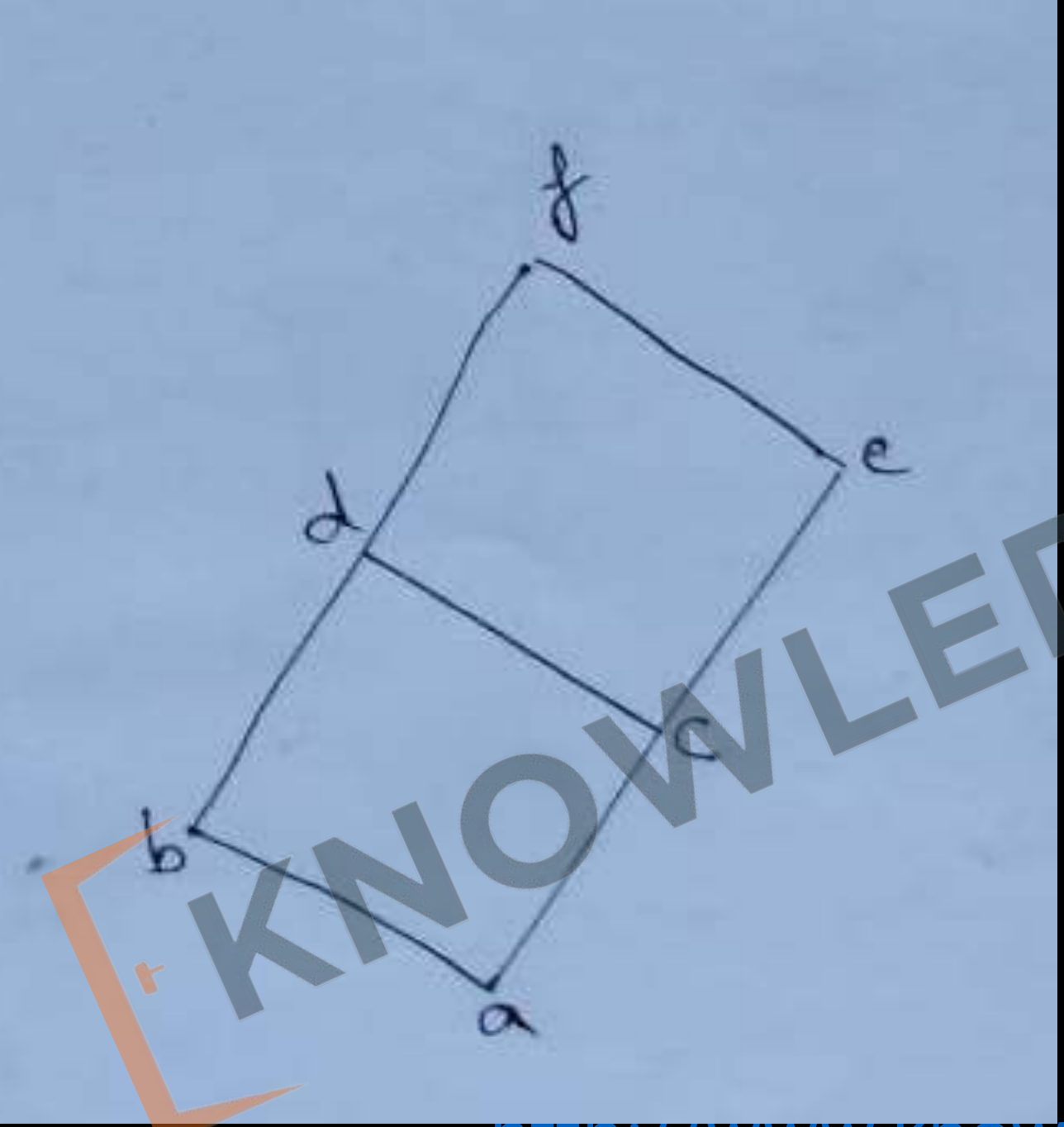
KNOWLEDGEGATE

- **Complement Lattice :-** A Lattice is said to be Complement lattice. if for every element their exist at least one complement(one or more).

KNOWLEDGEGATE

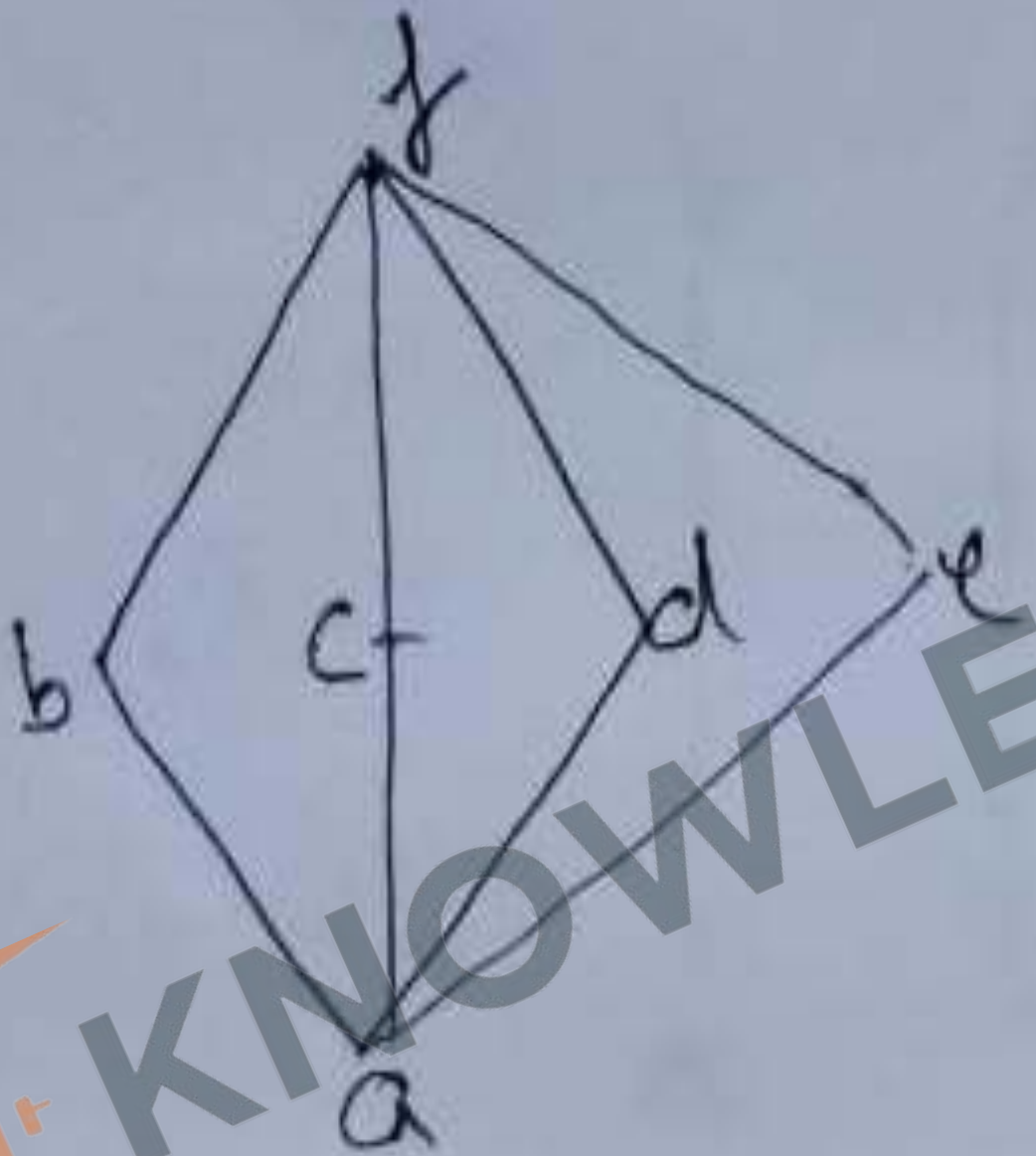
- **Boolean Algebra :-** A Lattice is said to be Boolean Algebra, if for every element their exist exactly one complement. Or if a lattice is both complemented and distributed then it is called Boolean Algebra.

KNOWLEDGEGATE



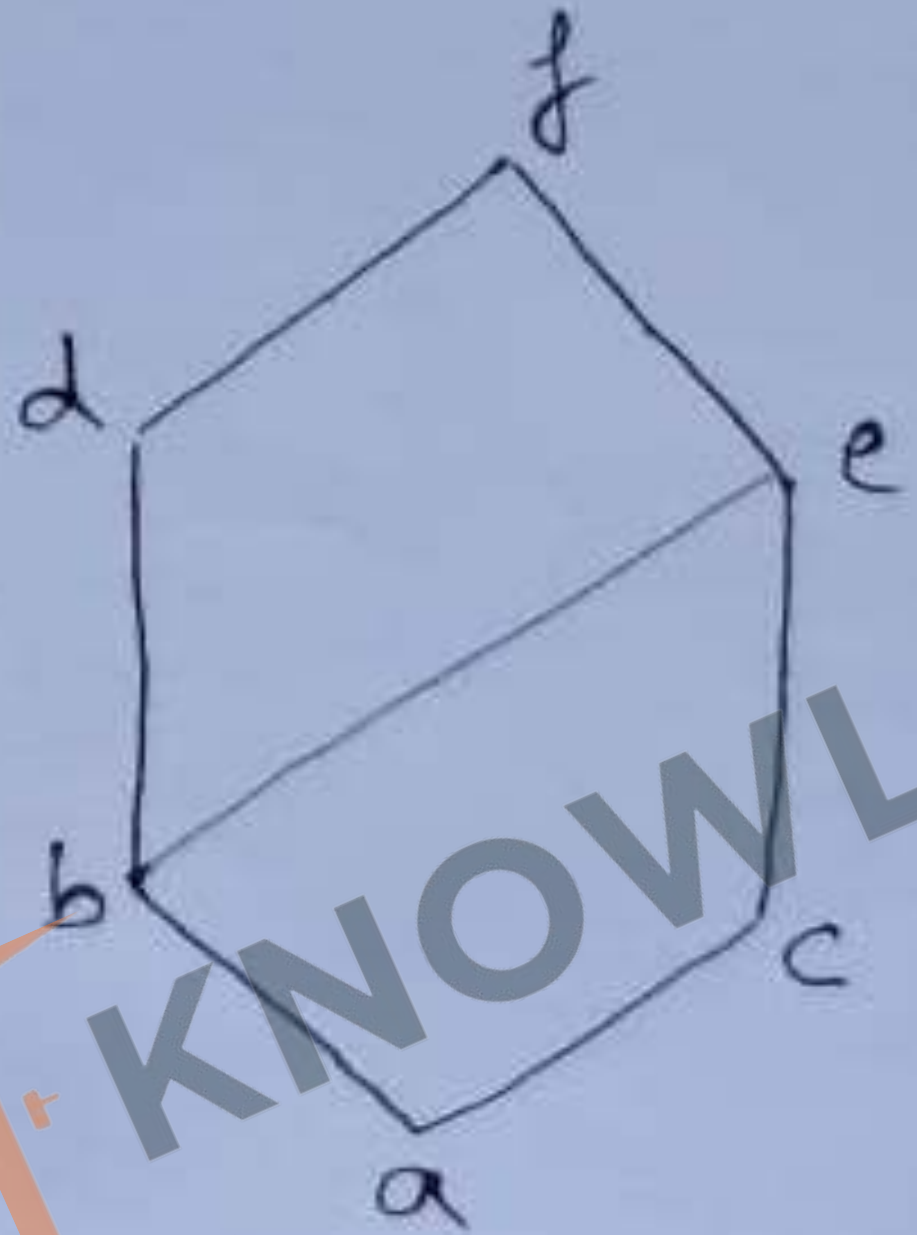
KNOWLEDGEGATE

http://www.knowledgegate.in/gate



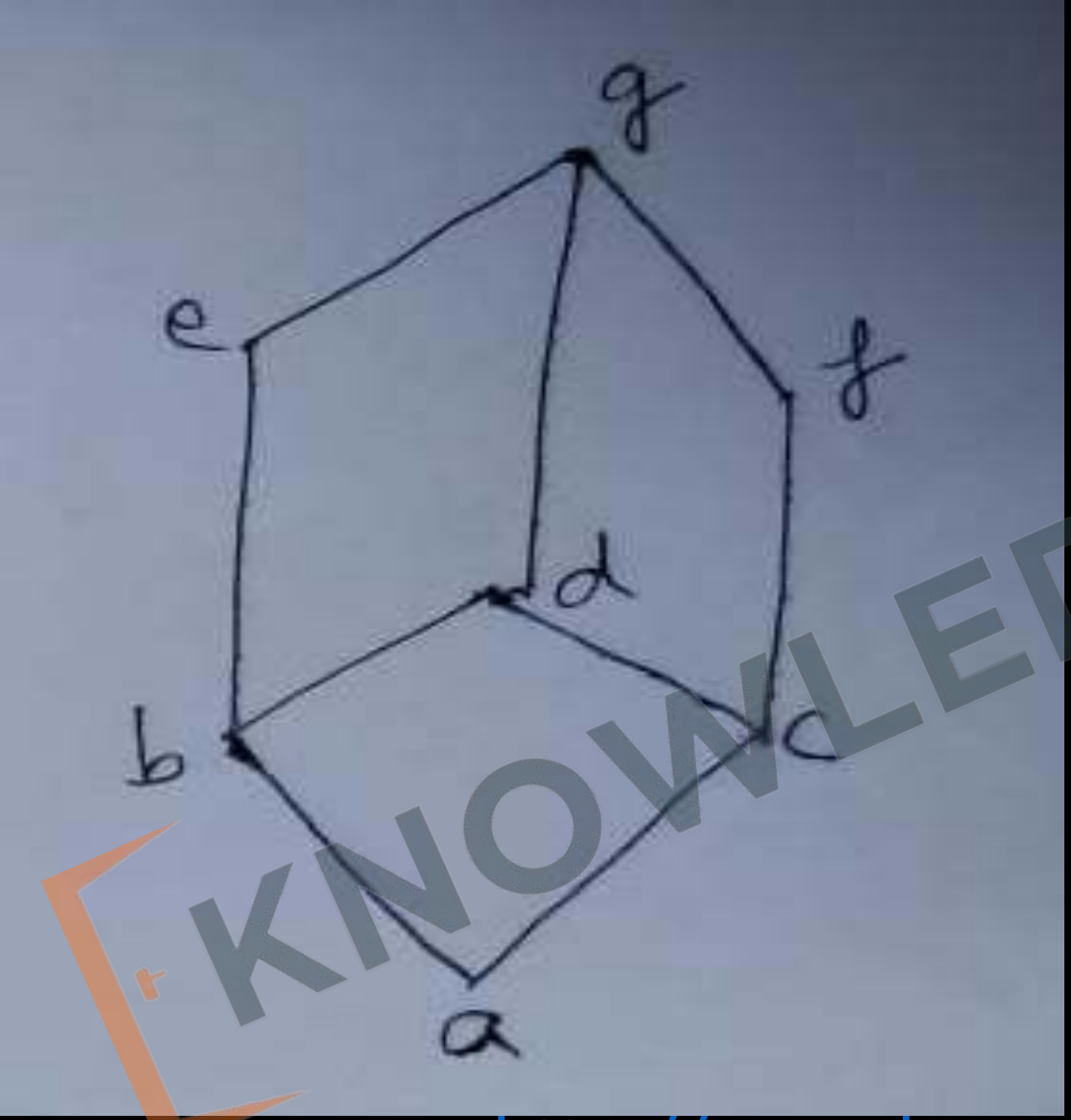
KNOWLEDGEGATE

wledgegate.in/gate



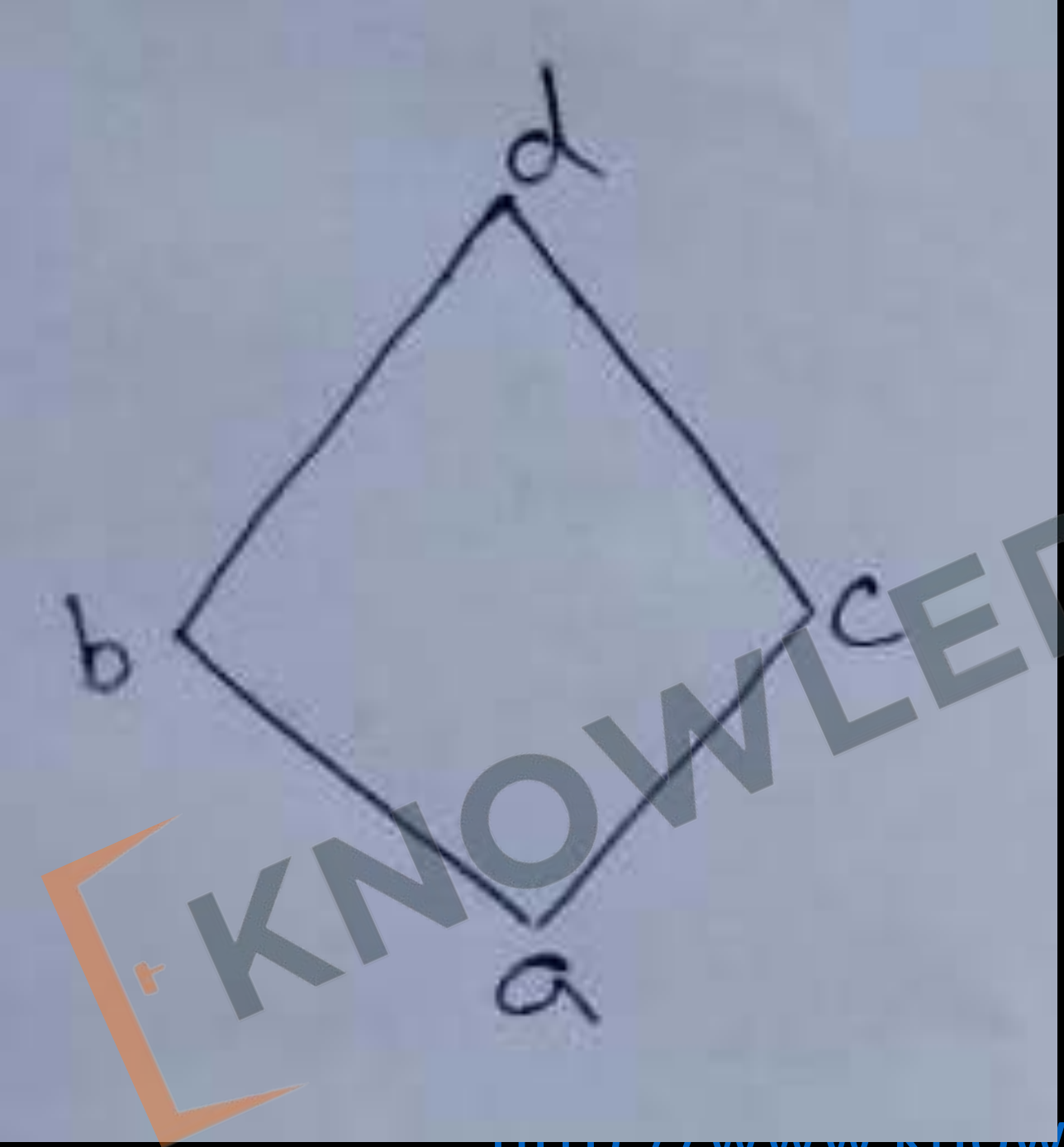
KNOWLEDGEGATE

wledgegate.in/gate



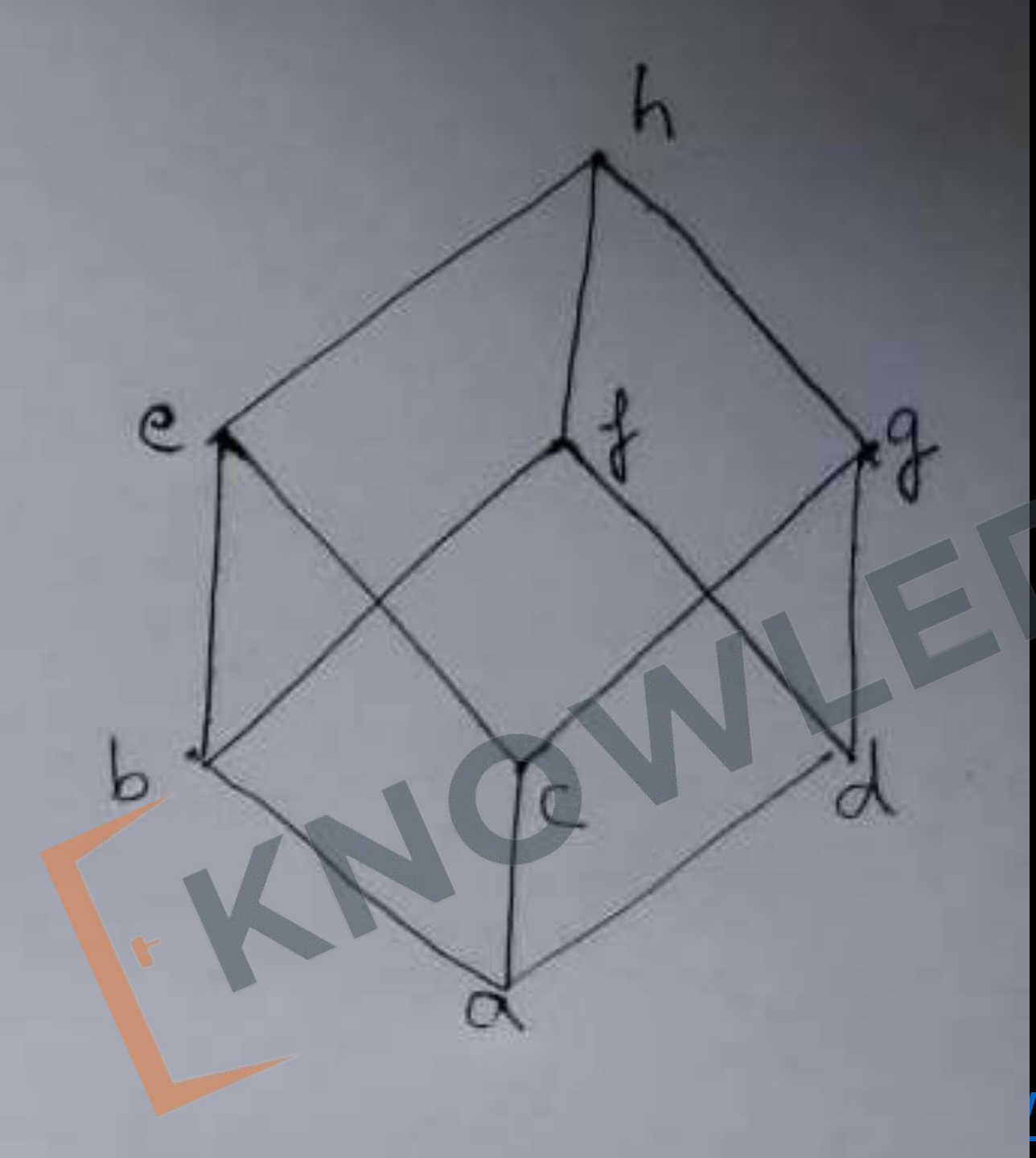
KNOWLEDGEGATE

http://www.knowledgegate.in/gate



KNOWLEDGEGATE

http://www.knowledgegate.in/gate



KNOWLEDGEGATE

wledgegate.in/gate

Q Find which of the following is a lattice and Boolean Algebra?

(1)$[D_{10}, /]$

(2) $[D_{12}, /]$

(3) $[D_{30}, /]$

(4) $[D_{45}, /]$

(5) $[D_{64}, /]$

(6) $[D_{81}, /]$

(7) $[D_{91}, /]$

(8) $[D_{110}, /]$

KNOWLEDGEGATE

Chapter-4 (Functions)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Function

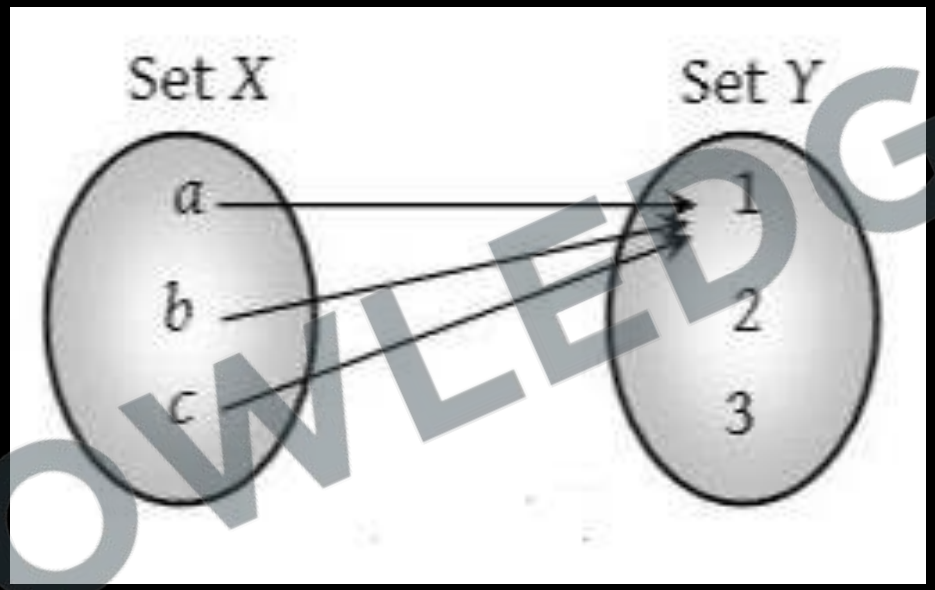
- Functions are widely used in science, and in most fields of mathematics. It has been said that functions are "the central objects of investigation" in most fields of mathematics.

- Let X = {1, 2, 3, 4, 5} and Y = {2, 3, 4, 5, 6, 7} and R ⊑ X * Y. Now this is a valid relation but not a function, because there is a element which is not participating in the relationship secondly 5 is relating with more than one element.

KNOWLEDGE GATE

Function

- In mathematics, a function is a relation between sets that associates to every element of a first set exactly one element of the second set.
- A relation ‘f’ from a set ‘A’ to a Set ‘B’ is called a function, if each element of A is mapped with a unique element on B.
- f: A→B



- Range of fun ⊑ B
- Range of f = { y | y ∈ B and (x, y) ∈ f}

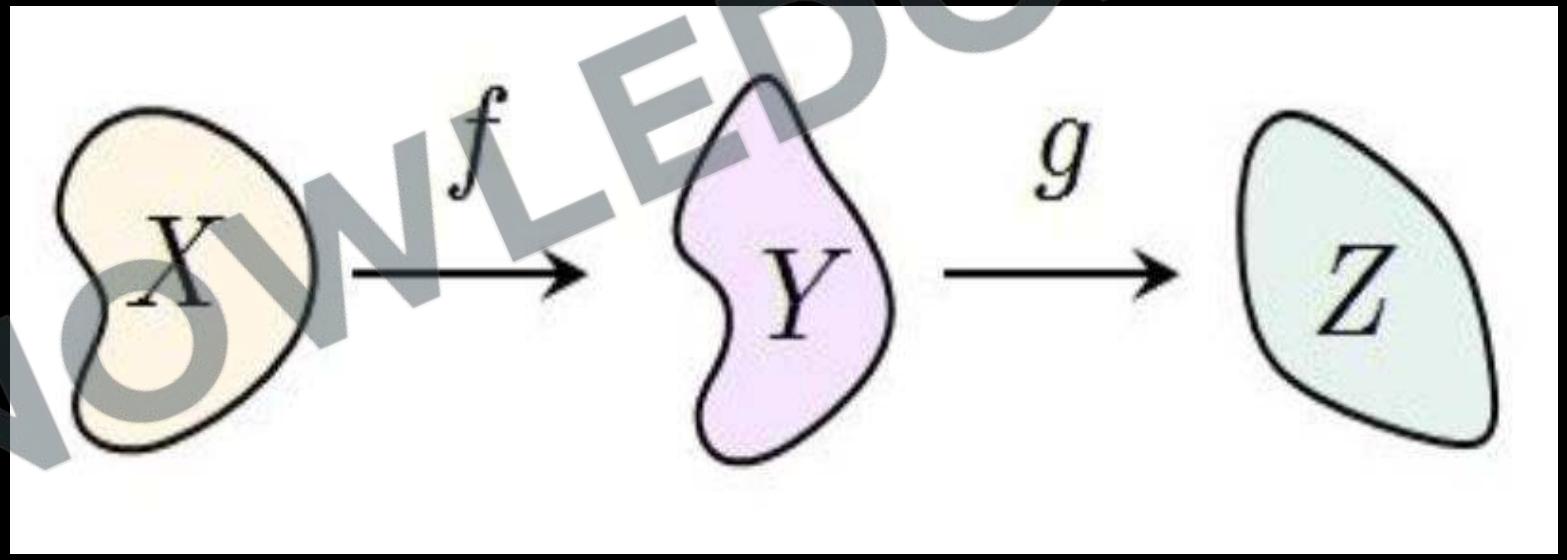
KNOWLEDGEGATE

- If |A| = m and |B|= n, then number of functions possible from A to B = n^m

KNOWLEDGEGATE

Function composition

- In mathematics, **function composition** is an operation that takes two functions *f* and *g* and produces a function *h* such that $h(x) = g(f(x))$.
- In this operation, the function *g* is applied to the result of applying the function *f* to *x*. That is, the functions $f: X \to Y$ and $g: Y \to Z$ are **composed** to yield a function that maps *x* in *X* to $g(f(x))$ in *Z*.



KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- fog(x) = f(g(x))
- gof(x) = g(f(x))

- Composition of functions on a finite set: If $f = \{(1, 3), (2, 1), (3, 4), (4, 6)\}$, and $g = \{(1, 5), (2, 3), (3, 4), (4, 1), (5, 3), (6, 2)\}$, then $g \circ f = \{(1, 4), (2, 5), (3, 1), (4, 2)\}$.
- The composition of functions is always associative—a property inherited from the composition of relations. That is, if f, g, and h are three functions with suitably chosen domains and codomains, then $f \circ (g \circ h) = (f \circ g) \circ h$

KNOWLEDGEGATE

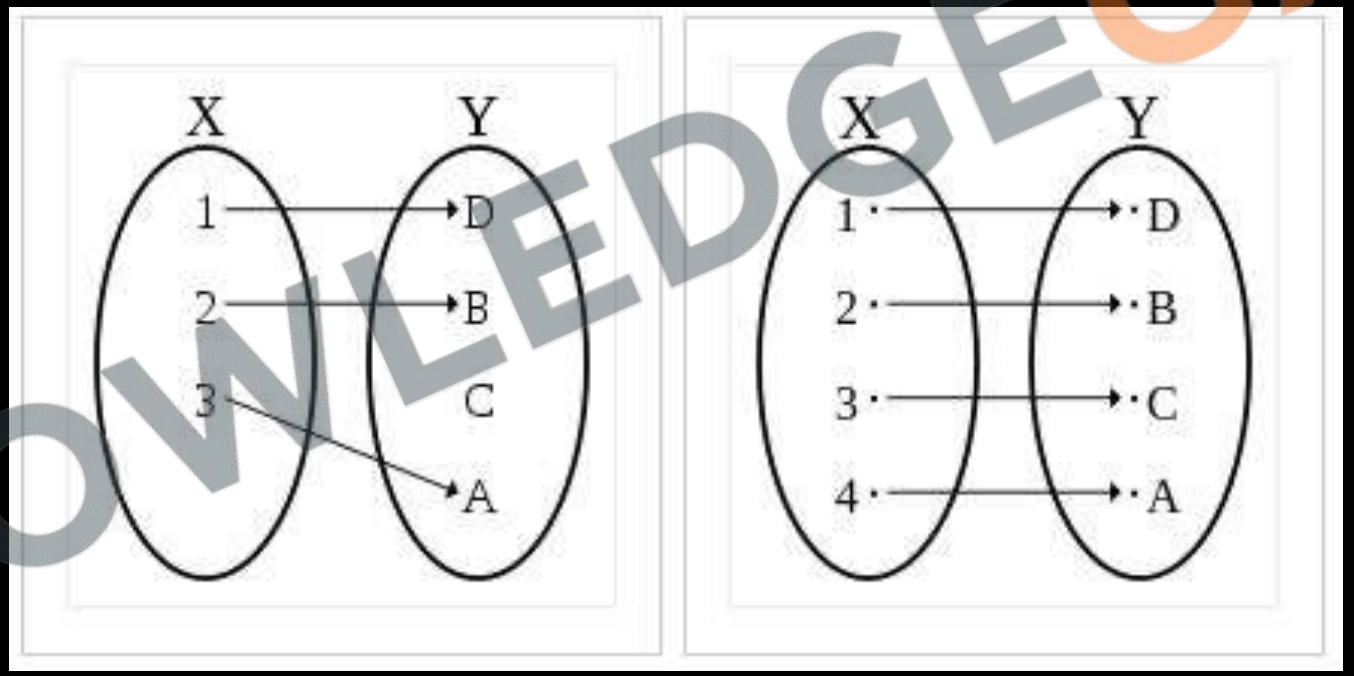
One-to-One (Injective Function)

- An **injective function** (also known as **injection**, or **one-to-one function**) is a function that maps distinct elements of its domain to distinct elements of its codomain. In other words, every element of the function's codomain is the image of *at most* one element of its domain.

KNOWLEDGEGATE

One-to-One (Injective Function)

- A function F: A→B is said to be one-to-one function if every element of A has distinct image in B
- If A and B are finite set, then one-to-one from A→B is possible
  - if |A| <= |B|



KNOWLEDGEGATE

- No of function possible = ${}^{n}p_{m}$ = P (n, m)
- If |A| = |B| = n, then no of functions possible is n!

KNOWLEDGEGATE

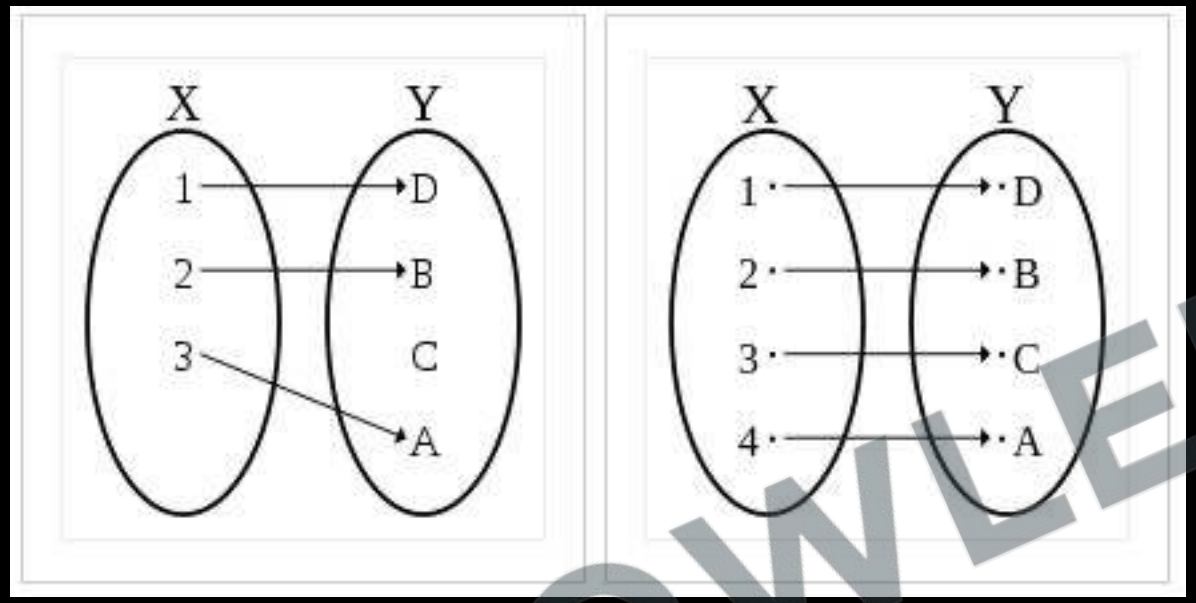
Onto (Surjective Function)

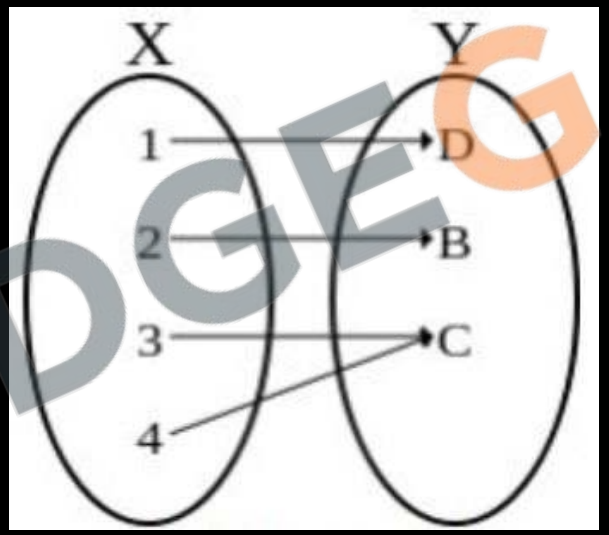
- A function *f* from a set *X* to a set *Y* is **surjective** (also known as **onto**, or a **surjection**), if for every element *y* in the co-domain *Y* of *f*, there is at least one element *x* in the domain *X* of *f* such that $f(x) = y$. It is not required that *x* be unique; the function *f* may map one or more elements of *X* to the same element of *Y*.

KNOWLEDGEGATE

Onto (Surjective Function)

- A function f: A → B is said to be onto if and only if every element of B is mapped by at least one element of A.
- Range of f = B





- If A and B are finite sets, then onto function from A→B is possible, |B|<=|A|
- If |A| = |B|, then every onto function from A to B is also one-to-one function.

KNOWLEDGEGATE

No of onto function possible from A to B

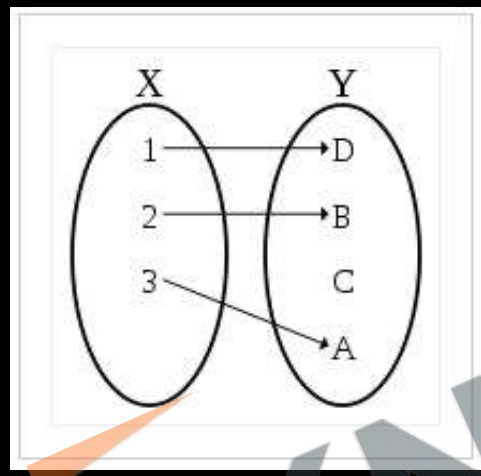
$= n^m - {}^nc_1(n-1)^m + {}^nc_2(n-2)^m - {}^nc_3(n-3)^m + \text{---------} + (-1)^n\ {}^nc_{n-1}\ 1^m$

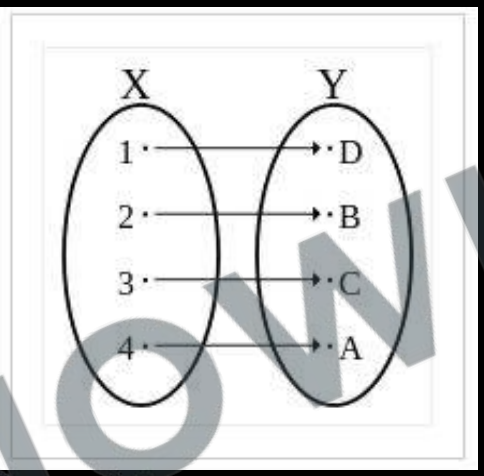
KNOWLEDGEGATE

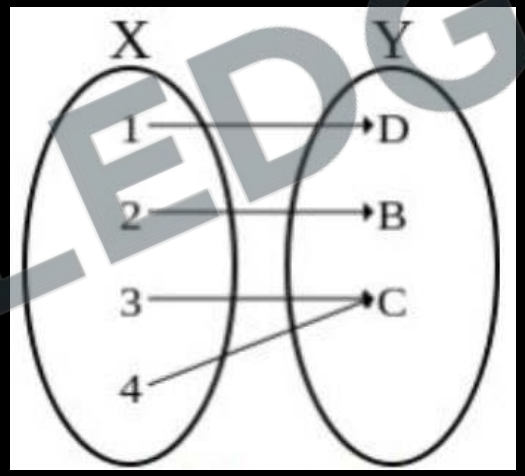
http://www.knowledgegate.in/gate

Bijective Function

- In mathematics, a **bijection**, **bijective function**, **one-to-one correspondence**, or **invertible function**, is a function between the elements of two sets, where each element of one set is paired with exactly one element of the other set, and each element of the other set is paired with exactly one element of the first set. There are no unpaired elements.







KNOWLEDGEGATE

- A function f: A→B is said to be bijection if f is one-to-one and onto.
- Bijection from A and B is possible, if |A|□|B|
- No of Bijection from A to B = ?

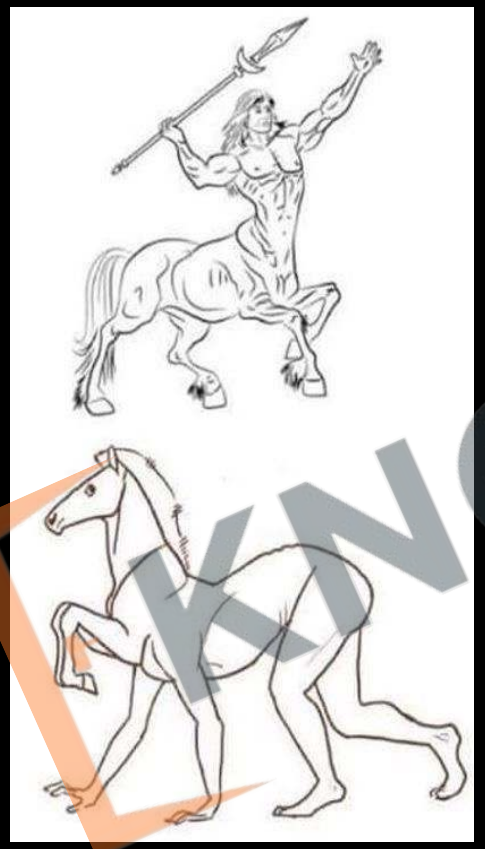
KNOWLEDGEGATE

- A function f: A→B is said to be bijection if f is one-to-one and onto.
- Bijection from A and B is possible, if |A| = |B|
- No of Bijection from A to B = n!

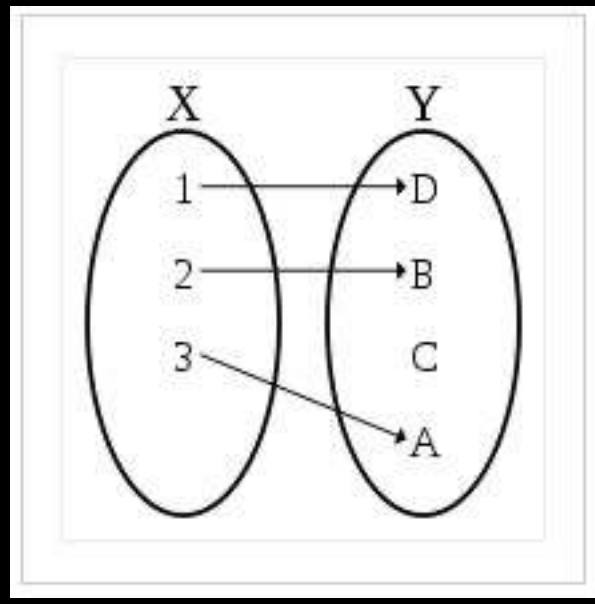
KNOWLEDGEGATE

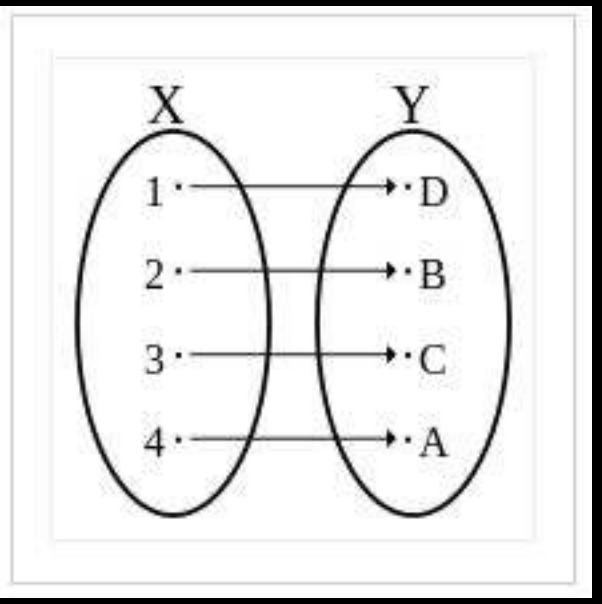
Inverse of a function

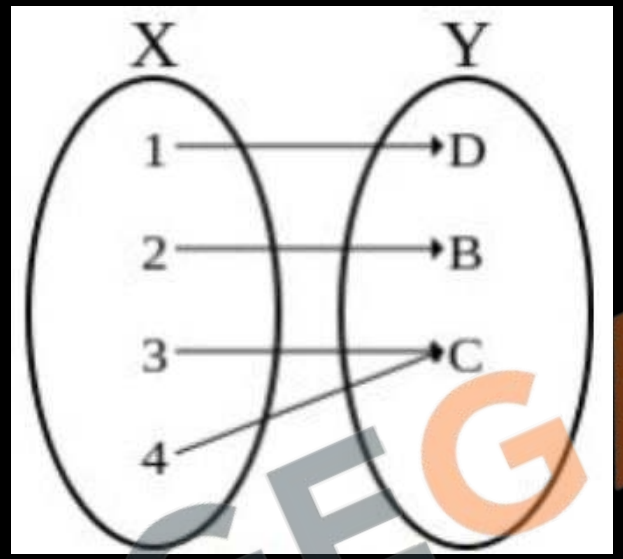
- In mathematics, an **inverse function** (or **anti-function**) is a function that "reverses" another function
- If the function *f* applied to an input *x* gives a result of *y*, then applying its inverse function f^{-1} to *y* gives the result *x*, and vice versa.
- $f(x) = y$ then $f^{-1}(y) = x$.



KNOWLEDGEGATE







KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- $f(x) = 5x - 7$
- $f^{-1}(y) = (y + 7)/5$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Let R denote the set of real numbers. Let $f: R \times R \rightarrow R \times R$ be a bijective function defined by $f(x, y) = (x + y, x - y)$. The inverse function of f is given by

a) $f^{-1}(x, y) = (1 / (x + y), 1 / (x - y))$

b) $f^{-1}(x, y) = (x - y, x + y)$

c) $f^{-1}(x, y) = ((x + y) / 2, (x - y) / 2)$

d) $f^{-1}(x, y) = [2(x - y), 2(x + y)]$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Chapter-5 (Theory of Logics)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- A **proposition** is a declarative sentence (that is, a sentence that declares a fact) that is either true or false, but not both.
  1. Delhi is the capital of USA
  2. How are you doing
  3. 5<= 11
  4. Temperature is less than 10 C
  5. It is cold today
  6. Read this carefully
  7. X + y = z

KNOWLEDGEGATE

- Premises(proposition) is always considered to be true. Premises is a statement that provides reason or support for the conclusion(proposition).
- If a set of Premises(P) yield another proposition Q(Conclusion), then it is called an Argument.
- An argument is said to be valid if the conclusion Q can be derived from the premises by applying the rules of inference.

$\{P_1, P_2, P_3..., P_N\} \vdash Q$	P_1 P_2 P_3 . . . P_N ..... Q	$\{P_1 \wedge P_2 \wedge P_3 \wedge .... \wedge P_N\} \vdash Q$

KNOWLEDGEGATE

Types of proposition

1. We use letters to denote **propositional variables** (or **statement variables**), that is, variables that represent propositions, just as letters are used to denote numerical variables.
2. The conventional letters used for propositional variables are *p, q, r, s.* The **truth value** of a proposition is true, denoted by T, if it is a true proposition, and the truth value of a proposition is false, denoted by F, if it is a false proposition.
3. Many mathematical statements are constructed by combining one or more propositions. New propositions, called **compound propositions**, are formed from existing propositions using logical operators.

KNOWLEDGEGATE

Operators / Connectives

1. **Negation: -** let p be a proposition, then negation of p new proposition, denoted by ¬*p*, is the statement "it is not the case that p".

2. The truth value of the negation of *p*, ¬*p*, is the opposite of the truth value of p. e.g. ¬"Michael's PC runs Linux" = "It is not the case that Michael's PC runs Linux." = "Michael's PC does not run Linux."

Negation	
P	¬P
F	
T	

KNOWLEDGE GATE

Conjunction

- Let *p* and *q* be propositions. The *conjunction* of *p* and *q*, denoted by *p* ∧ *q*, is the proposition "*p* and *q*."

- The conjunction *p* ∧ *q* is true when both *p* and *q* are true and is false otherwise.

Conjunction		
p	q	p ∧ q
F	F	
F	T	
T	F	
T	T	

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Q consider the following arguments and find which of them are valid?

1	
P_1	$(p \wedge q)$
Q	p

Simplification

2	
P_1	P
Q	$p \wedge q$

3	
P_1	$\neg(p \wedge q)$
P_2	P
Q	$\neg q$

Conjunctive Syllogism

4	
P_1	$\neg (p \wedge q)$
P_2	q
Q	$\neg p$

KNOWLEDGEGATE

Disjunction

- Let *p* and *q* be propositions. The *disjunction* of *p* and *q*, denoted by *p* ∨ *q*, is the proposition. "*p* or *q*." The disjunction *p* ∨ *q* is false when both *p* and *q* are false and is true otherwise.

Disjunction		
p	q	p ∨ q
F	F	
F	T	
T	F	
T	T	

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q consider the following arguments and find which of them are valid?

1	
P_1	$(p \wedge q)$
Q	$p \vee q$

2	
P_1	$p \vee q$
Q	$(p \wedge q)$

5	
P_1	$(p \vee q)$
P_2	$\neg p$
Q	q

6	
P_1	$(p \vee q)$
P_2	$\neg q$
Q	p

Disjunctive syllogism

$\frac{p}{p \vee q}$ Addition

KNOWLEDGEGATE

Implication

1. Let *p* and *q* be propositions. The *conditional statement* $p \rightarrow q$ is the proposition "if *p*, then q". The conditional statement $p \rightarrow q$ is false when *p* is true and *q* is false, and true otherwise.
2. In conditional statement $p \rightarrow q$, *p* is called the hypothesis (or antecedent or premise) and q is called the conclusion.
3. The statement $p \rightarrow q$ is called a conditional statement because $p \rightarrow q$ asserts that *q* is true on the condition that *p* holds.

Implication		
p	q	p q
F	F	
F	T	
T	F	
T	T	

KNOWLEDGEGATE

p	q	P → q	¬p	¬q	¬P → ¬q	q → p	¬q → ¬p	¬p ∨ q
F	F							
F	T							
T	F							
T	T							

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

1. *p → q implication*
2. *q → p converse*
3. *¬p → ¬q inverse*
4. *¬q → ¬p contra positive*

KNOWLEDGEGATE

1. $p \rightarrow q = \neg q \rightarrow \neg p$

2. $p \rightarrow q$ will be true if either p is false or q is true, $p \rightarrow q = \neg p \vee q$

KNOWLEDGEGATE

Q Express Converse, Inverse and Contrapositive of the following Statement "If x + 5 = 8 then x = 3"

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q consider the following arguments and find which of them are valid?

Modus Ponens	
P_1	$p \rightarrow q$
P_2	P
Q	Q

Modus Tollens	
P_1	$p \rightarrow q$
P_2	$\neg Q$
Q	$\neg p$

1	
P_1	$\neg p$
Q	$p \rightarrow q$

2	
P_1	q
Q	$p \rightarrow q$

KNOWLEDGEGATE

Bi-conditional

- Let *p* and *q* be propositions. The bi*conditional statement* $p \leftrightarrow q$ is the proposition.
  - *" p if and only* q".
- The biconditional statement $p \leftrightarrow q$ *is true when p and q have the same values, and false otherwise. Biconditional statements are also called bi-implications.*
  - "*p* is necessary and sufficient for *q*"
  - "if *p* then *q*, and conversely"
  - "*p* iff *q*."
  - $p \leftrightarrow q = (p \rightarrow q) \wedge (q \rightarrow p)$

Bi-conditional		
p	q	P q
F	F	
F	T	
T	F	
T	T	

KNOWLEDGEGATE

Q Construct the truth table for the following statement ?

$(P \rightarrow \neg Q) \rightarrow \neg P$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q The statement $(\neg p) \Longrightarrow (\neg q)$ is logically equivalent to which of the statement below?

1) $p \Longrightarrow q$ **2)** $q \Longrightarrow p$ **3)** $(\neg q) \vee (p)$ **4)** $(\neg p) \vee q$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Type of cases

- **Tautology/valid: -** A propositional function which is always having truth in the last column, is called tautology. E.g. $p \vee \neg p$

p	¬ p	p V ¬ p
F	T	
T	F	

KNOWLEDGE GATE

- **<u>Contradiction/ Unsatisfiable</u>: -** A propositional function which is always having false in the last column, is called Contradiction. E.g. $p \wedge \neg p$

p	¬ p	p ∧ ¬ p
F	T	
T	F	

KNOWLEDGEGATE

- **Contingency:** - A propositional function which is neither a tautology nor a contradiction, is called Contingency. E.g. ***p*** V ***q***

p	q	p V q
F	F	
F	T	
T	F	
T	T	

KNOWLEDGE GATE

- **Satisfiable: -** A propositional function which is not contradiction is satisfiable. i.e. it must have at least one truth value in the final column e.g. $p \vee q$

KNOWLEDGEGATE

- **Functionality Complete Set: -** A set of connectives is said to be functionally complete if it is able to write any propositional function.
  - {∧, ¬}
  - {∨, ¬}

KNOWLEDGEGATE

Q consider the following argument

I_1: if today is Gandhi ji's birthday, then today is oct 2nd

I_2: today is oct 2nd

C: today is Gandhi ji's birthday

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q consider the following argument

I_1: if Canada is a country, then London is a city

I_2: London is not a city

C: Canada is not a country

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the following logical inferences.

I_1: If it rains then the cricket match will not be played.

The cricket match was played.

Inference: There was no rain.

I_2: If it rains then the cricket match will not be played.

It did not rain.

Inference: The cricket match was played.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the following propositional statements:

$P_1 : ((A \wedge B) \rightarrow C)) \equiv ((A \rightarrow C) \wedge (B \rightarrow C))$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the following propositional statements:

$P_2 : ((A \vee B) \rightarrow C)) \equiv ((A \rightarrow C) \neg (B \rightarrow C))$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Use rules of inference to justify that the three hypotheses
i. "If it does not rain or if it is not foggy, then the sailing race will be held and the lifesaving demonstration will go on."
ii. "If the sailing race is held, then the trophy will be awarded."
iii. "The trophy was not awarded." imply the conclusion
iv. "It rained."

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Show that ((p ∨ q) ∧ ¬(¬p ∧ (¬q ∨ ¬r))) ∨ (¬p ∧ ¬q) ∨ (¬p ∨ r) is tautology without using truth table.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q "If the labour market is perfect then the wages of all persons in a particular employment will be equal. But it is always the case that wages for such persons are not equal therefore the labour market is not perfect". Test the validity of this argument using truth table.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Prove the validity of the following argument.
If Mary runs for office, she will be elected. If Mary attends the meeting, she will run for office. Either Mary will attend the meeting or she will go to India. But Mary cannot go to India."Thus Mary will be elected".

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Prove the validity of the following argument.
If Mary runs for office, she will be elected. If Mary attends the meeting, she will run for office. Either Mary will attend the meeting or she will go to India. But Mary cannot go to India."Thus Mary will be elected".

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Prove the validity of the following argument "if the races are fixed so the casinos are cooked, then the tourist trade will decline. If the tourist trade decreases, then the police will be happy. The police force is never happy. Therefore, the races are not fixed."

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Prove the validity of the following argument "If I get the job and work hard, then I will get promoted. If I get promoted, then I will be happy. I will not be happy. Therefore, either I will not get the job, or I will not work hard."

KNOWLEDGEGATE

- **Canonical SOP form**: - In a sum of product form expression, if each AND term (product term) consists all the literals(variables) appearing either in complements or uncomplemented form. E.g. a'bc + ab'c' + abc. Then the form is said to be canonical SOP. Here we have Principle Disjunction Normal Form

- **Canonical POS form**: - In a product of sum form expression, if each OR term (sum term) consists all the literals(variables) appearing either in complements or uncomplemented form. E.g. (a' + b + c). (a + b' + c'). (a + b + c). Then the form is said to be Canonical POS form. Here we have Principle Conjunctive Normal Form.

KNOWLEDGE GATE

First order Predicate Logic

- Sometime propositional logic cannot derive any meaningful information even though, we as human can understand that argument is meaningful or not.

- P_1: Every Indian like cricket
- P_2: Sunny is an Indian
- Q: Sunny Likes cricket

- The reason propositional logic fails here because using only inference system we can not conclude Q from P_1 and P_2.

KNOWLEDGEGATE

- In first order logic we understand, a new approach of subject and predicate to extract more information from a statement
  - 1 is a natural number (1 is subject, natural number is predicate)
  - we can write FOPL (short hand notation) for this as NatNo(1) = 1 is natural number
  - Similarly, we can understand the meaning of NatNo(2) as 2 is a natural number
  - NatNo(x): x is a natural number

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- Sometime subject is not a single element but representing the entire group.
  - Every Indian like Cricket.
  - We can have a propositional function Cricket(x): x likes Cricket.
  - We can fix domain of discussion or universe of discourse as, x is an Indian.

KNOWLEDGEGATE

- If i say four Indian are there I_1, I_2, I_3, I_4
- I_1 likes cricket $\wedge$ I_2 likes cricket $\wedge$ I_3 likes cricket $\wedge$ I_4 likes cricket
- Cricket(I_1) $\wedge$ Cricket(I_2) $\wedge$ Cricket(I_3) $\wedge$ Cricket(I_4)
- But problem with this notation is as there is 130+ corers Indian this formula will become very long and in some case we actually do not know how many subjects are there in the universe of discourse. so, we again need a short hand formula.
- $\forall_x$ Cricket(x), if we confine x to be Indian then it means every x like cricket.

KNOWLEDGEGATE

- **<u>Universal quantifiers</u>**: - The universal quantification of a propositional function is the proposition that asserts

- P(x) is true for all values of x in the universe of discourse.

- The universe of discourse specifies the possible value of x.

- $\forall_x$ P(x), i.e. for all value of a P(x) is true

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- Let try some other statement ‘Some Indian like samosa’
  - if i say four Indian are there I_1, I_2, I_3, I_4
  - I_1 like samosa $\vee$ I_2 like samosa $\vee$ I_3 like samosa $\vee$ I_4 like samosa
  - Samosa(I_1) $\vee$ Samosa(I_2) $\vee$ Samosa(I_3) $\vee$ Samosa(I_4)
  - $\exists_x$ Samosa(x), if we confine x to be Indian then it means some x likes samosa.

KNOWLEDGEGATE

- **<u>Existential quantifiers</u>**: - with existential quantifier of a propositional that is true if and only if P(x) is true for at least one value of x in the universe of discourse.

- There exists an element x is the universe of discourse such that P(x) is true.

- $\exists_x$ P(x), i.e. for at least one value of a P(x) is true

KNOWLEDGEGATE

- let's change the universe of discourse from Indian to human
  - if human is Indian then it likes cricket
  - Indian(x): x is an Indian
  - Cricket(x): x likes Cricket

  - if I_1 is Indian then likes cricket $\wedge$ if I_2 is Indian then likes cricket $\wedge$ if I_3 is Indian then likes cricket $\wedge$ if I_4 is Indian then likes cricket

  - $[Indian(I_1) \rightarrow cricket(I_1)] \wedge [Indian(I_2) \rightarrow cricket(I_2)] \wedge [Indian(I_3) \rightarrow cricket(I_3)] \wedge [Indian(I_4) \rightarrow cricket(I_4)]$

  - $\forall_x [Indian(x) \rightarrow cricket(x)]$

KNOWLEDGEGATE

- let’s change the universe of discourse from Indian to human
  - if human is Indian then it likes samosa
  - Indian(x): x is an Indian
  - Samosa(x): x likes Samosa

  - if I_1 is Indian then likes samosa $\vee$ if I_2 is Indian then likes samosa $\vee$ if I_3 is Indian then likes samosa $\vee$ if I_4 is Indian then likes samosa

  - $[Indian(I_1) \wedge samosa(I_1)] \vee [Indian(I_2) \wedge samosa(I_2)] \vee [Indian(I_3) \wedge samosa(I_3)] \vee [Indian(I_4) \wedge samosa(I_4)]$

  - $\exists_x [Indian(x) \wedge samosa(x)]$

[KNOWLEDGEGATE]

- let check validity of a statement “Some Indians like samosa” = $\exists_x$ [Indian(x) →samosa(x)], x is human
- let human contains four elements I_1, I_2, I_3, I_4 out of which I_1, I_2 are Indian while I_3, I_4 are not Indian
- Suppose I_1, I_2, I_3 do not likes samosa
  - [Indian(I_1) → samosa(I_1)] ∨ [Indian(I_2) → samosa(I_2)] ∨ [Indian(I_3) → samosa(I_3)]
  - [T → F] ∨ [T → F] ∨ [F → F]
  - [F] ∨ [F] ∨ [T]
  - T
- conclusion $\exists_x$ is not used with →

[KNOWLEDGEGATE]

Negation

- $\neg [\forall_x P(x)] = \exists_x \neg P(x)$
- $\neg [\exists_x P(x)] = \forall_x \neg P(x)$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Universal specification: By this rule if the premise $\forall_x P(x)$ is true then P(c) is true where c is particular member of Universe of Discourse.

$$\forall_x P(x)$$

$$P(c)$$

KNOWLEDGEGATE

Universal generalization: By this rule if P(c) is true for all c in Universe of Discourse then $\forall_x P(x)$ is true.

$$P(c)$$

$$\forall_x P(x)$$

KNOWLEDGEGATE

Existential specification: By this rule if $\exists_x P(x)$ is true then P(x) is true for some particular member of Universe of Discourse.

$$\exists_x P(x)$$

$$P(c)$$

KNOWLEDGEGATE

Existential generalization: By this rule if P(c) is true for some particular member c in Universe of Discourse, then $\exists_x P(x)$ is true

$$P(c)$$

$$\exists_x P(x)$$

KNOWLEDGEGATE

Universal modus ponens: By this rule if $P(x) \rightarrow Q(c)$ is true for every x and P(c) is true for some particular member c in Universe of Discourse then Q(c) is true.

$$\forall_x P(x) \rightarrow Q(x)$$
$$P(c)$$

$$Q(c)$$

KNOWLEDGEGATE

Universal modus tollens: By this rule if $P(x) \rightarrow Q(x)$ is true for every x and $\sim Q(c)$ is true for some particular c in Universe of Discourse then $\sim Q(c)$ is true.

$$\forall_x P(x) \rightarrow Q(x)$$
$$\sim Q(c)$$

$$\sim P(c)$$

KNOWLEDGEGATE

Q The CORRECT formula for the sentence is

"not all rainy days are cold"

$\exists_d (Rainy(d) \wedge \sim Cold(d))$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the statement

"Not all that glitters is gold"

$\exists_x$: glitters(x) $\wedge$ ¬gold(x)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the statement

"None of my friends are perfect."

$$\neg\exists_x(F(x) \wedge P(x))$$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the statement

"Some real numbers are rational"

$\exists_x (\text{real}(x) \wedge \text{rational}(x))$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the statement

"Gold and silver ornaments are precious".

$$\forall_x ((G(x) \vee S(x)) \rightarrow P(x))$$

KNOWLEDGEGATE

Q Consider the statement

"Not every graph is connected" ?

$\neg \forall_x$ (Graph(x) $\rightarrow$ Connected(x))

$\exists_x$ (Graph(x) $\wedge$ $\neg$Connected(x))

$\neg \forall_x$ ($\neg$Graph(x) $\vee$ Connected(x))

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the statement

“Tigers and lions attack if they are hungry or threatened”

$\forall_x[(\text{tiger}(x) \vee \text{lion}(x)) \rightarrow (\text{hungry}(x) \vee \text{threatened}(x)) \rightarrow \text{attacks}(x)]$

KNOWLEDGEGATE

Q Consider the statement

Every teacher is liked by some student

$\forall_{(x)}$ [teacher (x) → $\exists_{(y)}$[student (y) ^ likes (y, x)]]

KNOWLEDGEGATE

Q Consider the statement

Some boys in the class are taller than all the girls

$(\exists_x)\ (boy(x) \wedge (\forall_y)\ (girl(y) \rightarrow taller\ (x, y)))$

KNOWLEDGEGATE

Q Convert the following two statements in quantified expressions of predicate logic

- For every number there is a number greater than that number.

P(x) : x is a number
Q(y) : y is a number greater than x
$\forall_x(P(x) \rightarrow Q(y))$

- Sum of every two integer is an integer.

P(x) : x is a integer
S(x) : x is sum of integer
$\forall_x(S(x) \rightarrow P(x))$

KNOWLEDGEGATE

- Not Every man is perfect.

P(x) : x is perfect man

$\sim \forall_x(P(x))$

- There is no student in the class who knows Spanish and German.

P(x) : x is a student

L(x) : x knows Spanish and German

$\exists_x(P(x) \vee L(x))$

KNOWLEDGEGATE

Chapter-6 (Algebraic Structures)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

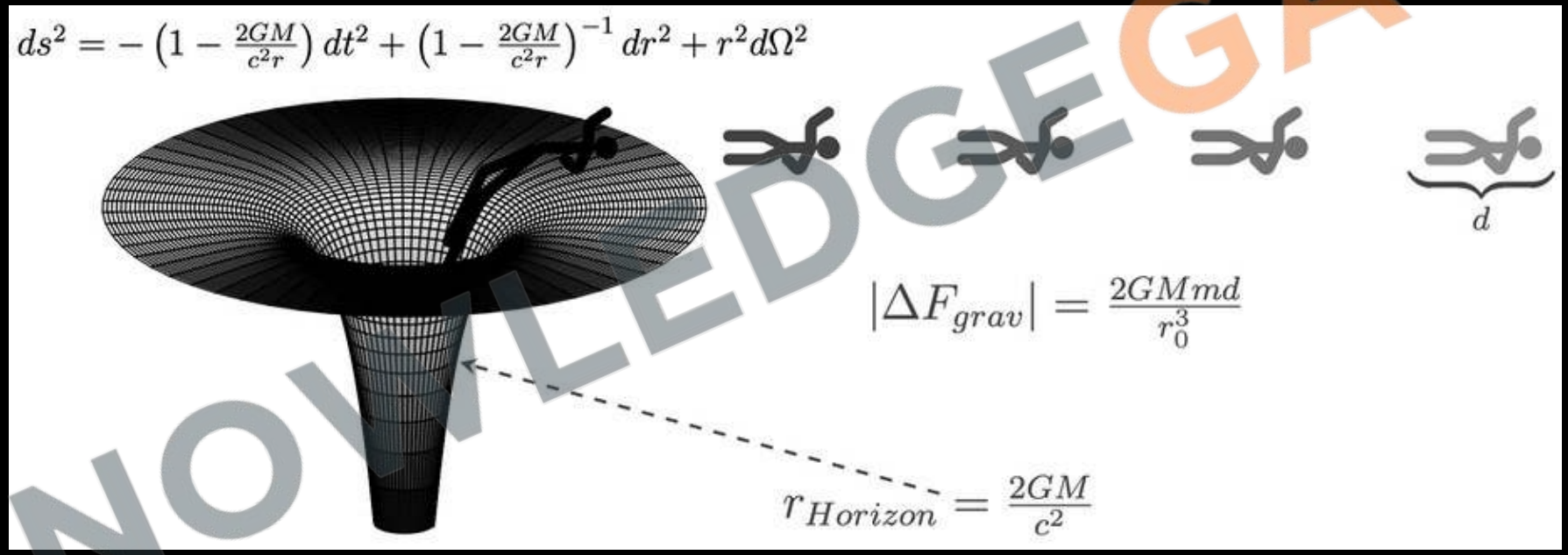
Group Theory

- Group theory is very important mathematical tool which is used in a number of areas in research and application. Using group theory, we can estimate the strength of a set with respect to an operator.

KNOWLEDGE GATE

http://www.knowledgegate.in/gate

- This idea will further help us in research field to identify the correct mathematical system to work in a particular research area. E.g. can we use natural numbers in complex problem area like soft computing or studying black holes.



KNOWLEDGEGATE

- Now we will directly study some of the basic set related properties and will define some structure of set and operator based on the properties and will check those properties on basics number systems like natural numbers, integers, real numbers etc.

KNOWLEDGE GATE

1. **Closure property: -** Consider a non-empty set A and a binary operation * on A. A is said to be closed with respect to *, if ∀ a, b ∈ A, then a*b ∈ A.

2. **Algebraic Structure: -** A non-empty set A is said to be an algebraic structure with respect to a binary operation *, if A satisfy closure property with respect to *.

		Algebraic Structure
1	(N, +)	
2	(N, -)	
3	(N, /)	
4	(N, x)	
5	(Z, +)	
6	(Z, -)	
7	(Z, /)	
8	(Z, x)	
9	(R, +)	
10	(R, -)	
11	(R, /)	
12	(R, x)	
13	(M, +)	
14	(M, x)	
15	(E, +)	
16	(E, x)	
17	(O, +)	
18	(O, x)	
19	(R-0, x)	
20	(R-0, /)	
21	(Non-Singular Matrix, x)	

KNOWLEDGEGATE

1. **Associative property: -** Consider a non-empty set A and a binary operation * on A. A is said to be associative with respect to *, if ∀ a, b, c ∈ A, then (a*b) *c = a*(b*c)

2. **Semi-Group: -** A non-empty set A is said to be a Semi-group with respect to a binary operation *, if A satisfy closure, Associative property with respect to *.

KNOWLEDGEGATE

http://www.knowledgegate.in

		Algebraic Structure	Semi Group
1	(N, +)	Y	
2	(N, -)	N	
3	(N, /)	N	
4	(N, x)	Y	
5	(Z, +)	Y	
6	(Z, -)	Y	
7	(Z, /)	N	
8	(Z, x)	Y	
9	(R, +)	Y	
10	(R, -)	Y	
11	(R, /)	N	
12	(R, x)	Y	
13	(M, +)	Y	
14	(M, x)	Y	
15	(E, +)	Y	
16	(E, x)	Y	
17	(O, +)	N	
18	(O, x)	Y	
19	(R-0, x)	Y	
20	(R-0, /)	Y	
21	(Non-Singular Matrix, x)	Y	

1. **Identity property: -** Consider a non-empty set A and a binary operation * on A. A is said to satisfy identity property with respect to *, if ∀ a ∈ A, there must be unique e ∈ A, such that
   a*e = e*a = a

2. There is exactly one Identity element in the set and will be same for all element in the set.

3. **Monoid: -** A non-empty set A is said to be a Monoid with respect to a binary operation *, if A satisfy closure, Associative, identity property with respect to *.

		Algebraic Structure	Semi Group	Monoid
1	(N, +)	Y	Y	
2	(N, -)	N	N	
3	(N, /)	N	N	
4	(N, x)	Y	Y	
5	(Z, +)	Y	Y	
6	(Z, -)	Y	N	
7	(Z, /)	N	N	
8	(Z, x)	Y	Y	
9	(R, +)	Y	Y	
10	(R, -)	Y	N	
11	(R, /)	N	N	
12	(R, x)	Y	Y	
13	(M, +)	Y	Y	
14	(M, x)	Y	Y	
15	(E, +)	Y	Y	
16	(E, x)	Y	Y	
17	(O, +)	N	N	
18	(O, x)	Y	Y	
19	(R-0, x)	Y	Y	
20	(R-0, /)	Y	N	
21	(Non-Singular Matrix, x)	Y	Y	

KNOWLEDGEGATE

1. **Inverse property: -** Consider a non-empty set A and a binary operation * on A. A is said to satisfy inverse property with respect to *, if ∀ a ∈ A, there must be unique element $a^{-1} \in A$, such that $a * a^{-1} = a^{-1} * a = e$

2. Every element has a exactly one unique inverse which is also present in the same set.

3. If a is the inverse of b, then b will be invers one a.

4. No two elements can have the same inverse

5. Identity element is its own inverse.

6. **Group: -** A non-empty set A is said to be a group with respect to a binary operation *, if A satisfy closure, Associative, identity, inverse property with respect to *.

KNOWLEDGE GATE

http://www.knowledge

		AS	Semi Group	Monoid	Group
1	(N, +)	Y	Y	N	
2	(N, -)	N	N	N	
3	(N, /)	N	N	N	
4	(N, x)	Y	Y	Y	
5	(Z, +)	Y	Y	Y	
6	(Z, -)	Y	N	N	
7	(Z, /)	N	N	N	
8	(Z, x)	Y	Y	Y	
9	(R, +)	Y	Y	Y	
10	(R, -)	Y	N	N	
11	(R, /)	N	N	N	
12	(R, x)	Y	Y	Y	
13	(M, +)	Y	Y	Y	
14	(M, x)	Y	Y	Y	
15	(E, +)	Y	Y	Y	
16	(E, x)	Y	Y	N	
17	(O, +)	N	N	N	
18	(O, x)	Y	Y	Y	
19	(R-0, x)	Y	Y	Y	
20	(R-0, /)	Y	N	N	
21	(Non-Singular Matrix, x)	Y	Y	Y	

1. If the total number of elements in a group is even then there exists at least one element in the group who is the inverse of itself.

2. Some time it is also possible that every element is inverse of itself in a group.

3. In a group $(a * b)^{-1} = b^{-1} * a^{-1}$ for $\forall\ a, b \in A$

4. Cancelation law holds good

    1. $a * b = a * c \rightarrow b = c$

    2. $a * c = b * c \rightarrow a = b$

KNOWLEDGEGATE

1. **Commutative property: -** Consider a non-empty set A and a binary operation * on A. A is said to satisfy commutative property with respect to *, if ∀ a, b ∈ A, such that a* b = b*a

2. **Abelian Group: -** A non-empty set A is said to be a group with respect to a binary operation *, if A satisfy closure, Associative, identity, inverse, commutative property with respect to *.

		AS	SG	Monoid	Group	Abelian Group
1	(N, +)	Y	Y	N	N	
2	(N, -)	N	N	N	N	
3	(N, /)	N	N	N	N	
4	(N, x)	Y	Y	Y	N	
5	(Z, +)	Y	Y	Y	Y	
6	(Z, -)	Y	N	N	N	
7	(Z, /)	N	N	N	N	
8	(Z, x)	Y	Y	Y	N	
9	(R, +)	Y	Y	Y	Y	
10	(R, -)	Y	N	N	N	
11	(R, /)	N	N	N	N	
12	(R, x)	Y	Y	Y	N	
13	(M, +)	Y	Y	Y	Y	
14	(M, x)	Y	Y	Y	N	
15	(E, +)	Y	Y	Y	Y	
16	(E, x)	Y	Y	N	N	
17	(O, +)	N	N	N	N	
18	(O, x)	Y	Y	Y	N	
19	(R-0, x)	Y	Y	Y	Y	
20	(R-0, /)	Y	N	N	N	
21	(Non-Singular Matrix, x)	Y	Y	Y	Y	

KNOWLEDGE GATE

Q let A = {1, 3, 5, ……., ∞} and B = {2, 4, 6, ……., ∞}, what is the highest structure achieved by anyone of them?

1) (A,+) **2)** (A,*) **3)** (B,+) **4)** (B, *)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider a set of natural numbers N, with respect to *, such that a * b = a^b which structure it will satisfy?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q let {p, q, r, s} be the set. A binary operation * is defined on the set and is given by the following table: Which of the following is true about the binary operation?

a) it is commutative but not associative
b) it is associative but not commutative
c) it is both associative and commutative
d) it is neither associative nor commutative

*	p	q	r	s
P	p	r	s	p
q	p	q	r	s
r	p	q	p	r
S	p	q	q	q

KNOWLEDGEGATE

Q which of the following is not a group?

a) {……. -6, -4, -2, 0, 2, 4, 6, ……}, +

b) {…. -3k, -2k, -k, 0, k, 2k, 3k, ….}, + [k ∈ Z]

c) {2^n, n ∈ Z}, *

d) set of complex number, *

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the set of all integers(Z) with the operation defined as $m * n = m + n + 2, m, n \in Z$

if (z, *) forms a group, then determine the identity element

a) 0 **b)** -1 **c)** -2 **d)** 2

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider a set of positive rational number with respect to an operation *, such that a*b = (a.b)/3, it is known that the it is an abelian group, which of the following is not true?

a) identity element e = 3

b) inverse of a = 9/a

c) inverse of 2/3 = 6

d) inverse of 3 = 3

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q A binary operation α on a set of integers is defined as $x * y = x^2 + y^2$. Which one of the following statements is TRUE about *?

(A) Commutative but not associative

(B) Both commutative and associative

(C) Associative but not commutative

(D) Neither commutative nor associative

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Which one of the following in NOT necessarily a property of a Group?

(A) Commutativity

(B) Associativity

(C) Existence of inverse for every element

(D) Existence of identity

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Finite Group: -** A group with finite number of elements is called a finite group.

- **Order of group: -** Order of a group is denoted by O(G) = no of elements in G
  - If there is only one element in the Group, it must be an identity element.

KNOWLEDGEGATE

Q Check out which of the following is a finite group?

1- {0}, +

+	0
0	

2- {0}, *

*	0
0	

3- {1}, +

+	1
1	

4- {1}, *

*	1
1	

5- {0,1}, +

+	0	1
0		
1		

6- {0,1}, *

*	0	1
0		
1		

7- {-1, 0, 1}, +

+	-1	0	1
-1			
0			
+1			

8- {-1, 0, 1}, *

*	-1	0	1
-1			
0			
1			

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Check out which of the following is a finite group?

9- {-1, 1}, +

10- {-1, 1}, *

11- {-2, -1, 0, 1, 2}, +

+	-1	1
-1		
1		

*	-1	1
-1		
1		

+	-2	-1	0	1	2
-2					
-1					
0					
1					
2					

KNOWLEDGEGATE

Q Check out which of the following is a finite group?

12- {-2, -1, 0, 1, 2}, *

*	-2	-1	0	1	2
-2					
-1					
0					
1					
2					

13- {1, ω, ω^2}, *

*	1	ω	ω^2
1			
ω			
ω^2			

14- {-1, 1, i, -i}, *

*	-1	1	i	-i
-1				
1				
i				
-i				

KNOWLEDGEGATE

1. **Conclusion: -** it is very difficult to design finite group as with number greater than 2 closure property fails with simple addition and multiplication operation.

2. So we will try to develop new modified addition and multiplication operators with which closure and other properties can be satisfied.

KNOWLEDGEGATE

- **<u>Addition modulo</u>: -** addition modulo is a binary operator denoted by $+_m$ such that

- $a +_m b = a + b$ if $(a + b < m)$

- $a +_m b = a + b - m$ if $(a + b >= m)$

$\{0,1,2,3\}, +_4$

$+_4$	0	1	2	3
0				
1				
2				
3				

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Multiplication modulo: -** Multiplication modulo is a binary operator denoted by $*_m$ such that

- $a *_m b = a * b$ if $(a * b < m)$

- $a *_m b = (a * b) \% m$ if $(a * b >= m)$

$\{1,2,3,4\}, *_5$

$*_5$	1	2	3	4
1				
2				
3				
4				

[KNOWLEDGEGATE]

http://www.k

Q Check out which of the following is a group?

1- $\{0,1,2,3\}, +_4$

$+_4$	0	1	2	3
0				
1				
2				
3				

2- $\{0,1,2,3\}, *_4$

$*_4$	0	1	2	3
0				
1				
2				
3				

3- $\{1,2,3\}, +_4$

$+_4$	1	2	3
1			
2			
3			

4- $\{1,2,3\}, *_4$

$*_4$	1	2	3
1			
2			
3			

KNOWLEDGEGATE
http://www.knowle

5- $\{0,1,2,3,4\}, +_5$

$+_5$	0	1	2	3	4
0					
1					
2					
3					
4					

6- $\{0,1,2,3,4\}, *_5$

$*_5$	0	1	2	3	4
0					
1					
2					
3					
4					

7- $\{1,2,3,4\}, +_5$

$+_5$	1	2	3	4
1				
2				
3				
4				

8- $\{1,2,3,4\}, *_5$

$*_5$	1	2	3	4
1				
2				
3				
4				

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

9- {0,1,2,3,4,5,6}, $+_7$

$+_7$	0	1	2	3	4	5	6
0							
1							
2							
3							
4							
5							
6							

10- {0,1,2,3,4,5,6}, $*_7$

$*_7$	0	1	2	3	4	5	6
0							
1							
2							
3							
4							
5							
6							

11- {1,2,3,4,5,6}, $+_7$

$+_7$	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						

12- {1,2,3,4,5,6}, $*_7$

$*_7$	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						

KNOWLEDGEGATE
www.knowle

13- $\{1,3,5,7\}, *_8$

$*_8$	1	3	5	7
1				
3				
5				
7				

14- $\{1,2,4,7,8,11,13,14\}, *_{15}$

$*_{15}$	1	2	4	7	8	11	13	14
1								
2								
4								
7								
8								
11								
13								
14								

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

15- {1,2,3, 4……., p-1}, $*_p$

16- {0,1,2,3, 4……., p-1}, $*_p$

17- {1,2,3, 4……., p-1}, $+_p$

18- {0,1,2,3, 4……., p-1}, $+_p$

KNOWLEDGEGATE

Q Let s = set of all integers. A binary operation * is defined by
a * b = a + b + 3
consider the following statements
S_1: (S,*) is a group

S_2: -3 is identity element of (S, *)

S_3: the inverse of -6 is 0

which of the following are true
a) Only S_1 and S_2
b) Only S_2 and S_3
c) Only S_1 and S_3
d) Only S_1 ,S_2 and S_3

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Q $\{0,1,2,3,4,5\}$, $+_6$ is a group which of the following is not true?

a) $1^{-1} = 5$ **b)** $2^{-1} = 4$ **c)** $3^{-1} = 6$ **d)** $0^{-1} = 0$

$+_6$	0	1	2	3	4	5
0						
1						
2						
3						
4						
5						

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q {1,2,3,4,5,6}, $*_7$ is a group which of the following is not true?

a) $1^{-1} = 1$ **b)** $2^{-1} = 4$ **c)** $3^{-1} = 5$ **d)** $6^{-1} = 6$

$*_7$	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q $\{1,3,5,7\}, *_8$ is a group which of the following is not true?

a) $1^{-1} = 1$ **b)** $3^{-1} = 3$ **c)** $5^{-1} = 5$ **d)** $7^{-1} = 7$

$*_8$	1	3	5	7
1				
3				
5				
7				

KNOWLEDGEGATE

Q {1,2,4,7,8,11,13,14}, $*_{15}$ is a group which of the following is not true?

a) $2^{-1} = 8$ **b)** $4^{-1} = 4$ **c)** $7^{-1} = 13$ **d)** $11^{-1} = 14$

$*_{15}$	1	2	4	7	8	11	13	14
1								
2								
4								
7								
8								
11								
13								
14								

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Sub Group

1. The subset of a group may or may not be a group.
2. When the subset of a group is also a group then it is called sub group.
3. The identity element of a group and its sub group is always same.
4. Union of two subgroup may or may not be a subgroup.
5. Intersection of two subgroup is always a subgroup.
6. Lagrange's theorem: - the order of a group is always exactly divisible by the order of a sub group.

KNOWLEDGEGATE

Q Consider a group G = {1,3,5,7}, $*_8$ which of the following sub set of this set does not form is sub group?

a) {0,1}

$*_8$	0	1
0		
1		

b) {1,3}

$*_8$	1	3
1		
3		

c) {1,5}

$*_8$	1	5
1		
5		

d) {1,7}

$*_8$	1	7
1		
7		

e) {1,3,7}

$*_8$	1	3	7
1			
3			
7			

KNOWLEDGEGATE

Q Let G be a group with 15 elements. Let L be a subgroup of G. It is known that L != G and that the size of L is at least 4. The size of L is _____.

(A) 3 **(B)** 5 **(C)** 7 **(D)** 9

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Let G be a finite group on 84 elements. The size of a largest possible proper subgroup of G is _____ .

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Order of an element: - (A, *) be a group, then ∀ a ∈ A, order of a is denoted by O(a).

1. O(a) =n (smallest positive integer), such that $a^n = e$
2. Order of identity element is always one.
3. Order of an element and its inverse is always same.
4. Order of an element in an infinite group does not exist or infinite expect identity.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q consider a group {0,1,2,3}, $+_4$ and find the order of each element?

$+_4$	0	1	2	3
0				
1				
2				
3				

KNOWLEDGEGATE

Q consider a set on cube root of unity $\{1, \omega, \omega^2\}$, * and find the order of each element?

*	1	ω	ω^2
1			
ω			
ω^2			

KNOWLEDGEGATE

Q consider a set on forth root of unity {-1, 1, I, -i}, * and find the order of each element?

*	-1	1	i	-i
-1				
1				
i				
-i				

KNOWLEDGEGATE

Generating element or Generator: - A element ‘a’ is said to be a generating element, if every element of A is an integral power of a, i.e. every element of A can be represented using power of a.

$A = \{a^1, a^2, a^3, a^4, a^5 \ldots.\}$

KNOWLEDGEGATE

Cyclic group: - A group (A, *) is said to be a cyclic group if it contains at least one generator.

1. In a cyclic group if an element is a generator than its inverse will also be a generator.

2. The order of a cyclic group is always the order of the generating element of G.

KNOWLEDGEGATE

Q Show that G = {1,2,4,5,7,8}, $+_{15}$ is a cyclic group. How many generators are there?

$*_{15}$	1	2	4	5	7	8
1						
2						
4						
5						
7						
8						

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q consider a group {1,2,4,7,8,11,13,14}, $*_{15}$ and find the order of each element?

$*_{15}$	1	2	4	7	8	11	13	14
1								
2								
4								
7								
8								
11								
13								
14								

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Q For the composition table of a cyclic group shown below

*	a	b	c	d
a	a	b	c	d
b	b	a	d	c
c	c	d	b	a
d	d	c	a	b

Which one of the following choices is correct?

(A) a, b are generators

(B) b, c are generators

(C) c, d are generators

(D) d, a are generators

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Number of generators

Lagrange's theorem: - let A be a cyclic group of order n, number of Generator in A is denoted by $\phi(n) = \{n(p_1-1)\ (p_2-1)\ (p_3-1)\ \ldots\ldots\ (p_k-1)\} / (p_1p_2p_3\ldots\ldots p_k)$

KNOWLEDGEGATE

Q let G be a cyclic group, O(G) = 8, number of generators in G =?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q let G be a cyclic group, O(G) = 12, number of generators in G =?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q let G be a cyclic group, O(G) = 70, number of generators in G =?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q let G be a cyclic group, O(G) = 23100, number of generators in G =?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Coset

Let H be a subgroup of a group G and $a \in G$, then the set
$aH = \{ah \mid h \in H\}$ is called a left coset of H in G
and $Ha = \{ha \mid h \in H\}$ is called a right coset of H in G.

By this definition, it is clear that corresponding to every element of G, we have a left coset and right coset of H in G. It is obvious that

$$aH \subset G,\ Ha \subset G,\ \forall\ a \in G$$

Further we may note that

$$eH = H = He$$

i.e., the left and the right cosets of H corresponding to the identity e coincide with H. Hence H itself is a left as well as a right coset of H in G.

[KNOWLEDGEGATE]

Let G= (Z, +) and H = 2Z = {0, ± 2, ± 4,....}
then the right cosets of H in G are :
H+ 0={0, ±2, ±4, ...} = H
H + 1={0 + 1, ± 2+1, ± 4+1, ...}
H + 2={0 + 2, ± 2+2, ± 4+2, ...} etc.
It can be easily observed that any right coset and its corresponding left coset are equal i.e.,
H+ 1=1 + H, H + 2=2 + H etc.
Again
H + 3 ={.., - 6, -3, 0, 3, 6, 9, 12,....} = H
H + 4 = {..., - 5, - 2, 1, 4, 7, 10, 13,..} = H + 1 etc.
Similarly H + 5 = H + 2, H + 6 = H
Thus H has only two distinct cosets H, H + 1 in Z.

KNOWLEDGEGATE

Find all the cosets of 3Z in the group (Z, +).

H = 3Z = {.....6, -3, 0, 3, 6, ...}

The following distinct cosets of H in G are obtained :

0 + H = H + 0 = {..., -6, -3, 0, 3, 6, ...} = H

1 + H = H + 1 = {.., - 5, -2, 1, 4, 7, ...}

2 + H = H + 2 = {.....-4, - 1, 2, 5, 8,..}

3 + H = H + 3 = 0 + H = H + 0

4 + H = H + 4 = 1 + H = H + 1

5 + H = H + 5 = 2 + H = H + 2

6 + H = H + 6 = 0 + H = H + 0

three cosets of H is G are H, H+1, H+2

so n cosets of nZ in Z

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Find all the cosets of H = {0, 3} in the group $(Z_6, +_6)$.

G = { 0, 1, 2, 3, 4, 5} H = {0, 3}, G

The following distinct cosets of H in G are obtained :

$0 + H = H + 0 = \{0, 3\} = H$

$1 + H = H + 1 = \{0 +_6 1, 3 +_6 1\} = \{1, 4\} = H + 1$

$2 + H = H + 2 = \{0 +_6 2, 3 +_6 2\} = \{2, 5\} = H + 2$

$3 + H = H + 3 = \{0 +_6 3, 3 +_6 3\} = \{3, 0\} = H$

$4 + H = H + 4 = \{0 +_6 4, 3 +_6 4\} = \{4, 1\} = H + 1$

$5 + H = H + 5 = \{0 +_6 5, 3 +_6 5\} = \{5, 2\} = H + 2$

$6 + H = H + 6 = \{0 +_6 6, 3 +_6 6\} = \{0, 3\} = H$

If we take union of all distinct cosets of G, we will get back G

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Ring

- A ring is an algebraic system (R, +, •) where R is a non-empty set and + and • are two binary operations (which can be different from addition and multiplication) and if the following conditions are satisfied :
  - (R, +) is an abelian group.
  - (R, •) is semigroup
- The operation • is distributive over +.
  - a • (b + c) = (a • b) + (a • c)
  - (a + b) • c = (a • c) + (b • c)

- E.g. (Z, +, X) is a ring

KNOWLEDGEGATE

Integral Domain

- A ring becomes integral domain, if it is a commutative ring with unity and without zero divisor
  - (D, +) is an abelian group.
  - (D, •) is semigroup with Commutative and With unity, Without zero divisor if a.b=0 then a=0 or b=0

- E.g. (Z, +, X) is a integral domain, (Q, +, X) is a integral domain, (R, +, X) is a integral domain, (C, +, X) is a integral domain, $(Z, +_3, X_3)$ is a integral domain

KNOWLEDGEGATE

Field

- A field is an algebraic system (R, +, •) where R is a non-empty set and + and • are two binary operations (which can be different from addition and multiplication) and if the following conditions are satisfied :
  - (R, +) is an abelian group
  - (R, •) is an abelian group(inverse should exist for every non-zero element)
  - The operation • is distributive over +.
  - a • (b + c) = (a • b) + (a • c)
  - (a + b) • c = (a • c) + (b • c)

- E.g. (R, +, X) is a field, (Q, +, X) is a field, ($Z, +_3, X_3$) is a field

KNOWLEDGEGATE

$\{0,1,2,3,4\}, +_5$

$+_5$	0	1	2	3	4
0					
1					
2					
3					
4					

$\{0,1,2,3,4\}, *_5$

$*_5$	0	1	2	3	4
0					
1					
2					
3					
4					

KNOWLEDGEGATE

This is ring, integral domain and field

http://www.knowledgegate.in/gate

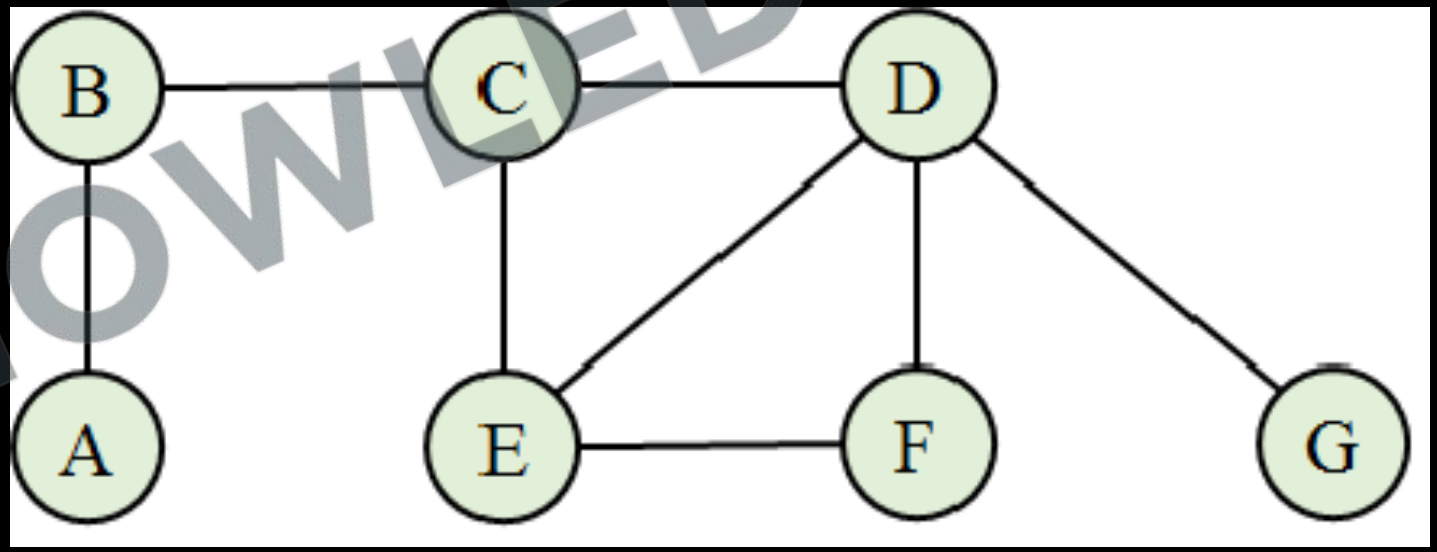
Chapter-7 (Graphs)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

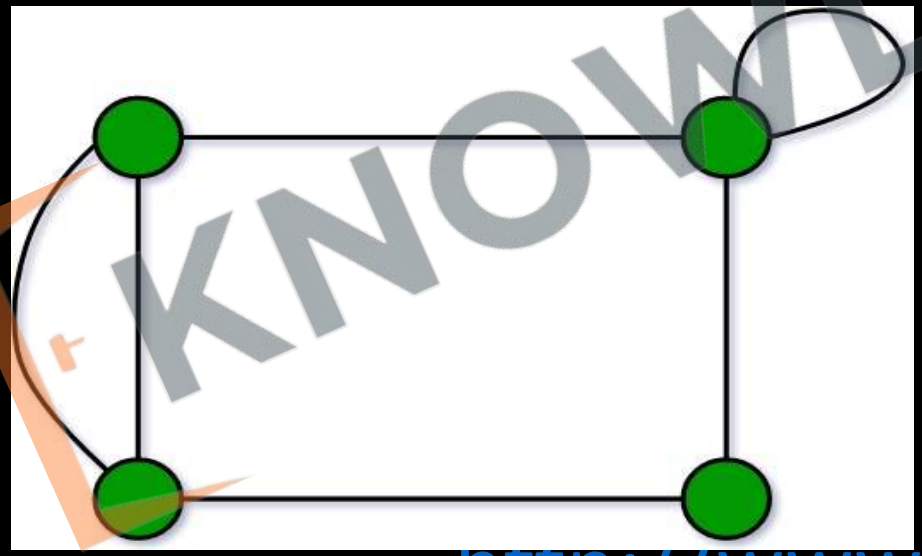
Graph Theory

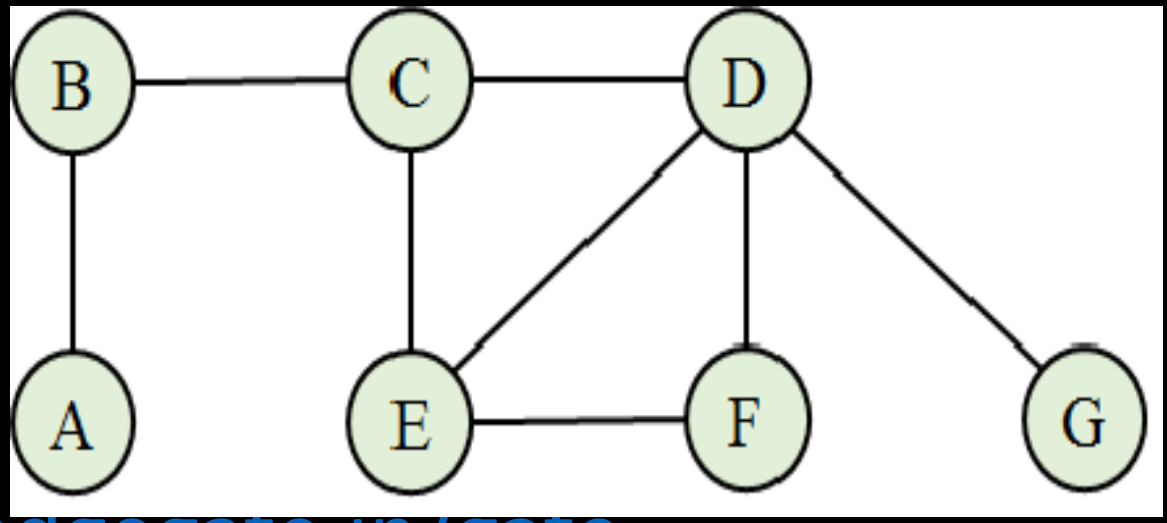
1. A graph G (V, E) consist of a set off objects V = {V_1, V_2, V_3….,V_N} called vertices and another set E = {E_1, E_2, E_3,….,E_n} whose elements are called edges.
2. Each edge e_k is identified with an unordered pair (v_i, v_j) of vertices.
3. The vertices v_i, v_j associated with edge e_k are called the end vertices of e_k.



KNOWLEDGE GATE

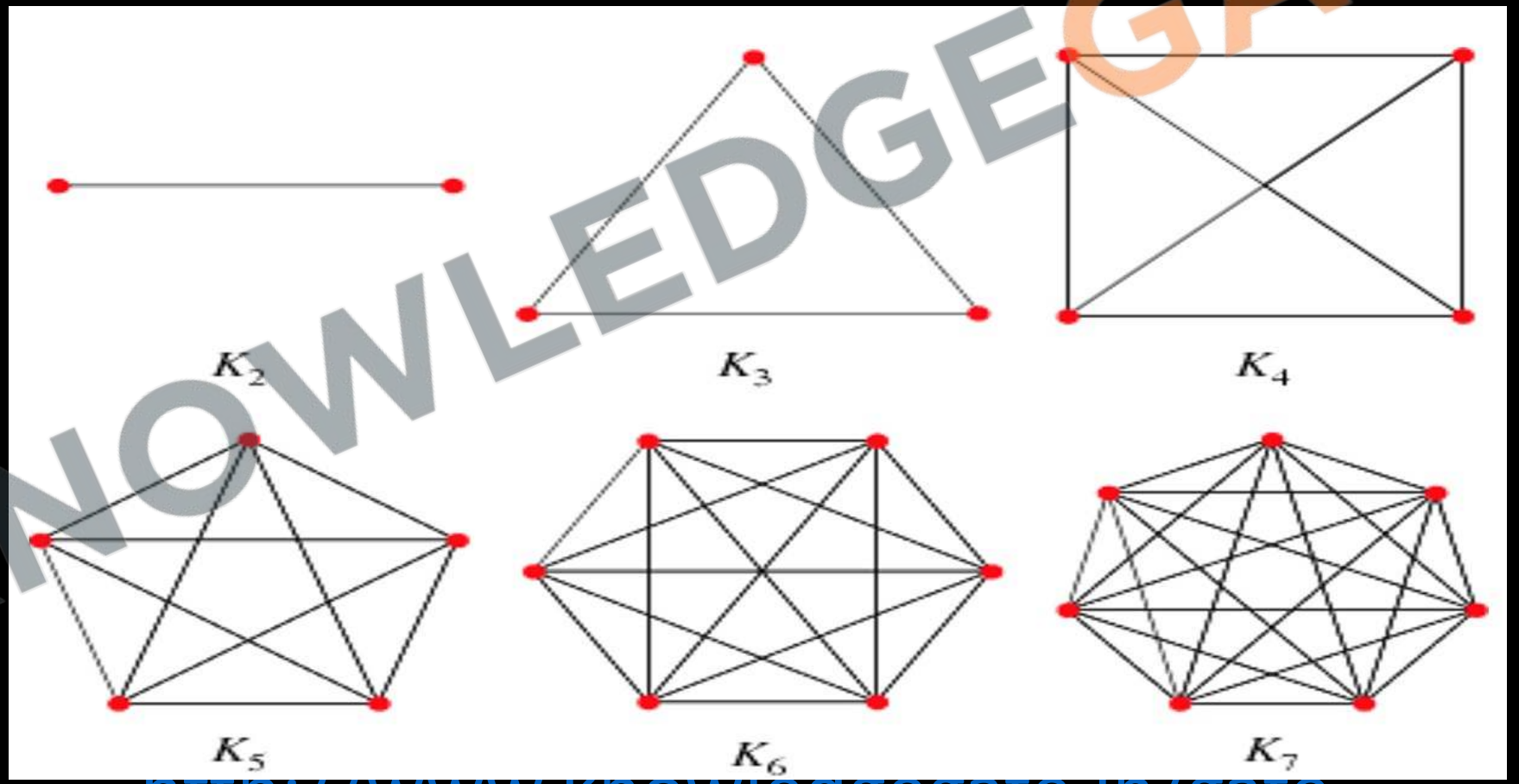
1. **Self-Loop:** Edge having the same vertex (v_i, v_i) as both its end vertices is called self-loop.
2. **Parallel Edge:** When more than one edge associated with a given pair of vertices such edges are referred as parallel edges.
3. **Adjacent Vertices:** If two vertices are joined by the same edges, they are called adjacent vertices.
4. **Adjacent Edges:** If two edges are incident on some vertex, they are called adjacent edges.





KNOWLEDGEGATE

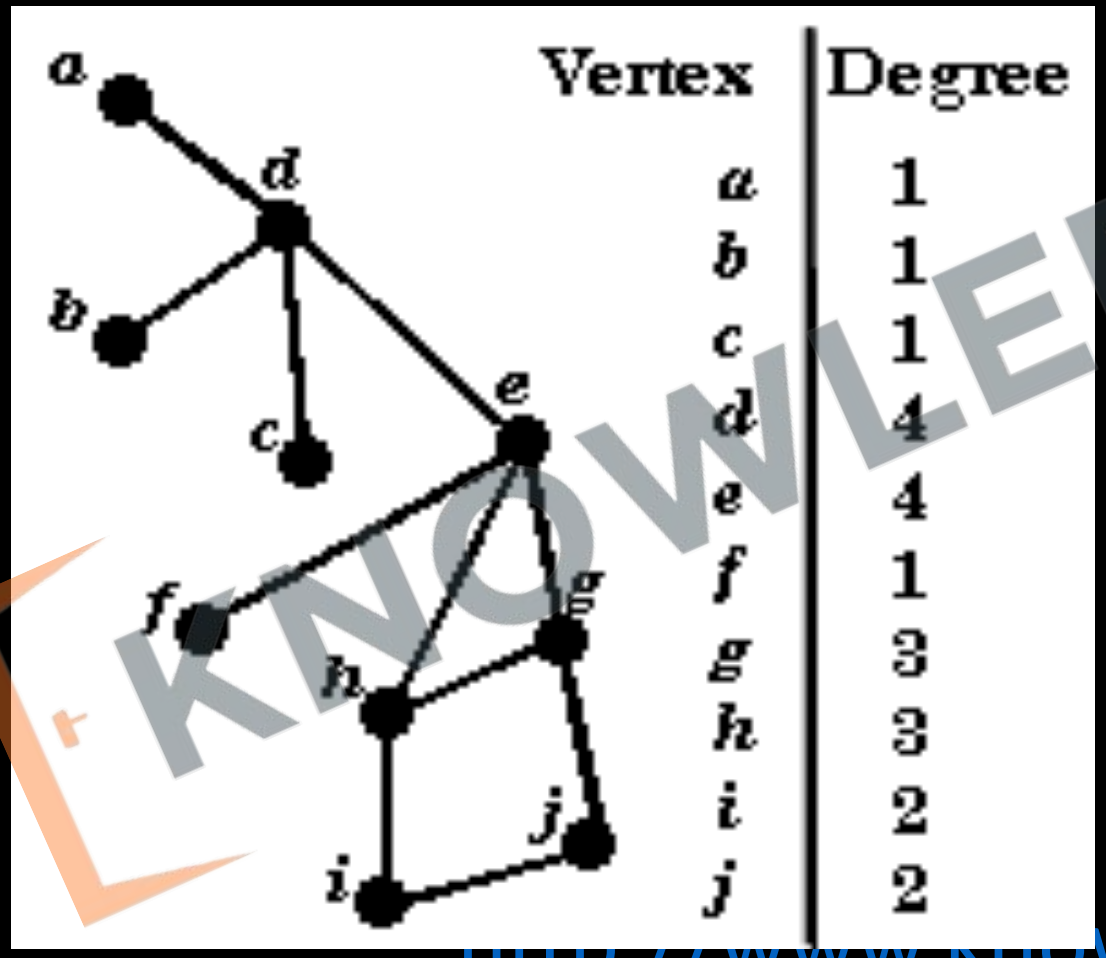
- **Complete or Full Graph:** In a simple graph there exist an edge between each and every pair of vertices i.e. every vertex are adjacent to each other, then the graph is said to be a complete graph, denoted by K_n. Number of edges in a Complete graph is n(n-1)/2



KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Degree

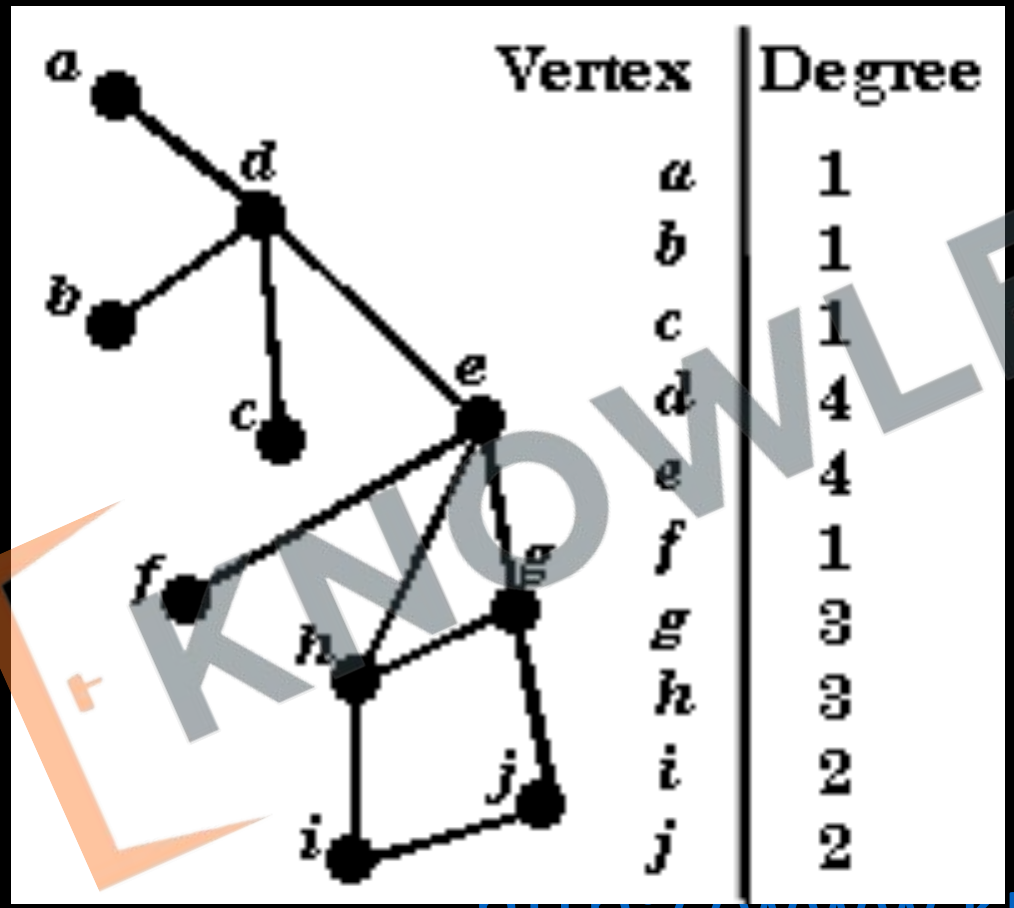
- **Degree of a Vertex:** The degree of a vertex in an undirected graph is the number of edges associated with it, denoted by $deg(v_i)$.



Vertex	Degree
a	1
b	1
c	1
d	4
e	4
f	1
g	3
h	3
i	2
j	2

KNOWLEDGEGATE

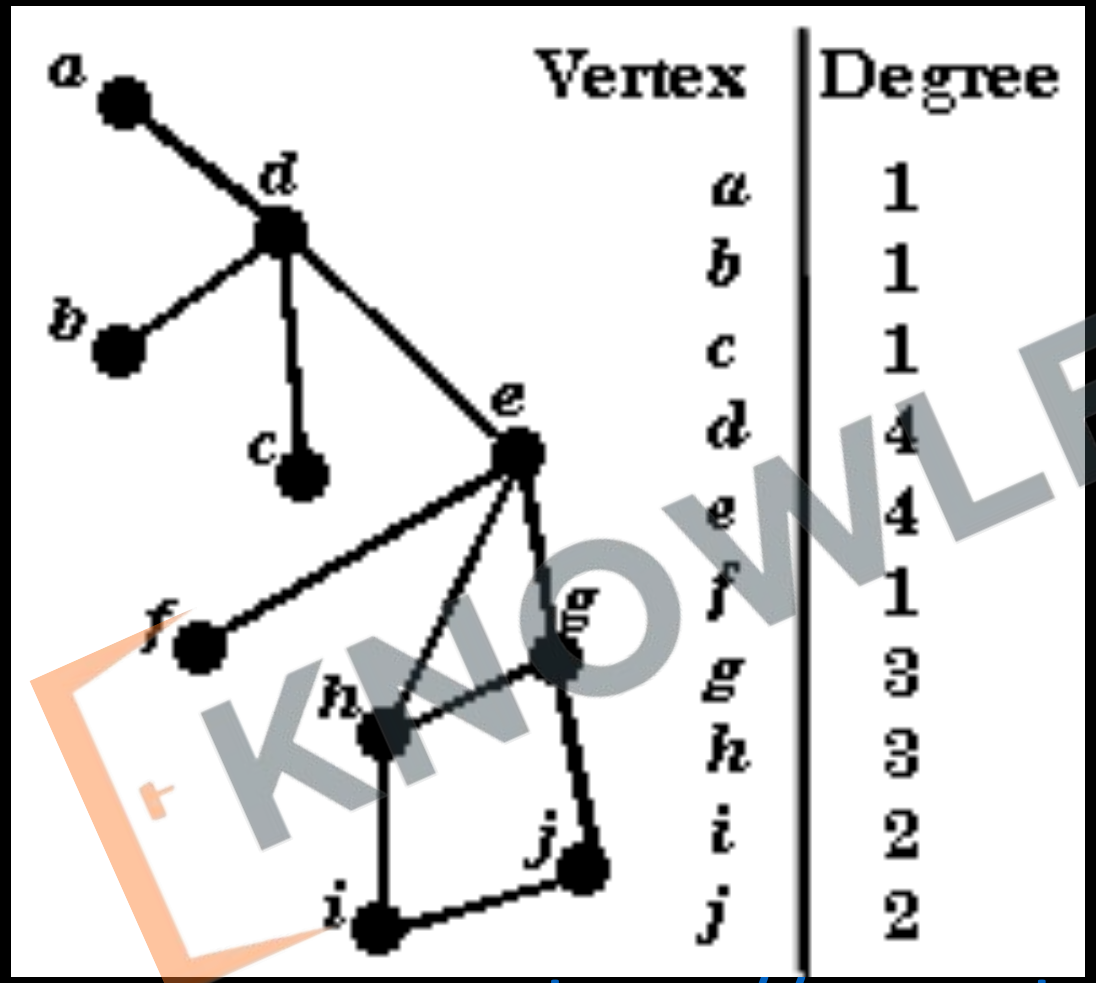
- **Hand-shaking theorem:** - Since each edge contribute two degree in the graph, the sum of the degree of all vertices in G is twice the number of edges in g.
  - $\sum_{i=1}^{n} d(vi) = 2|E|$



Vertex	Degree
a	1
b	1
c	1
d	4
e	4
f	1
g	3
h	3
i	2
j	2

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- The number of vertices of odd degree in a graph is always even.
  - $\sum_{i=1}^{n} d(vi) = \sum_{even} d(vi) + \sum_{odd} d(vi)$



Vertex	Degree
a	1
b	1
c	1
d	4
e	4
f	1
g	3
h	3
i	2
j	2

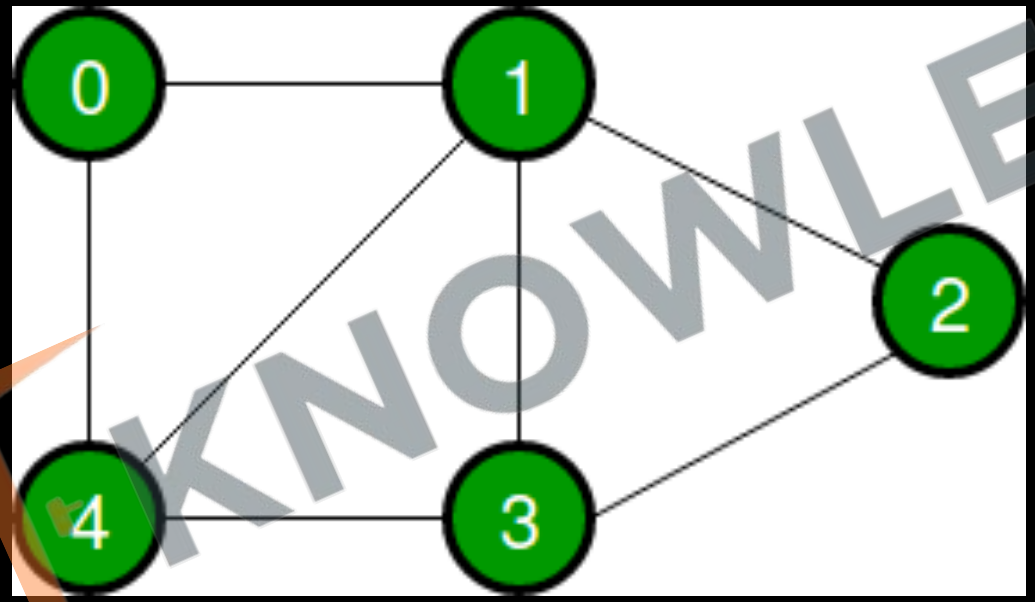
KNOWLEDGEGATE

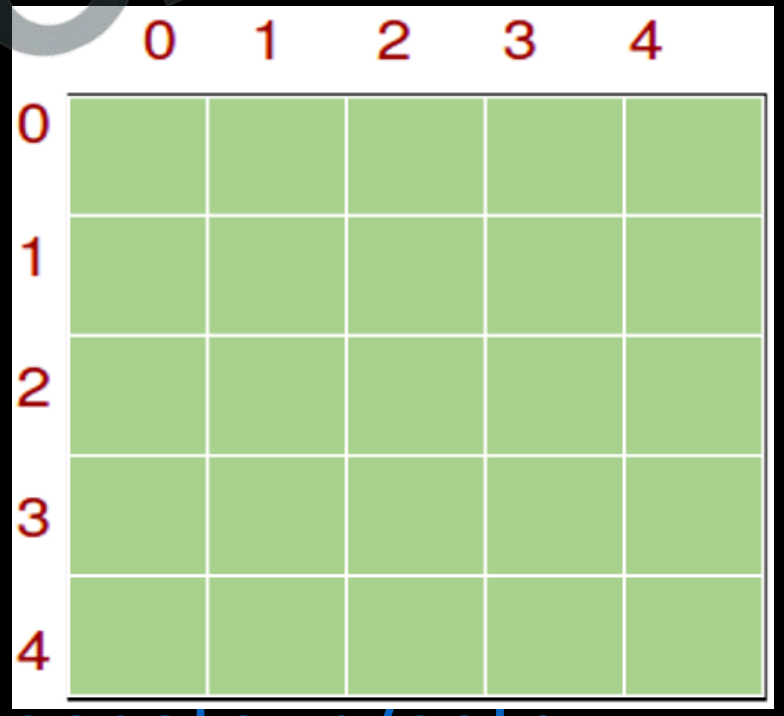
Representation of Graph in Memory

- Following two are the most commonly used representations of a graph.
  - Adjacency Matrix
  - Adjacency List
- There are other representations also like, Incidence Matrix and Incidence List. The choice of the graph representation is situation specific. It totally depends on the type of operations to be performed and ease of use.

KNOWLEDGEGATE

- **Adjacency Matrix:** Adjacency Matrix is a 2D array of size V x V where V is the number of vertices in a graph. Let the 2D array be adj[][], a slot adj[i][j] = 1 indicates that there is an edge from vertex i to vertex j.
- Adjacency matrix for undirected graph is always symmetric.
- Adjacency Matrix is also used to represent weighted graphs. If adj[i][j] = w, then there is an edge from vertex i to vertex j with weight w.





KNOWLEDGEGATE

Incidence Matrix

- **Representation of undirected graph** : Consider a undirected graph G = (V, E) which has n vertices and m edges all labelled. The incidence matrix I(G) = [bij], is then n x m matrix,
  - where bi,j=1 when edge ej is incident with vi
  - = 0 otherwise

- **Representation of directed graph** : The incidence matrix I(D) = [bij] of digraph D with n vertices and m edges is the n x m matrix in which.
  - Bi,j = 1 if arc j is directed away from vertex vi
  - = −1 if arc j is directed towards vertex vi
  - =0 otherwise.

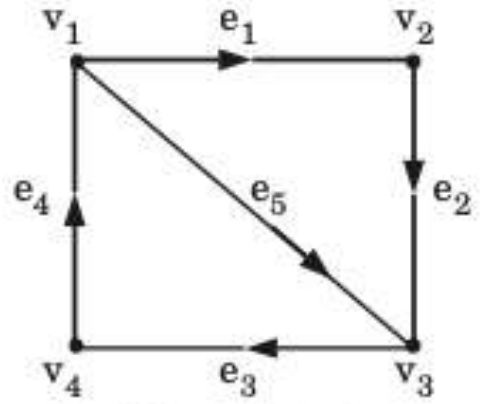


Fig. 4.3.3.

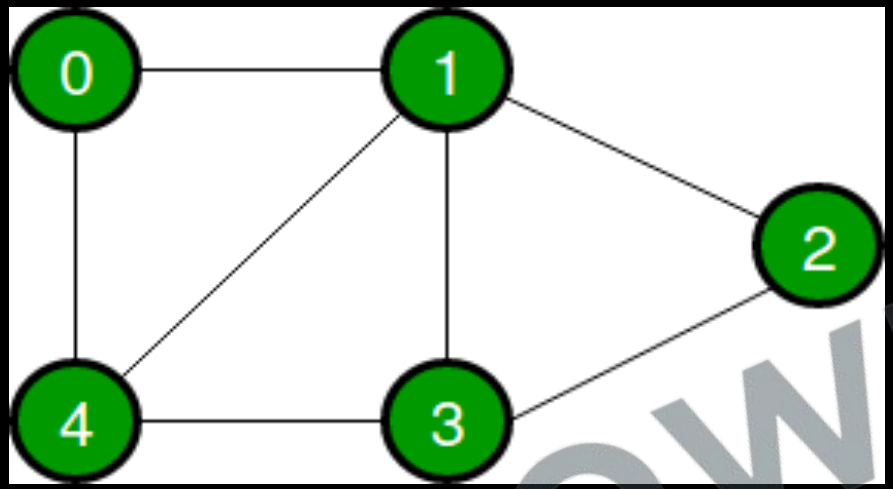
The incidence matrix of the digraph of Fig. 4.3.3 is

$$I(D) = \begin{bmatrix} 1 & 0 & 0 & -1 & 1 \\ -1 & 1 & 0 & 0 & 0 \\ 0 & -1 & 1 & 0 & -1 \\ 0 & 0 & -1 & 1 & 0 \end{bmatrix}$$

KNOWLEDGEGATE

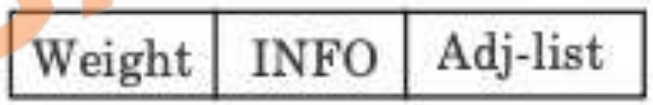
http://www.knowled

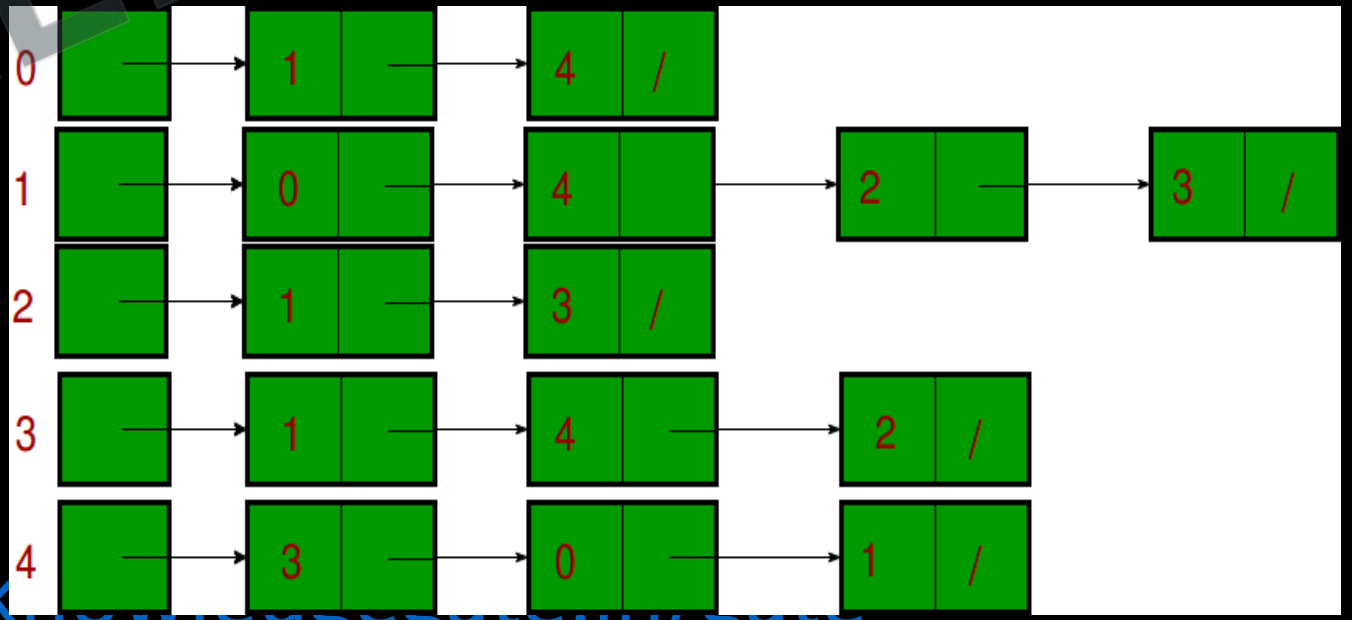
- **Adjacency List:** An array of lists is used. Size of the array is equal to the number of vertices. Let the array be array[]. An entry array[i] represents the list of vertices adjacent to the *i*th vertex. This representation can also be used to represent a weighted graph. The weights of edges can be represented as lists of pairs.



a. For non-weighted graph :

b. For weighted graph :

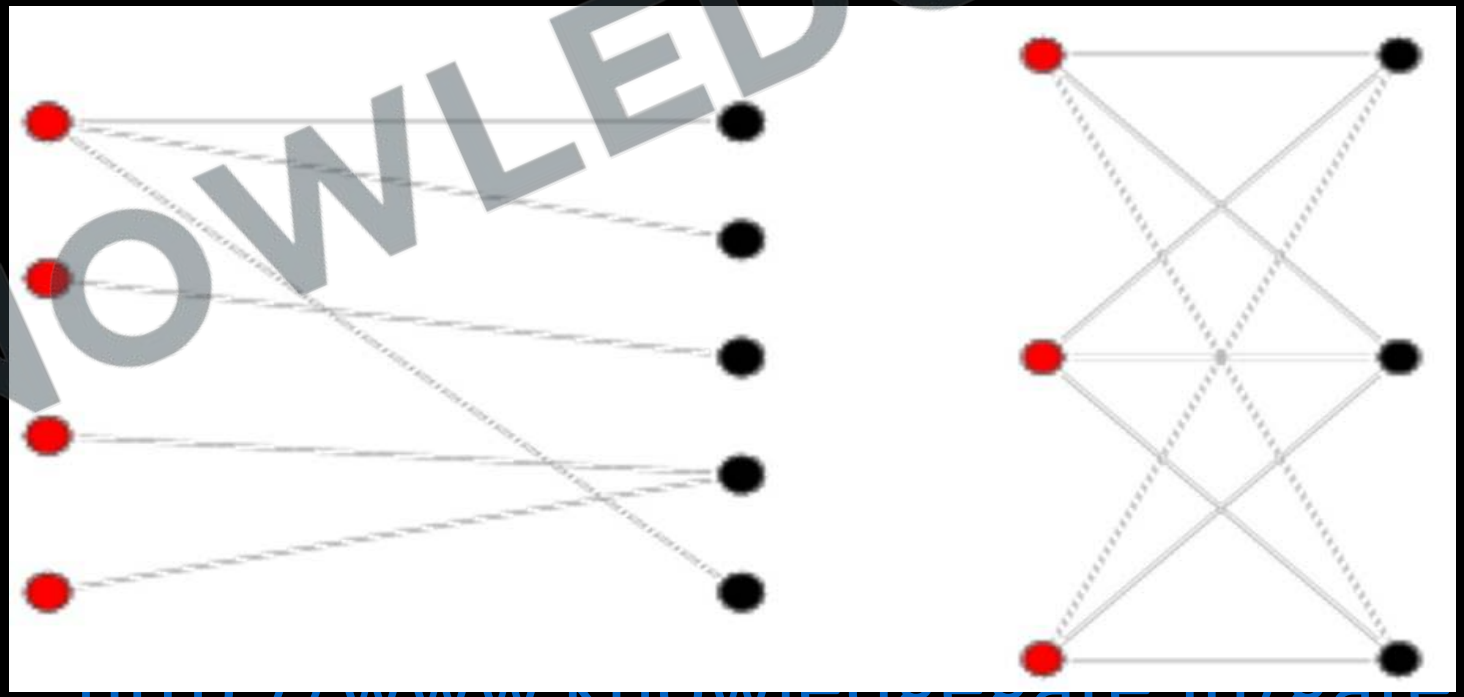




KNOWLEDGEGATE
http://www.knowledgegate.in/gate

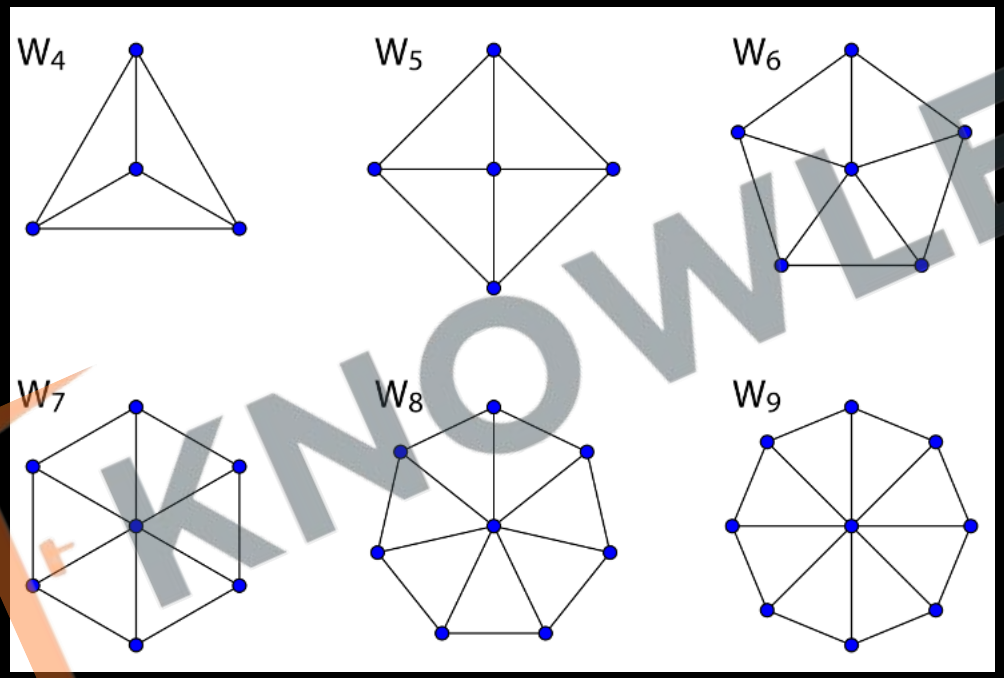
Some Popular Graph

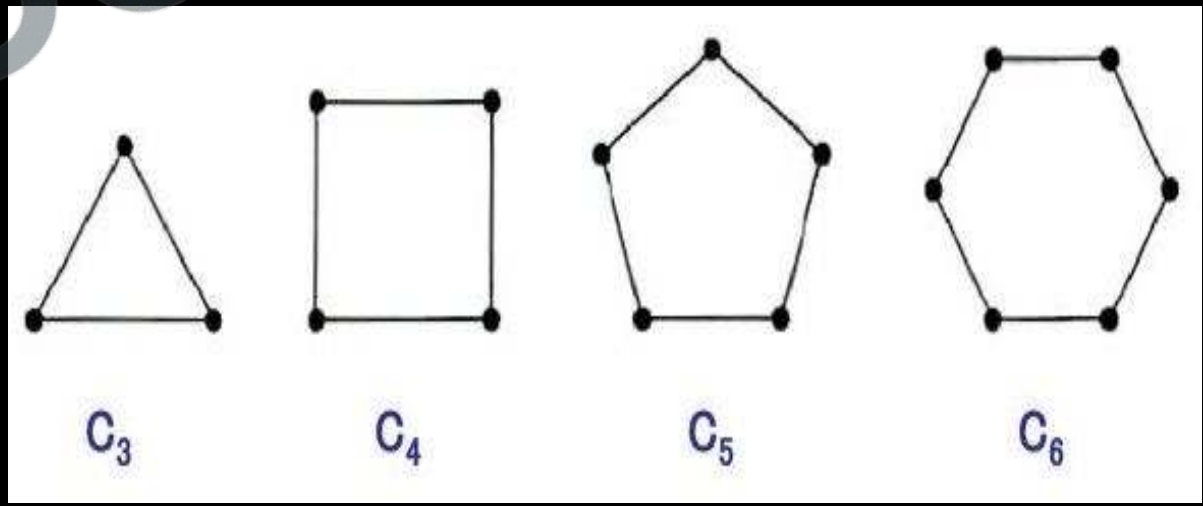
1. **Bi-partite graph:** - A graph G(V, E) is called bi-partite graph if it's vertex set V(G) can be partitioned into two non-empty disjoint subset $V_1(G)$ and $V_2(G)$ in such a way that each edge e ϵ E(G) has it's one end point in $V_1(g)$ and other end point in $V_2(g)$. The partition $V = V_1 \cup V_2$ is called bipartition of G.
2. **Complete Bi-partite graph:** - A Bi-partite graph G(V, E) is called Complete bi-partite graph if every vertex in the first partition is connected to every vertex in the second partition, denoted by $K_{m,n}$.



KNOWLEDGEGATE
http://www.knowledgegate.in/gate

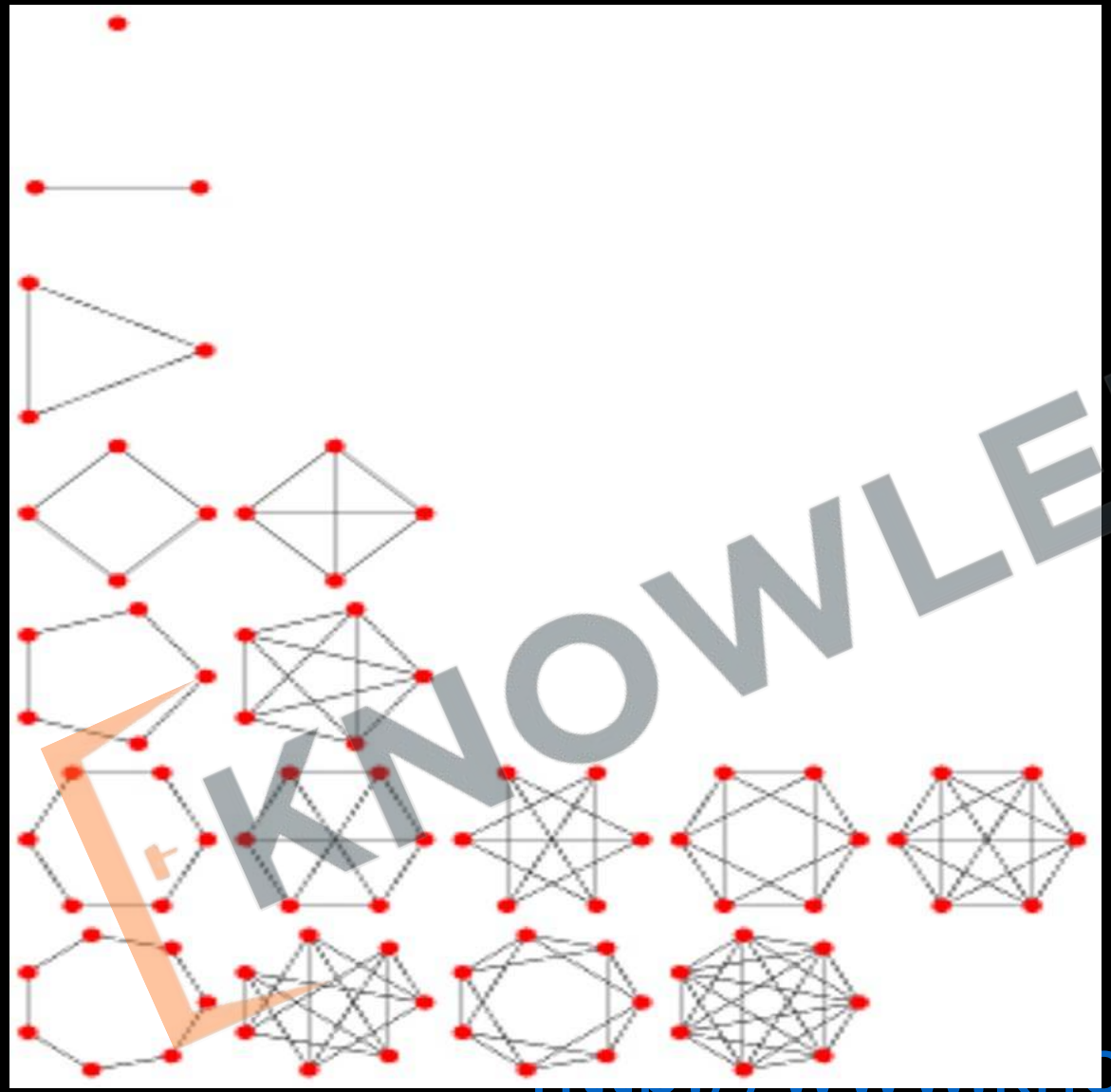
1. **Cycle Graph:** - A cycle graph or circular graph is a graph that consists of a single cycle, or in other words, some number of vertices (at least 3) connected in a closed chain. The cycle graph with n vertices is called C_n. The number of vertices in C_n equals the number of edges, and every vertex has degree 2; that is, every vertex has exactly two edges incident with it.
2. **Wheel graph:** - A wheel graph is a graph formed by connecting a single universal vertex to all vertices of a cycle. Some authors write W_n to denote a wheel graph with n vertices ($n \geq 4$); other authors instead use W_n to denote a wheel graph with n+1 vertices ($n \geq 3$), which is formed by connecting a single vertex to all vertices of a cycle of length n.





KNOWLEDGEGATE

- **Regular graph:** - A graph in which all the vertices are of equal degree is called a regular graph. E.g. 2-regular graph, 3-regular graph.

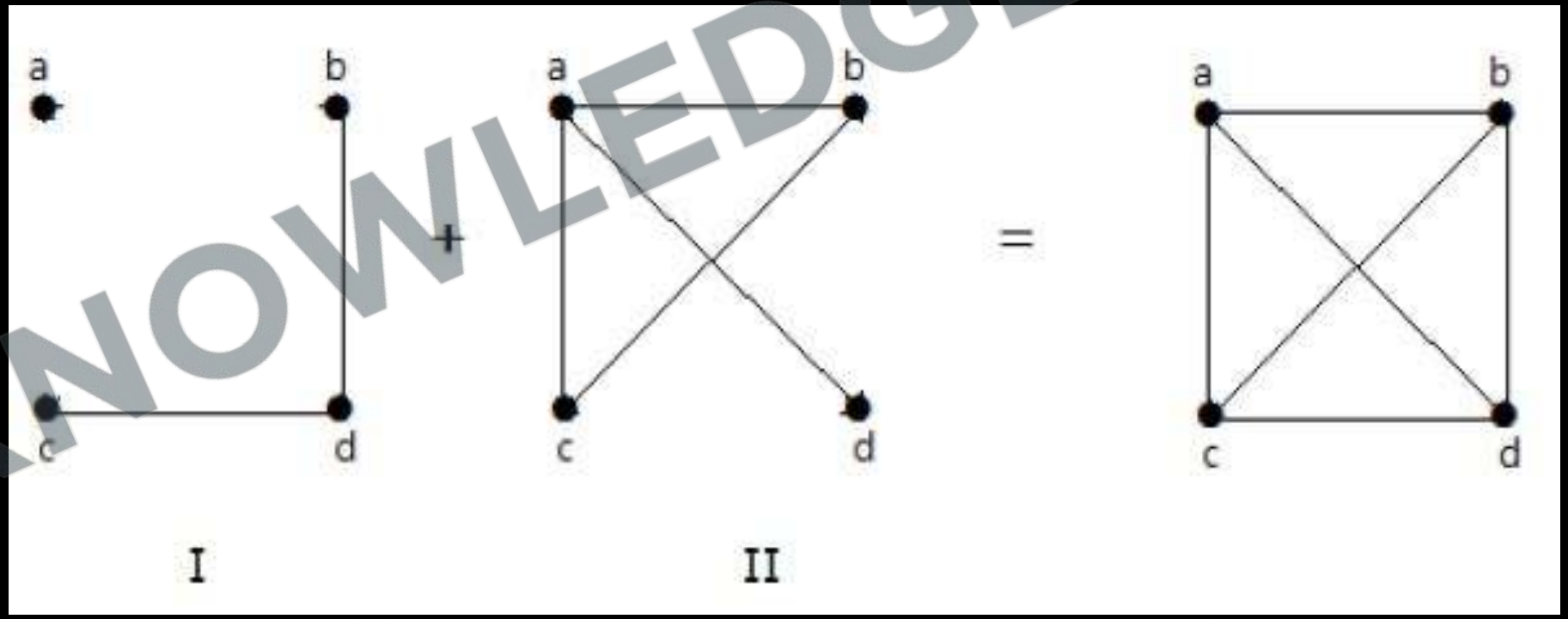


KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Complement of a Graph

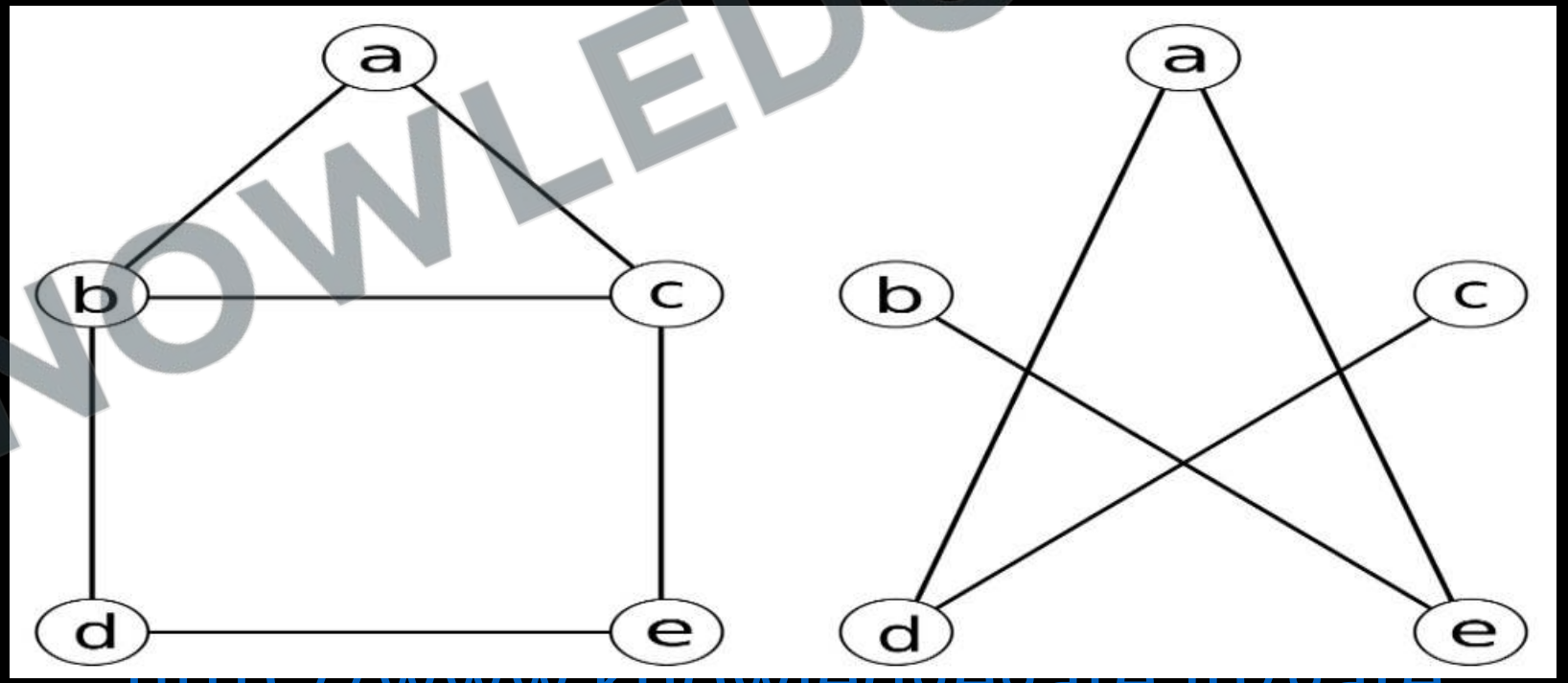
1. The complement of a simple graph G (V, E) is a graph G^c (V, E^c) on the same vertices set as of G, such that there will be an edge between two vertices u, v in G^c if ang only if there is no edge between u, v in G. i.e. two vertices of Gc are adjacent iff they are not adjacent in G.
2. $V(G) = V(G^c)$
3. $E(G^c) = \{(u, v) \mid (u, v) \notin E(G)\}$
4. $E(G^c) = E(K_n) - E(G)$



KNOWLEDGEGATE

Properties

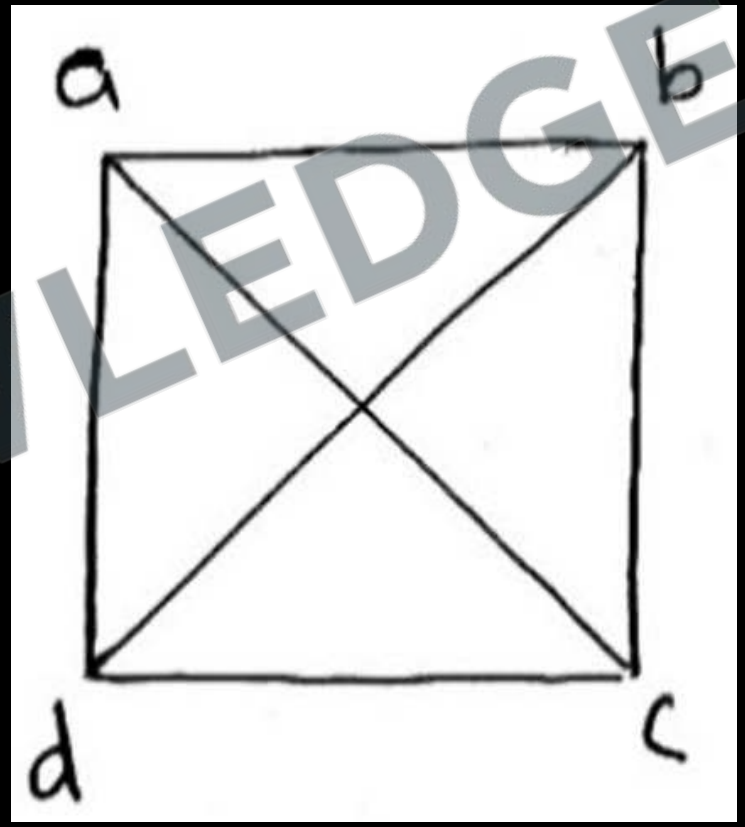
1. $G \cup G^c = K_n$
2. $G \cap G^c$ = null graph
3. $|E(G)| + |E(G^c)| = E(K_n) = n(n-1)/2$



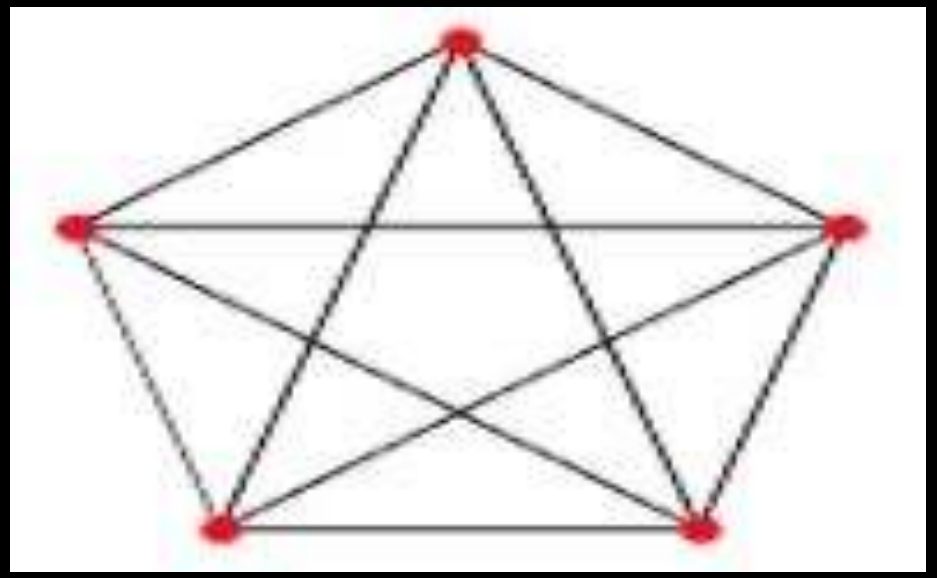
KNOWLEDGEGATE
http://www.knowledgegate.in/gate

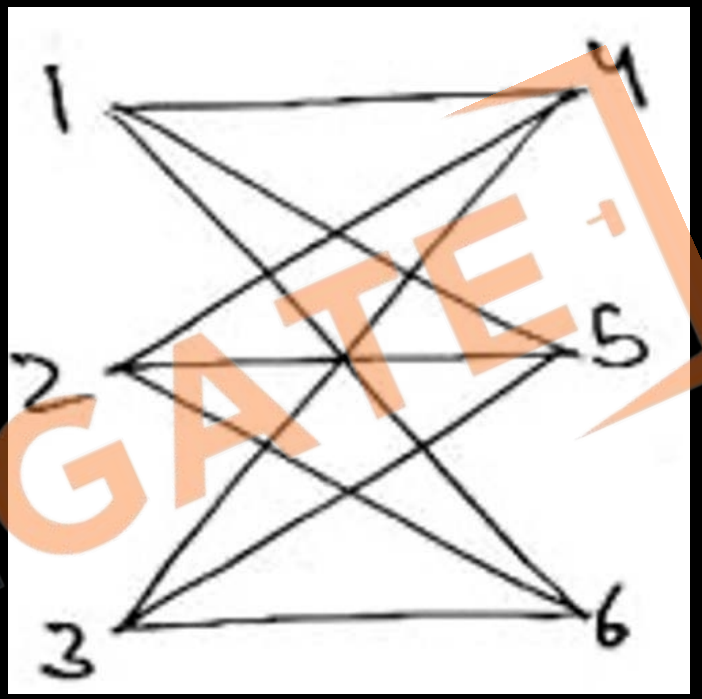
Planer Graph

Planer Graph: - A graph is called a planer graph if it can be drawn on a plan in such a way that no edges cross each other, otherwise it is called non-planer. Application: civil engineering, circuit designing



KNOWLEDGEGATE





KNOWLEDGEGATE

Simplest Non-Planer Graphs

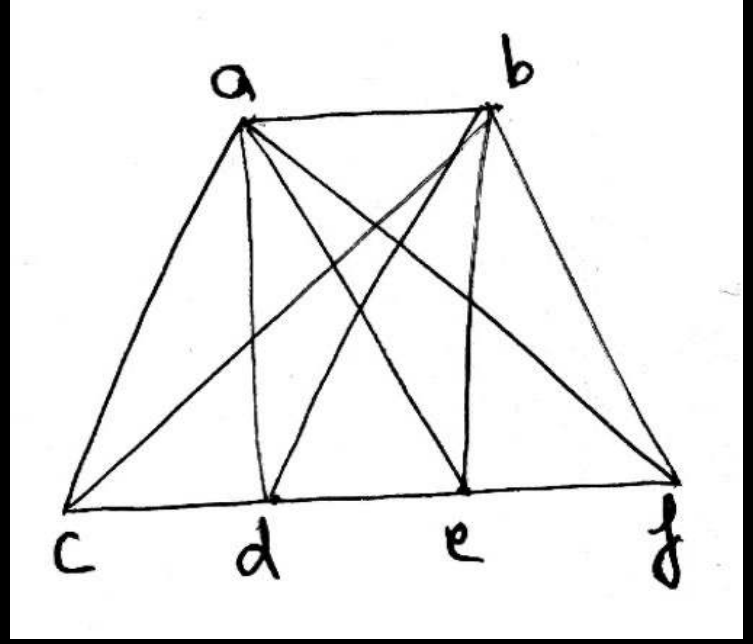
1. **Kuratowski's case I:** - K_5
2. **Kuratowski's case II:** - $K_{3,3}$
3. Both are simplest non-planer graph
4. Both are regular graph
5. If we delete either an edge or a vertex from any of the graph, they will become planer

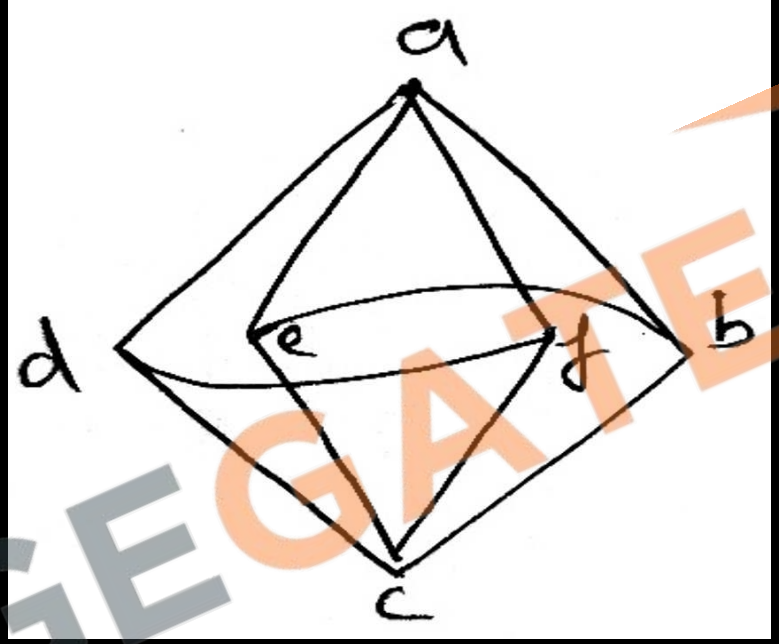
KNOWLEDGEGATE

- **Kazimierz Kuratowski** (2 February 1896 – 18 June 1980) was a Polish mathematician and logician.

KNOWLEDGEGATE

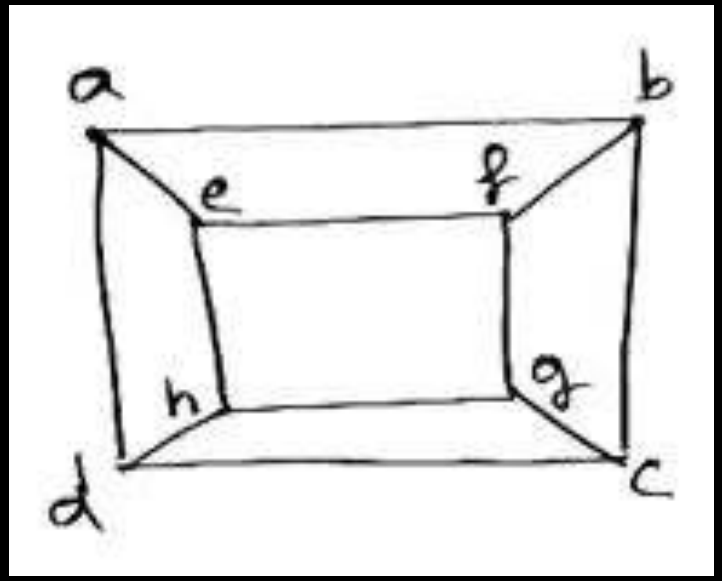
http://www.knowledgegate.in/gate

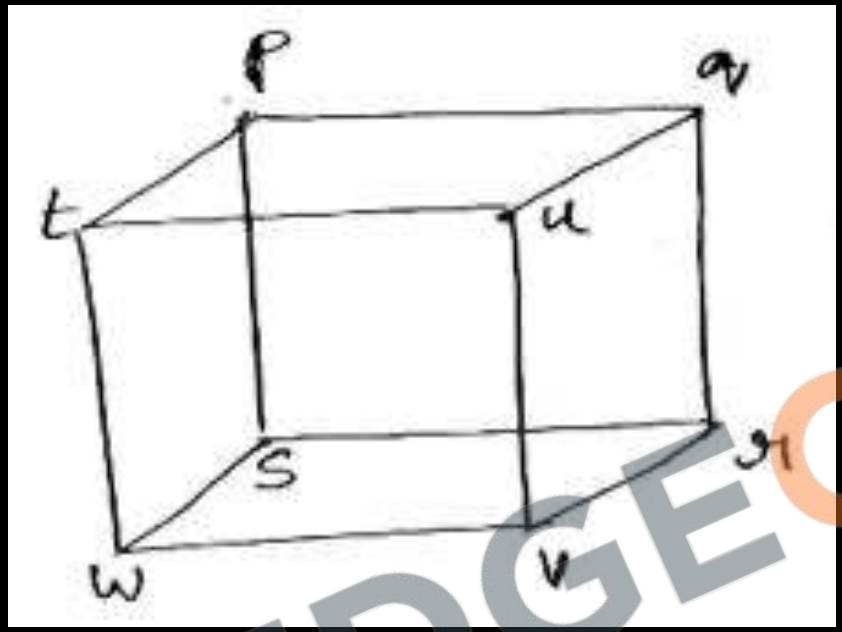


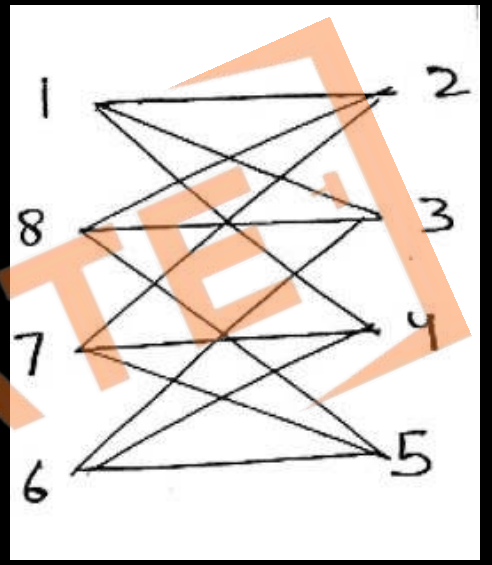


KNOWLEDGEGATE

http://www.knowledgegate.in/gate

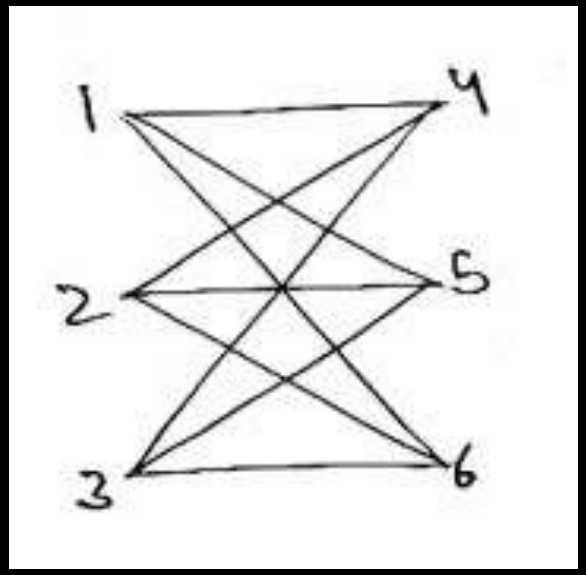


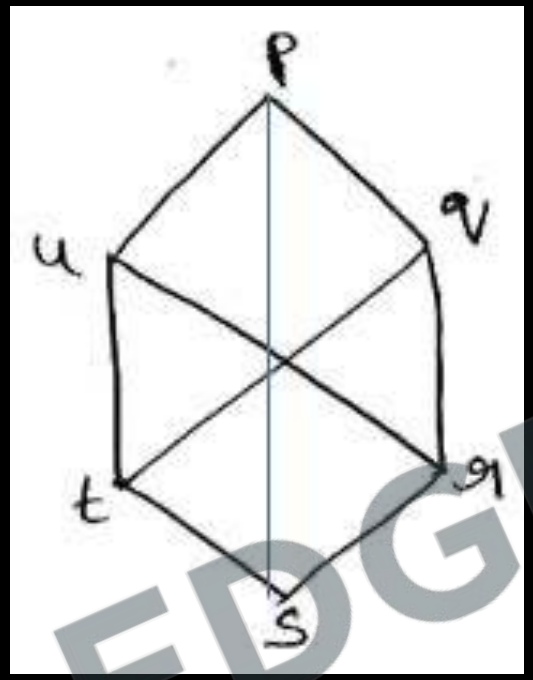


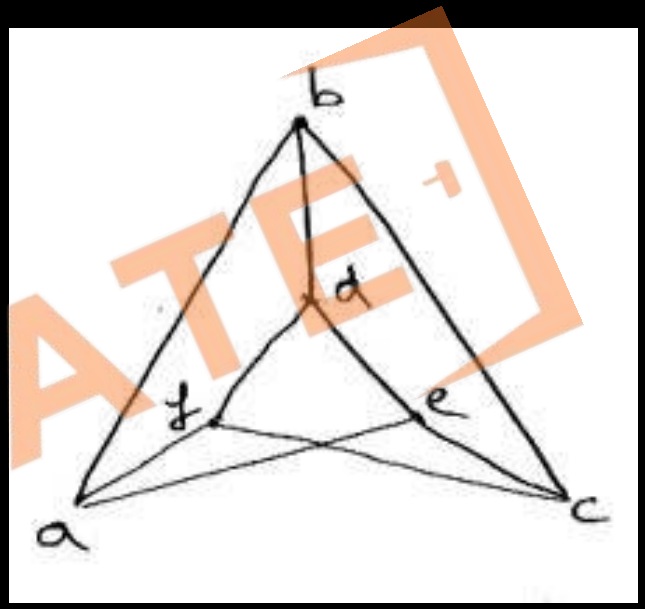


KNOWLEDGEGATE

http://www.knowledgegate.in/gate





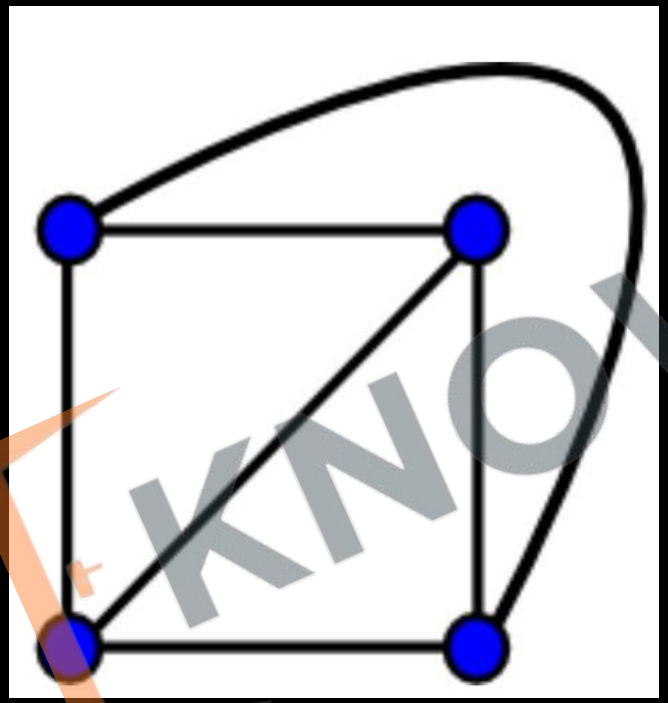


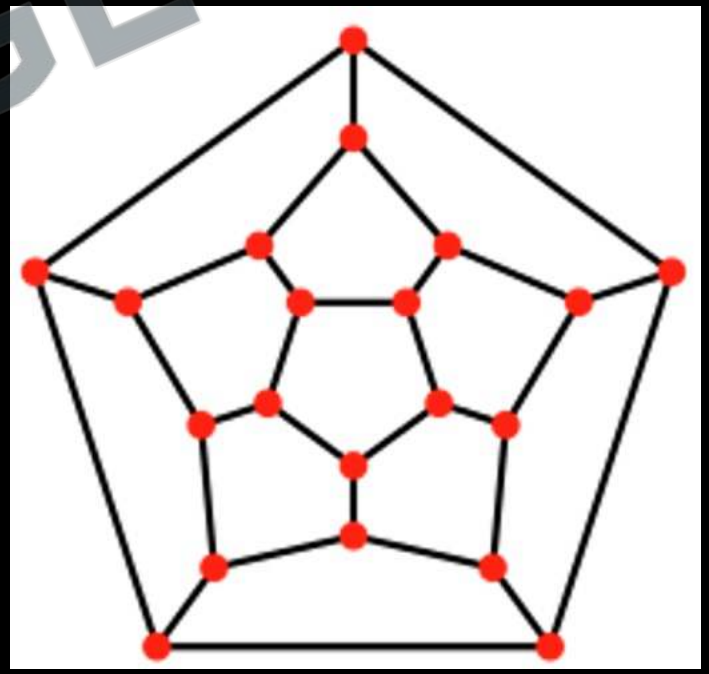
KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Euler's formula

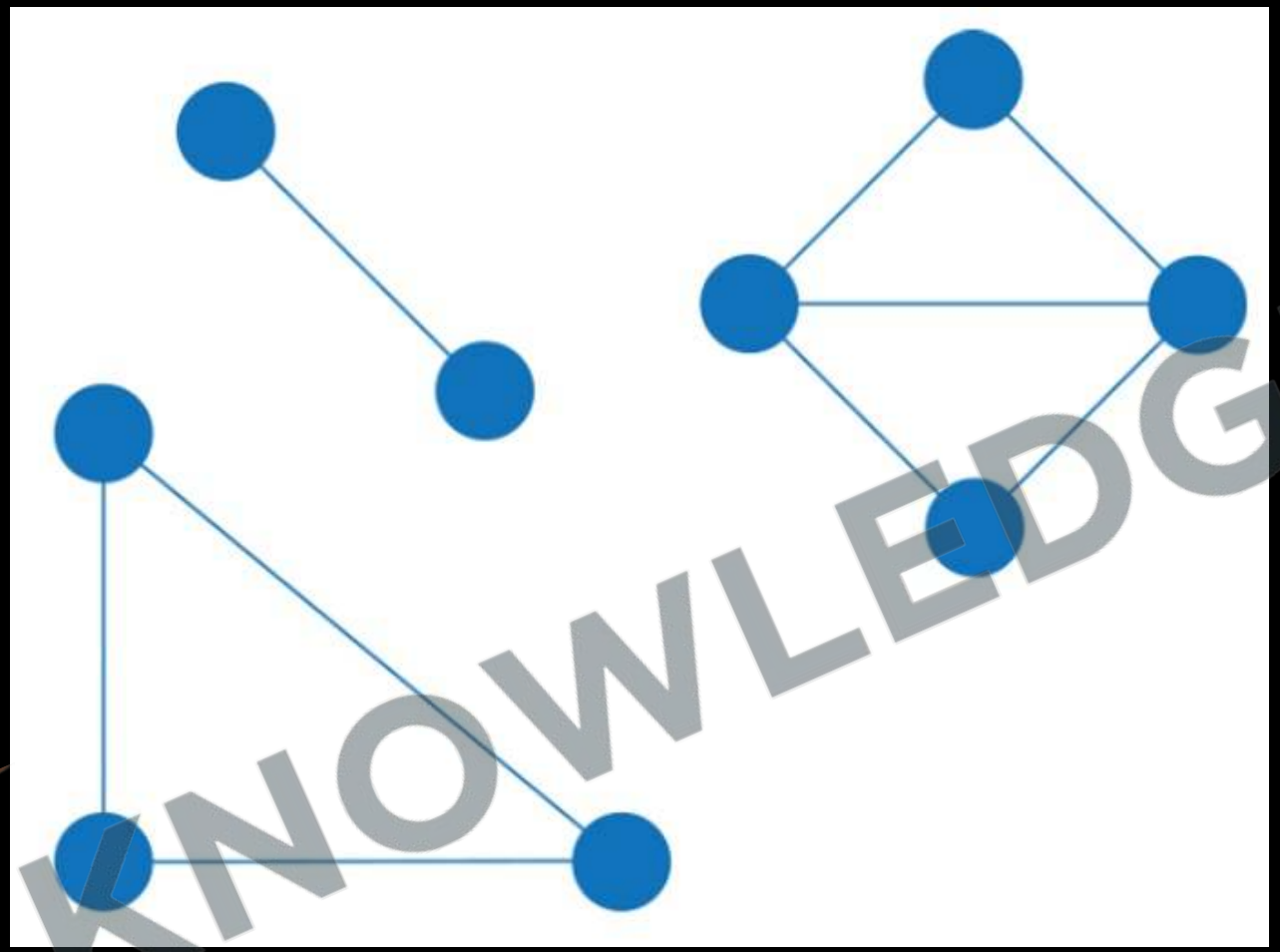
- A planer graph divides the plane into number or regions (faces, planer embedding), which are further divided into bounded(internal) and unbounded region(external). **Euler's formula** states that if a finite, connected, planar graph with *v* is the number of vertices, *e* is the number of edges and *r* is the number of faces (regions bounded by edges, including the outer, infinitely large region), then r = e – v + 2





KNOWLEDGEGATE

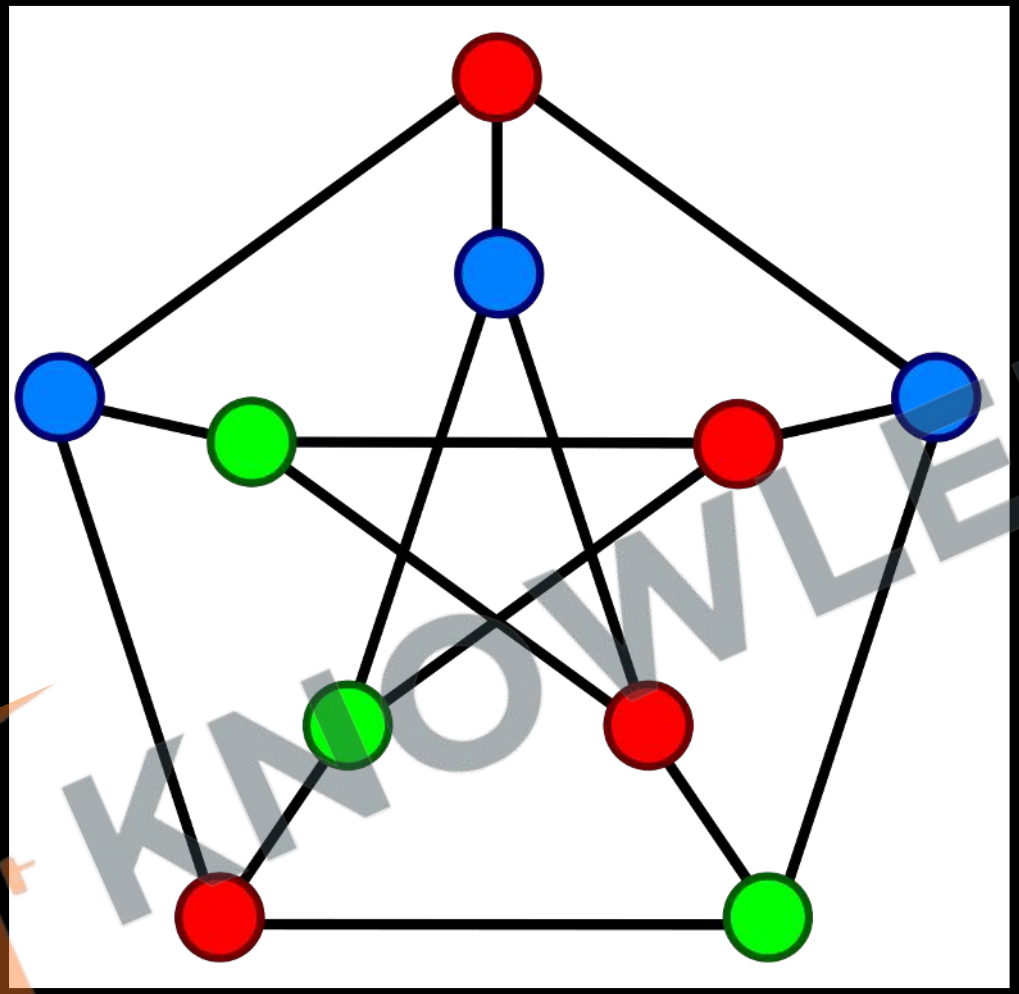
- Euler's formula (Disconnected graph): V – e + r – k = 1

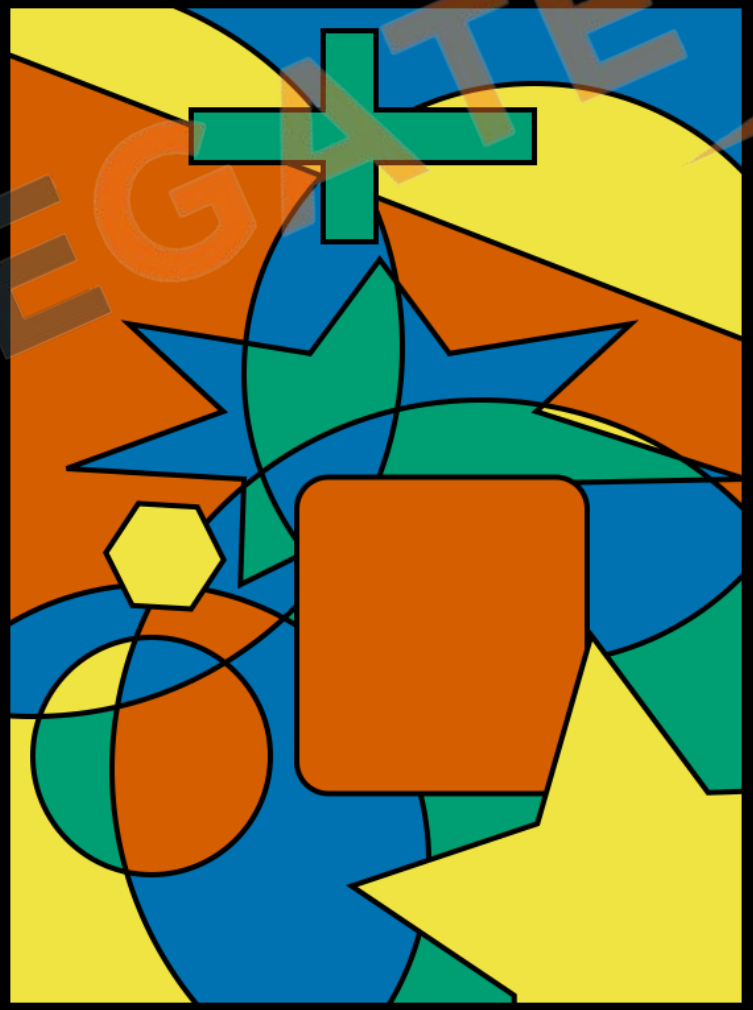


KNOWLEDGEGATE

Graph Coloring

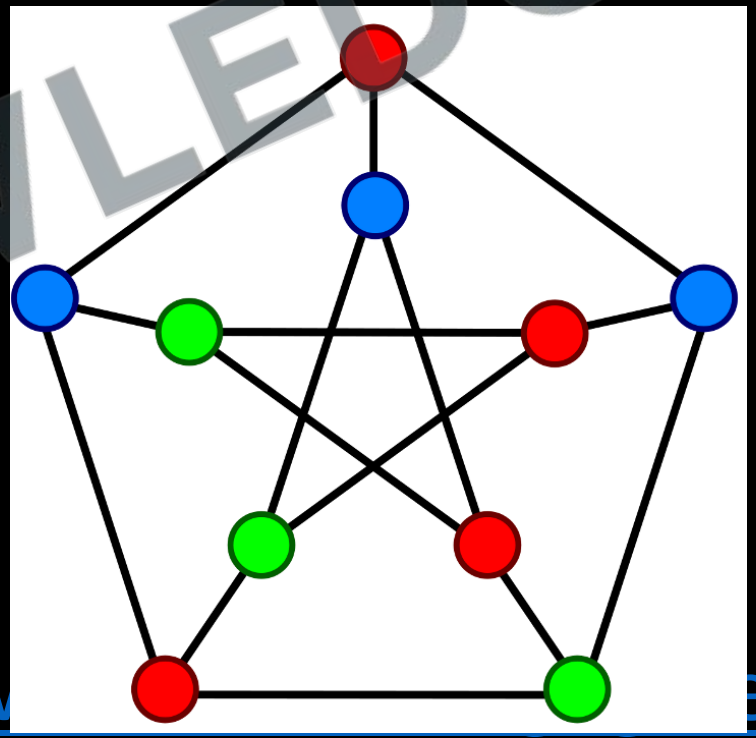
- Graph coloring can be of two types vertex coloring and region coloring.



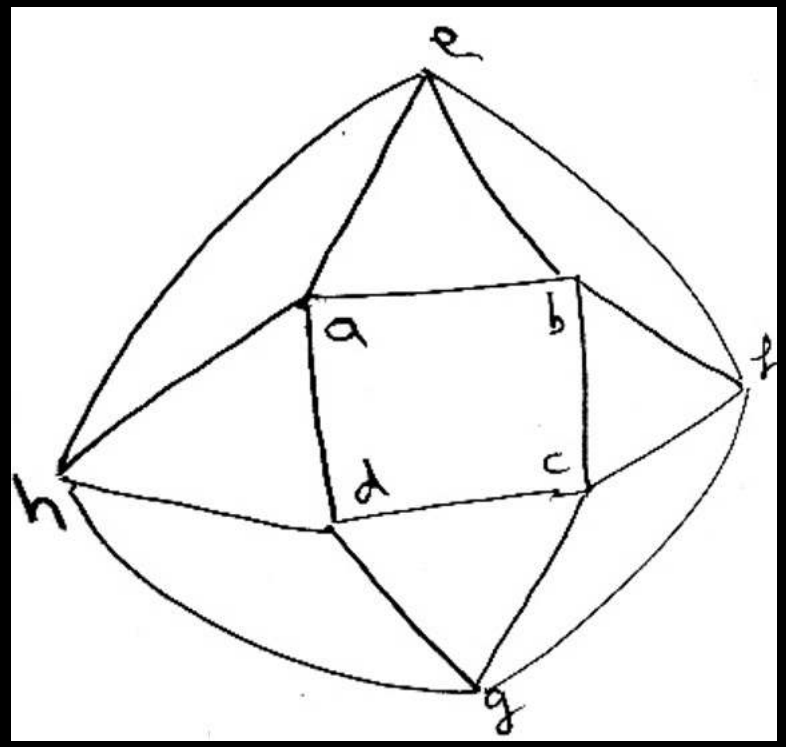


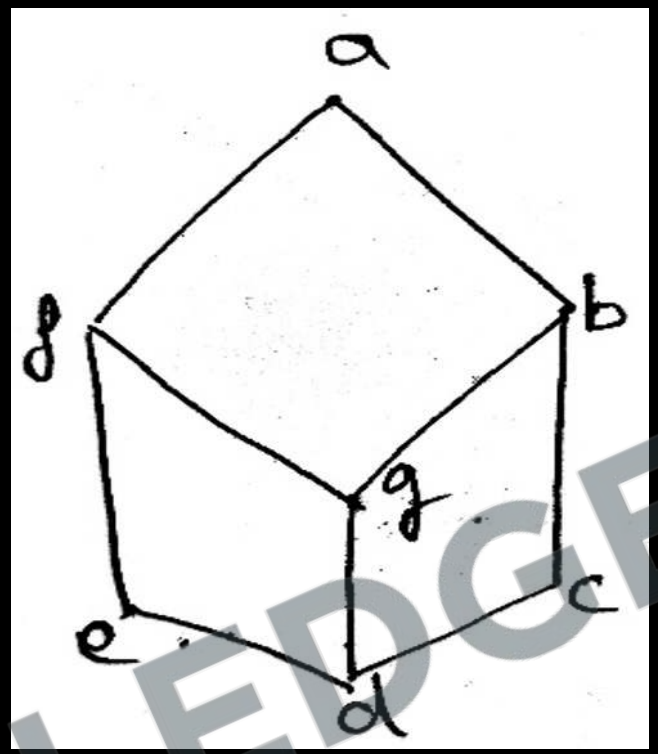
KNOWLEDGEGATE

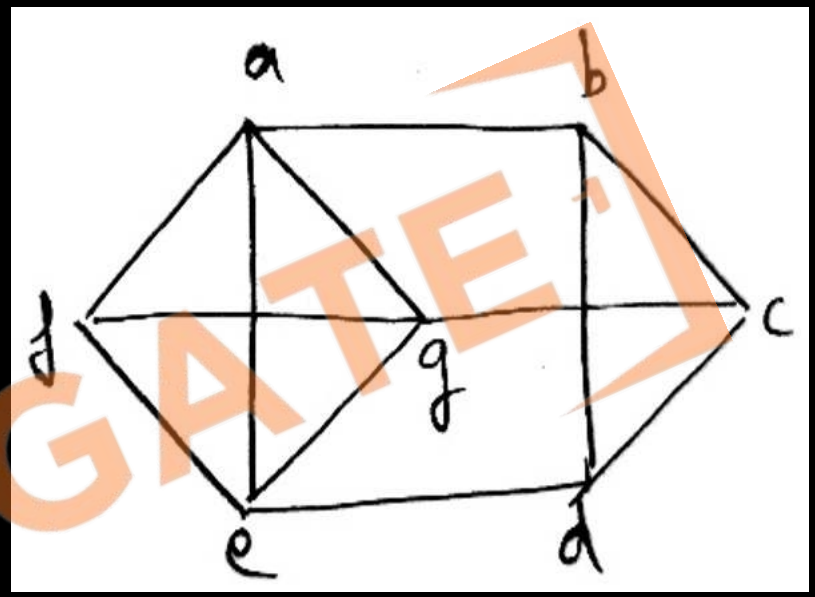
- Associating a color with each vertex of the graph is called vertex coloring.
- **Proper Vertex coloring:** - Associating all the vertex of a graph with colors such that no two adjacent vertices have the same color is called proper vertex coloring.
- **Chromatic number of the graph:** - Minimum number of colors required to do a proper vertex coloring is called the chromatic number of the graph, denoted by $\chi(G)$. the graph is called K-chromatic or K-colorable.



KNOWLEDGE GATE
http://w e.in/gate

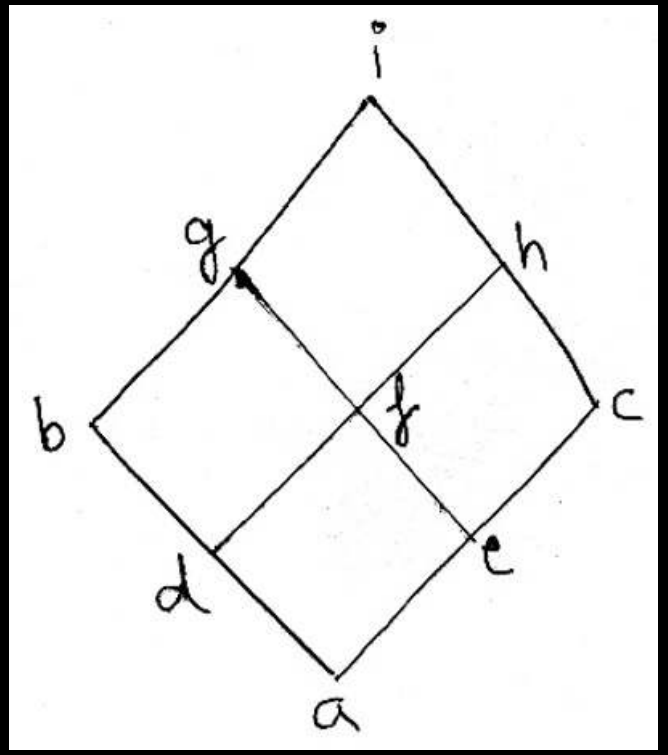


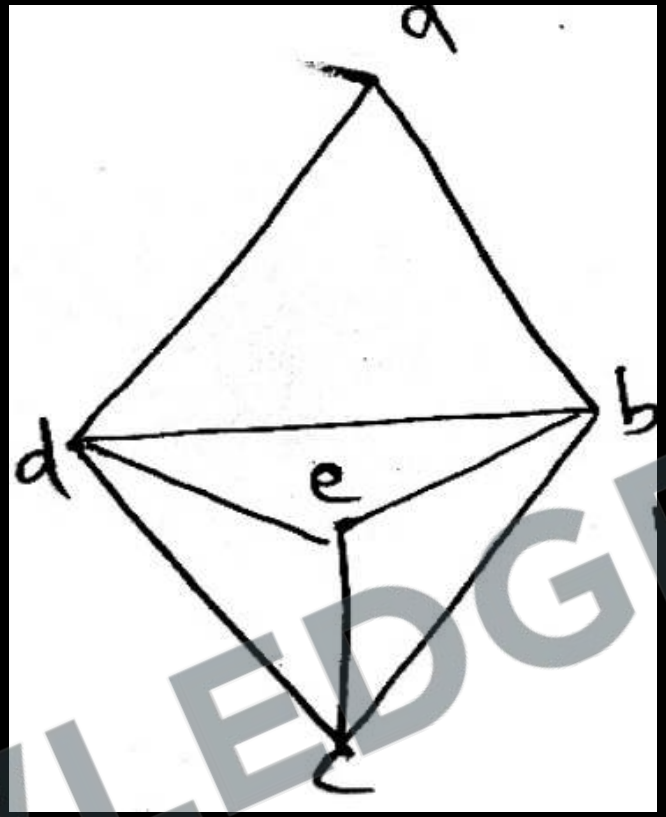


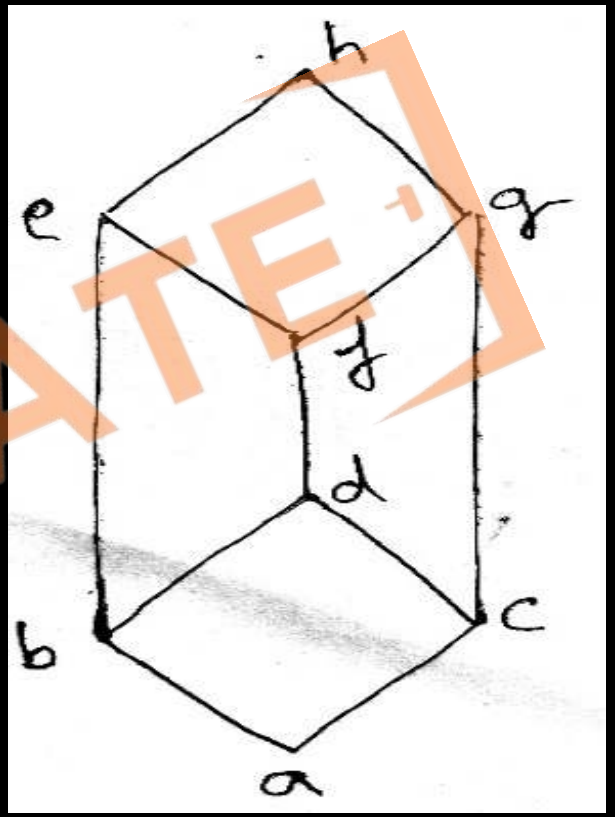


KNOWLEDGEGATE

http://www.knowledgegate.in/gate





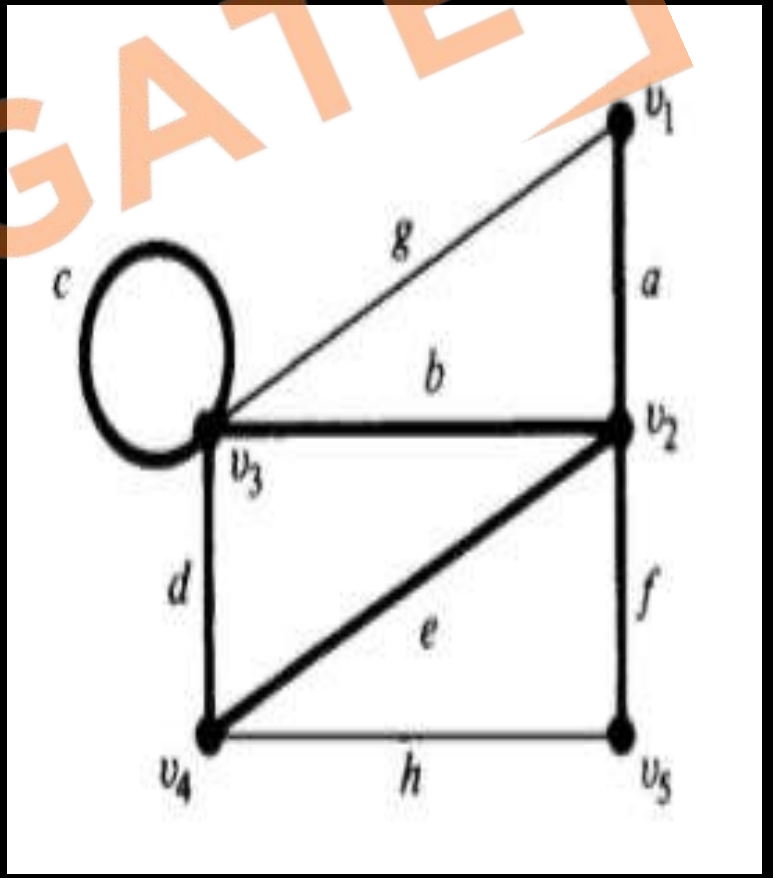


KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Traversal

- **Walk / Edge Train / Chain**: -A Walk is defined as a finite alternating sequence of vertices and edges, beginning and ending with vertices. Vertex and edge, may appear more than once in a walk.
- It is possible for a walk to begin and end at the same vertex. Such a walk is called a closed walk. A walk that is not closed is called an open walk.
- An open walk in which no vertex appears more than once is called a path (a path does not interact itself). Number of edges in a path is called length of a path.



Open Walk	
Closed Walk	
Path	

KNOWLEDGEGATE

- **Connected Graph:** A graph is said to be connected if there is at least one path between every pair of vertices in G.
- A graph with n vertices can be connected with minimum n -1 edges.
- A graph with n vertices will necessary be connected if it has more than (n - 1) (n - 2)/2 edges.

KNOWLEDGEGATE

Q Which condition is necessarily for a graph to be connected?

a) A graph with 6 vertices and 10 edges

b) A graph with 7 vertices and 14 edges

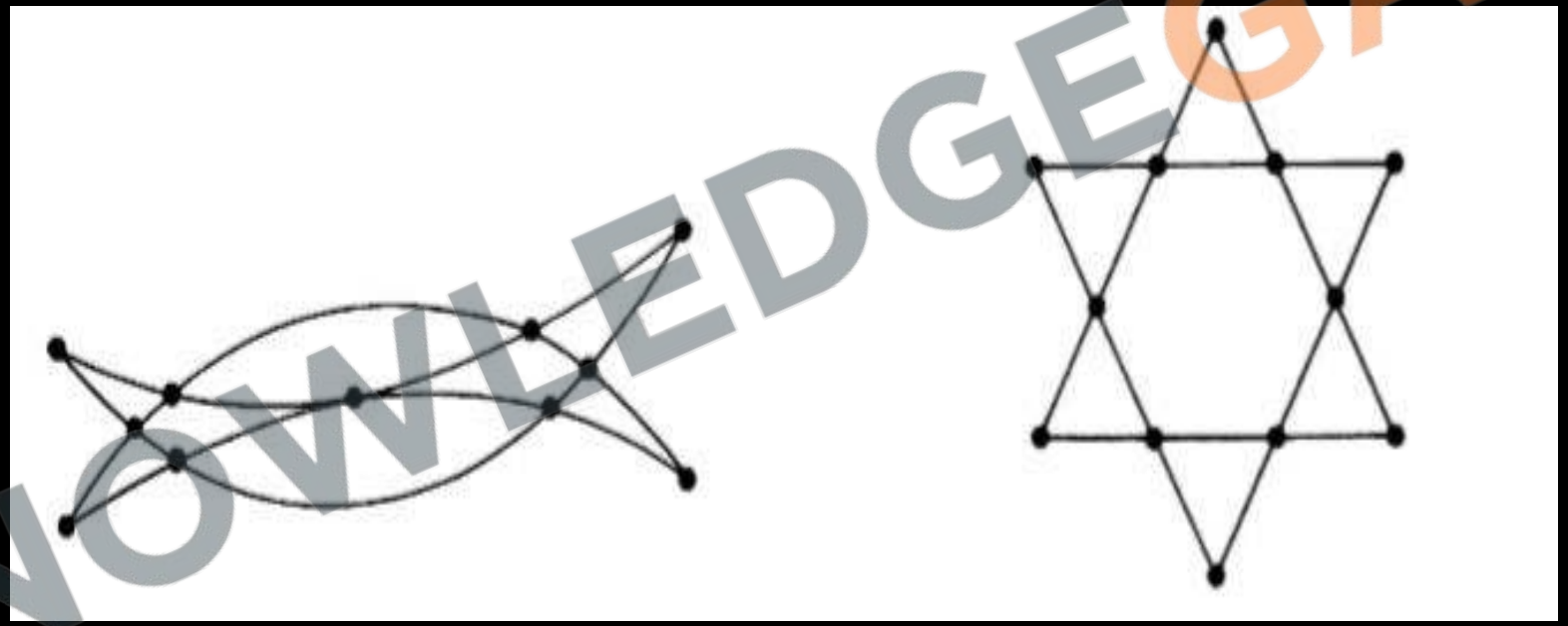
c) A graph with 8 vertices and 22 edges

d) A graph with 9 vertices and 28 edges

KNOWLEDGEGATE

Euler Graph

1. **Euler Graph:** - If some closed walk in a graph contains all the edges of the graph(connected), then the walk is called a Euler line and the graph a Euler Graph.
2. A given connected graph G is a Euler graph if and only if all vertices of G are of even degree.

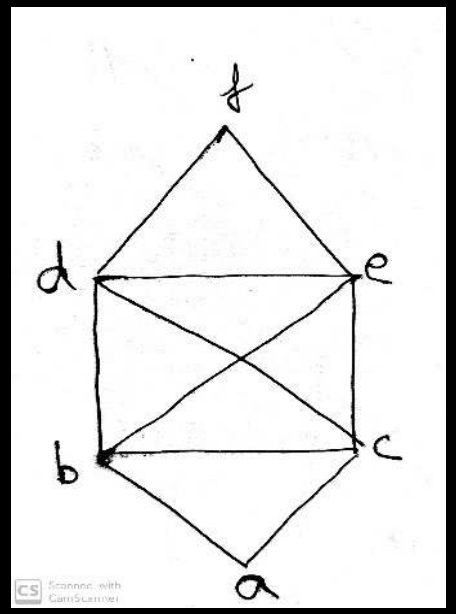


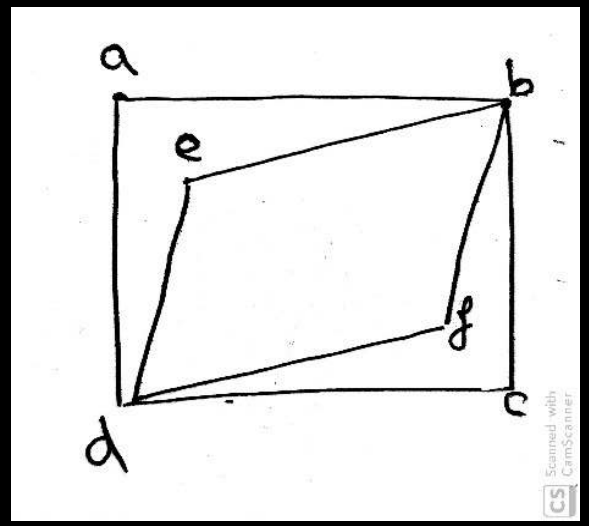
KNOWLEDGEGATE

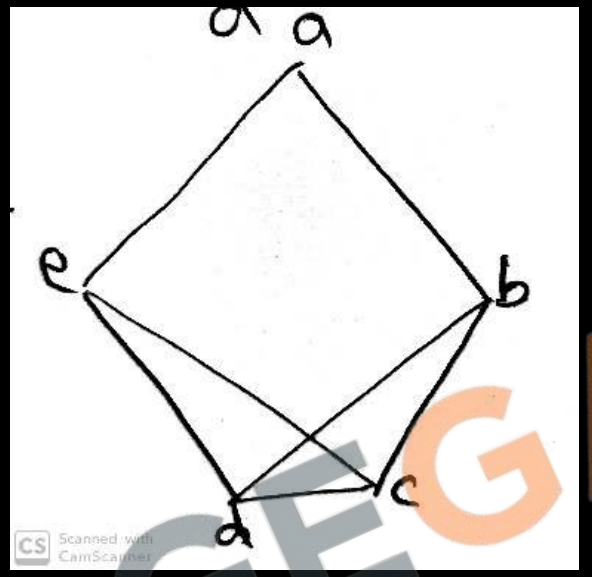
Hamiltonian

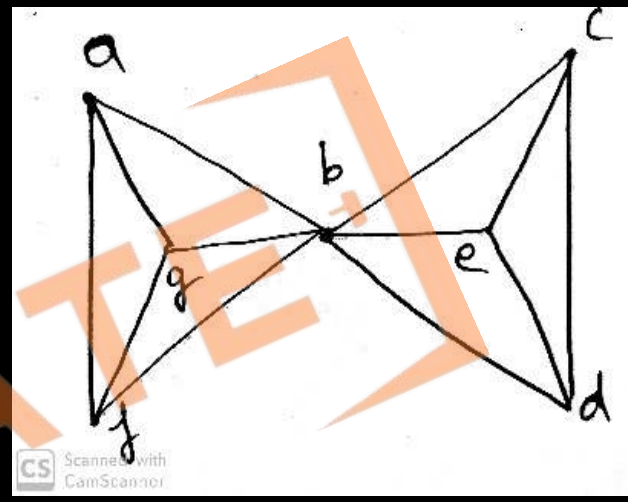
1. **Hamiltonian Graph:** - A Hamiltonian circuit in a connected graph is defined as a closed walk that traverses every vertex of G exactly once, except of course the starting vertex, at which the walk also terminates. A graph containing Hamiltonian circuit is called Hamiltonian graph.
2. Finding weather a graph is Hamiltonian or not is a NPC problem.

KNOWLEDGEGATE







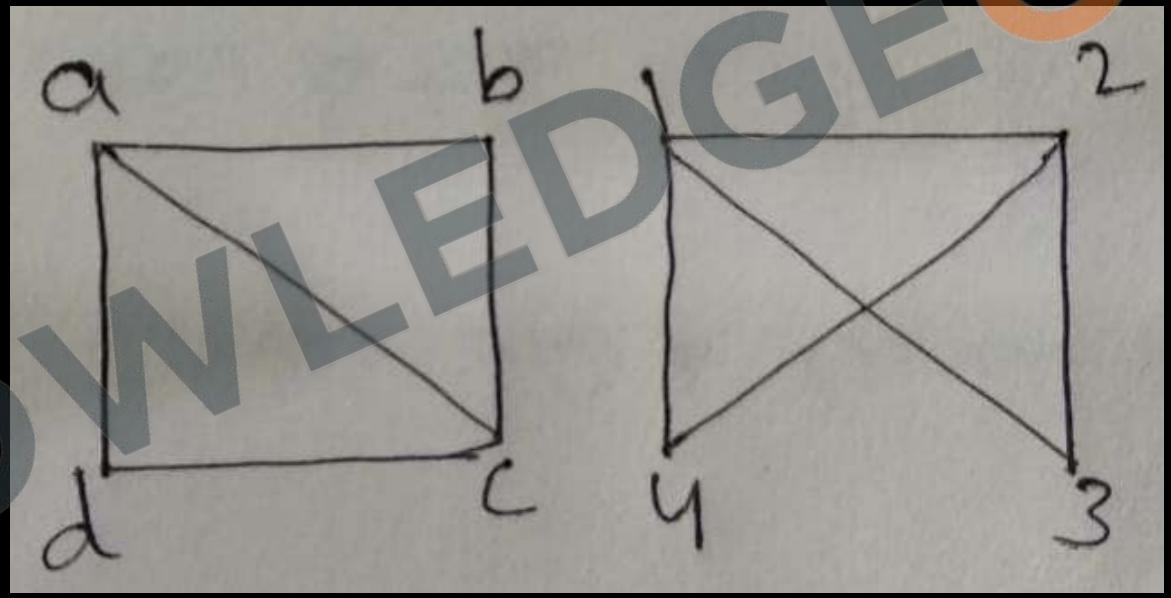


KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Isomorphism

1. In general, two graphs are said to be isomorphic if they are perhaps the same graphs, but just drawn differently with different names. i.e. two graphs are thought of as isomorphic if they have identical behavior in terms of graph-theoretic properties.

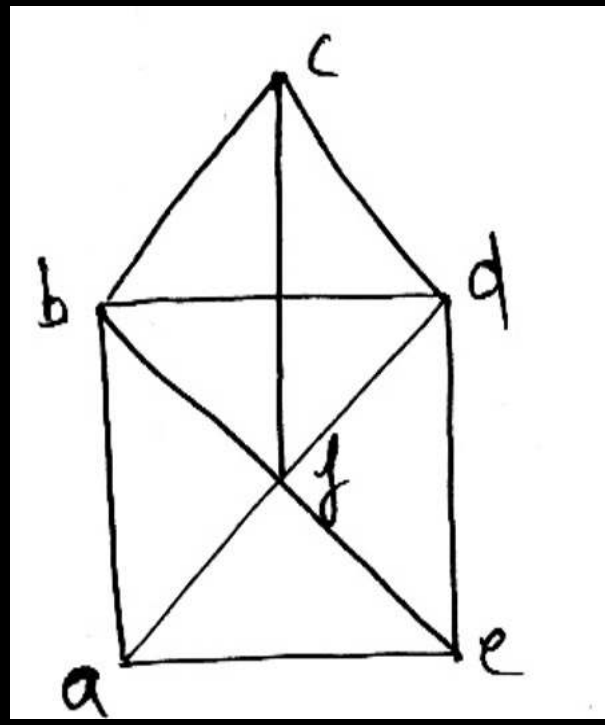


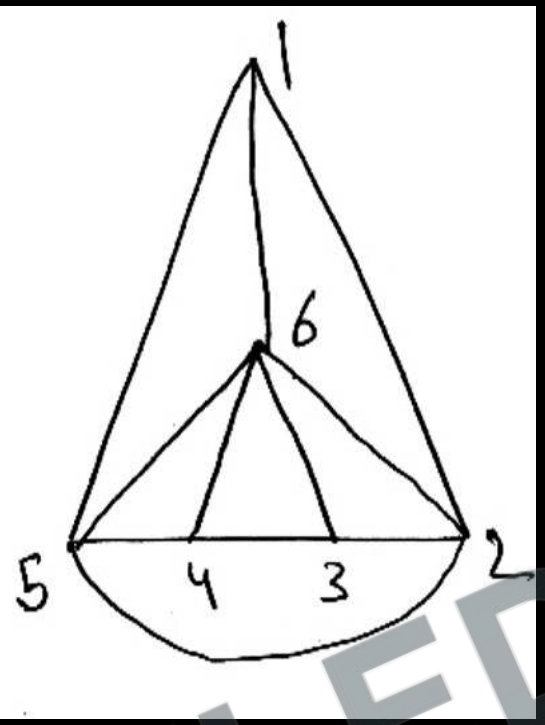
KNOWLEDGEGATE

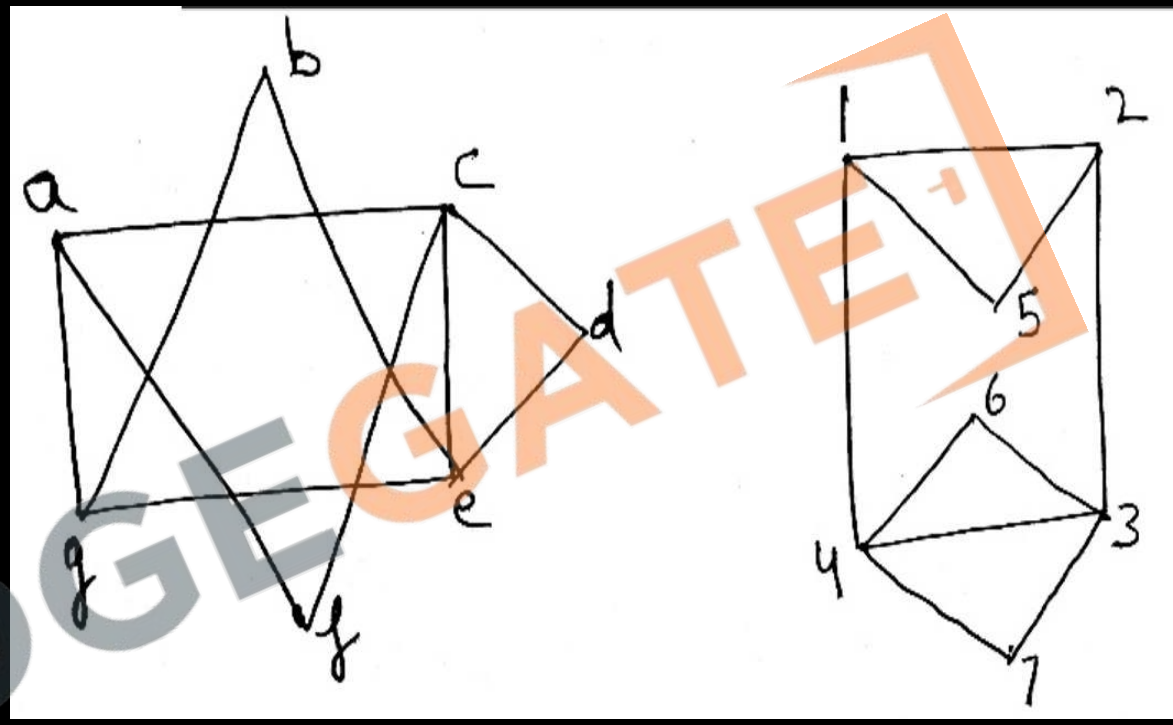
Q How many simple non isomorphic graphs are possible with 3 vertices ?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

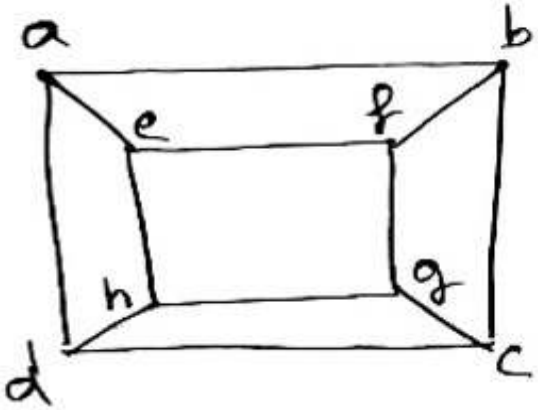


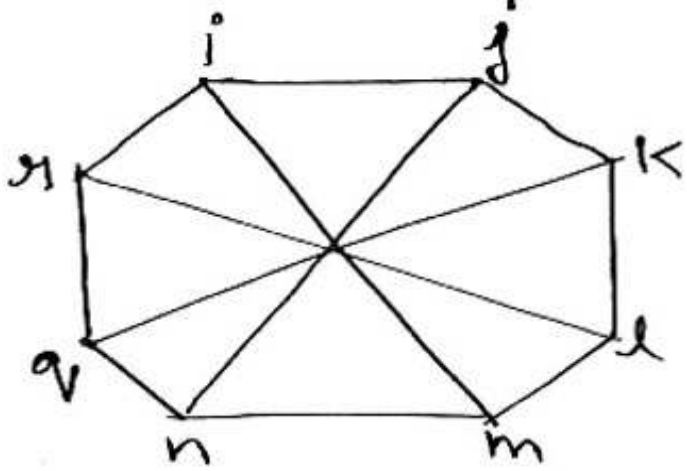


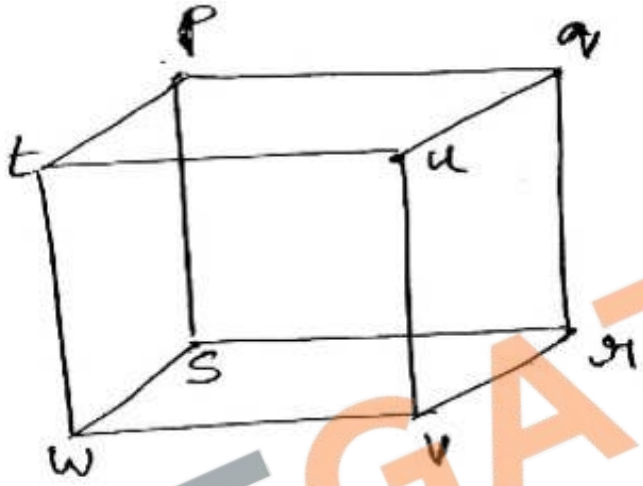


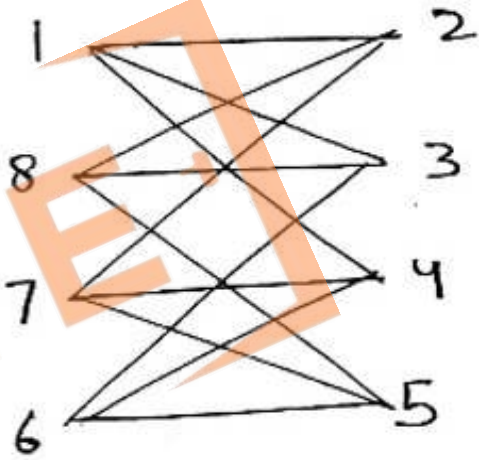
KNOWLEDGEGATE

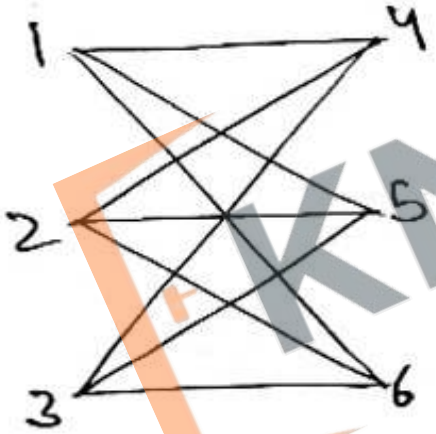
http://www.knowledgegate.in/gate

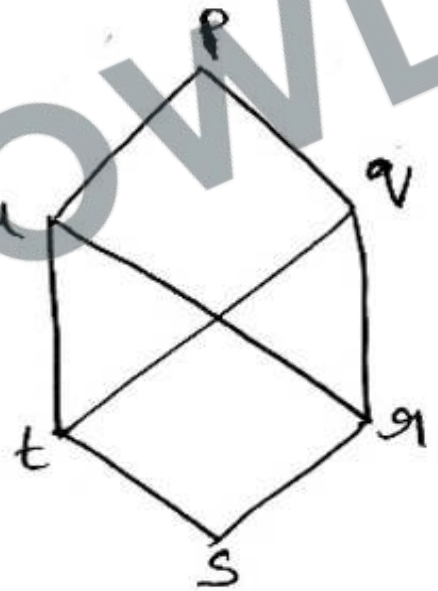


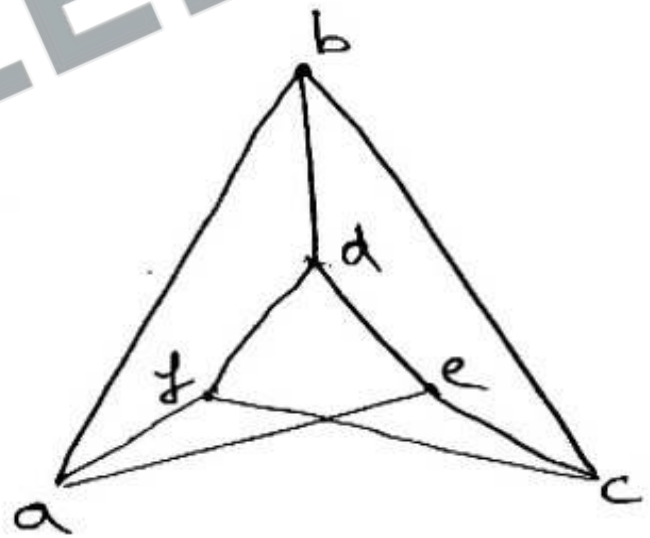


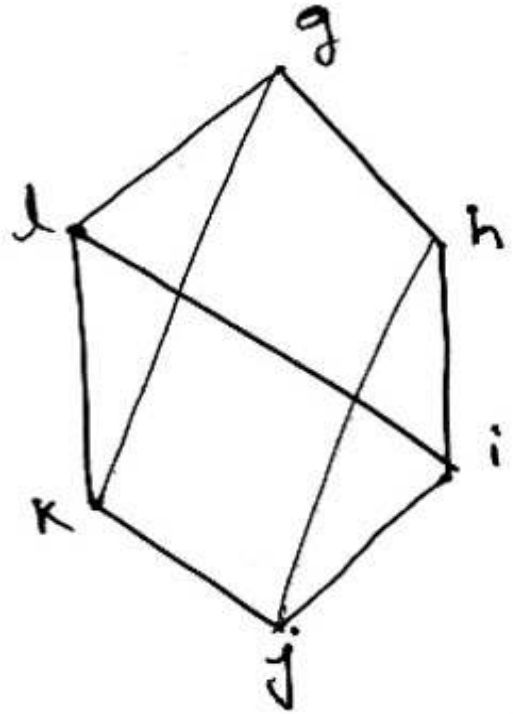




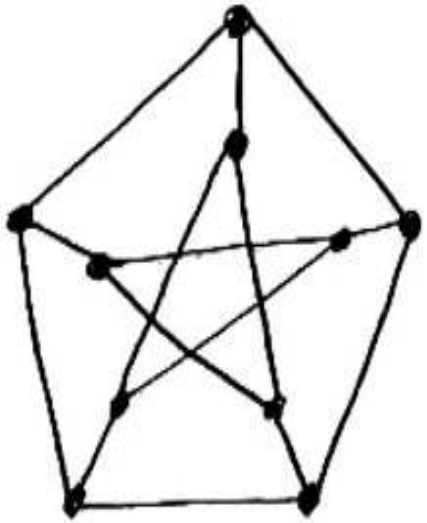
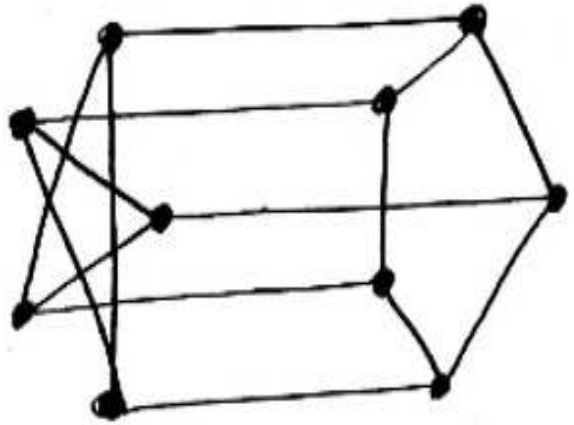


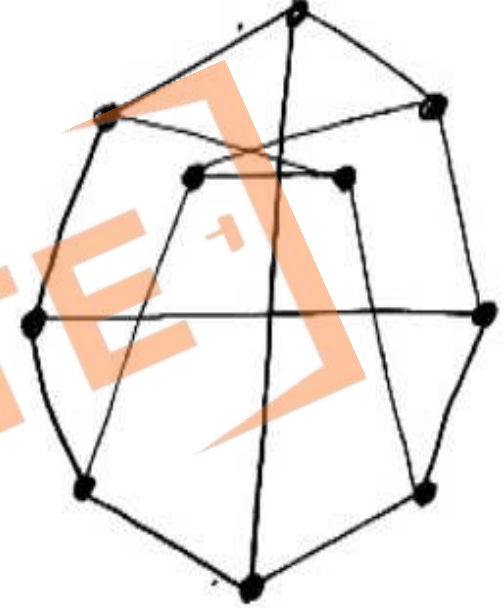


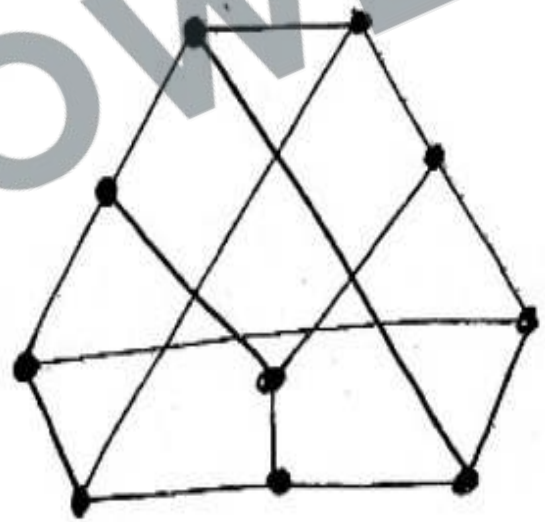
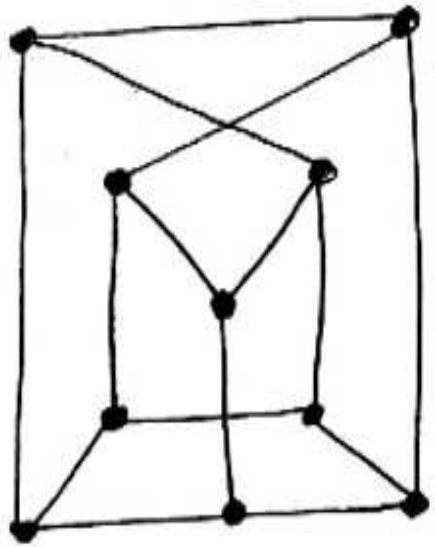
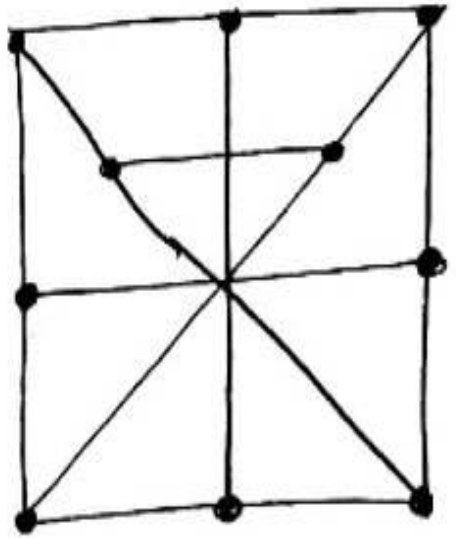




KNOWLEDGEGATE



KNOWLEDGEGATE

Chapter-8 (Combinatorics)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Permutation

- Permutation refers to the arrangement of a set of items or elements in a specific order. The key points about permutation are:
  - **Order Matters**: The arrangement is considered different if the order of items is different.
  - **Counting Permutations**: To find the number of permutations of 'n' items taken 'r' at a time, we use the formula ${}^{n}p_{m} = P(n, m)$.

KNOWLEDGEGATE

Examples:

- **Simple Example**:
  - Items: A, B
  - Permutations: AB, BA
  - Total: 2 permutations

- **Numerical Example**:
  - Items: 1, 2, 3
  - Permutations: 123, 132, 213, 231, 312, 321
  - Total: 6 permutations ($3! = 3 \times 2 \times 1 = 6$)

- **Real-world Example**:
  - Scenario: Arranging books on a shelf.
  - Books: Physics (P), Chemistry (C), Mathematics (M)
  - Permutations: PCM, PMC, CMP, CPM, MPC, MCP
  - Total: 6 arrangements

KNOWLEDGEGATE

Combination

- Combination refers to the selection of items from a larger set where the order of selection does not matter. The key points about combination are:
  - **Order Doesn't Matter**: Unlike permutations, the arrangement or order of the selected items is irrelevant in combinations.
  - **Counting Combinations**: To find the number of combinations of 'n' items taken 'r' at a time, we use the formula ${}^{n}C_{r} = (n!) / ((n-r)!n!)$.

KNOWLEDGEGATE

- **Simple Example**:
  - Items: A, B, C
  - Selecting 2 items at a time.
  - Combinations: AB, AC, BC
  - Total: 3 combinations

- **Numerical Example**:
  - Items: 1, 2, 3, 4
  - Selecting 2 items at a time.
  - Combinations: 12, 13, 14, 23, 24, 34
  - Total: 6 combinations (4 2=4!2!(4−2)!=64*C*2=2!(4−2)!4!=6)

- **Real-world Example**:
  - Scenario: Choosing 2 fruits from a basket containing an Apple (A), Banana (B), and Cherry (C).
  - Combinations: AB, AC, BC
  - Total: 3 ways to choose 2 fruits out of 3.

KNOWLEDGEGATE

Q A collection of 10 electric bulbs contain 3 defective ones?

- In how many ways can a sample of four bulbs be selected?

$$^{10}C_4$$

- In how many ways can a sample of 4 bulbs be selected which Contain 2 good bulbs and 2 defective ones ?

$$^{7}C_2 + {}^{3}C_2$$

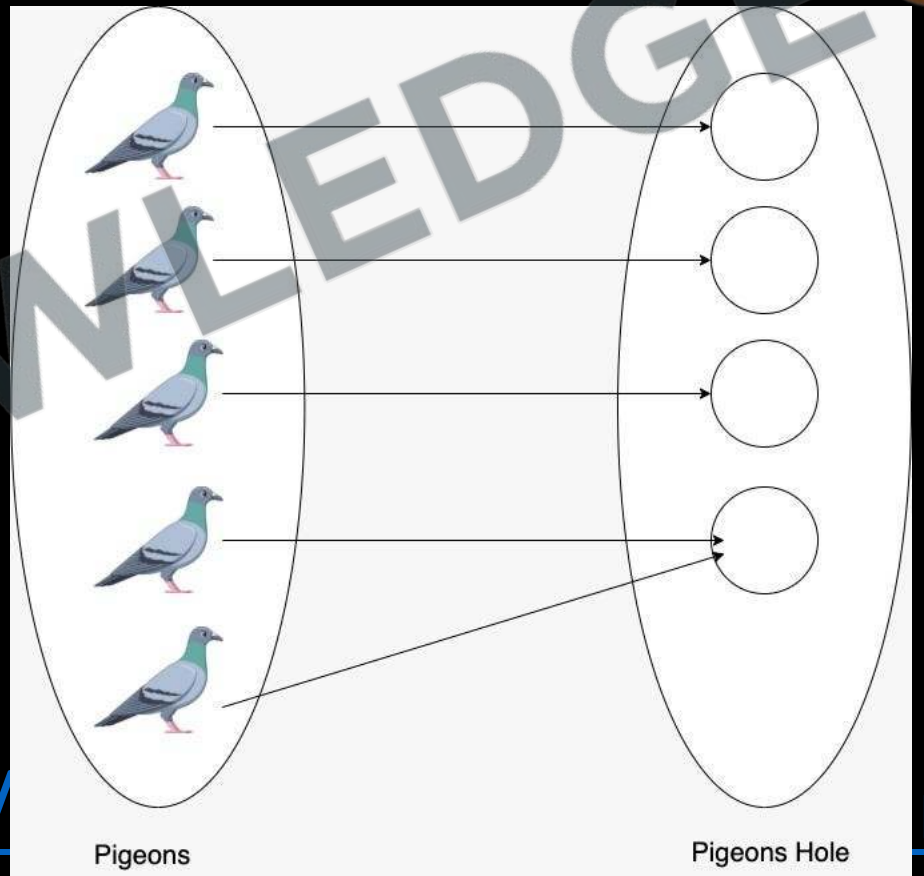
- In how many ways can a sample of 4 bullbs be selected so that either the sample contain 3 good ones and 1 defectives ones or 1 good and 3 defectives ones ?

$$^{7}C_3 \times {}^{3}C_1 + {}^{7}C_1 \times {}^{3}C_3$$

KNOWLEDGEGATE

Pigeonhole Principle

- It states that if more items (pigeons) are put into fewer containers (pigeonholes) than there are items, then at least one container must contain more than one item. Key points include:
  - **Basic Idea**: If 'n' items are distributed among 'm' containers, and if $n>m$, then at least one container has more than one item.



KNOWLEDGEGATE

http:/ n/gate

- **Simple Example**:
  - Socks: 10 pairs of socks (20 individual socks) and 19 drawers.
  - According to the Pigeonhole Principle, since there are more socks than drawers, at least one drawer must contain more than one sock.

- **Real-world Example**:
  - Scenario: A classroom with 30 students.
  - If you want to prove that at least two students have their birthdays in the same month, you consider the months (12 pigeonholes) and the students (30 pigeons).
  - According to the Pigeonhole Principle, at least one month (pigeonhole) will have more than [30/12] = 3 students (pigeons) sharing their birthdays.

[KNOWLEDGEGATE]

No of onto function possible from A to B

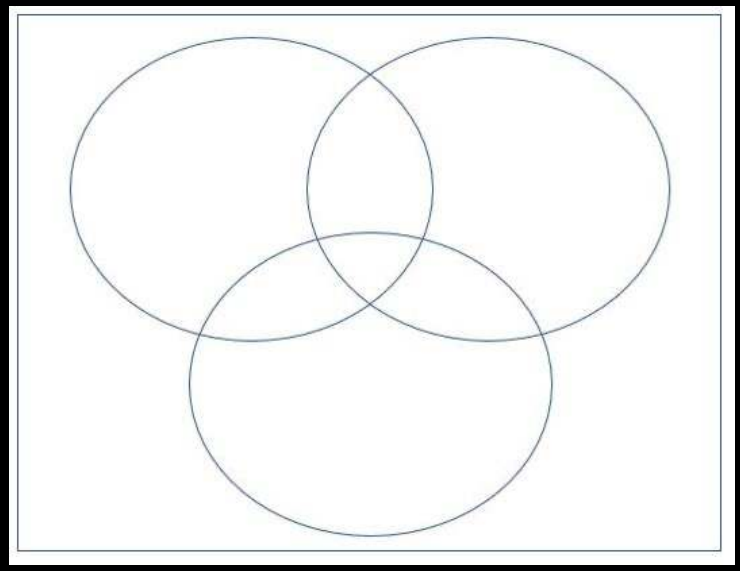
$= n^m - {}^nc_1(n-1)^m + {}^nc_2(n-2)^m - {}^nc_3(n-3)^m + \text{---------} + (-1)^n\ {}^nc_{n-1}\ 1^m$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q In a class of 200 students, 125 students have taken Programming Language course, 85 students have taken Data Structures course, 65 students have taken Computer Organization course; 50 students have taken both Programming Language and Data Structures, 35 students have taken both Data Structures and Computer Organization; 30 students have taken both Programming Language and Computer Organization, 15 students have taken all the three courses. How many students have not taken any of the three courses?

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |B \cap C| - |A \cap C| + |A \cap B \cap C|$$

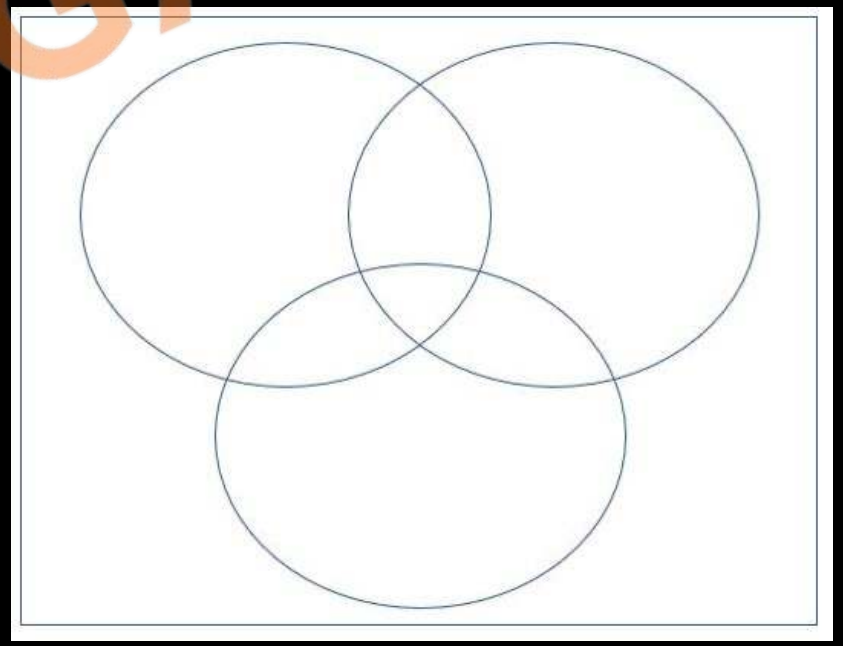


125+85+65-50−35−30+15

KNOWLEDGEGATE

Q The number of integers between 1 and 500 (both inclusive) that are divisible by 3 or 5 or 7 is _____.

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |B \cap C| - |A \cap C| + |A \cap B \cap C|$$

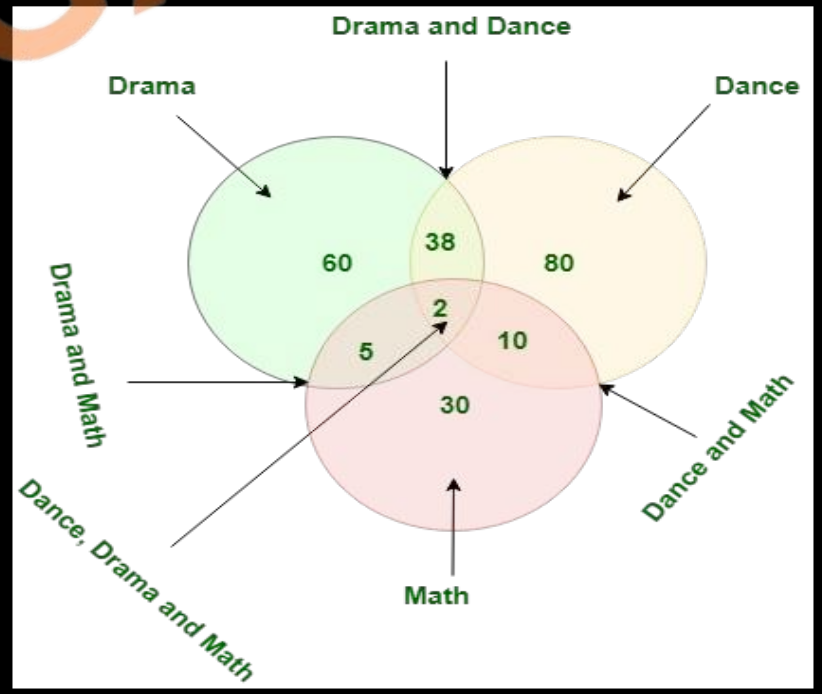


166 + 100 + 71 -33-14-23+4

KNOWLEDGEGATE

Q In a college, there are three student clubs, Sixty students are only in the Drama club, 80 students are only in the Dance club, 30 students are only in Maths club, 40 students are in both Drama and Dance clubs, 12 students are in both Dance and Maths clubs, 7 students are in both Drama and Maths clubs, and 2 students are in all clubs. If 75% of the students in the college are not in any of these clubs, then the total number of students in the college is _____.

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |B \cap C| - |A \cap C| + |A \cap B \cap C|$$



KNOWLEDGEGATE

Module

Free Semester One Shot Exam Course

Topic

Database Management Systems

Subtopic

DBMS One Shot

Lesson

DBMS_Notes

Video chapters

- **Chapter-1 (Basics):** Data & information, Database System vs File System, Views of Data Base, Data Independence, Instances & Schema, OLAP Vs OLTP, Types of Data Base, DBA, Architecture.
- **Chapter-2 (ER Diagram):** Entity, Attributes, Relationship, Degree of a Relationship, Mapping, Weak Entity set, Conversion from ER Diagram to Relational Model, Generalization, Specification, Aggregation.
- **Chapter-3 (RDBMS & Functional Dependency):** Basics & Properties, Update Anomalies, Purpose of Normalization, Functional Dependency, Closure Set of Attributes, Armstrong's axioms, Equivalence of two FD, Canonical cover, Keys.
- **Chapter-4 (Normalization):** 1NF, 2NF, 3NF, BCNF, Multivalued Dependency, 4NF, Lossy-Lossless Decomposition, 5NF, Dependency Preserving Decomposition.
- **Chapter-5 (Indexing):** Overview of indexing, Primary indexing, Clustered indexing and Secondary Indexing, B-Tree.
- **Chapter-6 (Relational Algebra) :** Query Language, Select, Project, Union, Set Difference, Cross Product, Rename Operator, Additional or Derived Operators.
- **Chapter-7(SQL) :** Introduction to SQL, Classification, DDL Commands, Select, Where, Set Operations, Cartesian Product, Natural Join, Outer Join, Rename, Aggregate Functions, Ordering, String, Group, having, Trigger, embedded, dynamic SQL.
- **Chapter-8 (Relational Calculus) :** Overview, Tuple Relation Calculus, Domain Relation Calculus.
- **Chapter-9 (Transaction) :** What is Transaction, ACID Properties, Transaction Sates, Schedule, Conflict Serializability, View Serializability, Recoverability, Cascade lessness, Strict Schedule.
- **Chapter-10 (Recovery & Concurrency Control)** : Log Based Recovery, Shadow Paging, Data Fragmentation, TIME STAMP ORDERING PROTOCOL, THOMAS WRITE RULE, 2 phase locking, Basic 2pl, Conservative 2pl, Rigorous 2pl, Strict 2pl, Validation based protocol Multiple Granularity.

Knowledge Gate Website

Syllabus

- **UNIT-1 : INTRODUCTION** Overview, Database System vs File System, Database System Concept and Architecture, Data Model Schema and Instances, Data Independence and Database Language and Interfaces, Data Definitions Language, DML, Overall Database Structure. **Data Modeling Using the Entity Relationship Model**: ER Model Concepts, Notation for ER Diagram, Mapping Constraints, Keys, Concepts of Super Key, Candidate Key, Primary Key, Generalization, Aggregation, Reduction of an ER Diagrams to Tables, Extended ER Model, Relationship of Higher Degree.
- **UNIT-2 : RELATIONAL DATA MODEL** Relational Data Model Concepts, Integrity Constraints, Entity Integrity, Referential Integrity, Keys Constraints, Domain Constraints, Relational Algebra, Relational Calculus, Tuple and Domain Calculus. **Introduction on SQL**: Characteristics of SQL, Advantage of SQL. SQl Data Type and Literals. Types of SQL Commands. SQL Operators and Their Procedure. Tables, Views and Indexes. Queries and Sub Queries. Aggregate Functions. Insert, Update and Delete Operations, Joins, Unions, Intersection, Minus, Cursors, Triggers, Procedures in SQL/PL SQL.
- **UNIT-3 : DATA BASE DESIGN & NORMALIZATION** Functional dependencies, normal forms, first, second, 3 third normal forms, BCNF, inclusion dependence, loss less join decompositions, normalization using FD, MVD, and JDs, alternative approaches to database design.
- **UNIT-4 : TRANSACTION PROCESSING CONCEPT** Transaction System, Testing of Serializability, Serializability of Schedules, Conflict & View Serializable Schedule, Recoverability, Recovery from Transaction Failures, Log Based Recovery, Checkpoints, Deadlock Handling. Distributed Database: Distributed Data Storage, Concurrency Control, Directory System.
- **UNIT-5 : CONCURRENCY CONTROL TECHNIQUES** Concurrency Control, Locking Techniques for Concurrency Control, Time Stamping Protocols for Concurrency Control, Validation Based Protocol, Multiple Granularity, Multi Version Schemes, Recovery with Concurrent Transaction, Case Study of Oracle.

Knowledge Gate Website

What is Data ?

- Data are characteristics or attributes, often numerical, collected through observation and can be qualitative(descriptive) or quantitative(numerical). Any facts and figures about an entity is called as Data.
- Data serves a crucial role in various sectors by facilitating analysis, supporting decision-making processes, and being fundamental to research activities.

Knowledge Gate Website

What is Information ?

- Data becomes information when analyzed and placed in context, providing a basis for understanding, decision-making, and further analysis. Processed Data is called information.

Knowledge Gate Website

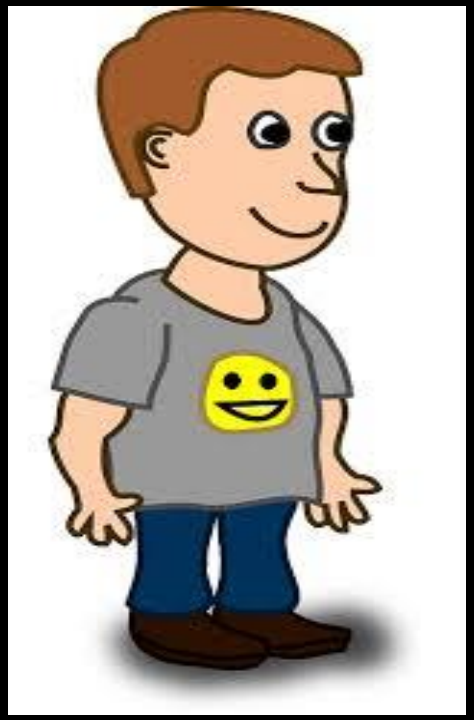
What is Data Base?

- A database is a structured collection of data, facilitating easy access, management, and updates., generally stored and accessed electronically from a computer system.

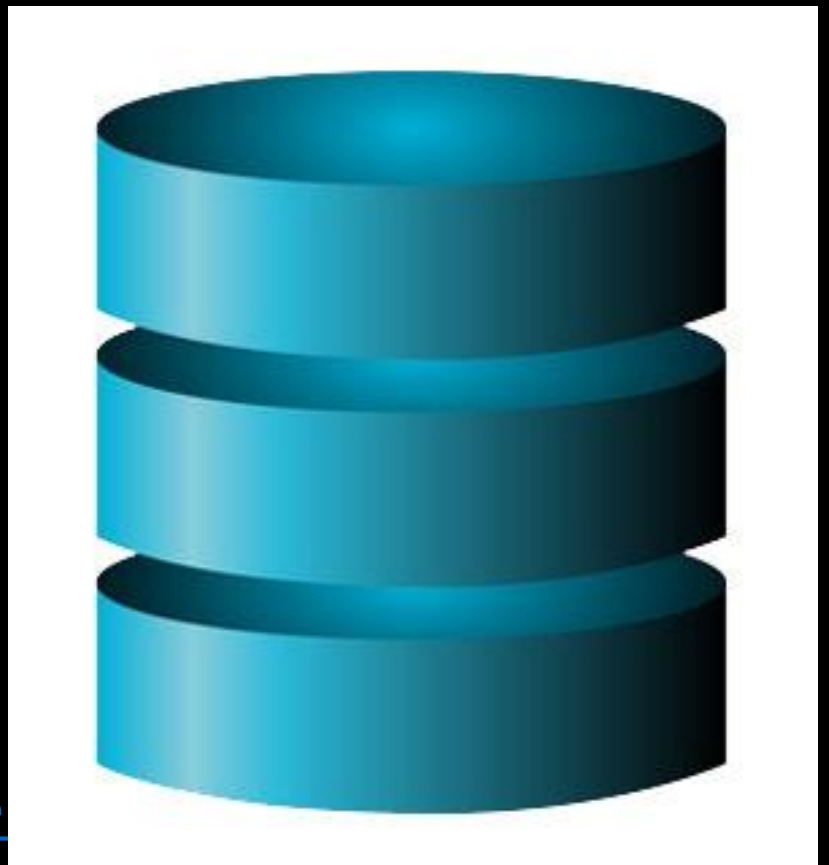
Knowledge Gate Website

What is Data Base Management System?

- A DBMS is software facilitating efficient data storage, retrieval, and management in databases.
- Ensures data safety and integrity, while offering accessibility and concurrency control.
- Supports functions like data querying, reporting, and analytics for informed decision-making.



Knowledge Gate

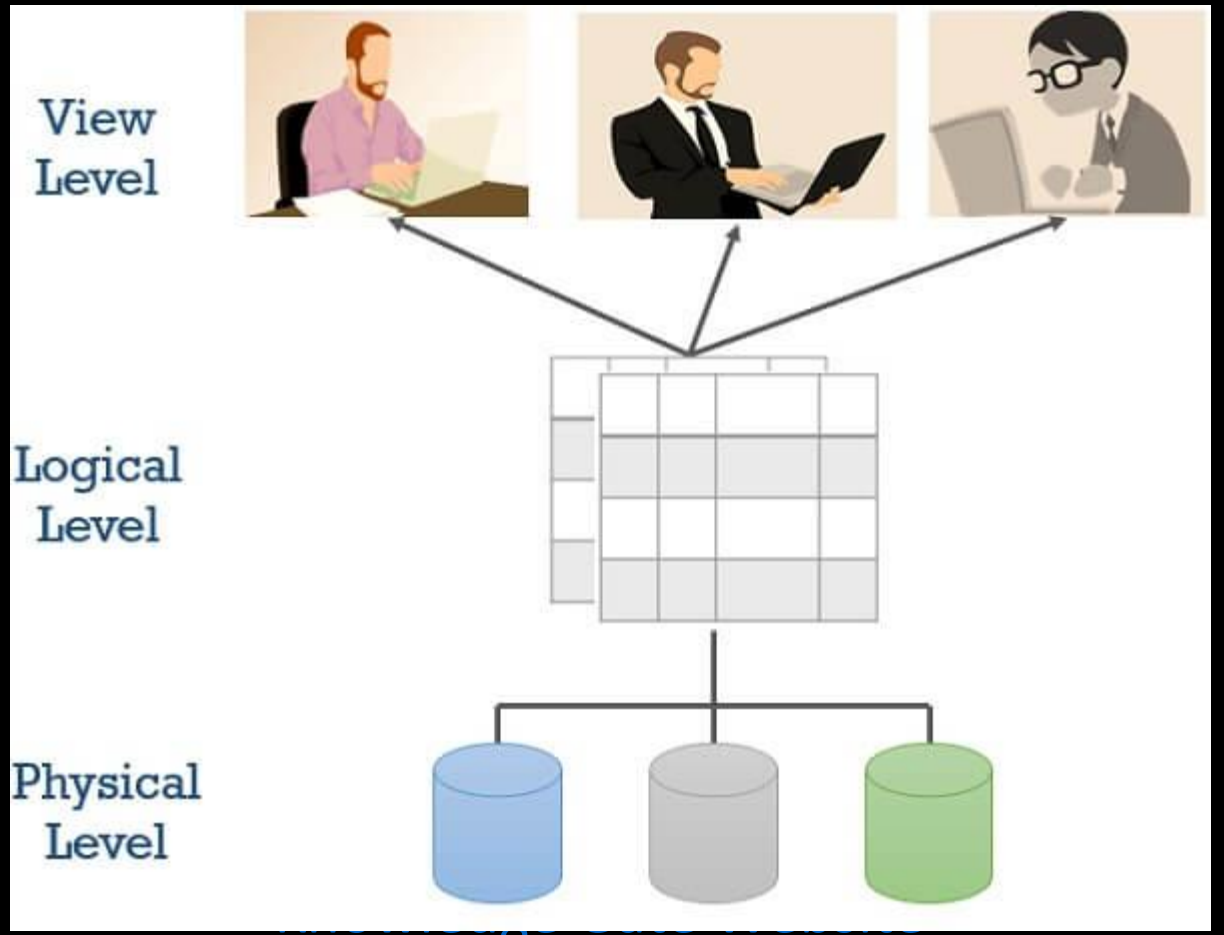


Aspect	File System	Database Management System
Data Access	Slower data retrieval due to unstructured querying capabilities.	Structured querying capabilities allow for quicker data access.
Data Isolation	Challenges in correlating data across separate files leading to data isolation.	Facilitates data integration, reducing data isolation issues.
Data Integrity	Risk of inadvertent data alterations or deletions creating integrity problems.	Features to prevent unauthorized data alterations, maintaining integrity.
Atomicity Problem	Potential for data inconsistency due to incomplete operations, leading to atomicity problems.	Supports transaction properties like atomicity, ensuring operations are either completed fully or not at all.
Concurrent Access Anomalies	Conflicts and inconsistencies from simultaneous data access/modifications, causing concurrent access anomalies.	Advanced concurrency controls to manage multiple users accessing the database simultaneously, reducing anomalies.

Knowledge Gate Website

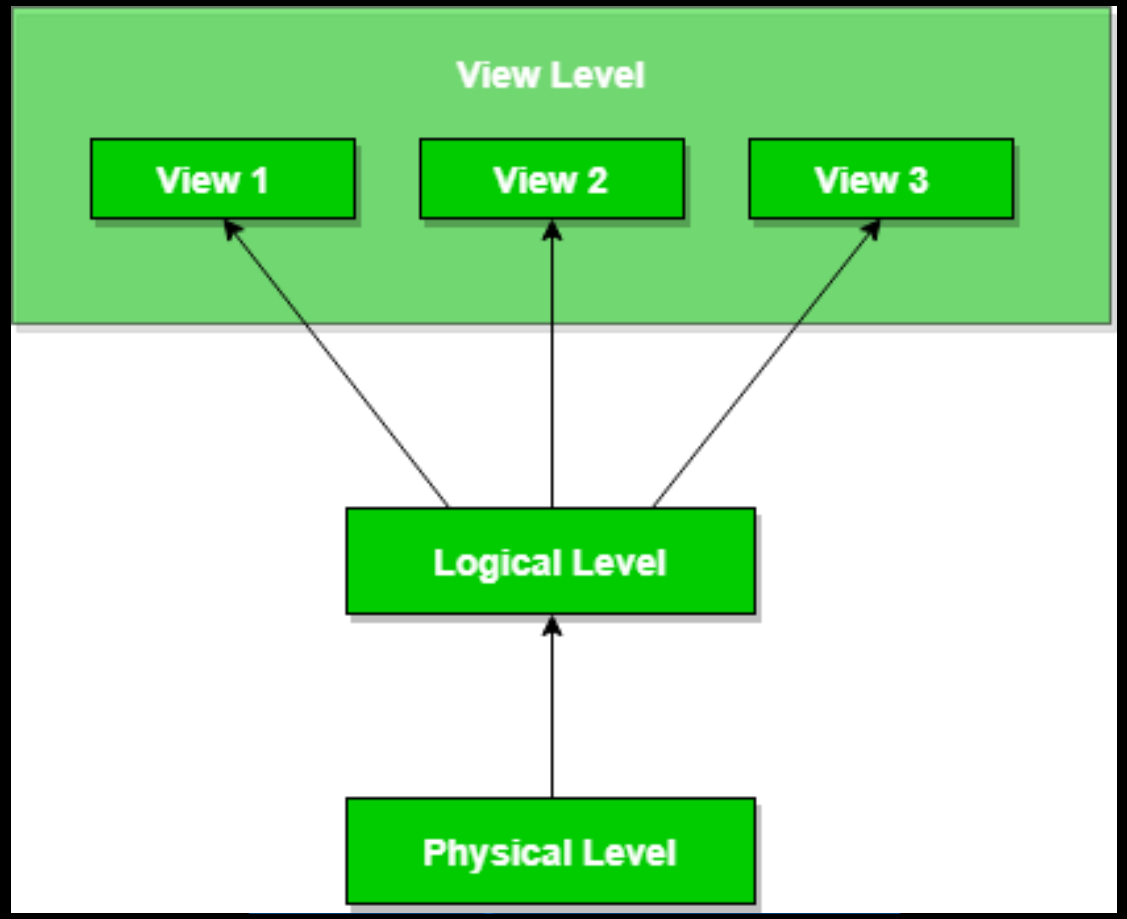
View of Data Base (Data Abstraction)

- Physical Level
- Logical Level/ Conceptual Level
- View Level



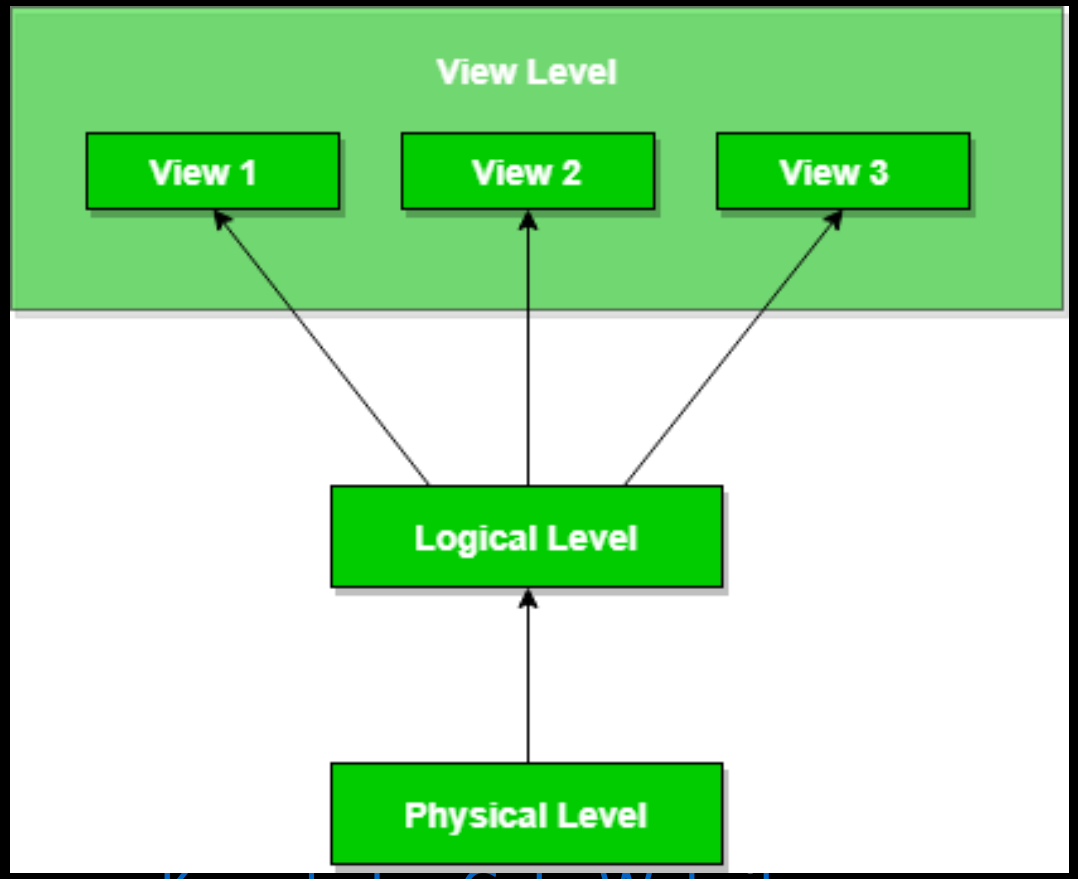
View of Data Base (Data Abstraction)

- **Physical Level:** The internal schema details data storage and access on hardware, featuring the lowest level of data abstraction with complex structures, predominantly managed by the database administrator.



View of Data Base

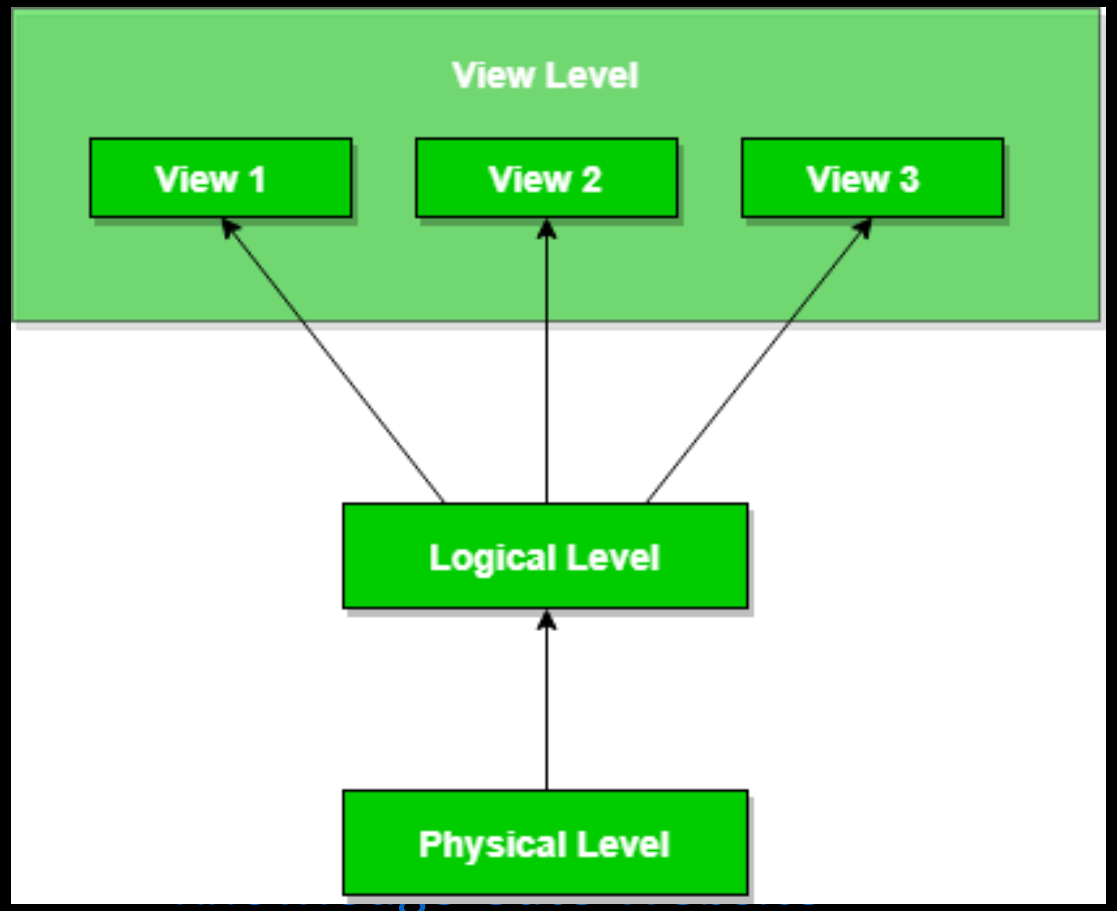
- **Logical Level/ Conceptual Level:** Above the physical level, this level showcases data as entity sets and their relationships, detailing the types and connections between stored data in the database.



Knowledge Gate Website

View of Data Base

- **View Level:** This is the pinnacle of data abstraction, displaying only a portion of the entire database focusing on user-interest areas. It can represent multiple views of the same data, allowing users to access information through various applications from the database.



Data independence

- Data independence is defined as the capacity to change the schema at one level of a database system without having to change the schema at the next higher level.
- Types of data independence :
  - **Physical data independence**: a. Physical data independence is the ability to modify internal schema without changing the conceptual schema. b. Modification at the physical level is occasionally necessary in order to improve performance. c. It refers to the immunity of the conceptual schema to change in the internal schema. d. Examples of physical data independence are reorganizations of files, adding a new access path or modifying indexes, etc.
  - **Logical data independence**: a. Logical data independence is the ability to modify the conceptual schema without having to change the external schemas or application programs. b. It refers to the immunity of the external model to changes in the conceptual model. c. Examples of logical data independence are addition/removal of entities.

Knowledge Gate Website

Instance and Schemas

- **Instance of the Database**:
  - The collection of information stored in the database at a specific moment is known as an instance of the database. It is a snapshot of the database that contains live data at that moment, showing the current state of all records and transactions.

- **Database Schema**:
  - The database schema refers to the overall design of the database, illustrating the logical structure and organization of data. It defines how data is organized and how relationships between data are handled, essentially serving as the blueprint for how the database is constructed.

Knowledge Gate Website

Aspect	OLAP (Online Analytical Processing)	OLTP (Online Transaction Processing)
Primary Function	Designed for complex data analysis and reporting.	Handles daily transactional data processing.
Database Design	Star or snowflake schema, optimizing for read operations.	Normalized schema, optimizing for write operations.
Query Complexity	Complex queries involving aggregations and computations across multiple dimensions.	Simple and standard queries focusing on CRUD operations (Create, Read, Update, Delete).
Data Volume	Deals with large volumes of data for historical analysis.	Processes a high number of small transactions.
Response Time	Slower response time due to complex queries.	Fast response time to support high transaction rates.

Knowledge Gate Website

Types of Data Base

- **Commercial Database**: Predominantly used in the business sector to handle large volumes of transactions and customer data. A CRM system like Salesforce which handles large volumes of customer data and transactions.

- **Multimedia Database**: Stores data types such as images, audio, and video files, facilitating the management and retrieval of multimedia content. A digital asset management system like Adobe Experience Manager that facilitates the storage and retrieval of multimedia content.

- **Deductive Database**: Utilizes logic programming to derive information from data stored in a database, allowing for more complex and analytical queries. A database using Datalog (a query language) which allows for complex logical queries and information derivation.

Knowledge Gate Website

- **Temporal Database**: Keeps track of changing data over time, allowing for queries concerning time-based data. A historical trading database in the financial sector which keeps track of stock prices over time.

- **Geological Information System (GIS)**: Stores, organizes, and analyzes geographical data, aiding in spatial analysis and mapping projects. A system like ArcGIS which enables the storage, analysis, and visualization of geographical data.

Knowledge Gate Website

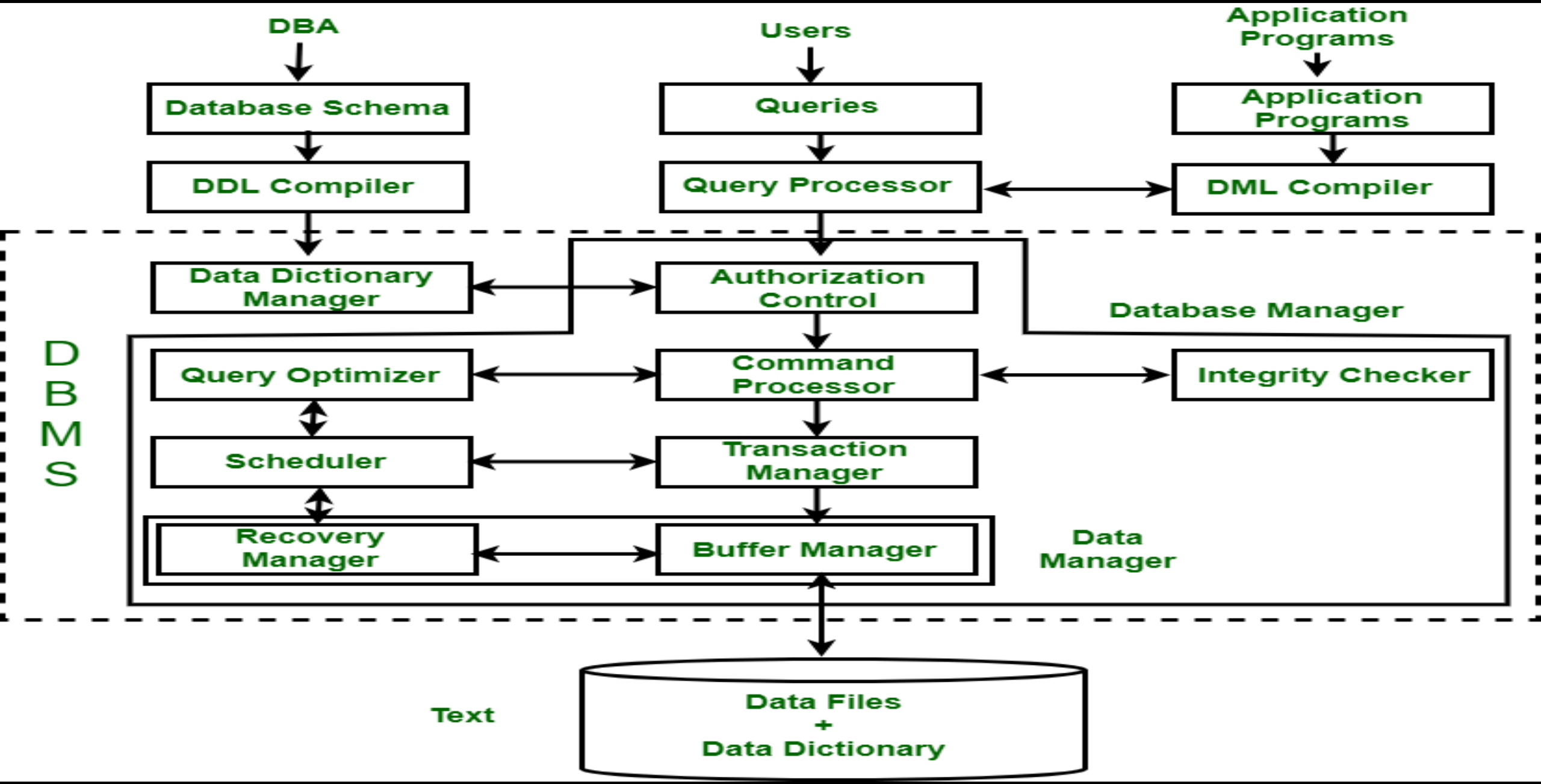
DBA(Database Administrator)

Database administrators hold authority over data and the programs facilitating data access. Their roles/functions are:

- **Schema Definition**: DBA outlines the original database schema. Achieved through writing definitions translated to permanent labels in the data dictionary by the DDL compiler.
- **Storage Structure and Access Method Definition**: Responsible for forming appropriate storage structures and access methods. Achieved through writing definitions translated by the data storage and definition language compiler.
- **Schema and Physical Organization Modification**: Involves altering the database schema or physical storage organization. Changes are implemented by writing definitions that modify the relevant internal system tables.
- **Granting of Authorization for Data Access**: DBA grants varied types of data access authorization to different database users.
- **Integrity Constraint Specification**: DBA implements and maintains integrity constraints to ensure data accuracy and consistency.

Knowledge Gate Website

DBMS Architecture



1. **Query Processor**: This is the component of a DBMS that interprets and executes user queries. It comprises several sub-components including:
    1. **DML Compiler**: Processes Data Manipulation Language (DML) statements into low-level instructions that can be executed.
    2. **DDL Interpreter**: Processes Data Definition Language (DDL) statements into metadata tables.
    3. **Embedded DML Pre-compiler**: Translates DML statements embedded in application programs into procedural calls.
    4. **Query Optimizer**: Determines the most efficient way to execute a query by evaluating different query plans.
2. **Storage Manager**: Also known as the Database Control System, it is responsible for managing the data stored in the database, ensuring its consistency and integrity. It includes the following sub-components:
    1. **Authorization Manager**: Manages access controls and privileges.
    2. **Integrity Manager**: Ensures that data modifications adhere to integrity constraints.
    3. **Transaction Manager**: Manages concurrent access to the database and maintains database consistency during transactions.
    4. **File Manager**: Manages file space and data structures representing information in the database.
    5. **Buffer Manager**: Manages data cache and data transfer between main memory and secondary storage.
3. **Disk Storage**: Represents the storage aspect of a DBMS, encompassing the following components:
    1. **Data Files**: Files where the actual data is stored.
    2. **Data Dictionary**: Repository containing information about the structure and characteristics of database objects.
    3. **Indices**: Data structures that facilitate faster data retrieval.

Knowledge Gate Website

A Database Management System (DBMS) consists of three primary components:

1. **Internal Level**: Concerns the physical storage of data in databases, overseeing data storage on hardware devices, and managing low-level aspects like data compression and indexing.

2. **Conceptual Level**: Represents the logical layout of the database, detailing the schema with tables and attributes and their interrelations. It's independent of specific DBMS implementations, focusing on organizing and connecting data elements.

3. **External Level**: Embodies the user interface of the database, facilitating data access and interaction through user-friendly views and interfaces tailored to various user groups.

Knowledge Gate Website

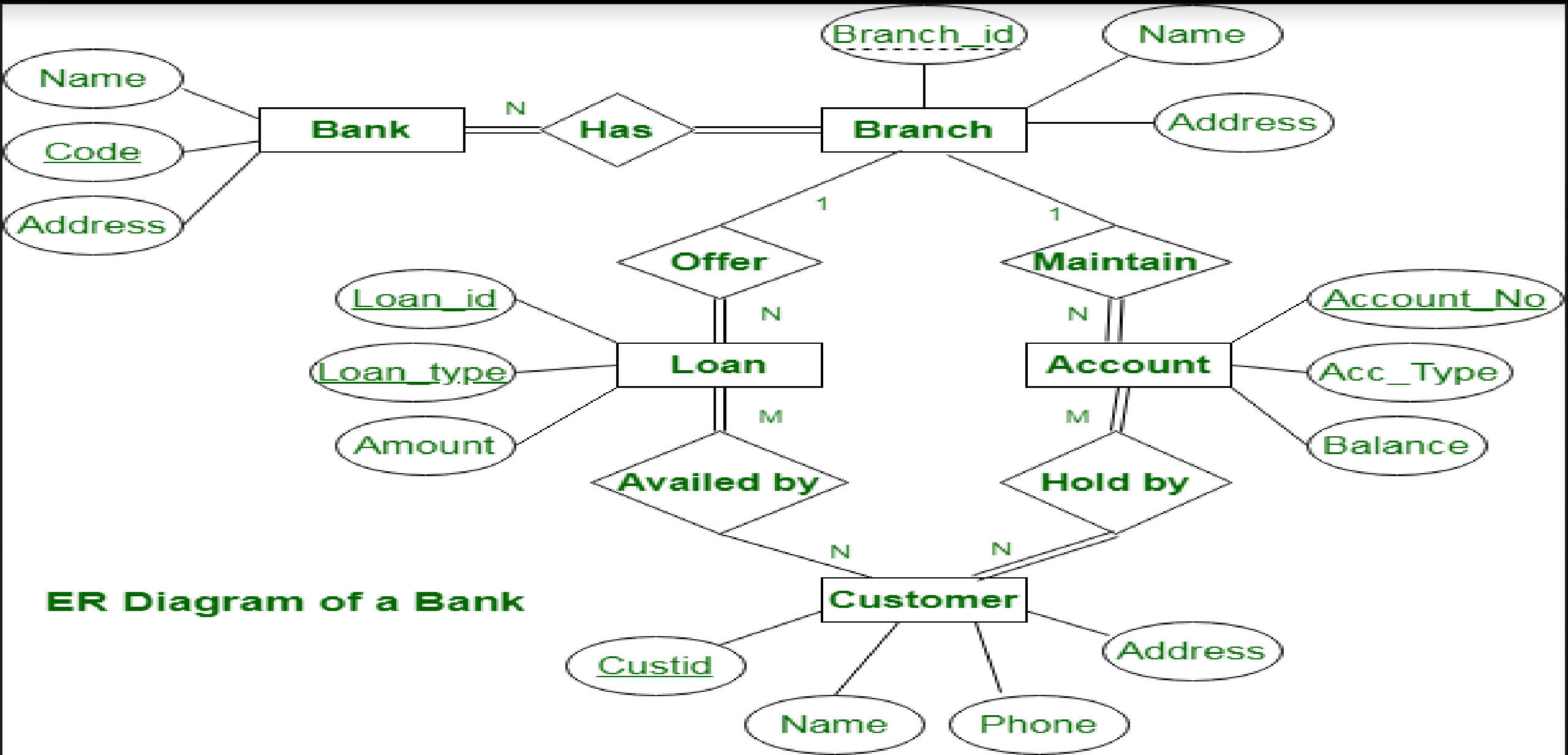
ER Diagram

- Developed by Dr. Peter Chen in 1976, this conceptual level method, grounded in real-world perceptions, facilitates diagrammatic data representation, simplifying comprehension for non-technical users.

- The E-R data model, central to database design, encapsulates entities and their attributes within an enterprise schema, serving as a clear, standardized tool for translating real-world enterprise interactions into a conceptual schema.

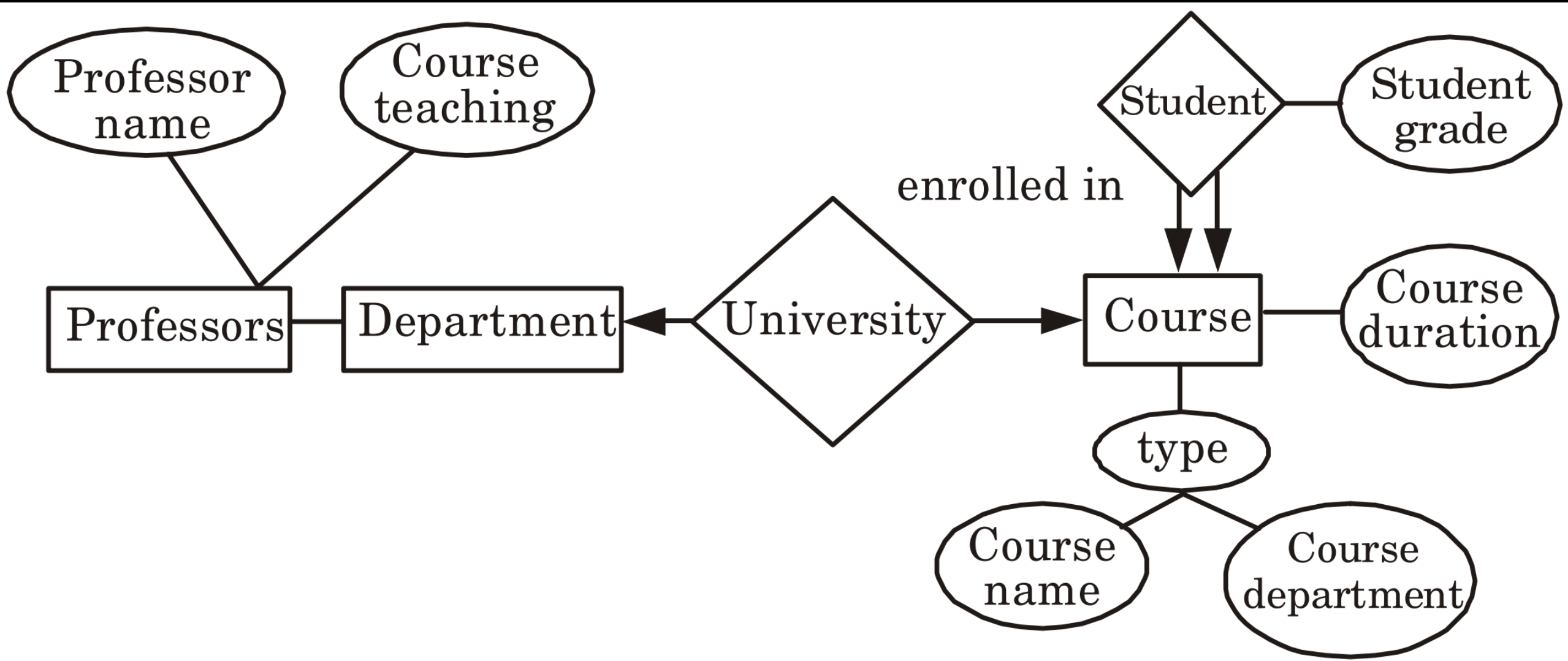
Knowledge Gate Website

ER diagram for bank

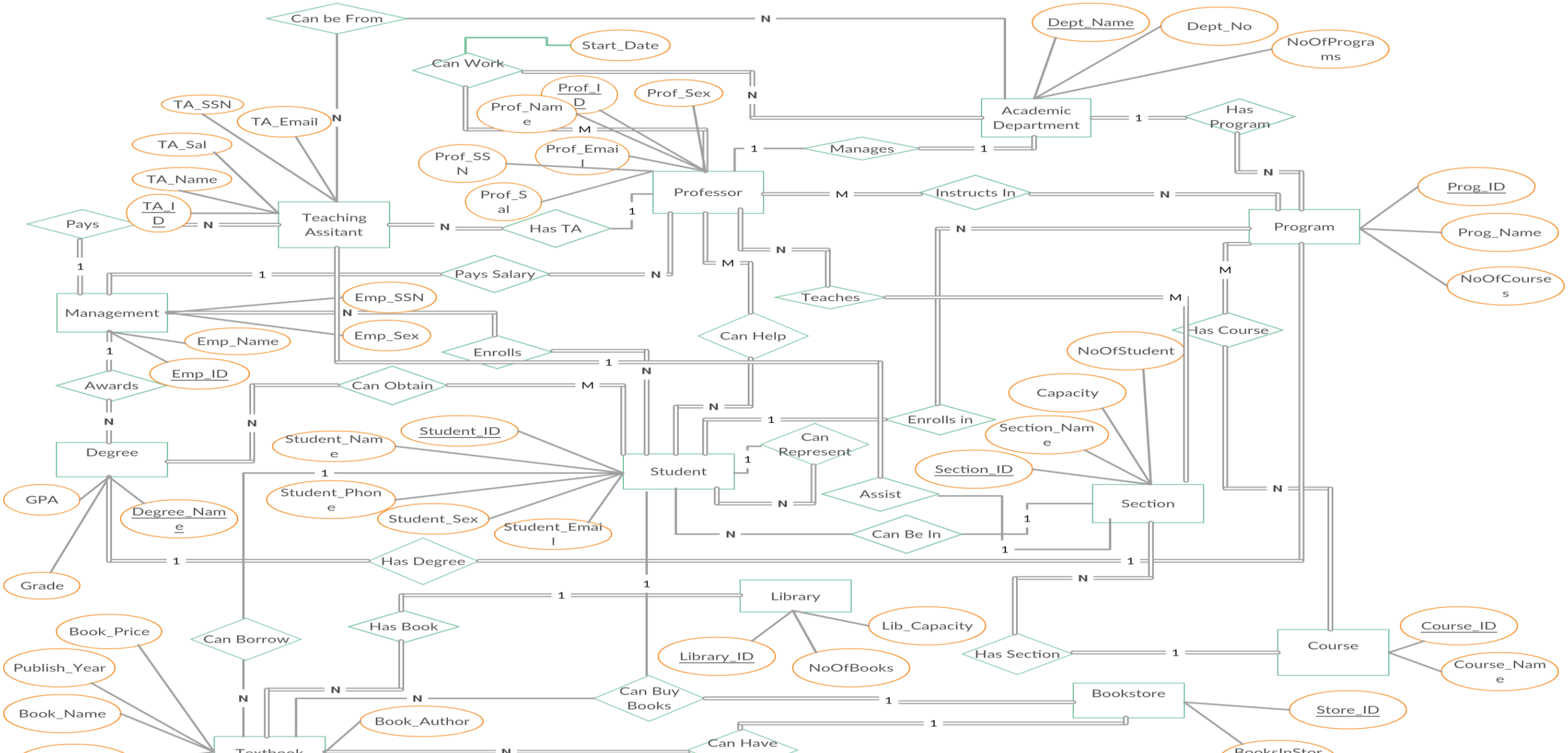


ER Diagram of a Bank

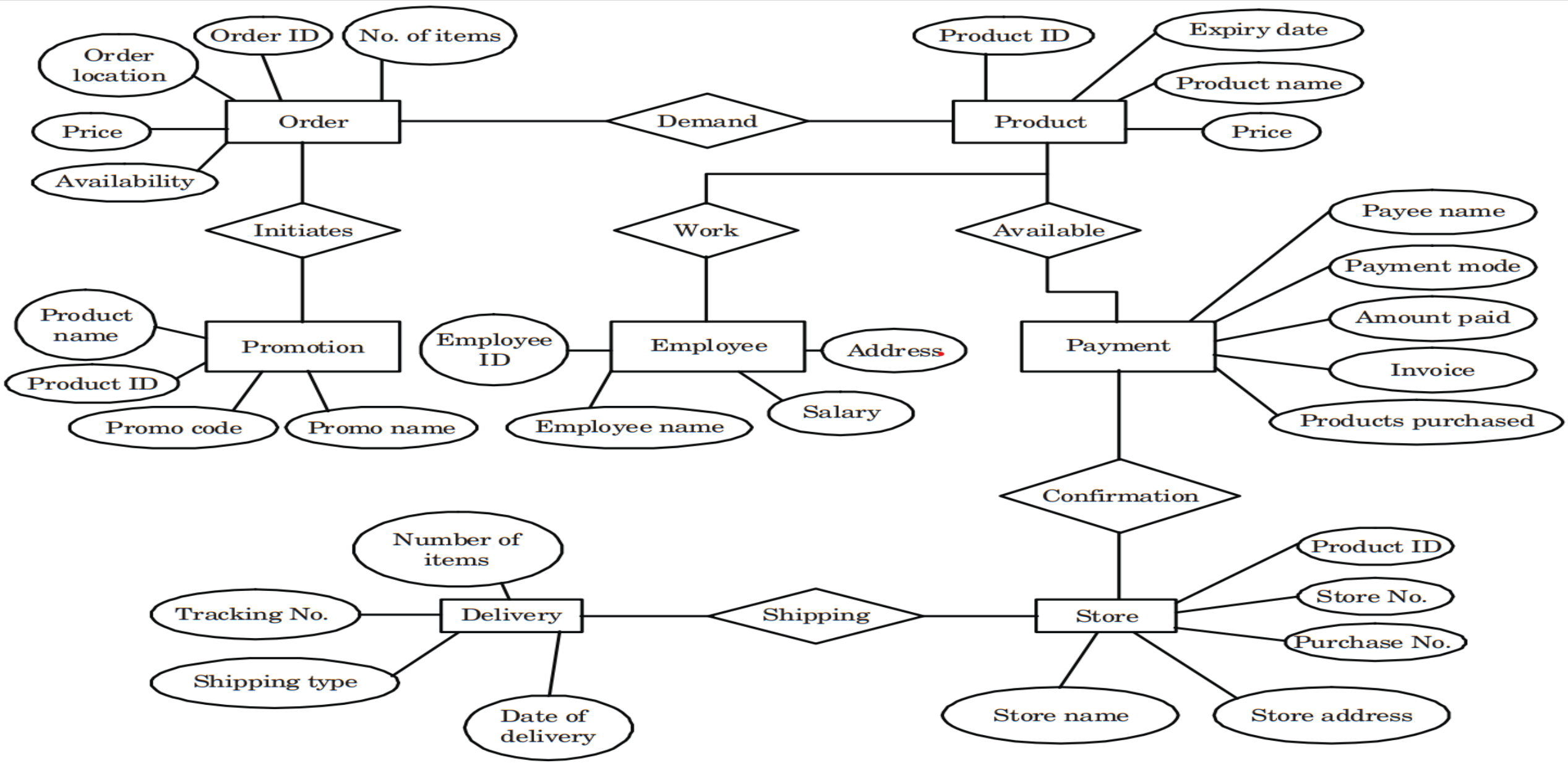
ER diagram for University System



ER diagram for University System



ER diagram for Marketing Company

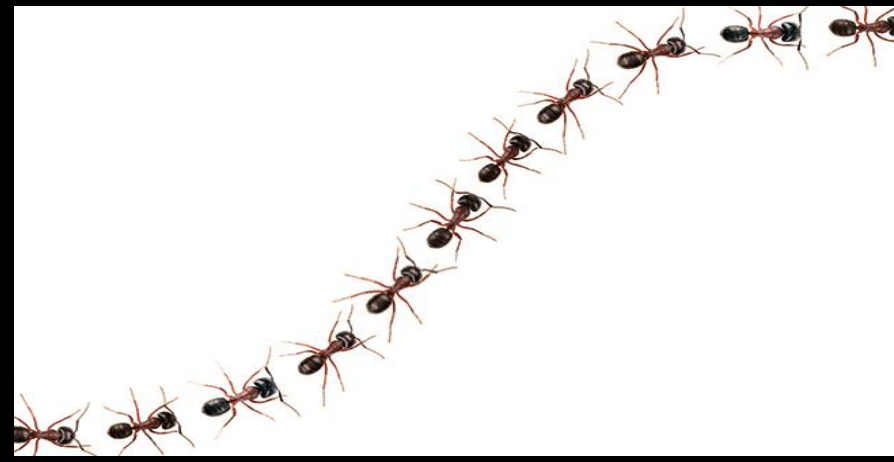


ENTITY

- An entity is a thing or an object in the real world that is distinguishable from other object based on the values of the attributes it possesses.

- An entity may be **concrete**, such as a person or a book, or it may be **abstract**, such as a course, a course offering, or a flight reservation.

Name	FName	City	Age	Salary
Smith	John	3	35	$280
Doe	Jane	1	28	$325
Brown	Scott	3	41	$265
Howard	Shemp	4	48	$359
Taylor	Tom	2	22	$250



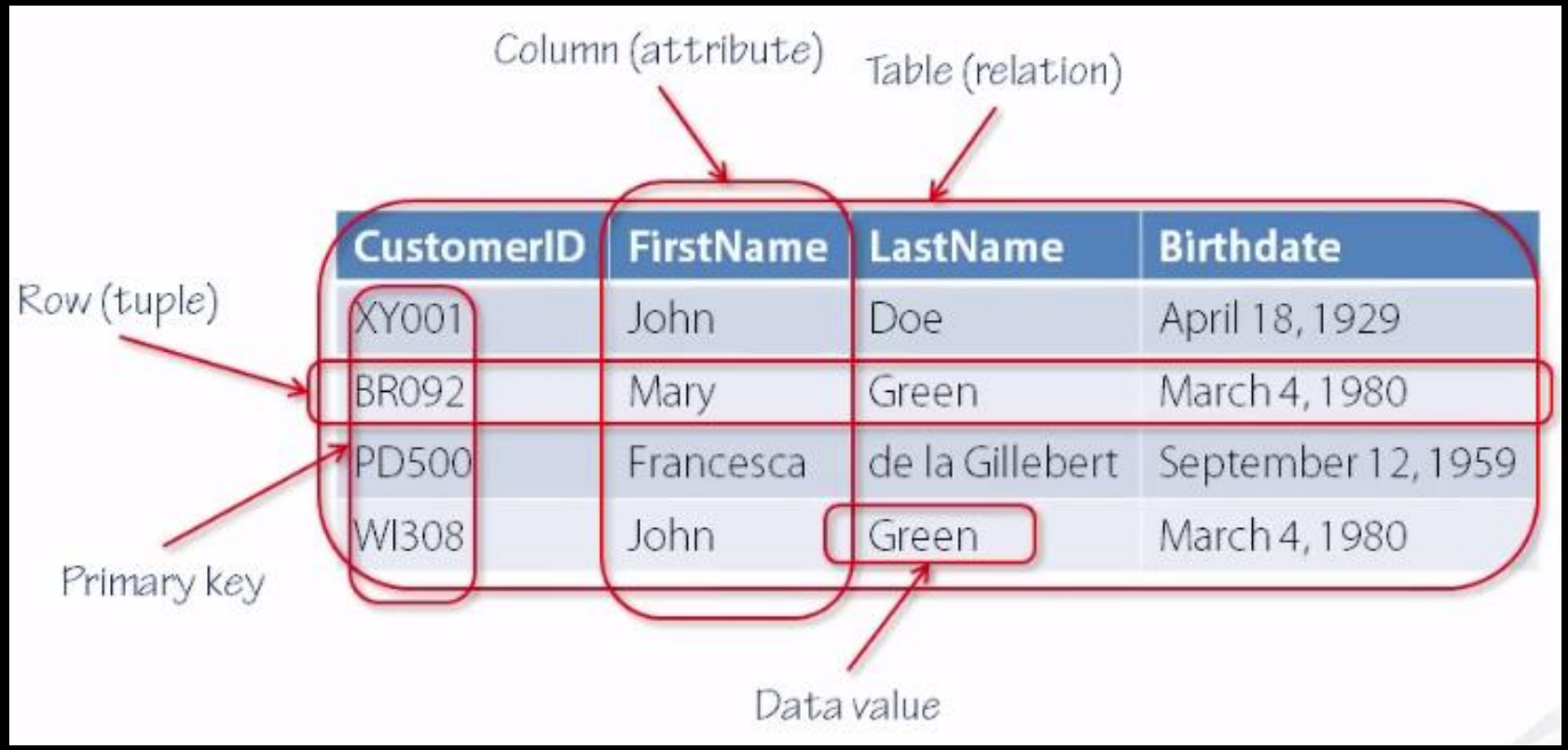
Knowledge Gate Website

- *Types of Entity*
  - **Tangible** - *Entities which physically exist in real world. E.g. - Car, Pen, locker*
  - **Intangible** - *Entities which exist logically. E.g. – Account, video.*

Knowledge Gate Website

- In ER diagram we cannot represent an entity, as entity is an instant not schema, and ER diagram is designed to understand schema

- In a relational model entity is represented by a row or a tuple or a record in a table.



Knowledge Gate Website

- **ENTIY SET-** Collection of same type of entities that share the same properties or attributes.

- In an ER diagram an entity set is represented by a rectangle

- In a relational model it is represented by a separate table

Name	FName	City	Age	Salary
Smith	John	3	35	$280
Doe	Jane	1	28	$325
Brown	Scott	3	41	$265
Howard	Shemp	4	48	$359
Taylor	Tom	2	22	$250

Knowledge Gate Website

ATTRIBUTES

- Attributes are the units defines and describe properties and characteristics of entities.

- Attributes are the descriptive properties possessed by each member of an entity set. for each attribute there is a set of permitted values called domain.

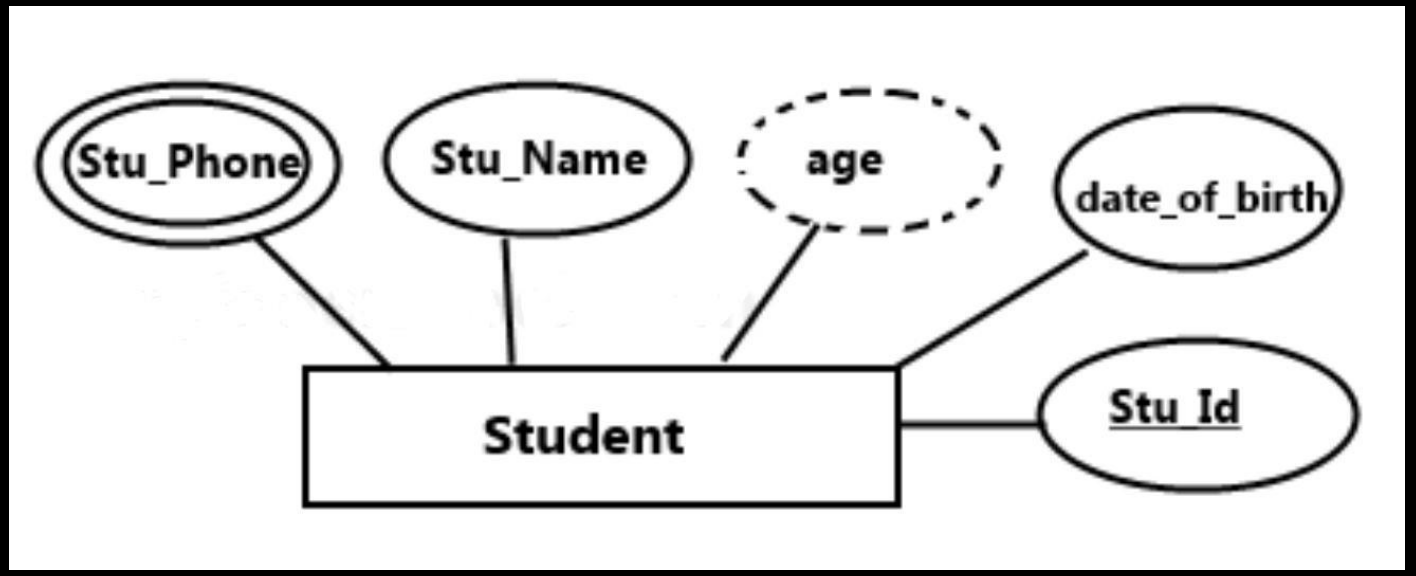
Name	FName	City	Age	Salary
Smith	John	3	35	$280
Doe	Jane	1	28	$325
Brown	Scott	3	41	$265
Howard	Shemp	4	48	$359
Taylor	Tom	2	22	$250

Knowledge Gate Website

- In an ER diagram attributes are represented by ellipse or oval connected to rectangle.

- While in a relational model they are represented by independent column. e.g. Instructor (ID, name, salary, dept_name)

Name	FName	City	Age	Salary
Smith	John	3	35	$280
Doe	Jane	1	28	$325
Brown	Scott	3	41	$265
Howard	Shemp	4	48	$359
Taylor	Tom	2	22	$250

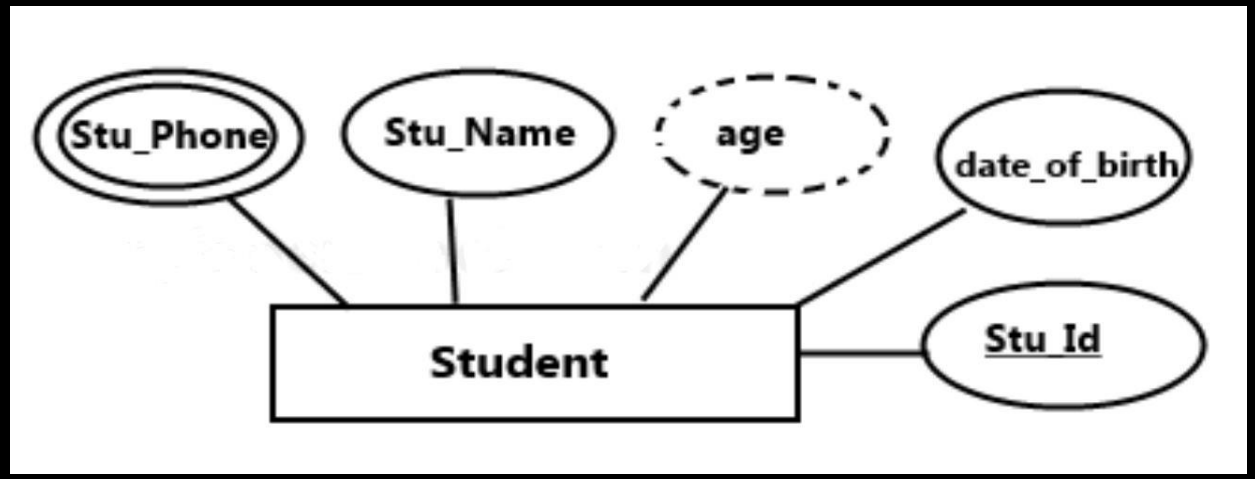


[Knowledge Gate Website]

Types of Attributes

- **Single valued**- Attributes having single value at any instance of time for an entity. E.g. – Aadhar no, dob.

- **Multivalued** - Attributes which can have more than one value for an entity at same time. E.g. - Phone no, email, address.
  - A multivalued attribute is represented by a double ellipse in an ER diagram and by an independent table in a relational model.
  - Separate table for each multivalued attribute, by taking mva and pk of main table as fk in new table



Knowledge Gate Website

Customer			
Customer ID	**First Name**	**Surname**	**Telephone Number**
123	Rabri Devi	Singh	555-861-2025, 192-122-1111
456	Imarti Devi	Zhang	(555) 403-1659 Ext. 53; 182-929-2929
789	Barfi Devi	Doe	555-808-9633

Customer			
Customer ID	**First Name**	**Surname**	**Telephone Number**
123	Pooja	Singh	555-861-2025, 192-122-1111
456	San	Zhang	(555) 403-1659 Ext. 53; 182-929-2929
789	John	Doe	555-808-9633

जुगाड़ technology

Customer				
Customer ID	**First Name**	**Surname**	**Telephone Number1**	**Telephone Number2**
123	Pooja	Singh	555-861-2025	192-122-1111
456	San	Zhang	(555) 403-1659 Ext. 53	182-929-2929
789	John	Doe	555-808-9633	

Knowledge Gate Website

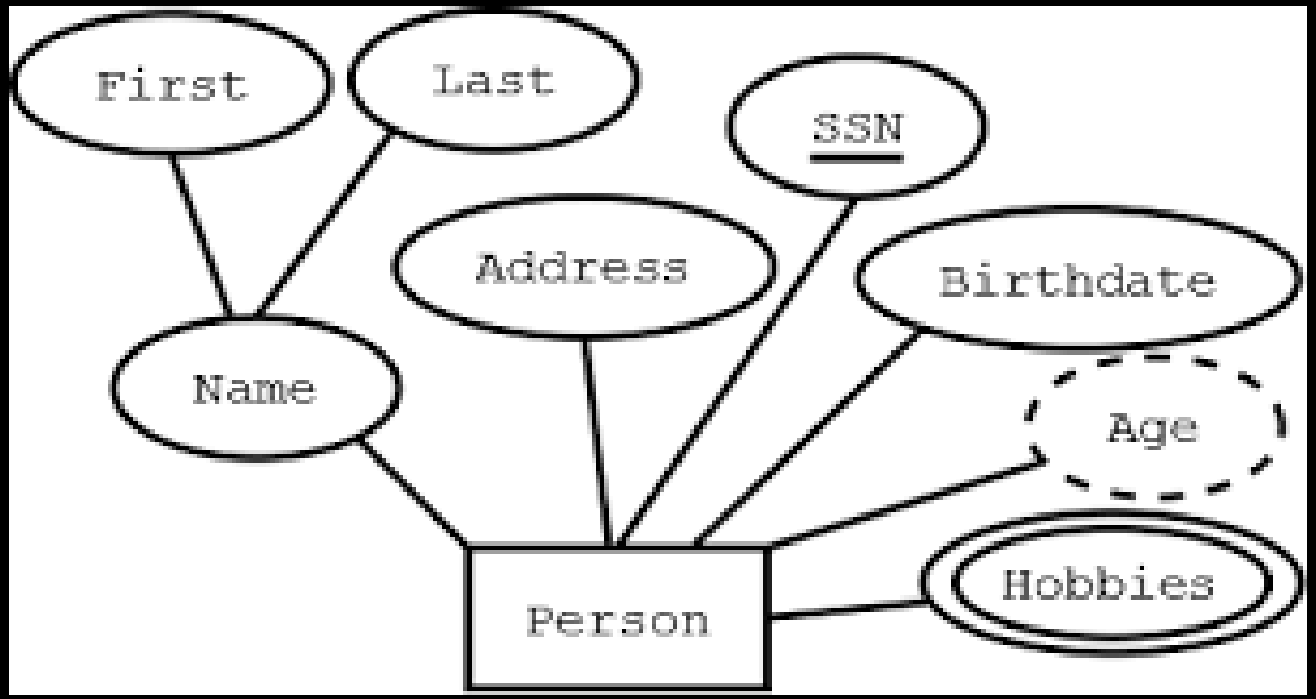
Customer			
Customer ID	**First Name**	**Surname**	**Telephone Number**
123	Rabri Devi	Singh	555-861-2025, 192-122-1111
456	Imarti Devi	Zhang	(555) 403-1659 Ext. 53; 182-929-2929
789	Barfi Devi	Doe	555-808-9633

Customer Name		
Customer ID	**First Name**	**Surname**
123	Pooja	Singh
456	San	Zhang
789	John	Doe

Customer Phone Number	
Customer ID	**Telephone Number**
123	555-861-2025
123	192-122-1111
456	(555) 403-1659 Ext. 53
456	182-929-2929
789	555-808-9633

Knowledge Gate Website

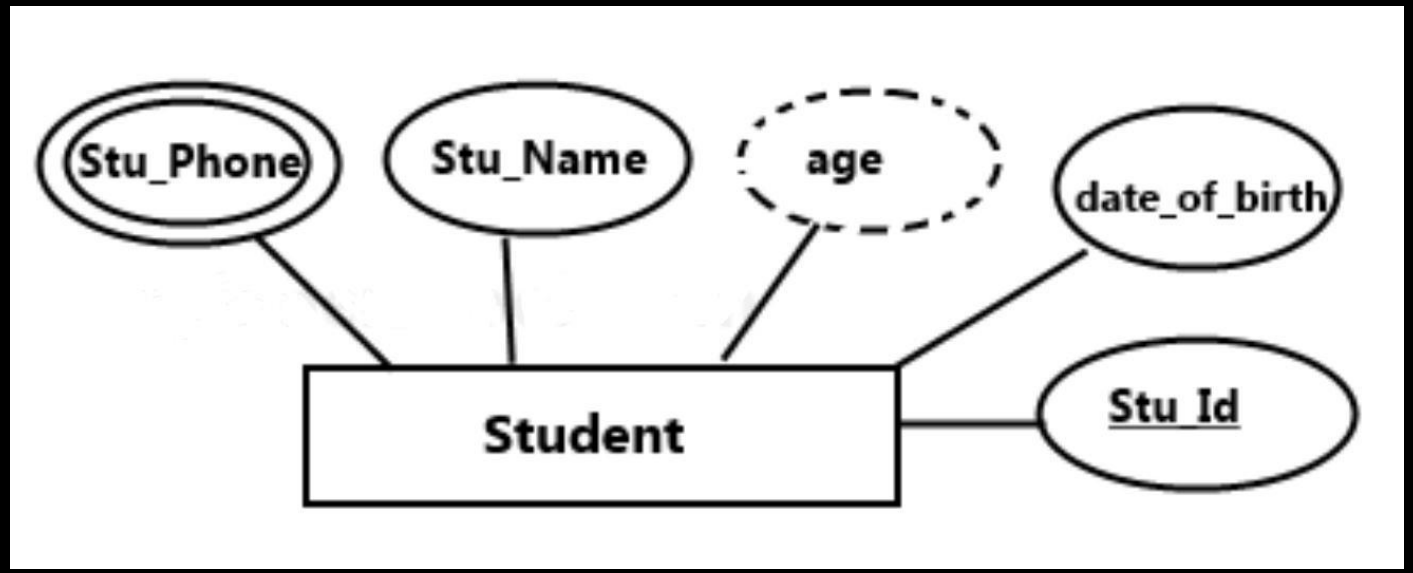
- **Simple** - Attributes which cannot be divided further into sub parts. E.g. Age
- **Composite** - Attributes which can be further divided into sub parts, as simple attributes. A composite attribute is represented by an ellipse connected to an ellipse and in a relational model by a separate column.



Knowledge Gate Website

- **Stored** - Main attributes whose value is permanently stored in database. E.g. date_of_birth

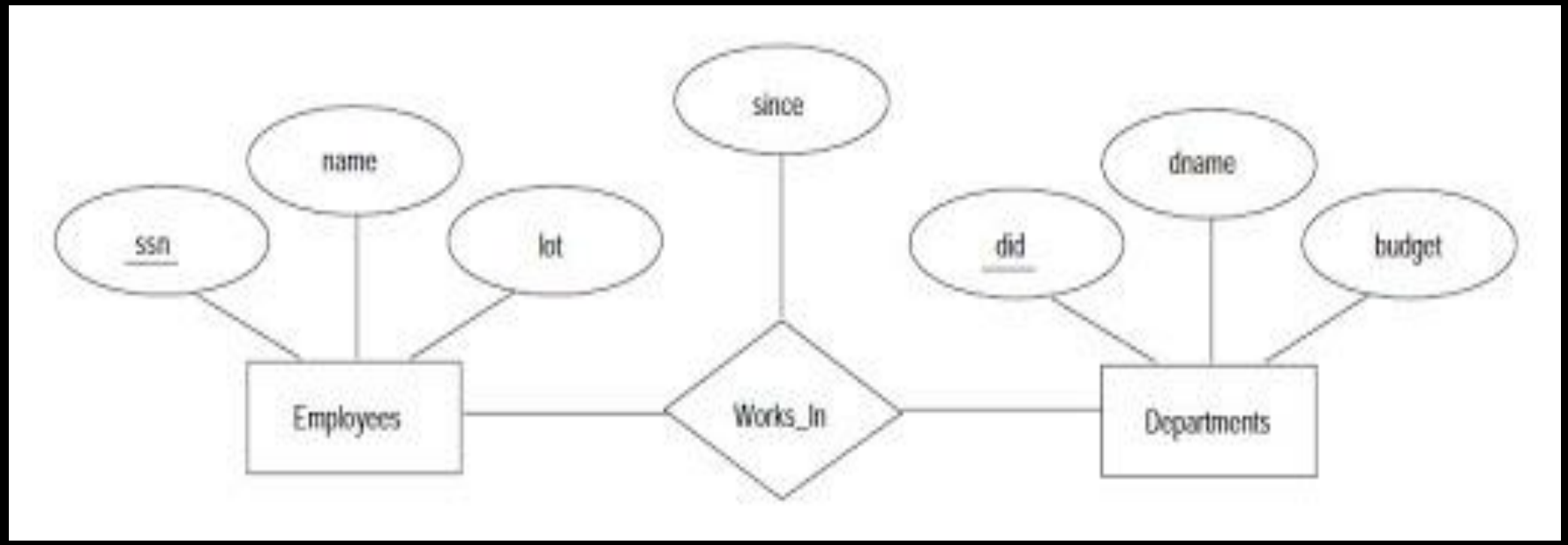
- **Derived** -The value of these types of attributes can be derived from values of other Attributes. E.g. - Age attribute can be derived from date_of_birth and Date attribute.



Knowledge Gate Website

Descriptive attribute - Attribute of relationship is called descriptive attribute.

- An attribute takes a null value when an entity does not have a value for it. The null value may indicate “not applicable”— that is, that the value does not exist for the entity.



Knowledge Gate Website

Relationship / Association

Knowledge Gate Website

Relationship / Association

- Is an association between two or more entities of same or different entity set.

- In ER diagram we cannot represent individual relationship as it is an instance or data.

- In an ER diagram it is represented by a diamond, while in relational model sometimes through foreign key and other time by a separate table.

Roll no	name	Age	Br_code	Br_name	Br_hod_name
1	A	19	101	Cs	Abc
2	B	18	101	Cs	Abc
3	C	20	101	Cs	Abc
4	D	20	102	Ec	Pqr

PK FK PK

Roll no	name	Age	Br_code
1	A	19	101
2	B	18	101
3	C	20	101
4	D	20	102

Br_code	Br_name	Br_hod_name
101	Cs	Abc
102	ec	Pqr

Knowledge Gate Website

- Every relationship type has three components.
  - Name
  - Degree
  - Structural constraints (cardinalities ratios, participation)

Knowledge Gate Website

- **NAME**- every relation must have a unique name.

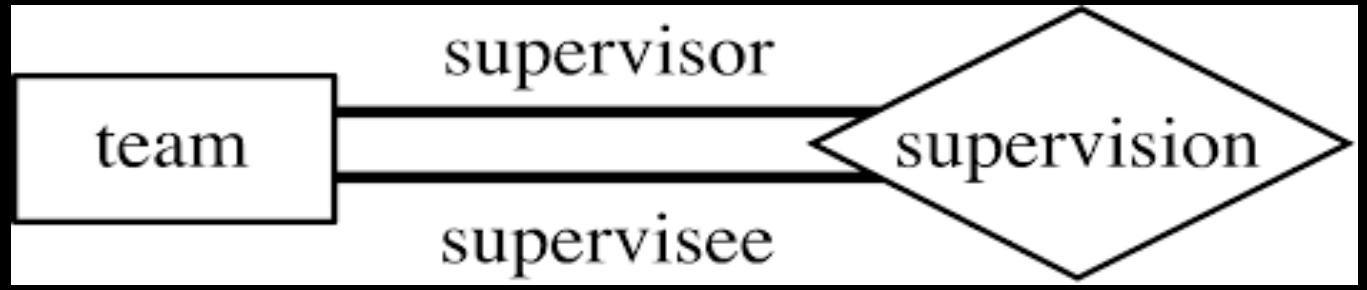
Degree of a relationship/relationship set

- Means number of entities set(relations/tables) associated(participate) in the relationship set.
- Most of the relationship sets in a data base system are binary.
- Occasionally however relationship sets involve more than two entity sets.
- Logically, we can associate any number of entity set in a relationship called N-ary Relationship.

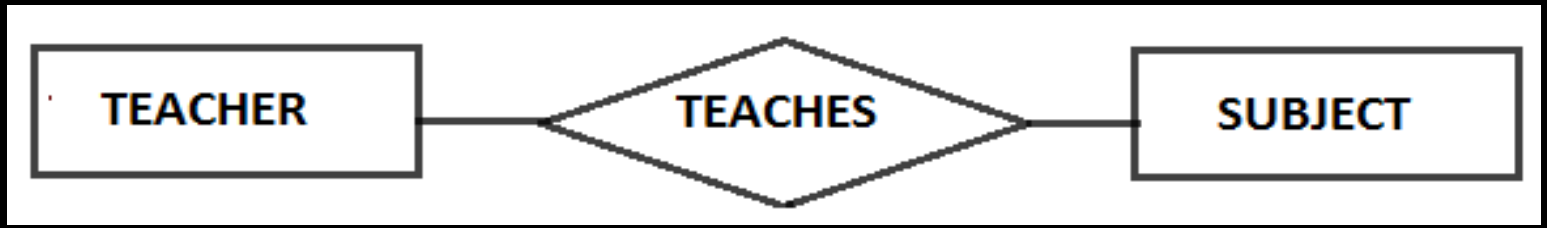
- **Unary Relationship** - One single entity set participate in a Relationship, means two entities of the same entity set are related to each other.

- These are also called as self -referential Relationship set.

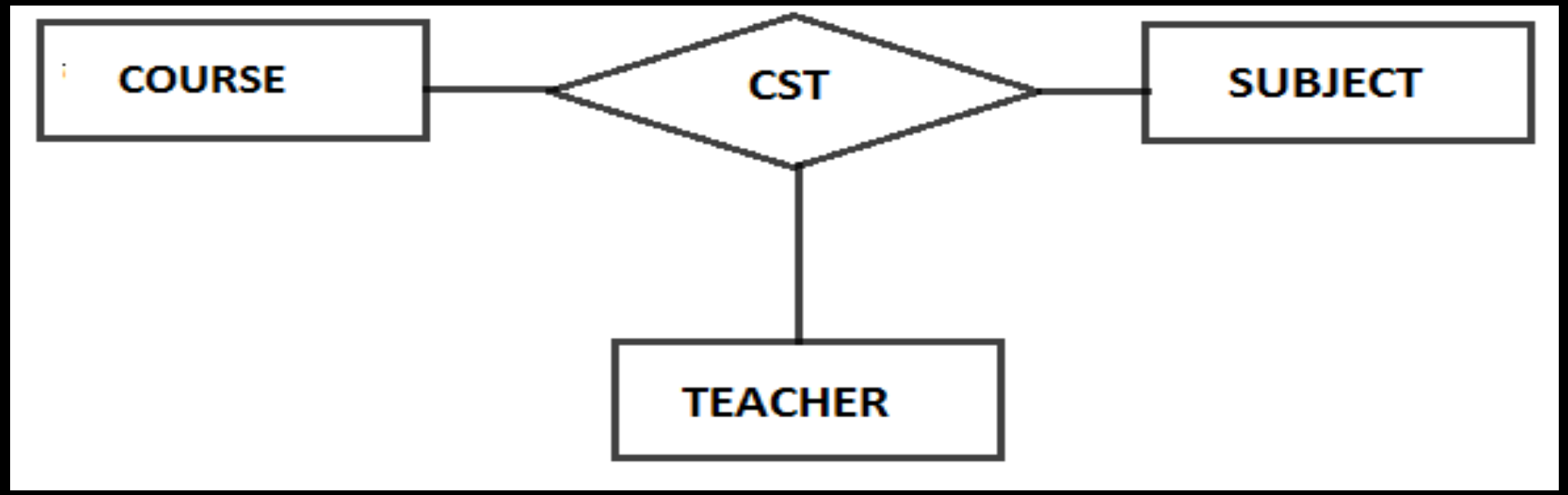
- E.g.- A member in a team maybe supervisor of another member in team.



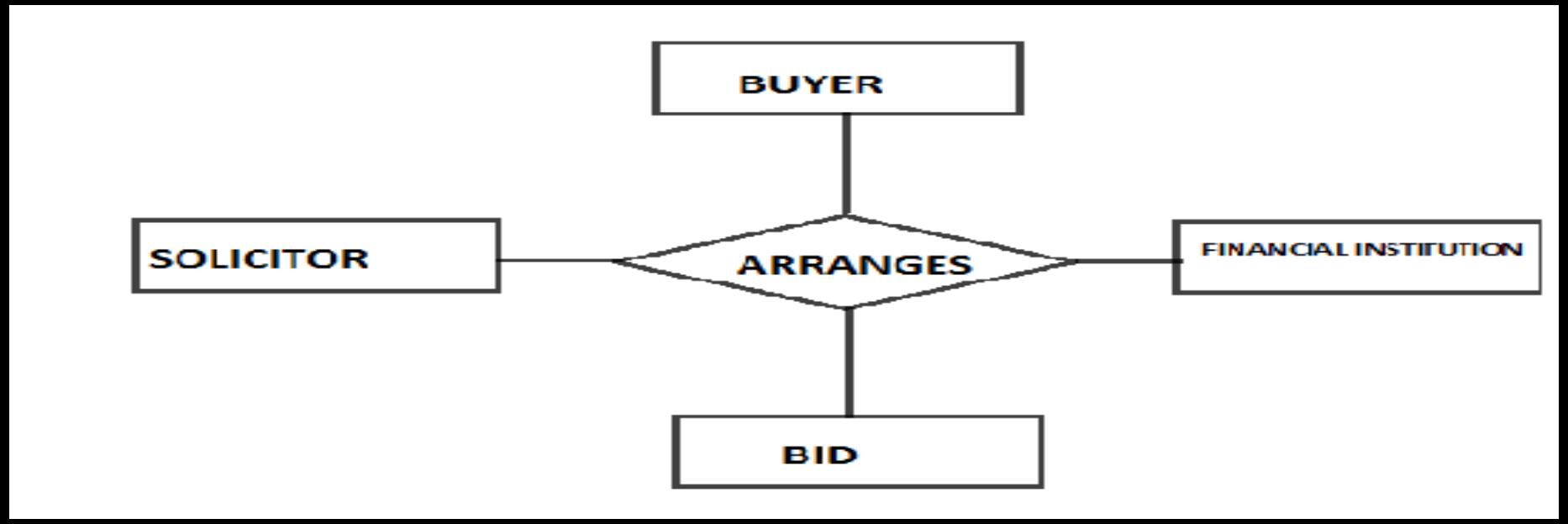
- **Binary Relationship** - Two entity sets participate in a Relationship. It is most common Relationship.



- **Ternary Relationship** - When three entities participate in a Relationship. E.g. The University might need to record which teachers taught which subjects in which courses.

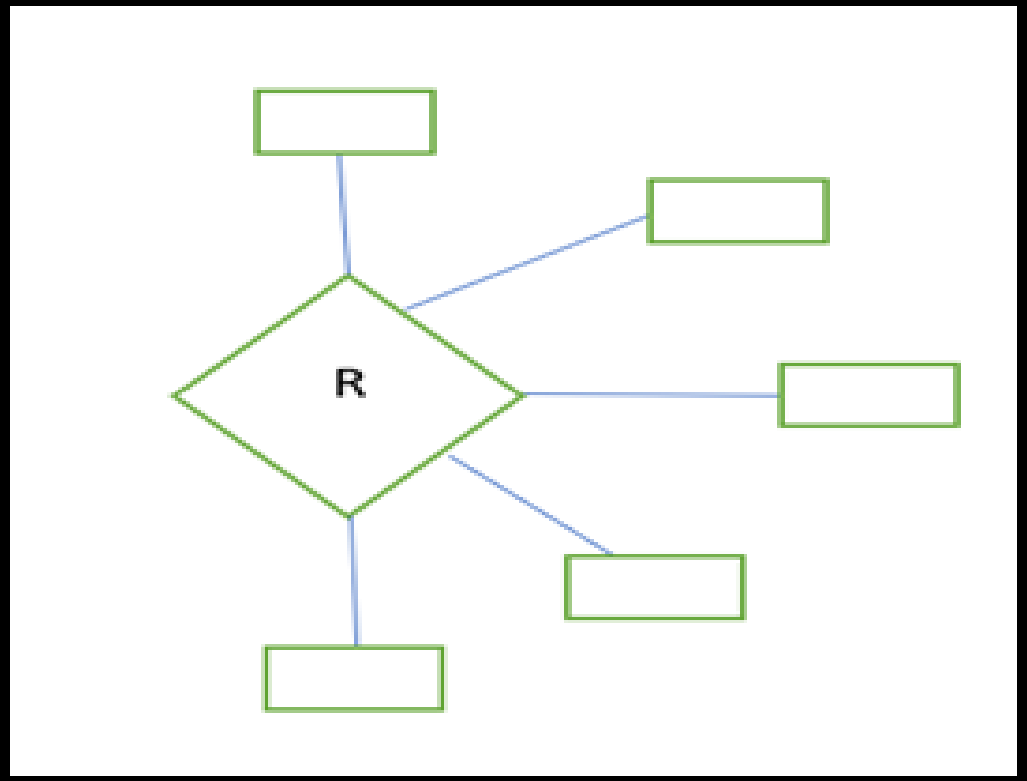


- **Quaternary Relationship** - When four entities participate in a Relationship.



Knowledge Gate Website

- **N-ary relationship** – where n number of entity set are associated



- But the most common relationships in ER models are ***Binary.***

Knowledge Gate Website

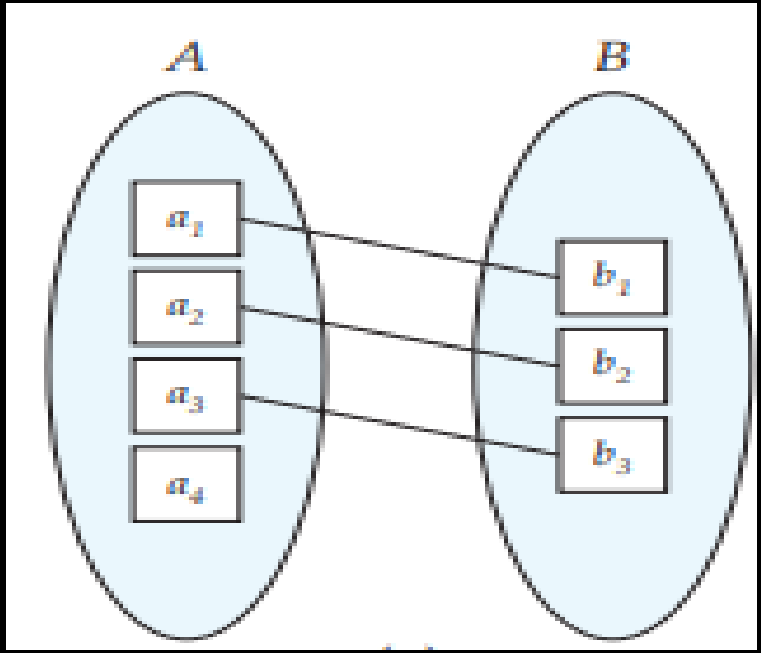
Structural constraints (Cardinalities Ratios, Participation)

- An E-R enterprise schema may define certain constraints to which the contents of a database must conform.

MAPPING CARDINALITIES / CARNINALITY RATIOS

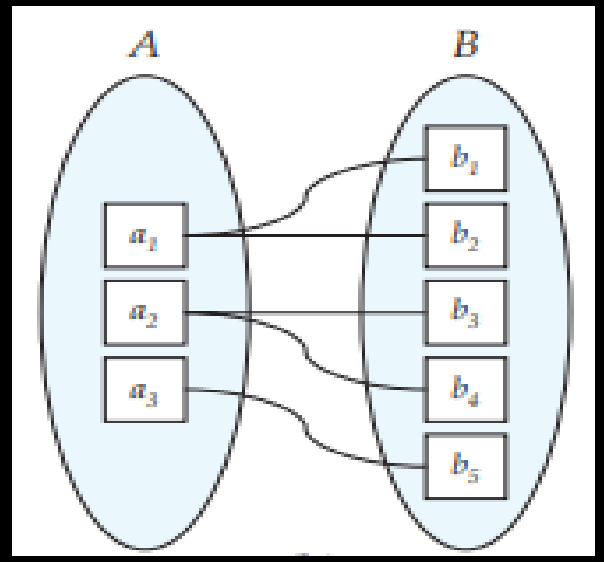
- Express the number of entities to which another entity can be associated via a relationship set. Four possible categories are-
  - One to One (1:1) Relationship.
  - One to Many (1: M) Relationship.
  - Many to One (M: 1) Relationship.
  - Many to Many (M: N) Relationship.

<u>One to One (1:1) Relationship</u> - An entity in A is associated with at most one entity in B, and an entity in B is associated with at most one entity in A.



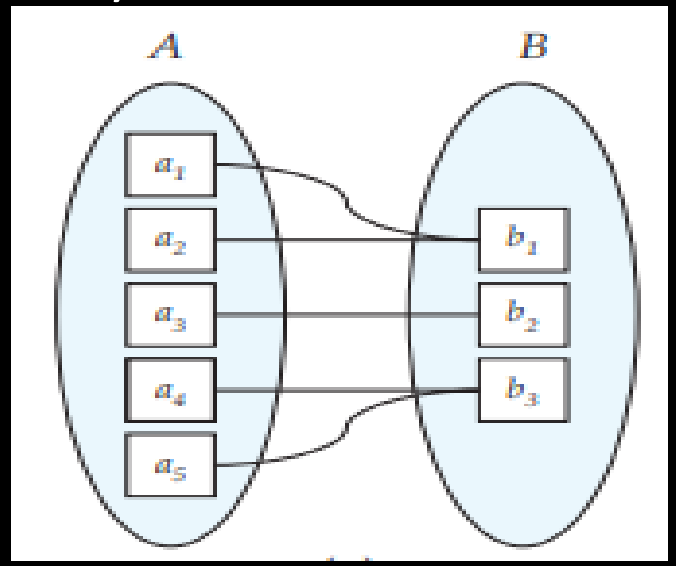
E.g.- The directed line from relationship set advisor to both entities set indicates that ‘an instructor may advise at most one student, and a student may have at most one advisor’.

One to Many (1: M) Relationship - An entity in A is associated with any number (zero or more) of entities in B. An entity in B, however, can be associated with at most one entity in A.



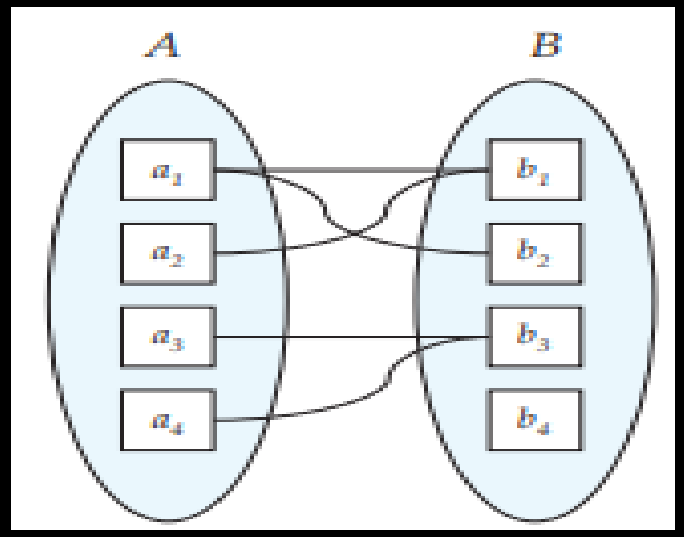
E.g.- This indicates that an instructor may advise many students, but a student may have at most one advisor.

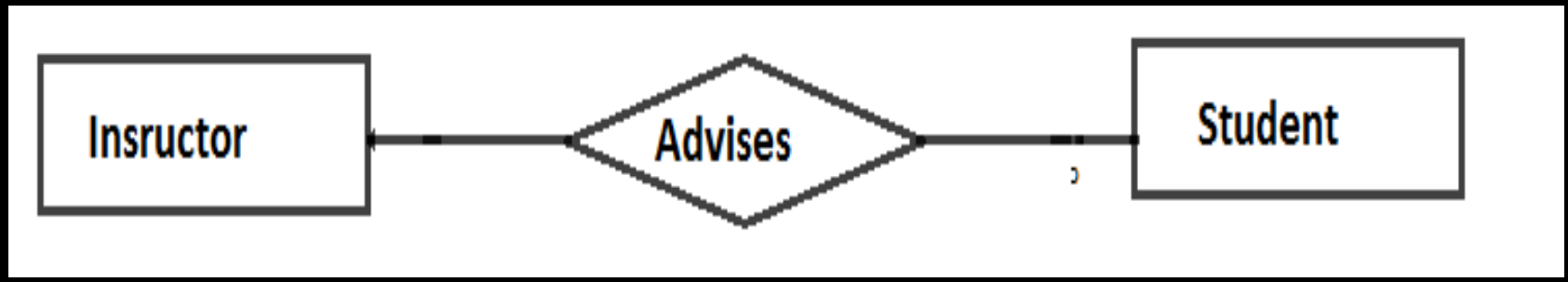
Many to One (M: 1) Relationship - An entity in A is associated with at most one entity in B. An entity in B, however, can be associated with any number (zero or more) of entities in A.



E.g.- This indicates that student may have many instructors but an instructor can advise at most one student.

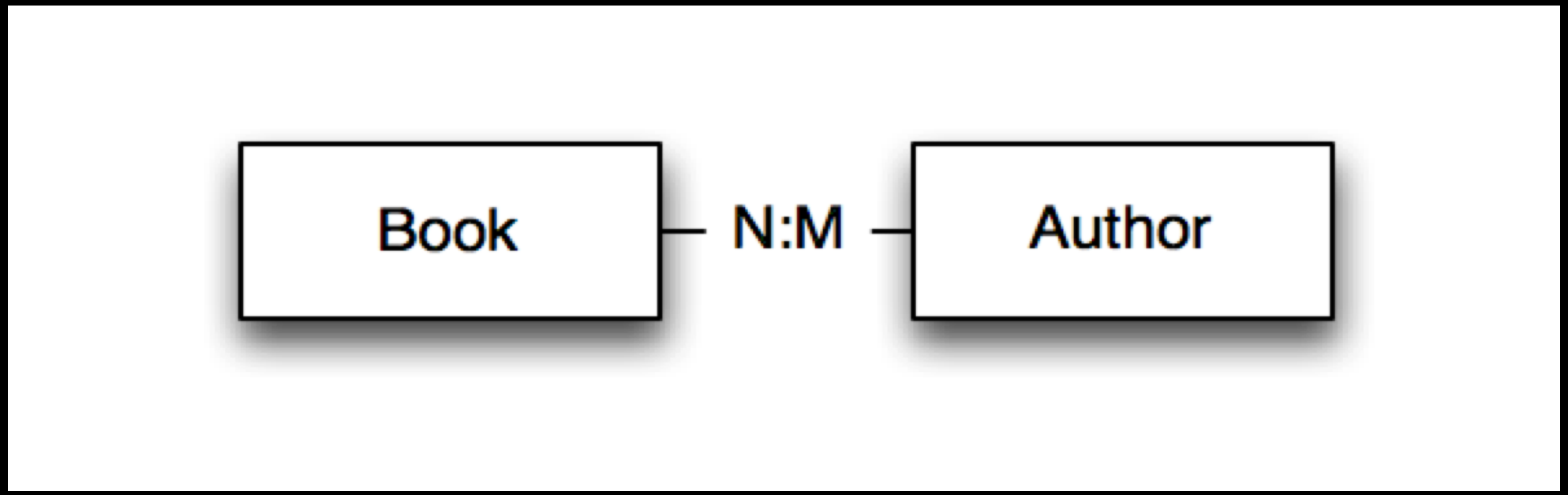
Many to Many(M:N) Relationship - An entity in A is associated with any number (zero or more) of entities in B, and an entity in B is associated with any number (zero or more) of entities in A.





E.g.- This indicates a student may have many advisors and an instructor may advise many students.

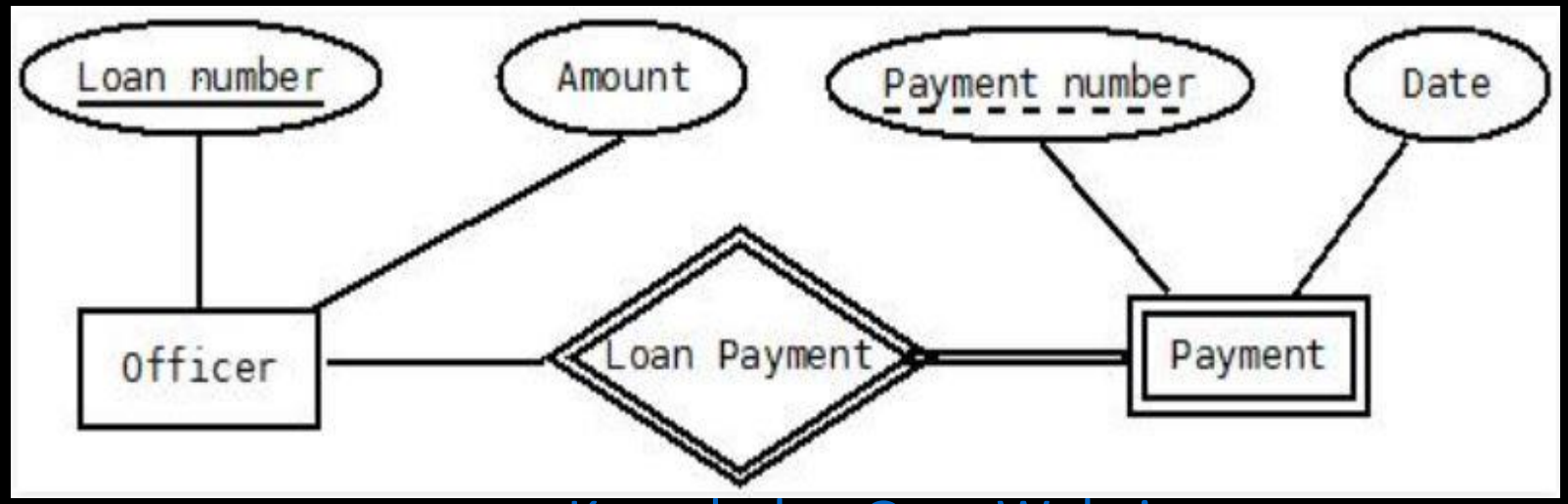
- **PARTICIPATION CONSTRAINTS**- it defines participations of entities of an entity type in a relationship.
  - **Partial participation**
  - **Total Participation**



Knowledge Gate Website

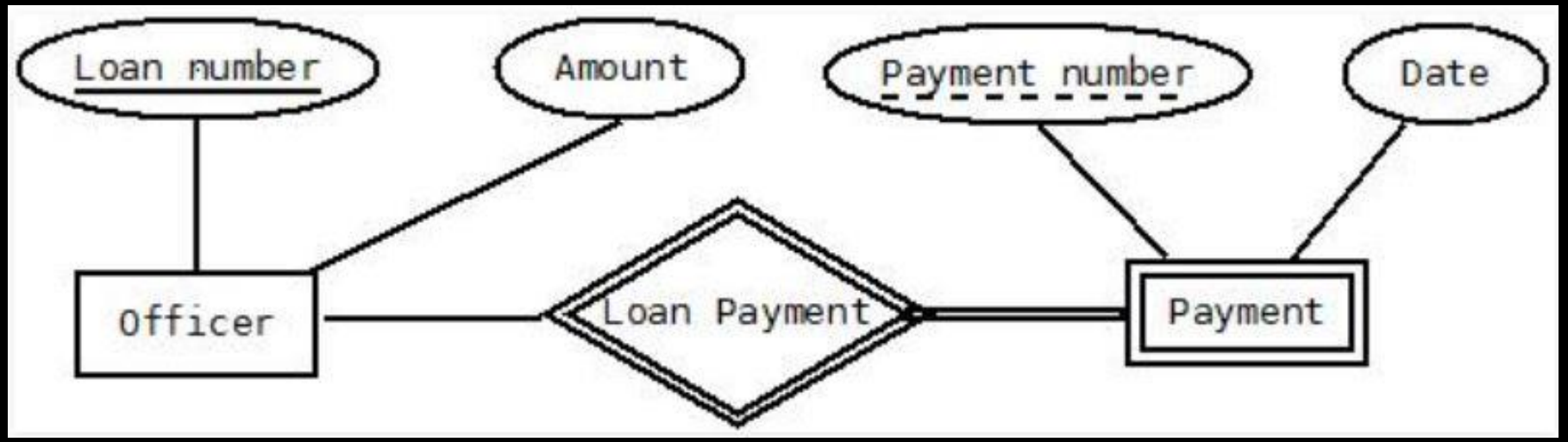
STRONG AND WEAK ENTITY SET

- An entity set is called strong entity set, if it has a primary key, all the tuples in the set are distinguishable by that key.
- An entity set that does not process sufficient attributes to form a primary key is called a weak entity set.
- It contains discriminator attributes (partial key) which contain partial information about the entity set, but it is not sufficient enough to identify each tuple uniquely. Represented by double rectangle.

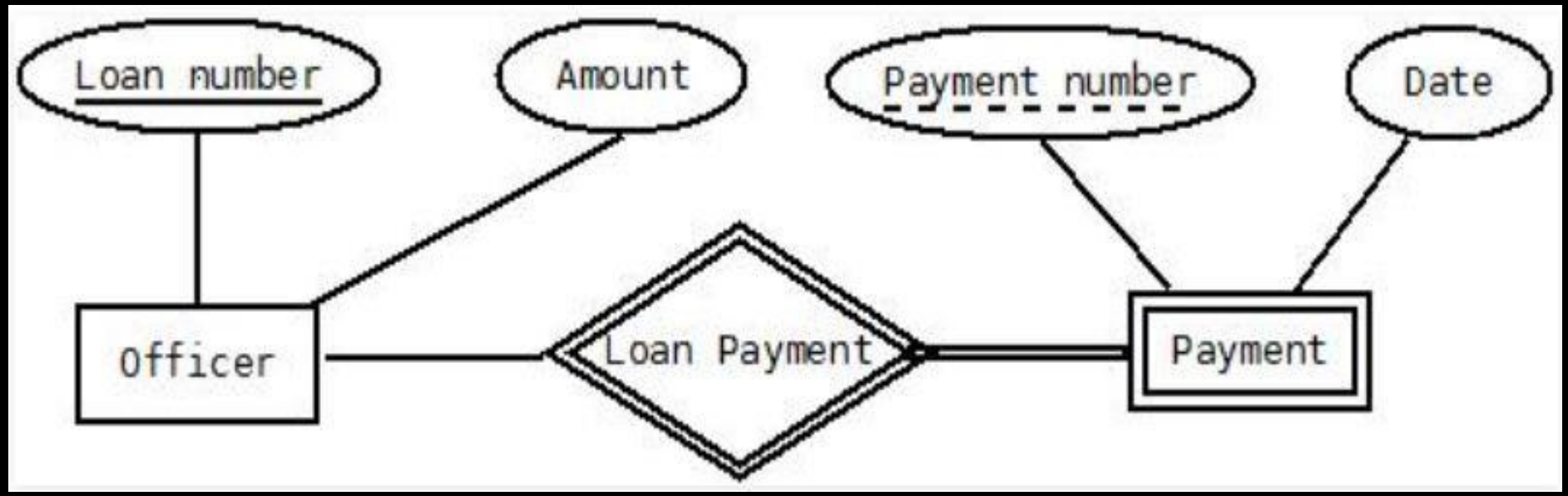


Knowledge Gate Website

- For a weak entity set to be meaningful and converted into strong entity set, it must be associated with another strong entity set called the **identifying or owner entity set** i.e. weak entity set is said to be **existence dependent** on the identity set.
- The identifying entity set is said to own weak entity set that it identifies.
- A weak entity set may participate as owner in an identifying relationship with another weak entity set.



- The relationship associating the weak entity set with the identifying entity set is called the **identifying relationship (double diamonds)**.
- The identifying relationship is many to one from the weak entity set to identifying entity set, and the participation of the weak entity set in relationship is always total.
- The primary key of weak entity set will be the union of primary key and discriminator attributes.



Knowledge Gate Website

REASONS TO HAVE WEAK ENTITY SET

- Weak entities reflect the logical structure of an entity being dependent on another.
- Weak entity can be deleted automatically when their strong entity is deleted.
- Without weak entity set it will lead to duplication and consequent possible inconsistencies.

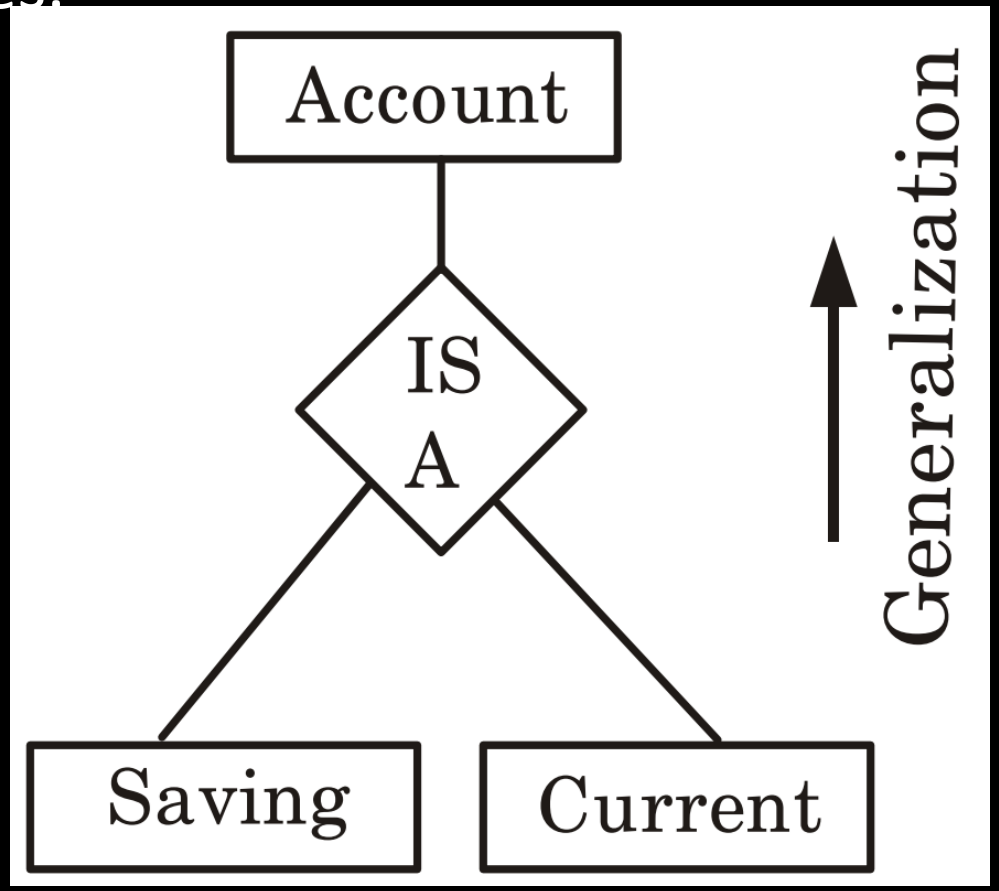
Conversion From ER Diagram To Relational Model

- Entity Set
  - Convert every strong, weak entity set into a separate table. In weak entity set we make it dependent onto one strong entity set **(identifying or owner entity set)**.
- Relationship
  - If Unary: No separate table is required, add a new column as fk which refer the pk of the same table.
  - if 1:1 No separate table is required, take pk of one side and put it as fk on other side, priority must be given to the side having total participation.
  - if 1:n or n:1 No separate table is required, modify n side by taking pk of 1 side a foreign key on n side.
  - If m-n Separate table is required take pk of both table and declare their combination as a pk of new table
  - (3 or More) Take the pk of all participating entity sets as fk and declare their combinations as pk in the new table.
- Attributes
  - Multivalued-A separate table must be taken for all multivalued attributes, where we take pk of the main table as fk and declare combination of fk and multivalued attribute are pk in the new table.
  - Composite Attributes-A separate column must be taken for all simple attributes of the composite attribute.

Knowledge Gate Website

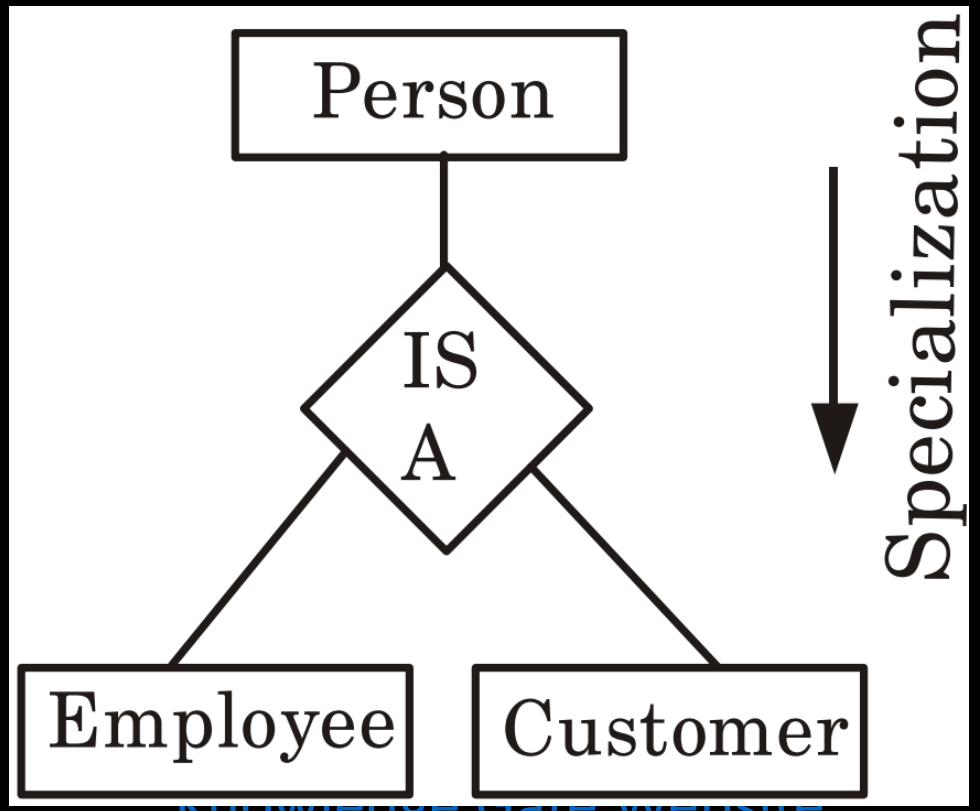
Generalization

- Involves merging two lower-level entities to create a higher-level entity.
- A bottom-up approach that builds complexity from simpler components.
- Highlights similarities among lower-level entity sets while hiding differences.
- Leads to a simplified, structured data representation, aiding in database design and querying processes.



Specialization

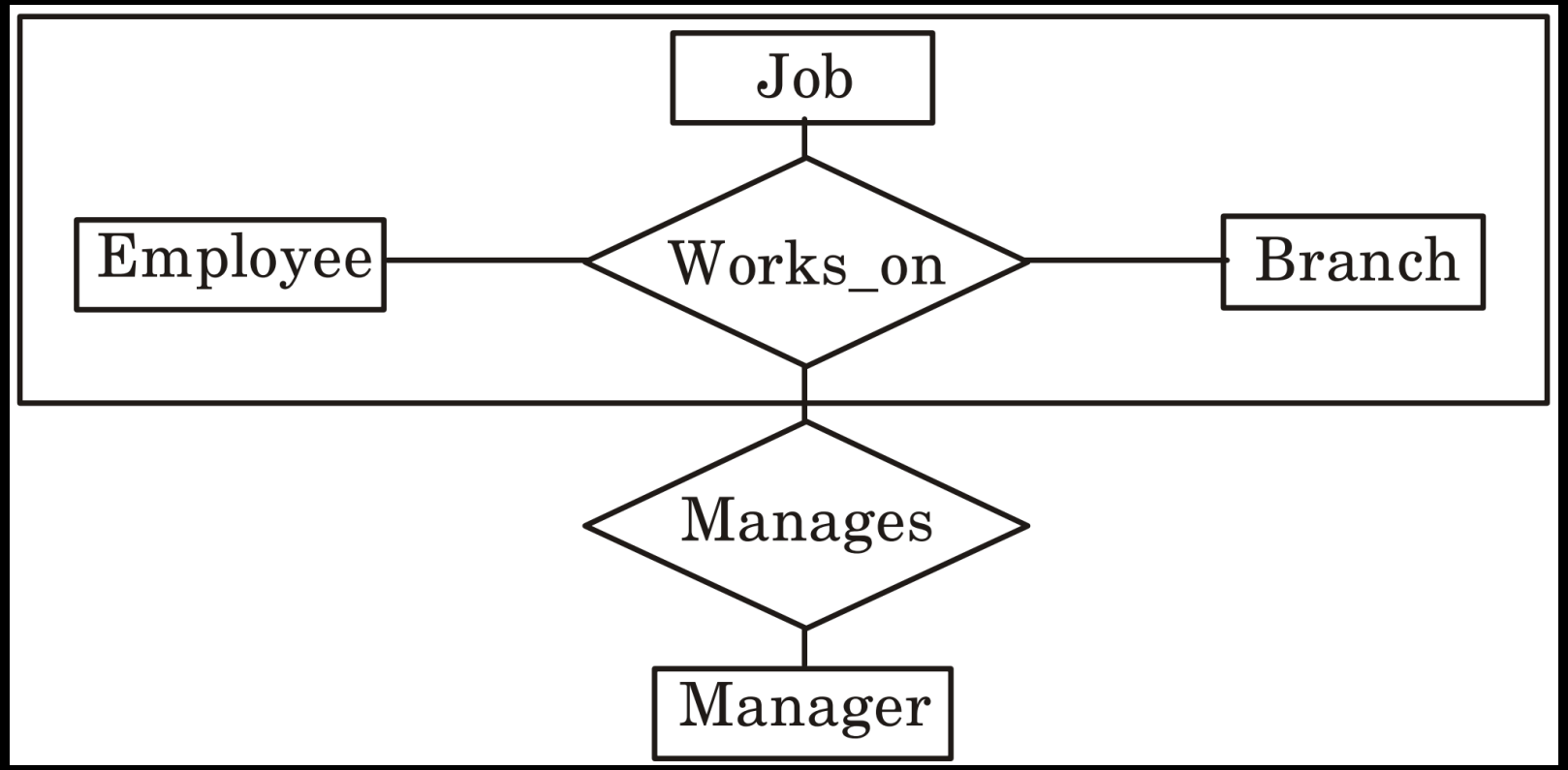
- A process where a higher-level entity is broken down into more specific, lower-level entities.
- This top-down approach delineates complexity into simpler components.
- Acts as the converse of the generalization process, focusing on differentiating properties rather than similarities.



Knowledge Gate Website

Aggregation

- A concept wherein relationships are abstracted to form higher-level entities, enabling a more organized representation of complex relationships.



Knowledge Gate Website

ADVANTAGES OF E-R DIGRAM

- Constructs used in the ER diagram can easily be transformed into relational tables.
- It is simple and easy to understand with minimum training.

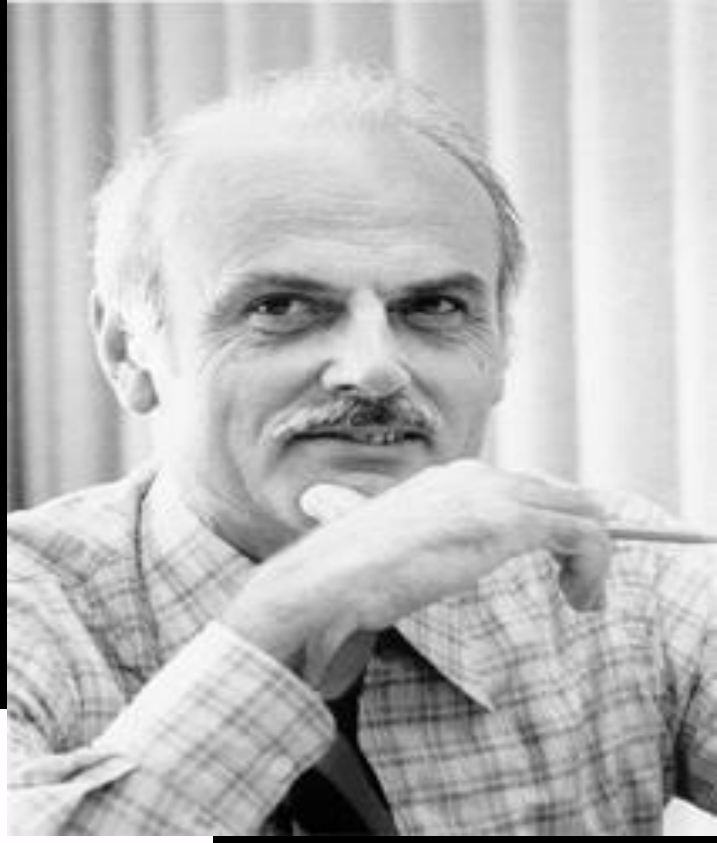
DISADVANTAGE OF E-R DIGRAM

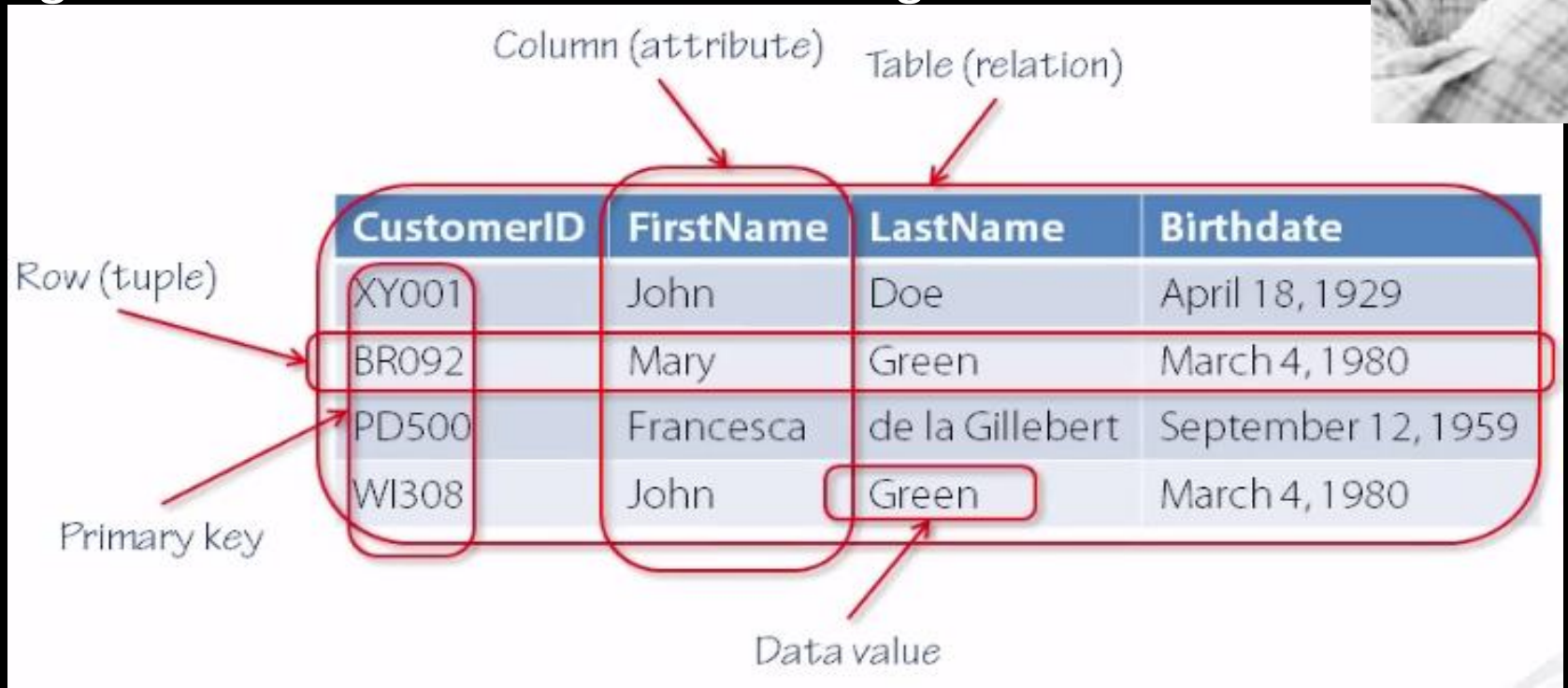
- Loss of information content
- Limited constraints representation
- It is overly complex for small projects

Knowledge Gate Website

RELATIONAL DATABASE MANAGEMENT SYSTEM

- A relational database management system (RDBMS), conceptualized by Edgar F. Codd in 1970, serves as the foundation for most contemporary commercial and open-source database applications.
- Central to its design is the utilization of tables for data storage, where it maintains and enforces specific data relationships, marking a significant evolution in database design.





CustomerID	FirstName	LastName	Birthdate
XY001	John	Doe	April 18, 1929
BR092	Mary	Green	March 4, 1980
PD500	Francesca	de la Gillebert	September 12, 1959
WI308	John	Green	March 4, 1980

BASICS OF RDBMS

- **Domain (set of permissible value in particular column)** is a set of atomic values.
- By **atomic** we mean that each value in the domain is indivisible as far as the formal relational model is concerned.
- A common method of specifying a domain is to specify a data type from which the data values forming the domain are drawn.
  - **E.g.** Names: The set of character strings that represent names of persons.

Domain/ Field/ Column/ Arity/ Degree

NAME	ID	CITY	COUNTRY	HOBBY
NISHA	1	AGRA	INDIA	PLAYING
NIKITA	2	DELHI	INDIA	DANCING
AJAY	3	AGRA	INDIA	CHESS
ARPIT	4	PATNA	INDIA	READING

Knowledge Gate Website

- **Table (Relation)** - A Relation is a set of tuples/rows/entities/records.
- **Tuple** - Each row of a Relation/Table is called Tuple.
- **Arity/Degree** - No. of columns/attributes of a Relation. E.g. - Arity is 5 in Table Student.
- **Cardinality** - No of rows/tuples/record of a Relational instance. E.g. - Cardinality is 4 in table Student.

Rows/Tuples/Record/ Cardinality →

NAME	ID	CITY	COUNTRY	HOBBY
NISHA	1	AGRA	INDIA	PLAYING
NIKITA	2	DELHI	INDIA	DANCING
AJAY	3	AGRA	INDIA	CHESS
ARPIT	4	PATNA	INDIA	READING

Properties of Relational tables

1. Cells contains atomic values
2. Values in a column are of the same kind
3. Each row is unique
4. No two tables can have the same name in a relational schema.
5. Each column has a unique name
6. The sequence of rows is insignificant
7. The sequence of columns is insignificant.

Knowledge Gate Website

Problems in relational database

- **Update Anomalies-** Anomalies that cause redundant work to be done during insertion into and Modification of a relation and that may cause accidental loss of information during a deletion from a relation
  - **Insertion Anomalies**
  - **Modification Anomalies**
  - **Deletion Anomalies**

- **Insertion anomalies:** An independent piece of information cannot be recorded into a relation unless an irrelevant information must be inserted together at the same time.

Roll no	name	Age	Br_code	Br_name	Br_hod_name
1	A	19	101	Cs	Abc
2	B	18	101	Cs	Abc
3	C	20	101	Cs	Abc
4	D	20	102	Ec	Pqr

- **Modification anomalies:** The update of a piece of information must occur at multiple locations.

Roll no	name	Age	Br_code	Br_name	Br_hod_name
1	A	19	101	Cs	Abc
2	B	18	101	Cs	Abc
3	C	20	101	Cs	Abc
4	D	20	102	Ec	Pqr

- **Deletion Anomalies:** The deletion of a piece of information unintentionally removes other information.

Roll no	name	Age	Br_code	Br_name	Br_hod_name
1	A	19	101	Cs	Abc
2	B	18	101	Cs	Abc
3	C	20	101	Cs	Abc
4	D	20	102	Ec	Pqr

Roll no	name	Age	Br_code	Br_name	Br_hod_name
1	A	19	101	Cs	Abc
2	B	18	101	Cs	Abc
3	C	20	101	Cs	Abc
4	D	20	102	Ec	Pqr

PK

FK

PK

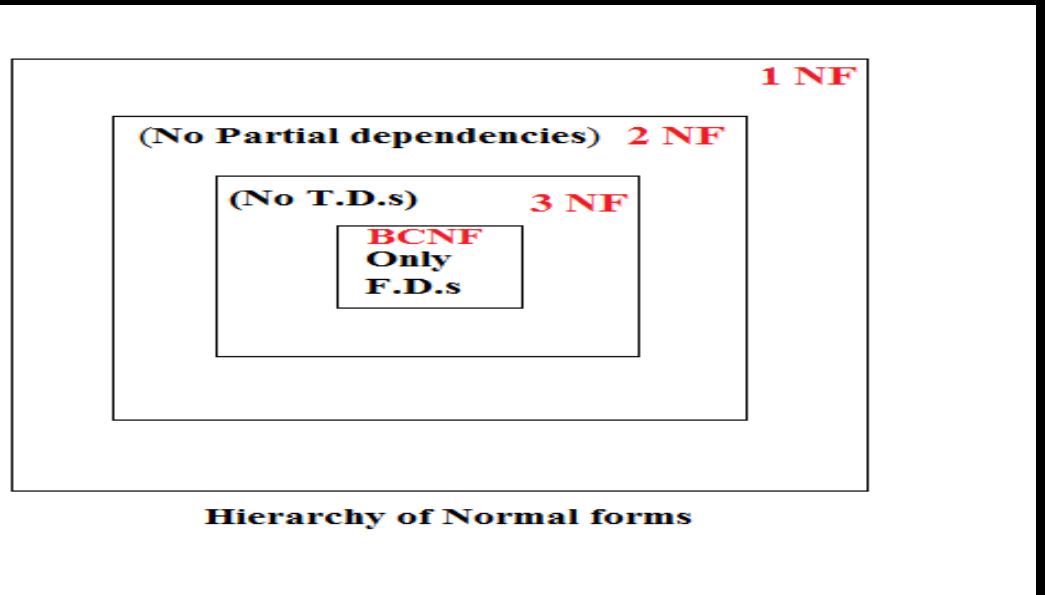
Roll no	name	Age	Br_code
1	A	19	101
2	B	18	101
3	C	20	101
4	D	20	102

Br_code	Br_name	Br_hod_name
101	Cs	Abc
102	ec	Pqr

Knowledge Gate Website

Purpose of Normalization

- Normalization may be simply defined as refinement process. Which includes creating tables and establishing relationships between those tables according to rules designed both to protect data and make the database more flexible by eliminating two factors;
  - Redundancy
  - Inconsistent Dependency
- With out normalization data base system may be inaccurate, slow and inefficient and they might not produce the data we expect. A series of normal form tests that can be carried out on individual relation schemas so that the relational database can be **normalized** to any desired degree.
- 1NF>>>2NF>>3NF>>BCNF



Hierarchy of Normal forms

Knowledge C

Conclusion

1. Like every paragraph must have a single idea similarly every table must have a single idea and if a table contains more than one idea then that table must be decomposed until each table contains only one idea.

2. We need some tools to approach this decomposition or normalization on large database which contains a number of table, and that tool is functional dependencies.

Roll no	name	Age	Br_code	Br_name	Br_hod_name
1	A	19	101	Cs	Abc
2	B	18	101	Cs	Abc
3	C	20	101	Cs	Abc
4	D	20	102	Ec	Pqr

Roll no	name	Age	Br_code
1	A	19	101
2	B	18	101
3	C	20	101
4	D	20	102

Br_code	Br_name	Br_hod_name
101	Cs	Abc
102	ec	Pqr

Knowledge Gate Website

Functional Dependency

Knowledge Gate Website

MARUTI SUZUKI

MARUTI SUZUKI

Knowledge Gate Website

Br_code → Br_hod_name

Br_code ← Br_hod_name

Knowledge Gate Website

Functional Dependency कोई बताता नहीं, इसकी feel आ जाती है

Knowledge Gate Website

FUNCTIONAL DEPENDENCY

- A formal tool for analysis of relational schemas.
- In a Relation R, if ‘α’ ⊑ R AND ‘β’ ⊑ R, then attribute or a Set of attribute ‘α’ Functionally derives an attribute or set of attributes ‘β’, iff each ‘α’ value is associated with precisely one ‘β’ value.
- For all pairs of tuples t_1 and t_2 in R such that
  - If $T_1[\alpha] = T_2[\alpha]$
  - Then, $T_1[\beta] = T_2[\beta]$

X	Y	Z
1	4	2
1	4	3
2	6	3
3	2	2

Q Consider the following relation instance, which of the following dependency doesn't hold

A) A → b

B) BC → A

C) B → C

D) AC → B

A	B	C
1	2	3
4	2	3
5	3	3

Knowledge Gate Website

- Trivial Functional dependency - If β is a subset of α, then the functional dependency α → β will always hold.

जिसका होना न होना बराबर हो

X	Y	Z
1	4	2
1	4	3
2	6	3
3	2	2

Knowledge Gate Website

ATTRIBUTES CLOSURE/CLOSURE ON ATTRIBUTE SET/ CLOSURE SET OF ATTRIBUTES

- Attribute closure of an attribute set can be defined as set of attributes which can be functionally determined from F.
  - DENOTED BY F^+

R (A, B, C, D)

A → B

B → C

AB → D

A^+ =

R(ABCDEFG)

A → B

BC → DE

AEG → G

$(AC)^+$ =?

Q R(ABCDE)

A → BC

CD → E

B → D

E → A

$(B)^+$ =

Knowledge Gate Website

ARMSTRONG'S AXIOMS

1. An **axiom** or **postulate** is a statement that is taken to be true, to serve as a premise or starting point for further reasoning and arguments.
2. **Armstrong's axioms** are a set of axioms (or, more precisely, inference rules) used to infer all the functional dependencies on a relational database. They were developed by William W. Armstrong in his 1974 paper.
3. The axioms are sound in generating only functional dependencies in the closure of a set of functional dependencies (denoted as F^+) when applied to that set (denoted as F).

Knowledge Gate Website

Armstrong Axioms

- **Reflexivity**: If Y is a subset of X, then $X \rightarrow Y$
- **Augmentation**: If $X \rightarrow Y$, then $XZ \rightarrow YZ$
- **Transitivity**: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

Knowledge Gate Website

From these rules, we can derive these secondary rules-

- **Union**: If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
- **Decomposition**: If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$
- **Pseudo transitivity**: If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$
- **Composition**: If $X \rightarrow Y$ and $Z \rightarrow W$, then $XZ \rightarrow YW$

Knowledge Gate Website

Why Armstrong axioms refers to the Sound and Complete

- By sound, we mean that given a set of functional dependencies F specified on a relation schema R, any dependency that we can infer from F by using the primary rules of Armstrong axioms holds in every relation state r of R that satisfies the dependencies in F.

- By complete, we mean that using primary rules of Armstrong axioms repeatedly to infer dependencies until no more dependencies can be inferred results in the complete set of all possible dependencies that can be inferred from F.

Equivalence of Two FD sets-

Two FD sets F_1 and F_2 are equivalent if –

$F_1^+ = F_2^+$

Or

$F_1 \sqsubseteq F_2^+$ and $F_2 \sqsubseteq F_1^+$

Q Consider the following set of fd R(ACDEH)

F	G
A → C	A → CD
AC → D	E → AH
E → AD	
E → H	

Knowledge Gate Website

To find the MINIMAL COVER / CANONICAL COVER / IRREDUCIBLE SET

- A **canonical cover** (also known as a minimal cover) for a set of functional dependencies in a database is a minimal set of functional dependencies that is equivalent to the original set, but with redundant dependencies and extraneous attributes removed. It is used in the normalization process of database design to simplify the set of functional dependencies and to find a good set of relations.

- There may be any following type of redundancy in the set of functional dependencies: -
  - Complete production may be Redundant.
  - One or more than one attributes may be redundant on right hand side of a production.
  - One or more than one attributes may be redundant on Left hand side of a production.

Q R(ABCD)

A → B

C → B

D → ABC

AC → D

Knowledge Gate Website

R(A,B,C)

A → B

B → C

A → C

AB → B

AB → C

AC → B

Knowledge Gate Website

Key

Knowledge Gate Website

Super key

- Set of attributes using which we can identify each tuple uniquely is called Super key.
- Let X be a set of attributes in a Relation R , if X^+(Closure of X) determines all attributes of R then X is said to be Super key of R .
- There should be at least one Super key in every relation.

Candidate key

- Minimal set of attributes using which we can identify each tuple uniquely is called Candidate key. A super key is called candidate key if it’s No proper subset is a super key. Also called as **MINIMAL SUPER KEY.**

- There should be at least one candidate key.

Prime attribute - Attributes that are member of at least one candidate Keys are called Prime attributes.

Primary key

- One of the candidate keys is selected by database administrator as a Primary means to identify tuple is called primary Key. Primary Key attribute are not allowed to have Null values. Exactly one Primary Key per table in RDMS.

- Candidate key which are not chosen as primary key is alternate key.

Foreign Keys

- A foreign key is a column or group of columns in a relational database table that refers the primary key of the same table or some other table to represent relationship.

- The ***concept of referential integrity*** is derived from foreign key theory.

Roll no	name	Age	Br_code	Br_name	Br_hod_name
1	A	19	101	Cs	Abc
2	B	18	101	Cs	Abc
3	C	20	101	Cs	Abc
4	D	20	102	Ec	Pqr

PK

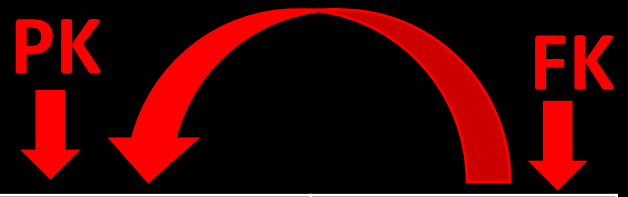
FK

PK

Roll no	name	Age	Br_code
1	A	19	101
2	B	18	101
3	C	20	101
4	D	20	102

Br_code	Br_name	Br_hod_name
101	Cs	Abc
102	ec	Pqr

Knowledge Gate Website



Roll no	CR
1	1
2	2
3	1
4	2
5	1
6	2
7	1
8	2
9	1
10	2
11	1
12	2
13	1
14	2
15	null

Knowledge Gate Website

- **Composite key** – Composite key is a key composed of more than one column sometimes it is also known as concatenated key.

- **Secondary key** – Secondary key is a key used to speed up the search and retrieval contrary to primary key, a secondary key does not necessary contain unique values.

NORMAL FORM

Knowledge Gate Website

FIRST NORMAL FORM

- 1NF is the initial step of database normalization.
- Implications of first normal form
  - **Atomic Values**: Each cell in a table contains indivisible, atomic values. Means a Relation should not contain any multivalued or composite attributes.
  - **Unique Columns**: Each column must have a distinct name to identify the data it contains.
  - **Primary Key**: A table in 1NF should have a primary key that uniquely identifies each record.
  - **Eliminating Duplicates**: Duplicate rows are removed to prevent data redundancy.

Prime attribute: - A attribute is said to be prime if it is part of any of the candidate key

Non-Prime attribute: - A attribute is said to be non-prime if it is not part of any of the candidate key

Eg R(ABCD)

AB→CD

Here candidate key is AB so, A and B are prime attribute, C and D are non-prime attributes.

PARTIAL DEPENDENCY- When a non – prime attribute is dependent only on a part (Proper subset) of candidate key then it is called partial dependency. (PRIME > NON-PRIME)

Full DEPENDENCY- When a non – prime attribute is dependent on the entire candidate key then it is called Full dependency.

e.g. R(ABCD)

AB→D

A→C

SECOND NORMAL FORM

- Relation R is in 2NF if,
  - R should be in 1 NF.
  - R should not contain any Partial dependency. (that is every non-prime attribute should be fully dependent upon candidate key)

Q R(A, B, C) B→C

A	B	C
a	1	X
b	2	Y
a	3	Z
C	3	Z
D	3	Z
E	3	Z

A	B
A	1
B	2
A	3
C	3
D	3
E	3

B	C
1	X
2	Y
3	z

Knowledge Gate Website

Knowledge Gate Website

TRANSITIVE DEPENDENCY – A functional dependency from non-Prime attribute to non-Prime attribute is called transitive

E.g.- R(A, B, C, D) with A as a candidate key
A→B
B→C [transitive dependency]
C→D [transitive dependency]

THIRD NORMAL FORM

- Let R be the relational schema, it is said to be in 3 NF
  - R should be in 2NF
  - It must not contain any transitive dependency

THIRD NORMAL FORM DIRECT DEFINATION

- A relational schema R is said to be 3 NF if every functional dependency in R from α-->β, either α is super key or β is the prime attribute

R(A, B, C)

A → B

B → C

A	B	C
A	1	P
B	2	Q
C	2	Q
D	2	Q
E	3	R
F	3	R
G	4	S

A	B
A	1
B	2
C	2
D	2
E	3
F	3
G	4

B	C
1	P
2	Q
3	R
4	S

Knowledge Gate Website

BCNF (BOYCE CODD NORMAL FORM)

- A relational schema R is said to be BCNF if every functional dependency in R from
  - $\alpha \rightarrow \beta$
  - α must be a super key

R(A, B, C)

AB → C

C → B

A	B	C
A	C	B
B	B	C
B	A	D
A	A	E
C	C	B
D	C	B
E	C	B
F	C	B

A	B
A	B
B	B
B	A
A	A
C	C
D	C
e	C
f	c

C	B
B	C
C	B
D	A
E	A

Knowledge Gate Website

Some important note points on Normalization:

- A Relation with two attributes is always in BCNF.
- A Relation schema R consist of only prime attributes then R is always in 3NF, but may or may not be in BCNF.

Q Consider the universal relational schema R (A, B, C, D, E, F, G, H, I, J) and a set of following functional dependencies.

F = {AB → C, A → DE, B → F, F → GH, D → IJ} determine the keys for R ?

Decompose R into 2nd normal form.

Q Find the normal form of relation R(A, B, C, D, E) having FD set F= {A → B, BC → E, ED → A}.

Knowledge Gate Website

Multivalued Dependency

- Denoted by, A →→ B, Means, for every value of A, there may exist more than one value of B.
- **E.g. S_name → → Club_name**

S_Name	Club_name
Kamesh	Dance
Kamesh	Guitar

- A **trivial multivalued dependency** X→→Y is one where either *Y* is a subset of *X*, or *X* and *Y* together form the whole set of attributes of the relation.

S_name →→ Club_name

S_Name	Club_name
Kamesh	Dance
Kamesh	Guitar

S_name →→ Club_name
S_name →→ P_no

S_Name	Club_name	P_no
Kamesh	Dance	123
Kamesh	Guitar	123
Kamesh	Dance	789
Kamesh	Guitar	789

Knowledge Gate Website

E.g. let the constraint specified by MVD in relation **Student as**

S_name →→ Club_name

S_name →→ P_no

S_Name	Club_name
Kamesh	Dance
Kamesh	Guitar

S_Name	P_no
Kamesh	123
Kamesh	789

S_Name	Club_name	P_no
Kamesh	Dance	123
Kamesh	Guitar	123
Kamesh	Dance	789
Kamesh	Guitar	789

NOTE: The above Student schema is in BCNF as no functional dependency holds on EMP, but still redundancy due to MVD.

Knowledge Gate Website

- Each row indicates that a given restaurant can deliver a given variety. The table has no non-key attributes because its only key is {Restaurant, Variety, Delivery Area}. Therefore, it meets all normal forms up to BCNF.
- If we assume, however, that Variety offered by a restaurant are not affected by delivery area (i.e. a restaurant offers all Variety it makes to all areas it supplies), then it does not meet 4NF. The problem is that the table features two non-trivial multivalued dependencies on the {Restaurant} attribute (which is not a super key). The dependencies are:
  - {Restaurant} {Variety}
  - {Restaurant} {Delivery Area}

Restaurant Delivery Permutations		
Restaurant	**Variety**	**Delivery Area**
Chatora Sweets	Samosa	Hatibagan Market
Chatora Sweets	Samosa	Chandni Chowk
Chatora Sweets	Samosa	Koramangala
Chatora Sweets	Dosa	Hatibagan Market
Chatora Sweets	Dosa	Chandni Chowk
Chatora Sweets	Dosa	Koramangala
Moolchand	Ladoo	Koramangala
Moolchand	Dosa	Koramangala
Thaggu	Samosa	Hatibagan Market
Thaggu	Samosa	Chandni Chowk
Thaggu	Ladoo	Hatibagan Market
Thaggu	Ladoo	Chandni Chowk

- If we have two or more multivalued *independent* attributes in the same relation schema, we get into a problem of having to repeat every value of one of the attributes with every value of the other attribute to keep the relation state consistent and to maintain the independence among the attributes involved. This constraint is specified by a multivalued dependency.

Delivery Areas By Restaurant	
Restaurant	Delivery Area
Chatora Sweets	Hatibagan Market
Chatora Sweets	Chandni Chowk
Chatora Sweets	Koramangala
Moolchand	Koramangala
Thaggu	Hatibagan Market
Thaggu	Chandni Chowk

Varieties By Restaurant	
Restaurant	Pizza Variety
Chatora Sweets	Samosa
Chatora Sweets	Dosa
Moolchand	Ladoo
Moolchand	Dosa
Thaggu	Samosa
Thaggu	Ladoo

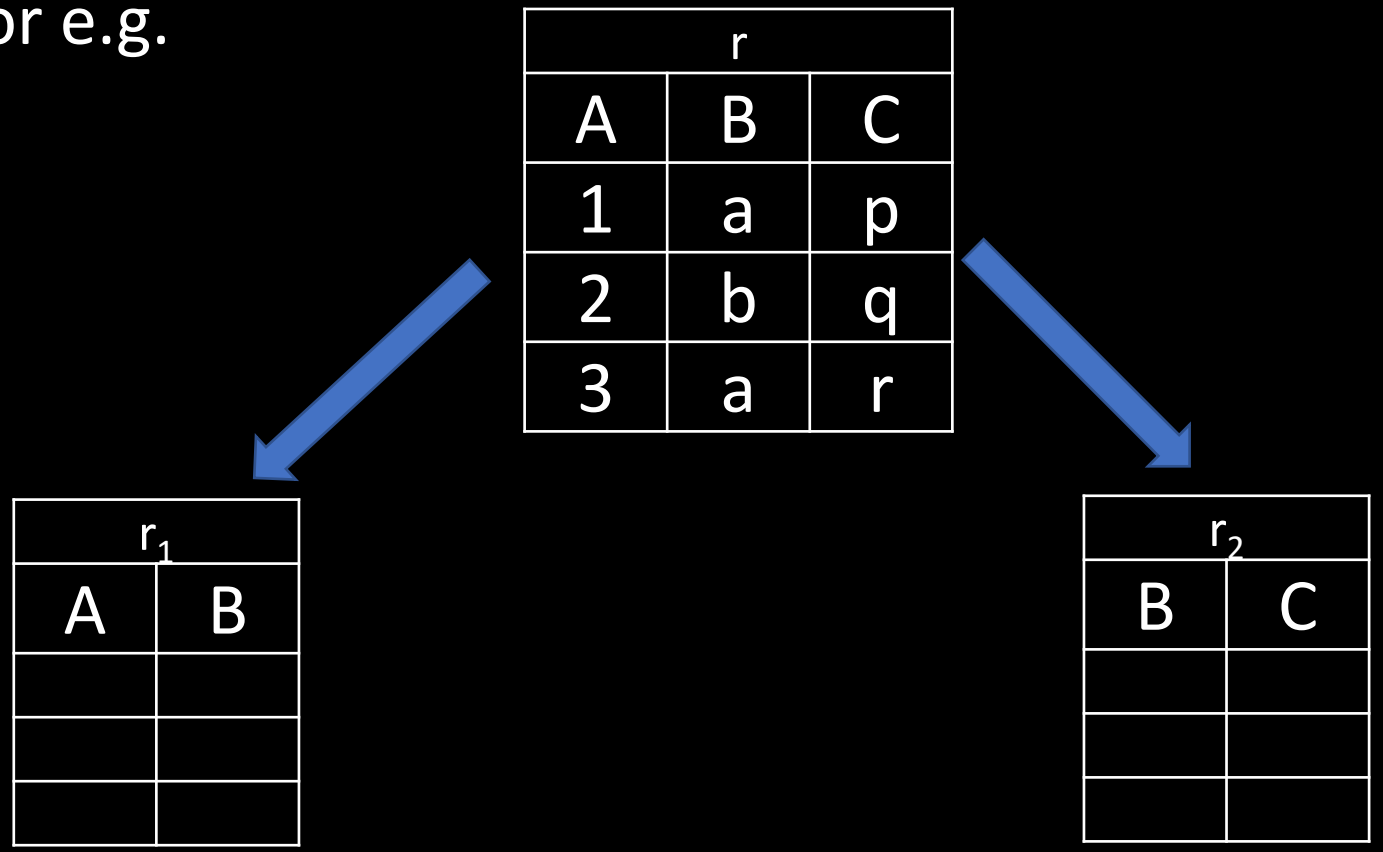
- A relation is in 4NF iff
  - It is in BCNF
  - There must not exist any non-trivial multivalued dependency.

- Each MVD is decomposed in separate table, where it becomes trivial MVD.

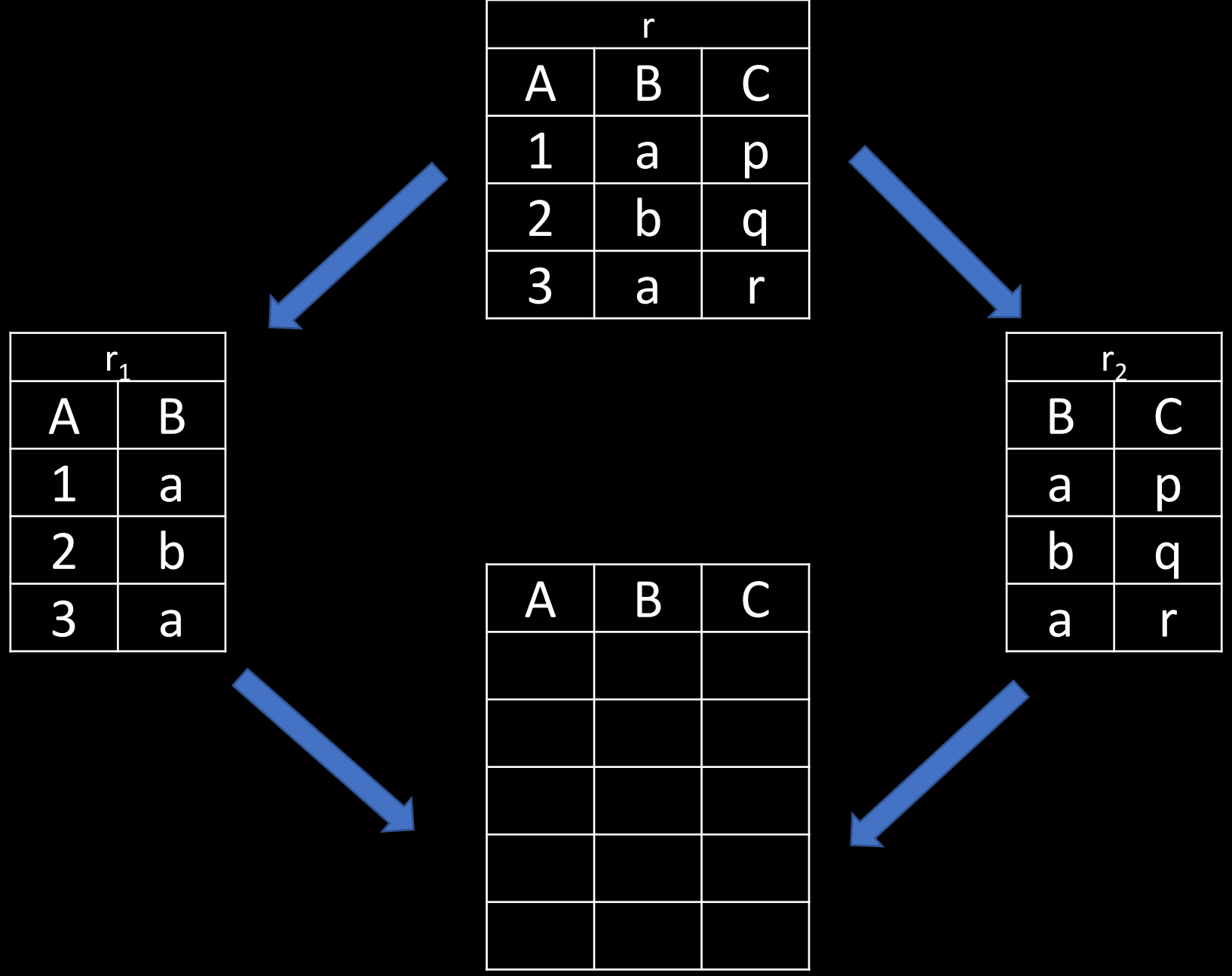
Lossy/Lossless-Dependency Preserving Decomposition

- Because of a normalization a table is Decomposed into two or more tables, but during this decomposition we must ensure satisfaction of some properties out of which the most important is lossless join property / decomposition.

- if we decompose a table r into two tables r_1 and r_2 because of normalization then at some later stage if we want to join(combine) (natural join) these tables r_1 and r_2, then we must get back the original table r, without any extra or less tuple. But some information may be lost during retrieval of original relation or table. For e.g.



Knowledge Gate Website



Knowledge Gate Website

- Decomposition is lossy if $R_1 \bowtie R_2 \supset R$
- Decomposition is lossy if $R \supset R_1 \bowtie R_2$

- Decomposition is lossless if $R_1 \bowtie R_2 = R$ "The decomposition of relation R into R_1 and R_2 is lossless when the join of R_1 and R_2 yield the same relation as in R." which guarantees that the spurious (extra or less) tuple generation problem does not occur with respect to the relation schemas created after decomposition.

- This property is extremely critical and must be achieved at any cost.

- lossless Decomposition / NonAdditive Join Decomposition

A	B	C	D	E
A	122	1	W	A
E	236	4	X	B
A	199	1	Y	C
B	213	2	Z	D

Knowledge Gate Website

How to check for lossless join decomposition using FD set, following conditions must hold:

- Union of Attributes of R_1 and R_2 must be equal to attribute of R. Each attribute of R must be either in R_1 or in R_2. $Att(R_1) \cup Att(R_2) = Att(R)$

- Intersection of Attributes of R_1 and R_2 must not be NULL. $Att(R_1) \cap Att(R_2) \neq \Phi$

- Common attribute must be a key for at least one relation (R_1 or R_2)
  - $Att(R_1) \cap Att(R_2) \rightarrow (R_1)$ or $Att(R_1) \cap Att(R_2) \rightarrow (R_2)$

Q R (A, B, C, D)

A→ B, B→C, C→D, D→A

R_1(A, B), R_2(B, C) AND R_3(C, D)

Knowledge Gate Website

Q R(ABCDE)(NF) R_1(AB) R_2(BC) R_3(ABCD) R_4(EG)

A→BC

C→DE

D→E

Knowledge Gate Website

5 NF/Project-Join Normal Form

- A Relational table R is said to be in 5^{th} normal form if
  - it is in 4 NF
  - it cannot be further non-loss decomposed

Dependency Preserving Decomposition

- Let relation R be decomposed into Relations $R_1, R_2, R_3 \ldots\ldots\ldots\ldots R_N$ with their respective functional Dependencies set as $F_1, F_2, F_3 \ldots\ldots\ldots\ldots F_N$, then the Decomposition is Dependency Preserving iff
  - $\{F_1 \cup F_2 \cup F_3 \cup F_4 \ldots\ldots\ldots \cup F_N\}^+ = F^+$

- Dependency preservation property, although desirable, is sometimes sacrificed.

Knowledge Gate Website

X = (PQRS)

F = {QR → S, R → P, S → Q}

Y = (PR) and Z = (QRS)

Knowledge Gate Website

Indexing

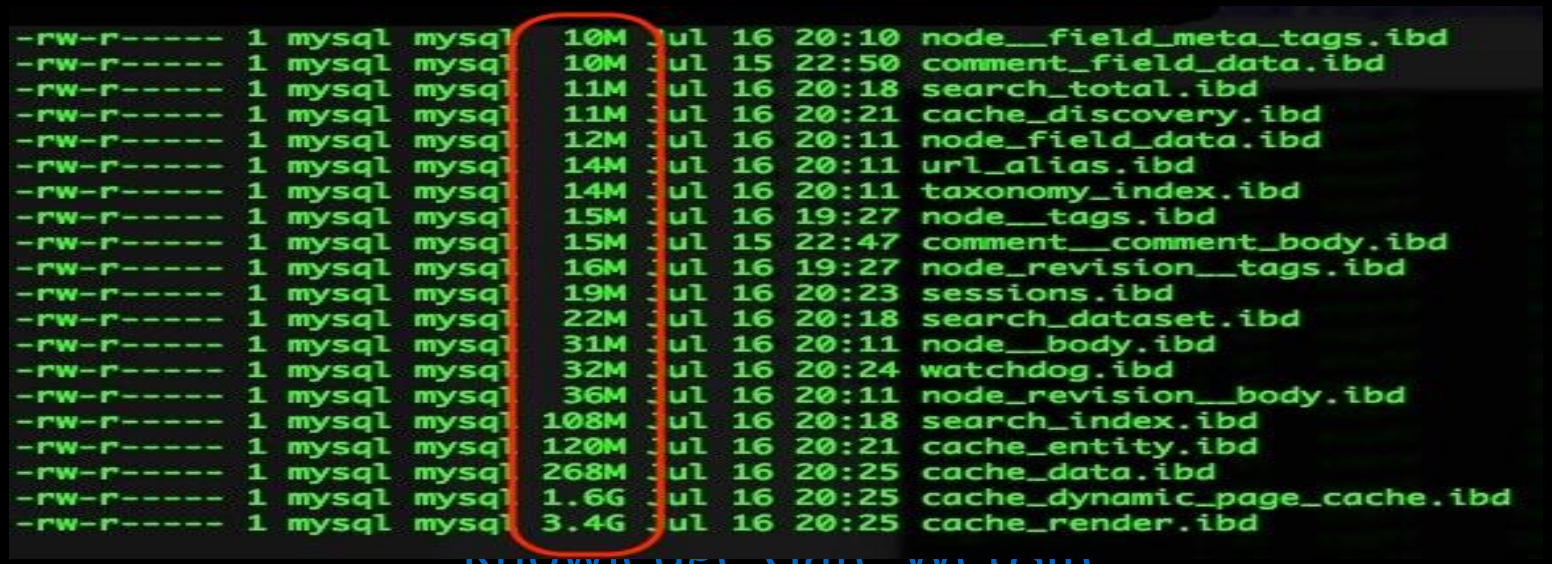
Relational databases are based on set theory.

- In set theory, the order of elements is unimportant, similarly in database tables.
- However, in practical implementation, element order in tables is often specified.
- Various operations such as search, insertion, and deletion are influenced by the order of elements in the tables.
- Elements in a table can be stored in two ways: sorted (ordered) or unsorted (unordered).

File organization/ organization of records in a file

Ordered file organization

- All the records in the file are ordered on some search key field.
- Here binary search is possible. (give example of book page searching)
- Maintenance (insertion & deletion) is costly, as it requires reorganization of entire file.
- Notes that we will get binary search only if we are using that key for searching on which indexing is done, otherwise it will behave as unsorted file.
- if file is unordered then no of block assesses required to reach correct block which contain the desired record is $O(\log_2 n)$, where n is the number of blocks.



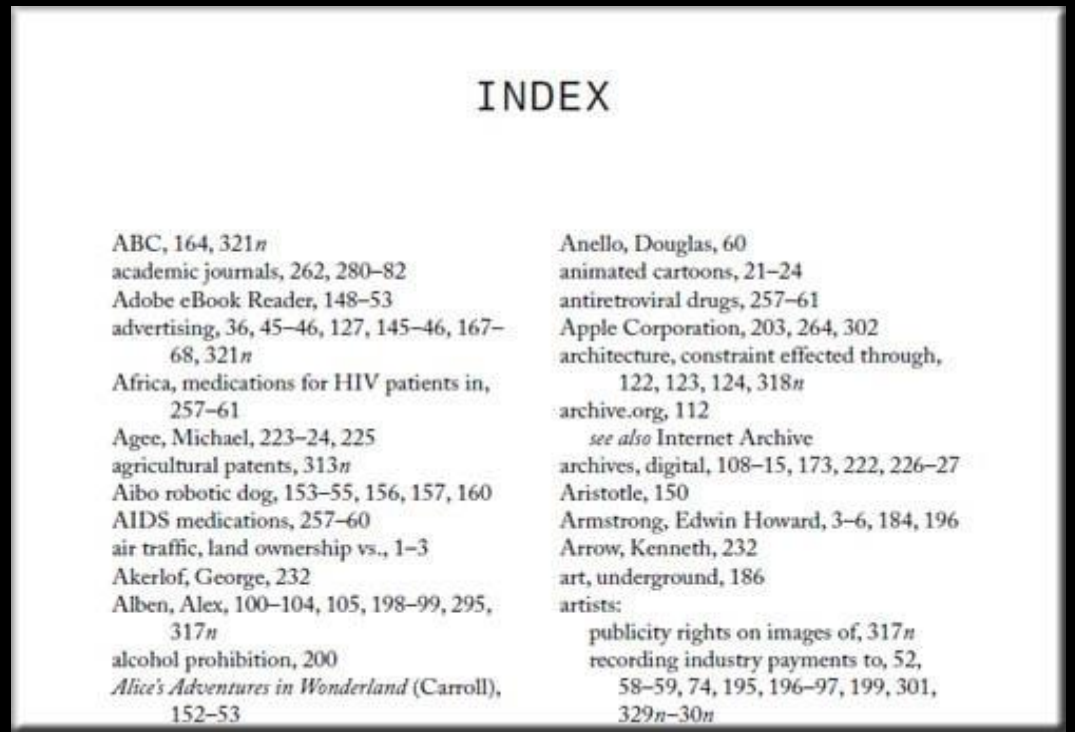
Knowledge Gate Website

Unordered file organization

- Records are typically added at the end of the file, without following any specific order.
- This insertion method allows only linear search, resulting in slower search times.
- Despite slow searches, maintenance including insertion and deletion is simpler.
- No reorganization of the entire file is needed, making maintenance easier.
- If file is unordered then no of block assesses required to reach correct block which contain the desired record is O(n), where n is the number of blocks.

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 2	A-215	Mianus	700
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600
record 8	A-218	Perryridge	700

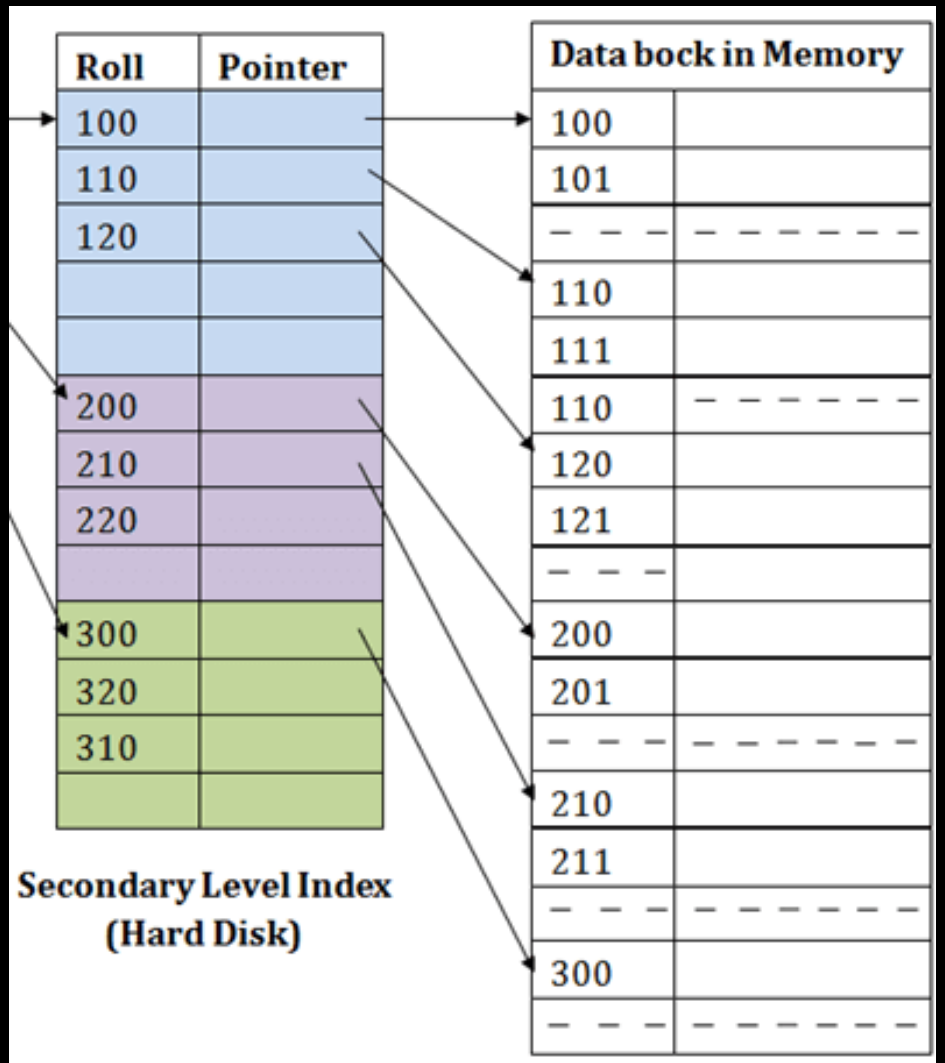
- Indexes are supplementary structures in databases, aiding in swift record retrieval.
- They enable quick data access based on particular attributes identified for indexing.
- This technique is similar to the index sections seen in books.
- Indexes provide secondary pathways to access records without changing their physical position in the main file.



INDEX

ABC, 164, 321*n*
academic journals, 262, 280–82
Adobe eBook Reader, 148–53
advertising, 36, 45–46, 127, 145–46, 167–68, 321*n*
Africa, medications for HIV patients in, 257–61
Agee, Michael, 223–24, 225
agricultural patents, 313*n*
Aibo robotic dog, 153–55, 156, 157, 160
AIDS medications, 257–60
air traffic, land ownership vs., 1–3
Akerlof, George, 232
Alben, Alex, 100–104, 105, 198–99, 295, 317*n*
alcohol prohibition, 200
Alice's Adventures in Wonderland (Carroll), 152–53
Anello, Douglas, 60
animated cartoons, 21–24
antiretroviral drugs, 257–61
Apple Corporation, 203, 264, 302
architecture, constraint effected through, 122, 123, 124, 318*n*
archive.org, 112
see also Internet Archive
archives, digital, 108–15, 173, 222, 226–27
Aristotle, 150
Armstrong, Edwin Howard, 3–6, 184, 196
Arrow, Kenneth, 232
art, underground, 186
artists:
publicity rights on images of, 317*n*
recording industry payments to, 52, 58–59, 74, 195, 196–97, 199, 301, 329*n*–30*n*

- The size of index file is way smaller than that of the main file, as index file record contain only two columns key (attribute in which searching is done) and block pointer (base address of the block of main file which contains the record holding the key), while main file contains all the columns.



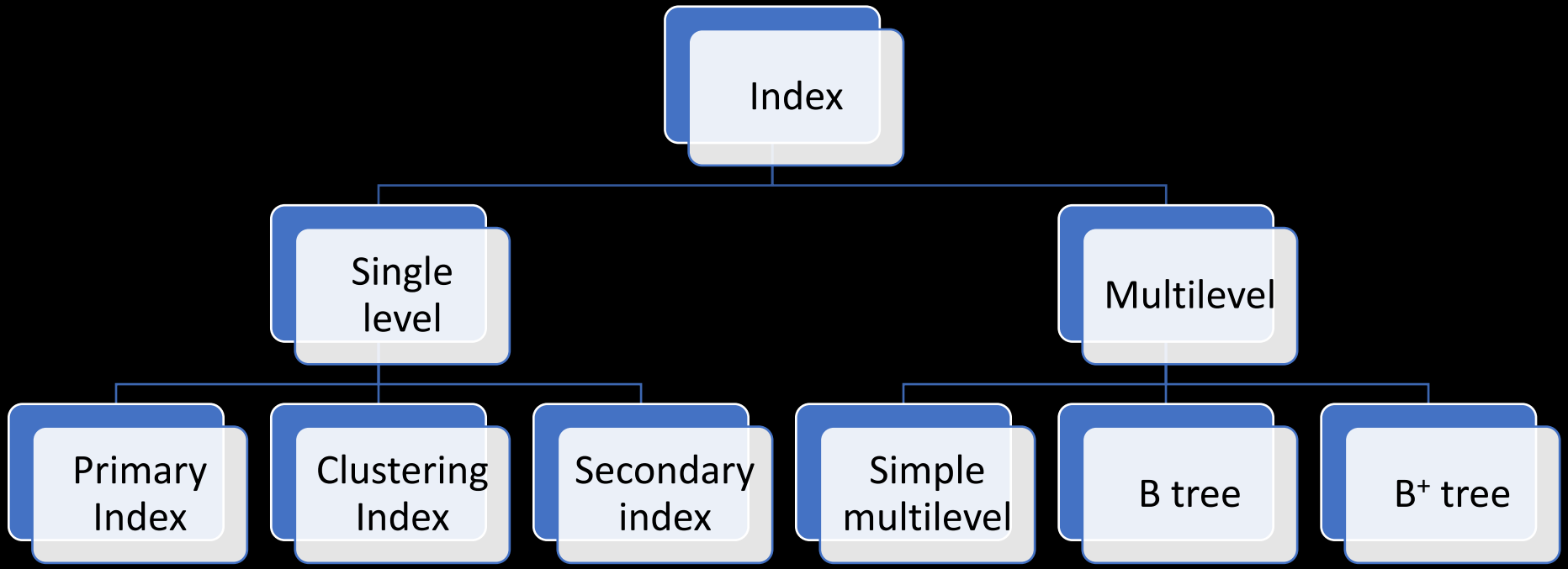
Q Suppose we have ordered file with records stored r = 30,000 on a disk with Block Size B = 1024 B. File records are of fixed size and are unspanned with record length R = 100 B. Suppose that ordering key field of file is 9 B long and a block pointer is 6 B long, Implement primary indexing?

Knowledge Gate Website

1. Indexes can be established on any relation field, be it primary key or non-key.
2. Each attribute can have a dedicated index file, meaning multiple index files may exist for one main file.
3. Index files are always organized, allowing for the utilization of binary search advantages, irrespective of the main file's order.
4. Indexing accelerates data retrieval time but also introduces space overhead for storing the index file.
5. The correct block in the main file can be located with $\log_2$(number of blocks in index file) + 1 accesses.

Knowledge Gate Website

TYPES OF INDEXING



- In single-level indexing, an index file is created for the main file, marking the end of the indexing process.
- Multiple-level indexing, on the other hand, involves creating an index for the index file and continually repeating this procedure until only a single block remains.

Knowledge Gate Website

PRIMARY INDEXING

- Main file is always sorted according to primary key.
- Indexing is done on Primary Key, therefore called as primary indexing
- Index file have two columns, first primary key and second anchor pointer (base address of block)
- It is an example of Sparse Indexing.
- Here first record (anchor record) of every block gets an entry in the index file
- No. of entries in the index file = No of blocks acquired by the main file.

CLUSTERED INDEXING

- Main file will be ordered on some non-key attributes
- No of entries in the index file = no of unique values of the attribute on which indexing is done.
- It is the example of Sparse as well as dense indexing

Q Suppose we have ordered file with records stored r = 30,000 on a disk with Block Size B = 1024 B. File records are of fixed size and are unspanned with record length R = 100 B. Suppose that ordering key field of file is 9 B long and a block pointer is 6 B long, Implement Secondary indexing?

Knowledge Gate Website

SECONDARY INDEXING

- Most common scenarios, suppose that we already have a primary indexing on primary key, but there is frequent query on some other attributes, so we may decide to have one more index file with some other attribute.
- Main file is ordered according to the attribute on which indexing is done(unordered).
- Secondary indexing can be done on key or non-key attribute.
- No of entries in the index file is same as the number of entries in the main file.
- It is an example of dense indexing.

Dense Vs Sparse

- Dense Index In dense index, there is an entry in the index file for every search key value in the main file. This makes searching faster but requires more space to store index records itself. Note that it is not for every record, it is for every search key value. Sometime number of records in the main file > number of search keys in the main file, for example if search key is repeated.

- Sparse Index-If an index entry is created only for some records of the main file, then it is called sparse index. No. of index entries in the index file < No. of records in the main file. Note: - dense and sparse are not complementary to each other, sometimes it is possible that a record is both dense and sparse.

B tree

- A B-tree of order m if non-empty is an m-way search tree in which.
  - The root has at least zero child nodes and at most m child nodes.
  - The internal nodes except the root have at least celling(m/2) child nodes and at most m child nodes.
  - The number of keys in each internal node is one less than the number of child nodes and these keys partition the subtrees of the nodes in a manner similar to that of m-way search tree.
  - All leaf nodes are on the same level(perfectly balanced).

Root

Rules	MAX	MIN
CHILD	m	0
DATA	m-1	1

Internal except Root

Rules	MAX	MIN
CHILD	m	⌈m/2⌉
DATA	m-1	⌈m/2⌉ - 1

Leaf

Rules	MAX	MIN
CHILD	0	0
DATA	m-1	⌈m/2⌉ - 1

Knowledge Gate Website

Insertion in B-TREE

- A B-tree starts with a single root node (which is also a leaf node) at level 0 (zero). Once the root node is full with m – 1 search key values and we attempt to insert another entry in the tree, the root node splits into two nodes at level 1.

- Only the middle value is kept in the root node, and the rest of the values are split evenly between the other two nodes. When a non-roof node is full and a new entry is inserted into it, that node is split into two nodes at the same level, and the middle entry is moved to the parent node along with two pointers to the new split nodes.

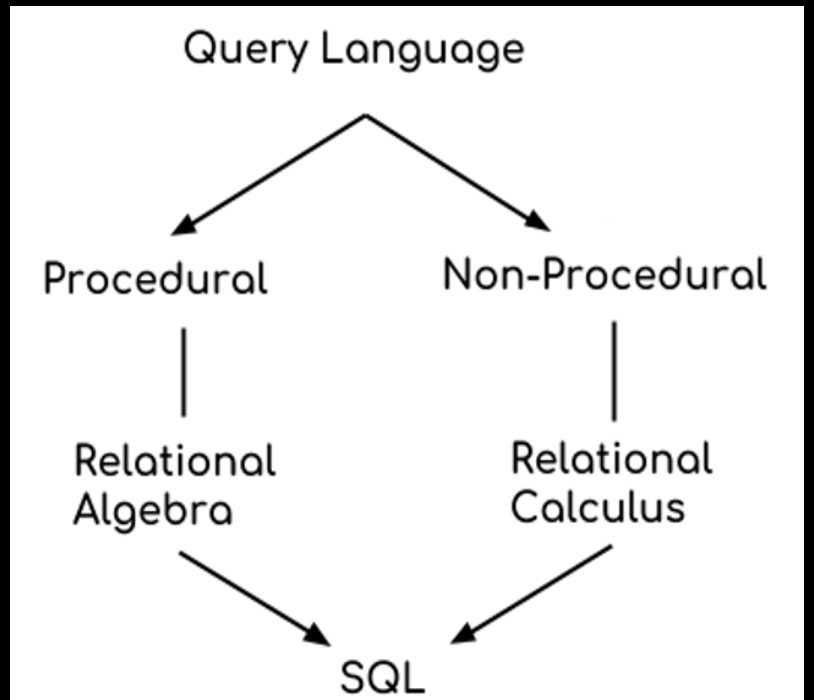
- If the parent node is full, it is also split. Splitting can propagate all the way to the root node, creating a new level if the root is split.

Knowledge Gate Website

Q Consider the following elements 5, 10, 12, 13, 14, 1, 2, 3, 4 insert them into an empty b-tree of order = 3.

Query Language

- After designing a data base, that is ER diagram followed by conversion in relational model followed by normalization and indexing, now next task is how to store, retrieve and modify data in the data base.
- So here we will be concentrating more on the retrieval part. Query languages are used for this purpose. **Query languages**, **data query languages** or **database query languages** (**DQLs**) are computer languages using which user request some information from the database. A well known example is the **Structured Query Language (SQL).**



Procedural Query Language

- Here users instruct the system to performs a sequence of operations on the data base in order to compute the desired result.

- Means user provides both what data to be retrieved and how data to be retrieved. e.g. Relational Algebra.

Non-Procedural Query Language

- In nonprocedural language, the user describes the desired information without giving a specific procedure for obtaining that information.

- What data to be retrieved e.g. Relational Calculus. **Tuple relational calculus, Domain relational calculus are** declarative query languages based on mathematical logic

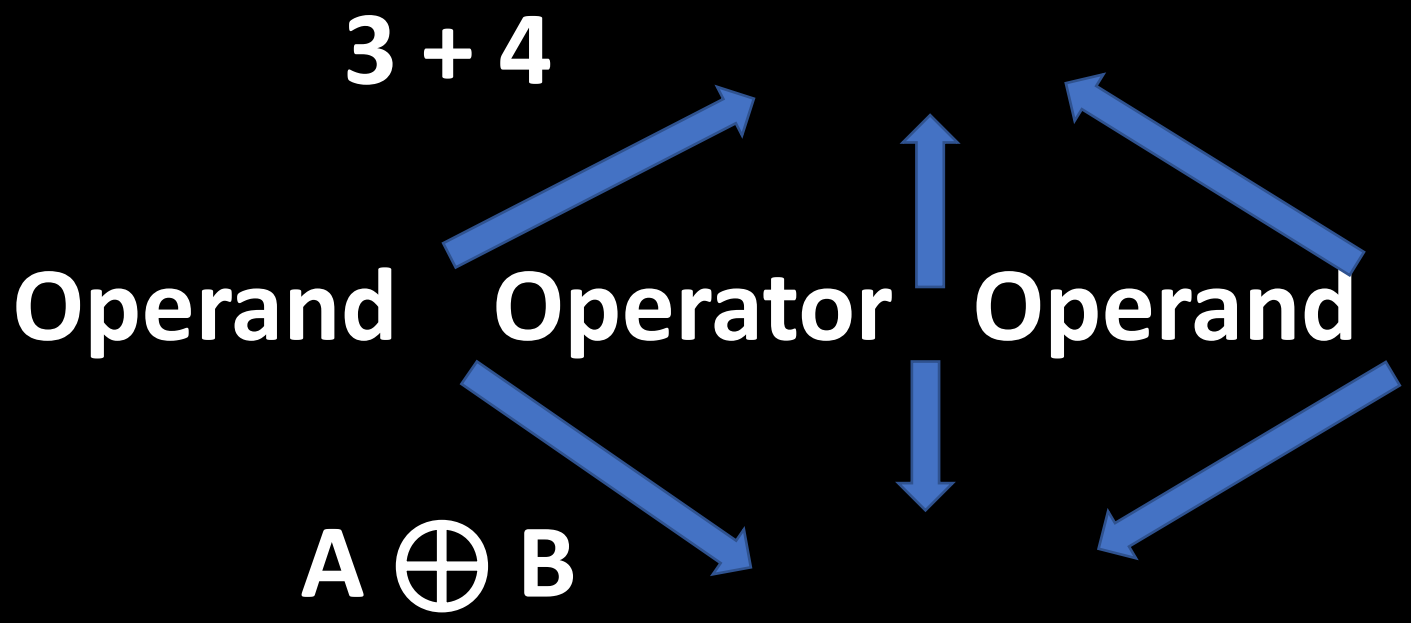
- Relational Algebra (Procedural) and Relational Calculus (non-procedural) are mathematical system/ query languages which are used for query on relational model.
- RA and RC are not executed in any computer they provide the fundamental mathematics on which SQL is based.
- SQL (structured query language) works on RDBMS, and it includes elements of both procedural or non-procedural query language.

Relational model	**RDBMS**
RA, RC	SQL
Algo	Code
Conceptual	Reality
Theoretical	Practical

Knowledge Gate Website

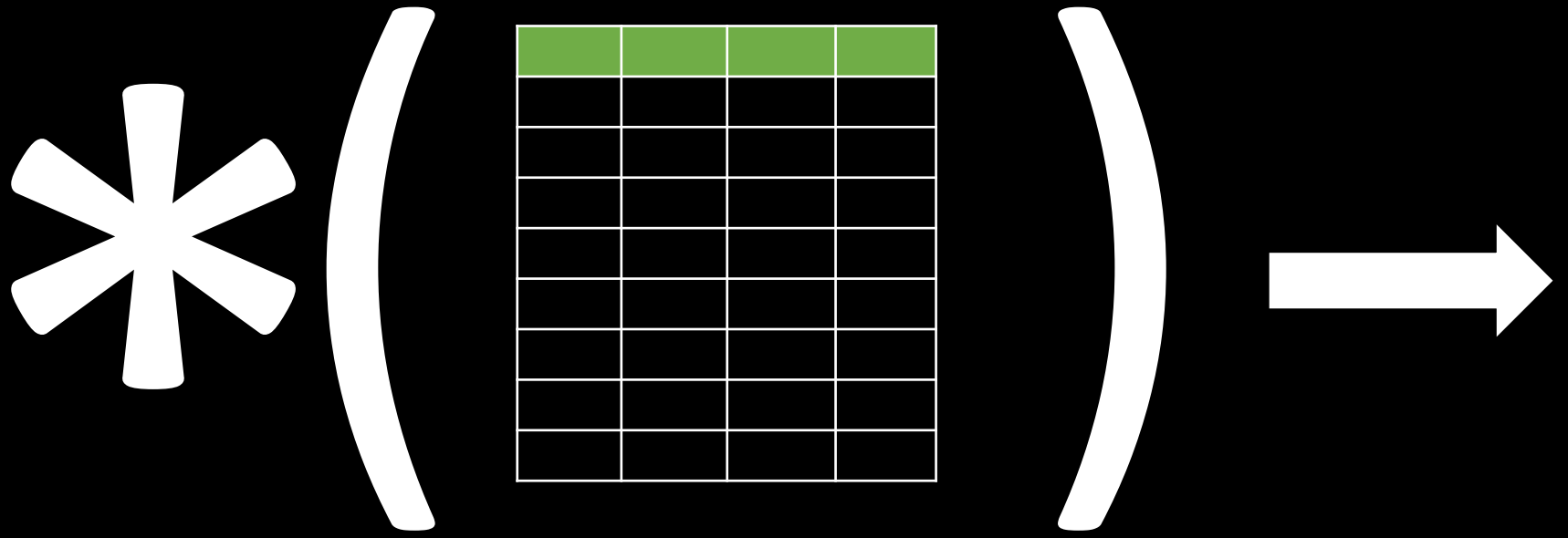
RELATIONAL ALGEBRA

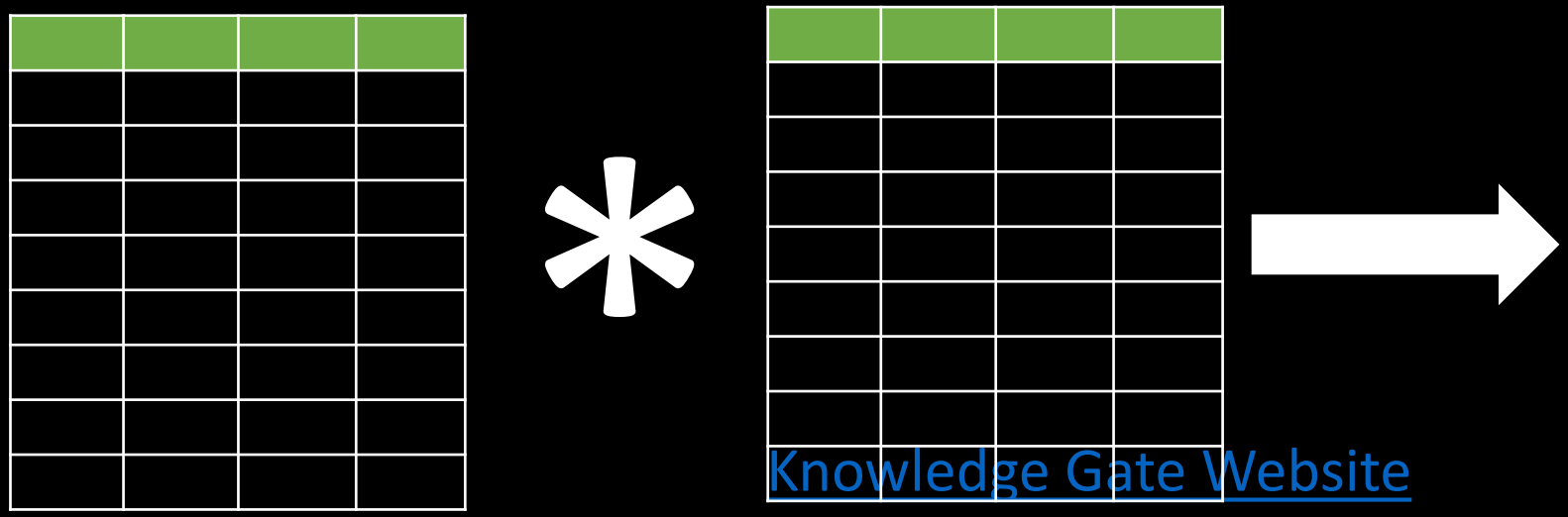
- RA like any other mathematical system provides a number of operators and use relations (tables) as operands and produce a new relation as their result.



Knowledge Gate Website

- Every operator in the RA accepts (one or two) relation/table as input arguments and returns always a single relation instance as the result without a name.





Knowledge Gate Website

- It also does not consider duplicity by default as it is based on set theory. Same query is written in RA and SQL the result may be different as SQL considers duplication.

- As it is pure mathematics no use of English keywords. Operators are represented using symbols.

BASIC / FUNDAMENTAL OPERATORS

- The fundamental operations in the relational algebra are **select, project, union, set difference, Cartesian product, and Rename.**

Name	Symbol
Select	(σ)
Project	(∏)
Union	(∪)
Set difference	(–)
Cross product	(X)
Rename	(ρ)

Knowledge Gate Website

DERIVED OPERATORS

- There are several other operations namely: **set intersection, natural join, and assignment.**

Name	Symbol	DERIVED FROM
Join	$(\bowtie)$	(X)
Intersection	$(\cap)$	(–) $A \cap B = A - (A - B)$
Division	$(\div)$	(X,-, ∏)
Assignment	$(=)$	

Knowledge Gate Website

- The **select, project, and rename** operations are called **unary operations**, because they operate on one relation.

- **Union, Cartesian product and set difference** operate on pairs of relations and are, therefore, called **binary operations.**

- Relational algebra also provides the framework for query optimization.

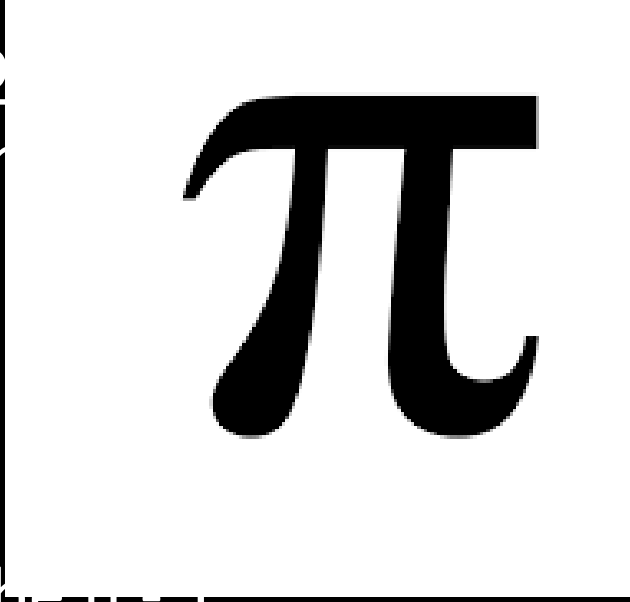
Relational schema - A **relation schema** R, denoted by R (A_1, A_2, ..., A_n), is made up of a relation name R and a list of attributes, A_1, A_2, ..., A_n. Each **attribute** A_i is the name of a role played by some domain D in the relation schema R. It is use to describe a Relation.

E.g. Schema representation of Table **Student** is as –

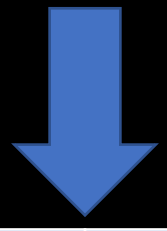
STUDENT (NAME, ID, CITY, COUNTRY, HOBBY).

Relational Instance - Relations with its data at particular instant of time.

The Project Operation (Vertical Selectio



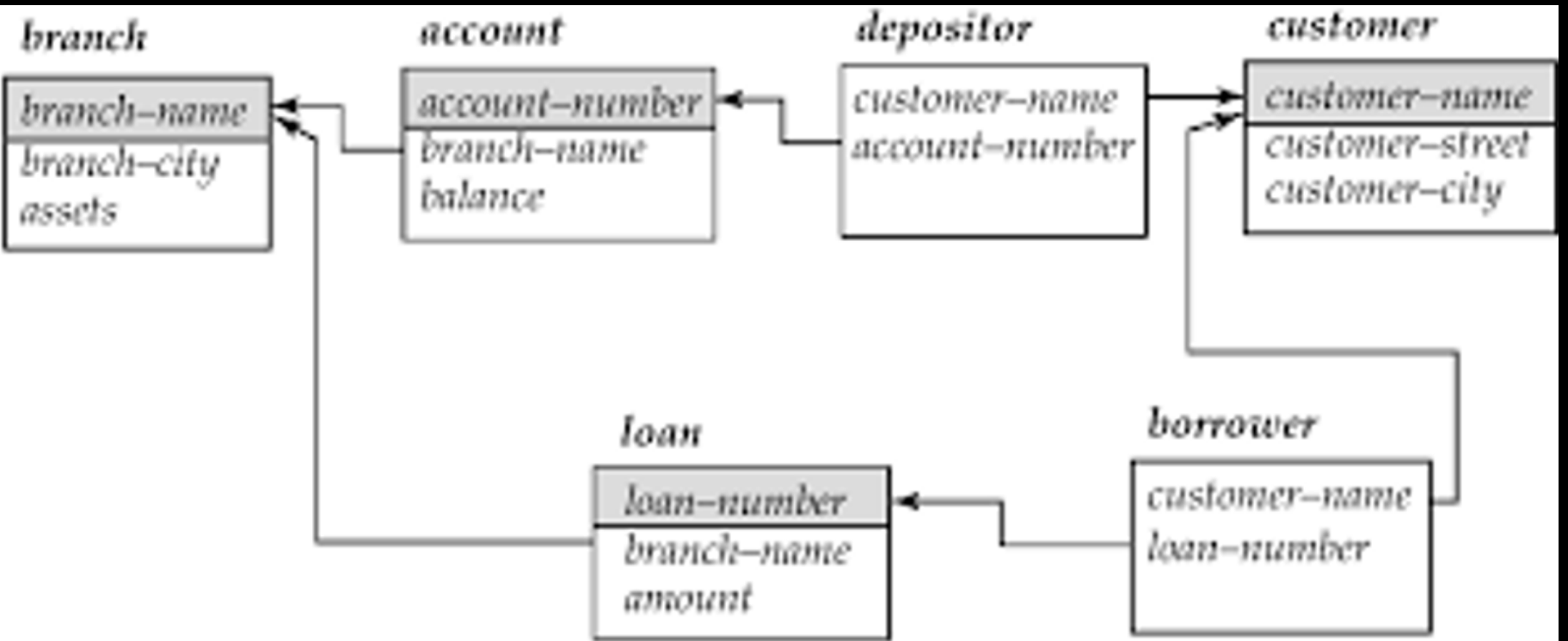
- Main idea behind project operator is to select desired columr
- The project operation is a unary operation that returns its argument relation, with certain attributes left out.
- Projection is denoted by the uppercase Greek letter pi (Π). M columns selected can be 1, Maximum selected Columns can be n - 1.
- **Π_{column_name} (table_name)**



Knowledge Gate Website

Q Write a RELATIONAL ALGEBRA query to find the name of all customer without duplication having bank account?

Q Write a RELATIONAL ALGEBRA query to find all the details of bank branches?

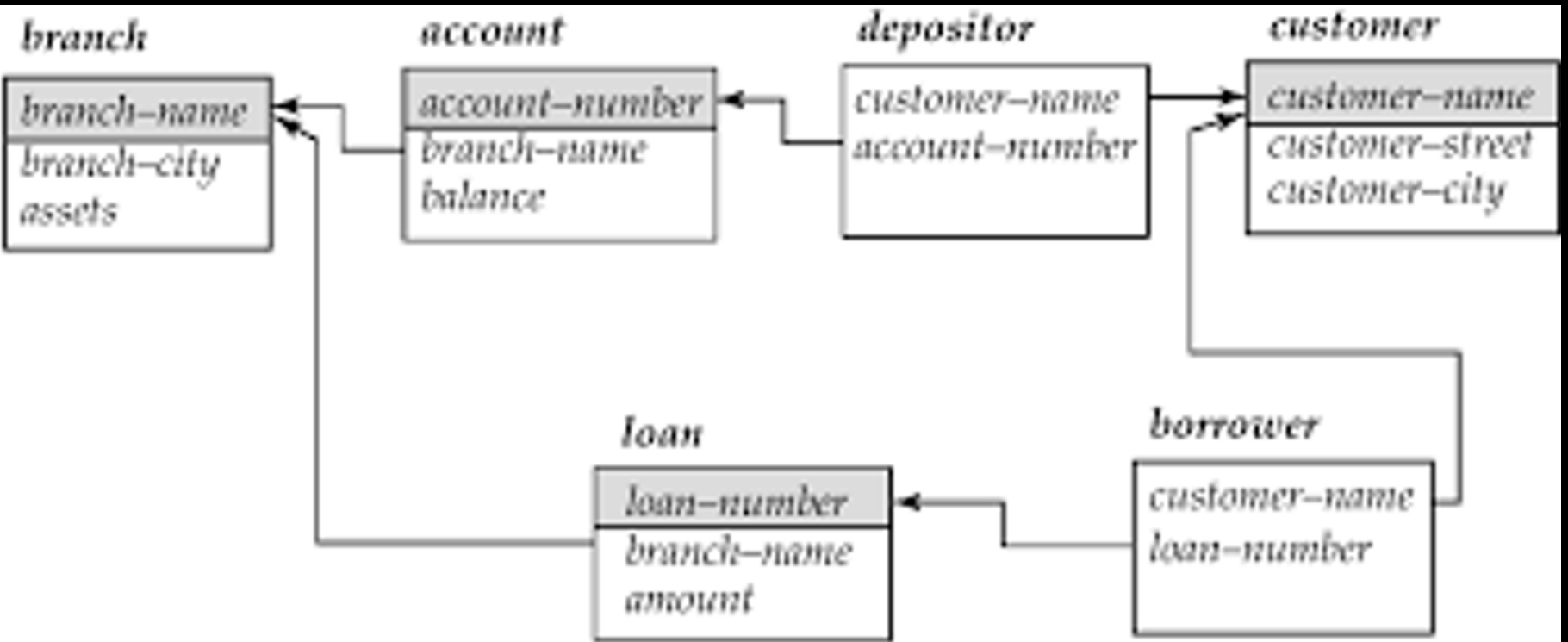


Q Write a RELATIONAL ALGEBRA query to find the name of all customer without duplication having bank account?

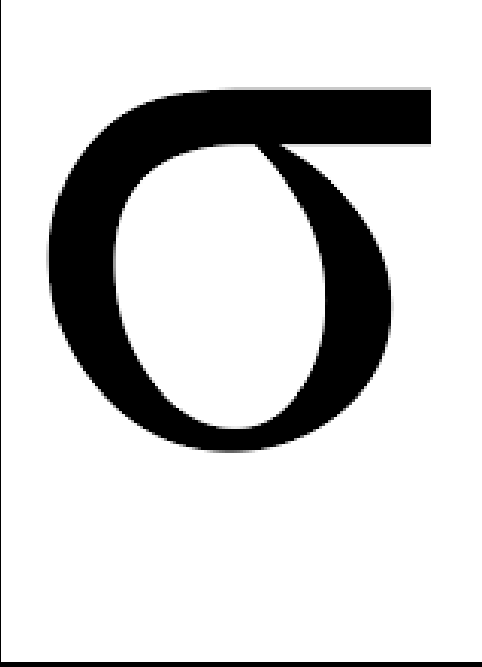
$$\Pi_{customer_name}\ (depositor)$$

Q Write a RELATIONAL ALGEBRA query to find all the details of bank branches?

$$(branch)$$



The Select Operation (Horizontal Selection)

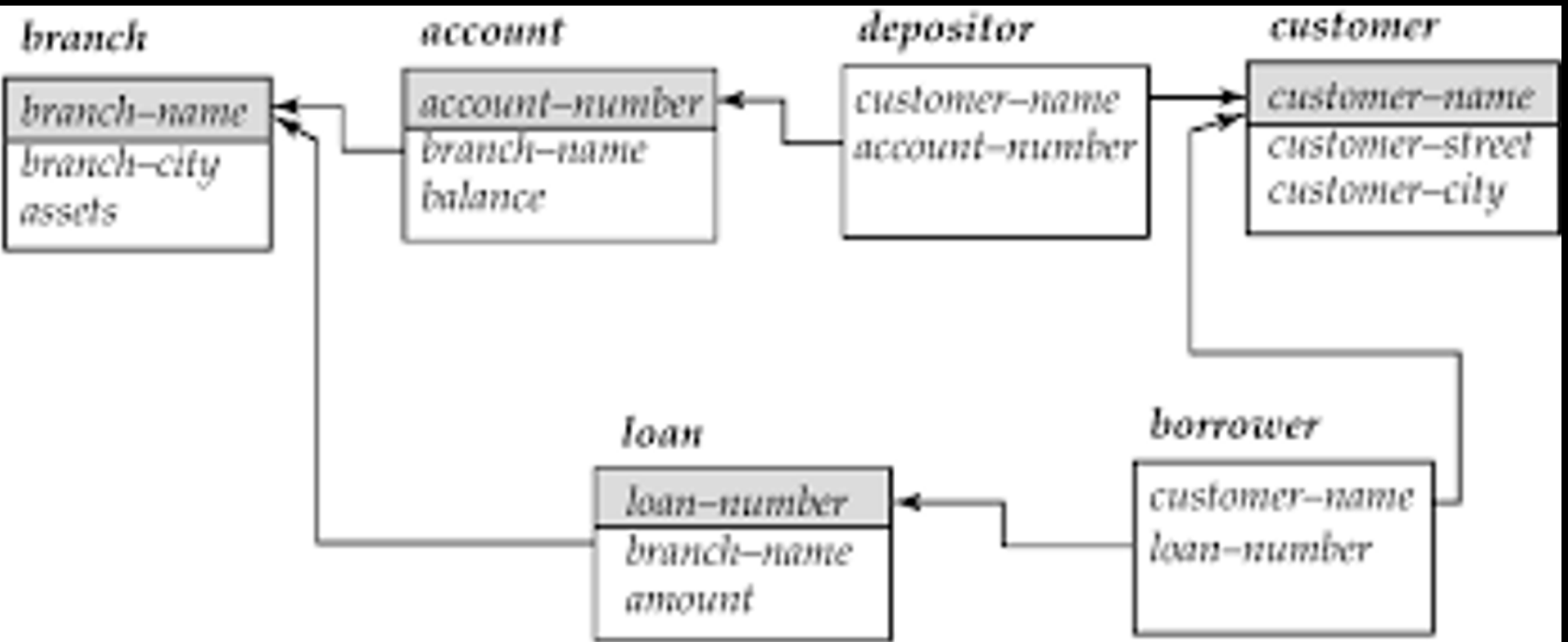


- The select operation selects tuples that satisfy a given predicate/Condition p.
- Lowercase Greek letter sigma (σ) is used to denote selection.
- It is a unary operator.
- Eliminates only tuples/rows.
- $\sigma_{condition}$ (table_name)

Knowledge Gate Website

Q Write a RELATIONAL ALGEBRA query to find all account_no where balance is less the 1000?

Q Write a RELATIONAL ALGEBRA query to find branch name which is situated in Delhi and having assets less than 1,00,000?

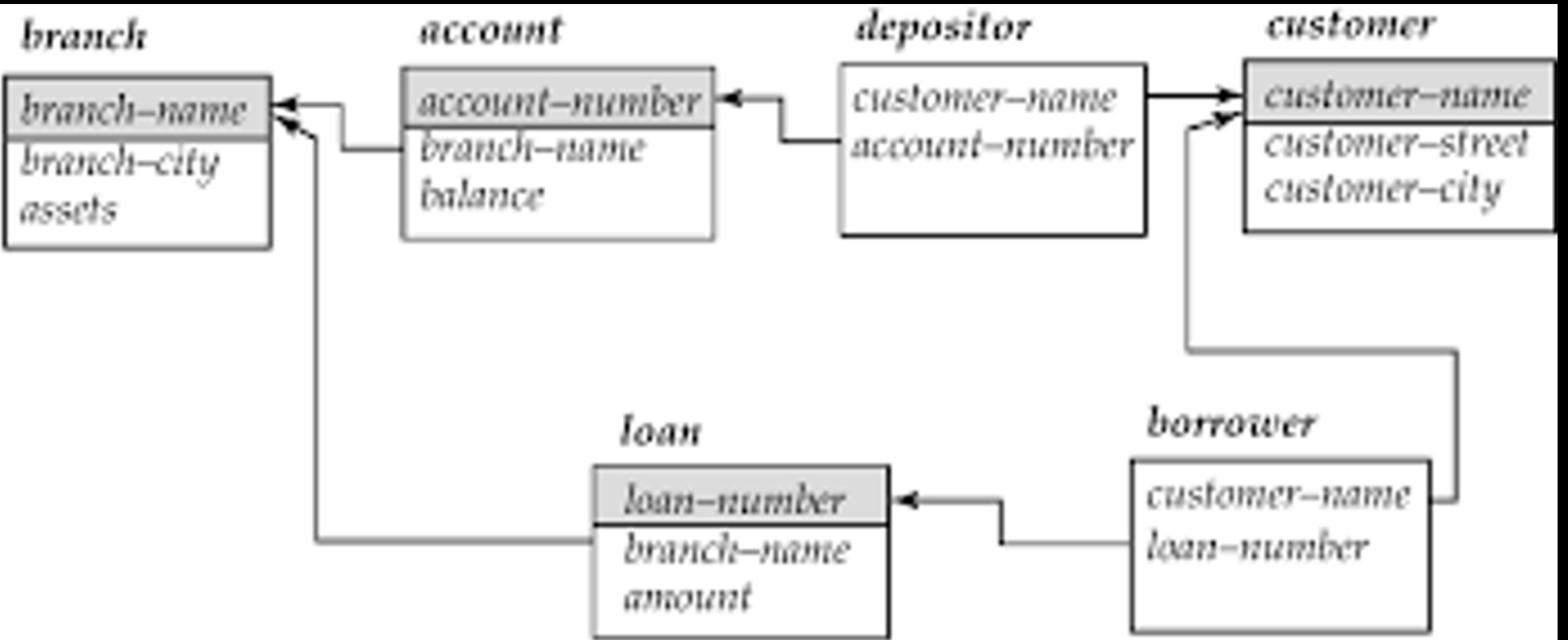


Q Write a RELATIONAL ALGEBRA query to find all account_no where balance is less the 1000?

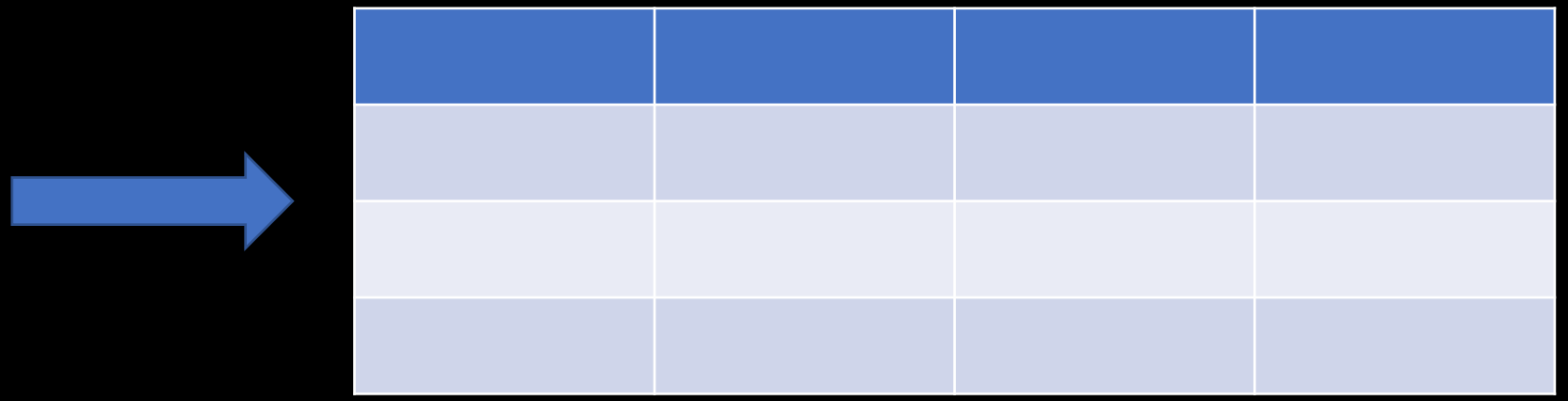
$$\Pi_{account_number}(\sigma_{balance<1000}(account))$$

Q Write a RELATIONAL ALGEBRA query to find branch name which is situated in Delhi and having assets less than 1,00,000?

$$\Pi_{branch_name}(\sigma_{(branch_city = delhi) \wedge (assets < 1000)}(branch))$$



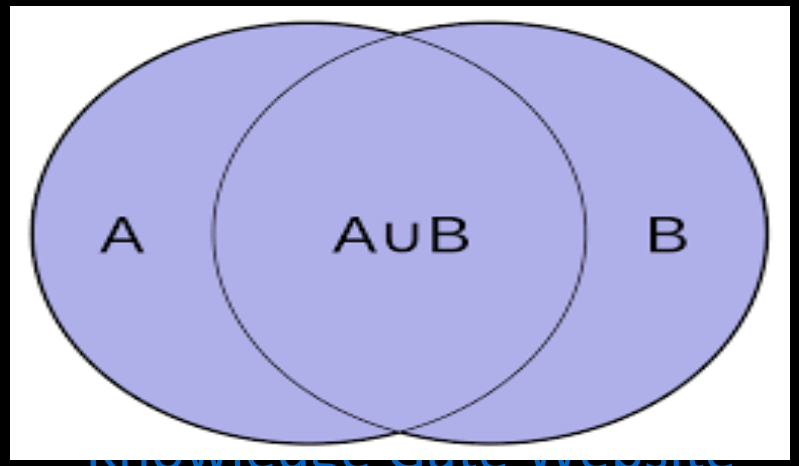
- Commutative in Nature, $\sigma_{p1 \wedge p2}(r) = \sigma_{p2 \wedge p1}(r) = \sigma_{p1}(\sigma_{p2}(r)) = \sigma_{p2}(\sigma_{p1}(r))$
- We allow comparisons using =, ≠, <, >, ≤ and ≥ in the selection predicate.
- Using the connectives and (∧), or (∨), and not (¬), we can combine several predicates into a larger predicate.
- Minimum number of tuples selected can be 0, Maximum selected tuples can be all.



Knowledge Gate Website

The Union Operation

- It is a binary operation, denoted, as in set theory, by ∪.
- Written as, $Expression_1 \cup Expression_2$, $r \cup s = \{t \mid t \in r \text{ or } t \in s\}$
- For a union operation r ∪ s to be valid, we require that two conditi
  - The relations r and s must be of the same arity. That is, they mu of attributes.
  - The domains of the i^{th} attribute of r and the i^{th} attribute of s must be the same, for all i.



Knowledge Gate Website

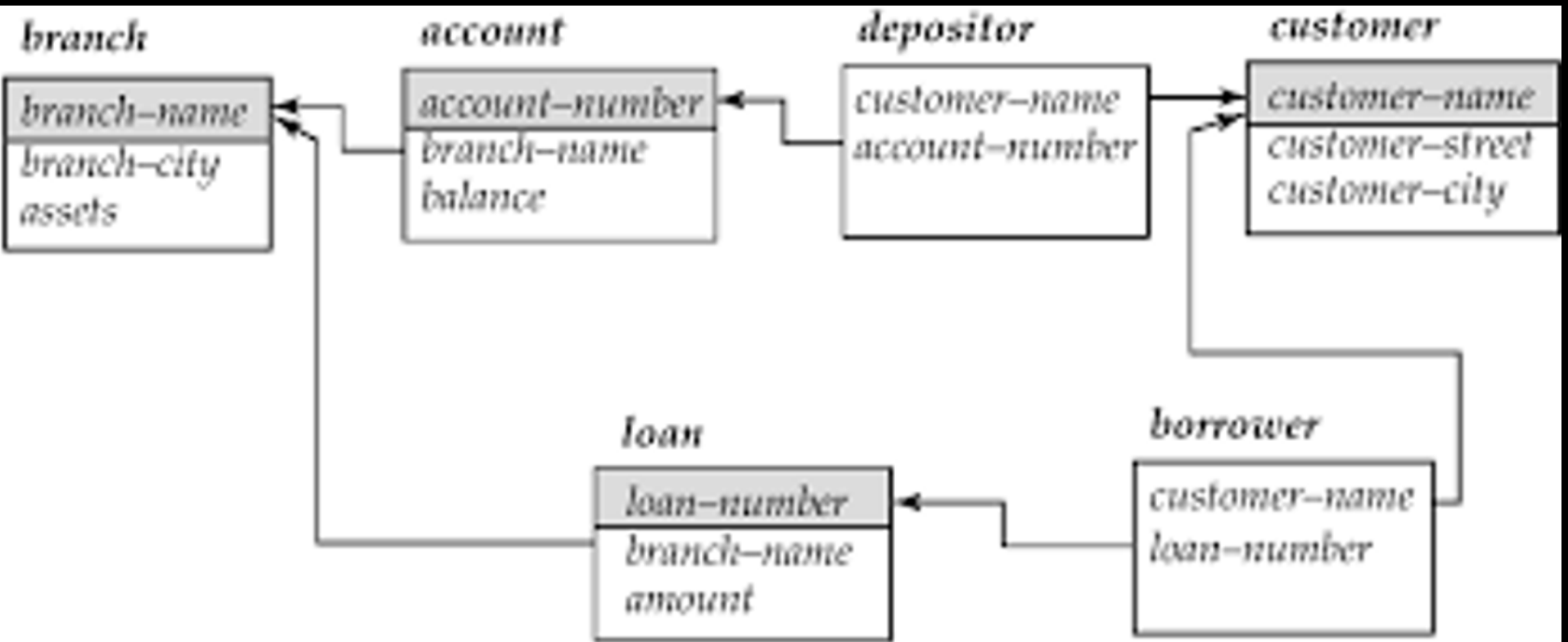
- **Some points to remember**

  - Deg (R ∪ S) = Deg(R) = Deg(S)

  - Max (IRI, ISI) <= IRUSI <= (IRI+ISI)

Q Write a RELATIONAL ALGEBRA query to find all the customer name who have a loan or an account or both?

Q Write a RELATIONAL ALGEBRA query to find all the customer name who have a loan but do not have an account?

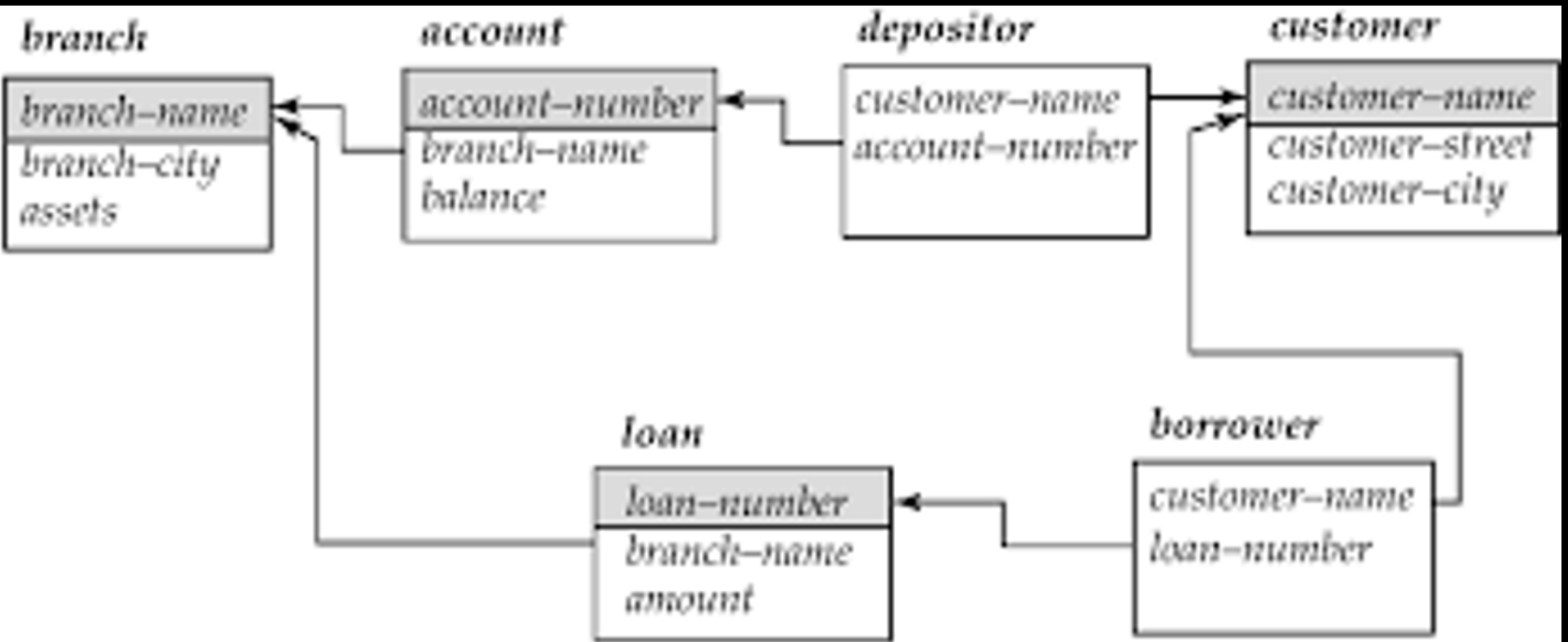


Q Write a RELATIONAL ALGEBRA query to find all the customer name who have a loan or an account or both?

$\Pi_{customer_name}(depositor)) \cup \Pi_{customer_name}(borrower))$

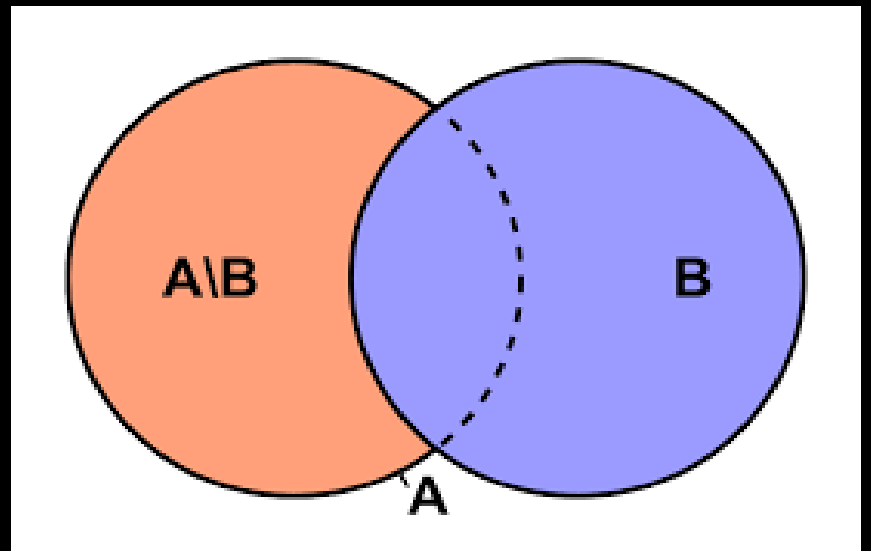
Q Write a RELATIONAL ALGEBRA query to find all the customer name who have a loan but do not have an account?

$\Pi_{customer_name}(borrower)) - (\Pi_{customer_name}(depositor)))$



The Set-Difference Operation

- The set-difference operation, denoted by −, allows us to find tuples that are in one relation but are not in another. It is a binary operator.
- The expression r − s produces a relation containing those tuples in r but not in s.
- We must ensure that set differences are taken between compatible relations.
- For a set-difference operation r − s to be valid, we require that the relations r and s be of the same arity, and that the domains of the ith attribute of r and the ith attribute of s be the same, for all i.
  - 0 <= IR - SI <= IRI



The Cartesian-Product Operation

- The Cartesian-product operation, denoted by a cross (×), allows us to combine information from any two relations.
- It is a binary operator; we write the Cartesian product of relations R_1 and R_2 as $R_1 \times R_2$.
- Cartesian-product operation associates every tuple of R_1 with every tuple of R_2.
  - R_1 X R_2 = {rs | r ∈ R_1 and s ∈ R_2}, contains one tuple <r, s> (concatenation of tuples r and s) for each pair of tuples r ϵ R_1, s ϵ R_2.

R_1	
A	**B**
1	P
2	Q
3	R

R_2	
B	**C**
Q	X
R	Y
S	Z

R_1 X R_2			
A	**R_1.B**	**R_2.B**	**C**

Knowle

The Cartesian-Product Operation

- The Cartesian-product operation, denoted by a cross (×), allows us to combine information from any two relations.
- It is a binary operator; we write the Cartesian product of relations R_1 and R_2 as $R_1 \times R_2$.
- Cartesian-product operation associates every tuple of R_1 with every tuple of R_2.
  - R_1 X R_2 = {rs | r ∈ R_1 and s ∈ R_2}, contains one tuple <r, s> (concatenation of tuples r and s) for each pair of tuples r ϵ R_1, s ϵ R_2.

R_1	
A	**B**
1	P
2	Q
3	R

R_2	
B	**C**
Q	X
R	Y
S	Z

R_1 X R_2			
A	**R_1.B**	**R_2.B**	**C**
1	P	Q	X
1	P	R	Y
1	P	S	Z
2	Q	Q	X
2	Q	R	Y
2	Q	S	Z
3	R	Q	X
3	R	R	Y
3	R	S	Z

Knowle

The Cartesian-Product Operation

- The Cartesian-product operation, denoted by a cross (×), allows us to combine information from any two relations.
- It is a binary operator; we write the Cartesian product of relations R_1 and R_2 as $R_1 \times R_2$.
- Cartesian-product operation associates every tuple of R_1 with every tuple of R_2.
  - R_1 X R_2 = {rs | r ∈ R_1 and s ∈ R_2}, contains one tuple <r, s> (concatenation of tuples r and s) for each pair of tuples r ϵ R_1, s ϵ R_2.

R_1	
A	**B**
1	P
2	Q
3	R

R_2	
B	**C**
Q	X
R	Y
S	Z

R_1 X R_2			
A	**R_1.B**	**R_2.B**	**C**
1	P	Q	X
1	P	R	Y
1	P	S	Z
2	Q	Q	X
2	Q	R	Y
2	Q	S	Z
3	R	Q	X
3	R	R	Y
3	R	S	Z

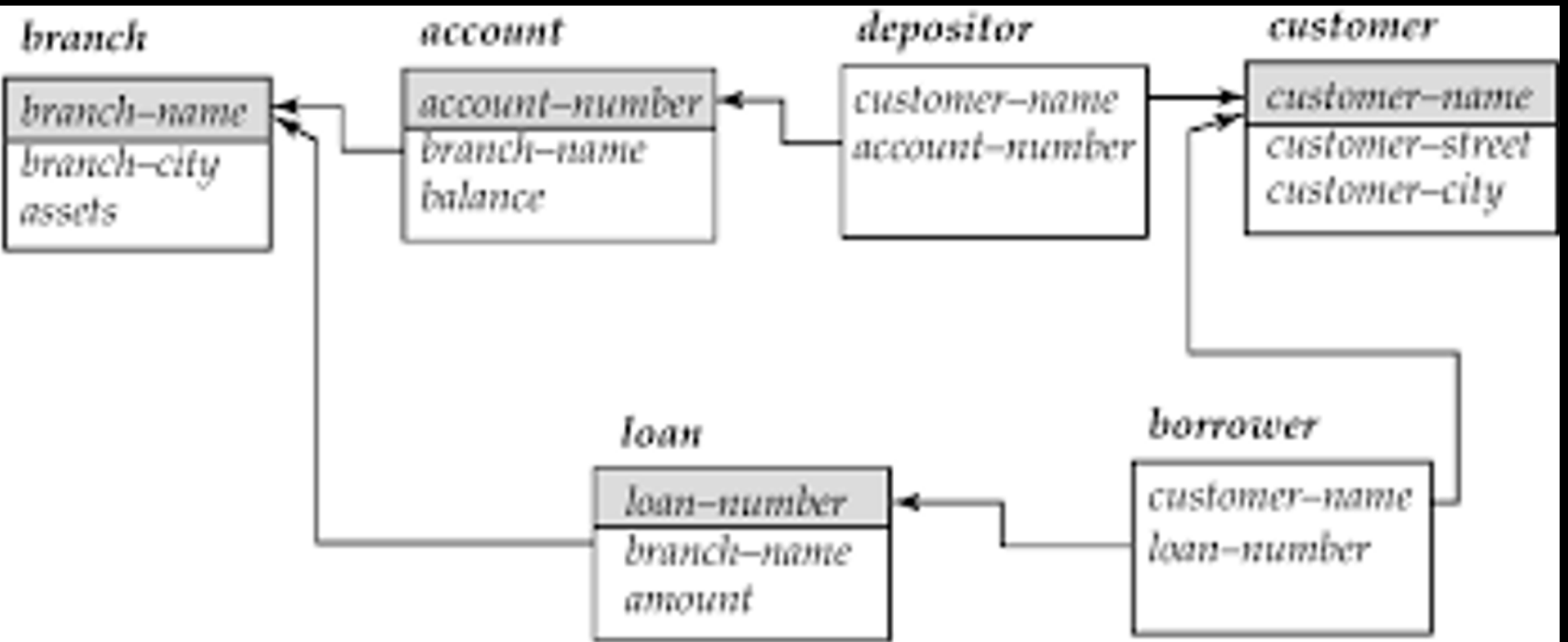
Knowle

- R_1 X R_2 returns a relational instance whose schema contains all the fields of R_1 (in order as they appear in R_1) and all fields of R_2 (in order as they appear in R_2).

- If R_1 has m tuples and R_2 has n tuples the result will be having = m*n tuples.

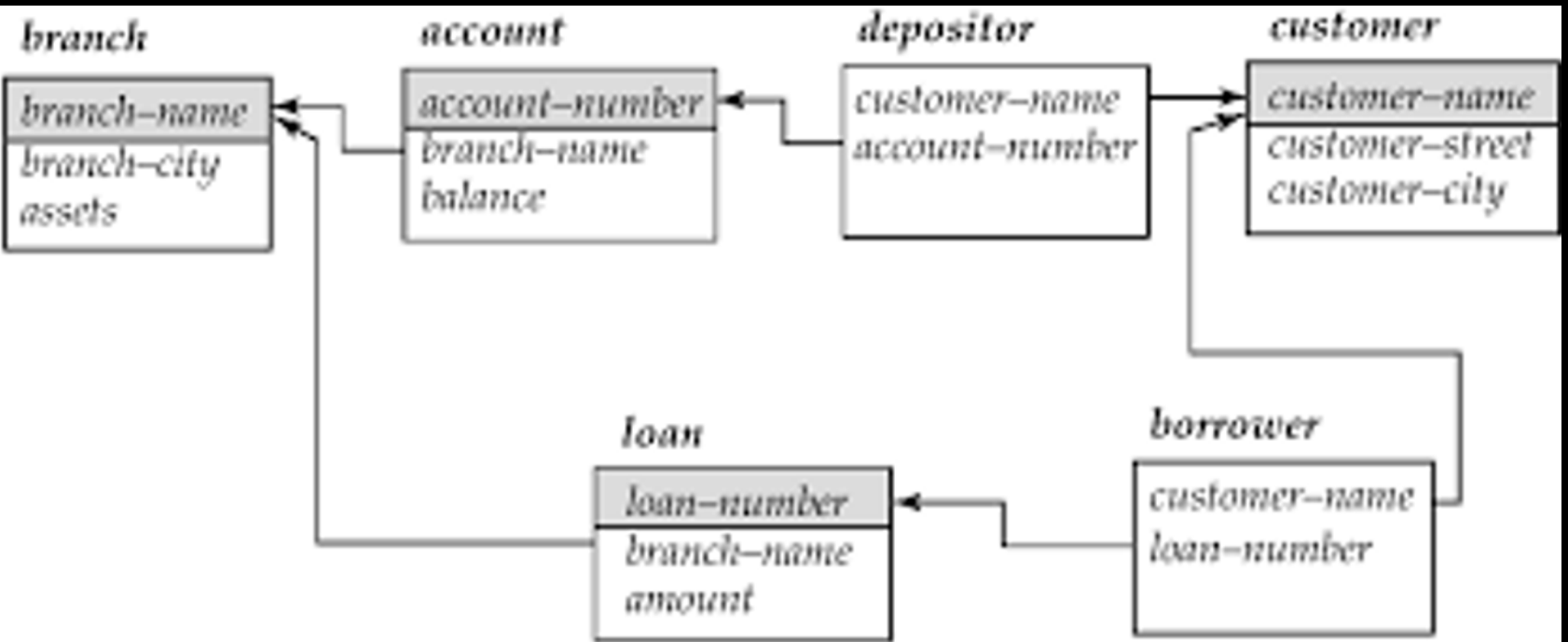
- Same attribute name may appear in both R_1 and R_2, we need to devise a naming schema to distinguish between these attributes.

Q Write a RELATIONAL ALGEBRA query to find the name of all the customers along with account balance, who have an account in the bank?



Q Write a RELATIONAL ALGEBRA query to find the name of all the customers along with account balance, who have an account in the bank?

$\Pi_{customer_name,balance}\ (\sigma_{account.account_number=depositor.account_number}\ (account\ X\ depositor))$



Rename Operation

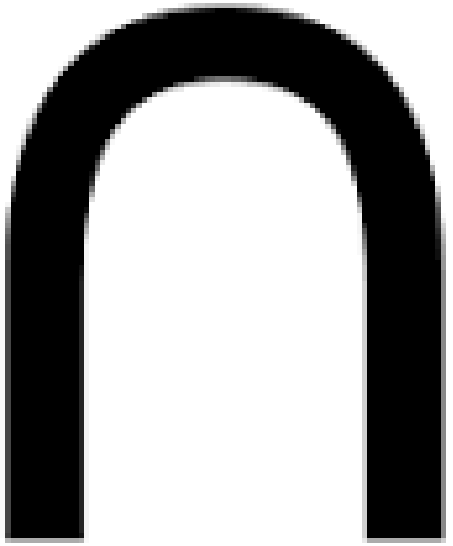
- The results of relational algebra are also relations but without any name. This Query do not change the name of the table in the original data base, but create a new copy of the table.
- The rename operation allows us to rename the output relation. It is denoted with small Greek letter **rho** ρ. Where the result of expression **E** is saved with name of **x**.
- $\rho_{x(A1, A2, A3, A4,.....AN)}(E)$
- $\rho_{Learner}(Student)$
- $\rho_{Learner(Stu_ID, User_Name, Age)}(Student(Roll_No, Name, Age))$

Knowledge Gate Website

Additional/Derived Relational-Algebra Operations

- If we restrict ourselves to just the fundamental operations, certain common queries are lengthy to express. Therefore, we use additional operations.
- These additional operations do not add any power to the algebra.
- They are used to simplify the queries.

Set-Intersection Operat

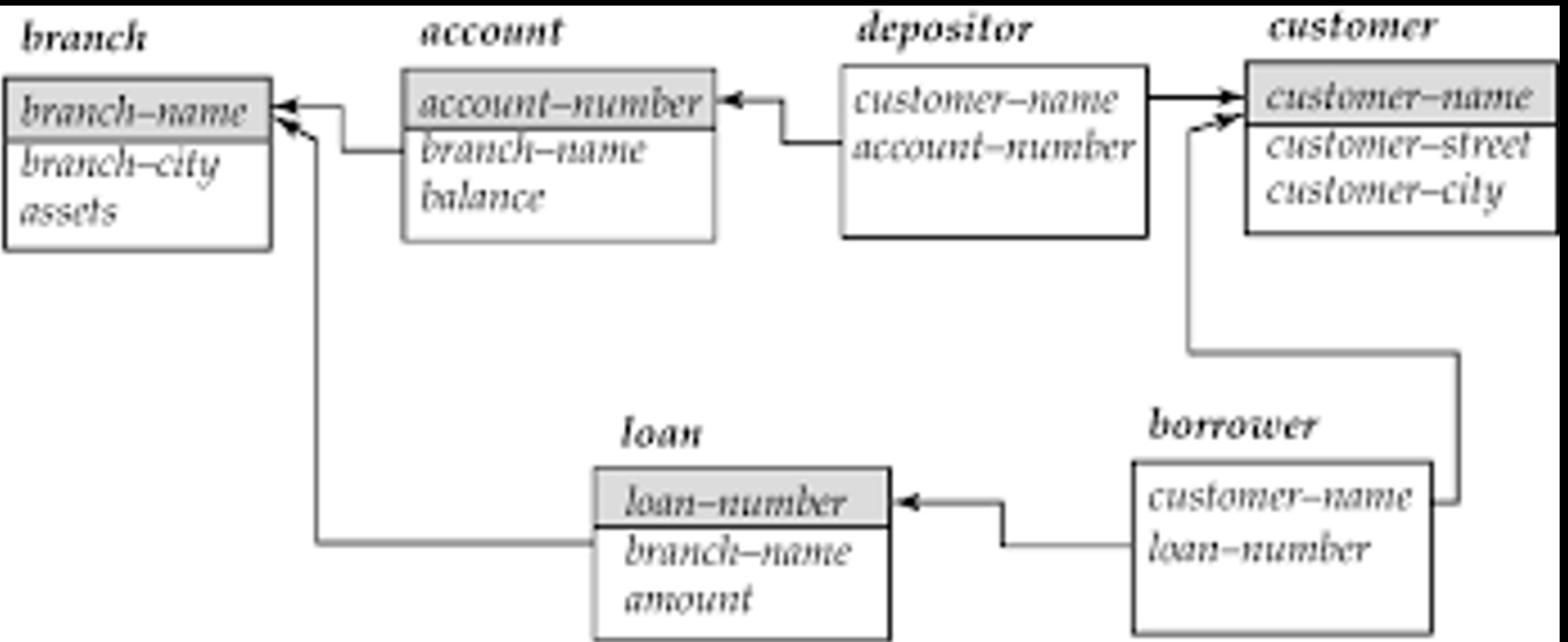


- We will be using ∩ symbol to denote set inter
- r ∩ s = r – (r – s)
- Set intersection is not a fundamental operatio power to the relational algebra.
- r ∩ s = {t | t ∈ r and t ∈ s}
- 0 <= I R∩S I <= min (IRI, ISI)

Knowledge Gate Website

Q Write a RELATIONAL ALGEBRA query to find all the customer name who have both a loan and an account?

$\Pi_{customer_name}(depositor)) \cup \Pi_{customer_name}(borrower))$



The Natural-Join Operation * ⋈

- The natural join is a binary operation that allows us to combine certain selections and a Cartesian product into one operation.

he natural join is a Lossy operator.

R_1	
A	**B**
1	P
2	Q
3	R

R_2	
B	**C**
Q	X
R	Y
S	Z

$R_1 \bowtie R_2$		
A	**B**	**C**
2	Q	X
3	R	Y

Knowledge Gate Website

- In general, the DIVISION operation is applied when we have query like student who have completed both database1 and data base2 tasks.

Completed

Student	Task
Fred	Database1
Fred	Database2
Fred	Compiler1
Eugene	Database1
Eugene	Compiler1
Sarah	Database1
Sarah	Database2

DBProject

Task
Database1
Database2

$$\pi_{Student}(R) - \{\pi_{Student}[(\pi_{Student}(R) \times S) - \pi_{Student,Task}(R)]\}$$

Completed

Student	Task
Fred	Database1
Fred	Database2
Fred	Compiler1
Eugene	Database1
Eugene	Compiler1
Sarah	Database1
Sarah	Database2

DBProject

Task
Database1
Database2

Completed ÷ ***DBProject***

Student
Fred
Sarah

Knowledge Gate Website

R

A	B
a1	b1
a2	b1
a3	b1
a4	b1
a1	b2
a3	b2
a2	b3
a3	b3
a4	b3
a1	b4
a2	b4
a3	b4

S

A
a1
a2
a3

T

B
b1
b4

Knowledge Gate Website

Introduction to SQL

- Structured Query Language is a domain-specific language (not general purpose) used in programming and design for managing data held in a relational database management system (RDBMS).

- Although we refer to the SQL language as a “query language,” it can do much more than just query a database. It can define the structure of the data base, modify data in the database, specify security constraints and number of other tasks.

- Originally based upon relational algebra(procedural) and tuple relational calculus (Non-procedural) mathematical model.

Overview of the SQL Query Language

1. IBM developed the original version of SQL, originally called Sequel (*Structured English Query Language*), as part of the System R project in the early 1970s.

2. The Sequel language has evolved since then, and its name has changed to SQL (Structured Query Language) (some other company has trademark on the word sequel). SQL has clearly established itself as *the* standard relational database language.

3. In 1986, the American National Standards Institute (ANSI) and the International Organization for Standardization (ISO) published an SQL standard, called SQL-86.

4. The next version of the standard was SQL-89, SQL-92, SQL:1999, SQL:2003, SQL:2006, SQL:2008, SQL:2011, SQL: 2016, SQL: 2019and most recently SQL:2023.

Knowledge Gate Website

Classification of database languages

1. **Data Definition Language (DDL)** :
    1. a. DDL is set of SQL commands used to create, modify and delete database structures but not data.
    2. b. They are used by the DBA to a limited extent, a database designer, or application developer.
    3. c. Create, drop, alter, truncate are commonly used DDL command. CREATE, ALTER, DROP, TRUNCATE, COMMENT, GRANT, REVOKE statement

2. **Data Manipulation Language (DML)** :
    1. a. A DML is a language that enables users to access or manipulates data as organized by the appropriate data model.
    2. b. There are two types of DMLs :
        1. i. Procedural DMLs : It requires a user to specify what data are needed and how to get those data.
        2. ii. Declarative DMLs (Non-procedural DMLs) : It requires a user to specify what data are needed without specifying how to get those data.
    3. c. Insert, update, delete, query are commonly used DML commands. INSERT, UPDATE, DELETE statement

Knowledge Gate Website

3. **Data Control Language (DCL)** :
    1. a. It is the component of SQL statement that control access to data and to the database.
    2. b. Commit, rollback command are used in DCL. GRANT and REVOKE statement

4. **Data Query Language (DQL)** :
    1. a. It is the component of SQL statement that allows getting data from the database and imposing ordering upon it.
    2. b. It includes select statement. SELECT statement

5. **View Definition Language (VDL)** :
    1. 1. VDL is used to specify user views and their mapping to conceptual schema.
    2. 2. It defines the subset of records available to classes of users.
    3. 3. It creates virtual tables and the view appears to users like conceptual level.
    4. 4. It specifies user interfaces. SQL is a DML language.

Knowledge Gate Website

```
CREATE TABLE table_name (
    column1 data_type [constraints],
    column2 data_type [constraints],
    column3 data_type [constraints],
    ...
);


CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Age INT,
    Email VARCHAR(100)
);
```

Knowledge Gate Website

- list of some common data types supported by SQL along with a brief description of each:

- Numeric Data Types:
  - 1. `INT`: For storing integer values.
  - 2. `SMALLINT`: A smaller range of integers compared to INT.
  - 3. `BIGINT`: For storing larger integers.
  - 4. `DECIMAL(p, s)`: For storing exact numerical values, where `p` is the precision and `s` is the scale.
  - 5. `FLOAT`: For storing floating-point numbers.
  - 6. `REAL`: A data type that can store floating-point numbers, generally with less precision compared to FLOAT.

- String Data Types:
  - 7. `VARCHAR(n)`: Variable-length character string, where `n` is the maximum length.
  - 8. `CHAR(n)`: Fixed-length character string, where `n` is the length.
  - 9. `TEXT`: For storing long text strings.

Knowledge Gate Website

Adding a New Column

```
ALTER TABLE Employees
ADD PhoneNumber VARCHAR(15);
```

Dropping a Column

```
ALTER TABLE Employees
DROP COLUMN PhoneNumber;
```

Modifying an Existing Column

```
ALTER TABLE Employees
MODIFY COLUMN PhoneNumber VARCHAR(20);

ALTER TABLE Employees
ALTER COLUMN PhoneNumber VARCHAR(20);
```

Renaming a Column

```
ALTER TABLE Employees
RENAME COLUMN PhoneNumber TO ContactNumber;
```

Renaming a table

```
ALTER TABLE Employees
RENAME TO Staff;

DROP TABLE table_name;
```

[Knowledge Gate Website]

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    OrderDate DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customers (CustomerID)
);


ALTER TABLE Orders
ADD FOREIGN KEY (CustomerID) REFERENCES Customers (CustomerID);
```

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

```
INSERT INTO Students (StudentID, FirstName, LastName, Age)
VALUES
(1, 'Amit', 'Sharma', 20),
(2, 'Priya', 'Gupta', 22),
(3, 'Ravi', 'Kumar', 19);
```

```
DELETE FROM table_name
WHERE condition;
```

```
DELETE FROM Students
WHERE StudentID = 1;
```

```
DELETE FROM table_name;
```

Basic Structure of SQL Queries

- For any SQL as query, input and output both are relations. Number of relations inputs to a query will be at least one, but output will always be a single relation without any name unless specified, but columns will have names from input tables.

- The basic structure of an SQL query consists of three clauses: select, from, and where. The query takes it's input the relations listed in the from clause, operates on them as specified in the where and select clauses, and then produces a relation as the result without any name unless specified.

- A typical SQL query has the form.

Select A_1, A_2,..., A_n (Column name)
from r_1, r_2,... , r_m (Relation/table name)
Where P; (Condition)

Knowledge Gate Website

Select A_1, A_2,..., A_n (Column name)
from r_1, r_2,... , r_m (Relation/table name)
Where P; (Condition)

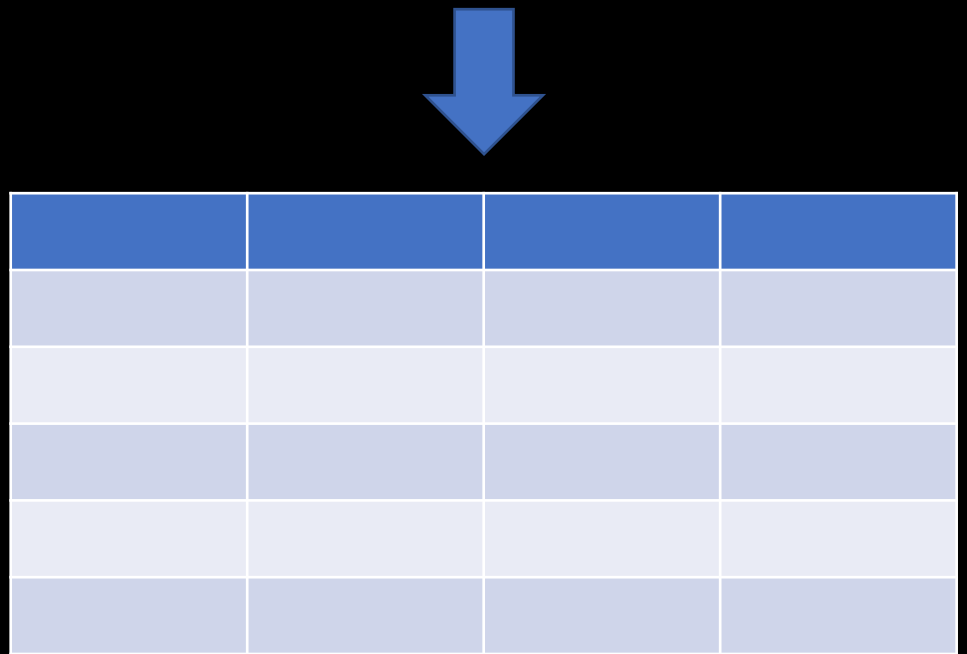
1. It is to be noted only select and from are mandatory clauses, and if not required then it is not essential to write where. If the **where** clause is omitted, the predicate *P* is **true**.

2. SQL in general is not case sensitive i.e. it doesn't matter whether we write query in upper or lower case.

3. In practice, duplicate elimination is time-consuming. Therefore, SQL allows duplicates in relations as well as in the results of SQL expressions. In those cases where we want to force the elimination of duplicates, we insert the keyword **distinct** after **select**, will discuss in detail later. SQL allows us to use the keyword **all** to specify explicitly that duplicates are not removed, Since duplicate retention is the default, we shall not use **all** in our examples.

Select Clause

- The function of Select clause in SQL is more or less same as that of '∏' projection in the relational algebra. It is used to pick the column required in result of the query out of all the columns in relation/table. (Vertical filtering)
  - **Select A_1, A_2,..., A_n (Column name)**
- We can use '*' to specify that we need all columns
  - **Select ***
- The **select** clause may also contain arithmetic expressions involving the operators +, -, / and * operating on constants or attributes of tuples. however, that it does not result in any change to the relation/table.

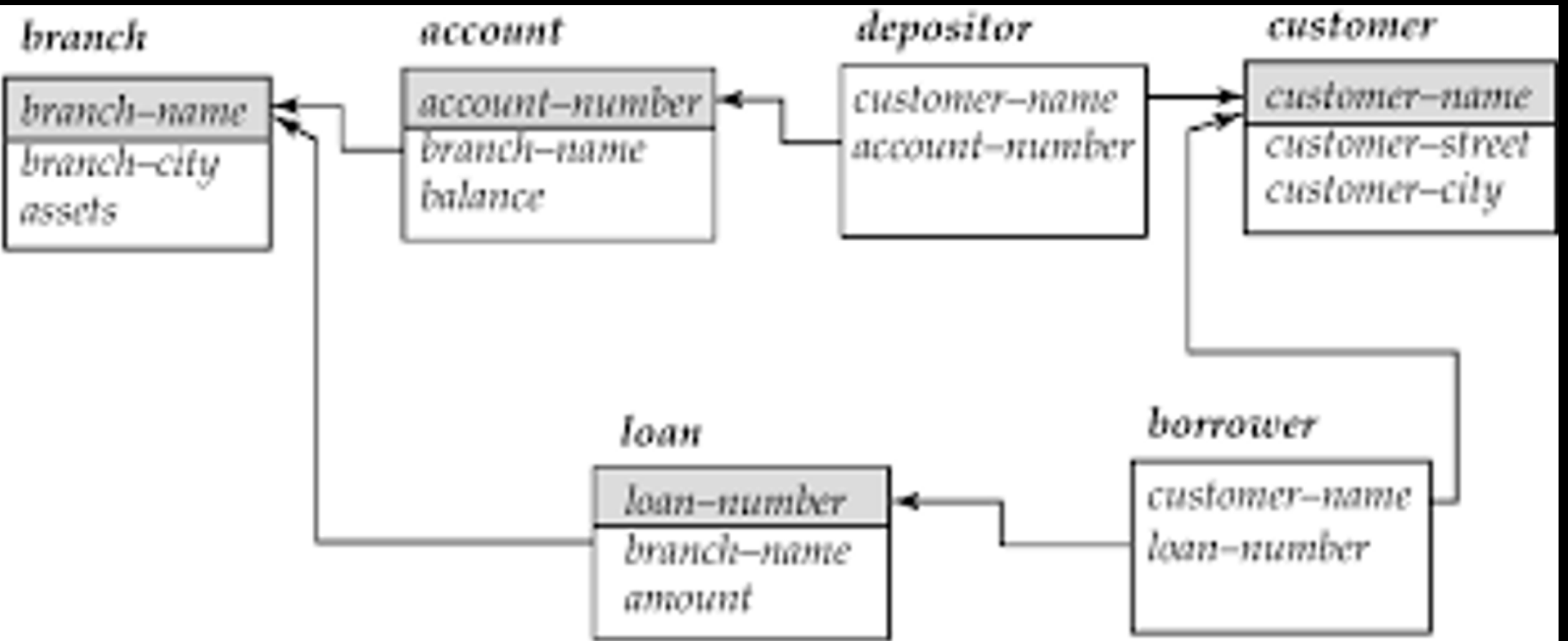


Q Write a SQL query to find all the details of bank branches?

Q Write a SQL query to find each loan number along with loan amount?

Q Write a SQL query to find the name of all customer without duplication having bank account?

Q Write a SQL query to find all account_no and balance with 6% yearly interest added to it?



Q find all the details of bank branches?

```
Select  *
from branch
```

Q find each loan number along with loan amount?

```
Select loan_number, amount
from loan
```

Q find the name of all customer without duplication having bank account?

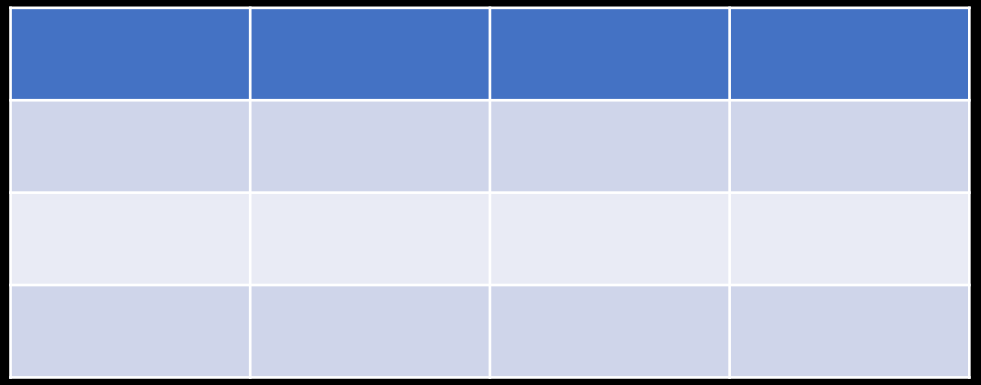
```
Select distinct customer_name
from depositor
```

Q find all account_no and balance with 6% yearly interest added to it?

```
Select account_number, balance*1.06
from account
```

Select Clause with where clause

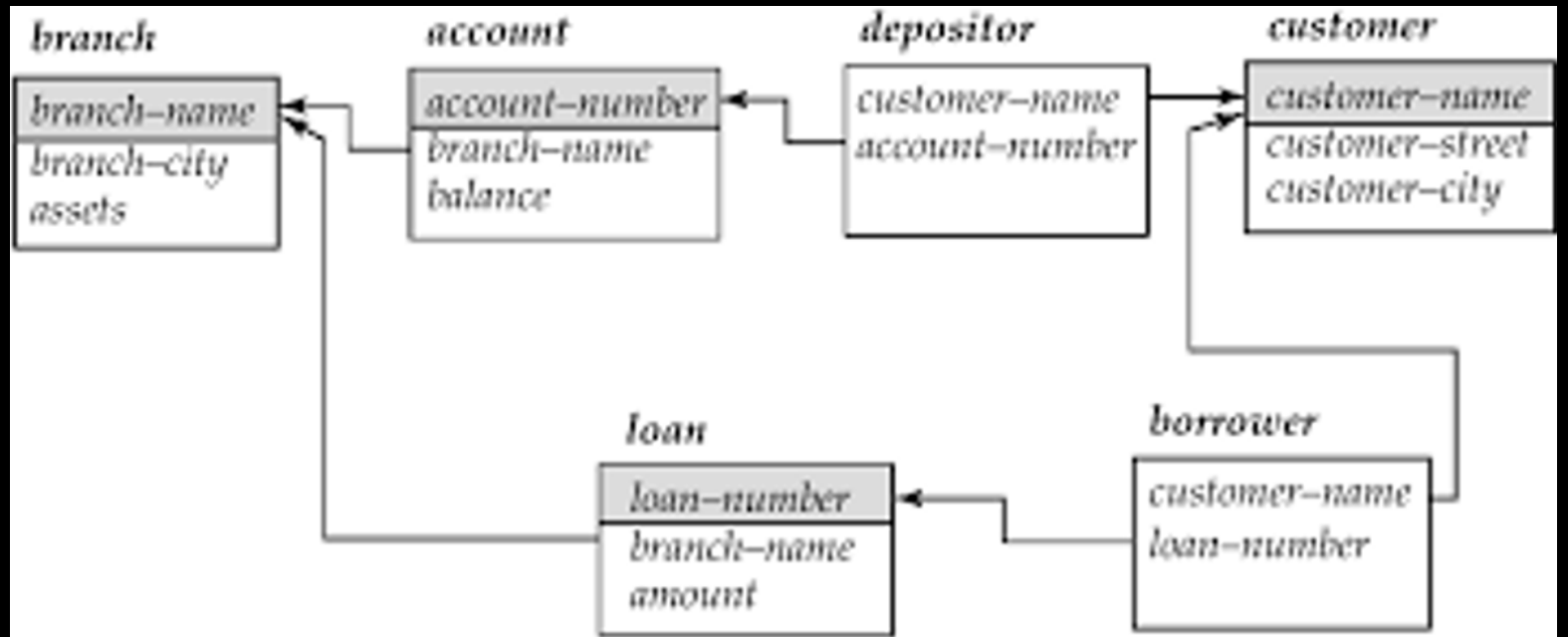
1. Where clause in SQL is same as ‘σ’ sigma of relational algebra where we specify the conditions/Predicate (horizontal filtering).
2. Where clause can have expressions involving the comparison operators <, <, >, >=, <= and <>. SQL allows us to use the comparison operators to compare strings and arithmetic expressions.
3. SQL allows the use of the logical connectives **and**, **or**, and **not** in the **where** clause.
4. SQL includes a **between** comparison operator to simplify **where** clauses that specify that a value be less than or equal to some value and greater than or equal to some other value.
5. Similarly, we can use the **not between** comparison operator.



Q Write a SQL query to find all account_no where balance is less the 1000?

Q Write a SQL query to find branch name which is situated in Delhi and having assets less than 1,00,000?

Q Write a SQL query to find branch name and account_no which has balance greater than equal to 1,000 but less than equal to 10,000?



Q find all account_no where balance is less the 1000?

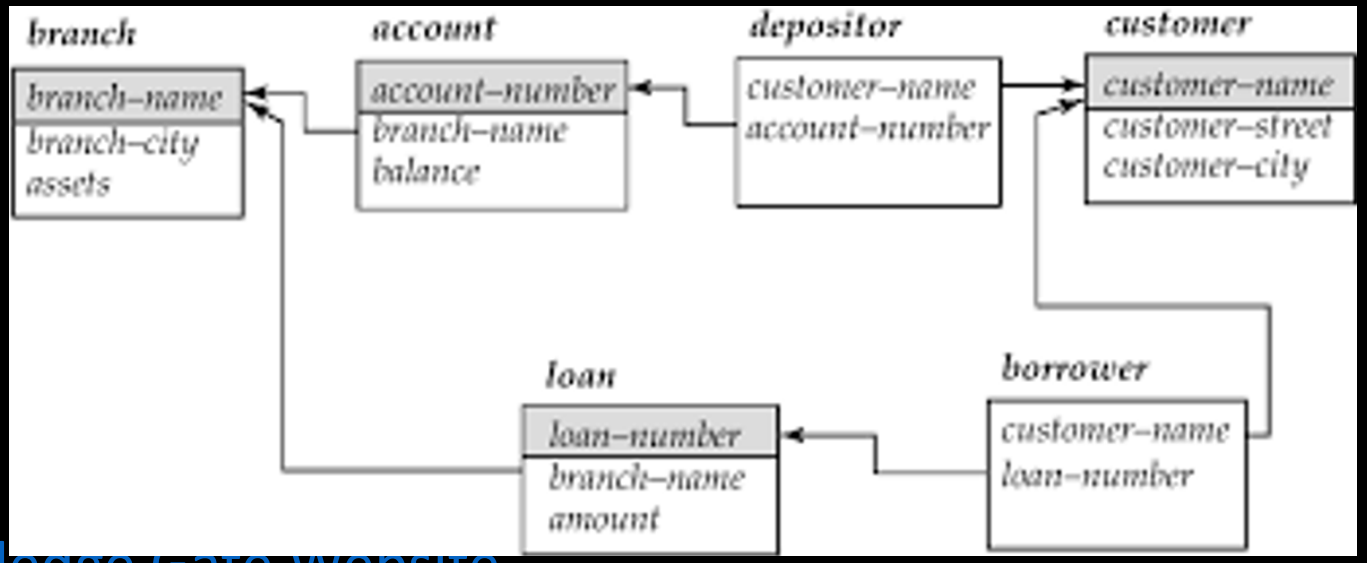
```
Select account_number
from account
Where balance < 1000
```

Q Write a SQL query to find branch name which is situated in Delhi and having assets less than 1,00,000?

```
Select branch_name
from branch
Where branch_city = ‘delhi’ and assets < 1,00,000
```

Q Write a SQL query to find branch name and account_no which has balance greater than equal to 1,000 but less than equal to 10,000?

```
Select branch_name, account_number
from account
Where balance between 1000 and 10000

Select branch_name, account_number
from account
Where balance >= 1000 and balance<=10000
```



Knowledge Gate Website

Set Operation

1. The SQL operations **union**, **intersect**, and **except/minus** operate on relations and corresponds to the mathematical set-theory operations ∪, ∩ and – respectively.

2. The **union** operation automatically eliminates duplicates, unlike the **select** clause, If we want to retain all duplicates, we must write **union all** in place of **union.**

3. The **intersect** operation automatically eliminates duplicates. If we want to retain all duplicates, we must write **intersect all** in place of intersect.

4. If we want to retain duplicates, we must write **except all** in place of **except.**

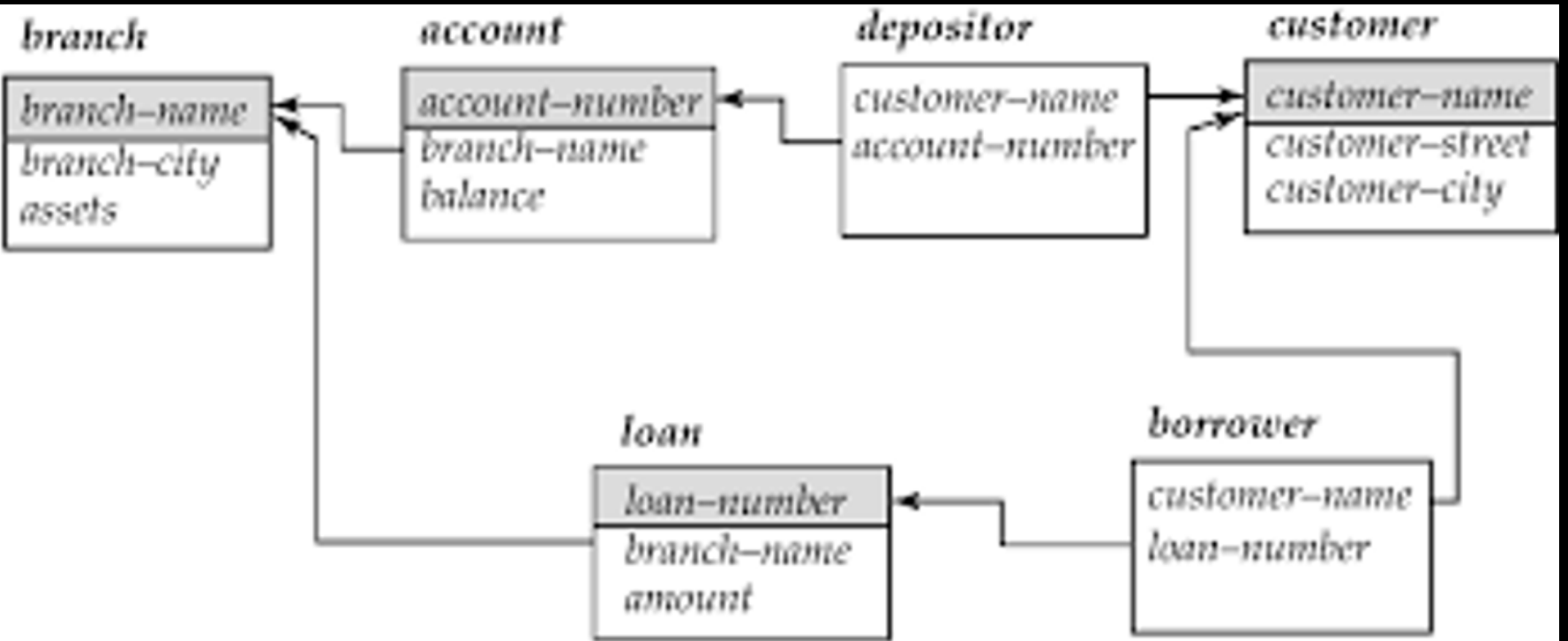
Knowledge Gate Website

Q Write a SQL query to find all the customer name

a) who have a loan or an account or both ?

b) who have both a loan and an account?

c) who have a loan but do not have an account?



Q Write a SQL query to find all the customer name

a) who have a loan or an account or both ?

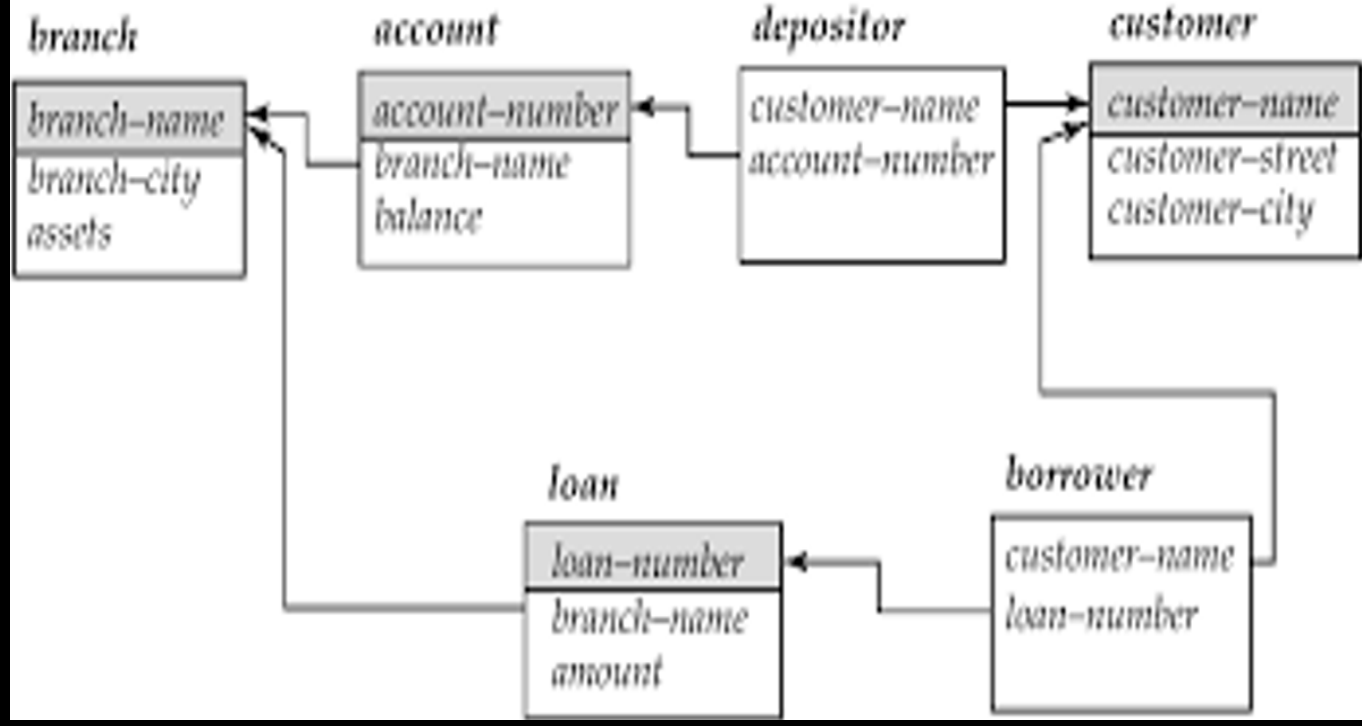
```
Select customer_name
From depositor
Union
Select customer_name
From borrower
```

b) who have both a loan and an account?

```
Select customer_name
From depositor
intersect
Select customer_name
From borrower
```

c) who have a loan but do not have an account?

```
Select customer_name
From borrower
Except
Select customer_name
From depositor
```



Knowledge Gate Website

Queries on Multiple Relations

- The **from** clause by itself defines a Cartesian product of the relations listed in the clause. Cartesian product of two relations, which concatenates each tuple of the first relation with every tuple of the second
- Since the same attribute name may appear in both r_1 and r_2, we prefix the name of the relation from which the attribute originally came, before the attribute name.For those attributes that appear in only one of the two schemas, we shall usually drop the relation-name prefix. This simplification does not lead to any ambiguity.
- Cartesian Product is commutative in nature

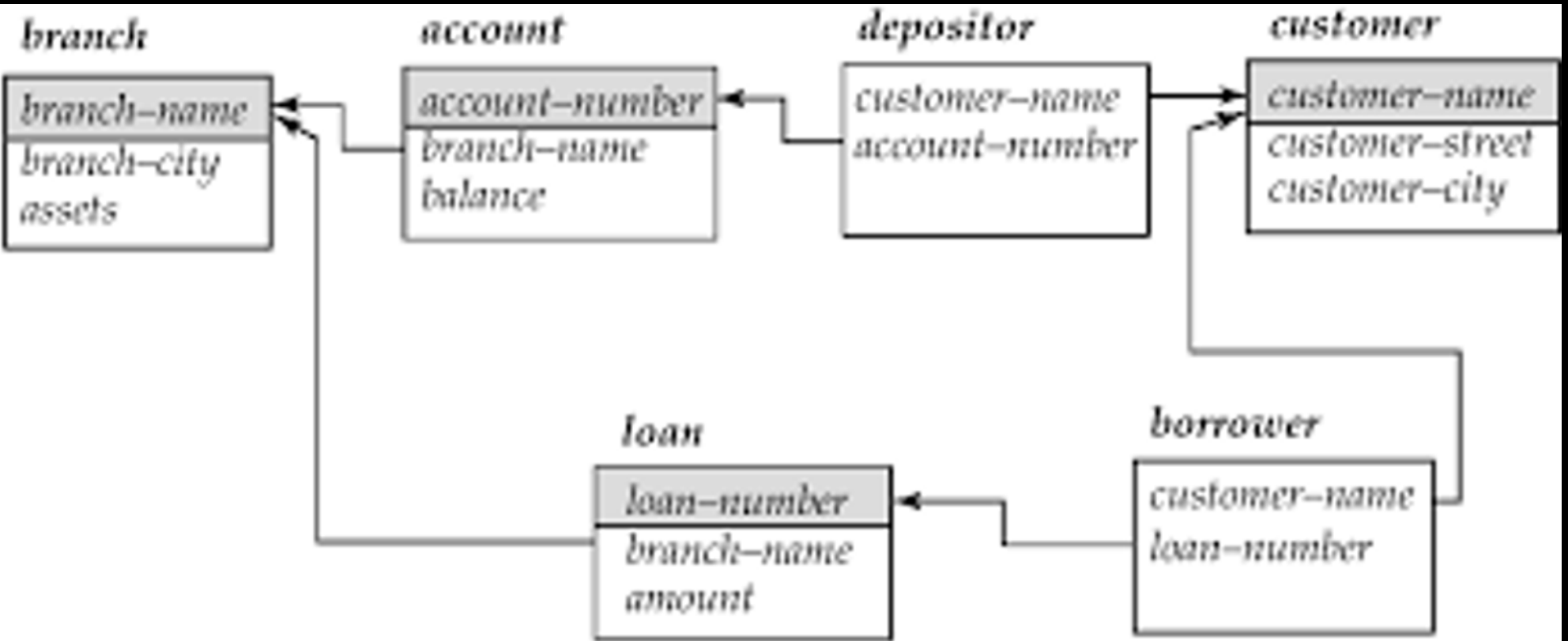
R_1	
A	**B**
1	P
2	Q
3	R

R_2	
B	**C**
Q	X
R	Y
S	Z

R_1 X R_2			
A	**R_1.B**	**R_2.B**	**C**
1	P	Q	X
1	P	R	Y
1	P	S	Z
2	Q	Q	X
2	Q	R	Y
2	Q	S	Z
3	R	Q	X
3	R	R	Y
3	R	S	Z

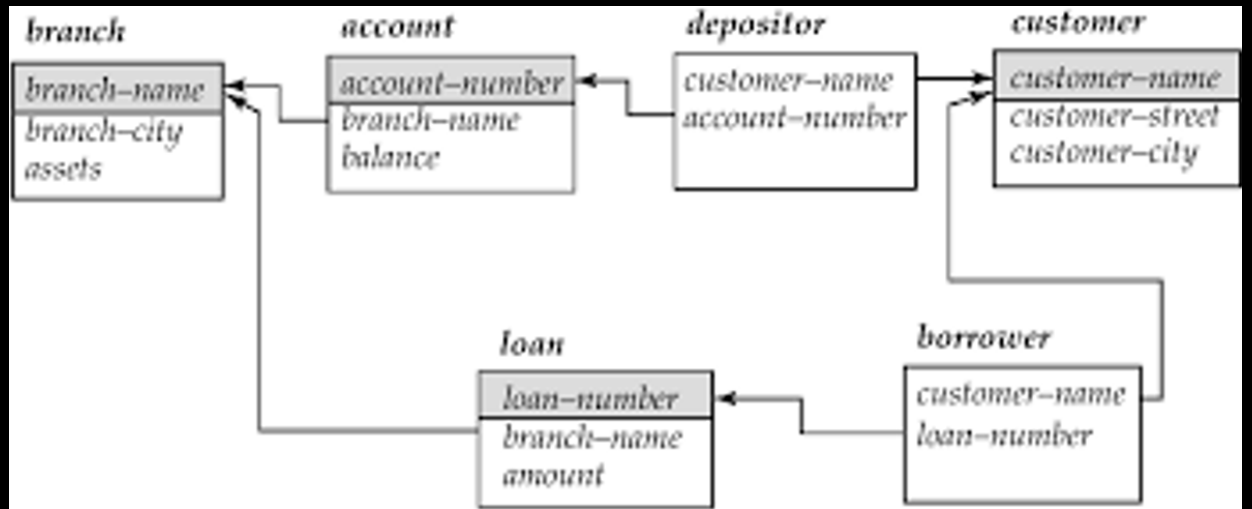
Knowledge Gate

Q Write a RELATIONAL ALGEBRA query to find the name of all the customers along with account balance, who have an account in the bank?



Q find the name of all the customers with account balance, who have an account in the bank?

Select customer_name, balance
From account, depositor
Where account.account_number = depositor.account_number



Natural Join

- To make the life of an SQL programmer easier for this common case, SQL supports an operation called the *natural join*. The **natural join** operation like cartesian product operates on two relations and produces a relation as the result.
- Natural join considers only those pairs of tuples with the same value on those attributes that appear in the schemas of both relations. Notice that we do not repeat those attributes that appear in the schemas of both relations; rather they appear only once.
- Notice also the order in which the attributes are listed: first the attributes common to the schemas of both relations, second those attributes unique to the schema of the first relation, and finally, those attributes unique to the schema of the second relation. Commutative in nature.

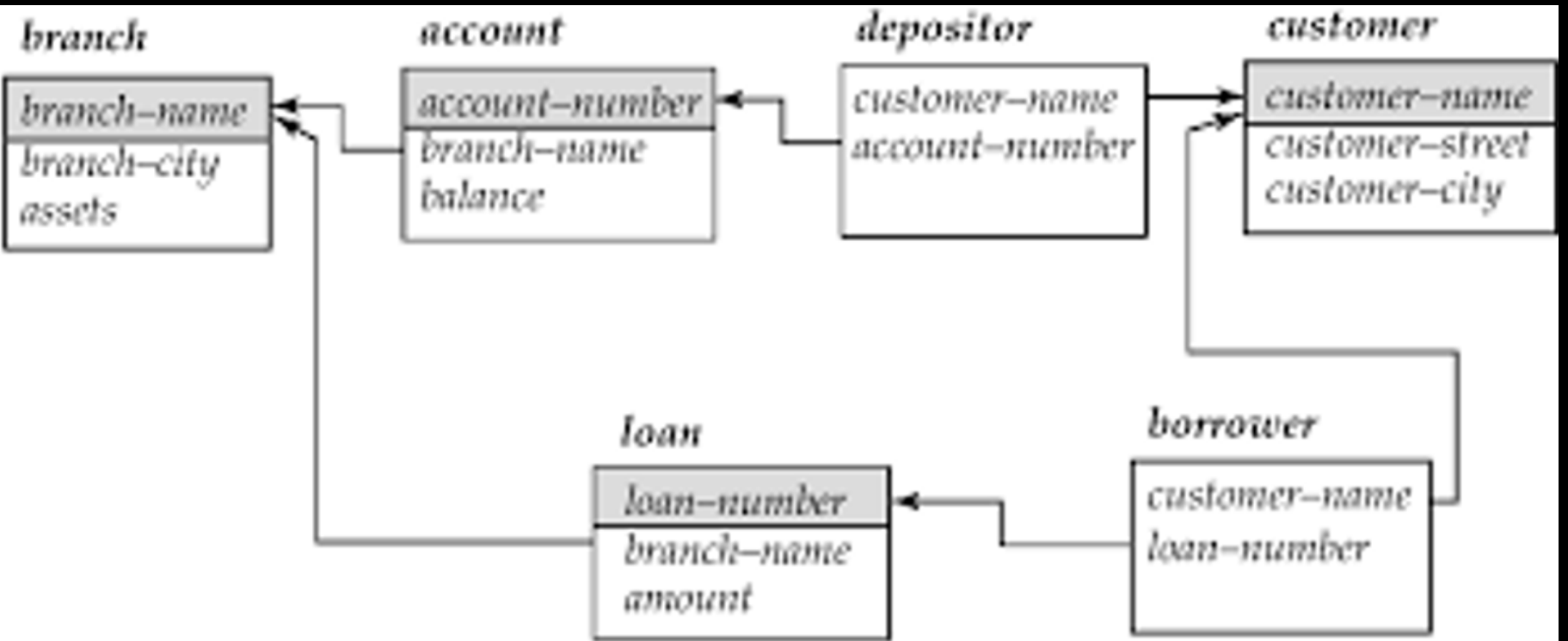
R_1	
A	**B**
1	P
2	Q
3	R

R_2	
B	**C**
Q	X
R	Y
S	Z

$R_1 \bowtie R_2$		
A	**B**	**C**
2	Q	X
3	R	Y

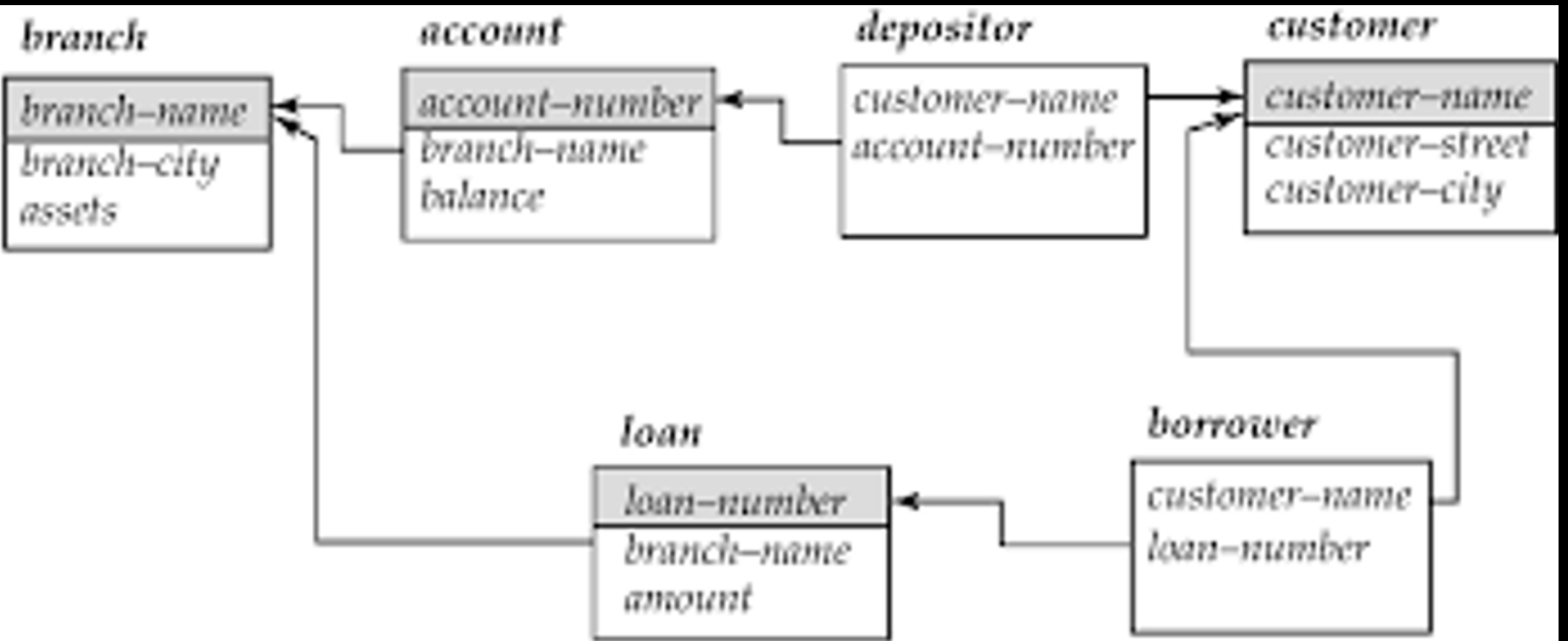
Knowledge Gate Website

Q Write a SQL query to find the name of all the customers along with account balance, who have an account in the bank?



Q Write a SQL query to find the name of all the customers along with account balance, who have an account in the bank?

Select customer_name, balance
From account natural join depositor



Outer Join

- The problem with natural join or join or inner join is only those values that appears in both relations will manage to reach final table, but if some value is explicitly in table one or in second table then that information will be lost, and that will be loss of information.
- The outer join operation works in a manner similar to the join operations we have already studied, but preserve those tuples that would be lost in a join, by creating tuples in the result containing null values.
- There are in fact three forms of outer join:
  - The **left outer join (**left join**)** preserves tuples only in the relation named before (to the left of) the left outer join operation.
  - The **right outer join (**right join**)** preserves tuples only in the relation named after (to the right of) the right outer join operation.
  - The **full outer** join preserves tuples in both relations.

Knowledge Gate Website

R_1	
A	B
1	P
2	Q
3	R

R_2	
B	C
Q	X
R	Y
S	Z

$R_1 * R_2$			
A	$R_1.B$	$R_2.B$	C
1	P	Q	X
1	P	R	Y
1	P	S	Z
2	Q	Q	X
2	Q	R	Y
2	Q	S	Z
3	R	Q	X
3	R	R	Y
3	R	S	Z

$R_1 \bowtie R_2$		
A	B	C
2	Q	X
3	R	Y

$R_1 ⟕ R_2$		
A	B	C
1	P	null
2	Q	X
3	R	Y

$R_1 ⟖ R_2$		
A	B	C
2	Q	X
3	R	Y
null	S	Z

$R_1 ⟗ R_2$		
A	B	C
1	P	null
2	Q	X
3	R	Y
null	S	Z

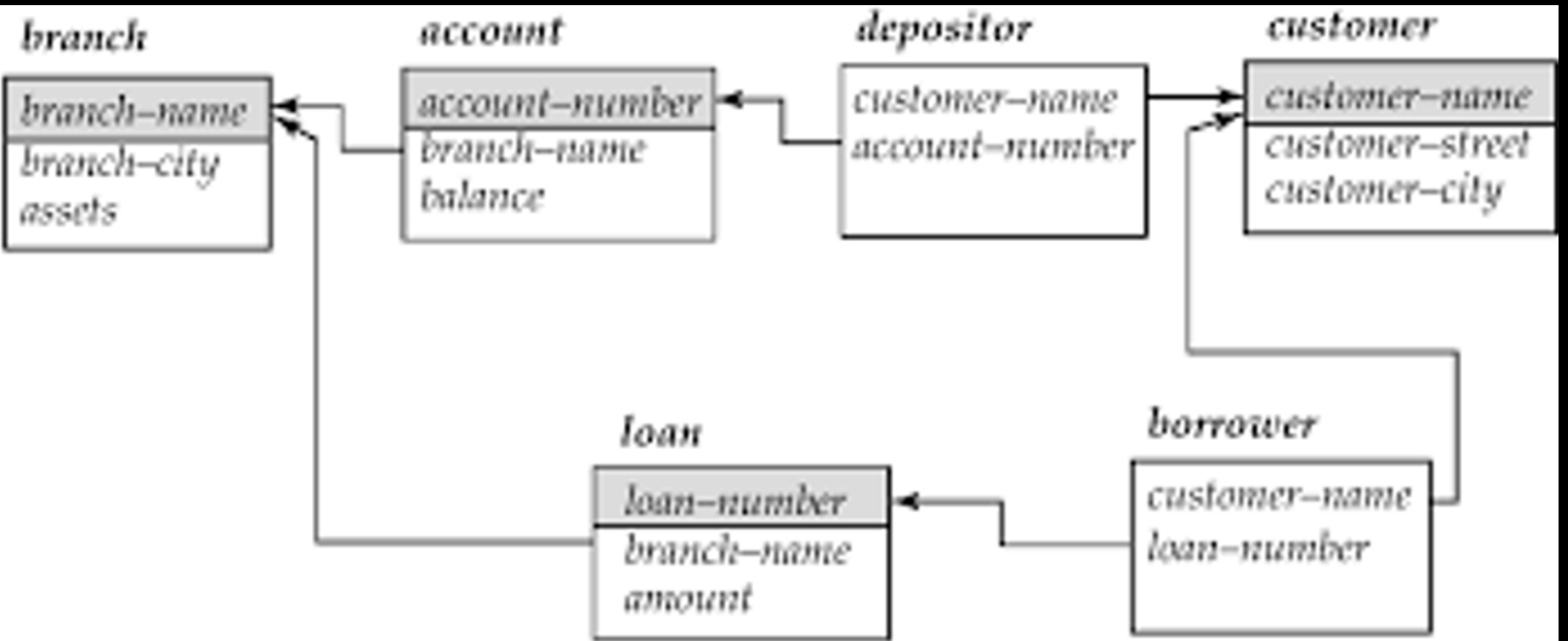
Knowledge Gate Website

Alias Operation/rename

- SQL aliases are used to give a table, or a column in a table, a temporary name. Just create a new copy but do not change anything in the data base. An alias only exists for the duration of the query.
- Aliases are often used to make column names more readable.
- It uses the as clause, taking the form: old-name as new-name. The as clause can appear in both the select and from clauses.

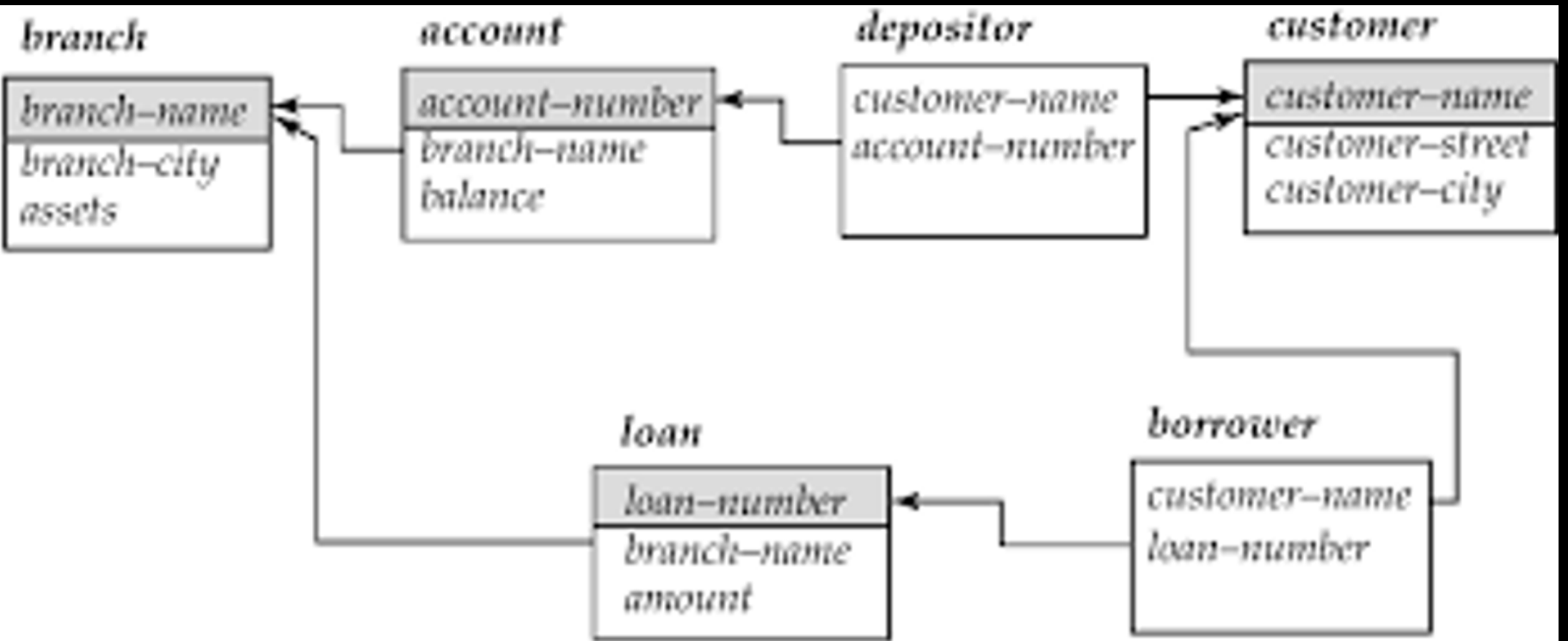
Knowledge Gate Website

Q Write a SQL query to find the account_no along and balance with 8% interest, as Account, total_balance?



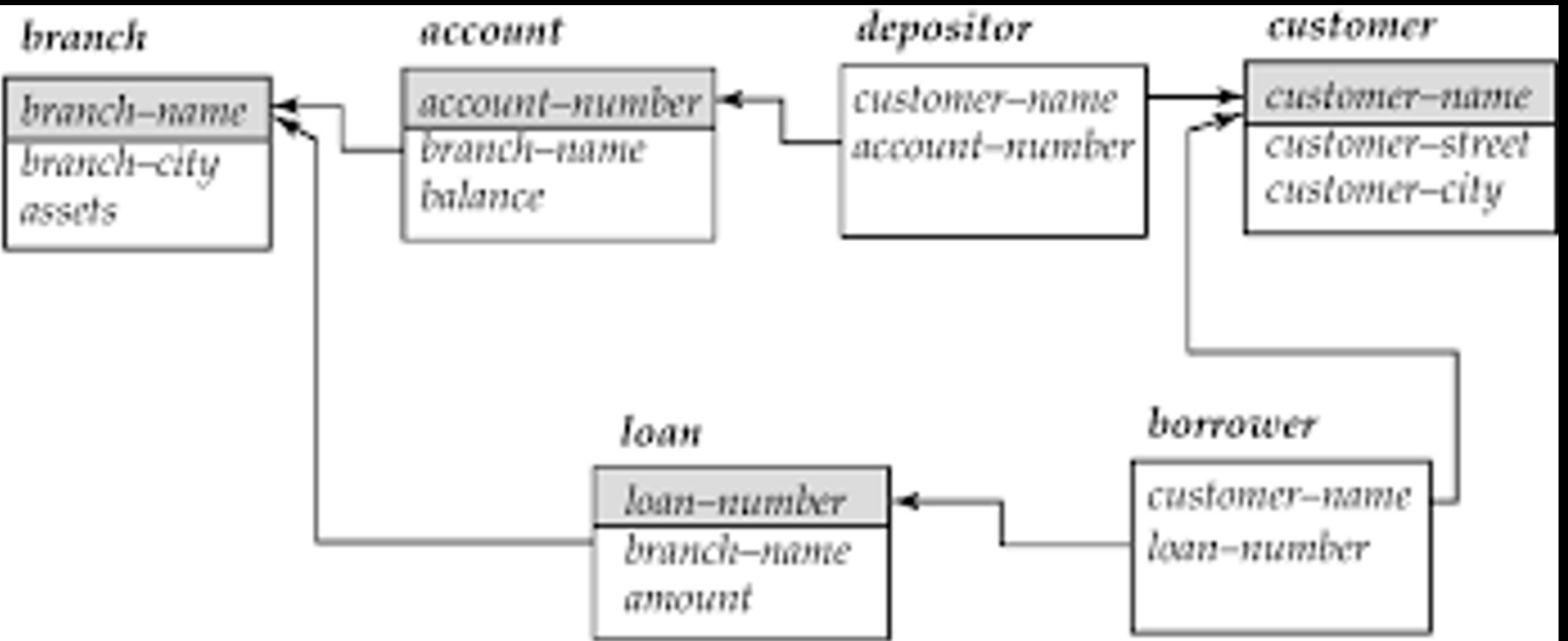
Q Write a SQL query to find the account_no along and balance with 8% interest, as Account, total_balance?

Select account_number, balance*1.06 as total_balance
From account



Q Write a SQL query to find the loan_no with maximum loan amount?

Select balance
From account
Except
Select A.balance
From account as A, account as B
Where A.balance <B.balance



Aggregate Functions

- *Aggregate functions* are functions that take a collection (a set or multiset) of values as input and return a single value. SQL offers five built-in aggregate functions:
  - Average: **avg**
  - Minimum: **min**
  - Maximum: **max**
  - Total: **sum**
  - Count: **count**
- The input to **sum** and **avg** must be a collection of numbers, but the other operators can operate on collections of nonnumeric data types, such as strings, as well. Count is the only aggregate function which can work with null, all other aggregate functions simply ignore null.
- We use the aggregate function **count** frequently to count the number of tuples in a relation. The notation for this function in SQL is **count** (*).

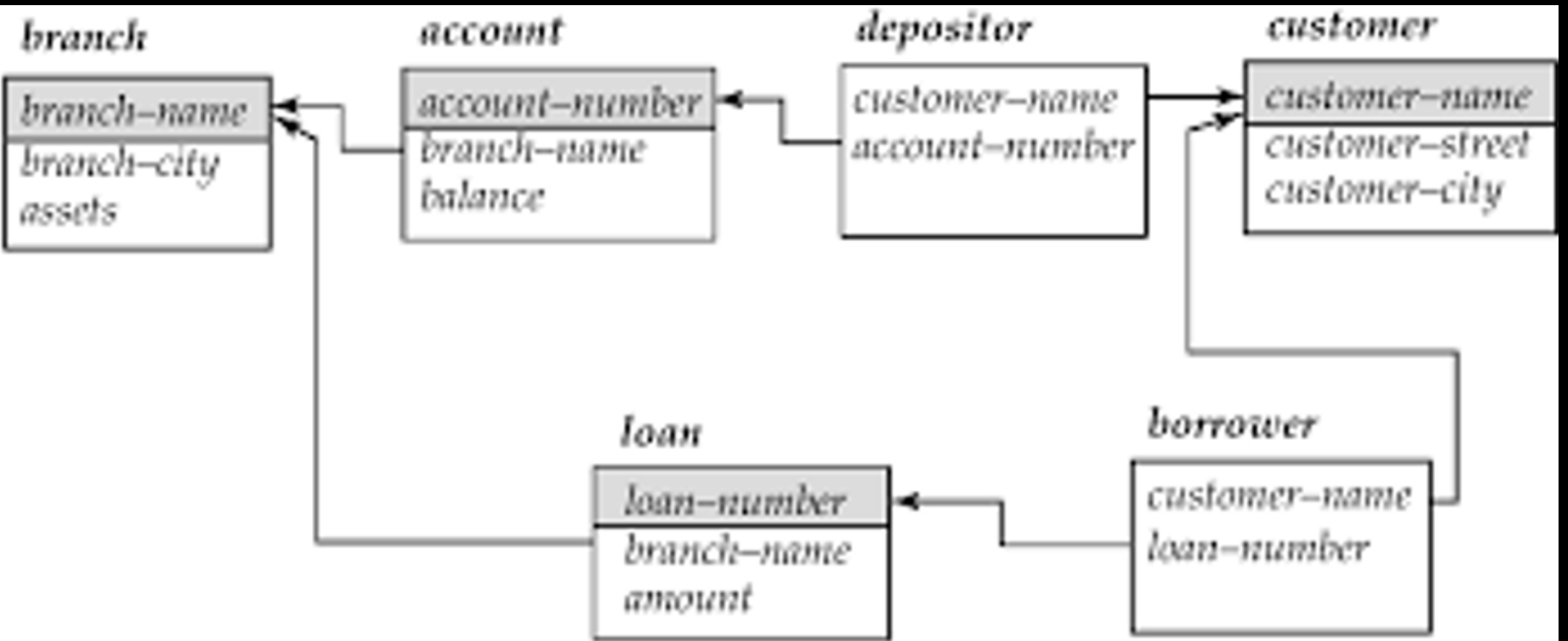
Knowledge Gate Website

Q find the number of accounts in the bank?

Select count(*)
from account

Q find the average balance of every account in the banks from south_delhi branch?

Select avg(balance)
from account
Where branch_name = ‘south_delhi’



Q Consider a table along with two query?

Select avg (balance)
from account

Account_no	balance	Branch_name
Abc123	100	N_delhi
Pqr123	500	S_mumbai
Wyz123	null	S_delhi

Select sum(balance)/count(balance)
from account

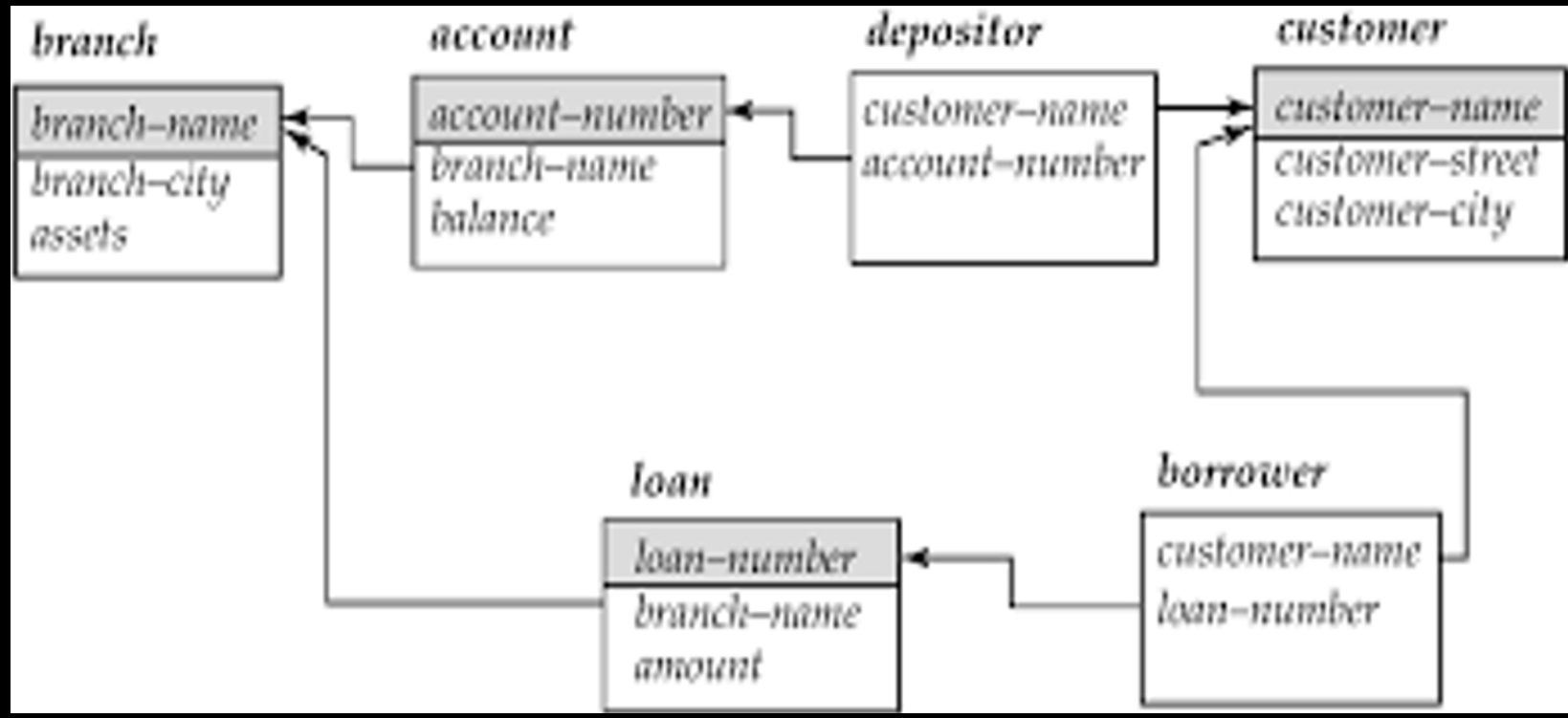
Ordering the Display of Tuples

- SQL offers the user some control over the order in which tuples in a relation are displayed. The **order by** clause causes the tuples in the result of a query to appear in sorted order.

Q find all the branch_name which are situated in Delhi in alphabetic order?

```
Select distinct branch_name
from branch
where branch_city = 'Delhi'
Order by branch_name aesc;
```

```
Select distinct branch_name
from branch
where branch_city = 'Delhi'
Order by branch_name desc;
```



Knowledge Gate Website

String Operations

- SQL specifies strings by enclosing them in single quotes, for example, ’Computer’. The SQL standard specifies that the equality operation on strings is case sensitive; as a result the expression ‘Computer’ = ’computer’ evaluates to false.
- However, some database systems, such as MySQL and SQL Server, do not distinguish uppercase from lowercase when matching strings; as a result, would evaluate to true on these databases. This default behavior can, however, be changed, either at the database level or at the level of specific attributes.

- SQL also permits a variety of functions on character strings, such as concatenating, extracting substrings, finding the length of strings, converting strings to uppercase and lowercase, removing spaces at the end of the string and so on. There are variations on the exact set of string functions supported by different database systems.

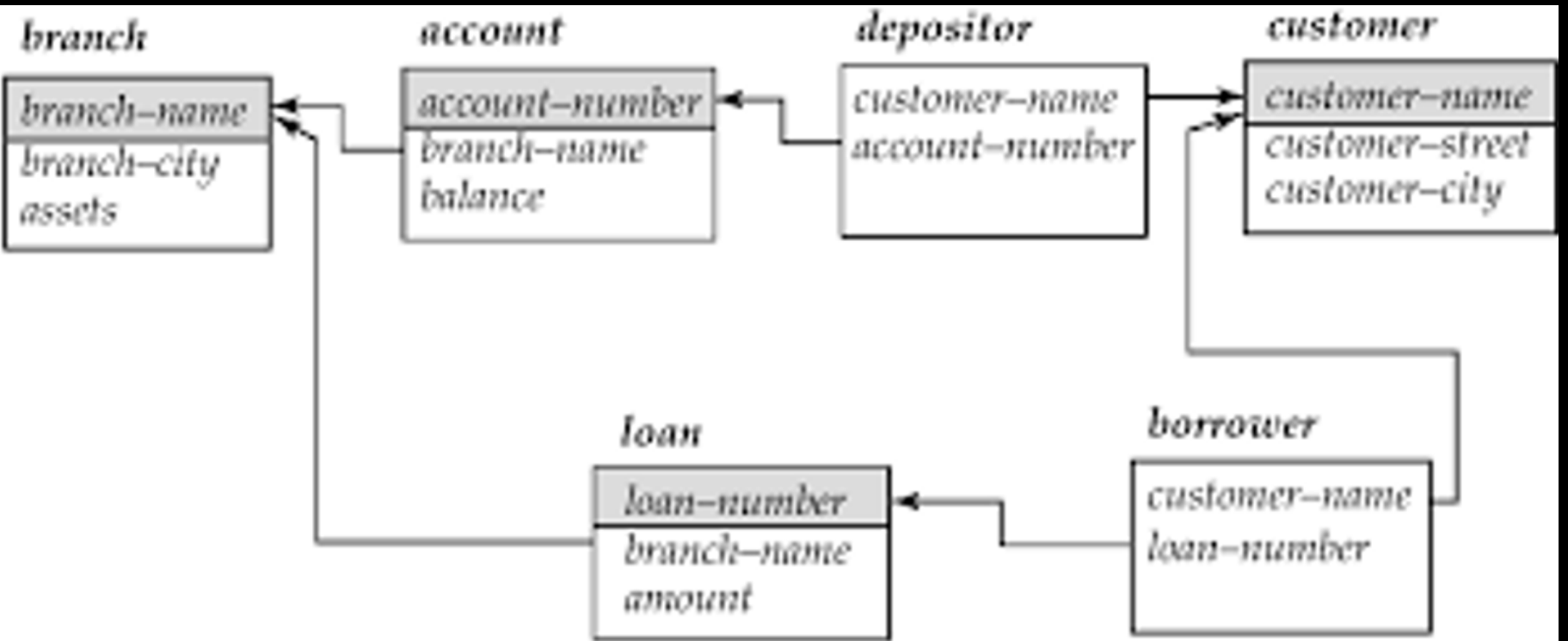
- Pattern matching can be performed on strings, using the operator **like**. We describe patterns by using two special characters:
  - Percent (%): The % character matches any substring.
  - Underscore (_): The _ character matches any character.

- ’%Comp%’ matches any string containing “Comp” as a substring, for example, ’Intro to Computer Science’, and ’Computational Biology’.
  - ’_ _ _’ matches any string of exactly three characters.
  - ’_ _ _%’ matches any string of at least three characters.

Knowledge Gate Website

Q find all the branch name who have exactly 5 character in their name ?

Q find all the customer name who have ‘kumar’ in their name ?

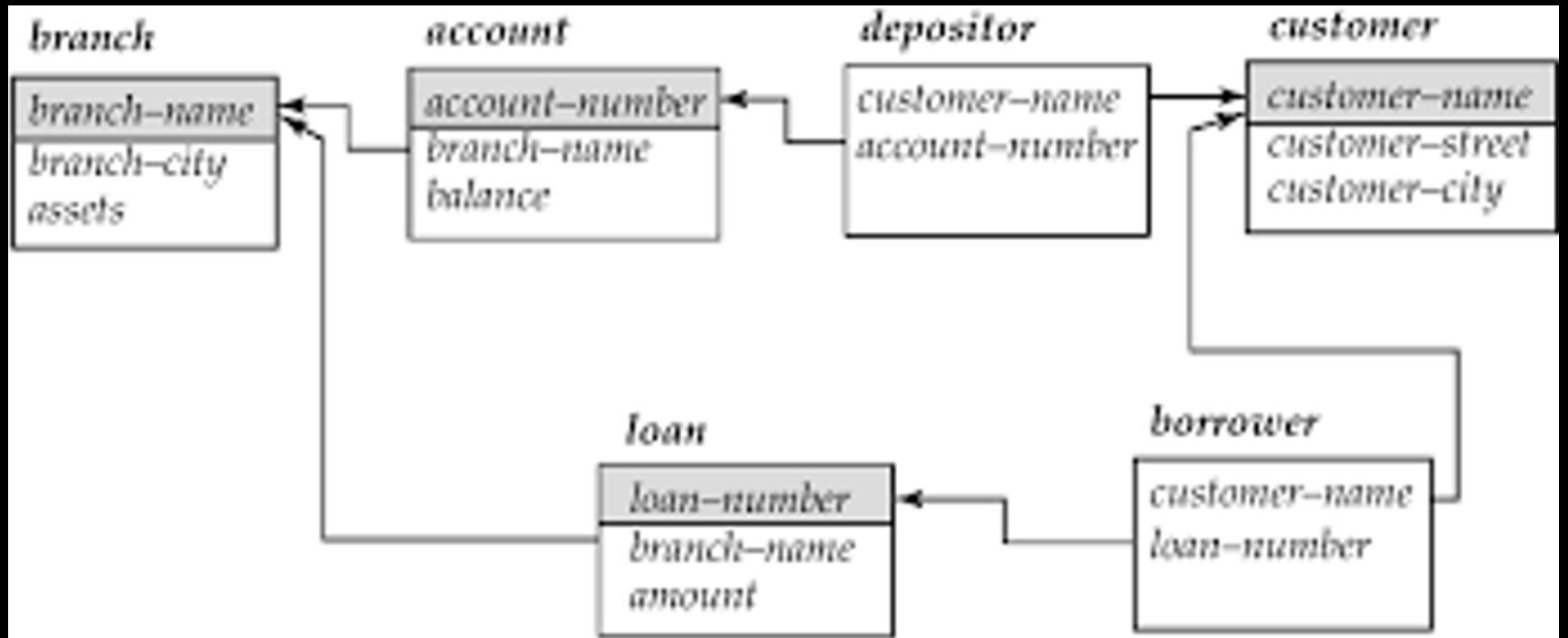


Q find all the branch name who have exactly 5 character in their name ?

```
Select branch_name
from branch
where branch_name like '_ _ _ _ _'
```

Q find all the customer name who have 'kumar' in their name ?

```
Select customer_name
from customer
where customer_name like '%kumar%'
```



- We define the escape character for a **like** comparison using the **escape** keyword. To illustrate, consider the following patterns, which use a backslash (\) as the escape character:

- **like** ’ab\%cd%’ **escape** ’\’ matches all strings beginning with “ab%cd”.

- **like** ’ab\\cd%’ **escape** ’\’ matches all strings beginning with “ab\cd”.

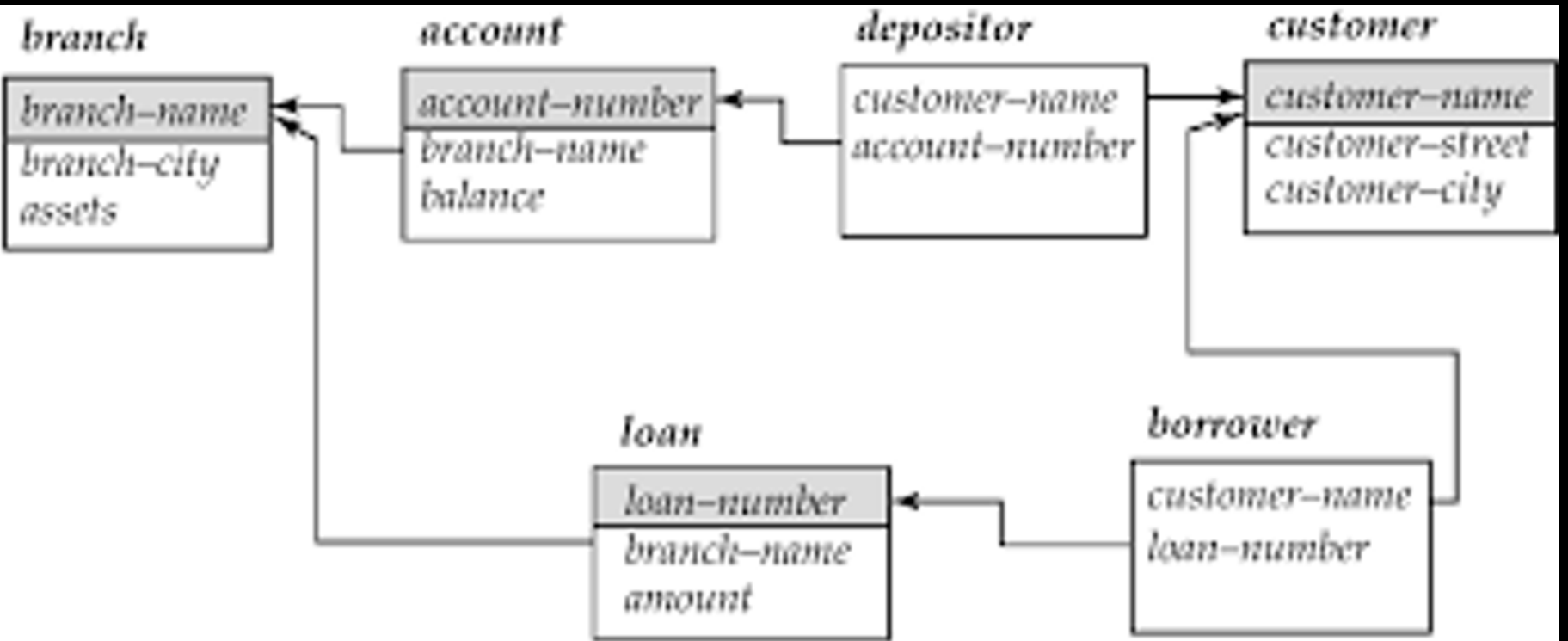
Group by clause

1. There are circumstances where we would like to work on a set of tuples in a relation rather than working on the whole table as one unit.

2. The attribute or attributes given in the **group by** clause are used to form groups. Tuples with the same value on all attributes in the **group by** clause are placed in one group.

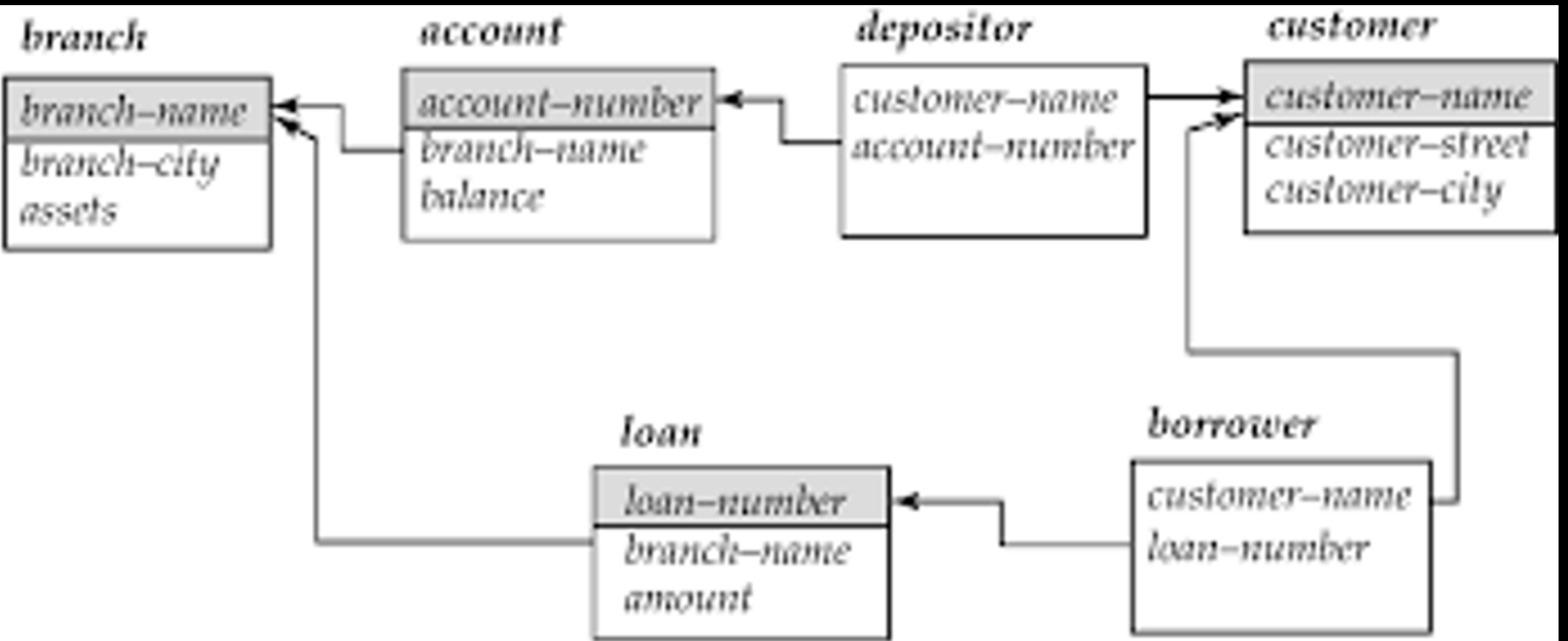
Q find the average account balance of each branch?

```
Select branch_name, avg(balance)
from account
Group by branch_name
```



Q find the branch name of Gwalior city with average balance more than 1500?

```
Select branch.branch_name, avg(balance)
from branch, account
Where branch.branch_name = account.branch_name and branch_city = ‘gwalior’
Group by branch_name
Having avg(balance) > 1500
```



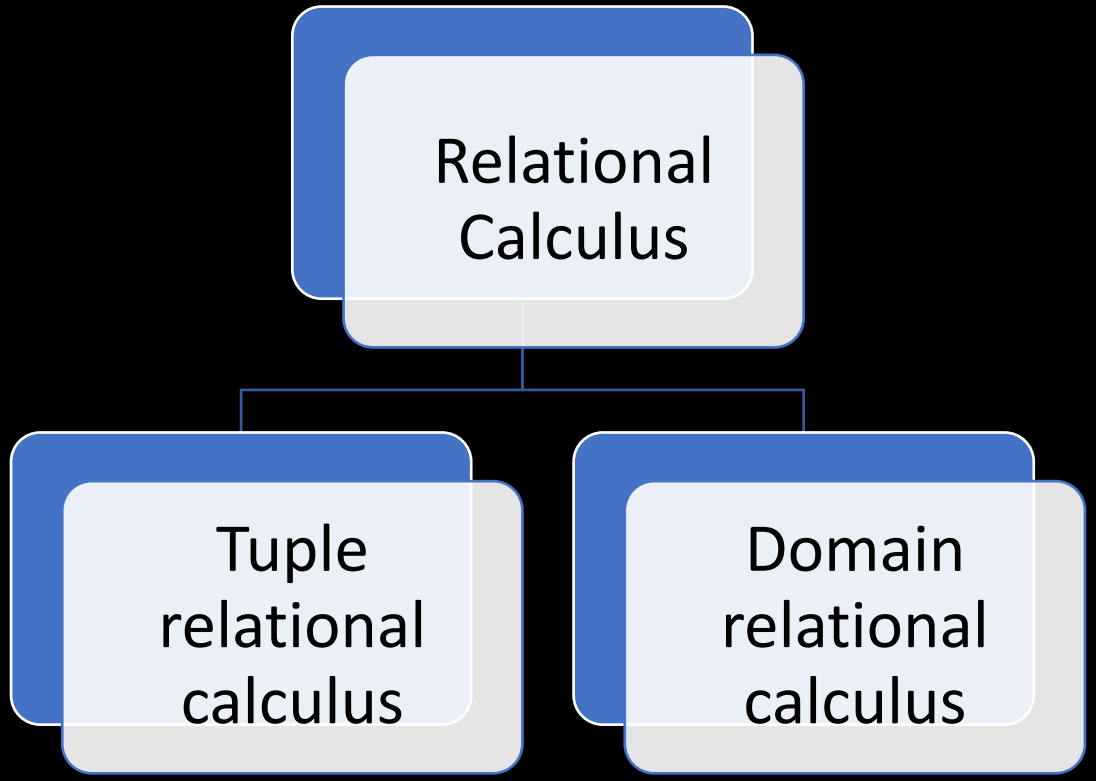
- **Trigger Definition**
  - A trigger is a special procedure that automatically activates in response to modifications on a table or view in a database, aiding in maintaining data integrity.

- **Usage of Triggers**
  - Triggers help maintain database integrity by automatically enforcing conditions or modifying data during database operations. They are crucial for applying business rules, auditing database alterations, and supporting data replication, ensuring compliance before any data insertions, updates, or deletions.

- **Embedded SQL**
  - Embedded SQL is a method where SQL statements are incorporated directly into a procedural programming language, such as C or Java. It allows the programmers to integrate SQL queries within their code, facilitating the interaction between the database and the application. Embedded SQL statements are static and defined at compile time.
- **Dynamic SQL**
  - Dynamic SQL, on the other hand, enables the construction of SQL statements dynamically at runtime. It allows the creation of more flexible and adaptable applications, where SQL statements can be generated and executed based on changing conditions or user inputs. This makes it possible to create more complex and adaptable database operations, though it might be more susceptible to SQL injection attacks if not handled carefully.

Relational Calculus

- Relational calculus is non-procedural query language, where we have to define what to get and not how to get it



Knowledge Gate Website

Tuple Relational Calculus

- TRC is based on specifying a number of tuple variables. Each tuple variable usually range over a particular database relation, meaning that the variable may takes as its value from any individual tuple of a relation.
- A simple tuple relational calculus query is of the form.
  - {t| Condition(t)}
- Where t is a tuple variable and condition(t) is a conditional expression involving. The result of such a query is the set of all tuple t that satisfy condition (t).

Student(Roll No, Name, Branch)

Q Find the details of all computer science students?

SQL: select * from student where branch = CSE

RA: $\{\sigma_{\text{branch = CSE}} (\text{Student})\}$

TRC: {t | Student(t) ∩ t. branch = CSE}

DRC:

Student(Roll No, Name, Branch)

Q Find the Roll No of all computer science students?

SQL: select Roll No from student where branch = CSE

RA: $\{\prod_{\text{roll no}} (\sigma_{\text{branch = CSE}} (\text{Student}))\}$

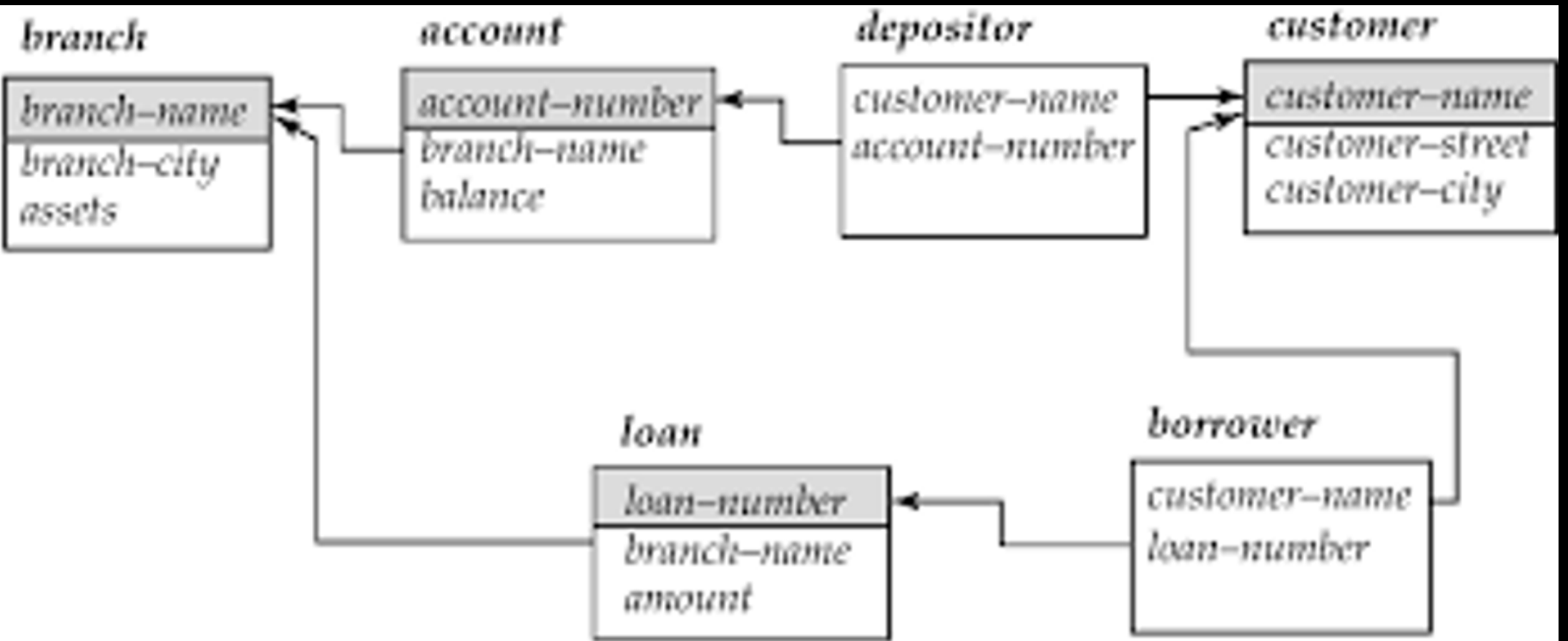
TRC: {t. Roll No | Student(t) ∩ t. branch = CSE}

DRC:

Knowledge Gate Website

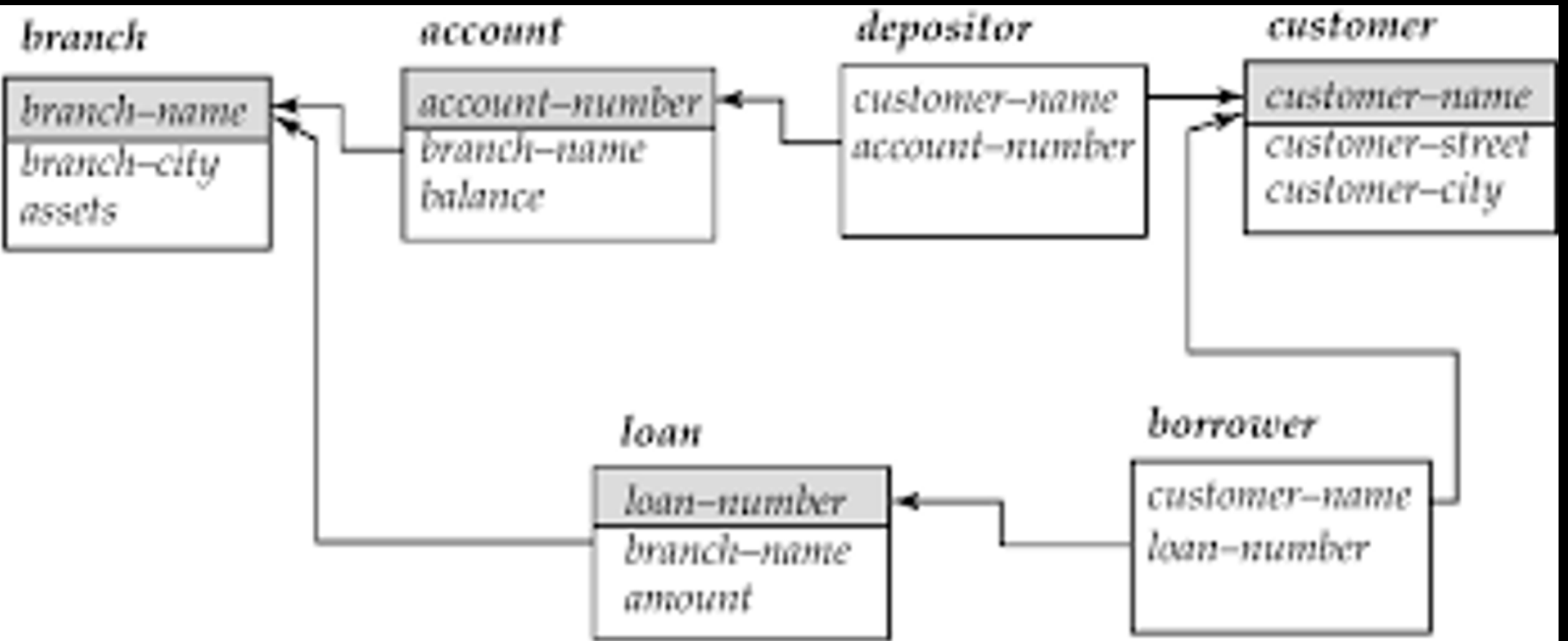
Q Find all the details of loan for amount over 1200?

$\{t \mid t \in loan \wedge t[amount] > 1200] \}$



Q Find the loan number for each loan of amount over 1200?

$\{t \mid \exists s \in loan\ (t[loan\ number] = s[loan\ number]) \wedge s[amount] > 1200\}$

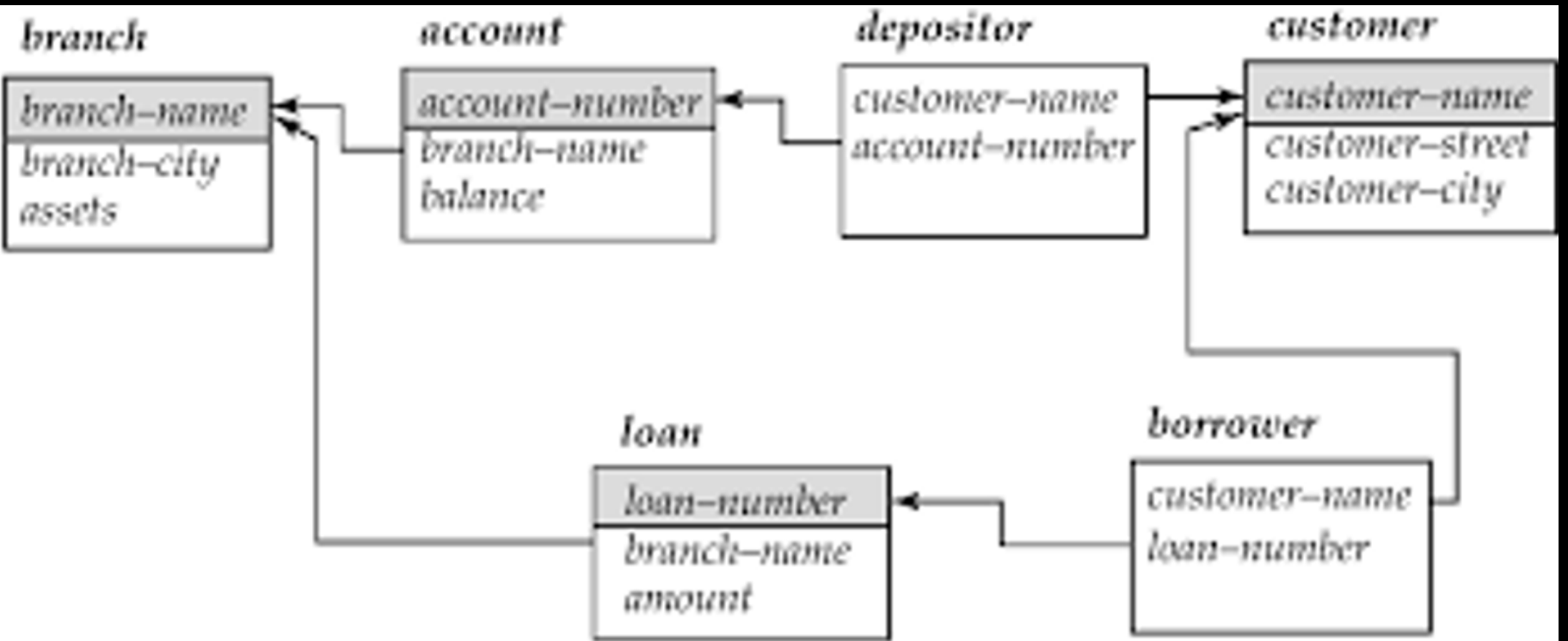


Q Find the name of all the customers who have a loan from Noida branch?

{t| ∃s ∈ borrower (t[customer name] = s[customer name])

∧ ∃u ∈ loan (u[customer name] = s[customer name])

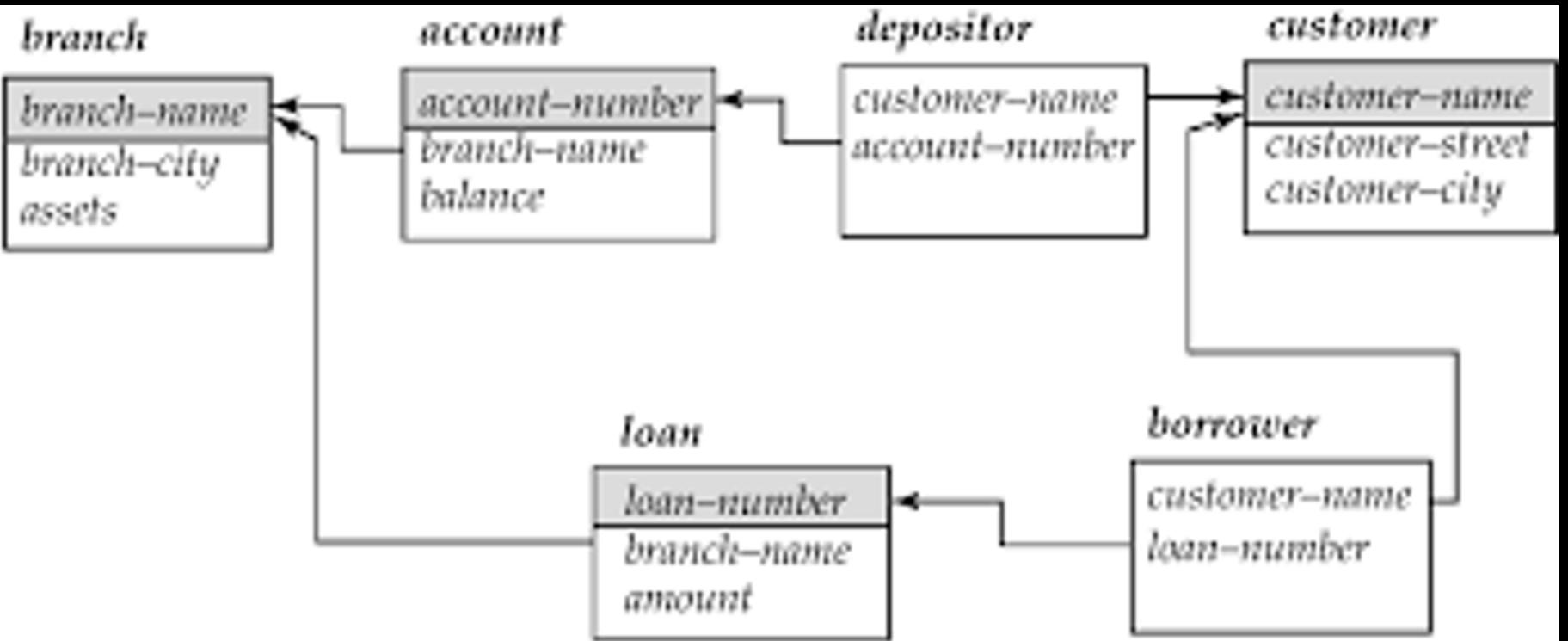
∧ u[branch name] = 'Noida' }



Q Find the name of all the customers who have a loan or account or both at the bank?

{t| ∃s ∈ borrower (t[customer name] = s[customer name])

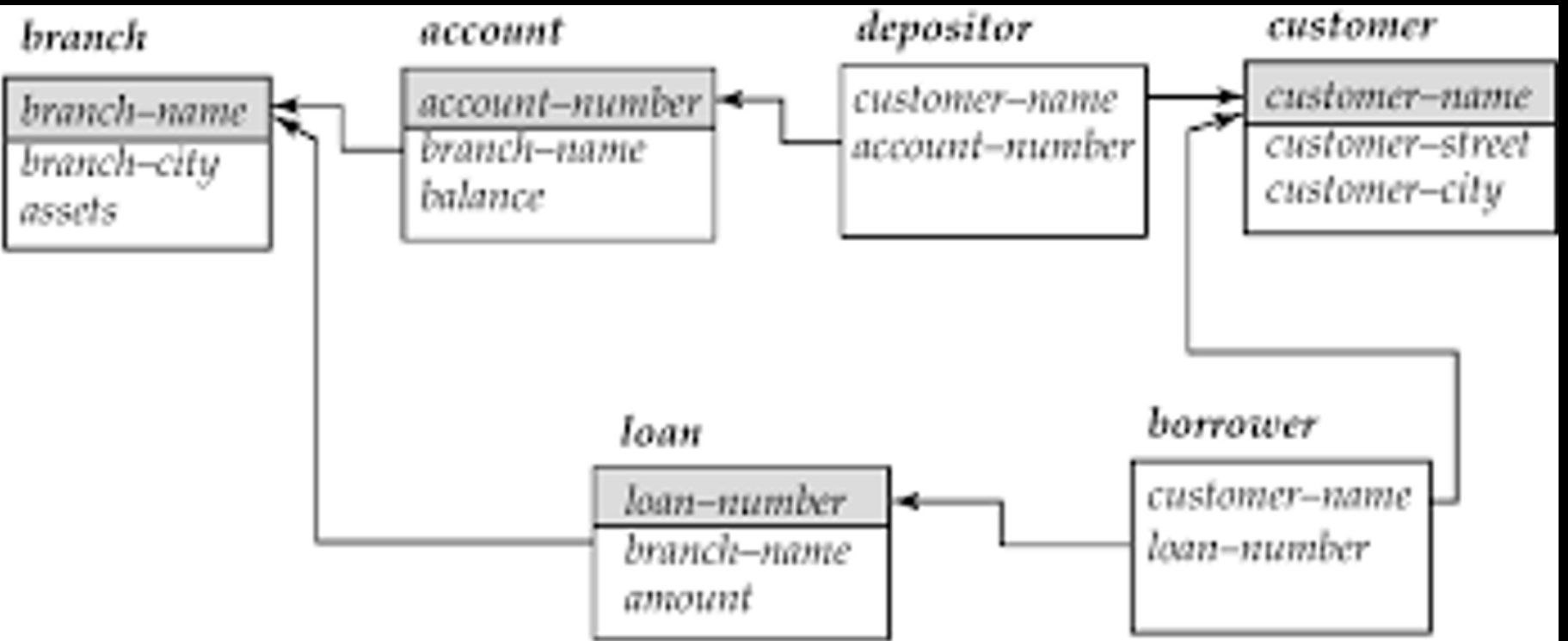
∨ ∃u ∈ depositor (t[customer name] = u[customer name]) }



Q Find the name of all the customers who have a loan and account both at the bank?

{t| ∃s ∈ borrower (t[customer name] = s[customer name])

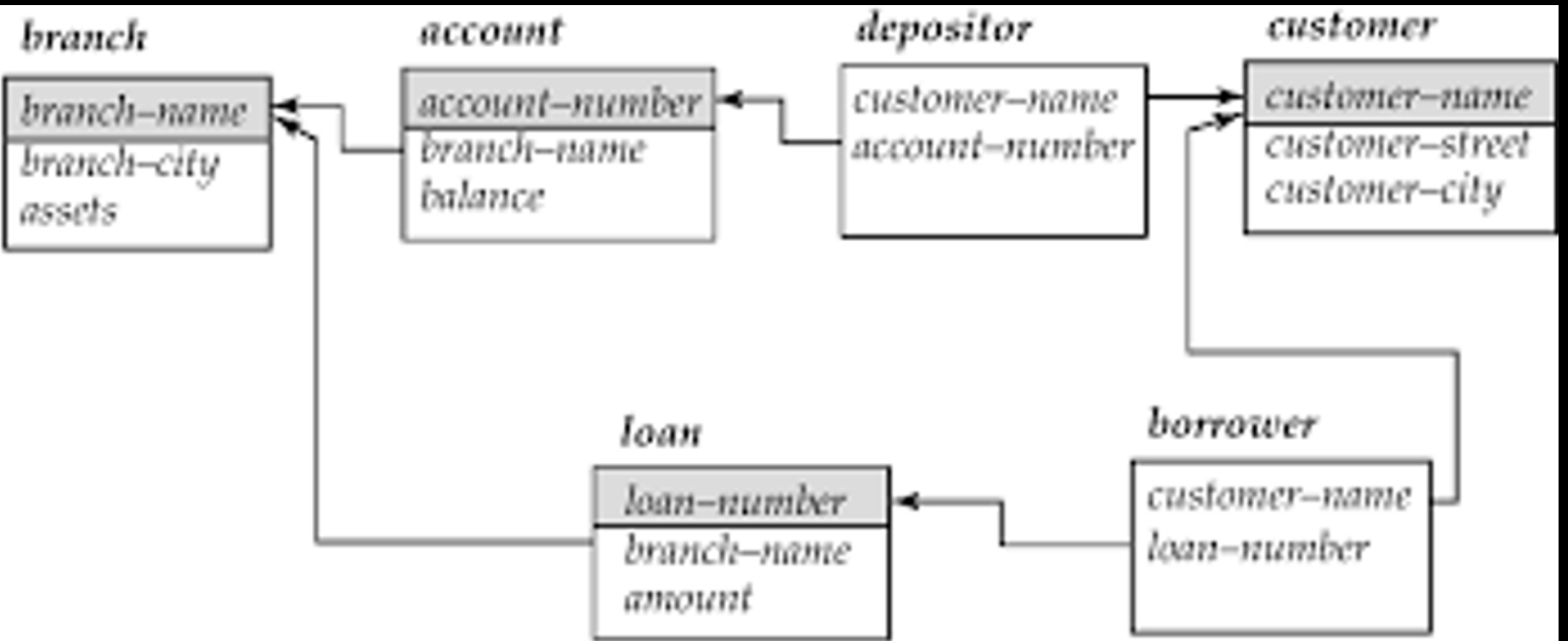
∧ ∃u ∈ depositor (t[customer name] = u[customer name]) }



Q Find the name of all the customers who have a loan from the bank and do not have a account?

{t| ∃u ∈ depositor borrower (t[customer name] = u[customer name])

∧ ¬∃s ∈ borrower (t[customer name] = s[customer name]) }



Knowledge Gate Website

Knowledge Gate Website

Domain Relational Calculus

- Domain calculus differs from the tuple calculus in the type of variables used in formulas: rather than having variables range over tuples.
- The variable range over single values from domains of attributes. To form a relation of degree n for a query result, we must have n of these domain variables. One for each attribute.
- An expression of the domain calculus is of the form
- $(x_1, x_2, \ldots, x_n \mid COND(x_1, x_2, \ldots, x_n, x_{n+1}, x_{n+2}, \ldots, x_{n+m}))$
- Where $x_1, x_2, \ldots, x_n$ are domain variables that range over domains and COND is a condition of the domain relational calculus.

Student(Roll No, Name, Branch)

Q Find the details of all computer science students?

SQL: select * from student where branch = CSE

RA: $\{\sigma_{\text{branch = CSE}} (\text{Student})\}$

TRC: {t | Student(t) ∩ t. branch = CSE}

DRC: {(Roll No, Name, Branch) | Student (Roll no, Name, Branch) ∩ branch = CSE}

Knowledge Gate Website

Student(Roll No, Name, Branch)

Q Find the Roll No of all computer science students?

SQL: select Roll No from student where branch = CSE

RA: $\{\prod_{\text{sname}} (\sigma_{\text{branch = CSE}} (\text{Student}))\}$

TRC: {t. Roll No | Student(t) ∩ t. branch = CSE}

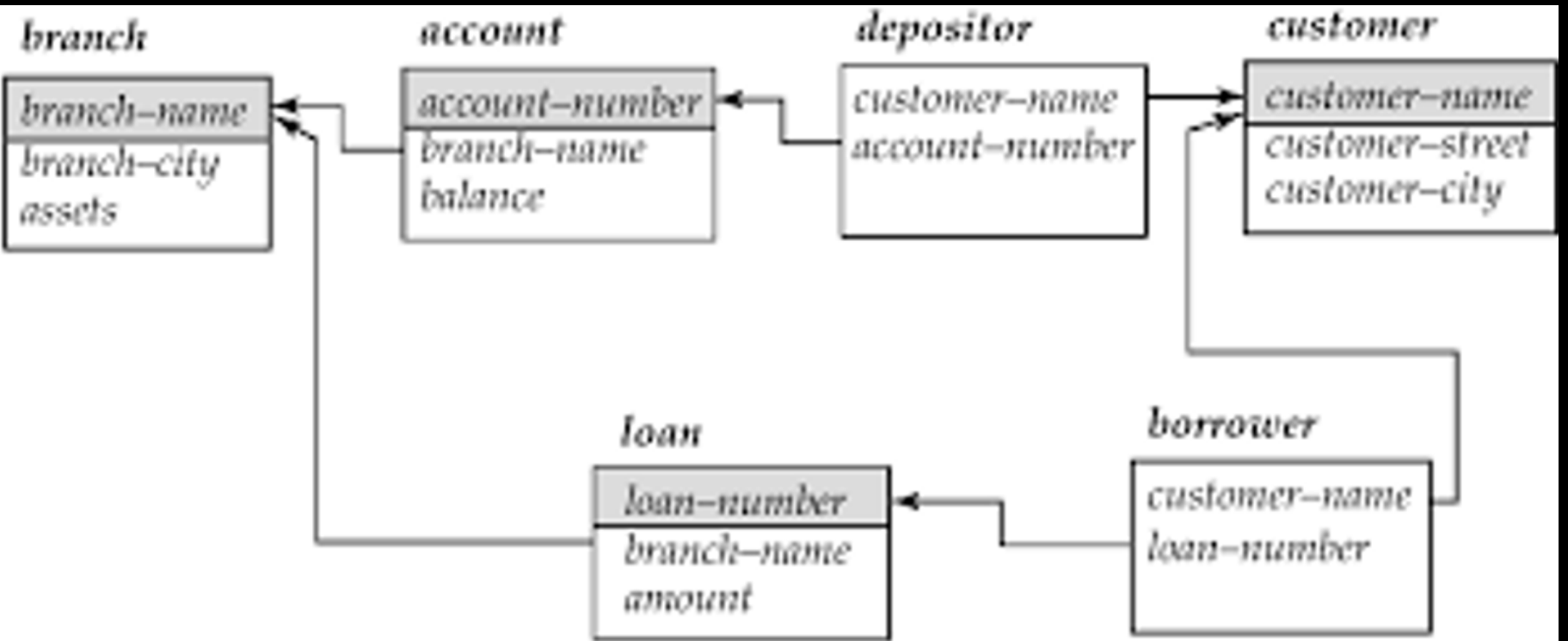
DRC: {(Roll No) | Student (Roll no, Name, Branch) (Name, Branch) ∩ branch = CSE}

Knowledge Gate Website

Example: Find all details of instructors whose salary is greater than $80,000

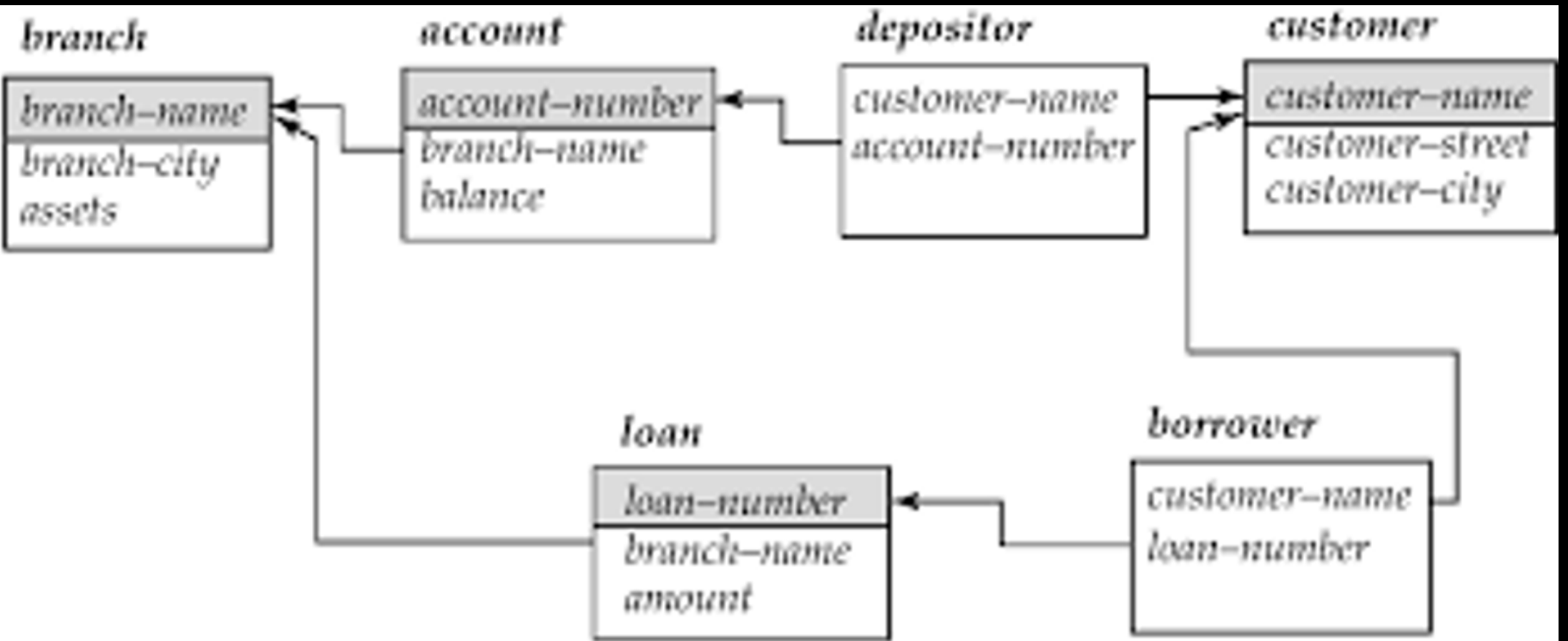
Q Find the branch name, loan number and amount for loan of amount over 1200?

$\{ < l, b, a > \mid < l, b, a > \in loan \wedge a > 1200] \}$



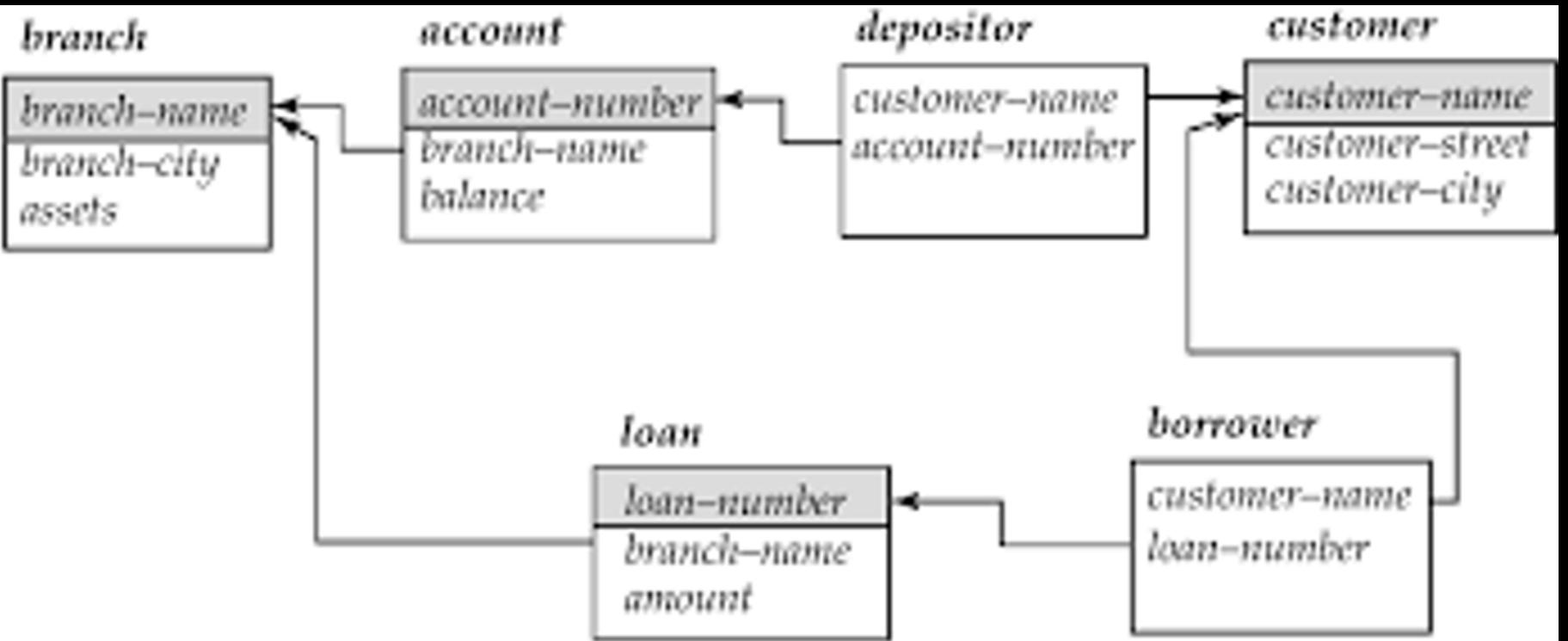
Q Find the loan number for each loan of amount over 1200?

$\{ < l > \mid \exists_{b, a} < l, b, a > \in loan \wedge a > 1200] \}$



Q Find the name of all the customers who have a loan from Noida branch with loan amount?

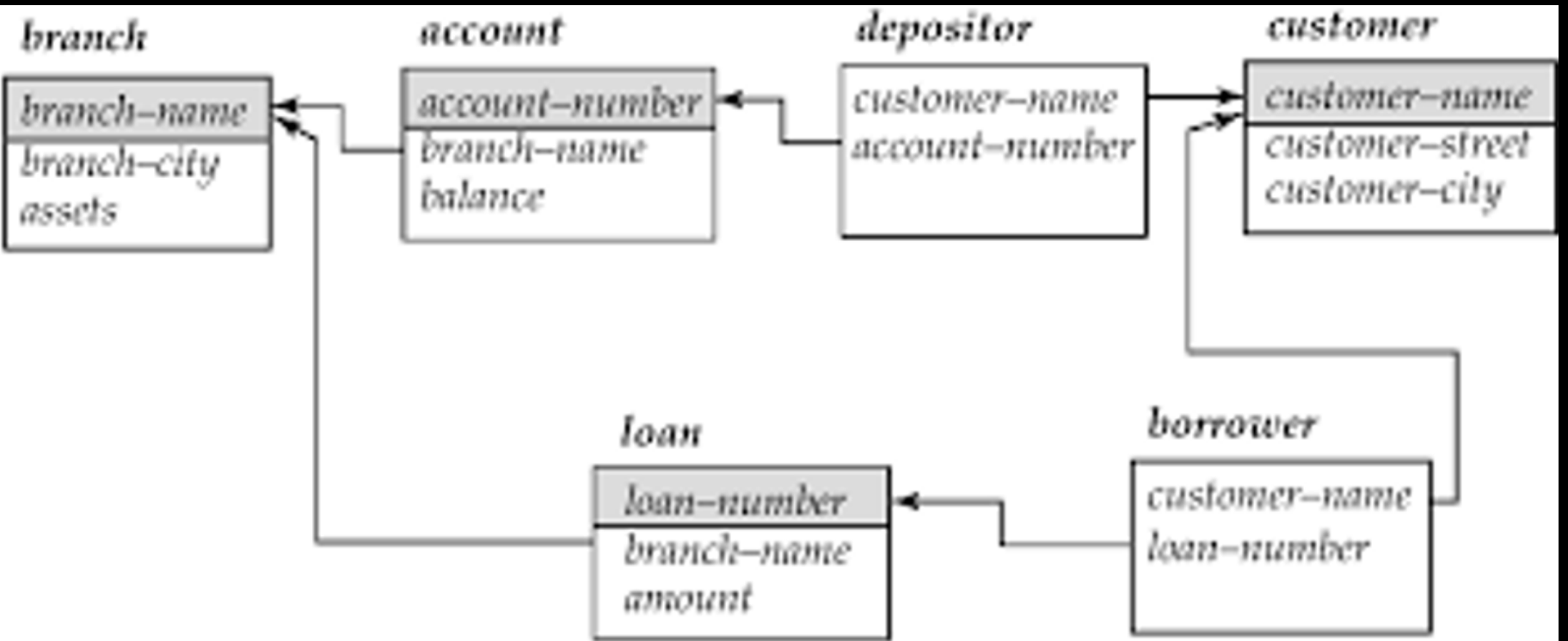
$\{ < c, a > \mid \exists_l \ (< c, l > \in \text{borrower} \wedge \exists_b (< l, b, a > \in \text{loan} \wedge b = \text{'Noida'}))\}$



Q Find the name of all the customers who have a loan or account or both at the bank?

$\{ < c > \mid \exists_l (< c, l > \in \text{borrower} \wedge \exists_{b,a} (< l, b, a > \in \text{loan} \wedge b = \text{'Noida'}))$

$\vee \exists_a (< c, a > \in \text{depositor} \wedge \exists_{b,a} (< a, bn, bal > \in \text{account} \wedge bn = \text{'Noida'}))\}$



Knowledge Gate Website

Knowledge Gate Website

TRANSACTION

- Why we study transaction?
  - According to general computation principle (operating system) we may have partially executed program, as the level of atomicity is instruction i.e. either an instruction is executed completely or not
  - But in DBMS view, user perform a logical work(operation) which is always atomic in nature i.e. either operation is execute or not executed, there is no concept like partial execution. For example, Transaction T_1 which transfer 100 units from account A to B.
  - In this transaction if a failure occurs after Read(B) then the final statue of the system will be inconsistent as 100 units are debited from account A but not credited in account B, this will generate inconsistency. Here for 'consistency' before (A + B) == after (A + B)".

T_1
Read(A)
A = A-100
Write(A)
Read(B)
B = B+100
Write(B)

Knowledge Gate Website

What is transaction

- To remove this partial execution problem, we increase the level of atomicity and bundle all the instruction of a logical operation into a unit called transaction.
- So formally ‘A transaction is a Set of logically related instructions to perform a logical unit of work’.

T_1
Read(A)
A = A-100
Write(A)
Read(B)
B = B+100
Write(B)

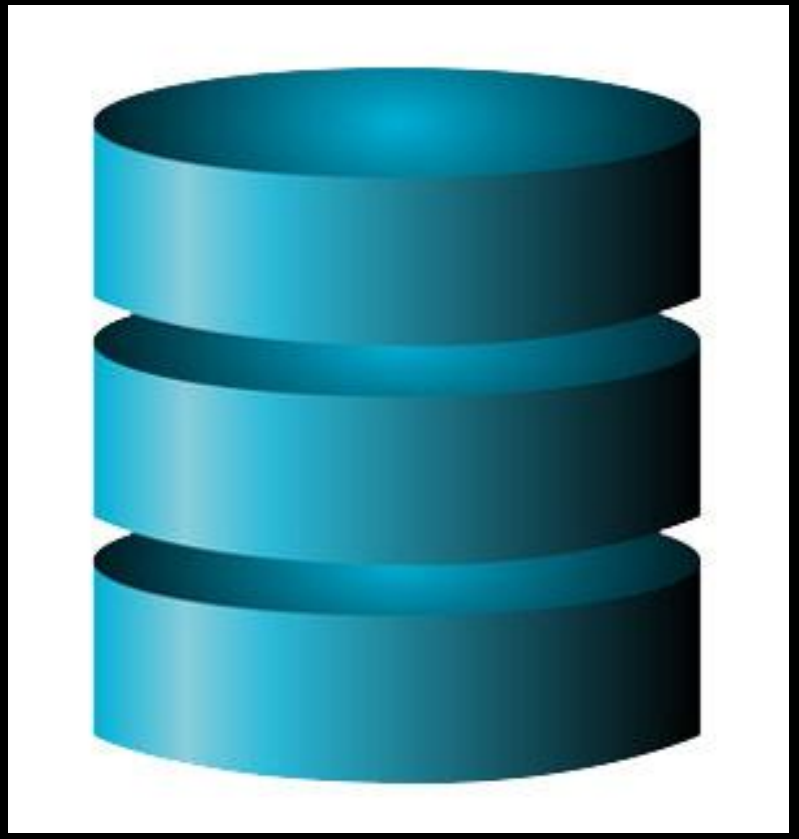
Knowledge Gate Website

- As here we are only concerned with DBMS so we well only two basic operation on database.
- **READ (X)** - Accessing the database item x from disk (where database stored data) to memory variable also name as X.
- **WRITE (X)** - Writing the data item from memory variable X to disk.

Desirable properties of transaction

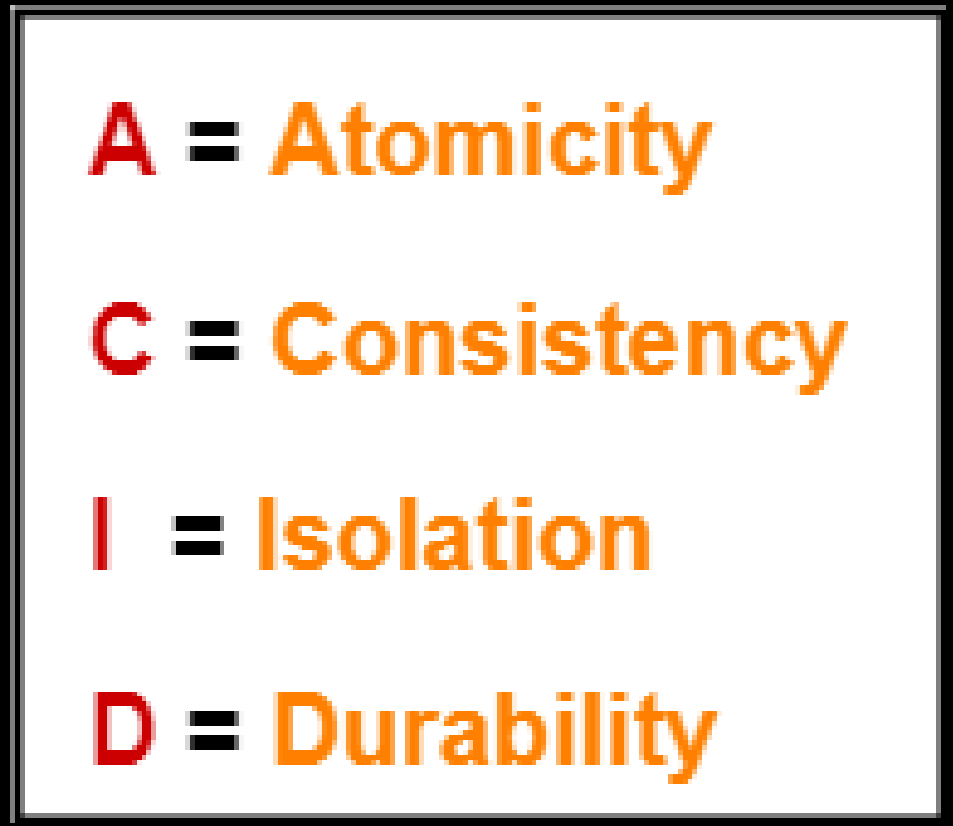
- Now as the smallest unit which have atomicity in DBMS view is transaction, so if want that our data should be consistent then instead of concentrating on data base, we must concentrate on the transaction for our data to be consistent.

T_1
Read(A)
A = A-100
Write(A)
Read(B)
B = B+100
Write(B)



Knowledge Gate Website

- Transactions should possess several properties, often called the **ACID** properties; to provide integrity and consistency of the data in the database. The following are the ACID properties:



Knowledge Gate Website

- **Atomicity** - A transaction is an atomic unit of processing; it should either be performed in its entirety or not performed at all.

T_1
Read(A)
A = A-100
Write(A)
Read(B)
B = B+100
Write(B)

Knowledge Gate Website

- **Consistency** - A transaction should be consistency preserving, meaning that if it is completely executed from beginning to end without interference from other transactions, it should take the database from one consistent state to another.

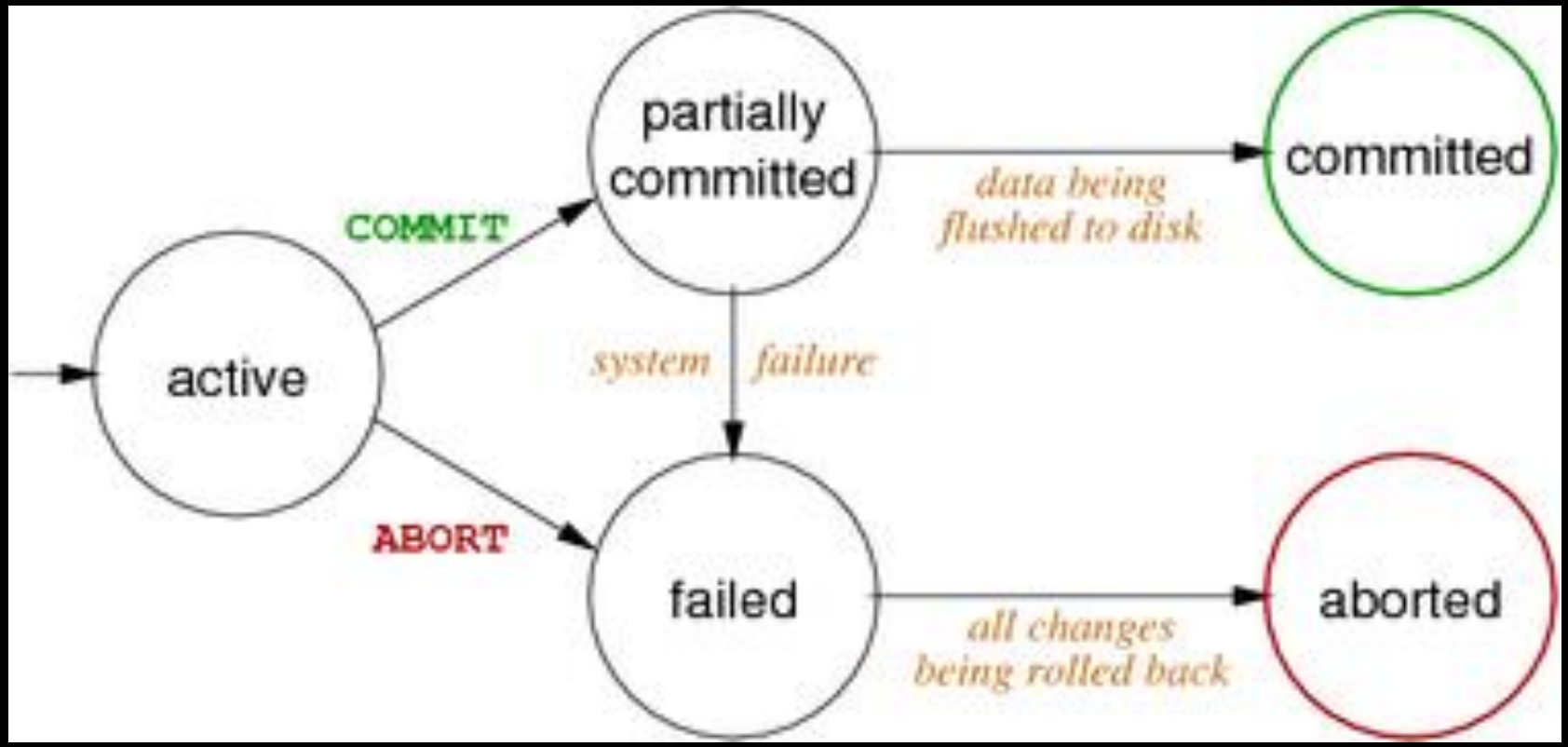
Knowledge Gate Website

- **Isolation -** A transaction should appear as though it is being executed in isolation from other transactions, even though many transactions are executing concurrently.
- That is, the execution of a transaction should not be interfered with by any other transactions executing concurrently.

- **Durability -** The changes applied to the database by a committed transaction must persist in the database.

Transaction states

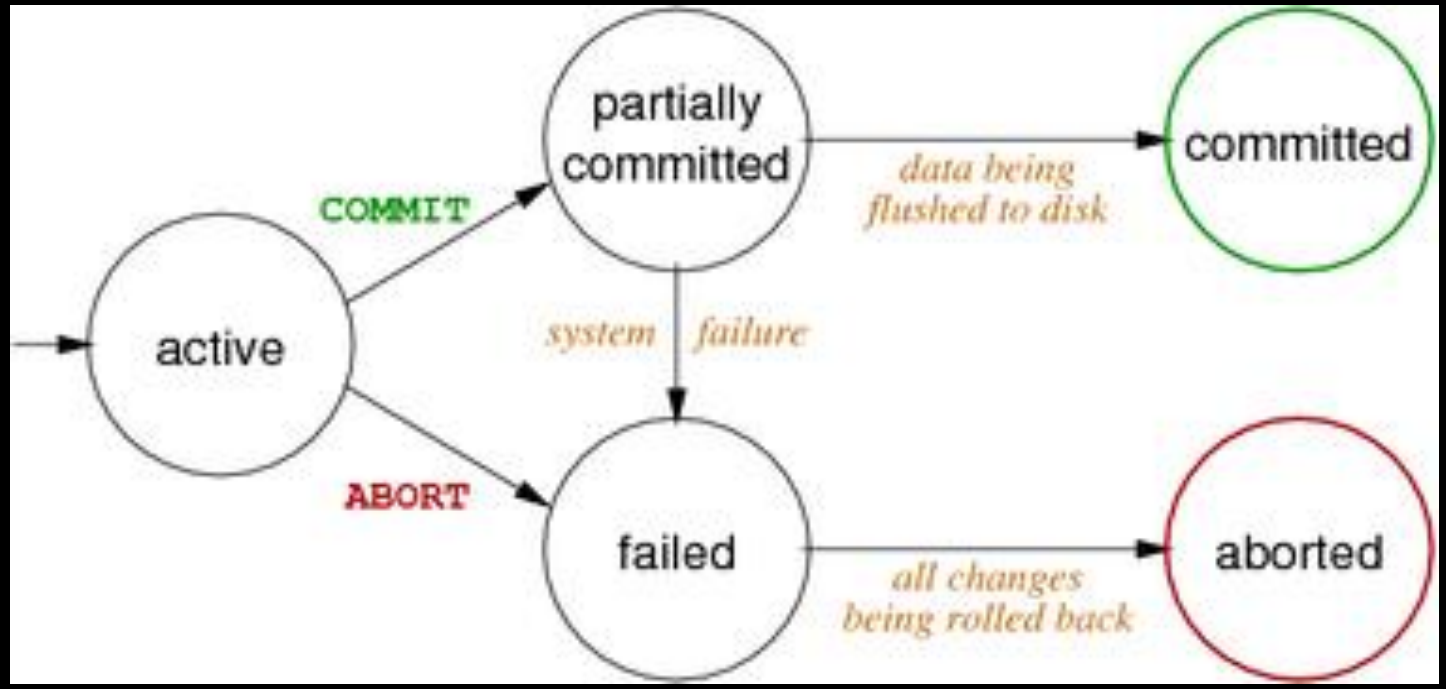
- **ACTIVE** - It is the initial state. Transaction remains in this state while it is executing operations.



Knowledge Gate Website

Transaction states

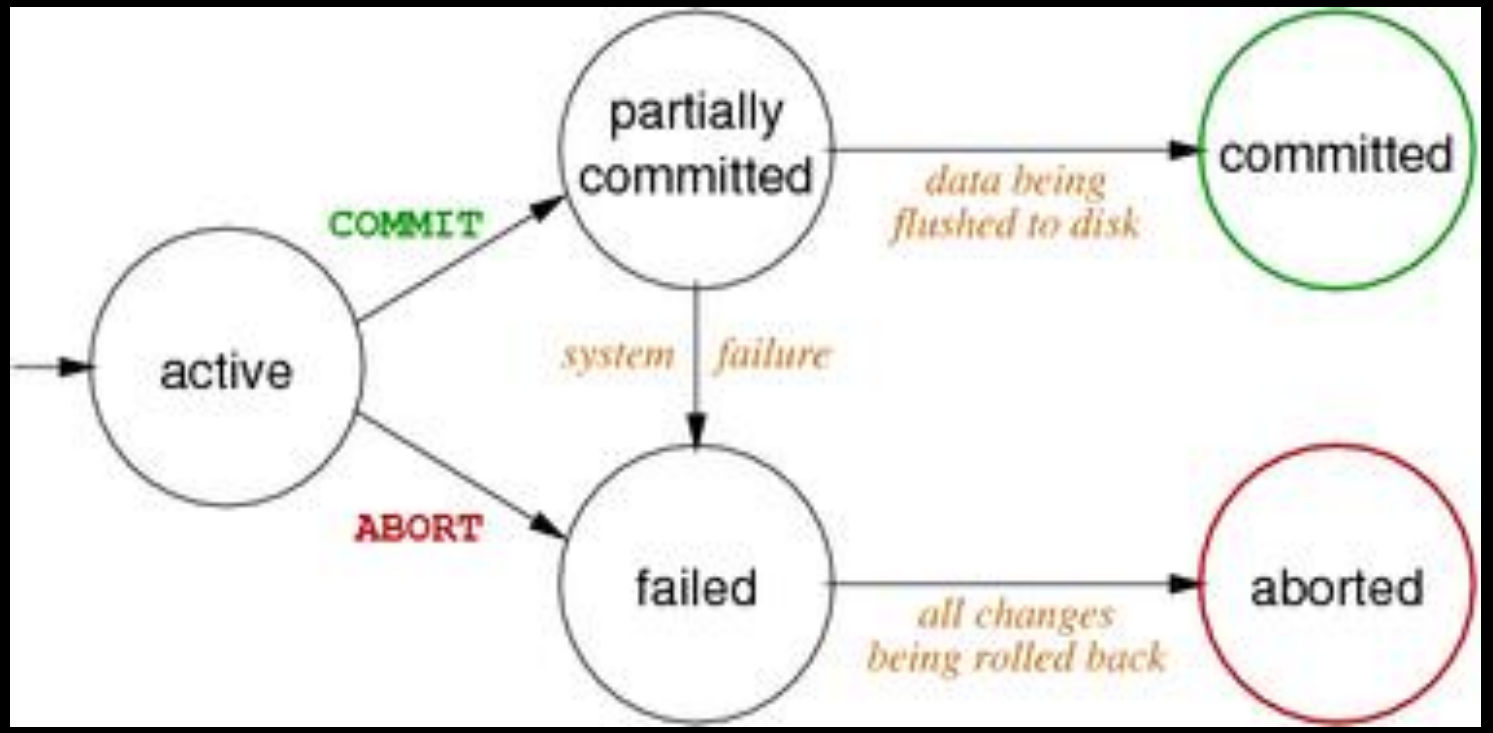
- **PARTIALLY COMMITTED** - After the final statement of a transaction has been executed, the state of transaction is partially committed as it is still possible that it may have to be aborted (due to any failure) since the actual output may still be temporarily residing in main memory and not to disk.



Knowledge Gate Website

Transaction states

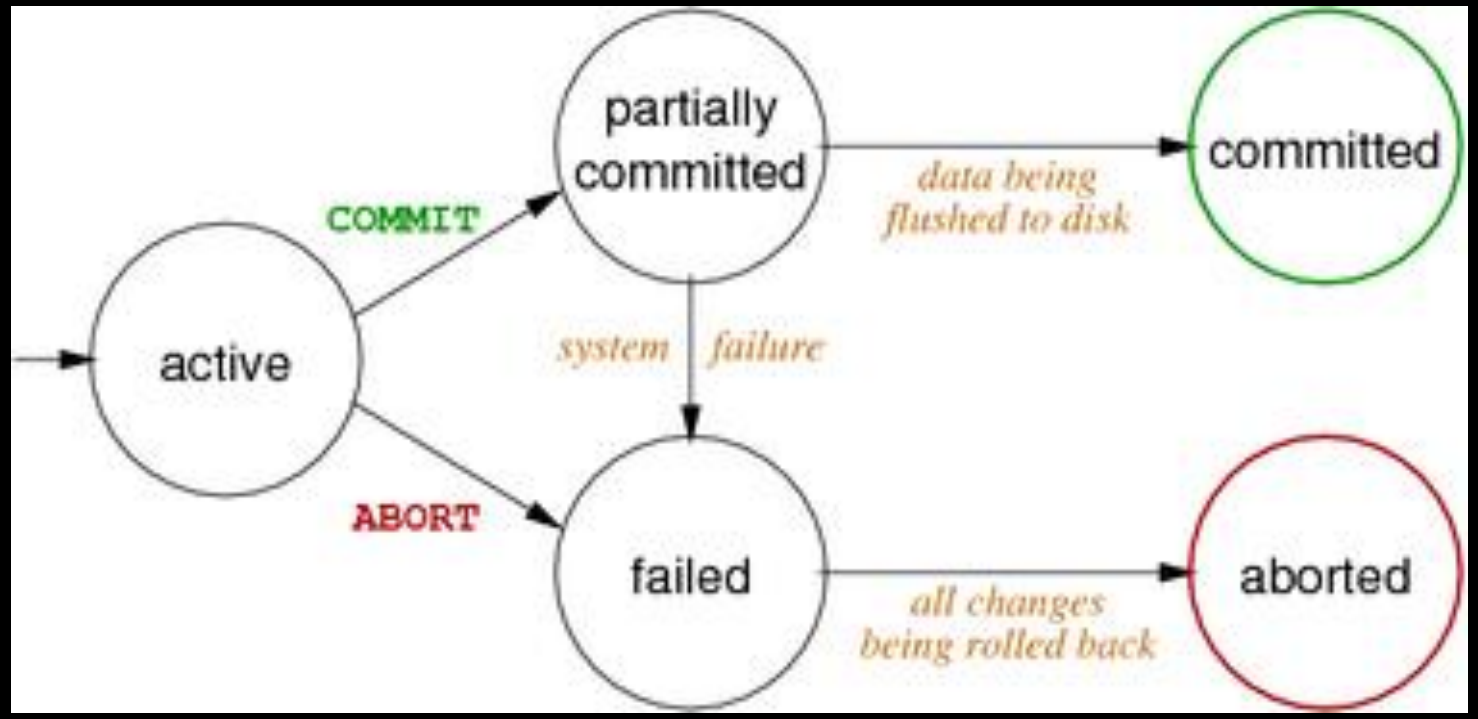
- **FAILED** - After the discovery that the transaction can no longer proceed (because of hardware /logical errors). Such a transaction must be rolled back.



Knowledge Gate Website

Transaction states

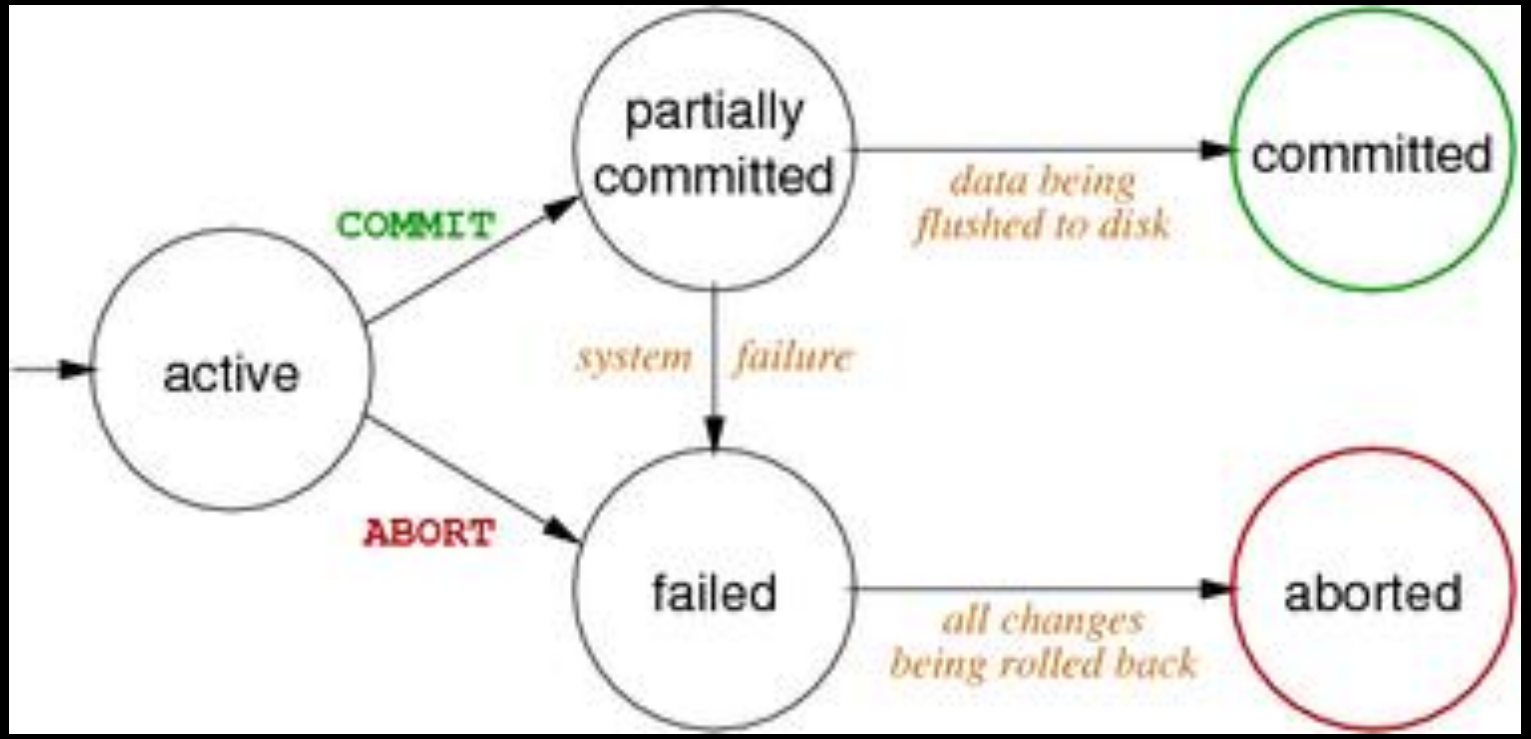
- **ABORTED** - A transaction is said to be in aborted state when the when the transaction has been rolled back and the database has been restored to its state prior to the start of execution.



Knowledge Gate Website

Transaction states

- **COMMITTED** - A transaction enters committed state after successful completion of a transaction and final updation in the database.



Knowledge Gate Website

Why we need concurrent execution

- Concurrent execution is necessary because-
  - It leads to good database performance , less weighting time.
  - Overlapping I/O activity with CPU increases throughput and response time.

Knowledge Gate Website

PROBLEMS DUE TO CONCURRENT EXECUTION OF TRANSACTION

- But interleaving of instructions between transactions may also lead to many problems that can lead to inconsistent database.

- Sometimes it is possible that even though individual transaction are satisfying the acid properties even though the final statues of the system will be inconsistent.

Solution is Schedule

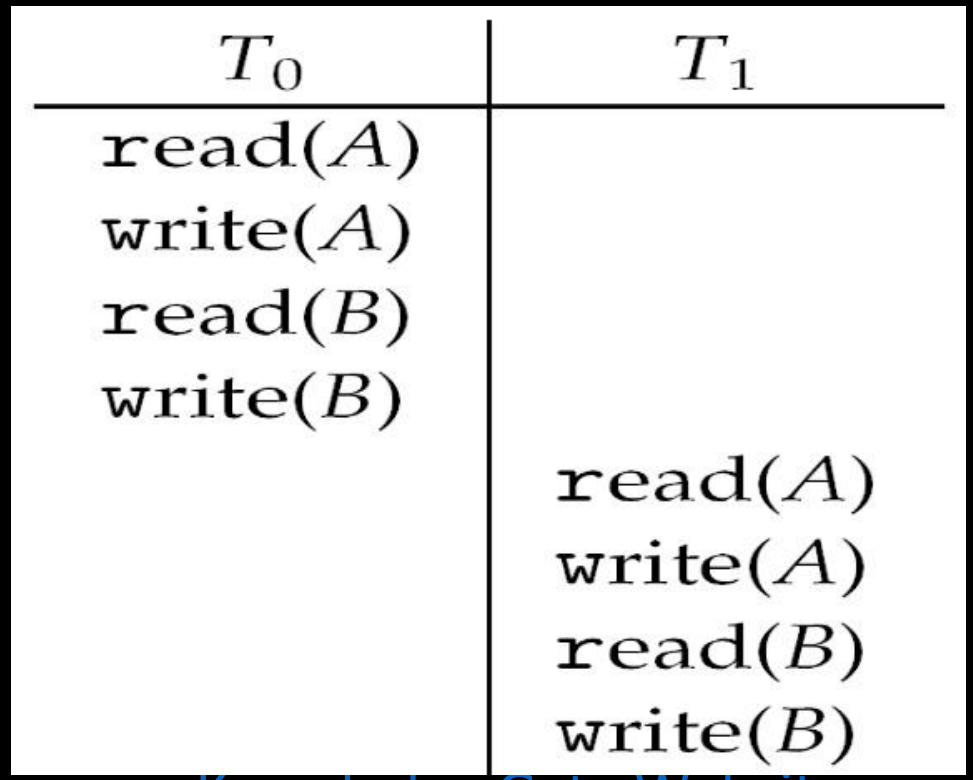
- When two or more transaction executed together or one after another then they can be bundled up into a higher unit of execution called schedule.

T_0	T_1
read(A)	
write(A)	
read(B)	
write(B)	
	read(A)
	write(A)
	read(B)
	write(B)

T_2	T_3
read(B)	
	read(B)
	write(B)
read(A)	
	read(A)
	write(A)

Knowledge Gate Website

- **Serial schedule** - A serial schedule consists of sequence of instruction belonging to different transactions, where instructions belonging to one single transaction appear together. Before complete execution of one transaction another transaction cannot be started. Every serial schedule lead database into consistent state.



Knowledge Gate Website

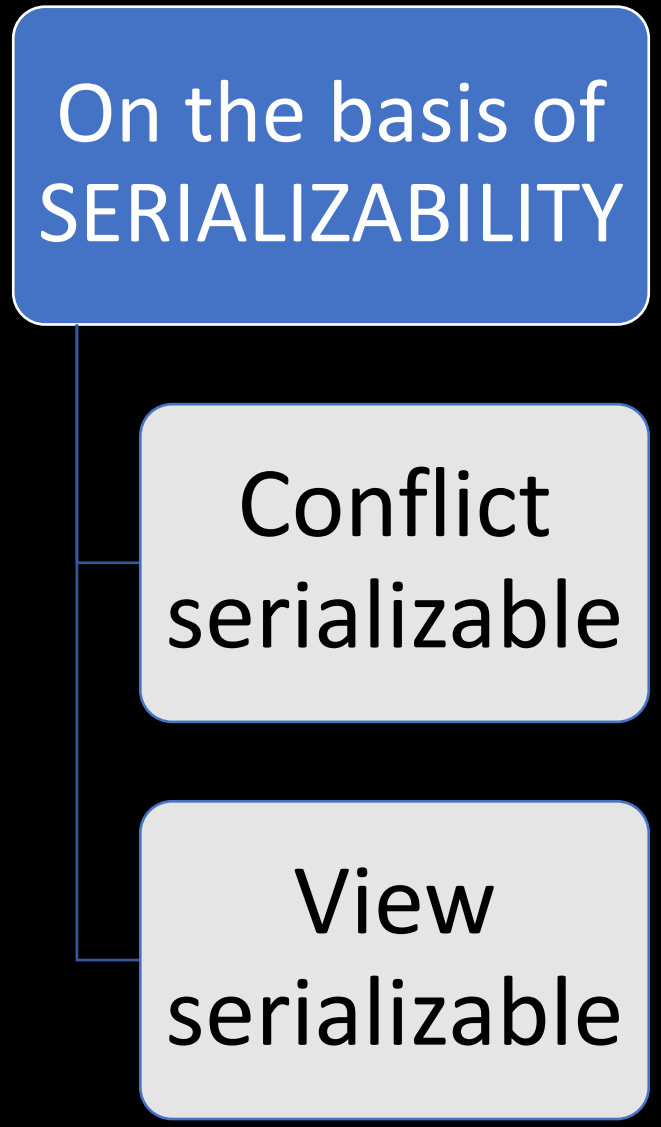
- **Non-serial schedule** - A schedule in which sequence of instructions of a transaction appear in the same order as they appear in individual transaction but the instructions may be interleaved with the instructions of different transactions i.e. concurrent execution of transactions takes place.

T_2	T_3
read(B)	
	read(B)
	write(B)
read(A)	
	read(A)
	write(A)

Knowledge Gate Website

- **Conclusion of schedules**
  - We do not have any method to proof that a schedule is consistent, but from the above discussion we understand that a serial schedule will always be consistent.

  - So if somehow we proof that a non-serial schedule will also have same effects as of a serial schedule than we can proof that, this particular non-serial schedule will also be consistent.



Knowledge Gate Website

T_1	T_2
R(A)	
	R(B)

T_1	T_2
	R(B)
R(A)	

T_1	T_2
R(A)	
	W(A)

T_1	T_2
	W(A)
R(A)	

T_1	T_2
R(A)	
	W(B)

T_1	T_2
	W(B)
R(A)	

T_1	T_2
	R(A)
W(A)	

T_1	T_2
W(A)	
	R(A)

T_1	T_2
R(A)	
	R(A)

T_1	T_2
	R(A)
R(A)	

T_1	T_2
	W(A)
W(A)	

T_1	T_2
W(A)	
	W(A)

Knowledge Gate Website

Conflict equivalent – if one schedule can be converted to another schedule by swapping of non- conflicting instruction then they are called conflict equivalent schedule.

T_1	T_2
R(A)	
A=A-50	
	R(B)
	B=B+50
R(B)	
B=B+50	
	R(A)
	A=A+10

T_1	T_2
	R(B)
	B=B+50
R(A)	
A=A-50	
R(B)	
B=B+50	
	R(A)
	A=A+10

Knowledge Gate Website

SERIALIZABILITY

- ***Conflicting instructions*** **-** Let I and J be two consecutive instructions belonging to two different transactions T_i and T_j in a schedule S, the possible I and J instruction can be as-
  - I= READ(Q), J=READ(Q) ->*Non-conflicting*
  - I= READ(Q), J=WRITE(Q) ->*Conflicting*
  - I= WRITE(Q), J=READ(Q) ->*Conflicting*
  - I= WRITE(Q), J=WRITE(Q) ->*Conflicting*

- So, the instructions I and J are said to be conflicting, if they are operations by different transactions on the same data item, and at least one of these instructions is a write operation.

CONFLICT SERIALIZABLE

- The schedules which are conflict equivalent to a serial schedule are called conflict serializable schedule.
- If a schedule S can be transformed into a schedule S' by a series of swaps of non- conflicting instructions, we say that S and S' are ***conflict equivalent.*** A schedule S is ***conflict serializable***, if it is conflict equivalent to a serial schedule.

T_1	T_2
read(A)	
write(A)	
	read(A)
	write(A)
read(B)	
write(B)	
	read(B)
	write(B)

T_1	T_2
read(A)	
write(A)	
read(B)	
write(B)	
	read(A)
	write(A)
	read(B)
	write(B)

Knowledge Gate Website

Q Consider the following schedule for transactions T_1, T_2 and T_3: what is the correct serialization of the above?

T1	T2	T3
Read (X)		
	Read (Y)	
		Read (Y)
	Write (Y)	
Write (X)		
		Write (X)
	Read (X)	
	Write (X)	

Knowledge Gate Website

Procedure for determining conflict serializability of a schedule

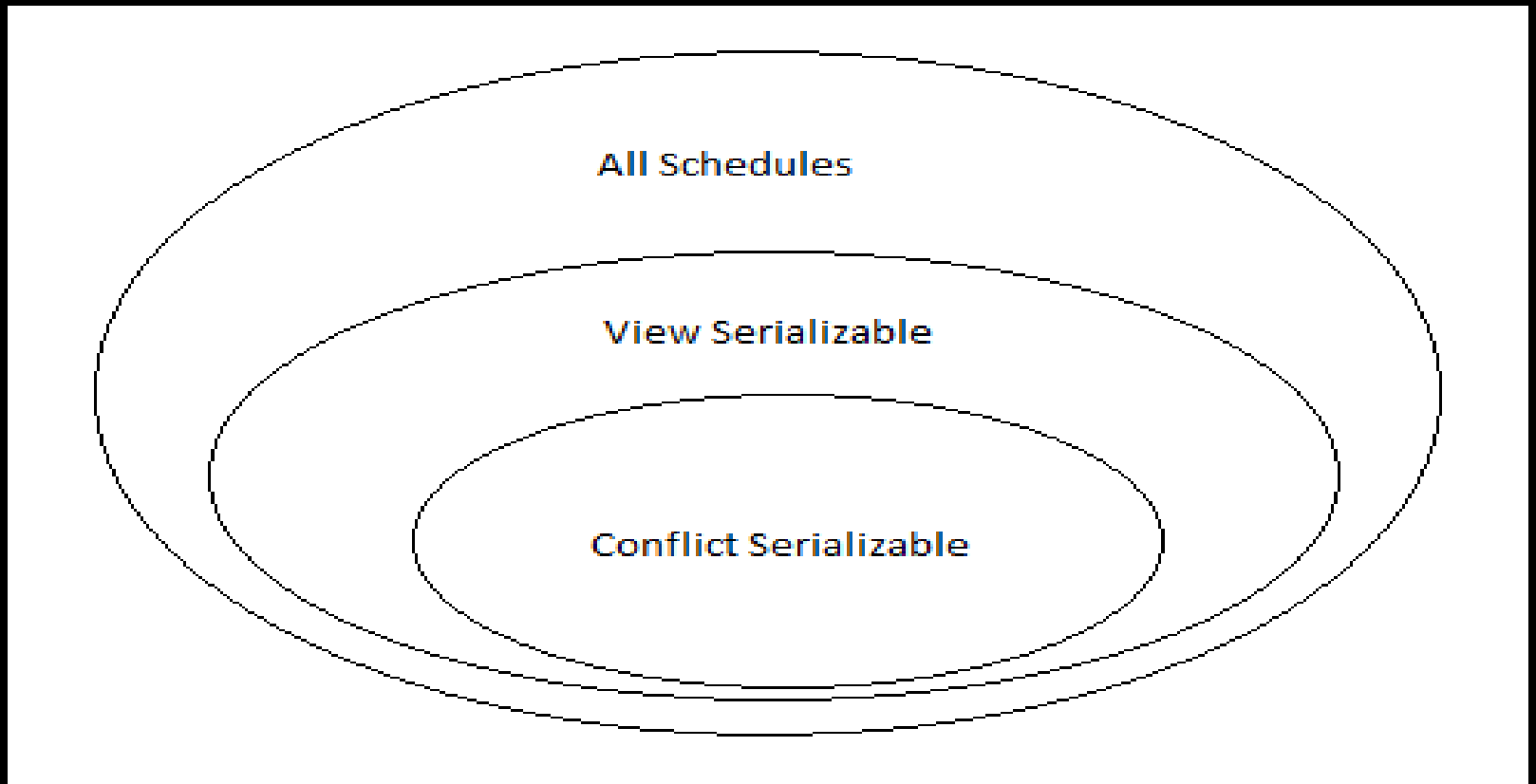
- It can be determined using PRECEDENCE GRAPH method:
- A precedence graph consists of a pair G (V, E)
- V= set of vertices consisting of all the transactions participating in the schedule.
- E= set of edges consists of all edges $T_i \rightarrow T_j$, for which one of the following conditions holds:
  - T_i executes write(Q) before T_j executes read(Q)
  - T_i executes read(Q) before T_j executes write(Q)
  - T_i executes write(Q) before T_j executes write(Q)

- If an edge $T_i \rightarrow T_j$ exists in the precedence graph, then in any serial schedule S' equivalent to S, Ti must appear before T_j.
- **If the precedence graph for S has no cycle, then schedule S is conflict serializable, else it is not.**

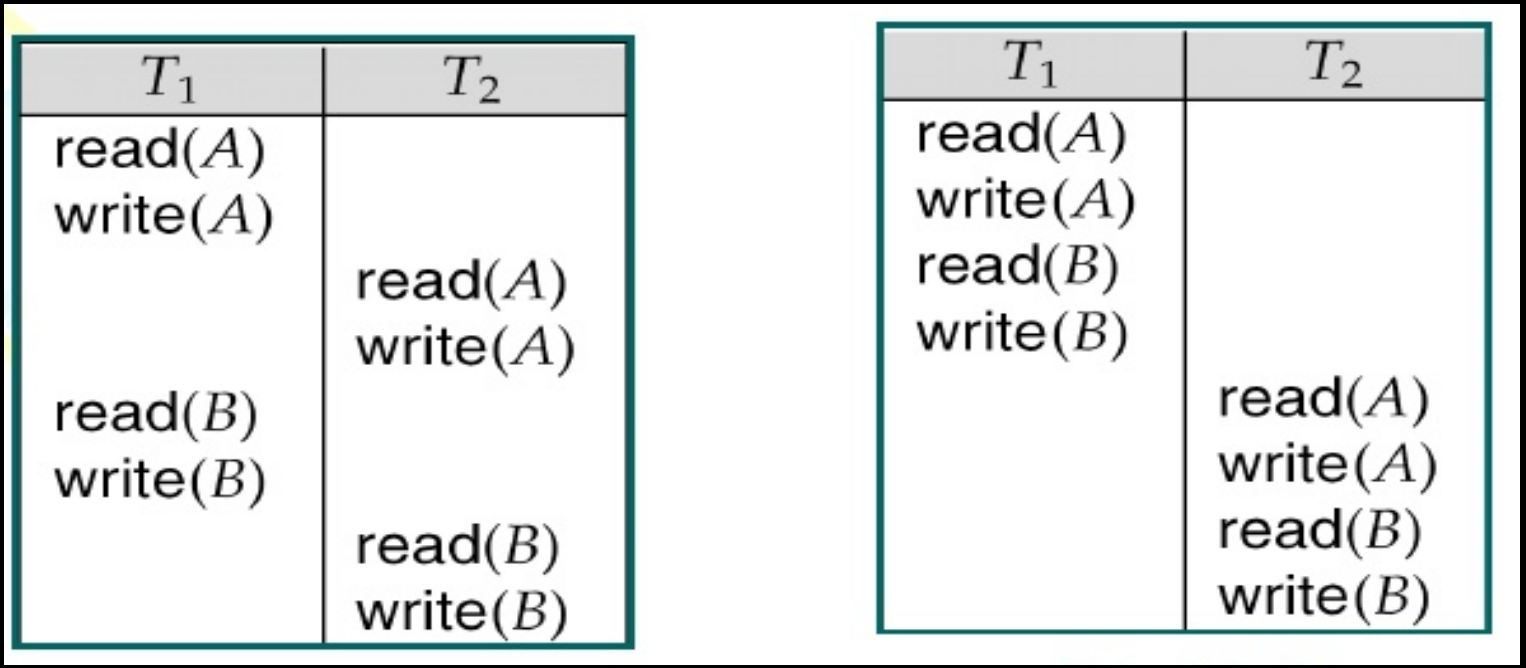
VIEW SERIALIZABLE

- If a schedule is not conflict serializable, still it can be consistent, so let us study a weaker form of serializability called View serializability, and even if a schedule is view serializable still it can be consistent.

- If a schedule is conflict serializable then it will also be view serializable, so we must check view serializability only if a schedule is not conflict serializable.



- Two schedules S and S' are view equivalent, if they satisfy following conditions –
  - For each data item Q, if the transaction T_i reads the initial value of Q in schedule S , then then the transaction T_i must, in schedule S' ,also read the initial value of Q.
  - If a transaction T_i in schedule S reads any data item Q, which is updated by transaction T_j, then a transaction T_i must in schedule S' also read data item Q updated by transaction T_j in schedule S'.
  - For each data item Q, the transaction (if any) that performs the final write(Q) operation in schedule S, then the same transaction must also perform final write(Q) in schedule S'.



T_1	T_2
read(A)	
write(A)	
	read(A)
	write(A)
read(B)	
write(B)	
	read(B)
	write(B)

T_1	T_2
read(A)	
write(A)	
read(B)	
write(B)	
	read(A)
	write(A)
	read(B)
	write(B)

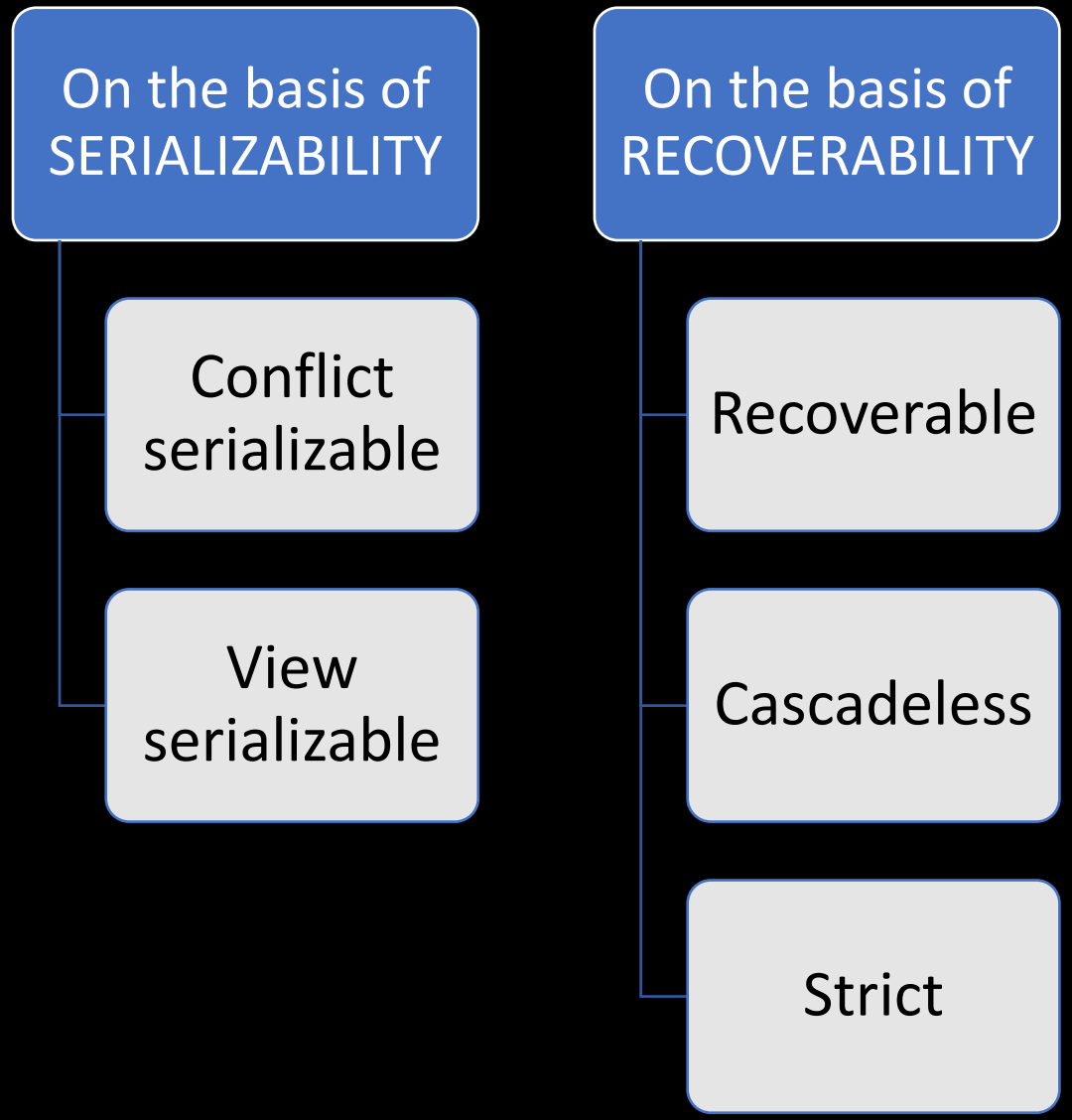
Knowledge Gate Website

View Serializable

A schedule S is view serializable, if it is view equivalent to a serial schedule.

Schedule A				Serial schedule <T3,T4,T6>		
T3	T4	T6		T3	T4	T6
read(Q)				read(Q)		
	write(Q)			write(Q)		
write(Q)					write(Q)	
		write(Q)				write(Q)

Knowledge Gate Website



Knowledge Gate Website

NON- RECOVERABLE Vs RECOVERABLE SCHEDULE

- A schedule in which for each pair of transaction T_i and T_j, such that if T_j reads a data item previously written by T_i, then the commit or abort operation of T_i appears before T_j. Such a schedule is called Non- Recoverable schedule.
- A schedule in which for each pair of transaction T_i and T_j, such that if T_j reads a data item previously written by T_i, then the commit or abort of T_i must appear before T_j. Such a schedule is called Recoverable schedule.

S	
T_1	T_2
R(X)	
W(X)	
	R(X)
	C
C	

S	
T_1	T_2
R(X)	
W(X)	
	R(X)
C	
	C

Knowledge Gate Website

CASCADING ROLLBACK Vs CASCADELESS SCHEDULE

- It is a phenomenon, in which even if the schedule is recoverable, the a single transaction failure leads to a series of transaction rollbacks, is called cascading rollback. Cascading rollback is undesirable, since it leads to undoing of a significant amount of work. Uncommitted reads are not allowed in cascade less schedule.

- A schedule in which for each pair of transactions T_i and T_j, such that if T_j reads a data item previously written by T_i then the commit or abort of T_i must appear before read operation of T_j. Such a schedule is called cascade less schedule.

S		
T_1	T_2	T_3
R(X)		
W(X)		
	R(X)	
	W(X)	
		R(X)
C		
	C	
		C

S		
T_1	T_2	T_3
R(X)		
W(X)		
C		
	R(X)	
	W(X)	
	C	
		R(X)
		C

Knowledge Gate Website

Strict Schedule

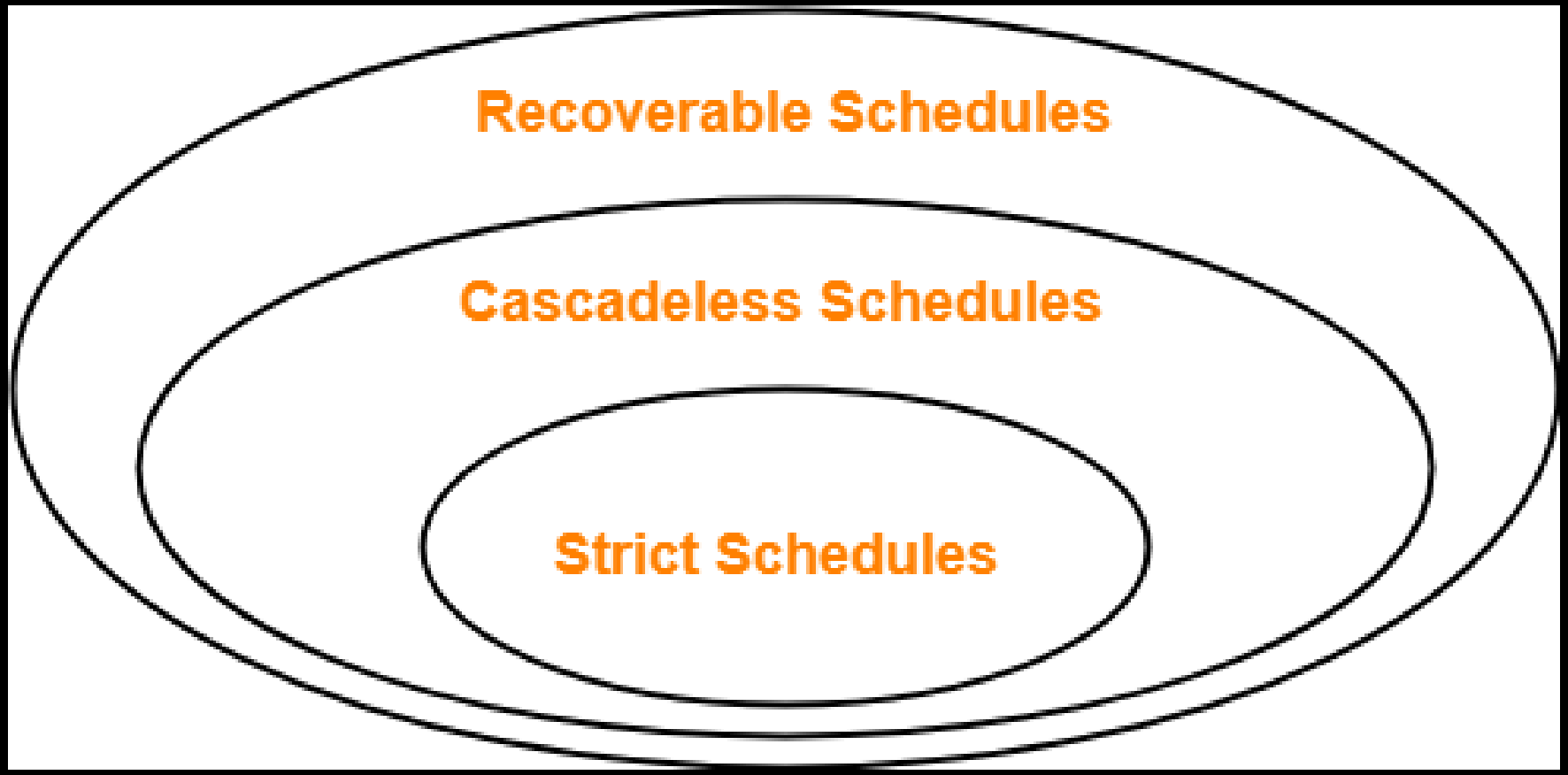
- A schedule in which for each pair of transactions T_i and T_j, such that if T_j reads a data item previously written by T_i then the commit or abort of T_i must appear before read and write operation of T_j.

S_1	
T_1	T_2
R(a)	
W(a)	
	W(a)
C	
	R(a)
	C

S_2	
T_1	T_2
R(a)	
W(a)	
C	
	W(a)
	R(a)
	C

S_3	
T_1	T_2
R(a)	
	R(b)
W(a)	
	W(b)
C	
	R(a)
	C

Knowledge Gate Website



Knowledge Gate Website

Log based Recovery

- The system log, or transaction log, records all the changes or activities happening in the database, ensuring that transactions are durable and can be recovered in the event of a failure.

- Different types of log records represent various stages or actions in a transaction.
  - <T_i start>: This log entry indicates that the transaction T_i has started its execution.
  - < T_i, X_j, V_1, V_2>: This log entry documents a write operation. It states that transaction T_i has changed the value of data item X_j from V_1 to V_2.
  - < T_i commit>: This log entry marks the successful completion of transaction T_i.
  - < T_i abort>: This log entry denotes that the transaction T_i has been aborted, either due to an error or a rollback operation.

- Before any write operation modifies the database, a log record of that operation needs to be created. This is to ensure that in case of a failure, the system can restore the database to a consistent state using the log records.

Knowledge Gate Website

Aspect	Deferred Database Modification	Immediate Database Modification
Write Operation Timing	Only at the commit point.	As soon as changes occur.
I/O Operations	Reduced, as changes are batched and written at once during the commit, saving on intermediate I/O operations.	Increased, as each change triggers immediate write operations, leading to more frequent I/O operations.
Recovery Complexity	Simpler, as uncommitted changes are not reflected in the database, making rollback easier in case of failures.	More complex, as it may require undoing changes from uncommitted transactions that have been written to the database.

Knowledge Gate Website

Shadow Paging Recovery Technique

In the shadow paging recovery technique, the database maintains two page tables during a transaction: the current page table (reflecting the state before the transaction began) and the shadow page table (which tracks changes made during the transaction). Here's how it operates:

- **Initialization**: When a transaction begins, the database creates a shadow copy of the page table. The database pages themselves are not duplicated; only the page table entries are duplicated.

- **Modifications**: As the transaction progresses, any changes are reflected in the current page table, while the shadow page table retains the original state. If a page is modified, a new copy of the page is created, and the current page table is updated to point to this new version, thereby ensuring that the shadow page table still points to the original unmodified page.

Knowledge Gate Website

- **Commit**: Upon transaction commit, the shadow page table is discarded, and the current page table becomes the new committed state of the database. The database atomically switches to using the current page table, ensuring that changes are installed all at once.

- **Recovery**: In case of a system failure before the transaction commits, the database can easily recover by discarding the current page table and reverting back to the shadow page table, thereby restoring the database to its state before the transaction began.

Knowledge Gate Website

Data fragmentation

- Data fragmentation in the context of a Database Management System (DBMS) refers to the process of breaking down a database into smaller, manageable pieces, and distributing these pieces across a network of computers. This is a significant component in distributed database systems.
  - **Horizontal Fragmentation**: In DBMS, horizontal fragmentation divides a table into sections based on rows. Each fragment contains a subset of rows, usually segmented based on certain conditions or attributes. This can enhance performance by facilitating parallel processing and quicker data retrieval, especially useful in distributed database systems.
  - **Vertical Fragmentation**: This fragmentation type divides a database table by columns, with different sets of columns stored in separate fragments. This approach is employed when different users require access to varied data attributes, promoting efficient data handling and minimizing data transfer times, which is advantageous in scenarios where queries only necessitate a subset of attributes.
  - **Hybrid Fragmentation**: Hybrid or mixed fragmentation in DBMS combines both horizontal and vertical strategies, optimizing data storage and retrieval for complex access patterns. This method can potentially offer performance improvements by utilizing the merits of both horizontal and vertical fragmentation, and is typically implemented in databases with complex, multifaceted data access requirements.

Knowledge Gate Website

Distributed database

- A distributed database is a database in which storage devices are not all attached to a common processor. It may be stored in multiple computers, located in the same physical location, or dispersed over a network of interconnected computers.

- **Data Replication**
  - ***Advantages:*** Increased Availability, Improved Performance, Enhanced Reliability
  - ***Disadvantages:*** Storage Costs, Maintenance Complexity, Write Complexity.

- **Data Fragmentation**
  - ***Advantages:*** Efficient Data Access, Distributed Processing, Localized Management.
  - ***Disadvantages:*** Complexity, Dependency on Network, Reconstruction Issues.

Knowledge Gate Website

CONCURRENCY CONTROL

Here we will study those protocol which guarantee to generate schedule which always satisfy desirable properties like conflict serializability. Along with we desire the following properties from schedule generating protocols

- Concurrency should be as high as possible, as this is our ultimate goal because of which we are making all the effort.
- The time taken by a transaction should also be less.
- Easy to understand and implement.

- **Time stamping based method: -** Where before entering the system, a specific order is decided among the transaction, so in case of a clash we can decide which one to allow and which to stop.

- **Lock based method: -** where we ask a transaction to first lock a data item before using it. So that no different transaction can use a data at the same time, removing any possibility of conflict.
  - 2 phase locking
    1. Basic 2pl
    2. Conservative 2pl
    3. Rigorous 2pl
    4. Strict 2pl
  - Graph based protocol

- **Validation based protocol –** Majority of transactions are read only transactions, the rate of conflicts among the transaction may be low, thus many of transaction, if executed without the supervision of a concurrency control scheme, would nerveless leave the system in a consistent state.

Knowledge Gate Website

TIME STAMP ORDERING PROTOCOL

- Basic idea of time stamping is to decide the order between the transaction before they enter in the system using a stamp (time stamp), in case of any conflict during the execution order can be decided using the time stamp.

- Let’s understand how this protocol works, here we have two idea of timestamping, one for the transaction, and other for the data item.

- Time stamp for transaction,
  - With each transaction t_i, in the system, we associate a unique fixed timestamp, denoted by $TS(t_i)$.
  - This timestamp is assigned by database system to a transaction at time transaction enters into the system.
  - If a transaction has been assigned a timestamp $TS(t_i)$ and a new transaction t_j , enters into the system with a timestamp $TS(t_j)$, then always $TS(t_i) < TS(t_j)$.

- Two things are to be noted

1. First time stamp of a transaction remain fixed throughout the execution

2. Second it is unique means no two transaction can have the same timestamp.

- Time stamp with data item, in order to assure such scheme, the protocol maintains for each data item Q two timestamp values:

    1. **W-timestamp(Q)** is the largest time-stamp of any transaction that executed write(Q) successfully.

    2. **R-timestamp(Q)** is the largest time-stamp of any transaction that executed read(Q) successfully.

- These timestamps are updated whenever a new read(Q) or write(Q) instruction is executed.

Knowledge Gate Website

- Suppose a transaction Ti request a ***read(Q)***

    1. If **TS(Ti) < W-timestamp(Q),** then Ti needs to read a value of Q that was already overwritten. Hence, the read operation is rejected, and Ti is rolled back.

    2. If **TS(Ti)≥ W-timestamp(Q),** then the read operation is executed, and R-timestamp(Q) is set to the maximum of R-timestamp(Q) and TS(T_i).

- Suppose that transaction Ti issues ***write(Q).***
  1. If **TS(Ti) < R-timestamp(Q),** then the value of Q that Ti is producing was needed previously, and the system assumed that that value would never be produced. Hence, the write operation is rejected, and Ti is rolled back.

  2. If **TS(Ti) < W-timestamp(Q),** then Ti is attempting to write an obsolete value of Q. Hence, this write operation is rejected, and Ti is rolled back.

  3. If **TS(Ti) ≥ R-timestamp(Q),** then the write operation is executed, and W-timestamp(Q) is set to max(W-timestamp(Q), TS(Ti)).

  4. If **TS(Ti) ≥ W-timestamp(Q),** then the write operation is executed, and W-timestamp(Q) is set to max(W-timestamp(Q), TS(Ti)).

	Conflict Serializability	View Serializability	Recoverability	Cascadelessness	Deadlock Freedom
Time Stamp Ordering	YES	YES	NO	NO	YES
Thomas Write Rule					
Basic 2PL					
Conservative 2PL					
Rigorous 2PL					
Strict 2PL					

Knowledge Gate Website

THOMAS WRITE RULE

- Thomas write is an improvement in time stamping protocol, which makes some modification and may generate those protocols that are even view serializable, because it allows greater potential concurrency.

- It is a Modified version of the timestamp-ordering protocol in which Blind write operations may be ignored under certain circumstances.

- The protocol rules for read operations remain unchanged. while for write operation, there is slightly change in Thomas write rule than timestamp ordering protocol.

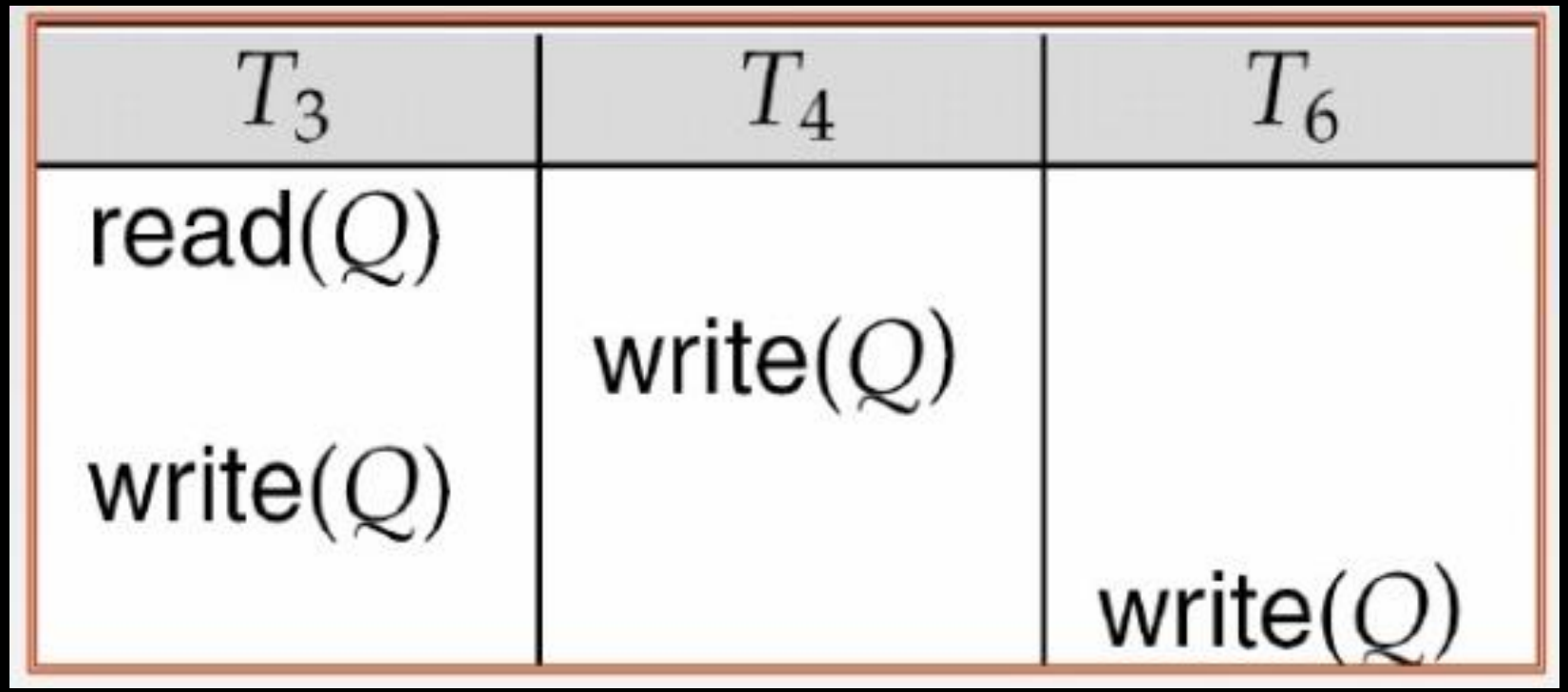
Knowledge Gate Website

When T_i attempts to write data item Q,

- **if TS(T_i) < W-timestamp(Q),** then T_i is attempting to write an obsolete value of {Q}. Rather than rolling back T_i as the timestamp ordering protocol would have done, this {write} operation can be ignored.

- This modification is valid as the any transaction with **TS(T_i) < W-timestamp(Q),** the value written by this transaction will never be read by any other transaction performing Read(Q) ignoring such obsolete write operation is considerable.

- Thomas' Write Rule allows greater potential concurrency. Allows some view-serializable schedules that are not conflict serializable.



Knowledge Gate Website

	Conflict Serializability	View Serializability	Recoverability	Cascadelessness	Deadlock Freedom
Time Stamp Ordering	YES	YES	NO	NO	YES
Thomas Write Rule	NO	YES	NO	NO	YES
Basic 2PL					
Conservative 2PL					
Rigorous 2PL					
Strict 2PL					

Knowledge Gate Website

Lock Based Protocols

- To ensure isolation is to require that data items be accessed in a mutually exclusive manner i.e. while one transaction is accessing a data item, no other transaction can modify that data item. Locking is the most fundamental approach to ensure this.

- Lock based protocols ensure this requirement. Idea is first obtain a lock on the desired data item then if lock is granted then perform the operation and then unlock it.

- In general, we support two modes of lock because, to provide better concurrency.

- ***Shared mode***
  - If transaction T_i has obtained a shared-mode lock (denoted by S) on any data item Q, then T_i can read, but cannot write Q, any other transaction can also acquire a shared mode lock on the same data item(this is the reason we called this shared mode).

- **Exclusive mode**
  - If transaction T_i has obtained an exclusive-mode lock (denoted by X) on any data item Q, then T_i can both read and write Q, any other transaction cannot acquire either a shared or exclusive mode lock on the same data item. (this is the reason we called this exclusive mode)

Knowledge Gate Website

Lock –Compatibility Matrix

- Conclusion shared is compatible only with shared while exclusive is not compatible either with shared or exclusive.
- To access a data item, transaction T_i must first lock that item, if the data item is already locked by another transaction in an incompatible mode, or some other transaction is already waiting in non-compatible mode, then concurrency control manager will not grant the lock until all incompatible locks held by other transactions have been released. The lock is then granted.

		Current State of lock of data items		
		Exclusive	Shared	Unlocked
Requested Lock	Exclusive	N	N	Y
	Shared	N	Y	Y
	Unlock	Y	Y	-

Knowledge Gate Website

- Lock based protocol *do not ensure serializability* as granting and releasing of lock do not follow any order and any transaction any time may go for lock and unlock. Here in the example below we can see, that even this transaction in using locking but neither it is conflict serializable nor independent from deadlock.

T_1	T_2
LOCK-X(A)	
READ(A)	
WRITE(A)	
UNLOCK(A)	
	LOCK-S(B)
	READ(B)
	UNLOCK(B)
LOCK-X(B)	
READ(B)	
WRITE(B)	
UNLOCK(B)	
	LOCK-S(A)
	READ(A)
	UNLOCK(A)

Knowledge Gate Website

- If we do not use locking, or if we unlock data items too soon after reading or writing them, we may get inconsistent states, as there exists a possibility of dirty read. On the other hand, if we do not unlock a data item before requesting a lock on another data item, concurrency will be poor.

- We shall require that each transaction in the system follow a set of rules, called a **locking protocol**, indicating when a transaction may lock and unlock each of the data items for e.g. 2pl or graph based locking.

- Locking protocols restrict the number of possible schedules.

Knowledge Gate Website

Two phase locking protocol(2PL)

- The protocol ensures that each transaction issue lock and unlock requests in two phases, note that each transaction will be 2 phased not schedule.

- **Growing phase**- A transaction may obtain locks, but not release any locks.

- **Shrinking phase**- A transaction may release locks, but may not obtain any new locks.

- Initially a transaction is in growing phase and acquires lock as needed and in between can perform operation reach to lock point and once a transaction releases a lock, it can issue no more lock requests i.e. it enters the shrinking phase.

T_1	T_2
LOCK-X(A)	
READ(A)	
WRITE(A)	
	LOCK-S(B)
	READ(B)
LOCK-X(B)	
READ(B)	
WRITE(B)	
	LOCK-S(A)
	READ(A)
	UNLOCK(B)
UNLOCK(A)	
UNLOCK(B)	
	UNLOCK(A)

Knowledge Gate Website

	Conflict Serializability	View Serializability	Recoverability	Cascadelessness	Deadlock Freedom
Time Stamp Ordering	YES	YES	NO	NO	YES
Thomas Write Rule	NO	YES	NO	NO	YES
Basic 2PL	YES	YES	NO	NO	NO
Conservative 2PL					
Rigorous 2PL					
Strict 2PL					

Knowledge Gate Website

Properties

- 2PL ensures conflict serializability, and the ordering of transaction over lock points is itself a serializability order of a schedule in 2PL.
- If a schedule is allowed in 2PL protocol then definitely it is always conflict serializable. But it is not necessary that if a schedule is conflict serializable then it will be generated by 2pl. Equivalent serial schedule is based on the order of lock points.
- View serializability is also guaranteed.
- Does not ensure freedom from deadlock
- May cause non-recoverability.
- Cascading rollback may occur.

Knowledge Gate Website

Conservative 2PL

- The idea is there is no growing phase transaction start directly from lock point, i.e. transaction must first acquire all the required locks then only it can start execution. If all the locks are not available then transaction must release the acquired locks and must wait.

  - Shrinking phase will work as usual, and transaction can unlock any data item anytime.

  - we must have a knowledge in future to understand what is data required so that we can use it

	Conflict Serializability	View Serializability	Recoverability	Cascadelessness	Deadlock Freedom
Time Stamp Ordering	YES	YES	NO	NO	YES
Thomas Write Rule	NO	YES	NO	NO	YES
Basic 2PL	YES	YES	NO	NO	NO
Conservative 2PL	YES	YES	NO	NO	YES
Rigorous 2PL					
Strict 2PL					

Knowledge Gate Website

RIGOROUS 2PL

- Requires that all locks be held until the transaction commits.
- This protocol requires that locking be two phase and also all the locks taken be held by transaction until that transaction commit.
- Hence there is no shrinking phase in the system.

	Conflict Serializability	View Serializability	Recoverability	Cascadelessness	Deadlock Freedom
Time Stamp Ordering	YES	YES	NO	NO	YES
Thomas Write Rule	NO	YES	NO	NO	YES
Basic 2PL	YES	YES	NO	NO	NO
Conservative 2PL	YES	YES	NO	NO	YES
Rigorous 2PL	YES	YES	YES	YES	NO
Strict 2PL					

Knowledge Gate Website

STRICT 2PL

- that all exclusive-mode locks taken by a transaction be held until that transaction commits. This requirement ensures that any data written by an uncommitted transaction are locked in exclusive mode until the transaction commits, preventing any other transaction from reading the data.

- This protocol requires that locking be two phase and also that exclusive –mode locks taken by transaction be held until that transaction commits.

- So it is simplified form of rigorous 2pl

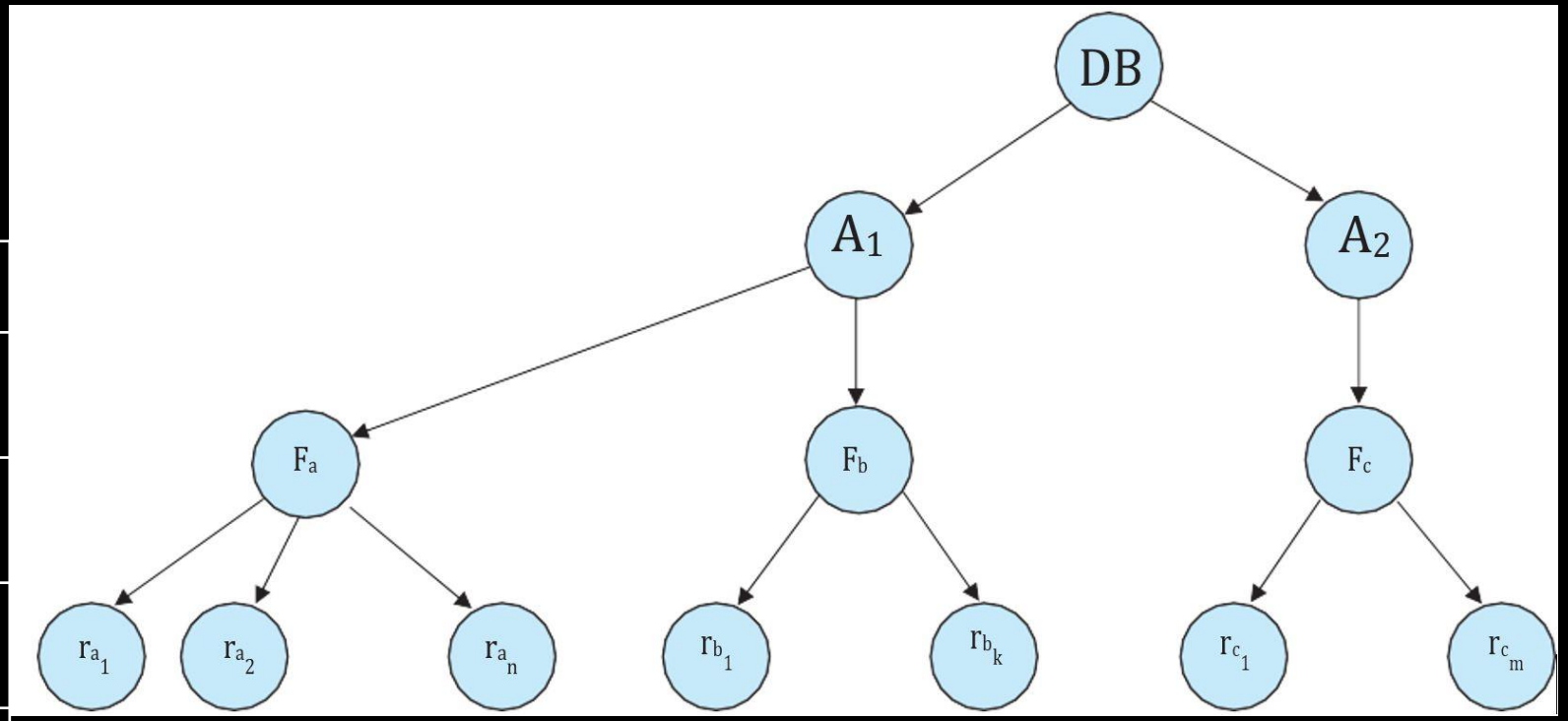
Knowledge Gate Website

	Conflict Serializability	View Serializability	Recoverability	Cascadelessness	Deadlock Freedom
Time Stamp Ordering	YES	YES	NO	NO	YES
Thomas Write Rule	NO	YES	NO	NO	YES
Basic 2PL	YES	YES	NO	NO	NO
Conservative 2PL	YES	YES	NO	NO	YES
Rigorous 2PL	YES	YES	YES	YES	NO
Strict 2PL	YES	YES	YES	YES	NO

Knowledge Gate Website

Multiple Granularity

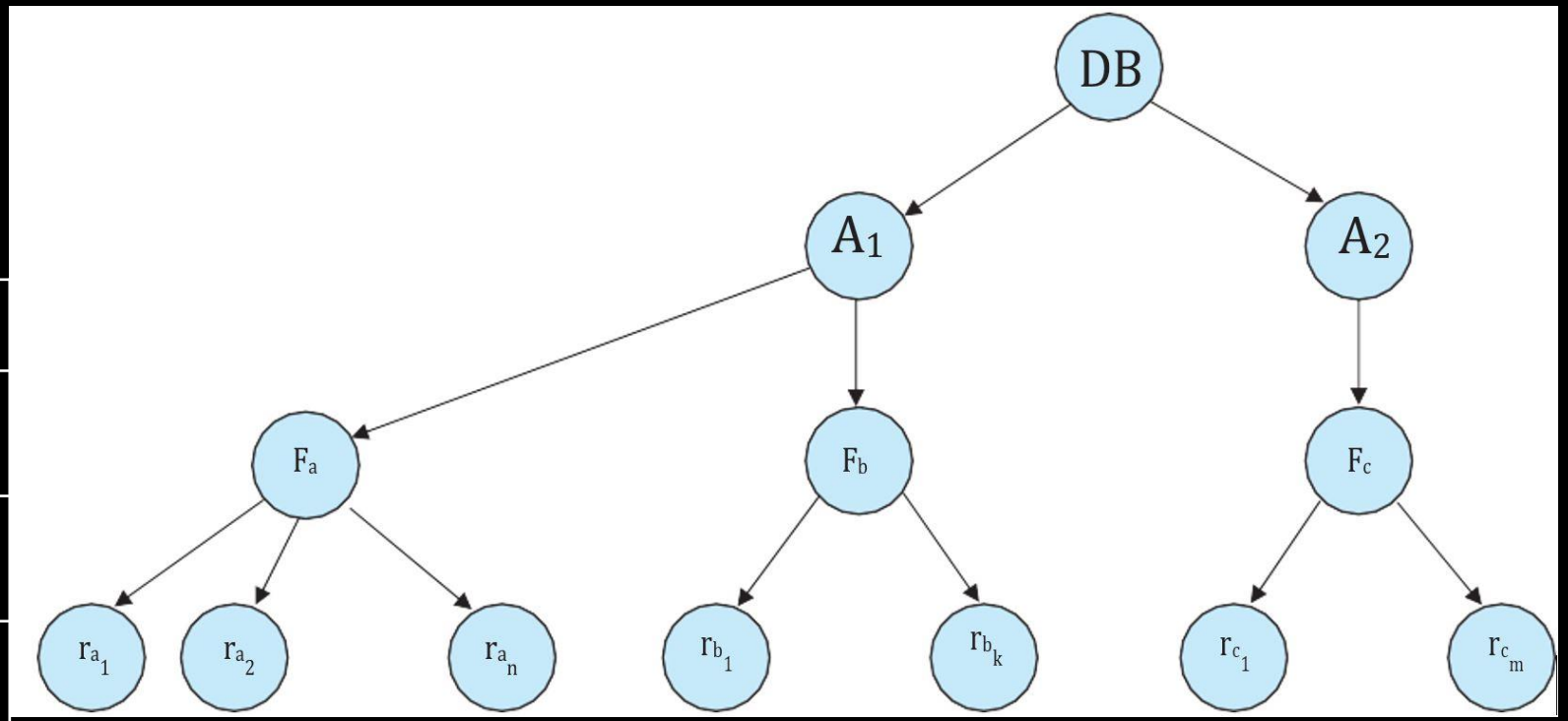
	IS	IX	S	X
IS				
IX				
S				
X				



Knowledge Gate Website

Multiple Granularity

	IS	IX	S	X
IS	true	true	true	false
IX	true	true	false	false
S	true	false	true	false
X	false	false	false	false



Knowledge Gate Website

Validation-Based Protocols

- In cases where a majority of transactions are read-only transactions, the rate of conflicts among transactions may be low.
- Thus, many of these transactions, if executed without the supervision of a concurrency-control scheme, would nevertheless leave the system in a consistent state.
- A concurrency-control scheme imposes overhead of code execution and possible delay of transactions. It may be better to use an alternative scheme that imposes less overhead.
- A difficulty in reducing the overhead is that we do not know in advance which transactions will be involved in a conflict. To gain that knowledge, we need a scheme for *monitoring* the system.
- The **validation protocol** requires that each transaction T_i executes in two or three different phases in its lifetime, depending on whether it is a read-only or an update transaction. The phases are, in order:

Knowledge Gate Website

- **Read phase**. During this phase, the system executes transaction T_i. It reads the values of the various data items and stores them in variables local to T_i. It performs all write operations on temporary local variables, without updates of the actual database.

- **Validation phase**. The validation test (described below) is applied to trans- action T_i. This determines whether T_i is allowed to proceed to the write phase without causing a violation of serializability. If a transaction fails the validation test, the system aborts the transaction.

- **Write phase**. If the validation test succeeds for transaction T_i, the temporary local variables that hold the results of any write operations performed by T_i are copied to the database. Read-only transactions omit this phase.

Knowledge Gate Website

Module

Free Semester One Shot Exam Course

Topic

Digital Electronics

Subtopic

DE One Shot

Lesson

Digital Electronics_Notes

SEMESTER EXAM SERIES

DIGITAL ELECTRONICS

IN 6 HOURS + NOTES

KNOWLEDGE GATE

Video chapters

Chapter-1 (Boolean Algebra & Logic Gates): Introduction to Digital Electronics, Advantage of Digital System, Boolean Algebra, Laws, Not, OR, AND, NOR, NAND, EX-OR, EX-NOR, AND-OR, OR-AND, Universal Gate Functionally Complete Function.

- **Chapter-2 (**Boolean Expressions**):** Boolean Expressions, SOP(Sum of Product), SOP Canonical Form, POS(Product of Sum), POS Canonical Form, No of Functions Possible, Complementation, Duality, Simplification of Boolean Expression, K-map, Quine Mc-CluskyMethod.

- **Chapter-3 (Combinational Circuits):**Basics, Design Procedure, Half Adder, Half subtractor, Full Adder, Full Subtractor, Four-bit parallel binary adder / Ripple adder, Look ahead carry adder, Four-bit ripple adder/subtractor, Multiplexer, Demultiplexer, Decoder, Encoder, Priority Encoder

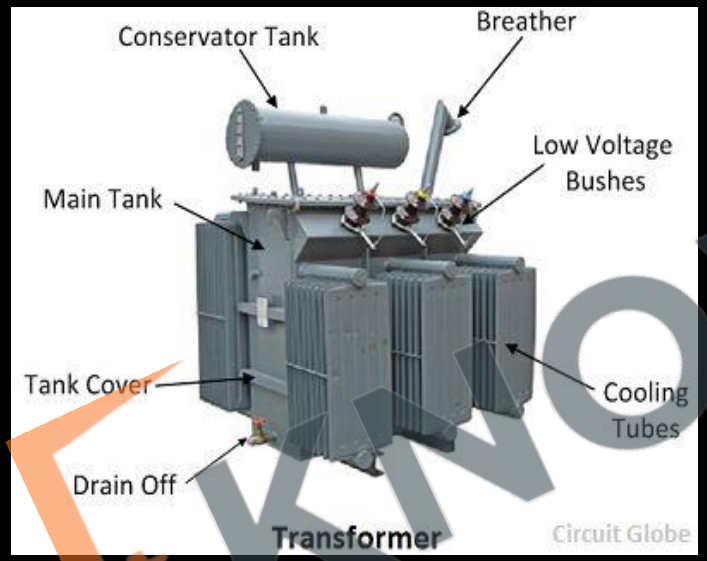
- **Chapter-4 (Sequential Circuits):**Basics,NOR Latch, NAND Latch, SR flip flop, JK flip flop, T(Toggle) flip flop, D flip flop, Flip Flops Conversion, Basics of counters, Finding Counting Sequence Synchronous Counters, Designing Synchronous Counters, Asynchronous/Ripple Counter, Registers, Serial In-Serial Out (SISO), Serial-In Parallel-Out shift Register (SIPO), Parallel-In Serial-Out Shift Register (PISO), Parallel-In Parallel-Out Shift Register (PIPO), Ring Counter, Johnson Counter

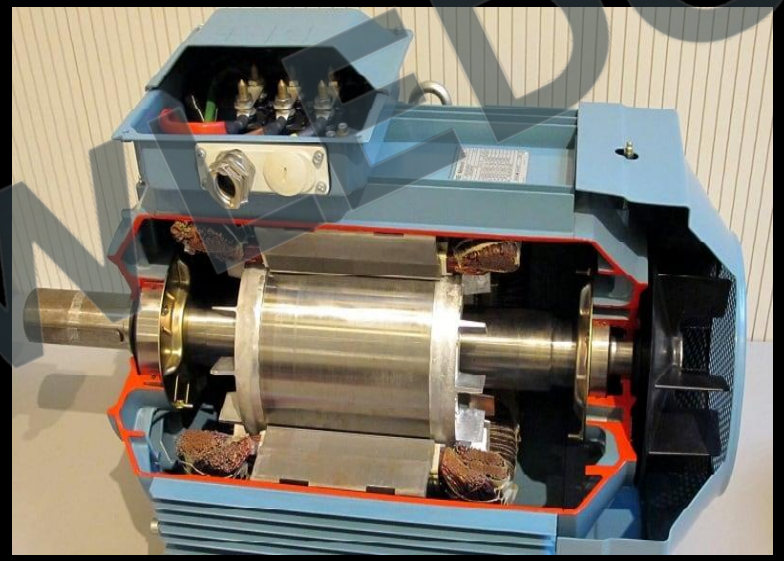
- **Chapter-5 (Number Sysem& Representations):** Basics, Conversion, Signed number Representation, Signed Magnitude, 1's Complement, 2's Complement, Gray Code, Binary-Coded Decimal Code (BCD), Excess-3 Code.

KNOWLEDGEGATE
www.knowledgegate.in

Basics

- **Electrical engineering** is a professional engineering discipline that generally deals with the study and application of electricity, electronics and electromagnetism and heavy voltage devices like transformers, motors etc.

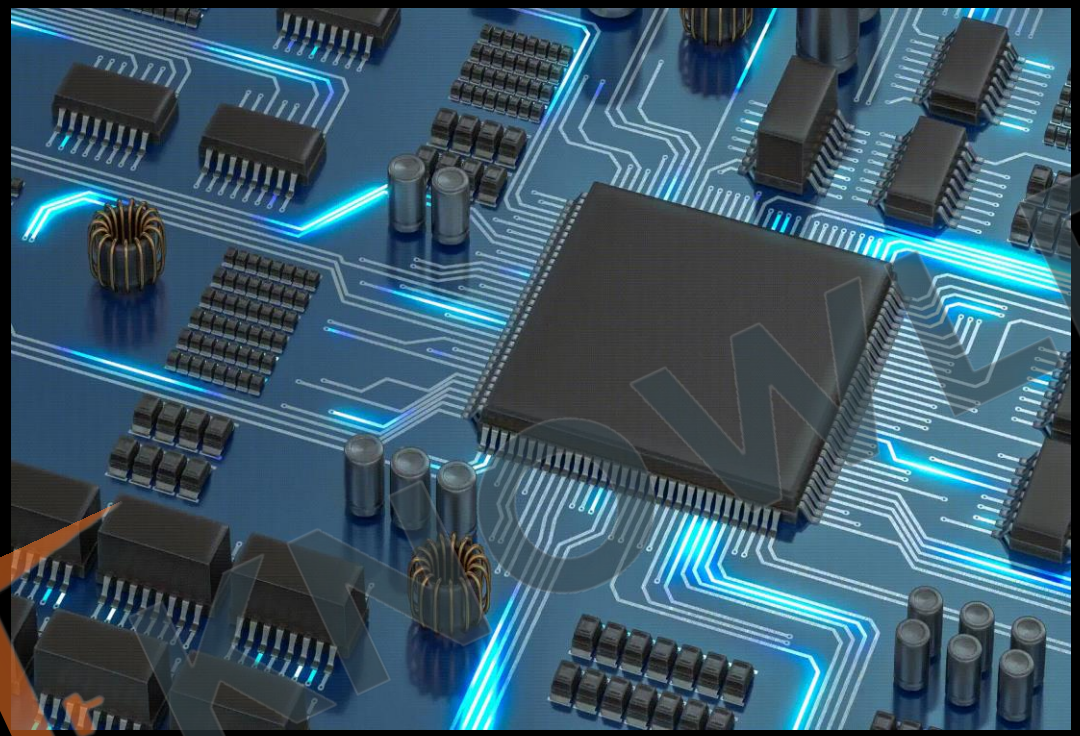




KNOWLEDGEGATE

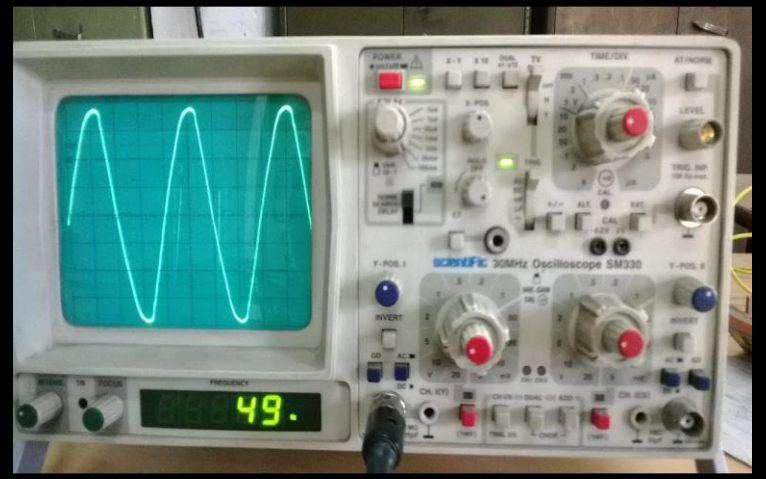
www.knowledgegate.in

- **Electronic engineering** is an electrical engineering discipline where we work on low voltage devices (such as semiconductor devices, especially transistor, diodes and integrated circuits) to design electronic circuits, VLSI devices and systems.



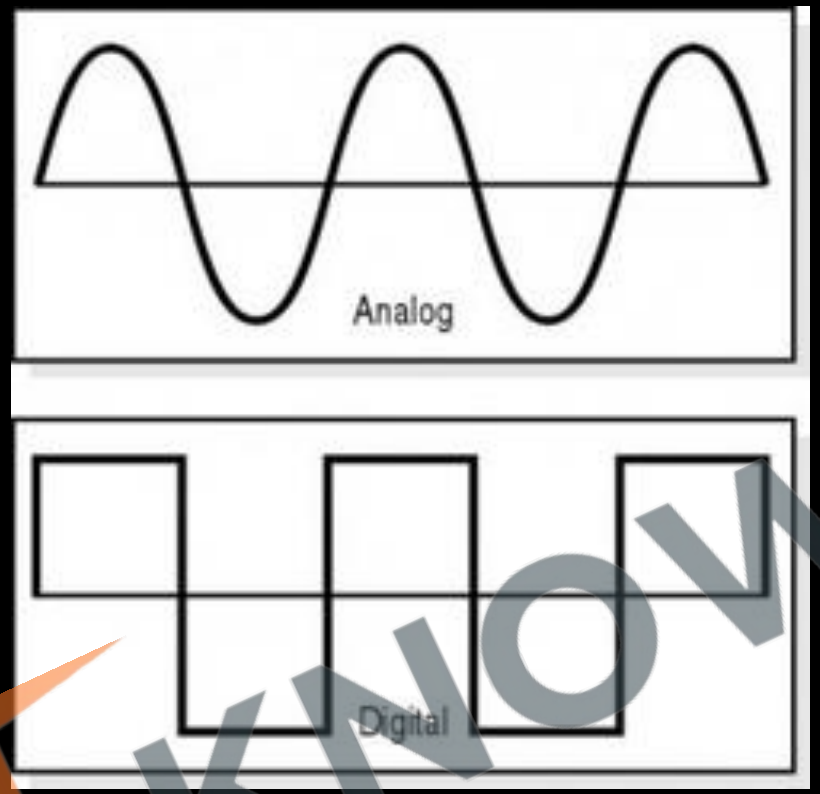
KNOWLEDGEGATE

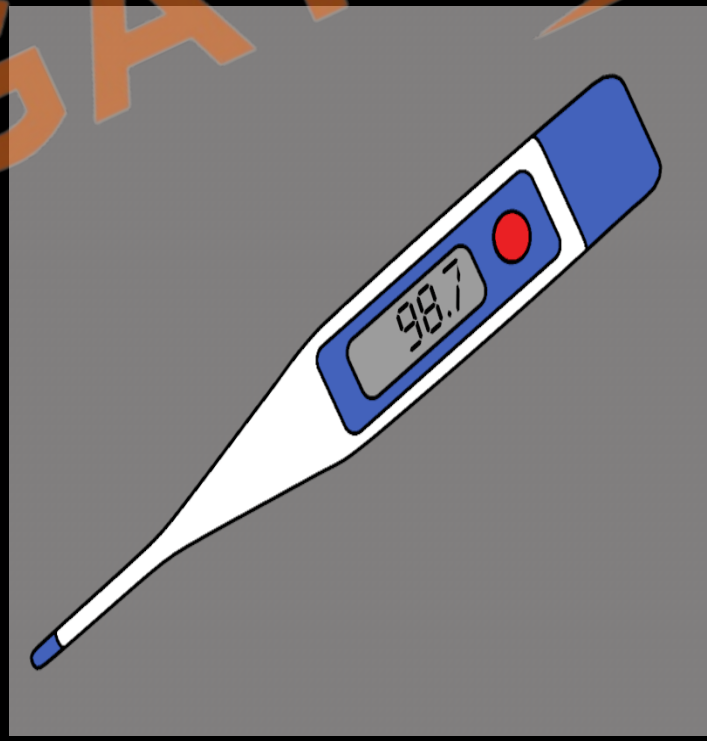
- Electronic systems are generally of two types –
  - **Analog system** in an analog representation, a continuous value is used to denote the information. Analog signal is defined as any physical quantity which varies continuously with respect to time. e.g. amplifier, cro, ecg. Watch, radio.



KNOWLEDGEGATE

- **Digital system** – the information is denoted by a finite sequence of discrete value or digits. Digital signal is defined as any physical quantity having discrete values. E.g. digital watch, calculator, bp machine, thermometer etc.





KNOWLEDGEGATE

Advantage of Digital system

- **Easy to Design -** Digital systems are simpler to design because you mostly need knowledge of switching circuits. You don't need to be an expert in engineering or math.

- **Long-Term Storage and Easy Processing -** You can store information for a very long time, and it's easier to process data in digital systems.

- **Higher Accuracy and Precision -** Digital devices give more accurate and precise results. They are less affected by things like noise or electric fields.

- **Better Modularity and Fabrication -** Digital systems are modular, meaning they are built in parts that can be easily replaced or upgraded. They are also smaller in size, thanks to things like integrated circuits.

- **Cost-Effective -** Generally, digital systems are less expensive to build and maintain.

KNOWLEDGEGATE

Boolean Algebra Laws

Idempotent Law

$a \cdot a = a$

$a + a = a$

Associative law

$a.(b.c) = (a.b).c$

$a + (b + c) = (a + b) + c$

Commutative law

$a.b = b.a$

$a + b = b + a$

Distributive law

$a.(b + c) = a.b + a.c$

$a + (b.c) = (a + b).(a + c)$

KNOWLEDGEGATE

www.knowledgegate.in

De-Morgan law

$(a + b)' = a' \cdot b'$

$(a. b)' = a' + b'$

Identity law

$a + 0 = a$

$a. 0 = 0$

$a + 1 = 1$

$a. 1 = a$

Complementation law

$0' = 1$

$1' = 0$

$a. a' = 0$

$a + a' = 1$

Involution law

$(a')' = a$

KNOWLEDGEGATE

www.knowledgegate.in

Absorption law

- a + ab = a
- a.(a+b) =a

Compensation theorem

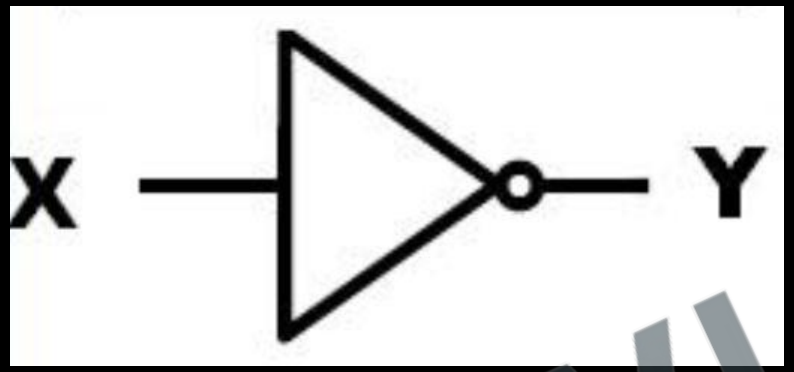
- ab + a’c + bc = ab + a’c
- (a+b) (a’ + c) (b + c) = (a+b) (a’ + c)

- a + a’b = a + b
- a.(a’+b) = a. b

KNOWLEDGEGATE

Not Gate(Inverter)

- It represents not logical operator is also known as inverter, it is a unary operator, which simply complement the input.

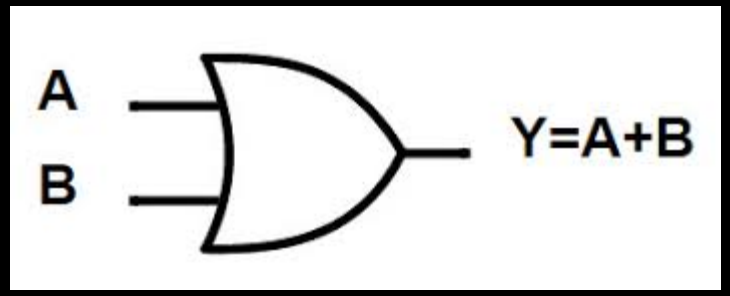


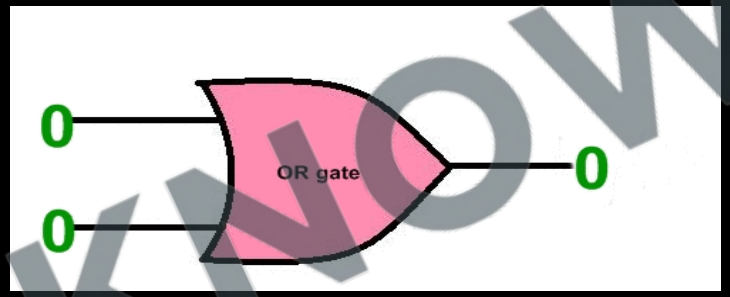
Truth Table	
Input	Output
X	Y = X’
0	
1	

KNOWLEDGE GATE

OR Gate

- It is a digital logic gate, that implements logical disjunction. The output will be high if at least one of the input lines is high.



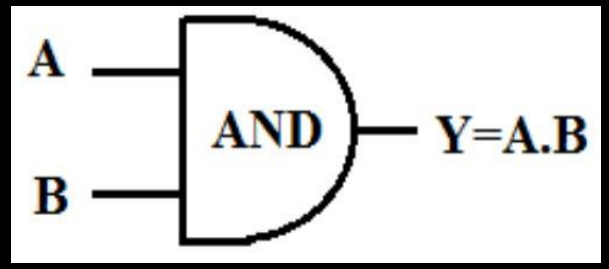


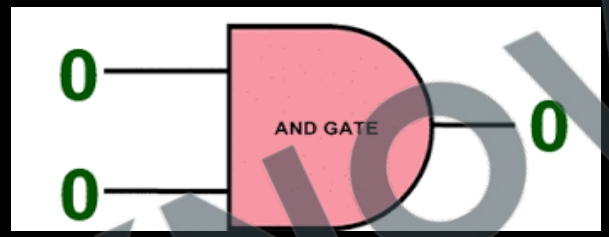
Truth Table		
Input		Output
A	B	Y = A + B
0	0	
0	1	
1	0	
1	1	

KNOWLEDGE GATE

And Gate

- It is a digital logic gate, that implements logical conjunction. Output will be high if and only if all input are high otherwise low.



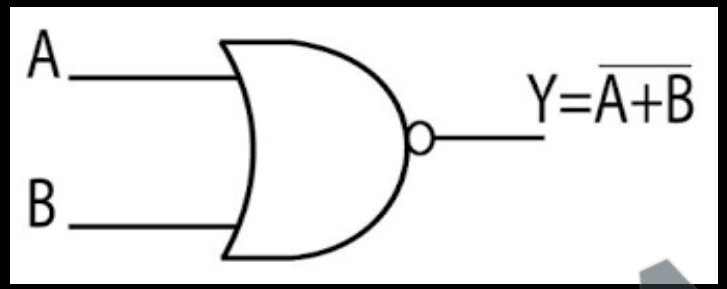


Truth Table		
Input		Output
A	B	Y = A . B
0	0	
0	1	
1	0	
1	1	

KNOWLEDGEGATE

Nor gate

- The output will be high if and only if all inputs are low. Or simply a OR gate followed by an invertor.
- NOR gate is also called universal gate because it can be used to implement any other logic gate.

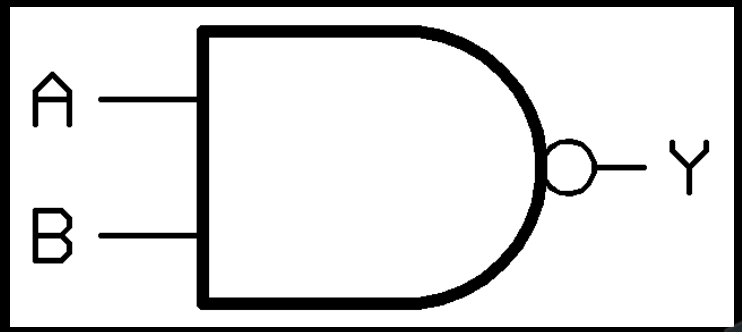


Truth Table		
Input		Output
A	B	Y = (A + B)’
0	0	
0	1	
1	0	
1	1	

KNOWLEDGE GATE

NAND Gate

- The output will be low if and only if all inputs are high. Or simply an and gate followed by an invertor
- NAND gate is also called universal gate.

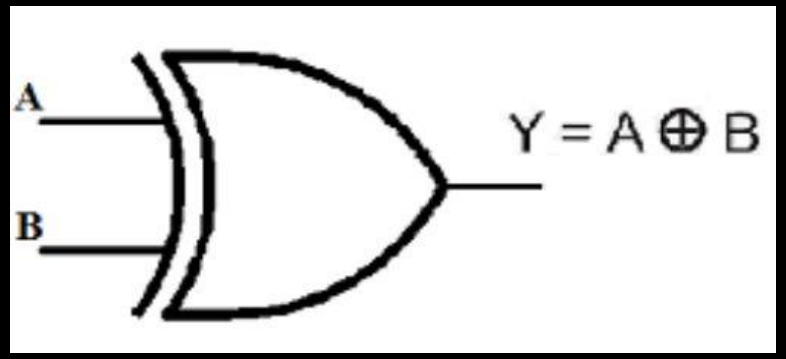


Truth Table		
Input		Output
A	B	Y = (A . B)’
0	0	
0	1	
1	0	
1	1	

KNOWLEDGE GATE

EX-OR

- For two inputs, output will be high if and only if both the input values are different.
- $a \oplus b = a'.b + a.b'$

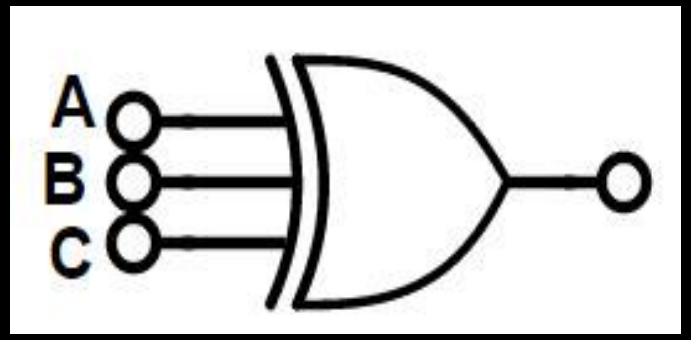


Truth Table		
Input		Output
A	B	Y = A ⊕ B
0	0	
0	1	
1	0	
1	1	

KNOWLEDGEGATE

EX-OR

- The XOR gate is a digital logic gate that gives High as output when the number of inputs High are odd.

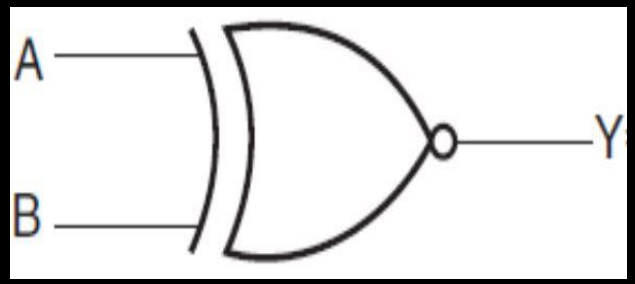


A	B	C	a ⊕ b ⊕ c
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

KNOWLEDGEGATE

EX-NOR

- For two input, output will be high if and only if both the input values are same
- a ⊙ b = a'. b' + a. b

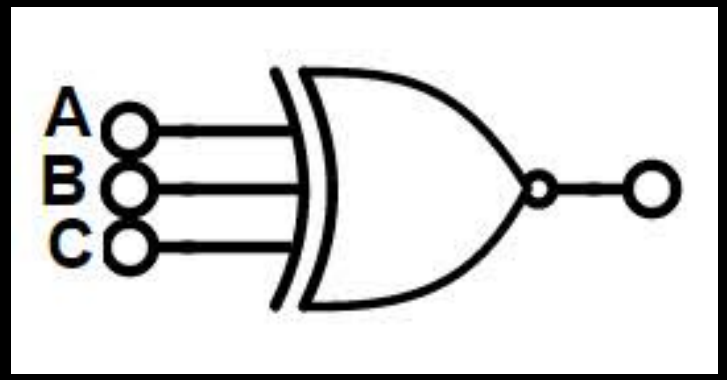


Truth Table		
Input		Output
A	B	Y = A ⊙ B
0	0	
0	1	
1	0	
1	1	

KNOWLEDGE GATE

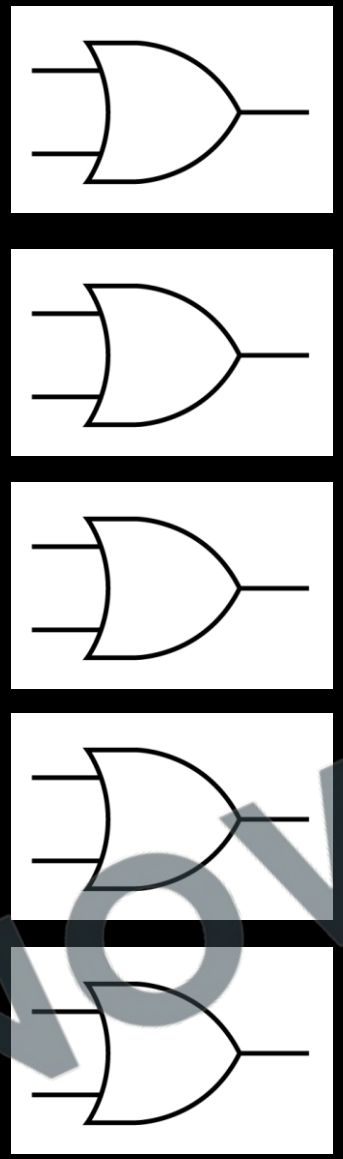
EX-NOR

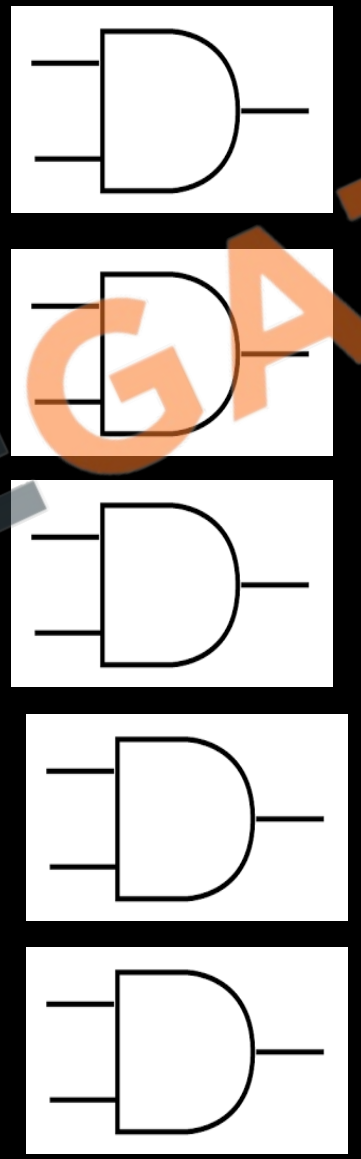
- The EX-NOR gate is a digital logic gate that gives output High when the number inputs low are even.



A	B	C	a ⊙ b ⊙ c
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

KNOWLEDGEGATE





KNOWLEDGEGATE

www.knowledgegate.in

AND-OR(NAND-NAND) Implementation

KNOWLEDGEGATE

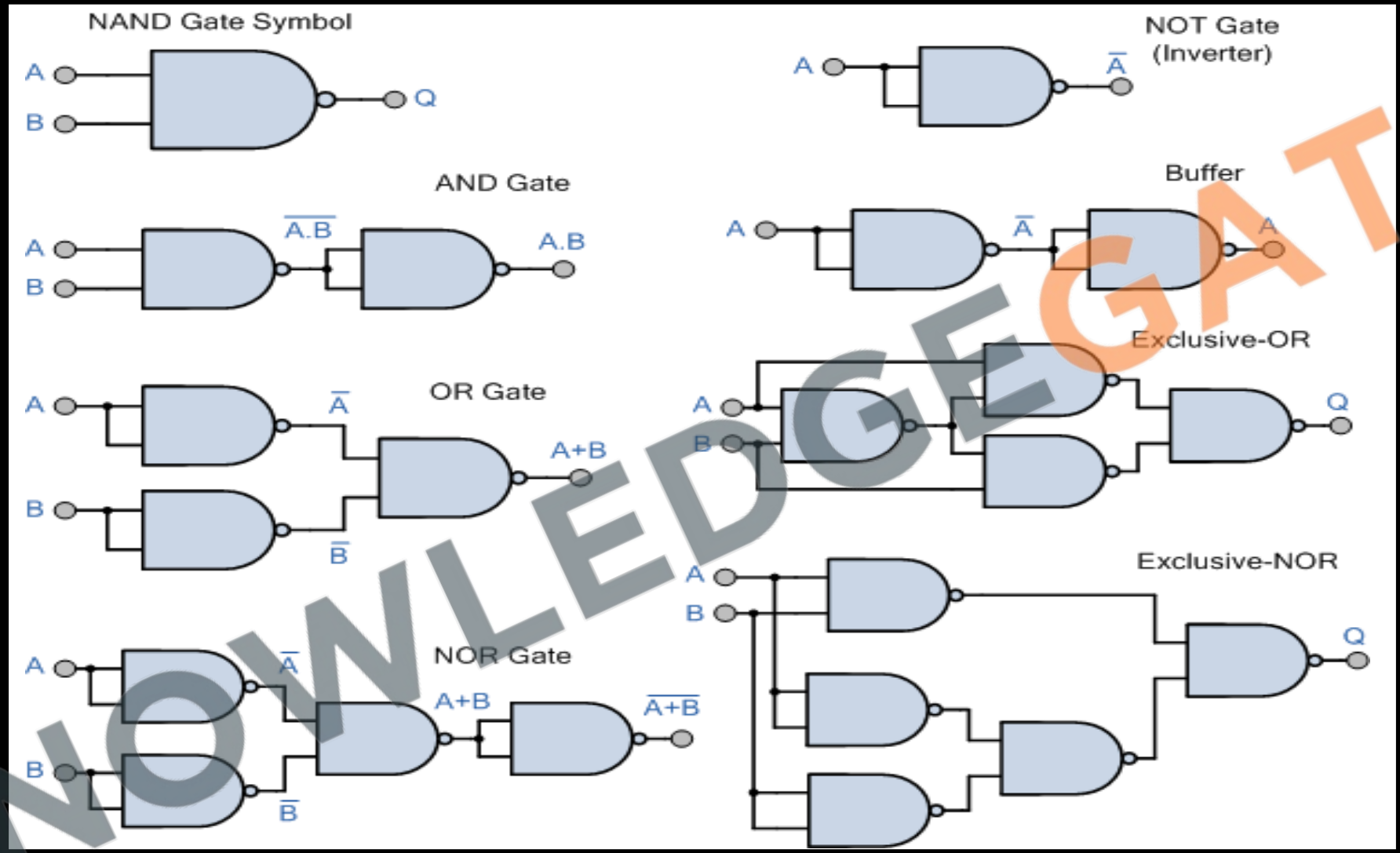
www.knowledgegate.in

OR-AND(NOR-NOR) Implementation

KNOWLEDGEGATE

www.knowledgegate.in

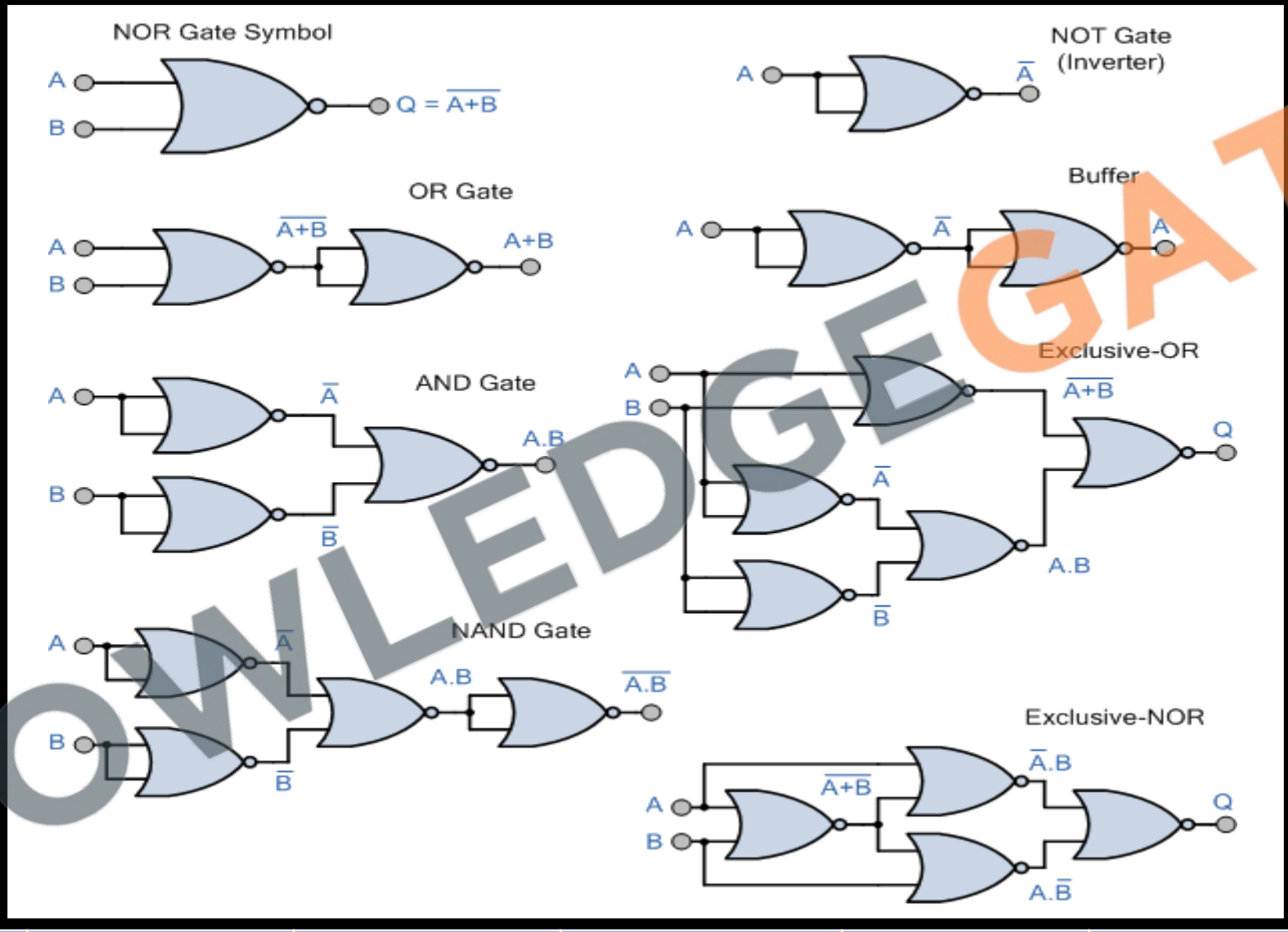
Implementing Every Gate with NAND Gate



KNOWLEDGEGATE

	NOT	AND	OR	NAND	NOR	EX-OR	EX-NOR
NAND	1	2	3	1	4	4	5

Implementing Every Gate with NOR Gate



KNOWLEDGEGATE

	NOT	AND	OR	NAND	NOR	EX-OR	EX-NOR
NOR	1	3	2	4	1	5	4

Functionally Complete Function

- As we know there are only three fundamental Boolean operator NOT, AND and OR. All the other operators are derived from these operators
- We understand NOT along with OR(NOR) and NOT along with AND(NAND) are used as universal gates.
- Any Boolean operation or function can be implemented with NAND or NOR operation.

KNOWLEDGEGATE

Implementing any function with universal gates

$F = x'y + xy' + z$

KNOWLEDGEGATE

Implementing any function with universal gates

F = bd + bc + acd’

KNOWLEDGEGATE

Implementing any function with universal gates

$F = (x' + y)\,(x + y')(z)$

KNOWLEDGEGATE

Boolean Expressions

- Boolean expressions are the method using which we save the information about the Boolean function, that when we get value 1 and when we get value 0 as output.
- So, we convert the truth table of the function into an expression, reading which we can understand when the output is 1 and when it is zero.

Light	Day	Engine	Warning
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

KNOWLEDGEGATE

www.knowledgegate.in

- There are two popular approaches for writing these expressions.
  - Sum of Product (SOP) (which remember when we get 1)
  - Product of Sum (POS) (which remember when we get 0)
- Make a note of this as we are studying Boolean function remembering both 0 and 1 is not required, so we can either concentrate on 0 or 1.

Light	Day	Engine	Warning
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

- $W(L, D, E) = \sum_m (1, 4, 6, 7)$
- $W(L, D, E) = \prod_M (0, 2, 3, 5)$

KNOWLEDGEGATE

- Power of SOP and POS bother are same, i.e. any Boolean functions can be represented using both SOP and POS.

- Mathematically speaking we should choose (0 or 1), whatever is less in number because then it will be easy to represent

A	B	f_1 =	f_2 =
0	0	0	0
0	1	1	0
1	0	1	0
1	1	1	1

KNOWLEDGE GATE

SOP (Sum of Product)

- A sum of product form expression contains product terms (AND terms) which are sum (OR) together, that's why called sum of product.
- Each product terms (AND terms) consists of one or more literals (variables) appearing either in complements or uncomplemented form. E.g. a'b + b'c' + abc
- A product term which contains all the literals (variables) either in complemented or uncomplemented form is called minterm.
- In a n variable function, there will be 2^n minterms.

Binary Representation	Sequence	Minterm	Designation
000	0	a'b'c'	m_0
001	1	a'b'c	m_1
010	2	a'bc'	m_2
011	3	a'bc	m_3
100	4	ab'c'	m_4
101	5	ab'c	m_5
110	6	abc'	m_6
111	7	abc	m_7

KNOWLEDGEGATE
www.knowledgegate.in

- The result of a product term must always be 1, If a literal is having value 1 then it is ok, but if not, then we complement those which is 0, it to make it 1.
- There is only 1 input sequence of variables for any minterm on which the output is 1, so it represents information.
- Then the sum of all product term(minterm) to from a function, and functions will have value 1, if at least one of the product term(minterm) is 1.

Binary Representation	Sequence	Minterm	Designation
000	0	a'b'c'	m_0
001	1	a'b'c	m_1
010	2	a'bc'	m_2
011	3	a'bc	m_3
100	4	ab'c'	m_4
101	5	ab'c	m_5
110	6	abc'	m_6
111	7	Abc	m_7

KNOWLEDGEGATE
www.knowledgegate.in

Canonical logic forms

- Either in POS or SOP form it is not essential that all product or sum terms contains all the literals.
- **Canonical SOP form**: - In a sum of product form expression, if each AND term (product term) consists all the literals(variables) appearing either in complements or uncomplemented form. E.g. a'bc + ab'c' + abc. Then the form is said to be canonical SOP.

- F (a, b, c) = a + a'b + abc + bc'

KNOWLEDGEGATE

POS (Product of Sum)

- A product of sum (POS) form of expression contains OR (sum) terms which are AND (product) together, that's why called product of sum expression.
- Each OR term (sum term) consists of one or more literals(variables) appearing either in complemented or uncomplemented form. (a' + b). (b' + c'). (a + c)
- A OR (sum) term which contains all the literals(variables) either in complemented or uncomplemented form is called maxterm. In a n variable function, there will be 2^n maxterms.

Binary Representation	Sequence	Maxterm	Designation
000	0	a + b + c	M_0
001	1	a + b + c'	M_1
010	2	a + b' + c	M_2
011	3	a + b' + c'	M_3
100	4	a' + b + c	M_4
101	5	a' + b + c'	M_5
110	6	a' + b' + c	M_6
111	7	a' + b' + c'	M_7

KNOWLEDGEGATE

- The result of the OR (sum) term must be 0, so If a variable is having value 0 then it is ok, but if not then we complement the variable it to make it 0.
- There is only 1 input sequence for which the output of a Maxterm is 0. Because, it requires all values as zero.
- Then the product of all sum term (maxterm), from a function, and functions will have a value 0, if any of the sum term(maxterm) is 0.

Binary Representation	Sequence	Maxterm	Designation
000	0	a + b + c	M_0
001	1	a + b + c’	M_1
010	2	a + b’ + c	M_2
011	3	a + b’ + c’	M_3
100	4	a’ + b + c	M_4
101	5	a’ + b + c’	M_5
110	6	a’ + b’ + c	M_6
111	7	a’ + b’ + c’	M_7

KNOWLEDGEGATE

Canonical logic forms

- **Canonical POS form**: - In a product of sum form expression, if each OR term (sum term) consists all the literals(variables) appearing either in complements or uncomplemented form. E.g. (a’ + b + c). (a + b’ + c’). (a + b + c). Then the form is said to be Canonical POS form.

KNOWLEDGEGATE

No of functions possible

Q With n-Boolean variables how many different Boolean functions are possible?

With n binary variable we can generate 2^n combinations. And as we know that a Boolean function can also have only 2 possible values either 0 or 1, so total number of different functions possible will be $2^{(2^n)}$.

Similarly, we can generalize this idea as $x^{(y^z)}$ are the total number of functions possible, x is the nature of the function, y is the nature of the variable and z is the number of variables.

A	B	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

KNOWLEDGEGATE

Complementation

- Let us consider a function f(a, b, c, d,……, 0, 1, +, ·), then the complement of the function is defined as f’(a’, b’, c’,……, z’, 1, 0, ., +).

- i.e. When all the variables are replaced by their compliments, 0→1,1→0, or →and, and → or, then we will be called them compliment of a function.

KNOWLEDGEGATE

Duality

- Let us consider a function f(a, b, c, d,……,z,0,1,+,·), then the dual of the function is defined as f^d(a, b, c,……,z,1,0,·.,+).

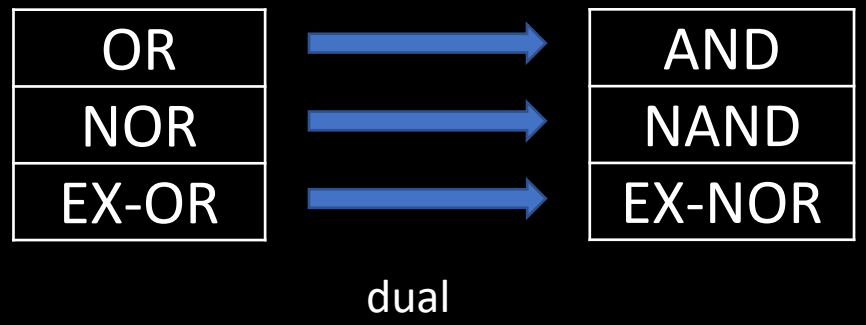
- When the nature of variable remains same but 0 → 1, 1 → 0, or → and, and → or, then they are called dual functions.

a	b		
L 0 H	L 0 H	L 0 H	L 0 H
L 0 H	H 1 L	H 1 L	L 0 H
H 1 L	L 0 H	H 1 L	L 0 H
H 1 L	H 1 L	H 1 L	H 1 L

KNOWLEDGEGATE

- When we take dual of a function, then the functionality of a function remains the same but a positive logic system is transformed to negative logic system.

- If a function works corrects in positive logic system, then it must also work correct in negative logic system as it do not depends on magnitude.

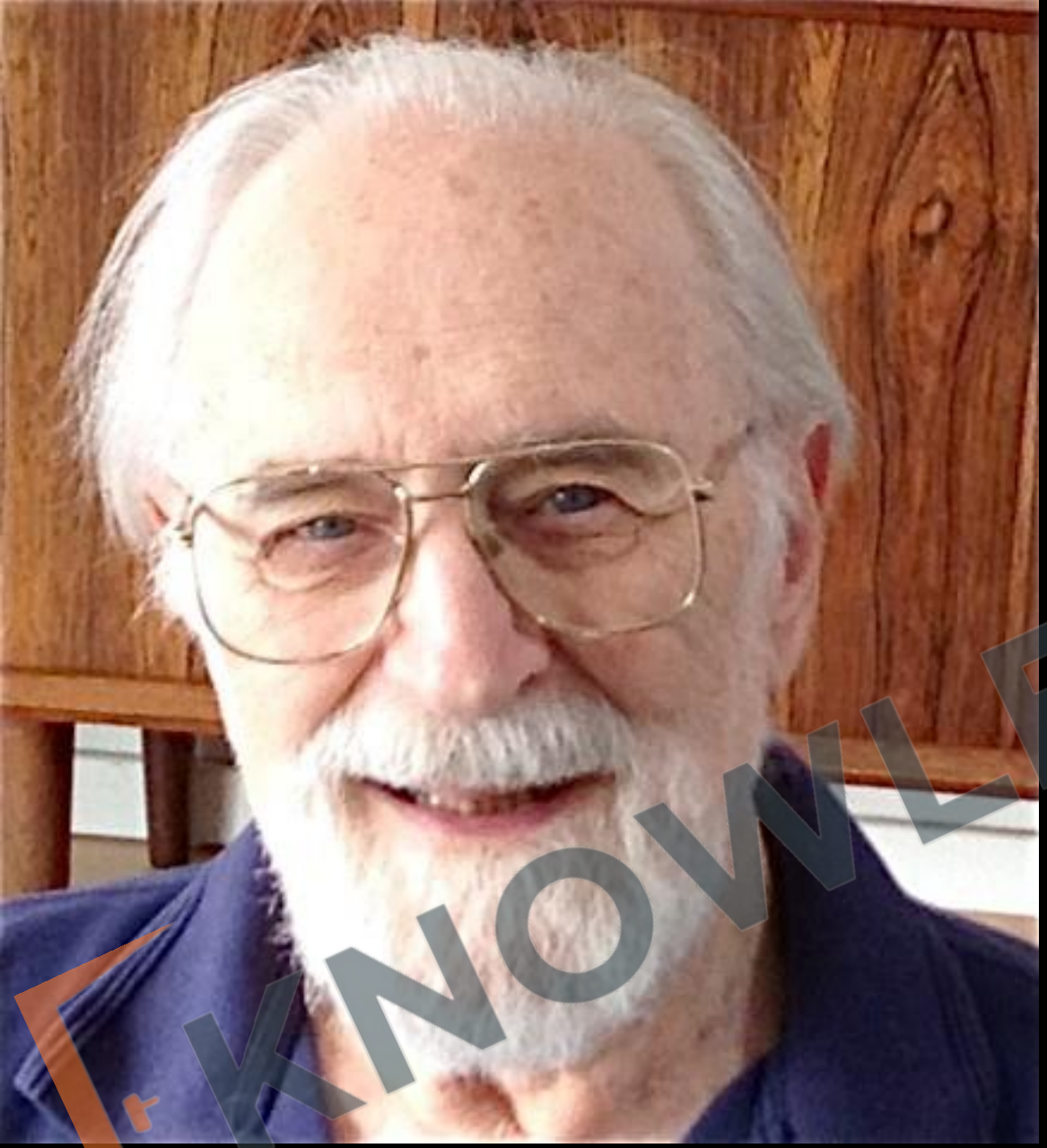


KNOWLEDGEGATE

Simplification of Boolean expression

- After deriving a Boolean expression from the truth table next important step is to minimise it, so that the cost of implementation into hardware and delay because of additional gates can be reduced. There are following methods for simplifying a Boolean expression
- Using Karnaugh-map
- Using Quine Mc-Clusky Method(Tabular Method)
- Using Algebraic method (Boolean Laws)

KNOWLEDGEGATE



- Maurice Karnaugh (4 October 1924 – 8 Nov 2022) is an American physicist, mathematician and inventor known for the Karnaugh map used in Boolean algebra.

KNOWLEDGEGATE

- Karnaugh map is one of the most extensively used tool, it is a graphical representation, represents truth table by pictorial form, provides a systematic method for simplifying or minimizing a Boolean expression. For a n-variable k-map, there will be 2^n cells addressed by a gray code. Each cell corresponds to one minterm or maxterm.

	a	b	c	d	F
0	0	0	0	0	
1	0	0	0	1	
2	0	0	1	0	
3	0	0	1	1	
4	0	1	0	0	
5	0	1	0	1	
6	0	1	1	0	
7	0	1	1	1	
8	1	0	0	0	
9	1	0	0	1	
10	1	0	1	0	
11	1	0	1	1	
12	1	1	0	0	
13	1	1	0	1	
14	1	1	1	0	
15	1	1	1	1	

	ab				
cd					

KNOWLEDGEGATE

	ab	a’b’	a’b	ab	ab’
cd		00	01	11	10
c’d’	00	0	4	12	8
c’d	01	1	5	13	9
cd	11	3	7	15	11
cd’	10	2	6	14	10

	cd	c’d’	c’d	cd	cd’
ab		00	01	11	10
a’b’	00	0	1	3	2
a’b	01	4	5	7	6
ab	11	12	13	15	14
ab’	10	8	9	11	10

KNOWLEDGEGATE

www.knowledgegate.in

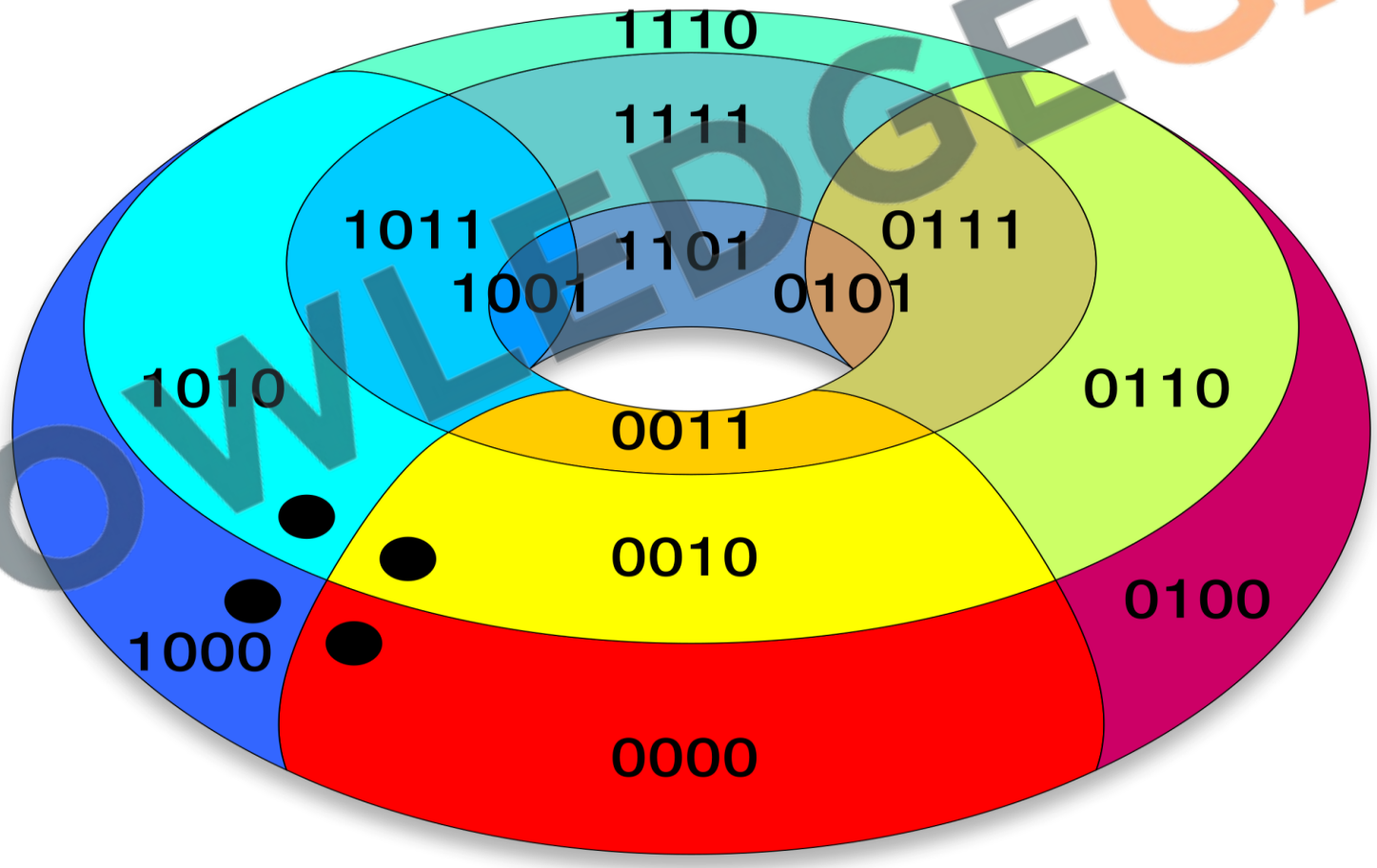
	ad	a’d’	a’d	ad	ad’
bc		00	01	11	10
b’c’	00	0	1	9	8
b’c	01	2	3	11	10
bc	11	6	7	15	14
bc’	10	4	5	13	12

	db	d’b’	d’b	db	db’
ca		00	01	11	10
c’a’	00	0	4	5	1
c’a	01	8	12	13	9
ca	11	10	14	15	11
ca’	10	2	6	7	3

KNOWLEDGEGATE

www.knowledgegate.in

0000	0100	1100	1000
0001	0101	1101	1001
0011	0111	1111	1011
0010	0110	1110	1010



KNOWLEDGEGATE

	ab	a’b’	a’b	ab	ab’
c		00	01	11	10
c’	0	0	2	6	4
c	1	1	3	7	5

	bc	b’c’	b’c	bc	bc’
a		00	01	11	10
a’	0	0	1	3	2
a	1	4	5	7	6

	a	a’	a
bc		0	1
b’c’	00	0	4
b’c	01	1	5
bc	11	3	7
bc’	10	2	6

KNOWLEDGEGATE

www.knowledgegate.in

	a	a’	a
b		0	1
b’	0	0	2
b	1	1	3

	b	b’	b
a		0	1
a’	0	0	1
a	1	2	3

KNOWLEDGEGATE

www.knowledgegate.in

5-Variable K-Map

a'

	bc	b'c'	b'c	bc	bc'
de		00	01	11	10
d'e'	00	0	4	12	8
d'e	01	1	5	13	9
de	11	3	7	15	11
de'	10	2	6	14	10

a

	bc	b'c'	b'c	bc	bc'
de		00	01	11	10
d'e'	00	16	20	28	24
d'e	01	17	21	29	25
de	11	19	23	31	27
de'	10	18	22	30	26

KNOWLEDGEGATE

6-Variable K-Map

a'b'

	cd	c'd'	c'd	cd	cd'
ef		00	01	11	10
e'f'	00	0	4	12	8
e'f	01	1	5	13	9
ef	11	3	7	15	11
ef'	10	2	6	14	10

a'b

	cd	c'd'	c'd	cd	cd'
ef		00	01	11	10
e'f'	00	16	20	28	24
e'f	01	17	21	29	25
ef	11	19	23	31	27
ef'	10	18	22	30	26

ab'

	cd	c'd'	c'd	cd	cd'
ef		00	01	11	10
e'f'	00	32	36	44	40
e'f	01	33	37	45	41
ef	11	35	39	47	43
ef'	10	34	38	46	42

ab

	cd	c'd'	c'd	cd	cd'
ef		00	01	11	10
e'f'	00	48	52	60	56
e'f	01	49	53	61	57
ef	11	51	55	63	59
ef'	10	50	54	62	58

KNOWLEDGEGATE
www.knowledgegate.in

7-Variable K-Map

a'b'c'

	cd	c'd'	c'd	cd	cd'
ef		00	01	11	10
e'f'	00				
e'f	01		0-15		
ef	11				
ef'	10				

a'b'c

	cd	c'd'	c'd	cd	cd'
ef		00	01	11	10
e'f'	00				
e'f	01		16-31		
ef	11				
ef'	10				

a'bc'

	cd	c'd'	c'd	cd	cd'
ef		00	01	11	10
e'f'	00				
e'f	01		32-47		
ef	11				
ef'	10				

a'bc

	cd	c'd'	c'd	cd	cd'
ef		00	01	11	10
e'f'	00				
e'f	01		48-63		
ef	11				
ef'	10				

ab'c'

	cd	c'd'	c'd	cd	cd'
ef		00	01	11	10
e'f'	00				
e'f	01		64-79		
ef	11				
ef'	10				

ab'c

	cd	c'd'	c'd	cd	cd'
ef		00	01	11	10
e'f'	00				
e'f	01		80-95		
ef	11				
ef'	10				

abc'

	cd	c'd'	c'd	cd	cd'
ef		00	01	11	10
e'f'	00				
e'f	01		96-111		
ef	11				
ef'	10				

abc

	cd	c'd'	c'd	cd	cd'
ef		00	01	11	10
e'f'	00				
e'f	01		112-127		
ef	11				
ef'	10				

KNOWLEDGEGATE

a'b’c’d’

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		0-15		
ef	11				
ef’	10				

a’b’c’d

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		16-31		
ef	11				
ef’	10				

a’b’cd’

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		32-47		
ef	11				
ef’	10				

a’b’cd

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		48-63		
ef	11				
ef’	10				

a’bc’d’

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		64-79		
ef	11				
ef’	10				

a’bc’d

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		80-95		
ef	11				
ef’	10				

a’bcd’

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		96-111		
ef	11				
ef’	10				

a’bcd

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		112-127		
ef	11				
ef’	10				

ab’c’d’

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		128-143		
ef	11				
ef’	10				

ab’c’d

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		144-159		
ef	11				
ef’	10				

ab’cd’

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		160-175		
ef	11				
ef’	10				

ab’cd

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		176-191		
ef	11				
ef’	10				

abc’d’

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		192-207		
ef	11				
ef’	10				

abc’d

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		208-223		
ef	11				
ef’	10				

abcd’

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		224-239		
ef	11				
ef’	10				

abcd

	cd	c’d’	c’d	cd	cd’
ef		00	01	11	10
e’f’	00				
e’f	01		240-255		
ef	11				
ef’	10				

KNOWLEDGEGATE
www.knowledgegate.in

K-Map for POS

	a+b	a+b	a+b'	a'+b'	a'+b
c+d		0+0	0+1	1+1	1+0
c+d	0+0	0	4	12	8
c+d'	0+1	1	5	13	9
c'+d'	1+1	3	7	15	11
c'+d	1+0	2	6	14	10

KNOWLEDGEGATE

- **Minimal function**: - A function is said to be minimal if it is representing the function expression, if it is using minimal no of literals to represent the function.

KNOWLEDGEGATE

www.knowledgegate.in

- Rules of grouping: -

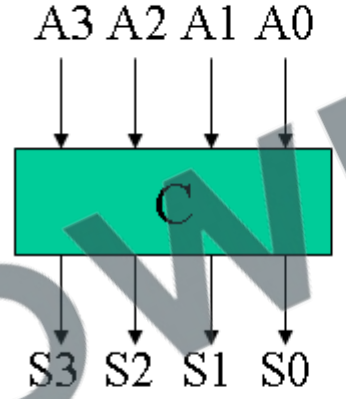
1. Every minterm must be covered
2. Group must have contiguous Cells(circular)
3. Group must be in horizontal or vertical fashion(circular)
4. Number of cells in a group must be in power of 2
5. Will Try to make largest group possible, so that number of literals in the expression can be reduced
6. Can also take don't care if it helps in creating the larger groups, other wise don't care.
7. Will consider new implicant if it is covering some new minterm.

	ab	a'b'	a'b	ab	ab'
cd		00	01	11	10
c'd'	00	0	4	12	8
c'd	01	1	5	13	9
cd	11	3	7	15	11
cd'	10	2	6	14	10

KNOWLEDGEGATE

Don’t care condition

- Don’t care cases are those cases which can never occur logically in that function.
- It is not essential to cover don’t care conditions, but if don’t care are helping to generate bigger prime implicants, then we can use them.



A3	A2	A1	A0	S3	S2	S1	S0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

KNOWLEDGEGATE

	ab	a’b’	a’b	ab	ab’
cd		00	01	11	10
c’d’	00			1	
c’d	01	D	D	1	D
cd	11		1	1	
cd’	10		1	1	

KNOWLEDGEGATE

www.knowledgegate.in

First try to cover those minterms which do not have an option.

	ab	a'b'	a'b	ab	ab'
cd		00	01	11	10
c'd'	00		1		
c'd	01		1	1	1
cd	11	1	1	1	
cd'	10			1	

KNOWLEDGEGATE

www.knowledgegate.in

For some functions more than one minimal Boolean expression are possible

	ab	a'b'	a'b	ab	ab'
cd		00	01	11	10
c'd'	00		D		D
c'd	01	1	1	D	
cd	11		D	1	1
cd'	10		1	D	

KNOWLEDGEGATE

www.knowledgegate.in

Q $f(a, b, c) = \sum_m \{1,2,3,4,5\}$

	ab	a'b'	a'b	ab	ab'
c		00	01	11	10
c'	0	0	2	6	4
c	1	1	3	7	5

KNOWLEDGEGATE

www.knowledgegate.in

	ab	a’b’	a’b	ab	ab’
cd		00	01	11	10
c’d’	00	1			1
c’d	01	1	1		
cd	11		1	1	
cd’	10			1	1

KNOWLEDGEGATE

www.knowledgegate.in

	ab	a’b’	a’b	ab	ab’
cd		00	01	11	10
c’d’	00	1	D	D	1
c’d	01	D			
cd	11				
cd’	10	1			D

KNOWLEDGEGATE

www.knowledgegate.in

	ab	a’b’	a’b	ab	ab’
cd		00	01	11	10
c’d’	00		1	1	
c’d	01	D			1
cd	11	D			1
cd’	10		1	1	D

KNOWLEDGEGATE

www.knowledgegate.in

	ab	a’b’	a’b	ab	ab’
cd		00	01	11	10
c’d’	00	D	1		1
c’d	01		1	D	
cd	11	1	D	D	
cd’	10	D			D

KNOWLEDGEGATE

www.knowledgegate.in

K-Map for POS

	a+b	a+b	a+b’	a’+b’	a’+b
c+d		0+0	0+1	1+1	1+0
c+d	0+0			D	1
c+d’	0+1		1	D	1
c’+d’	1+1		1	D	D
c’+d	1+0		1	D	D

KNOWLEDGEGATE

www.knowledgegate.in

Using Quine Mc-Clusky Method(Tabular Method)

Y(a, b, c, d) = $\sum_m$ (0, 1, 3, 7, 8, 9, 11, 15)

Group	Minterm	Binary Representation
0	0	0 0 0 0
1	1 8	0 0 0 1 1 0 0 0
2	3 9	0 0 1 1 1 0 0 1
3	7 11	0 1 1 1 1 0 1 1
4	15	1 1 1 1

Group	Minterm	Binary Representation
0	0,1 0,8	0 0 0 _ _ 0 0 0
1	1,3 1,9 8,9	0 0 _ 1 _ 0 0 1 1 0 0 _
2	3,7 3,11 9,11	0 _ 1 1 _ 0 1 1 1 0 _1
3	7,15 11,15	_ 1 1 1 1 _ 1 1

KNOWLEDGEGATE
www.knowledgegate.in

Group	Minterm	Binary Representation
0	0,1 0,8	0 0 0 _ _ 0 0 0
1	1,3 1,9 8,9	0 0 _ 1 _ 0 0 1 1 0 0 _
2	3,7 3,11 9,11	0 _ 1 1 _ 0 1 1 1 0 _ 1
3	7,15 11,15	_ 1 1 1 1 _ 1 1

Group	Minterm	Binary Representation
0	0,1,8,9 0,8,1,9	_ 0 0 _ _ 0 0 _
1	1,3,9,11 1,9,3,11	_ 0 _ 1 _ 0 _ 1
2	3,7,11,15 3,11,7,15	_ _ 1 1 _ _ 1 1

KNOWLEDGE GATE

Group	Minterm	Binary Representation	
0	0,1,8,9 0,8,1,9	_ 0 0 _ _ 0 0 _	b'c'
1	1,3,9,11 1,9,3,11	_ 0 _ 1 _ 0 _ 1	b'd
2	3,7,11,15 3,11,7,15	_ _ 1 1 _ _ 1 1	cd

Y(a, b, c, d) = b'c' + cd

PI	Minterm	0	1	3	7	8	9	11	15
b'c'	0,1,8,9	X	X			X	X		
b'd	1,3,9,11		X	X			X	X	
cd	3,7,11,15			X	X			X	X

KNOWLEDGEGATE

PI	Minterm	0	1	3	7	8	9	11	15
b'c'	0,1,8,9	X	X			X	X		
b'd	1,3,9,11		X	X			X	X	
cd	3,7,11,15			X	X			X	X

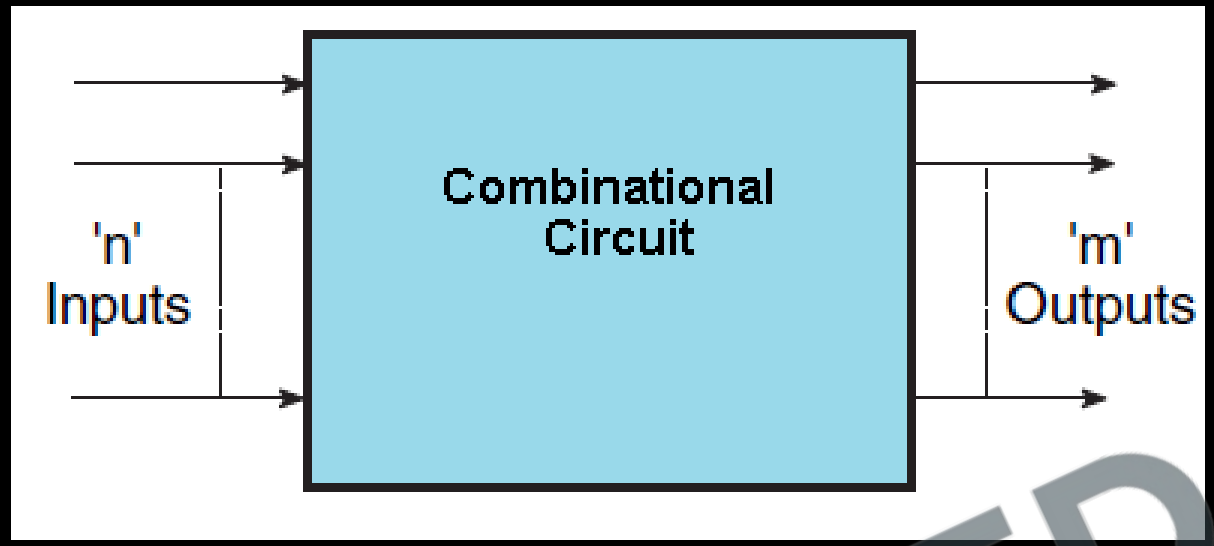
Y(a, b, c, d) = $\sum_m$ (0, 1, 3, 7, 8, 9, 11, 15)

Y(a, b, c, d) = b'c' + cd

	ab	a'b'	a'b	ab	ab'
cd		00	01	11	10
c'd'	00	1			1
c'd	01	1			1
cd	11	1	1	1	1
cd'	10				

KNOWLEDGEGATE

Combinational Circuits



O/p = f(i/p)

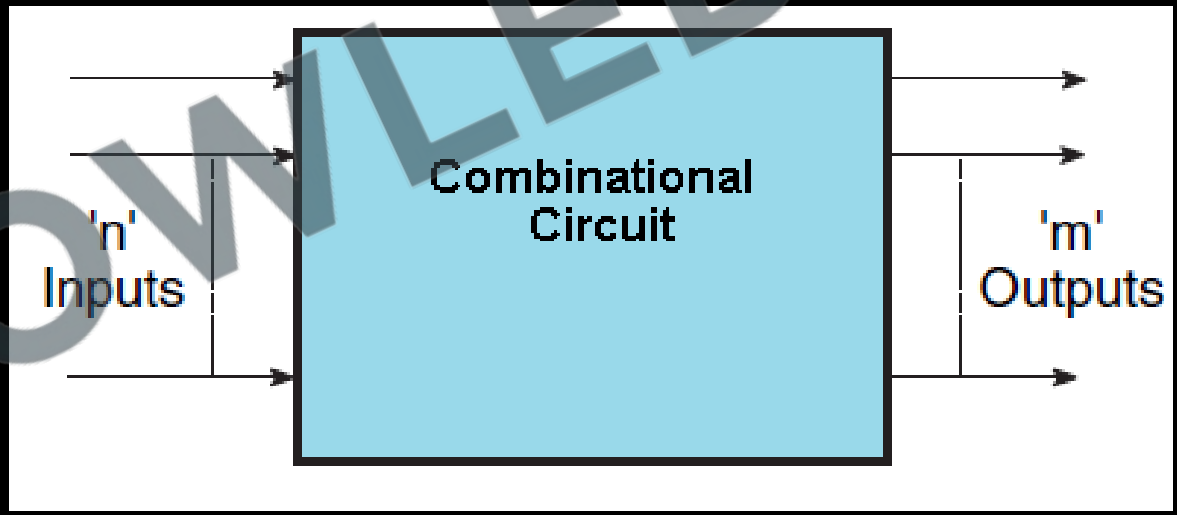
KNOWLEDGEGATE

Combinational Circuits

- When logic gates are connected together to produce a specified out put on certain specified combinations of input variables, with no memory involved, then the resulting circuit is called a combinational circuit.

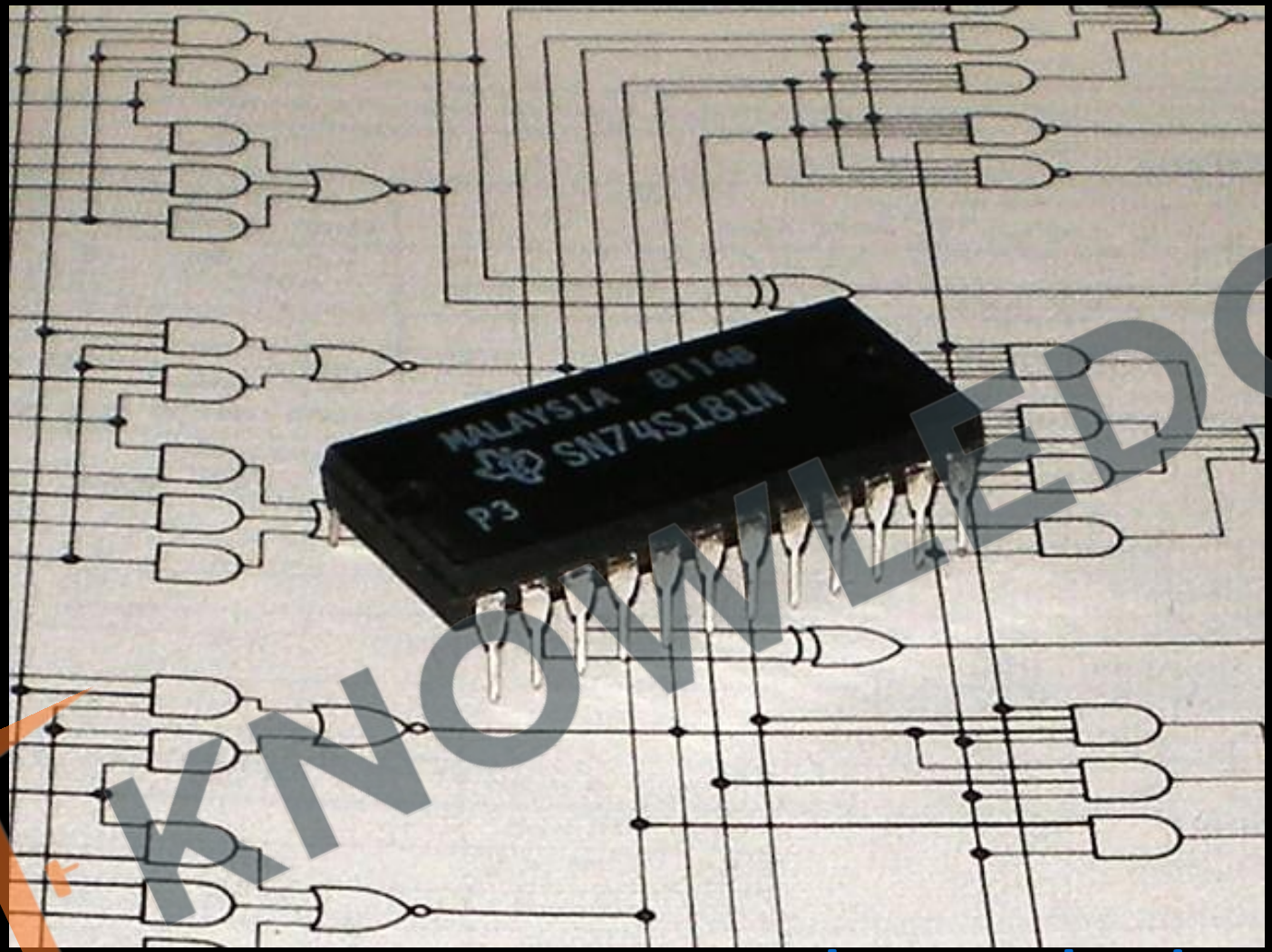
- Output depends only on present input. O/p = f(i/p), combinational circuit performs an operation that can be specified logically by a set of Boolean function.

- A combinational circuit may have n-binary inputs and m-binary outputs.



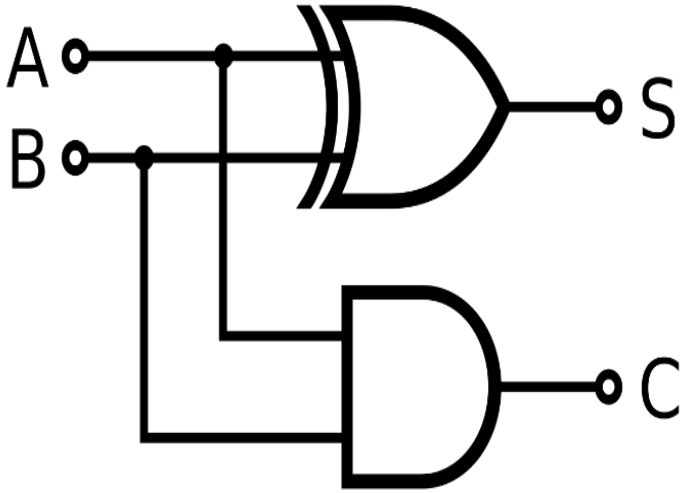
KNOWLEDGEGATE

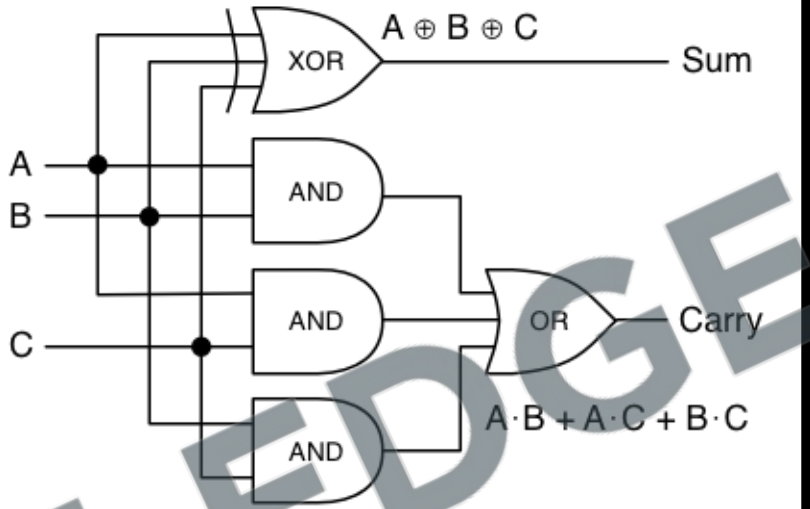
- Application, Practical computer circuits normally contain combinational and sequential logic. For e.g. the part of ALU, that does mathematical calculations is constructed using combinational logic.

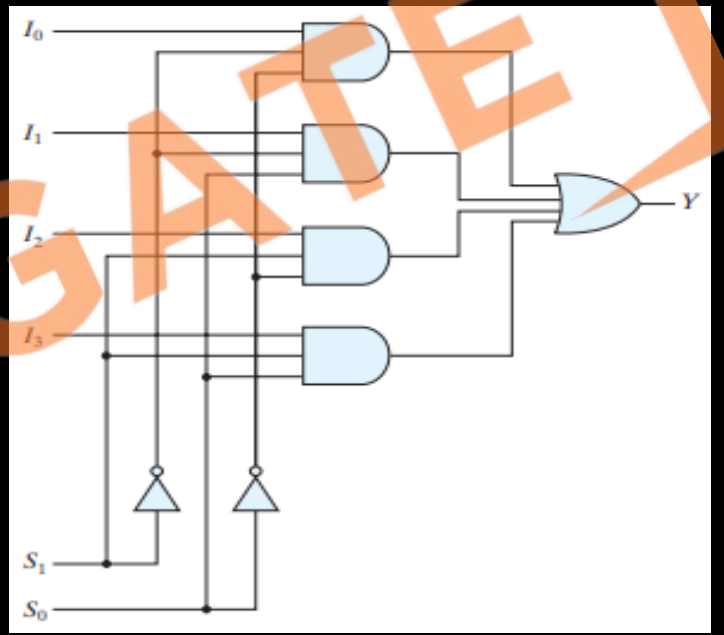


KNOWLEDGEGATE

- Other circuits such as half adders, full adders, half subtractors, full subtractors, multiplexers, demultiplexers, encoders and decoders are also made by using combinational logic.







KNOWLEDGEGATE

www.knowledgegate.in

Design procedure: -

1. Analyse the given problem and identify the number of i/p and o/p variables.

2. Write the truth table based on the specification of the problem.

3. Convert the truth table in minimized Boolean expression using k-map.

4. Draw the logic circuit for the above obtained output expression.

KNOWLEDGEGATE

Q Design a digital system for a car manufacturing company, where we want to design a warning signal for a car, there are three inputs, lights of the car(L), day or night(D), ignition (on/off).?

Solution

1) Understand the problem- here we will Understand the definition of the problem

Design the truth table –

Light	Day	Engine	Warning
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

KNOWLEDGEGATE

2) Write the Boolean expression

- $W(L, D, E) = \sum_m (1, 4, 6, 7)$

- $W(L, D, E) = \prod_M (0, 2, 3, 5)$

Light	Day	Engine	Warning
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

KNOWLEDGEGATE

3) Minimize Boolean expression

W = a’b’c + ab’c’ + abc’ + abc

	ab	a’b’	a’b	ab	ab’
c		00	01	11	10
c’	0	0	2	6	4
c	1	1	3	7	5

W =

Light	Day	Engine	Warning
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

KNOWLEDGEGATE

4) Implement the expression using logic gates, draw the implementation using logic gates

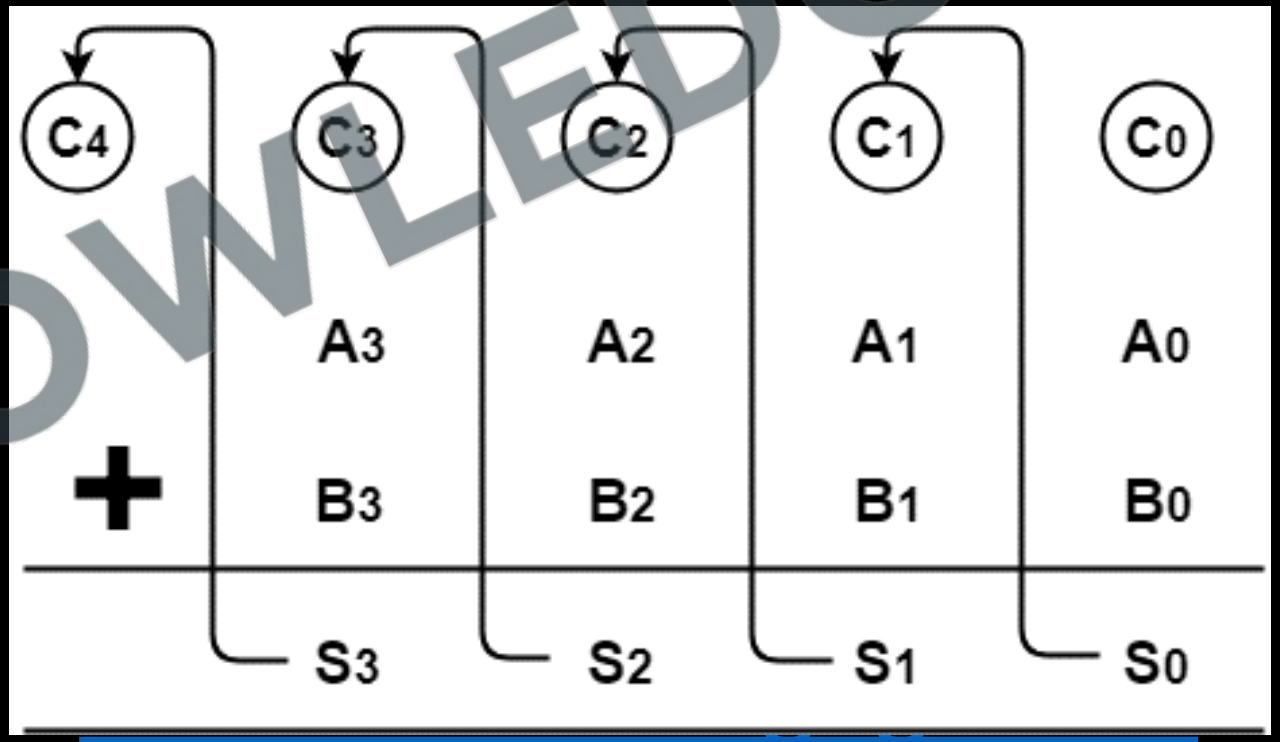
$W = ac' + ab + a'b'c$

Light	Day	Engine	Warning
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

KNOWLEDGEGATE

Adder

- An **adder** is a digital combinational circuit that performs addition of numbers. Are used in the arithmetic logic units or ALU.
- They are also utilized in other parts of the processor, where they are used to calculate addresses, table indices, increment and decrement operators, and similar operations.
- Although adders can be constructed for many number representations, such as binary-coded decimal or excess-3, the most common adders operate on binary numbers.
- Can also be used to perform substruction In cases of 2's representation.



KNOWLEDGEGATE

Half adder

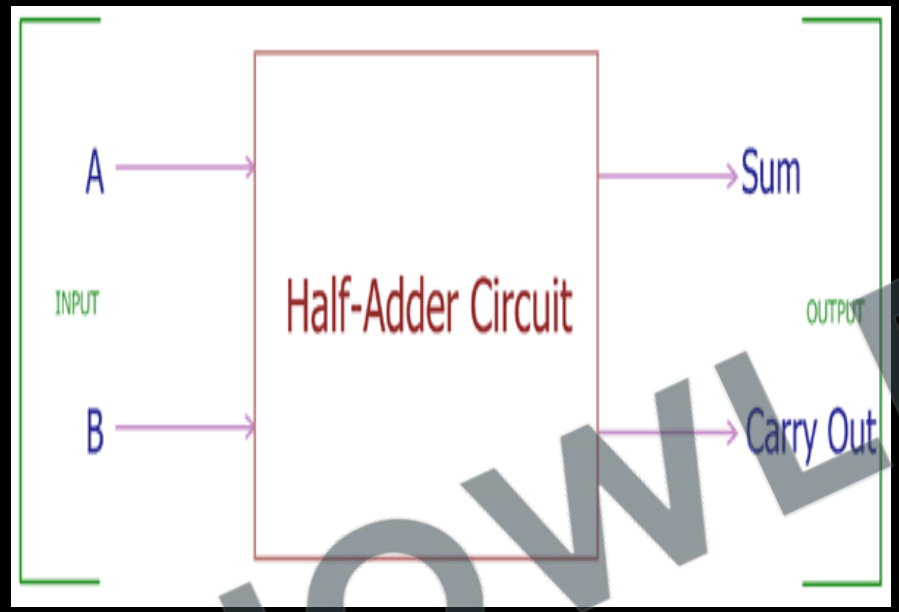
- The simplest form of addition is addition of two binary digits, consists of four possible elementary operations

$$\begin{array}{r} 1 \\ +1 \\ \hline \\ \hline \end{array} \qquad \begin{array}{r} 1 \\ +0 \\ \hline \\ \hline \end{array} \qquad \begin{array}{r} 0 \\ +1 \\ \hline \\ \hline \end{array} \qquad \begin{array}{r} 0 \\ +0 \\ \hline \\ \hline \end{array}$$

- The first three operations produce a sum of two digits, but when both augend and addend bits are equal to 1, the binary sum consists of two digits. The higher significant bit of this result is called a carry.

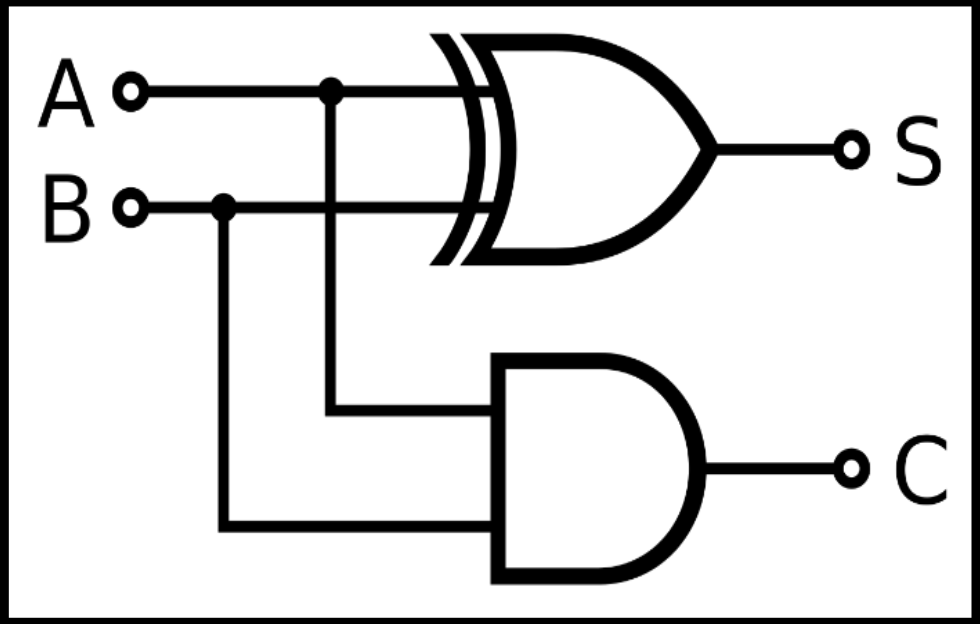
KNOWLEDGEGATE

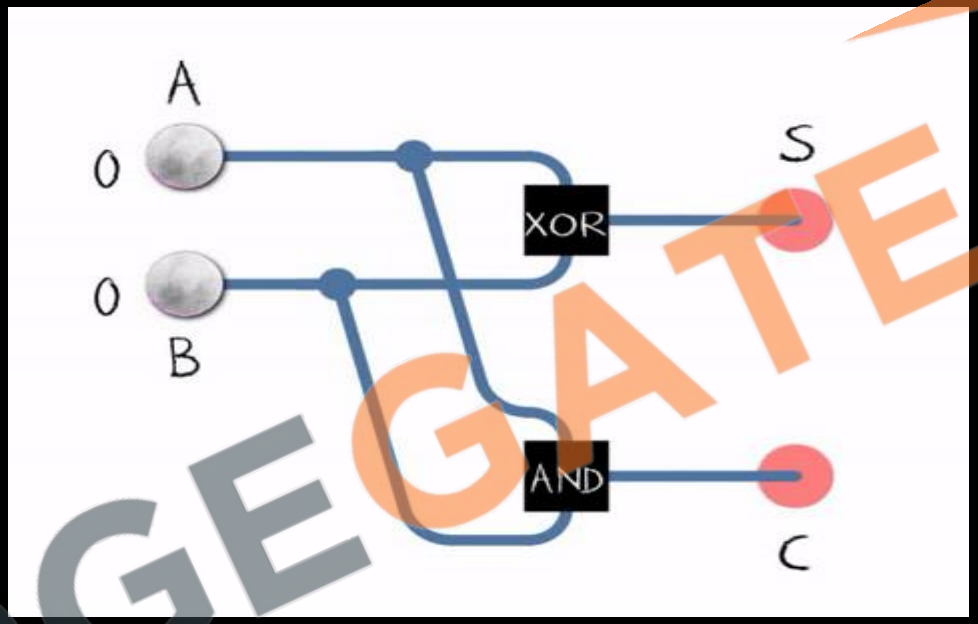
- It is a combinational circuit, which perform the arithmetic addition of two one-bit binary numbers is referred to as an half-adder.
- So, in half adder inputs are adds two single binary bits *A* and *B*, and two outputs, sum (*S*) and carry (C).



INPUTS		OUTPUTS	
A	B	Carry	Sum
0	0		
0	1		
1	0		
1	1		

KNOWLEDGEGATE

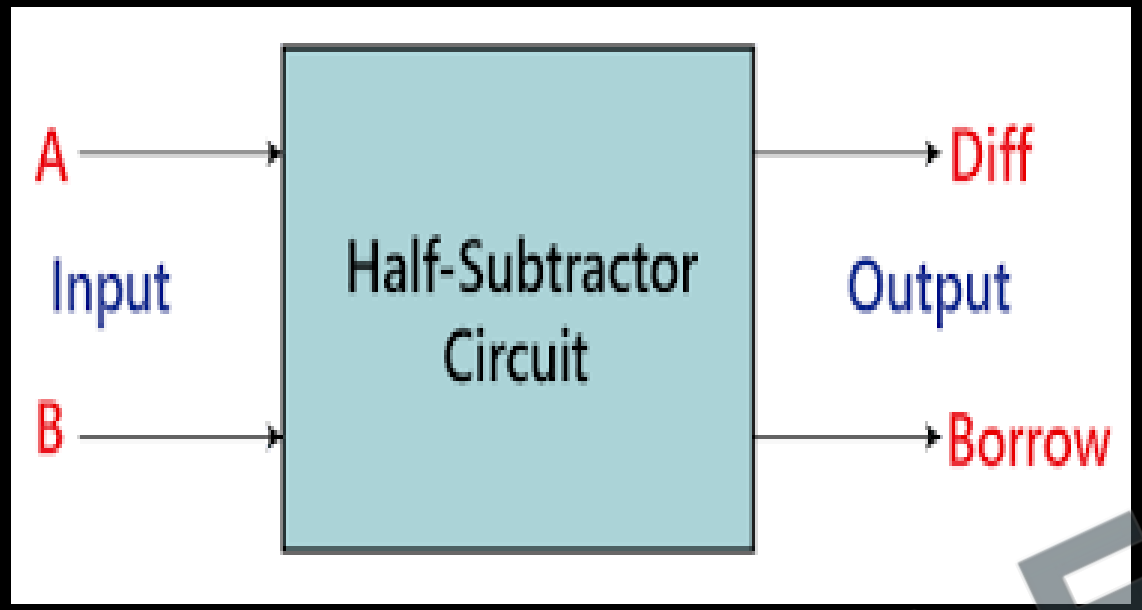




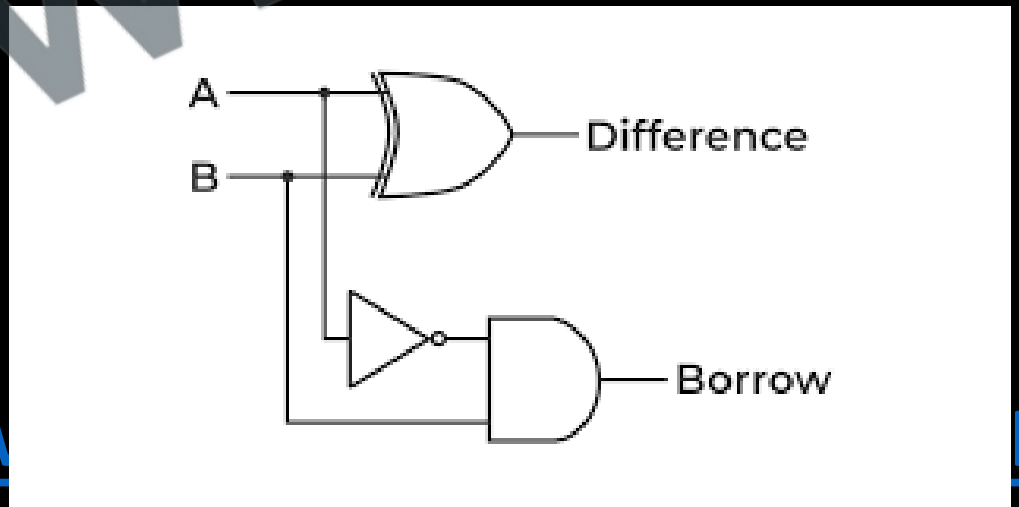
- Cost of implementation a half adder is one EX-OR gate and one AND gate.
- A half adder has only two inputs and there is no provision to add a carry coming from the lower order bits when multi bit number addition is performed. For this reason, we have designed a full adder.

KNOWLEDGEGATE

Half subtractor



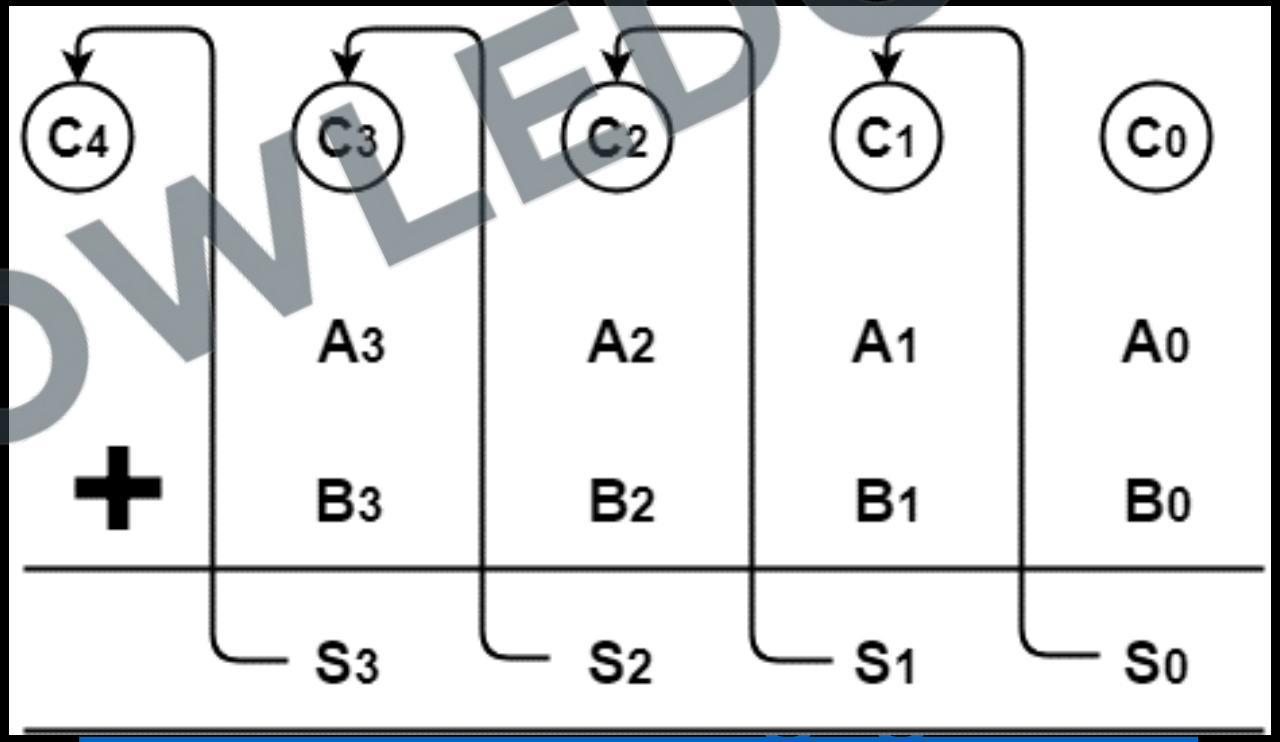
Inputs		Outputs	
A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0



KNOWLEDGEGATE

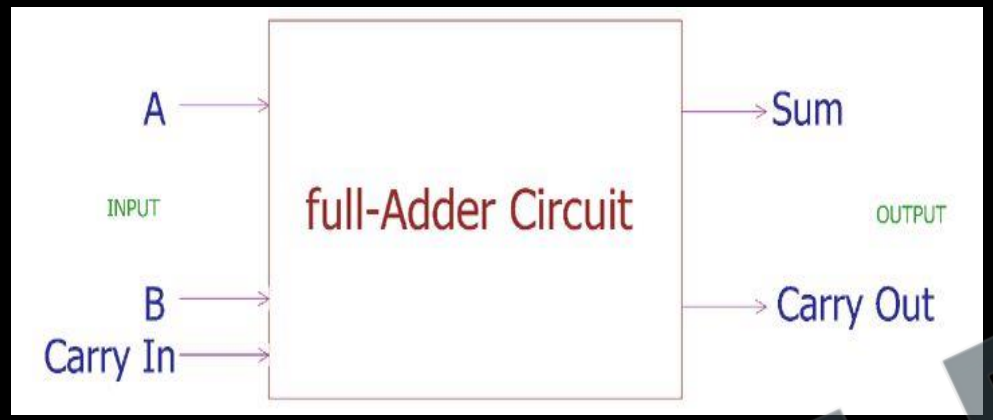
Full adder

1. A full adder is a combinational logic circuit that performs the arithmetic sum of three input bits.
2. Where A_n, B_n are the n^{th} order bits of the number A and B respectively and C_n is the carry generated from the addition of $(n-1)^{th}$ order bits.
3. It consists of three input bits, denoted by A (First operand), B (Second operand), C_{in} (Represents carry from the previous lower significant position).



KNOWLEDGE GATE

- Two output bits are same as of half adder, which is Sum and $Carry_{out}$.
- When the augend and addend number contain more significant digits, the carry obtained from the addition of two bits is added to the next higher order pair of significant bits.



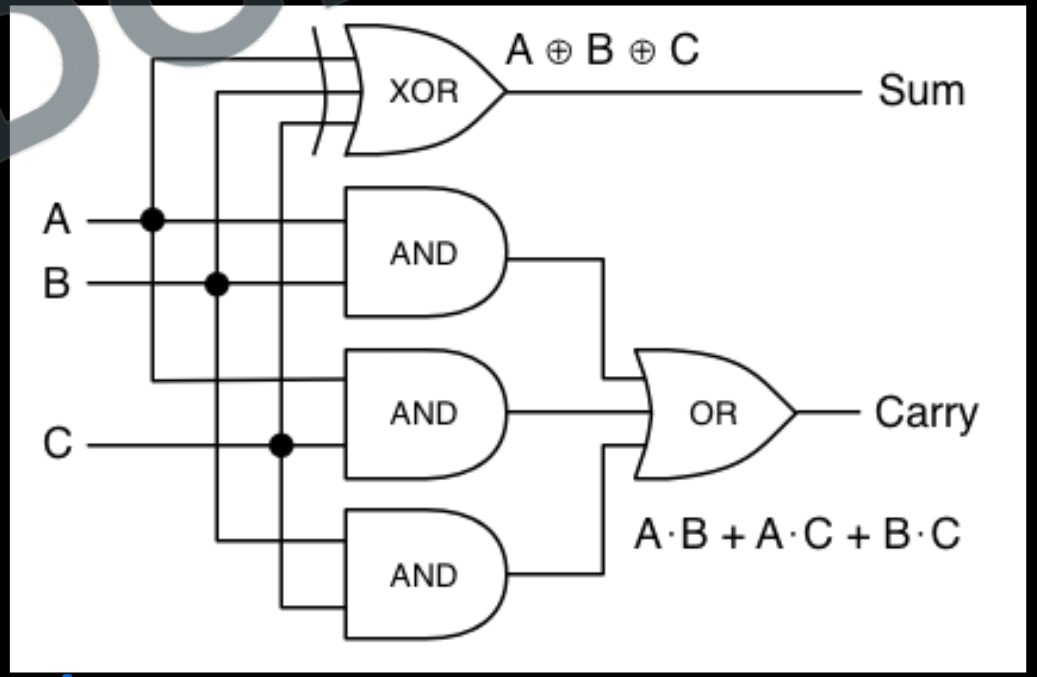
INPUTS			OUTPUTS	
A	B	C_{in}	C_{out}	Sum
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

KNOWLEDGEGATE
www.knowledgegate.in

	ab	a'b'	a'b	ab	ab'
c_{in}		00	01	11	10
c_{in}'	0	0	2	6	4
c_{in}	1	1	3	7	5

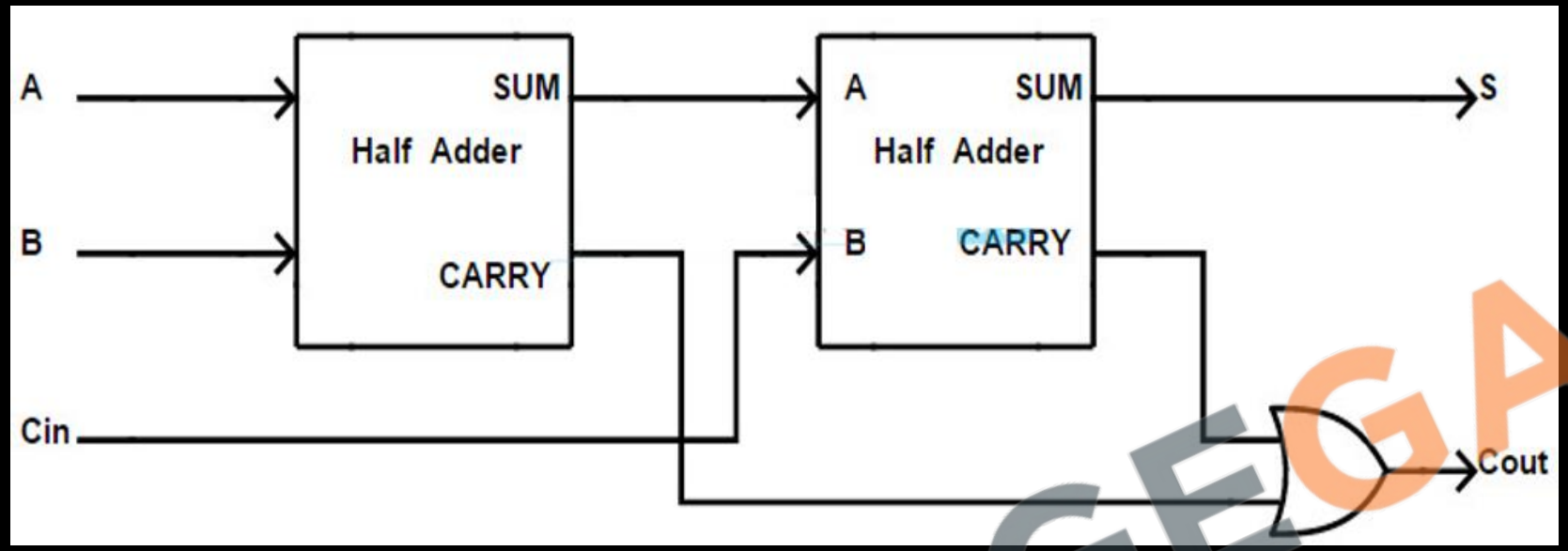
	ab	a'b'	a'b	ab	ab'
c_{in}		00	01	11	10
c_{in}'	0	0	2	6	4
c_{in}	1	1	3	7	5

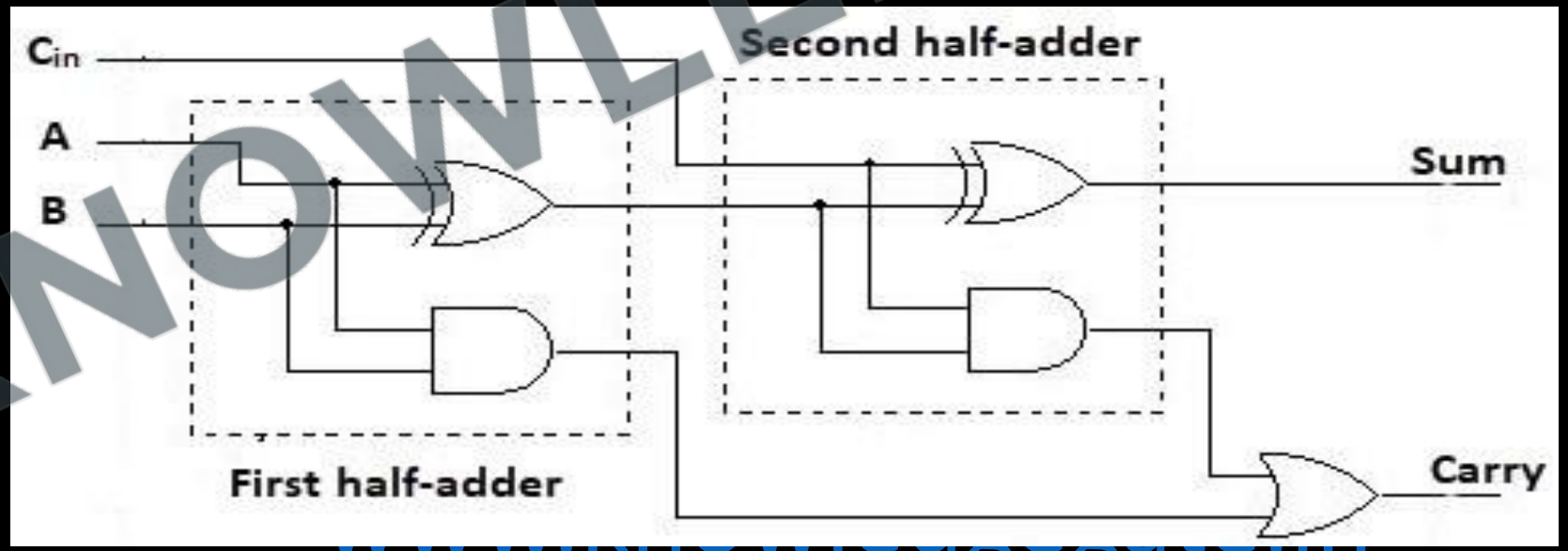
INPUTS			OUTPUTS	
A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



KNOWLEDGEGATE

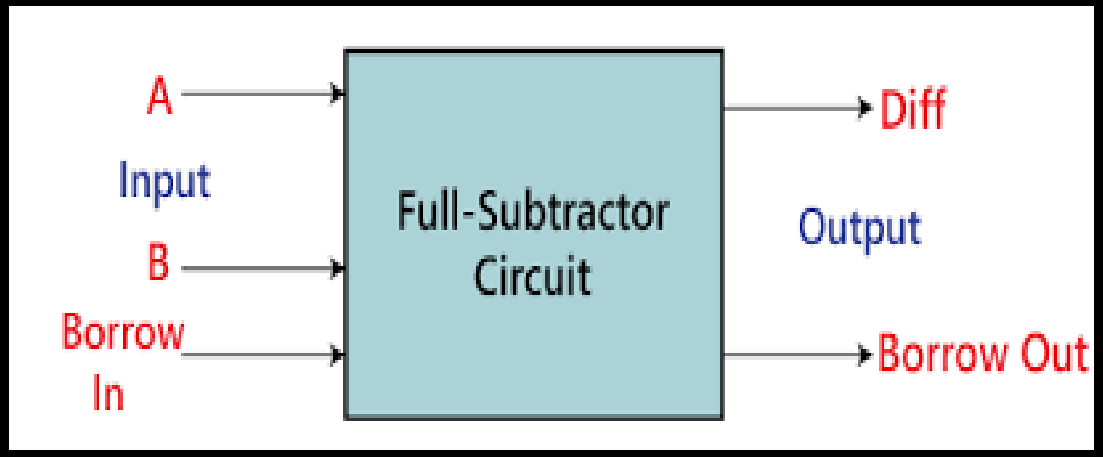
www.knowledgegate.in

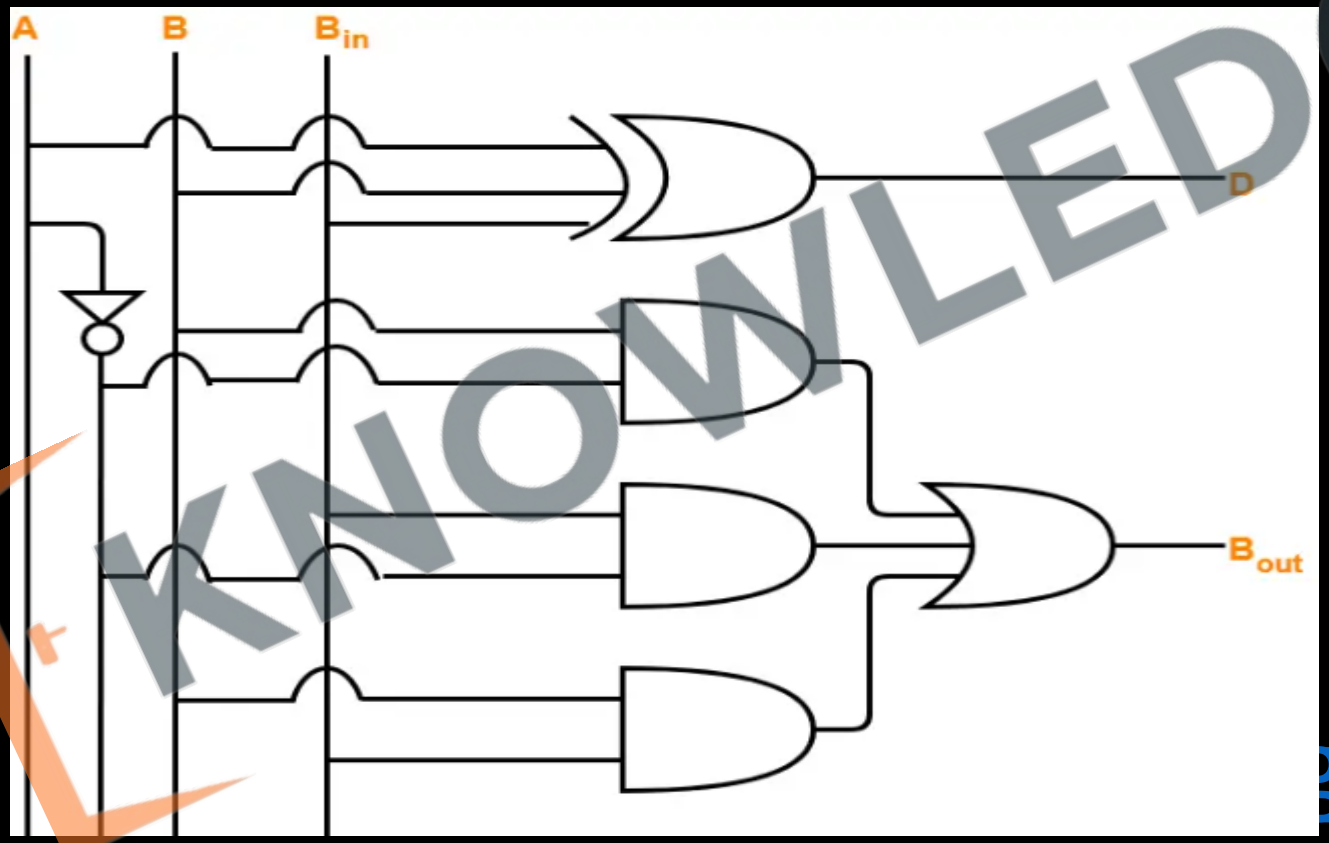




KNOWLEDGEGATE

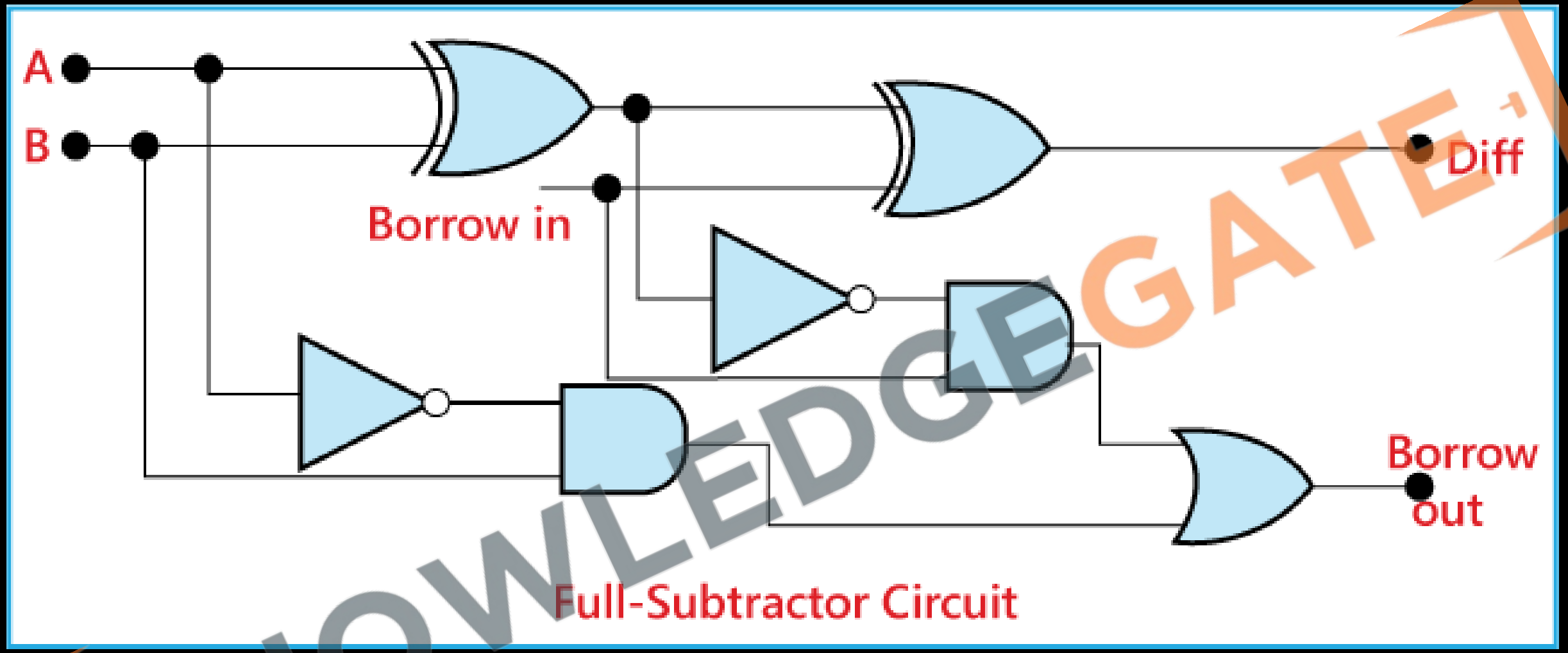
Full subtractor





Inputs			Outputs	
A	B	$Borrow_{in}$	Diff	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

KNOWLEDGEGATE
gegate.in



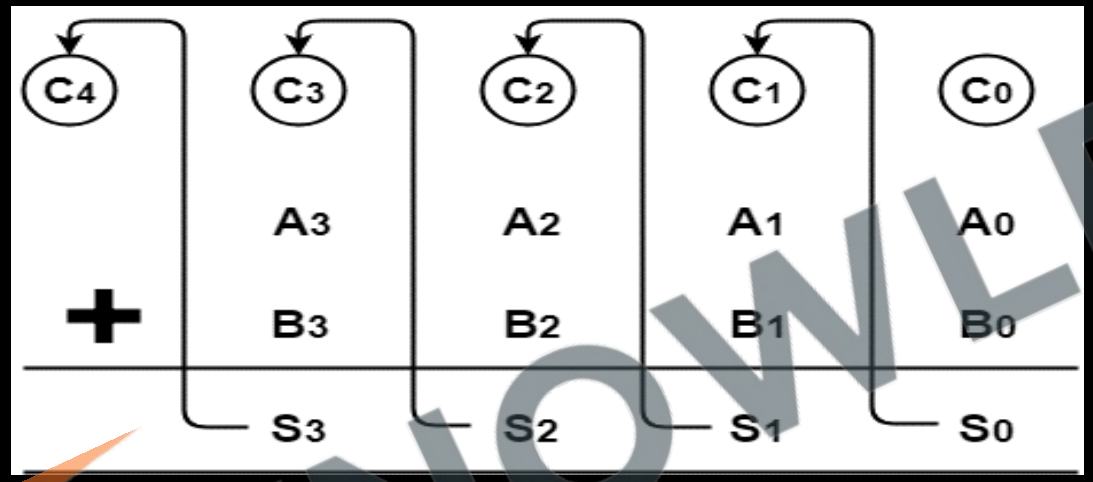
Full-Subtractor Circuit

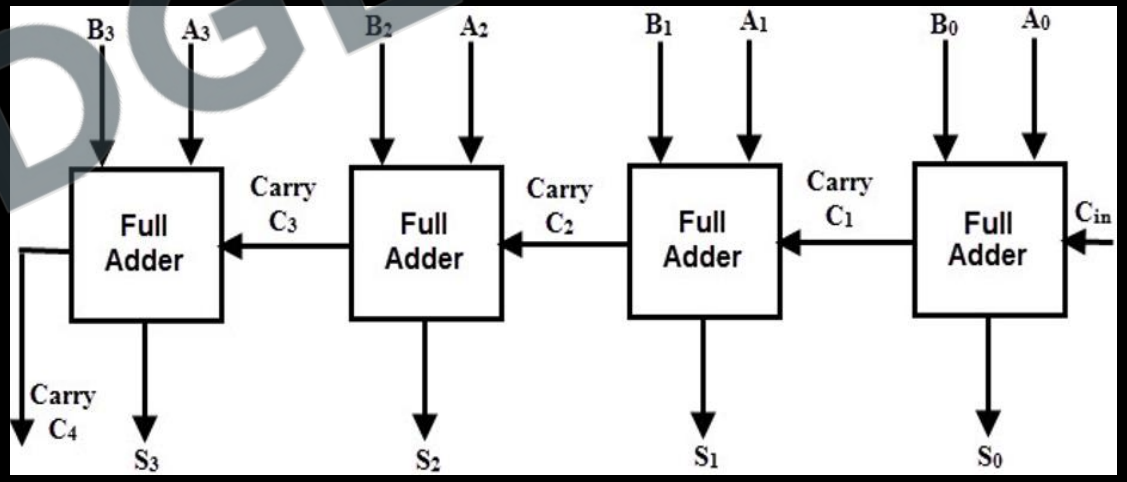
KNOWLEDGEGATE

www.knowledgegate.in

Four-bit parallel binary adder / Ripple adder

- As we know that full adder is capable of adding two 1 bit number and 1 previous carry, but in order to add binary numbers with more than one bits, additional full adders must be employed. For e.g. a four bit binary adder can be constructed using four full adders.
- Theses four full adders are connected in cascade, carry output of each adder is connected to the carry input of the next higher-order adder. So a n-bit parallel adder is constructed using ‘n’ number of full adders.

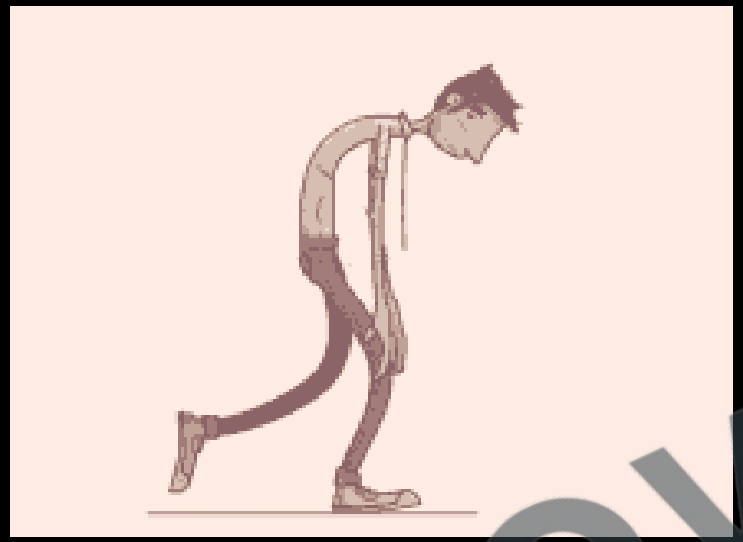




KNOWLEDGEGATE

- There are some scope of improvement in parallel binary adder / Ripple adder is it is very slow

Carry propagation delay



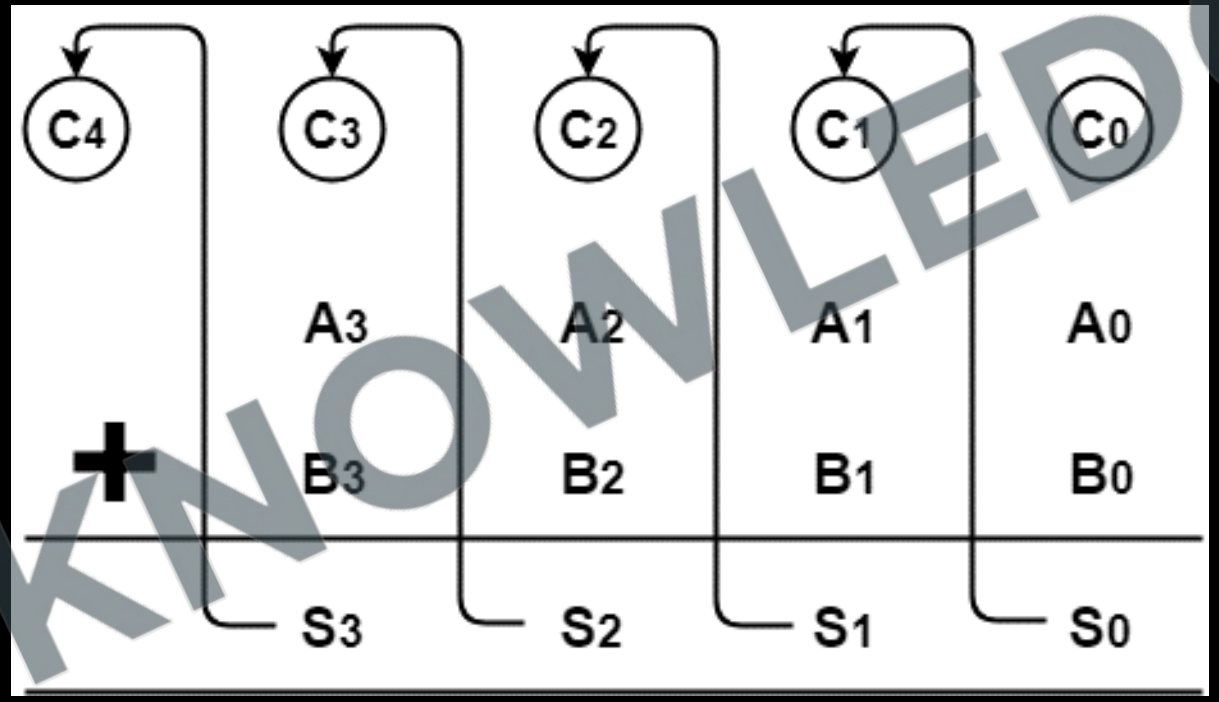
Look ahead Carry Generator

KNOWLEDGEGATE

Look ahead carry adder

- The carry propagation time is an important attribute of the adder because it limits the speed with which two numbers are added. The solution to delay is to increase the complexity of the equipment in such a way that the carry delay time is reduced.

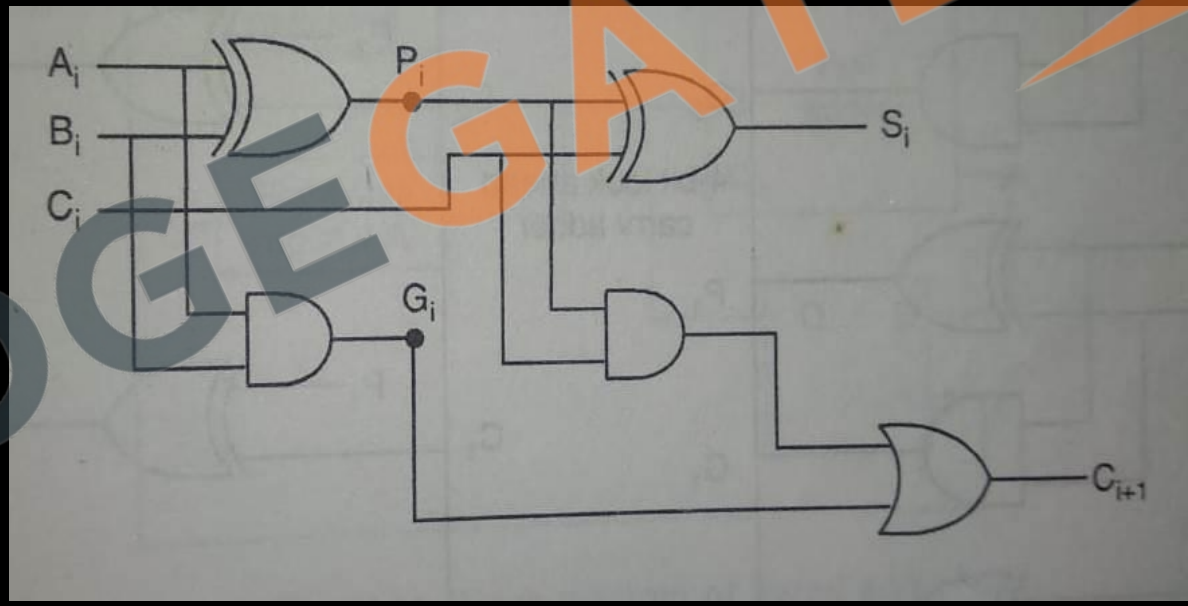
- To solve this problem most widely used technique employs the principle of 'look ahead carry'. This method utilizes logic gates to look at the lower order bits of the augend and addend to see if a higher order carry is to be generated. It uses two functions carry generate G_i and carry propagate P_i



$A(A_3\ A_2\ A_1\ A_0)$
$B(B_3\ B_2\ B_1\ B_0)$

KNOWLEDGEGATE
www.knowledgegate.in

- G_i is called a ***carry generate***, and it produces a carry of 1 when both A_i and B_i are 1, regardless of the input carry C_i.
- P_i is called a ***carry propagate***, because it determines whether a carry into stage i will propagate into stage i + 1.
- We now write the Boolean functions for the carry outputs of each stage and substitute the value of each C_i from the previous equations:



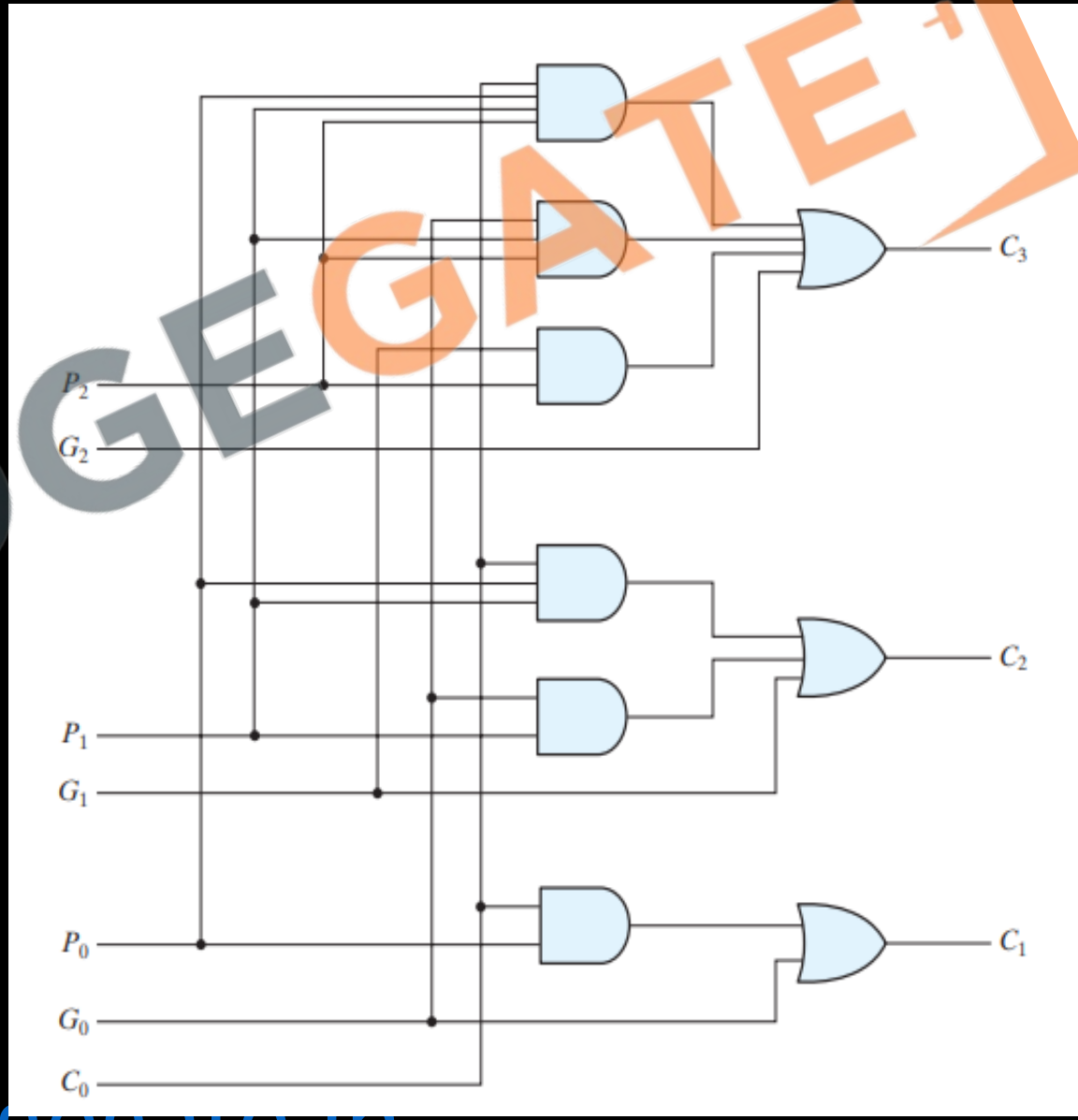
- $P_i = A_i \oplus B_i$
  $G_i = A_i \cdot B_i$
  $S_i = P_i \oplus C_i$
  $C_{i+1} = G_i + P_i C_i$

- $C_0 = 0$
- $C_1 = G_0 + P_0 C_0$
- $C_2 = G_1 + P_1G_0 + P_1P_0 C_0$
- $C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1 P_0 C_0$
- $C_4 = G_3 + G_2.P_3 + G_1.P_2.P_3 + G_0.P_1.P_2.P_3 + C_0.P_0.P_1.P_2.P_3$

KNOWLEDGEGATE

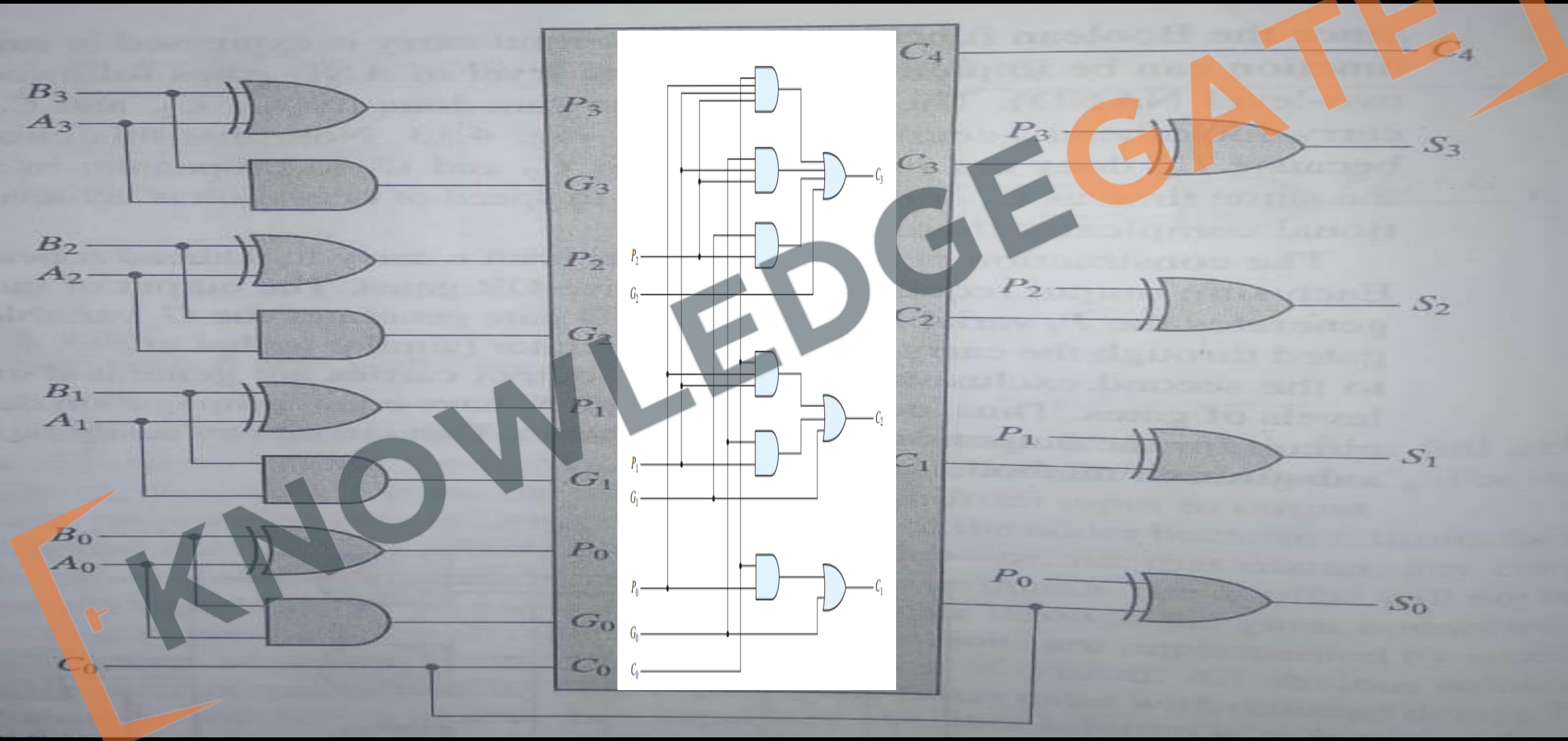
- Since the Boolean function for each output carry is expressed in sum-of-products form. Each function can be implemented with one level of AND gates followed by an OR gate (or by a two-level NAND).

- $C_0 = 0$
- $C_1 = G_0 + P_0 C_0$
- $C_2 = G_1 + P_1G_0 + P_1P_0 C_0$
- $C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1 P_0 C_0$
- $C_4 = G_3 + G_2.P_3 + G_1.P_2.P_3 + G_0.P_1.P_2.P_3 + C_0.P_0.P_1.P_2.P_3$



KNOWLEDGEGATE
www.knowledgegate.in

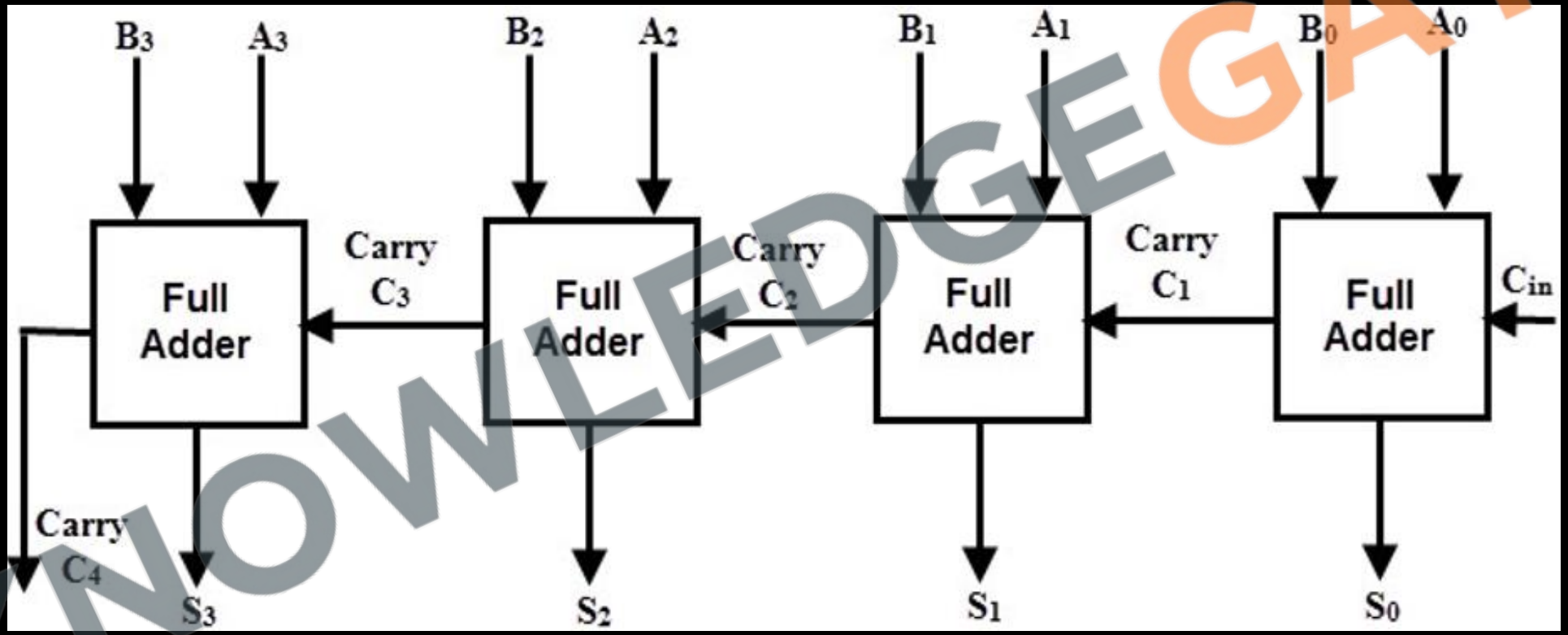
- All output carries are generated after a delay through two levels of gates. Thus, outputs S_1 through S_3 have equal propagation delay times.



KNOWLEDGEGATE

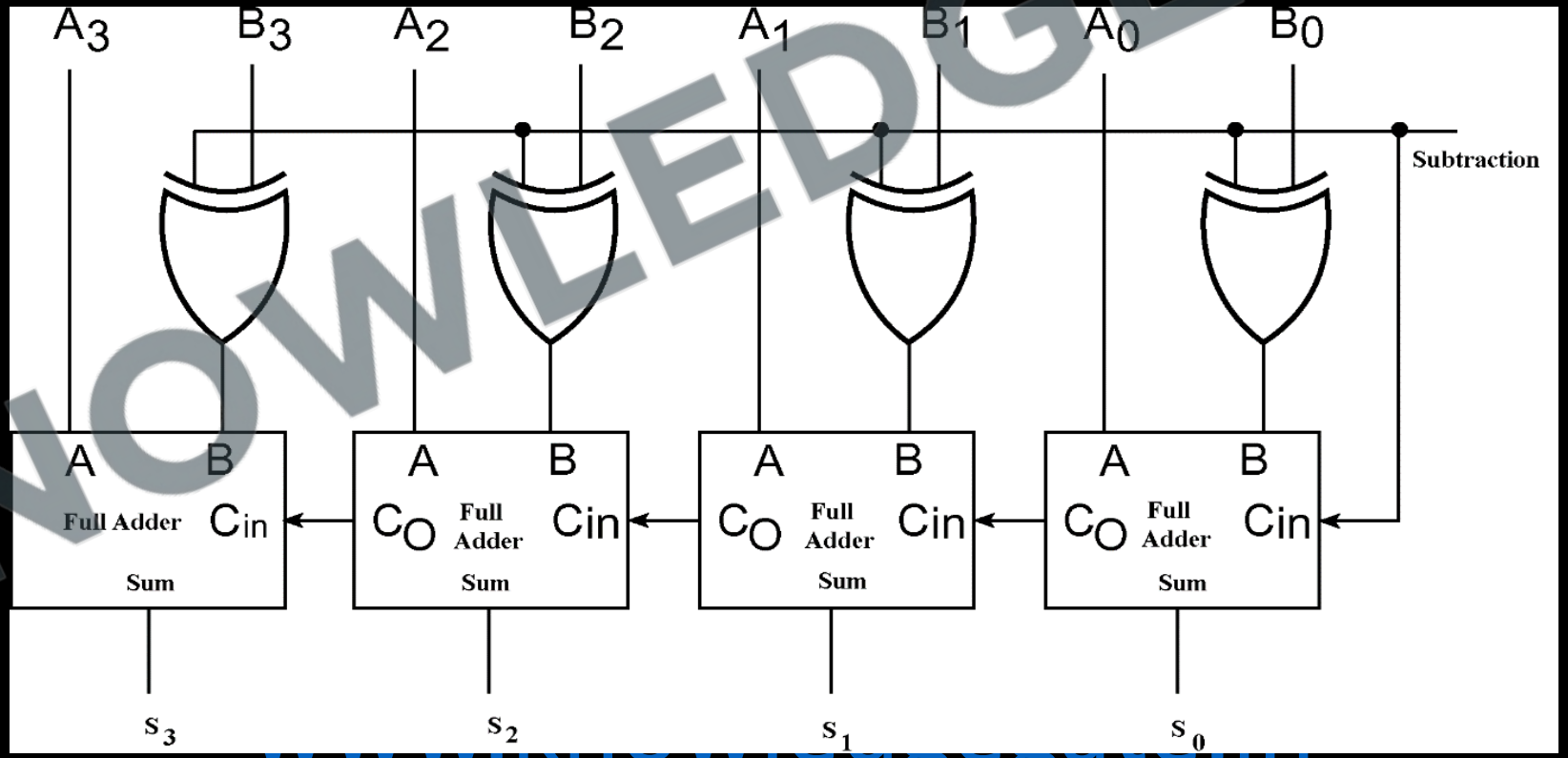
Four-bit ripple adder/subtractor

- The subtraction A - B can be done by taking the 2's complement of B and adding it to A.
- A + (-B)



KNOWLEDGEGATE

- The circuit for subtracting A - B consists of an adder with inverters placed between each data input B and the corresponding input of the full adder. The mode input M controls the operation. When M = 0, the circuit is an adder, and when M = 1, the circuit becomes a subtractor.
- When M = 0, we have B $\oplus$ 0 = B. The full adders receive the value of B, the input carry is 0, and the circuit performs A plus B.
- When M = 1, we have B $\oplus$ 1 = B' and C_0 = 1. The B inputs are all complemented and a 1 is added through the input carry. The circuit performs the operation A plus the 2's complement of B.



KNOWLEDGEGATE

A_1	A_0	B_1	B_0	A>B	A<B	A=B
0	0	0	0	0	0	1
0	0	0	1	0	1	0
0	0	1	0	0	1	0
0	0	1	1	0	1	0
0	1	0	0	1	0	0
0	1	0	1	0	0	1
0	1	1	0	0	1	0
0	1	1	1	0	1	0
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	0	1
1	0	1	1	0	1	0
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	0	1

A>B	a_1a_0	$a'_1a'_0$	a'_1a_0	a_1a_0	$a_1a'_0$
b_1b_0		00	01	11	10
$b'_1b'_0$	00		1	1	1
b'_1b_0	01			1	1
B_1b_0	11				
$B_1b'_0$	10			1	

A=B	a_1a_0	$a'_1a'_0$	a'_1a_0	a_1a_0	$a_1a'_0$
b_1b_0		00	01	11	10
$b'_1b'_0$	00	1			
b'_1b_0	01		1		
B_1b_0	11			1	
$B_1b'_0$	10				1

A<B	a_1a_0	$a'_1a'_0$	a'_1a1_0	a_1a_0	$a_1a'_0$
b_1b_0		00	01	11	10
$b'_1b'_0$	00				
b'_1b_0	01	1			
B_1b_0	11	1	1		1
$B_1b'_0$	10	1	1		

KNOWLEDGE GATE
www.knowledgega

A>B	a_1a_0	$a'_1a'_0$	a'_1a_0	a_1a_0	$a_1a'_0$
b_1b_0		00	01	11	10
$b'_1b'_0$	00		1	1	1
b'_1b_0	01			1	1
B_1b_0	11				
$B_1b'_0$	10			1	

$$A>B = a_0b_1b_0 + a_1a_1b_1 + a_1b_1$$

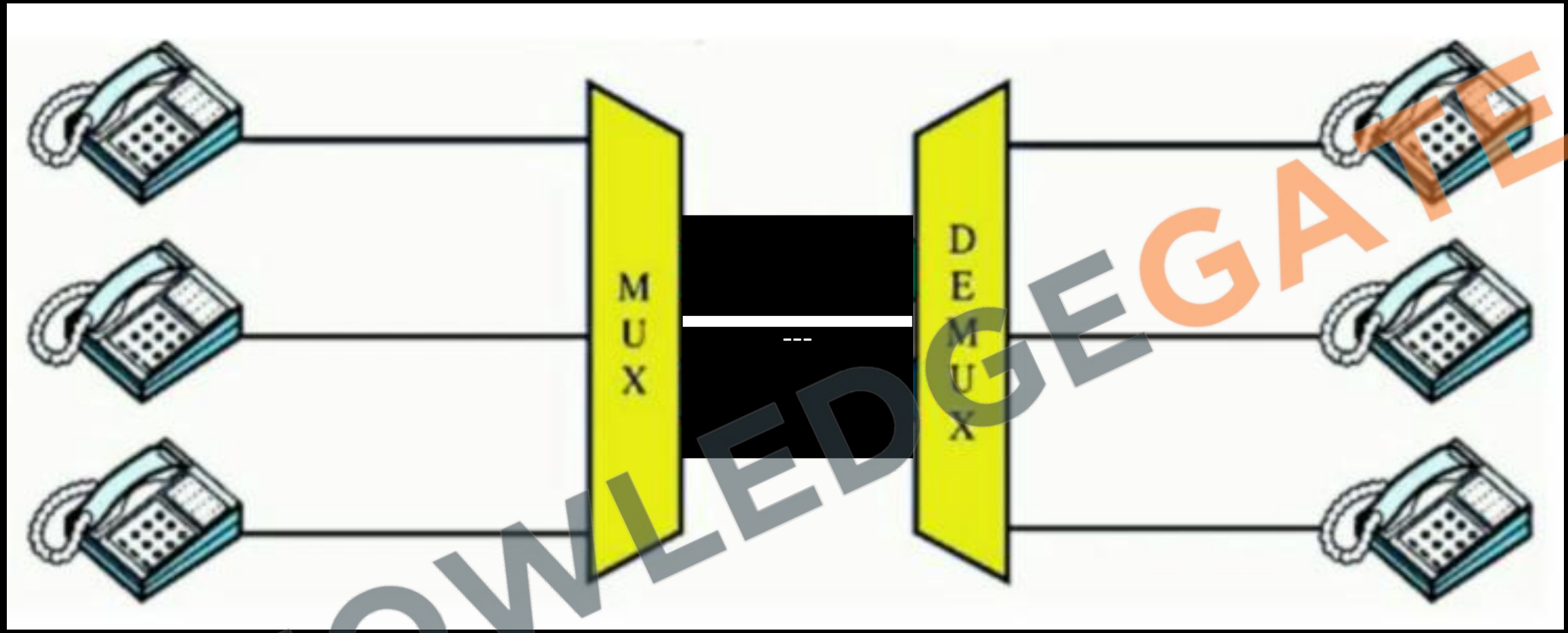
A=B	a_1a_0	$a'_1a'_0$	a'_1a_0	a_1a_0	$a_1a'_0$
b_1b_0		00	01	11	10
$b'_1b'_0$	00	1			
b'_1b_0	01		1		
B_1b_0	11			1	
$B_1b'_0$	10				1

$$A=B = (a_1 \odot b_1)\,(a_0 \odot b_0)$$

A<B	a_1a_0	$a'_1a'_0$	a'_1a1_0	a_1a_0	$a_1a'_0$
b_1b_0		00	01	11	10
$b'_1b'_0$	00				
b'_1b_0	01	1			
B_1b_0	11	1	1		1
$B_1b'_0$	10	1	1		

$$A<B = a_1a_0b_0 + a_1b_1 + a_0b_1b_0$$

KNOWLEDGEGATE
www.knowledgegate.in



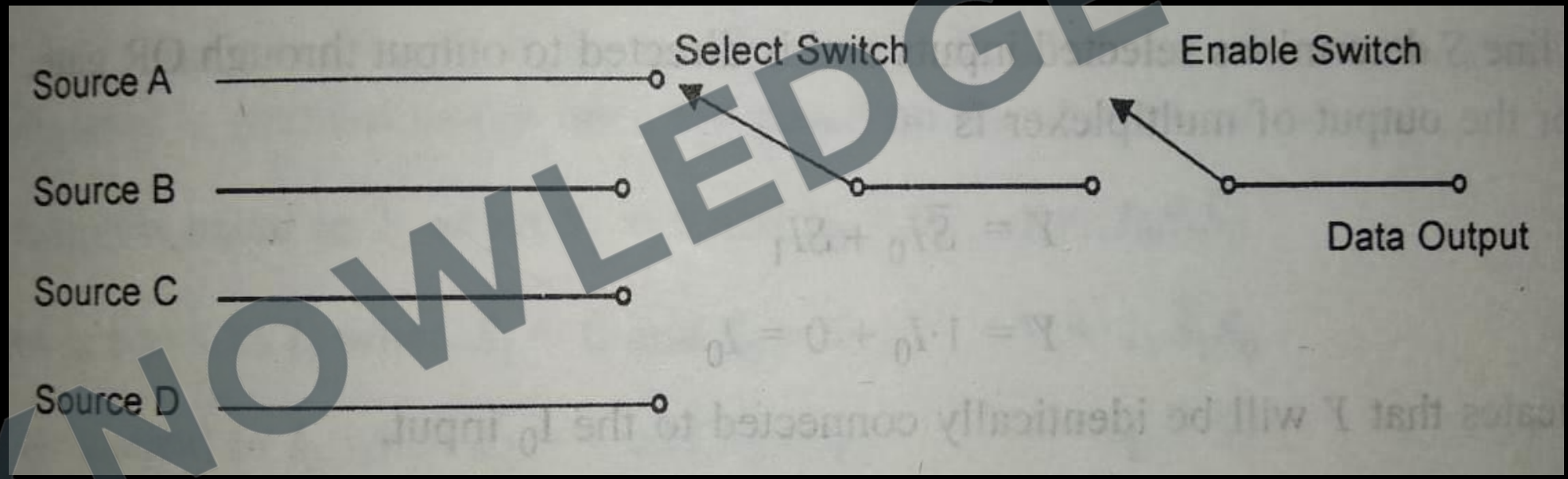
KNOWLEDGEGATE

www.knowledgegate.in

KNOWLEDGE GATE

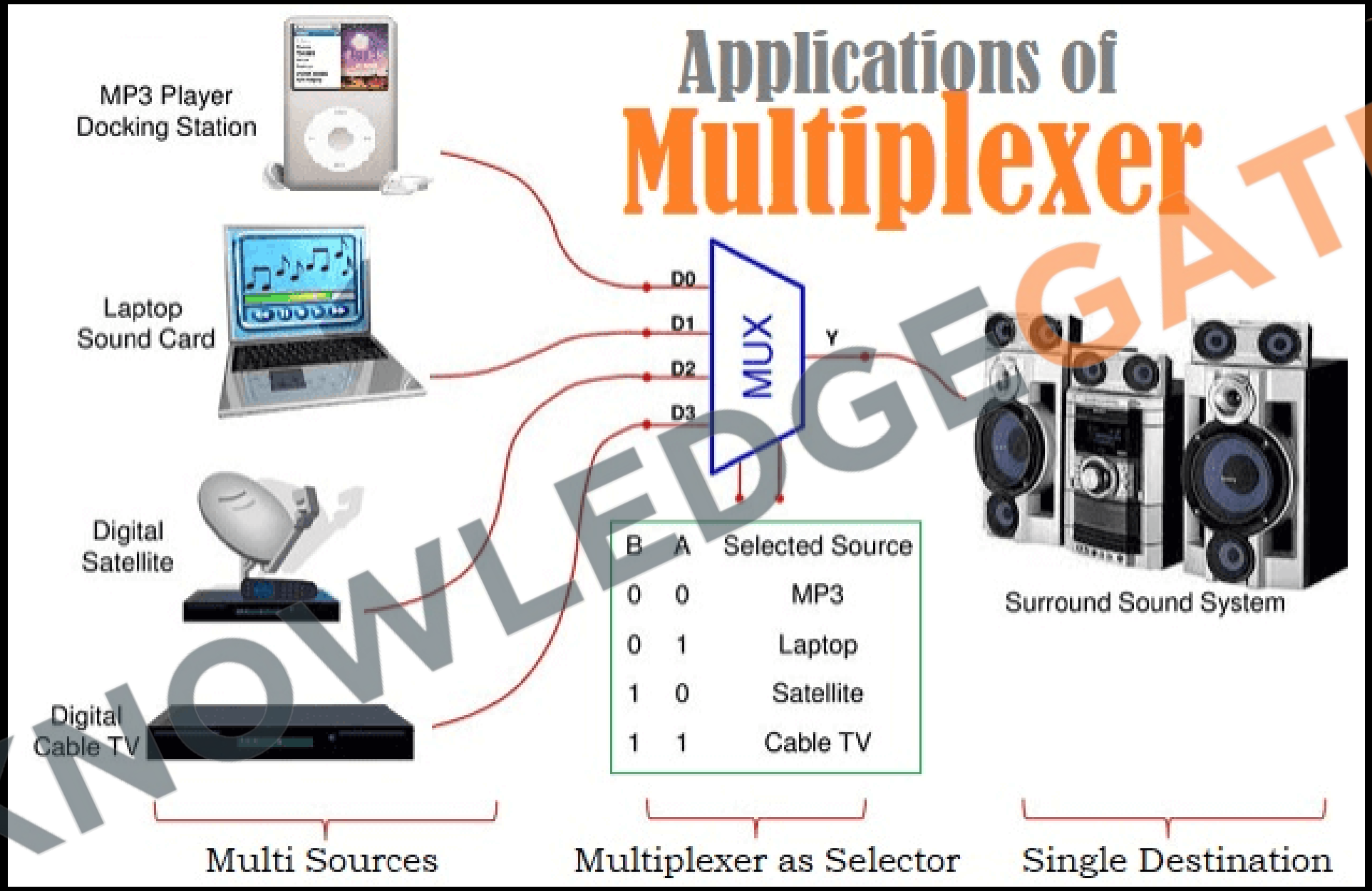
Multiplexer (Selector)

- Multiplexer are special and one of the most widely used combinational circuits.
- Main requirement is out of many inputs we have to select one for e.g. telephone or train leaving the station. So multiplexer do not perform any logical operation or comparison, it just acts as a switch or relay.



KNOWLEDGEGATE

Applications of Multiplexer



B	A	Selected Source
0	0	MP3
0	1	Laptop
1	0	Satellite
1	1	Cable TV

KNOWLEDGEGATE

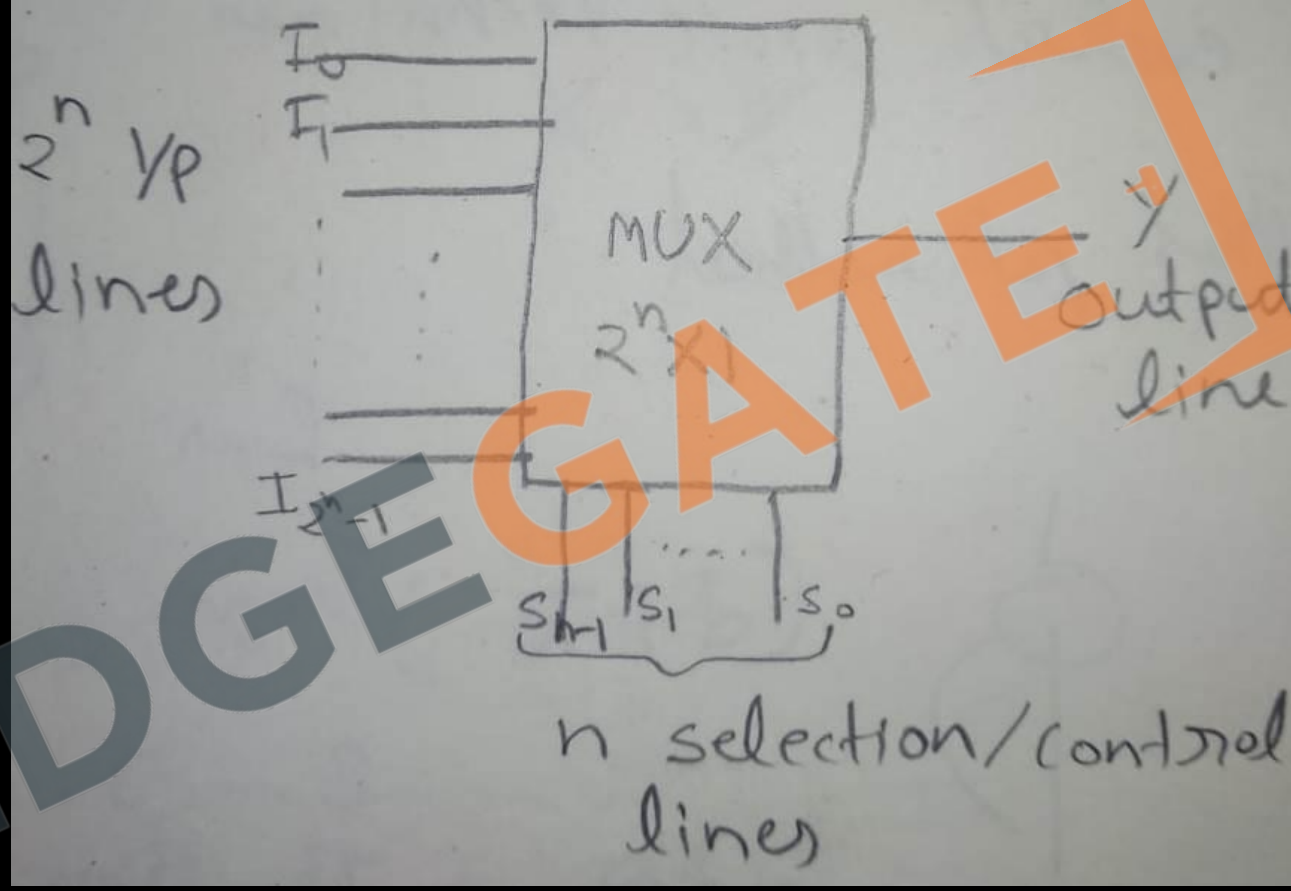
1. A multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line.

2. A multiplexer is also called a data selector, since it selects one of many inputs and steers the binary information to the output line.

3. The selection of a particular input line is controlled by a set of selection lines.

4. There are 2^n input lines and n selection lines whose bit combinations determine which input is to be selected.

5. No of Input line =< 2^n



KNOWLEDGEGATE

Note:

- Can never have two i/p connected to out at any time

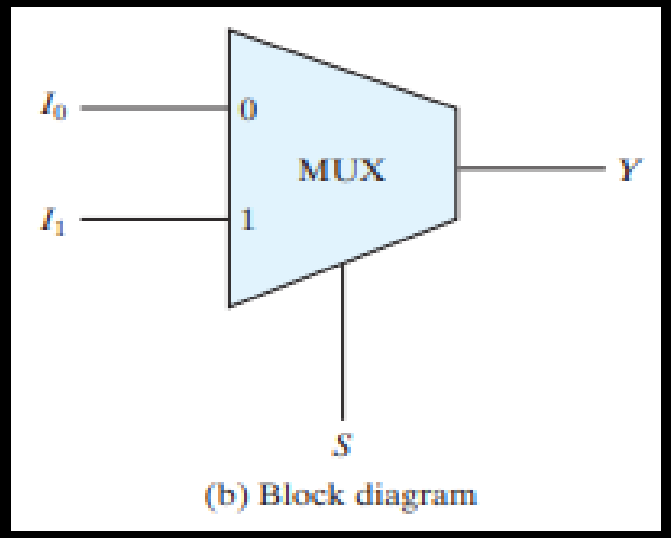
Application

- Parallel data to serial data conversion
- Used as data selector, as the output of a multiplexer is directed from one of various inputs
- Used in implementation of Boolean functions
- Used in communication systems, **Computer Memory, Telephone Network, Transmission from the Computer System of a Satellite**

KNOWLEDGEGATE

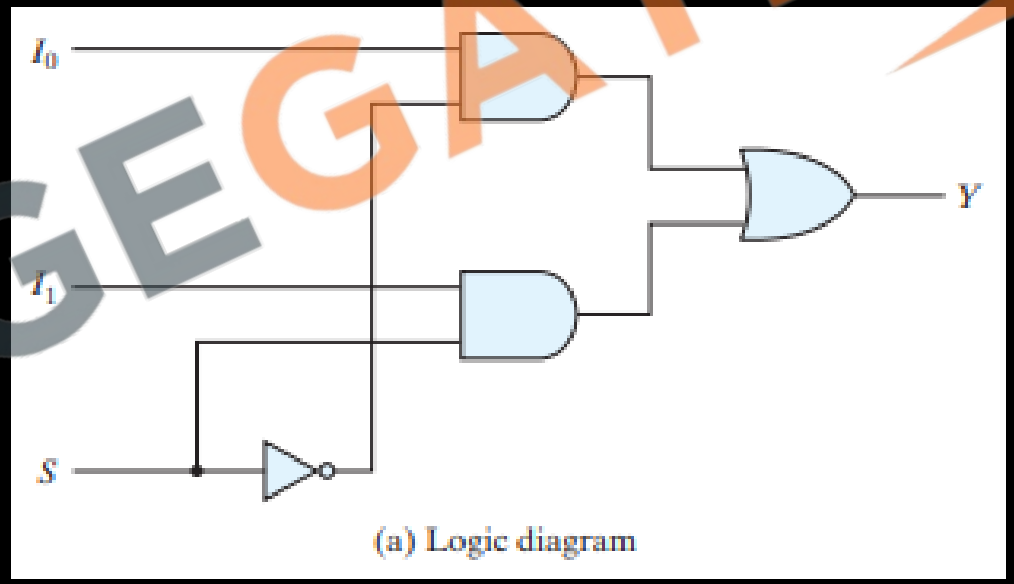
A two-to-one-line multiplexer

- When S = 0, the upper AND gate is enabled and I_0 has a path to the output.
- When S = 1, the lower AND gate is enabled and I_1 has a path to the output.



(b) Block diagram

Input	Output
S	Y
0	I_0
1	I_1



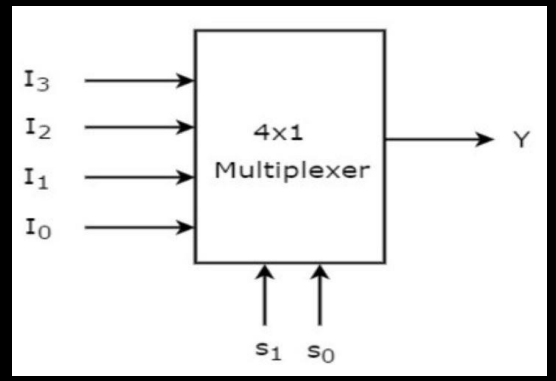
(a) Logic diagram

Characteristic equation: $Y = S_0' I_0 + S_0 I_1$

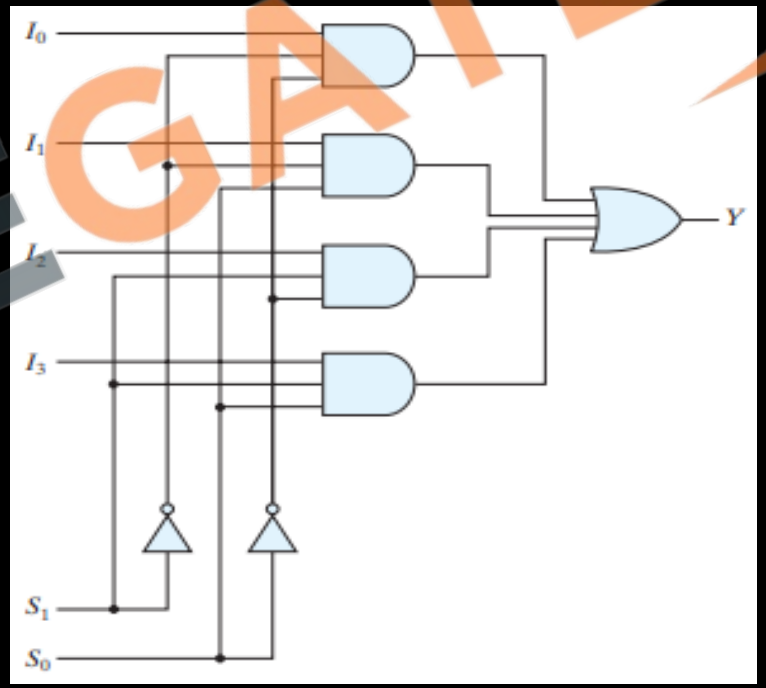
KNOWLEDGEGATE

Case study of 4 to 1

- Each of the four inputs, I_0 through I_3, is applied to one input of an AND gate.
- Selection lines S_1 and S_0 are decoded to select a particular AND gate.
- The outputs of the AND gates are applied to a single OR gate that provides the one-line output.



Input		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



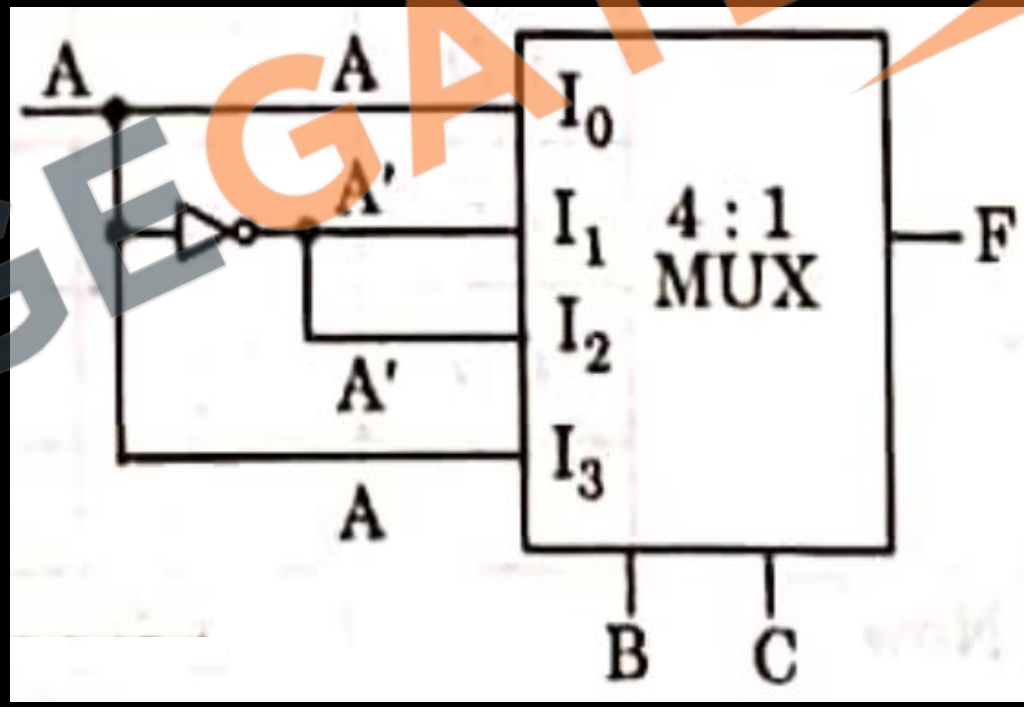
Characteristic equation

$$Y = E\,[(S'_1\,S'_0\,I_0) + (S'_1\,S_0\,I_1) + (S_1\,S'_0\,I_2) + (S_1\,S_0\,I_3)]$$

KNOWLEDGE GATE

Q Implement the function F(a,b,c) = a'b'c + a'bc' + ab'c' + abc. With b and c variable on select line?

	I_0	I_1	I_2	I_3
A'	0	1	2	3
A	4	5	5	7



KNOWLEDGEGATE

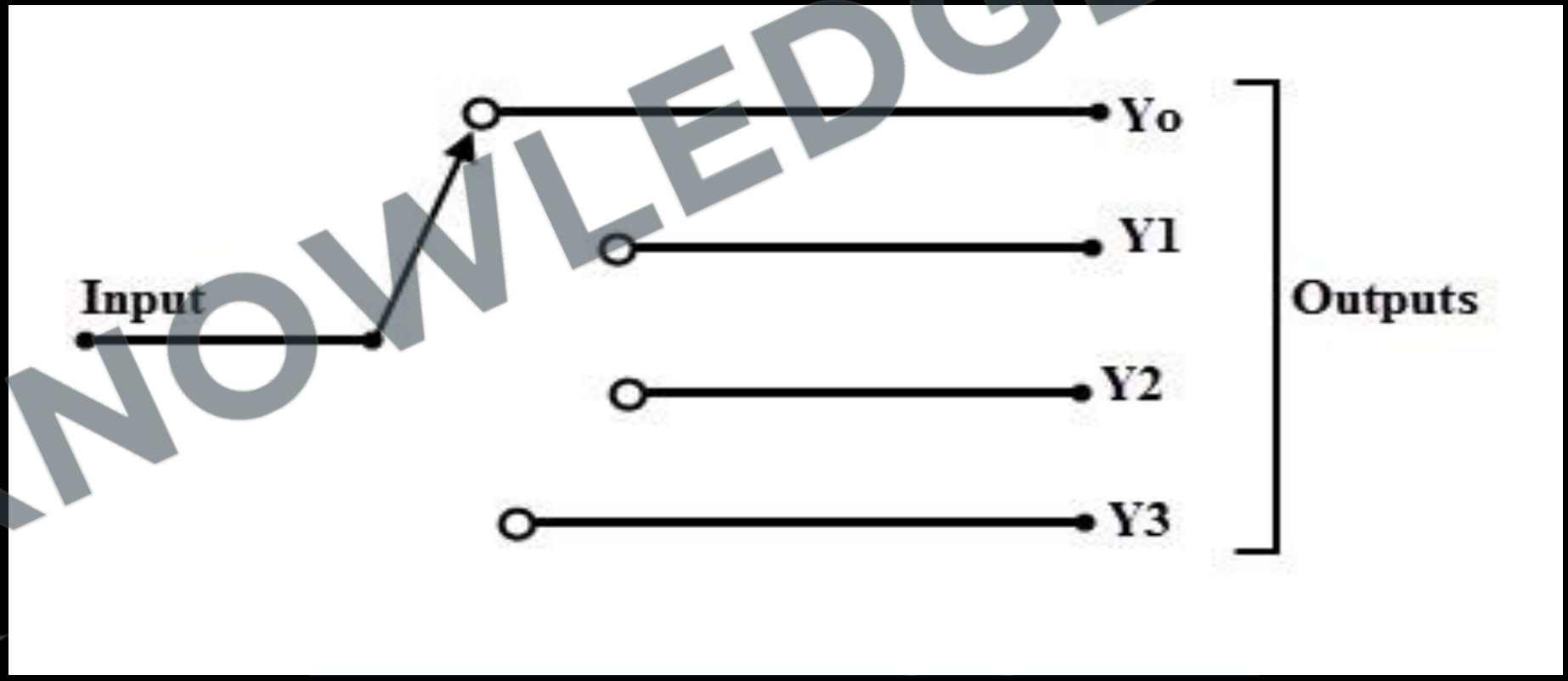
Q Implement the function F(a,b,c) = a’b’c + a’bc’ + ab’c’ + abc. With one, two and three variable on the select line?

KNOWLEDGEGATE

www.knowledgegate.in

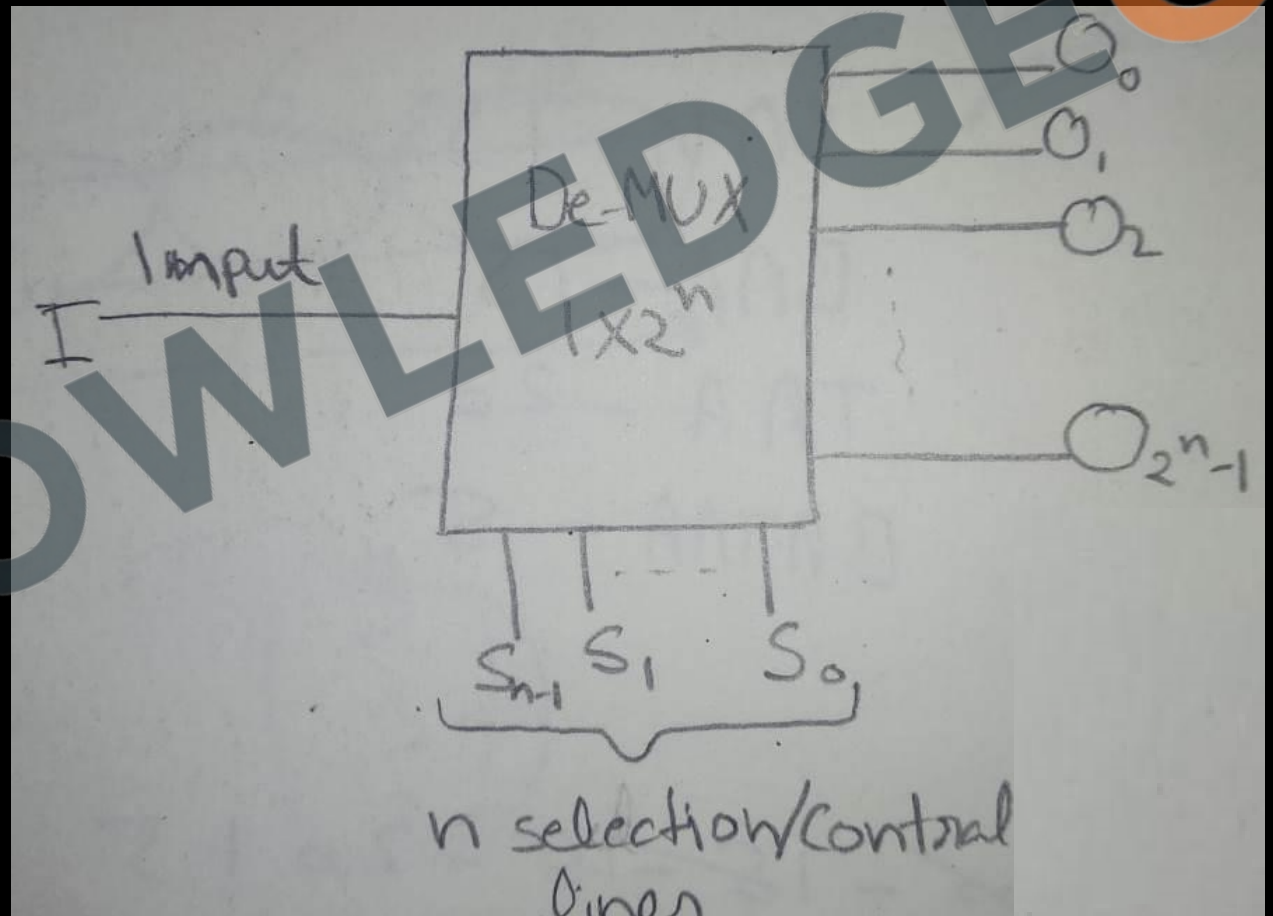
Demultiplexer

1. A demultiplexer (or DeMux) is a device that takes a single input line and routes it to one of several digital output lines.
2. A demultiplexer is also called a data distributor.
3. It is conceptually same as Mux just with reverse logic.



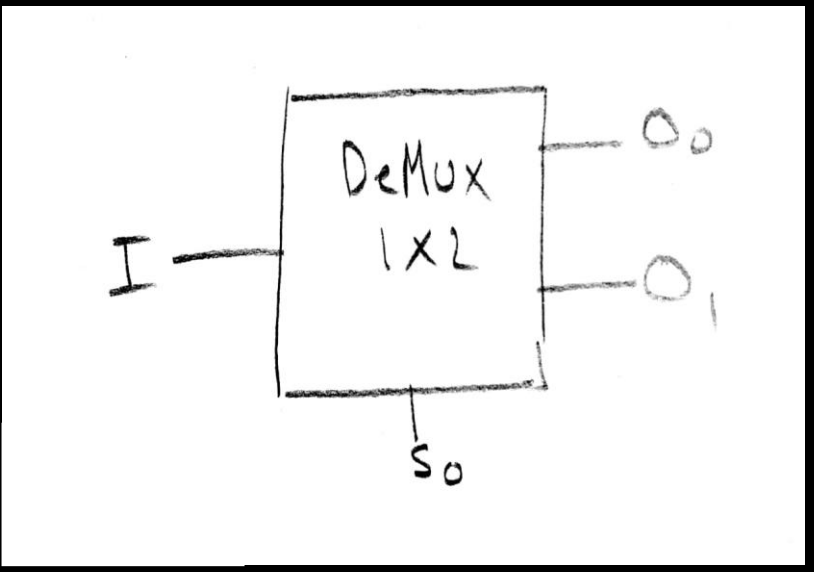
KNOWLEDGEGATE

- A demultiplexer of 2^n outputs has n select lines, which are used to select which output line to send the input.
- A DeMux is a combinational circuit, which is used in data communication
- Serial to parallel conversation
- Demultiplexers are mainly used in Boolean function generators and decoder circuits.

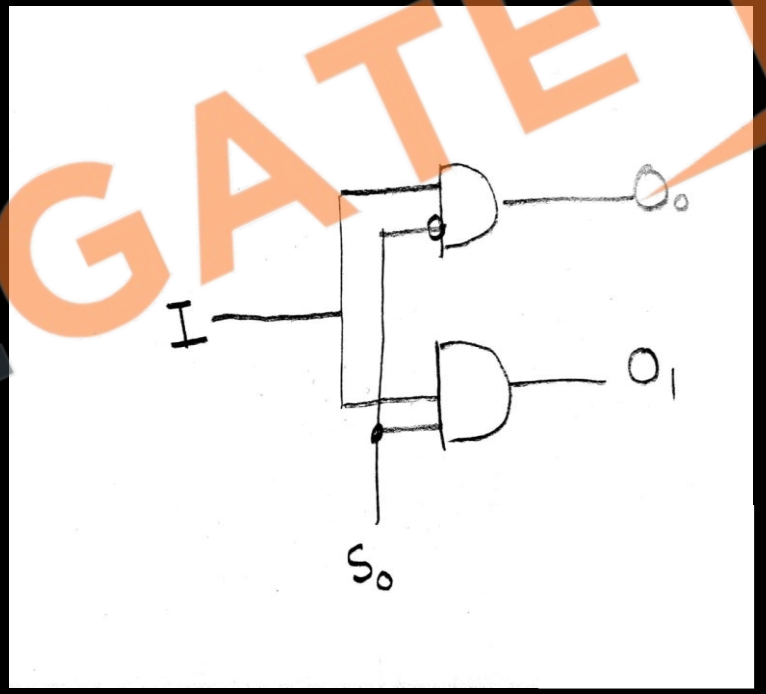


KNOWLEDGEGATE

1 to 2 Demultiplexer

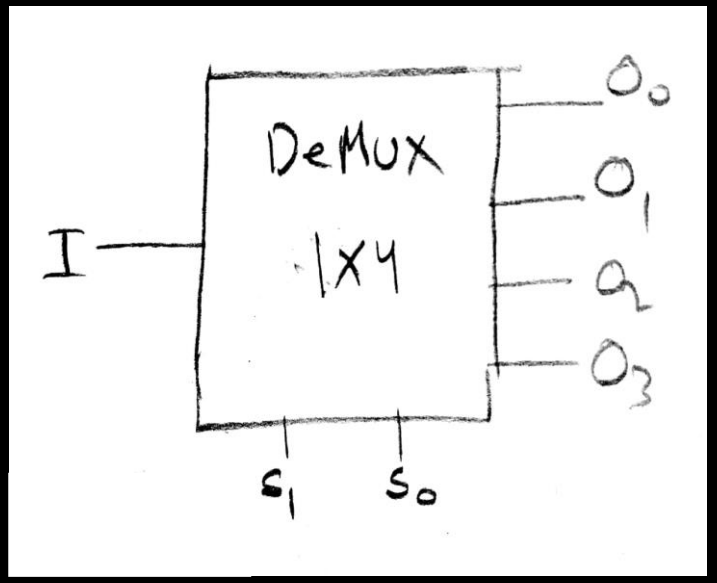


S_0	O_1	O_0
0	0	I
1	I	0

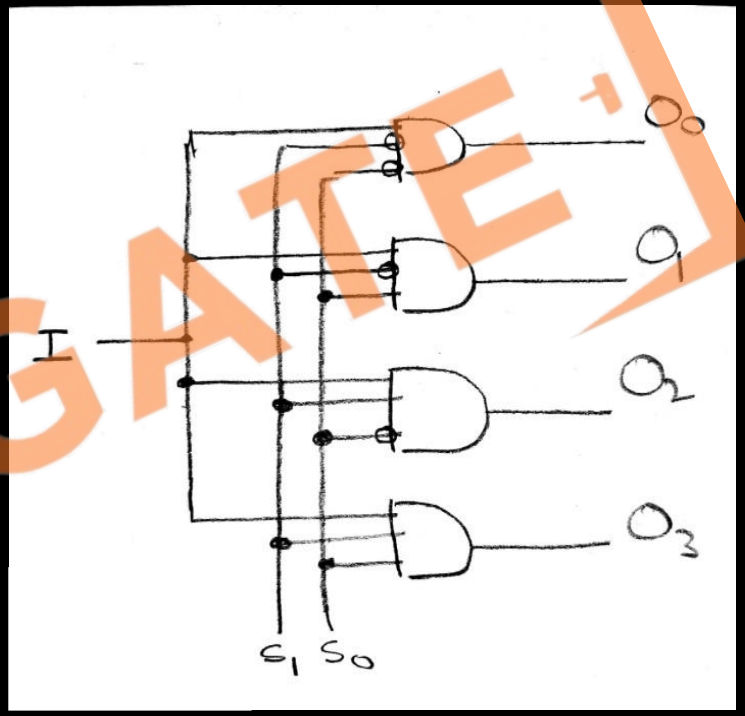


KNOWLEDGE GATE

1 to 4 Demultiplexer



S_1	S_0	O_3	O_2	O_1	O_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

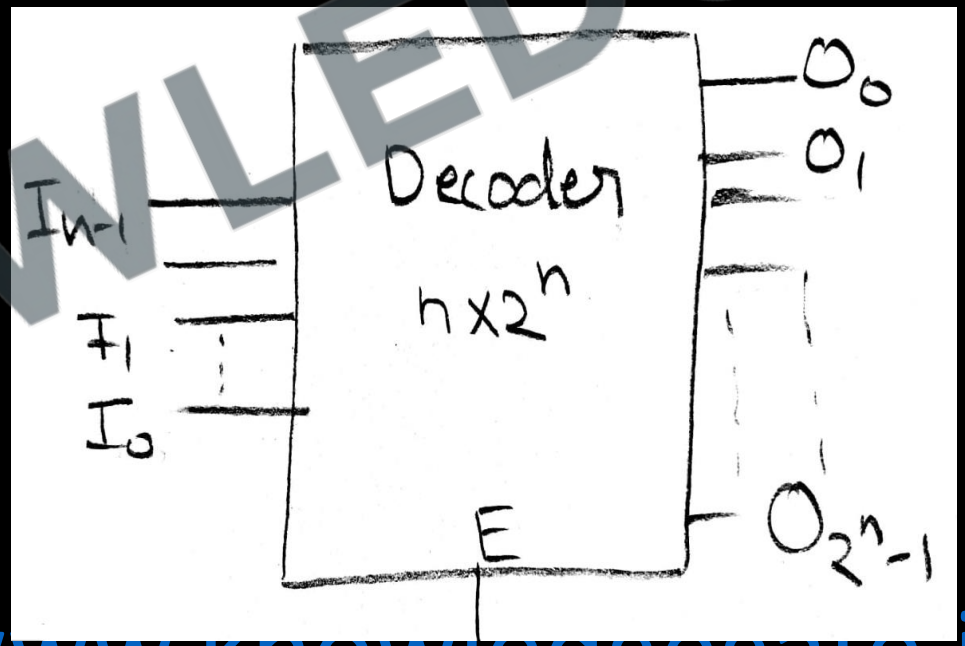


KNOWLEDGEGATE

www.knowledgegate.in

Decoder

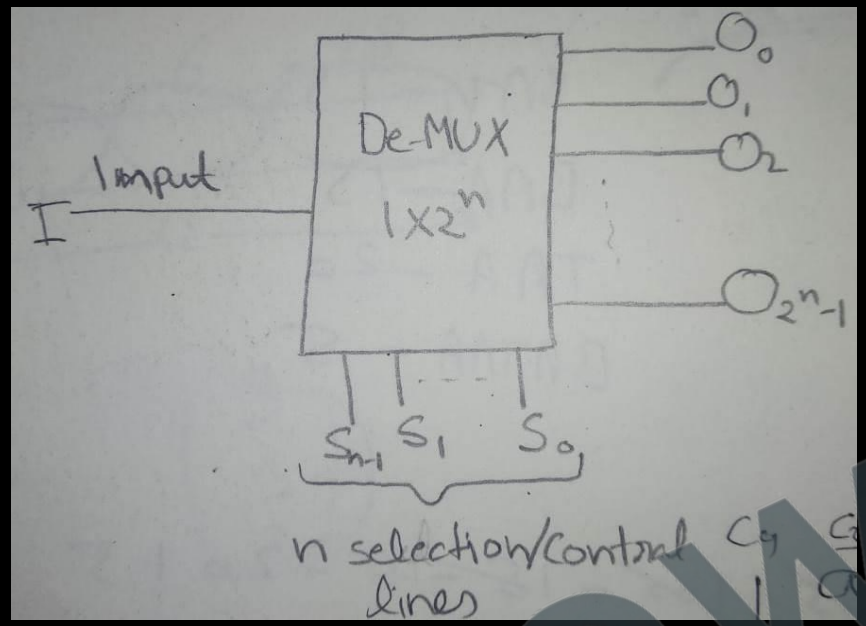
- A decoder is a combinational circuit that decodes binary information from n input lines to a maximum of 2^n unique output lines.
- The decoders are called n -to- m -line decoders, where $m \leq 2^n$.
- Their purpose is to generate the 2^n (or fewer) minterms of n input variables. Each combination of inputs will assert a unique output.
- If the n -bit coded information has unused combinations, the decoder may have fewer than 2^n outputs.

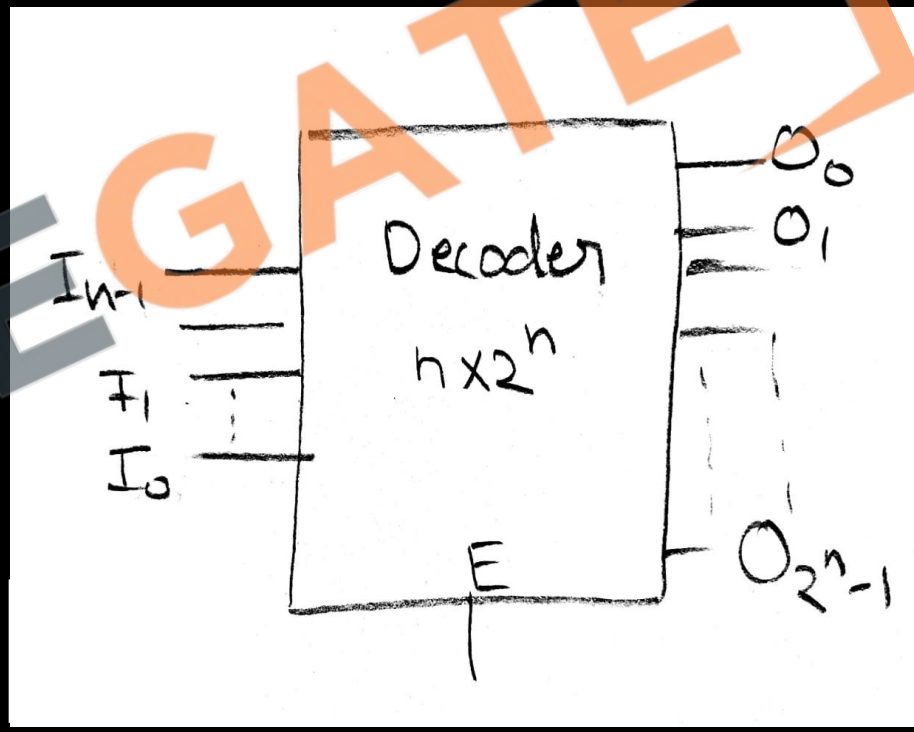


KNOWLEDGE GATE
www.knowledgegate.in

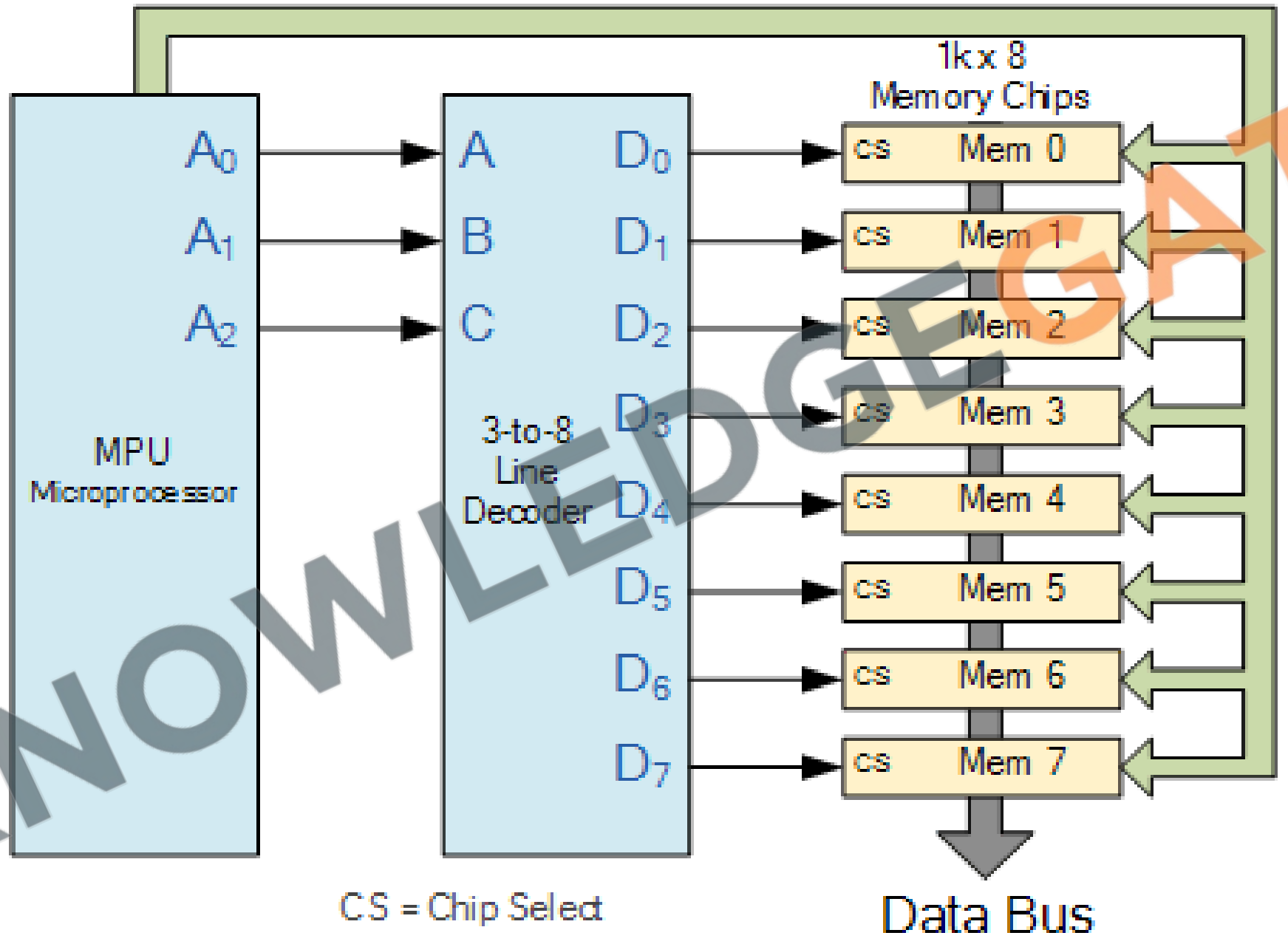
Decoder

- Decoders are also combinational circuits logically we can say a DeMux can be converted into a decoder by setting input line as enable line and selection line as input lines.



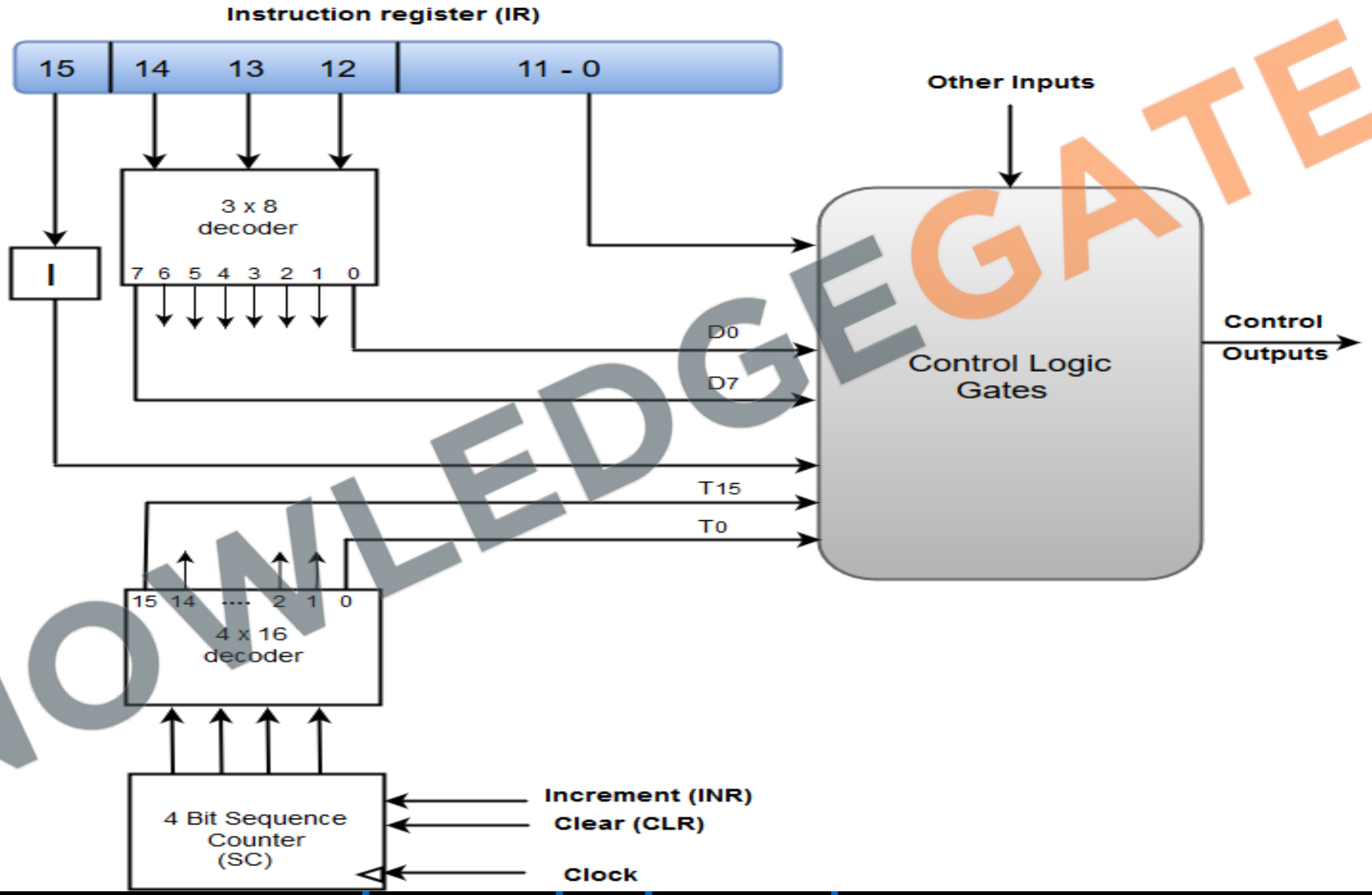


KNOWLEDGEGATE



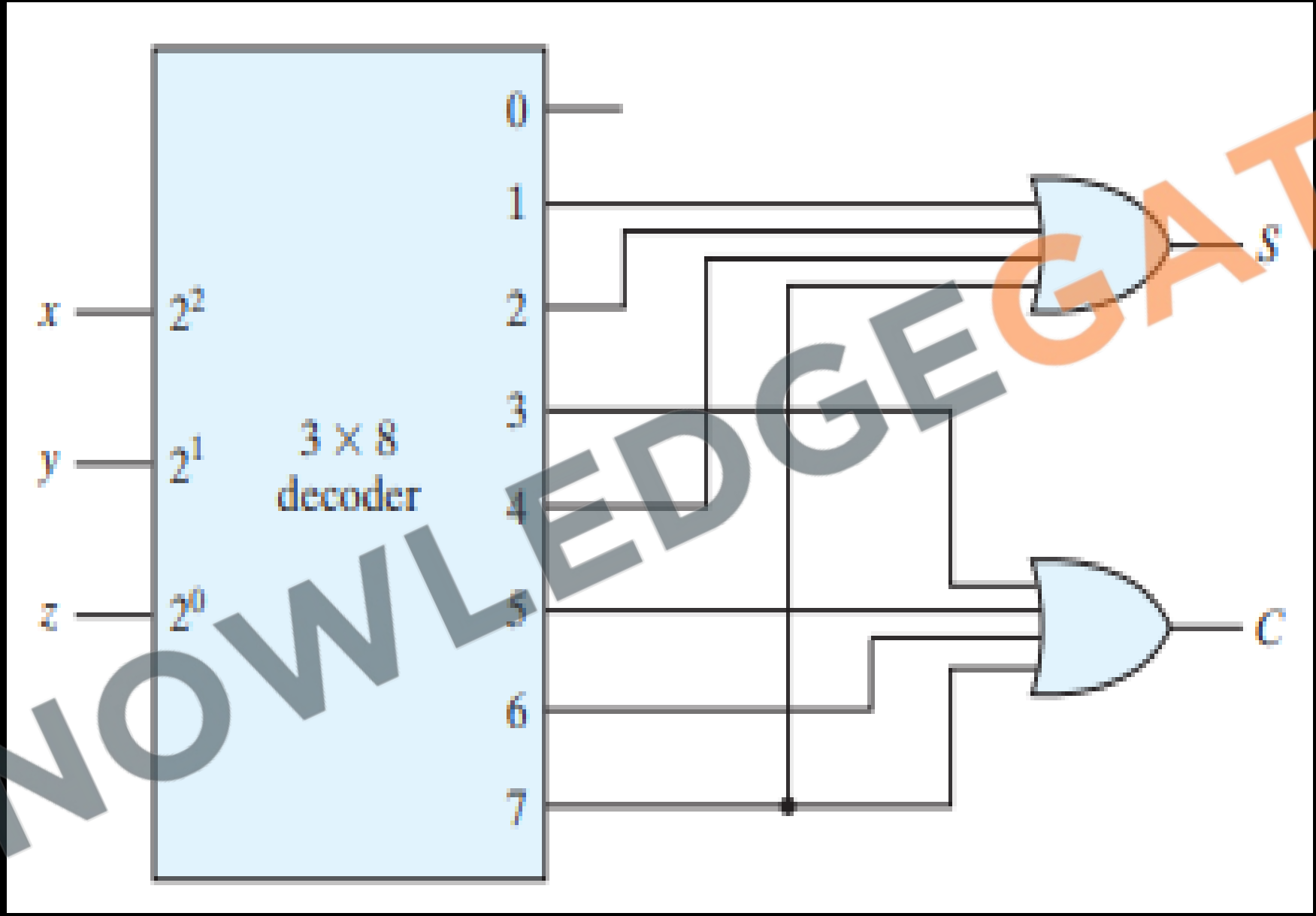
KNOWLEDGEGATE

Control Unit of a Basic Computer:



KNOWLEDGEGATE

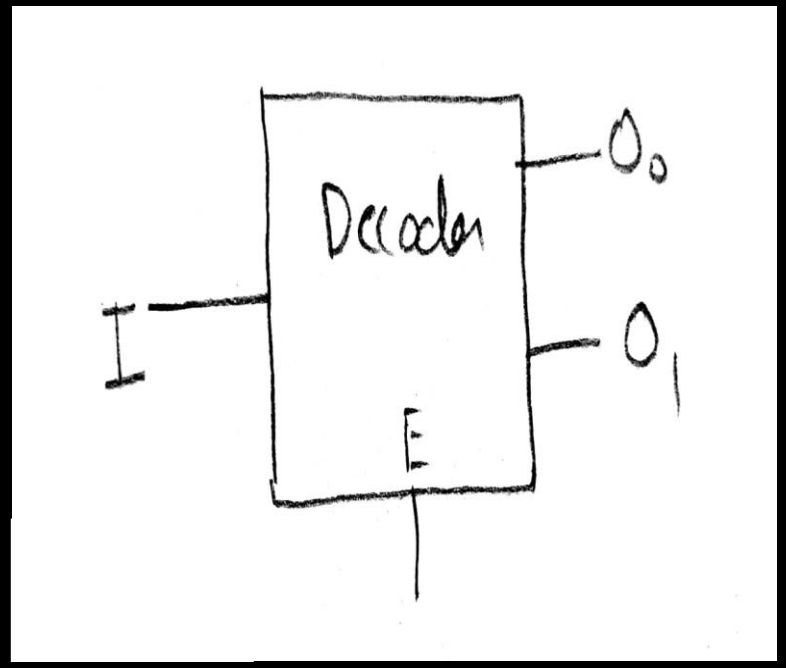
www.knowledgegate.in



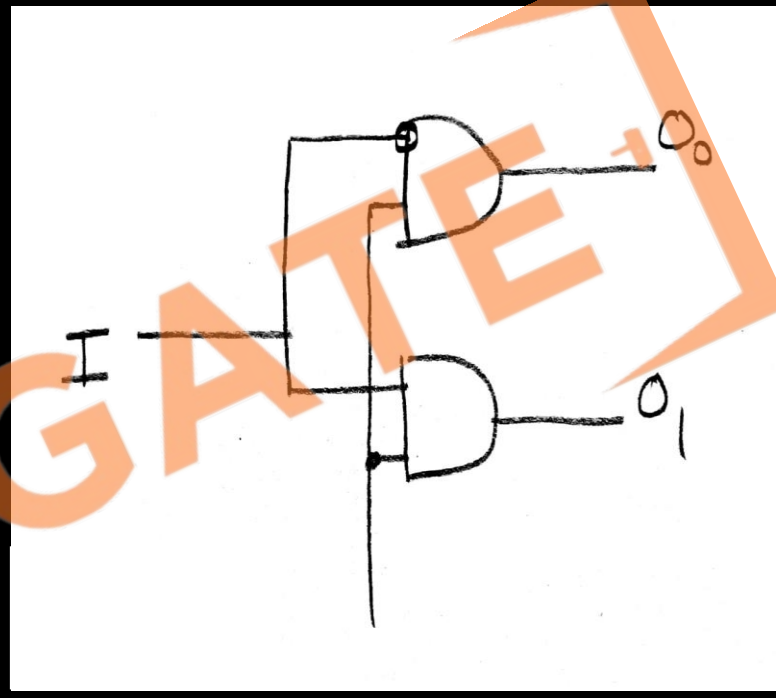
KNOWLEDGEGATE

www.knowledgegate.in

1-to-2 Decoder

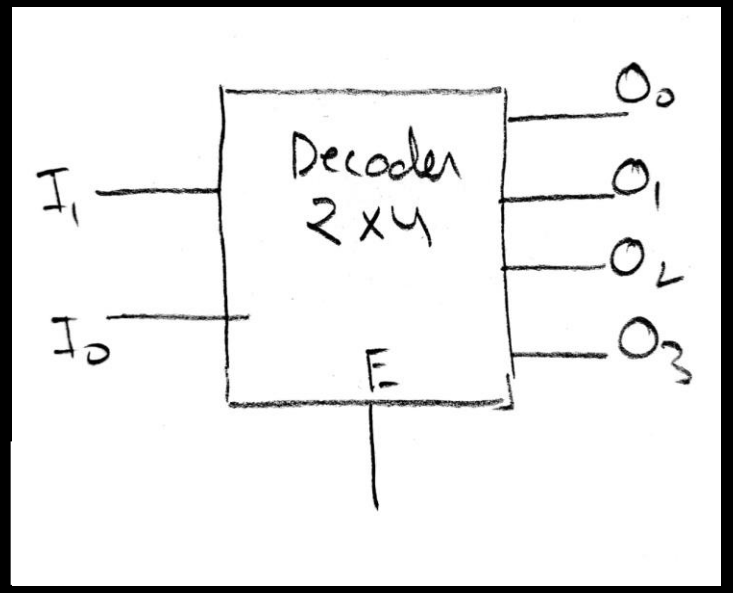


I	O_1	O_0
0	0	1
1	1	0

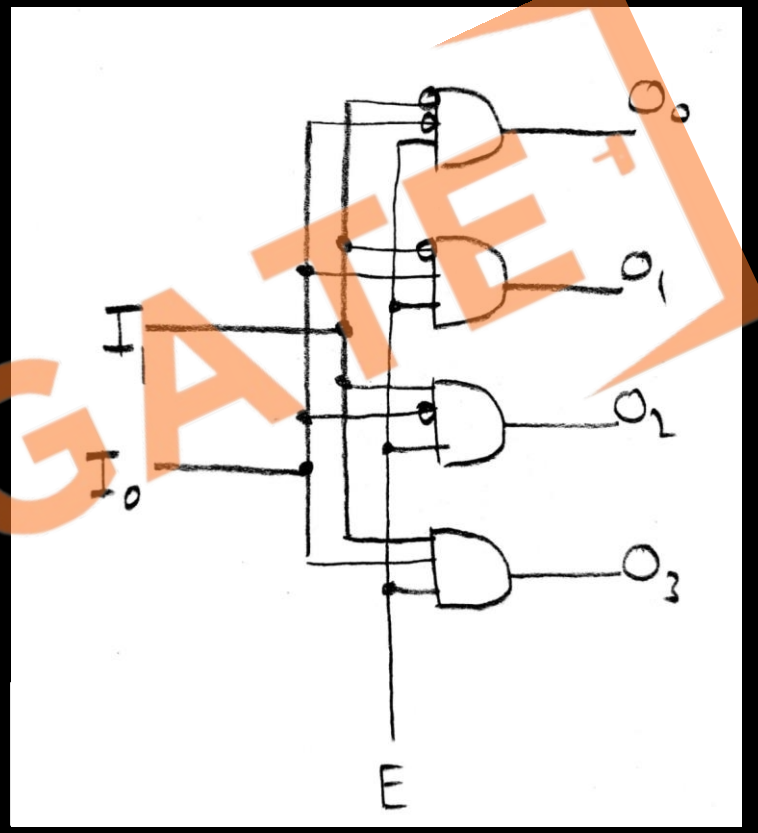


KNOWLEDGEGATE

2-to-4 Decoder



Input		Output			
I_1	I_0	O_3	O_2	O_1	O_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



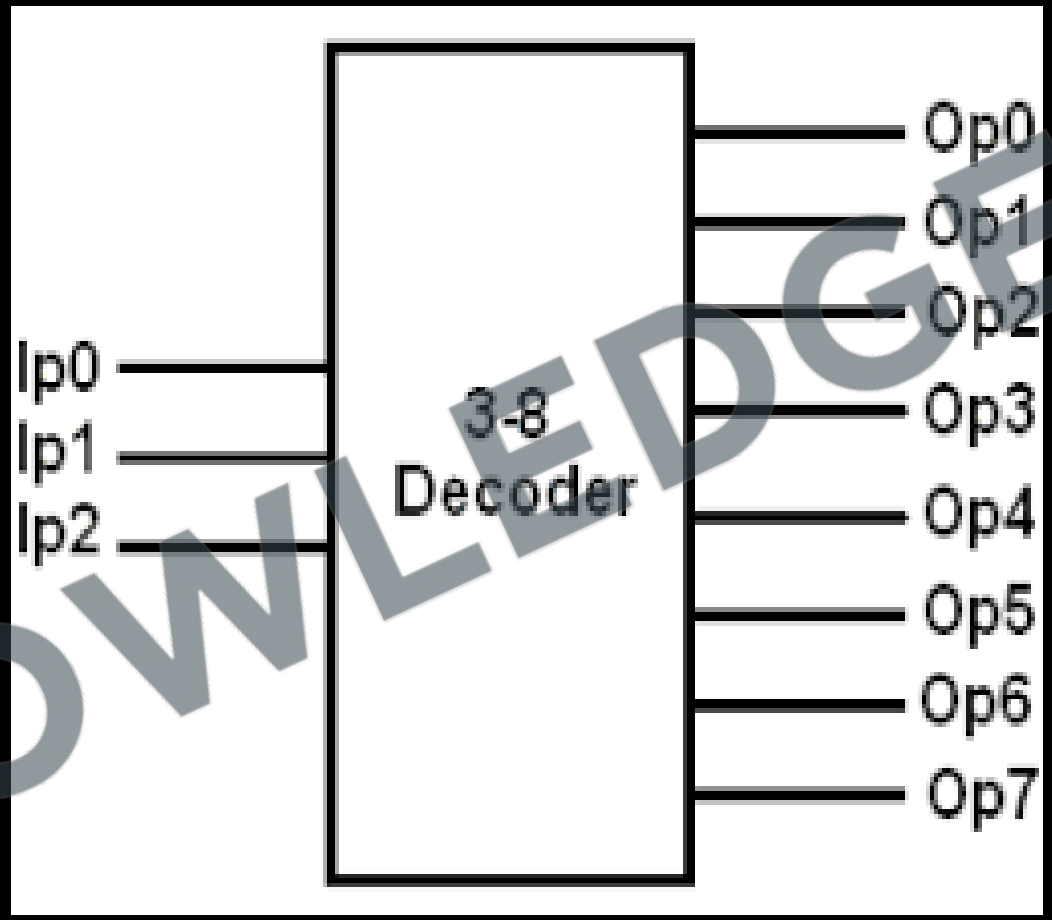
KNOWLEDGEGATE

Q Implementation of a full adder with a decoder

From the truth table of the full adder, we obtain the functions for the combinational circuit in sum-of-minterms form:

S(x, y, z) = ∑ (1, 2, 4, 7)

C(x, y, z) = ∑ (3, 5, 6, 7)



KNOWLEDGEGATE

www.knowledgegate.in

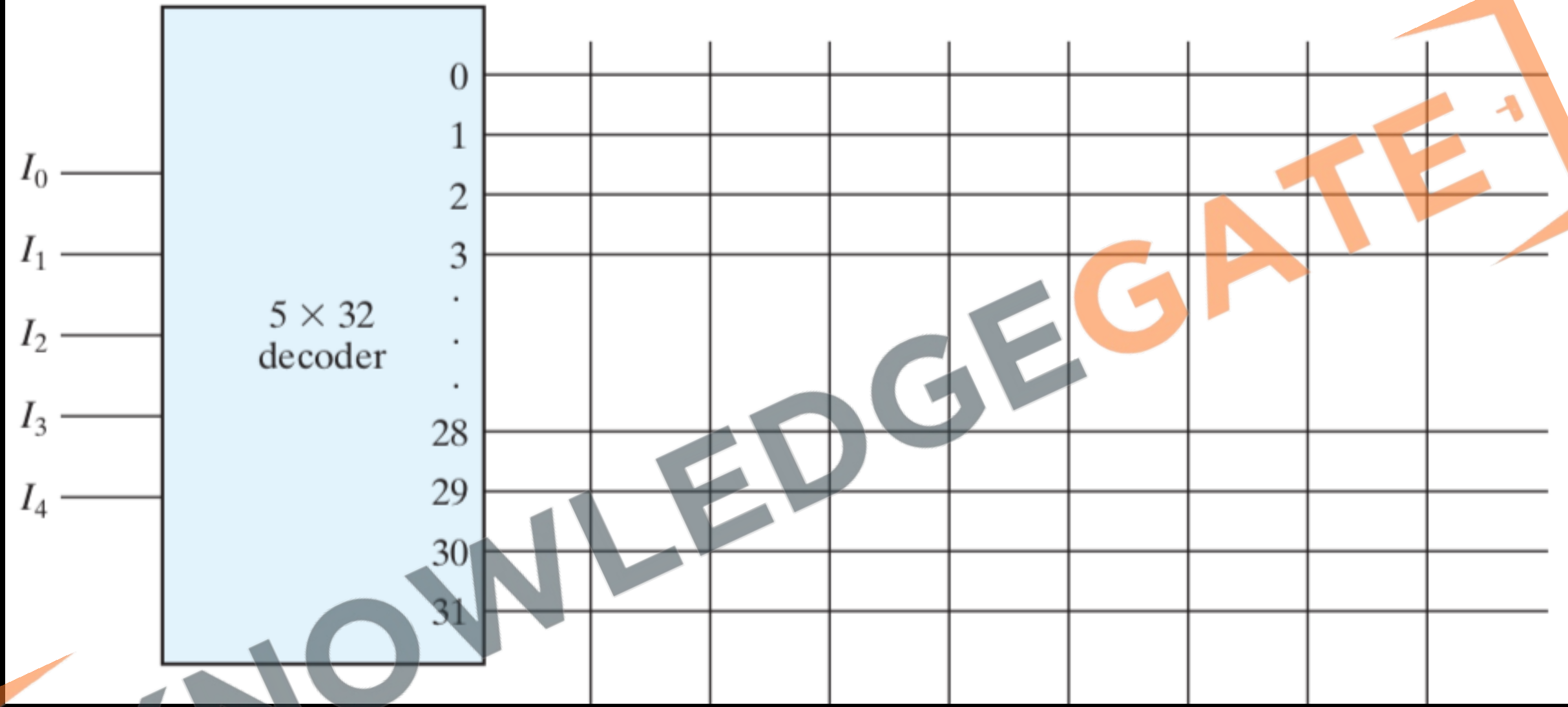
Combinational Logic Implementation

- Any combinational circuit with n inputs and m outputs can be implemented with an n -to-2^n -line decoder and OR gates.

- The procedure for implementing a combinational circuit by means of a decoder and OR gates requires that the Boolean function for the circuit be expressed as a sum of minterms.

- A decoder is then chosen that generates all the minterms of the input variables. The inputs to each OR gate are selected from the decoder outputs according to the list of minterms of each function.

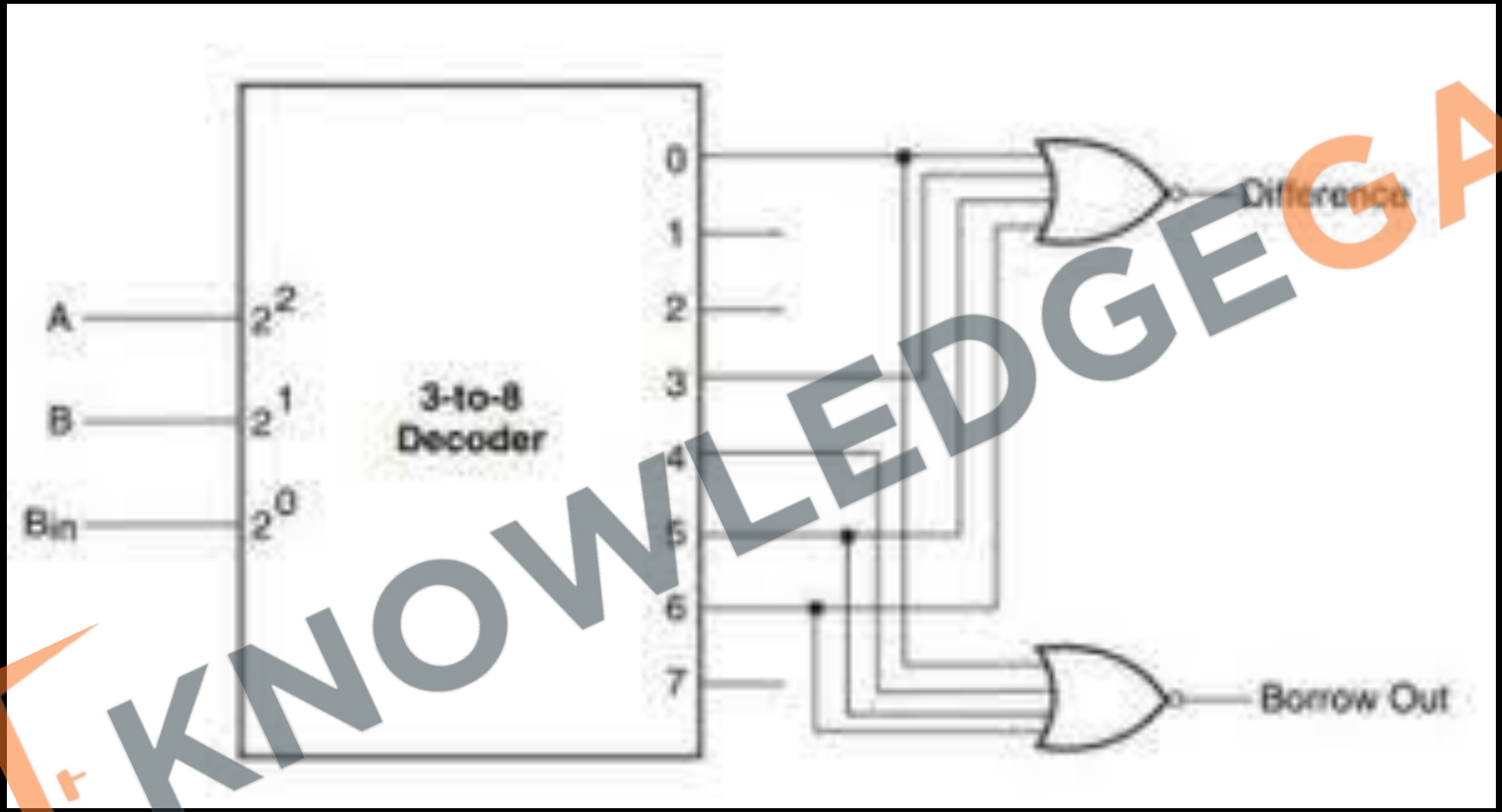
KNOWLEDGEGATE



KNOWLEDGEGATE

www.knowledgegate.in

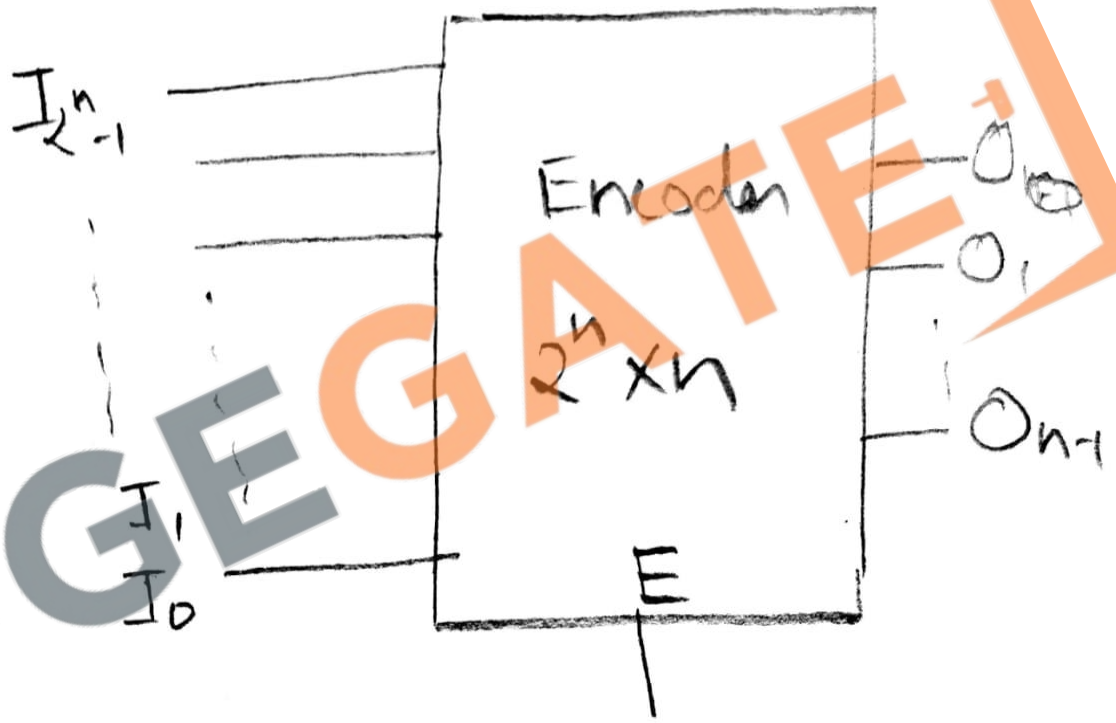
Q A full Subtractor using Decoder?



KNOWLEDGEGATE

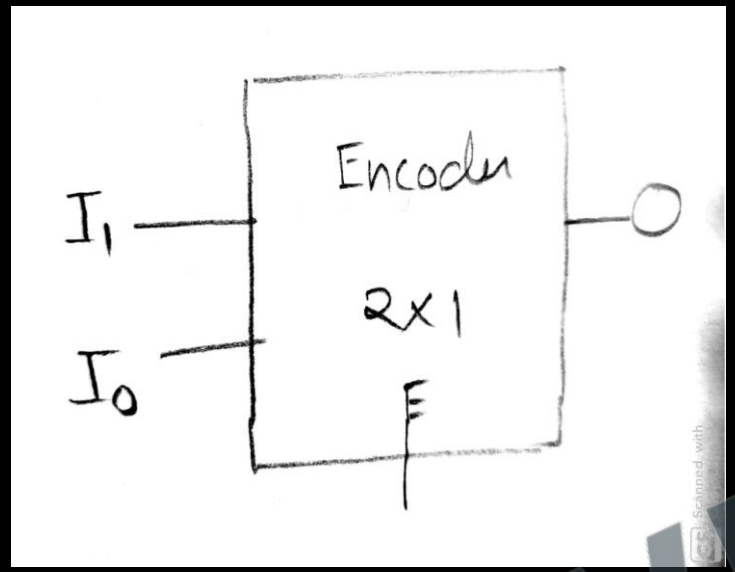
Encoder

- An encoder is a combinational circuit that encode binary information form one of a 2^N input lines and encode it into N output lines, which represent N bit code for the input.
- For simple encoders, it is assumed that only one input line is active at a time.
- Encoder performs the inverse operation of a decoder.



KNOWLEDGEGATE

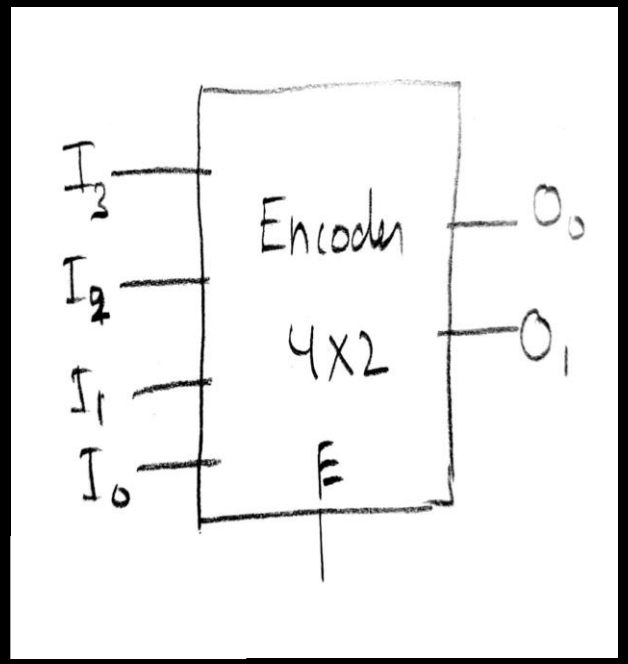
2-to-1 Encoder



I_1	I_0	O_0
0	1	0
1	0	1

KNOWLEDGEGATE

4-to-2 Encoder



I_3	I_2	I_1	I_0	O_1	O_0
0	0	0	1	**0**	**0**
0	0	1	0	**0**	**1**
0	1	0	0	**1**	**0**
1	0	0	0	**1**	**1**

- $O_0 = I_0' I_1' (I_2 \oplus I_3)$
- $O_1 = I_0' I_2' (I_1 \oplus I_3)$

KNOWLEDGEGATE

Priority Encoder

- In some practical cases more than one input can be high at a time, there we can not use simple encoder. In a priority encoder more than one input can be high at a time. A priority encoder is an encoder circuit that includes the priority function.
- The operation of the priority encoder is such that if two or more inputs are equal to 1 at the same time, the input having the highest priority will take precedence.

KNOWLEDGEGATE

- They are often used to control interrupt requests by acting on the highest priority interrupt input. The *priority encoders* output corresponds to the currently active input which has the highest priority. So when an input with a higher priority is present, all other inputs with a lower priority will be ignored.

- $I_3 > I_2 > I_1 > I_0$
- $O_0 = I_3 + I_3' I_2' I_1$
- $O_1 = I_2 + I_3$

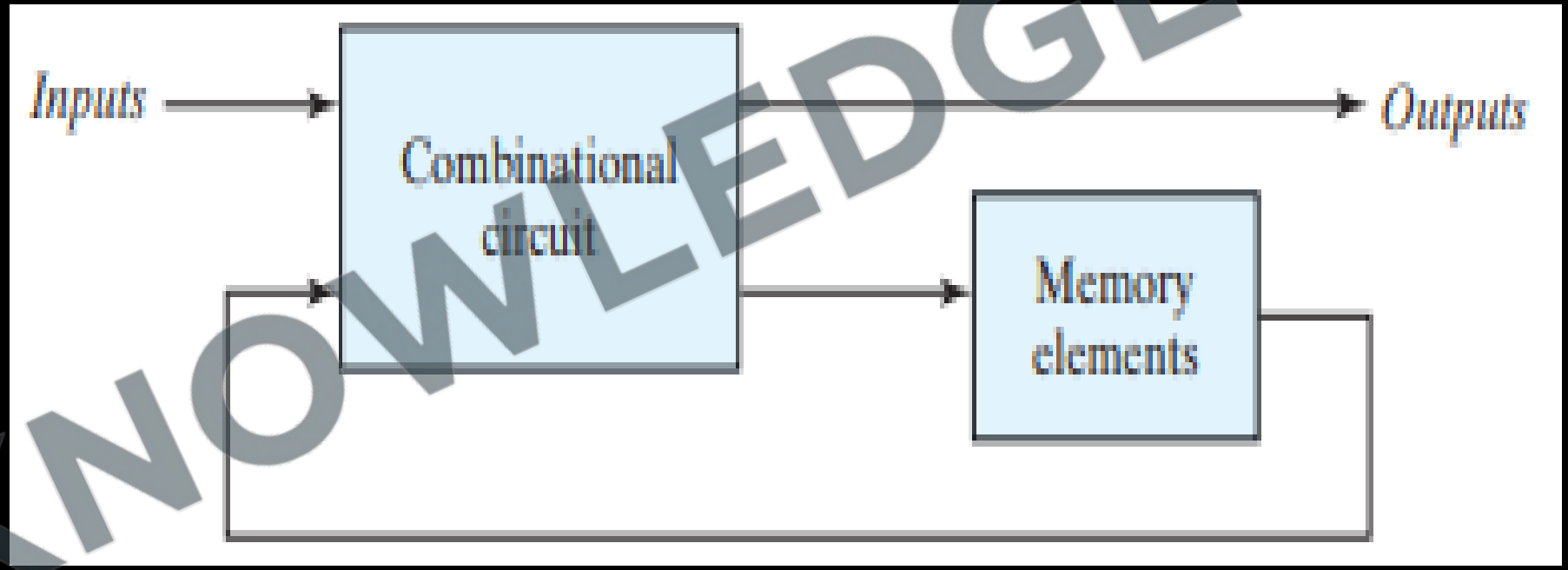
I_3	I_2	I_1	I_0	O_1	O_0
0	0	0	1	**0**	**0**
0	0	1	d	**0**	**1**
0	1	d	d	**1**	**0**
1	d	d	d	**1**	**1**

KNOWLEDGE GATE

www.knowledgegate.in

SEQUENTIAL CIRCUITS

- Sequential Circuits consists of a combinational circuit to which memory elements are connected to form a feedback path. The memory elements are devices capable of storing binary information.
- The binary information stored in these elements at any given time defines the state of the sequential circuit at that time.

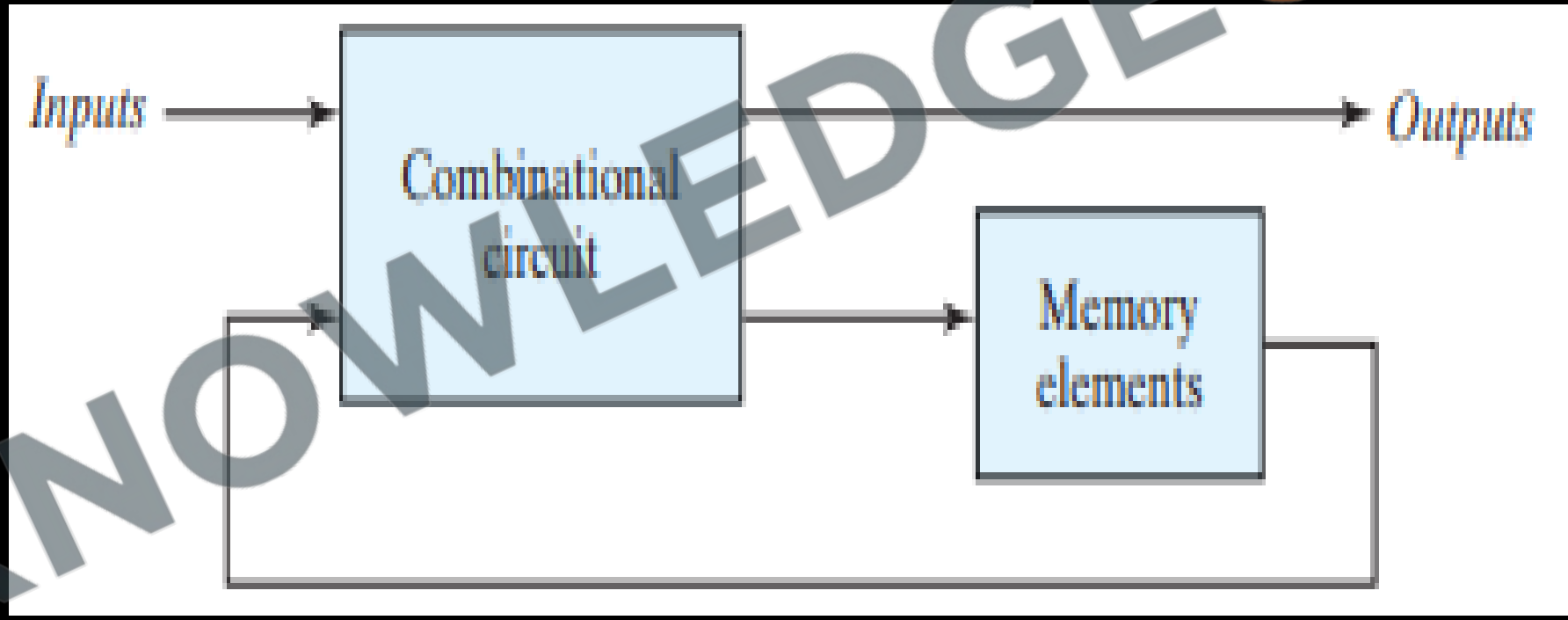


KNOWLEDGEGATE
www.knowledgegate.in

- The sequential circuit receives binary information from external inputs (x_n) that, together with the present state (y_{n-1}) of the memory elements, determine the binary value of the outputs(y_n).

- A sequential circuit is specified by a time sequence of inputs, outputs, and internal states.

$$f(x_n, y_{n-1}) = y_n$$



KNOWLEDGEGATE
www.knowledgegate.in

Latches

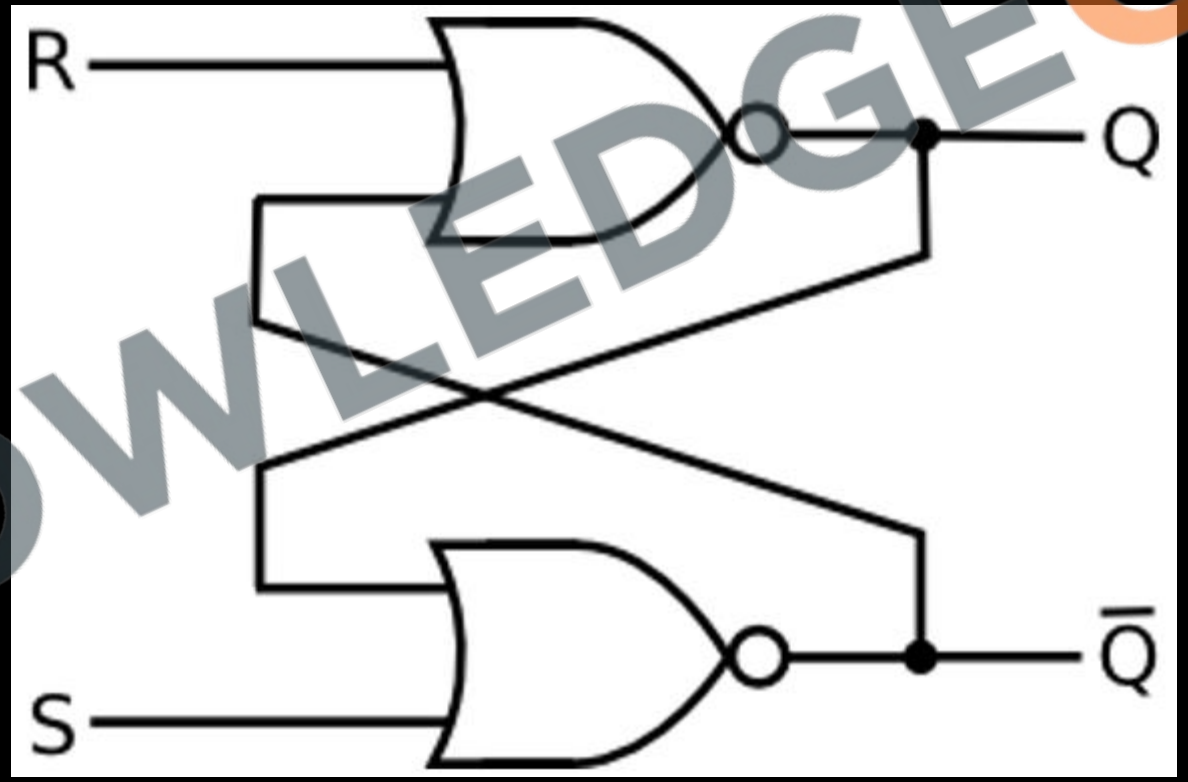
- Latch means to hold something or something which do not change.
- latches are the basic building blocks of any flip flop and they are capable of holding 1 bit until necessary.
- Storage elements that operate with signal levels are referred to as latches.
- Latches are level sensitive devices.

KNOWLEDGEGATE

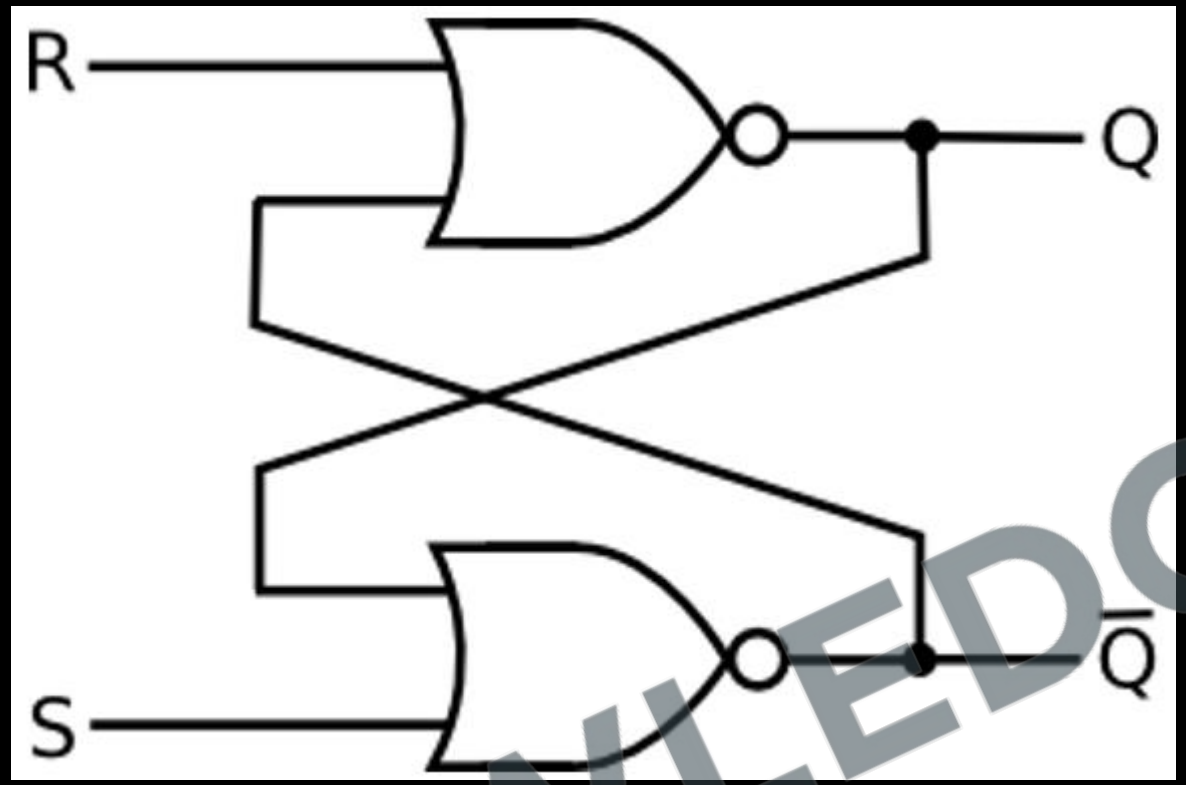
NOR Latch

- The SR latch is a circuit with two cross-coupled NOR gates or two cross-coupled NAND gates, and two inputs labeled S for set and R for reset.

- Outputs Q_n and Q_n' are the complement of each other, in valid scenario.

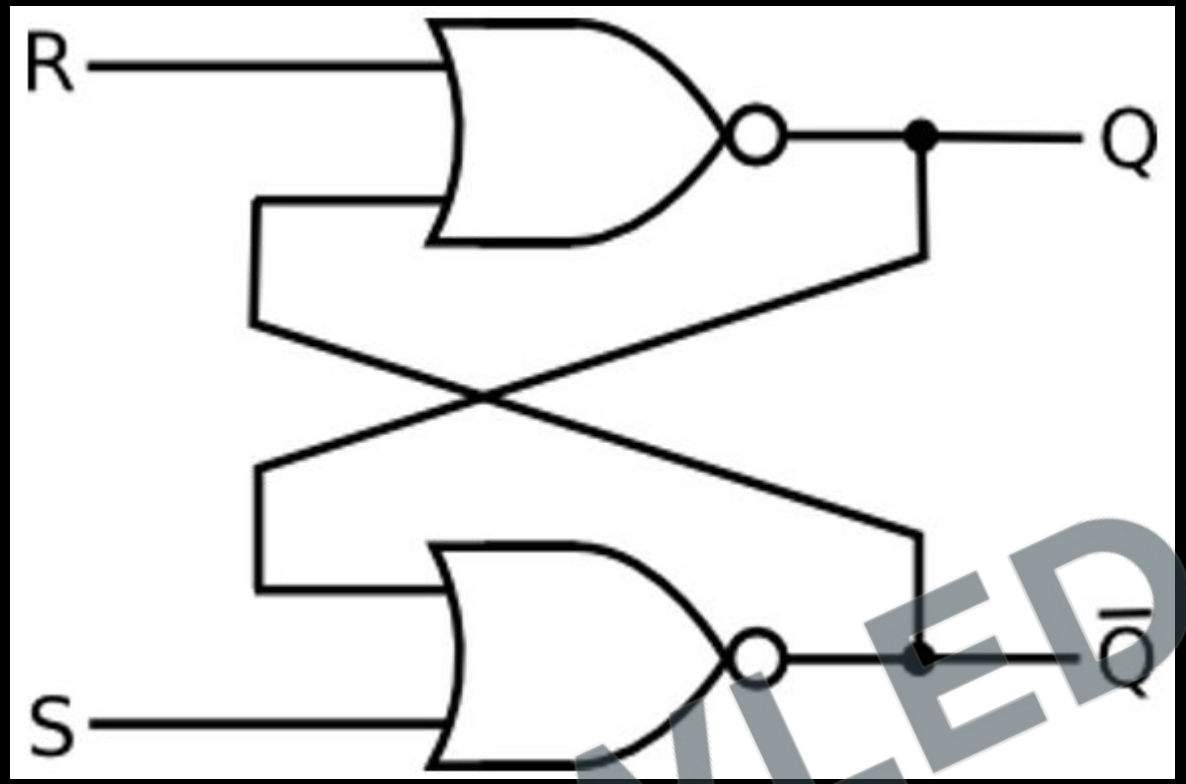


KNOWLEDGEGATE
www.knowledgegate.in



S	R	Q_n	Q_{n+1}
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

KNOWLEDGEGATE
www.knowledgegate.in

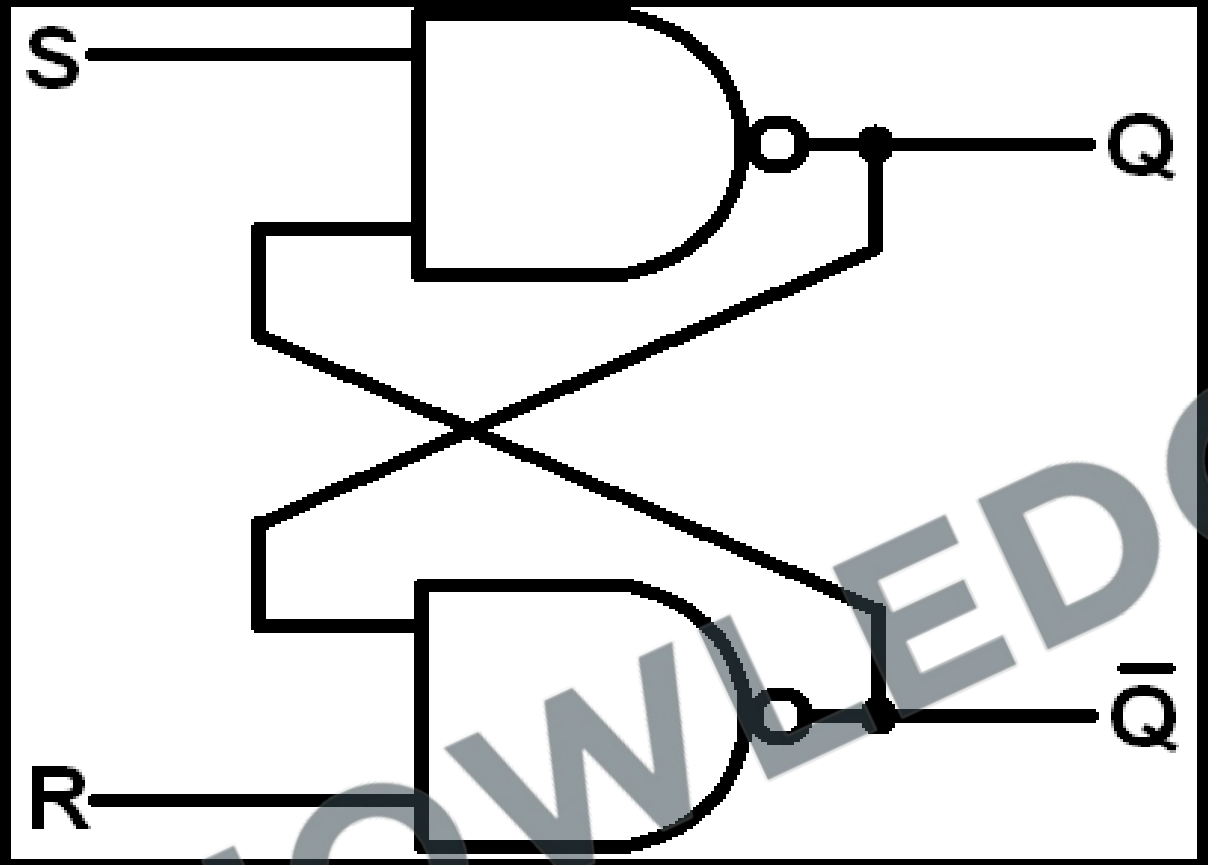


S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

KNOWLEDGEGATE

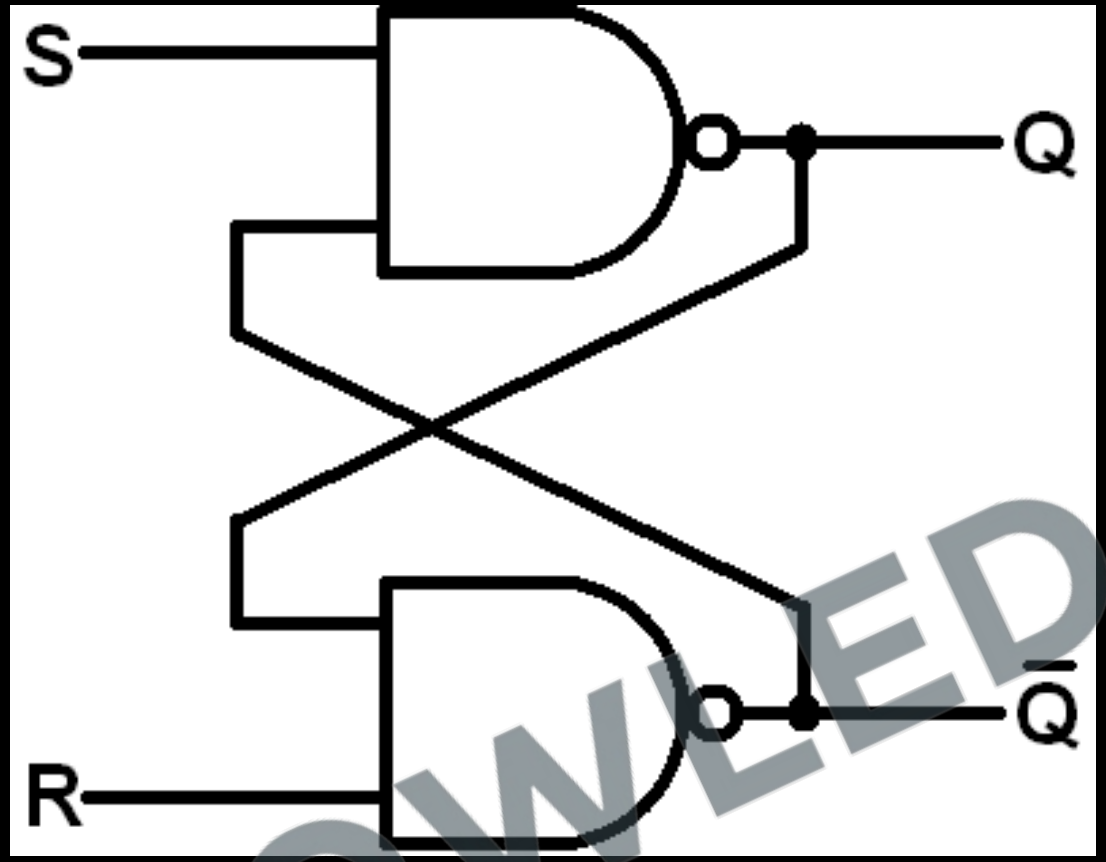
www.knowledgegate.in

NAND Latch



S	R	Q_n	Q_{n+1}
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

KNOWLEDGEGATE



S	R	Q_n	Q_{n+1}
0	0	0	X
0	0	1	X
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

KNOWLEDGEGATE

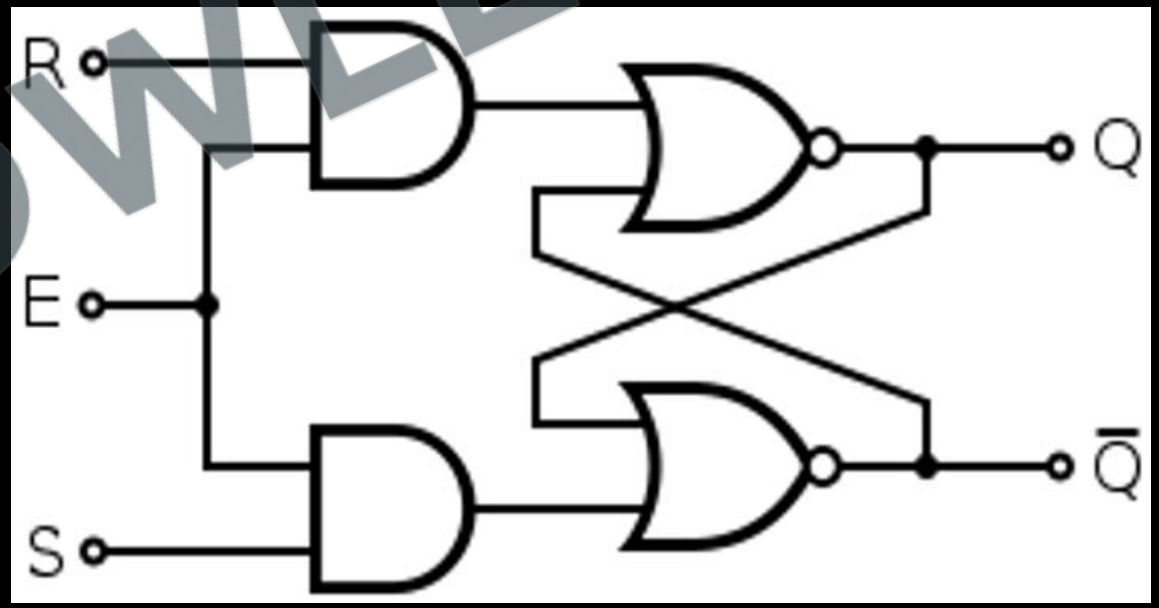
Notes

- Latches are the basic circuits from which all flip-flops are constructed.
- The storage elements (memory) used in clocked sequential circuits are called flipflops.
- A flip-flop is a binary storage device capable of storing one bit of information. In a stable state, the output of a flip-flop is either 0 or 1 (it is called bi-stable multi-vibrator).
- A flip-flop is said to be stable if it has complementary behavior.
- Latches are level sensitive devices; flip-flops are edge-sensitive devices.
- Storage elements that operate with signal levels are referred to as latches; those controlled by a clock transition are flip-flops.

KNOWLEDGEGATE

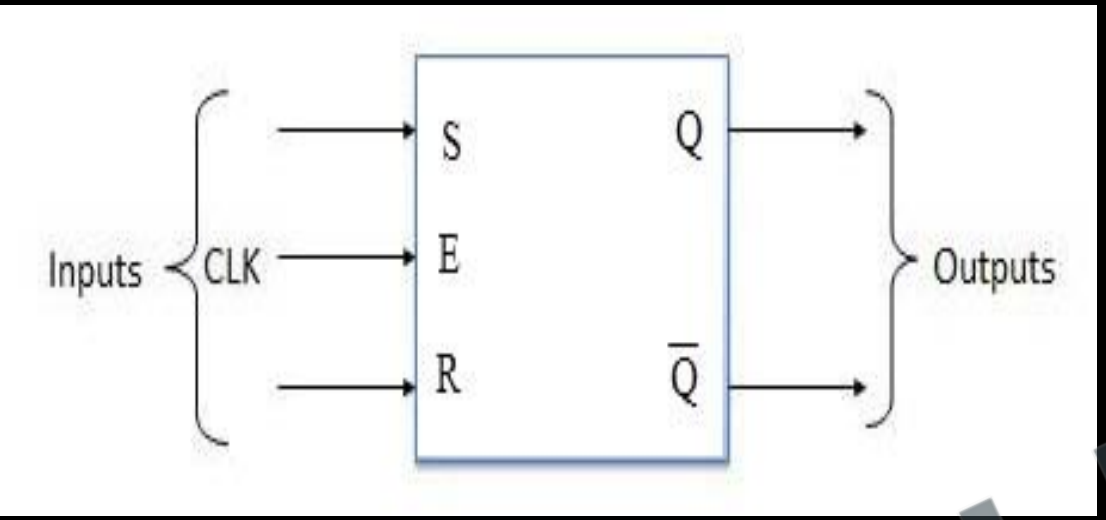
SR flip flop

1. The operation of the basic SR latch can be modified by providing an additional input signal that determines (controls), when the state of the latch can be changed by determining whether S and R (or S and R) can affect the circuit.
2. It consists of the basic SR latch and two additional AND gates. The control input CP / E_n acts as an enable signal for the other two inputs.
3. The outputs of the AND gates stay at the 0 as long as the enable signal remains at 0 as one input of AND gate gets 0 resulting in 0 as output. When the enable input goes to 1, information from the S or R input is allowed to affect the latch.

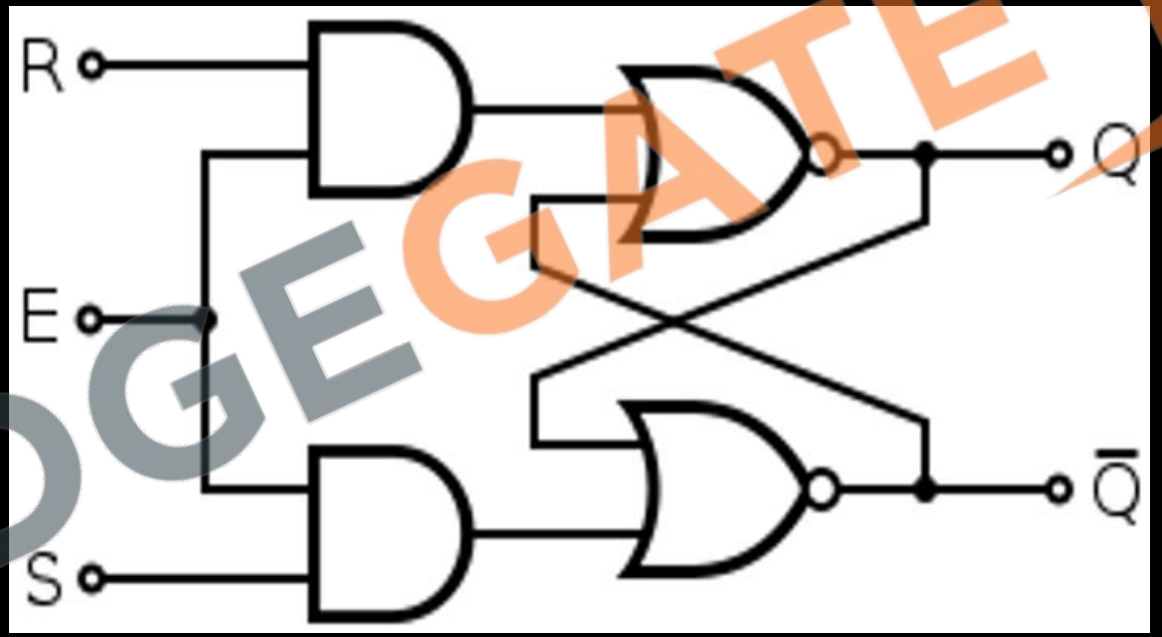


KNOWLEDGEGATE

Block Diagram

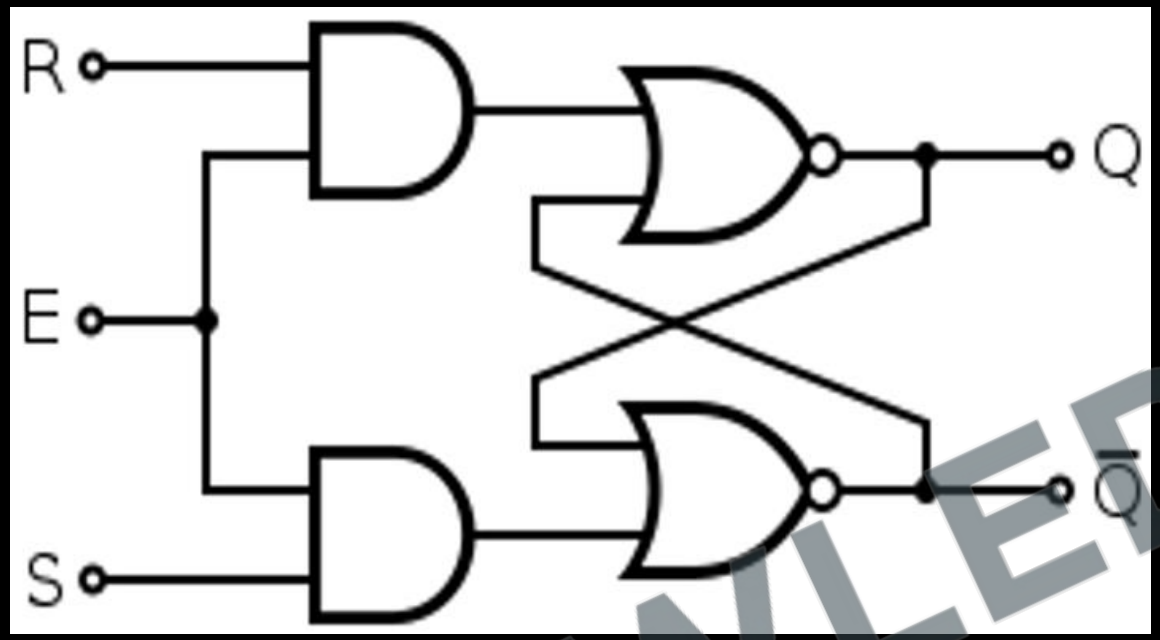


Implementation



KNOWLEDGEGATE

Implementation



Truth Table

S	R	Q_n	Q_{n+1}
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

KNOWLEDGEGATE

Truth Table

S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

K-Map

	sr	s'r'	s'r	sr	sr'
q		00	01	11	10
q'	00				
q	01				

Characteristics Equation

Q_{n+1} =

KNOWLEDGEGATE

Truth Table

S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

Characteristics Equation

$$Q_{n+1} = S + R' Q_n$$

Function Table

S	R	Q_{n+1}
0	0	
0	1	
1	0	
1	1	

KNOWLEDGEGATE

Truth Table

S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

Excitation Table

Q_n	Q_{n+1}	S	R
0	0		
0	1		
1	0		
1	1		

KNOWLEDGEGATE

Truth Table

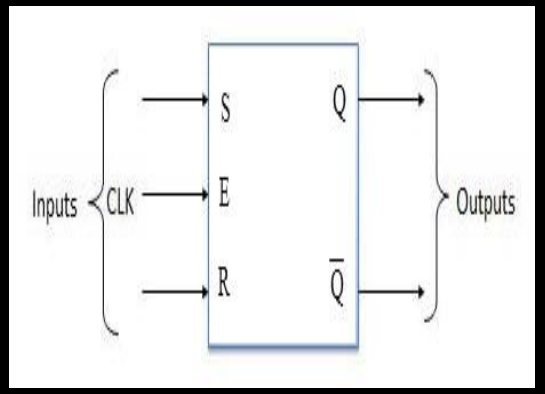
S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

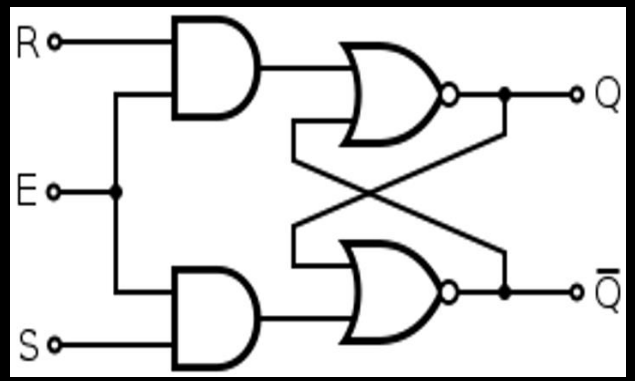
State Diagram

KNOWLEDGEGATE

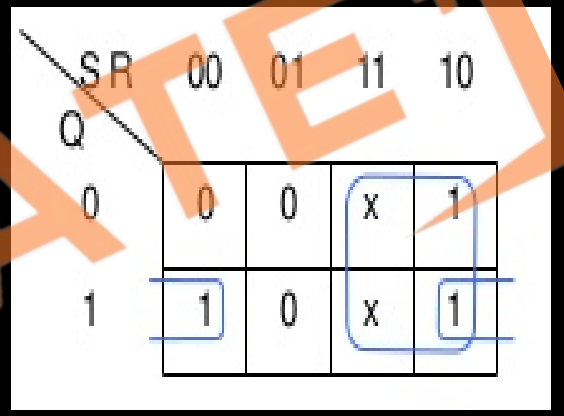
www.knowledgegate.in

Block Diagram	Implementation	Truth Table	K-Map





S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

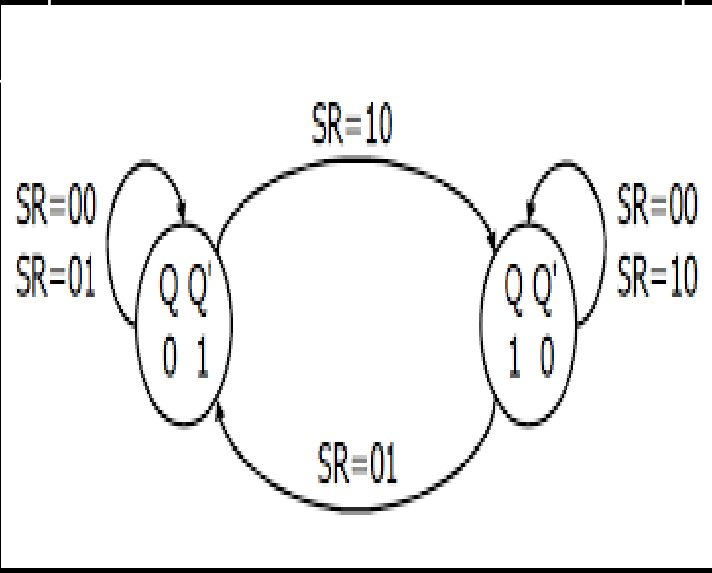


Characteristics Equation	Function Table	Excitation Table

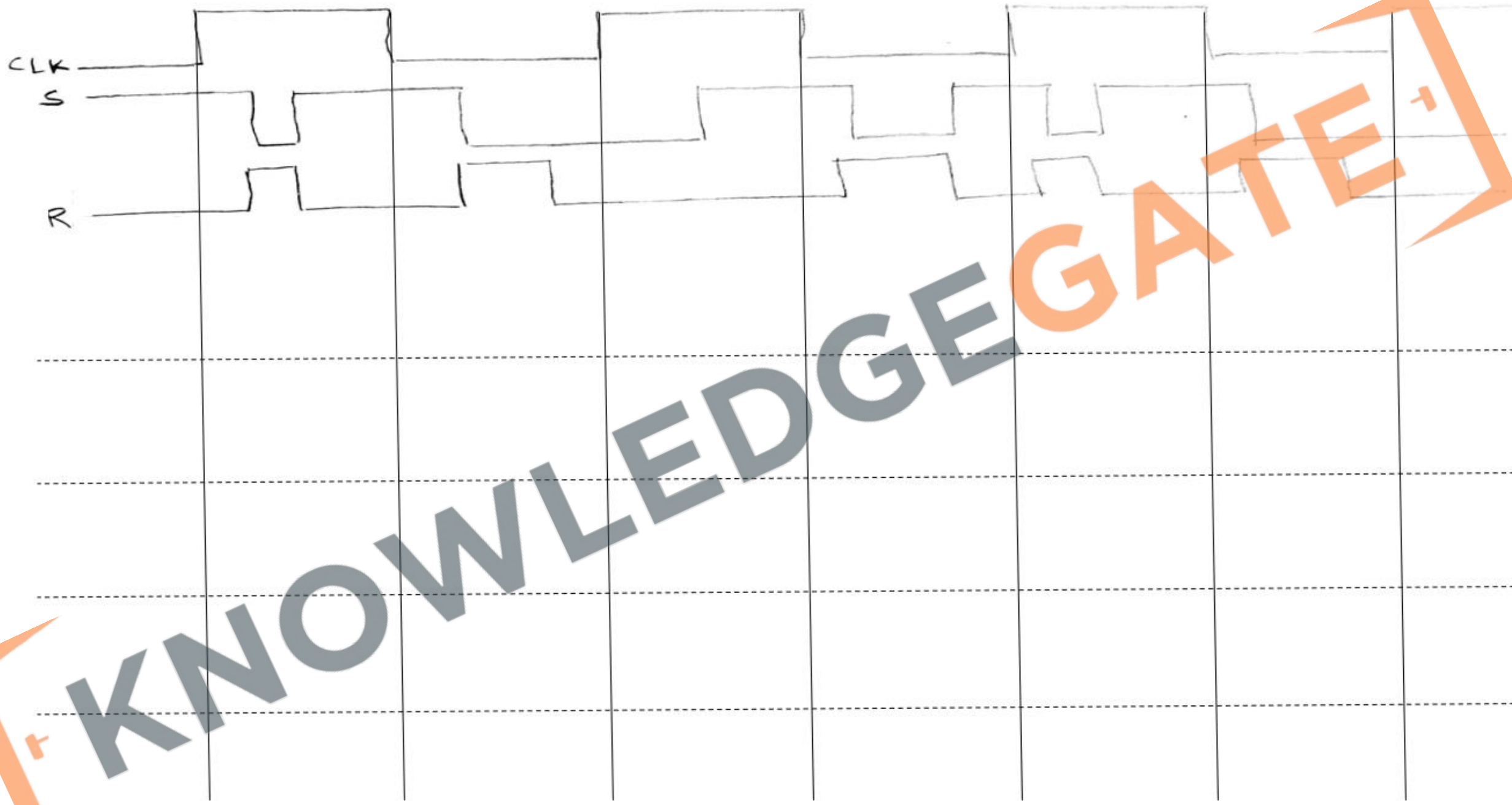
$$Q_{n+1} = S + R' Q_n$$

S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	X

Q_n	Q_{n+1}	S	R
0	0	0	d
0	1	1	0
1	0	0	1
1	1	d	0



KNOWLEDGEGATE



KNOWLEDGEGATE

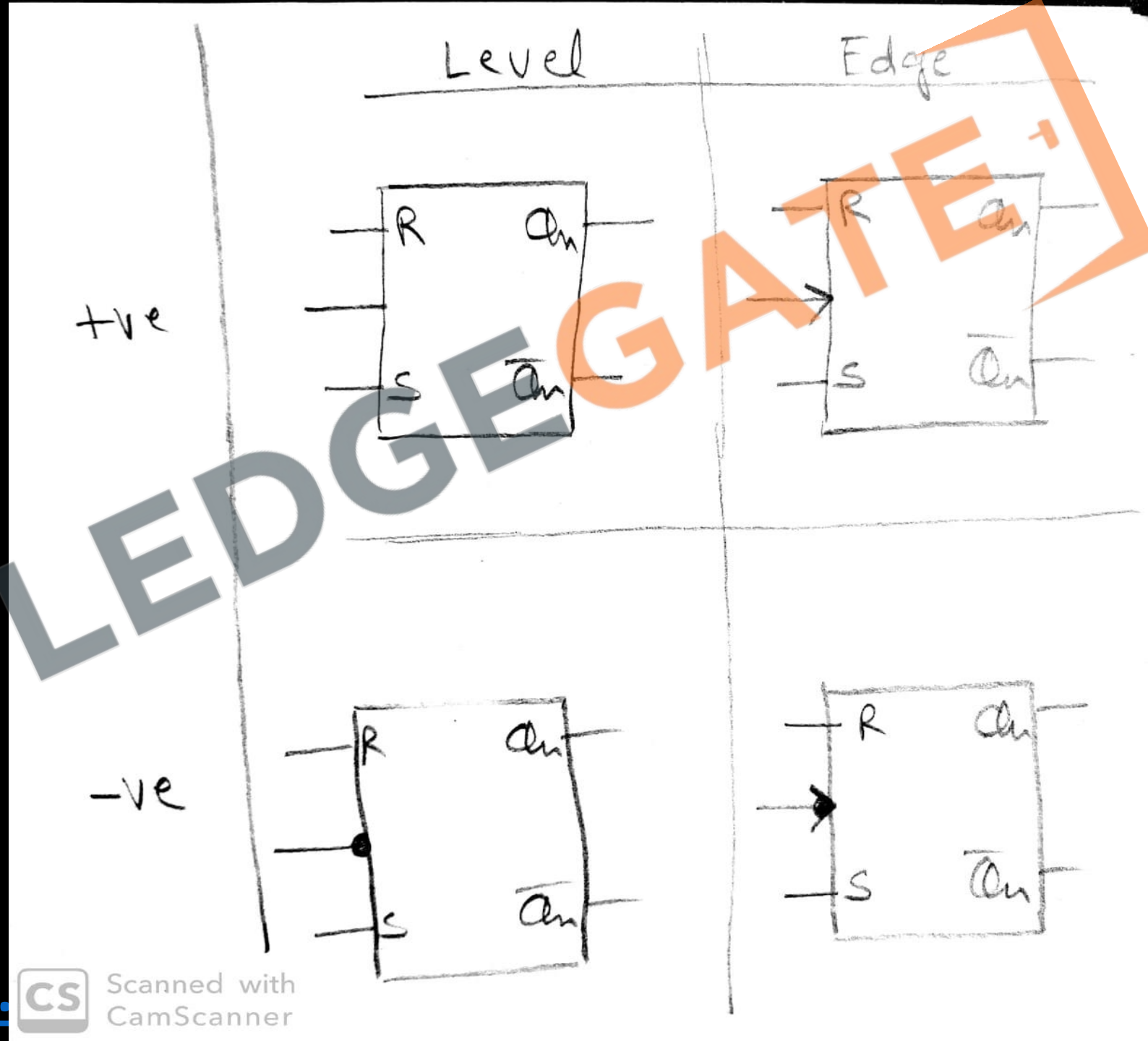
Order of Priority

-ve edge

+ve edge

-ve level

+ve level



KNOWLEDGEGATE
www.
Scanned with CamScanner

Notes

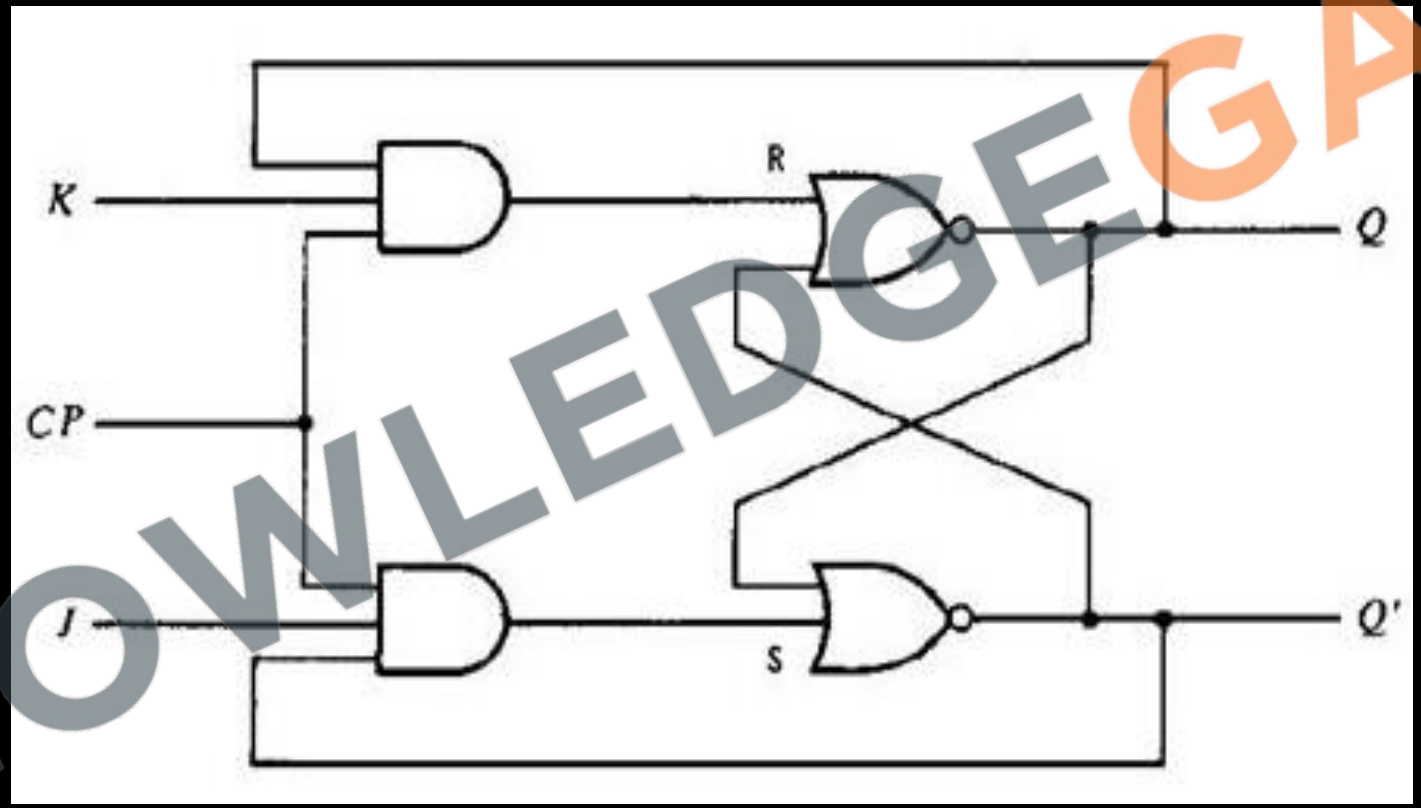
- Flip-flops can be either level-triggered (asynchronous, transparent or opaque) or edge-triggered (synchronous, or clocked).
- The term flip-flop has historically referred generically to both level-triggered and edge-triggered circuits that store a single bit of data using gates.
- Recently, some authors reserve the term *flip-flop* exclusively for discussing clocked circuits;
  - the simple ones are commonly called *transparent latches*. Using this terminology, a level-sensitive flip-flop is called a transparent latch
  - Whereas an edge-triggered flip-flop is simply called a flip-flop.
- Using either terminology, the term "flip-flop" refers to a device that stores a single bit of data. The terms "edge-triggered", and "level-triggered" may be used to avoid ambiguity.

KNOWLEDGE GATE

www.knowledgegate.in

JK flip flop

- SR flip flop when both S and R = 1, results in invalid output. To resolve the problem we use JK Flip Flop. We take a feedback from the outputs.

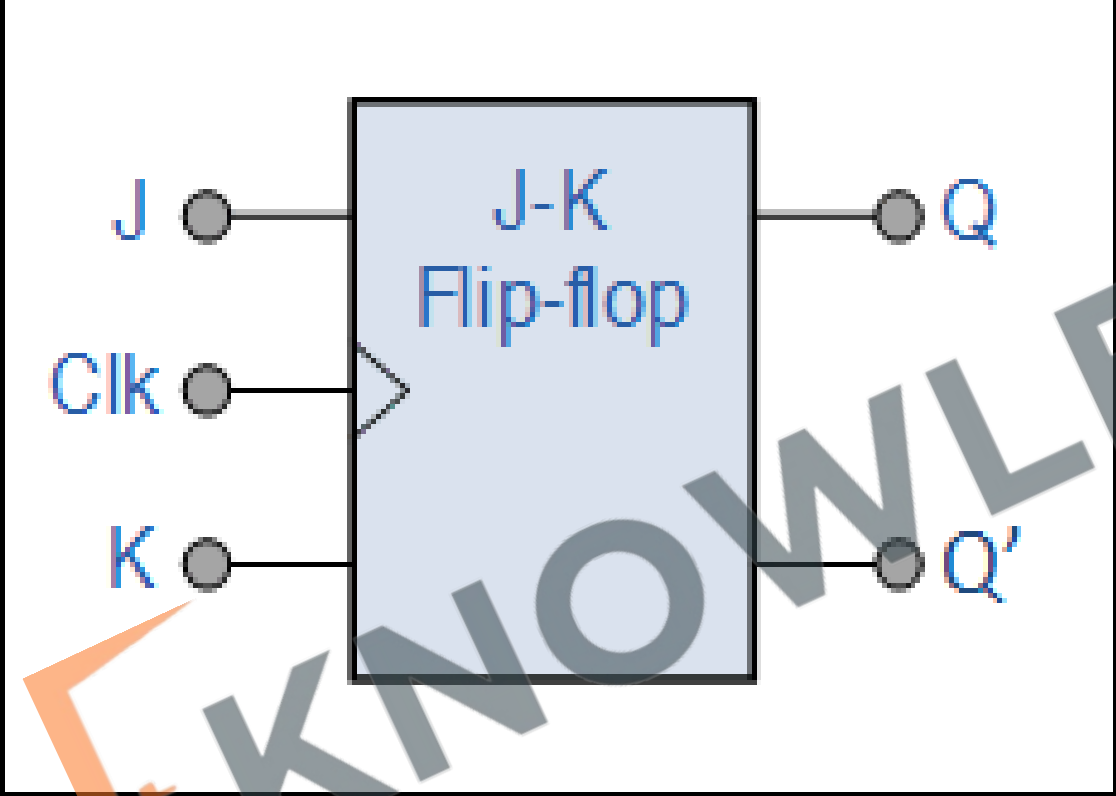


KNOWLEDGEGATE

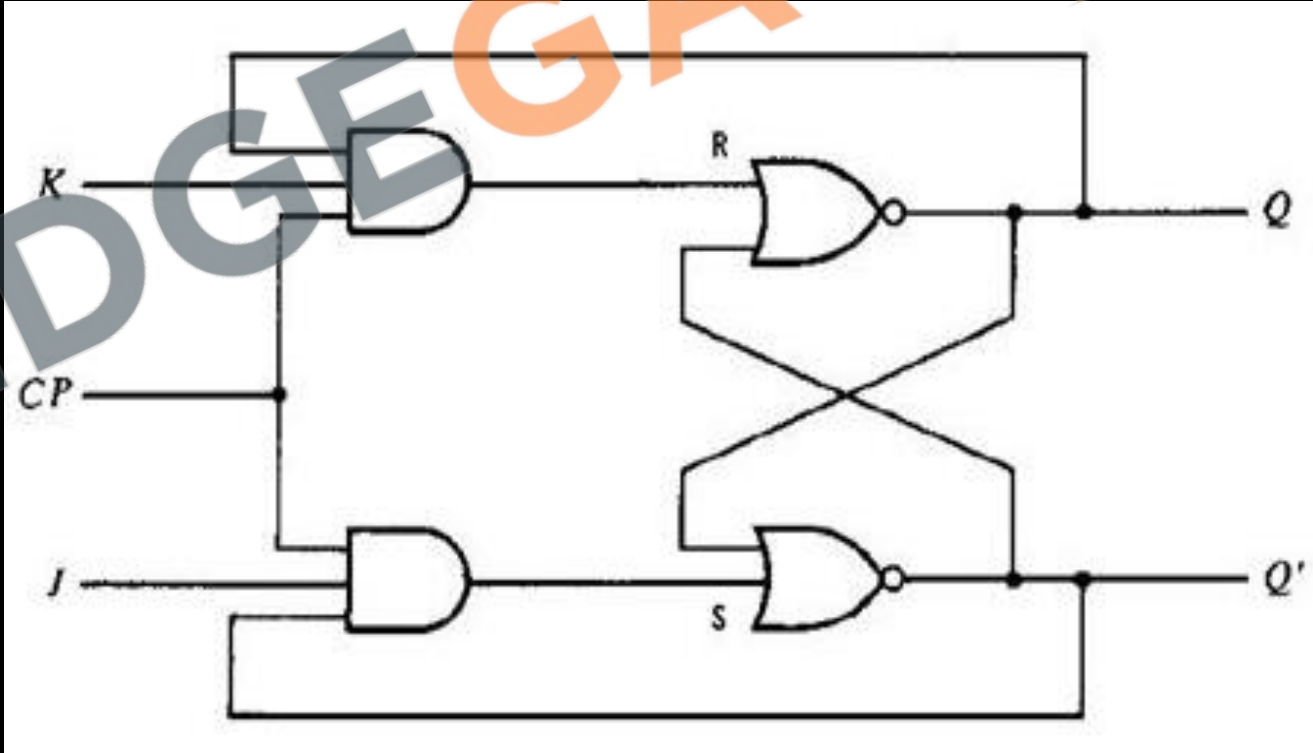
JK flip flop

- SR flip flop when both S and R = 1, results in invalid output. To resolve the problem we use JK Flip Flop. We take a feedback from the outputs.

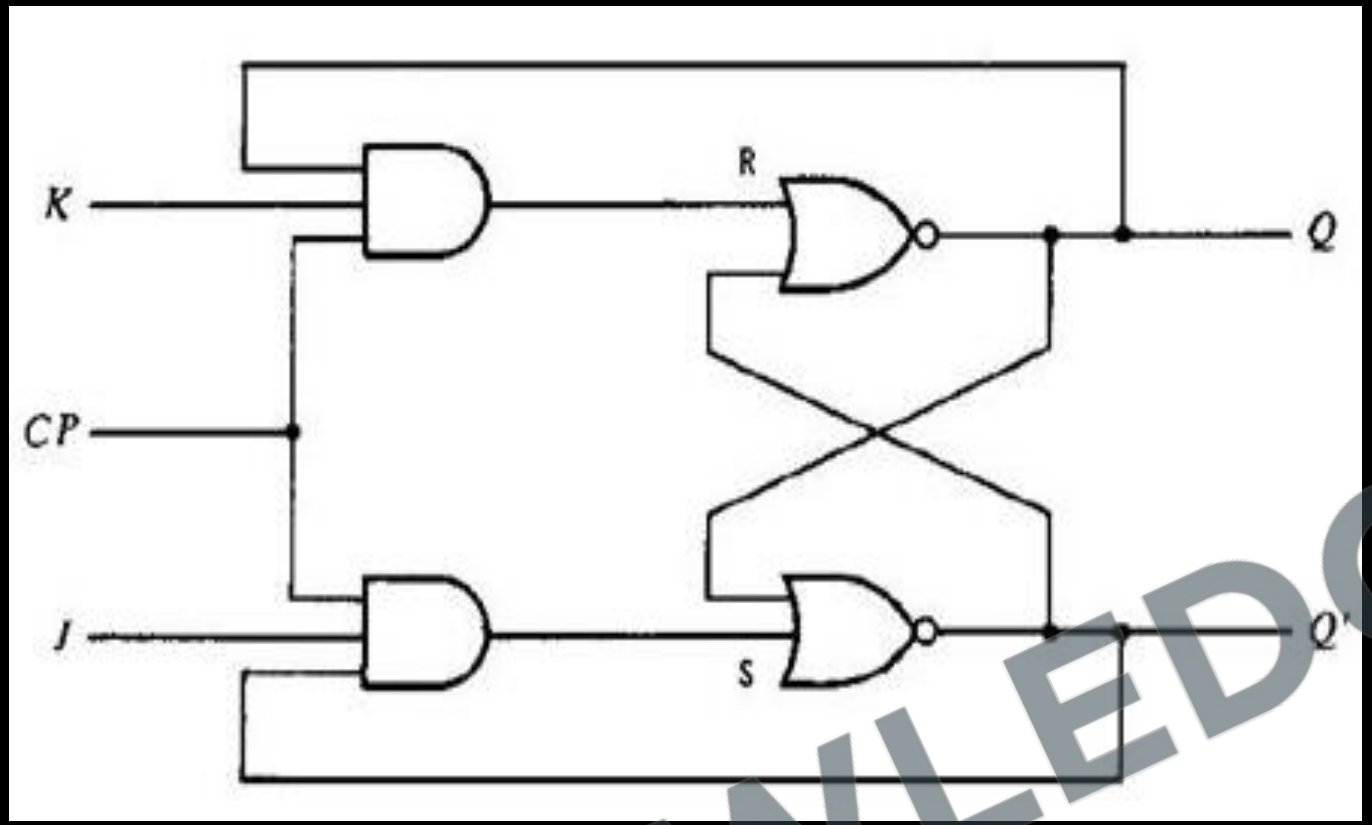
Block Diagram



Implementation



KNOWLEDGEGATE



Truth Table

J	K	Q_n	Q_{n+1}
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

KNOWLEDGEGATE

Truth Table

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

K-Map

	JK	J'K'	J'K	JK	JK'
q		00	01	11	10
q'	00				
q	01				

Characteristics Equation

Q_{n+1} =

KNOWLEDGE GATE

Truth Table

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Characteristics Equation

$$Q_{n+1} = J\,Q'_n + K'Q_n$$

Function Table

J	K	Q_{n+1}
0	0	
0	1	
1	0	
1	1	

KNOWLEDGEGATE

Truth Table

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Excitation Table

Q_n	Q_{n+1}	J	K
0	0		
0	1		
1	0		
1	1		

KNOWLEDGEGATE

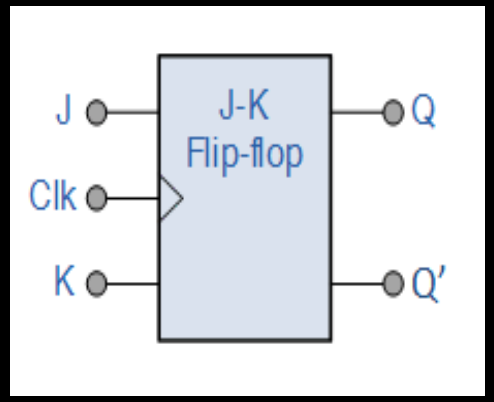
Truth Table

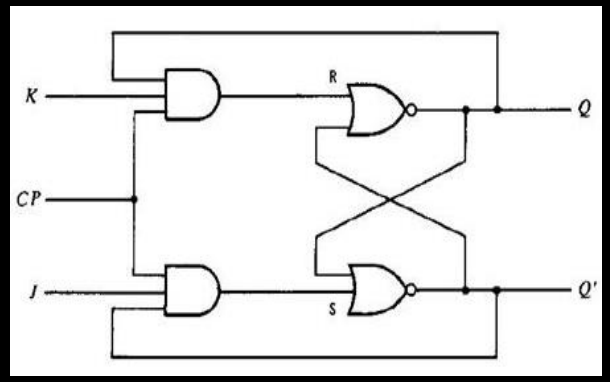
J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

State Diagram

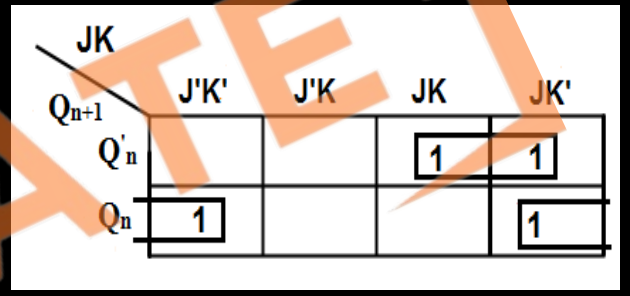
KNOWLEDGEGATE

Block Diagram	Implementation	Truth Table	K-Map





J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

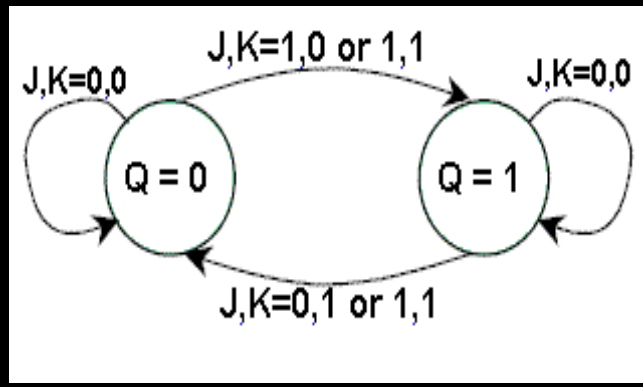


Characteristics Equation	Function Table	Excitation Table	State Diagram

$$Q_{n+1} = J\,Q'_n + K'Q_n$$

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	Q'_n

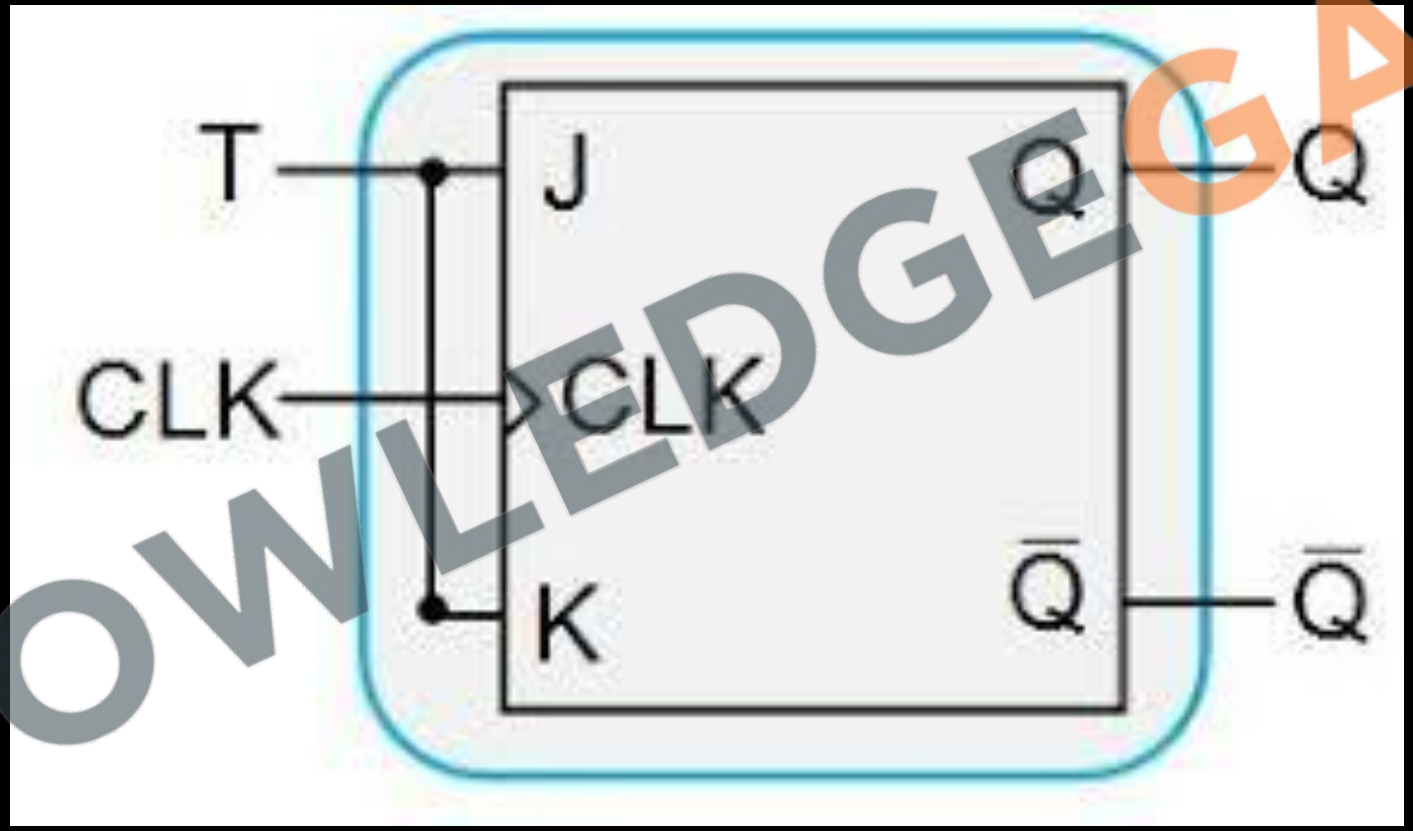
Q_n	Q_{n+1}	J	K
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0



KNOWLEDGEGATE

T(Toggle) flip flop

- The T (toggle) flip-flop is a complementing flip-flop and can be obtained from a JK flip-flop when inputs J and K are tied together.



KNOWLEDGEGATE

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Truth Table

T	Q_n	Q_{n+1}
0	0	
0	1	
1	0	
1	1	

KNOWLEDGE GATE

www.knowledgegate.in

Truth Table

T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

K-Map

	T	T’
q		0
q’	0	
q	1	

Characteristics Equation

Q_{n+1} =

KNOWLEDGE GATE

Truth Table

T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

Characteristics Equation

$$Q_{n+1} = T \oplus Q_n$$

Function Table

T	Q_{n+1}
0	
1	

KNOWLEDGEGATE

Truth Table

T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

Excitation Table

Q_n	Q_{n+1}	T
0	0	
0	1	
1	0	
1	1	

KNOWLEDGE GATE

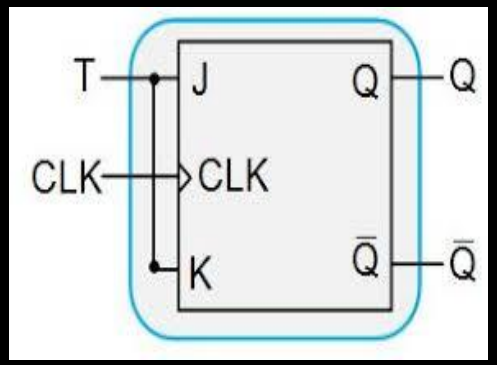
Truth Table

T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

State Diagram

KNOWLEDGEGATE

Block Diagram	Truth Table	K-Map



T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

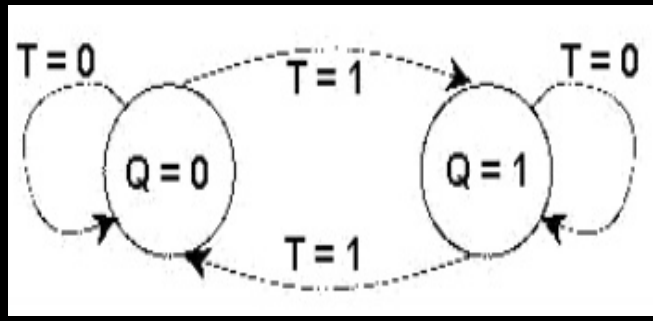
	T	T'	T
q		0	1
q'	0		1
q	1	1	

Characteristics Equation	Function Table	Excitation Table	State Diagram

$$Q_{n+1} = T \oplus Q_n$$

T	Q_{n+1}
0	Q_n
1	Q_n'

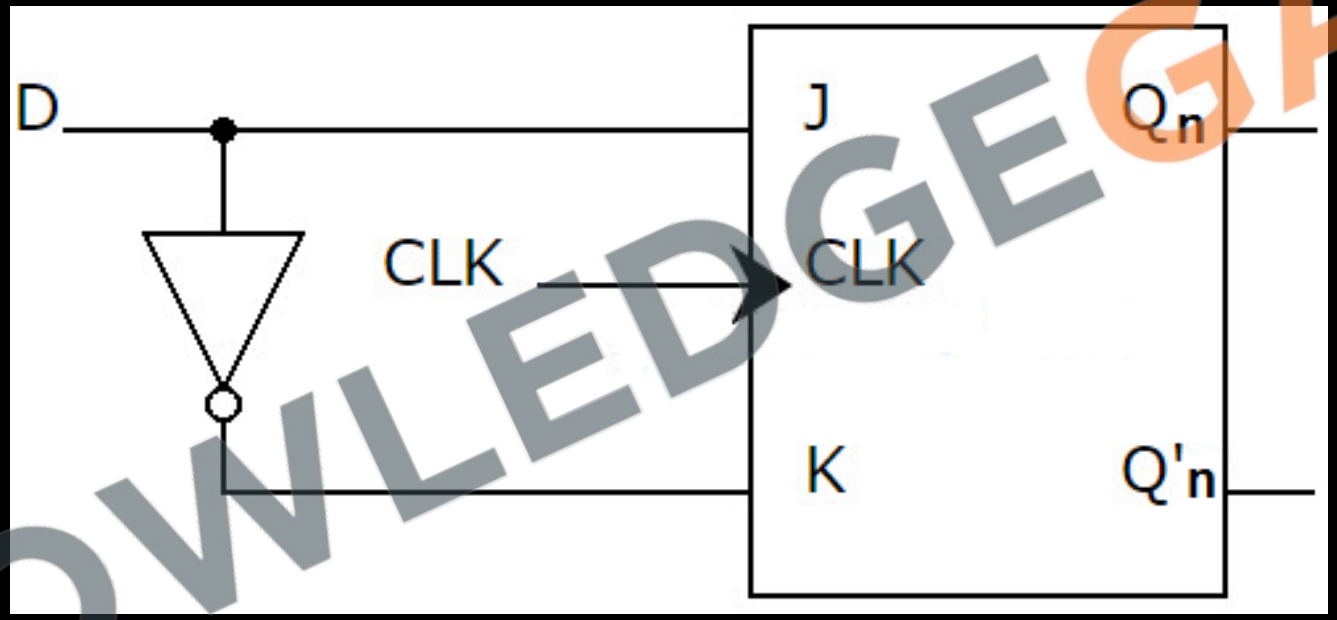
Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0



KNOWLEDGEGATE

D flip flop

- The D (Data/Delay) flip-flop, tracks the input at D and produces the same value as output.



KNOWLEDGEGATE

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Truth Table

D	Q_n	Q_{n+1}
0	0	
0	1	
1	0	
1	1	

KNOWLEDGE GATE

www.knowledgegate.in

Truth Table

D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

K-Map

	D	D’	D
q		0	1
q’	0		
q	1		

Characteristics Equation

Q_{n+1} =

KNOWLEDGE GATE

Truth Table

D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

Characteristics Equation

$$Q_{n+1} = D$$

Function Table

D	Q_{n+1}
0	
1	

KNOWLEDGEGATE

Truth Table

D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

Excitation Table

Q_n	Q_{n+1}	D
0	0	
0	1	
1	0	
1	1	

KNOWLEDGE GATE

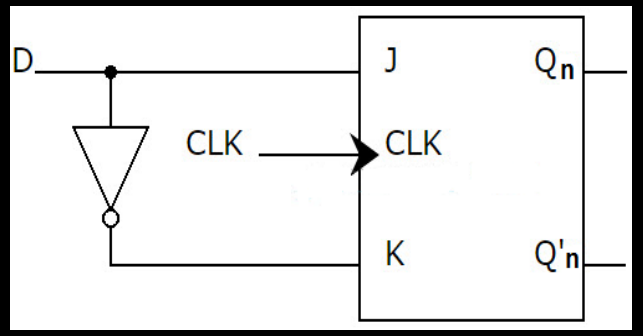
Truth Table

D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

State Diagram

KNOWLEDGEGATE

Block Diagram	Truth Table	K-Map



D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

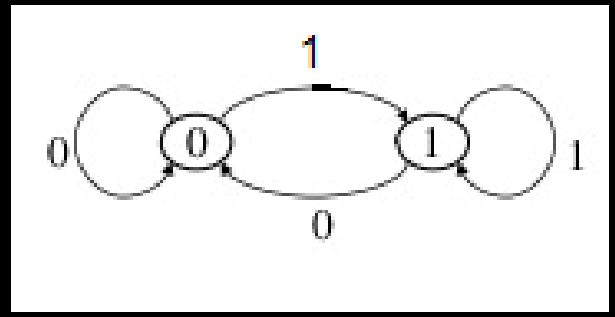
	D	D'	D
q		0	1
q'	0		1
q	1		1

Characteristics Equation	Function Table	Excitation Table	State Diagram

$$Q_{n+1} = D$$

D	Q_{n+1}
0	0
1	1

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1



KNOWLEDGEGATE

www.knowledgegate.in

Flip Flops Conversion

1. We will require the **Characteristics Table** of **target flip flop.**
2. We will require the **Excitation Table** of **given flip flop.**
3. Determine the excitation values for characteristics table.
4. Obtain the expressions for input of given flip flop in terms of target.

KNOWLEDGEGATE

Convert D Flip Flop to T Flip Flop

Characteristics table of T- Flip Flop

T	Q_n	Q_{n+1}
0	0	
0	1	
1	0	
1	1	

Excitation Table of D flip flop

Q_n	Q_{n+1}	D
0	0	
0	1	
1	0	
1	1	

KNOWLEDGE GATE

Characteristics table of T- Flip Flop

T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

T	Q_n	Q_{n+1}	D
0	0	0	
0	1	1	
1	0	1	
1	1	0	

Excitation Table of D flip flop

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

KNOWLEDGE GATE

www.knowledgegate.in

Convert D Flip Flop to T Flip Flop

Excitation values for characteristics table

T	Q_n	Q_{n+1}	D
0	0	0	0
0	1	1	1
1	0	1	1
1	1	0	0

Obtaining simplified expression

T	Q_n	D
0	0	
0	1	
1	0	
1	1	

KNOWLEDGE GATE

Obtaining simplified expression

T	Q_n	D
0	0	0
0	1	1
1	0	1
1	1	0

Using K-Map to Simplify

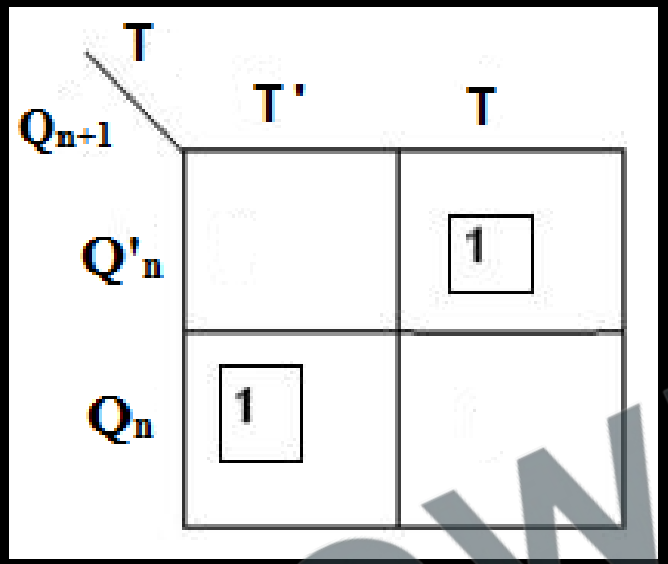
	T	T'	T
q	0	0	1
q'	1		
q			

KNOWLEDGE GATE

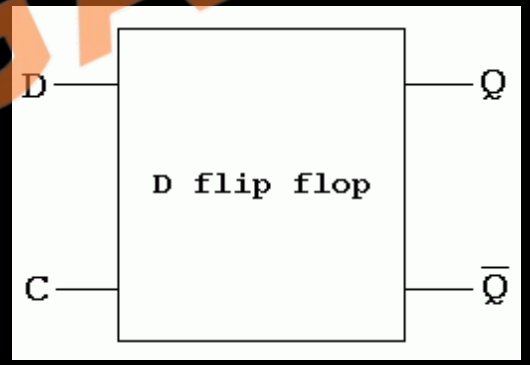
www.knowledgegate.in

Convert D Flip Flop to T Flip Flop

Using K-Map to Simplify



Block Diagram

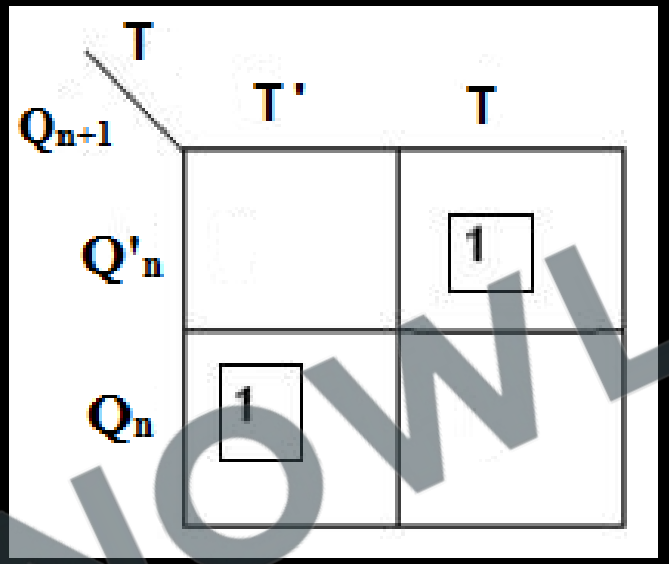


$$D = Q_n \oplus T$$

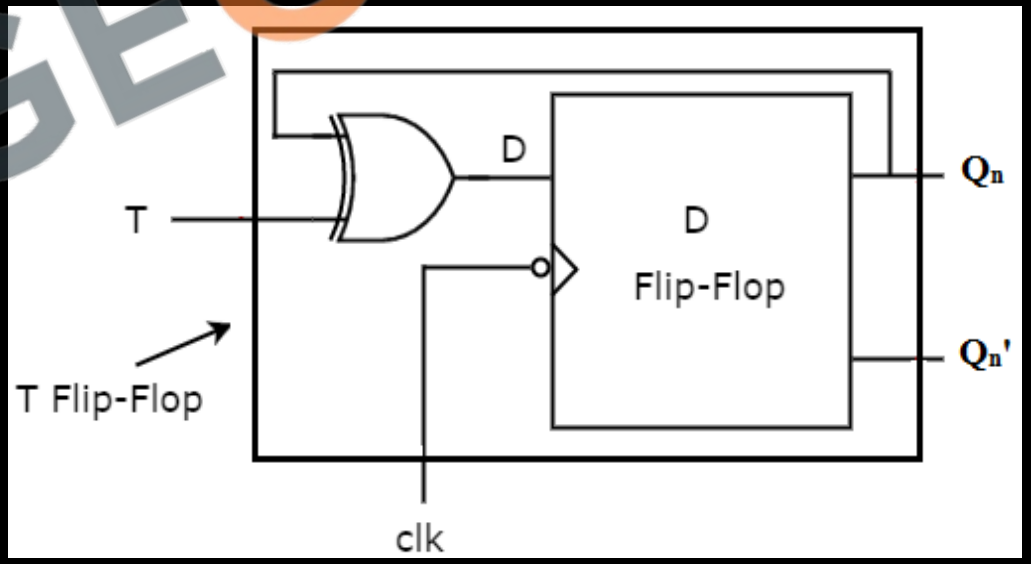
KNOWLEDGEGATE

Convert D Flip Flop to T Flip Flop

Using K-Map to Simplify



Block Diagram



$$D = Q_n \oplus T$$

KNOWLEDGE GATE
www.knowledgegate.in

Flip Flops Conversion

1. We will require the **Characteristics Table** of **target flip flop.**
2. We will require the **Excitation Table** of **given flip flop.**
3. Determine the excitation values for characteristics table.
4. Obtain the expressions for input of given flip flop in terms of target.

KNOWLEDGEGATE

Convert T flip flop to JK flip flop

Characteristics table of JK- Flip Flop

J	K	Q_n	Q_{n+1}
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Excitation Table of T flip flop

Q_n	Q_{n+1}	T
0	0	
0	1	
1	0	
1	1	

KNOWLEDGE GATE

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

J	K	Q_n	Q_{n+1}	T
0	0	0	0	
0	0	1	1	
0	1	0	0	
0	1	1	0	
1	0	0	1	
1	0	1	1	
1	1	0	1	
1	1	1	0	

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

KNOWLEDGE GATE

www.knowledgegate.in

Convert T flip flop to JK flip flop

Excitation values for characteristics table

J	K	Q_n	Q_{n+1}	T
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	0
1	1	0	1	1
1	1	1	0	1

Obtaining simplified expression

J	K	Q_n	T
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

KNOWLEDGE GATE

Convert T flip flop to JK flip flop

Using K-Map to Simplify

J	K	Q_n	T
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

	JK	J'K'	J'K	JK	JK'
q		00	01	11	10
q'	00				
q	01				

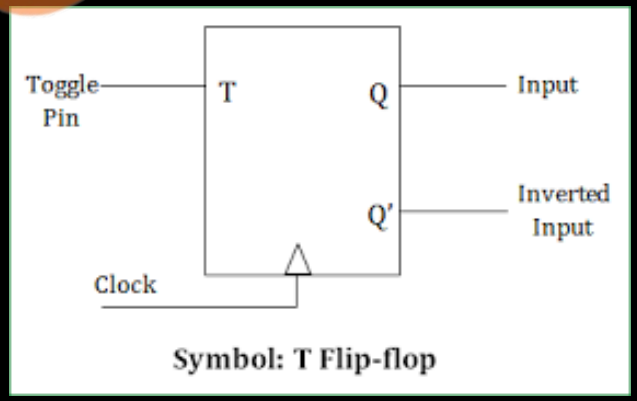
T =

KNOWLEDGE GATE

Convert T flip flop to JK flip flop

Block Diagram

T =

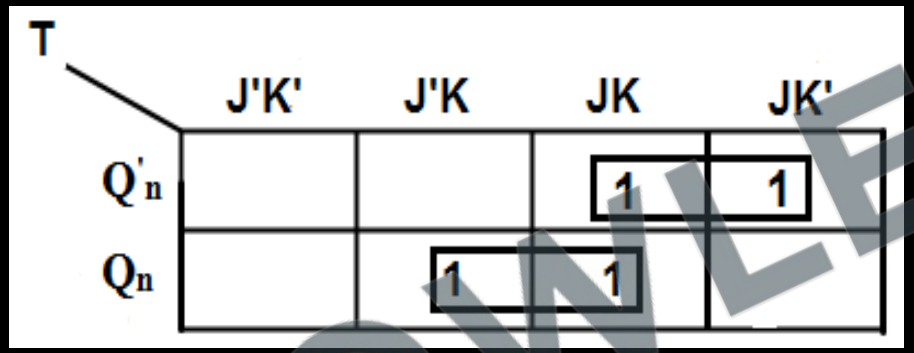


Symbol: T Flip-flop

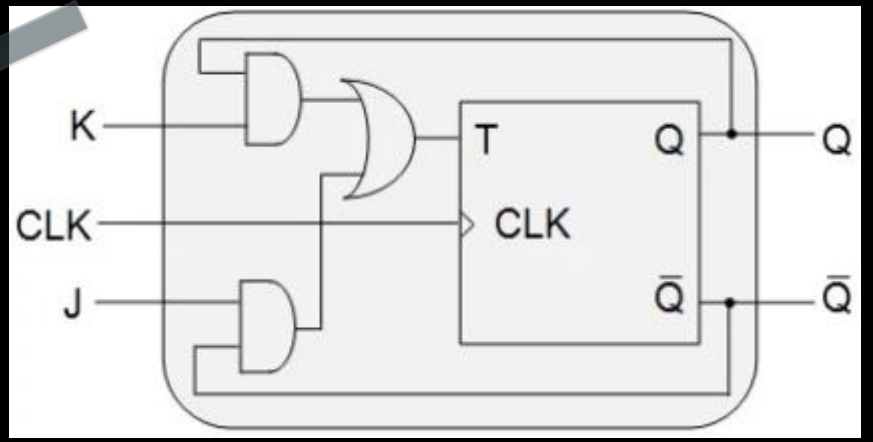
KNOWLEDGEGATE

Convert T flip flop to JK flip flop

Using K-Map to Simplify



Block Diagram



$$T = J Q_n' + K Q_n$$

KNOWLEDGEGATE
www.knowledgegate.in

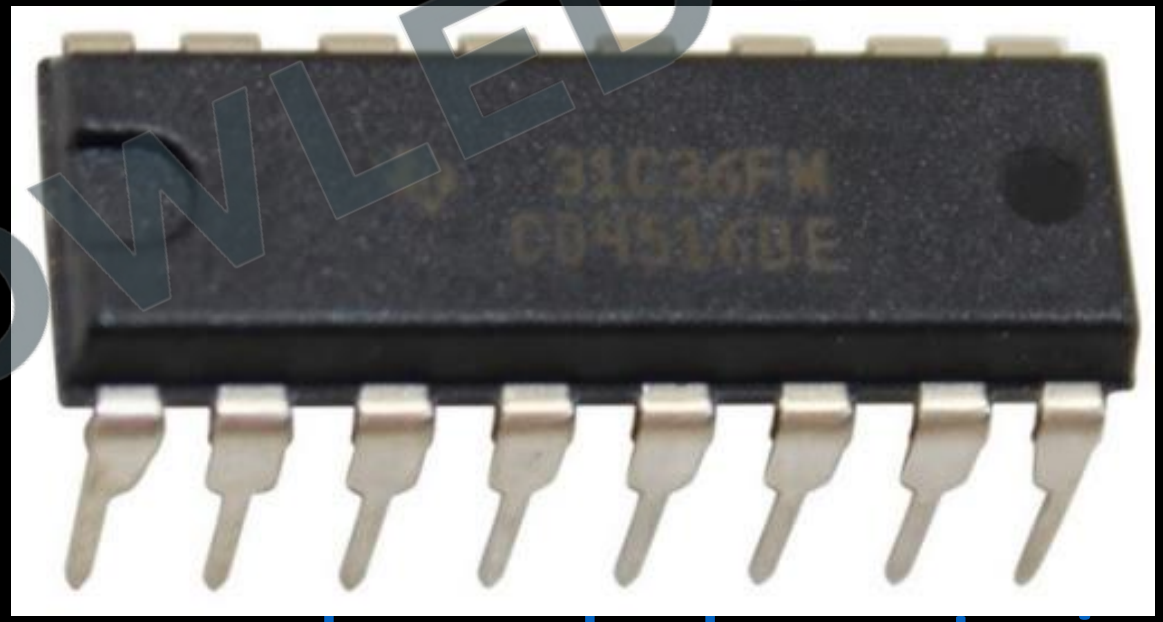
Flip Flops Conversion

1. We will require the **Characteristics Table** of **target flip flop.**
2. We will require the **Excitation Table** of **given flip flop.**
3. Determine the excitation values for characteristics table.
4. Obtain the expressions for input of given flip flop in terms of target.

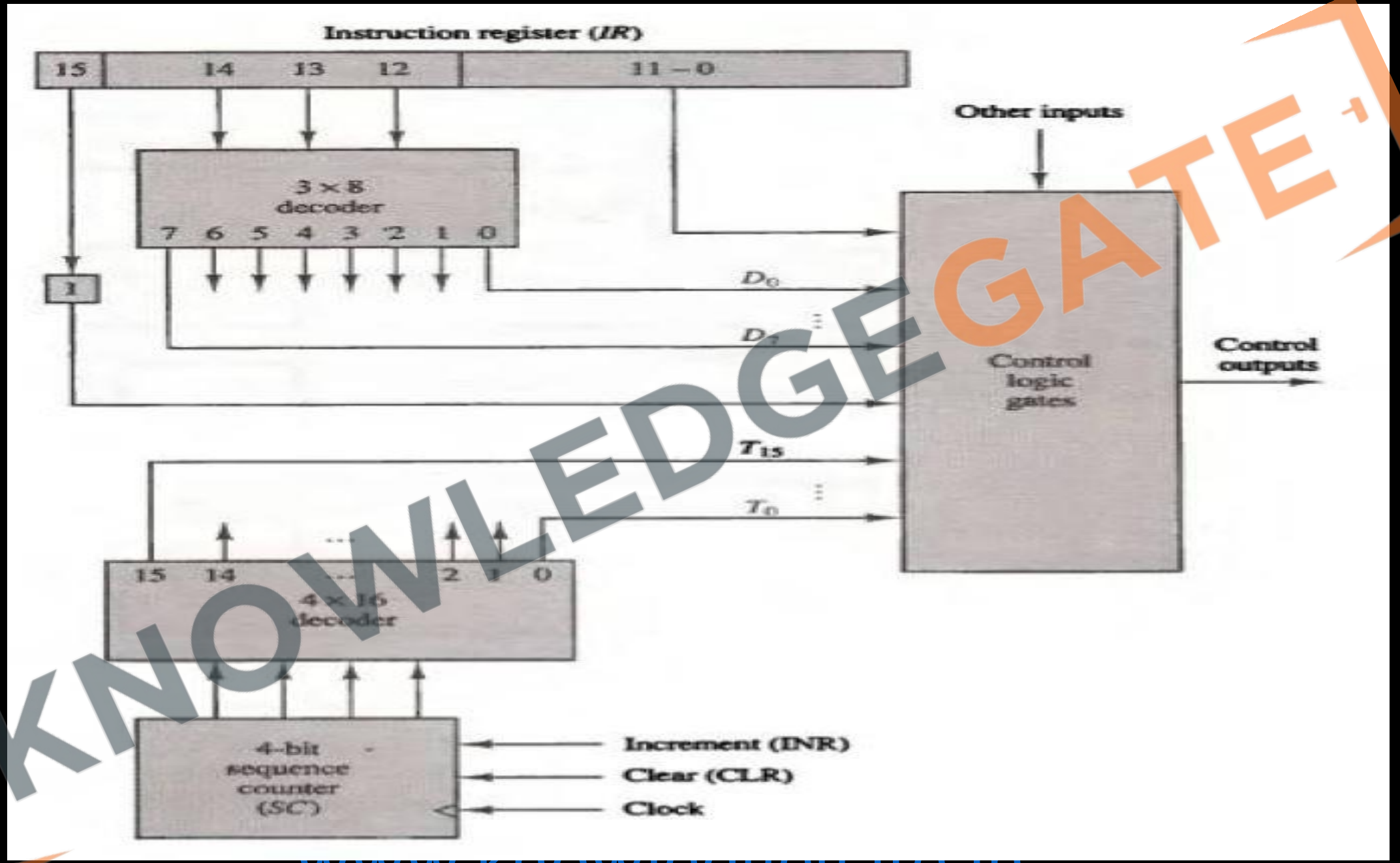
KNOWLEDGEGATE

Basics of counters

- A counter is a sequential circuit that goes through a predetermined sequence of binary states upon the application of input pulses. A counter that follows the binary number sequence is called a binary counter.
- An n - bit binary counter consists of n flip-flops and can count in binary from 0 through $2^n - 1$. Counters are available in two categories: **synchronous counters** and **Ripple counters (asynchronous)**.



KNOWLEDGEGATE
www.knowledgegate.in



KNOWLEDGEGATE
www.knowledgegate.in

KNOWLEDGE GATE
www.knowledgegate.in

KNOWLEDGEGATE
www.knowledgegate.in

KNOWLEDGEGATE
www.knowledgegate.in

KNOWLEDGEGATE
www.knowledgegate.in

KNOWLEDGE GATE
www.knowledgegate.in

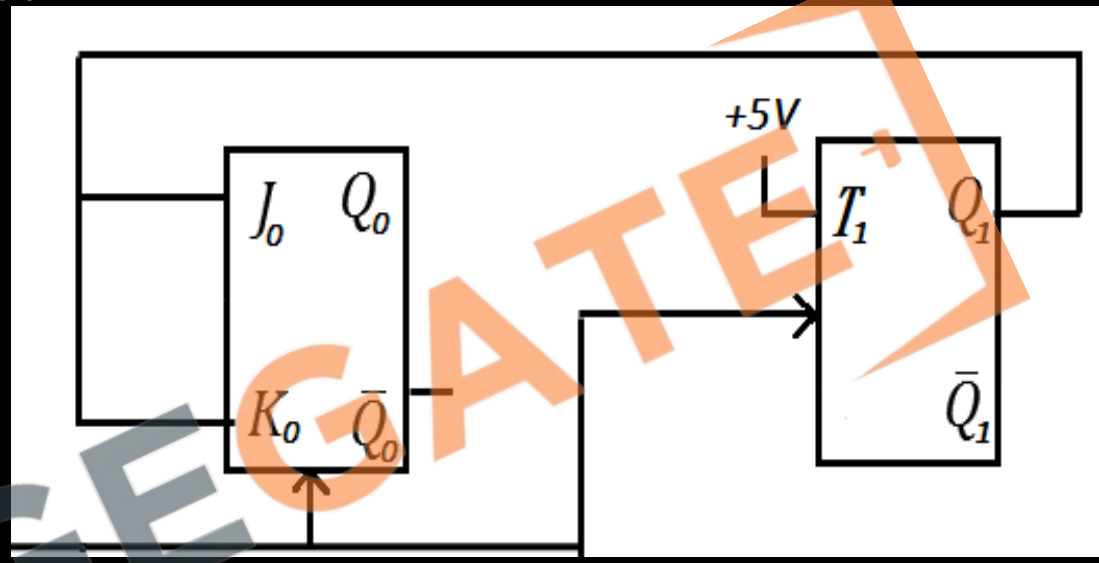
Q Find out the counting sequence for the counter below?

The characteristics equation for JK Flip Flop is Q_{n+1} =

Q_{0N} =

The characteristics equation for T Flip Flop is Q_{n+1} =

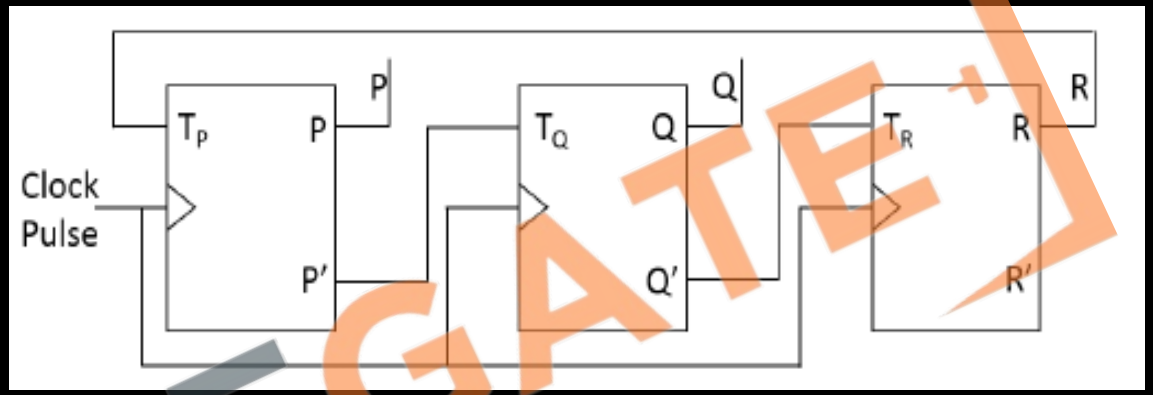
Q_{1N} =



Present State		Next State	
Q_{1p}	Q_{0p}	Q_{1N}	Q_{0N}
0	0		
0	1		
1	0		
1	1		

KNOWLEDGEGATE
www.knowledgegate.in

Q Consider a 3-bit counter, designed using T flip-flops, as shown below:



Present State			Next State		
Q_{2p}	Q_{1p}	Q_{0p}	Q_{2N}	Q_{1N}	Q_{0N}
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

KNOWLEDGEGATE
www.knowledgegate.in

Q Design a synchronous counter for sequence: 0 → 1 → 2 → 3 → 0, using T flip flop.

Present State		Next State	
Q_{1p}	Q_{0p}	Q_{1N}	Q_{0N}
0	0		
0	1		
1	0		
1	1		

KNOWLEDGEGATE

www.knowledgegate.in

Q Example: Design a synchronous counter for sequence: 0 → 1 → 2 → 3 → 0, using D flip flop.

Present State		Next State	
Q_{1p}	Q_{0p}	Q_{1N}	Q_{0N}
0	0		
0	1		
1	0		
1	1		

KNOWLEDGEGATE

www.knowledgegate.in

Q Design synchronous counter for sequence: 0 → 1 → 3 → 4 → 5 → 7 → 0, using T flip-flop.

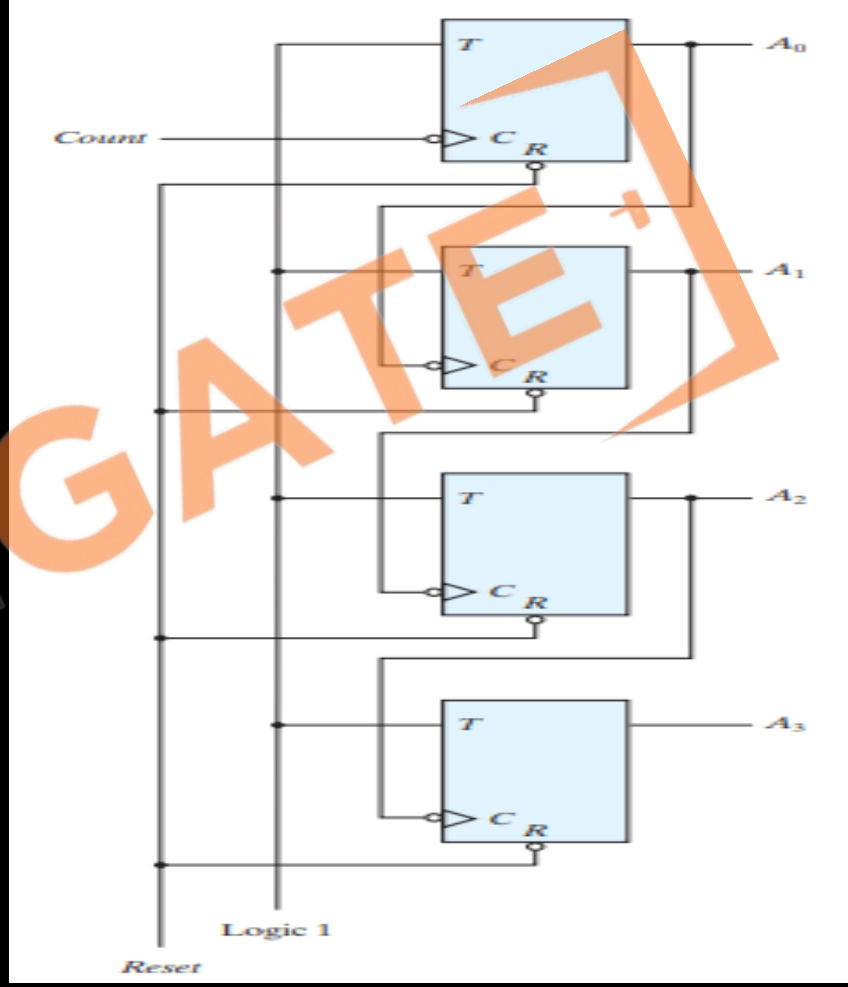
Present State			Next State		
Q_{2p}	Q_{1p}	Q_{0p}	Q_{2N}	Q_{1N}	Q_{0N}
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

KNOWLEDGEGATE
www.knowledgegate.in

RIPPLE COUNTERS

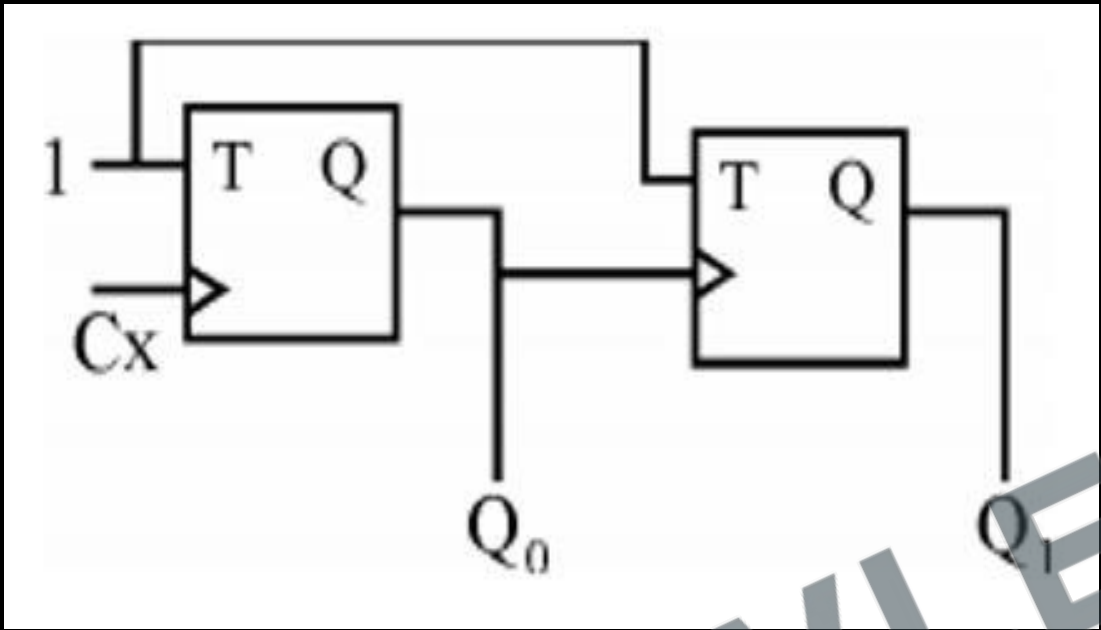
1. A binary ripple counter consists of a series connection of complementing flip-flops, with the output of each flip-flop connected to the input of the next higher order flip-flop.

2. The flip-flop holding the least significant bit receives the incoming count pulses.



KNOWLEDGE GATE

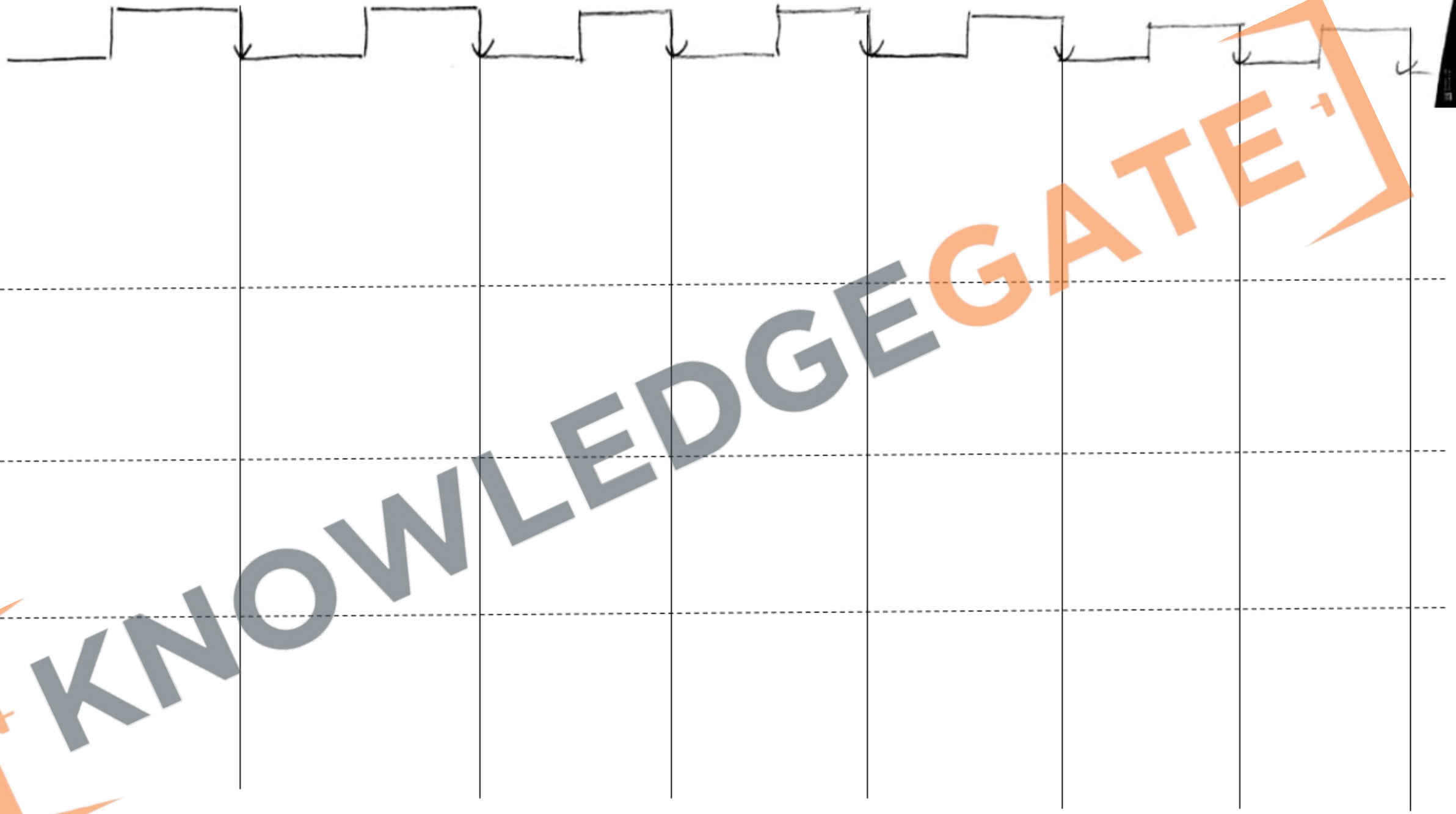
Q In the sequential circuit shown below, if the initial value of the output Q_1Q_0 is 00, what are the next four values of Q_1Q_0?



Present State		Next State	
Q_{1p}	Q_{0p}	Q_{1N}	Q_{0N}
0	0		
0	1		
1	0		
1	1		

KNOWLEDGEGATE

www.knowledgegate.in



KNOWLEDGEGATE

Nature of Clock	Nature of Feedback	Nature of counting
+ve		UP Counting
+ve		Down Counting
-ve		UP Counting
-ve		Down Counting

KNOWLEDGEGATE

www.knowledgegate.in

Asynchronous counters

SYNCHRONOUS COUNTER	ASYNCHRONOUS COUNTER
In synchronous counter, all flip flops are triggered with same clock simultaneously.	In asynchronous counter, different flip flops are triggered with different clock, not simultaneously.
Synchronous Counter is faster than asynchronous counter in operation. $T_{delay} = T_{FF} + T_{CC}$	Asynchronous Counter is slower than synchronous counter in operation. $T_{delay} = n \times T_{FF} + T_{CC}$
Synchronous Counter is also called Parallel Counter.	Asynchronous Counter is also called Serial Counter.
Synchronous Counter designing as well implementation are complex due to increasing the number of states.	Asynchronous Counter designing as well as implementation is very easy.
Synchronous Counter will operate in any desired count sequence.	Asynchronous Counter will operate only in fixed count sequence (UP/DOWN).
Synchronous Counter examples are: Ring counter, Johnson counter.	Asynchronous Counter examples are: Ripple UP counter, Ripple DOWN counter.
A synchronous sequential circuit is a system whose behavior can be defined from the knowledge of its signals at discrete instants of time.	The behavior of an asynchronous sequential circuit depends upon the input signals at any instant of time and the order in which the inputs change.

KNOWLEDGE GATE
www.knowledgegate.in

Registers

- Registers are basically storing devices which are also designed using basic element called flip-flop.
- D-flip-flop are most popular choice for register because they don’t perform any functionality and output is simply based on current input so, they act as a buffer.
- Apart from storing registers sometimes also be used in performing basic mathematical operation like multiply by 2 by left shit and divide by 2 by right shift.

KNOWLEDGE GATE

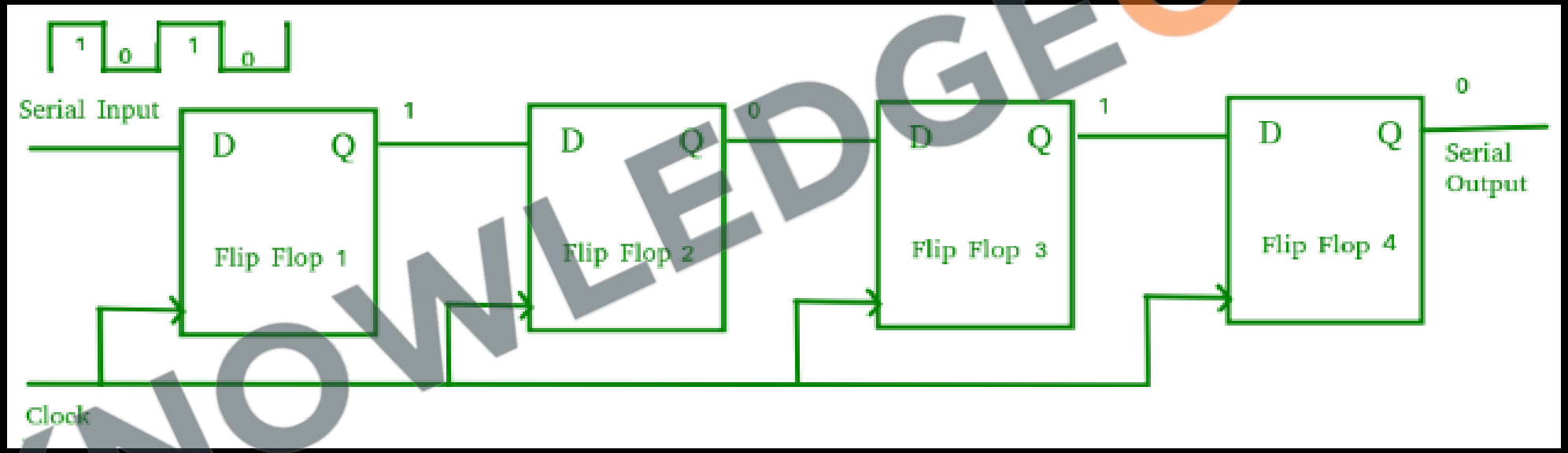
Shift Register Types & Operations

- There are four different modes in which registers operate.
  1. Serial In-Serial Out (SISO)
  2. Serial In-Parallel Out (SIPO)
  3. Parallel In-Serial Out (PISO)
  4. Parallel In- Parallel Out (PIPO)

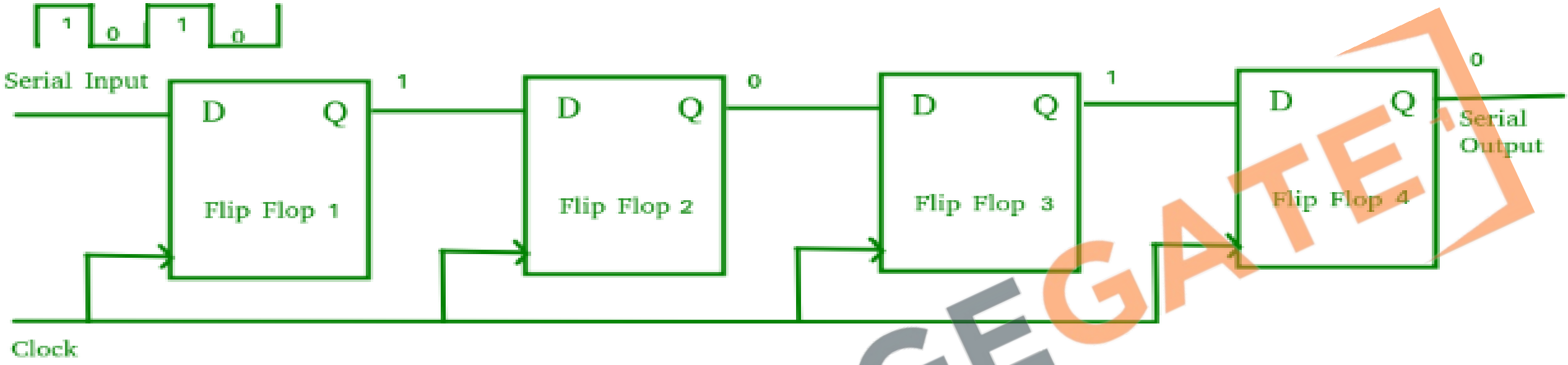
KNOWLEDGEGATE

Serial In-Serial Out (SISO)

- The shift register, which allows serial input (one bit after the other through a single data line) and produces a serial output is known as Serial-In Serial-Out shift register.
- Since there is only one output, the data leaves the shift register one bit at a time in a serial pattern, thus the name Serial-In Serial-Out Shift Register.



KNOWLEDGEGATE
www.knowledgegate.in



Sequence	Input	Q_2	Q_1	Q_0	Output

KNOWLEDGEGATE
www.knowledgegate.in

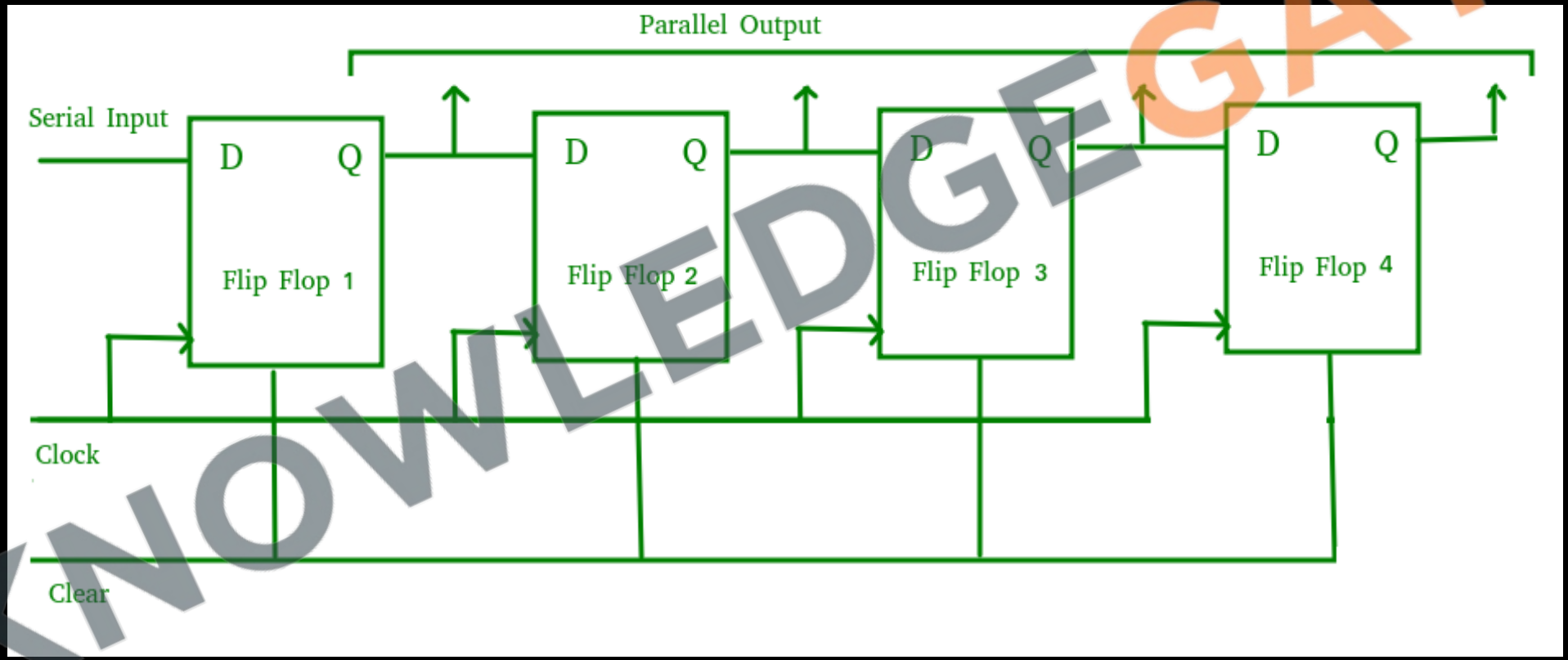
- The main use of a SISO is to act as a delay element.
- In SISO registers to provide n bit data serially in it requires n clock pulse and to provide serial output it requires n - 1 clock pulses.

	No of clock (write)	No of clock (Read)	total
SISO			
SIPO			
PISO			
PIPO			

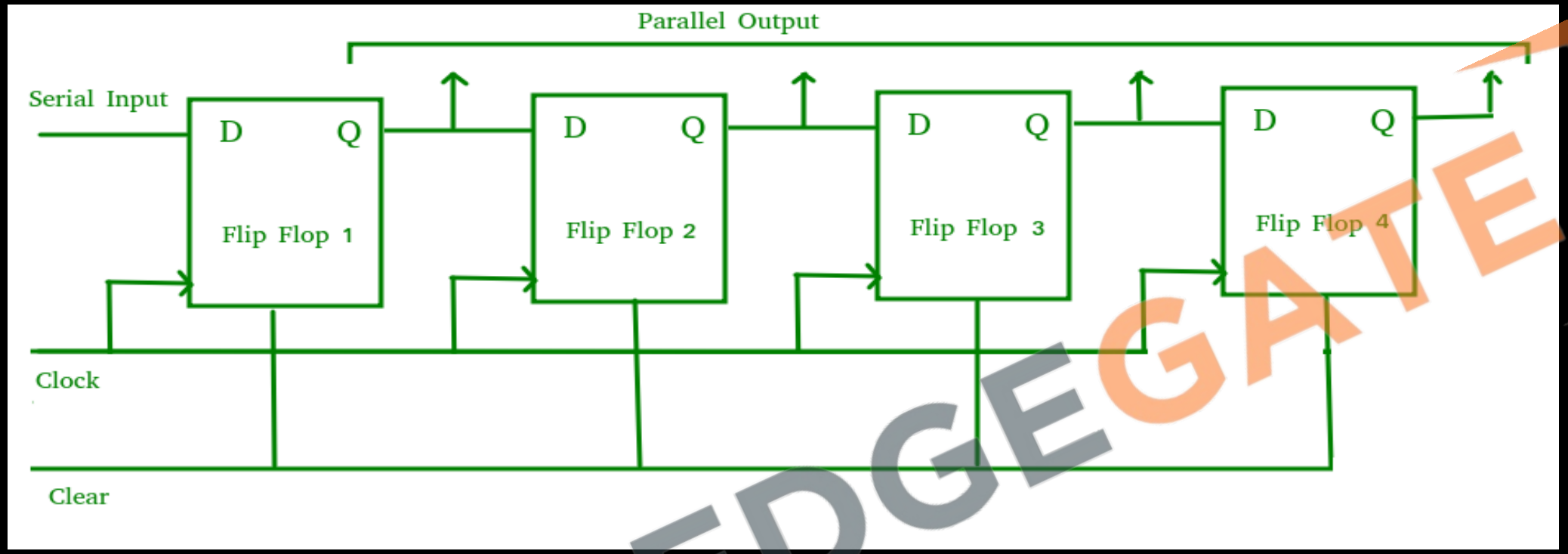
KNOWLEDGEGATE

Serial-In Parallel-Out shift Register (SIPO)

- The shift register, which allows serial input (one bit after the other through a single data line) and produces a parallel output is known as Serial-In Parallel-Out shift register.



KNOWLEDGEGATE



- They are used in communication lines where demultiplexing of a data line into several parallel lines is required because the main use of the SIPO register is to convert serial data into parallel data.

KNOWLEDGEGATE

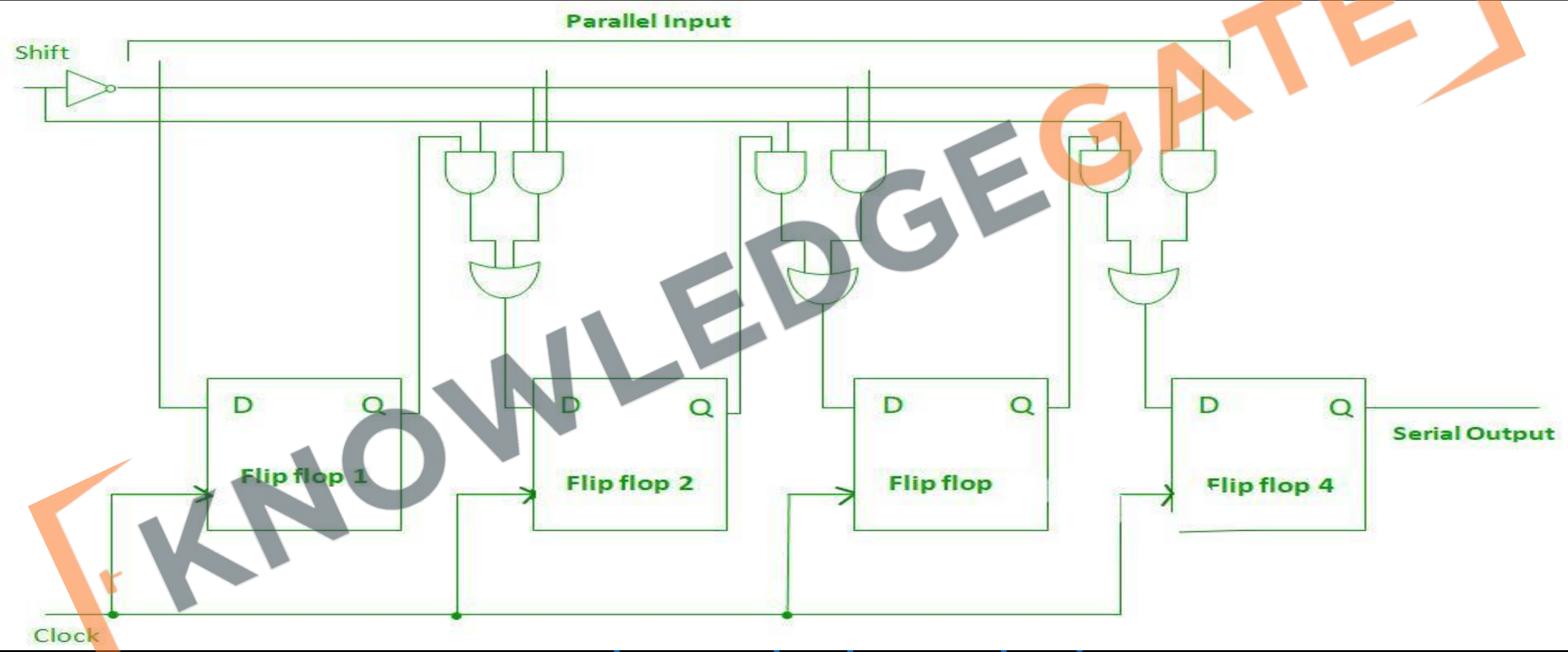
	No of clock (write)	No of clock (Read)	total
SISO	n	n-1	2n-1
SIPO			
PISO			
PIPO			

KNOWLEDGEGATE

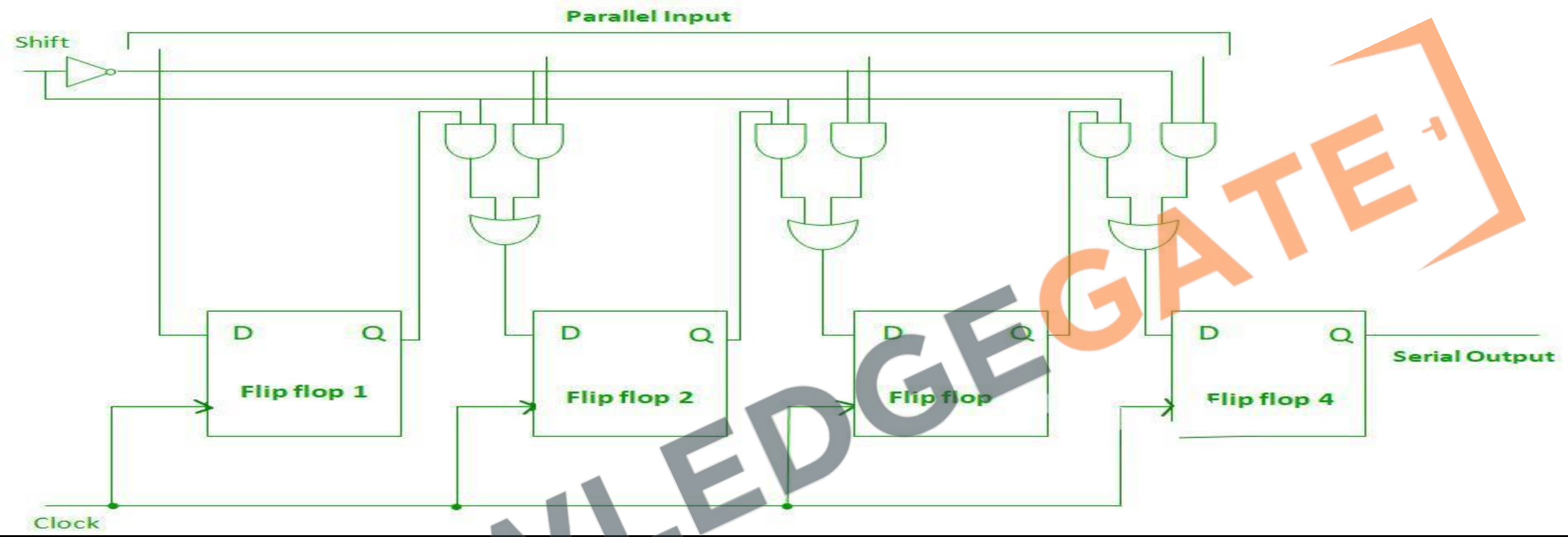
www.knowledgegate.in

Parallel-In Serial-Out Shift Register (PISO)

- The shift register, which allows parallel input (data is given separately to each flip flop and in a simultaneous manner) and produces a serial output is known as Parallel-In Serial-Out shift register.



KNOWLEDGEGATE



- The circuit consists of four D flip-flops which are connected.
- The clock input is directly connected to all the flip flops but the input data is connected individually to each flip flop through a multiplexer at the input of every flip flop.
- The output of the previous flip flop and parallel data input are connected to the input of the MUX and the output of MUX is connected to the next flip flop.
- A Parallel in Serial out (PISO) shift register us used to convert parallel data to serial data.

KNOWLEDGEGATE
www.knowledgegate.in

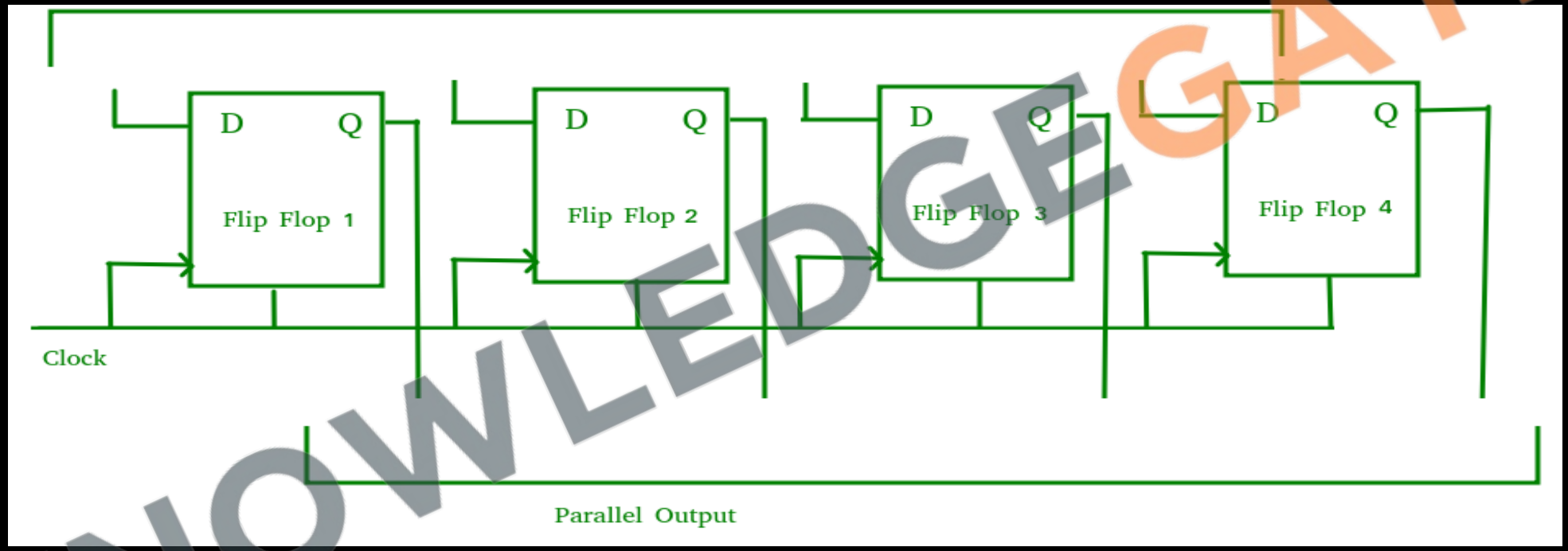
	No of clock (write)	No of clock (Read)	total
SISO	n	n-1	2n-1
SIPO	n	0	n
PISO			
PIPO			

KNOWLEDGEGATE

www.knowledgegate.in

Parallel-In Parallel-Out Shift Register (PIPO)

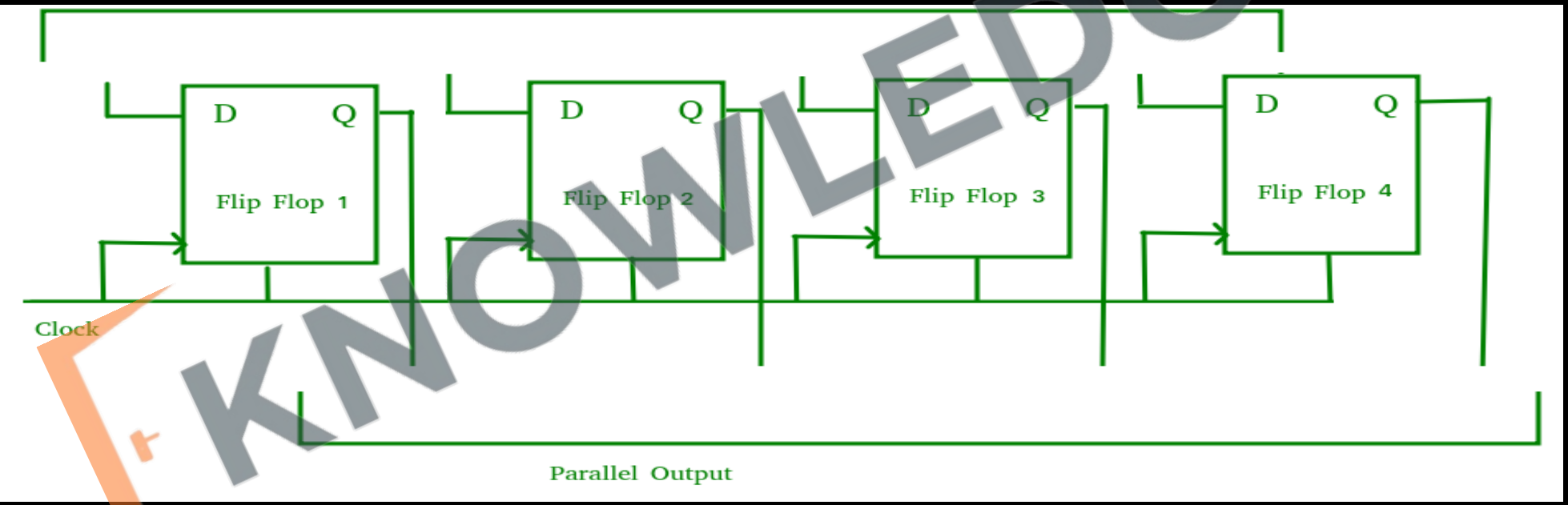
- The shift register, which allows parallel input (data is given separately to each flip flop and in a simultaneous manner) and also produces a parallel output is known as Parallel-In parallel-Out shift register.



KNOWLEDGEGATE

www.knowledgegate.in

- The circuit consists of four D flip-flops which are connected.
- In this type of register, there are no interconnections between the individual flip-flops since no serial shifting of the data is required.
- Data is given as input separately for each flip flop and in the same way, output also collected individually from each flip flop.
- A Parallel in Parallel out (PIPO) shift register is used as a temporary storage device and like SISO Shift register it acts as a delay element.
- For parallel input it requires 1 clock pulse and for parallel output it requires 0 clock.



KNOWLEDGEGATE
www.knowledgegate.in

	No of clock (write)	No of clock (Read)	total
SISO	n	n-1	2n-1
SIPO	n	0	n
PISO	1	n-1	n
PIPO			

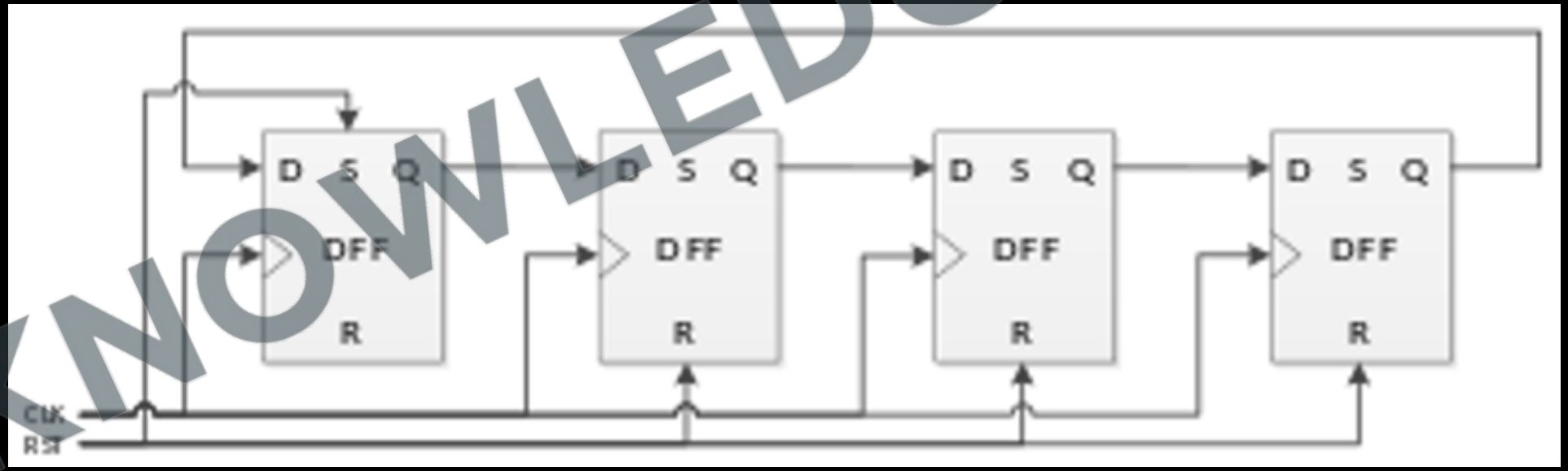
KNOWLEDGEGATE

www.knowledgegate.in

Ring Counter/ Straight Ring Counter

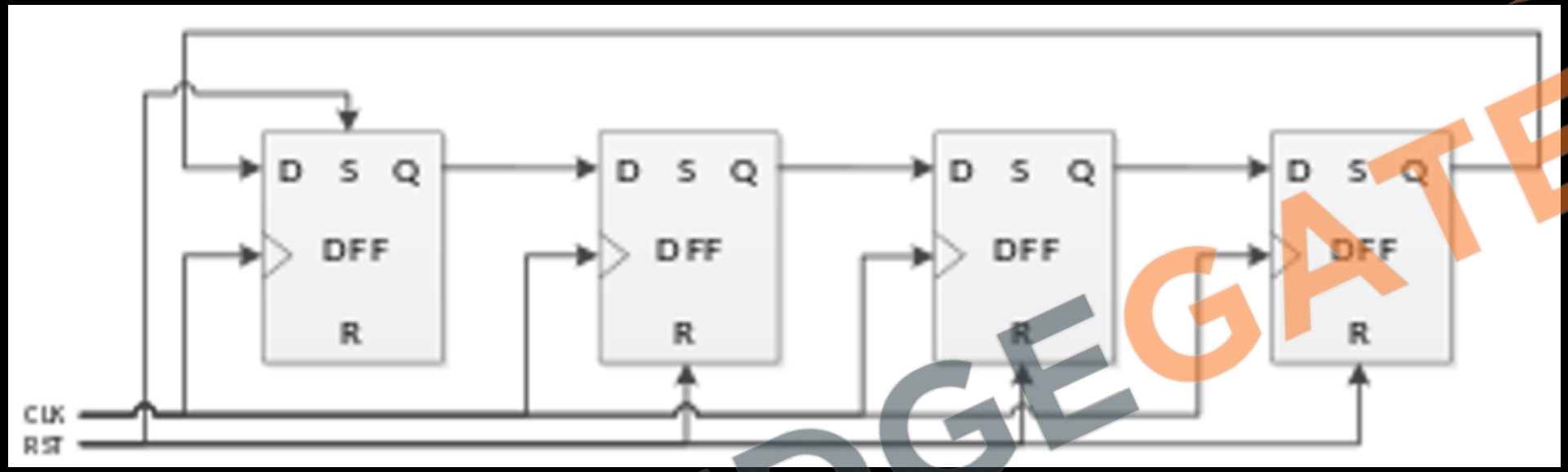
4-Bit Ring Counter

- A ring counter is a circular shift register with only one flip-flop being set at any particular time; all others are cleared.
- The single bit is shifted from one flip-flop to the next to produce the sequence of timing signals.
- Output of the last flip-flop is connected to the input of the first flip-flop in case of ring counter.
- ***No. of states in Ring counter = No. of flip-flop used***



KNOWLEDGEGATE

- Number of unused states in Ring Counter is 2^n-n

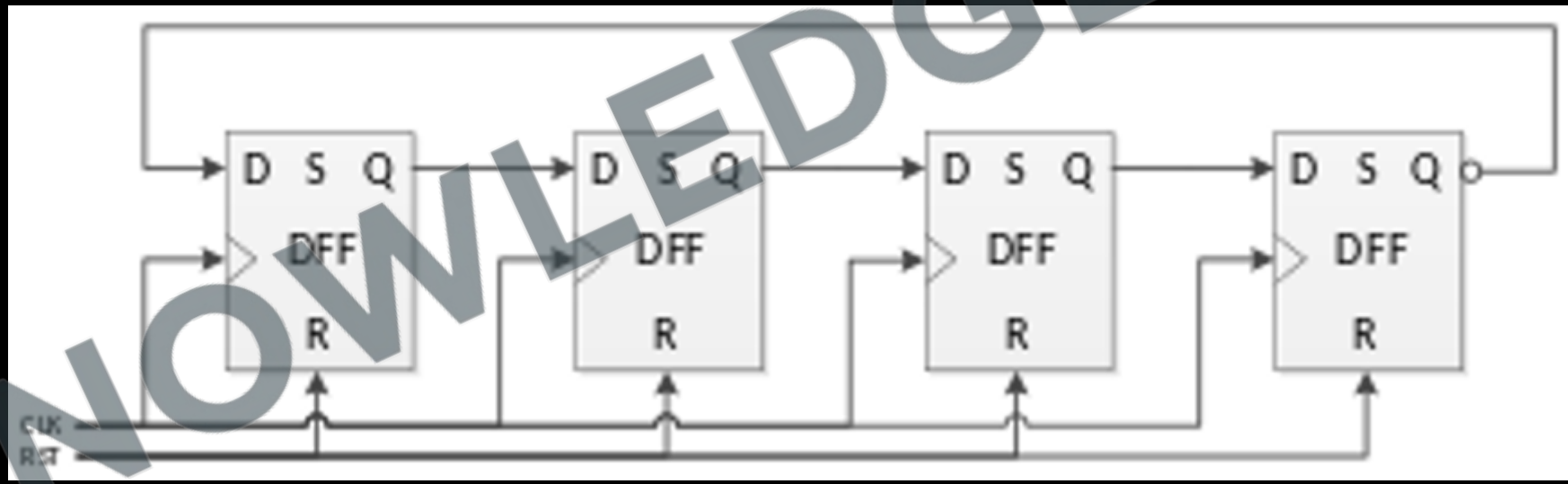


CLK	Q_3	Q_2	Q_1	Q_0
0	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
0	1	0	0	0

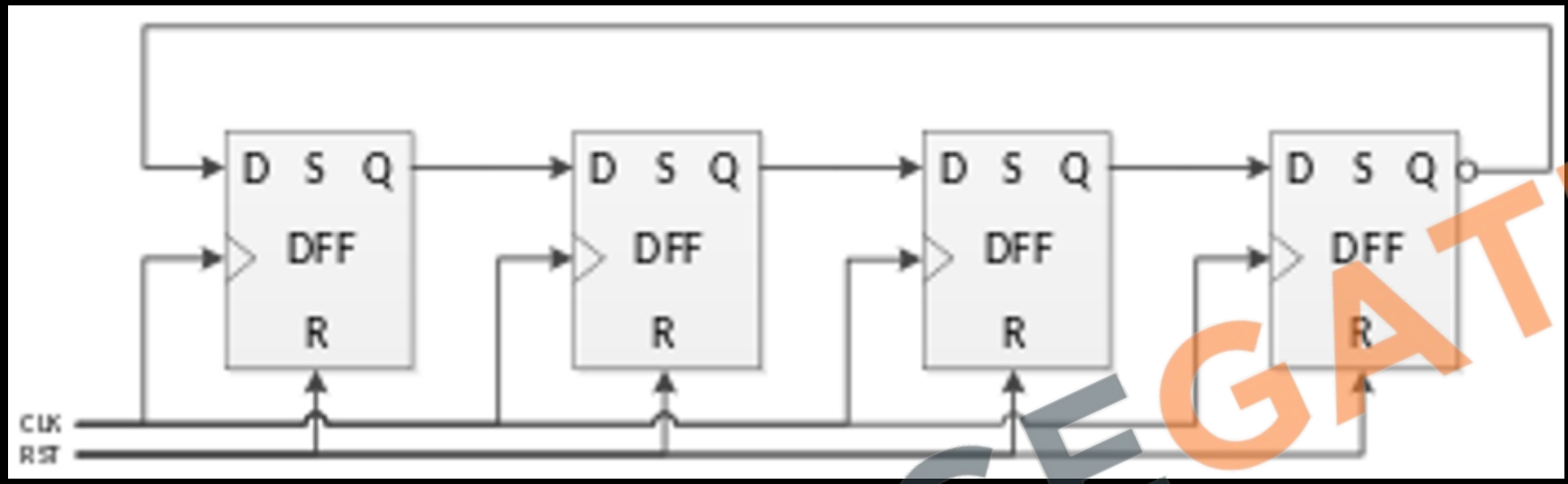
KNOWLEDGEGATE
www.knowledgegate.in

Johnson Counter/ Twisted ring counter/ Switch-tail ring counter/ Walking ring counter

- A k -bit ring counter circulates a single bit among the flip-flops to provide k distinguishable states.
- The number of states can be doubled if the shift register is connected as a switch-tail ring counter. A switch-tail ring counter is a circular shift register with the complemented output of the last flip-flop connected to the input of the first flip-flop.



KNOWLEDGEGATE



CLK	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1
5	0	1	1	1
6	0	0	1	1
7	0	0	0	1
0	0	0	0	0

KNOWLEDGEGATE
www.knowledgegate.in

Basics

- Main idea in number system is counting and representation of quantity
- In stone age or even today the basic idea of counting in Unary counting
  - Remember how we started counting on fingers and using abacus
  - How can we use it for counting sheep or anything?

KNOWLEDGE GATE

- But when we want to count larger quantity in anything, day to day life, business, maths or research, we cannot work with unary system.

Samsung pays Apple $1 Billion sending 30 trucks full of 5 cent coins

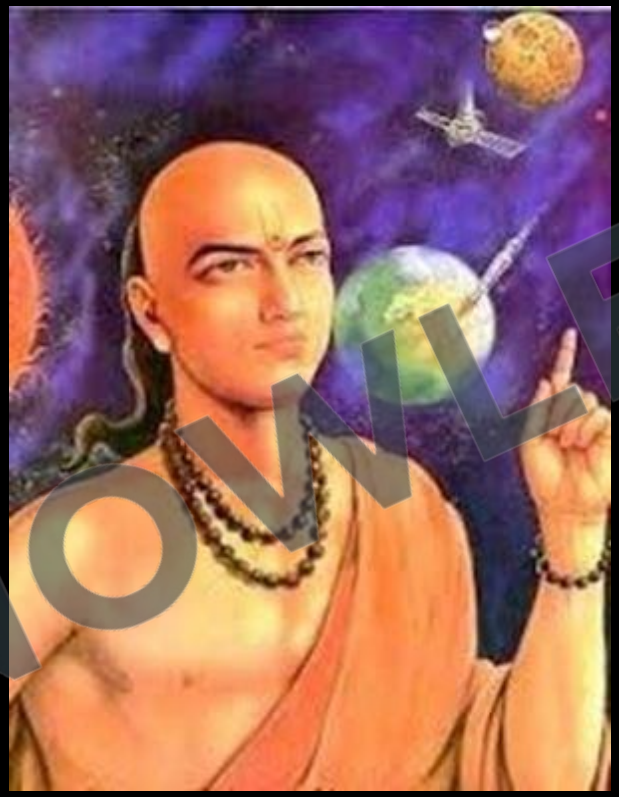
KNOWLEDGEGATE

www.knowledgegate.in

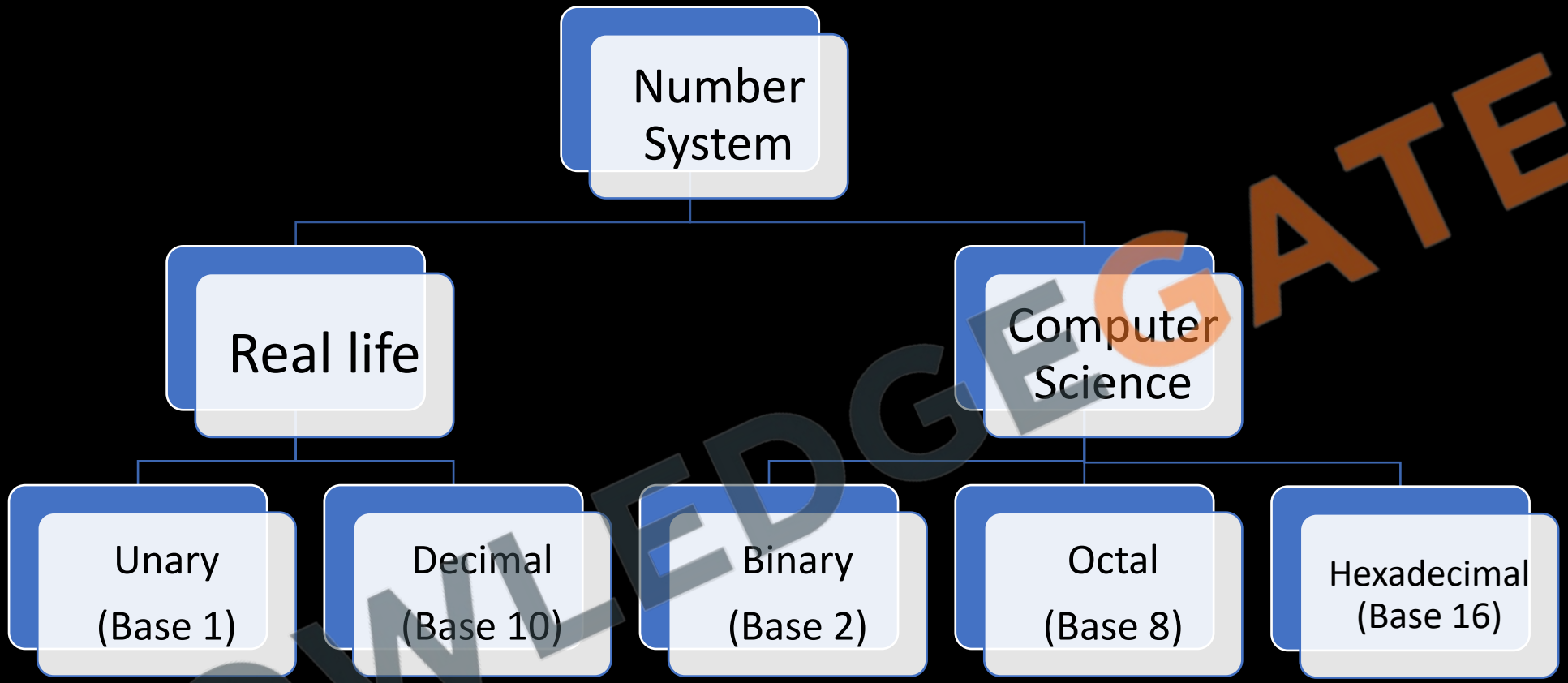
- So next popular system we use in real life is decimal system, may be because we have 10 fingers on over hands, in history different cultures have been using base 10 for general purpose counting, Indian, British, roman, Arabic etc.

KNOWLEDGEGATE
www.knowledgegate.in

- Thought **Aryabhata (3500 years back)** was the first one known to extensively worked on the idea of zero, pie, number system, trigonometry, quadratic equations, astronomy etc.
- In Indian culture we were working on very complex math from much early time, Aryabhata was may be first one, who have properly written and published his understanding, and some of his work even reached today.



KNOWLEDGE GATE



KNOWLEDGEGATE

www.knowledgegate.in

Number System	Base or Radix	Digits or symbols
Binary	2	0,1
Octal	8	0,1,2,3,4,5,6,7
Hexadecimal	16	0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

Number System	Base or Radix	Digits or symbols
Unary	1	0/1
Decimal	10	0,1,2,3,4,5,6,7,8,9

KNOWLEDGEGATE

Decimal Number System

- The Decimal Number System uses 10 different digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. It's called a "base-10" system because it relies on these 10 digits and uses powers of 10 to form numbers. This is the number system we commonly use in everyday activities.
- Example: $(739.2)_{10}$, where 739.2 is a shorthand notation for what should be written as

$$7 * 10^2 + 3 * 10^1 + 9 * 10^0 + 2 * 10^{-1}$$

KNOWLEDGEGATE

Binary Number System

- In the Binary Number System, you only use two digits: 0 and 1. Each digit in a binary number is multiplied by a power of 2 (like 2^0, 2^1, 2^2, and so on). Then you add these products together to get the number in decimal form.

Example: $(11010.11)_2$ value in Decimal?

$$1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 + 1 * 2^{-1} + 1 * 2^{-2} = 26.75$$

KNOWLEDGEGATE

- Similarly, there can be many bases and they can be easily converted to decimal number system.
- An example of a base-5 number is

$$(4021.2)_5$$

$$4 * 5^3 + 0 * 5^2 + 2 * 5^1 + 1 * 5^0 + 2 * 5^{-1} = (511.4)_{10}$$

KNOWLEDGEGATE

Convert from Decimal to Any Base

$(4213.21)_5$ → $(558.44)_{10}$ → $(1425.3036)_7$

KNOWLEDGEGATE

www.knowledgegate.in

$(83)_{10} \longrightarrow (1010011)_2$

KNOWLEDGEGATE

www.knowledgegate.in

$(109.125)_{10} \longrightarrow (1101101.001)_2$

KNOWLEDGEGATE

www.knowledgegate.in

Octal Number System

$$(127.4)_8$$

$$1 * 8^2 + 2 * 8^1 + 7 * 8^0 + 4 * 8^{-1} = (87.5)_{10}$$

KNOWLEDGEGATE

Hexadecimal Number System

- When the base of a number system is higher than 10, we use letters from the alphabet to represent the extra digits. For instance, in the Hexadecimal System, which is base-16, the first 10 digits are the same as in the Decimal System: 0 to 9. After that, the letters A, B, C, D, E, and F stand for the numbers 10, 11, 12, 13, 14, and 15 respectively. The hexadecimal system is most popular in digital systems.

$$(B65F)_{16}$$

$$11 * 16^3 + 6 * 16^2 + 5 * 16^1 + 15 * 16^0 = (46687)_{10}$$

KNOWLEDGEGATE

- In case a binary number usually becomes so large so better idea is to club digits in a group of three or four, leading to octal, hexadecimal

Hexadecimal System / Base 16 System	
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10-A
1011	11-B
1100	12-C
1101	13-D
1110	14-E
1111	15-F

Octal System / Base 8 System	
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

base 4 System	
00	0
01	1
10	2
11	3

KNOWLEDGEGATE

www.knowledgegate.in

$(\ 110010.111 \ )_2$

$(_____.___)_4$

$(\ 110010.111 \ )_2$

$(____._)_8$

$(\ 110010.111 \ )_2$

$(____.___)_{16}$

KNOWLEDGEGATE
www.knowledgegate.in

$(62.7)_8$

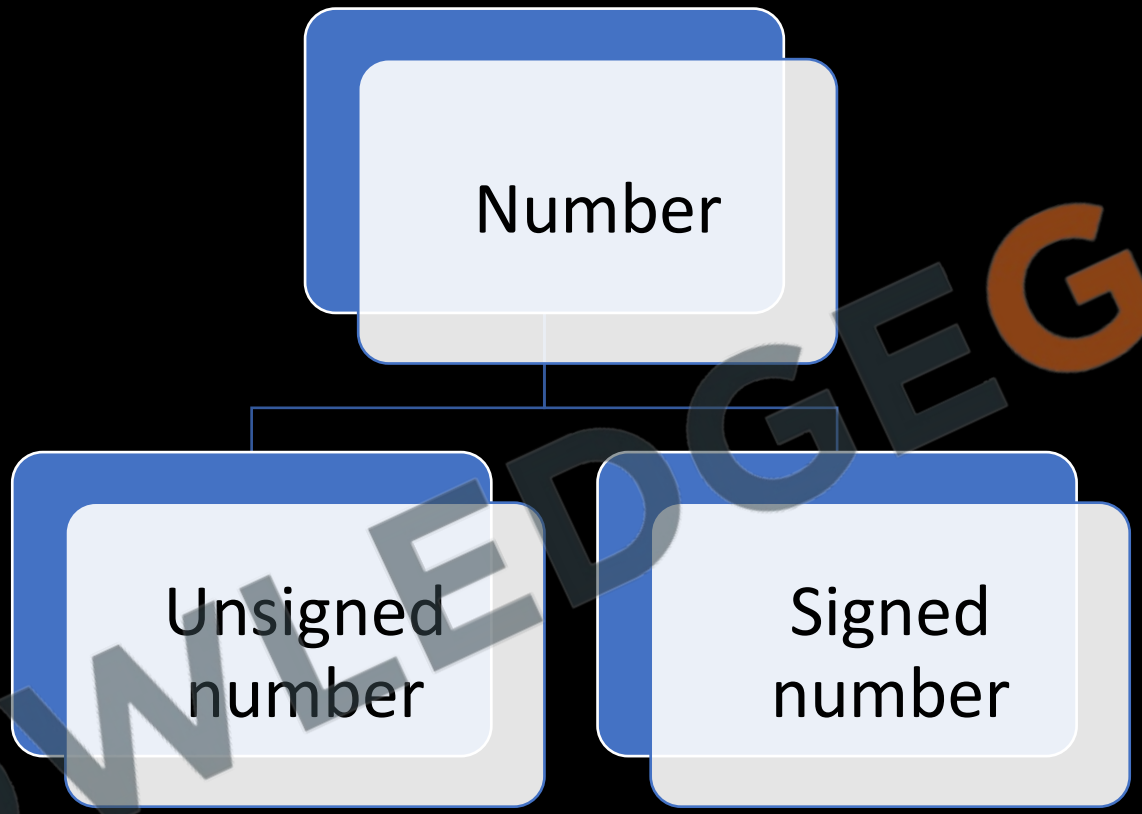
$(_____\cdot____)_2$

$(____\cdot___)_{16}$

KNOWLEDGEGATE

www.knowledgegate.in

Representation of a number



KNOWLEDGEGATE

www.knowledgegate.in

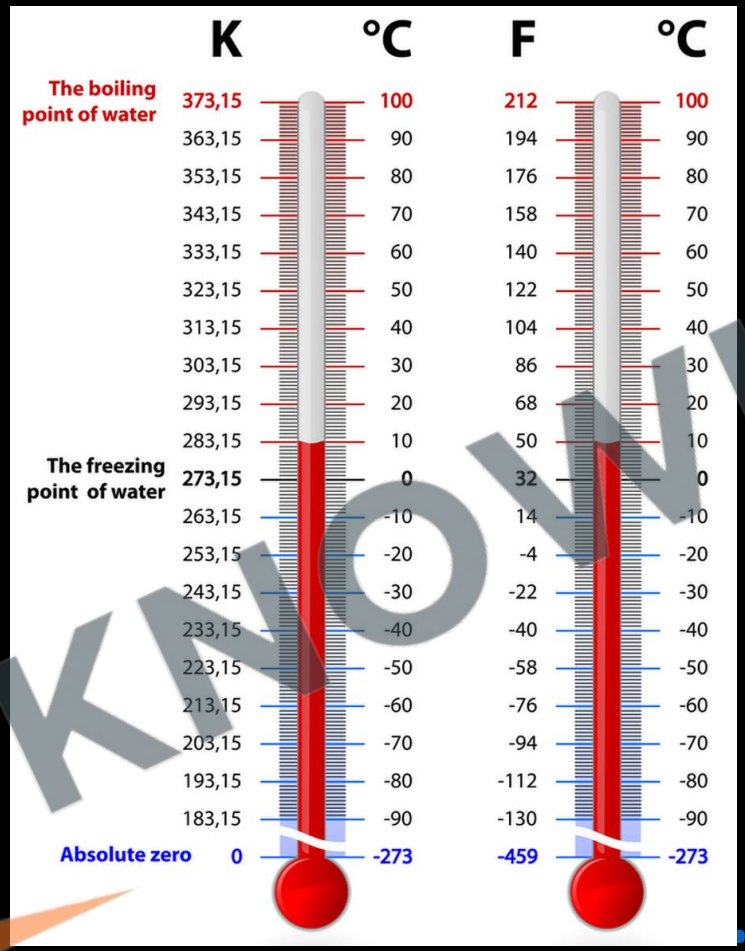
Unsigned number

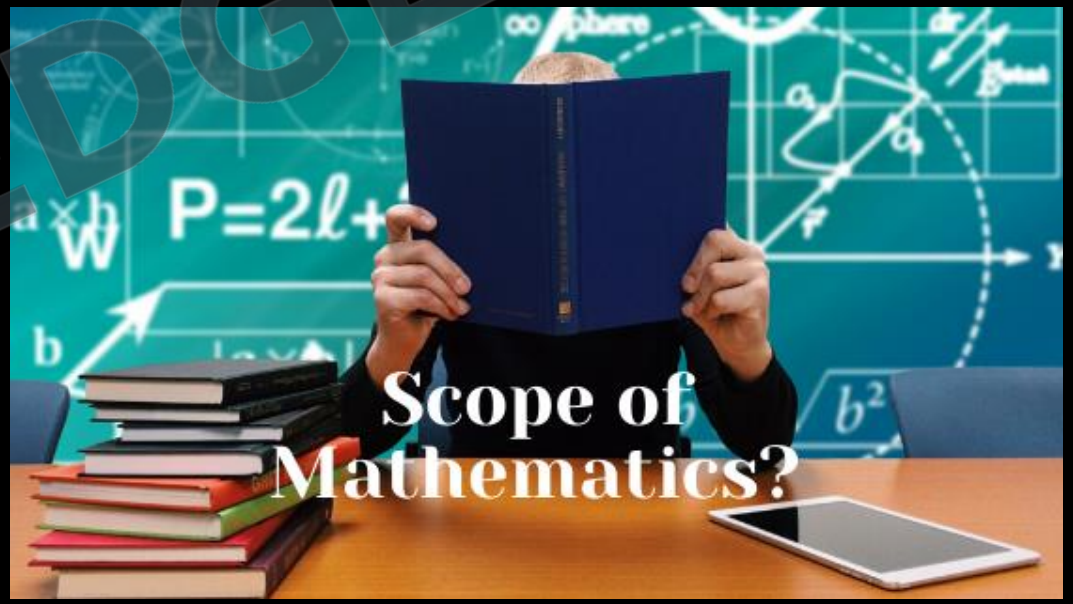
- Until now, we have considered only unsigned numbers. Unsigned numbers without any positive or negative sign, these numbers represent only magnitude. In real life, we have only unsigned numbers (considered as positive) (natural numbers) for e.g. number of stars, number of trees etc.
- If n bits are used to store the number, then all the n bits are used to store number / magnitude / absolute value. If we have a n bit Unsigned number than range is from 0 to 2^n-1

KNOWLEDGEGATE

Signed number

- For modern-day math tasks like calculating profit and loss or measuring temperature, we often need to use negative numbers. In daily life, we write these numbers with both their sign and magnitude. For example, +9 shows a positive number and -543 indicates a negative one. We use the symbols "+" and "-" to show if the number is positive or negative, respectively.

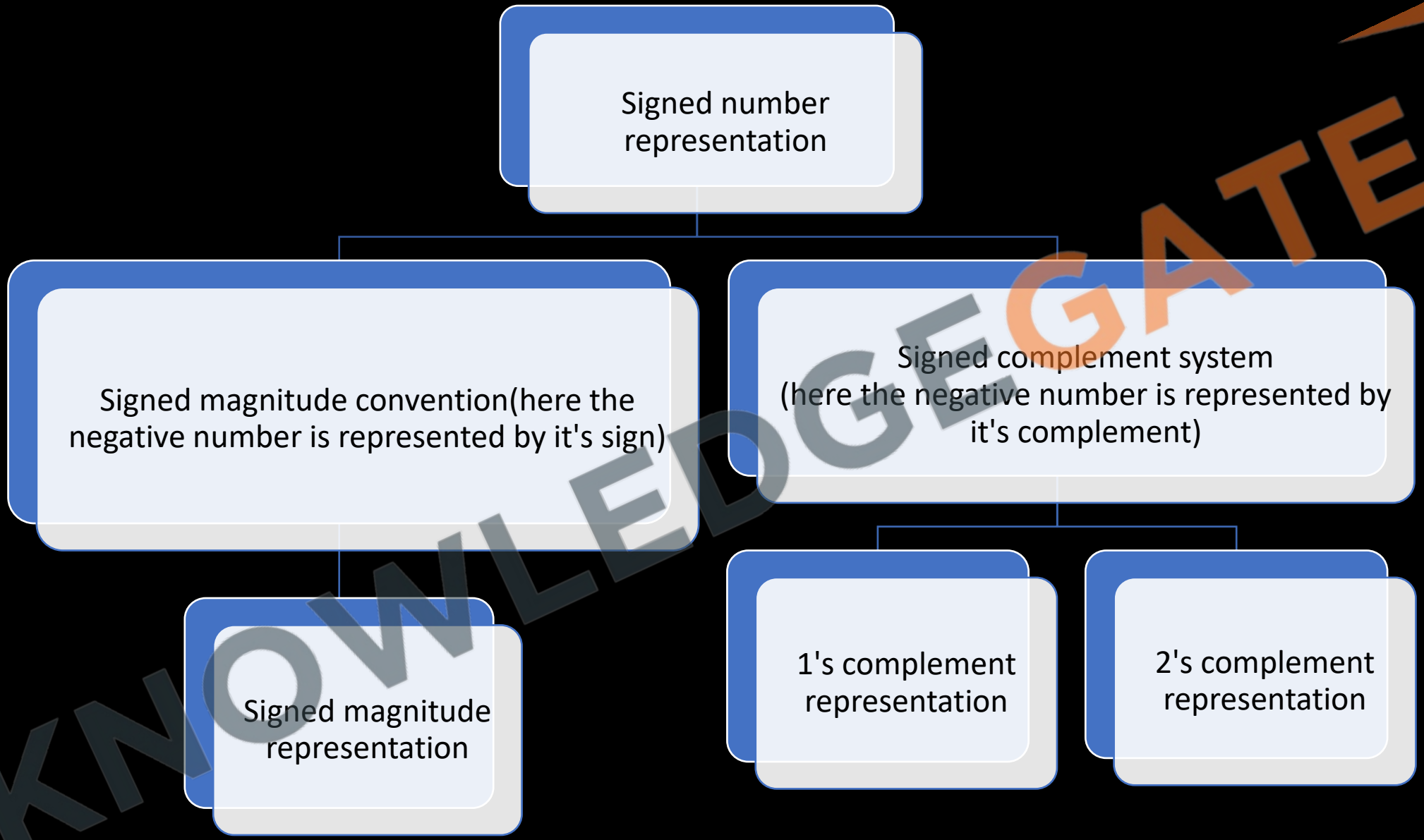




KNOWLEDGEGATE
.knowledgegate.in

- In a computer, an n-bit word can store a signed binary number. One usually the leftmost bit is set aside to show if the number is positive or negative, and the other n-1 bits represent the size or magnitude of the number.
- Because of the limitation of the computer hardware as circuit only understand 0 and 1, every piece of information in computer must be represented in terms of numbers i.e. 0 and 1, weather it is sign or magnitude. where first left most bit 0 means +Ve number and 1 means -Ve number.

KNOWLEDGEGATE



KNOWLEDGE GATE

www.knowledgegate.in

Decimal	Signed Magnitude	1's Complement	2's Complement
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	1000	1111	0000
-1	1001	1110	1111
-2	1010	1101	1110
-3	1011	1100	1101
-4	1100	1011	1100
-5	1101	1010	1011
-6	1110	1001	1010
-7	1111	1000	1001
-8	-------	-------	1000

KNOWLEDGE GATE
www.knowledgegate.in

- Represent decimal number -13 in all three methods?
  - Signed Magnitude
  - 1’s Complement
  - 2’s Complement

KNOWLEDGEGATE

www.knowledgegate.in

- Read number 10101 according to all three representation?
  - Signed Magnitude
  - 1’s Complement
  - 2’s Complement

KNOWLEDGEGATE

www.knowledgegate.in

Sign magnitude representation

- **Introduction: -** It is the simplest form of representation, where in an n-bit word, the right most n-1 bits (from LSB) hold the magnitude of the number and n^{th} bit (Left most, MSB) is assigned for sign, where 0→+ve and 1→-ve.
- **Range: -** from $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$ total $2^n - 1$
- **Advantage: -** Easy to read and write, because it is a weighted code system, directly reading and writing is possible.
- **Disadvantage: -** One drawback in sign-magnitude representation is addition and subtraction require separate handling of the sign and the magnitude. One more problem is there are two representation for 0, i.e. -0 & +0, because of which there is a loss of one presentation, and create confusion.
- **Number extension : -** In computer the size of a location is fixed, so it is often required to write a smaller number in a larger space, this can be done by sign extension, where, the magnitude is written as far right as possible, sign bit is written left most, and all the empty cells must be filled/padded with zero.

KNOWLEDGE GATE

1’s Complement Representation

- **Introduction: -** here the negative number is represented by taking complement.
- **Range: -** from $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$, total 2^n -1 representation
- **Advantage: -** Easy to calculate, used as logical operations.
- **Disadvantage: -** One drawback in 1’s complement representation is addition and subtraction in difficult, Not a weighted code system. there are two representation for 0, i.e. -0 & +0, because of which there is a loss of one presentation. Neither we can write a negative number directly nor we can read a negative number directly.
- **Number extension:** In computer the size of a location is fixed, so it is often required to write a smaller number in a larger space, this can be done by sign extension, where, if the number is positive extension is done same as that of sign magnitude extension. If the number is -ve then we must write the number as far right as possible and then all the empty cells must be filled with 1(in negative logic 0 is represented by 1).

KNOWLEDGE GATE

2's Complement Representation

- **Introduction: -** if 1 is added to 1's complement of a binary number, the resulting number known as the 2's complement of the binary number.
- **Range: -** from $-(2^{n-1})$ to $+(2^{n-1}-1)$, total 2^n representation

Advantage: -

- Easy to do arithmetic operations with 2's complement representation. As end round carry can be discarded. Has only one representation for zero which is always positive, therefore provides better clarity, efficiency and more range with no wastage. It is a waited code system, direct reading and writing is possible.

- **Disadvantage: -** One drawback in 2's complement representation is relatively difficult to understand. But in application like floating point representation it is not much required, and it takes more space.

- **Number extension:** In computer the size of a location is fixed, so it is often required to write a smaller number in a larger space, this can be done by sign extension, where, if the number is positive extension is done same as that of sign magnitude extension. If the number is -ve then we must write the number as far right as possible and then all the empty cells must be filled with 1(in negative logic 0 is represented by 1).

KNOWLEDGEGATE

- Subtraction using 2’s Complement arithmetic

- 9-4 =

- 7-11 =

KNOWLEDGEGATE

www.knowledgegate.in

Gray Code

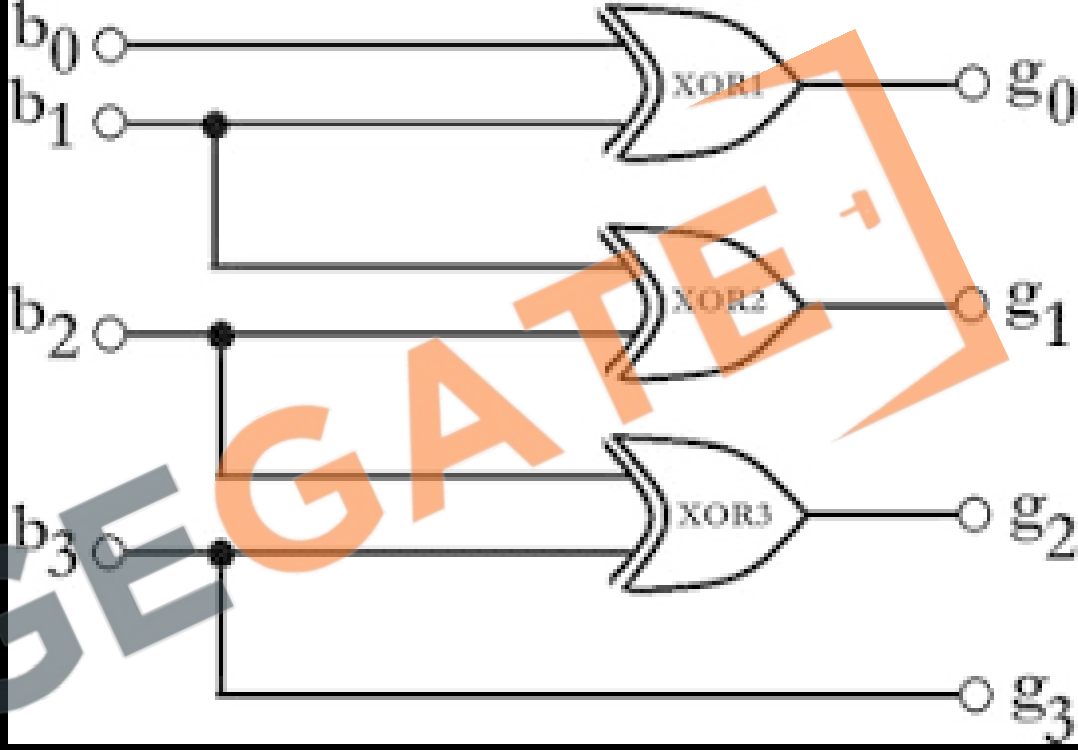
- Gray Code is a binary numbering system where two successive numbers differ in only one bit. It's essential because it reduces errors in digital systems, particularly in analog-to-digital conversions.

- Applications include rotary encoders in mechanical systems and minimizing errors in multi-bit data transfer. Gray Code contributes to system reliability by ensuring that only one bit changes at a time, reducing the risk of transitional errors.

- It's particularly useful in situations where an error in multiple bits changing at once could be catastrophic, like in control systems.

KNOWLEDGEGATE

1 1 0 0 1 1 1 0



KNOWLEDGEGATE

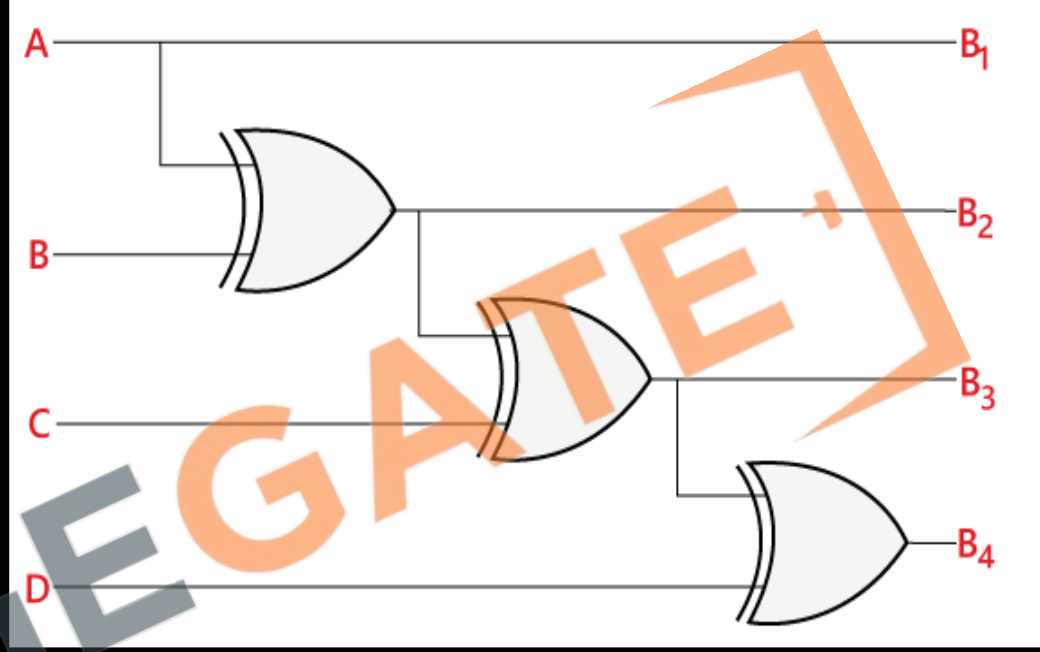
www.knowledgegate.in

- $G_0 = B_1 \oplus B_0$
- $G_1 = B_2 \oplus B_1$
- $G_2 = B_3 \oplus B_2$
- $G_3 = B_3$

4-bit binary				4-bit gray			
B_4	B_3	B_2	B_1	G_4	G_3	G_2	G_1
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

KNOWLEDGE GATE
www

1 1 0 0 1 1 1 0



KNOWLEDGEGATE

www.knowledgegate.in

- $B_0 = B_1 \oplus G_0$
- $B_1 = B_2 \oplus G_1$
- $B_2 = B_3 \oplus G_2$
- $B_3 = G_3$

G3	G2	G1	G0	B3	B2	B1	B0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
0	1	0	1	0	1	1	0
0	1	0	0	0	1	1	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	1	1	0	1	0
1	1	1	0	1	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	0	0	1	1	1	1	0
1	0	0	0	1	1	1	1

KNOWLEDGEGATE
www.knowledgegate.in

Binary-Coded Decimal Code (BCD)

- BCD (Binary-Coded Decimal) is a class of binary encodings where each decimal digit is represented by a fixed number of binary digits, usually four. It's needed because it simplifies the process of converting between binary and decimal numbers, making calculations more accurate and easier to check.
- Applications of BCD are widespread, including in digital clocks, calculators, and financial systems where decimal representation is critical for accuracy. It is also useful in systems that require easy conversion to human-readable decimal numbers.
- BCD contributes to computational efficiency and accuracy. It makes it simpler to convert between binary and decimal, reducing the chance of errors that could occur in calculations or data transfers.

KNOWLEDGE GATE

Decimal	B_3	B_2	B_1	B_0	D_3	D_2	D_1	D_0	D_3	D_2	D_1	D_0
0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	0	0	0	0	1
2	0	0	1	0	0	0	0	0	0	0	1	0
3	0	0	1	1	0	0	0	0	0	0	1	1
4	0	1	0	0	0	0	0	0	0	1	0	0
5	0	1	0	1	0	0	0	0	0	1	0	1
6	0	1	1	0	0	0	0	0	0	1	1	0
7	0	1	1	1	0	0	0	0	0	1	1	1
8	1	0	0	0	0	0	0	0	1	0	0	0
9	1	0	0	1	0	0	0	0	1	0	0	1
10	1	0	1	0	0	0	0	1	0	0	0	0
11	1	0	1	1	0	0	0	1	0	0	0	1
12	1	1	0	0	0	0	0	1	0	0	1	0
13	1	1	0	1	0	0	0	1	0	0	1	1
14	1	1	1	0	0	0	0	1	0	1	0	0
15	1	1	1	1	0	0	0	1	0	1	0	1

KNOWLEDGE GATE
www.knowledgegate.in

- Represent the unsigned decimal number 965 and 672 in BCD and then show the steps necessary to find their sum?

965	1001	0110	0101
672	0110	0111	0010
FD7	1111	1101	0111

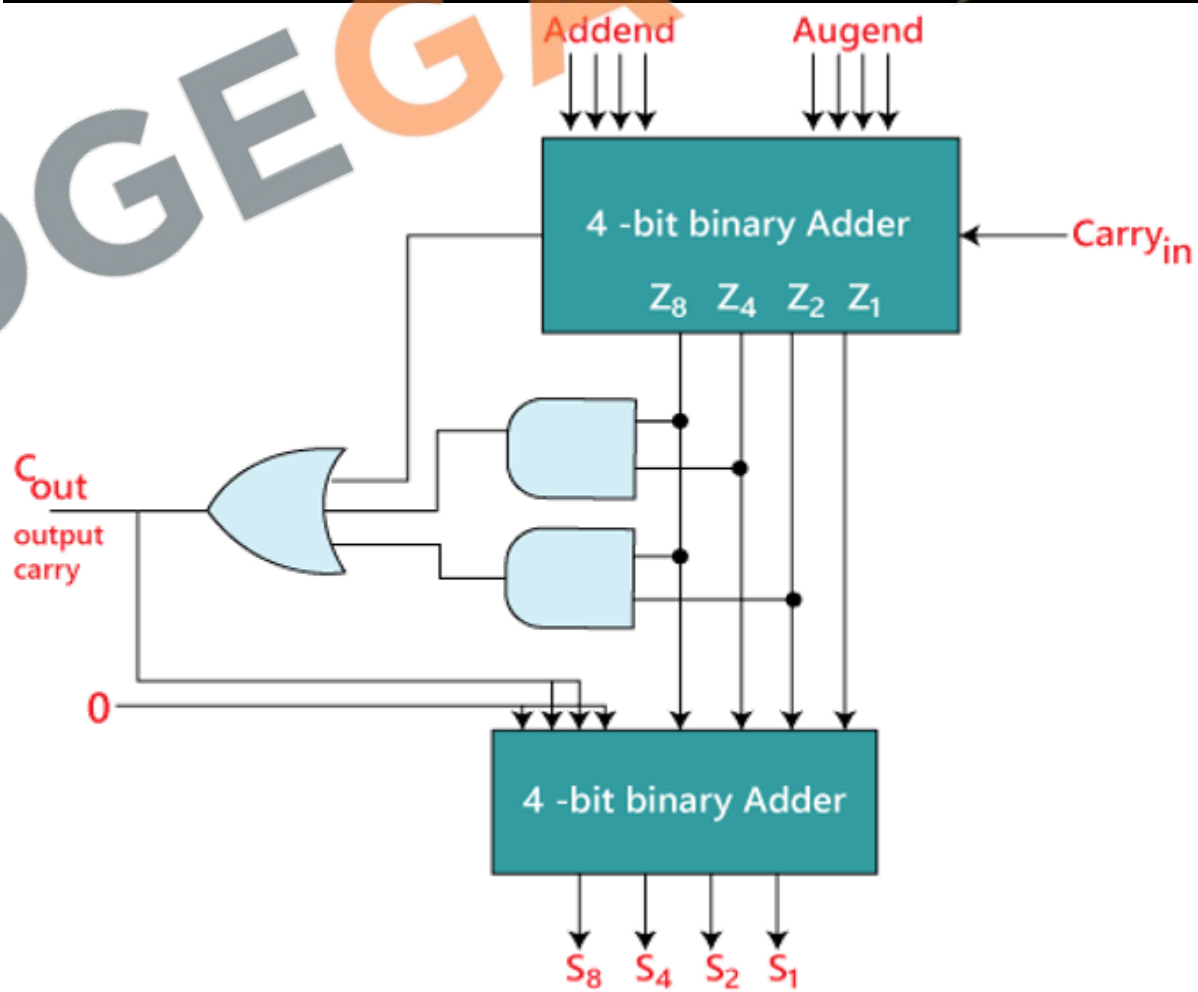
FD7	1111	1101	0111
+	0110	0110	0000
0001	0110	0011	0111
1	6	3	7

If sum > 9 then add 6 for correction

KNOWLEDGEGATE
www.knowledgegate.in

Decimal	Binary Sum					BCD Sum				
	C'	S3'	S2'	S1'	S0'	C	S3	S2	S1	S0
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	1
2	0	0	0	1	0	0	0	0	1	0
3	0	0	0	1	1	0	0	0	1	1
4	0	0	1	0	0	0	0	1	0	0
5	0	0	1	0	1	0	0	1	0	1
6	0	0	1	1	0	0	0	1	1	0
7	0	0	1	1	1	0	0	1	1	1
8	0	1	0	0	0	0	1	0	0	0
9	0	1	0	0	1	0	1	0	0	1
10	0	1	0	1	0	1	0	0	0	0
11	0	1	0	1	1	1	0	0	0	1
12	0	1	1	0	0	1	0	0	1	0
13	0	1	1	0	1	1	0	0	1	1
14	0	1	1	1	0	1	0	1	0	0
15	0	1	1	1	1	1	0	1	0	1
16	1	0	0	0	0	1	0	1	1	0
17	1	0	0	0	1	1	0	1	1	1
18	1	0	0	1	0	1	1	0	0	0
19	1	0	0	1	1	1	1	0	0	1

$$C = C' + S'_3S'_2 + S'_3S'_1$$



KNOWLEDGEGATE

Excess-3 Code

- Excess-3 Code is a binary-coded decimal (BCD) system that represents each digit of a decimal number by its corresponding 4-bit binary representation, but adds an offset of 3 to each digit. This makes arithmetic operations like addition and subtraction easier to implement.
- Excess-3 Code is primarily used in older computer systems and some types of calculators. It was important for minimizing hardware in early digital systems, where computational resources were limited.
- The main contribution of Excess-3 Code is that it simplifies the hardware required for arithmetic operations. Its self-complementing feature helps in carrying out subtraction by addition, thereby reducing computational complexity.

Decimal	Excess-3
0	0011
1	0100
2	0101
3	0110
4	0111
5	1000
6	1001
7	1010
8	1011
9	1100

KNOWLEDGE GATE

- Example: 395 in decimal will be represented in excess-3 code as:

  0110 1100 1000.

  3 9 5

- 3 in excess-3 representation is: 3 + 3 = 6 (0110)
- 9 in excess-3 representation is: 9 + 3 = 12 (1100)
- 5 in excess-3 representation is: 5 + 3 = 8 (1000)

- First, convert the decimal digits to their 4-bit Excess-3 codes:
  - 5 in Excess-3: 5 + 3 = 8 in decimal, which is 1000 in binary.
  - 7 in Excess-3: 7 + 3 = 10 in decimal, which is 1010 in binary.
- Add the Excess-3 coded numbers:
  - 1000 + 1010 = 10010 in binary, which is 18 in decimal.
- Now, subtract 3 from the sum to remove the excess and get the real sum:
  - 18 − 3 = 15
  - 15 in Excess-3 code, real sum is 12

KNOWLEDGEGATE

Module

Free Semester One Shot Exam Course

Topic

Computer Organization and Architecture

Subtopic

COA One Shot

Lesson

COA_Notes

Video chapters

Chapter-1 (Introduction): Boolean Algebra, Types of Computer, Functional units of digital system and their interconnections, buses, bus architecture, types of buses and bus arbitration. Register, bus and memory transfer. Processor organization, general registers organization, stack organization and addressing modes.

Chapter-2 (Arithmetic and logic unit) : Look ahead carries adders. Multiplication: Signed operand multiplication, Booth's algorithm and array multiplier. Division and logic operations. Floating point arithmetic operation, Arithmetic & logic unit design. IEEE Standard for Floating Point Numbers

Chapter-3 (Control Unit) : Instruction types, formats, instruction cycles and sub cycles (fetch and execute etc), micro-operations, execution of a complete instruction. Program Control, Reduced Instruction Set Computer,. Hardwire and micro programmed control: micro programme sequencing, concept of horizontal and vertical microprogramming.

Chapter-4 (Memory) : Basic concept and hierarchy, semiconductor RAM **memories**, 2D & 2 1/2D memory organization. ROM memories. Cache memories: concept and design issues & performance, address mapping and replacement Auxiliary memories: magnetic disk, magnetic tape and optical disks Virtual memory: concept implementation.

Chapter-5 (Input / Output) : Peripheral devices, 1/0 interface, 1/0 ports, Interrupts: interrupt hardware, types of interrupts and exceptions. Modes of Data Transfer: Programmed 1/0, interrupt initiated 1/0 and Direct Memory Access., 1/0 channels and processors. Serial Communication: Synchronous & asynchronous communication, standard communication interfaces.

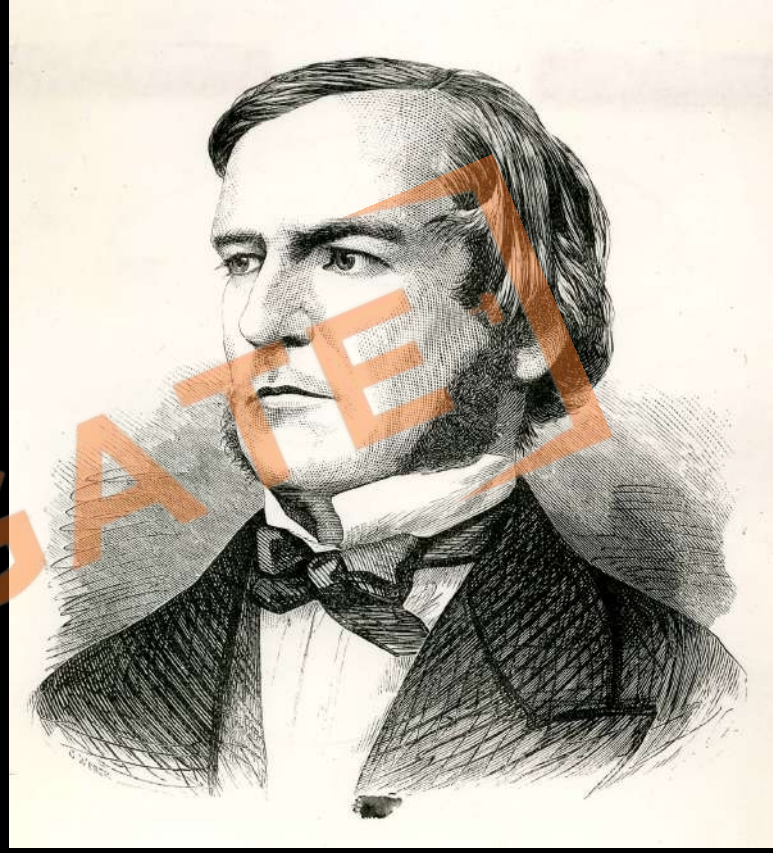
Chapter-6 (Pipelining): Uniprocessing, Multiprocessing, Pipelining, Speed UP, Structural hazards, Control hazards, Data hazards, Operand Forwarding.

KNOWLEDGE GATE

www.knowledgegate.in

Boolean algebra

- The signal in most present day electronic digital system uses just two discrete values and are therefore said to be binary.
- Boolean algebra was introduced by George Boole in his first book The Mathematical Analysis of Logic (1847), and set forth more fully in his An Investigation of the Laws of Thought (1854).
- George boole introduced the concept of binary number system in the studies of the mathematical theory of logic and developed its algebra known as Boolean algebra.



KNOWLEDGEGATE

Boolean algebra

1. In mathematics and mathematical logic, Boolean algebra is the branch of algebra in which the values of the variables are the truth values true and false, usually denoted 1 and 0 respectively.

2. Instead of elementary algebra where the values of the variables are numbers, and the prime operations are addition and multiplication. The main operations of Boolean algebra are
   - The conjunction and denoted as ∧
   - The disjunction or denoted as ∨
   - The negation not denoted as ¬

KNOWLEDGEGATE

www.knowledgegate.in

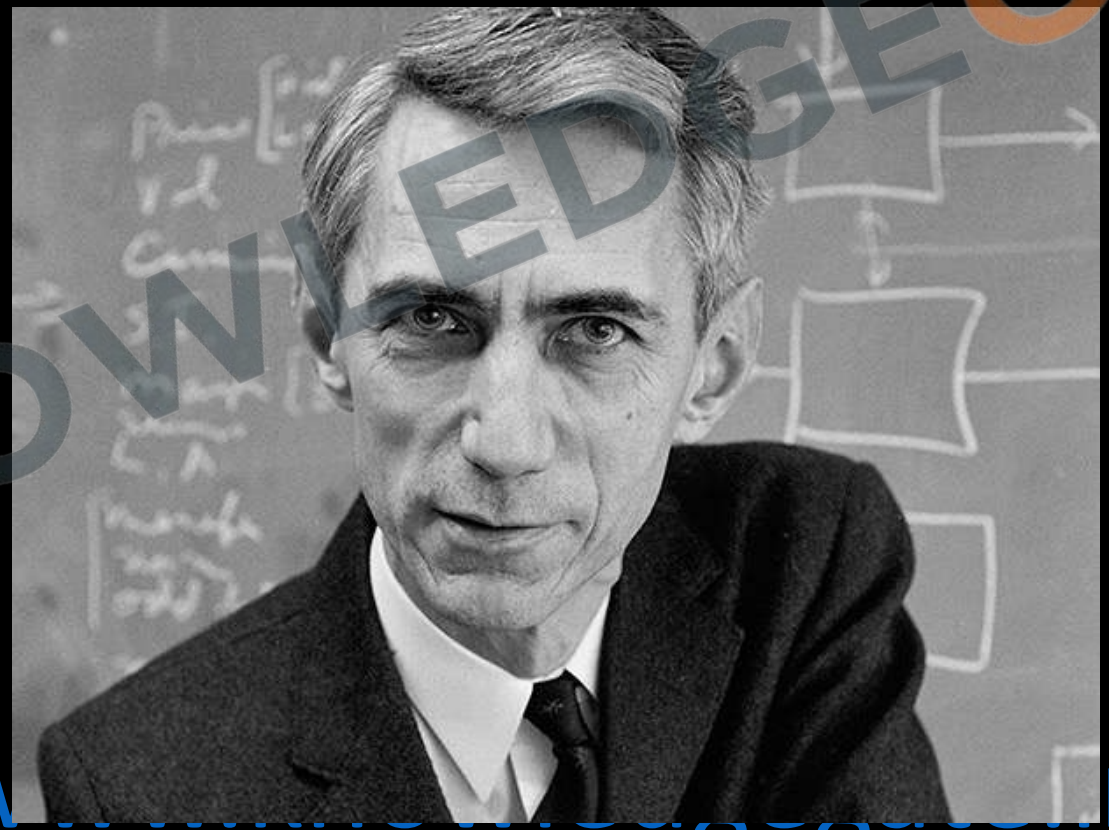
Turing Machine

- The Church-Turing thesis states that any algorithmic procedure that can be carried out by human beings/computer can be carried out by a Turing machine.(1936)
- It has been universally accepted by computer scientists that the Turing machine provides an ideal theoretical model of a computer.

KNOWLEDGEGATE

Digital System

- In the 1930s, while studying switching circuits, Claude Shannon observed that one could also apply the rules of Boole's algebra in this setting, and he introduced switching algebra as a way to analyze and design circuits by algebraic means in terms of logic gates.
- These logic concepts have been adapted for the design of digital hardware since 1938 Claude Shannon (father of information theory), organized and systematized Boole’s work.



KNOWLEDGE GATE

- Shannon already had at his disposal the Boolean algebra, thus he cast his switching algebra as the two-element Boolean algebra. Efficient implementation of Boolean functions is a fundamental problem in the design of combinational logic circuits.

- Boolean constants are denoted by 0 or 1. Boolean variables are quantities that can take different values at different times. They may represent input, output or intermediate signals.

- Here we can have n number of variables usually written as a, b, c…. (lower case) and it satisfy all Boolean laws, which will be discussed later.

KNOWLEDGEGATE

www.knowledgegate.in

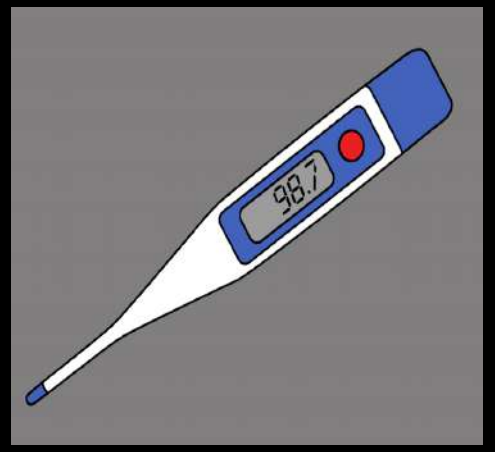
- Historically there have been 2 types of Computers:
  - **Fixed Program Computers / Dedicated device / Embedded system –** Their function is very specific and they couldn't be reprogrammed, e.g. Calculators washing machine.
  - **Stored Program Computers / General purpose computer / von Neumann architecture** These can be programmed to carry out many different tasks, applications are stored on them, hence the name.

KNOWLEDGEGATE

www.knowledgegate.in

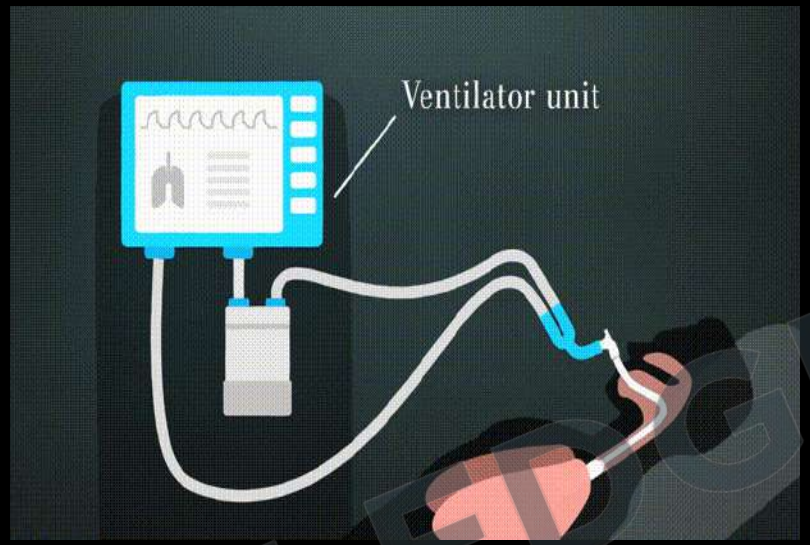
Fixed Program Computers / Dedicated device / Embedded system

- They are designed to perform a specific task their functionality is written in terms of program which is permanently fused in a chipset. E.g. washing machine, microwave etc.



KNOWLEDGEGATE

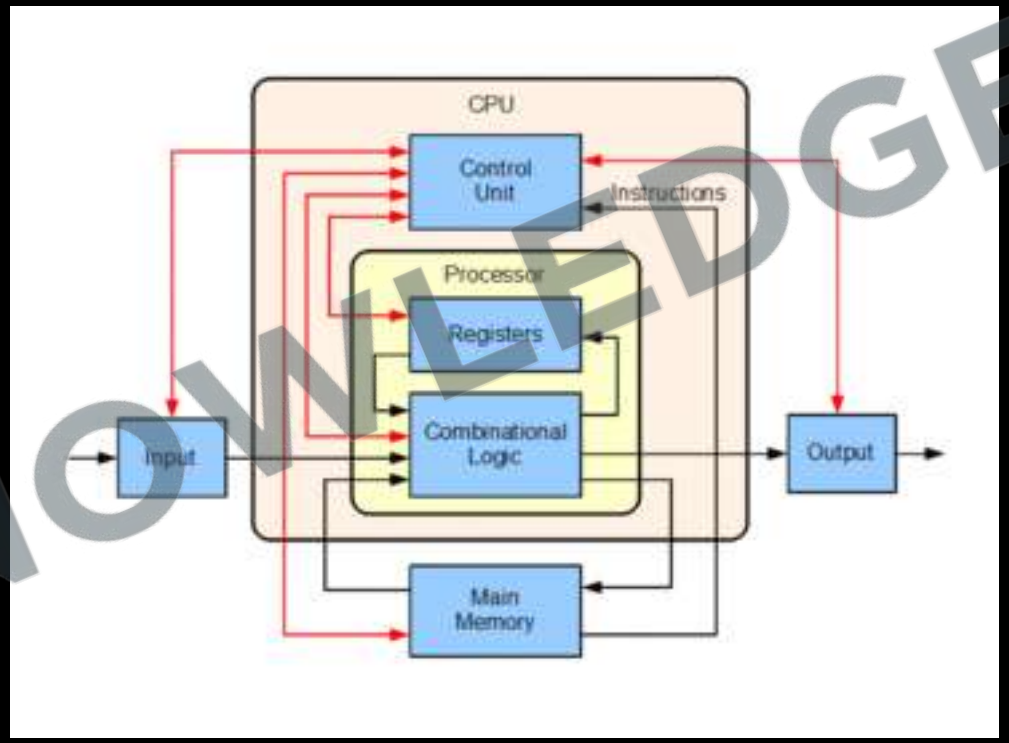
- Embedded system are kind of simple computers designed to do Specific task for e.g. microwave, washing machine etc., they have small microprocessors inside them. These microprocessors are hardwired and can not do everything, except specific task.



KNOWLEDGE GATE

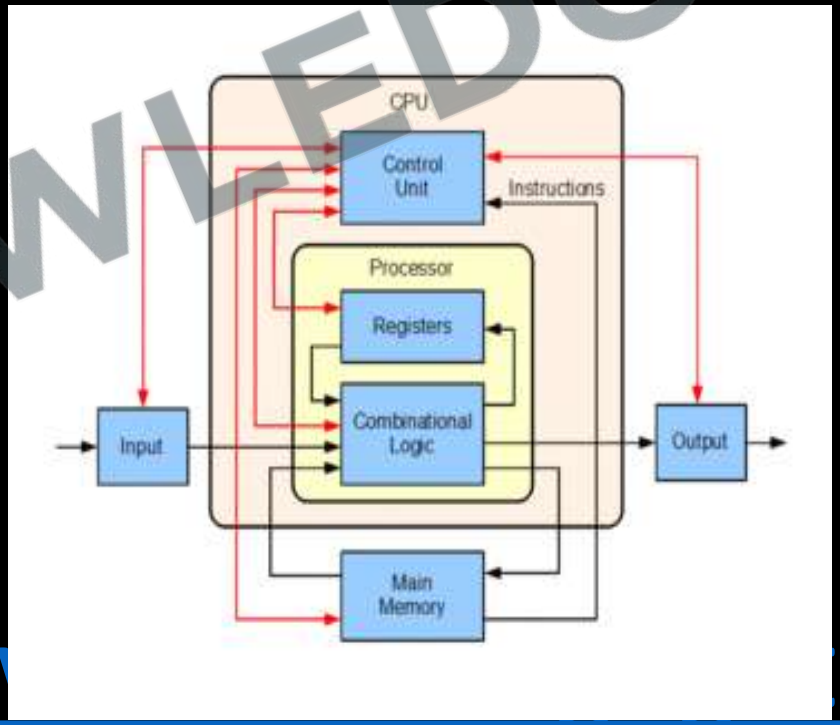
Stored Program Computers / General purpose computer / von Neumann architecture

- Modern computers are based on a stored-program concept introduced by John Von Neumann, Which can perform anything which can be theoretically done by a Turing machine. These computer works on stored program concept where a programmer, writes a program store it in the memory and then computer executes it, based on the requirement program can be changed and so does the functionality.



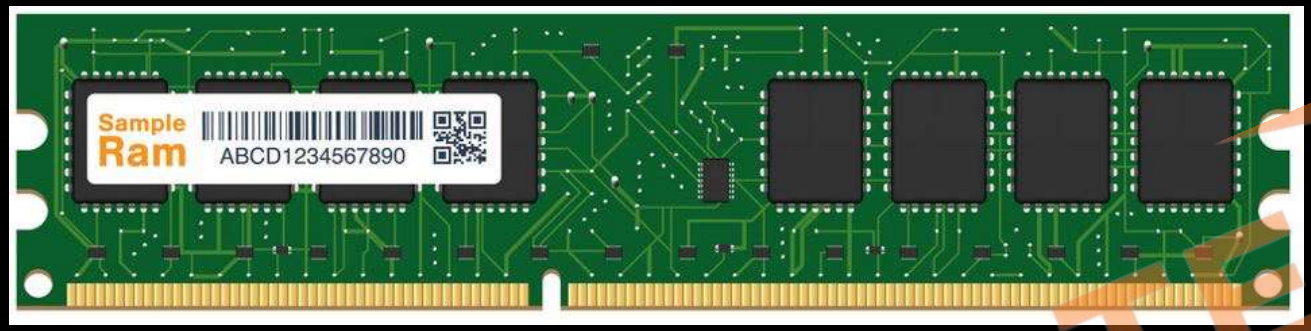
KNOWLEDGEGATE

- **Machine / CPU architecture: -** In general a CPU contain 3 important components
  - **Control unit: -** Which generates control signal (master generator) to control every other part of the CPU. It directs all input and output flow, fetches code for instructions, and controls how data moves around the system.
  - **Registers: -** few important registers are in every processor like program counter which contains address of the next instruction then instruction register which contain address of the current register and base register which contain base address of program.
  - **ALU: -** ALU is a complex combination circuit which can perform arithmetic Operations, Bit Shifting Operations, and logical operation. e.g. Addition, Subtraction, Comparisons etc.

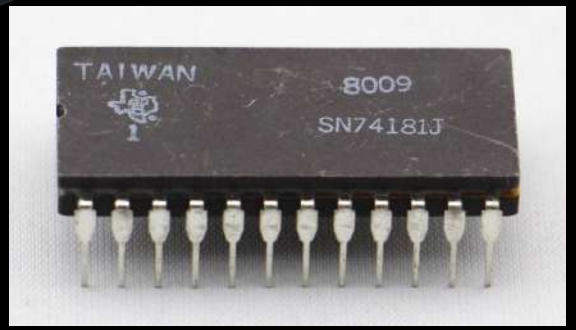


KNOWLEDGEGATE
www.knowledgegate.in

- Now the question is what are the main elements we need
  - Memory (store a program)



  - ALU (circuit which perform operations)

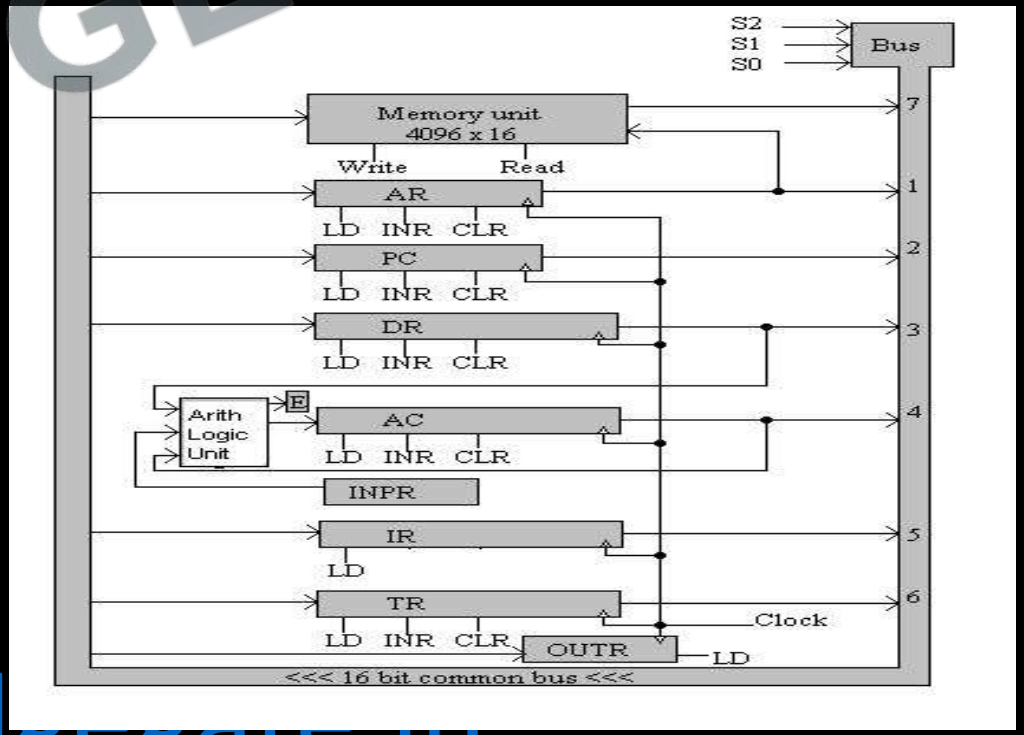


  - Register (fast memory, sequence of flip-flop) (load, clear, increment pin)

KNOWLEDGE GATE

- Timing circuit (sequence counter) (to order certain operations, like fetch, decode, execute), will generate timing signals

- Control unit will generate control signals-to select registers, select other circuit, to give inputs to registers, So we need a special unit called control unit, which will give signals to all the components



- Flags – one-bit information

- Bus – using which we will connect different component together, and perform data transfer using multiplexer

KNOWLEDGEGATE
www.knowledgegate.in

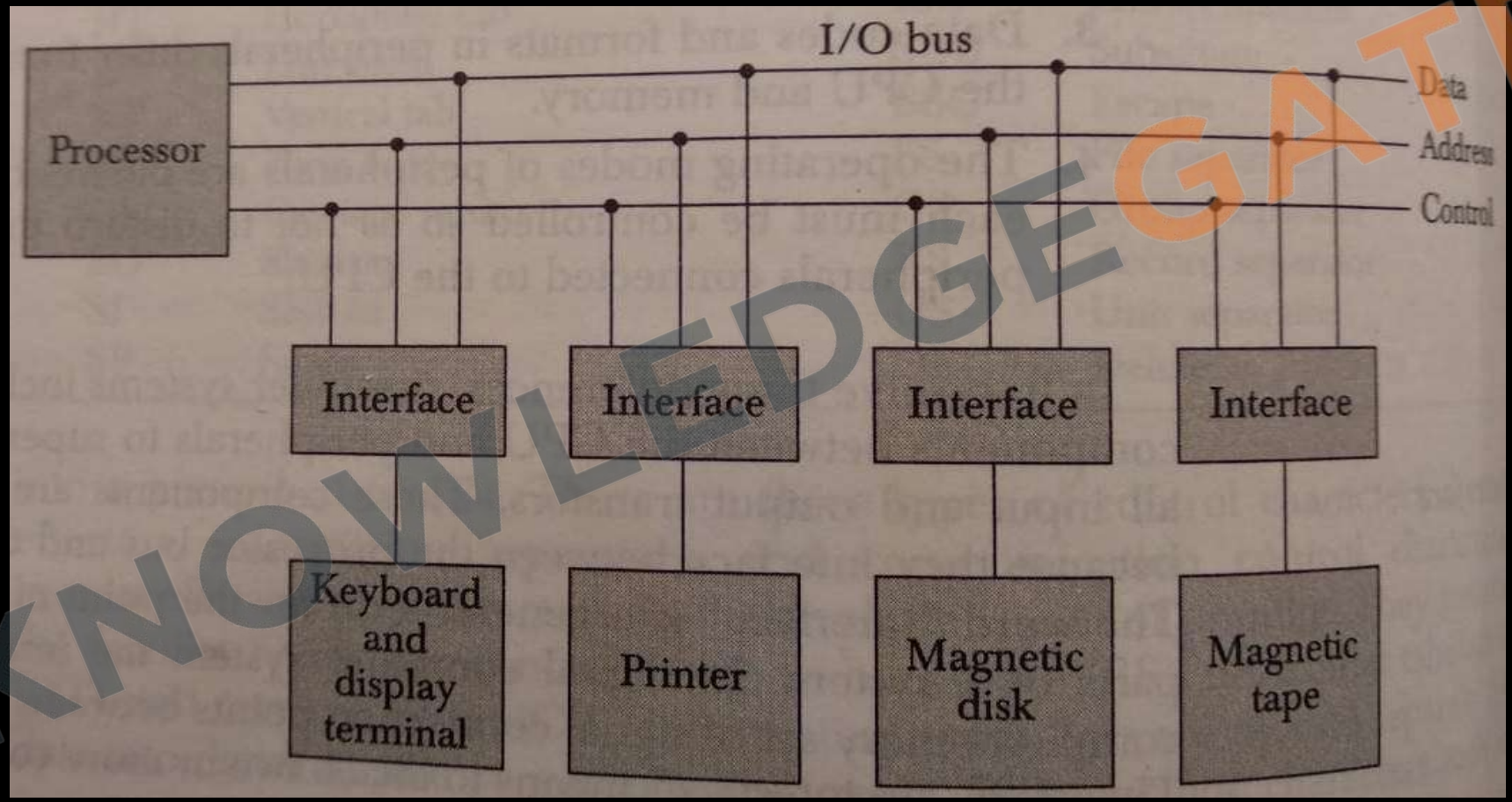
- how in general operation are performed?

  - ***memory -> register -> ALU (perform operation) register -> memory.***

KNOWLEDGEGATE

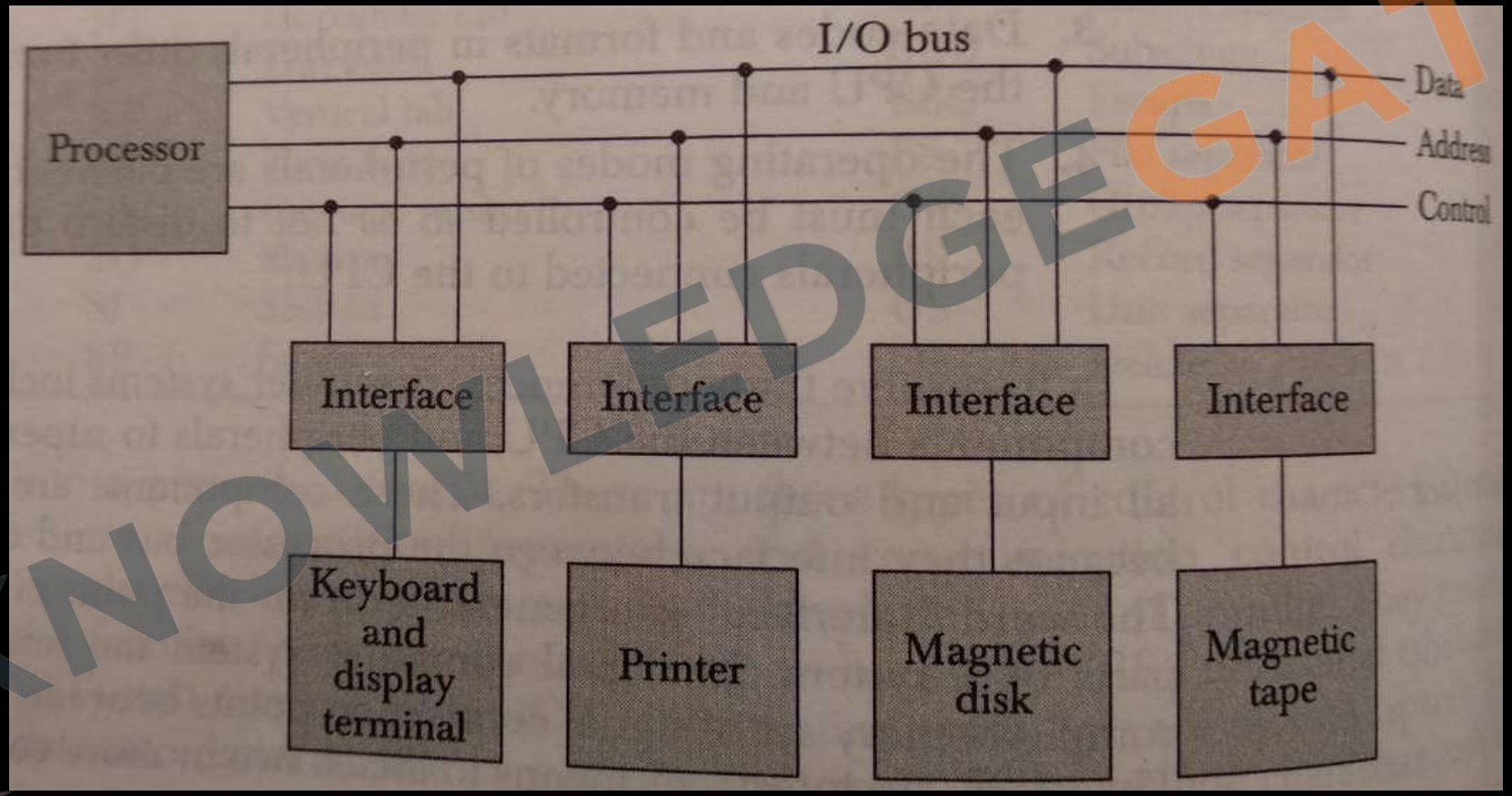
www.knowledgegate.in

- **Address bus: -** Is used to identify the correct i/o devices among the number of i/o device , so CPU put an address of a specific i/o device on the address line, all devices keep monitoring this address bus and decode it, and if it is a match then it activates control and data lines.



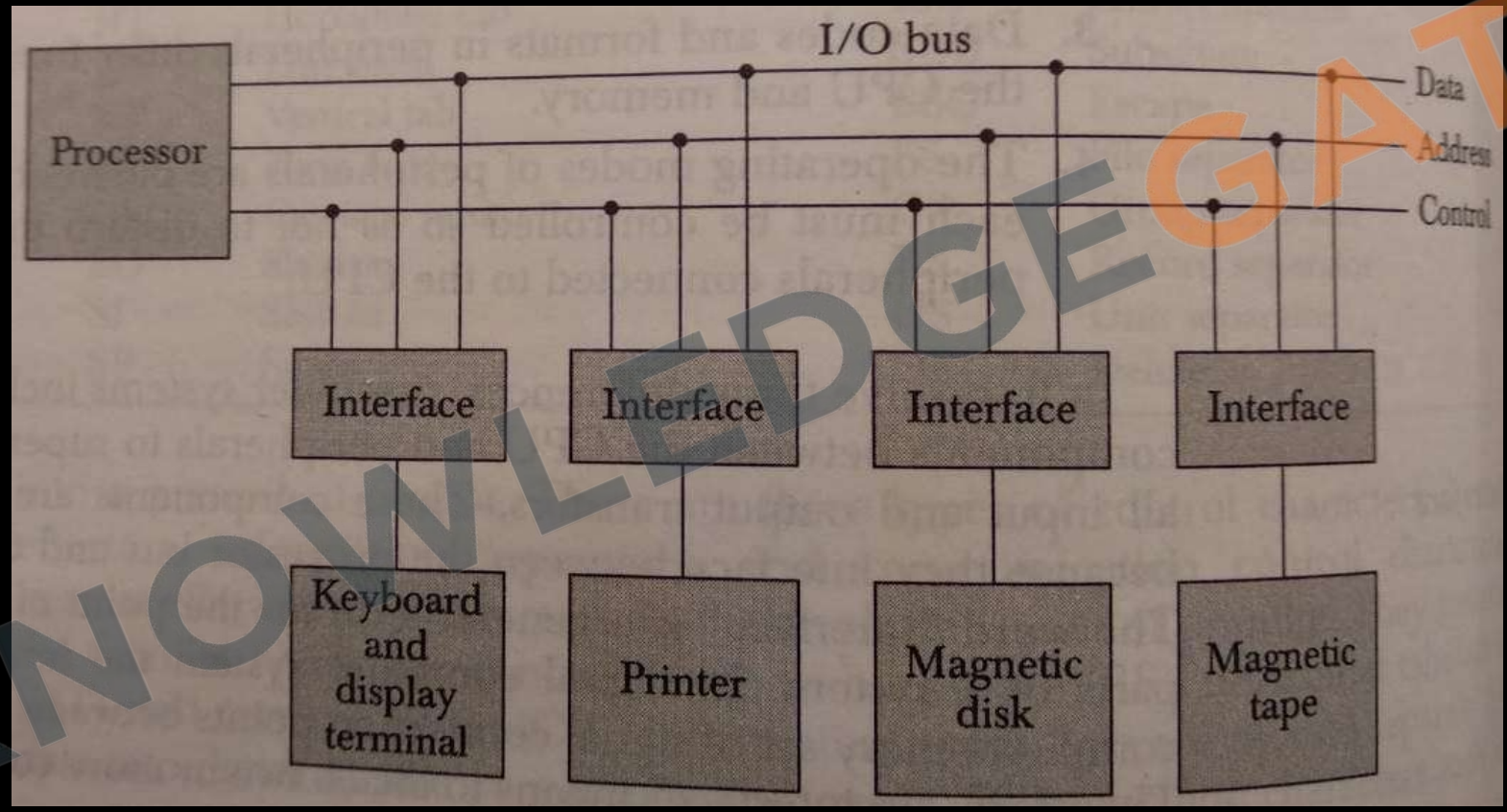
KNOWLEDGEGATE
www.knowledgegate.in

- **Control bus: -** After selecting a specific i/o device CPU sends a functional code on the control line. The selected device(interface) reads that functional code and execute it. E.g. i/o command, control command, status command etc.



KNOWLEDGEGATE
www.knowledgegate.in

- **Data bus:** - In the final step depending on the operation either CPU will put data on the data line and device will store it or device will put data on the data line and CPU will store it.



KNOWLEDGEGATE

www.knowledgegate.in

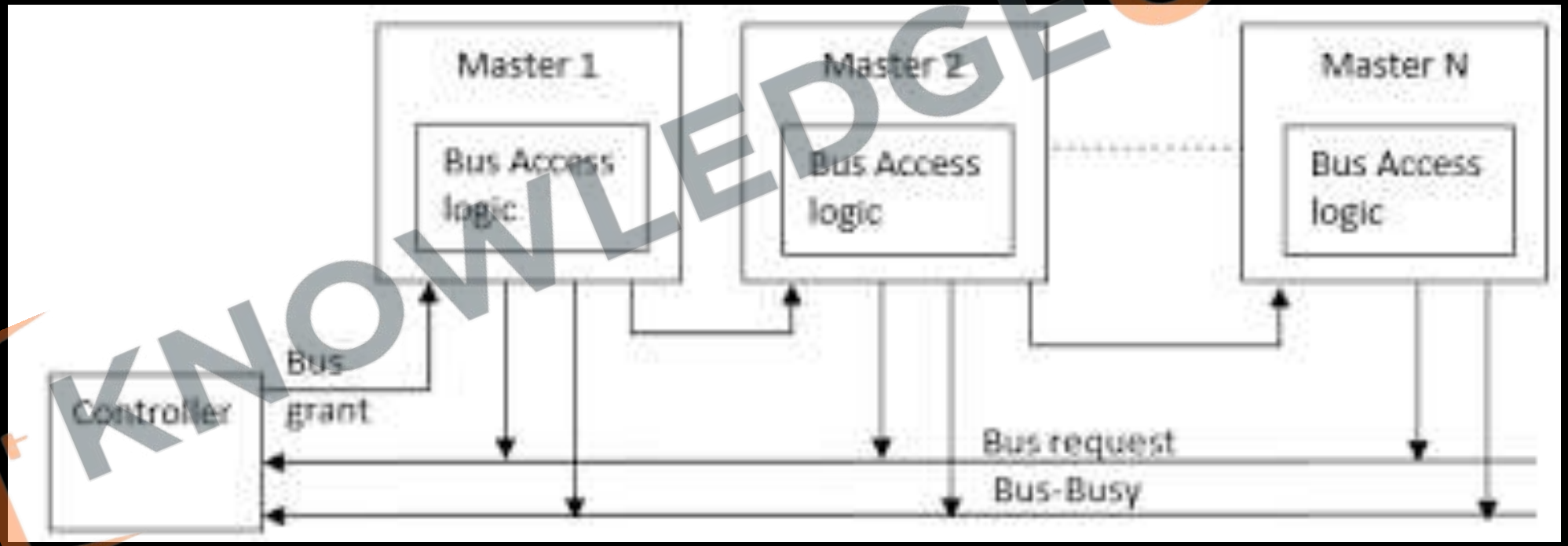
Bus Arbitration

- **Bus Arbitration**: This is the method used to decide which device gets access to the common bus when multiple devices request it simultaneously, ensuring data integrity and system stability.
- **Conflict Resolution**: Without bus arbitration, simultaneous access could result in data corruption and system malfunctions, making this mechanism essential for orderly and reliable data transfer.

KNOWLEDGEGATE

www.knowledgegate.in

- **Daisy Chaining method:** It is a simple and cheaper method where all the bus masters use the same line for making bus requests. The bus grant signal serially propagates through each master until it encounters the first one that is requesting access to the bus. This master blocks the propagation of the bus grant signal, therefore any other requesting module will not receive the grant signal and hence cannot access the bus.

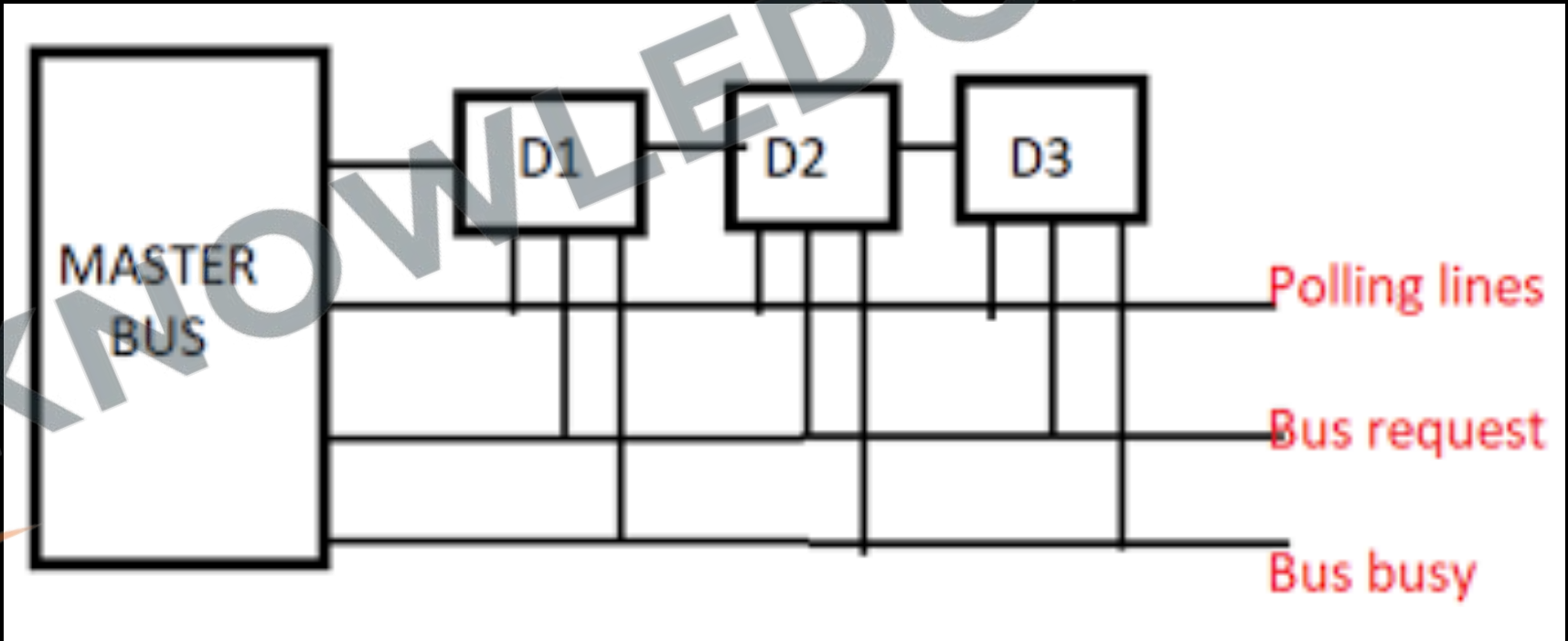


KNOWLEDGEGATE

www.knowledgegate.in

Polling

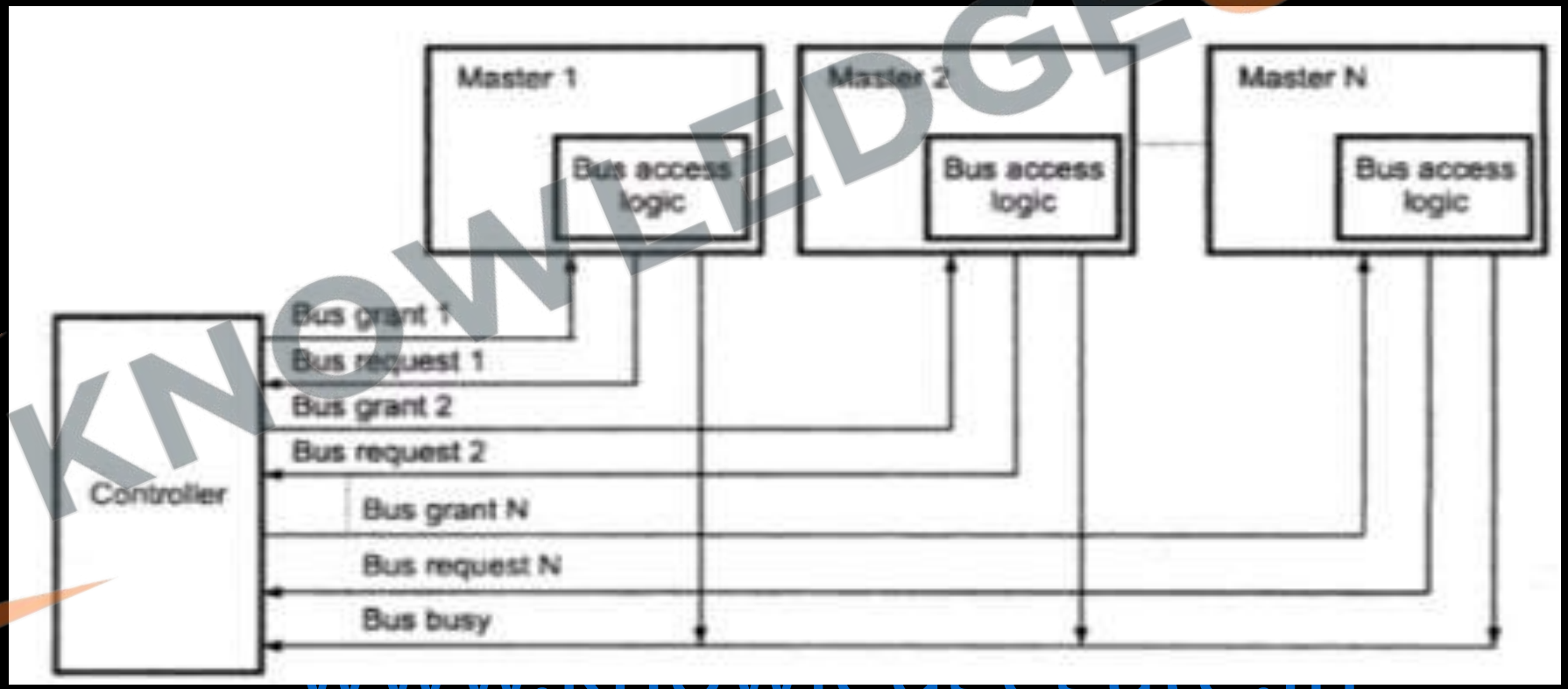
- **Address Generation:** In this method, a controller generates unique addresses for each master (device) based on their priority. The number of address lines correlates with the number of masters in the system.
- **Sequence & Activation**: The controller cycles through the generated addresses. When a master recognizes its own address, it activates a "busy" signal and gains access to the bus for data transfer.



KNOWLEDGEGATE
www.knowledgegate.in

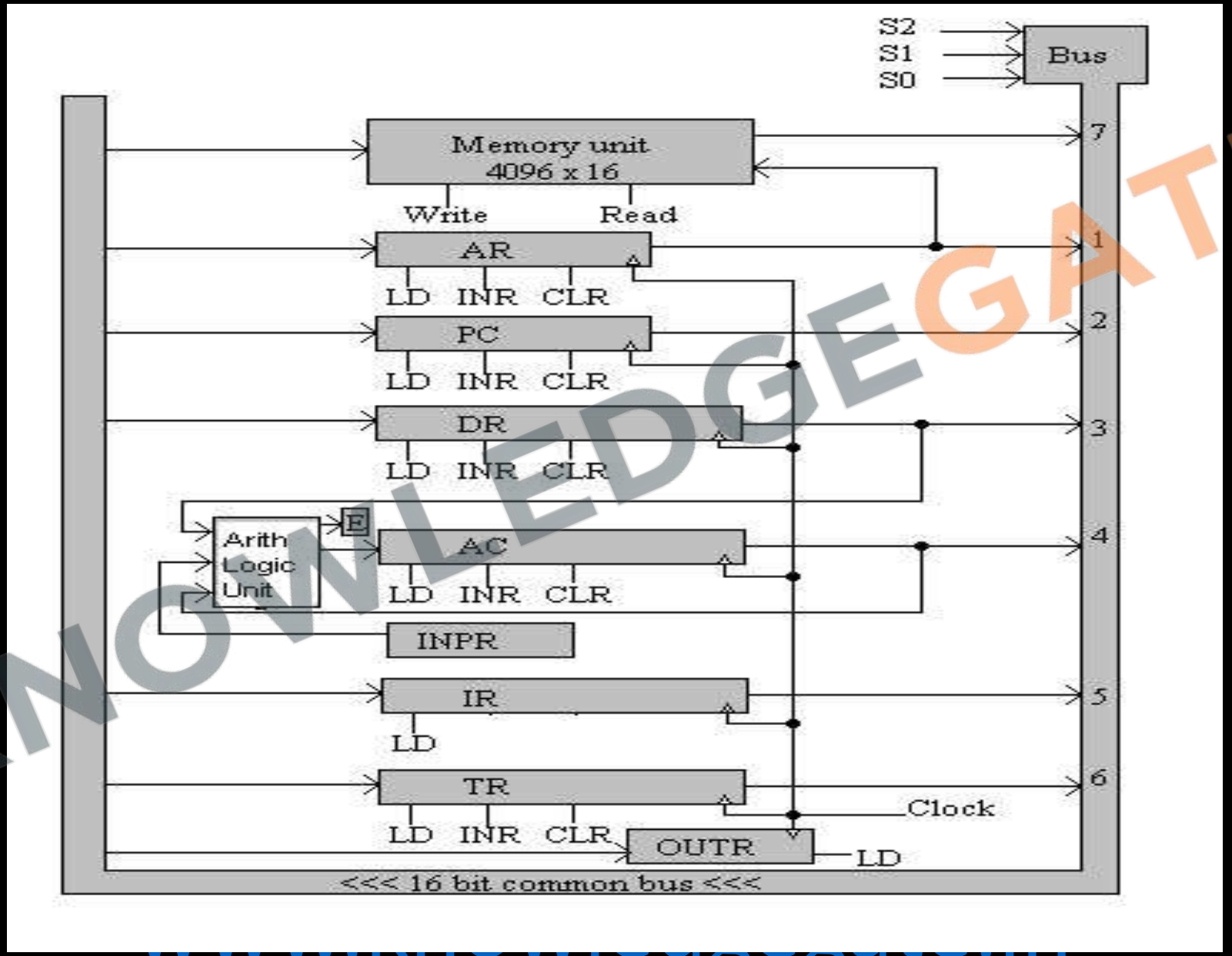
Fixed priority or Independent Request method

- In this, each master has a separate pair of bus request and bus grant lines and each pair has a priority assigned to it.
- The built-in priority decoder within the controller selects the highest priority request and asserts the corresponding bus grant signal.



KNOWLEDGEGATE
www.knowledgegate.in

Processor Organization



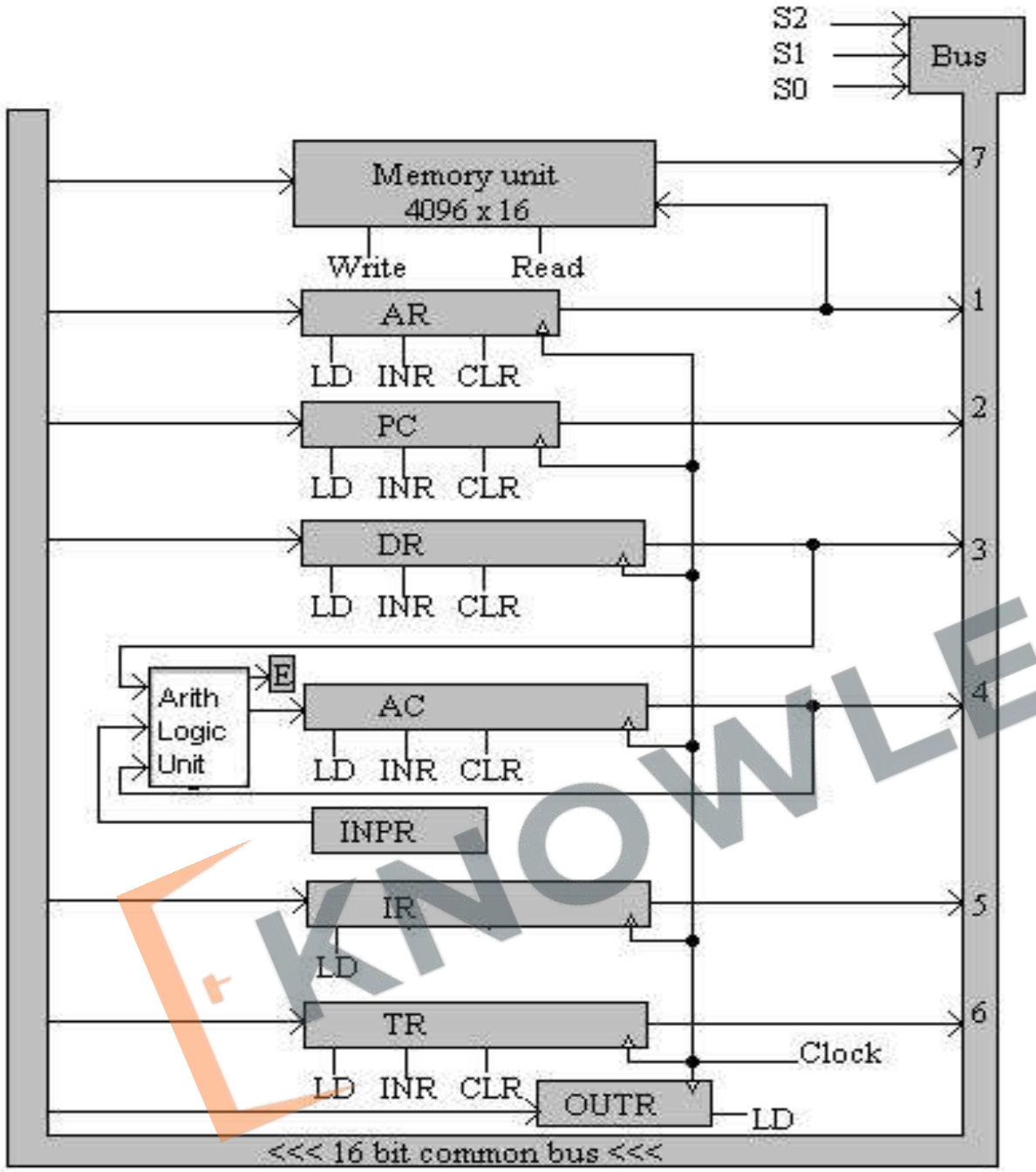
KNOWLEDGEGATE

Registers - Registers refer to high-speed storage areas in the CPU. The data processed by the CPU are fetched from the registers. There are different types of registers used in architecture.

Register symbol	Number of bits	Register name	Function
DR	16	Data register	Holds memory operand
AR	12	Address register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction register	Holds instruction code
PC	12	Program counter	Holds address of instruction
TR	16	Temporary register	Holds temporary data
INPR	8	Input register	Holds input character
OUTR	8	Output register	Holds output character

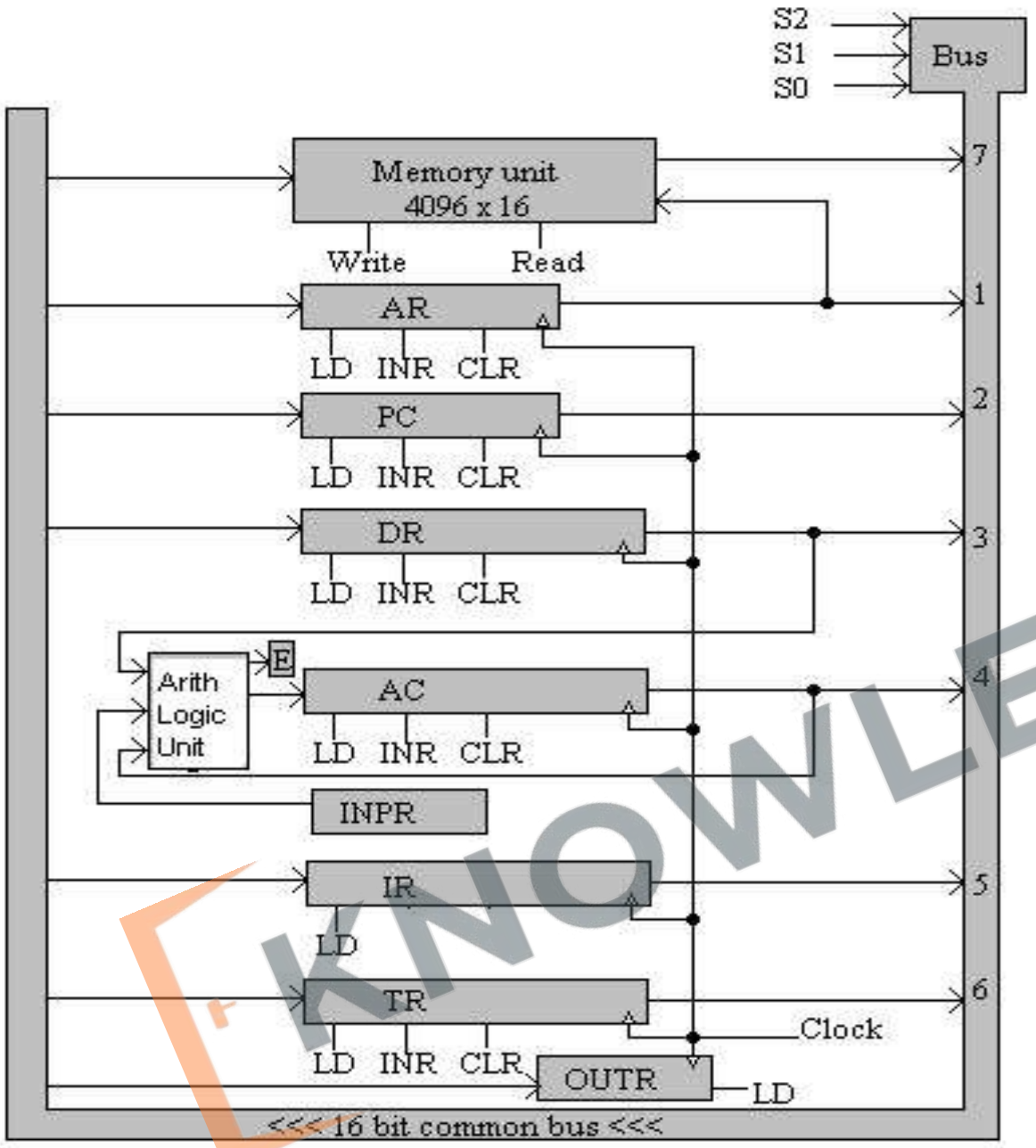
KNOWLEDGEGATE

www.knowledgegate.in



- The **data register (DR)** is a register which is used to store any data that is to be transferred to memory or which are fetched from memory.
- The **accumulator (AC)** register is a general-purpose processing register. Will hold the intermediate arithmetic and logic results of the operation performed in the ALU, as ALU is not directly connected to the memory.

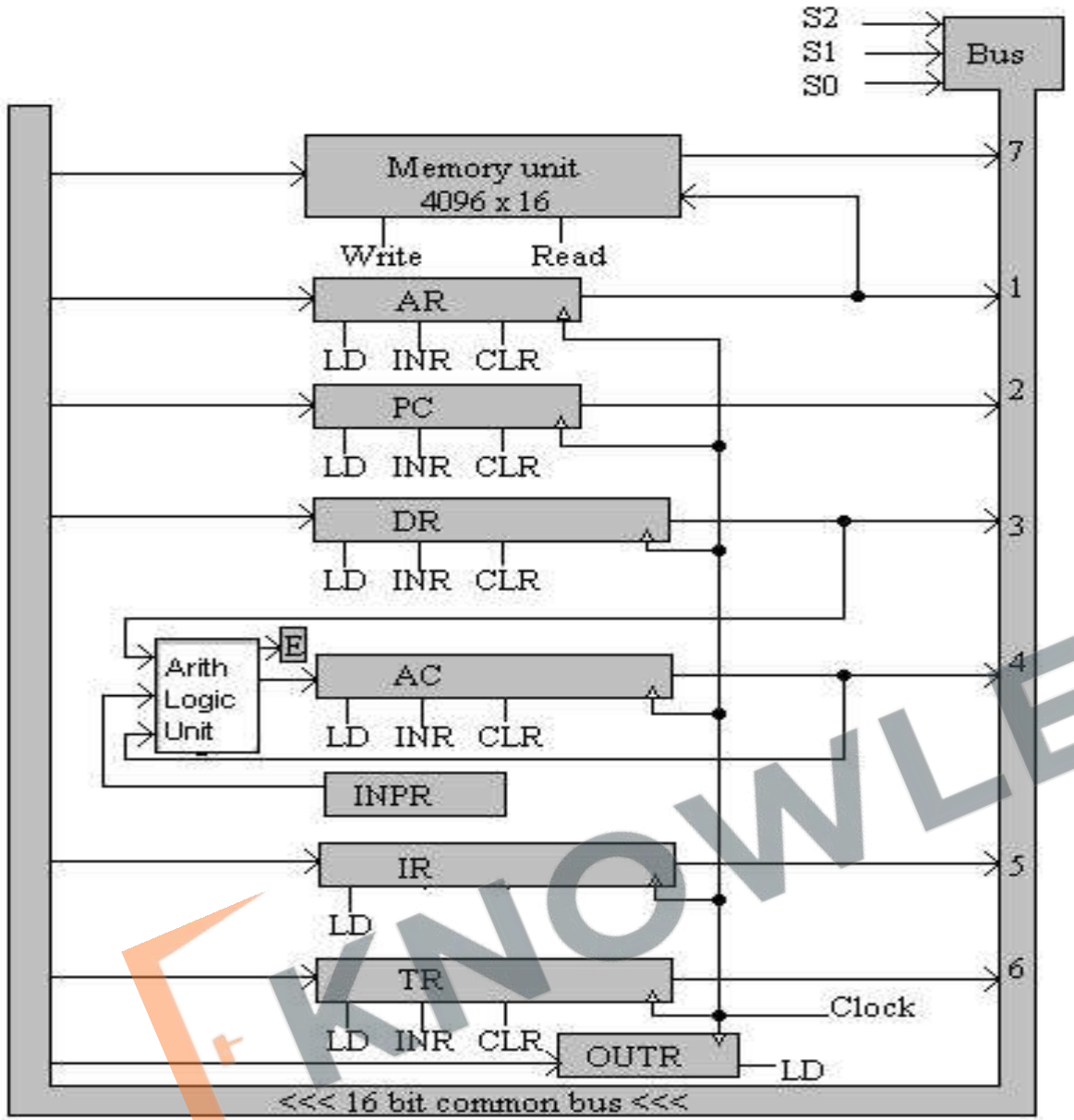
KNOWLEDGEGATE
edgegate.in



- **Instruction register**- is used to store instruction which we fetched from memory, so that we analyses the instruction using a decoder and can understand what instruction want to perform.

- The **temporary register (TR)** is used for holding temporary data during the processing.

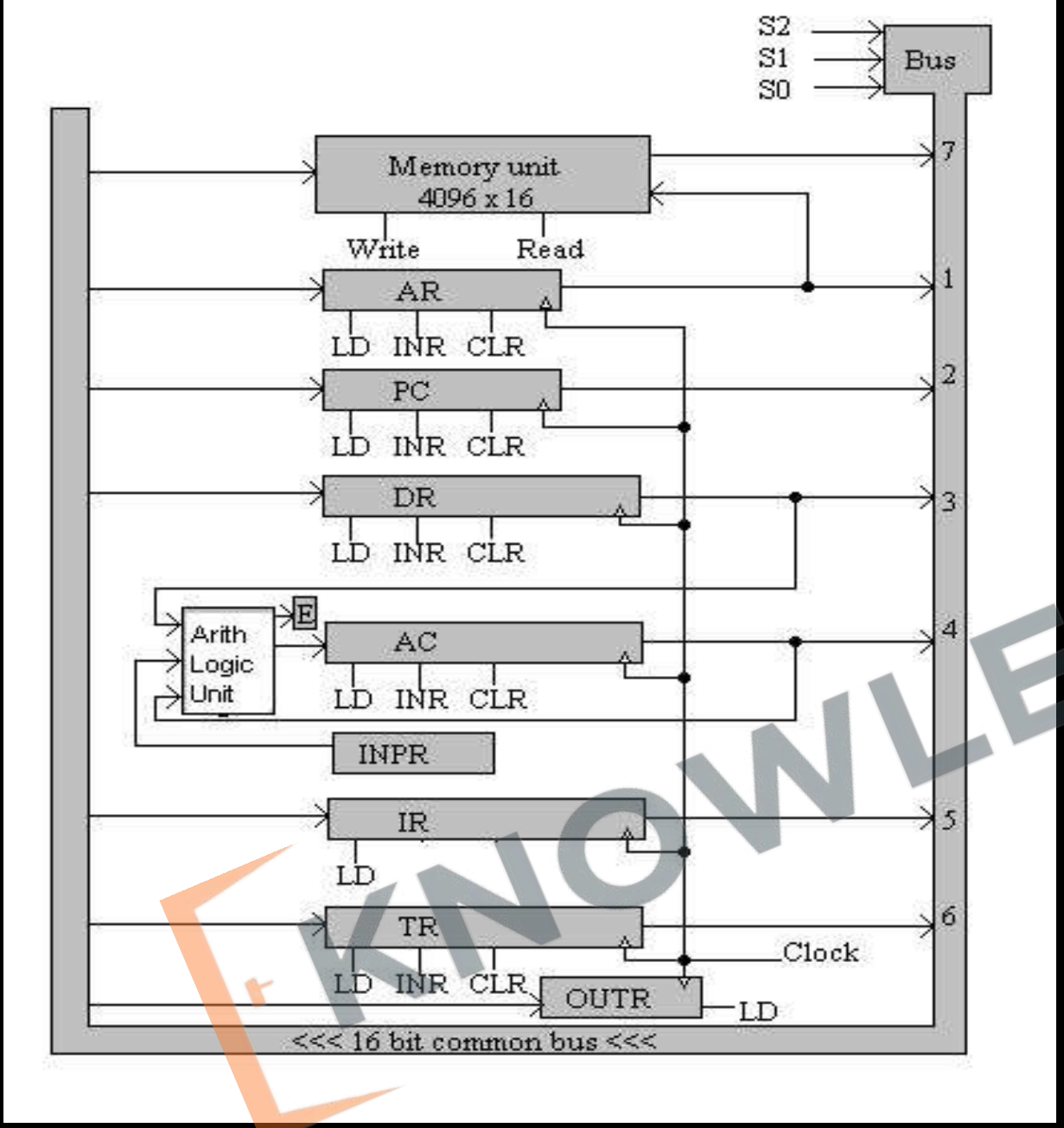
KNOWLEDGEGATE



- The memory **address register (AR)** has 12 bits since this is the width of a memory address (always used least significant bits of bus). It stores the memory locations of instructions or data that need to be fetched from memory or stored in memory.

- The **program counter (PC)** also has 12 bits and it holds the address of the next instruction to be read from memory after the current instruction is executed. The PC goes through a counting sequence and causes the computer to read sequential instructions previously stored in memory.

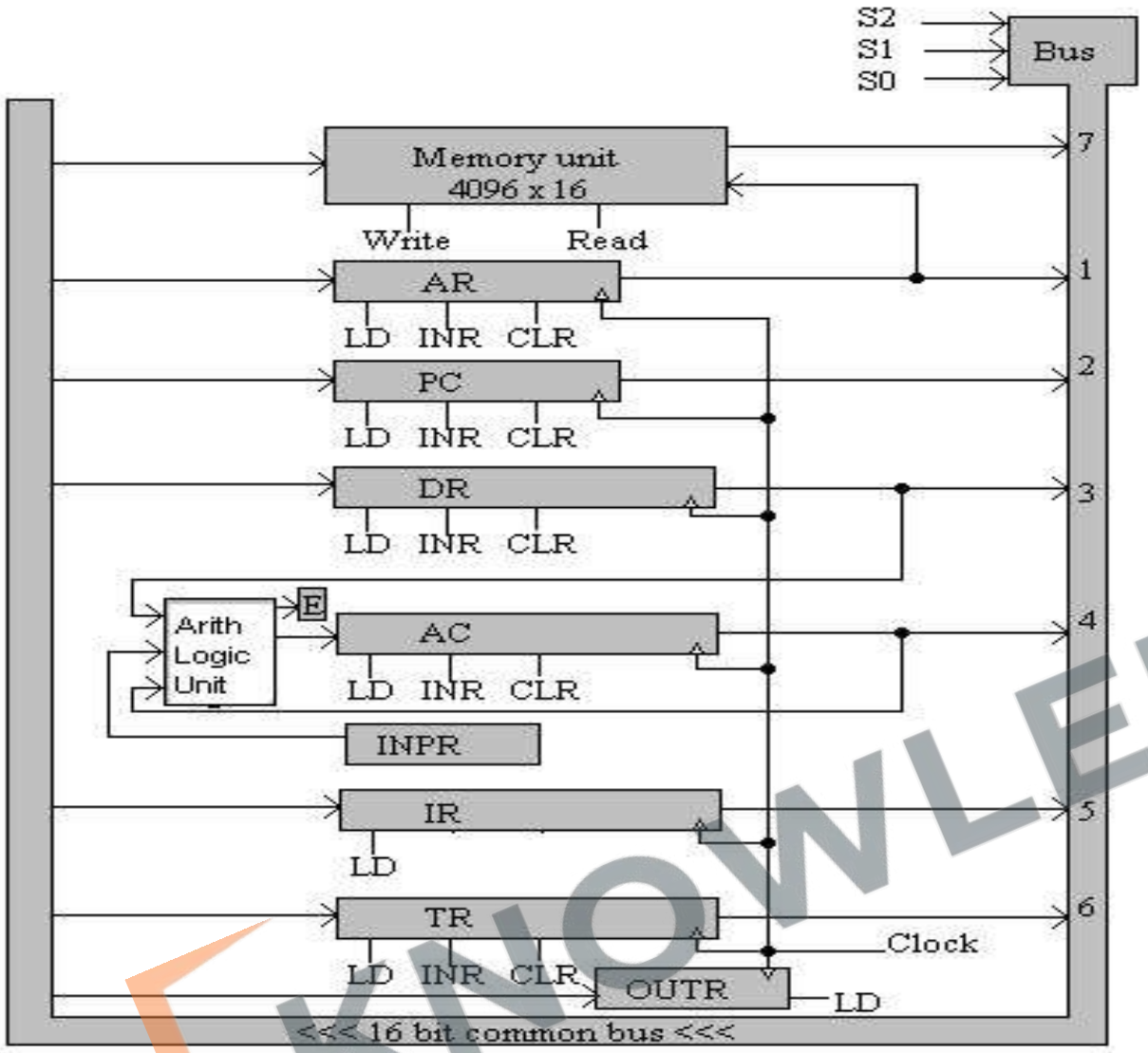
KNOWLEDGEGATE



- Two registers are used for input and output. The **input register (INPR)** receives an 8-bit character from an input device, and pass it to ALU then accumulator and then to memory.

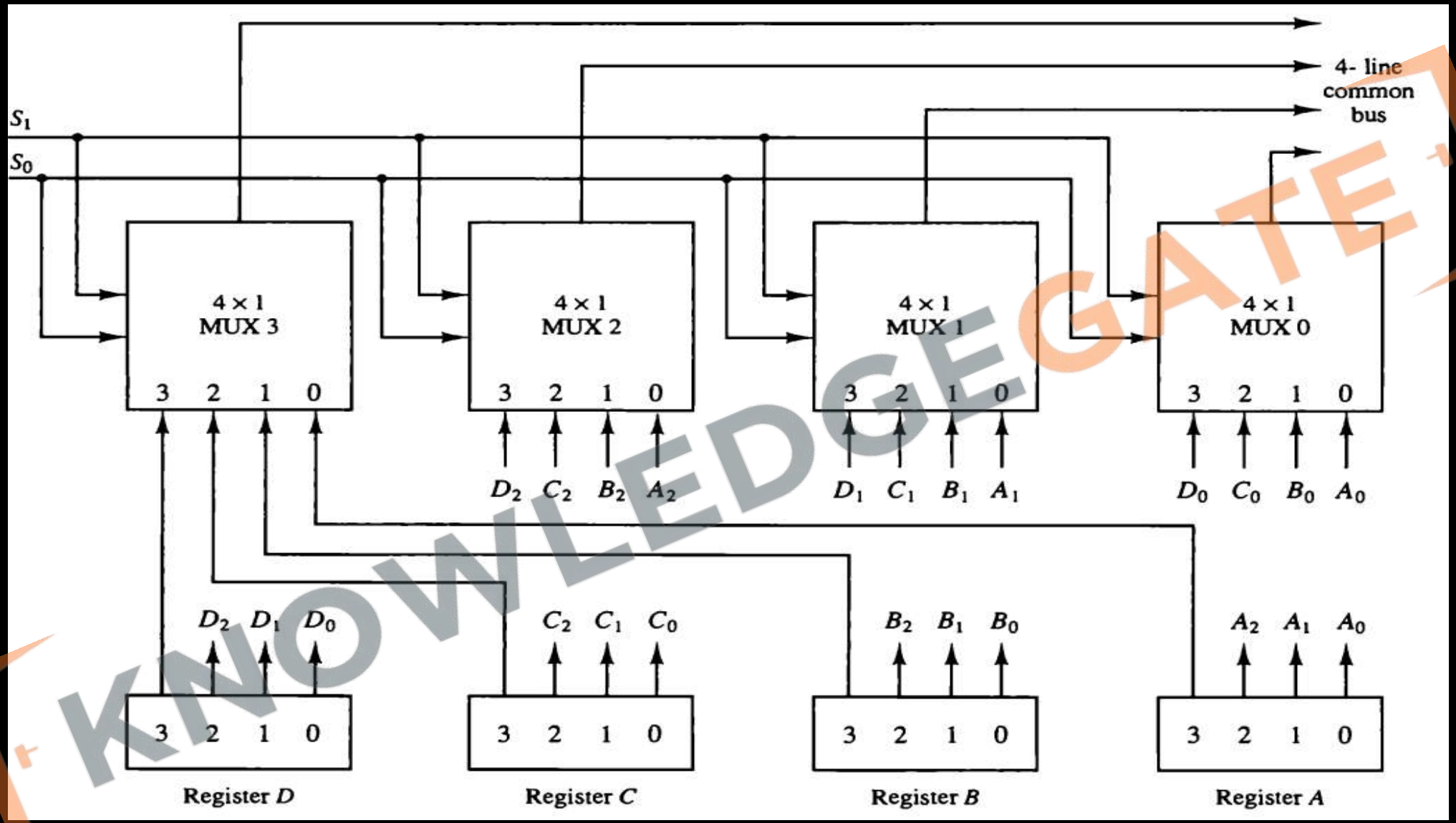
- The output **register (OUTR)** holds an 8-bit character for an output device, screen, printer etc.

KNOWLEDGEGATE



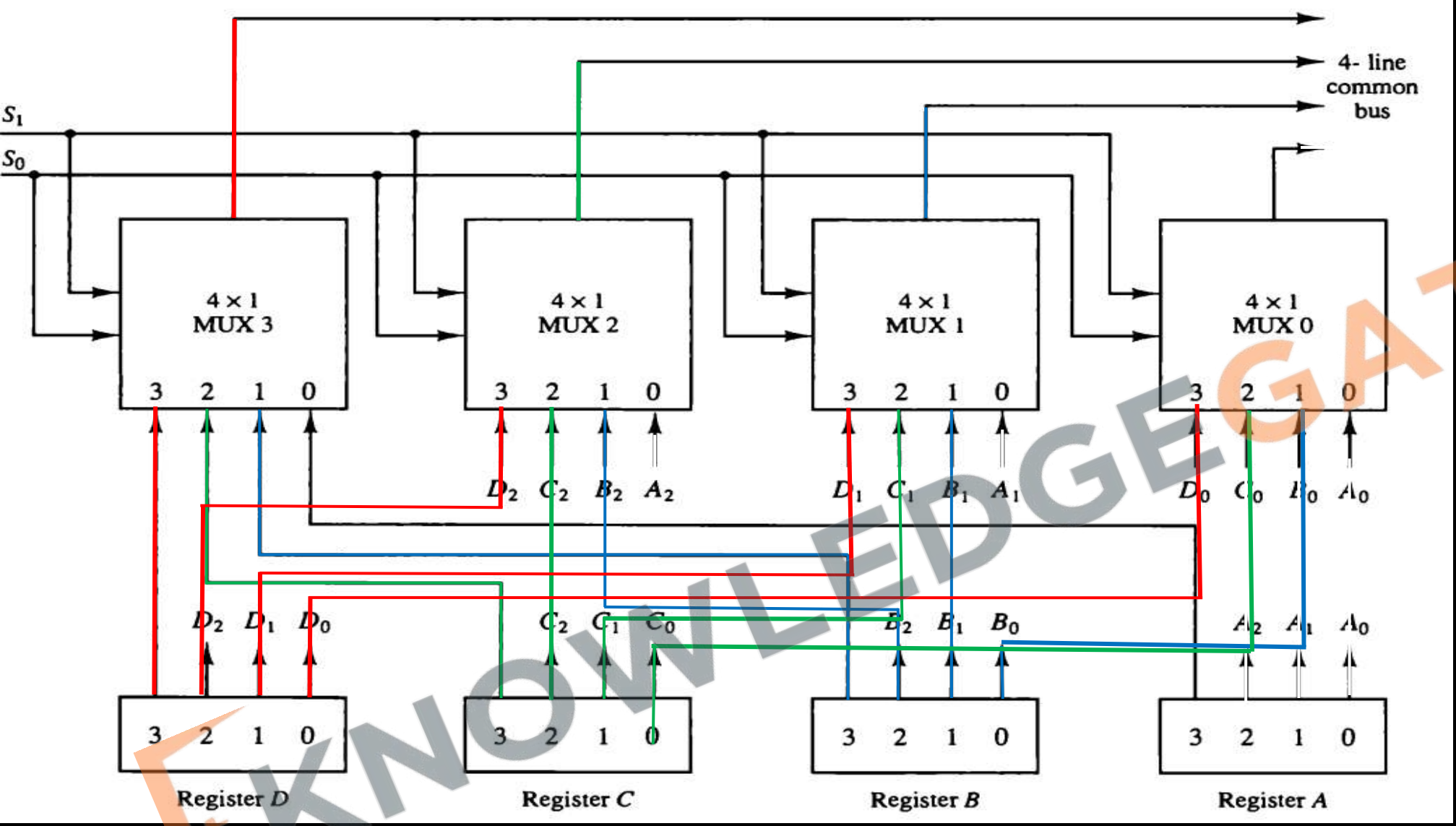
- The outputs of seven registers and memory are connected to the common bus.
- The specific output that is selected for the bus lines at any given time is determined from the binary value of the selection variables S_2, S_1, and S_0.
- The number along each output shows the decimal equivalent of the required binary selection.
- Example: the number along the output of DR is 3. The 16-bit outputs of DR are placed on the bus lines when $S_2S_1S_0 = 011$.

KNOWLEDGEGATE



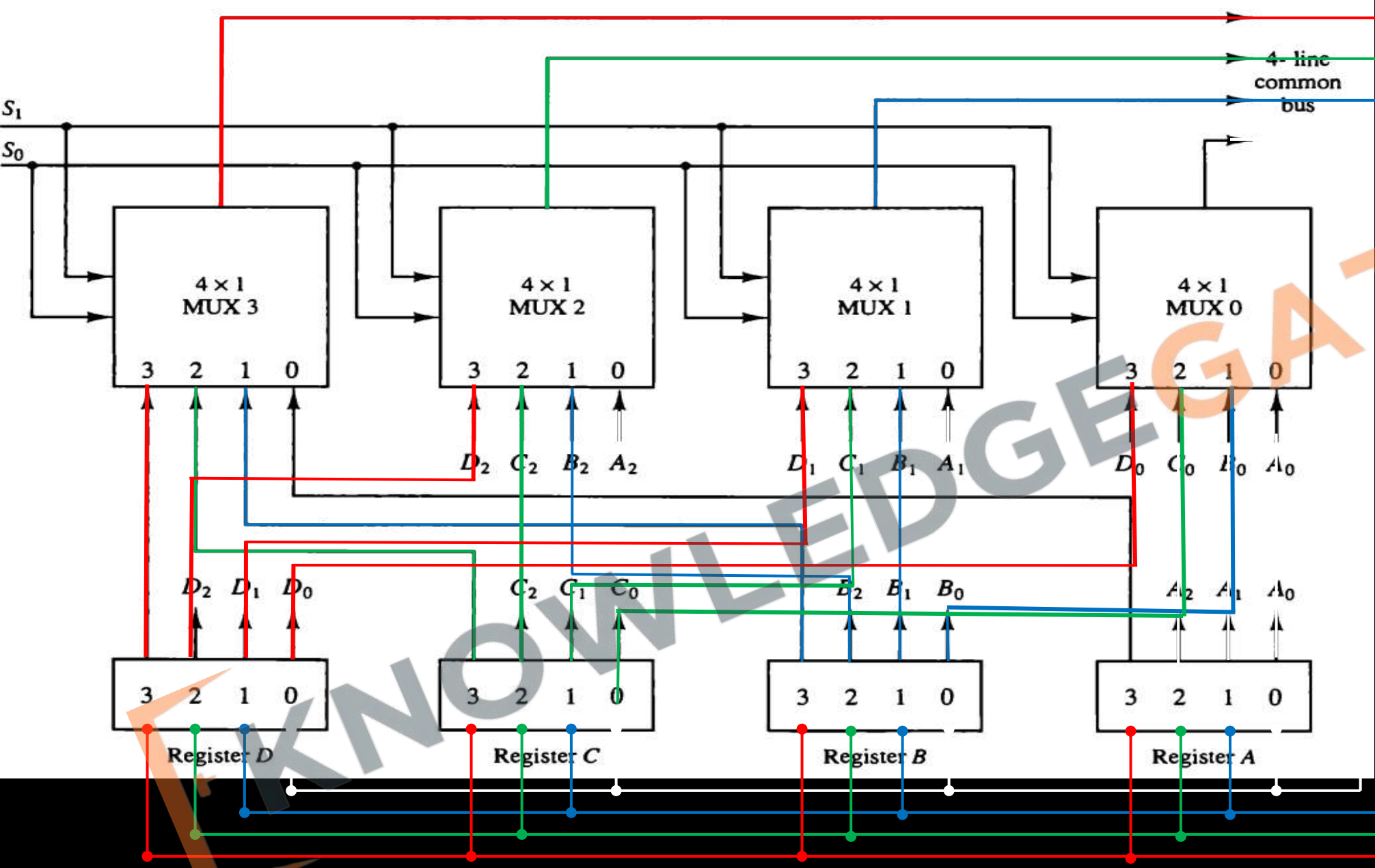
KNOWLEDGEGATE

www.knowledgegate.in



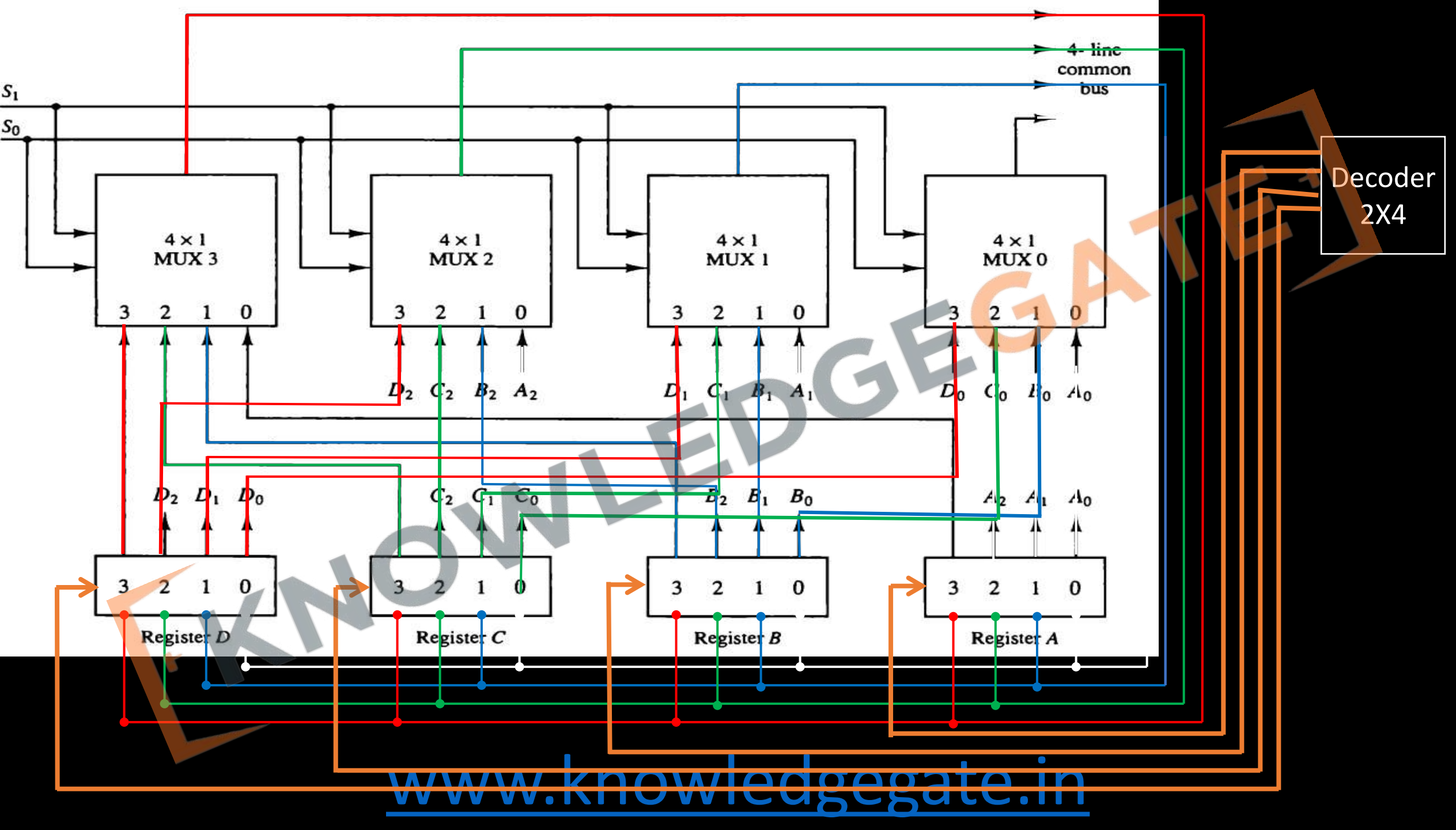
KNOWLEDGEGATE

www.knowledgegate.in

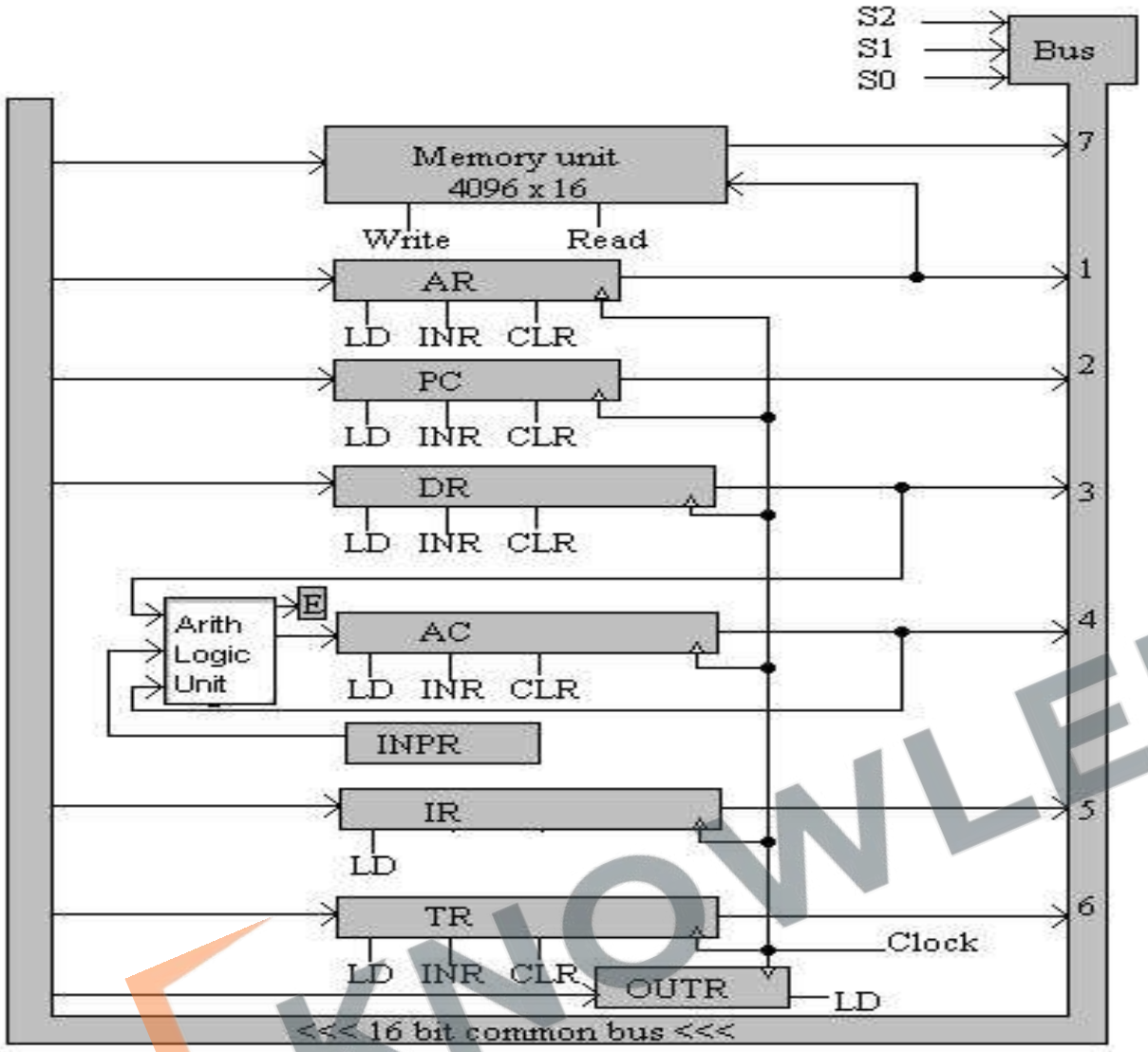


KNOWLEDGEGATE

www.knowledgegate.in

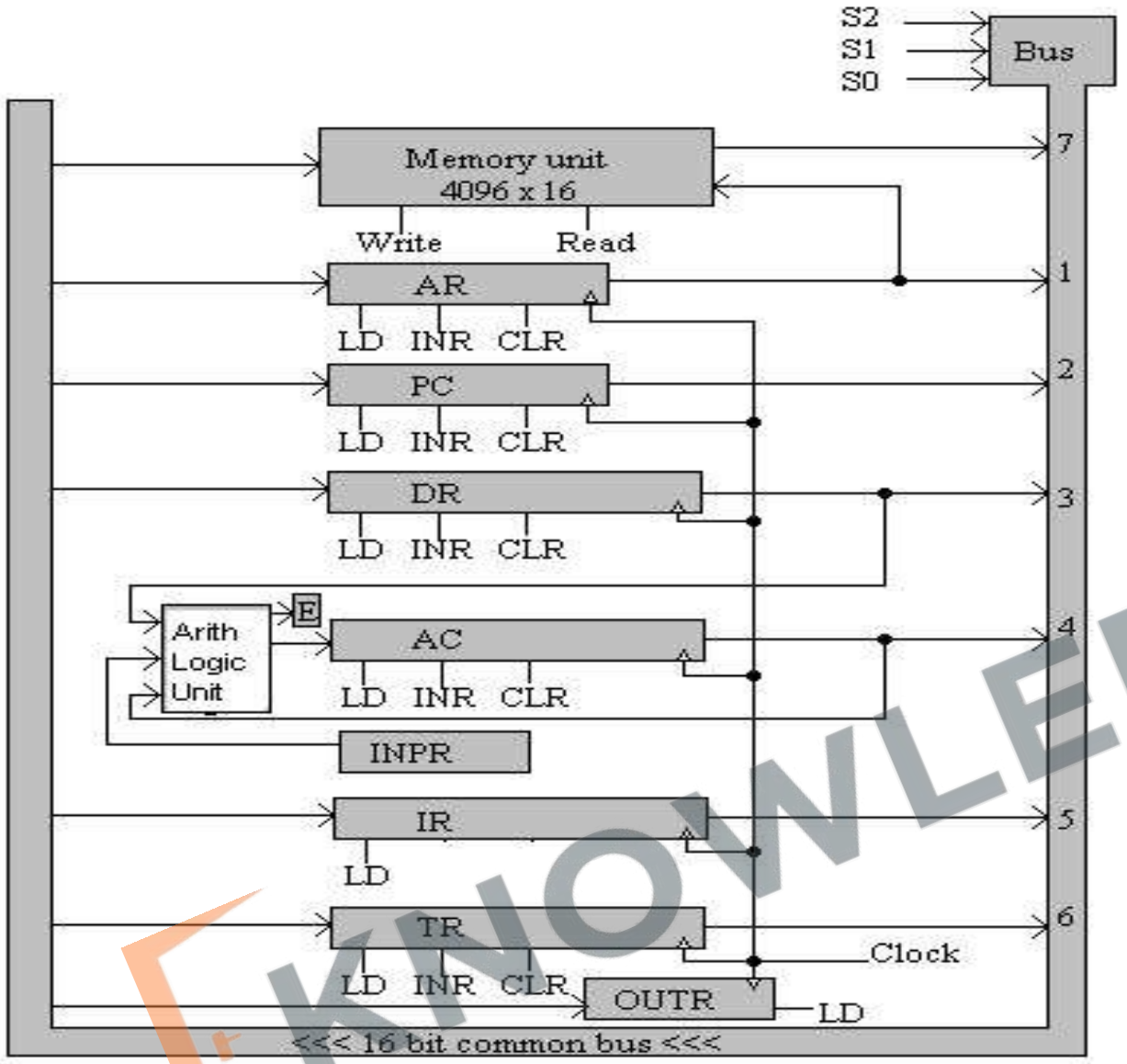


KNOWLEDGEGATE
www.knowledgegate.in



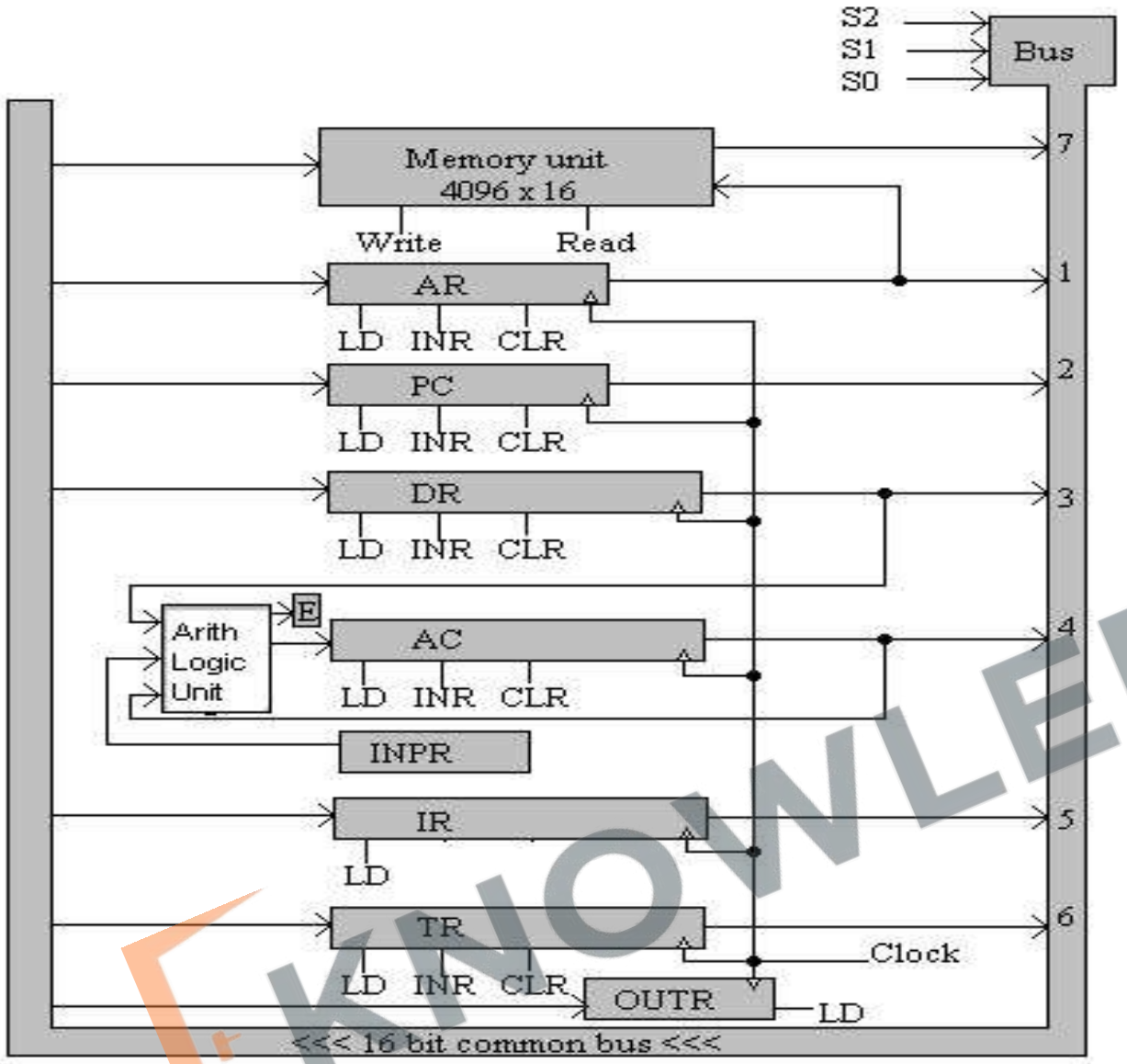
- The particular register whose LD (load) input is enabled receives the data from the bus during the next clock pulse transition.
- The memory receives the contents of the bus when its write input is activated.
- The memory places its 16-bit output onto the bus when the read input is activated and $S_2S_1S_0 = 111$.

KNOWLEDGEGATE



- The input register INPR and the output register OUTR have 8 bits each and communicate with the eight least significant bits in the bus.
- INPR is connected to provide information to the bus but OUTR can only receive information from the bus. There is no transfer from OUTR to any of the other registers.

KNOWLEDGEGATE



- Some additional points to notice
  - The 16 lines of the common bus receive information from six registers and the memory unit.
  - The bus lines are connected to the inputs of six registers and the memory.
  - Five registers have three control inputs: LD (load), INR (increment), and CLR (clear)

KNOWLEDGE GATE

General registers organization

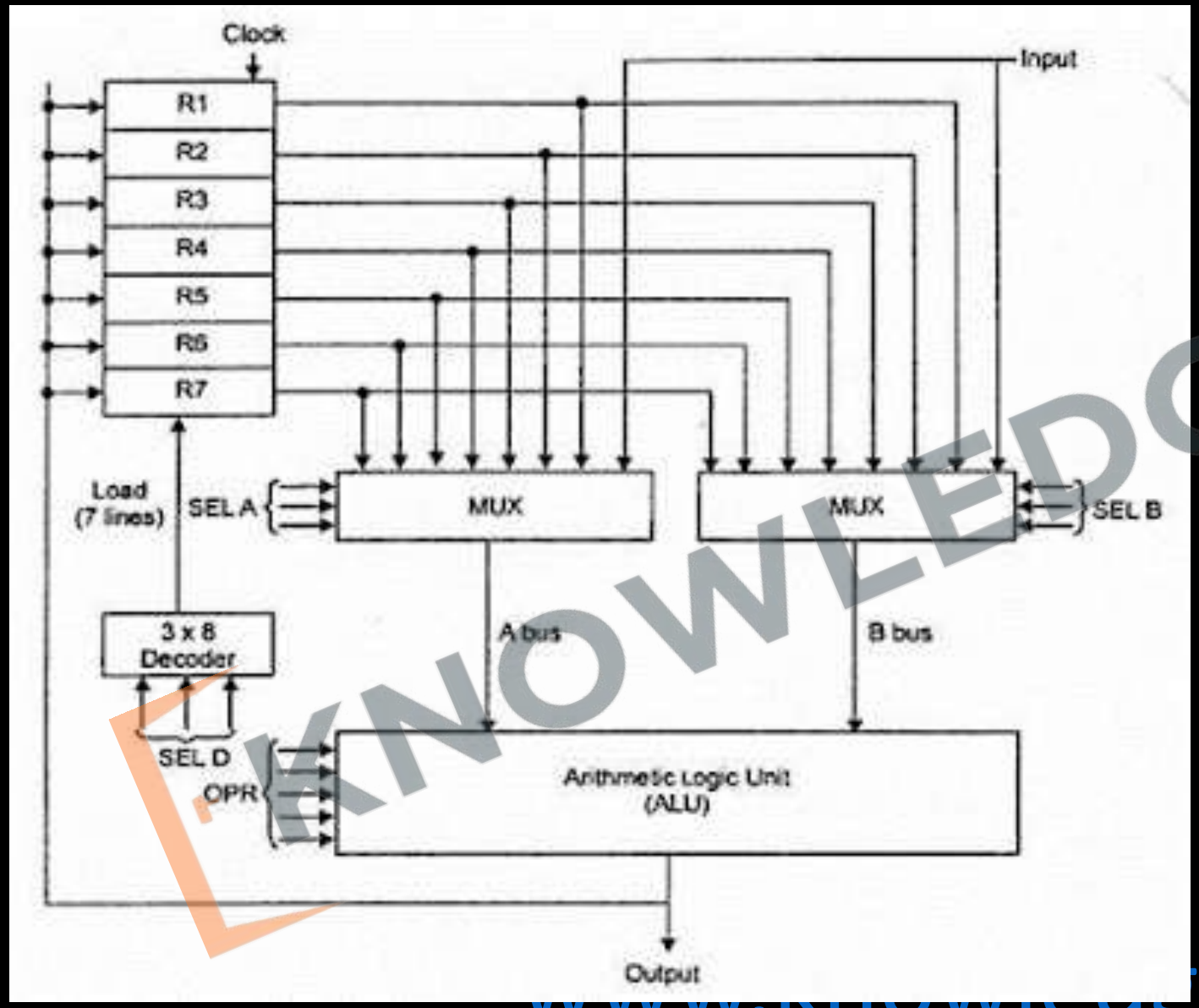


TABLE 8-1 Encoding of Register Selection Fields

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

TABLE 8-2 Encoding of ALU Operations

OPR Select	Operation	Symbol
00000	Transfer *A*	TSFA
00001	Increment *A*	INCA
00010	Add *A* + *B*	ADD
00101	Subtract *A* − *B*	SUB
00110	Decrement *A*	DECA
01000	AND *A* and *B*	AND
01010	OR *A* and *B*	OR
01100	XOR *A* and *B*	XOR
01110	Complement *A*	COMA
10000	Shift right *A*	SHRA
11000	Shift left *A*	SHLA

KNOWLEDGEGATE
www.knowledgegat

General registers organization

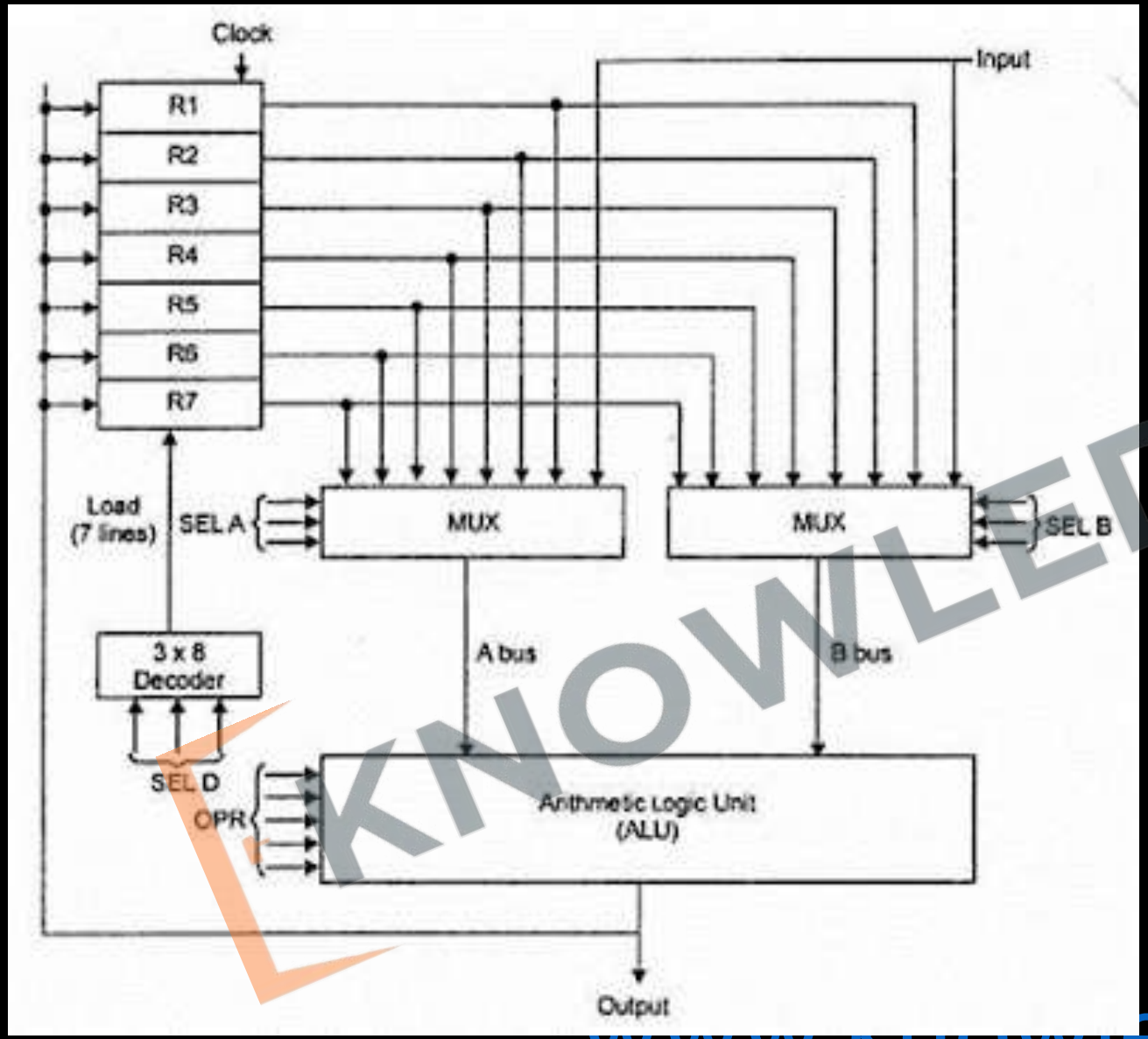


TABLE 8-3 Examples of Microoperations for the CPU

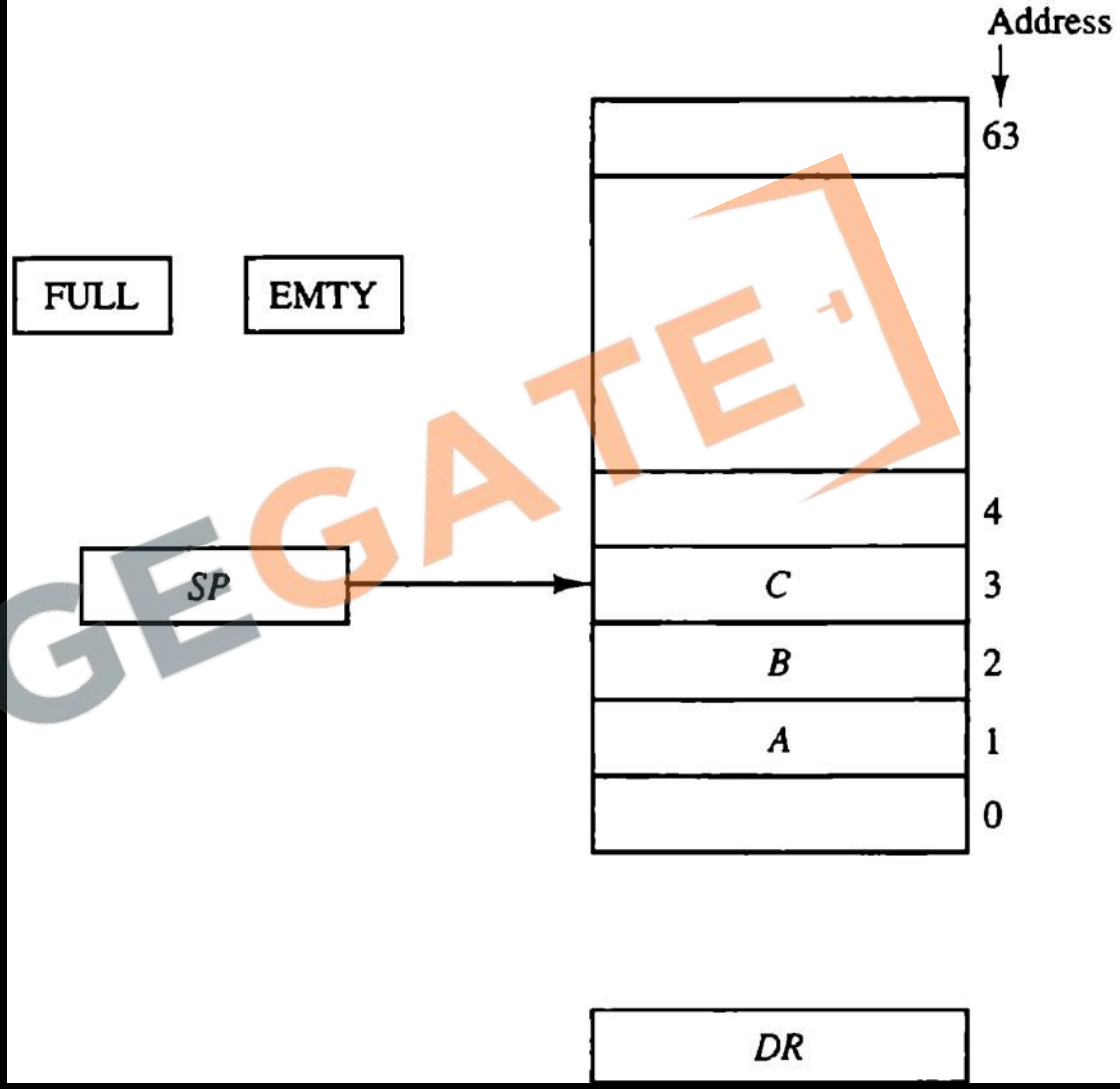
Microoperation	Symbolic Designation SELA	SELB	SELD	OPR	Control Word
$R1 \leftarrow R2 - R3$	R2	R3	R1	SUB	010 011 001 00101
$R4 \leftarrow R4 \vee R5$	R4	R5	R4	OR	100 101 100 01010
$R6 \leftarrow R6 + 1$	R6	—	R6	INCA	110 000 110 00001
$R7 \leftarrow R1$	R1	—	R7	TSFA	001 000 111 00000
Output $\leftarrow R2$	R2	—	None	TSFA	010 000 000 00000
Output $\leftarrow$ Input	Input	—	None	TSFA	000 000 000 00000
$R4 \leftarrow$ shl $R4$	R4	—	R4	SHLA	100 000 100 11000
$R5 \leftarrow 0$	R5	R5	R5	XOR	101 101 101 01100

KNOWLEDGEGATE.in

www.Knowledgegate.in

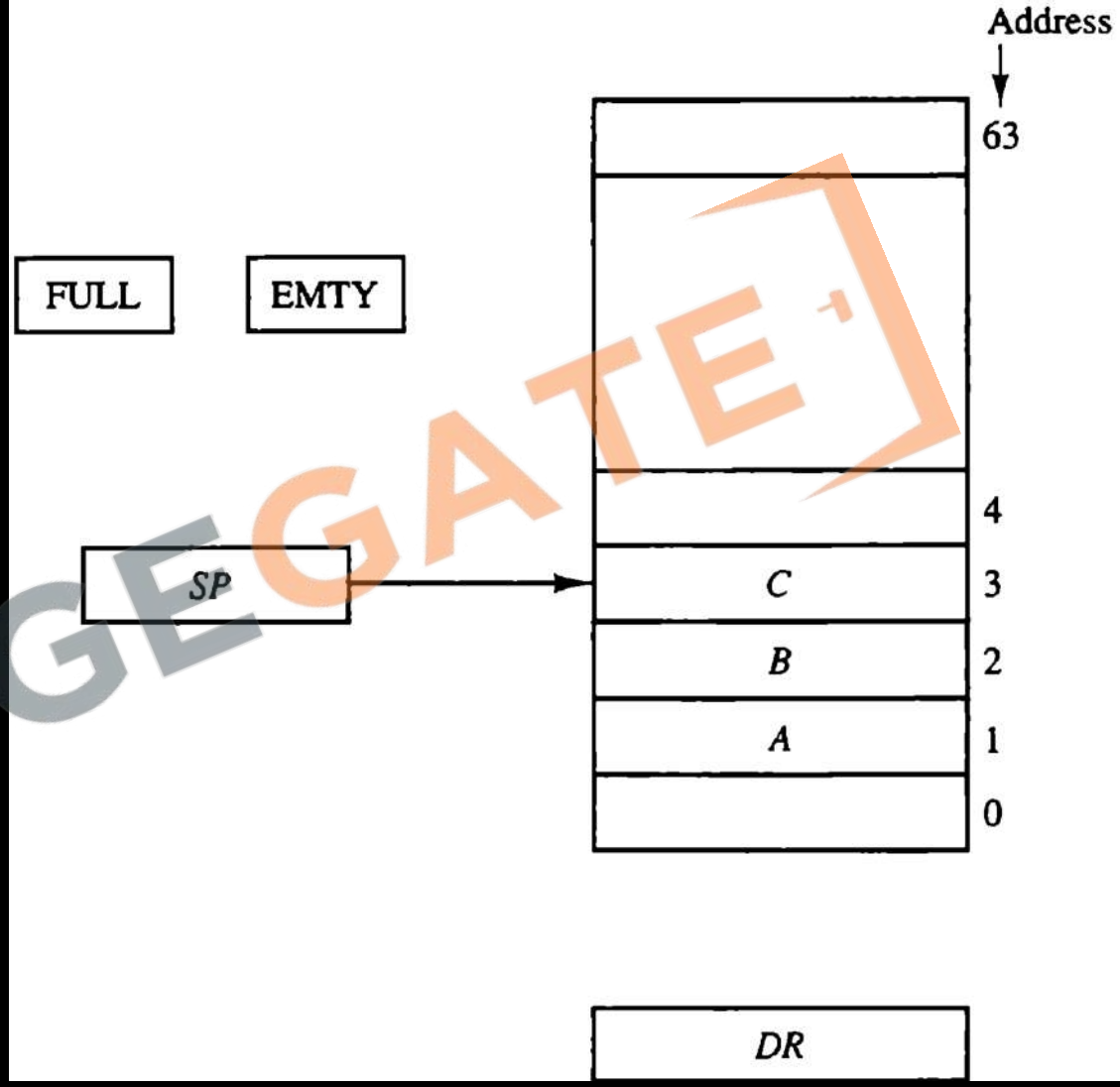
Stack organization

- It is an ordered list in which addition of a new data item and deletion of already existing data item is done from only one end known as top of stack (TOS)
- The element which is added in last will be first to be removed and the element which is inserted first will be removed in last, so it is called last in first out (LIFO) or first in last out (FILO) type of list.
- Most frequently accessible element in the stack is the topmost element, whereas the least accessible element is the bottom of the stack.



KNOWLEDGEGATE

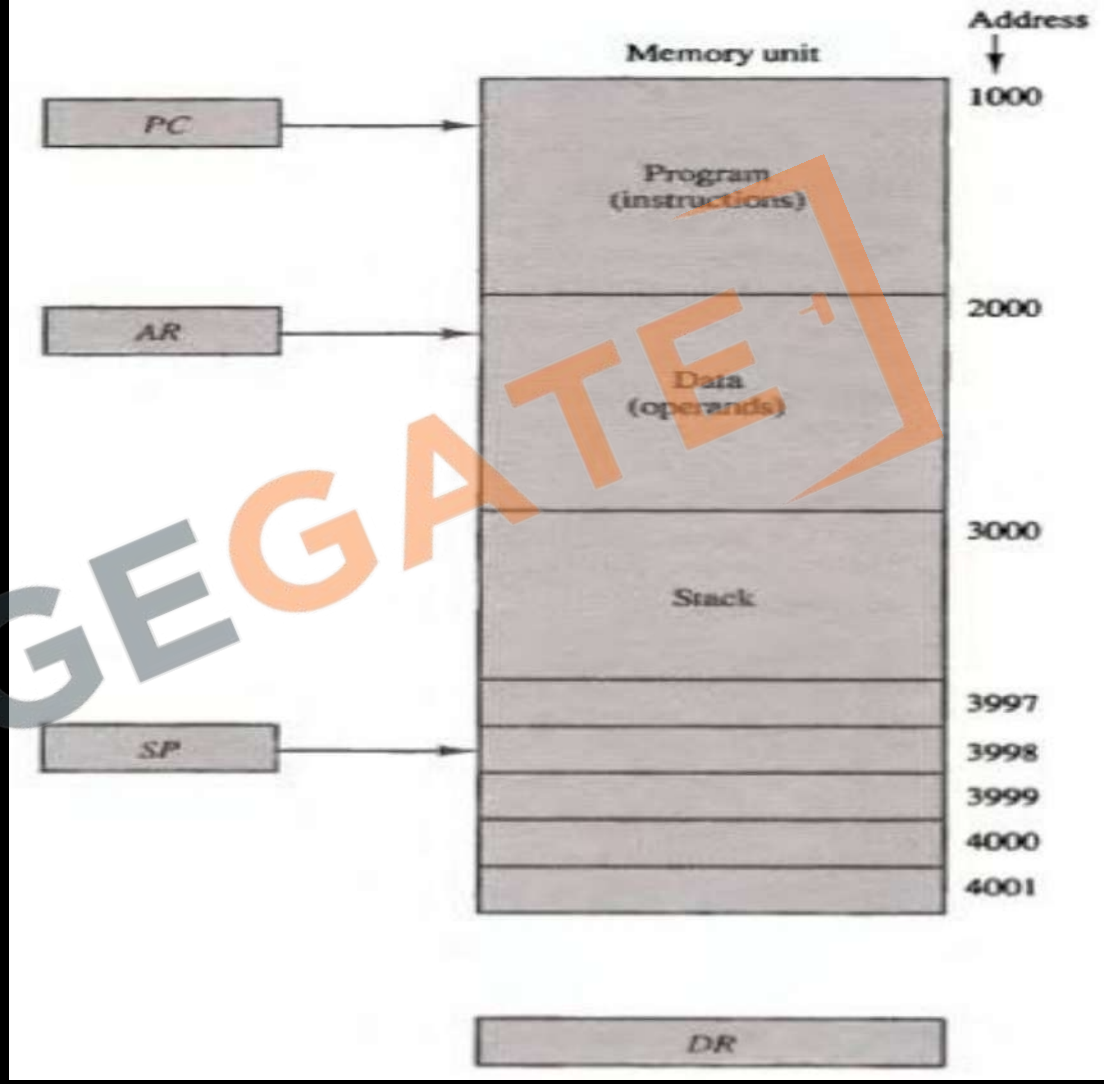
- Stack Pointer register (SP): It contains a value in binary each of 6 bits, which is the address of the top of the stack. Here, the stack pointer SP contains 6 bits and SP cannot contain a value greater than 111111 i.e. 63
- **Full Register**: it can store 1 bit of information, set to 1 when stack is full.
- **Empty Register**: it can store 1 bit of information, set to 1 when the stack is empty.
- **Data Register**: It hold the data to be written into into be read from the stack.



KNOWLEDGEGATE

Memory Stack

- Memory stacks operate on a Last-In, First-Out (LIFO) principle, where the most recently added element is the first to be removed.
- A special register called the Stack Pointer (SP) points to the top of the stack, and it can contain binary values up to a limit, often determined by the architecture, such as 6 bits equating to 63 in decimal.
- To monitor the stack's status, Full and Empty Registers are used, each storing a single bit to indicate whether the stack is full or empty.
- A Data Register holds the data to be written into or read from the stack, serving as an intermediary between the CPU and the stack. These elements collectively form the essential organization of a memory stack.



KNOWLEDGEGATE
www.knowledgegate.in

Addressing Mode

- It specifies the different ways possible in which reference to the operand can be made.
- **Effective address**: - It is the final address of the location where the operand is stored.
- Calculation of the effective address can be done in two ways
  - Non-computable addressing
  - Computable addressing (which involve arithmetic's)

KNOWLEDGEGATE

www.knowledgegate.in

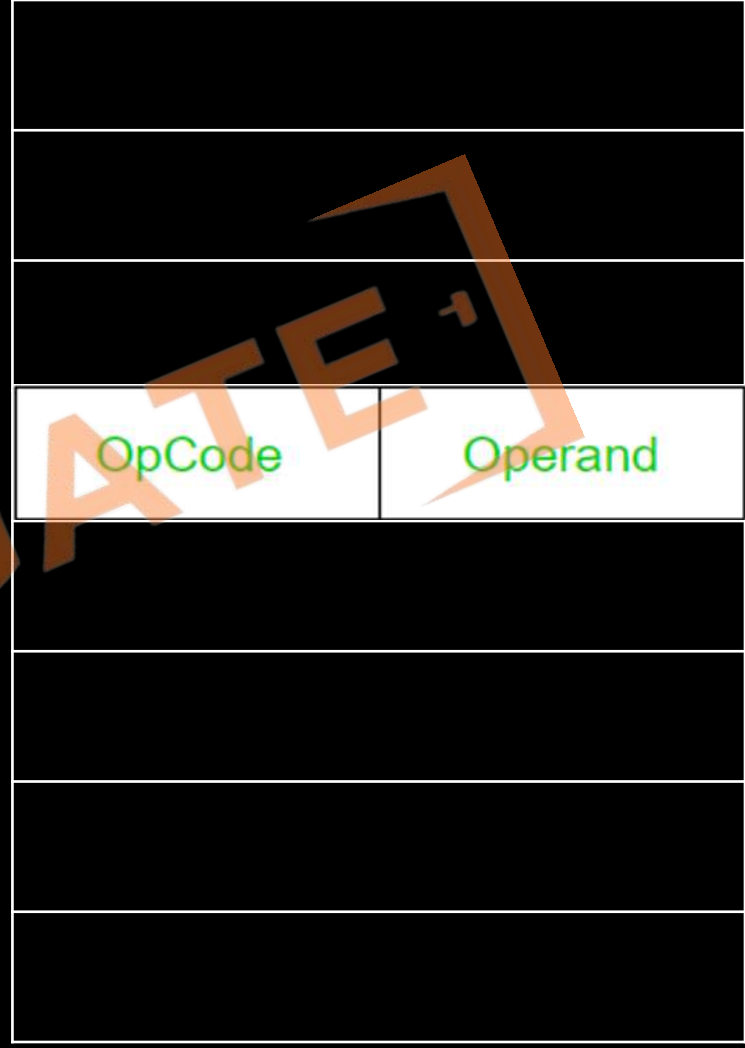
- Criteria for different addressing mode
  - It should be fast
  - The length of the instruction must be small
  - They should support pointers
  - They should support looping constructs, indexing of data structure
  - Program relocation

KNOWLEDGEGATE

www.knowledgegate.in

Immediate mode addressing

- It means the operand is itself part of the instruction.
  - E.g. ADD 3
  - Means Add 3 to the accumulator



KNOWLEDGEGATE

- Advantage
  - Can be used for constants, where values are already known.
  - Extremely fast, no memory reference is required.

- Disadvantage
  - Cannot be used with variables whose values are unknown at the time of program writing.
  - Cannot be used for large constant whose values cannot be stored in the small part of instruction.

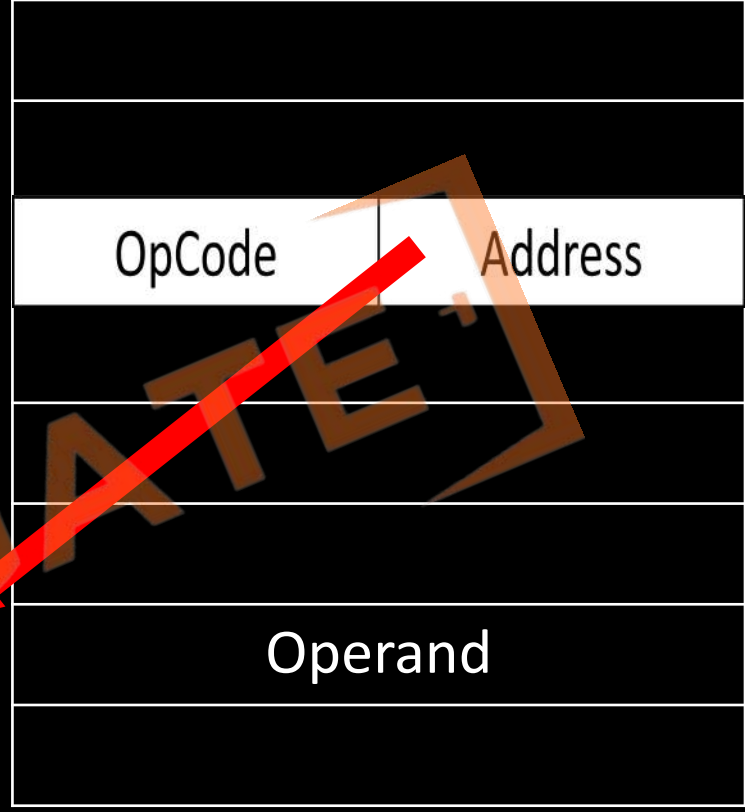
- Application
  - Mostly used when required data is directly moved to required register or memory

KNOWLEDGEGATE

www.knowledgegate.in

Direct mode addressing (absolute address mode)

- It means instruction contain address of the memory location where data is present (effective address).
- Here only one memory reference operation is required to access the data.



KNOWLEDGEGATE

- Advantage
  - With variable whose values are unknown direct addressing is the simplest one.
  - No restriction on the range of data values and largest value which system can hold, can be used in this mode.
  - Can be used to access global variables whose address is known at compile time.

- Disadvantage
  - Relatively slow compare to immediate mode
  - No of variable used are limited
  - In large calculation it will fail

KNOWLEDGEGATE

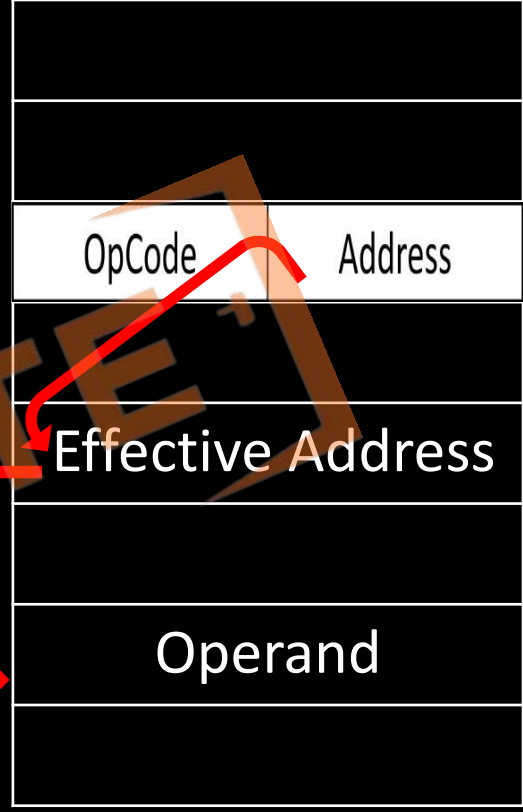
www.knowledgegate.in

Indirect mode addressing

- Here in the instruction we store the address where the (address of the variable) effective address is stored using which we can access the actual data.

- Here two references are required.

- 1st reference to get effective address and 2nd reference to access the data.



KNOWLEDGEGATE

- **Advantage**
  - No limitation on no of variable or size of variables.
  - Implementation of pointer are feasible and relatively more secure.

- **Disadvantage**
  - Relatively slow as memory must be referred more than one time.

KNOWLEDGEGATE

www.knowledgegate.in

Implied mode addressing

- In implied addressing mode, the operands are specified implicitly in the definition of the instruction.
- All the instructions which reference registers that use an accumulator are implied mode instructions.
- Zero address instructions in a stack organized computer are also implied mode instructions. Thus, it is also known as stack addressing mode.
- E.g. Increment accumulator, Complement accumulator

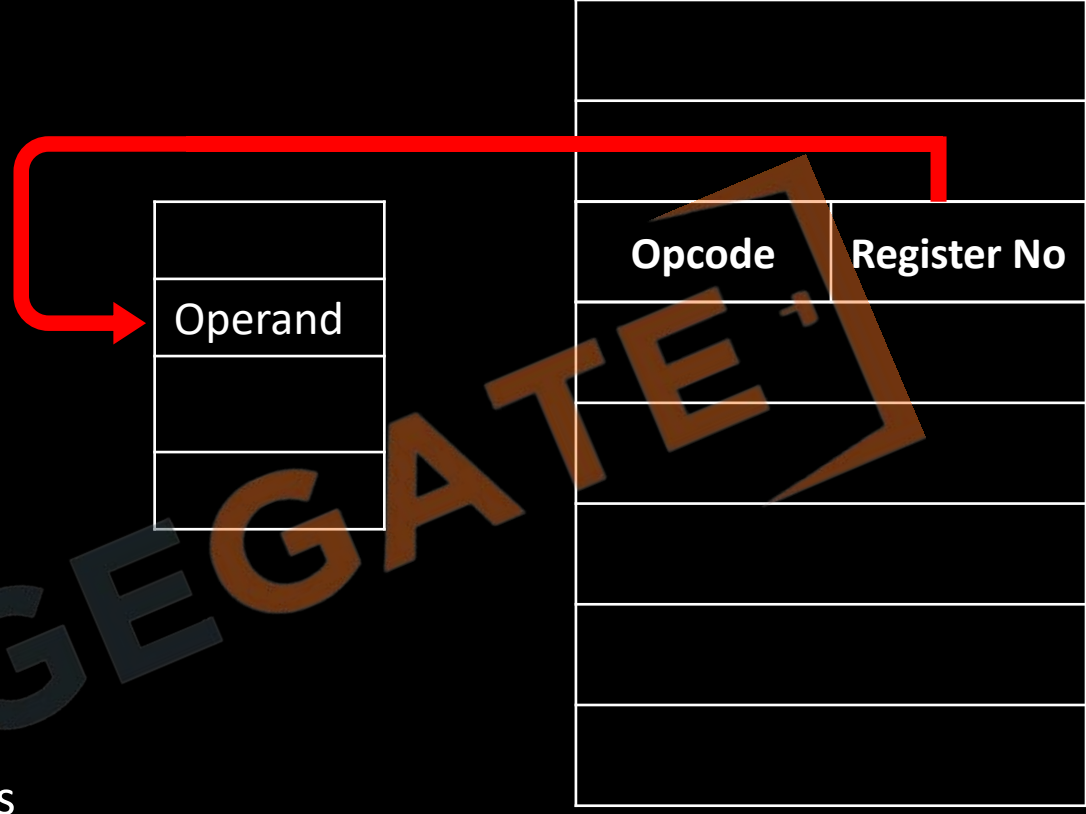
KNOWLEDGEGATE

Register mode addressing

- It means variable are stored in register of the CPU instead of memory, in the instruction we will give the register no

- Advantage
  - Will be extremely fast as register access time will be less then cache access time.
  - Because no of registers is less, so bits required to specify a register is also less.

- Disadvantage
  - Because no of registers is very less so only with few variables this method can be used

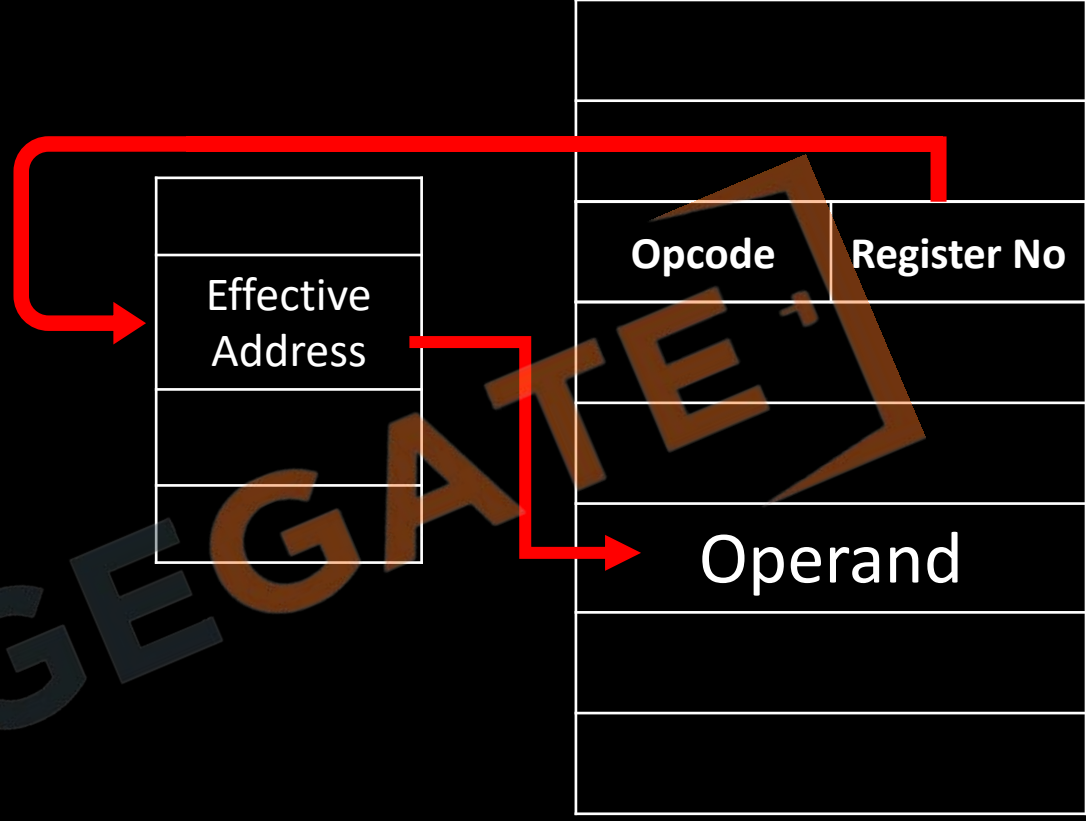


KNOWLEDGEGATE

www.knowledgegate.in

Register indirect mode addressing

- In this mode in the instruction we will specify the address of the register where inside the register actual memory address of the variable is stored effective address.
- When same address is required again and again, this approach is very useful
- Here the same register can be used to provide different data by charging the value of register, i.e. it can reference memory without paying the price of having a full memory in the instruction.
- In pointer arithmetic, it provides more flexibility compare to register mode.
- Register indirect mode can further be improved by auto increment and auto decrement command where we read or work on some continuous data structure like array or matrix.



KNOWLEDGEGATE

www.knowledgegate.in

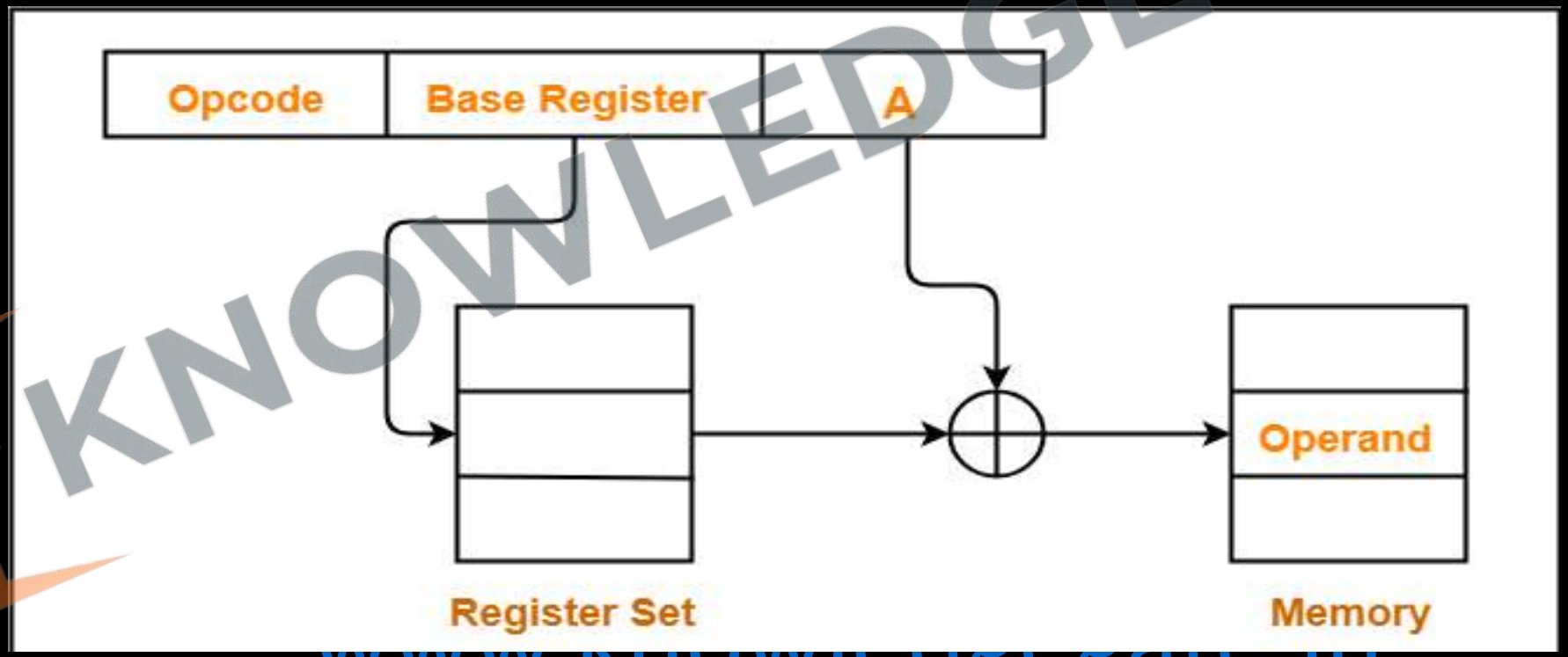
Base register (off set) mode

- In multiprogramming environment, the location of the process in the memory keeps on changing from one place to another.
- But if we are using direct address in the process then it will create a problem.
- So, to solve this problem we try to save the starting of the program address in a register called base register. It is a very popular approach in most of the computer.

KNOWLEDGEGATE

www.knowledgegate.in

- Then instead to giving the direct branch address we give off set in the instruction
- Effective address = base address + off set (instruction)
- Now the advantage is even if we try to shift process in the memory, we only need to change the content of the base register.
- Final address will be the sum of what is given in instruction and given in base register.



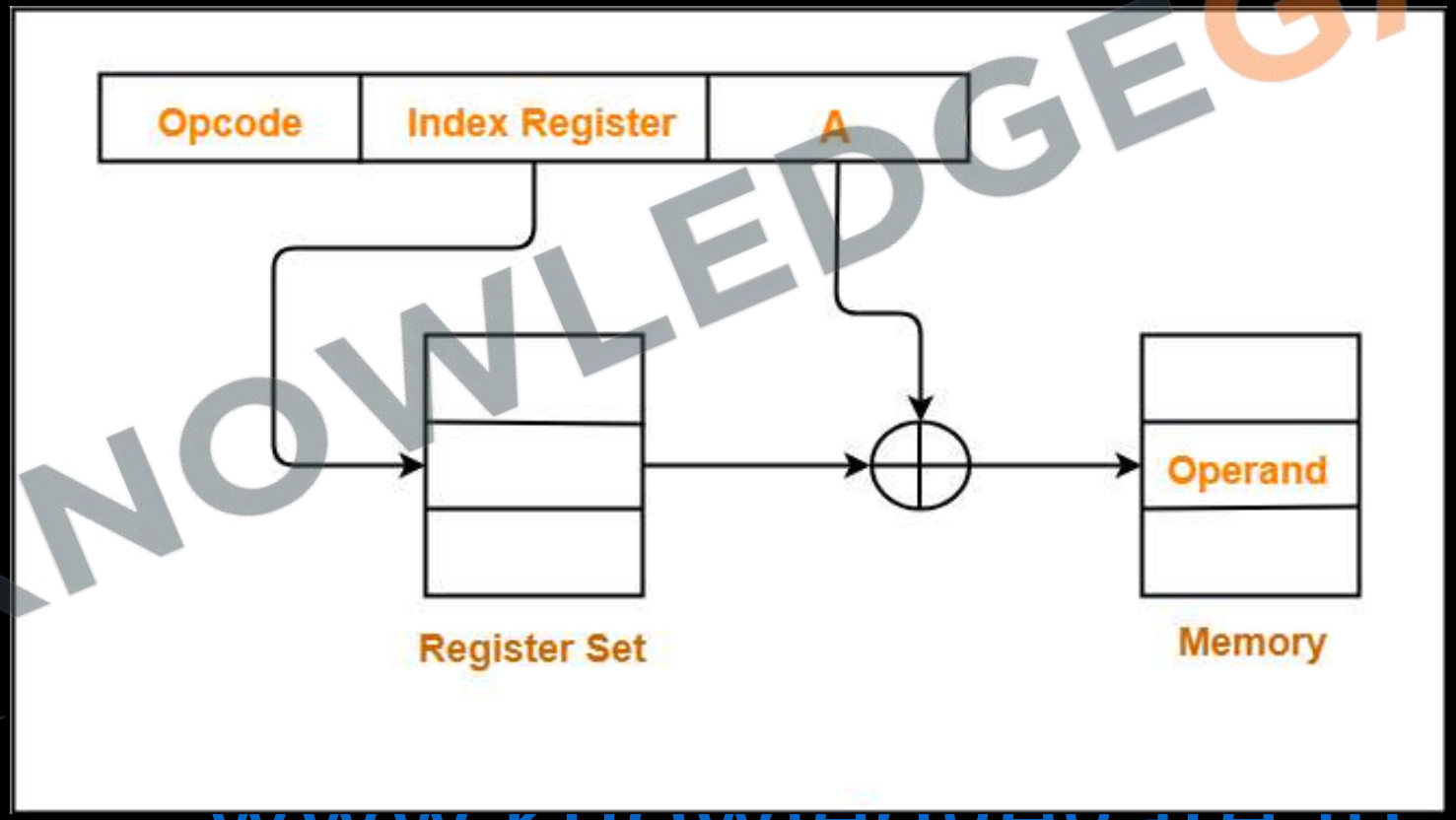
KNOWLEDGEGATE
www.knowledgegate.in

Index addressing mode

- This mode is generally used when CPU has a number of registers, and out of which one can be used an index register
- This mode is especially useful, when we try to access large sixed array elements.

KNOWLEDGEGATE

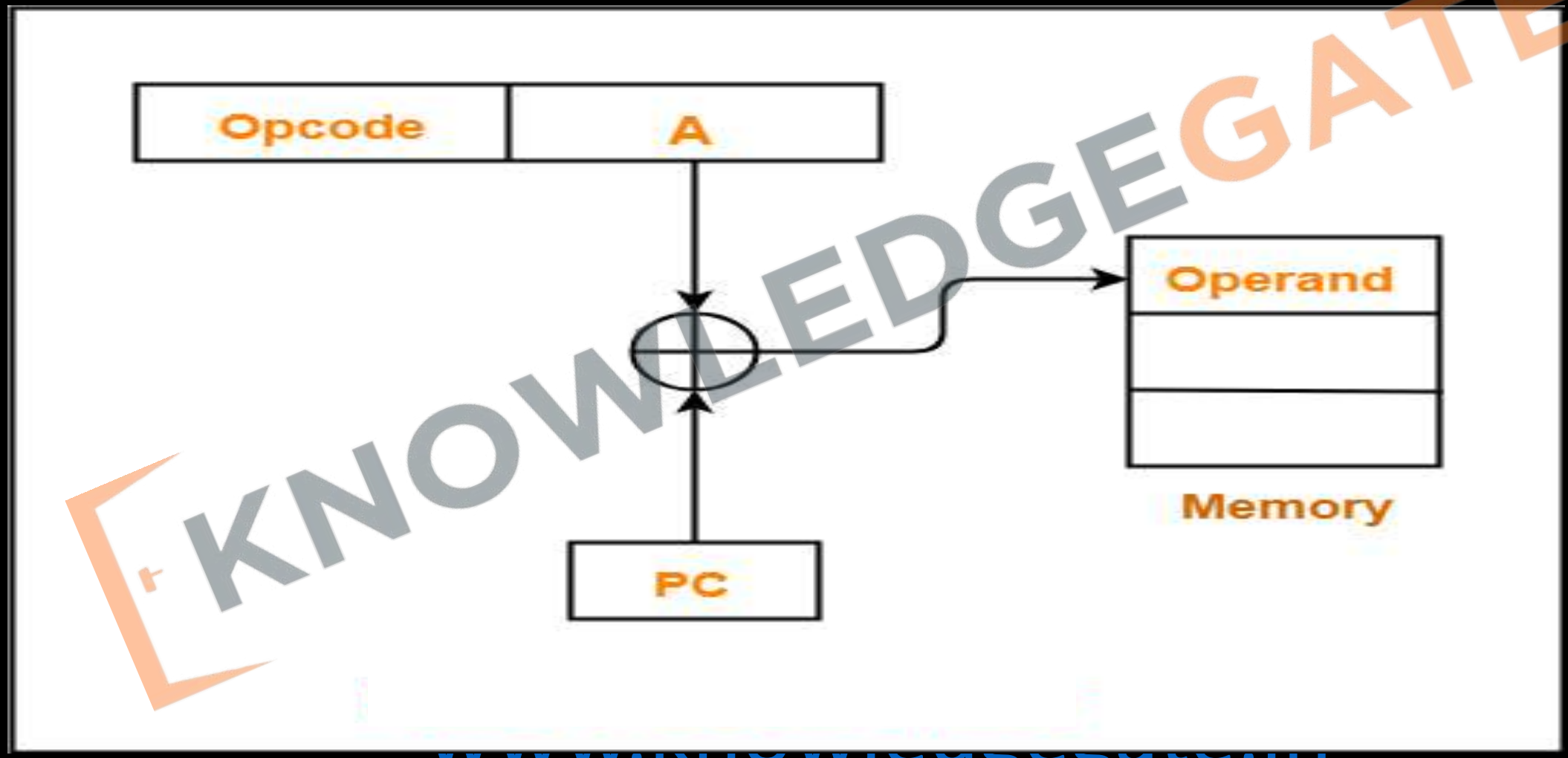
- Idea is, give the base address of the array in the instruction and the index which we want to access will be there in the register.
- So by changing the index in the index register we can use the same instruction to access the different element in the array.
- Base in present inside the instruction and index is present inside the register



KNOWLEDGEGATE
www.knowledgegate.in

Relative Addressing Mode

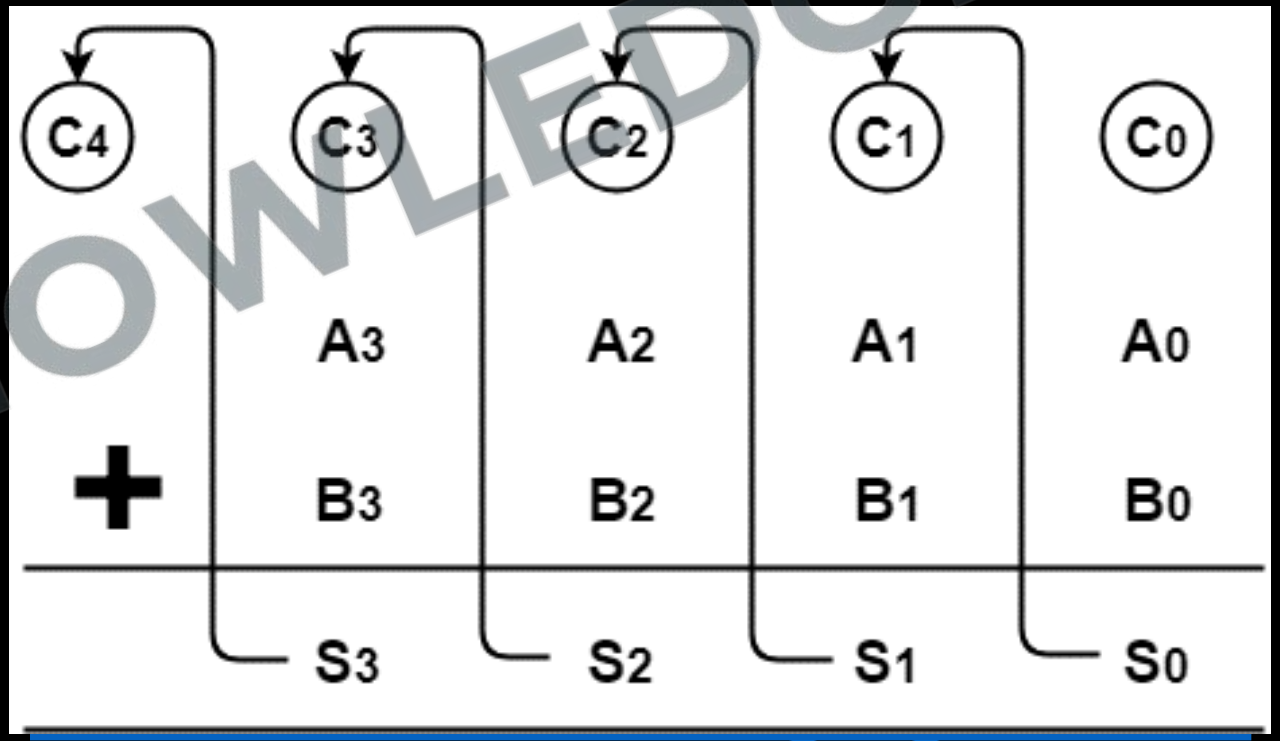
- Effective address of the operand is obtained by adding the content of program counter with the address part of the instruction.



KNOWLEDGEGATE

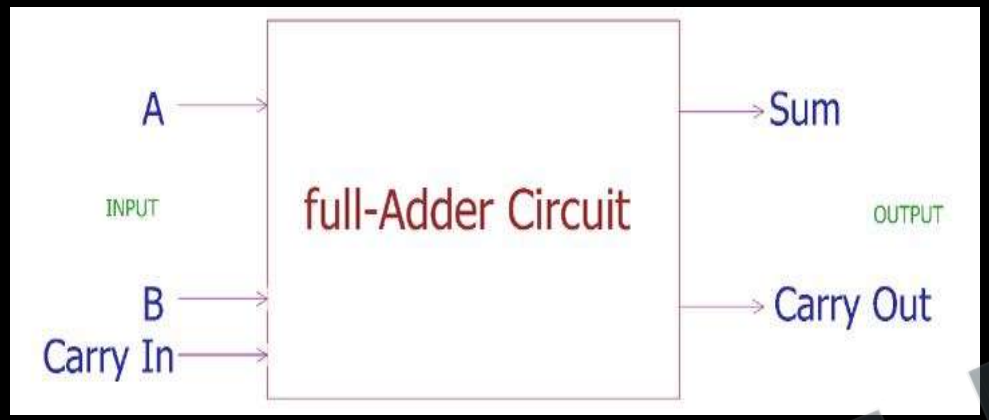
Full adder

1. A full adder is a combinational logic circuit that performs the arithmetic sum of three input bits.
2. Where A_n, B_n are the n^{th} order bits of the number A and B respectively and C_n is the carry generated from the addition of $(n-1)^{th}$ order bits.
3. It consists of three input bits, denoted by A (First operand), B (Second operand), C_{in} (Represents carry from the previous lower significant position).



KNOWLEDGEGATE

- Two output bits are same as of half adder, which is Sum and $Carry_{out}$.
- When the augend and addend number contain more significant digits, the carry obtained from the addition of two bits is added to the next higher order pair of significant bits.



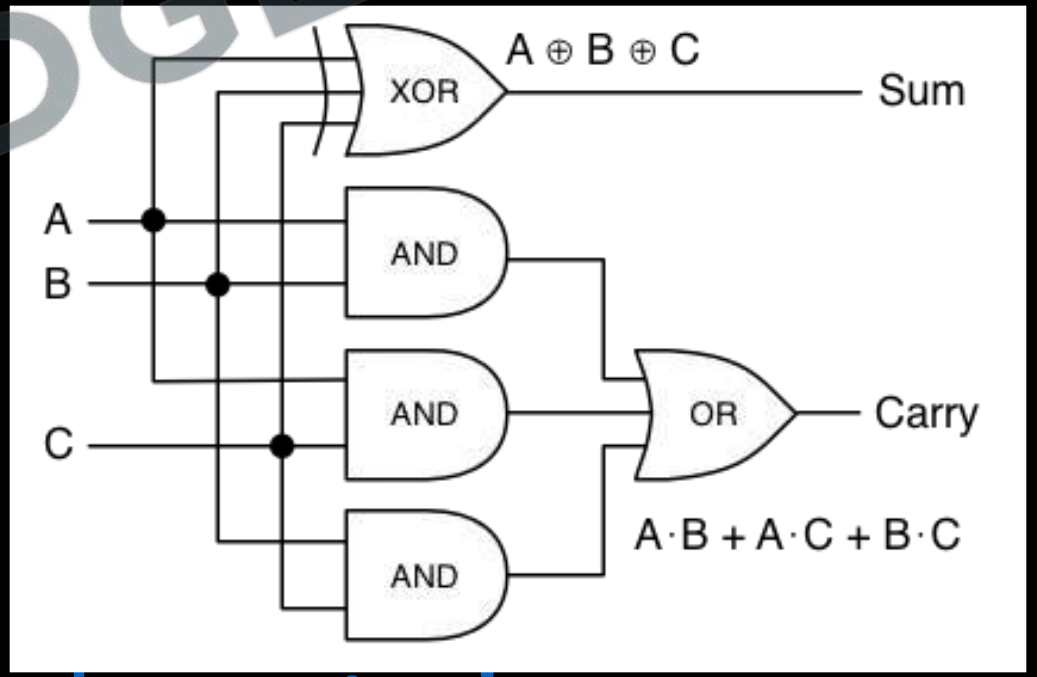
INPUTS			OUTPUTS	
A	B	C_{in}	C_{out}	Sum
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

KNOWLEDGEGATE
www.knowledgegate.in

	ab	a’b’	a’b	ab	ab’
C_{in}		00	01	11	10
C_{in}’	0	0	2	6	4
C_{in}	1	1	3	7	5

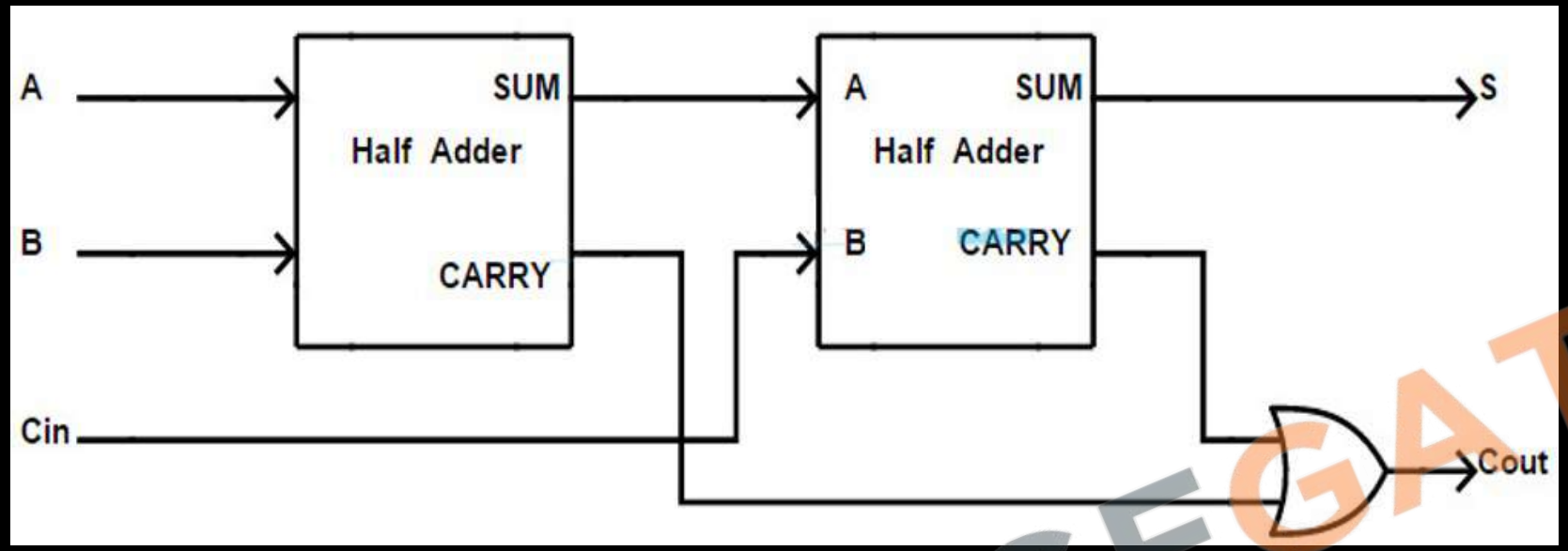
	ab	a’b’	a’b	ab	ab’
C_{in}		00	01	11	10
C_{in}’	0	0	2	6	4
C_{in}	1	1	3	7	5

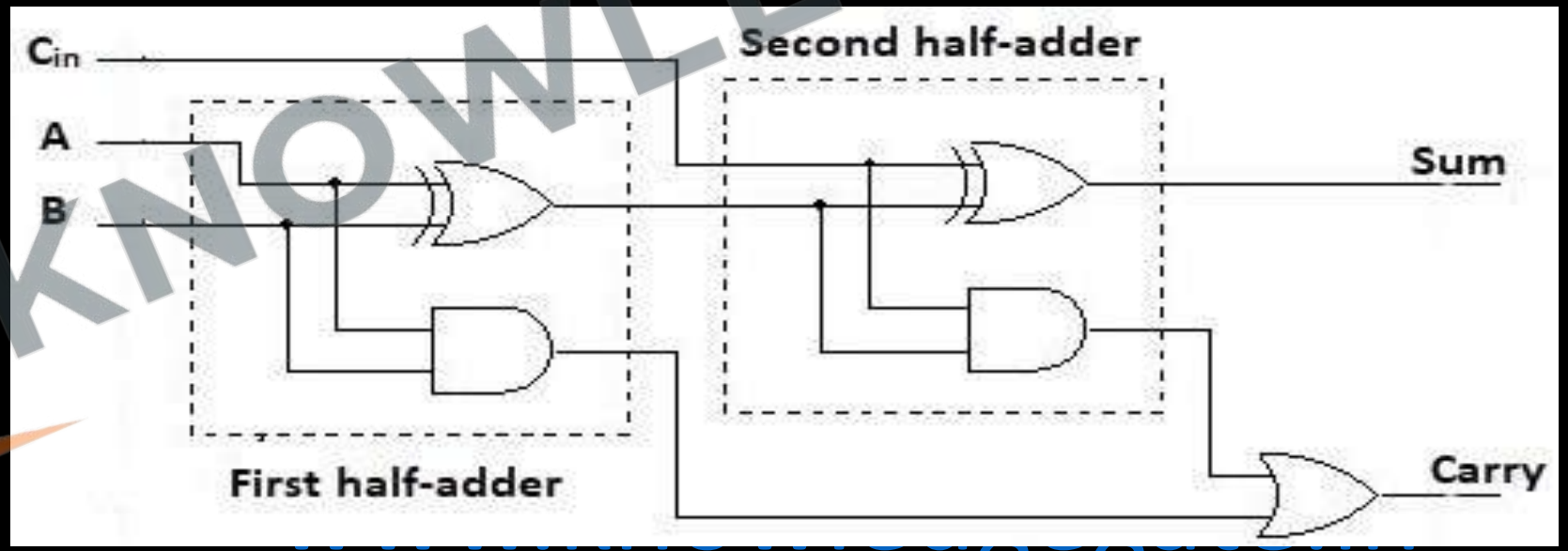
INPUTS			OUTPUTS	
A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



KNOWLEDGEGATE

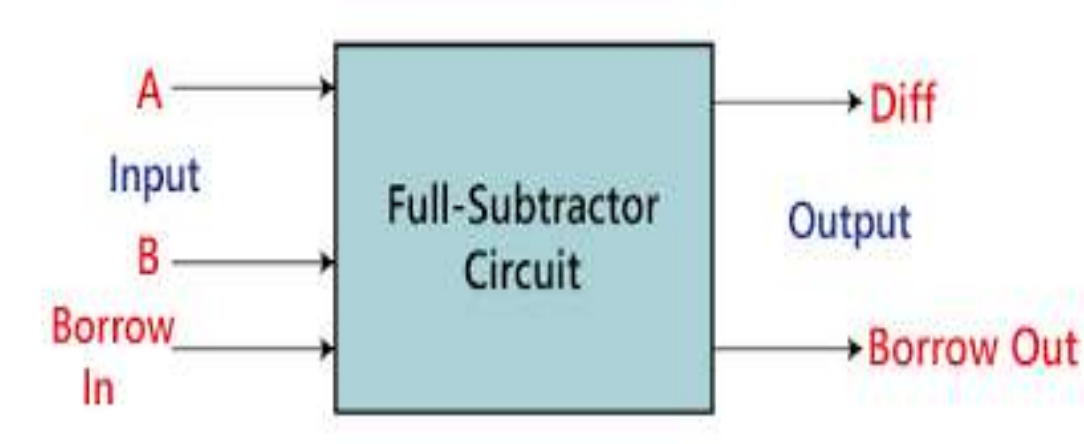
www.knowledgegate.in

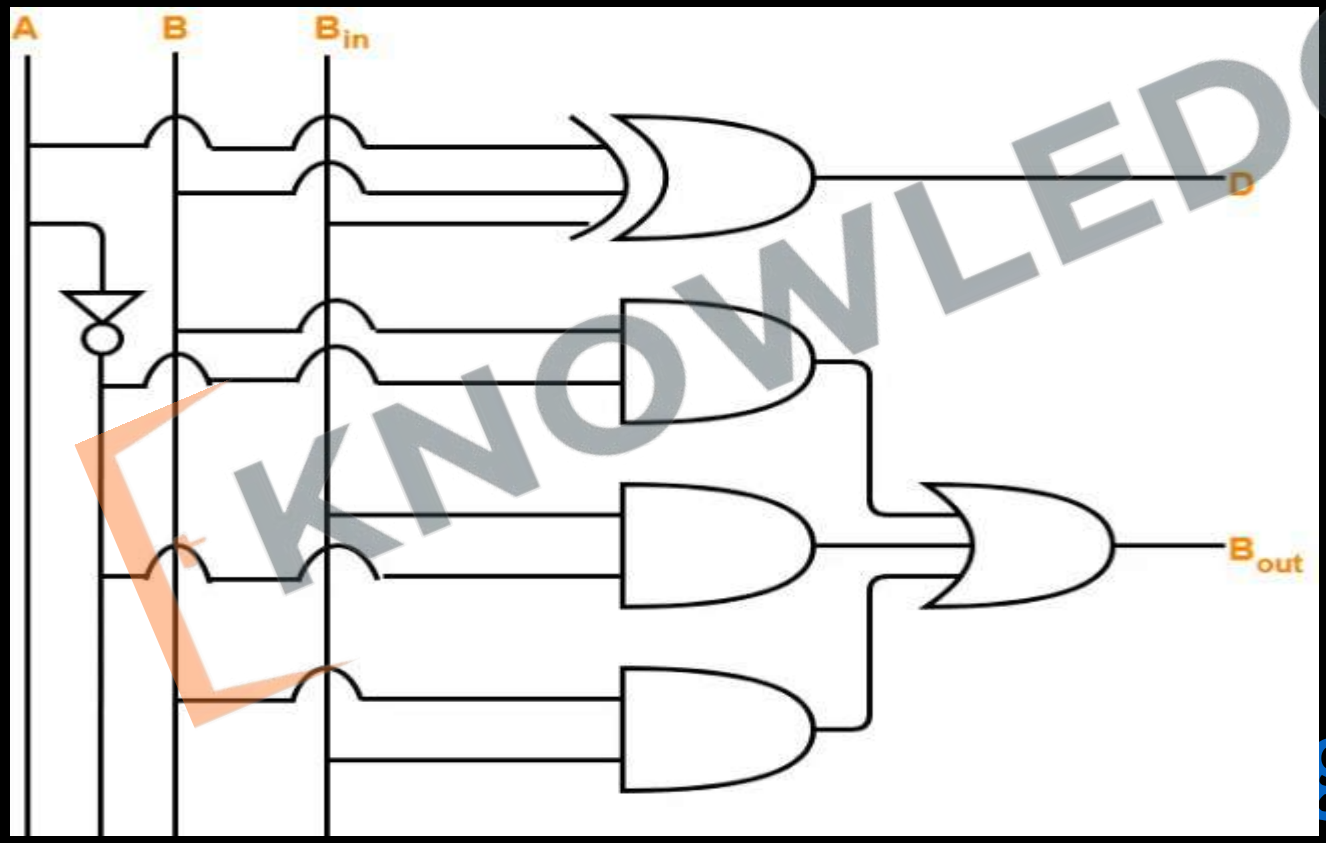




KNOWLEDGEGATE

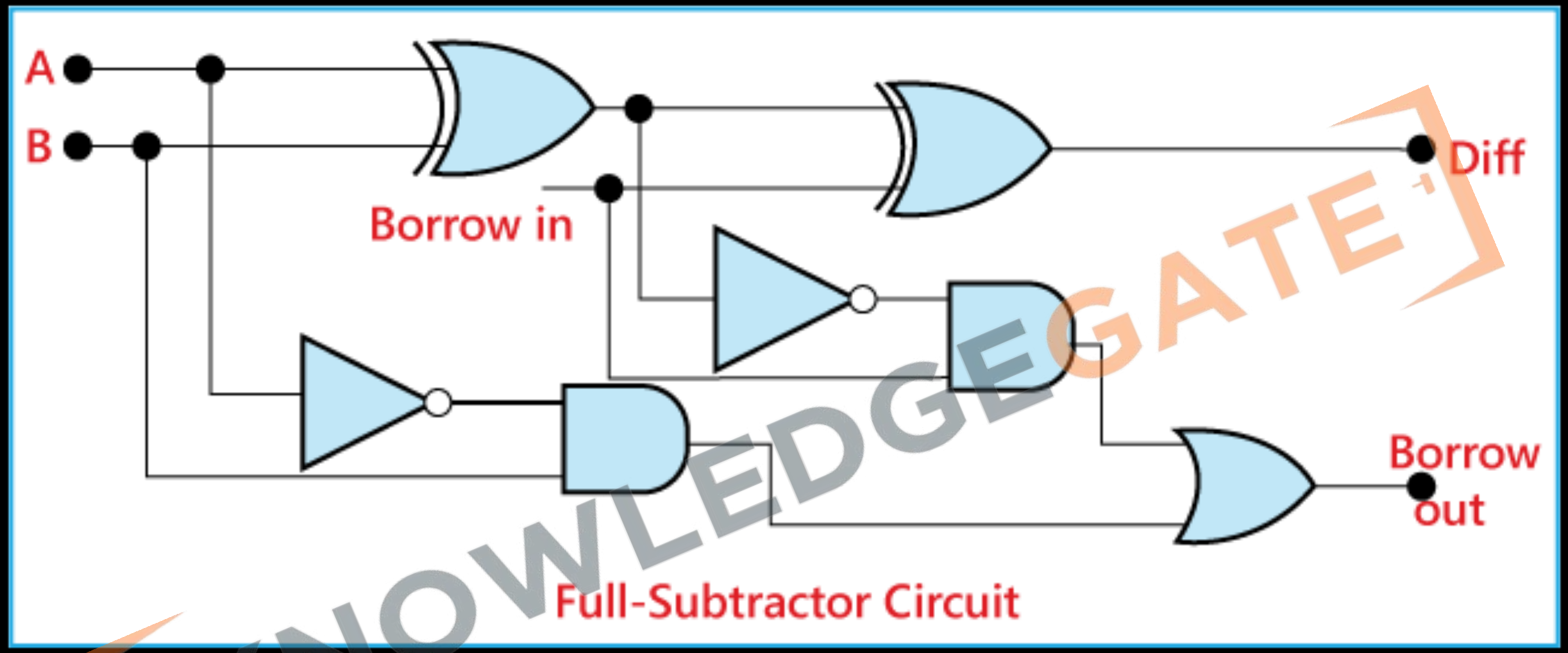
Full subtractor





Inputs			Outputs	
A	B	$Borrow_{in}$	Diff	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

KNOWLEDGEGATE
gegate.in

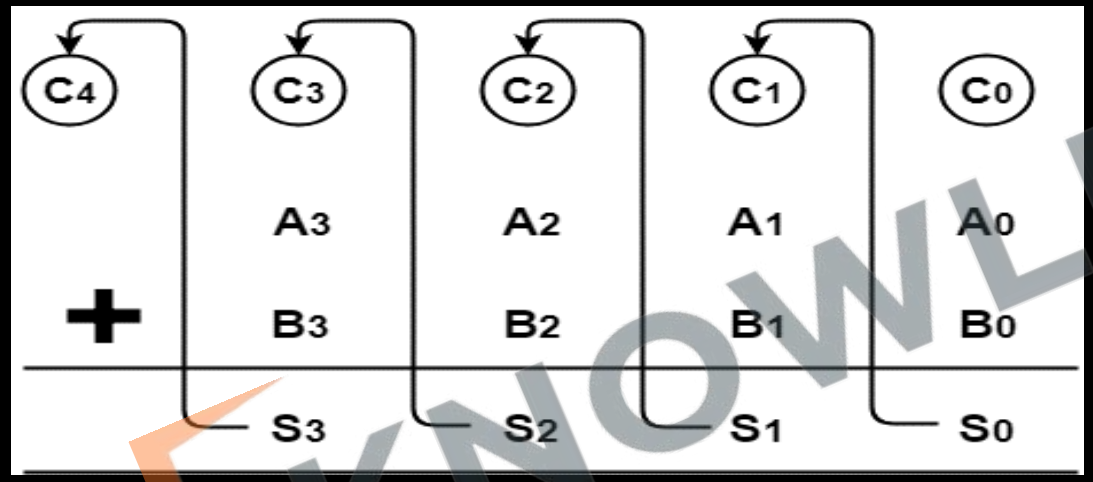


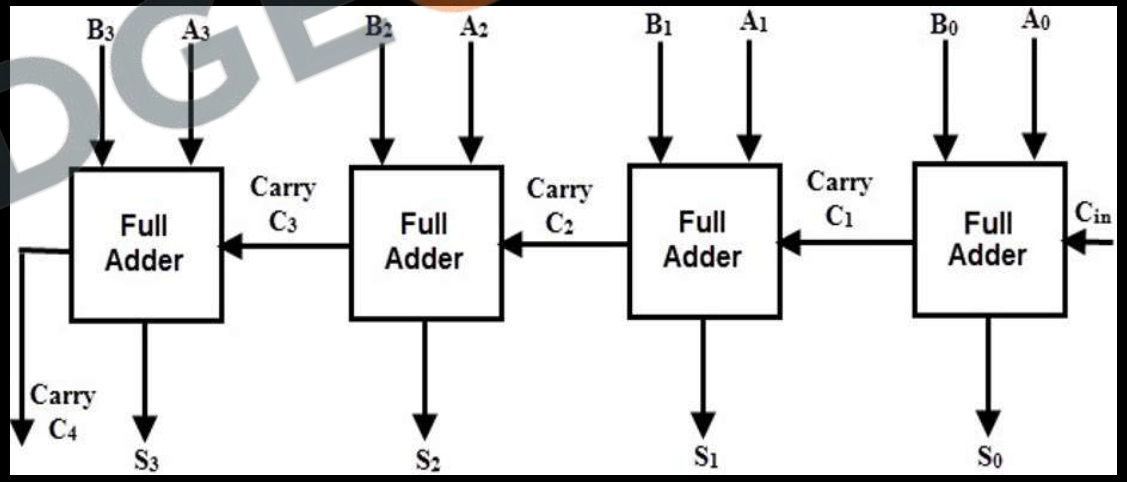
KNOWLEDGEGATE

www.knowledgegate.in

Four-bit parallel binary adder / Ripple adder

- As we know that full adder is capable of adding two 1 bit number and 1 previous carry, but in order to add binary numbers with more than one bits, additional full adders must be employed. For e.g. a four bit binary adder can be constructed using four full adders.
- Theses four full adders are connected in cascade, carry output of each adder is connected to the carry input of the next higher-order adder. So a n-bit parallel adder is constructed using ‘n’ number of full adders.

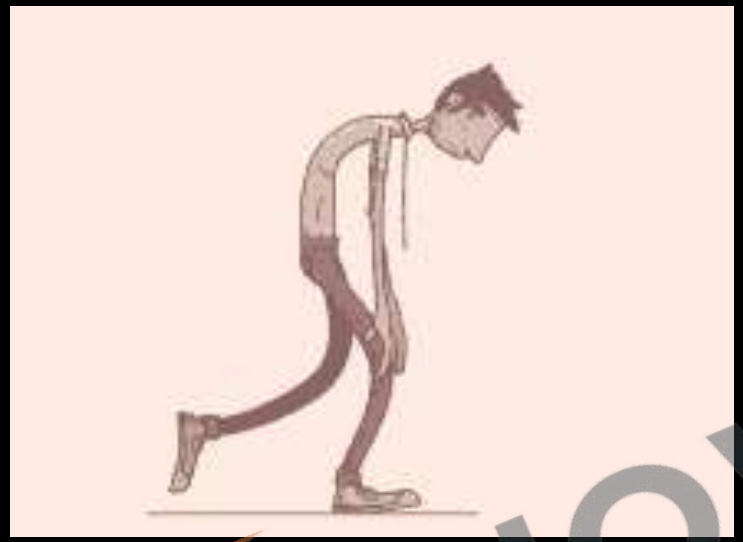




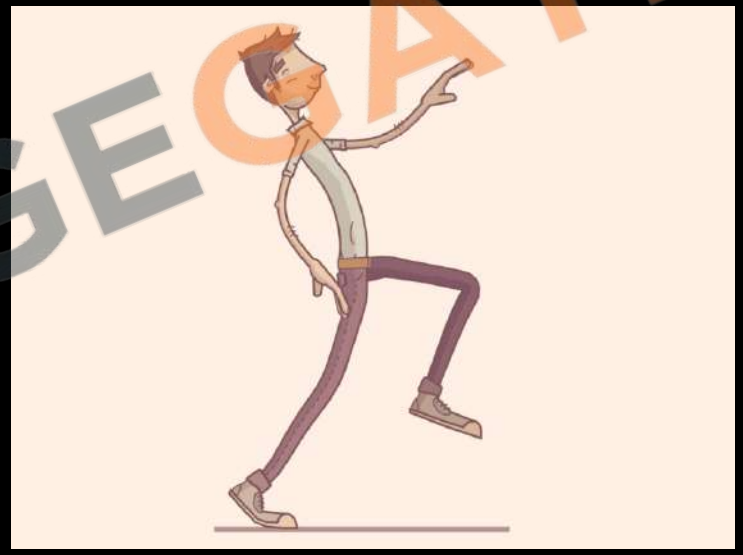
KNOWLEDGEGATE

- There are some scope of improvement in parallel binary adder / Ripple adder is it is very slow

Carry propagation delay



Look ahead Carry Generator

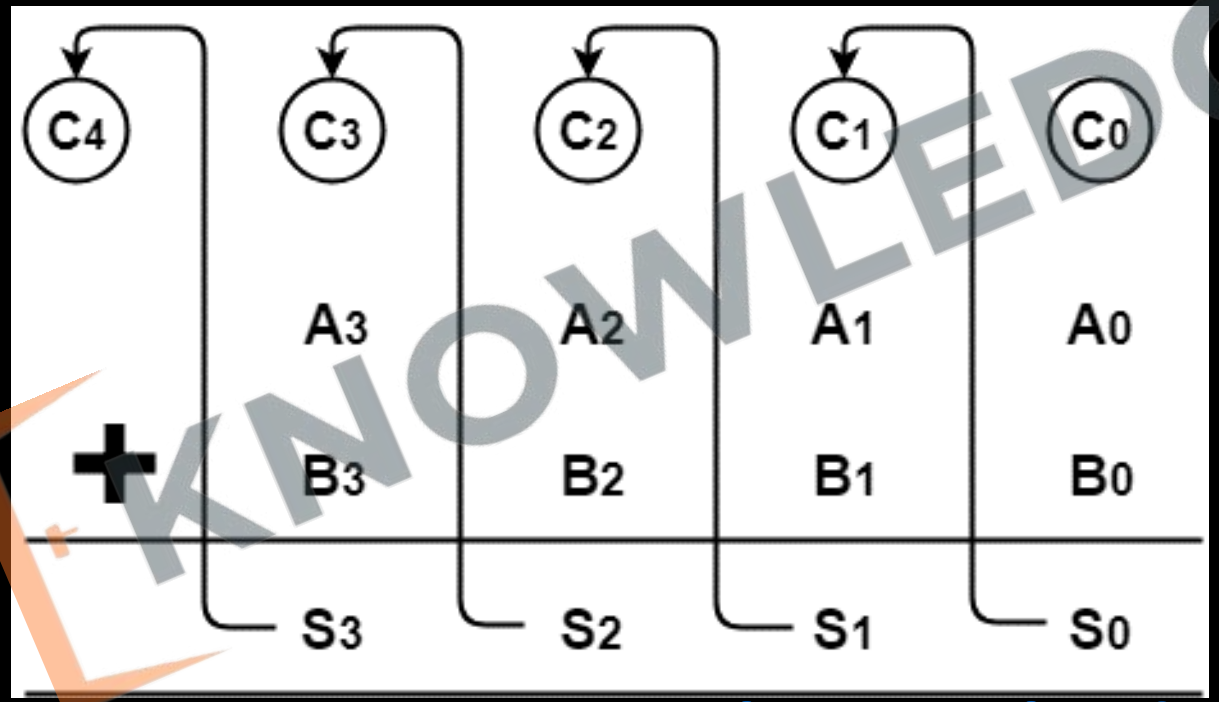


KNOWLEDGEGATE

Look ahead carry adder

- The carry propagation time is an important attribute of the adder because it limits the speed with which two numbers are added. The solution to delay is to increase the complexity of the equipment in such a way that the carry delay time is reduced.

- To solve this problem most widely used technique employs the principle of 'look ahead carry'. This method utilizes logic gates to look at the lower order bits of the augend and addend to see if a higher order carry is to be generated. It uses two functions carry generate G_i and carry propagate P_i

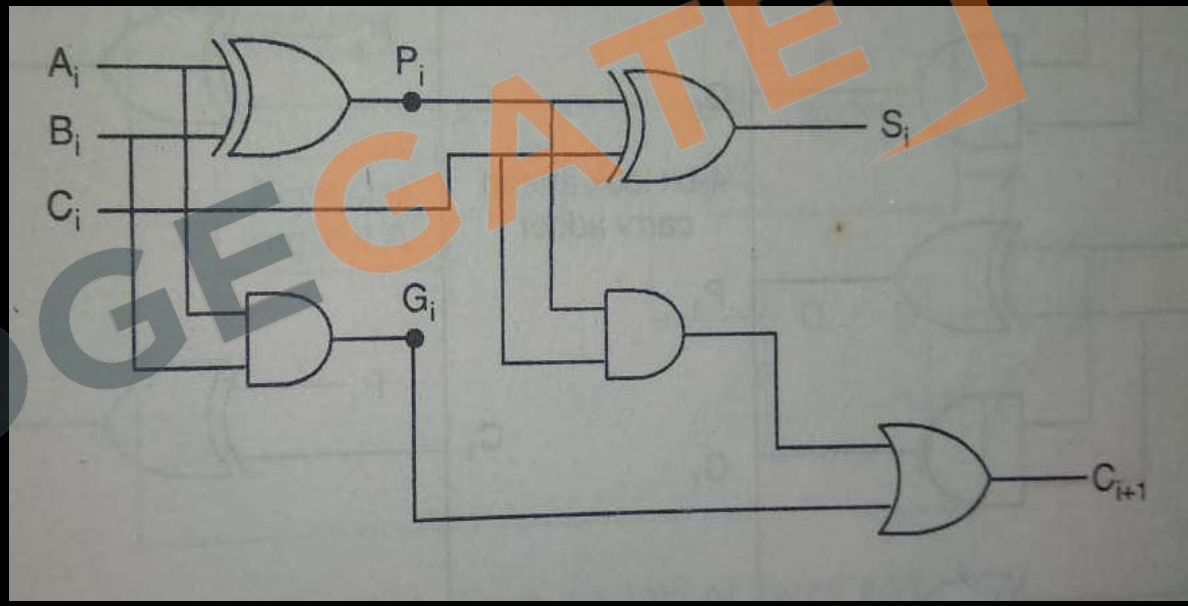


$A(A_3\ A_2\ A_1\ A_0)$

$B(B_3\ B_2\ B_1\ B_0)$

KNOWLEDGEGATE

- G_i is called a ***carry generate***, and it produces a carry of 1 when both A_i and B_i are 1, regardless of the input carry C_i.
- P_i is called a ***carry propagate***, because it determines whether a carry into stage i will propagate into stage i + 1.
- We now write the Boolean functions for the carry outputs of each stage and substitute the value of each C_i from the previous equations:



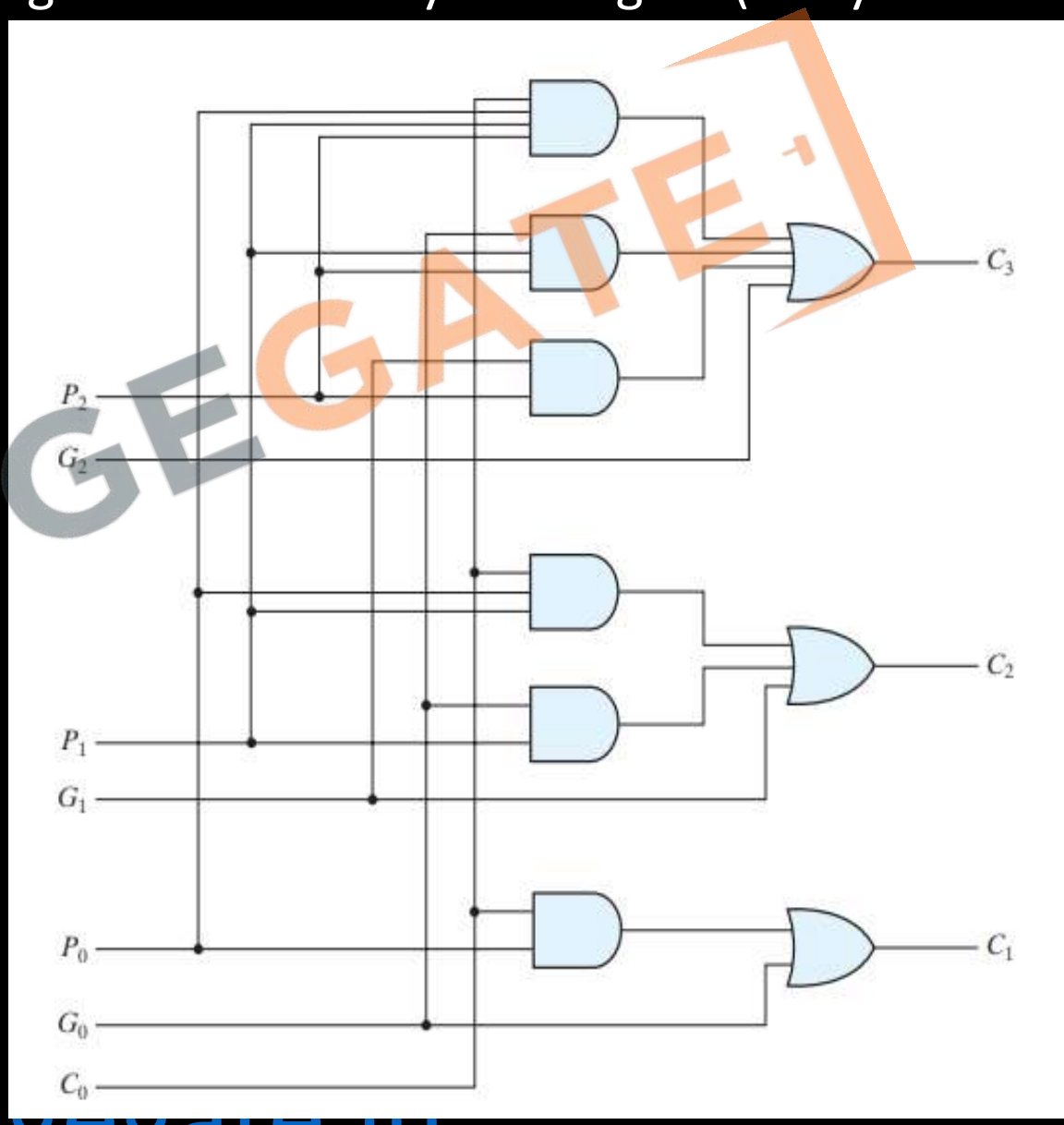
- $P_i = A_i \oplus B_i$
  $G_i = A_i \cdot B_i$
  $S_i = P_i \oplus C_i$
  $C_{i+1} = G_i + P_i C_i$

- $C_0 = 0$
- $C_1 = G_0 + P_0 C_0$
- $C_2 = G_1 + P_1G_0 + P_1P_0 C_0$
- $C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1 P_0 C_0$
- $C_4 = G_3 + G_2.P_3 + G_1.P_2.P_3 + G_0.P_1.P_2.P_3 + C_0.P_0.P_1.P_2.P_3$

KNOWLEDGE GATE

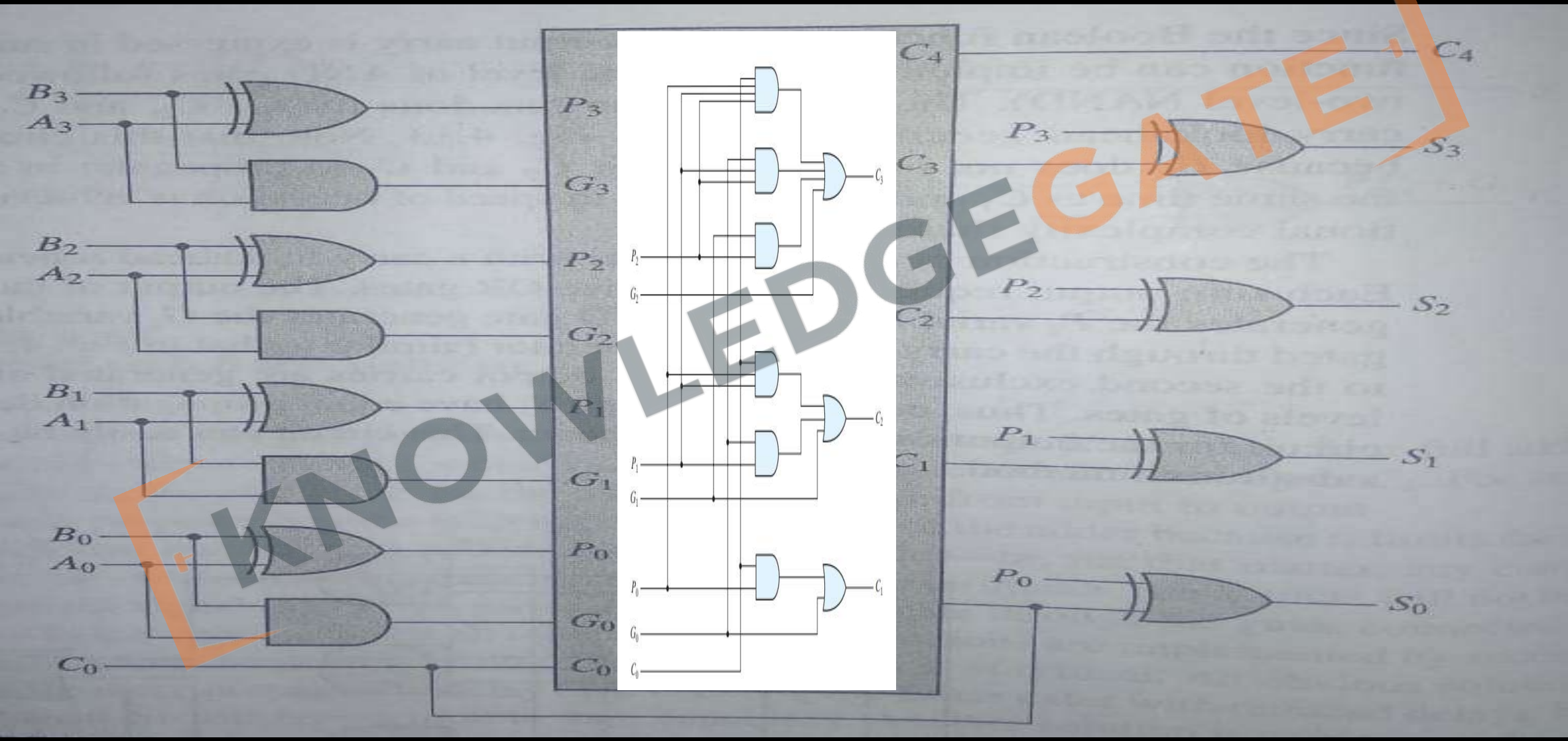
- Since the Boolean function for each output carry is expressed in sum-of-products form. Each function can be implemented with one level of AND gates followed by an OR gate (or by a two-level NAND).

- $C_0 = 0$
- $C_1 = G_0 + P_0 C_0$
- $C_2 = G_1 + P_1G_0 + P_1P_0 C_0$
- $C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1 P_0 C_0$
- $C_4 = G_3 + G_2.P_3 + G_1.P_2.P_3 + G_0.P_1.P_2.P_3 + C_0.P_0.P_1.P_2.P_3$



KNOWLEDGEGATE
www.knowledgegate.in

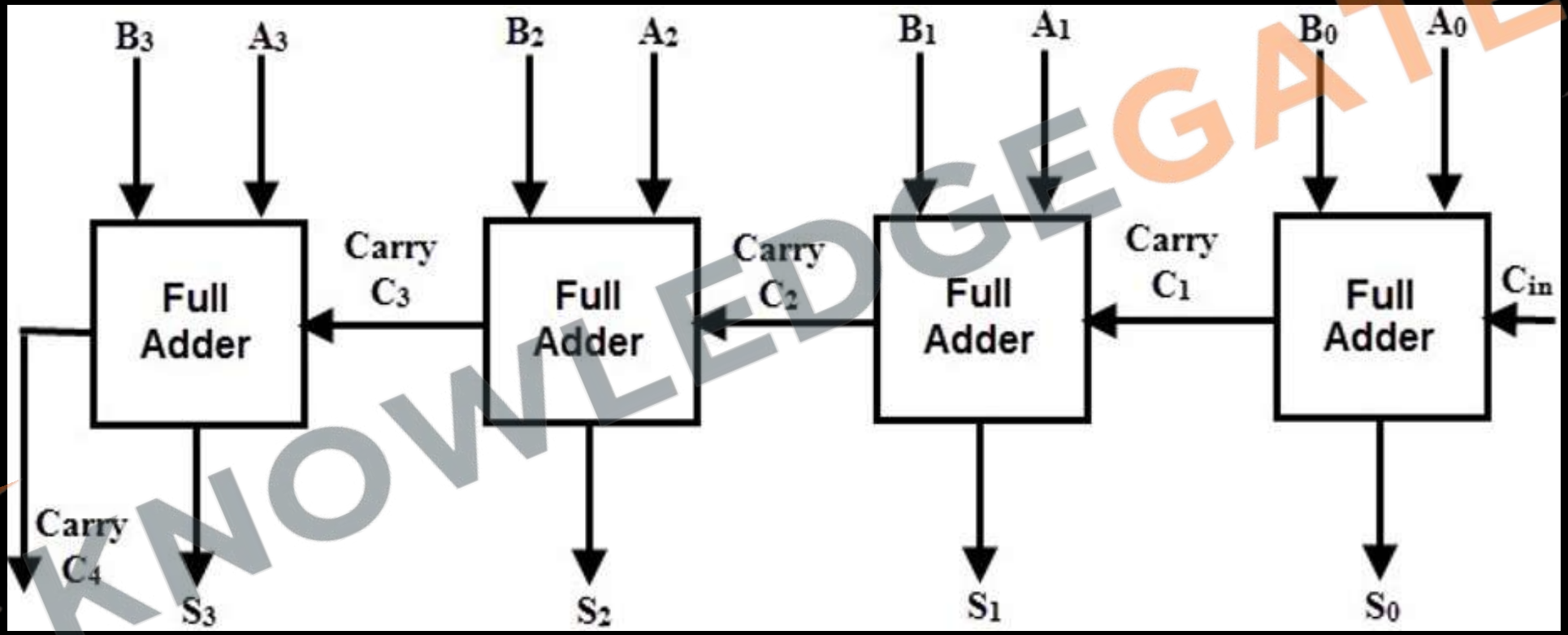
- All output carries are generated after a delay through two levels of gates. Thus, outputs S_1 through S_3 have equal propagation delay times.



KNOWLEDGEGATE

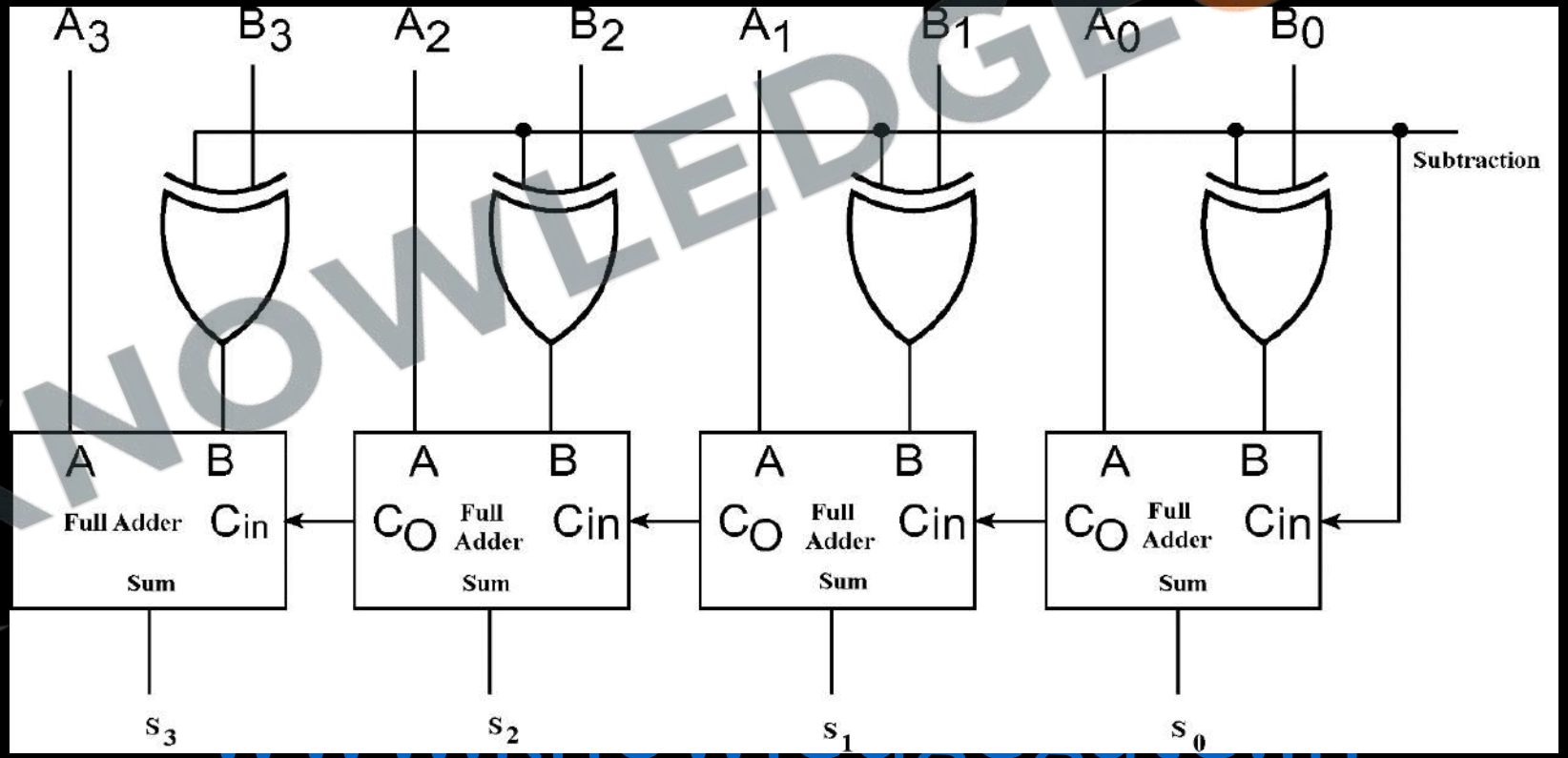
Four-bit ripple adder/subtractor

- The subtraction A - B can be done by taking the 2's complement of B and adding it to A.
- A + (-B)



KNOWLEDGEGATE

- The circuit for subtracting A - B consists of an adder with inverters placed between each data input B and the corresponding input of the full adder. The mode input M controls the operation. When M = 0, the circuit is an adder, and when M = 1, the circuit becomes a subtractor.
- When M = 0, we have B $\oplus$ 0 = B. The full adders receive the value of B, the input carry is 0, and the circuit performs A plus B.
- When M = 1, we have B $\oplus$ 1 = B' and C_0 = 1. The B inputs are all complemented and a 1 is added through the input carry. The circuit performs the operation A plus the 2's complement of B.



KNOWLEDGEGATE

Booth’s algorithm

- **Andrew Donald Booth** (11 February 1918 – 29 November 2009) was a British electrical engineer, physicist and computer scientist, who was an early developer of the magnetic drum memory for computers. He is known for Booth's multiplication algorithm.

- Booth algorithm optimizes binary multiplication by reducing the number of additions and subtractions based on the multiplier bits.

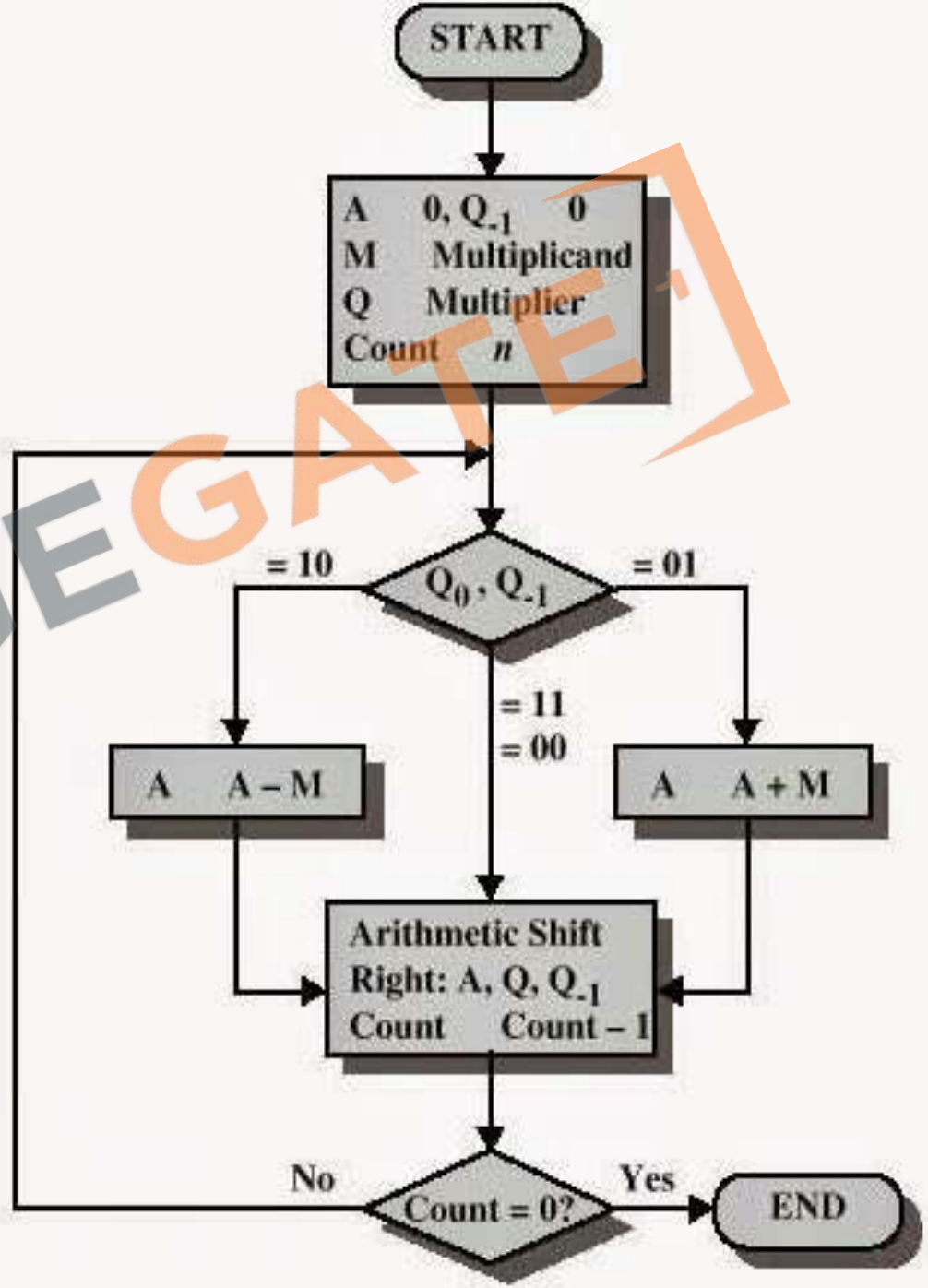
- The process involves examining multiplier bits, then either adding, subtracting, or leaving the multiplicand unchanged before shifting the partial product.

KNOWLEDGEGATE

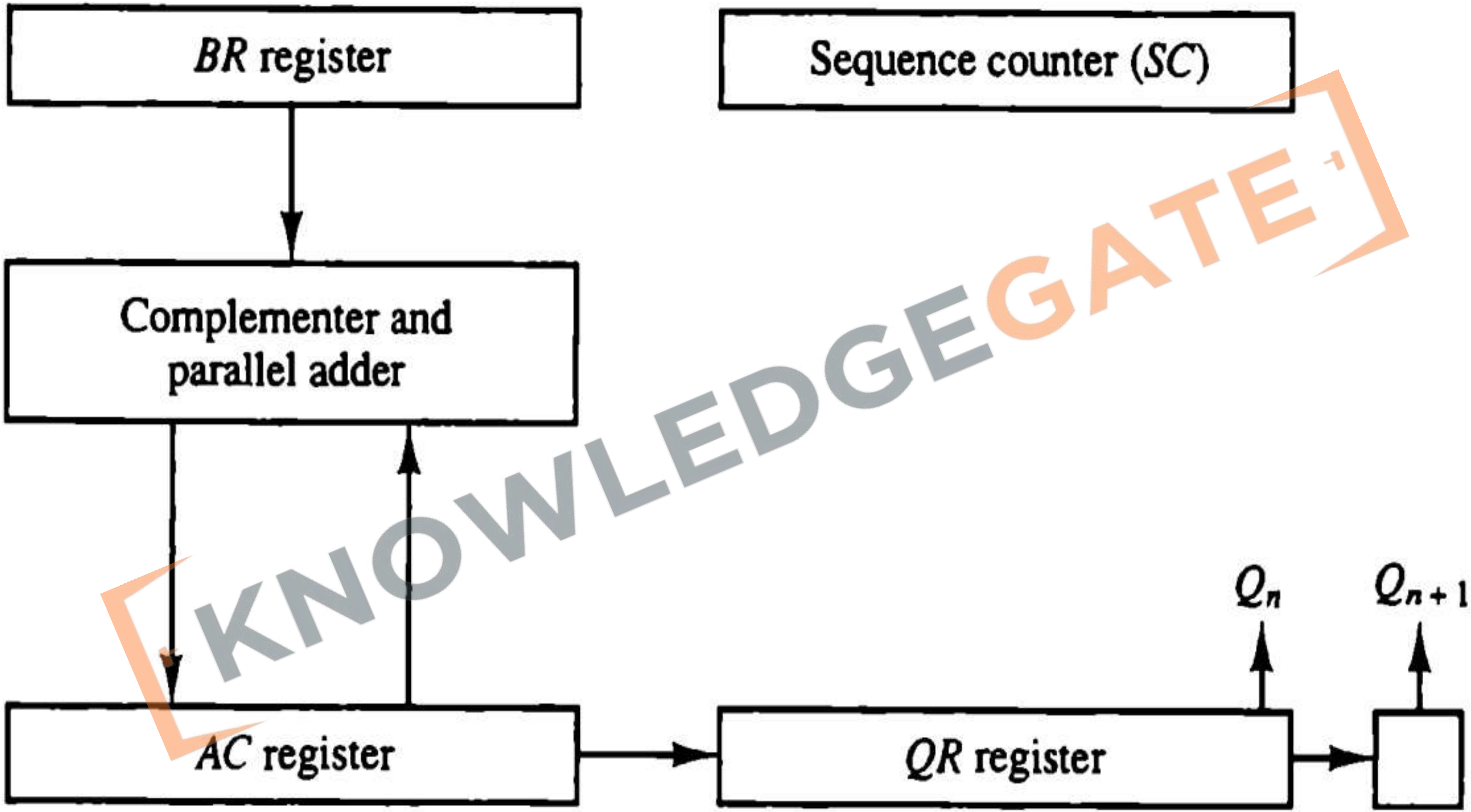
www.knov

A	Q	Q_{-1}	M		
0000	0011	0	0111	Initial values	
1001	0011	0	0111	A ← A − M	First cycle
1100	1001	1	0111	Shift	
1110	0100	1	0111	Shift	Second cycle
0101	0100	1	0111	A ← A + M	Third cycle
0010	1010	0	0111	Shift	
0001	0101	0	0111	Shift	Fourth cycle

Example of Booth's Algorithm (7 × 3)



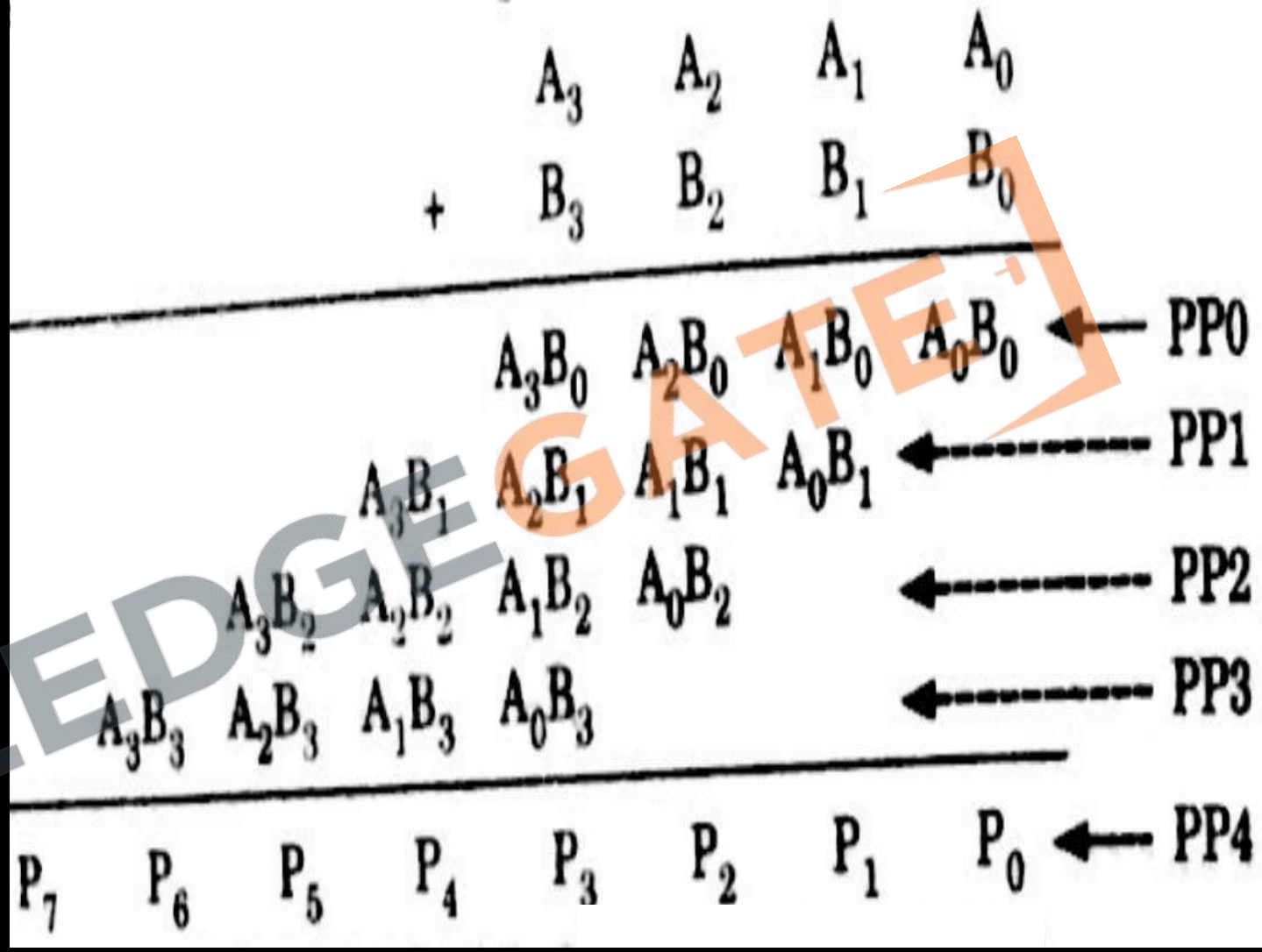
KNOWLEDGEGATE



KNOWLEDGEGATE

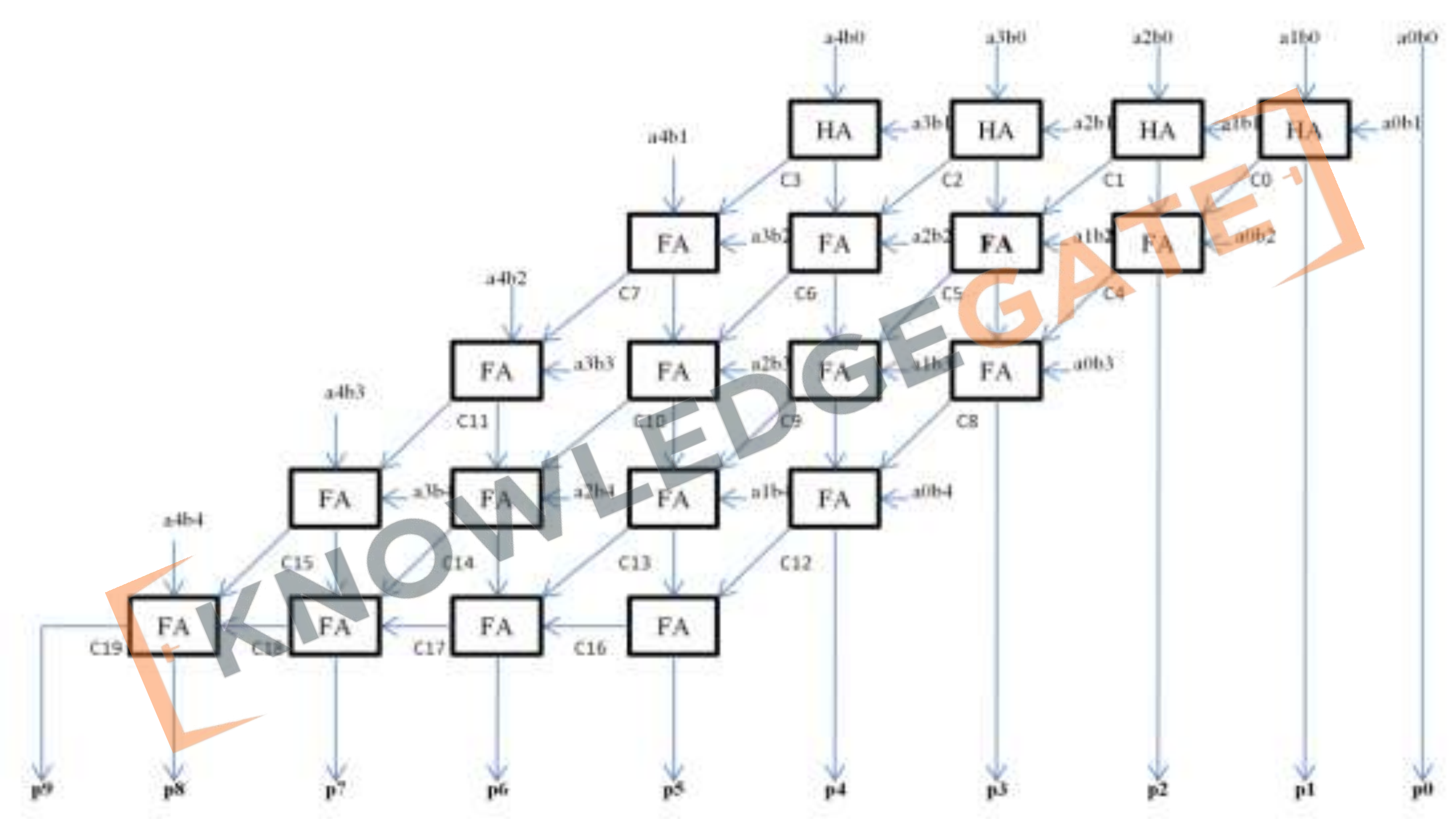
Array multiplier

- Array multiplier uses a grid of full and half adders to perform nearly simultaneous addition of product terms.
- AND gates are used to form these product terms before they are fed into the adder array.
- Unlike sequential multipliers that check bits one at a time, array multipliers form the product bits all at once, making the operation faster.
- The speed advantage comes at a cost of requiring a large number of gates, which became economical with the advent of integrated circuits.



KNOWLEDGEGATE

www.knowledgegate.in



KNOWLEDGEGATE

1 1 0 0

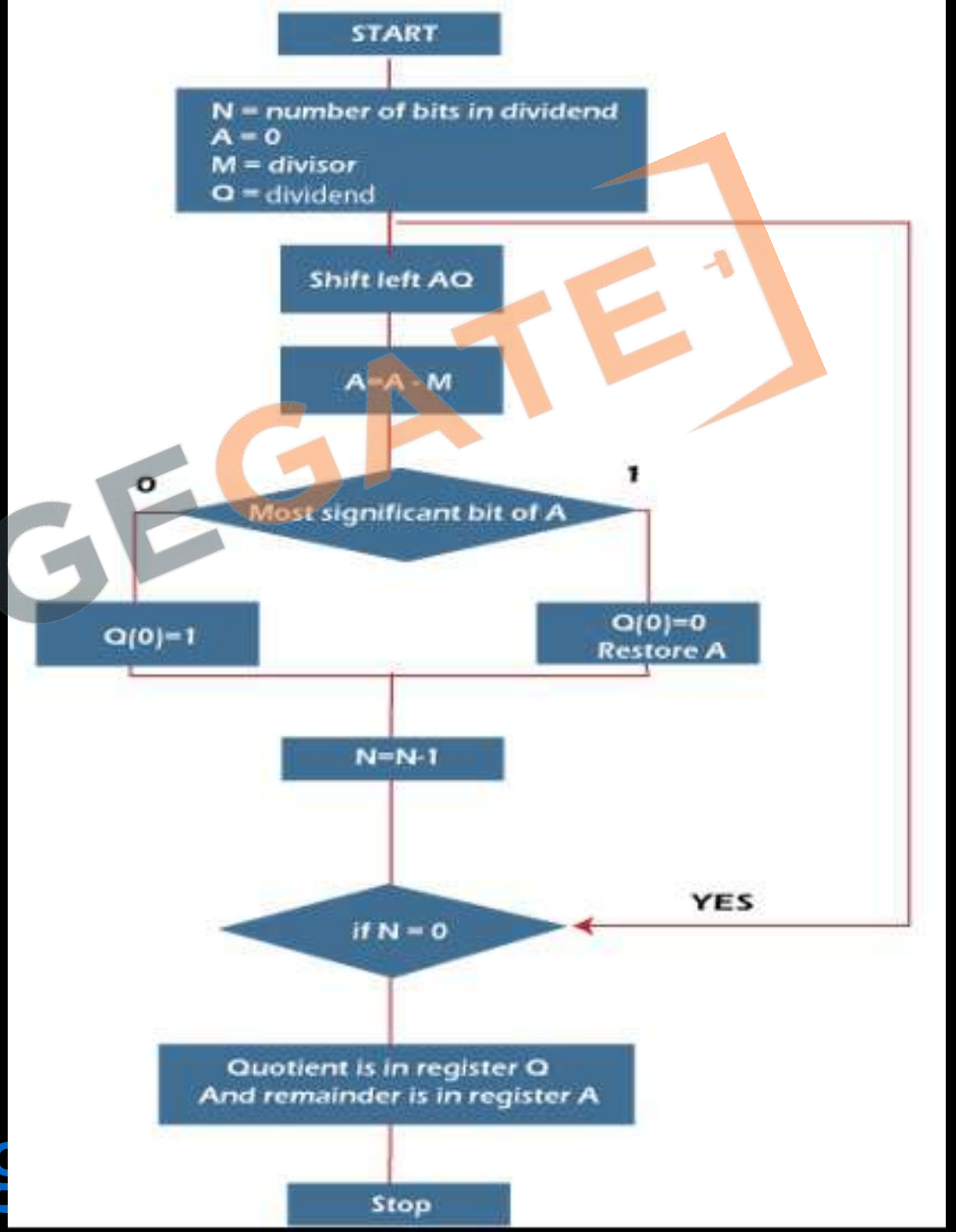
0 1 1 0

KNOWLEDGEGATE

www.knowledgegate.in

Restoring Division Algorithm

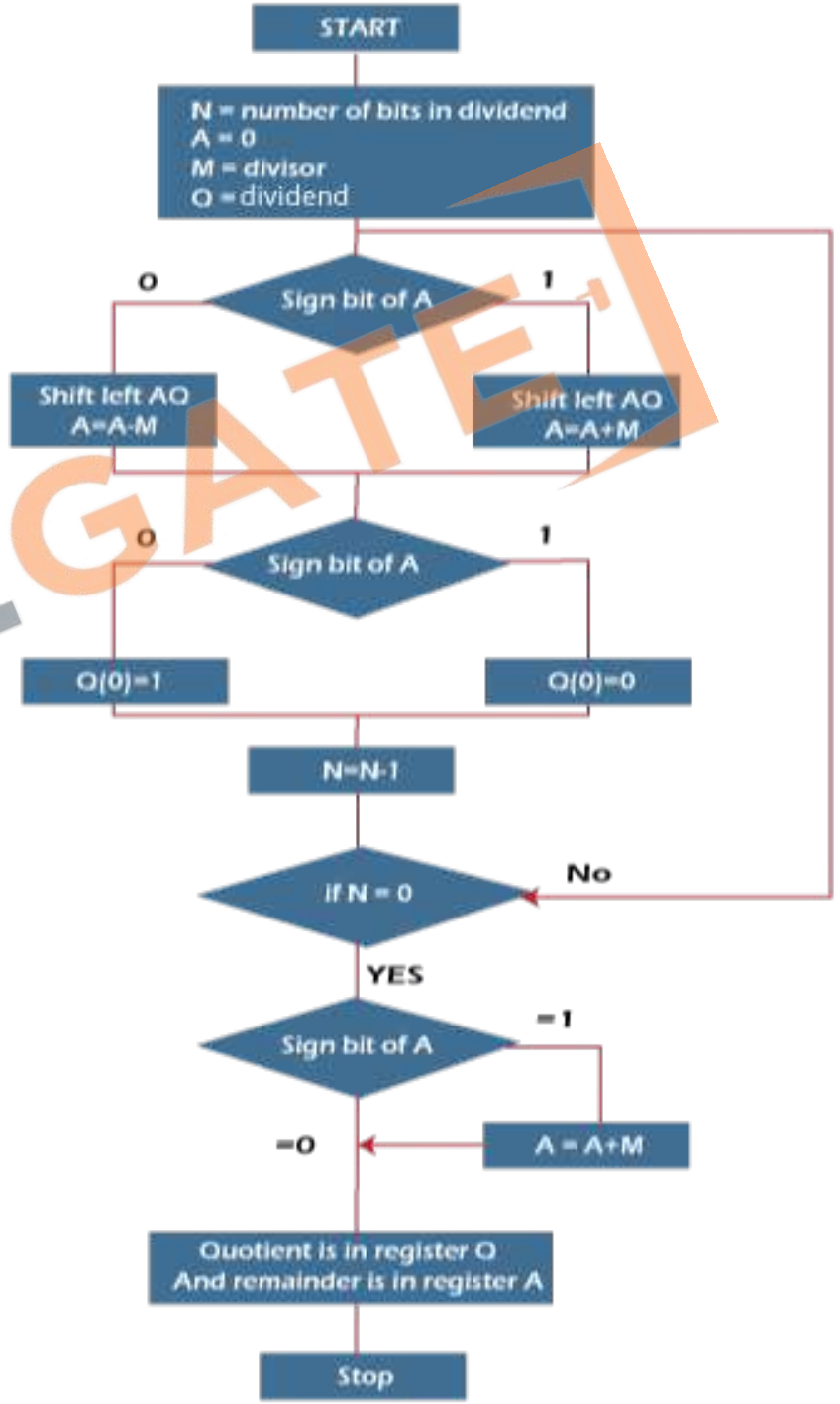
n	M	A	Q	Action/Operation
4	00011	00000	1011	Initialization
		00001	011?	SL AQ
		11110	011?	A = A-M
3	00011	00001	0110	QO ← 0
		00010	110?	SL AQ
		11111	110?	A = A-M
2	00011	00010	1100	QO ← 0
		00101	100?	SL AQ
		00010	100?	A = A-M
1	00011	00010	1001	QO ← 1
		00101	001?	SL AQ
		00010	001?	A = A-M
0	00011	00010	0011	QO ← 1



KNOWLEDGEGATE

Non-Restoring Division Algorithm

n	M	A	Q	Action/Operation
4	00011	00000	1011	Initialization
		00001	011?	SL AQ
		11110	011?	A = A-M
3	00011	11110	0110	QO ← 0
		11100	110?	SL AQ
		11111	110?	A = A+M
2	00011	11111	1100	QO ← 0
		11111	100?	SL AQ
		00010	100?	A = A+M
1	00011	00010	1001	QO ← 1
		00101	001?	SL AQ
		00010	001?	A = A-M
0	00011	00010	0011	QO ← 1



KNOWLEDGEGATE

Q Add -35 and -31 in binary using 8-bit registers, in signed 1's complement and signed 2's complement?

KNOWLEDGEGATE

www.knowledgegate.in

Floating point representation

- Problem with representation we have already studied is that it do not works well if the number to be stored is either too small or too large, is it take very large amount of memory.
- Imagine a number $6.023 * 10^{23}$ will require around 70 bits to be stored.
- So in scientific application or statics it is a problem to store very small or very large number.

KNOWLEDGEGATE

- Floating point representation a special kind of sign magnitude representation.
- Floating point number is stored in mantissa/exponent form i.e. $m*r^e$
- Mantissa is a signed magnitude fraction for most of the cases.
- The exponent is stored in biased form.

KNOWLEDGEGATE

www.knowledgegate.in

- The biased exponent is an unsigned number representing signed true exponent.
- If the biased exponent field contains K bits, the biased = 2^{k-1}
- True value expression is $V = (-1)^{S}(.M)_2 * 2^{E-Bias}$, note it is explicit representation
- True value expression is $V = (-1)S(1.M)_2 * 2^{E-Bias}$, note it is implicit representation, it has more precision than explicit normalization.

KNOWLEDGEGATE

- Biased exponent(E) = True exponent(e) + Bias
- Range of true exponent: from -2^{k-1} to $+2^{k-1}-1$, where k is number of bits assigned for exponent
- After adding bias 2^{k-1}, new range go from 0 to $2^{k}-1$

KNOWLEDGEGATE

www.knowledgegate.in

- How to convert a signed number into floating point representation
- The floating-point normalized number distribution is not uniform. Representable number are dense towards zero and spared towards max value
- This uneven distribution result negligible effect of rounding towards zero and dominant towards max value.

KNOWLEDGEGATE

www.knowledgegate.in

Q Represent -21.75 (s=1, k=7, m=8)

KNOWLEDGEGATE

www.knowledgegate.in

IEEE 754 floating point standard

- The **IEEE Standard for Floating-Point Arithmetic** (**IEEE 754**) is a technical standard for floating-point arithmetic established in 1985 by the Institute of Electrical and Electronics Engineers (IEEE).

- The standard addressed many problems found in the diverse floating-point implementations that made them difficult to use reliably and portably. Many hardware floating-point units use the IEEE 754 standard.

KNOWLEDGEGATE

IEEE 754 floating point standard

- IEEE 754 standard represent floating point number standards
- It gives a provision for +-0 & +-∞ (by reserving certain pattern for Mantissa/Exponent pattern)
- there are number of modes for storage from half precision (16 bits) to very lengthy notations
- based of the system is 2
- the floating-point number can be stored either in implicit normalized form or in fractional form
- If the biased exponent field contains K bits, the biased = $2^{k-1} -1$
- Certain Mantissa/Exponent pattern does not denote any number (NAN i.e. not a number)

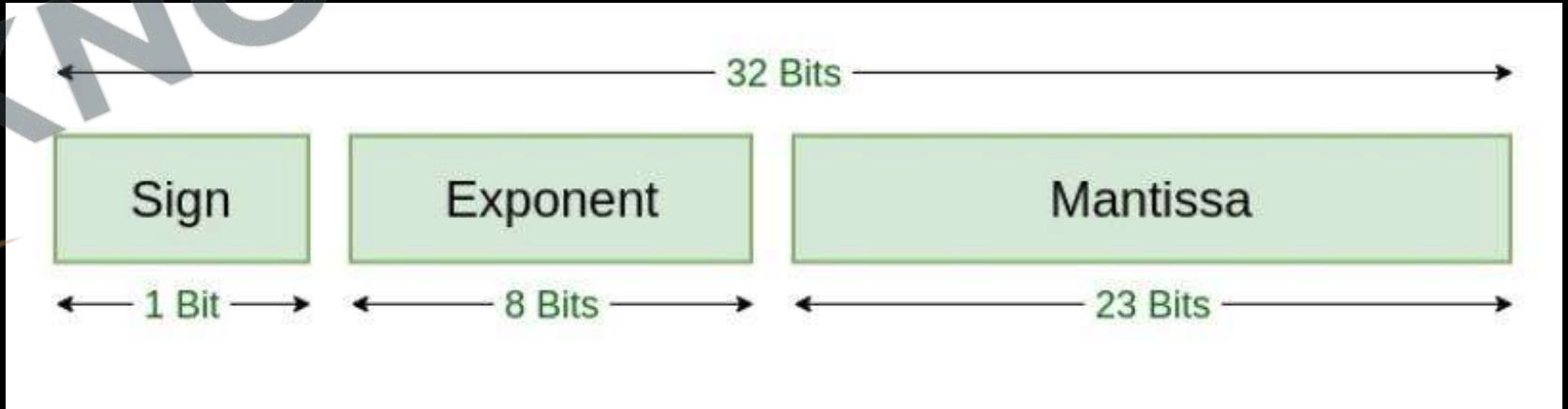
KNOWLEDGEGATE

Name	Common name	Mantissa Bits	Exponent bits	Exponent bias	E min	E max
binary16	Half precision	10	5	$2^{5-1}-1 = 15$	−14	+15
binary32	Single precision	23	8	$2^{8-1}-1 = 127$	−126	+127
binary64	Double precision	52	11	$2^{11-1}-1 = 1023$	−1022	+1023
binary128	Quadruple precision	112	15	$2^{15-1}-1 = 16383$	−16382	+16383
binary256	Octuple precision	236	19	$2^{19-1}-1 = 262143$	−262142	+262143

KNOWLEDGEGATE
www.knowledgegate.in

Single pression

Sign bit (1)	Exponent (8)	Mantissa (23)	Value
0	00…….0(E=0)	00…….0(M=0)	+0
1	00…….0(E=0)	00…….0(M=0)	-0
0	11……1(E=255)	00…….0(M=0)	+∞
1	11……1(E=255)	00…….0(M=0)	-∞
0/1	1<=E<=254	XX……X (M! =0)	Implicit normalised number
0/1	00…….0(E=0)	XX……X (M! =0)	Fraction
0/1	11……1(E=255)	XX……X (M! =0)	NAN (Not a Number)



KNOWLEDGEGATE

Q Consider the following representation of a number in IEEE 754 single-precision floating point format with a bias of 127.

S : 1 E : 10000001 F : 11110000000000000000000

Here, S,E and F denote the sign, exponent, and fraction components of the floating point representation. The decimal value corresponding to the above representation (rounded to 2 decimal places) is _____.

KNOWLEDGEGATE

www.knowledgegate.in

Q Given the following binary number in 32-bit (single precision) IEEE-754 format:

00111110011011010000000000000000

The decimal value closest to this floating-point number is

KNOWLEDGEGATE

www.knowledgegate.in

Q The decimal value 0.5 in IEEE single precision floating point representation has

a) fraction bits of 000…000 and exponent value of 0

b) fraction bits of 000…000 and exponent value of −1

c) fraction bits of 100…000 and exponent value of 0

d) no exact representation

KNOWLEDGEGATE

www.knowledgegate.in

Double Precision

KNOWLEDGEGATE

www.knowledgegate.in

Instruction format

- In general, based on the number of operands or reference made in the instructions. Instructions can be classified into the following types: -
  - 3 Address Instruction
  - 2 Address Instruction
  - 1 Address Instruction
  - 0 Address Instruction

KNOWLEDGEGATE

www.knowledgegate.in

3 Address Instruction

Mode/Opcode	Destination	$Source_1$	$Source_2$

- Three-address instruction is a format of machine instruction. It has one opcode and three address fields. One address field is used for destination and two address fields for source.

- X = (A + B) x (C + D)

ADD R_1, A, B	R_1 <- M[A] + M[B]
ADD R_2, C, D	R_2 <- M[C] + M[D]
MUL X, R_1, R_2	M[X] <- R_1 x R_2

KNOWLEDGEGATE

www.knowledgegate.in

- It produces short program much easier to understand, but it does not mean that program will run much faster because now instruction only contain more information but each micro operation (changing content of register, loading address in address bus etc.) will be performed in one cycle only.

- Many bits are needed for instructions encoded in binary to define three addresses. More advanced processing and decoding circuits are needed.

KNOWLEDGEGATE

www.knowledgegate.in

2 Address Instruction

Mode/Opcode	Destination/Source$_1$	Source$_2$

- Here two addresses can be specified in the instruction. In effort to reduce the size of instruction here we remove the reference for result and Source$_1$ is used for storing the result.
- X = (A + B) x (C + D)

MOV	R_1, A	$R_1 \leftarrow M[A]$
ADD	R_1, B	$R_1 \leftarrow R_1 + M[B]$
MOV	R_2, C	$R_2 \leftarrow C$
ADD	R_2, D	$R_2 \leftarrow R_2 + D$
MUL	R_1, R_2	$R_1 \leftarrow R_1 * R_2$
MOV	X, R_1	$M[X] \leftarrow R_1$

KNOWLEDGEGATE
www.knowledgegate.in

- Less length compares to 3 address, more efficient, no of register required are less.

- Here code optimization is relatively difficult.

KNOWLEDGEGATE

www.knowledgegate.in

1 Address Instruction

Mode/Opcode	Destination/Source

- One address instruction uses an implied accumulator (AC) register for all data manipulation.
- One operand is in the accumulator and the other is in the register or memory location. Implied means that the CPU already knows that one operand is in the accumulator so there is no need to specify it.
- X = (A + B) x (C + D)

LOAD	A	AC ← M[A]
ADD	B	AC ← AC + M[B]
STORE	T	M[T] ← AC
LOAD	C	AC ← M[C]
ADD	D	AC ← AC + M[D]
MUL	T	AC ← AC * M[T]
STORE	X	M[X] ← AC

KNOWLEDGEGATE
www.knowledgegate.in

0 Address Instruction

Mode/Opcode

- Early computers which do not have complex circuit like ALU use few registers which are used as organised stack. While working on these stacks we do not require location of result as it is implied at the top of the stack. A stack organized computer does not use an address field for the instructions ADD and MUL. The push and pop instruction, however, need an address field to specify the operand that communicates with stack.
- X = (A + B) x (C + D)

PUSH	A	TOP ← A
PUSH	B	TOP ← B
ADD		TOP ← A+B
PUSH	C	TOP ← C
PUSH	D	TOP ← D
ADD		TOP ← C+D
MUL		TOP ← (C+D)*(A+B)
POP	X	M[X] ← TOP

KNOWLEDGEGATE
www.knowledgegate.in

- To evaluate arithmetic expression in a stack computer, it is necessary to Covert the expression into reverse polish notation. The name zero-address is given to this type of computer because of the absences of an address field in the com.

- Here mode/opcode is sufficient to perform any operation.

- It is very simple and very short, but complex to design and speed is very poor.

KNOWLEDGEGATE

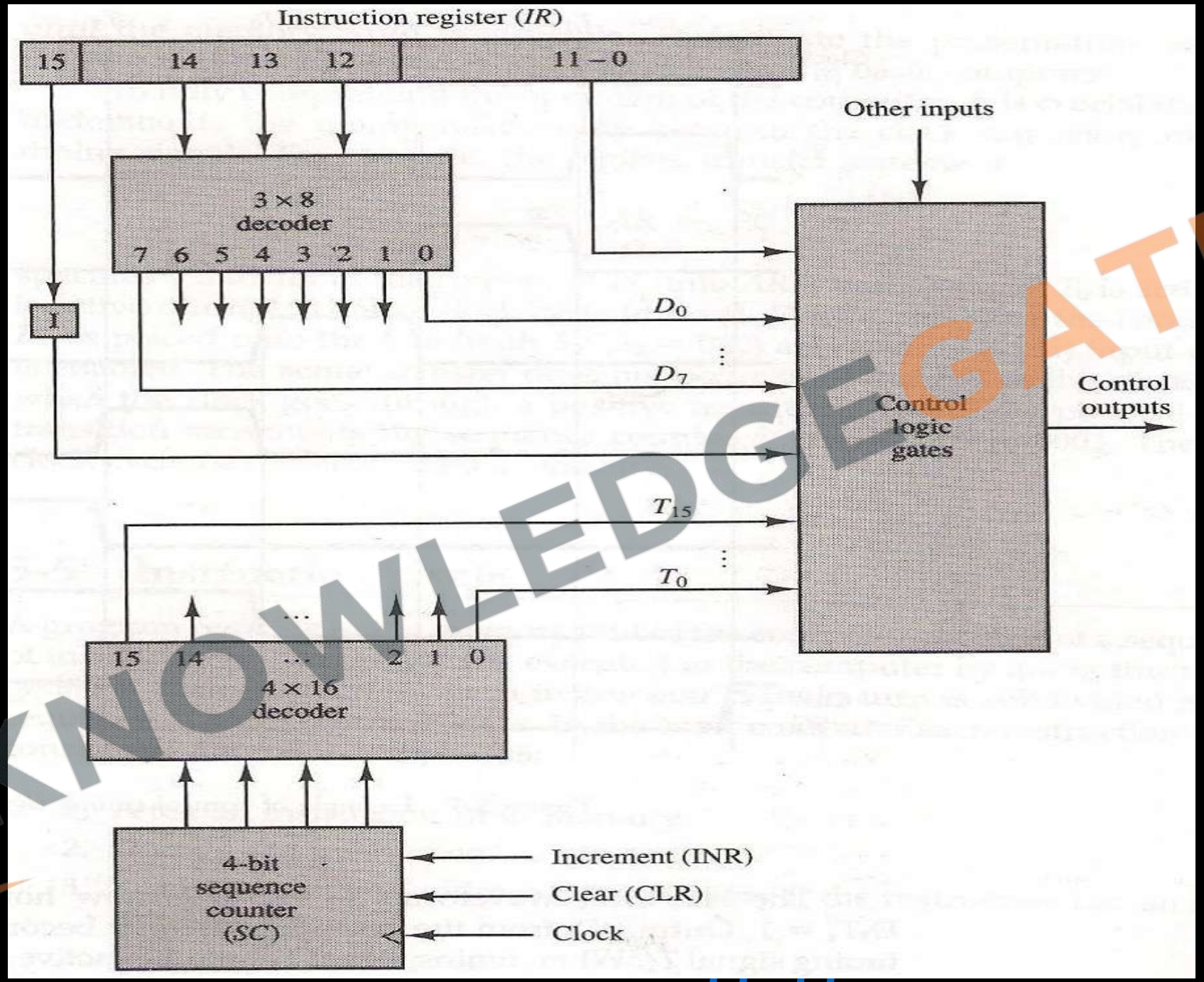
www.knowledgegate.in

Timing Circuit

- In order to perform a instruction we need to perform a number of micro-operators, and these micro-operations must be timed
  - AR ← PC
  - IR ← M[AR], PC ← PC + 1
- we should distinguish one clock pulse from another during the execution of instruction
- Sequence counter followed by decoder is used to generate time signals

KNOWLEDGEGATE

www.knowledgegate.in


Instruction register (IR)
15
14
13
12
11 – 0
Other inputs
3 × 8
decoder
7 6 5 4 3 2 1 0
1
D_0
D_7
Control
logic
gates
Control
outputs
T_{15}
T_0
15 14 ... 2 1 0
4 × 16
decoder
4-bit
sequence
counter
(SC)
Increment (INR)
Clear (CLR)
Clock

Instruction Cycle - A program residing in the memory unit of the computer consists of a sequence of instructions. The program is executed in the computer by going through a cycle for each instruction. Each instruction cycle in turn is subdivided into a sequence of sub cycles or phases. from fetching of instruction to the completion of execution of instruction whatever happens is called instruction cycle. The reason we called this cycle because it will happen for every instruction.

- Each instruction cycle consists of the following phases:
  - Fetch an instruction from memory.
  - Decode the instruction.
  - Read the effective address from memory if the instruction has an indirect address.
  - Execute the instruction.
  - Fetch and Decode

KNOWLEDGEGATE

www.knowledgegate.in

Micro-Operation

- A micro-operation is a simple operation that can be performed during one clock period.
- The result of this operation may replace the previous binary information of register or the result may be transferred to another register.
- Examples of micro-operations are shift, move, count, add and load etc.
- The micro-operations most often encountered in digital computers are classified into four categories:
  - **Register transfer micro-operations**: It transfer binary information from one register to another.
  - **Arithmetic micro-operations**: It perform arithmetic operation on numeric data stored in registers.
  - **Logic micro-operations**: It perform bit manipulation operations on non-numeric data stored in registers.
  - **Shift micro-operations**: It perform shift operations on data stored in registers.

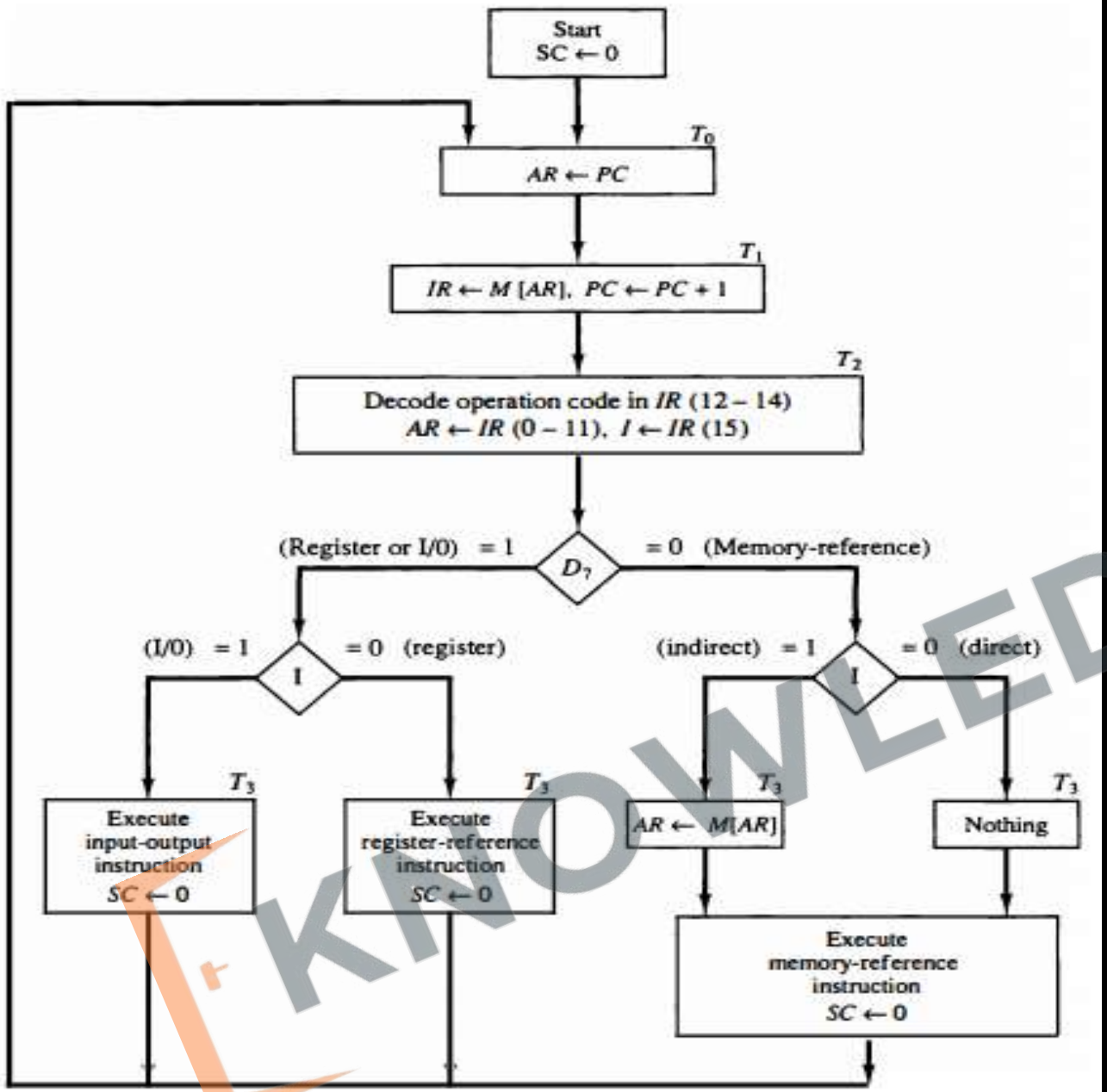
KNOWLEDGEGATE

- Initially, the program counter PC is loaded with the address of the first instruction in the program.
- The sequence counter SC is cleared to 0, providing a decoded timing signal T_0.
- After each clock pulse, SC is incremented by one, so that the timing signals go through a sequence T_0, T_1, T_2, and so on.
- The microoperations for the fetch and decode phases can be specified by the following register transfer statement

$$T_0: \quad AR \leftarrow PC$$
$$T_1: \quad IR \leftarrow M[AR], \quad PC \leftarrow PC + 1$$
$$T_2: \quad D_0, \ldots, D_7 \leftarrow \text{Decode } IR(12\text{–}14), \quad AR \leftarrow IR(0\text{–}11), \quad I \leftarrow IR(15)$$

KNOWLEDGEGATE

www.knowledgegate.in



- Determine the Type of Instruction
- The timing signal that is active after the decoding is T_3.
- During time T_3, the control unit determines the type of instruction that was just read from memory.
- Decoder output D, is equal to 1 if the operation code is equal to binary 111.
- If $D_7 = 1$, the instruction must be a register-reference or input-output type.
- If $D_7 = 0$, the operation code must be one of the other seven values 000 through 110, specifying a memory-reference instruction.
- Control then inspects the value of the first bit of the instruction, which is now available in flip-flop I. If $D_7 = 0$ and I = 1, we have a memory reference instruction with an indirect address.
- In case of indirect address, it is necessary to read effective address from memory. The microoperation for the indirect address condition can be symbolized by the register transfer statement
- AR ← M [AR]
- When a memory-reference instruction with I = 0 is encountered, it is not necessary to do anything since the effective address is already in AR.
- The sequence counter SC is either incremented or cleared to 0 with every positive clock transition.

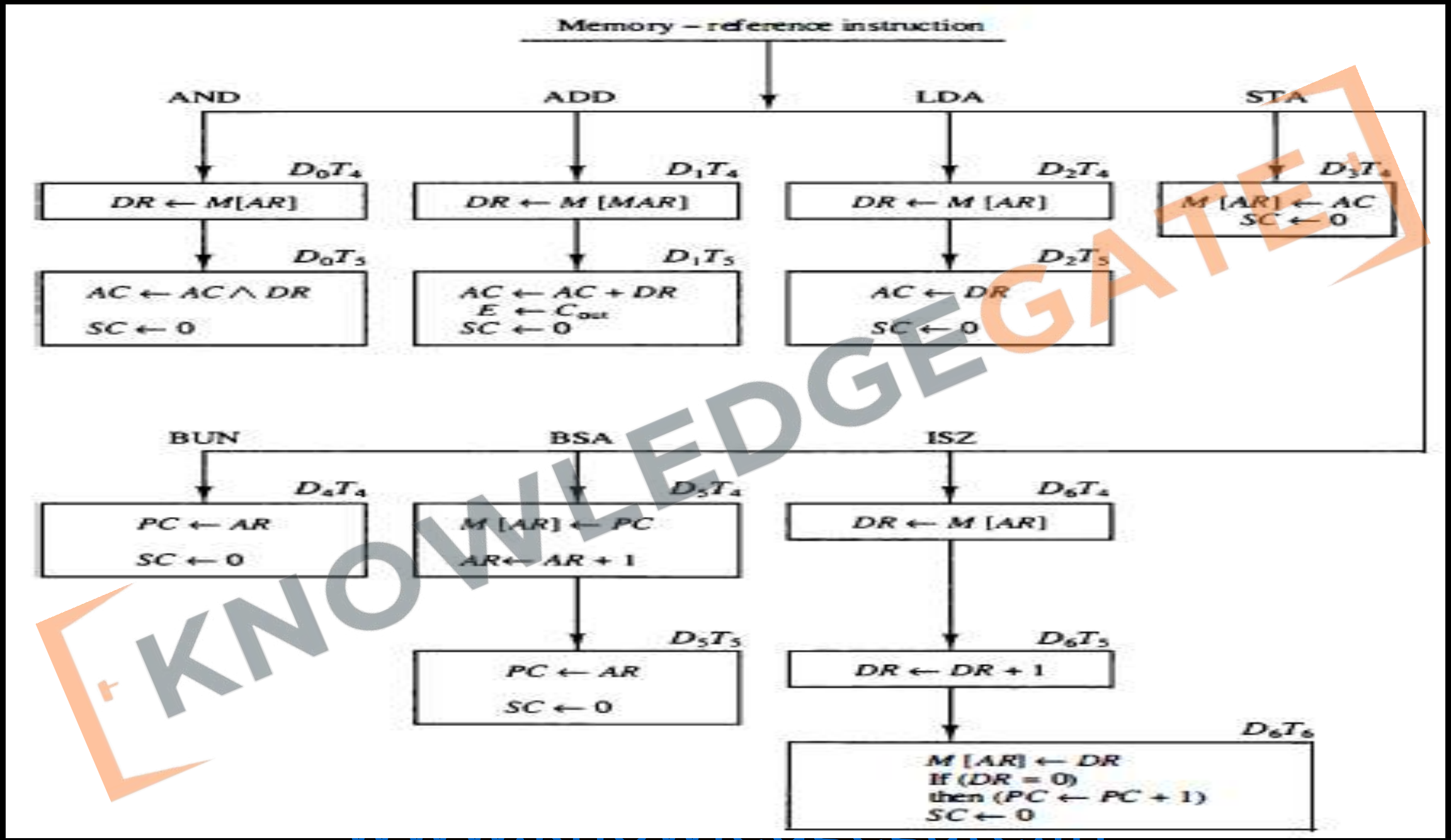
KNOWLEDGEGATE
www.knowledgegate.in

Memory-Reference Instructions

Symbol	Operation decoder	Symbolic description
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR], \quad E \leftarrow C_{out}$
LDA	D_2	$AC \leftarrow M[AR]$
STA	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC, \quad PC \leftarrow AR + 1$
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1,$ If $M[AR] + 1 = 0$ then $PC \leftarrow PC + 1$

- The effective address of the instruction is in the address register AR and was placed there during timing signal T2 when I = 0, or during timing signal T3 when I = 1. The execution of the memory-reference instructions starts with timing signal T4.
- Operation of each Instruction

KNOWLEDGEGATE
www.knowledgegate.in



KNOWLEDGEGATE
www.knowledgegate.in

- **AND to AC** - This is an instruction that performs the AND logic operation on pairs of bits in AC and the memory word specified by the effective address.
- The result of the operation is transferred to AC. The microoperations that execute this instruction are:

$$D_0T_4: \quad DR \leftarrow M[AR]$$
$$D_0T_5: \quad AC \leftarrow AC \wedge DR, \quad SC \leftarrow 0$$

KNOWLEDGEGATE

www.knowledgegate.in

- **ADD to AC** - This instruction adds the content of the memory word specified by the effective address to the value of AC.

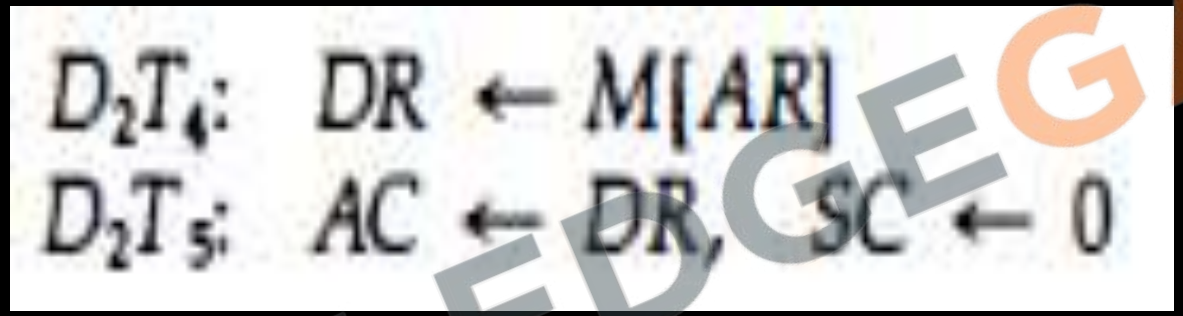
- The sum is transferred into AC and the output carry Cout is transferred to the E (extended accumulator) flip-flop. The microoperations needed to execute this instruction are:

$$D_1T_4: \quad DR \leftarrow M[AR]$$
$$D_1T_5: \quad AC \leftarrow AC + DR, \quad E \leftarrow C_{out}, \quad SC \leftarrow 0$$

KNOWLEDGEGATE

www.knowledgegate.in

LDA: Load to AC - This instruction transfers the memory word specified by the effective address to AC. The microoperations needed to execute this instruction are



KNOWLEDGEGATE

www.knowledgegate.in

STA: Store AC - This instruction stores the content of AC into the memory word specified by the effective address. Since the output of AC is applied to the bus and the data input of memory is connected to the bus, we can execute this instruction with one microoperation:

$$D_3T_4: \quad M[AR] \leftarrow AC, \quad SC \leftarrow 0$$

KNOWLEDGE GATE

www.knowledgegate.in

BUN: Branch Unconditionally - This instruction transfers the program to the instruction specified by the effective address. The BUN instruction allows the programmer to specify an instruction out of sequence and we say that the program branches (or jumps) unconditionally. The instruction is executed with one microoperation:

$$D_4T_4: \quad PC \leftarrow AR, \quad SC \leftarrow 0$$

KNOWLEDGEGATE

www.knowledgegate.in

BSA: Branch and Save Return Address - This instruction is useful for branching to a portion of the program called a subroutine or procedure

$$M[AR] \leftarrow PC, \quad PC \leftarrow AR + 1$$

KNOWLEDGEGATE

www.knowledgegate.in

ISZ: Increment and Skip if Zero - This instruction increments the word specified by the effective address, and if the incremented value is equal to 0, PC is incremented by 1.

D_6T_4: $DR \leftarrow M[AR]$

D_6T_5: $DR \leftarrow DR + 1$

D_6T_6: $M[AR] \leftarrow DR$, if $(DR = 0)$ then $(PC \leftarrow PC + 1)$, $SC \leftarrow 0$

KNOWLEDGEGATE

www.knowledgegate.in

Register-Reference Instructions

- Register-reference instructions are recognized by the control when $D_7 = 1$ and $I = 0$.
- These instructions use bits 0 through 11 of the instruction code to specify one of 12 instructions. These 12 bits are available in IR(0-11).
- These instructions are executed with the clock transition associated with timing variable T_3.
- Each control function needs the Boolean relation $D_7I' T_3$.

$D_7I'T_3 = r$ (common to all register-reference instructions)
$IR(i) = B_i$ [bit in $IR(0–11)$ that specifies the operation]

	r:	$SC \leftarrow 0$	Clear SC
CLA	rB_{11}:	$AC \leftarrow 0$	Clear AC
CLE	rB_{10}:	$E \leftarrow 0$	Clear E
CMA	rB_9:	$AC \leftarrow \overline{AC}$	Complement AC
CME	rB_8:	$E \leftarrow \overline{E}$	Complement E
CIR	rB_7:	$AC \leftarrow$ shr AC, $AC(15) \leftarrow E$, $E \leftarrow AC(0)$	Circulate right
CIL	rB_6:	$AC \leftarrow$ shl AC, $AC(0) \leftarrow E$, $E \leftarrow AC(15)$	Circulate left
INC	rB_5:	$AC \leftarrow AC + 1$	Increment AC
SPA	rB_4:	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$	Skip if positive
SNA	rB_3:	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$	Skip if negative
SZA	rB_2:	If $(AC = 0)$ then $PC \leftarrow PC + 1$	Skip if AC zero
SZE	rB_1:	If $(E = 0)$ then $(PC \leftarrow PC + 1)$	Skip if E zero
HLT	rB_0:	$S \leftarrow 0$ (S is a start–stop flip-flop)	Halt computer

KNOWLEDGEGATE
www.knowledgegate.in

Input-Output Configuration

- The terminal sends and receives serial information. Each quantity of information has eight bits of an alphanumeric code.
- The serial information from the keyboard is shifted into the input register INPR.
- The serial information for the printer is stored in the output register OUTR.
- These two registers communicate with a communication interface serially and with the AC in parallel.

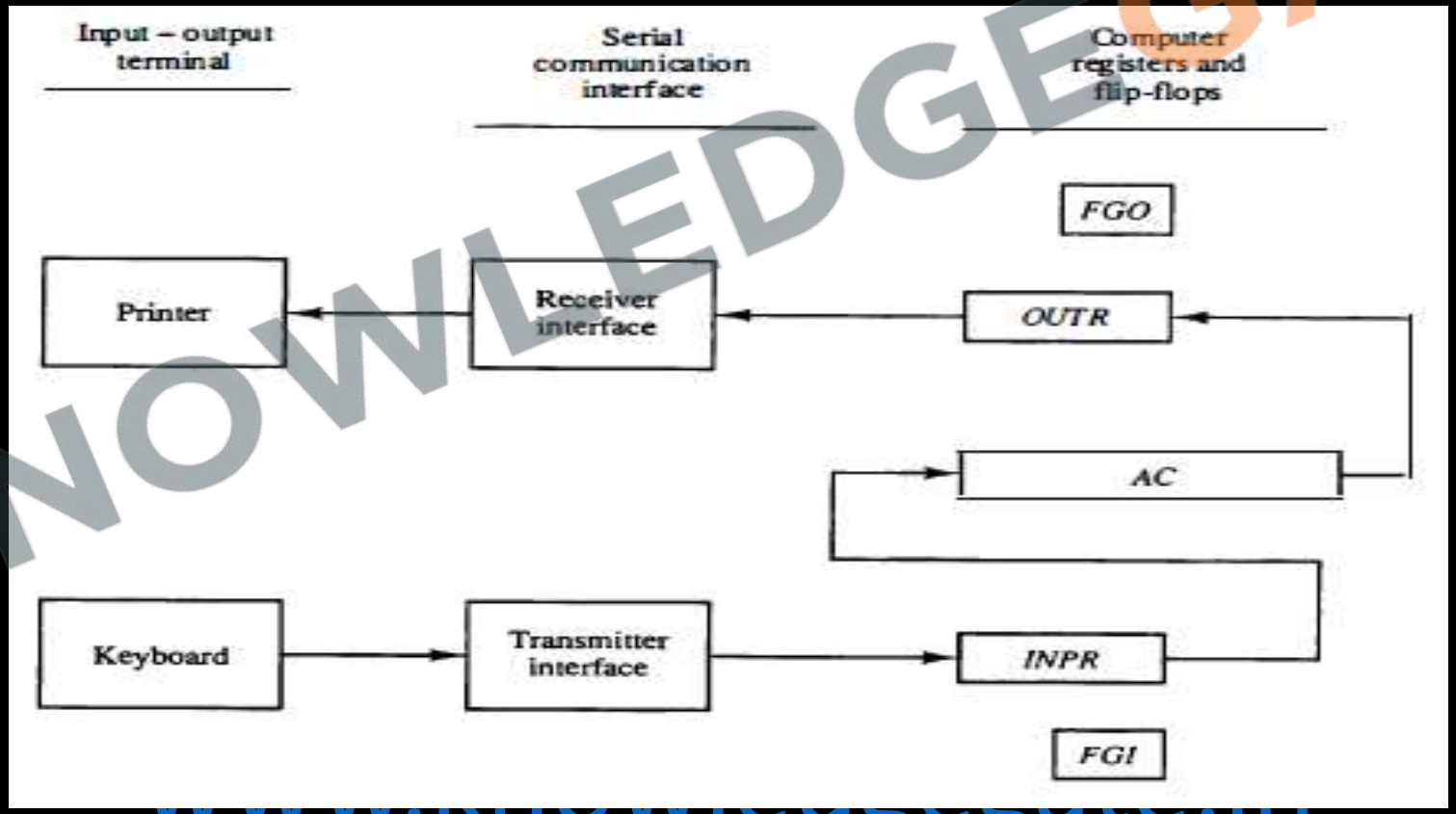
$D_7IT_3 = p$
$IR(i) = B_i$, $i = 6, \ldots, 11$

	p:	$SC \leftarrow 0$	Clear SC
INP	pB_{11}:	$AC(0\text{-}7) \leftarrow INPR, FGI \leftarrow 0$	Input char. to AC
OUT	pB_{10}:	$OUTR \leftarrow AC(0\text{-}7), FGO \leftarrow 0$	Output char. from AC
SKI	pB_9:	$if(FGI = 1)$ then $(PC \leftarrow PC + 1)$	Skip on input flag
SKO	pB_8:	$if(FGO = 1)$ then $(PC \leftarrow PC + 1)$	Skip on output flag
ION	pB_7:	$IEN \leftarrow 1$	Interrupt enable on
IOF	pB_6:	$IEN \leftarrow 0$	Interrupt enable off

KNOWLEDGEGATE
www.knowledgegate.in

Input-Output Configuration

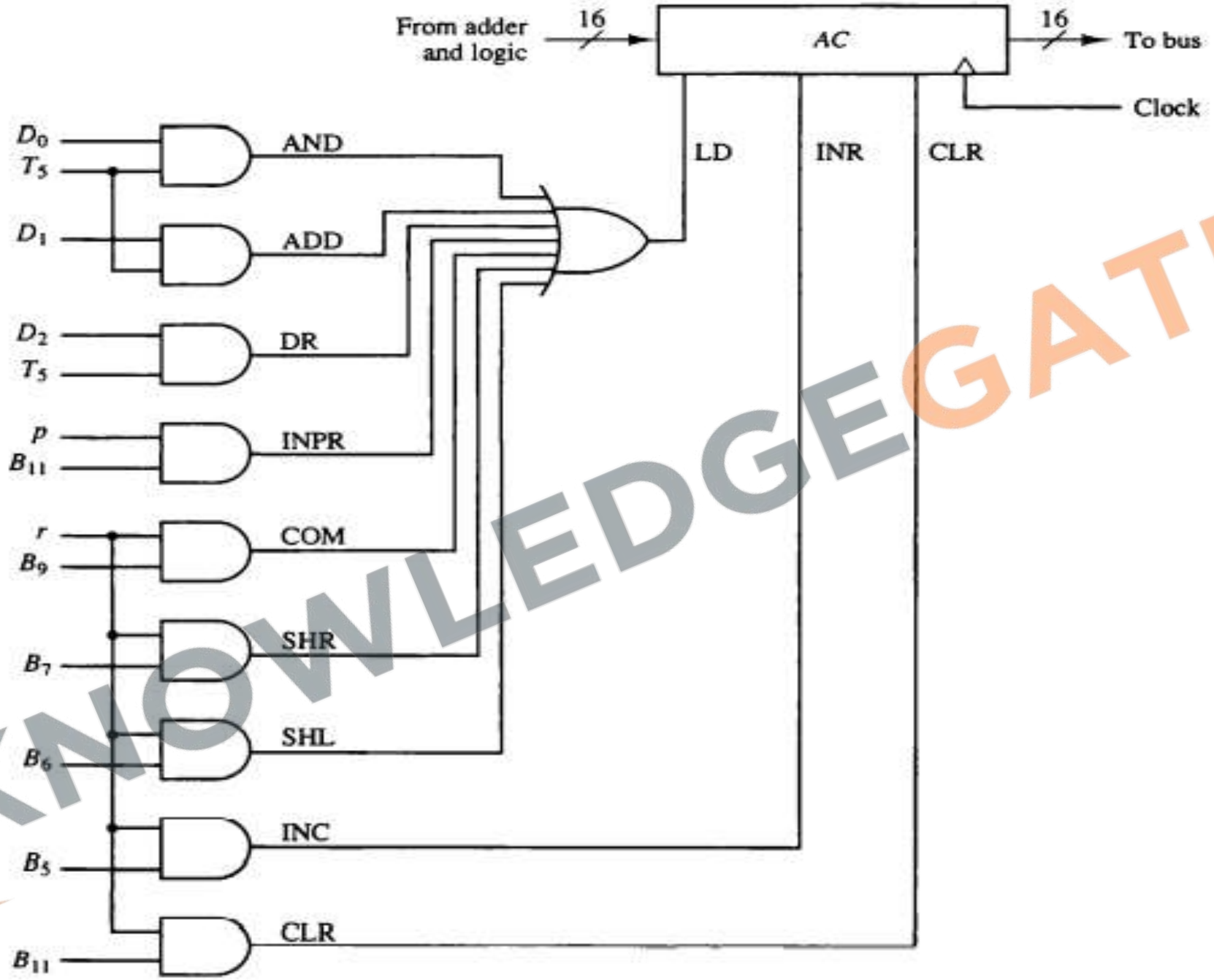
- The transmitter interface receives serial information from the keyboard and transmits it to INPR.
- The receiver interface receives information from OUTR and sends it to the printer serially.
- The 1-bit input flag FGI is a control flip-flop.
- The flag bit is set to 1 when new information is available in the input device and is cleared to 0 when the information is accepted by the computer.
- The flag is needed to synchronize the timing rate difference between the input device and the computer.



KNOWLEDGE GATE
www.knowledgegate.in

Symbol	Hexadecimal code $I = 0$	Hexadecimal code $I = 1$	Description
AND	0xxx	8xxx	AND memory word to *AC*
ADD	1xxx	9xxx	Add memory word to *AC*
LDA	2xxx	Axxx	Load memory word to *AC*
STA	3xxx	Bxxx	Store content of *AC* in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear *AC*
CLE	7400		Clear *E*
CMA	7200		Complement *AC*
CME	7100		Complement *E*
CIR	7080		Circulate right *AC* and *E*
CIL	7040		Circulate left *AC* and *E*
INC	7020		Increment *AC*
SPA	7010		Skip next instruction if *AC* positive
SNA	7008		Skip next instruction if *AC* negative
SZA	7004		Skip next instruction if *AC* zero
SZE	7002		Skip next instruction if *E* is 0
HLT	7001		Halt computer
INP	F800		Input character to *AC*
OUT	F400		Output character from *AC*
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

KNOWLEDGEGATE



KNOWLEDGEGATE

Reduced Instruction Set Computer (RISC)

- Computers that use fewer instructions with simple constructs so they can be executed much faster within the CPU without having to use memory as often are classified as RISC.
- Relatively few instructions and few addressing modes
- Memory access limited to load and store instructions as all operations done within the registers of the CPU
- Fixed-length, easily decoded instruction format, single-cycle instruction execution.
- Hardwired rather than microprogrammed control.
- The small set of instructions of a typical RISC processor consists mostly of register-to-register operations, Thus, each operand is brought into a processor register with a load instruction. All computations are done among the data stored in processor registers. Results are transferred to memory by means of store instructions
- A RISC computer has a small set of simple and general instructions, rather than a large set of complex and specialized ones.

KNOWLEDGEGATE

- Another success from this era was IBM's effort that eventually led to the IBM POWER instruction set architecture, PowerPC, and Power ISA. As these projects matured, a variety of similar designs flourished in the late 1980s and especially the early 1990s, representing a major force in the Unix workstation market as well as for embedded processors in laser printers, routers and similar products.
- The many varieties of RISC designs include ARC, Alpha, Am29000, ARM, Atmel AVR, Blackfin, i860, i960, M88000, MIPS, PA-RISC, Power ISA (including PowerPC), RISC-V, SuperH, and SPARC.
- The use of ARM architecture processors in smartphones and tablet computers such as the iPad and Android devices provided a wide user base for RISC-based systems. RISC processors are also used in supercomputers such as Summit, which, as of January 2020, is the world's fastest supercomputer as ranked by the TOP500 project.

Capable of 200 petaFLOPS

www.knowledgegate.in

Complex Instruction Set Computer (CISC)

- A computer with a large number of instructions is classified as a complex instruction set computer, abbreviated CISC.
- A large number of instructions-typically from 100 to 250 instructions
- Some instructions that perform specialized tasks and are used infrequently
- A large variety of addressing modes-typically from 5 to 20 different modes
- Variable-length instruction formats
- Instructions that manipulate operands in memory

KNOWLEDGEGATE

- A **complex instruction set computer** (**CISC**) is a computer in which single instructions can execute several low-level operations (such as a load from memory, an arithmetic operation, and a memory store) or are capable of multi-step operations or addressing modes within single instructions.

- The term was retroactively coined in contrast to reduced instruction set computer (RISC) and has therefore become something of an umbrella term for everything that is not RISC, from large and complex mainframe computers to simplistic microcontrollers where memory load and store operations are not separated from arithmetic instructions.

- A modern RISC processor can therefore be much more complex than, say, a modern microcontroller using a CISC-labeled instruction set, especially in the complexity of its electronic circuits, but also in the number of instructions or the complexity of their encoding patterns.

- The only typical differentiating characteristic is that most RISC designs use uniform instruction length for almost all instructions, and employ strictly separate load/store-instructions.

KNOWLEDGEGATE

www.knowledgegate.in

- Examples of instruction set architectures that have been retroactively labeled CISC are System/360 through z/Architecture, the PDP-11 and VAX architectures, Data General Nova and many others.
- Well known microprocessors and microcontrollers that have also been labeled CISC in many academic publications include the Motorola 6800, 6809 and 68000-families; the Intel 8080, iAPX432 and x86-family; the Zilog Z80, Z8 and Z8000-families; the National Semiconductor 32016 and NS320xx-line; the MOS Technology 6502-family; the Intel 8051-family; and others.
- Some designs have been regarded as borderline cases by some writers. For instance, the Microchip Technology PIC has been labeled RISC in some circles and CISC in others. The 6502 and 6809 have both been described as "RISC-like", although they have complex addressing modes as well as arithmetic instructions that operate on memory, contrary to the RISC-principles.

KNOWLEDGE GATE

www.knowledgegate.in

Complex instruction set format (CISC)	Reduced instruction set format (RISC)
Large number of instructions (around 1000) Application point of view they are useful but for system designer it is a headache	Relatively Few numbers of instruction, only those instructions which are strictly required, and in case of implementation of a complex function a set of simple instruction will perform together
Some instruction is there which perform specific task and are used infrequently for e.g. instructions for testing	Few instructions will be there which will be preforming more frequent work
Large number of addressing mode, leading to lengthen instruction, but compilation and translation will be faster	Number of instruction mode will be less, hardly 3 to 4
Variable length instruction format	Fixed length easy to decode instruction format
Instruction that manipulate operand in memory	Do not perform operation directly in memory, first load them in register and then perform, so all operations will be done with in the registers of CPU
Powerful but costly	Relatively less powerful but cheap
Microprogrammed control unit, relatively slow	Hardwired control unit, so fast

KNOWLEDGEGATE
www.knowledgegate.in

Designing of Control Unit

- The Control signal can be generated either by hardware (hardware Control unit design) or by (memory + h/w) (micro-program control unit design)
- In the hardware control unit design, each control signal is expressed as SOP expression and realized by digital hardware.
- The sequence of the operation carried out by this machine is determined by the wiring of the logic elements and hence named as “hardwired”.
- Fixed logic circuits that correspond directly to the Boolean expressions are used to generate the control signals.

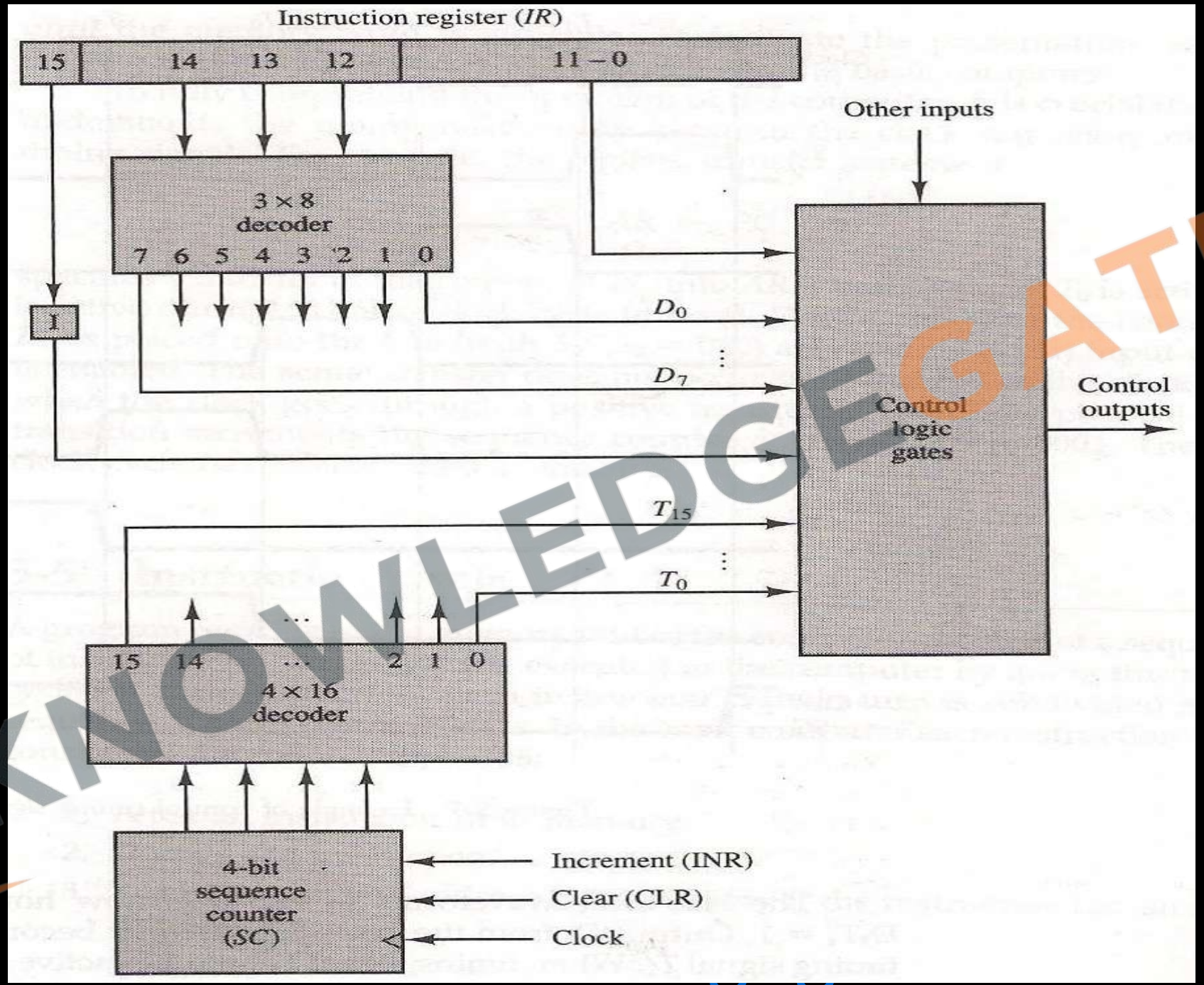
KNOWLEDGEGATE

Q Consider a hypothetical control unit implemented by hardware it uses three, 8-bits register A, B, C. It supports the two instruction I_1 and I_2, the following table gives control signals required for each micro-operation for both instructions?

Micro-operation	Control Signal	
	I_1	I_2
T_1	A_{in}, B_{out}	A_{in}, A_{out}
T_2	B_{in}, C_{out}	C_{in}, A_{out}
T_3	B_{in}, B_{out}	C_{in}, C_{out}
T_4	A_{in}, A_{out}	A_{in}, B_{out}

KNOWLEDGEGATE

www.knowledgegate.in


Instruction register (IR)
15
14
13
12
11 – 0
3 × 8
decoder
7 6 5 4 3 2 1 0
1
Other inputs
D_0
D_7
Control
logic
gates
Control
outputs
T_{15}
T_0
15 14 ... 2 1 0
4 × 16
decoder
4-bit
sequence
counter
(SC)
Increment (INR)
Clear (CLR)
Clock

Conclusion

- It is the fastest control unit design and it is suitable for real time application. (The RISC machine employ hardware control unit), Hardwired control is faster than micro-programmed control.
- A controller that uses this approach can operate at high speed. RISC architecture is based on hardwired control unit.
- Limitation: any modification to the design require re-design & re-connection of H/W component. it is not flexible; hence hardware control unit is not suitable for design & de-bugging environment. This problem is resolved using Micro-programmed control unit design.

KNOWLEDGEGATE

Micro-Programmed Control Unit

- The idea of microprogramming was introduced by Maurice Wilkes in 1951 as an intermediate level to execute computer program instructions.
- Microprograms were organized as a sequence of *microinstructions* and stored in special control memory.
- The main advantage of the microprogram control unit is the simplicity of its structure. Outputs of the controller are organized in microinstructions and they can be easily replaced.
- In this, the binary patterns of control signals are stored in control memory. After accessing word from the control memory, hardware is used to generate the control signals.

KNOWLEDGEGATE

- Any changes can affects only the control word but not the hardware
- The design is flexible & it is used in CISC system.
- Based on the no of words in the control word, the micro-programming may be
  - Horizontal micro-Program
  - Vertical Micro-program

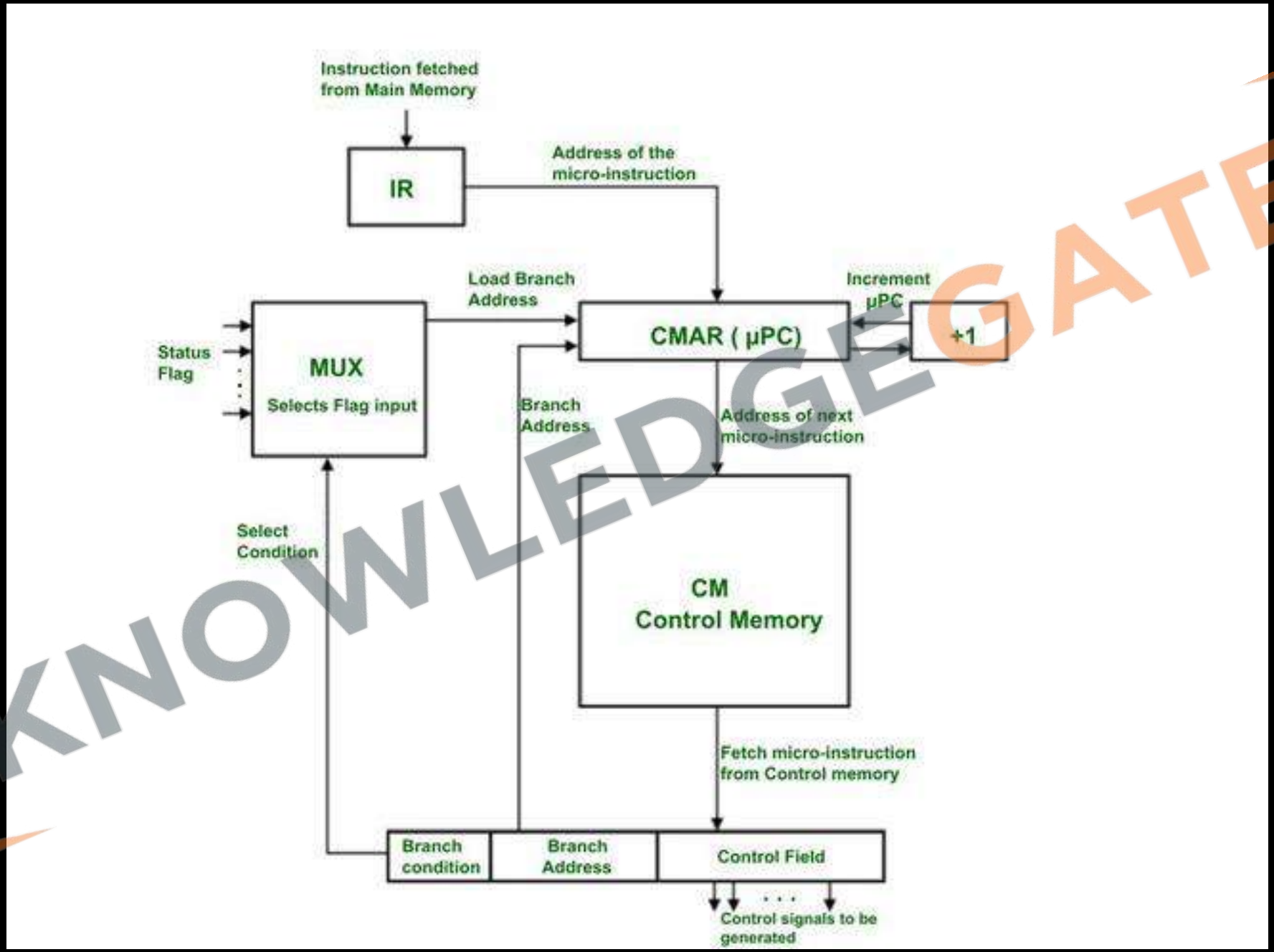
KNOWLEDGEGATE

www.knowledgegate.in

Characteristics	Hardwired	Micro-programmed Control
Speed	Fast	Slow
Implementation	Hardware	Software
Flexibility	Not Flexible	Flexible
Ability to handle complex instruction set	Diffcult	Easier
Design process	Diffcult for more opertion	Easy
Memory	Not Used	Control memory used
Chip are efficiency	Uses less area	Uses more area
Used in	RISC	CISC

KNOWLEDGEGATE
www.knowledgegate.in

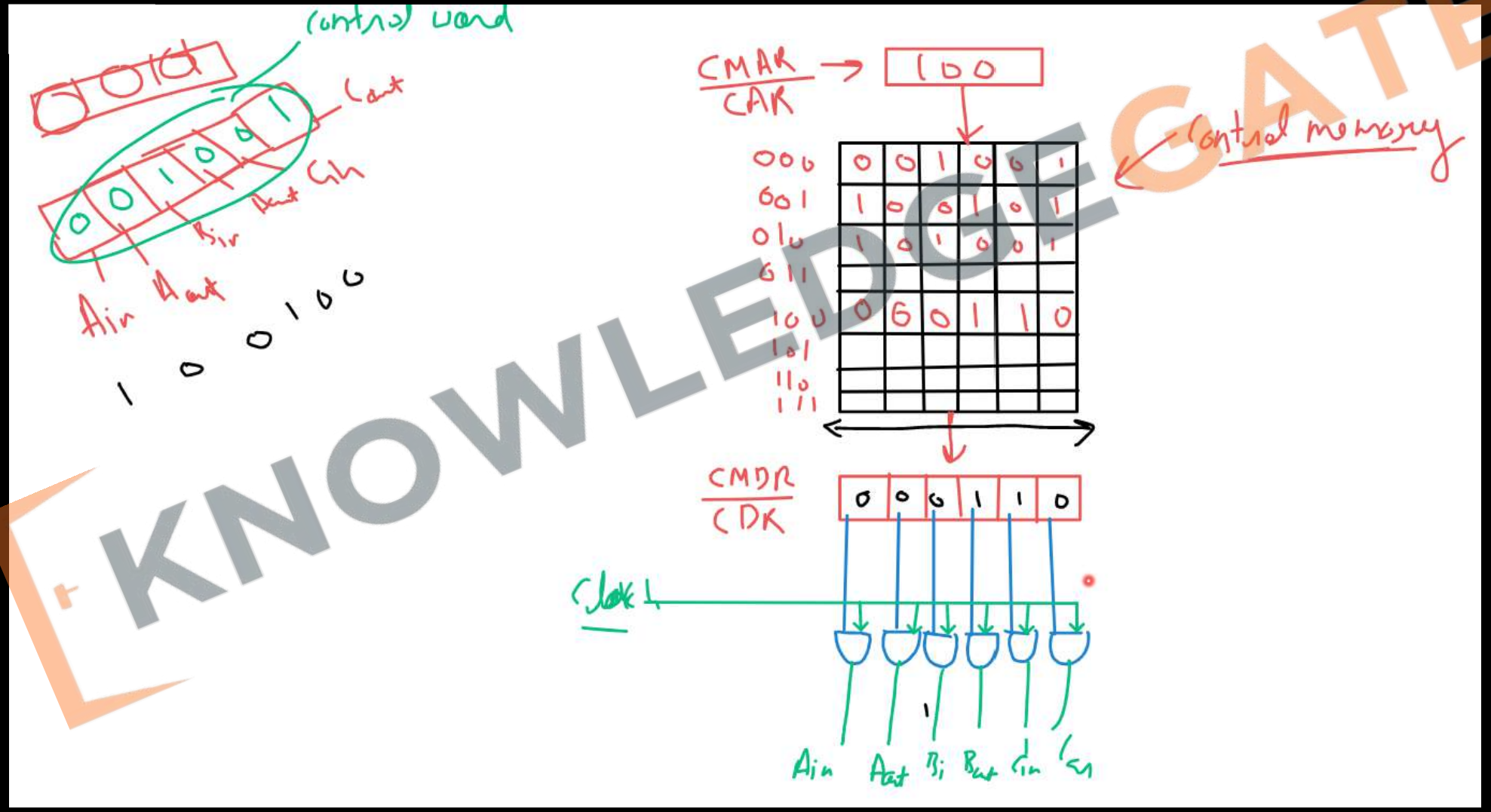
Horizontal Micro-Programmed Control Unit



KNOWLEDGEGATE

Horizontal micro-Program

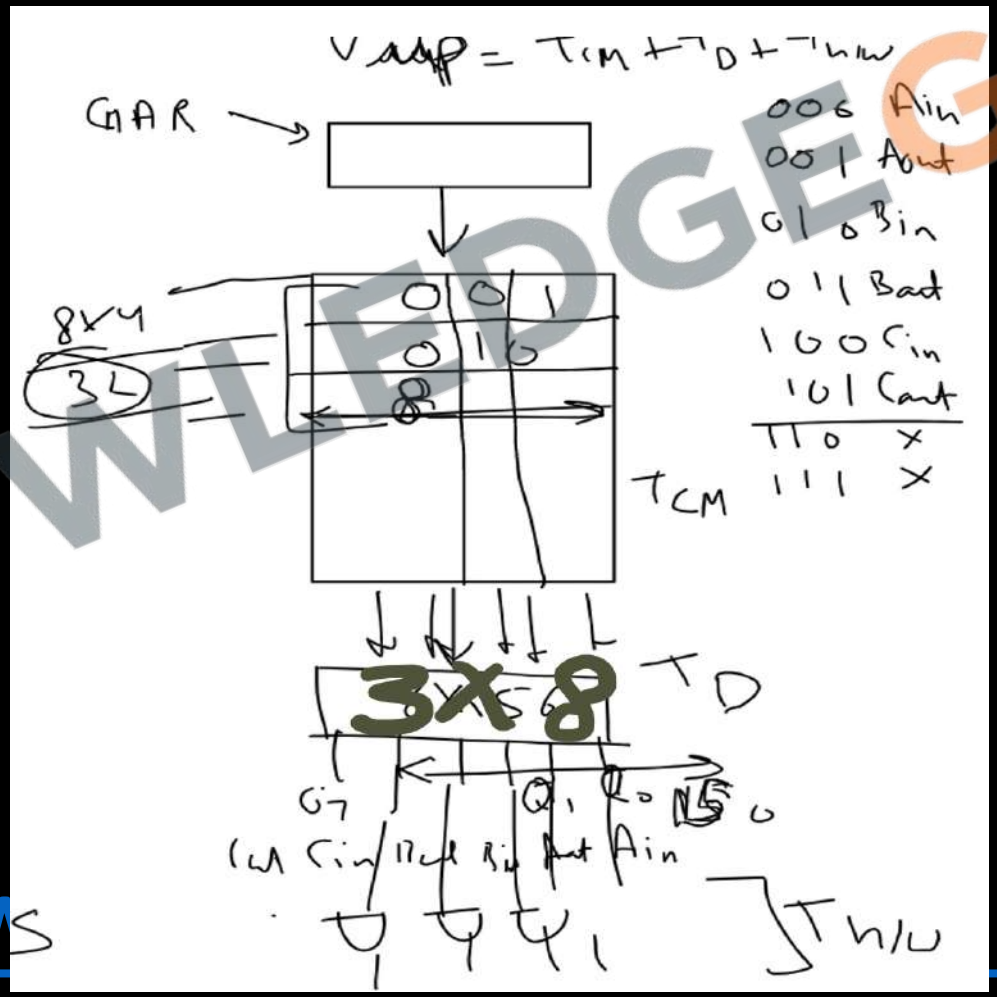
- The horizontal micro-programmed provides higher degree of parallelism & it is suitable in multi-processor system. it requires more bits for control word (1 it for every control signal)



KNOWLEDGE GATE

Vertical Micro-program

- The vertical micro-programming reduces the size of control words by encoding, control signal pattern before it is stored in control memory. it offers more flexibility than horizontal micro-programming.
- The pattern with vertical-programming is the maximum degree of parallelism is 1(due to the decoder).



KNOWLEDGE GATE

ww in

Horizontal	Vertical
Long Format	Short Format
Ability to express a high degree of parallelism	Limited ability to express parallel micro-operations
Little encoding of control information	Considerable encoding of the control information
Usefull when higher operating speed is desired	Slower operating speed

KNOWLEDGEGATE
www.knowledgegate.in

Conclusion

- The combinational Horizontal micro-Program and Vertical Micro-program controlled unit is preferred in most application.

KNOWLEDGEGATE

www.knowledgegate.in

Control word sequencing

- In order to implement, the micro-program, control words are to be sequentially from control memory. One address instruction is used for performing the control word sequencing.

Flag	Control Signal	Address

KNOWLEDGEGATE

Memory in Computer

- Memory is one of the most crucial components of a computer. Some reasons why memory plays a vital role in computer's overall performance :
  - **Storage of data**: Memory allows the processor to access the data quickly.
  - **Multitasking**: Memory enables multiple tasks simultaneously, as it allows the processor to switch between different programs and data efficiently.
  - **Running applications**: Applications and programs require a certain amount of memory to run.
  - **Operating system**: The operating system also requires memory to run smoothly.
- In summary, memory is critical for the efficient operation of a computer and plays a key role in determining its performance.

KNOWLEDGEGATE

Importance of Memory

- The criteria for a ideal memory in a computer system depend on the specific requirements of the system, but there are several general factors that are important for any type of memory:
  - Speed:
  - Capacity:
  - Stability:
  - Cost:
  - Scalability

KNOWLEDGEGATE

www.knowledgegate.in

Cycle

Car

Airbus

KNOWLEDGEGATE

www.knowledgegate.in

Lathi

303

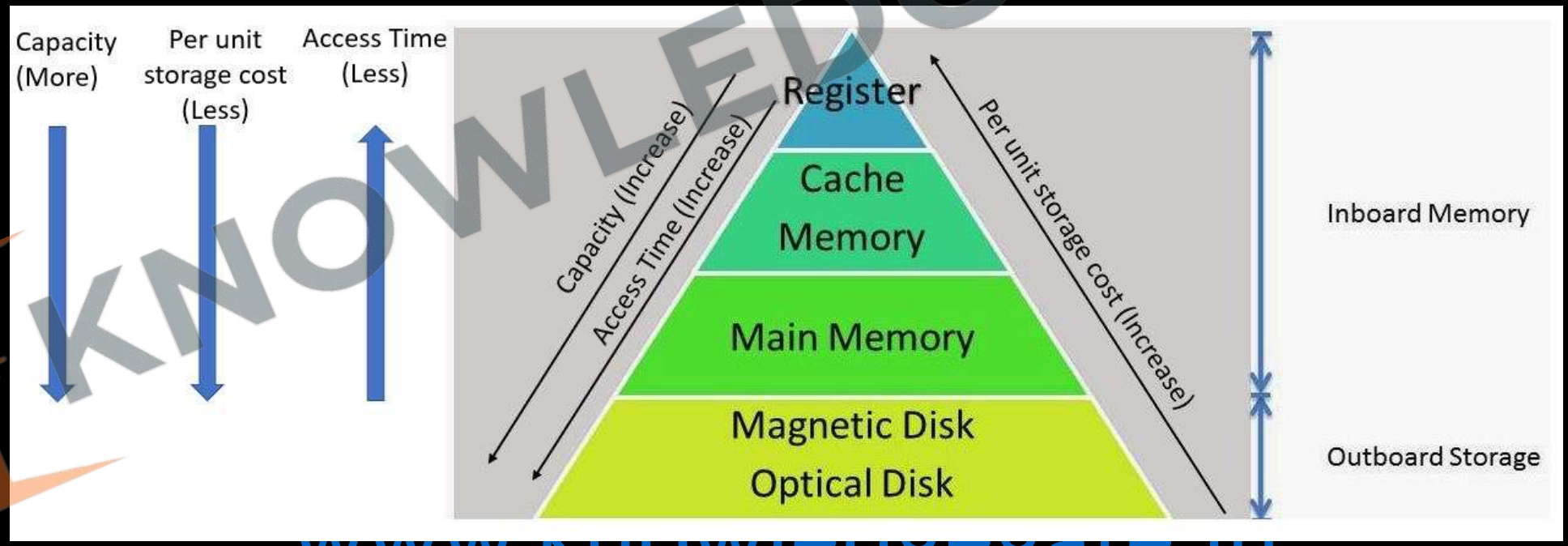
AK-47

KNOWLEDGEGATE

सत्यमेव जयते

www.knowledgegate.in

- The memory hierarchy in a computer system refers to different levels of memory storage with varying capacity, speed, and cost. The memory hierarchy is designed to provide the CPU with fast access to data and a large storage area for longer-term data. The levels of the memory hierarchy, in order of speed, are:
  - **Registers**: Fastest and smallest memory built into the CPU.
  - **Cache**: Small, fast memory usually built into the CPU and used for frequently accessed data.
  - **Main Memory (RAM)**: Primary storage for data and instructions, faster than storage devices.
  - **Secondary Memory (Storage Devices):** Used for permanent storage, slower than main memory but with larger capacity.



KNOWLEDGEGATE
www.knowledgegate.in

How CPU Uses Memory Hierarchy

Cache

Showroom

Main Memory

Go down

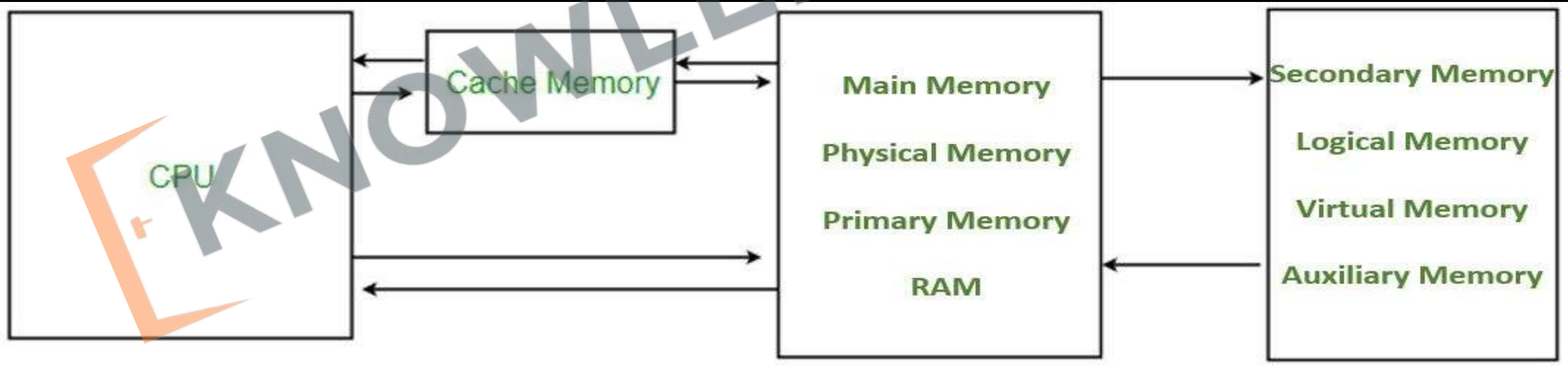
Secondary Memory

Factory

KNOWLEDGEGATE

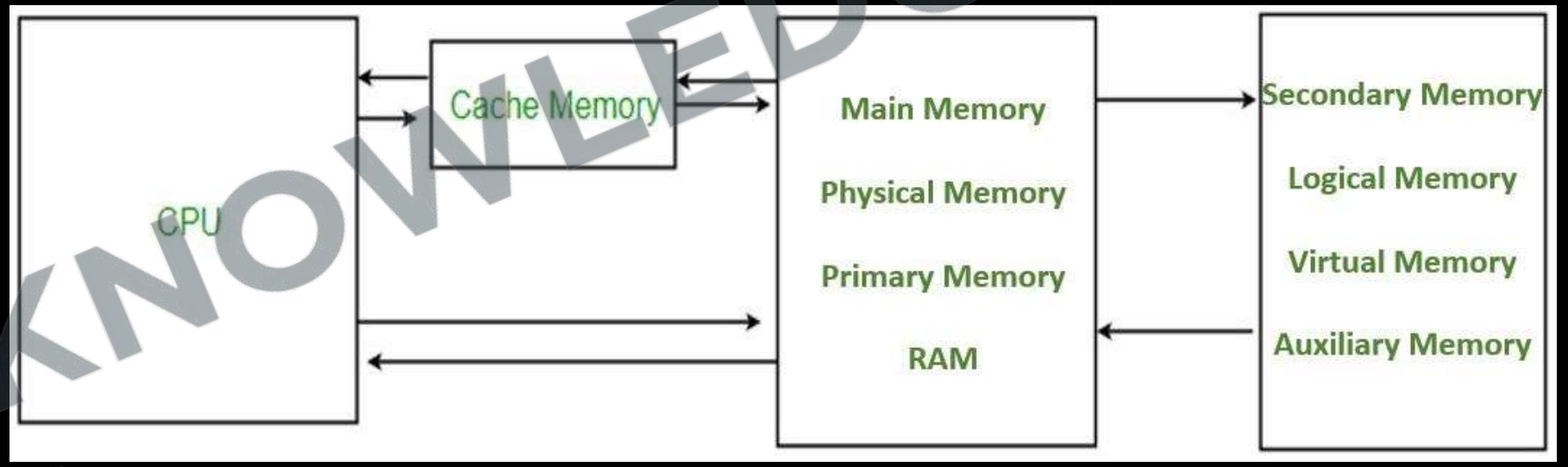
How CPU Uses Memory Hierarchy

- Cache memory boosts processing speed by swiftly providing data to the CPU. It bridges the speed gap between the processor and main memory.
- Main memory holds data for immediate access by the CPU. If data isn't in the main memory, it's fetched from the auxiliary or secondary memory.
- For efficient processing, data sought by the CPU should ideally be in Cache. If not, it's fetched from main memory or, as a last resort, secondary memory.



KNOWLEDGEGATE
www.knowledgegate.in

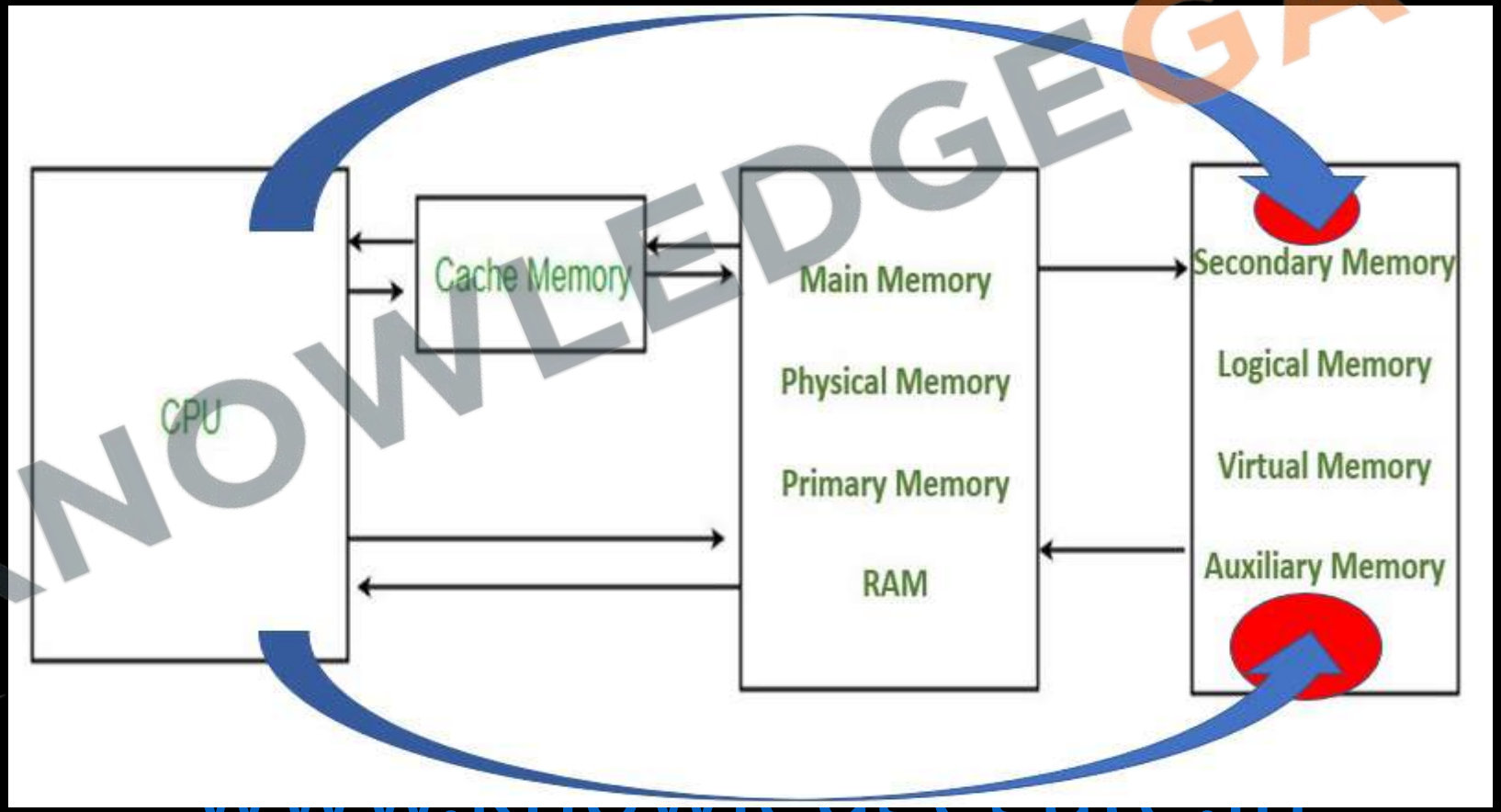
- But this is a difficult task to do on your computer as it has, 1 TB of Secondary Memory, 16 GB of Main Memory, but only 768KB, 4MB, 16 MB of L_1, L_2, L_3 cache respectively.
- If somehow we can estimate what data CPU will require if future we can prefetch in Cache and Main Memory. Locality of reference Helps we to perform this estimation.



KNOWLEDGEGATE
www.knowledgegate.in

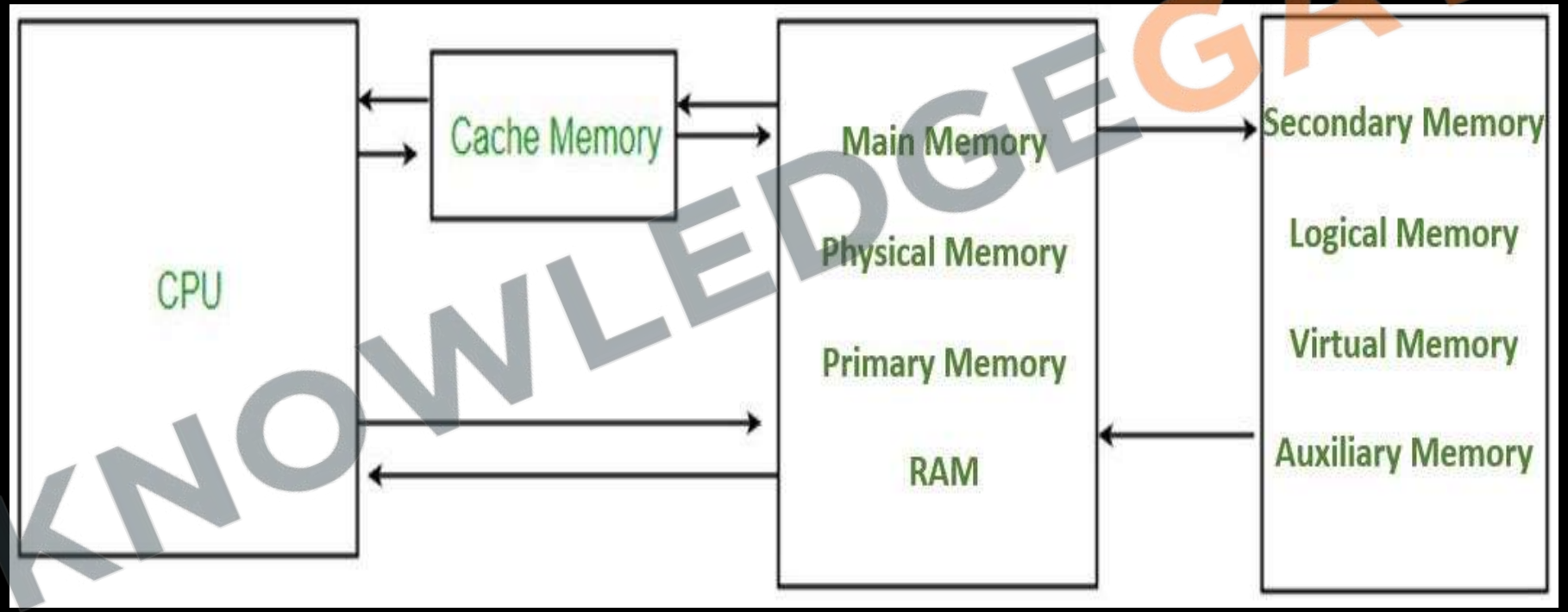
Locality of Reference

- Locality of reference refers to the tendency for a program to access a small portion of its memory at any given time, while the vast majority of its memory remains unused. There are two types of locality of reference: temporal locality and spatial locality.



KNOWLEDGEGATE
www.knowledgegate.in

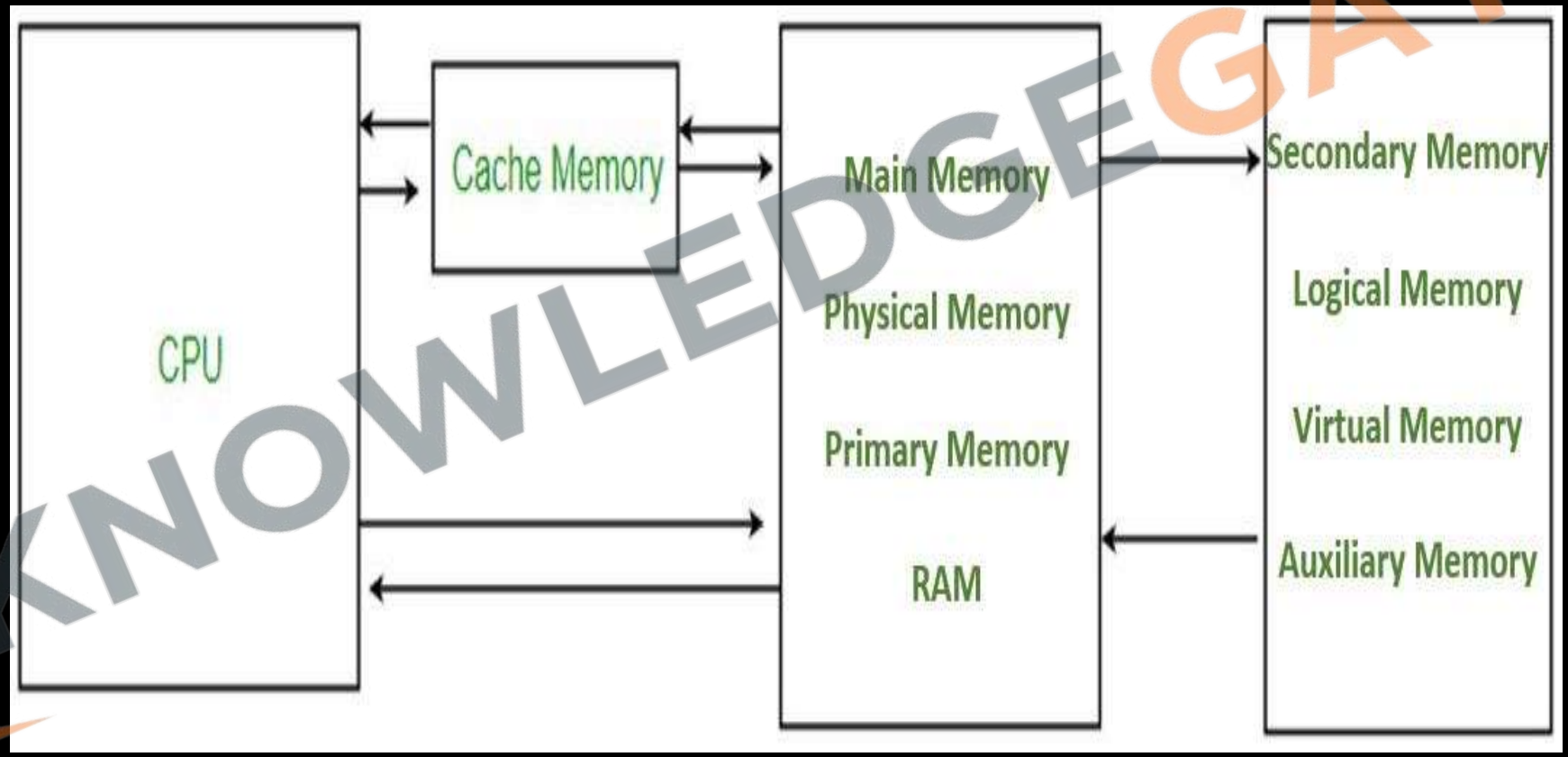
- **Spatial Locality:** Refers to the tendency for a program to access memory locations that are close to each other in memory. This means that if a program accesses a memory location, it is likely to access other nearby memory locations soon after.



KNOWLEDGEGATE

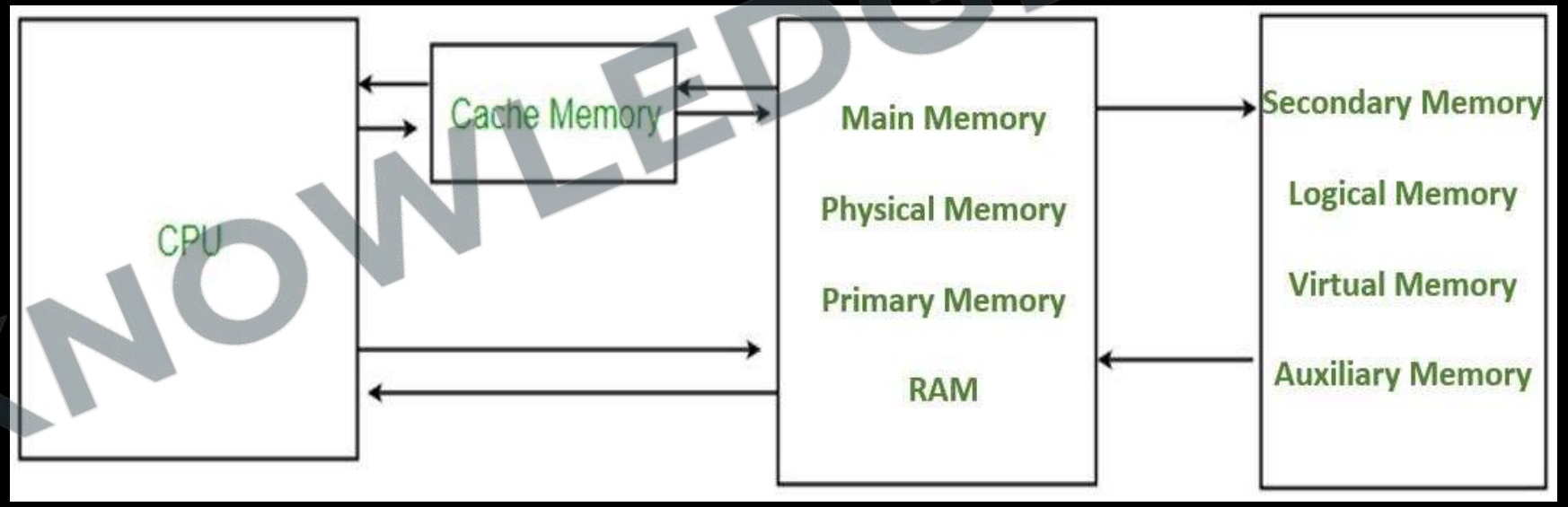
www.knowledgegate.in

- **Temporal Locality:** Temporal locality refers to the tendency for a program to access recently used memory locations repeatedly. This means that if a program accesses a memory location, it is likely to access that same location again in the near future.



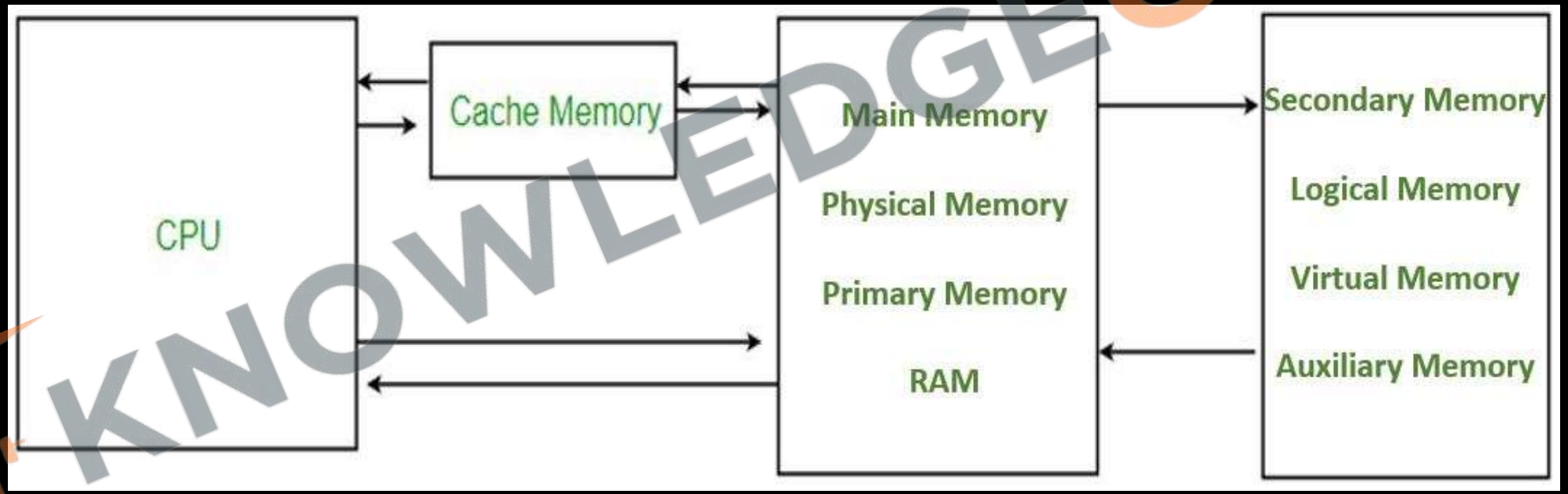
KNOWLEDGEGATE
www.knowledgegate.in

- **Cache Hit:** When a program requests data, the cache is searched first, and if the data is found in the cache, it is returned to the program. This is known as a cache hit.
- **Hit Ratio:** The cache hit rate is the ratio of data access requests that are being satisfied by the cache.
- **Hit Latency:** Hit latency refers to the time it takes for a computer system to retrieve data from the cache memory when a cache hit occurs.

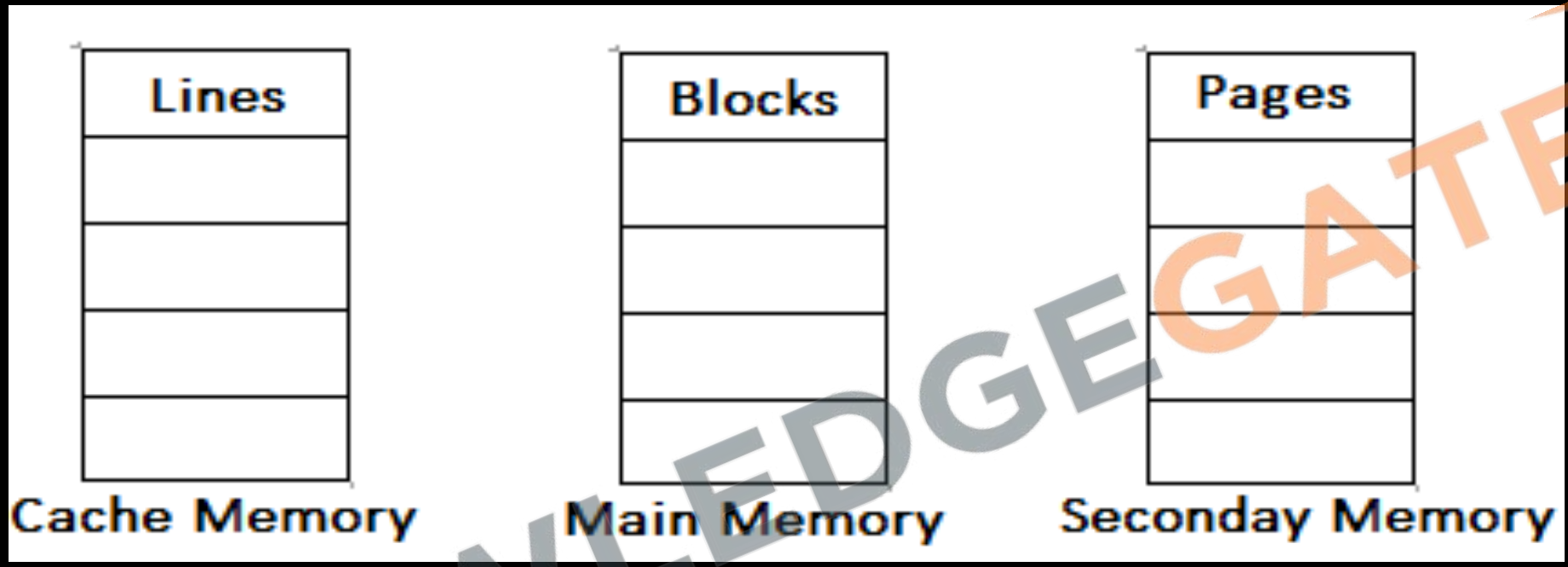


KNOWLEDGEGATE
www.knowledgegate.in

- **Cache Miss:** A cache miss occurs when a computer system tries to retrieve data from the cache memory, but the data is not found in the cache.
- **Miss Latency:** Cache miss latency refers to the time it takes for a computer system to retrieve data from the main memory or disk storage when a cache miss occurs.



KNOWLEDGEGATE



KNOWLEDGEGATE

www.knowledgegate.in

10^3	1 Thousand
10^6	1 Million
10^9	1 Billion
10^{12}	1 Trillion

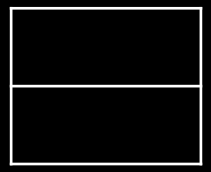
10^3	1 kilo
10^6	1 Mega
10^9	1 Giga
10^{12}	1 Tera
10^{15}	1 Peta

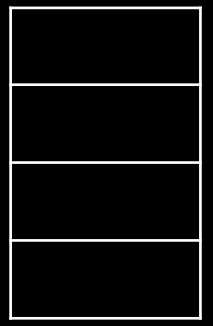
2^{10}	1 kilo
2^{20}	1 Mega
2^{30}	1 Giga
2^{40}	1 Tera
2^{50}	1 Peta

KNOWLEDGEGATE

www.knowledgegate.in

Address Length in bits	n
No of Locations	2^n

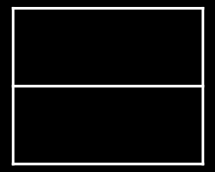


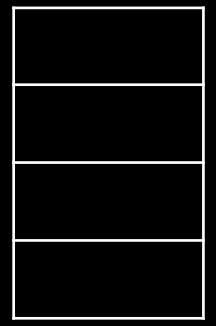


KNOWLEDGEGATE

www.knowledgegate.in

Address Length in bits	n
No of Locations	2^n



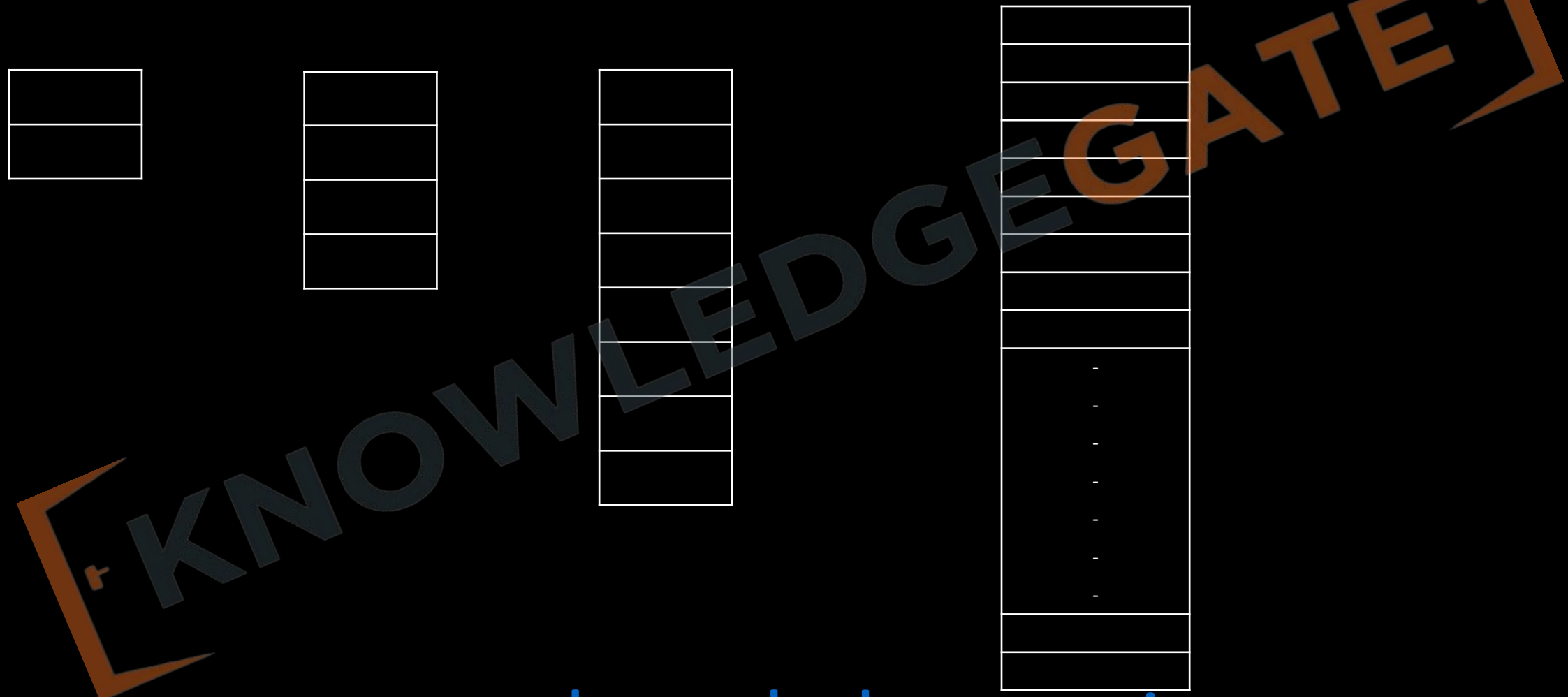


KNOWLEDGEGATE

Memory Size = Number of Location * Size of each Location

www.knowledgegate.in

Address Length in bits	Upper Bound($Log_2 n$)
No of Locations	n



KNOWLEDGEGATE

www.knowledgegate.in

Semiconductor RAM

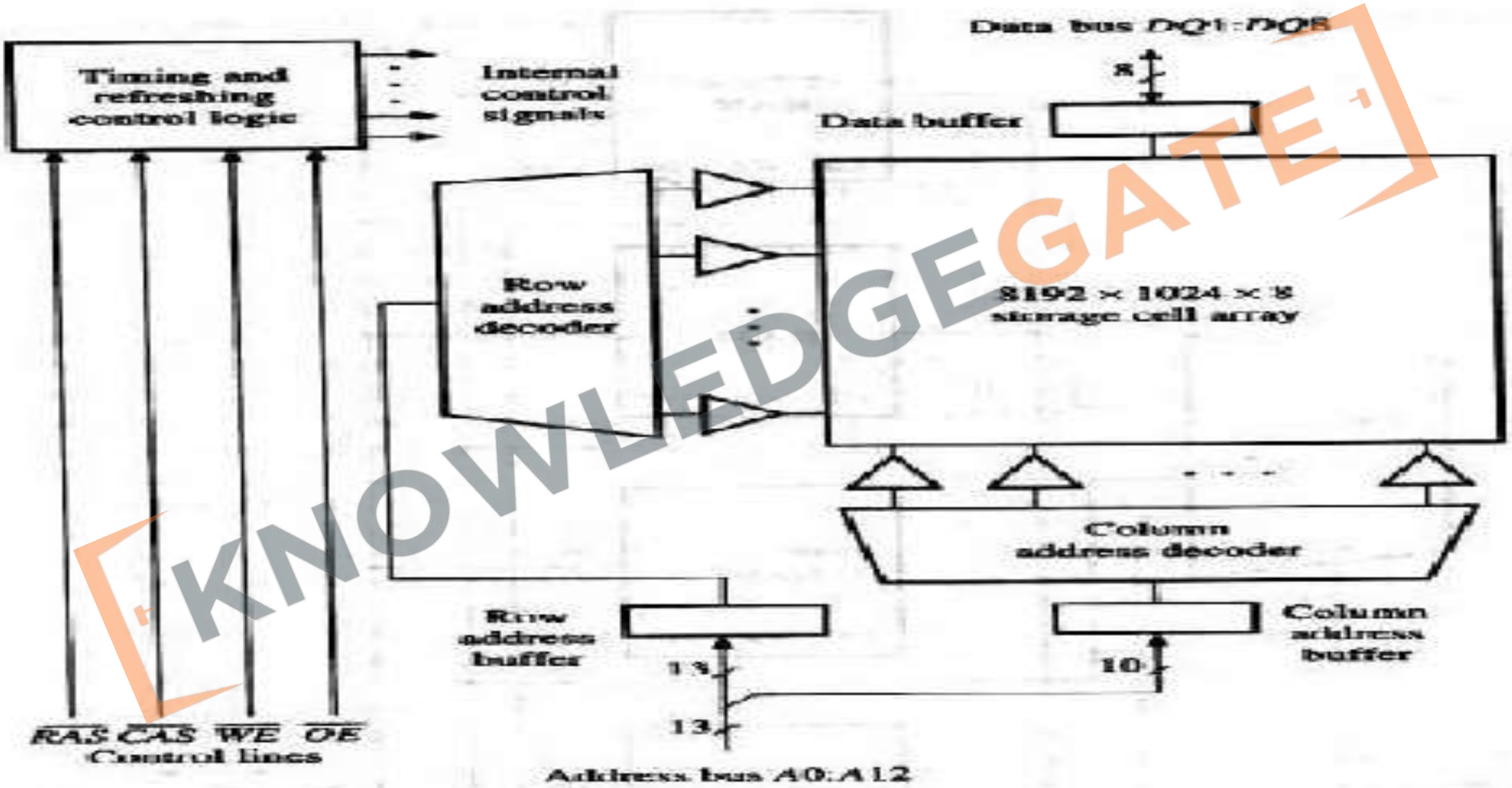
- Semiconductor RAM (Random Access Memory) is a type of computer memory that allows data to be read and written in almost the same amount of time irrespective of the physical location of data inside the memory. It uses semiconductor technology to store data.

KNOWLEDGEGATE

Feature	Dynamic RAM (DRAM)	Static RAM (SRAM)
Storage	Uses capacitors	Uses bistable latching
Speed	Slower	Faster
Refresh	Requires frequent refresh	No refresh needed
Power	Lower power consumption	Higher power consumption
Cost	Cheaper	More expensive
Usage	Main system memory	Cache memory

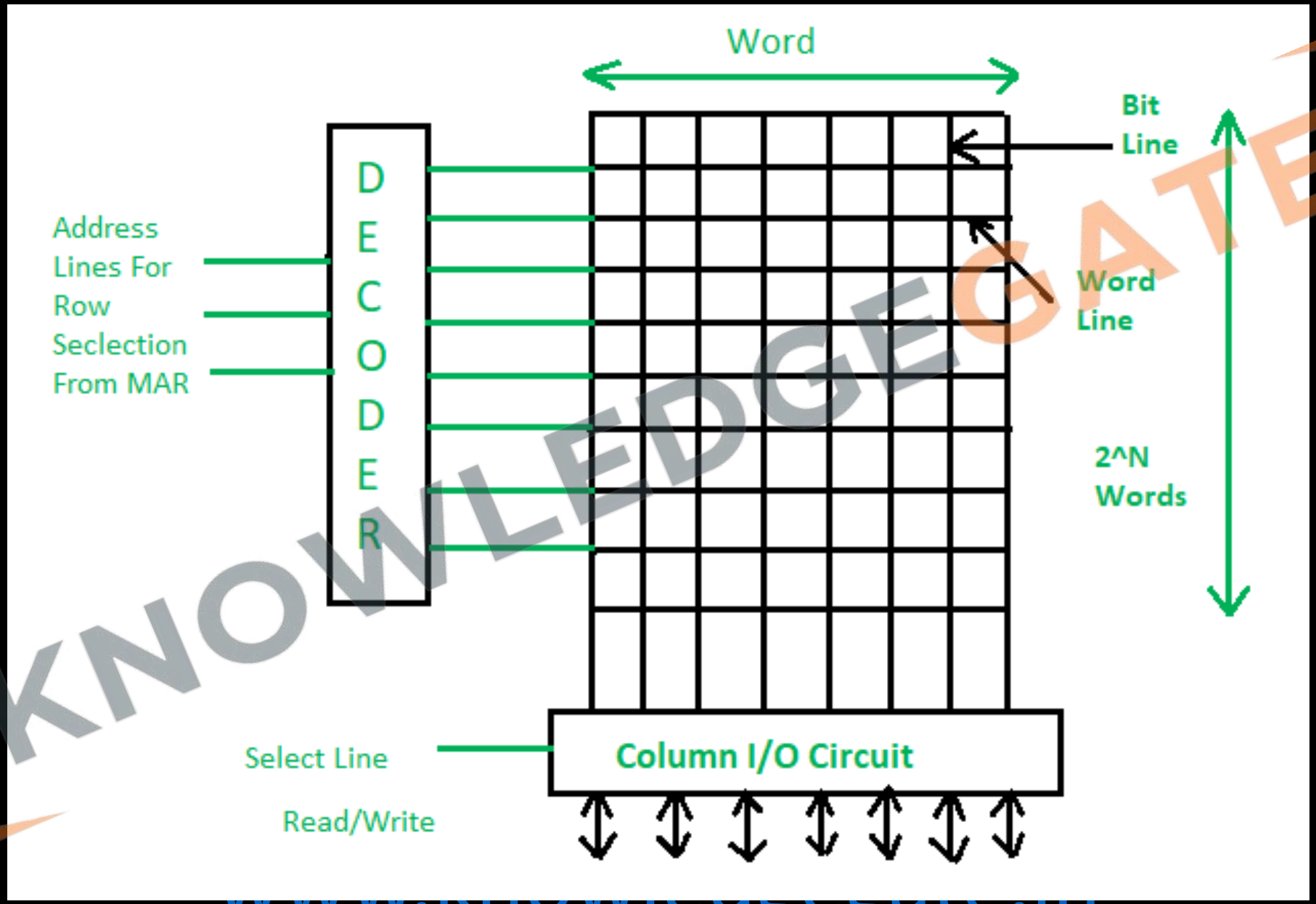
KNOWLEDGEGATE
www.knowledgegate.in

Commercial DRAM Chip



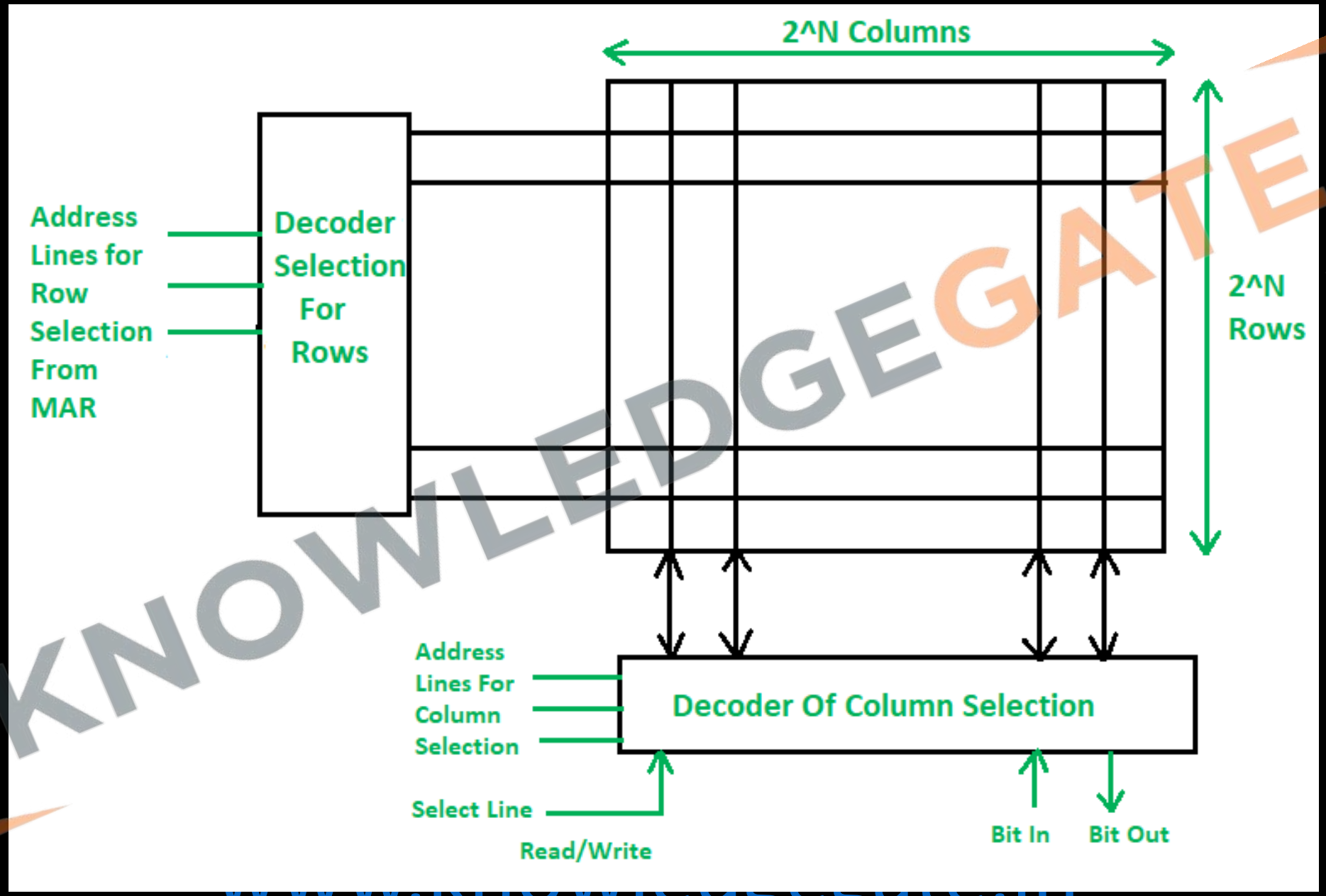
KNOWLEDGEGATE

2D Organization



KNOWLEDGEGATE
www.knowledgegate.in

2.5 D Organization



KNOWLEDGEGATE
www.knowledgegate.in

ROM (Read-Only Memory)

- Non-volatile memory, retains data even when power is off.
- Used primarily for firmware, which contains the basic instructions for hardware.
- Data written during the manufacturing process; traditionally, it cannot be modified (hence "read-only").

KNOWLEDGEGATE

www.knowledgegate.in

Types of ROM:

1.MROM (Masked ROM):

1. Programmed during the manufacturing process.
2. Cannot be reprogrammed by the user.
3. Cheapest type of ROM.

2.PROM (Programmable Read-Only Memory):

1. Can be programmed by the user only once.
2. Uses a device called a PROM programmer or PROM burner.
3. Once programmed, data cannot be erased.

3.EPROM (Erasable Programmable Read-Only Memory):

1. Can be erased and reprogrammed multiple times.
2. Erased using UV light, hence often has a transparent window on the chip.
3. Takes more time to erase and reprogram compared to EEPROM.

4.EEPROM (Electrically Erasable Programmable Read-Only Memory):

1. Can be erased and reprogrammed using electrical signals.
2. Erasure can be done byte by byte, not all at once, allowing for selective editing.
3. Used in scenarios where data needs frequent updates, like in BIOS chips.

5.Flash Memory (a type of EEPROM):

1. Can erase and reprogram blocks of data, not just byte-by-byte.
2. Faster than regular EEPROM.
3. Commonly used for USB drives, memory cards, and Solid-State Drives (SSDs).

KNOWLEDGEGATE
www.knowledgegate.in

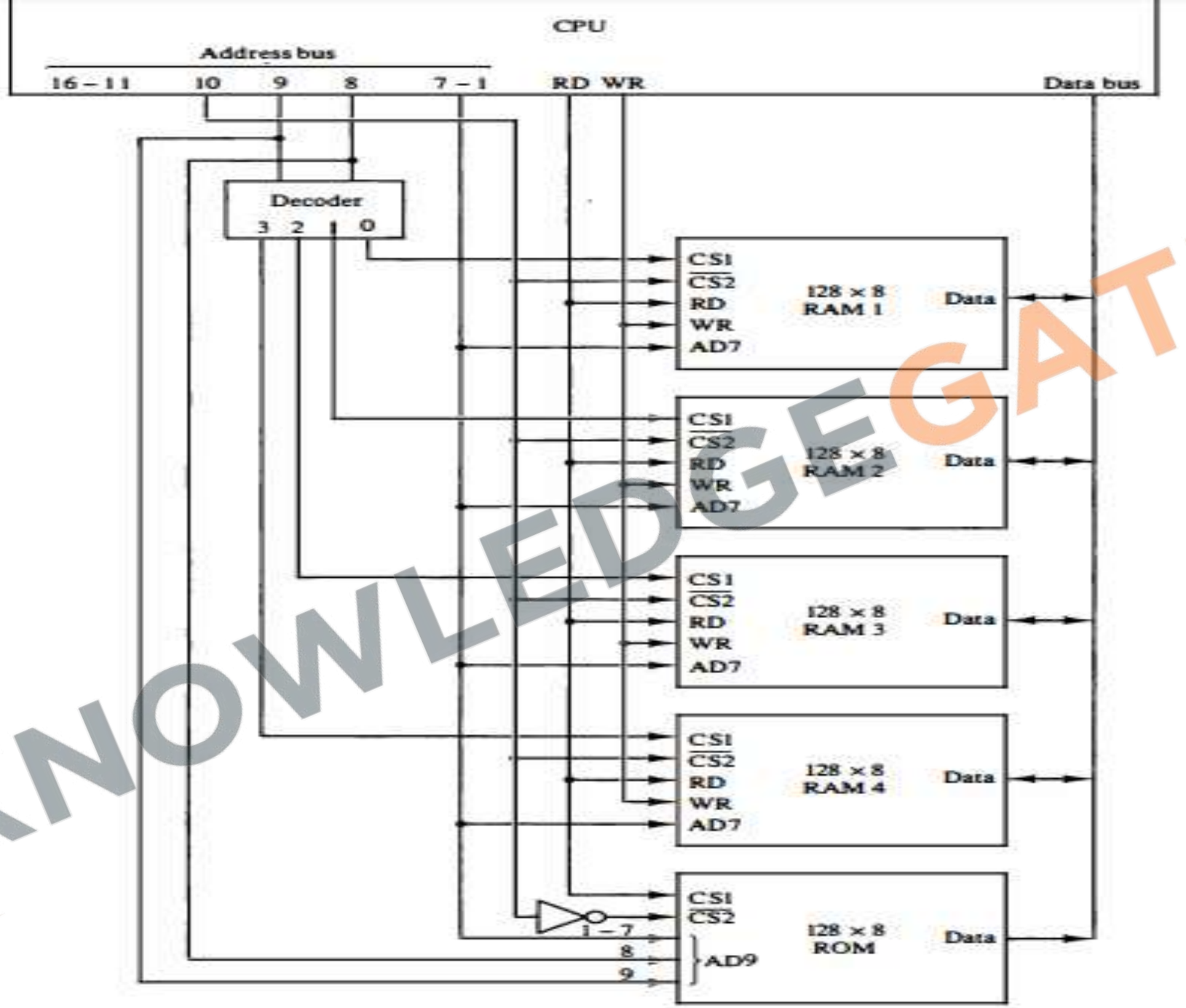


Figure 4 Memory connection to the CPU.

KNOWLEDGEGATE

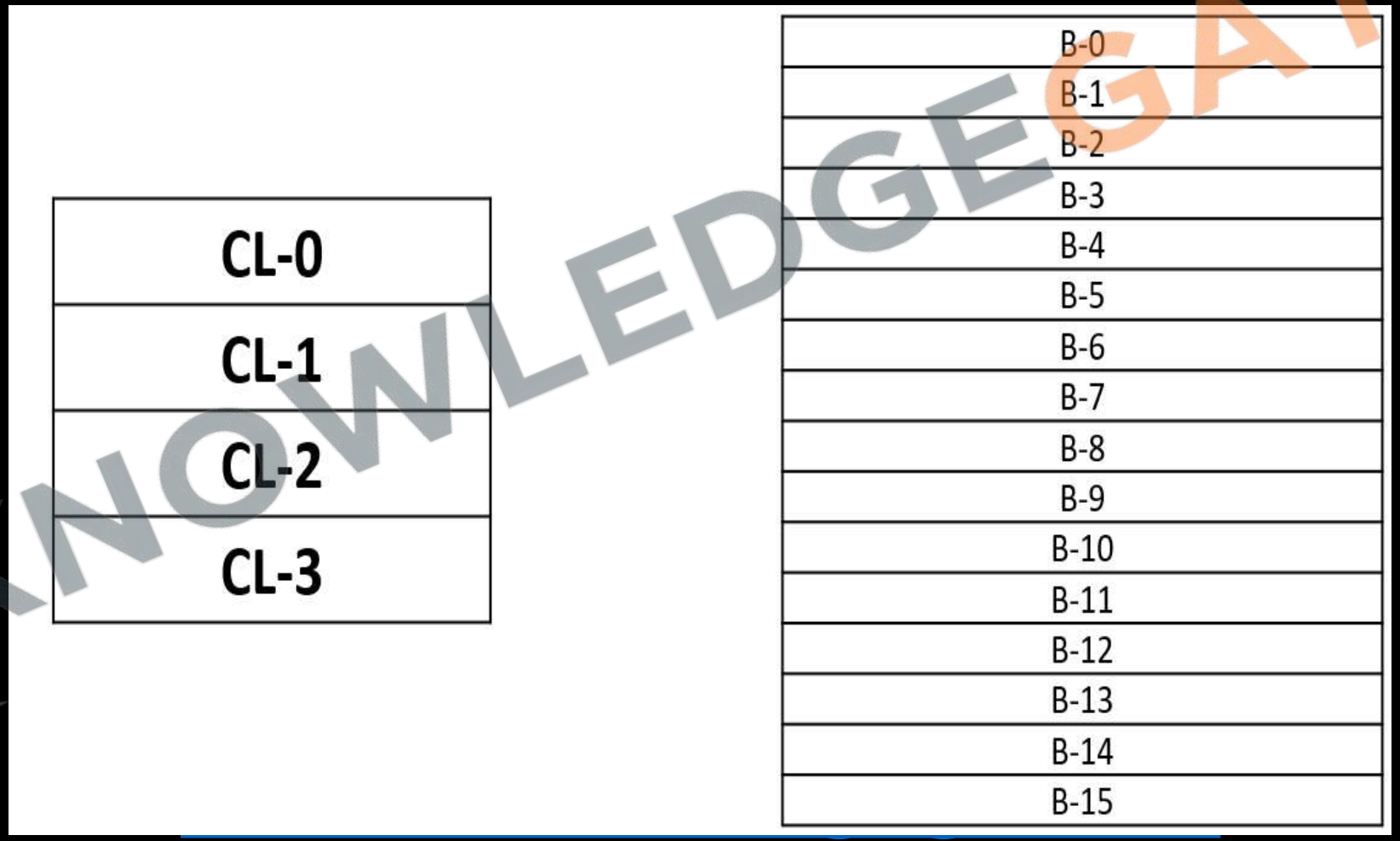
B-0	W-0
	W-1
	W-2
	W-3
B-1	W-4
	W-5
	W-6
	W-7
B-2	W-8
	W-9
	W-10
	W-11
B-3	W-12
	W-13
	W-14
	W-15
B-4	W-16
	W-17
	W-18
	W-19
B-5	W-20
	W-21
	W-22
	W-23
B-6	W-24
	W-25
	W-26
	W-27
B-7	W-28
	W-29
	W-30
	W-31
B-8	W-32
	W-33
	W-34
	W-35
B-9	W-36
	W-37
	W-38
	W-39
B-10	W-40
	W-41
	W-42
	W-43
B-11	W-44
	W-45
	W-46
	W-47
B-12	W-48
	W-49
	W-50
	W-51
B-13	W-52
	W-53
	W-54
	W-55
B-14	W-56
	W-57
	W-58
	W-59
B-15	W-60
	W-61
	W-62
	W-63

Main Memory				
B-0	W-0	W-1	W-2	W-3
B-1	W-4	W-5	W-6	W-7
B-2	W-8	W-9	W-10	W-11
B-3	W-12	W-13	W-14	W-15
B-4	W-16	W-17	W-18	W-19
B-5	W-20	W-21	W-22	W-23
B-6	W-24	W-25	W-26	W-27
B-7	W-28	W-29	W-30	W-31
B-8	W-32	W-33	W-34	W-35
B-9	W-36	W-37	W-38	W-39
B-10	W-40	W-41	W-42	W-43
B-11	W-44	W-45	W-46	W-47
B-12	W-48	W-49	W-50	W-51
B-13	W-52	W-53	W-54	W-55
B-14	W-56	W-57	W-58	W-59
B-15	W-60	W-61	W-62	W-63

KNOWLEDGEGATE
www.knowledgegate.in

- Cache mapping refers to the process of determining where data should be stored in the cache memory.

- The cache mapping algorithm determines which cache lines are assigned to which main memory blocks.



KNOWLEDGEGATE

- There are several types of cache mapping algorithms, including:
  - **Direct mapping**: Each main memory block can be stored in only one specific cache line.
  - **Associative mapping**: Any main memory block can be stored in any cache line.
  - **Set-associative mapping**: The cache is divided into sets, each of which contains several cache lines. A main memory block can be stored in any cache line within a set.

KNOWLEDGEGATE

www.knowledgegate.in

Direct Mapping

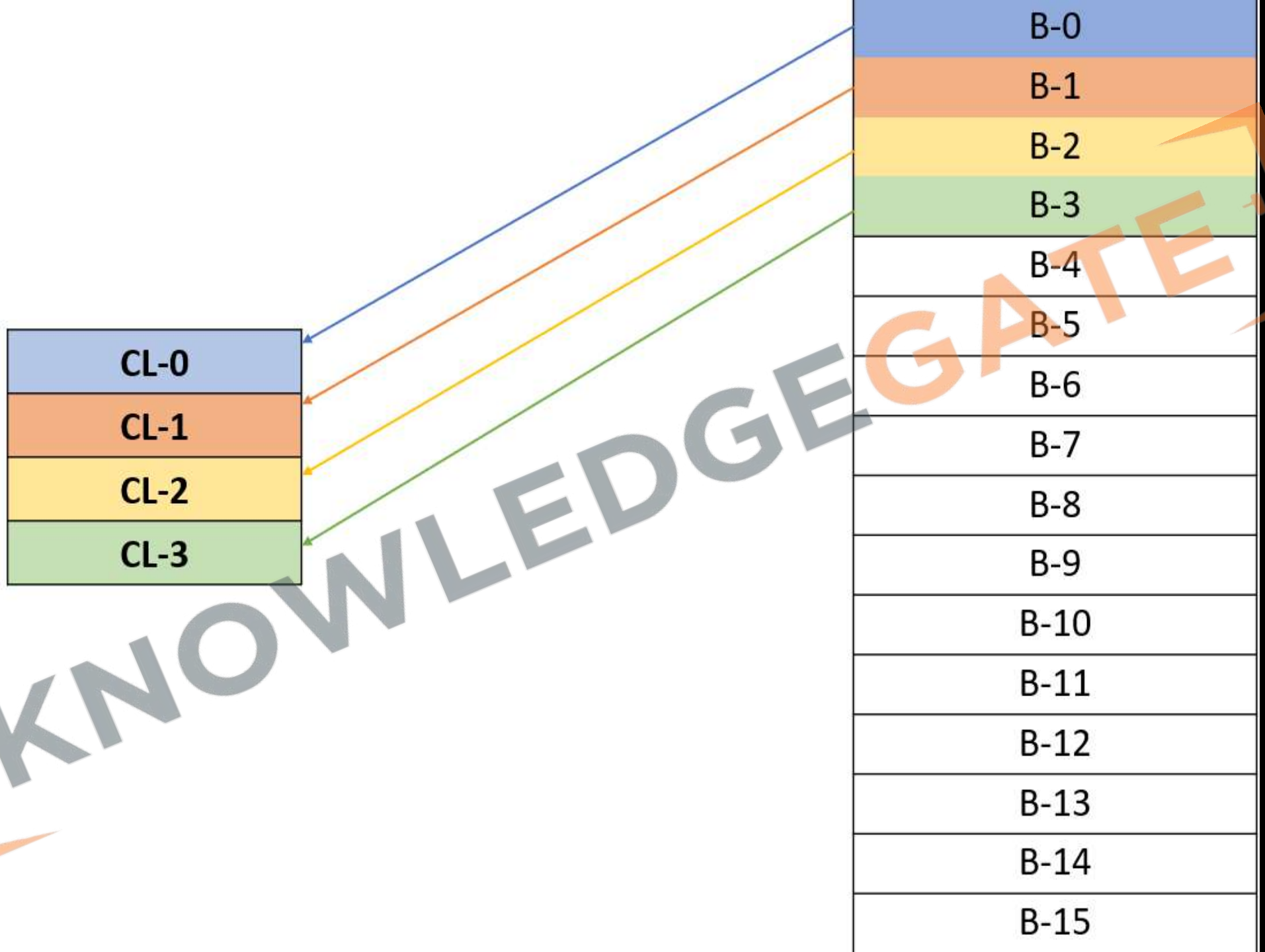
- Direct mapping is a type of cache mapping algorithm that maps each main memory block to a specific cache line.

CL-0
CL-1
CL-2
CL-3

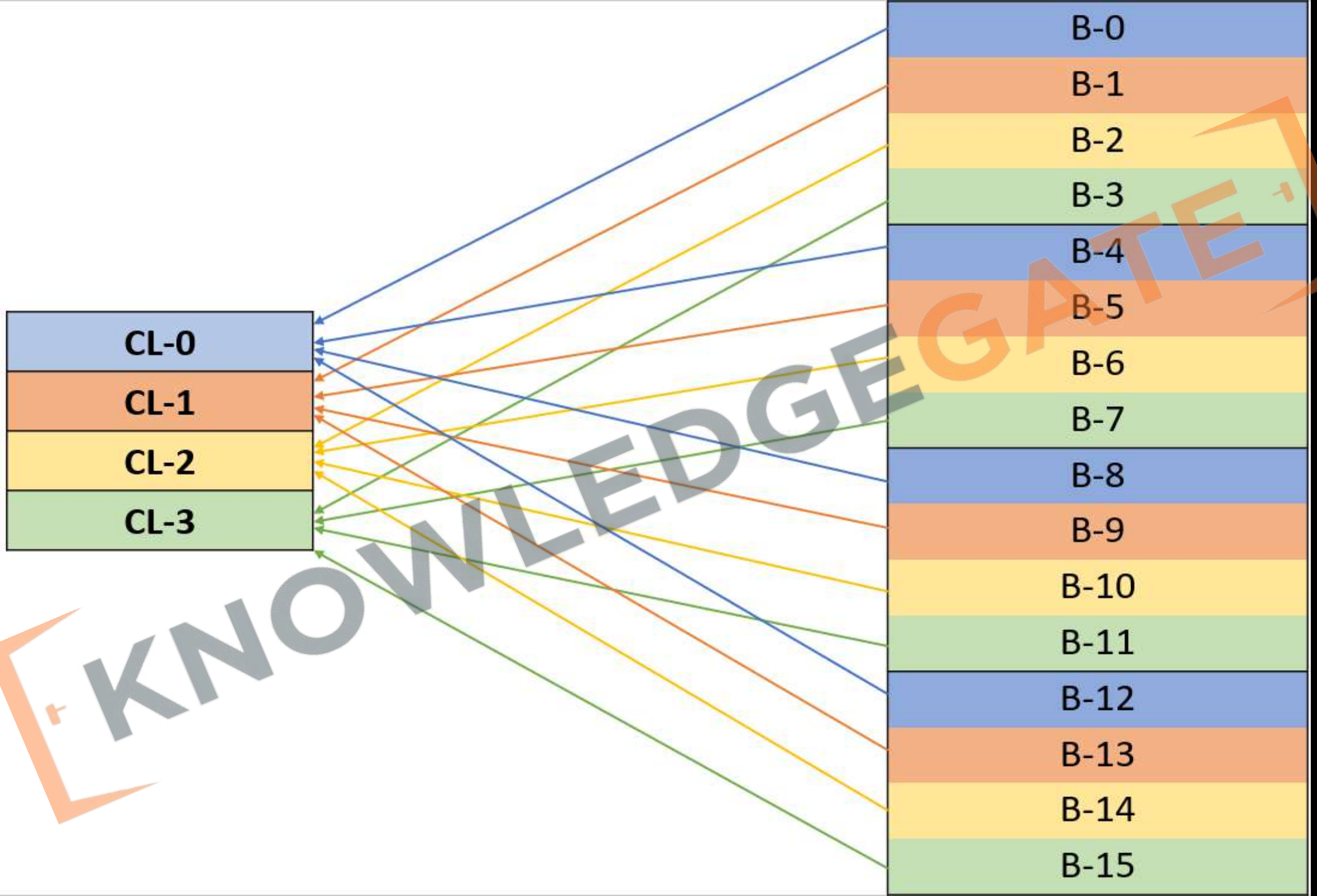
B-0
B-1
B-2
B-3
B-4
B-5
B-6
B-7
B-8
B-9
B-10
B-11
B-12
B-13
B-14
B-15

KNOWLEDGEGATE

www.knowledgegate.in

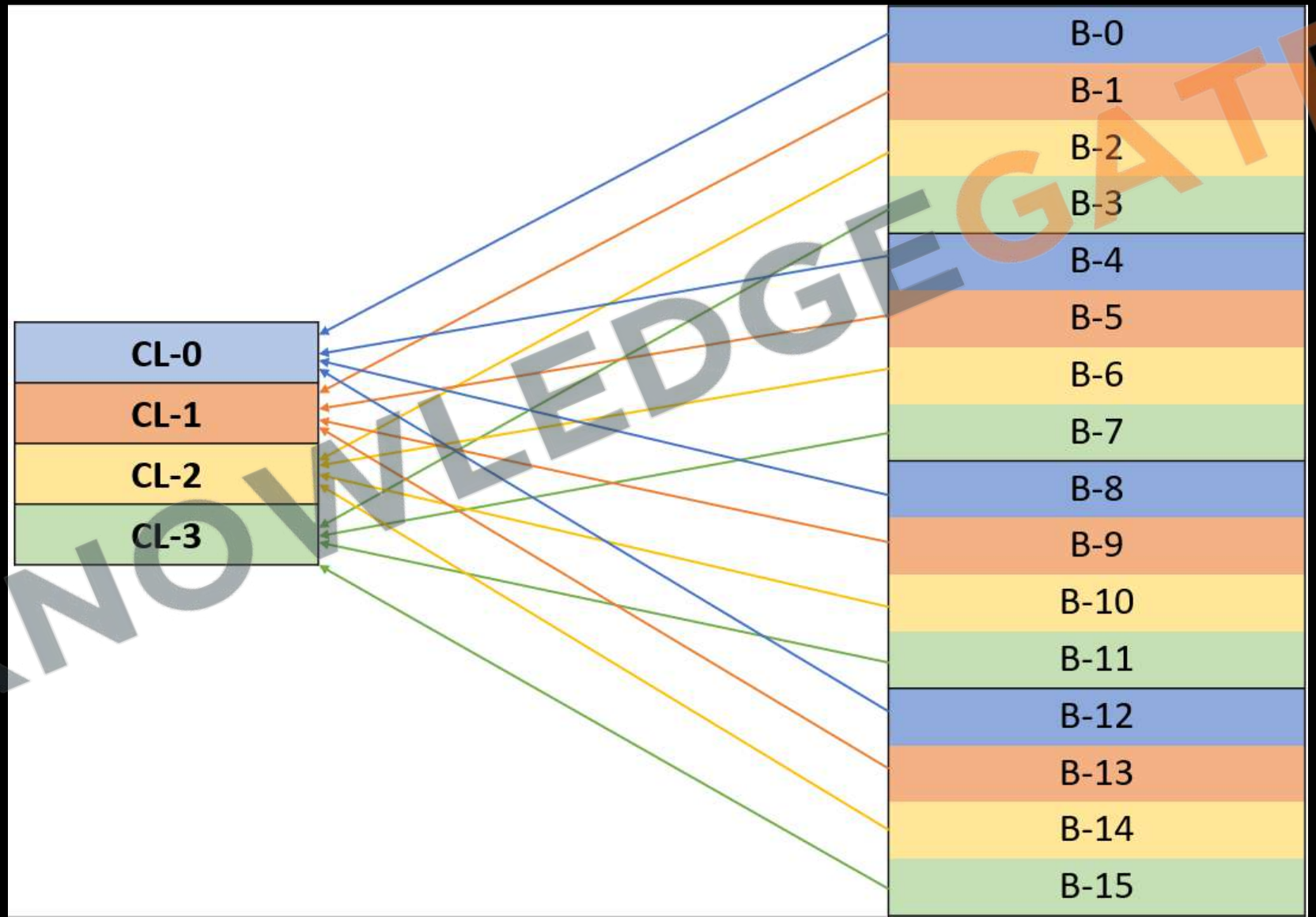


KNOWLEDGEGATE



KNOWLEDGEGATE

$$\frac{Block\ No}{No\ of\ Cache\ Line} = \text{Remainder is the Address of block in Cache}$$



KNOWLEDGE GATE

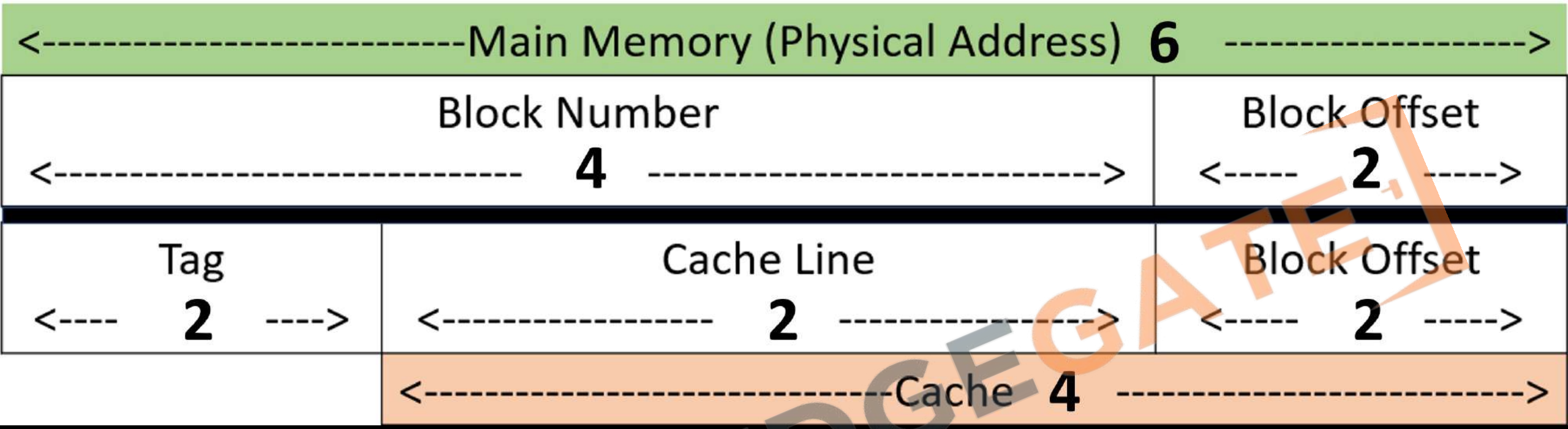
Cache Memory	
CL-0	B-0 / B-4 / B-8 / B-12
CL-1	B-1 / B-5 / B-9 / B-13
CL-2	B-2 / B-6 / B-10 / B-14
CL-3	B-3 / B-7 / B-11 / B-15

Cache Memory		
CL-0	TAG	W-
		W-
		W-
		W-
CL-1	TAG	W-
		W
		W-
		W-
CL-2	TAG	W-
		W-
		W-
		W-
CL-3	TAG	W-
		W-
		W-
		W-

Cache								
CL-0	B-0	W-0	B-4	W-16	B-8	W-32	B-12	W-48
		W-1		W-17		W-33		W-49
		W-2		W-18		W-34		W-50
		W-3		W-19		W-35		W-51
CL-1	B-1	W-4	B-5	W-20	B-9	W-36	B-13	W-52
		W-5		W-21		W-37		W-53
		W-6		W-22		W-38		W-54
		W-7		W-23		W-39		W-55
CL-2	B-2	W-8	B-6	W-24	B-10	W-40	B-14	W-56
		W-9		W-25		W-41		W-57
		W-10		W-26		W-42		W-58
		W-11		W-27		W-43		W-59
CL-3	B-3	W-12	B-7	W-28	B-11	W-44	B-15	W-60
		W-13		W-29		W-45		W-61
		W-14		W-30		W-46		W-62
		W-15		W-31		W-47		W-63

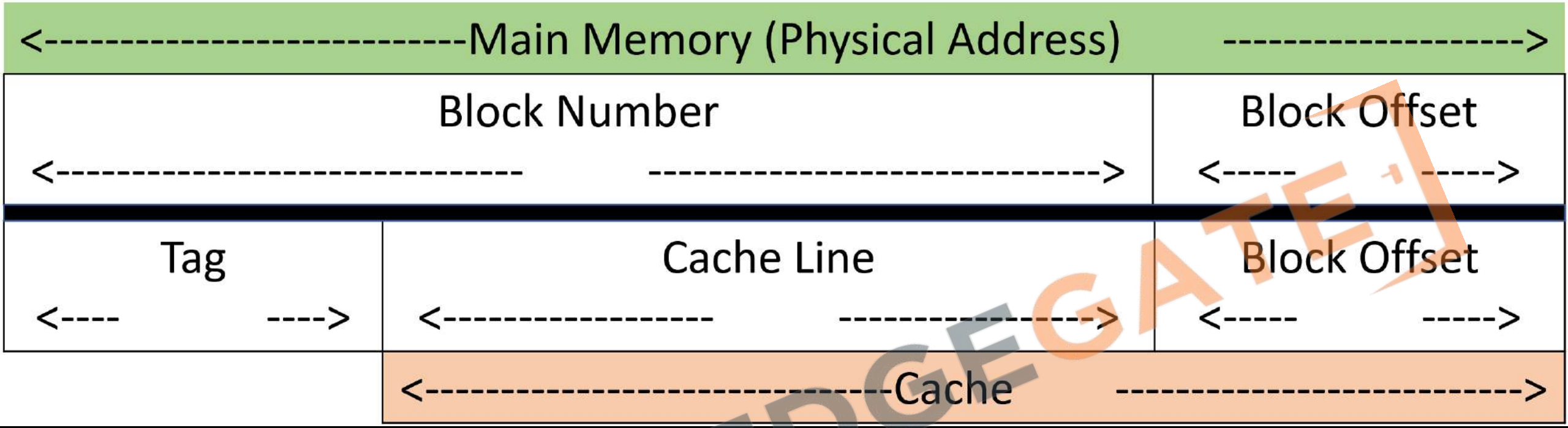
KNOWLEDGEGATE

v.knowledgegate.in



KNOWLEDGEGATE

www.knowledgegate.in



KNOWLEDGEGATE

www.knowledgegate.in

Cache Memory	
CL-0	B-0 / B-4 / B-8 / B-12
CL-1	B-1 / B-5 / B-9 / B-13
CL-2	B-2 / B-6 / B-10 / B-14
CL-3	B-3 / B-7 / B-11 / B-15

Cache Memory		
CL-0	TAG 11	W- W- W- 100111 W-
CL-1	TAG 01	W- W W- W-
CL-2	TAG 01	W- W- W- W-
CL-3	TAG 10	W- W- W- W-

Cache

CL-0	B-0	W-0	B-4	W-16	B-8	W-32	B-12	W-48
		W-1		W-17		W-33		W-49
		W-2		W-18		W-34		W-50
		W-3		W-19		W-35		W-51
CL-1	B-1	W-4	B-5	W-20	B-9	W-36	B-13	W-52
		W-5		W-21		W-37		W-53
		W-6		W-22		W-38		W-54
		W-7		W-23		W-39		W-55
CL-2	B-2	W-8	B-6	W-24	B-10	W-40	B-14	W-56
		W-9		W-25		W-41		W-57
		W-10		W-26		W-42		W-58
		W-11		W-27		W-43		W-59
CL-3	B-3	W-12	B-7	W-28	B-11	W-44	B-15	W-60
		W-13		W-29		W-45		W-61
		W-14		W-30		W-46		W-62
		W-15		W-31		W-47		W-63

1) W-18
0 1 0 0 1 0

2) W-51
1 1 0 0 1 1

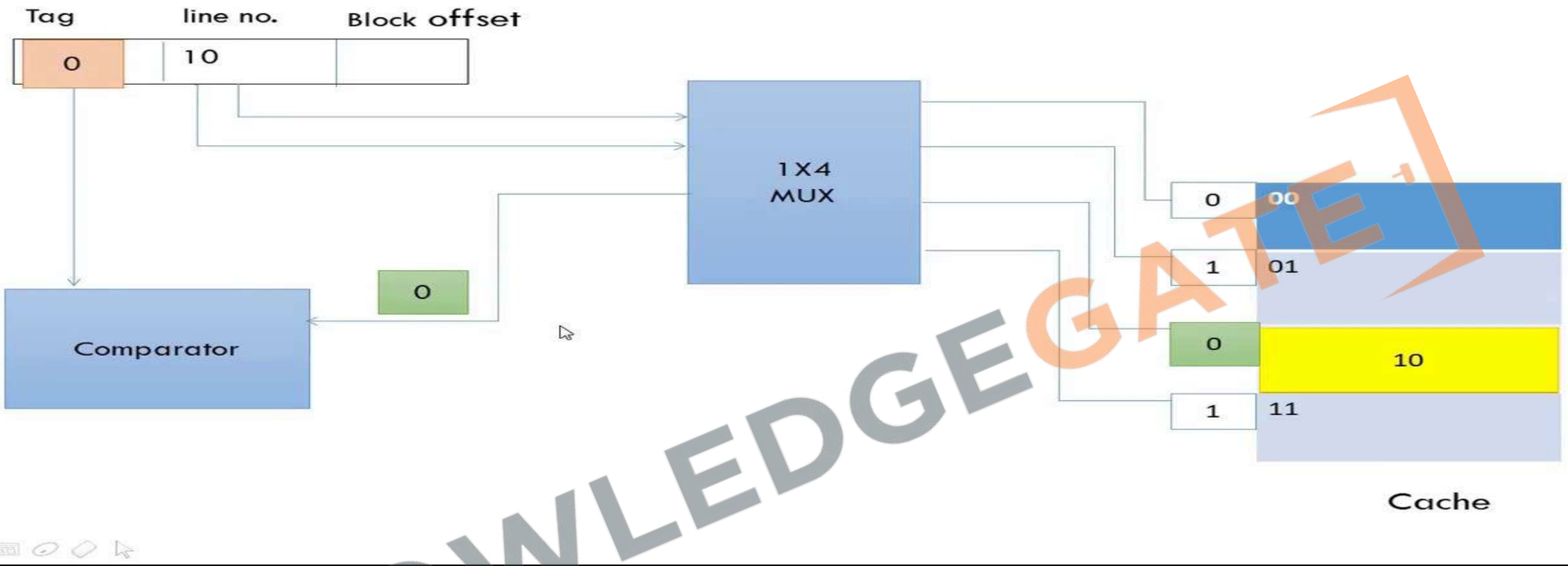
3) W-39
1 0 0 1 1 1

4) W-22
0 1 0 1 1 0

5) W-24
0 1 1 0 0 0

6) W-56
1 1 1 0 0 0

KNOWLEDGEGATE
knowledgegate.in



- **Tag directory size = Number of tags x Tag size = Number of lines in cache x Number of bits in tag**

KNOWLEDGEGATE

www.knowledgegate.in

- The main advantage of direct mapping is its simplicity. Since each main memory block maps to only one specific cache line, there is no need for complicated search algorithms to determine the location of a memory block in the cache. This makes direct mapping relatively fast and efficient.

KNOWLEDGEGATE

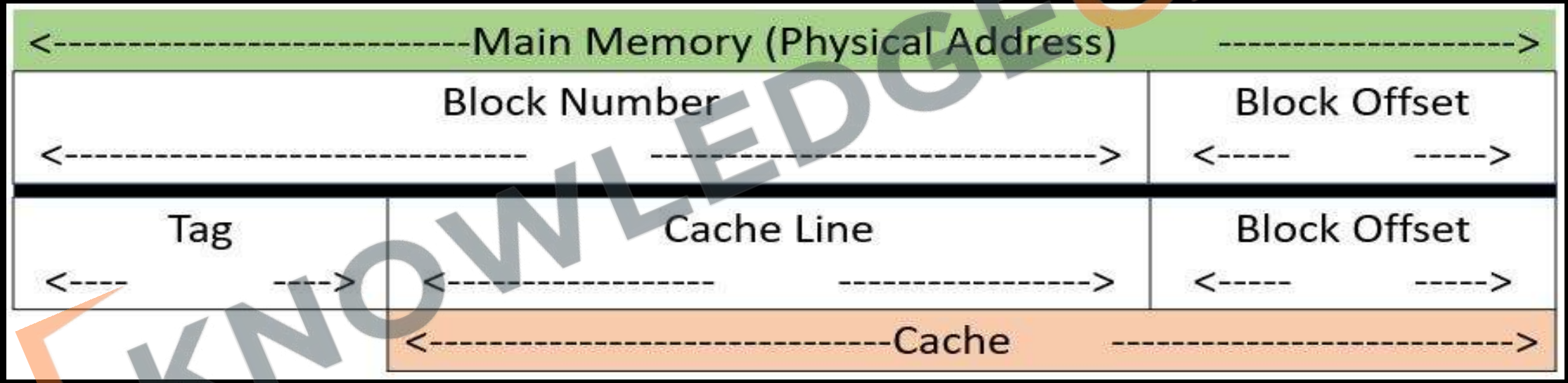
www.knowledgegate.in

- However, the simplicity of direct mapping also has its drawbacks. Since each cache line can store only one main memory block, it is possible for two memory blocks with different addresses to map to the same cache line. This is known as a cache conflict and can lead to cache thrashing, where the cache is constantly replacing one block with another, reducing the cache hit rate.
- Overall, direct mapping is suitable for small cache memories, where the likelihood of cache conflicts is low, and the benefits of simplicity outweigh the drawbacks.

KNOWLEDGE GATE

www.knowledgegate.in

MM Size	Cache Size	Block Size	No of bits in Tag	Tag Directory Size
16 GB	32 MB	4 KB		
128 MB	256 KB	512 B		
32 GB	128 MB	1 KB		
256 MB	16 KB	1 KB		
4 GB	8 MB	2 KB		
512 KB	2 KB	128 B		



KNOWLEDGE GATE
www.knowledgegate.in

MM Size	Cache Size	Block Size	No of bits in Tag	Tag Directory Size
128 KB	16 KB	256 B	3	
32 GB	32 KB	1 KB	20	
2^{26}B	512 KB	1 KB	7	
16 GB	2^{24} B	4 KB	10	
64 MB	2^{16} B	?	10	
2^{26} B	512 KB	?	7	

KNOWLEDGEGATE

www.knowledgegate.in

MM Size	Cache Size	Block Size	No of bits in Tag	Tag Directory Size
128 KB	16 KB	256 B	3	$3 * 2^6$
32 GB	32 KB	1 KB	20	$20 * 2^5$
2^{26}B	512 KB	1 KB	7	$7*2^9$
16 GB	2^{24} B	4 KB	10	$10*2^{12}$
64 MB	2^{16} B	?	10	?
2^{26} B	512 KB	?	7	?

KNOWLEDGEGATE

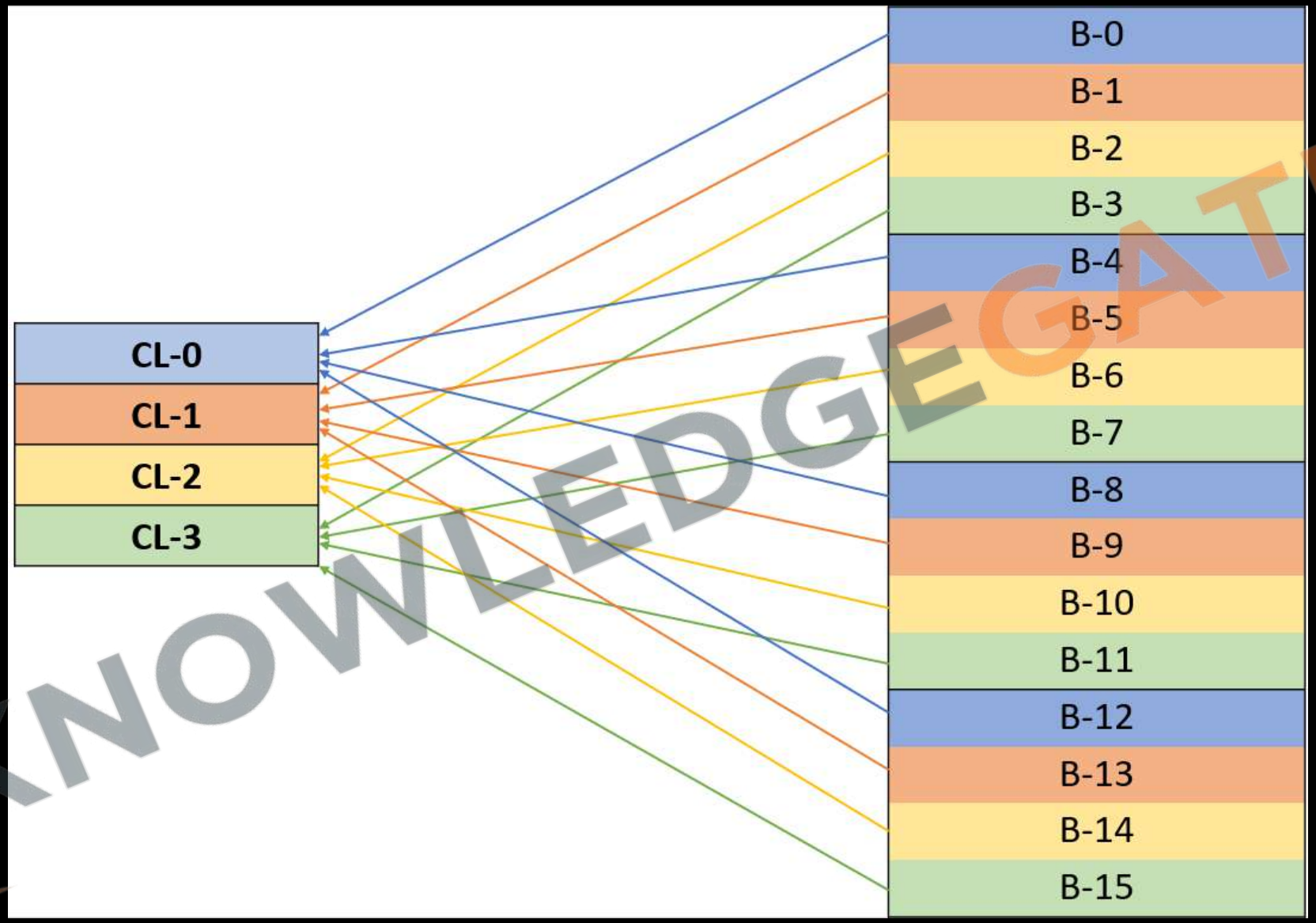
www.knowledgegate.in

KNOWLEDGE GATE

Associative Mapping

- To overcome the problem of conflict-miss in direct mapping we have Associative Mapping.
- A block of main memory can be mapped to any freely available cache line. This makes fully associative mapping more flexible than direct mapping.
- It is also known as many to many mappings.

KNOWLEDGEGATE
www.knowledgegate.in



KNOWLEDGEGATE

www.knowledgegate.in

Cache Memory		
CL-0	TAG = Block no	W-
		W-
		W-
		W-
CL-1	TAG = Block no	W-
		W
		W-
		W-
CL-2	TAG = Block no	W-
		W-
		W-
		W-
CL-3	TAG = Block no	W-
		W-
		W-
		W-

Main Memory				
B-0	W-0	W-1	W-2	W-3
B-1	W-4	W-5	W-6	W-7
B-2	W-8	W-9	W-10	W-11
B-3	W-12	W-13	W-14	W-15
B-4	W-16	W-17	W-18	W-19
B-5	W-20	W-21	W-22	W-23
B-6	W-24	W-25	W-26	W-27
B-7	W-28	W-29	W-30	W-31
B-8	W-32	W-33	W-34	W-35
B-9	W-36	W-37	W-38	W-39
B-10	W-40	W-41	W-42	W-43
B-11	W-44	W-45	W-46	W-47
B-12	W-48	W-49	W-50	W-51
B-13	W-52	W-53	W-54	W-55
B-14	W-56	W-57	W-58	W-59
B-15	W-60	W-61	W-62	W-63

KNOWLEDGE GATE

www.knowledgegate.in

Cache Memory	
CL-0	B-0 / B-4 / B-8 / B-12
CL-1	B-1 / B-5 / B-9 / B-13
CL-2	B-2 / B-6 / B-10 / B-14
CL-3	B-3 / B-7 / B-11 / B-15

Cache Memory		
CL-0	TAG	W-
		W-
		W-
		W-
CL-1	TAG	W-
		W
		W-
		W-
CL-2	TAG	W-
		W-
		W-
		W-
CL-3	TAG	W-
		W-
		W-
		W-

Cache								
CL-0	B-0	W-0	B-4	W-16	B-8	W-32	B-12	W-48
		W-1		W-17		W-33		W-49
		W-2		W-18		W-34		W-50
		W-3		W-19		W-35		W-51
CL-1	B-1	W-4	B-5	W-20	B-9	W-36	B-13	W-52
		W-5		W-21		W-37		W-53
		W-6		W-22		W-38		W-54
		W-7		W-23		W-39		W-55
CL-2	B-2	W-8	B-6	W-24	B-10	W-40	B-14	W-56
		W-9		W-25		W-41		W-57
		W-10		W-26		W-42		W-58
		W-11		W-27		W-43		W-59
CL-3	B-3	W-12	B-7	W-28	B-11	W-44	B-15	W-60
		W-13		W-29		W-45		W-61
		W-14		W-30		W-46		W-62
		W-15		W-31		W-47		W-63

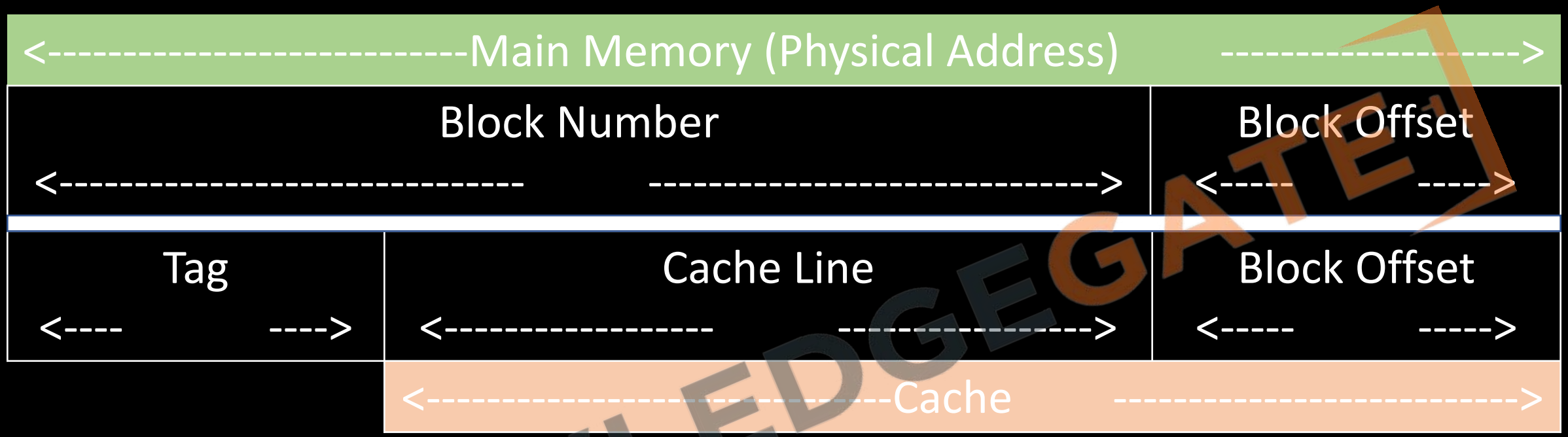
KNOWLEDGEGATE

v.knowledgegate.in

- A replacement algorithm is needed to replace a block if the cache is full.
- In fully associative mapping we only have two fields: Tag/Block Number field and a Block offset field.
- Here the number of bits in tag = number of bits to require to represent block number

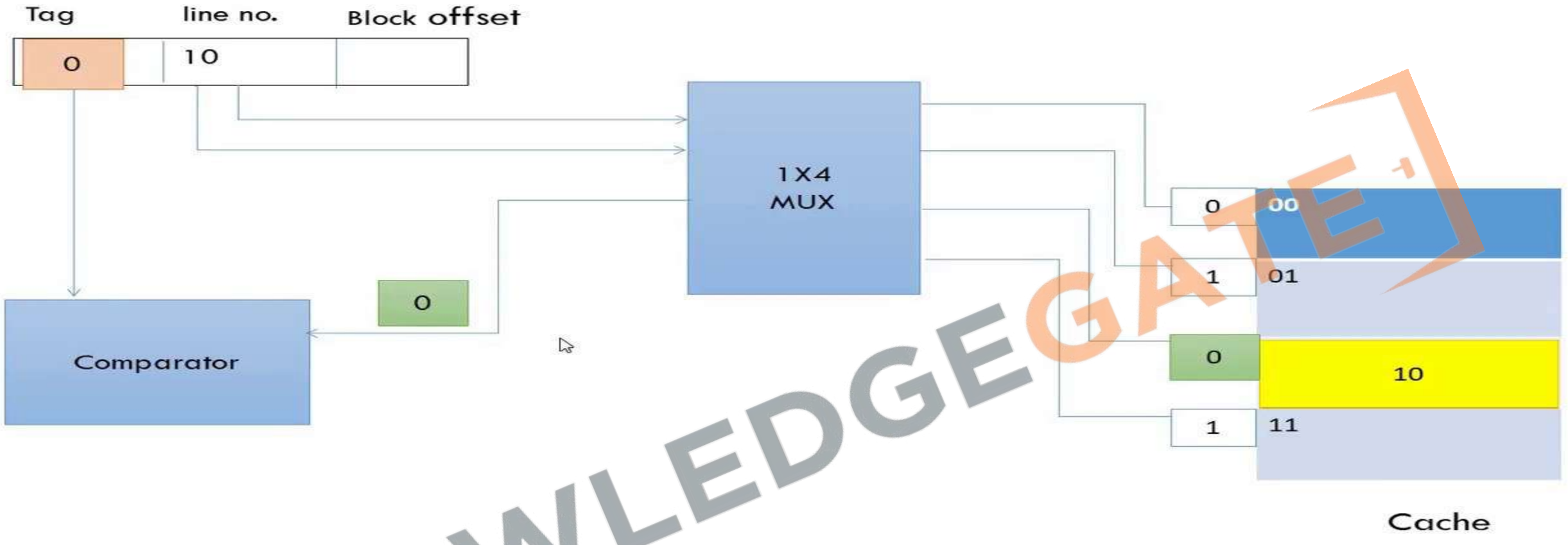
KNOWLEDGEGATE

www.knowledgegate.in



KNOWLEDGEGATE

www.knowledgegate.in



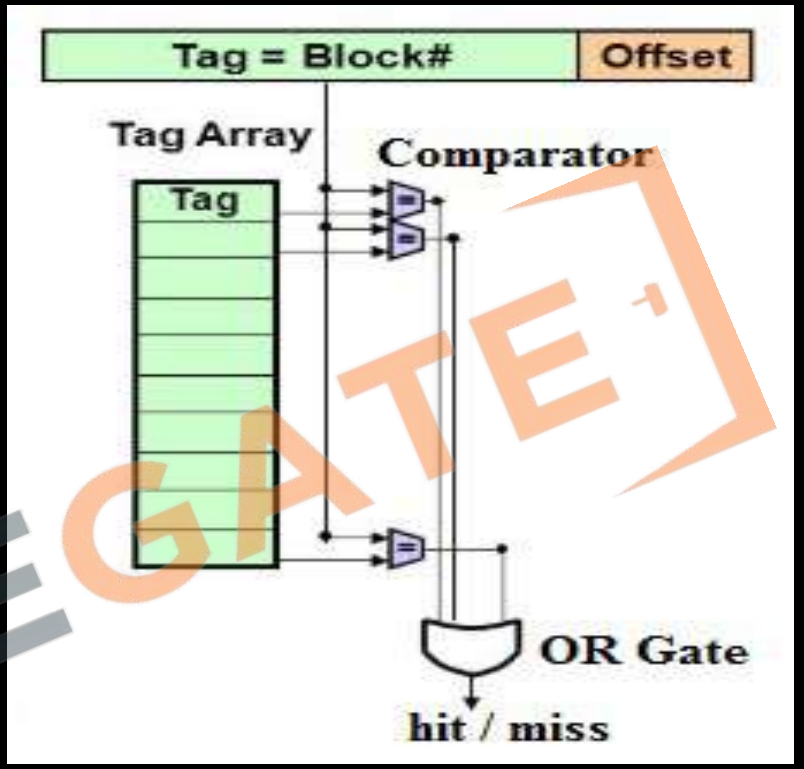
KNOWLEDGEGATE

Tag directory size = Number of tags x Tag size = Number of lines in cache x Number of bits in tag

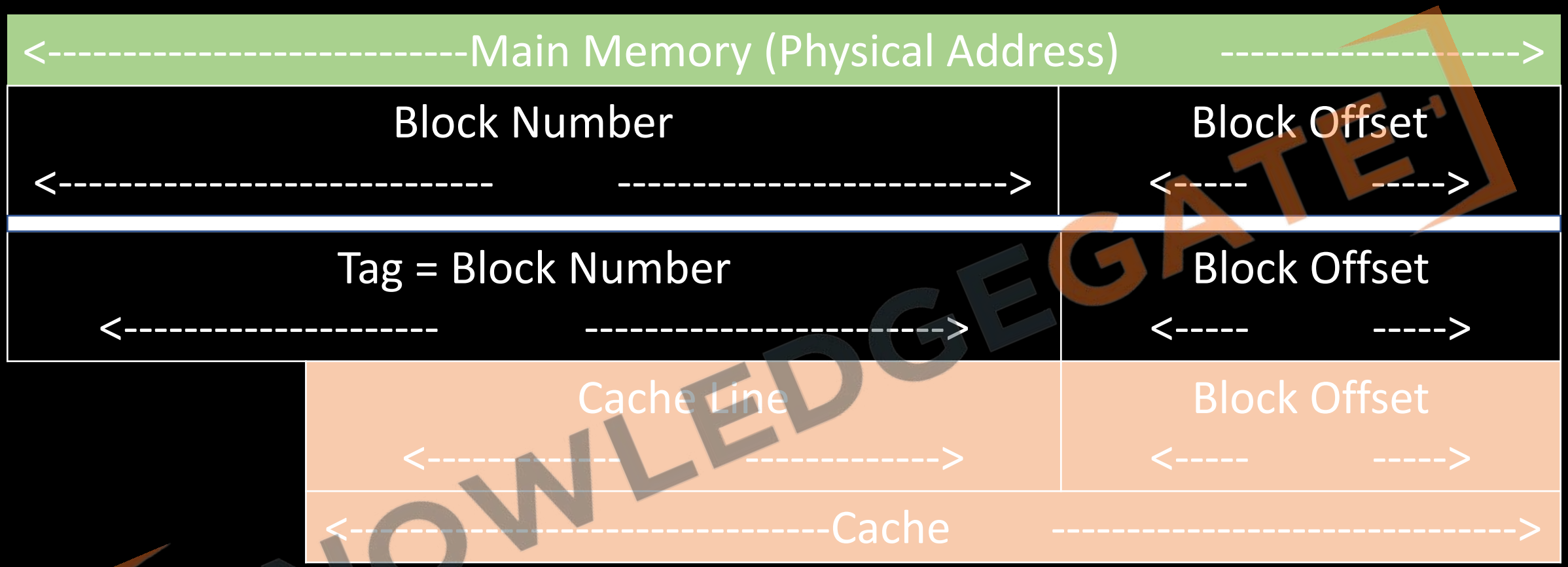
www.knowledgegate.in

Hardware Architecture

- If we have ‘n’ lines in cache then ‘n’ number of comparators are required.
- Size of comparator = Size of Tag
- If we have ‘n’ bit tag then we require ‘n’ bit comparator.

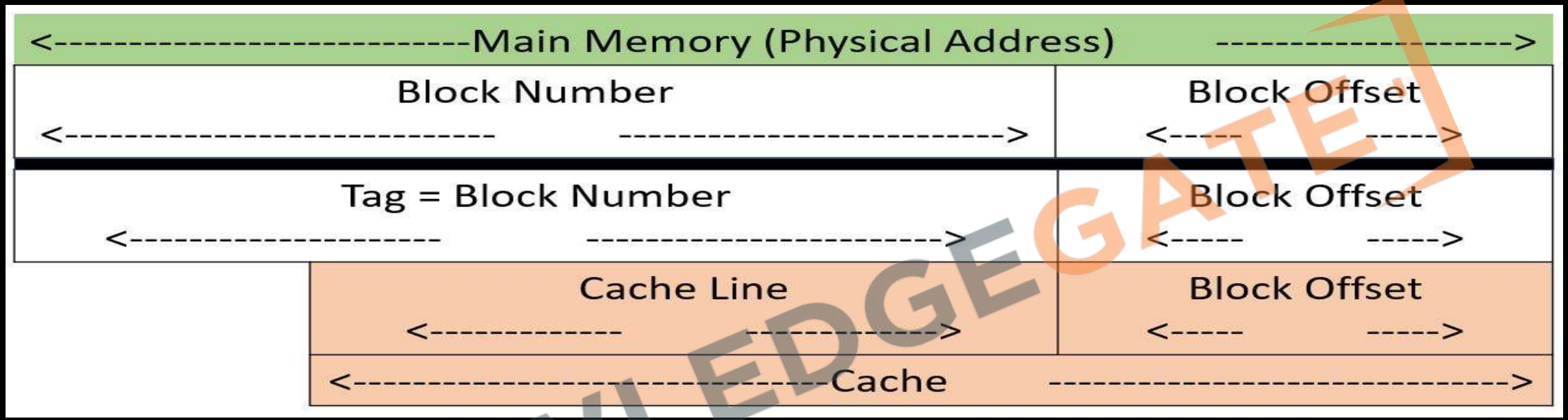


KNOWLEDGE GATE



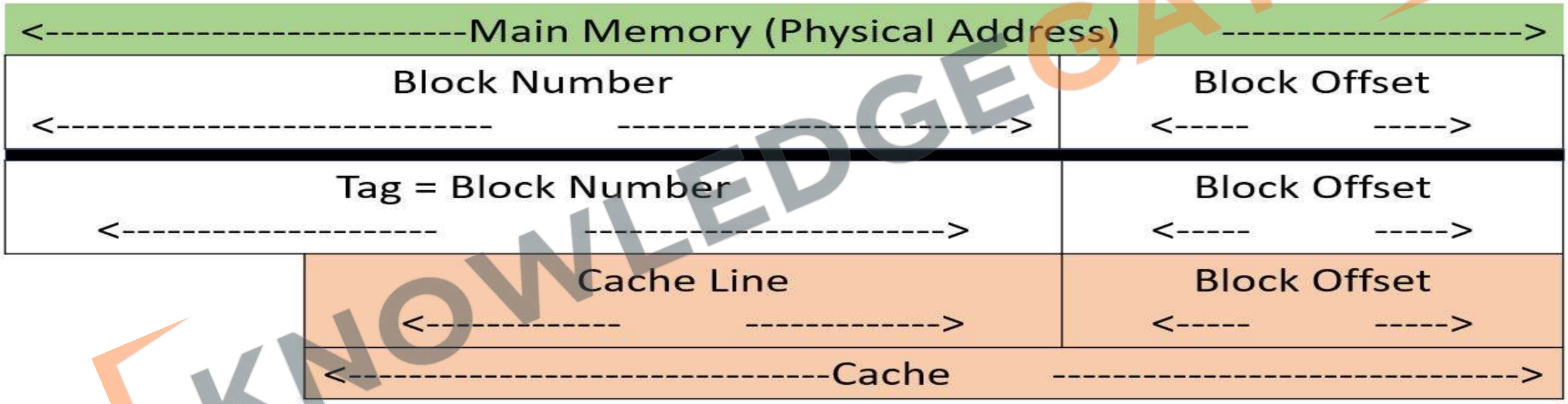
www.knowledgegate.in

Q Consider a fully associative mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find out the: Number of bits in tag and Tag directory size?



www.knowledgegate.in

MM Size	Cache Size	Block Size	No of bits in Tag	Tag Directory Size	Comp
128 KB	16 KB	256 B			
32 GB	32 KB	1 KB			
	512 KB	1 KB	17		
16 GB		4 KB	22		
64MB			10		
	512 KB		7		



KNOWLEDGEGATE

www.knowledgegate.in

MM Size	Cache Size	Block Size	No of bits in Tag	Tag Directory Size	Comp
128 KB	16 KB	256 B	9	9 * 64	64
32 GB	32 KB	1 KB	25	25 * 32	32
128 MB	512 KB	1 KB	17	512 *17	512
16 GB	?	4 KB	22	?	?
64MB	?	64 KB	10	?	?
?	512 KB		7	?	?

KNOWLEDGEGATE

www.knowledgegate.in

Disadvantage

- Hardware cost is high as compared to direct mapping.
- Tag directory size is more as compared to direct mapping.

KNOWLEDGEGATE

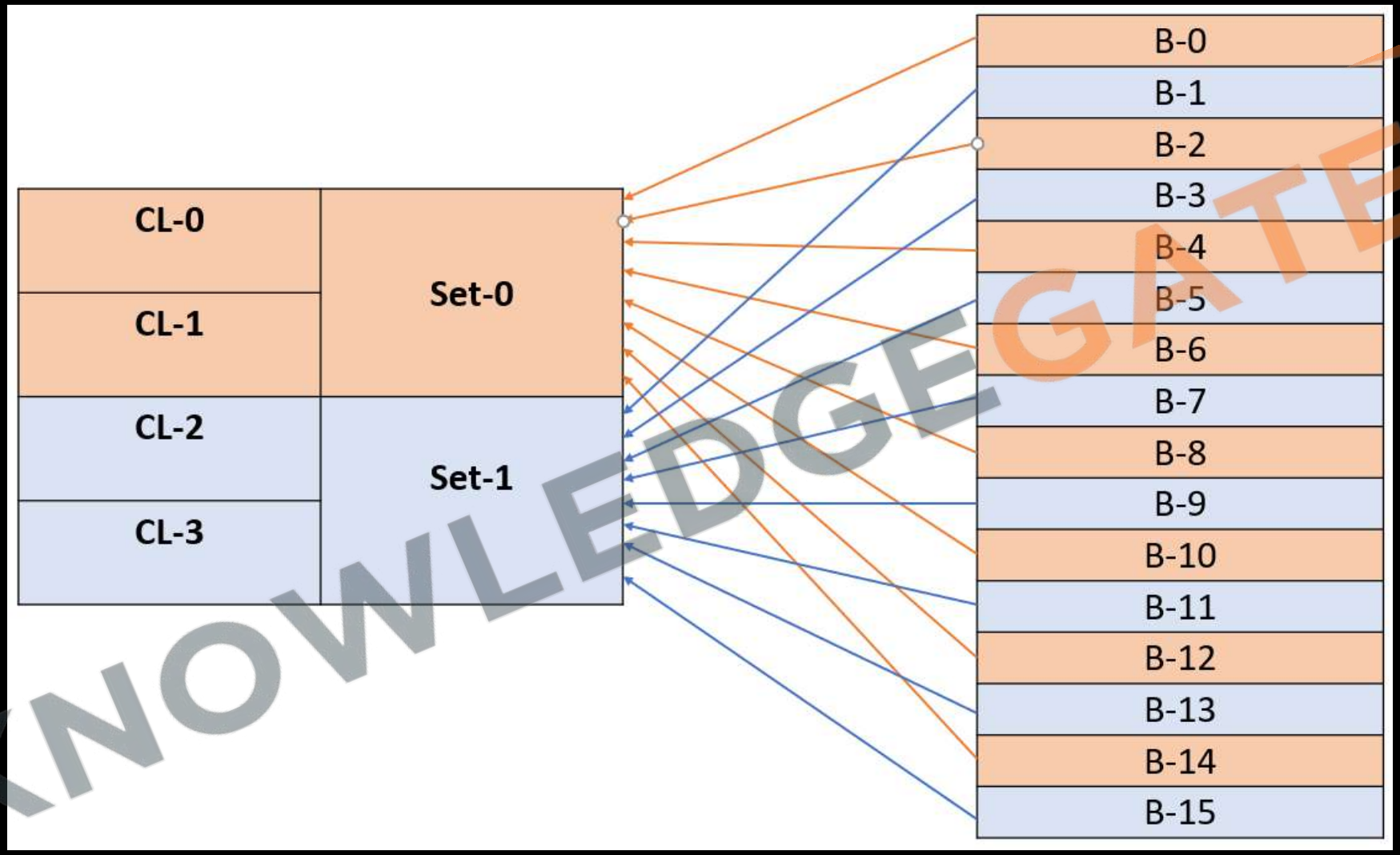
www.knowledgegate.in

Set Associative Mapping

- In k-way set associative mapping, cache lines are grouped into sets where each set contains “k” number of lines.
- A particular block of main memory can map to only one particular set of the cache.
- However, within that set, the memory block can map to any freely available cache line.

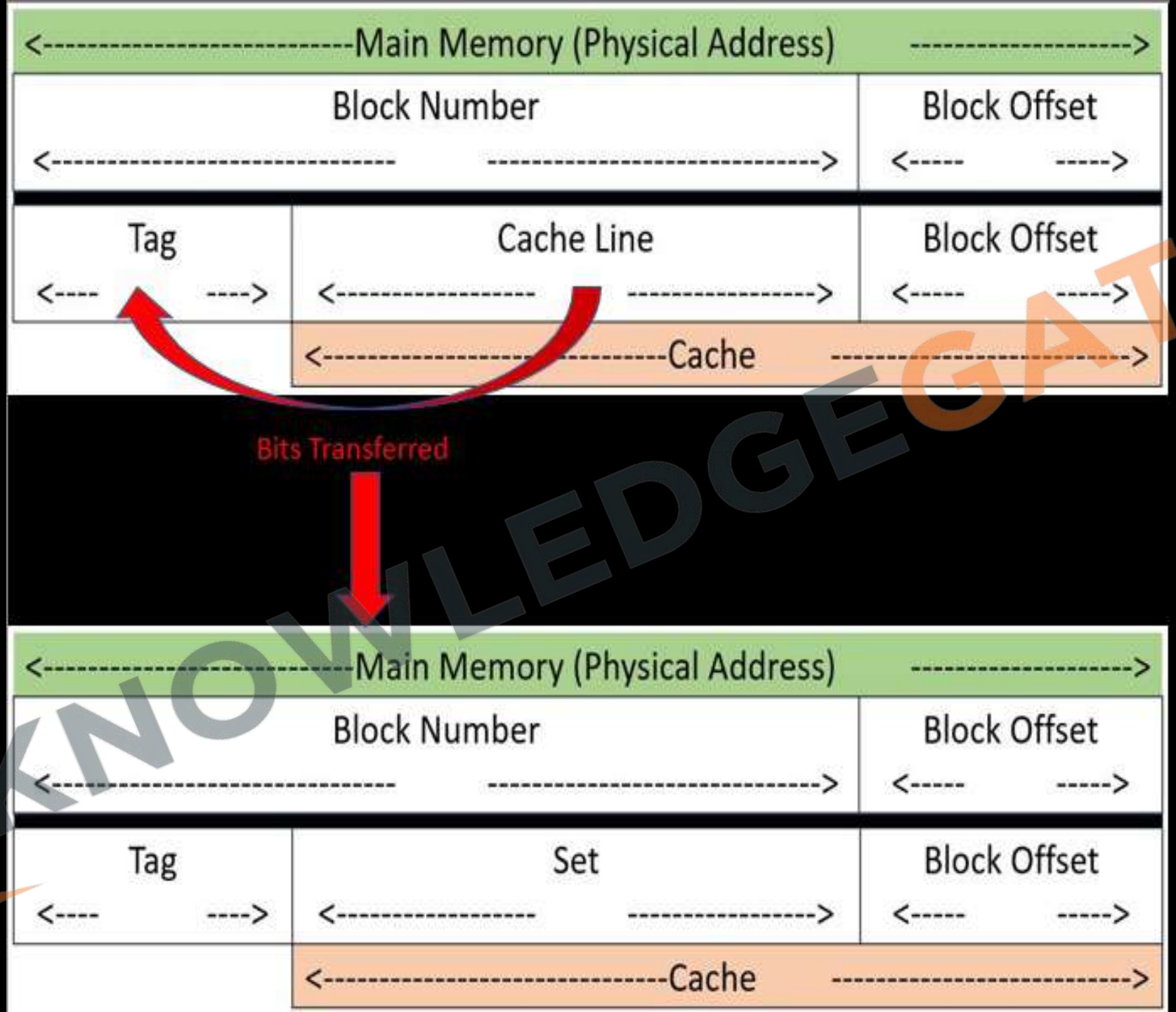
KNOWLEDGEGATE

www.knowledgegate.in



KNOWLEDGEGATE

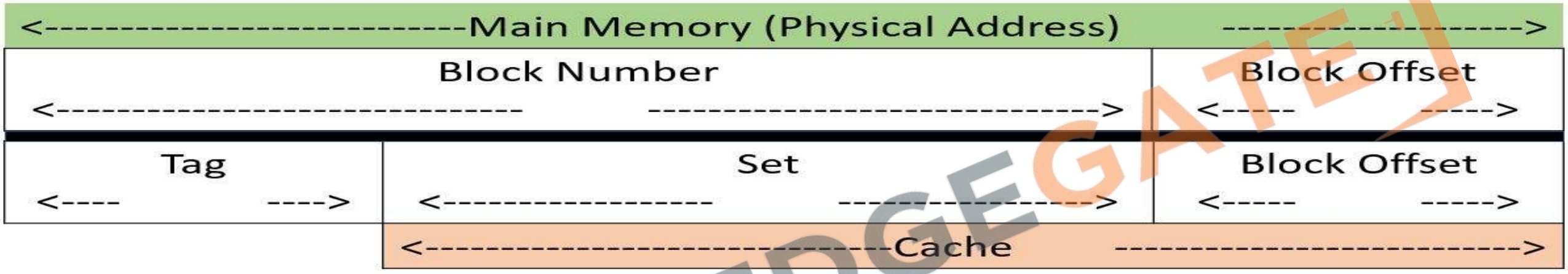
www.knowledgegate.in



KNOWLEDGEGATE

Formulas

- Number of Sets = No of Lines in Cache / No of Cache line in a set(k)(k-way set associative)



KNOWLEDGEGATE

Main Memory				
B-0	W-0	W-1	W-2	W-3
B-1	W-4	W-5	W-6	W-7
B-2	W-8	W-9	W-10	W-11
B-3	W-12	W-13	W-14	W-15
B-4	W-16	W-17	W-18	W-19
B-5	W-20	W-21	W-22	W-23
B-6	W-24	W-25	W-26	W-27
B-7	W-28	W-29	W-30	W-31
B-8	W-32	W-33	W-34	W-35
B-9	W-36	W-37	W-38	W-39
B-10	W-40	W-41	W-42	W-43
B-11	W-44	W-45	W-46	W-47
B-12	W-48	W-49	W-50	W-51
B-13	W-52	W-53	W-54	W-55
B-14	W-56	W-57	W-58	W-59
B-15	W-60	W-61	W-62	W-63

Cache			
Set Number-0	CL-0	TAG = Block no – CL	W-
			W-
			W-
			W-
	CL-1	TAG = Block no – CL	W-
			W
			W-
			W-
Set Number-1	CL-2	TAG = Block no – CL	W-
			W-
			W-
			W-
	CL-3	TAG = Block no - CL	W-
			W-
			W-
			W-

KNOWLEDGEGATE

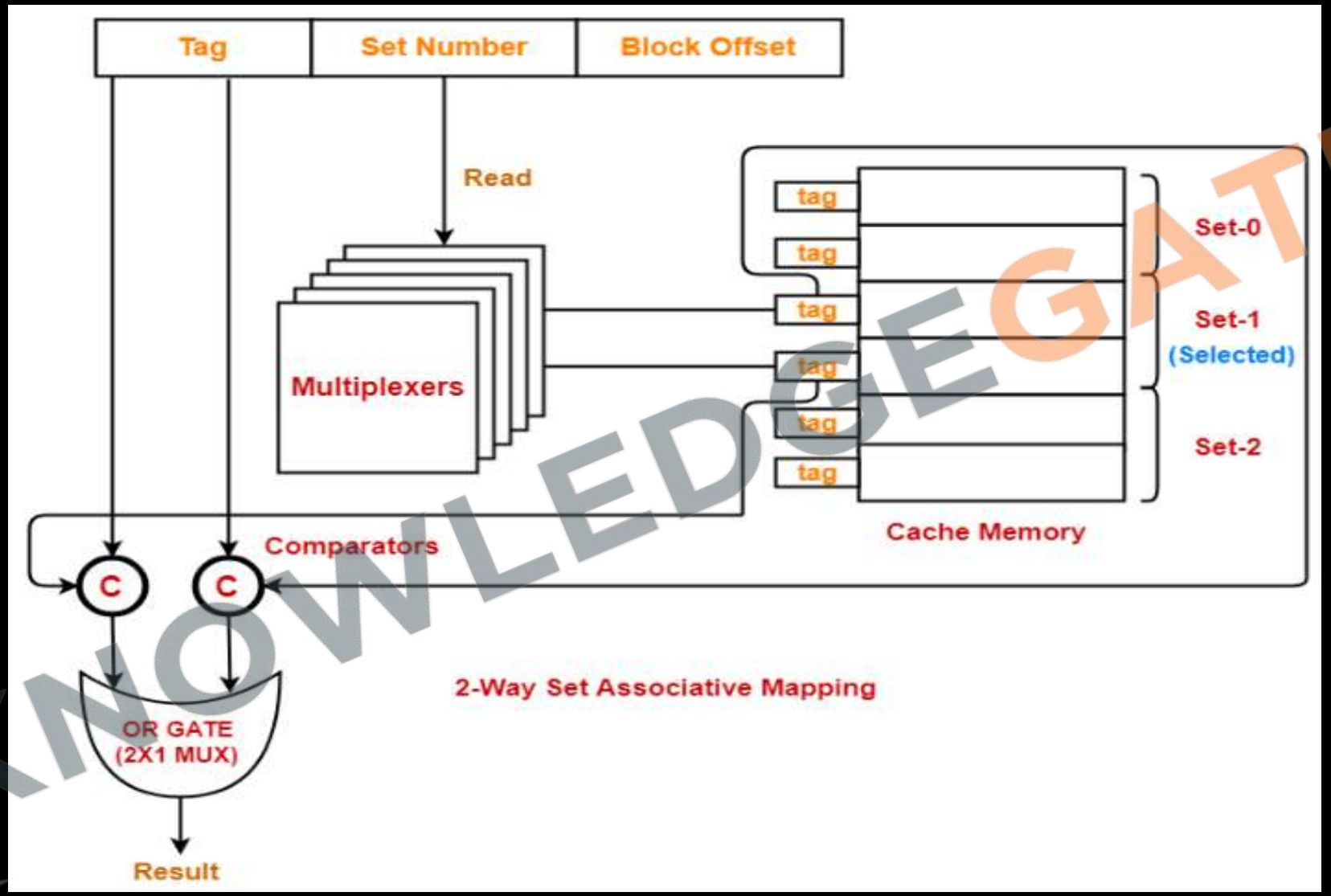
www.knowledgegate.in

	Cache									
Set Number-0	CL-0	Tag	B-0	W-0	B-2	W-8	B-4	W-16	B-6	W-24
				W-1		W-9		W-17		W-25
				W-2		W-10		W-18		W-26
				W-3		W-11		W-19		W-27
	CL-1	Tag	B-8	W-32	B-10	W-40	B-12	W-48	B-14	W-56
				W-33		W-41		W-49		W-57
				W-34		W-42		W-50		W-58
				W-35		W-43		W-51		W-59
Set Number-1	CL-2	Tag	B-1	W-4	B-3	W-12	B-5	W-20	B-7	W-28
				W-5		W-13		W-21		W-29
				W-6		W-14		W-22		W-30
				W-7		W-15		W-23		W-31
	CL-3	Tag	B-9	W-36	B-11	W-44	B-13	W-52	B-15	W-60
				W-37		W-45		W-53		W-61
				W-38		W-46		W-54		W-62
				W-39		W-47		W-55		W-63

KNOWLEDGEGATE

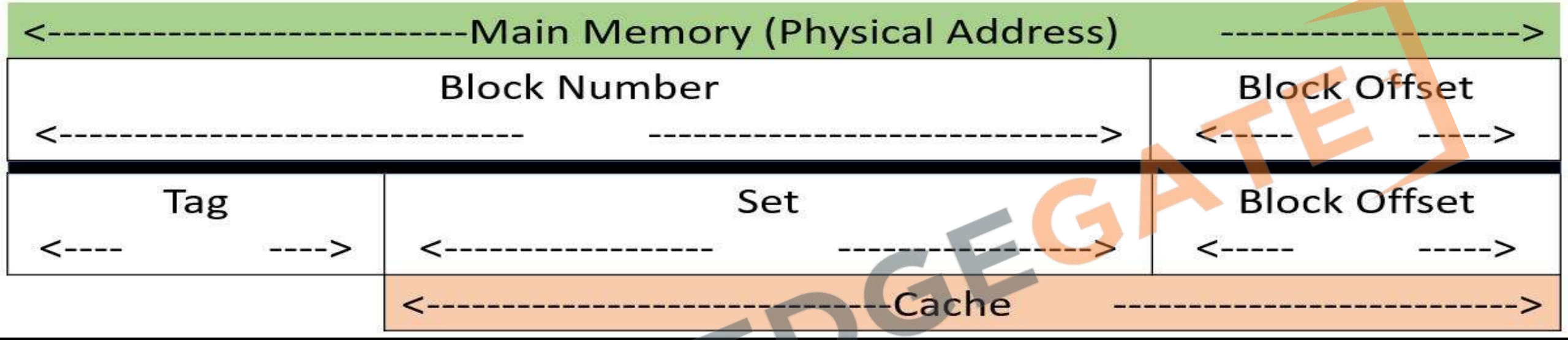
www.knowledgegate.in

Hardware Architecture



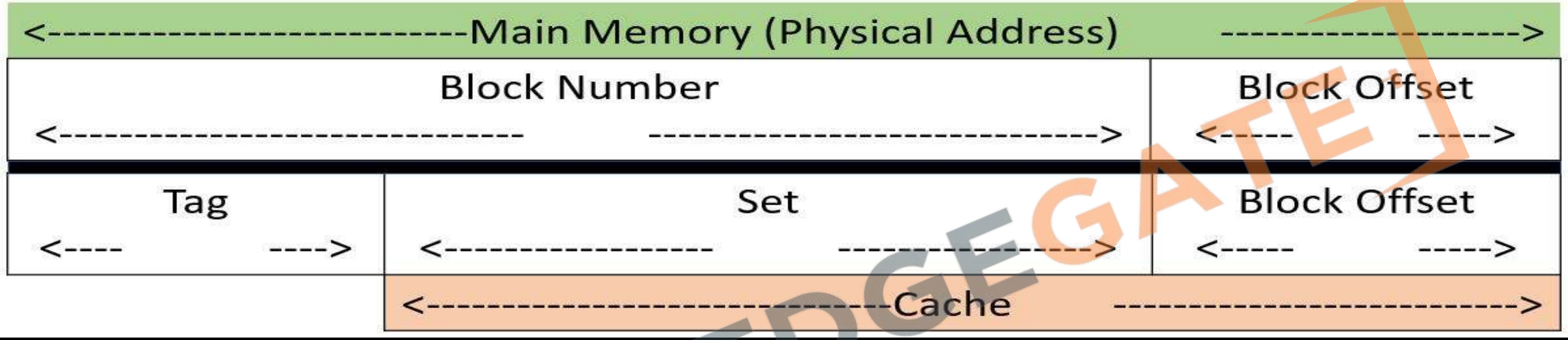
KNOWLEDGEGATE

Q Consider the main memory size is of 128 KB, the cache size is of 16 KB, the block size is of 256 B, the set size is 2. Find Tag.



KNOWLEDGEGATE

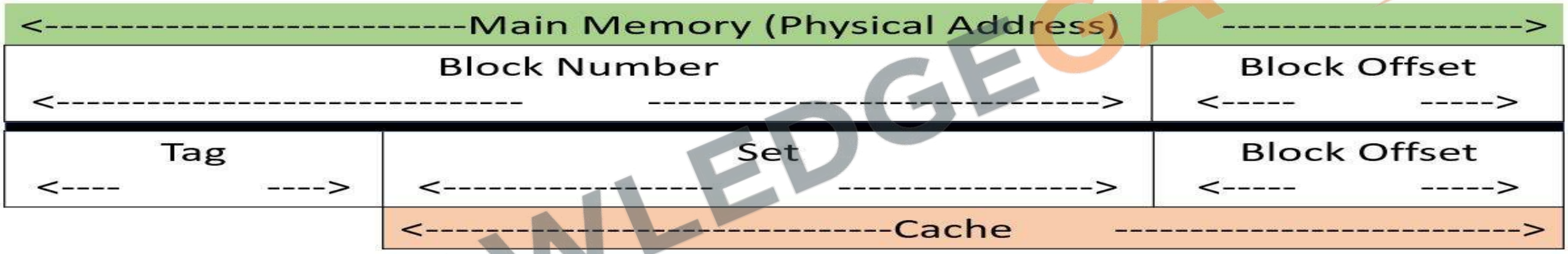
Q Main Memory = 32 GB, Cache Size = 32 KB, Block Size = 1 KB and it is a 4-way set associative cache?



KNOWLEDGEGATE

www.knowledgegate.in

MM Size	Cache Size	Block Size	No of bits in Tag	Tag Directory Size	Set Associative
128 KB	16 KB	256 B			2-way
32 GB	32 KB	1 KB			4-way
	512 KB	1 KB	7		8-way
16 GB		4 KB	10		4-way
64MB			10		4-way
	512 KB		7		8-way



KNOWLEDGEGATE
www.knowledgegate.in

MM Size	Cache Size	Block Size	No of bits in Tag	Tag Directory Size	Set Associative
128 KB	16 KB	256 B	4	$4 * 2^6$	2-way
32 GB	32 KB	1 KB	22	22 * 32	4-way
2^{23} B	512 KB	1 KB	7	$7 * 2^9$	8-way
16 GB	2^{26} B	4 KB	10	$10 * 2^{14}$	4-way
64MB	?	?	10	?	4-way
?	512 KB	?	7	?	8-way

- The choice of cache mapping algorithm depends on several factors, including the size of the cache, the size of the main memory, and the type of data being stored. The most common cache mapping algorithm used in practice is set-associative mapping, as it provides a good balance between the flexibility of associative mapping and the simplicity of direct mapping.
- In general, the cache mapping algorithm plays an important role in determining the performance of the cache memory, as it affects the number of cache hits and misses and the speed at which data can be retrieved from the cache.

KNOWLEDGEGATE

www.knowledgegate.in

Cache Replacement Policies

- In direct mapped cache, the position of each block is predetermined hence no replacement policy exists.
- In fully associative and set associative caches there exists policies.
- When a new block is brought into the cache and all the positions that it may occupy are full, then the controller needs to decide which of the old blocks it can overwrite.

KNOWLEDGEGATE

www.knowledgegate.in

FIFO Policy

- The block which have entered first in the memory will be replaced first.
- This can lead to a problem known as "**Belady's Anomaly**", it states that if we increase the number of lines in cache memory the cache miss will increase.

Example: Let the blocks be in the sequence: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 and the cache memory has 4 lines.

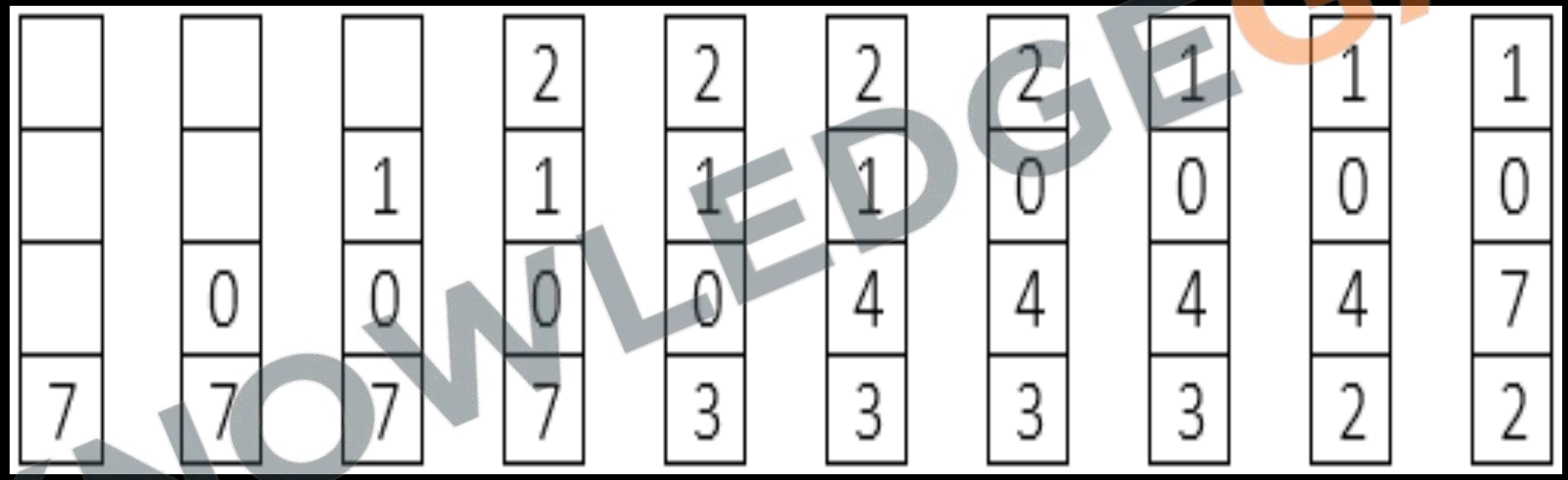
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

KNOWLEDGEGATE

FIFO Policy

- The block which have entered first in the memory will be replaced first.
- This can lead to a problem known as "**Belady's Anomaly**", it states that if we increase the number of lines in cache memory the cache miss will increase.

Example: Let the blocks be in the sequence: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 and the cache memory has 4 lines.

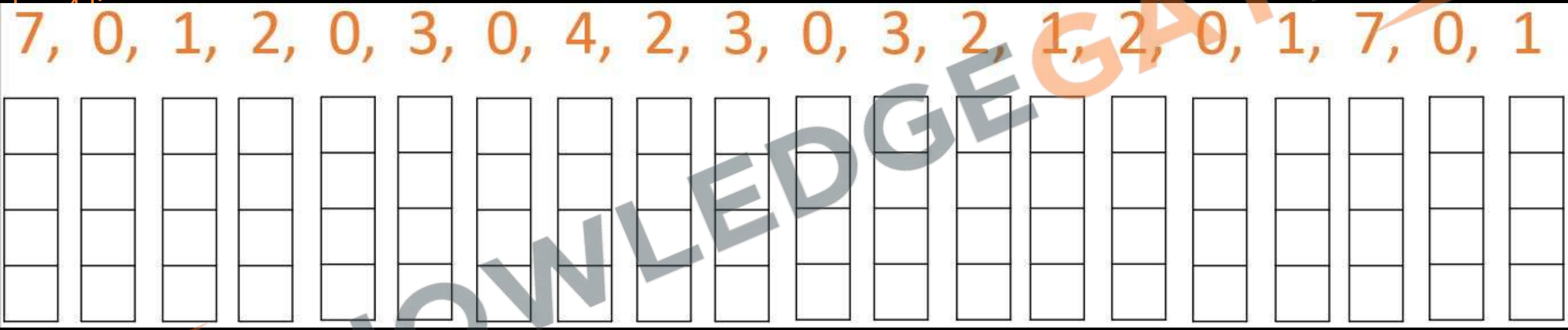


KNOWLEDGEGATE
www.knowledgegate.in

Optimal Algorithm

- The page which will not be used for the longest period of time in future references will be replaced first.
- The optimal algorithm will provide the best performance but it is difficult to implement as it requires the future knowledge of pages which is not possible.
- It is used as a benchmark for cache replacement algorithms.

Example: Let the blocks be in the sequence: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 and the cache memory

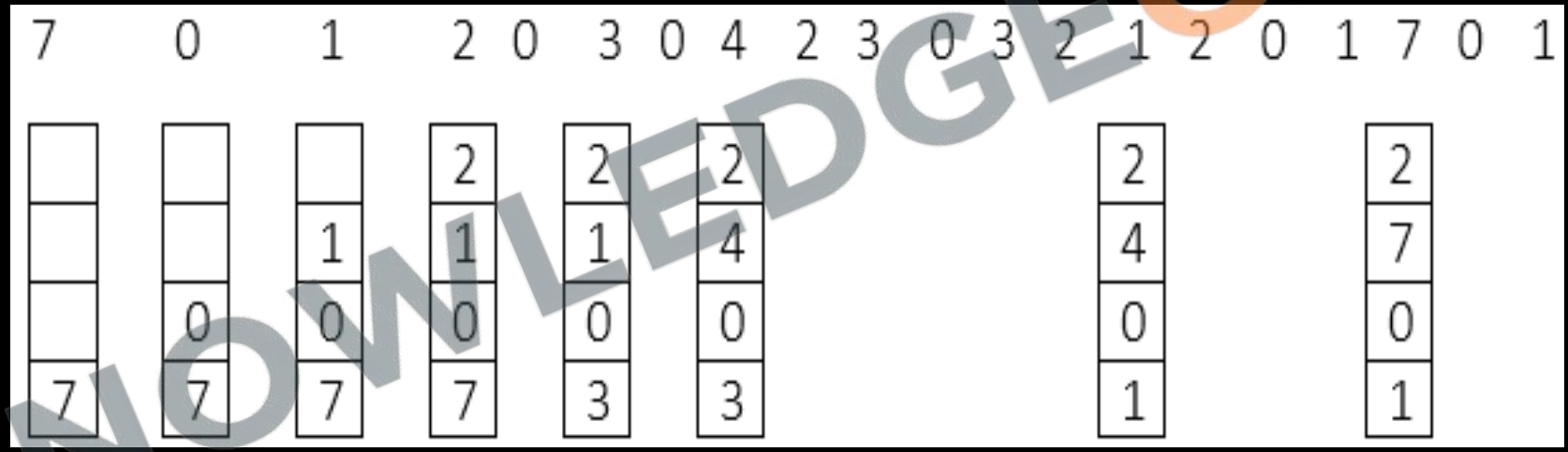


KNOWLEDGEGATE

Optimal Algorithm

- The page which will not be used for the longest period of time in future references will be replaced first.
- The optimal algorithm will provide the best performance but it is difficult to implement as it requires the future knowledge of pages which is not possible.
- It is used as a benchmark for cache replacement algorithms.

Example: Let the blocks be in the sequence: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 and the cache memory has 4 lines.

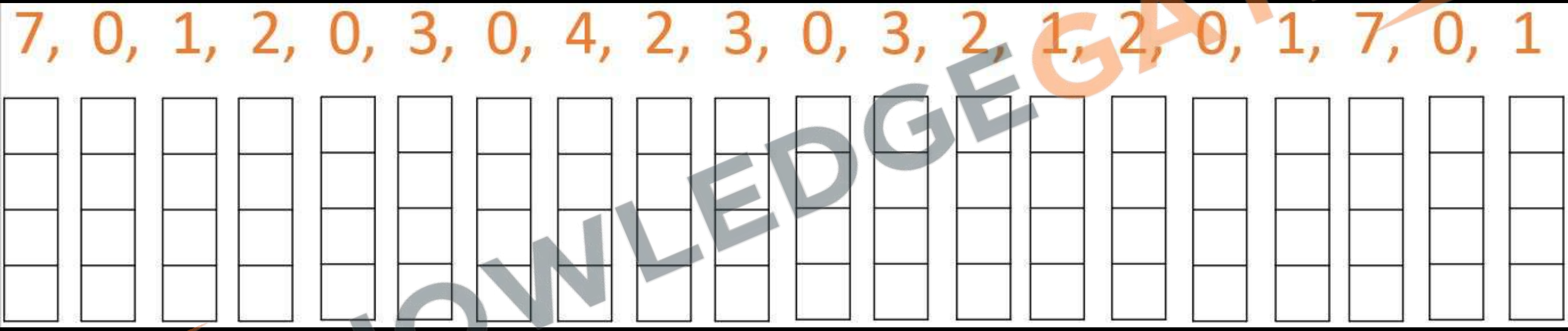


KNOWLEDGEGATE

LRU (Least Recently Used)

- The page which was not used for the longest period of time in the past will get replaced first.

Example: Let the blocks be in the sequence: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 and the cache memory has 4 lines.



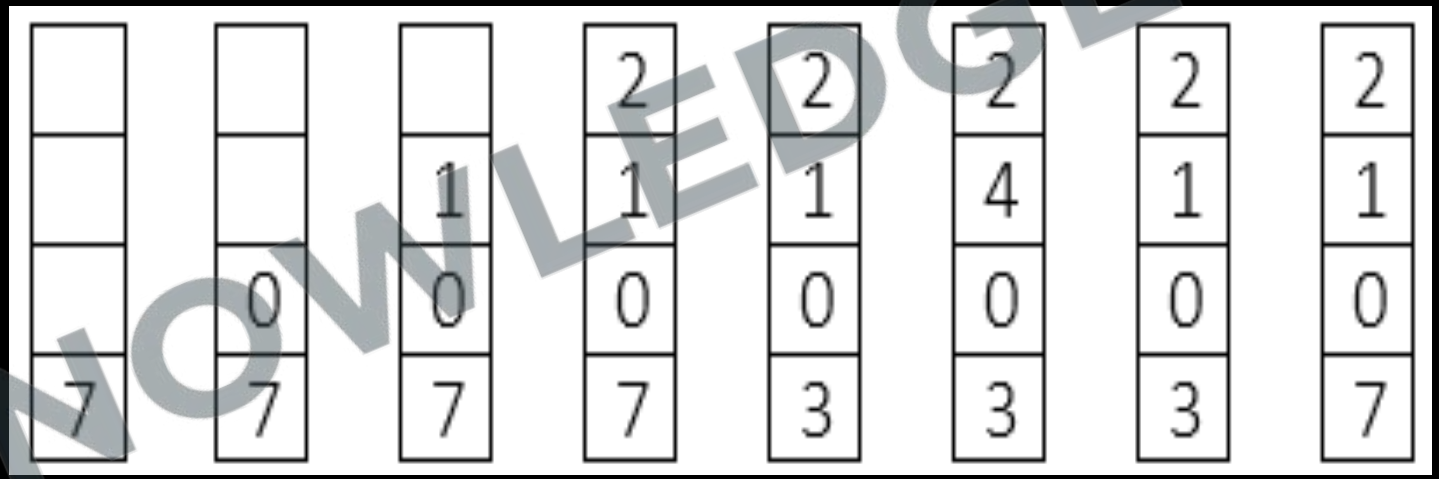
KNOWLEDGEGATE

www.knowledgegate.in

LRU (Least Recently Used)

- The page which was not used for the longest period of time in the past will get replaced first.

Example: Let the blocks be in the sequence: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 and the cache memory has 4 lines.

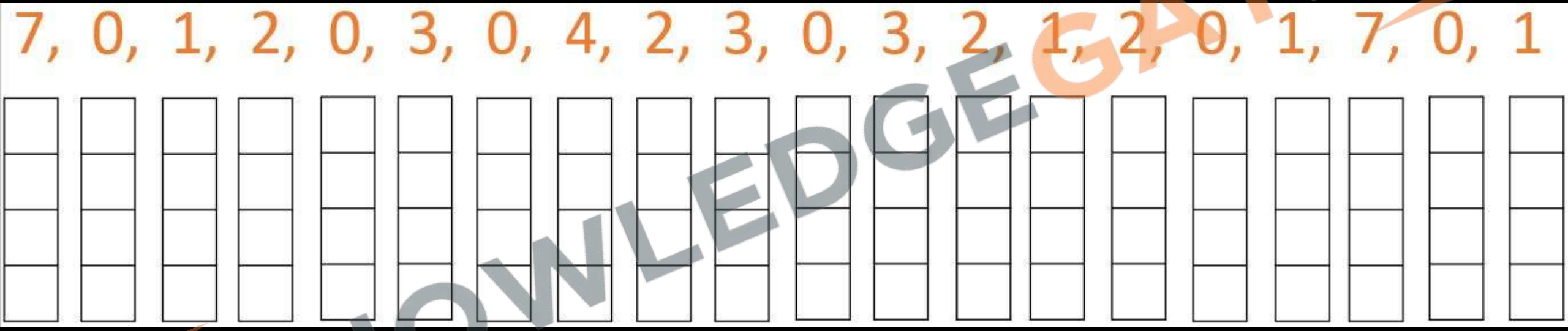


KNOWLEDGEGATE

Most Recently Used (MRU)

- The page which was used recently will be replaced first.

Example: Let the blocks be in the sequence: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 and the cache memory has 4 lines.

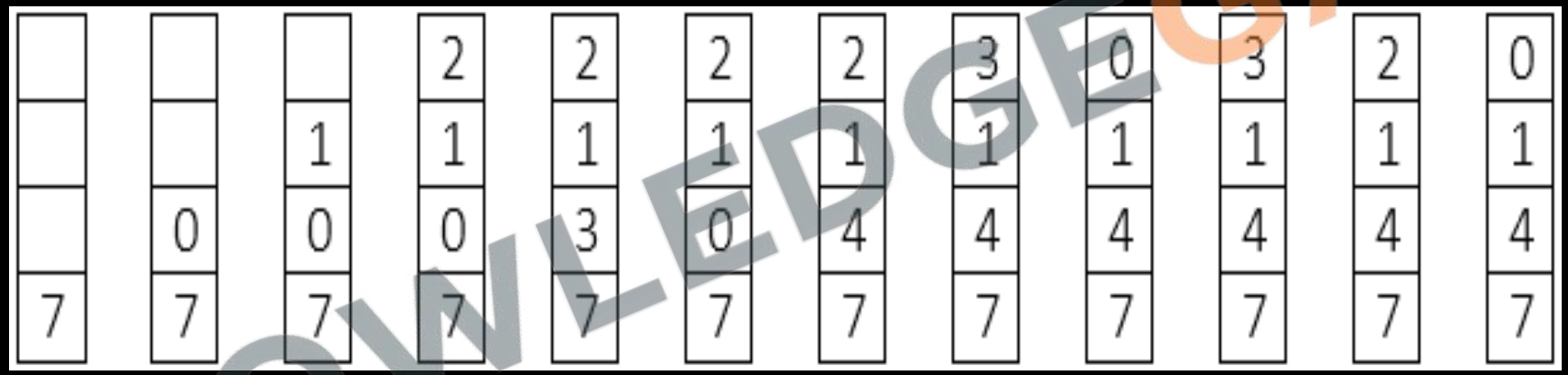


KNOWLEDGEGATE
www.knowledgegate.in

Most Recently Used (MRU)

- The page which was used recently will be replaced first.

Example: Let the blocks be in the sequence: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 and the cache memory has 4 lines.



KNOWLEDGE GATE

www.knowledgegate.in

Types of Miss

- **Compulsory Miss**
  - When CPU demands for any block for the first time then definitely a miss is going to occur as the block needs to be brought into the cache, it is known as Compulsory miss.

- **Capacity Miss**
  - Occur because blocks are being discarded from cache because cache cannot contain all blocks needed for program execution.

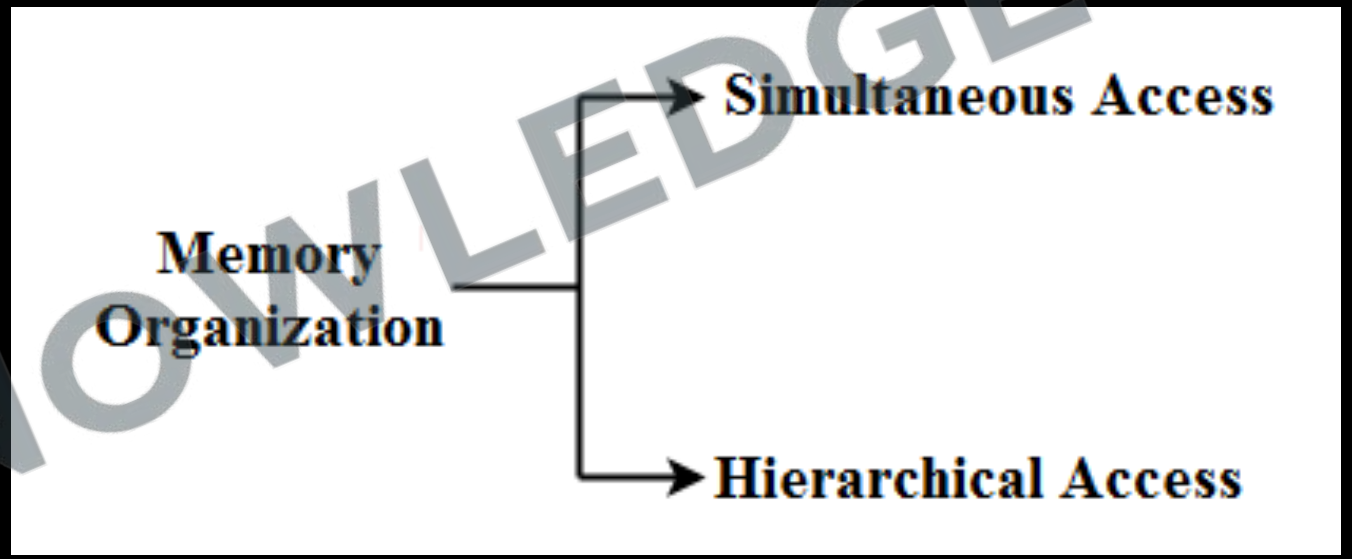
- **Conflict Miss**
  - In the case of set associative or direct mapped block placement strategies, conflict misses occur when several blocks are mapped to the same set or block frame; also called collision misses or interference misses.

KNOWLEDGEGATE

www.knowledgegate.in

Memory Organization

- Memory is organized at different levels.
- CPU may try to access different levels of memory in different ways.
- On this basis, the memory organization is broadly divided into two types



KNOWLEDGEGATE

www.knowledgegate.in

Hierarchical Access Memory Organization

- In this memory organization, memory levels are organized as
  - Level-1 is directly connected to the CPU.
  - Level-2 is directly connected to level-1.
  - Level-3 is directly connected to level-2 and so on

- Whenever CPU requires any word,
  - It first searches for the word in level-1.
  - If the required word is not found in level-1, it searches for the word in level-2.
  - If the required word is not found in level-2, it searches for the word in level-3 and so on.

KNOWLEDGEGATE

T_1 = Access time of level L_1	T_2 = Access time of level L_2	T_3 = Access time of level L_3
S_1 = Size of level L_1	S_2 = Size of level L_2	S_3 = Size of level L_3
C_1 = Cost per byte of level L_1	C_2 = Cost per byte of level L_2	C_3 = Cost per byte of level L_3
H_1 = Hit rate of level L_1	H_2 = Hit rate of level L_2	H_3 = Hit rate of level L_3

Effective Memory Access Time

Average time required to access memory per operation =

$$H_1 * T_1 + (1 - H_1) * H_2 * (T_1 + T_2) + (1 - H_1)(1 - H_2) * H_3 * (T_1 + T_2 + T_3)$$

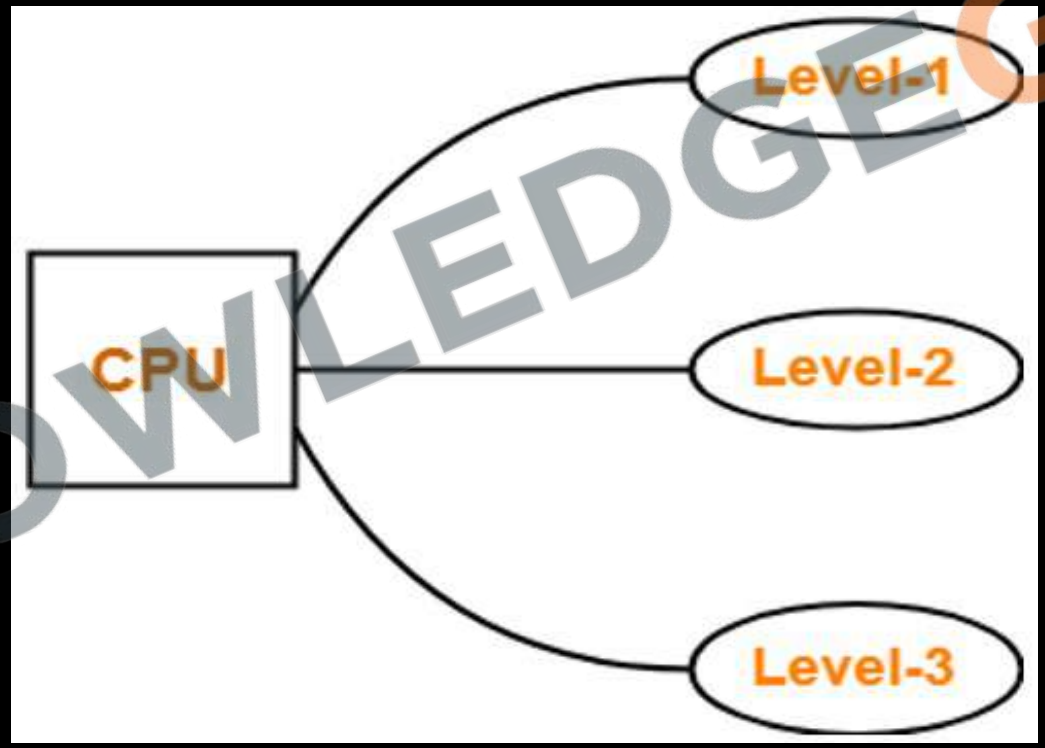
$$H_1 * T_1 + (1 - H_1) * H_2 * (T_1 + T_2) + (1 - H_1)(1 - H_2) * (T_1 + T_2 + T_3)$$

KNOWLEDGEGATE

www.knowledgegate.in

Simultaneous Access Memory Organization

- In simultaneous access all the levels of memory are directly connected to the CPU, whenever CPU requires any word, it starts searching for it in all the levels simultaneously.



KNOWLEDGEGATE
www.knowledgegate.in

- T_1 = Access time of level L_1
- S_1 = Size of level L_1
- C_1 = Cost per byte of level L_1
- H_1 = Hit rate of level L_1

- T_2 = Access time of level L_2
- S_2 = Size of level L_2
- C_2 = Cost per byte of level L_2
- H_2 = Hit rate of level L_2

- T_3 = Access time of level L_3
- S_3 = Size of level L_3
- C_3 = Cost per byte of level L_3
- H_3 = Hit rate of level L_3

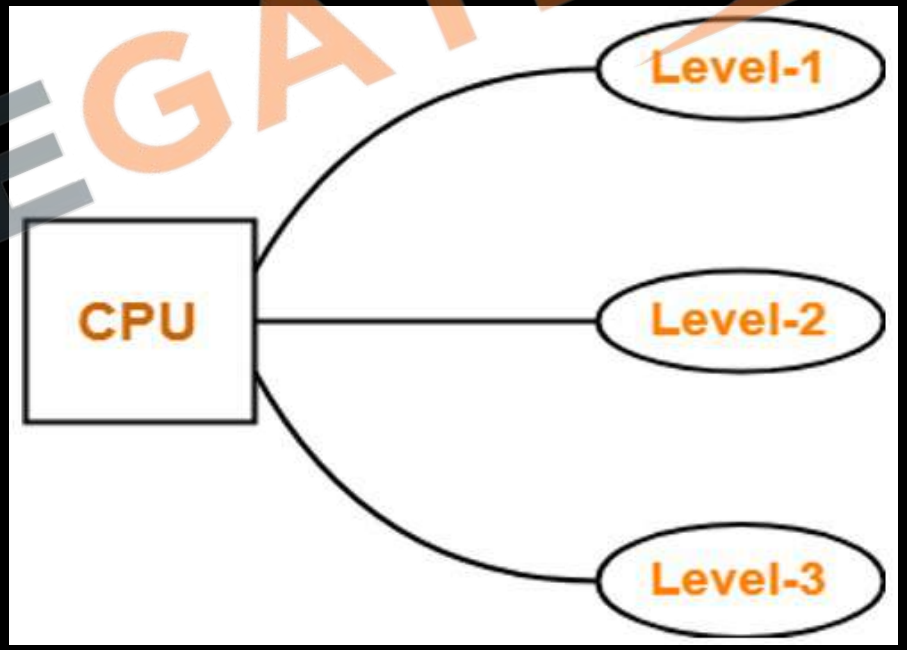
Effective Memory Access Time (EMAT) =

$$H_1 * T_1 + (1 - H_1) * H_2 * T_2 + (1 - H_1) (1 – H_2) * H_3 * T_3$$

In any memory organization, the data item being searched will definitely be present in the last level (or secondary memory).

Thus, hit rate for the last level is always 1. So,

$$H_1 * T_1 + (1 - H_1) * H_2 * T_2 + (1 - H_1) (1 – H_2) * T_3$$



[KNOWLEDGEGATE]

Example: Calculate the EMAT for a machine with a cache hit rate of 80% where cache access time is 5ns and main memory access time is 100ns, both for simultaneous and hierarchical access.

KNOWLEDGEGATE

www.knowledgegate.in

Cache Coherence Problem

- If multiple copy of same data is maintained at different level of memories then inconsistency may occur, this problem is known as cache coherence problem.

- Cache coherence problem can be resolved using the following techniques:
  - Write Through
  - Write Back

KNOWLEDGEGATE

Write Through

- Write through is used to maintain the consistency between the cache and main memory.
- According to it if the cache copy is updated, at the same time main memory is also updated.

- **Advantages**
  - It provides the highest level of consistency.

- **Disadvantages**
  - It requires more number of memory access.

KNOWLEDGEGATE

www.knowledgegate.in

Write Back

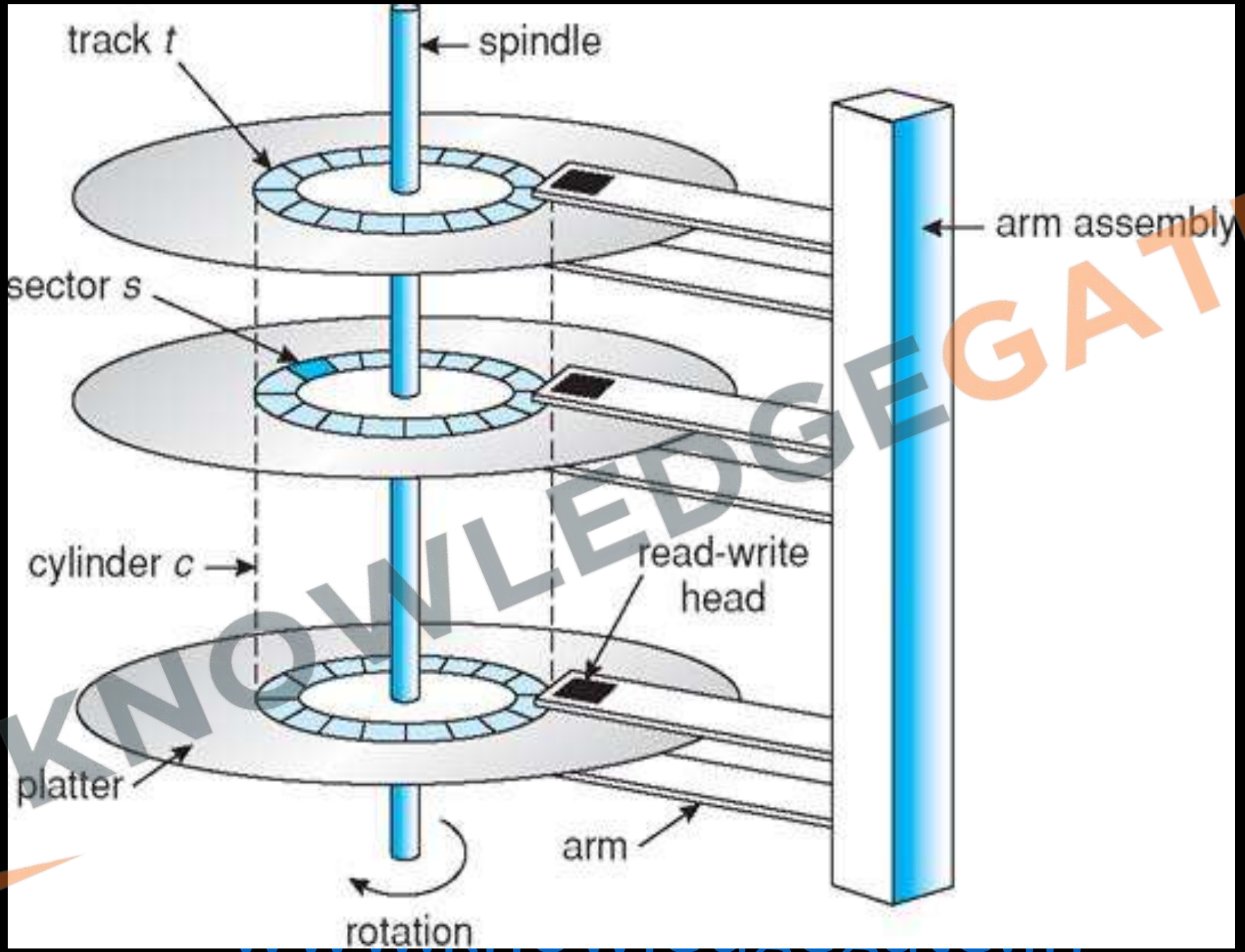
- Write back is also used to maintain the consistency between the cache and main memory.
- According to it all the changes performed on cache are reflected back to the main memory in the end.

- **Advantage**
  - Less number of memory accesses and less write operations.

- **Disadvantage**
  - Inconsistency may occur.

KNOWLEDGEGATE

www.knowledgegate.in



KNOWLEDGEGATE

KNOWLEDGEGATE

www.knowledgegat

Total Transfer Time = Seek Time + Rotational Latency + Transfer Time

Seek Time: - It is a time taken by Read/Write header to reach the correct track. (Always given in question)

Rotational Latency: - It is the time taken by read/Write header during the wait for the correct sector. In general, it's a random value, so far average analysis, we consider the time taken by disk to complete half rotation.

Transfer Time: - it is the time taken by read/write header either to read or write on a disk. In general, we assume that in 1 complete rotation, header can read/write the either track, so
total time will be = (File Size/Track Size) *time taken to complete one revolution.

KNOWLEDGE GATE

Q Consider a disk pack with 16 surfaces, 128 tracks per surface and 256 sectors per track. 512 bytes of data are stored in a bit serial manner in a sector. The capacity of the disk pack and the number of bits required to specify a particular sector in the disk are respectively: **(GATE-2007) (1 Marks)**

(A) 256 Mbyte, 19 bits

(B) 256 Mbyte, 28 bits

(C) 512 Mbyte, 20 bits

(D) 64 Gbyte, 28 bit

KNOWLEDGEGATE

www.knowledgegate.in

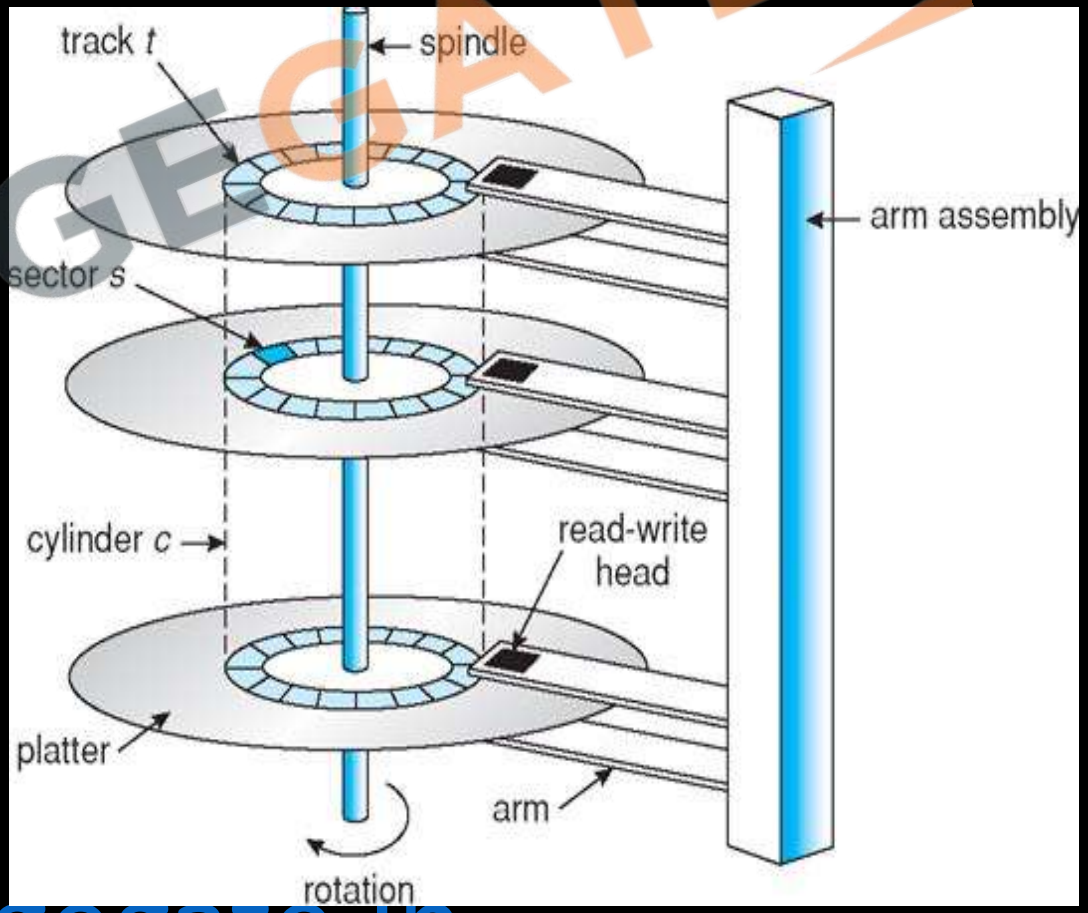
Q consider a disk where each sector contains 512 bytes and there are 400 sectors per track and 1000 tracks on the disk. If disk is rotating at speed of 1500 RPM, find the total time required to transfer file of size 1 MB. Suppose seek time is 4ms?

KNOWLEDGEGATE

www.knowledgegate.in

Q A hard disk has 63 sectors per track, 10 platters each with 2 recording surfaces and 1000 cylinders. The address of a sector is given as a triple (c, h, s), where c is the cylinder number, h is the surface number and s is the sector number. Thus, the 0th sector is addressed as (0, 0, 0), the 1st sector as (0, 0, 1), and so on The address <400,16,29> corresponds to sector number:

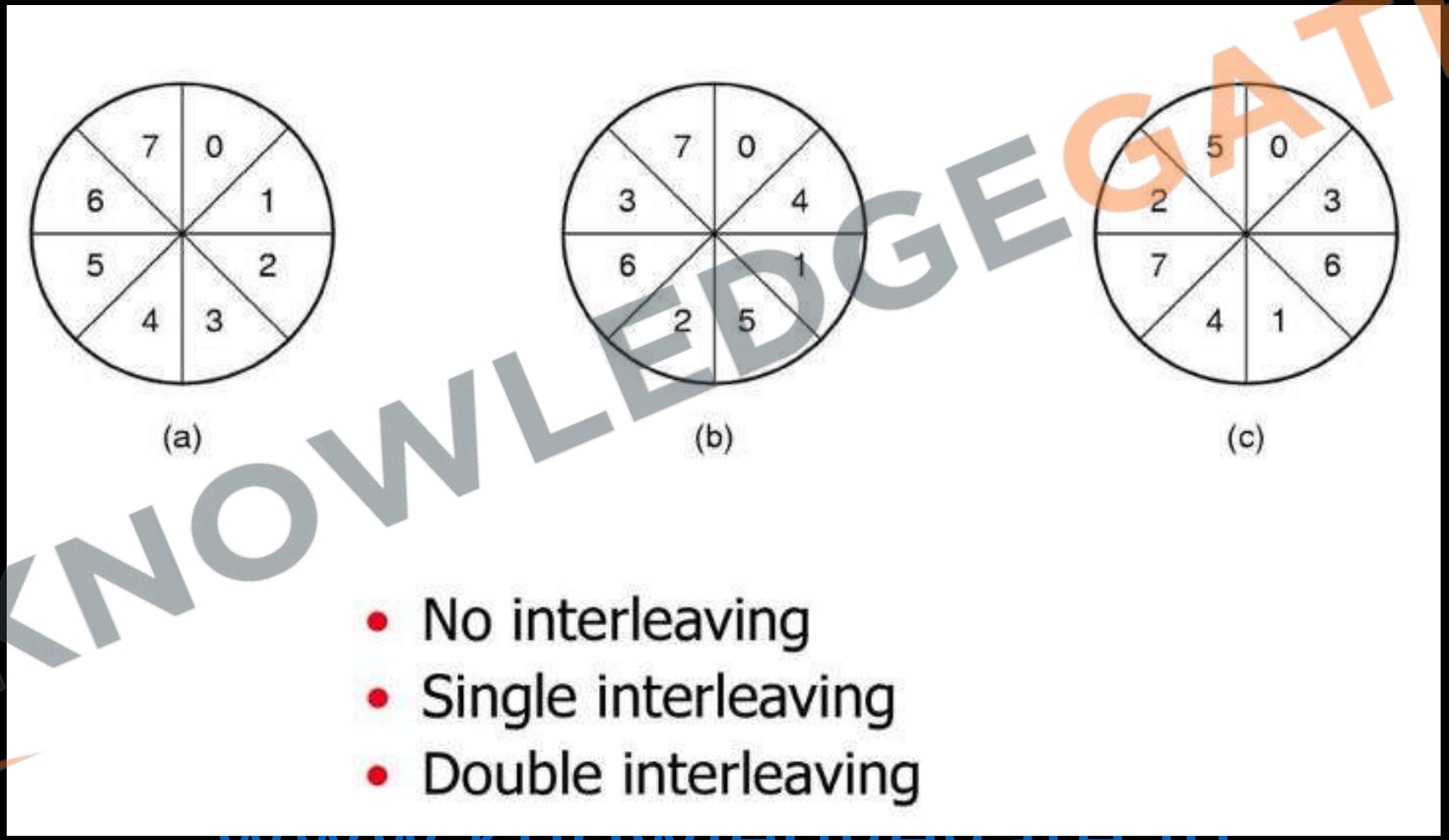
(A) 505035 **(B)** 505036 **(C)** 505037 **(D)** 505038



KNOWLEDGEGATE
www.knowledgegate.in

- **Single interleaving: -** in 2 rotation we read 1 track

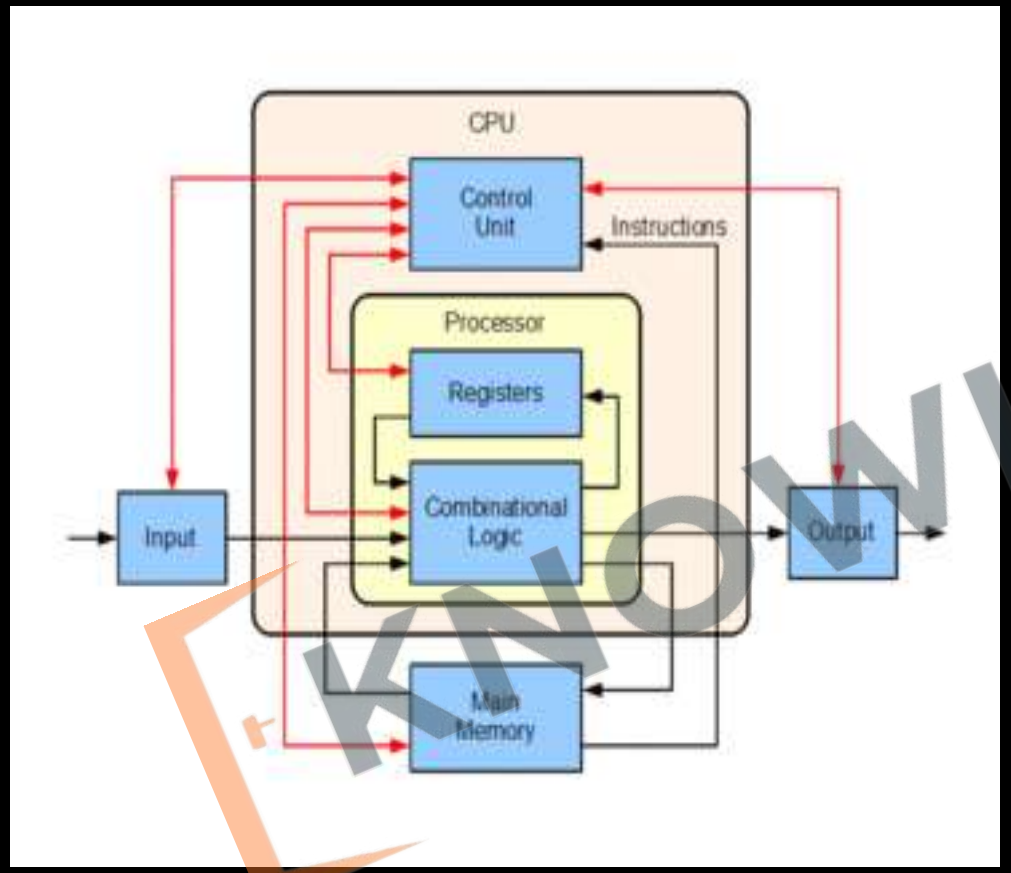
- **Double interleaving: -** in 2.75 rotation we read 1 track



KNOWLEDGEGATE
www.knowledgegate.in

Input / Output management

- In general, when we say a computer, we understand a CPU + Memory (cache, main memory).
- But a computer does not serve any purpose if it cannot receive data from the outside world or cannot transmit the data to outside world.



KNOWLEDGEGATE

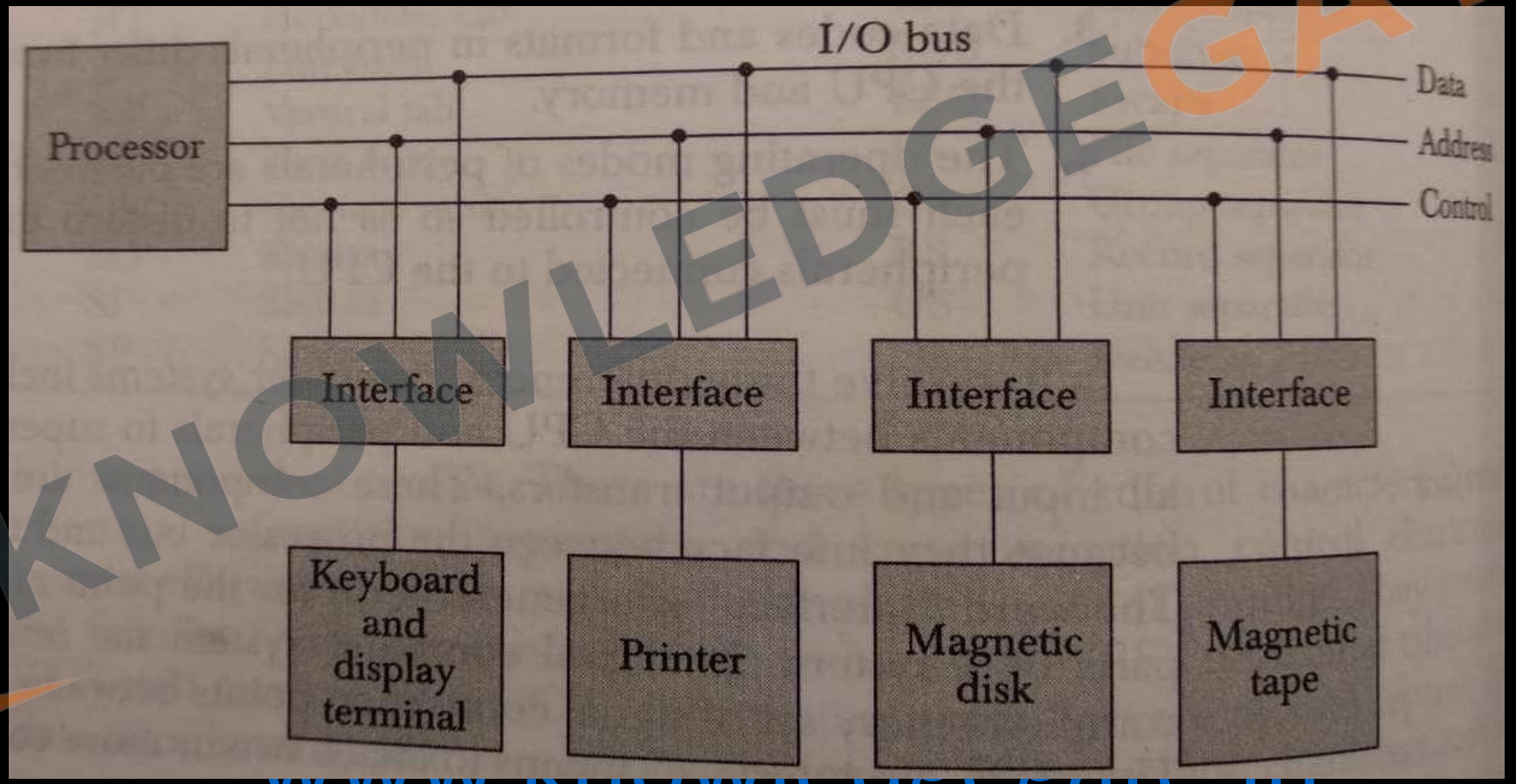
- I/o or peripheral devices are those independent devices which serve this purpose.
- So, while designing i/o for a computer we must know the number of i/o device and the capacity of each device.

KNOWLEDGE GATE
www.knowledgegate.in

Interface

we cannot directly connect a i/o device to computer because of the following reasons

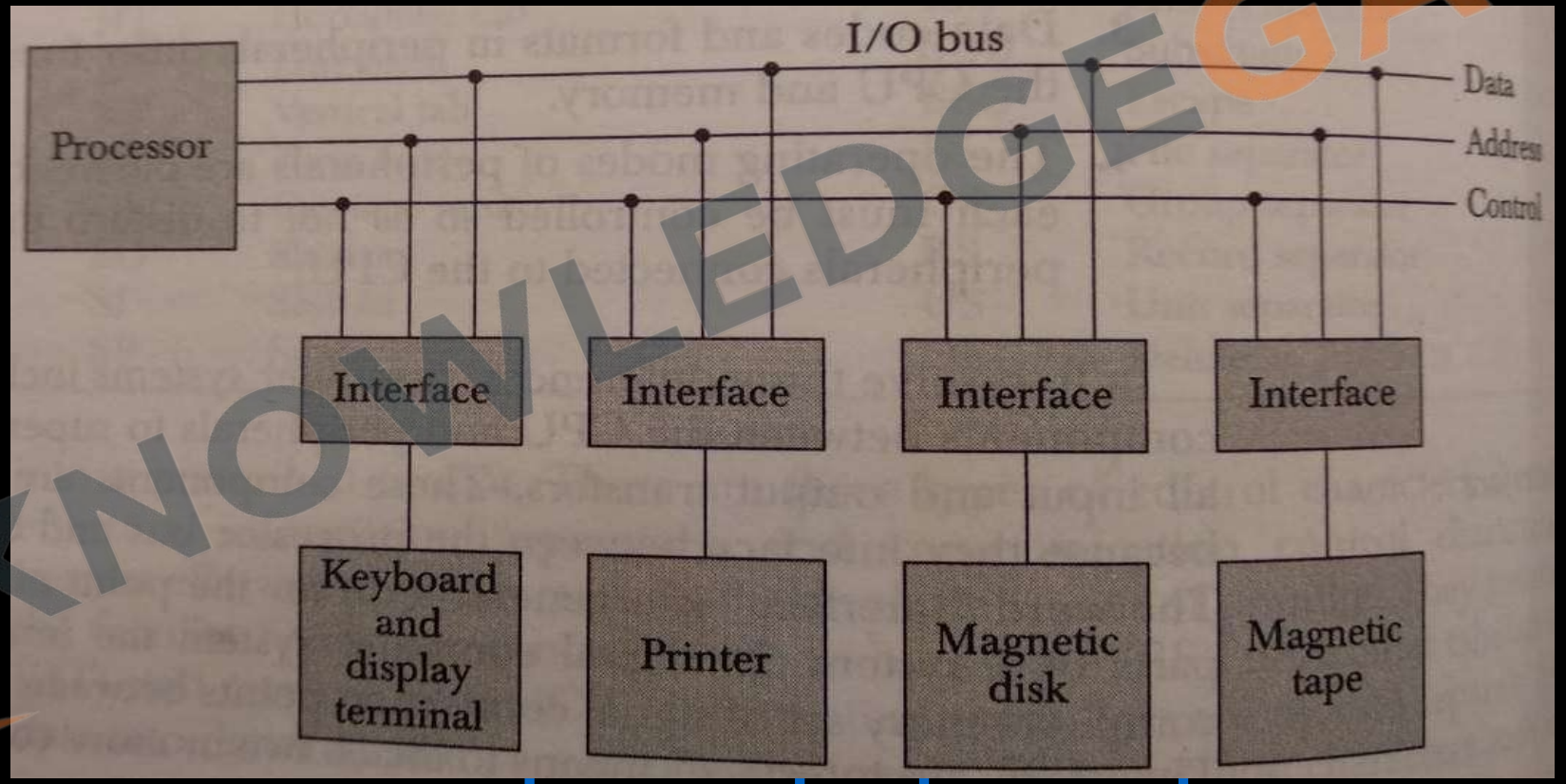
- **Speed: -** The speed of CPU and i/o device will usually different.
- **Format: -** The data code and format of CPU and peripherals may be different. E.g. ASCII, Unicode etc.



KNOWLEDGEGATE
www.knowledgegate.in

- **Physical orientation: -** Different device have organizations like optical, magnetic, electrochemical and different controlling functions.

- **Signal conversion: -** peripherals are electromagnetic and electrochemical device and their manner of operation is different from the operations of CPU and memory, which are electronic device, signal conversion is required.



KNOWLEDGEGATE
www.knowledgegate.in

- **I/O ports,** 1/0 ports, Interrupts: interrupt hardware, types of interrupts and exceptions

KNOWLEDGEGATE

www.knowledgegate.in

Interrupt

- It's a signal from a hardware device (like a keyboard or mouse) to get the processor's attention.
- **Polled Interrupt:**
  - Polled interrupt is like a "raising your hand" system in a classroom. In this method, the processor regularly checks each device one by one to see if it needs attention. It's like a teacher asking each student individually if they have a question. This can be slow because the processor has to keep asking every device, "Do you need anything?"
-
- **Vectored Interrupt:**
  - Vectored interrupt is more efficient, like a student coming directly to the teacher with a question. In this method, when a device sends an interrupt, it also sends a special code (vector) to the processor. The code tells the processor exactly what the device needs. So, the processor doesn't waste time checking all devices; it goes directly to the one that asked for help.

KNOWLEDGEGATE

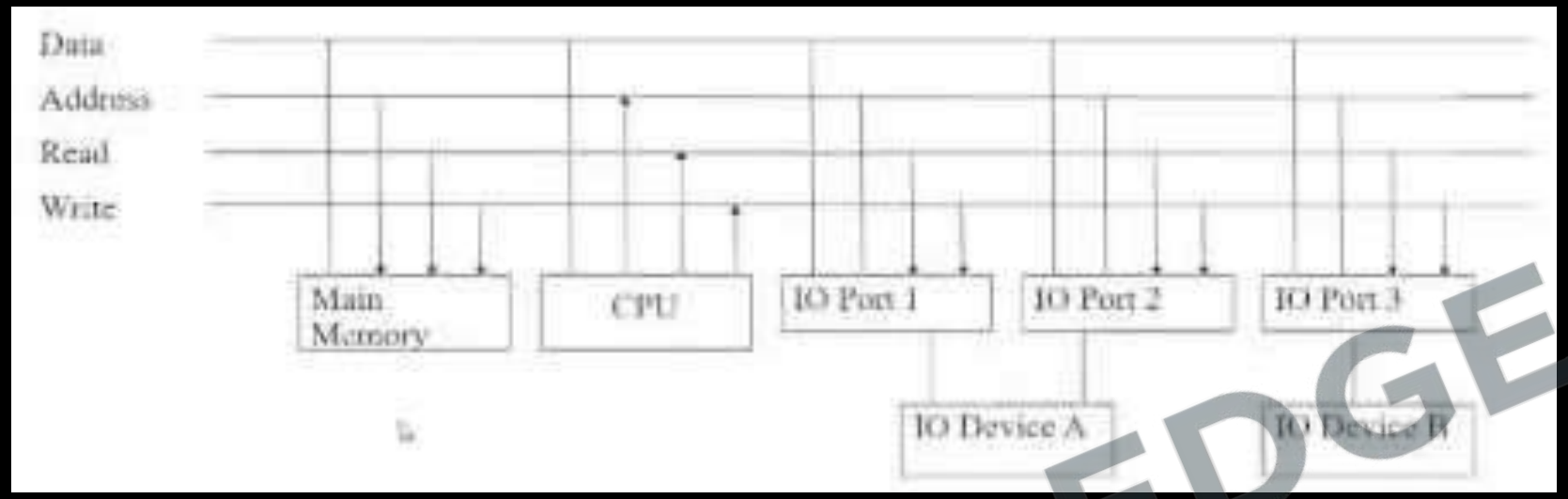
- each of the steps related to interrupt handling as single bullet points:
- **Interrupt Recognition**:
  - Processor identifies the interrupt, either from an external request or an internal mechanism.
  - Determines which device or CPU component issued the request.
- **Status Saving**:
  - Processor saves important information like flags and registers that might change during the interrupt.
  - Ensures the system can return to its previous state after handling the interrupt.
- **Interrupt Masking**:
  - Initially, all interrupts are blocked to prioritize the current interrupt.
  - Later, the processor allows higher-priority interrupts to be processed.
- **Interrupt Acknowledgment**:
  - Processor acknowledges the specific interrupt being serviced.
  - Allows the interrupting device to continue its task.
  - This acknowledgment can occur through an external signal line.
- **Interrupt Service Routine**:
  - Processor starts executing a specific routine designed to handle the interrupt.
  - The routine's address can be determined in various ways depending on the computer's architecture.
  - It's like following a set of instructions to deal with the interrupt.
- **Restoration and Return**:
  - After the interrupt service routine is completed, the processor restores all saved registers and flags to their initial state.
  - Ensures the system returns to its state before the interrupt occurred.

KNOWLEDGEGATE

www.knowledgegate.in

How a computer (CPU) deals with Memory and I/O devices

Memory Mapped i/o



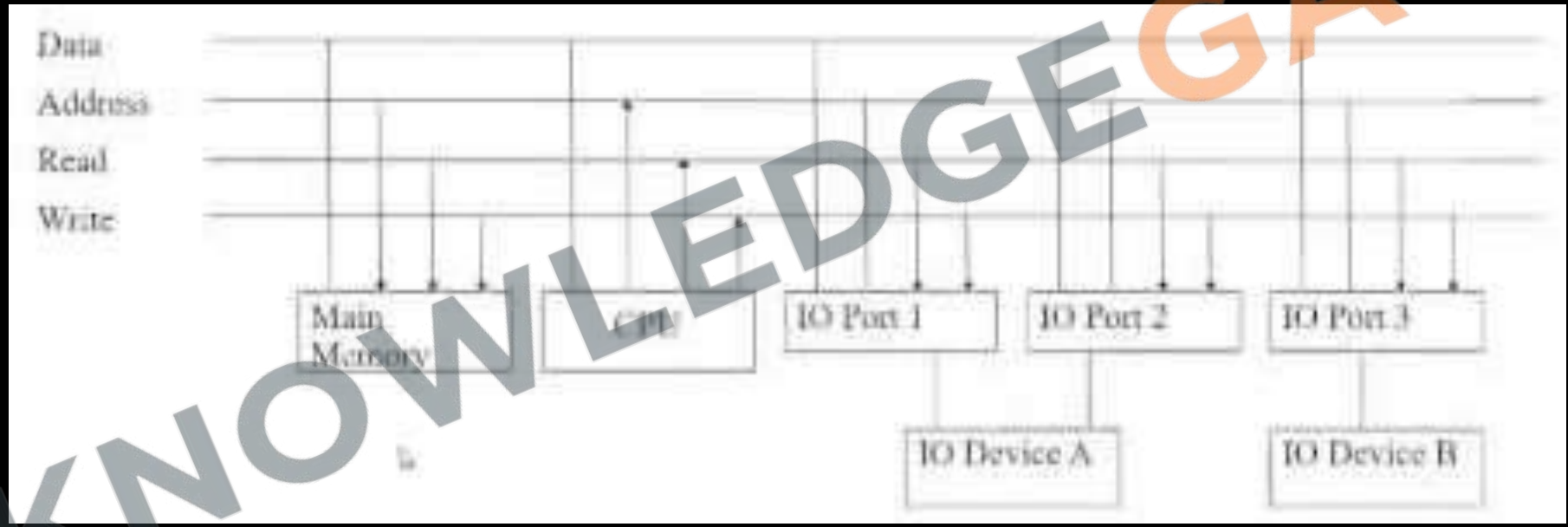
8085

- Here there is no separate i/o instructions. The CPU can manipulate i/o data residing in the interface registers with the same instructions that are used to manipulate memory words.
- Here computer can use memory type instructions for i/o data.
- E.g. 8085

KNOWLEDGEGATE

- **Advantage: -** In typical computer there are more memory reference instruction than i/o instruction, but in memory mapped i/o all instructions that refer to memory are also available for i/o.

- **Disadvantage: -**Total address get divided, some range is occupied by i/o while some memory.



KNOWLEDGEGATE

www.knowledgegate.in

- The **Commodore 64**, also known as the **C64** or the **CBM 64**, is an 8-bit home computer introduced in January 1982 by Commodore International (first shown at the Consumer Electronics Show, in Las Vegas, January 7–10, 1982).

- It has been listed in the Guinness World Records as the highest-selling single computer model of all time, with independent estimates placing the number sold between 10 and 17 million units.

KNOWLEDGE GATE

www.knowledgegate.in

Isolated i/o

- Here common bus to transfer data between memory or i/o and CPU. The distinction between a memory and i/o transfer is made through separate read and write line.
- i/o read and i/o write control lines are enabled during an i/o transfer.
- Memory read and memory write control lines are enabled during a memory transfer. E.g 8086

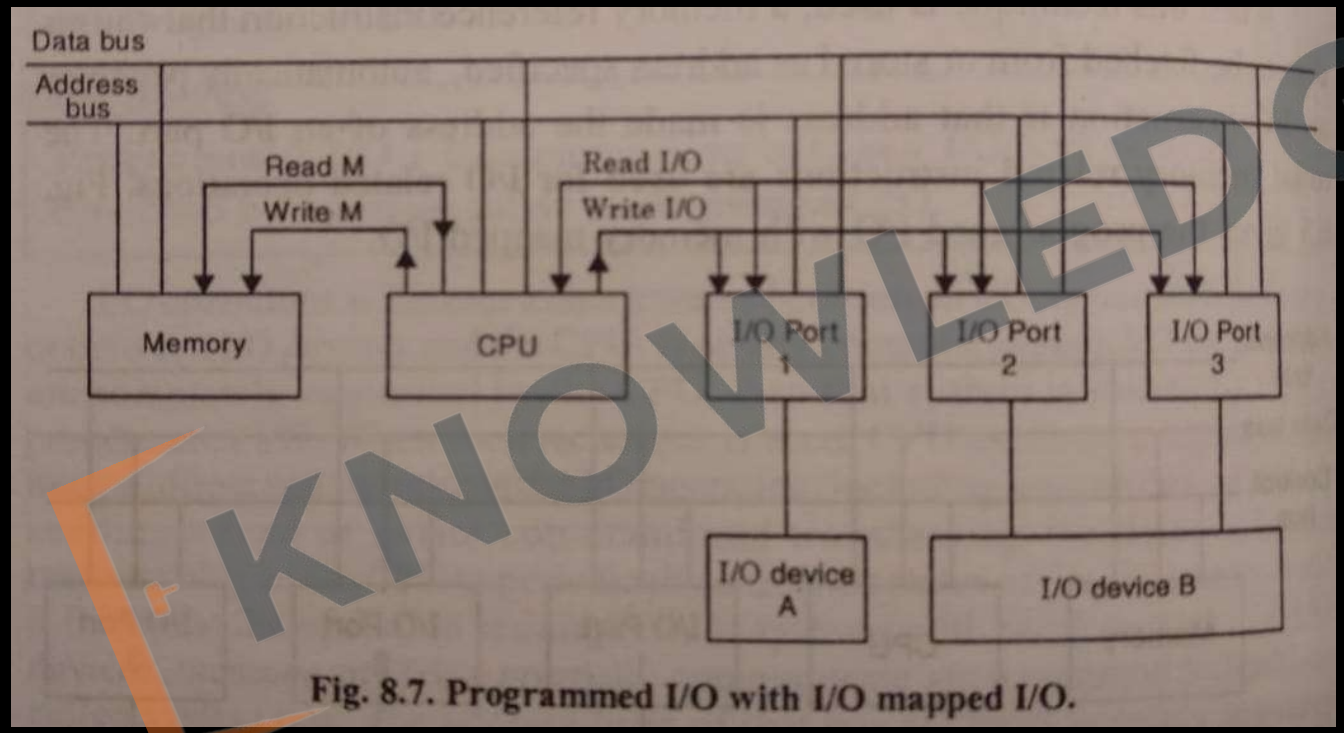
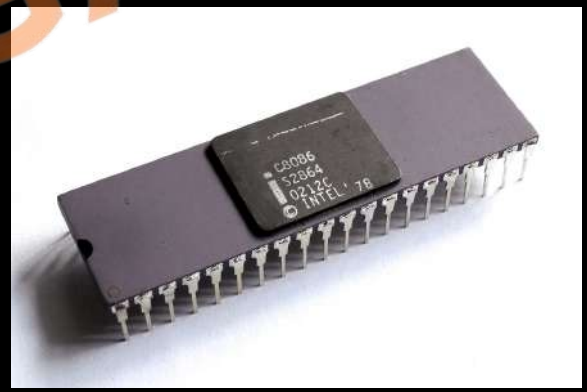


Fig. 8.7. Programmed I/O with I/O mapped I/O.



KNOWLEDGEGATE

- **Advantage: -** Here memory is used efficiently as the same address can be used two times.

- **Disadvantage: -** Need different control lines one for memory and other for i/o devices.

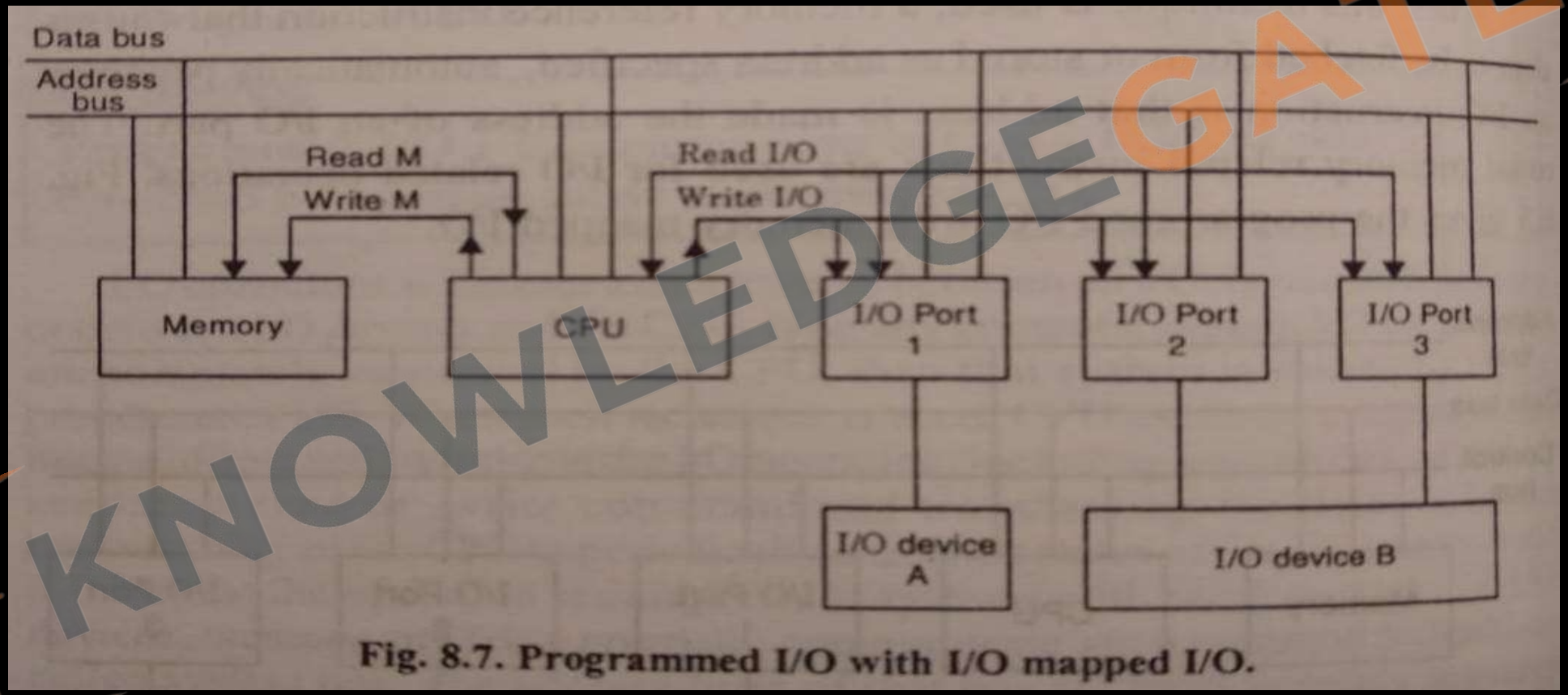


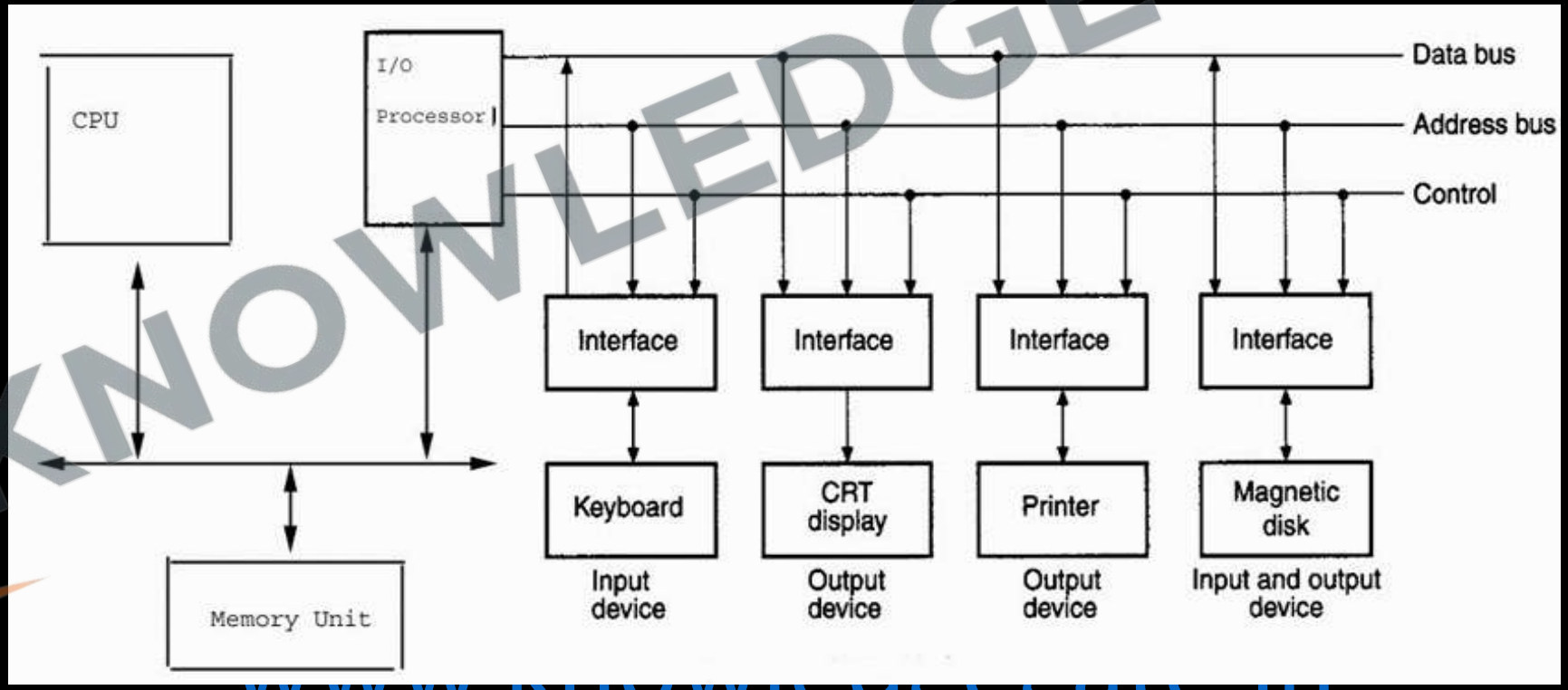
Fig. 8.7. Programmed I/O with I/O mapped I/O.

KNOWLEDGEGATE

www.knowledgegate.in

I/O Processor

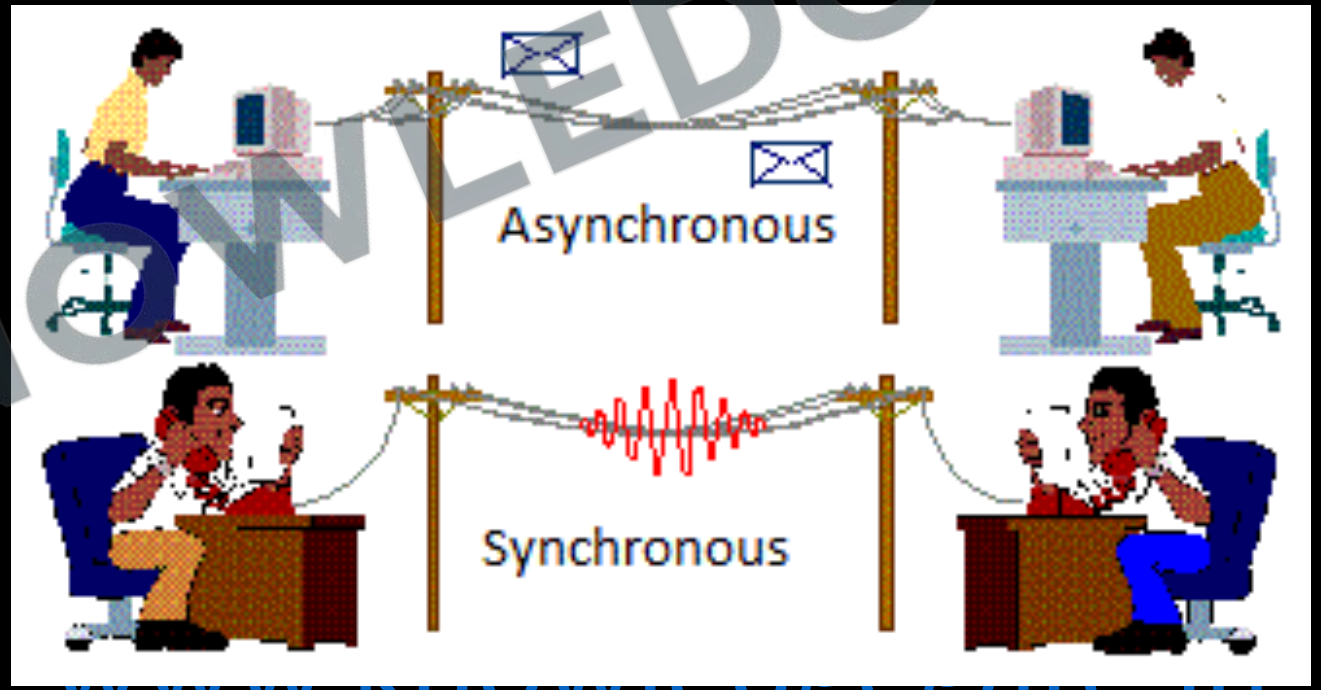
- Computer has independent sets of data, address and control buses, one for accessing memory and the other for i/o. this is done in computers that provide a separate i/o processor other than CPU.
- Memory communicate with both the CPU and iop through a memory bus.
- Iop communicates also with the input and output devices through a separate i/o bus with its own address, data and control lines. The purpose of iop is to provide an independent pathway for the transfer of information between external devices and internal memory.



KNOWLEDGEGATE
www.knowledgegate.in

Synchronous Vs Asynchronous data transfer

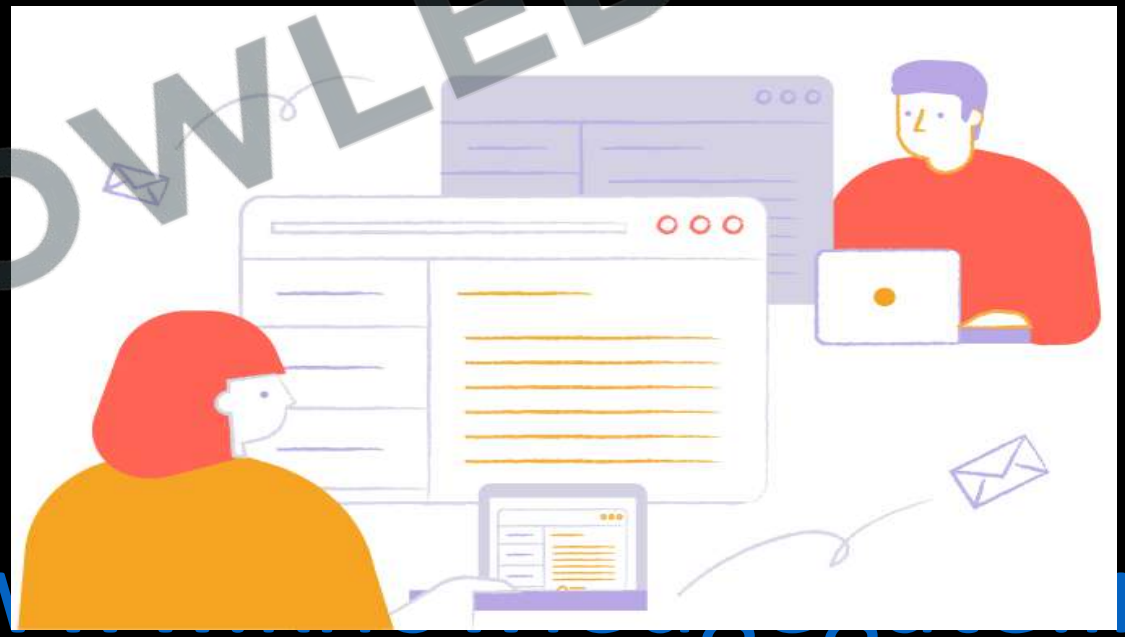
- Synchronization is achieved by a device called master generator, which generate a periodic train of clock pulse.
- The internal operations in a digital system are synchronized by means of clock pulses supplied by a common pulse generator.
- When communication happens between devices which are under a same control unit or same clock then it is called synchronous communication. e.g. communication between CPU and its registers.



KNOWLEDGEGATE
www.knowledgegate.in

Asynchronous communication

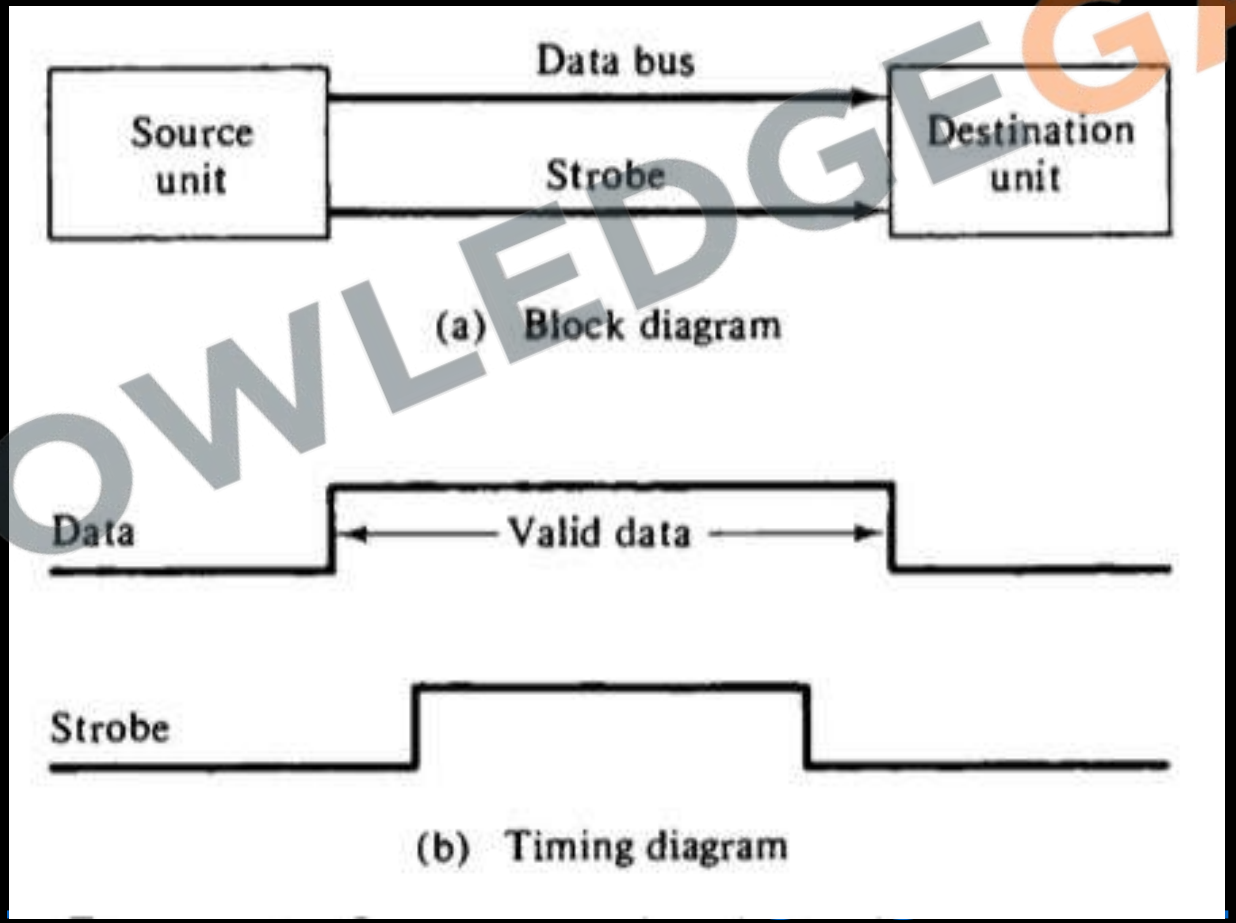
- When the timing units of two devices are independent that is they are under different control then it is called asynchronous communication.
- Asynchronous data transfer between two independent units required that control signals be transmitted between the communicating units to indicate the time at which data is being transmitted.
- One way to achieving this by means of strobe pulse supplied by one of the units to indicate to the other unit which when the transfer has to occur.



KNOWLEDGEGATE

Source Initiated I/O

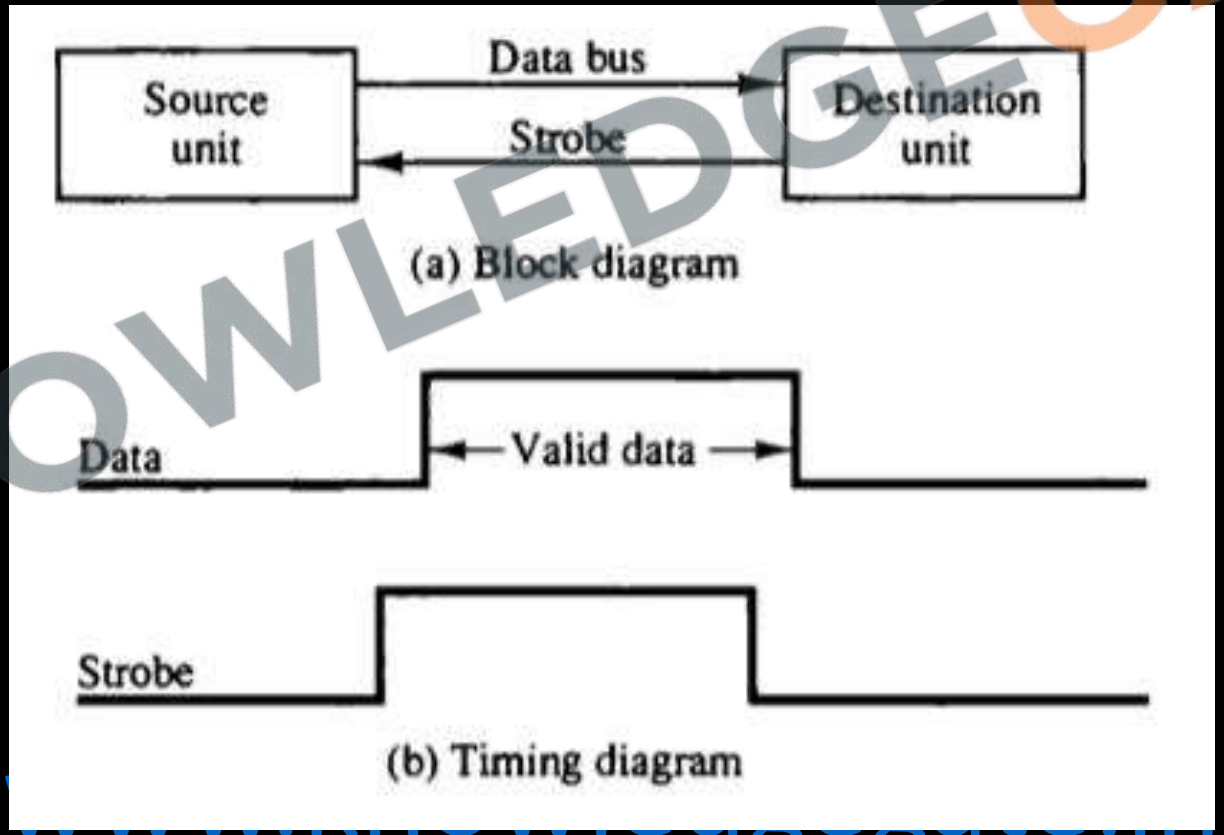
- the unit receiving the data item response with another control signal to acknowledge the receipt of the data. this type of agreement between two independent units is referred to as handshaking.
- The strobe pulse method and the handshaking method of asynchronous data transfer are not restricted to input output transfer. in fact they are used extensively on numerous occasions requiring the transfer of data between two independent units.



KNOWLEDGEGATE

Destination Initiated I/O

- the unit receiving the data item response with another control signal to acknowledge the receipt of the data. this type of agreement between two independent units is referred to as handshaking.
- The strobe pulse method and the handshaking method of asynchronous data transfer are not restricted to input output transfer. in fact they are used extensively on numerous occasions requiring the transfer of data between two independent units.



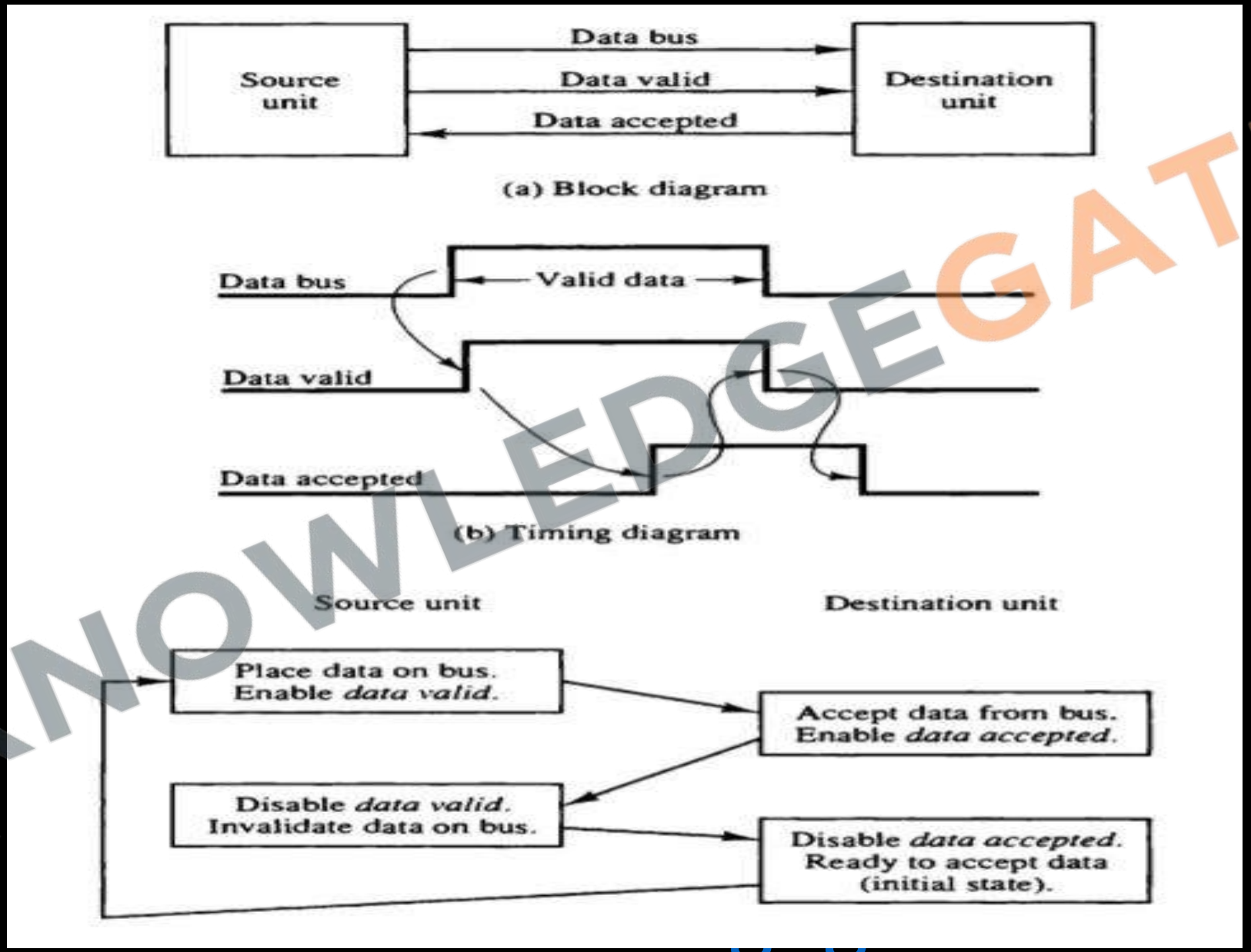
KNOWLEDGE GATE

- the unit receiving the data item response with another control signal to acknowledge the receipt of the data. this type of agreement between two independent units is referred to as handshaking.
- The strobe pulse method and the handshaking method of asynchronous data transfer are not restricted to input output transfer. in fact they are used extensively on numerous occasions requiring the transfer of data between two independent units.

KNOWLEDGEGATE

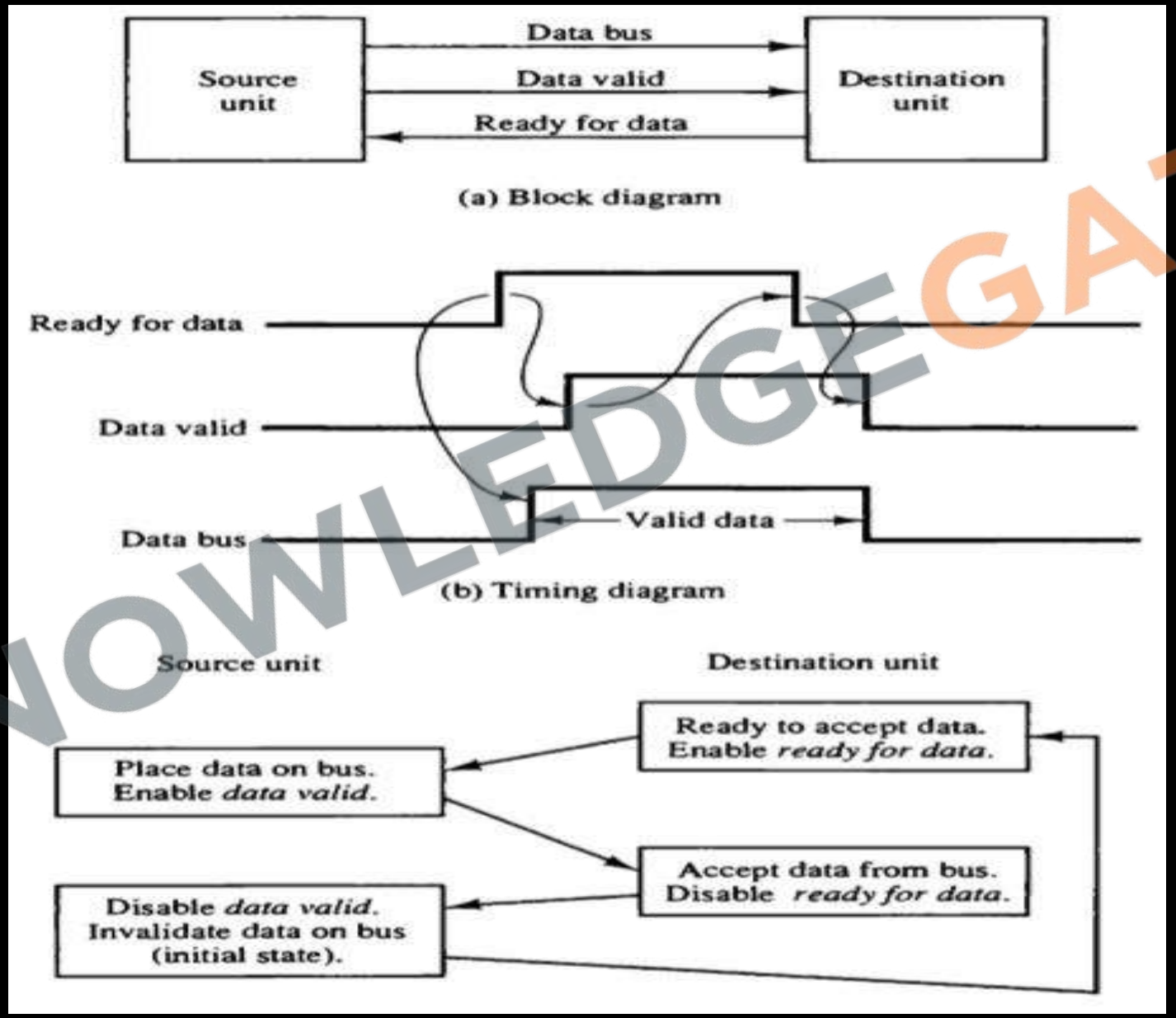
www.knowledgegate.in

Source Initiated Transfer



KNOWLEDGEGATE

Destination initiated transfer



KNOWLEDGEGATE

Modes of Data Transfer

Here we deal with the problem that how data communication will take place CPU and i/o device.

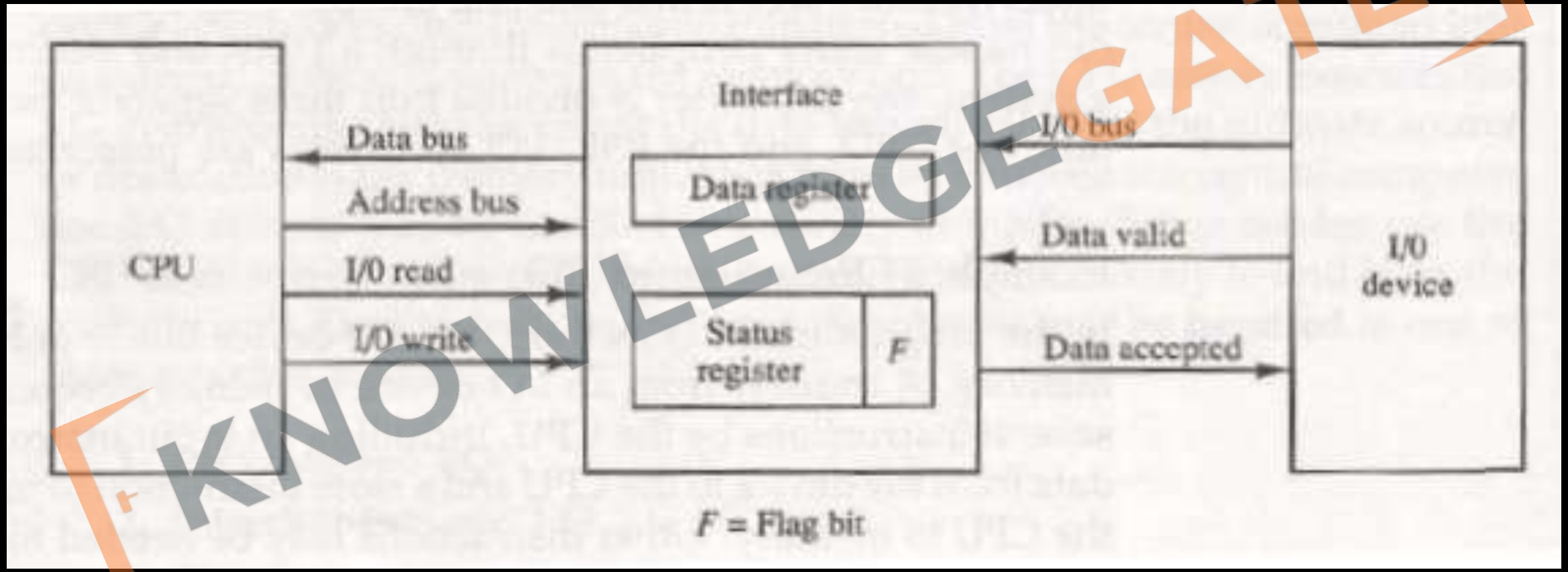
There are popularly three methods of data transfer: -

- **Programmed i/o**
- **Interrupt initiated i/o**
- **Direct Memory Transfer**

KNOWLEDGEGATE

Programmed I/O

- In this i/o device cannot access the memory directly. To complete data transfer number of instructions will be executed out of which input instruction are those which transfer data from device to CPU and store instructions from CPU to memory.

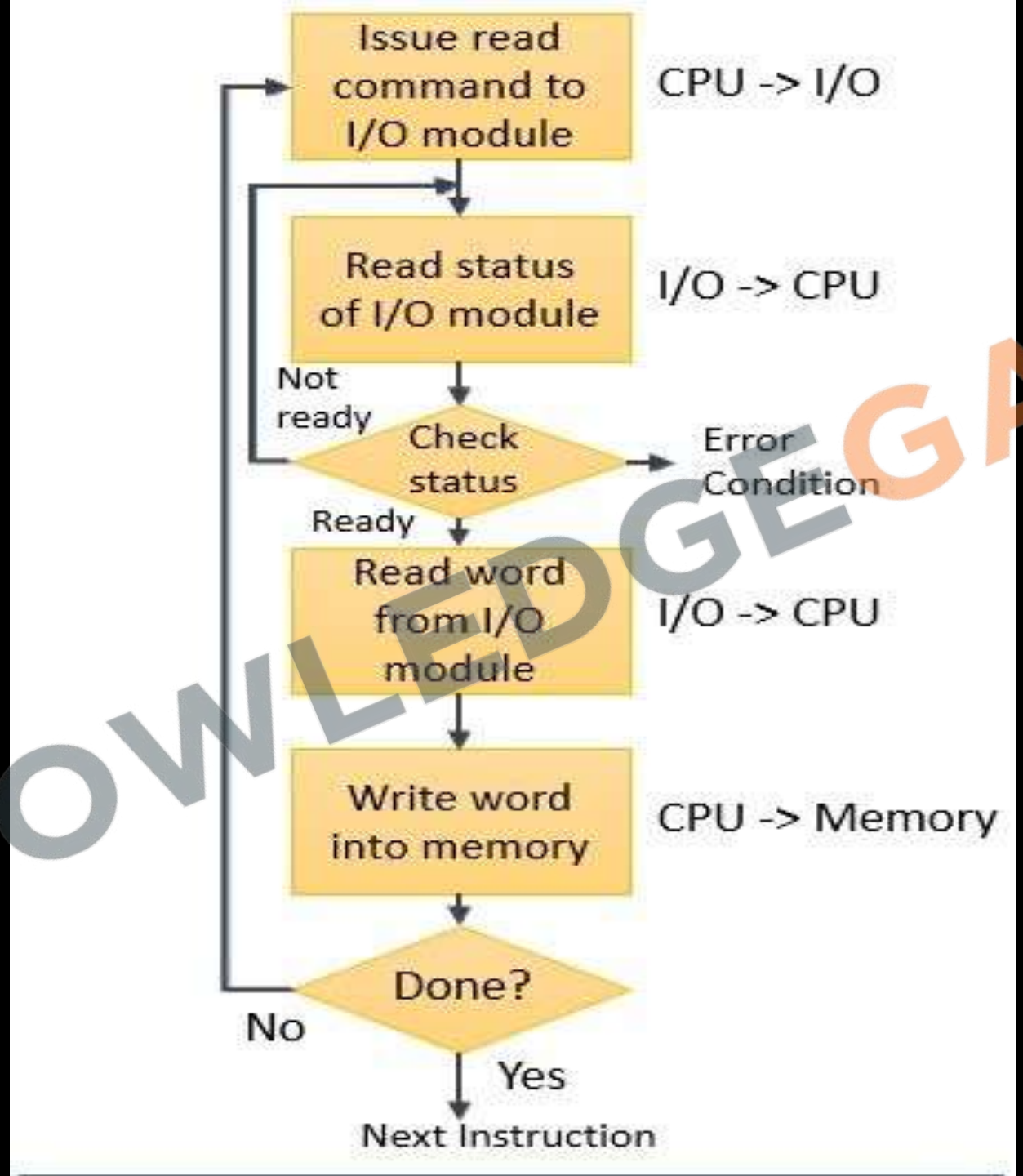


KNOWLEDGEGATE

www.knowledgegate.in

- **Step 1: -** i/o device will put data on the data bus and will enable valid data signal
- **Step 2: -** Interface is continuously sensing for data valid signal and when it receives the signal it will copy data from the data bus into it's data register and set its flag bit to 1 and enable data accepted line.
- **Step 3: -** CPU is continuously monitoring the status register in the programmed mode and as soon as it sees flag bit as 1, it immediately copies data from data register on the data bus and clear flag bit to zero.
- **Step 4: -** Now interface will disable data accepted line to tell i/o device, I am ready for new transitions.
- **Conclusion: -** CPU works in programmed mode or in busy wait mode so no of clock cycles are wasted. It will be difficult to handle multiple i/o device at the same time. It is not appropriate with the high-speed i/o devices.

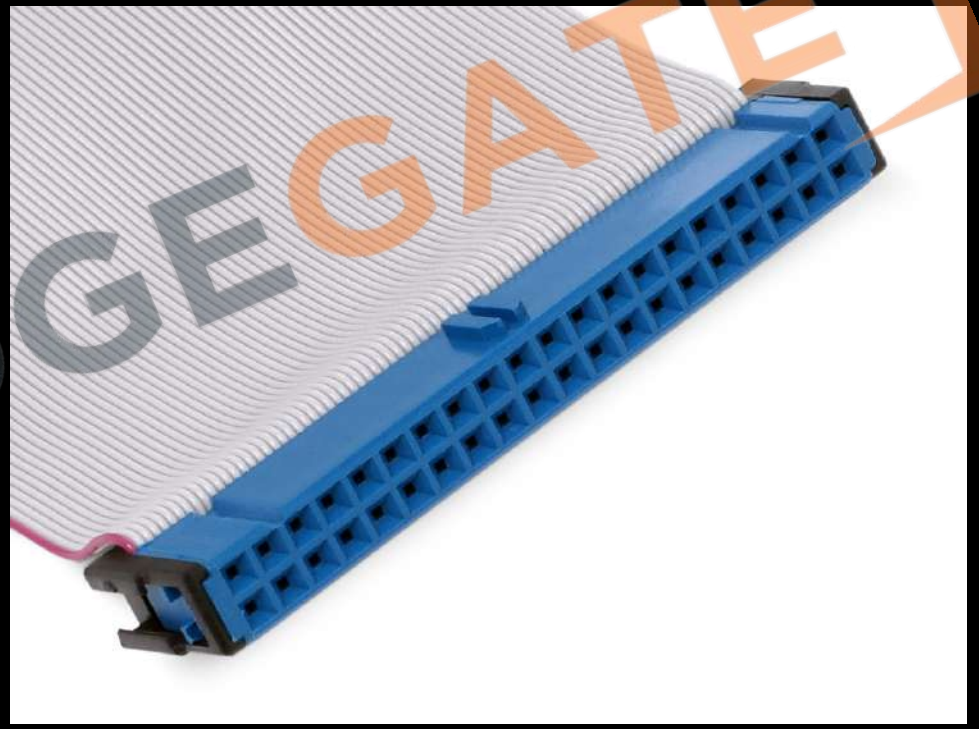
KNOWLEDGEGATE



KNOWLEDGEGATE

- The best known example of a PC device that uses programmed I/O is the ATA interface

- **Parallel ATA** (**PATA**), originally **AT Attachment**, is an interface standard for the connection of storage devices such as hard disk drives, floppy disk drives, and optical disc drives in computers.



KNOWLEDGEGATE

www.Knowledgegate.in

Interrupt initiated i/o

- **In** this method I/o device interrupt CPU when it is ready for data transfer.
- CPU keep executing instructions and after executing one instruction and before starting another instructions CPU wait and see if there is interrupt or not and if there is interrupt then takes a decision whether CPU should entertain this interrupt or continue with the execution.
- **Note: -** instruction is always absolute in nature and there is nothing like partial execution of instruction.
- The method of handling interrupts of different i/o devices are different, therefor every device has a program or routine called ISR (interrupt service routine) which tell CPU how the interrupt should be managed (and it saves CPU time).

KNOWLEDGEGATE

www.knowledgegate.in

- Interrupt can be of two types: -
  - **Non-vectored interrupt: -** Here there is a mutual understanding between CPU and device that where this routine is stored in memory (high priority device)
  - **Vectored interrupt: -** Some interrupts may be vectored where the interrupting device will also tell the address that where this routine is stored in the memory.
- It is possible that different i/o devices interrupt at the same time. Now CPU must have a priority decision that which interrupt should be service first and which should be service later.

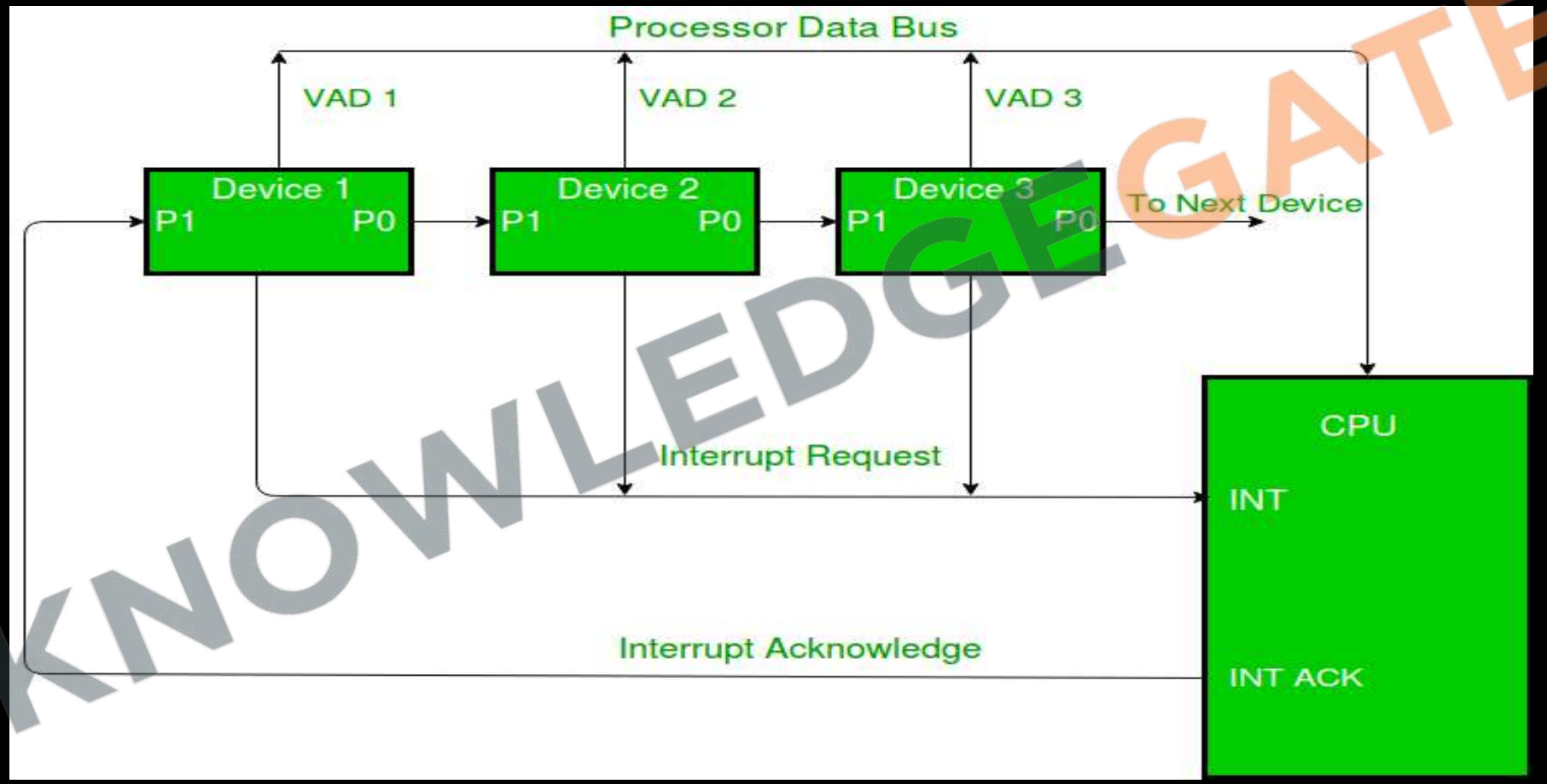
KNOWLEDGEGATE

www.knowledgegate.in

Feature	Vectored Interrupt	Non-Vectored Interrupt
Addressing	Directly jumps to specific ISR	Jumps to a fixed location
Efficiency	Faster, less overhead	Slower due to extra steps
Complexity	More complex to set up	Simpler to implement
Flexibility	Higher, allows multiple ISRs	Lower, usually one ISR
Use Case	Suitable for systems with multiple devices requiring different handling	More appropriate for simpler systems with fewer interrupt sources

KNOWLEDGEGATE
www.knowledgegate.in

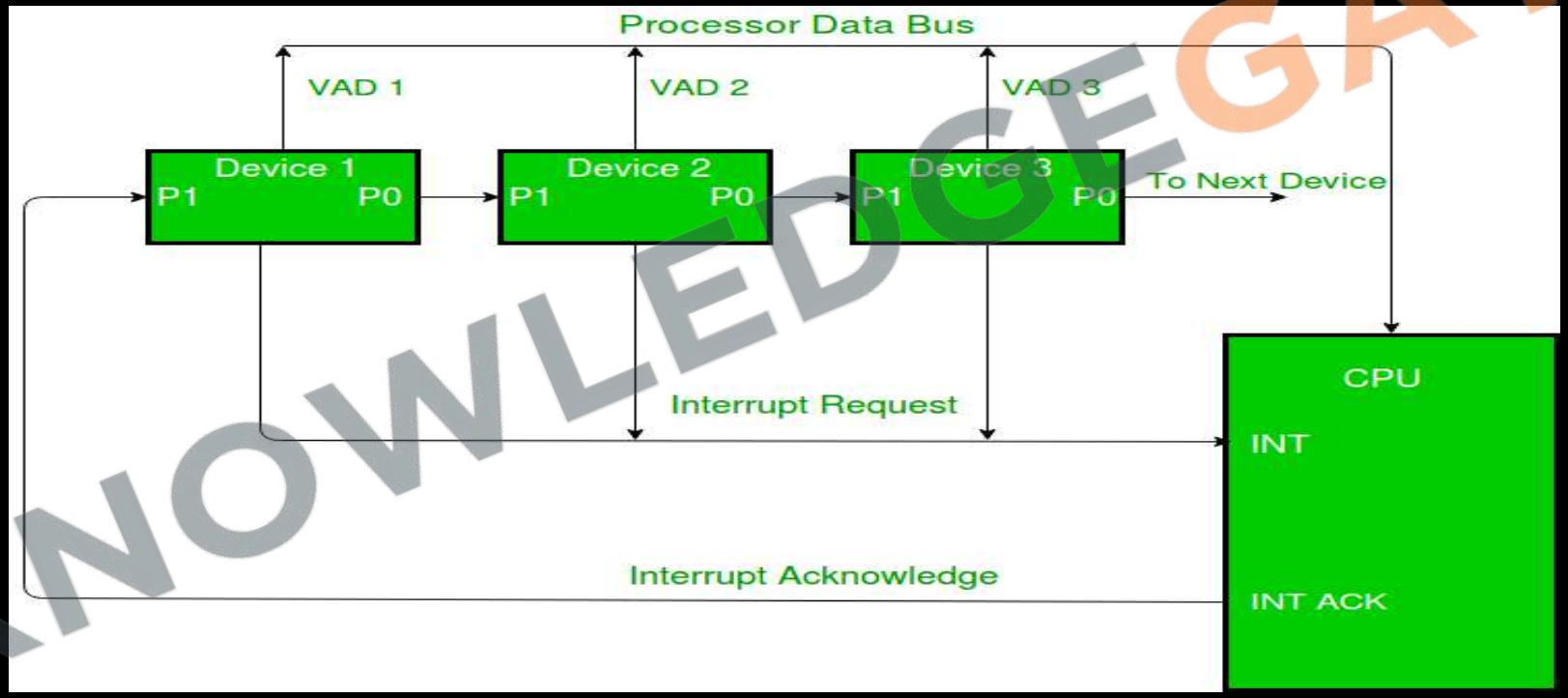
- **H/w solution: -** It can be serial or parallel, serial solution is known as Daisy Channing is a h/w solution which is used to decide priority among different i/o devices (VAD- vector address device)



KNOWLEDGEGATE

www.knowledgegate.in

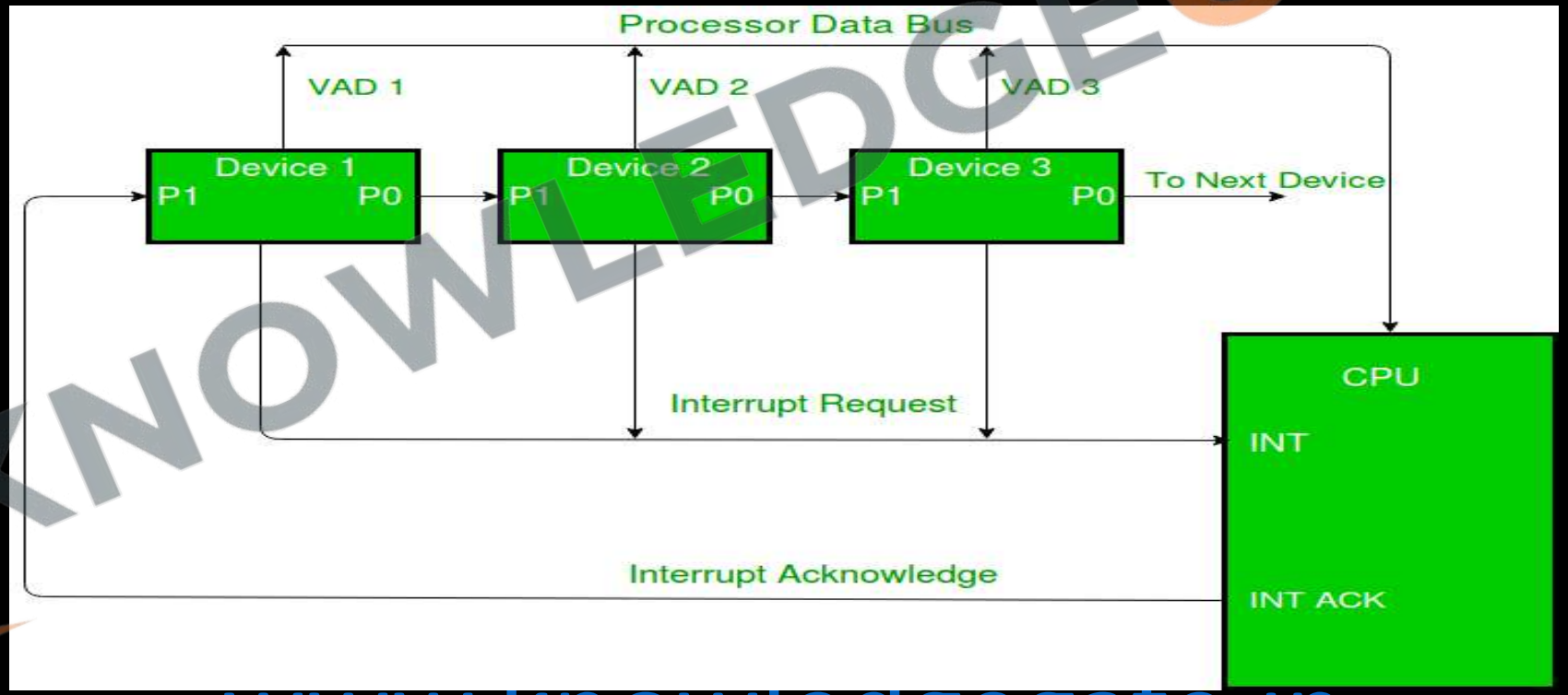
- Out of all possible devices 1 or more device may send an interrupt with a common line.
- When CPU completes 1 instruction and check interrupt in line and found an interrupt, then CPU will enable interrupt acknowledgment line to 1.



KNOWLEDGEGATE

www.knowledgegate.in

- The i/o device placed first in the arrangement will always get acknowledgement first. If it wants to perform i/o then it will put 0 on priority outline and will put the address of its interrupt service routine (ISR) on the vector address line.
- If device do not want to perform i/o then will set priority output as 1 and will give chance to the second device, and the processor continue.
- **Advantage: -** very simple, easy to use, easy to understand, relatively fast.
- **Disadvantage: -** here priority fixed and even in case of requirement we cannot change it.



KNOWLEDGEGATE
www.knowledgegate.in

- **Polling for Identification**: When multiple devices send interrupts, the CPU polls them in a predefined sequence to identify which device sent the interrupt. The first device to confirm gets serviced first.
- **Static Priority via Polling**: The order in which devices are polled effectively sets their priority. Devices polled earlier are serviced first, making this a form of static priority assignment.
- **Simplicity & Limitations**: This approach is straightforward but rigid. It's suitable for systems with fixed priorities but may not be ideal for dynamic environments where priority needs can change rapidly.

KNOWLEDGEGATE

www.knowledgegate.in

- **Hardware and software interrupts** :
  - The interrupts initiated by an external hardware by sending an appropriate signal to the interrupt pin of the CPU is called hardware interrupt. The software interrupts are program instructions. These instructions are inserted at desired location in a program. While running a program, if software interrupt instruction is encountered the CPU initiates an interrupt

- **Maskable and non-maskable interrupts**.
  - The interrupts whose request can be either accepted or rejected by the CPU are called maskable interrupts.
  - The interrupts whose request has to be definitely accepted by the CPU are called non-maskable interrupts.

KNOWLEDGEGATE

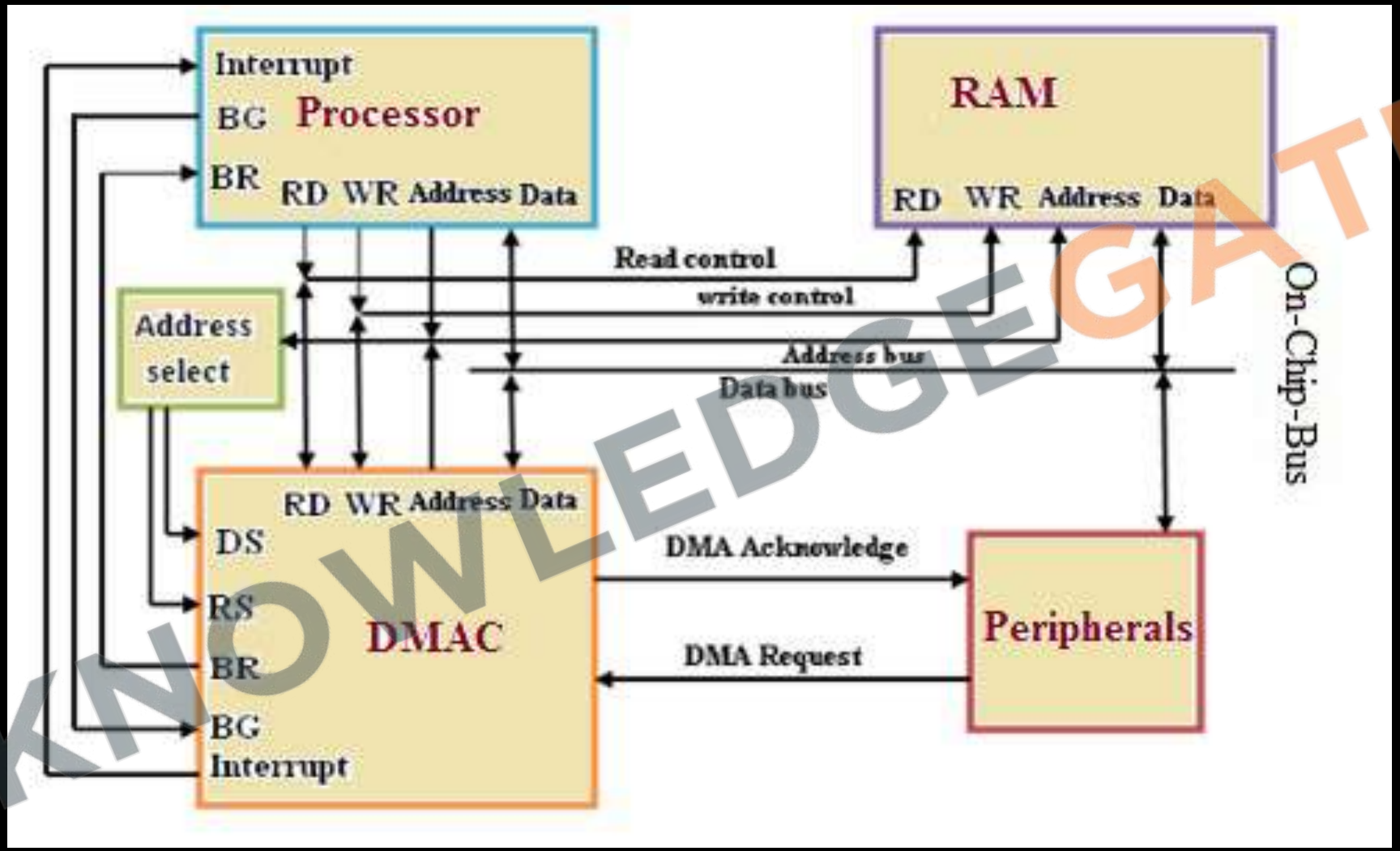
www.knowledgegate.in

- Intel 82C59A

KNOWLEDGEGATE

www.knowledgegate.in

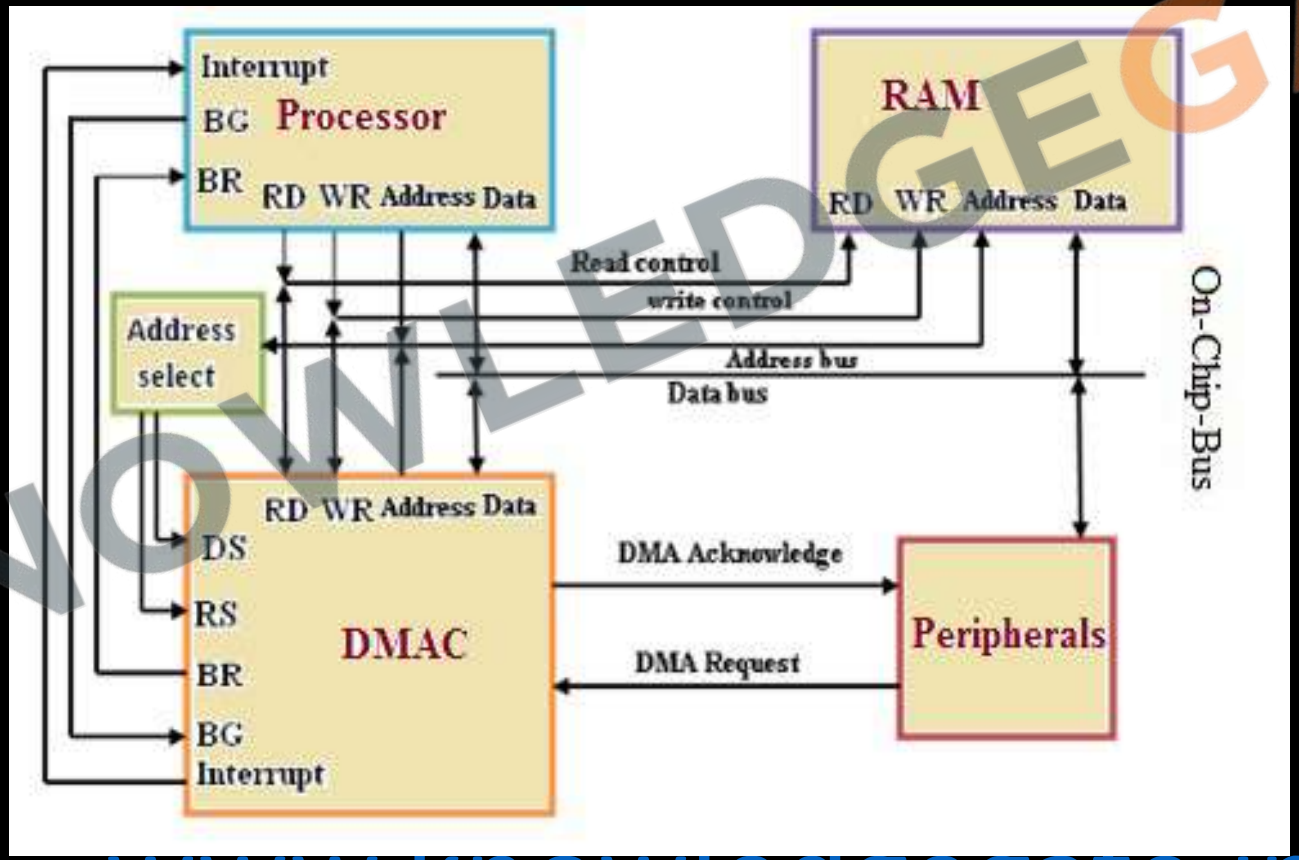
Direct Memory Access (DMA)



KNOWLEDGEGATE

Direct Memory Access (DMA)

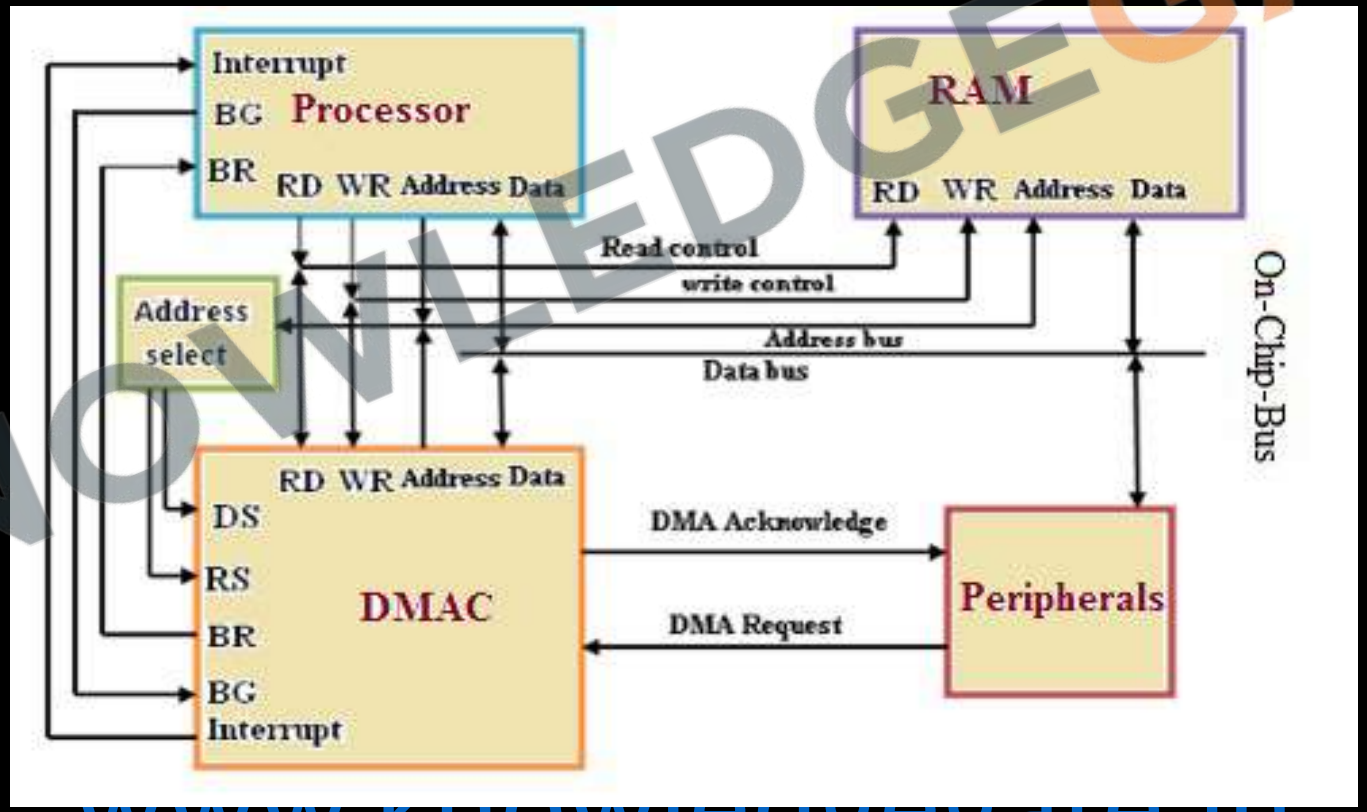
- When we want to perform i/o operations then the actual source or destination is either i/o device or memory, but CPU is placed in between just to manage and control the transfer.
- **DMA is** the idea where we use a new device is call DMA controller using which CPU allow DMA controller to take control of system buses and perform direct data transfer either from device to memory or from memory to device.



KNOWLEDGEGATE
www.knowledgegate.in

Sequence of DMA transfer: -

- **Step 1: -** I/O device will send a DMA request to DMA controller to perform an i/o operation.
- **Step 2: -** DMA controller will set interrupt and bus request line to 1.
- **Step 3: -** CPU using address bus will select device and registers and then will initiate i/o address register(location), counter (no of words)
- **Step 4: -** CPU will put on the bus grant line to tell DMA controller, now you are the master of system buses.
- **Step 5: -** now DMA controller will put 1 in DMA acknowledgement and using read/write control lines and address line will perform i/o directly between memory and device.



KNOWLEDGEGATE
www.knowledgegate.in

Mode of transfer: -

- **Burst mode: -** when entire i/o transfer is completed and then control comes back to CPU then it is called burst mode transfer. i.e. with high speed device like magnetic disk.
- **Cycle stealing mode: -** When CPU executes an instruction then normally their could following phases.
  - **IF –** Instruction Fetch
  - **ID –** Instruction Decode
  - **OF –** Operand fetch
  - **IX –** Instruction execute
  - **WB –** write back or store result

- Normally in ID and IE phases CPU don't require system buses and if only in that time control is giving to DMA controller then it is called cycle stealing.

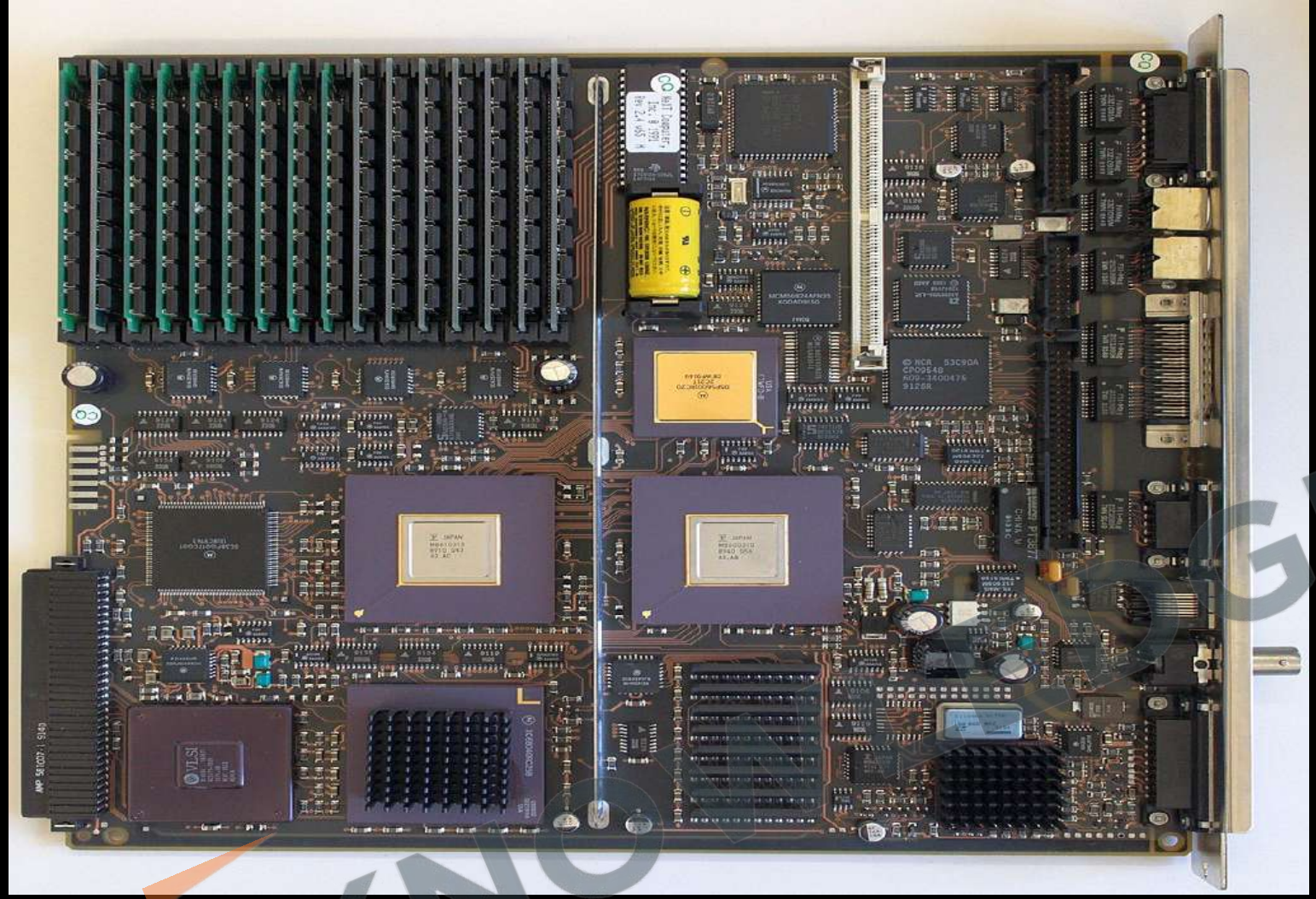
KNOWLEDGEGATE

- Many hardware systems use DMA, including disk drive controllers, graphics cards, network cards and sound cards.

- In the original IBM PC (and the follow-up PC/XT), there was only one Intel 8237 DMA controller capable of providing four DMA channels (numbered 0–3).

KNOWLEDGEGATE

www.knowledgegate.in



- Motherboard of a NeXTcube computer (1990).

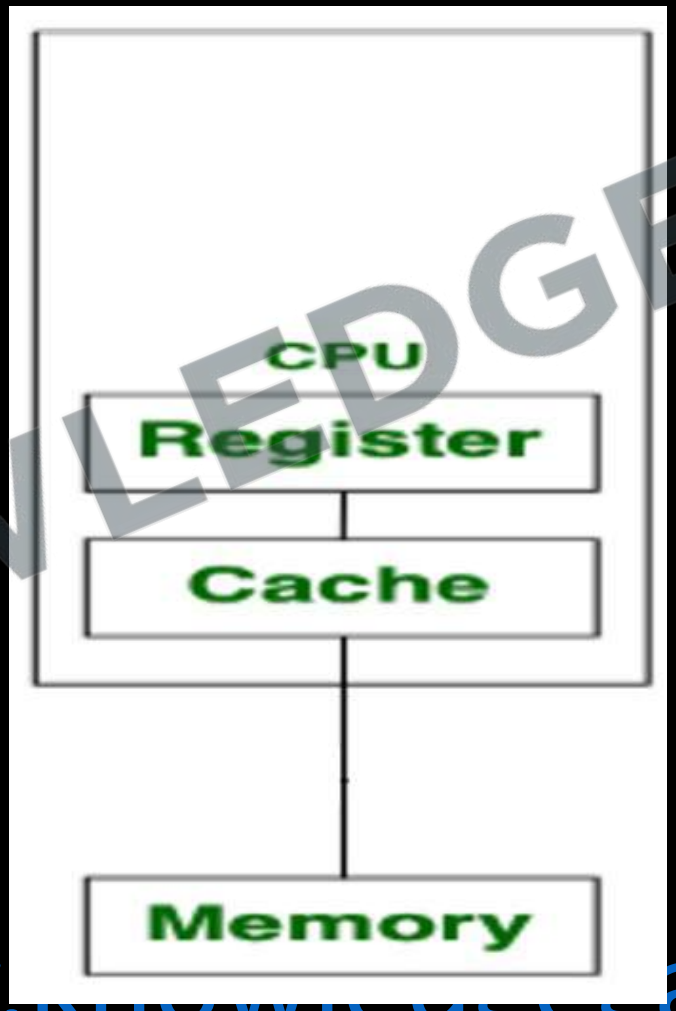
- The two large integrated circuits below the middle of the image are the DMA controller (l.) and - unusual - an extra dedicated DMA controller (r.) for the magneto-optical disc used instead of a hard disk drive in the first series of this computer model.

KNOWLEDGE GATE

www.knowledgegate.in

Uniprocessing

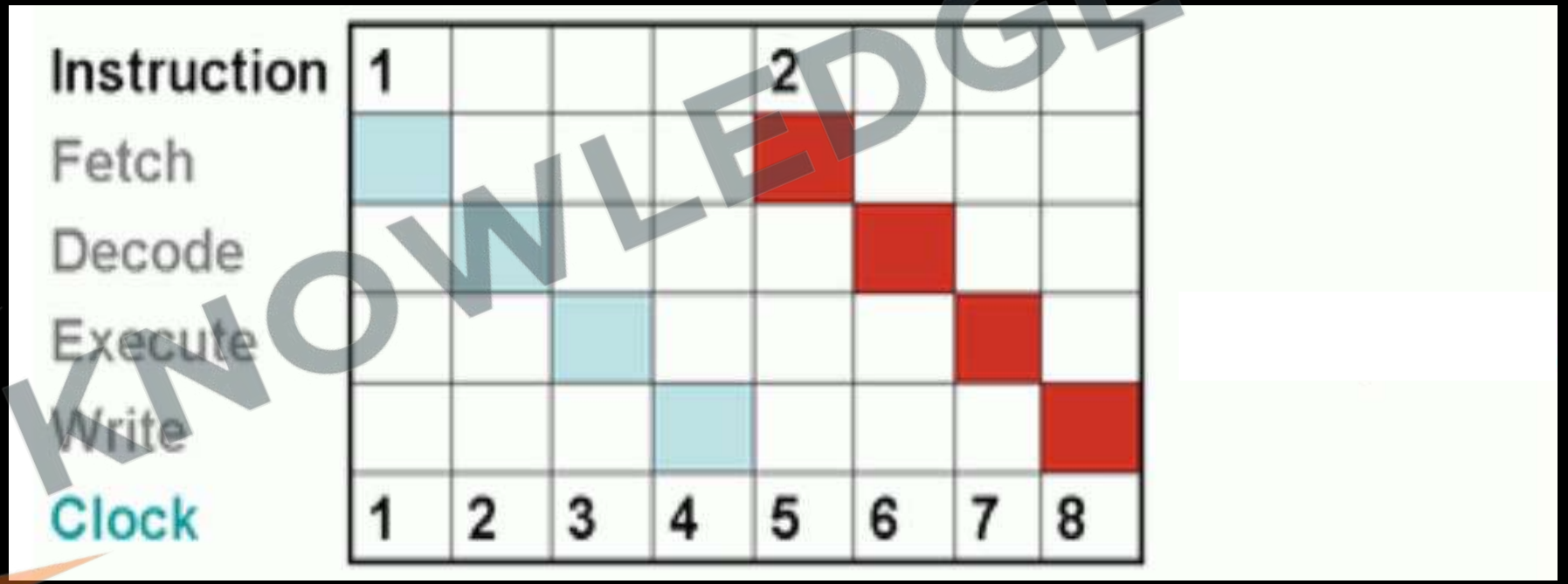
- If the system has only one processor then at most one instruction can be executed at a time.



KNOWLEDGEGATE
www.knowledgegate.in

- When we actually execute an instruction, it is executed into number of phases, like
  - Instruction Fetch
  - Instruction Decode
  - Instruction executes
  - Instruction Store

- Where when one phase is completed then only we start with next phase.



KNOWLEDGEGATE

www.knowledgegate.in

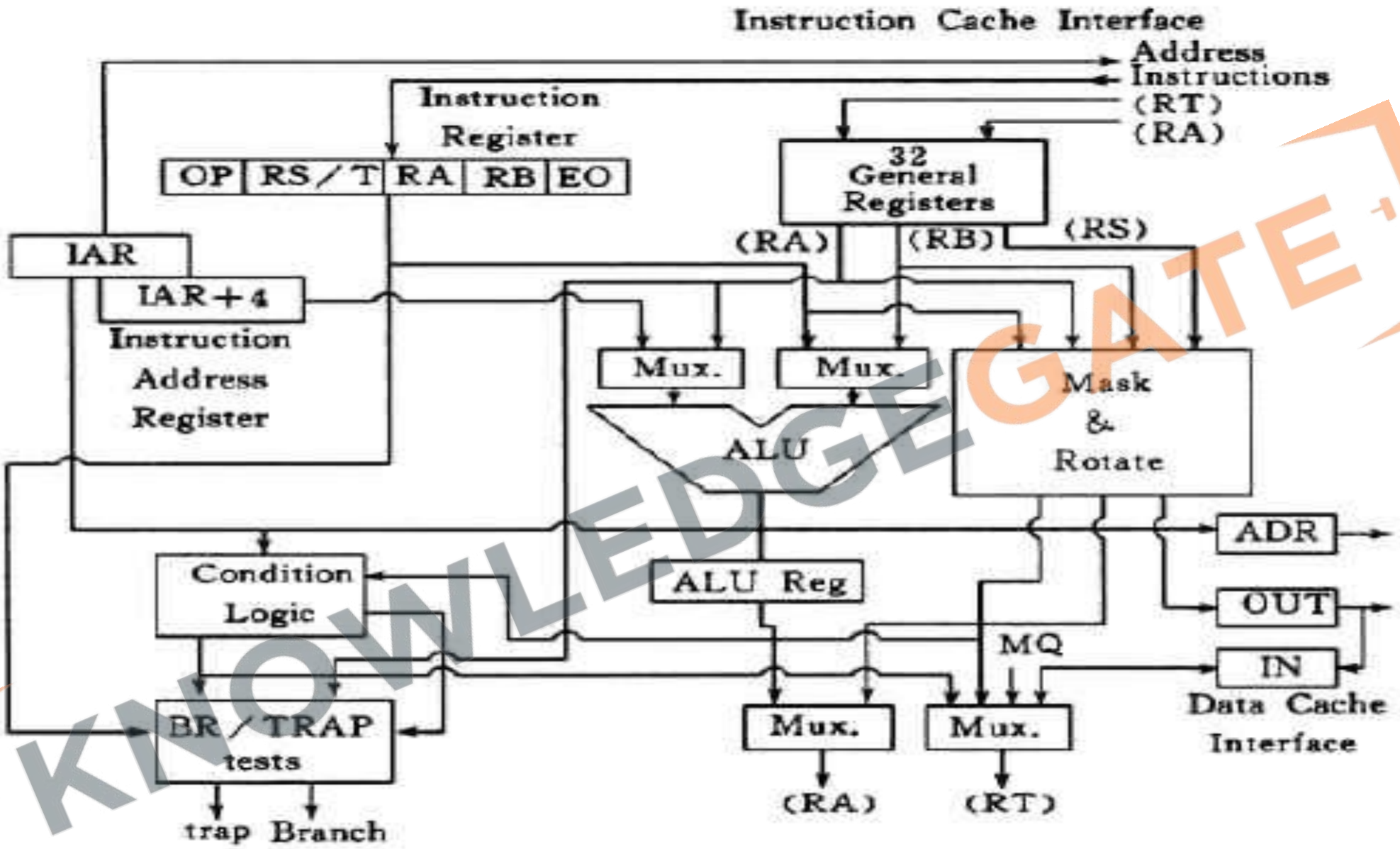
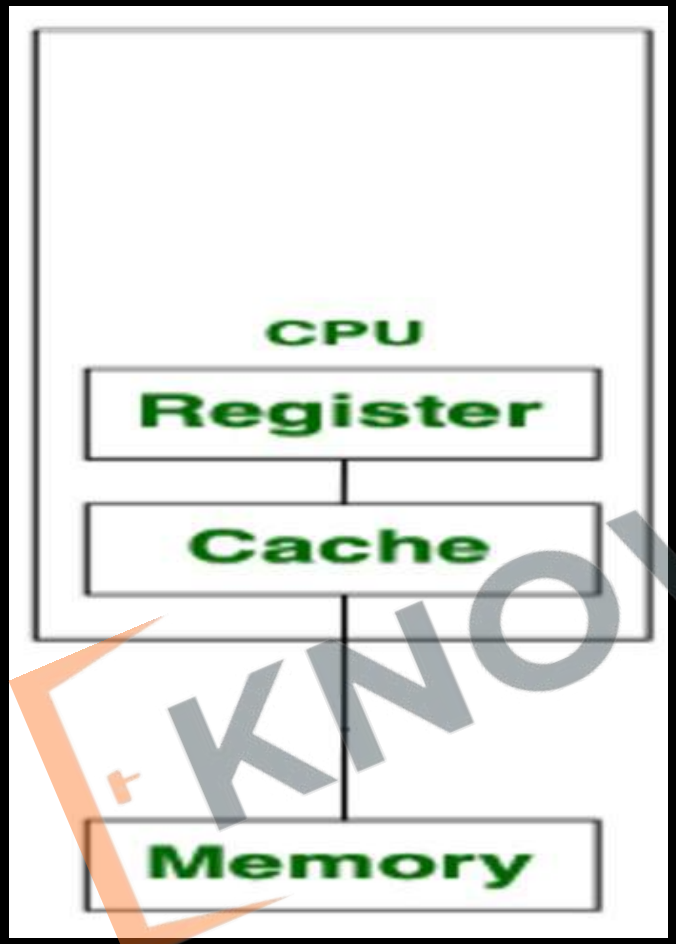


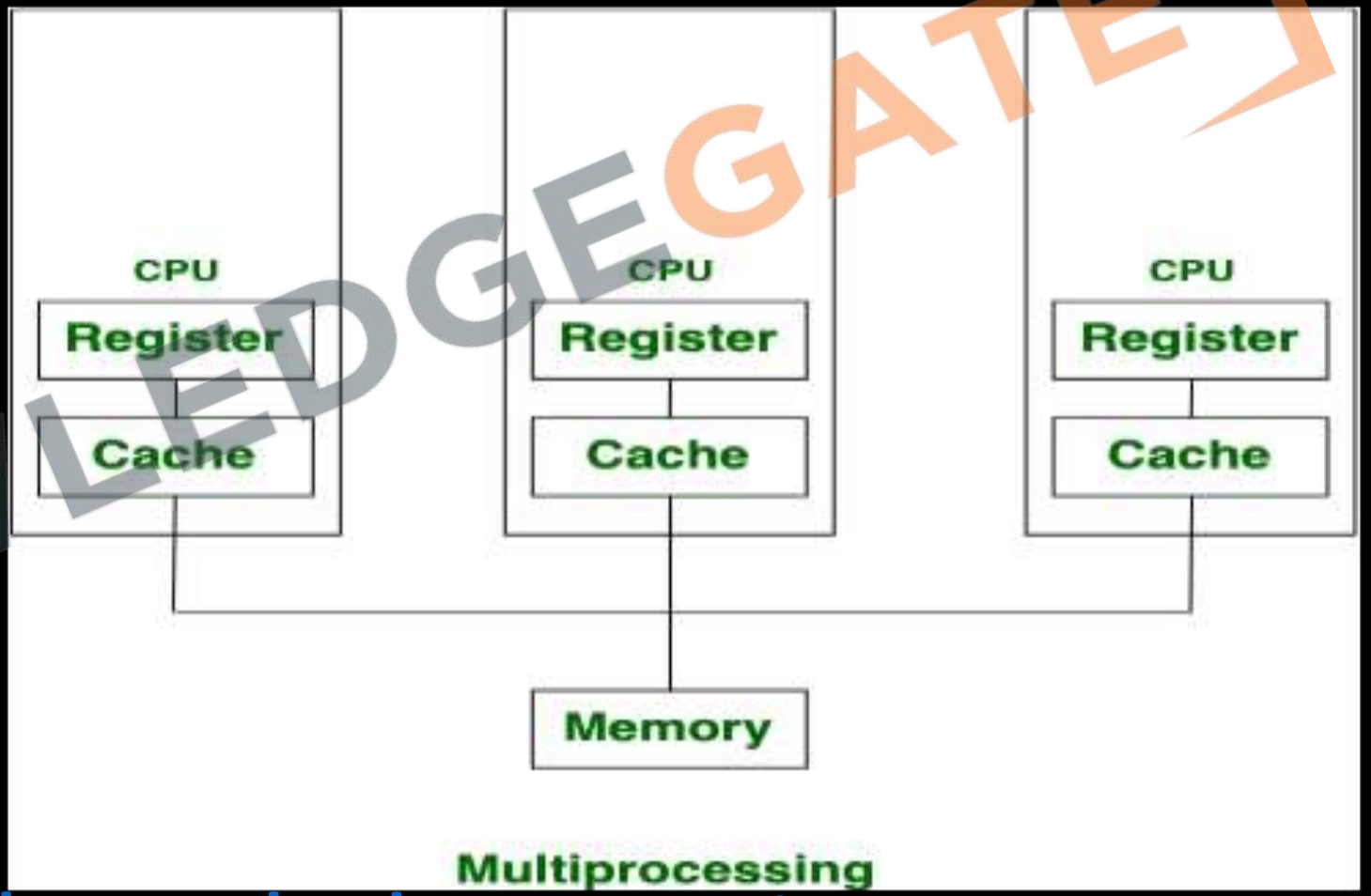
Fig. 3-2 IBM 801 Architecture

KNOWLEDGE GATE

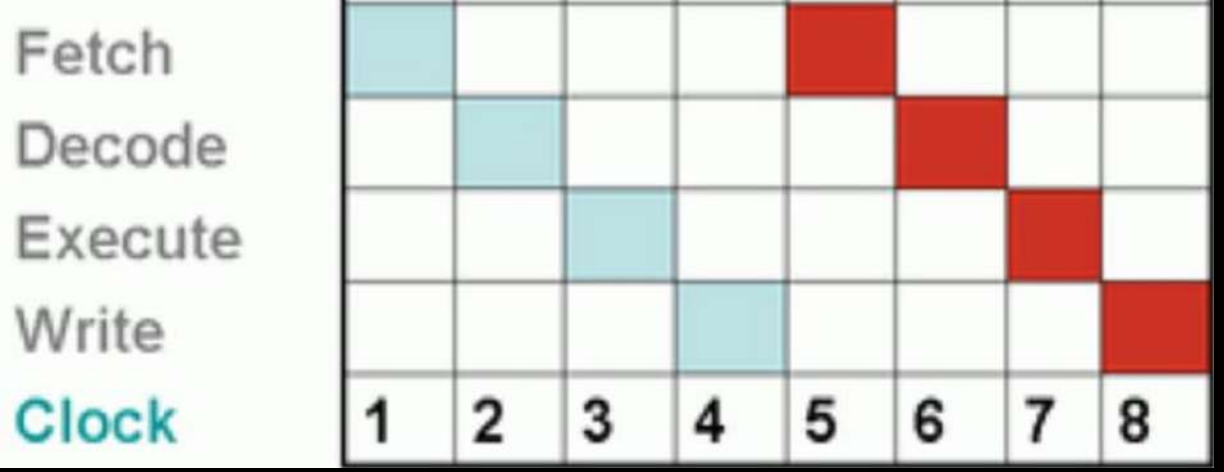
Uniprocessing vs Multiprocessing

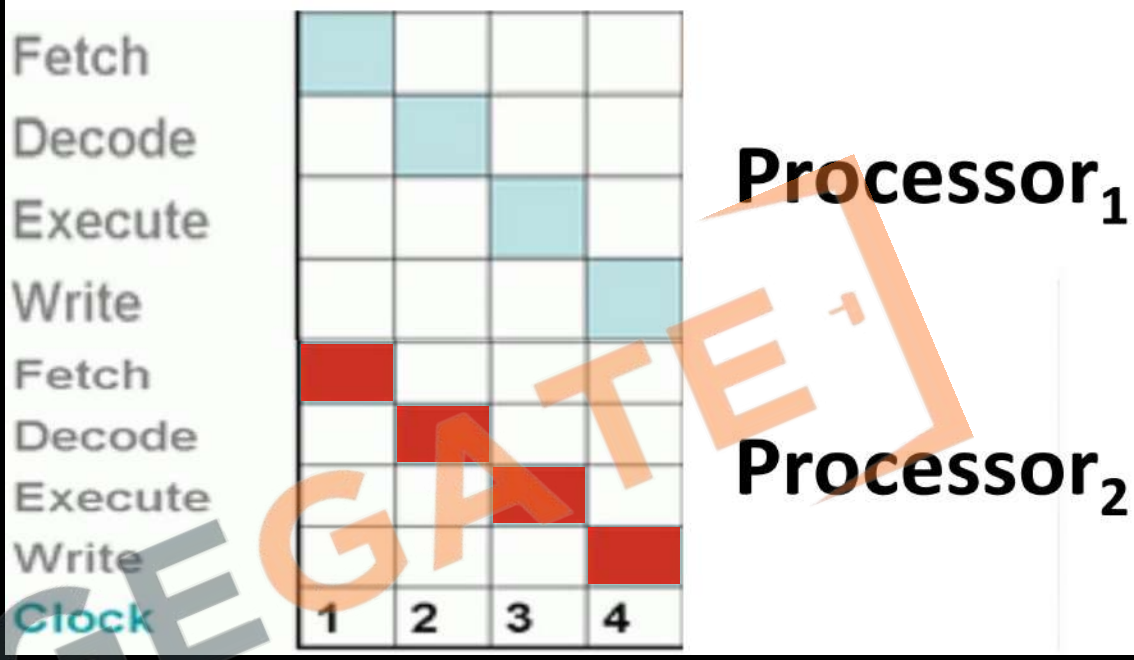
- If we really want to execute multiple instruction together or concurrently then we must have multiple processors.





KNOWLEDGEGATE





KNOWLEDGEGATE

www.knowledgegate.in

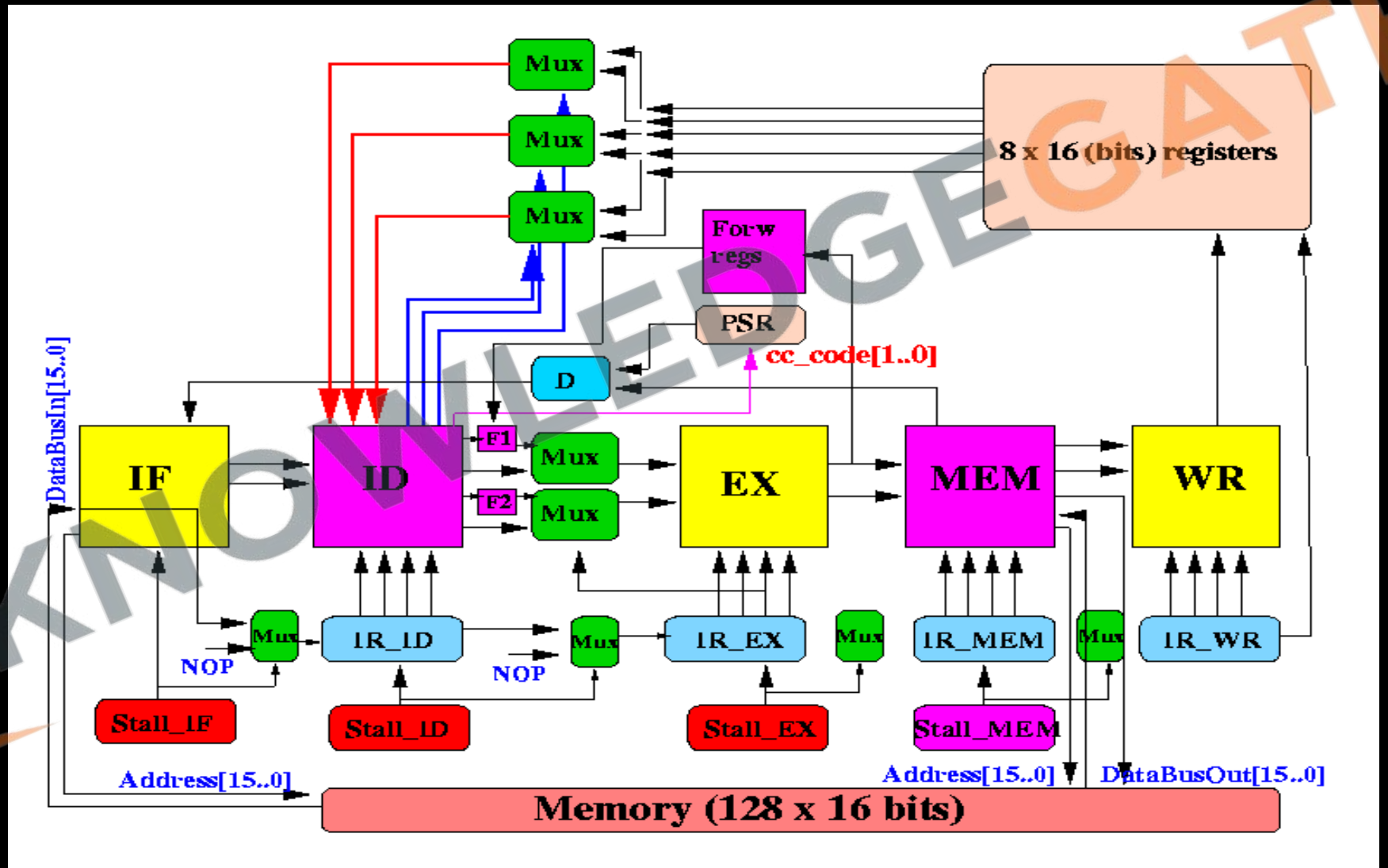
Pipelining

- Pipelining is a phenomena or method using which, we will able to run more than one instruction at the same time, on a single processor.

KNOWLEDGEGATE

www.knowledgegate.in

- The idea if make special processor (pipelined processor), where the circuit of every phase is different and buffers are placed between stages. Then we can start executing next instruction before completing all the phases of the current executing one. This idea is called pipelining.



- Note: Hardware architecture of non-pipelined and pipelined processor are different.

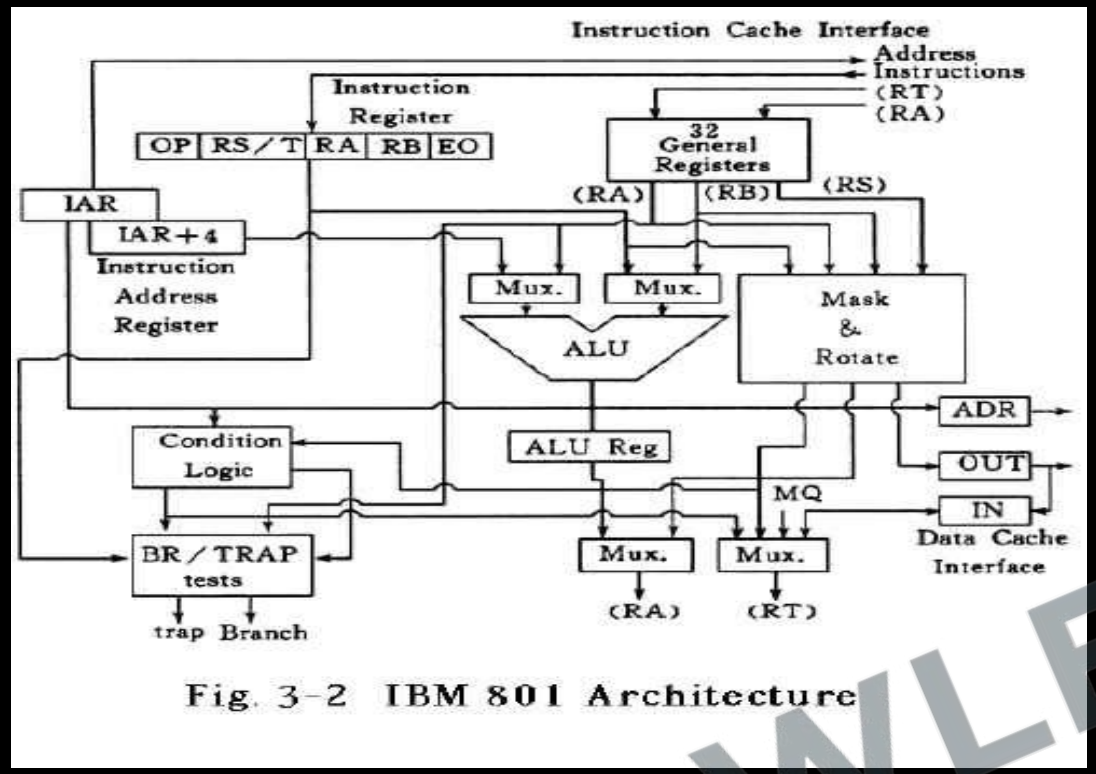
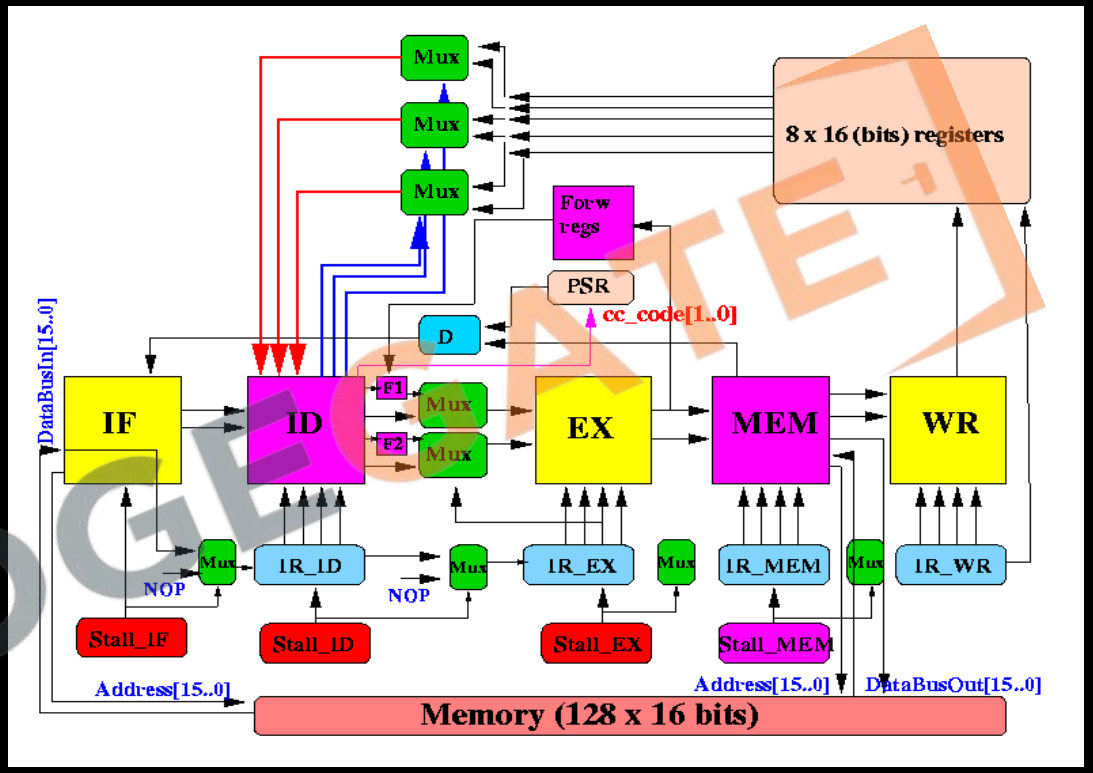


Fig. 3-2 IBM 801 Architecture



KNOWLEDGEGATE

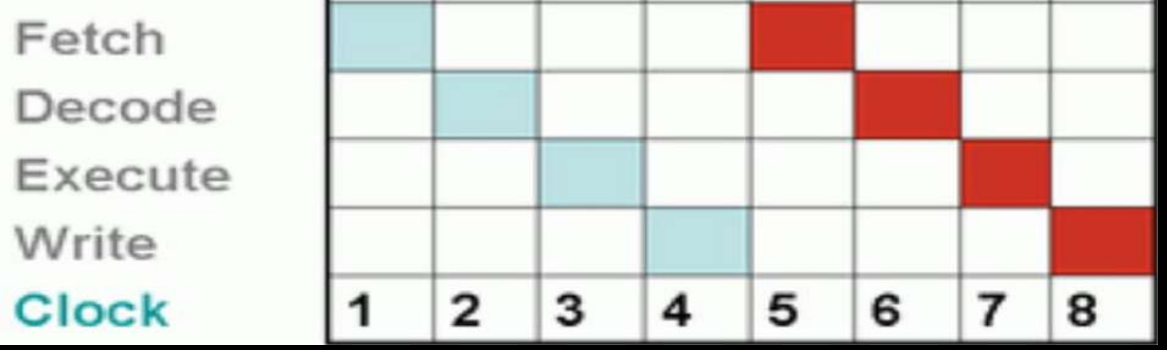
www.knowledgegate.in

KNOWLEDGEGATE

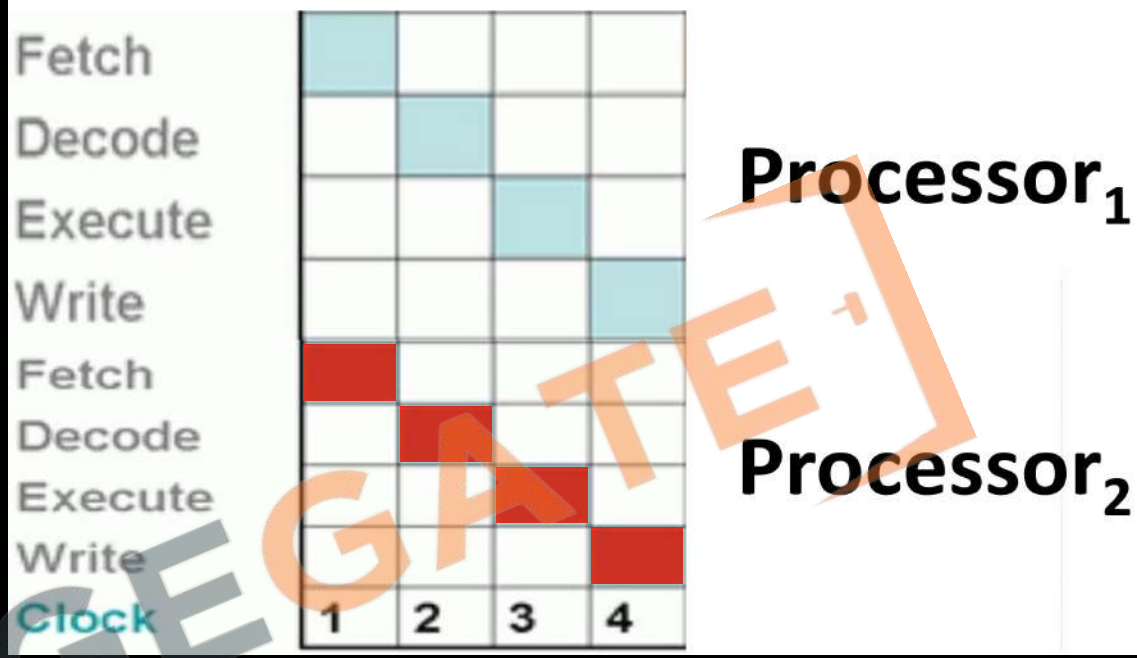
www.knowledgegate.in

KNOWLEDGEGATE

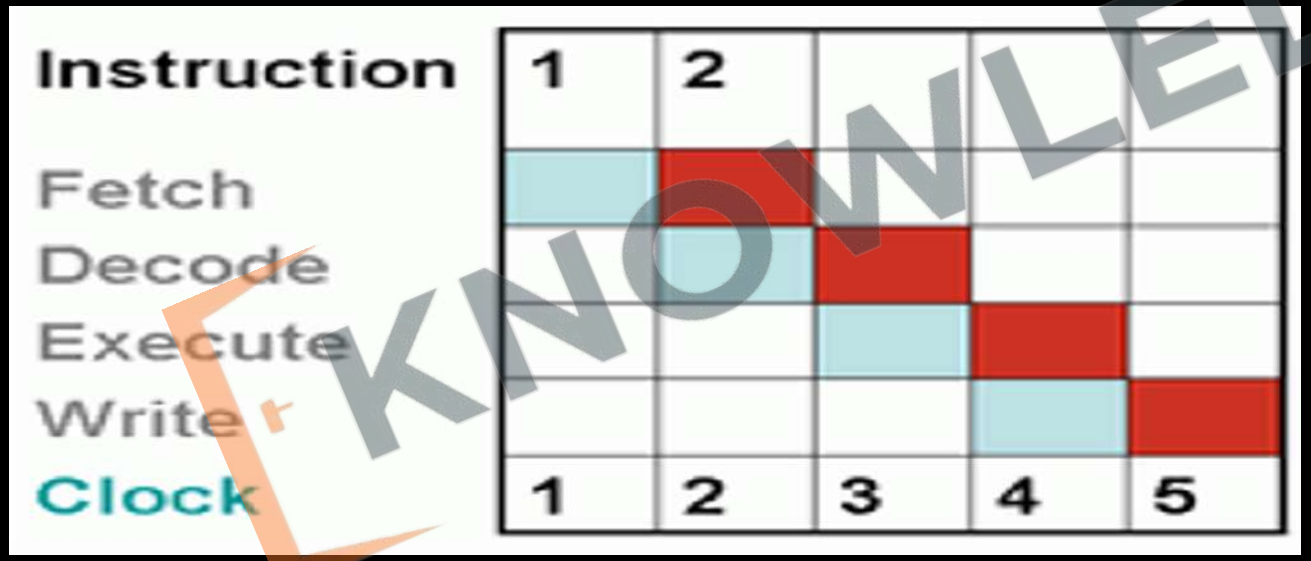
www.knowledgegate.in



Uniprocessing



Multiprocessing



Pipelining

KNOWLEDGEGATE

www.knowledgegate.in

Q Consider a system where clock is triggering at a speed of 1MHz (1 clock = 1 µs). In a pipelined processor there are 4 stages and each stage take only 1 clock, if a program has 10 instruction then it will take what time?

- On a non-pipelined processor

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
IF	I_1				I_2				I_3				I_4				I_5				I_6				I_7				I_8				I_9				I_{10}			
ID		I_1				I_2				I_3				I_4				I_5				I_6				I_7				I_8				I_9				I_{10}		
EX			I_1				I_2				I_3				I_4				I_5				I_6				I_7				I_8				I_9				I_{10}	
WB				I_1				I_2				I_3				I_4				I_5				I_6				I_7				I_8				I_9				I_{10}

KNOWLEDGEGATE

www.knowledgegate.in

- If all instructions are identical (time taken for specific phase is same for all instruction)

- Time without pipeline (T_{wp}) =
  - (sum of clocks for each phase of one instruction) *(no of instruction)*time of one clock

- If each phase requires same clock usually one (as we set the frequency in such a way)
  = (no of phases * no of instruction) * time of one clock

KNOWLEDGEGATE

www.knowledgegate.in

Q Consider a system where clock is triggering at a speed of 1MHz (1 clock = 1 µs). In a pipelined processor there are 4 stages and each stage take only 1 clock, if a program has 10 instruction then it will take what time?

- On a pipelined processor

	1	2	3	4	5	6	7	8	9	10	11	12	13
IF	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	I_9	I_{10}			
ID		I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	I_9	I_{10}		
EX			I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	I_9	I_{10}	
WB				I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	I_9	I_{10}

KNOWLEDGEGATE

www.knowledgegate.in

- If all instructions are identical (time taken for specific phase is same for all instruction)

- If each phase requires same clock usually one (as we set the frequency in such a way)

- Time with pipeline (T_p) = ((no of phase) + (no of instruction - 1)) *time of one clock

KNOWLEDGEGATE

www.knowledgegate.in

- Speed up = (Time without pipeline (T_{wp}))/ (Time with pipeline (T_p)) = 40/13

- Max Speed up = no of stages = (In this case 4)

- Efficiency = (speed up/max speed up) * 100 = 76.9%

KNOWLEDGEGATE

N = 10

www.knowledgegate.in

- Speed up = (Time without pipeline (T_{wp}))/ (Time with pipeline (T_p)) =

- Max Speed up = no of stages = (In this case 4)

- Efficiency = (speed up/max speed up) * 100 = 97.08%

KNOWLEDGEGATE

N = 100

www.knowledgegate.in

- Speed up = (Time without pipeline (T_{wp}))/ (Time with pipeline (T_p)) = 40000/10003

- Max Speed up = no of stages = (In this case 4)

- Efficiency = (speed up/max speed up) * 100 = 99.97%

KNOWLEDGEGATE

N = 10,000

www.knowledgegate.in

Q Consider a system where we have ‘m’ stages and program contains ‘n’ instruction such that m<<n, then find speed up?

KNOWLEDGEGATE

www.knowledgegate.in

Q Consider 5 instruction with following clock requirement?

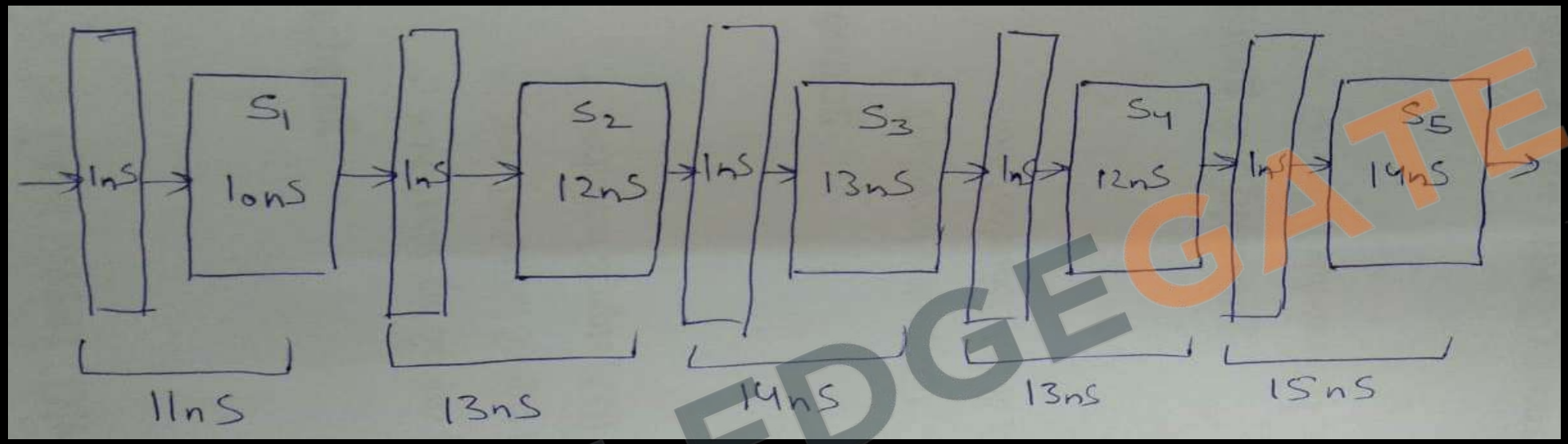
	F	D	E	WB
I_1	1	2	1	1
I_2	1	2	2	1
I_3	2	1	3	2
I_4	1	3	2	1
I_5	1	2	1	2

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
I_1																		
I_2																		
I_3																		
I_4																		
I_5																		

KNOWLEDGEGATE

www.knowledgegate.in

Q After considering this diagram what must be the frequency of the processor to ensure that work of every stage will complete in 1 clock stage wise?



	S_1	S_2	S_3	S_4	S_5
1ns	11	13	14	13	15
2ns	6	7	7	7	8
3ns	4	5	5	5	5
5ns3	3	3	3	3	3
15ns	1	1	1	1	1

KNOWLEDGE GATE

- We understand that different stages in a pipeline may have different delays, it also depends on the type of instruction that how much time a particular stage will take for a specific instruction.

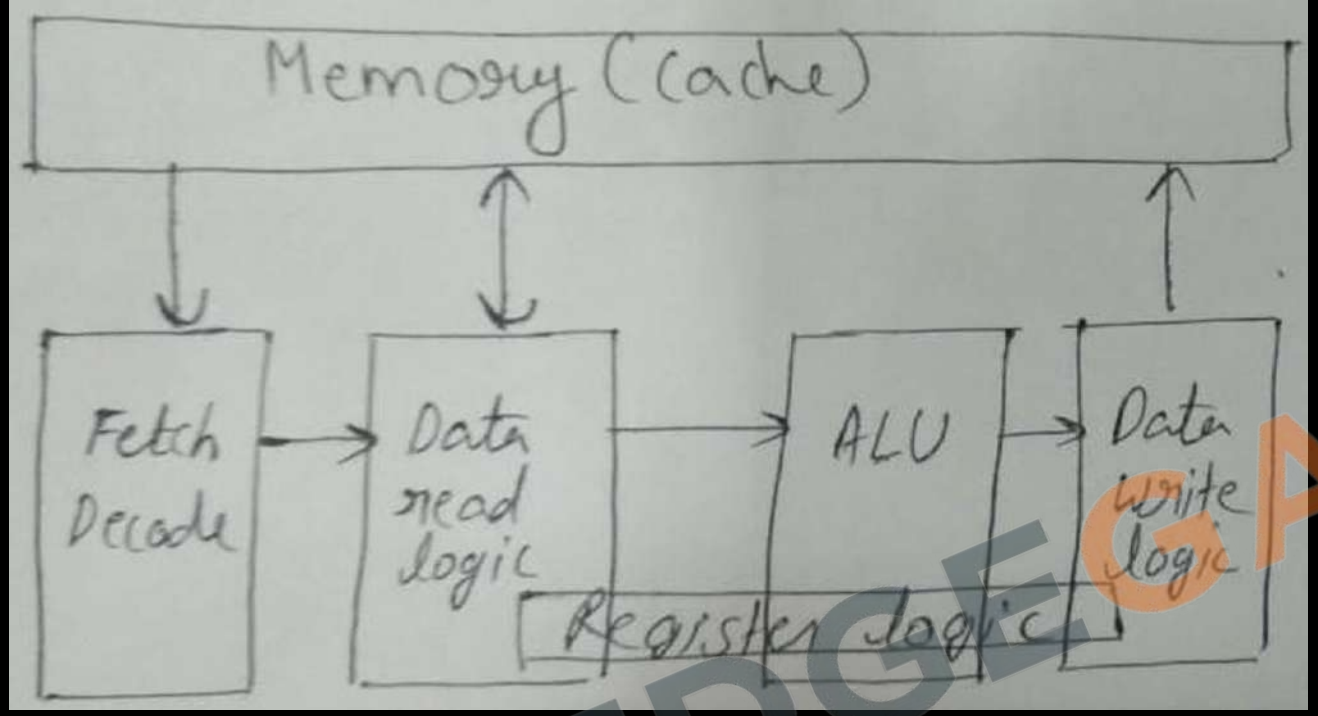
- If we increase the time of a clock to the time taken by the slowest stage of the pipeline, then each instruction takes one clock then with pipe-line processor in long run, we achieve
  - CPI(Clock Per Instruction) = 1

KNOWLEDGEGATE

- Sometimes we cannot execute instructions with full efficiency in a pipeline because of certain dependency or hazards.
  - **Structural hazards**
  - **Control hazards**
  - **Data hazards**

KNOWLEDGEGATE

www.knowledgegate.in



	1	2	3	4	5	6	7	8	9	10	11	12
I_1	FD	DR	ALU	DW								
I_2			FD		DR	ALU	DW					
I_3												
I_4												

KNOWLEDGE GATE

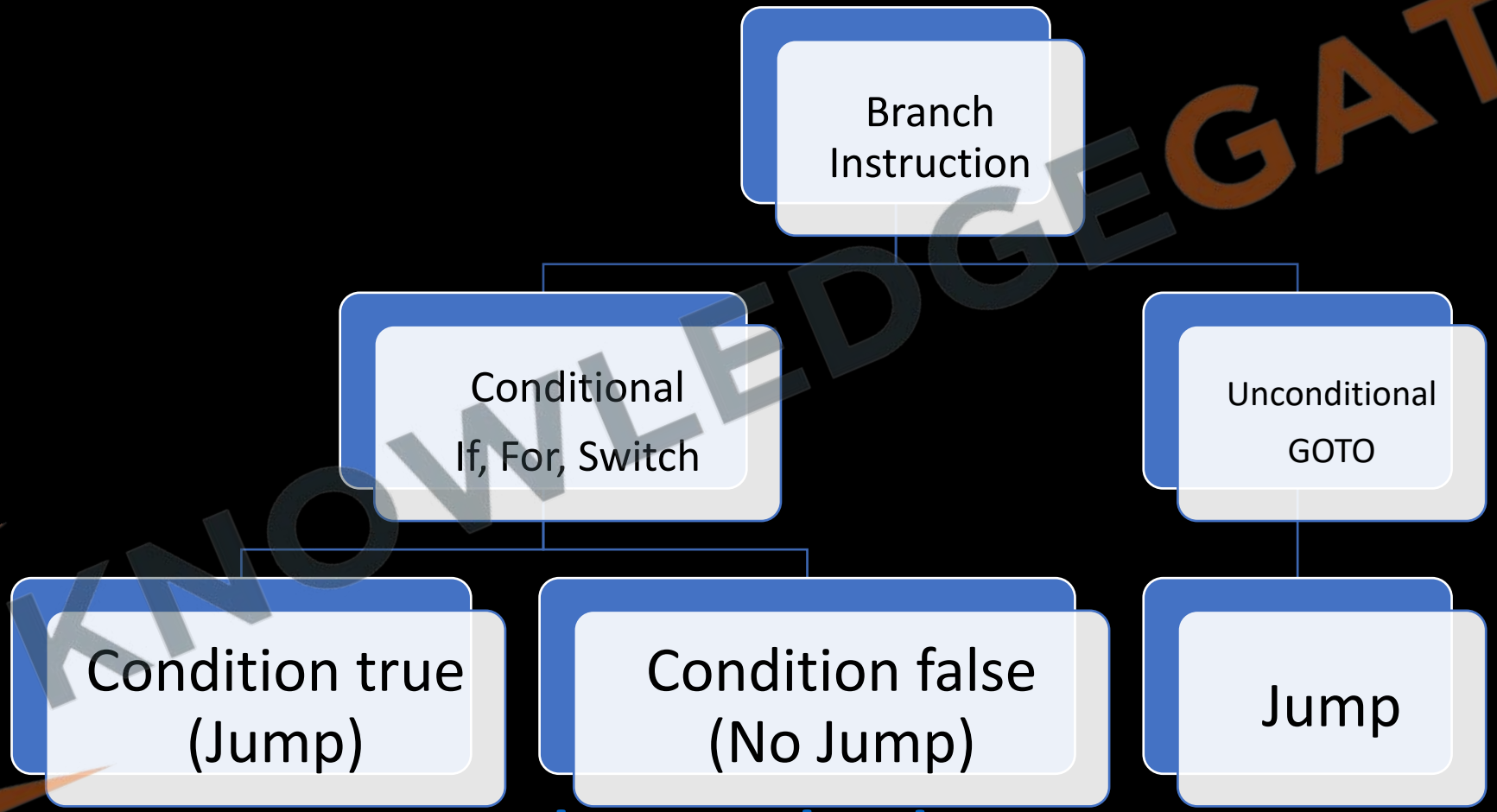
KNOWLEDGEGATE

- A structural hazard cannot be removed using efficient program. Therefore, one solution could be resource duplication but the cost of implementation will be very high.

KNOWLEDGEGATE

www.knowledgegate.in

- If some instruction I_1 is branch instruction and after executing the entire instruction we understand that there is jump and next instruction to be executed is I_{10}. This time we have already partially executed I_1, I_2, I_3 which is the problem.



KNOWLEDGEGATE
www.knowledgegate.in

	1	2	3	4	5	6	7	8	9	10	11	12
I_1	IF	ID	EX	WB								
I_2		IF	ID	EX	WB							
I_3			IF	ID	EX	WB						
I_4				IF	ID	EX						
I_5					IF	ID						
I_6												
I_7						IF	ID	EX	WB			
I_8							IF	ID	EX	WB		

I_3----------->I_7

KNOWLEDGEGATE

www.knowledgegate.in

KNOWLEDGE GATE

Data Hazards

- Data hazards occur when instructions that exhibit data dependence, modify data in different stages of a pipeline. Hazard cause delays in the pipeline. E.g.

Instruction	Meaning of instruction
I_0: MUL R_2, R_0, R_1	$R_2 \neg R_0 * R_1$
I_1: DIV R_5, R_3, R_4	$R_5 \neg R_3 / R_4$
I_2: ADD R_2, R_5, R_2	$R_2 \neg R_5 + R_2$
I_3: SUB R_5, R_2, R_6	$R_5 \neg R_2 - R_6$

KNOWLEDGEGATE

KNOWLEDGEGATE

www.knowledgegate.in

- There are mainly three types of data hazards: This condition is called **Bernstein condition.** RAW (Read after Write) [Flow/True data dependency].

- RAW hazard occurs when instruction J tries to read data before instruction I write it.

  I: $R_2 \leftarrow R_1 + R_3$

  J: $R_4 \leftarrow R_2 + R_3$

Here, J is trying to read R_2 before I have written it.

KNOWLEDGEGATE

- WAR (Write after Read) [Anti-Data dependency]

- WAR hazard occurs when instruction J tries to write data before instruction, I read it.

  I: $R_2 <- R_1 + R_3$

  J: $R_3 <- R_4 + R_5$

Here, J is trying to write R_3 before I have read it.

KNOWLEDGEGATE

www.knowledgegate.in

- WAW (Write after Write) [Output data dependency]

- WAW hazard occurs when instruction J tries to write output before instruction, I write it.

  I: R_2 <- R_1 + R_3

  J: R_2 <- R_4 + R_5

Here J is trying to write R_2 before I.

WAR and WAW hazards occur during the out-of-order execution of the instructions.

KNOWLEDGEGATE

www.knowledgegate.in

We say that instruction S_2 depends in instruction S_1, when: This condition is called **Bernstein condition.**

Three cases exist:

Flow (data) dependence: $O(S_1) \cap I(S_2)$, $S_1 \rightarrow S_2$ and S_1 writes after something read by S_2

Anti-dependence: $I(S_1) \cap O(S_2)$, $S_1 \rightarrow S_2$ and S_1 reads something before S_2 overwrites it

Output dependence: $O(S_1) \cap O(S_2)$, $S_1 \rightarrow S_2$ and both write the same memory location.

$$[I(S1) \cap O(S2)] \cup [O(S1) \cap I(S2)] \cup [O(S1) \cap O(S2)] \neq \phi$$

KNOWLEDGEGATE

www.knowledgegate.in

Solution of Data dependency

- We can use code movement or code relocation and can execute the dependent instruction after some time.
- Here we can use operator forwarding using which we can directly access the result after execution instead of waiting that it gets store in memory.

KNOWLEDGEGATE

Q A 5-stage pipelined processor has Instruction Fetch (IF), Instruction Decode (ID), Operand Fetch (OF), Perform Operation (PO)and Write Operand (WO)stages. The IF, ID, OF and WO stages take 1 clock cycle each for any instruction. The PO stage takes 1 clock cycle for ADD and SUB instructions,3 clock cycles for MUL instruction, and 6 clock cycles for DIV instruction respectively. Operand forwarding is used in the pipeline. What is the number of clock cycles needed to execute the following sequence of instructions?

Instruction	Meaning of instruction
I_0: MUL R_2, R_0, R_1	$R_2 \neg R_0 * R_1$
I_1: DIV R_5, R_3, R_4	$R_5 \neg R_3 / R_4$
I_2: ADD R_2, R_5, R_2	$R_2 \neg R_5 + R_2$
I_3: SUB R_5, R_2, R_6	$R_5 \neg R_2 - R_6$

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
IF	ID	OF	PO	PO	PO	WO													
	IF	ID	OF			PO	PO	PO	PO	PO	PO	WO							
		IF	ID										OF	PO	WO				
			IF	ID												OF	PO	WO	

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
IF	ID	OF	PO	PO	PO	WO													
	IF	ID	OF			PO	PO	PO	PO	PO	PO	WO							
		IF	ID									PO	WO						
			IF	ID									PO	WO					

KNOWLEDGEGATE
www.knowledgegate.in

KNOWLEDGEGATE
edgegate.in

Module

Free Semester One Shot Exam Course

Topic

Operating Systems

Subtopic

OS One Shot

Lesson

Operating System_Notes

Syllabus(Semester Exam)

Unit – I Introduction : Operating system and functions, Classification of Operating systems- Batch, Interactive, Time sharing, Real Time System, Multiprocessor Systems, Multiuser Systems, Multiprocess Systems, Multithreaded Systems, Operating System Structure- Layered structure, System Components, Operating System services, Reentrant Kernels, Monolithic and Microkernel Systems.

Unit – II CPU Scheduling: Scheduling Concepts, Performance Criteria, Process States, Process Transition Diagram, Schedulers, Process Control Block (PCB), Process address space, Process identification information, Threads and their management, Scheduling Algorithms, Multiprocessor Scheduling. Deadlock: System model, Deadlock characterization, Prevention, Avoidance and detection, Recovery from deadlock.

Unit – III Concurrent Processes: Process Concept, Principle of Concurrency, Producer / Consumer Problem, Mutual Exclusion, Critical Section Problem, Dekker's solution, Peterson's solution, Semaphores, Test and Set operation; Classical Problem in Concurrency- Dining Philosopher Problem, Sleeping Barber Problem; Inter Process Communication models and Schemes, Process generation.

Unit – IV Memory Management: Basic bare machine, Resident monitor, Multiprogramming with fixed partitions, Multiprogramming with variable partitions, Protection schemes, Paging, Segmentation, Paged segmentation, Virtual memory concepts, Demand paging, Performance of demand paging, Page replacement algorithms, Thrashing, Cache memory organization, Locality of reference.

Unit – V I/O Management and Disk Scheduling: I/O devices, and I/O subsystems, I/O buffering, Disk storage and disk scheduling, RAID. File System: File concept, File organization and access mechanism, File directories, and File sharing, File system implementation issues, File system protection and security.

Knowledge Gate
Knowledge Gate Website

Chapters of This Video

(Chapter-1: Introduction)- Operating system, Goal & functions, System Components, Operating System services, Classification of Operating systems- Batch, Interactive, Multiprogramming, Multiuser Systems, Time sharing, Multiprocessor Systems, Real Time System.

(Chapter-2: Operating System Structure)- Layered structure, Monolithic and Microkernel Systems, Interface, System Call.

Chapter-3: Process Basics)- Process Control Block (PCB), Process identification information**,** Process States, Process Transition Diagram, Schedulers, CPU Bound and i/o Bound, Context Switch.

(Chapter-4: CPU Scheduling)- Scheduling Performance Criteria, Scheduling Algorithms.

(Chapter-5: Process Synchronization)- Race Condition, Critical Section Problem, Mutual Exclusion,, Dekker's solution, Peterson's solution, Process Concept, Principle of Concurrency,

(Chapter-6: Semaphores)- Classical Problem in Concurrency- Producer/Consumer Problem, Reader-Writer Problem, Dining Philosopher Problem, Sleeping Barber Problem, Test and Set operation.

(Chapter-7: Deadlock)- System model, Deadlock characterization, Prevention, Avoidance and detection, Recovery from deadlock.

(Chapter-8)- Fork Command, Multithreaded Systems, Threads and their management

(Chapter-9: Memory Management)- Memory Hierarchy, Locality of reference, Multiprogramming with fixed partitions, Multiprogramming with variable partitions, Protection schemes, Paging, Segmentation, Paged segmentation.

(Chapter-10: Virtual memory)- Demand paging, Performance of demand paging, Page replacement algorithms, Thrashing.

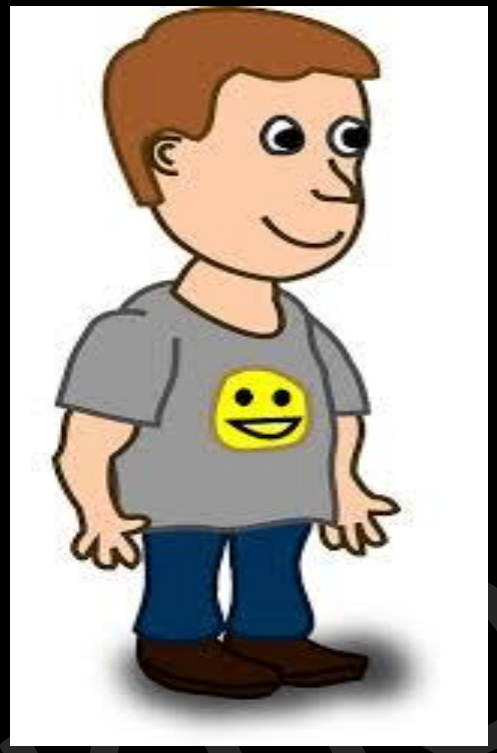
Chapter-11: Disk Management)- Disk Basics, Disk storage and disk scheduling, Total Transfer time.

(Chapter-12: File System)- File allocation Methods, Free-space Management, File organization and access mechanism, File directories, and File sharing, File system implementation issues, File system protection and security.

Knowledge Gate
Knowledge Gate Website

What is Operating System

1. **Intermediatory** – Acts as an **intermediary** between user & h/w .



Knowledge Gate

Knowledge Gate Website

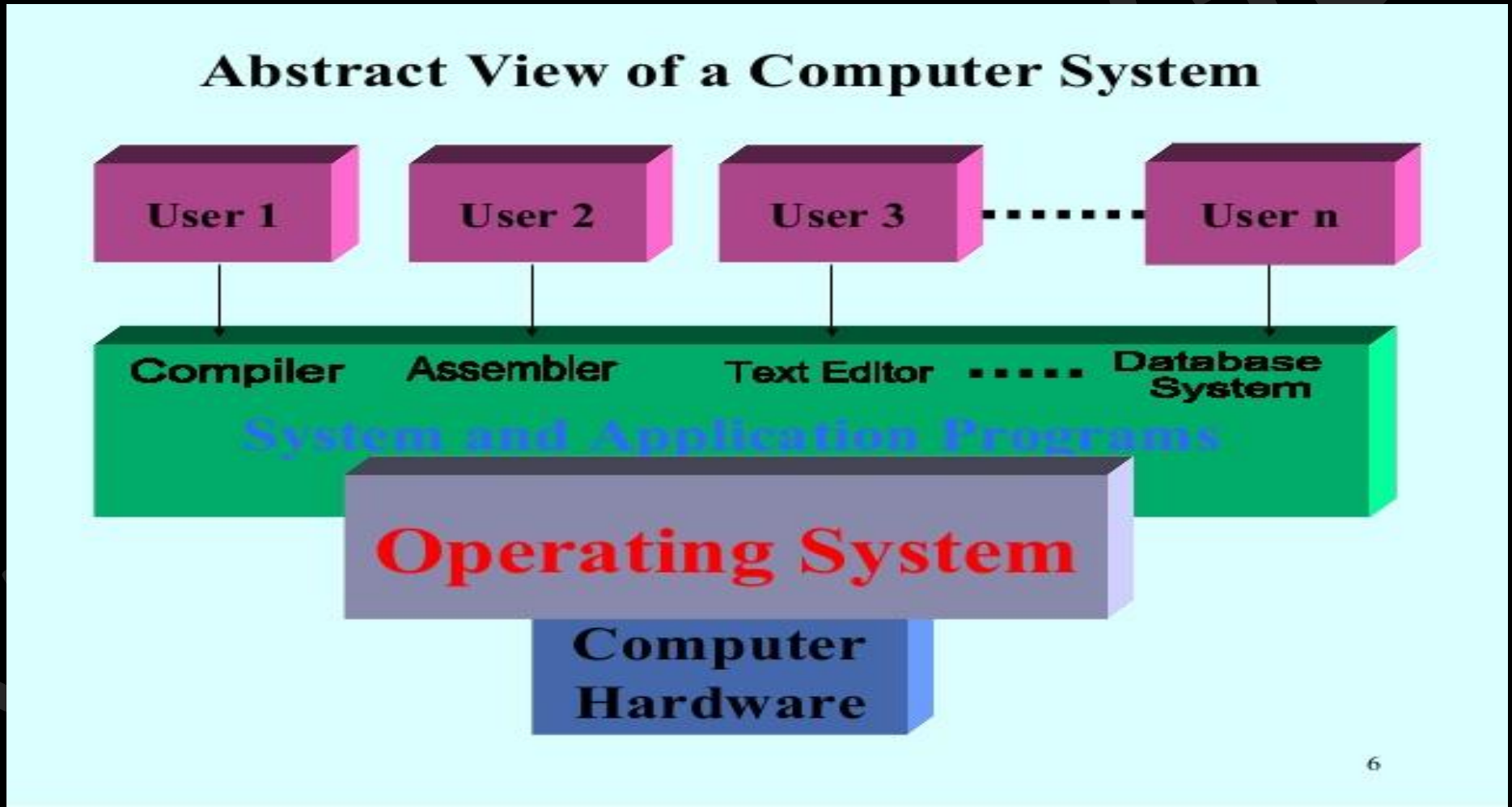
What is Operating System

2. <u>Resource Manager/Allocator</u> – Operating system controls and coordinates the use of system resources among various application programs in an unbiased fashion.

Knowledge Gate Website

What is Operating System

3. Platform - OS provides platform on which other application programs can be installed, provides the environment within which programs are executed.



Example

Knowledge Gate Website

Goals and Functions of operating system

- Goals are the ultimate destination, but we follow functions to implement goals.

सबका साथ सबका विकास

Ministries	Departments
58	93

Knowledge Gate

Knowledge Gate Website

Goals of operating system

1. Primary goals (Convenience / user friendly)

2. Secondary goals (Efficiency (Using resources in efficient manner) / Reliability / maintainability)

Knowledge Gate

Knowledge Gate Website

Functions of operating system

1. **Process Management**: Involves handling the creation, scheduling, and termination of processes, which are executing programs.

2. **Memory Management**: Manages allocation and deallocation of physical and virtual memory spaces to various programs.

3. **I/O Device Management**: Handles I/O operations of peripheral devices like disks, keyboards, etc., including buffering and caching.

4. **File Management**: Manages files on storage devices, including their information, naming, permissions, and hierarchy.

5. **Network Management**: Manages network protocols and functions, enabling the OS to establish network connections and transfer data.

6. **Security & Protection**: Ensures system protection against unauthorized access and other security threats through authentication, authorization, and encryption.

Knowledge Gate

Knowledge Gate Website

Major Components of operating system

1. **Kernel**
- **Central Component**: Manages the system's resources and communication between hardware and software.

2. **Process Management**
- **Process Scheduler**: Determines the execution of processes.
- **Process Control Block (PCB)**: Contains process details such as process ID, priority, status, etc.
- **Concurrency Control**: Manages simultaneous execution.

3. **Memory Management**
- **Physical Memory Management**: Manages RAM allocation.
- **Virtual Memory Management**: Simulates additional memory using disk space.
- **Memory Allocation**: Assigns memory to different processes.

4. **File System Management**
- **File Handling**: Manages the creation, deletion, and access of files and directories.
- **File Control Block**: Stores file attributes and control information.
- **Disk Scheduling**: Organizes the order of reading or writing to disk.

Knowledge Gate

Knowledge Gate Website

5. Device Management

- **Device Drivers**: Interface between the hardware and the operating system.
- **I/O Controllers**: Manage data transfer to and from peripheral devices.

6. Security and Access Control

- **Authentication**: Verifies user credentials.
- **Authorization**: Controls access permissions to files and directories.
- **Encryption**: Ensures data confidentiality and integrity.

7. User Interface

- **Command Line Interface (CLI)**: Text-based user interaction.
- **Graphical User Interface (GUI):** Visual, user-friendly interaction with the OS.

8. Networking

- **Network Protocols**: Rules for communication between devices on a network.
- **Network Interface**: Manages connection between the computer and the network.

Knowledge Gate

Knowledge Gate Website

Batch Operating System

1. Early computers were not interactive device, there user use to prepare a job which consist three parts
    1. Program
    2. Control information
    3. Input data
2. Only one job is given input at a time as there was no memory, computer will take the input then process it and then generate output.
3. Common input/output device were punch card or tape drives. So these devices were very slow, and processor remain ideal most of the time.

Knowledge Gate

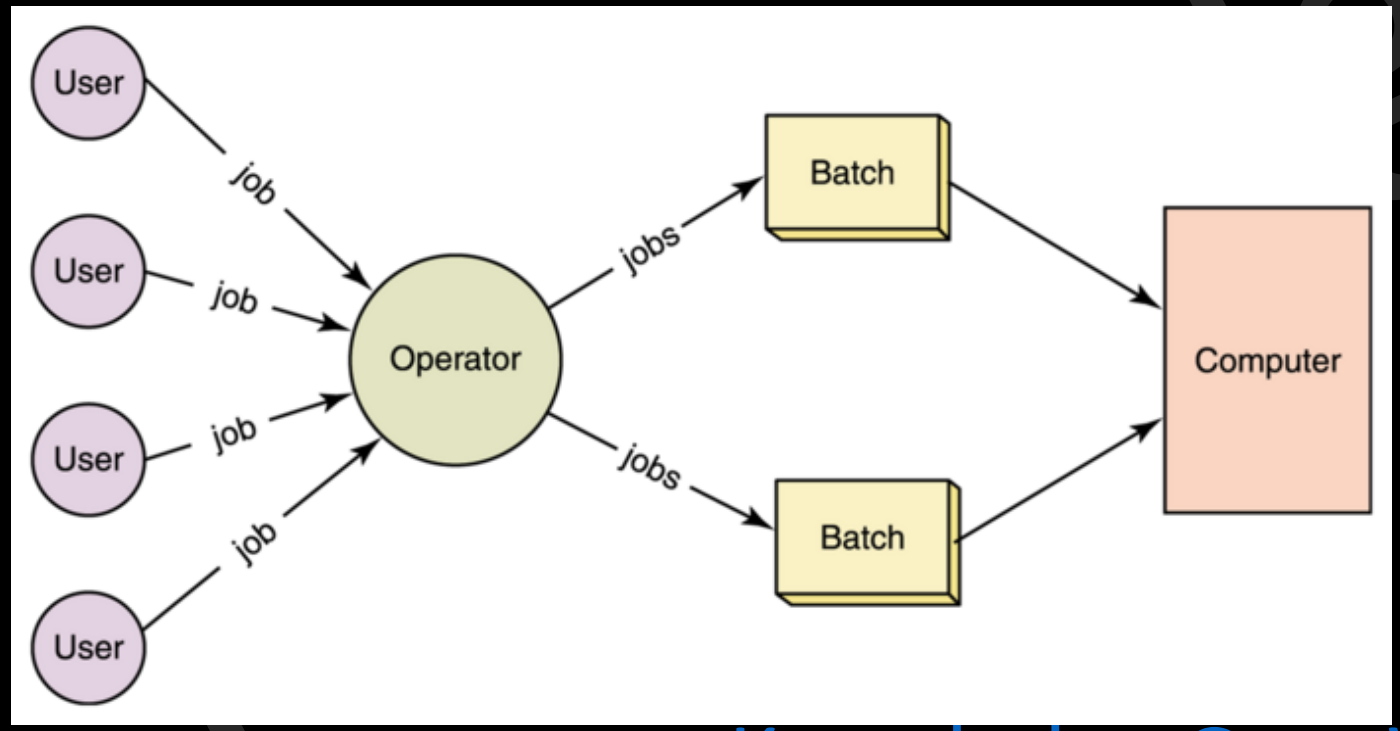
Knowledge Gate Website

Batch Operating System

4. To speed up the processing job with similar types (for e.g. FORTRAN jobs, COBOL jobs etc.) were batched together and were run through the processor as a group (batch).

5. In some system grouping is done by the operator while in some systems it is performed by the 'Batch Monitor' resided in the low end of main memory)

6. Then jobs (as a deck of punched cards) are bundled into batches with similar requirement.



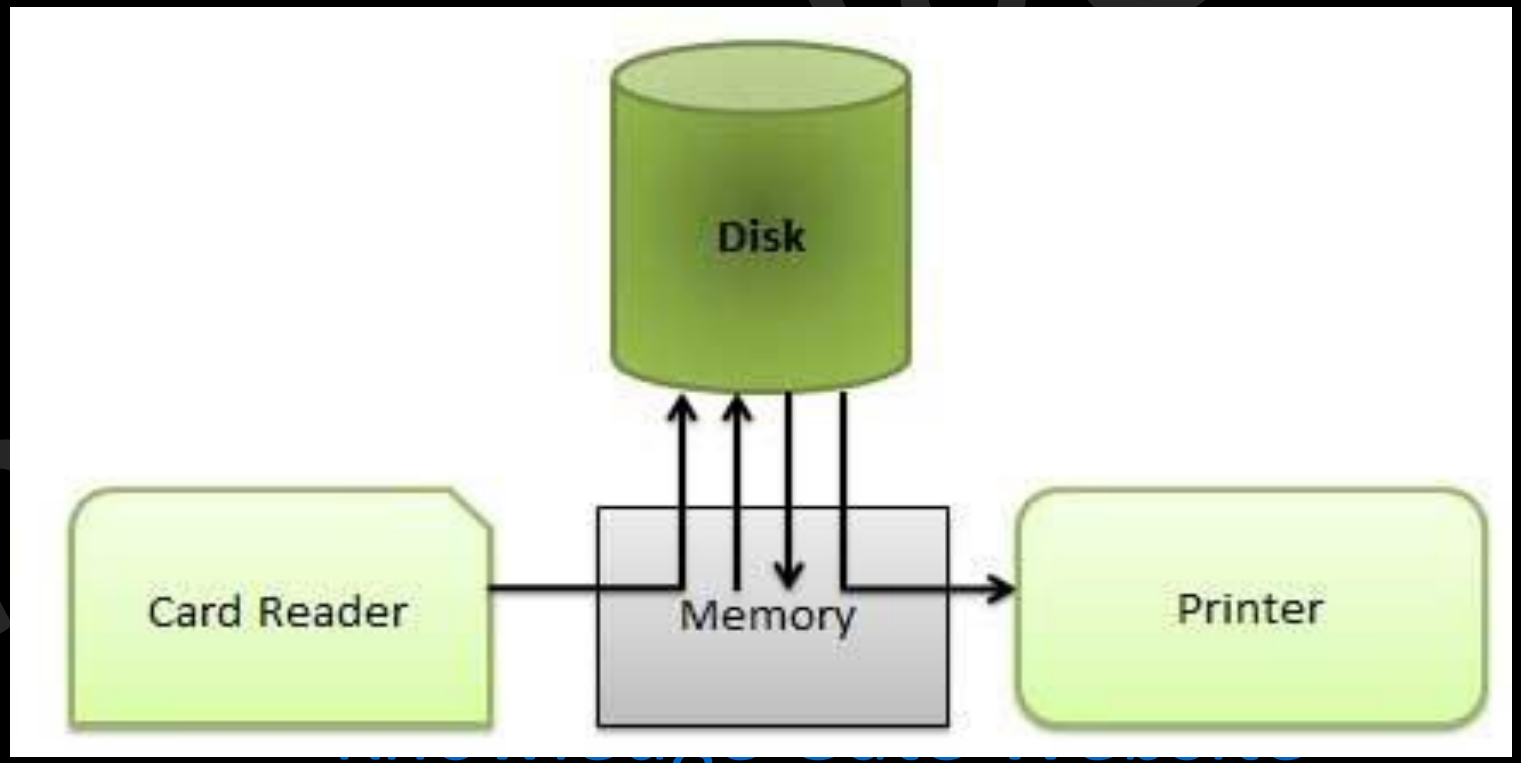
Knowledge Gate Website

Spooling

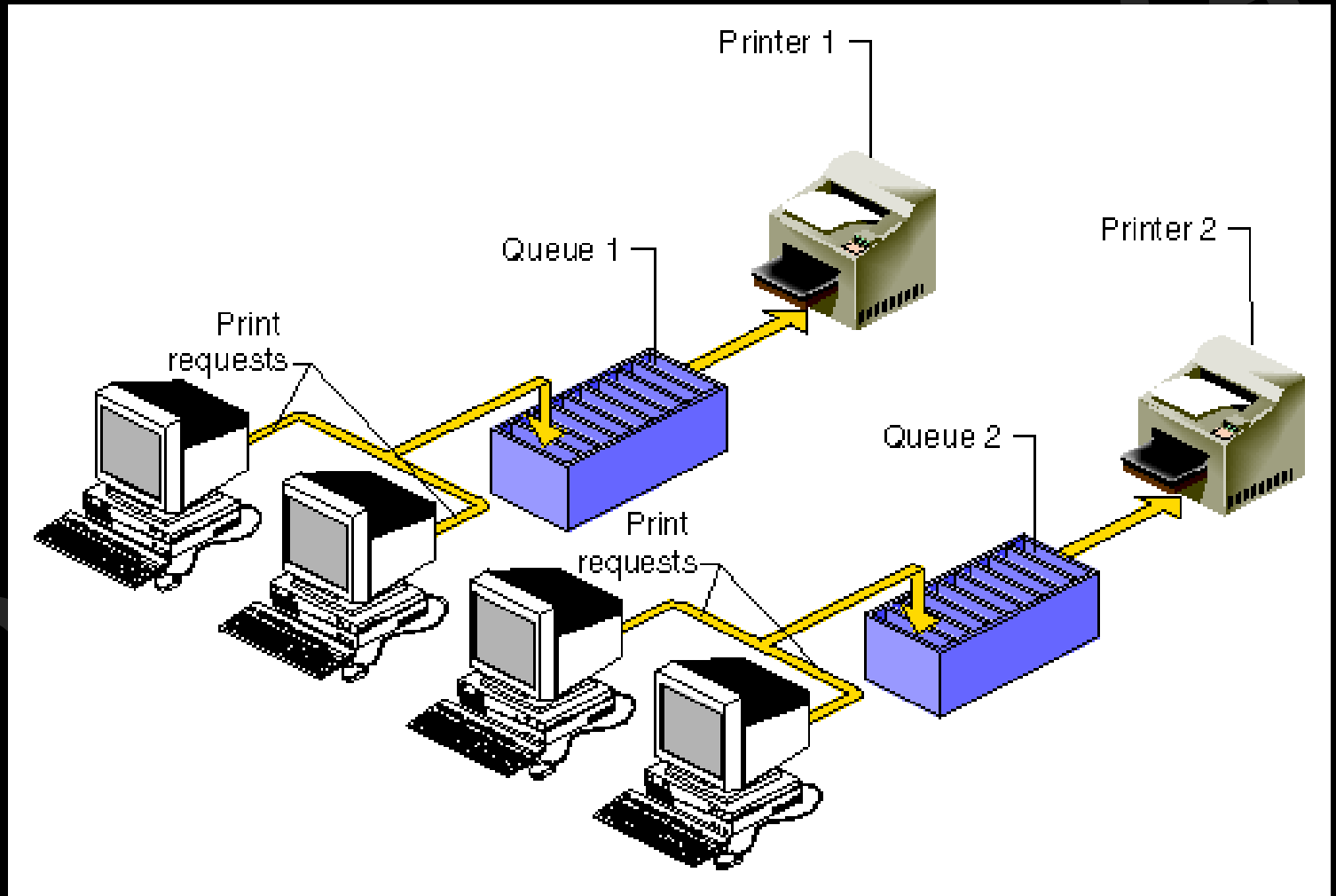
Simultaneous peripheral operations online

1. In a computer system input-output devices, such as printers are very slow relative to the performance of the rest of the system.

2. Spooling is a process in which data is temporarily held in memory or other volatile storage to be used by a device or a program.



3. The most common implementation of spooling can be found in typical input/output devices such as the keyboard, mouse and printer. For example, in printer spooling, the documents/files that are sent to the printer are first stored in the memory. Once the printer is ready, it fetches the data and prints it.

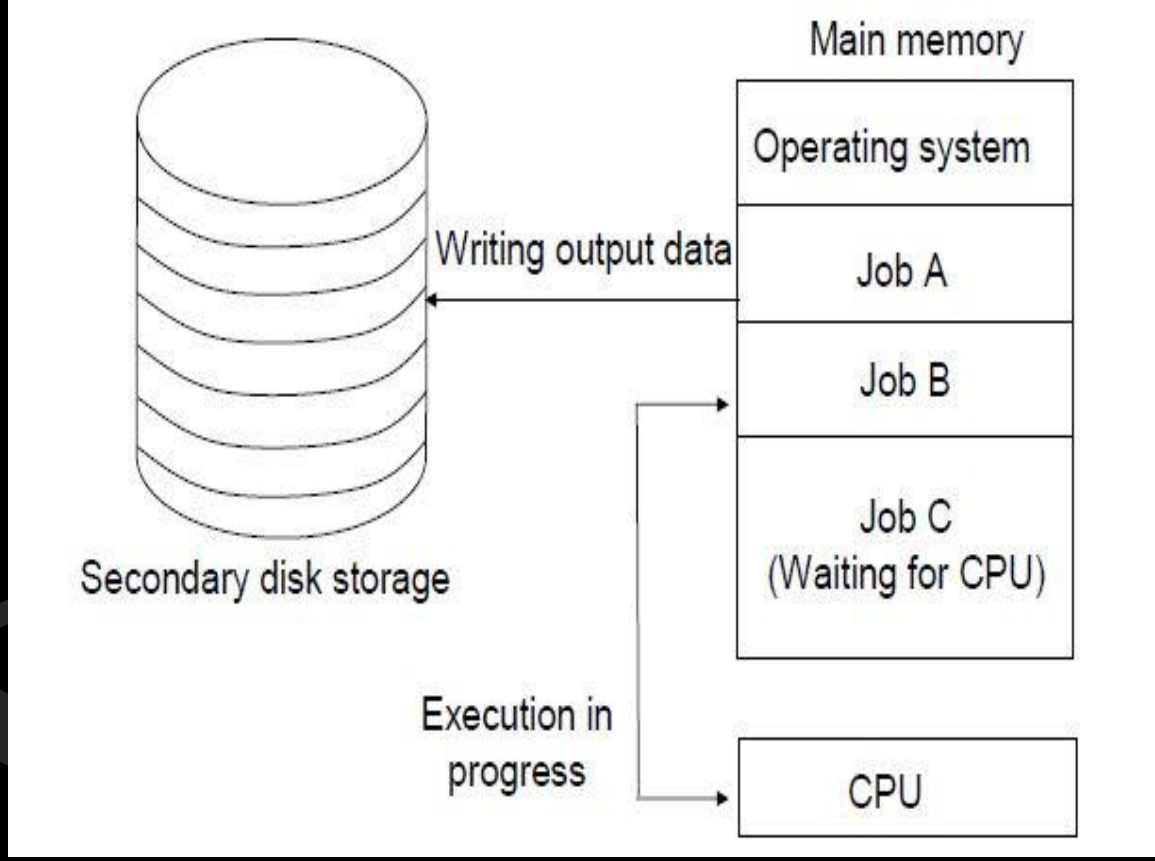


4. Ever had your mouse or keyboard freeze briefly? We often click around to test if it's working. When it unfreezes, all those stored clicks execute rapidly due to the device's spool.

Knowledge Gate Website

Multiprogramming Operating System

- **Multiple Jobs**: Keeps several jobs in main memory simultaneously, allowing more efficient utilization of the CPU.
- **Job Execution**: The OS picks and begins to execute one of the jobs in memory.
- **Waiting Jobs**: Eventually, a job may need to wait for a task, such as an I/O operation, to complete.

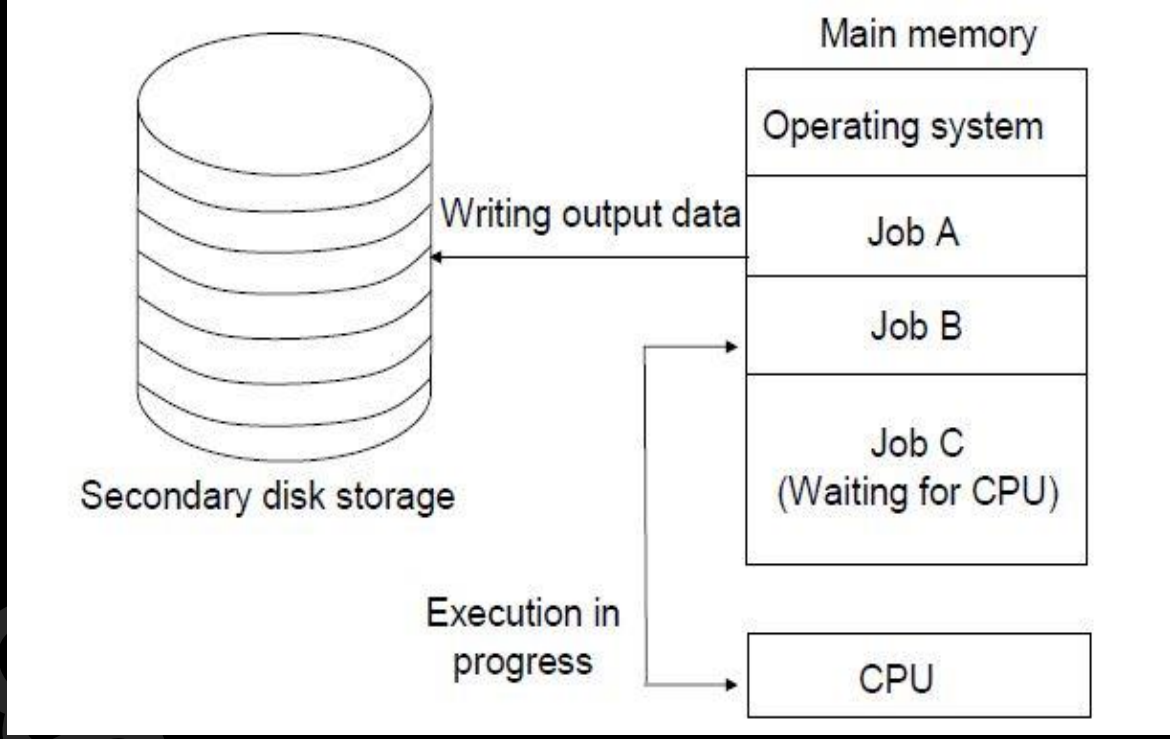


Processor किसी के लिए wait नहीं करेगा

Knowledge Gate Website

- **Non-Multiprogrammed**: CPU sits idle while waiting for a job to complete.

- **Multiprogrammed**: The OS switches to and executes another job if the current job needs to wait, utilizing the CPU effectively.



Show must go on

- **Conclusion**
- **Efficient Utilization**: Ensures that the CPU is never idle as long as at least one job needs to execute, leading to better utilization of resources.

Knowledge

Knowledge Gate Website

- **Advantages:**
  - **High CPU Utilization:** Enhances processing efficiency.
  - **Less Waiting Time:** Minimizes idle time.
  - **Multi-Task Handling:** Manages concurrent tasks effectively.
  - **Shared CPU Time:** Increases system efficiency.

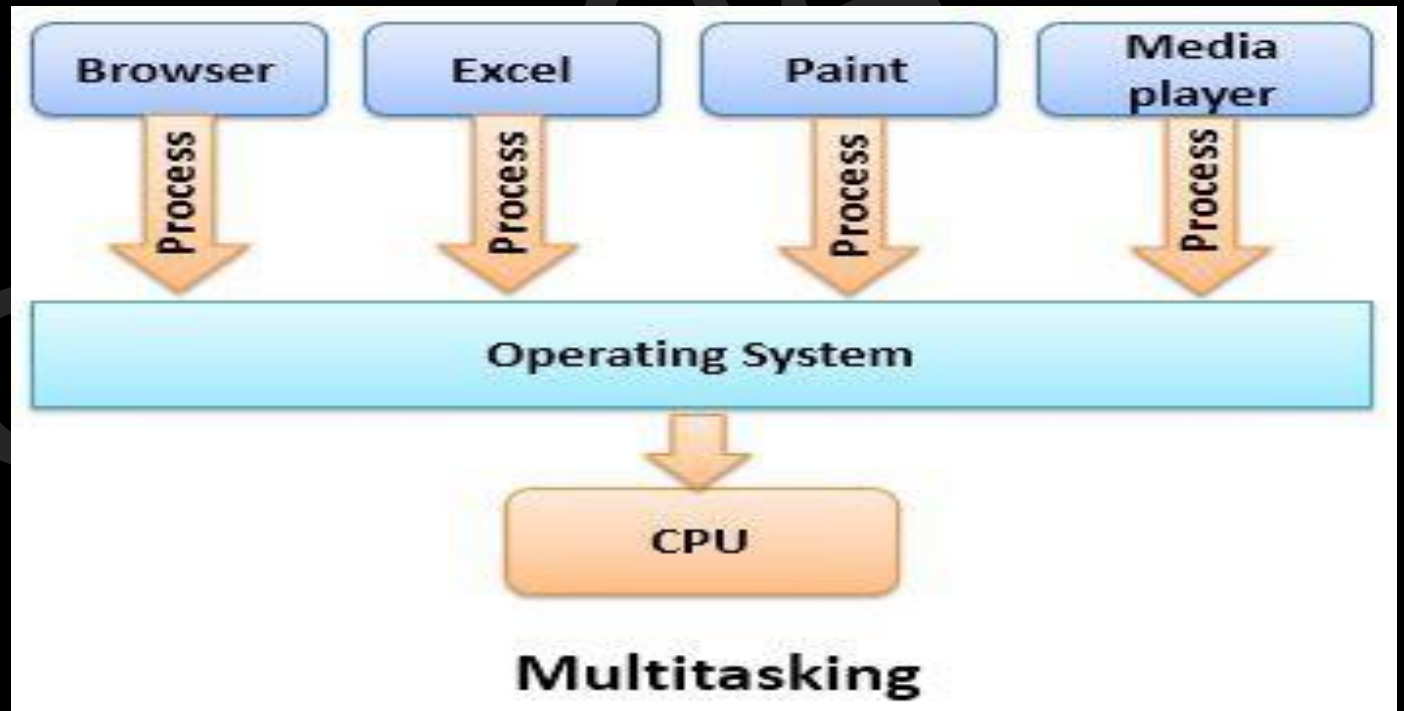
- **Disadvantages:**
  - **Complex Scheduling:** Difficult to program.
  - **Complex Memory Management:** Intricate handling of memory is required.

Knowledge Gate

Knowledge Gate Website

Multitasking Operating system/time sharing/Multiprogramming with Round Robin/ Fair Share

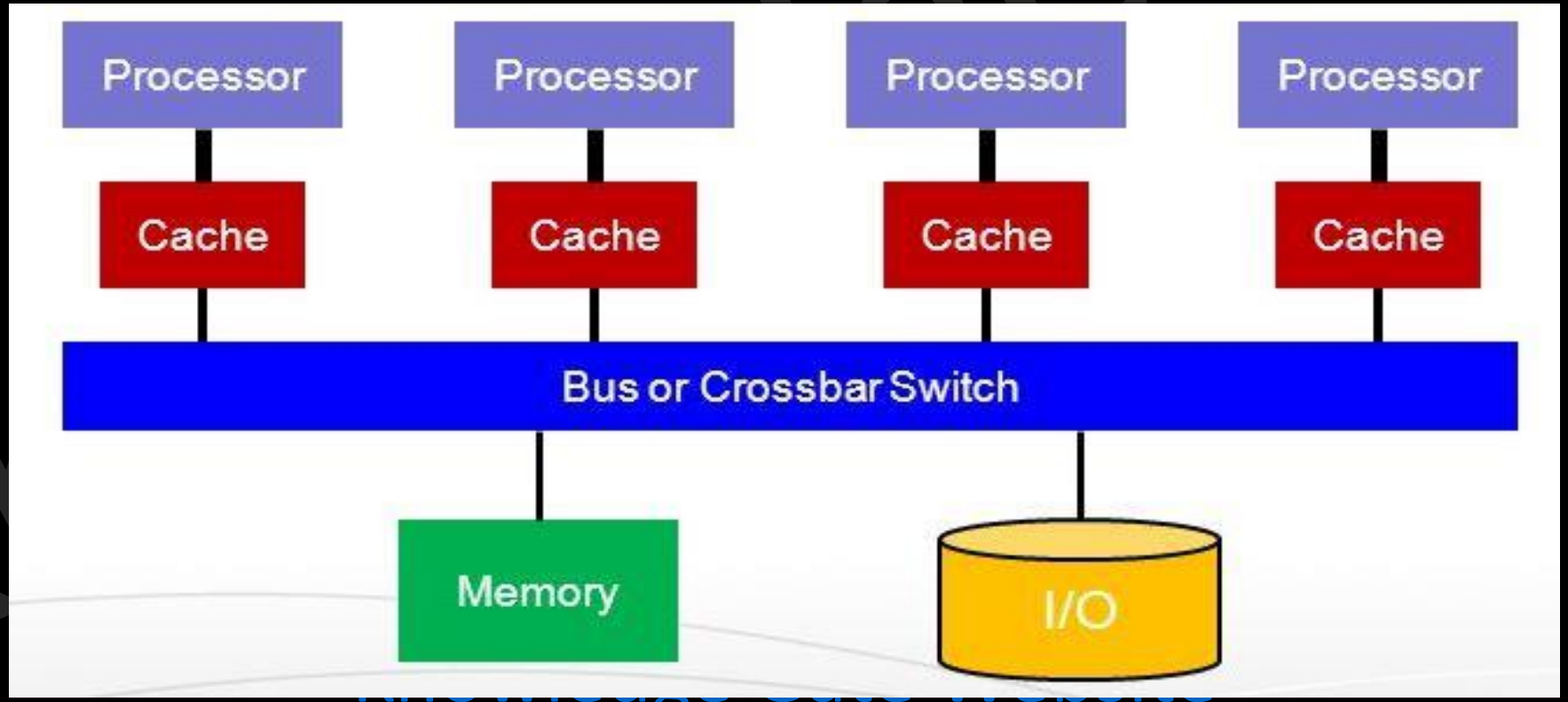
1. Time sharing (or multitasking) is a logical extension of multiprogramming, it allows many users to share the computer simultaneously. the CPU executes multiple jobs (May belong to different user) by switching among them, but the switches occur so frequently that, each user is given the impression that the entire computer system is dedicated to his/her use, even though it is being shared among many users.
2. In the modern operating systems, we are able to play MP3 music, edit documents in Microsoft Word, surf the Google Chrome all running at the same time. (by context switching, the illusion of parallelism is achieved)
3. For multitasking to take place, firstly there should be multiprogramming i.e. presence of multiple programs ready for execution. And secondly the concept of time sharing.



9

Multiprocessing Operating System/ tightly coupled system

1. Multiprocessor Operating System refers to the use of two or more central processing units (CPU) within a single computer system. These multiple CPU's share system bus, memory and other peripheral devices.
2. Multiple concurrent processes each can run on a separate CPU, here we achieve a true parallel execution of processes.
3. Becomes most important in computer system, where the complexity of the job is more, and CPU divides and conquers the jobs. Generally used in the fields like artificial intelligence and expert system, image processing, weather forecasting etc.



Point	Symmetric Processing	Asymmetric Processing
Definition	All processors are treated equally and can run any task.	Each processor is assigned a specific task or role.
Task Allocation	Any processor can perform any task.	Tasks are divided according to processor roles.
Complexity	Generally simpler as all processors are treated the same.	More complex due to the dedicated role of each processor.
Scalability	Easily scalable by adding more processors.	May require reconfiguration as processors are added.
Performance	Load is evenly distributed, enhancing performance.	Performance may vary based on the specialization of tasks.

Knowledge Gate

Knowledge Gate Website

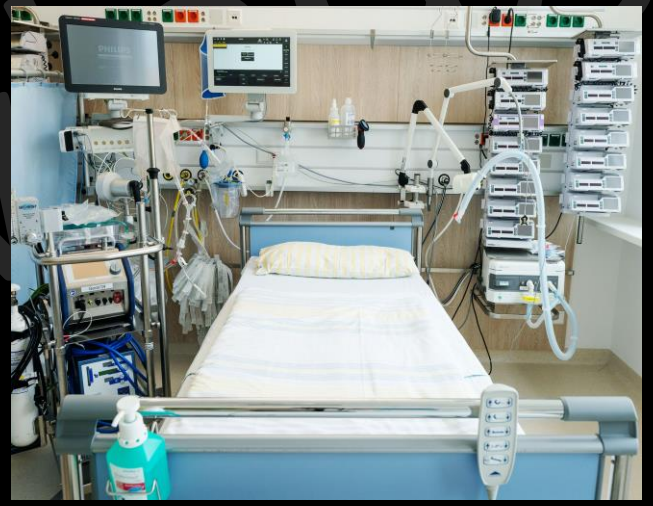
Point	Multi-Programming	Multi-Processing
Definition	Allows multiple programs to share a single CPU.	Utilizes multiple CPUs to run multiple processes concurrently.
Concurrency	Simulates concurrent execution by rapidly switching between tasks.	Achieves true parallel execution of processes.
Resource Utilization	Maximizes CPU utilization by keeping it busy with different tasks.	Enhances performance by allowing tasks to be processed simultaneously.
Hardware Requirements	Requires only one CPU and manages multiple tasks on it.	Requires multiple CPUs, enabling parallel processing.
Complexity and Coordination	Less complex, primarily managing task switching on one CPU.	More complex, requiring coordination among multiple CPUs.

KnowledgeGate

Knowledge Gate Website

Real time Operating system

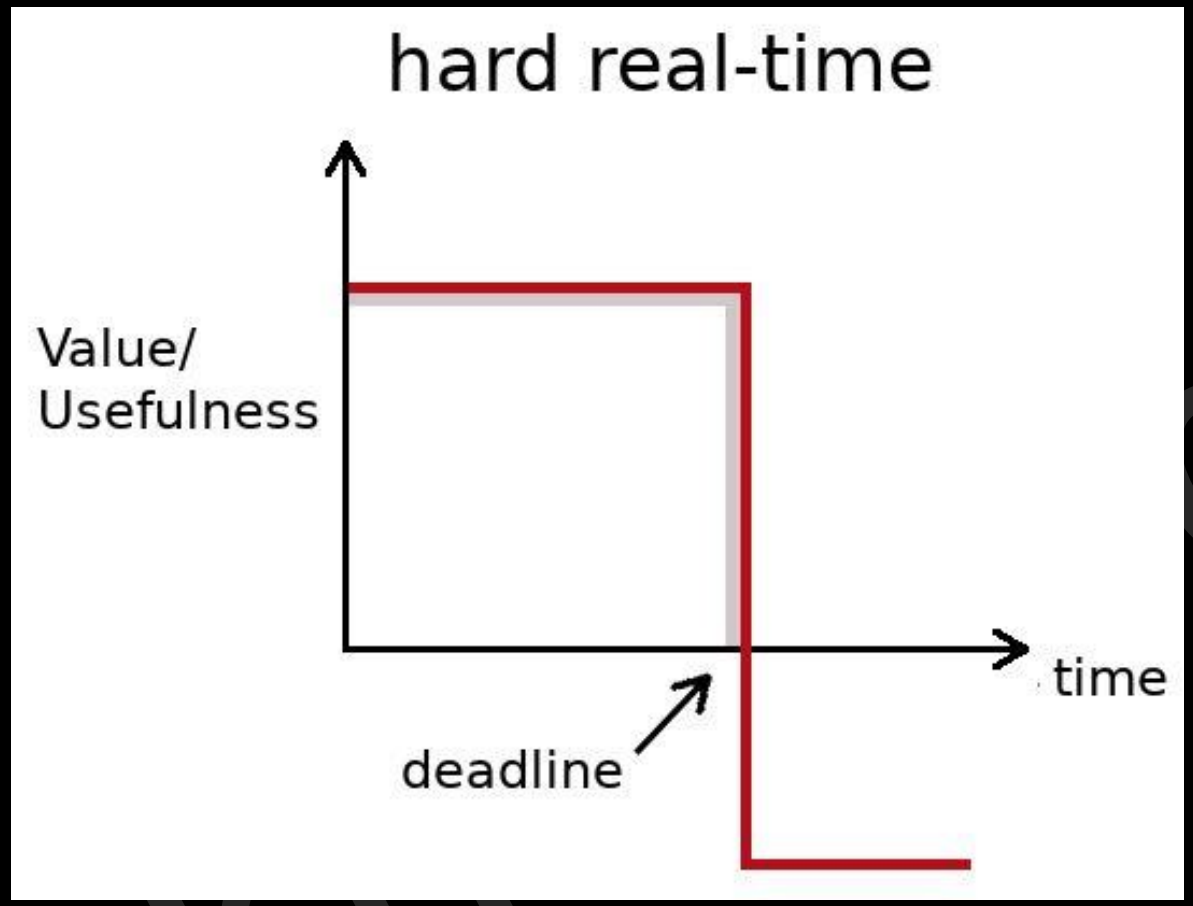
1. A real time operating system is a special purpose operating system which has well defined fixed time constraints. Processing must be done within the defined time limit or the system will fail.
2. Valued more for how quickly or how predictably it can respond, without buffer delays than for the amount of work it can perform in a given period of time.
3. For example, a petroleum refinery, Airlines reservation system, Air traffic control system, Systems that provide up to the minute information on stock prices, Defense application systems like as RADAR.



Knowledge Gate

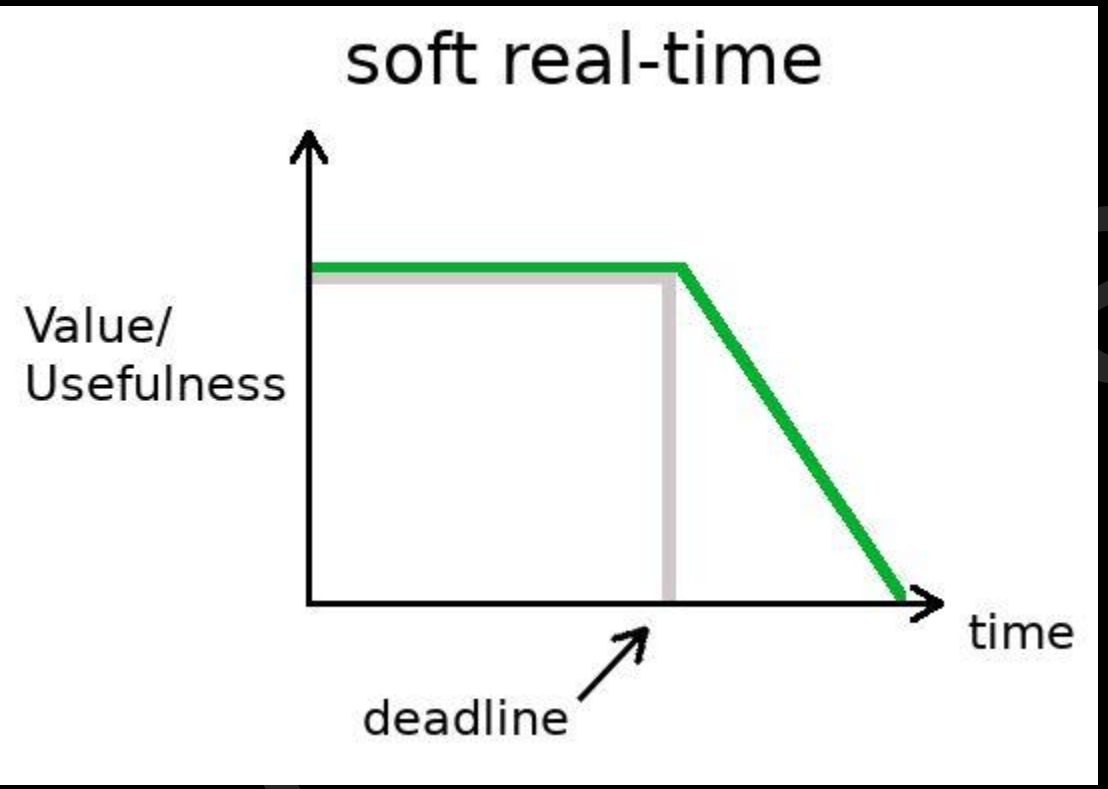
Knowledge Gate Website

- **Hard real-time operating system** - This is also a type of OS and it is predicted by a deadline. The predicted deadlines will react at a time t = 0. Some examples of this operating system are air bag control in cars.



Knowledge Gate Website

- **Soft real-time operating system** - The soft real-time operating system has certain deadlines, may be missed and they will take the action at a time t=0+. The critical time of this operating system is delayed to some extent. The examples of this operating system are the digital camera, mobile phones and online data etc.



Knowledge Gate Website

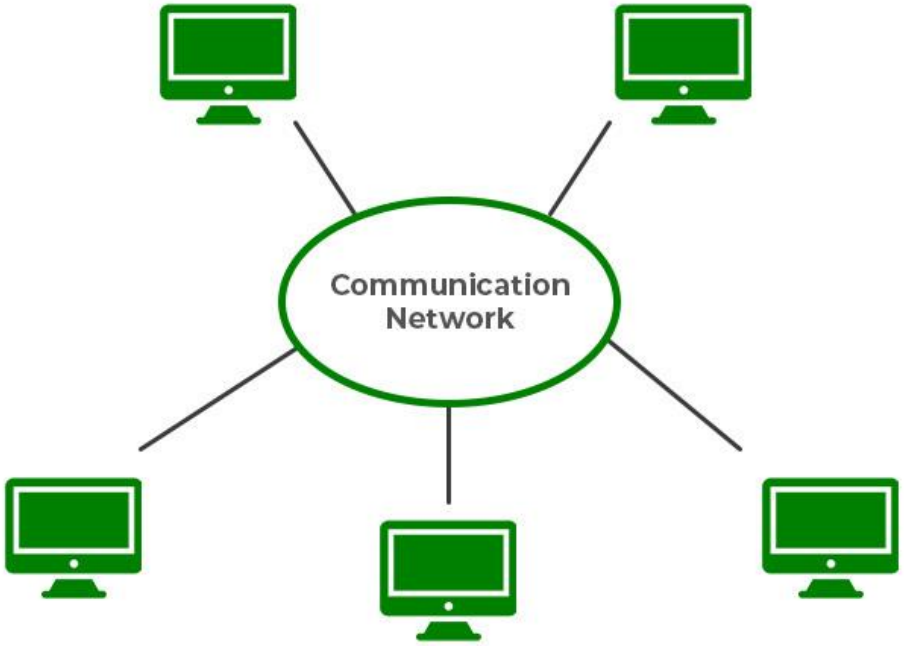
Point	Hard Real-Time Operating System	Soft Real-Time Operating System
Deadline Constraints	Must meet strict deadlines without fail.	Can miss deadlines occasionally without failure.
Response Time	Fixed and guaranteed.	Predictable, but not guaranteed.
Applications	Used in life-critical systems like medical devices, nuclear reactors.	Used in multimedia, user interfaces, etc.
Complexity and Cost	Typically more complex and costlier.	Less complex and usually less expensive.
Reliability	Must be highly reliable and fault-tolerant.	High reliability desired, but some failures are tolerable.

Knowledge Gate

Knowledge Gate Website

Distributed OS

1. A **distributed operating system** is a software over a collection of independent, networked, communicating, loosely coupled nodes and physically separate computational nodes.
2. The nodes communicate with one another through various networks, such as high-speed buses and the Internet. They handle jobs which are serviced by multiple CPUs. Each individual node holds a specific software subset of the global aggregate operating system.
3. There are four major reasons for building distributed systems: resource sharing, computation speedup, reliability, and communication.



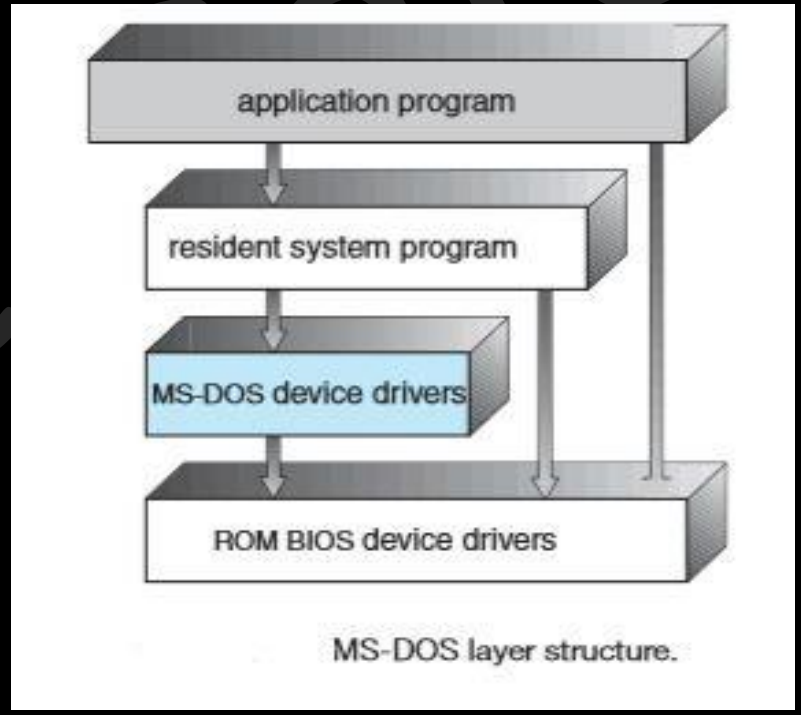
Knowledge Gate

Structure of Operating System

- A common approach is to partition the task into small components, or modules, rather than have one monolithic system. Each of these modules should be a well-defined portion of the system, with carefully defined inputs, outputs, and functions.

- **Simple Structure** - Many operating systems do not have well-defined structures. Frequently, such systems started as small, simple, and limited systems and then grew beyond their original scope. MS-DOS is an example of such a system.

- Not divided into modules. Its interface , levels and functionality are not well separated

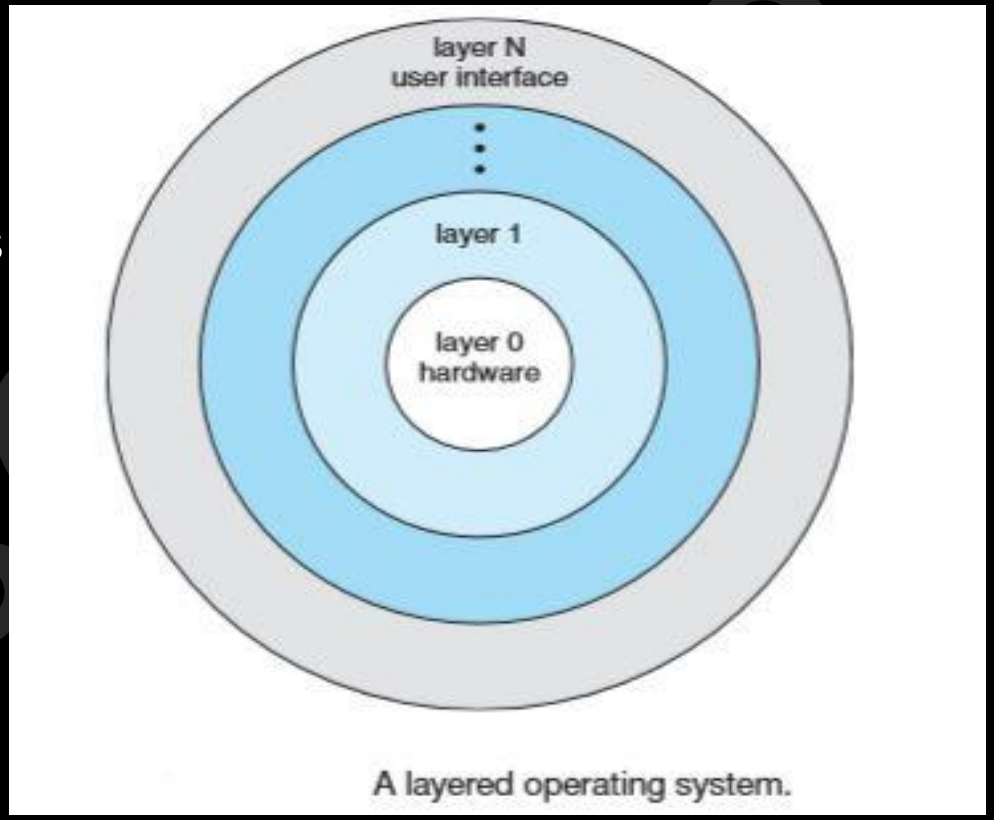


MS-DOS layer structure.

Knowledge Gate Website

- **Layered Approach** - With proper hardware support, operating systems can be broken into pieces. The operating system can then retain much greater control over the computer and over the applications that make use of that computer.

1. Implementers have more freedom in changing the inner workings of the system and in creating modular operating systems.
2. Under a top-down approach, the overall functionality and features are determined and are separated into components.
3. Information hiding is also important, because it leaves programmers free to implement the low-level routines as they see fit.
4. A system can be made modular in many ways. One method is the layered approach, in which the operating system is broken into a number of layers (levels). The bottom layer (layer 0) is the hardware; the highest (layer *N*) is the user interface.

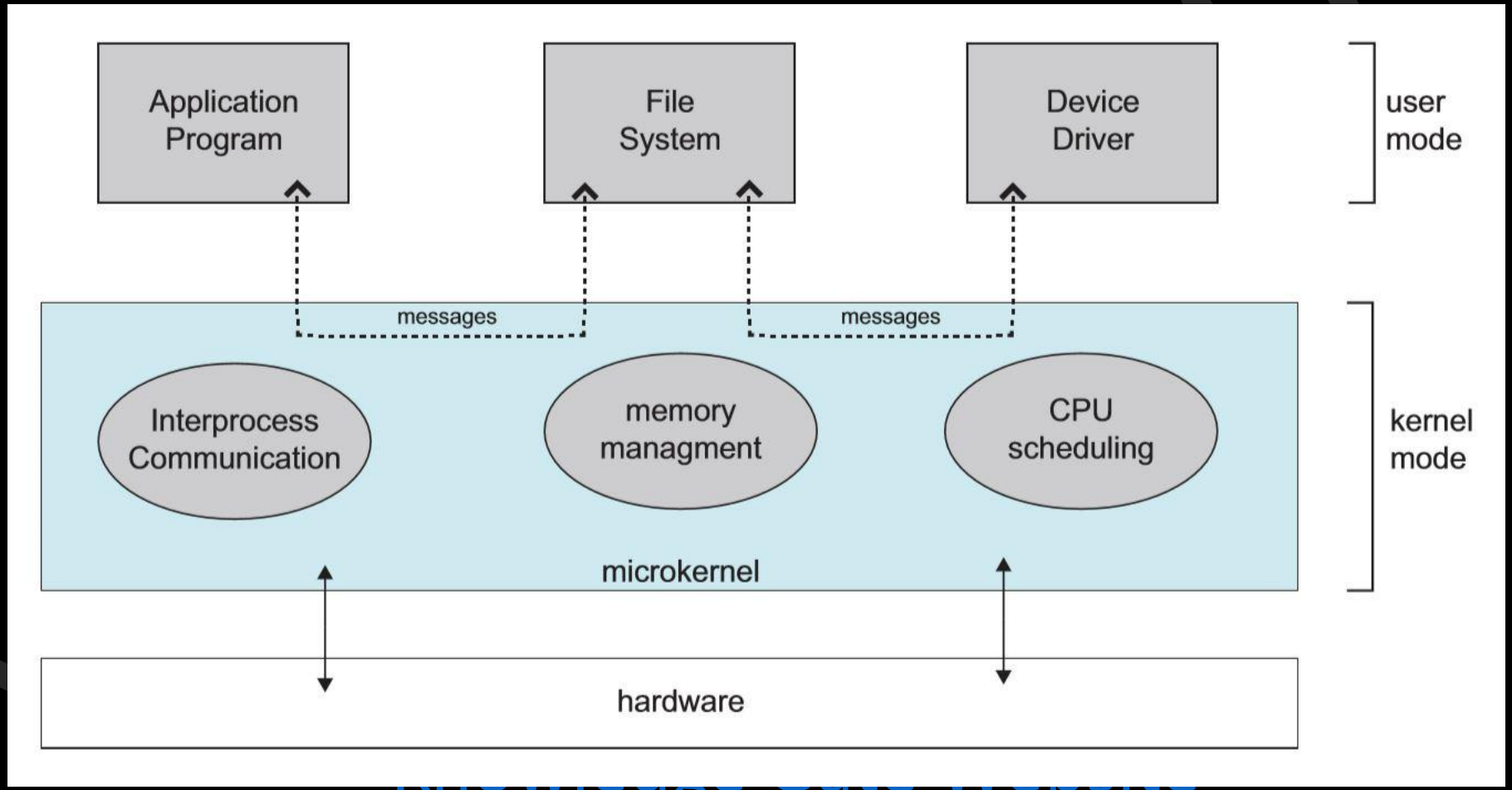


A layered operating system.

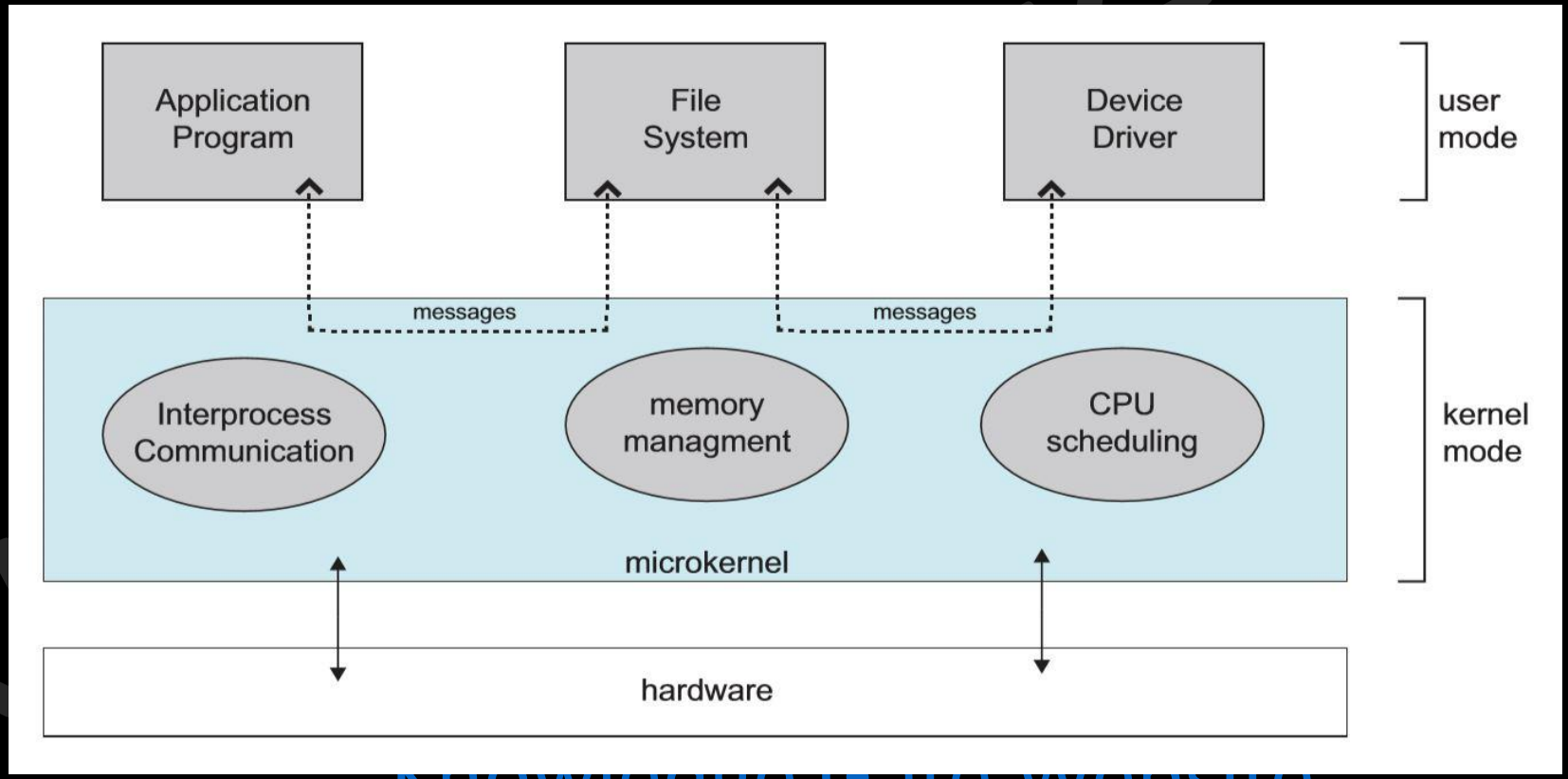
Knowledge Gate Website

Micro-Kernel approach

- In the mid-1980s, researchers at Carnegie Mellon University developed an operating system called **Mach** that modularized the kernel using the **microkernel** approach.
- This method structures the operating system by removing all nonessential components from the kernel and implementing them as system and user-level programs. The result is a smaller kernel.



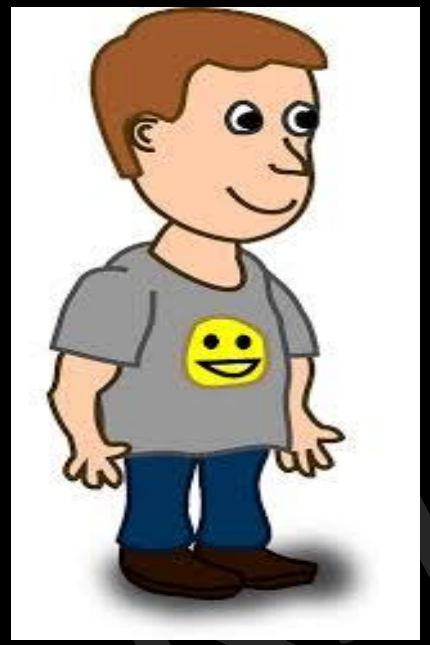
- One benefit of the microkernel approach is that it makes extending the operating system easier. All new services are added to user space and consequently do not require modification of the kernel.
- When the kernel does have to be modified, the changes tend to be fewer, because the microkernel is a smaller kernel.
- The MINIX 3 microkernel, for example, has only approximately 12,000 lines of code. Developer Andrew S. Tanenbaum



Knowledge Gate Website

User and Operating-System Interface

- There are several ways for users to interface with the operating system. Here, we discuss two fundamental approaches.
  - Command-line interface, or command interpreter.
  - Graphical User Interfaces.



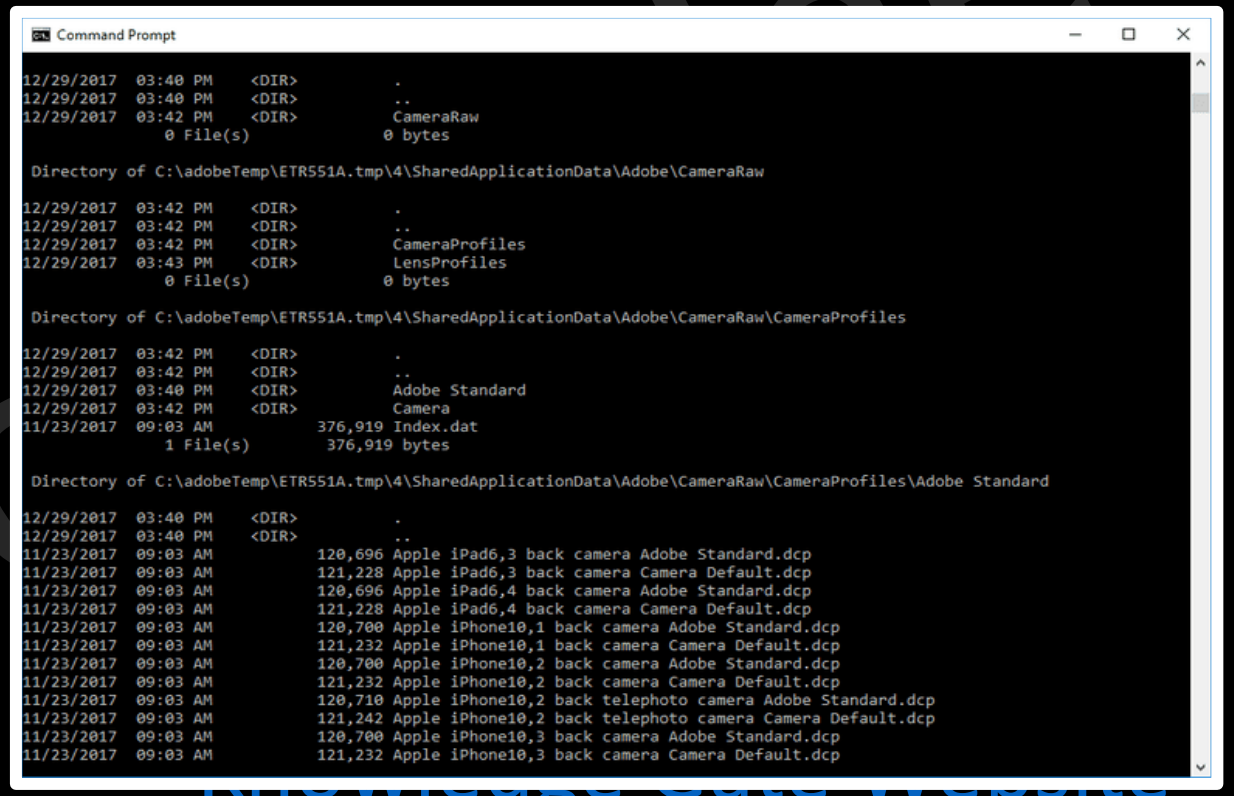
?

Operating System

Knowledge Gate

Knowledge Gate Website

- **Command Interpreters** - Some operating systems include the command interpreter in the kernel. Others, such as Windows and UNIX, treat the command interpreter as a special program that is running when a job is initiated or when a user first logs on (on interactive systems).
- On systems with multiple command interpreters to choose from, the interpreters are known as shells. For example, on UNIX and Linux systems, a user may choose among several different shells, including the *Bourne shell*, *C shell*, *Bourne-Again shell*, *Korn shell*, and others.



Knowledge Gate
Knowledge Gate Website

- **Graphical User Interfaces** - A second strategy for interfacing with the operating system is through a user- friendly graphical user interface, or GUI. Here, users employ a mouse-based window- and-menu system characterized by a desktop.
- The user moves the mouse to position its pointer on images, or icons, on the screen (the desktop) that represent programs, files, directories, and system functions. Depending on the mouse pointer's location, clicking a button on the mouse can invoke a program, select a file or directory—known as a folder —or pull down a menu that contains commands.

Knowledge Gate Website

- Because a mouse is impractical for most mobile systems, smartphones and handheld tablet computers typically use a touchscreen interface.
- Here, users interact by making gestures on the touchscreen—for example, pressing and swiping fingers across the screen.

Knowledge Gate Website

- The choice of whether to use a command-line or GUI interface is mostly one of personal preference.
- System administrators who manage computers and power users who have deep knowledge of a system frequently use the command-line interface. For them, it is more efficient, giving them faster access to the activities they need to perform.
- Indeed, on some systems, only a subset of system functions is available via the GUI, leaving the less common tasks to those who are command-line knowledgeable.

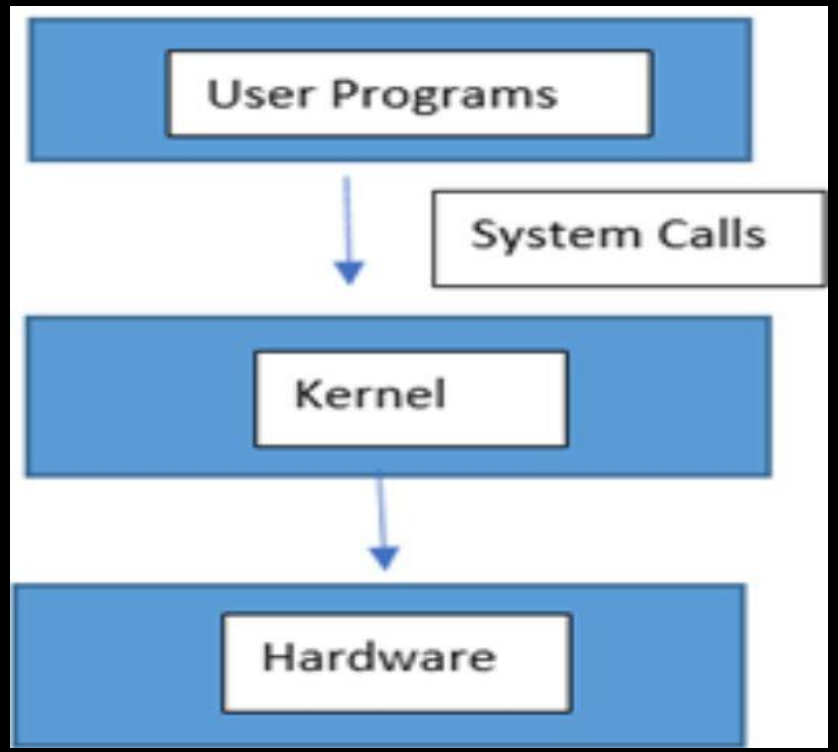
Knowledge Gate Website

System call

- System calls provide the means for a user program to ask the operating system to perform tasks reserved for the operating system on the user program's behalf.

- System calls provide an interface to the services made available by an operating system. These calls are generally available as routines written in C and C++.

- The API specifies a set of functions that are available to an application programmer, including the parameters that are passed to each function and the return values the programmer can expect.

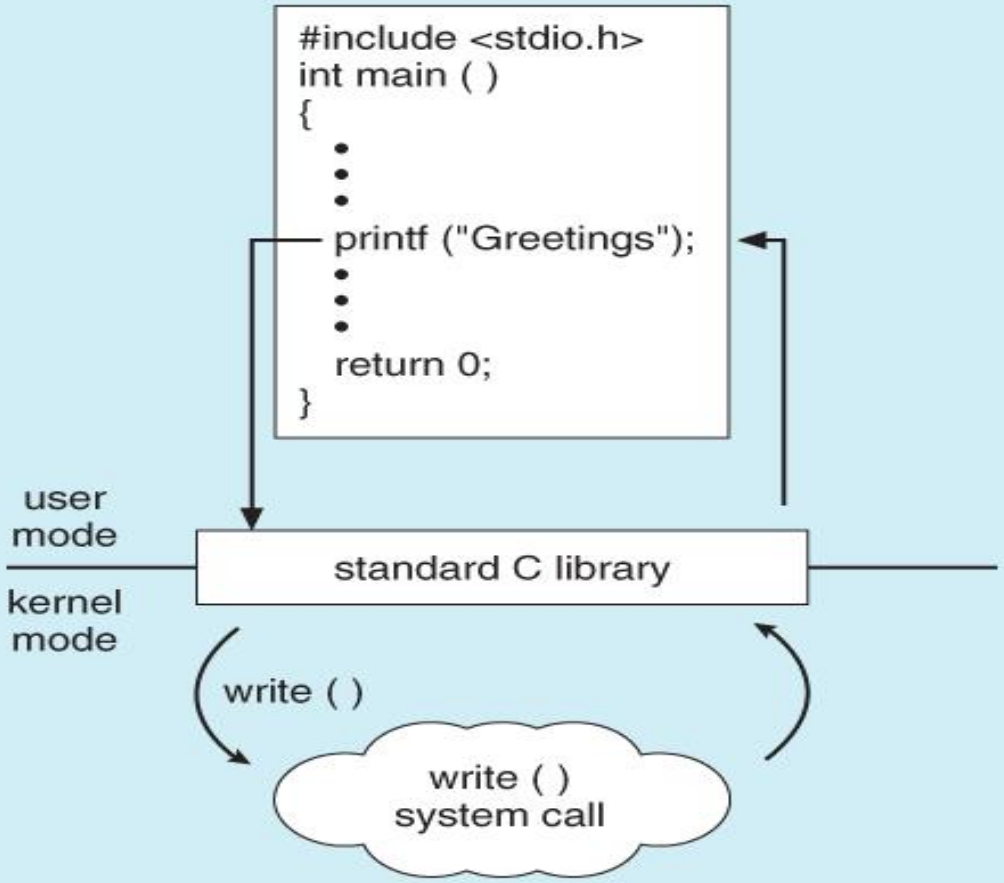


Knowledge

Knowledge Gate Website

EXAMPLE OF STANDARD C LIBRARY

The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. For example, let's assume a C program invokes the printf( ) statement. The C library intercepts this call and invokes the necessary ssystem call(s) in the operating system - in this instance, the write( ) system call. The C library takes the value returned by write( ) and passes it back to the user program. This is shown below:



- **Types of System Calls** - System calls can be grouped roughly into six major categories: process control, file manipulation, device manipulation, information maintenance, communications, and protection.
- **Process control**
    1. end, abort
    2. load, execute
    3. create process, terminate process
    4. get process attributes, set process attributes
    5. wait for time
    6. wait event, signal event
    7. allocate and free memory

Knowledge Gate

Knowledge Gate Website

- **File management**

1. create file, delete file
2. open, close
3. read, write, reposition
4. get file attributes, set file attributes

Knowledge Gate

Knowledge Gate Website

- **Device management**

1. request device, release device
2. read, write, reposition
3. get device attributes, set device attributes
4. logically attach or detach devices

Knowledge Gate

Knowledge Gate Website

- **Information maintenance**

1. get time or date, set time or date
2. get system data, set system data
3. get process, file, or device attributes
4. set process, file, or device attributes

Knowledge Gate

Knowledge Gate Website

- **Communications**

1. create, delete communication connection

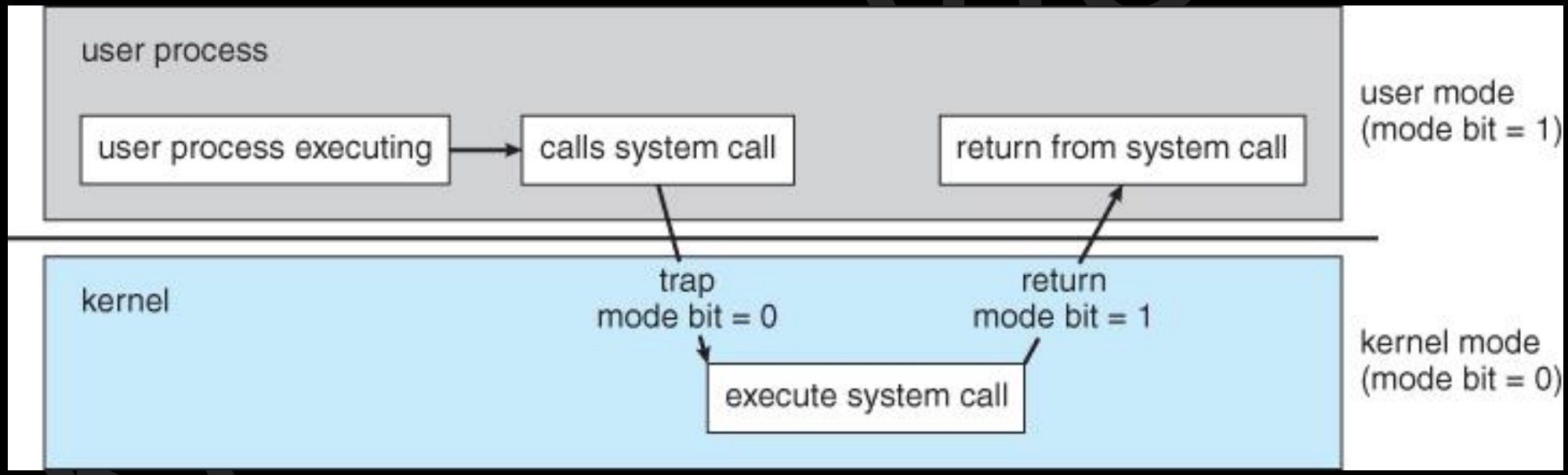
2. send, receive messages transfer status information

Knowledge Gate

Knowledge Gate Website

Mode

- We need two separate *modes* of operation: User mode and Kernel mode (also called supervisor mode, system mode, or privileged mode). A bit, called the mode bit, is added to the hardware of the computer to indicate the current mode: kernel (0) or user (1).
- When the computer system is executing on behalf of a user application, the system is in user mode. However, when a user application requests a service from the operating system (via a system call), the system must transition from user to kernel mode to fulfill the request.



Knowledge Gate Website

User Mode

Kernel Mode

[Knowledge Gate Website]

Break

Knowledge Gate

Knowledge Gate Website

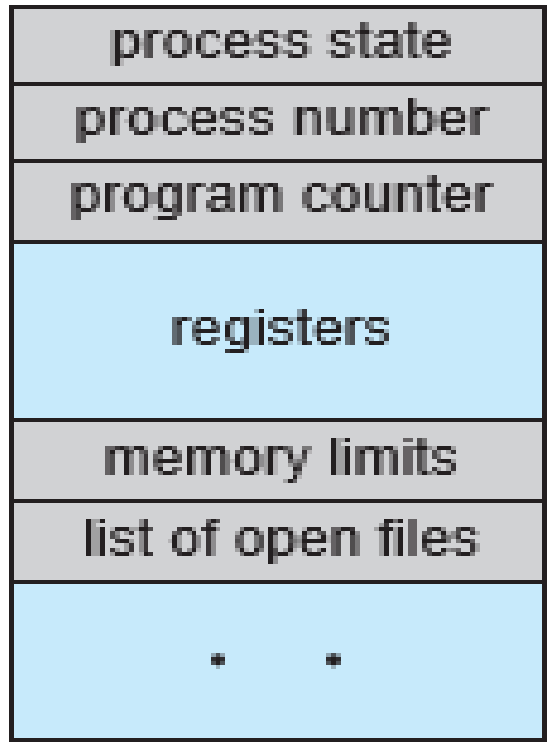
Process

In general, a process is a program in execution.

A Program is not a Process by default. A program is a passive entity, i.e. a file containing a list of instructions stored on disk (secondary memory) (often called an executable file).

A program becomes a Process when an executable file is loaded into main memory and when it's PCB is created.

A process on the other hand is an Active Entity, which require resources like main memory, CPU time, registers, system buses etc.



Knowledge Gate

Knowledge Gate Website

Even if two processes may be associated with same program, they will be considered as two separate execution sequences and are totally different process.

For instance, if a user has invoked many copies of web browser program, each copy will be treated as separate process. even though the text section is same but the data, heap and stack sections can vary.

Knowledge Gate

Knowledge Gate Website

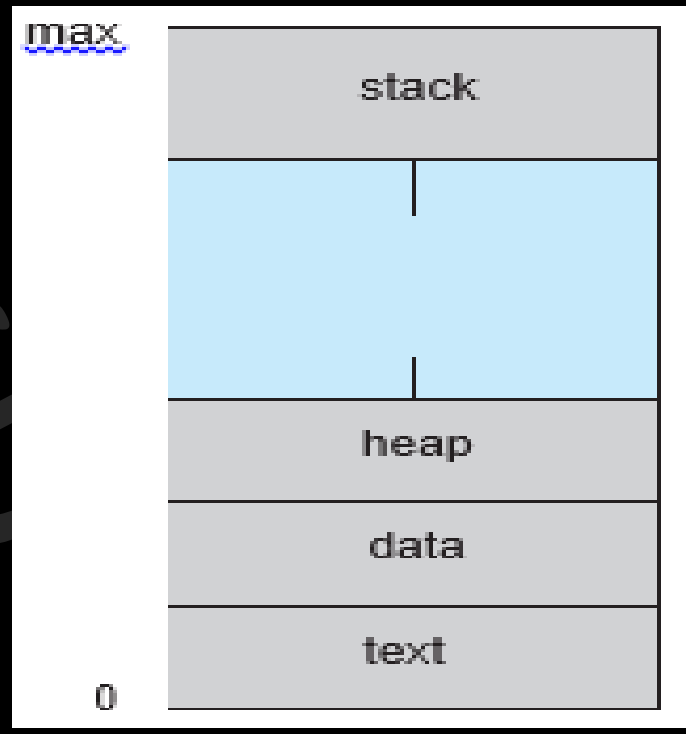
A Process consists of following sections:

Text section: Also known as Program Code.

Stack: Which contains the temporary data (Function Parameters, return addresses and local variables).

Data Section: Containing global variables.

Heap: Which is memory dynamically allocated during process runtime.



Knowledge G

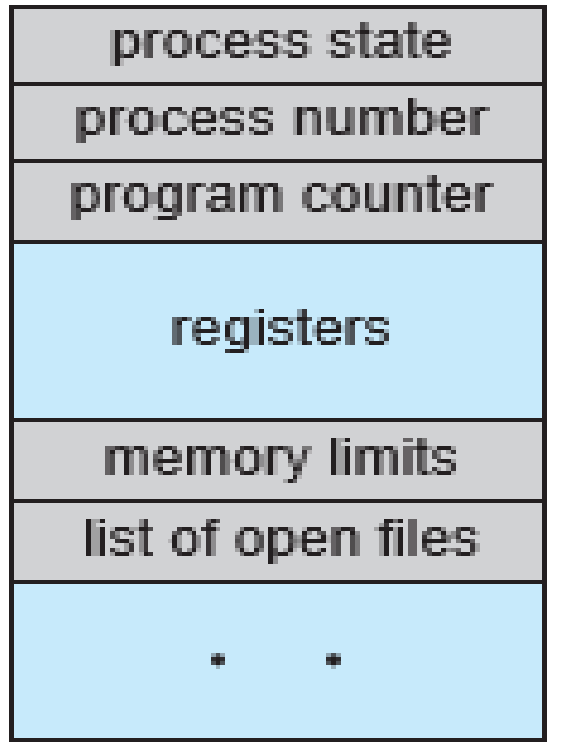
Knowledge Gate Website

Point	Program	Process
Definition	A set of instructions written to perform a specific task.	An instance of a program being executed.
State	Static; exists as code on disk or in storage.	Dynamic; exists in memory and has a state (e.g., running, waiting).
Resources	Does not require system resources when not running.	Requires CPU time, memory, and other resources during execution.
Independence	Exists independently and is not executing.	Can operate concurrently with other processes.
Interaction	Does not interact with other programs or the system.	Can interact with other processes and the operating system through system calls and inter-process communication.

Knowledge Gate

Knowledge Gate Website

Process Control Block (PCB)



Each process is represented in the operating system by a process control block (PCB) — also called a task control block.

PCB simply serves as the repository for any information that may vary from process to process. It contains many pieces of information associated with a specific process, including these:

1. Process state: The state may be new, ready, running, waiting, halted, and so on.

2. Program counter: The counter indicates the address of the next instruction to be executed for this process.

3. CPU registers: The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information. Along with the program counter, this state information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward.

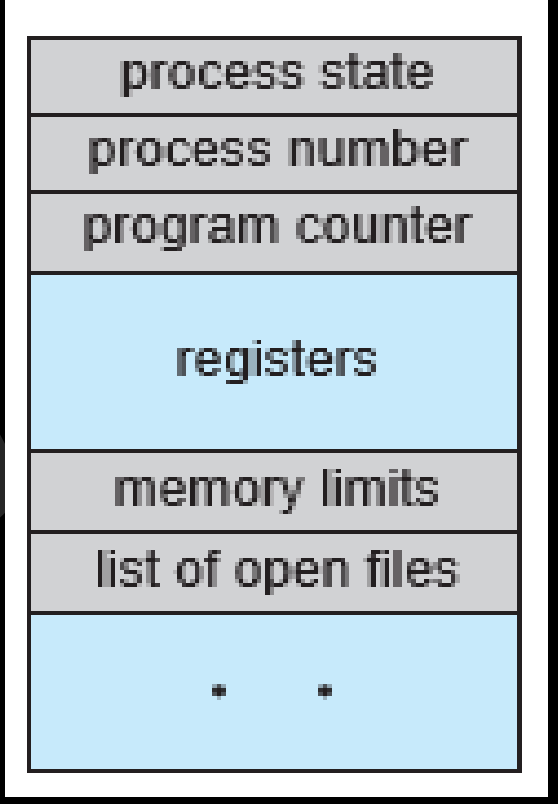
Knowledge Gate Website

4. CPU-scheduling information: This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.

5. Memory-management information: This information may include such items as the value of the base and limit registers and the page tables, or the segment tables, depending on the memory system used by the operating system.

6. Accounting information: This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.

7. I/O status information: This information includes the list of I/O devices allocated to the process, a list of open files, and so on.



Knowledge Gate Website

The Human Life Cycle

Knowledge Gate Website

Process States

A Process changes states as it executes. The state of a process is defined in parts by the current activity of that process. A process may be in one of the following states:

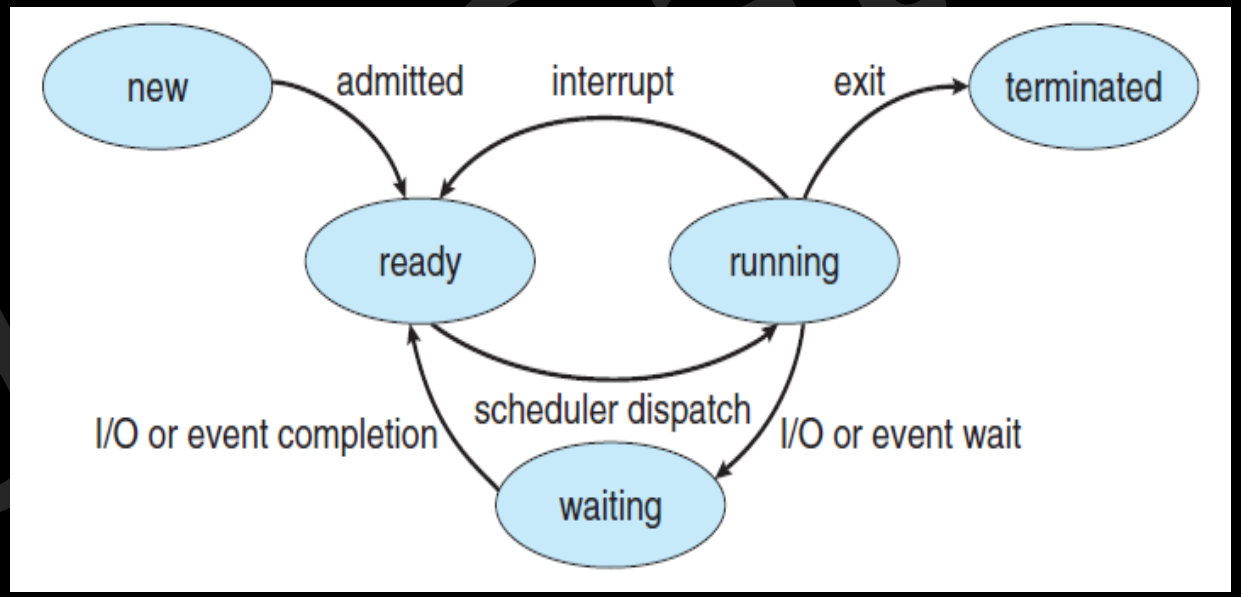
New: The process is being created.

Running: Instructions are being executed.

Waiting (Blocked): The process is waiting for some event to occur (such as an I/O completion or reception of a signal).

Ready: The process is waiting to be assigned to a processor.

Terminated: The process has finished execution.



Knowledge Gate

Knowledge Gate Website

Schedulers

Schedulers: A process migrates among the various scheduling queues throughout its lifetime. The operating system must select, for scheduling purposes, processes from these queues in some fashion. The selection process is carried out by the appropriate scheduler.

Types of Schedulers

Long Term Scheduler (LTS)/Spooler: Long-term schedulers determine which processes enter the ready queue from the job pool. Operating less frequently than short-term schedulers, they focus on long-term system goals such as maximizing throughput.

Medium-term scheduler: The medium-term scheduler swaps processes in and out of memory to optimize CPU usage and manage memory allocation. By doing so, it adjusts the degree of multiprogramming and frees up memory as needed. Swapping allows the system to pause and later resume a process, improving overall system efficiency.

Short Term Scheduler (STS): The short-term scheduler, or CPU scheduler, selects from among the processes that are ready to execute and allocates the CPU to one of them.

Knowledge Gate

Knowledge Gate Website

Point	Long-Term Scheduler	Short-Term Scheduler	Middle Scheduler
Function	Controls the admission of new processes into the system.	Selects which ready process will execute next.	Adjusts the degree of multiprogramming, moving processes between ready and waiting queues.
Frequency	Executes infrequently as it deals with the admission of new processes.	Executes frequently to rapidly switch between processes.	Executes at an intermediate frequency, balancing long-term and short-term needs.
Responsibility	Determines which programs are admitted to the system from the job pool.	Manages CPU scheduling and the switching of processes.	Controls the mix of CPU-bound and I/O-bound processes to optimize throughput.
Impact on System Performance	Influences overall system performance and degree of multiprogramming.	Directly impacts CPU utilization and response time.	Balances system load to prevent resource bottlenecks or idle resources.
Decision Making	Makes decisions based on long-term goals like system throughput.	Makes decisions based on short-term goals like minimizing response time.	Makes decisions considering both short-term and long-term goals, optimizing resource allocation.

Knowledge Gate

Knowledge Gate Website

Dispatcher - The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler.

This function involves the following: Switching context, switching to user mode, jumping to the proper location in the user program to restart that program.

The dispatcher should be as fast as possible, since it is invoked during every process switch. The time it takes for the dispatcher to stop one process and start another running is known as the dispatch latency.

Knowledge Gate

Knowledge Gate Website

CPU Bound and I/O Bound Processes

A process execution consists of a cycle of CPU execution or wait and i/o execution or wait. Normally a process alternates between two states.

Process execution begin with the CPU burst that may be followed by a i/o burst, then another CPU and i/o burst and so on. Eventually in the last will end up on CPU burst. So, process keep switching between the CPU and i/o during execution.

I/O Bound Processes: An I/O-bound process is one that spends more of its time doing I/O than it spends doing computations.

CPU Bound Processes: A CPU-bound process, generates I/O requests infrequently, using more of its time doing computations.

It is important that the long-term scheduler select a good process mix of I/O-bound and CPU-bound processes. If all processes are I/O bound, the ready queue will almost always be empty, and the short-term scheduler will have little to do. Similarly, if all processes are CPU bound, the I/O waiting queue will almost always be empty, devices will go unused, and again the system will be unbalanced.

Knowledge Gate

Knowledge Gate Website

Context Switch

Switching the CPU to another process requires performing a state save of the current process and a state restore of a different process. This task is known as a context switch.

When a context switch occurs, the kernel saves the context of the old process in its PCB and loads the saved context of the new process scheduled to run. Context-switch time is pure overhead, because the system does no useful work while switching.

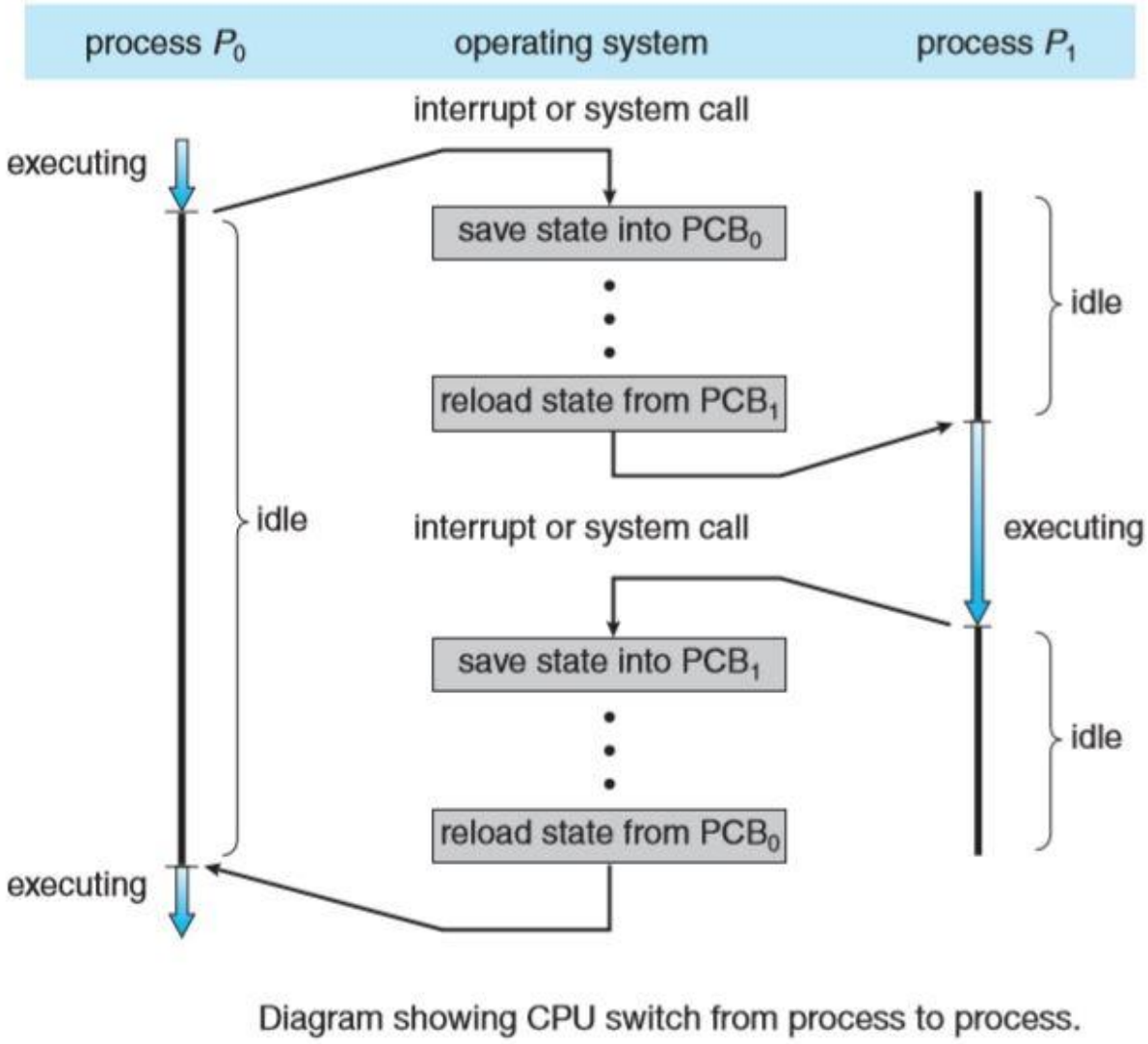


Diagram showing CPU switch from process to process.

Knowledge Gate Website

Break

Knowledge Gate

Knowledge Gate Website

Sector-78
Rohtak
Sadar bazar
Grand father paper industry machenical
May-2023,
Cacha 1 km away
Elder brother marrage in

Knowledge Gate

Knowledge Gate Website

Knowledge Gate

Knowledge Gate Website

CPU Scheduling

1. CPU scheduling is the process of determining which process in the ready queue is allocated to the CPU.
2. Various scheduling algorithms can be used to make this decision, such as First-Come-First-Served (FCFS), Shortest Job Next (SJN), Priority and Round Robin (RR).
3. Different algorithm support different class of process and favor different scheduling criterion.

Knowledge Gate

Knowledge Gate Website

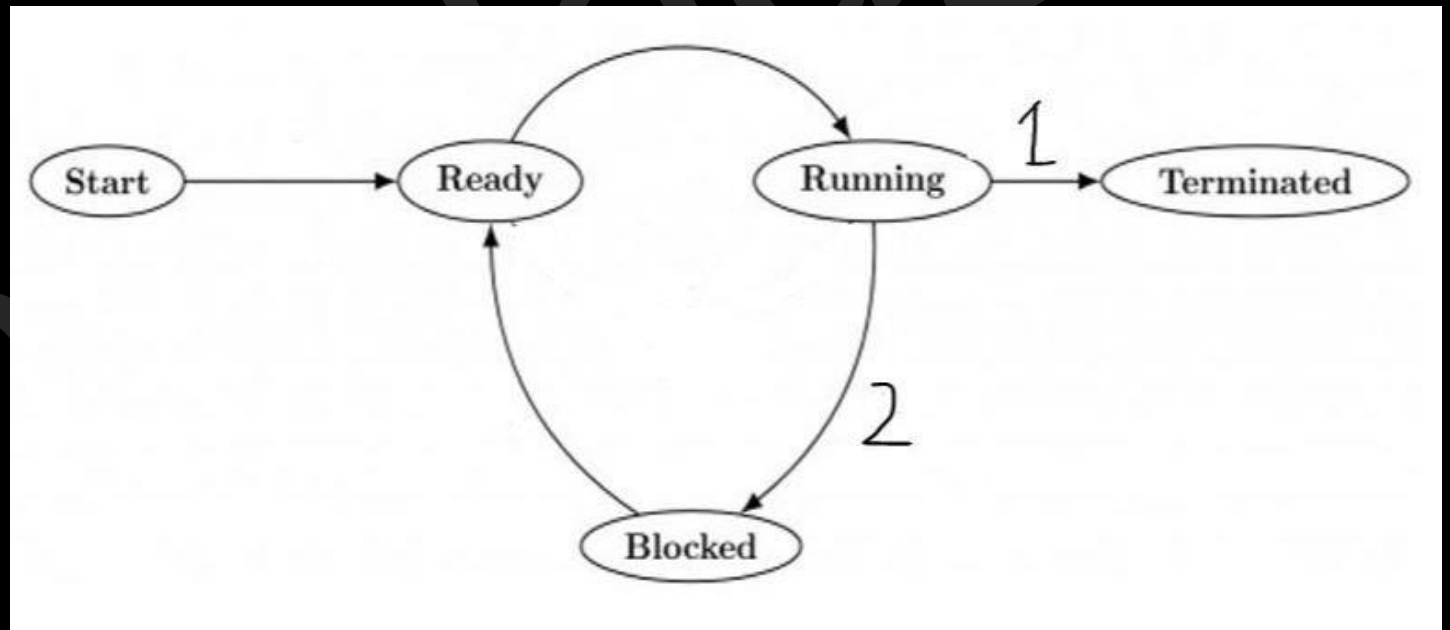
Type of scheduling

Non-Pre-emptive: Under Non-Pre-emptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU willingly.

A process will leave the CPU only

When a process completes its execution (Termination state)

When a process wants to perform some i/o operations(Blocked state)

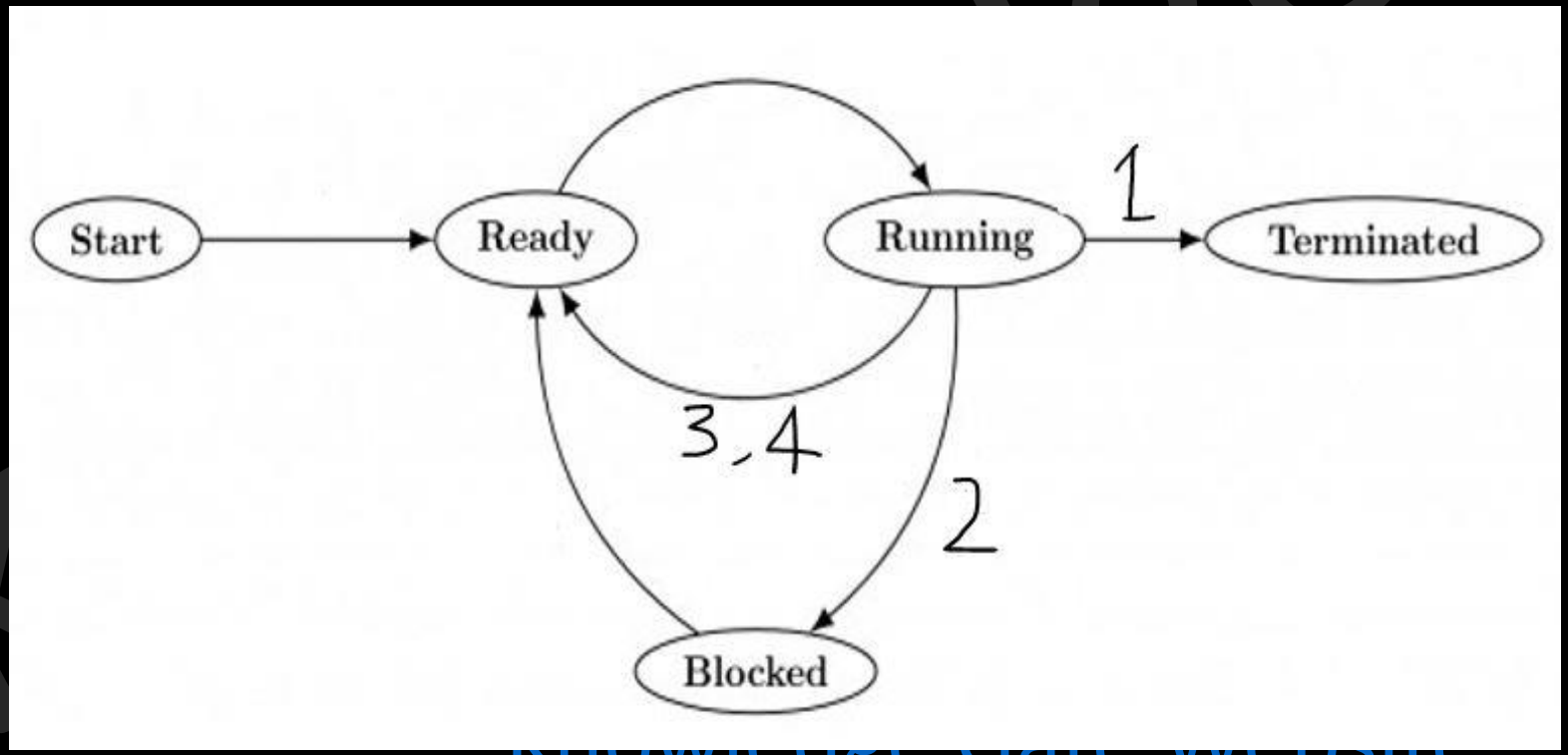


Knowledge Gate

Pre-emptive

Under Pre-emptive scheduling, once the CPU has been allocated to a process, A process will leave the CPU willingly or it can be forced out. So it will leave the CPU

- When a process completes its execution
- When a process leaves CPU voluntarily to perform some i/o operations
- If a new process enters in the ready states (new, waiting), in case of high priority
- When process switches from running to ready state because of time quantum expire.



Knowledge Gate Website

Point	Non-Pre-emptive Scheduling	Pre-emptive Scheduling
CPU Allocation	Once a process starts, it runs to completion or wait for some event.	A process can be interrupted and moved to the ready queue.
Response Time	Can be longer, especially for short tasks.	Generally shorter, as higher-priority tasks can pre-empt others.
Complexity	Simpler to implement.	More complex, requiring careful handling of shared resources.
Resource Utilization	May lead to inefficient CPU utilization.	Typically more efficient, as it can quickly switch tasks.
Suitable Applications	Batch systems and applications that require predictable timing.	Interactive and real-time systems requiring responsive behavior.

Knowledge Gate

Knowledge Gate Website

Scheduling criteria - Different CPU-scheduling algorithms have different properties, and the choice of a particular algorithm may favour one class of processes over another. So, in order to efficiently select the scheduling algorithms following criteria should be taken into consideration:

Knowledge Gate

Knowledge Gate Website

CPU utilization: Keeping the CPU as busy as possible.

Knowledge Gate

Knowledge Gate Website

Throughput: If the CPU is busy executing processes, then work is being done. One measure of work is the number of processes that are completed per time unit, called throughput.

Knowledge Gate Website

Waiting time: Waiting time is the sum of the periods spent waiting in the ready queue.

Gate

Knowledge Gate Website

Response Time: Is the time it takes to start responding, not the time it takes to output the response.

Knowledge Gate Website

Note: The CPU-scheduling algorithm does not affect the amount of time during which a process executes or perform I/0; it affects only the amount of time that a process spends waiting in the ready queue.

It is desirable to maximize CPU utilization and throughput and to minimize turnaround time, waiting time, and response time.

Knowledge Gate

Knowledge Gate Website

Terminology

Arrival Time (AT): Time at which process enters a ready state.

Burst Time (BT): Amount of CPU time required by the process to finish its execution.

Completion Time (CT): Time at which process finishes its execution.

Turn Around Time (TAT): Completion Time (CT) – Arrival Time (AT), Waiting Time + Burst Time (BT)

Waiting Time: Turn Around Time (TAT) – Burst Time (BT)

Knowledge Gate

Knowledge Gate Website

FCFS (FIRST COME FIRST SERVE)

FCFS is the simplest scheduling algorithm, as the name suggest, the process that requests the CPU first is allocated the CPU first.

Implementation is managed by FIFO Queue.

It is always non pre-emptive in nature.

Knowledge Gate

Knowledge Gate Website

P. No	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P_0	2	4			
P_1	1	2			
P_2	0	3			
P_3	4	2			
P_4	3	1			
Average					

Knowledge Gate

Knowledge Gate Website

Advantage

Easy to understand, and can easily be implemented using Queue data structure.

Can be used for Background processes where execution is not urgent.

Knowledge Gate

Knowledge Gate Website

P. No	AT	BT	TAT=CT-AT	WT=TAT -BT
P_0	0	100		
P_1	1	2		
Average				

P. No	AT	BT	TAT=CT-AT	WT=TAT -BT
P_0	1	100		
P_1	0	2		
Average				

Knowledge Gate

Knowledge Gate Website

Convoy Effect

If the smaller process have to wait more for the CPU because of Larger process then this effect is called Convoy Effect, it result into more average waiting time.

Solution, smaller process have to be executed before longer process, to achieve less average waiting time.

Knowledge Gate

Knowledge Gate Website

Disadvantage

FCFS suffers from convoy which means smaller process have to wait larger process, which result into large average waiting time.

The FCFS algorithm is thus particularly troublesome for time-sharing systems (due to its non-pre-emptive nature), where it is important that each user get a share of the CPU at regular intervals.

Higher average waiting time and TAT compared to other algorithms.

Knowledge Gate

Knowledge Gate Website

Shortest Job First (SJF)(non-pre-emptive)

Shortest Remaining Time First (SRTF)/ (Shortest Next CPU Burst) (Pre-emptive)

Whenever we make a decision of selecting the next process for CPU execution, out of all available process, CPU is assigned to the process having smallest burst time requirement. When the CPU is available, it is assigned to the process that has the smallest next CPU burst. If there is a tie, FCFS is used to break tie.

Knowledge Gate

Knowledge Gate Website

It supports both version non-pre-emptive and pre-emptive (purely greedy approach)

In Shortest Job First (SJF)(non-pre-emptive) once a decision is made and among the available process, the process with the smallest CPU burst is scheduled on the CPU, it cannot be pre-empted even if a new process with the smaller CPU burst requirement then the remaining CPU burst of the running process enter in the system.

Knowledge Gate

Knowledge Gate Website

In Shortest Remaining Time First (SRTF) (Pre-emptive) whenever a process enters in ready state, again we make a scheduling decision weather, this new process with the smaller CPU burst requirement then the remaining CPU burst of the running process and if it is the case then the running process is pre-empted and new process is scheduled on the CPU.

This version (SRTF) is also called optimal is it guarantee minimal average waiting time.

Knowledge Gate

Knowledge Gate Website

P. No	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P_0	1	7			
P_1	2	5			
P_2	3	1			
P_3	4	2			
P_4	5	8			
Average					

Knowledge Gate

Knowledge Gate Website

Advantage

Pre-emptive version guarantees minimal average waiting time so some time also referred as optimal algorithm. Provide a standard for other algo in terms of average waiting time. Provide better average response time compare to FCFS.

Disadvantage

Here process with the longer CPU burst requirement goes into starvation and have response time.
This algo cannot be implemented as there is no way to know the length of the next CPU burst. As SJF is not implementable, we can use the one technique where we try to predict the CPU burst of the next coming process.

Knowledge Gate

Knowledge Gate Website

Priority scheduling

Knowledge Gate Website

Priority scheduling

Here a priority is associated with each process. At any instance of time out of all available process, CPU is allocated to the process which possess highest priority (may be higher or lower number).

Tie is broken using FCFS order. No importance to senior or burst time. It supports both non-pre-emptive and pre-emptive versions.

In Priority (non-pre-emptive) once a decision is made and among the available process, the process with the highest priority is scheduled on the CPU, it cannot be pre-empted even if a new process with higher priority more than the priority of the running process enter in the system.

In Priority (pre-emptive) once a decision is made and among the available process, the process with the highest priority is scheduled on the CPU. if it a new process with priority more than the priority of the running process enter in the system, then we do a context switch and the processor is provided to the new process with higher priority.

There is no general agreement on whether 0 is the highest or lowest priority, it can vary from systems to systems.

Knowledge Gate

Knowledge Gate Website

P. No	AT	BT	Priority	CT	TAT = CT - AT	WT = TAT - BT
P_0	1	4	4			
P_1	2	2	5			
P_2	2	3	7			
P_3	3	5	8(H)			
P_4	3	1	5			
P_5	4	2	6			
Average						

Knowledge Gate

Knowledge Gate Website

Advantage

Gives a facility specially to system process.

Allow us to run important process even if it is a user process.

Disadvantage

Here process with the smaller priority may starve for the CPU

No idea of response time or waiting time.

Note: - Specially use to support system process or important user process

Ageing: - a technique of gradually increasing the priority of processes that wait in the system for long time. E.g. priority will increase after every 10 mins

Knowledge Gate

Knowledge Gate Website

Round robin

This algo is designed for time sharing systems, where it is not, the idea to complete one process and then start another, but to be responsive and divide time of CPU among the process in the ready state(circular).

The CPU scheduler goes around the ready queue, allocating the CPU to each process for a maximum of 1 Time quantum say q. Up to which a process can hold the CPU in one go, with in which either a process terminates if process have a CPU burst of less than given time quantum or context switch will be executed and process must release the CPU voluntarily and enter the ready queue and wait for the next chance.

If there are n processes in the ready queue and the time quantum is q, then each process gets 1/n of the CPU time in chunks of at most q time units. Each process must wait no longer than (n - 1) x q time units until its next time quantum.

Knowledge Gate Website

P. No	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P_0	0	4			
P_1	1	5			
P_2	2	2			
P_3	3	1			
P_4	4	6			
P_5	6	3			
Average					

Knowledge Gate

Knowledge Gate Website

Advantage

Perform best in terms of average response time

Works will in case of time-sharing systems, client server architecture and interactive system

kind of SJF implementation

Disadvantage

Longer process may starve

Performance depends heavily on time quantum - If value of the time quantum is very less, then it will give lesser average response time (good but total no of context switches will be more, so CPU utilization will be less), If time quantum is very large then average response time will be more bad, but no of context switches will be less, so CPU utilization will be good.

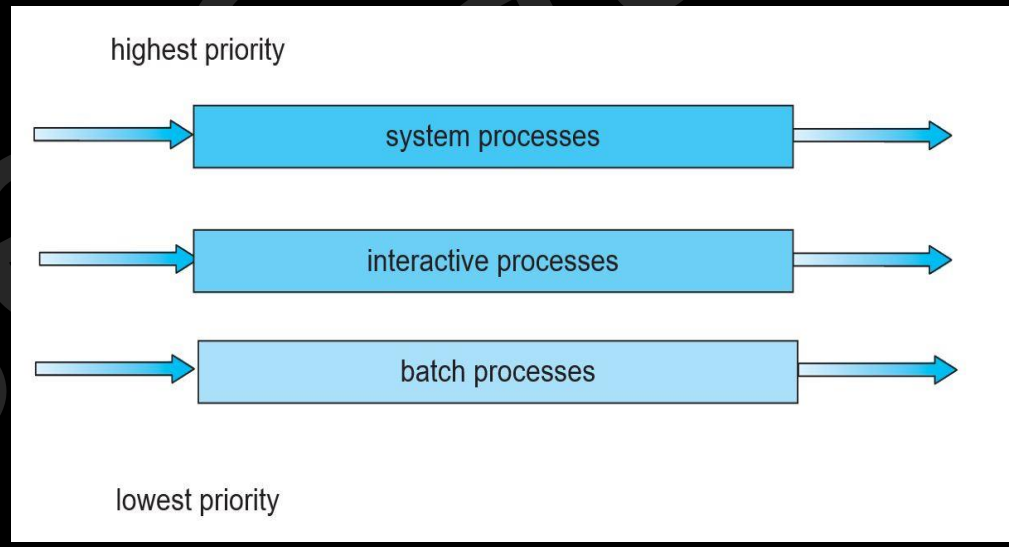
No idea of priority

Knowledge Gate

Knowledge Gate Website

Multi Level-Queue Scheduling

- After studying all important approach to CPU scheduling, we must understand anyone of them alone is not good for every process in the system, as different process have different scheduling needs so, we must have a kind of hybrid scheduling idea, supporting all classes of processes.



- Here processes are easily classified into different groups.
  - **System** process
  - **foreground** (interactive) processes
  - **background** (batch) processes.

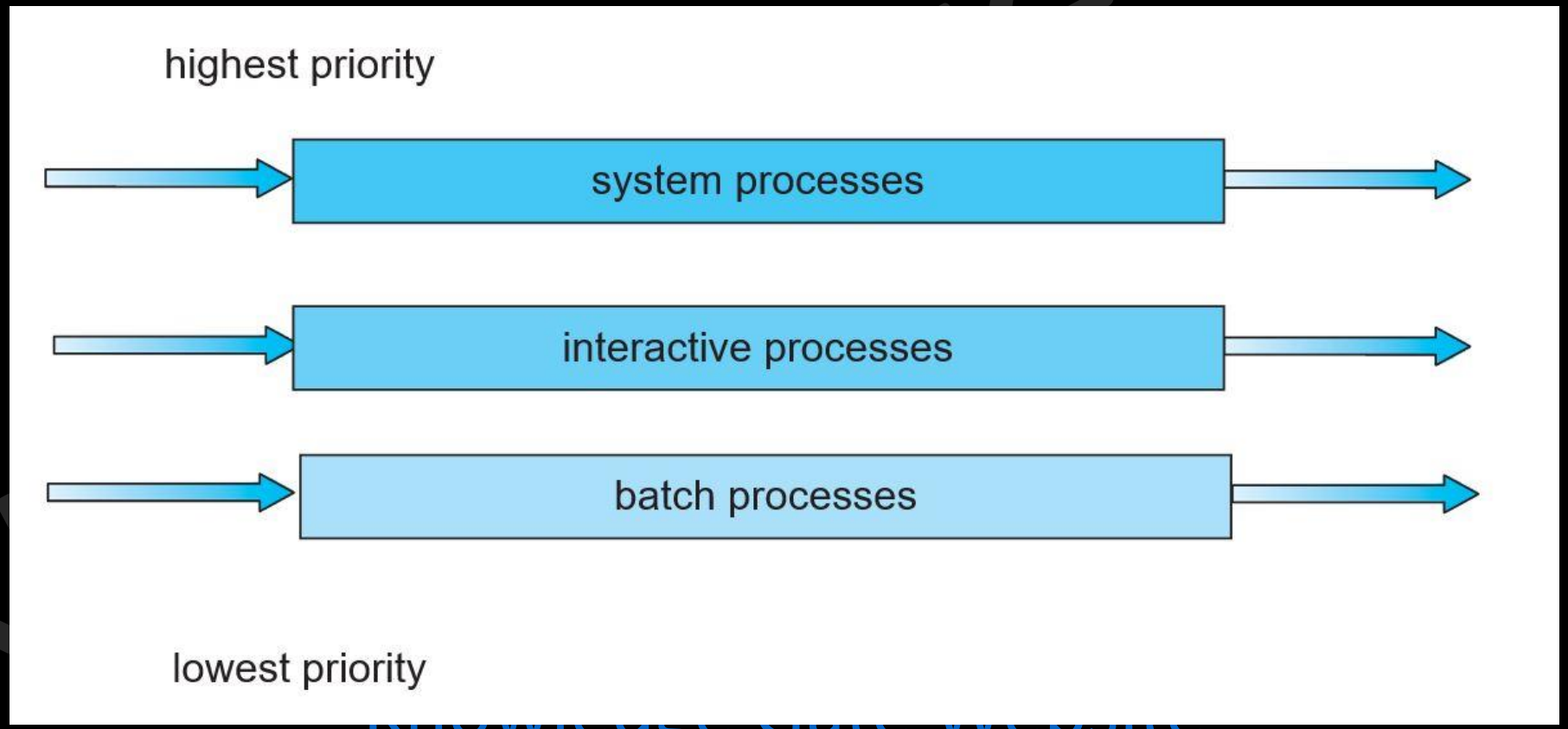
- A **multilevel queue** scheduling algorithm, partitions the ready queue into several separate queues. The processes are permanently assigned to one queue, generally based on properties and requirement of the process.

Knowledge Gate

Knowledge Gate Website

- Each queue has its own scheduling algorithm. For example
  - System process might need priority algorithm
  - Interactive process might be scheduled by an RR algorithm
  - Batch processes is scheduled by an FCFS algorithm.

- In addition, there must be scheduling among the queues, which is commonly implemented as fixed-priority preemptive scheduling or round robin with different time quantum.



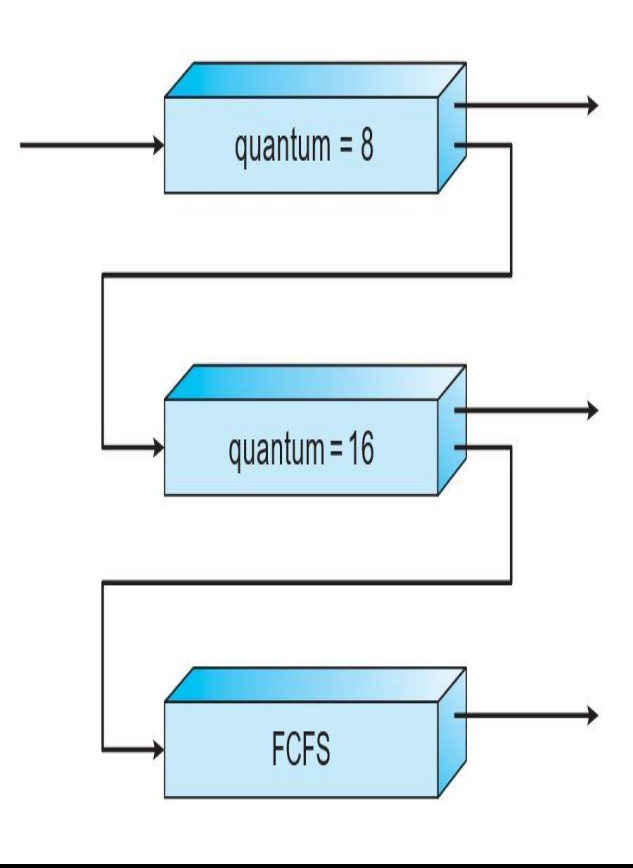
Knowledge Gate
Knowledge Gate Website

Multi-level Feedback Queue Scheduling

Problem with multi-level queue scheduling is how to decide number of ready queue, scheduling algorithm inside the queue and between the queue and once a process enters a specific queue we can not change and queue after that.

The **multilevel feedback queue** scheduling algorithm, allows a process to move between queues. The idea is to separate processes according to the characteristics of their CPU bursts. If a process uses too much CPU time, it will be moved to a lower-priority queue. In addition, a process that waits too long in a lower-priority queue may be moved to a higher-priority queue. This form of aging prevents starvation.

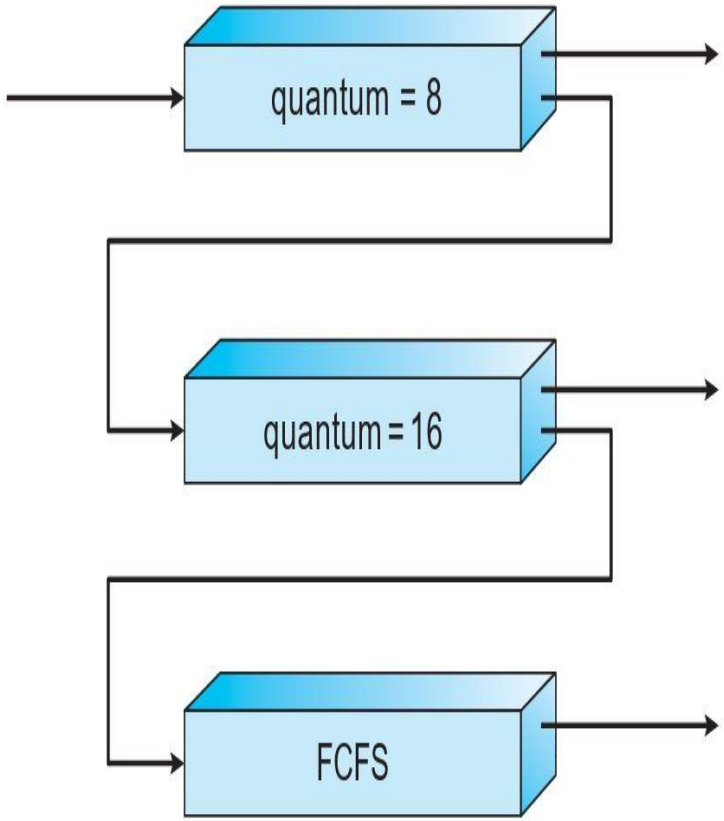
A process entering the ready queue is put in queue 0. A process in queue 0 is given a time quantum of 8 milliseconds. If it does not finish within this time, it is moved to the tail of queue 1. If queue 0 is empty, the process at the head of queue 1 is given a quantum of 16 milliseconds. If it does not complete, it is preempted and is put into queue 2. Processes in queue 2 are run on an FCFS basis but are run only when queues 0 and 1 are empty.



Knowledge Gate Website

In general, a multilevel feedback queue scheduler is defined by the following parameters:

- The number of queues
- The scheduling algorithm for each queue
- The method used to determine when to upgrade a process to a higher-priority queue
- The method used to determine when to demote a process to a lower-priority queue.
- The definition of a multilevel feedback queue scheduler makes it the most general CPU-scheduling algorithm. It can be configured to match a specific system under design. Unfortunately, it is also the most complex algorithm, since defining the best scheduler requires some means by which to select values for all the parameters.



Knowledge Gate Website

Break

Knowledge Gate

Knowledge Gate Website

Process Synchronization & Race Condition

- As we understand in a multiprogramming environment a good number of processes compete for limited number of resources. Concurrent access to shared data at some time may result in data inconsistency for e.g.

```
P ()
{
        read ( i );
        i = i + 1;
        write( i );
}
```

- Race condition is a situation in which the output of a process depends on the execution sequence of process. i.e. if we change the order of execution of different process with respect to other process the output may change.

Knowledge Gate

Knowledge Gate Website

General Structure of a process

- **Initial Section:** Where process is accessing private resources.
- **Entry Section:** Entry Section is that part of code where, each process request for permission to enter its critical section.
- **Critical Section:** Where process is access shared resources.
- **Exit Section:** It is the section where a process will exit from its critical section.
- **Remainder Section:** Remaining Code.

```
P()
{
    While(T)
    {
        Initial Section
        Entry Section
        Critical Section
        Exit Section
        Remainder Section
    }
}
```

Knowledge Gate

Knowledge Gate Website

Criterion to Solve Critical Section Problem

- **Mutual Exclusion:** No two processes should be present inside the critical section at the same time, i.e. only one process is allowed in the critical section at an instant of time.

- **Progress:** If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next(means other process will participate which actually wish to enter). there should be no deadlock.

- **Bounded Waiting:** There exists a bound or a limit on the number of times a process is allowed to enter its critical section and no process should wait indefinitely to enter the CS.

Knowledge Gate

Knowledge Gate Website

Some Points to Remember

- Mutual Exclusion and Progress are mandatory requirements that needs to be followed in order to write a valid solution for critical section problem.

- Bounded waiting is optional criteria, if not satisfied then it may lead to starvation.

Knowledge Gate

Knowledge Gate Website

Solutions to Critical Section Problem

We generally have the following solutions to a Critical Section Problems:

1. Two Process Solution
    1. Using Boolean variable turn
    2. Using Boolean array flag
    3. Peterson's Solution

2. Operating System Solution
    1. Counting Semaphore
    2. Binary Semaphore

3. Hardware Solution
    1. Test and Set Lock
    2. Disable interrupt

Knowledge Gate

Knowledge Gate Website

Two Process Solution

- In general it will be difficult to write a valid solution in the first go to solve critical section problem among multiple processes, so it will be better to first attempt two process solution and then generalize it to N-Process solution.

- There are 3 Different idea to achieve valid solution, in which some are invalid while some are valid.

- **1- Using Boolean variable turn**

- **2- Using Boolean array flag**

- **3- Peterson's Solution**

Knowledge Gate

Knowledge Gate Website

- Here we will use a Boolean variable turn, which is initialize randomly(0/1).

P_0	P_1

while (1)
{
while (turn! = 0);
Critical Section
turn = 1;
Remainder section
}
	while (1)
	{
	while (turn! = 1);
	Critical Section
	turn = 0;
	Remainder Section
	}

Knowledge Gate

Knowledge Gate Website

- The solution follows Mutual Exclusion as the two processes cannot enter the CS at the same time.

- The solution does not follow the Progress, as it is suffering from the strict alternation. Because we never asked the process whether it wants to enter the CS or not?

Knowledge Gate

Knowledge Gate Website

- Here we will use a Boolean array flag with two cells, where each cell is initialized to F

P_0	P_1

while (1)
{
flag [0] = T;
while (flag [1]);
Critical Section
flag [0] = F;
Remainder section
}
	while (1)
	{
	flag [1] = T;
	while (flag [0]);
	Critical Section
	flag [1] = F;
	Remainder Section
	}

Knowledge Gate

Knowledge Gate Website

- This solution follows the Mutual Exclusion Criteria.
- But in order to achieve the progress the system ended up being in a deadlock state.

Knowledge Gate

Knowledge Gate Website

Dekker's algorithm

P_i	P_j

do
{
flag[i] = true;
while (flag[j])
{
if (turn == j)
{
flag[i] = false;
while (turn == j) ;
flag[i] = true;
}
}
/* critical section */
turn = j;
flag[i] = false;
/* remainder section */
}
while (true);
	do
	{
	flag[j] = true;
	while (flag[i])
	{
	if (turn == i)
	{
	flag[j] = false;
	while (turn == i) ;
	flag[j] = true;
	}
	}
	/* critical section */
	turn = i;
	flag[j] = false;
	/* remainder section */
	}
	while (true);

Knowledge Gate

Knowledge Gate Website

- Peterson's solution is a classic Software-based solution to the critical-section problem for two process. We will be using both: **turn and Boolean flag**.

P_0	P_1

while (1)
{
flag [0] = T;
turn = 1;
while (turn = = 1 && flag [1] = = T);
Critical Section
flag [0] = F;
Remainder section
}
	while (1)
	{
	flag [1] = T;
	turn = 0;
	while (turn = = 0 && flag [0] = =T);
	Critical Section
	flag [1] = F;
	Remainder Section
	}

- This solution ensures **Mutual Exclusion, Progress and Bounded Wait.**

Knowledge Gate
Knowledge Gate Website

Operation System Solution (Semaphores)

1. Semaphores are synchronization tools using which we will attempt n-process solution.
2. A semaphore S is a simple integer variable that, but apart from initialization it can be accessed only through two standard atomic operations: wait(S) and signal(S).
3. The wait(S) operation was originally termed as P(S) and signal(S) was originally called V(S).

```
Wait(S)
{
    while(s<=0);
    s--;
}
```

```
Signal(S)
{
    s++;
}
```

Knowledge Gate

Knowledge Gate Website

- Peterson's Solution was confined to just two processes, and since in a general system can have n processes, Semaphores provides n-processes solution.
- While solving Critical Section Problem only we initialize semaphore S = 1.
- Semaphores are going to ensure Mutual Exclusion and Progress but does not ensures bounded waiting.

Wait(S)
{
while(s<=0);
s--;
}

Signal(S)
{
s++;
}

$P_i()$
{
While(T)
{
Initial Section
wait(s)
Critical Section
signal(s)
Remainder Section
}
}

Knowledge Gate

Knowledge Gate Website

Classical Problems on Synchronization

- There are number of actual industrial problem we try to solve in order to improve our understand of Semaphores and their power of solving problems.
- Here in this section we will discuss a number of problems like
  - Producer consumer problem/ Bounder Buffer Problem
  - Reader-Writer problem
  - Dining Philosopher problem
  - The Sleeping Barber problem

Knowledge Gate

Knowledge Gate Website

Producer-Consumer Problem

- **Problem Definition** – There are two process Producer and Consumers, producer produces information and put it into a buffer which have n cell, that is consumed by a consumer. Both Producer and Consumer can produce and consume only one article at a time.
- A producer needs to check whether the buffer is overflowed or not after producing an item, before accessing the buffer.
- Similarly, a consumer needs to check for an underflow before accessing the buffer and then consume an item.
- Also, *the producer and consumer must be synchronized, so that once a producer and consumer it accessing the buffer the other must wait.*

Knowledge Gate

Knowledge Gate Website

Solution Using Semaphores

- Now to solve the problem we will be using three semaphores:
  - Semaphore S = 1 // CS
  - Semaphore E = n // Count Empty cells
  - Semaphore F = 0 // Count Filled cells

Knowledge Gate

Knowledge Gate Website

Semaphore S =

Semaphore E =

Semaphore F =

Producer()	Consumer()

Knowledge Gate

Knowledge Gate Website

Semaphore S =

Semaphore E =

Semaphore F =

Producer()	Consumer()
Producer()	Consumer()
{	{
while(T)	while(T)
{	{
}	}

Knowledge Gate

Knowledge Gate Website

Semaphore S =

Semaphore E =

Semaphore F =

Producer()	Consumer()
Producer()	Consumer()
{	{
while(T)	while(T)
{	{
// Produce an item	
	// Pick item from buffer
// Add item to buffer	
	// Consume item
}	}

Knowledge Gate

Knowledge Gate Website

Semaphore S =

Semaphore E =

Semaphore F =

Producer()	Consumer()
Producer()	Consumer()
{	{
while(T)	while(T)
{	{
// Produce an item	
	wait(S)
wait(S)	// Pick item from buffer
// Add item to buffer	signal(S)
signal(S)	
	// Consume item
}	}

Knowledge Gate

Knowledge Gate Website

Total three resources are used

- semaphore E take count of empty cells and over flow
- semaphore F take count of filled cells and under flow
- Semaphore S take care of buffer

Producer()	Consumer()
{	{
while(T)	while(T)
{	{
// Produce an item	wait(F)//UnderFlow
wait(E)//OverFlow	wait(S)
wait(S)	// Pick item from buffer
// Add item to buffer	signal(S)
signal(S)	wait(E)
wait(F)	Consume item
}	}
}	}

Knowledge Gate

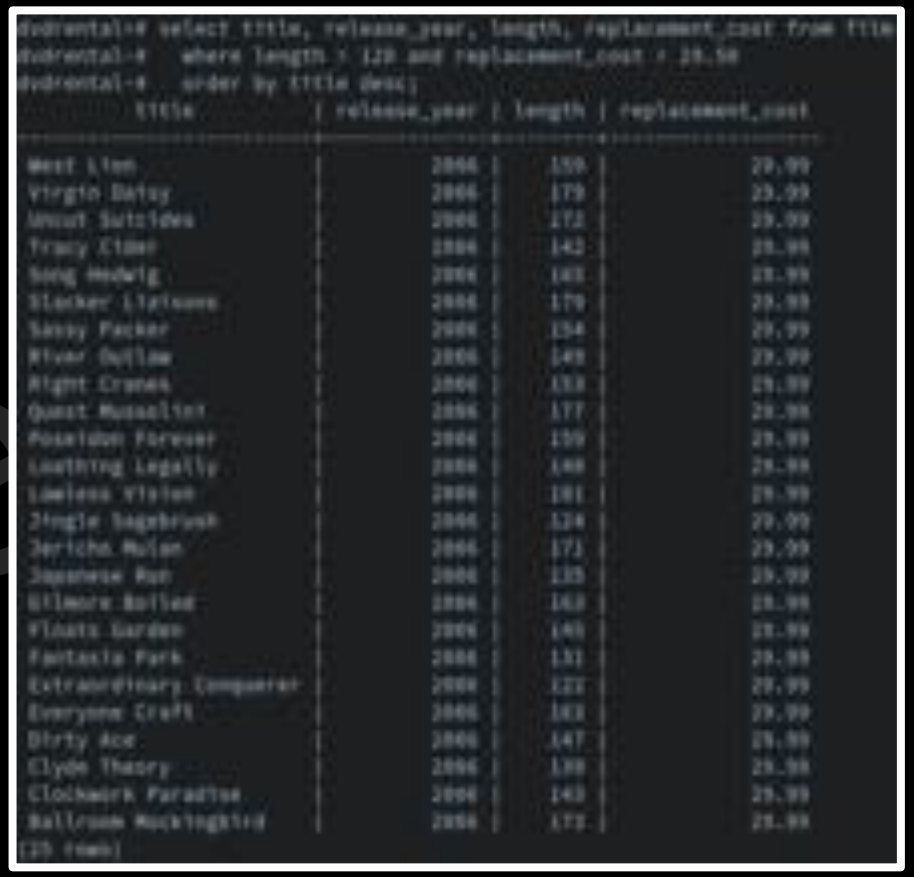
Knowledge Gate Website

Reader-Writer Problem

- Suppose that a database is to be shared among several concurrent processes. Some of these processes may want only to read the database (readers), whereas others may want to update (that is, to read and write) the database(writers).

- If two readers access the shared data simultaneously, no adverse effects will result. But, if a writer and some other process (either a reader or a writer) access the database simultaneously, chaos may ensue.

- To ensure that these difficulties do not arise, we require that the writers have exclusive access to the shared database while writing to the database.



Knowledge

Knowledge Gate Website

- **Points that needs to be taken care for generating a Solutions:**
  - The solution may allow more than one reader at a time, but should not allow any writer.
  - The solution should strictly not allow any reader or writer, while a writer is performing a write operation.

Knowledge Gate

Knowledge Gate Website

- **Solution using Semaphores**
  - The reader processes share the following data structures:
  - semaphore mutex = 1, wrt =1; // Two semaphores
  - int readcount = 0; // Variable

Three resources are used

- Semaphore Wrt is used for synchronization between WW, WR, RW
- Semaphore reader is used to synchronize between RR
- Readcount is simple int variable which keep counts of number of readers

Knowledge Gate

Knowledge Gate Website

Mutex =

Wrt =

Readcount =

Writer()	Reader()
CS //Write	CS //Read

Knowledge Gate

Knowledge Gate Website

Mutex =

Wrt =

Readcount =

Writer()	Reader()
Wait(wrt)	
CS //Write	CS //Read
Signal(wrt)	

Knowledge Gate

Knowledge Gate Website

Mutex =

Wrt =

Readcount =

	Readcount++
Wait(wrt)	
CS //Write	CS //Read
Signal(wrt)	
	Readcount--

[Knowledge Gate Website]

Knowledge Gate

Mutex =

Wrt =

Readcount =

Writer()	Reader()
	Wait(mutex)
	Readcount++
Wait(wrt)	**signal(mutex)**
CS //Write	CS //Read
Signal(wrt)	**Wait(mutex)**
	Readcount--
	signal(mutex)

Knowledge Gate

Knowledge Gate Website

Mutex =

Wrt =

Readcount =

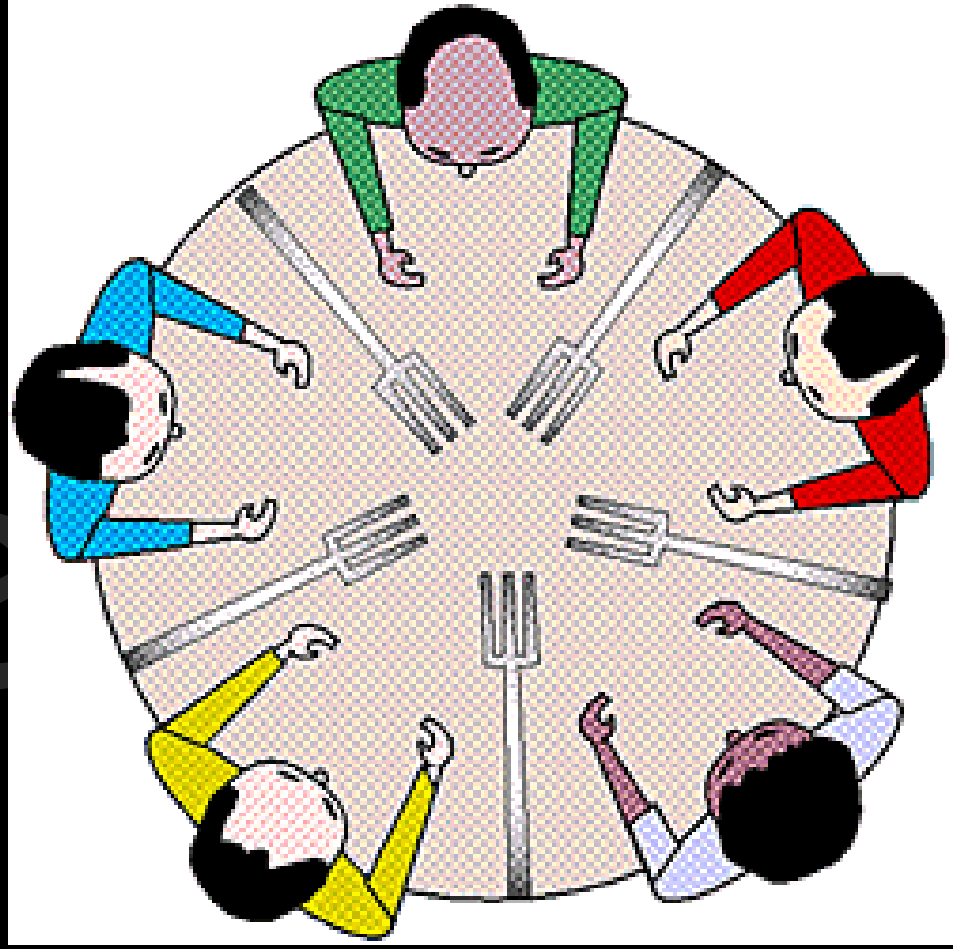
Writer()	Reader()
	Wait(mutex)
	Readcount++
	If(readcount ==1)
	wait(wrt) // first
Wait(wrt)	signal(mutex)
CS //Write	CS //Read
Signal(wrt)	Wait(mutex)
	Readcount--
	If(readcount ==0)
	signal(wrt) // last
	signal(mutex)

Knowledge Gate

Knowledge Gate Website

Dining Philosopher Problem

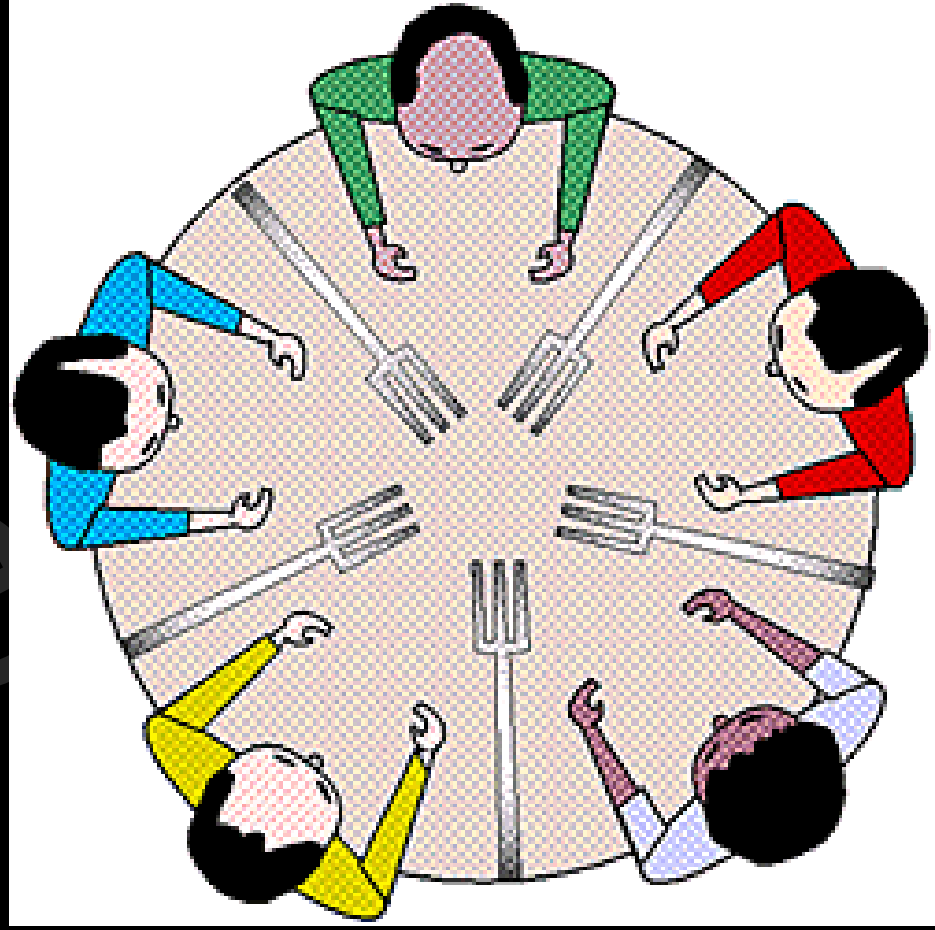
- Consider five philosophers who spend their lives thinking and eating. The philosophers share a circular table surrounded by five chairs, each belonging to one philosopher.
- In the center of the table is a bowl of rice, and the table is laid with five single chopsticks.
- When a philosopher thinks, she does not interact with her colleagues.



Knowledge

Knowledge Gate Website

- From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to her (the chopsticks that are between her and her left and right neighbors).
- A philosopher may pick up only one chopstick at a time. Obviously, she can’t pick up a chopstick that is already in the hand of a neighbor. When a hungry philosopher has both her chopsticks at the same time, she eats without releasing her chopsticks. When she is finished eating, she puts down both of her chopsticks and starts thinking again.



Knowledge

Knowledge Gate Website

Knowledge Gate

Knowledge Gate Website

Indian Chopsticks

Knowledge Gate

Knowledge Gate Website

Solution for Dining Philosophers

```
Void Philosopher (void)
{
        while ( T )
        {
                Thinking ( ) ;
                wait(chopstick [i]);
                wait(chopstick[(i+1)%5]);
                Eat( );
                signal(chopstick [i]);
                signal(chopstick[(i+1)%5]);
        }
}
```

Knowledge Gate

Knowledge Gate Website

- Here we have used an array of semaphores called chopstick[]
- Solution is not valid because there is a possibility of deadlock.

Knowledge Gate

Knowledge Gate Website

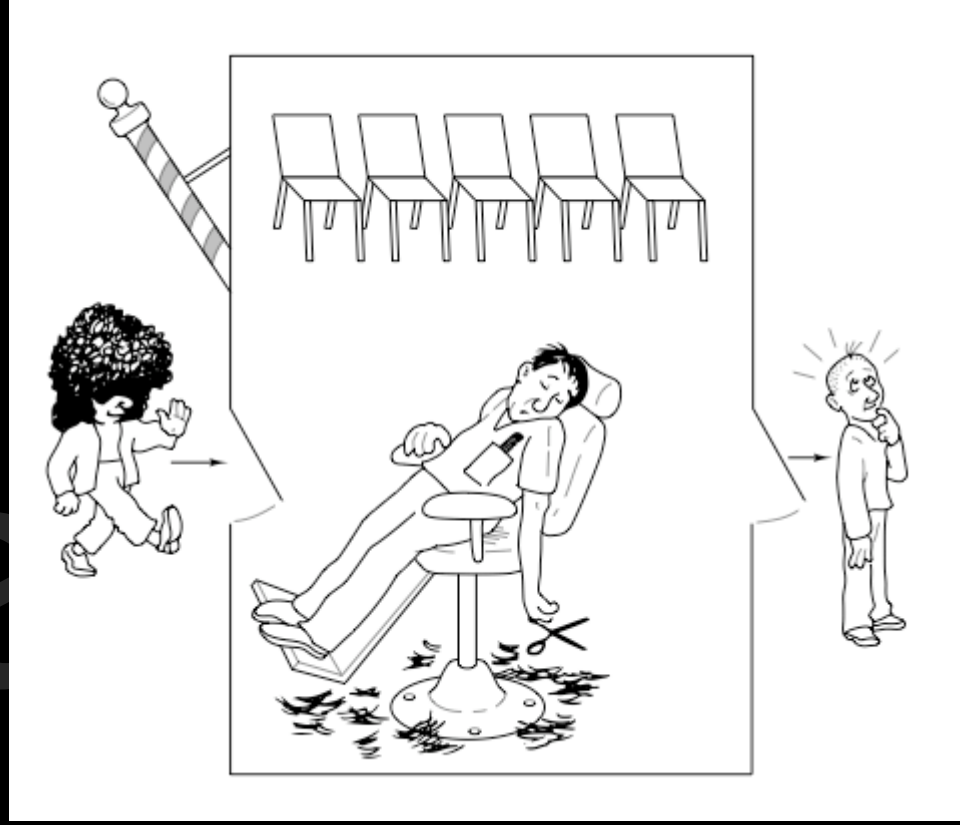
- The proposed solution for deadlock problem is
  - Allow at most four philosophers to be sitting simultaneously at the table.
  - Allow six chopstick to be used simultaneously at the table.
  - Allow a philosopher to pick up her chopsticks only if both chopsticks are available (to do this, she must pick them up in a critical section).
- One philosopher picks up her right chopstick first and then left chop stick, i.e. reverse the sequence of any philosopher.
- Odd philosopher picks up first her left chopstick and then her right chopstick, whereas an even philosopher picks up her right chopstick and then her left chopstick.

Knowledge

Knowledge Gate Website

The Sleeping Barber problem

- **Barbershop**: A barbershop consists of a waiting room with n chairs and a barber room with one barber chair.
- **Customers**: Customers arrive at random intervals. If there is an available chair in the waiting room, they sit and wait. If all chairs are taken, they leave.
- **Barber**: The barber sleeps if there are no customers. If a customer arrives and the barber is asleep, they wake the barber up.
- **Synchronization**: The challenge is to coordinate the interaction between the barber and the customers using concurrent programming mechanisms.



Knowledge

Knowledge Gate Website

```
semaphore barber = 0; // Indicates if the barber is available
semaphore customer = 0; // Counts the waiting customers
semaphore mutex = 1; // Mutex for critical section int
waiting = 0; // Number of waiting customers
```

Barber	Customer

while(true)
{
wait(customer);
wait(mutex);
waiting = waiting - 1;
signal(barber);
signal(mutex);
// Cut hair
}
	wait(mutex);
	if(waiting < n)
	{
	waiting = waiting + 1;
	signal(customer);
	signal(mutex);
	wait(barber);
	// Get hair cut
	}
	else
	{
	signal(mutex);
	}

Knowledge Gate

Knowledge Gate Website

Hardware Type Solution Test and Set

- Software-based solutions such as Peterson's are not guaranteed to work on modern computer architectures. In the following discussions, we explore several more solutions to the critical-section problem using techniques ranging from hardware to software, all these solutions are based on the premise of locking —that is, protecting critical regions through the use of locks.

- The critical-section problem could be solved simply in a single-processor environment if we could prevent interrupts from occurring while a shared variable was being modified.

Knowledge Gate

Knowledge Gate Website

```
Boolean test and set (Boolean *target)
{
        Boolean rv = *target;
        *target  = true;
        return rv;
}
```

```
While(1)
{
        while (test and set(&lock));
        /* critical section */
        lock = false;
        /* remainder section */
}
```

Knowledge Gate

Knowledge Gate Website

- Many modern computer systems therefore provide special hardware instructions that allow us either to test and modify the content of a word atomically —that is, as one uninterruptible unit. We can use these special instructions to solve the critical-section problem in a relatively simple manner.

- The important characteristic of this instruction is that it is executed atomically. Thus, if two test and set() instructions are executed simultaneously (each on a different CPU), they will be executed sequentially in some arbitrary order.

Knowledge Gate

Knowledge Gate Website

Basics of Dead-Lock

- In a multiprogramming environment, several processes may compete for a finite number of resources.
- A process requests resources; if the resources are not available at that time, the process enters a waiting state. Sometimes, a waiting process is never again able to change state, because the resources it has requested are held by other waiting processes. This situation is called a deadlock.
- A set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set.

P_1 P_2

R_1 R_2

Knowledge Gate Website

Knowledge Gate

11455

Tax

Services

Knowledge Gate

Knowledge Gate Website

Knowle

Knowledge Gate Website

Necessary conditions for deadlock

A deadlock can occur if all these 4 conditions occur in the system simultaneously.

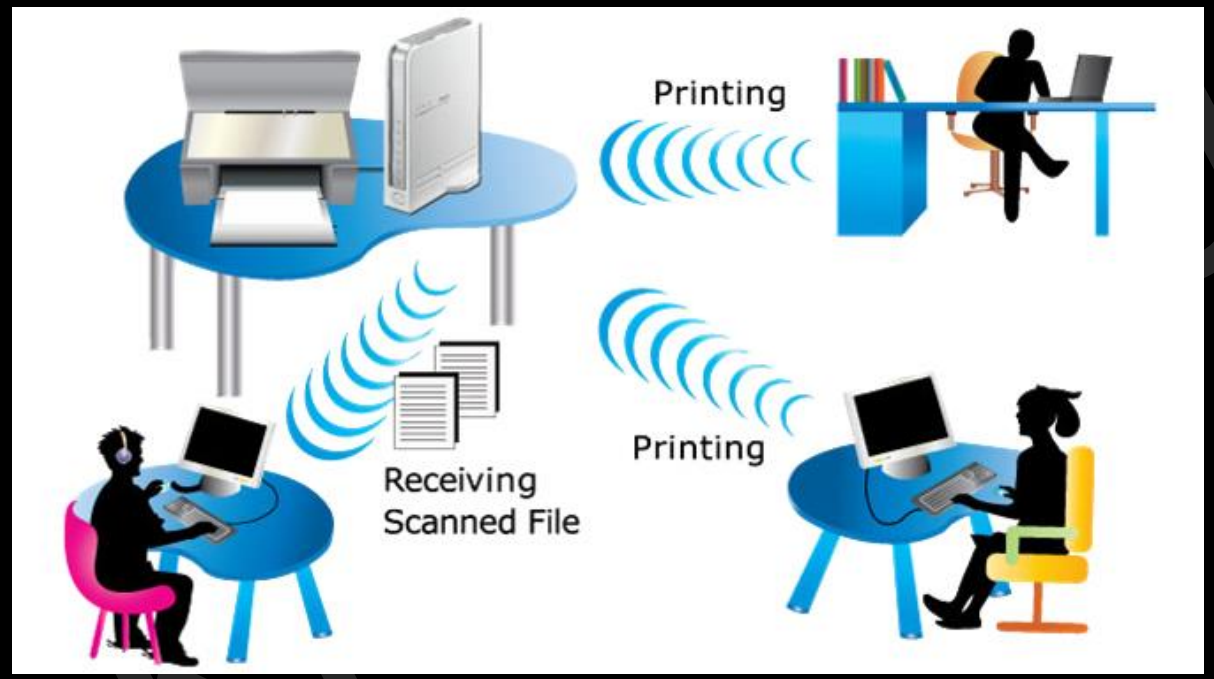
- Mutual exclusion
- Hold and wait
- No pre-emption
- Circular wait

Knowledge Gate

Knowledge Gate Website

- **Mutual exclusion**: - At least one resource must be held in a non-sharable mode; that is, only one process at a time can use the resource.

- If another process requests that resource, the requesting process must be delayed until the resource has been released. And the resource Must be desired by more than one process.



Knowledge Gate

Knowledge Gate Website

- **Hold and wait**: - A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes. E.g. Plate and spoon

Knowledge Gate Website

- **No pre-emption**: - Resources cannot be pre-empted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.

Knowledge Gate Website

- **Circular wait**: - A set $P_0, P_1, \ldots, P_n$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1, P_1 is waiting for a resource held by P_2, ..., P_{n-1} is waiting for a resource held by P_n, and P_n is waiting for a resource held by P_0.

Knowledge Gate

Knowledge Gate Website

Deadlock Handling methods

1. **Prevention**: - Design such protocols that there is no possibility of deadlock.

2. **Avoidance**: - Try to avoid deadlock in run time so ensuring that the system will never enter a deadlocked state.

3. **Detection**: - We can allow the system to enter a deadlocked state, then detect it, and recover.

4. **Ignorance**: - We can ignore the problem altogether and pretend that deadlocks never occur in the system.

Knowledge Gate

Knowledge Gate Website

Prevention

- It means designing such systems where there is no possibility of existence of deadlock. For that we have to remove one of the four necessary condition of deadlock.

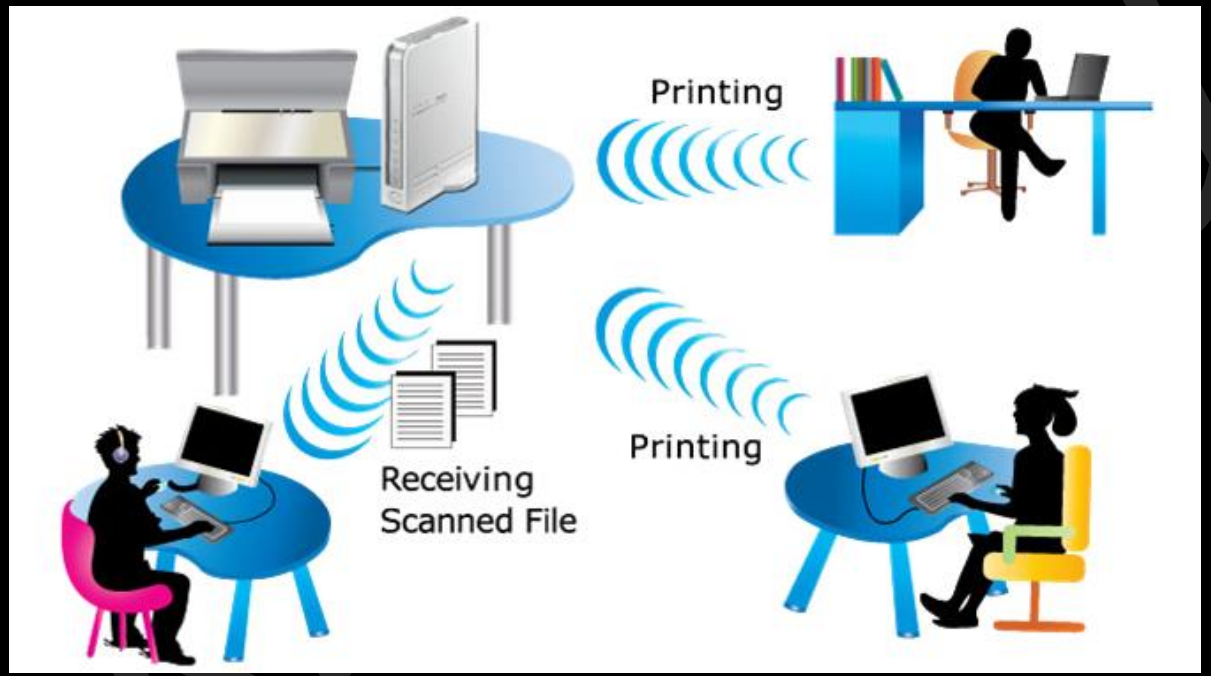
Polio vaccine

Knowledge Gate

Knowledge Gate Website

- **Mutual exclusion**: - In prevention approach, there is no solution for mutual exclusion as resource can't be made sharable as it is a hardware property and process also can't be convinced to do some other task.

- In general, however, we cannot prevent deadlocks by denying the mutual-exclusion condition, because some resources are intrinsically non-sharable.



Knowledge Gate

Knowledge Gate Website

Hold & wait

1. **Conservative Approach**: Process is allowed to run if & only if it has acquired all the resources.

2. **Alternative protocol**: A process may request some resources and use them. Before it can request any additional resources, it must release all the resources that it is currently allocated.

3. **Wait time out**: We place a max time outs up to which a process can wait. After which process must release all the holding resources & exit.

P_1 P_2

R_1 R_2

Knowledge Gate Website

Knowledge Gate

No pre-emption

- If a process requests some resources
  - We first check whether they are available. If they are, we allocate them.
  - If they are not,
    - We check whether they are allocated to some other process that is waiting for additional resources. If so, we pre-empt the desired resources from the waiting process and allocate them to the requesting process (Considering Priority).
    - If the resources are neither available nor held by a waiting process, the requesting process must wait, or may allow to pre-empt resource of a running process Considering Priority.

Knowledge Gate

Knowledge Gate Website

Circular wait

- We can eliminate circular wait problem by giving a natural number mapping to every resource and then any process can request only in the increasing order and if a process wants a lower number, then process must first release all the resource larger than that number and then give a fresh request.

Knowledge Gate

Knowledge Gate Website

- **Problem with Prevention**
  - Different deadlock Prevention approach put different type of restrictions or conditions on the processes and resources Because of which system becomes slow and resource utilization and reduced system throughput.

Knowledge Gate Website

Avoidance

Knowledge Gate

Knowledge Gate Website

- So, in order to avoid deadlock in run time, System try to maintain some books like a banker, whenever someone ask for a loan(resource), it is granted only when the books allow.

Knowledge Gate

Knowledge Gate Website

Avoidance

- To avoiding deadlocks we require additional information about how resources are to be requested. which resources a process will request during its lifetime.

- With this additional knowledge, the operating system can decide for each request whether process should wait or not.

	Max Need		
	E	F	G
P_0			
P_1			
P_2			
P_3			

	Allocation		
	E	F	G
P_0			
P_1			
P_2			
P_3			

	Current Need		
	E	F	G
P_0			
P_1			
P_2			
P_3			

System Max		
E	F	G

Available		
E	F	G

Knowledge Gate Website

	Max Need		
	E	F	G
P_0	4	3	1
P_1	2	1	4
P_2	1	3	3
P_3	5	4	1

	Allocation		
	E	F	G
P_0	1	0	1
P_1	1	1	2
P_2	1	0	3
P_3	2	0	0

	Current Need		
	E	F	G
P_0			
P_1			
P_2			
P_3			

System Max		
E	F	G
8	4	6

Available		
E	F	G

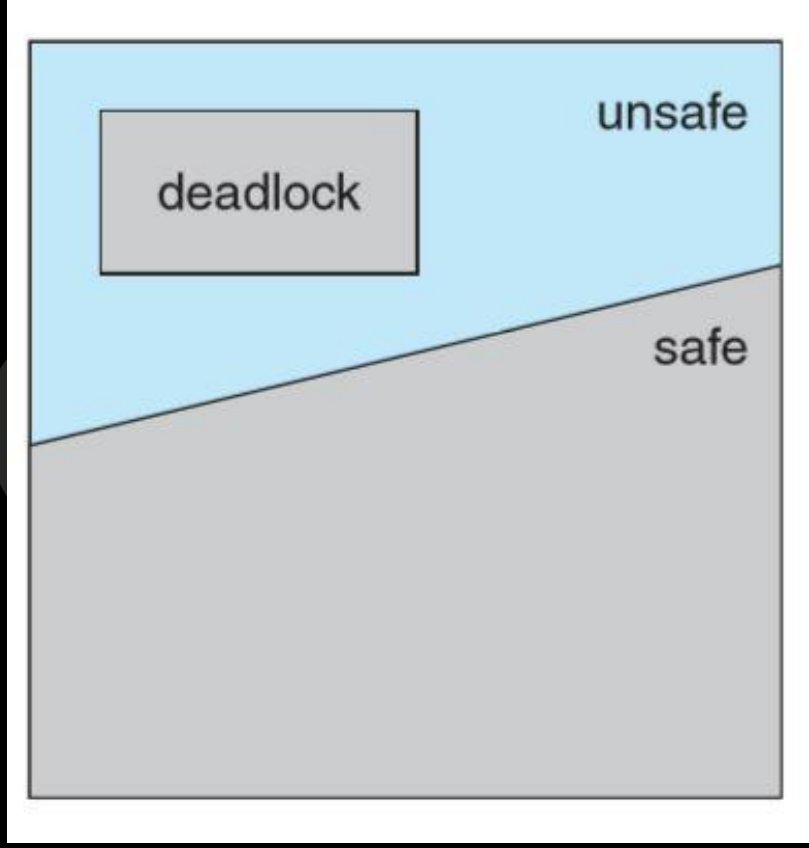
Knowledge Gate

Knowledge Gate Website

- **Safe sequence**: some sequence in which we can satisfies demand of every process without going into deadlock, if yes, this sequence is called safe sequence.

- **Safe Sate**: If their exist at least one possible safe sequence.

- **Unsafe Sate**: If their exist no possible safe sequence.



Knowledge Gate Website

Banker's Algorithm

Several data structures must be maintained to implement the banker's algorithm. These data structures encode the state of the resource-allocation system. We need the following data structures, where n is the number of processes in the system and m is the number of resource types:

Available: A vector of length m indicates the number of available resources of each type. If Available[j] equals k, then k instances of resource type R_j are available.

Available		
E	F	G
3	3	0

Max: An n*m matrix defines the maximum demand of each process. If Max[i][j] equals k, then process P_i may request at most k instances of resource type R_j.

	Max Need		
	E	F	G
P_0	4	3	1
P_1	2	1	4
P_2	1	3	3
P_3	5	4	1

Knowledge Gate

Knowledge Gate Website

Allocation: An n*m matrix defines the number of resources of each type currently allocated to each process. If Allocation[i][j] equals k, then process P_i is currently allocated k instances of resource type R_j.

	Allocation		
	E	F	G
P_0	1	0	1
P_1	1	1	2
P_2	1	0	3
P_3	2	0	0

Need/Demand/Requirement: An n*m matrix indicates the remaining resource need of each process. If Need[i][j] equals k, then process P_i may need k more instances of resource type R_j to complete its task.
Note that Need[i][j] = Max[i][j] − Allocation[i][j].
These data structures vary over time in both size and value.

	Current Need		
	E	F	G
P_0	3	3	0
P_1	1	0	2
P_2	0	3	0
P_3	3	4	1

Knowledge Gate

Knowledge Gate Website

Safety Algorithm

We can now present the algorithm for finding out whether or not a system is in a safe state. This algorithm can be described as follows:

1- Let Work and Finish be vectors of length m and n, respectively. Initialize Work = Available and Finish[i] = false for i = 0, 1, ..., n − 1.

2- Find an index i such that both
Finish[i] == false
$Need_i \leq Work$
If no such i exists, go to step 4.

3- Work = Work + $Allocation_i$
Finish[i] = true
Go to step 2.

4- If Finish[i] == true for all i, then the system is in a safe state.
This algorithm may require an order of $m*n^2$ operations to determine whether a state is safe.

	Need		
	E	F	G
P_0	3	3	0
P_1	1	0	2
P_2	0	3	0
P_3	3	4	1

Work		
E	F	G
3	3	0

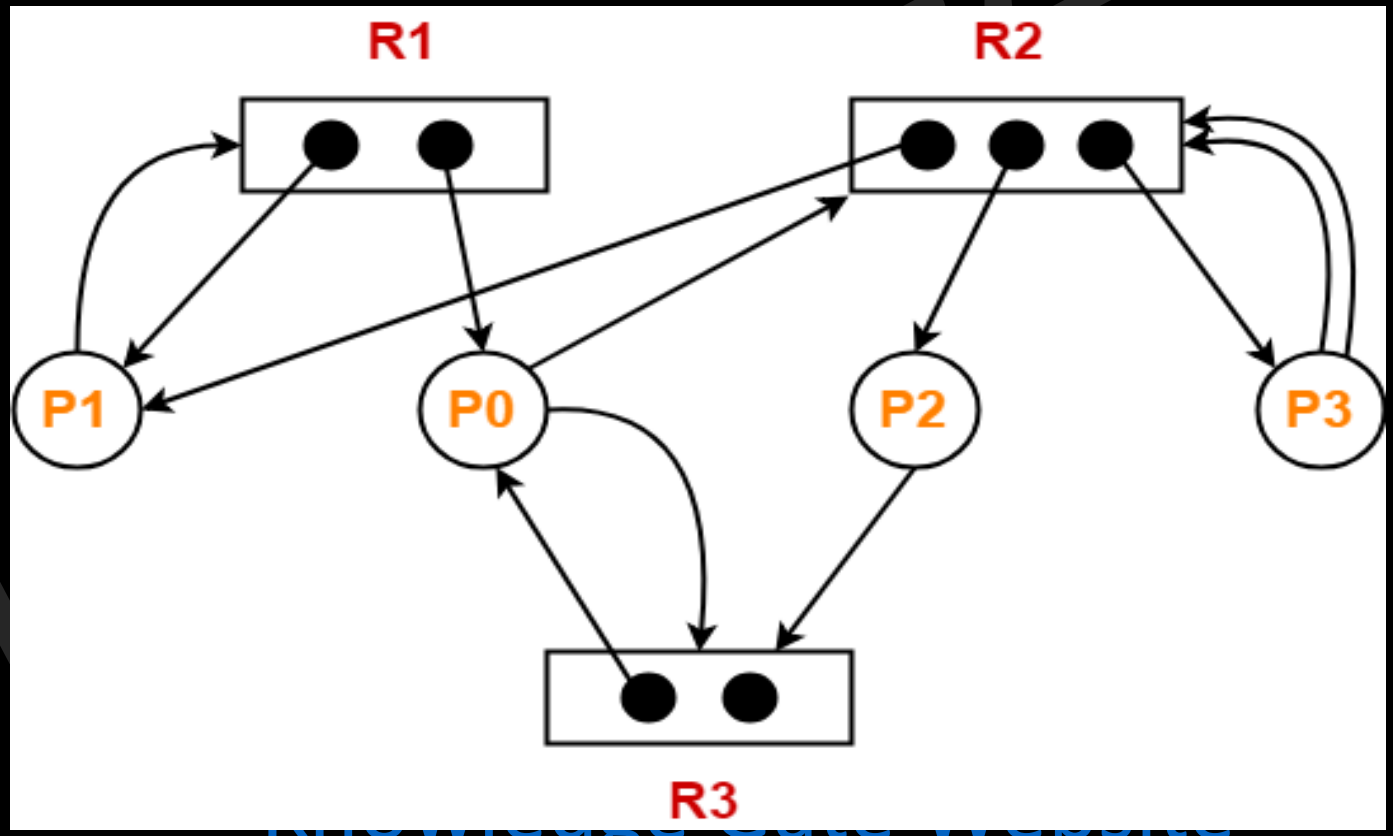
Finish[i]		
E	F	G
F	F	F

Knowledge Gate
Knowledge Gate Website

Resource Allocation Graph

- Deadlock can also be described in terms of a directed graph called a system resource-allocation graph. This graph consists of a set of vertices V and a set of edges E.

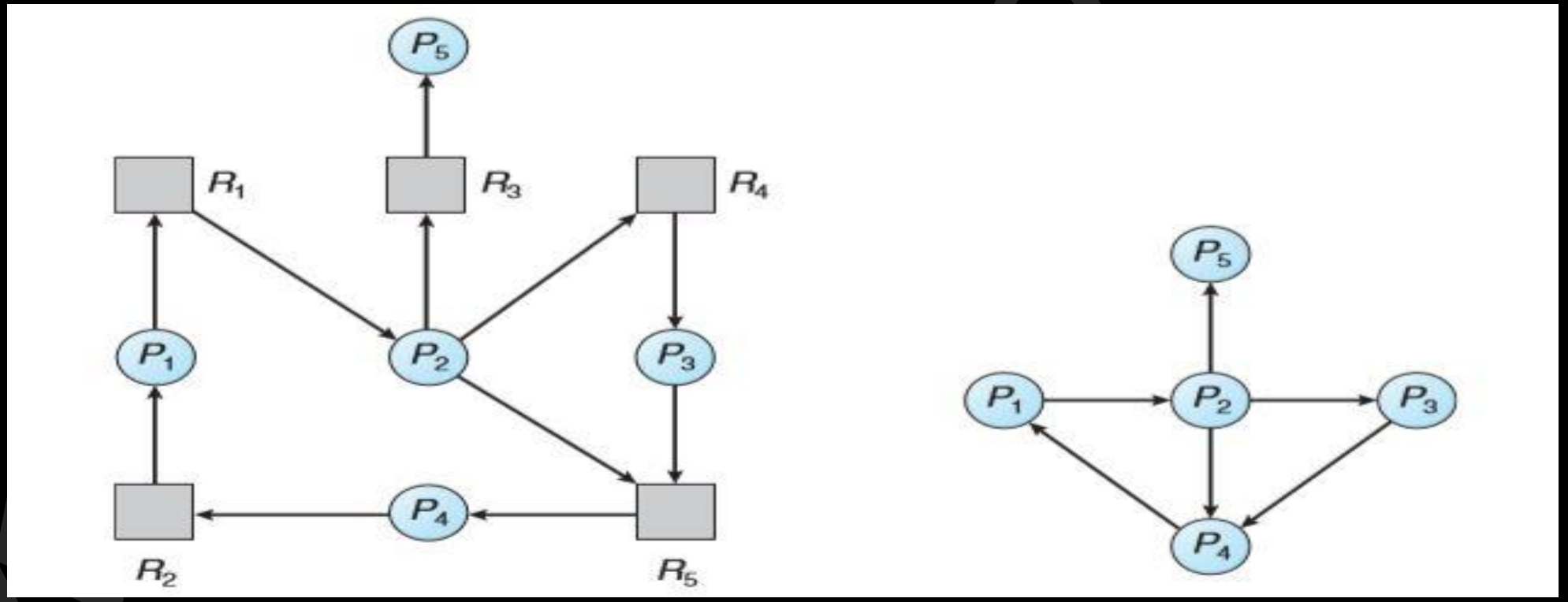
- The set of vertices V is partitioned into two different types of nodes: $P = \{P_1, P_2, ..., P_n\}$, the set consisting of all the active processes in the system, and $R = \{R_1, R_2, ..., R_m\}$, the set consisting of all resource types in the system.



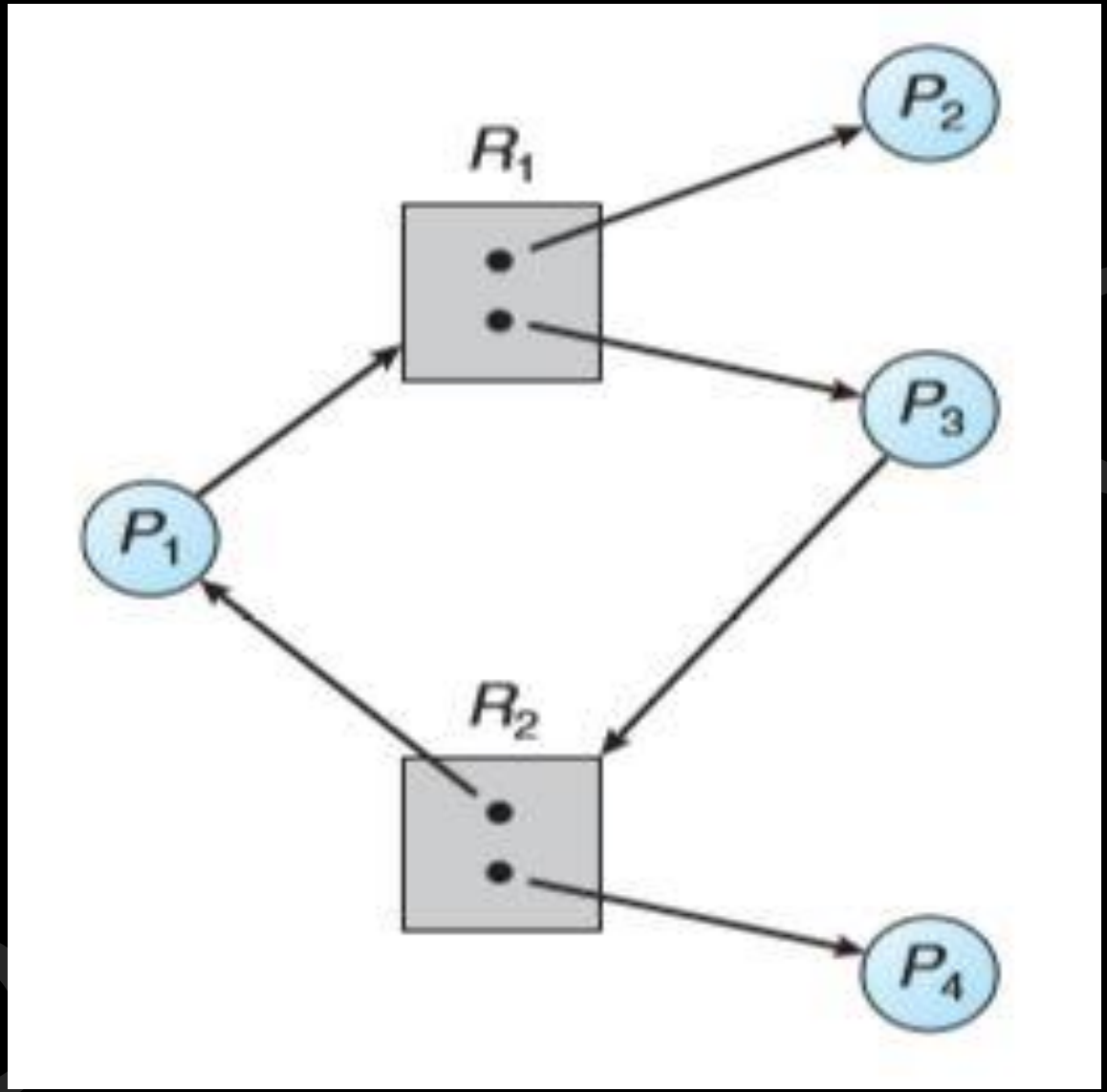
Knowledge Gate Website

- A directed edge from process P_i to resource type R_j is denoted by $P_i \rightarrow R_j$ is called a request edge; it signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource.

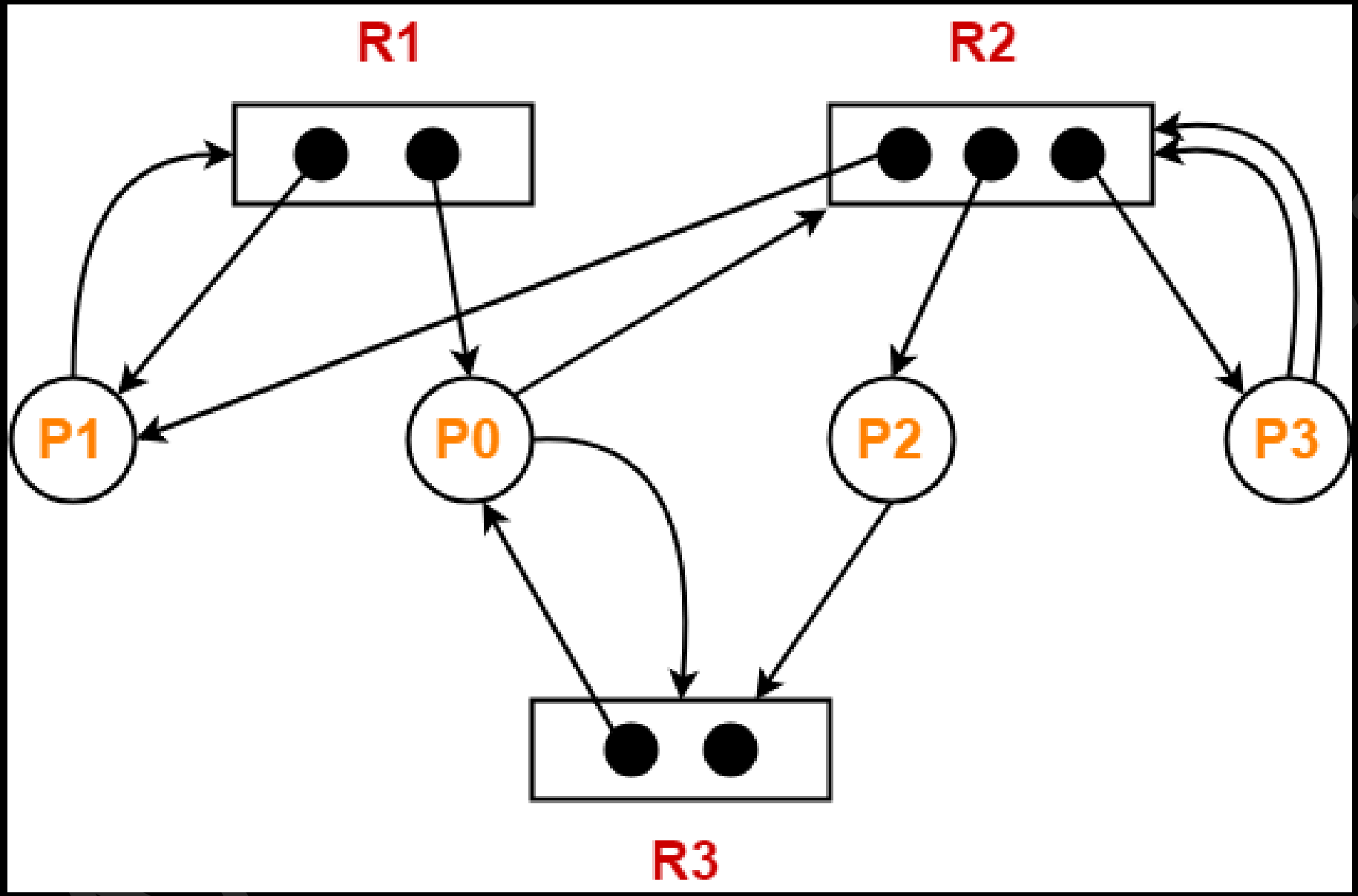
- A directed edge from resource type R_j to process P_i is denoted by $R_j \rightarrow P_i$ is called an assignment edge; it signifies that an instance of resource type R_j has been allocated to process P_i.



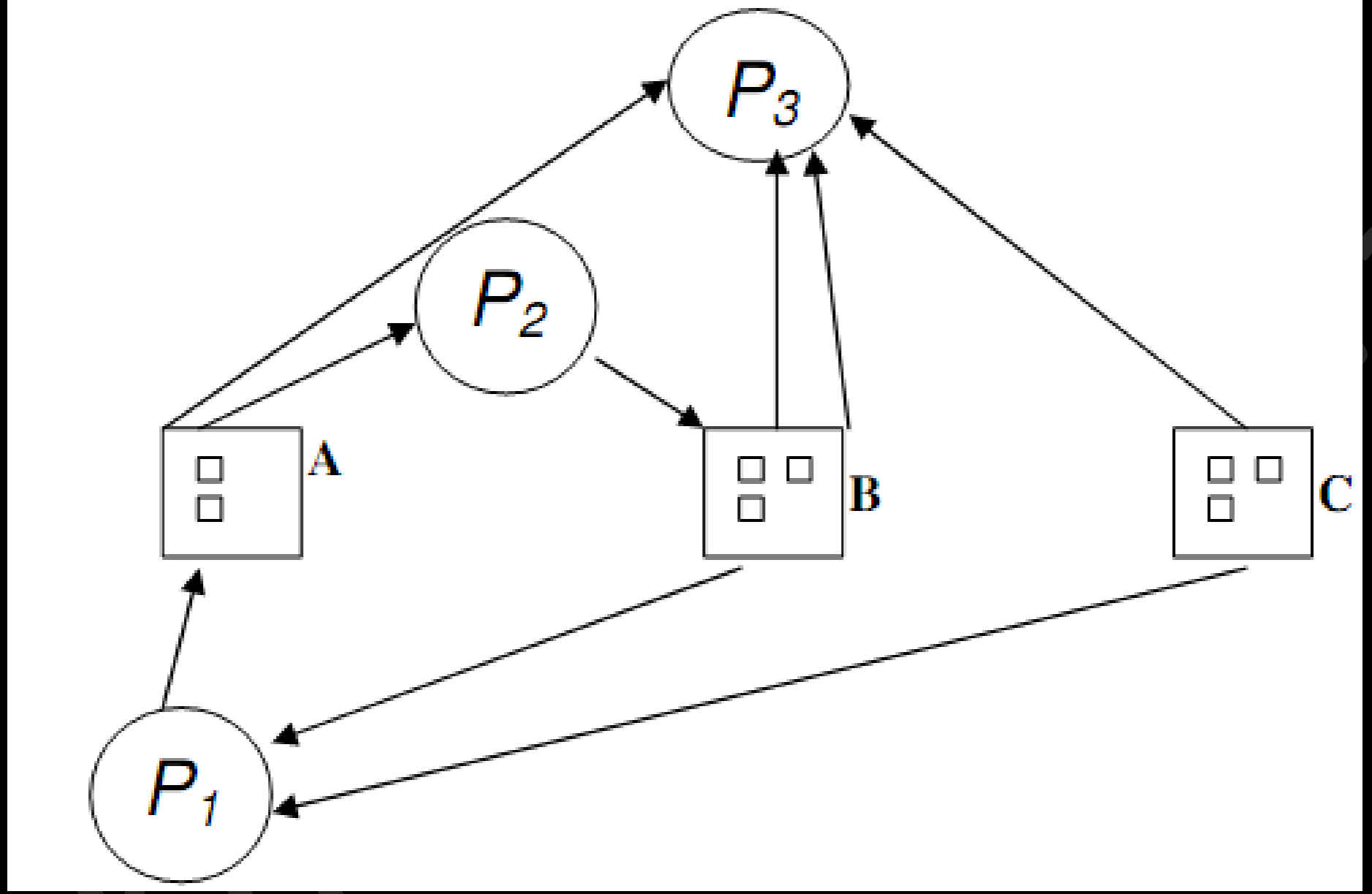
Knowledge Gate Website



Knowledge Gate Website



Knowledge Gate Website



Knowledge Gate Website

- Cycle in resource allocation graph is necessary but not sufficient condition for detection of deadlock.

- If every resource have only one resource in the resource allocation graph than detection of cycle is necessary and sufficient condition for deadlock detection.

Knowledge Gate

Knowledge Gate Website

Deadlock detection and recovery

- Once a dead-lock is detected there are two options for recovery from a deadlock
  - Process Termination
    - Abort all deadlocked processes
    - Abort one process at a time until the deadlock is removed
  - Recourse pre-emption
    - Selecting a victim
    - Partial or Complete Rollback

Knowledge Gate

Knowledge Gate Website

Ignorance(Ostrich Algorithm)

1. Operating System behaves like there is no concept of deadlock.
2. Ignoring deadlocks can lead to system performance issues as resources get locked by idle processes.
3. Despite this, many operating systems opt for this approach to save on the cost of implementing deadlock detection.
4. Deadlocks are often rare, so the trade-off may seem justified. Manual restarts may be required when a deadlock occurs.

Knowledge

Knowledge Gate Website

Break

Knowledge Gate

Knowledge Gate Website

Fork

Requirement of Fork command

In number of applications specially in those where work is of repetitive nature, like web server i.e. with every client we have to run similar type of code. Have to create a separate process every time for serving a new request.

So it must be a better solution that instead to creating a new process every time from scratch we must have a short command using which we can do this logic.

Knowledge Gate

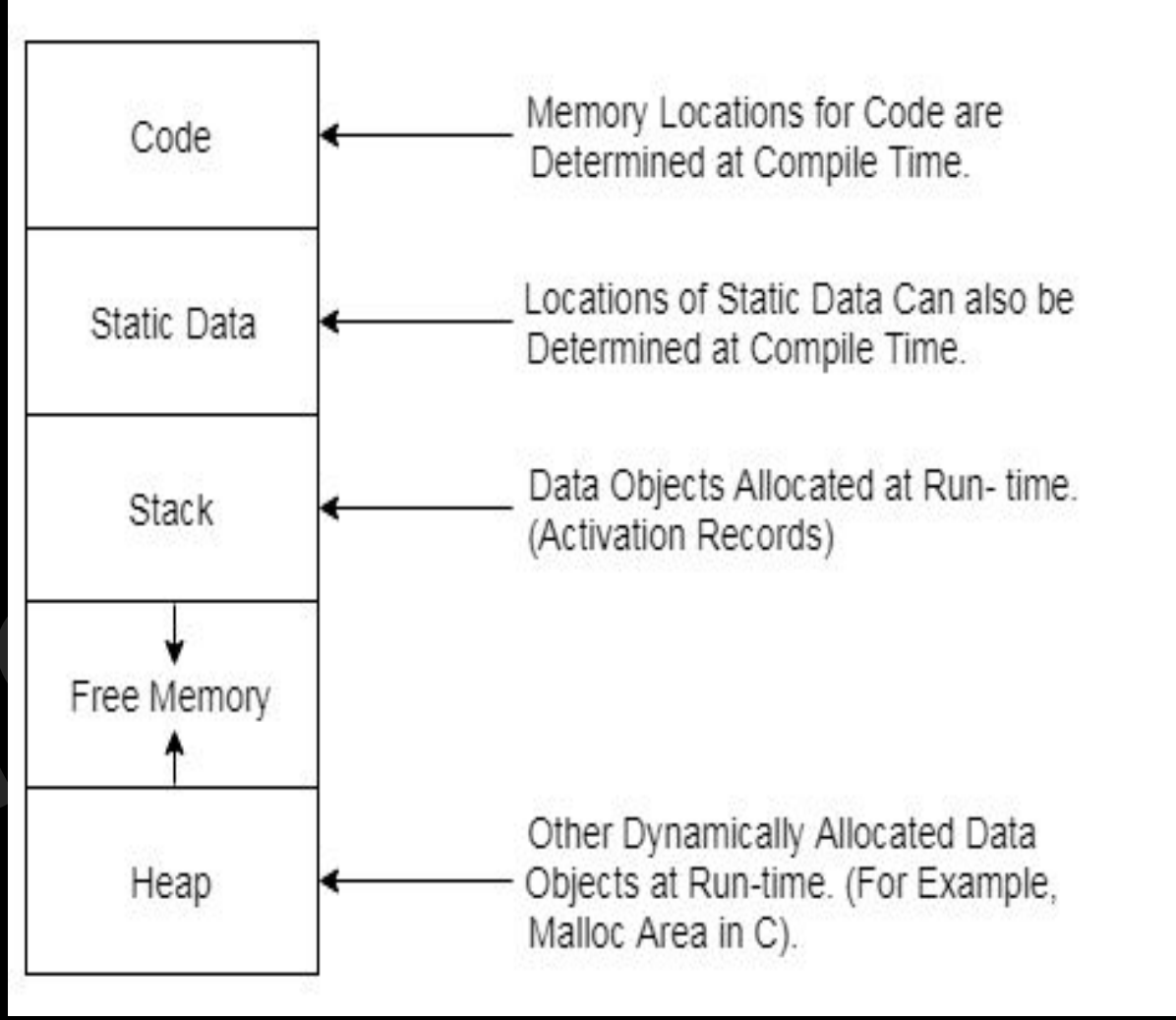
Idea of fork command

Here fork command is a system command using which the entire image of the process can be copied and we create a new process, this idea help us to complete the creation of the new process with speed.

After creating a process, we must have a mechanism to identify weather in newly created process which one is child and which is parent.

Implementation of fork command

In general, if fork return 0 then it is child and if fork return 1 then it is parent, and then using a programmer level code we can change the code of child process to behave as new process.



Knowledge Gate Website

Advantages of using fork commands

Now it is relatively easy to create and manage similar types of process of repetitive nature with the help of fork command.

Disadvantage

To create a new process by fork command we have to do system call as, fork is system function

Which is slow and time taking

Increase the burden over Operating System

Different image of the similar type of task have same code part which means we have the multiple copy of the same data waiting the main memory

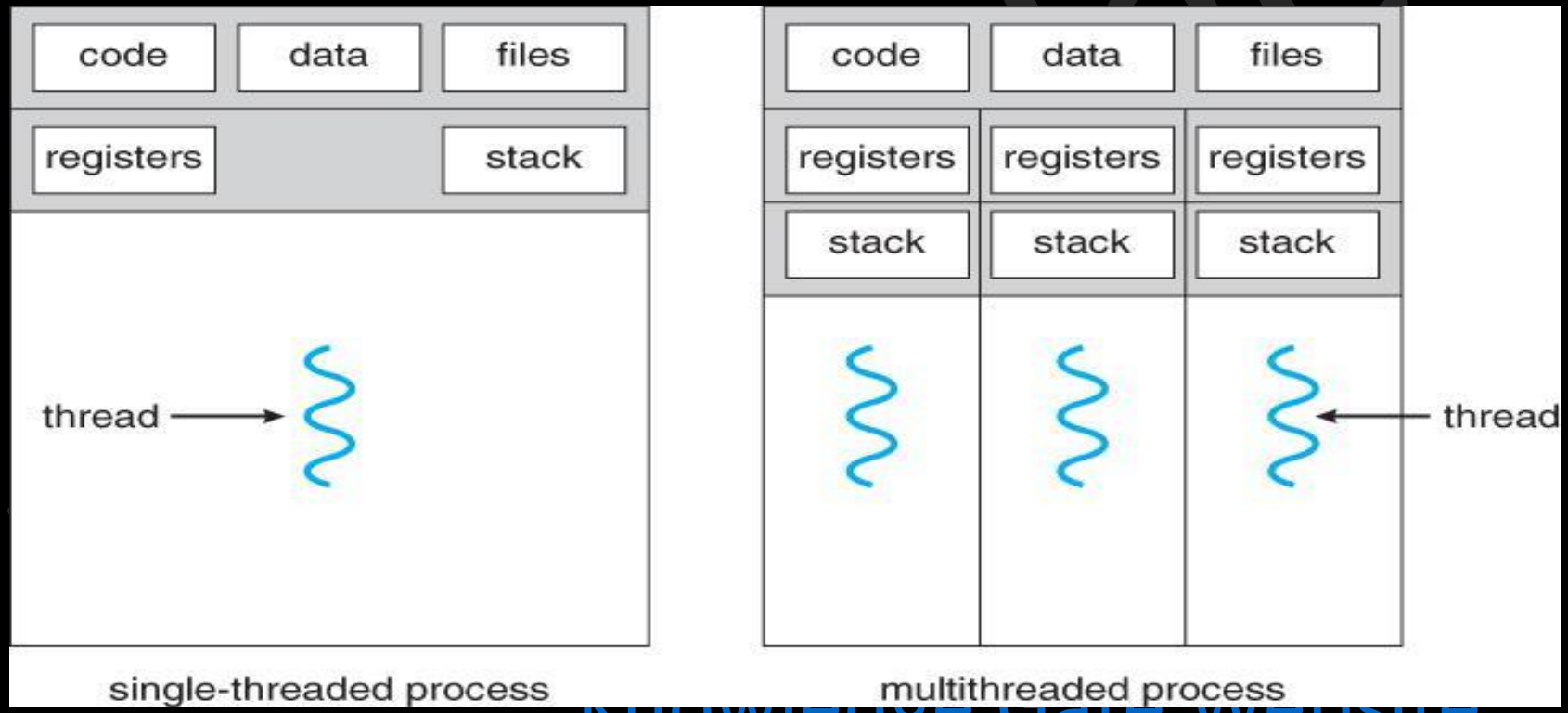
Knowledge Gate

Knowledge Gate Website

A ***thread*** is a basic unit of CPU utilization, consisting of a program counter, a stack, and a set of registers, (and a thread ID.)

Traditional (heavyweight) processes have a single thread of control - There is one program counter, and one sequence of instructions that can be carried out at any given time.

Multi-threaded applications have multiple threads within a single process, each having their own program counter, stack and set of registers, but sharing common code, data, and certain structures such as open files.



Knowledge Gate Website

Multithreading Models

There are two types of threads to be managed in a modern system: User threads and kernel threads.

User threads are supported above the kernel, without kernel support. These are the threads that application programmers would put into their programs.

Kernel threads are supported within the kernel of the OS itself. All modern OS support kernel level threads, allowing the kernel to perform multiple simultaneous tasks and/or to service multiple kernel system calls simultaneously

Knowledge Gate

Knowledge Gate Website

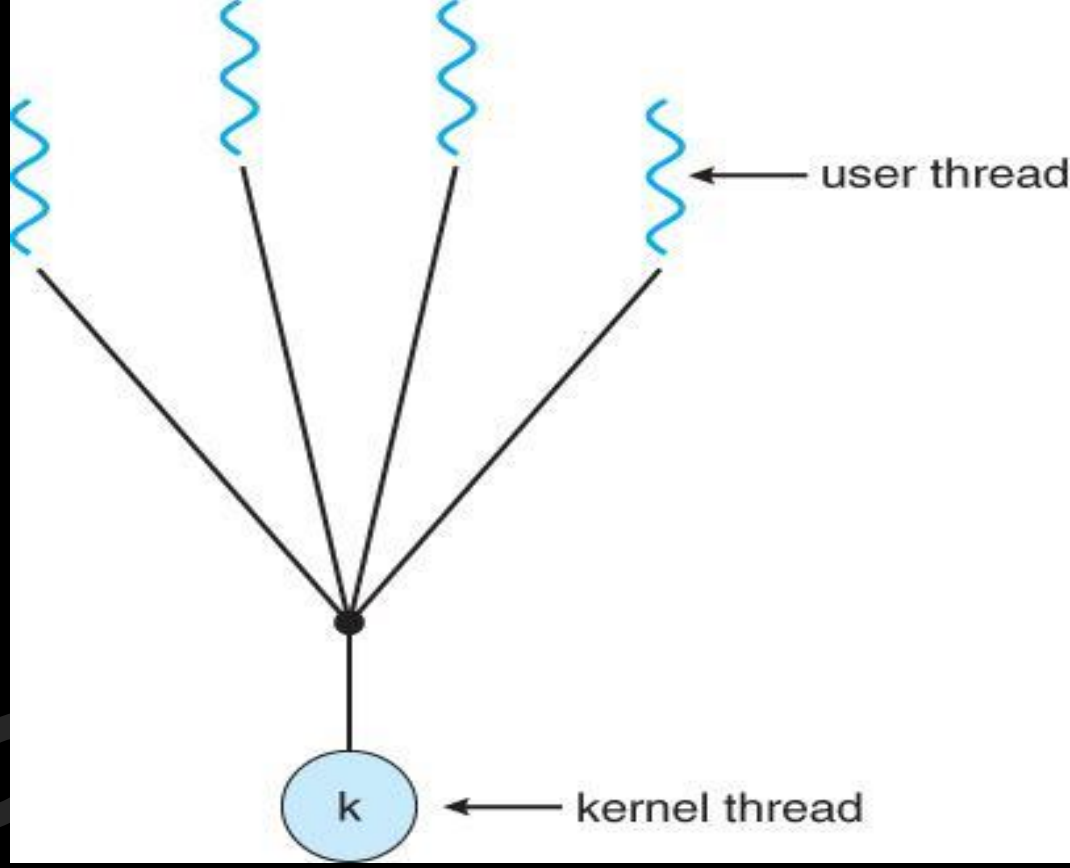
Many-To-One Model

In the many-to-one model, many user-level threads are all mapped onto a single kernel thread.

However, if a blocking system call is made, then the entire process blocks, even if the other user threads would otherwise be able to continue.

Because a single kernel thread can operate only on a single CPU, the many-to-one model does not allow individual processes to be split across multiple CPUs.

Green threads for Solaris implement the many-to-one model in the past, but few systems continue to do so today.



Knowledg

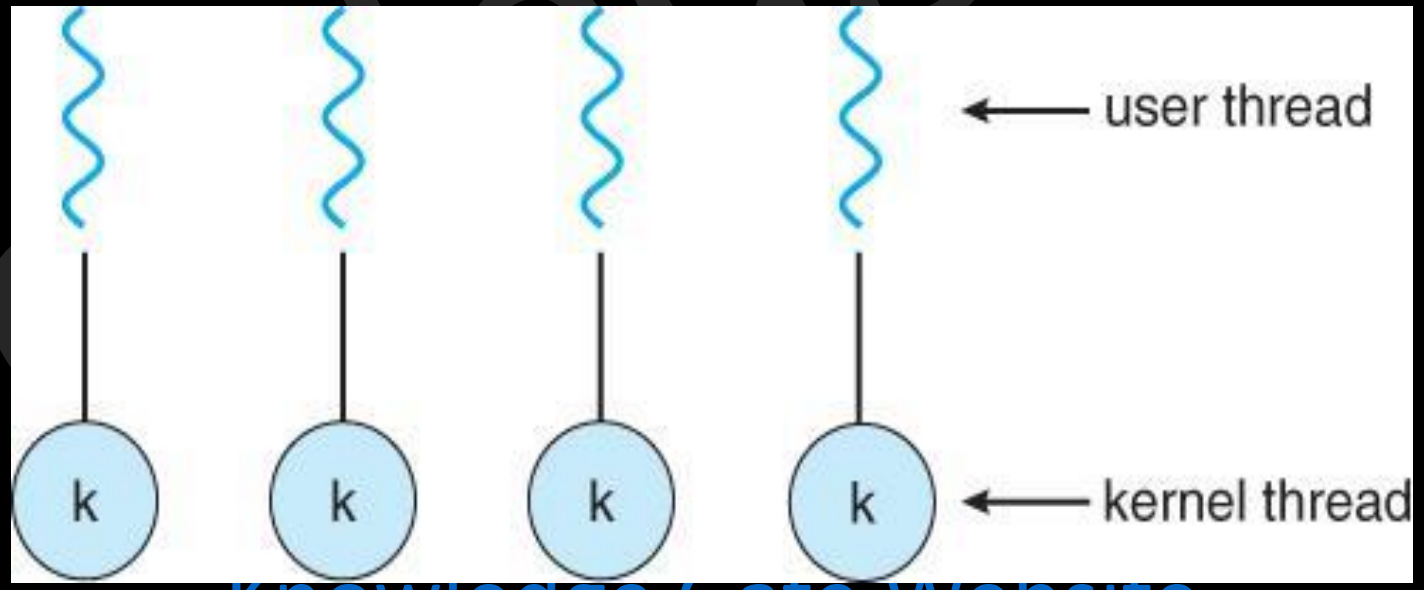
Knowledge Gate Website

One-To-One Model

The one-to-one model creates a separate kernel thread to handle each user thread. It overcomes the problems listed above involving blocking system calls and the splitting of processes across multiple CPUs.

However, the overhead of managing the one-to-one model is more significant, involving more overhead and slowing down the system. Most implementations of this model place a limit on how many threads can be created.

Linux and Windows from 95 to XP implement the one-to-one model for threads.



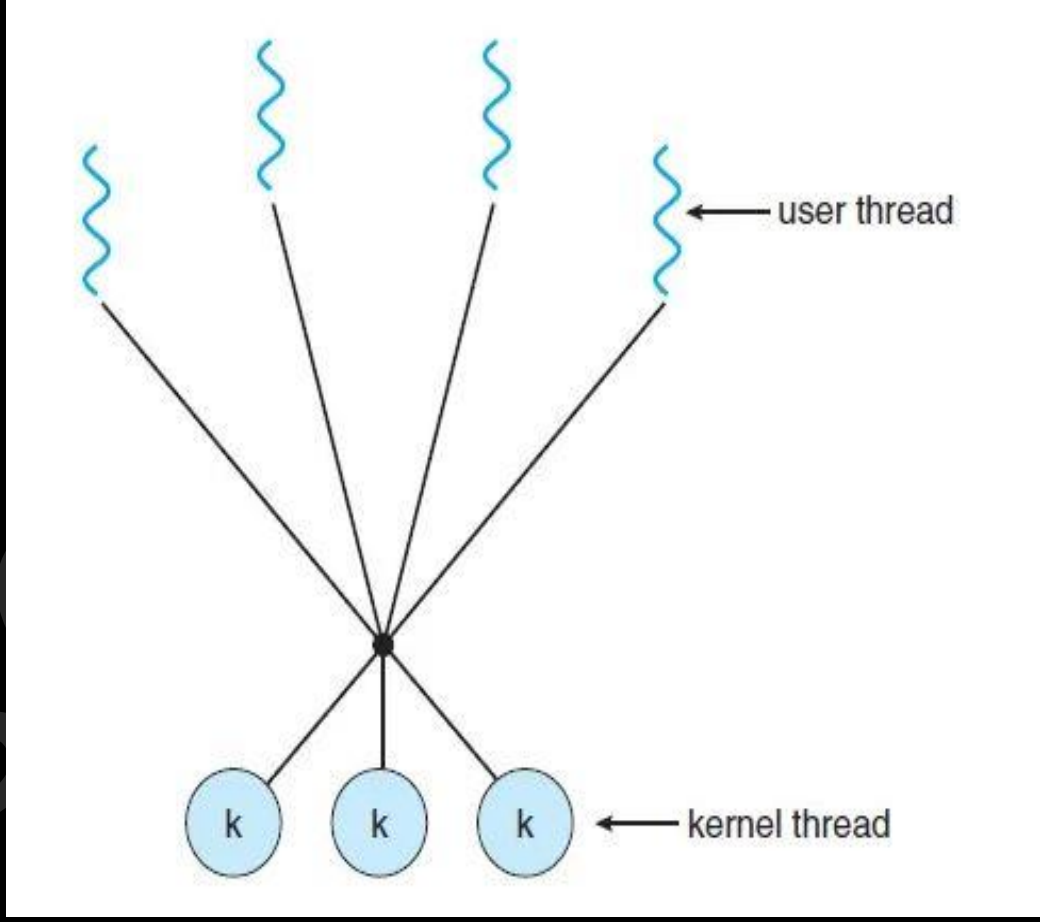
Knowledge Gate Website

Many-To-Many Model

The many-to-many model multiplexes any number of user threads onto an equal or smaller number of kernel threads, combining the best features of the one-to-one and many-to-one models.

Users have no restrictions on the number of threads created. Blocking kernel system calls do not block the entire process.

Processes can be split across multiple processors. Individual processes may be allocated variable numbers of kernel threads, depending on the number of CPUs present and other factors.



Knowledge

Knowledge Gate Website

Knowledge Gate

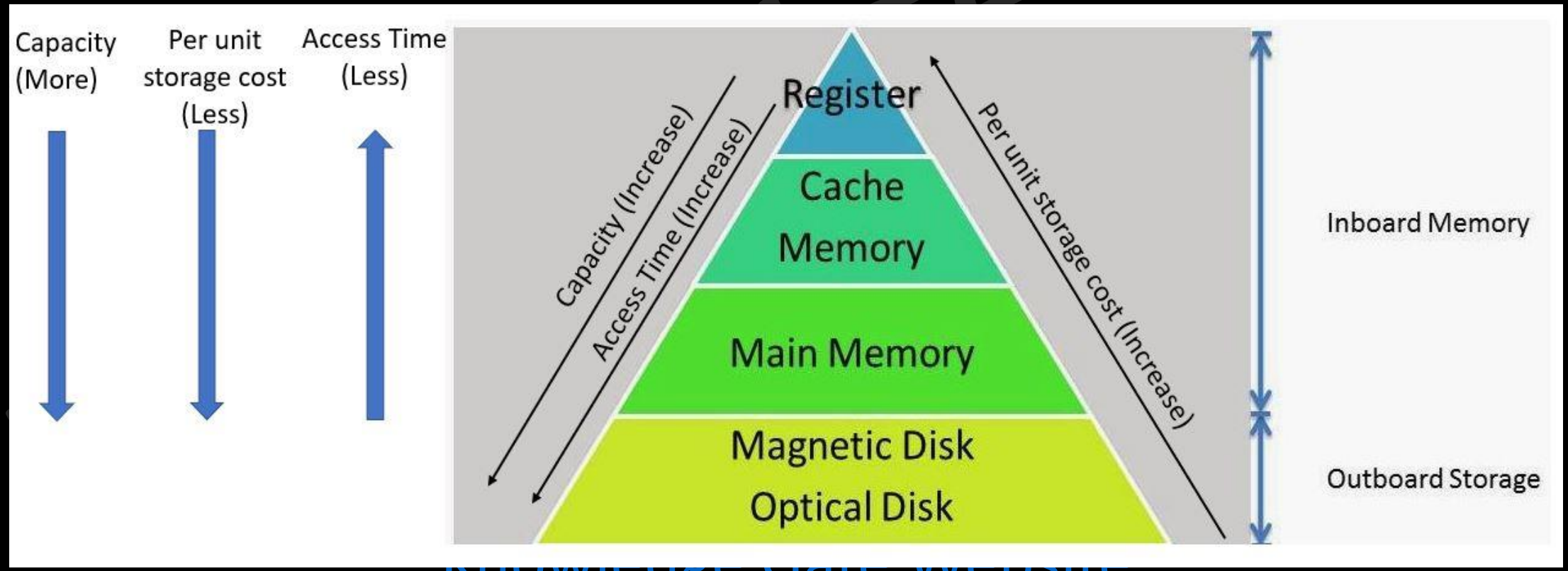
Knowledge Gate Website

Memory Hierarchy

Let first understand what we need from a memory

- Large capacity
- Less per unit cost
- Less access time(fast access)

The memory hierarchy system consists of all storage devices employed in a computer system.



Knowledge Gate Website

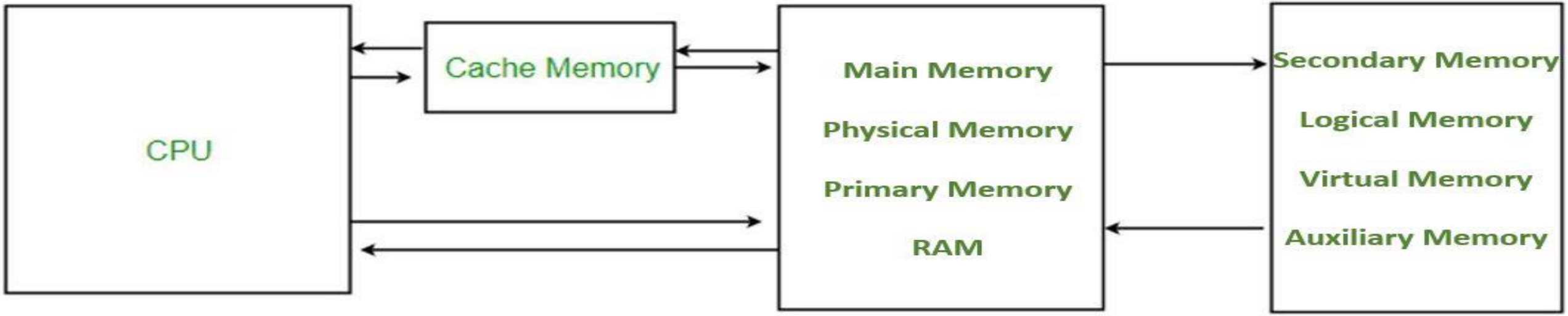
Cycle

Car

Airbus

Knowledge Gate

Knowledge Gate Website



Showroom

4 MB

Go down

32 GB

Factory

8 TB

Knowledge Gate

Knowledge Gate Website

Locality of Reference

The references to memory at any given interval of time tend to be confined within a few localized areas in memory. This phenomenon is known as the property of locality of reference. There are two types of locality of reference.

Spatial Locality: Use of data elements in the nearby locations.

Temporal Locality: Temporal locality refers to the reuse of specific data or resources, within a relatively small-time duration, i.e. Most Recently Used.

Knowledge Gate

Knowledge Gate Website

Duty of Operating System

Operating system is responsible for the following activities in connection with memory management:

1. **Address Translation**: Convert logical addresses to physical addresses for data retrieval.
2. **Memory Allocation and Deallocation**: Decide which processes or data segments to load or remove from memory as needed.
3. **Memory Tracking**: Monitor which parts of memory are in use and by which processes.
4. **Memory Protection**: Implement safeguards to restrict unauthorized access to memory, ensuring both process isolation and data integrity.

Knowledge Gate

Knowledge Gate Website

There can be two approaches for storing a process in main memory.

Contiguous allocation policy

Non-contiguous allocation policy

Knowledge Gate

Knowledge Gate Website

Contiguous allocation policy

We know that when a process is required to be executed it must be loaded to main memory, by policy has two implications.

It must be loaded to main memory completely for execution.

Must be stored in main memory in contiguous fashion.

Knowledge Gate

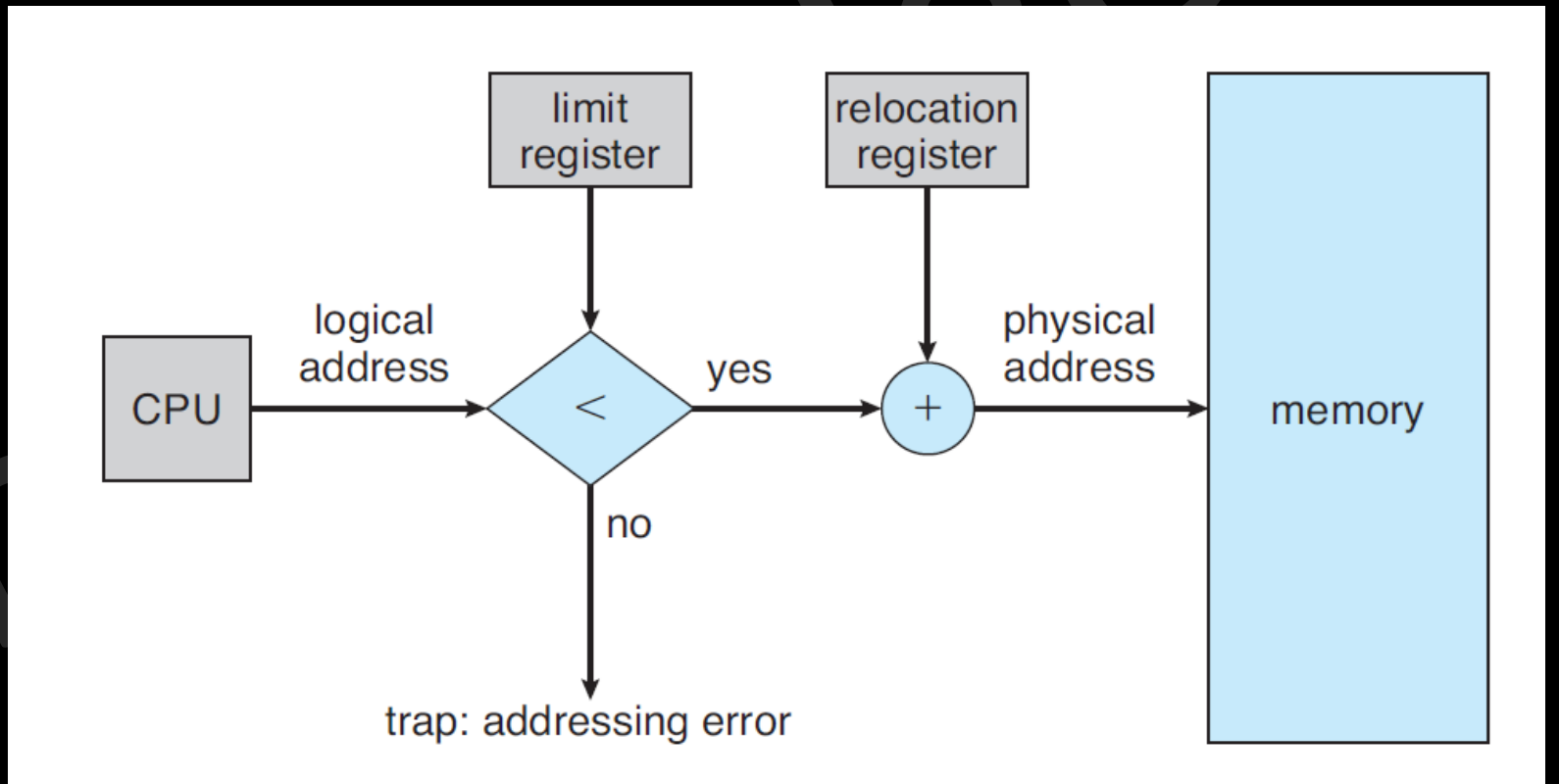
Knowledge Gate Website

Address Translation in Contiguous Allocation

Here we use a Memory Management Unit(OS) which contains a relocation register, which contains the base address of the process in the main memory and it is added in the logical address every time.

In order to check whether address generated to CPU is valid(with in range) or invalid, we compare it with the value of limit register, which contains the max no of instructions in the process.

So, if the value of logical address is less than limit, then it means it's a valid request and we can continue with translation otherwise, it is a illegal request which is immediately trapped by OS.



Space Allocation Method in Contiguous Allocation

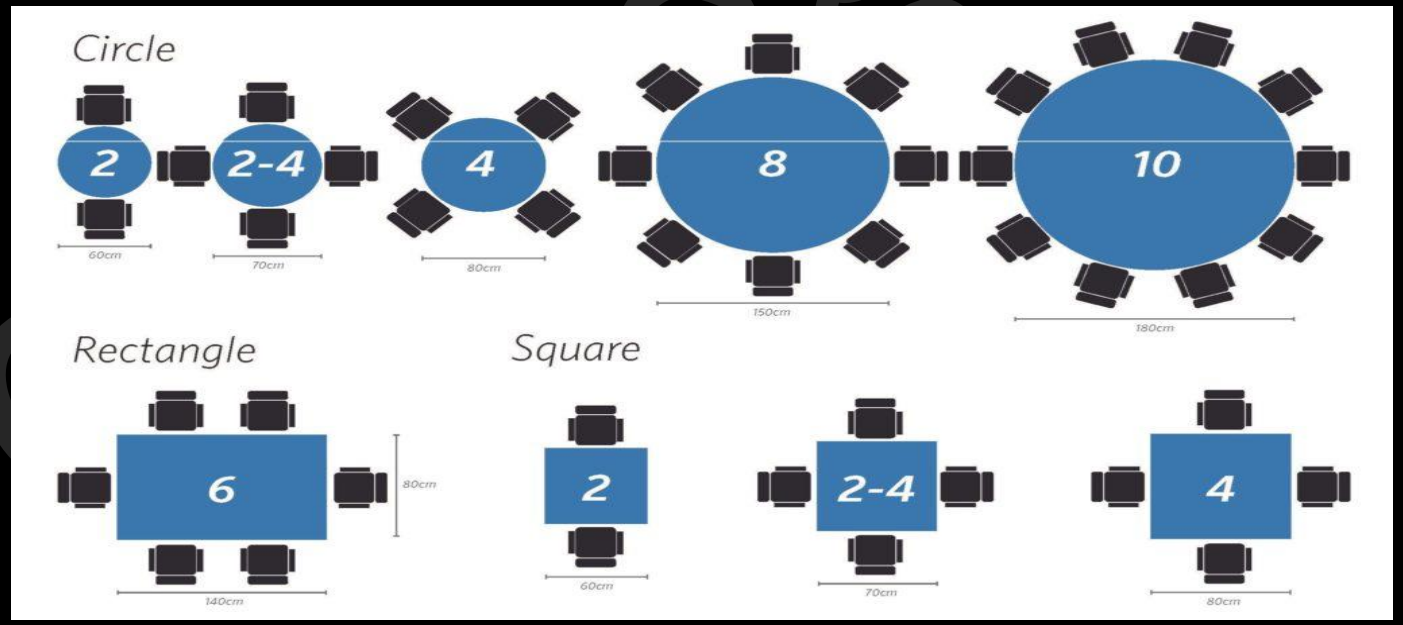
Variable size partitioning: -In this policy, in starting, we treat the memory as a whole or a single chunk & whenever a process request for some space, exactly same space is allocated if possible and the remaining space can be reused again.

Knowledge Gate

Knowledge Gate Website

Fixed size partitioning: - here, we divide memory into fixed size partitions, which may be of different sizes, but here if a process request for some space, then a partition is allocated entirely if possible, and the remaining space will be waisted internally.

xcvxc



Knowledge Gate Website

First fit policy: - It states searching the memory from the base and will allocate first partition which is capable enough.

Advantage: - simple, easy to use, easy to understand

Disadvantage: -poor performance, both in terms of time and space

Knowledge Gate

Knowledge Gate Website

Best fit policy: - We search the entire memory and will allocate the smallest partition which is capable enough.

Advantage: - perform best in fix size partitioning scheme.

Disadvantage: - difficult to implement, perform worst in variable size partitioning as the remaining spaces which are of very small size.

Knowledge Gate

Knowledge Gate Website

Worst fit policy: - It also searches the entire memory and allocate the largest partition possible.

Advantage: - perform best in variable size partitioning

Disadvantage: - perform worst in fix size partitioning, resulting into large internal fragmentation.

Knowledge Gate

Knowledge Gate Website

Q Consider five memory partitions of size 100 KB, 500 KB, 200 KB, 300 KB, and 600 KB, where KB refers to kilobyte. These partitions need to be allotted to four processes of sizes 212 KB, 417 KB, 112 KB and 426 KB in that order?

100	500	200	300	600

100	500	200	300	600

100	500	200	300	600

Knowledge Gate

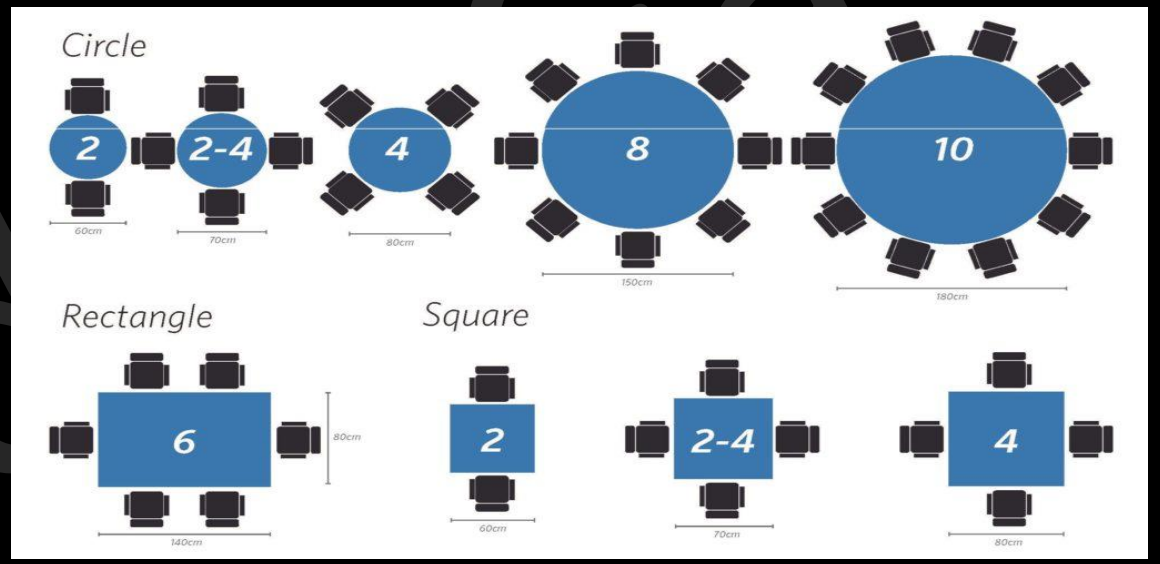
Knowledge Gate Website

Next fit policy: - Next fit is the modification in the best fit where, after satisfying a request, we start satisfying next request from the current position.

Knowledge Gate

Knowledge Gate Website

External fragmentation: - External fragmentation is a function of contiguous allocation policy. The space requested by the process is available in memory but, as it is not being contiguous, cannot be allocated this wastage is called external fragmentation.



The Big Cason Family want to celebrate a party

Knowledge Gate

Knowledge Gate Website

Internal fragmentation: - Internal fragmentation is a function of fixed size partition which means, when a partition is allocated to a process. Which is either the same size or larger than the request then, the unused space by the process in the partition Is called as internal fragmentation

Knowledge Gate Website

How can we solve external fragmentation?

We can also swap processes in the main memory after fixed intervals of time & they can be swapped in one part of the memory and the other part become empty(Compaction, defragmentation). This solution is very costly in respect to time as it will take a lot of time to swap process when system is in running state.

Either we should go for non-contiguous allocation, which means process can be divided into parts and different parts can be allocated in different areas.

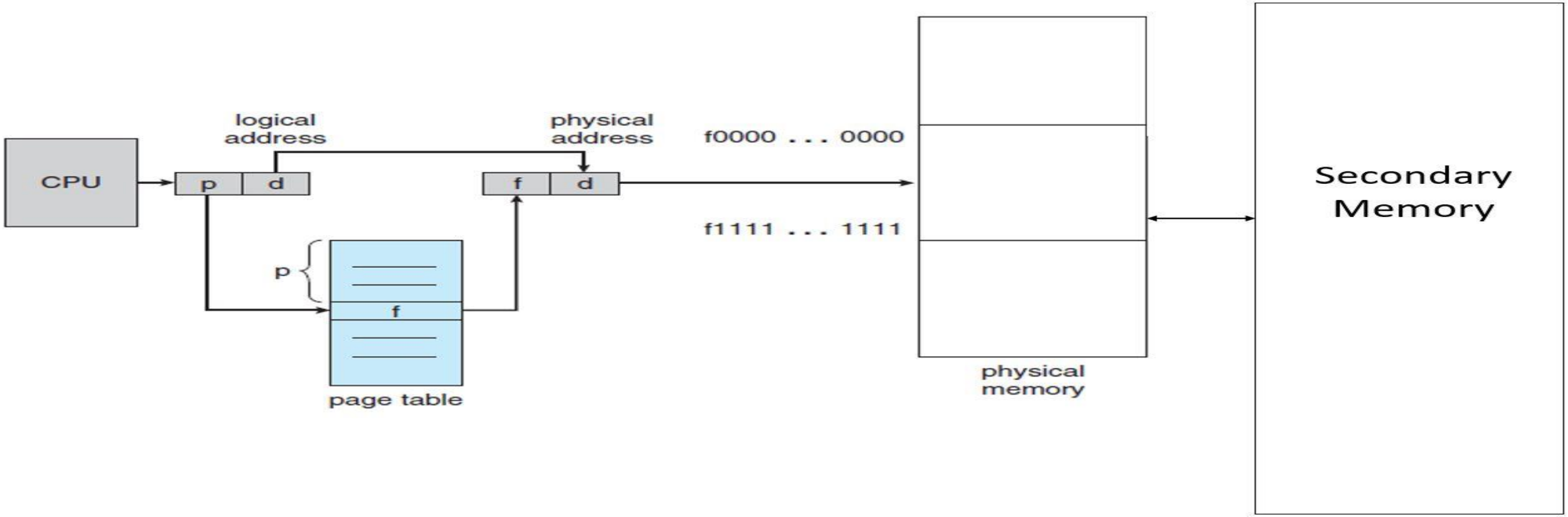
Knowledge Gate

Knowledge Gate Website

Non-Contiguous Memory allocation(Paging)

Paging is a memory-management scheme that permits the physical address space of a process to be non-contiguous.

Paging avoids external fragmentation

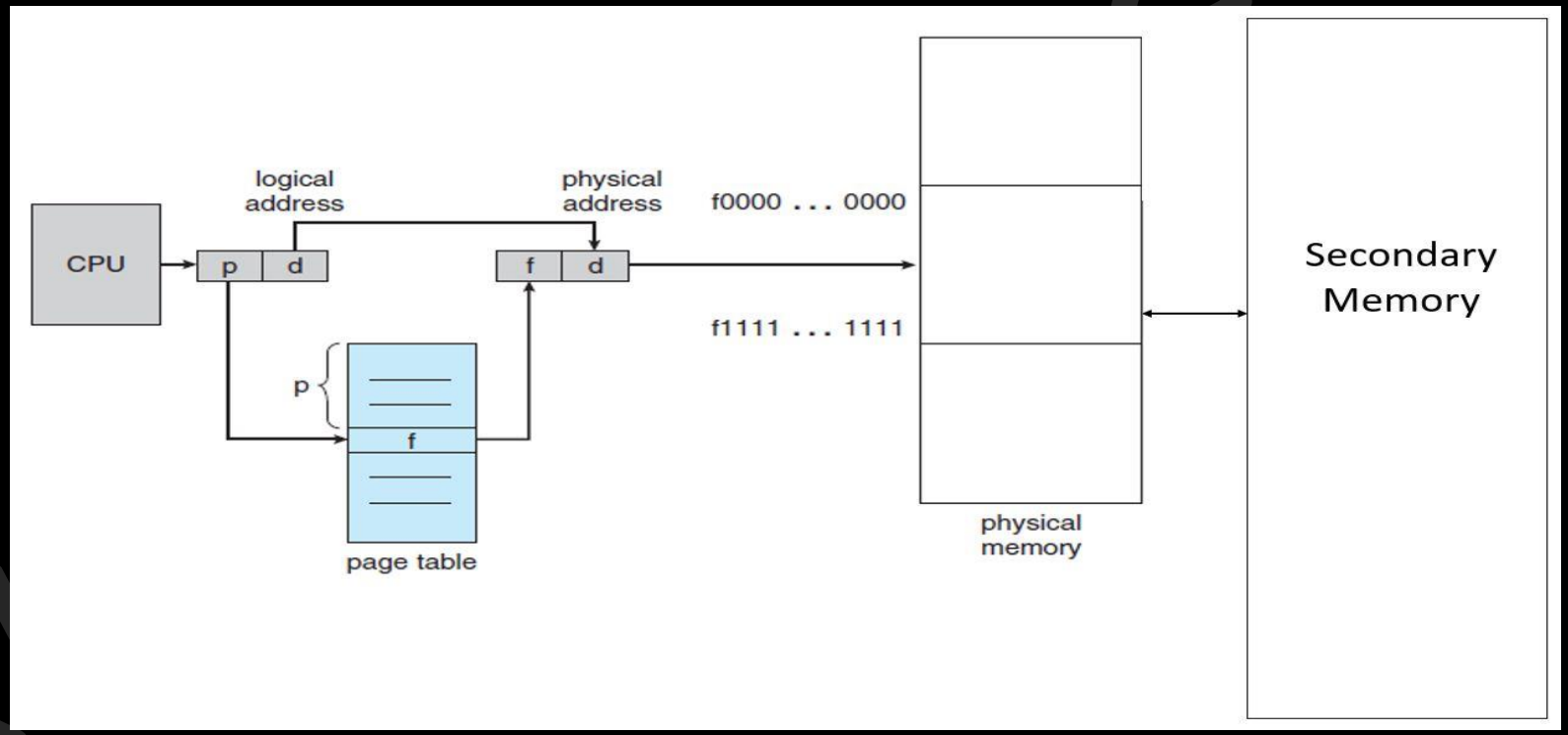


Secondary memory is divides into fixed size partition(because management is easy) all of them of same size called pages(easy swapping and no external fragmentation).

Main memory is divided into fix size partitions (because management is easy), each of them having same size called frames(easy swapping and no external fragmentation).

Size of frame = size of page

In general number of pages are much more than number of frames (approx. 128 time)



Knowledge Gate Website

Translation process

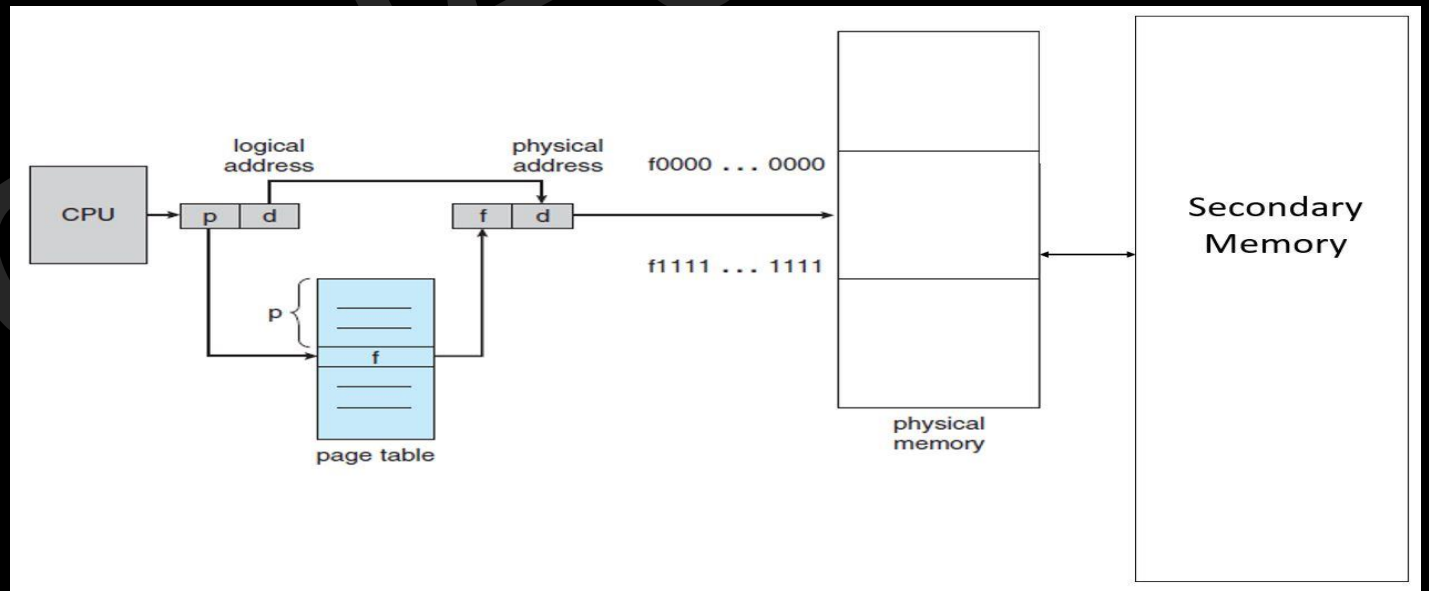
CPU generate a logical address is divided into two parts - **p** and **d**
where p stands for page no and d stands for instruction offset.

The **page number(p)** is used as an index into a **Page table**

Page table base register(PTBR) provides the base of the page table and then the corresponding page no is accessed using p.

Here we will finds the corresponding frame no (the base address of that frame in main memory in which the page is stored)

Combine corresponding frame no with the instruction offset and get the physical address. Which is used to access main memory.



Knowledge Gate

Page Table

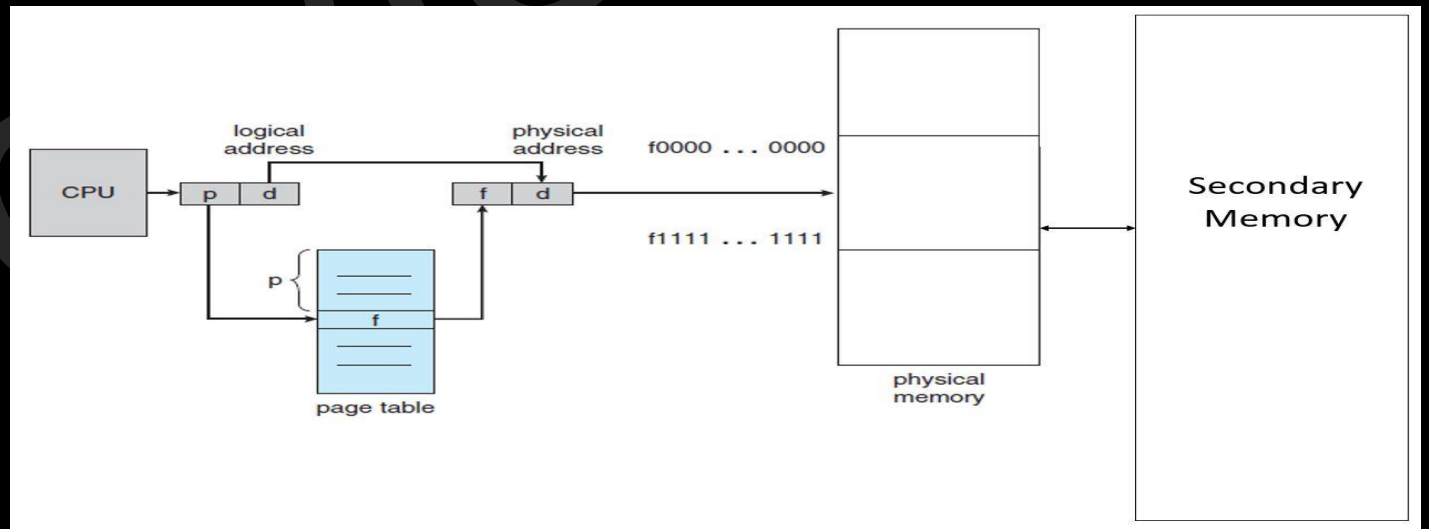
Page table is a data structure not hardware.

Every process have a separate page table.

Number of entries a process have in the page table is the number of pages a process have in the secondary memory.

Size of each entry in the page table is same it is corresponding frame number.

Page table is a data structure which is it self stored in main memory.



Knowledge Gate

Advantage

Removal of External Fragmentation

Disadvantage

Translation process is slow as Main Memory is accessed two times(one for page table and other for actual access).

A considerable amount of space a waisted in storing page table(meta data).

System suffers from internal fragmentation(as paging is an example of fixed size partition).

Translation process is difficult and complex to understand and implement.

Knowledge Gate

Knowledge Gate Website

10^3	1 Thousand	10^3	1 kilo	2^{10}	1 kilo
10^6	1 Million	10^6	1 Mega	2^{20}	1 Mega
10^9	1 Billion	10^9	1 Giga	2^{30}	1 Giga
10^{12}	1 Trillion	10^{12}	1 Tera	2^{40}	1 Tera
		10^{15}	1 Peta	2^{50}	1 Peta
		10^{18}	1 Exa	2^{60}	1 Exa
		10^{21}	1 Zetta	2^{70}	1 Zetta
		10^{24}	1 Yotta	2^{80}	1 Yotta

Knowledge Gate

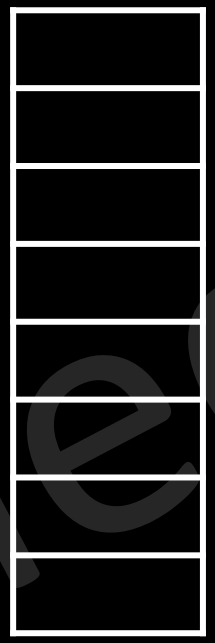
Knowledge Gate Website

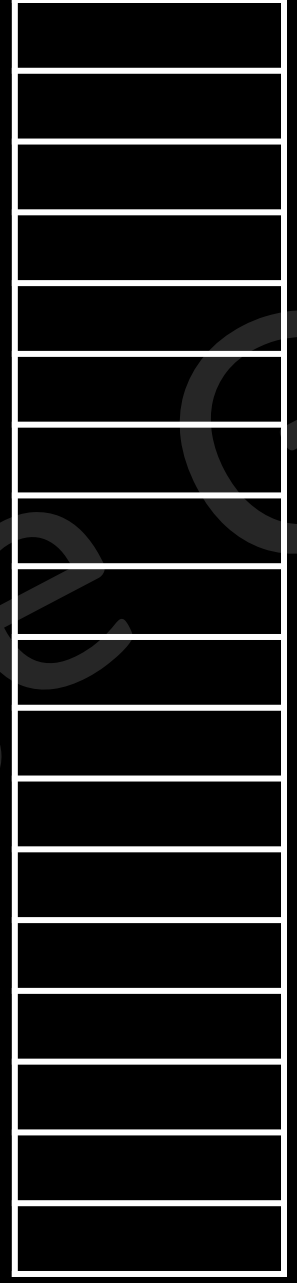
Address Length in bits	
No of Locations	2^n

Knowledge Gate

Knowledge Gate Website

Address Length in bits	
No of Locations	2^n



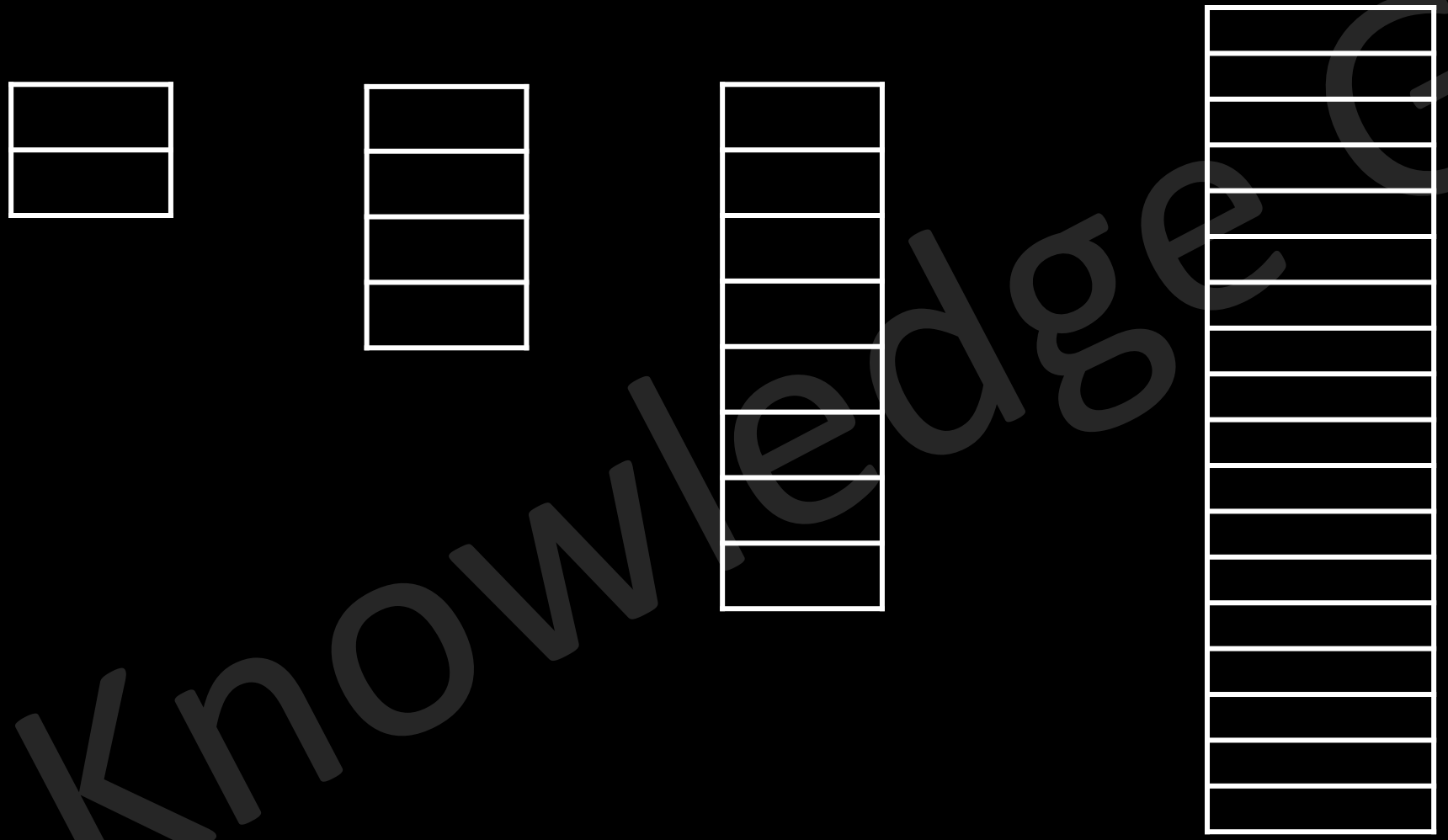


Memory Size = Number of Location * Size of each Location

Knowledge Gate

Knowledge Gate Website

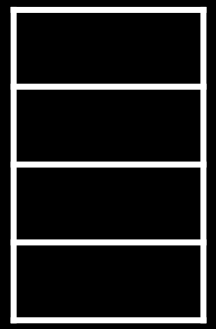
Address Length in bits	Upper Bound($Log_2 n$)
No of Locations	n

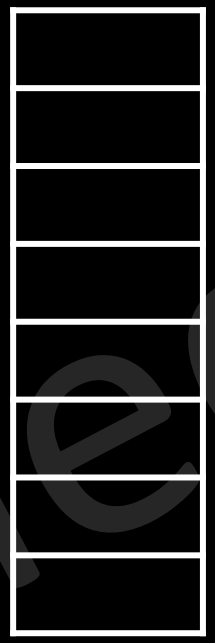


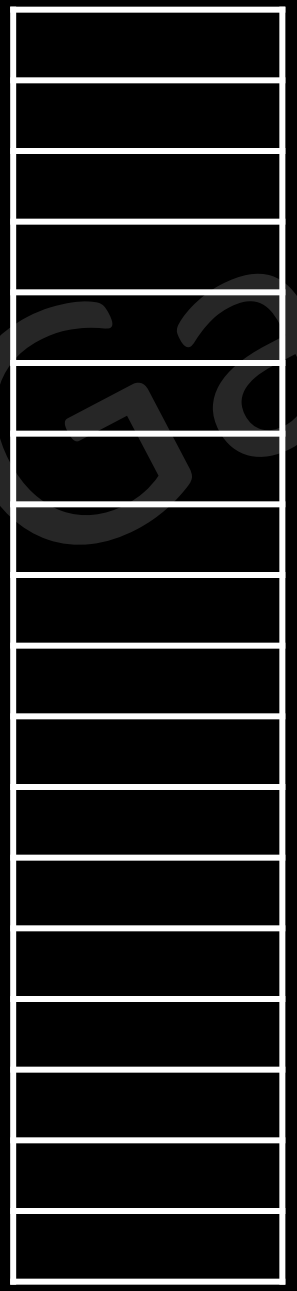
Knowledge Gate

Knowledge Gate Website

Address Length in bits	Upper Bound($Log_2 n$)
No of Locations	n







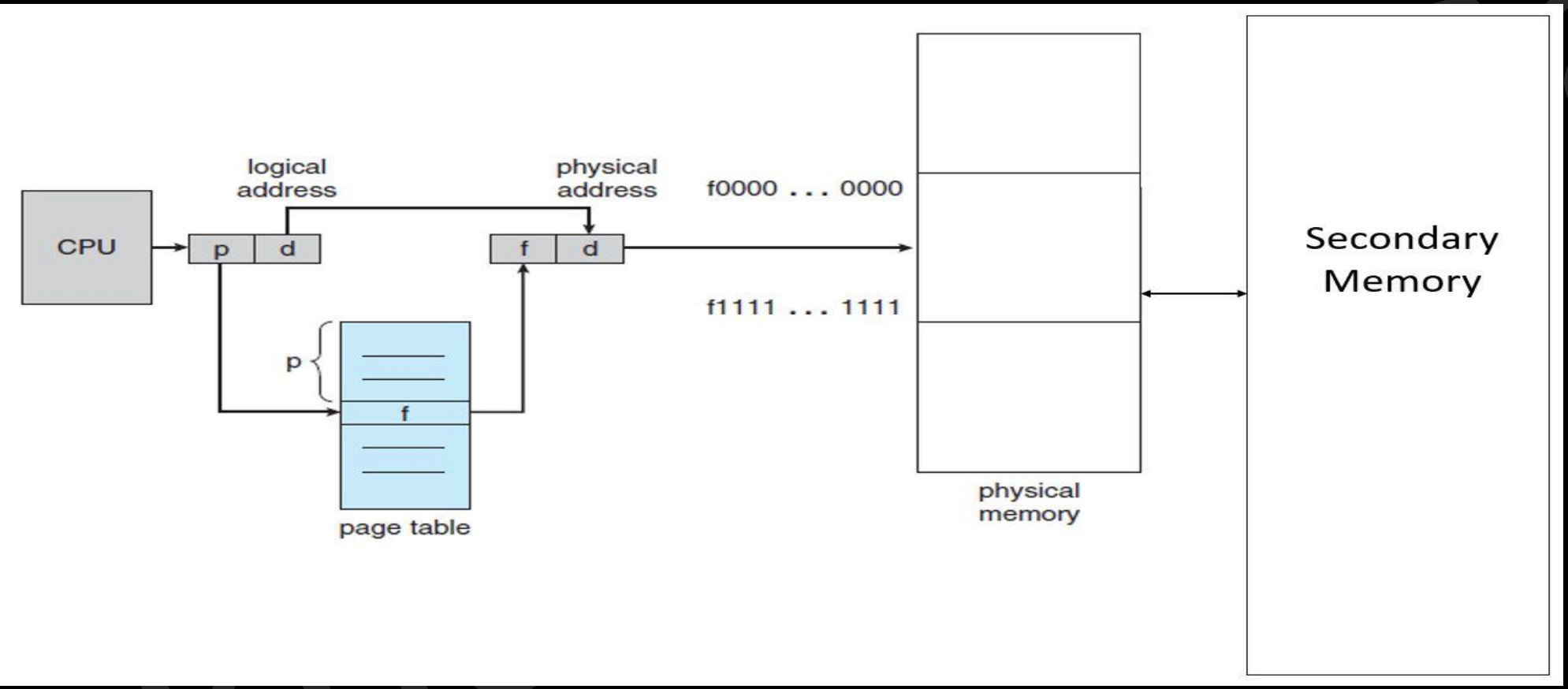
Memory Size/ Size of each Location = Number of Location

Knowledge Gate

Knowledge Gate Website

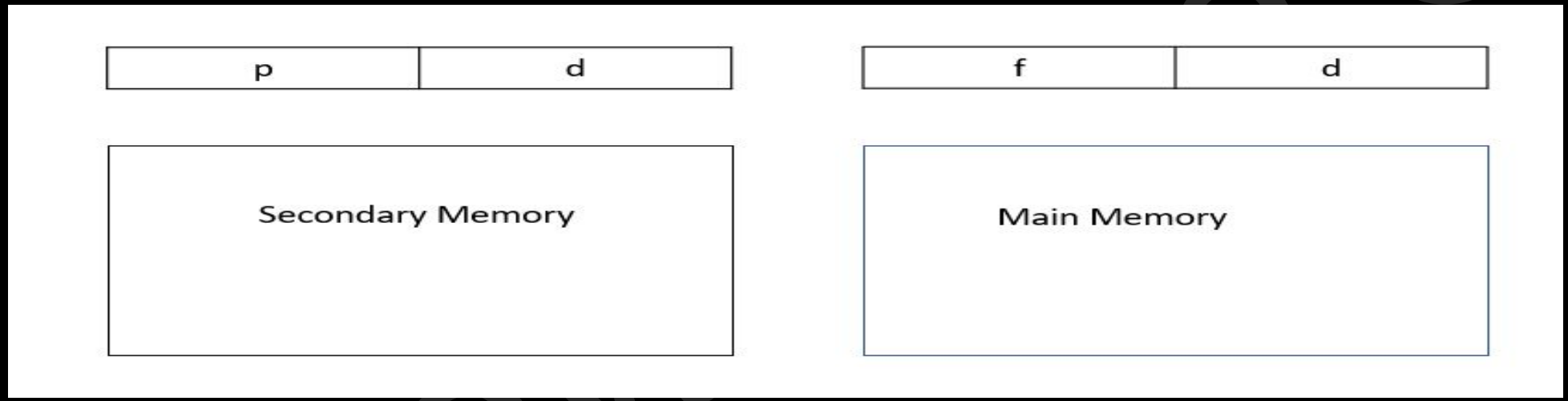
Page Table Size = No of entries in Page table * Size of each entry(f)

Process Size = No of Pages * Size of each page



Knowledge Gate Website

S No	SM	LA	MM	PA	p	f	d	addressable	Page Size
1	32 GB		128 MB					1B	1KB
2		42		33			11	1B	
3	512GB			31				1B	512B
4	128GB		32GB		30			1B	
5					28	14			4096B

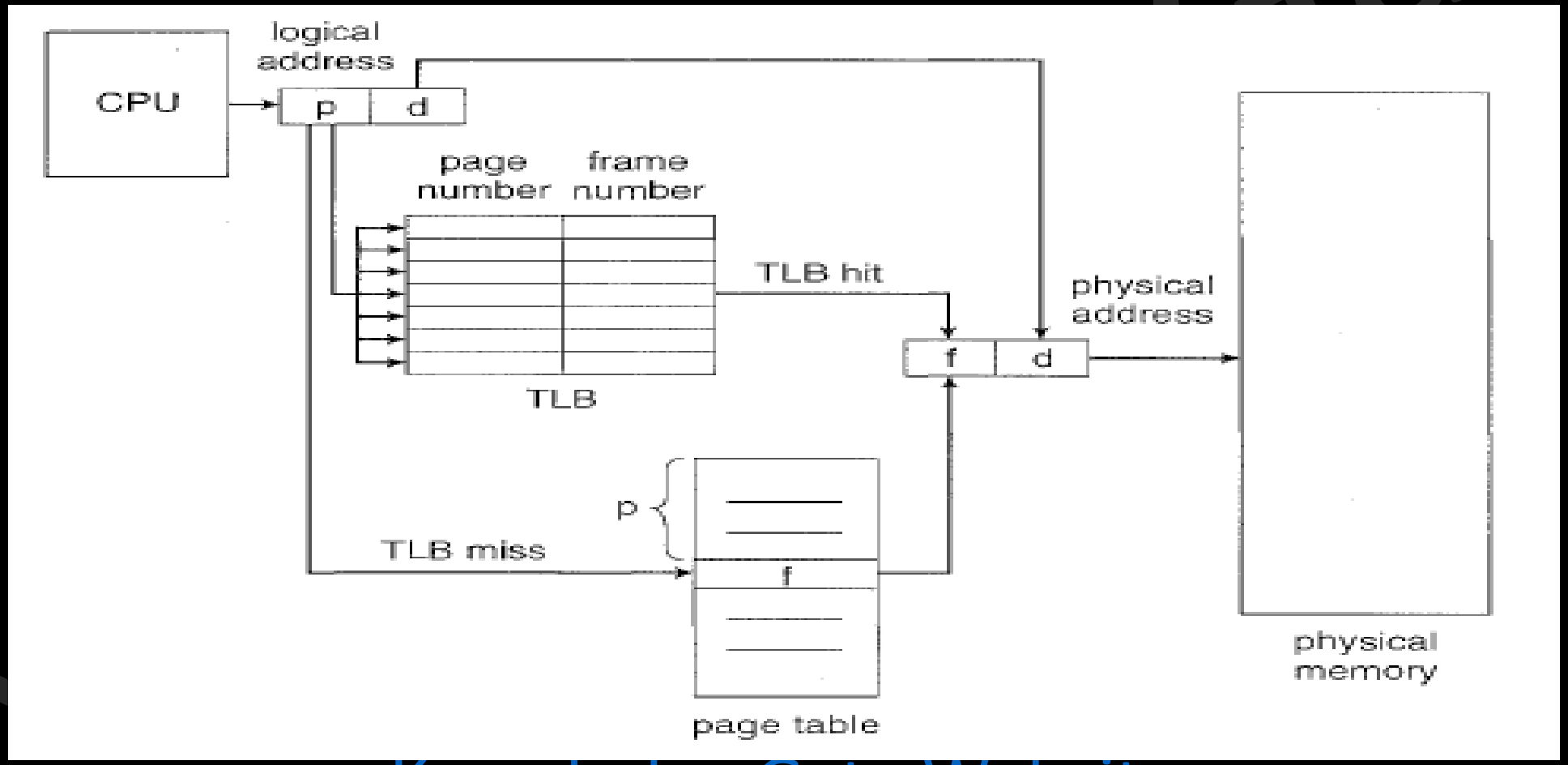


Knowledge Gate

Knowledge Gate Website

A serious problem with page is, translation process is slow as page table is accessed two times (one for page table and other for actual access).

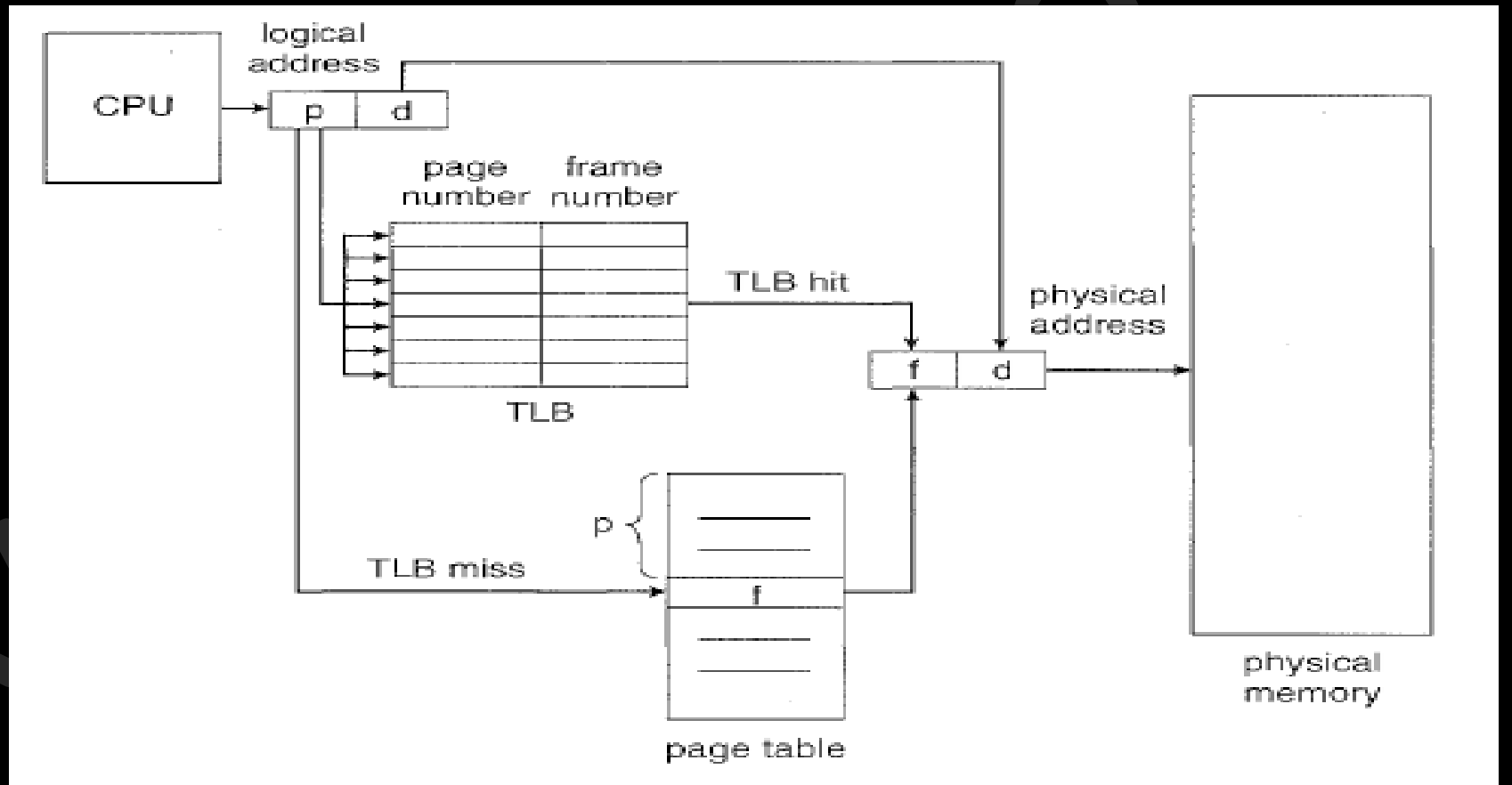
To solve the problems in paging we take the help of TLB. The TLB is associative, high-speed memory.



Knowledge Gate Website

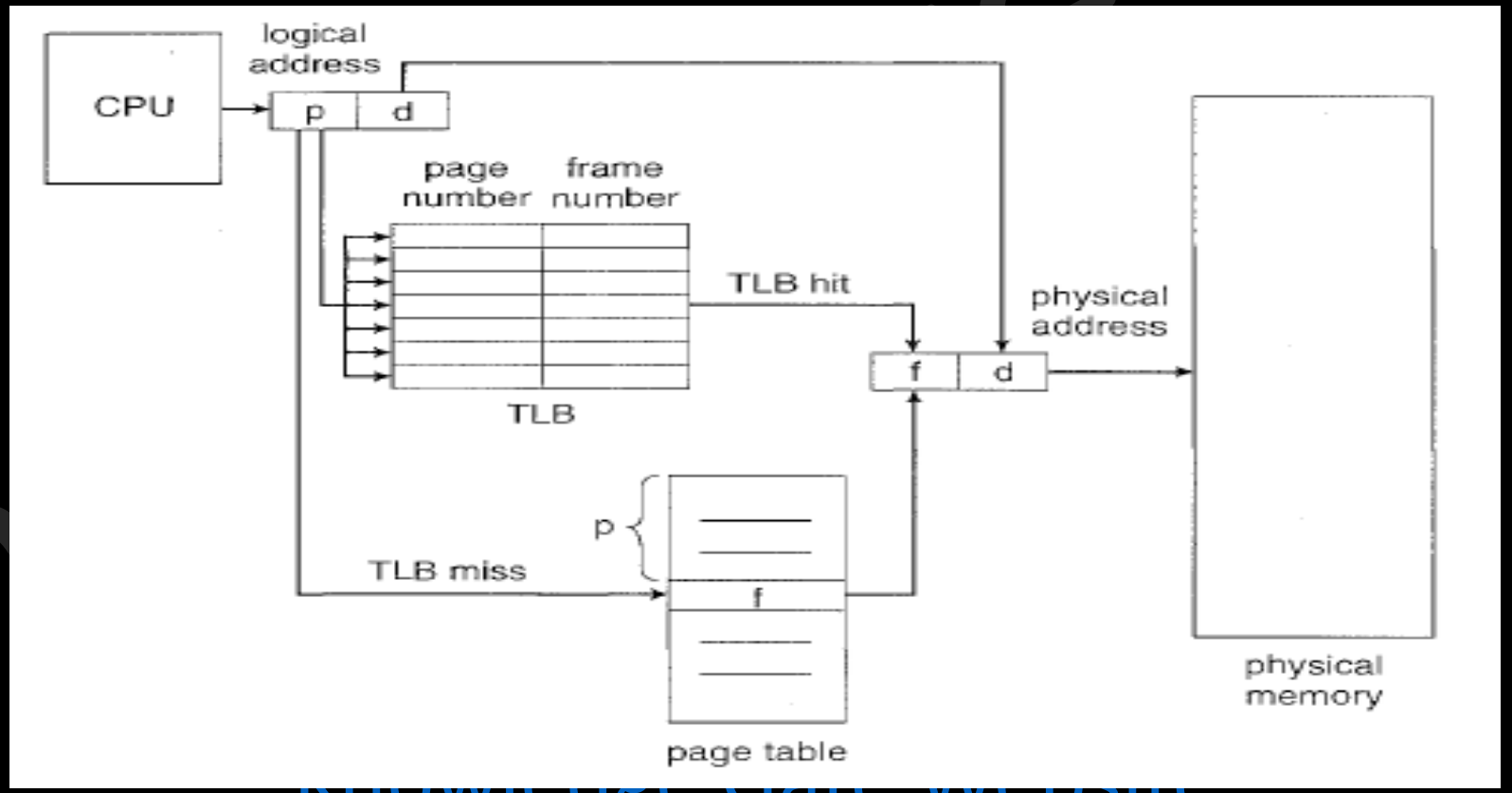
Each entry in the TLB consists of two parts: a key (Page no) and a value (frame no). When the associative memory is search for page no, the page no is compared with all page no simultaneously. If the item is found, the corresponding frame no field is returned.

The search is fast; the hardware, however, is expensive, TLB Contains the frequently referred page numbers and corresponding frame number.



The TLB is used with page tables in the following way. The TLB contains only a few of the page-table entries. When a logical address is generated by the CPU, its page number is presented to the TLB. If the page number is found, its frame number is immediately available and is used to access memory.

If the page number is not in the TLB (known as a **TLB Miss**), then a memory reference to the page table must be made.

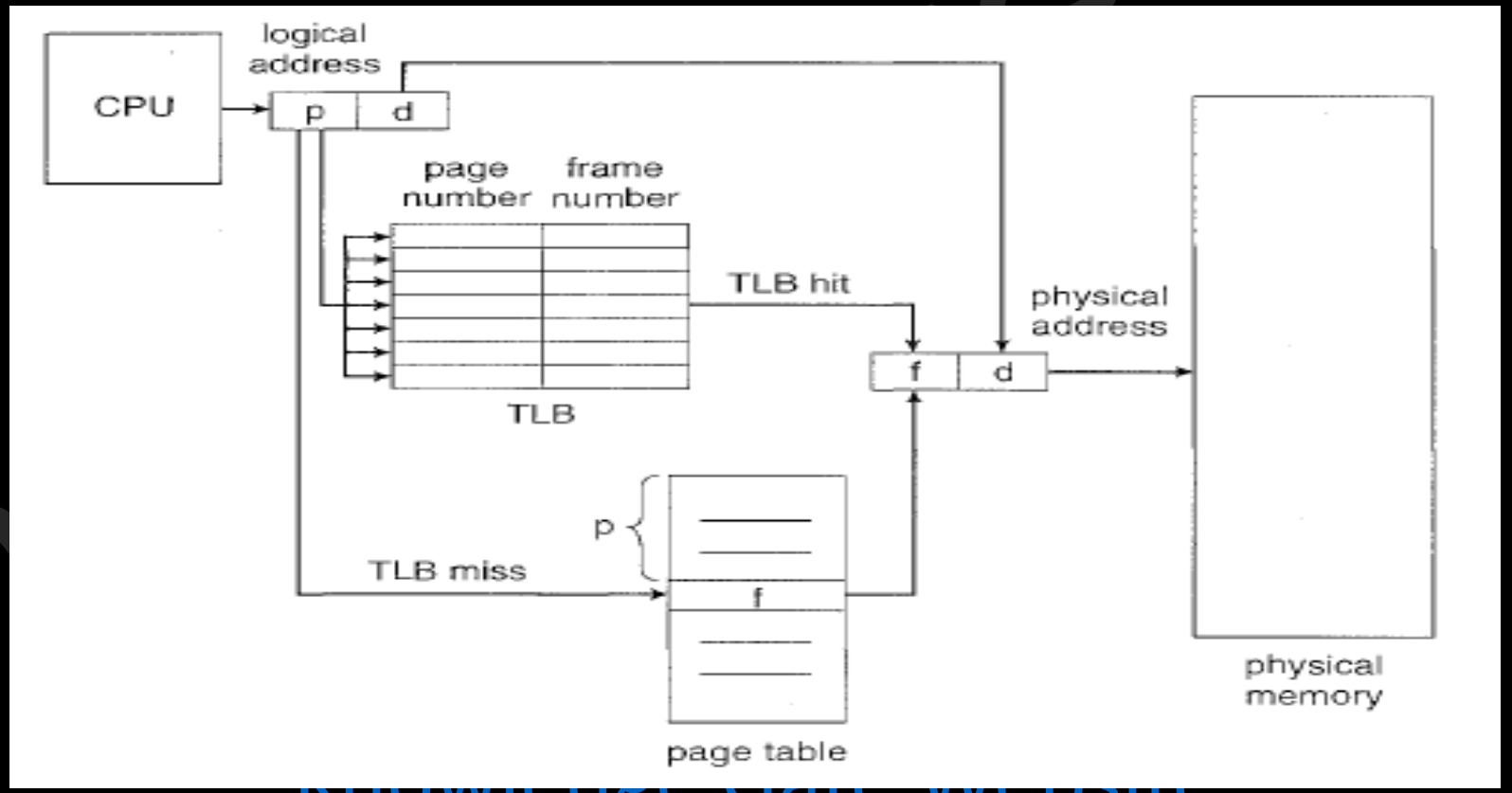


Knowledge Gate Website

Also we add the page number and frame number to the TLB, so that they will be found quickly on the next reference.

If the TLB is already full of entries, the operating system must select one for replacement i.e. Page replacement policies.

The percentage of times that a particular page number is found in the TLB is called the **Hit Ratio**.

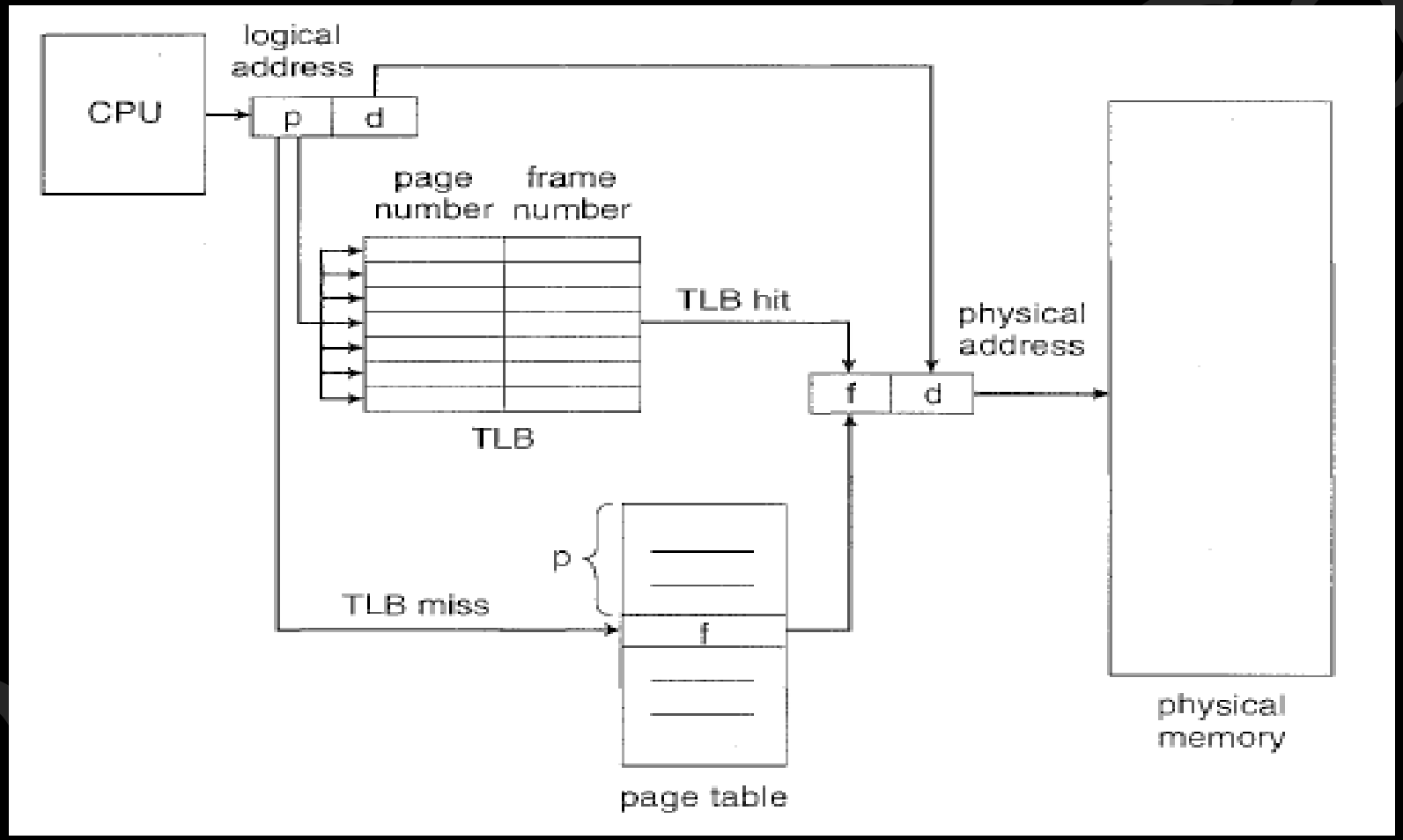


Knowledge Gate Website

Effective Memory Access Time:

Hit [TLB + Main Memory] + 1-Hit [TLB + 2 Main Memory]

TLB removes the problem of slow access.



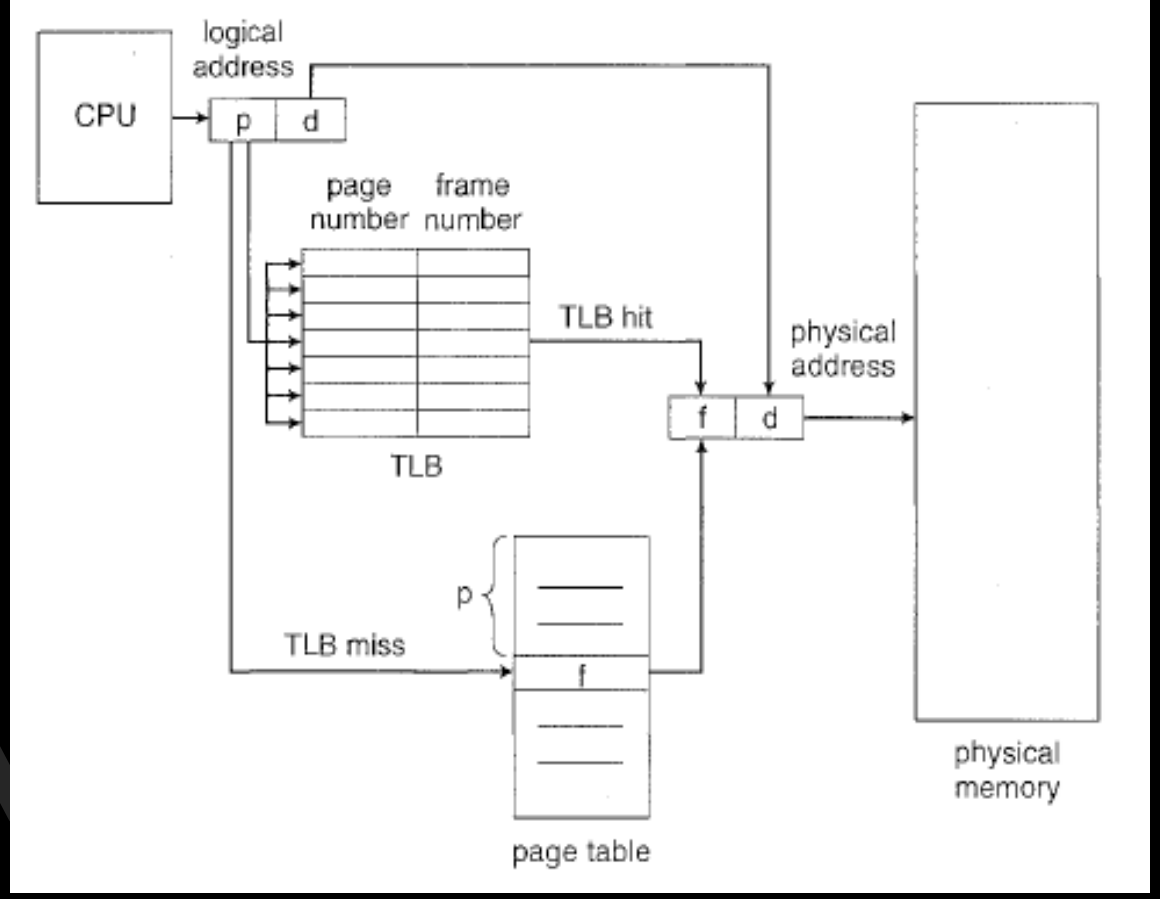
Knowledge Gate Website

Disadvantage of TLB:

TLB can hold the data of one process at a time and in case of multiple context switches TLB will be required to flush frequently.

Solution:

Use multiple TLB's but it will be costly. Some TLBs allow certain entries to be **wired down**, meaning that they cannot be removed from the TLB. Typically, TLB entries for kernel code are wired down.



Knowledge Gate Website

Size of Page

If we increase the size of page table then internal fragmentation increase but size of page table decreases.

If we decrease the size of page then internal fragmentation decrease but size of page table increases.

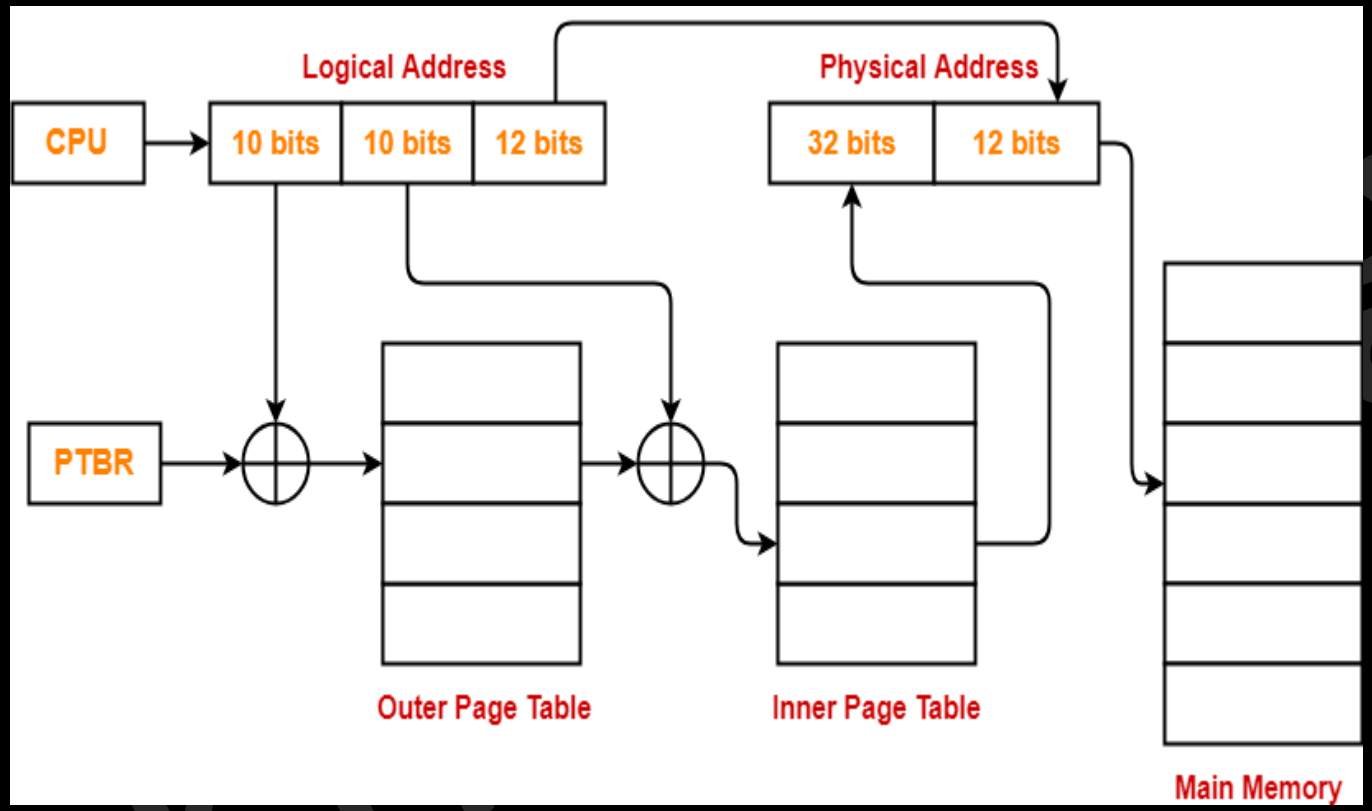
So we have to find what should be the size of the page, where both cost are minimal.

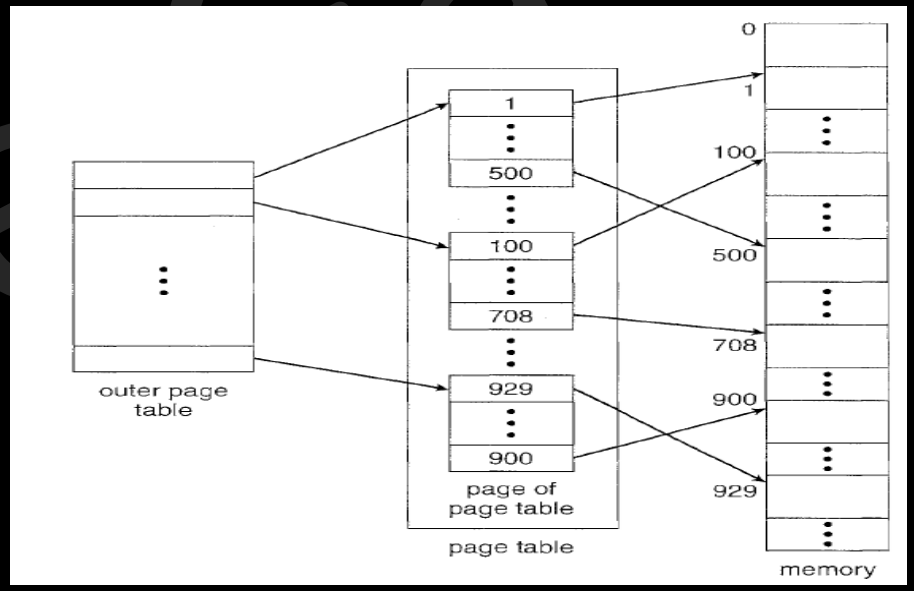
Knowledge Gate

Knowledge Gate Website

Multilevel Paging / Hierarchical Paging

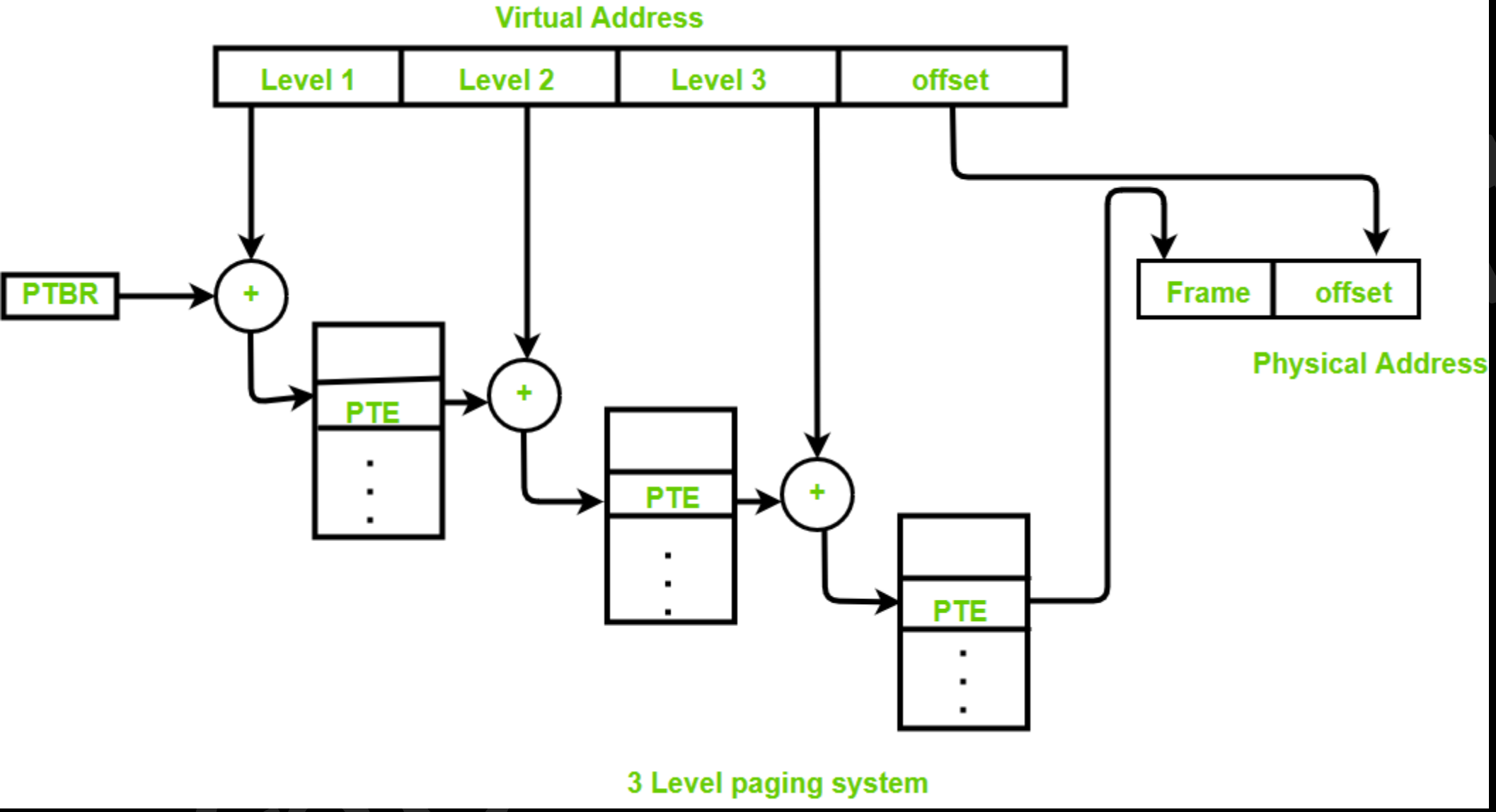
Modern systems support a large logical address space (2^32 to 2^64).
In such cases, the page table itself becomes excessively large and can contain millions of entries and can take a lot of space in memory, so cannot be accommodated into a single frame.
A simple solution to this is to divide page table into smaller pieces.
One way is to use a **two-level paging algorithm**, in which the page table itself is also paged.





Knowledge Gate

Knowledge Gate Website



3 Level paging system

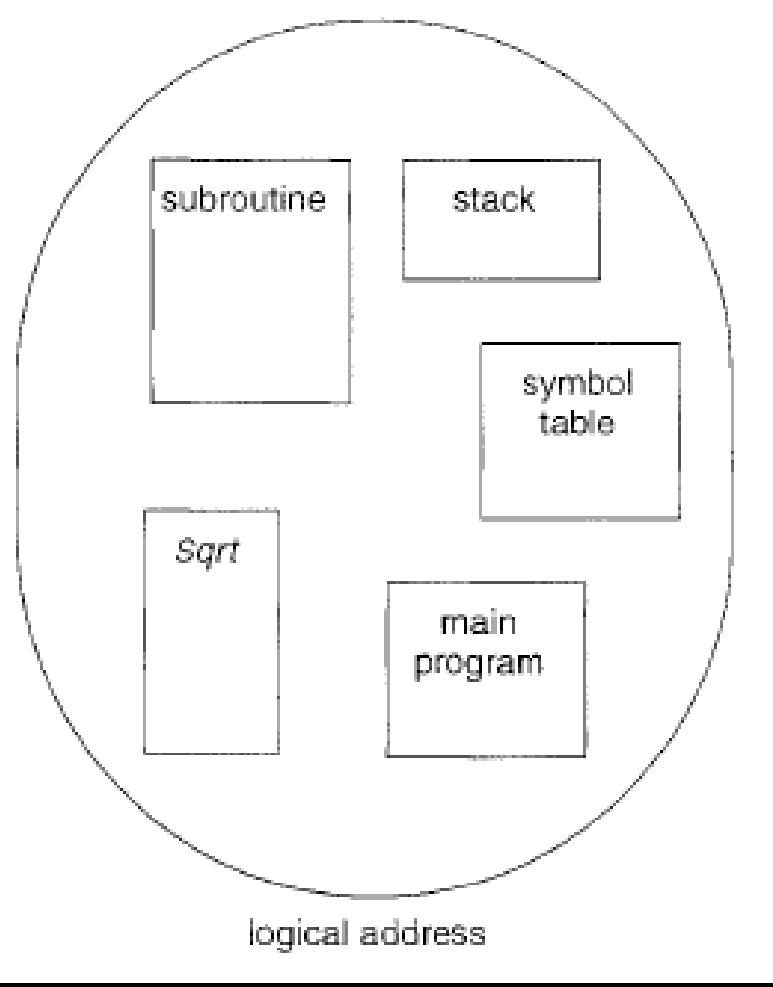
Knowledge Gate Website

Segmentation

Paging is unable to separate the user's view of memory from the actual physical memory. Segmentation is a memory-management scheme that supports this user view of memory.

A logical address space is a collection of segments. Each segment has a name and a length. The addresses specify both the segment name and the offset within the segment. The user therefore specifies each address by two quantities: **a segment name and an offset.**

Segments can be of variable lengths unlike pages and are stored in main memory.

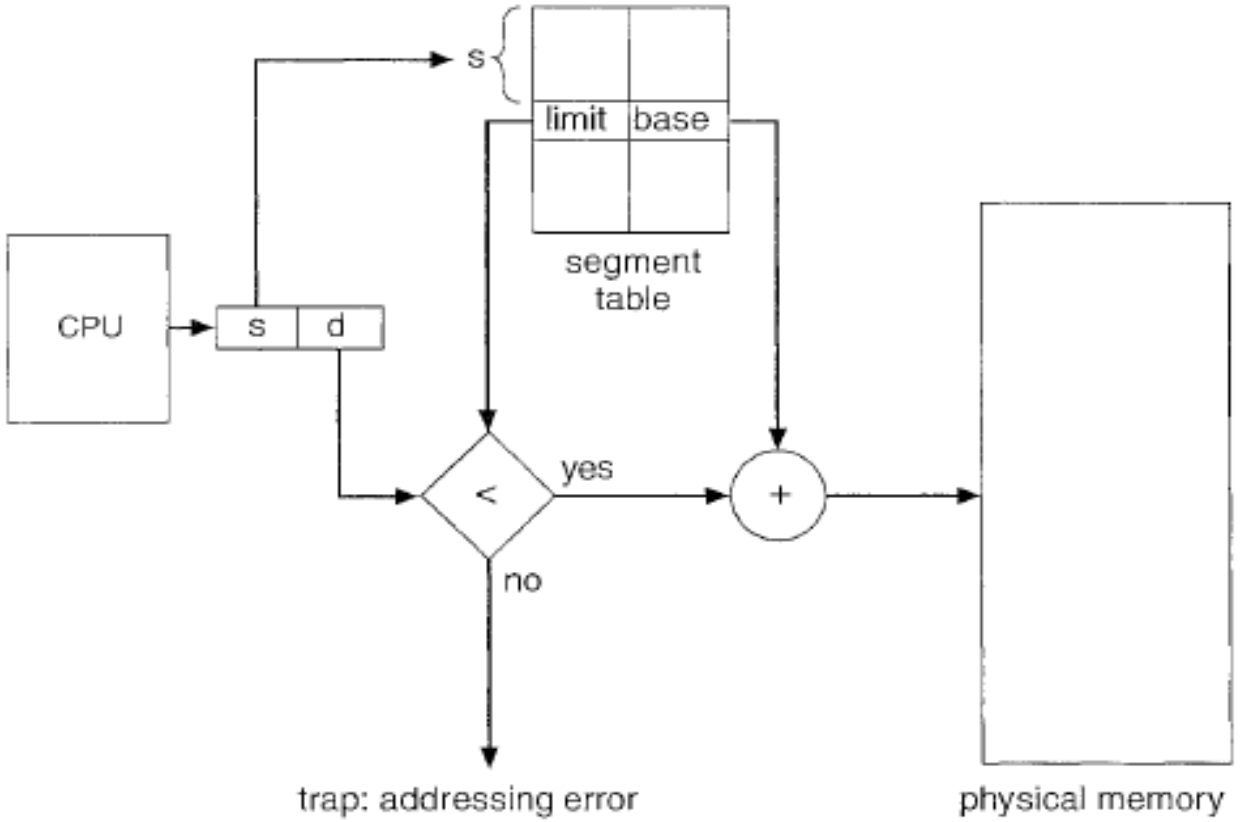


Knowledge Gate Website

Segment Table: Each entry in the segment table ha a **segment base** and a **segment limit**. The segment base contains the starting physical address where the segment resides in memory, and the segment limit specifies the length of the segment.

The segment number is used as an index to the segment table. The offset d of the logical address must be between 0 and the segment limit. If it is not, we trap to the operating system.

When an offset is legal, it is added to the segment base to produce the address in physical memory of the desired byte.Segmentation suffers from **External Fragmentation.**



Knowle

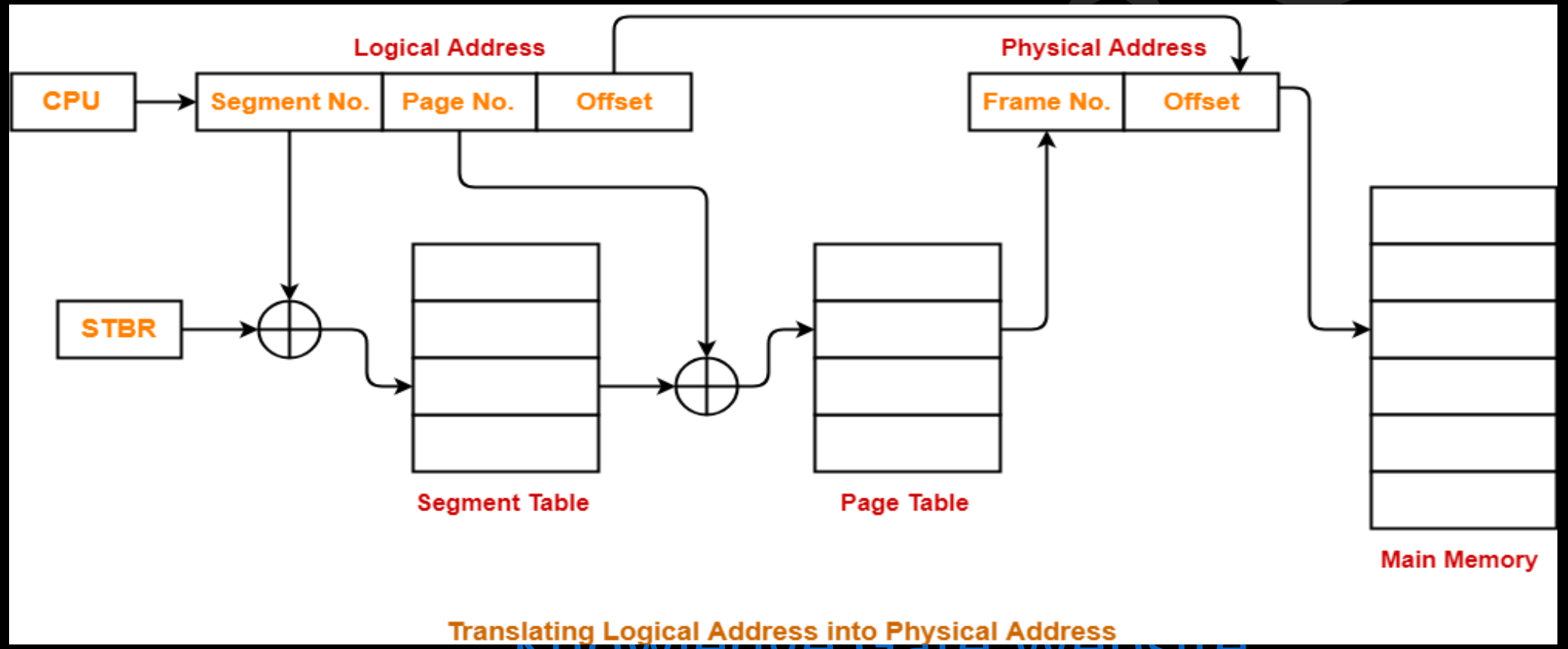
Knowledge Gate Website

Segmentation with Paging

Since segmentation also suffers from external fragmentation, **it is better to divide the segments into pages.**

In segmentation with paging a process is divided into segments and further the segments are divided into pages.

One can argue it is segmentation with paging is quite similar to multilevel paging, but actually it is better, because here when page table is divided, the size of partition can be different (as actually the size of different chapters can be different). All properties of segmentation with paging is same as multilevel paging.



Translating Logical Address into Physical Address

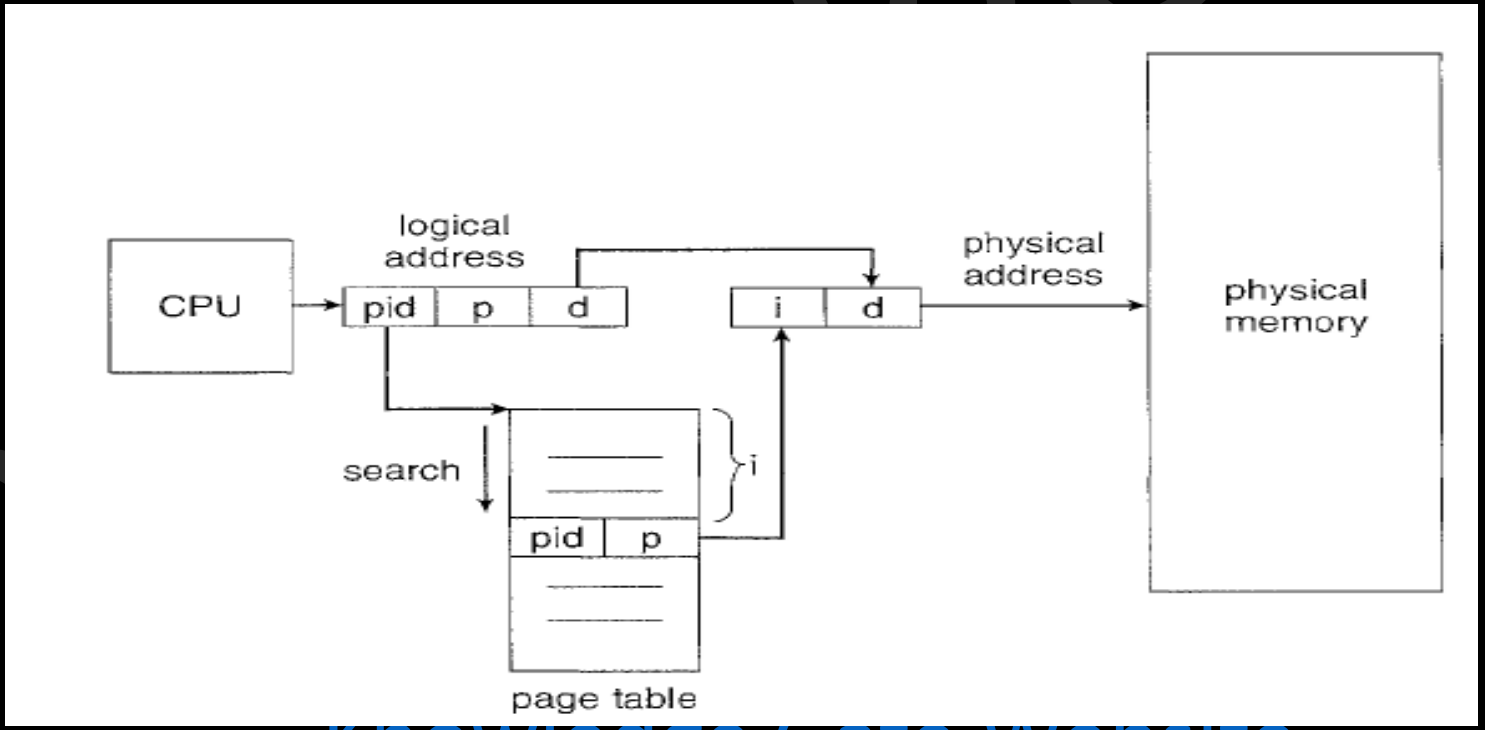
Knowledge Gate Website

Inverted Page Table

The drawback of paging is that each page table may consist of millions of entries. These tables may consume large amounts of physical memory just to keep track of how other physical memory is being used. To solve this problem, we can use an **Inverted Page Table.**

An inverted page table has one entry for each real page (or frame) of memory. Each entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns the page. Thus, only one page table is in the system, and it has only one entry for each page of physical memory.

Thus number of entries in the page table is equal to the number of frames in the physical memory.



Knowledge Gate Website

Virtual Memory

To enable multiprogramming and optimize memory, modern computing often uses pure demand paging to keep multiple processes in memory.

2. **Pure Demand Paging**: A memory management scheme where a process starts with no pages in memory. Pages are only loaded when explicitly required during execution.
    1. Process starts with zero pages in memory. Immediate page fault occurs.
    2. Load the needed page into memory. Execution resumes after the required page is loaded into memory.
    3. Additional page faults occur as the process continues and requires new pages.
    4. Execution proceeds without further faults once all necessary pages are in memory. The key principle is to only load pages when absolutely necessary.

Knowledge Gate

Knowledge Gate Website

Advantage

A program would no longer be constrained by the amount of physical memory that is available, Allows the execution of processes that are not completely in main memory, i.e. process can be larger than main memory.

More programs could be run at the same time as use of main memory is less.

Disadvantages

Virtual memory is not easy to implement.

It may substantially decrease performance if it is used carelessly (Thrashing)

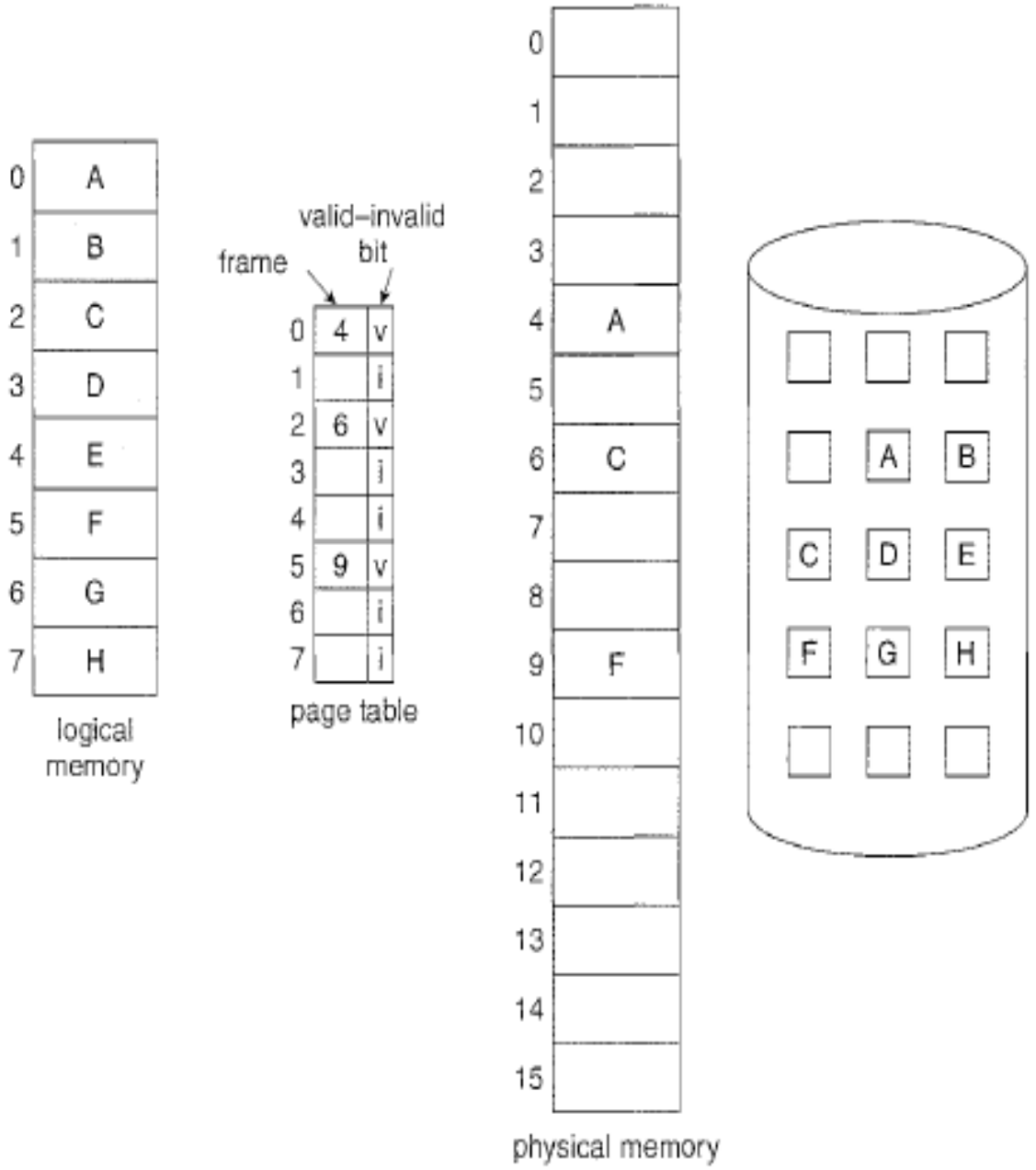
Knowledge Gate

Knowledge Gate Website

Implementation of Virtual Memory

We add a new column in page table, which have binary value 0 or Invalid which means page is not currently in main memory, 1 or valid which means page is currently in main memory.

Page Fault: - When a process tries to access a page that is not in main memory then a **Page Fault Occurs.**

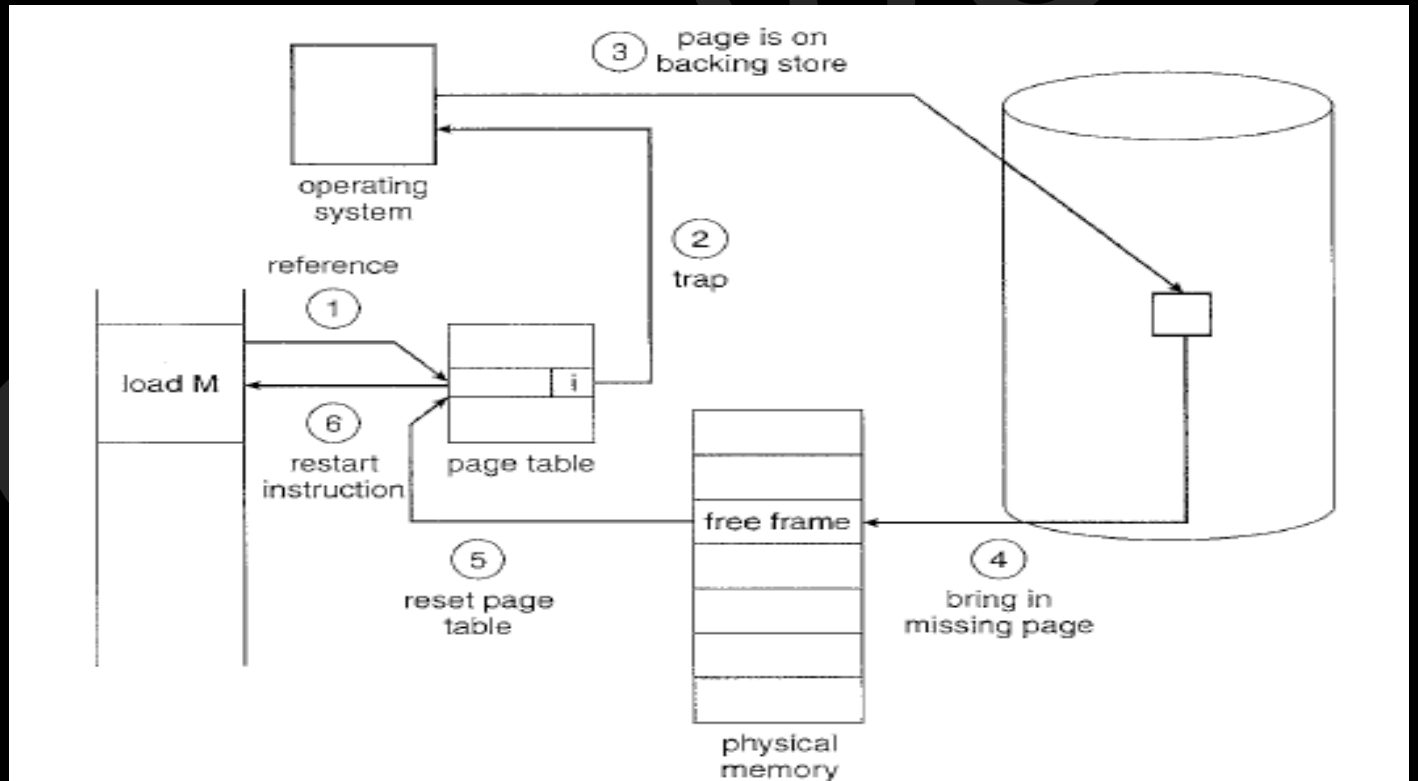


Knowled

Steps to handle Page Fault

1. If the reference was invalid, it means there is a page fault and page is not currently in main-memory, now we have to load this required page in main-memory.

2. We find a free frame if available we can brought in desired page, but if not we have to select a page as a victim and swap it out from main memory to secondary memory and then swap in the desired page(situation effectively doubles the page-fault service time).

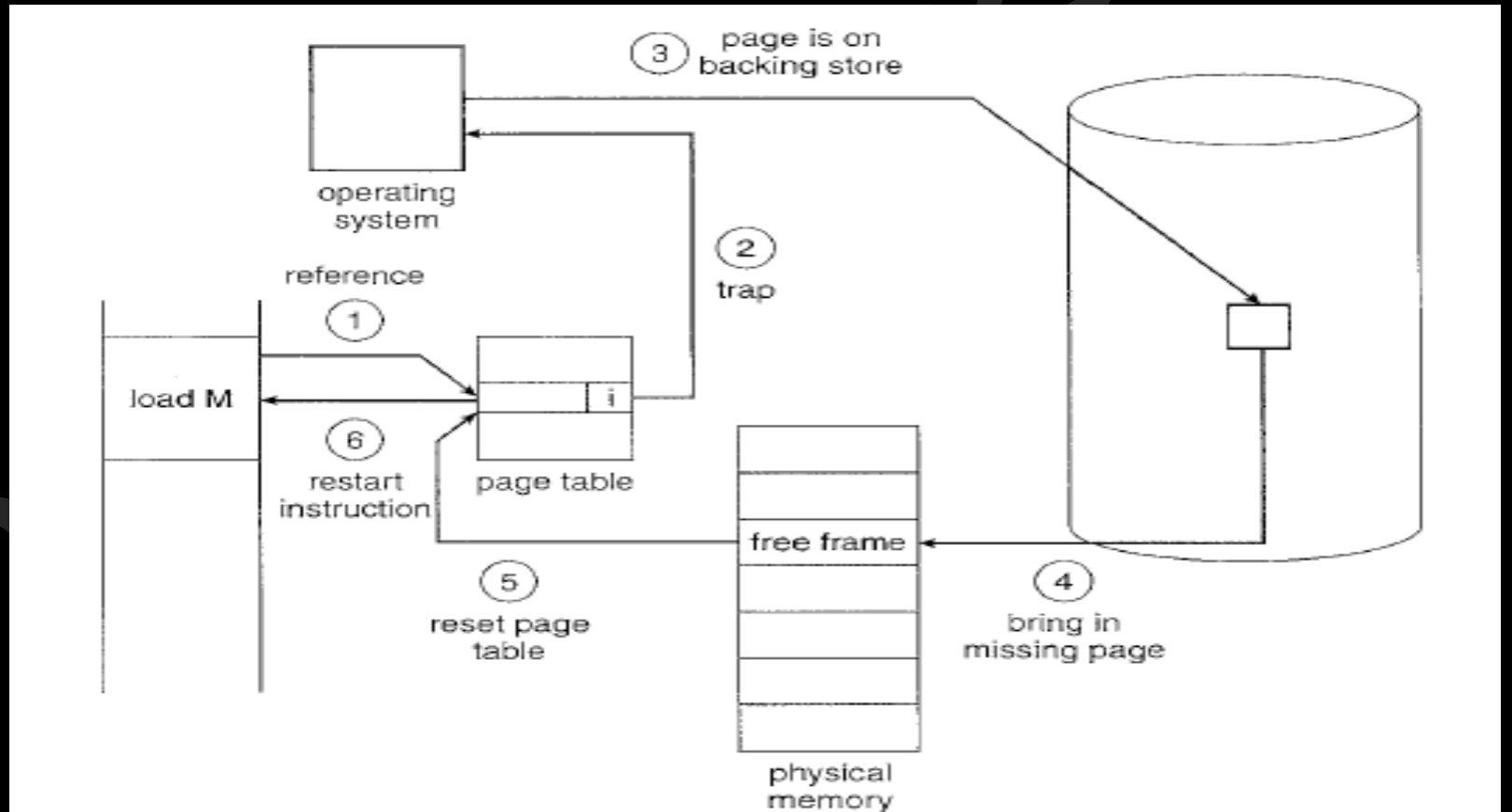


Knowledge Gate

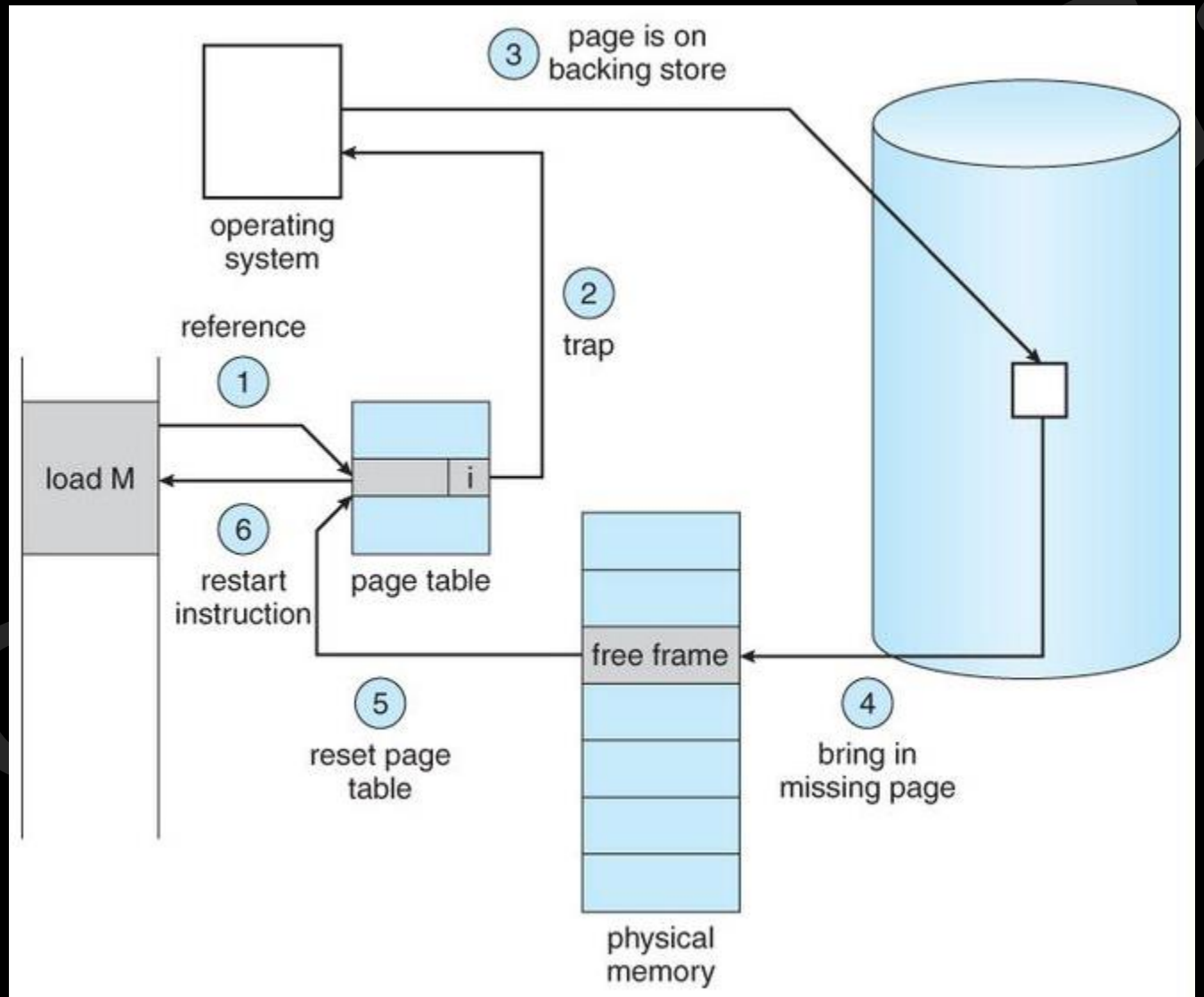
We can reduce this overhead by using a **Modify bit or Dirty Bit** as a new column in page table.

3.1. The modify bit for a page is set whenever the page has been modified. In this case, we must write the page to the disk.

3.2. If the modify bit is not set: It means the page has not been modified since it was read into the main memory. We need not write the memory page to the disk: it is already there.



4. Now we modify the internal table kept with the process(PCB) and the page table to indicate that the page is now in memory. We restart the instruction that was interrupted by the trap. The process can now access the page as though it had always been in memory.



Performance of Demand Paging

Effective Access time for Demand Paging:

(1 - p) x ma + p x page fault service time.

Here, p: Page fault rate or probability of a page fault.

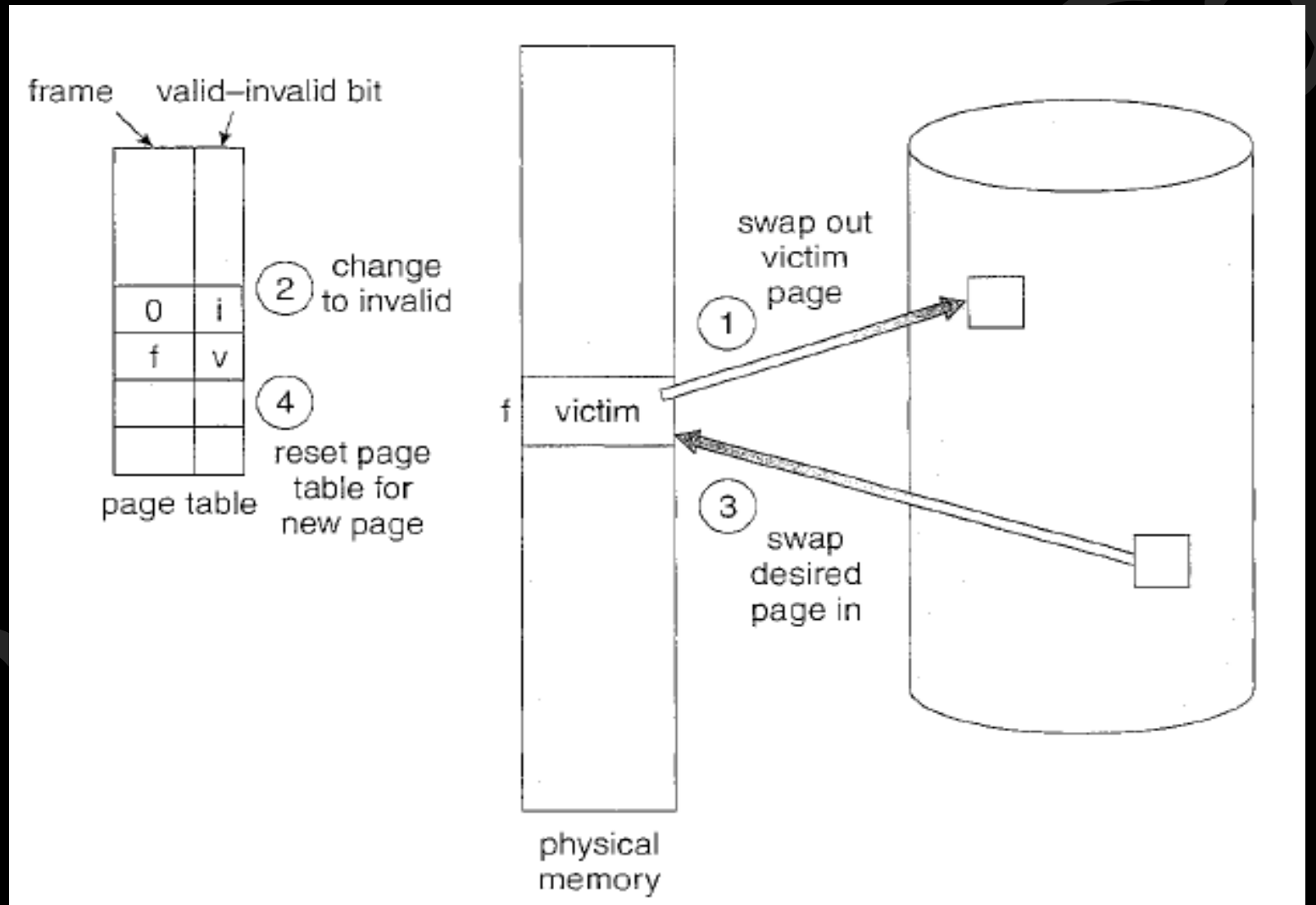
ma is memory access time.

Knowledge Gate

Knowledge Gate Website

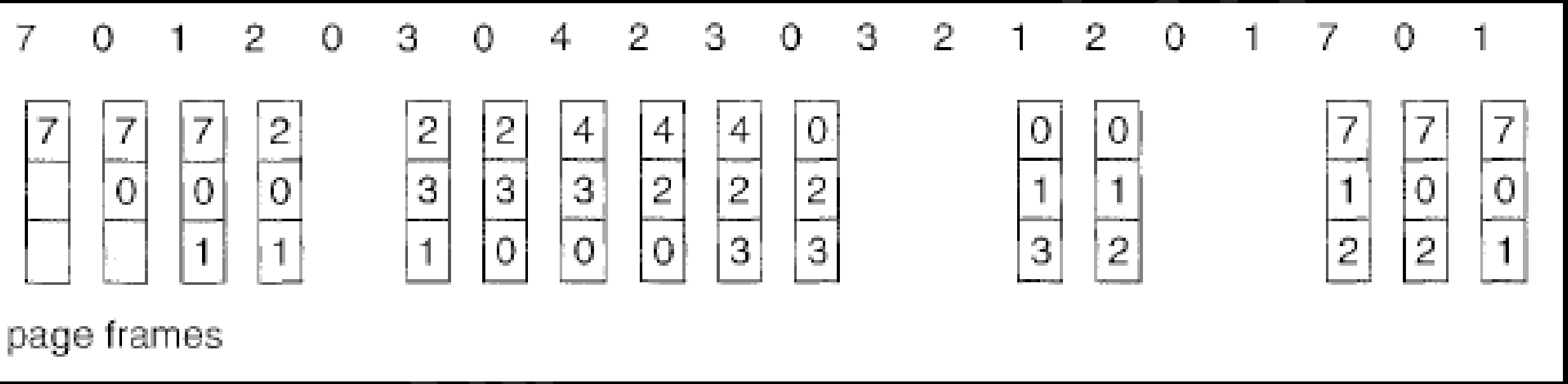
Page Replacement Algorithms

Now, we must solve a major problems to implement demand paging i.e. a **page replacement algorithm**. Page Replacement will decide which page to replace next.



Page Replacement Algorithms

First In First Out Page Replacement Algorithm: - A FIFO replacement algorithm associates with each page, the time when that page was brought into memory. When a page must be replaced, the oldest page is chosen. i.e. the first page that came into the memory will be replaced first.



In the above example the number of page fault is 15.

Knowledge Gate

Knowledge Gate Website

The FIFO page-replacement algorithm is easy to understand and program. However, its performance is not always good.

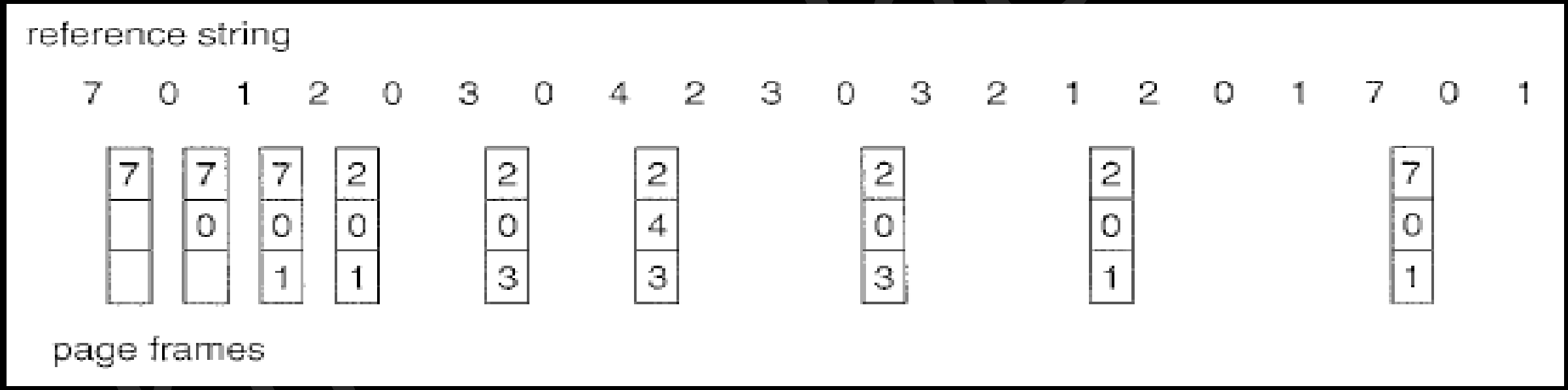
Belady's Anomaly: for some page-replacement algorithms, the page-fault rate may increase as the number of allocated frames increases.

Knowledge Gate

Knowledge Gate Website

Optimal Page Replacement Algorithm

Replace the page that will not be used for the longest period of time.
It has the lowest page-fault rate of all algorithms.
Guarantees the lowest possible page fault rate for a fixed number of frames and will never suffer from Belady's anomaly.
Unfortunately, the optimal page-replacement algorithm is difficult to implement, because it requires future knowledge of the reference string. It is mainly used for comparison studies.

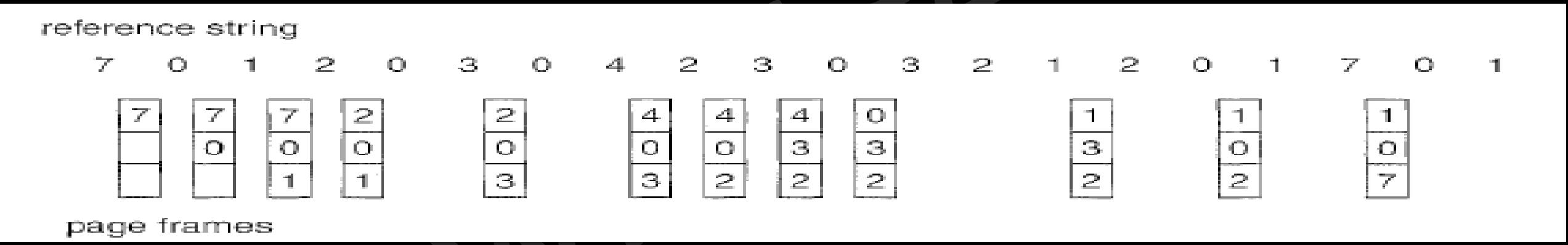


Knowledge Gate Website

Least Recently Used (LRU) Page Replacement Algorithm

We can think of this strategy as the optimal page-replacement algorithm looking backward in time, rather than forward. Replace the page that has not been used for the longest period of time.

LRU is much better than FIFO replacement in term of page-fault. The LRU policy is often used in industry. LRU also does not suffer from Belady's Anomaly.

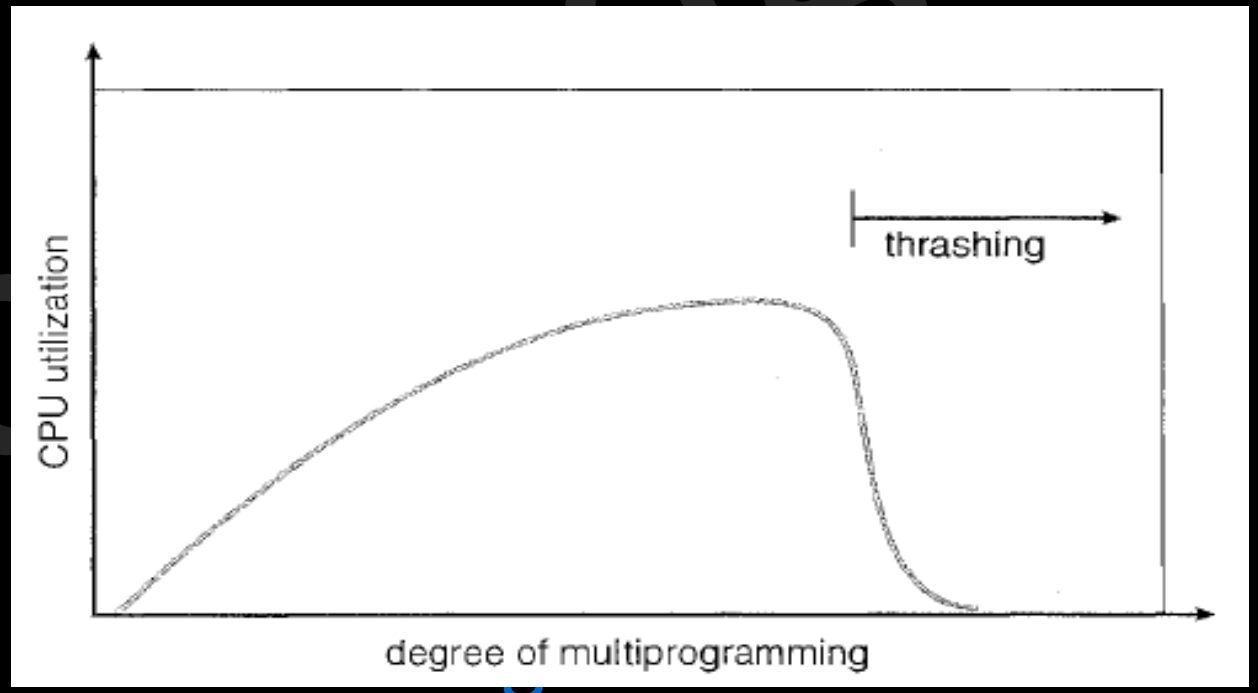


Knowledge Gate

Knowledge Gate Website

Thrashing

1. When a process spends more time swapping pages than executing, it's called Thrashing. Low CPU utilization prompts adding more processes to increase multiprogramming.
2. If a process needs more frames, it starts taking them from others, causing those processes to also fault and swap pages. This empties the ready queue and lowers CPU utilization.
3. Responding to decreased CPU activity, the scheduler adds more processes, worsening the issue. This leads to Thrashing: a cycle of increasing page faults, plummeting system throughput, and rising memory-access times, ultimately resulting in no productive work.



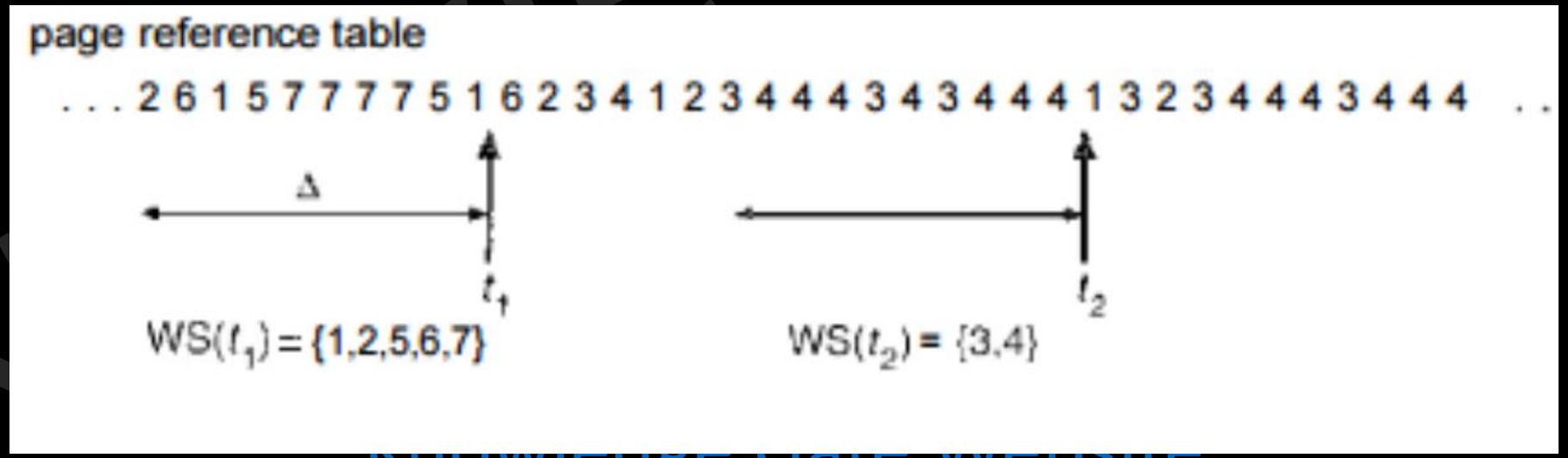
Knowledge Gate

Solution-The Working Set Strategy

This model uses a parameter Δ, to define the **working set window.** The set of pages in the most recent Δ page references is the working set.

If a page is in active use, it will be in the working set. If it is no longer being used, it will drop from the working set.

The working set is an approximation of the program's locality. The accuracy of the working set depends on the selection of Δ . If Δ is too small, it will not encompass the entire locality; if Δ is too large, it may overlap several localities.



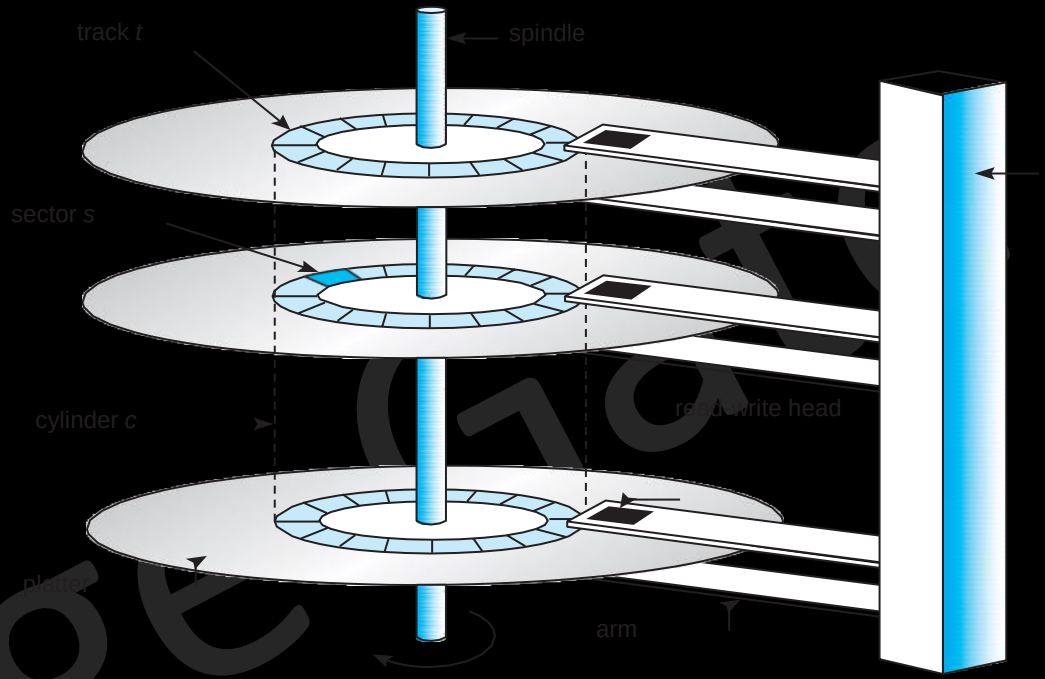
Knowledge Gate Website

Disk

Magnetic disks serve as the main secondary storage in computers. Each disk has a flat, circular platter with magnetic surfaces for data storage.

A read-write head hovers over these surfaces, moving in unison on a disk arm. Platters have tracks divided into sectors for logical data storage.

Disks spin at speeds ranging from 60 to 250 rotations per second, commonly noted in RPM like 5,400 or 15,000.



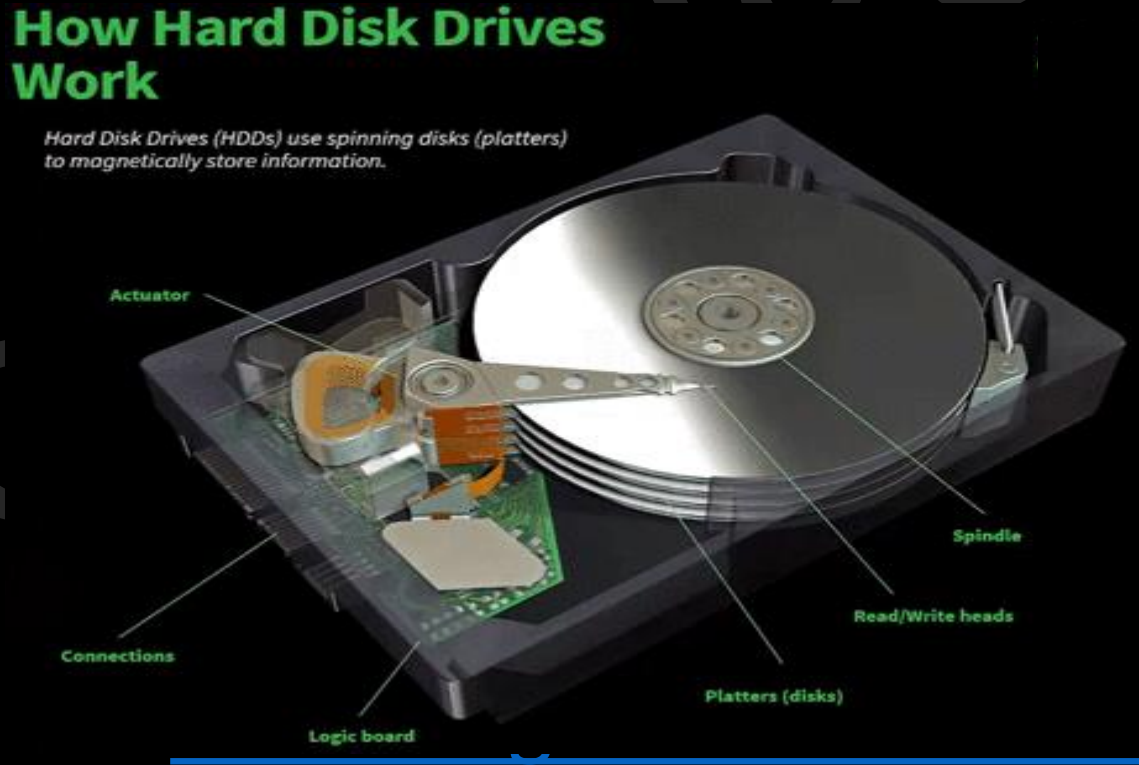
Knowledge Gate

Total Transfer Time = Seek Time + Rotational Latency + Transfer Time

Seek Time: - It is a time taken by Read/Write header to reach the correct track. (Always given in question)

Rotational Latency: - It is the time taken by read/Write header during the wait for the correct sector. In general, it's a random value, so far average analysis, we consider the time taken by disk to complete half rotation.

Transfer Time: - it is the time taken by read/write header either to read or write on a disk. In general, we assume that in 1 complete rotation, header can read/write the either track, so
total time will be = (File Size/Track Size) *time taken to complete one revolution.



Kn re Gate

Disk scheduling

One of the responsibilities of the operating system is to use the hardware efficiently. For the disk drives, efficiency means having less seek time, less waiting time and high data transfer rate. We can improve all of these by managing the order in which disk I/O requests are serviced.

2. Whenever a process needs I/O to or from the disk, it issues a system call to the operating system. The request may specifies several pieces of information: Whether this operation is input or output, disk address, Memory address, number of sectors to be transferred.

   If the desired disk drive and controller are available, the request can be serviced immediately. If the drive or controller is busy, any new requests for service will be placed in the queue of pending requests for that drive.

   When one request is completed, the operating system chooses which pending request to service next. How does the operating system make this choice? Any one of several disk-scheduling algorithms can be used.

Knowledge Gate

Knowledge Gate Website

FCFS (First Come First Serve)

The simplest form of disk scheduling is, of course, the first-come, first-served (FCFS) algorithm. In FCFS, the requests are addressed in the order they arrive in the disk queue. This algorithm is intrinsically fair, but it generally does not provide the fastest service.

Advantages:

- Easy to understand easy to use
- Every request gets a fair chance
- No starvation (may suffer from convoy effect)

Disadvantages:

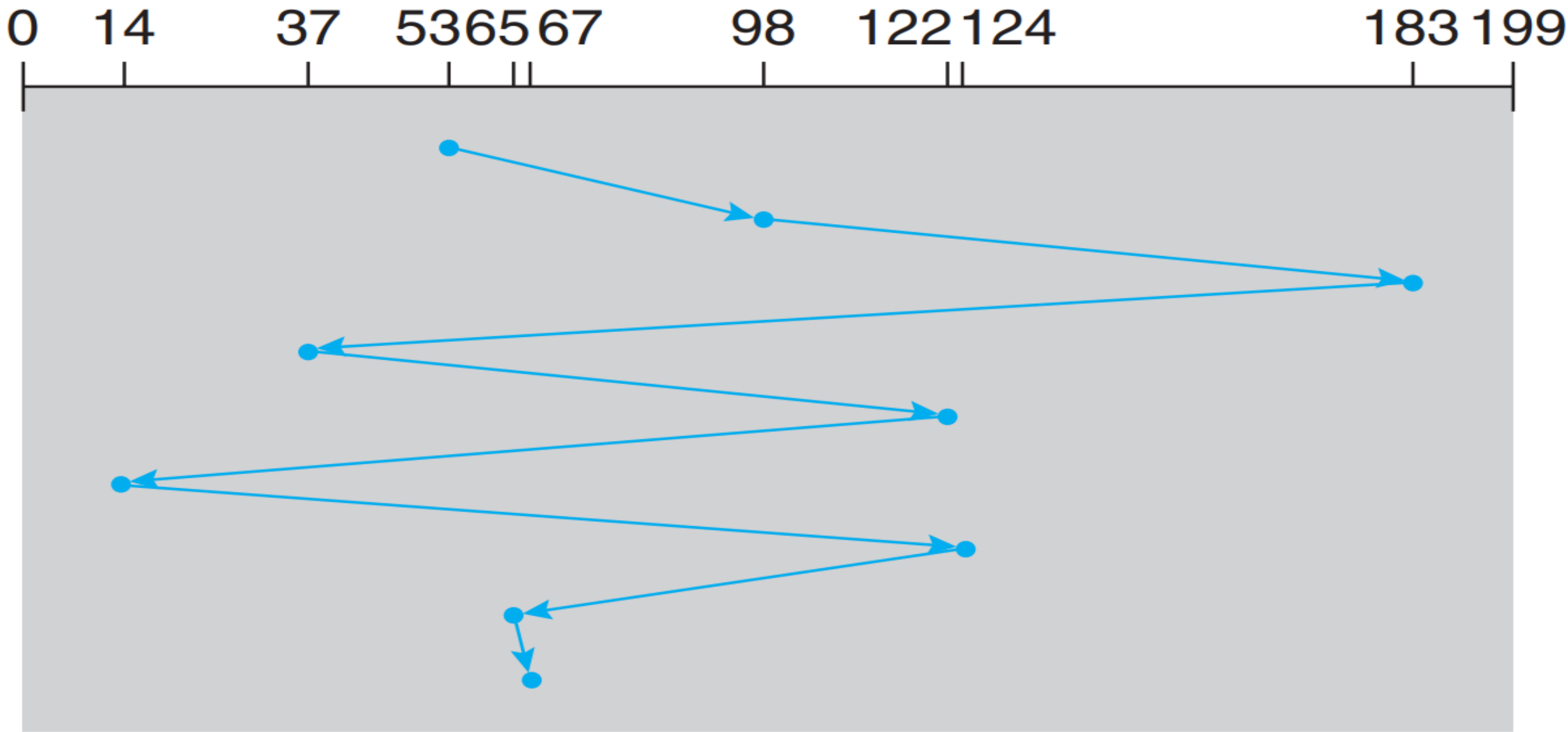
- Does not try to optimize seek time, or waiting time.

Knowledge Gate

Knowledge Gate Website

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



SSTF(Shortest Seek Time First) Scheduling

Major component in total transfer time is seek time, in order to reduce seek time if we service all the requests close to the current head position, this idea is the basis for the SSTF algorithm. In SSTF, the request nearest to the disk arm will get executed first i.e. requests having shortest seek time are executed first. Although the SSTF algorithm is a substantial improvement over the FCFS algorithm, it is not optimal.

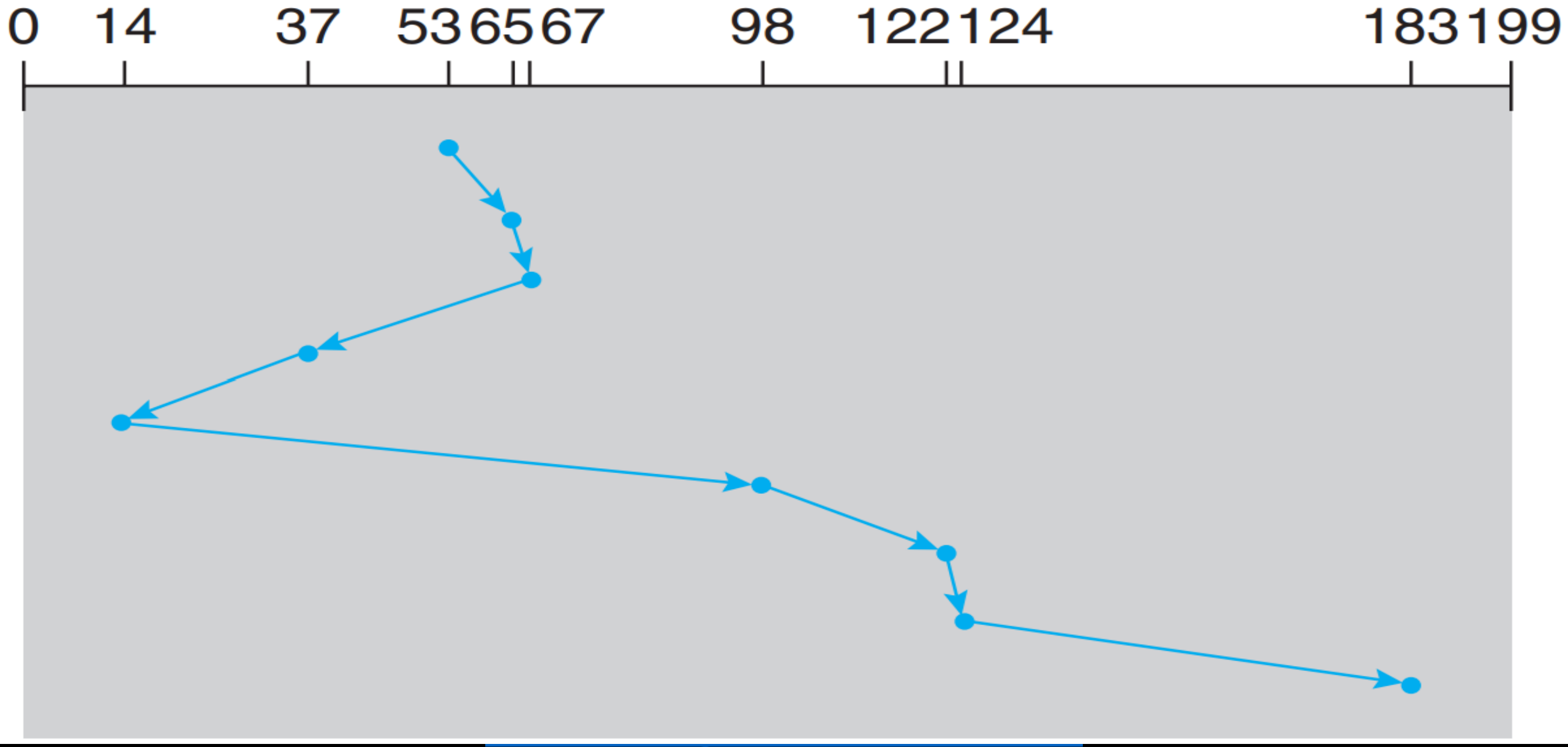
- **Advantages:**
  Seek movements decreases
  Throughput increases
-
- **Disadvantages:**
  Overhead to calculate the closest request.
  Can cause Starvation for a request which is far from the current location of the header
  High variance of response time and waiting time as SSTF favors only closest requests

Knowledge Gate

Knowledge Gate Website

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



SCAN/ Elevator Algorithm

- The disk arm starts at one end of the disk and moves towards the other end, servicing requests as it reaches each track, until it gets to the other end of the disk.

- At the other end, the direction of head movement is reversed, and servicing continues. The head continuously scans back and forth across the disk.

Advantages:

Simple easy to understand and use
No starvation but more wait for some random process
Low variance and Average response time

Disadvantages:

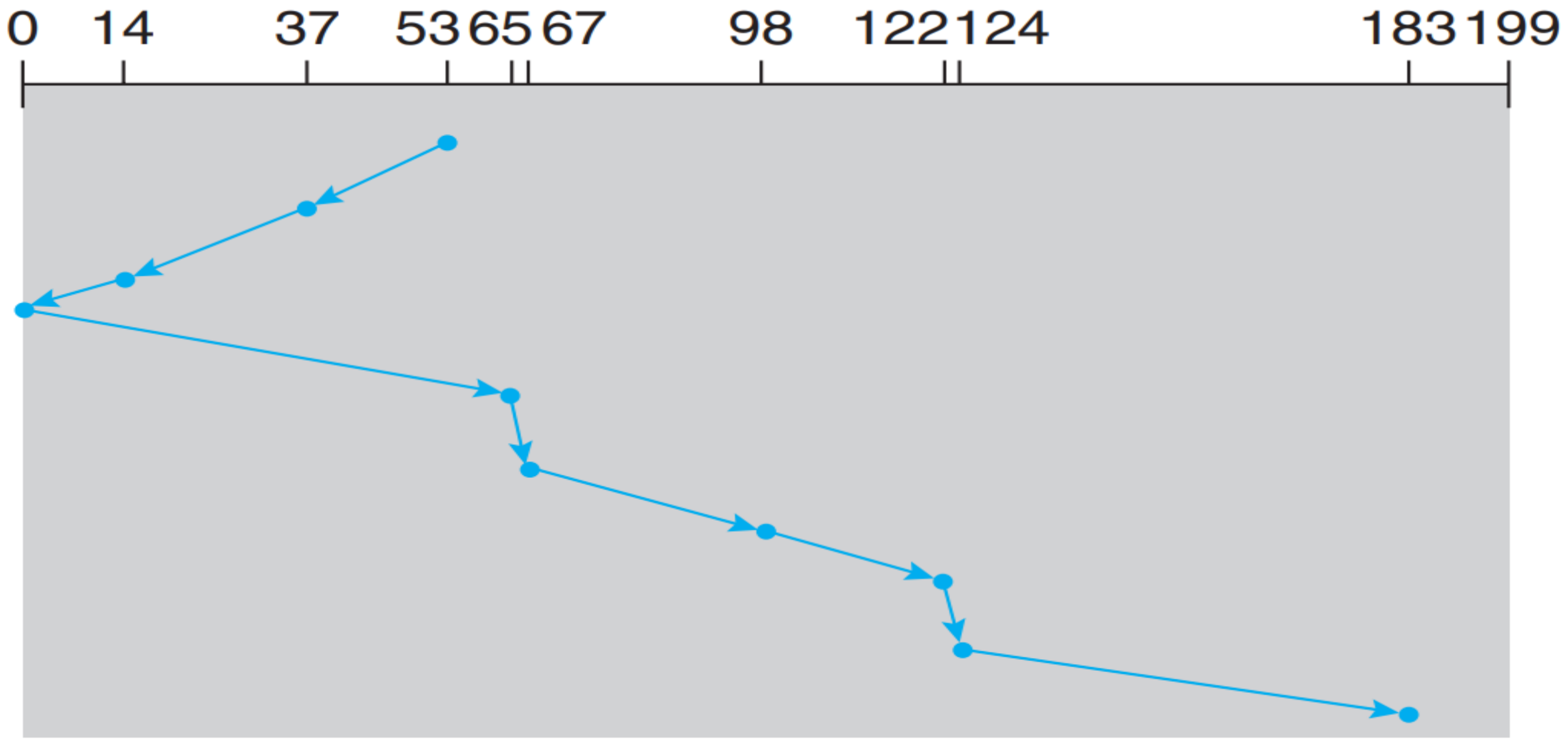
Long waiting time for requests for locations just visited by disk arm.
Unnecessary move to the end of the disk, even if there is no request.

Knowledge Gate

Knowledge Gate Website

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



C-SCAN Scheduling

- When the disk head reaches one end and changes direction, fewer requests are nearby since those cylinders were just serviced. Most pending requests are at the opposite end, having waited the longest.
- Circular-scan is a variant of SCAN designed to provide a more uniform wait time. Like SCAN, C-SCAN moves the head from one end of the disk to the other, servicing requests along the way. When the head reaches the other end, however, it immediately returns to the beginning of the disk without servicing any requests on the return trip.

Advantages:

Provides more uniform wait time compared to SCAN
Better response time compared to scan

Disadvantage:

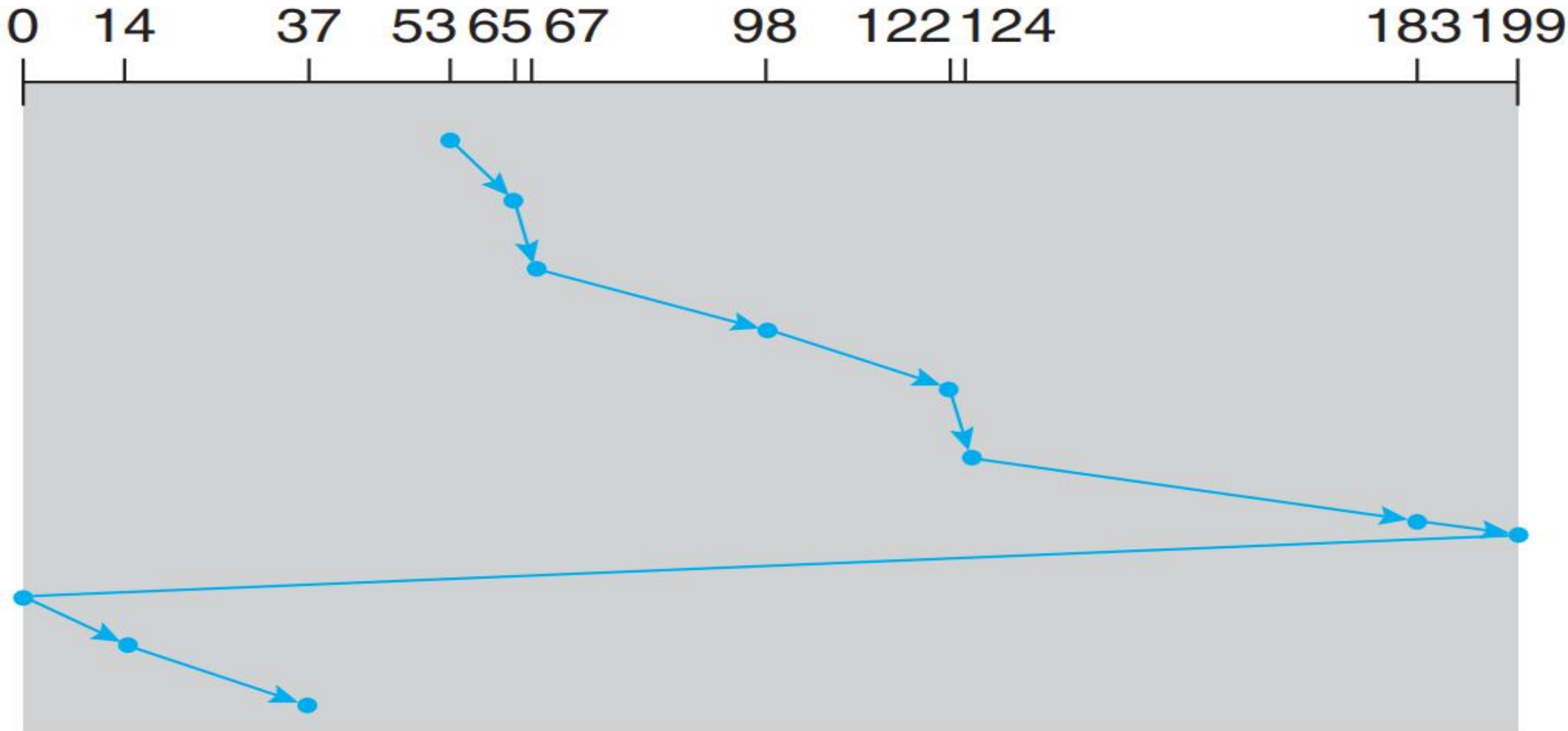
More seeks movements in order to reach starting position

Knowledge Gate

Knowledge Gate Website

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



LOOK Scheduling

- It is similar to the SCAN disk scheduling algorithm except the difference that the disk arm inspite of going to the end of the disk goes only to the last request to be serviced in front of the head and then reverses its direction from there only. Thus, it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

Advantage: -

Better performance compared to SCAN
Should be used in case to less load

Disadvantage: -

Overhead to find the last request
Should not be used in case of more load.

Knowledge Gate

Knowledge Gate Website

C LOOK

- As LOOK is similar to SCAN algorithm, in similar way, C-LOOK is similar to C-SCAN disk scheduling algorithm. In C-LOOK, the disk arm in spite of going to the end goes only to the last request to be serviced in front of the head and then from there goes to the other end's last request. Thus, it also prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

Advantage: -

Provides more uniform wait time compared to LOOK

Better response time compared to LOOK

Disadvantage: -

Overhead to find the last request and go to initial position is more

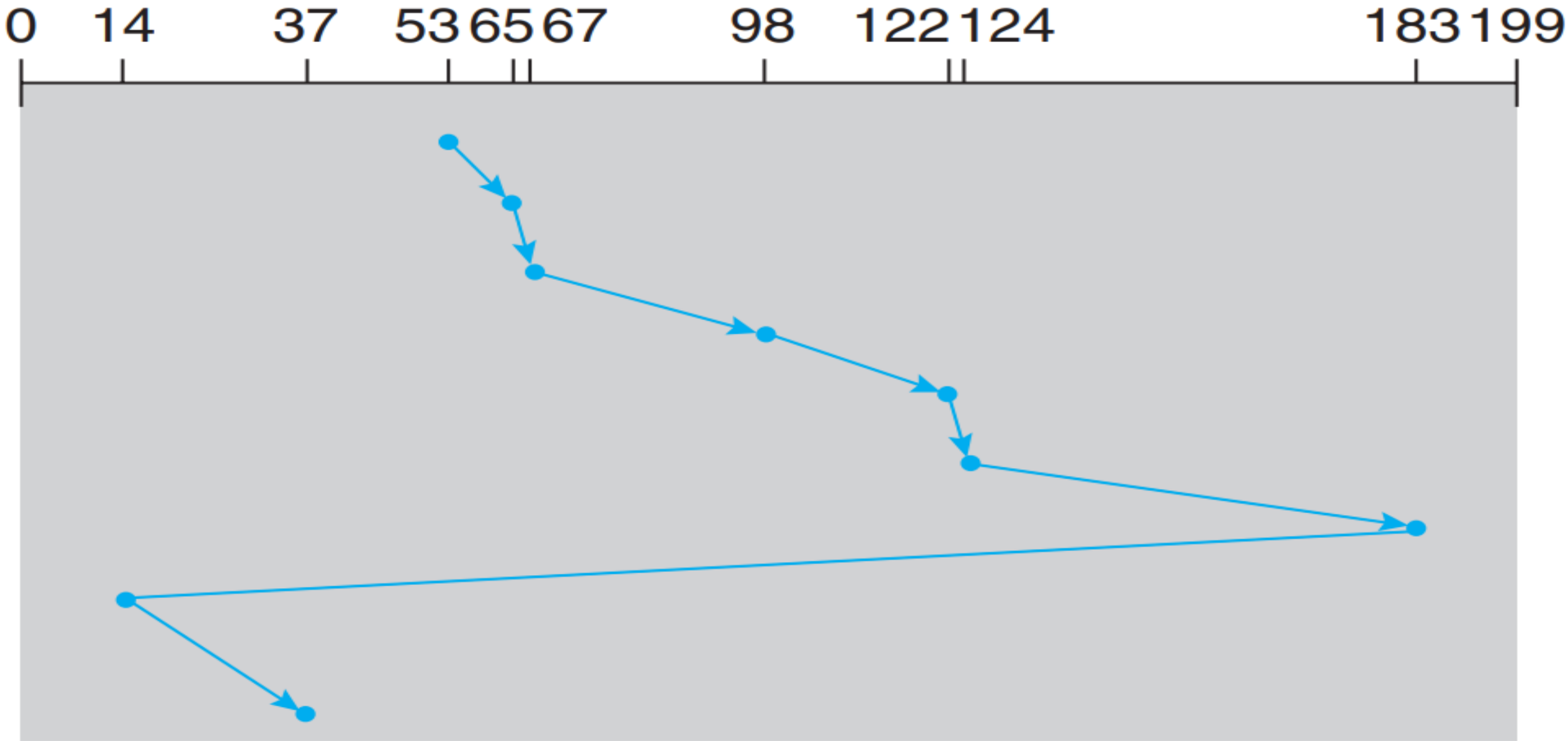
Should not be used in case of more load.

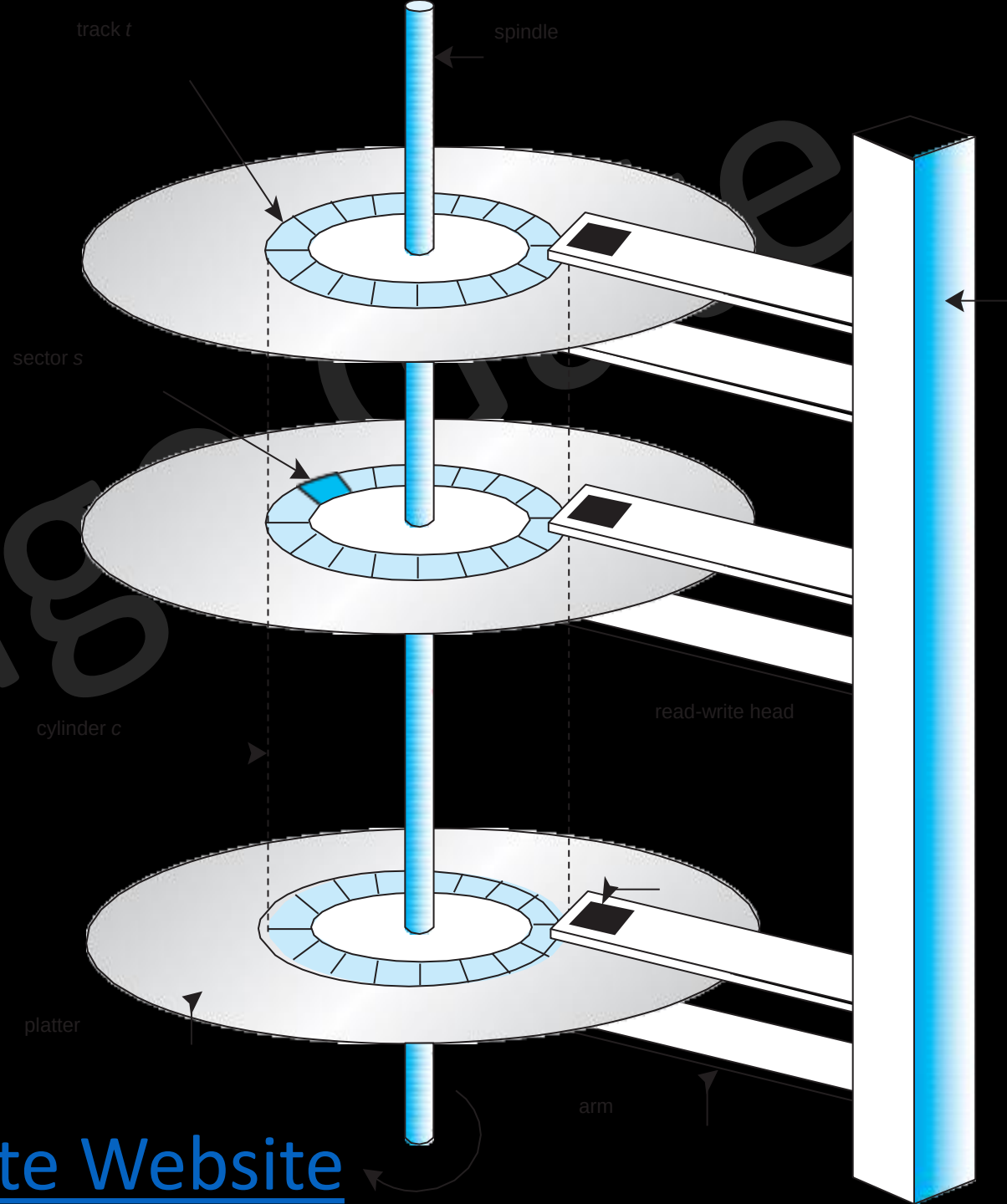
Knowledge Gate

Knowledge Gate Website

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53





Knowledge Gate Website

Total Transfer Time = Seek Time + Rotational Latency + Transfer Time

Seek Time: - It is a time taken by Read/Write header to reach the correct track. (Always given in question)

Rotational Latency: - It is the time taken by read/Write header during the wait for the correct sector. In general, it's a random value, so far average analysis, we consider the time taken by disk to complete half rotation.

Transfer Time: - it is the time taken by read/write header either to read or write on a disk. In general, we assume that in 1 complete rotation, header can read/write the either track, so

total time will be = (File Size/Track Size) *time taken to complete one revolution.

Knowledge Gate

Knowledge Gate Website

Q Consider a disk where there are 512 tracks, each track is capable of holding 128 sector and each sector holds 256 bytes, find the capacity of the track and disk and number of bits required to reach correct track, sector and disk.

Knowledge Gate

Knowledge Gate Website

Q consider a disk where each sector contains 512 bytes and there are 400 sectors per track and 1000 tracks on the disk. If disk is rotating at speed of 1500 RPM, find the total time required to transfer file of size 1 MB. Suppose seek time is 4ms?

Knowledge Gate

Knowledge Gate Website

Q Consider a system with 8 sector per track and 512 bytes per sector. Assume that disk rotates at 3000 rpm and average seek time is 15ms standard. Find total time required to transfer a file which requires 8 sectors to be stored.

a) Assume contiguous allocation

b) Assume Non- contiguous allocation

Knowledge Gate

Knowledge Gate Website

File allocation methods

The main aim of file allocation problem is how disk space is utilized effectively and files can be accessed quickly. Three major methods of allocating disk space are in wide use:

Contiguous
Linked
Indexed

Each method has advantages and disadvantages. Although some systems support all three, it is more common for a system to use one method for all files.

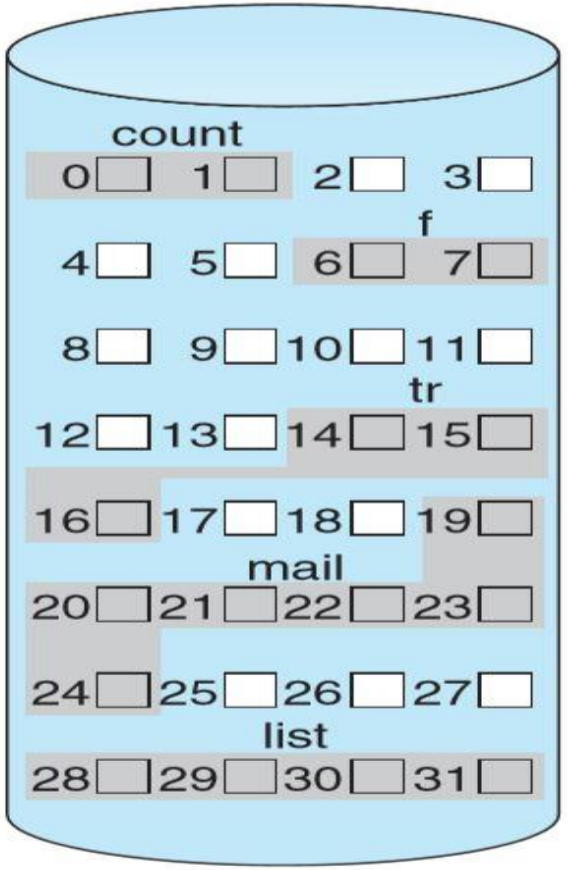
Knowledge Gate

Knowledge Gate Website

Contiguous Allocation

Contiguous allocation requires that each file occupy a set of contiguous blocks on the disk.

In directory usually we store three column file name, start dba and length of file in number of blocks.



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Advantage

Accessing a file that has been allocated contiguously is easy. Thus, both sequential and direct access can be supported by contiguous allocation.

Disadvantage

Suffer from huge amount of external fragmentation.

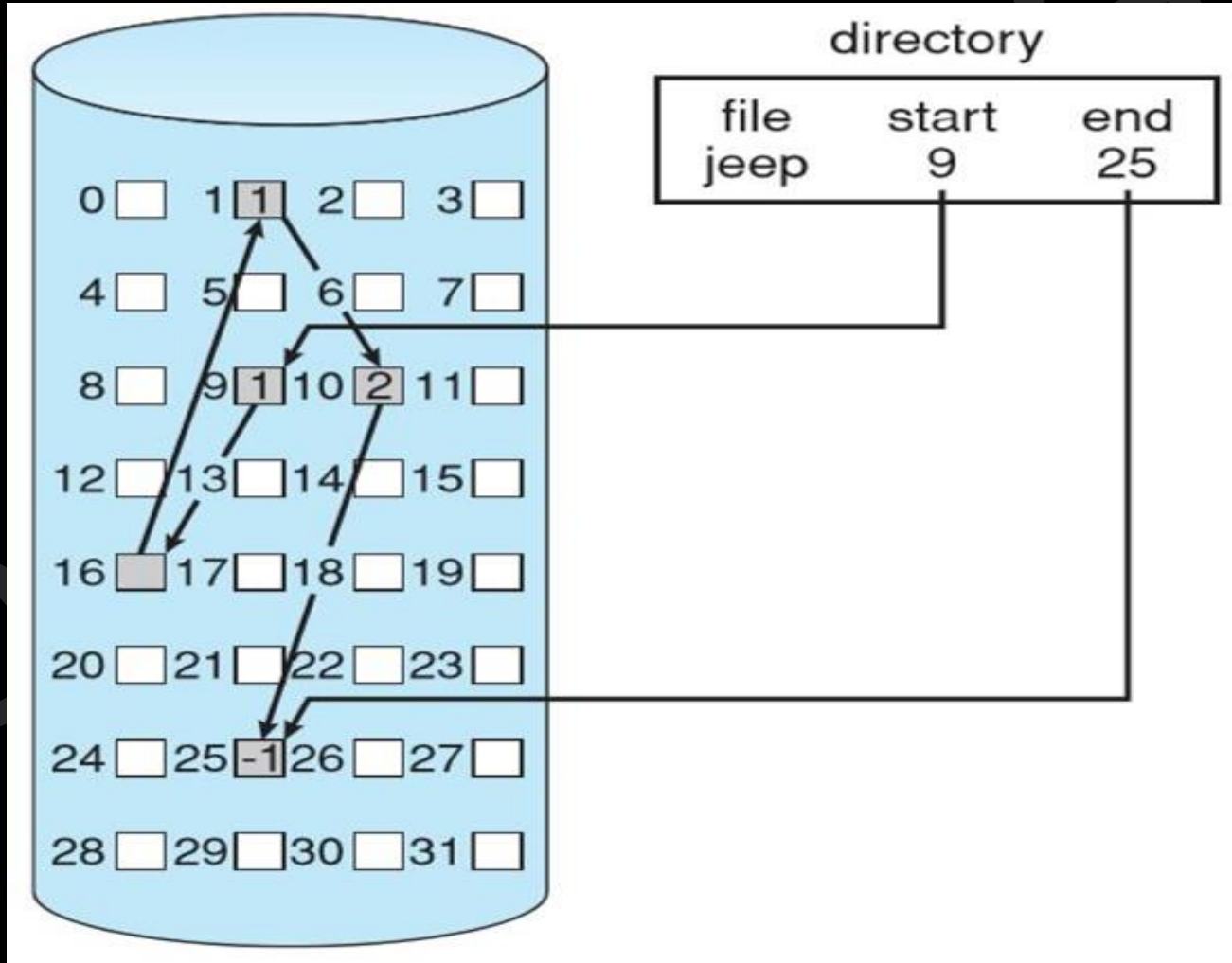
Another problem with contiguous allocation is file modification

Knowledge Gate

Knowledge Gate Website

Linked Allocation

With linked allocation, each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks of the file.



Advantage: -

To create, read, write a new file is simply easy. The size of a file need not be declared when the file is created. A file can continue to grow as long as free blocks are available.

There is no external fragmentation with linked allocation, and any free block on the free-space list can be used to satisfy a request.

Disadvantage: -

Only sequential access is possible, To find the i^{th} block of a file, we must start at the beginning and follow the pointers until we get to the i^{th} block.

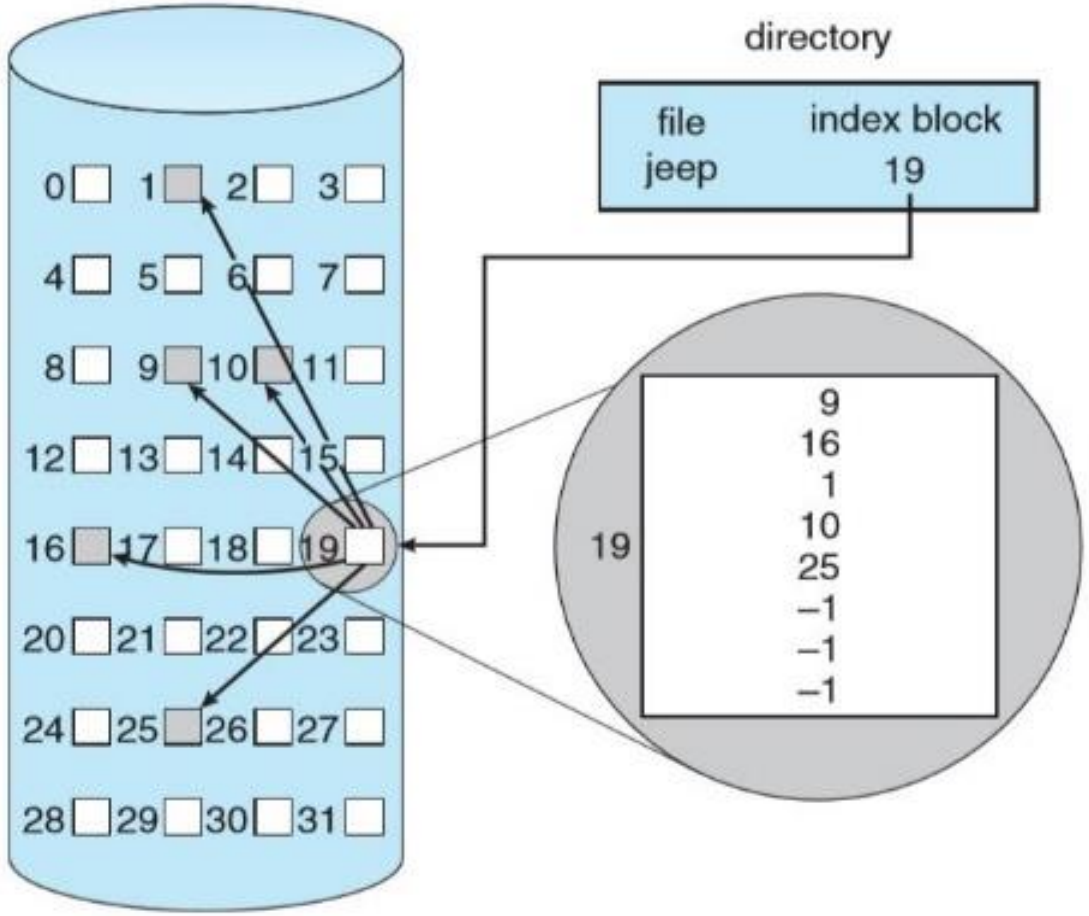
Another disadvantage is the space required for the pointers, so each file requires slightly more space than it would otherwise.

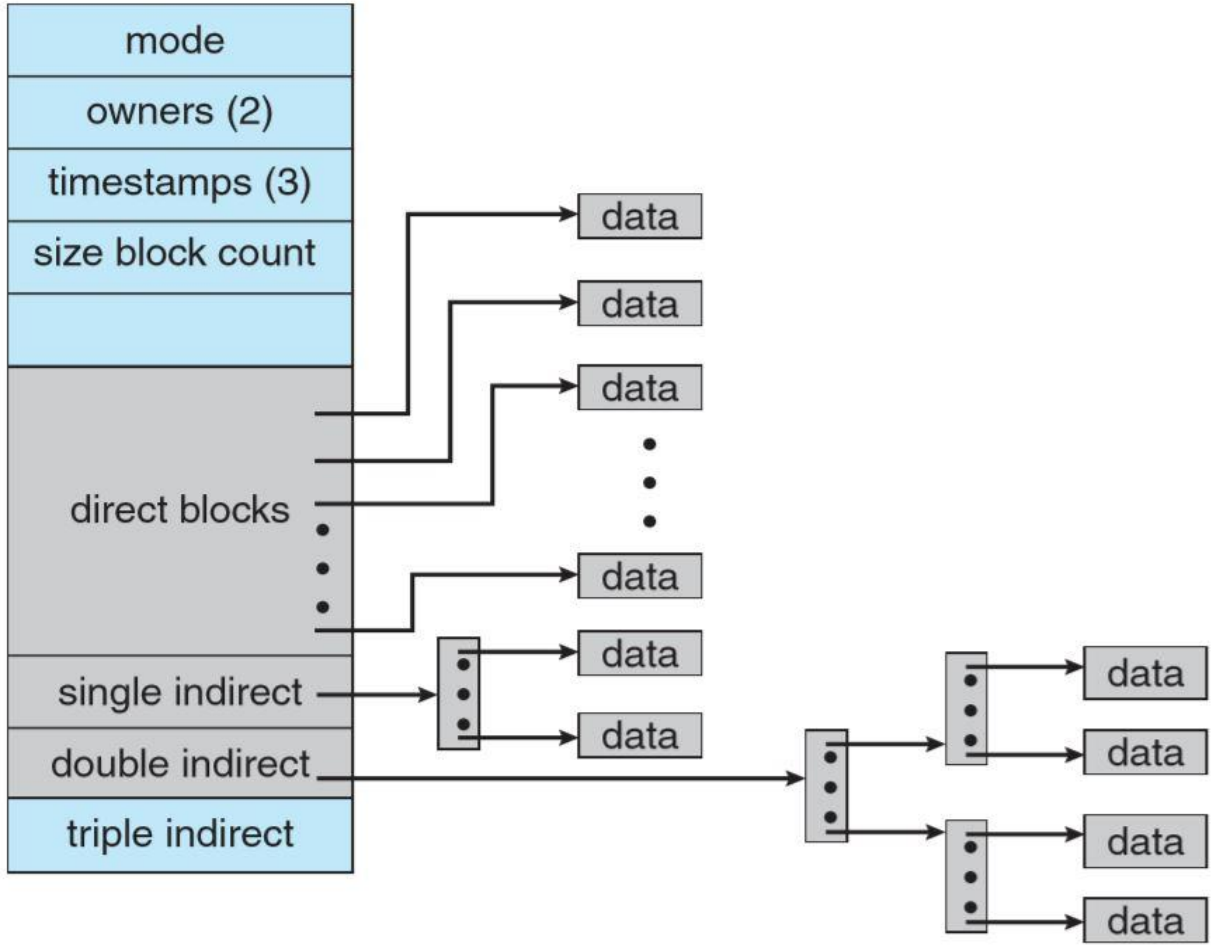
Knowledge Gate

Knowledge Gate Website

Indexed Allocation

Indexed allocation solves problems of contiguous and linked allocation, by bringing all the pointers together into one location: the index block.





Knowledge Gate Website

Each file has an index block containing an array of disk-block addresses. The directory entry points to this index block.

When a file is created, all index block pointers are null. Writing to the i^{th} block updates the corresponding index-block entry with a block address from the free-space manager.

The size of the index block is a trade-off: it should be small to save space but large enough to accommodate pointers for big files.

Knowledge Gate

Knowledge Gate Website

Linked scheme: To allow for large files, we can link together several index blocks. For example, an index block might contain a small header giving the name of the file and a set of the first 100 disk-block addresses. The next address (the last word in the index block) is null (for a small file) or is a pointer to another index block (for a large file).

Knowledge Gate

Knowledge Gate Website

Multilevel index. A variant of linked representation uses a first-level index block to point to a set of second-level index blocks, which in turn point to the file blocks.

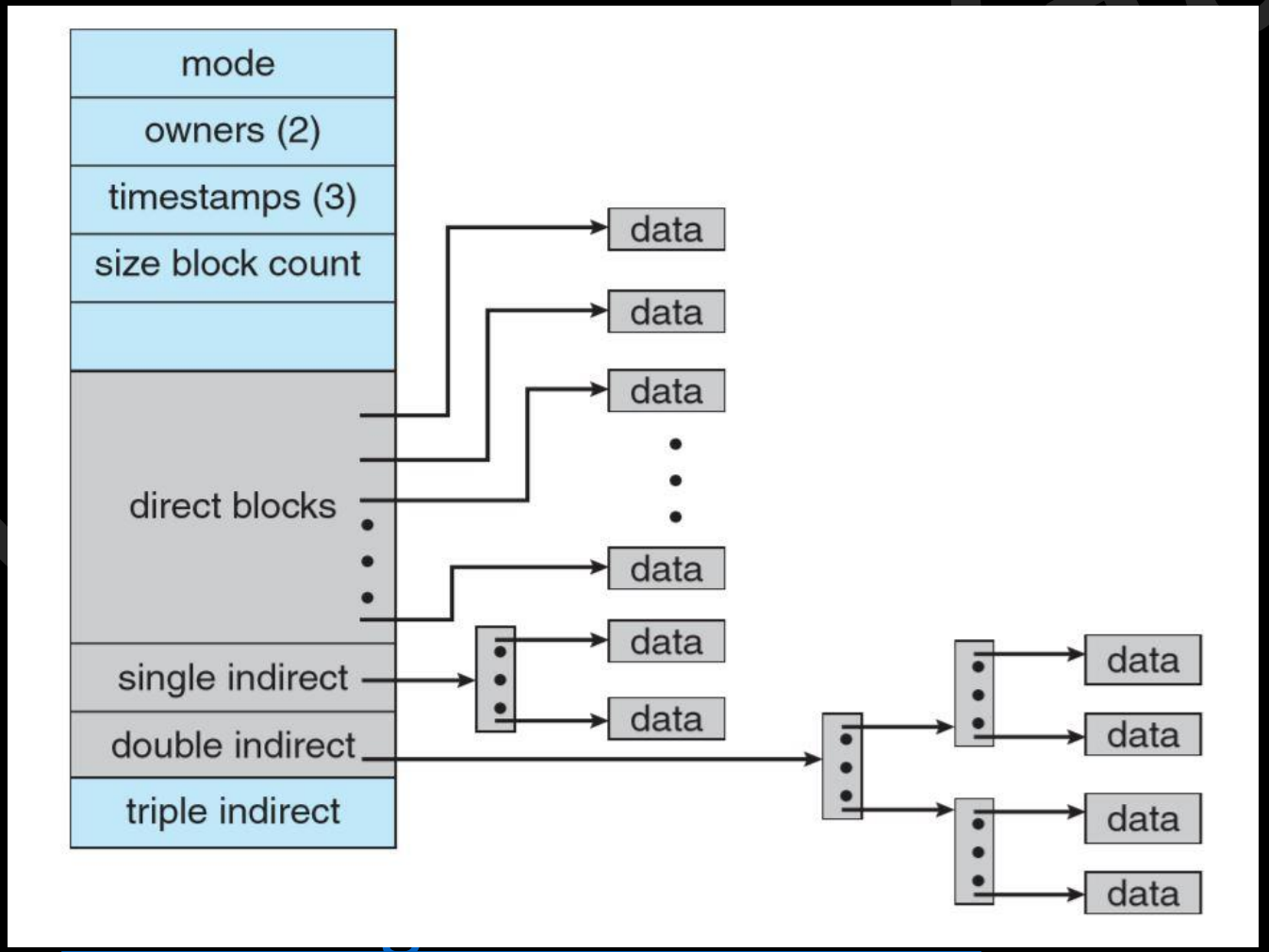
To access a block, the operating system uses the first-level index to find a second-level index block and then uses that block to find the desired data block. This approach could be continued to a third or fourth level, depending on the desired maximum file size.

Knowledge Gate

Knowledge Gate Website

Combined scheme: In UNIX-based systems, the file's Inode stores the first 15 pointers from the index block. The first 12 point directly to data blocks, eliminating the need for a separate index block for small files.

The next three pointers are for indirect blocks: the first for a single indirect block, the second for a double indirect block, and the last for a triple indirect block, each increasingly indirecting to the actual data blocks.



Advantage

Indexed allocation supports direct access, without suffering from external fragmentation, because any free block on the disk can satisfy a request for more space.

Disadvantage

Indexed allocation does suffer from wasted space. The pointer overhead of the index block is generally greater than the pointer overhead of linked allocation.

Knowledge Gate

Knowledge Gate Website

Break

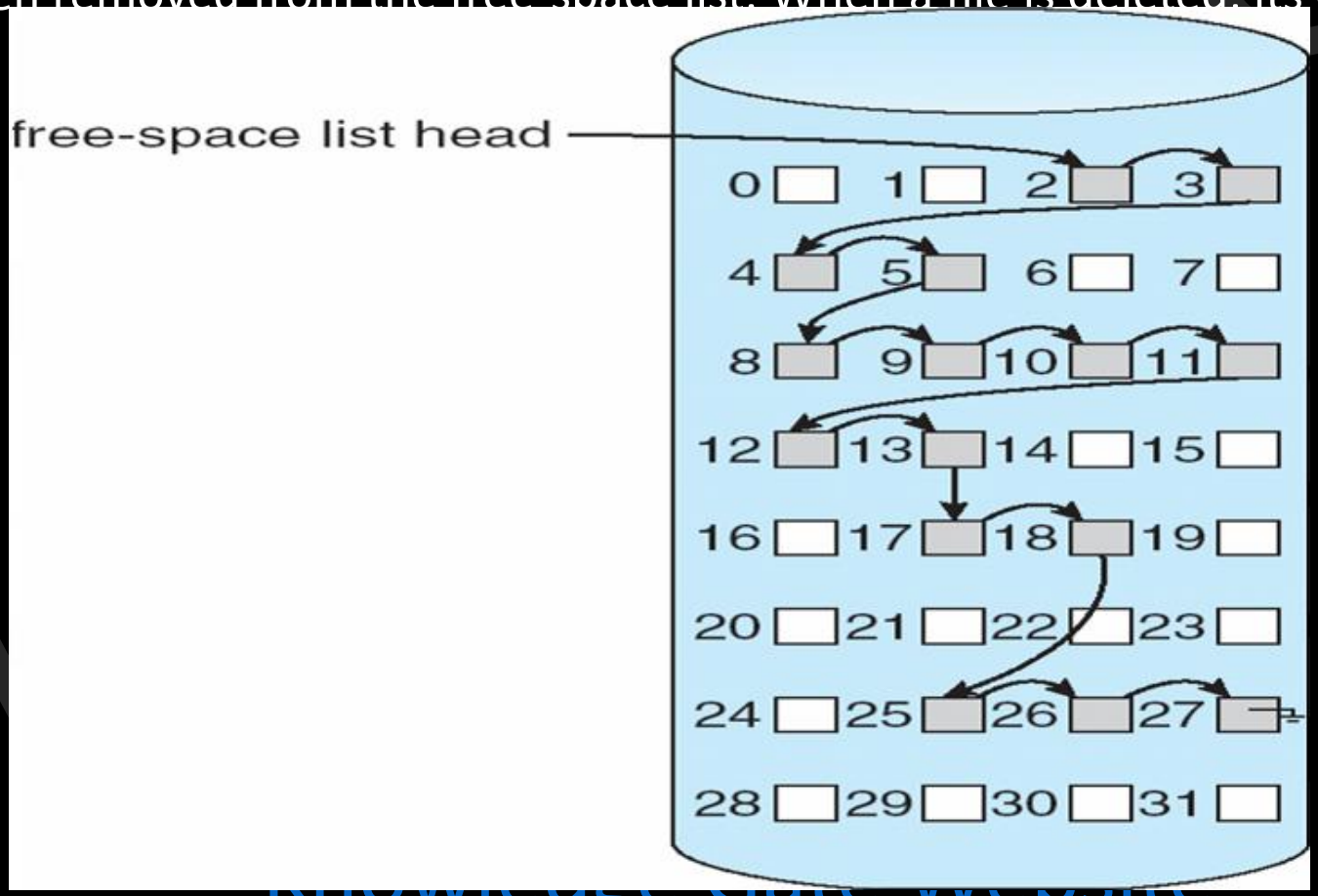
Knowledge Gate

Knowledge Gate Website

Free-Space Management

Since disk space is limited, we need to reuse the space from deleted files for new files, if possible. To keep track of free disk space, the system maintains a free-space list. The free-space list records all free disk blocks—those not allocated to some file or directory.

To create a file, we search the free-space list for the required amount of space and allocate that space to the new file. This space is then removed from the free-space list. When a file is deleted, its disk space is added to the free-space list.



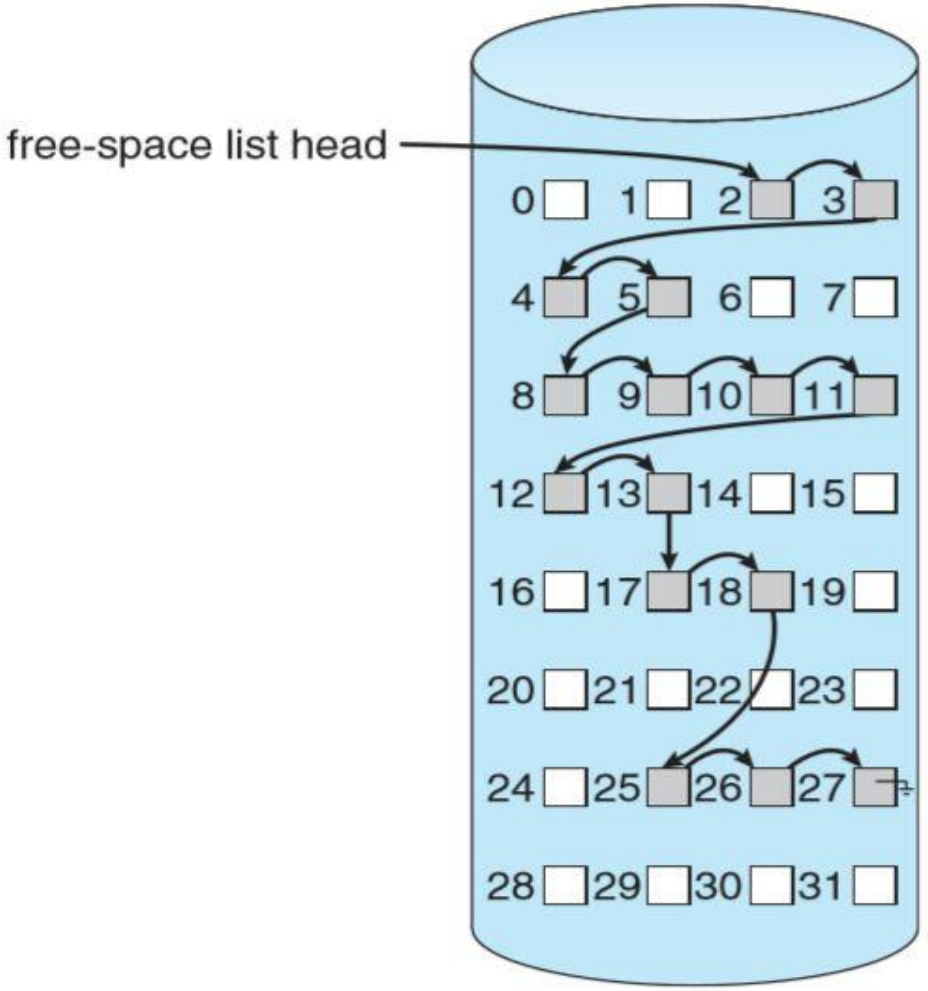
Knowledge Gate Website

Linked List

A approach to free-space management is to link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory.

This first block contains a pointer to the next free disk block, and so on.

This scheme is not efficient; to traverse the list, we must read each block, which requires substantial I/O time. However, operating system simply needs a free block so that it can allocate that block to a file, so the first block in the free list is used.



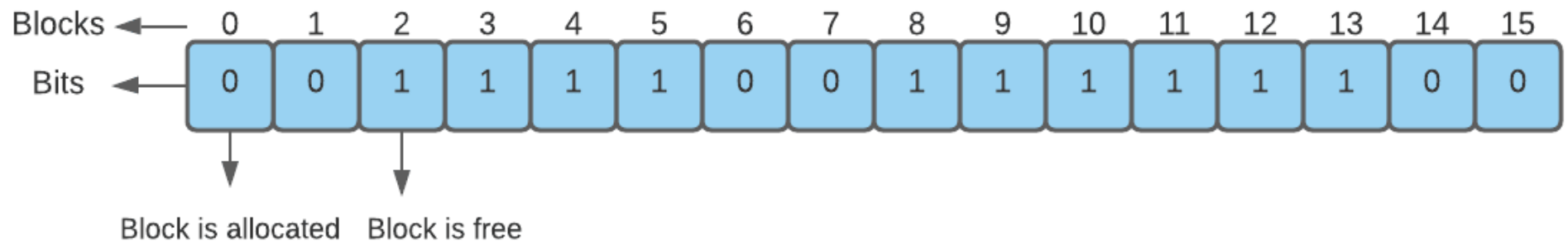
Knowledge Gate Website

Bit Vector

Frequently, the free-space list is implemented as a bit map or bit vector. Each block is represented by 1 bit. If the block is free, the bit is 1; if the block is allocated, the bit is 0.

For example, consider a disk where blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 are free and the rest of the blocks are allocated. The free-space bit map would be 00111100111111000110000011100000 ...

The main advantage of this approach is its relative simplicity and its efficiency in finding the first free block or n consecutive free blocks on the disk.

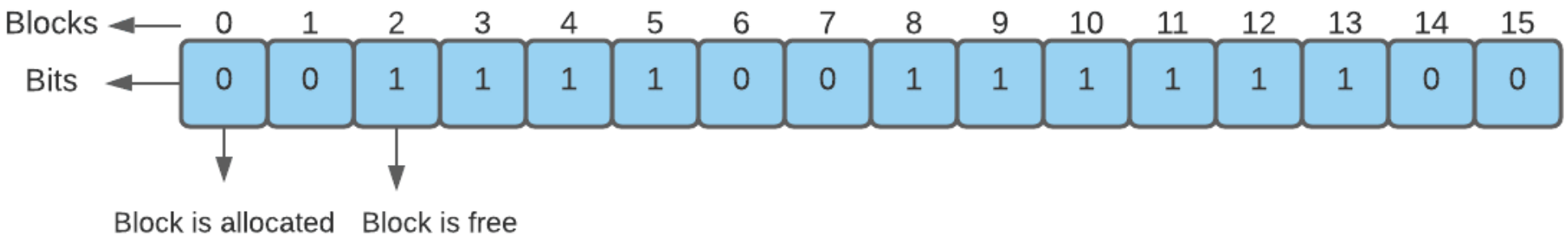


Knowledge Gate Website

Unfortunately, bit vectors are inefficient unless the entire vector is kept in main memory. Keeping it in main memory is possible for smaller disks but not necessarily for larger ones.

A 1.3-GB disk with 512-byte blocks would need a bit map of over 332 KB to track its free blocks.

A 1-TB disk with 4-KB blocks requires 256 MB to store its bit map. Given that disk size constantly increases, the problem with bit vectors will continue to escalate as well.



Knowledge Gate Website

Break

Knowledge Gate

Knowledge Gate Website

File organization

- File organization refers to the way data is stored in a file. File organization is very important because it determines the methods of access, efficiency, flexibility and storage devices to use.
- Four methods of organizing files:
  - 1. **Sequential file organization**:
    - a. Records are stored and accessed in a particular sorted order using a key field.
    - b. Retrieval requires searching sequentially through the entire file record by record to the end.
  - 2. **Random or direct file organization**:
    - a. Records are stored randomly but accessed directly.
    - b. To access a file which is stored randomly, a record key is used to determine where a record is stored on the storage media.
    - c. Magnetic and optical disks allow data to be stored and accessed randomly.

Knowledge Gate

Knowledge Gate Website

- 3. **Serial file organization**:
  - a. Records in a file are stored and accessed one after another.
  - b. This type of organization is mainly used on magnetic tapes.
- 4. **Indexed-sequential file organization method**:
  - Almost similar to sequential method only that, an index is used to enable the computer to locate individual records on the stage media
  - For example, on a magnetic drum, records are stored sequentially on the tracks. However, each record is assigned an index that can be used to access it directly.

Knowledge Gate

Knowledge Gate Website

File access mechanism

- Sequential access:
  - it is the simplest access mechanism in which information is stored in a file are exist in an order such that one record is process after the other
  - for example editors and compilers usually access file in this manner. next line

- Direct Access:
  - It is an alternative method for accessing a file, which is based on the disk model of a file, since disk allow random access to any block or a record of a file
  - for this method, a file is viewed as a numbered sequence of blocks or records which are read/written in an arbitrary manner that is there is no restriction on the order of recording or writing
  - it is well suited for database management system.

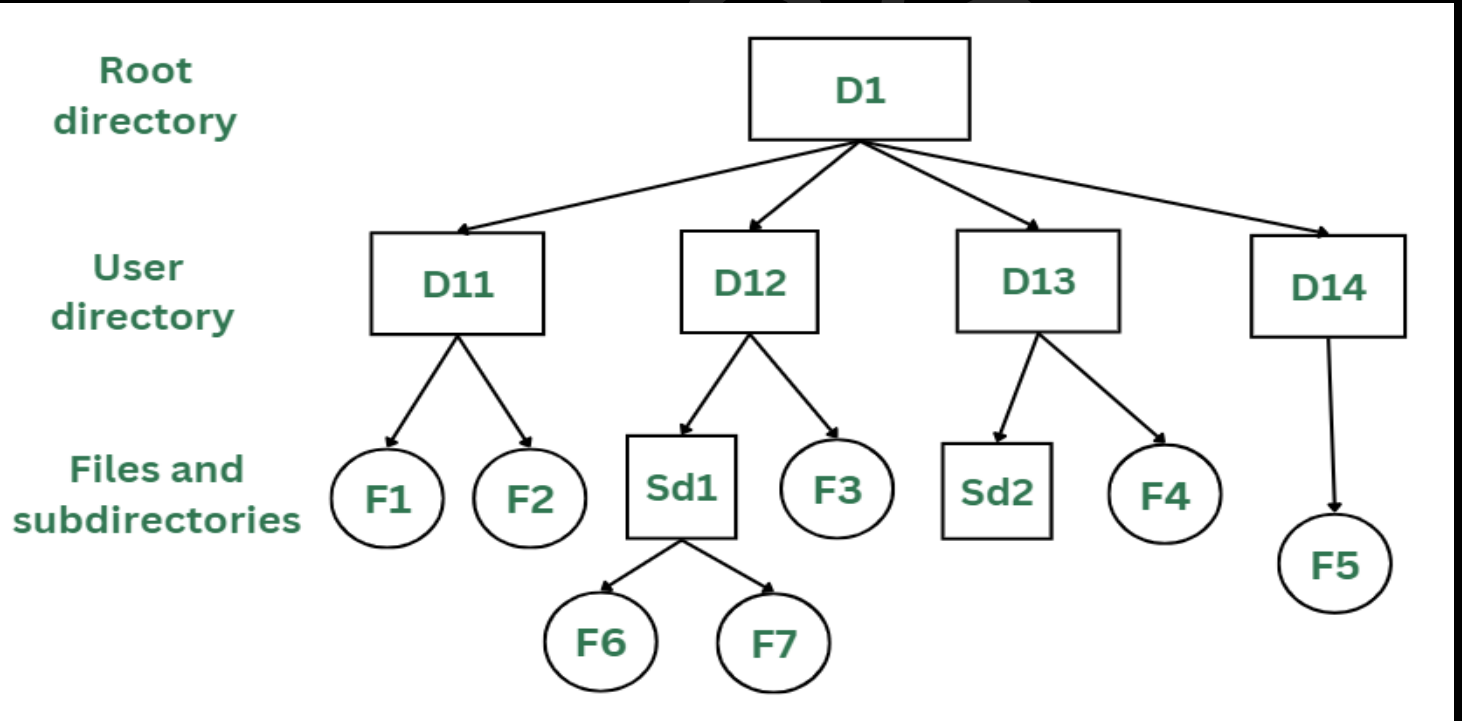
- Index access
  - In this method and alternate index is created which contain key field and a pointer to the various blocks.
  - To find and entry in the file for a key value we first search the index and then use the pointer to directly excess of file and find the desired entry

Knowledge Gate

Knowledge Gate Website

Directory

- A directory is similar to a "folder" in everyday terminology, and it exists within a file system.
- It's a virtual container where multiple files and other directories (often called subdirectories) can reside.
- It organizes the file system in a hierarchical manner, meaning directories can contain subdirectories, which may contain further subdirectories, and so on.



Knowledge Gate

Operations that Can Be Performed on a Directory

1. **Create Directory**: Make a new directory to store files and subdirectories.
2. **Delete Directory**: Remove an existing directory, usually only if it's empty.
3. **Rename Directory**: Change the name of a directory.
4. **List Contents**: View the files and subdirectories within a directory.
5. **Move Directory**: Relocate a directory to a different path in the file system.
6. **Copy Directory**: Make a duplicate of a directory, including its files and subdirectories.
7. **Change Directory**: Switch the working directory to a different one.
8. **Search Directory**: Find specific files or subdirectories based on certain criteria like name or file type.
9. **Sort Files**: Arrange the files in a directory by name, date, size, or other attributes.
10. **Set Permissions**: Change the access controls for a directory (read, write, execute).

Knowledge Gate

Knowledge Gate Website

Feature	Single-Level Directory	Two-Level Directory
User Isolation	No user-specific directories. All users share the same directory space.	Each user has their own private directory.
Organization	All files are stored in one directory, making it less organized.	Files can be organized under user-specific directories, allowing for better file management.
Search Efficiency	Can be less efficient as all files are in a single directory, requiring more time to find a specific file.	More efficient due to fewer files in each user-specific directory.
Access Control	Hard to implement user-specific access controls because all files reside in the same directory.	Easier to implement user-specific access controls, enhancing security.
Complexity	Simpler to implement but can become cluttered and difficult to manage with many files.	Slightly more complex due to the need for user management, but offers better organization.

Knowledge Gate
Knowledge Gate Website

Why It's Necessary

- **Organization**: It helps in sorting and locating files more efficiently.
- **User-Friendliness**: Directories make it easier for users to categorize their files by project, file type, or other attributes.
- **Access Control**: Using directories, different levels of access permission can be applied, providing an extra layer of security.

Features of Directories

- **Metadata**: Directories also store metadata about the files and subdirectories they contain, such as permissions, ownership, and timestamps.
- **Dynamic Nature**: As files are added or removed, the directory dynamically updates its list of contents.
- **Links and Shortcuts**: Some systems support the creation of pointers or links within directories to other files or directories.

Knowledge Gate

Knowledge Gate Website

Feature	Sequential File	Indexed File
Access Method	Records accessed one after another in order	Records can be accessed directly using an index
Speed of Access	Slower, especially for large files	Faster for random access, thanks to index
Storage Efficiency	Generally more efficient as no space is used for index	Less efficient due to storage needed for index
Update Complexity	Simpler, usually involves appending	More complex; updating index needed for record change
Use Case	Suitable for batch processing, backups	Suitable for databases, directories with quick lookup

Knowledge Gate

Knowledge Gate Website

File Protection System

- **Reliability**:
  - Reliability in a file protection system ensures that files are accessible and retrievable whenever needed, without loss of data. Techniques such as backup, mirroring, and RAID configurations contribute to high reliability.
- **Security**:
  - Security mechanisms protect files from unauthorized access, modification, or deletion. Encryption, firewall settings, and secure file transfer protocols like SFTP can be employed to safeguard files.
- **Controlled Access**:
  - Controlled access specifies who can do what with a file. Users are given permissions like read, write, and execute (r-w-x), often categorized into roles for easy management. Controlled access is crucial for maintaining the integrity and confidentiality of files.
- **Access Control**:
  - Access control mechanisms like Access Control Lists (ACLs) or Role-Based Access Control (RBAC) define rules specifying which users or system processes are granted access to files and directories. They can also specify the type of operations (read, write, execute) permitted.
- In summary, a robust file protection system is multi-layered, incorporating reliability measures, strong security protocols, and detailed access control mechanisms to ensure the safe and efficient management of files.

Knowledge Gate

Knowledge Gate Website

Access Matrix

- The Access Matrix is a conceptual framework used in computer security to describe the permissions that different subjects (such as users or processes) have when accessing different objects (such as files, directories, or resources).

- In this matrix, each row represents a subject and each column represents an object. The entry at the intersection of a row and column defines the type of access that the subject has to the object.

Knowledge Gate

Knowledge Gate Website

- **Matrix Form**: In its most straightforward representation, the Access Matrix is a table where the cell at the intersection of row *i* and column *j* contains the set of operations that subject *i* can perform on object *j*.

	File A	File B	File C
User 1	r-w	r	-
User 2	r	w	r-w
User 3	-	r	w

- Here, 'r' indicates read permission, 'w' indicates write permission, and '-' indicates no permission.

Knowledge Gate

Knowledge Gate Website

- **Access Control Lists (ACLs)**: Each object's column in the matrix can be converted to an Access Control List, which lists all subjects and their corresponding permissions for that object.

- File A:
  - User 1: r-w
  - User 2: r

- File B:
  - User 1: r
  - User 2: w
  - User 3: r

Knowledge Gate

Knowledge Gate Website

- **Capability Lists**: Each subject's row in the matrix can be converted into a Capability List, which lists all objects and the operations the subject can perform on them.

- User 1:
  - File A: r-w
  - File B: r

- User 2:
  - File A: r
  - File B: w
  - File C: r-w

Knowledge Gate

Knowledge Gate Website

- **Sparse Matrix**: In large systems, the Access Matrix is usually sparse. Special data structures can be used to represent only the non-empty cells to save space.

Knowledge Gate

Knowledge Gate Website

Implementation of Access Matrix

1. **Global Table**: A global table is essentially the raw access matrix itself, where each cell denotes the permissions a subject has on an object. While straightforward, this method is not practical for large systems due to the sparsity of the matrix and the associated storage overhead.

2. **Access Lists for Objects**: Here, the focus is on objects like files or directories. Each object maintains an Access Control List (ACL) that records what operations are permissible by which subjects. ACLs are object-centric and make it easy to determine all access rights to a particular object. However, this approach makes it cumbersome to list all capabilities of a particular subject across multiple objects.

3. **Capability Lists for Domains**: In this subject-centric approach, each subject or domain maintains a list of objects along with the operations it can perform on them, known as a Capability List. This makes it straightforward to manage and review the permissions granted to each subject. On the downside, revoking or changing permissions across all subjects for a specific object can be more challenging.

4. **Lock-Key Mechanism**: In a lock-key mechanism, each object is assigned a unique "lock," and subjects are granted "keys" to unlock these locks. When a subject attempts to access an object, the system matches the key with the lock to determine if the operation is permissible. This approach can be seen as an abstraction over the access matrix and can be used to dynamically change permissions with minimal overhead.

Knowledge Gate

Knowledge Gate Website

Module

Free Semester One Shot Exam Course

Topic

Computer Networks

Subtopic

CN One Shot

Lesson

Computer Network_Notes

Syllabus

- **UNIT-1 : Introduction Concepts:** Goals and Applications of Networks, Network structure and architecture, The OSI reference model, services, Network Topology Design - Delay Analysis, Back Bone Design, Local Access Network Design, Physical Layer Transmission Media, Switching methods, ISDN, Terminal Handling.

- **UNIT-2 : Medium Access sub layer:** Medium Access sub layer - Channel Allocations, LANprotocols - ALOHA protocols - Overview of IEEE standards - FDDI. Data Link Layer - Elementary Data Link Protocols, Sliding Window protocols, Error Handling.

- **UNIT-3 : Network Layer:** Network Layer - Point - to Pont Networks, routing, Congestion control Internetworking TCP / IP, IP packet, IP address, IPv6.

- **UNIT-4 : Transport Layer:** Transport Layer - Design issues, connection management, session Layer-Design issues, remote procedure call. Presentation Layer-Design issues, Data compression techniques, cryptography - TCP - Window Management.

- **UNIT-5 : Application Layer:** Application Layer: File Transfer, Access and Management, Electronic mail, Virtual Terminals, Other application. Example Networks - Internet and Public Networks.

KnowledgeGate

Video chapters

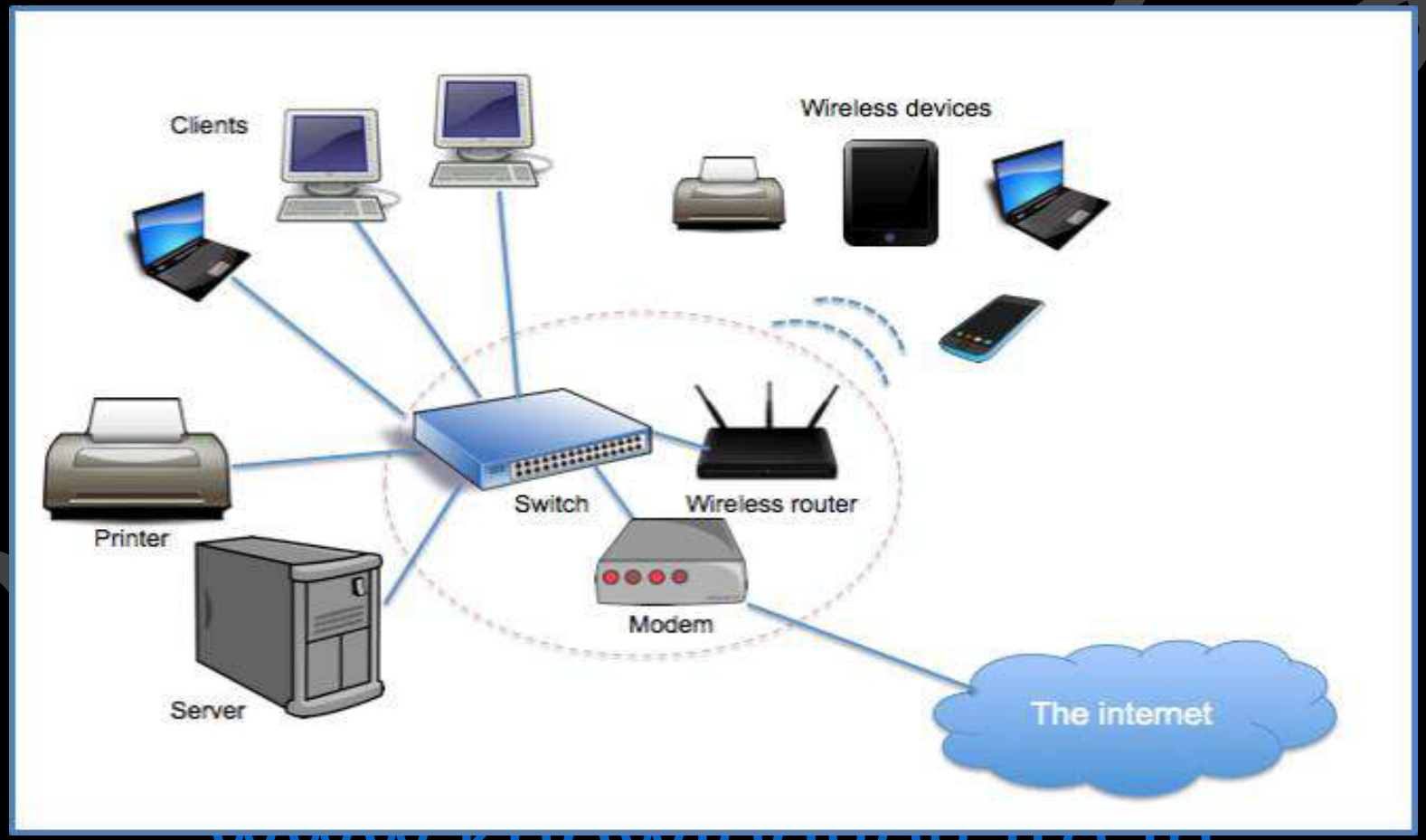
- **Chapter-1 (Basics):** What is Computer Networks, Goals, Application, Data Communication, Transmission Mode, Network Criteria, Connection Type, Topology, LAN, WAN, MAN, OSI Model, All Layer Duties, Transmission Media, Switching, ISDN.
- **Chapter-2 (Data Link Layer):** Random Access, ALOHA, Slotted ALOHA, CSMA, (CSMA/CD), (CSMA/CA), Sliding Window Protocol, Stop-and-Wait, Go-Back-N, Selective Repeat ARQ, Error Handling, Parity Check, Hamming Codes, CheckSum, CRC, Ethernet, Token Bus, Token Ring, FDDI, Manchester Encoding.
- **Chapter-3 (Network Layer):** Basics, IPv4 Header, IPv6 Header, ARP, RARP, ICMP, IGMP, IPv4 Addressing, Notations, Classful Addressing, Class A, Class B, Class C, Class D, Class E, Casting, Subnetting, Classless Addressing, Routing, Flooding, Intra-Domain Vs Inter-Domain, Distance Vector Routing, Two-Node Instability, Split Horizon, Link State Routing.
- **Chapter-4 (Transport Layer):** Basics, Port Number, Socket Addressing, TCP-Header, Three-way-Handshake, User Datagram Protocol, Data Compression, Cryptography, Symmetric Key, DES, Asymmetric Key, RSA Algorithm, Block-Transposition Cipher.
- **Chapter-5 (Application Layer):** E-Mail, SMTP, POP3/IMAP4, MIME, Web-Based Mail, FTP, WWW, Cookies, HTTP, DNS, Name Space, Telnet, ARPANET, X.25, SNMP, Voice over IP, RPC, Firewall, Repeater, Hub, Bridge, Switch, Router, Gateway.

KnowledgeGate

www.knowledgegate.in

What is Computer Networks

- A **computer network** is a telecommunications network, which allows autonomous digital devices(nodes) to exchange data between each other using either wired or wireless connections to share resources (h/w or s/w) interconnected by a single technology e.g. internet.



www.knowledgegate.in

Internet means empowerment

nowledgegate.in

Internet Of Things

Any Device

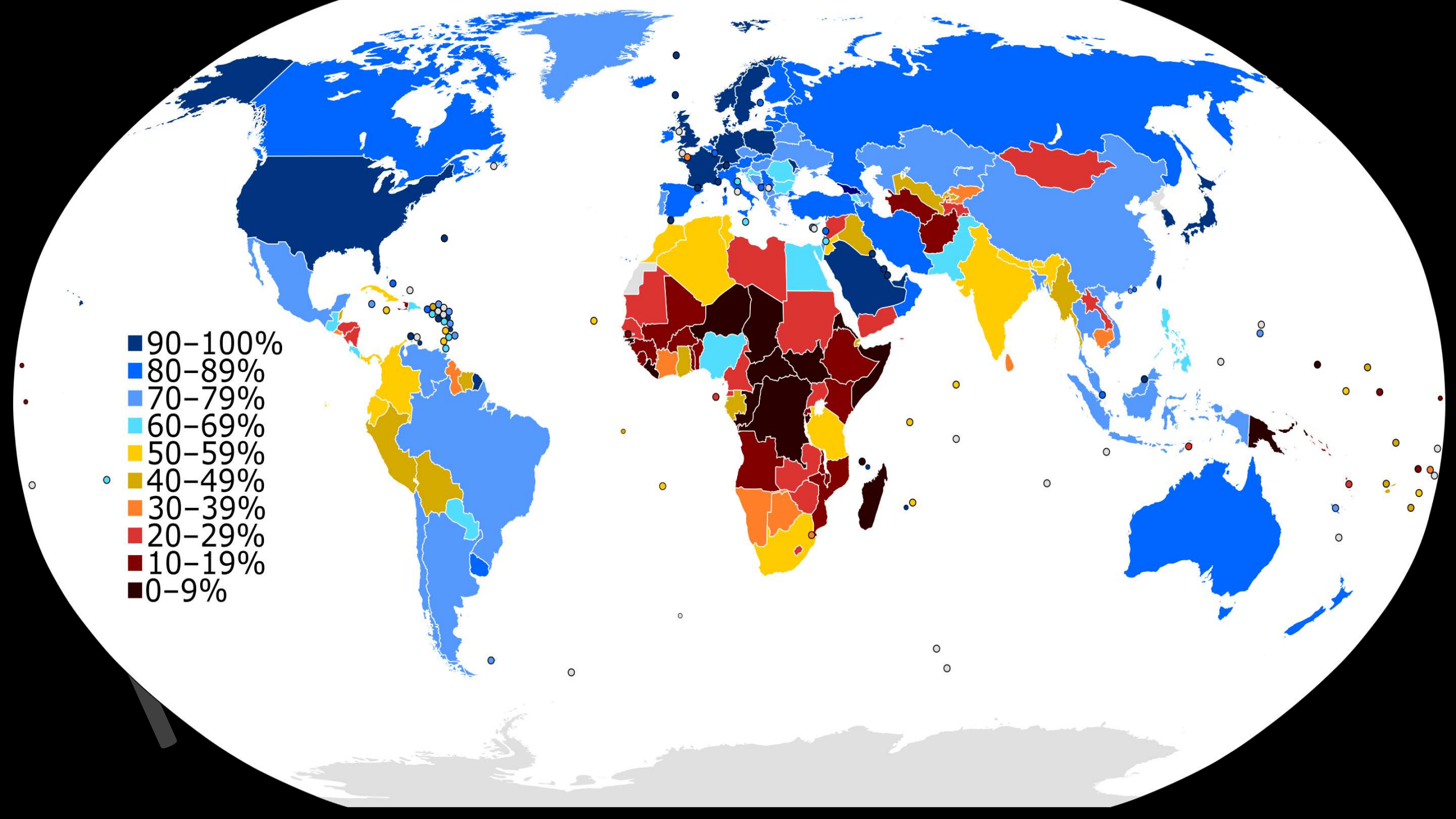
Anybody

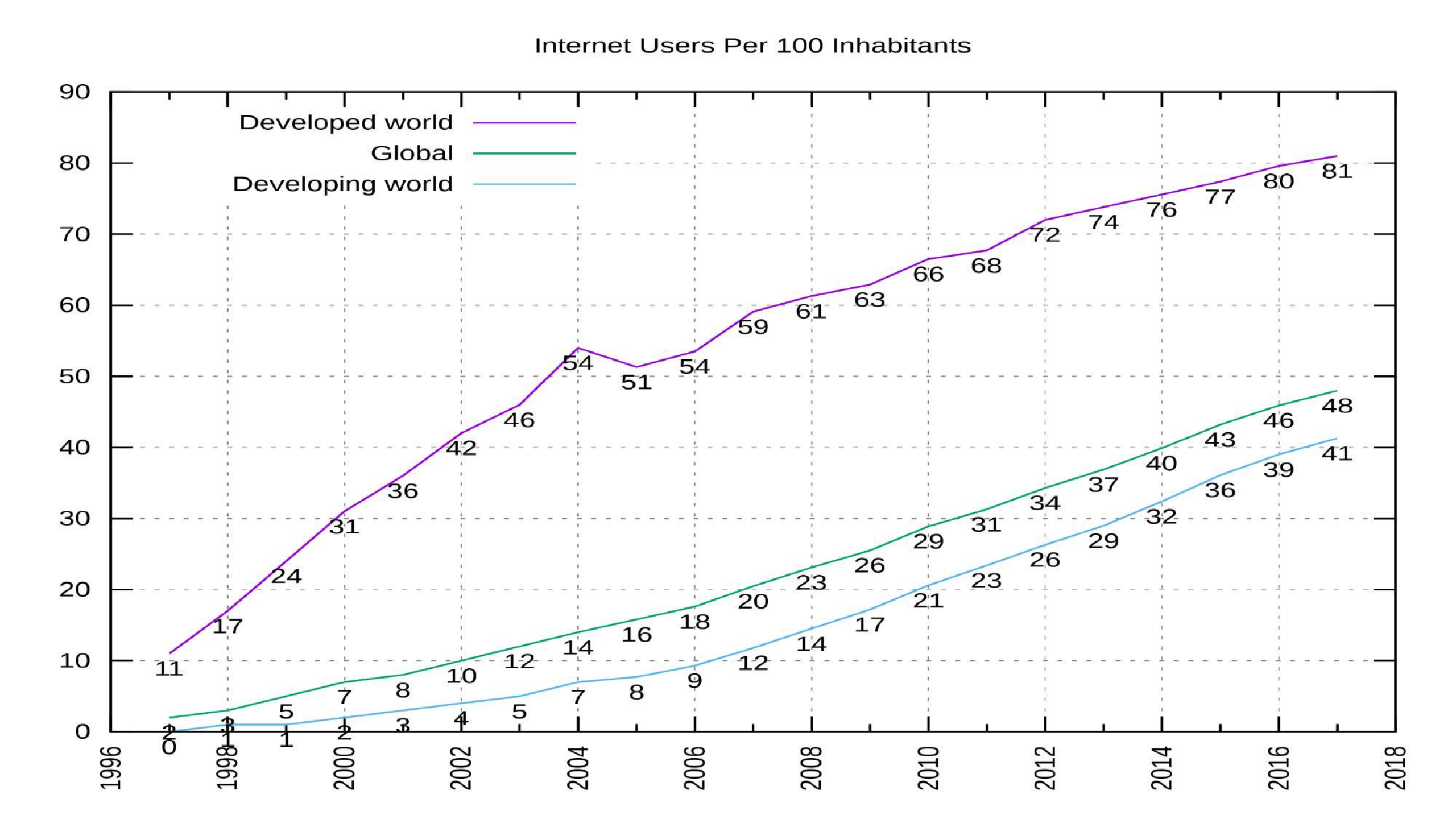
Anywhere

Any Business

Any Network

Anytime


90–100%
80–89%
70–79%
60–69%
50–59%
40–49%
30–39%
20–29%
10–19%
0–9%


Internet Users Per 100 Inhabitants
Developed world
Global
Developing world
90
80
70
60
50
40
30
20
10
0
1996
1998
2000
2002
2004
2006
2008
2010
2012
2014
2016
2018
11
17
24
31
36
42
46
54
51
54
59
61
63
66
68
72
74
76
77
80
81
2
3
5
7
8
10
12
14
16
18
20
23
26
29
31
34
37
40
43
46
48
0
1
1
2
3
4
5
7
8
9
12
14
17
21
23
26
29
32
36
39
41

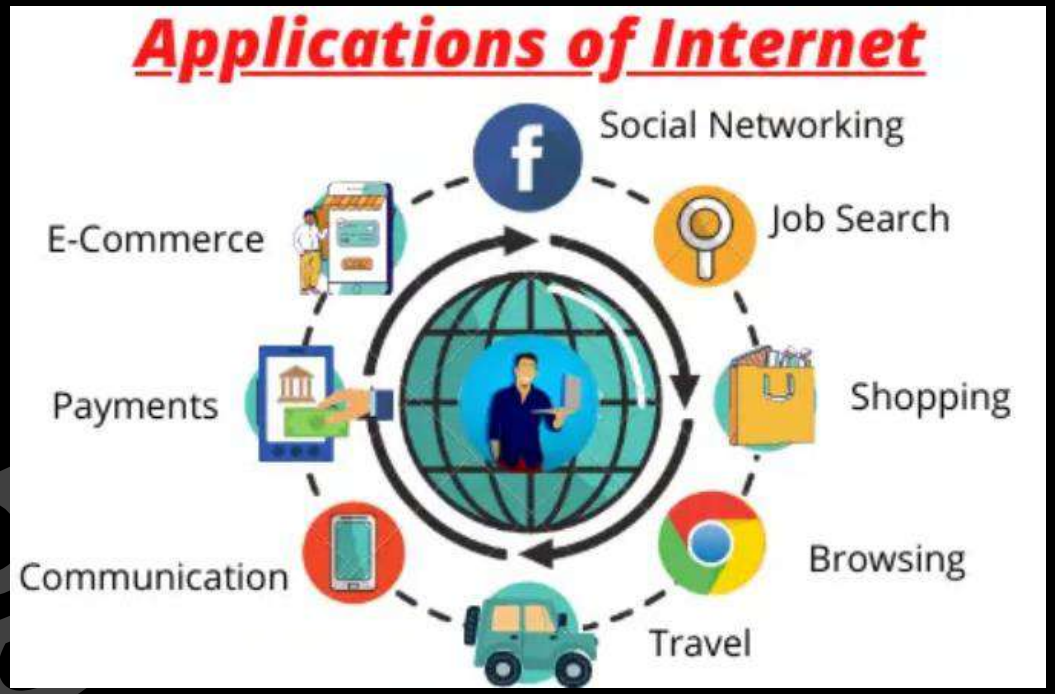
Goals of Computer Networks

- **Facilitating Communication**:
  - Enabling swift and efficient communication between individuals and organizations.
  - Supports video conferencing, emails, instant messaging, etc.
- **Resource Sharing**:
  - Allows users to share hardware and software resources.
  - Enables printer sharing, file sharing, etc.
- **Data Storage and Access**:
  - Centralized storage systems that allow data access from any connected device.
  - Helps in easy data backup and recovery.
- **Cost Efficiency**:
  - Reduces costs by sharing resources and avoiding duplication of hardware and software.
- **Reliability and Redundancy**:
  - Enhances reliability through alternate paths and redundant systems in case of failures.

KnowledgeGate

Applications of Computer Networks

- **Business and Commerce**:
  - E-commerce, online banking, stock trading, etc.
  - Facilitates remote working and global collaboration.
- **Education**:
  - E-learning platforms, virtual classrooms, online exams, etc.
  - Facilitates research and knowledge sharing.

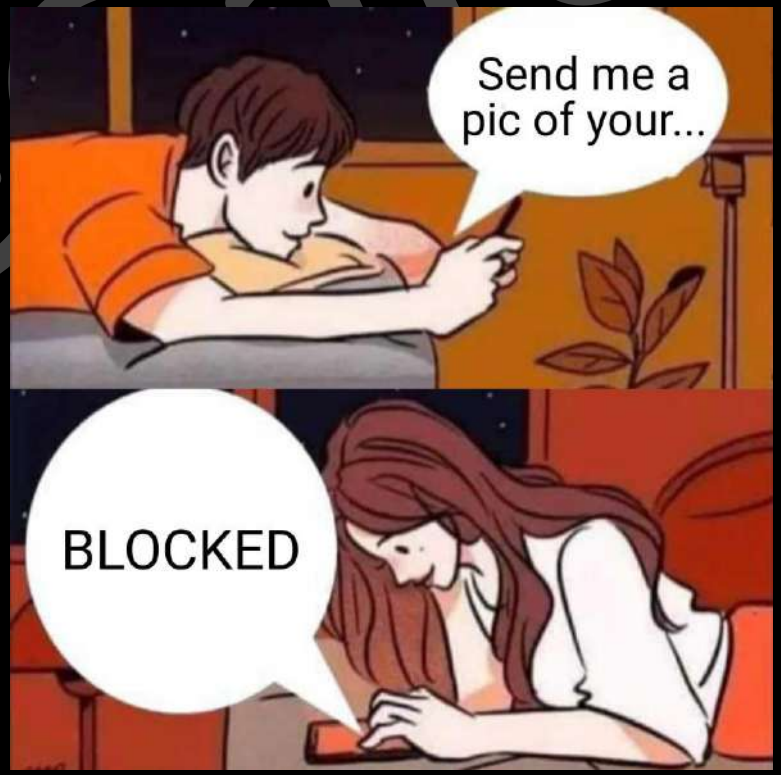


- **Healthcare**:
  - Telemedicine, electronic health records, remote patient monitoring, etc.
- **Government Services**:
  - E-governance, online public services, secure communication between government agencies, etc.
- **Entertainment**:
  - Online gaming, streaming services, social media platforms, etc.
- **Scientific Research**:
  - Facilitates data sharing and collaboration on research projects between institutions worldwide.
- **Travel and Hospitality**:
  - Online ticket booking, hotel reservations, GPS and navigation services, etc.

Knowledge
www.knowledgegate.in

Data communication

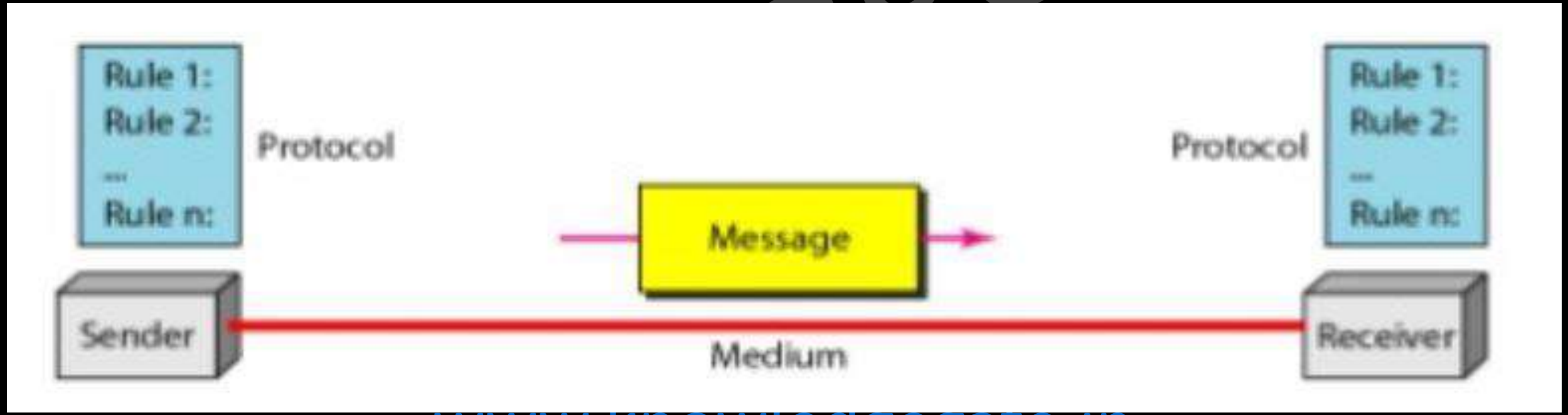
- Data communications are the exchange of data between to two devices via some transmission medium.



Knowledge Gate

Data communication system has five components

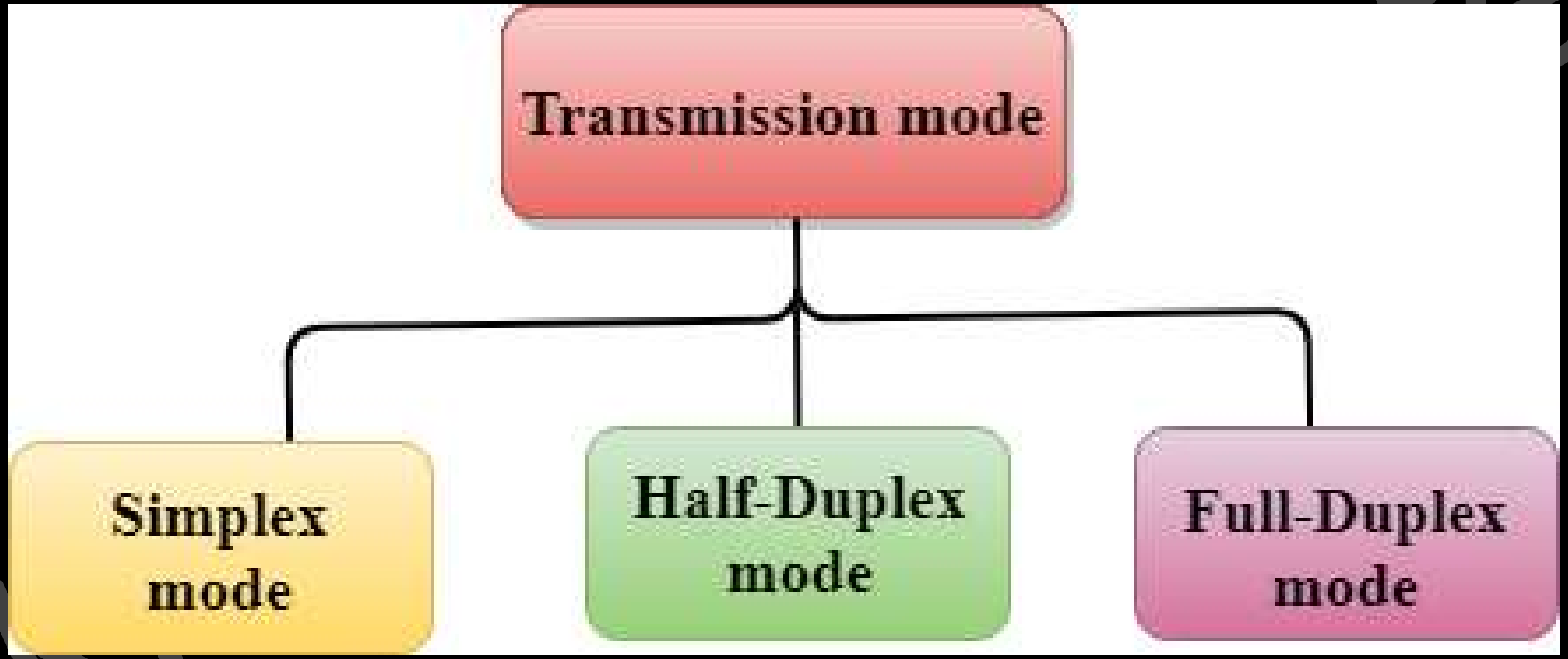
1. **Message**- information (data) to be communicated e.g. text, audio, video.
2. **Sender**- device how sends the message (computer, phone, camera etc.)
3. **Receiver**- device how receives the message (computer, phone, television etc.)
4. **Transmission medium** – is the physical path by which a message travels from sender to receiver.
5. **Protocol** – Which includes Syntax, Semantics, Timing, De facto, De jure.



www.knowledgegate.in

Transmission Mode

- Data flow between two systems can be categorised into three types –



www.knowledgegate.in

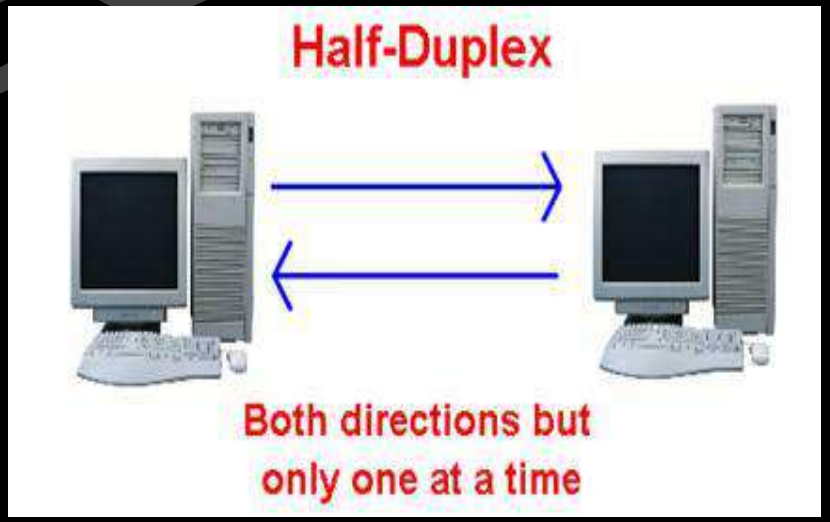
Simplex

- The communication is unidirectional as a one-way street, one device always, other can always receive. E.g. radio, mouse. The simplex mode can use the entire capacity of the channel to send data in one direction.

Knowledge Gate
www.knowledg

Half duplex

- **E**ach station can both transmit and receive, but not at the same time. E.g. like a one lane road, walkie-talkie etc.
  - When one device is sending, the other can only receive, and vice versa.
  - In a half-duplex transmission, the entire capacity of a channel is taken over by whichever of the two devices is transmitting at the time.
  - Walkie-talkies are both half-duplex systems.

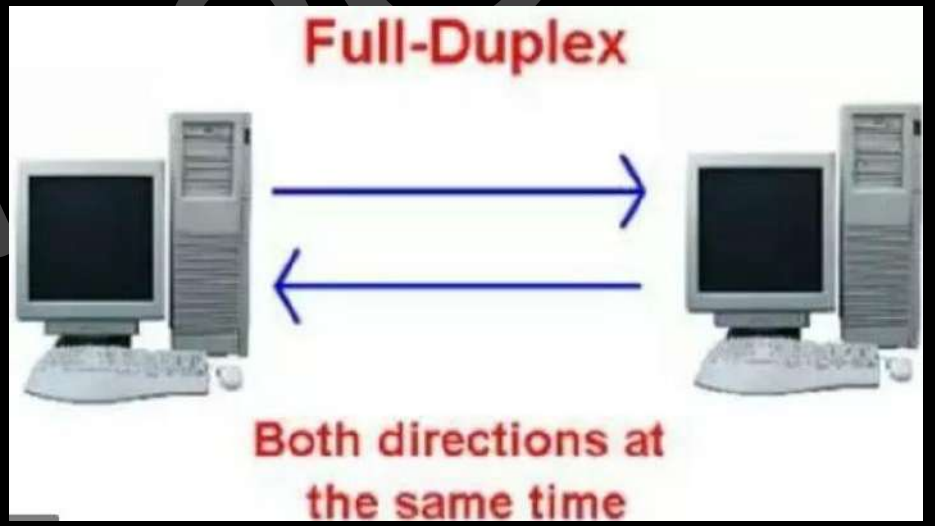


Walkie-talkies

www.knowledgegate.in

Full duplex

- Both stations can transmit and receive at the same time. Actually, it is two half duplex connections.
- Telephone network is an example of full-duplex mode, when two people are communicating by a telephone line, both can talk and listen at the same time.
- The capacity of the channel, must be divided between the two directions.



www.knowledgegate.in

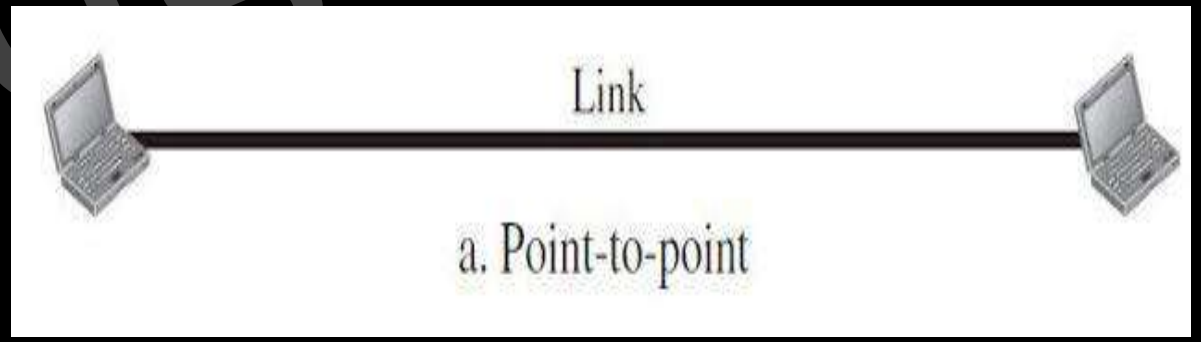
Network criteria

- A network must be able to meet a certain number of criteria. The most important of these are.
  1. **Delivery & Accuracy**- Must deliver the data to correct destination without any error.
  2. **Performance** – Can be measured in many ways including transit time, response time, number of users, type of transmission medium, capabilities of connected hardware's and efficiency of software.
  3. **Reliability** – Is a measure of frequency of failure and the time taken to resolve from the failure.
  4. **Security** – Includes protecting data from unauthorised access, protecting data from damage and development.

KnowledgeGate

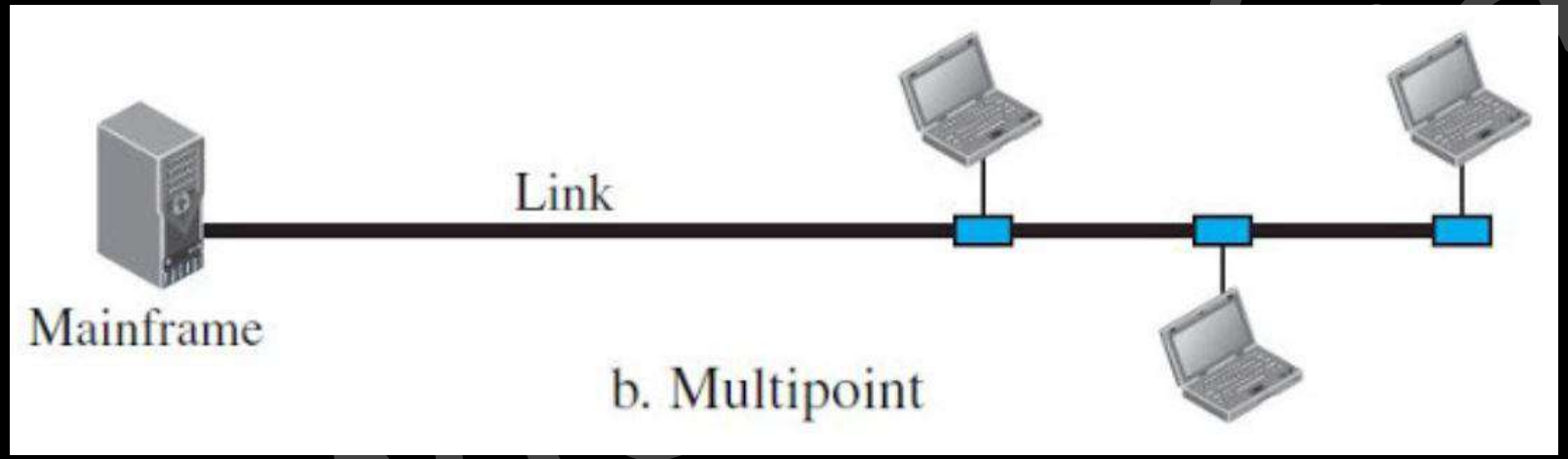
Types of connection

- **Point to point -** A point-to-point connection provides a dedicated link between two devices.
- Most point-to-point connections use an actual length of wire or cable to connect the two ends, but other options, such as microwave or satellite links, are also possible.



KnowledgeGate

- **Multipoint** - A multipoint (also called multidrop) connection is one in which more than two specific devices share a single link.

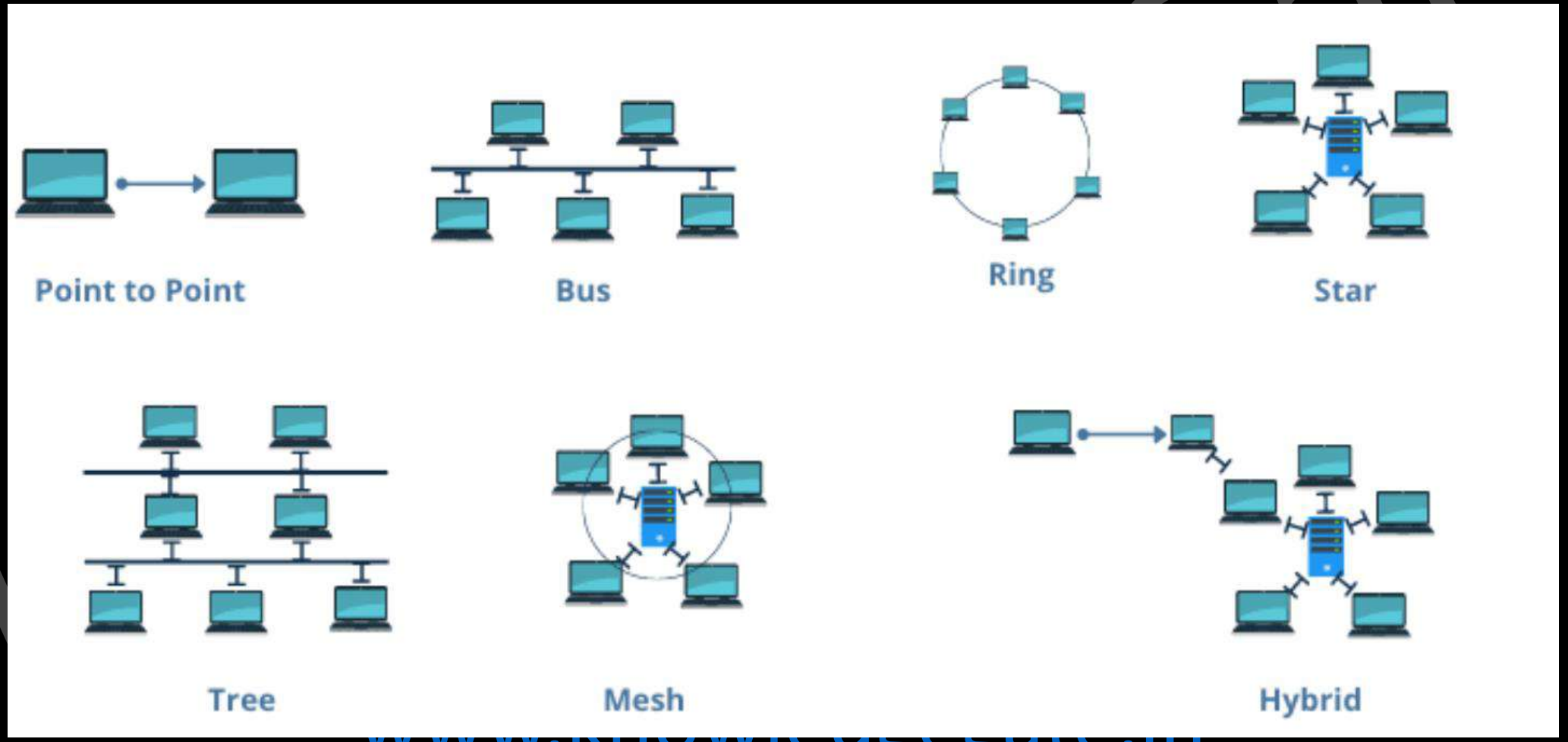


Knowledge Gate

www.knowledgegate.in

Physical topology

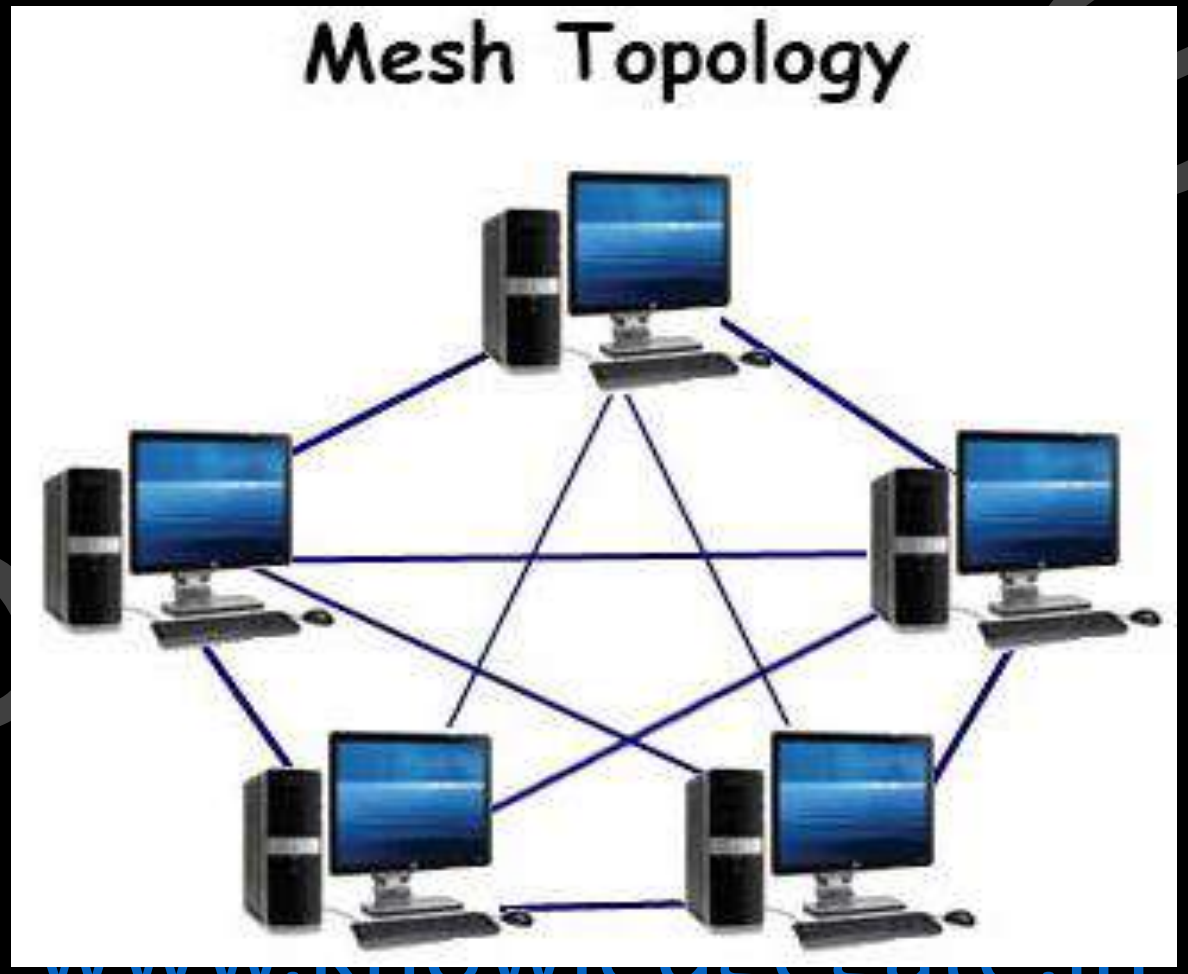
- Refers to the way in which a network is laid out physically. Topology of a network is the geometric representation of the relationship of all the links and linking devices to one another.



www.knowledgegate.in

Mesh Topology

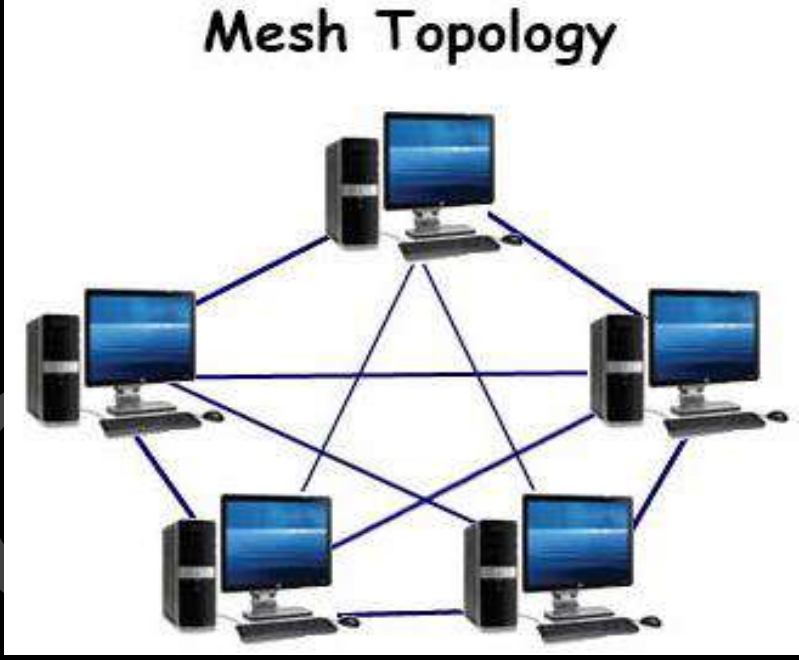
- In a mesh topology, every device has a dedicated point-to-point link to every other device. We need, **n (n -1) /2,** duplex-mode links, where n is number of nodes.



Knowledge Gate
www.knowledgegate.in

- **Advantages**
  1. No **traffic problems**
  2. **Robust**
  3. **Privacy or security**
  4. **Fault identification and fault isolation easy.**

- **Disadvantage**
  1. **Installation and reconnection are difficult.**
  2. The **sheer bulk** of the wiring.
  3. **Expensive.**



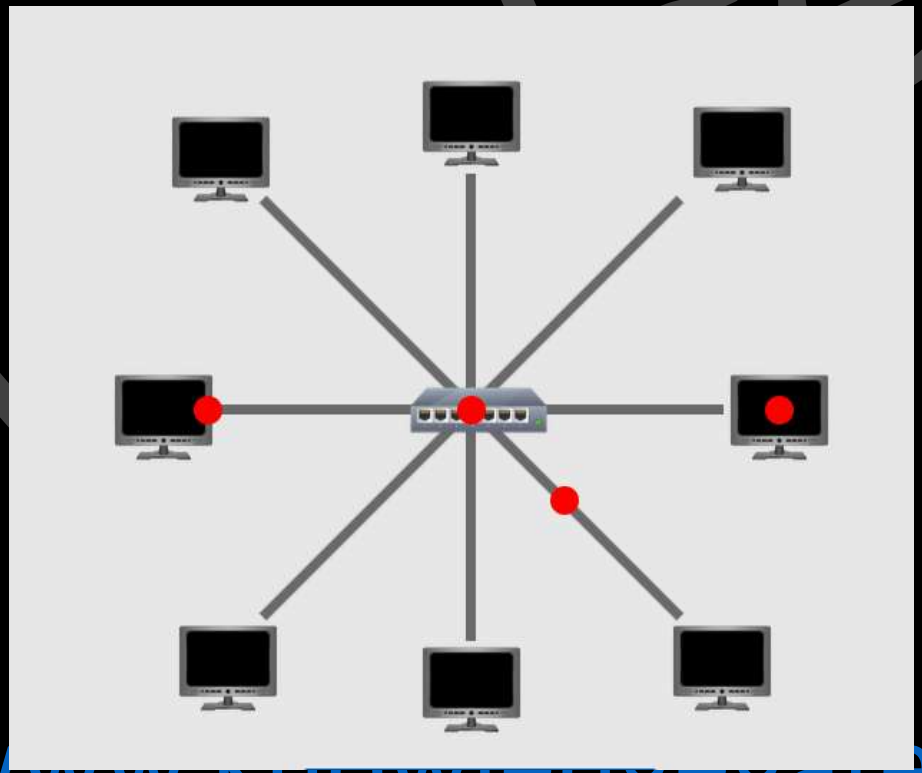
Knowledge

www.knowledgegate.in

Star Topology

- In a star topology, each device has a dedicated point-to-point link only to a central controller, usually called a hub. The devices are not directly linked to one another.

- The controller acts as an exchange: If one device wants to send data to another, it sends the data to the controller, which then relays the data to the other connected device.

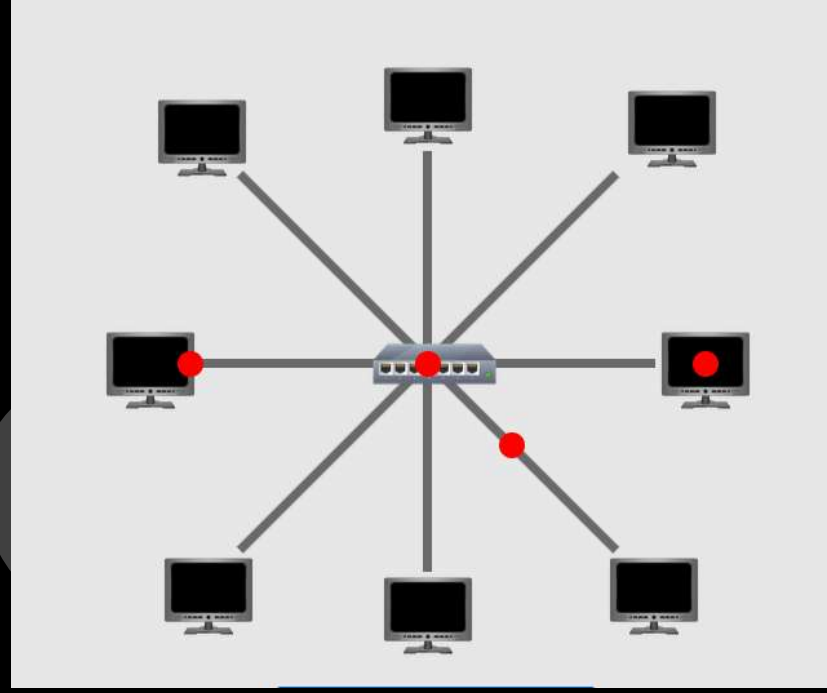


KnowledgeGate

www.knowledgegate.in

Advantages

1. Less expensive than a mesh topology.
2. Easy to install and reconfigure and less costly.
3. It is robust. If one link fails, only that link is affected.
4. Easy fault identification and fault isolation.



Disadvantage

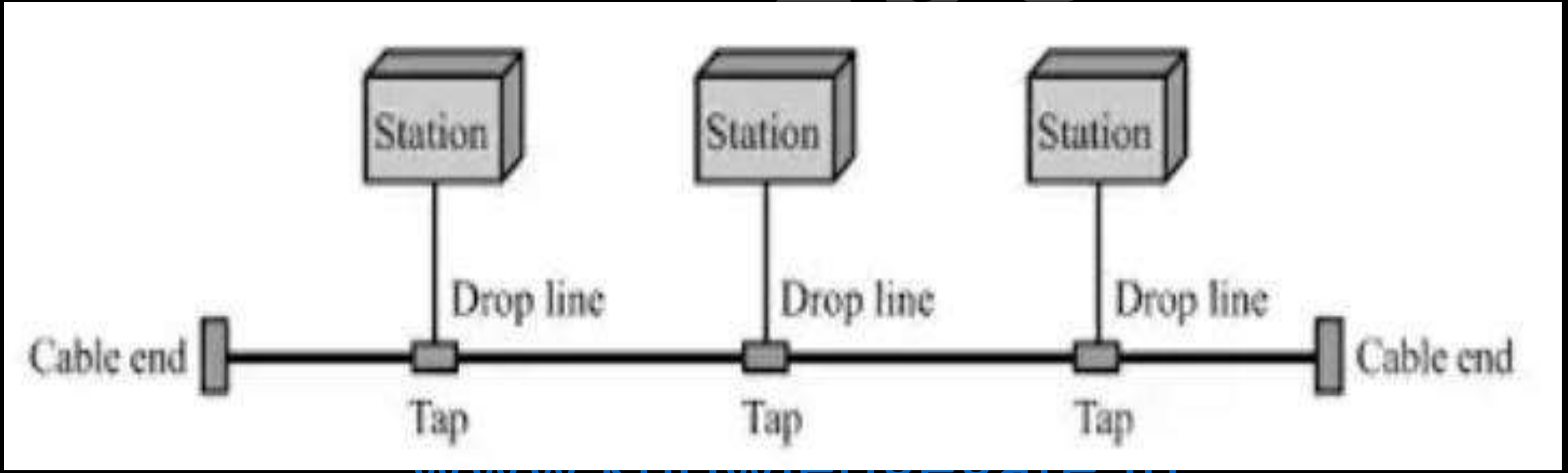
1. Dependency of the whole topology on one single point, the hub.
2. Often more cabling is required in a star than in some other topologies.

Knowledge

www.knowledgegate.in

Bus Topology

- A bus topology, is multipoint. One long cable acts as a backbone to link all the devices in a network.
- Nodes are connected to the bus cable by drop lines and taps.
- A drop line is a connection running between the device and the main cable.
- A tap is a connector that either splices into the main cable or punctures the sheathing of a cable to create a contact with the metallic core.



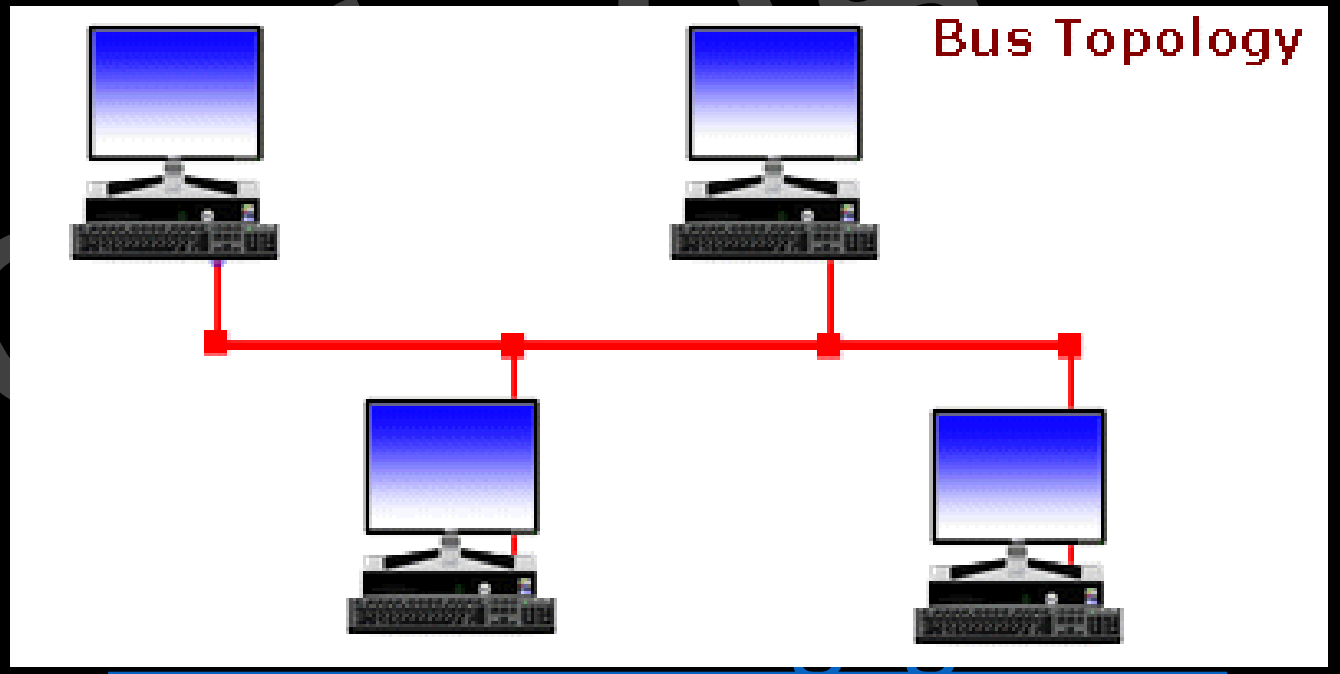
KnowledgeGate
www.knowledgegate.in

Advantages

1. Ease of installation.
2. Uses less cabling than mesh or star topologies.

Disadvantage

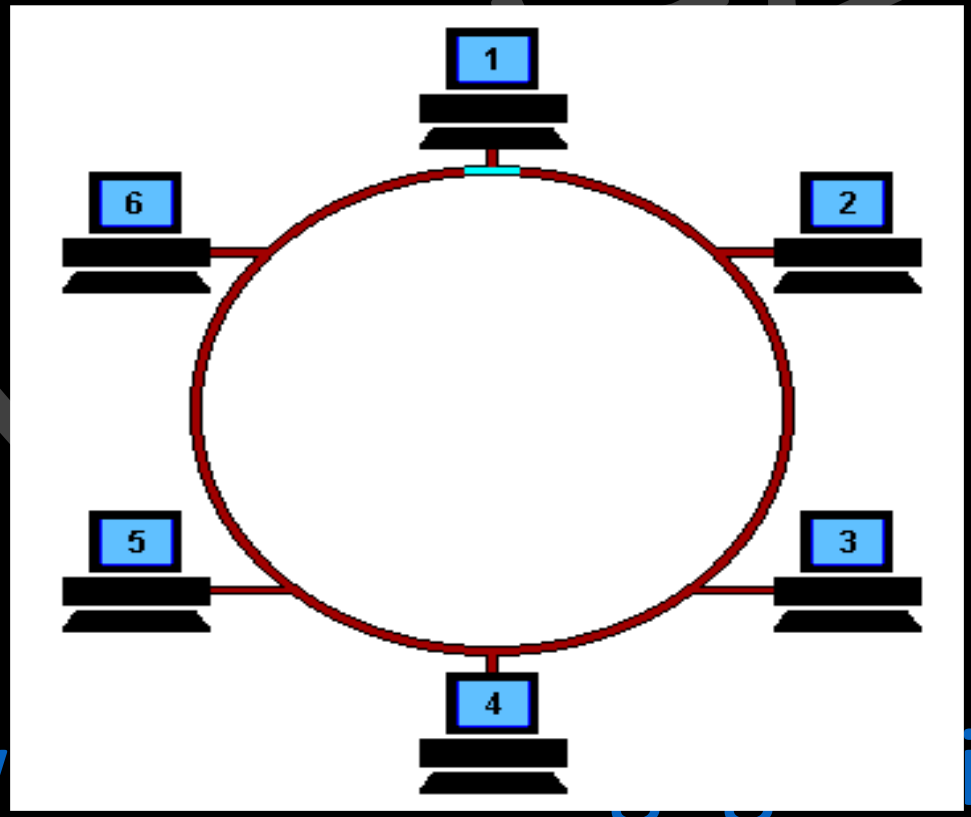
1. Difficult reconnection and fault isolation.
2. Difficult to add new devices to network.
3. A fault or break in the bus cable stops all transmission.



KnowledgeGate

Ring Topology

- In a ring topology, each device has a dedicated point-to-point connection with only the two devices on either side of it.
- A signal is passed along the ring in one direction, from device to device, until it reaches its destination. Each device in the ring incorporates a repeater.
- When a device receives a signal intended for another device, its repeater regenerates the bits and passes them along.



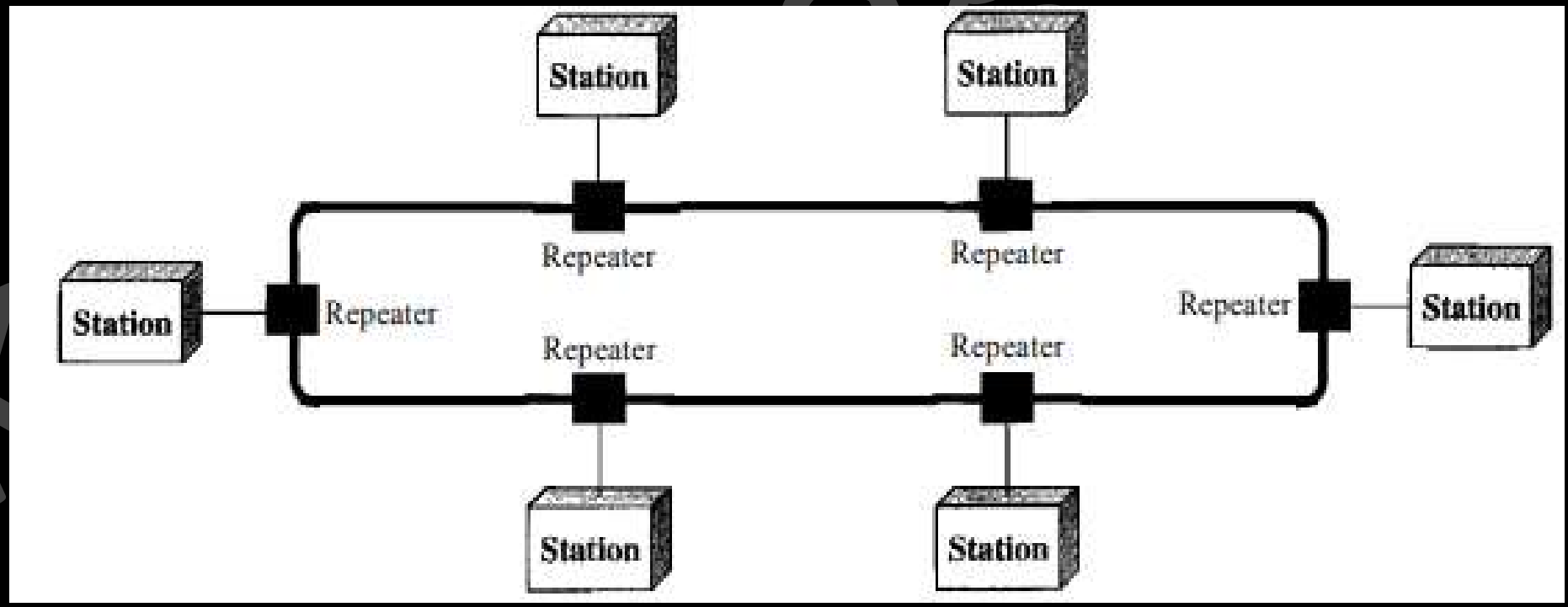
KnowledgeGate

Advantages

1. A ring is relatively easy to install and reconfigure.
2. Fault isolation is simplified.

Disadvantages

1. A break in the ring (such as a disabled station) can disable the entire network.



KnowledgeGate

- **Local Area Network** (LAN) :
  - LAN is usually limited to a few kilometers of area.
  - It may be privately owned and could be a network inside an office on one of the floor of a building or a LAN could be a network consisting of the computers in an entire building.

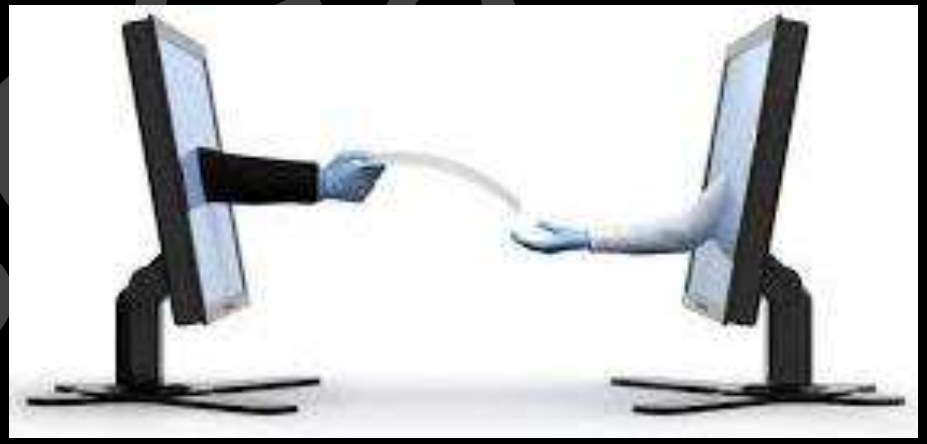
- **Wide Area Network** (WAN) :
  - WAN is made of all the networks in a (geographically) large area.
  - The network in the entire state of UP could be a WAN.

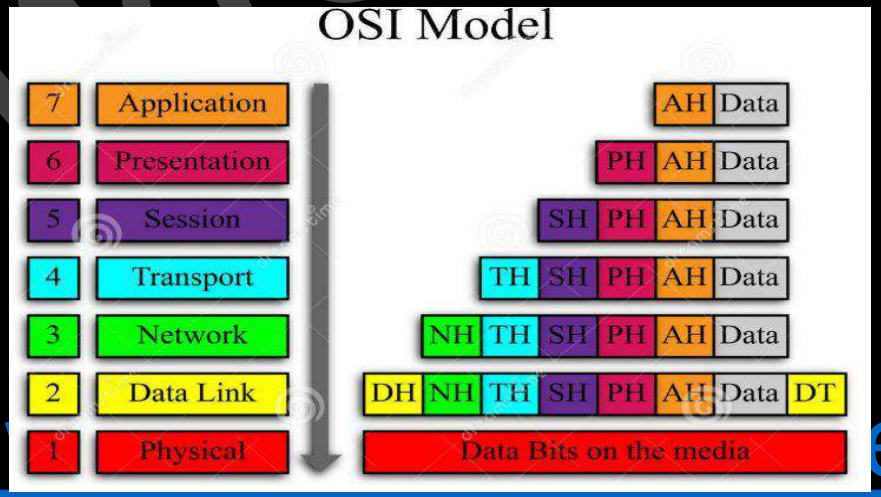
- **Metropolitan Area Network** (MAN) :
  - MAN is of size between LAN and WAN.
  - It is larger than LAN but smaller than WAN.
  - It may comprise the entire network in a city like Mumbai.

KnowledgeGate

Network Models

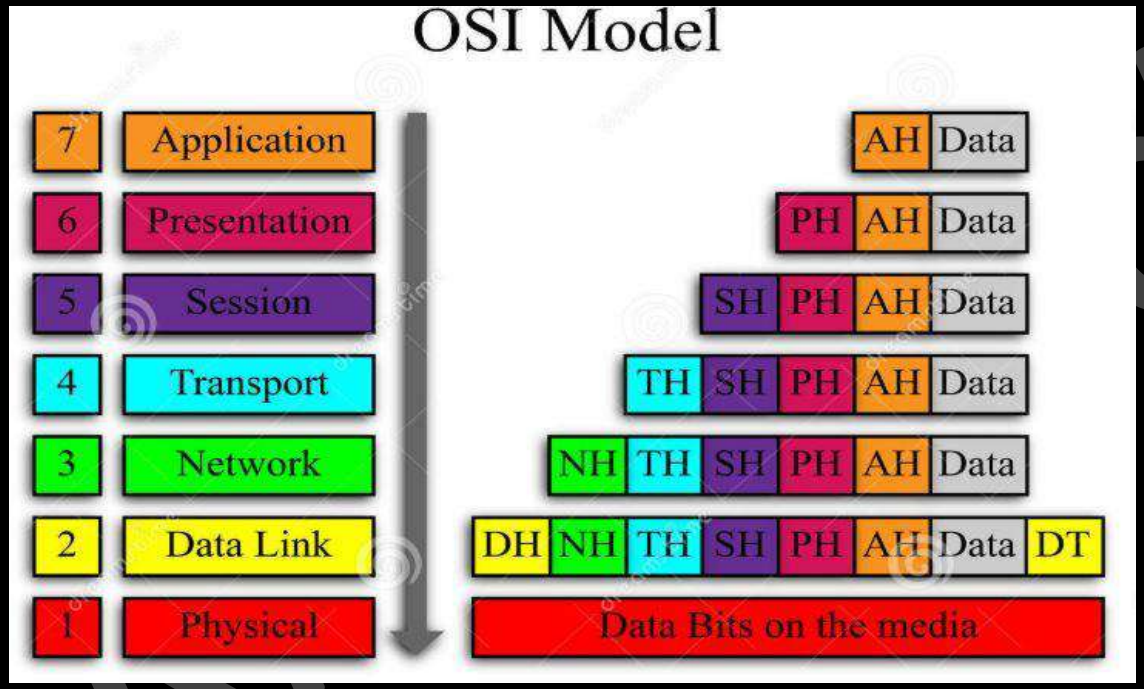
- International standard organization (ISO)- proposed an open system interconnection (OSI) model that allows two system to communicate regardless of their architecture.

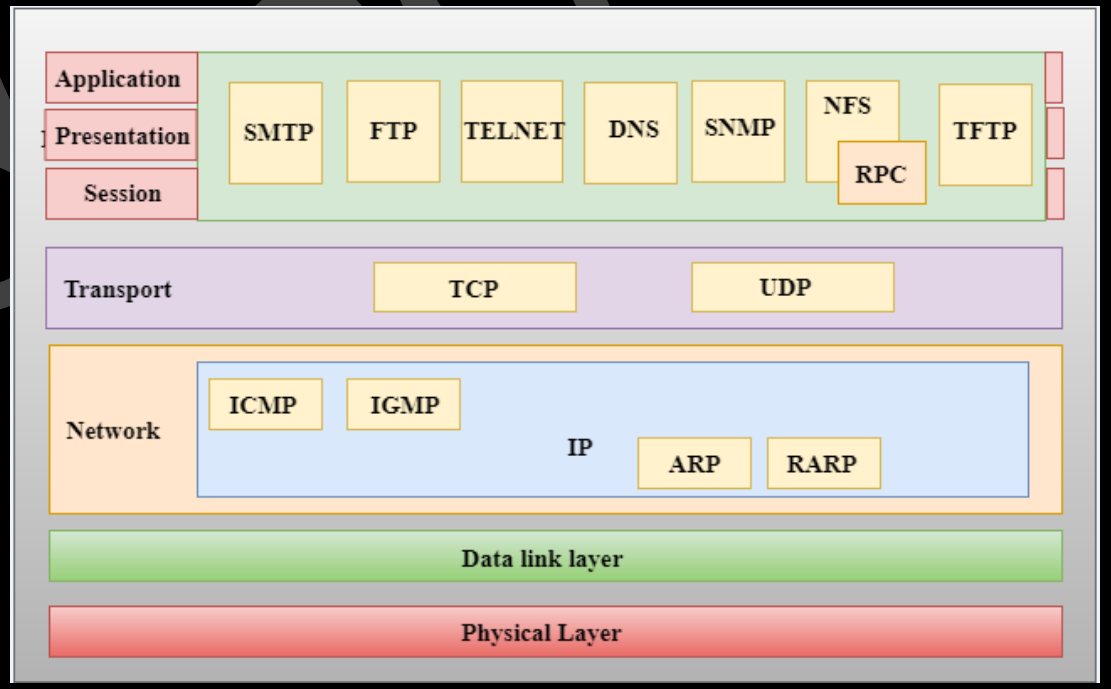




Knowledge Gate
w e.in

- The purpose of the OSI model is to show how to facilitate communication between different systems without requiring changes to the logic of the underlying hardware and software.
- The OSI model is not a protocol; it is a model for understanding and designing a network architecture that is flexible, robust, and interoperable. It consists of seven separate but related layers, each of which defines a part of the process of moving information across a network.





Directive Principles Constitution

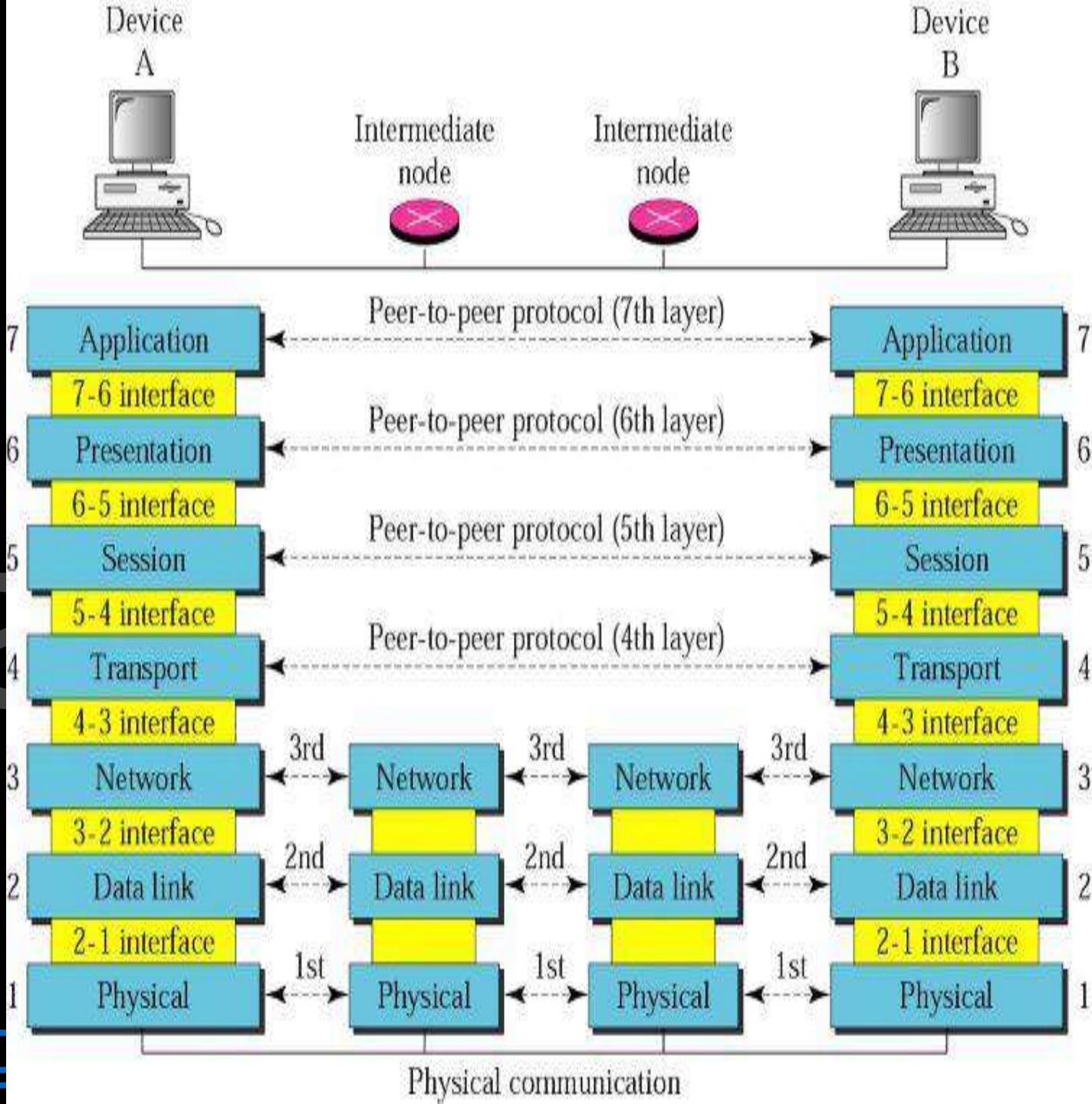
www.knowledgegate.in

Layered Architecture

1. The OSI model is composed of seven ordered layers, within a single machine, each layer calls upon the services of the layer just below it and provide services to the layer above it.
2. Between machines, layer x on one machine communicates with layer x on another machine. This communication is governed by an agreed-upon series of rules and conventions called protocols.
3. The processes on each machine that communicate at a given layer are called peer-to-peer processes.

Knowle

www.kno

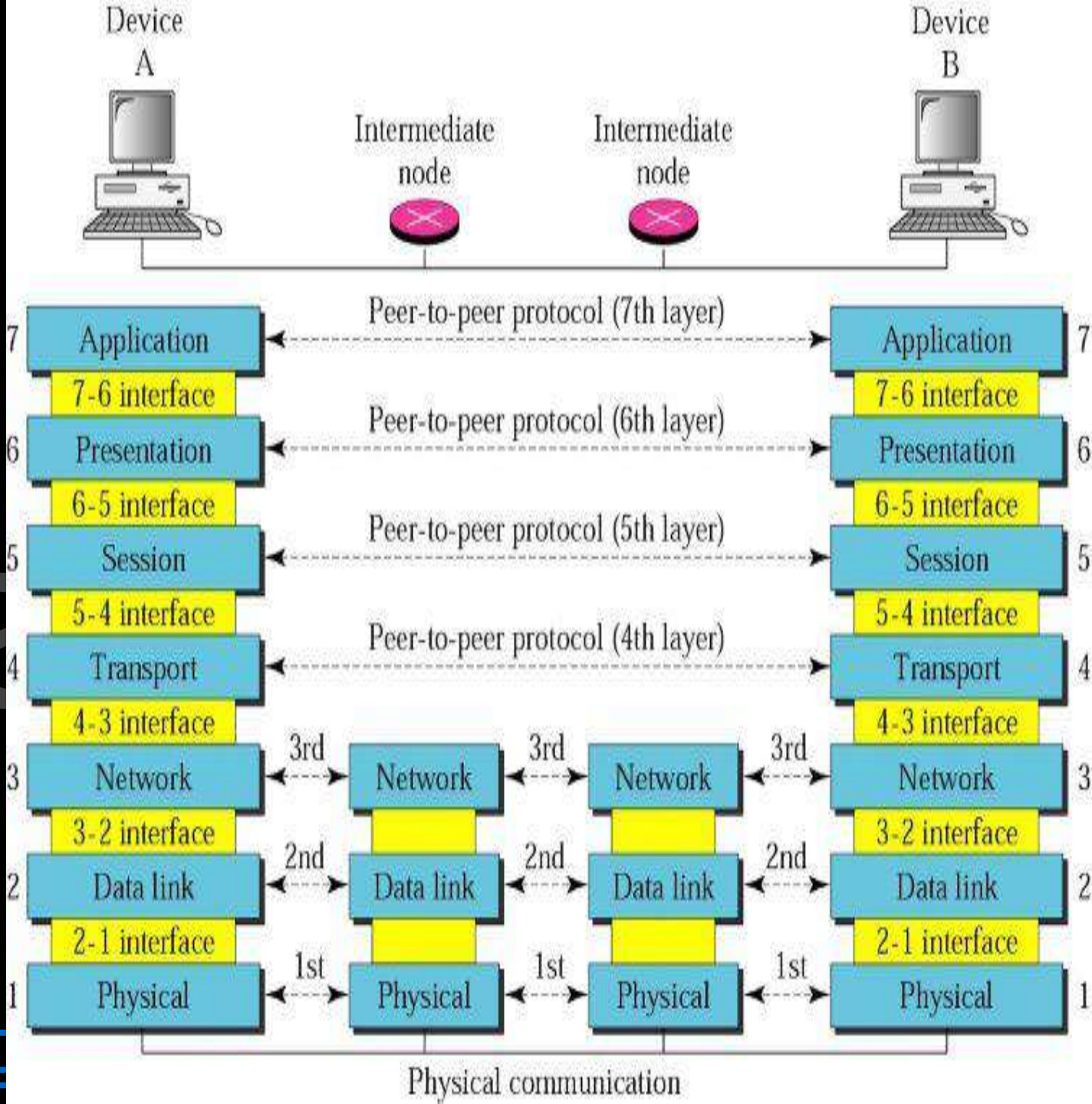


Peer-to-Peer Processes

- At the physical layer, communication is direct: device A sends a stream of bits to device B (through intermediate nodes).
- At the higher layers, communication must move down through the layers on device A, over to device B, and then back up through the layers.
- Each layer in the sending device adds its own information to the message it receives from the layer just above it and passes the whole package to the layer just below it.
- At layer 1 the entire package is converted to a form that can be transmitted to the receiving device.
- At the receiving machine, the message is unwrapped layer by layer, with each process receiving and removing the data meant for it.

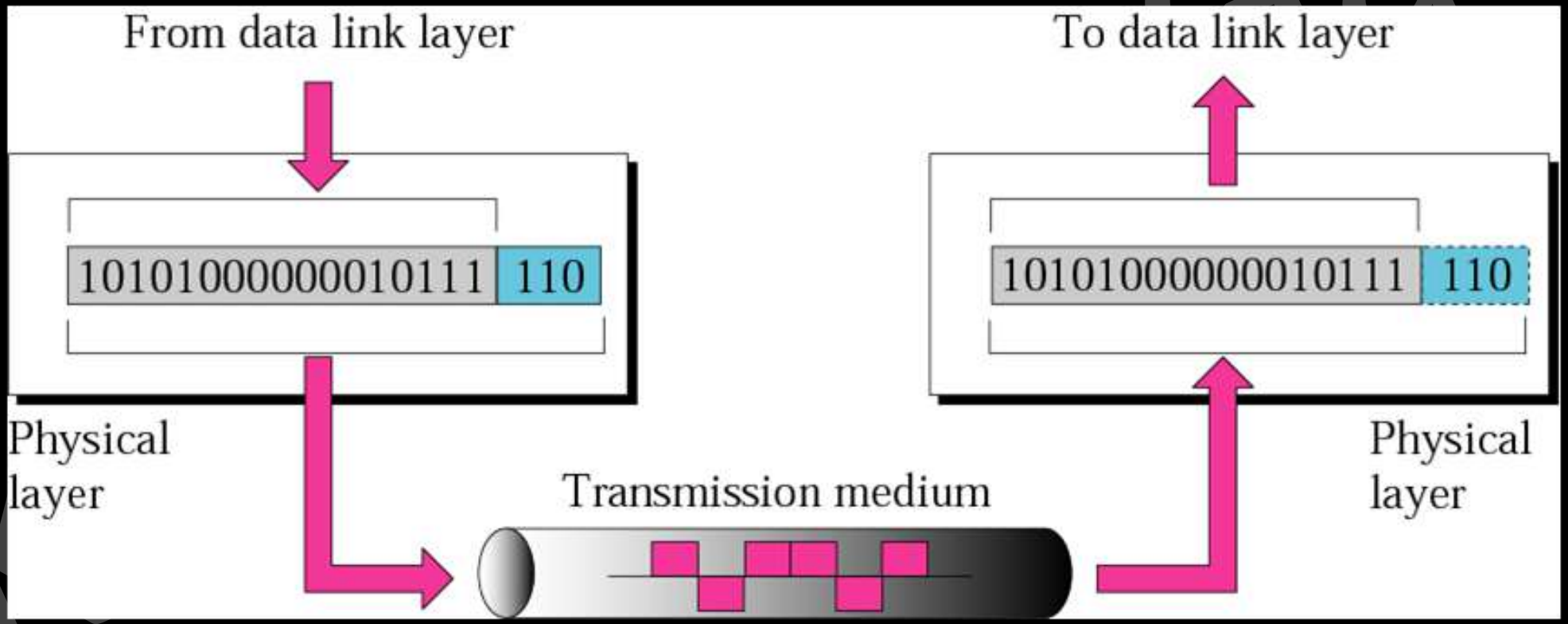
Knowle

www.kno



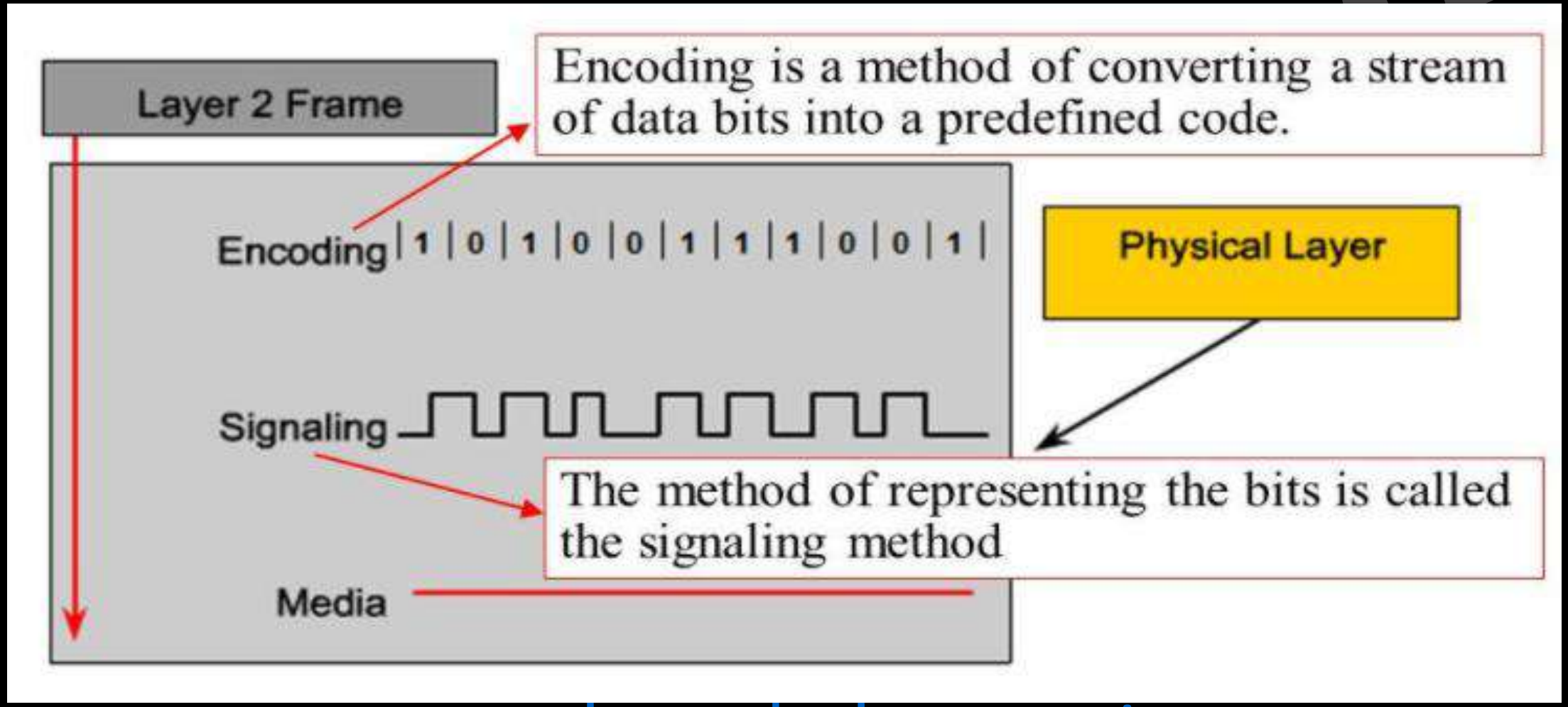
Physical layer

- The physical layer defines the **characteristics of the interface** between the devices and the transmission medium.



www.knowledgegate.in

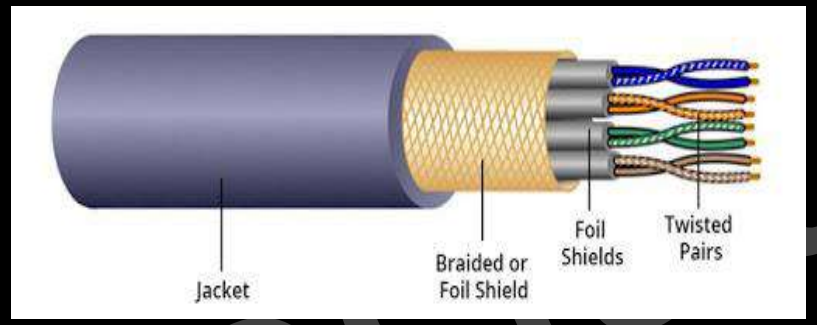
1. Representation of bits: The physical layer data consists of a stream of bits (sequence of 0s or 1s) with no interpretation. To be transmitted, bits must be encoded into signals- electrical or optical.

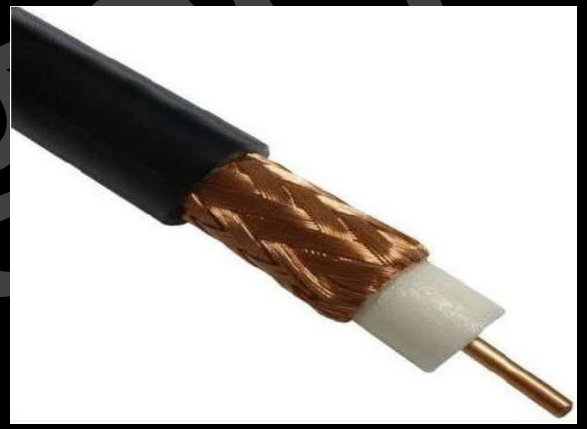


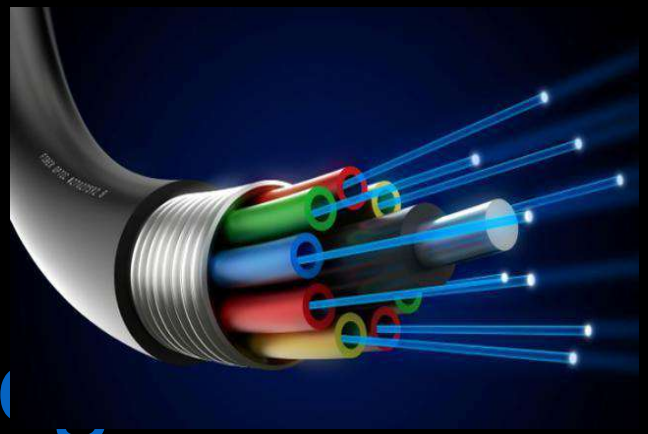
www.knowledgegate.in

2. Data rate: The transmission rate-the number of bits sent each second-is also defined by the physical layer.

Type of Cable and LAN	Transfer Rates
Twisted Pair	
• 10Base-T (Ethernet)	10 Mbps
• 100Base-T (Fast Ethernet)	100 Mbps
• 1000Base-T (Gigabit Ethernet)	1000 Mbps
• Token ring	4 - 16 Mbps
Coaxial Cable	
• 10Base2 (ThinWire Ethernet)	10 Mbps
• 10Base5 (ThickWire Ethernet)	10 Mbps
Fiber-Optic Cable	
• 10Base-F (Ethernet)	10 Mbps
• 100Base-FX (Fast Ethernet)	100 Mbps
• FDDI (Fiber Distributed-Data Interface) token ring	100 Mbps

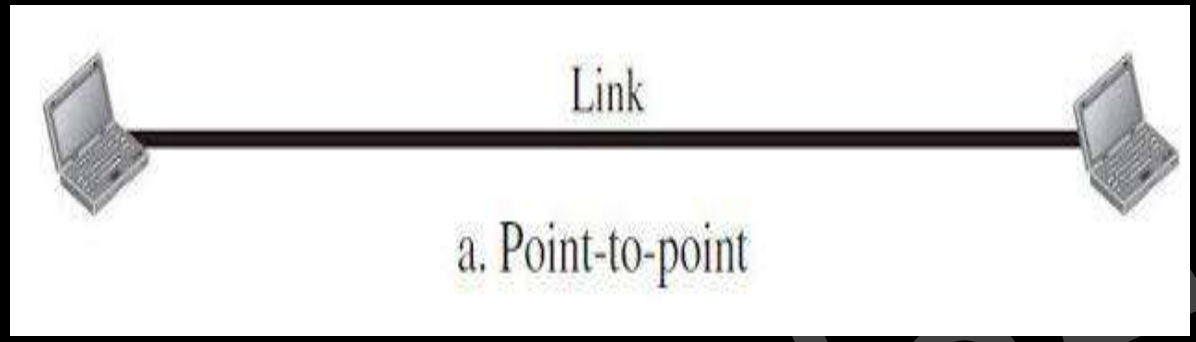


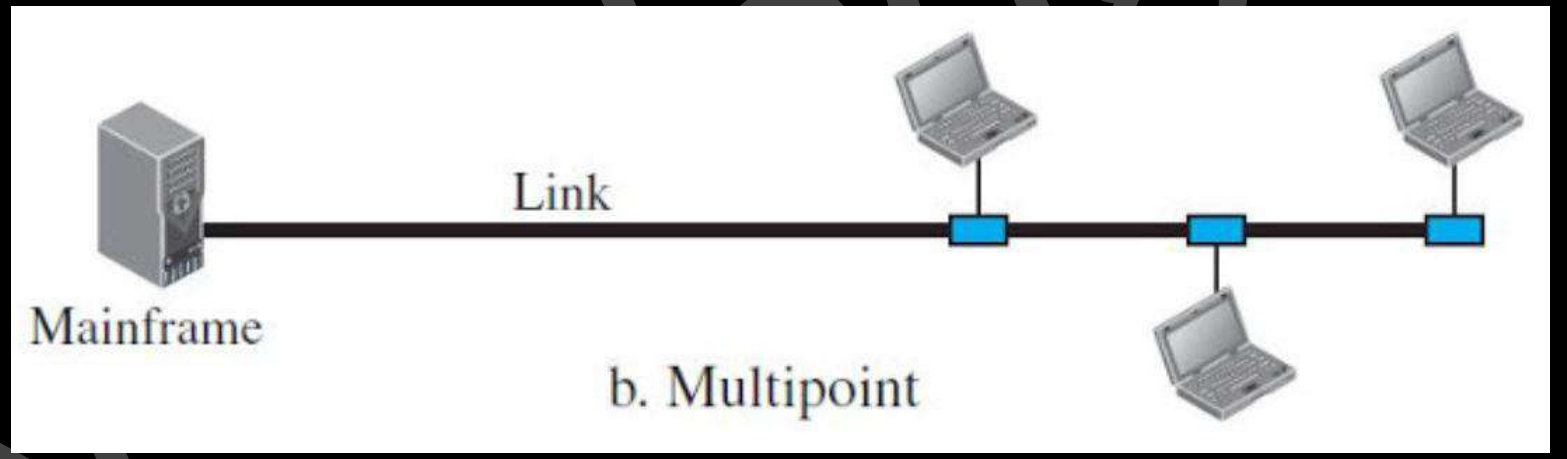




www.knowledg

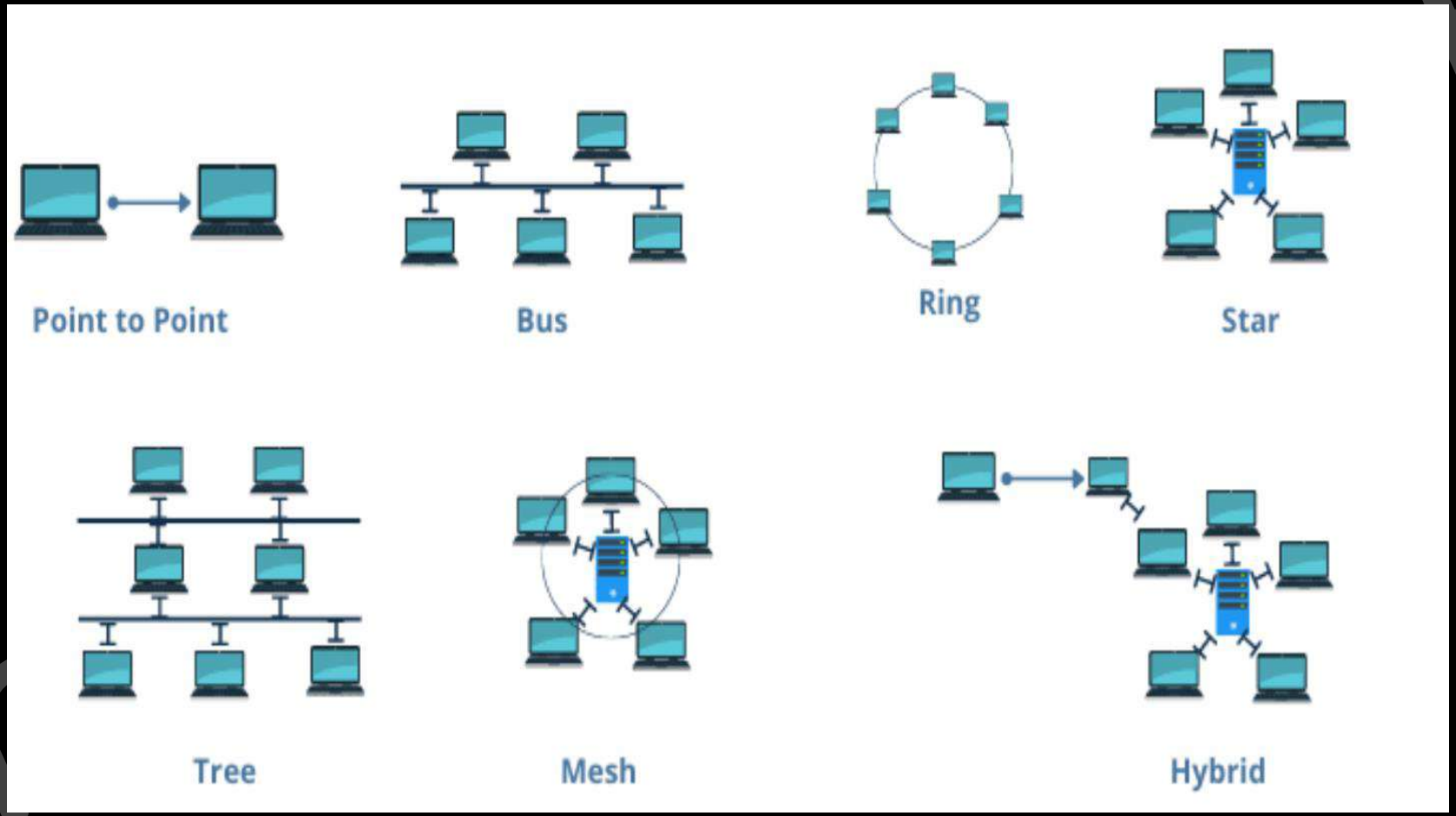
3. Line configuration: The physical layer is concerned with the connection of devices to the media.





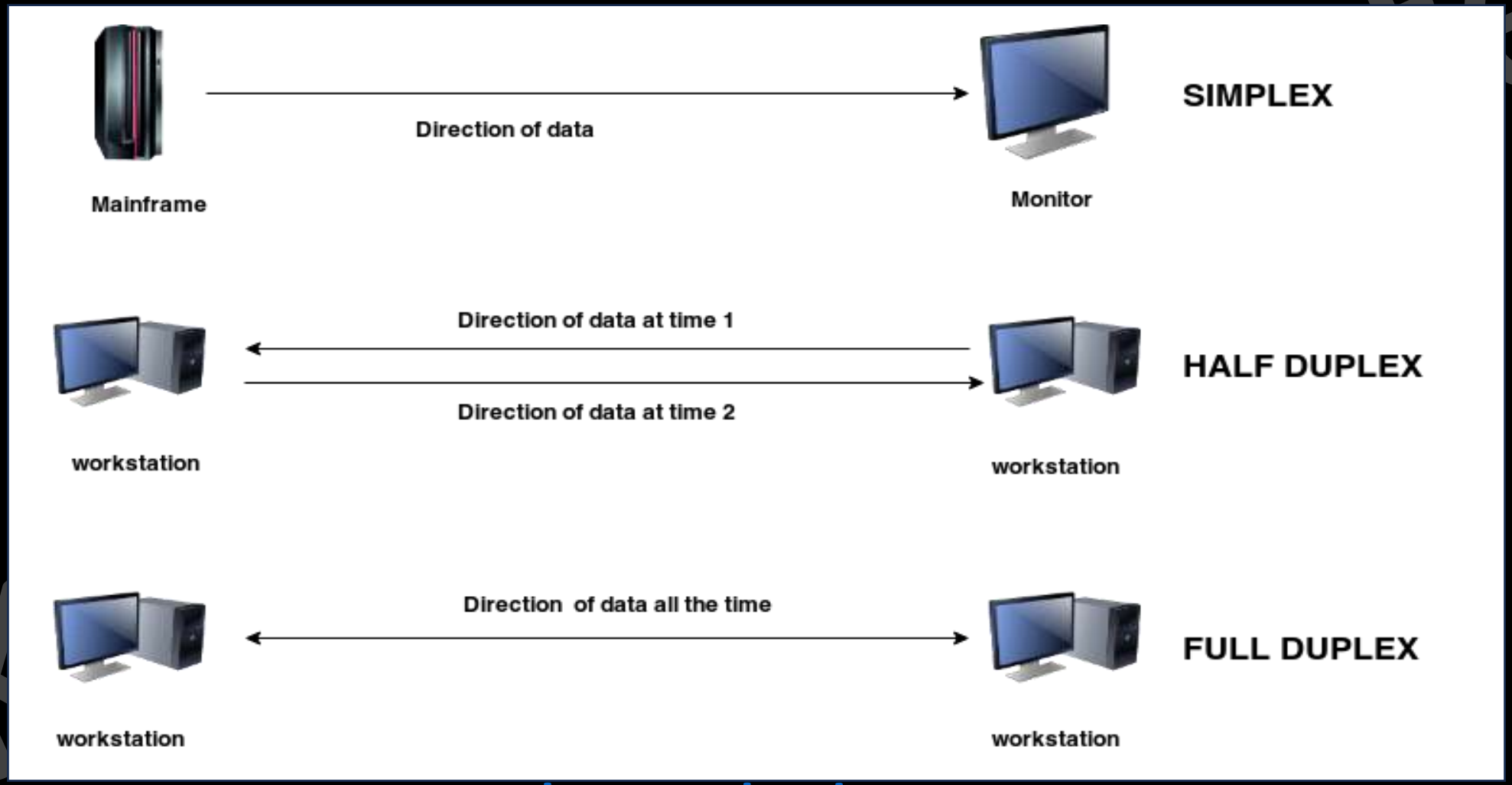
KnowledgeGate

4. Physical topology: The physical topology defines how devices are connected to make a network.



www.knowledgegate.in

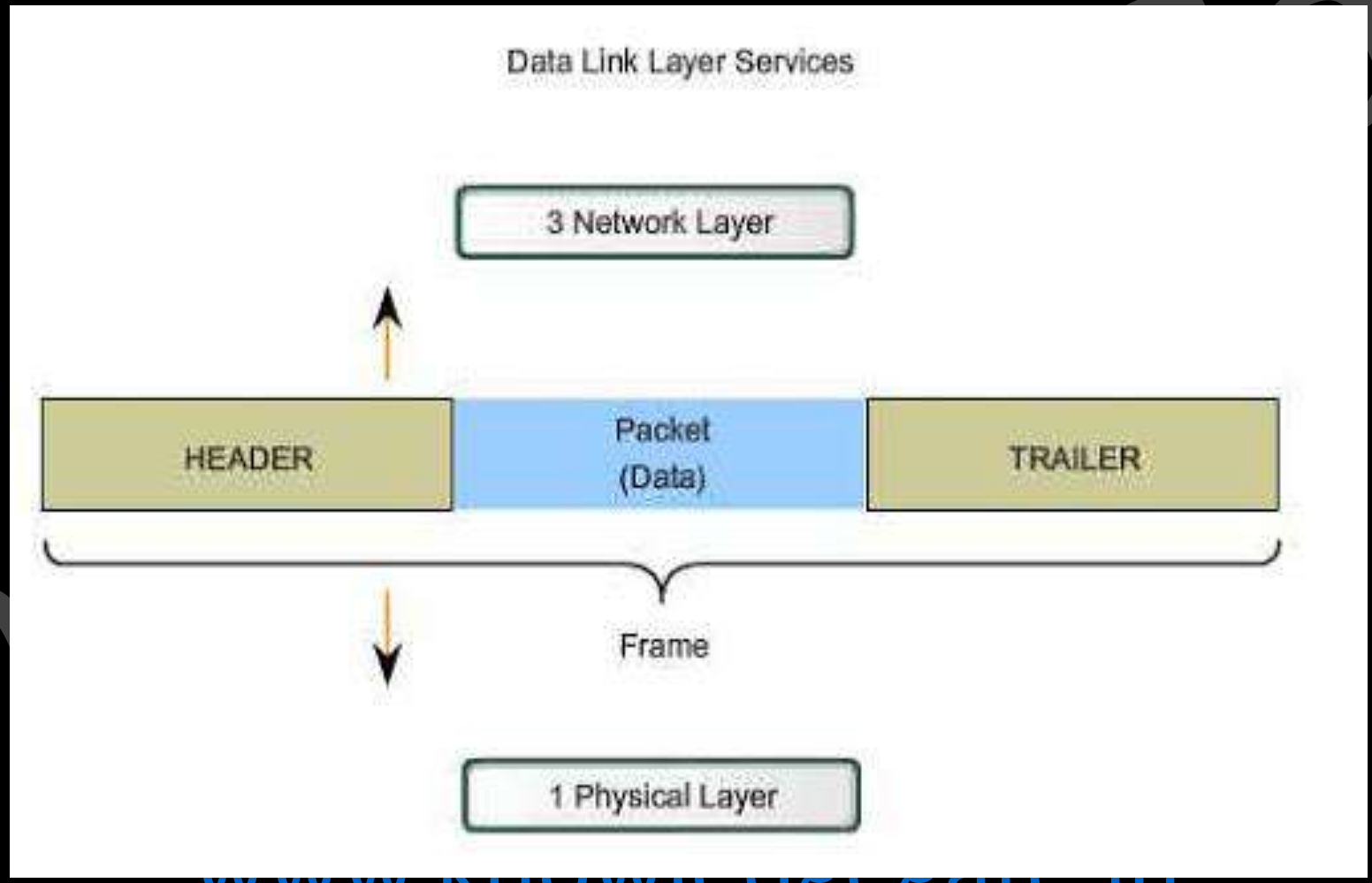
5. Transmission mode: The physical layer also defines the direction of transmission between two devices: simplex, half-duplex, or full-duplex.



www.knowledgegate.in

Data link layer

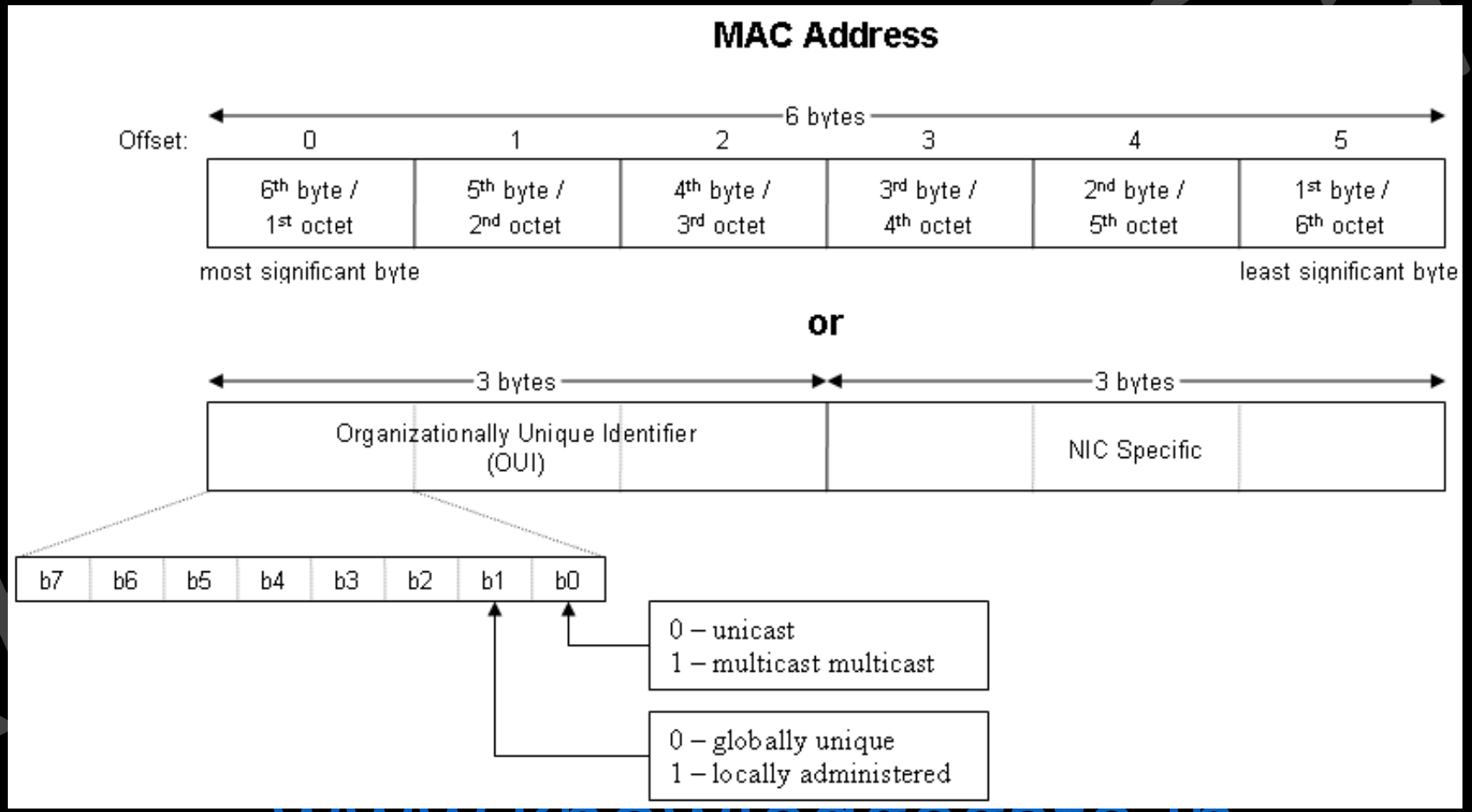
1. Framing: The data link layer divides the stream of bits received from the network layer into manageable data units called frames.



www.knowledgegate.in

Data link layer

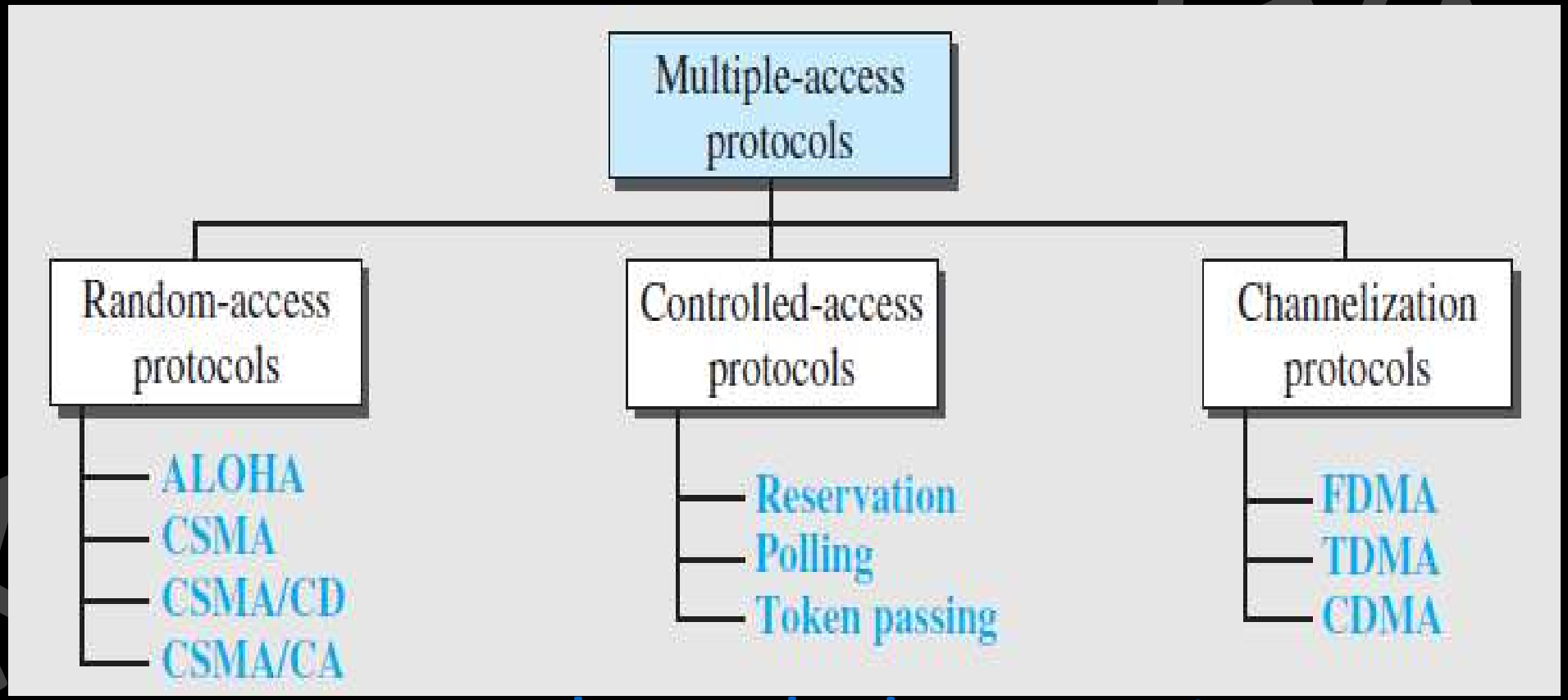
2. Physical addressing: If frames are to be distributed to different systems on the network, the data link layer adds a header to the frame to define the sender and/or receiver of the frame.



www.knowledgegate.in

Data link layer

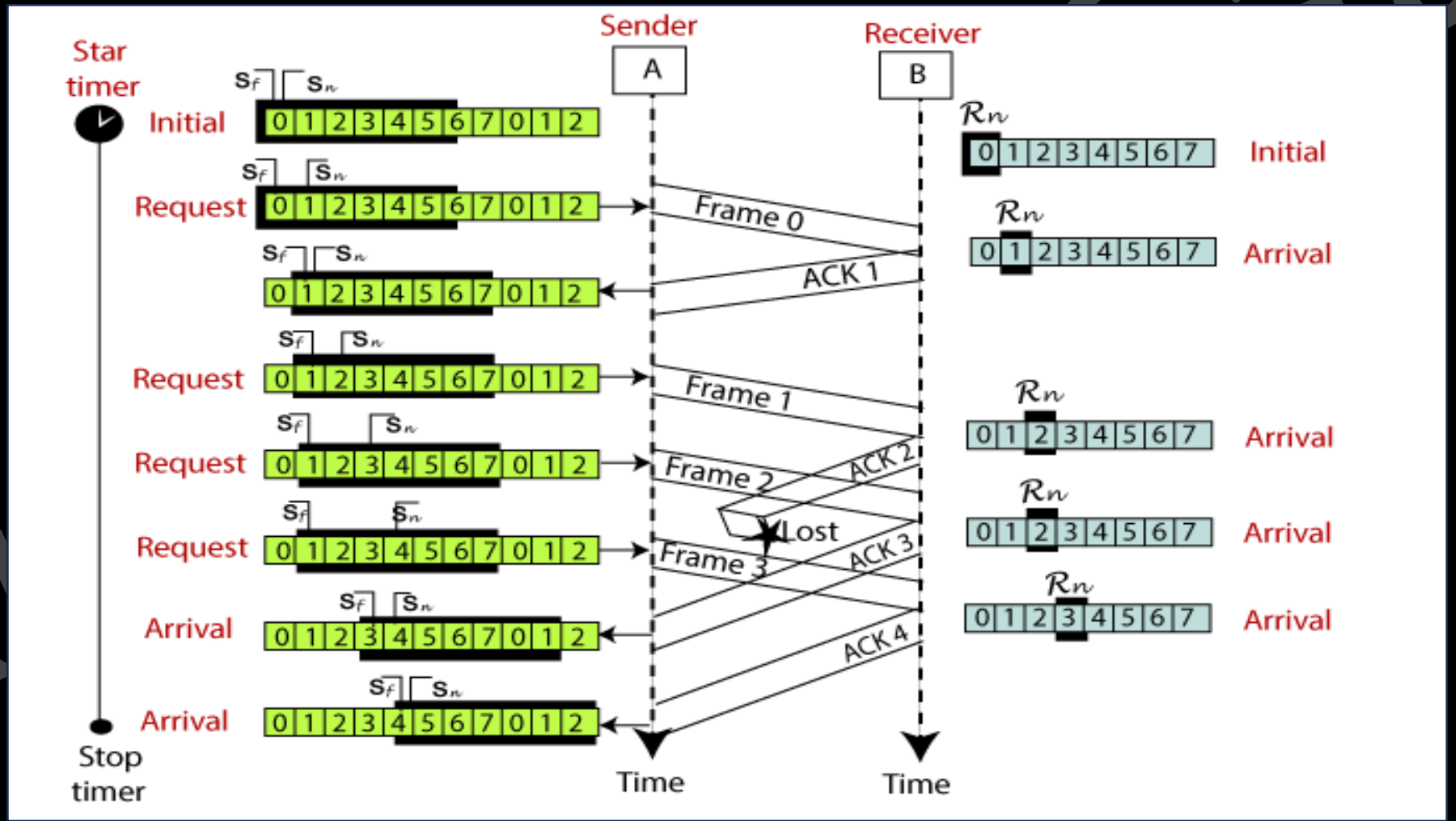
3. Access control: When two or more devices are connected to the same link, data link layer protocols are necessary to determine which device has control over the link at any given time.



www.knowledgegate.in

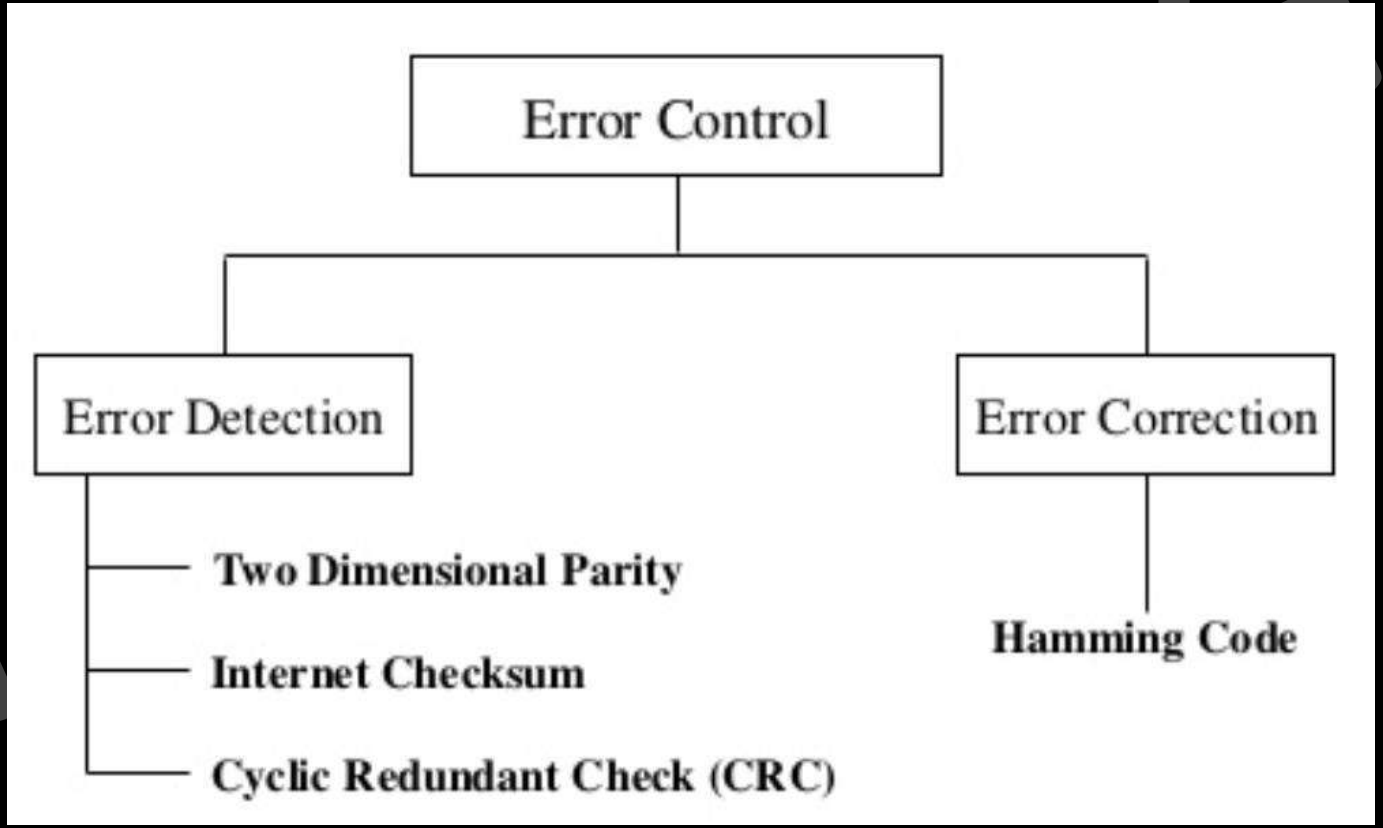
Data link layer

4. Flow control: If the rate at which the data are absorbed by the receiver is less than the rate at which data are produced in the sender, the data link layer imposes a flow control mechanism to avoid overwhelming the receiver.



Data link layer

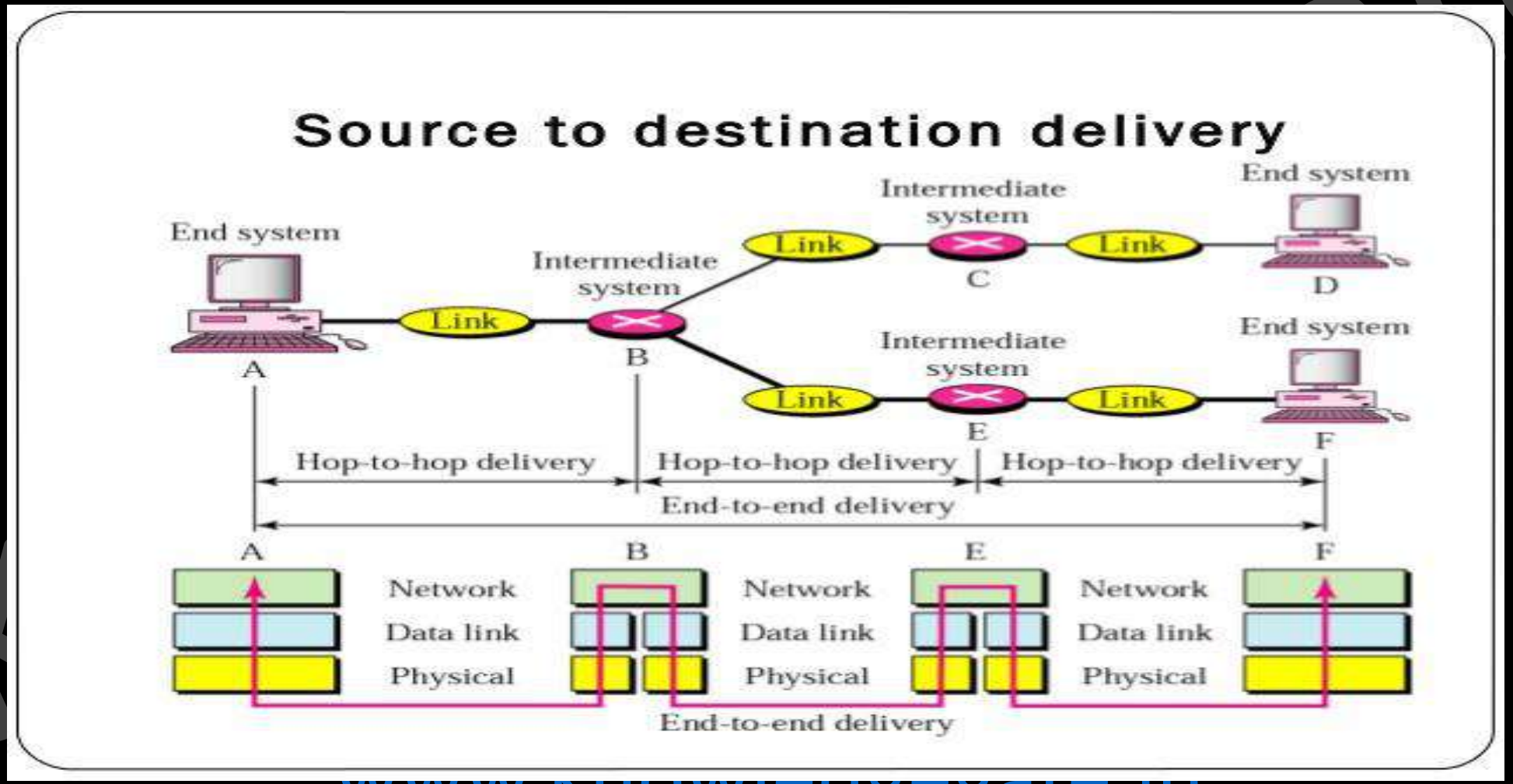
5. Error control: The data link layer adds reliability to the physical layer by adding mechanisms to detect and retransmit damaged or lost frames. It also uses a mechanism to recognize duplicate frames. Error control is normally achieved through a trailer added to the end of the frame.



www.knowledgegate.in

Network layer

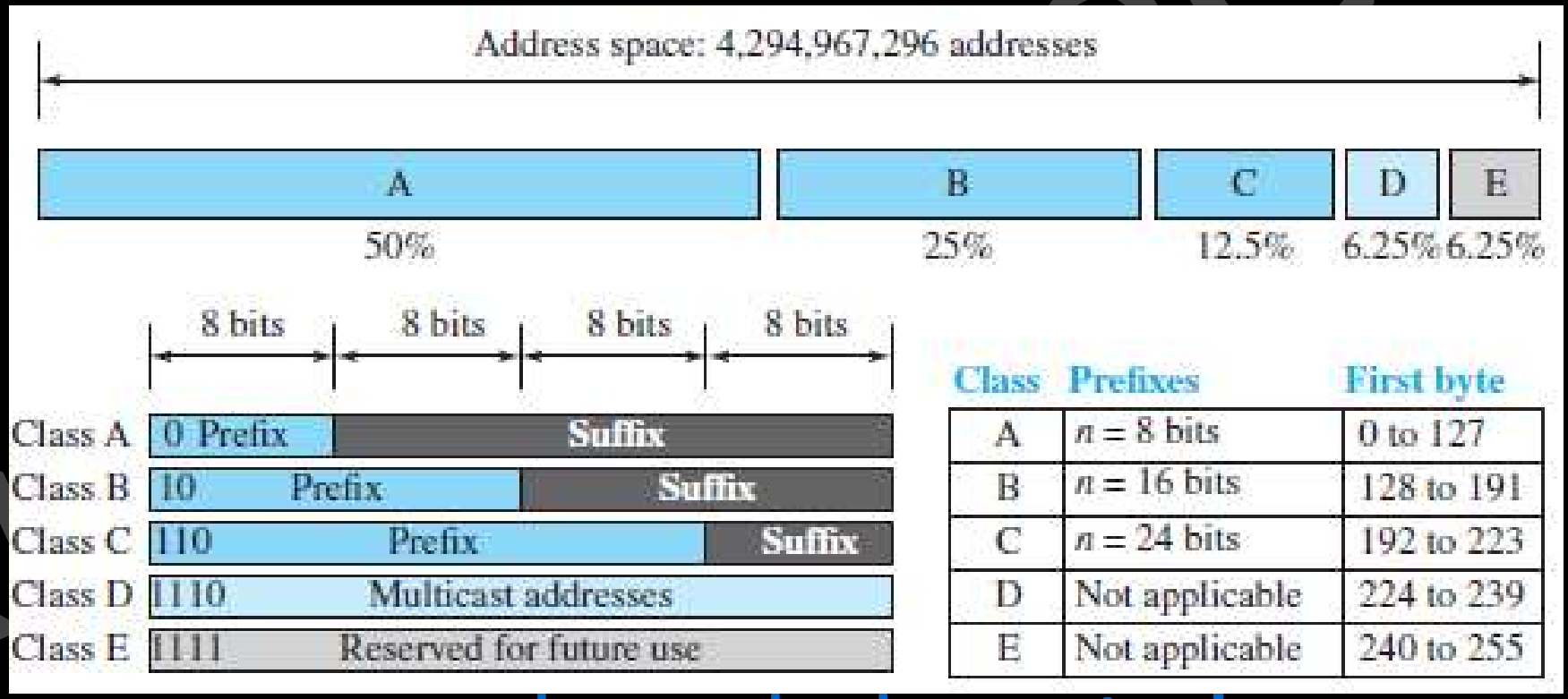
- The network layer is responsible for the **source-to-destination delivery** of a packet, possibly across multiple networks (links).



www.knowledgegate.in

Network layer

1. Logical addressing: If a packet passes the network boundary, we need another addressing system to help distinguish the source and destination systems. The network layer adds a header to the packet coming from the upper layer that, among other things, includes the logical addresses of the sender and receiver.

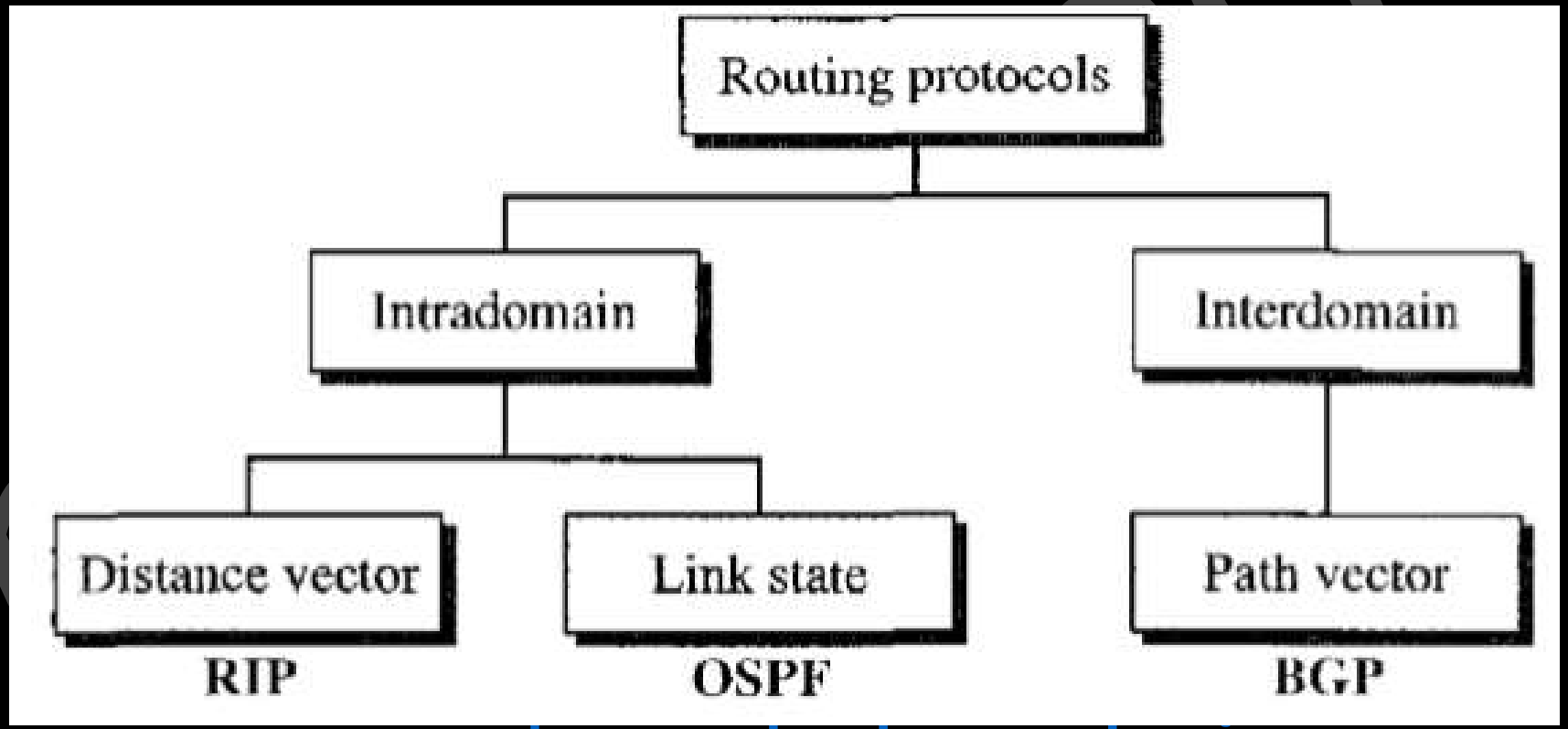


Class	Prefixes	First byte
A	$n = 8$ bits	0 to 127
B	$n = 16$ bits	128 to 191
C	$n = 24$ bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

www.knowledgegate.in

Network layer

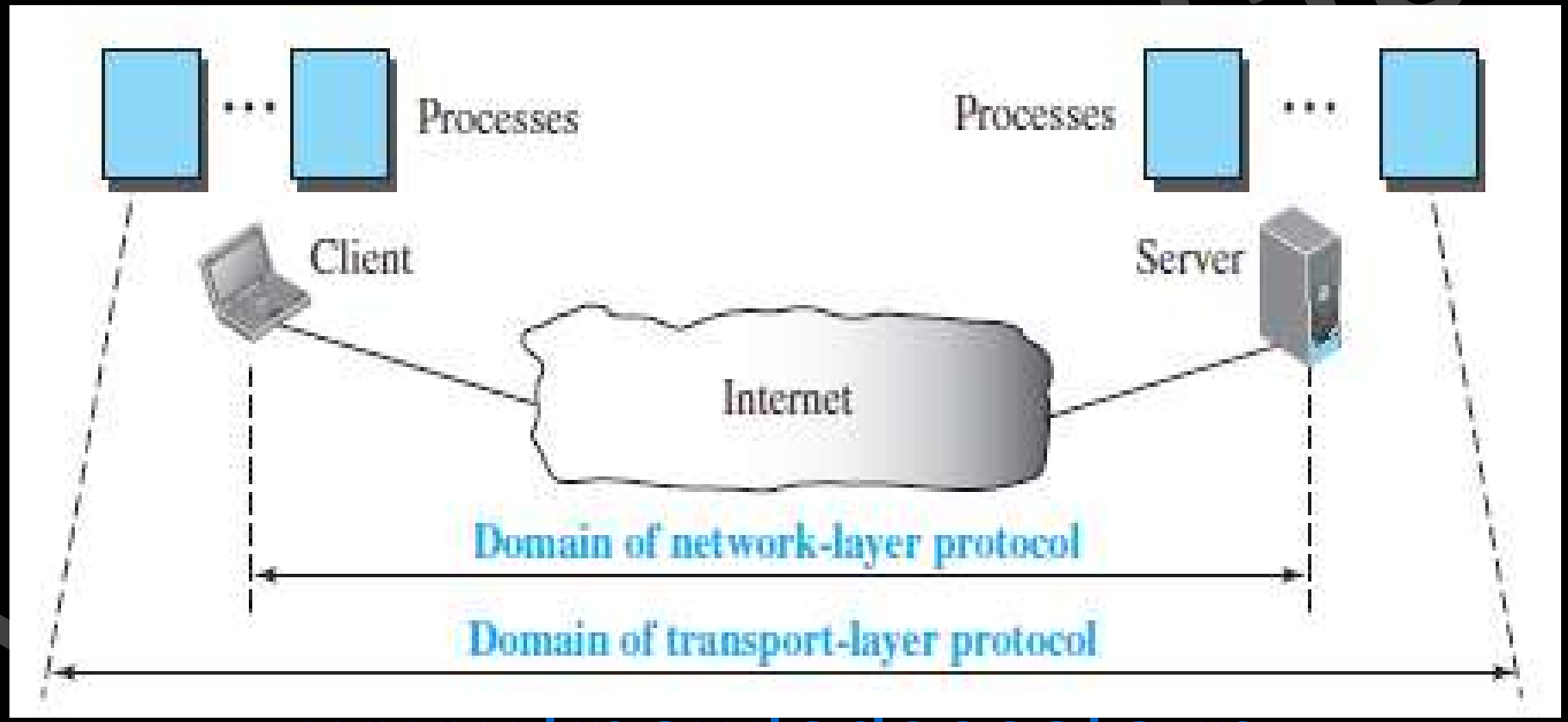
2. Routing: When independent networks or links are connected to create internetworks (network of networks) or a large network, the connecting devices (called routers or switches) route or switch the packets to their final destination. One of the functions of the network layer is to provide this mechanism.



www.knowledgegate.in

Transport layer

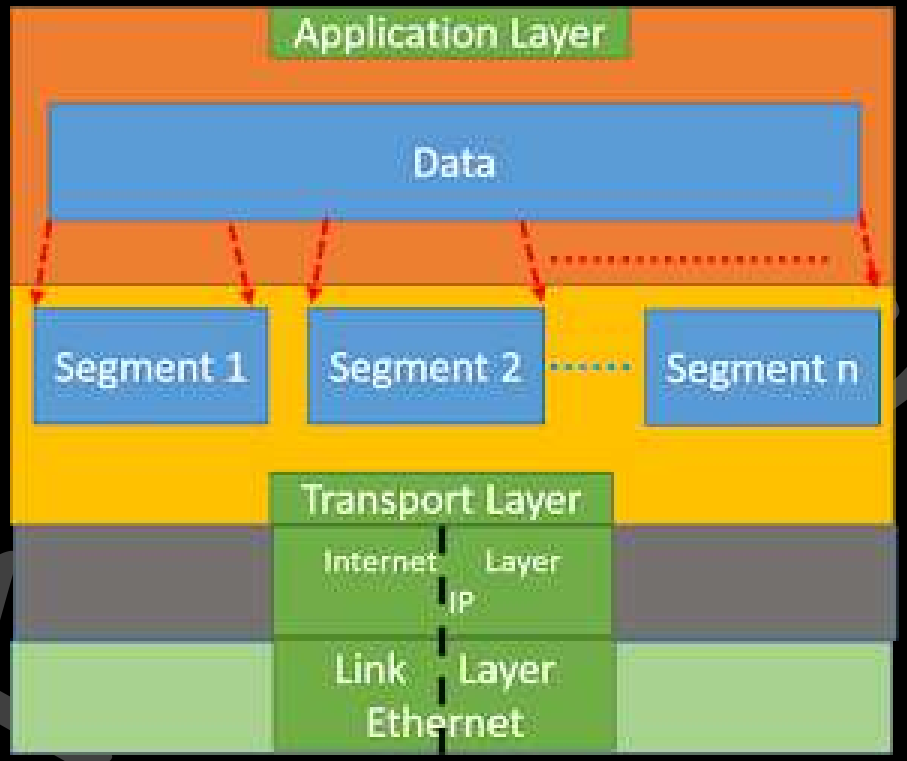
1. Service-point addressing: The transport layer header must include a type of address called a service-point address (or port address). The network layer gets each packet to the correct computer; the transport layer gets the entire message to the correct process on that computer. The transport layer is responsible for process-to-process delivery of the entire message.

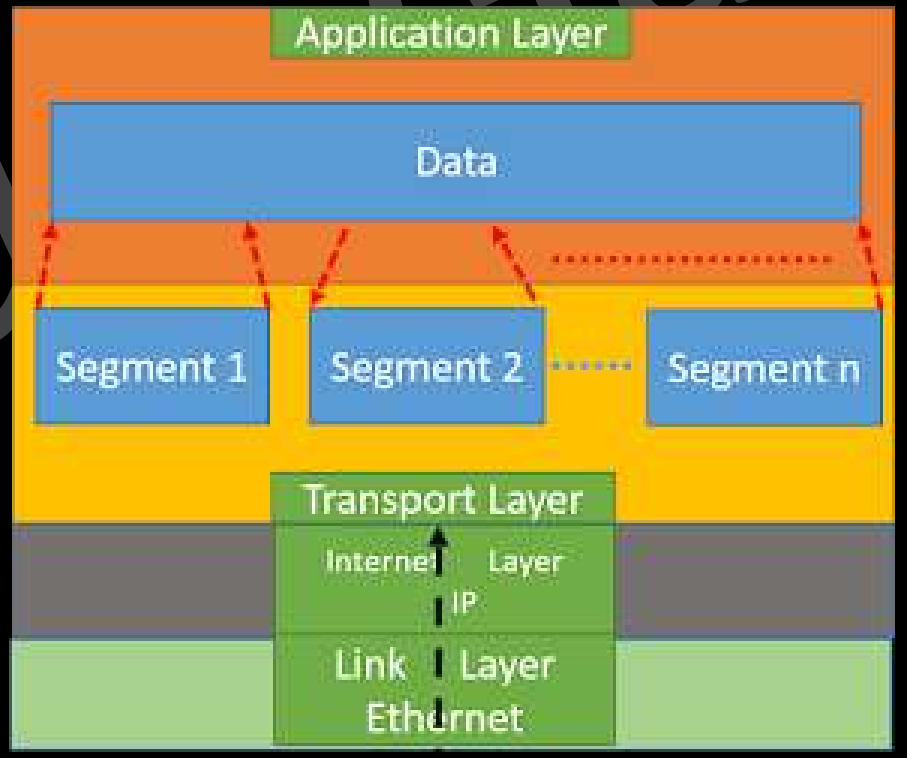


www.knowledgegate.in

Transport layer

2. Segmentation and reassembly: A message is divided into transmittable segments, with each segment containing a sequence number. These numbers enable the transport layer to reassemble the message correctly upon arriving at the destination and to identify and replace packets that were lost in transmission.

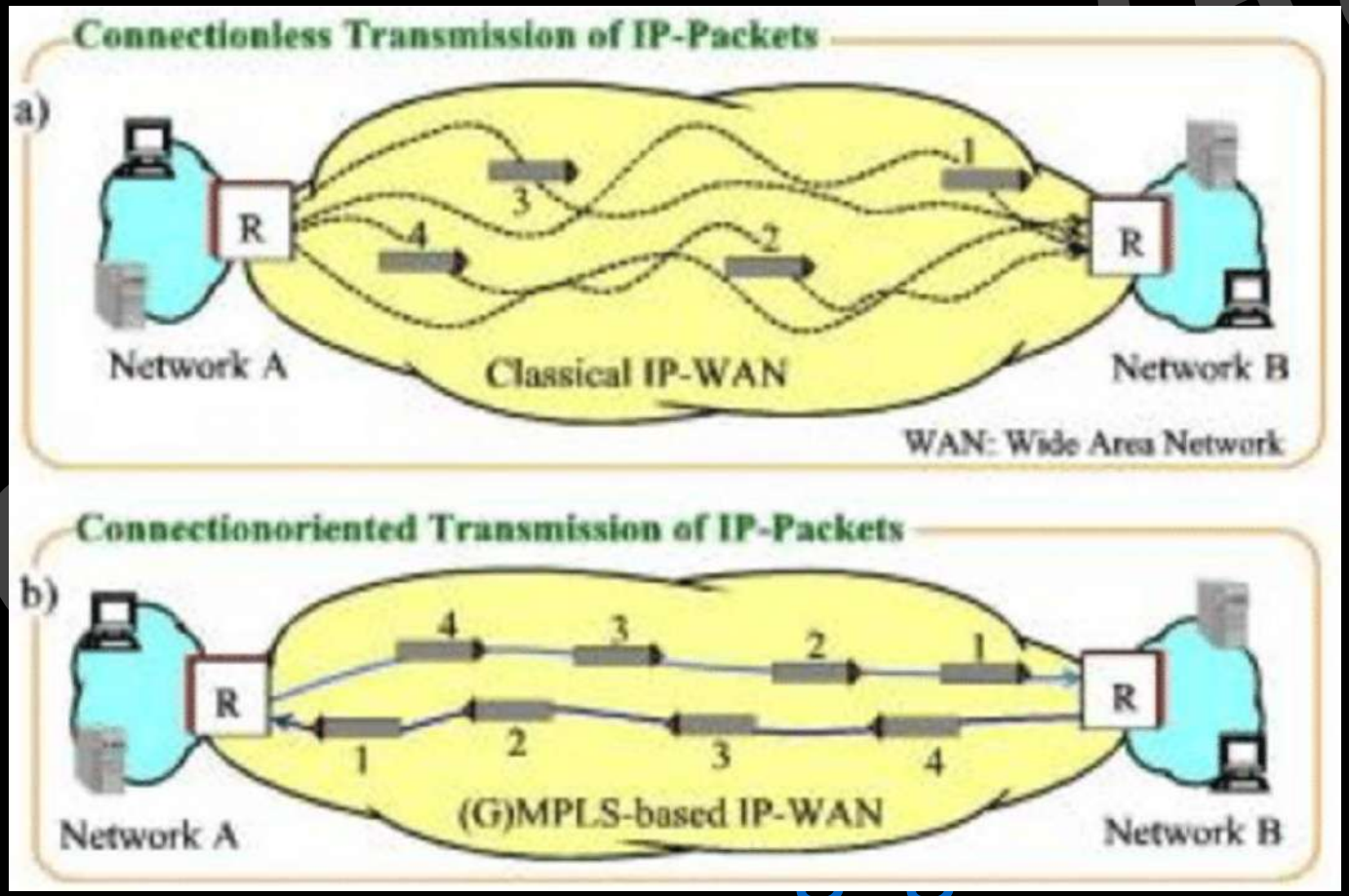




www.knowledgegate.in

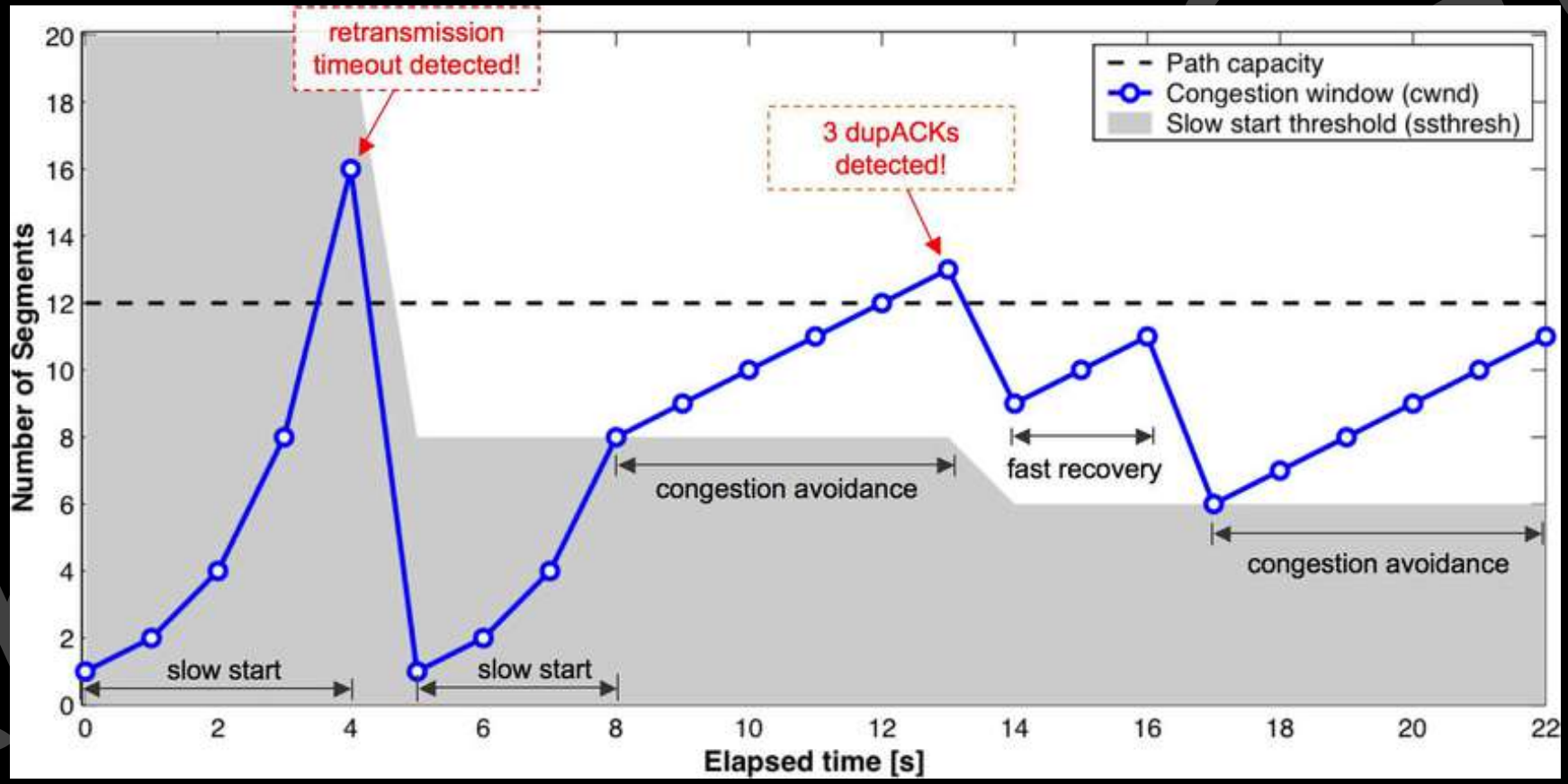
Transport layer

3. Connection control: The transport layer can be either connectionless or connection oriented. A connectionless transport layer treats each segment as an independent packet and delivers it to the transport layer at the destination machine. A connection-oriented transport layer makes a connection with the transport layer at the destination machine first before delivering the packets. After all the data are transferred, the connection is terminated.



Transport layer

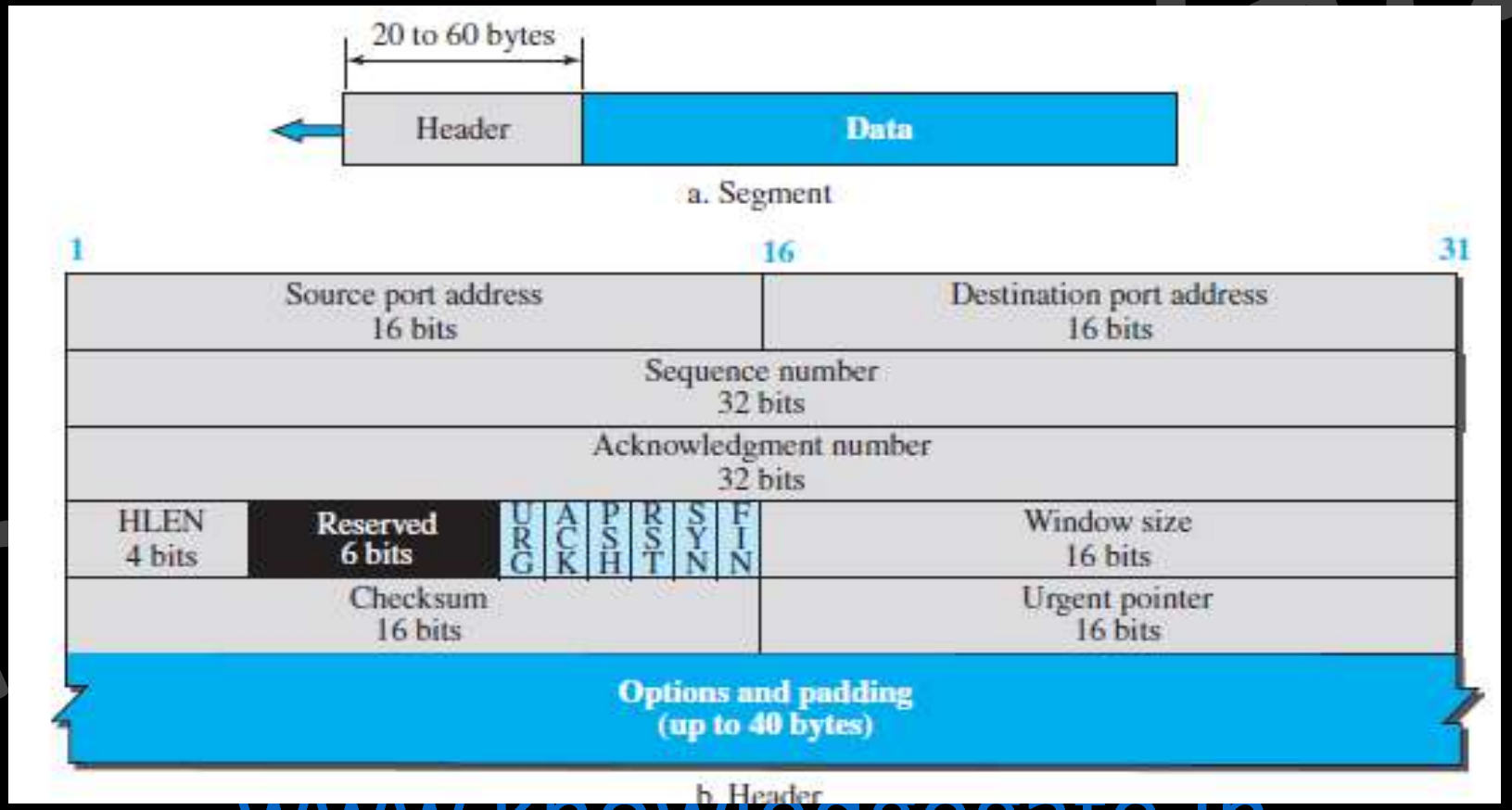
4. Flow control: Like the data link layer, the transport layer is responsible for flow control. However, flow control at this layer is performed end to end rather than across a single link.



www.knowledgegate.in

Transport layer

5. Error control: Like the data link layer, the transport layer is responsible for error control. However, error control at this layer is performed process-to process rather than across a single link. The sending transport layer makes sure that the entire message arrives at the receiving transport layer without error (damage, loss, or duplication). Error correction is usually achieved through retransmission.



www.knowledgegate.in

Session layer

- The session layer is the **network dialog controller**. It establishes, maintains, and synchronizes the interaction among communicating systems.

1. The session layer is responsible for **dialog control and synchronization**.

2. Dialog control: The session layer allows two systems to enter into a dialog. It allows the communication between two processes to take place in either half duplex (one way at a time) or full-duplex (two ways at a time) mode.

3. Synchronization: The session layer allows a process to add checkpoints, or synchronization points, to a stream of data.

KnowledgeGate

www.knowledgegate.in

Presentation layer

1. Translation: The processes (running programs) in two systems are usually exchanging information in the form of character strings, numbers, and so on.

- The information must be changed to bit streams before being transmitted. Because different computers use different encoding systems, the presentation layer is responsible for interoperability between these different encoding methods.
- The presentation layer at the sender changes the information from its sender-dependent format into a common format. The presentation layer at the receiving machine changes the common format into its receiver-dependent format.

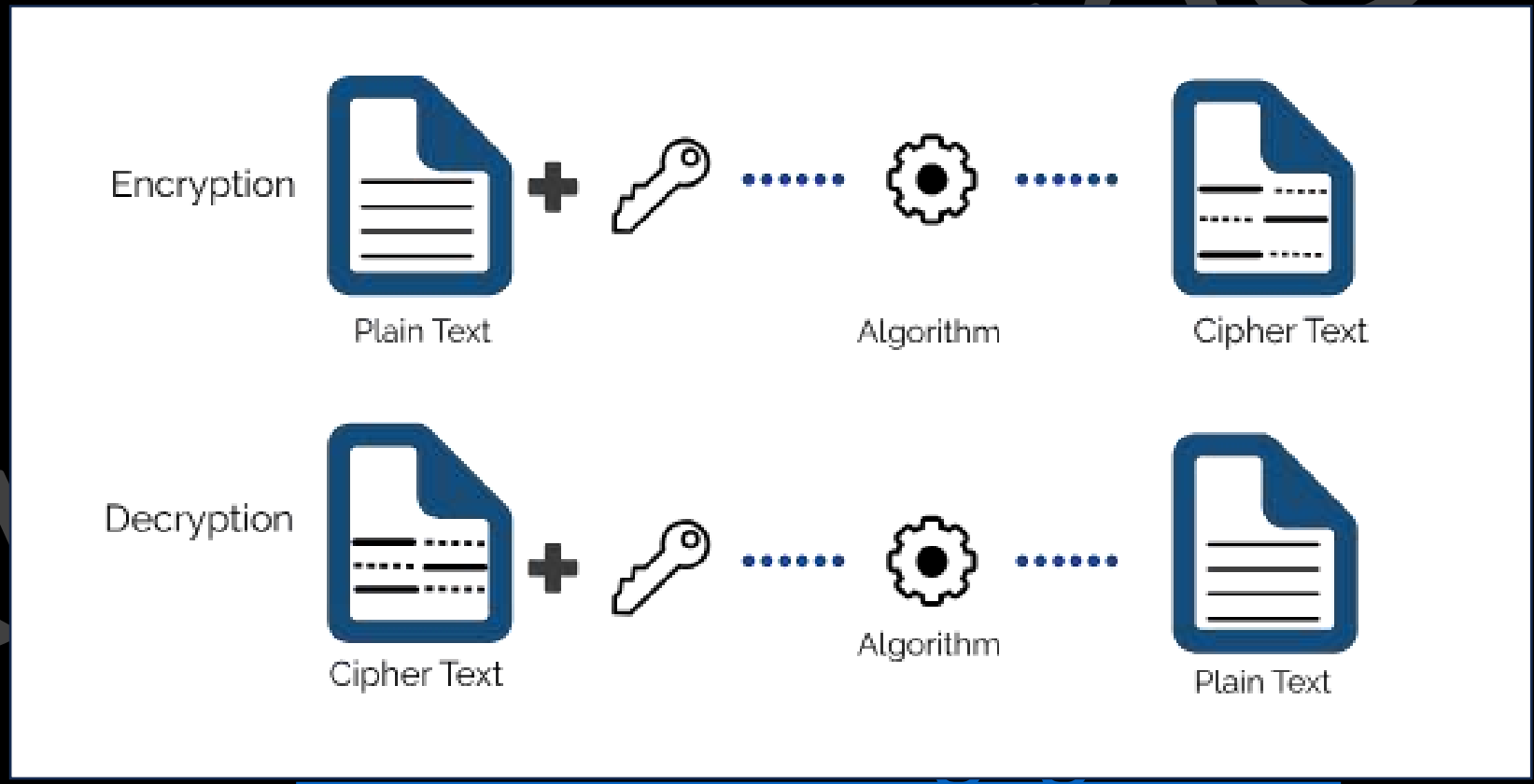
KnowledgeGate

www.knowledgegate.in

Presentation layer

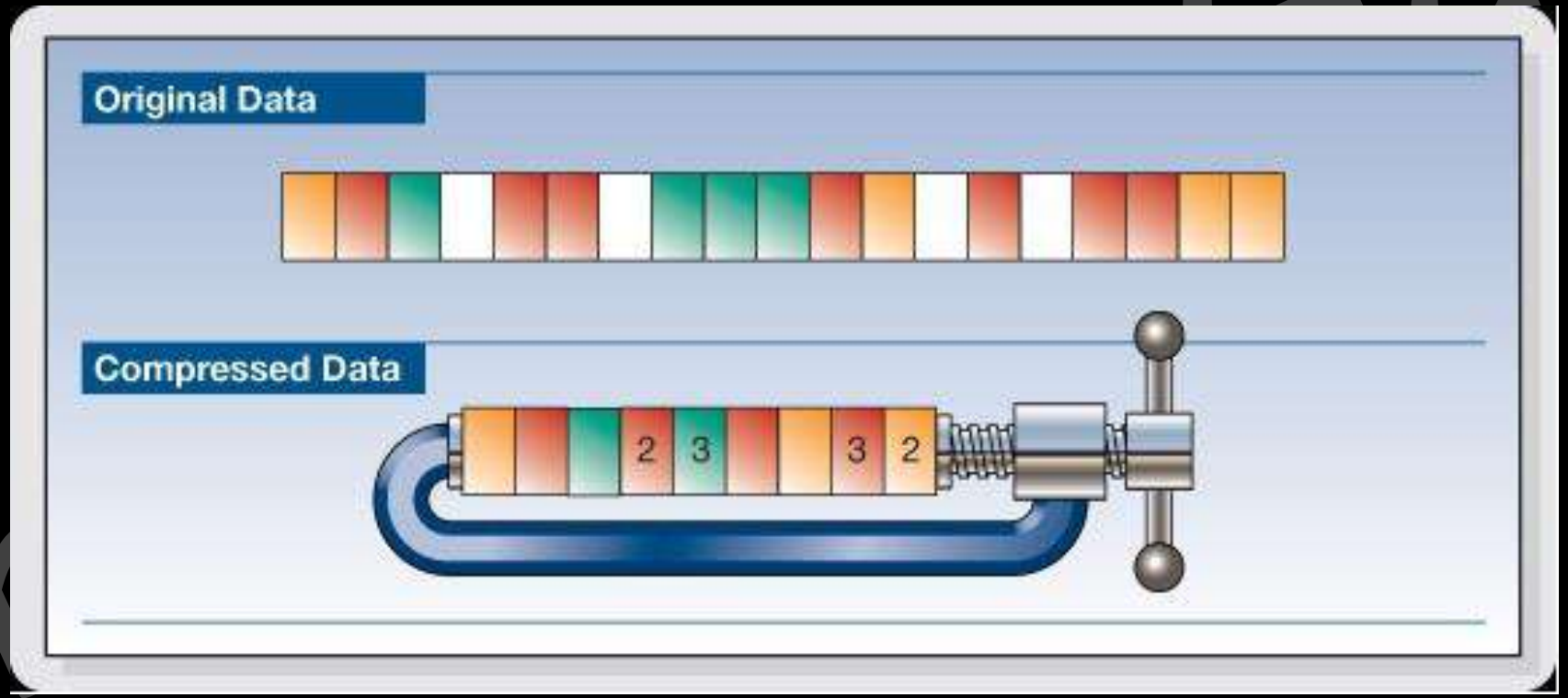
2. Encryption: To carry sensitive information, a system must be able to ensure privacy. Encryption means that the sender transforms the original information to another form and sends the resulting message out over the network.

- Decryption reverses the original process to transform the message back to its original form.



Presentation layer

3. Compression: Data compression reduces the number of bits contained in the information. Data compression becomes particularly important in the transmission of multimedia such as text, audio, and video.



www.knowledgegate.in

Application layer

- The application layer enables the user, whether human or software, to access the network.
- It provides user interfaces and support for services such as electronic mail, remote file access and transfer, shared database management, and other types of distributed information services.

Services

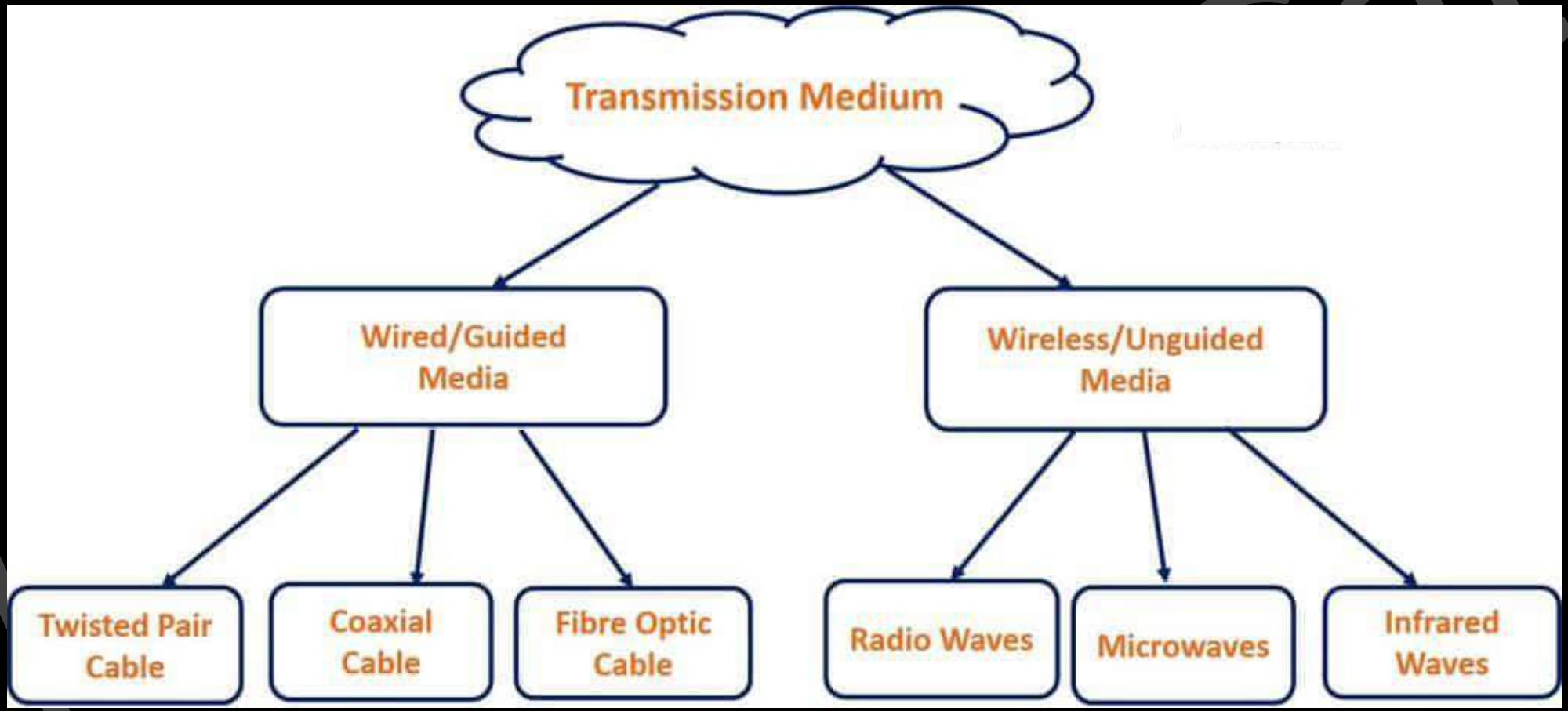
1. **Network virtual terminal**: A network virtual terminal is a software version of a physical terminal, and it allows a user to log on to a remote host. To do so, the application creates a software emulation of a terminal at the remote host. The user's computer talks to the software terminal which, in turn, talks to the host, and vice versa. The remote host believes it is communicating with one of its own terminals and allows the user to log on.
2. **File transfer, access, and management**: This application allows a user to access files in a remote host (to make changes or read data), to retrieve files from a remote computer for use in the local computer, and to manage or control files in a remote computer locally.
3. **Mail services**: This application provides the basis for e-mail forwarding and storage.
4. **Directory services**: This application provides distributed database sources and access for global information about various objects and services.

KnowledgeGate

www.knowledgegate.in

Transmission media

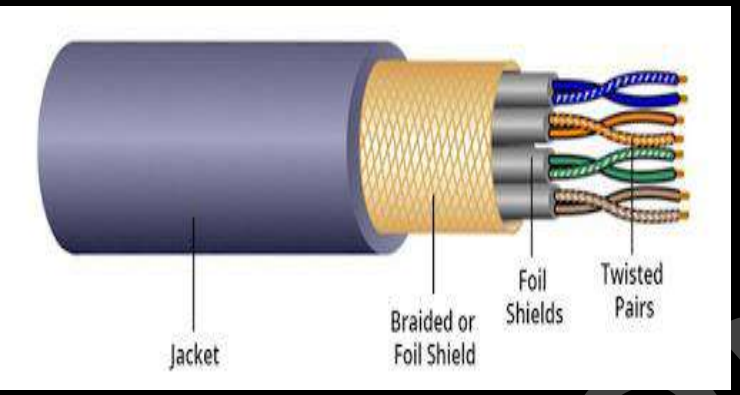
- (layer-0) can broadly be defined anything that can carry information from source to destination.



www.knowledgegate.in

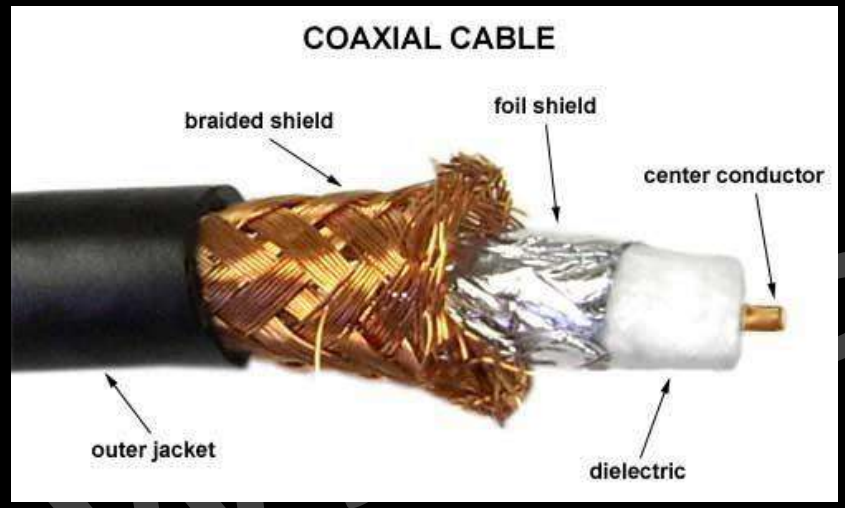
Guided media

- Are those which provide a connection from one device to another.
- **Twisted pair cable** - Consists of two conductors (copper), each with it's own plastic insulation, twisted together. (shielded and unshielded twisted pair of cables)(telephone line)



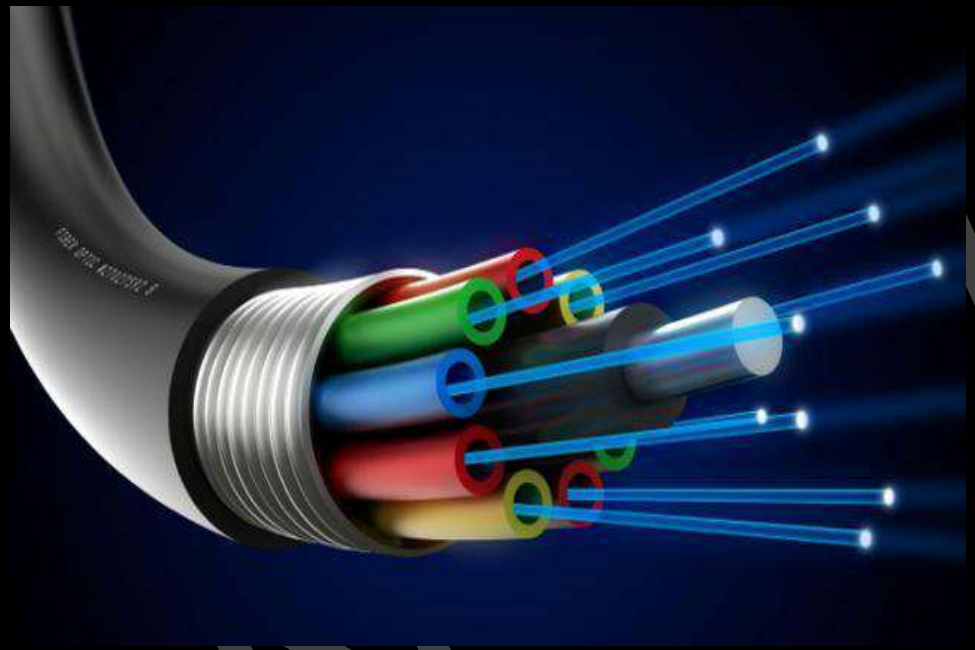
KnowledgeGate

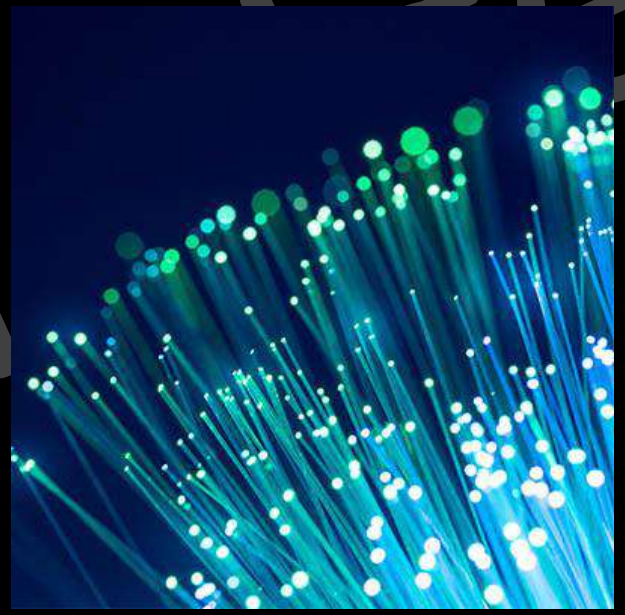
- **Coaxial Cable** – Has a central core conductor of solid wire enclosed in an insulating sheath, which in turn, encased in an outer conductor of metal foil, braid or a combination of two. (Cable tv)



www.knowledgegate.in

- **Fibre optic** - Made of glass or plastic and transmit signal in the form of light, using the principle of total internal reflection, a glass or plastic core is surrounded by a cladding of less dense glass or plastic.
- Backbone network cost effective can go up to 1600 Gbps (higher bandwidth, less signal attenuation, no noise problem, no corrosion, light weight, greater immunity to tapping) (installation and maintenance, unidirectional light propagation, cost)

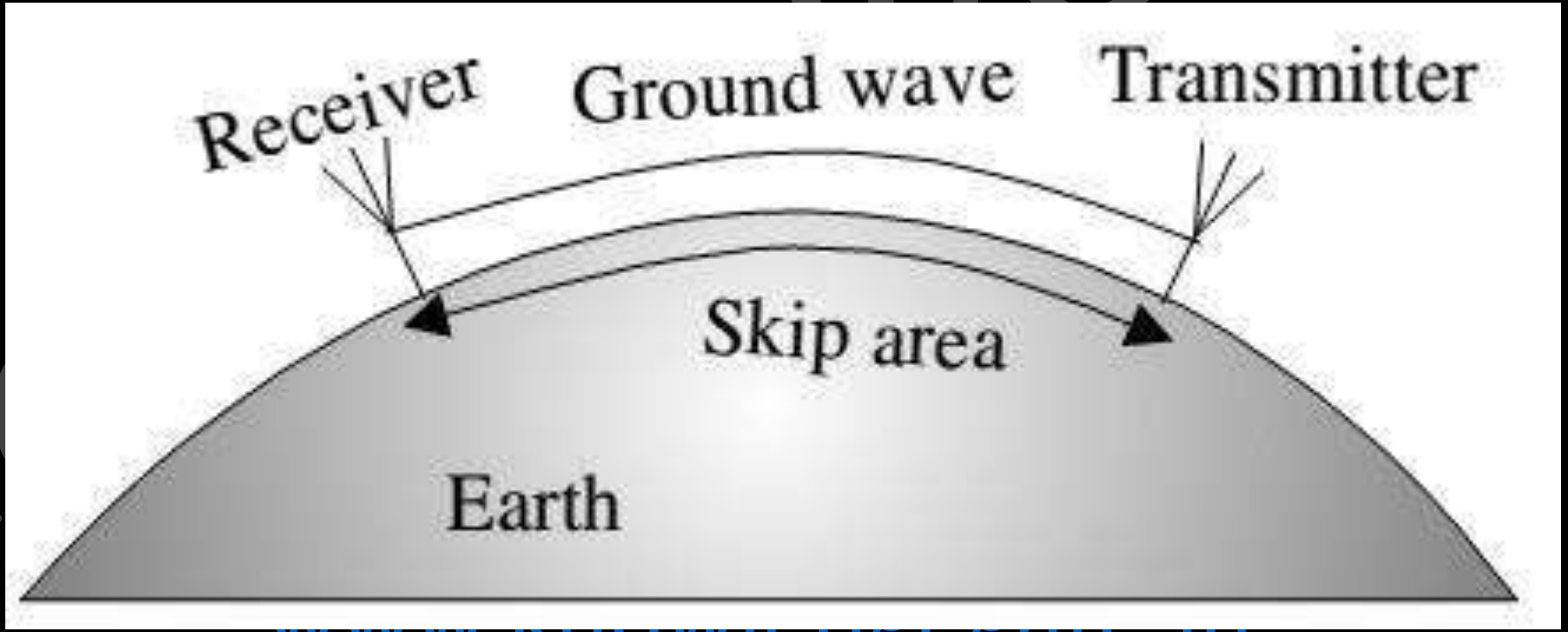




www.knowledgegate.in

Unguided Media: Wireless

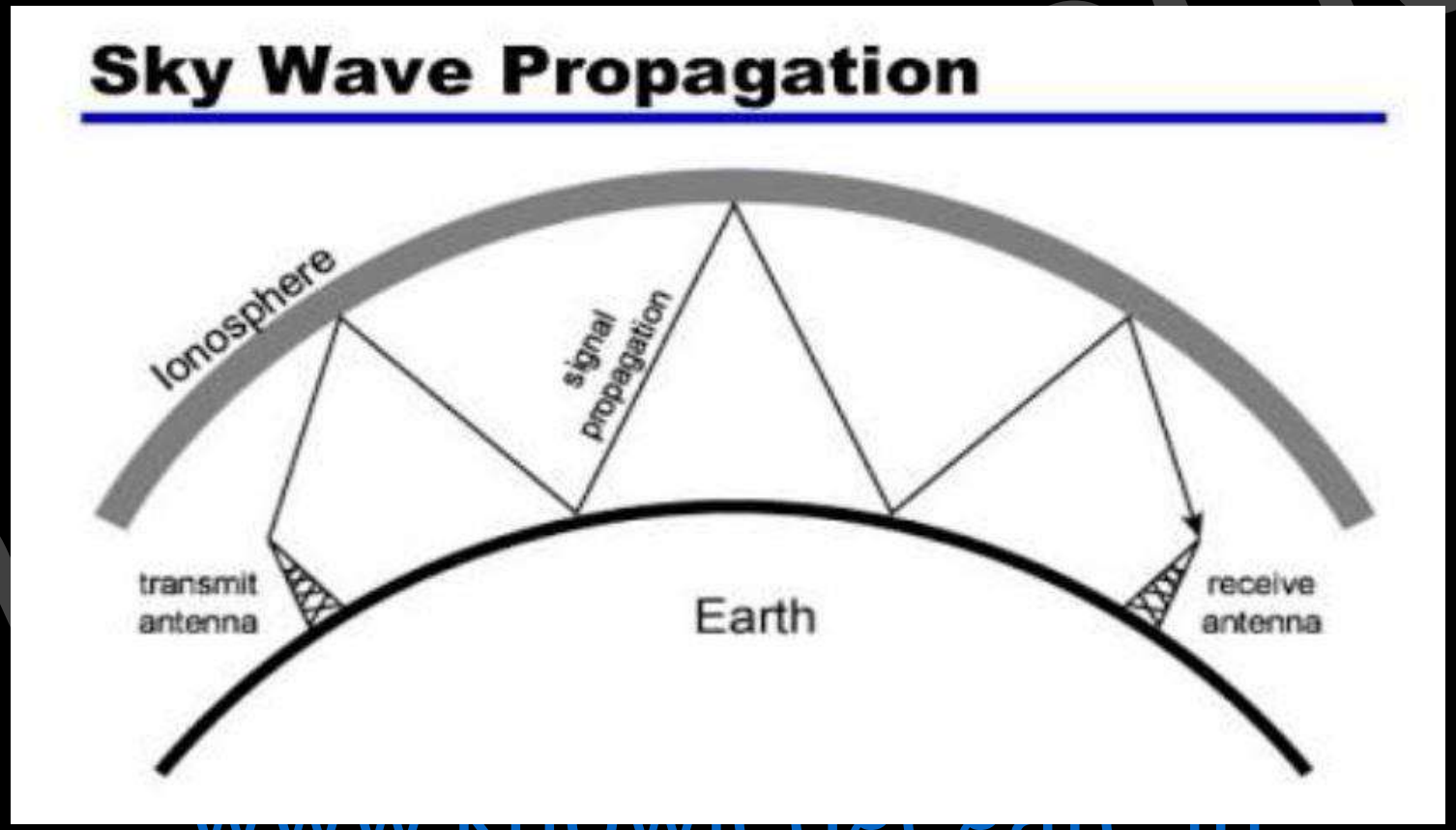
- **Ground propagation** – Waves travel through lower portion of the atmosphere hugging the earth, they are omni directional, distance depends on the amount of power.
- Will having low frequency and large wave length (Khz - Mhz)
- Bend round the obstructions, because large of Wave Length(e.g. light and sound)
- Attenuate in short range.



www.knowledgegate.in

Unguided Media: Wireless

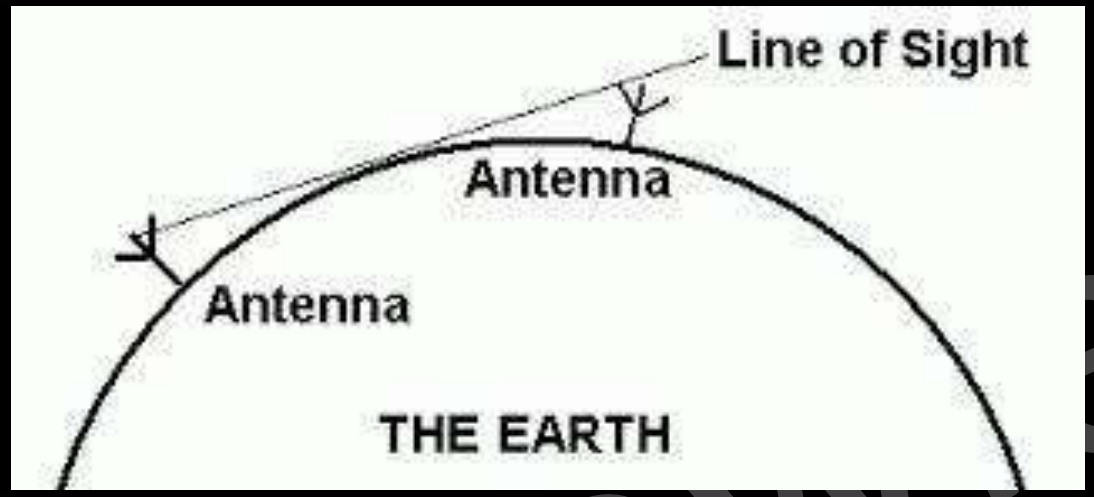
- **Sky propagation** – high frequency radio waves, radiated upward into the ionosphere where they are reflected back to earth. greater distance with lower output.
- 3 Mhz to 32 Mhz
- Range go up to 5000 km

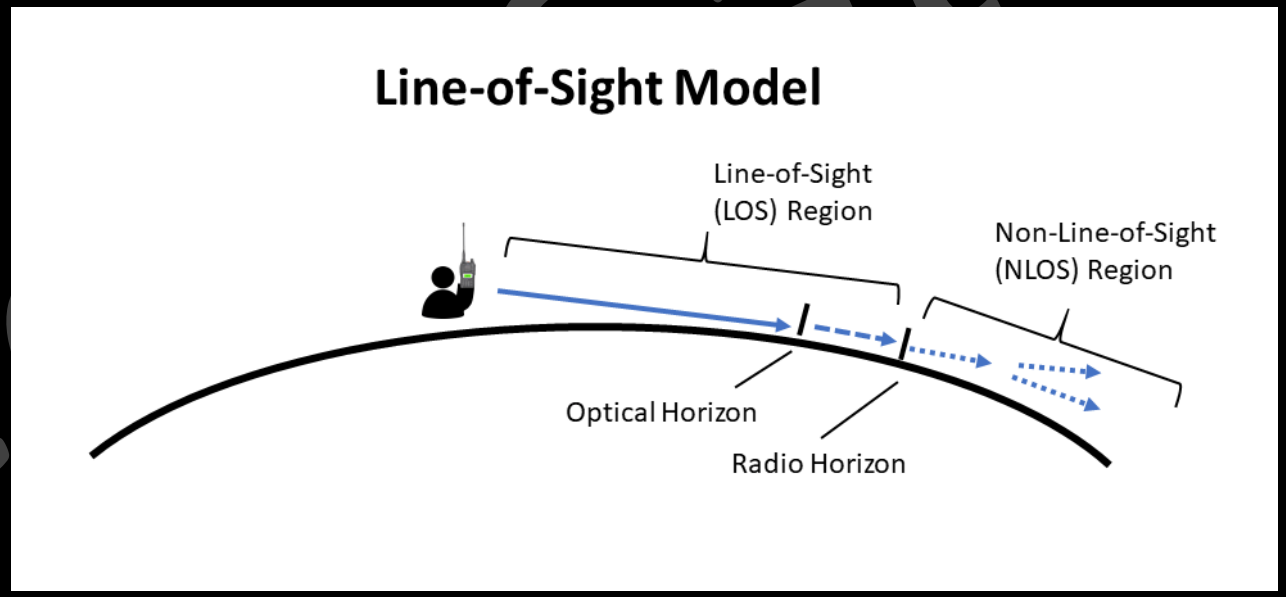


KnowledgeGate
www.knowledgegate.in

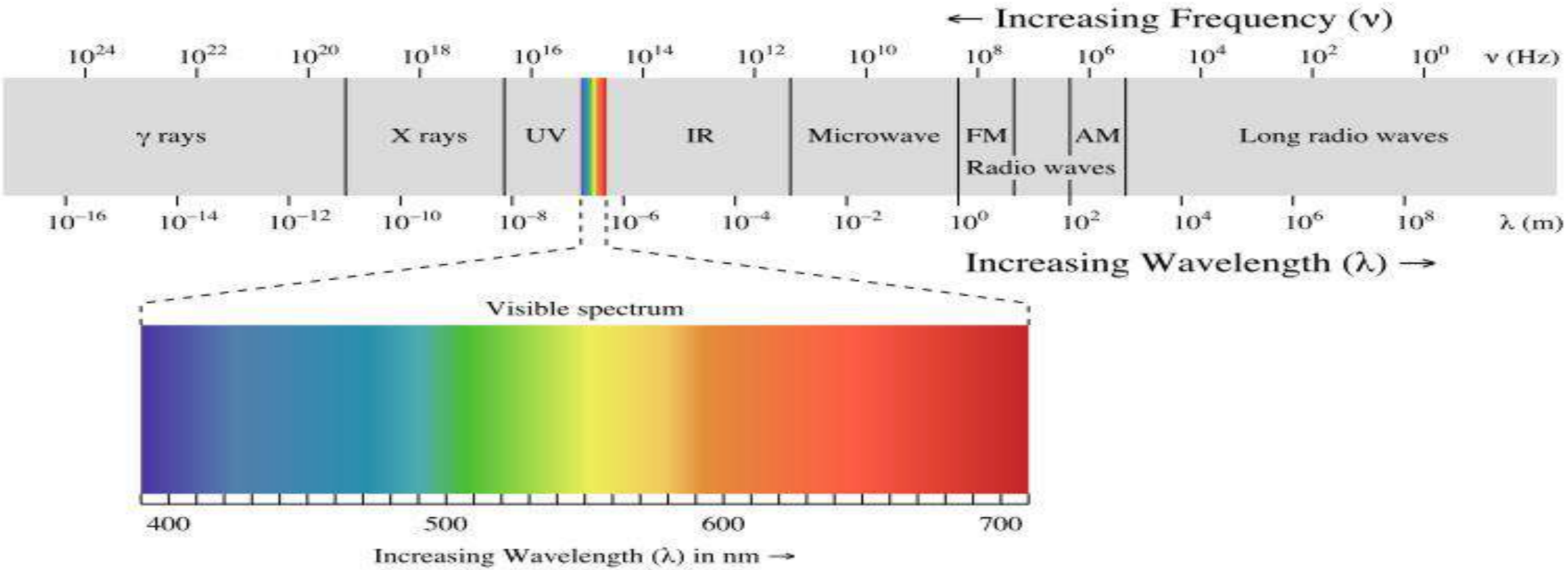
Unguided Media: Wireless

- **Line of sight propagation** – Very high frequency signals transmitted in straight lines directly from antenna to antenna.



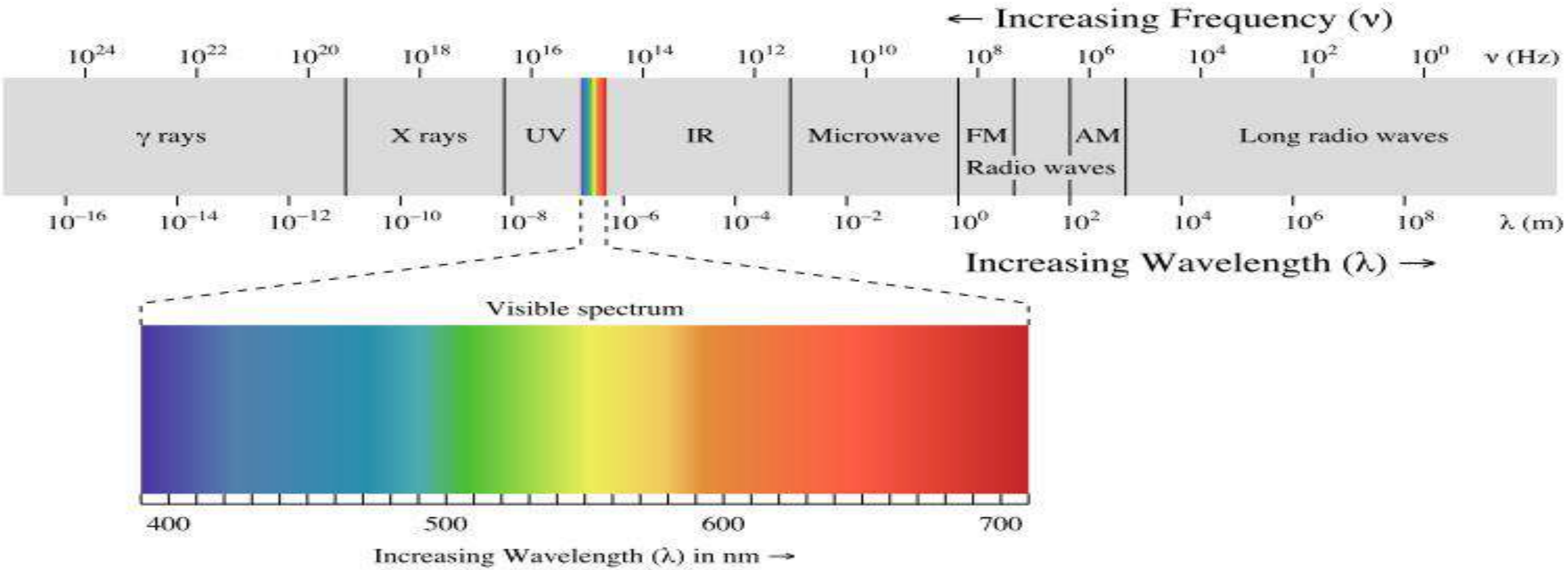


Knowledge Gate



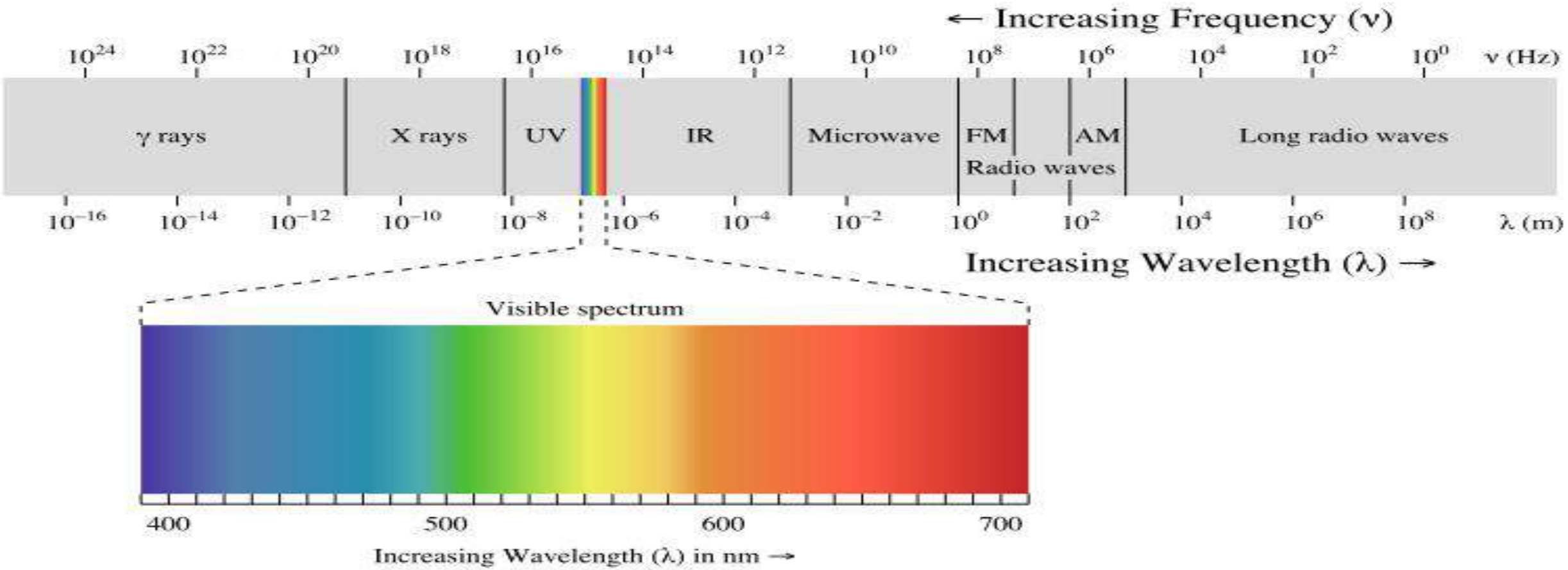
- **<u>Radio Waves</u>** – (3KHz- 1GHz) (omnidirectional) (interference problem because of omnidirectional) (sky mode) (long distance) (AM radio) (can penetrate through wall) (managed by government)

www.knowledgegate.in



- **Microwaves** – (1GHZ- 300GHz) (unidirectional) (can be focused narrowly) (sending and receiving antenna needed to be aligned) (cannot penetrate wall) (wide band so high data rates are possible) (managed by government)

www.knowledgegate.in

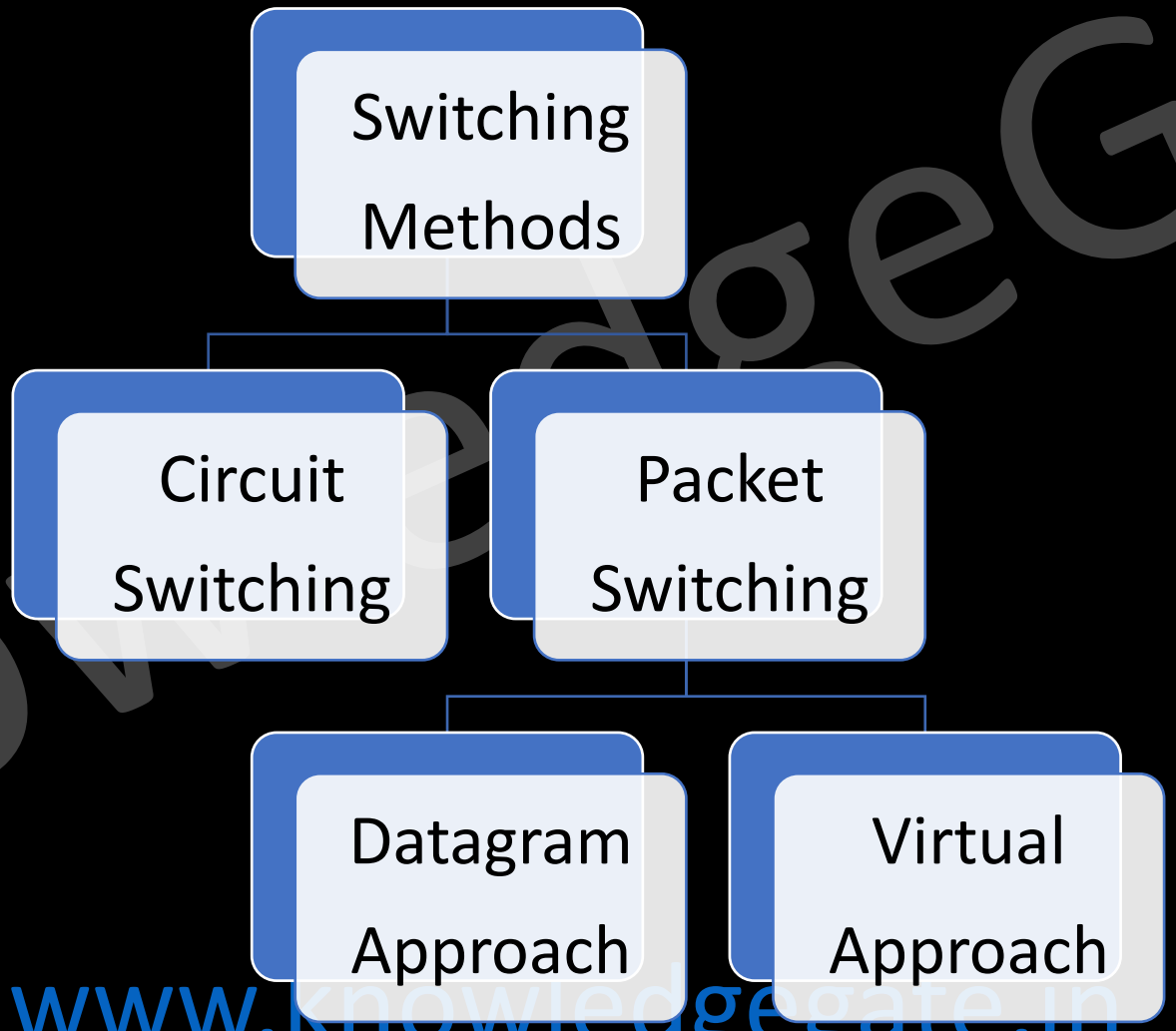


- **Infrared** – (300GHz- 400THz) (short range communication) (home appliances)

www.knowledgegate.in

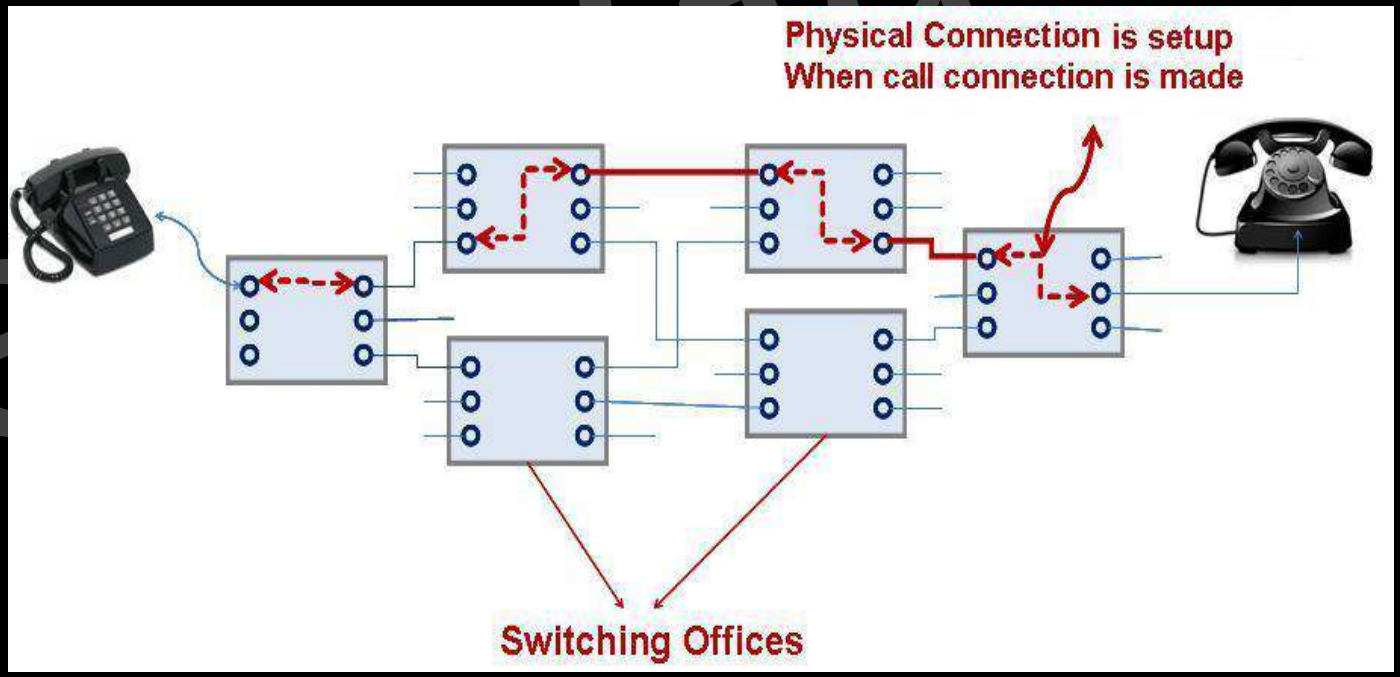
Switching

- Switching is the technique by which nodes control or switch data to transmit it between specific points on a network.



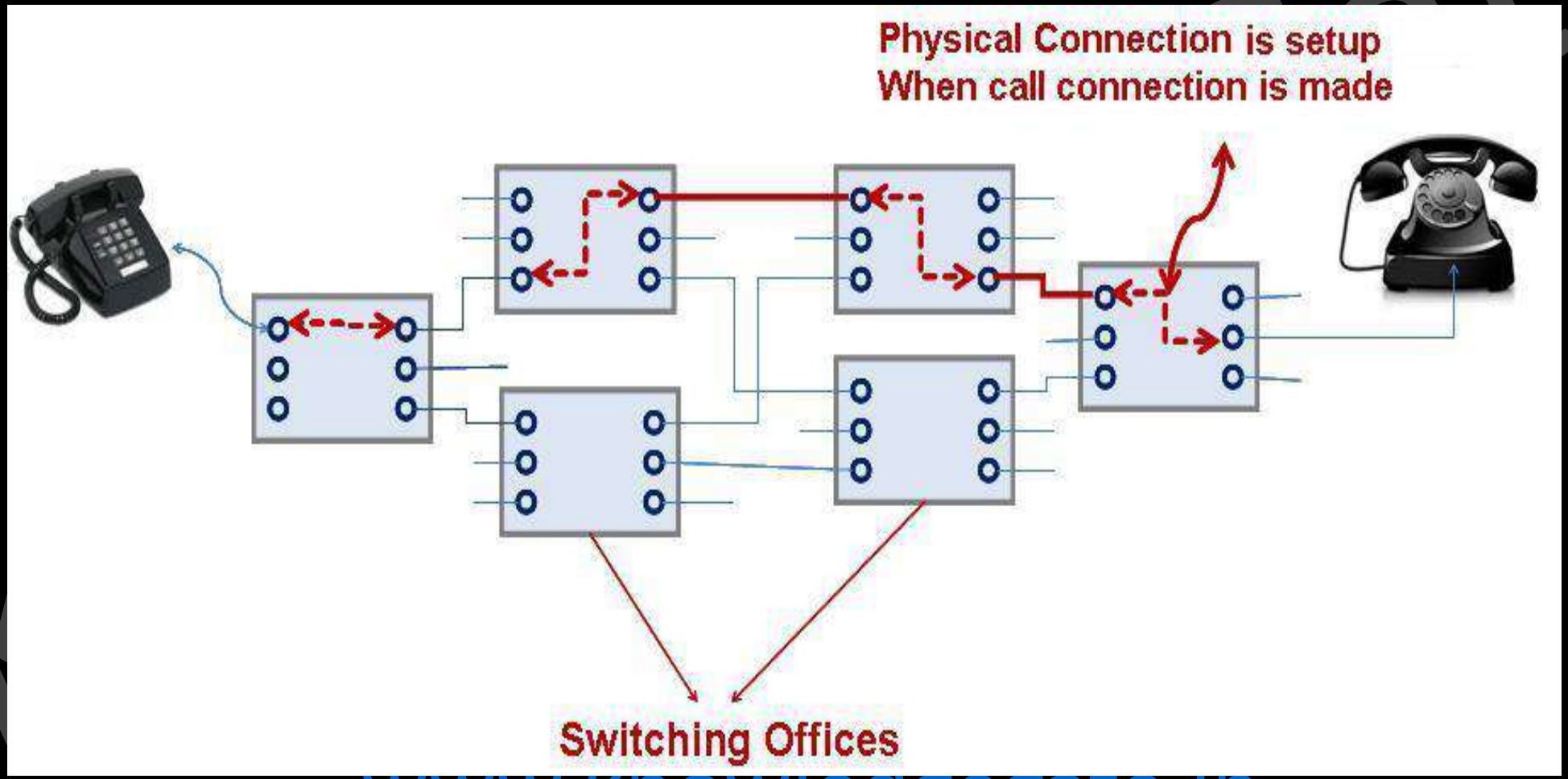
KnowledgeGate
www.knowledgegate.in

- In circuit switching network resources (bandwidth) is divided into pieces and bit delay is constant during a connection.
- The dedicated path/circuit established between sender and receiver provides a guaranteed data rate. Data can be transmitted without any delays once the circuit is established. Telephone system network is the one of example of Circuit switching.



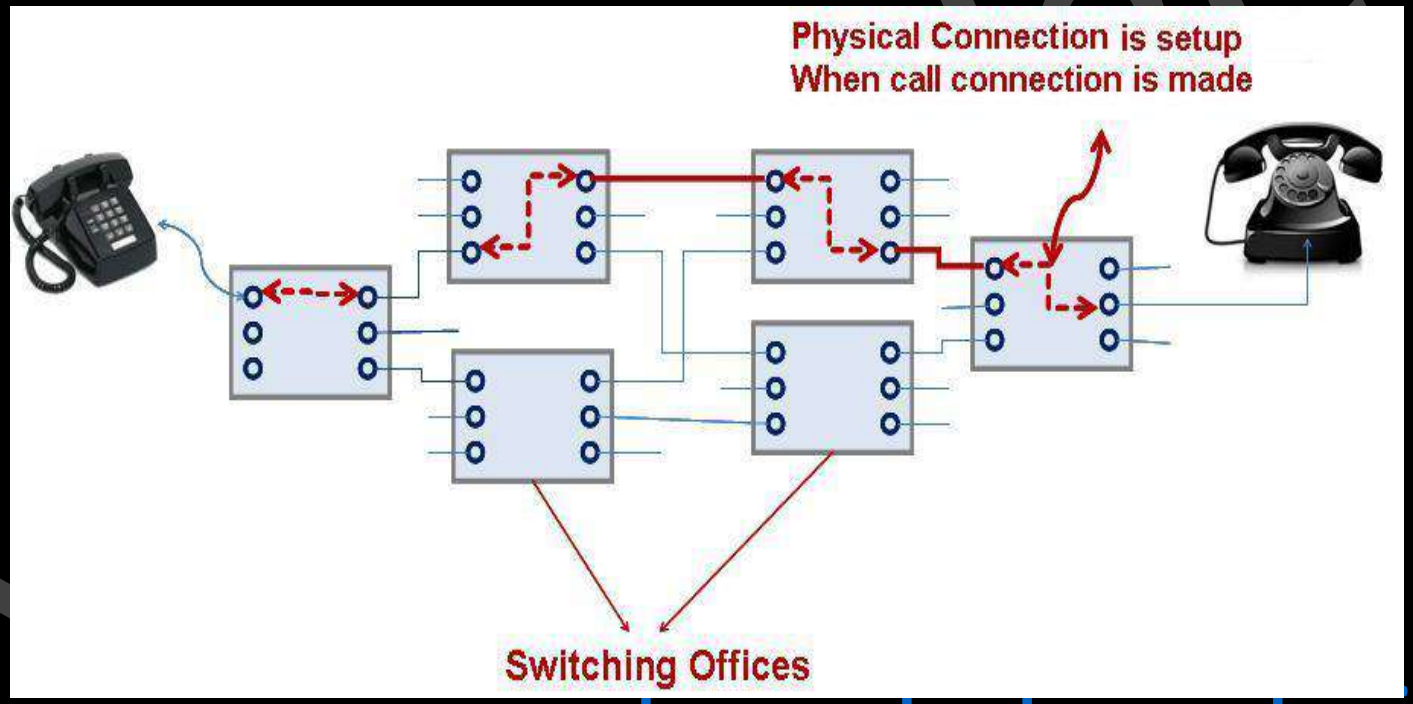
www.knowledgegate.in

- Telephone system network is the one of example of Circuit switching.
  - **TDM (Time Division Multiplexing)**
  - **FDM (Frequency Division Multiplexing)**
- are two methods of multiplexing multiple signals into a single carrier.



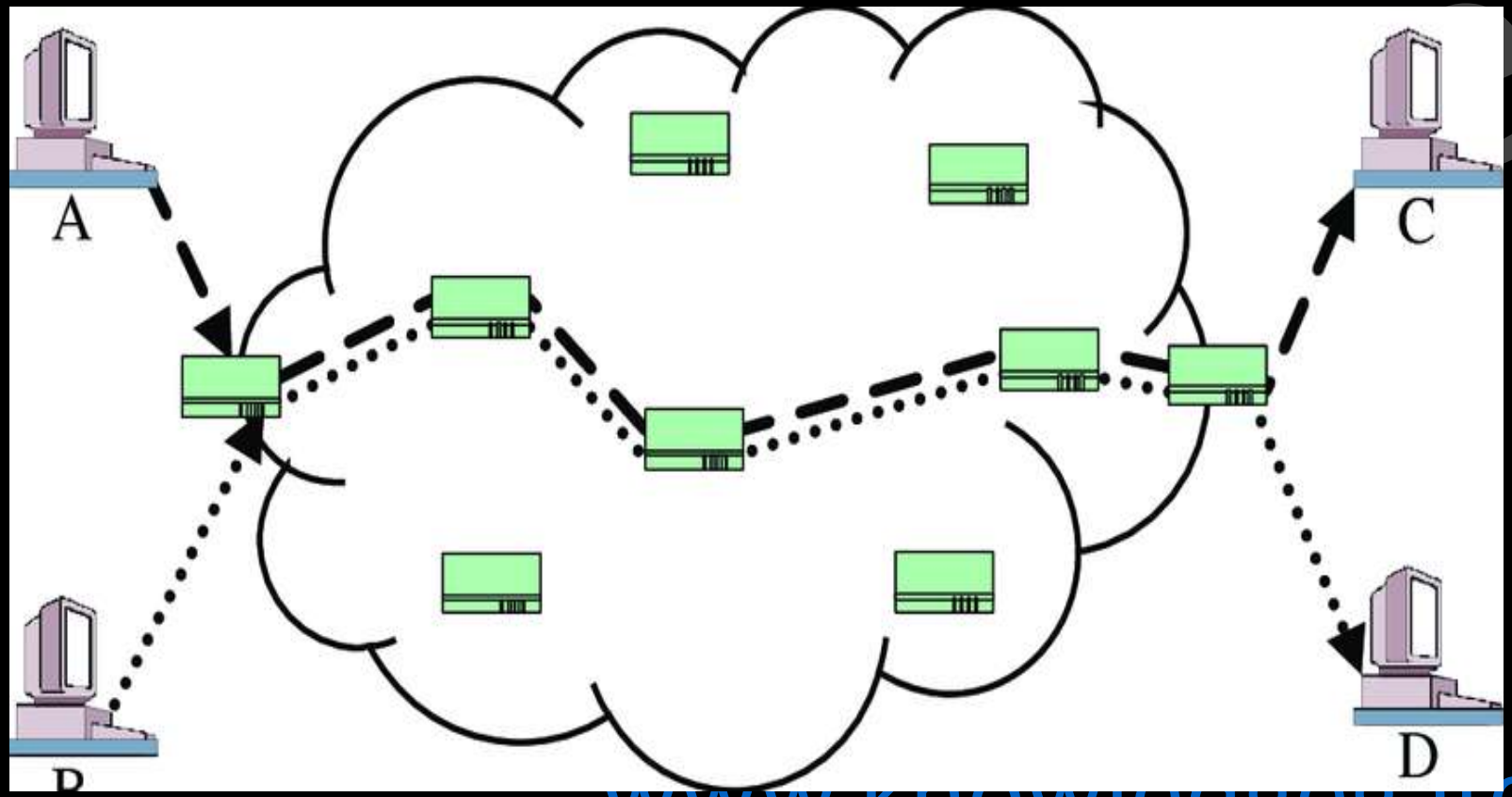
www.knowledgegate.in

- **Time Division Multiplexing :** *Divides into frames*
  - Time-division multiplexing (TDM) is a method of transmitting and receiving independent signals over a common signal path by means of synchronized switches at each end of the transmission line.
  - TDM is used for long-distance communication links and bears heavy data traffic loads from end user.
  - Time division multiplexing (TDM) is also known as a digital circuit switched.



KnowledgeGate
www.knowledgegate.in

- **Frequency Division Multiplexing :** *Divides into multiple bands*
  - Frequency Division Multiplexing or FDM is used when multiple data signals are combined for simultaneous transmission via a shared communication medium.
  - It is a technique by which the total bandwidth is divided into a series of non-overlapping frequency sub-bands, where each sub-band carry different signal. Practical use in radio spectrum & optical fiber to share multiple independent signals.



www.knowledgegate.in

- **Advantages of Circuit Switching:** It has the following advantages :
  - The main advantage of circuit switching is that a committed transmission channel is established between the computers which gives a guaranteed data ratee.
  - In circuit switching there is no delay in data flow because of the dedicated transmission path.
  - No Header is required
  - Reordering of data cannot happen.

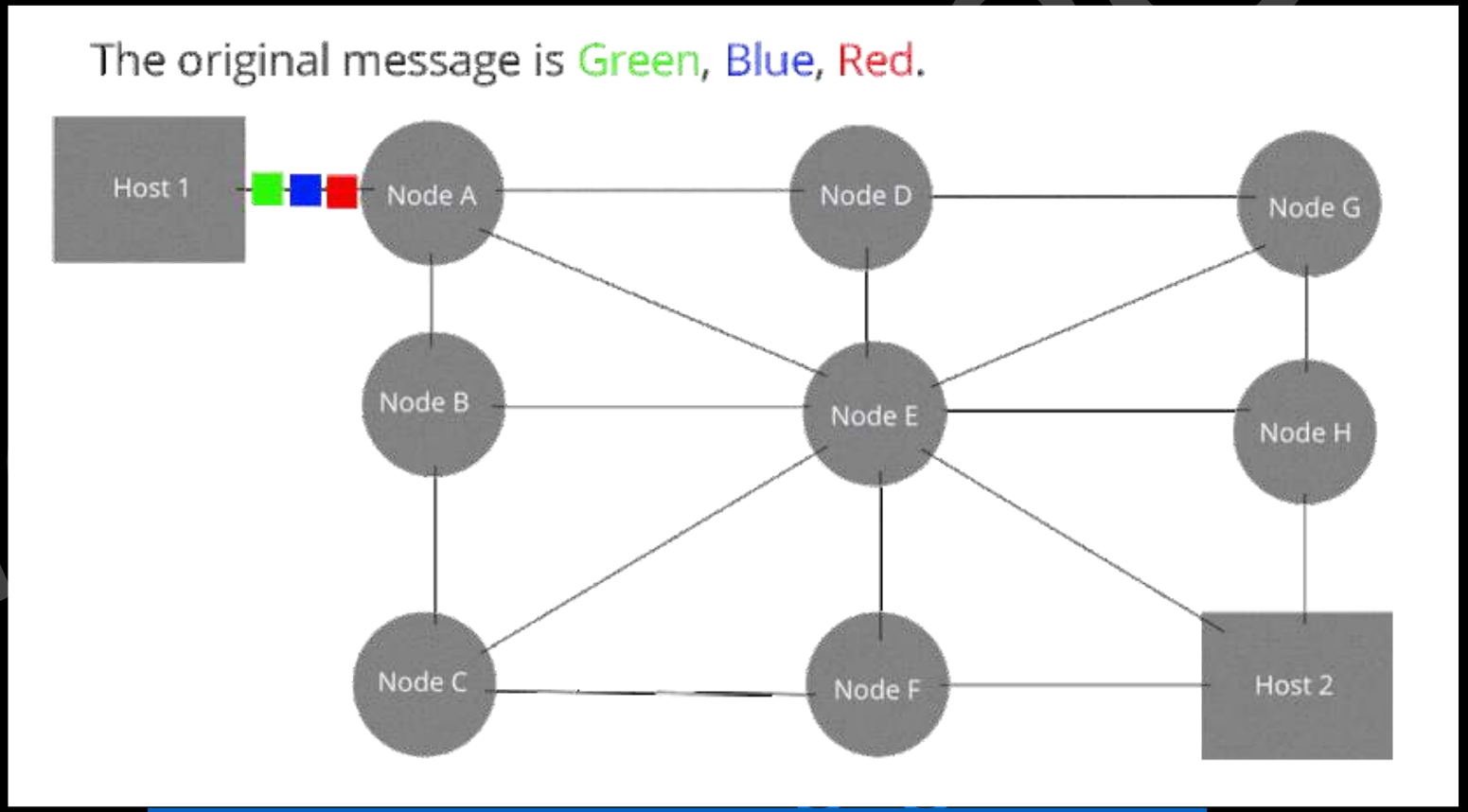
- **Disadvantages of Circuit Switching:** It has the following disadvantages :
  - It takes long time to establish connection.
  - More bandwidth is required in setting up of dedicated channels.
  - It cannot be used to transmit any other data even if the channel is free as the connection is dedicated in circuit switching.
  - Outdated, not used now a days

KnowledgeGate

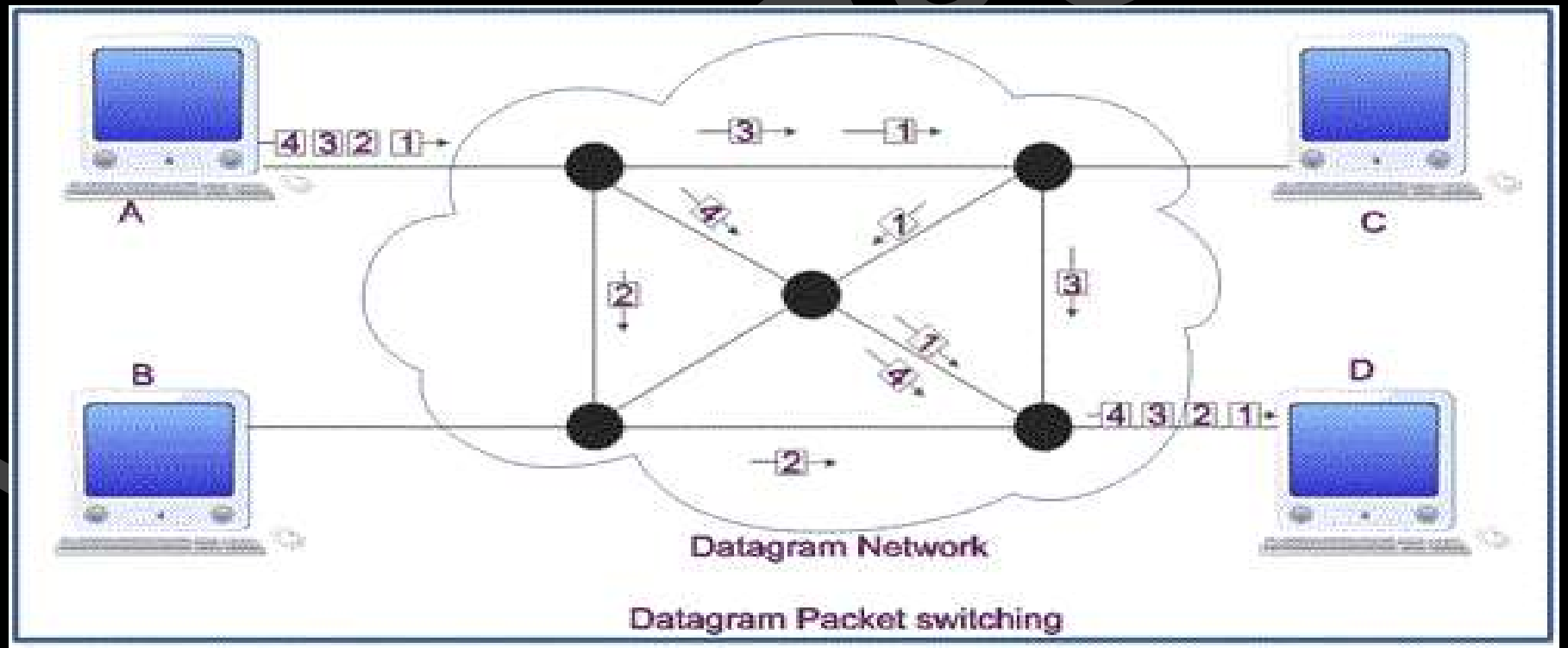
www.knowledgegate.in

Packet Switching

- **Datagram network** - If the message is going to pass through a packet-switched network, it needs to be divided into packets of fixed or variable size. The size of the packet is determined by the network and the governing protocol.
- In packet switching, there is no resource allocation for a packet. This means that there is no reserved bandwidth on the links, and there is no scheduled processing time for each packet. Resources are allocated on demand. The allocation is done on a first come, first-served basis.



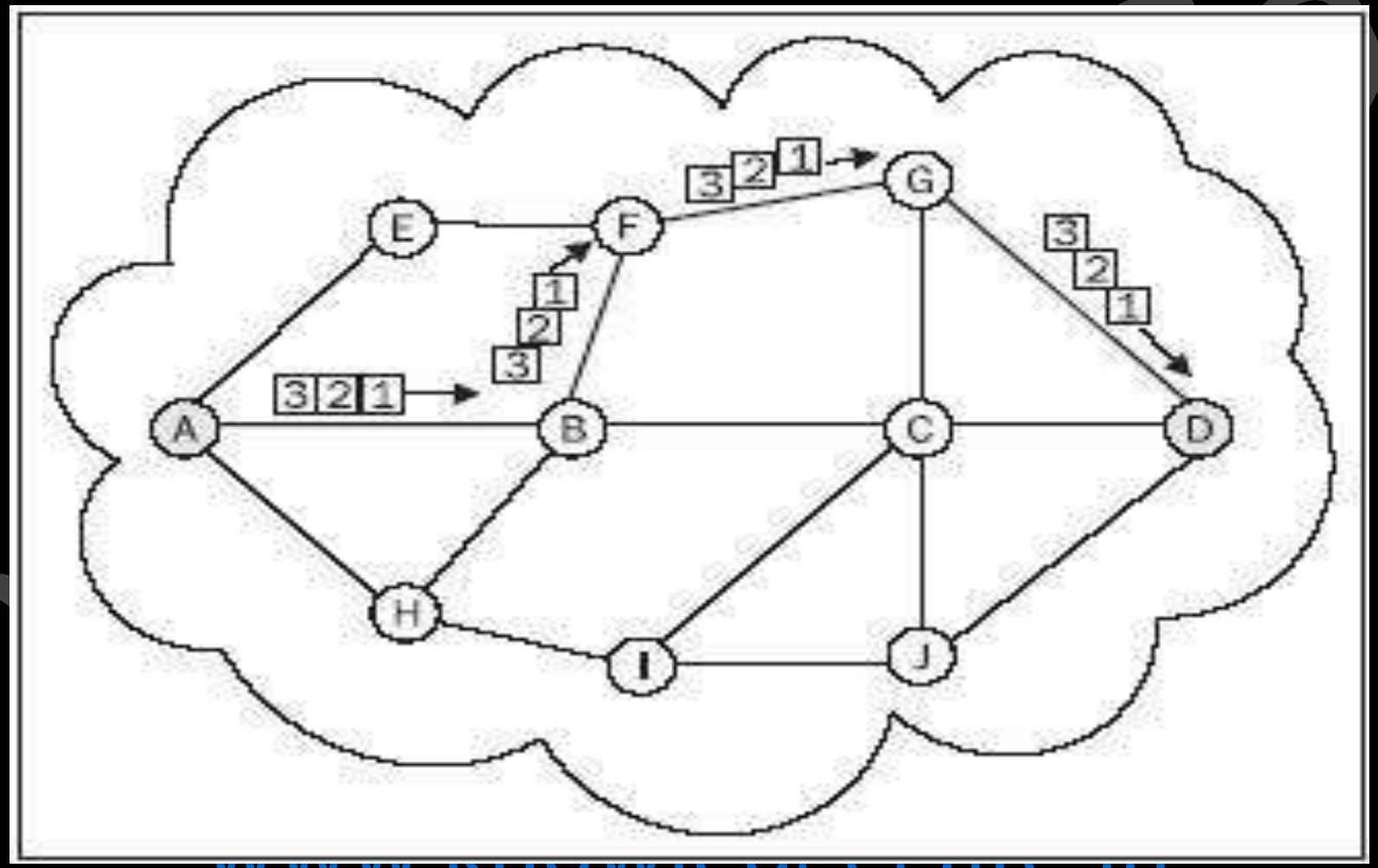
- When a switch receives a packet, no matter what is the source or destination, the packet must wait if there are other packets being processed. As with other systems in our daily life, this lack of reservation may create delay.
- Even if a packet is part of a multipacket transmission, the network treats it as though it existed alone. Packets in this approach are referred to as datagrams. Packets may also be lost or dropped because of a lack of resources.
- In most protocols, it is the responsibility of an upper-layer protocol to reorder the datagrams or ask for lost datagrams before passing them on to the application.



Datagram Packet switching

KodeGate

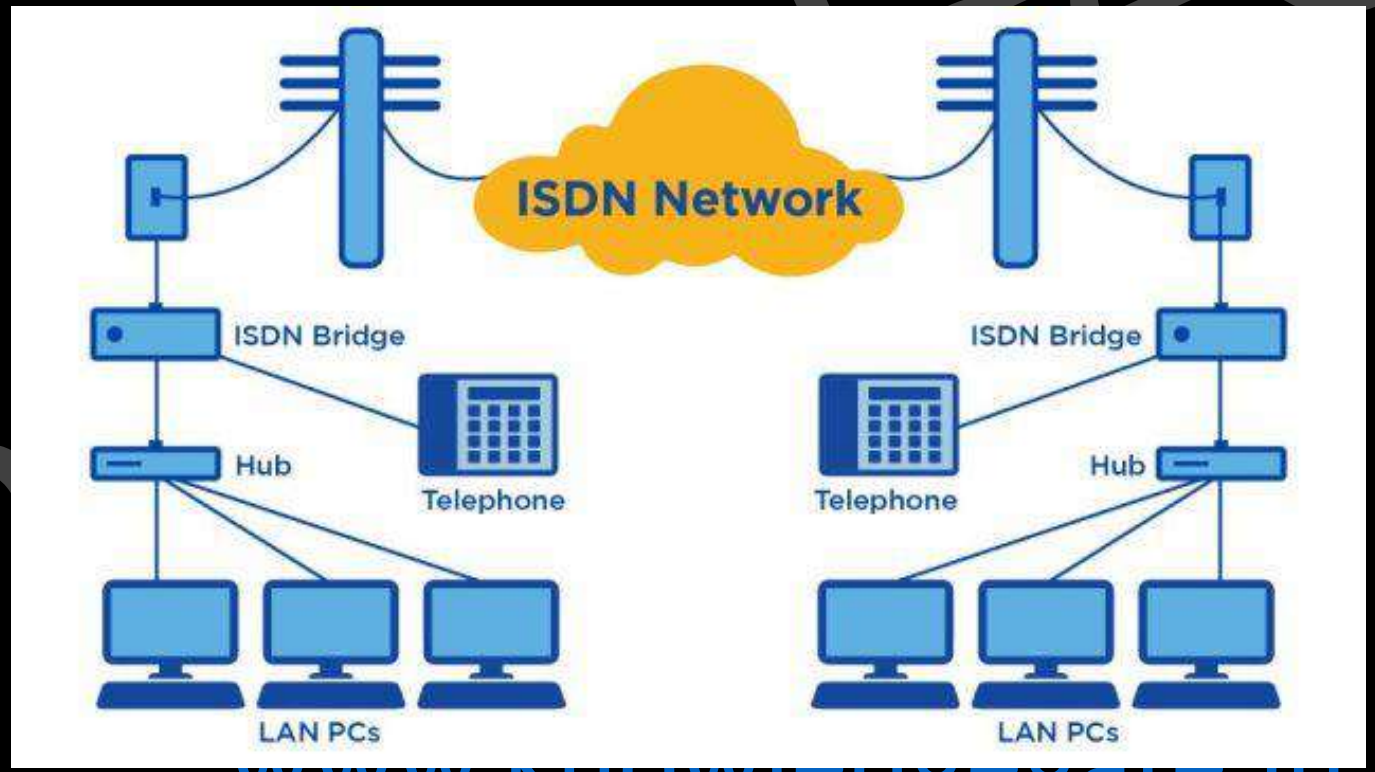
- **Virtual network** - A virtual-circuit network is a cross between a circuit-switched network and a datagram network. It has some characteristics of both. Used now-a-days in telephone networks



www.knowledgegate.in

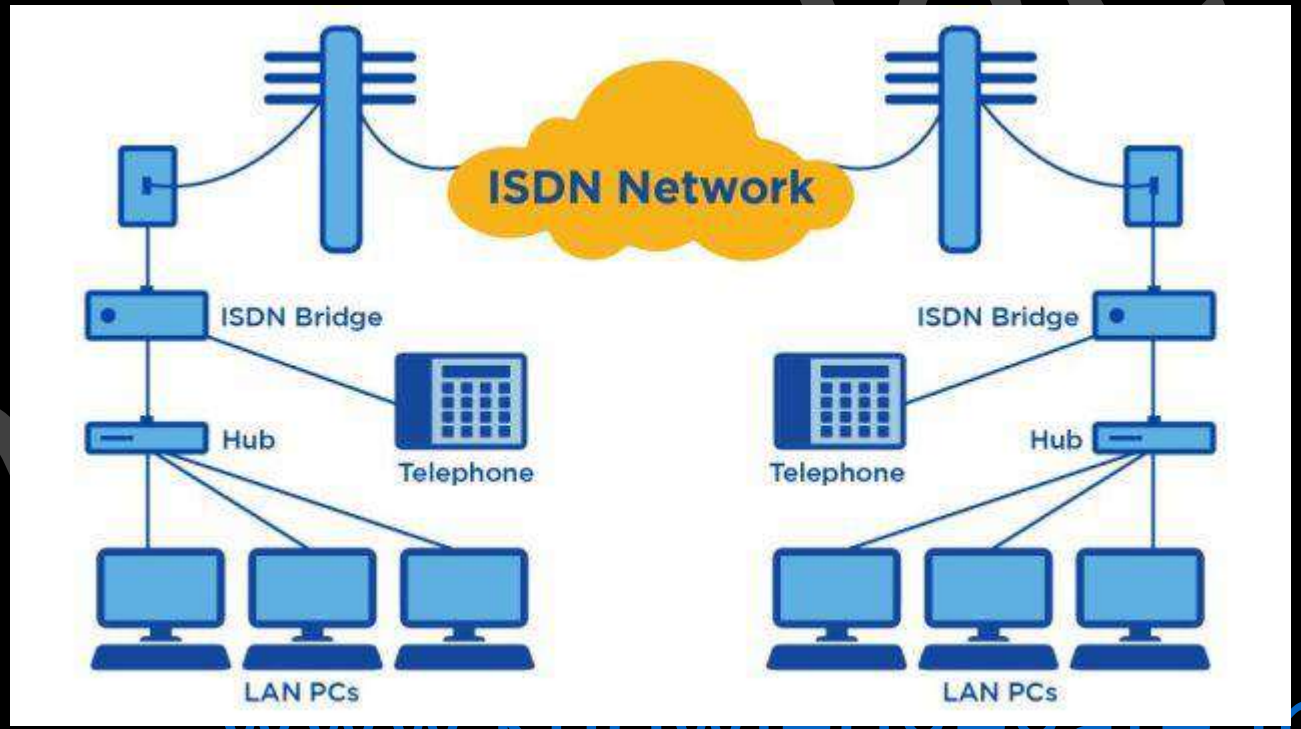
ISDN (Integrated Services Digital Network)

1. **Definition**: Integrated Services Digital Network - a set of protocols for establishing and breaking circuit-switched connections, and for advanced call features for the user.
2. **Development Period**: Developed during the late 1980s and early 1990s.
3. **Digital Transmission**: Unlike traditional telephone services which use analog signals, ISDN uses digital signals for transmission.
4. **Channels**: ISDN provides channels known as B-channels (for data) and D-channels (for control and signaling).



www.knowledgegate.in

5. **BRI and PRI**: There are two types of ISDN interfaces - Basic Rate Interface (BRI) and Primary Rate Interface (PRI). BRI is suitable for home and small enterprise, while PRI is used for larger installations.
6. **Speed**: ISDN provides data rates up to 128 kbps in the case of BRI, and up to 1.544 Mbps for PRI in North America and 2.048 Mbps in Europe.
7. **Usage**: Initially popular for internet access before the widespread availability of broadband.
8. **Decline**: Its popularity has declined with the advent of faster, more reliable broadband internet services.



KnowledgeGate
www.knowledgegate.in

Unit-2

KnowledgeGate

www.knowledgegate.in

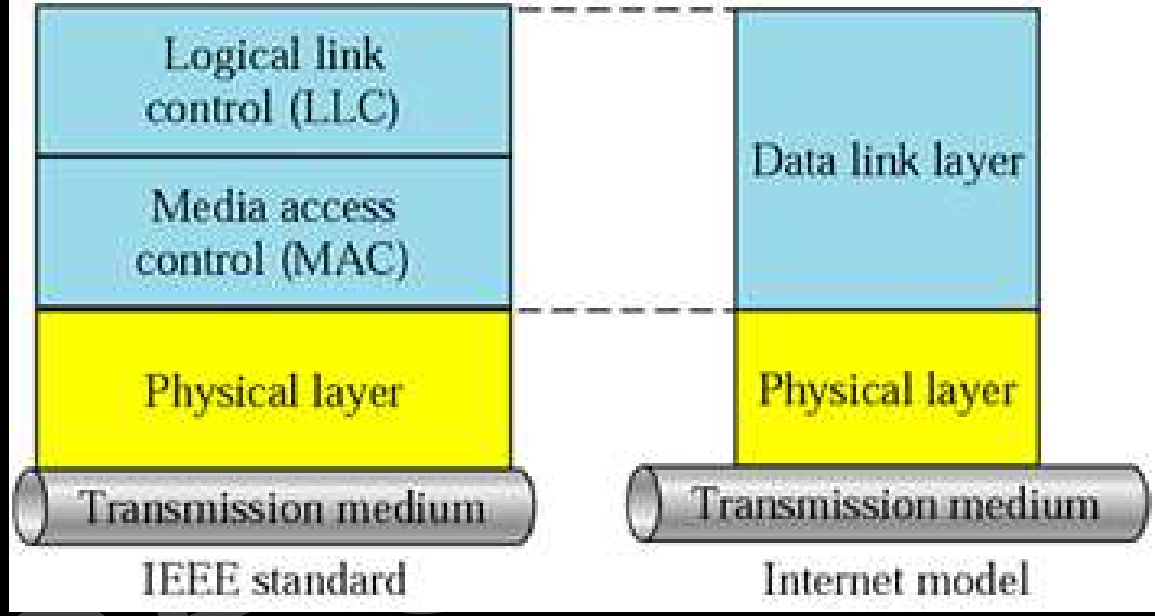
Two Sublayers

1. The IEEE has subdivided the data-link layer into two sublayers: **logical link control (LLC)** (TOP) **and media access control (MAC)** (BOTTOM)**.**

2. ***Media Access Control (MAC):*** It defines the specific access method for each LAN, Ethernet and Take care of Addressing at the level (Lan technology).

3. Flow control, error control, and part of the framing duties are collected into one sublayer called the *logical link control* (LLC).

4. Framing is handled in both the LLC sublayer and the MAC sublayer.

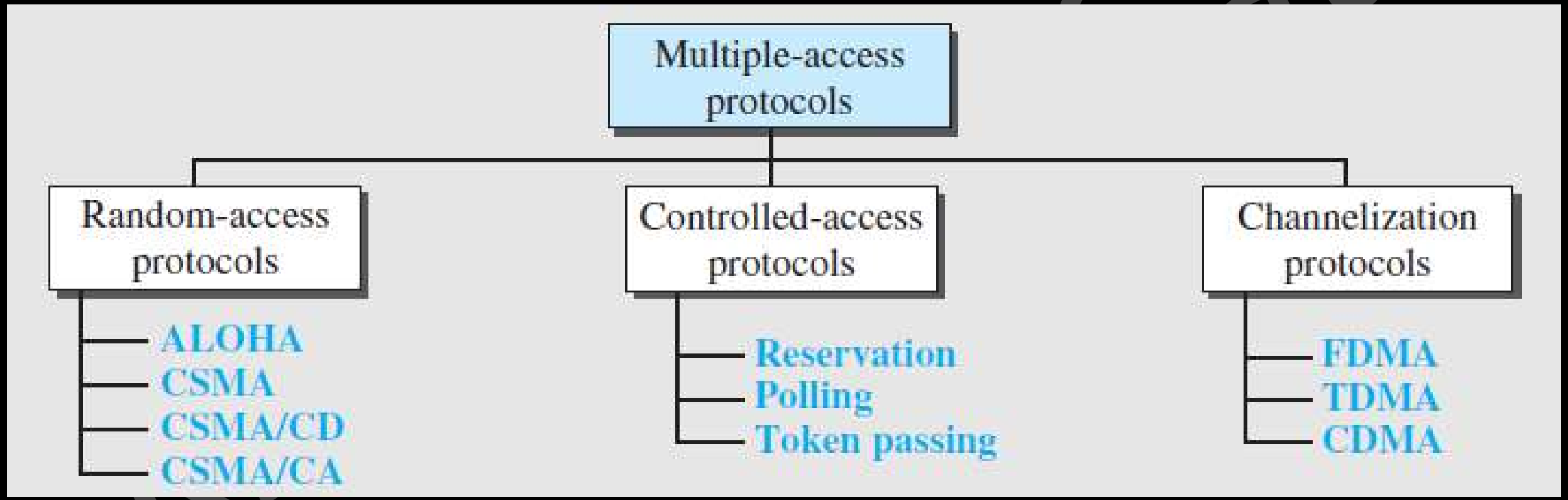


Knowledge

www.knowledgegate.in

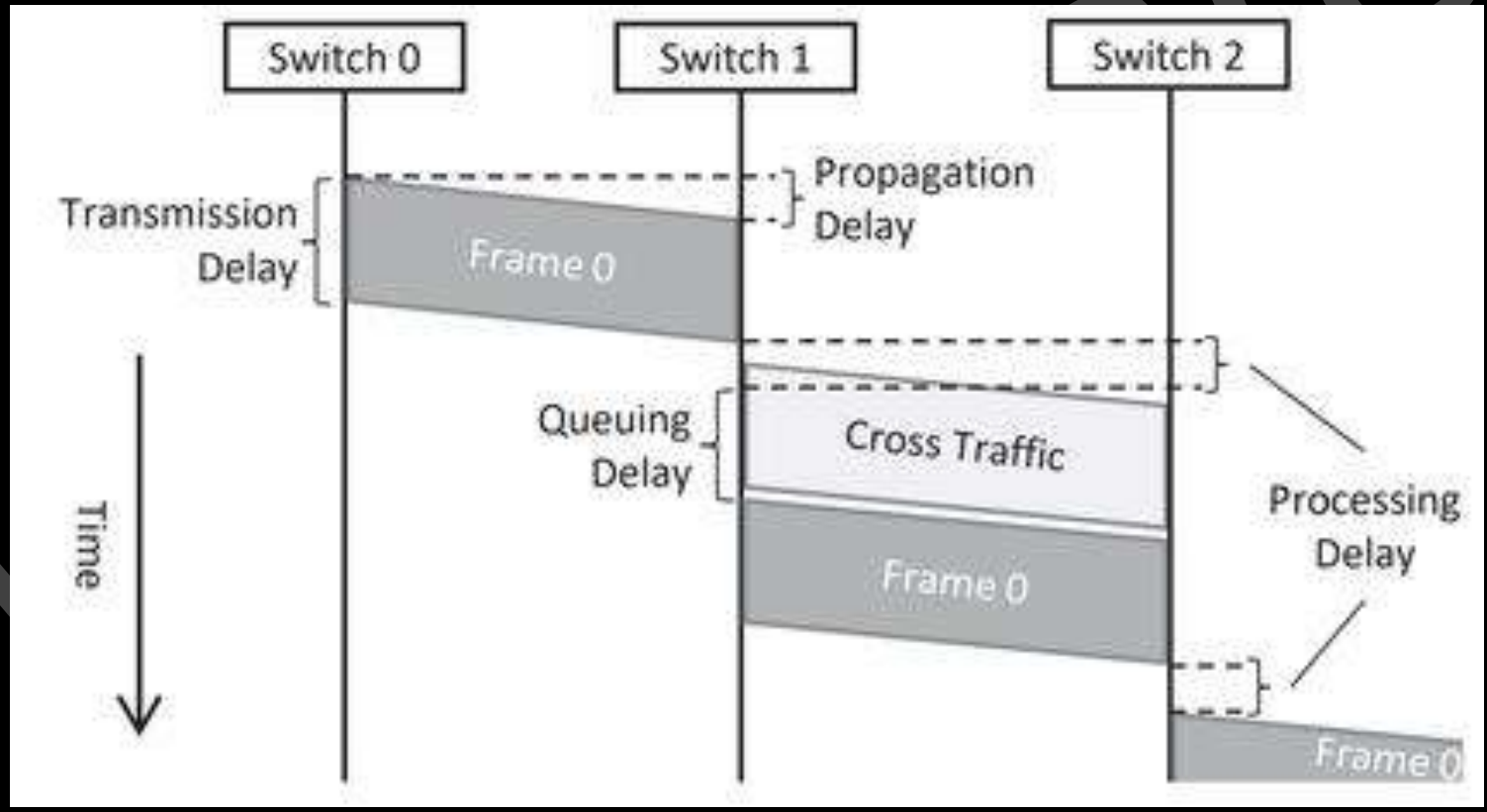
Media Access Control

- When nodes or stations are connected and use a common link, we need a multiple-access protocol to coordinate access to the link. Many protocols have been devised to handle access to a shared link.



www.knowledgegate.in

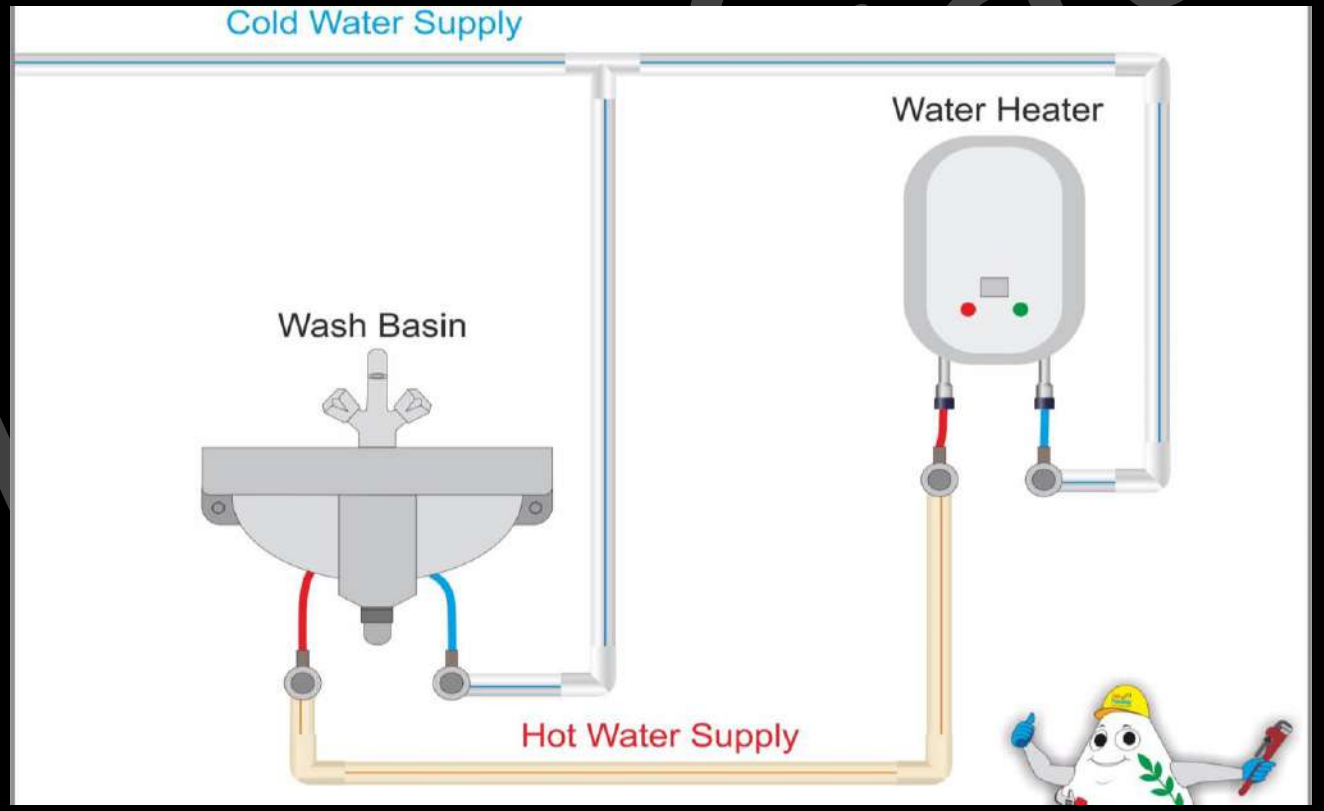
- **<u>Propagation Delay</u>***:* Propagation delay is the time it takes for a bit to travel from point A to point B in the transmission media.
  T_P = (Distance) / (Propagation speed)



Know

www.knowledgegate.in

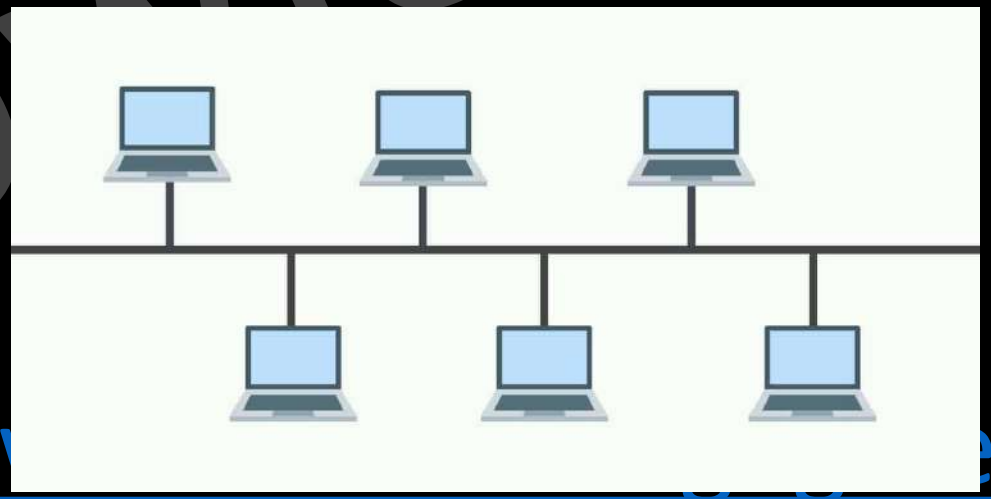
- **<u>Transmission Delay (TT)</u> :** A sender needs to put the bits in a packet on the line one by one. If the first bit of the packet is put on the line at time t_1 and the last bit is put on the line at time t_2, transmission delay of the packet is $(t_2 - t_1)$.
  T_t = (Packet length (L)) / (Transmission rate or Bandwidth (B)) = L / B



www.knowledgegate.in

RANDOM ACCESS

1. In random access methods, no station is superior to another station and none is assigned the control over another.
2. No station permits, or does not permit, another station to send.
3. Two features give this method its name.
   - First, there is no scheduled time for a station to transmit. Transmission is random among the stations. That is why these methods are called random access.
   - Second, no rules specify which station should send next. Stations compete with one another to access the medium. That is why these methods are also called contention methods.
4. However, if more than one station tries to send, there is an access conflict-collision-and the frames will be either destroyed or modified.



KnowledgeGate

e.in

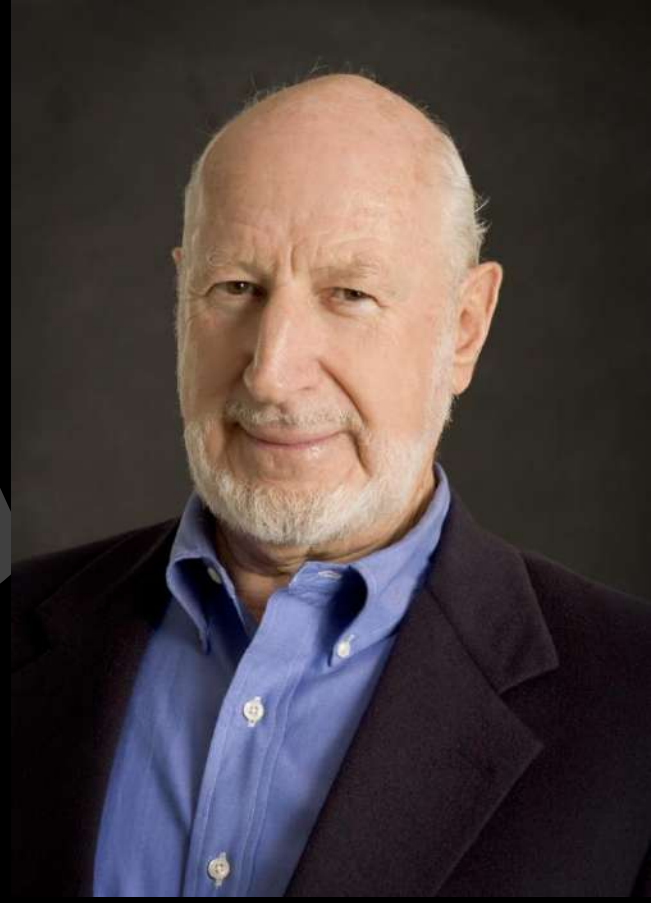
- All the protocols in Random access approach will answer the following questions
  1. When can the station access the medium?
  2. What can the station do if the medium is busy?
  3. How can the station determine the success or failure of the transmission?
  4. What can the station do if there is an access conflict?

KnowledgeGate

www.knowledgegate.in

Aloha

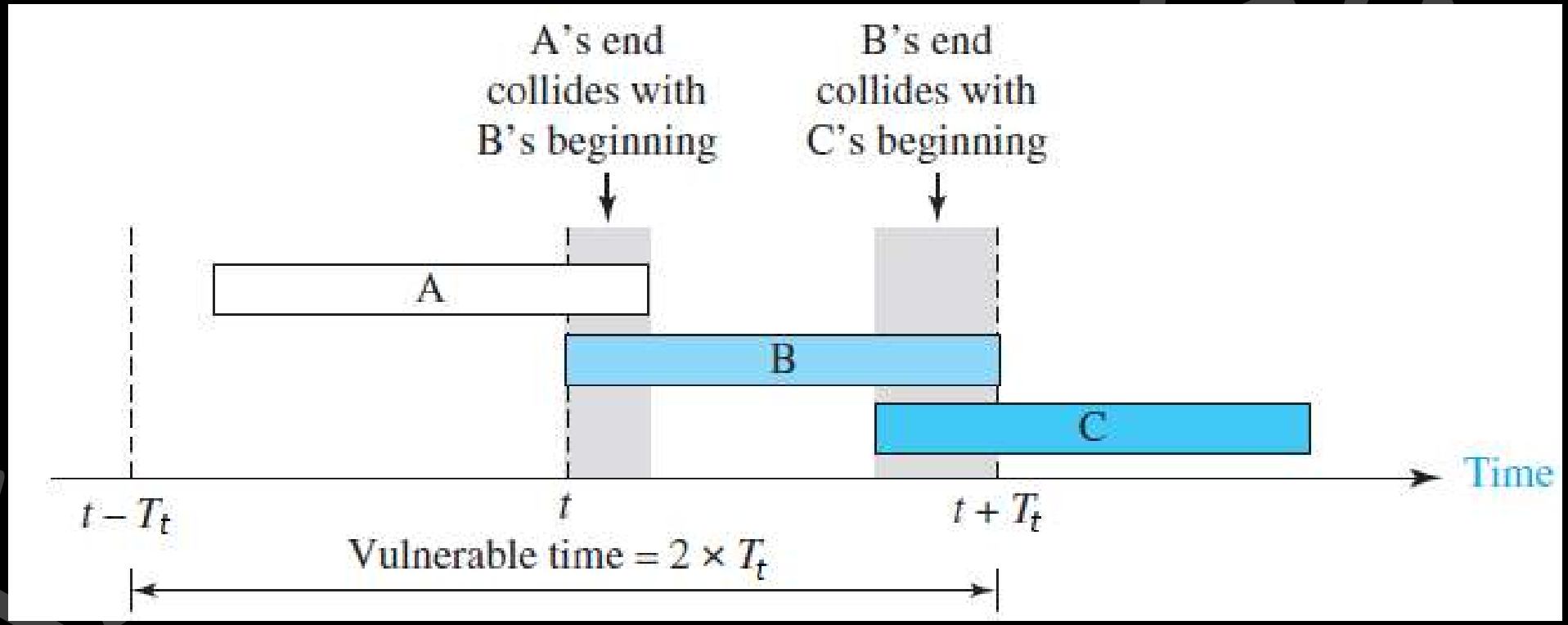
- Earliest random-access method, was developed at the University of Hawaii around 1970.
- It was designed for a radio (wireless) LAN, but it can be used on any shared medium.
- The original ALOHA protocol is called pure ALOHA. This is a simple, but elegant protocol.
- The idea is that each station sends a frame whenever it has a frame to send. However, there is the possibility of collision between frames from different stations.



Norman Manuel Abramson

KnowledgeG

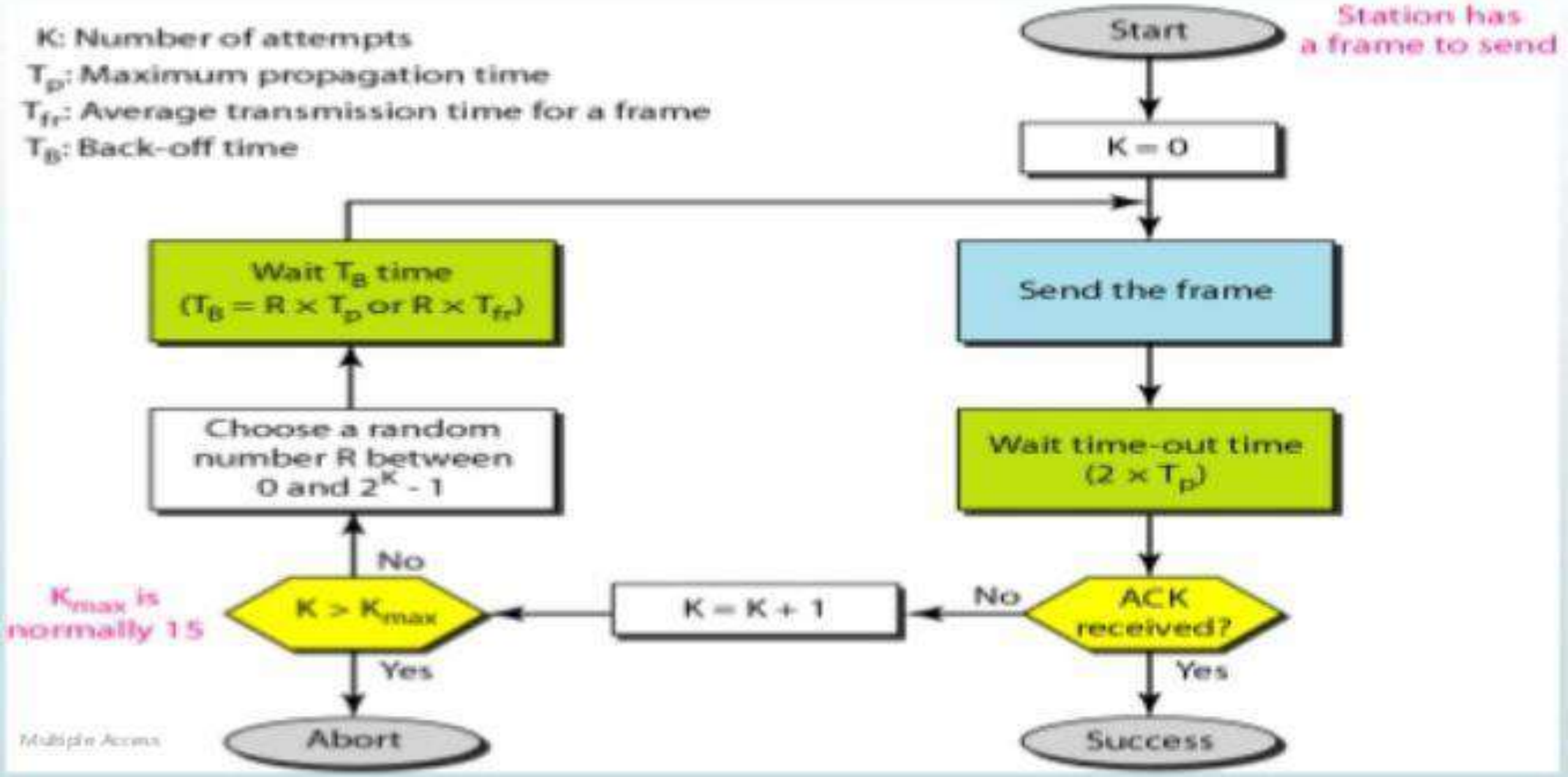
- We assume that the stations send fixed-length frames with each frame taking T_{fr} to send.
- Vulnerable time in which there is a possibility of collision, we see that the time during which a collision may occur in pure ALOHA, is 2 times the frame transmission time. Pure ALOHA vulnerable time= 2 x T_{fr}



www.knowledgegate.in

Procedure for Pure ALOHA protocol

K: Number of attempts
T_p: Maximum propagation time
T_{fr}: Average transmission time for a frame
T_B: Back-off time



- If all these stations try to resend their frames after the time-out, the frames will collide again.
- Pure ALOHA dictates that when the time-out period passes, each station waits a random amount of time before resending its frame. The randomness will help avoid more collisions. We call this time the back-off time T_B.

- Pure ALOHA has a second method to prevent congesting the channel with retransmitted frames. After a maximum number of retransmissions attempts K_{max} a station must give up and try later.

KnowledgeGate

www.knowledgegate.in

Example 12.2

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the requirement to make this frame collision-free?

Solution

Average frame transmission time T_{fr} is 200 bits/200 kbps or 1 ms. The vulnerable time is 2×1 ms = 2 ms. This means no station should send later than 1 ms before this station starts transmission and no station should start sending during the period (1 ms) that this station is sending.

Throughput

Let us call G the average number of frames generated by the system during one frame transmission time. Then it can be proven that the average number of successfully transmitted frames for pure ALOHA is $S = G \times e^{-2G}$. The maximum throughput S_{max} is 0.184, for $G = 1/2$. (We can find it by setting the derivative of S with respect to G to 0; see Exercises.) In other words, if one-half a frame is generated during one frame transmission time (one frame during two frame transmission times), then 18.4 percent of these frames reach their destination successfully. We expect $G = 1/2$ to produce the maximum throughput because the vulnerable time is 2 times the frame transmission time. Therefore, if a station generates only one frame in this vulnerable time (and no other stations generate a frame during this time), the frame will reach its destination successfully.

The throughput for pure ALOHA is $S = G \times e^{-2G}$.
The maximum throughput $S_{max} = 1/(2e) = 0.184$ when $G = (1/2)$.

Example 12.3

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the throughput if the system (all stations together) produces

a. 1000 frames per second?

b. 500 frames per second?

c. 250 frames per second?

Solution

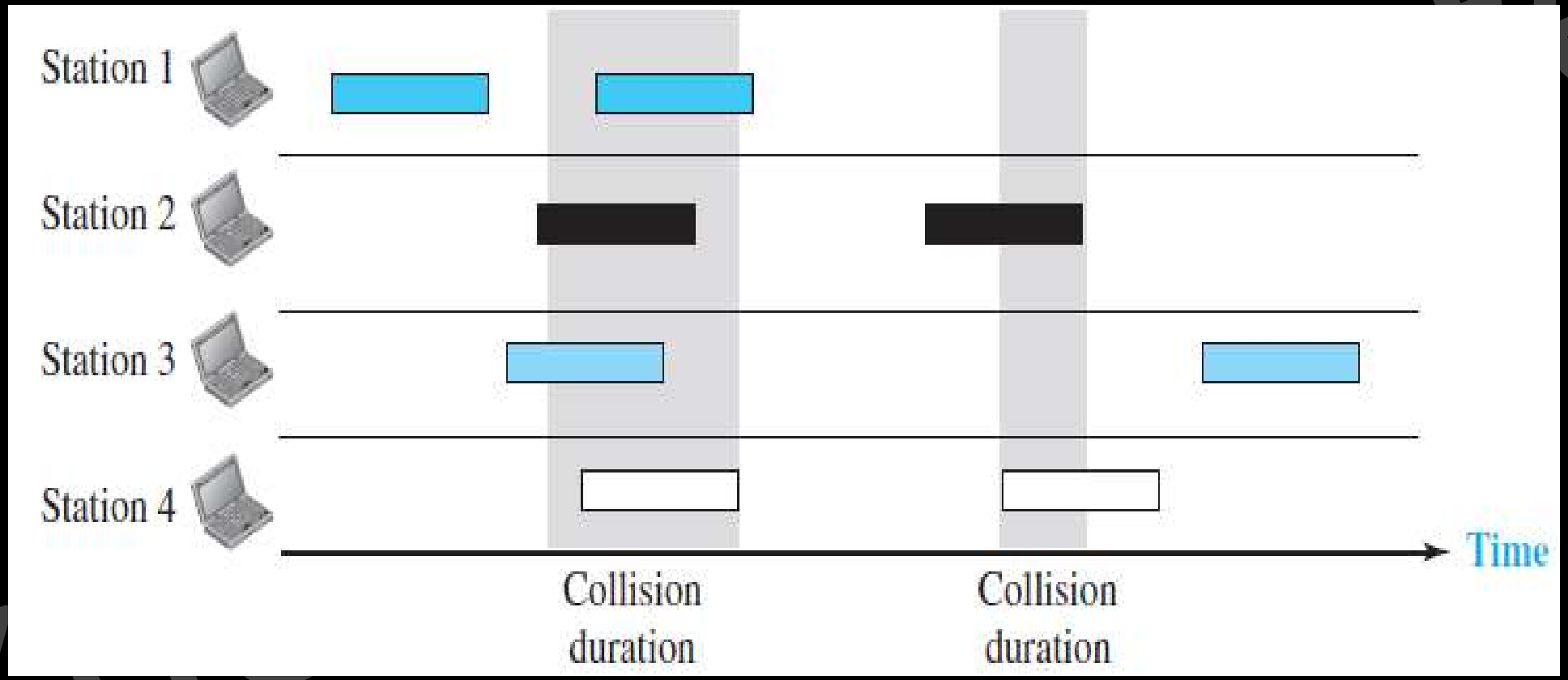
The frame transmission time is 200/200 kbps or 1 ms.

a. If the system creates 1000 frames per second, or 1 frame per millisecond, then $G = 1$. In this case $S = G \times e^{-2G} = 0.135$ (13.5 percent). This means that the throughput is $1000 \times 0.135 = 135$ frames. Only 135 frames out of 1000 will probably survive.

b. If the system creates 500 frames per second, or 1/2 frames per millisecond, then $G = 1/2$. In this case $S = G \times e^{-2G} = 0.184$ (18.4 percent). This means that the throughput is $500 \times 0.184 = 92$ and that only 92 frames out of 500 will probably survive. Note that this is the *maximum* throughput case, percentagewise.

c. If the system creates 250 frames per second, or 1/4 frames per millisecond, then $G = 1/4$. In this case $S = G \times e^{-2G} = 0.152$ (15.2 percent). This means that the throughput is $250 \times 0.152 = 38$. Only 38 frames out of 250 will probably survive.

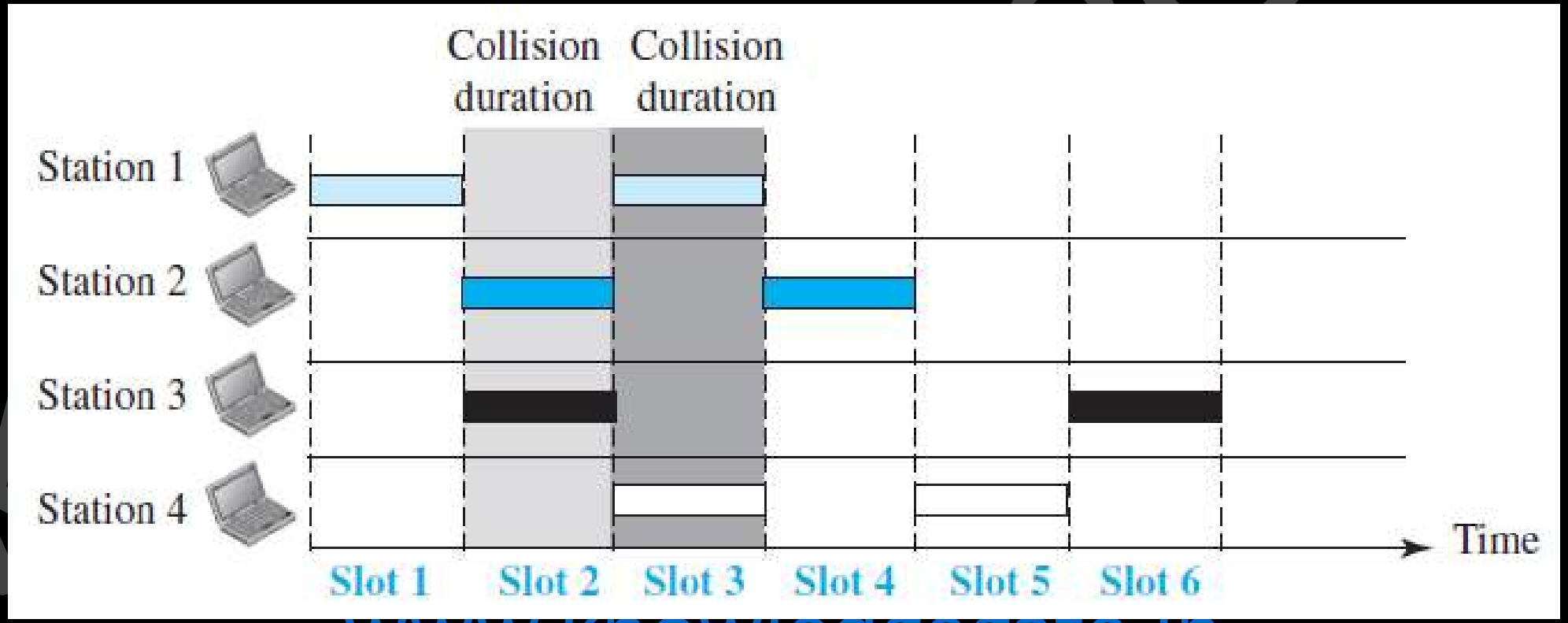
ALOHA



www.knowledgegate.in

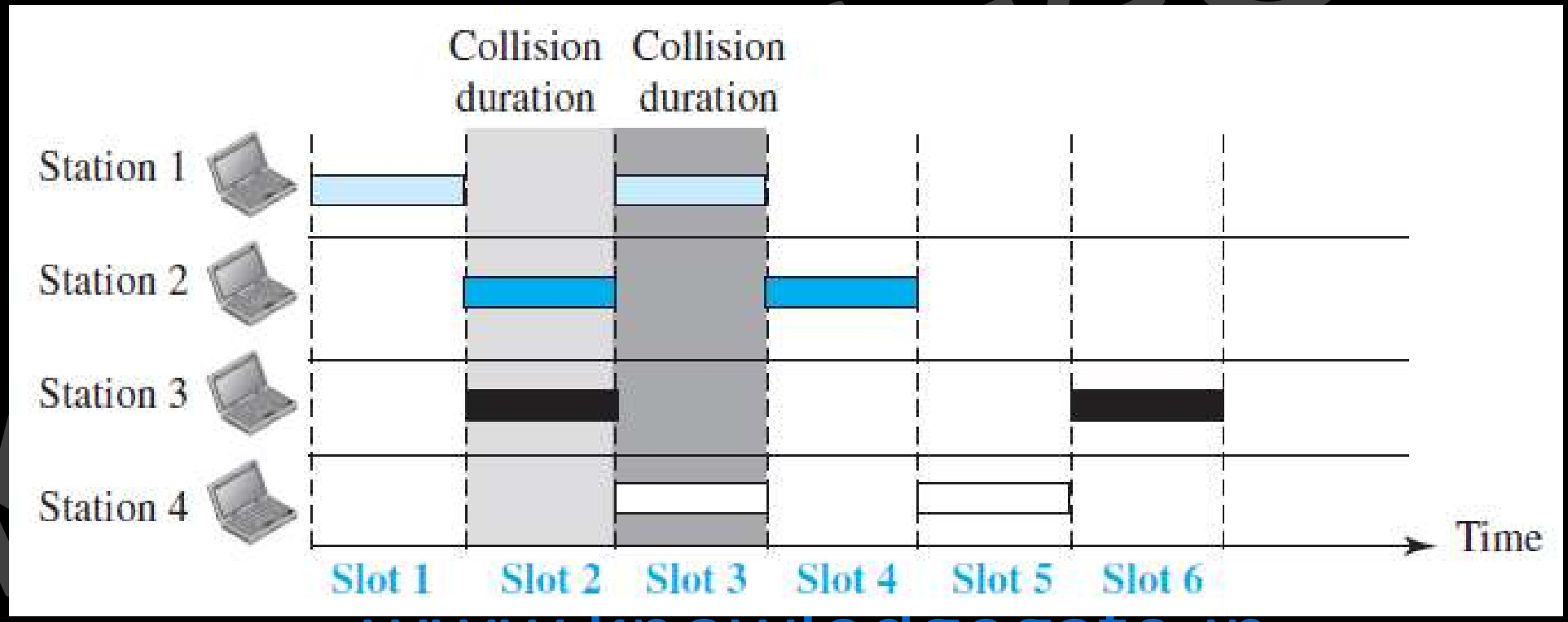
Slotted ALOHA

- Pure ALOHA has a vulnerable time of 2 x T_{fr}. This is so because there is no rule that defines when the station can send. A station may send soon after another station has started or soon before another station has finished.
- Slotted ALOHA was invented to improve the efficiency of pure ALOHA. In slotted ALOHA we divide the time into slots of T_{fr} s and force the station to send only at the beginning of the time slot.



www.knowledgegate.in

- Because a station is allowed to send only at the beginning of the synchronized time slot, if a station misses this moment, it must wait until the beginning of the next time slot. This means that the station which started at the beginning of this slot has already finished sending its frame.
- Off course, there is still the possibility of collision if two stations try to send at the beginning of the same time slot. However, the vulnerable time is now reduced to one-half, equal to T_{fr}



www.knowledgegate.in

Aspect	Pure ALOHA	Slotted ALOHA
Time Structure	No specific time structure; data can be sent at any time.	Operates on time slots; data is sent at the beginning of a time slot.
Efficiency	Lower efficiency (~18.4%) due to higher probability of collisions.	Higher efficiency (~37%) as it reduces the chance of collisions.
Collision Handling	If a collision occurs, the message is resent after a random time interval.	Messages are sent in synchronized time slots, reducing the chance of collision but if a collision occurs, it is handled similarly to Pure ALOHA.
Complexity	Less complex as it doesn't require synchronization.	More complex due to the need for time synchronization.
Implementation	Easier to implement due to lack of synchronization requirements.	Implementation is moderately complex because of the synchronization of time slots.

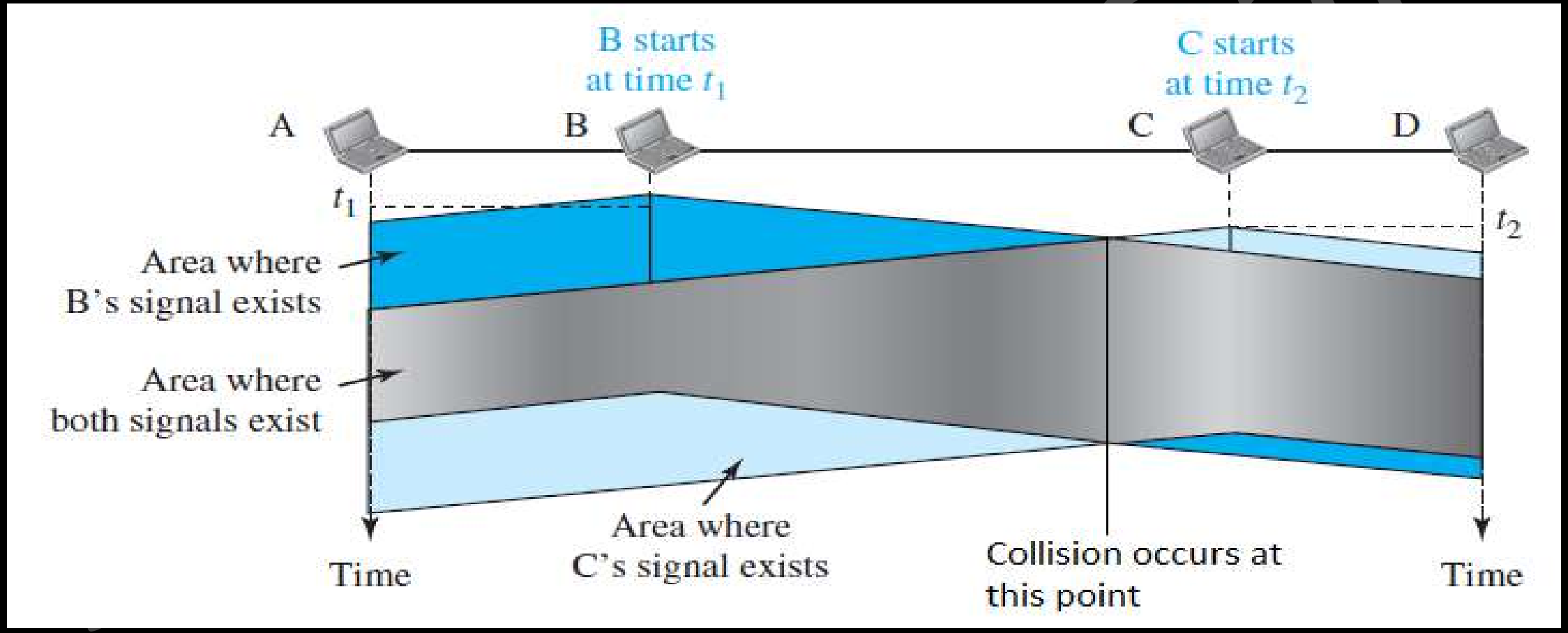
KnowledgeGate
www.knowledgegate.in

Carrier Sense Multiple Access (CSMA)

- To minimize the chance of collision and, therefore, increase the performance, the CSMA method was developed. The chance of collision can be reduced if a station senses the medium before trying to use it.
- Carrier sense multiple access (CSMA) requires that each station first listen to the medium (or check the state of the medium) before sending, so "sense before transmit" or" listen before talk." CSMA can reduce the possibility of collision, but it cannot eliminate it.

KnowledgeGate

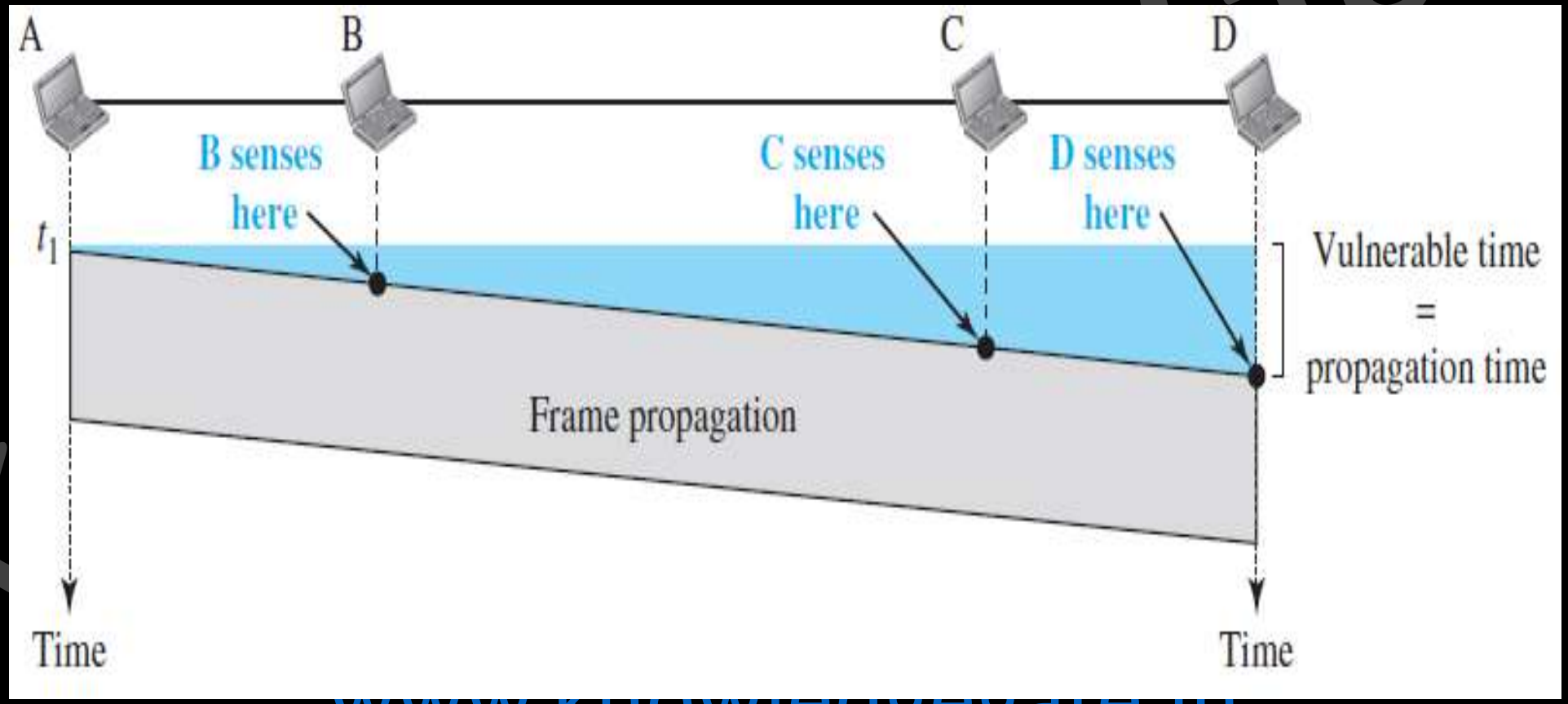
- The possibility of collision still exists because of propagation delay; when a station sends a frame, it still takes time (although very short) for the first bit to reach every station and for every station to sense it. In other words, a station may sense the medium and find it idle, only because the first bit sent by another station has not yet been received.



www.knowledgegate.in

Vulnerable Time

- The vulnerable time for CSMA is the ***propagation time*** *Tp*. When a station sends a frame and any other station tries to send a frame during this time, a collision will result.
- But if the first bit of the frame reaches the end of the medium, every station will already have heard the bit and will refrain from sending.



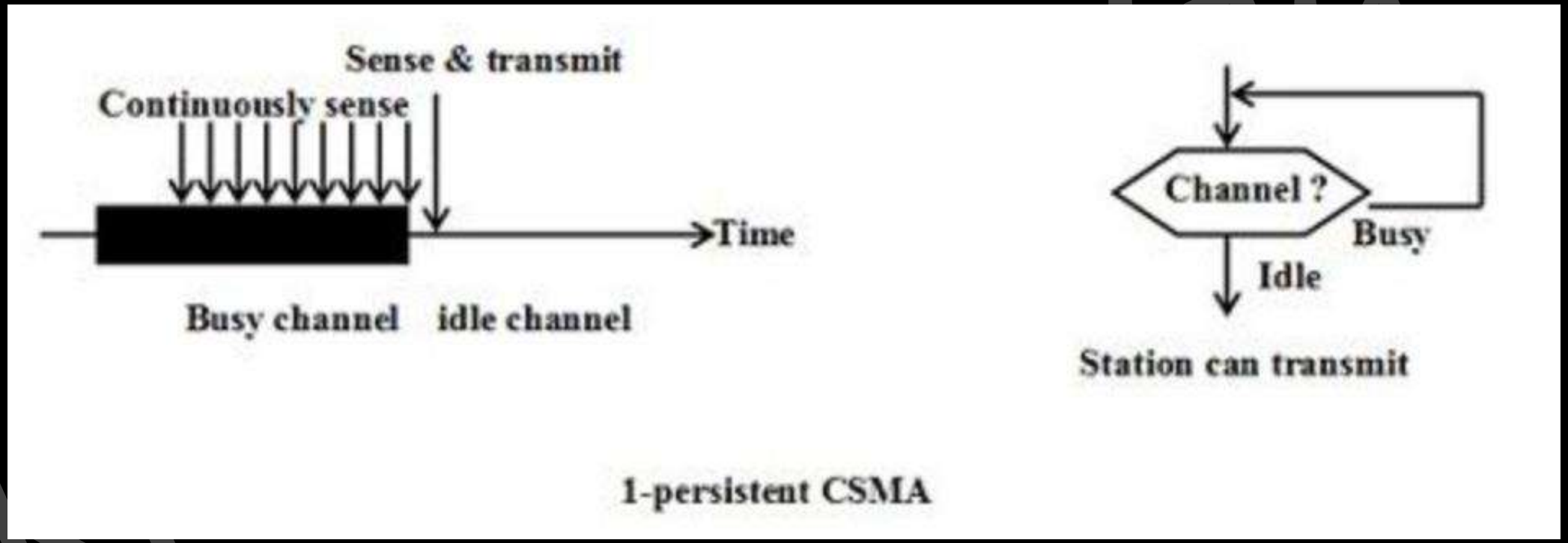
www.knowledgegate.in

Persistence Methods

- What should a station do if the channel is busy? What should a station do if the channel is idle?
- Three methods have been devised to answer these questions:
  - 1-persistent method
  - Non-persistent method
  - P-persistent method.

KnowledgeGate

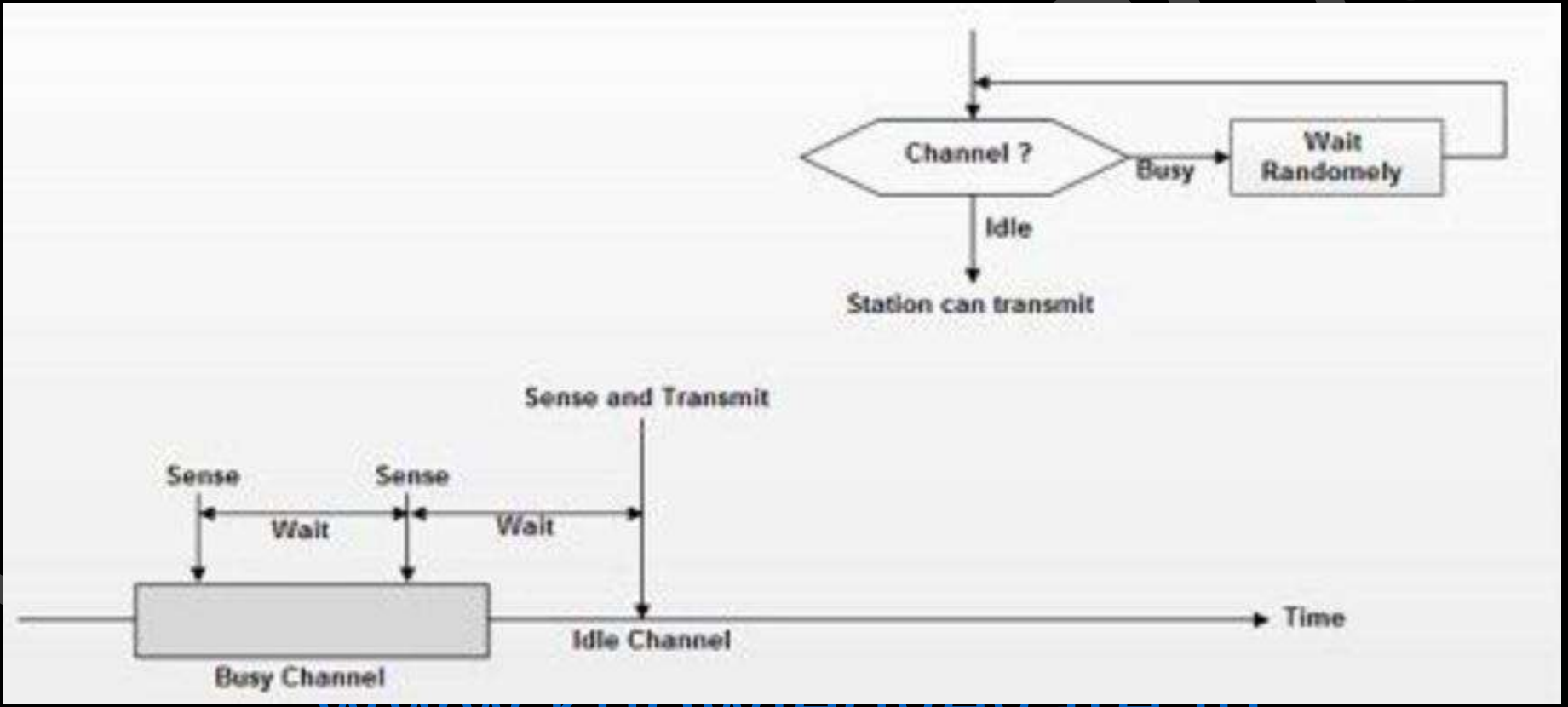
- **1-Persistent**
  - The 1-persistent method is simple and straightforward. In this method, after the station finds the line idle, it sends its frame immediately (with probability 1). This method has the highest chance of collision because two or more stations may find the line idle and send their frames immediately.



1-persistent CSMA

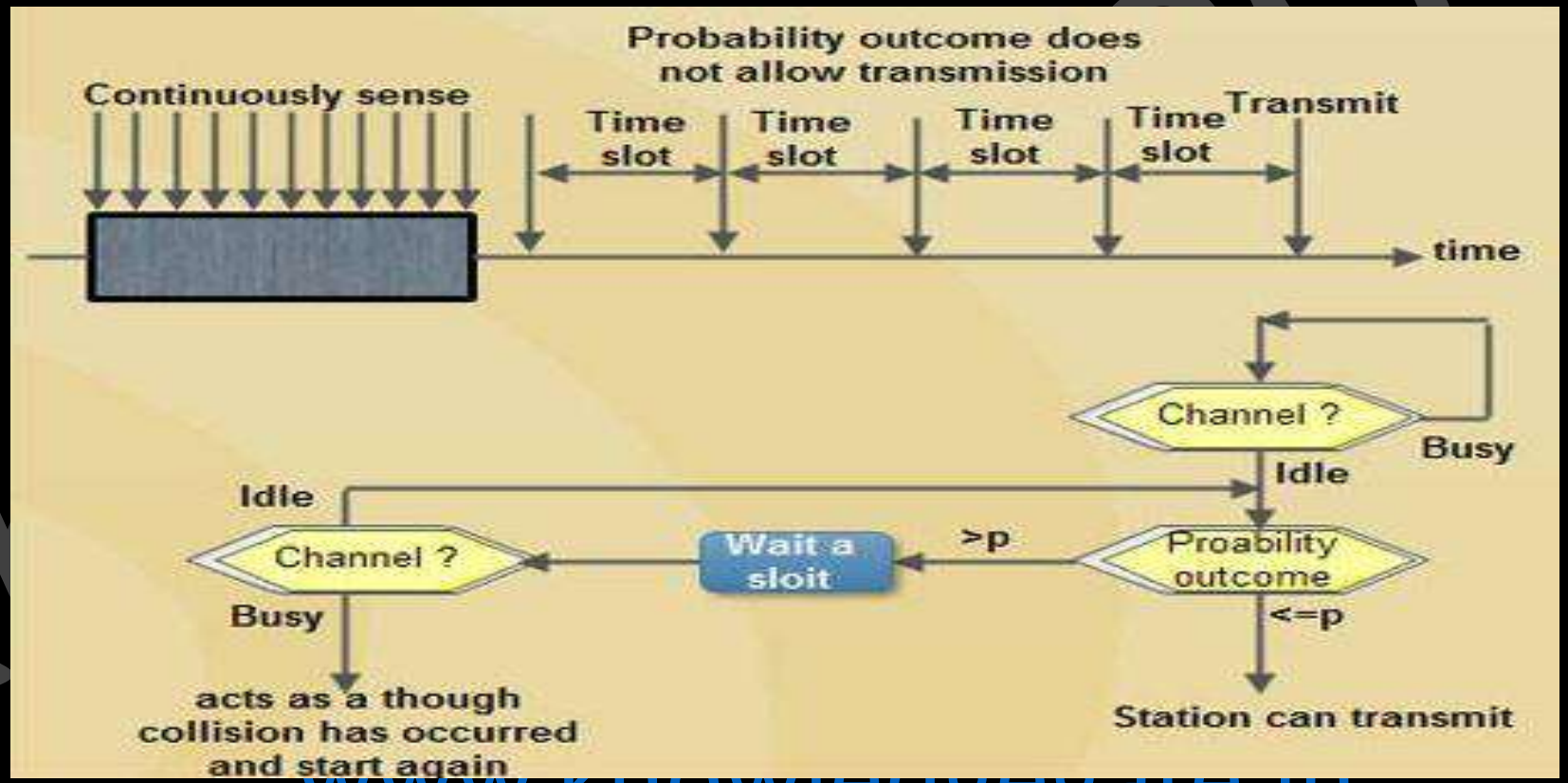
www.knowledgegate.in

- **Nonpersistent**
  - In the nonpersistent method, a station that has a frame to send senses the line. If the line is idle, it sends immediately. If the line is not idle, it waits a random amount of time and then senses the line again.
  - The nonpersistent approach reduces the chance of collision because it is unlikely that two or more stations will wait the same amount of time and retry to send simultaneously. However, this method reduces the efficiency of the network because the medium remains idle when there may be stations with frames to send.



www.knowledgegate.in

- **P-Persistent**
  - The p-persistent approach combines the advantages of the other two strategies. It reduces the chance of collision and improves efficiency. In this method, after the station finds the line idle it follows these steps:
  - With probability p, the station sends its frame.
  - With probability q = 1 - p, the station waits for the beginning of the next time slot and checks the line again.
    - a. If the line is idle, it goes to step 1.
    - b. If the line is busy, it acts as though a collision has occurred and uses the backoff procedure.



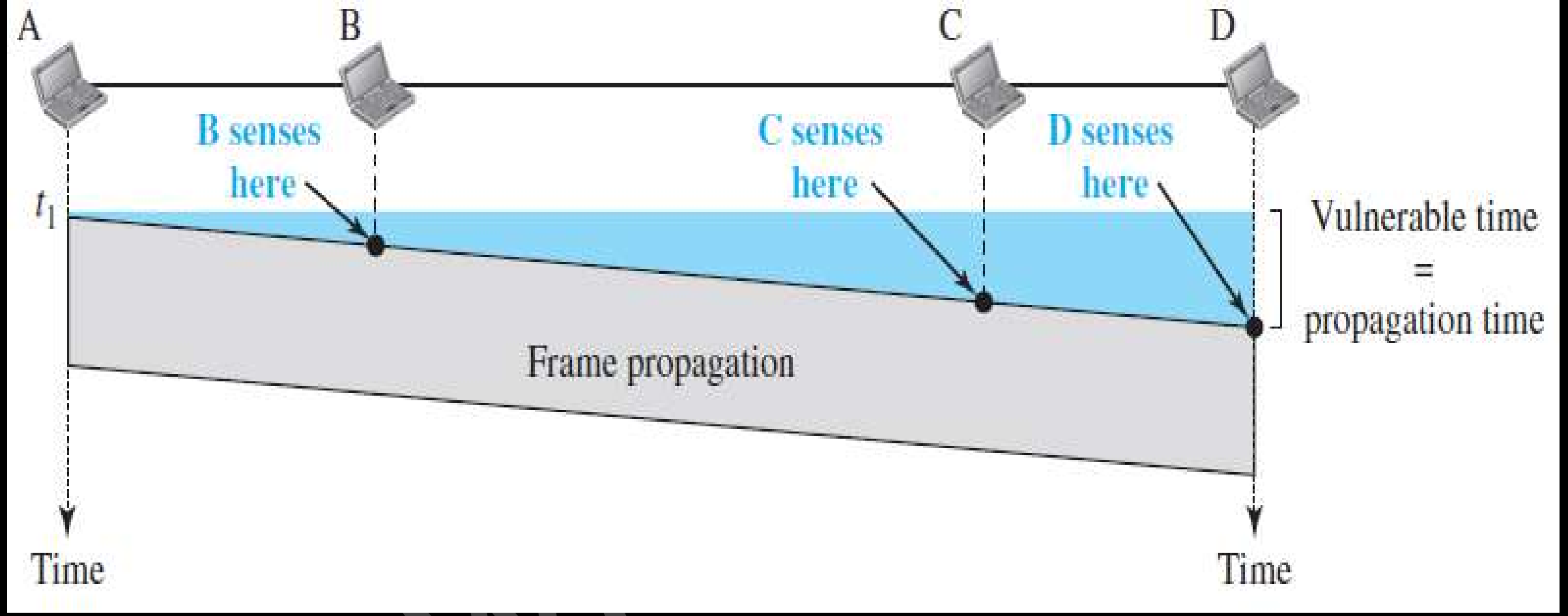
www.knowledgegate.in

Carrier Sense Multiple Access with Collision Detection (CSMA/CD)

- In this method, a station monitors the medium after it sends a frame to see if the transmission was successful. If so, the station is finished. If, however, there is a collision, the frame is sent again.
- Minimum Frame Size - For CSMA / CD to work, we need a restriction on the minimum frame size. Before sending the last bit of the frame, the sending station must detect a collision, if any, and abort the transmission.
- This is so because the station, once the entire frame is sent, does not monitor the line for collision detection. Therefore, the frame transmission time T_{fr} must be at least two times the maximum propagation time Tp.
- To understand the reason, let us think about the worst-case scenario. If the two stations involved in a collision are the maximum distance apart, the signal from the first takes time T_p to reach the second, and the effect of the collision takes another time T_p to reach the first. So the requirement is that the first station must still be transmitting after $2T_p$.

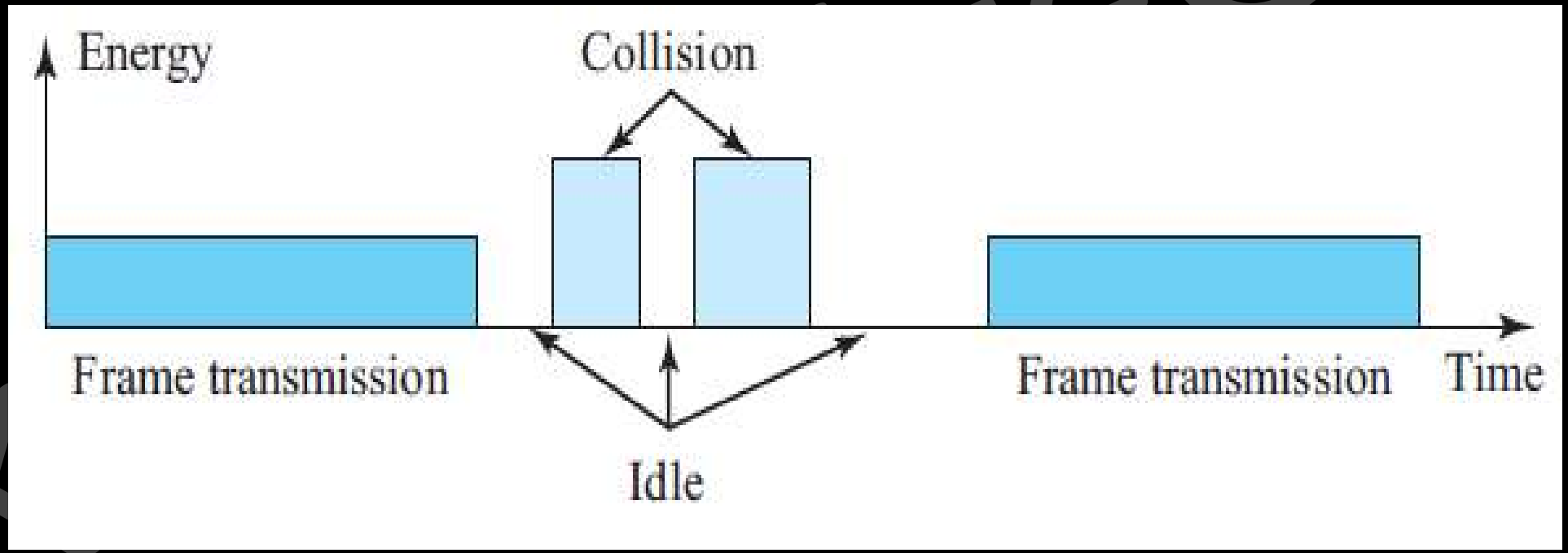
KnowledgeGate

www.knowledgegate.in

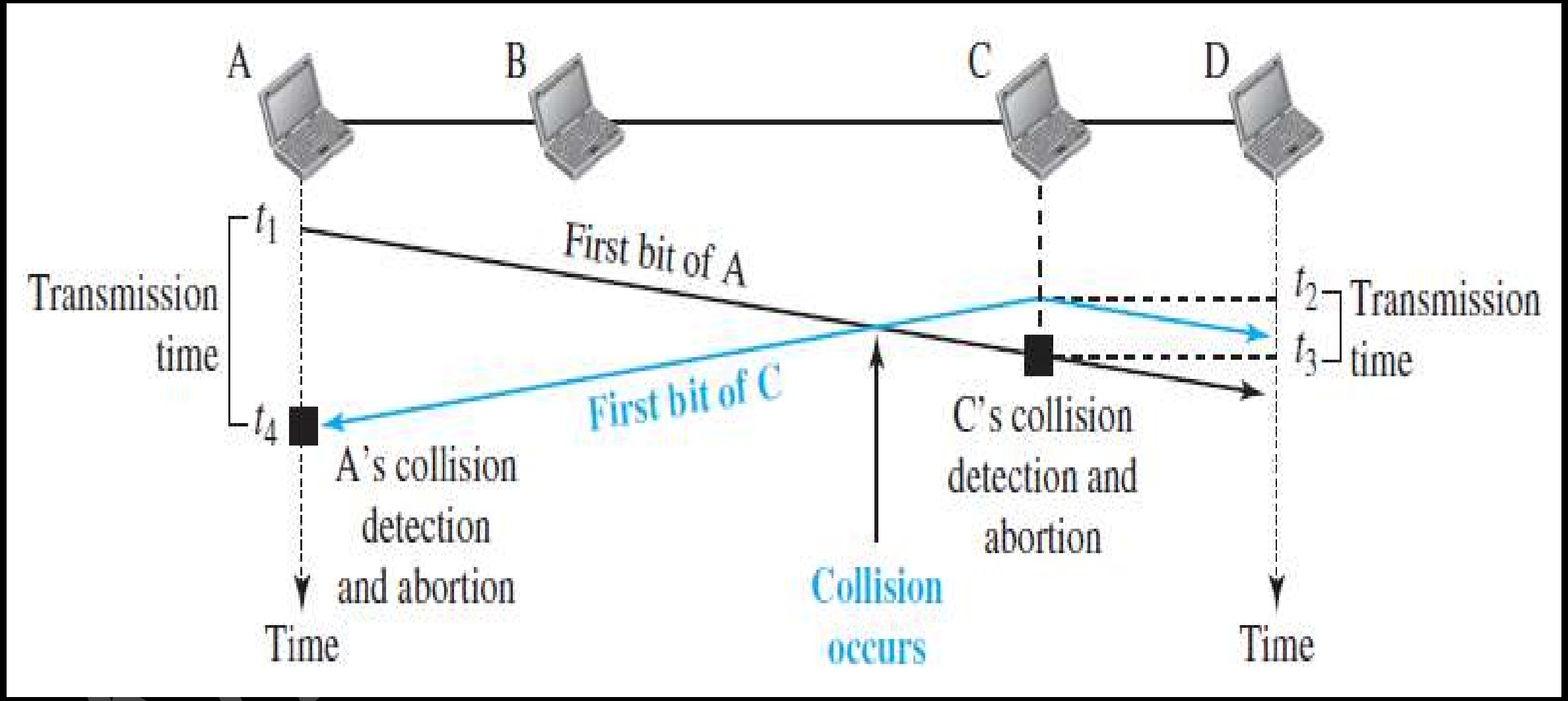


www.knowledgegate.in

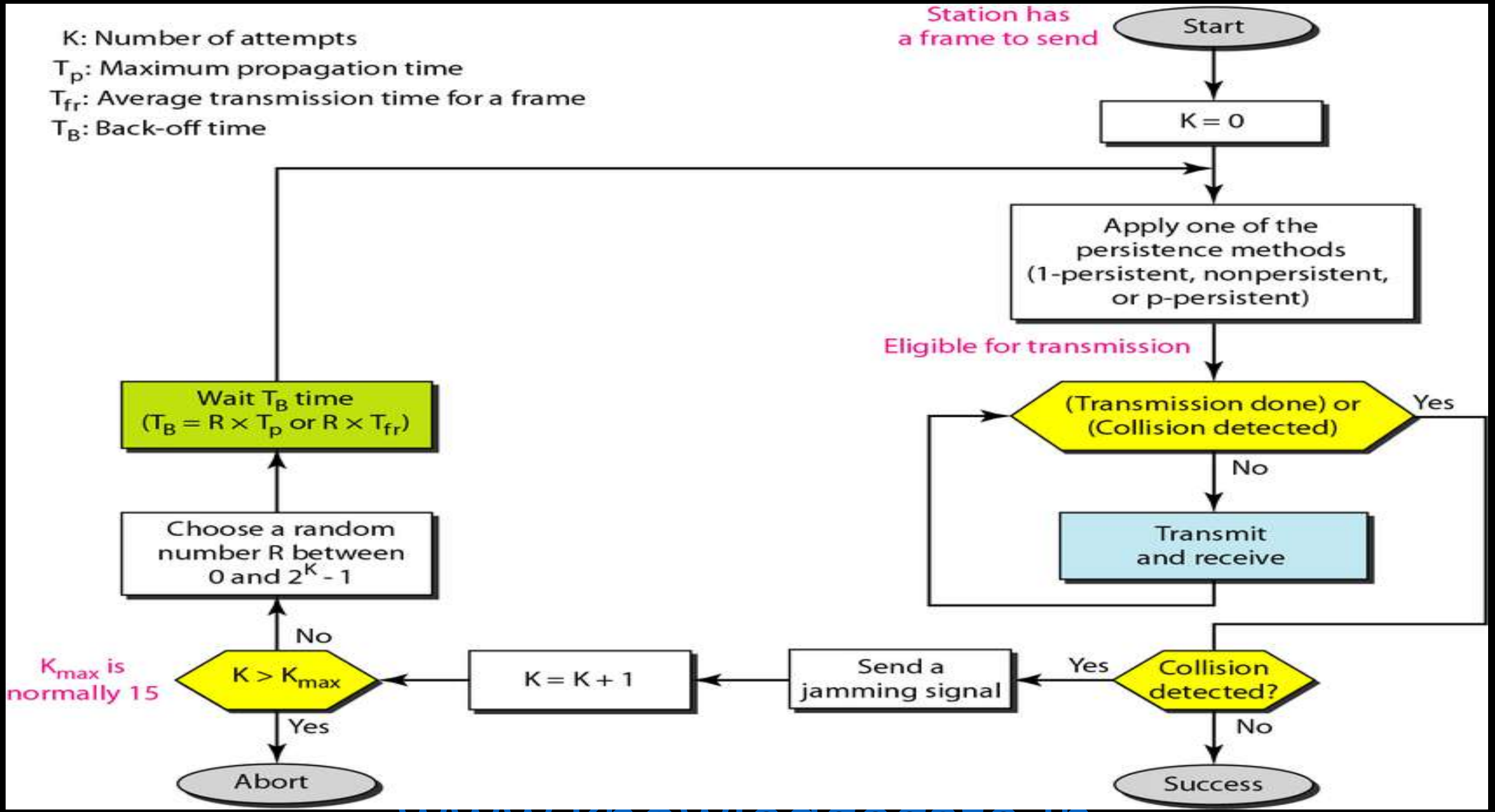
- **Energy Level**
  - We can say that the level of energy in a channel can have three values: zero, normal, and abnormal. At the zero level, the channel is idle. At the normal level, a station has successfully captured the channel and is sending its frame.
  - At the abnormal level, there is a collision and the level of the energy is twice the normal level. A station that has a frame to send or is sending a frame needs to monitor the energy level to determine if the channel is idle, busy, or in collision mode.



www.knowledgegate.in



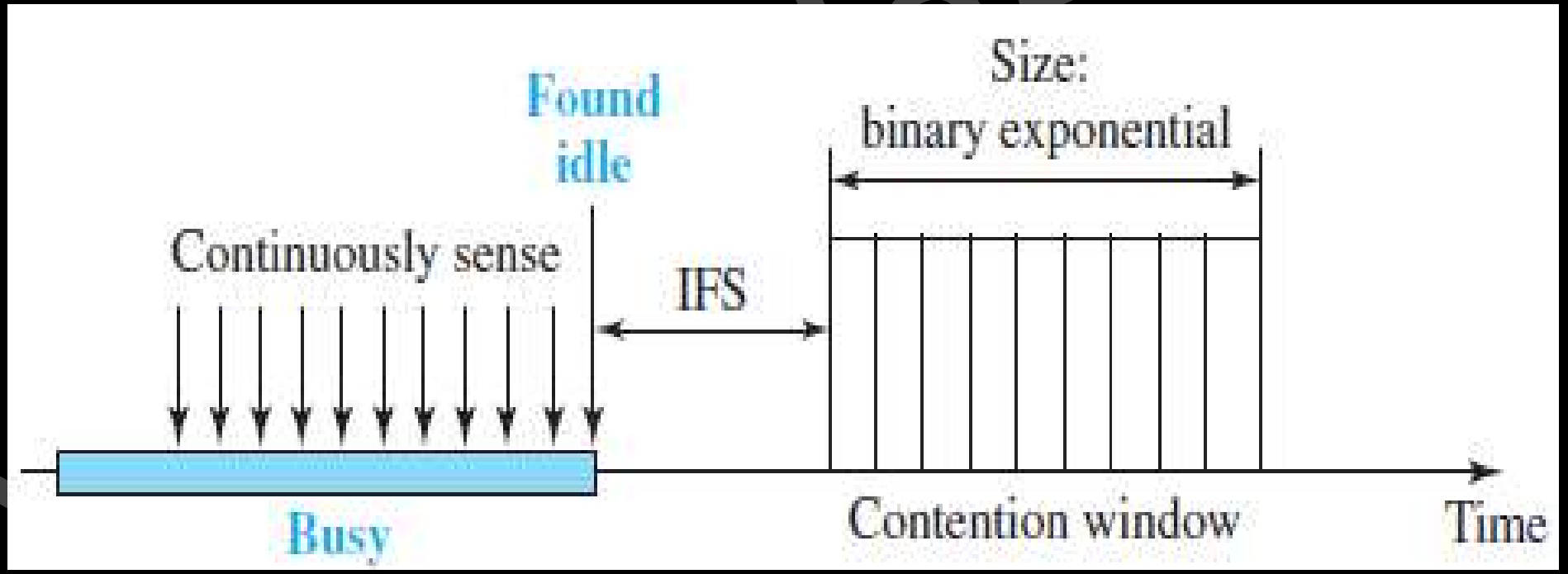
www.knowledgegate.in



www.knowledgegate.in

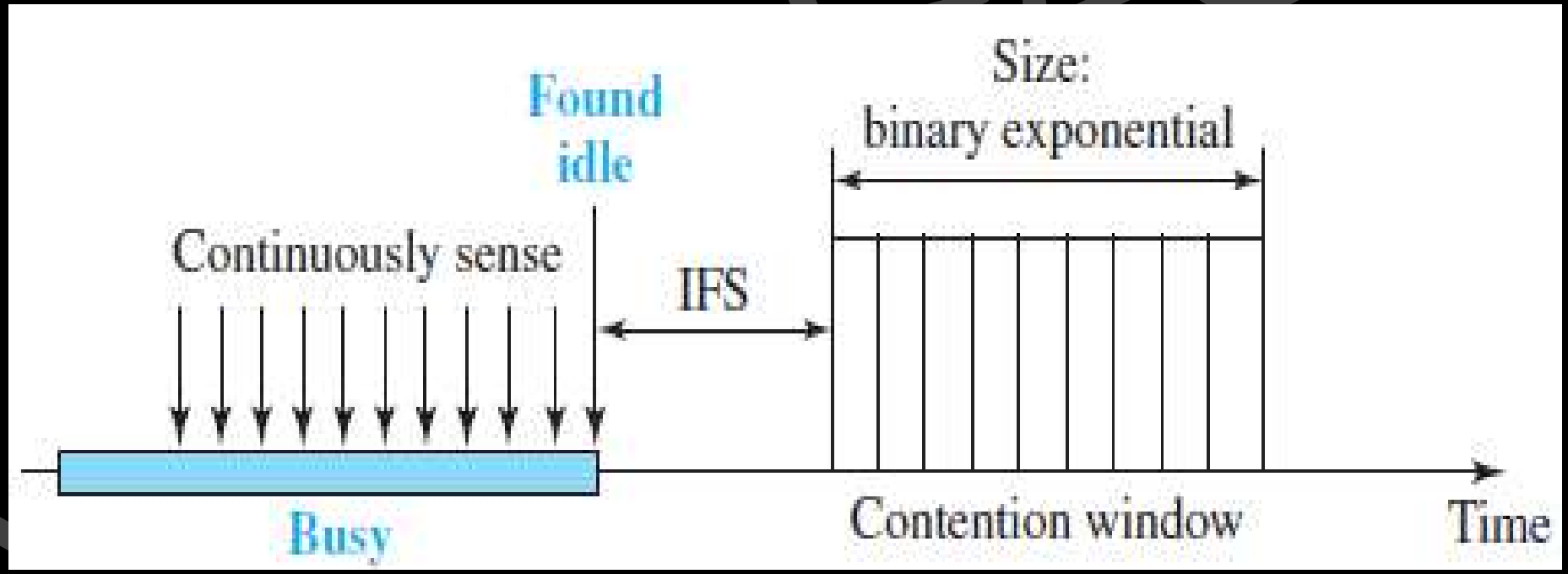
Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)

- In a wireless network, much of the sent energy is lost in transmission. The received signal has very little energy. Therefore, a collision may add only 5 to 10 percent additional energy. This is not useful for effective collision detection.
- We need to avoid collisions on wireless networks because they cannot be detected. Carrier sense multiple access with collision avoidance (CSMA / CA) was invented for this network. Collisions are avoided through the use of CSMA / CA
- three strategies: the interframe space, the contention window, and acknowledgment



www.knowledgegate.in

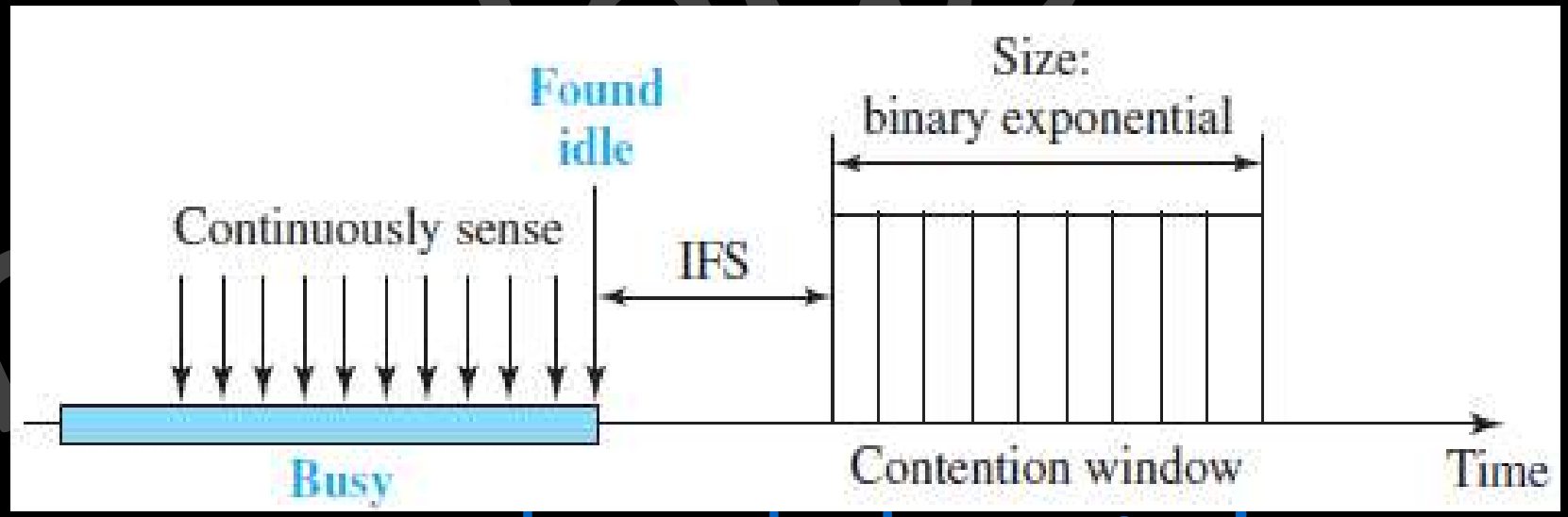
- **Interframe Space (IFS)**
  - First, collisions are avoided by deferring transmission even if the channel is found idle. When an idle channel is found, the station does not send immediately. It waits for a period of time called the interframe space or IFS.
  - The IFS time allows the front of the transmitted signal by the distant station to reach this station. If after the IFS time the channel is still idle, the station can send, but it still needs to wait a time equal to the contention time. The IFS variable can also be used to prioritize stations or frame types.



www.knowledgegate.in

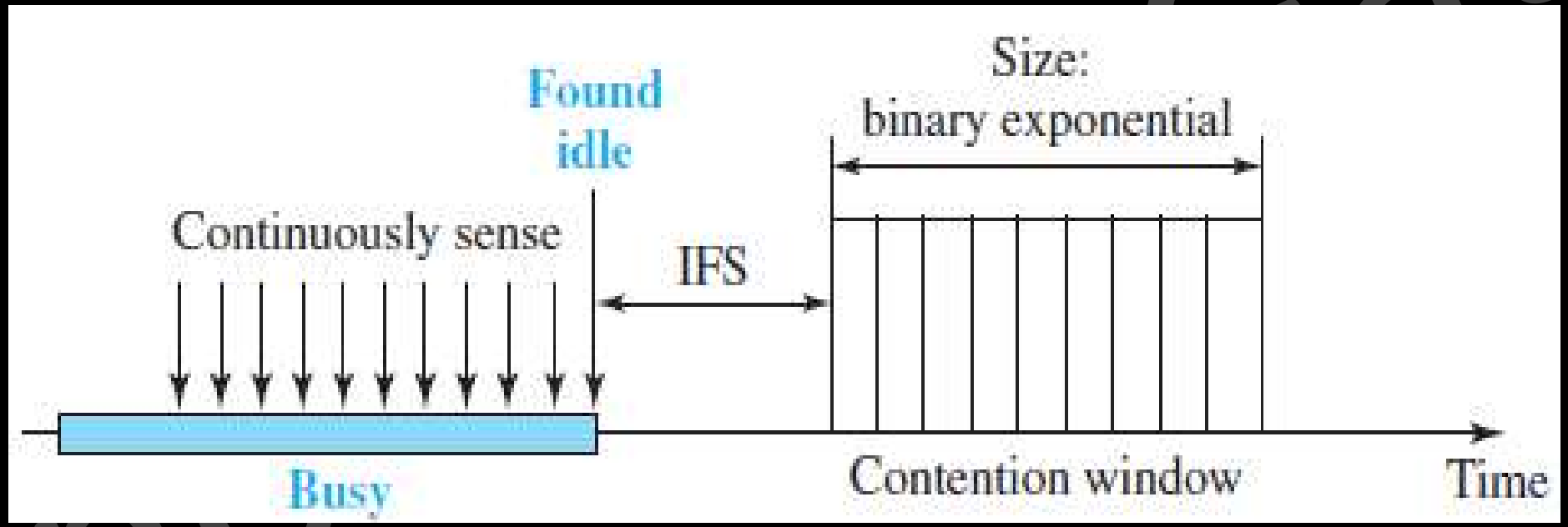
Contention Window

- The contention window is an amount of time divided into slots. A station that is ready to send chooses a random number of slots as its wait time.
- The number of slots in the window changes according to the binary exponential back-off strategy.
- This means that it is set to one slot the first time and then doubles each time.
- One interesting point about the contention window is that the station needs to sense the channel after each time slot.
- However, if the station finds the channel busy, it does not restart the process; it just stops the timer and restarts it when the channel is sensed as idle. This gives priority to the station with the longest waiting time.

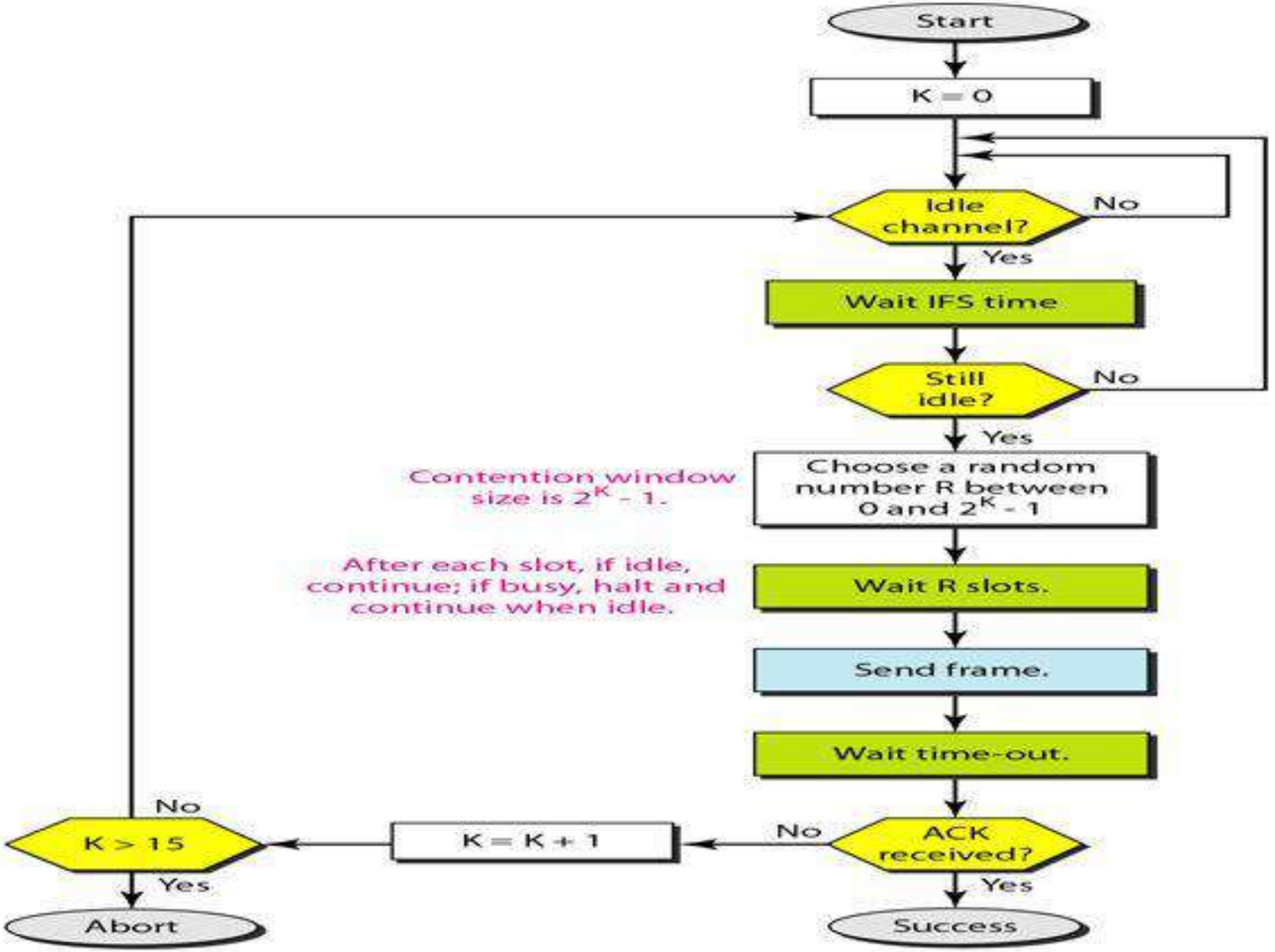


KnowledgeGate
www.knowledgegate.in

- **Acknowledgment**
  - With all these precautions, there still may be a collision resulting in destroyed data. In addition, the data may be corrupted during the transmission. The positive acknowledgment and the time-out timer can help guarantee that the receiver has received the frame.



www.knowledgegate.in


Start
K = 0
Idle channel?
No
Yes
Wait IFS time
Still idle?
No
Yes
Choose a random number R between 0 and 2^K - 1
Contention window size is 2^K - 1.
Wait R slots.
After each slot, if idle, continue; if busy, halt and continue when idle.
Send frame.
Wait time-out.
ACK received?
No
Yes
Success
K = K + 1
K > 15
No
Yes
Abort

FLOW CONTROL

- To prevent data overflow, the receiving device has a buffer memory where incoming data are temporarily stored until processed; the data processing speed often lags behind the transmission rate.
- If the buffer is nearing its capacity, the receiver somehow must communicates with the sender to reduce the number of frames sent or pause transmission temporarily, ensuring smooth data flow and preventing system overload.

KnowledgeGate

www.knowledgegate.in

ERROR CONTROL

- **(lost, out of order, corrupt) (detection and retransmission)**
- Error control, encompassing error detection and correction, enables the receiver to notify the sender about lost or damaged frames during transmission, facilitating the coordination for their retransmission.
- In the data link layer, error control is commonly implemented through the Automatic Repeat Request (ARQ) process, where detected errors trigger the retransmission of specific frames to maintain data integrity.

KnowledgeGate

Sliding Window Protocol

1. Stop-and-Wait Automatic Repeat Request
2. Go-Back-N Automatic Repeat Request
3. Selective Repeat Automatic Repeat Request

KnowledgeGate

Stop-and-Wait Automatic Repeat Request

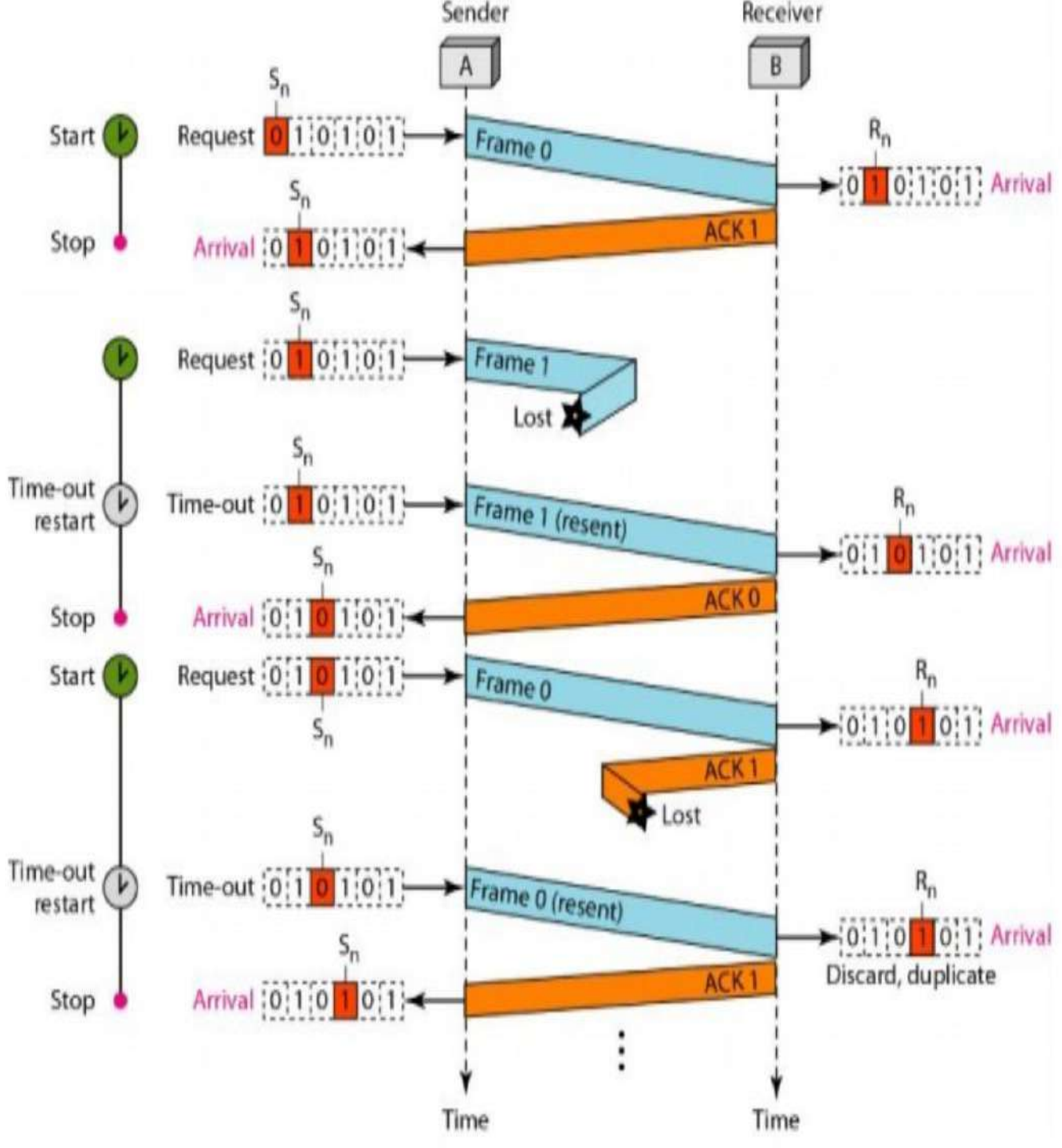
- When the frame arrives at the receiver site, it is checked and if it is corrupted, it is silently discarded. The detection of errors in this protocol is manifested by the silence of the receiver and no ack in send to sender.

- The sender keeps a copy of the sent frame. At the same time, it starts a timer. If the timer expires and there is no ACK for the sent frame, the frame is resent, the copy is held, and the timer is restarted.

- Lost frames are more difficult to handle than corrupted ones. The solution is to number the frames. When the receiver receives a data frame that is out of order, this means that frames were either lost or duplicated.

Knowle

www.kno



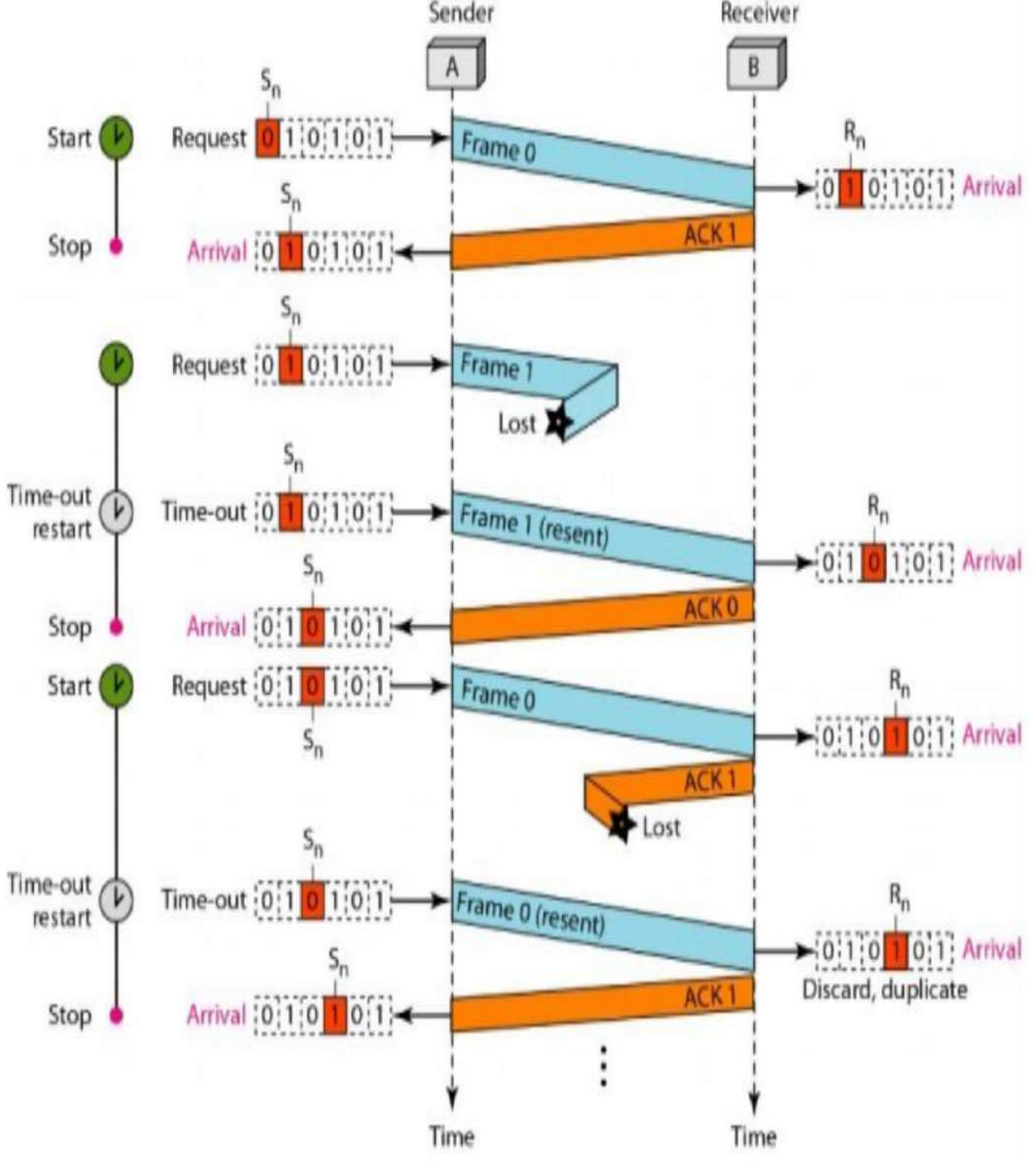
Sequence Numbers

- Frames are numbered using sequence numbers, added in a field within the data frame, to facilitate unambiguous communication with minimized frame size; the numbering can wrap around in a defined range.

- Sequence numbers only need to alternate between 0 and 1, enabling the receiver to differentiate between new frames and retransmitted frames without confusion, thereby optimizing the communication process.

Knowle

www.know

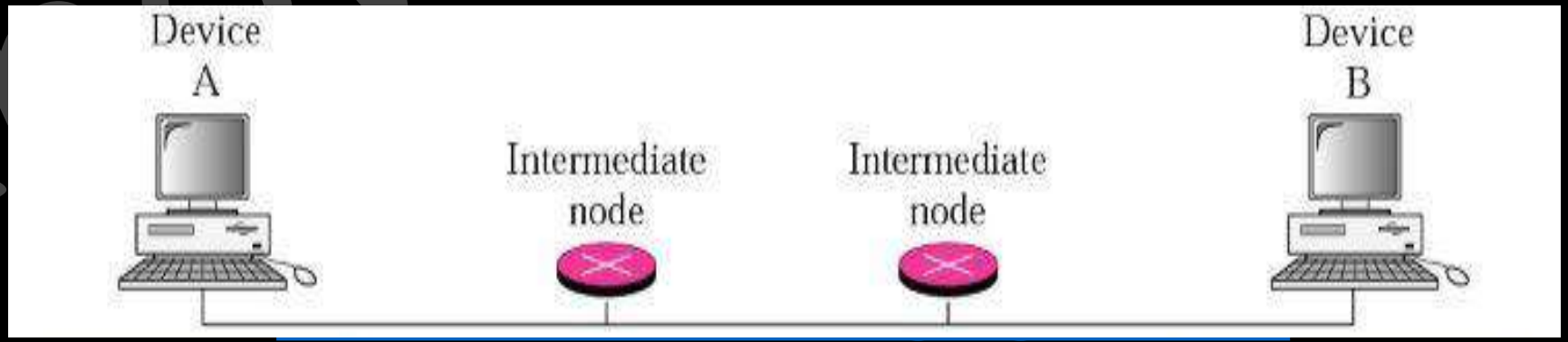


- **Propagation Delay:** Propagation delay is the time it takes for a bit to travel from point A to point B in the transmission media.
  T_P = (Distance) / (Propagation speed)

- **Transmission Delay (TT):** A sender needs to put the bits in a packet on the line one by one. If the first bit of the packet is put on the line at time t_1 and the last bit is put on the line at time t_2, transmission delay of the packet is $(t_2 - t_1)$.
  T_t = (Packet length (L)) / (Transmission rate or Bandwidth (B)) = L / B

- ***Queuing Delay*** **-** It is measured as the time a packet waits in the input queue and output queue of a router.
  - $Delay_{qu}$ = The time a packet waits in input and output queues in a router

- ***Processing Delay*** **-** It is the time required for a destination host to receive a packet from its input port, remove the header, perform an error detection procedure, and deliver the packet to the output port or deliver the packet to the upper-layer protocol (in the case of the destination host).
  - $Delay_{pr}$ = Time required to process a packet in a destination host

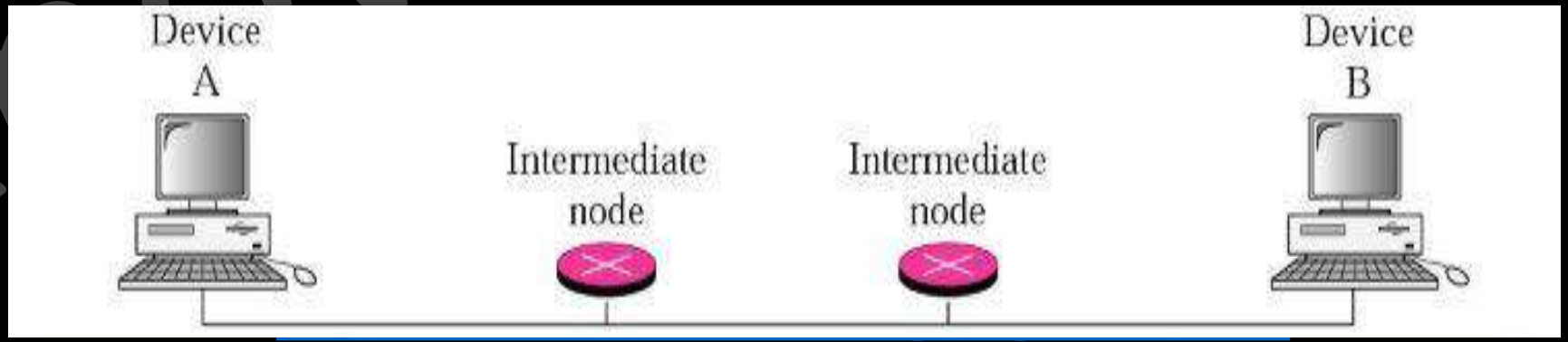


KnowledgeGate

Measuring Performance for Stop and Wait

1. Total time = $T_{t\,(data)} + T_{p\,(data)} + Dealy_{que} + Delay_{pro} + T_{t\,(ack)} + T_{p\,(ack)} + Dealy_{que} + Delay_{pro}$
2. Queuing delay and processing delays are generally kept 0.
   - Total Time = $T_{t\,(data)} + T_{p\,(data)} + T_{t\,(ack)} + T_{p\,(ack)}$
3. In general we have taken $T_{t\,(ack)}$ as negligible as the ack size is generally very less
   - Total Time = $T_{t\,(data)} + T_{p\,(data)} + T_{p\,(ack)}$
4. The T_p for data and ack are going to be same
   - Total Time = $T_{t\,(data)} + 2*T_p$

- Note:- Some times *2 * T_p time is also called Round Trip Time (RTT)*



KnowledgeGate

- **Efficiency(η)**:-
  - Useful Time / Total Cycle time = T_t / T_t + 2 * T_p
  - Here, Useful time in the entire cycle time is T_t and for the rest 2 * T_p time we are waiting for the processing, whereas instead of waiting we could have sent more packets.
  - Dividing numerator and denominator with T_t, we get: η = 1 / 1 + (2 * T_p/T_t)
  - So, η = 1 / 1 + 2a, (where a = T_p / T_t)

- Effective Bandwidth / Throughput / Bandwidth Utilization is calculated as:
- Throughput = η * B (efficiency * bandwidth)

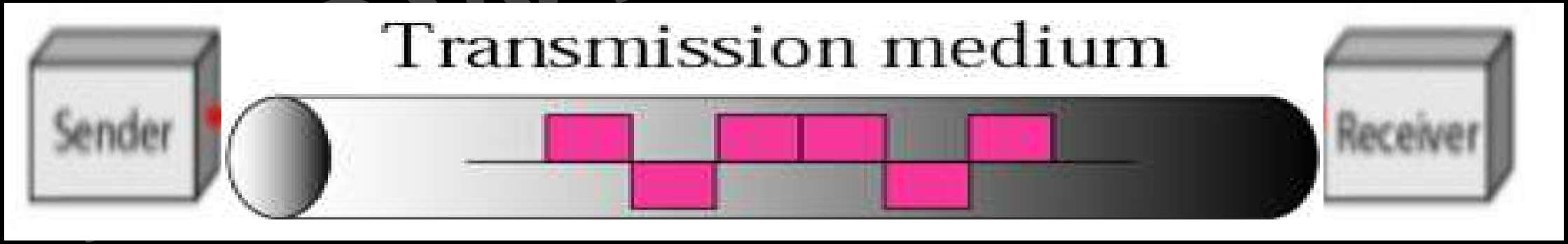
KnowledgeGate

www.knowledgegate.in

Go-Back-N Automatic Repeat Request

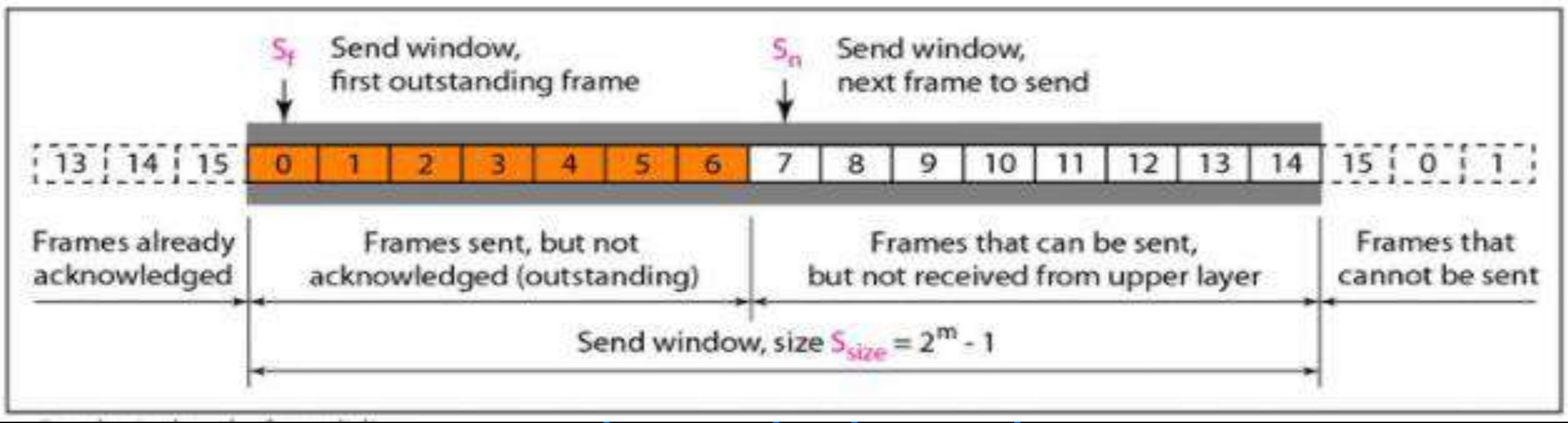
- The Stop-and-Wait ARQ method can be inefficient for channels with high bandwidth, resulting in underutilization of the available data pipeline. Solution is Go-Back-N.

- To enhance transmission efficiency, the Go-Back-N ARQ protocol allows multiple frames to be transmitted before acknowledgments are received, utilizing a "sliding window" concept.



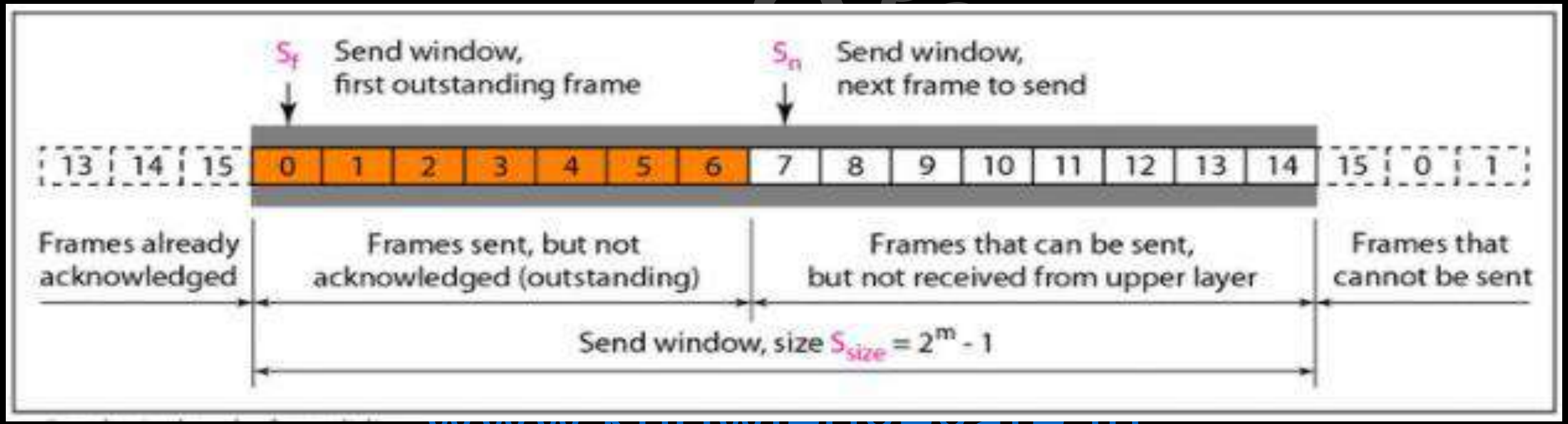
www.knowledgegate.in

- The send window, an imaginary concept, covers the sequence numbers of frames that can be in transit, having a maximum size of $2^m - 1$.
- Sequence numbers are divided into four regions: the first represents acknowledged frames (no copies kept), the second represents frames sent but with unknown status (outstanding frames), the third denotes frames ready to be sent pending data packets from the network layer, and the fourth signifies sequence numbers inaccessible until the window slides further.



www.knowledgegate.in

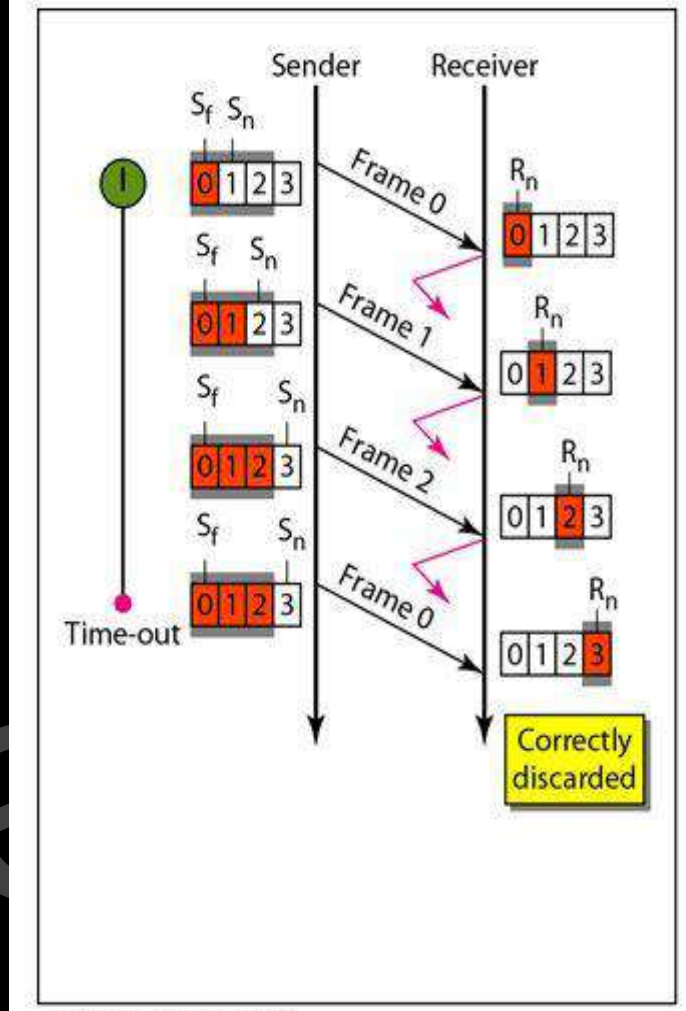
- The send window is characterized by three variables:
  - S_f (marks the first outstanding frame)
  - S_n (indicates the next frame to be dispatched)
  - S_{size} (represents the fixed window size).
- Conversely, the receive window, with a constant size of one, ensures the receipt of correct data frames and the issuance of appropriate acknowledgments, discarding and necessitating the resending of any out-of-order frames.



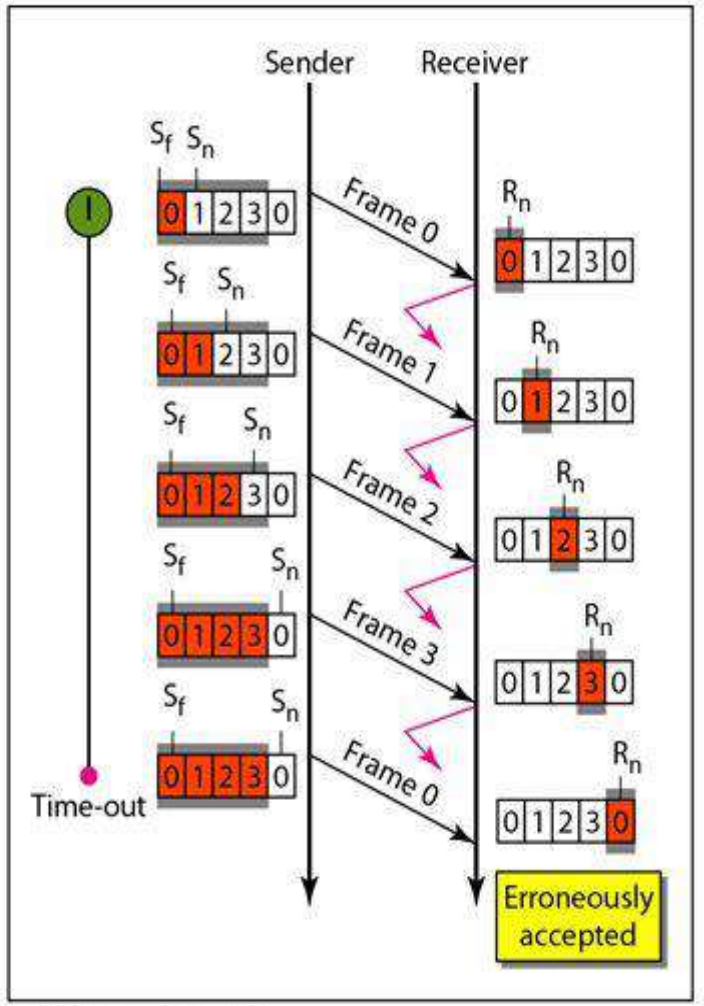
KnowledgeGate
www.knowledgegate.in

Timers

- Although there can be a timer for each frame that is sent, in our protocol we use only one. The reason is that the timer for the first outstanding frame always expires first; we send all outstanding frames when this timer expires.



a. Window size < 2^m



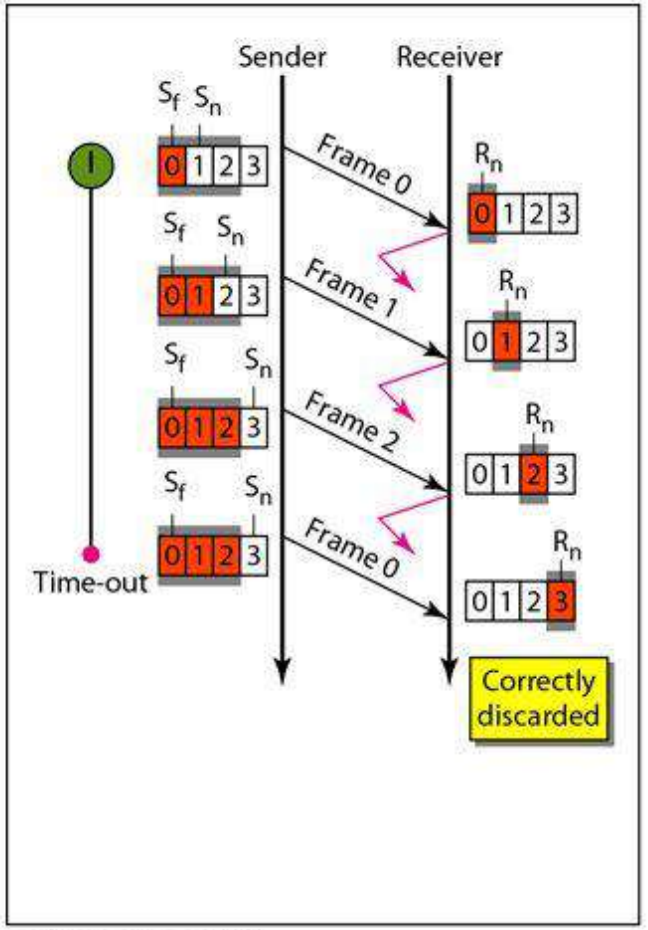
b. Window size = 2^m

Knowle

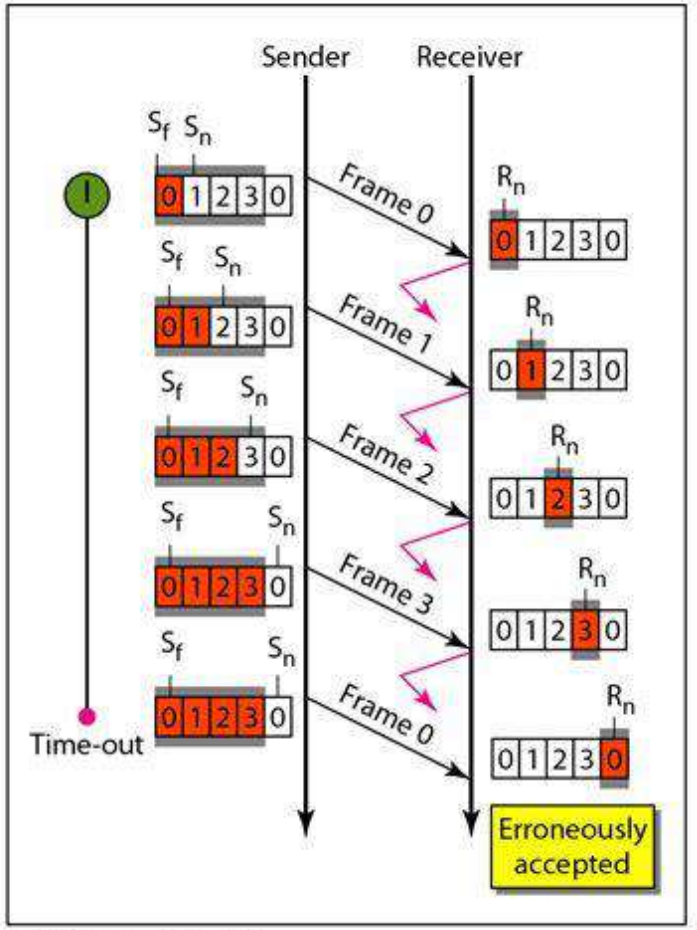
www.knowledgegate.in

Acknowledgment

- The receiver sends a positive acknowledgment for frames that arrive intact and in the correct order, whereas it remains silent and discards frames that are damaged or received out of sequence, maintaining a focus on the expected frame's arrival.
- The receiver's silence triggers the timer of the unacknowledged frame at the sender site to expire, prompting the sender to resend all frames from the one with the expired timer
- The receiver can issue a cumulative acknowledgment for multiple frames, enhancing efficiency.



a. Window size < 2^m



b. Window size = 2^m

Knowle

www.knowledgegate.in

Sequence and Acknowledgement Numbers

- To improve the efficiency of transmission (to fill the pipe), multiple packets must be in transition while the sender is waiting for acknowledgment

- In order to maximize the efficiency, the window size (W_s) = (1+ 2a)

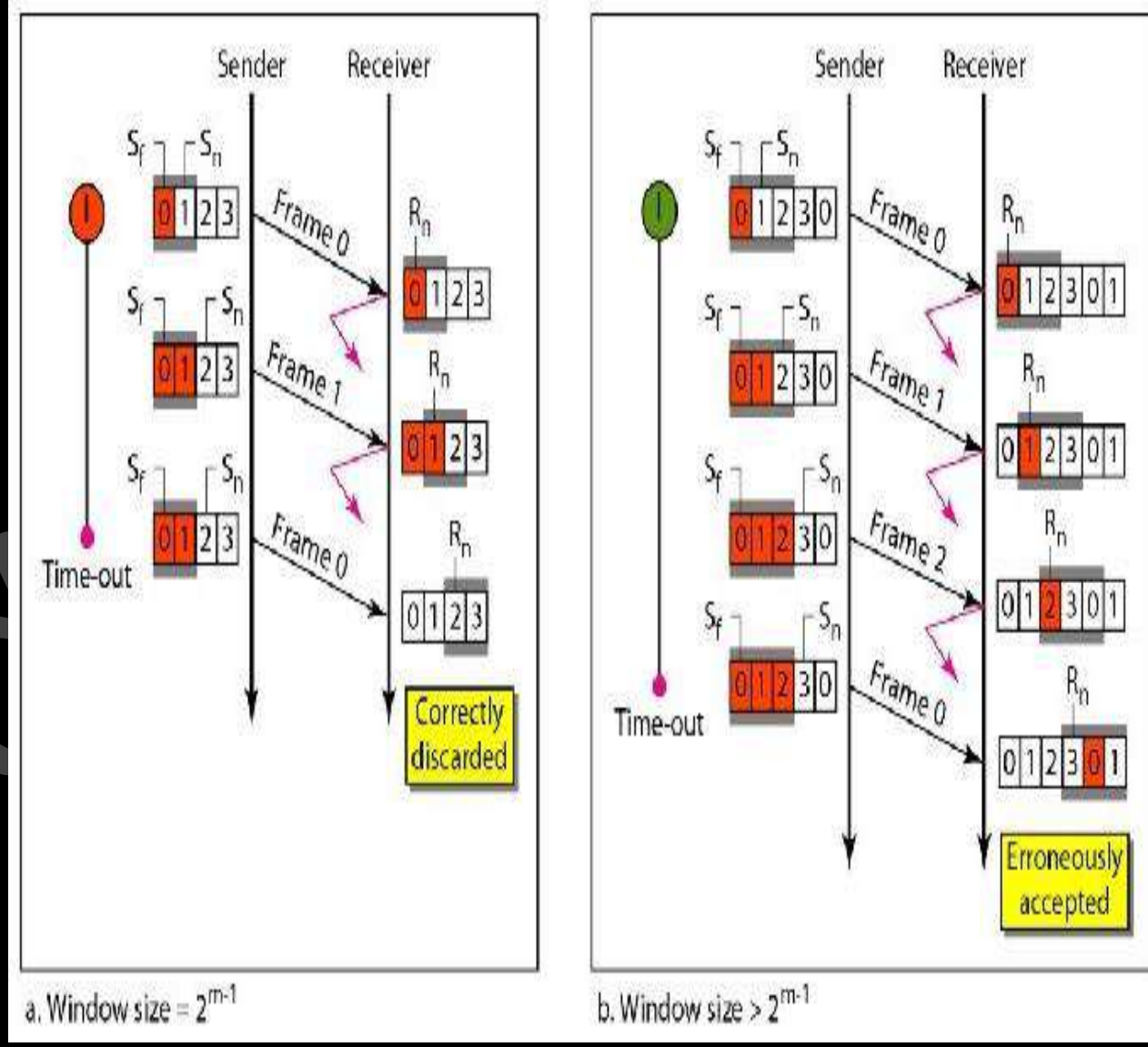
- Number of bits required for sequence numbers = ceil ($\log_2$ (1 + 2a))

KnowledgeGate

www.knowledgegate.in

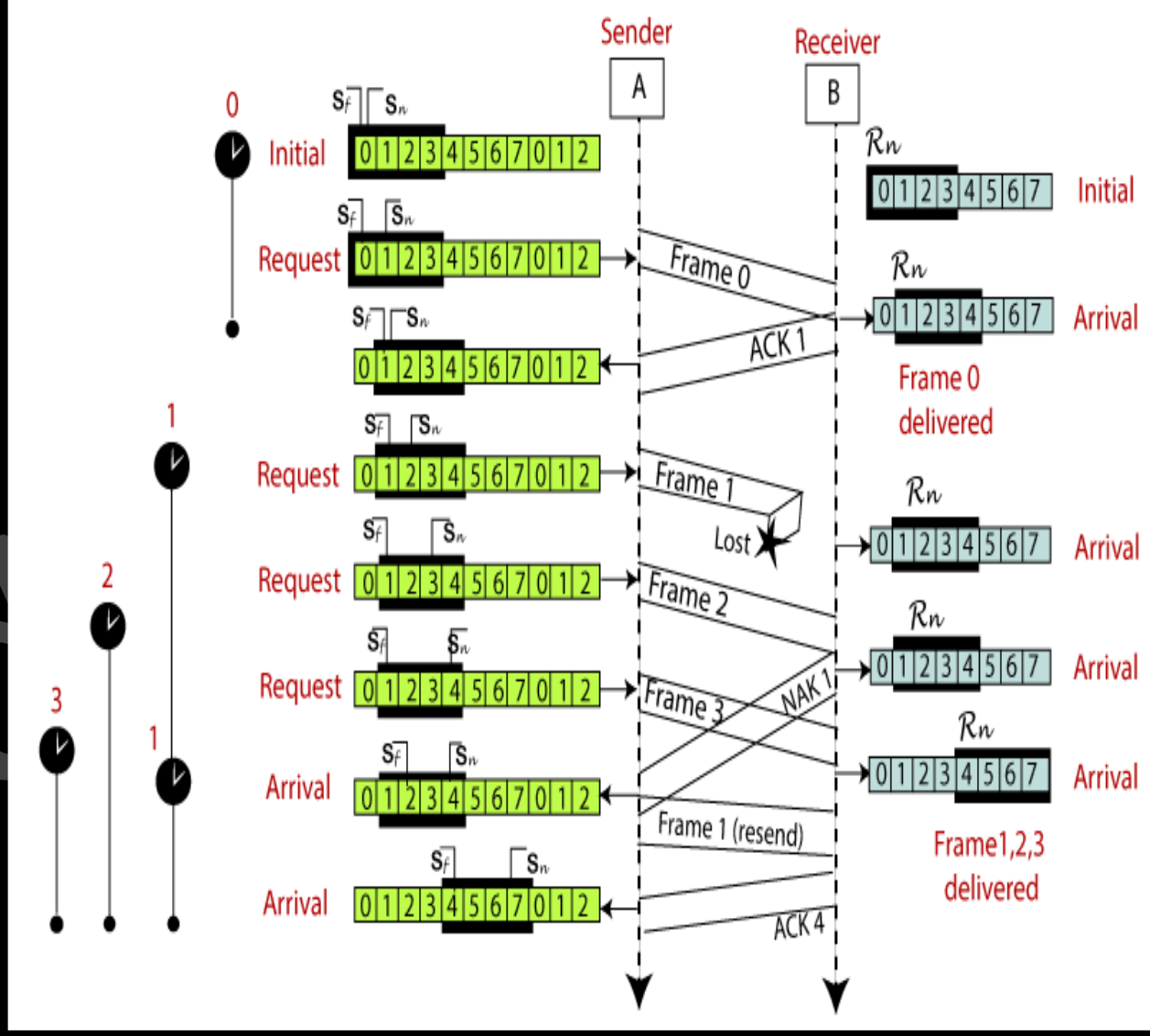
Selective Repeat Automatic Repeat Request

- Go-Back-N ARQ makes it easier for the receiver by only keeping track of one thing and ignoring frames that arrive in the wrong order. However, if the connection is not great, it can waste time and data by having to resend a bunch of frames often.
- If the connection is not stable, the Selective Repeat ARQ is better as it only resends the frames that didn't arrive correctly, saving time and data. But, it makes the receiver's job a bit tougher.
- The Selective Repeat ARQ also lets the receiver gather frames even if they come in the wrong order and keeps them until they can be arranged correctly and passed on, using equal room for storing and sending frames to work more efficiently.



Knowledge
www.knowledgegate.in

- In Selective Repeat ARQ, the data windows for sending and receiving are kept fairly small to manage data better 2^{m-1}; each frame sent has its own countdown timer, making the tracking of requests quite similar to earlier methods but more detailed when data arrives.
- When a clear NAK message arrives, we simply send the frame again; if a clear ACK message comes, we clear old data, stop the related countdown, and move the window's starting point. If a timeout happens, only the late frame is sent again.



Knowledge

www.knowledgegate.in

Piggybacking

- The discussed protocols are unidirectional, primarily allowing data frames to move in one direction, although control signals like ACK and NAK can go both ways. In practice, data frames flow bidirectionally, requiring a method called piggybacking to enhance bidirectional protocol efficiency.

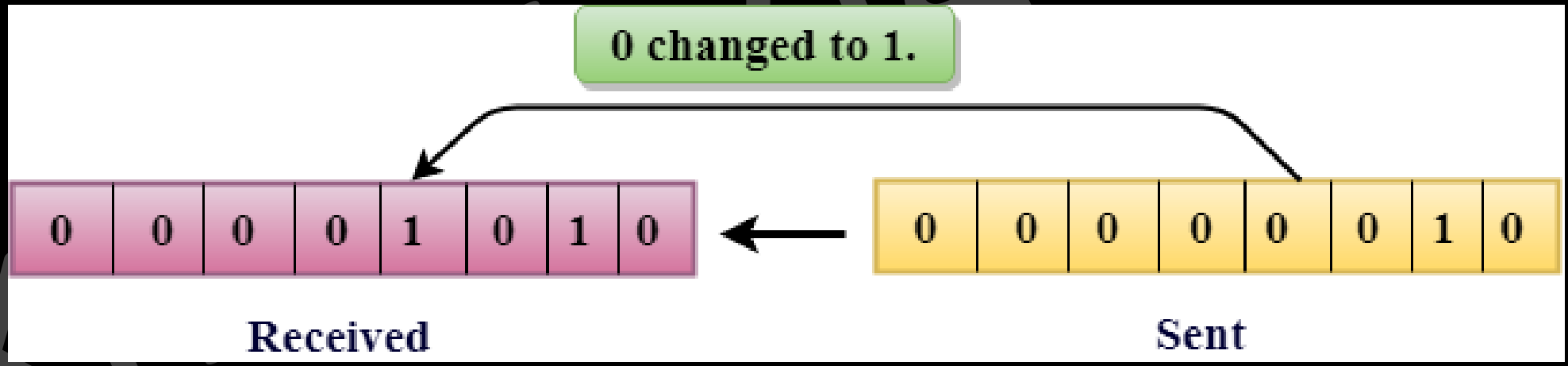
- Piggybacking enables a data frame moving from A to B to also convey control information regarding B's frames and vice versa, streamlining the communication process. Every node maintains two operational windows: a send and a receive window, along with a timer overseeing three events: request, arrival, and time-out.

- This strategy complicates the arrival event, necessitating the simultaneous handling of control information and the actual data frame within a singular event, employing both windows at the respective site. A uniform algorithm, albeit complex, must be employed by both sites to effectively manage the unified arrival events.

KnowledgeGate

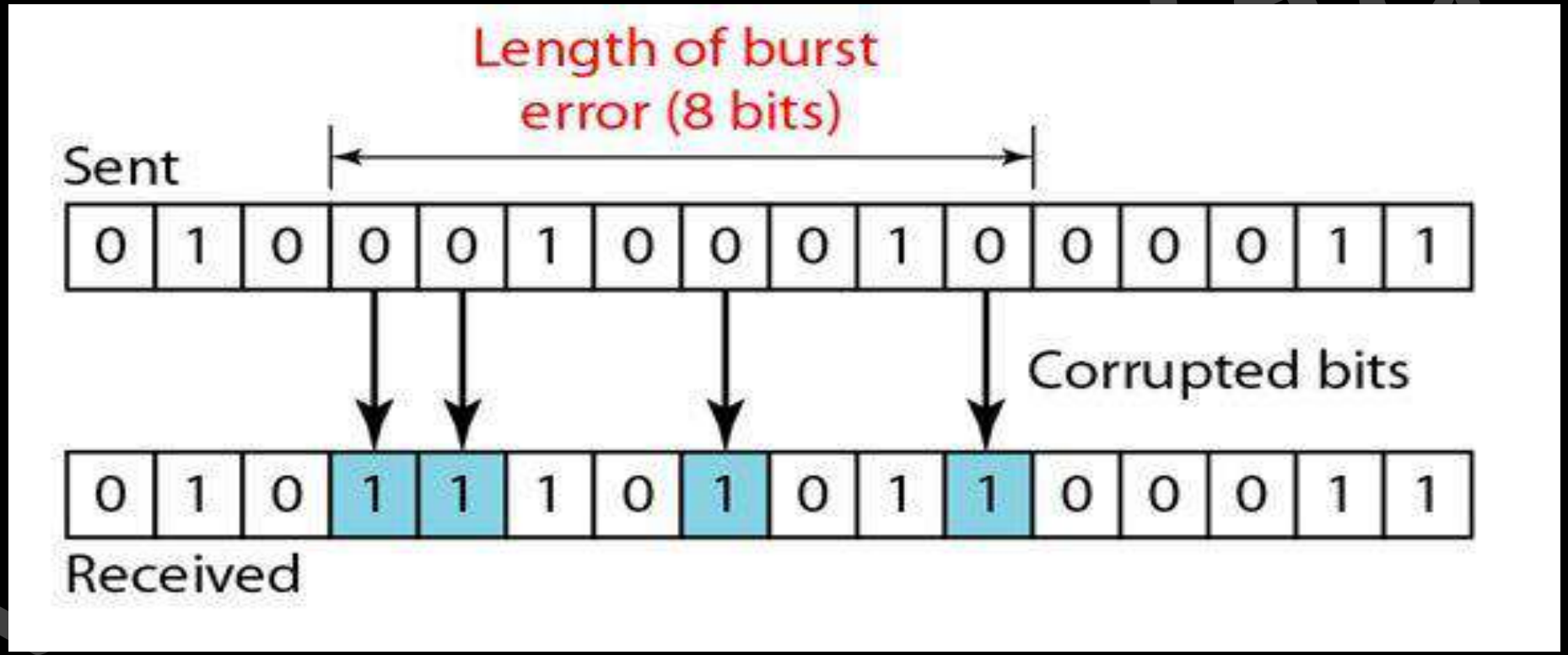
Types of Errors

- Data transmitted between nodes can sometimes be altered or corrupted during transmission. This corruption can manifest as either a single-bit error or a burst error.
- A single-bit error refers to a scenario where only one bit in a data unit (like a byte or packet) is changed, either from 1 to 0 or vice versa. This type of error is less common.



KnowledgeGate

- Burst error, on the other hand, involves the alteration of two or more bits within a data unit. These changes might not be in successive bits. It's a more common error type as noise typically affects a set of bits, the extent of which is influenced by the data rate and noise duration.



www.knowledgegate.in

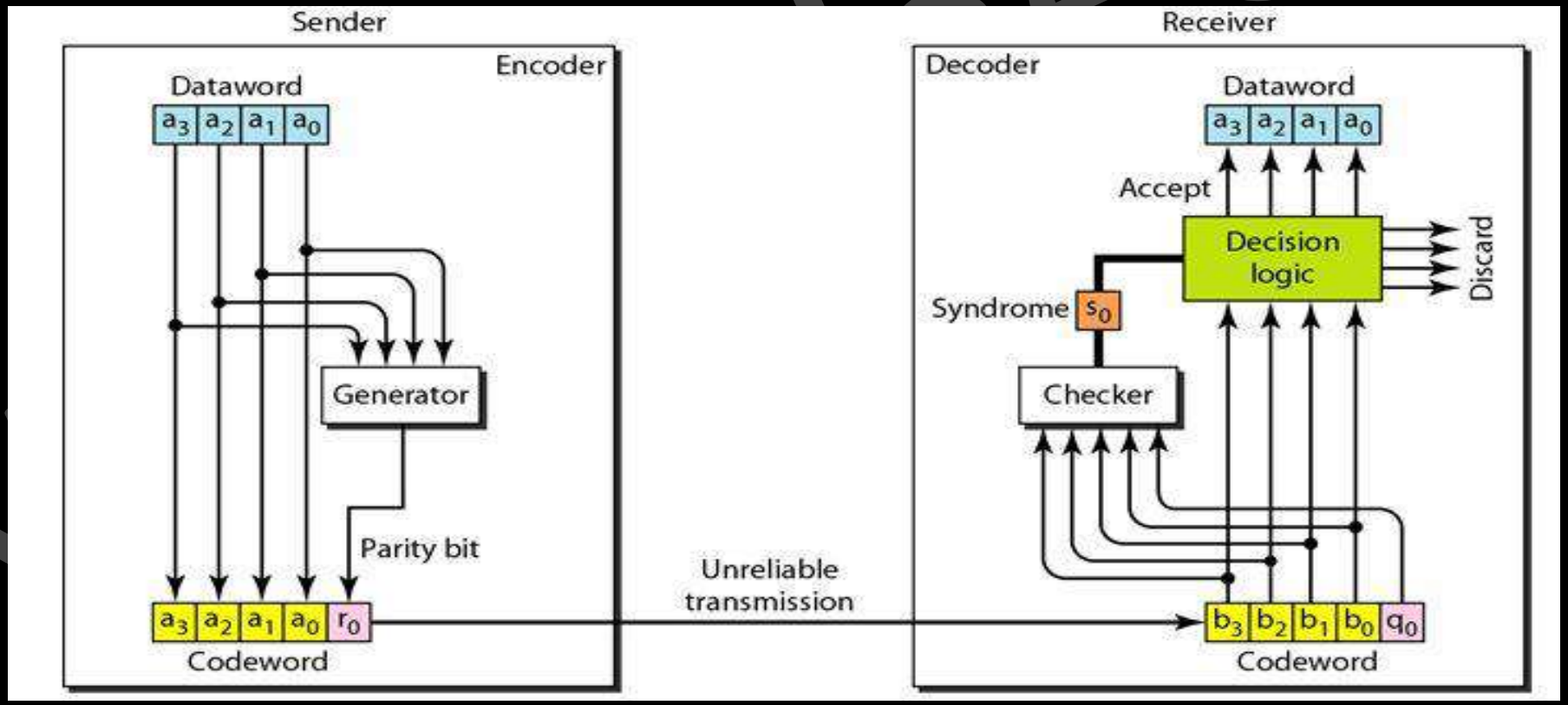
Hamming Distance

- Hamming distance, represented as d(x, y), is a measure of the difference between two words of equal size, calculated by counting the number of dissimilar bits at the corresponding positions in the two words.
- To find the Hamming distance, perform an XOR operation on the two words and count the number of 1s in the outcome.
- For instance, the Hamming distance d(000, 011) equals 2, illustrated through the XOR operation: 000 XOR 011 equals 011, which has two 1s.
- The Hamming distance d(10101, 11110) is _____ ?

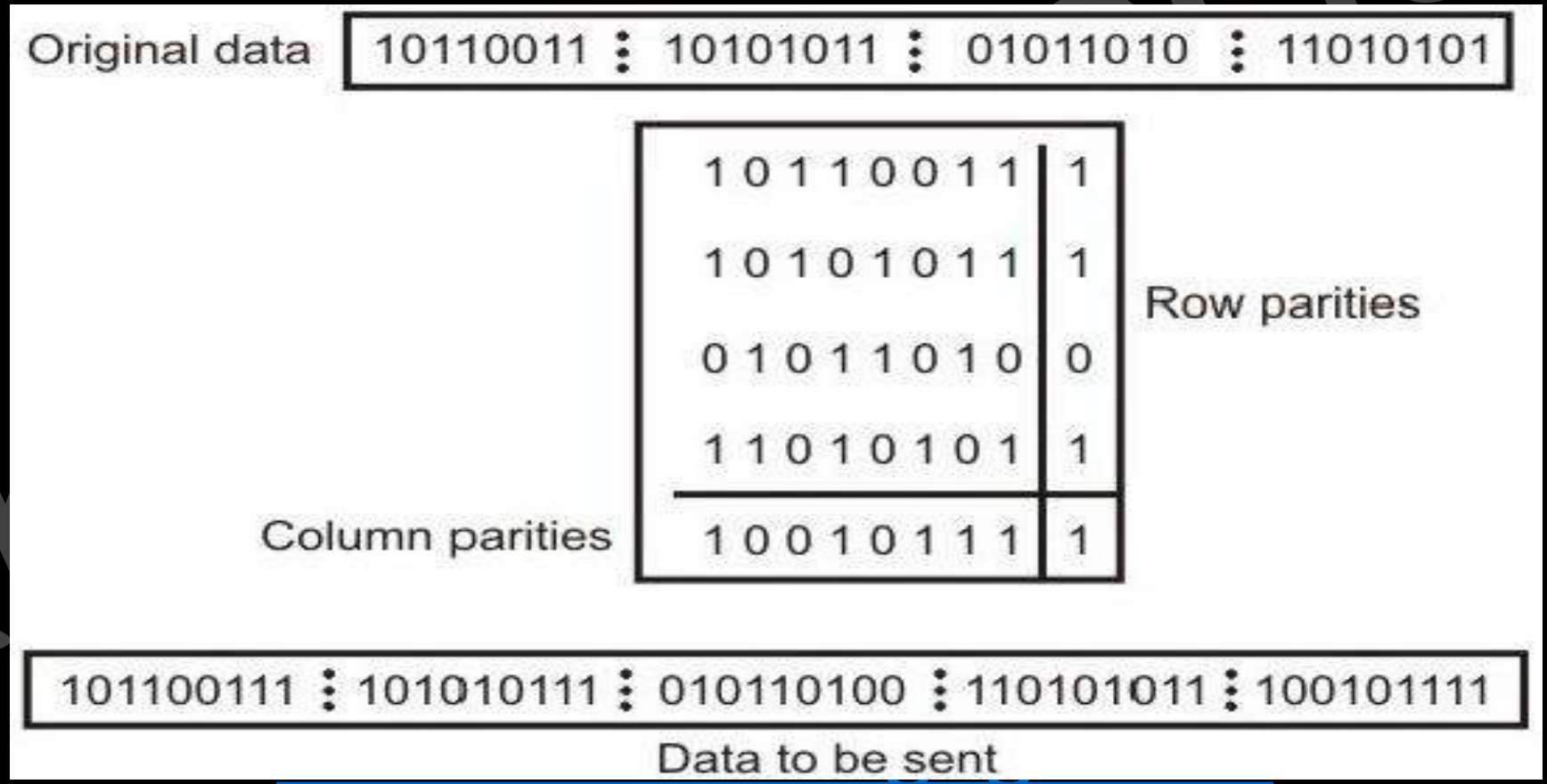
KnowledgeGate

Simple Parity-Check Code

- The simple parity-check code expands a k-bit dataword to an n-bit codeword (n = k + 1), with the additional bit serving as a parity bit to ensure an even total count of 1s in the codeword, though some variations aim for an odd count instead.
- This code acts as a single-bit error-detecting mechanism, characterized by n = k + 1 and a minimum Hamming distance (d_{min}) of 2.
- It is designed to identify an odd number of errors within the data transmitted.



- In the two-dimensional parity check, data words are arranged in a table with rows and columns. A special bit called a parity-check bit is added to each row and column to help identify errors in the data.
- After the table is sent, the receiver checks the parity bits for each row and column to find "syndromes", which are indicators of where errors might have occurred.
- This method can identify up to three errors in the table, but might miss errors if four or more bits are affected.



Hamming Codes

- Now let us discuss a category of error-correcting codes called Hamming codes. These codes were originally designed to detect up to two errors or correct one single error.

- First let us find the relationship between n and k in a Hamming code. The values of n and k are then calculated from r as $k = 2^r - r - 1$.

Bit position		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	
Encoded data bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15	
Parity bit coverage	p1	X		X		X		X		X		X		X		X		X		X		
	p2		X	X			X	X			X	X			X	X			X	X		...
	p4				X	X	X	X					X	X	X	X					X	
	p8								X	X	X	X	X	X	X	X						
	p16																X	X	X	X	X	

Richard Hamming

www.knowledgegate.in

- For example, if m = 3, then n = 7 and k = 4. This is a Hamming code C(7, 4) with dmin =3. shows the datawords and codewords for this code.

- Position of parity bits 2^0, 2^1, 2^2, …,2^n

- If we use even parity then
  - For parity bit P_1 we check position 1, 3, 5, 7 (take 1, leave 1)(which have 1 at 2^0)
  - For parity bit P_2 we check position 2, 3, 6, 7 (take 2, leave 2)(which have 1 at 2^1)
  - For parity bit P_4 we check position 4, 5, 6, 7 (take 4, leave 4)(which have 1 at 2^2)

7	6	5	4	3	2	1
111	110	101	100	011	010	001
D_4	D_3	D_2	P_4	D_1	P_2	P_1

KnowledgeGate

www.knowledgegate.in

CHECKSUM

- The checksum method is a way to check for errors when sending a list of numbers over the internet. In this method, we add up all the numbers we want to send and send this total sum, along with the original numbers.

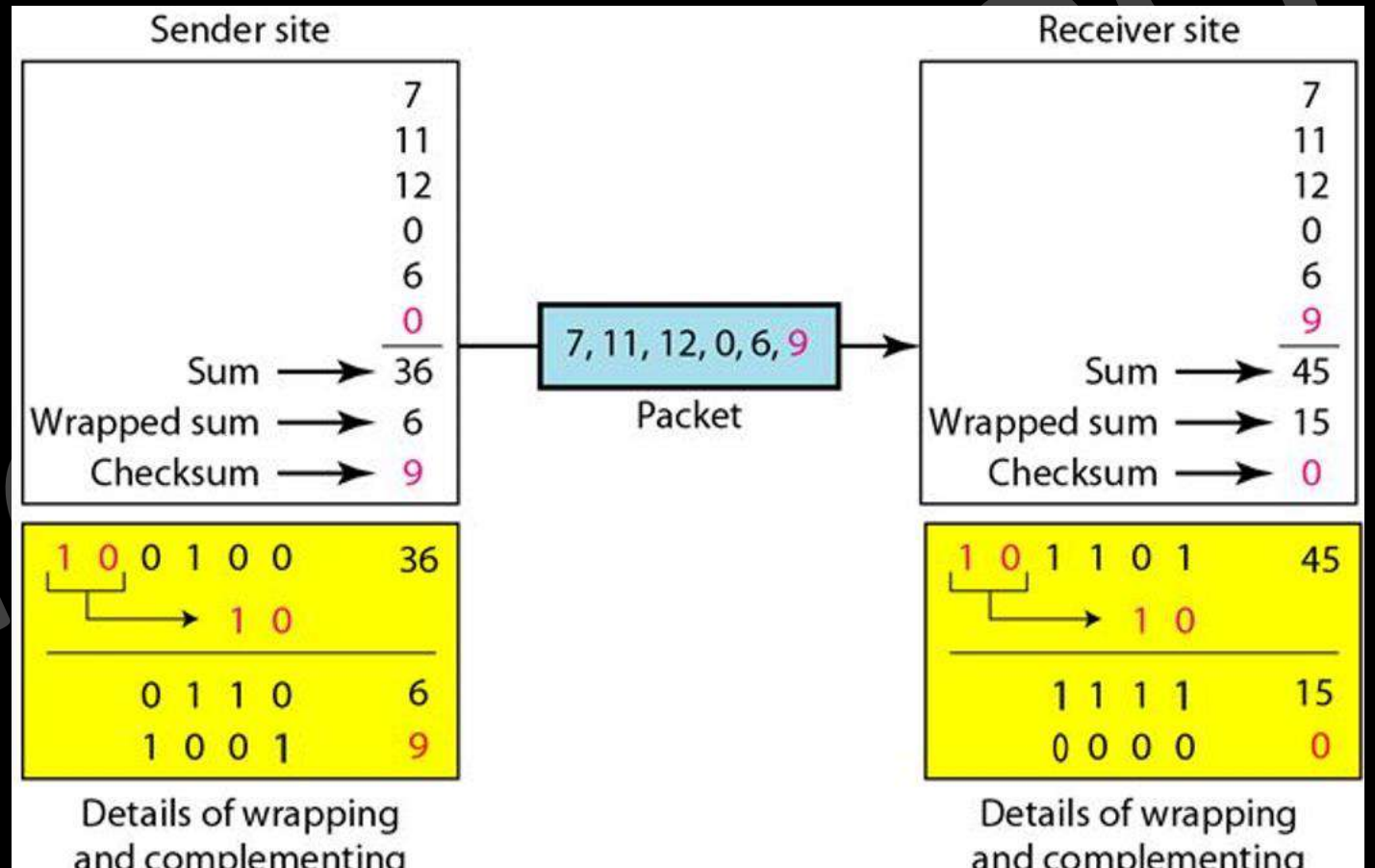
- At the receiving end, the numbers are added up again and compared with the received sum. If everything adds up correctly, it means that no errors occurred during transmission. Otherwise, it indicates that there was an error and the data is not accepted.For example, if the set of numbers is (7, 11, 12, 0, 6), we send (7, 11, 12, 0, 6, 36), where 36 is the sum of the original numbers. The receiver adds the five numbers and compares the result with the sum.

- We can make the job of the receiver easier if we send the negative (complement) of the sum, called the checksum. In this case, we send (7, 11, 12, 0, 6, -36). The receiver can add all the numbers received (including the checksum). If the result is 0, it assumes no error; otherwise, there is an error.

KnowledgeGate

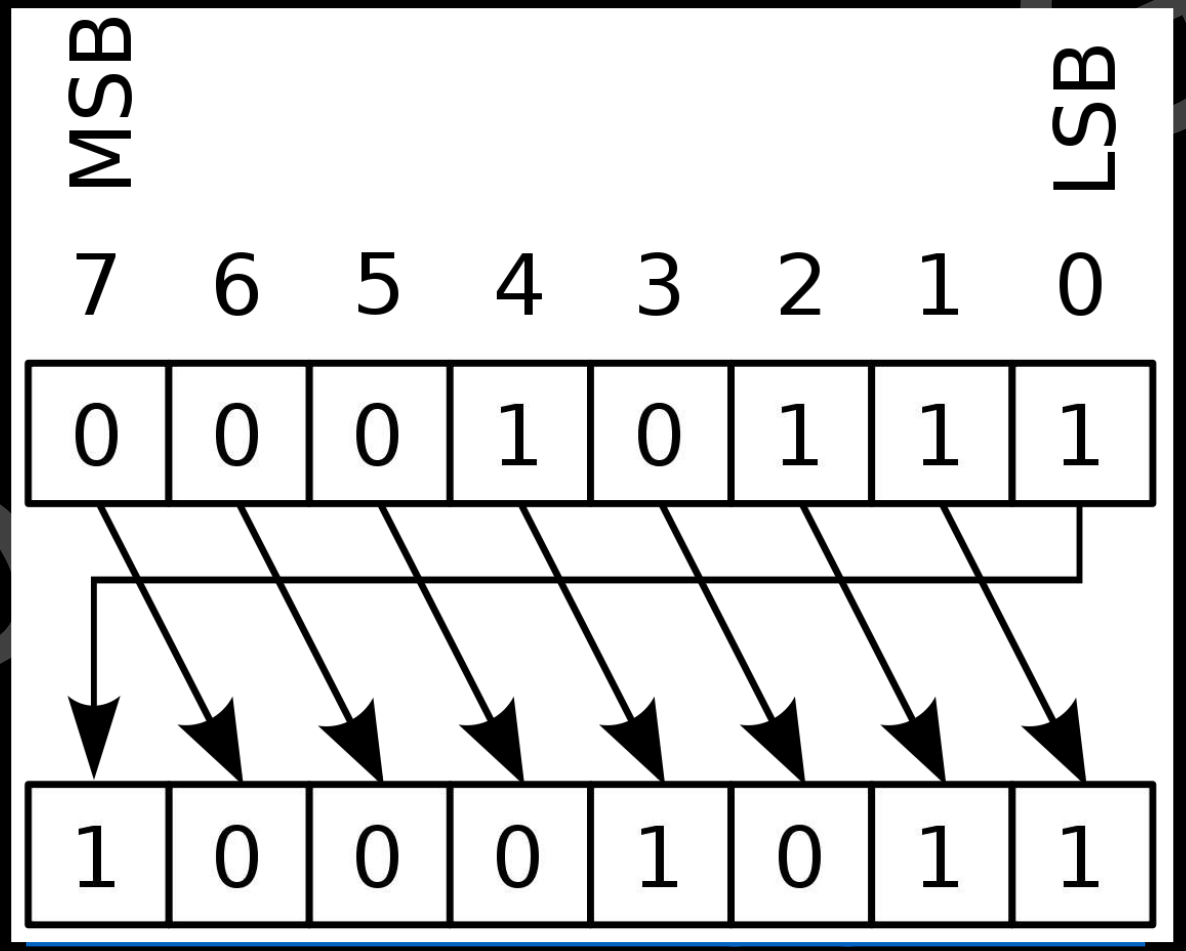
One's Complement

- The previous example has one major drawback. All of our data can be written as a 4-bit word (they are less than 15) except for the checksum.
- One solution is to use one's complement arithmetic. In one's complement arithmetic, a negative number can be represented by inverting all bits (changing a 0 to a 1 and a 1 to a 0). This is the same as subtracting the number from $2^n - 1$. If the number has more than n bits, the extra leftmost bits need to be added to the n rightmost bits (wrapping).



CYCLIC CODES

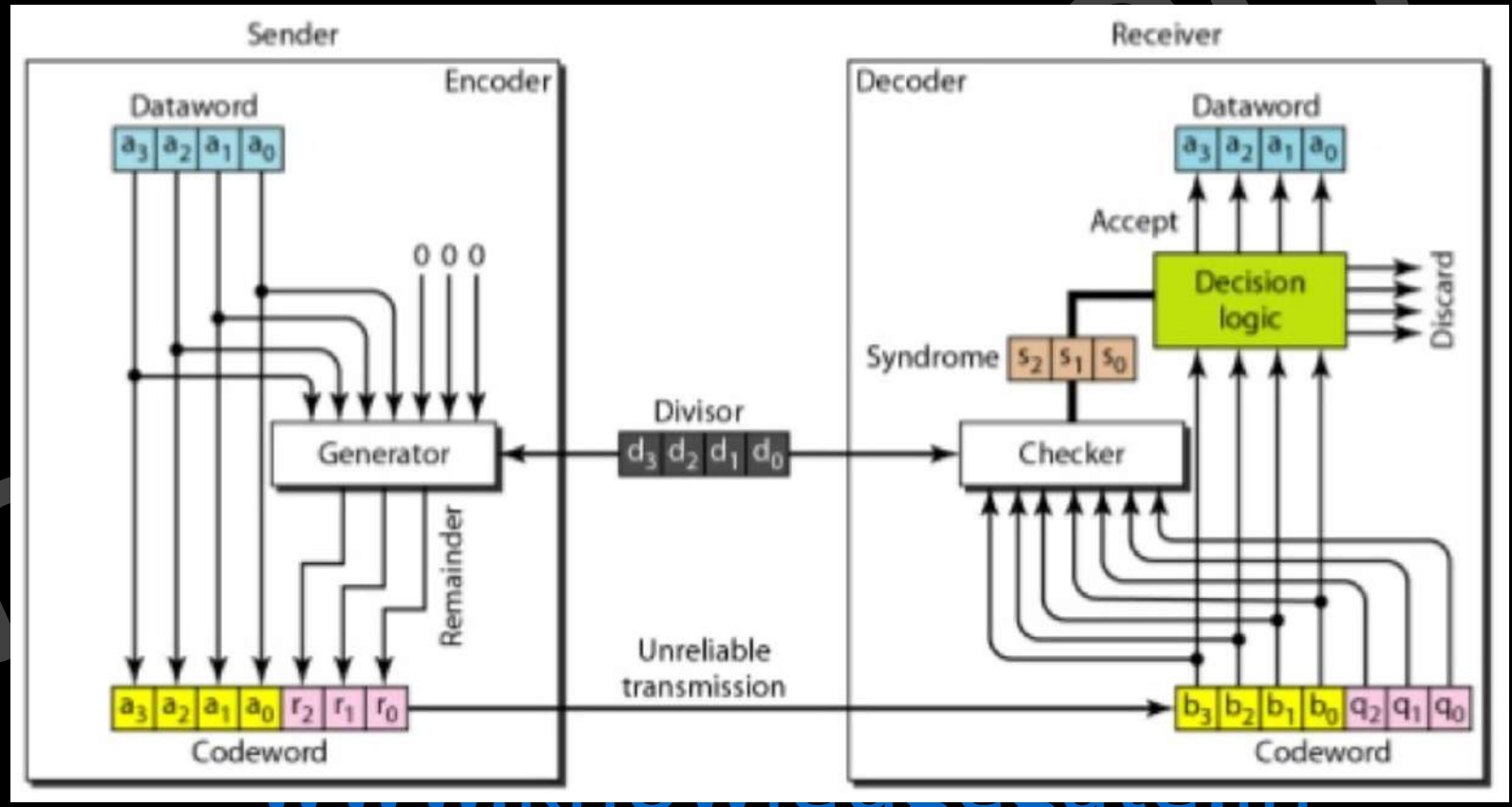
- Cyclic codes are special linear block codes with one extra property. In a cyclic code, if a codeword is cyclically shifted (rotated), the result is another codeword.



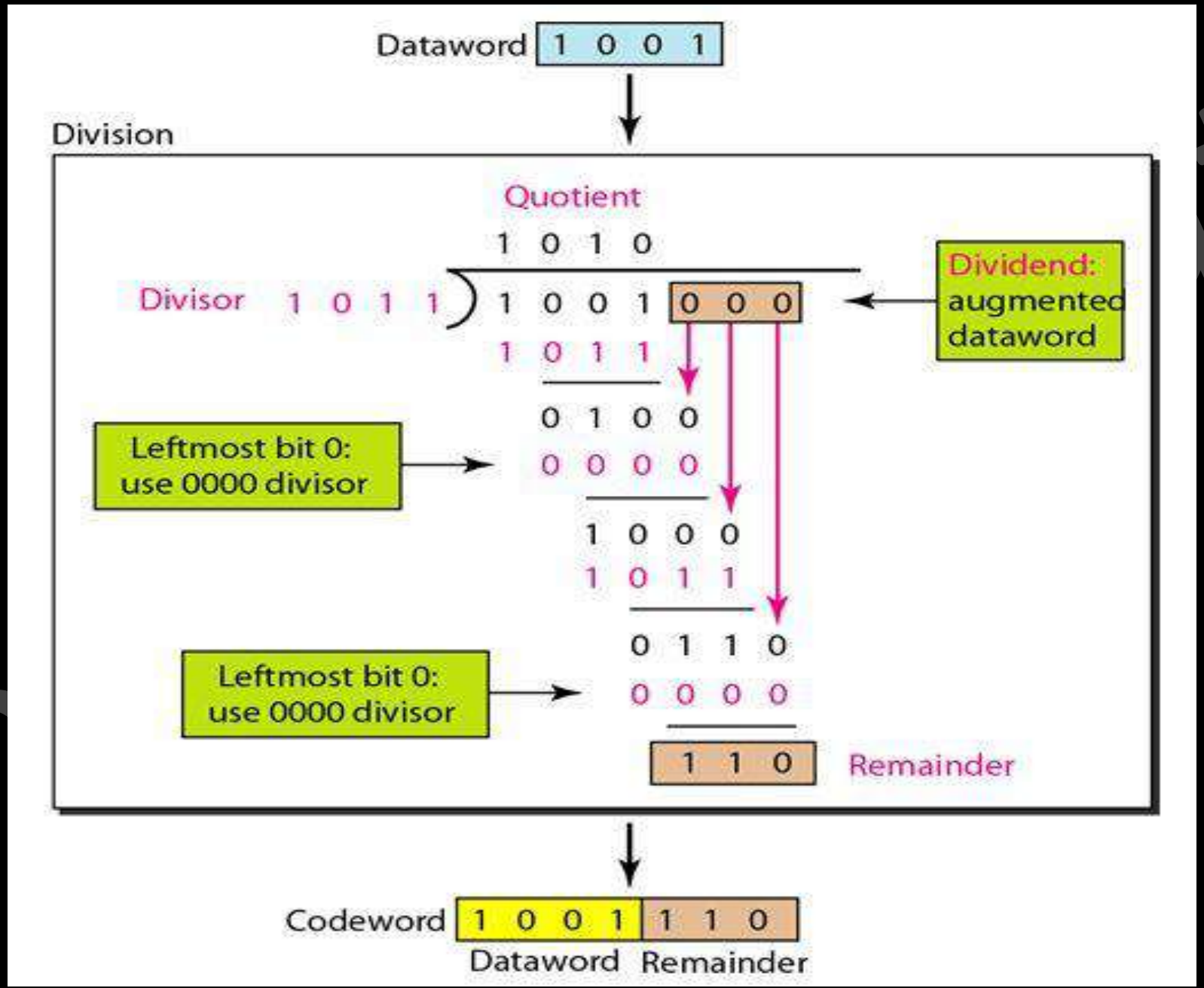
Know Gate

Cyclic Redundancy Check

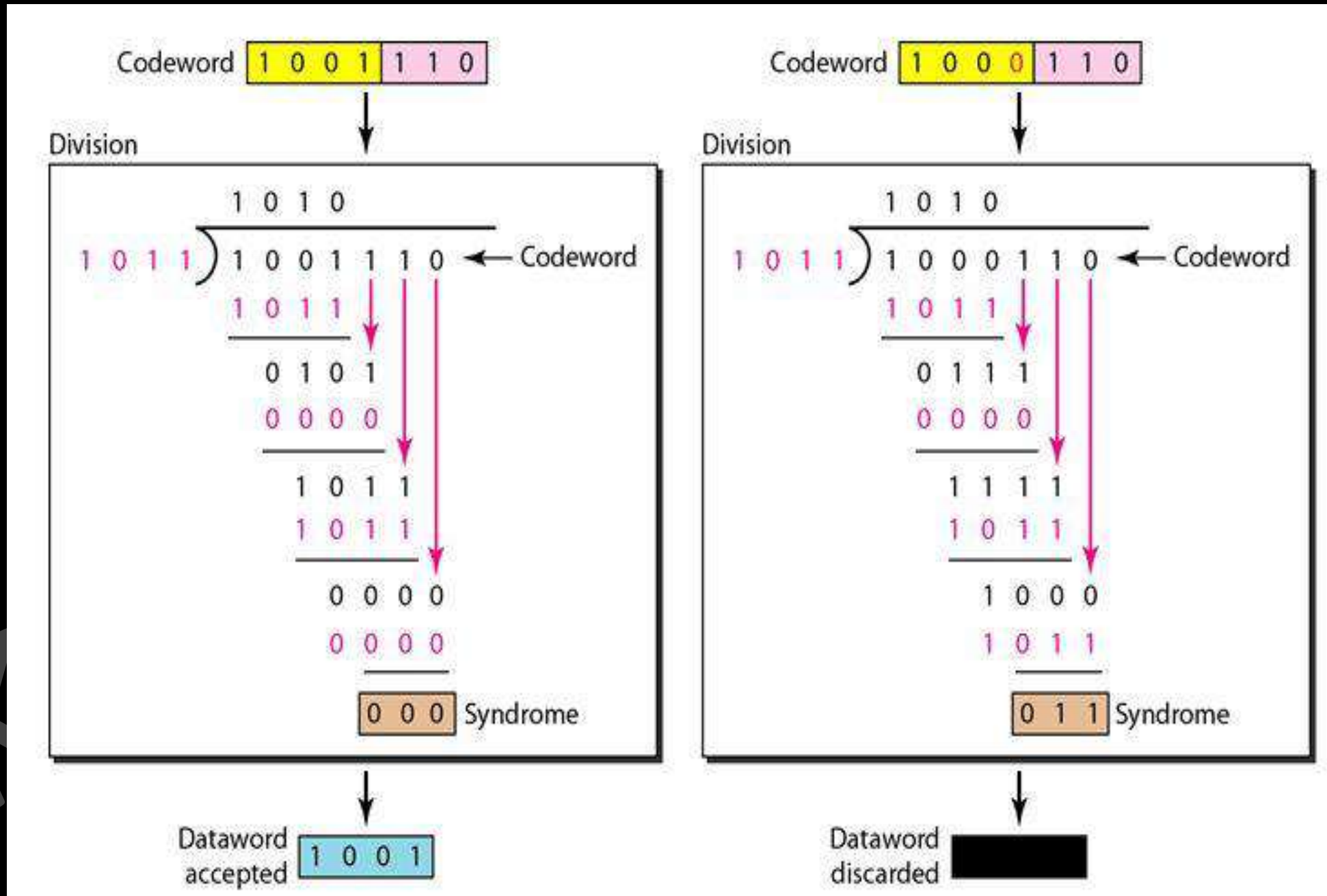
- We can create cyclic codes to correct errors. In the encoder, the dataword has k bits (4 here); the codeword has n bits (7 here). The size of the dataword is augmented by adding n - k (3 here) O^s to the right-hand side of the word.
- The n-bit result is fed into the generator. The generator uses a divisor of size n - k + 1 (4 here), predefined and agreed upon. The generator divides the augmented dataword by the divisor (modulo-2 division). The quotient of the division is discarded; the remainder (r_2 r_1 r_0) is appended to the dataword to create the codeword.



Encoder

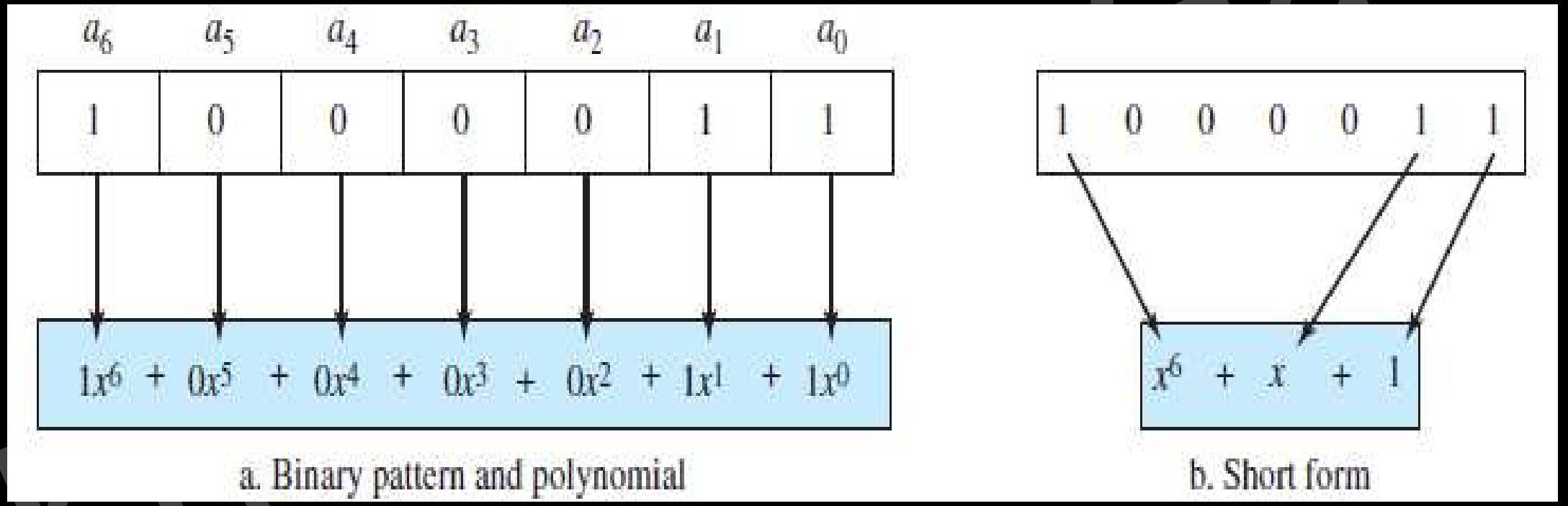

Dataword 1 0 0 1
Division
Quotient
1 0 1 0
Divisor 1 0 1 1
1 0 0 1 0 0 0
Dividend: augmented dataword
1 0 1 1
0 1 0 0
Leftmost bit 0: use 0000 divisor
0 0 0 0
1 0 0 0
1 0 1 1
0 1 1 0
Leftmost bit 0: use 0000 divisor
0 0 0 0
1 1 0
Remainder
Codeword 1 0 0 1 1 1 0
Dataword
Remainder

Decoder

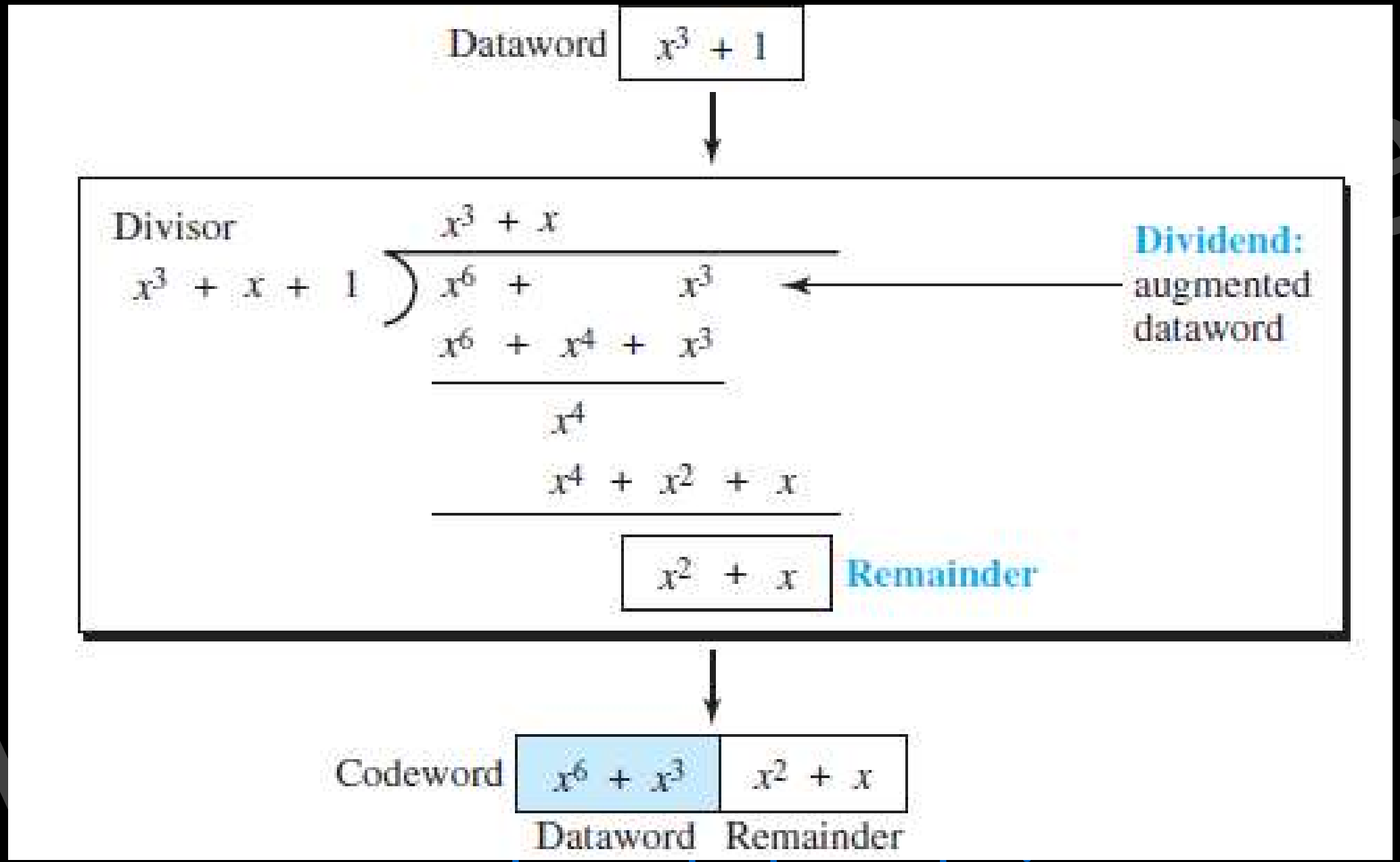

Codeword 1 0 0 1 1 1 0
Division
1 0 1 0
1 0 1 1) 1 0 0 1 1 1 0 ← Codeword
1 0 1 1
0 1 0 1
0 0 0 0
1 0 1 1
1 0 1 1
0 0 0 0
0 0 0 0
0 0 0 Syndrome
Dataword accepted 1 0 0 1
Codeword 1 0 0 0 1 1 0
Division
1 0 1 0
1 0 1 1) 1 0 0 0 1 1 0 ← Codeword
1 0 1 1
0 1 1 1
0 0 0 0
1 1 1 1
1 0 1 1
1 0 0 0
1 0 1 1
0 1 1 Syndrome
Dataword discarded

Polynomials

- A pattern of 0s and 1s can be represented as a **polynomial** with coefficients of 0 and 1. The power of each term shows the position of the bit; the coefficient shows the value of the bit. An advantage is that a large binary pattern can be represented by short terms.



www.knowledgegate.in



www.knowledgegate.in

Ethernet

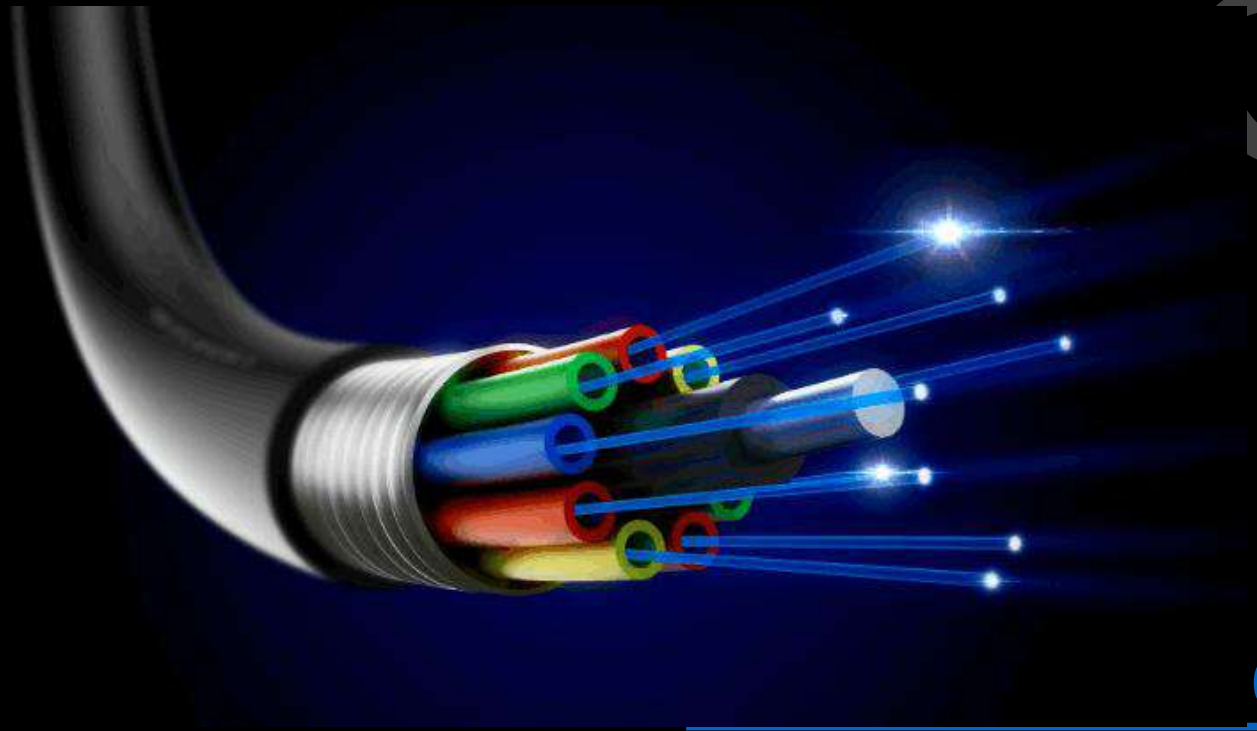
- Introduced commercially in 1980 and first standardized as IEEE 802.3 in 1983, Ethernet has undergone numerous refinements to support higher bit rates and extended link distances, replacing other LAN technologies like token ring, FDDI, and ARCNET.
- Initially utilizing coaxial cables as a shared medium, modern Ethernet variations have adopted twisted pair and fiber optic links, coupled with hubs or switches, to facilitate communication and data transfer within LANs and MANs.
- Over its evolution, Ethernet has boosted data transfer rates from an initial 2.94 Mbit/s to a remarkable 100 Gbit/s, offering several wirings and signaling options as part of the OSI physical layer in sync with Ethernet standards.

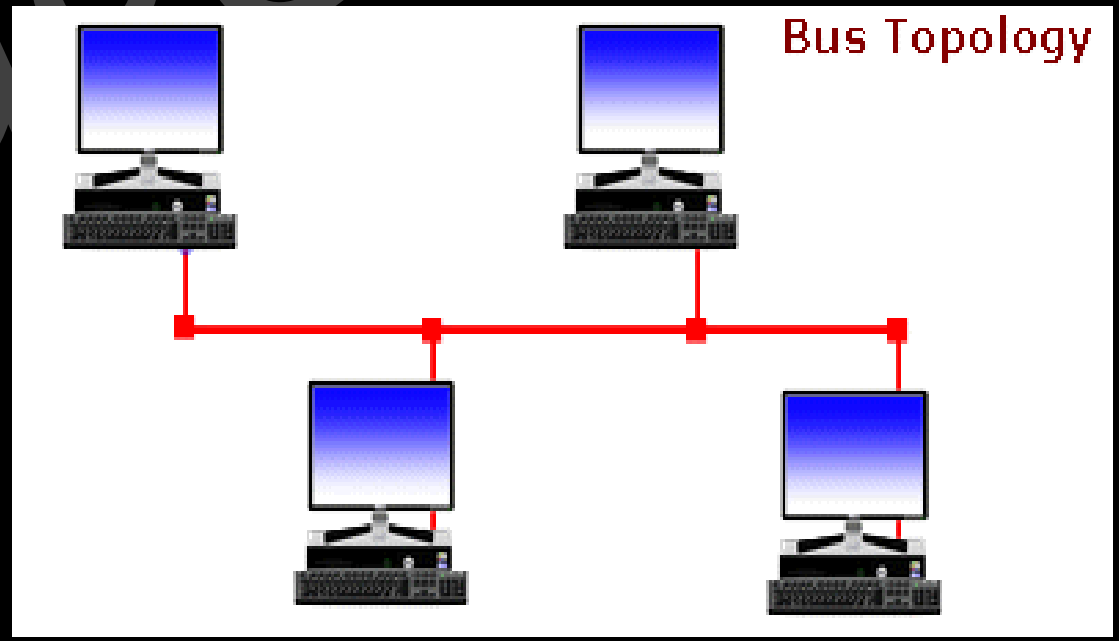
KnowledgeGate

Protocol	*MTU*
Hyperchannel	65,535
Token Ring (16 Mbps)	17,914
Token Ring (4 Mbps)	4,464
FDDI	4,352
Ethernet	1,500
X.25	576
PPP	296

www.knowledgegate.in

- Data is segmented into "frames" that carry source and destination addresses along with error-checking data, facilitating the detection and discarding of damaged frames, and enabling higher-layer protocols to trigger the retransmission of lost frames.
- Wi-Fi, a wireless protocol standardized as IEEE 802.11, serves as a prominent alternative to Ethernet in contemporary Local Area Networks (LANs).
- Operates using a Bus topology, connecting all devices to a common communication line; acknowledgments are not inherently used, but can be incorporated as data packets if necessary; utilizes Manchester encoding techniques for data transmission.





edgegate.in

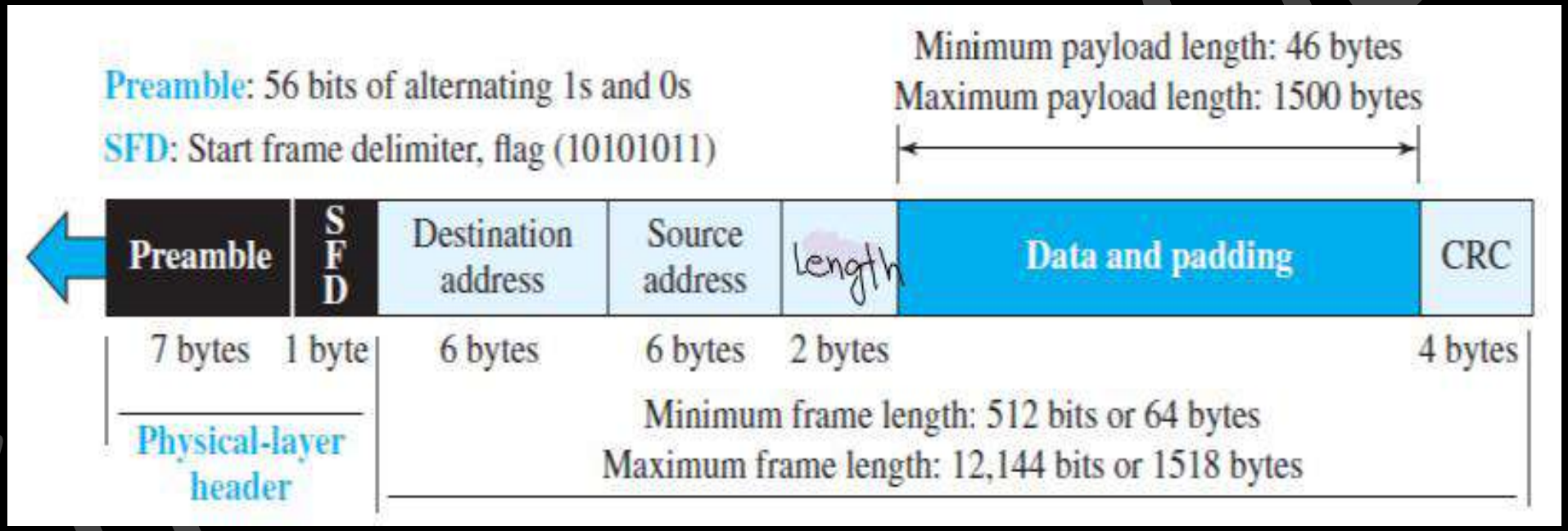
STANDARD ETHERNET

Connectionless and Unreliable Service:

- Each frame sent is independent of the previous or next frame. Ethernet has no connection establishment or connection termination phases.
- Ethernet is also unreliable, if a frame is corrupted during transmission and the receiver finds out about the corruption, the receiver drops the frame silently.
- In case of requirement ack can be sent separately at data packets.

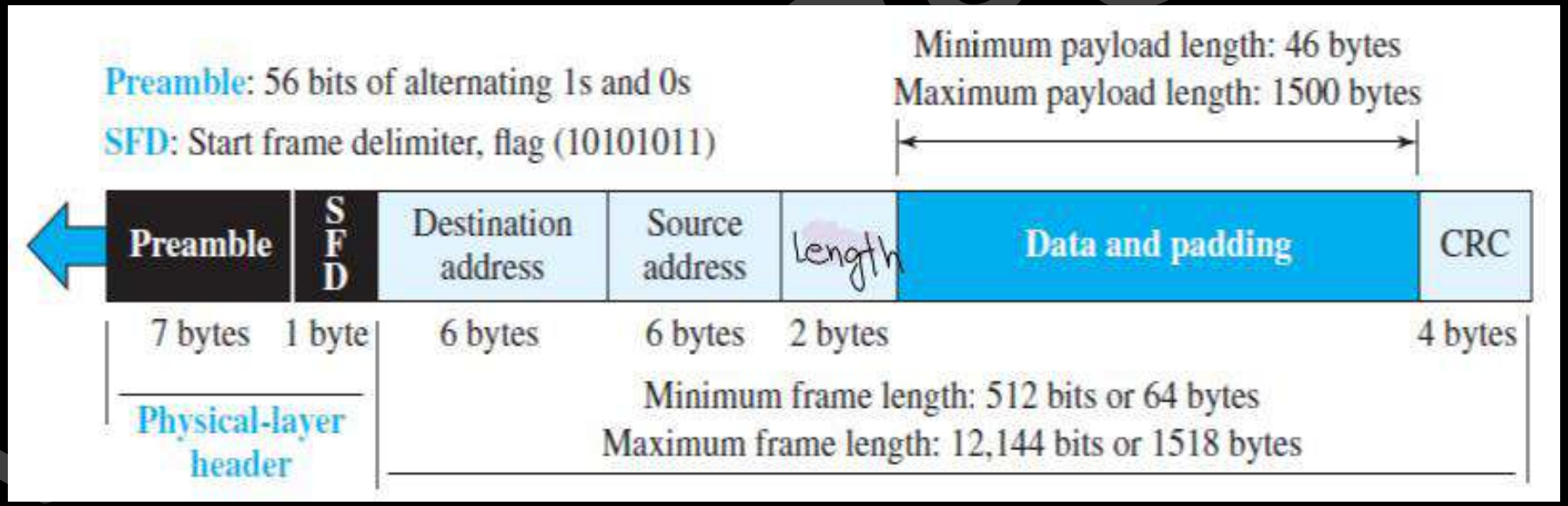
KnowledgeGate

- **Preamble**
  - It is a 7-byte field that contains a pattern of alternating 0's and 1's.
  - It alerts the stations that a frame is going to start.
  - It also enables the sender and receiver to establish bit synchronization.
  - The Preamble field is added at the physical layer.



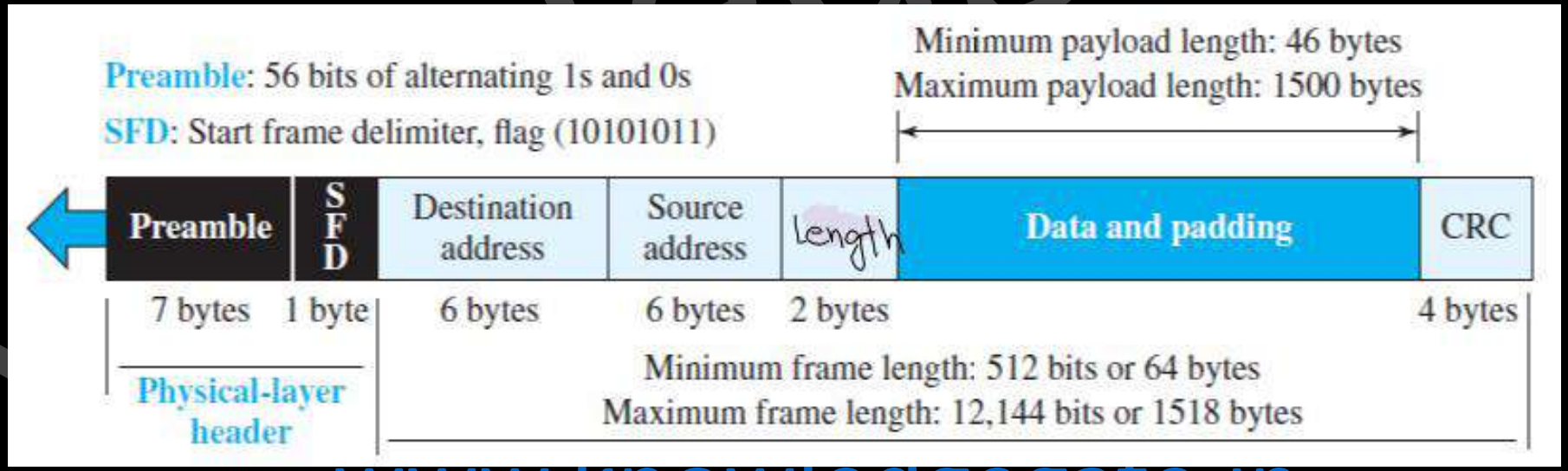
www.knowledgegate.in

- **Start Frame Delimiter (SFD)**
  - It is a 1-byte field which is always set to 10101011.
  - The last two bits “11” indicate the end of Start Frame Delimiter and marks the beginning of the frame.
  - The SFD field is also added at the physical layer.
  - Initial only SFD was there Preamble was added later



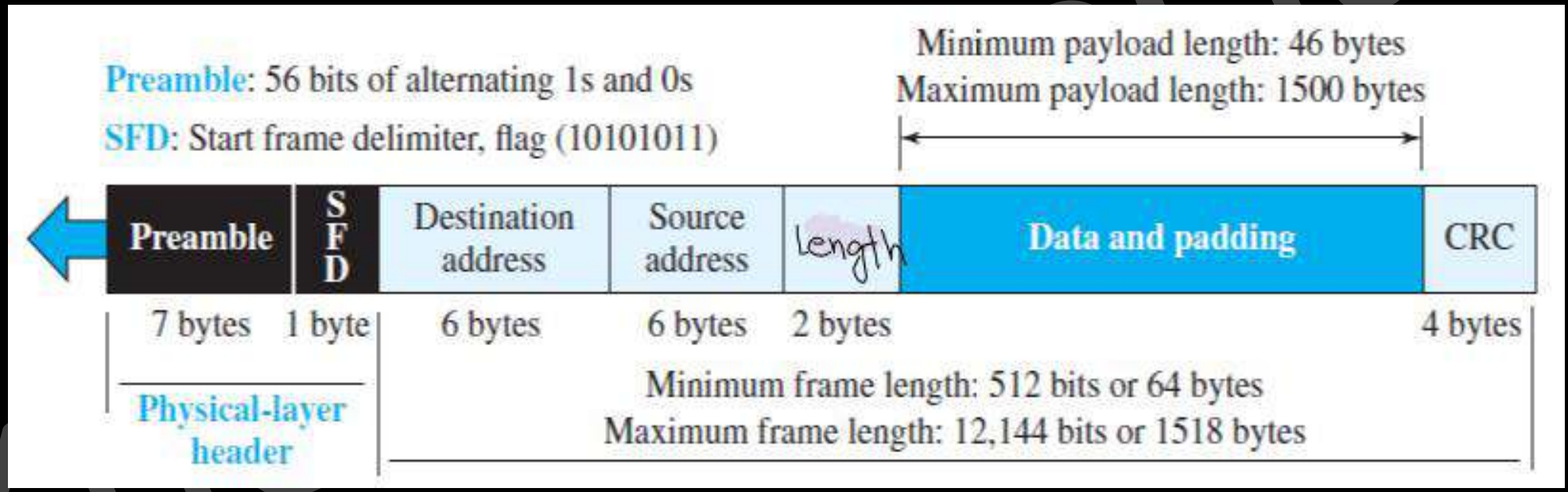
KnowledgeGate

- **Destination Address**
  - It is a 6-byte field that contains the MAC address of the destination for which the data is destined. e.g. 2D : 8A : 7B : C5
  - MAC address is present on NIC card.
  - MAC address can be of three types
    - Unicast-LSB of the first byte is 0 (Source address will always be unicast)
    - Multicast- LSB of the first byte is 1, if we want to send, repeated messages to a group of station on the network then we can group these stations together and can assign a Multicast address to the group.
    - Broadcast-all bit are assigned 1's



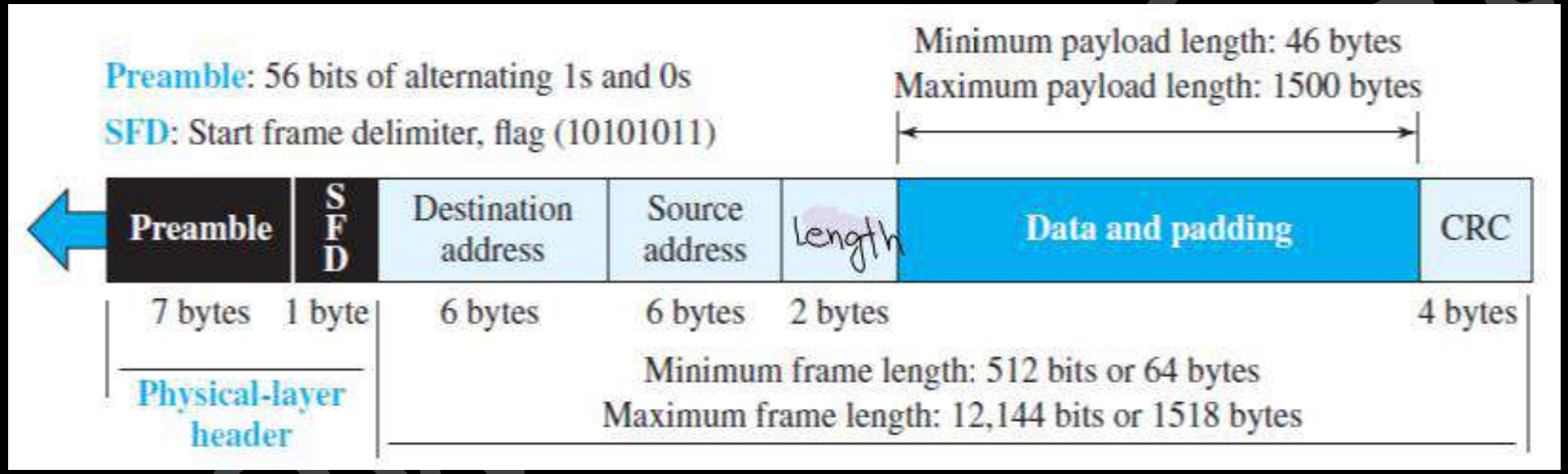
KnowledgeGate
www.knowledgegate.in

- **Source Address**
  - It is a 6-byte field that contains the MAC address of the source which is sending the data.
  - Using some protocol, we can broadcast a request message asking MAC address of every other station in the network.



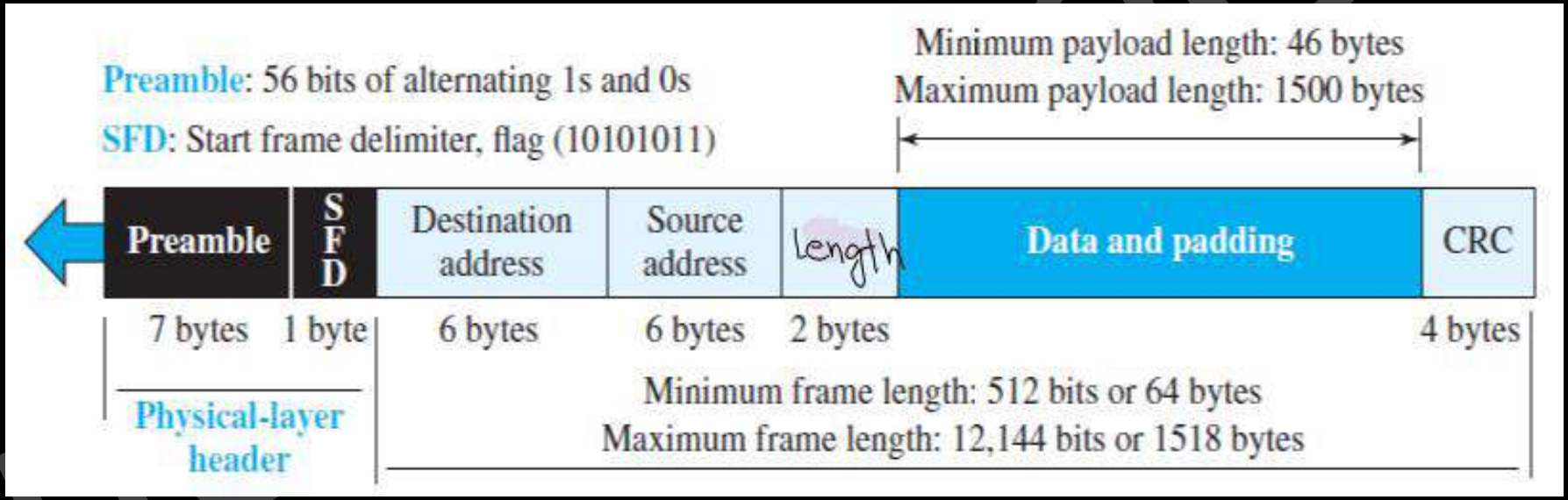
www.knowledgegate.in

- **Length**
  - As ethernet use variable size frames therefore we need Length field
  - It is a 16-bit field.



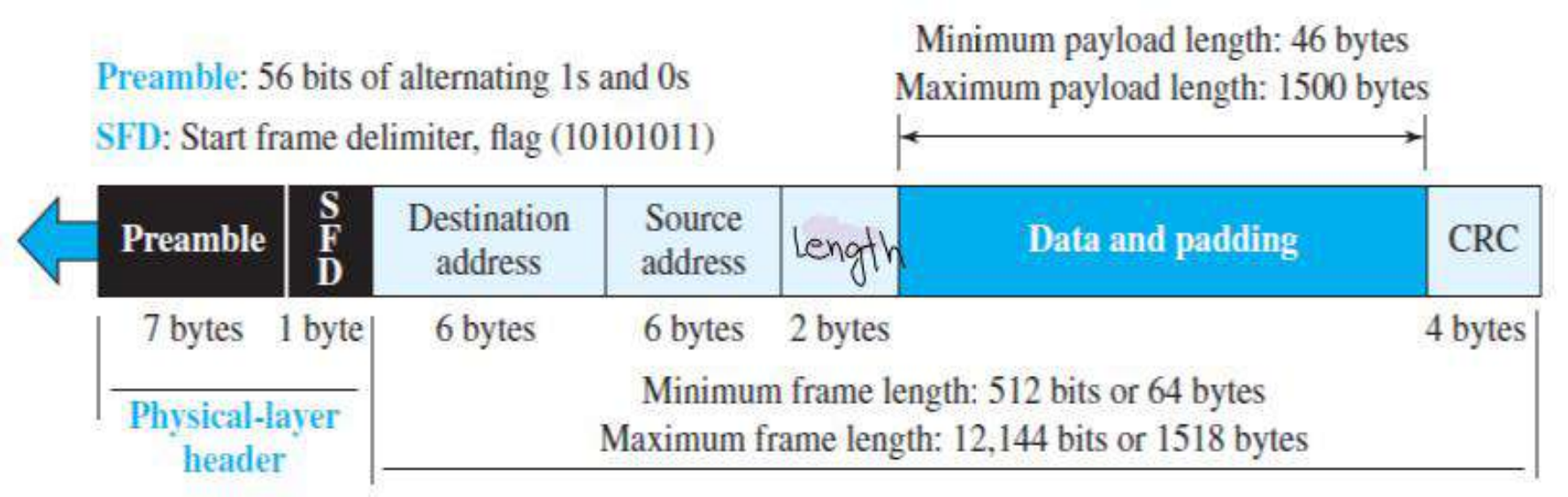
www.knowledgegate.in

- **Data**
  - It is a variable length field which contains the actual data, also called as a payload field.
  - The length of this field lies in the range [46 bytes, 1500 bytes], i.e. in an Ethernet frame, minimum data has to be 46 bytes and maximum data can be 1500 bytes.
  - If it is less than 46 bytes, it needs to be padded with extra 0s.
  - If more than 1500 bytes, it should be fragmented and encapsulated in more than one frame.



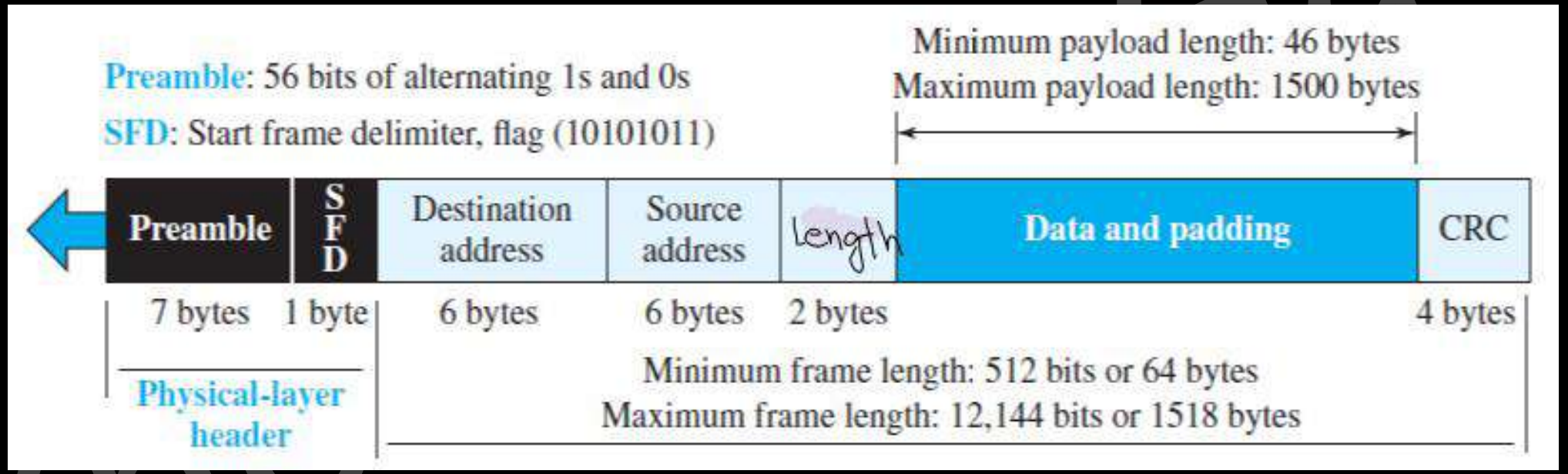
www.knowledgegate.in

- **Data**
  - The minimum length restriction is required for the correct operation of CSMA/CD, value in general come to be 64B, 64-6-6-2-4 = 46B.
  - The maximum length restriction has two historical reasons:
    - Memory was very expensive when Ethernet was designed; a maximum length restriction helped to reduce the size of the buffer.
    - The maximum length restriction prevents one station from monopolizing the shared medium, blocking other stations that have data to send.



www.knowledgegate.in

- **CRC**
  - The last field contains error detection information.
  - At the time of transmission CRC is calculated so it is in the last.
  - It is a 4-Byte field



www.knowledgegate.in

Point to Note

- Ethernet is very simple easy to install and reconfigure.
- Should not be used with real time applications, because of collision possibility.
- if amount of data is very less then also should not be used.
- No idea of priority (Server suffer).

KnowledgeGate

Token Bus (IEEE 802.4)

- IEEE 802.4 refers to the Token Bus network standard, another member of the IEEE 802 series of networking standards. Token Bus is designed to work over a bus topology but shares the token-passing access method for controlling the network traffic. Here are some key points about IEEE 802.4 Token Bus:
- **Topology**: As the name suggests, it operates over a bus topology, where all devices are connected to a central backbone.
- **Token Access**: Similar to Token Ring, it uses a token-passing mechanism to manage network access and prevent data collisions.
- **Data Rate**: Originally designed to operate at speeds up to 10 Mbps, although this may vary based on specific implementations.
- **Industries**: Often found in industrial applications and settings that require deterministic network behaviour, such as manufacturing or automation systems.
- **Deterministic**: Provides more predictable network timing compared to Ethernet, making it useful for real-time applications.
- **Obsolescence**: Like Token Ring, Token Bus has been largely superseded by Ethernet networks due to their higher speeds and easier configuration and maintenance.
- Token Bus was less common than both Ethernet and Token Ring but had specific use-cases where its deterministic behaviour was valuable.

KnowledgeGate

www.knowledgegate.in

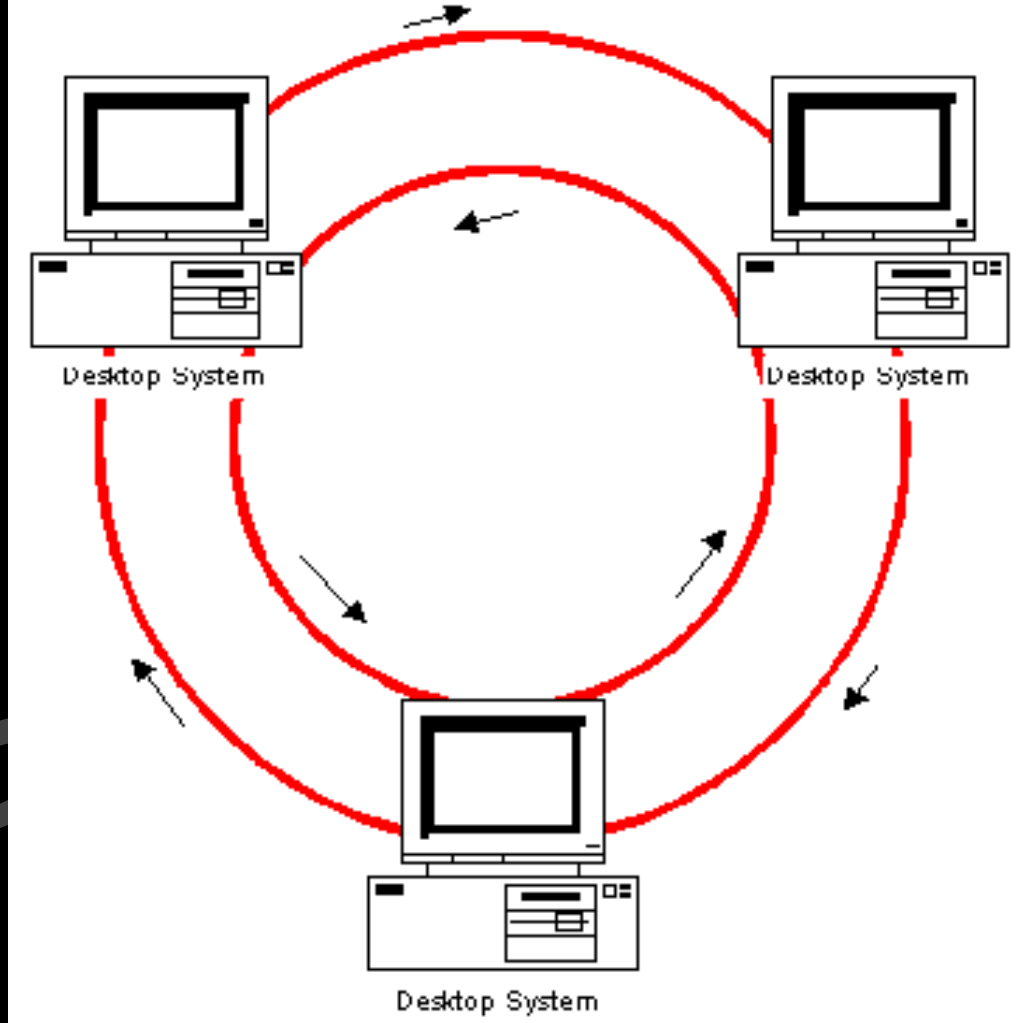
Token Ring (IEEE 802.5)

- 802.5 refers to the IEEE standard for Token Ring networks. Token Ring networks were initially developed by IBM and later standardized by IEEE as 802.5. Here are a few points about it:
- Operates over a star or ring topology, where data packets are circulated in one direction from one device to the next until they reach their destination.
- Utilizes a "token" for managing network access; a device can only send data if it seizes the token, which helps to prevent collisions.
- Typically offers data transfer rates ranging from 4 Mbps to 16 Mbps, although later developments increased speeds up to 100 Mbps.
- Over the years, it has been largely replaced by Ethernet networks due to their greater speeds and simpler configuration and maintenance.

KnowledgeGate

Fiber Distributed Data Interface (FDDI)

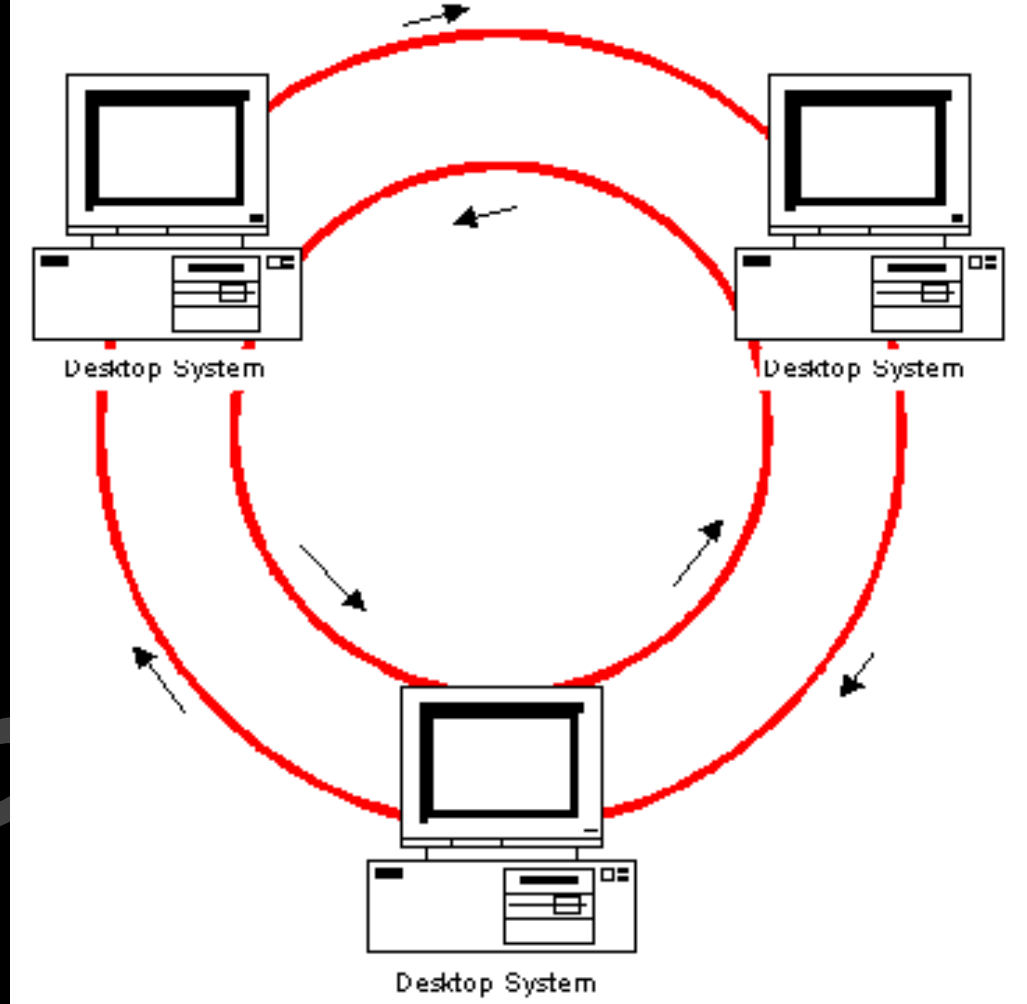
- Fiber Distributed Data Interface (FDDI) is a standard for data transmission in a local area network (LAN) that can extend in range up to 200 kilometers (124 miles). It uses optical fiber as its standard underlying physical medium, although it was later adapted for copper wiring as well (CDDI or Copper Distributed Data Interface). Here are some key points about FDDI:
- **Dual Ring**: FDDI uses a dual-ring architecture. In the event of a failure on one ring, data can be automatically sent over the second ring, providing fault tolerance.
- **High Speed**: Operates at a data rate of 100 Mbps, which was quite high when FDDI was first introduced in the late 1980s.
- **Token Passing**: Similar to Token Ring and Token Bus, FDDI also uses token-passing as the access method to prevent data collisions.



FDDI - all stations functioning

Knowledg

- **Deterministic**: Offers predictable latency, making it suitable for real-time applications and time-sensitive data.
- **Scalability**: Due to its fiber-optic nature, FDDI networks can be quite large, spanning distances that are impractical for copper-based Ethernet networks.
- **Industry Use**: Initially used for mission-critical, high-availability environments like banking, telecoms, and air traffic control systems.
- **Obsolescence**: With advancements in Ethernet technology, including Gigabit and 10-Gigabit Ethernet, FDDI has largely been phased out in favor of Ethernet solutions that offer higher speeds and are easier to manage.
- FDDI was an important step in the development of robust, high-speed networking but has largely been replaced by newer technologies.

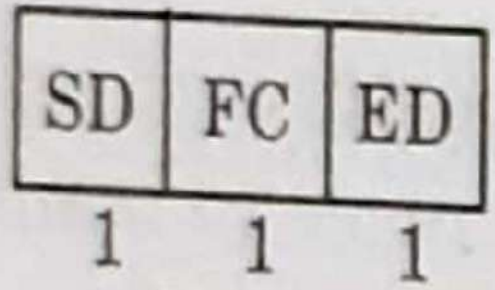


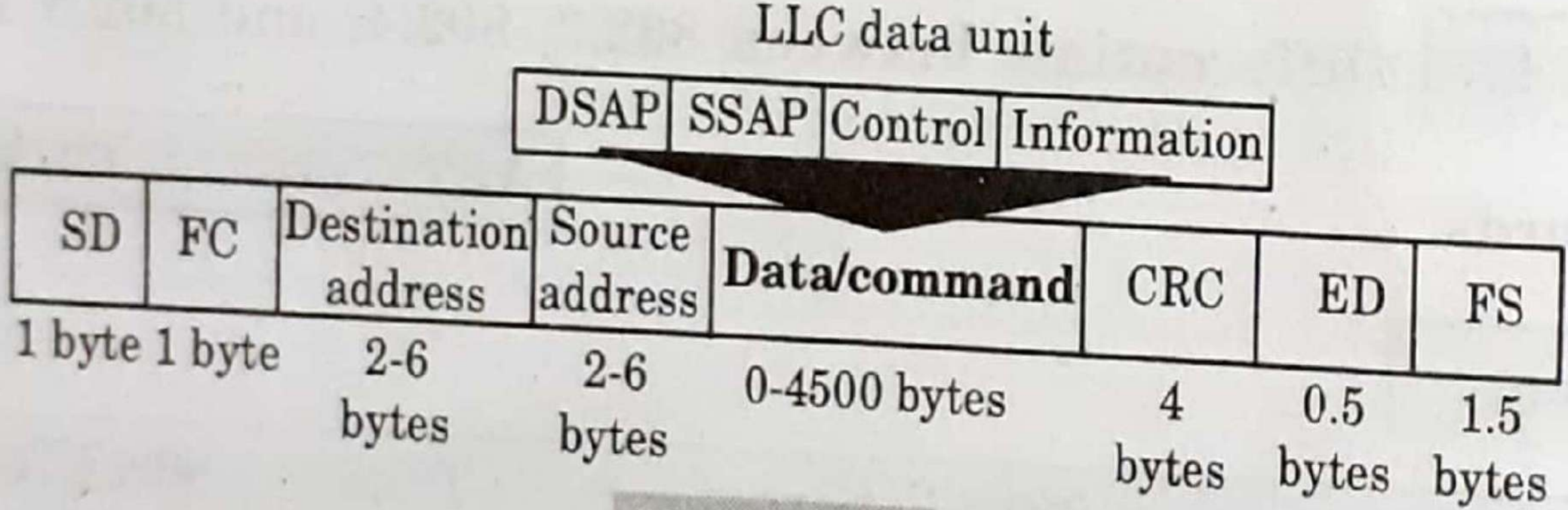
FDDI - all stations functioning

Knowledg

www.knowledgegate.in

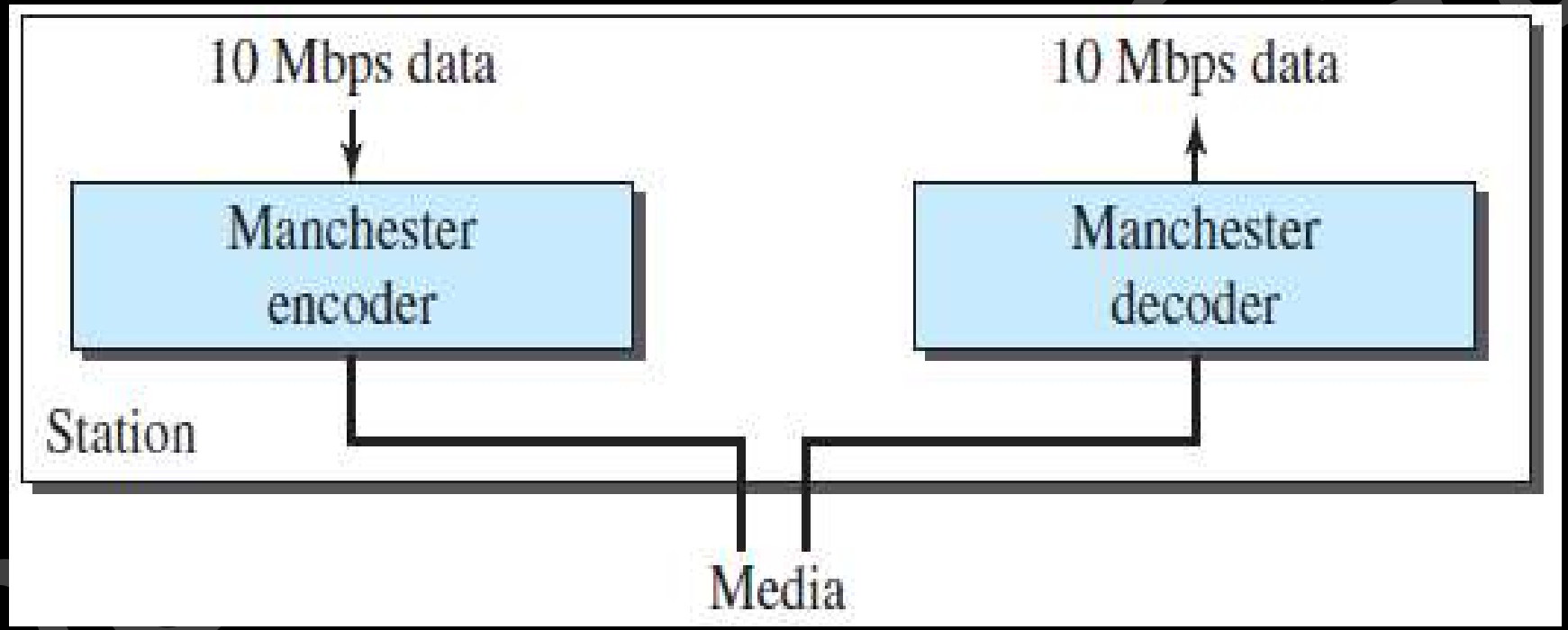
Frame format :





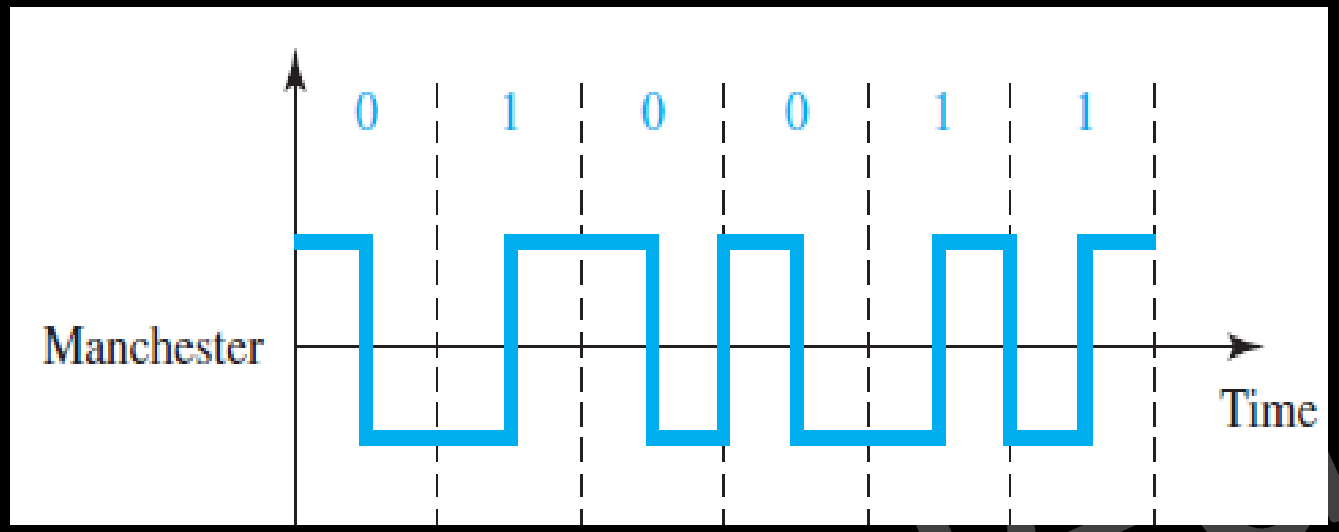
Implementation

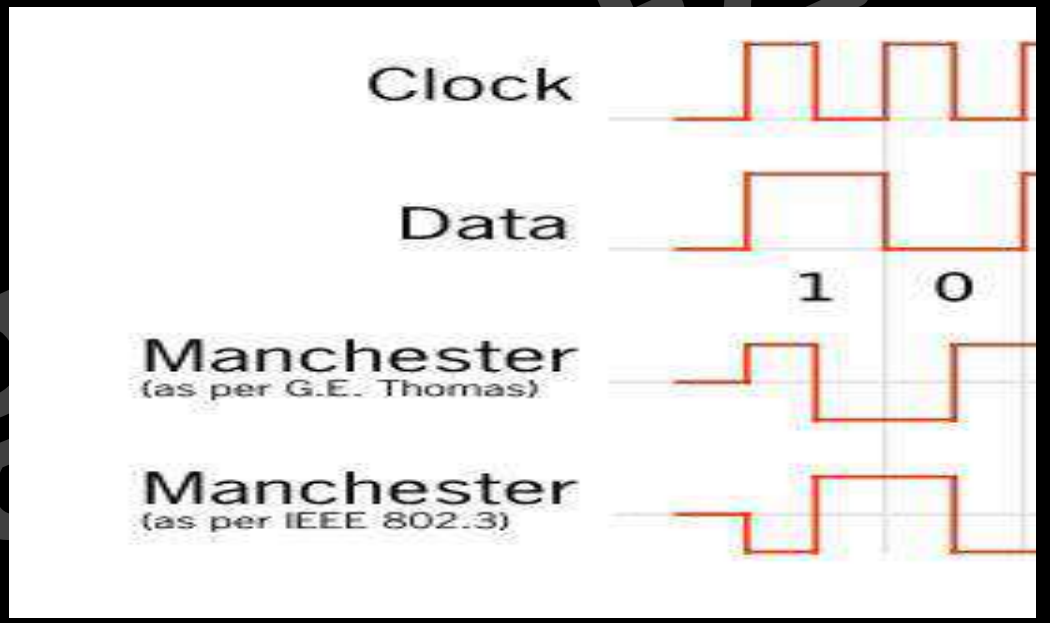
- At the sender, data are converted to a digital signal using the Manchester scheme; at the receiver, the received signal is interpreted as Manchester and decoded into data.



www.knowledgegate.in

- In Manchester encoding, the duration of the bit is divided into two halves. The voltage remains at one level during the first half and moves to the other level in the second half. The transition at the middle of the bit provides synchronization





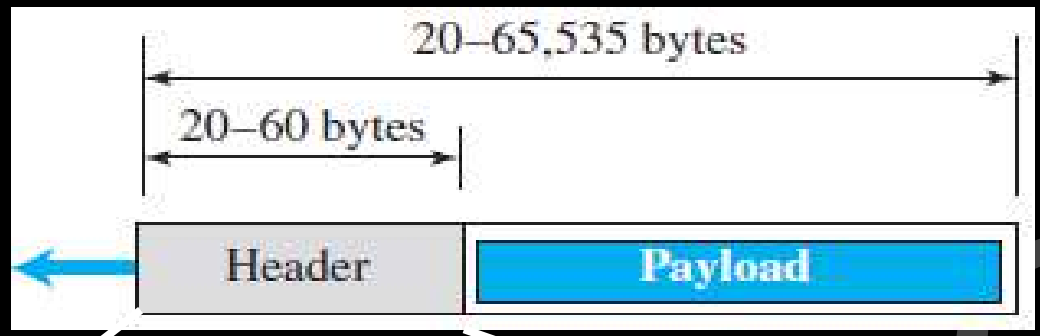
Knowledgegate

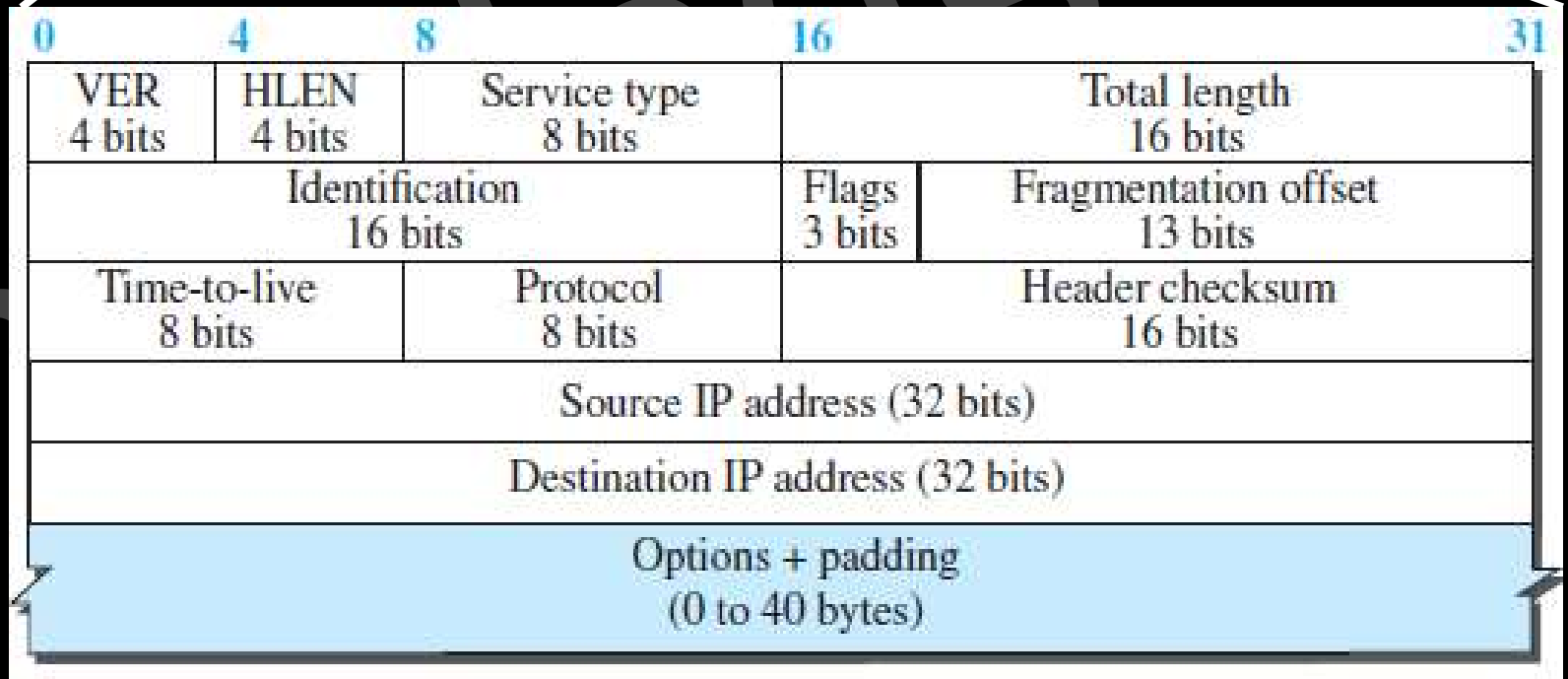
Network layer

- **Source to Destination Delivery**: Delivers packets from source to destination, possibly over multiple networks.
- **Logical Addressing**: Uses logical addresses to identify the sender and receiver.
- **Routing**: Responsible for finding the best route to send packets using routing protocols.
- **Packetizing**: Involves encapsulating payload at the source, adding a header with essential details, and preserving payload integrity during transit, barring fragmentation cases.
- **Error and Flow Control**: Encompasses adding a checksum in the datagram header for detecting corruption (not covering the entire datagram), with limited direct involvement in flow control, and using ICMP for some error control activities.
- **Congestion Control**: Manages network congestion, handling situations when too many datagrams crowd a network segment and addressing capacity exceedance issues in networks or routers.

KnowledgeGate

IPv4





KnowledgeGate

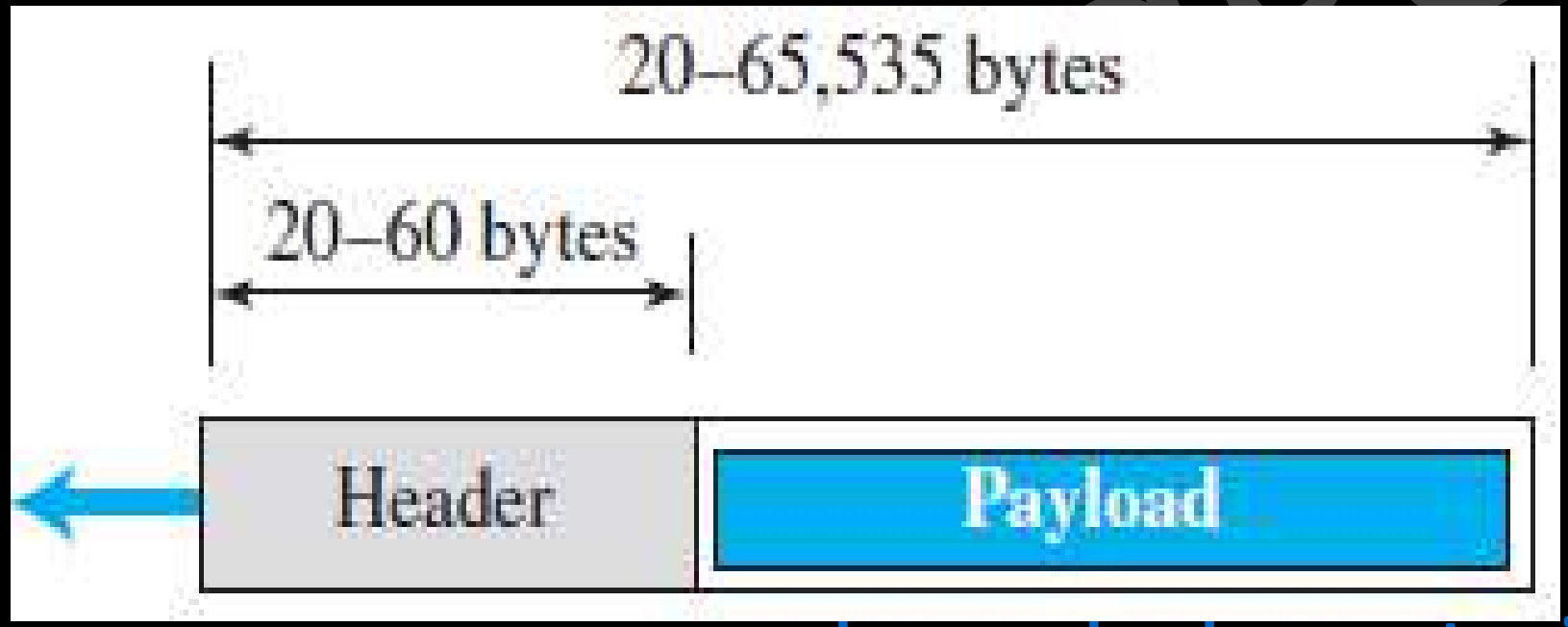
IPv4

- IPv4 operates as an unreliable connectionless datagram protocol, offering a best-effort delivery service which doesn't guarantee packet safety or order.
- The "best-effort" notion implies that IPv4 packets might experience corruption, loss, delays, or out-of-order arrival, potentially causing network congestion.
- Employing a datagram approach, IPv4 treats each datagram independently, allowing them to traverse different routes to their destination.
- To enhance reliability, IPv4 should be coupled with a reliable protocol like TCP, forming the TCP/IP protocol stack for secured data delivery.

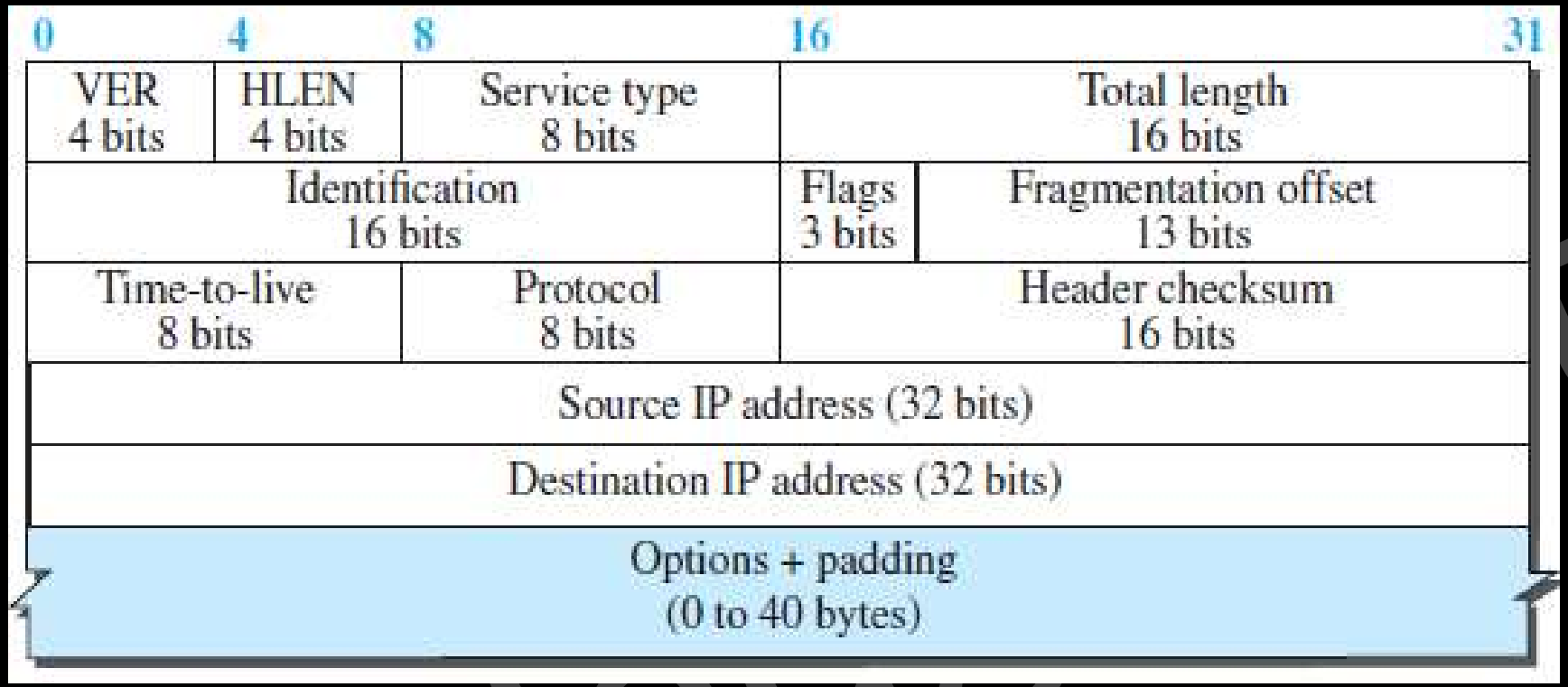
KnowledgeGate

Datagram Format

- Packets used by the IP are called ***datagrams*.**
- A datagram is a variable-length packet consisting of two parts: header and payload (data).
- The header is 20 to 60 bytes in length and contains information essential to routing and delivery.



www.knowledgegate.in

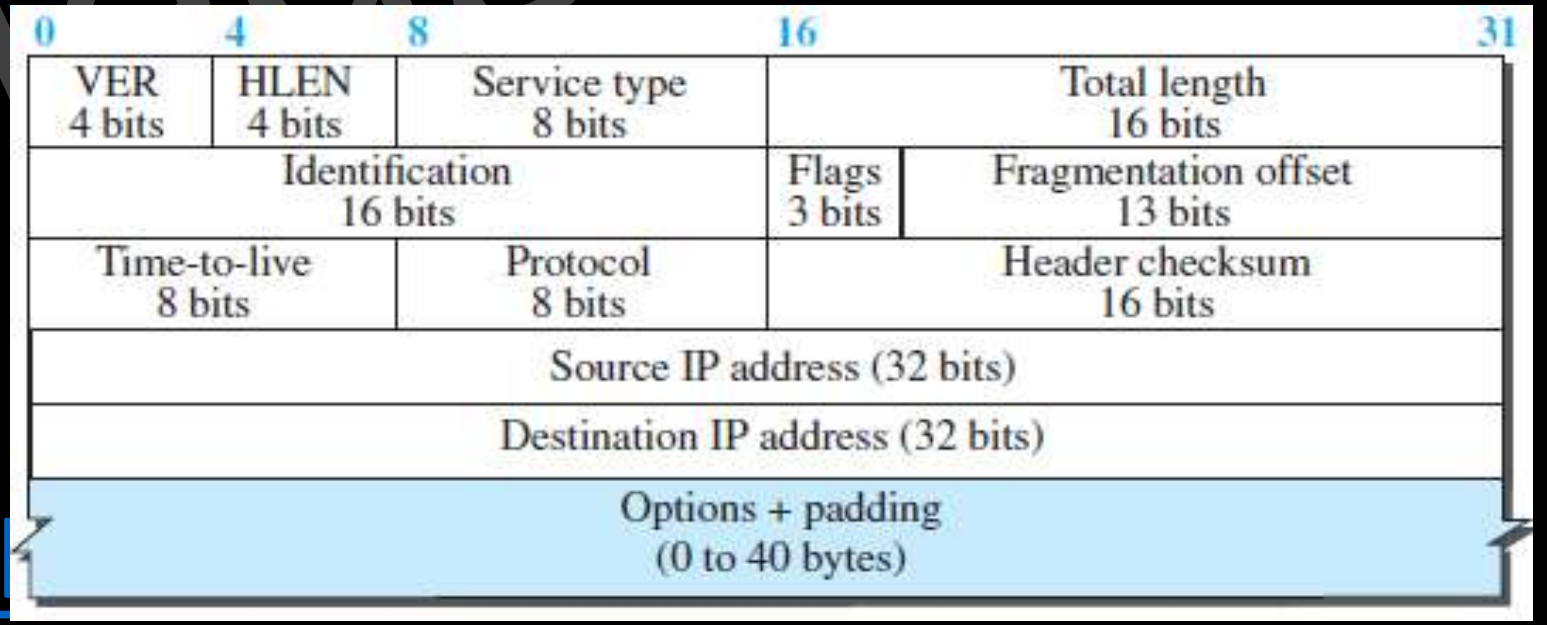


- **Version Number:** The 4-bit version number (VER) field defines the version of the IPv4 protocol, which, has the value of 4.

Knowledgegate

- **Header Length:** The 4-bit header length (HLEN) field defines the total length of the datagram header in 4-byte words. The IPv4 datagram has a variable-length header.

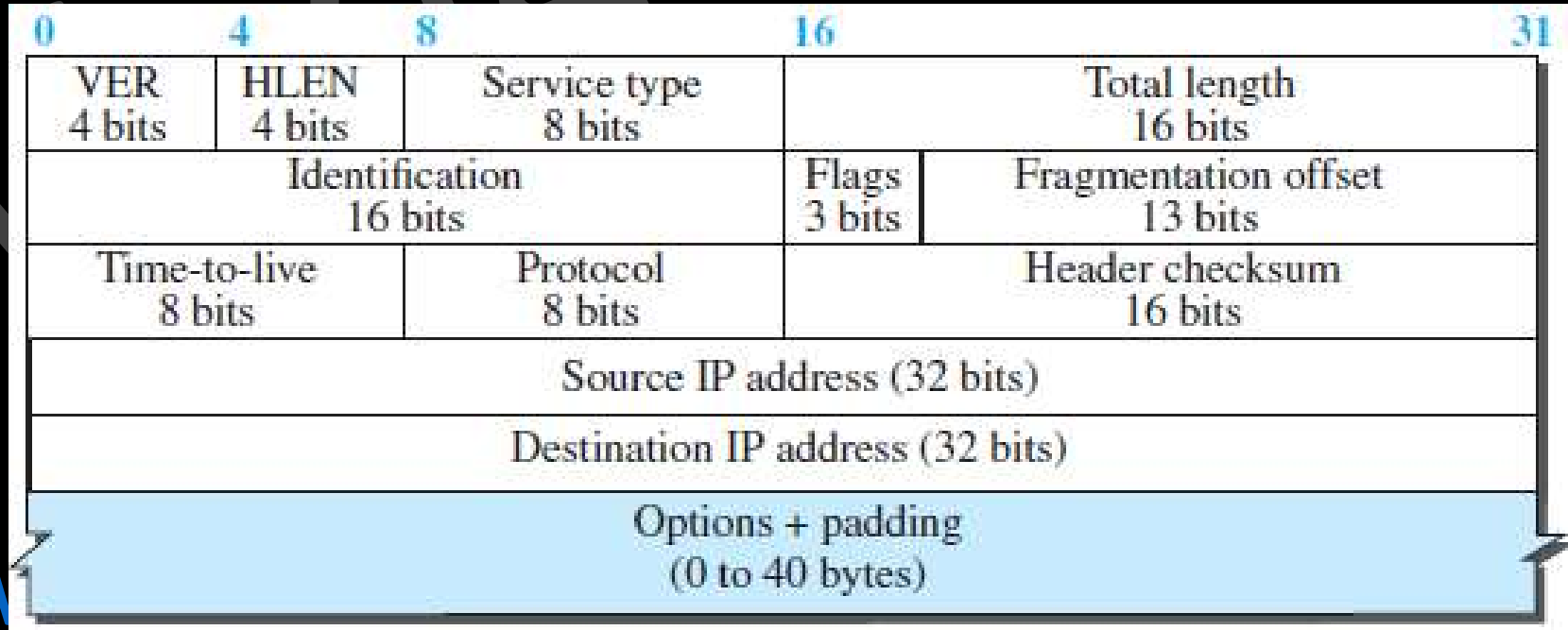
- **Scaling Factor:**
  - To make the value of the header length (number of bytes) fit in a 4-bit header length, the total length of the header is calculated as 4-byte words.
  - The total length is divided by 4 and the value is inserted in the field.
  - The receiver needs to multiply the value of this field by 4 to find the total length.
  - Example: If header length field contains decimal value 5 (represented as 0101), then Header length = 5 x 4 = 20 bytes



KnowledgeGate

www.k

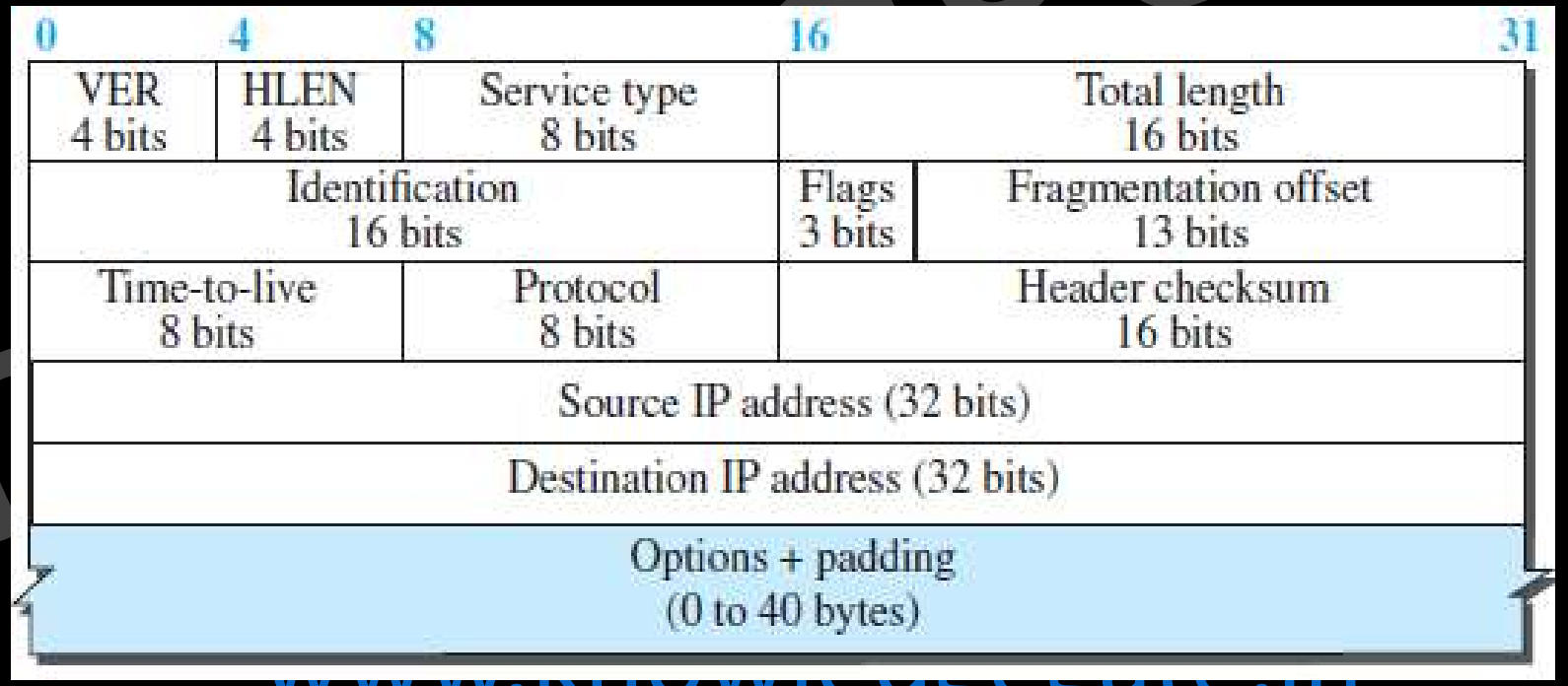
- **Point to Note**
  - The length of IP header always lies in the range of [20 bytes, 60 bytes]
  - The initial 5 rows of the IP header are always used. So, ***minimum length of IP header*** = 5 x 4 bytes = 20 bytes.
  - The size of Options field can go up to 40 bytes. So, ***maximum length of IP header*** = 20 bytes + 40 bytes = 60 bytes.
  - The range of header length field value is always [5, 15] as [20/4 = 5, 60/4 = 15]
  - The range of header length is always [20, 60].



KnowledgeGate

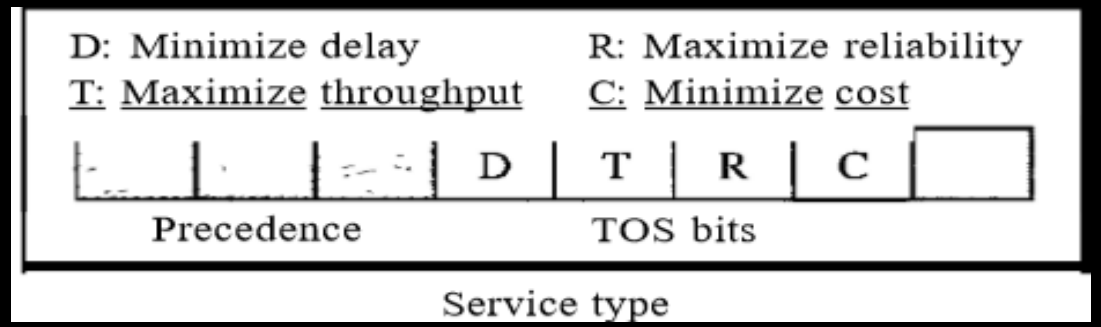
www

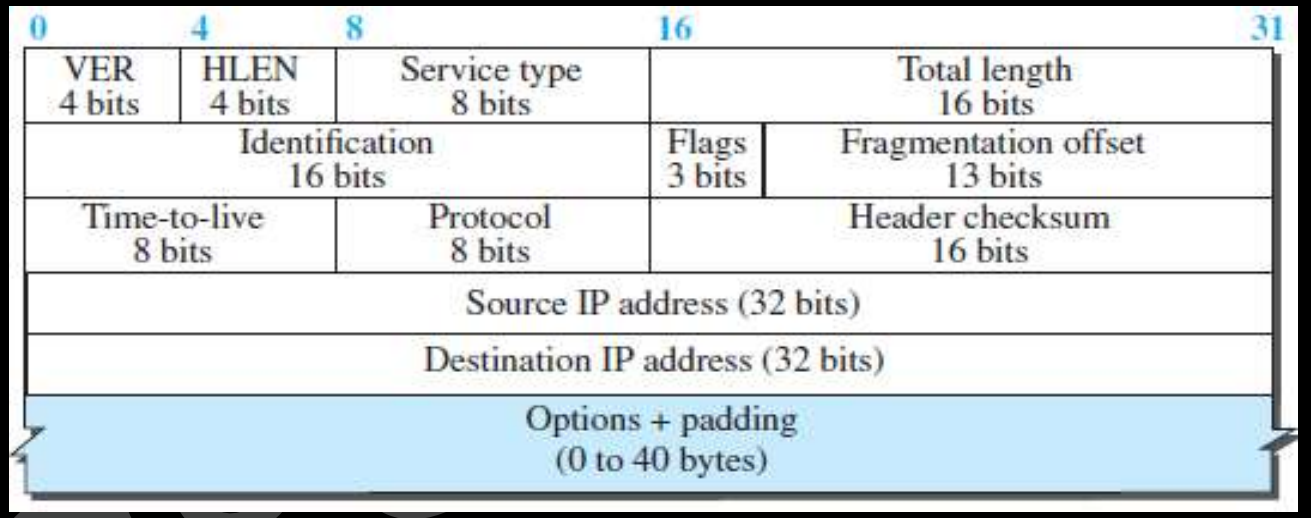
- **Services:** - IETF has changed the interpretation and name of this 8-bit field. This field, previously called service type, is now called differentiated services.
- Precedence is a 3-bit subfield ranging from 0 (000 in binary) to 7 (111 in binary). The precedence defines the priority of the datagram in issues such as congestion. If a router is congested and needs to discard some datagrams, those datagrams with lowest precedence are discarded first.



KnowledgeGate
www.knowledgegate.in

- **Service Type:** It defines how the datagram should be handled. Service type is an 8-bit field that is used for Quality of Service (QoS).





TOS Bits	*Description*
0000	Normal (default)
0001	Minimize cost
0010	Maximize reliability
0100	Maximize throughput
1000	Minimize delay

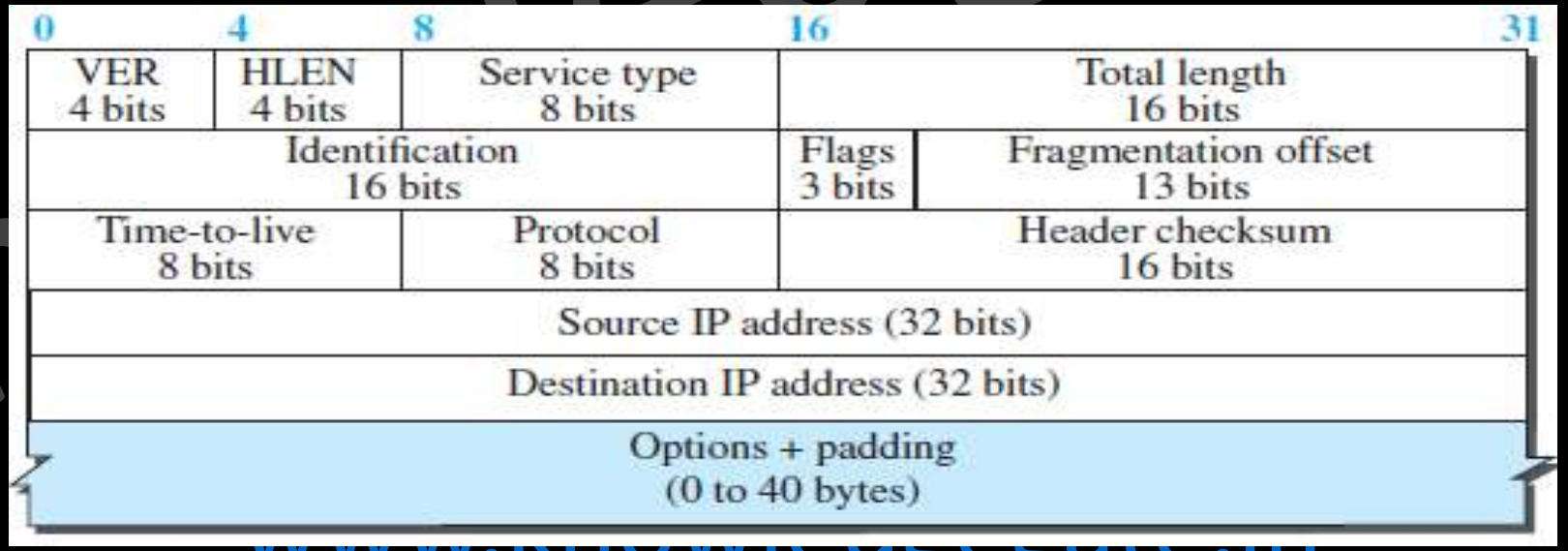
www.knowledgegate.in

- TOS bits is a 4-bit subfield with each bit having a special meaning. Although a bit can be either 0 or 1, one and only one of the bits can have the value of 1 in each datagram

Protocol	*TOS Bits*	*Description*
ICMP	0000	Normal
BOOTP	0000	Normal
NNTP	0001	Minimize cost
IGP	0010	Maximize reliability
SNMP	0010	Maximize reliability
TELNET	1000	Minimize delay
FTP (data)	0100	Maximize throughput
FTP (control)	1000	Minimize delay
TFTP	1000	Minimize delay
SMTP (command)	1000	Minimize delay
SMTP (data)	0100	Maximize throughput
DNS (UDP query)	1000	Minimize delay
DNS (TCP query)	0000	Normal
DNS (zone)	0100	Maximize throughput

www.knowledgegate.in

- **Total Length:** It defines the total length (header plus data) of the IP datagram in bytes. This field helps the receiving device to know when the packet has completely arrived.
- **Minimum total length of datagram** = 20 bytes (20 bytes header + 0 bytes data)
- **Maximum total length of datagram** = Maximum value of 16-bit word = 65535 bytes
- To find the length of the data coming from the upper layer, subtract the header length from the total length.
- **Length of data = total length – (HLEN) × 4**

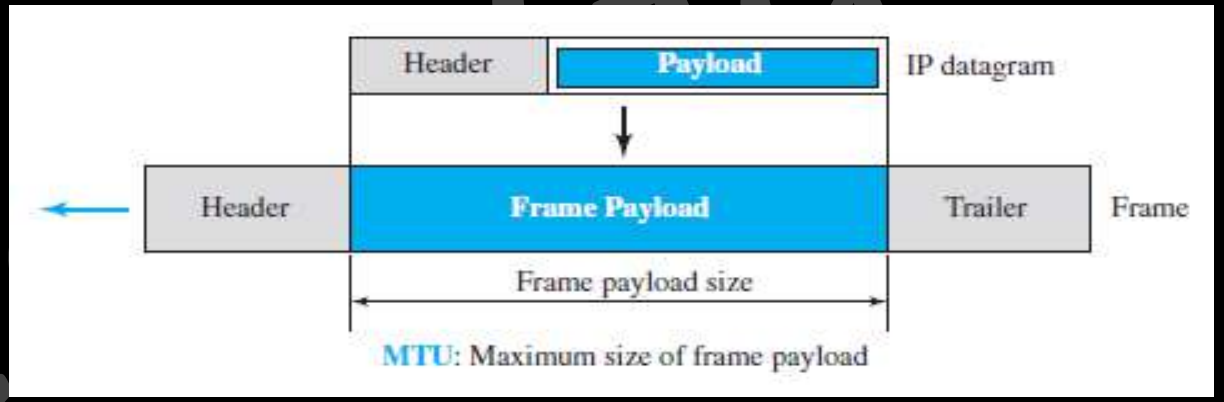


KnowledgeGate
www.knowledgegate.in

Maximum Transfer Unit (MTU)

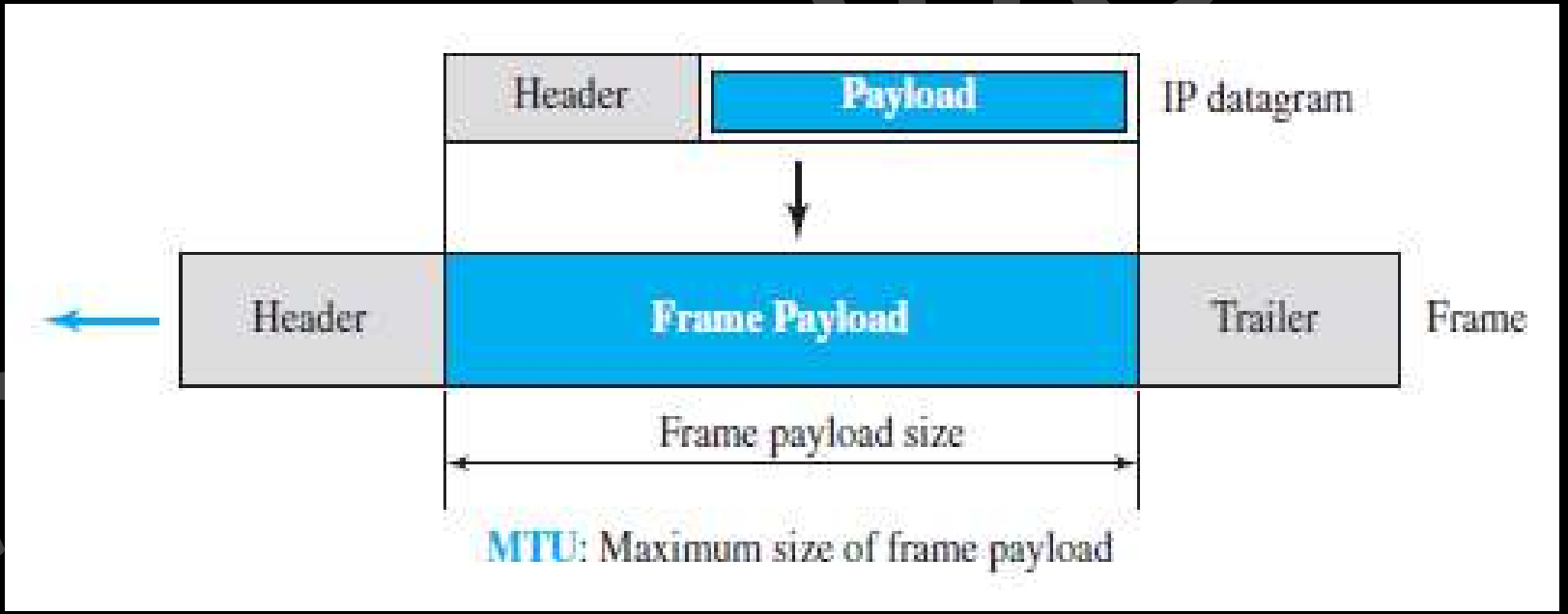
- Each link-layer protocol has its own frame format. One of the features of each format is the maximum size of the payload that can be encapsulated. In other words, when a datagram is encapsulated in a frame, the total size of the datagram must be less than this maximum size.

Protocol	*MTU*
Hyperchannel	65,535
Token Ring (16 Mbps)	17,914
Token Ring (4 Mbps)	4,464
FDDI	4,352
Ethernet	1,500
X.25	576
PPP	296



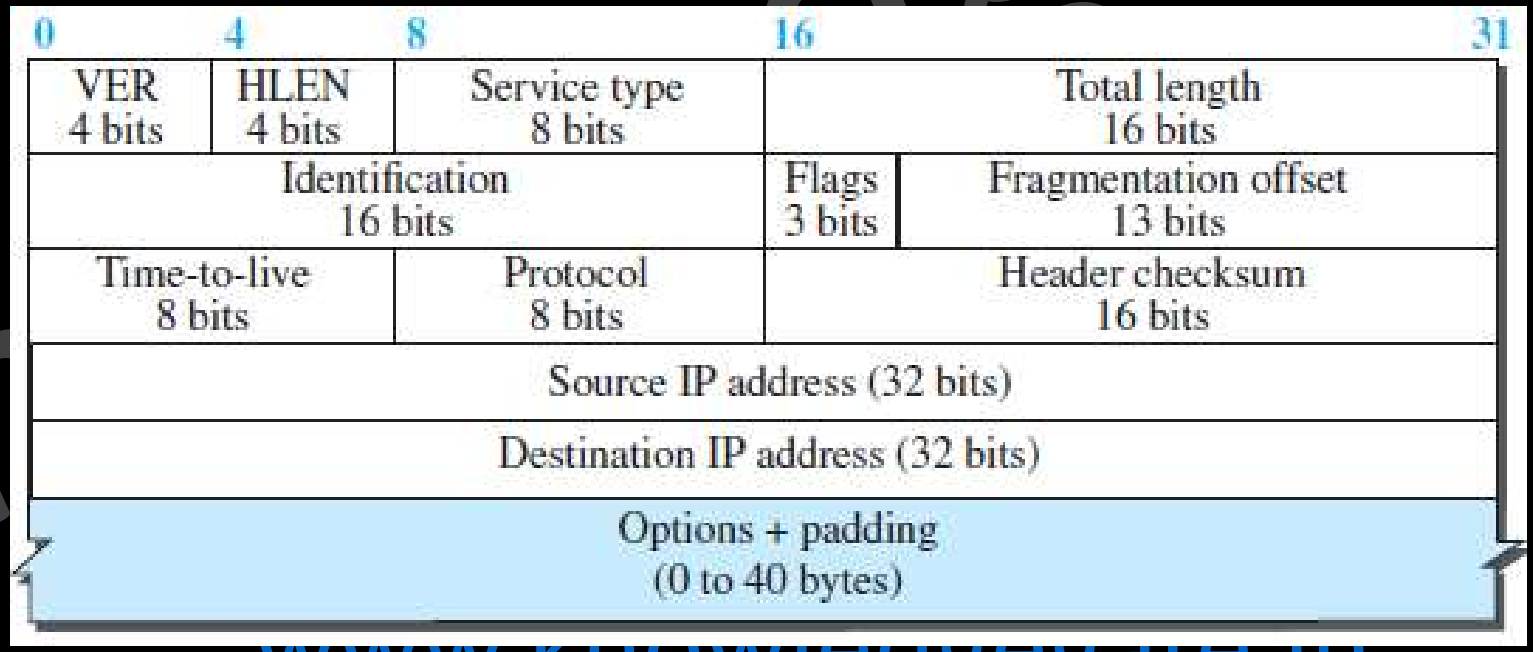
www.knowledgegate.in

- ***The value of the MTU differs from one physical network protocol to another.*** For example, the value for a LAN is normally 1500 bytes, but for a WAN it can be larger or smaller.
- When a datagram is fragmented it means that the payload of the IP datagram is fragmented and each fragment has its own header with most of the fields repeated, but some have been changed such as flags, fragmentation offset, and total length and checksum is recalculated at each point.
- A fragmented datagram may itself be fragmented if it encounters a network with an even smaller MTU. Thus, ***datagram may be fragmented several times before it reaches the final destination.***



www.knowledgegate.in

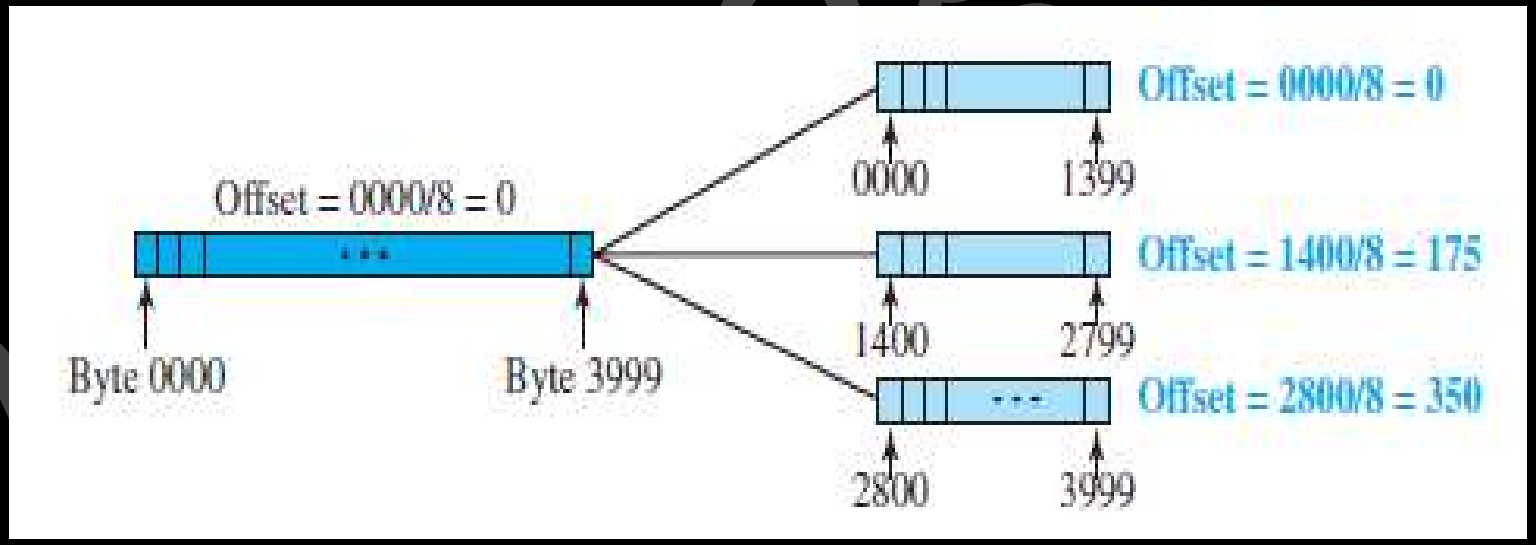
- **Identification**: 16-bit *identification field* identifies a datagram originating from the source host. To guarantee uniqueness, IP protocol uses a counter to label the datagrams.
- The counter is initialized to a positive number. When the IP protocol sends a datagram, it copies the current value of the counter to the identification field and increments the counter by one.
- When a datagram is fragmented, the value in the identification field is copied into all fragments, used for the identification of the fragments of an original IP datagram. The identification number helps the destination in reassembling the datagram.



www.knowledgegate.in

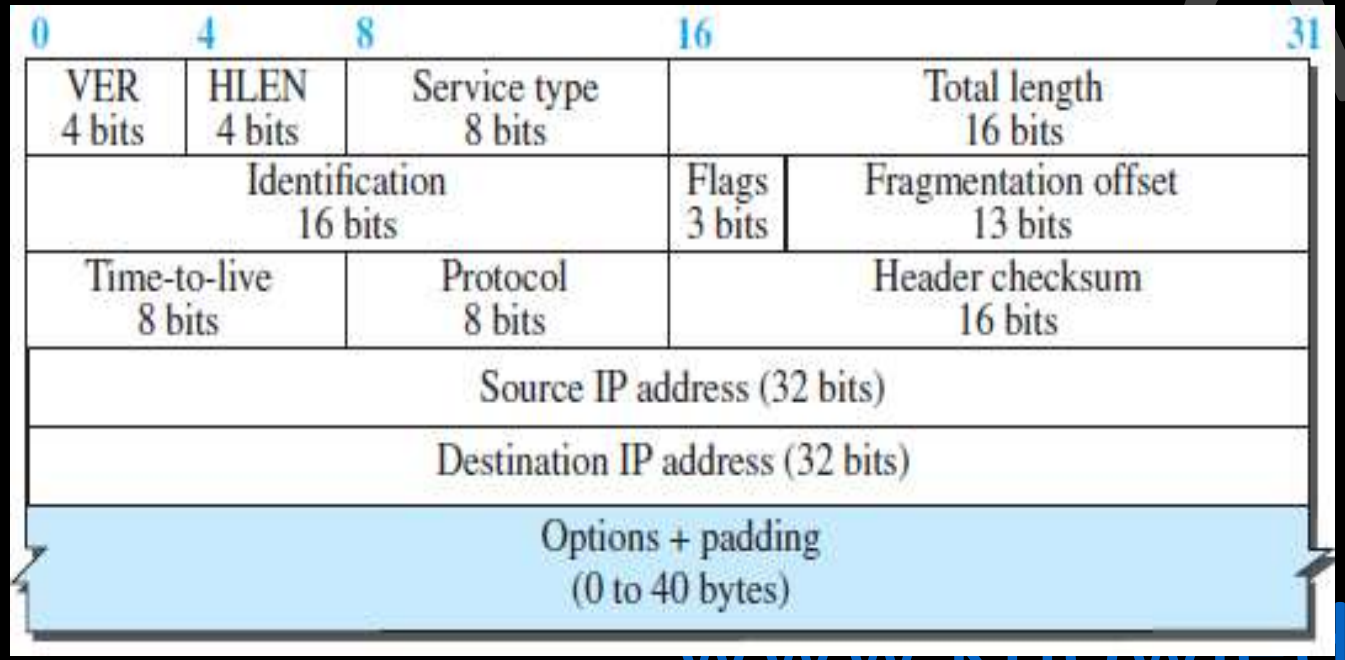
Fragmentation

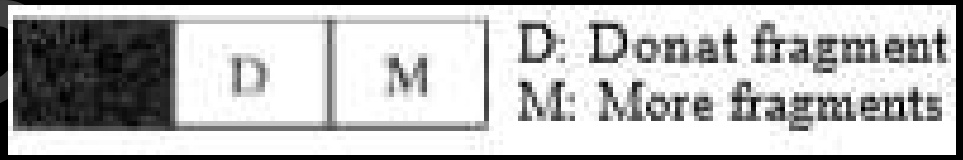
- Fragmentation is a process of dividing the datagram into fragments during its transmission.
- Datagram can be fragmented by the source host or any router in the path.
- The reassembly of the datagram, is done only by the destination host, because each fragment becomes an independent datagram.
- The fragmented datagram can travel through different routes.



KnowledgeGate

- **Flag Field:** The 3-bit *flags field* defines three flags.
  - The leftmost bit is reserved (not used).
  - The second bit (D bit) is called the *do not fragment* bit.
    - If its value is 1, the machine must not fragment the datagram.
    - If its value is 0, the datagram can be fragmented if necessary.
  - The third bit (M bit) is called the *more fragment bit*.
    - If its value is 1, it means the datagram is not the last fragment; there are more fragments after this one.
    - If its value is 0, it means this is the last or only fragment.

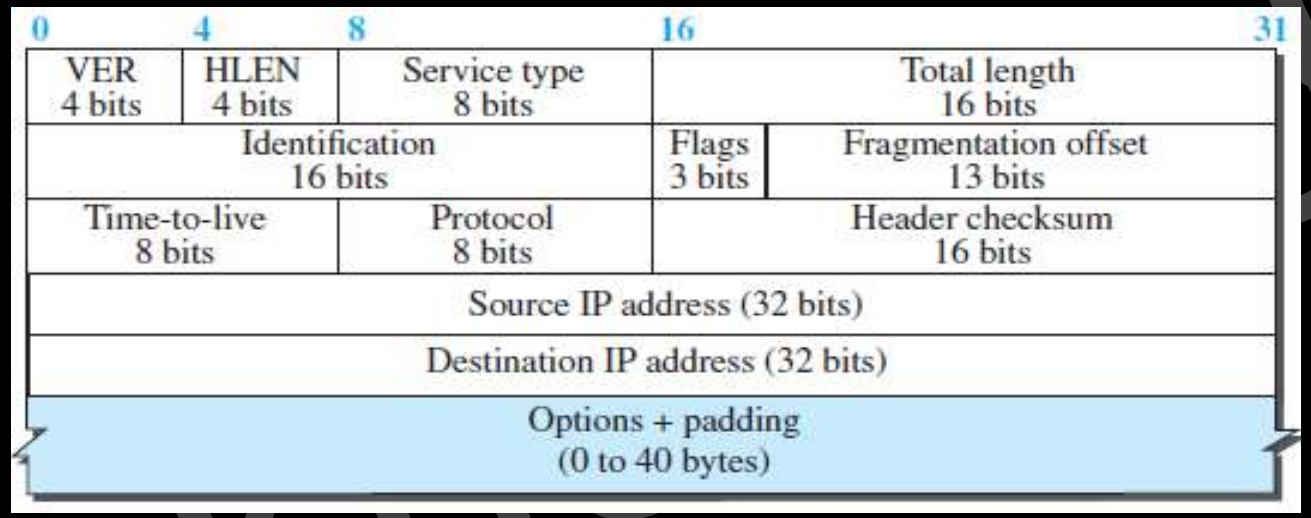


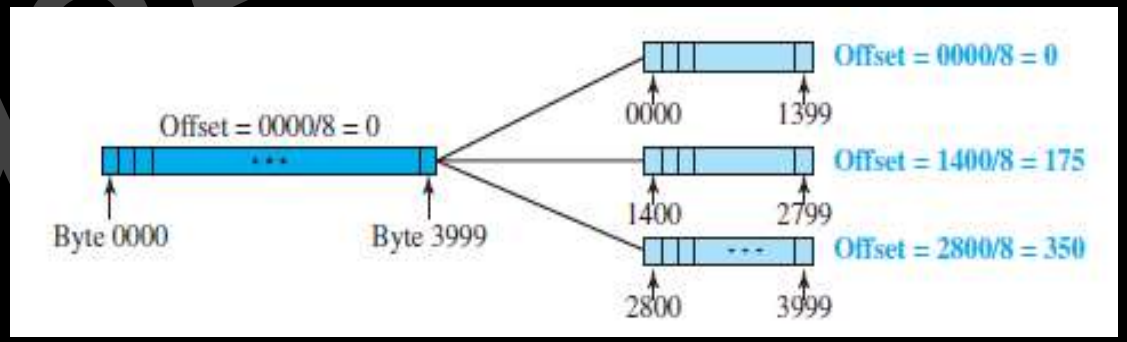


KnowledgeGate

www.knowledgegate.in

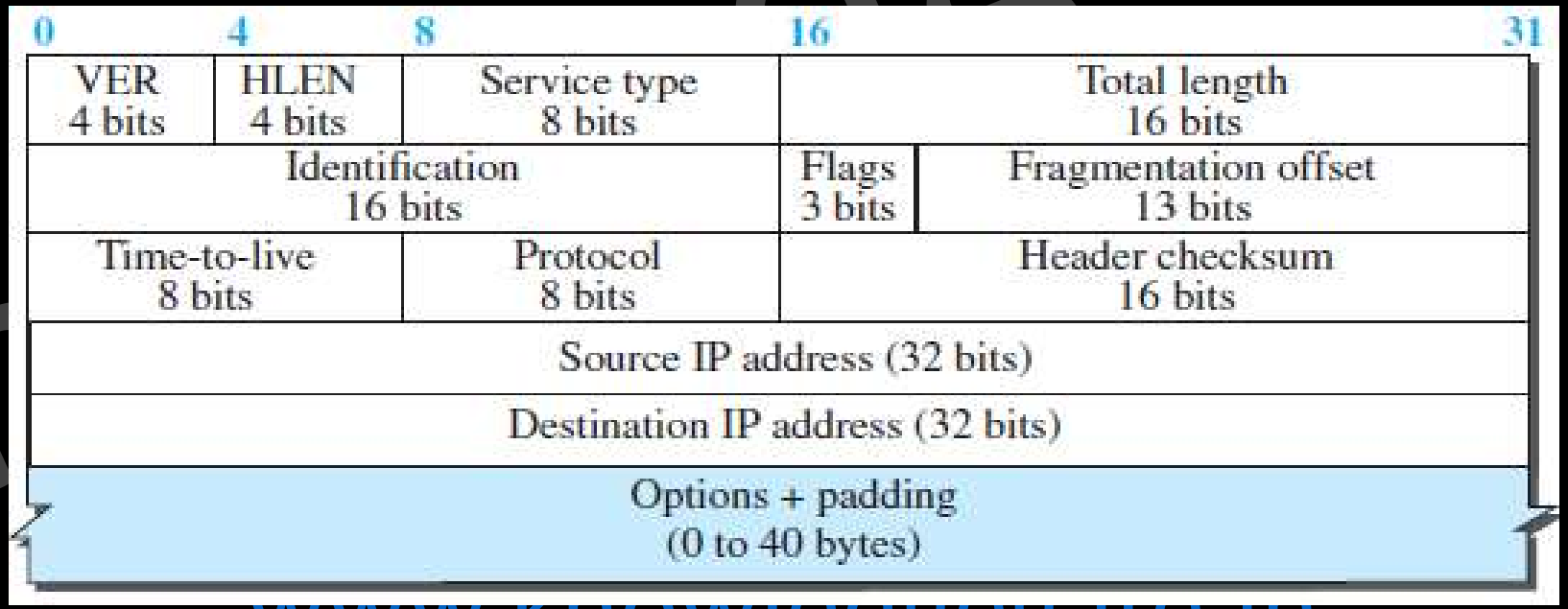
- **Fragmentation Offset**: The 13-bit *fragmentation offset field* shows the relative position of this fragment with respect to the whole datagram.
- It is the offset of the data in the original datagram measured in units of 8 bytes.
- The bytes in the original datagram are numbered 0 to 3999.
- The first fragment carries bytes 0 to 1399. The offset value => 0/8 = 0.
- The second fragment carries bytes 1400 to 2799; the offset value => 1400/8 = 175.
- The third fragment carries bytes 2800 to 3999. The offset value => 2800/8 = 350.





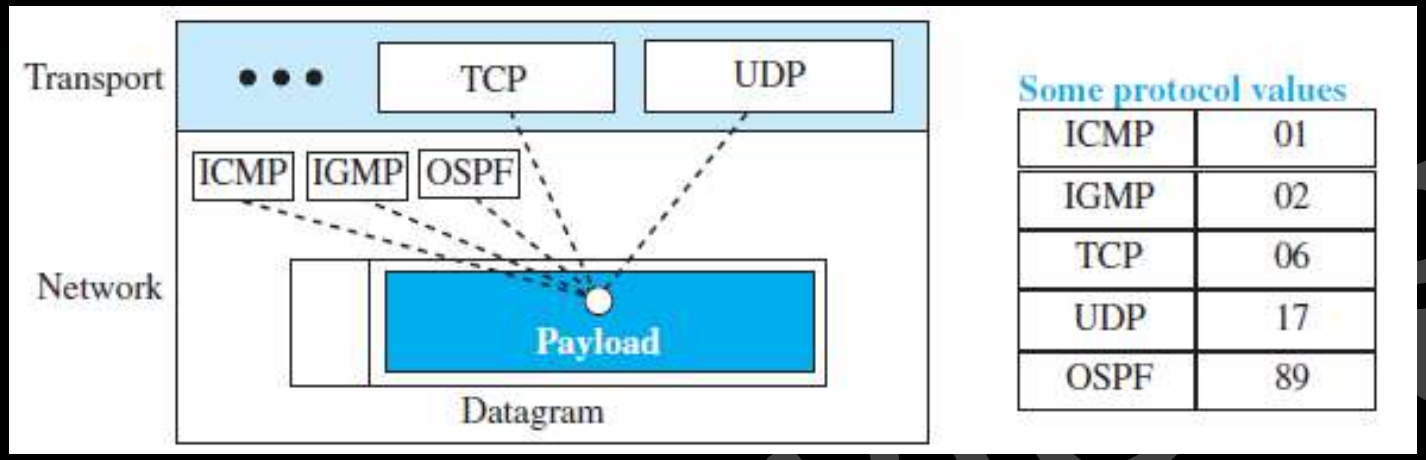
www.knowledgegate.in

- **Time-to-Live** (TTL): field in a datagram dictates the maximum number of hops (via routers) it can take, generally set to twice the highest number of routers between any two hosts.
- Every router the datagram passes through decreases the TTL value by one; the datagram is discarded if the TTL reaches zero, preventing it from circulating indefinitely due to potential routing table errors.
- Besides limiting a datagram's lifespan, the TTL field can be used to restrict a packet's journey deliberately, like confining it to a local network by setting the TTL value to 1, causing its discard at the first router.



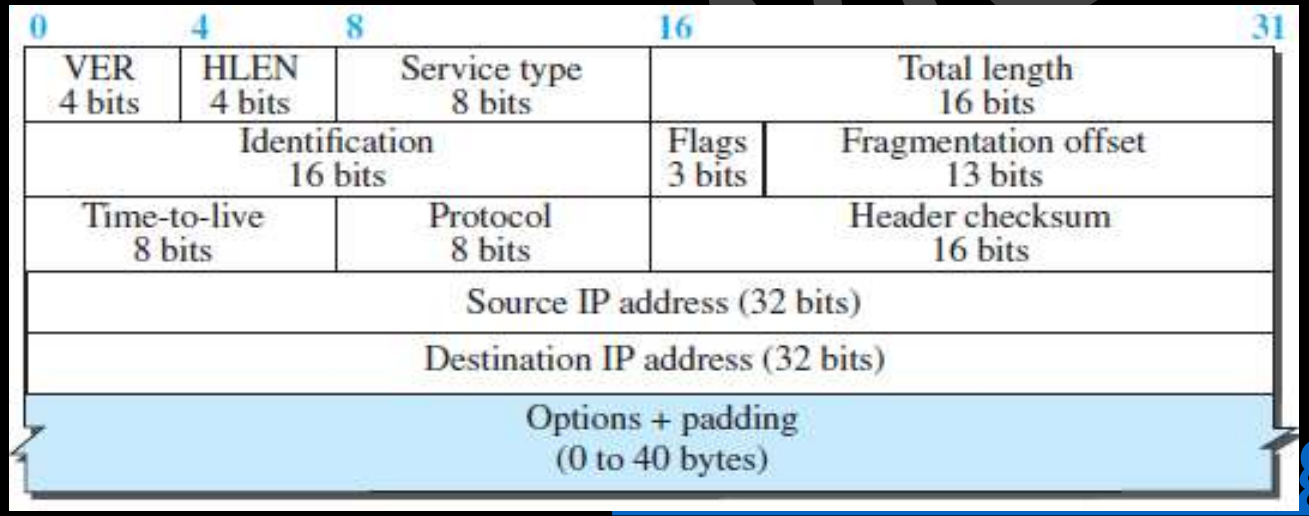
KnowledgeGate
www.knowledgegate.in

- **Protocol:** In TCP/IP, the data section of a packet, called the payload, carries the whole packet from another protocol. A datagram, for example, can carry a packet belonging to any transport-layer protocol such as UDP or TCP.
- When the datagram arrives at the destination, the value of this field helps to define to which protocol the payload should be delivered.



Some protocol values

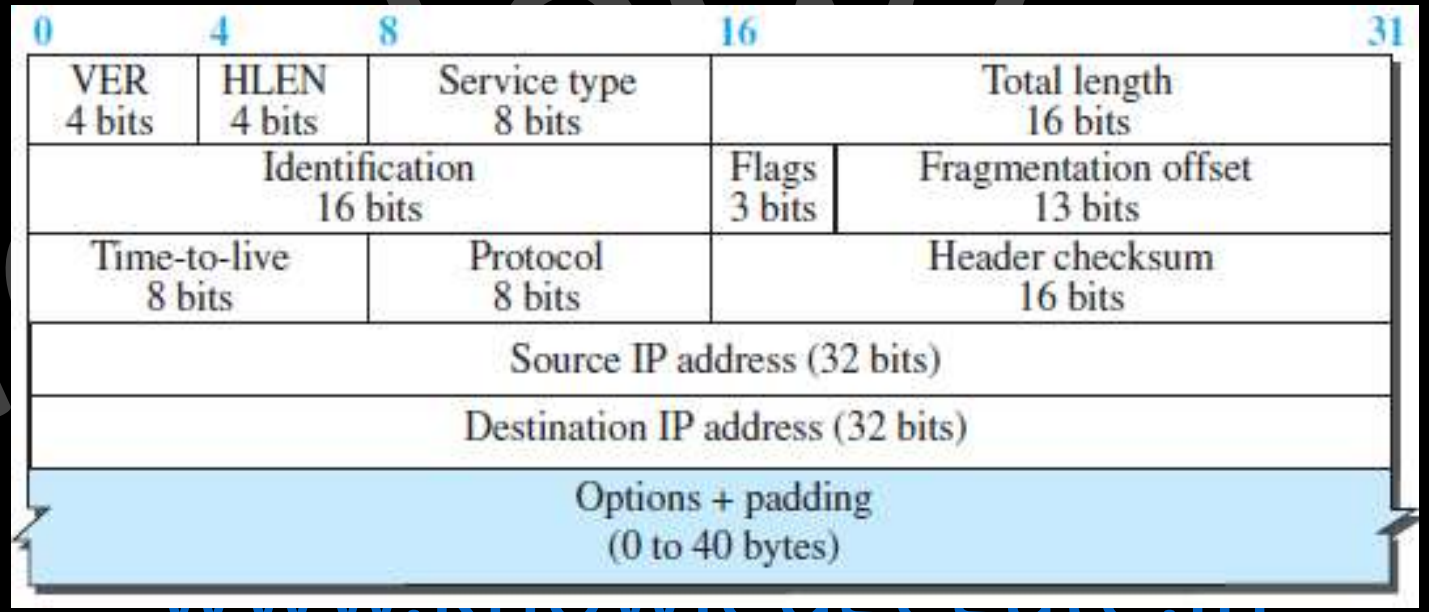
ICMP	01
IGMP	02
TCP	06
UDP	17
OSPF	89



geGate
gegate.in

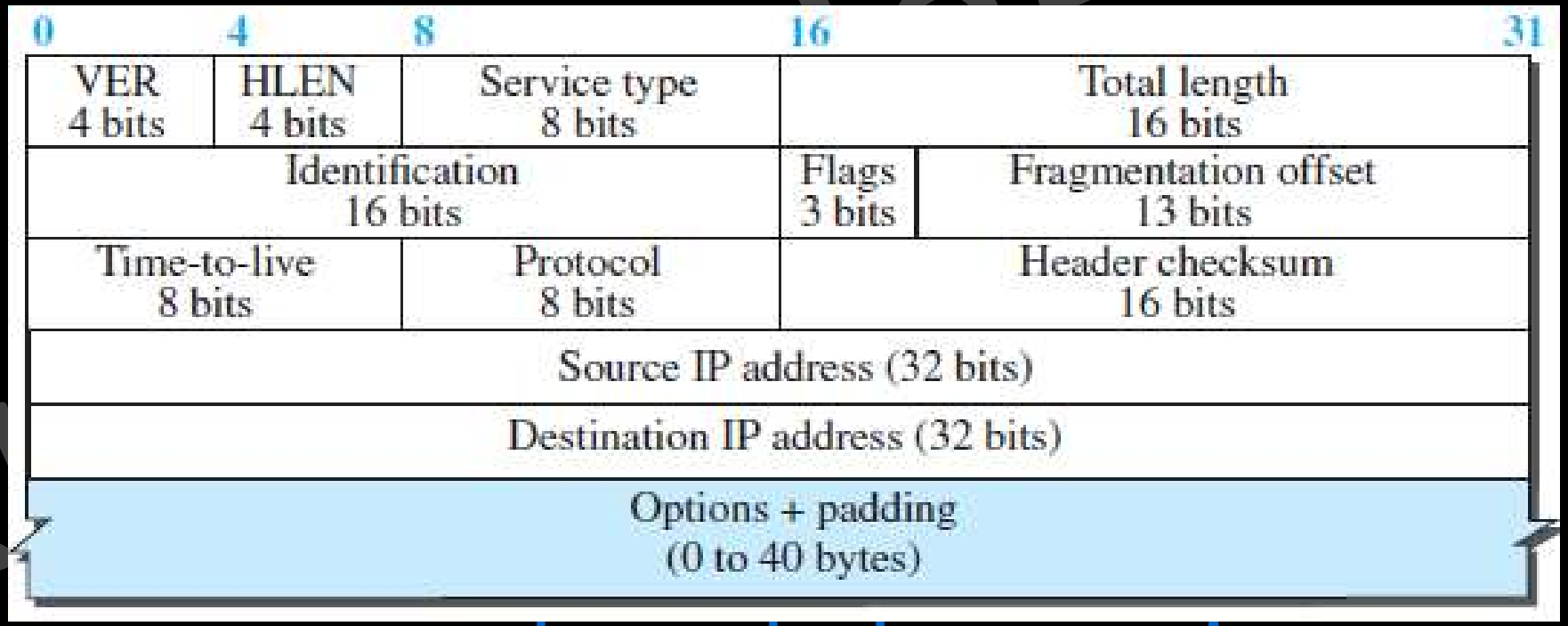
Header checksum: The IP header checksum field only verifies the header, not the payload, indicating that IP is not entirely reliable as it doesn't affirm the payload remains unaltered during transmission.

- Due to alterations in fields such as TTL at every router, the checksum needs frequent recalculations.
- Upper-level protocols encapsulating data in the IPv4 datagram maintain separate checksums that cover the complete packet, thus the IPv4 datagram checksum doesn't validate the contained data.
- The IPv4 packet's header, which changes at each visited router (but not the data), is the only section included in the checksum, preventing unnecessary increases in processing time from recalculating the entire packet's checksum at every router.



KnowledgeGate
www.knowledgegate.in

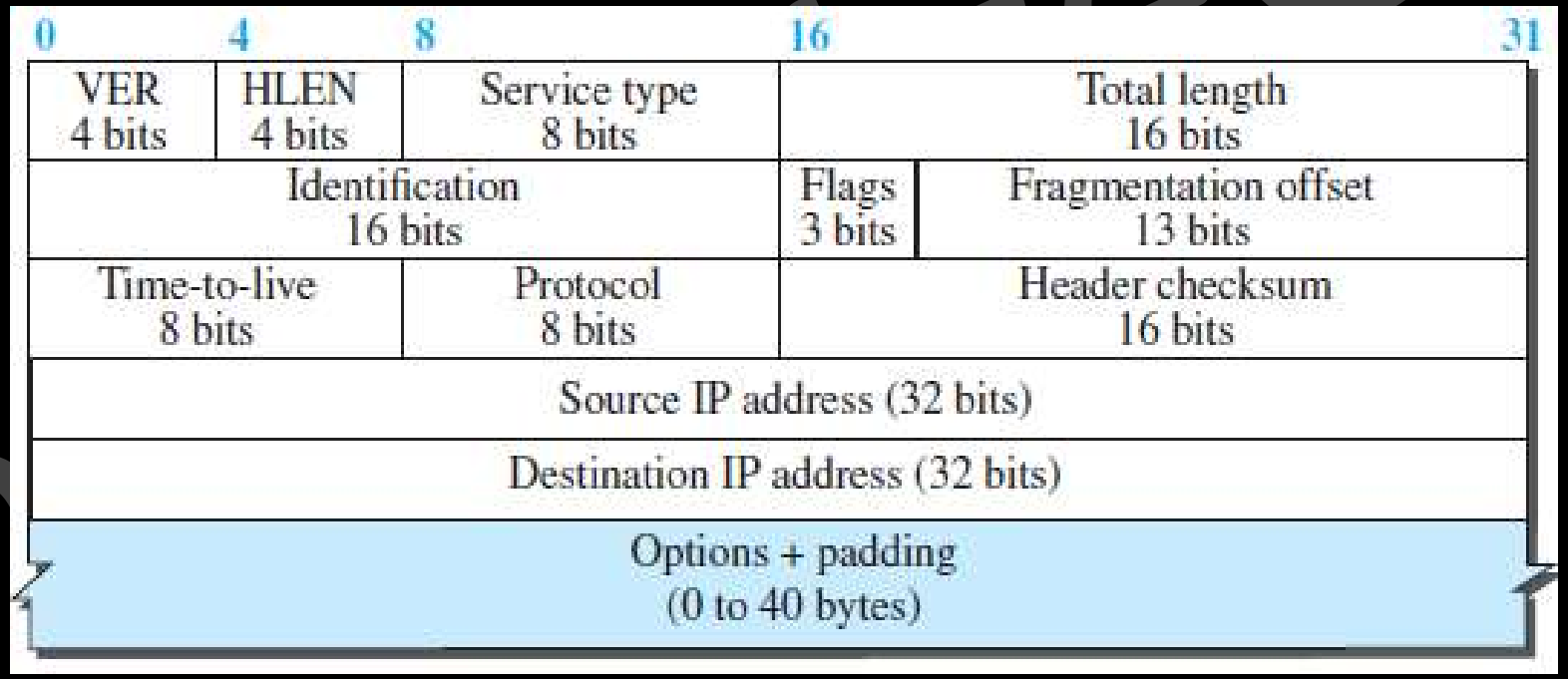
- **Source and Destination Addresses:** These 32-bit source and destination address fields define the IP address of the source and destination respectively.



www.knowledgegate.in

Variable part

- The variable part comprises the options that can be a maximum of 40 bytes. Options, as the name implies, are not required for a datagram. They can be used for network testing and debugging.



www.knowledgegate.in

- **End of Option**
  - An end-of-option option is a 1-byte option used for padding at the end of the option field. It, however, can only be used as the last option.

- **Record Route**
  - A record route option is used to record the Internet routers that handle the datagram. It can list up to nine router addresses. It can be used for debugging and management purposes.

KnowledgeGate

- **Strict Source Route**
  - A strict source route option is used by the source to predetermine a route for the datagram as it travels through the Internet. Dictation of a route by the source can be useful for several purposes.
  - The sender can choose a route with a specific type of service, such as minimum delay or maximum throughput.
  - Alternatively, it may choose a route that is safer or more reliable for the sender's purpose. For example, a sender can choose a route so that its datagram does not travel through a competitor's network.
  - If a datagram specifies a strict source route, all the routers defined in the option must be visited by the datagram. A router must not be visited if its IPv4 address is not listed in the datagram. If the datagram visits a router that is not on the list, the datagram is discarded and an error message is issued.
  - If the datagram arrives at the destination and some of the entries were not visited, it will also be discarded and an error message issued.

- **Loose Source Route**
  - A loose source route option is similar to the strict source route, but it is less rigid. Each router in the list must be visited, but the datagram can visit other routers as well.

KnowledgeGate

www.knowledgegate.in

- **Timestamp**
  - A timestamp option is used to record the time of datagram processing by a router. The time is expressed in milliseconds from midnight, Universal time or Greenwich mean time.
  - Knowing the time, a datagram is processed can help users and managers track the behaviour of the routers in the Internet. We can estimate the time it takes for a datagram to go from one router to another. We say estimate because, although all routers may use Universal time, their local clocks may not be synchronized.

KnowledgeGate

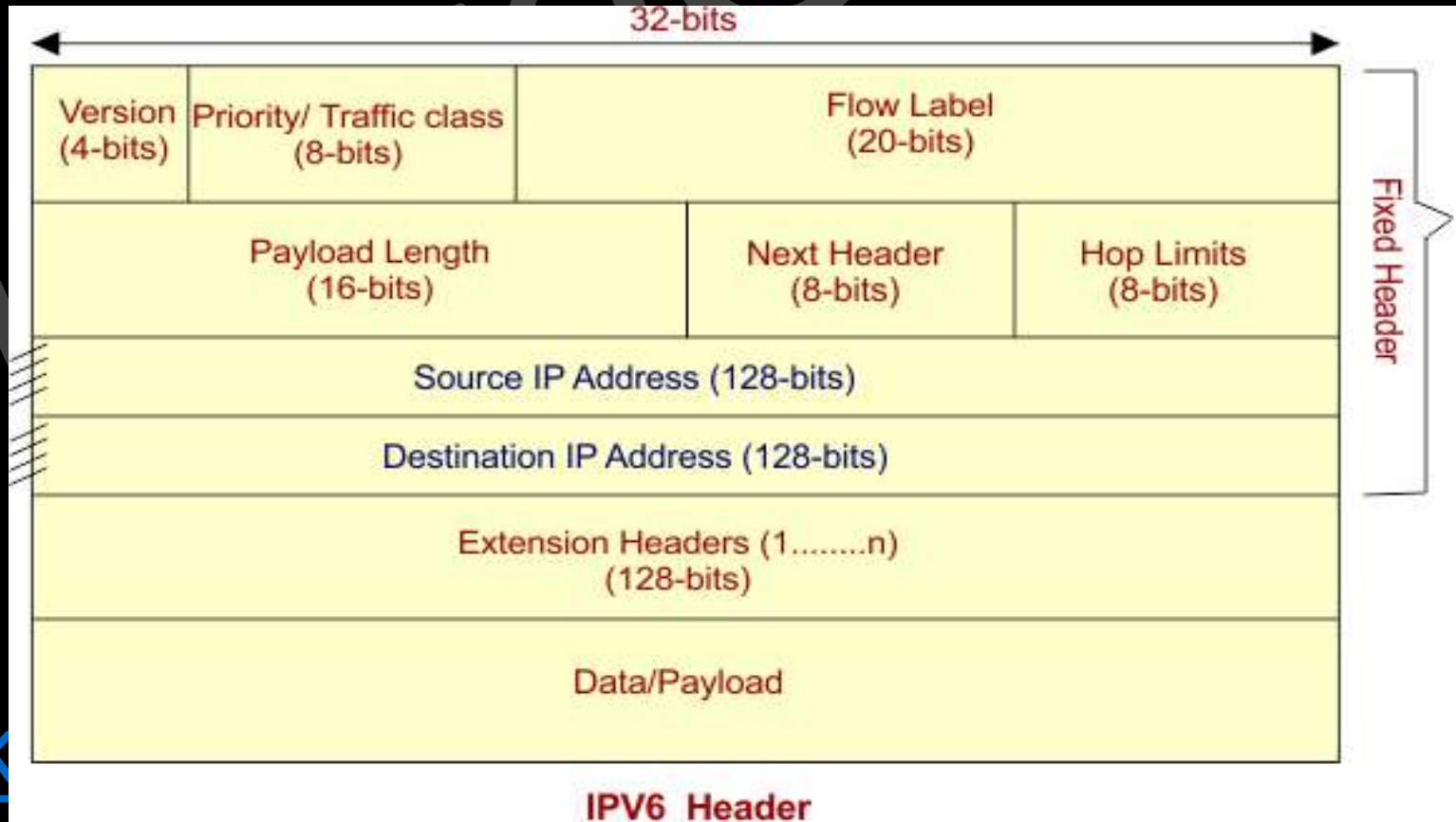
www.knowledgegate.in

IPv6

- Each packet can be divided into two parts :
  - Base header
  - Payload

- The payload is made up of two parts :
  - An optional extension header
  - The upper layer data

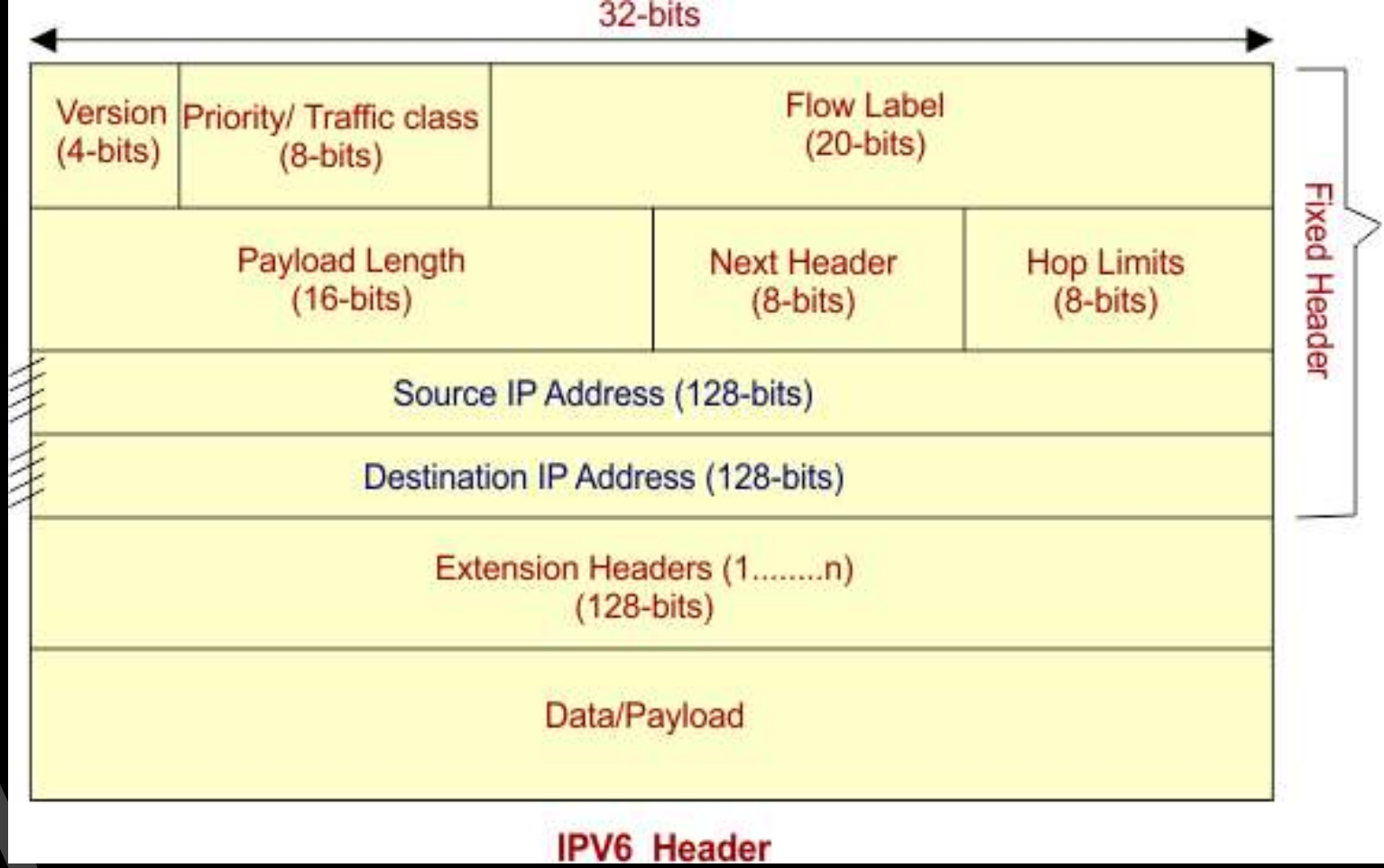
- Base header has eight fields



IPV6 Header

www.k
Know
oGate

i. Version: This is 4-bit field which defines the version number of IP. For IPv6, the value is 6.
ii. Priority: The 4-bit priority field defines the priority of the packet with respect to traffic congestion.
iii. Flow label: The flow label is a 3-byte field that is designed to provide special handling for a particular flow of data.
iv. Payload length: The 2-byte payload length field defines the total length of IP datagram excluding the base header.
v. Next header: The next header is an 8-bit field defines the header that follows the base header in the datagram.



IPV6 Header

vi. Hop limit: This 8-bit hop limit field serves the same purpose as TTL field in IPv4.
vii. Source address: The source address field is a 16-bytes internet address that identifies the original source of datagram.
viii. Destination address: The destination address field is a 16-byte internet address that usually identifies the final destination of the datagram.
ix. Extension header: Extension header field help in processing of data packets by appending different extension header. Each extension header has a length equal to multiple of 64-bits.

www.knowledgegate.in

1. Address Length
    1. IPv4: 32-bit (4 bytes)
    2. IPv6: 128-bit (16 bytes)
2. Address Notation
    1. IPv4: Decimal (e.g., 192.168.1.1)
    2. IPv6: Hexadecimal (e.g., 2001:0db8:85a3:0000)
3. Number of Addresses
    1. IPv4: Approximately 4.3 billion
    2. IPv6: Approximately 340 undecillion
4. Configuration
    1. IPv4: Manual or DHCP
    2. IPv6: Stateless address autoconfiguration (SLAAC) or DHCPv6
5. Security
    1. IPv4: Initially lacked, added later as extensions
    2. IPv6: Inbuilt support for IPsec, providing network security at the IP layer

KnowledgeGate

www.knowledgegate.in

Advancements in IPv6:

- **Larger Address Space**: Can accommodate a virtually unlimited number of unique addresses, facilitating the growth of the internet.
- **Simplified Header**: IPv6 has a simpler header structure, which improves the speed of routing by allowing routers to process packets more efficiently.
- **Improved Multicast**: IPv6 utilizes multicast addressing to send data packets to multiple destinations in a single transmission, which conserves bandwidth compared to IPv4.
- **No NAT Required**: IPv6 was designed to eliminate the need for NAT, making end-to-end connection more straightforward and reducing latency and complexity.
- **Mobility and Security**: IPv6 was built considering modern requirements, offering better support for mobile devices and higher levels of security.

KnowledgeGate

www.knowledgegate.in

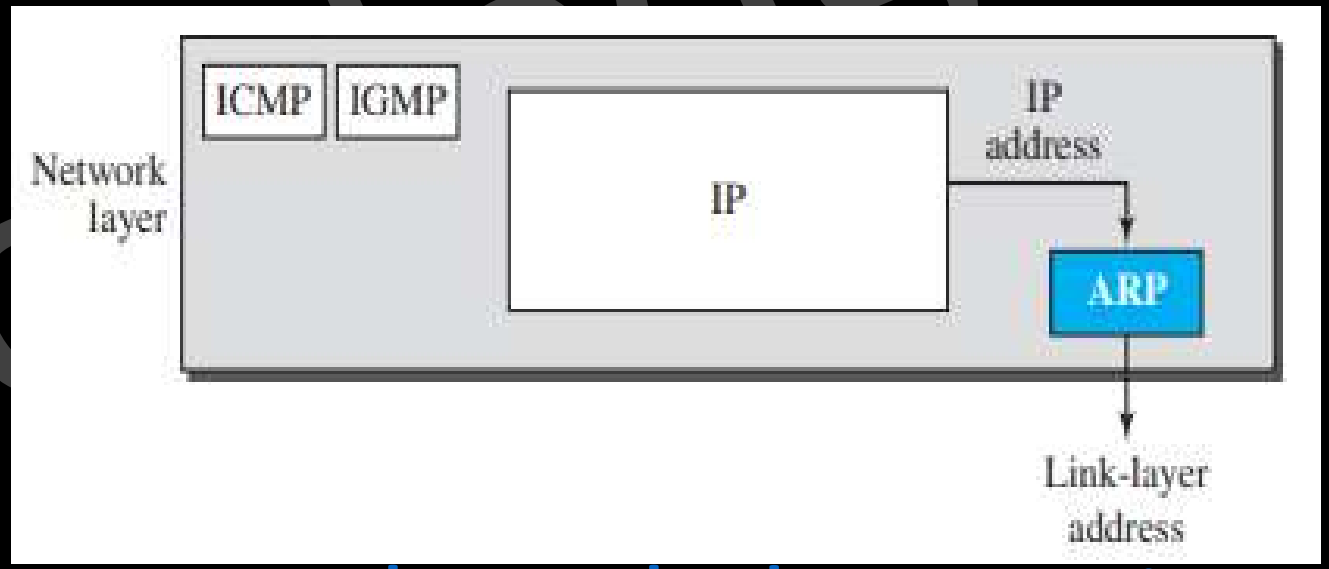
Need of Additional protocols

- IP packets, however, need to be encapsulated in a frame, which needs physical addresses (node-to-node). We will see that a protocol called ARP, the **Address Resolution Protocol**.
- We sometimes need reverse mapping-mapping a physical address to a logical address. For example, when booting a diskless network or leasing an IP address to a host, **RARP** is used.
- Lack of flow and error control in the Internet Protocol has resulted in another protocol, **ICMP**, that provides alerts. It reports congestion and some types of errors in the network or destination host
- IP was originally designed for unicast delivery, one source to one destination. As the Internet has evolved, the need for multicast delivery, one source to many destinations, has increased tremendously. **IGMP** gives IP a multicast capability.

KnowledgeGate

Address Resolution Protocol (ARP)

- The IP address of the next node alone is not helpful in moving a frame through a link; we need the link-layer address of the next node.
- ARP maps an IP address to a logical-link address. ARP accepts an IP address from the IP protocol, maps the address to the corresponding link-layer address, and passes it to the data-link layer.
- The ARP protocol is one of the auxiliary protocols defined in the **network layer.**



KnowledgeGate

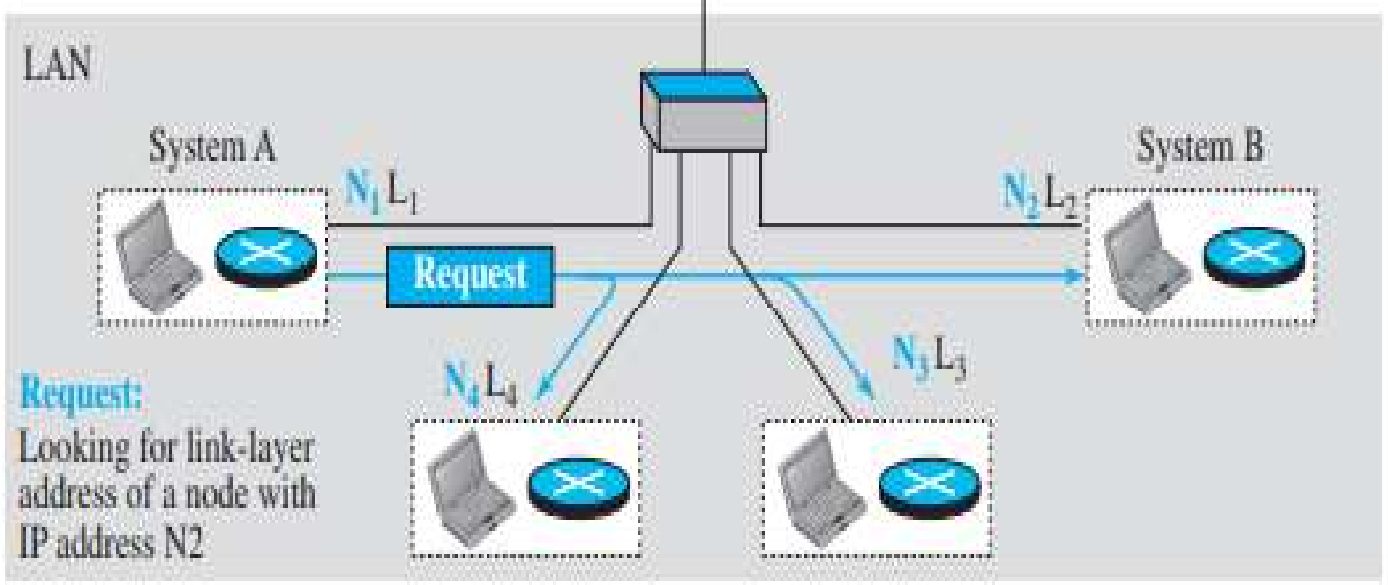
www.knowledgegate.in

1. Anytime a host or a router needs to find the link-layer address of another host or router in its network, it sends an ARP request packet. The packet includes the ***link-layer and IP addresses of the sender and the IP address of the receiver.***

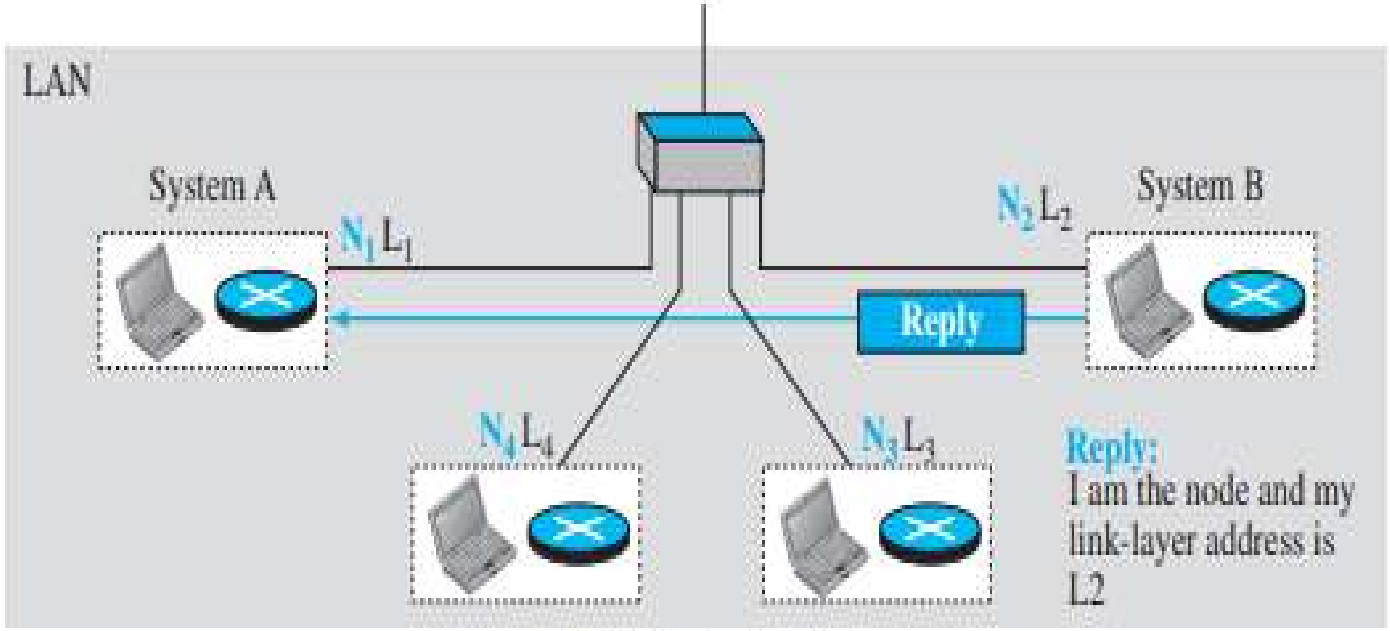
2. Because the sender does not know the link-layer address of the receiver, the query is broadcast over the link.

3. Every host or router on the network receives and processes the ARP request packet, but only the intended recipient recognizes its IP address and sends back an ARP response packet.

4. The response packet contains the recipient's IP and link-layer addresses. ***The packet is unicast directly to the node that sent the request packet.***



a. ARP request is broadcast



b. ARP reply is unicast

Knowle
www.kn

RARP

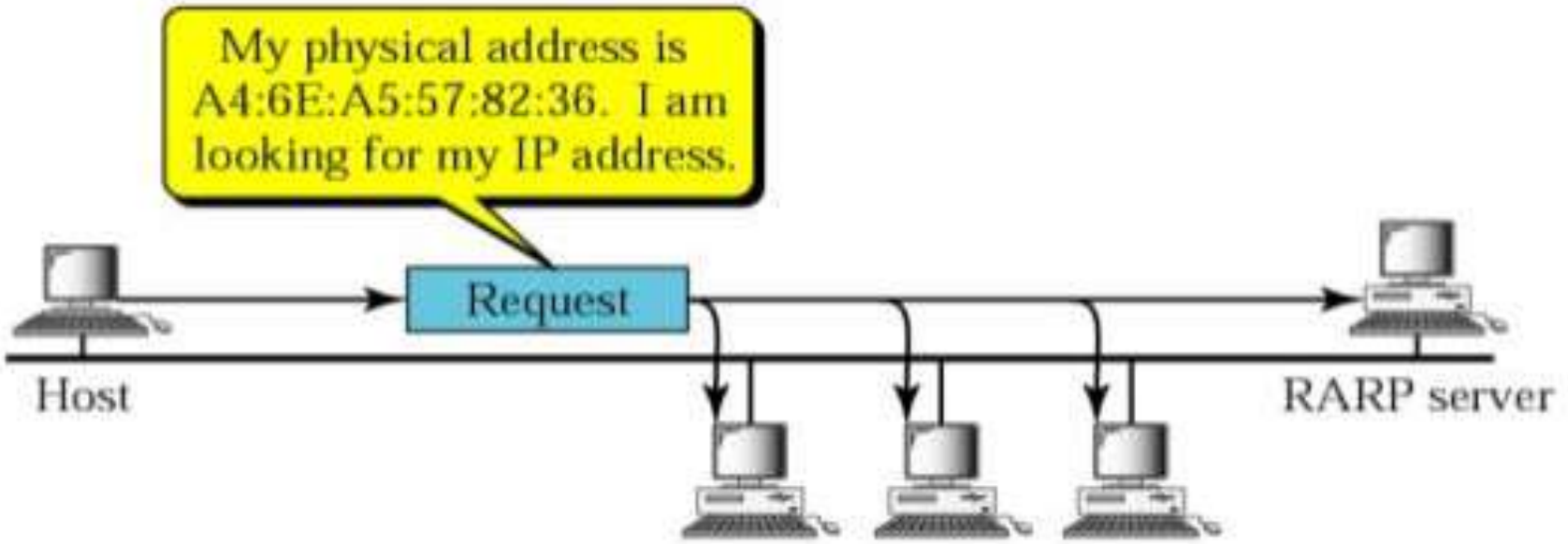
- Reverse Address Resolution Protocol (RARP) finds the logical address for a machine that knows only its physical address. Each host or router is assigned one or more logical (IP) addresses, which are unique and independent of the physical (hardware) address of the machine. To create an IP datagram, a host or a router needs to know its own IP address.

- The IP address of a machine is usually read from its configuration file stored on a disk file. However, a diskless machine is usually booted from ROM, which has minimum booting information. The ROM is installed by the manufacturer. It cannot include the IP address because the IP addresses on a network are assigned by the network administrator.

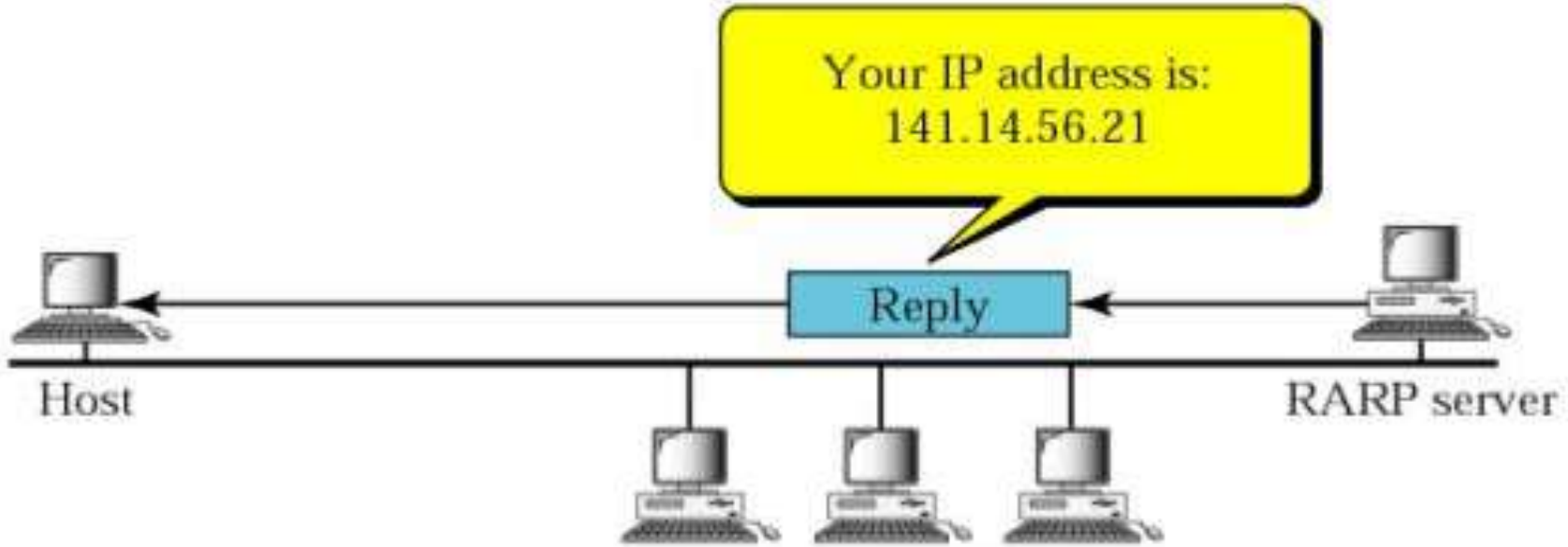
- The machine can get its physical address (by reading its NIC, for example), which is unique locally. It can then use the physical address to get the logical address by using the RARP protocol.

KnowledgeGate

- A RARP request is created and broadcast on the local network. Another machine on the local network that knows all the IP addresses will respond with a RARP reply.
- The requesting machine must be running a RARP client program; the responding machine must be running a RARP server program.



a. RARP request is broadcast



b. RARP reply is unicast

ICMP

- IP has two deficiencies: lack of error control and lack of assistance mechanisms. The IP protocol has no error-reporting or error-correcting mechanism.
  - What happens if something goes wrong?
  - What happens if a router must discard a datagram because it cannot find a router to the final destination,
  - or because the time-to-live field has a zero value?
  - What happens if the final destination host must discard all fragments of a datagram because it has not received all fragments within a predetermined time limit?

KnowledgeGate

- These are examples of situations where an error has occurred and the IP protocol has no built-in mechanism to notify the original host. The IP protocol also lacks a mechanism for host and management queries. A host sometimes needs to determine if a router or another host is alive. And sometimes a network administrator needs information from another host or router.

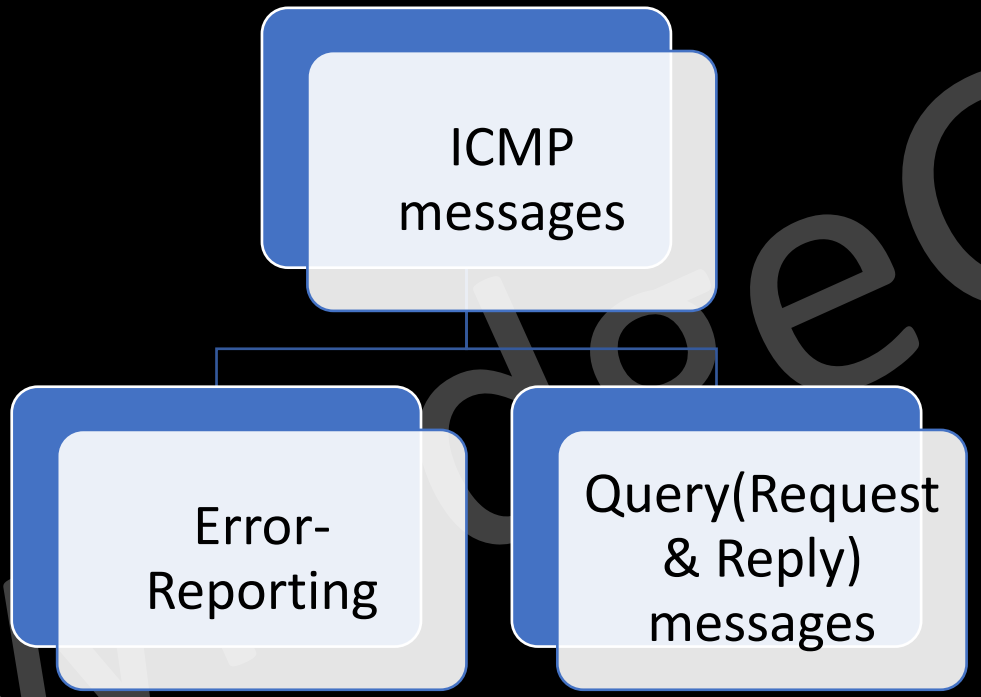
- The Internet Control Message Protocol (ICMP) has been designed to compensate for the above two deficiencies. It is a companion to the IP protocol.

KnowledgeGate

www.knowledgegate.in

Types of Messages

- ICMP messages are divided into two broad categories: Error-Reporting messages and query(request & reply) messages.



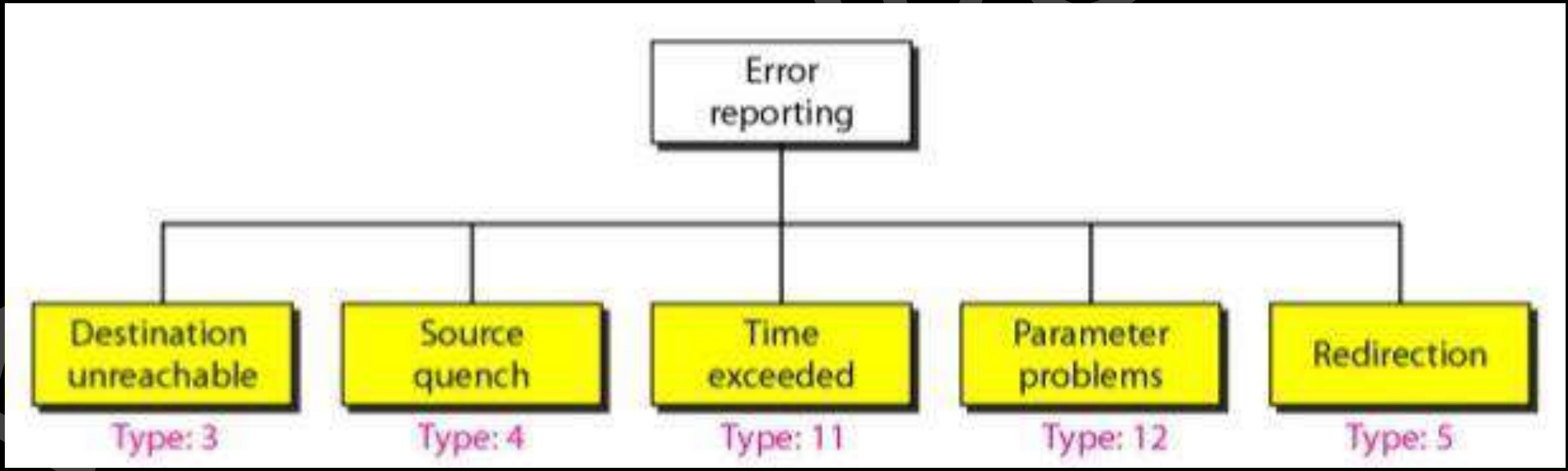
The error-reporting messages report problems that a router or a host (destination) may encounter when it processes an IP packet.

The query messages, which occur in pairs, help a host or a network manager get specific information from a router or another host.

KnowledgeGate

Error Reporting

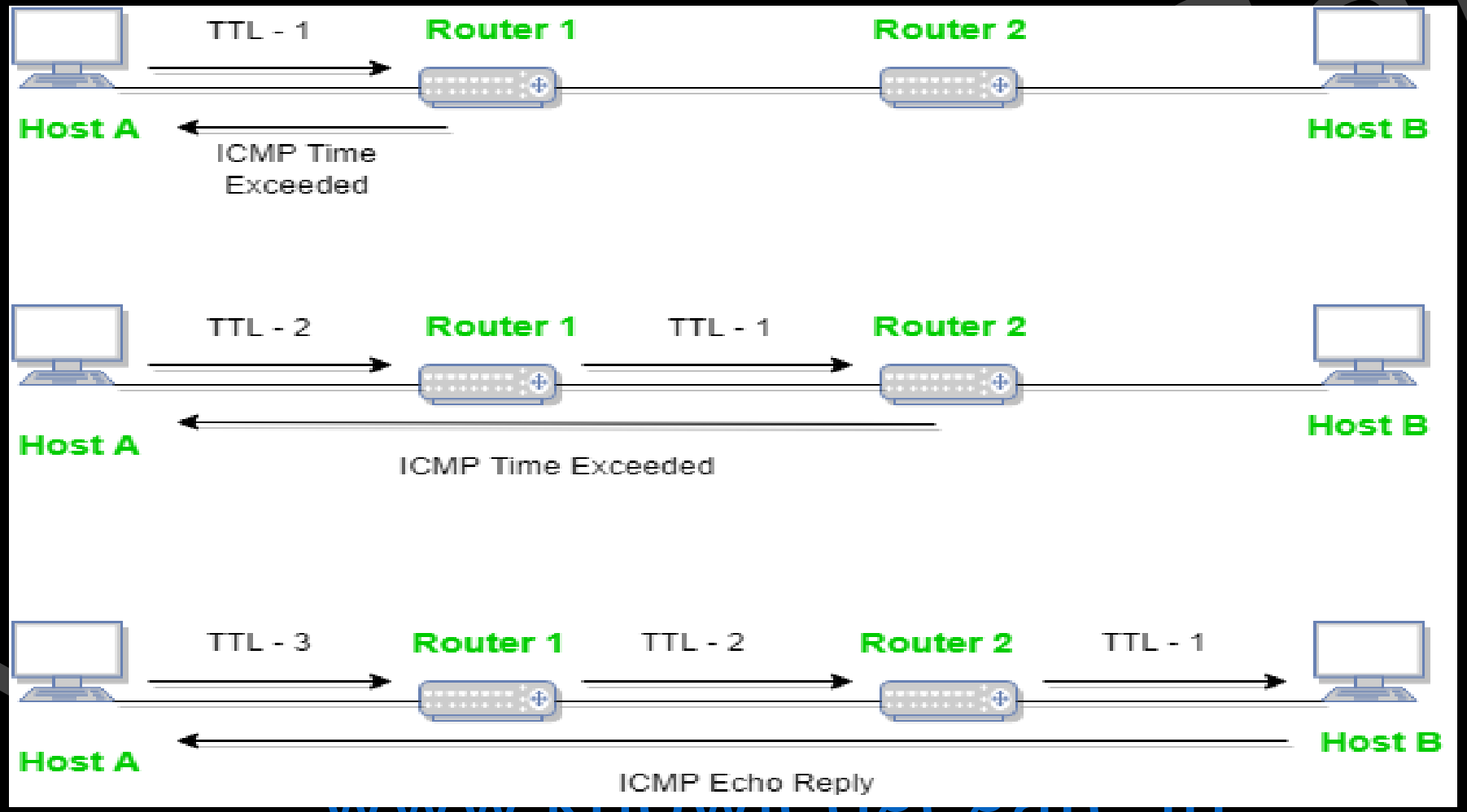
- ICMP does not correct errors-it simply reports them. Error correction is left to the higher-level protocols. Error messages are always sent to the original source because the only information available in the datagram about the route is the source and destination IP addresses. ICMP uses the source IP address to send the error message to the source (originator) of the datagram.



www.knowledgegate.in

Time exceeded message

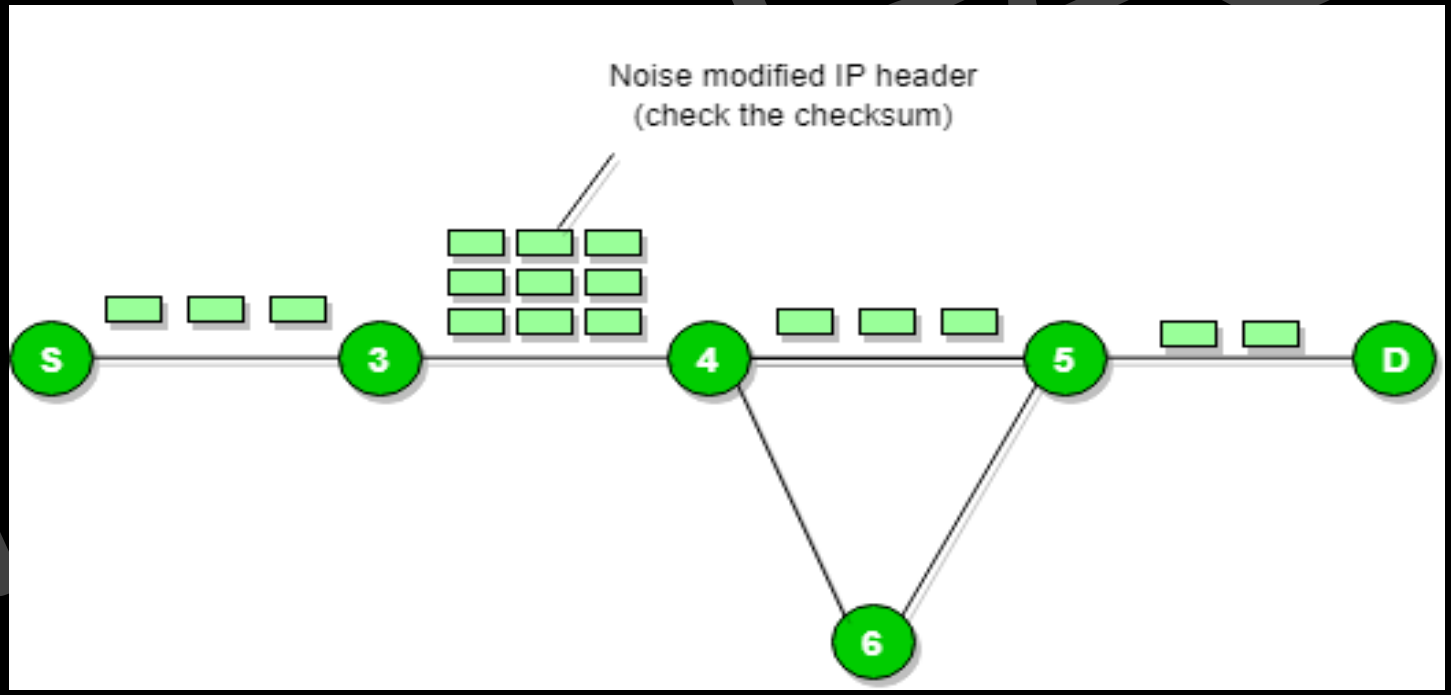
- ICMP will take source IP from discarded packet and informs to the source, of discarded datagram due to time to live field reaches to zero, by sending time exceeded message.



www.knowledgegate.in

Parameter problem

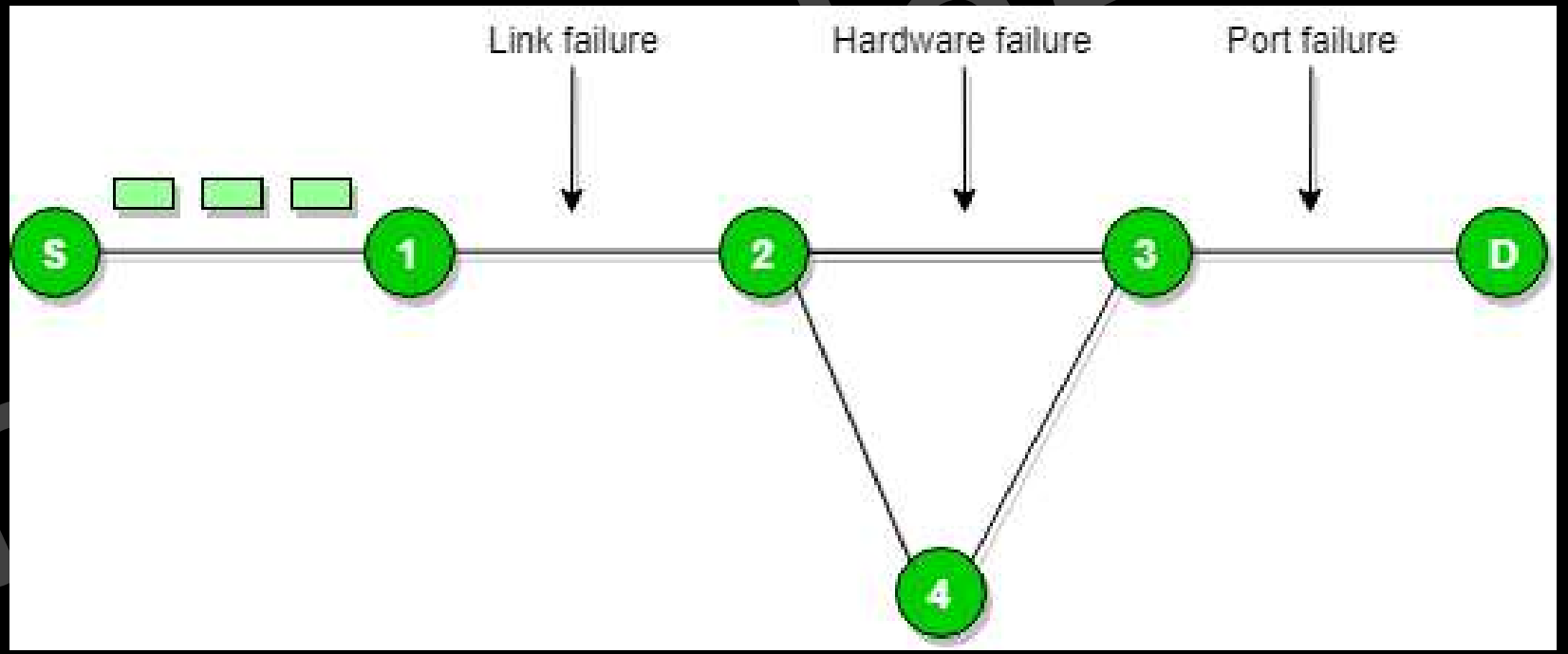
- Whenever packets come to the router then calculated header checksum should be equal to received header checksum then only packet is accepted by the router.
- If there is mismatch packet will be dropped by the router. ICMP will take the source IP from the discarded packet and informs to source by sending parameter problem message.



KnowledgeGate

Destination Un-reachable

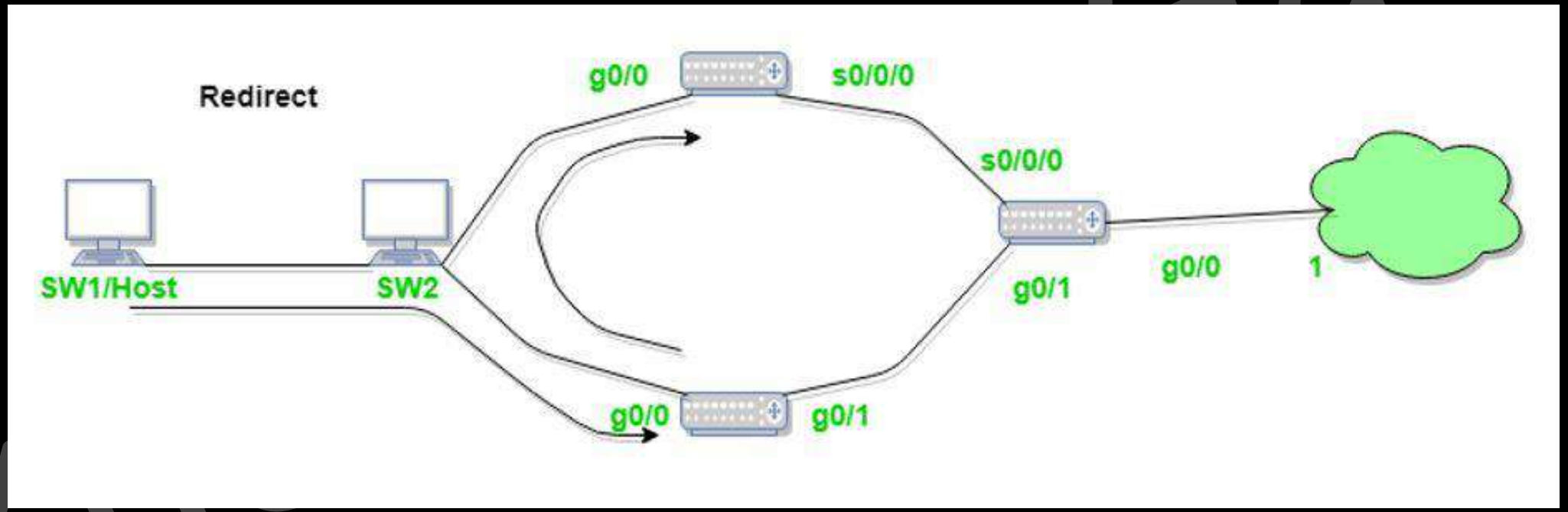
- Destination unreachable is generated by the host or its inbound gateway to inform the client that the destination is unreachable for some reason.
- There is no necessary condition that only router give the ICMP error message some time destination host send ICMP error message when any type of failure (link failure, hardware failure, port failure etc.) happen in the network.



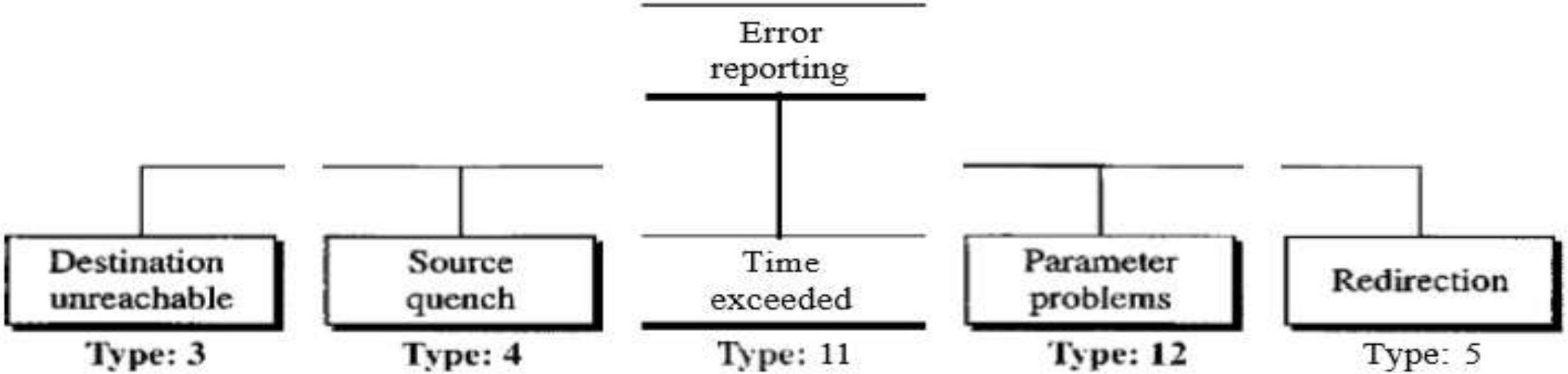
www.knowledgegate.in

Redirection message

- Redirect requests data packets be sent on an alternate route. The message informs to a host to update its routing information (to send packets on an alternate route).



www.knowledgegate.in

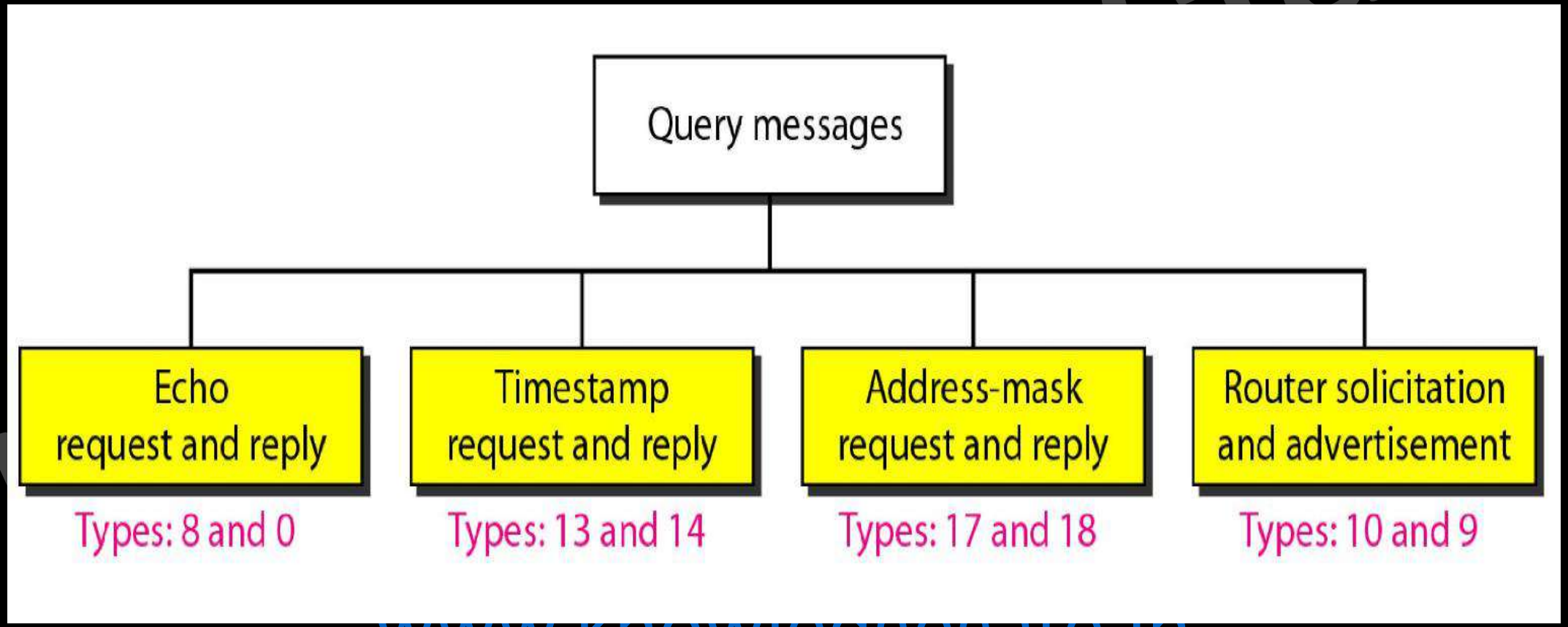


The following are important points about ICMP error messages:

- No ICMP error message will be generated in response to a datagram carrying an ICMP error message.
- No ICMP error message will be generated for a fragmented datagram that is not the first fragment.
- No IeMP error message will be generated for a datagram having a multicast address.
- No ICMP error message will be generated for a datagram having a special address such as 127.0.0.0 or 0.0.0.0.

Query

- In addition to error reporting, ICMP can diagnose some network problems. This is accomplished through the query messages, a group of four different pairs of messages.
- In this type of ICMP message, a node sends a message that is answered in a specific format by the destination node. A query message is encapsulated in an IP packet, which in turn is encapsulated in a data link layer frame.



www.knowledgegate.in

Echo Request and Reply

- Echo-request and echo-reply messages, encapsulated within IP datagrams, are essential diagnostic tools enabling network managers and users to pinpoint network issues and confirm that IP protocols in sender and receiver systems are in sync.
- Modern systems offer an advanced version of the ping command, capable of generating a series of these messages, facilitating comprehensive network analysis through the collection of vital statistical data.

KnowledgeGate

Router Solicitation and Advertisement

- As we discussed in the redirection message section, a host that wants to send data to a host on another network needs to know the address of routers connected to its own network. Also, the host must know if the routers are alive and functioning. The router-solicitation and router-advertisement messages can help in this situation.
- A host can broadcast (or multicast) a router-solicitation message. The router or routers that receive the solicitation message broadcast their routing information using the router-advertisement message.
- A router can also periodically send router-advertisement messages even if no host has solicited. Note that when a router sends out an advertisement, it announces not only its own presence but also the presence of all routers on the network of which it is aware.

KnowledgeGate

Address-Mask Request and Reply

- A host may know its IP address, but it may not know the corresponding mask. For example, a host may know its IP address as 159.31.17.24, but it may not know that the corresponding mask is /24.
- To obtain its mask, a host sends an address-mask-request message to a router on the LAN. If the host knows the address of the router, it sends the request directly to the router. If it does not know, it broadcasts the message.
- The router receiving the address-mask-request message responds with an address-mask-reply message, providing the necessary mask for the host. This can be applied to its full IP address to get its subnet address.

KnowledgeGate

Timestamp Request and Reply

- Two machines (hosts or routers) can use the timestamp request and timestamp reply messages to determine the round-trip time needed for an IP datagram to travel between them. It can also be used to synchronize the clocks in two machines.

KnowledgeGate

www.knowledgegate.in

IGMP

- The IP protocol can be involved in two types of communication: unicasting and multicasting. Unicasting is the communication between one sender and one receiver. It is a one-to-one communication.
- However, some processes sometimes need to send the same message to a large number of receivers simultaneously. This is called multicasting, which is a one-to-many communication.
- Multicasting has many applications. For example, multiple stockbrokers can simultaneously be informed of changes in a stock price, or travel agents can be informed of a plane cancellation.

KnowledgeGate

Video-on-Demand

www.knowledgegate.in

Distance Learning

www.knowledgegate.in

Congestion

- Congestion is a situation which may occur if users send data into the network at a rate greater than that allowed by network resources.
- **Congestion control** : Congestion control refers to techniques and mechanisms that can either prevent congestion, before it happens, or remove congestion, after it has happened. Techniques to prevent congestion :
- **Open-loop congestion control** : In open-loop congestion control, policies are applied to prevent congestion before it happens. In these mechanisms, congestion control is handled by either the source or the destination. Following are the policies that can prevent congestion :
  - i. **Retransmission policy** : The retransmission policy is designed to optimize efficiency and at the same time prevent congestion.
  - ii. **Window policy** : The type of window at the sender may also affect congestion. The selective repeat window is better than the Go-Back-N window for congestion control.
  - iii. **Acknowledgement policy** : The acknowledgement policy imposed by the receiver may also affect congestion. If the receiver does not acknowledge every packet it receives, it may slow down the sender and help to prevent congestion.

KnowledgeGate

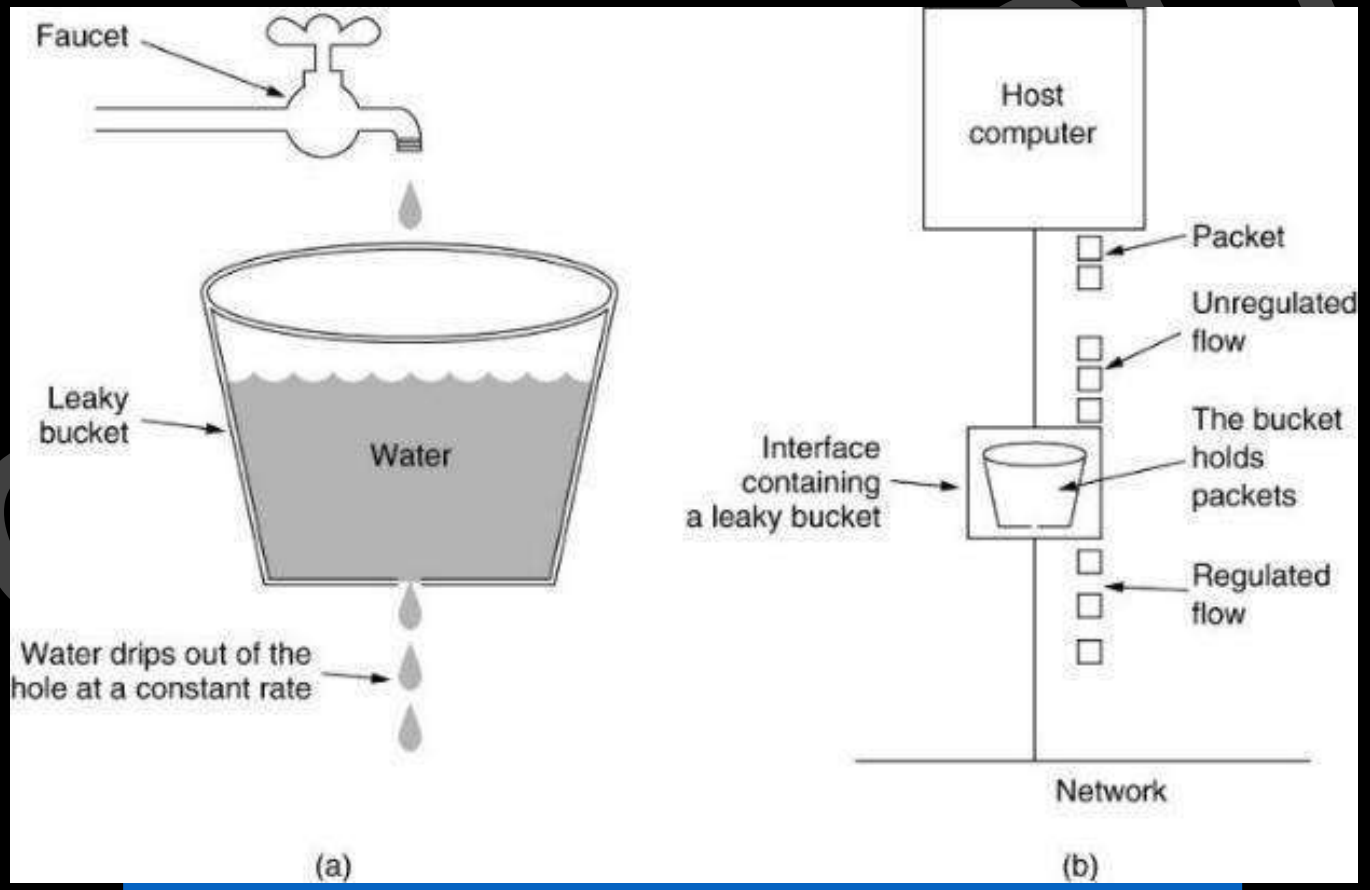
- Closed-loop congestion control : Closed-loop congestion control mechanisms try to reduce congestion after it happens. Several mechanisms have been used by different protocols which are as follows :
  - i. **Backpressure** : The technique of backpressure refers to mechanism in which a congested node stops congestion control m)receiving data from the immediate upstream node or nodes.
  - ii. **Choke packet** : A choke packet is a packet sent by a node to the source to inform about congestion. In the choke packet method ,the warning is from the router, which has encountered congestion to the source station directly.
  - iii. **Implicit signaling** : In implicit signaling, there is no communication between the congested node or nodes and the source.
  - iv. **Explicit signaling** : The node that experiences congestion can explicitly send a signal to the source or destination.

KnowledgeGate

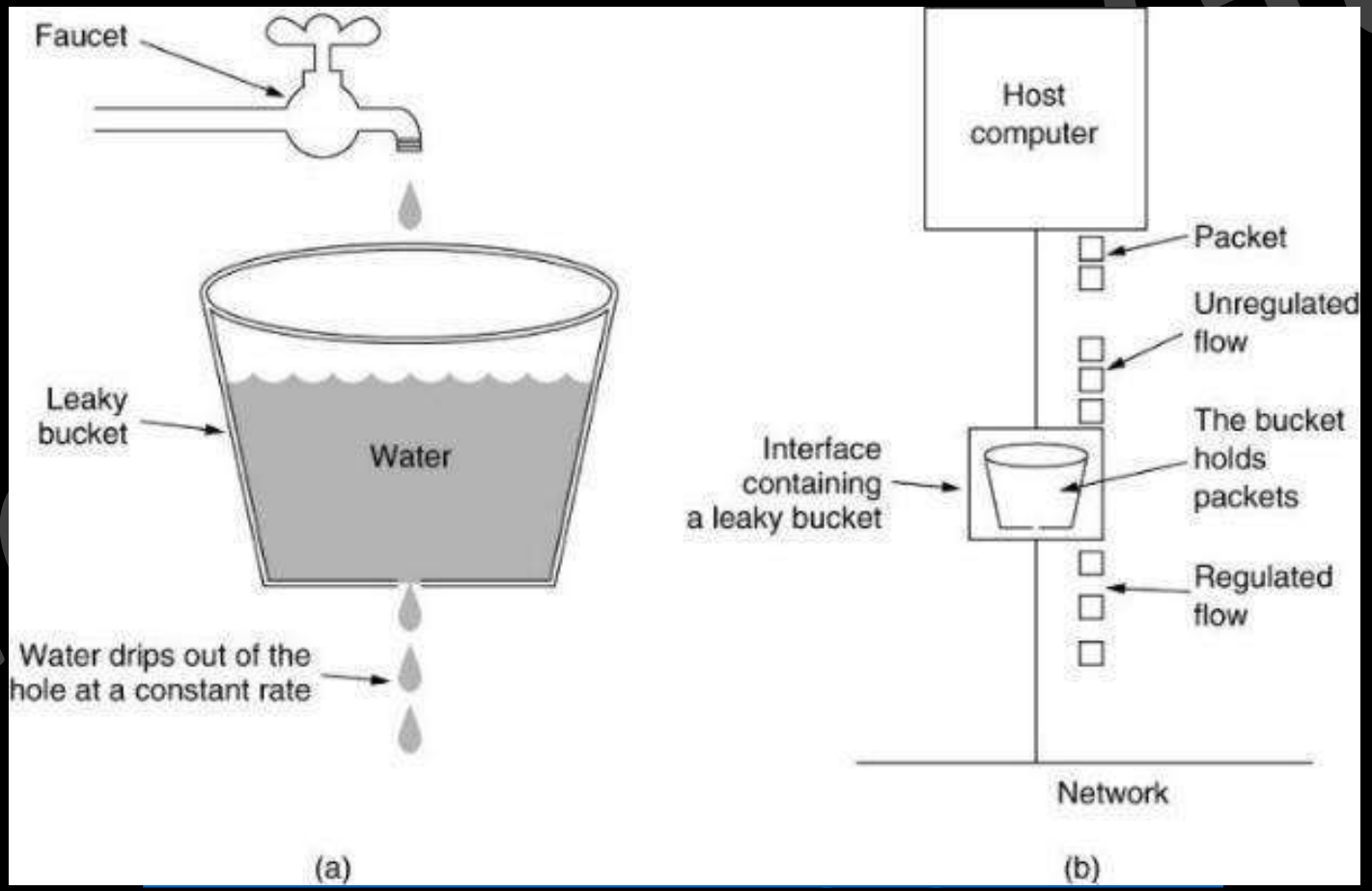
www.knowledgegate.in

Leaky bucket algorithm

- If a bucket has a small hole at the bottom, the water leaks from the bucket at a constant rate as long as there is water in the bucket. The rate at which the water leaks does not depend on the rate at which the water is input to the bucket unless the bucket is empty. The input rate can vary, but the output rate remains constant.
- Similarly, in networking, a technique called leaky bucket which can smooth out bursty traffic. A simple leaky bucket implementation a FIFO queue holds the packets. If the traffic consists of fixed size packets the process removes a fixed number of packets from the queue at each tick of the clock. If the traffic consists of variable length packets, the fixed output rate must be based on the number of bytes or bits.



- The following is an algorithm for variable length packets:
  - Initialize a counter to n at the tick of the clock.
  - If n is greater then the size of the packet, send the packet and decrement the counter by the packet size. Repeat this step until n is smaller than the packet size.
  - Reset the counter and go to step(i).



Token Bucket Algorithm:

1.Concept: It is another algorithm used to control data rate in networks.

2.Working: Tokens are added to the bucket at a fixed rate. Data packets can be transmitted if sufficient tokens are available.

3.Solution: It allows data to be sent at varying speeds, permitting bursts of data until the bucket is empty of tokens.

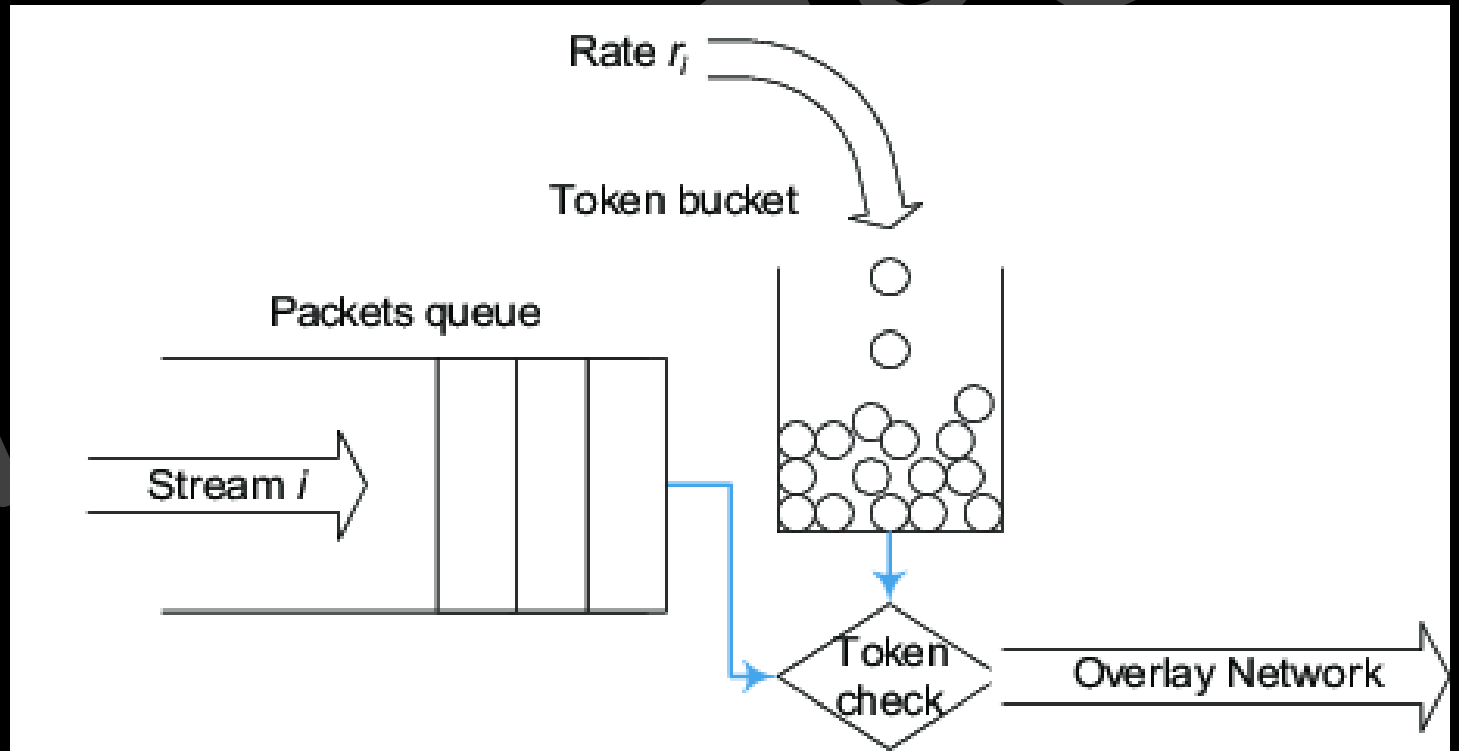
Advantage: Offers more flexibility than the leaky bucket as it accommodates bursts of high-speed data without data loss.

How Token Bucket Solves the Problem of Leaky Bucket:

1.Flexibility: It allows for bursts of data, adapting better to varying network speeds.

2.Prevents Data Loss: Data is not discarded suddenly as in the leaky bucket algorithm, reducing the chances of data loss.

3.Better Utilization of Bandwidth: By allowing data bursts, it utilizes available bandwidth more efficiently.



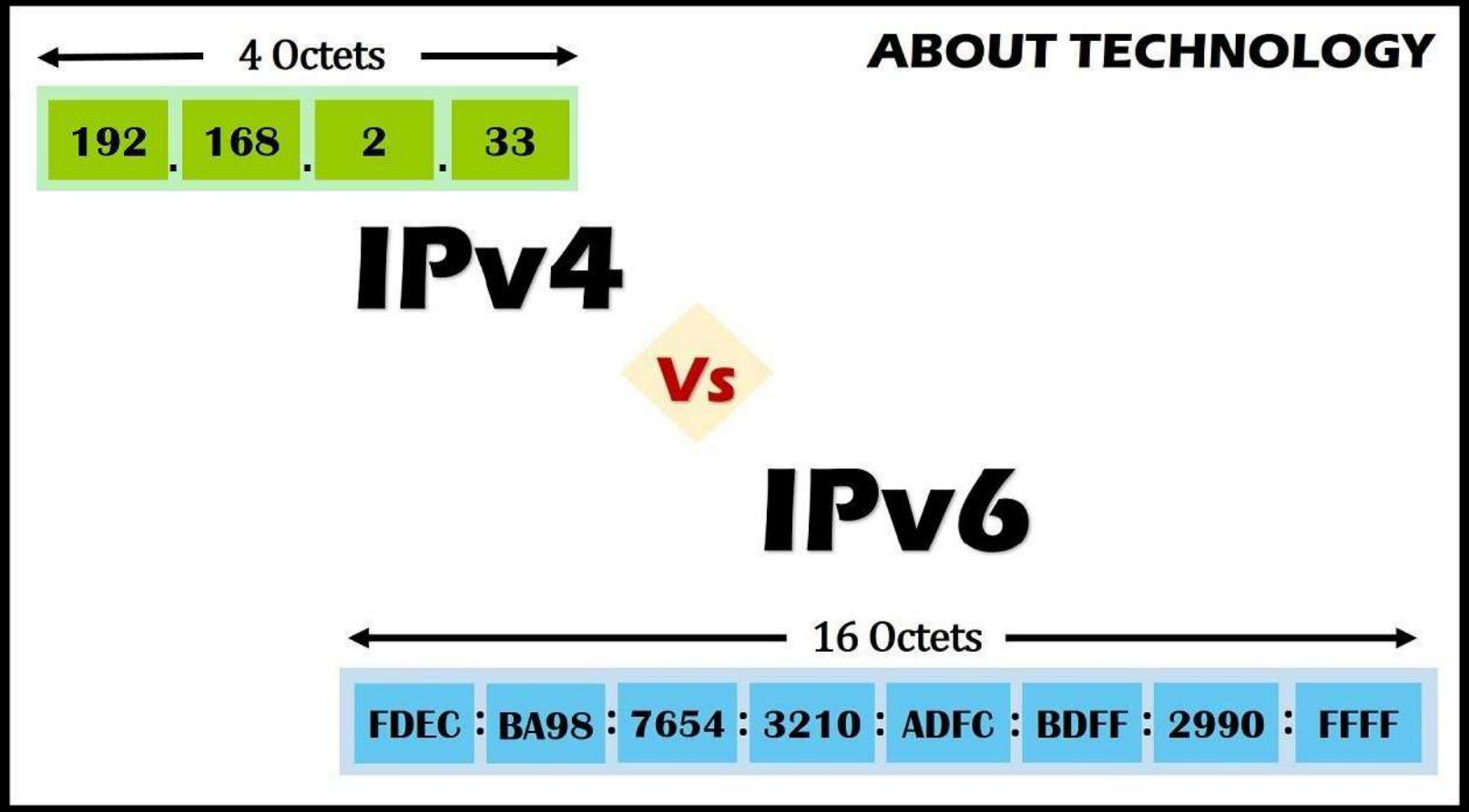
KnowledgeGate

IPV4 ADDRESSES

- The Internet Protocol addresses are 32 bits in length; this gives us a maximum of 2^{32} addresses. These addresses are referred to as IPv4.
- This means that, theoretically, if there were no restrictions, more than 4 billion (4,29,49,67,296) devices could be connected to the Internet. The actual number is much less because of the restrictions imposed on the addresses.
- **World population** is often used to refer to the total number of humans currently living, and was estimated to have exceeded 8.0 billion as of Sept 2023

KnowledgeGate

- The need for more addresses, in addition to other concerns about the IP layer, motivated a new design of the IP layer called the new generation of IP or IPv6 (IP version 6). In this version, the Internet uses 128-bit addresses that give much greater flexibility in address allocation ($3.4 * 10^{38}$). These addresses are referred to as IPv6 (IP version 6) addresses.



www.knowledgegate.in

Unique and Universal

- An IP address is uniquely and universally defining the connection of a host or a router to the Internet.
- They are unique in the sense that each address defines one, and only one, connection to the Internet. Two devices on the Internet can never have the same address at the same time.
- The IPv4 addresses are universal in the sense that the addressing system must be accepted by any host that wants to be connected to the Internet.
- The IP address is the address of the connection, not the host or the router, because if the device is moved to another network, the IP address may be changed.

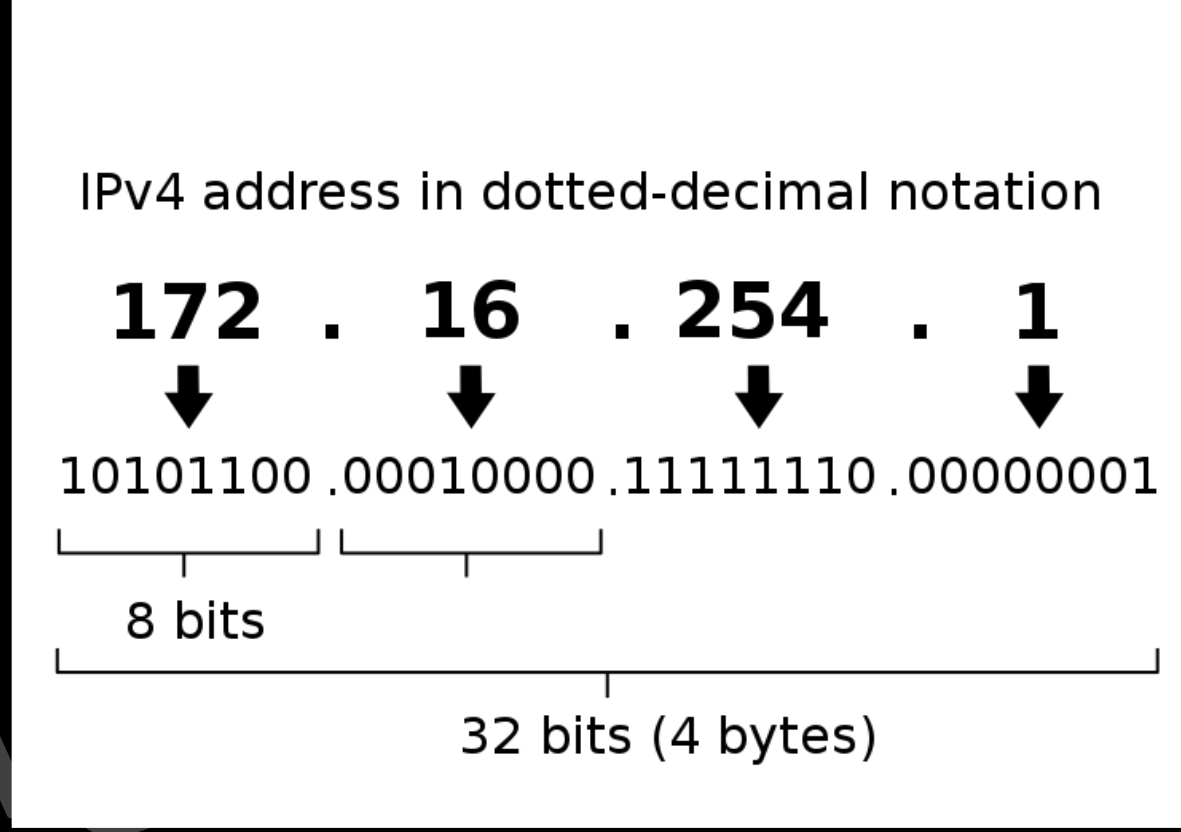
KnowledgeGate

Notations

- There are two prevalent notations to show an IPv4 address: binary notation and dotted decimal notation.

- **Binary Notation** - In binary notation, the IPv4 address is displayed as 32 bits. Each octet is often referred to as a byte. So, it is common to hear an IPv4 address referred to as a 32-bit address or a 4-byte address. An example of an IPv4 address in binary notation:
  01110101 10010101 00011101 00000010

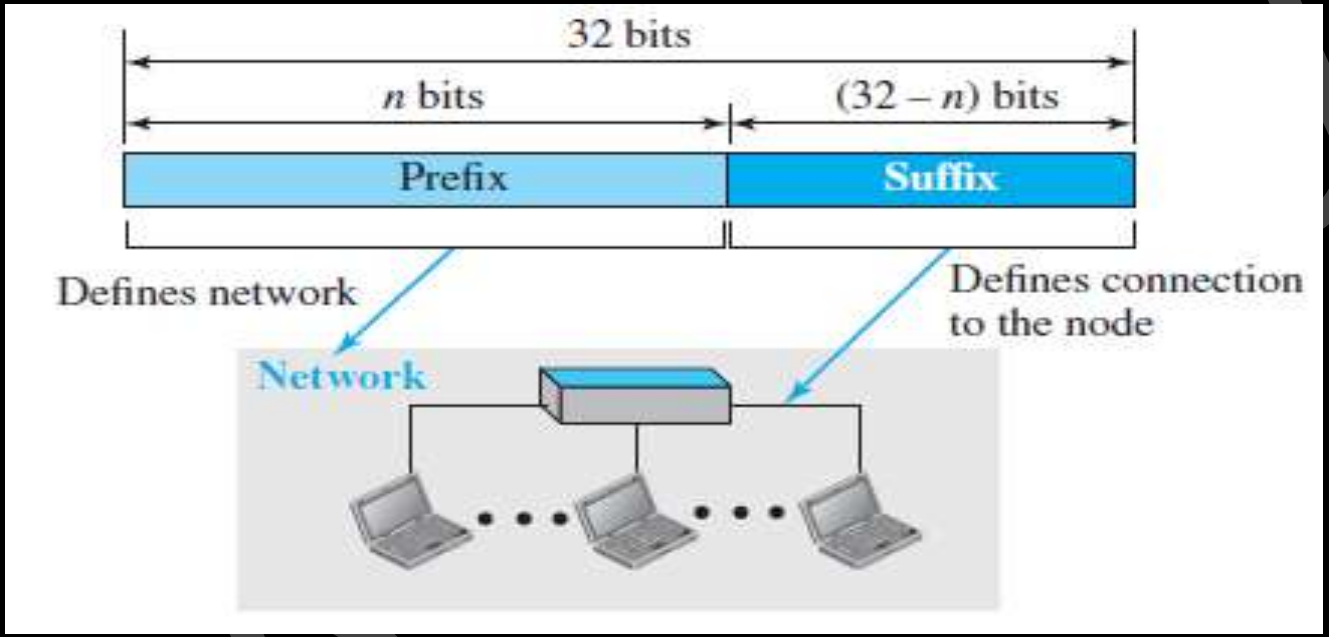
- **Dotted-Decimal Notation** - To make the IPv4 address more compact and easier to read, Internet addresses are usually written in decimal form with a decimal point (dot) separating the bytes. The following is the dotted decimal notation of the above address:
  117.149.29.2

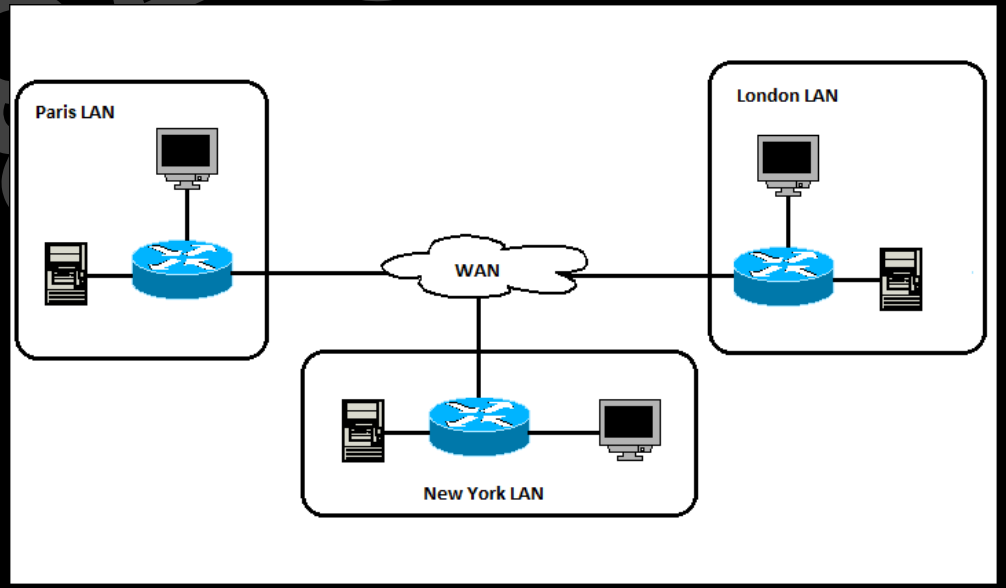


Knowledge

Classful Addressing

- IPv4 addressing, at its inception, used the concept of classes. This architecture is called classful addressing.
- A 32-bit IPv4 address is hierarchical and divided only into two parts:
- The first part of the address, called the *prefix*, defines the network (NetworkID).
- The second part of the address, called the *suffix*, defines the node (connection of a device to the Internet (HostID)).





www.knowledgegate.in

- IPv4 was first designed as a fixed-length prefix and is referred to as classful addressing
- In classful addressing, the address space is divided into five classes: A, B, C, D, and E. Each class occupies some part of the address space.
- To accommodate both small and large networks, three fixed-length prefixes were designed (*n* = 8, *n* = 16, and *n* = 24).

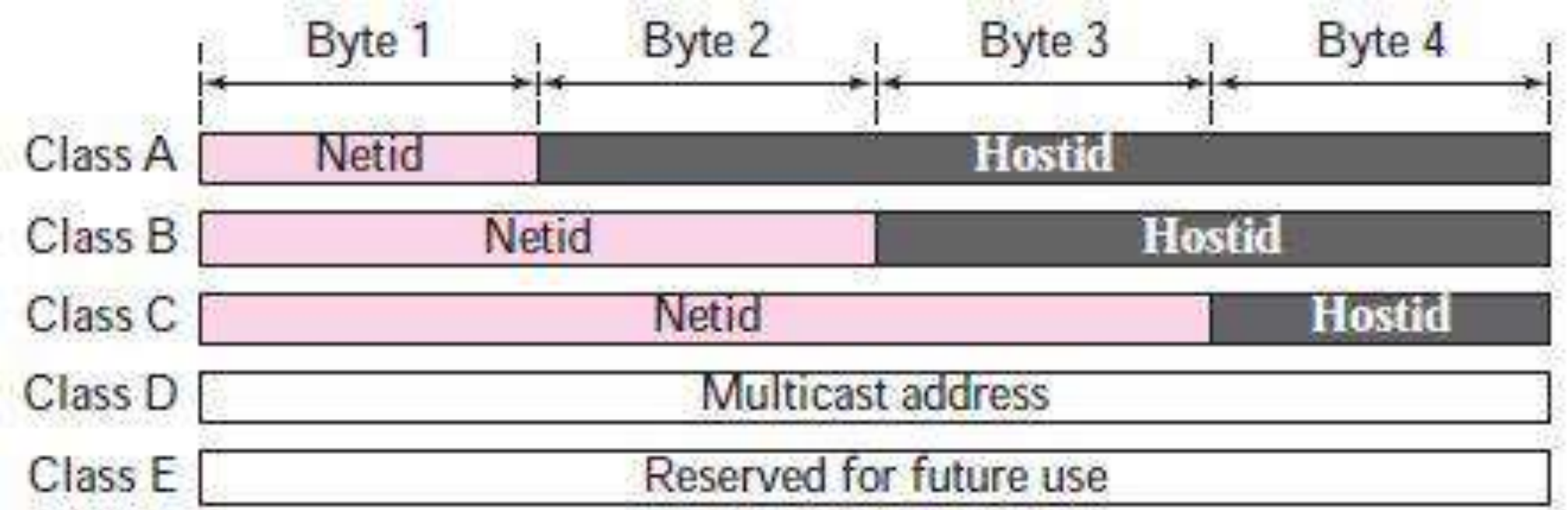
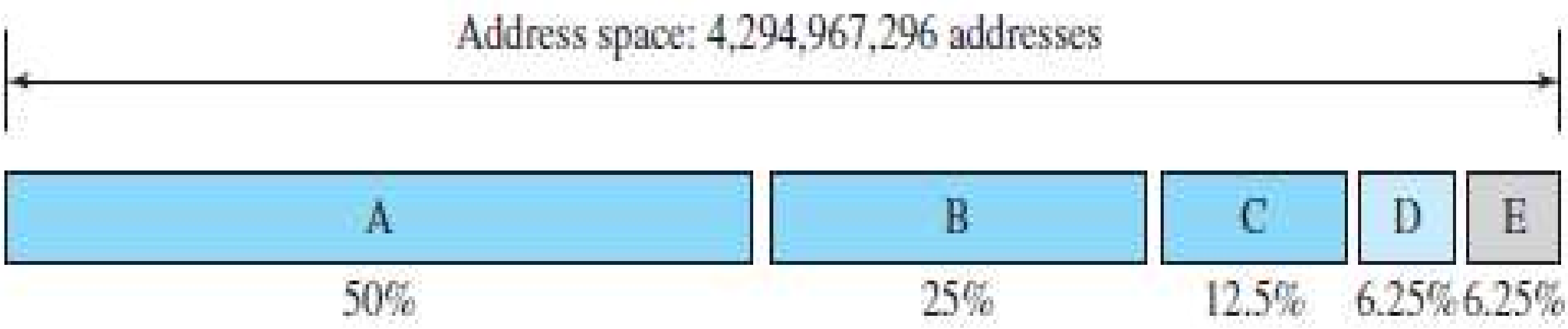
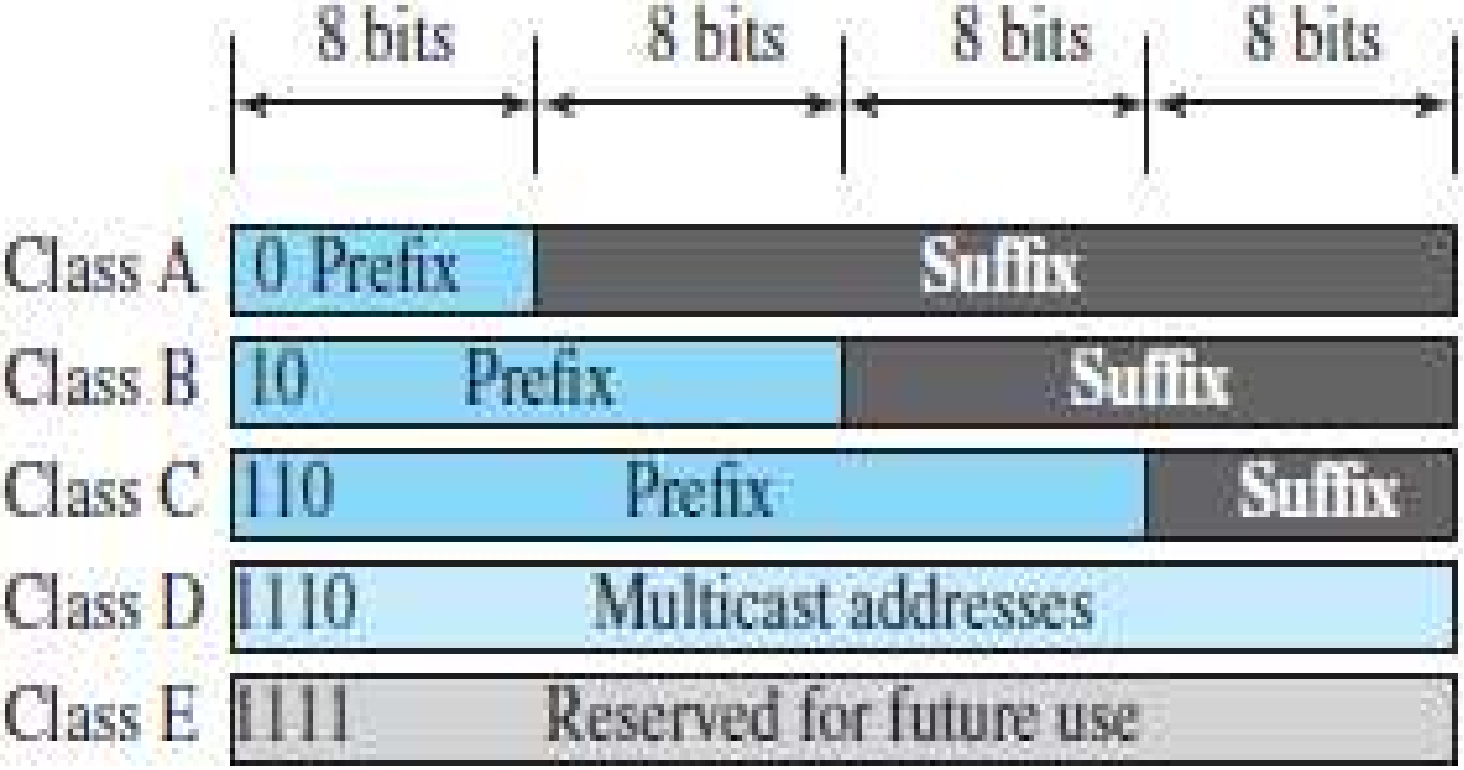


Fig1: Netid and hostid

www.knowledgegate.in

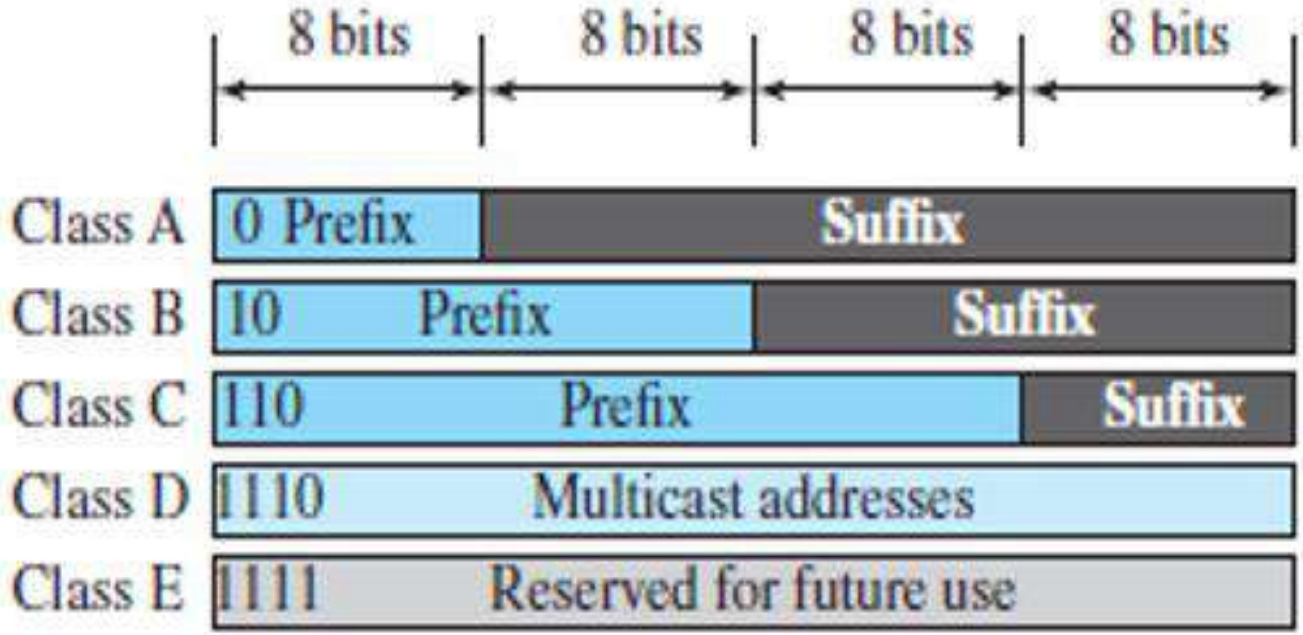




Class	Prefixes	First byte
A	$n = 8$ bits	0 to 127
B	$n = 16$ bits	128 to 191
C	$n = 24$ bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

Class A

- In Class A NetID = 8 bits and HostID = 24.
- How to identify class A address
  - First bit is reserved to 0 in binary notation
  - Range of 1st octet is [0, 127] in dotted decimal notation



Class	Prefixes	First byte
A	$n = 8$ bits	0 to 127
B	$n = 16$ bits	128 to 191
C	$n = 24$ bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

KnowledgeGate
www.knowledgegate.in

- Total number of connections in class A is 2^{31} (2,14,74,83,648)

- There are 2^7 -2 = 126 networks in Class A network.
  - In Class A, total network available are 2 less, because:
  - IP Address 0.0.0.0 is reserved for broadcasting requirements
  - IP Address 127.0.0.1 is reserved for loopback address / self-connectivity.
- The range of 1st octet is [0, 127] but since two addresses are reserved it is: [1, 126].

KnowledgeGate

www.knowledgegate.in

- There are $2^{24} – 2$ (1,67,77,214) HostID in Class A.

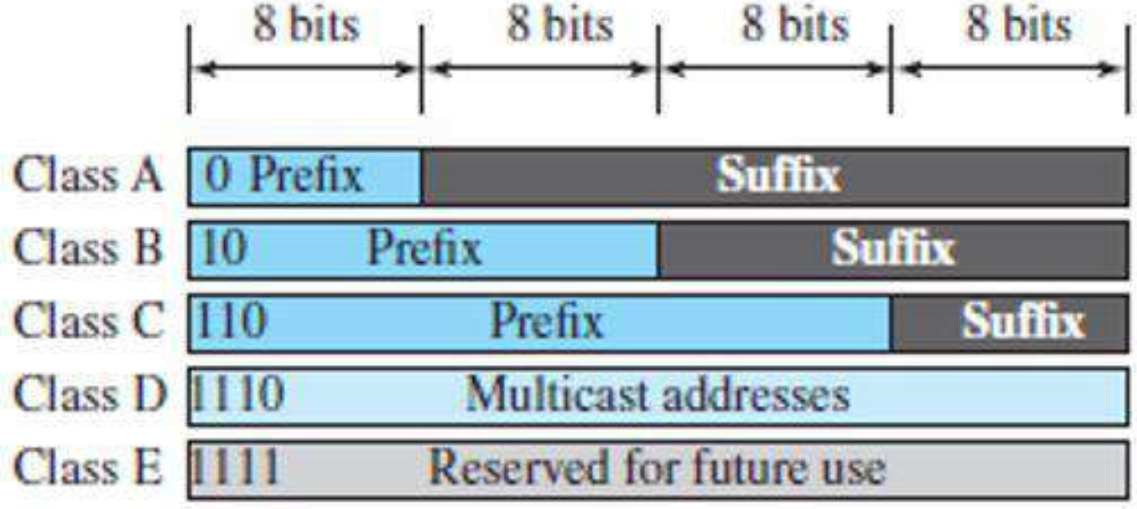
  - In all the classes, total number of hosts that can be configured are 2 less because:
  - This is to account for the two reserved IP addresses in which all the bits for host ID are either zero or one.
  - When all Host ID bits are 0, it represents the Network ID for the network.
  - When all Host ID bits are 1, it represents the Broadcast Address.

- Class A is used by organizations requiring very large size networks like Indian Railways.

KnowledgeGate

Class B

- In Class B NetID = 16 bits and HostID = 16.
- How to identify class B address
  - First two bits are reserved to 10 in binary notation
  - Range of 1st octet is [128, 191] in dotted decimal notation



Class	Prefixes	First byte
A	n = 8 bits	0 to 127
B	n = 16 bits	128 to 191
C	n = 24 bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

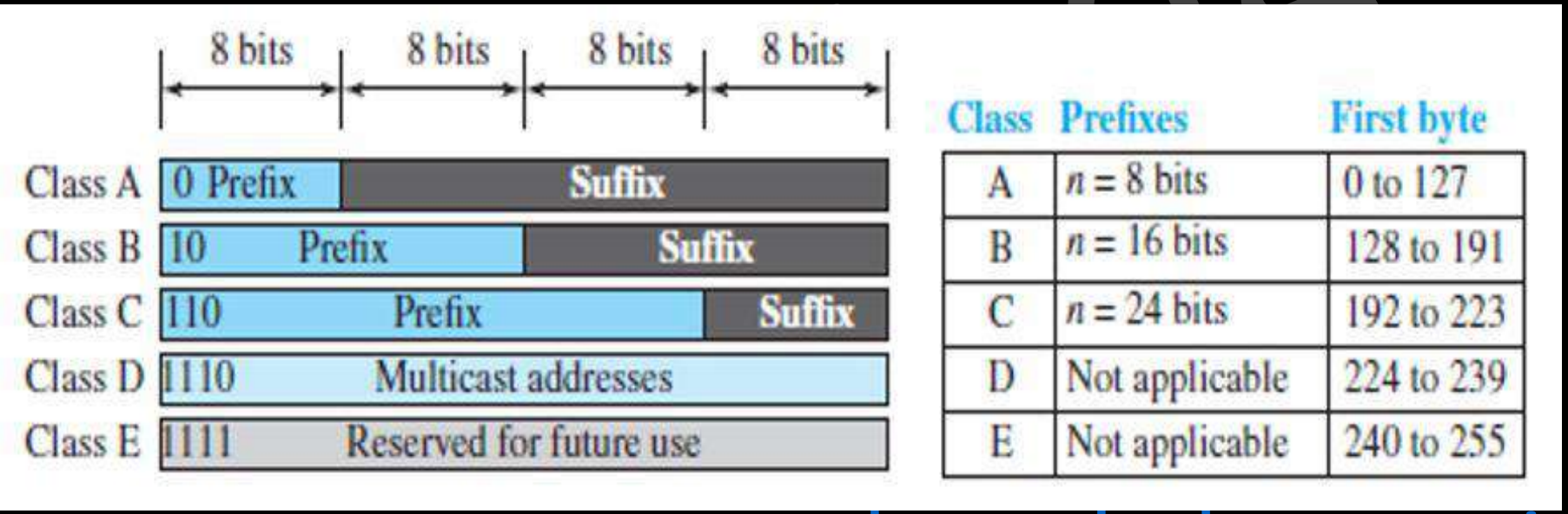
www.knowledgegate.in

- Total number of connections in class B is 2^{30} (1,07,37,41,824)
- Total number of networks available in class B is 2^{14} (16,384)
- Total number of hosts that can be configured in every network in class B is 2^{16} – 2 (65,534)
- Class B is used by organizations requiring medium size networks

KnowledgeGate

Class C

- In Class C NetID = 24 bits and HostID = 8.
- How to identify class C address
  - First three bits are reserved to 110 in binary notation
  - Range of 1st octet is [192, 223] in dotted decimal notation



Class	Prefixes	First byte
A	n = 8 bits	0 to 127
B	n = 16 bits	128 to 191
C	n = 24 bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

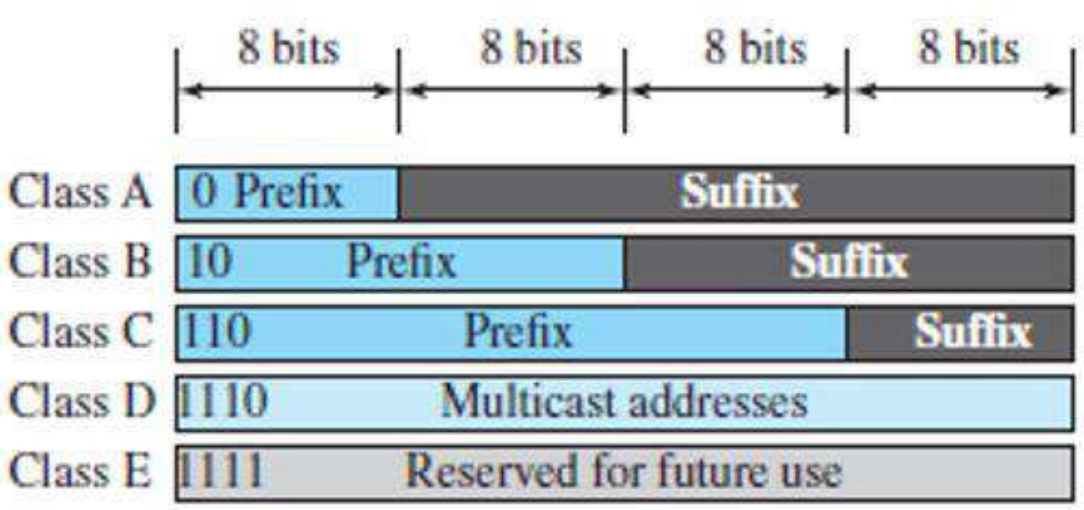
KnowledgeGate

- Total number of connections in class C is 2^{29} (53,68,70,912)
- Total number of networks available in class C is 2^{21} (20,97,152)
- Total number of hosts that can be configured in every network in class C is $2^8 – 2$ (254)
- Class C is used by organizations requiring small to medium size networks.

KnowledgeGate

Class D

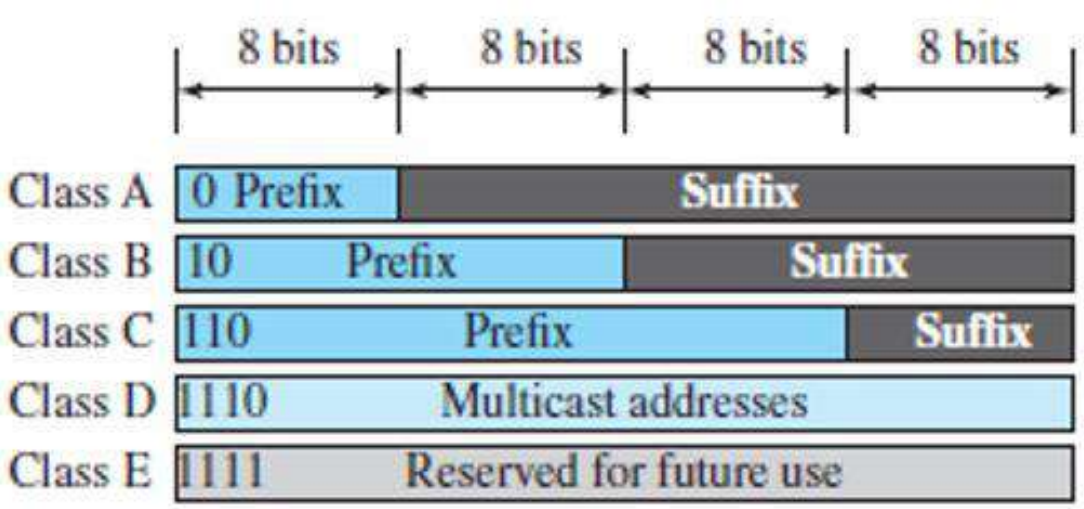
- Class D is not divided into Network ID and Host ID.
- How to identify class D address
  - First four bits are reserved to 1110 in binary notation
  - Range of 1st octet is [224, 239] in dotted decimal notation



Class	Prefixes	First byte
A	n = 8 bits	0 to 127
B	n = 16 bits	128 to 191
C	n = 24 bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

KnowledgeGate

- Total number of IP Addresses available in class D = 2^{28} (26,84,35,456)
- Class D is reserved for multicasting, in multicasting, there is no need to extract host address from the IP Address, this is because data is not destined for a particular host.

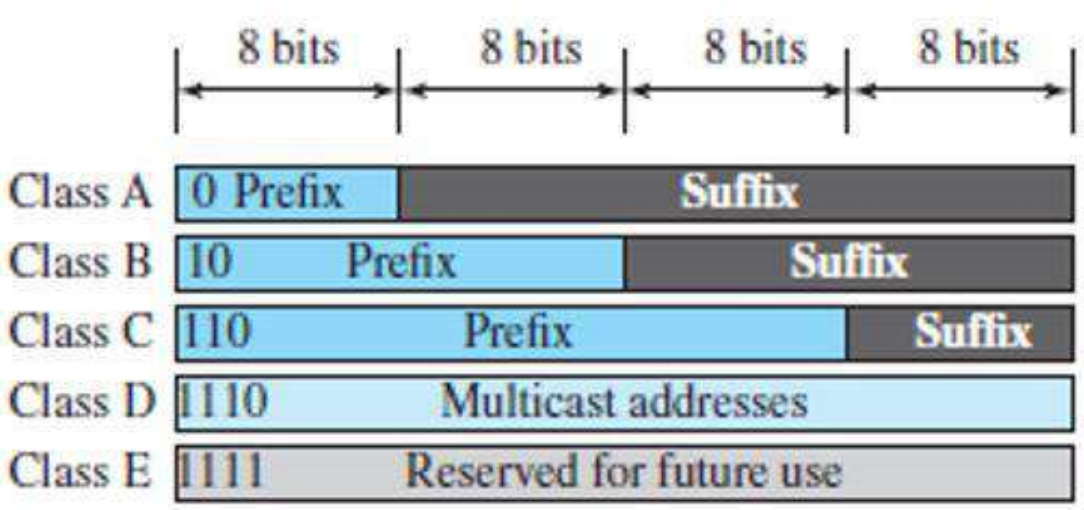


Class	Prefixes	First byte
A	n = 8 bits	0 to 127
B	n = 16 bits	128 to 191
C	n = 24 bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

KnowledgeGate

Class E

- Class E is not divided into Network ID and Host ID.
- How to identify class E address
  - First four bits are reserved to 1111 in binary notation
  - Range of 1st octet is [240, 255] in dotted decimal notation



Class	Prefixes	First byte
A	n = 8 bits	0 to 127
B	n = 16 bits	128 to 191
C	n = 24 bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

KnowledgeGate

- Total number of IP Addresses available in class E = 2^{28} (26,84,35,456)
- Class E is reserved for future or experimental purposes.

KnowledgeGate

Points to note

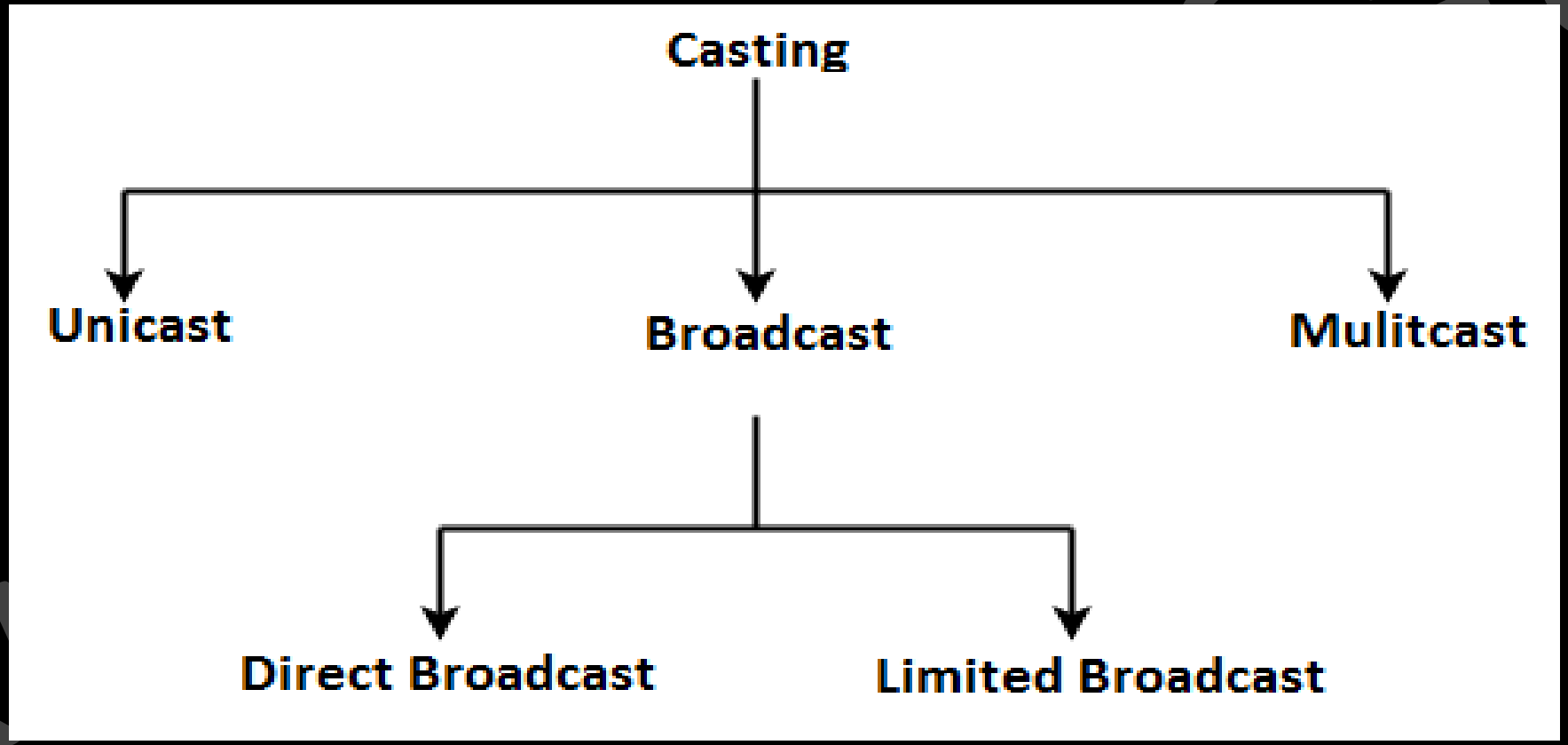
- All the hosts in a single network always have the same network ID but different Host ID.
- Two hosts in two different networks can have the same host ID.
- Only those devices which have the network layer will have IP Address, switches, hubs and repeaters does not have any IP Address.

KnowledgeGate

Casting in Networks

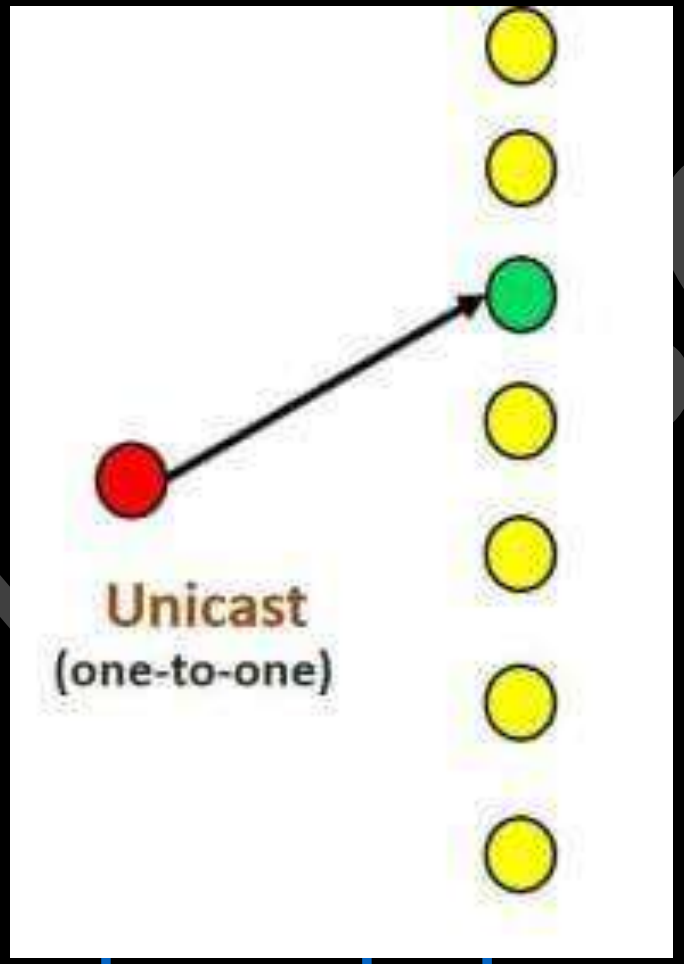
Types of Casting

- Casting in a network is basically of three type: Unicast, Multicast and Broadcast.



www.knowledgegate.in

. **Unicast**: Transmitting data from one source host to one destination host is called as **unicast**. It is a one to one transmission.



KnowledgeGate

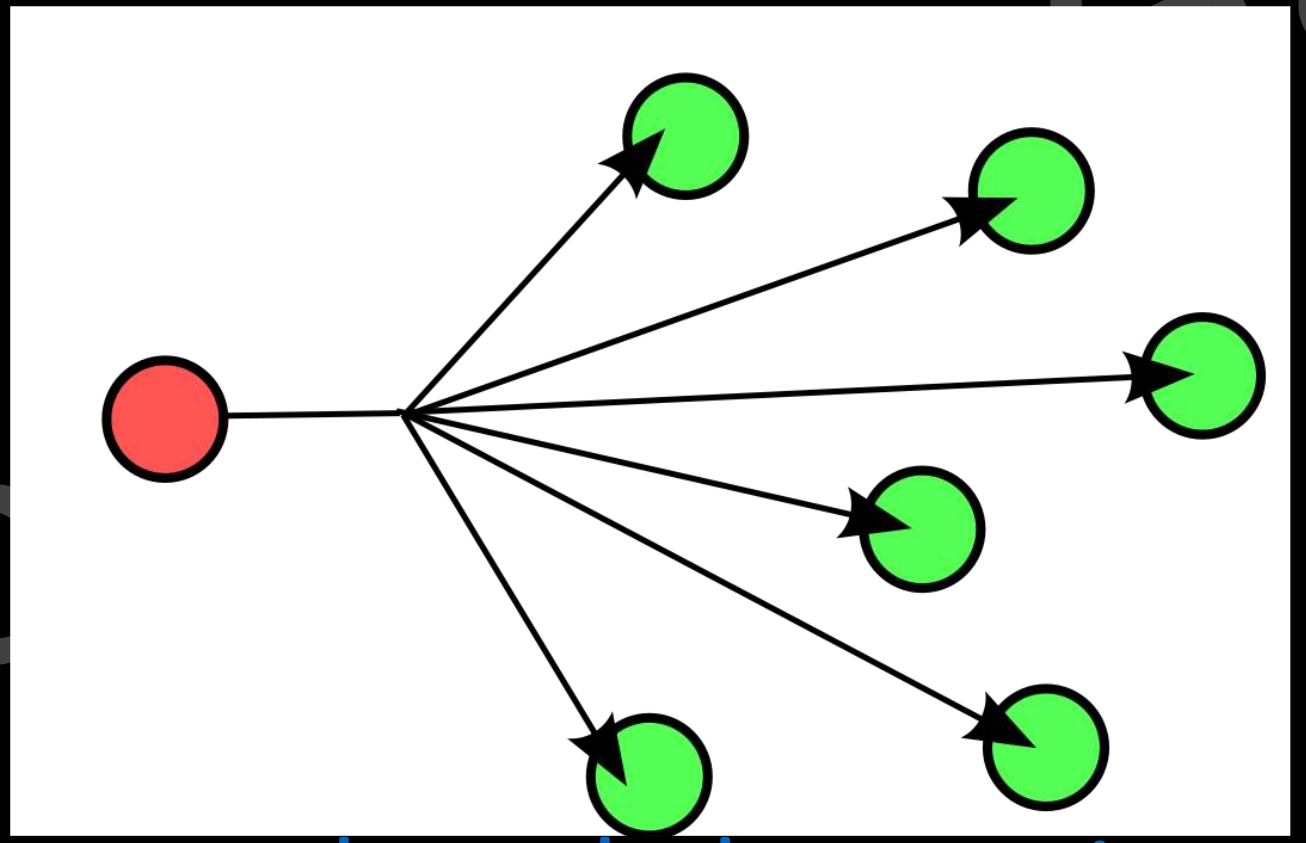
www.knowledgegate.in

- **Broadcast:** Transmitting data from one source host to all other hosts residing in a network either same or other network is called as **broadcast**. It is a one to all transmission.

  - **Limited Broadcast**:
  - **Direct Broadcast**:

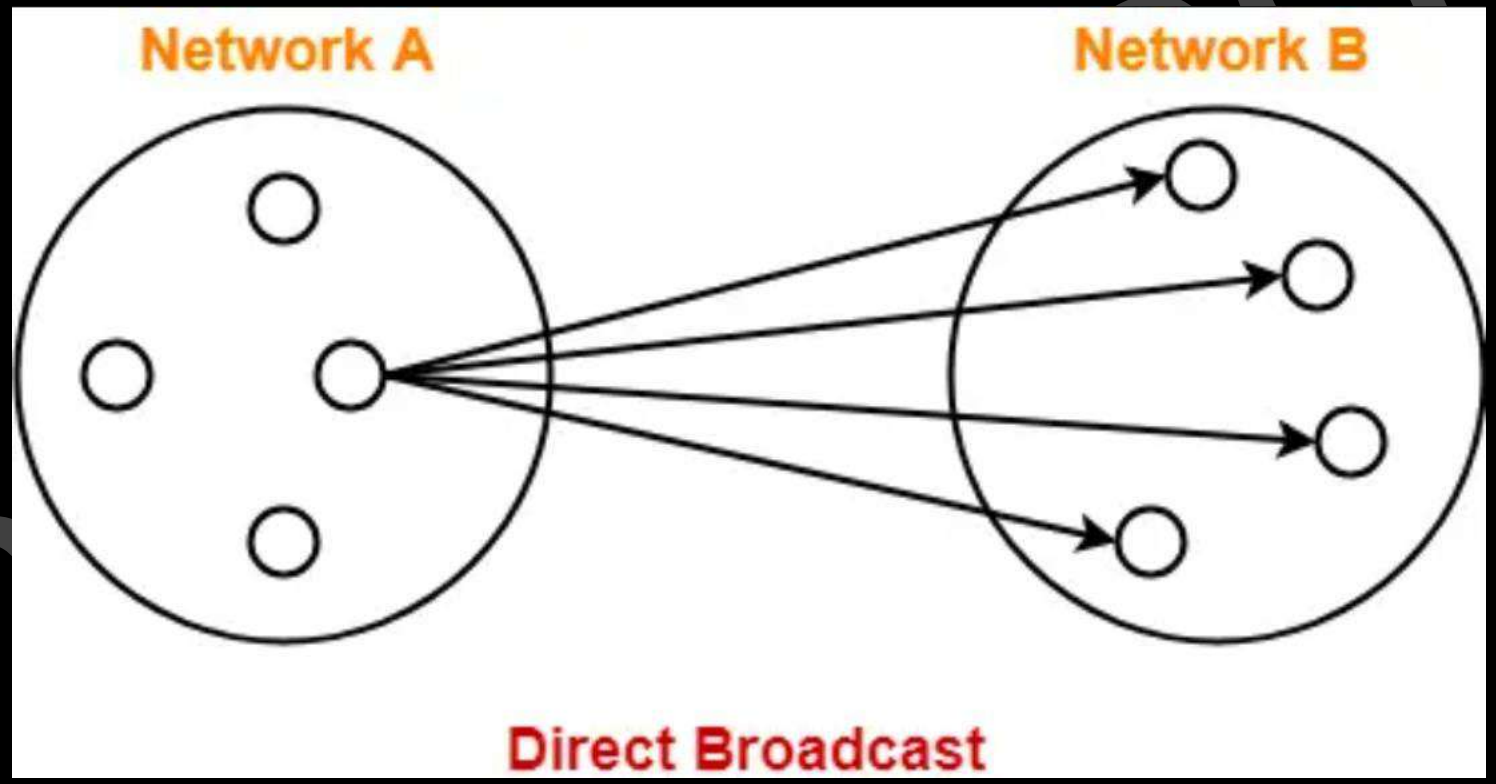
KnowledgeGate

- **Limited Broadcast**: Transmitting data from one source host to all other hosts residing in the same network is called as limited broadcast. Limited Broadcast Address for any network is All 32 bits set to 1 = 11111111.11111111.11111111.11111111 = 255.255.255.255



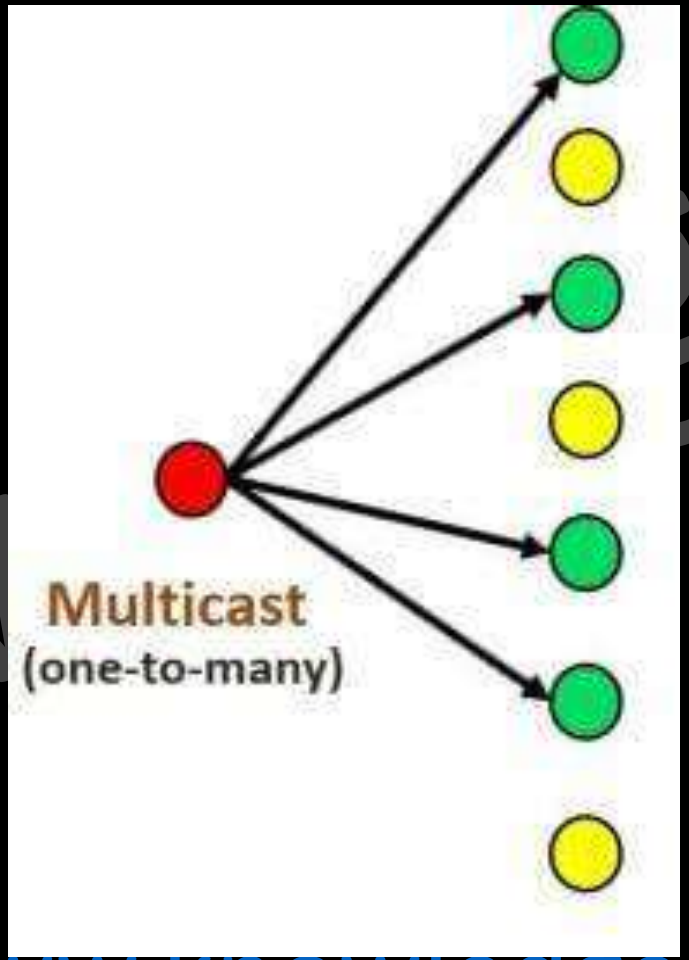
www.knowledgegate.in

- **Direct Broadcast**: Transmitting data from one source host to all other hosts residing in some other network is called as direct broadcast.
  - Direct Broadcast Address for any network is the IP Address where, Network ID is the IP Address of the network where all the destination hosts are present and Host ID bits are all set to 1.



www.knowledgegate.in

- **Multicast**: Transmitting data from one source host to a particular group of hosts having interest in receiving the data is called as multicast. It is a one to many transmissions.



KnowledgeGate

www.knowledgegate.in

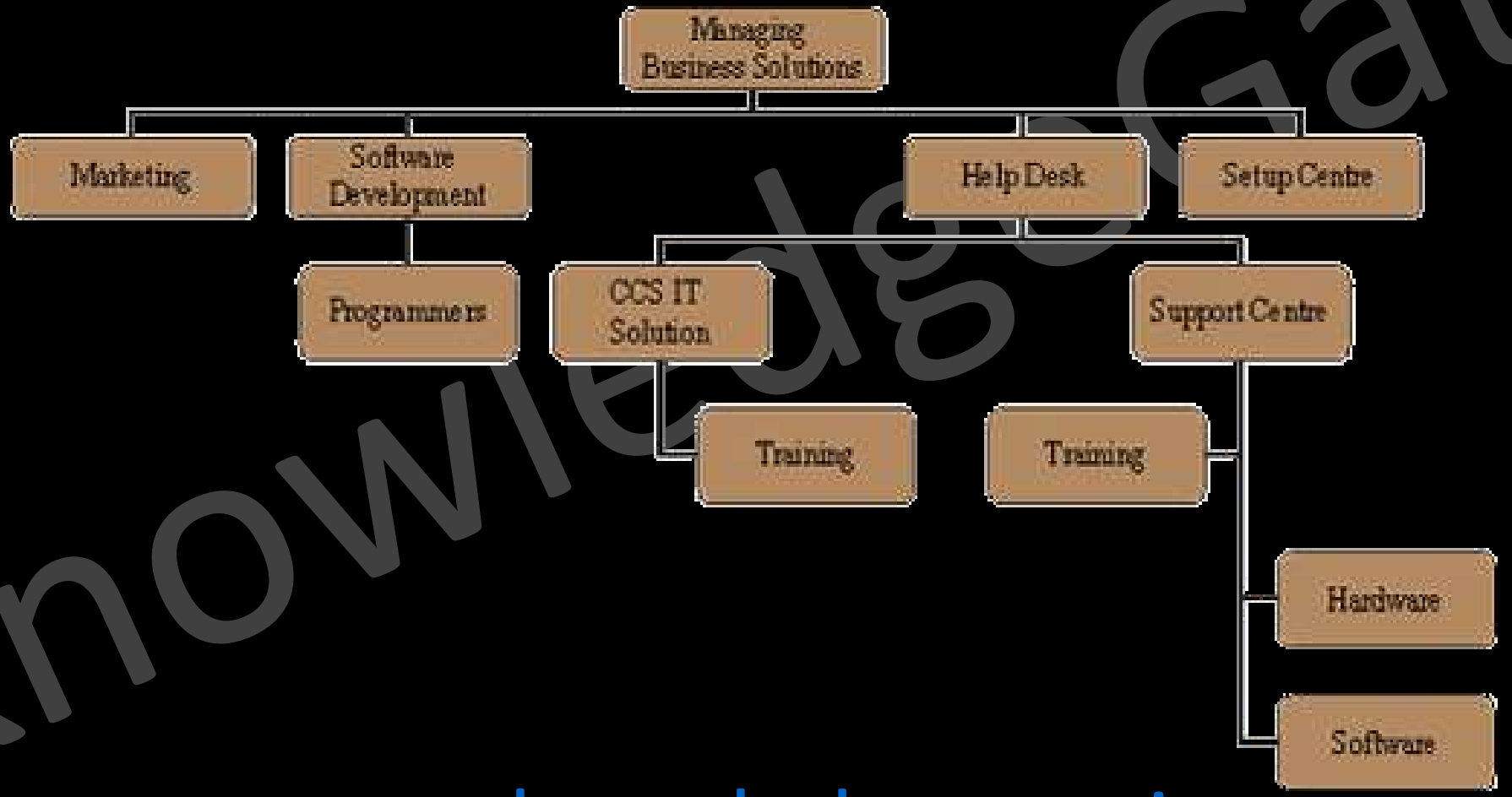
Reason For Subnetting

- Maintenance of a very big network like class A and class B is very difficult for network administrator.

www.knowledgegate.in

Reason For Subnetting

- Having all the computer from different departments in a company on the same networks is less secure from company prospective.



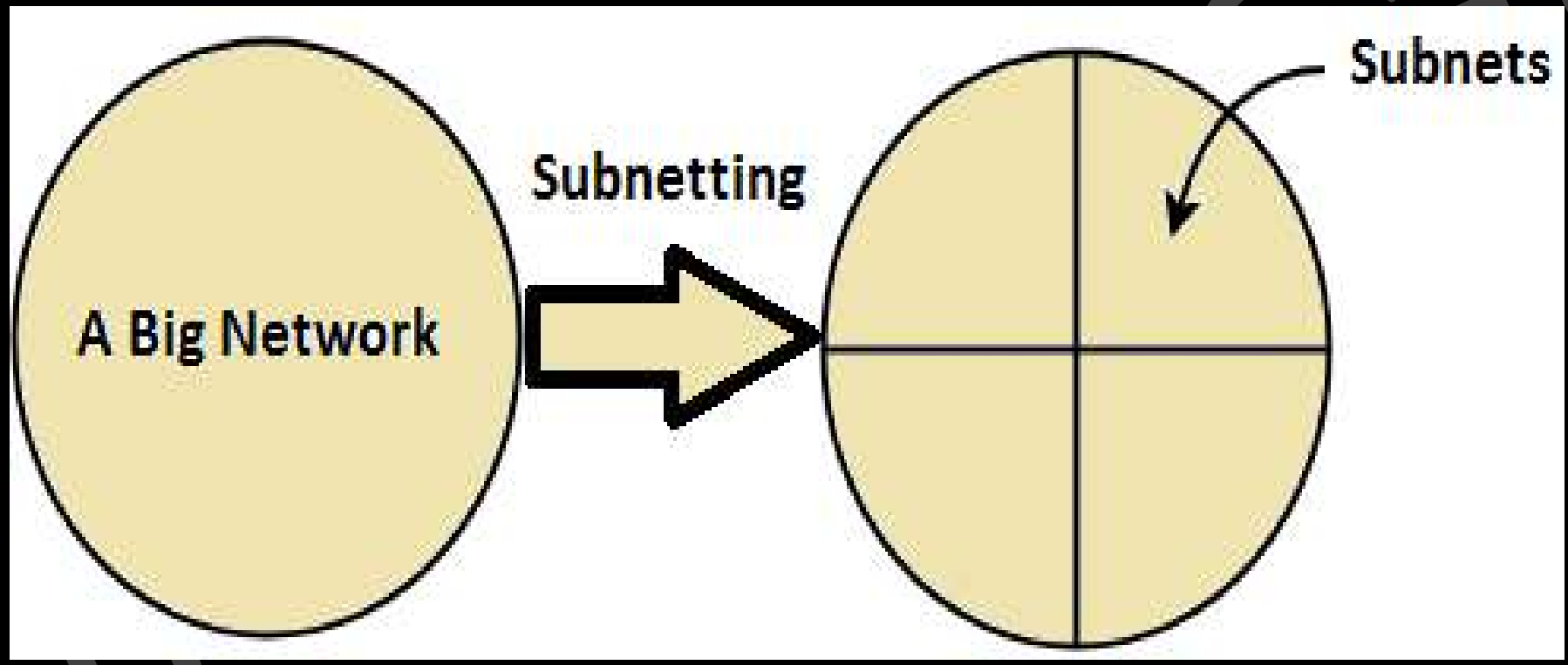
KnowledgeGate

Reason For Subnetting

- So, if an organization was granted a large block in class A or B, it could divide the addresses into several contiguous groups and assign each group to smaller networks (called subnets) or, in rare cases, share part of the addresses with neighbours.

www.knowledgegate.in

- **Conclusion:** An organization (or an ISP) that is granted a range of addresses may divide the range into several subranges and assign each subrange to a subnetwork (or subnet). A subnetwork can be divided into several sub-subnetworks. A sub-subnetwork can be divided into several sub-sub-subnetworks, and so on.



www.knowledgegate.in

Advantages

- It improves the security.
- The maintenance and administration of subnets is easy.

Disadvantages

- Identification of a station is difficult
- Not possible to directed broadcast from outside network.

KnowledgeGate

www.knowledgegate.in

Types of Subnetting

Subnets can be of two types:

1. Fixed Length Subnetting
2. Variable Length Subnetting

KnowledgeGate

Fixed Length Subnetting

- Fixed length subnetting (classful subnetting) divides the network into subnets such that:
  - All the subnets are of same size.
  - All the subnets have equal number of hosts.
  - All the subnets have same subnet mask.

KnowledgeGate

Q Consider the network having IP Address 200.1.2.0. Divide this network into two subnets.

KnowledgeGate

www.knowledgegate.in

Q Consider the network having IP Address 200.1.2.0. Divide this network into two subnets.

1st Subnet

- IP Address of the subnet / Subnet id = 200.1.2.0
- Direct Broadcast Address = 200.1.2.01111111 = 200.1.2.127
- Total number of IP Addresses = 2^7 = 128
- Range of IP Addresses = [200.1.2.0, 200.1.2.127]
- Total number of hosts that can be configured = 128 – 2 = 126
- Range of Allocated IP Addresses = [200.1.2.1, 200.1.2.126]

2nd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.128
- Direct Broadcast Address = 200.1.2.11111111 = 200.1.2.255
- Total number of IP Addresses = 2^7 = 128
- Range of IP Addresses = [200.1.2.128, 200.1.2.255]
- Total number of hosts that can be configured = 128 – 2 = 126
- Range of Allocated IP Addresses = [200.1.2.129, 200.1.2.254]

KnowledgeGate

Q Consider we have a big single network having IP Address 200.1.2.0. We want to do subnetting and divide this network into 4 subnets.

KnowledgeGate

www.knowledgegate.in

Q Consider we have a big single network having IP Address 200.1.2.0. We want to do subnetting and divide this network into 4 subnets.

1st Subnet

- IP Address of the subnet / Subnet id = 200.1.2.0
- Direct Broadcast Address = 200.1.2.**00**111111 = 200.1.2.63
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.0, 200.1.2.63]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.1, 200.1.2.62]

2nd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.64
- Direct Broadcast Address = 200.1.2.**01**111111 = 200.1.2.127
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.64, 200.1.2.127]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.65, 200.1.2.126]

3rd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.128
- Direct Broadcast Address = 200.1.2.**10**111111 = 200.1.2.191
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.128, 200.1.2.191]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.129, 200.1.2.190]

4th Subnet

- IP Address of the subnet / Subnet id = 200.1.2.192
- Direct Broadcast Address = 200.1.2.**11**111111 = 200.1.2.255
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.192, 200.1.2.255]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.193, 200.1.2.254]

KnowledgeGate

www.knowledgegate.in

Variable Length Subnetting

- Variable length subnetting (classless subnetting) divides the network into subnets such that:
  - All the subnets are not of same size.
  - All the subnets do not have equal number of hosts.
  - All the subnets do not have same subnet mask.

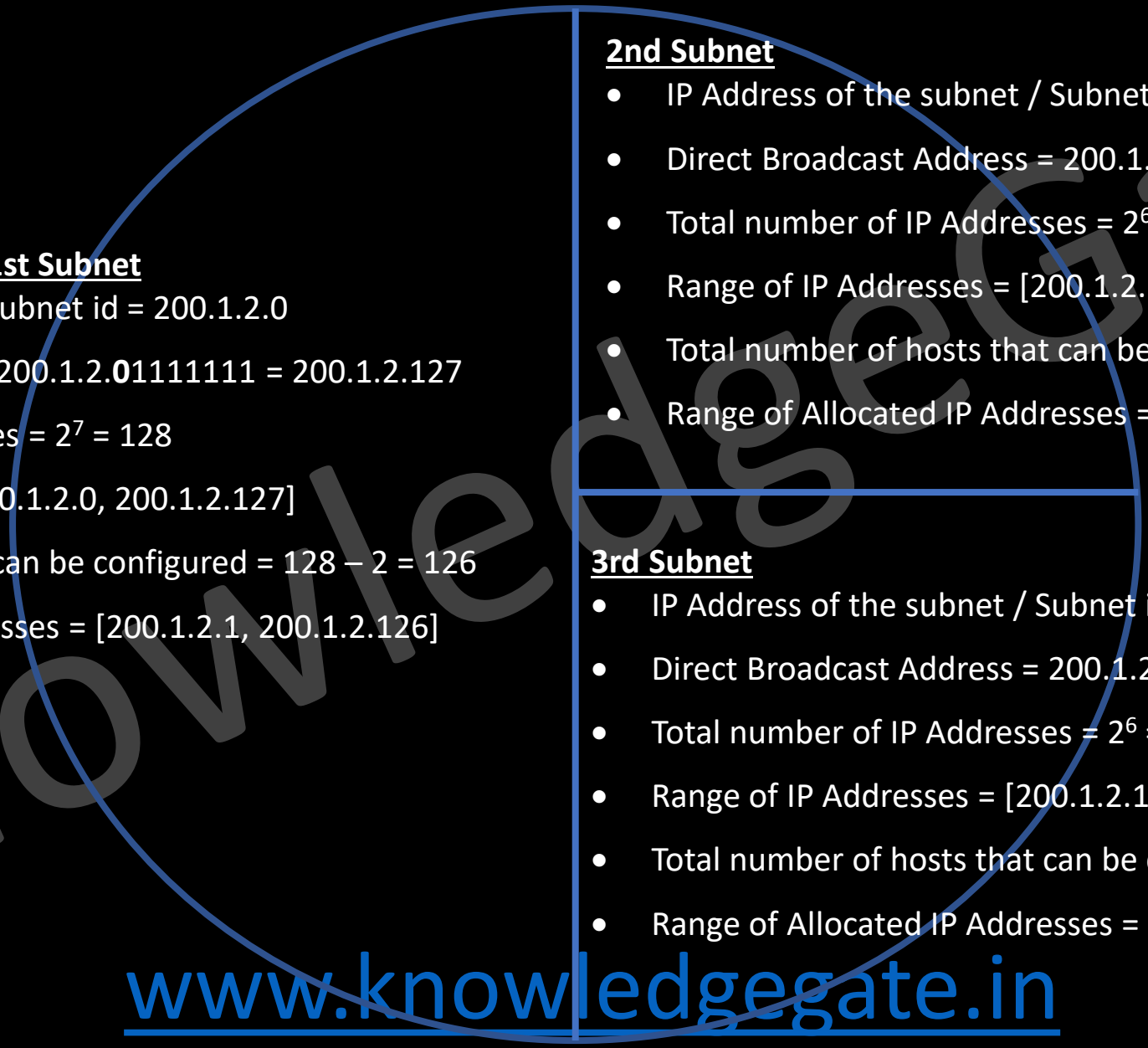
KnowledgeGate

Q Consider we have a big single network having IP Address 200.1.2.0. We want to do subnetting and divide this network into 3 subnets, such that first contains 126 hosts, and other two contains 62 hosts each?

KnowledgeGate

www.knowledgegate.in

Q Consider we have a big single network having IP Address 200.1.2.0. We want to do subnetting and divide this network into 3 subnets, such that first contains 126 hosts, and other two contains 62 hosts each?



1st Subnet

- IP Address of the subnet / Subnet id = 200.1.2.0
- Direct Broadcast Address = 200.1.2.**0**1111111 = 200.1.2.127
- Total number of IP Addresses = 2^7 = 128
- Range of IP Addresses = [200.1.2.0, 200.1.2.127]
- Total number of hosts that can be configured = 128 – 2 = 126
- Range of Allocated IP Addresses = [200.1.2.1, 200.1.2.126]

2nd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.128
- Direct Broadcast Address = 200.1.2.**10**1111111 = 200.1.2.191
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.128, 200.1.2.191]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.129, 200.1.2.190]

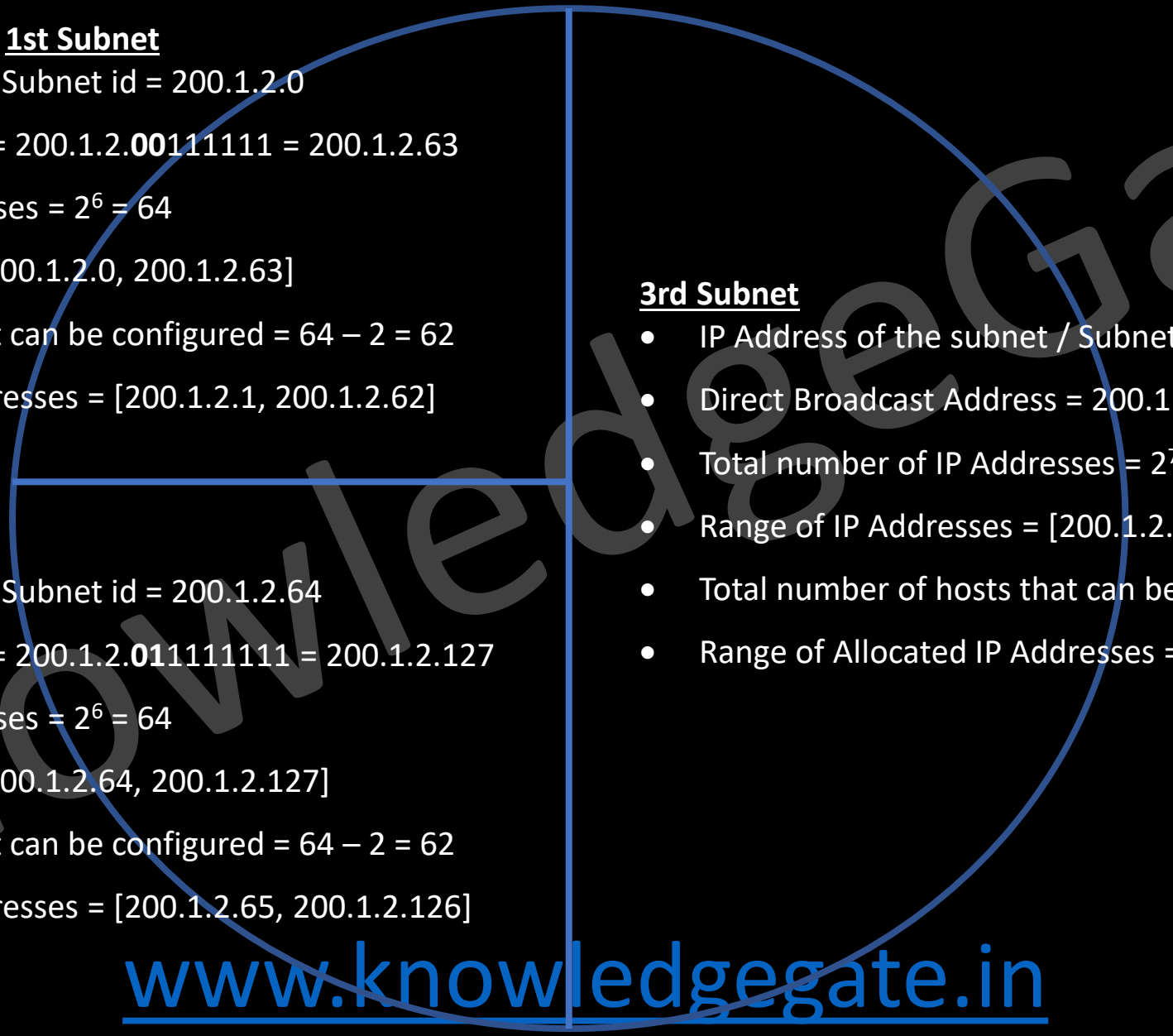
3rd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.192
- Direct Broadcast Address = 200.1.2.**11**1111111 = 200.1.2.255
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.192, 200.1.2.255]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.193, 200.1.2.254]

KnowledgeGate

www.knowledgegate.in

Q Consider we have a big single network having IP Address 200.1.2.0. We want to do subnetting and divide this network into 3 subnets, such that first contains 126 hosts, and other two contains 62 hosts each?



1st Subnet

- IP Address of the subnet / Subnet id = 200.1.2.0
- Direct Broadcast Address = 200.1.2.**00**111111 = 200.1.2.63
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.0, 200.1.2.63]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.1, 200.1.2.62]

2nd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.64
- Direct Broadcast Address = 200.1.2.**01**111111 = 200.1.2.127
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.64, 200.1.2.127]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.65, 200.1.2.126]

3rd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.128
- Direct Broadcast Address = 200.1.2.**1**11111111 = 200.1.2.255
- Total number of IP Addresses = 2^7 = 128
- Range of IP Addresses = [200.1.2.128, 200.1.2.255]
- Total number of hosts that can be configured = 128 – 2 = 126
- Range of Allocated IP Addresses = [200.1.2.129, 200.1.2.254]

KnowledgeGate

www.knowledgegate.in

Point to Note:

- Subnetting increases the number of 1’s in the mask

KnowledgeGate

www.knowledgegate.in

Subnet Masks

- In case of subnetting the problem is how to identify to which subnet the incoming packet from outside the network must be delivered. To solve this problem, we use the idea of subnet mask.

- Subnet mask is a 32-bit number which is a sequence of 1's followed by a sequence of 0's where:
  - 1's represents the Network ID part along with the subnet ID.
  - 0's represents the host ID part.

- Default mask for different classes of IP Address are:
  - Default subnet mask of Class A = 255.0.0.0
  - Default subnet mask for Class B = 255.255.0.0
  - Default subnet mask for Class C = 255.255.255.0

- Networks of same size always have the same subnet mask.

KnowledgeGate

www.knowledgegate.in

Address Depletion

- The addresses were not distributed properly as class A and B are usually very large for any organization and class C is usually very small
- Flexibility is not there is classful addressing, we cannot have the exact allocation as we want for e.g. if some company wants 150 IP address then must go for 256, resulting into address depletion.
- Wastage of addresses, for example: Class E addresses were almost never used, wasting the whole class.
- **Conclusion:** The Internet was faced with the problem of the addresses being rapidly used up, resulting in no more addresses available for organizations and individuals that needed to be connected to the Internet.

KnowledgeGate

Classless Addressing (Blocks/Network)

- Classless Addressing is an improved IP Addressing system.
- The class privilege is removed from the distribution to compensate for the address depletion, so no class.
- Here we can ask exact set of IP address which are required and a Variable-length blocks are assigned which satisfy the request.

KnowledgeGate

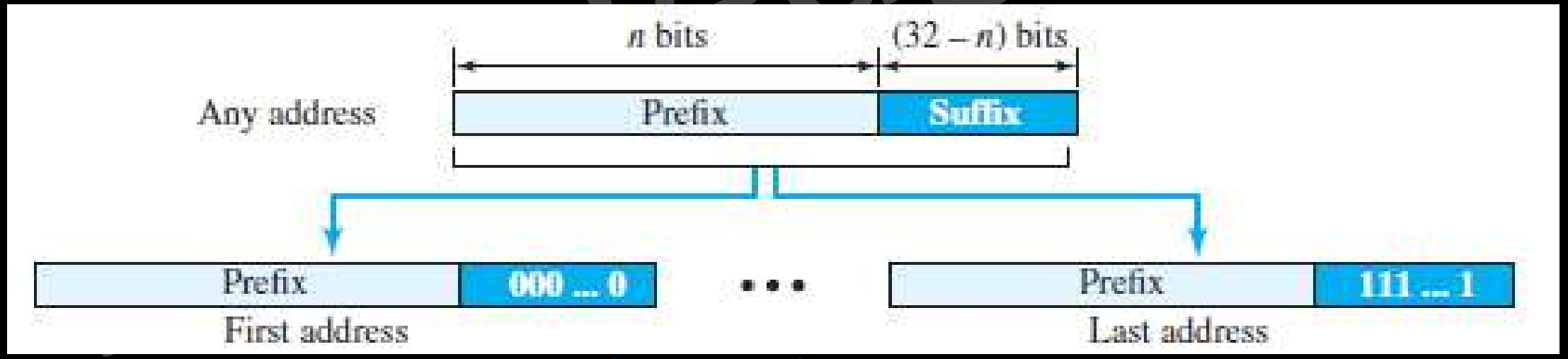
CIDR Notation

- The question is as there are no classes, how to identify block id and host id, as address in classless addressing does not define the block or network to which the address belongs.
- To solve this problem now we have a new CIDR notation, this notation is informally referred to as *slash notation* and formally as ***classless interdomain routing*** or ***CIDR.***
- To find the prefix(net_id), *n* is added to the address, separated by a slash.
- n represent number of bits in net_id

www.knowledgegate.in

Extracting Information from an Address

- The number of addresses in the block is found as $N = 2^{32-n}$.
- To find the first address, we keep the n leftmost bits and set the $(32 - n)$ rightmost bits all to 0s.
- To find the last address, we keep the n leftmost bits and set the $(32 - n)$ rightmost bits all to 1s.



KnowledgeGate

Q Find the Number of addresses, first and last address of the CIDR block to which Following Address belongs to
167.199.170.82/**27** (10100111 11000111 10101010 01010010)

KnowledgeGate

www.knowledgegate.in

167.199.170.82/**27** (10100111 11000111 10101010 01010010)

10100111 11000111 10101010 010_ _ _ _ _

No of Address = 2^5

KnowledgeGate

167.199.170.82/**27** (10100111 11000111 10101010 01010010)

64

10100111 11000111 10101010 0100 0 0 0 0 0

10100111 11000111 10101010 0101 1 1 1 1 1

95

KnowledgeGate

Address Mask

- The address mask is a 32-bit number in which the *n* leftmost bits are set to 1s and the rest of the bits (32 – *n*) are set to 0s.
- It is another way to find the first and last addresses in the block.
- Using the three bit-wise operations NOT, AND, and OR a computer can find:
  1. The number of addresses in the block N = **NOT** (mask) + 1.
  2. The first address in the block = (Any address in the block) **AND** (mask).
  3. The last address in the block = (Any address in the block) **OR** [(**NOT** (mask)].

KnowledgeGate

Rules for Creating CIDR Block (Network)

- All the IP Addresses in the CIDR block must be contiguous.
- The size of the block (total number of IP Addresses contained in the block) must be presentable as power of 2, size of any CIDR block will always be in the form 2^1, 2^2, 2^3, 2^4, 2^5 and so on. (calculation can be easy)
- First IP Address of the block must be divisible by the size of the block. (so that we get the host id from all 0 to all 1)

KnowledgeGate

Subnetting in CIDR

Q Consider the network having IP Address 40.30.20.10/25 Divide this network into two subnets.

KnowledgeGate

Q Consider the network having IP Address 40.30.20.10/25 Divide this network into two subnets.

40.30.20.0**0001010**

1st Subnet

- 40.30.20.00001010
- Total number of IP Addresses = 2^6 = 64
- First Address of the Subnet = 40.30.20.00000000
- Last Address of the Subnet = 40.30.20.00111111
- Range of IP Addresses = [40.30.20.0, 40.30.20.63]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [40.30.20.1, 40.30.20.62]
- CIRD Representation 40.30.20. __/26

0,1,2,3,4,5,----------,63

2st Subnet

- 40.30.20.01001010
- Total number of IP Addresses = 2^6 = 64
- First Address of the Subnet = 40.30.20.01000000
- Last Address of the Subnet = 40.30.20.01111111
- Range of IP Addresses = [40.30.20.64, 40.30.20.127]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [40.30.20.65, 40.30.20.126]
- CIRD Representation 40.30.20. __/26

64,65,66,67,----------,127

KnowledgeGate

Q Consider we have a big single network having IP Address 200.1.2.0/24 We want to do subnetting and divide this network into 4 subnets.

KnowledgeGate

www.knowledgegate.in

Q Consider we have a big single network having IP Address 200.1.2.0/24. We want to do subnetting and divide this network into 4 subnets.

1st Subnet

- IP Address of the subnet / Subnet id = 200.1.2.0
- Direct Broadcast Address = 200.1.2.**00**111111 = 200.1.2.63
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.0, 200.1.2.63]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.1, 200.1.2.62]

2nd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.64
- Direct Broadcast Address = 200.1.2.**01**111111 = 200.1.2.127
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.64, 200.1.2.127]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.65, 200.1.2.126]

3rd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.128
- Direct Broadcast Address = 200.1.2.**10**111111 = 200.1.2.191
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.128, 200.1.2.191]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.129, 200.1.2.190]

4th Subnet

- IP Address of the subnet / Subnet id = 200.1.2.192
- Direct Broadcast Address = 200.1.2.**11**111111 = 200.1.2.255
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.192, 200.1.2.255]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.193, 200.1.2.254]

KnowledgeGate

www.knowledgegate.in

Q Consider we have a big single network having IP Address 200.1.2.0/24. We want to do subnetting and divide this network into 3 subnets, such that first contains 126 hosts, and other two contains 62 hosts each?

KnowledgeGate

www.knowledgegate.in

Q Consider we have a big single network having IP Address 200.1.2.0/24. We want to do subnetting and divide this network into 3 subnets, such that first contains 126 hosts, and other two contains 62 hosts each?

1st Subnet

- IP Address of the subnet / Subnet id = 200.1.2.0
- Direct Broadcast Address = 200.1.2.**0**1111111 = 200.1.2.127
- Total number of IP Addresses = 2^7 = 128
- Range of IP Addresses = [200.1.2.0, 200.1.2.127]
- Total number of hosts that can be configured = 128 – 2 = 126
- Range of Allocated IP Addresses = [200.1.2.1, 200.1.2.126]

2nd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.128
- Direct Broadcast Address = 200.1.2.**10**1111111 = 200.1.2.191
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.128, 200.1.2.191]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.129, 200.1.2.190]

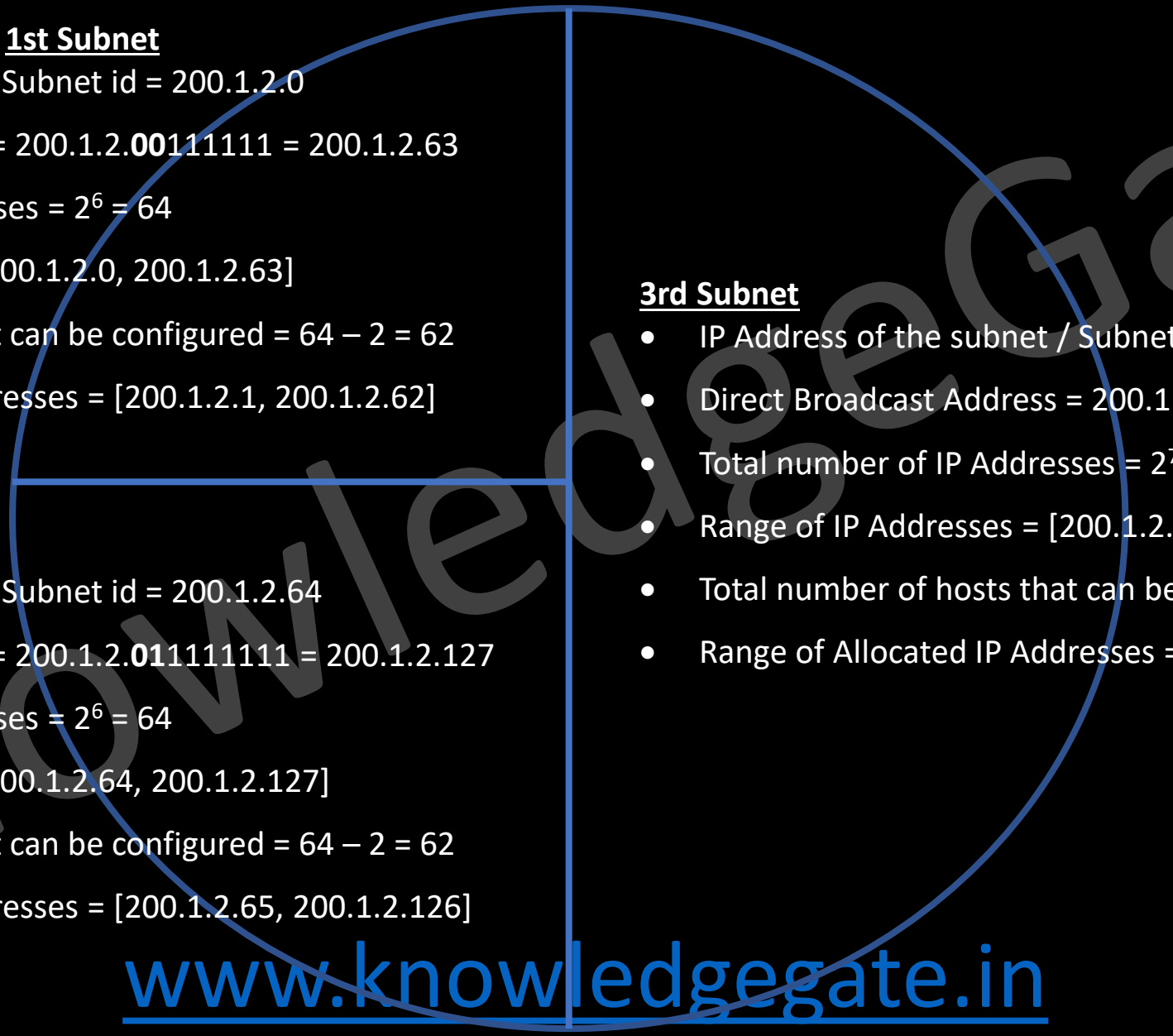
3rd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.192
- Direct Broadcast Address = 200.1.2.**11**1111111 = 200.1.2.255
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.192, 200.1.2.255]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.193, 200.1.2.254]

KnowledgeGate

www.knowledgegate.in

Q Consider we have a big single network having IP Address 200.1.2.0/24. We want to do subnetting and divide this network into 3 subnets, such that first contains 126 hosts, and other two contains 62 hosts each?



1st Subnet

- IP Address of the subnet / Subnet id = 200.1.2.0
- Direct Broadcast Address = 200.1.2.**00**111111 = 200.1.2.63
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.0, 200.1.2.63]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.1, 200.1.2.62]

2nd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.64
- Direct Broadcast Address = 200.1.2.**01**111111 = 200.1.2.127
- Total number of IP Addresses = 2^6 = 64
- Range of IP Addresses = [200.1.2.64, 200.1.2.127]
- Total number of hosts that can be configured = 64 – 2 = 62
- Range of Allocated IP Addresses = [200.1.2.65, 200.1.2.126]

3rd Subnet

- IP Address of the subnet / Subnet id = 200.1.2.128
- Direct Broadcast Address = 200.1.2.**1**11111111 = 200.1.2.255
- Total number of IP Addresses = 2^7 = 128
- Range of IP Addresses = [200.1.2.128, 200.1.2.255]
- Total number of hosts that can be configured = 128 – 2 = 126
- Range of Allocated IP Addresses = [200.1.2.129, 200.1.2.254]

KnowledgeGate

www.knowledgegate.in

Designing subnets for CIDR Notations

- Assume:
  - The total number of addresses granted to the organization is **N**
  - The prefix length is ***n***
  - The assigned number of addresses to each sub-network is $\mathbf{N_{sub}}$
  - The prefix length for each sub-network is $\boldsymbol{n_{sub}}$.
- Then, The number of addresses in each sub-network should be a power of 2.

- The prefix length for each sub-network should be found using the following formula: $\mathbf{n_{sub} = 32 - \log_2 N_{sub}}$

- The starting address in each sub-network should be divisible by the number of addresses in that sub-network. This can be achieved if we first assign addresses to larger sub-networks.

KnowledgeGate

Super Netting in Classful addressing

- In super netting, an organization can combine several blocks to create a larger range of addresses. In other words, several networks are combined to create a super network or a supernet.

- An organization can apply for a set of class C blocks instead of just one. For example, an organization that needs 1000 addresses can be granted four contiguous class C blocks. The organization can then use these addresses to create one super network.

- Super netting decreases the number of 1’s in the mask.

KnowledgeGate

Super Netting / Aggregation with CIDR

- Rules for Super netting in CIDR
  - All network should be contiguous
  - first net id should be divisible by size of the block

KnowledgeGate

Q Perform CIDR aggregation on the following IP Addresses-

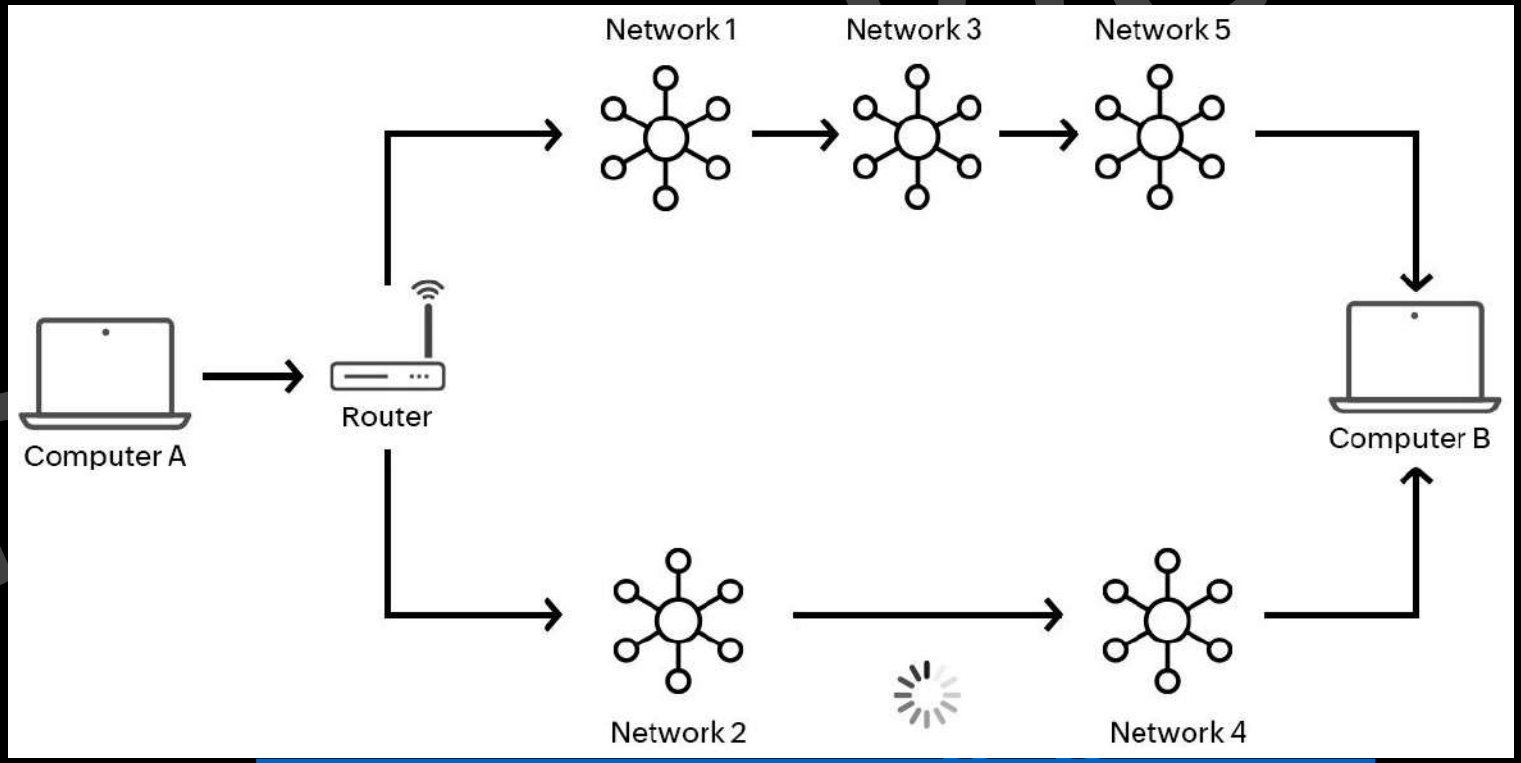
128.56.24.0/24
128.56.25.0/24
128.56.26.0/24
128.56.27.0/24

- Rules for Super netting in CIDR
  - All network should be contiguous
  - first net id should be divisible by size of the block

KnowledgeGate

Routing

- When a router receives an IP packet with destination address then how can it decide to which interface the packet must be send. This decision at the router, is taken with the help of a routing table.
- Actually the process of designing a routing table is called routing. Taking a packet and sending it to some path is actually switching.



KnowledgeGate

Flooding

- One question, is it possible that a packet reaches its destination without routing table, actually yes, the process is called flooding. That is instead of trying to identify the shortest path, we can send it to all possible way and then we can be sure that at least one packet will reach the destination.
- Flooding Advantage
  - No Routing Algorithm is required
  - Shortest Path is guaranteed
  - Highly Reliable
- Flooding Disadvantage
  - Duplicate packets will arrive at destination and intermediate router
  - Traffic is high.

KnowledgeGate

ROUTING

- A routing table contains information about the network, and it helps deciding to which interface the incoming packet should be sent inorder to reach destination. Routing table can be either static or dynamic.
- A static table is one with manual entries, i.e. if someone has information about all the routers in the network and can compute the shortest distance from one router to another and can upload the routing information in a table for each router then it is called Static Routing.
  - Now-a days as we know the internet is so complex that no one can have complete information about the entire internet
  - Internet keeps on changing some new routers come and some old routers may go down, means topology and traffic keeps on changing. Conclusion Static routing is not possible.
- A dynamic table, on the other hand, is one that is updated automatically, without human intervention, when there is a change somewhere in the internet either in topology or traffic.

KnowledgeGate

UNICAST ROUTING PROTOCOLS

- Routing protocols have been created in response to the demand for dynamic routing tables. A routing protocol is a combination of rules and procedures that lets routers in the internet inform each other of changes.

- It allows routers to share whatever they know about the internet or their neighbourhood. The sharing of information allows a router in Delhi to know about the failure of a network in Singapore. The routing protocols also include procedures for combining information received from other routers. Router to have several routing tables based on the required type of service.

KnowledgeGate

www.knowledgegate.in

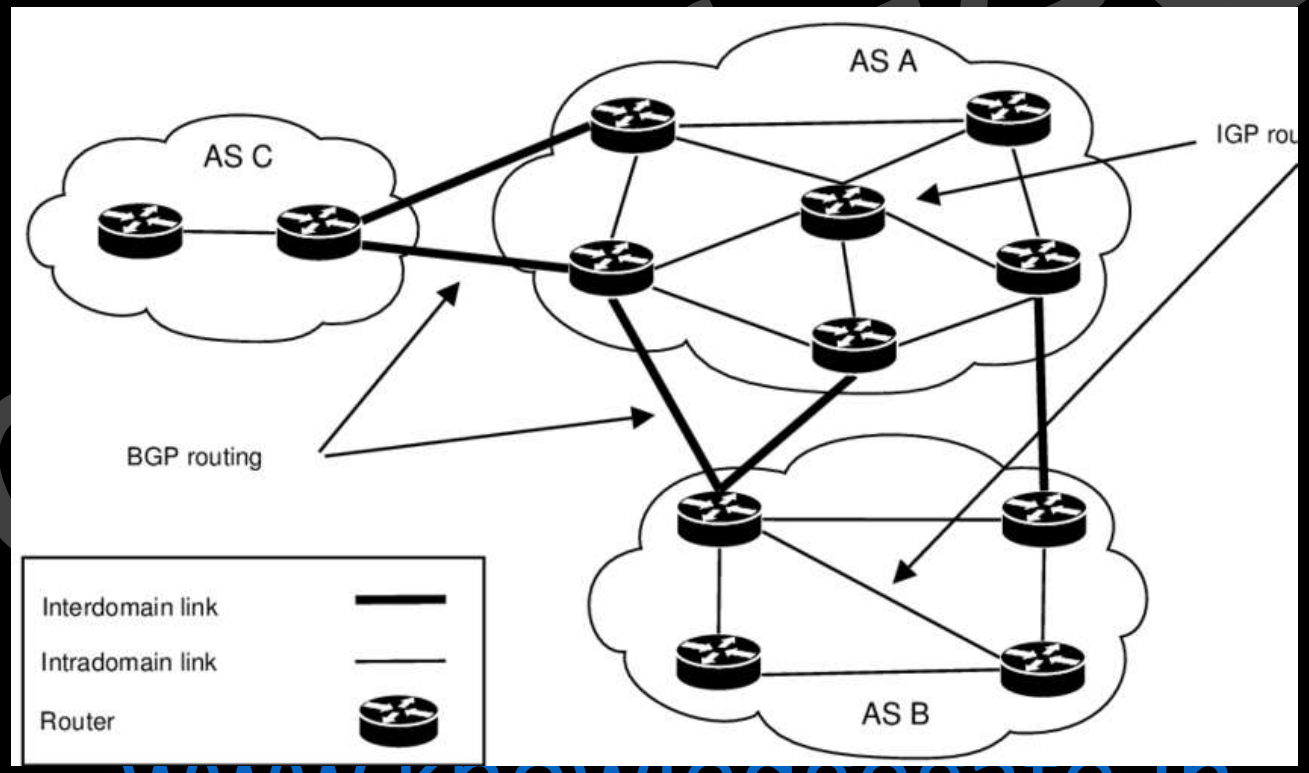
IPv4 Route Table

Active Routes:

Network Destination	Netmask	Gateway	Interface	Metric
0.0.0.0	0.0.0.0	10.0.0.1	10.0.0.75	35
10.0.0.0	255.255.255.0	On-link	10.0.0.75	291
10.0.0.75	255.255.255.255	On-link	10.0.0.75	291
10.0.0.255	255.255.255.255	On-link	10.0.0.75	291
127.0.0.0	255.0.0.0	On-link	127.0.0.1	331
127.0.0.1	255.255.255.255	On-link	127.0.0.1	331
127.255.255.255	255.255.255.255	On-link	127.0.0.1	331
192.168.56.0	255.255.255.0	On-link	192.168.56.1	281
192.168.56.1	255.255.255.255	On-link	192.168.56.1	281
192.168.56.255	255.255.255.255	On-link	192.168.56.1	281
224.0.0.0	240.0.0.0	On-link	127.0.0.1	331
224.0.0.0	240.0.0.0	On-link	192.168.56.1	281
224.0.0.0	240.0.0.0	On-link	10.0.0.75	291
255.255.255.255	255.255.255.255	On-link	127.0.0.1	331

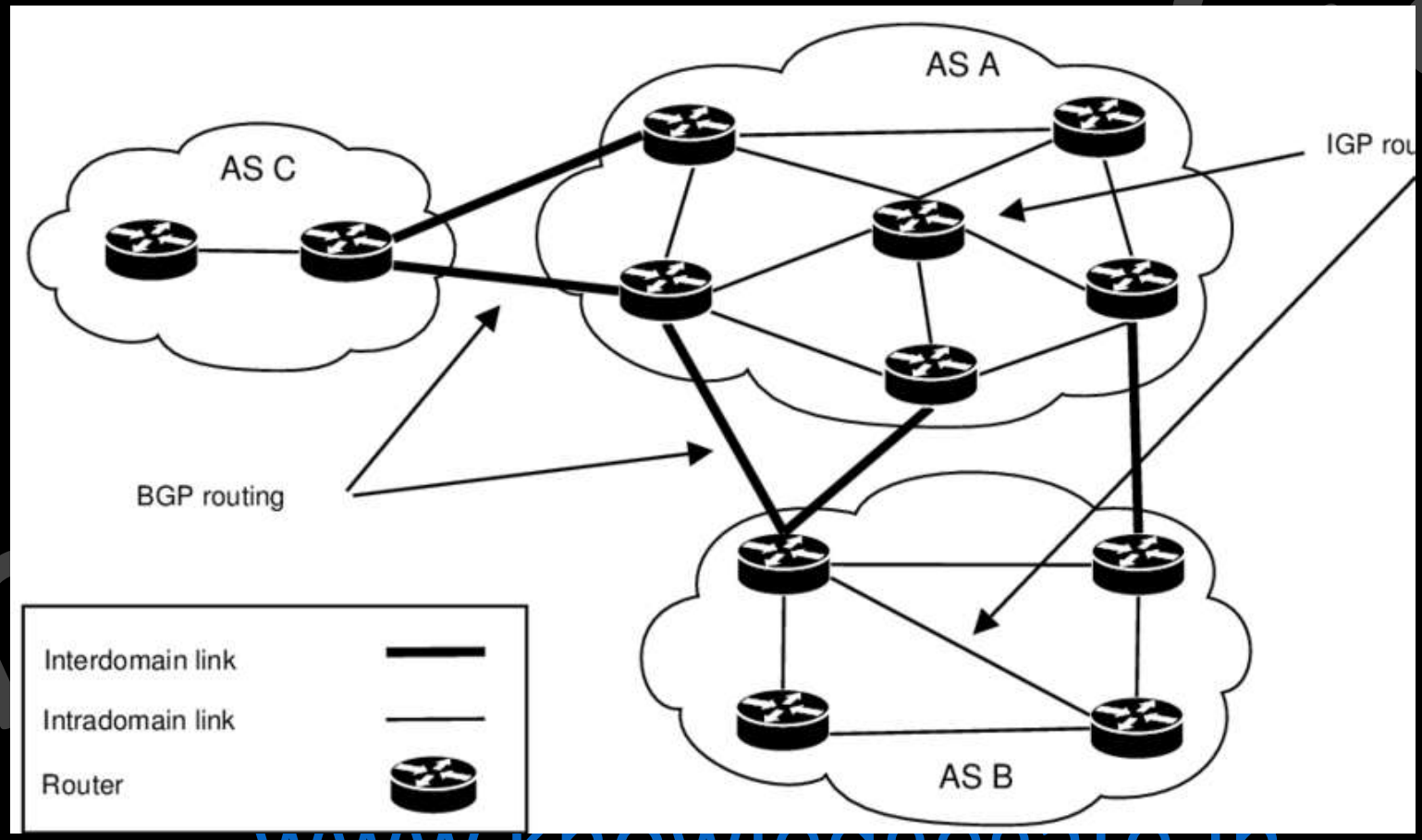
Intra-domain and Interdomain Routing

- Today, an internet can be so large that one routing protocol cannot handle the task of updating the routing tables of all routers. For this reason, an internet is divided into autonomous systems.
- An autonomous system (AS) is a group of networks and routers under the authority of a single administration. Routing inside an autonomous system is referred to as intradomain routing.

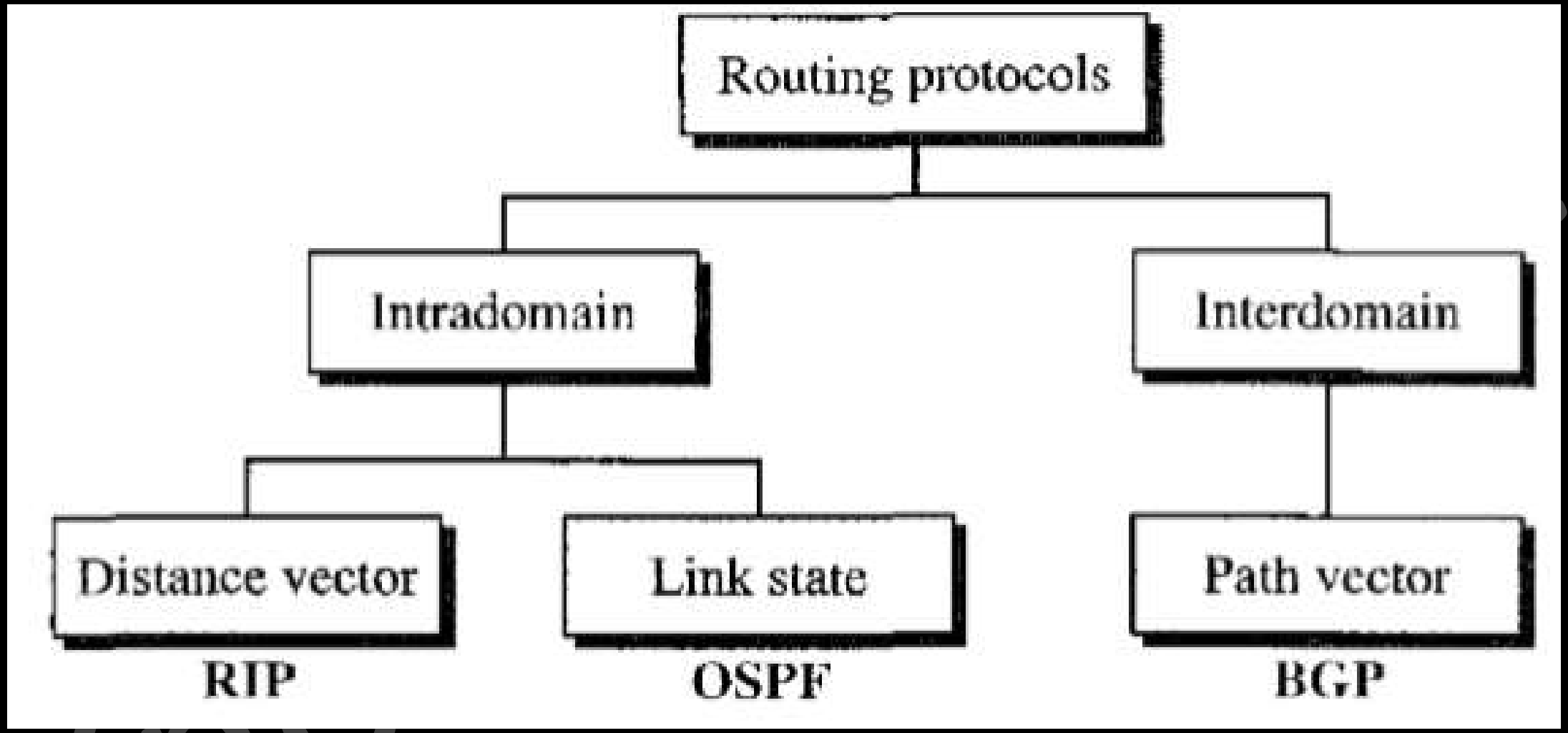


www.knowledgegate.in

- Routing between autonomous systems is referred to as interdomain routing.
- Each autonomous system can choose one or more intradomain routing protocols to handle routing inside the autonomous system. However, only one interdomain routing protocol handles routing between autonomous systems.



www.knowledgegate.in



www.knowledgegate.in

- Which of the available pathways is the optimum pathway? What is the definition of the term optimum? And What is the definition of the term cost

- However, the metric assigned to each network depends on the type of protocol. Some simple protocols, such as the Routing Information Protocol (RIP), treat all networks as equals. The cost of passing through a network is the same; it is one hop count. So if a packet passes through 10 networks to reach the destination, the total cost is 10 hop counts.

- Other protocols, such as Open Shortest Path First (OSPF), allow the administrator to assign a cost for passing through a network based on the type of service required. A route through a network can have different costs (metrics).

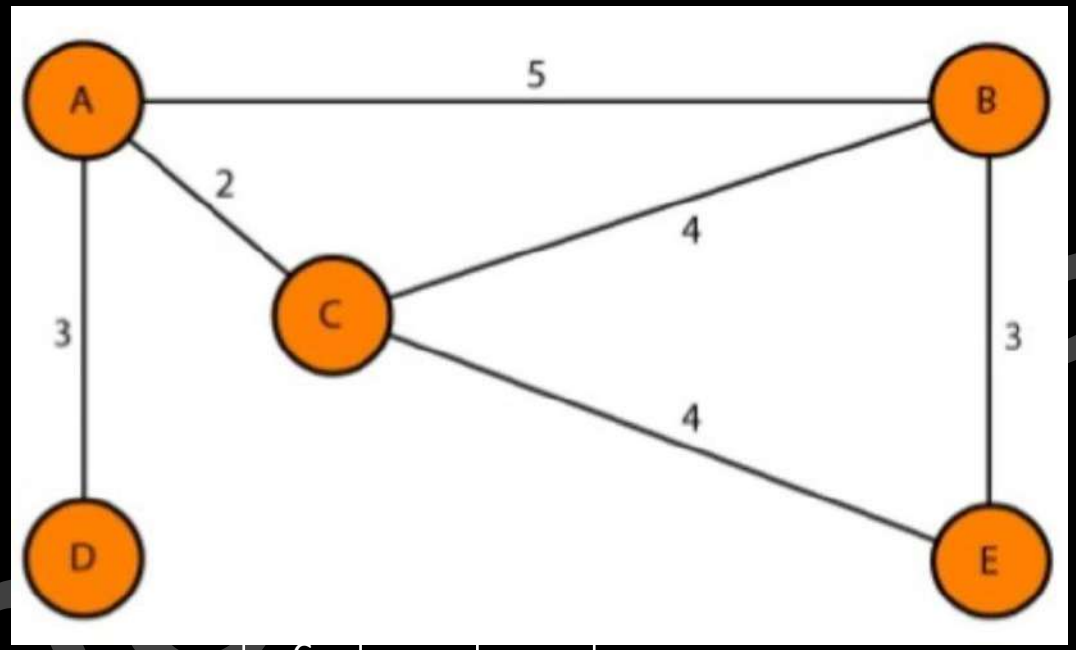
KnowledgeGate

www.knowledgegate.in

Distance Vector Routing

- In distance vector routing, the least-cost route between any two nodes is the route with minimum distance.
- In this protocol, as the name implies, each node maintains a vector (table) of minimum distances to every node. The table at each node also guides the packets to the desired node by showing the next stop in the route (next-hop routing).

To	Cost	Next
A		
B		
C		
D		
E		



To	Cost	Next
A		
B		
C		
D		
E		

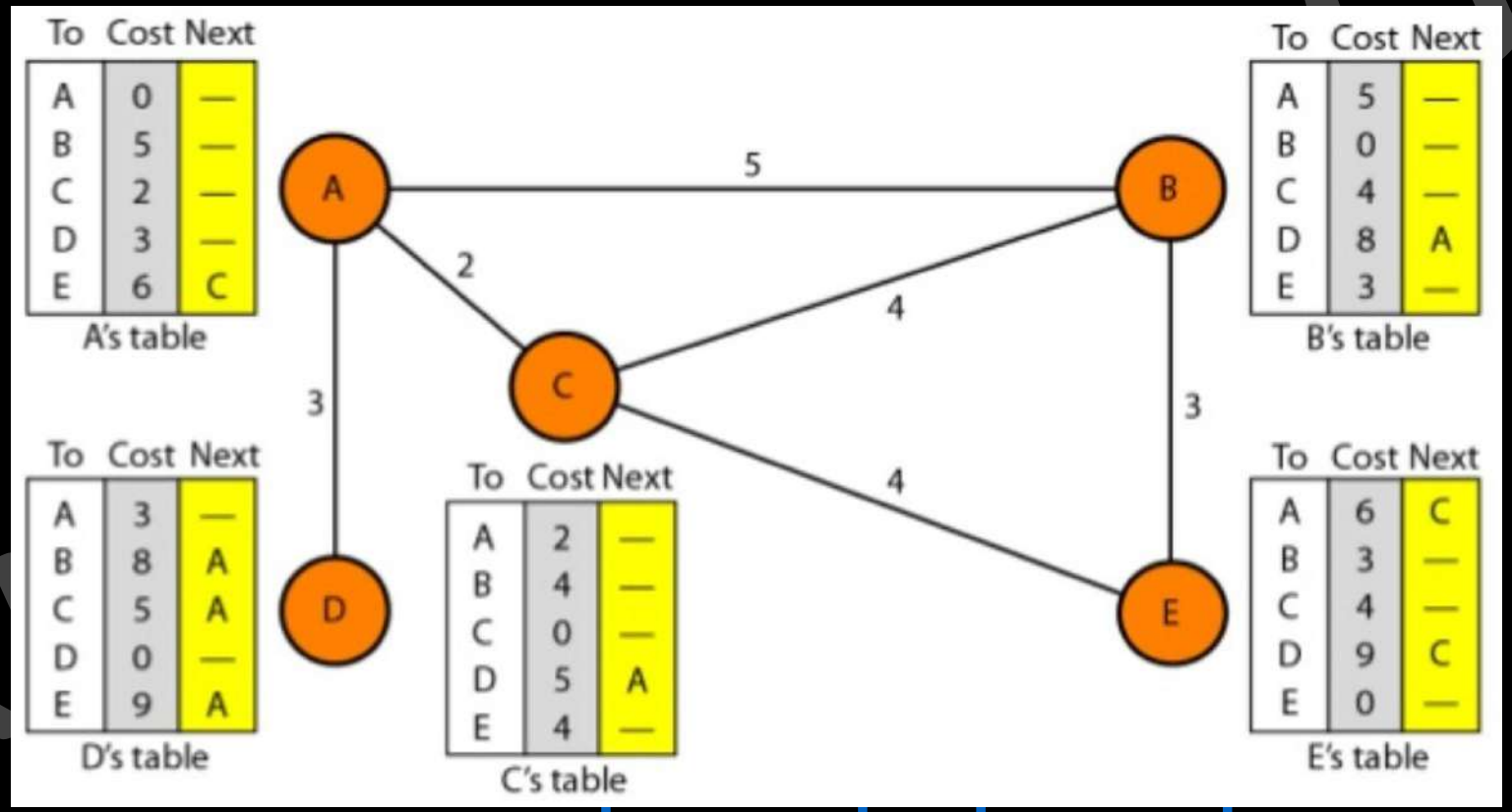
To	Cost	Next
A		
B		
C		
D		
E		

To	Cost	Next
A		
B		
C		
D		
E		

www.knowledgegate.in

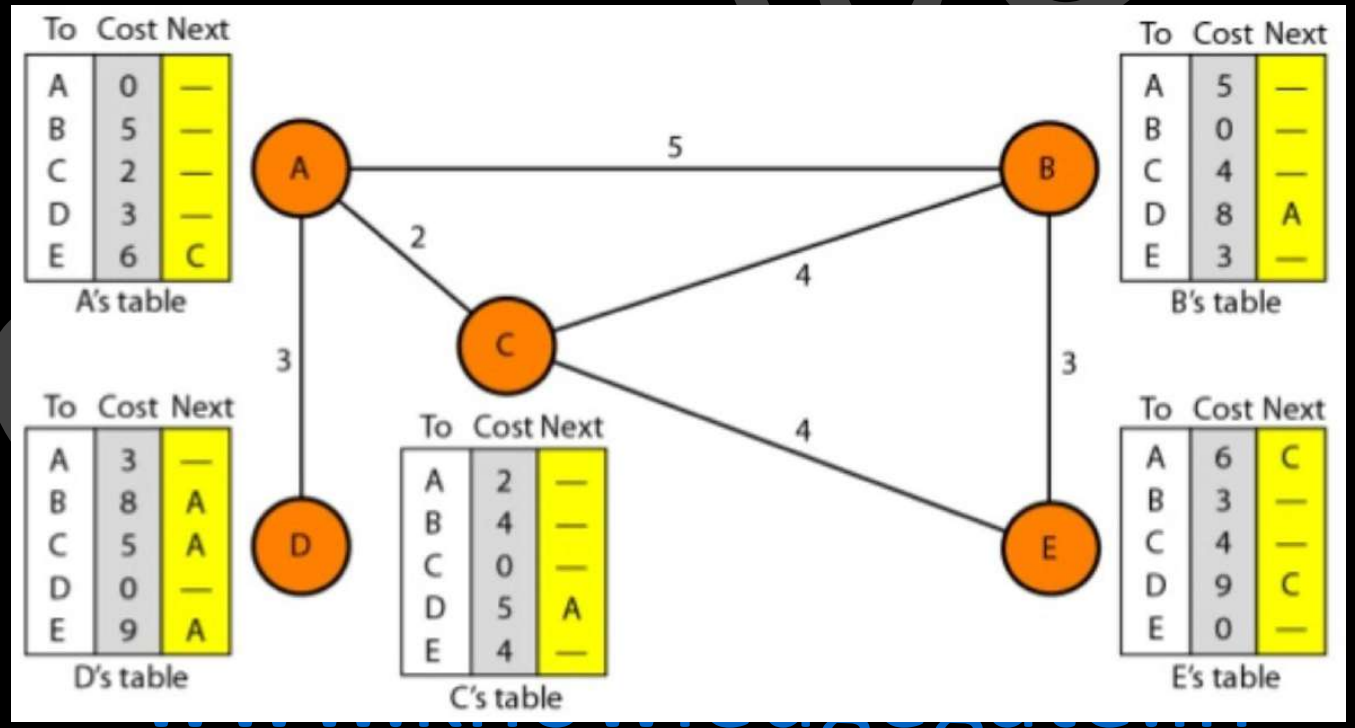
- The whole idea of distance vector routing is the sharing of information between neighbours. Although node A does not know about node E, node C does. So, if node C shares its routing table with A, node A can also know how to reach node E.

- On the other hand, node C does not know how to reach node D, but node A does. If node A shares its routing table with node C, node C also knows how to reach node D. In other words, nodes A and C, as immediate neighbours, can improve their routing tables if they help each other.



www.knowledgegate.in

- There is only one problem. How much of the table must be shared with each neighbour? A node is not aware of a neighbour's table. The best solution for each node is to send its entire table to the neighbour and let the neighbour decide what part to use and what part to discard.
- However, the third column of a table (next stop) is not useful for the neighbour. When the neighbour receives a table, this column needs to be replaced with the sender's name. If any of the rows can be used, the next node is the sender of the table.
- A node therefore can send only the first two columns of its table to any neighbour. In other words, sharing here means sharing only the first two columns.



- The question now is, when does a node send its partial routing table (only two columns) to all its immediate neighbours? The table is sent both periodically and when there is a change in the table.

- **Periodic Update** A node sends its routing table, normally every 30 s, in a periodic update. The period depends on the protocol that is using distance vector routing.

- **Triggered Update** A node sends its two-column routing table to its neighbours anytime there is a change in its routing table. This is called a triggered update. The change can result from the following.
  - A node receives a table from a neighbour, resulting in changes in its own table after updating.
  - A node detects some failure in the neighbouring links which results in a distance change to infinity.

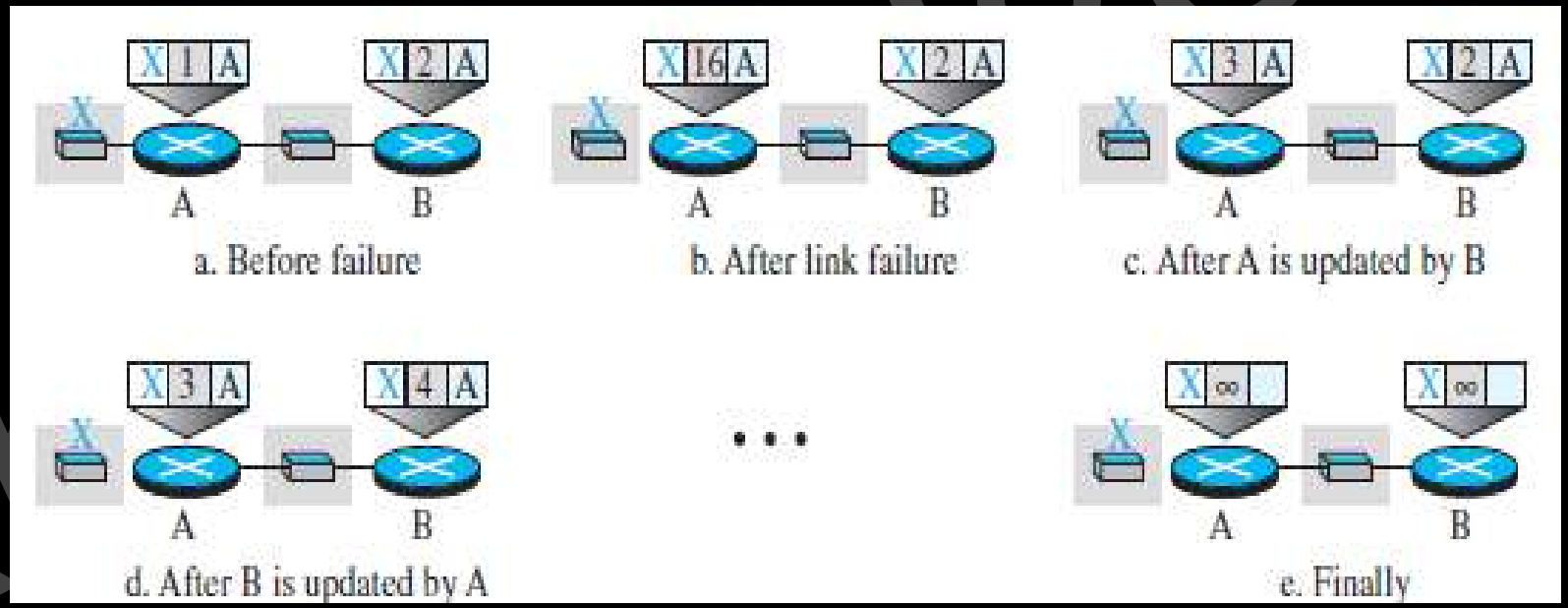
KnowledgeGate

www.knowledgegate.in

Two-Node Loop Instability

- If a link is broken (cost becomes infinity), every other router should be aware of it immediately, but in distance-vector routing, this takes some time as the algorithms is designed in such a way that it reports the minimum first.

- The problem is referred to as *count to infinity*. It sometimes takes several updates before the cost for a broken link is recorded as infinity by all routers.



www.knowledgegate.in

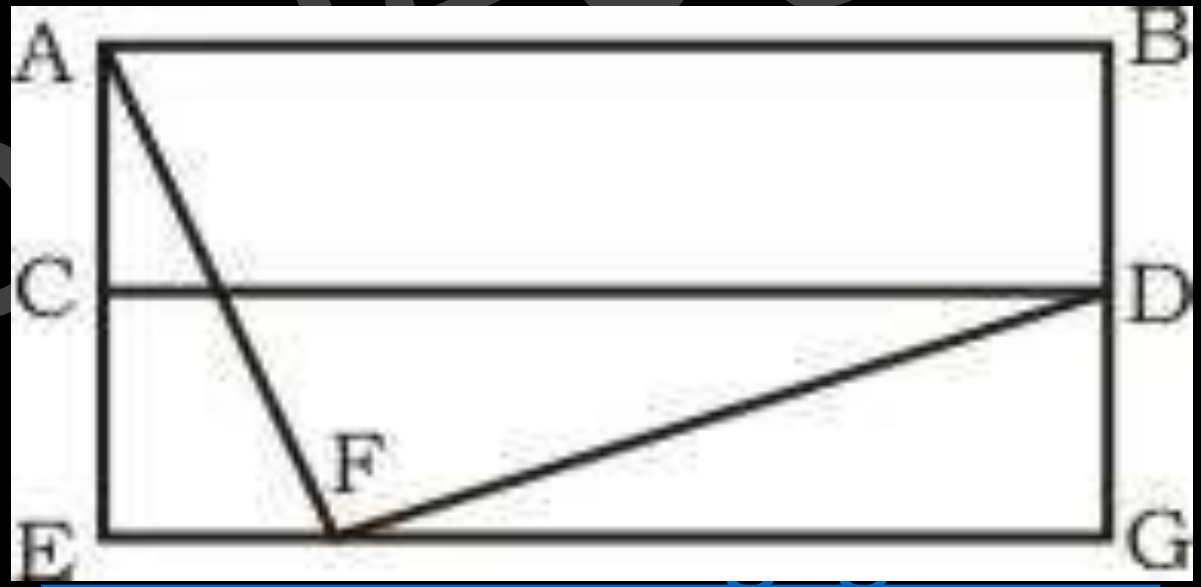
Split Horizon

- In this strategy, instead of flooding the table through each interface, each node sends only part of its table through each interface.
- If, according to its table, node B thinks that the optimum route to reach X is via A, it does not need to advertise this piece of information to A; the information has come from A (A already knows).
- Taking information from node A, modifying it, and sending it back to node A creates the confusion. In our scenario, node B eliminates the last line of its routing table before it sends it to A.
- In this case, node A keeps the value of infinity as the distance to X. Later when node A sends its routing table to B, node B also corrects its routing table. The system becomes stable after the first update: both node A and B know that X is not reachable

KnowledgeGate

Link State Routing

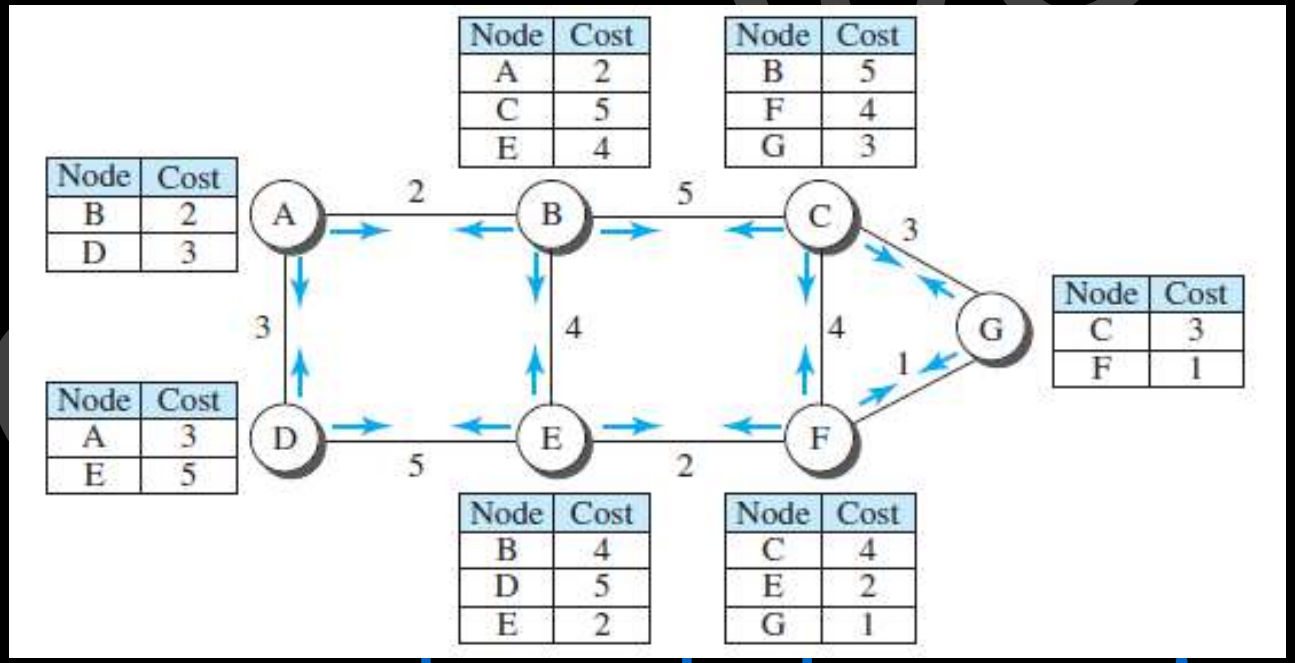
- Link state routing has a different philosophy from that of distance vector routing. In link state routing, if each node in the domain has the entire topology of the domain the list of nodes and links, how they are connected including the type, cost (metric), and condition of the links (up or down)-the node can use Dijkstra's algorithm to build a routing table.
- Each node uses the same topology to create a routing table, but the routing table for each node is unique because the calculations are based on different interpretations of the topology.
- The topology must be dynamic, representing the latest state of each node and each link. If there are changes in any point in the network (a link is down, for example), the topology must be updated for each node.



KnowledgeGate

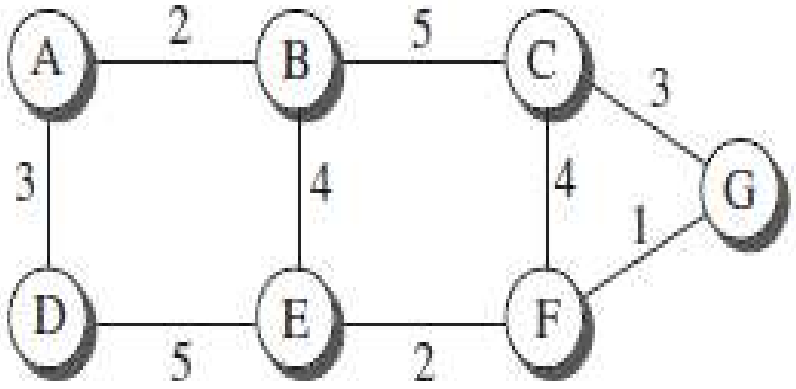
Building Routing Tables

- In link state routing, four sets of actions are required to ensure that each node has the routing table showing the least-cost node to every other node.
  1. Creation of the states of the links by each node, called the link state packet (LSP).
  2. Dissemination of LSPs to every other router, called flooding, in an efficient and reliable way.
  3. Formation of a shortest path tree for each node.
  4. Calculation of a routing table based on the shortest path tree



www.knowledgegate.in

- The collection of state for all links is called the ***link-state database (LSDB).***
- There is only one LSDB for the whole internet; each node needs to have a duplicate of it to be able to create the least-cost tree.
- The LSDB can be represented as a two-dimensional array (matrix) in which the value of each cell defines the cost of the corresponding link.



a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

b. Link state database

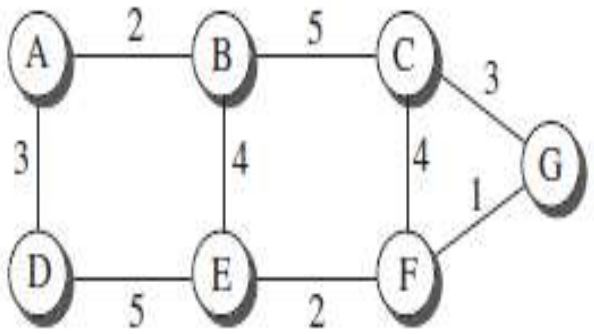
www.knowledgegate.in

- When a node receives an LSP from one of its interfaces, it compares the LSP with the copy it may already have.
  - If the newly arrived LSP is older than the one it has (found by checking the sequence number), it discards the LSP.
  - If it is newer or the first one received, the node discards the old LSP (if there is one) and keeps the received one.
  - It then sends a copy of it out of each interface except the one from which the packet arrived.

KnowledgeGate

www.knowledgegate.in

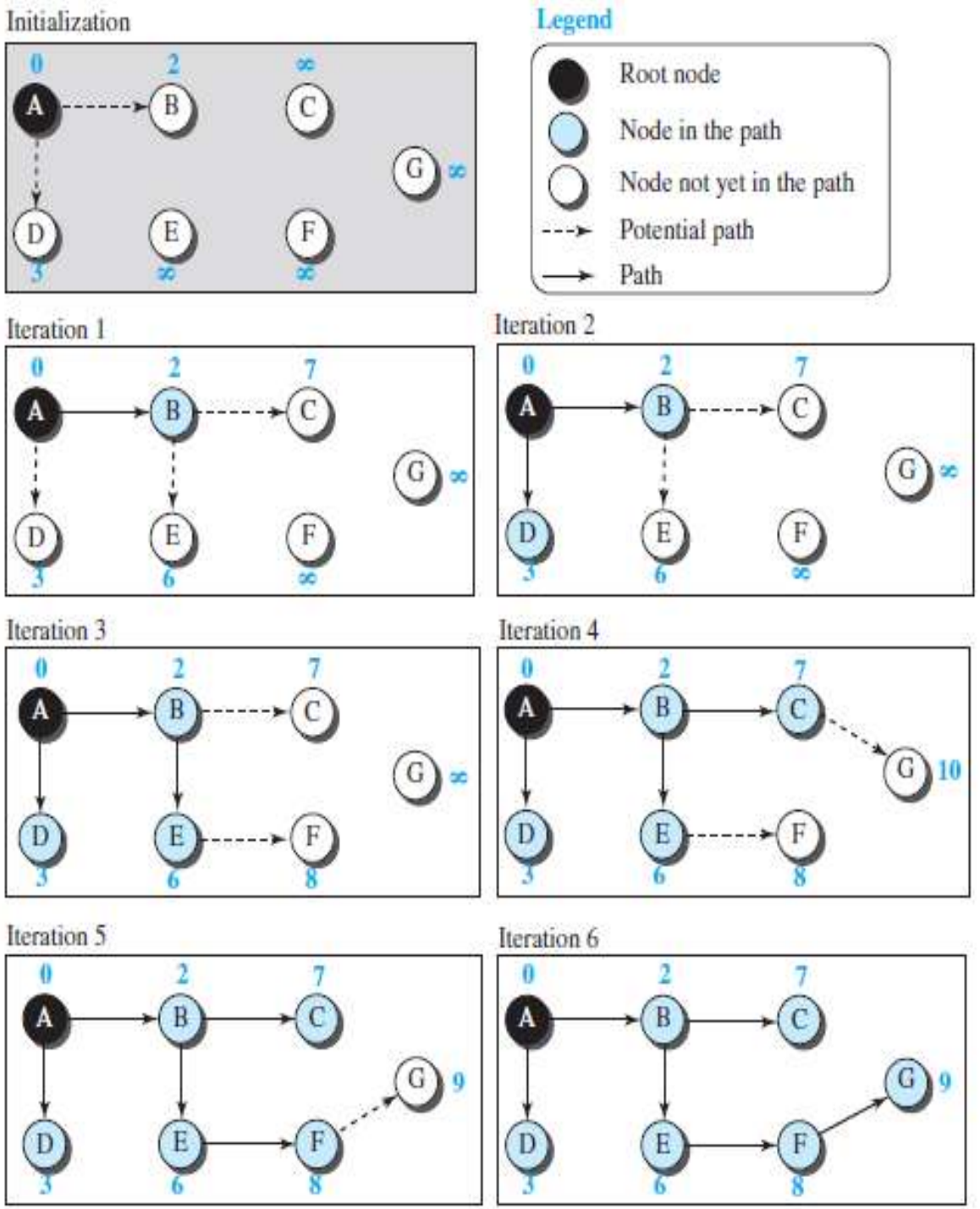
- The Dijkstra algorithm creates a shortest path tree from a graph. The algorithm divides the nodes into two sets: tentative and permanent. It finds the neighbours of a current node, makes them tentative, examines them, and if they pass the criteria, makes them permanent.
- To find the shortest path in each step, we need the cumulative cost from the root to each node, which is shown next to the node. Nodes and lists with the cumulative costs.



a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

b. Link state database



Difference between DVR and LSR

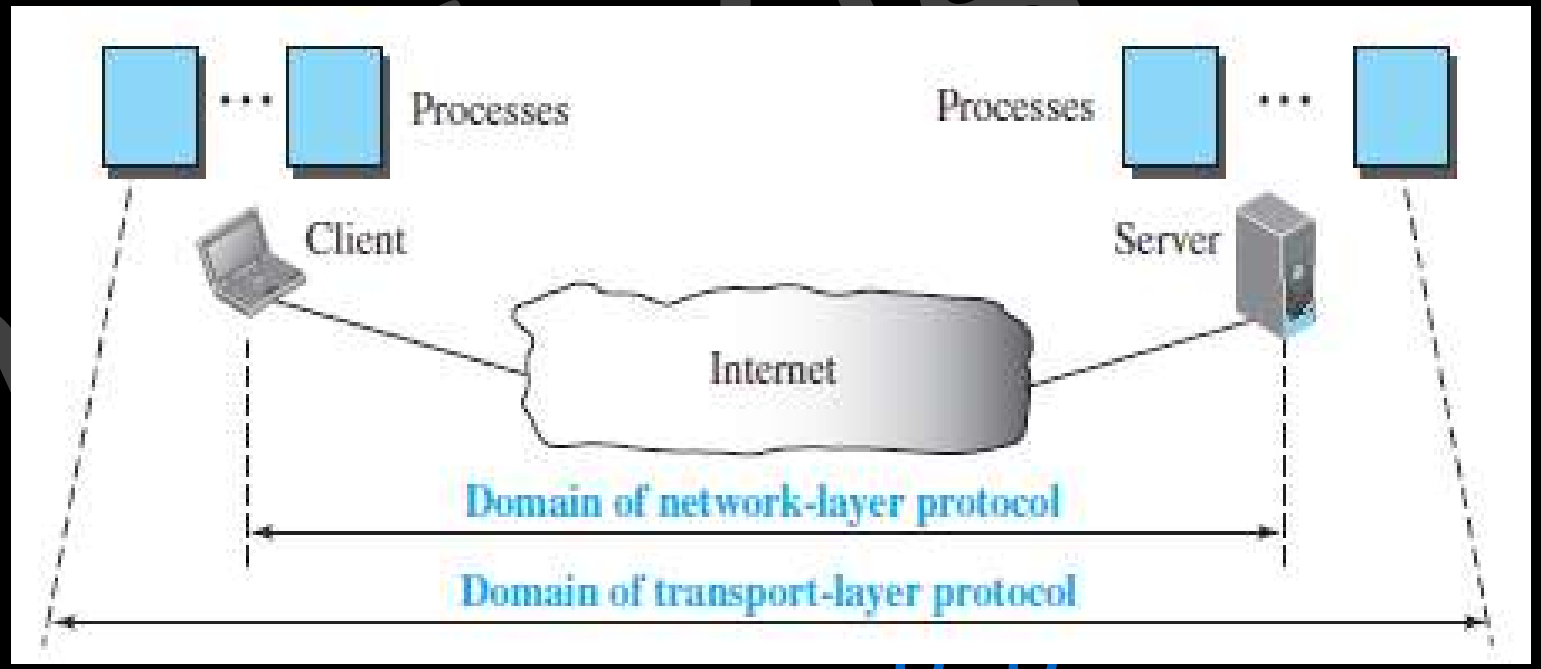
• In the distance-vector routing algorithm, each router tells its neighbours what it knows about the whole internet.	• In the link-state routing algorithm, each router tells the whole internet what it knows about its neighbours.
• Was most popularly used around 1980's	• Was most popularly used around 1990's
• Based on the idea of Local Knowledge	• Based on the idea of Global Knowledge
• Bandwidth requirement is Less	• Bandwidth requirement is high
• Roughly based on the idea of Bellman-Ford Algo	• Directly based on the idea of Dijkstra's
• Traffic is usually less	• Traffic is Usually high
• Converge slowly	• Converge faster
• Counts to Infinity	• No Counts to infinity
• RIP	• OSPF

KnowledgeGate

www.knowledgegate.in

Transport-Layer Services

- The network layer is responsible for communication at the computer level (host-to-host communication). A network-layer protocol can deliver the message only to the destination computer.
- However, this is an incomplete delivery, as the message still needs to be handed to the correct process. A transport-layer protocol is responsible for delivery of the message to the appropriate process. TL provides end to end or process to process communication.



KnowledgeGate

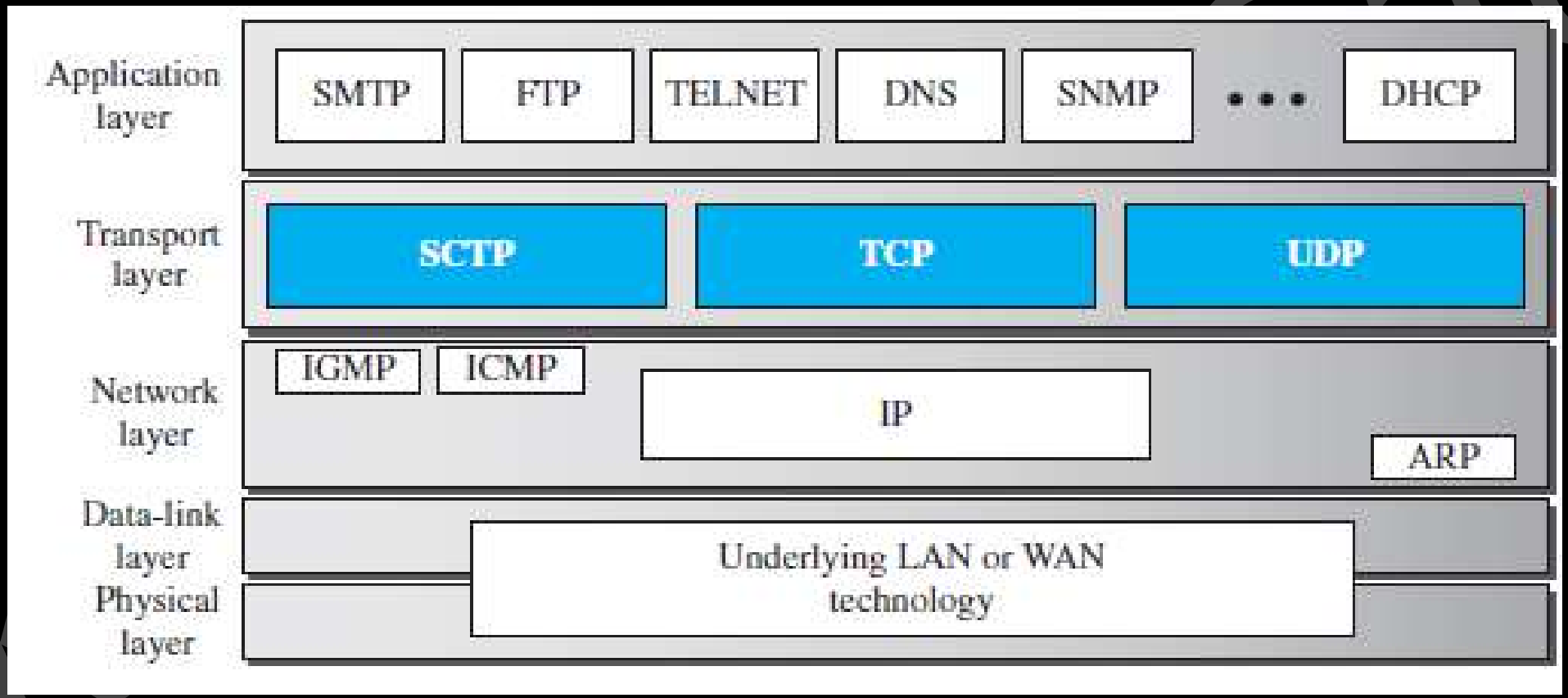
- A transport layer protocol can be either connectionless or connection-oriented.
  - A connectionless transport layer treats each segment as an independent packet and delivers it to the transport layer at the destination machine.
  - A connection-oriented transport layer makes a connection with the transport layer at the destination machine first before delivering the packets. After all the data is transferred, the connection is terminated.

- Reliable Versus Unreliable
  - If the application layer program needs reliability, we use a reliable transport layer protocol by implementing flow and error control at the transport layer. This means a slower and more complex service.
  - On the other hand, if the application program does not need reliability because it uses its own flow and error control mechanism or it needs fast service or the nature of the service does not demand flow and error control (real-time applications), then an unreliable protocol can be used.

KnowledgeGate

www.knowledgegate.in

- There are three common different transport layer protocols.
  - UDP is connectionless and unreliable;
  - TCP and SCTP are connection oriented and reliable. These three can respond to the demands of the application layer programs.



www.knowledgegate.in

Addressing: Port Numbers

- For communication, we must define the local host, local process, remote host, and remote process. Local and Remote host are defined by IP Addresses. To define the processes inside a host, we need second identifiers, called port numbers, they are 16-bits integers ranging from (0 to $2^{16} - 1$) or (0 to 65535).

- The IANA (Internet Assigned Number Authority) has divided the port numbers into three ranges: well known, registered, and dynamic (or private).

KnowledgeGate

- **Well-known ports:**
  - The ports ranging from 0 to 1023 are assigned and controlled by IANA. The server process must also define itself with a port number. This port number, however, cannot be chosen randomly as the client has to request the data from server.
- **Registered ports:**
  - The ports ranging from 1024 to 49,151 are not assigned or controlled by IANA. They can only be registered with IANA to prevent duplication.
- **Dynamic/Ephemeral ports:**
  - The ports ranging from 49,152 to 65,535 are neither controlled nor registered. They can be used by any process.
  - The client program defines itself with a port number, chosen randomly by the transport layer software running on the client host.
  - Ephemeral means “short-lived” and is used because the life of a client is normally short.

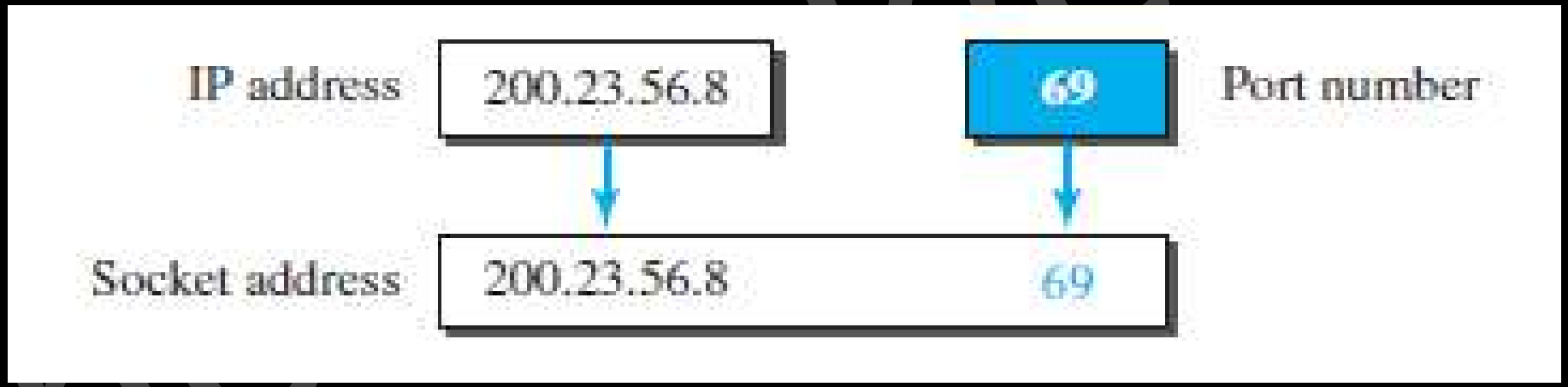
Port Number	Protocol	Application
20	TCP	FTP data
21	TCP	FTP control
22	TCP	SSH
23	TCP	Telnet
25	TCP	SMTP
53	UDP, TCP	DNS
67, 68	UDP	DHCP
69	UDP	TFTP
80	TCP	HTTP (WWW)
110	TCP	POP3
161	UDP	SNMP
443	TCP	HTTPS (SSL)
16384–32767	UDP	RTP-based voice (VoIP) and video

Knowledg

www.knowledgegate.in

Socket Addresses

- A transport-layer protocol in the TCP suite needs both the IP address and the port number, at each end, to make a connection. To use the services of the transport layer in the Internet, we need a pair of socket addresses: the client socket address and the server socket address.
- The combination of an IP address and a port number is called a ***socket address.***

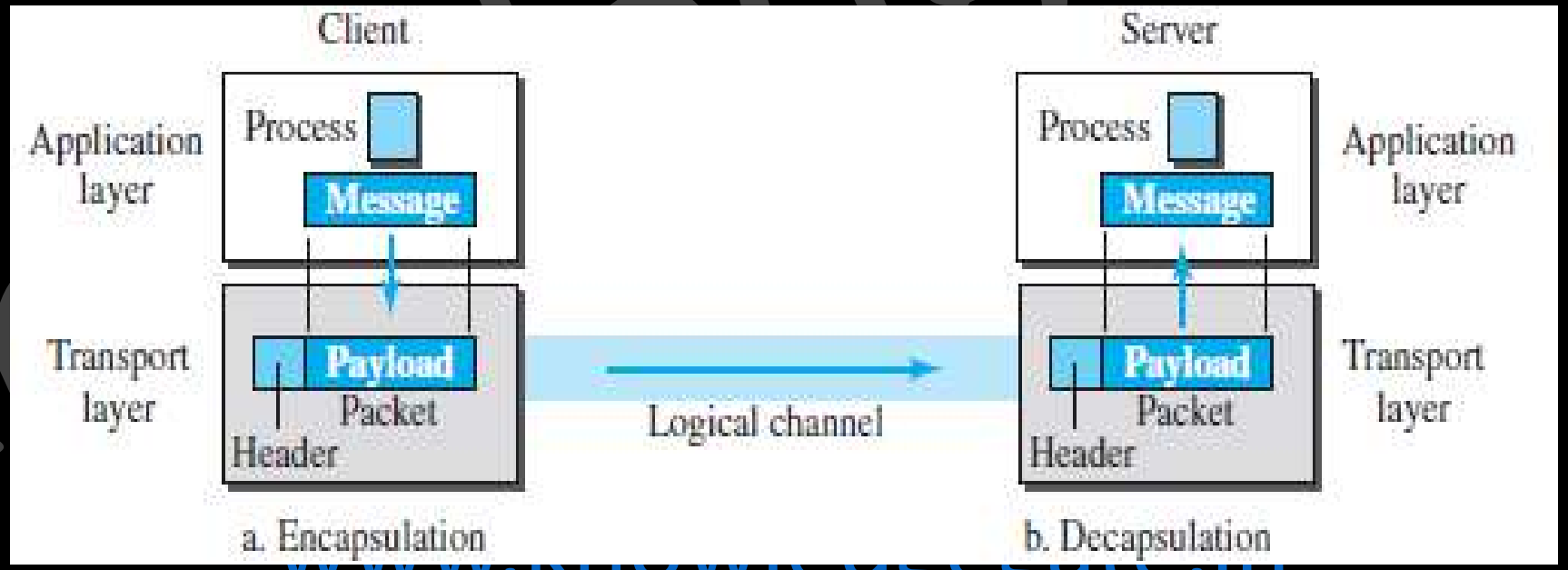


KnowledgeGate

Encapsulation and Decapsulation

- Encapsulation happens at the sender site. When a process has a message to send, it passes the message to the transport layer along with a pair of socket addresses. The transport layer receives the data and adds the transport-layer header. The packets at the transport layer in the Internet are called ***segments.***

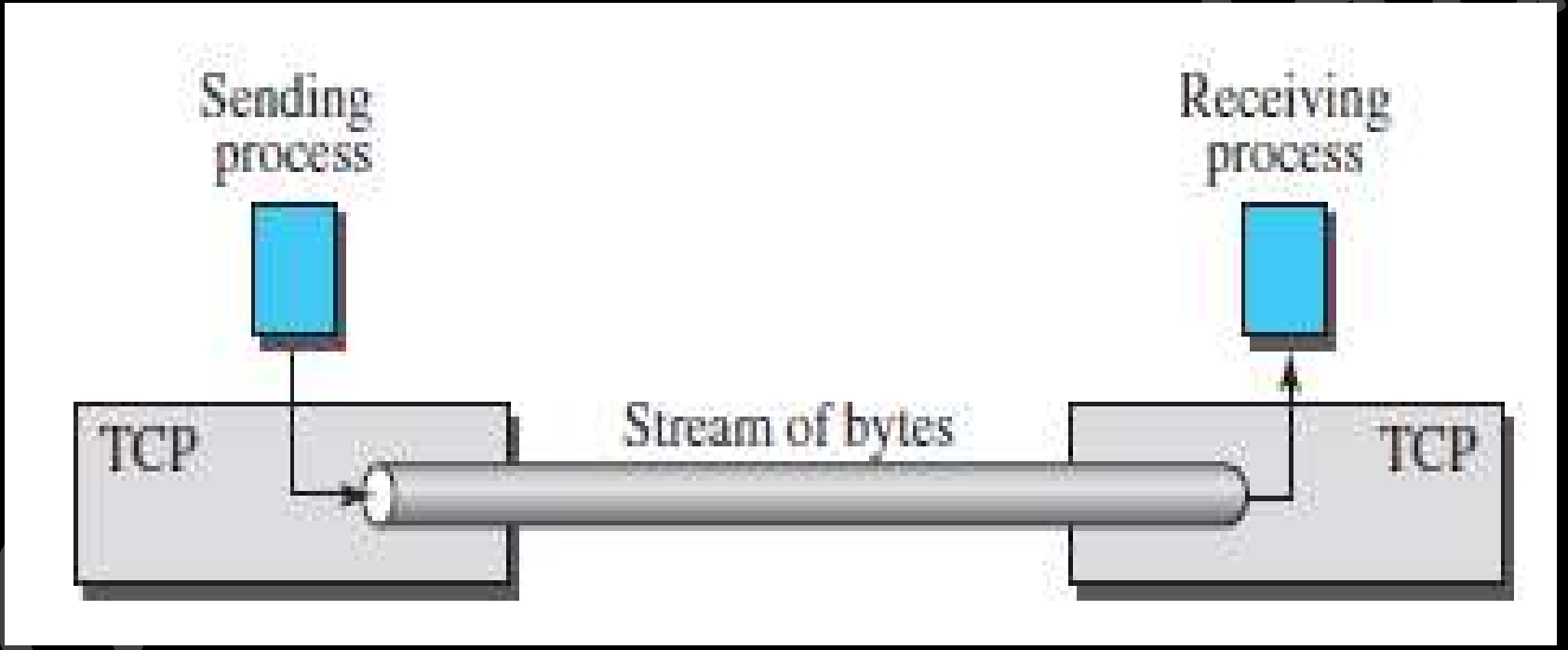
- Decapsulation happens at the receiver site. When the message arrives at the destination transport layer, the header is dropped and the transport layer delivers the message to the process running at the application layer.



KnowledgeGate
www.knowledgegate.in

TCP (Transmission Control Protocol)

- TCP creates a virtual connection between two TCPs to send data. TCP allows the sending process to deliver data as a stream of bytes and allows the receiving process to obtain data as a stream of bytes. TCP creates an environment in which the two processes seem to be connected by an imaginary "tube" that carries their data across the Internet.



www.knowledgegate.in

- **Flow Control**
  - The receiver of the data controls the amount of data that are to be sent by the sender. This is done to prevent the receiver from being overwhelmed with data. The numbering system allows TCP to use a byte-oriented flow control.

- **Error Control**
  - To provide reliable service, TCP implements an error control mechanism. Although error control considers a segment as the unit of data for error detection (loss or corrupted segments), error control is byte-oriented.

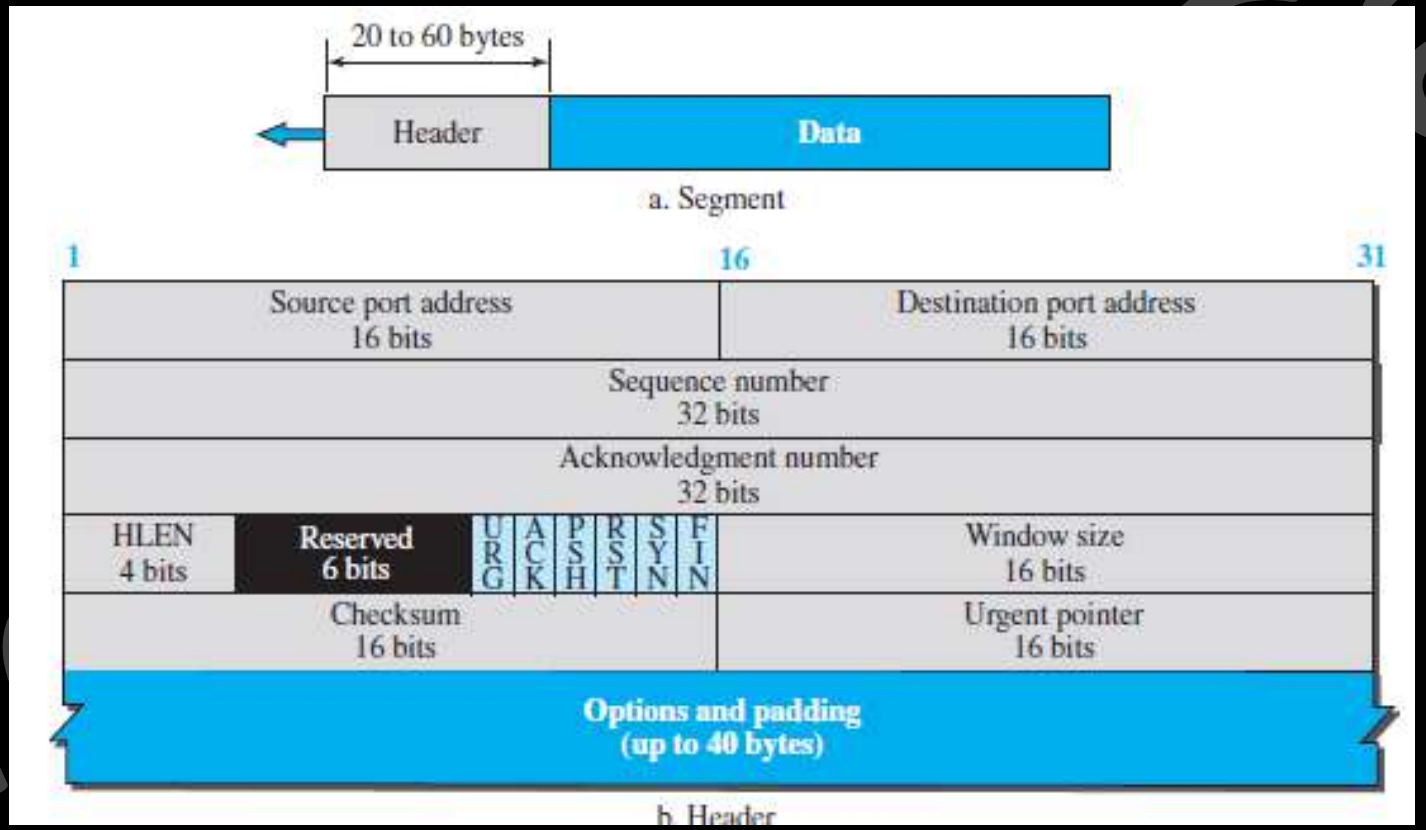
- **Congestion Control**
  - TCP, unlike UDP, takes into account congestion in the network. The amount of data sent by a sender is not only controlled by the receiver (flow control), but is also determined by the level of congestion in the network.

KnowledgeGate

www.knowledgegate.in

TCP Header

- The segment consists of a 20- to 60-byte header, followed by data from the application program. The header is 20 bytes if there are no options and up to 60 bytes if it contains options.

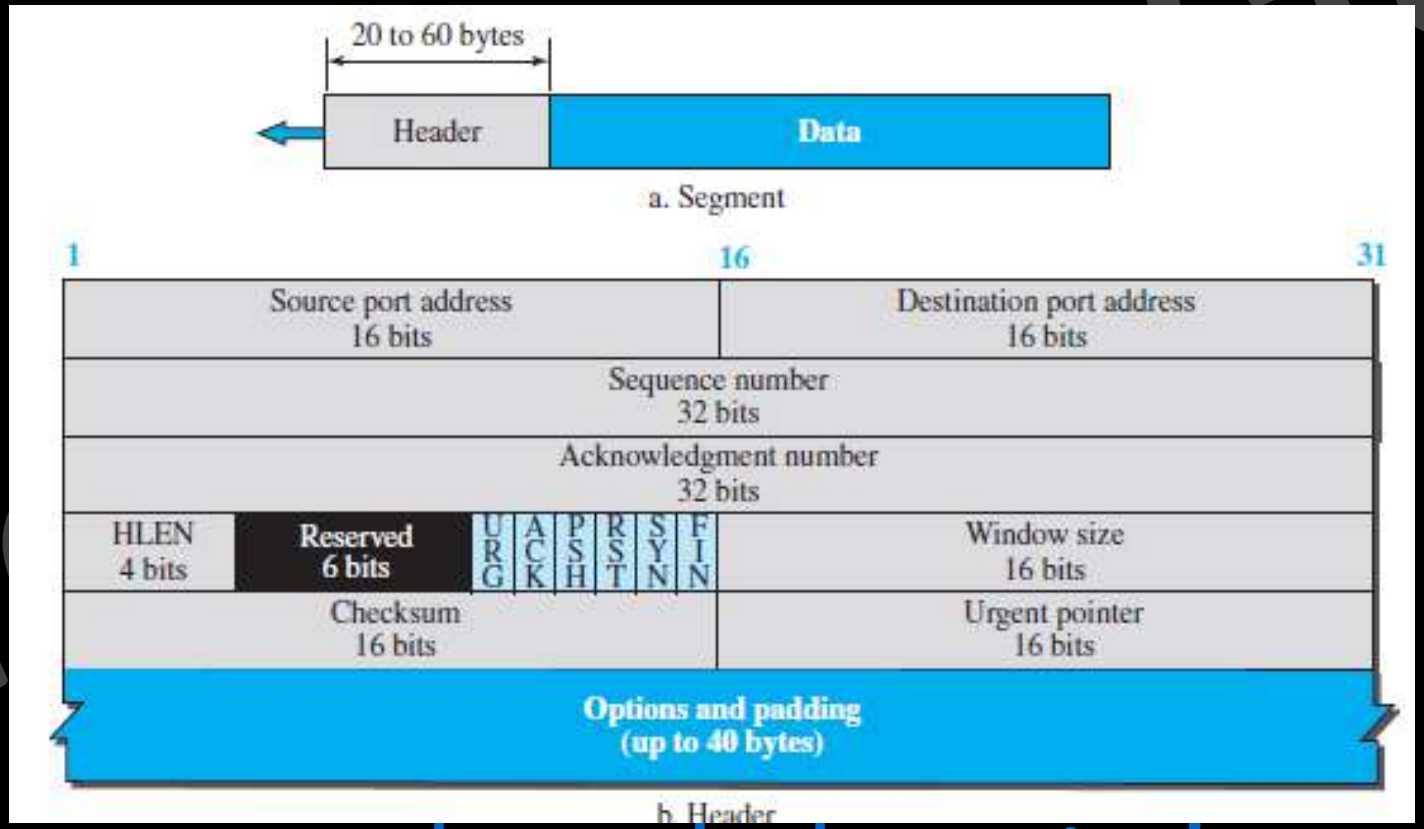


www.knowledgegate.in

Source & Destination port address

- **Source port address**. This is a 16-bit field that defines the port number of the application program in the host that is sending the segment.

- **Destination port address**. This is a 16-bit field that defines the port number of the application program in the host that is receiving the segment.

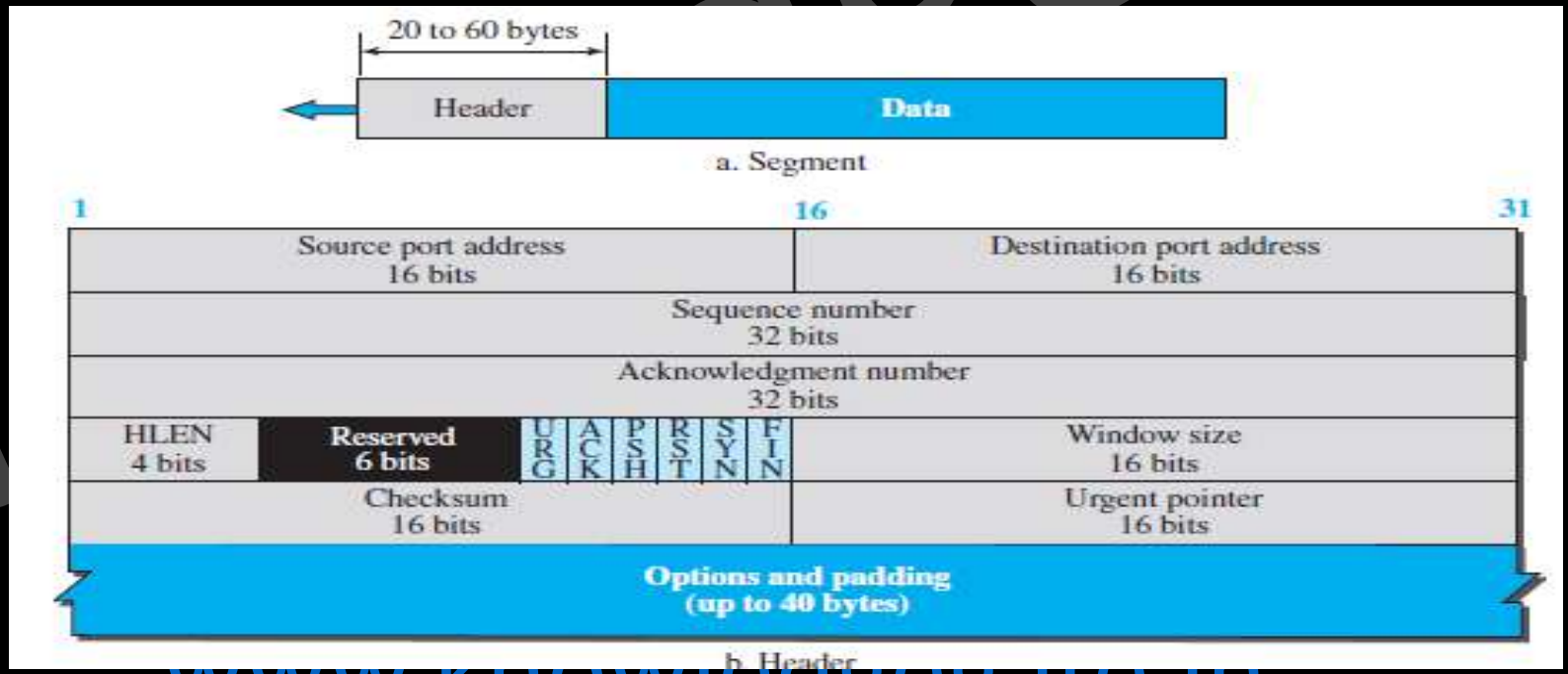


www.knowledgegate.in

Sequence number

- TCP is a stream transport protocol. To ensure connectivity, each byte to be transmitted is numbered. Sequence number is 32-bit field defines the number assigned to the first byte of data contained in this segment. So, maximum number of possible sequence numbers = 2^{32}. These sequence numbers lie in the range [0, $2^{32} - 1$].

- During connection establishment, each party uses a random number generator to create an initial sequence number (ISN), which is usually different in each direction. Sequence number should be started at random, to remove duplication problem. The sequence number of any other segment is the sequence number of the previous segment plus the number of bytes (real or imaginary) carried by the previous segment.



www.knowledgegate.in

- This does not imply that only 2^{32} bytes = 4 GB data can be sent using TCP. The concept of wrap around allows to send unlimited data using TCP.

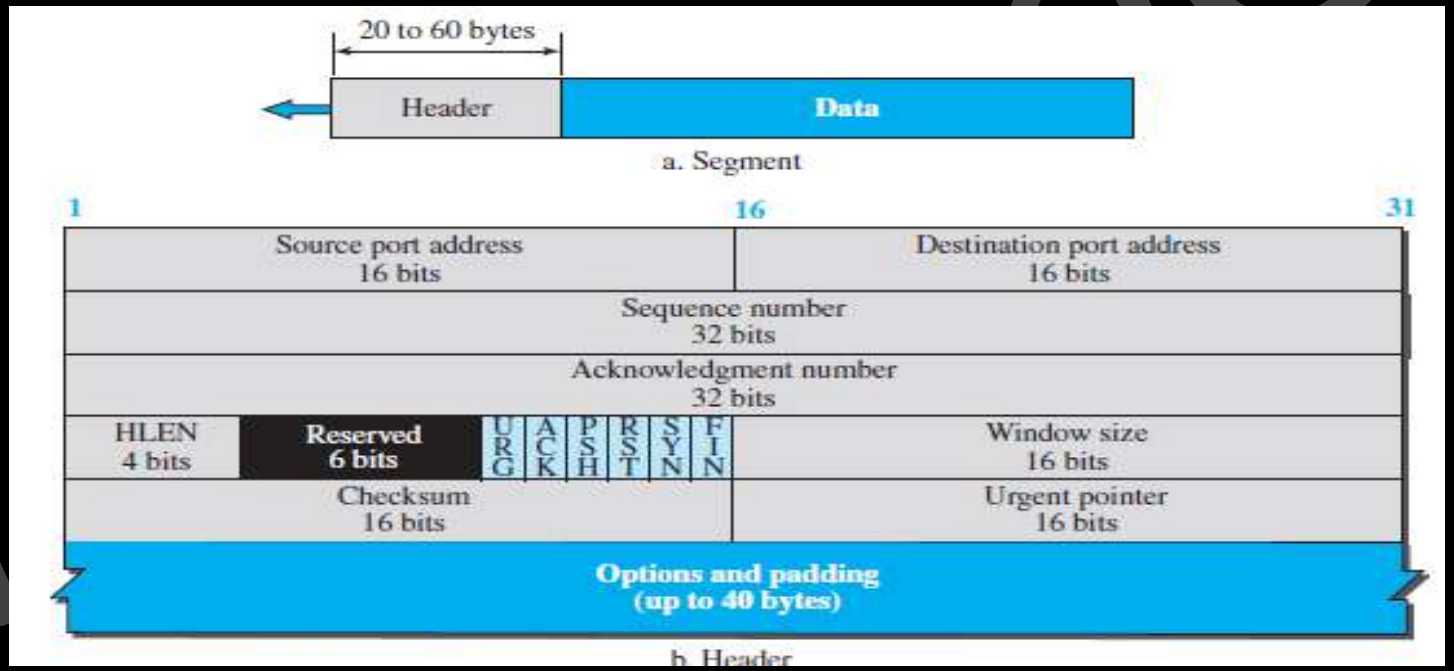
- After all the 2^{32} sequence numbers are used up and more data is to be sent, the sequence numbers can be wrapped around and used again from the starting.

KnowledgeGate

Acknowledgment Number

- If the receiver of the segment has successfully received byte number x from the other party, it defines x + 1 as the acknowledgment number. Acknowledgment and data can be piggybacked together.

- The acknowledgment number is cumulative, which means that the party takes the number of the last byte that it has received, safe and sound, adds 1 to it, and announces this sum as the acknowledgment number.



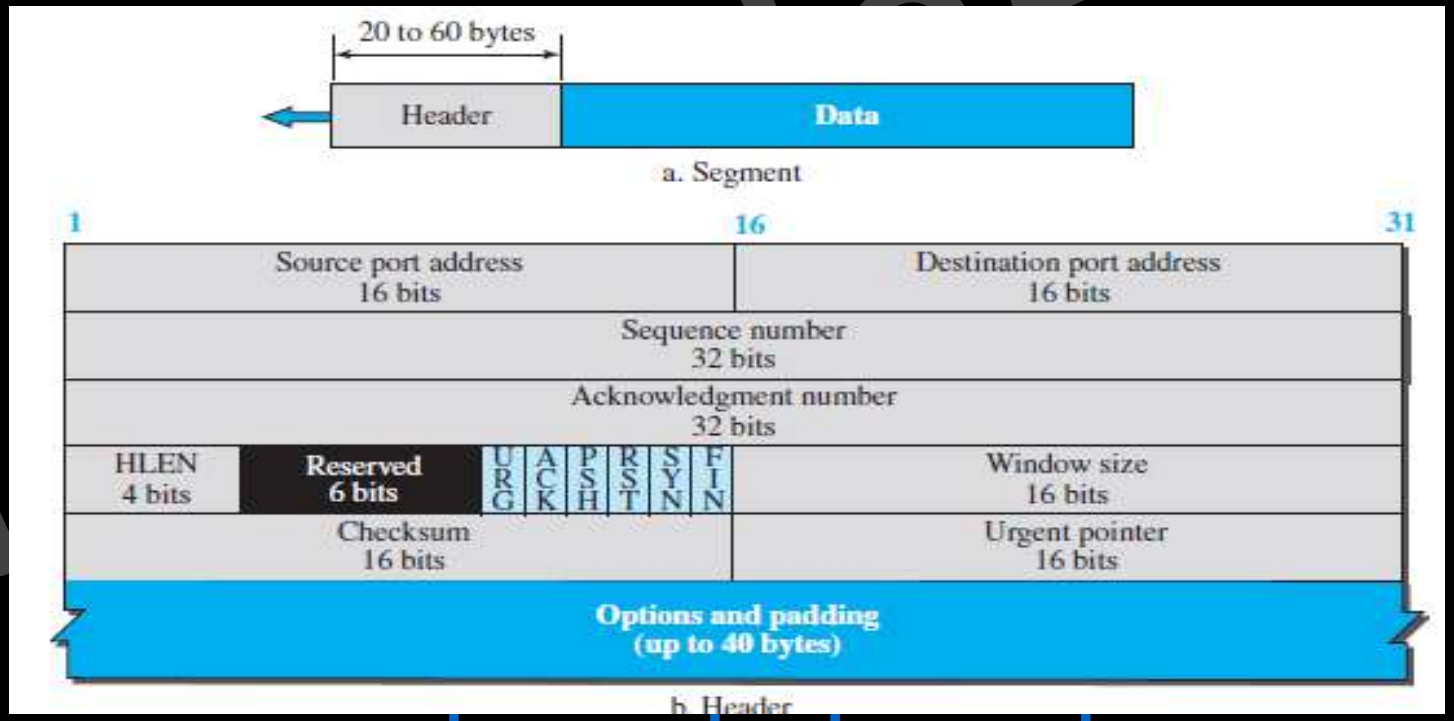
www.knowledgegate.in

Header length

- Header length: This 4-bit field indicates the number of 4-byte words in the TCP header. The length of the header can be between 20 and 60 bytes. Therefore, the value of this field can be between 5 (5 x 4 =20) and 15 (15 x 4 =60).

- Concept of Scaling Factor
  - *Header length = Header length field value x 4 bytes*

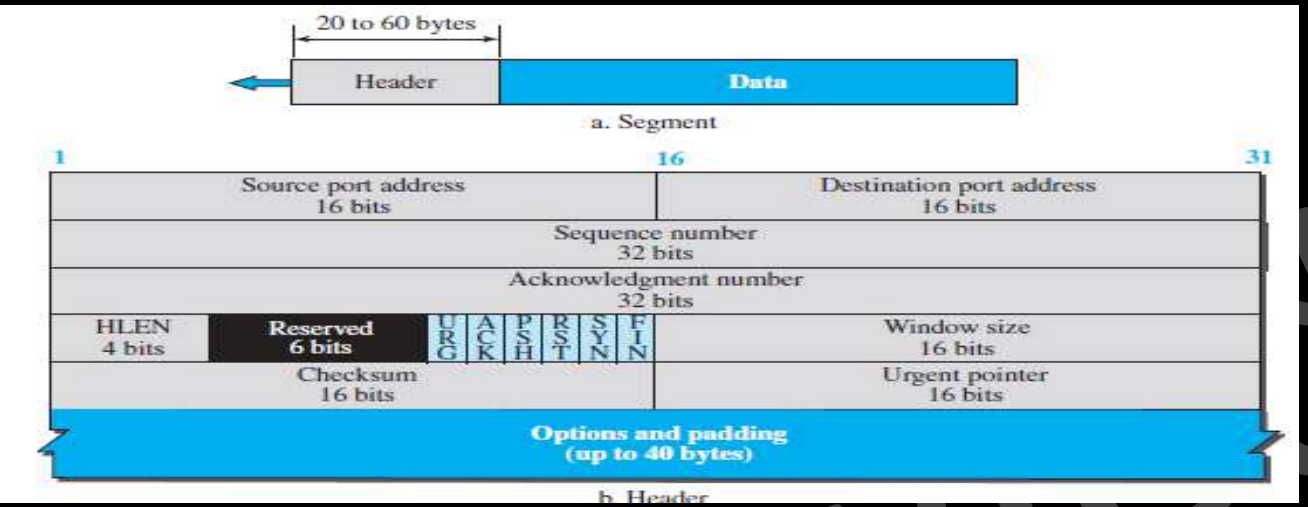
- Reserved. This is a 6-bit field reserved for future use.

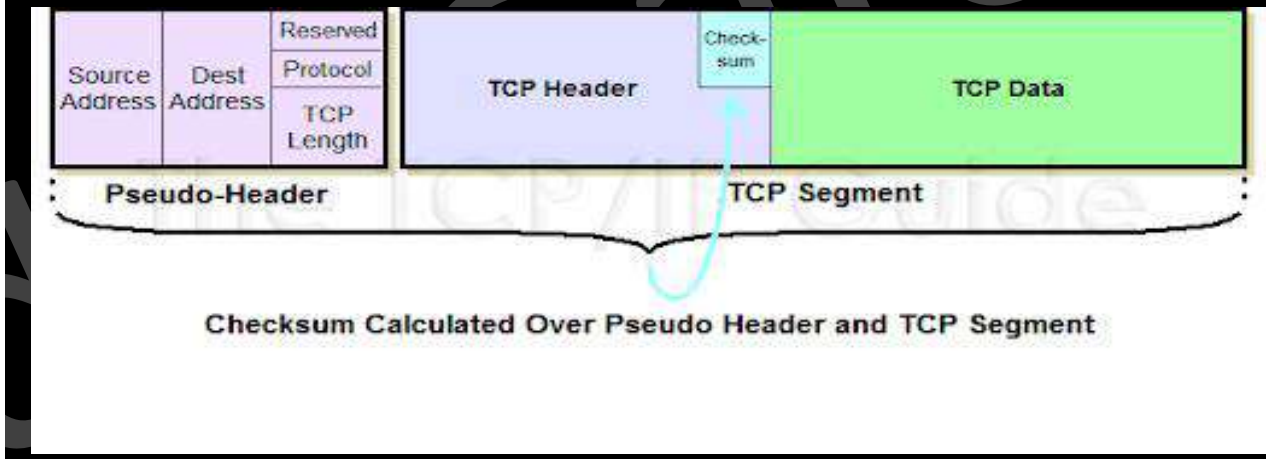


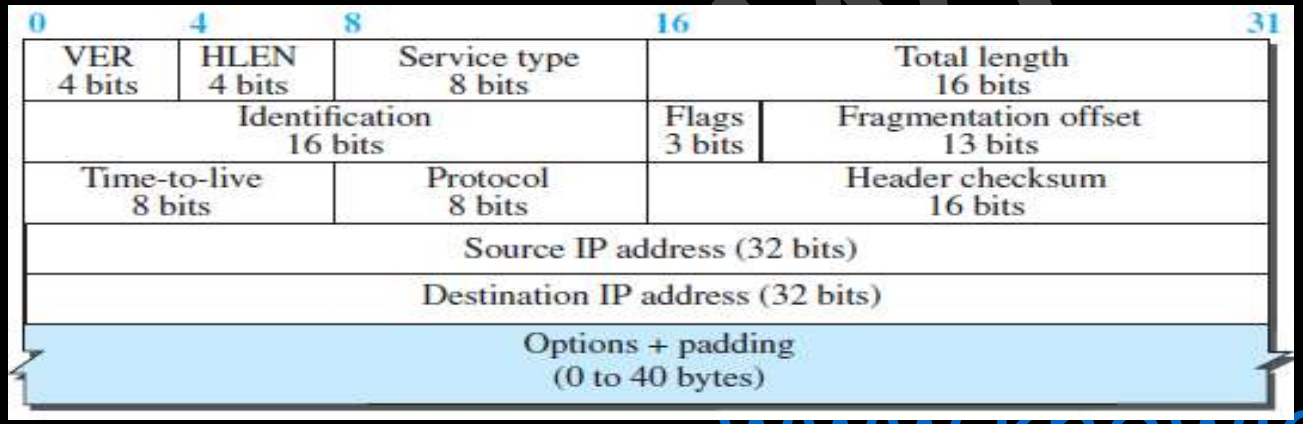
www.knowledgegate.in

Checksum

- **Checksum:** This 16-bit field contains the checksum.
- While calculation of the checksum for TCP, Entire TCP segment and pseudo header (IP) is considered.
- For the TCP pseudo header, the value for the protocol field is 6.



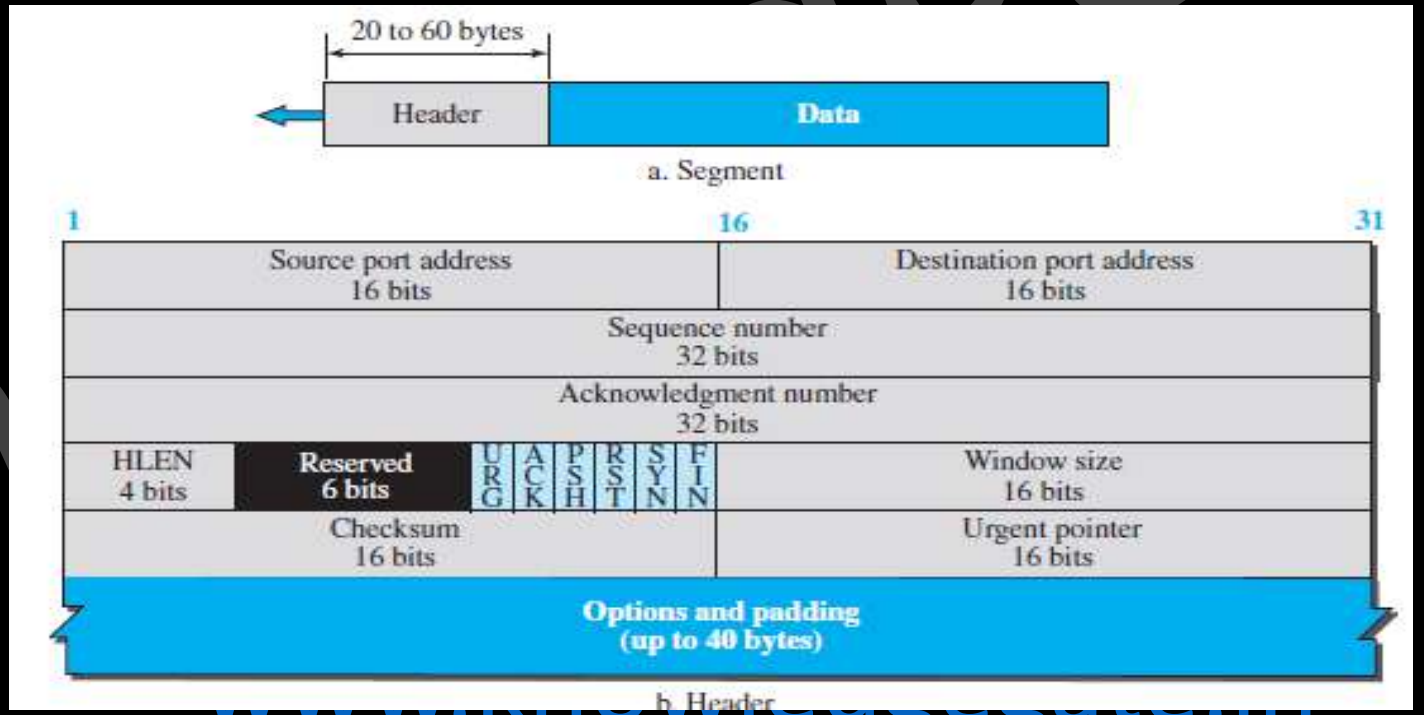




www.knowledgegate.in

Window Size/Advertisement window(Flow control)

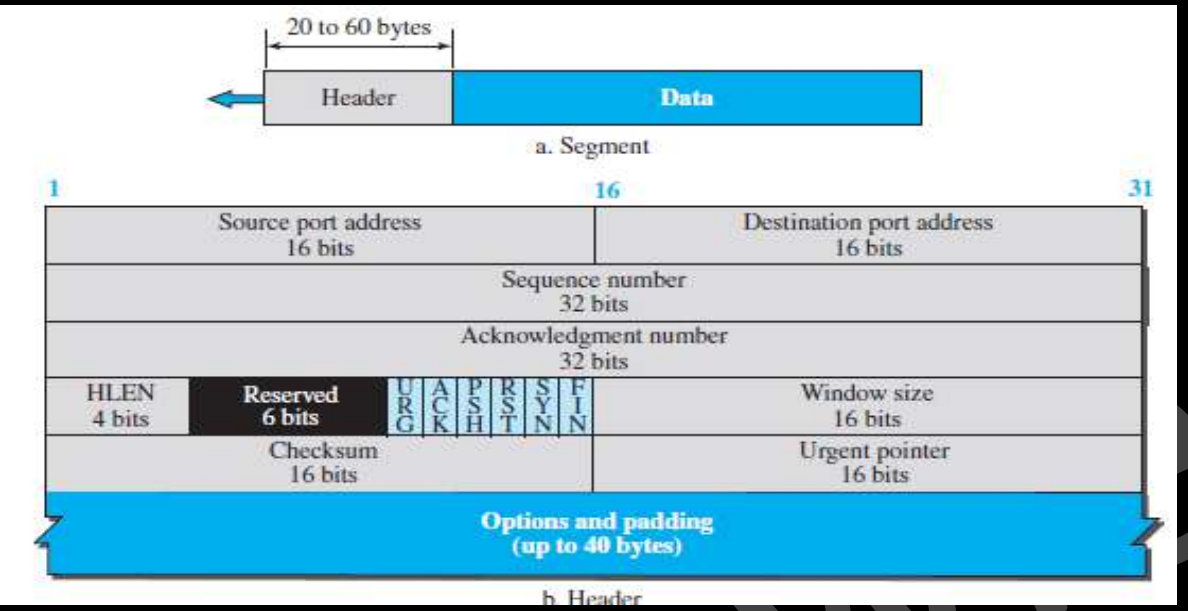
- Basics idea is a sender should never send what a receiver can not receive. Window size**:** This field defines the size of the window, in bytes, that the receiver have reserved for the incoming data from sender.
- Note that the length of this field is 16 bits, which means that the maximum size of the window is 65,535 bytes. This value is normally referred to as the receiving window and is determined by the receiver. The sender must obey the dictation of the receiver in this case.
- One problem with this idea is window size is only 16 bits long, so very small amount of data can be advertise in todays world context.A solution is additional 14 bits can be taken from options so total size become 30 bits or 1GB.



KnowledgeGate
www.knowledgegate.in

Control Flag

- **Control Flag:** This field defines 6 different control bits or flags. One or more of these bits can be set at a time.
- These bits enable flow control, connection establishment and termination, connection abortion, and the mode of data transfer in TCP.

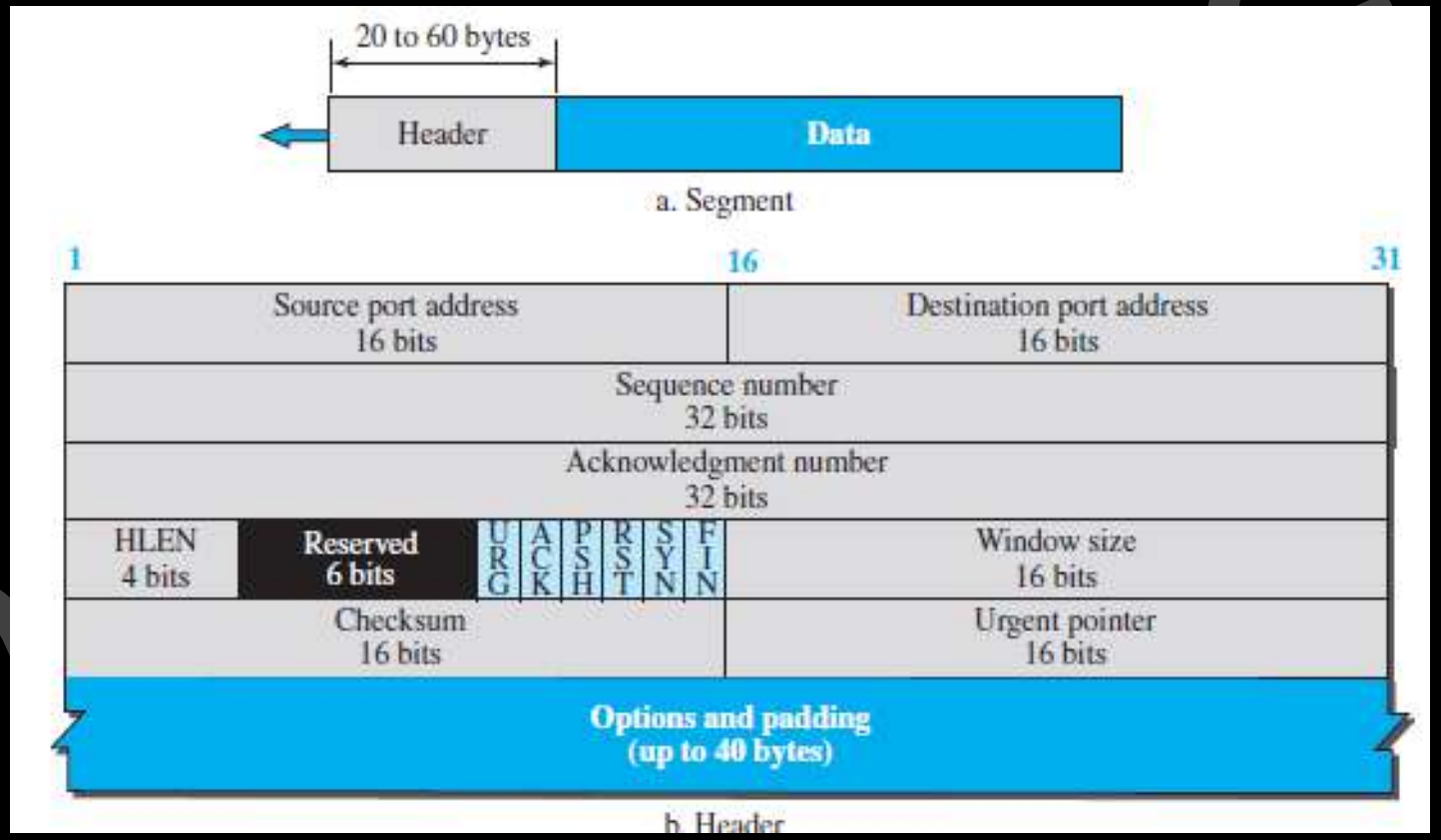


Flag	*Description*
URG	The value of the urgent pointer field is valid.
ACK	The value of the acknowledgment field is valid.
PSH	Push the data.
RST	Reset the connection.
SYN	Synchronize sequence numbers during connection.
FIN	Terminate the connection.

Knowledgegate

Urgent pointer

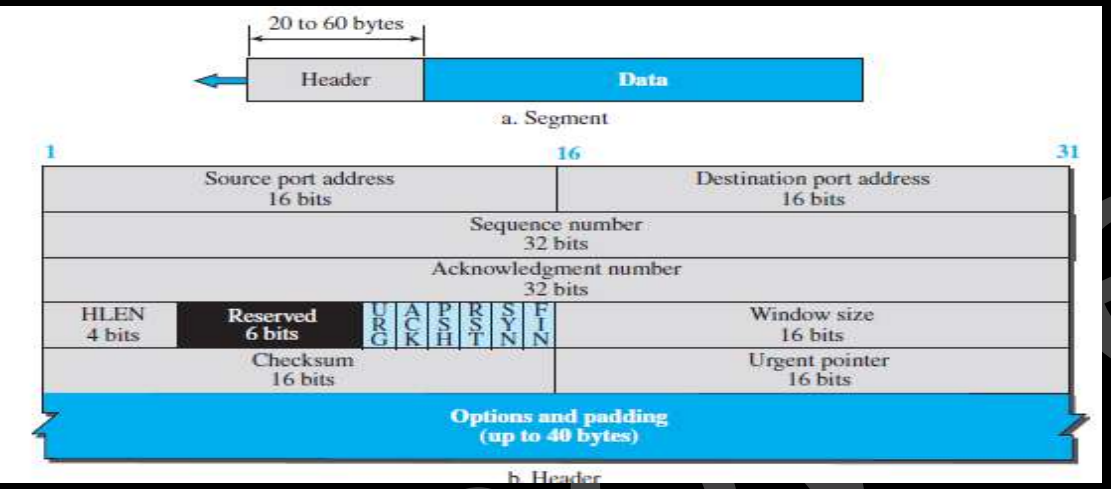
- **Urgent pointer:** This 16-bit field, which is valid only if the urgent flag is set, is used when the segment contains urgent data. It defines the number that must be added to the sequence number to obtain the number of the last urgent byte in the data section of the segment.



www.knowledgegate.in

PUSH Flag

- **Push (PSH) –** Transport layer by default waits for some time for application layer to send enough data equal to maximum segment size so that the number of packets transmitted on network minimizes which is not desirable by some application like interactive applications(chatting).
- This problem is solved by using PSH. Transport layer sets PSH = 1 and immediately sends the segment to network layer as soon as it receives signal from application layer. Receiver transport layer, on seeing PSH = 1 immediately forwards the data to application layer. In general, it tells the receiver to process these packets as they are received instead of buffering them.



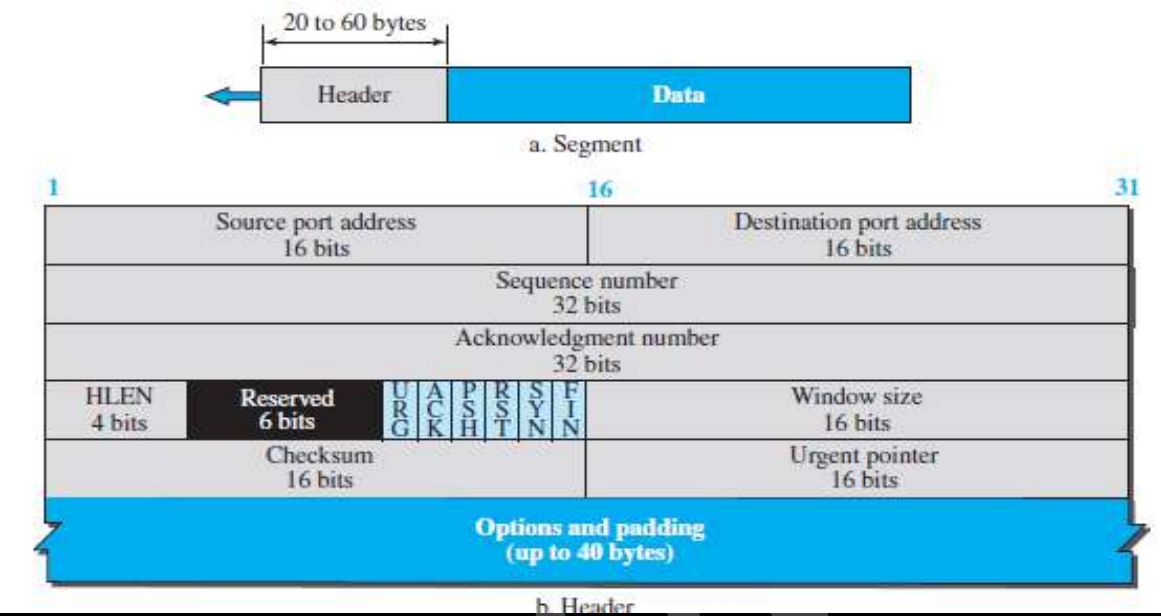
Flag	*Description*
URG	The value of the urgent pointer field is valid.
ACK	The value of the acknowledgment field is valid.
PSH	Push the data.
RST	Reset the connection.
SYN	Synchronize sequence numbers during connection.
FIN	Terminate the connection.

KnowledgeGate

RST Flag

- **Reset (RST) –** It is used to terminate the connection if the sender or receiver feels something is wrong with the TCP connection or that the conversation should not exist.

- It can get send from receiver side when packet is send to particular host that was not expecting it.

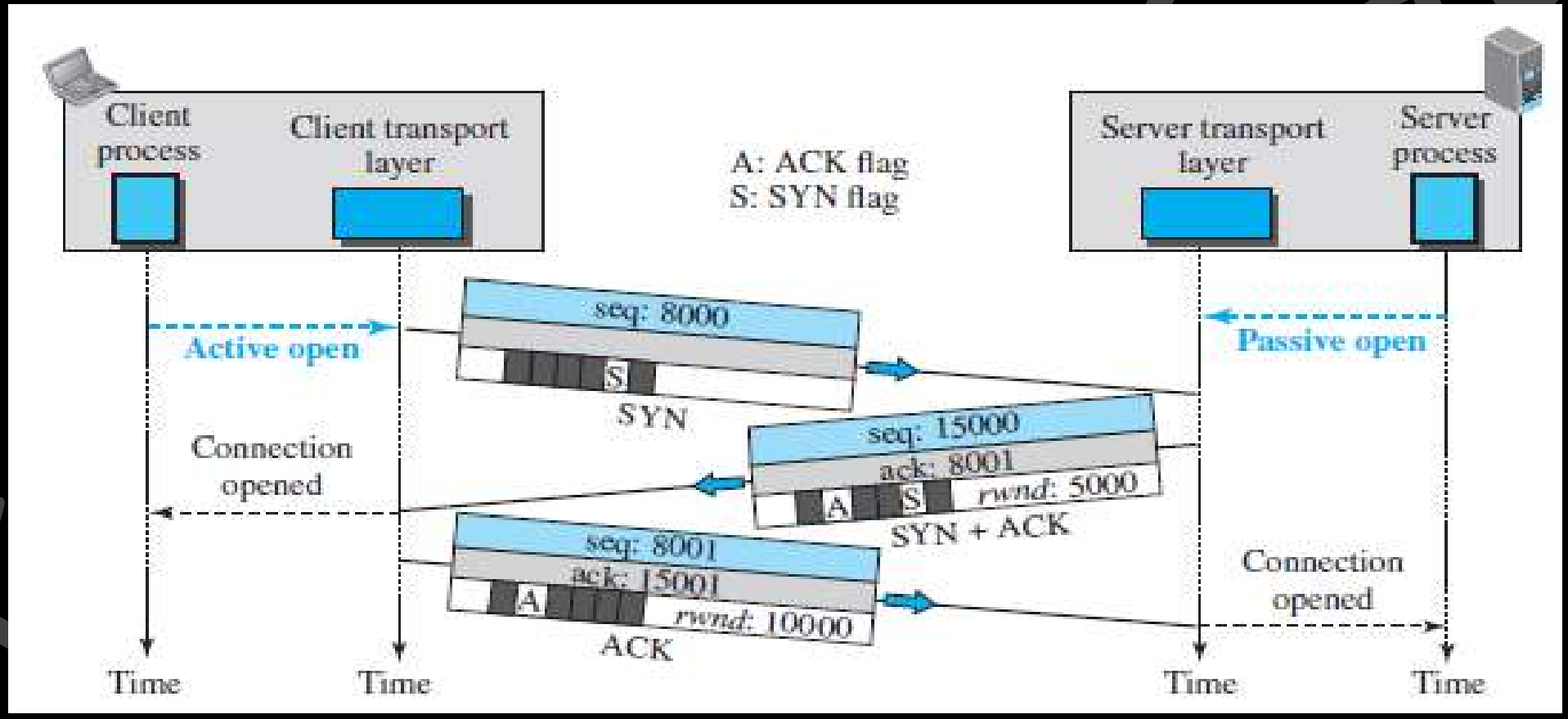


Flag	*Description*
URG	The value of the urgent pointer field is valid.
ACK	The value of the acknowledgment field is valid.
PSH	Push the data.
RST	Reset the connection.
SYN	Synchronize sequence numbers during connection.
FIN	Terminate the connection.

www.knowledgegate.in

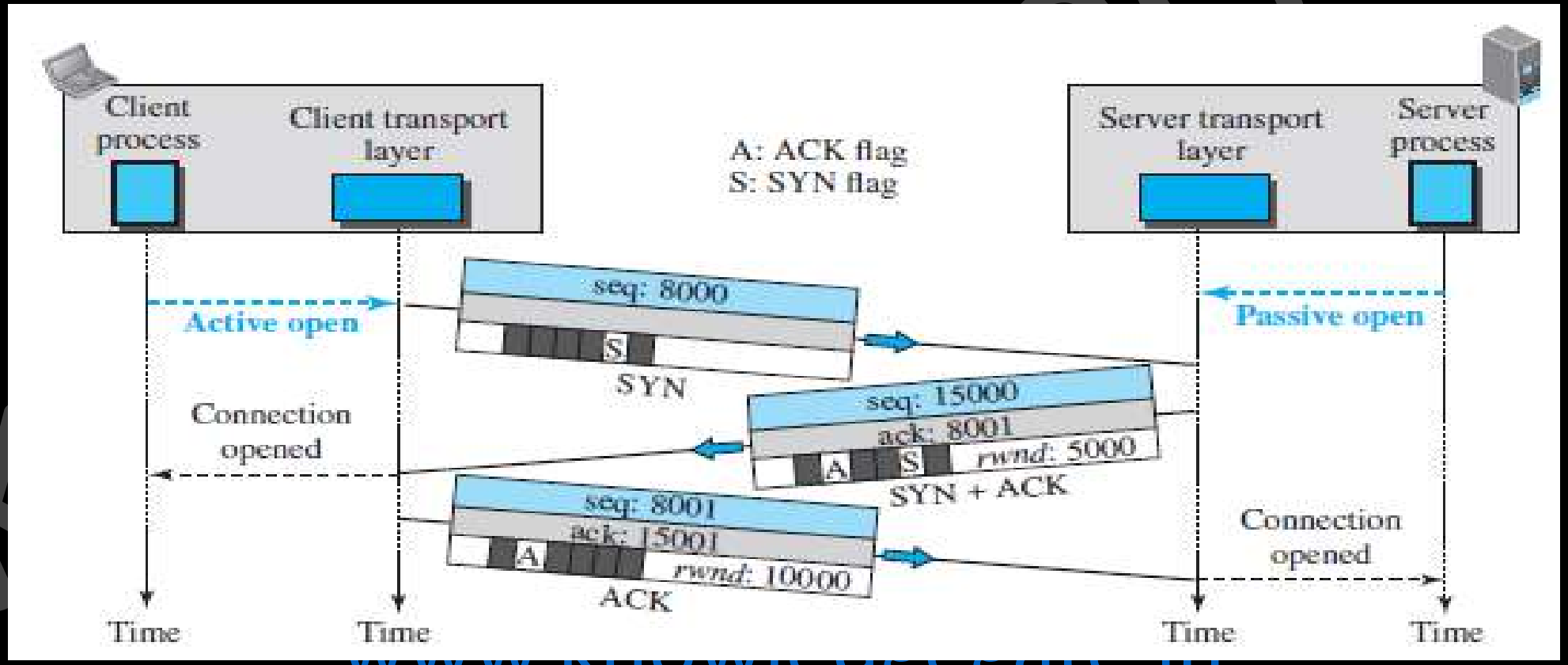
A TCP Connection

- The connection establishment in TCP is called ***three-way handshaking.***
- The client program issues a request for an *active open*. A client that wishes to connect to an open server tells its TCP to connect to a particular server. TCP can now start the three-way handshaking process.



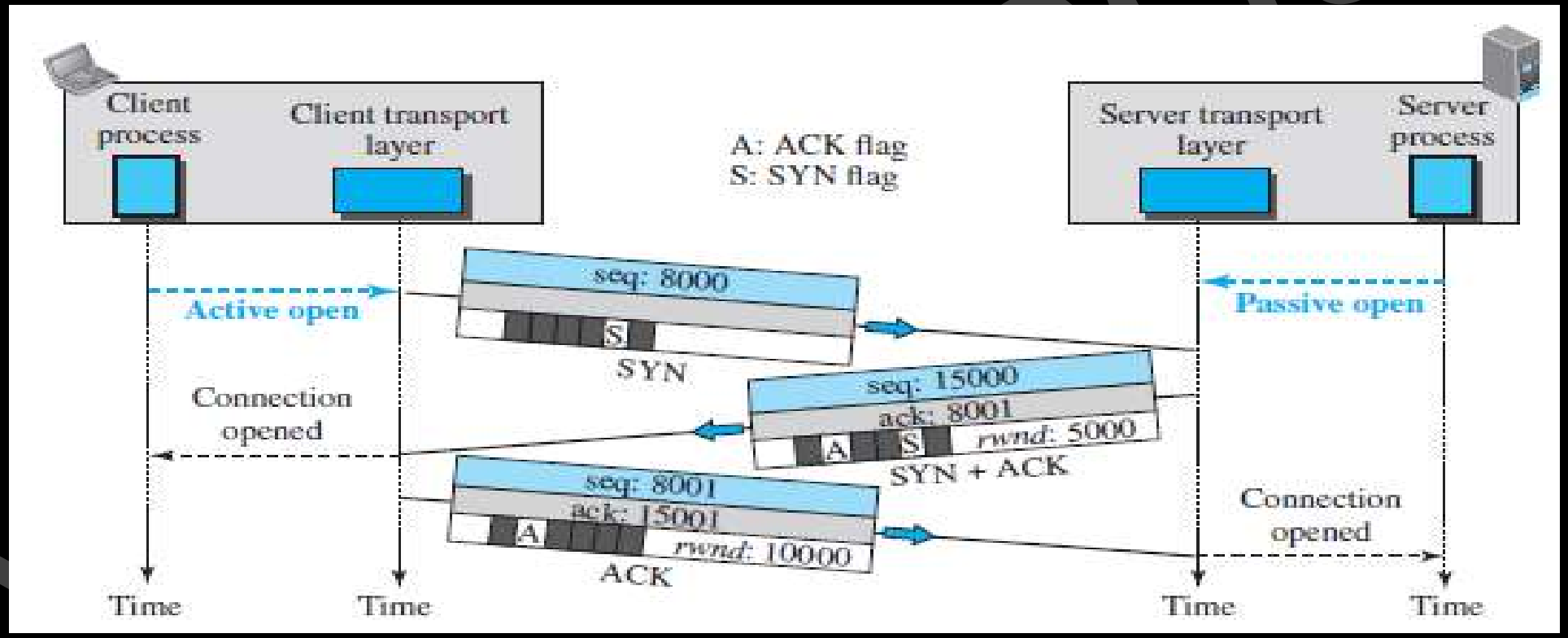
www.knowledgegate.in

- The server sends the second segment, a SYN + ACK segment with two flag bits set as: SYN and ACK.
  - It is a SYN segment for communication in the other direction.
  - The server uses this segment to initialize a sequence number for numbering the bytes sent from the server to the client.
  - The server also acknowledges the receipt of the SYN segment from the client by setting the ACK flag and displaying the next sequence number it expects to receive from the client.
  - A SYN + ACK segment cannot carry data, but it does consume one sequence number.



www.knowledgegate.in

- The client sends the third segment.
  - This is just an ACK segment.
  - It acknowledges the receipt of the second segment with the ACK flag and acknowledgment number field.
- ACK segment does not consume any sequence numbers if it does not carry data.
- After connection is established, bidirectional data transfer can take place.



www.knowledgegate.in

USER DATAGRAM PROTOCOL (UDP)

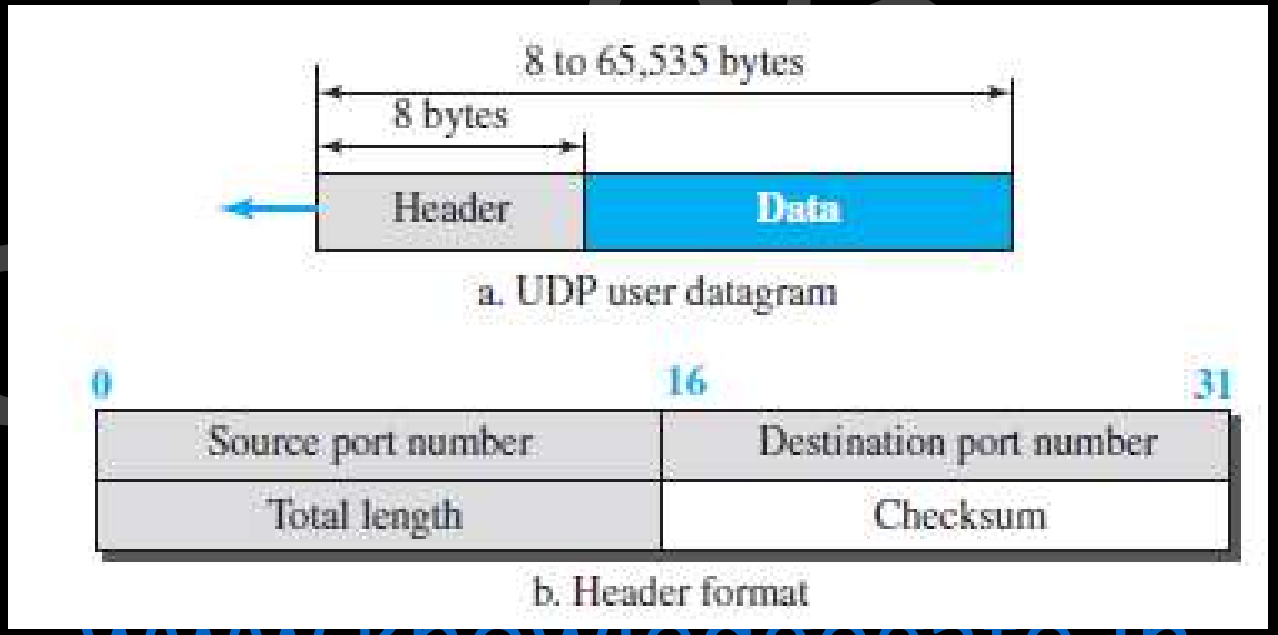
- The **User Datagram Protocol (UDP)** is a connectionless, unreliable transport protocol.
- It does not add anything to the services of IP except for providing process-to-process communication instead of host-to-host communication.

- **Why to use UDP**
  - UDP is a very simple protocol using a minimum of overhead.
  - If a process wants to send a small message and does not care much about reliability, it can use UDP.
  - Sending a small message using UDP takes much less interaction between the sender and receiver than using TCP.

KnowledgeGate

User Datagram

- UDP packets, called ***user datagrams,*** have a fixed-size header of 8 bytes made of four fields, each of 2 bytes (16 bits).
- The first two fields define the source and destination port numbers.
- The third field defines the total length of the user datagram, header plus data.
- The 16 bits can define a total length of 0 to 65,535 bytes. However, the total length needs to be less because a UDP user datagram is stored in an IP datagram with the total length of 65,535 bytes.
- The last field can carry the optional checksum



www.knowledgegate.in

UDP Services

- ***Process-to-Process Communication***
  - UDP provides process-to-process communication using **socket addresses,** a combination of IP addresses and port numbers.
- ***Connectionless Services***
  - Each user datagram sent by UDP is an independent datagram. There is no relationship between the different user datagrams. The user datagrams are not numbered. There is no connection establishment and no connection termination unlike TCP.
- ***Flow Control***
  - There is no *flow control,* and hence no window mechanism.
- ***Error Control***
  - There is no *error control* mechanism in UDP except for the checksum.
- ***Congestion Control***
  - It does not provide congestion control.

KnowledgeGate

UDP Applications

- UDP is suitable for a process with internal flow- and error-control mechanisms. For example, the ***Trivial File Transfer Protocol (TFTP)*** process includes flow and error control. It can easily use UDP.
- UDP is a suitable transport protocol for multicasting. Multicasting capability is embedded in the UDP software but not in the TCP software.
- UDP is used for management processes such as SNMP
- UDP is used for some route updating protocols such as Routing Information Protocol (RIP)
- UDP is normally used for interactive real-time applications that cannot tolerate uneven delay between sections of a received message.
- DNS,

KnowledgeGate

S.No.	TCP	RTP
1	TCP stands for transmission control protocol.	RTP stand for Real Time Transport Protocol.
2	It is a lossless protocol.	RTP is a stateless protocol.
3	It is a slow process.	It is a faster than TCP.
4	It cannot tolerate packet loss.	It can tolerate packet loss.
5	TCP is not generally used for ‘real-time’ streaming.	RTP is used for ‘real-time’ streaming.

KnowledgeGate
www.knowledgegate.in

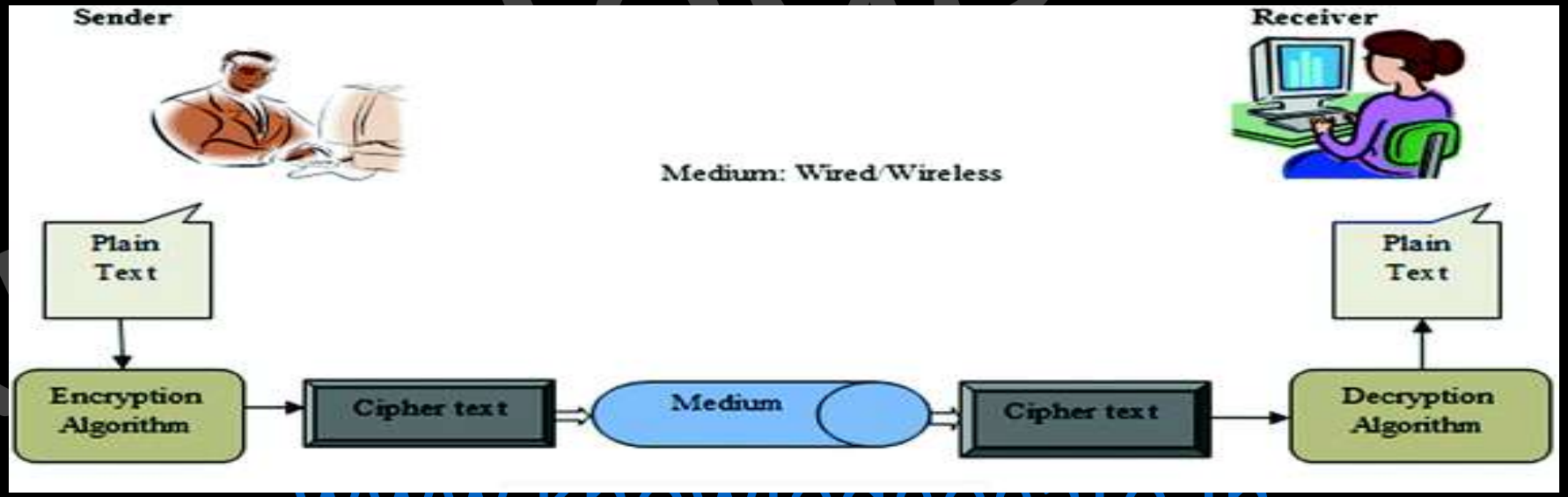
- Data compression :
  - Data compression is the way of downloading the compressed form of text, audio and video data using the computer.
  - Data compression is essential for efficient storage and transmission of different type of data.
  - A data compression system consists of an encoder and a decoder.
  - The encoder performs compression of the incoming data and decoder is used for decompression and reconstruction.
- Types of compression :
  - Lossless compression (or data compaction) :
    - In lossless compression, the redundant information contained in the data is removed.
    - In lossless compression, there is no loss of information.
    - Lossless compression has lower compression ratio.
  - Lossy compression :
    - In lossy compression, there is a loss of information in a controlled manner.
    - The lossy compression is not completely reversible.
    - The lossy compression has higher compression ratio.

KnowledgeGate

www.knowledgegate.in

Cryptography

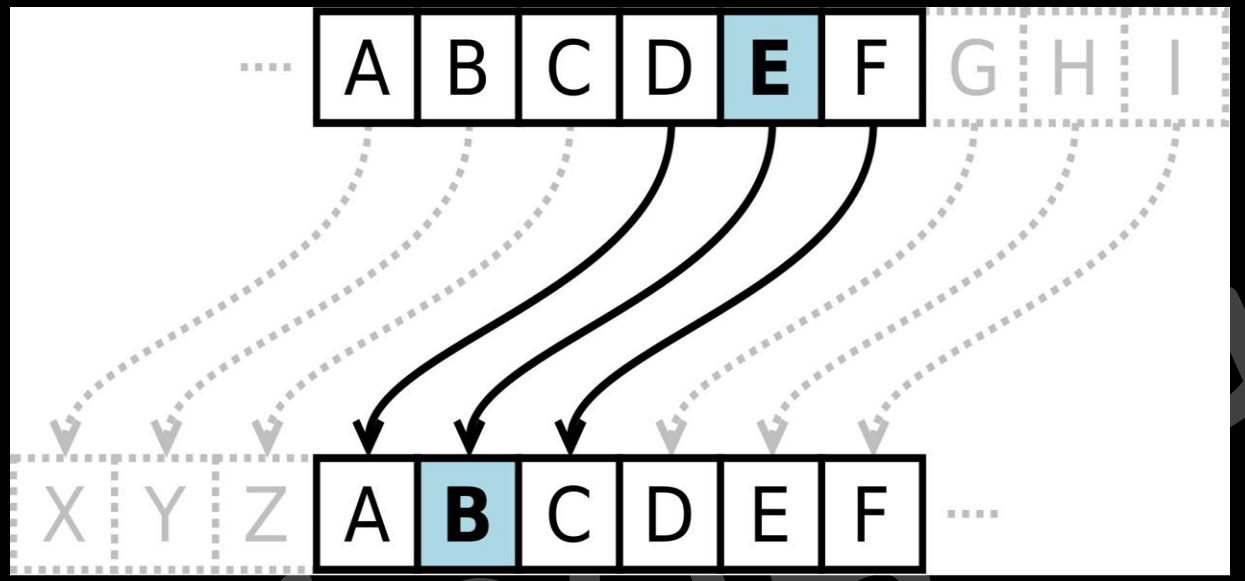
- **Cryptography** is the practice and study of techniques for secure communication in the presence of third parties called adversaries.
- More generally, cryptography is about constructing and analysing protocols that prevent third parties or the public from reading private messages; various aspects in information security such as data confidentiality, data integrity, authentication, and non-repudiation are central to modern cryptography.
- Applications of cryptography include electronic commerce, chip-based payment cards, digital currencies, computer passwords, and military communications



KnowledgeGate

www.knowledgegate.in

- Alphabet shift ciphers are believed to have been used by Julius Caesar over 2,000 years ago.

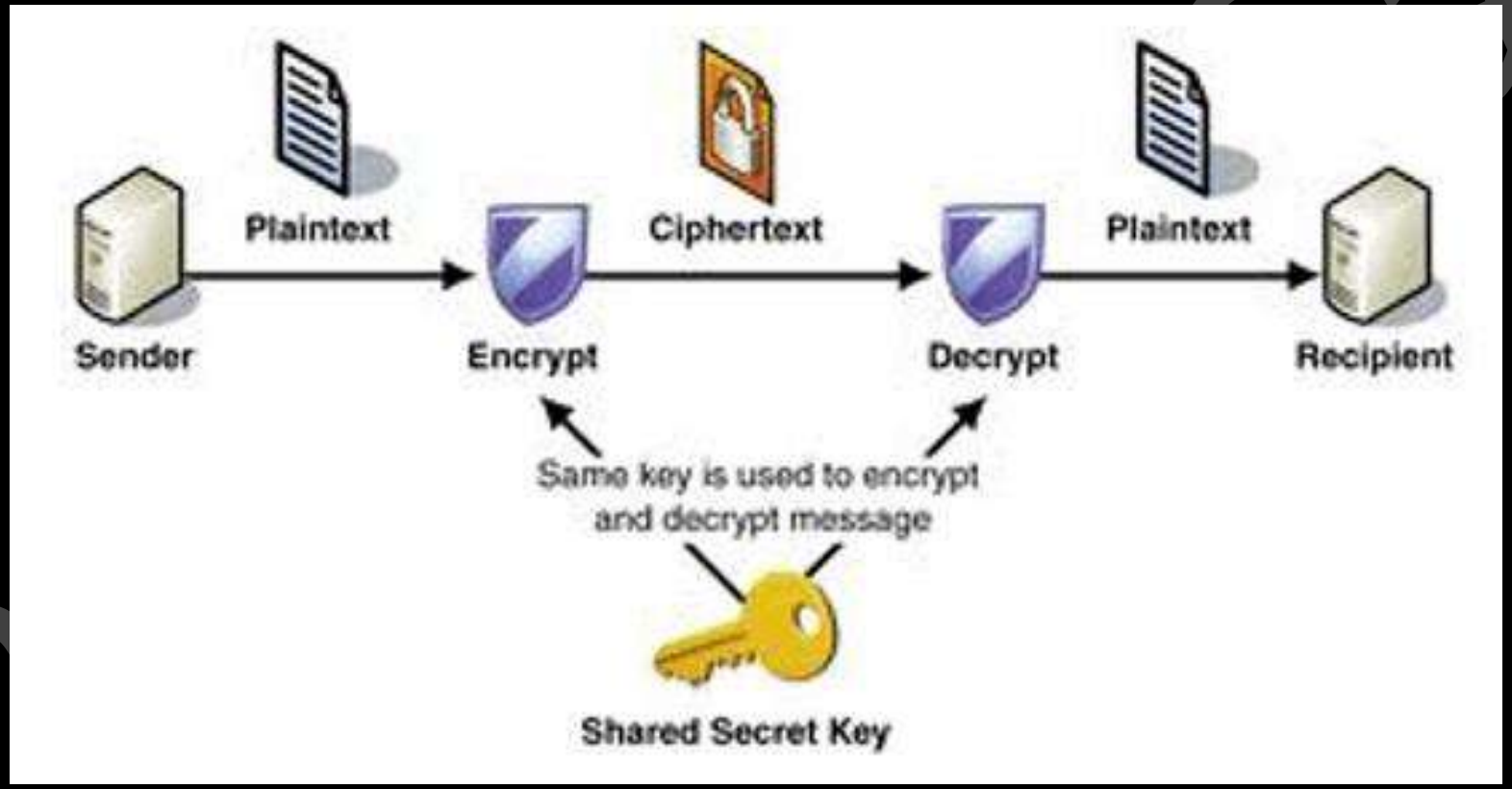


Knowledgegate

www.knowledgegate.in

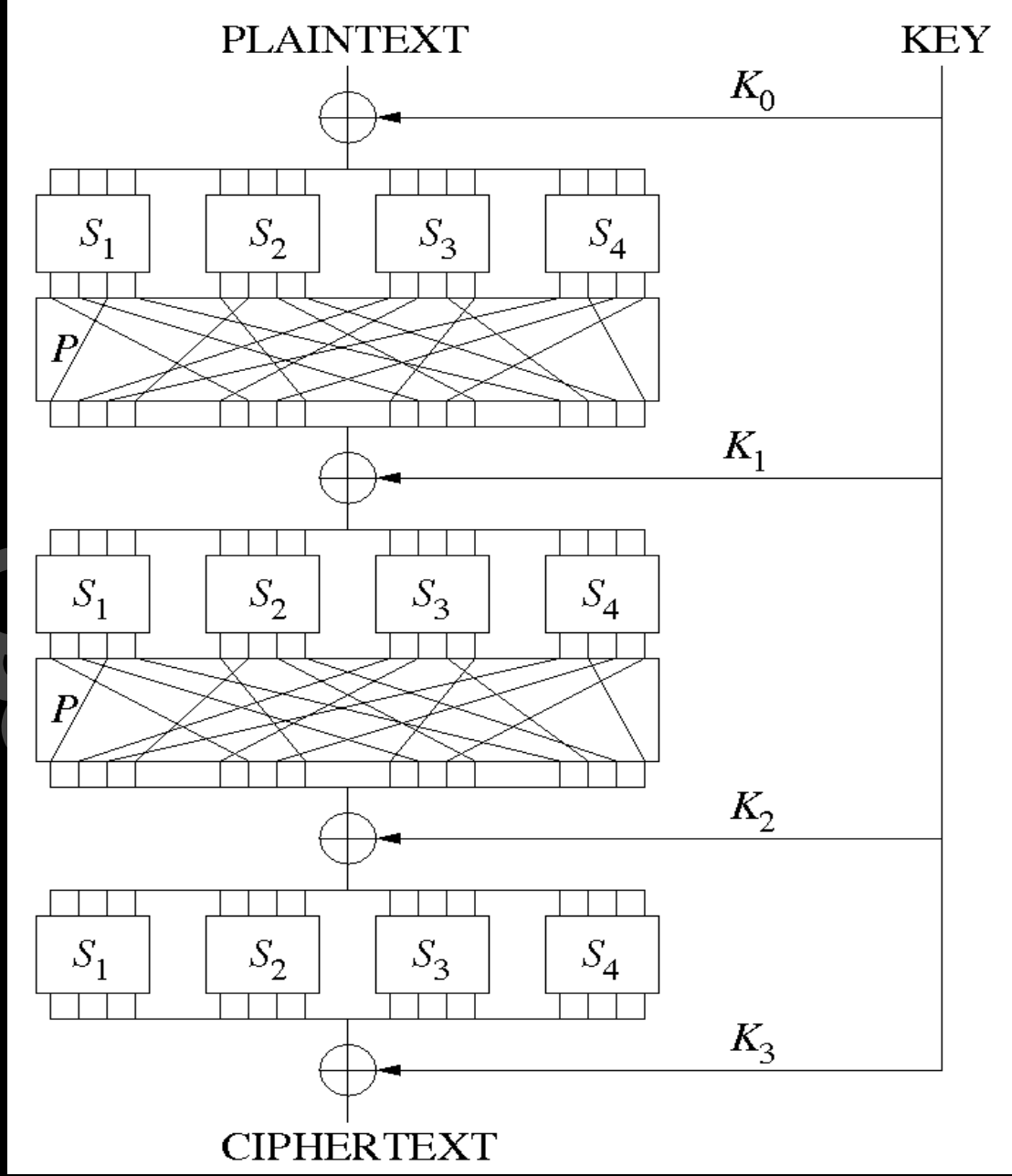
Symmetric key

- Symmetric-key cryptography refers to encryption methods in which both the sender and receiver share the same key (or, less commonly, in which their keys are different, but related in an easily computable way). This was the only kind of encryption publicly known until June 1976.



www.knowledgegate.in

- The Data Encryption Standard (DES) and the Advanced Encryption Standard (AES) are block cipher designs that have been designated cryptography standards by the US government (though DES's designation was finally withdrawn after the AES was adopted).
- Despite its deprecation as an official standard, DES (especially its still-approved and much more secure triple-DES variant) remains quite popular; it is used across a wide range of applications, from ATM encryption to e-mail privacy and secure remote access.



Knowledg

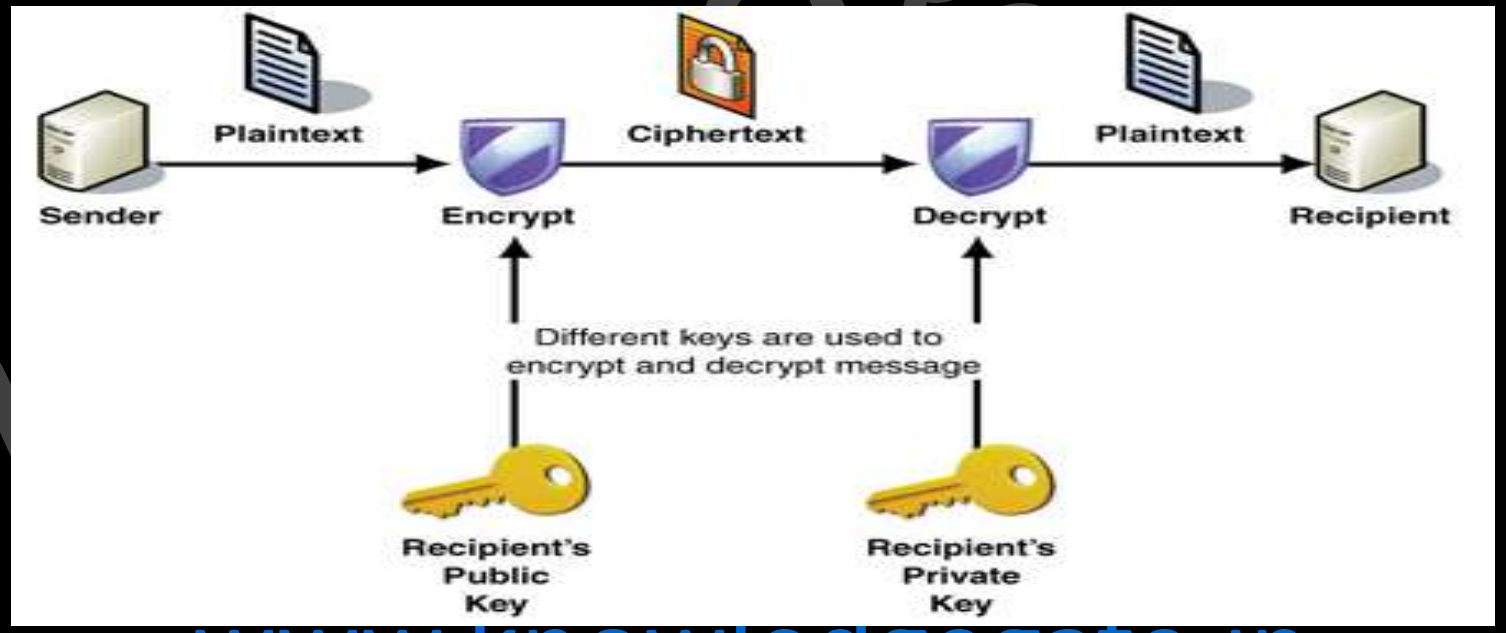
www.knowledgegate.in

- The data encryption standard(DES) is a block cipher that used shared secret encryption.
- DES is based on a symmetric key algorithm that uses a 56-bit key.
- DES is basically a mono-alphabetic substitution cipher using a 64-bit character.
- Whenever the same 64-bit plaintext block goes in, the same 64-bit ciphertext block comes out.
- Working of DES involves the following stages:
  - The first stage is a key independent transposition on the 64-bit plaintext.
  - The last stage is the exact inverse, before that is an exchange of the leftmost with the rightmost 32 bits.
  - The remaining 16 stages are functionally identical but are parameterized by different functions of the key.
  - The left output of an iteration stage is simply a copy of the right input. The right output is the exclusive OR of the left input and a function of the right input and the key for this iteration. All the complexity lies in this functions which consists of four sequential steps.

KnowledgeGate

Asymmetric key (Public-key cryptography)

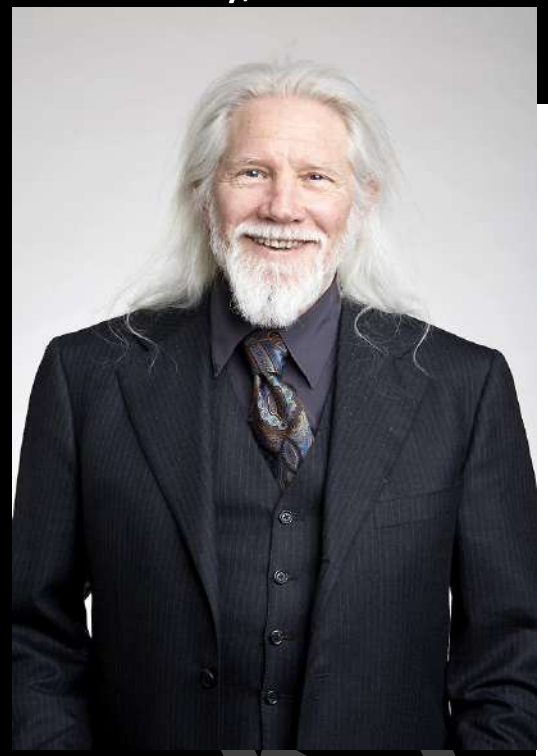
- Symmetric-key cryptosystems use the same key for encryption and decryption of a message, although a message or group of messages can have a different key than others.
- A significant disadvantage of symmetric ciphers is the key management necessary to use them securely. Each distinct pair of communicating parties must, ideally, share a different key, and perhaps for each ciphertext exchanged as well.
- The number of keys required increases as the square of the number of network members, which very quickly requires complex key management schemes to keep them all consistent and secret.



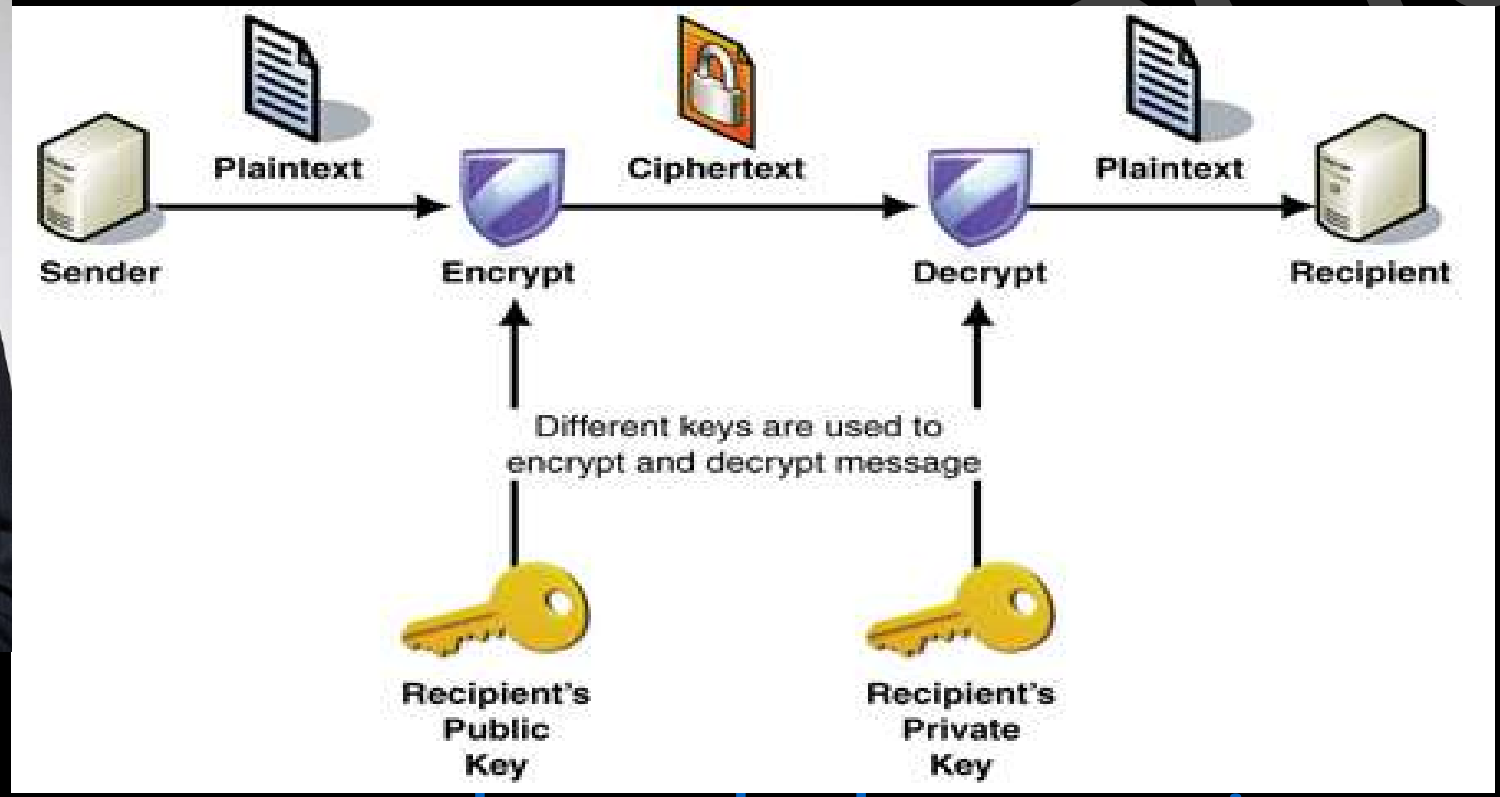
KnowledgeGate

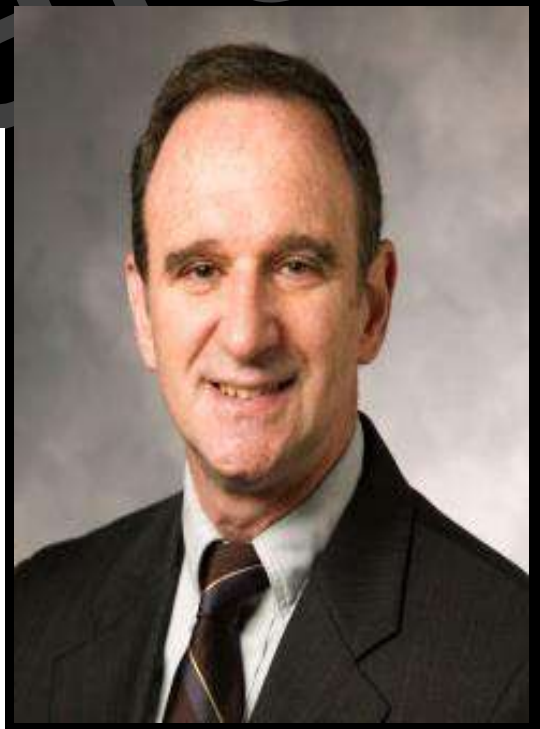
www.knowledgegate.in

- In a ground breaking 1976 paper, Whitfield Diffie and Martin Hellman proposed the notion of *public-key* (also, more generally, called *asymmetric key*) cryptography in which two different but mathematically related keys are used—a *public* key and a *private* key.
- A public key system is so constructed that calculation of one key (the 'private key') is computationally infeasible from the other (the 'public key'), even though they are necessarily related. Instead, both keys are generated secretly, as an interrelated pair.



Whitfield Diffie





Martin Hellman

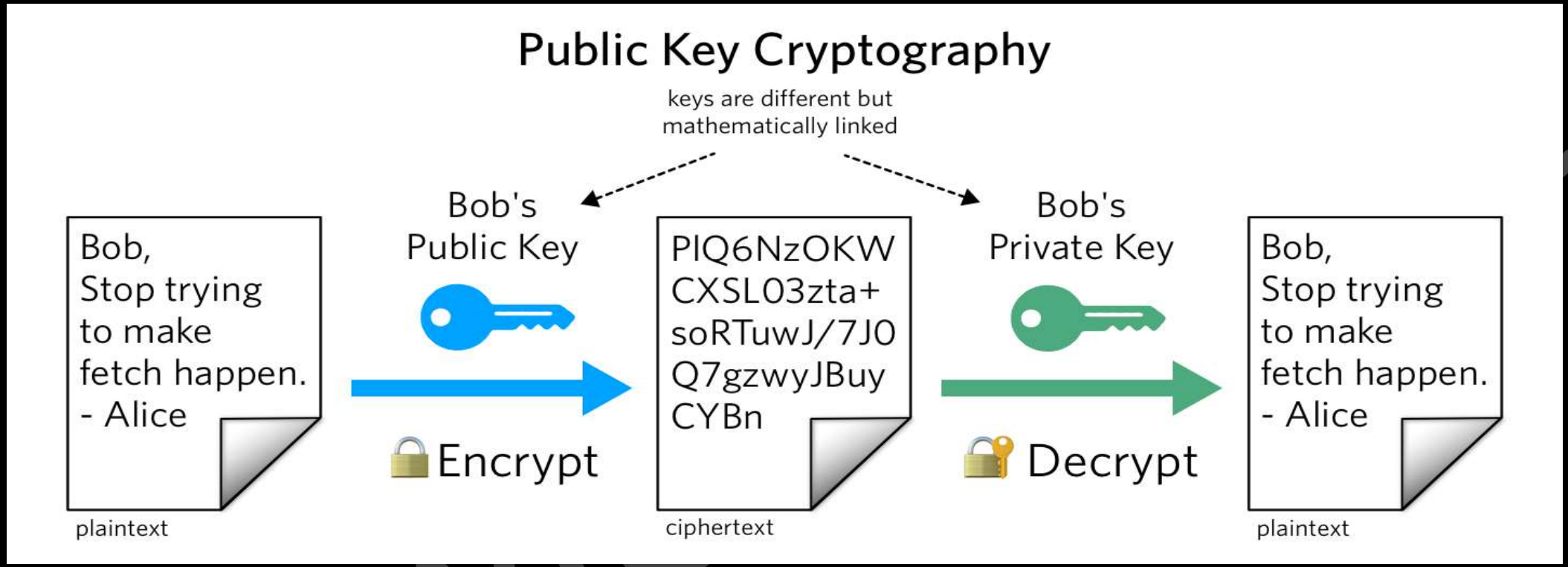
www.knowledgegate.in

- Diffie and Hellman's publication sparked widespread academic efforts in finding a practical public-key encryption system. This race was finally won in 1978 by Ronald Rivest, Adi Shamir, and Len Adleman, whose solution has since become known as the RSA algorithm.

KnowledgeGate

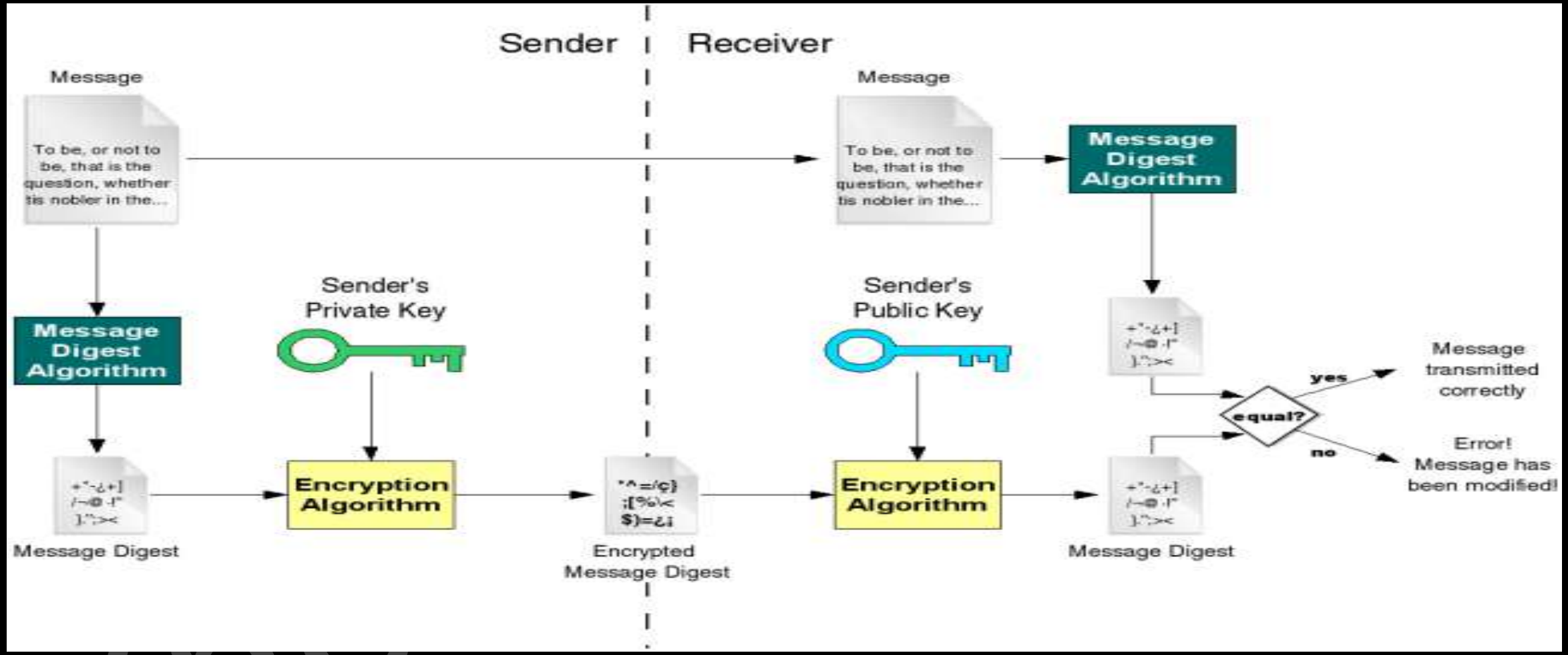
www.knowledgegate.in

Confidentiality



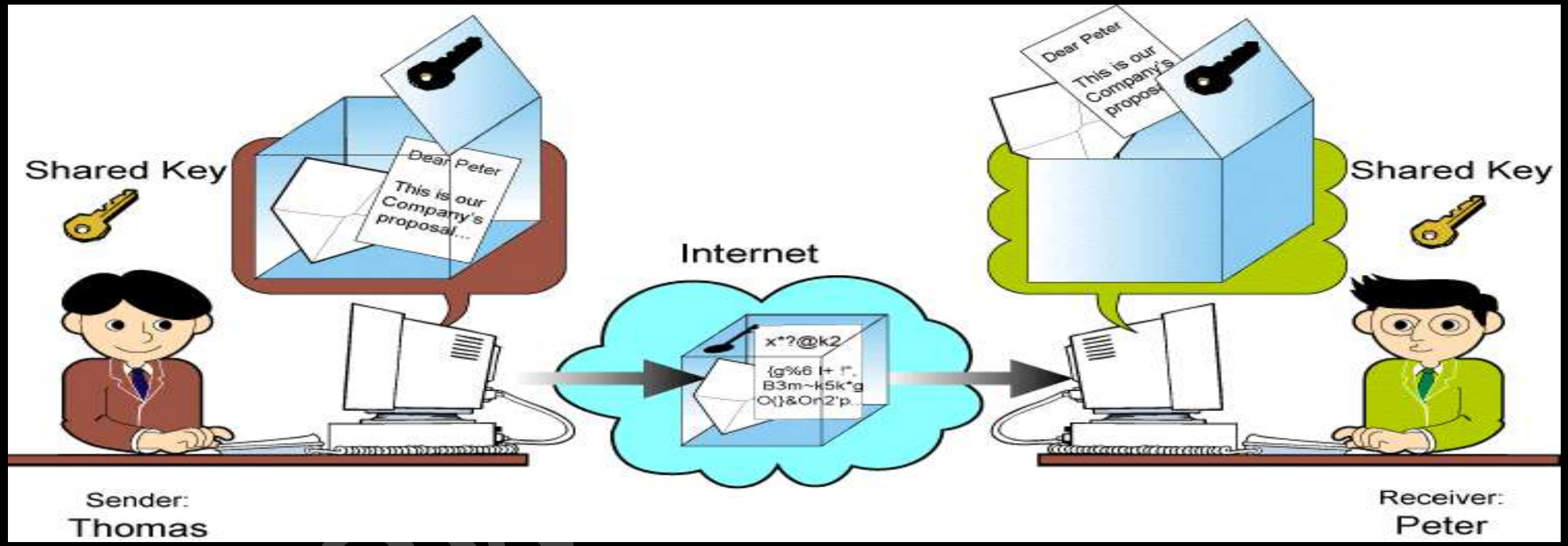
Knowledgegate

Authentication



www.knowledgegate.in

Authentication Confidentiality



www.knowledgegate.in

RSA Algorithm

1. Bob chooses two large numbers, p and q, and calculates n = p × q and 0 = (p - 1) × (g - 1).
2. Bob then selects e and d such that (e x d) mod 0 = 1.
3. Bob advertises e and n to the community as the public key; Bob keeps d as the private key.
4. Anyone, including Alice, can encrypt a message and send the ciphertext to Bob, using $C = (P^e)$ mod n; only Bob can decrypt the message, using $P = (C^d)$ mod n.
5. An intruder such as Eve cannot decrypt the message if p and q are very large numbers (she does not know d).

KnowledgeGate

For the sake of demonstration, let Bob choose 7 and 11 as p and q and calculate $n = 7 \times 11 = 77$. The value of $\phi(n) = (7 - 1)(11 - 1)$, or 60. If he chooses e to be 13, then d is 37. Note that $e \times d \bmod 60 = 1$. Now imagine that Alice wants to send the plaintext 5 to Bob. She uses the public exponent 13 to encrypt 5. This system is not safe because p and q are small.

KnowledgeGate
www.knowledgegate.in

Plaintext: 5

$C = 5^{13} = 26 \bmod 77$

Ciphertext: 26

Ciphertext: 26

$P = 26^{37} = 5 \bmod 77$

Plaintext: 5

KnowledgeGate

www.knowledgegate.in

Here is a more realistic example calculated using a computer program in Java. We choose a 512-bit p and q, calculate n and $\phi(n)$. We then choose e and calculate d. Finally, we show the results of encryption and decryption. The integer p is a 159-digit number.

$p =$ 961303453135835045741915812806154279093098455949962158225831508796479404550564706384912571601803475031209866660649242019180878066742109606335421992666120 9

The integer q is a 160-digit number.

$q =$ 120601919572314469182767942044508960015559250546370339360617983217314821484837646592153894532091752252732268301071206956046025138871455249690003596600456 17

The modulus $n = p \times q$. It has 309 digits.

$n =$ 115935041739676149688925098646158875237714573754541447754855261376147885408326350817276878815968325168468849300625485764111250162414552339182927162507656772727460097082714127730434960500556347274566628060099924037102991424472292215772798531727033839381334692684137327622000966676671831831088373420823444370953

$\phi(n) = (p - 1)(q - 1)$ has 309 digits.

Bob chooses $e = 35535$ (the ideal is 65537). He then finds d.

www.knowledgegate.in

$\phi(n) =$	11593504173967614968892509864615887523771457375454144775485526137614788540832635081727687881596832516846884930062548576411125016241455233918292716250765675105423360849291675203448262798811755478765701392344440571698958172819609822636107546721186461217135910735864061400888517026537727726446734106624385766412 8
$e =$	35535
$d =$	58008302860037763936093661289677917594669062089650962180422866111380593852822358731706286910030021710859044338402170729869087600611530620252495988444804756824096624708148581713046324064407770483313401085094738529564507193677406119732655742423721761767462077637164207600337085333288532144708859551366702948 31

Alice wants to send the message "THIS IS A TEST", which can be changed to a numeric value using the 00–26 encoding scheme (26 is the *space* character).

The ciphertext calculated by Alice is $C = P^e$, which is shown below.

P =	19070818260818260026190418 19
C =	47530912364622682720636555061054518094237179607049171652323924305445296061319932856661784341835911415119741125200568297979457173603610127821884789274156609048002350719071527718591497518846588863210114835410336165789846796838676373376577746562507928052114814184404814184430812773059004692874248559166462108656

Bob can recover the plaintext from the ciphertext using $P = C^d$, which is shown below.

P =	19070818260818260026190418 19

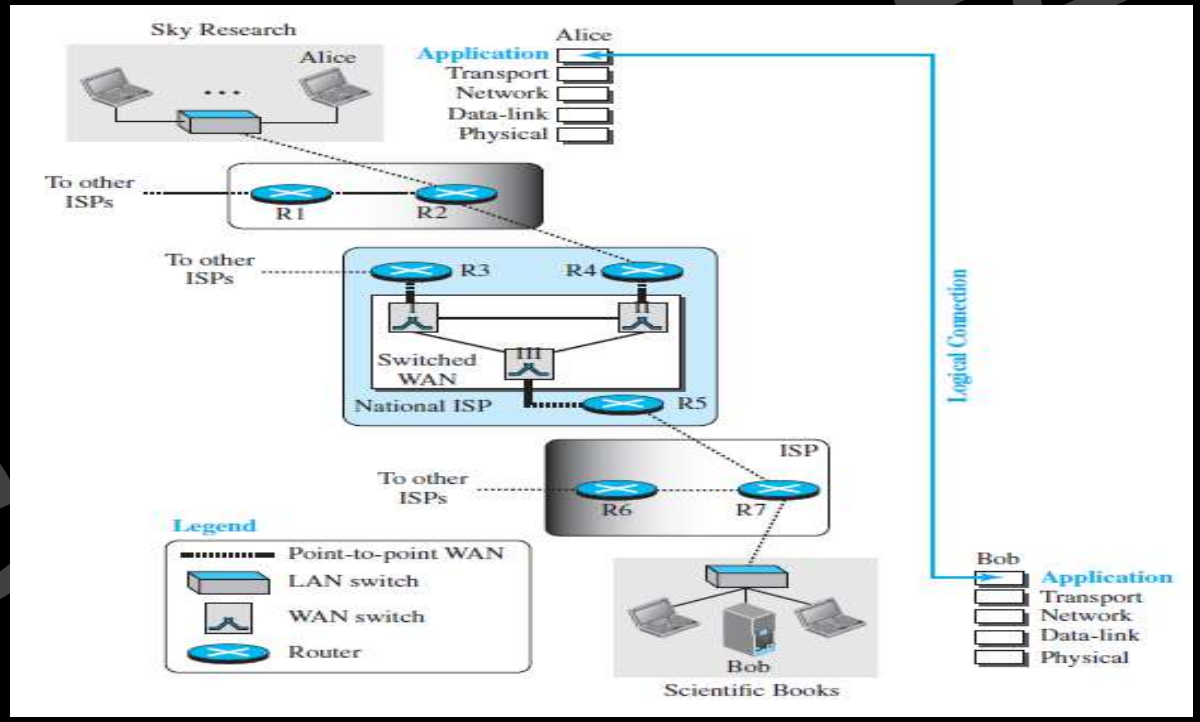
The recovered plaintext is "THIS IS A TEST" after decoding.

S.No.	Block Cipher	Transposition Cipher
1	Block cipher A block cipher is one in which a block of plaintext is treated as a whole and used to produce a ciphertext block of equal length.	Transposition cipher Transposition cipher is the cipher in which the plaintext is written -down as a sequence of diagonals and then read off as a sequence of rows.
2	Errors in transmitting one block generally do not affect other blocks.	Error in one letter will affect the whole ciphertext.
3	Encryption process is slow.	Encryption process is fast.
4	Security of block cipher depends on the design of encryption function.	Transposition cipher can be made more secure by performing more than one transposition.
5	Algorithm breaks the plaintext into blocks and operates on each block independently.	Algorithm breaks the plaintext into letters and operates on each letter independently.

KnowledgeGate
www.knowledgegate.in

Application Layer

- The application layer enables the user, whether human or software, to access the network. It provides user interfaces and responsible for providing services to the user such as
  - Electronic mail
  - File access and transfer
  - Access to system resources
  - Surfing the world wide web
  - Network management.



- Communication between two application layers happens over a logical connection. This means both layers act like they're directly connected for message exchange. To use the Internet, you need two application programs: one on one computer and another on a different computer. Both interact to send and receive data.

www.knowledgegate.in

ELECTRONIC MAIL

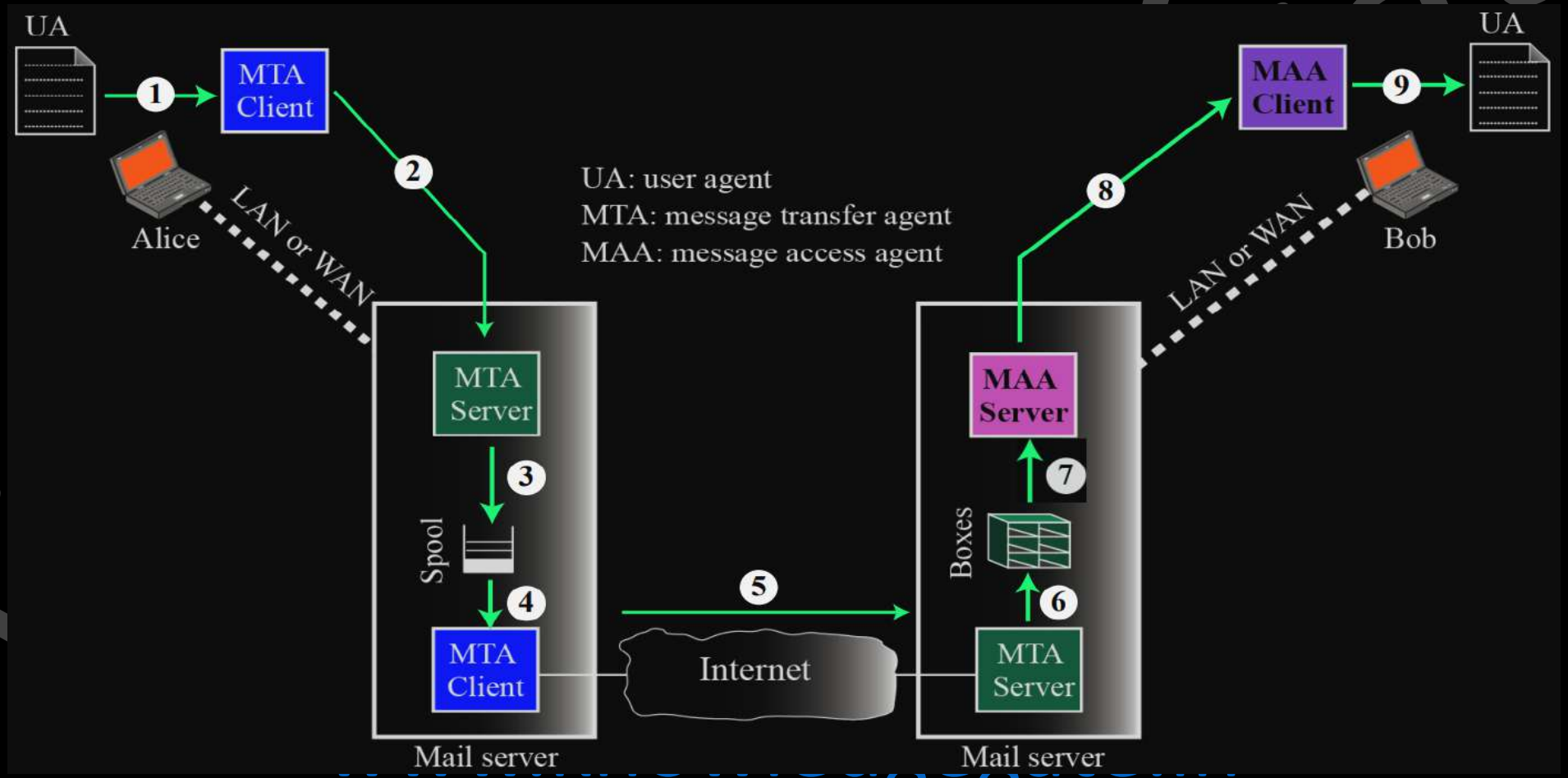
- E-mail has evolved significantly since the early days of the Internet. Originally designed for simple text-based communication, it was essentially a way to send digital memos between users. The creators of this system likely couldn't have foreseen just how versatile and popular e-mail would become.
  - **Multimedia**: Modern e-mails can include not just text, but also images, audio, and video attachments.
  - **Multiple Recipients**: E-mails can be sent to multiple recipients at once, with features like CC (carbon copy) and BCC (blind carbon copy) for added flexibility.
  - **Hyperlinks and HTML**: E-mails now often include hyperlinks and even HTML content, allowing for richly formatted messages.
  - **Encryption and Security**: Advanced security protocols like SSL/TLS are often used to encrypt e-mails for secure transmission.
  - **Filters and Folders**: Advanced organizational tools let you filter and sort e-mails into various folders, helping manage the influx of messages.
  - **Integration**: E-mails are now often integrated with other services such as calendars, task management tools, and even AI-based assistants to help manage appointments and reminders.

KnowledgeGate

Message Transfer Agent: SMTP

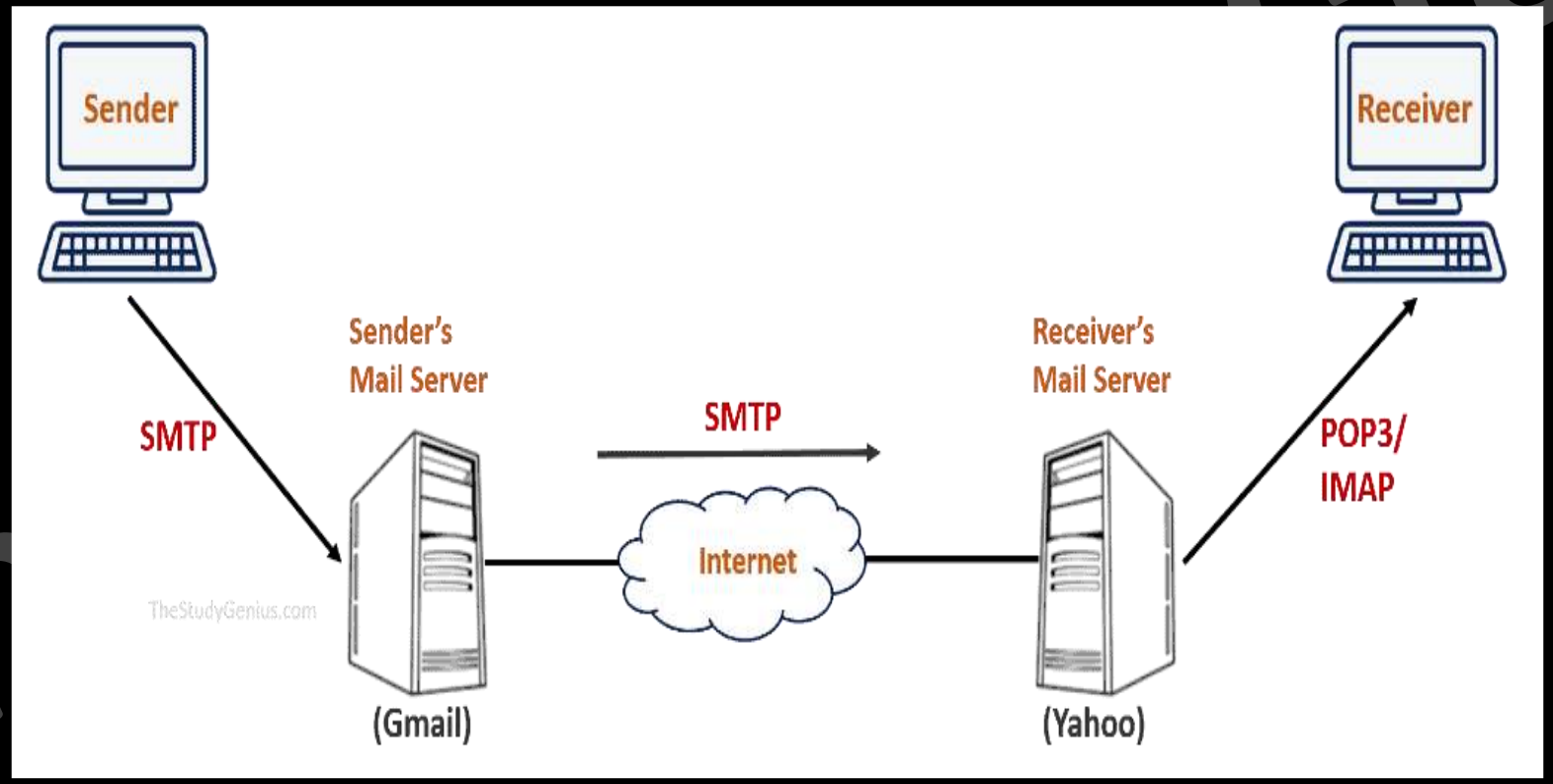
- The actual mail transfer is done through message transfer agents. To send mail, a system must have the client MTA, and to receive mail, a system must have a server MTA.

- The formal protocol that defines the MTA client and server in the Internet is called the Simple Mail Transfer Protocol (SMTP).



- SMTP is used two times, between the sender and the sender's mail server and between the two mail servers

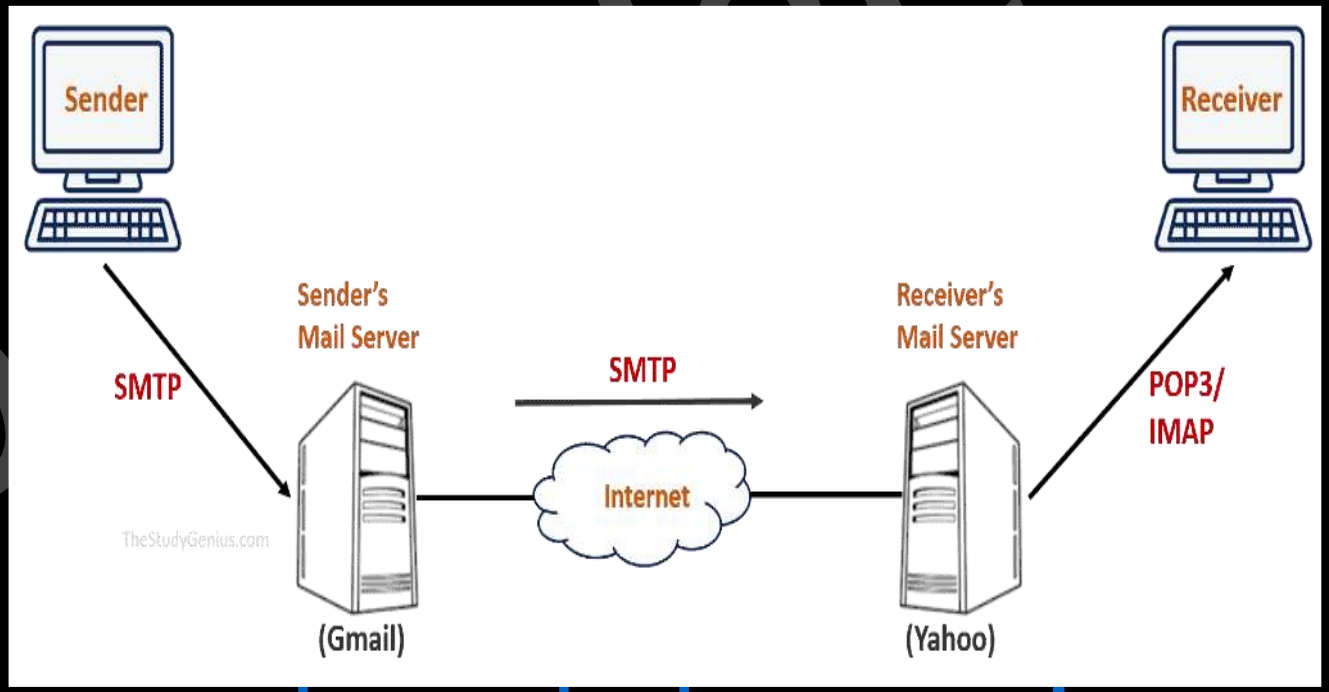
- SMTP simply defines how commands and responses must be sent back and forth.



www.knowledgegate.in

Message Access Agent: POP and IMAP

- The first and the second stages of mail delivery use SMTP. However, SMTP is not involved in the third stage because SMTP is a push protocol; it pushes the message from the client to the server.
- On the other hand, the third stage needs a pull protocol; the client must pull messages from the server.
- The third stage uses a message access agent. Currently two message access protocols are available: Post Office Protocol, version 3 (POP_3) and Internet Mail Access Protocol, version 4 ($IMAP_4$).



www.knowledgegate.in

POP3

- Post Office Protocol, version 3 (POP3) is simple and limited in functionality. The client POP3 software is installed on the recipient computer; the server POP3 software is installed on the mail server.

- Mail access starts with the client when the user needs to download e-mail from the mailbox on the mail server. The client opens a connection to the server on TCP port 110. It then sends its user name and password to access the mailbox.

- The user can then list and retrieve the mail messages, one by one. POP3 has two modes: the delete mode and the keep mode. In the delete mode, the mail is deleted from the mailbox after each retrieval. In the keep mode, the mail remains in the mailbox after retrieval.

- The delete mode is normally used when the user is working at her permanent computer and can save and organize the received mail after reading or replying.

- The keep mode is normally used when the user accesses her mail away from her primary computer (e.g., a laptop). The mail is read but kept in the system for later retrieval and organizing.

KnowledgeGate

IMAP4

- Another mail access protocol is Internet Mail Access Protocol, version 4 (IMAP4). IMAP4 is similar to POP3, but it has more features; IMAP4 is more powerful and more complex.
- POP3 is deficient in several ways. It does not allow the user to organize her mail on the server; the user cannot have different folders on the server. (Of course, the user can create folders on her own computer.)
- In addition, POP3 does not allow the user to partially check the contents of the mail before downloading. IMAP4 provides the following extra functions:
    - A user can check the e-mail header prior to downloading.
    - A user can search the contents of the e-mail for a specific string of characters prior to downloading.
    - A user can partially download e-mail. This is especially useful if bandwidth is limited and the e-mail contains multimedia with high bandwidth requirements.
    - A user can create, delete, or rename mailboxes on the mail server.
    - A user can create a hierarchy of mail boxes in a folder for e-mail storage.

KnowledgeGate

MIME

- Electronic mail has a simple structure. Its simplicity, however, comes at a price. It can send messages only in NVT 7-bit ASCII format. In other words, it has some limitations. For example, it cannot be used for languages that are not supported by 7-bit ASCII characters (such as Hindi, French, German, Hebrew, Russian, Chinese, and Japanese).
- Also, it cannot be used to send binary files or video or audio data.
- Multipurpose Internet Mail Extensions (MIME) is a supplementary protocol that allows non-ASCII data to be sent through e-mail.
- MIME transforms non-ASCII data at the sender site to NVT ASCII data and delivers them to the client MTA to be sent through the Internet. The message at the receiving side is transformed back to the original data.

KnowledgeGate

www.knowledgegate.in

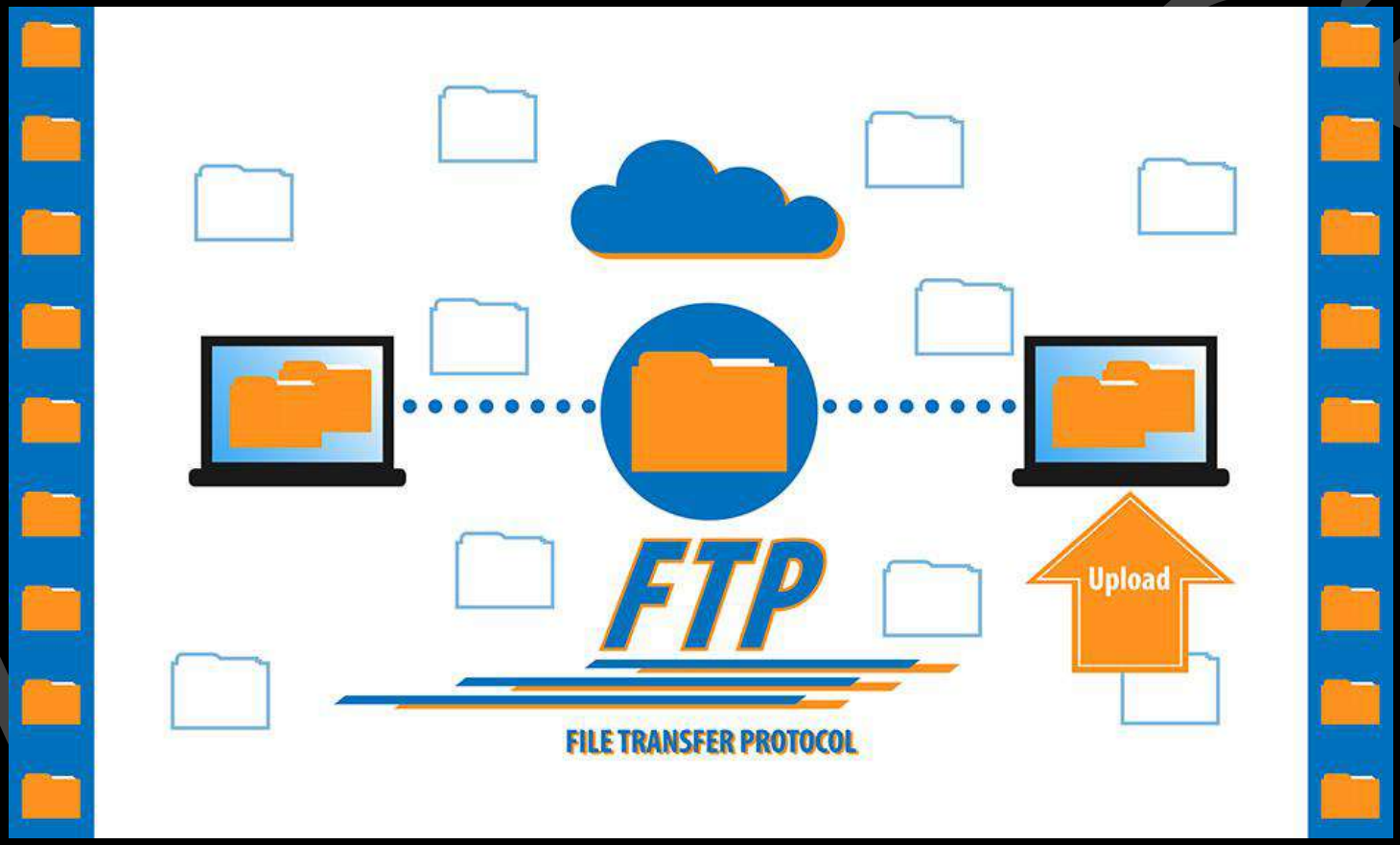
Web-Based Mail

- E-mail is such a common application that some websites today provide this service to anyone who accesses the site. Two common sites are Hotmail and Yahoo.
- The idea is very simple. Mail transfer from Alice's browser to her mail server is done through HTTP.
- The transfer of the message from the sending mail server to the receiving mail server is still through SMTP.
- Finally, the message from the receiving server (the Web server) to Bob's browser is done through HTTP.
- The last phase is very interesting. Instead of POP3 or IMAP4, HTTP is normally used. When Bob needs to retrieve his e-mails, he sends a message to the website (Hotmail, for example).
- The website sends a form to be filled in by Bob, which includes the log-in name and the password. If the log-in name and password match, the e-mail is transferred from the Web server to Bob's browser in HTML format.

KnowledgeGate

FILE TRANSFER

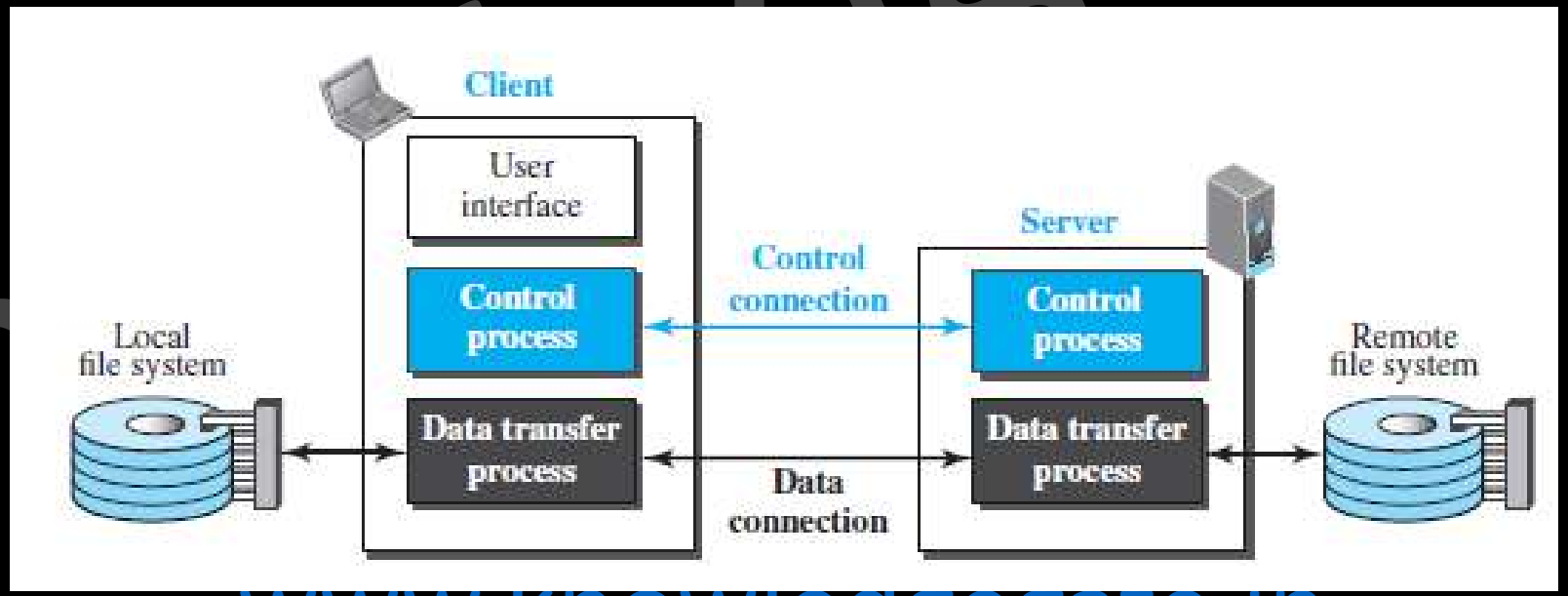
- Transferring files from one computer to another is one of the most common tasks expected from a networking or internetworking environment. As a matter of fact, the greatest volume of data exchange in the Internet today is due to file transfer.



www.knowledgegate.in

File Transfer Protocol (FTP)

- It is used for exchanging files over the internet and enables the users to upload and download the files from the internet.
- File Transfer Protocol (FTP) is the standard mechanism provided by TCP/IP for copying a file from one host to another. Although transferring files from one system to another seems simple and straightforward, some problems must be dealt with first. For example, two systems may use different file name conventions.
- Two systems may have different ways to represent text and data. Two systems may have different directory structures.
- All these problems have been solved by FTP in a very simple and elegant approach. FTP differs from other client/server applications in that it establishes two connections between the hosts.

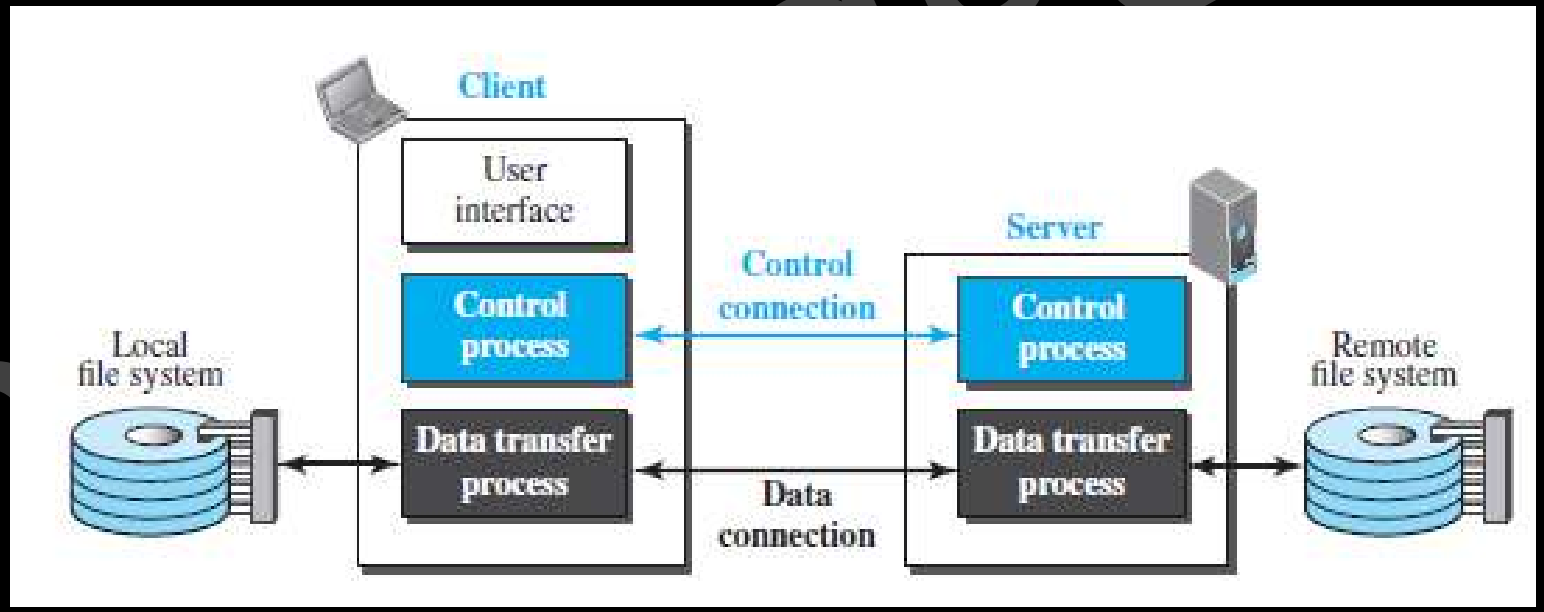


KnowledgeGate

www.knowledgegate.in

- One connection is used for data transfer, the other for control information (commands and responses). Separation of commands and data transfer makes FTP more efficient. The control connection uses very simple rules of communication. We need to transfer only a line of command or a line of response at a time. The data connection, on the other hand, needs more complex rules due to the variety of data types transferred.

- However, the difference in complexity is at the FTP level, not TCP. For TCP, both connections are treated the same. FTP uses two well-known TCP ports: Port 21 is used for the control connection, and port 20 is used for the data connection. The control connection remains connected during the entire interactive FTP session. The data connection is opened and then closed for each file transfer activity.



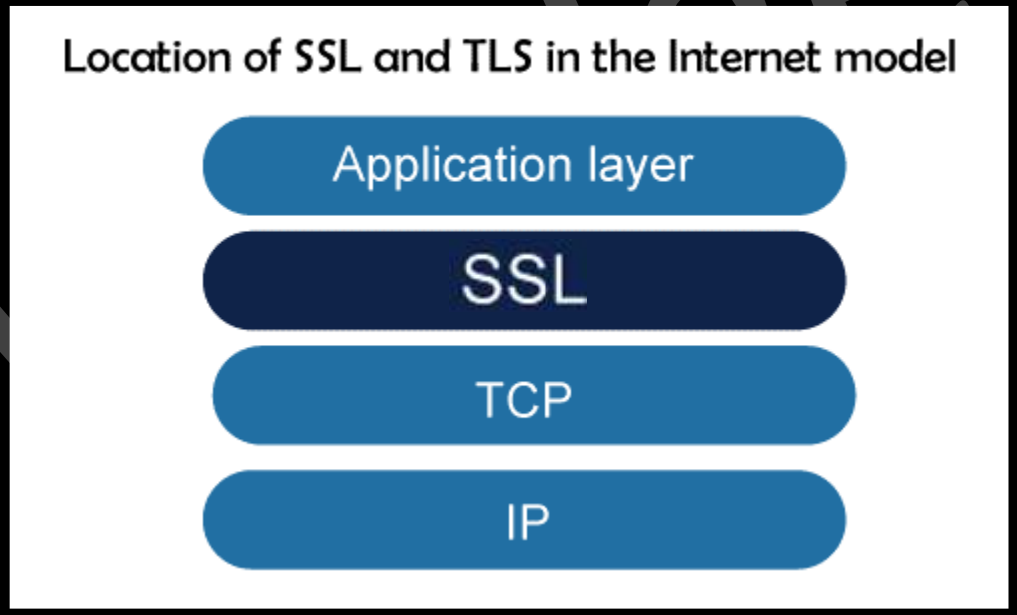
KnowledgeGate

www.knowledgegate.in

Security for FTP

- The FTP protocol was designed when security was not a big issue. Although FTP requires a password, the password is sent in plaintext (unencrypted), which means it can be intercepted and used by an attacker. The data transfer connection also transfers data in plaintext, which is insecure.

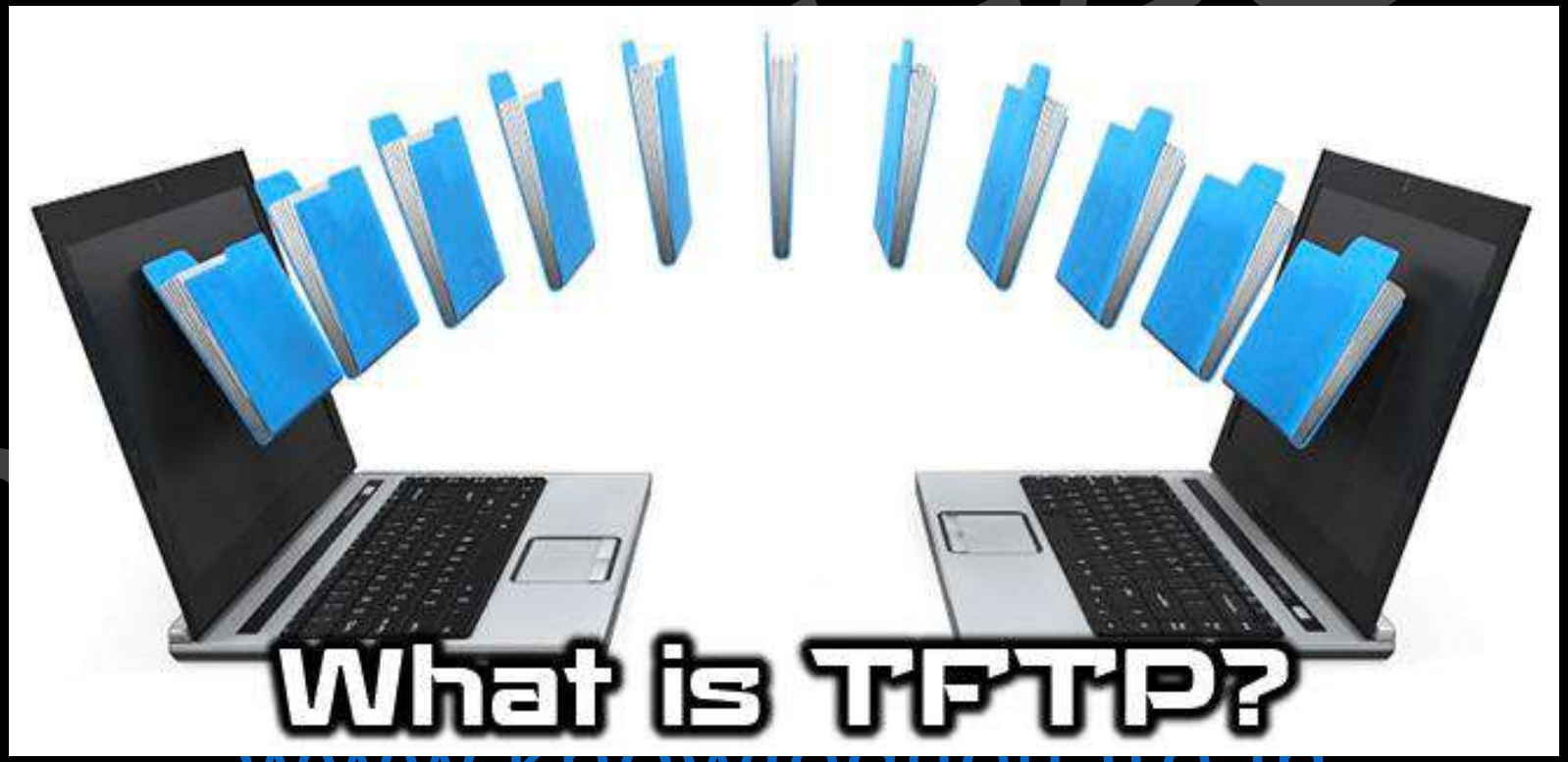
- To be secure, one can add a Secure Socket Layer between the FTP application layer and the TCP layer. In this case FTP is called SSL-FTP.



KnowledgeGate

Trivial File Transfer Protocol (TFTP)

- TFTP is a simple, UDP-based file transfer protocol that supports only basic read and write operations, typically using port 69, and is often used for booting systems or initial configurations.
- Due to its lack of authentication and encryption, as well as minimal error handling, TFTP is not suitable for secure or reliable long-distance file transfers.
- Although largely replaced by more secure and feature-rich protocols like FTPS and SFTP, TFTP's simplicity and minimal resource requirements make it useful for specific, local network applications.



KnowledgeGate
www.knowledgegate.in

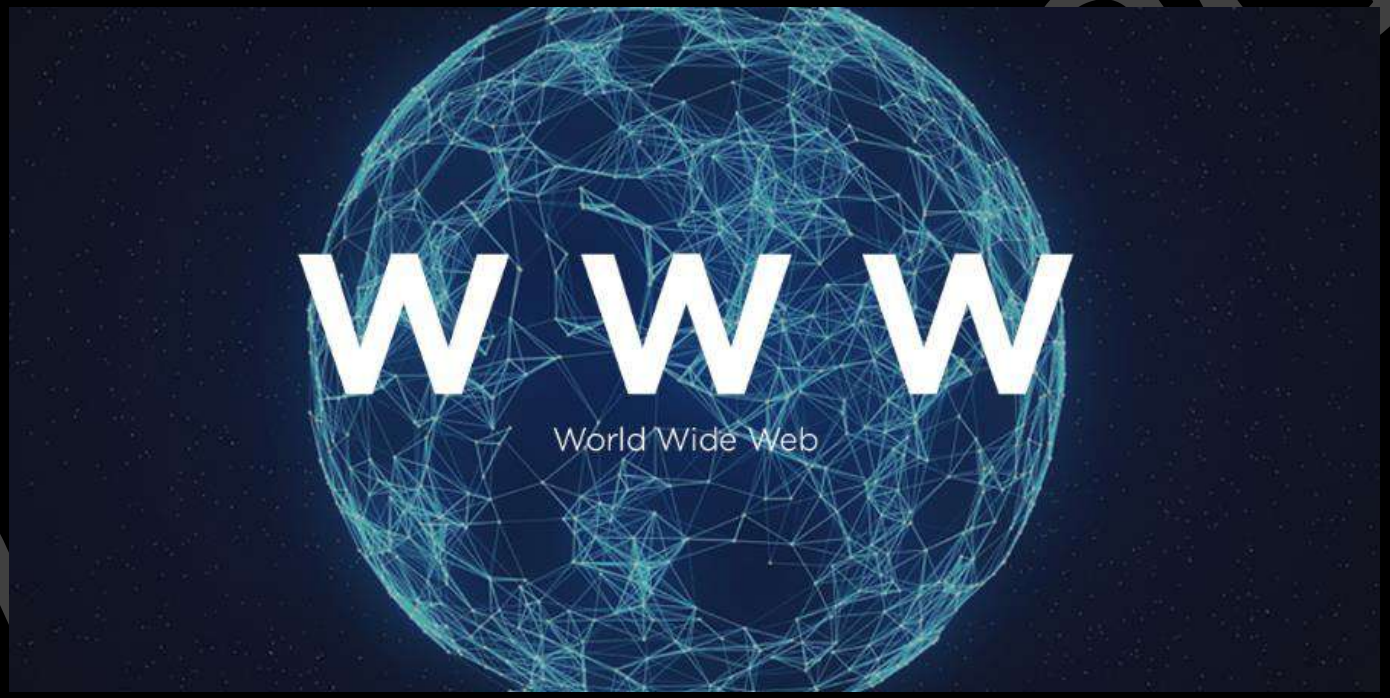
WWW

- The idea of the Web was first proposed by Tim Berners-Lee in 1989 CERN (European Laboratory for Particle Physics) to create a system to handle distributed resources necessary for scientific research.
- The World Wide Web (WWW) is a repository of information linked together from points all over the world. It a unique combination of flexibility, portability, and user-friendly features that distinguish it from other services provided by the Internet.

KnowledgeG

www.knowledgegate.in

- **ARCHITECTURE**
  - The WWW today is a distributed client server service, in which a client using a browser can access a service using a server. However, the service provided is distributed over many locations called sites.
  - Each site holds one or more documents, referred to as Web pages. Each Web page can contain a link to other pages in the same site or at other sites. The pages can be retrieved and viewed by using browsers.



www.knowledgegate.in

- **Client (Browser)**
  - A variety of vendors offer commercial browsers that interpret and display a Web document, and all use nearly the same architecture.
  - Each browser usually consists of three parts: a controller, client protocol, and interpreters.
  - The controller receives input from the keyboard or the mouse and uses the client programs to access the document. After the document has been accessed, the controller uses one of the interpreters to display the document on the screen.

KnowledgeGate

- **Server**
  - The Web page is stored at the server. Each time a client request arrives, the corresponding document is sent to the client.
  - To improve efficiency, servers normally store requested files in a cache in memory; memory is faster to access than disk.
  - A server can also become more efficient through multithreading or multiprocessing. In this case, a server can answer more than one request at a time.
  - A client that wants to access a Web page needs the address. To facilitate the access of documents distributed throughout the world, HTTP uses locators.

KnowledgeGate

www.knowledgegate.in

- **Uniform Resource Locator (URL)**
  - A web page, as a file, needs to have a unique identifier to distinguish it from other web pages.
  - To define a web page, we need four identifiers in general: ***Protocol***, ***host, port, and path.***
  - ***Protocol*.** Which client-server application we are using is called protocol. Although most of the time the protocol is HTTP (Hyper Text Transfer Protocol), we can also use other protocols such as FTP (File Transfer Protocol).
  - ***Host.*** The host identifier can be the IP address of the server or the unique name to the server.
  - ***Port*.** The port, a 16-bit integer, is normally predefined for the client-server application.

KnowledgeGate

Cookies

- Cookies enable stateful interactions on the World Wide Web, allowing websites to remember users and their activities, such as login status or items in a shopping cart.
- Upon a client's request, the server creates a cookie containing user information and sends it back to the client; the client's browser stores this cookie.
- During subsequent interactions, the client's browser sends the stored cookie back to the server, allowing the server to recognize returning clients and provide a more personalized experience.

KnowledgeGate

HTTP

- The Hypertext Transfer Protocol (HTTP) is a protocol used mainly to access data on the World Wide Web. The **Hyper Text Transfer Protocol (HTTP)** is used to define how the client-server programs can be written to retrieve web pages from the Web.
- HTTP uses the services of TCP on well-known port 80, the client uses a temporary port number.
- It is a connection-oriented and reliable protocol.
- HTTP functions as a combination of FTP and SMTP.

KnowledgeGate

www.knowledgegate.in

- It is similar to FTP because it transfers files and uses the services of TCP. However, it is much simpler than FTP because it uses only one TCP connection. There is no separate control connection; only data are transferred between the client and the server.

- HTTP is like SMTP because the data transferred between the client and the server look like SMTP messages. Unlike SMTP, the HTTP messages are not destined to be read by humans; they are read and interpreted by the HTTP server and HTTP client (browser).

- SMTP messages are stored and forwarded, but HTTP messages are delivered immediately. The commands from the client to the server are embedded in a request message. The contents of the requested file or other information are embedded in a response message.

KnowledgeGate

www.knowledgegate.in

Nonpersistent versus Persistent Connections

- ***Nonpersistent Connections***: - In a **nonpersistent connection**, one TCP connection is made for each request/response. The following lists the steps in this strategy:
  - The client opens a TCP connection and sends a request.
  - The server sends the response and closes the connection.
  - The client reads the data until it encounters an end-of-file marker; it then closes the connection.
  - **For example:** If a file contains links to N different pictures in different files (all located on the same server), the connection must be opened and closed N + 1 times.

- **Disadvantage** The nonpersistent strategy imposes high overhead on the server because the server needs N + 1 different buffer each time a connection is opened.

KnowledgeGate

Persistent Connections

- HTTP version 1.1 specifies a **persistent connection** by default.
- In a persistent connection, the server leaves the connection open for more requests after sending a response.
- The server can close the connection at the request of a client or if a time-out has been reached.

- **Advantages**
  - Time and resources are saved using persistent connections.
  - Only one set of buffers and variables needs to be set for the connection at each site.
  - The round-trip time for connection establishment and connection termination is saved.

KnowledgeGate

- ***It is important to know that HTTP is a stateless protocol as:***
  - HTTP server does not maintain any state. It forgets about the client after sending the response.
  - It treats every new request independently.

- ***HTTP Security***
  - HTTP per se does not provide security.
  - HTTP can be run over the Secure Socket Layer (SSL). In this case, HTTP is referred to as HTTPS.
  - HTTPS provides confidentiality, client and server authentication, and data integrity.

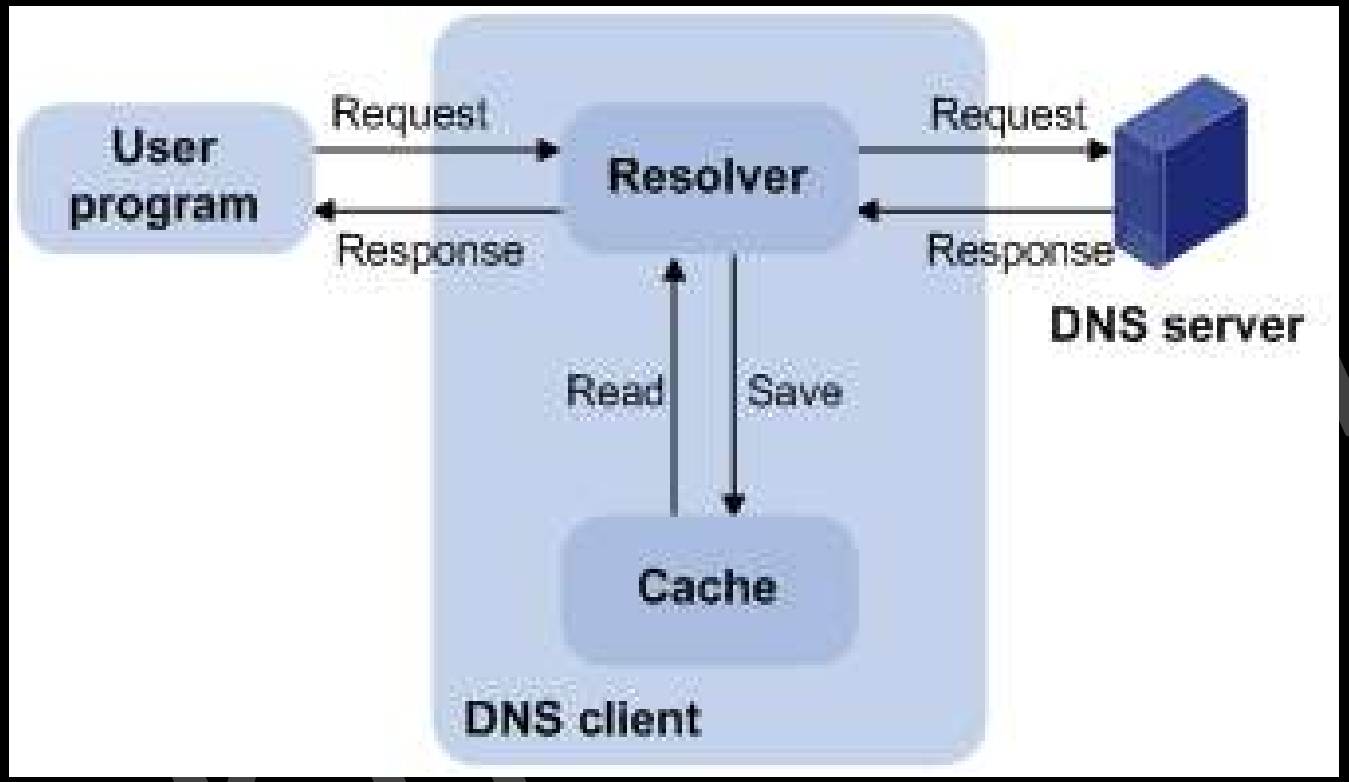
KnowledgeGate

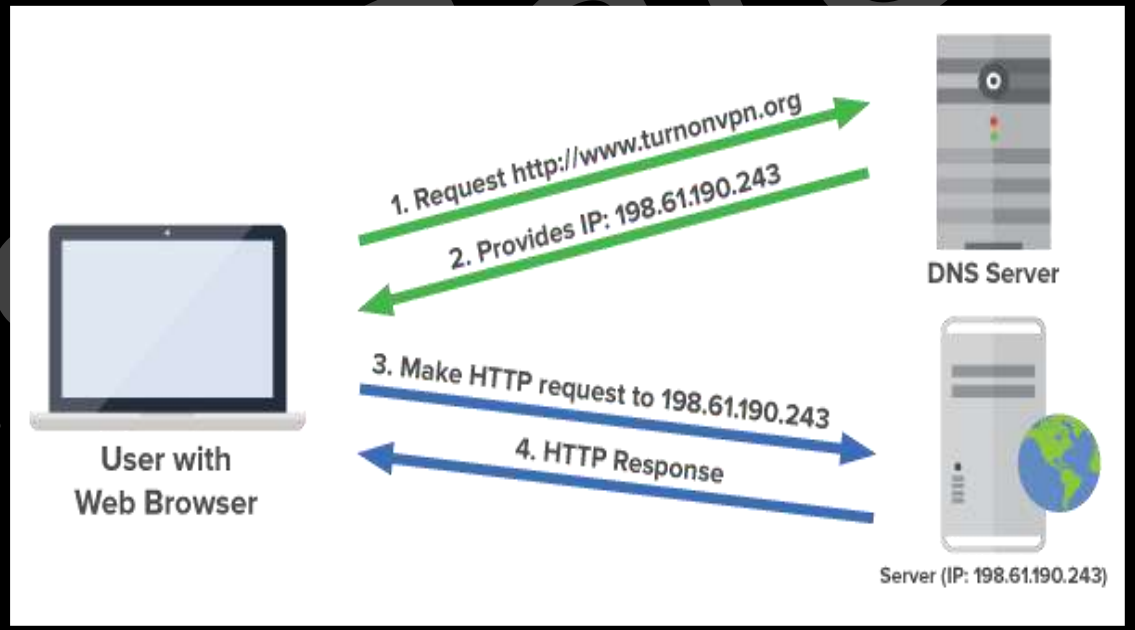
DNS

- As we know human beings are not comfortable in remembering numbers so to remember IP address of a website or mail account in internet is difficult. Secondly IP addresses of mail or websites keeps on changing, so we have to come up with one more level of addressing which is easy to remember and do not change with time.
- Solution is Name addressing, i.e. we give some names to websites and mail account like we do to humans in real world. But then if someone write a name of the website in the browser we need some mechanism to convert it back into IP address.
- Domain Name System solve this problem.

KnowledgeGate

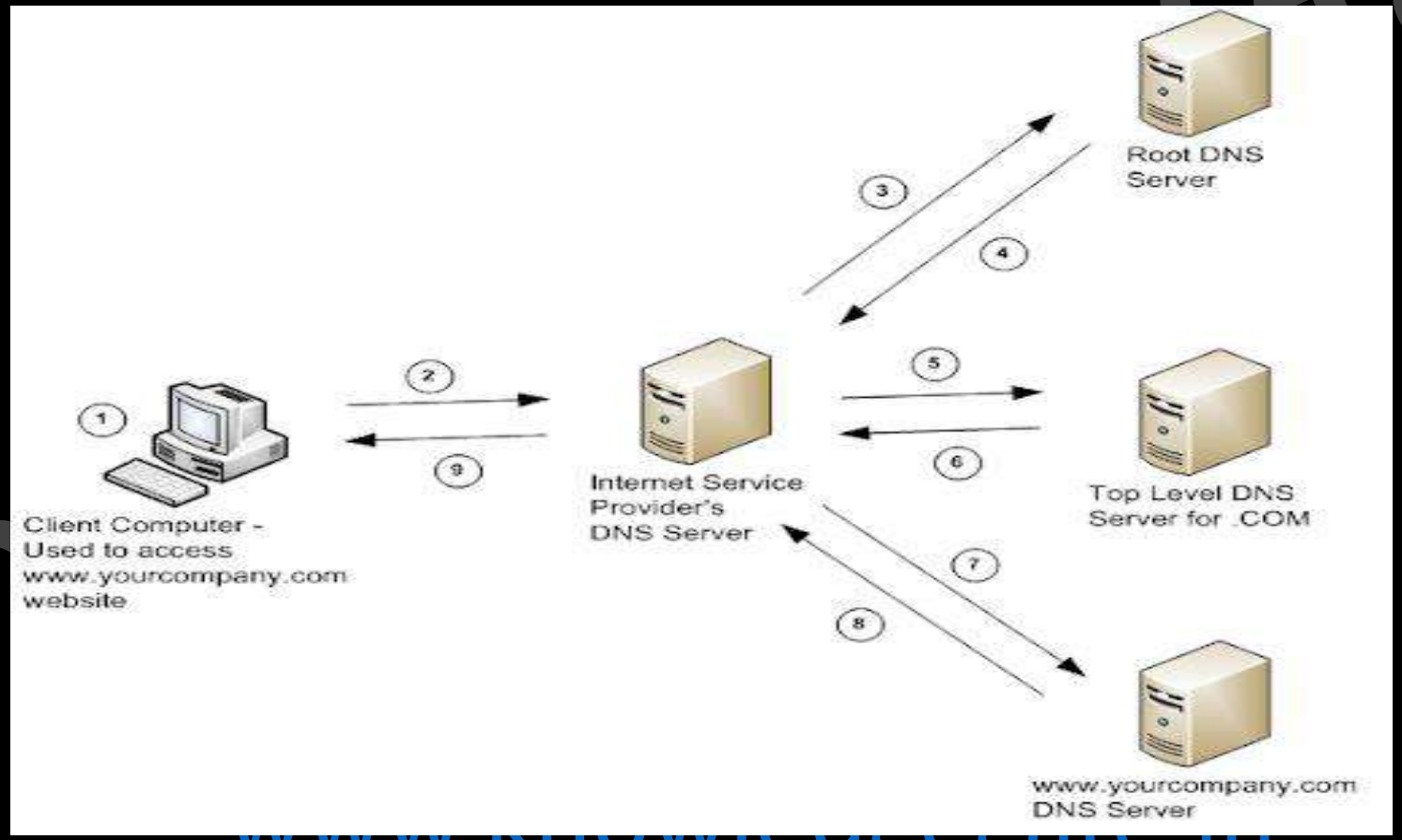
- This diagram perfectly represent how DNS works, A user of a website may know the name of the website; however, the IP protocol needs the IP address.
- The DNS client program sends a request to a DNS server to map the Web-site address to the corresponding IP address.





www.knowledgegate.in

- Today we divide this huge amount of information into smaller parts and store each part on a different computer. In this method, the host that needs mapping can contact the closest computer holding the needed information.
- It is Very difficult to find out the ip address associated to a website because there are millions of websites and with all those websites, we should be able to generate the ip address immediately, there should not be a lot of delay for that to happen organization of database is very important.

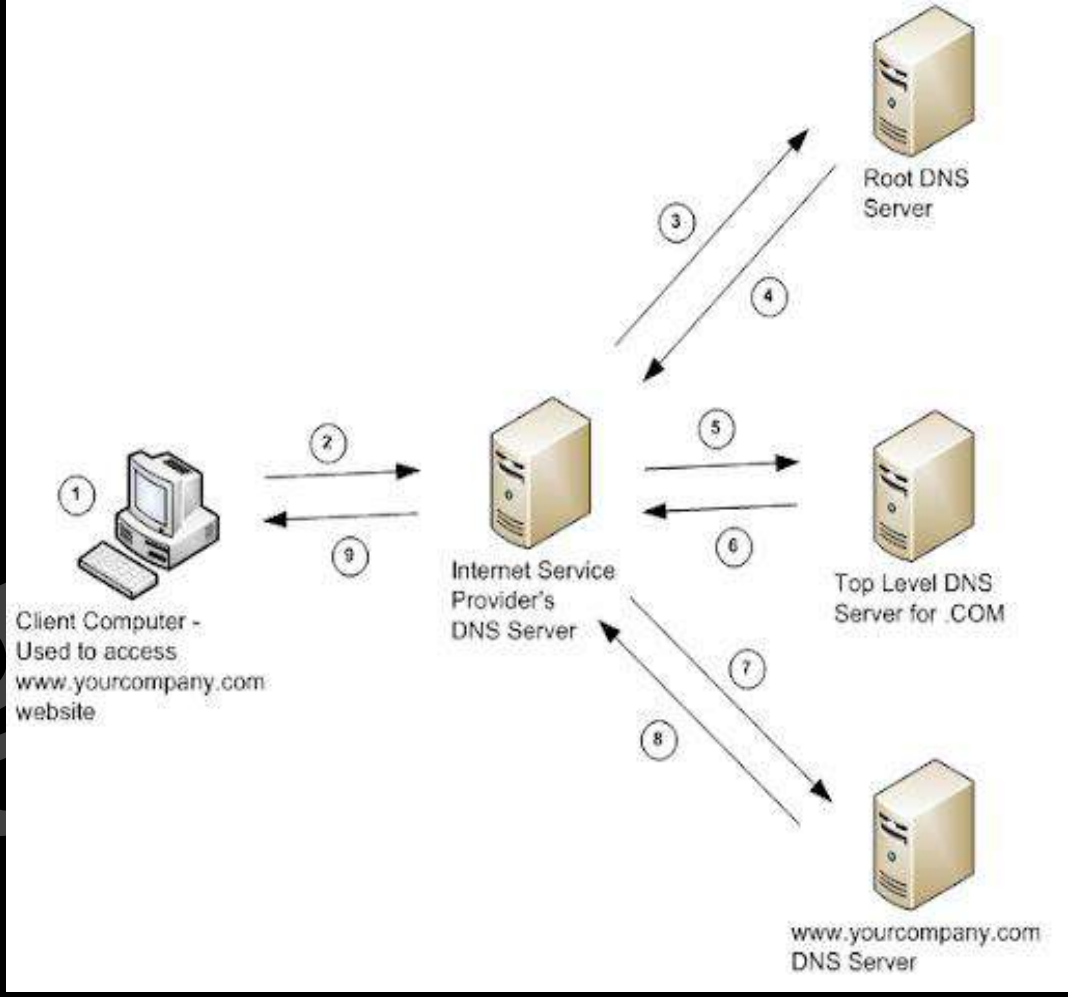


Knowledge Gate
www.knowledgegate.in

- **Hierarchy of Name Servers**
  - **Root name servers** – It is contacted by name servers that cannot resolve the name. It contacts authoritative name server if name mapping is not known. It then gets the mapping and return the IP address to the host.

  - **Top level server** – It is responsible for com, org, edu etc and all top-level country domains like uk, fr, ca, in etc. They have info about authoritative domain servers and know names and IP addresses of each authoritative name server for the second level domains.

  - **Authoritative name servers** This is organization's DNS server, providing authoritative hostName to IP mapping for organization servers. It can be maintained by organization or service provider. In order to reach cse.dtu.in we have to ask the root DNS server, then it will point out to the top-level domain server and then to authoritative domain name server which actually contains the IP address. So, the authoritative domain server will return the associative ip address.



Knowledge

www.knowledgegate.in

NAME SPACE

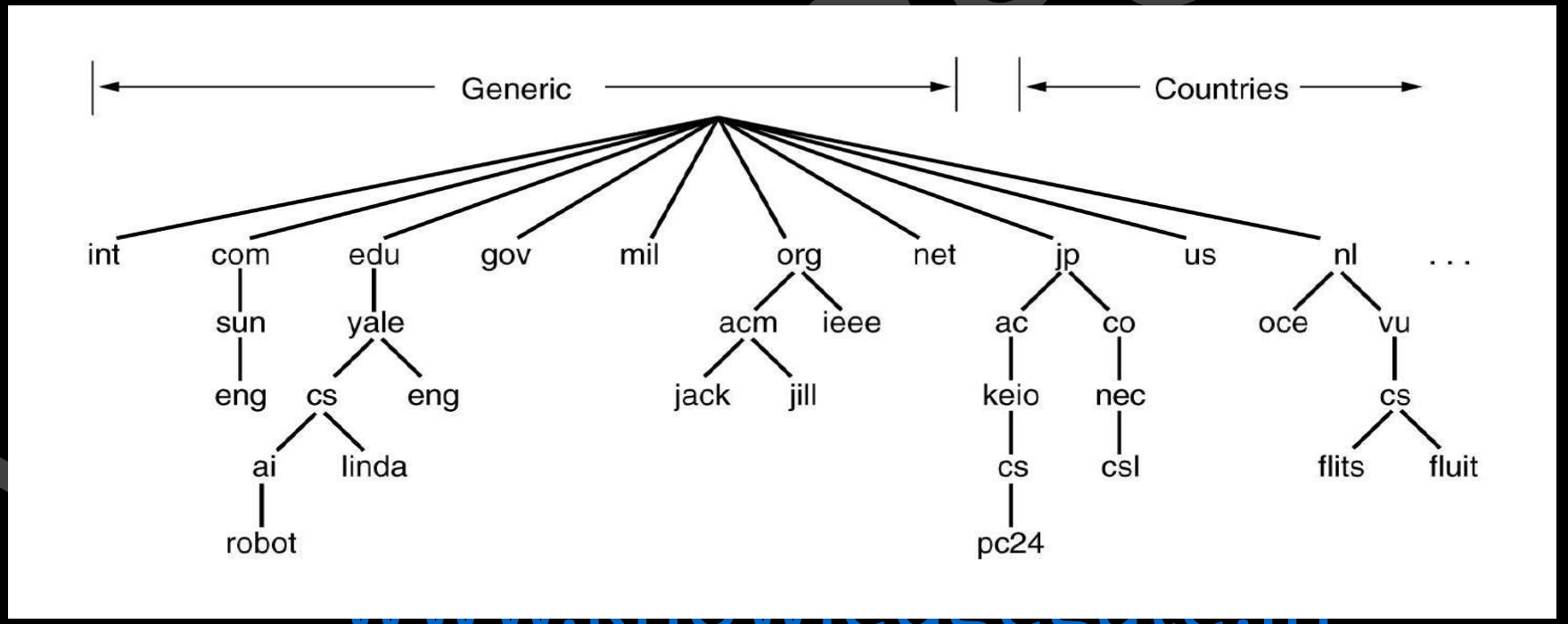
- To be unambiguous, the names must be unique because the addresses are unique. A name space that maps each address to a unique name can be organized in two ways: flat or hierarchical.

- **Flat Name Space**
  - In a flat name space, a name is assigned to an address. A name in this space is a sequence of characters without structure.
  - The main disadvantage of a flat name space is that it cannot be used in a large system such as the Internet because it must be centrally controlled to avoid ambiguity and duplication.
  - So, Solution is Hierarchical Name Space

KnowledgeGate

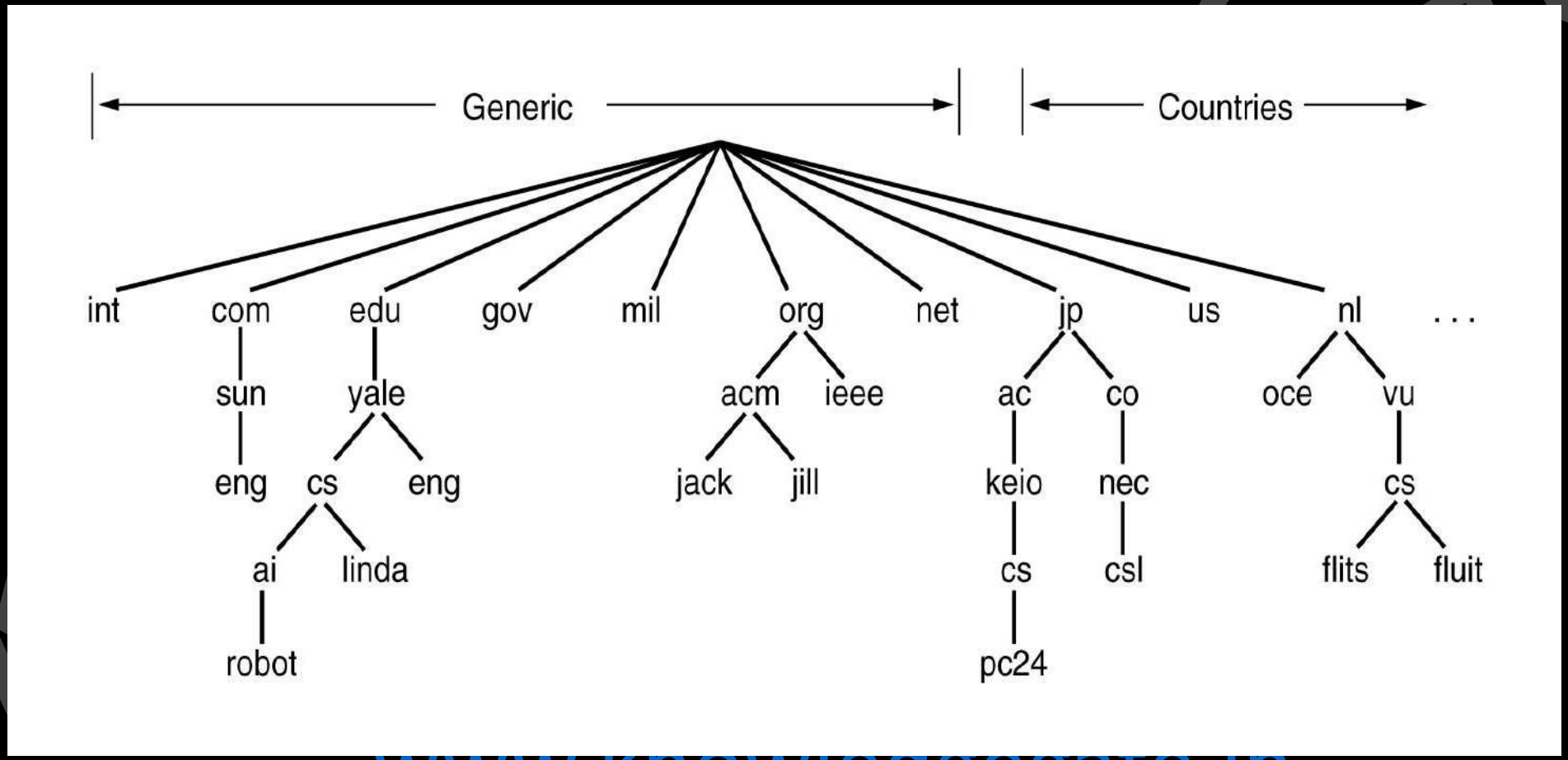
Hierarchical Name Space

- In a hierarchical name space, each name is made of several parts. The first part can define the nature of the organization
- the second part can define the name of an organization, the third part can define departments in the organization, and so on.
- In this case, the authority to assign and control the name spaces can be decentralized. A central authority can assign the part of the name that defines the nature of the organization and the name of the organization.
- The responsibility of the rest of the name can be given to the organization itself.
- The management of the organization need not worry that the prefix chosen for a host is taken by another organization because, even if part of an address is the same, the whole address is different



KnowledgeGate
www.knowledgegate.in

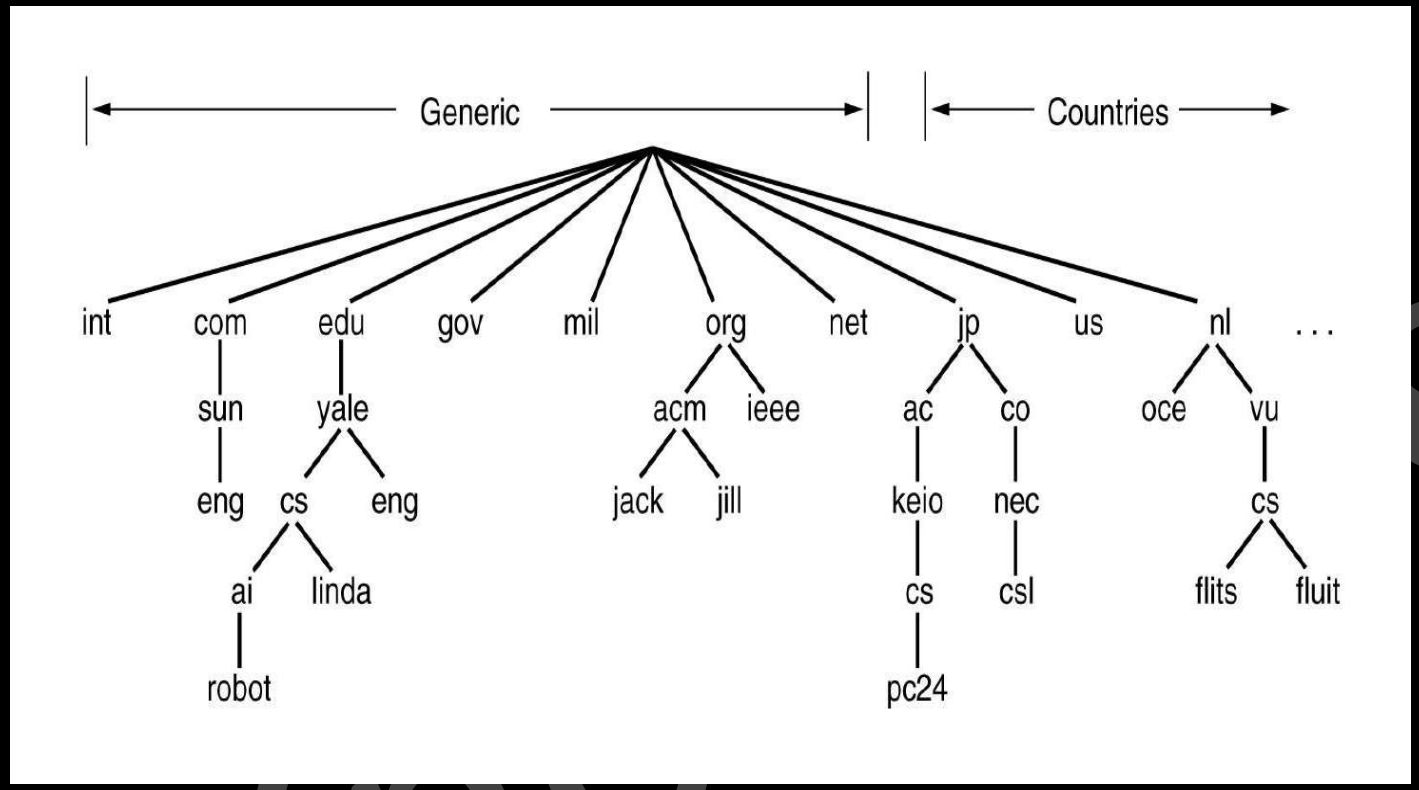
- Generic domain: .com(commercial) .edu(educational) .mil(military) .org (non-profit organization) .net (similar to commercial) all these are generic domain.
- Country domain .in (india) .us .uk
- Inverse domain if we want to know what is the domain name of the website. Ip to domain name mapping. So, DNS can provide both the mapping for example to find the ip addresses of www.unacademy.com then we have to type nslookup www.unacademy.com



www.knowledgegate.in

DOMAIN NAME SPACE

- To have a hierarchical name space, a domain name space was designed. In this design the names are defined in an inverted-tree structure with the root at the top. The tree can have only 128 levels: level 0 (root) to level 127

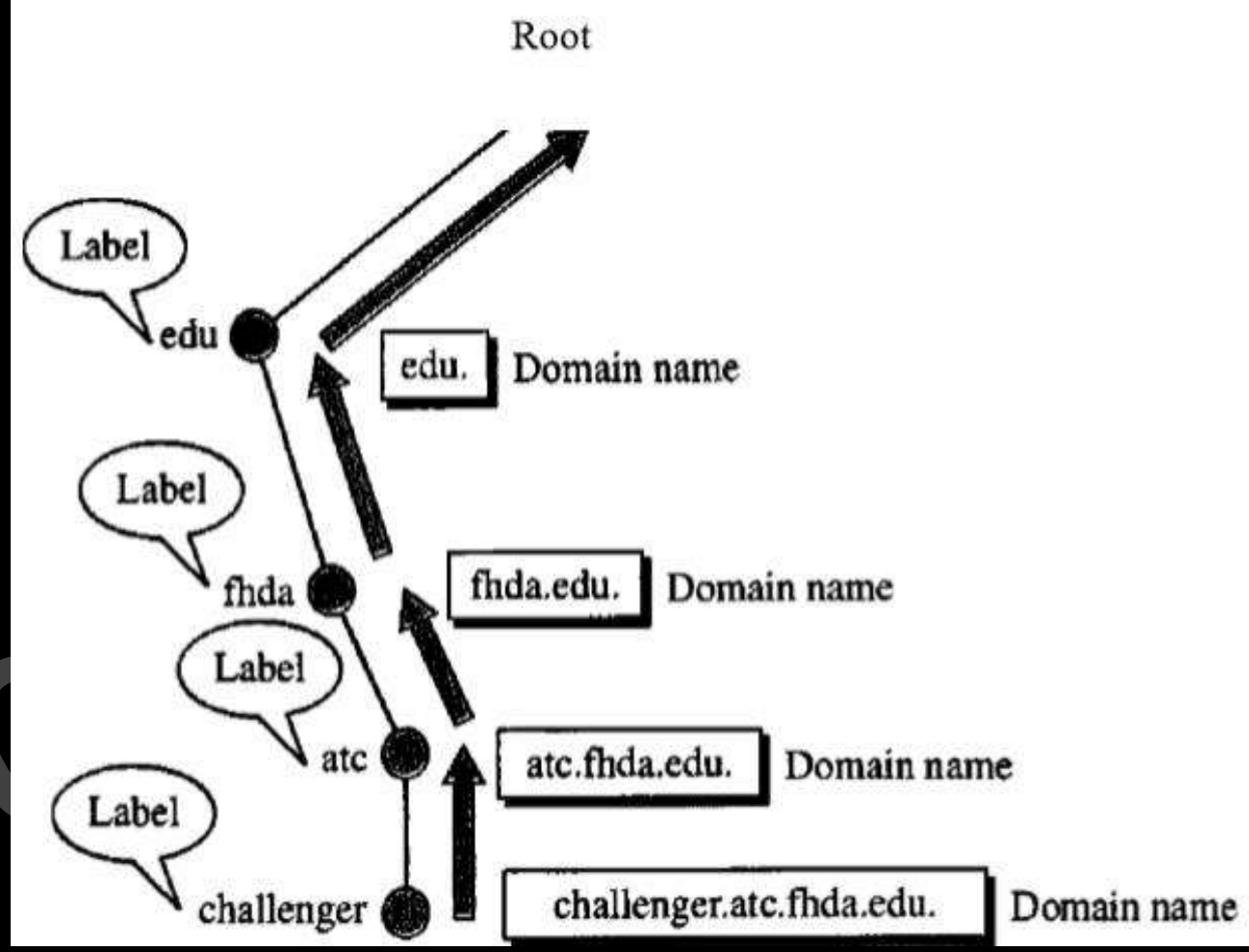


Label	Description
aero	Airlines and aerospace companies
biz	Businesses or firms (similar to "com")
com	Commercial organizations
coop	Cooperative business organizations
edu	Educational institutions
gov	Government institutions
info	Information service providers
int	International organizations
mil	Military groups
museum	Museums and other nonprofit organizations
name	Personal names (individuals)
net	Network support centers
org	Nonprofit organizations
pro	Professional individual organizations

www.knowledgegate.in

- **Label**
  - Each node in the tree has a label, which is a string with a maximum of 63 characters.
  - The root label is a null string (empty string).
  - DNS requires that children of a node (nodes that branch from the same node) have different labels, which guarantees the uniqueness of the domain names.

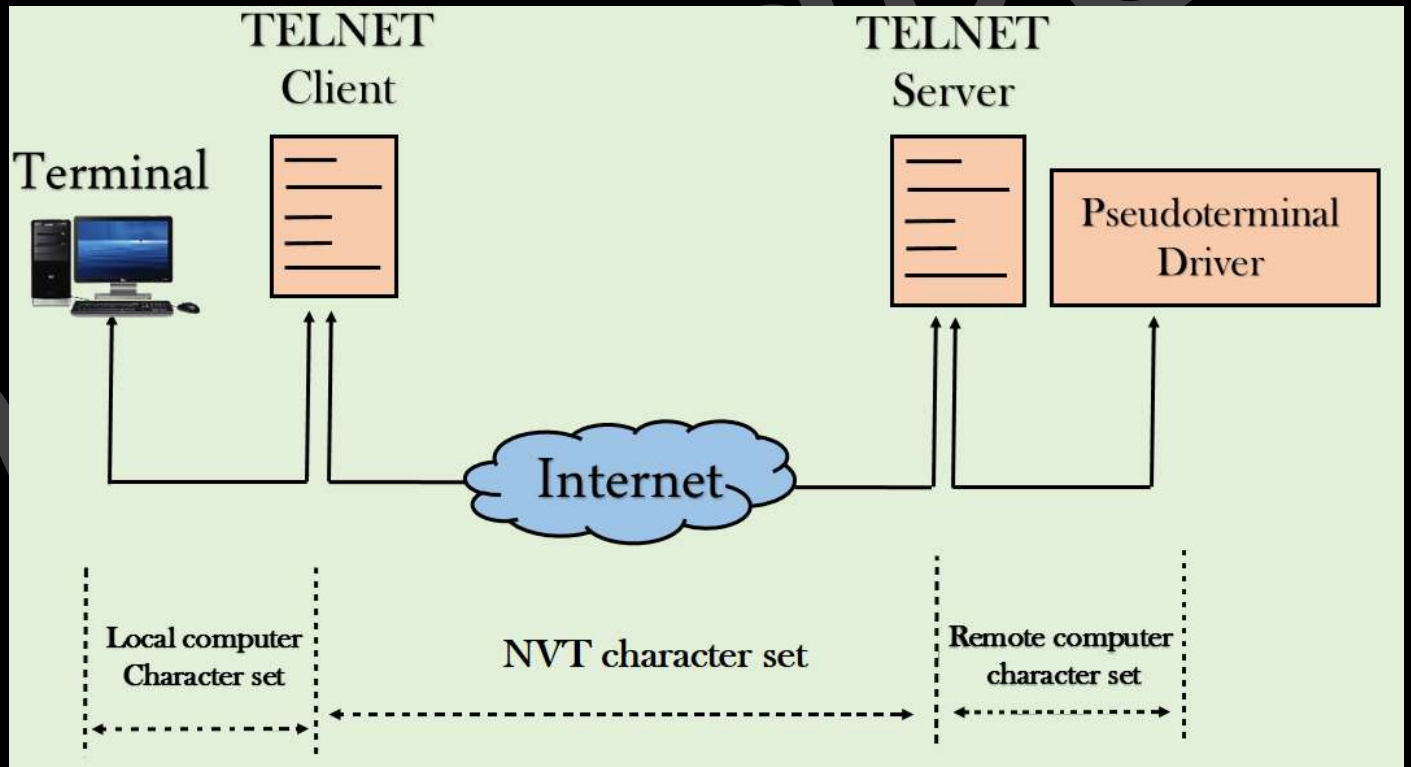
- **Domain Name**
  - Each node in the tree has a domain name. A full domain name is a sequence of labels separated by dots (.).
  - The domain names are always read from the node up to the root. The last label is the label of the root (null). This means that a full domain name always ends in a null label, which means the last character is a dot because the null string is nothing.



Knowledge

Telnet (Telecommunication Network)

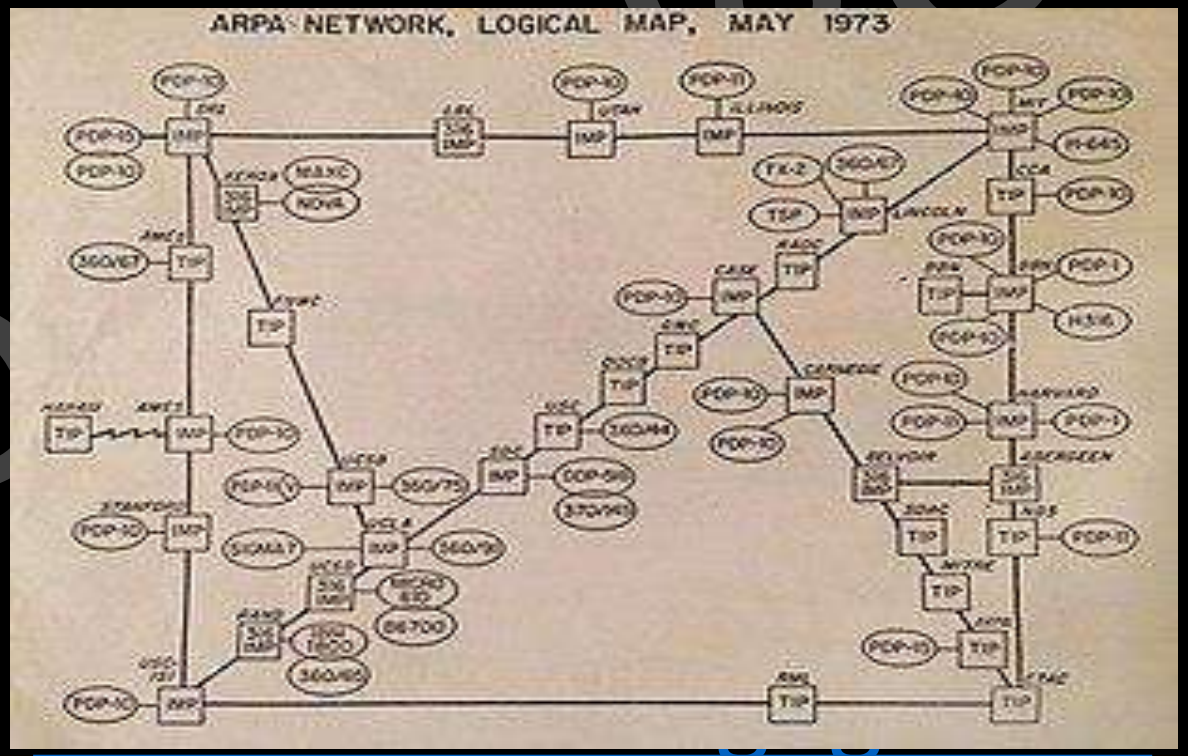
- Telnet is a text-based protocol used for remote access to servers, operating on TCP port 23 and following a client-server model, but lacks data encryption.
- Although useful for debugging and interactive sessions, its lack of security measures makes it susceptible to eavesdropping and unsuitable for transmitting sensitive information.
- Largely replaced by more secure alternatives like SSH, Telnet still finds use in legacy systems and specialized applications where high security is not crucial.



KnowledgeGate

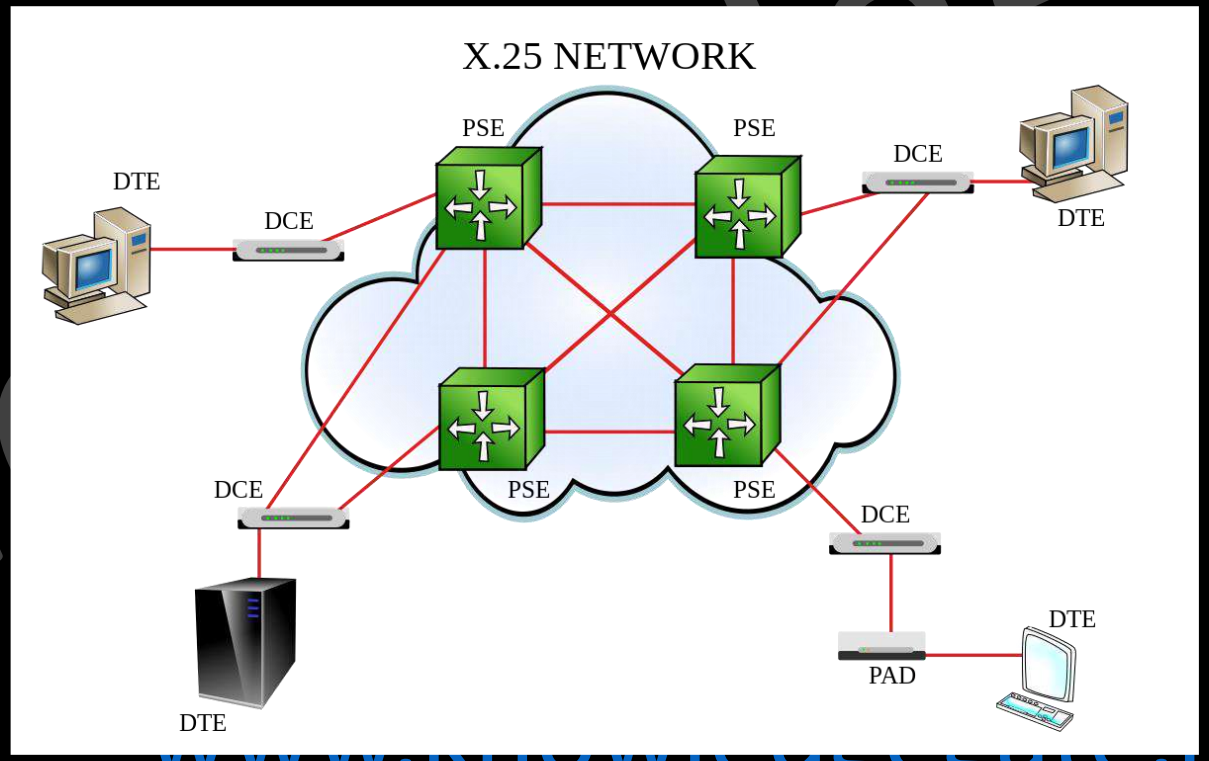
ARPANET (Advanced Research Projects Agency Network)

- ARPANET was the first wide-area network using packet switching, created in the late 1960s by the U.S. Department of Defense, and it laid the groundwork for today's internet.
- It introduced early networking protocols like NCP, gave rise to applications like email, and was designed for research and resource sharing, expanding to connect hundreds of institutions over time.
- Though decommissioned in 1990, its technologies and concepts had a lasting impact, inspiring subsequent networks and internet protocols, but initially lacked strong security measures.



X.25

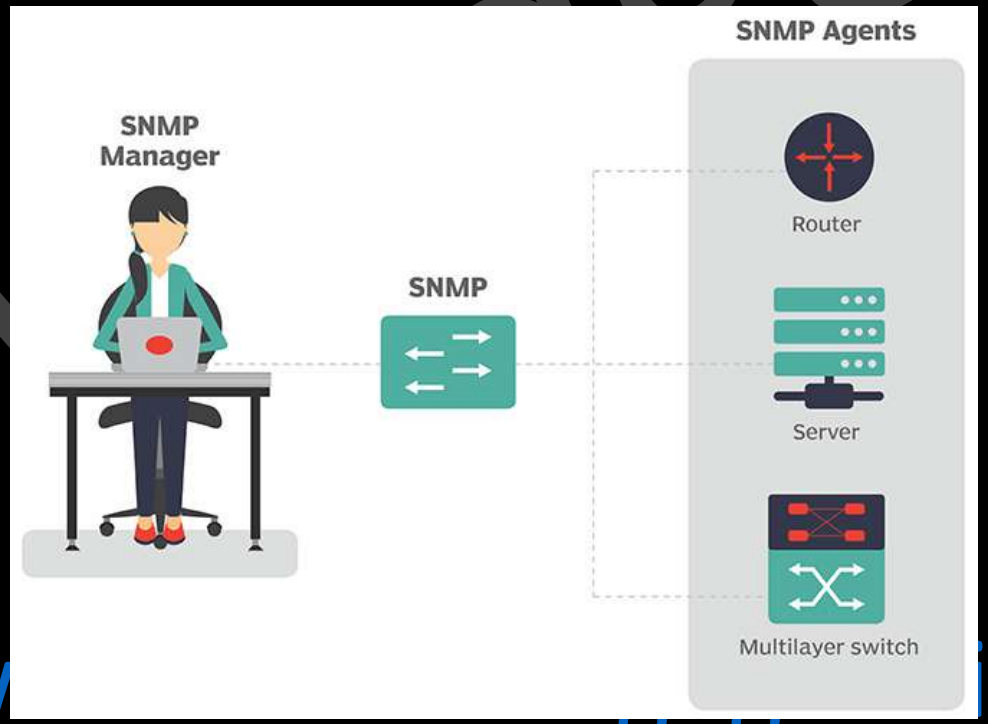
- X.25 is an old protocol for wide area networks that includes features for error checking, virtual circuit setup, and usage-based billing, operating across three layers of the OSI model.
- It was widely used in the past for applications like credit card processing and ATMs, supporting slower data rates initially but evolving over time.
- Although its usage has declined due to faster protocols like IP, it inspired newer technologies like Frame Relay and ATM and was a backbone for international data services before the Internet.



KnowledgeGate
www.knowledgegate.in

Simple Network Management Protocol (SNMP)

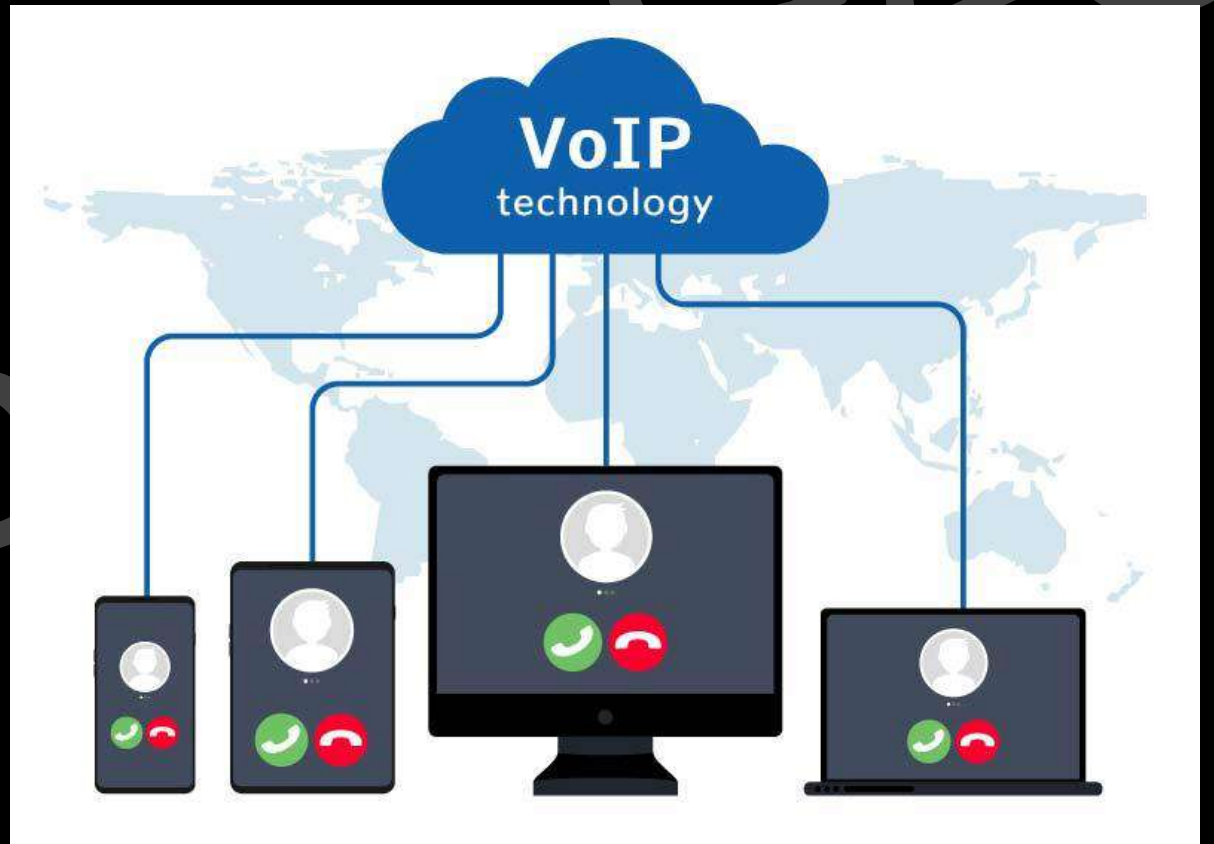
- SNMP is a common tool for managing and monitoring network devices; it operates over UDP and interacts with databases known as MIBs to control device properties.
- It works on a client-server model with an SNMP manager and agents, supports various operations like GET, SET, and TRAP, and comes in different versions with varying security features.
- It's scalable and used for real-time monitoring, collecting data either by polling from the manager to the agent or trapping where the agent notifies the manager about specific events.



KnowledgeGate

Voice over IP

- VoIP allows for versatile communication, including voice calls and multimedia, over IP networks, offering cost savings and network efficiency.
- Relies on a stable internet connection and computer hardware; any disruption can affect the telephone service.
- Susceptible to delays, security risks, and challenges in routing emergency calls due to the nature of IP networks.



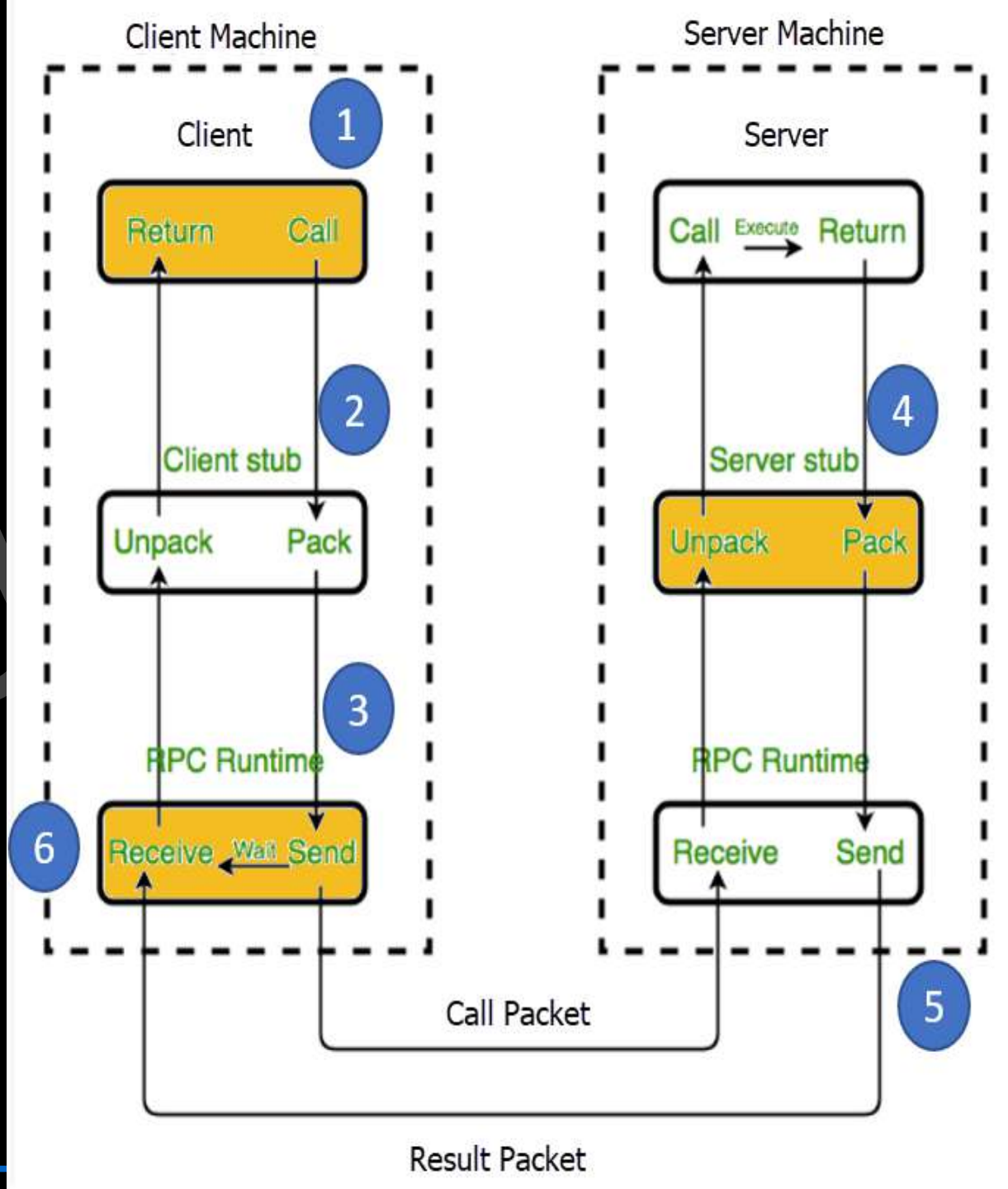
KnowledgeGate

Remote Procedure Call

- Remote Procedure Call (RPC) allows programs to execute procedures (functions) on a remote server, as if they were local, facilitating distributed computing.
- Operates over various transport protocols such as TCP or HTTP and may include authentication and encryption features for secure communication.
- Often used in client-server architectures and distributed systems, but can introduce complexities like network latency and failure handling.

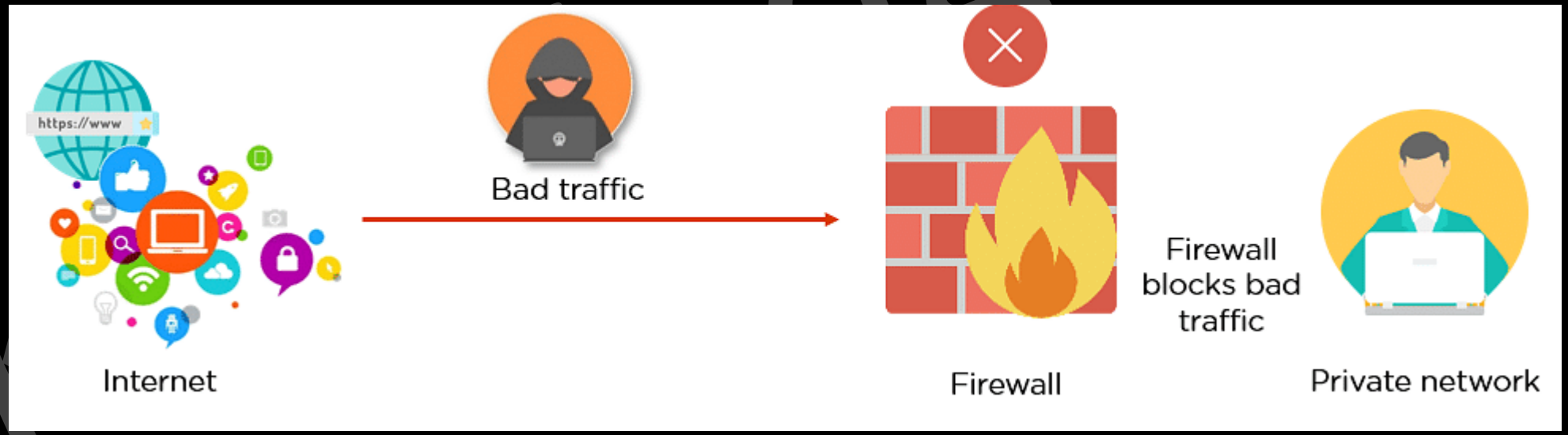
Knowled

www.knowle



firewall

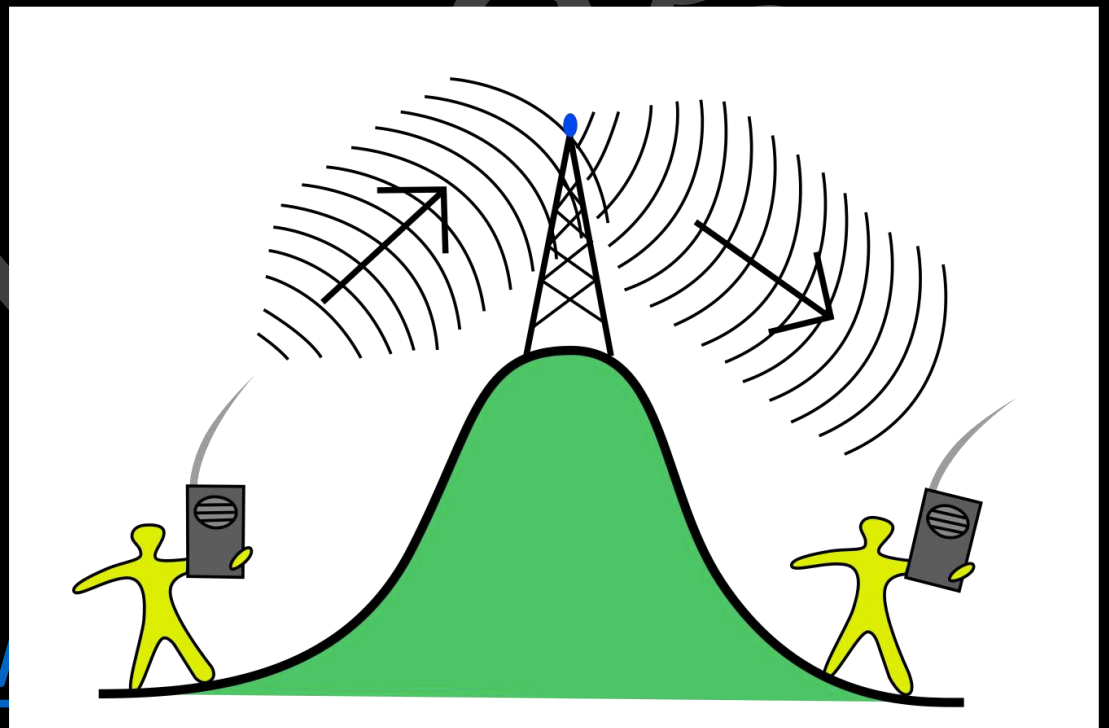
- Firewalls act as security guards for network traffic, checking data packets and allowing or blocking them based on set rules like IP addresses and port numbers.
- They offer different features like Stateful Inspection, Proxy Services, and VPN Support to enhance security measures, some even include intrusion detection systems.
- Types of firewalls range from hardware versions that are separate devices to software ones installed on individual computers, and they often come with logging and reporting features for network monitoring.



www.knowledgegate.in

Repeater

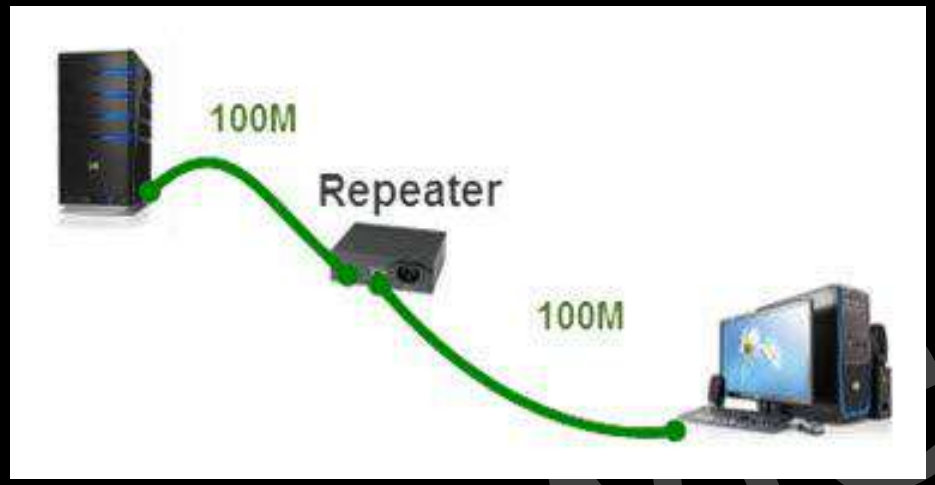
- Repeaters boost the strength of a signal as it travels through a communication channel, like a telephone line or a radio frequency, to help it cover longer distances.
- In situations where signals lose power due to resistance or distance, a repeater amplifies the signal before sending it further.
- In computer networking, repeaters operate on the physical layer of the OSI model, as they only work with the actual signal and don't interpret data.



KnowledgeGate

- Repeaters are used to extend transmissions so that the signal can cover longer distances or be received on the other side of an obstruction.

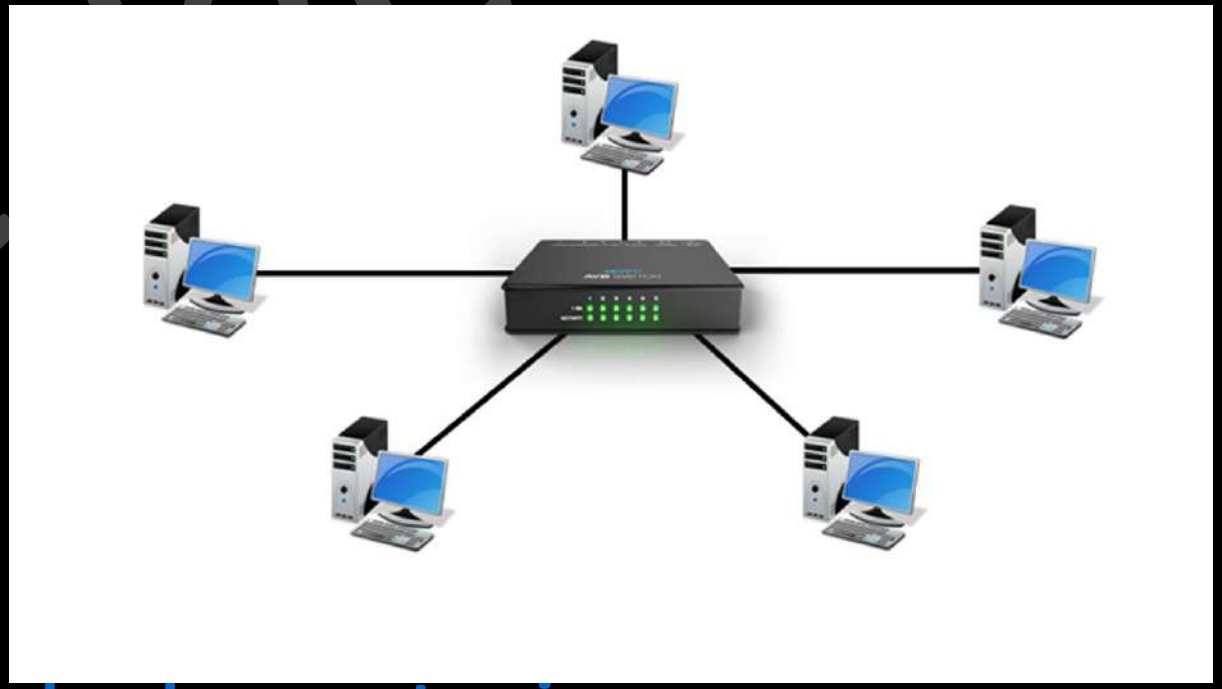
- In computer networking, because repeaters work with the actual physical signal, and do not attempt to interpret the data being transmitted, they operate on the physical layer, the first layer of the OSI model.



KnowledgeGate

Hub

- Hubs are multiport repeater. An **Ethernet hub**, **active hub**, **network hub**, **repeater hub**, **multiport repeater**, or simply **hub** is a network hardware device for connecting multiple Ethernet devices together and making them act as a single network segment.
- It has multiple input/output (I/O) ports, in which a signal introduced at the input of any port appears at the output of every port except the original incoming. A hub works at the physical layer (layer 1) of the OSI model. Hubs are now largely obsolete, having been replaced by network switches except in very old installations

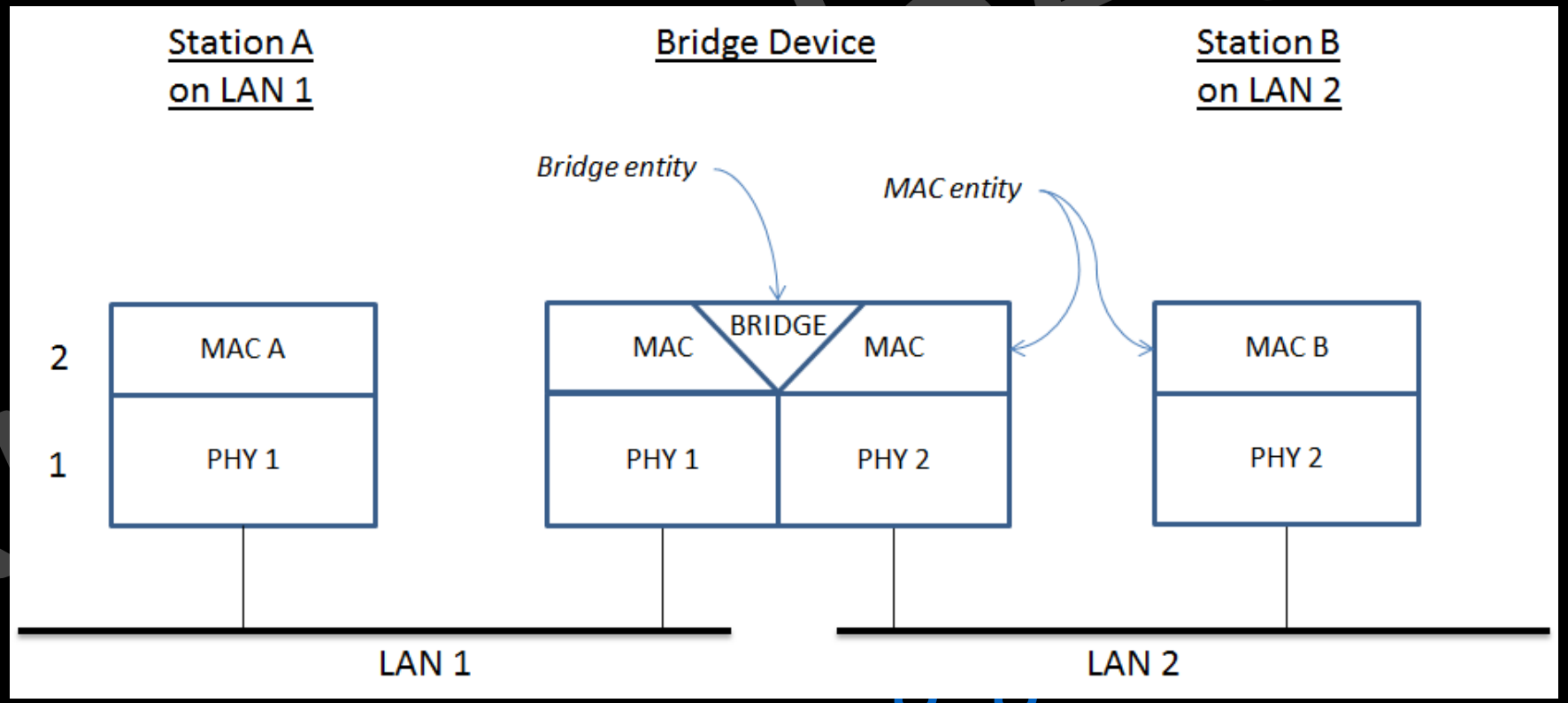


www.knowledgegate.in

Bridge

- Bridge is used to connect two different lan. A **network bridge** is a computer networking device that creates a single, aggregate network from multiple communication networks or network segments. Bridging connects two separate networks as if they were a single network.

- In the OSI model, bridging is performed in the data link layer (layer 2).



Switch

- A network switch connects multiple devices on a network and uses MAC addresses to send data directly to the right device.
- Unlike simpler devices like repeater hubs that send data to all ports, a switch is smarter and only sends data to the specific device it's meant for.
- The most common type of switch is for Ethernet networks, and the first one was made by Kalpana in 1990.

KnowledgeGate

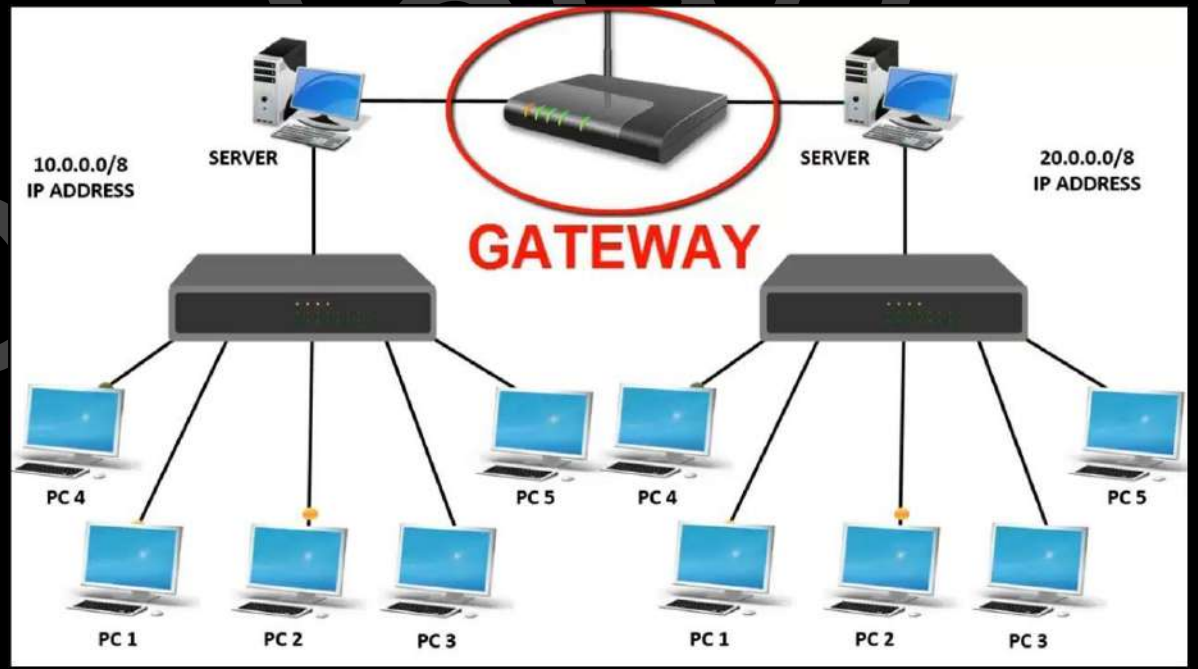
Router

- A router moves data between different computer networks, directing it based on destination information.
- It checks a data packet's header to know where to send it next, using its own set of rules or a "routing table."
- Simple routers are used in homes, while more advanced ones are for big businesses and internet service providers.

KnowledgeGate

Gateway

- A **gateway** is a piece of networking hardware or software used in telecommunications for telecommunications networks that allows data to flow from one discrete network to another.
- Gateways are distinct from routers or switches in that they communicate using more than one protocol to connect multiple networks and can operate at any of the seven layers of the open systems interconnection model (OSI).
- The term gateway can also loosely refer to a computer or computer program configured to perform the tasks of a gateway, such as a default gateway or router.



KnowledgeGate

Module

Free Semester One Shot Exam Course

Topic

Data Structure

Subtopic

DS One Shot

Lesson

Data Structure_Notes

SEMESTER EXAM SERIES

DATA STRUCTURES

IN 7 HOURS + NOTES

KNOWLEDGE GATE

Video Chapters

- **(Ch-1) Introduction** : Basic Terminology, Elementary Data Organization, Built in Data Types in C. Abstract Data Types (ADT)
- **(Ch-2) Array:** Definition, Single and Multidimensional Arrays, Representation of Arrays: Row Major Order, and Column Major Order, Derivation of Index Formulae for 1-D,2-D,3-D and n-D Array Application of arrays, Sparse Matrices and their representations.
- **(Ch-3) Linked lists**: Array Implementation and Pointer Implementation of Singly Linked Lists, Doubly Linked List, Circularly Linked List, Operations on a Linked List. Insertion, Deletion, Traversal, Polynomial Representation and Addition Subtraction & Multiplications of Single variable & Two variables Polynomial.
- **(Ch-4) Stack**: Abstract Data Type, Primitive Stack operations: Push & Pop, Array and Linked Implementation of Stack in C, Application of stack: Prefix and Postfix Expressions, Evaluation of postfix expression, Iteration and Recursion- Principles of recursion, Tail recursion, Removal of recursion Problem solving using iteration and recursion with examples such as binary search, Fibonacci numbers, and Hanoi towers. Trade offs between iteration and recursion.
- **(Ch-5) Queue**: Create, Add, Delete, Full and Empty, Circular queues, Array and linked implementation of queues in C, Dequeue and Priority Queue.
- **(Ch-6) Tree**: Basic terminology used with Tree, Binary Trees, Binary Tree Representation: Array Representation and Pointer(Linked List) Representation, Binary Search Tree, Strictly Binary Tree ,Complete Binary Tree . A Extended Binary Trees, Tree Traversal algorithms: Inorder, Preorder and Postorder, Constructing Binary Tree from given Tree Traversal, Operation of Insertion , Deletion, Searching & Modification of data in Binary Search . Threaded Binary trees, Traversing Threaded Binary trees. Huffman coding using Binary Tree. Concept & Basic Operations for AVL Tree , B Tree & Binary Heaps
- **(Ch-7) Graphs**: Terminology used with Graph, Data Structure for Graph Representations: Adjacency Matrices, Adjacency List, Adjacency. Graph Traversal: Depth First Search and Breadth First Search.
- **(Ch-8) Hashing**: Concept of Searching, Sequential search, Index Sequential Search, Binary Search. Concept of Hashing & Collision resolution Techniques used in Hashing

KNOWLEDGE GATE
http://www.knowledgegate.in/GATE

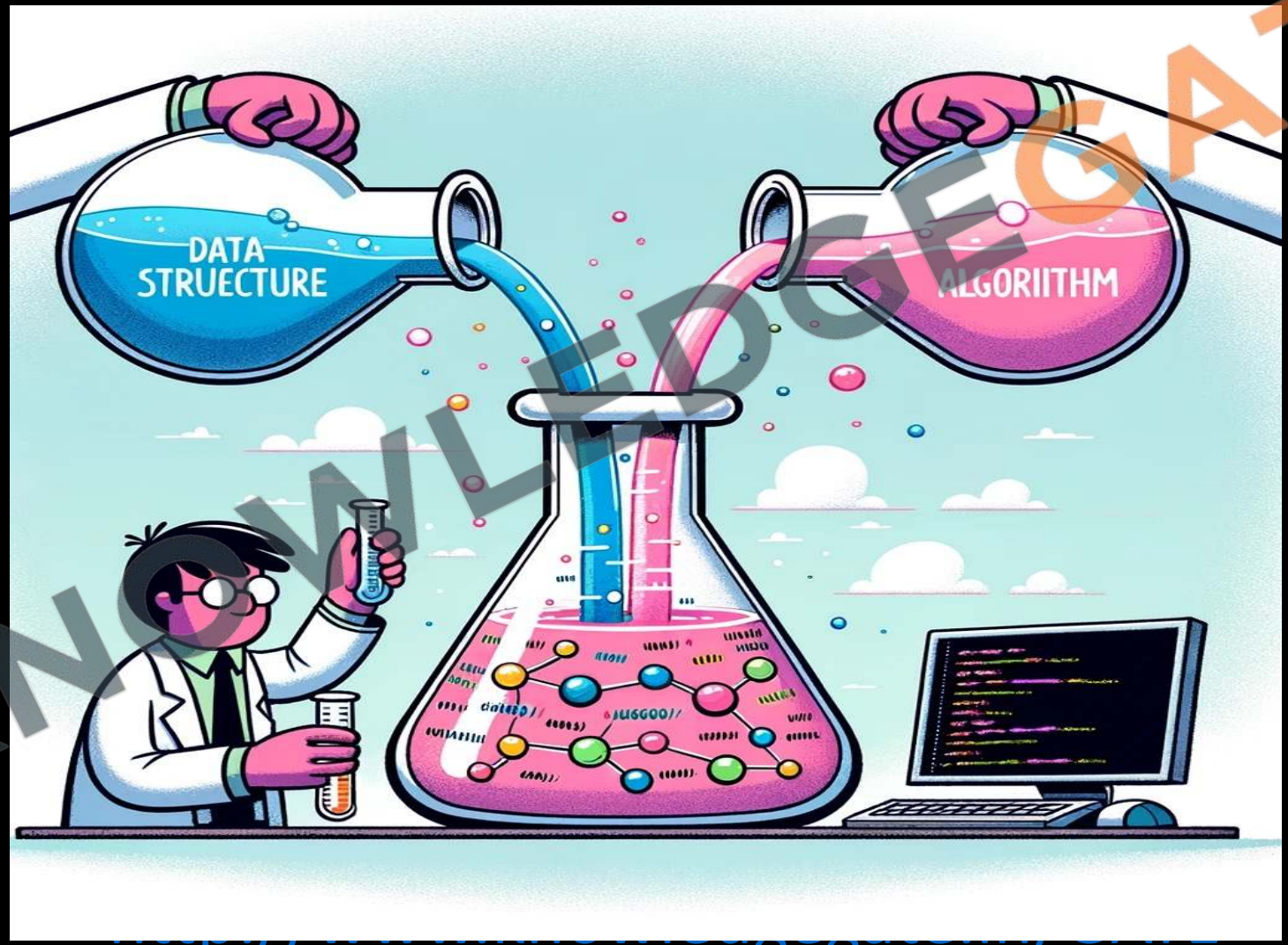
Idea of computer science

- Computer science deals with solving a problem correctly in the form of Algorithm which then can be converted into a program, in most efficient time and memory.
- Problem --> Solution(Algorithm) →Program (Efficient)

KNOWLEDGE GATE

http: GATE

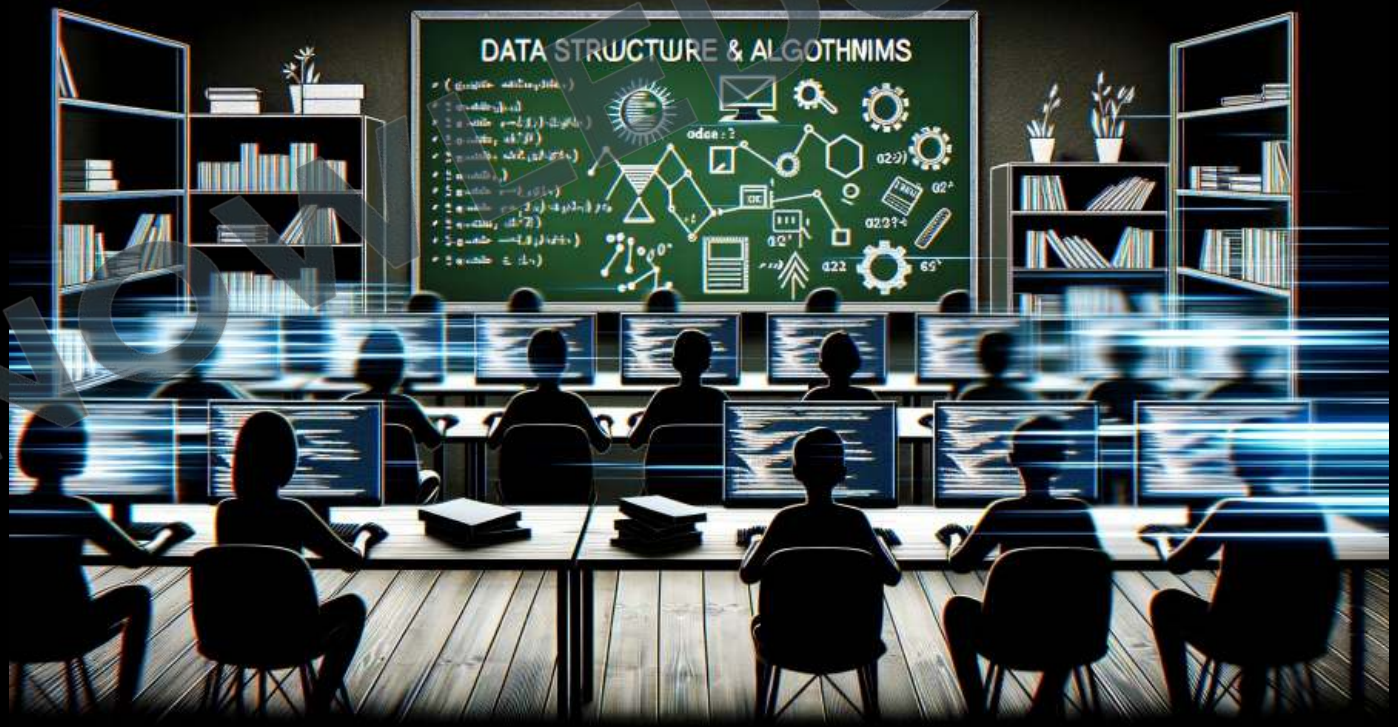
- To write an efficient program we need knowledge of both Data Structures and Algorithms.
  - DATA STRUCTURE + ALGORITHM = PROGRAM



KNOWLEDGEGATE

Why we study data structure and algorithms

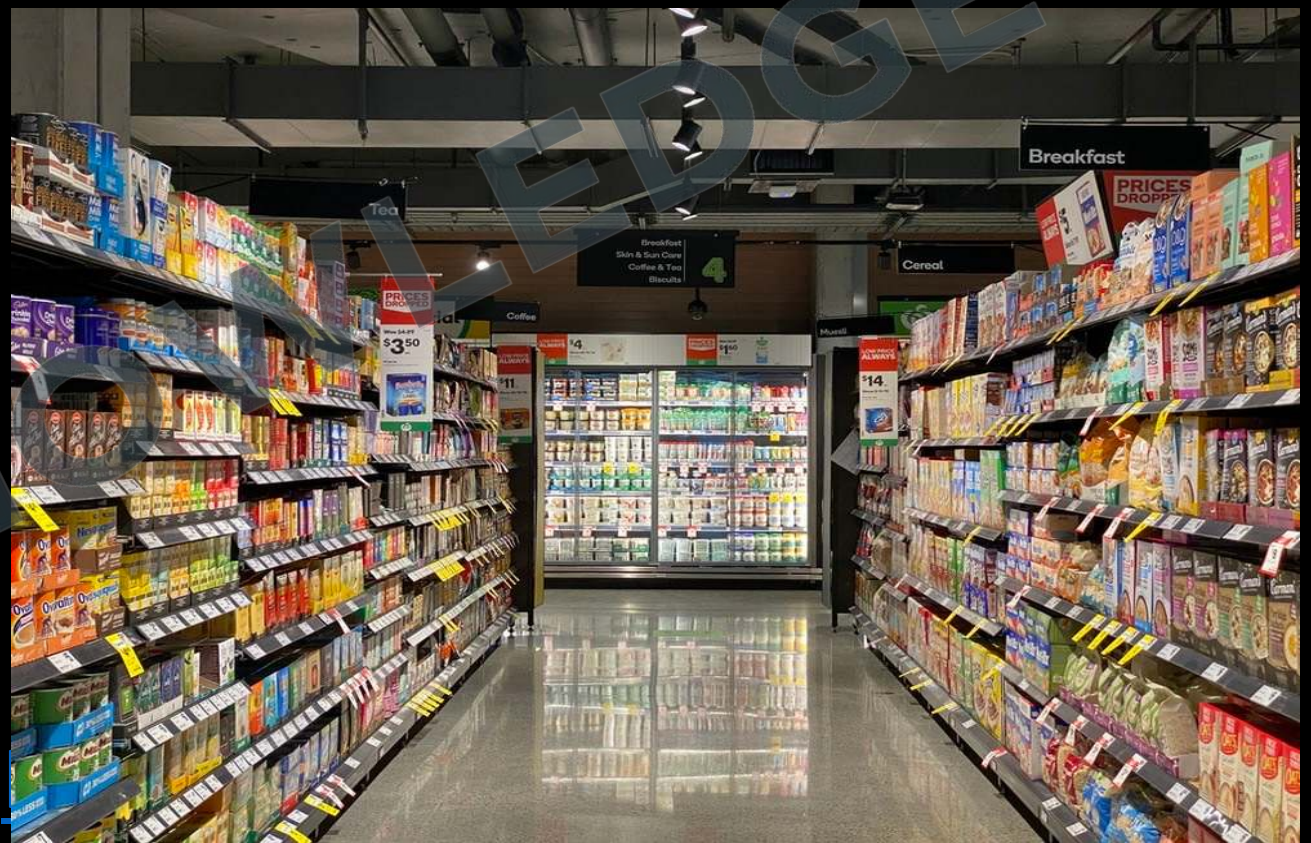
- Course objective is to teach you how to code efficiently.
- What is the meaning of efficiency (time, space, battery, system buses, register etc) time is considered as most important.
- Better running time is obtained from the use of most appropriate data structure and algorithms, rather than through removing a few statements by clever coding.



KNOWLEDGE GATE

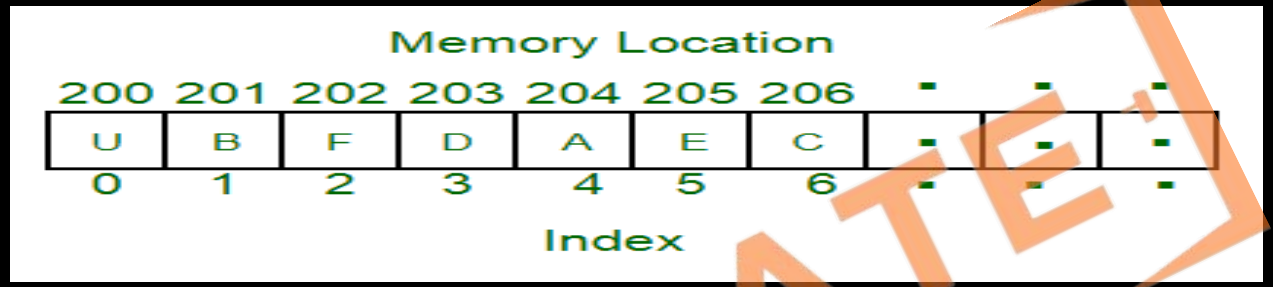
What is data structure

- **Data structure** is a particular way of organizing data in a computer memory (cache, main, secondary) so that Memory can be used efficiently both in terms of time and space.
- It is a logical relationship existing between individual elements of data, it considers elements stored and also their relationship to each other.

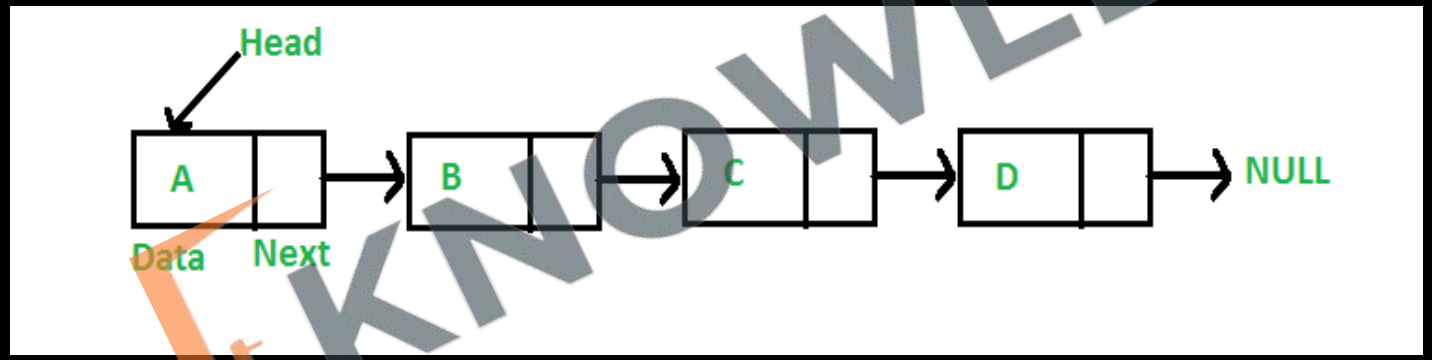


KNOWLEDGE GATE

- Data structure mainly specifies the following four things: -
  - Organization of data
  - Accessing methods
  - Degree of association
  - Processing methods

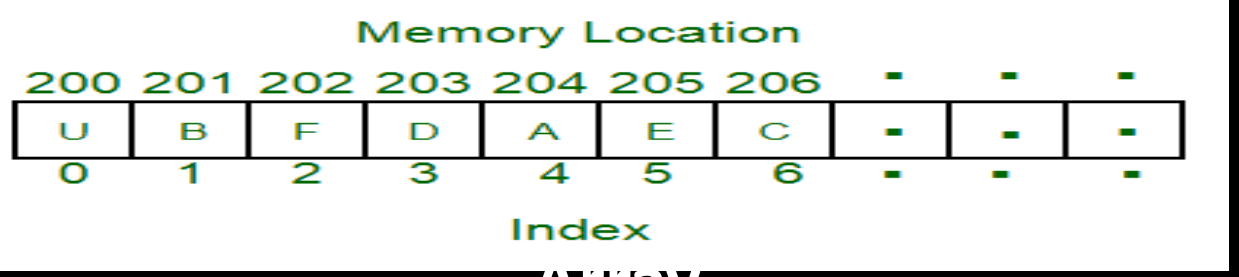


Array

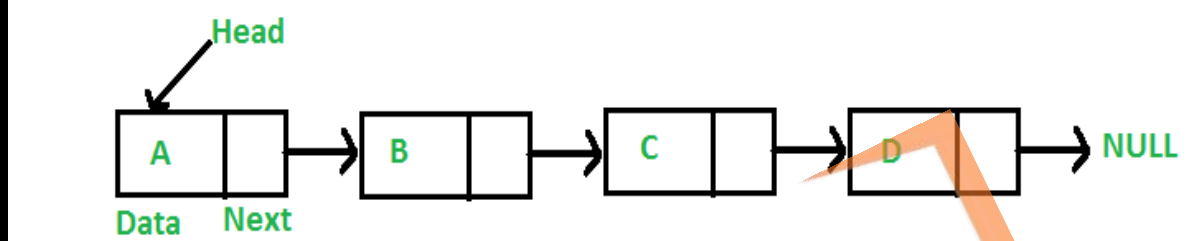


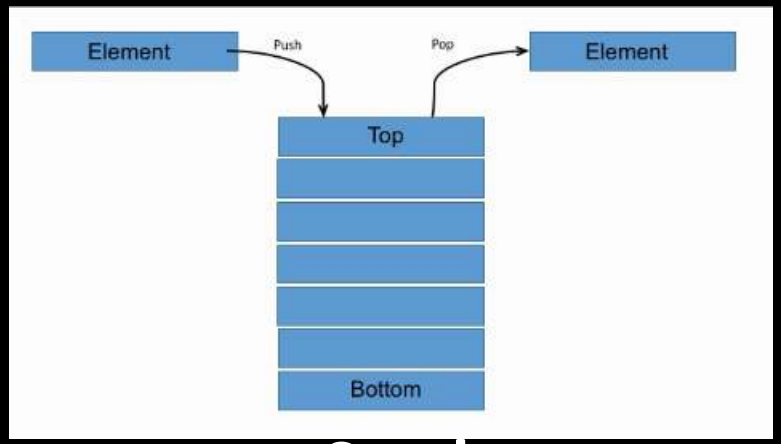
Link List

KNOWLEDGEGATE

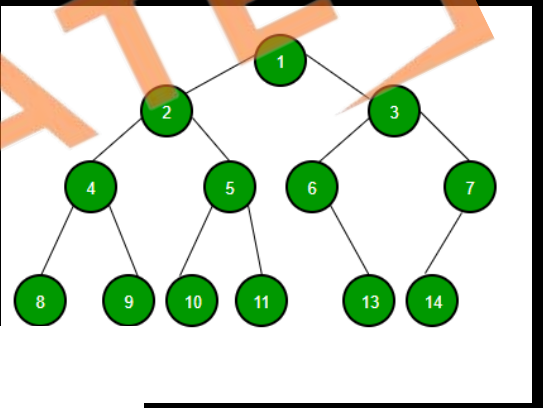


Array

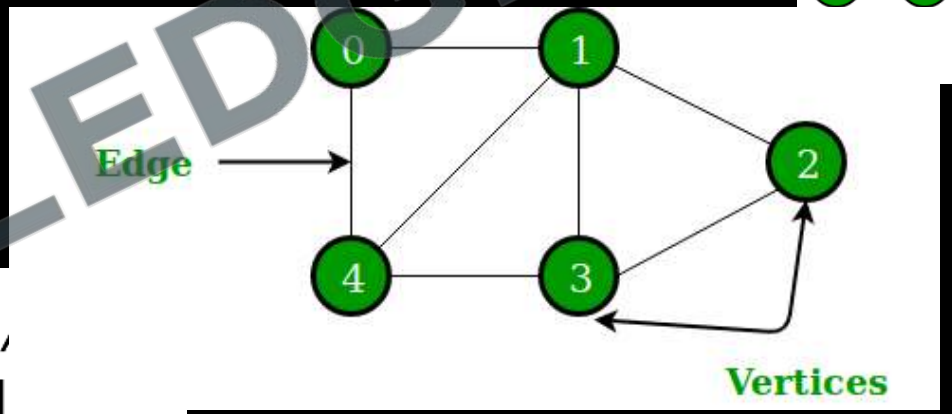




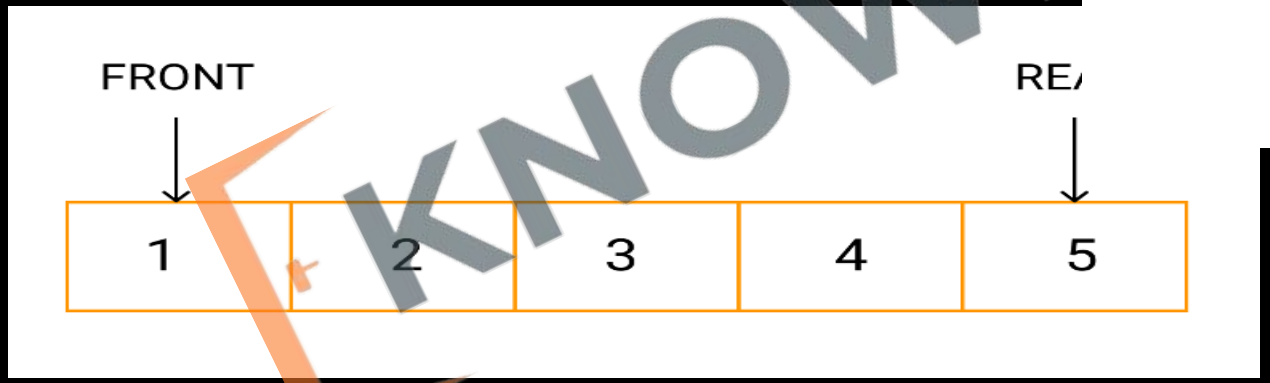
Stack



Tree



Graph

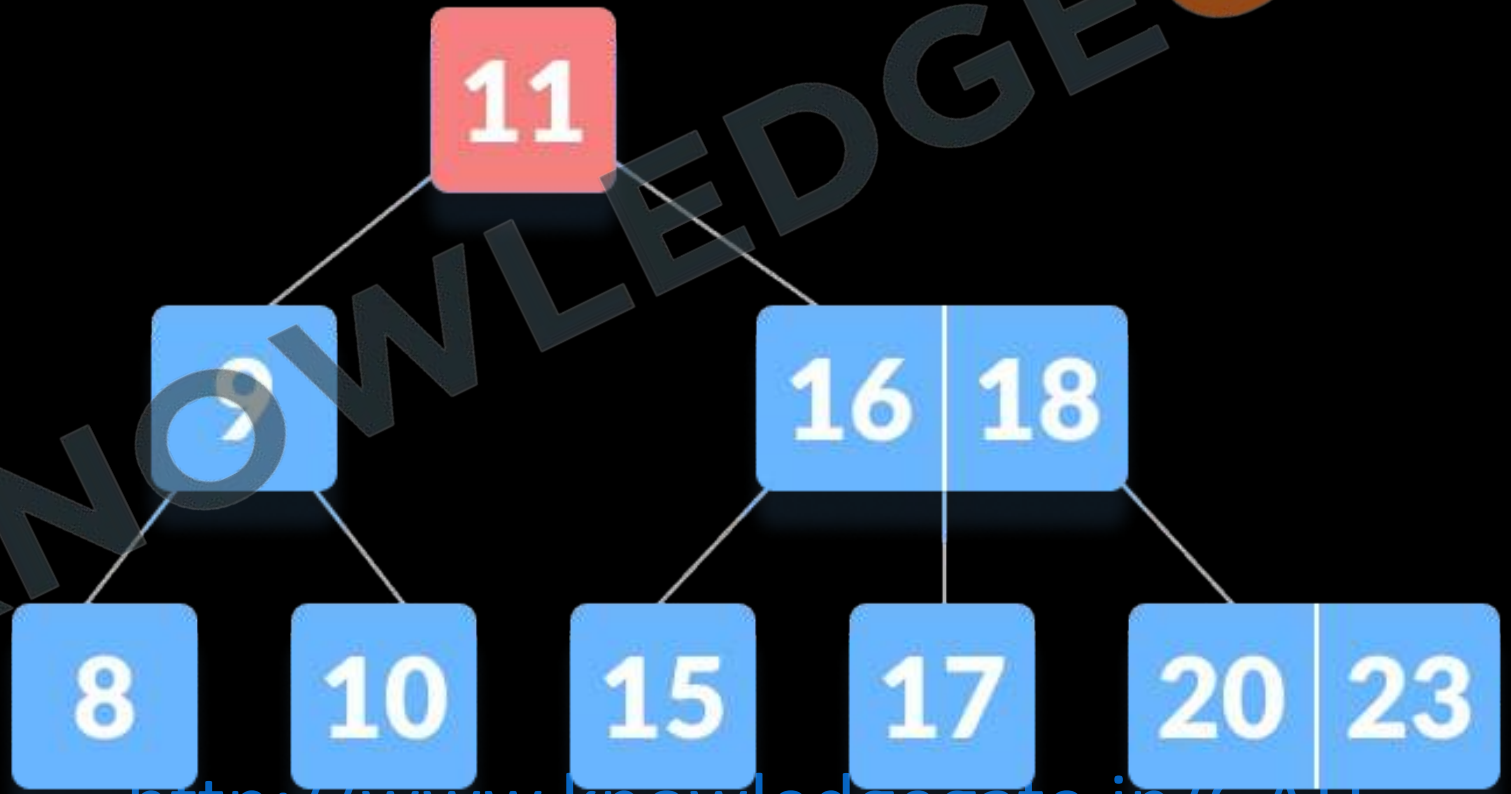


Queue

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Effect of Data Structure

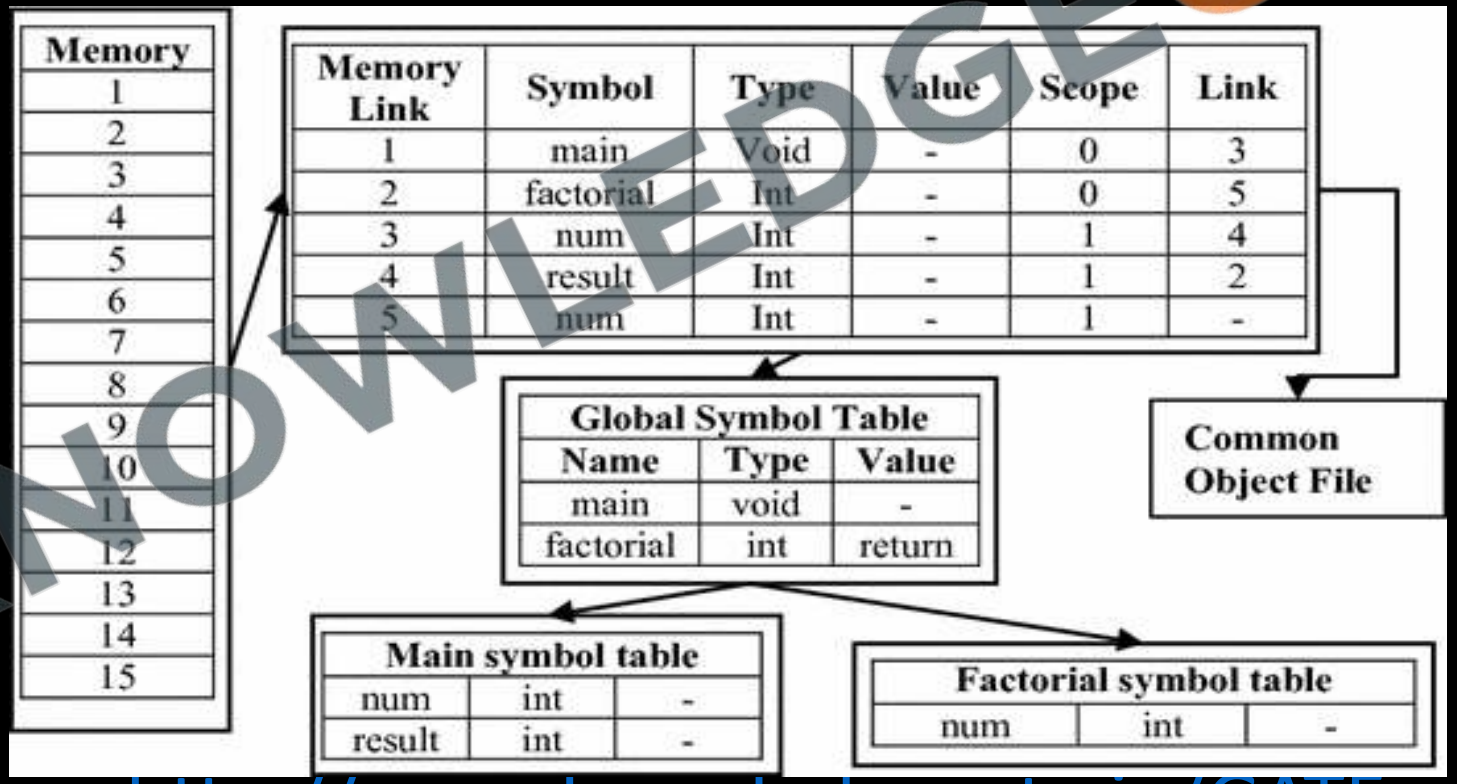
- Data structure effect both the structural and functional aspects of the program.
- Different kinds of data structures are suited to different kinds of applications, and some are highly specialized to specific tasks, For example.
  - Relational databases commonly use B-tree indexes for data retrieval
  - Compiler implementations usually use hash tables to look up identifiers.



KNOWLEDGEGATE

- Usually, efficient data structures are key to designing efficient algorithms. Some formal design methods and programming languages emphasize data structures, rather than algorithms, as the key organizing factor in software design.

- The implementation of a data structure usually requires writing a set of procedures that create and manipulate instances of that structure.

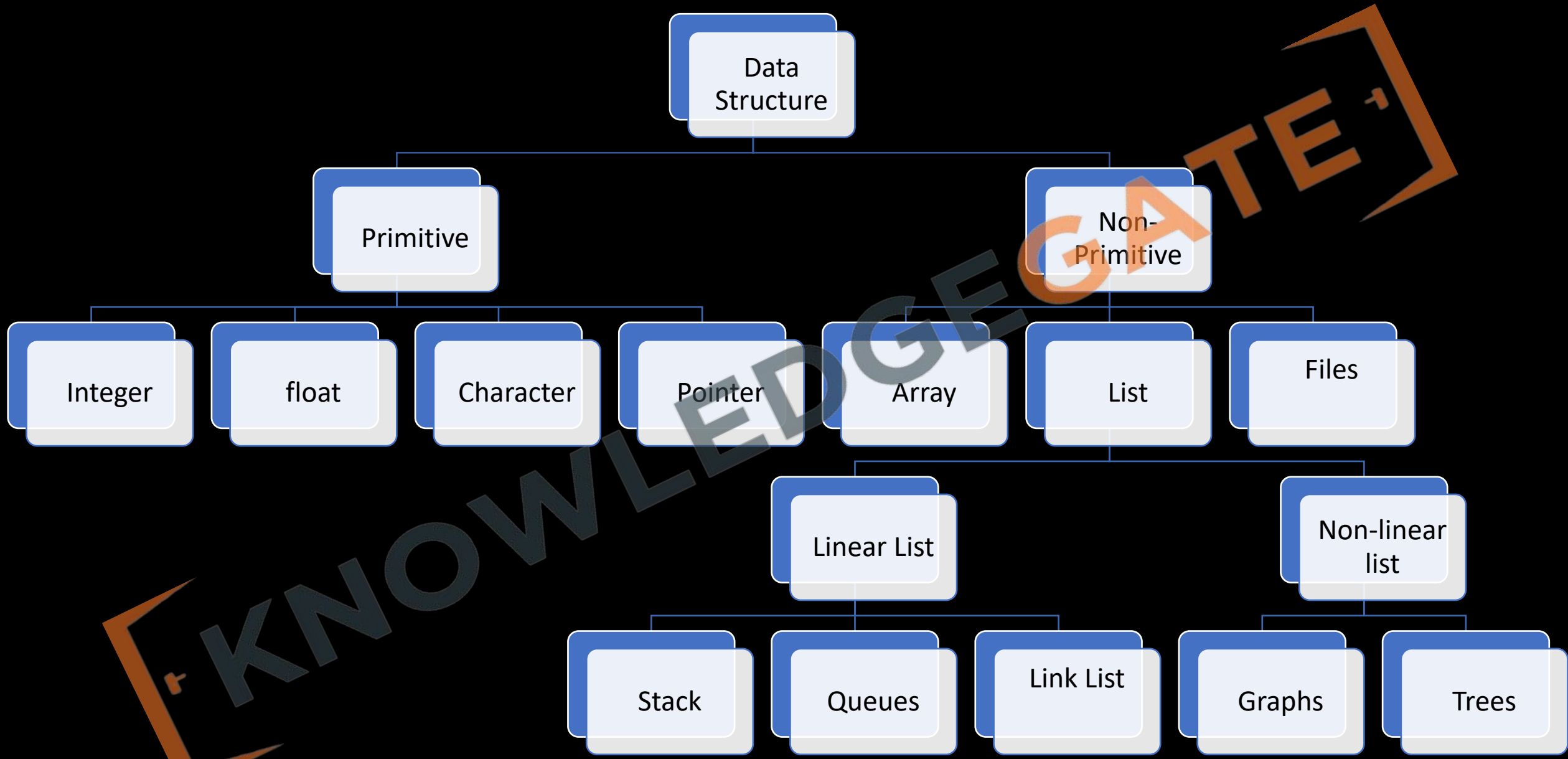


KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Primitive data structure

- Primitive data structures are those which have predefined way of storing data by the system. And the set of operations that can be performed on these data are also predefined. They are directly operated upon by the machine instruction.
- Primitive data structures are char, int, float, double. The predefined operations are addition, subtraction, etc.

KNOWLEDGE GATE



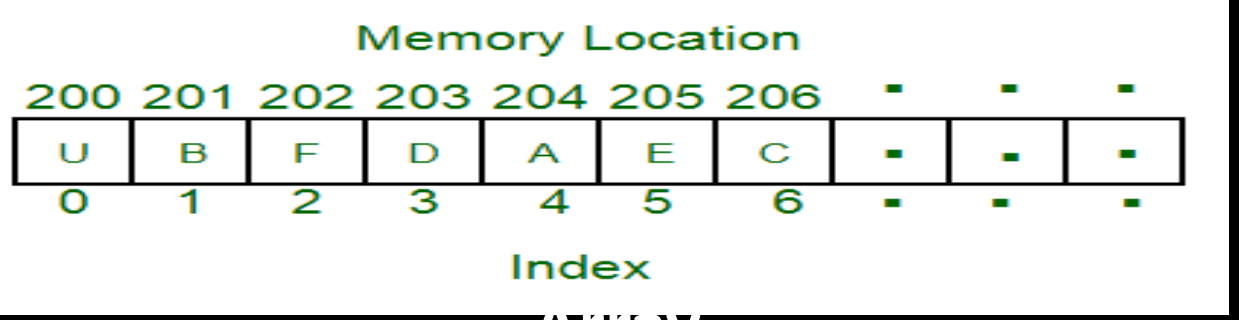
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

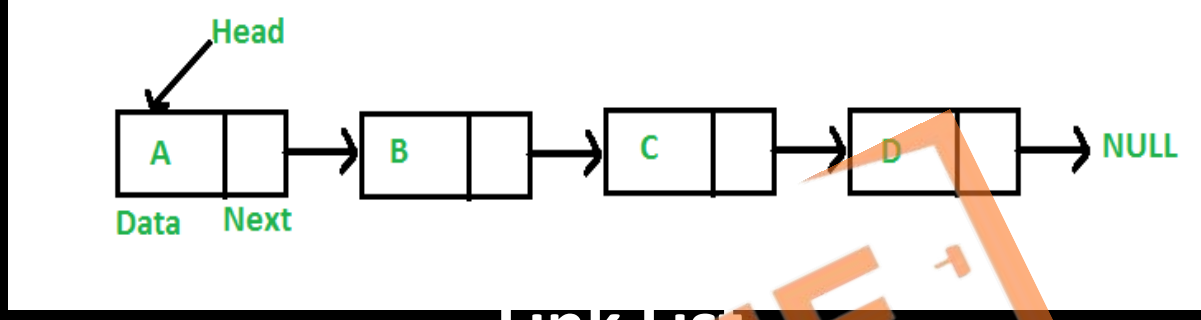
Non-Primitive data structure

- But there are certain situations when primitive data structures are not sufficient for our job. There comes derived data structures and user defined data structures.
- Derived data structures are also provided by the system but are made using primitives like an array. It can be array of chars, array of int, etc. The set of operations that can be performed on derived data structures are also predefined.
- Finally, there are user defined data types which the user defines using the primitive and derived data types using language constructs like structure or class and uses according to their needs. And the user has to define the set or operations that we can perform on them. User defines data types are Linked Lists, Trees, etc.

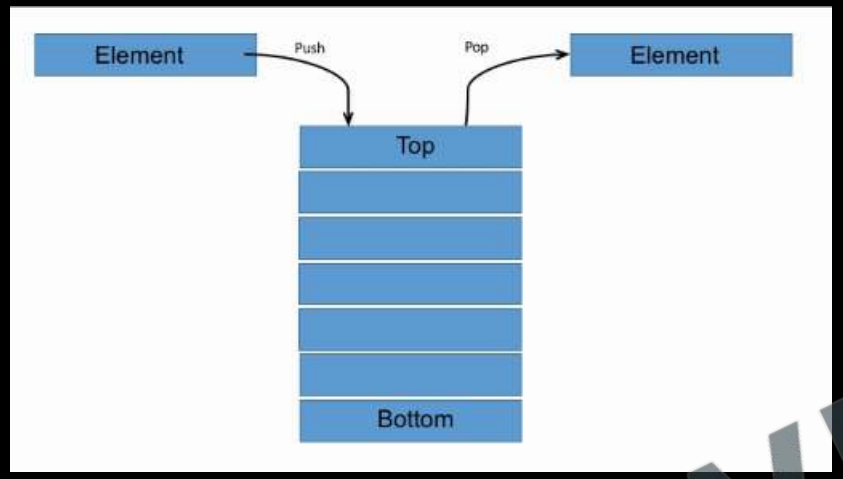
KNOWLEDGEGATE



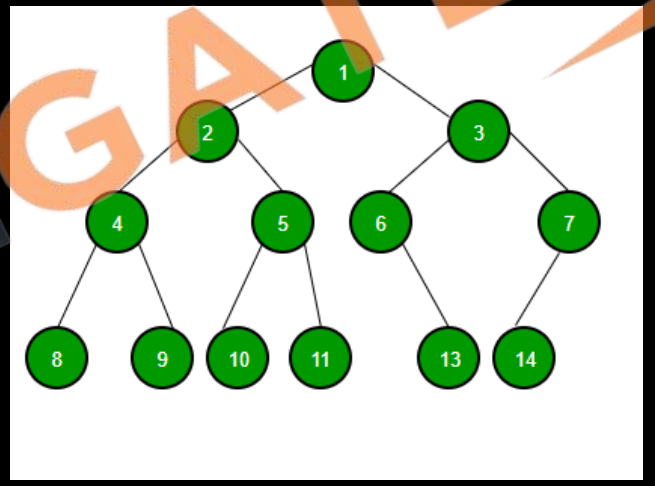
Array



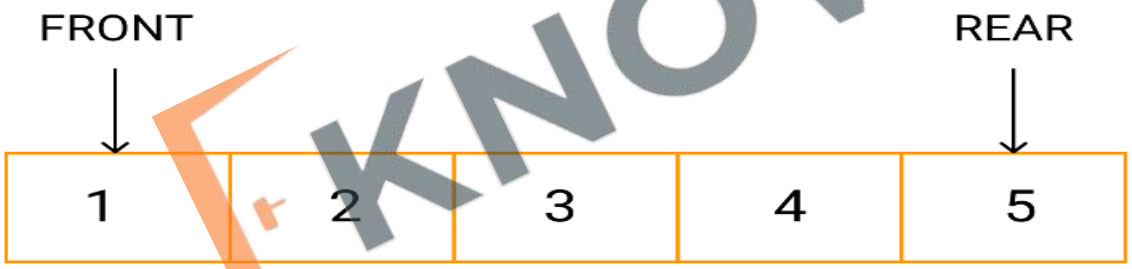
Link List



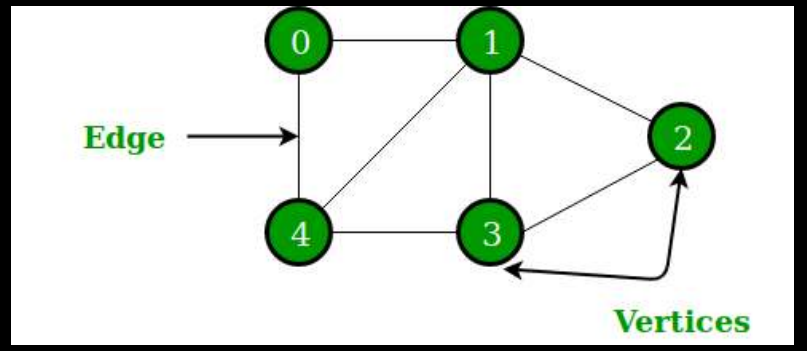
Stack



Tree



Queue



Graph

KNOWLEDGE GATE
http://www.knowledgegate.in/GATE

LINEAR DATA STRUCTURE	NON-LINEAR DATA STRUCTURE
• In a linear data structure, data elements are arranged in a linear order where each and every element are attached to its previous and next adjacent.	• In a non-linear data structure, data elements are attached in hierarchical manner.
• In linear data structure, single level is involved.	• Whereas in non-linear data structure, multiple levels are involved.
• Its implementation is easy in comparison to non-linear data structure.	• While its implementation is complex in comparison to linear data structure.
• In linear data structure, data elements can be traversed in a single run only.	• While in non-linear data structure, data elements can't be traversed in a single run only.
• Its examples are: array, stack, queue, linked list, etc.	• While its examples are: trees and graphs.

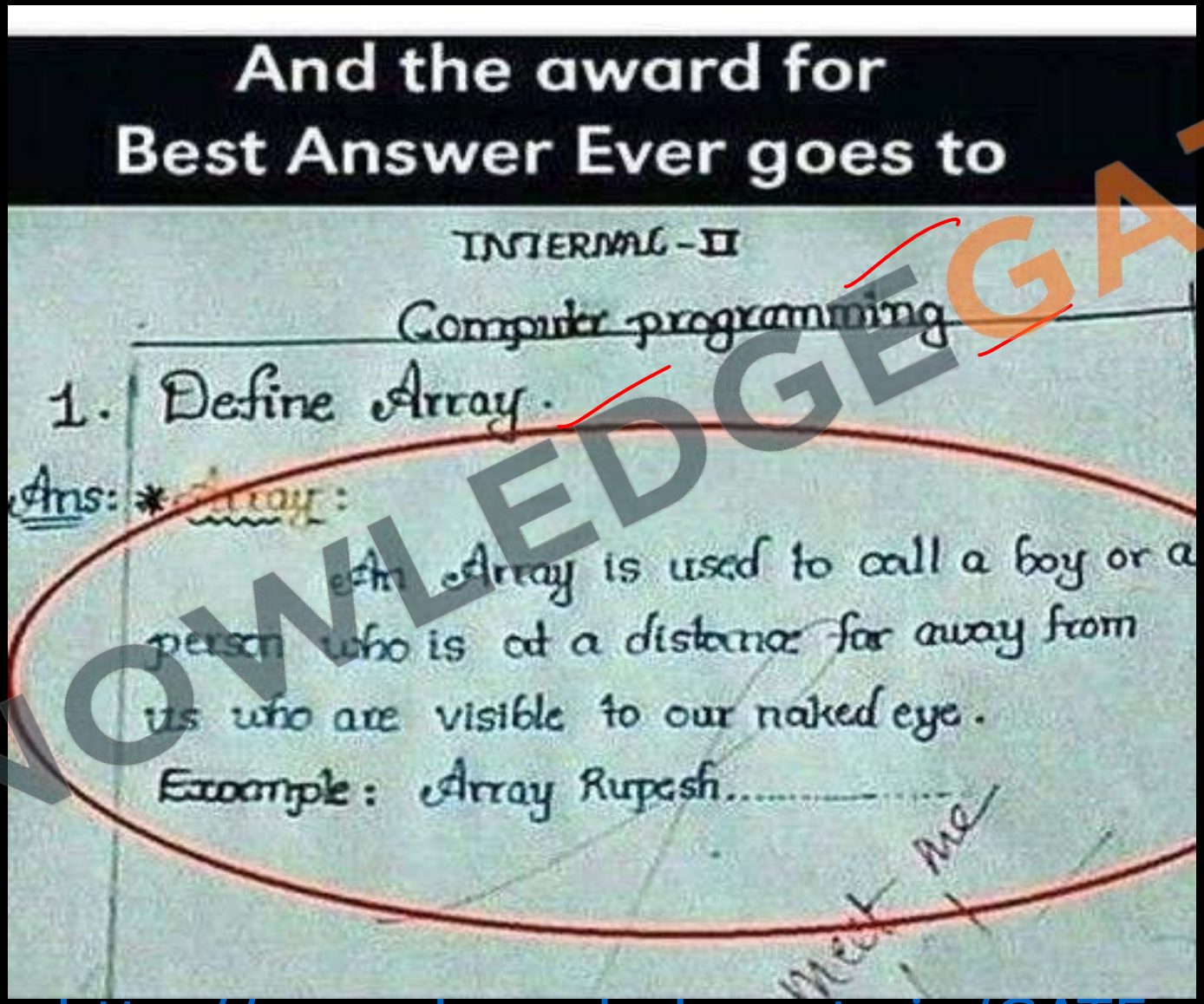
KNOWLEDGE GATE
http://www.knowledgegate.in/GATE

- **Homogeneous data:** Homogeneous data structures are those data structures that contain only similar type of data e.g. like a data structure containing only integer or float values. The simplest example of such type of data structures is an Array.

- **Heterogeneous Data:** Heterogeneous Data Structures are those data structures that contains a variety or dissimilar type of data, for e.g. a data structure that can contain various data of different data types like integer, float and character. The examples of such data structures include structures, union etc.

KNOWLEDGEGATE

Array

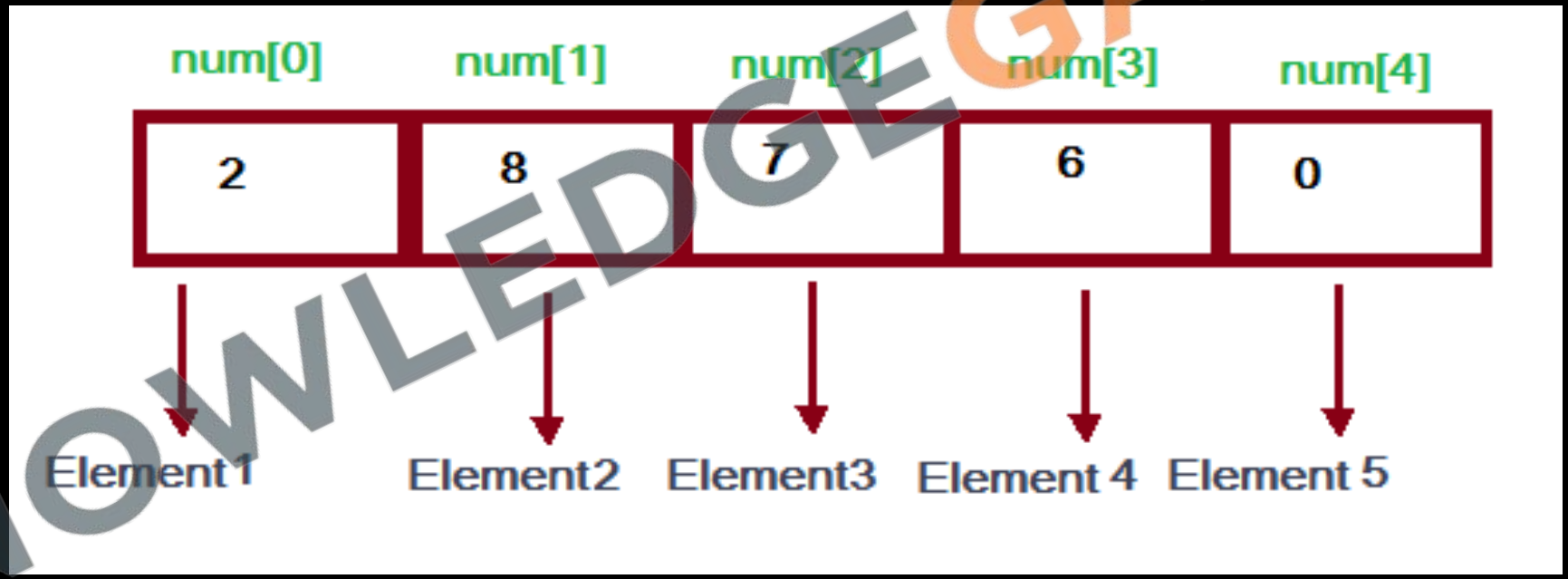


KNOWLEDGEGATE

Array

- An array is a is a data structure that stores collection of elements of same type stored at contiguous memory locations and can be accessed using an index.

int num[5]; ⟶



KNOWLEDGE GATE

How to declare an array in C

- datatype arrayName[array Size];
- int myarray[5];

- In C, the default value of the elements in an array is undefined or garbage. When an array is declared, the memory is allocated for the elements of the array, but the values of those elements are not initialized.
- It is important to note that some programming languages, like Java, automatically initialize the elements of an array to a default value (e.g., 0 for integers, false for booleans, and null for objects) if no initial values are specified.
- Note that in C, you cannot change the size of the array once it has been declared.

KNOWLEDGEGATE

How to initialize an array in C

- dataType arrayName[arraySize] = {$value_1$, $value_2$, ..., $value_N$};
- int myarray[5] = {1, 2, 3, 4, 5};
- You can also initialize an array like this.
  - int myarray[] = {1, 2, 3, 4, 5};
- Here, we haven't specified the size. However, the compiler knows it's size is 5 as we are initializing it with 5 elements.

KNOWLEDGEGATE

Change Value of Array elements

- int myarray[5] = {1, 2, 3, 4, 5};
- make the value of the third element to -1
  - myarray[2] = -1;
- make the value of the fifth element to 0
  - myarray[4] = 0;

KNOWLEDGEGATE

- Arrays have several advantages, like:
  - **Efficient storage and retrieval**: Arrays store elements in contiguous memory locations, which makes it easy to retrieve elements using their index. So very efficient with large amounts of data.
  - **Random access(fast access)**: Arrays allow access to individual elements using their index, which means that accessing any element of the array takes the same amount of time.
  - **Easy to sort and search**: Arrays can be easily sorted and searched using algorithms like binary search, which can be more efficient than searching through unsorted data.
  - **Flexibility**: Arrays can be used to represent a wide variety of data structures, including stacks, queues etc.
  - **Easy to use**: Arrays are a simple and easy-to-use data structure that can be easily understood by programmers of all skill levels.

KNOWLEDGEGATE

- Arrays also have some disadvantages, likes:
  - **Fixed size**: In most programming languages, arrays have a fixed size that cannot be changed once they are created. This can make it difficult to work with data structures that need to grow or shrink dynamically**(Internal Fragmentation) (External Fragmentation)**.
  - **No built-in support for insertion or deletion**: Inserting or deleting an element in an array can be time-consuming and require shifting all the elements after the insertion or deletion point.
  - **Homogeneous elements**: Arrays can only store elements of the same type, which can be limiting for many requirements.
  - **Poor performance for some operations**: Some operations, such as searching or inserting elements in a sorted array, can have poor performance compared to other data structures like hash tables or binary search trees.

KNOWLEDGEGATE

Applications of Arrays

- **Memory Management**: Arrays enable efficient storage of multiple items of the same type, especially when size is known beforehand.
- **Data Representation**: Used for vectors and matrices in mathematical operations like matrix multiplication.
- **Database Management**: Arrays store and manage datasets in relational databases, allowing efficient querying and updates.
- **Implementing Data Structures**: Arrays are foundational for structures like heaps, hash tables, and strings.
- **Caching & Buffering**: Arrays act as buffers in systems, storing data temporarily before writing to slower mediums or transmitting over networks.

KNOWLEDGEGATE

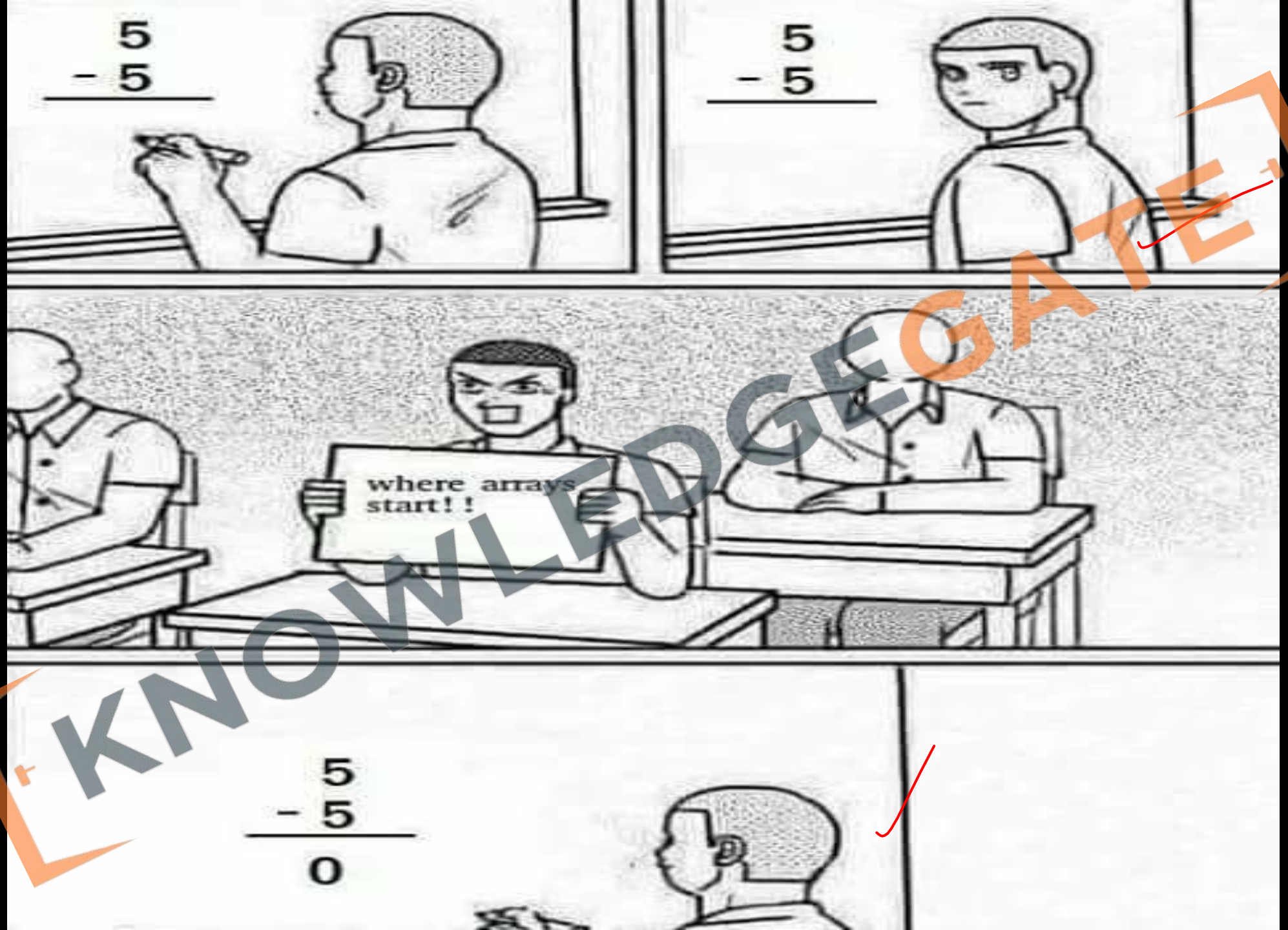
KNOWLEDGE GATE

- Types of indexing in array:
  - 0 (zero-based indexing): The first element of the array is indexed by subscript of 0
  - 1 (one-based indexing): The first element of the array is indexed by subscript of 1
  - n (n-based indexing): The base index of an array can be freely chosen. Usually programming languages allowing n-based indexing also allow negative index values and other scalar data types like enumerations, or characters may be used as an array index.

KNOWLEDGEGATE

-FATHER-

KNOWLEDGEGATE



KNOWLEDGEGATE

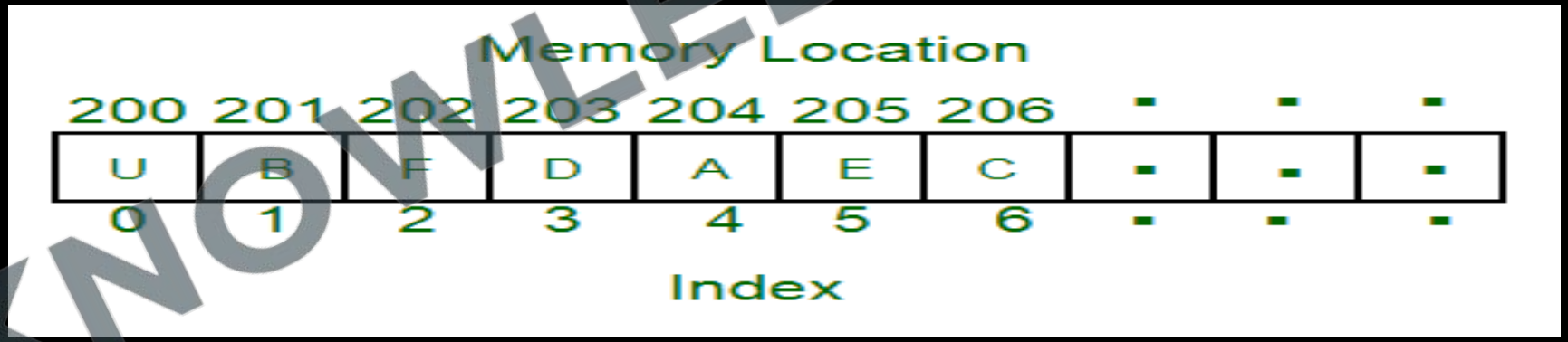
KNOWLEDGEGATE

Size of an array

- Number of elements = (Upper bound – Lower Bound) + 1
  - Lower bound index of the first element of the array
  - Upper bound index of the last element of the array

KNOWLEDGEGATE

- Size = number of elements * Size of each elements in bytes



KNOWLEDGEGATE

One Dimensional array

- Address of the element at k^{th} index
  - a[k] = B + W*k
  - a[k] = B + W*(k – Lower bound)
    - B is the base address of the array
    - W is the size of each element
    - K is the index of the element
    - Lower bound index of the first element of the array
    - Upper bound index of the last element of the array

KNOWLEDGEGATE

Q Let the base address of the first element of the array is 250 and each element of the array occupies 3 bytes in the memory, then address of the fifth element of a one- dimensional array a[10] ?

$$a[k] = B + W*(k - \text{Lower bound})$$

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q An array has been declared as follows
A: array [-6--------6] of elements where every element takes 4 bytes, if the base address of the array is 3500 find the address of array[0]?

$$a[k] = B + W*(k - \text{Lower bound})$$

KNOWLEDGEGATE

Two-Dimensional array

- The two-dimensional array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns.
- However, 2D arrays are created to implement a relational database look a like data structure. It provides ease of holding the bulk of data at once which can be passed to any number of functions wherever required.

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

KNOWLEDGEGATE

Two-Dimensional array

KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

```
data_type array_name[rows][columns];
int disp[2][4] = {
                    {10, 11, 12, 13},
                    {14, 15, 16, 17}
                 };
```

OR

```
int disp[2][4] = { 10, 11, 12, 13, 14, 15, 16, 17};
```

KNOWLEDGEGATE

Implementation of 2D array

- In computing, row-major order and column-major order are methods for storing multidimensional arrays in linear storage such as random access memory.

- The difference between the orders lies in which elements of an array are contiguous in memory.

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

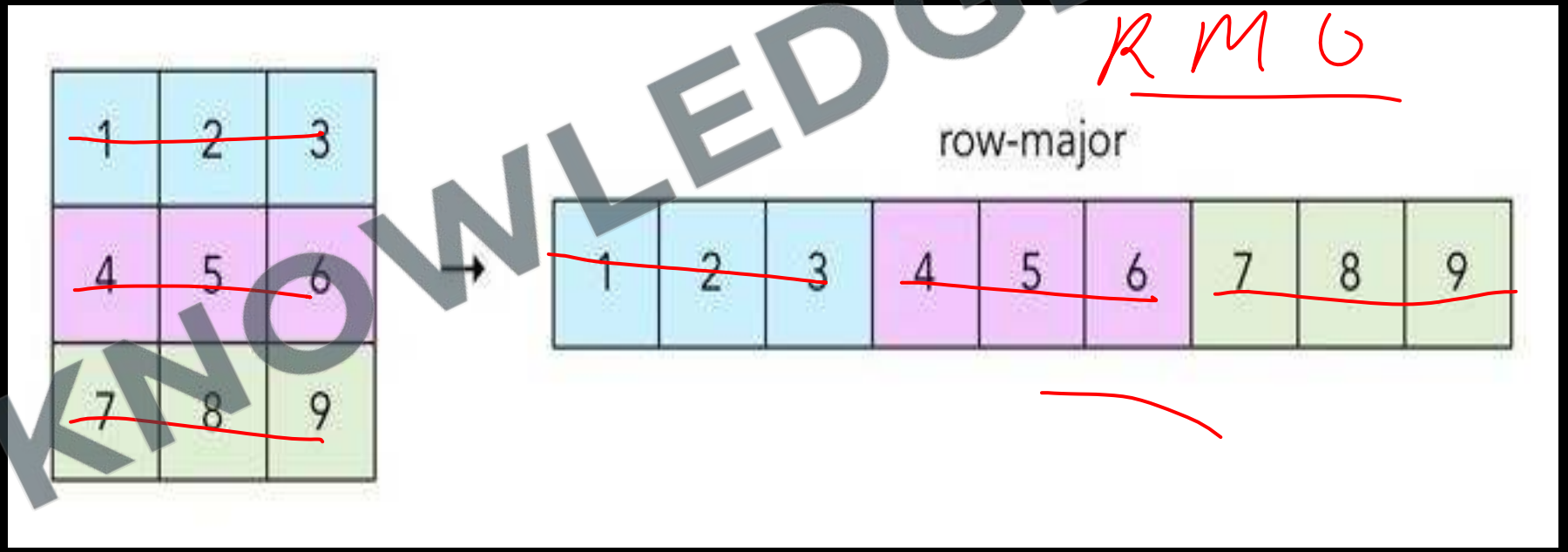
KNOWLEDGEGATE

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Row Major implementation of 2D array

- In Row major method elements of an array are arranged sequentially row by row.
- Thus, elements of first row occupies first set of memory locations reserved for the array, elements of second row occupies the next set of memory and so on.



KNOWLEDGEGATE

Row Major implementation of 2D array

$$\text{Address of } a[i][j] = B + W*[(U_2-L_2+1) (i-L_1) + (j-L_2)]$$

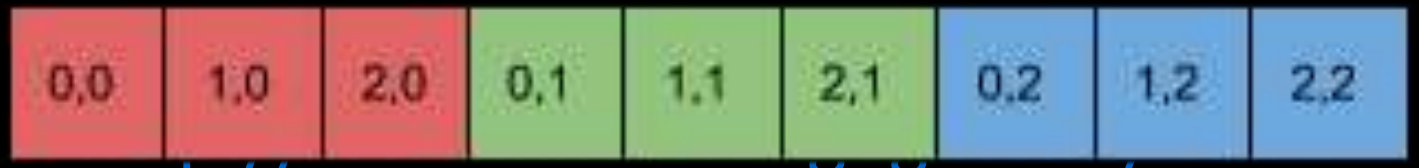
B = Base address
W = Size of each element
L_1 = Lower bound of rows
U_1 = Upper bound of rows
L_2 = Lower bound of columns
U_2 = Upper bound of columns
(U_2-L_2+1) = numbers of columns
$(i-L_1)$ = number of rows before us
$(j-L_2)$ = number of elements before us in current row

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Column Major implementation of 2D array

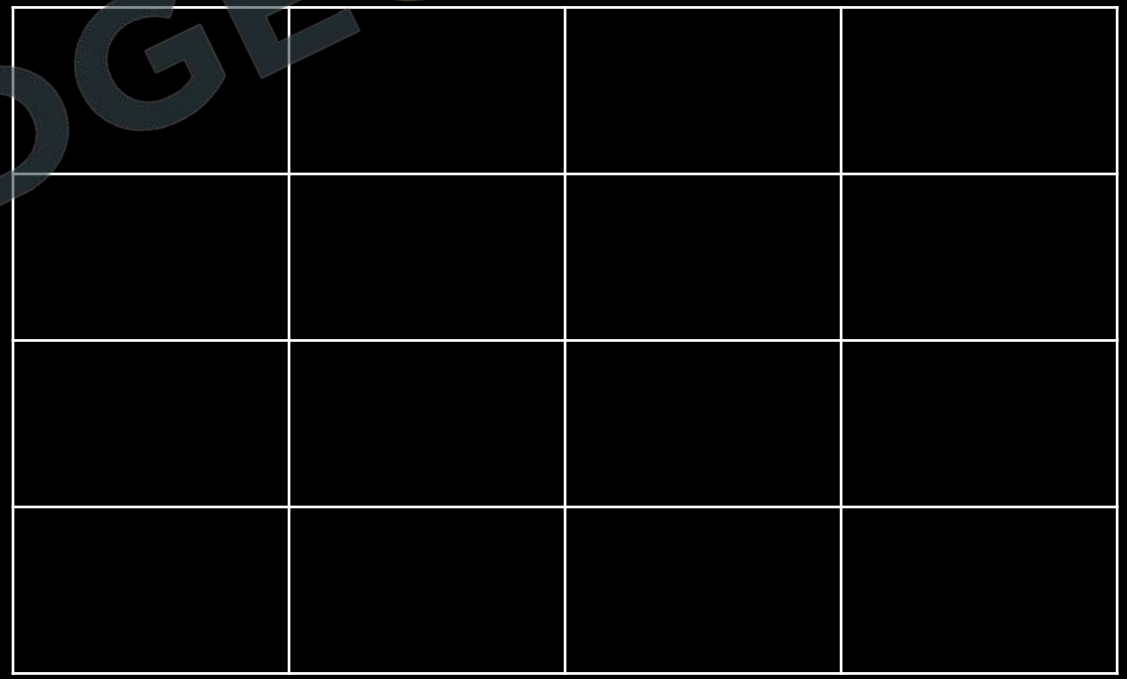
- In Column major method elements of an array are arranged sequentially column by column. Thus, elements of first column occupies first set of memory locations reserved for the array, elements of second column occupies the next set of memory and so on.



KNOWLEDGEGATE

Column Major implementation of 2D array

Address of a[i][j] = B + W*[$(U_1-L_1+1)(j-L_2) + (i-L_1)$]



B = Base address
W = Size of each element
L_1 = Lower bound of rows
U_1 = Upper bound of rows
L_2 = Lower bound of columns
U_2 = Upper bound of columns
(U_1-L_1+1) = numbers of rows
$(j-L_2)$ = number of columns before us
$(i-L_1)$ = number of elements before us in current column

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Q An array VAL[1…15][1…10] is stored in the memory with each element requiring 4 bytes of storage. If the base address of the array VAL is 1500, determine the location of VAL[12][9] when the array VAL is stored

(i) Row wise

Address of a[i][j] = B + W*[$(U_2-L_2+1)(i-L_1) + (j-L_2)$]

(ii) Column wise

Address of a[i][j] = B + W*[$(U_1-L_1+1)(j-L_2) + (i-L_1)$]

KNOWLEDGEGATE

3-Dimensional array

$A([L_1]---[U_1]), ([L_2]---[U_2]), ([L_3]---[U_3])$

Location of A [I, j, k] =

$$B + (i-L_1)(U_2-L_2+1)(U_3-L_3+1) + (j-L_2)(U_3-L_3+1) + (k-L_3)$$

KNOWLEDGE GATE

Q Suppose multidimensional arrays *Q is* declared *Q*(1: 8, – 5: 5, – 10 : 5) stored in column major order

- Find the length of each dimension of *Q*.
- The number of elements in *Q*.
- Assuming base address (*Q*) = 400, *W* = 4, find the effective indices *E*1, *E*2, *E*3 and address of the element *Q*[3, 3, 3].

$$A[I, j, k] = B + (i-L_1)(U_2-L_2+1)(U_3-L_3+1) + (j-L_2)(U_3-L_3+1) + (k-L_3)$$

KNOWLEDGEGATE

N-Dimensional array

A([L_1]---[U_1]), ([L_2]---[U_2]), ([L_3]---[U_3]), ([L_4]---[U_4])--------([L_N]---[U_N])

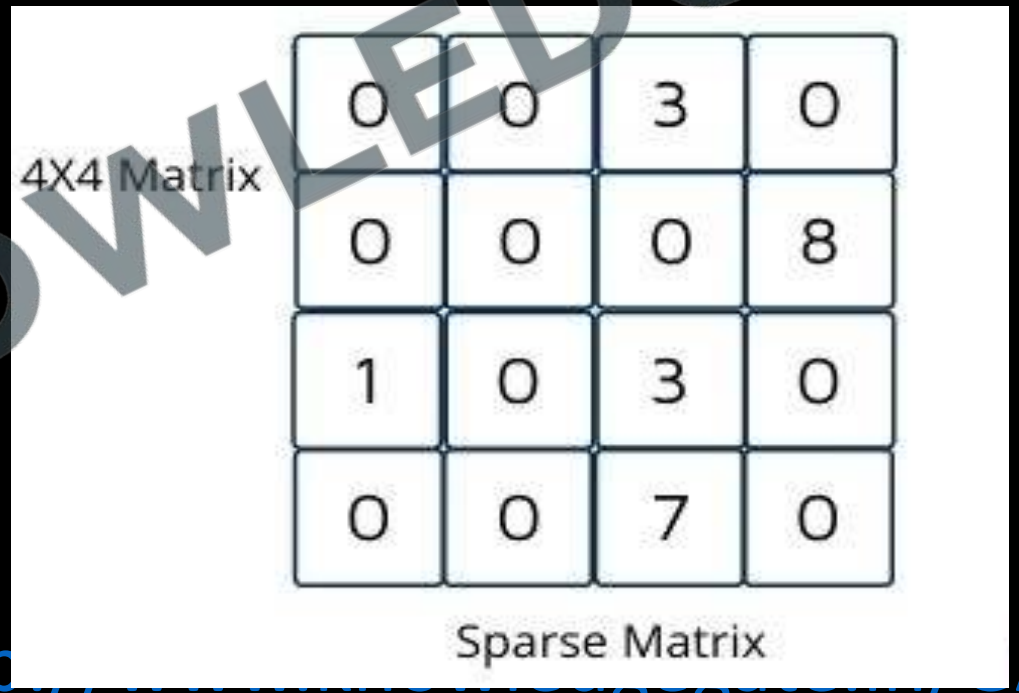
Location of A [I, j, k, ----, x] =

$$B + (i-L_1)(U_2-L_2+1)(U_3-L_3+1)(U_4-L_4+1)\text{----}(U_n-L_n+1)$$
$$+ (j-L_2)(U_3-L_3+1)(U_4-L_4+1)\text{----}(U_n-L_n+1)$$
$$+ (k-L_3)(U_4-L_4+1)\text{----}(U_n-L_n+1)$$
$$+$$
$$+$$
$$+$$
$$+ (x-L_n)$$

KNOWLEDGEGATE

Sparse Matrix

- A matrix is considered sparse if a large number of its elements are zero Conversely, a matrix with most of its elements being non-zero is termed dense.
- Using a sparse matrix over a regular matrix has distinct advantages:
  - **Storage Efficiency**: Given that a majority of the elements are zeros, sparse matrices allow for memory conservation by only storing the non-zero elements.
  - **Computational Speed**: By structuring the data to only account for non-zero elements, operations become faster, as they skip over the zero values.



KNOWLEDGE GATE

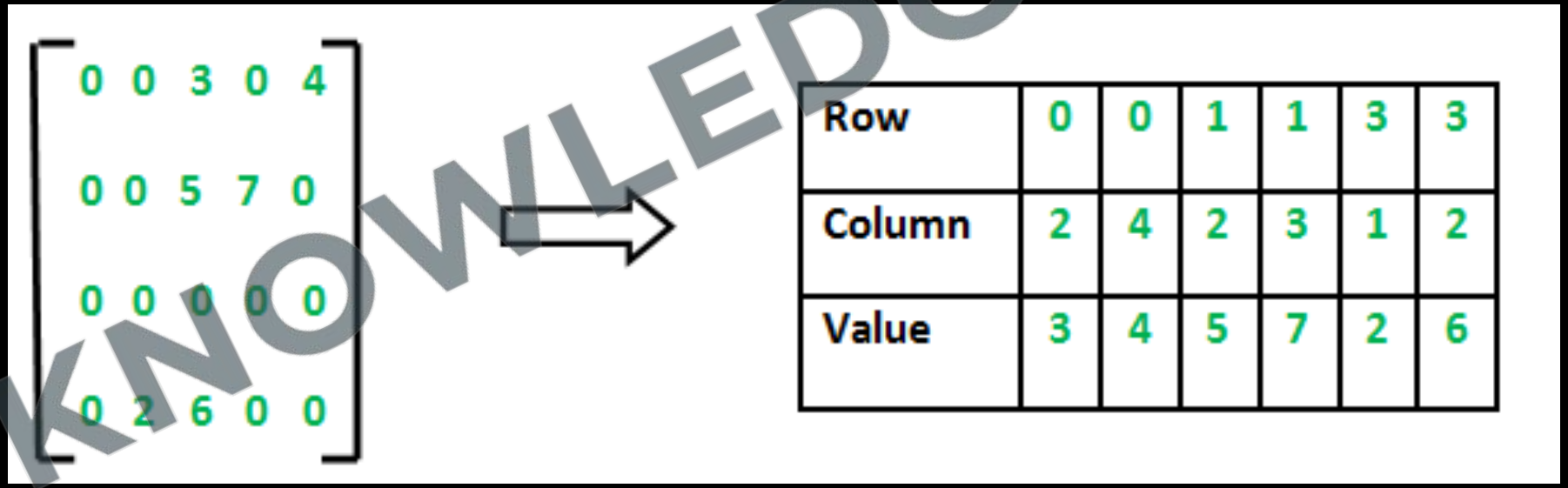
http... ATE

- Sparse Matrix Representations can be done in many ways following are two common representations:
  - Array representation
  - Linked list representation

KNOWLEDGEGATE

Array representation

- 2D array is used to represent a sparse matrix in which there are three rows named as
  - **Row:** Index of row, where non-zero element is located
  - **Column:** Index of column, where non-zero element is located
  - **Value:** Value of the non zero element located at index – (row, column)

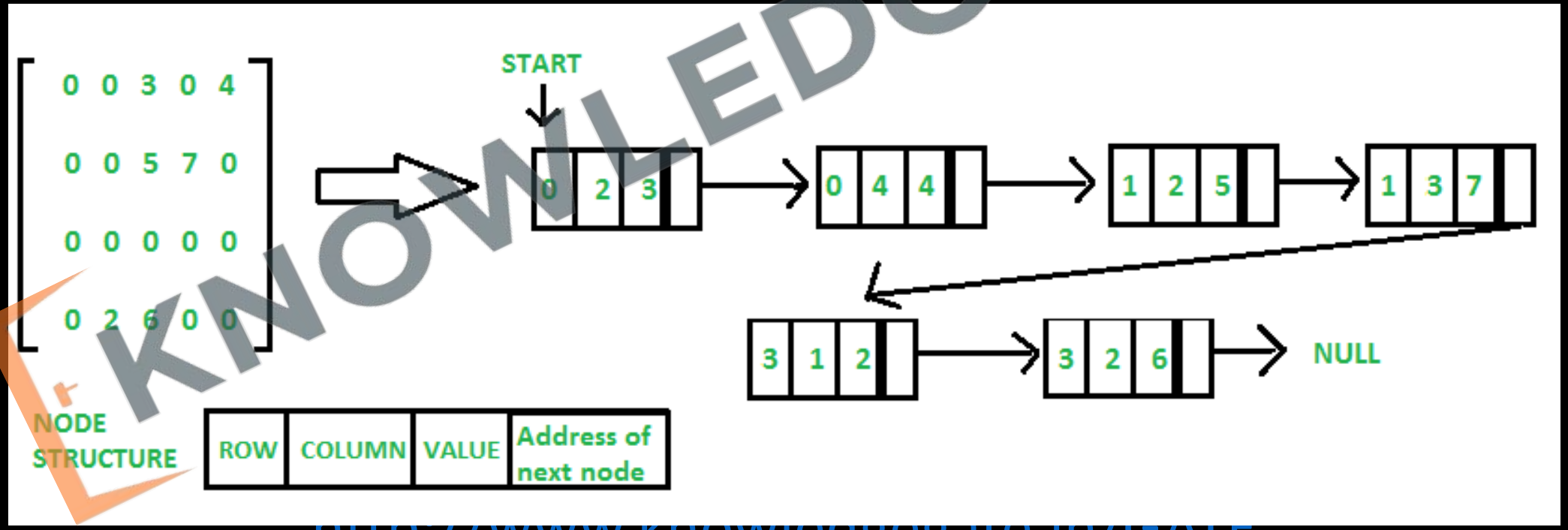


Row	0	0	1	1	3	3
Column	2	4	2	3	1	2
Value	3	4	5	7	2	6

KNOWLEDGEGATE

Linked List Representation

- In linked list, each node has four fields. These four fields are defined as: **Row:** Index of row, where non-zero element is located
  - **Column:** Index of column, where non-zero element is located
  - **Value:** Value of the non zero element located at index – (row,column)
  - **Next node:** Address of the next node



KNOWLEDGEGATE

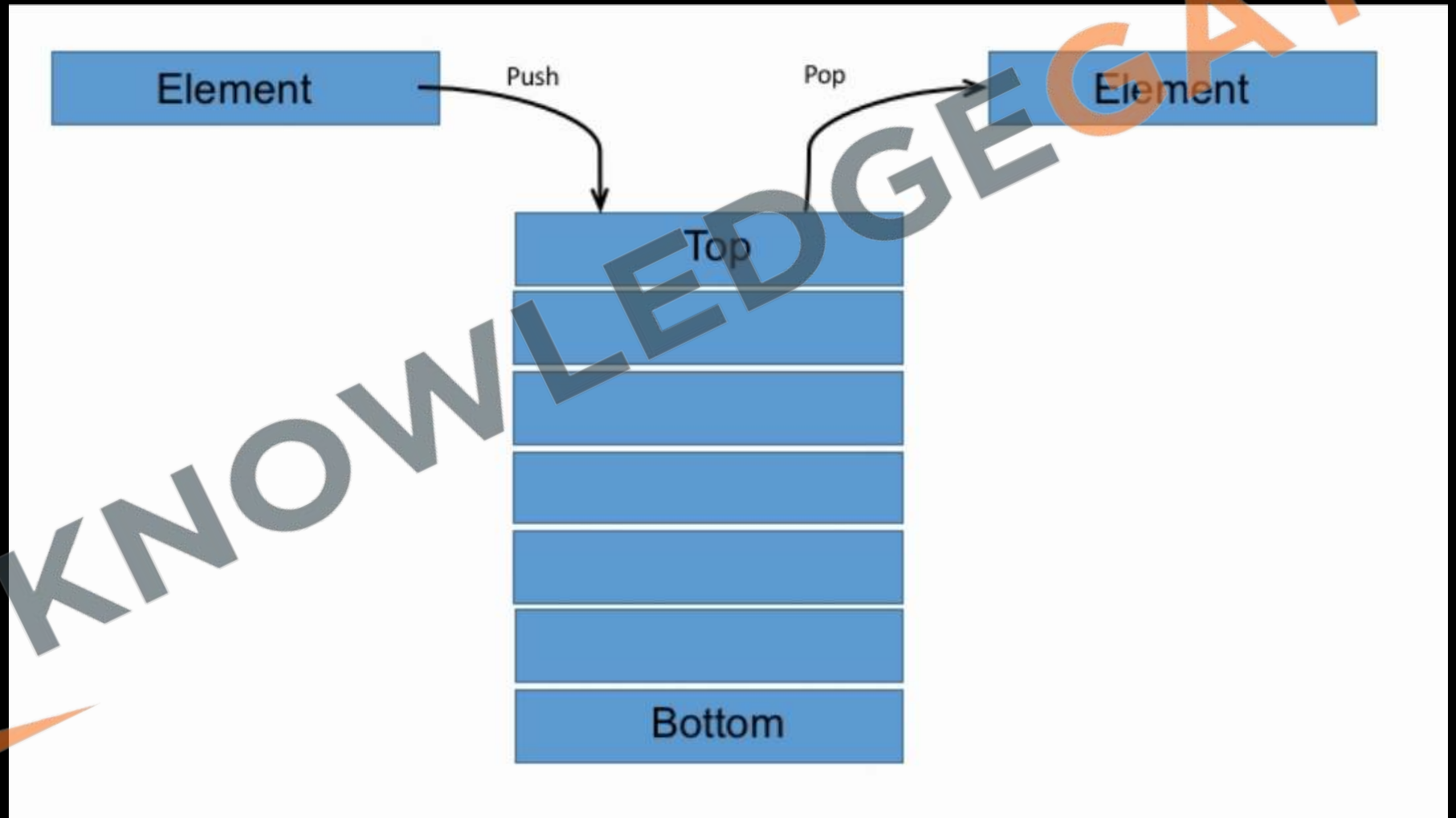
Basics of Stack

Basics of Stack

KNOWLEDGE GATE

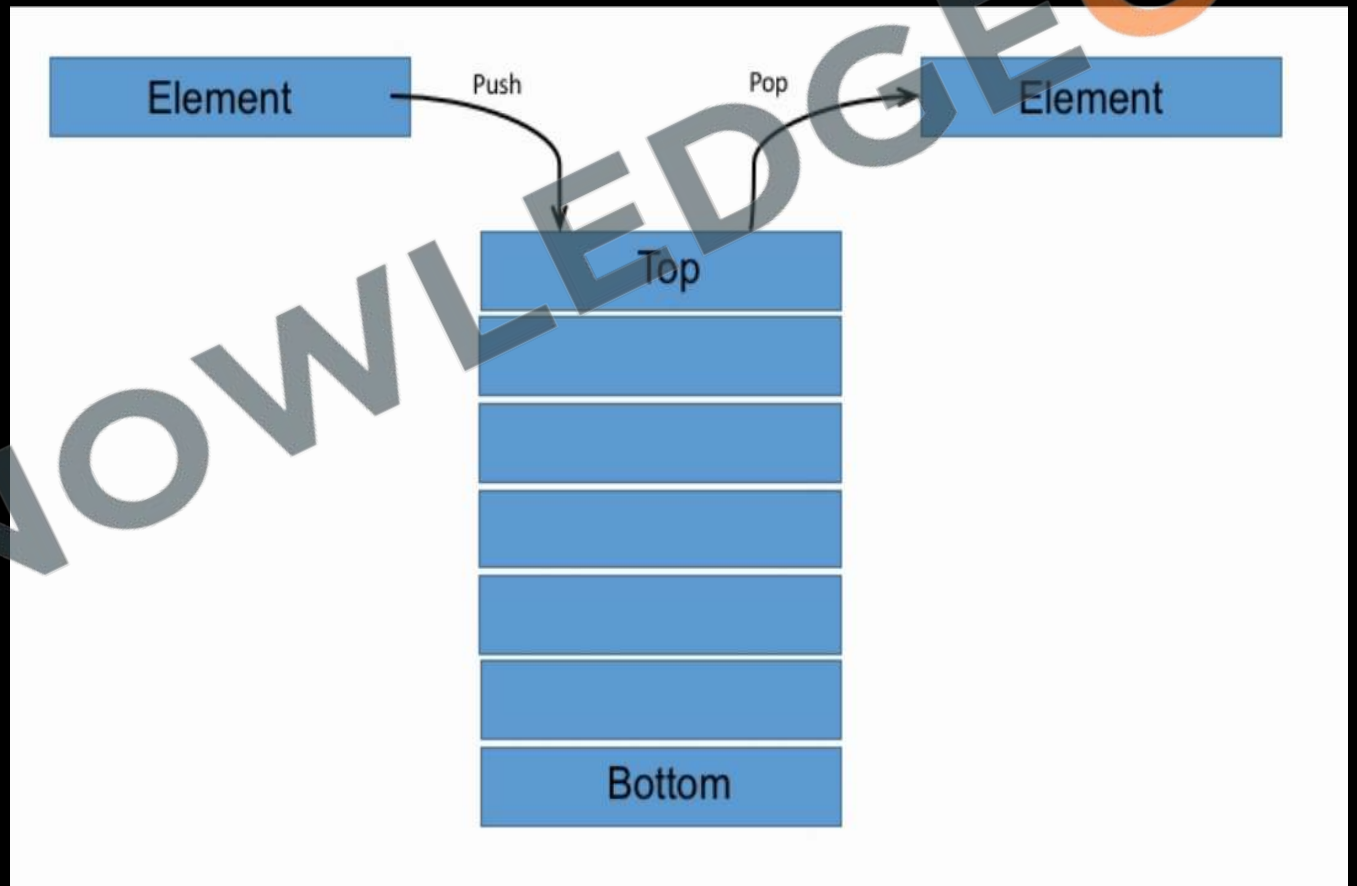
STACK

- A stack is a non-primitive linear data structure. it is an ordered list in which addition of a new data item and deletion of already existing data item is done from only one end known as top of stack (TOS).



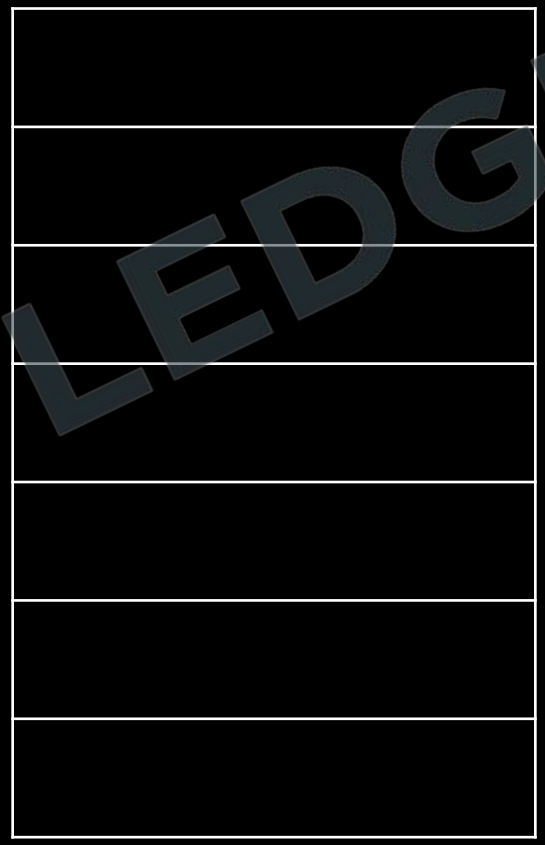
KNOWLEDGEGATE

- The element which is added in last will be first to be removed and the element which is inserted first will be removed in last.
- That is why it is called last in first out (LIFO) or first in last out (FILO) type of list.
- Most frequently accessible element in the stack is the top most element, whereas the least accessible element is the bottom of the stack.



KNOWLEDGEGATE

Q Choose the correct alternatives (more than one may be correct) and write the corresponding letters only: The following sequence of operations is performed on a stack: PUSH (10), PUSH (20), POP, PUSH (10), PUSH (20), POP, POP, POP, PUSH (20), POP The sequence of values popped out is ?



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Applications of Stack

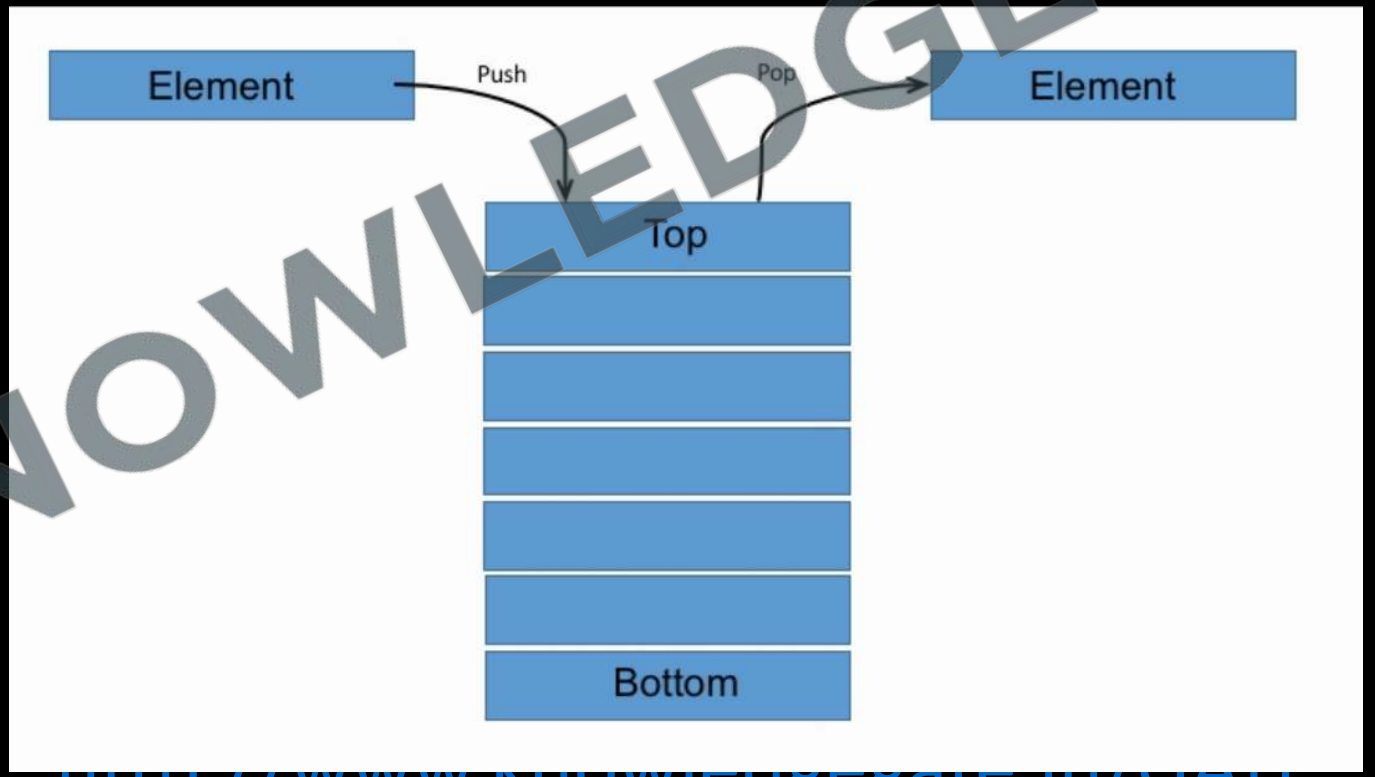
- **Expression Parsing**: Stacks help evaluate and check programming expressions, ensuring balanced parentheses.
- **Backtracking**: Used in algorithms like maze-solving and the "Eight Queens" puzzle.
- **Function Calls**: Manage function details during calls in programming languages.
- **Undo Feature**: Implement undo in text editors and browsers.
- **Syntax Checking**: Compilers use stacks to match syntax elements like 'if' with 'else'.

KNOWLEDGEGATE

Stack Implementation

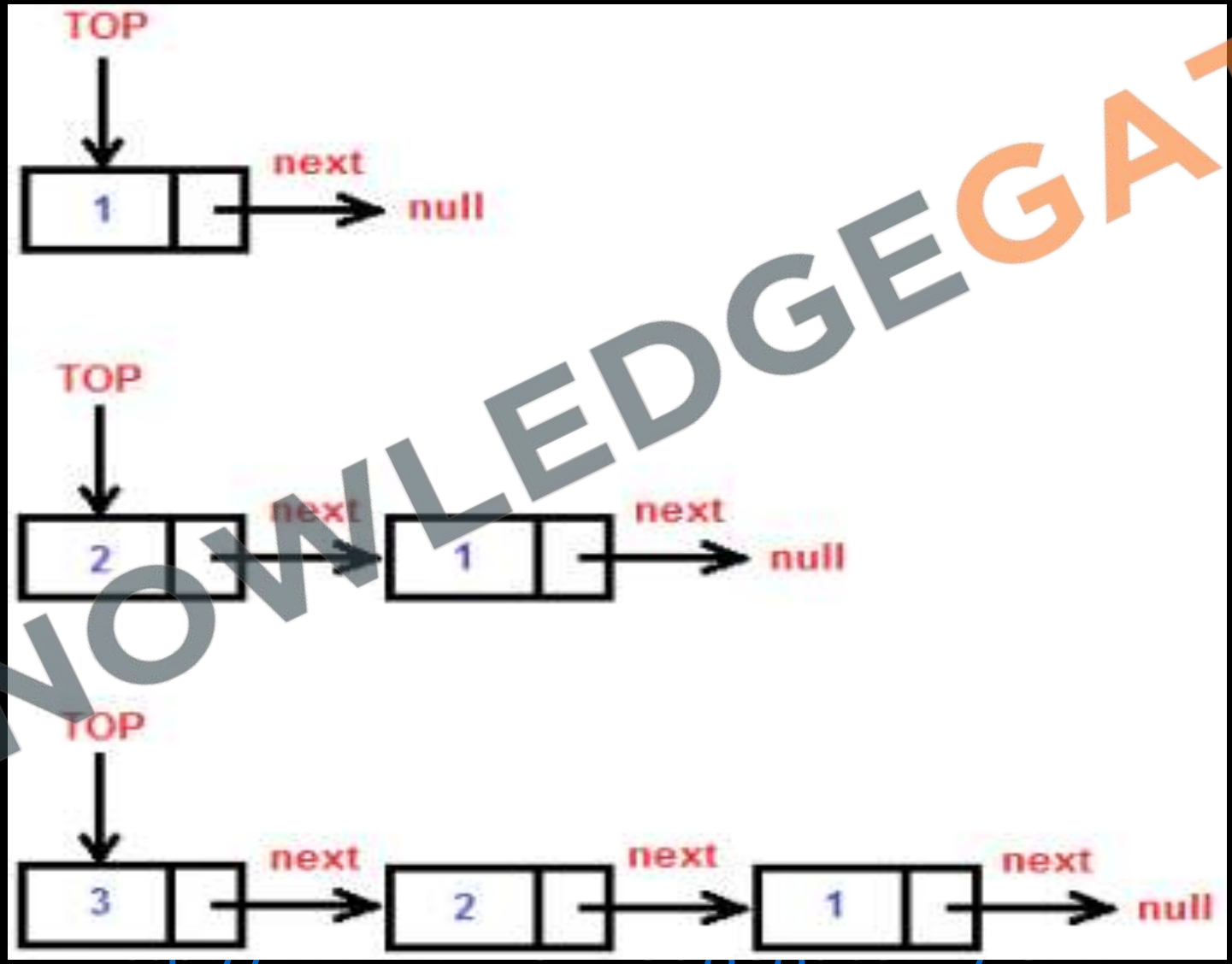
Stack is generally implemented in two ways.

- **Static Implementation**: - Here array is used to create stack. it is a simple technique but is not a flexible way of creation, as the size of stack has to be declared during program design, after that size implementation is not efficient with respect to memory utilization.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

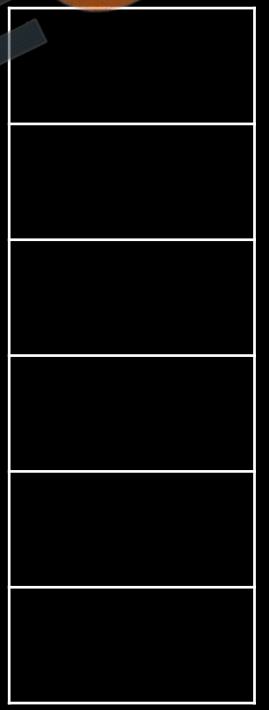
- **Dynamic implementation**: - It is also called linked list representation and uses pointer to implement the stack type of data structure.



KNOWLEDGEGATE

Push operation: - The process of adding new element to the top of stack is called push operation. the new element will be inserted at the top after every push operation the top is incremented by one. in the case the array is full and no new element can be accommodated it is called over-flow condition.

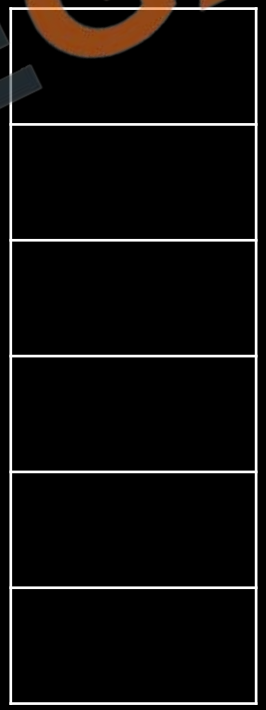
```
PUSH (S, N, TOP, x)
{
    if (TOP==N-1)
        Print stack overflow and exit
    TOP = TOP + 1
    S[TOP] = x
    exit
}
```



KNOWLEDGEGATE

Pop: - The process of deleting an element. from the top of stack is called POP operation, after every POP operation the stack is decremented by one if there is no element in the stack and the POP operation is requested then this will result into a stack underflow condition.

```
POP (S, N, TOP)
{
    if (TOP==-1)
        print underflow and exit
    y = S[TOP]
    TOP=TOP-1
    return(y) and exit
}
```



KNOWLEDGEGATE

```
typedef struct
{
    int arr[MAX_SIZE];
    int top;
} Stack;


void initialize(Stack *s)
{
     s->top = -1;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
int isEmpty(Stack *s)
{
    return s->top == -1;
}


int isFull(Stack *s)
{
    return s->top == MAX_SIZE - 1;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void push(Stack *s, int item)
{
    if (isFull(s))
    {
        printf("Stack is full!\n");
        return;
    }
     s->arr[++(s->top)] = item;
}
```

KNOWLEDGEGATE

```
int pop(Stack *s)
{
   if (isEmpty(s))
   {
      printf("Stack is empty!\n");
      exit(1);
   }
    return s->arr[(s->top)--];
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```c
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 100
int main()
{
    Stack s;
    initialize(&s);
    return 0;
}
```

KNOWLEDGEGATE

```
typedef struct Node
{
    int data;
    struct Node* next;
} Node;

typedef struct
{
    Node* top;
} Stack;
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void initialize(Stack* s)
{
    s->top = NULL;
}

int isEmpty(Stack* s)
{
    return s->top == NULL;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void push(Stack* s, int item)
{
    Node* newNode = (Node*)
    malloc(sizeof(Node));
    if (newNode == NULL)
    {
        printf("Stack overflow!\n");
        exit(1); // Exit with an error code
    }
    newNode->data = item;
    newNode->next = s->top;
    s->top = newNode;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
int pop(Stack* s)
{
    if (isEmpty(s))
    {
        printf("Stack underflow!\n");
        exit(1);
    }
    Node* temp = s->top;
    int poppedData = temp->data;
    s->top = s->top->next;
    free(temp);
    return poppedData;
}
```

KNOWLEDGEGATE

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    Stack s;
    initialize(&s);



    return 0;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void reverseString(char str[])
{
    int length = strlen(str);
    Stack s;
    initialize(&s);
    for (int i = 0; i < length; i++)
    {
        push(&s, str[i]);
    }
    for (int i = 0; i < length; i++)
    {
        str[i] = pop(&s);
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
int main()
{
    char str[] = "Hello, World!";
    printf("Original String: %s\n", str);
    reverseString(str);
    printf("Reversed String: %s\n", str);
    return 0;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q if the input sequence is 1, 2, 3, 4, 5 then identify the wrong stack permutation (possible pop sequence)?

a) 3, 5, 4, 2, 1

b) 2, 4, 3, 5, 1

c) 4, 3, 5, 2, 1

d) 5, 4, 3, 1, 2

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Infix notation**: the operator is written in between the operands. e.g. A+B. the reason why this notation is called infix is the place of operator in the expression.

- **Prefix notation:** In which the operator is written before the operands it is also called as polish notation. e.g. +AB

- **Postfix:** In the postfix notation the operator are written after the operands, so it is called the postfix notation. It is also known as suffix notation or reverse polish notation. AB+

[KNOWLEDGEGATE]

http://www.knowledgegate.in/GATE

- The description "Polish" refers to the nationality of logician Jan Łukasiewicz, who invented Polish notation in 1924.

KNOWLEDGE GATE

http /GATE

- Postfix notation is type of notation which is most suitable for a computer to calculate any expression. It is universally accepted notation for designing arithmetic and logical unit (ALU) of the CPU.

- Any expression entered into the computer is first converted into postfix notation, stored in stack and then calculated.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider an expression a + b * c / d ^ e ^ f * d – c + b, convert it into both prefix and post fix notation?

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider an expression log(x!), convert it into both prefix and post fix notation?

KNOWLEDGEGATE

Let Q is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression P.

1. Push "("onto STACK, and add ")" to end of Q.
2. Scan Q from left to right and repeat steps 3 to 6 for each element of Q until the STACK is empty.
3. If an operand is encountered, add it to P.
4. If a left parenthesis is encountered push it onto STACK.
5. If an operator $\otimes$ is encountered, then :
   a. Repeatedly pop from STACK and add to P each operator (on the top of STACK) which has the same precedence as or higher precedence than $\otimes$.
   b. Add $\otimes$ to STACK.

   [End if]
6. If a right parenthesis is encountered, then :
   a. Repeatedly pop from STACK and add to P each operator (on the top of STACK) until a left parenthesis is encountered.
   b. Remove the left parenthesis [Do not add it to P]

   [End if]

   [End of step 2]
7. End.

KNOWLEDGEGATE

(A + (B *C + D)/E)

Character	Stack	Postfix
(	(	
A	(	A
+	(+	A
(	(+(	A
B	(+(	AB
*	(+(*	AB
C	(+(*	ABC
+	(+(+	ABC*
D	(+(+	ABC*D
)	(+	ABC*D+
/	(+/	ABC*D+
E	(+/	ABC*D+E
)	(	ABC*D+E/+

KNOWLEDGEGATE

$A + (B*C - (D/E \wedge F)*H)$

Character	Stack	Postfix
A	(	A
+	(+	A
(	(+ (	A
B	(+ (	AB
*	(+ (*	AB
C	(+ (*	ABC
–	(+ (–	ABC*
(	(+ (– (	ABC*
D	(+ (– (	ABC*D
/	(+ (– (/	ABC*D
E	(+ (– (/	ABC*DE
^	(+ (– (/^	ABC*DE
F	(+ (– (/^	ABC*DEF
)	(+ (– (/^	ABC*DEF
*	(+ (– *	ABC*DEF ^/
H	(+ (– *	ABC*DEF ^/ H

KNOWLEDGEGATE

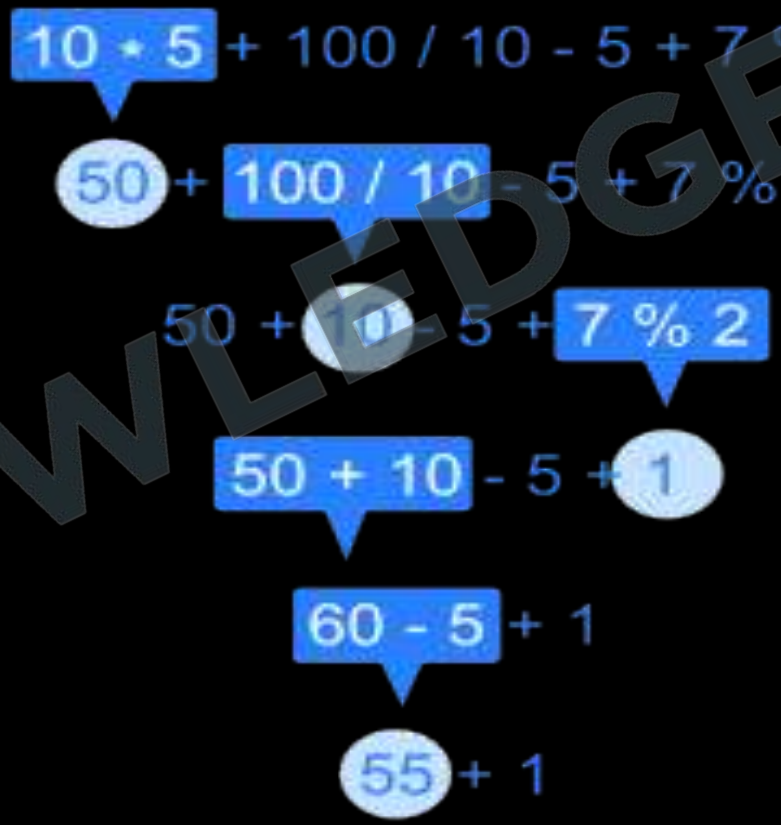
Q Consider an expression a + (b * c) / (d ^ e) convert it into post fix notation using stack?

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Evaluation of arithmetic expression

- An expression is defined as a number of operands or data items combined using several operators. There are basically three of notation to represent an expression.



KNOWLEDGEGATE

Q The result evaluating the postfix expression

8 2 3 * 1 / + 4 1 * 2 / +

KNOWLEDGEGATE

Q The result evaluating the prefix expression

+ + 8 / * 2 3 1 / * 4 1 2

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

1. Add a right parenthesis ")" to P.
   [This acts as a sentinel]
2. Scan P from left to right and repeat step 3 and 4 for each element of P until the sentinel ")" is encountered.
3. If an operand is encountered, put it on STACK.
4. If an operator $\otimes$ is encountered then :
   a. Remove the top two elements of STACK, where A is the top element and B is the next-to-top element.
   b. Evaluate $B \otimes A$.
   c. Place the result of (b) back on STACK.
   [End of if structure]
   [End of step 2 loop]
5. Set value equal to top element on STACK.
6. End.

KNOWLEDGEGATE

Recursion

- Recursion is defined as defining anything in terms of itself Recursion is a programming concept where a function calls itself in order to solve a larger problem by breaking it down into smaller, more manageable sub-problems. It's a fundamental idea in computer science and mathematics and is used to design algorithms and solve problems that have repetitive structures.
- **Base Case**: Essential to halt recursion. It provides a direct solution without further recursive calls.
- **Recursive Case**: The function calls itself to address smaller instances of the problem.
- **Call Stack**: Each recursive call is added to the program's call stack. Deep recursion might cause a "stack overflow" error.

KNOWLEDGEGATE

```
int factorial(int n)
{
    if (n == 0)
    {
        return 1;  // Base case: factorial of 0 is 1
    }
    else
    {
        return n * factorial(n-1);  // Recursive case
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Iteration

- Iteration refers to the process of repeatedly executing a set of statements as long as a specified condition remains true. In programming, iteration is commonly implemented using loops.

- **Loop Types**:
  - **For Loop**: Used for a known number of repetitions.
  - **While Loop**: Runs as long as a condition is true.
  - **Do-While Loop**: Executes at least once before checking the condition.

- **Control Statements**:
  - **Break**: Exits the loop.
  - **Continue**: Skips to the next iteration.

- **Nested Loops**: Loops within loops, often seen in matrix tasks.

KNOWLEDGEGATE

```
int factorial(int n)
{
    int result = 1;
    for(int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Recursion vs. Iteration

- Any problem that can be solved recursively can also be solved iteratively (using loops), and vice versa. However, the choice between the two often depends on problem characteristics, readability, and efficiency considerations.

KNOWLEDGEGATE

Aspect	Recursion	Iteration
Basic Concept	Function calls itself to solve sub-problems.	Uses loops to repeatedly execute code blocks.
Memory Usage	Typically uses more memory due to call stack.	Uses less memory as it doesn't rely on the call stack.
Termination	Requires a base case to prevent infinite loops.	Requires a loop exit condition.
Ease of Implementation	Can be more intuitive for certain problems.	Often simpler and more straightforward for repetitive tasks.
Performance	Might be slower due to overhead of function calls.	Typically faster due to direct loop mechanics.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- Recursion can be categorized based on how and where functions call themselves or other functions. Here's a succinct breakdown of the types of recursion:

- **Direct Recursion**:
  - A function calls itself directly.
  - **Tail Recursion**: If the recursive call is the last operation in the function, before it returns a value. It's more memory efficient since it can be optimized by the compiler to use constant stack space.
  - **Head Recursion**: If the recursive call is made before any other operation in the function. The operations are executed after the recursive call, which makes it use more stack space compared to tail recursion.

- **Indirect Recursion**:
  - Two or more functions call each other in a cyclic manner. For example, function A calls function B, and function B calls function A.

[KNOWLEDGEGATE]

Q Find the output of the following pseudo code?

```
Void main()
{
    fun(4);
}

Void fun(int x)
{
    if (x > 0)
    {
        Printf("%d", x);
        fun(x - 1);
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Find the output of the following pseudo code?

```
Void main()
{
    fun(4);
}

Void fun(int x)
{
    if (x > 0)
    {
        fun(x - 1);
        Printf("%d", x);
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Find the output of the following pseudo code?

```
Void main()
{
    fun(3);
}

Void fun(int x)
{
    if (x > 0)
    {
        Printf("%d", x);
        fun(x - 1);
        Printf("%d", x);
        fun(x - 1);
        Printf("%d", x);
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Find the output of the following pseudo code on n = 5?

```
int x(int n)
{
    if (n < 3)
        return 1;
    Else
        return x(n-1) + x(n-1) + 1;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider the following recursive C function. If get (5) function is being called in main () then how many times will the get () function be invoked before returning to the main ()?

```
void get (int n)
{
    if (n < 1)
        return;
    get(n-1);
    get(n-3);
    printf ("%d", n);
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- In mathematics, the **Fibonacci numbers**, commonly denoted F_n, form a sequence, called the **Fibonacci sequence**, such that each number is the sum of the two preceding ones, starting from 0 and 1.
  - if n==0, then f(n) = 0
  - if n==1, then f(n) = 1
  - if n > 1, then f(n-1) + f(n-2)

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- if n==0, then f(n) = 0
- if n==1, then f(n) = 1
- if n > 1, then f(n-1) + f(n-2)

n	0	1	2	3	4	5	6	7	8	9	10	11	12
f(n)													
No of invocation													
No of addition													

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

no of invocation = 2f(n+1) - 1

no of addition = f(n+1) – 1

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- Fibonacci numbers are named after Italian mathematician Leonardo of Pisa, later known as Fibonacci. In his 1202 book *Liber Abaci*, Fibonacci introduced the sequence to Western European mathematics

- Although the sequence had been described earlier in Indian mathematics,as early as 200 BC in work by Pingala on enumerating possible patterns of Sanskrit poetry formed from syllables of two lengths.

KNOWLEDGEGATE

http://w .in/GATE

- Fibonacci numbers appear unexpectedly often in mathematics, so much so that there is an entire journal dedicated to their study, the *Fibonacci Quarterly*.

- Applications of Fibonacci numbers include computer algorithms such as the Fibonacci search technique and the Fibonacci heap data structure, and graphs called Fibonacci cubes used for interconnecting parallel and distributed systems.

KNOWLEDGEGATE

- They also appear in biological settings, such as branching in trees, the arrangement of leaves on a stem, the fruit sprouts of a pineapple, the flowering of an artichoke, an uncurling fern, and the arrangement of a pine cone's bracts.

KNOWLEDGEGATE

http://ww gegate.in/GATE

- Although the sequence had been described earlier in Indian mathematics, as early as 200 BC in work by Acharya Pingala on enumerating possible patterns of Sanskrit poetry formed from syllables of two lengths.

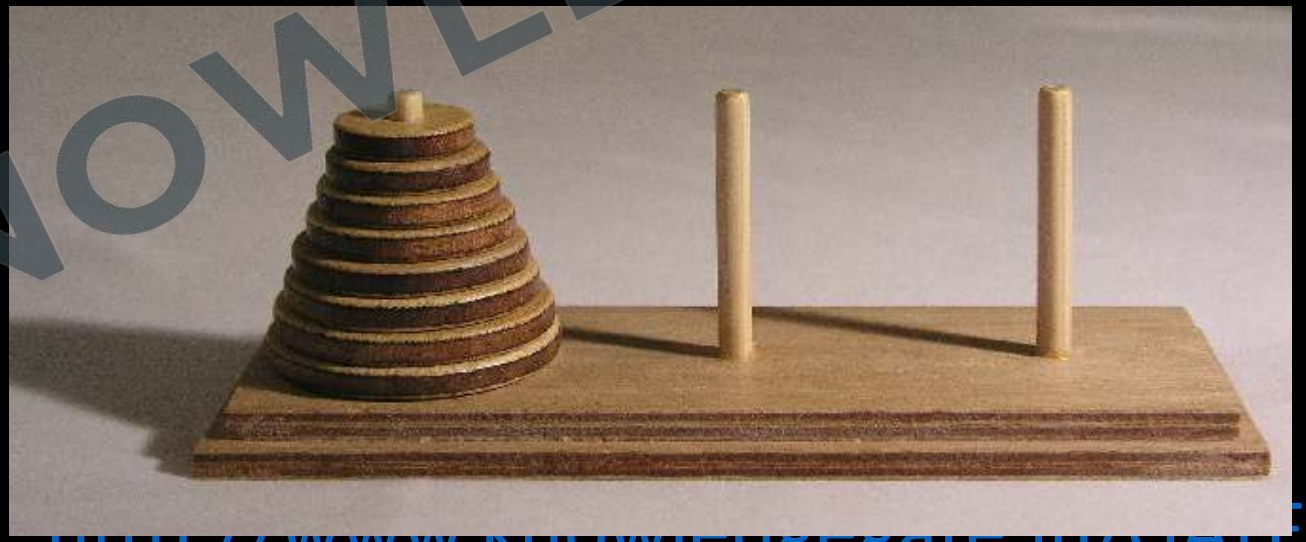
- binary numeral system
- binomial theorem
- Pascal's triangle
- zero

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Tower of hanoi

- The **Tower of Hanoi** (also called the **Tower of Brahma**) is a mathematical game or puzzle.
- It consists of three rods and a number of disks of different sizes, which can slide onto any rod.
- The puzzle starts with the disks in a neat stack in ascending order of size on one rod, the smallest at the top, thus making a conical shape.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

The objective of the puzzle is to move the entire stack to another rod, obeying the following simple rules:

1.Only one disk can be moved at a time.

2.Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty rod.

3.No larger disk may be placed on top of a smaller disk.

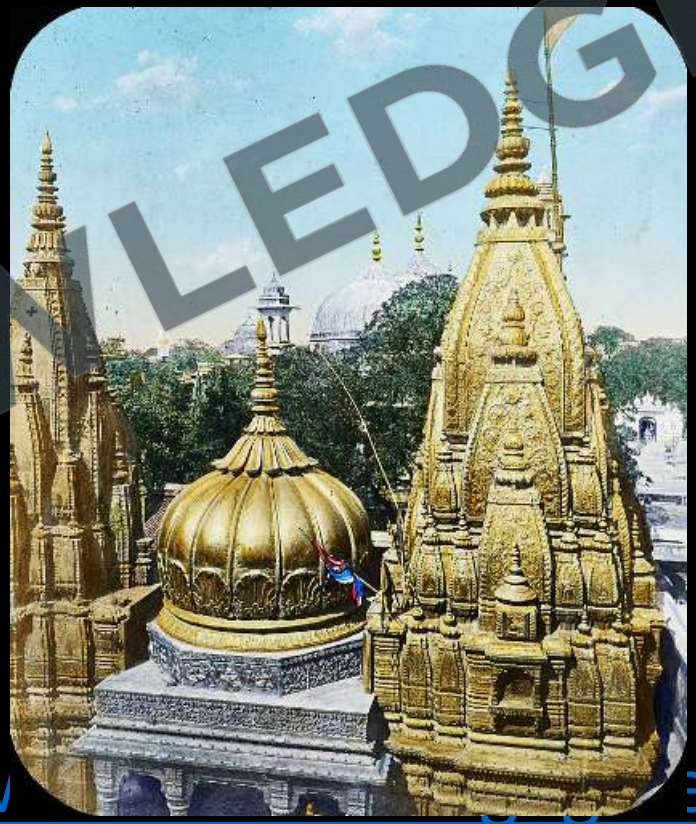
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- There is a story about an Indian temple in Kashi Vishwanath which contains a large room with three time-worn posts in it, surrounded by 64 golden disks.

- Brahmin priests, acting out the command of an ancient prophecy, have been moving these disks in accordance with the immutable rules of Brahma since that time.

- The puzzle is therefore also known as the Tower of Brahma puzzle.



KNOWLEDGEGATE

http://v e.in/GATE

```
Tower(N, B, A, E)
{
    if(n = 1)
    {
        B → E
        return
    }
    tower(n-1, B, E, A);
    B → E
    tower(n-1, A, B, E);
    Return
}
```

total disk moves = $2^n - 1$

total number of function call = $2^{n+1} - 1$

how many invocation are required for the first disk to move = n

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Find the output of the following pseudo code?

```
void Print_array(a, i, j)
{
    if (i = = j)
    {
        printf(“%d’, a[i]);
        return;
    }
    Else
    {
        printf(“%d’, a[i]);
        print_array(a, i+1, j)
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Find the output of the following pseudo code?

```
void Print_array(a, i, j)
{
    if (i = = j)
    {
        printf(‘’%d’, a[i]);
        return;
    }
    Else
    {
        print_array(a, i+1, j)
        printf(‘’%d’, a[i]);
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Find the output of the following pseudo code?

```
void Print_somthing(a, i, j)
{
    if (i = = j)
    {
        printf(‘’%d’, a[i]);
        return;
    }
    Else
    {
        if(a[i] < a[j])
            Print_somthing (a, i+1, j);
        Else
            Print_somthing (a, i, j-1);
    }
}
```

KNOWLEDGEGATE

Q Find what this function is doing?

```
void what(struct Bnode *t)
{
    if (t)
    {
        what(t → LC);
        printf(''%d', t → data);
        what(t → RC);
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Find what this function is doing?

```
void what(struct Bnode *t)
{
    if (t)
    {
        printf(''%d', t → data);
        what(t → LC);
        what(t → RC);
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Find what this function is doing?

```
void what(struct Bnode *t)
{
    if (t)
    {
        what(t → LC);
        what(t → RC);
        printf(''%d', t → data);
    }
}
```

KNOWLEDGEGATE

Q Find what this function is doing?

```
Void what(struct Bnode *t)
{
    if (t)
    {
        printf(''%d', t → data);
        what(t → LC);
        printf(''%d', t → data);
        what(t → RC);
        printf(''%d', t → data);
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Find what this function is doing?

```
Void A(struct Bnode *t)
{
    if (t)
    {
        B(t → LC);
        printf(“%d’, t → data);
        B(t → RC);
    }
}

Void B(struct Bnode *t)
{
    if (t)
    {
        printf(“%d’, t → data);
        A(t → LC);
        A(t → RC);
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q What does the following function print for n = 25?

```
void fun(int n)
{
    if (n == 0)
        return;
    printf("%d", n%2);
    fun(n/2);
}
```

(A) 11001
(B) 10011
(C) 11111
(D) 00000

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider the following recursive function fun (x, y). What is the value of fun (4, 3)

```
int fun(int x, int y)
{
      if (x == 0)
            return y;
      return fun(x - 1,  x + y);
}
```

(A) 13

(B) 12

(C) 9

(D) 10

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q What does the following function do?

```
int fun(int x, int y)
{
    if (y == 0)
        return 0;
    return (x + fun(x, y-1));
}
```

(A) x + y

(B) x + x*y

(C) x*y

(D) x^y

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q What does fun2() do in general?

```
int fun(int x, int y)
{
if (y == 0)
return 0;
    return (x + fun(x, y-1));
}

int fun2(int a, int b)
{
    if (b == 0)
        return 1;
    return fun(a, fun2(a, b-1));
}
```

(A) x*y
(B) x+x*y
(C) x^y
(D) y^x

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Queue

- A queue is a linear list of elements in which deletions can take place only at one end called the front, and insertions can take place only at the end called rear.
- Queue is a first in first out types of data structure(FIFO), the terms ‘Front’ and ‘Rear’ are used in describing a linear list only when it is implemented as a queue.

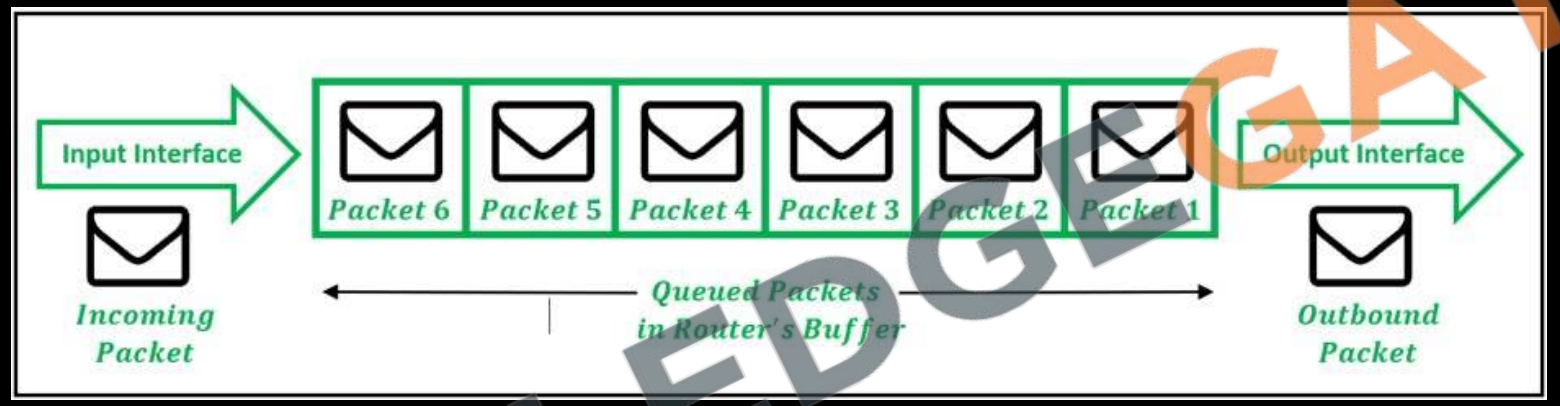
KNOWLEDGE GATE

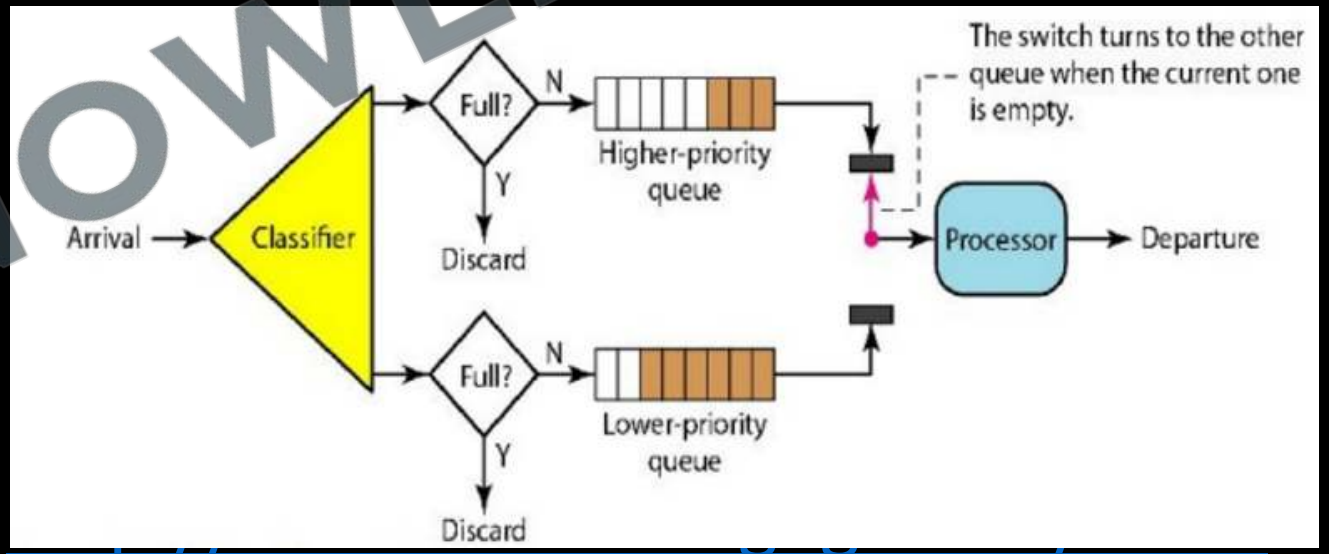
KNOWLEDGEGATE
4GIFS.com

KNOWLEDGE GATE
knowle

- In computer science queue are used in multiple places e.g. in time sharing system program with the same priority from a queue waiting to be executed.

- A queue is a non-primitive linear data structure. it is homogeneous collection of elements.

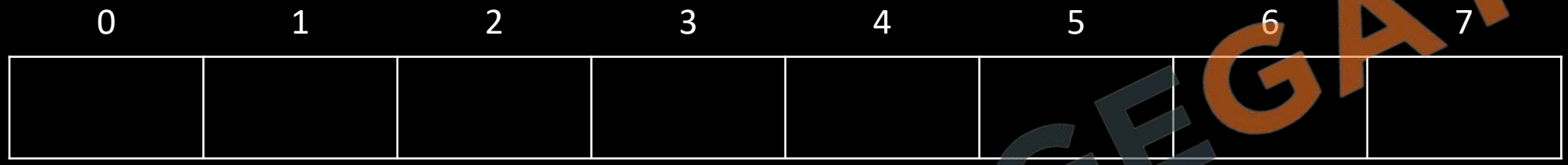




KNOWLEDGEGATE

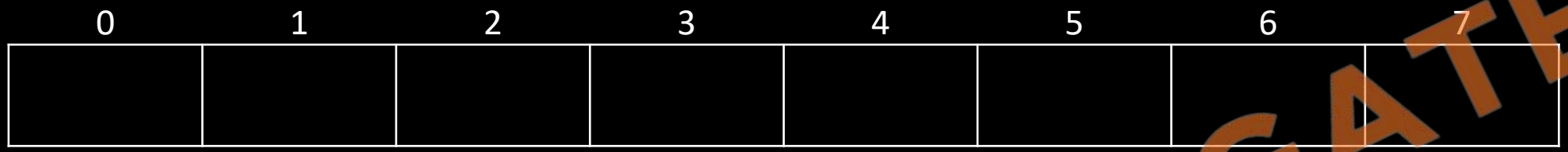
Representation of Queues

- Mostly each of our queues will be maintained by a linear array QUEUE and two pointer variables: FRONT containing the location of the Front element of the queue and REAR, containing the location of the rear element of the queue.



KNOWLEDGEGATE

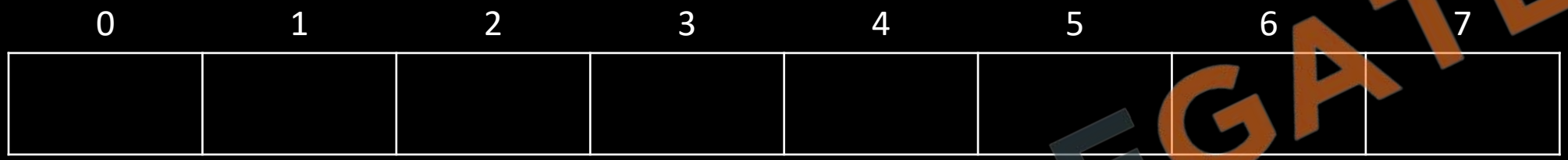
- Whenever an element is added to the queue, the value of REAR is increased by 1
  - REAR = REAR + 1



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- The condition FRONT = null, will indicate that the queue is empty. Whenever an element is deleted from the queue, the value of FRONT is increased by 1
  - Front = Front + 1



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

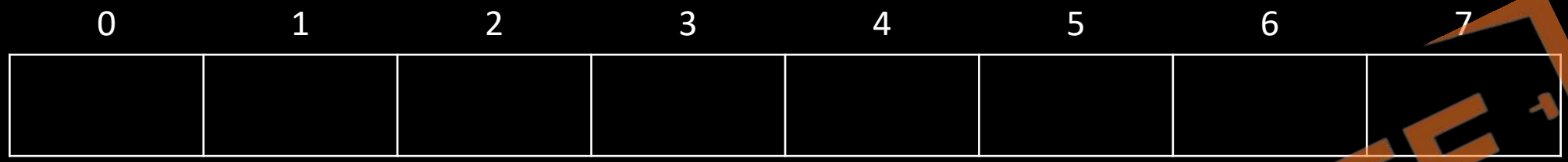
- This means that after N insertion the rear element of the queue will occupy QUEUE[N] or queue will occupy the last part of the array. This may occur even though the queue itself may not contain many elements.

- Total number of elements in a queue
  - Rear – Front + 1

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

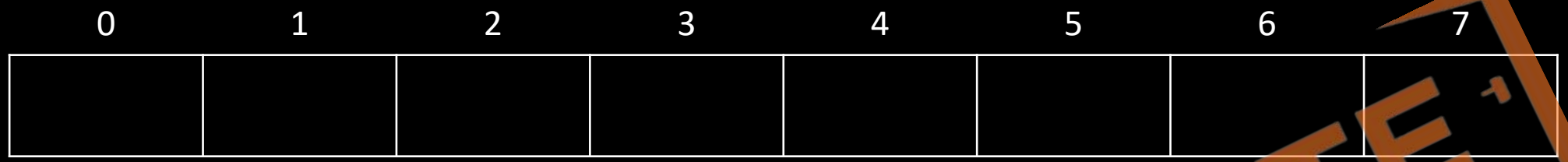
Insertion




```
Enqueue (QUEUE, N, F, R, ITEM)
{
    if (R == N - 1)
        Write over flow and exit
    if (F = = -1)
        Set F = 0 && R = 0
    Else
        R = R + 1
    Queue[R] = ITEM
}
```

KNOWLEDGEGATE

Deletion




```
Dequeue (QUEUE, N, F, R, ITEM)
{
    if (F == - 1)
        Write under flow and exit
    ITEM = QUEUE[F]
    if (F = = R)
        Set F = -1 && R = -1
    Else
        F = F + 1
    Return item
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
typedef struct
{
    int arr[MAX_SIZE];
    int front;
    int rear;
} Queue;

void initialize(Queue *q)
{
    q->front = -1;
    q->rear = -1;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
int isEmpty(Queue *q)
{
    return q->front == -1;
}


int isFull(Queue *q)
{
    return q->rear == MAX_SIZE - 1;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void enqueue(Queue *q, int item)
{
    if (isFull(q))
    {
        printf("Queue is full!\n");
        return;
    }
    if (isEmpty(q))
    {
        q->front = 0;
    }
    q->arr[++(q->rear)] = item;
}
```

KNOWLEDGEGATE

```
int dequeue(CircularQueue *q)
{
    if (isEmpty(q))
    {
        printf("Queue is empty!\n");
        exit(1);
    }
    int dequeuedItem = q->arr[q->front];
    if (q->front == q->rear)
    {
        q->front = -1;
        q->rear = -1;
    }
    else
    {
        q->front = (q->front + 1) % MAX_SIZE;
    }
    return dequeuedItem;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```c
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 100
int main()
{
    Queue q;
    initialize(&q);



    return 0;
}
```

KNOWLEDGEGATE

```
typedef struct Node
{
    int data;
    struct Node* next;
} Node;


typedef struct
{
    Node* front;
    Node* rear;
} Queue;
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void initialize(Queue* q)
{
    q->front = NULL;
    q->rear = NULL;
}


int isEmpty(Queue* q)
{
    return q->front == NULL;
}
```

KNOWLEDGEGATE

```
void enqueue(Queue* q, int item)
{
    Node* newNode = (Node*) malloc(sizeof(Node));
    if (newNode == NULL)
    {
        printf("Queue overflow!\n");
        return;
    }
    newNode->data = item;
    newNode->next = NULL;
    if (isEmpty(q))
    {
        q->front = newNode;
    }
    else
    {
        q->rear->next = newNode;
    }
    q->rear = newNode;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
int dequeue(Queue* q)
{
    if (isEmpty(q))
    {
        printf("Queue underflow!\n");
        exit(1);
    }
    Node* temp = q->front;
    int dequeuedItem = temp->data;
    q->front = q->front->next;
    if (q->front == NULL)
    {
        q->rear = NULL;
    }
    free(temp);
    return dequeuedItem;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    Queue q;
    initialize(&q);



    return 0;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

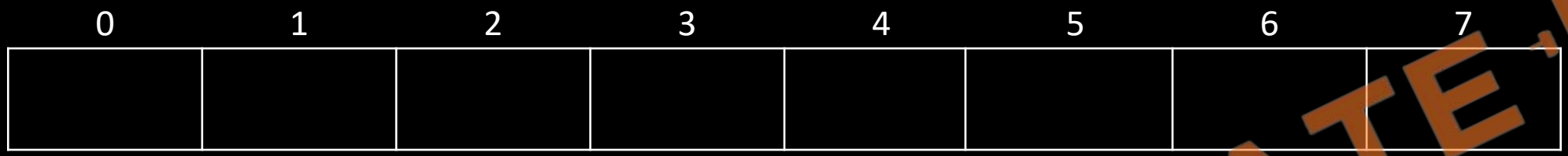
Q. Consider the following sequence of operations on an empty stack.
Push(54);push(52);pop();push(55);push(62);s=pop();

Consider the following sequence of operations on an empty queue.
enqueue(21);enqueue(24);dequeue();enqueue(28);enqueue(32);q=dequeue();
The value of s+q is _____.

KNOWLEDGEGATE

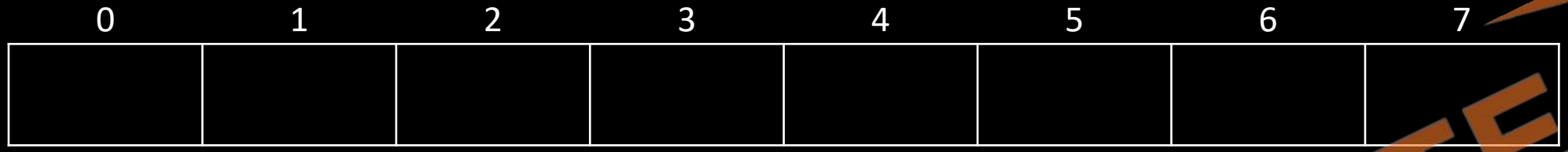
http://www.knowledgegate.in/GATE

Analysis

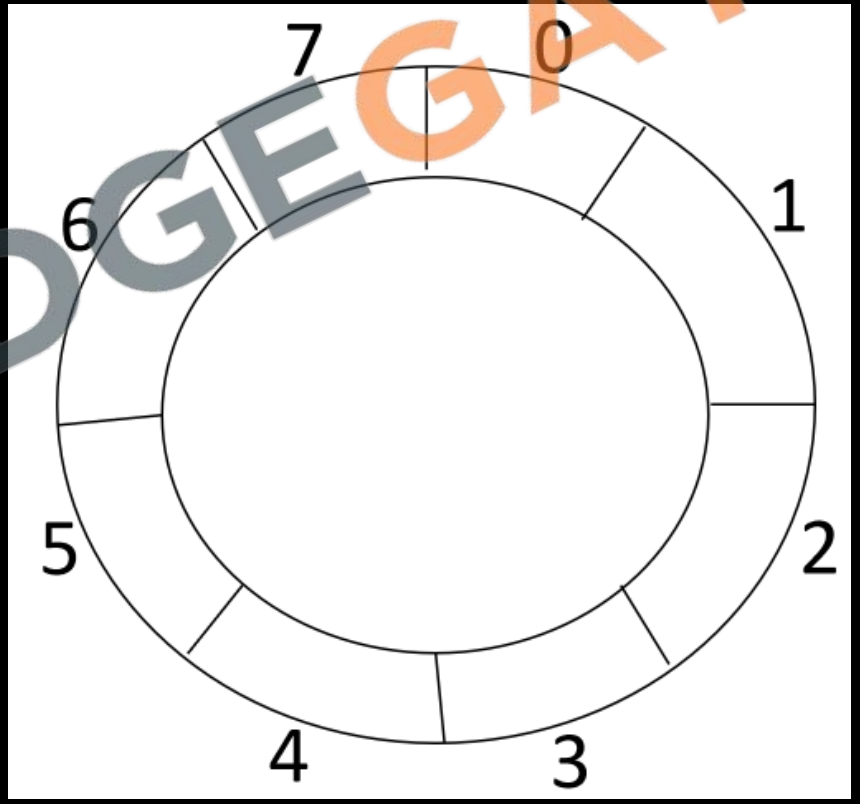


KNOWLEDGEGATE

Circular Queue



1. EnQ a, b, c
2. DeQ 1 element
3. EnQ d, e, f
4. EnQ g, h, I
5. DeQ 4 element
6. EnQ j, k, l, m, n

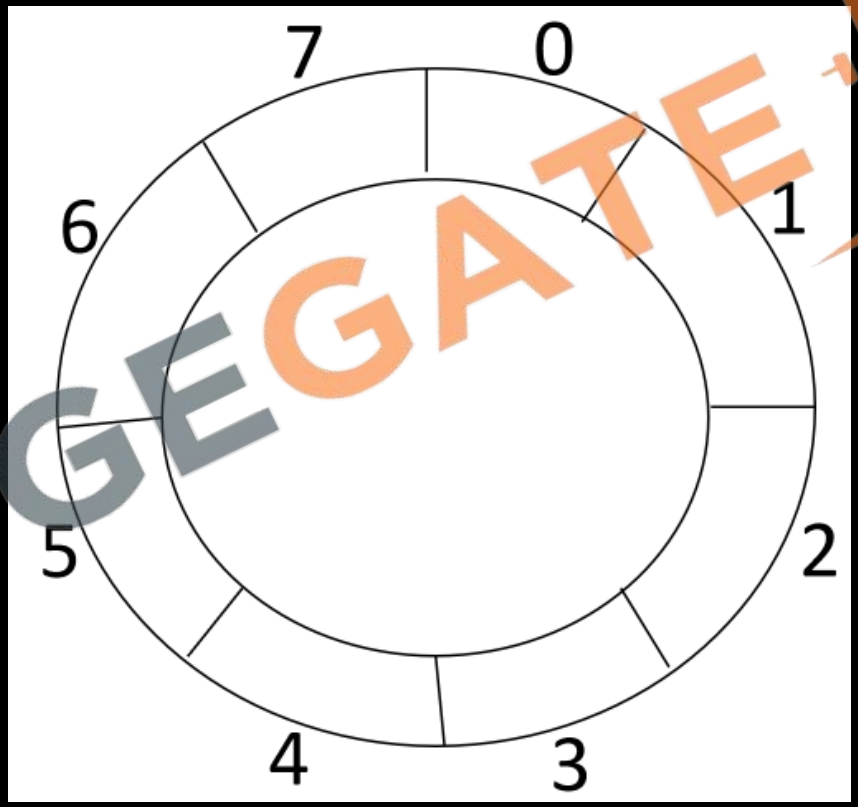


KNOWLEDGEGATE

Circular Queue

Insertion

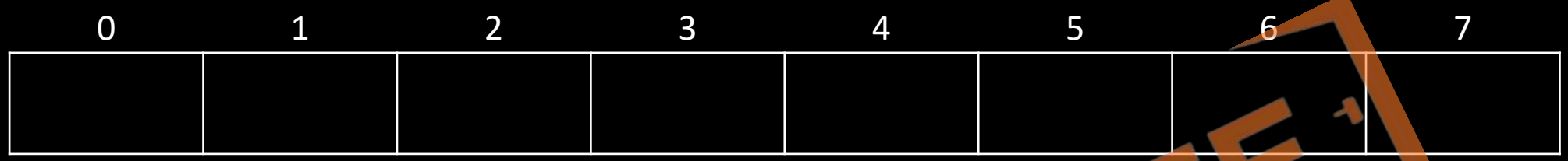
```
Enqueue(QUEUE, N, F, R, ITEM)
{
    if ((F==0 && R==N-1) :: (F == R + 1))
        Write over flow and exit
    if (F = = -1)
        Set F = 0 && R = 0
    Else
        R = (R + 1)%N
    Queue[R] = ITEM
}
```



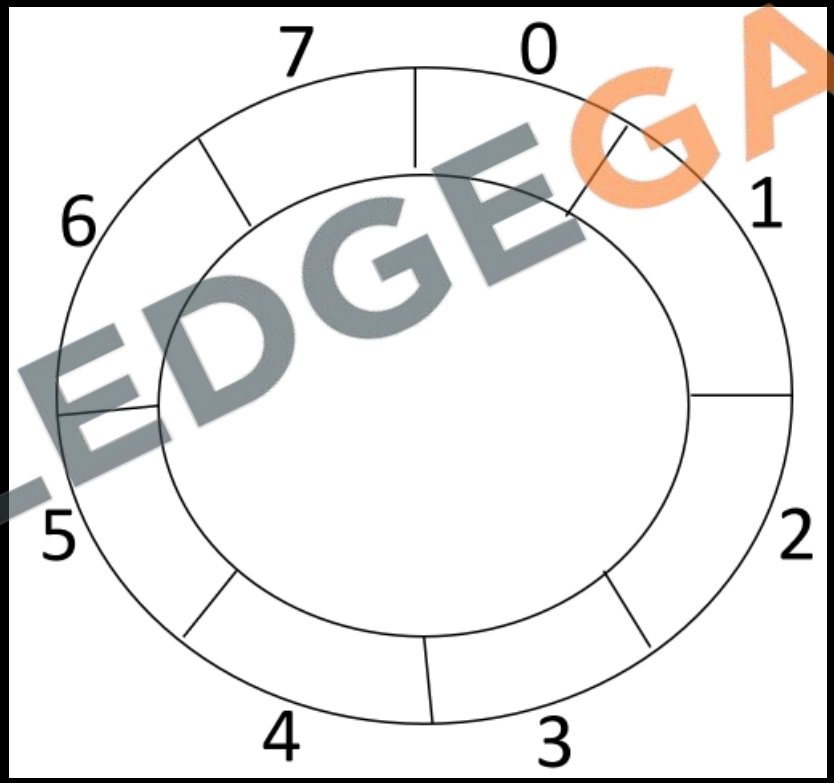
KNOWLEDGEGATE

Circular Queue

Deletion




```
Dequeue(QUEUE, N, F, R, ITEM)
{
    if (F == - 1)
        Write under flow and exit
    ITEM = QUEUE[F]
    if (F = = R)
        Set F = -1 && R = -1
    Else
        F = (F + 1)%N
    Return item
}
```



KNOWLEDGEGATE

```
typedef struct
{
    Int arr[MAX_SIZE];
    int front;
    int rear;
} CircularQueue;


void initialize(CircularQueue *q)
{
    q->front = -1;
    q->rear = -1;
}
```

KNOWLEDGEGATE

```
Int isEmpty(CircularQueue *q)
{
    return q->front == -1;
}


int isFull(CircularQueue *q)
{
    return (q->rear + 1) % MAX_SIZE == q->front;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void enqueue(CircularQueue *q, int item)
{
    if (isFull(q))
    {
        printf("Queue is full!\n");
        return;
    }
    if (isEmpty(q))
    {
        q->front = 0;
        q->rear = 0;
    }
    else
    {
        q->rear = (q->rear + 1) % MAX_SIZE;
    }
    q->arr[q->rear] = item;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
int dequeue(CircularQueue *q)
{
    if (isEmpty(q))
    {
        printf("Queue is empty!\n");
        exit(1);
    }
    int dequeuedItem = q->arr[q->front];
    if (q->front == q->rear)
    {
        q->front = -1;
        q->rear = -1;
    }
    else
    {
        q->front = (q->front + 1) % MAX_SIZE;
    }
    return dequeuedItem;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
int main()
{
    CircularQueue q;
    initialize(&q);



    return 0;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
typedef struct Node
{
    int data;
    struct Node* next;
} Node;


typedef struct
{
    Node* front;
    Node* rear;
} CircularQueue;
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void initialize(CircularQueue* q)
{
    q->front = NULL;
    q->rear = NULL;
}

int isEmpty(CircularQueue* q)
{
    return q->front == NULL;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
// Enqueue an item to the queue
void enqueue(CircularQueue* q, int item)
{
	Node* newNode = (Node*) malloc(sizeof(Node));
	if (newNode == NULL)
	{
		printf("Queue overflow!\n");
		return;
	}
	newNode->data = item;
	newNode->next = NULL;
	if (isEmpty(q))
	{
		q->front = newNode;
		q->rear = newNode;
		newNode->next = newNode;
	}
	else
	{
		newNode->next = q->front;
		q->rear->next = newNode;
		q->rear = newNode;
	}
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
// Dequeue an item from the queue
int dequeue(CircularQueue* q)
{
	if (isEmpty(q))
	{
		printf("Queue underflow!\n");
		exit(1);
	}
	Node* temp = q->front;
	int dequeuedItem = temp->data;
	if (q->front == q->rear)
	{
		q->front = NULL;
		q->rear = NULL;
	}
	else
	{
		q->front = q->front->next;
		q->rear->next = q->front;
	}
	free(temp);
	return dequeuedItem;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

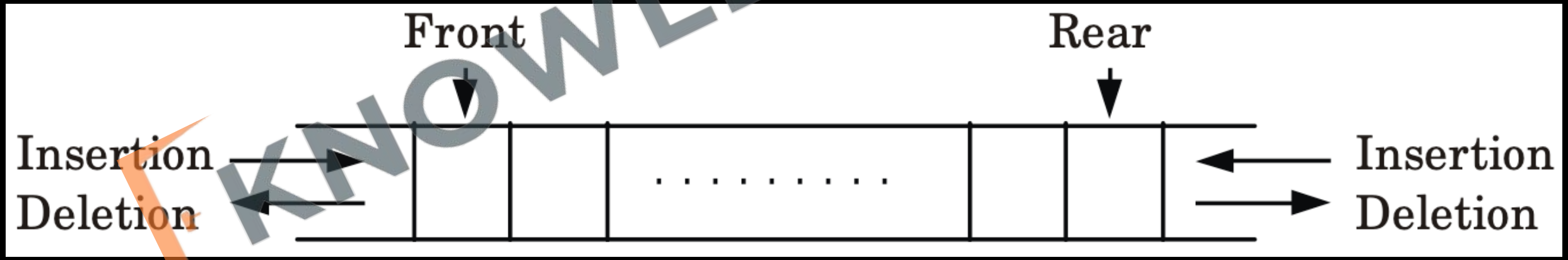
```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    CircularQueue q;
    initialize(&q);



    return 0;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

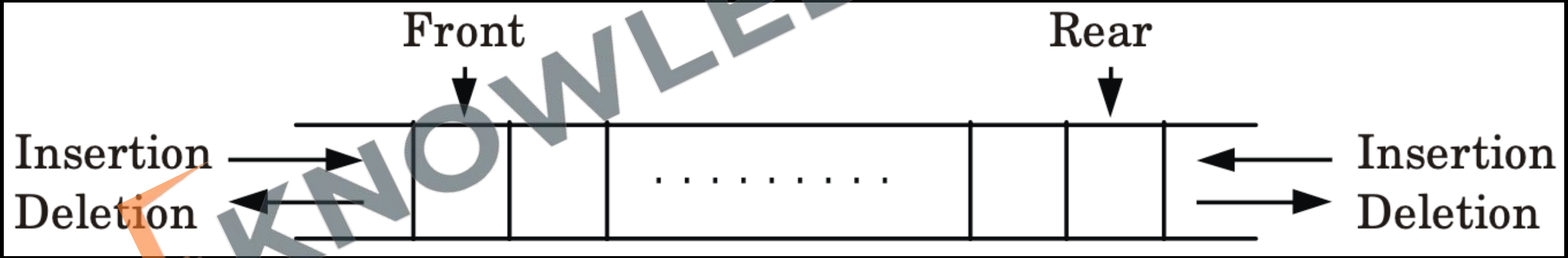
Dequeue

- In a dequeue, both insertion and deletion operations are performed at either end of the queues. That is, we can insert an element from the rear end or the front end. Also deletion is possible from either end.
- This dequeue can be used both as a stack and as a queue.
- There are various ways by which this dequeue can be represented. The most common ways of representing this type of dequeue are :
  - Using a doubly linked list
  - Using a circular array



KNOWLEDGEGATE

- Types of dequeue :
  - **Input-restricted dequeue** : In input-restricted dequeue, element can be added at only one end but we can delete the element from both ends.
  - **Output-restricted dequeue** : An output-restricted dequeue is a dequeue where deletions take place at only one end but allows insertion at both ends.



KNOWLEDGEGATE

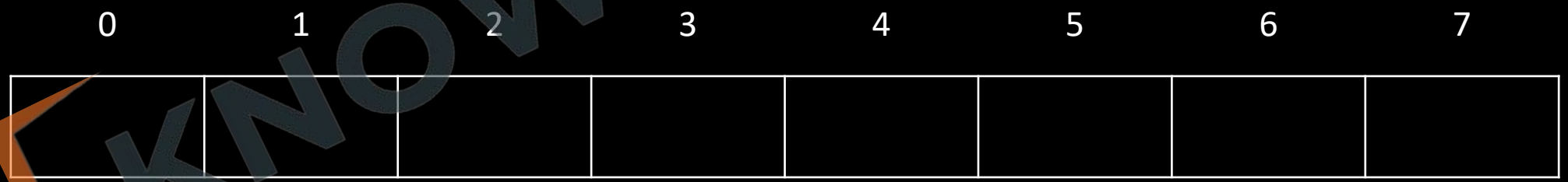
Priority Queue

- A priority queue is a collection of elements such that each element has been assigned a priority and such that the order in which elements are deleted and processed comes from the following rules.
  - An element of higher priority is processed before any element of lower priority
  - Two element with the same priority are processed according to the order in which they were added to the queue.

KNOWLEDGEGATE

Problem with Array

- **Fixed size and reallocation**: Arrays have a fixed size, which can lead to memory waste if the allocated size is larger than the actual data. Resizing an array often requires creating a new one and copying elements, which can be inefficient.
- **Inefficient insertion and deletion**: Adding or removing elements in the middle of an array requires shifting the remaining elements, resulting in a higher time complexity (O(n)) compared to linked lists.
- **Less flexible**: Arrays can only store elements of the same data type, and their structure cannot be easily adapted to different types (e.g., singly, doubly, circular) like linked lists.

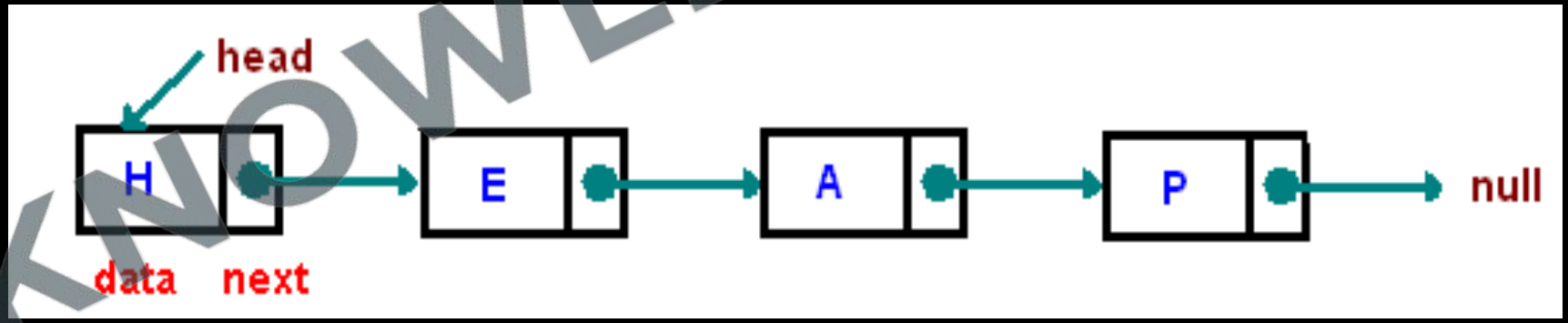


KNOWLEDGEGATE

Linked list

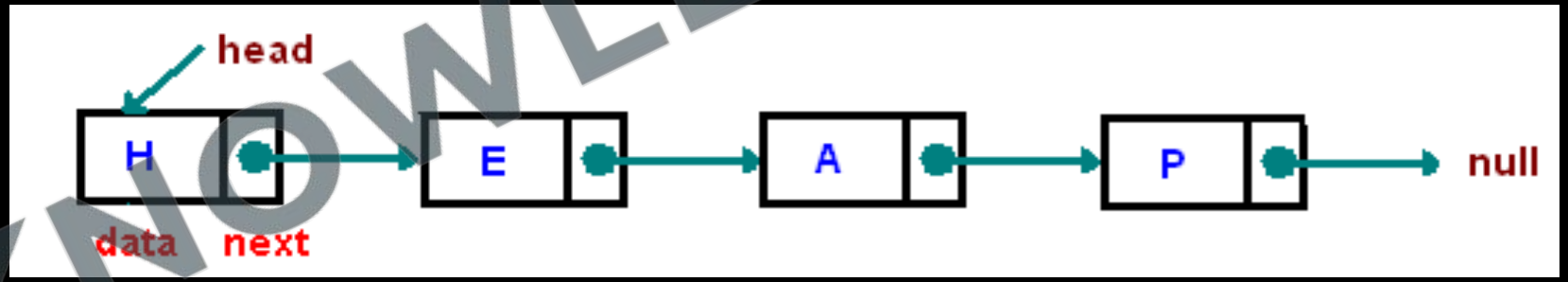
KNOWLEDGEGATE

- Solution is linked list
  - A linked list is a dynamic data structure that consists of elements called nodes, which are connected in a linear sequence. Each node contains two parts: data and a reference to the next node.
  - The first part is the information part of the node, which can store any type of information, such as integers, characters, or objects.
  - The second part called linked field or next pointer field, contains the address of the next node of the list.



KNOWLEDGEGATE

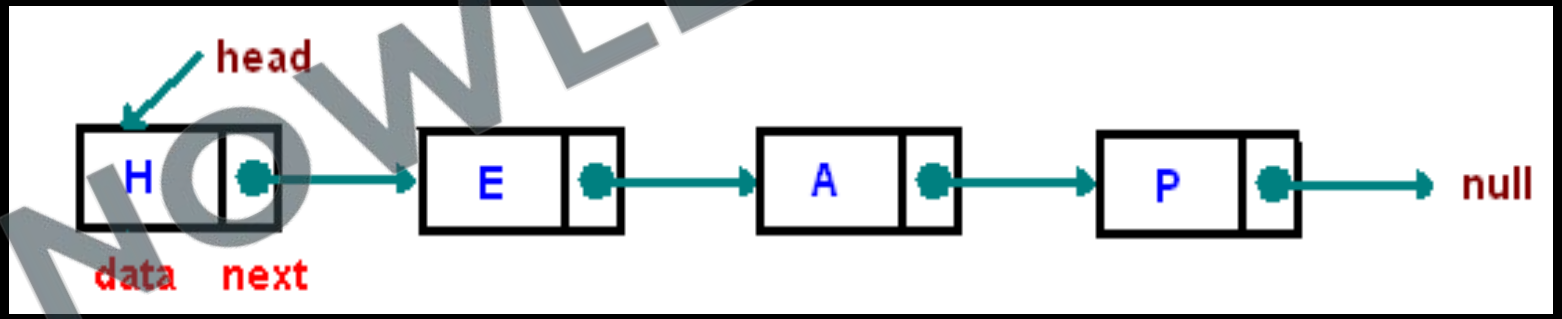
- The pointer of the last node contains a null pointer, which is an invalid address (0 or negative value).
- The linked also contains a list pointer variable called start/first/head which contain the address of the first node in the list.
- A special case is the list that has no nodes, such a list is called null list or empty list and is denoted by a null pointer in the variable start/first/head.



KNOWLEDGEGATE

Implementation of link list

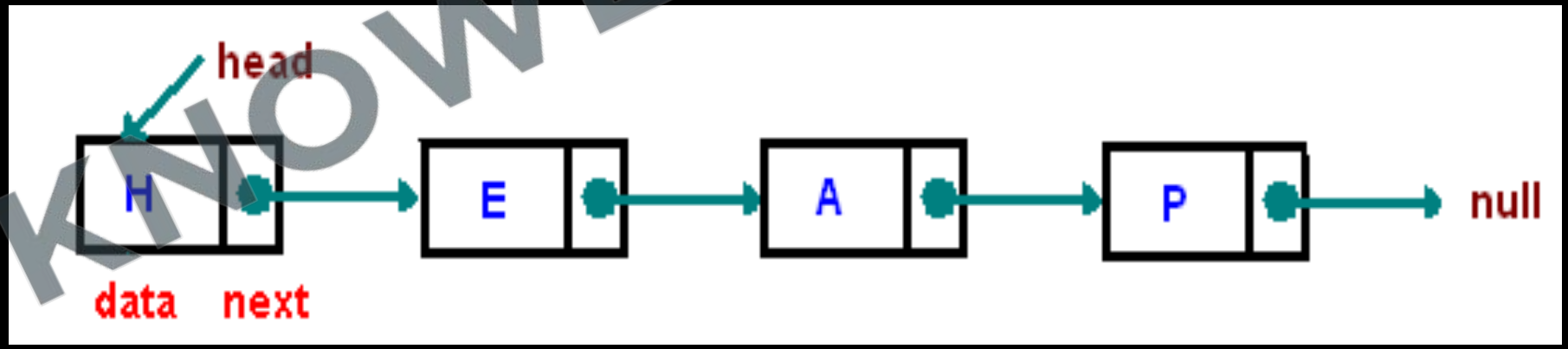
```
struct node
{
    int data;
    struct node *next;
};
```



KNOWLEDGEGATE

Advantage of link list

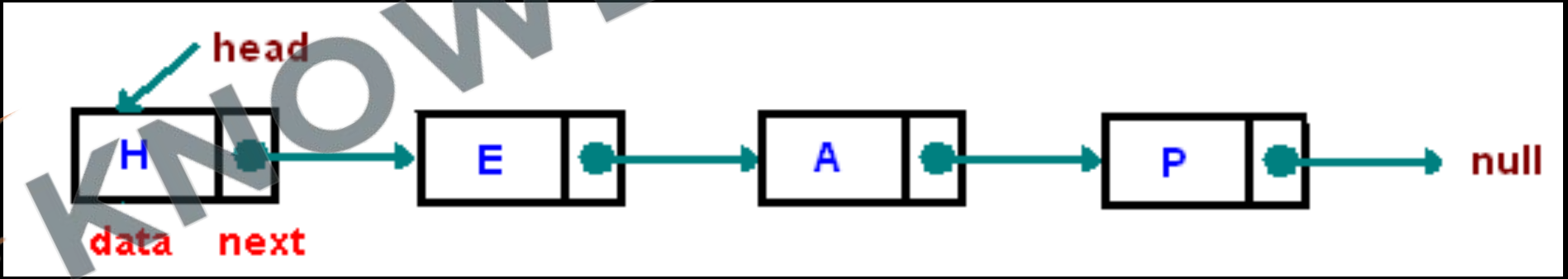
- **Dynamic size and efficient memory usage**: Linked lists can easily grow or shrink, allowing for efficient memory allocation and reduced waste as elements are added or removed.
- **Fast insertion and deletion**: Operations like inserting or removing elements can be performed in constant time (O(1)) if the position is known, offering better performance compared to array-based structures.
- **Versatility**: Linked lists can be adapted to various types (singly, doubly, circular) and can store elements of different data types or objects, providing a flexible data structure for a wide range of applications.



KNOWLEDGEGATE

Disadvantage of link list

- **Slower access times**: Linked lists have a higher time complexity for element access (O(n)) compared to arrays, as elements must be accessed sequentially from the head of the list.
- **Memory overhead**: Each node in a linked list requires additional memory to store the reference (or pointer) to the next node, increasing the overall memory usage compared to array-based structures.
- **Pointer manipulation**: Implementing linked lists involves managing pointers, which can increase code complexity and lead to potential issues, such as memory leaks or segmentation faults, if not handled carefully.



KNOWLEDGEGATE

Aspect	Array	Linked List
Memory Allocation	Contiguous memory locations.	Non-contiguous memory locations.
Size Flexibility	Fixed size.	Dynamic size, can grow or shrink as required.
Access Time	O(1) for direct access due to indexing.	O(n) for accessing an element as it requires traversal.
Insertion/Deletion	O(n) in worst case as shifting may be required.	O(1) if the pointer to the node is known.
Memory Efficiency	More memory efficient for a known size of data.	Extra memory for pointers, which can be overhead for small data sizes.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

```c
#include <stdio.h>
#include <stdlib.h>
// Define the Node structure
typedef struct Node
{
    int data;
    struct Node* next;
} Node;

Node* createNode(int data)
{
    Node* newNode = (Node*) malloc(sizeof(Node));
    if (!newNode)
    {
        printf("Memory error\n");
        exit(1);
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

KNOWLEDGEGATE

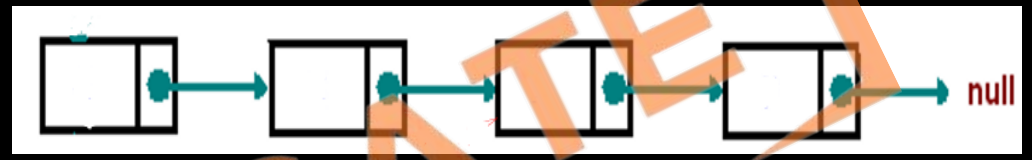
http://www.knowledgegate.in/GATE

```
int main()
{
    Node* head = createNode(10);
    head->next = createNode(20);
    head->next->next = createNode(30);
    printf("Linked List: ");
    traverseList(head);
    return 0;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Write a C-style pseudocode for Traversing a link list iteratively, where pointer head have the address of the first node of the list?

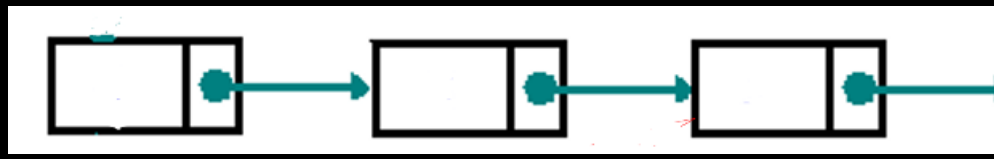



```
void traverseList(Node* head)
{
    Node* current = head;
    while (current != NULL)
    {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}
```

KNOWLEDGEGATE

Q Write a C-style pseudocode for Traversing a link list recursively, where pointer head have the address of the first node of the list?

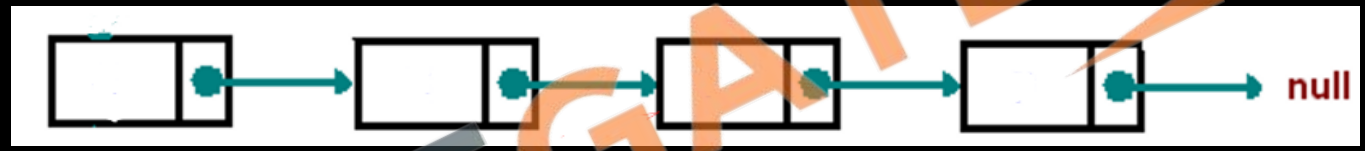
```
void traverseListRecursive(Node* current)
{
    if (current == NULL)
    {
        printf("NULL\n");
        return;
    }
    printf("%d -> ", current->data);
    traverseListRecursive(current->next);
}
```



KNOWLEDGEGATE

Q Write a C-style pseudocode for searching a key in a link list iteratively, where pointer head have the address of the first node of the list?

```
Node* searchKeyIterative(Node* head, int key)
{
    Node* current = head;
    while (current != NULL)
    {
        if (current->data == key)
        {
            return current;
        }
        current = current->next;
    }
    return NULL;
}
```



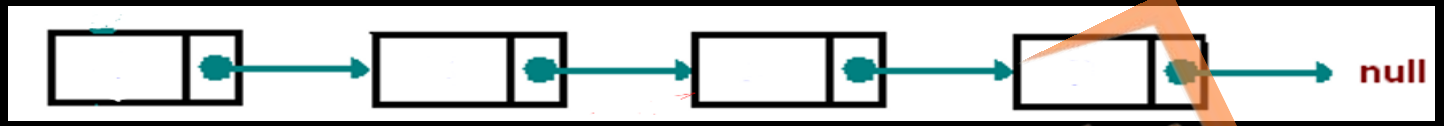
KNOWLEDGEGATE

Q Write a C-style pseudocode for searching a key in a link list recursively, where pointer head have the address of the first node of the list?

```
Node* searchKeyRecursive(Node* current, int key)
{
    if (current == NULL)
    {
        return NULL;
    }
    if (current->data == key)
    {
        return current;
    }
    return searchKeyRecursive(current->next, key);
}
```

KNOWLEDGEGATE

Q Write a C-style pseudocode for inserting a node with a key in the starting of link-list?




```
void insertAtBeginning(Node** head, int key)
{
    Node* newNode = createNode(key);
    newNode->data = data;
    newNode->next = *head;
    *head = newNode;
}
```

KNOWLEDGEGATE

Q Write a C-style pseudocode for inserting a node with a key after a location in a link-list?

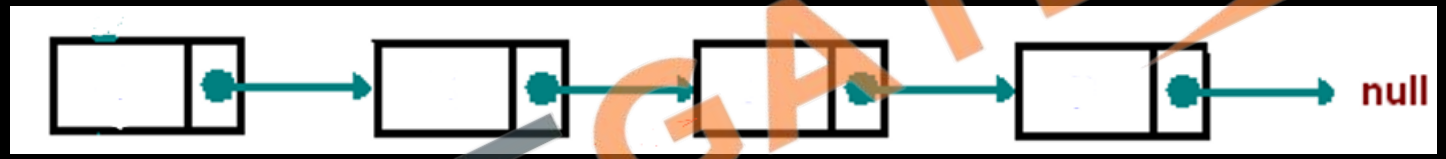
```
void insertAfter(Node* prevNode, int key)
{
    if (prevNode == NULL)
    {
        printf("The given previous node cannot be NULL.\n");
        return;
    }
    Node* newNode = createNode(key);
    newNode->data = data;
    newNode->next = prevNode->next;
    prevNode->next = newNode;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Write a C-style pseudocode for deleting a node from the starting of link-list?

```
void deleteAtBeginning(Node** head)
{
    if (*head == NULL)
    {
        printf("List is already empty.\n");
        return;
    }
    Node* temp = *head;
    *head = (*head)->next;
    free(temp);
}
```



KNOWLEDGEGATE

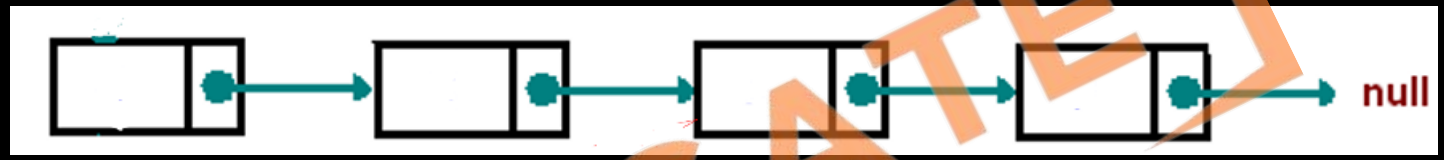
Q Write a C-style pseudocode for deleting a node after a given location from the starting of link-list?

```
void deleteAfter(Node* prevNode)
{
    if (prevNode == NULL || prevNode->next == NULL)
    {
        printf("The given node is NULL or there's no node after it to delete.\n");
        return;
    }
    Node* temp = prevNode->next;
    prevNode->next = temp->next;
    free(temp);
}
```

KNOWLEDGEGATE

Q Write a C-style pseudocode for reversing a link-list in a iteratively?

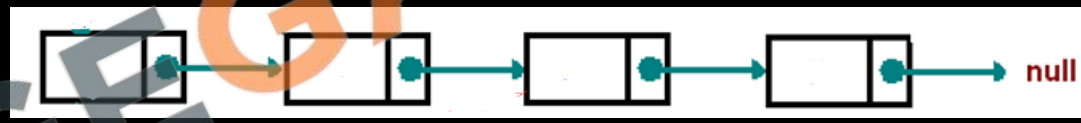
```
void reverseListIterative(Node** head)
{
    Node* prev = NULL;
    Node* current = *head;
    Node* next = NULL;
    while (current != NULL)
    {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    *head = prev;

}
```



KNOWLEDGEGATE

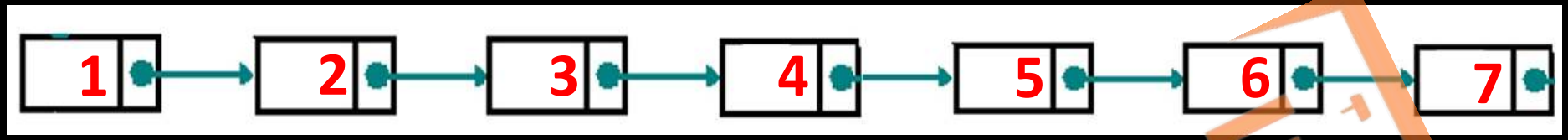
Q Write a C-style pseudocode for reversing a link-list in a recursively?

```
Node* reverseListRecursive(Node* head)
{
    if (head == NULL || head->next == NULL)
    {
        return head;
    }
    Node* rest = reverseListRecursive(head->next);
    head->next->next = head;
    head->next = NULL;
    return rest;
}
```



KNOWLEDGE GATE

Q The following C function takes a single-linked list of integers as a parameter and rearranges the elements of the list. The function is called with the list containing the integers 1, 2, 3, 4, 5, 6, 7 in the given order. What will be the contents of the list after the function completes execution?




```
struct node
{
        int value;
        struct node *next;
};
void rearrange(struct node *list)
{
        struct node *p, * q;
        int temp;
        if ((!list) || !list->next)
                return;
        p = list;
        q = list->next;
        while(q)
        {
                temp = p->value;
                p->value = q->value;
                q->value = temp;
                p = q->next;
                q = p ? p->next:0;
        }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

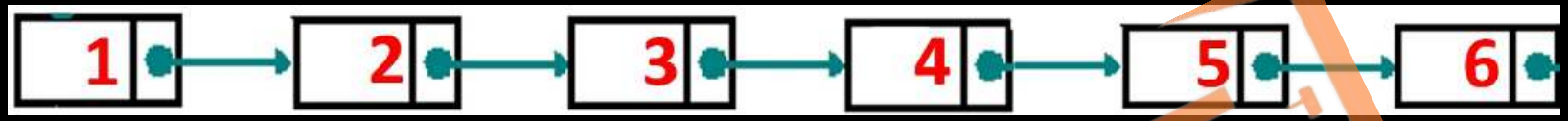
Q The following C function takes a simply-linked list as input argument. It modifies the list by moving the last element to the front of the list and returns the modified list. Some part of the code is left blank. Choose the correct alternative to replace the blank line.

```
typedef struct node
{
        int value;
        struct node *next;
}Node;

Node *move_to_front(Node *head)
{
        Node *p, *q;
        if ((head == NULL: || (head->next == NULL))
                return head;
        q = NULL; p = head;
        while (p-> next !=NULL)
        {
                q = p;
                 p = p->next;
        }
        _____________________________
        return head;
}
```

(A) q = NULL; p->next = head; head = p;
(B) q->next = NULL; head = p; p->next = head;
(C) head = p; p->next = q; q->next = NULL;
(D) q->next = NULL; p->next = head; head = p;

KNOWLEDGEGATE

Q What is the output of following function for start pointing to first node of following linked list?



1->2->3->4->5->6

```
void fun(struct node* start)
{
    if(start == NULL)
        return;
    printf("%d  ", start->data);
    if(start->next != NULL )
        fun(start->next->next);
    printf("%d  ", start->data);
}
```

(A) 1 4 6 6 4 1

(B) 1 3 5 1 3 5

(C) 1 2 3 5

(D) 1 3 5 5 3 1

KNOWLEDGEGATE

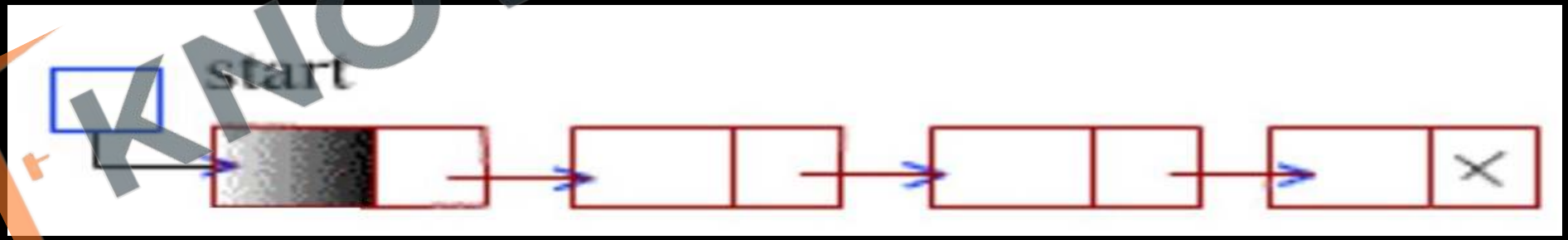
Header link list

- A header linked list is a variation of a standard linked list that includes a special node, called the header node, at the beginning of the list.
- The header node does not store any actual data; instead, it serves as a fixed reference point that simplifies some operations on the linked list like inserting or deleting elements at the beginning of the list.

KNOWLEDGEGATE

Here are some features of header linked lists:

- The header node is always present, even when the list is empty.
- The header node's 'next'pointer points to the first actual data node in the list or 'Null' if the list is empty.
- The header node simplifies operations like insertion or deletion at the beginning, middle, or end of the list, as well as traversal, since the header node acts as a consistent starting point.
- The header node can also store metadata about the list, such as its length, although this is not a requirement.



KNOWLEDGEGATE

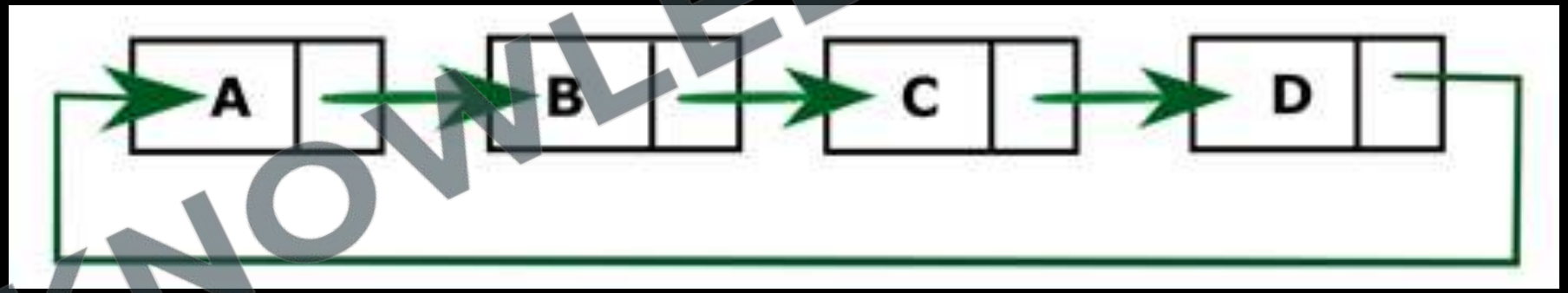
```
void traverseHeaderLinkedList(Node* header)
{
    if (header == NULL)
    {
        printf("List is empty.\n");
        return;
    }
    Node* temp = header->next;
    while (temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Circular Linked List

- A singly circular linked list is a variation of a singly linked list in which the last node in the list points back to the first node, creating a loop or circular structure.
- The primary difference between a standard singly linked list and a singly circular linked list is that the last node's 'Null' pointer refers to the first node in the list, rather than being 'Null'.



KNOWLEDGEGATE

Here are some key features of singly circular linked lists:

- Singly circular linked lists can be used to implement data structures like queues or circular buffers, where elements are added to the end and removed from the front, with constant-time complexity for both operations.
- Traversal of the list requires a stopping condition, such as iterating until you reach the starting node again or using a counter to limit the number of iterations, to avoid infinite loops.

KNOWLEDGEGATE

```
void traverseCircularLinkedList(Node* head)
{
    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }
    Node* temp = head;
    do
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    while (temp != head);
    printf("%d (head)\n", head->data);
}
```

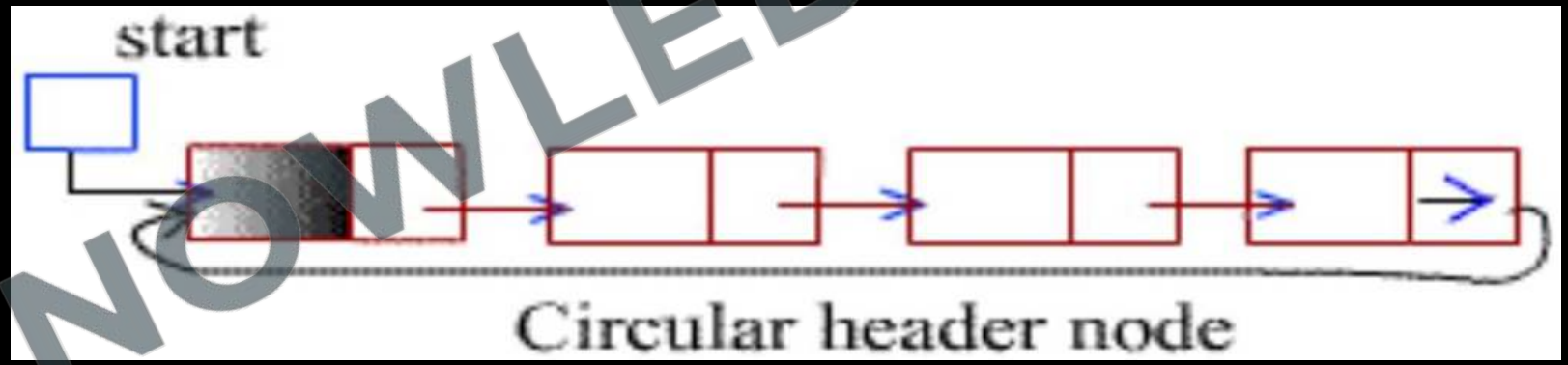
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Header circular link list

- A header singly circular linked list is a variation of a singly circular linked list that includes a special node, called the header node, at the beginning of the list.

- The header node does not store any actual data; instead, it serves as a fixed reference point that simplifies some operations on the linked list. The header node's primary purpose is to eliminate the need for special cases when performing certain operations like inserting or deleting elements at the beginning or end of the list.



KNOWLEDGEGATE

```
void traverseHeaderCircularLinkedList(Node* header)
{
    if (header->next == header)
    {
        printf("List is empty.\n");
        return;
    }
    Node* temp = header->next;
    while (temp != header)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("HEADER\n");
}
```

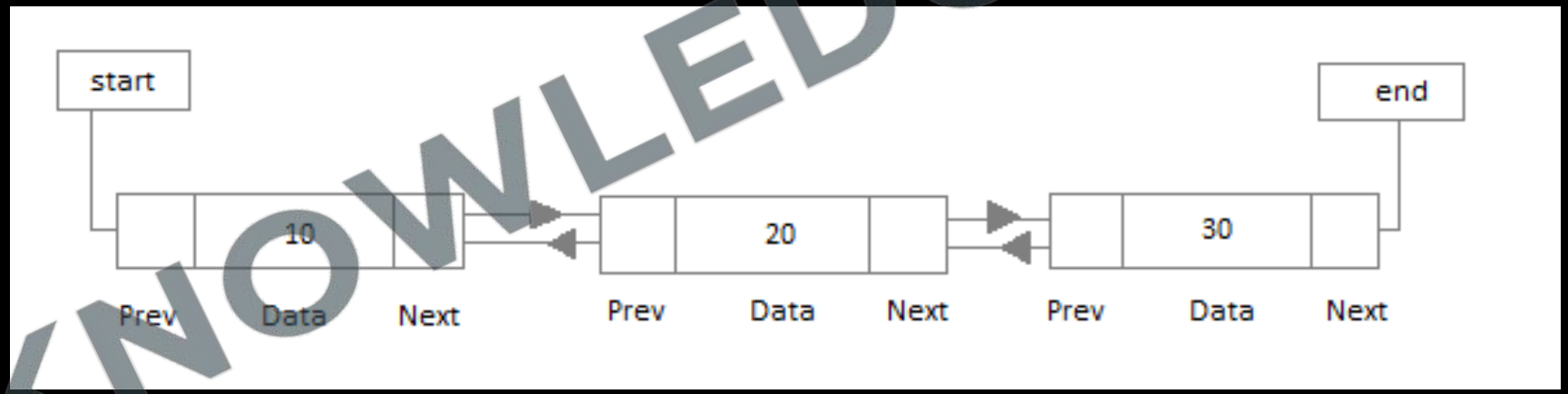
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Doubly link list

- A doubly linked list is a data structure in which each node contains a data element and two pointers, one pointing to the previous node (the ‘previous’ pointer) and the other pointing to the next node (the ‘next’ pointer) in the sequence.

- This bidirectional linking allows for easier traversal and manipulation of the list in both forward and backward directions, as well as simplifying some operations such as insertion or deletion of nodes at any position in the list.



KNOWLEDGEGATE

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node
{
	int data;
	struct Node* prev;
	struct Node* next;
} Node;

Node* createNode(int data)
{
	Node* newNode = (Node*) malloc(sizeof(Node));
	if (!newNode)
	{
		printf("Memory error\n");
		exit(1);
	}
	newNode->data = data;
	newNode->prev = NULL;
	newNode->next = NULL;
	return newNode;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void insertAtBeginning(Node** head, int data)
{
	Node* newNode = createNode(data);
	if (*head != NULL)
	{
		(*head)->prev = newNode;
	}
	newNode->next = *head;
	*head = newNode;
}

void insertAtEnd(Node** head, int data)
{
	Node* newNode = createNode(data);
	if (*head == NULL)
	{
		*head = newNode;
		return;
	}
	Node* temp = *head;
	while (temp->next != NULL)
	{
		temp = temp->next;
	}
	temp->next = newNode;
	newNode->prev = temp;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void traverse(Node* head)
{
	Node* temp = head;
	while (temp != NULL)
	{
		printf("%d -> ", temp->data);
		temp = temp->next;
	}
	printf("NULL\n");
}

Node* search(Node* head, int key)
{
	Node* temp = head;
	while (temp != NULL)
	{
		if (temp->data == key)
		{
			return temp;
		}
		temp = temp->next;
	}
	return NULL;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void deleteAtBeginning(Node** head)
{
        if (*head == NULL)
                return;
        Node* temp = *head;
        *head = (*head)->next;
        if (*head != NULL)
        {
                (*head)->prev = NULL;
        }
        free(temp);
}

void deleteAtEnd(Node** head)
{
        if (*head == NULL)
                return;
        Node* temp = *head;
        while (temp->next != NULL)
        {
                temp = temp->next;
        }
        if (temp->prev != NULL)
        {
                temp->prev->next = NULL;
        }
        else
        {
                *head = NULL;
        }
        Free(temp);
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void deleteByPointer(Node** head, Node* loc)
{
    if (*head == NULL || loc == NULL)
        return;
    if (loc->prev == NULL)
    {
        *head = loc->next;
    }
    else
    {
        loc->prev->next = loc->next;
    }
    if (loc->next != NULL)
    {
        loc->next->prev = loc->prev;
    }
    free(loc);
}
```

KNOWLEDGEGATE

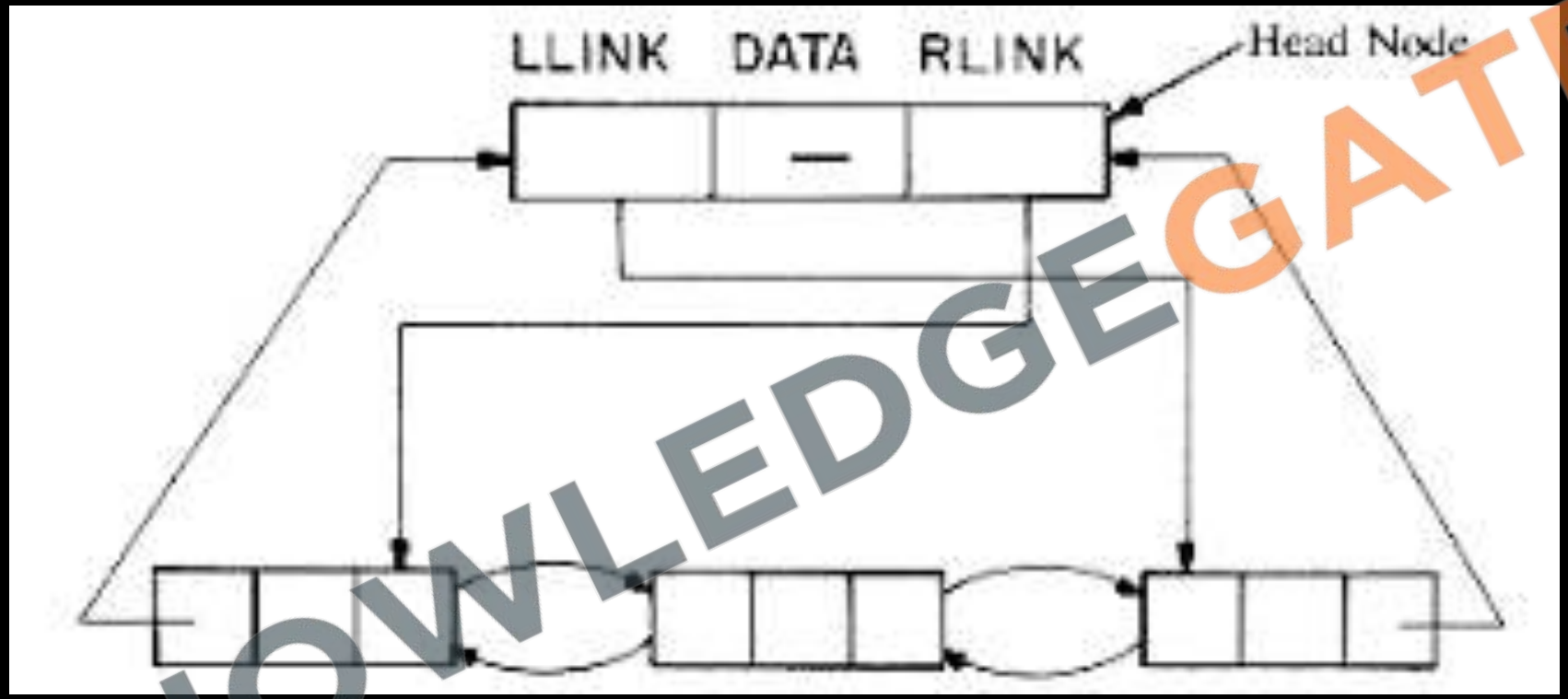
http://www.knowledgegate.in/GATE

Here are some key features of doubly linked lists:

- Each node has two pointers: ‘next’ pointing to the subsequent node and ‘previous’ pointing to the preceding node in the list.
- The first node’s ‘previous’ pointer and the last node’s ‘next’ pointer are set to ‘null’ indicating the beginning and end of the list, respectively.
- Doubly linked lists allow for easier traversal and manipulation in both forward and backward directions compared to singly linked lists.
- Doubly linked lists consume more memory than singly linked lists due to the additional ‘previous’ pointer.

KNOWLEDGEGATE

Header Circular Doubly Link List



KNOWLEDGEGATE

Q Consider the following function that takes reference to head of a Doubly Linked List as parameter. Assume that a node of doubly linked list has previous pointer as *prev* and next pointer as *next*.

```
void fun(struct node **head_ref)
{
    struct node *temp = NULL;
    struct node *current = *head_ref;
    while (current !=  NULL)
    {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;
        current = current->prev;
    }
    if(temp != NULL )
        *head_ref = temp->prev;
    }
```

Assume that reference of head of following doubly linked list is passed to above function

1 <--> 2 <--> 3 <--> 4 <--> 5 <-->6.

What should be the modified linked list after the function call?

(A) 2 <--> 1 <--> 4 <--> 3 <--> 6 <-->5

(B) 5 <--> 4 <--> 3 <--> 2 <--> 1 <-->6.

(C) 6 <--> 5 <--> 4 <--> 3 <--> 2 <--> 1.

(D) 6 <--> 5 <--> 4 <--> 3 <--> 1 <--> 2

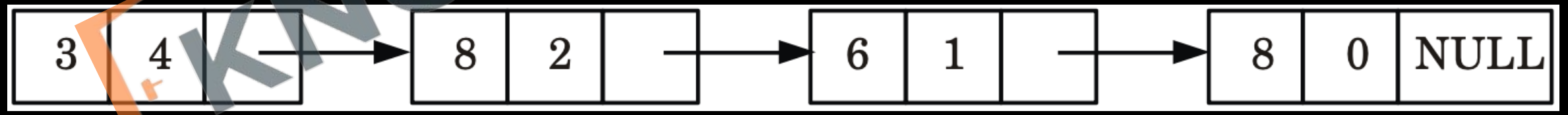
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Polynomial Representation Using Linked List

- In the linked representation of polynomials, each node should consist of three elements, namely coefficient, exponent and a link to the next term.
- The coefficient field holds the value of the coefficient of a term, the exponent field contains the exponent value of that term and the link field contains the address of the next term in the polynomial.

- $3x^4 + 8x^2 + 6x + 8$



KNOWLEDGEGATE

- $3x^2 + 2xy^2 + 5y^3 + 7yz$

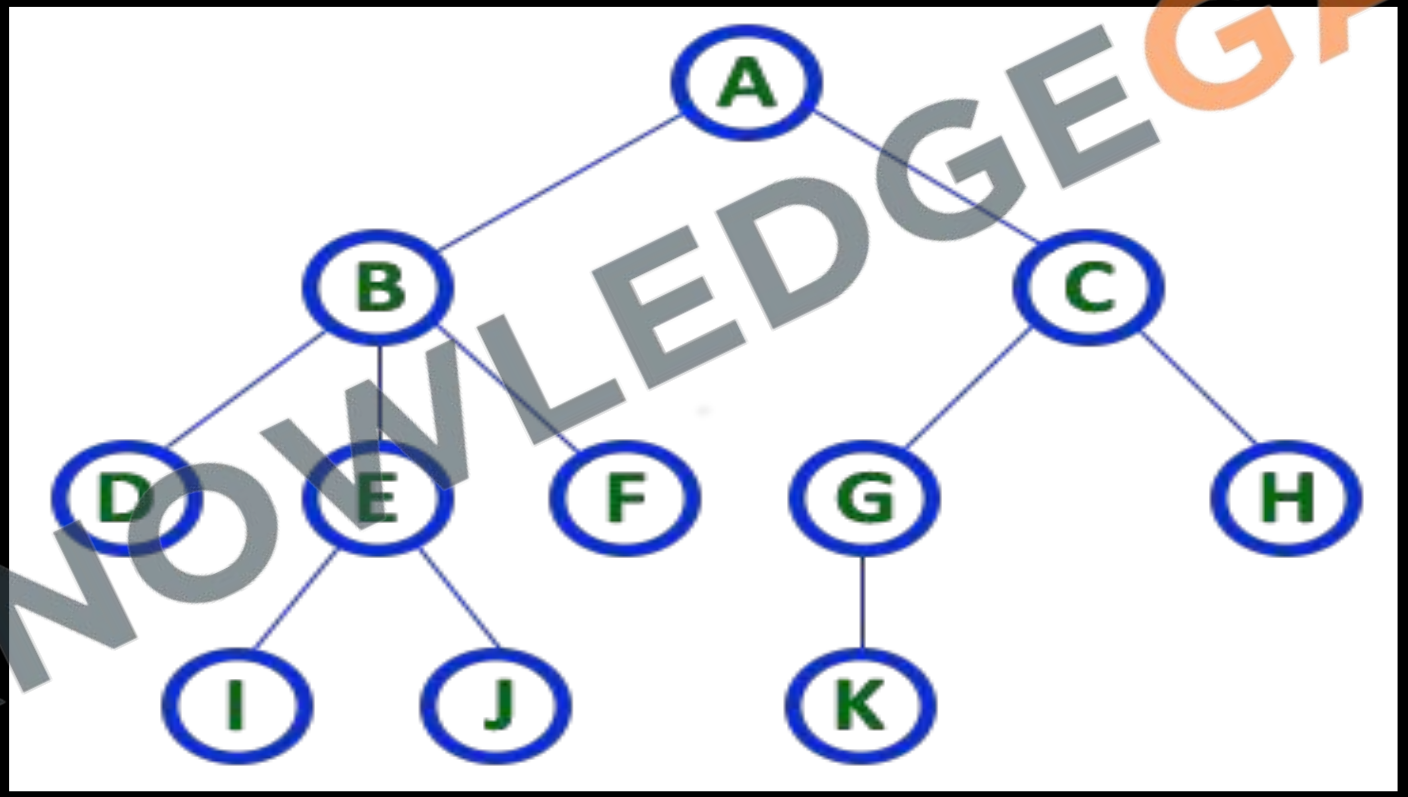
KNOWLEDGEGATE

Polynomial Addition Using Linked List

KNOWLEDGEGATE

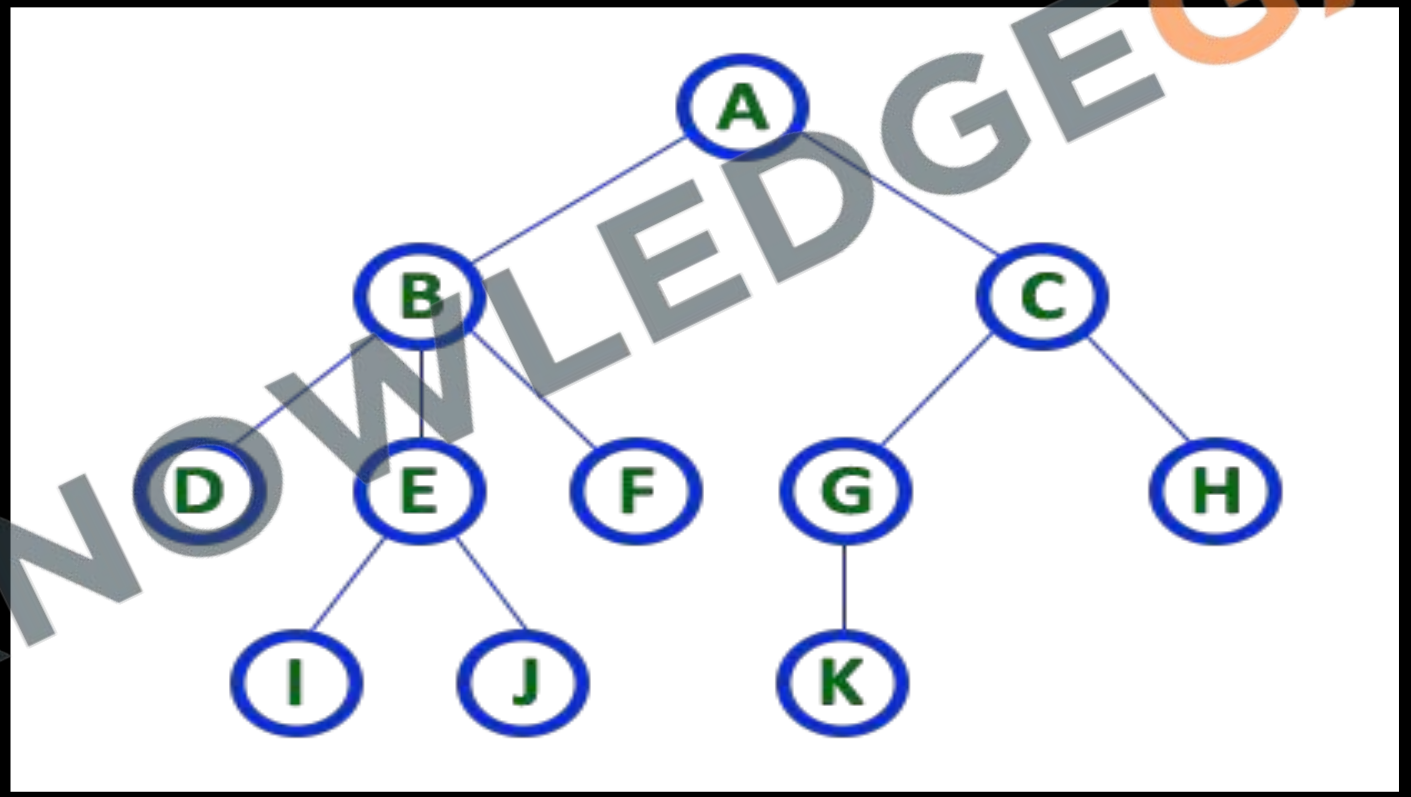
Tree

- The tree is one of the most powerful, flexible, versatile and nonlinear advanced data structures, it represents hierarchical relationship existing between several data items. it is used in wide range of applications.



KNOWLEDGEGATE

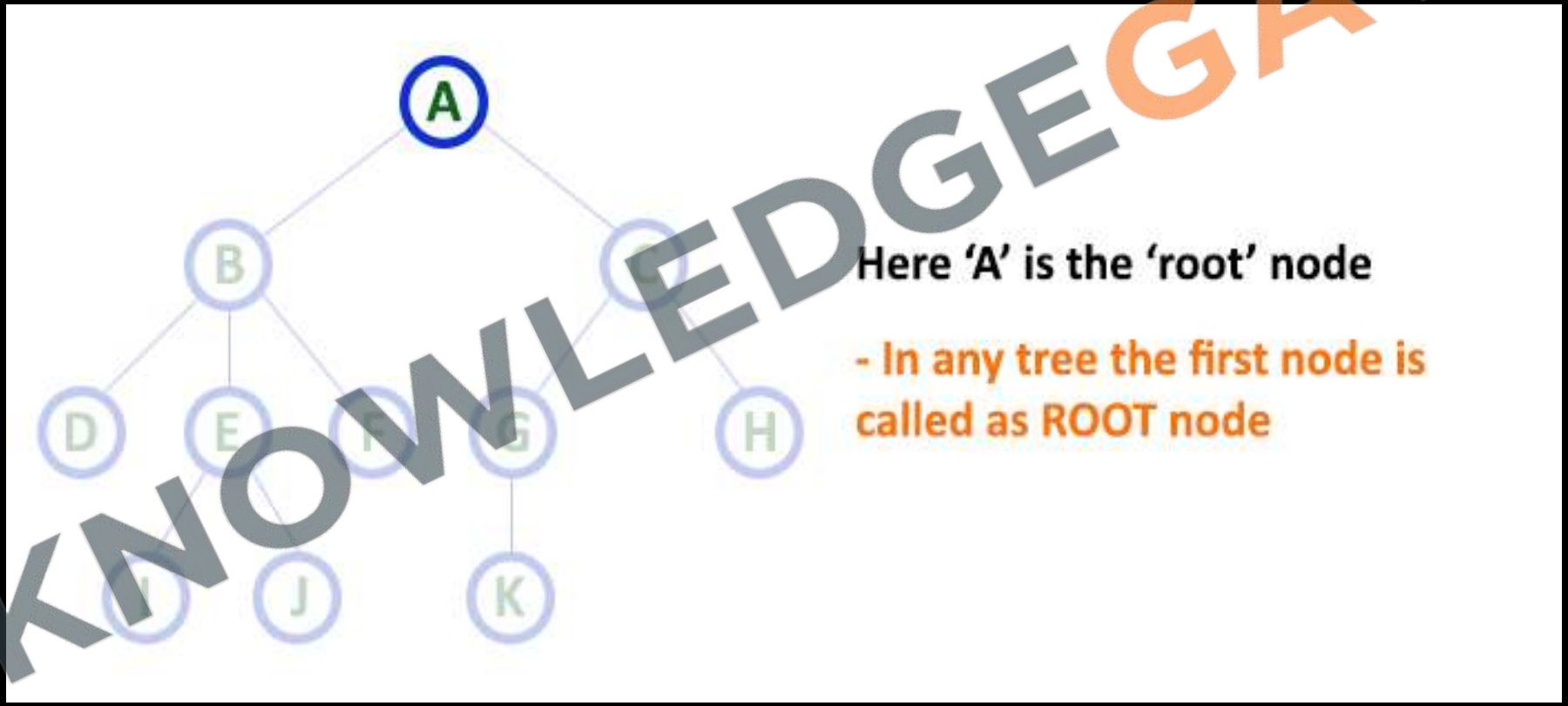
- A tree is a finite set of one or more data items(nodes) such that
  - There is a special data item called root of the tree
  - And its remaining data items are partitioned into number of mutually exclusive (disjoint) subsets, each of which is itself a tree and they are called subtree. i.e. Every node (exclude a root) is connected by a directed edge *from* exactly one other node; A direction is: *parent -> children*



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Root

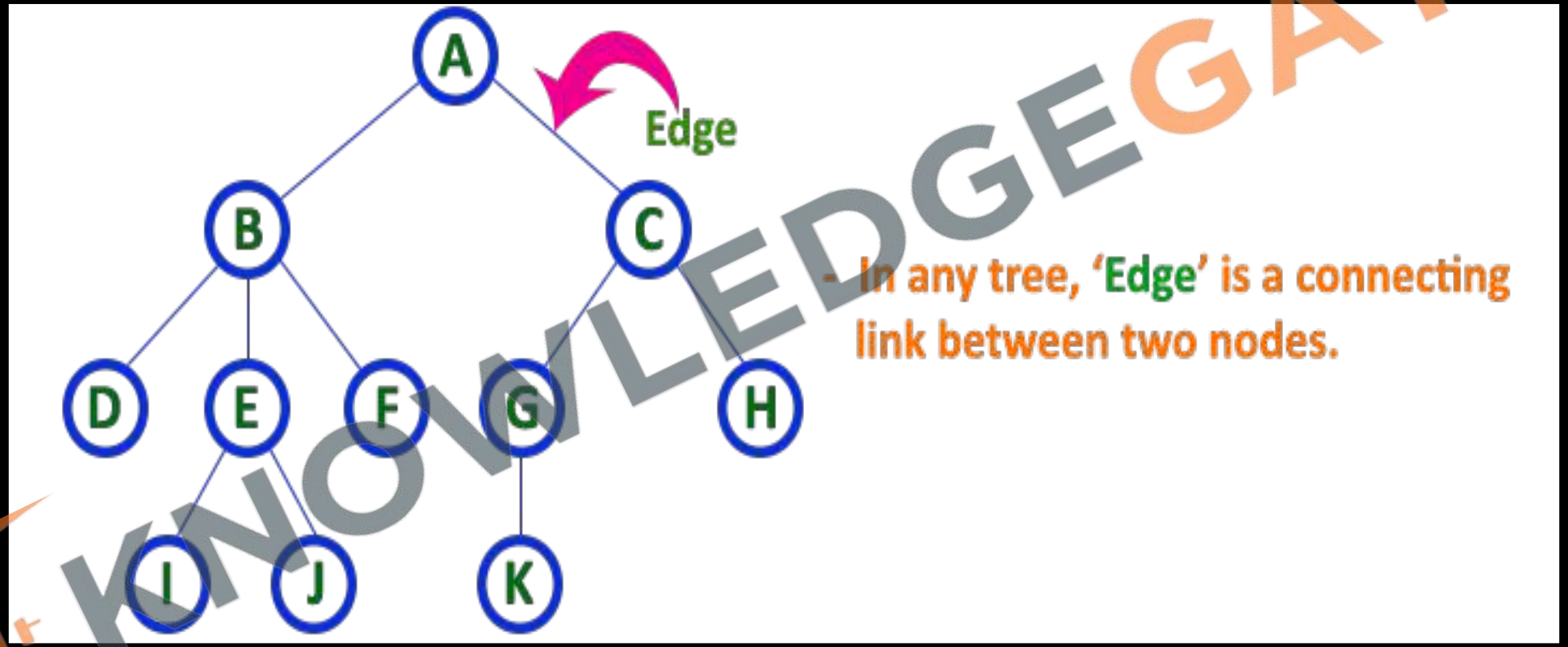
- The first/Top most node is called as Root Node. We always have exactly one root node in every tree. We can say that root node is the origin of tree data structure.



KNOWLEDGEGATE

Edge

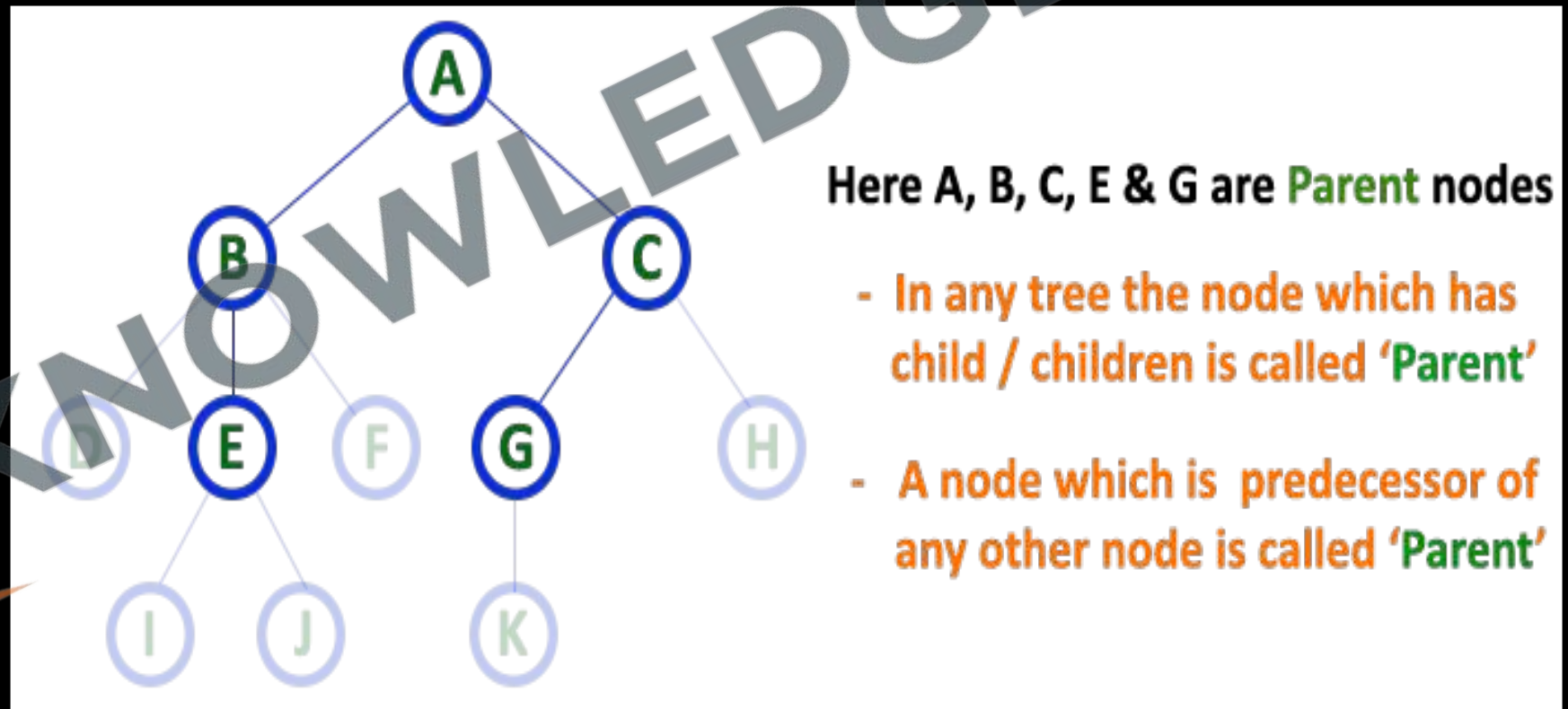
- In a tree data structure, the connecting link between any two nodes is called as EDGE. In a tree with 'N' number of nodes there will be exactly of 'N-1' number of edges.



KNOWLEDGEGATE

Parent

- In a tree data structure, the node which is predecessor of any node is called as PARENT NODE.
- In simple words, the node which has branch from it to any other node is called as parent node. Parent node can also be defined as "The node which has child / children".

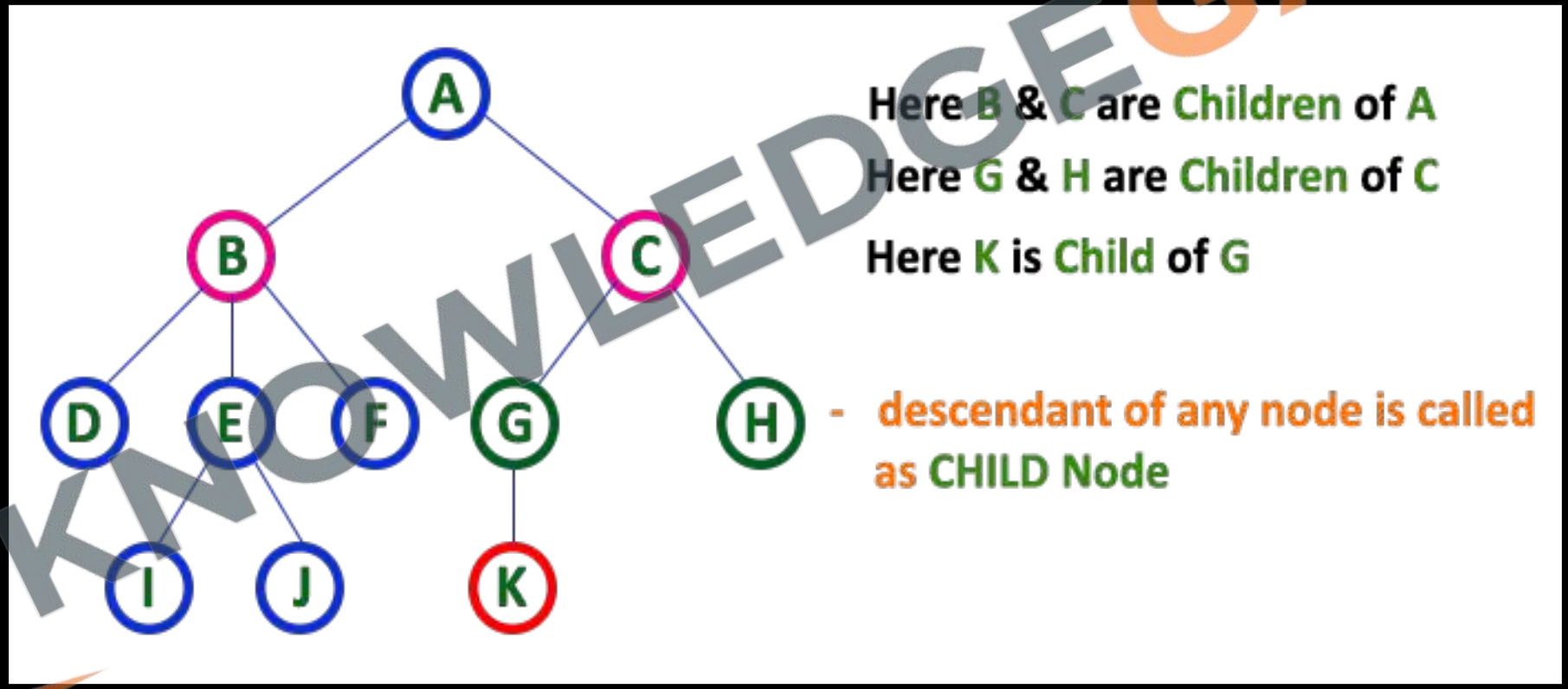


KNOWLEDGEGATE

Child

- In a tree data structure, the node which is descendant of any node is called as CHILD Node.

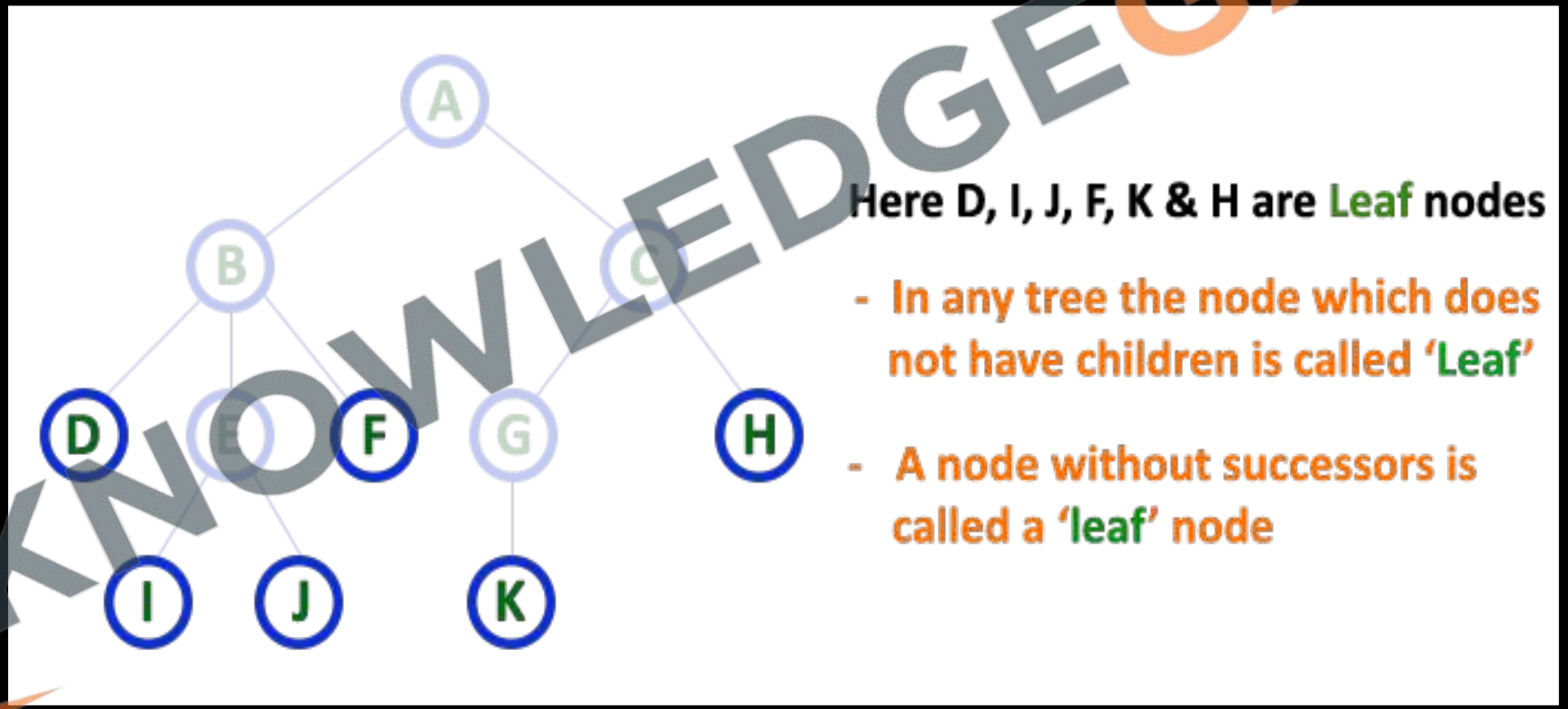
- In simple words, the node which has a link from its parent node is called as child node. In a tree, any parent node can have any number of child nodes. In a tree, all the nodes except root are child nodes.



KNOWLEDGEGATE

Leaf / External

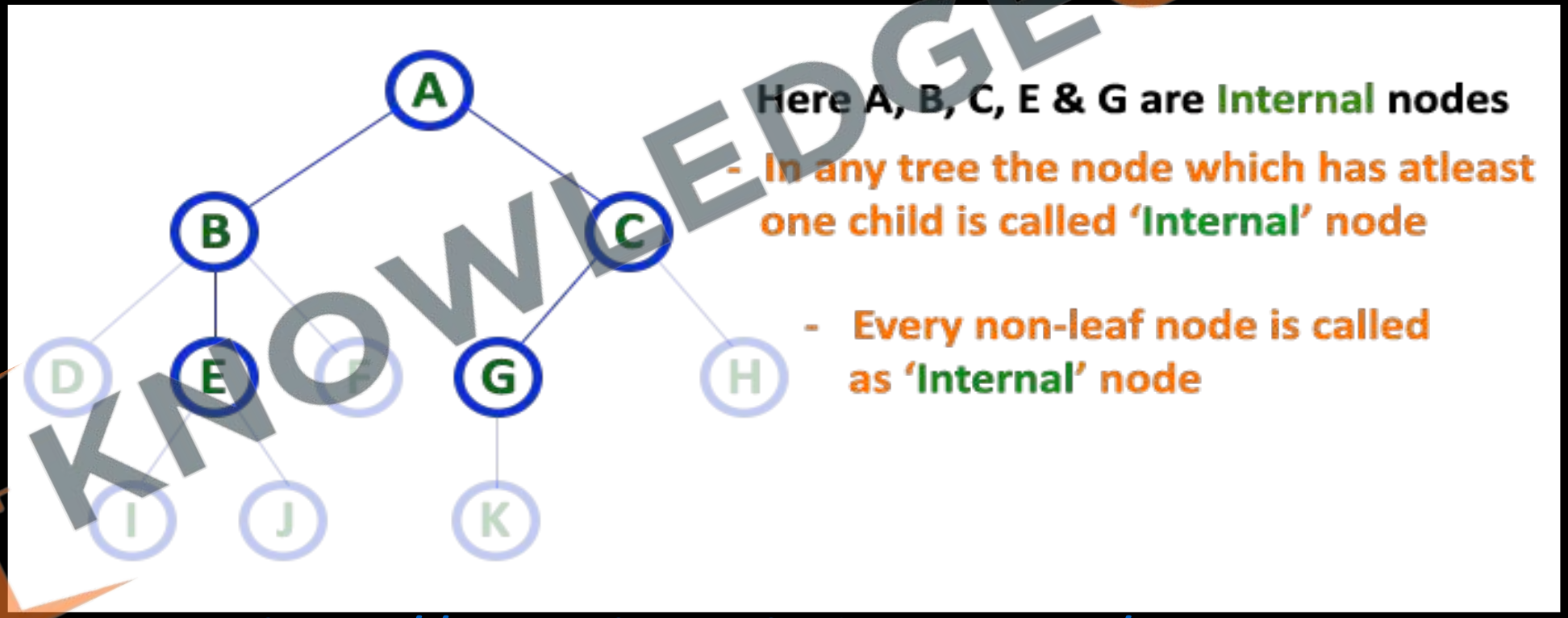
- In a tree data structure, the node which does not have a child is called as LEAF Node. In simple words, a leaf is a node with no child.
- In a tree data structure, the leaf nodes are also called as External Nodes. External node is also a node with no child. In a tree, leaf node is also called as 'Terminal' node.



KNOWLEDGEGATE

Internal Nodes

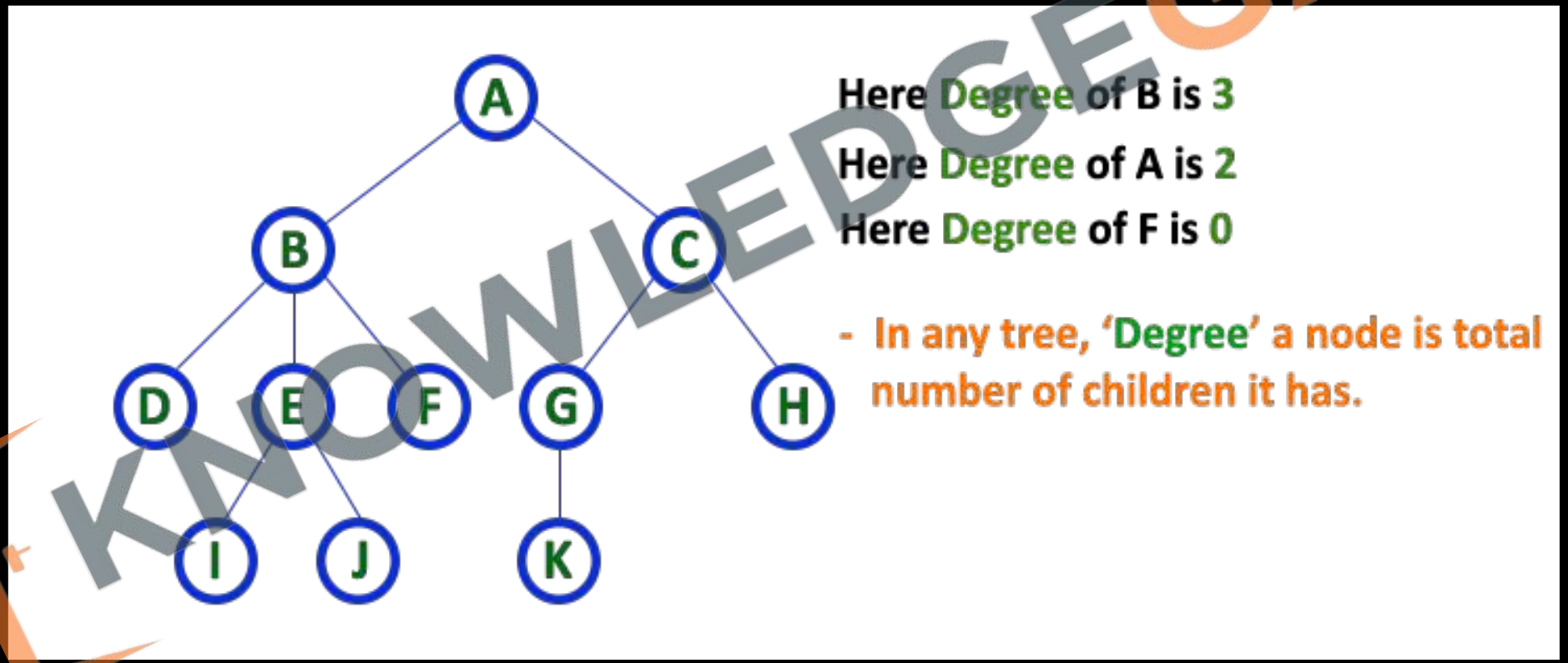
- In a tree data structure, the node which has at least one child is called as INTERNAL Node. In simple words, an internal node is a node with at least one child.
- In a tree data structure, nodes other than leaf nodes are called as Internal Nodes. The root node is also said to be Internal Node if the tree has more than one node. Internal nodes are also called as 'Non-Terminal' nodes.



KNOWLEDGEGATE

Degree

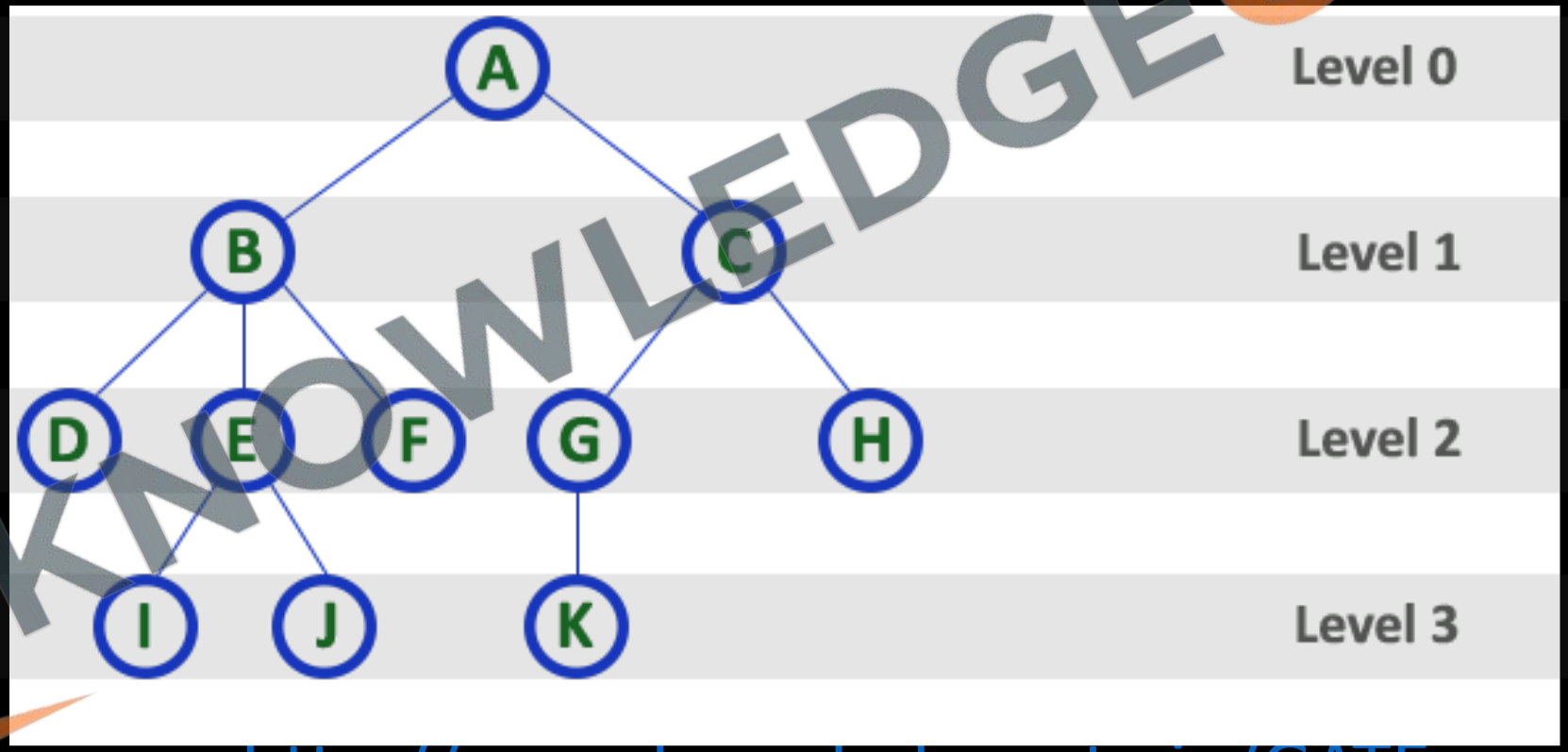
- In a tree data structure, the total number of children of a node is called as DEGREE of that Node. In simple words, the Degree of a node is total number of children it has.
- The highest degree allowed of a node in a tree is called as 'Degree of Tree'



KNOWLEDGEGATE

Level / Depth / Height

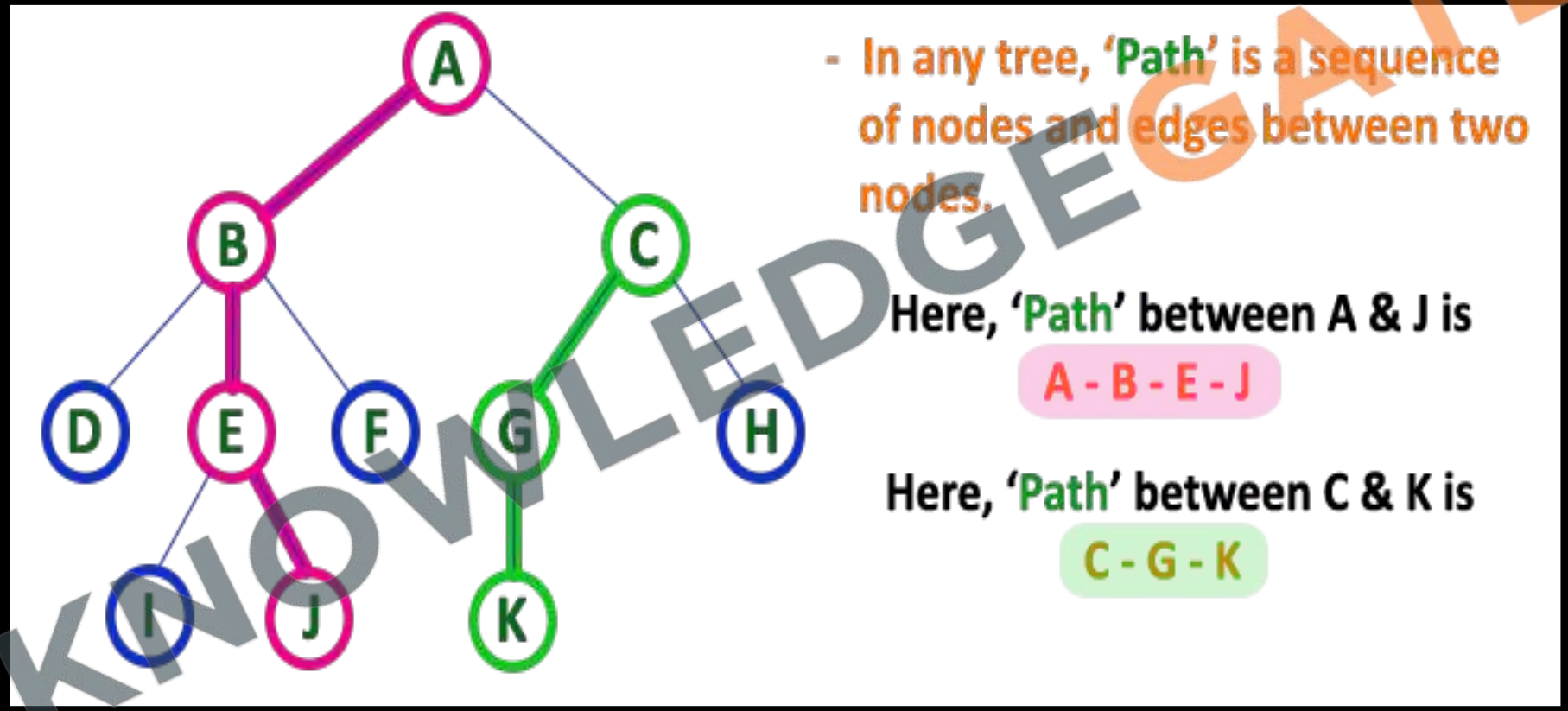
- In a tree data structure, the root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on...
- In simple words, in a tree each step from top to bottom is called as a Level and the Level count starts with '0' and incremented by one at each level (Step).



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Path

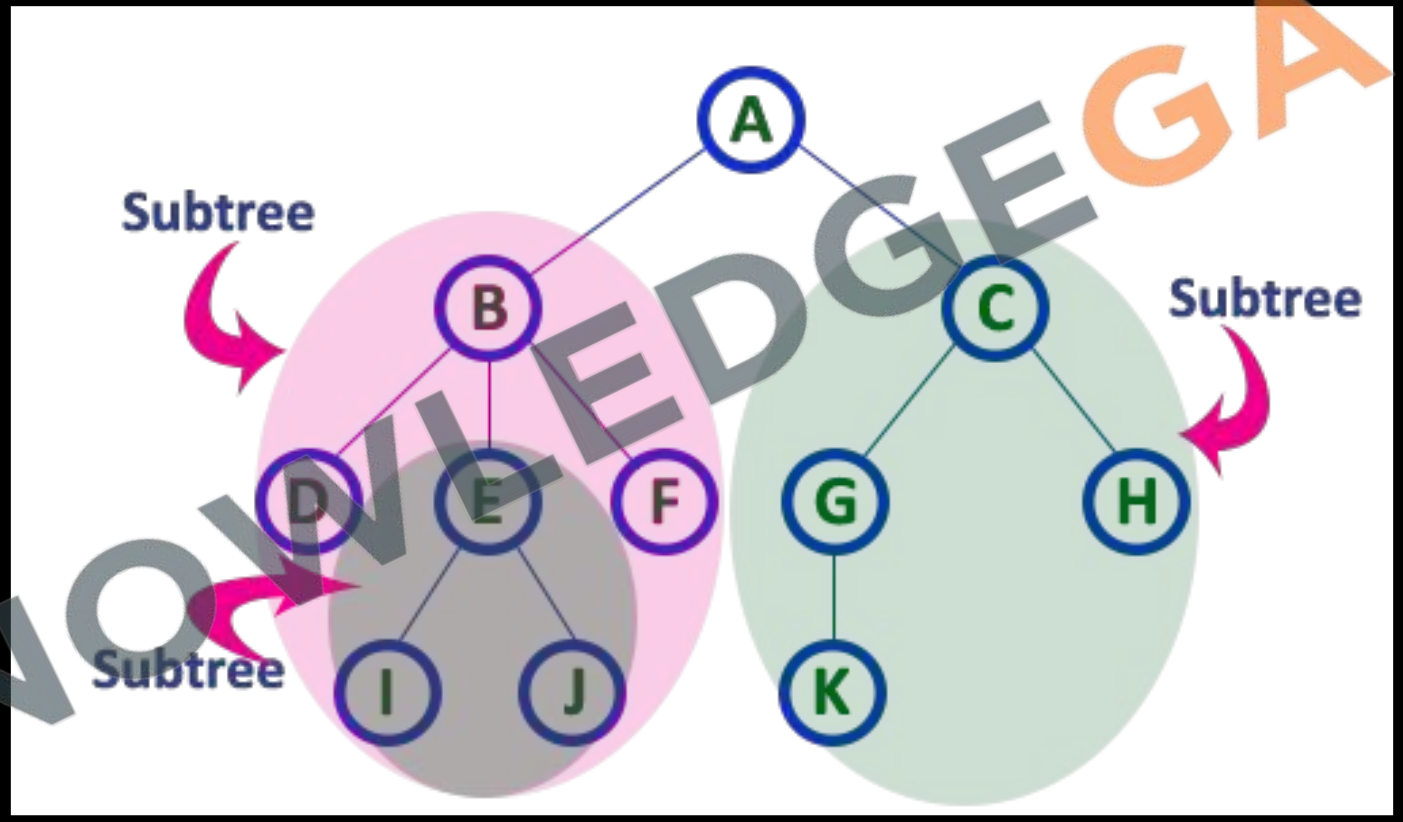
- In a tree data structure, the sequence of Nodes and Edges from one node to another node is called as PATH between that two Nodes. Length of a Path is total number of edge in that path. In below example the path A - B - E - J has length 4.



KNOWLEDGEGATE

Sub Tree

- In a tree data structure, each child from a node forms a subtree recursively. Every child node will form a subtree on its parent node.

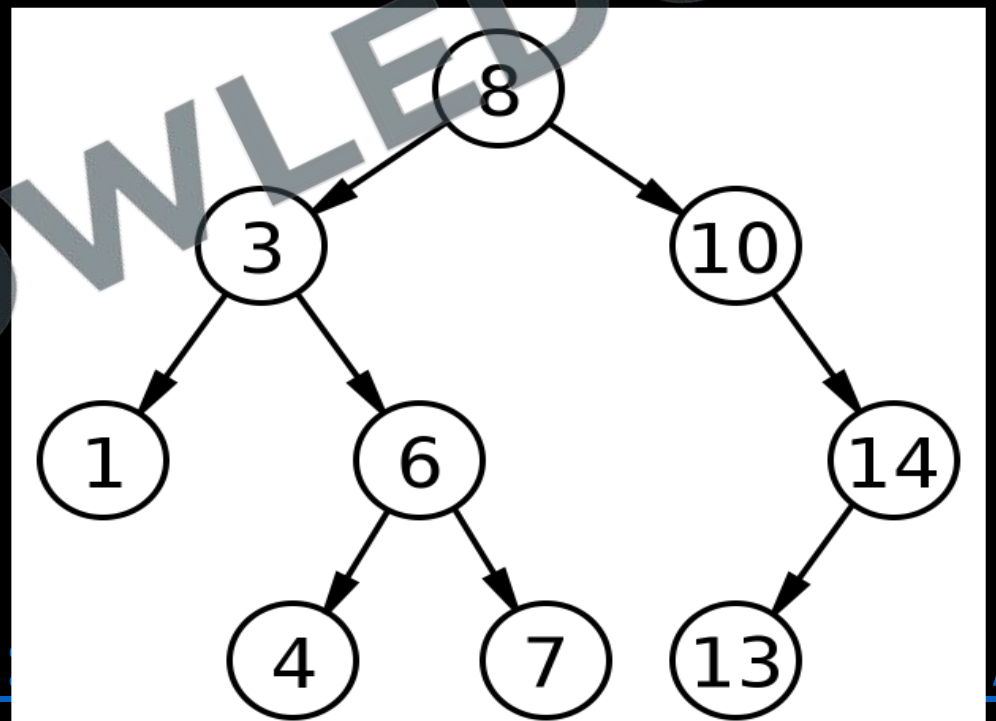


KNOWLEDGEGATE

Binary tree

- A binary tree T is defined as a finite set of elements called nodes such that,
  - T is empty (null tree)
  - T contain a distinguished node R, called the root of T, and the remaining nodes of T form an ordered pair of disjoint binary tree T_1 and T_2
- Direct: - A tree T in which any node can have maximum two children (left and right)

```
struct node {
   int data;
   struct node* left;
   struct node* right;
}
```



KNOWLEDGEGATE

http ATE

Q Let T be a binary search tree with 15 nodes. The minimum and maximum possible heights of T are: _____

(A) 4 and 15 respectively

(B) 3 and 14 respectively

(C) 4 and 14 respectively

(D) 3 and 15 respectively

KNOWLEDGEGATE

Q The height of a tree is the length of the longest root-to-leaf path in it. The maximum and minimum number of nodes in a binary tree of height 5 are

(A) 63 and 6, respectively

(B) 64 and 5, respectively

(C) 32 and 6, respectively

(D) 31 and 5, respectively

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q The height of a binary tree is the maximum number of edges in any root to leaf path. The maximum number of nodes in a binary tree of height h is:

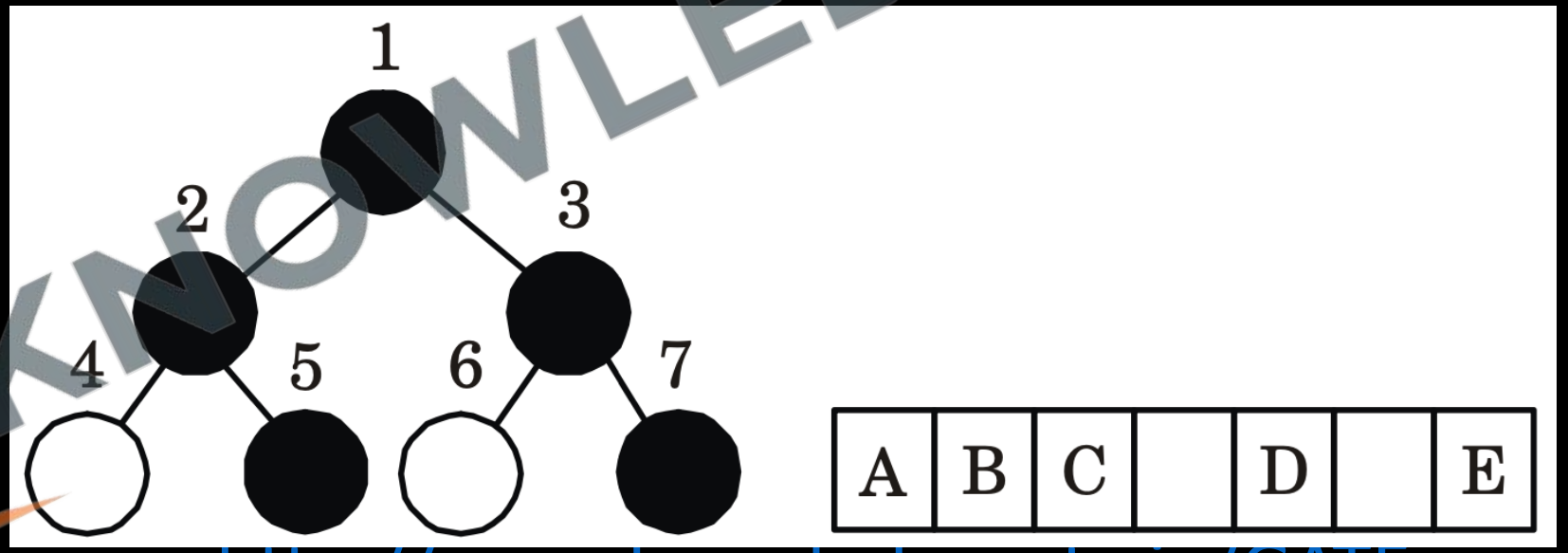
a) $2^h - 1$ **b)** $2^{h-1} - 1$ **c)** $2^{h+1} - 1$ **d)** 2^{h+1}

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Binary tree representation using array

- Binary tree can be represented using an array
- General representation
- The root is at index ‘1’
- For any given node at position ‘i’
  - Left Child is at position 2*i
  - Right Child is at position 2*i + 1
- If a node does not have a left or right child, that position in the array remains empty or is filled with a special value indicating it's vacant (like null or -1)



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- In fact, a binary tree that has *n*-elements may require an array of size up to 2^n (including position 0) for its representation.

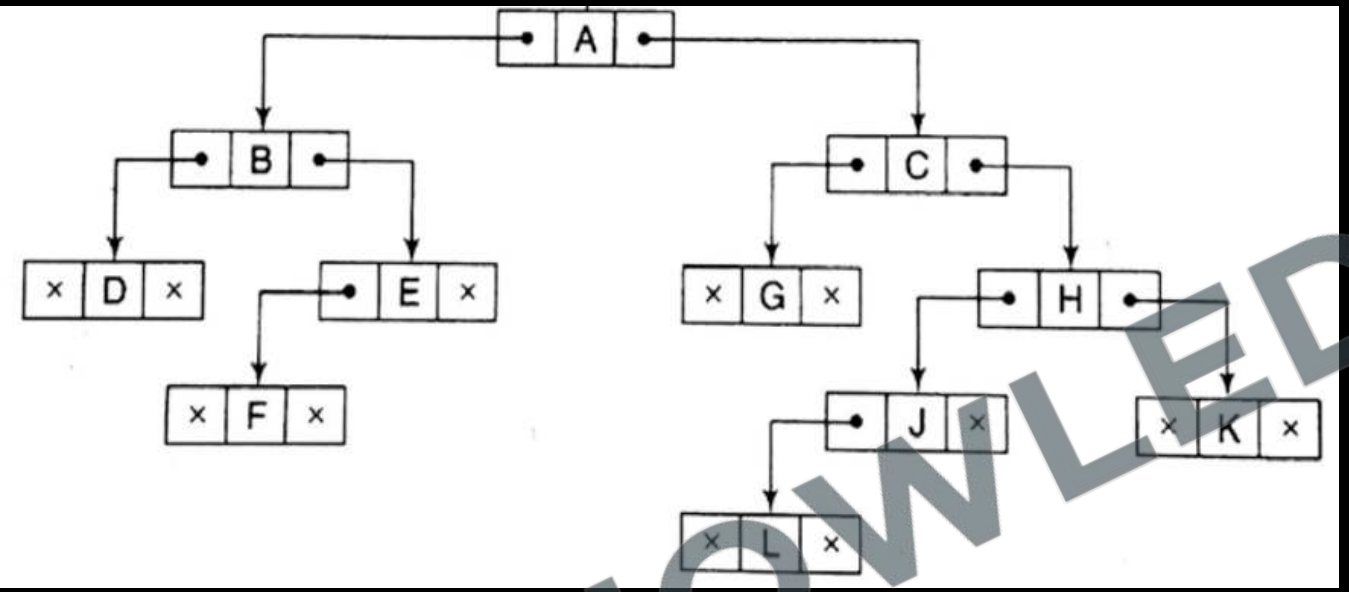
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Linked representation of binary tree

1. Consider a binary tree T which uses three parallel arrays, INFO, LEFT and RIGHT, and a pointer variable ROOT.
2. First of all, each node N of T will correspond to a location K such that :
   a. INFO[K] contains the data at the node N.
   b. LEFT[K] contains the location of the left child of node N.
   c. RIGHT[K] contains the location of the right child of node N.
3. ROOT will contain the location of the root R of T.
4. If any subtree is empty, then the corresponding pointer will contain the null value.
5. If the tree T itself is empty, then ROOT will contain the null value.
6. INFO may actually be a linear array of records or a collection of parallel arrays.

KNOWLEDGEGATE



ROOT
5

AVAIL
8

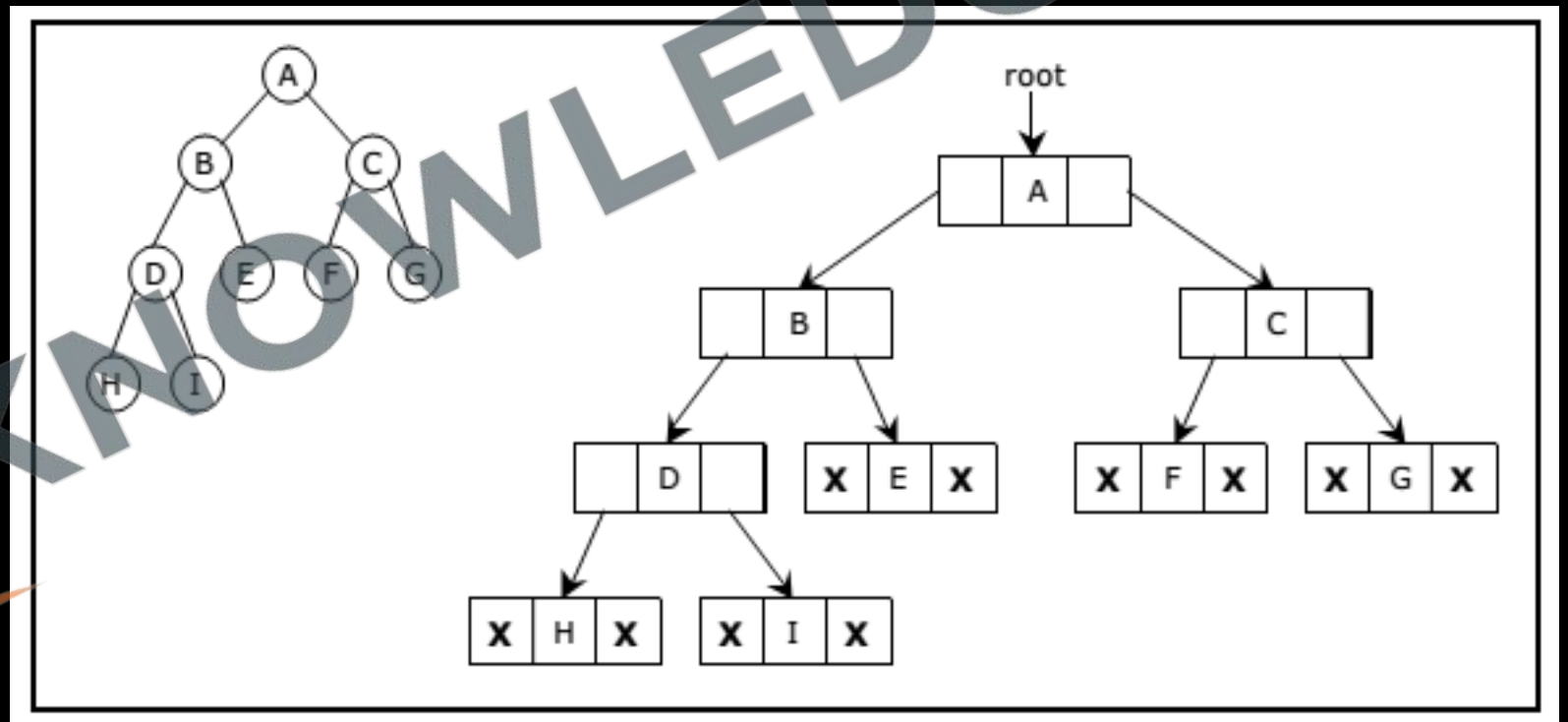
	INFO	LEFT	RIGHT
1	K	0	0
2	C	3	6
3	G	0	0
4		14	
5	A	10	2
6	H	17	1
7	L	0	0
8		9	
9		4	
10	B	18	13
11		19	
12	F	0	0
13	E	12	0
14		15	
15		16	
16		11	
17	J	7	0
18	D	0	0
19		20	
20		0	

KNOWLEDGEGATE
http://www.knowledgeg

Binary tree representation using Linked List

- A binary tree can be efficiently represented using a linked list structure where each node of the tree is represented by a separate node in the linked list. This linked structure is typically referred to as a "node-based" representation.

- Each node in the linked list contains the following components:
  - **Data**: The value stored in the node.
  - **Left Pointer**: A pointer pointing to the left child node.
  - **Right Pointer**: A pointer pointing to the right child node.



KNOWLEDGEGATE

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node
{
	int data;
	struct Node* left;
	struct Node* right;
} Node;

Node* createNode(int data)
{
	Node* newNode = (Node*) malloc(sizeof(Node));
	if (!newNode)
	{
		printf("Memory error\n");
		exit(1);
	}
	newNode->data = data;
	newNode->left = NULL;
	newNode->right = NULL;
	return newNode;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

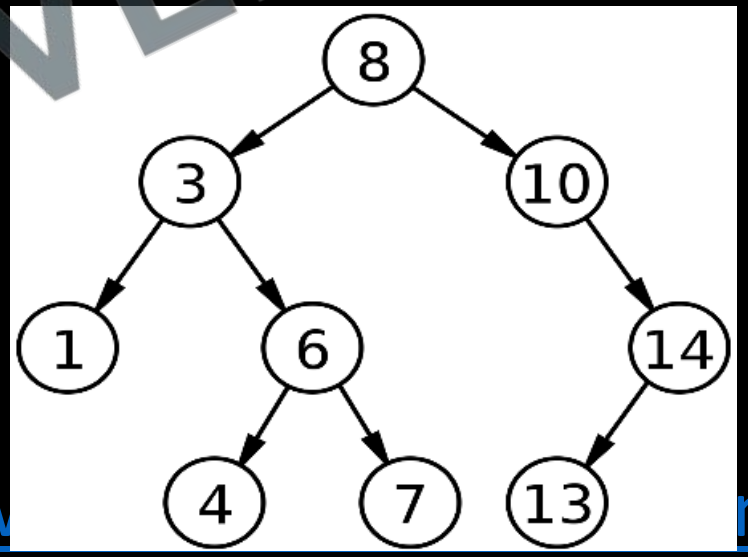
```
void insert(Node** root, int data)
{
    if (*root == NULL)
    {
        *root = createNode(data);
        return;
    }
    if (data < (*root)->data)
    {
        insert(&((*root)->left), data);
    }
    else
    {
        insert(&((*root)->right), data);
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Traversal of binary tree

- The process of visiting (checking and/or updating) each node in a tree data structure, exactly once in called tree traversal. Such traversals are classified by the order in which the nodes are visited.
- Unlike linked lists, one-dimensional arrays and other linear data structures, which are canonically traversed in linear order, trees may be traversed in multiple ways.
- They may be traversed in depth-first or breadth-first order. There are three common ways to traverse them in depth-first order: in-order, pre-order and post-order. Beyond these basic traversals, various more complex or hybrid schemes are possible, such as depth-limited searches like iterative deepening depth-first search.



KNOWLEDGEGATE

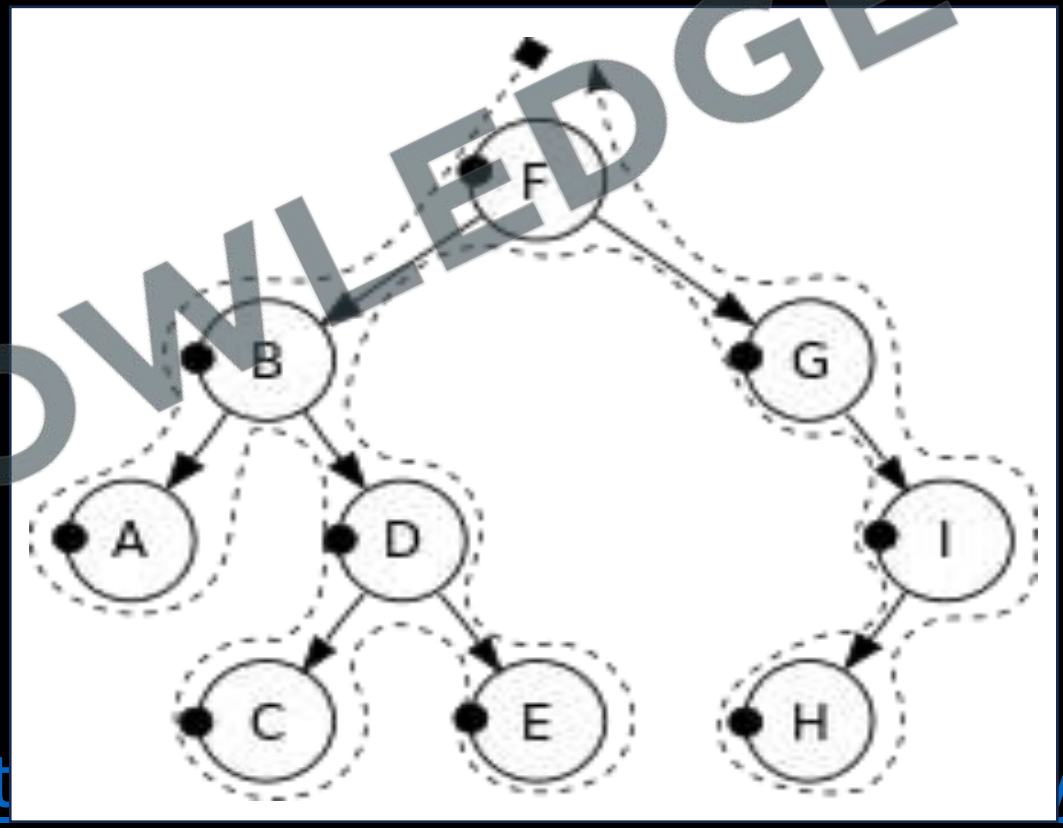
- Some applications do not require that the nodes be visited in any particular order as long as each node is visited precisely once. For other applications, nodes must be visited in an order that preserves some relationship.
- These steps can be done *in any order*. If (L) is done before (R), the process is called left-to-right traversal, otherwise it is called right-to-left traversal. The following methods show left-to-right traversal:

KNOWLEDGEGATE

Pre-order (Root L R)

Pre-order: F, B, A, D, C, E, G, I, H.

- Check if the current node is empty or null.
- Display the data part of the root (or current node).
- Traverse the left subtree by recursively calling the pre-order function.
- Traverse the right subtree by recursively calling the pre-order function.



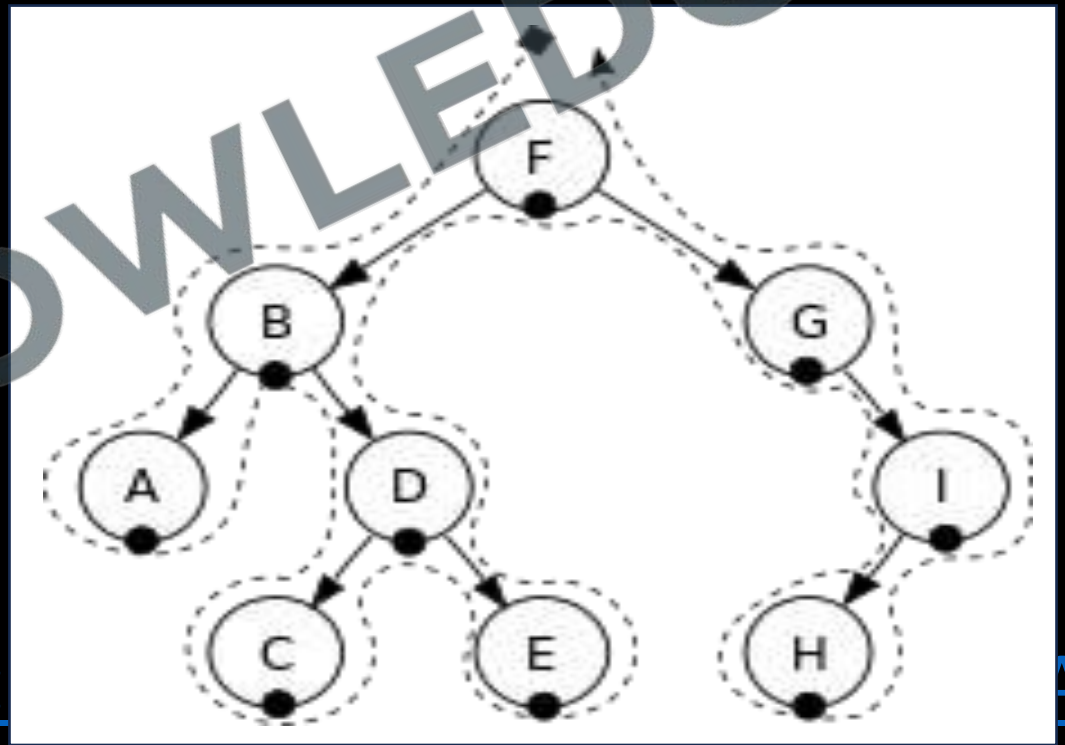
KNOWLEDGEGATE

htt ATE

In-order (L root R)

In-order: A, B, C, D, E, F, G, H, I.

- Check if the current node is empty or null.
- Traverse the left subtree by recursively calling the in-order function.
- Display the data part of the root (or current node).
- Traverse the right subtree by recursively calling the in-order function.
- In a binary search tree, in-order traversal retrieves data in sorted order.



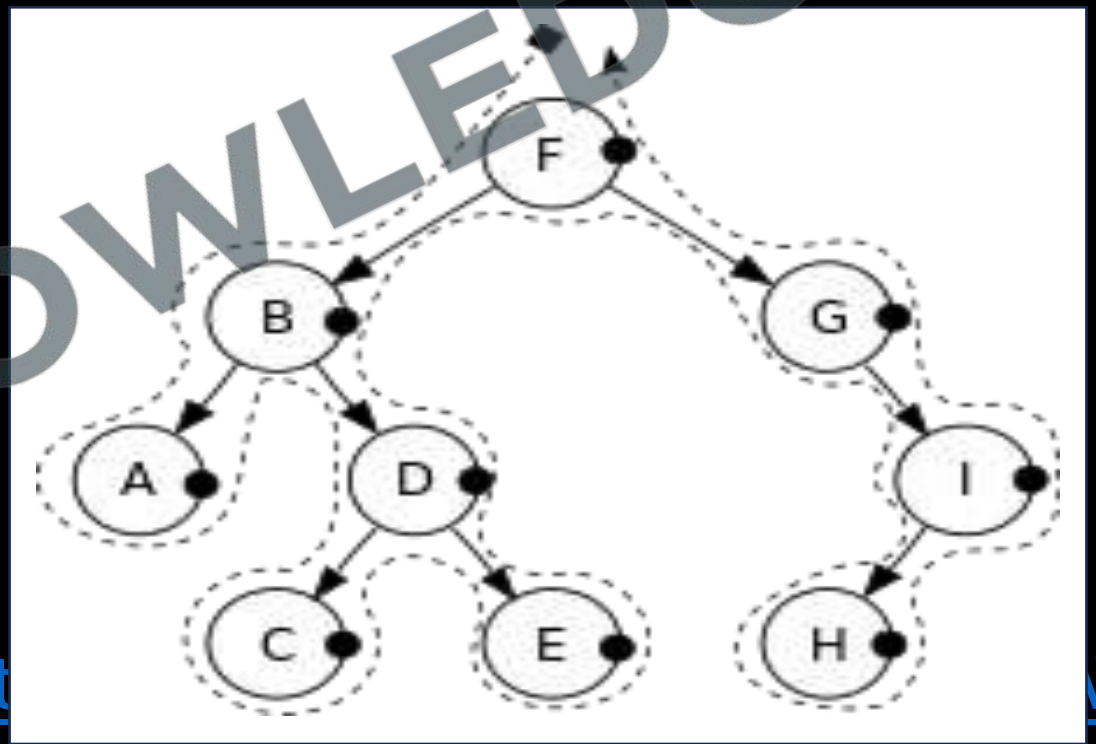
KNOWLEDGEGATE

htt ATE

Post-order (L R Root)

Post-order: A, C, E, D, B, H, I, G, F.

- Check if the current node is empty or null.
- Traverse the left subtree by recursively calling the post-order function.
- Traverse the right subtree by recursively calling the post-order function.
- Display the data part of the root (or current node).



KNOWLEDGEGATE

Q The post order traversal of a binary tree is 8, 9, 6, 7, 4, 5, 2, 3, 1. The inorder traversal of the same tree is 8, 6, 9, 4, 7, 2, 5, 1, 3. The height of a tree is the length of the longest path from the root to any leaf. The height of the binary tree above is _____.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

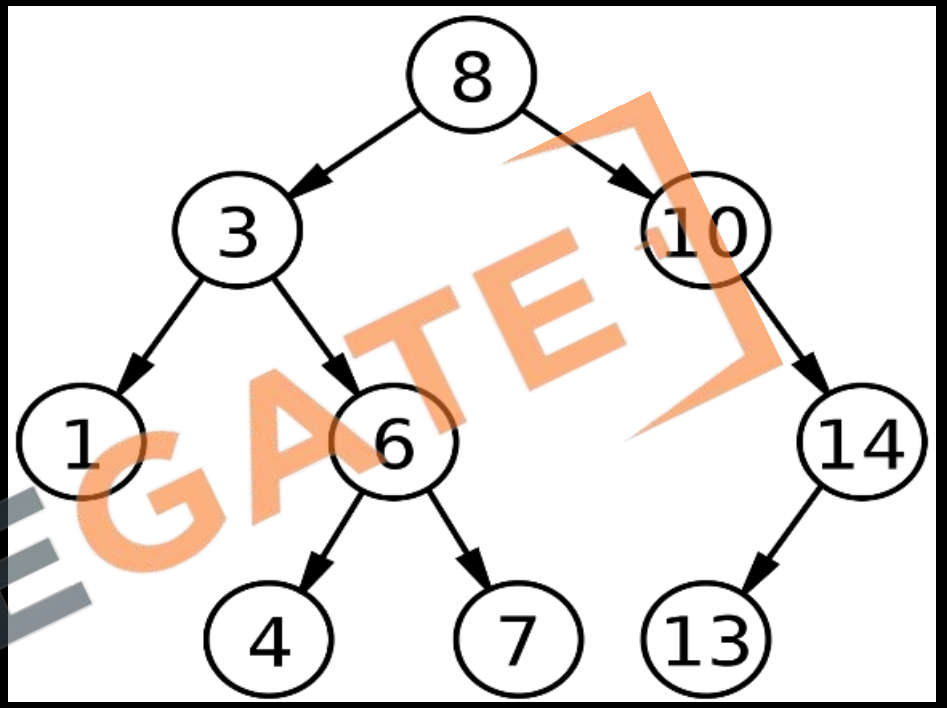
Inorder : D B H E A I F J C G

Preorder : A B D E H C F I J G

KNOWLEDGEGATE

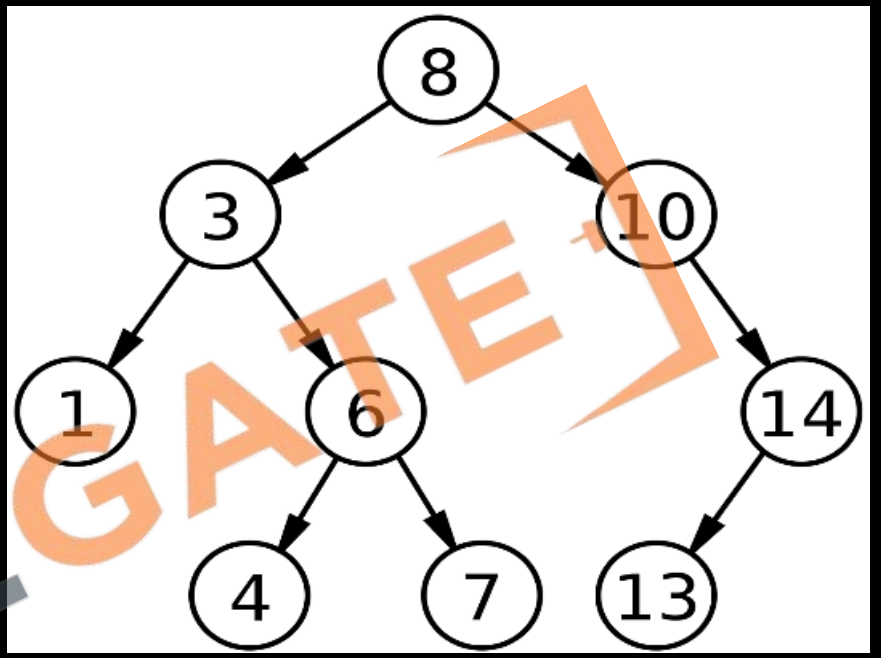
http://www.knowledgegate.in/GATE

```
void inorderTraversal(Node* root)
{
    if (root == NULL)
        return;
    inorderTraversal(root->left);
    printf("%d ", root->data);
    inorderTraversal(root->right);
}
```



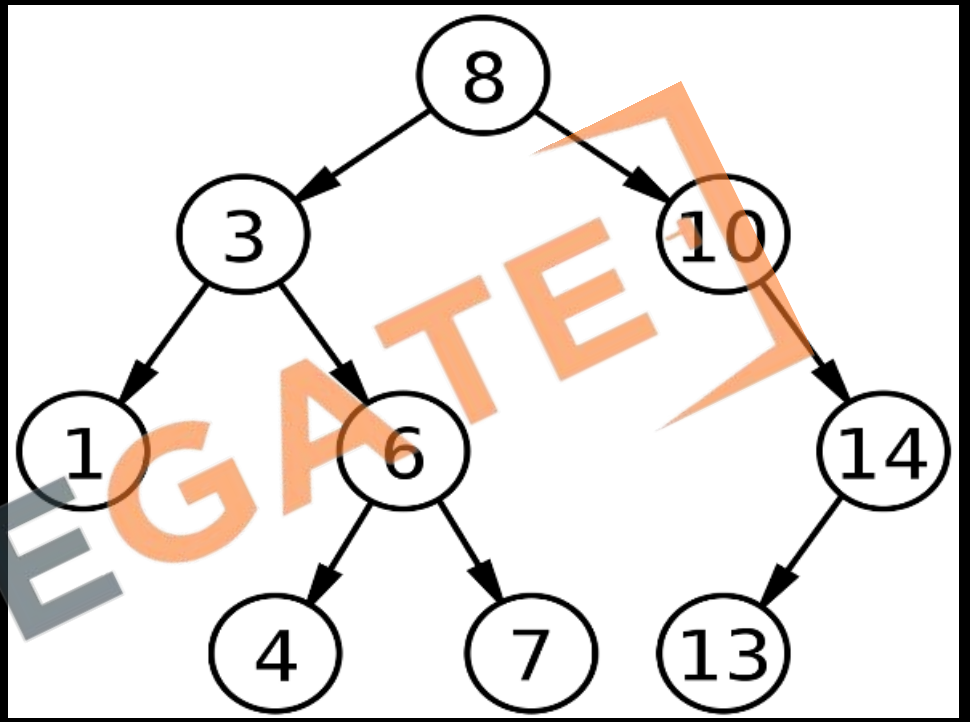
KNOWLEDGEGATE

```
void preorderTraversal(Node* root)
{
    if (root == NULL)
        return;
    printf("%d ", root->data);
    preorderTraversal(root->left);
    preorderTraversal(root->right);
}
```



KNOWLEDGE GATE

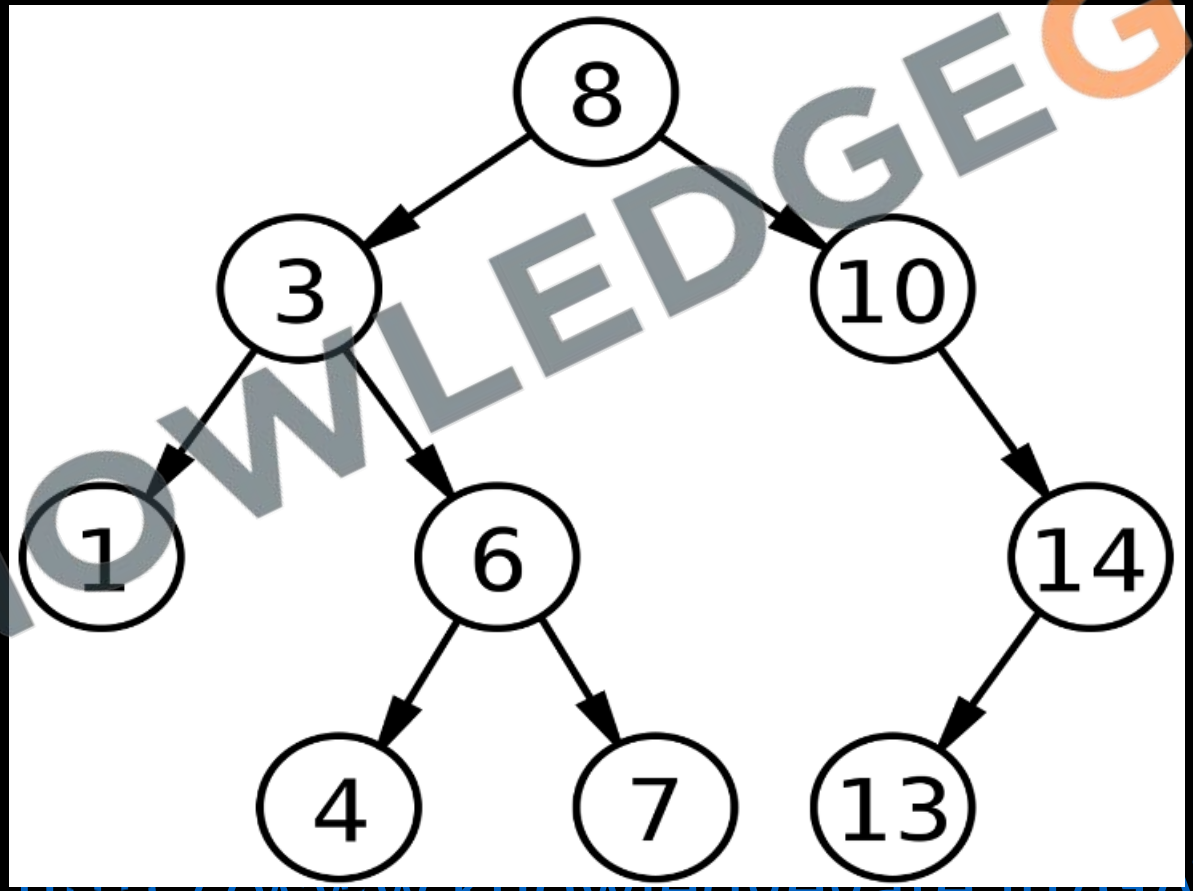
```
void postorderTraversal(Node* root)
{
    if (root == NULL)
        return;
    postorderTraversal(root->left);
    postorderTraversal(root->right);
    printf("%d ", root->data);
}
```



KNOWLEDGEGATE

Binary search tree / Ordered tree / Sorted binary tree

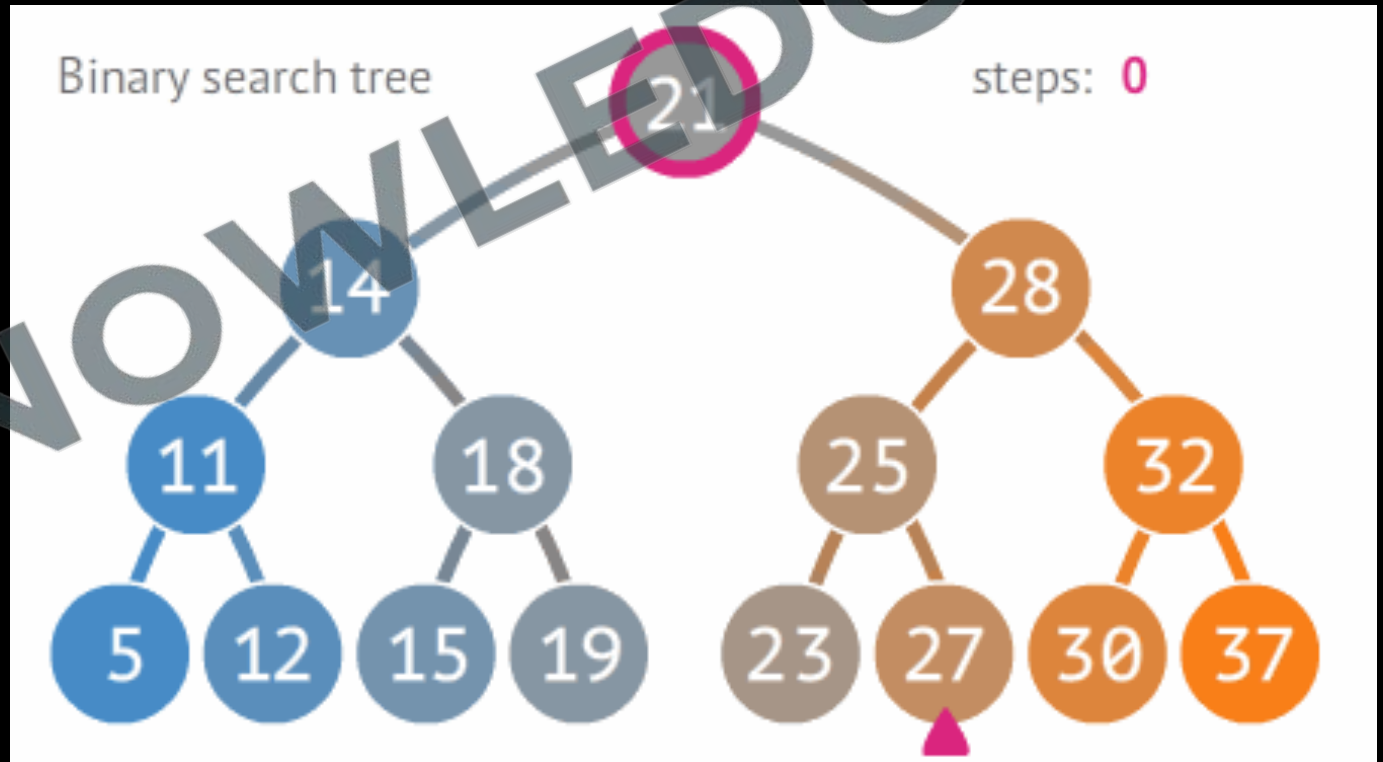
- A binary search tree (BST) is a binary tree in which left subtree of a node contains a key less than the node's key and right subtree of a node contains only the nodes with key greater than the node's key. Left and right sub tree must each also be a binary search tree.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Searching

- We begin by examining the root node. If the tree is *null*, the key we are searching for does not exist in the tree. Otherwise, if the key equals that of the root, the search is successful and we return the node.
- If the key is less than that of the root, we search the left subtree. Similarly, if the key is greater than that of the root, we search the right subtree.
- This process is repeated until the key is found or the remaining subtree is *null*. If the searched key is not found after a *null* subtree is reached, then the key is not present in the tree.



KNOWLEDGEGATE

Operations

- Binary search trees support three main operations: insertion of elements, deletion of elements, and lookup (checking whether a key is present).

KNOWLEDGEGATE

Q While inserting the elements 71, 65, 84, 69, 67, 83 in an empty binary search tree (BST) in the sequence shown, the element in the lowest level is

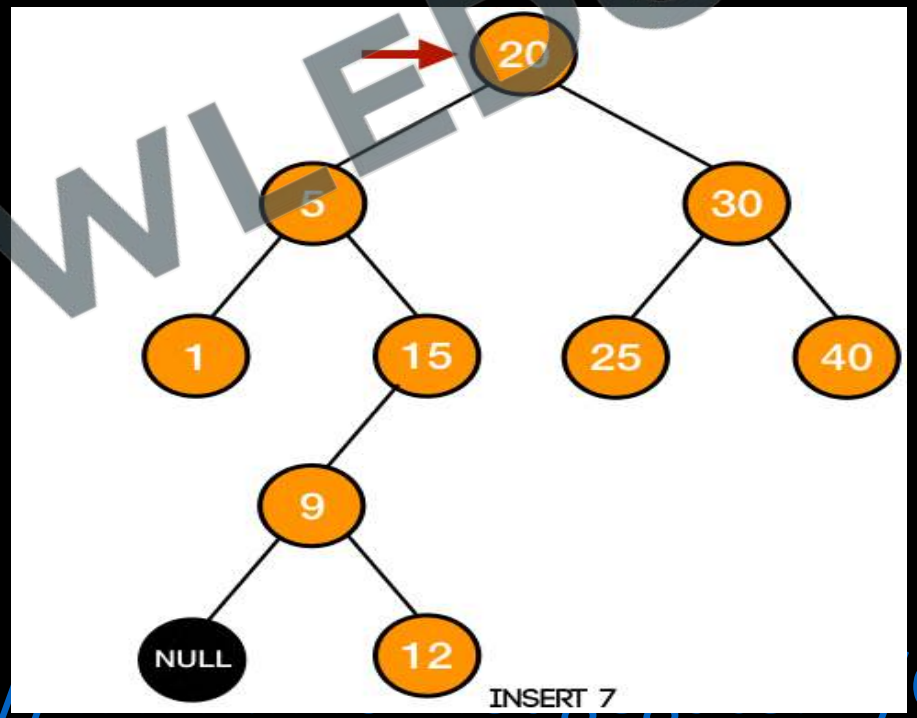
(A) 65 **(B)** 67 **(C)** 69 **(D)** 83

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Insertion

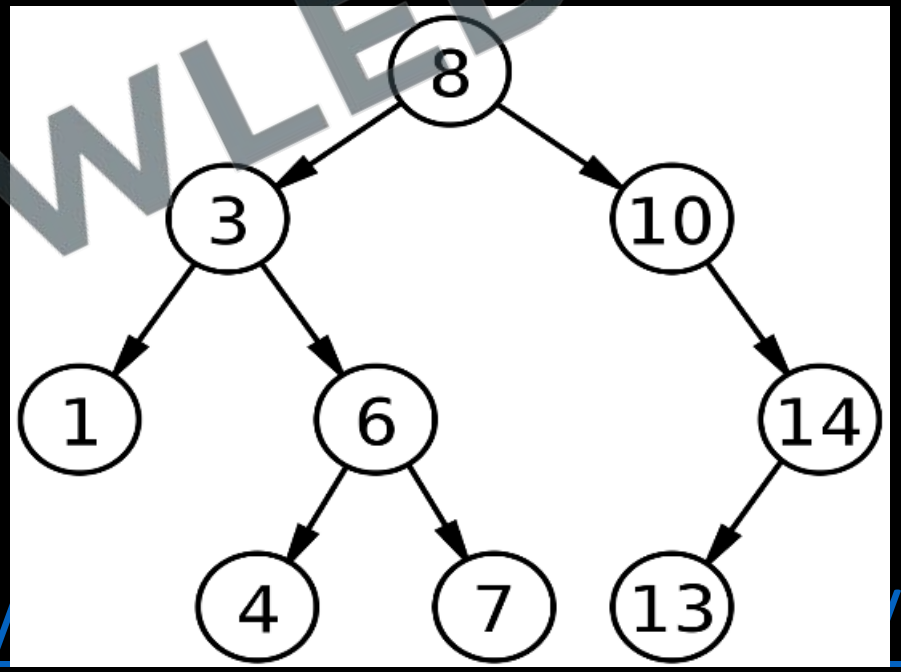
- Insertion begins as a search would begin; if the key is not equal to that of the root, we search the left or right subtrees as before.
- Eventually, we will reach an external node and add the new key-value pair (here encoded as a record ‘new Node') as its right or left child, depending on the node's key.
- In other words, we examine the root and recursively insert the new node to the left subtree if its key is less than that of the root, or the right subtree if its key is greater than or equal to the root.



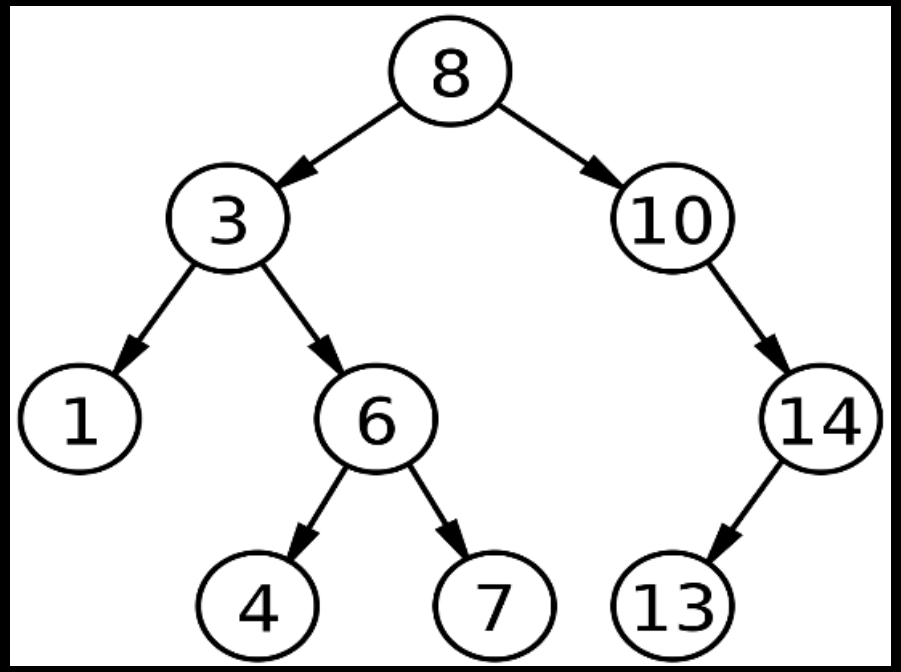
KNOWLEDGEGATE

Deletion

- Deleting a node with no children: simply remove the node from the tree.
- Deleting a node with one child: remove the node and replace it with its child.
- Deleting a node with two children: call the node to be deleted *D*. Do not delete *D*.
- Instead, choose either its in-order predecessor node or its in-order successor node as replacement node *E* (s. figure). Copy the user values of *E* to *D*.
- If *E* does not have a child simply remove *E* from its previous parent *G*. If *E* has a child, say *F*, it is a right child. Replace *E* with *F* at *E*'s parent.



KNOWLEDGEGATE



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Algorithm	Average	Worst case
Space	O(n)	O(n)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

- The major advantage of binary search trees over other data structures is that the related sorting algorithms and search algorithm such as in-order traversal can be very efficient; they are also easy to code

KNOWLEDGEGATE

Q The pre-order traversal of a binary search tree is given by 12, 8, 6, 2, 7, 9, 10, 16, 15, 19, 17, 20. Then the post-order traversal of this tree is:

a) 2, 6, 7, 8, 9, 10, 12, 15, 16, 17, 19, 20

b) 2, 7, 6, 10, 9, 8, 15, 17, 20, 19, 16, 12

c) 7, 2, 6, 8, 9, 10, 20, 17, 19, 15, 16, 12

d) 7, 6, 2, 10, 9, 8, 15, 16, 17, 20, 19, 12

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Which of the following is/are correct inorder traversal sequence(s) of binary search tree(s)? **1**. 3, 5, 7, 8, 15, 19, 25

2. 5, 8, 9, 12, 10, 15, 25

3. 2, 7, 10, 8, 14, 16, 20

4. 4, 6, 7, 9, 18, 20, 25

a) 1 and 4 only **b)** 2 and 3 only **c)** 2 and 4 only **d)** 2 only

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Suppose that we have numbers between 1 and 100 in a binary search tree and want to search for the number 55. Which of the following sequences CANNOT be the sequence of nodes examined?

(a) {10, 75, 64, 43, 60, 57, 55}

(b) 190, 12, 68, 34, 62, 45, 55}

(c) (9, 85, 47, 68, 43, 57, 55}

(d) {79, 14, 72, 56, 16, 53, 55}

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
Node* searchBST(Node* root, int data)
{
    if (root == NULL || root->data == data)
    {
        return root;
    }
    if (data < root->data)
    {
        return searchBST(root->left, data);
    }
    return searchBST(root->right, data);
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
void insertBST(Node** root, int data)
{
    if (*root == NULL)
    {
        *root = createNode(data);
        return;
    }
    if (data < (*root)->data)
    {
        insertBST(&((*root)->left), data);
    }
    else if (data > (*root)->data)
    {
        insertBST(&((*root)->right), data);
    }
    else
    {
        printf("Element %d already exists in the BST.\n", data);
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
Node* deleteBST(Node* root, int data)
{
	if (!root)
		return root;
	if (data < root->data)
	{
		root->left = deleteBST(root->left, data);
	}
	else if (data > root->data)
	{
		root->right = deleteBST(root->right, data);
	}
	else
	{
		if (!root->left)
		{
			Node* temp = root->right;
			free(root);
			return temp;
		}
		else if (!root->right)
		{
			Node* temp = root->left;
			free(root);
			return temp;
		}
		Node* temp = findMinValueNode(root->right);
		root->data = temp->data;
		root->right = deleteBST(root->right, temp->data);
	}
	return root;
}
```

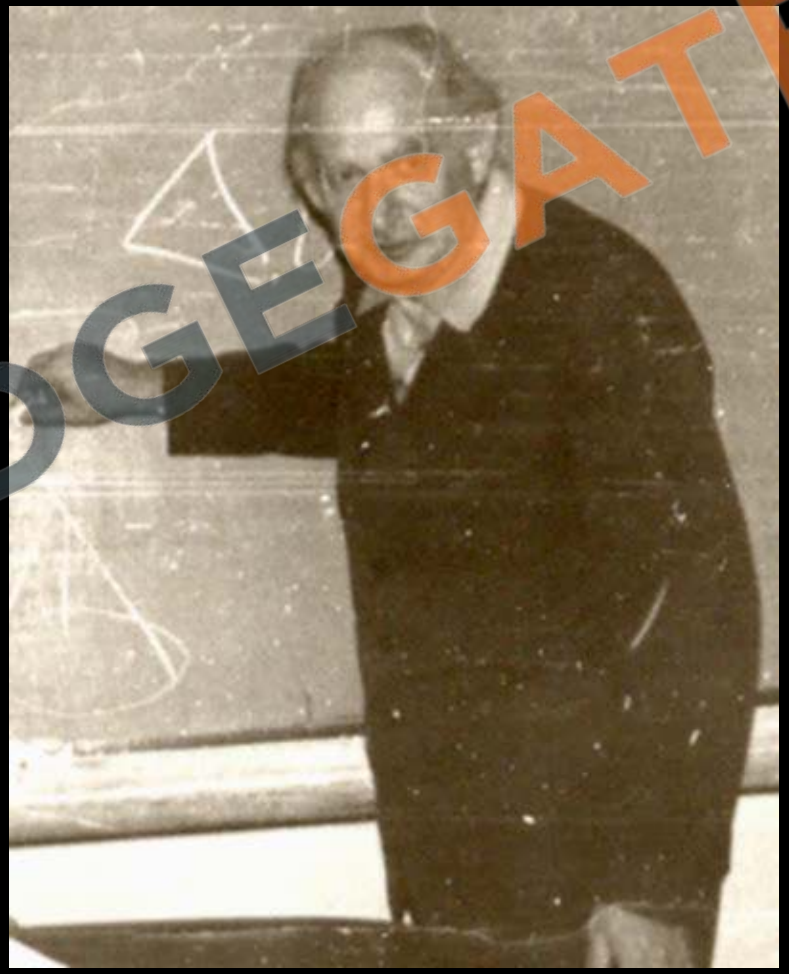
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

AVL tree

- In computer science, an **AVL tree** (named after inventors **A**delson-**V**elsky and **L**andis) is a self-balancing binary search tree. It was the first such data structure to be invented.

Adelson-Velsky

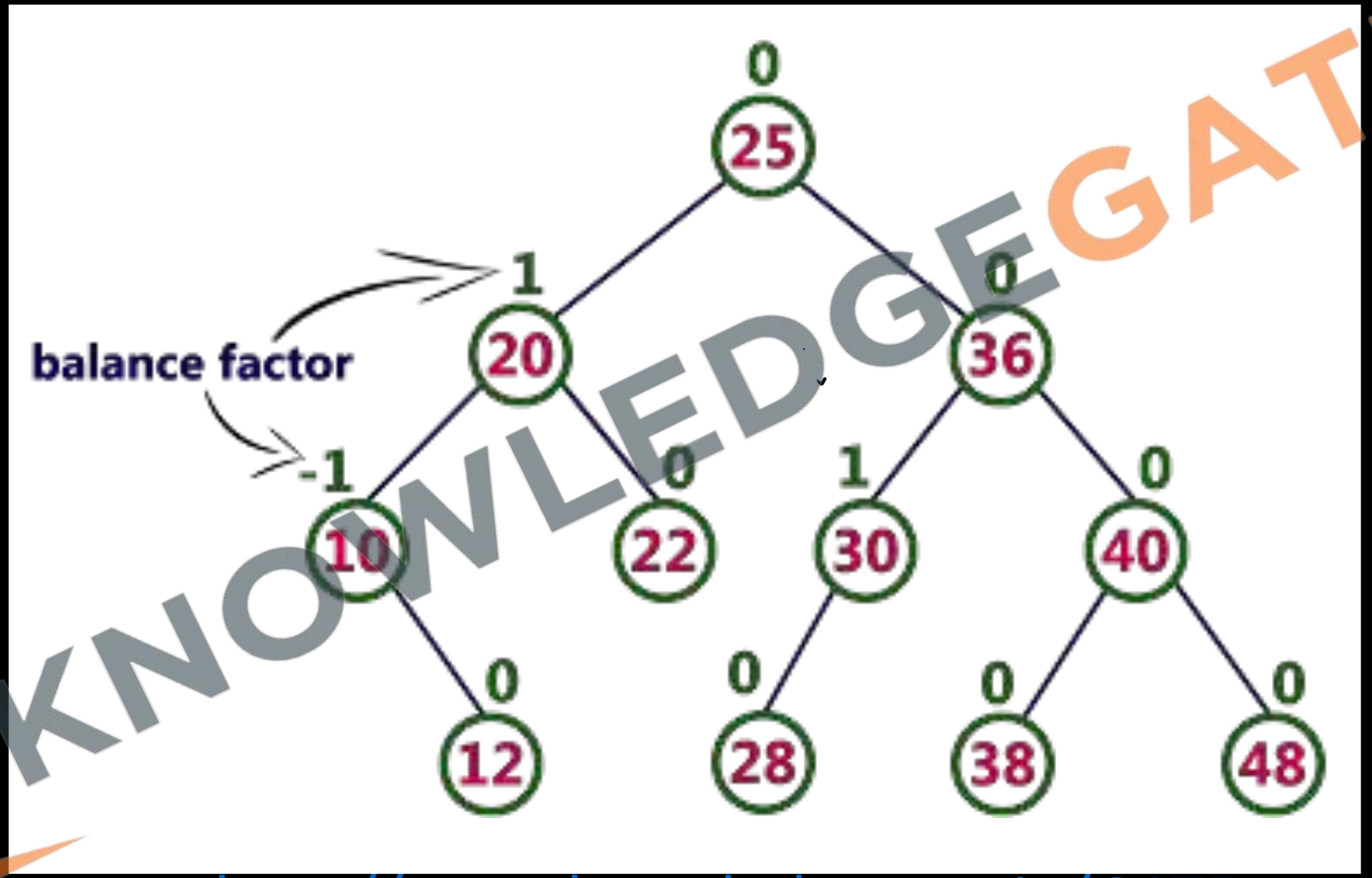


Landis

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- In an AVL tree, the heights of the two child subtrees of any node differ by at most one; if at any time they differ by more than one, rebalancing is done to restore this property.



KNOWLEDGEGATE

Balance factor

- In a binary tree the *balance factor* of a node N is defined to be the height difference Balance Factor(N): = Height (LeftSubtree(N)) – Height (RightSubtree(N)) of its two child subtrees.

- A binary tree is defined to be an *AVL tree* if the invariant Balance Factor(N) ∈ {–1, 0, +1} holds for every node N in the tree.

KNOWLEDGEGATE

- A node N with Balance Factor(N) > 0 is called "left-heavy“
- One with Balance Factor(N) < 0 is called "right-heavy“
- One with Balance Factor(N) = 0 is sometimes simply called "balanced".

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Insertion in an AVL tree

- Insert a node similarly as we do in binary search tree.
- After insertion start checking the balancing factor of each node in a bottom up fashion that is from newly inserted node towards the root.
- Stop on the first node whose balancing factor is violated and go two steps towards the newly inserted nodes. watch the movement, which is identified as the problem.

Problem	Solution
LL	R
RR	L
LR	LR
RL	RL

KNOWLEDGEGATE

Q Consider an empty AVL tree and insert the following nodes in sequence 21, 26, 30, 9, 4, 14, 28, 18, 15, 10, 2, 3, 7?

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

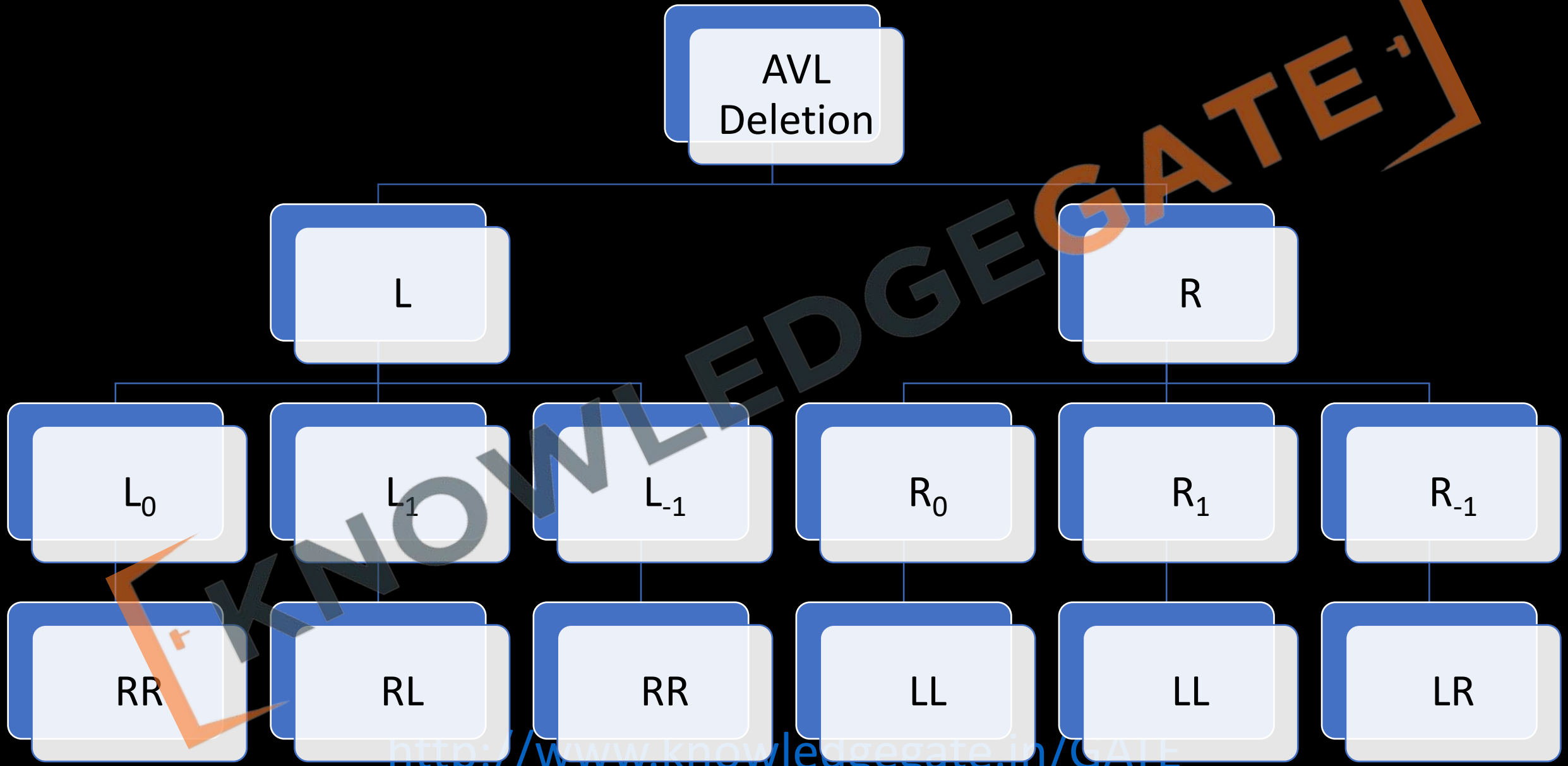
Insertion in an AVL tree

- After every insertion at most two rotations are sufficient to balance the AVL tree

Problem	Solution
LL	R
RR	L
LR	LR
RL	RL

KNOWLEDGEGATE

Deletion in an AVL tree



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Deletion in an AVL tree

Q delete the following nodes in sequence 2, 3, 10, 18, 4, 9, 14, 7, 15 ?

KNOWLEDGEGATE

Analysis of AVL tree

- Lookup, insertion, and deletion all take O(log *n*) time in both the average and worst cases, where *n* is the number of nodes in the tree prior to the operation.
- Insertions and deletions may require the tree to be rebalanced by one or more tree rotations.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Algorithm	Average	Worst case
Space	O(n)	O(n)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

Binary search tree

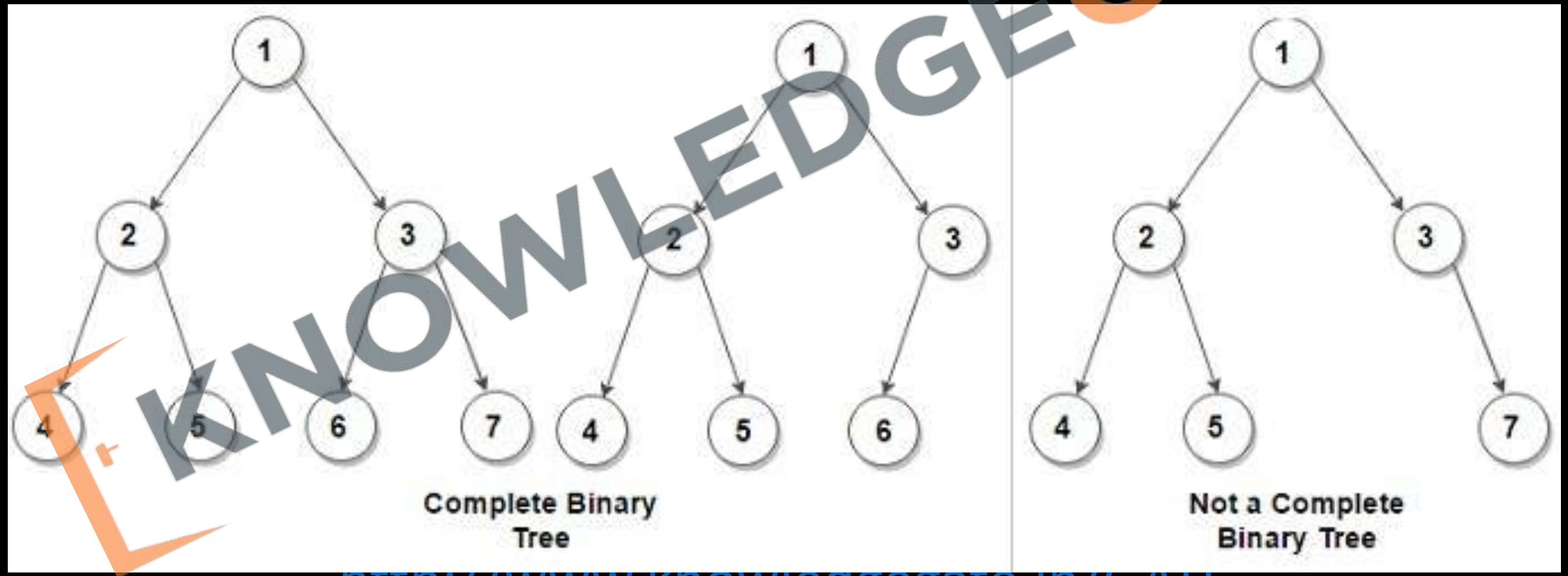
Algorithm	Average	Worst case
Space	O(n)	O(n)
Search	O(log n)	O(log n)
Insert	O(log n)	O(log n)
Delete	O(log n)	O(log n)

AVL tree

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

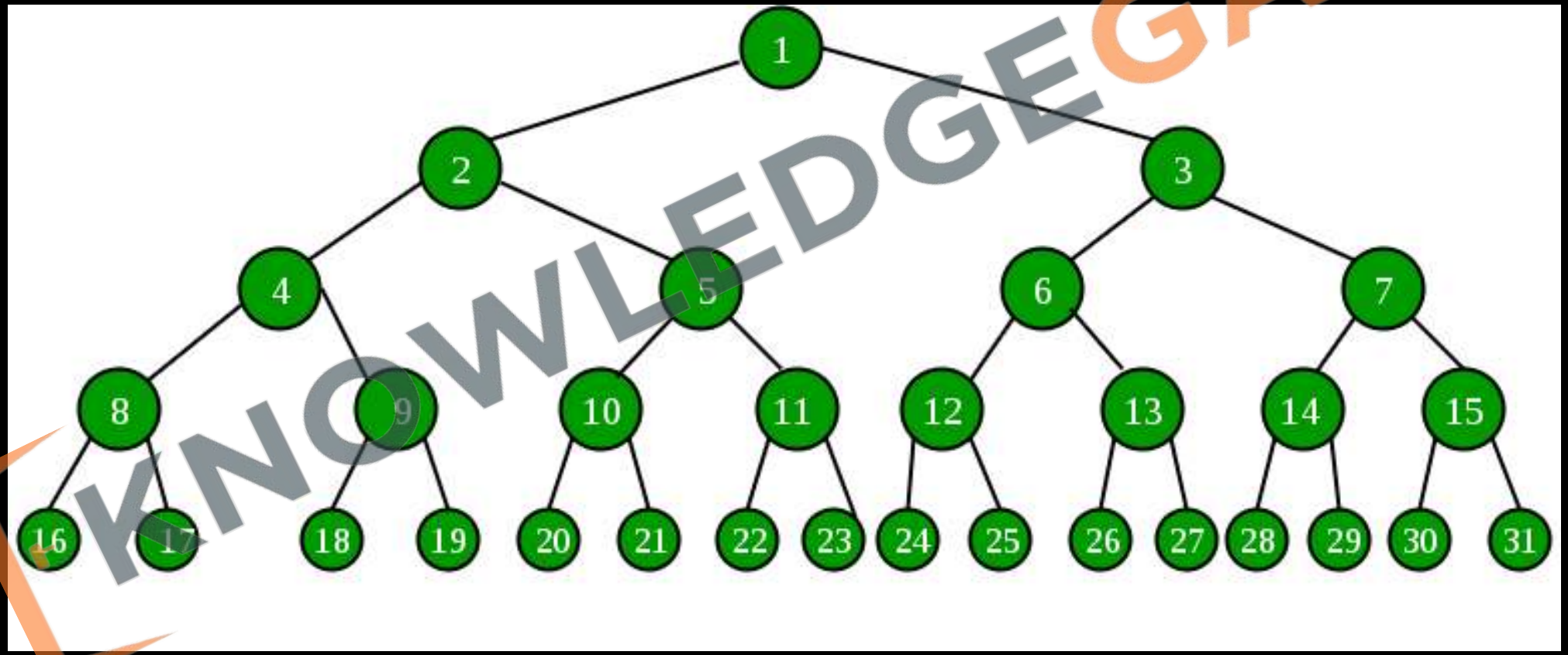
Complete Binary Tree

- Consider a binary tree T, the maximum number of nodes at height h is 2^h nodes.
- The binary tree T is said to be complete binary tree, if all its level except possibly the last, have the maximum number of nodes and if all the nodes at the last level appear as far left as possible.



KNOWLEDGEGATE

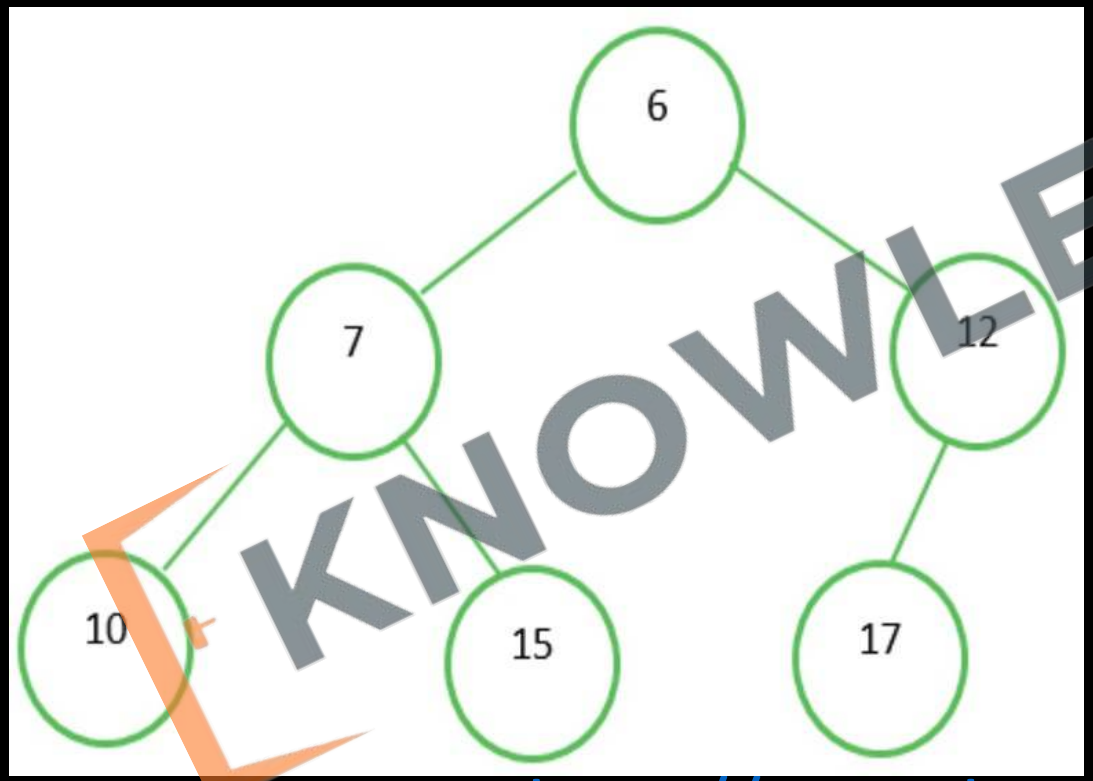
- One can easily determine the children and parent of a node k in any complete tree T
- Specially the left and right children of the node K are 2*k, 2*k + 1 and the parent of k is the node lower bound(k/2)

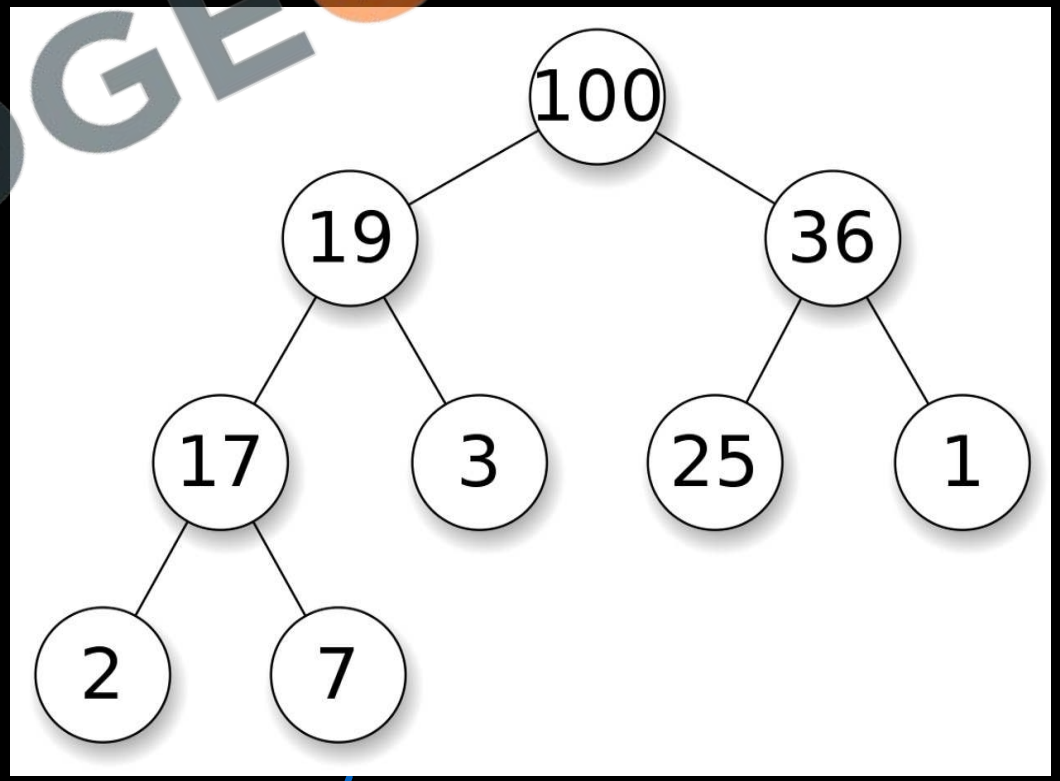


KNOWLEDGEGATE

Heap

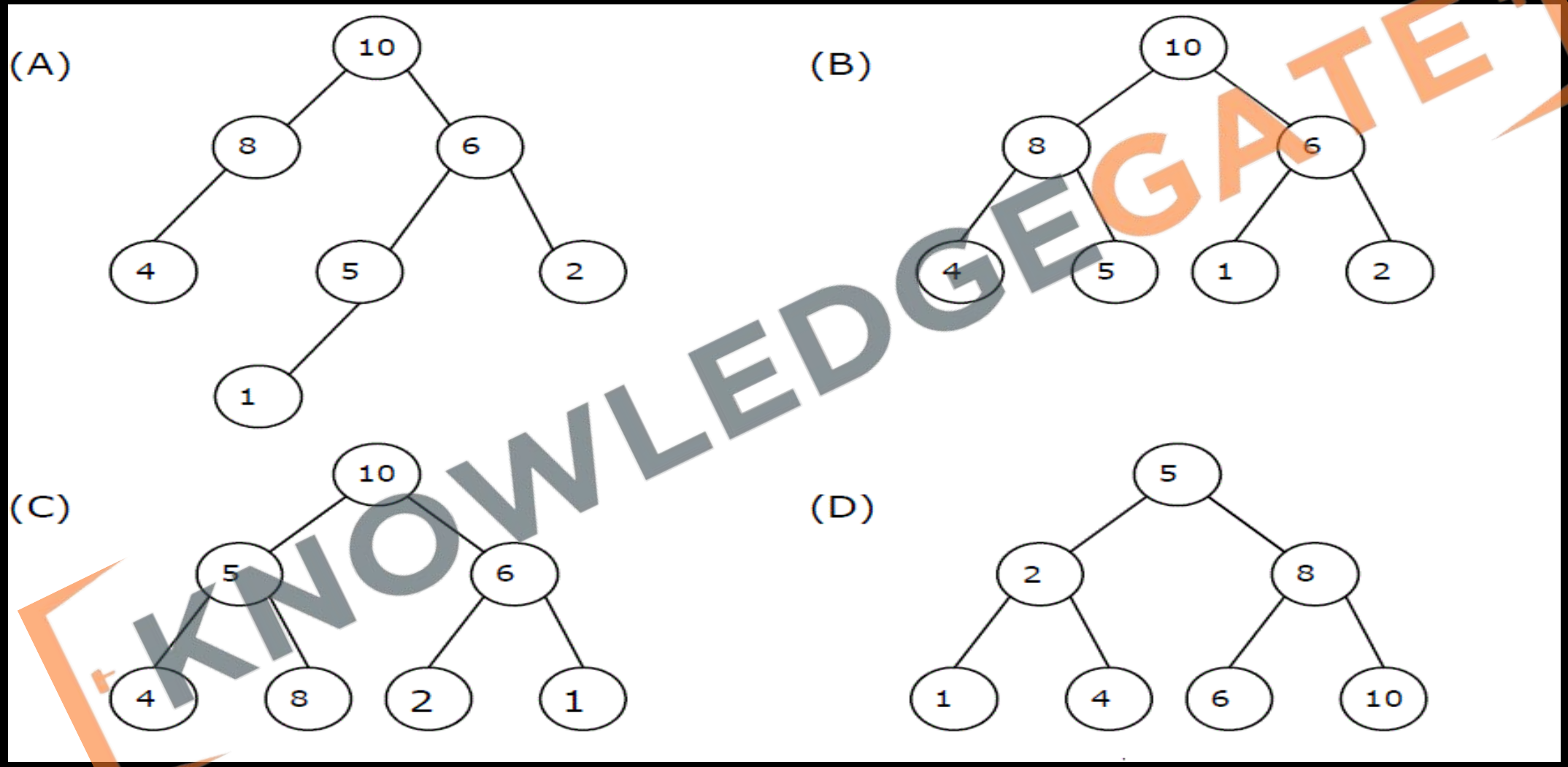
- Suppose H is a complete binary tree with n elements, H is called a Heap, if each node N of H has following properties:
  - The value of N is greater than to the value at each of the children of N then it is called Max heap.
  - A min heap is defined as the value at N is less than the value at any of the children of N.





KNOWLEDGEGATE

Q A max-heap is a heap where the value of each parent is greater than or equal to the values of its children. Which of the following is a max-heap?



KNOWLEDGEGATE

Q Consider a binary max-heap implemented using an array. Which one of the following arrays represents a binary max-heap?

(A) 23,17,14,6,13,10,1,12,7,5

(B) 23,17,14,6,13,10,1,5,7,12

(C) 23,17,14,7,13,10,1,5,6,12

(D) 23,17,14,7,13,10,1,12,5,7

KNOWLEDGEGATE

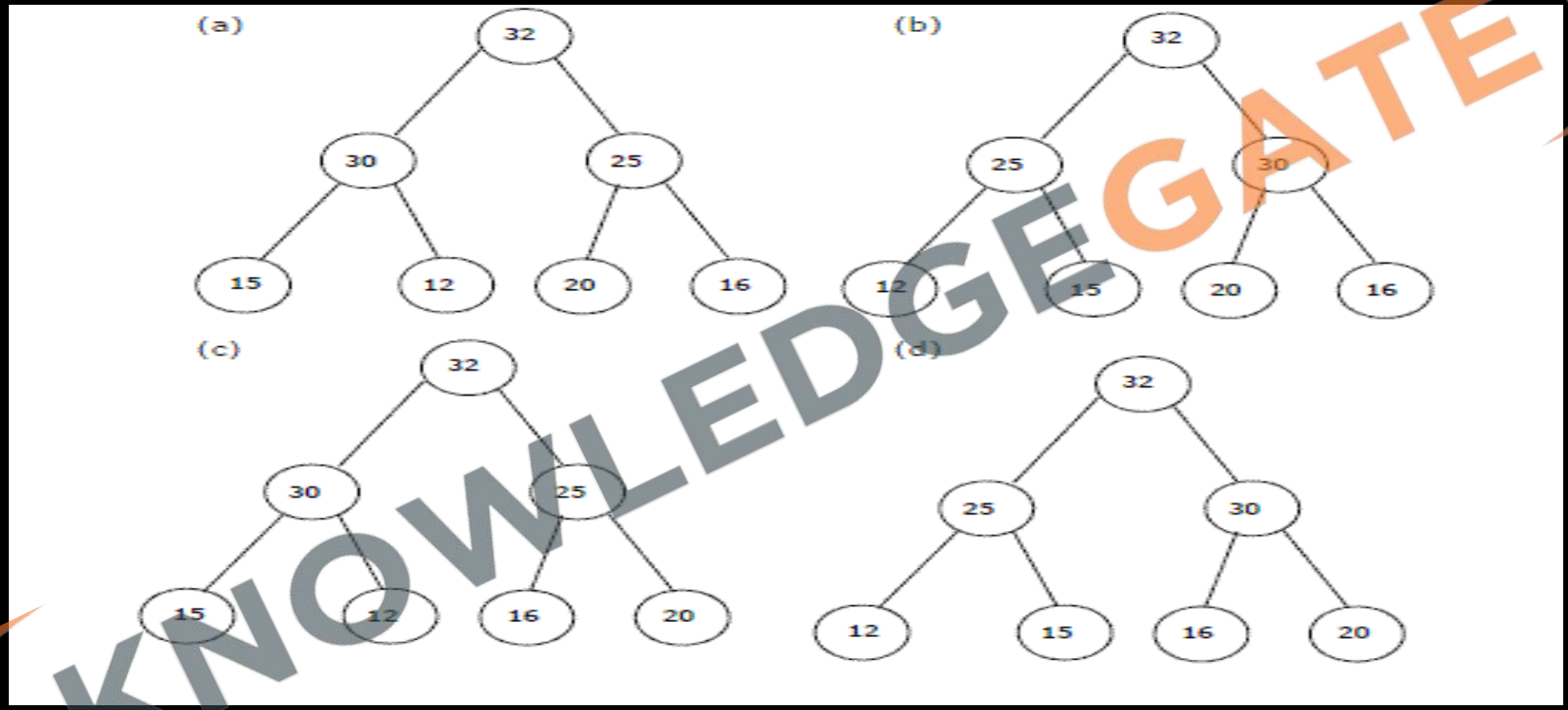
http://www.knowledgegate.in/GATE

Q The elements 32, 15, 20, 30, 12, 25, 16 are inserted one by one in the given order into a Max Heap. The resultant Max Heap is.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q The elements 32, 15, 20, 30, 12, 25, 16 are inserted one by one in the given order into a Max Heap. The resultant Max Heap is.



KNOWLEDGEGATE

Q Consider the following array of elements. ⟨89, 19, 50, 17, 12, 15, 2, 5, 7, 11, 6, 9, 100⟩. The minimum number of interchanges needed to convert it into a max-heap is

(A) 4 **(B)** 5 **(C)** 2 **(D)** 3

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q A priority queue is implemented as a Max-Heap. Initially, it has 5 elements. The level-order traversal of the heap is: 10, 8, 5, 3, 2. Two new elements 1 and 7 are inserted into the heap in that order. The level-order traversal of the heap after the insertion of the elements is:

(A) 10, 8, 7, 3, 2, 1, 5

(B) 10, 8, 7, 2, 3, 1, 5

(C) 10, 8, 7, 1, 2, 3, 5

(D) 10, 8, 7, 5, 3, 2, 1

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider a binary max-heap implemented using an array. Which one of the following arrays represents a binary max-heap?

(A) 25,12,16,13,10,8,14

(B) 25,12,16,13,10,8,14

(C) 25,14,16,13,10,8,12

(D) 25,14,12,13,10,8,16

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q What is the content of the array after two delete operations on the correct answer to the previous question?

(A) 14,13,12,10,8

(B) 14,12,13,8,10

(C) 14,13,8,12,10

(D) 14,13,12,8,10

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q We are given a set of n distinct elements and an unlabeled binary tree with n nodes. In how many ways can we populate the tree with the given set so that it becomes a binary search tree?

(A) 0 **(B)** 1 **(C)** n! **(D)** (1/(n+1)).2nCn

KNOWLEDGEGATE

Q The maximum number of binary trees that can be formed with three unlabelled nodes is:

a) 1 **b)** 5 **c)** 4 **d)** 3

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q How many distinct binary search trees can be created out of 4 distinct keys?

(A) 4 **(B)** 14 **(C)** 24 **(D)** 42

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q how many distinct BST can be constructed with 3 distinct keys?

a) 4 **b)** 5 **c)** 6 **d)** 9

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q A complete n-ary tree is one in which every node has 0 or n sons. If x is the number of internal nodes of a complete n-ary tree, the number of leaves in it is given by

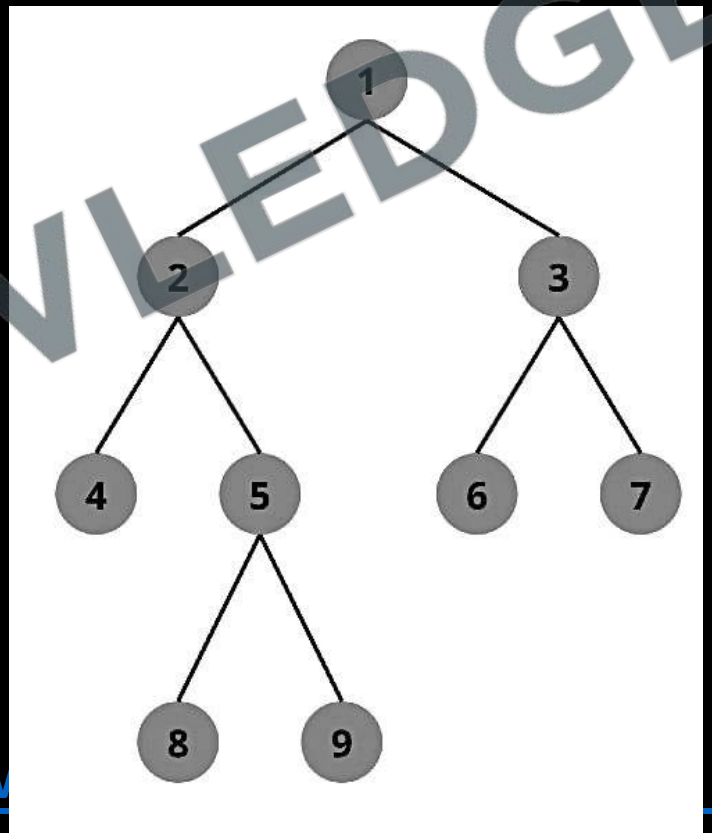
a) x(n−1)+1 **b)** xn−1 **c)** xn+1 **d)** x(n+1)

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Strictly binary tree

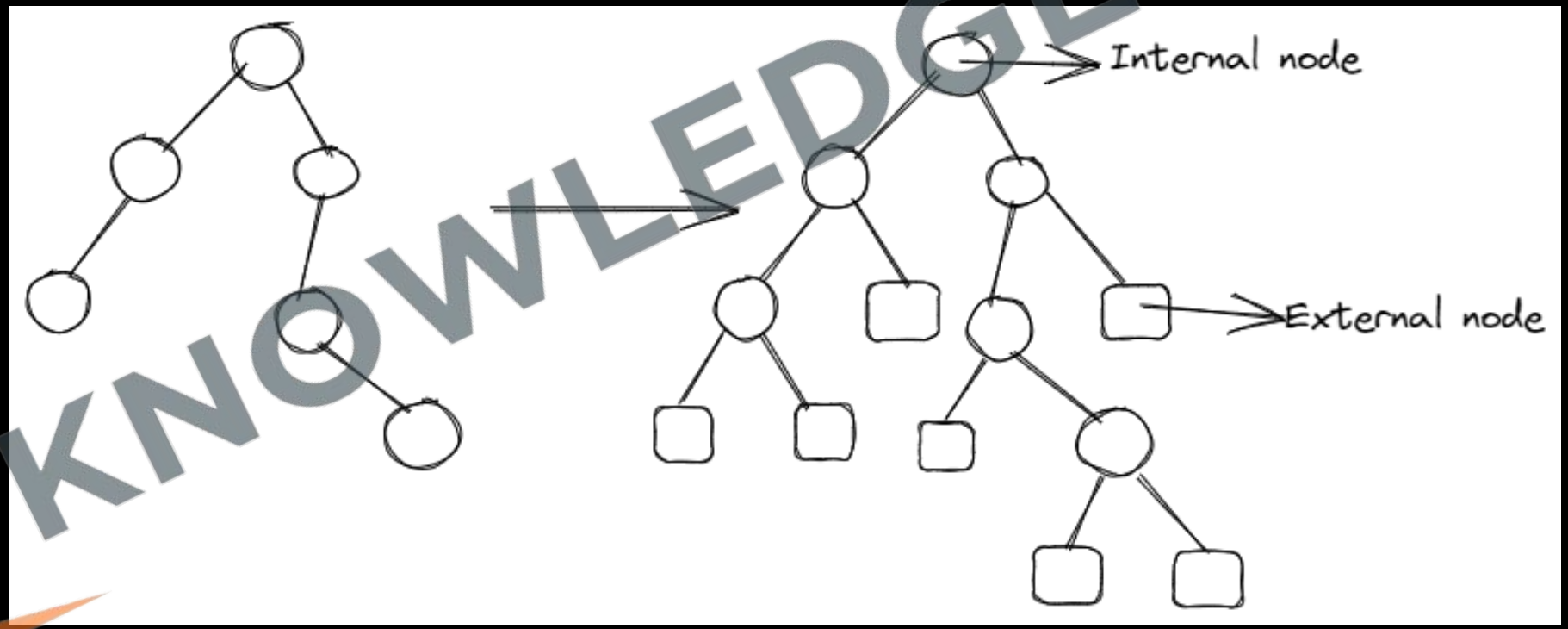
- If every non-leaf node in a binary tree has non-empty left and right subtree, the tree is termed as strictly binary tree.
- b. A strictly binary tree with n leaves always contains 2n - 1 nodes.
- c. If every non-leaf node in a binary tree has exactly two children, the tree is known as strictly binary tree.



KNOWLEDGEGATE

Extended binary tree

- A binary tree is said to be 2-tree or extended binary tree if each node has either 0 or 2 children
- Nodes with 2 children are called internal nodes and nodes with 0 children are called external nodes.



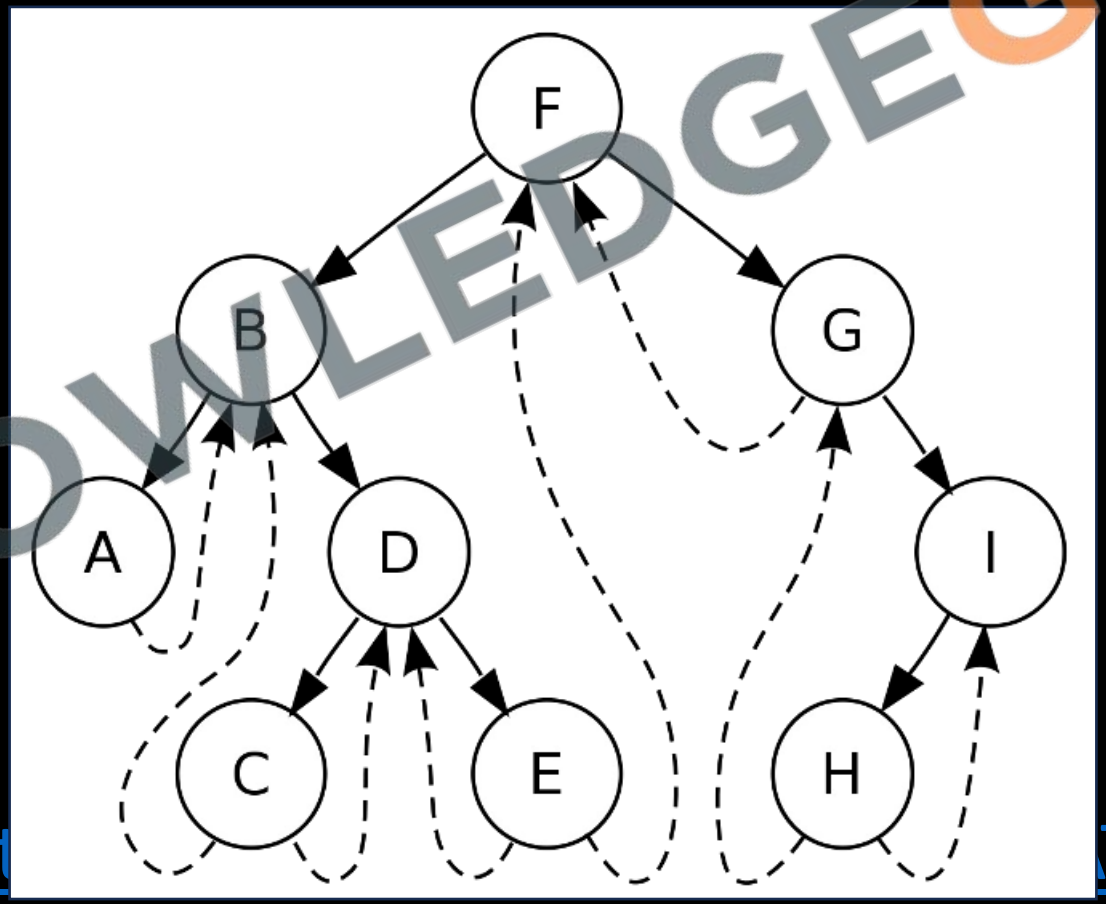
KNOWLEDGEGATE

Feature/Property	Strictly Binary Tree	Extended (Full) Binary Tree
Definition	Every non-leaf node has exactly two children.	Every node has either 0 or 2 children.
Nodes with One Child	No nodes with only one child.	No nodes with only one child.
Special Nodes	None.	May have external (dummy) nodes to make it full.
Typical Usage	Less common in practical applications.	Used in scenarios like Huffman coding trees where it's beneficial to consider the tree as full.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

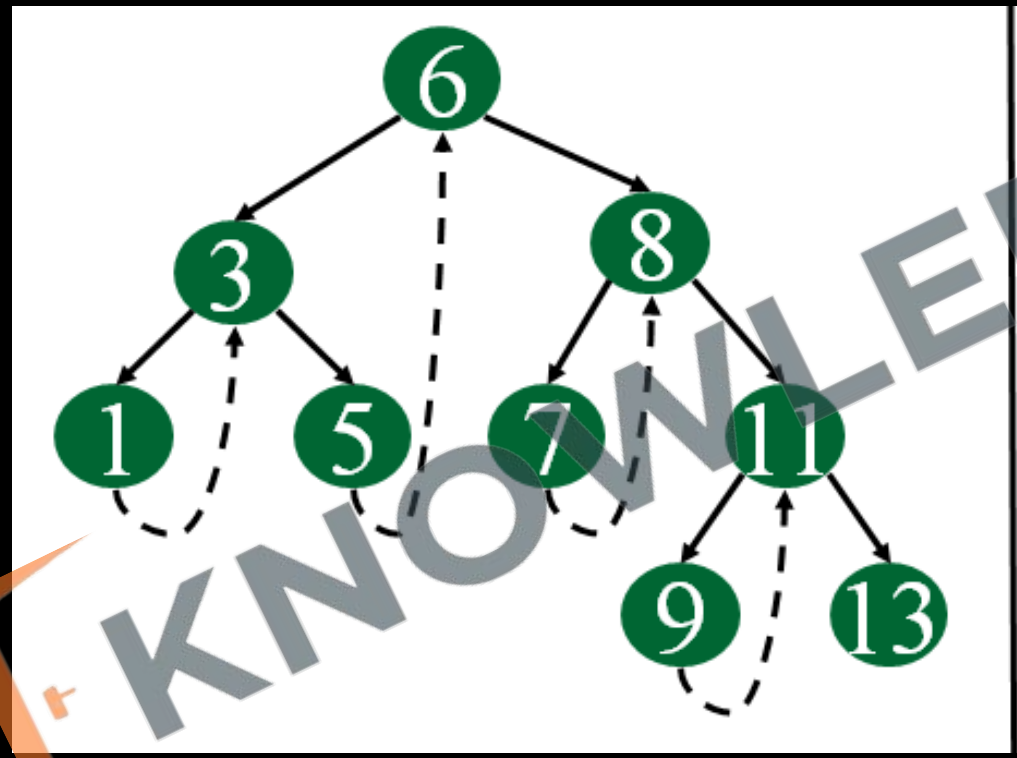
Threaded binary tree

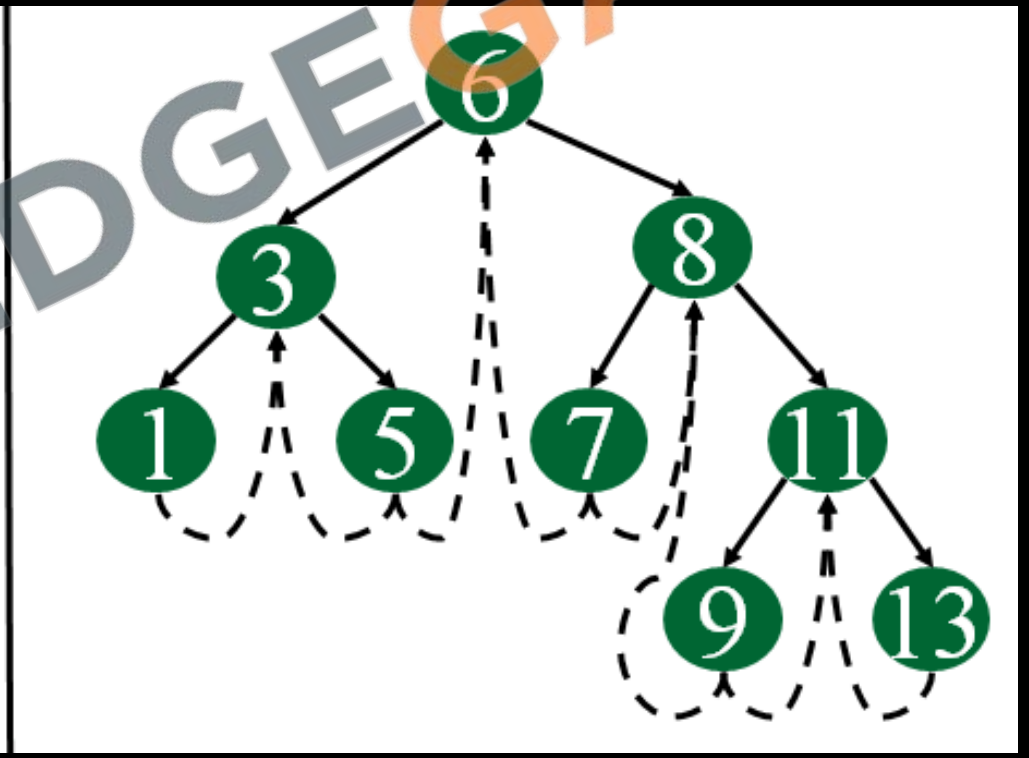
- A threaded binary tree is a modified binary tree that uses null pointers to link to the next node in an in-order sequence, optimizing in-order traversal.
- **Purpose**: Utilizes null pointers to store references (threads) to nodes, aiding efficient in-order traversal without recursion or stacks.



KNOWLEDGEGATE

- **Types**:
  - **Single Threaded**: Nodes threaded towards either in-order predecessor or successor.
  - **Double Threaded**: Nodes threaded towards both predecessor and successor.





KNOWLEDGEGATE

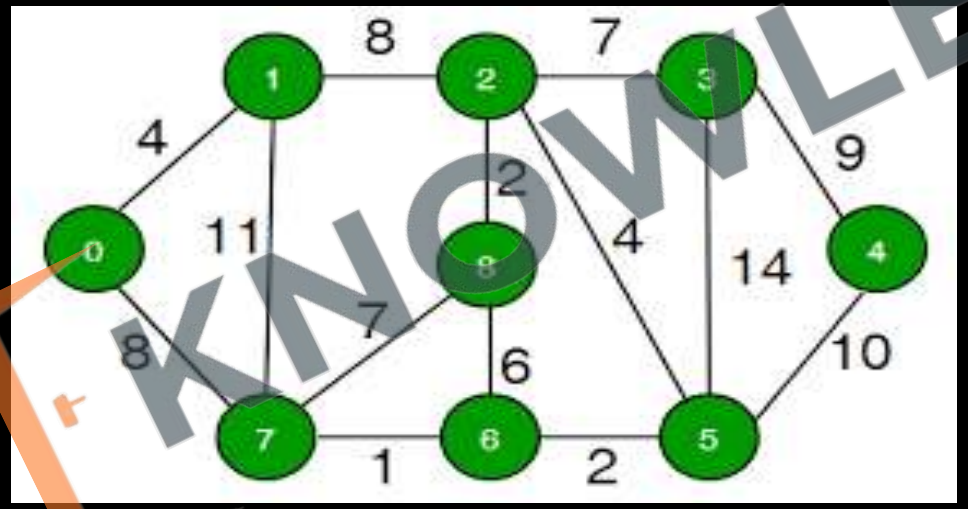
- **Benefits**:
  - Allows stack-less in-order traversal.
  - Makes efficient use of memory by replacing null pointers with threads.
- This tree variant is beneficial when recursive or stack-based traversals aren't feasible. However, its popularity has decreased with newer data structures.

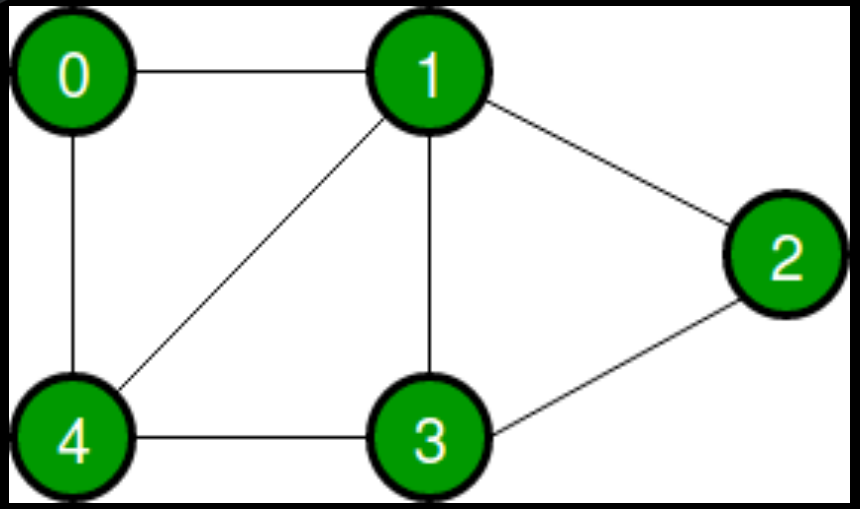
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Graph

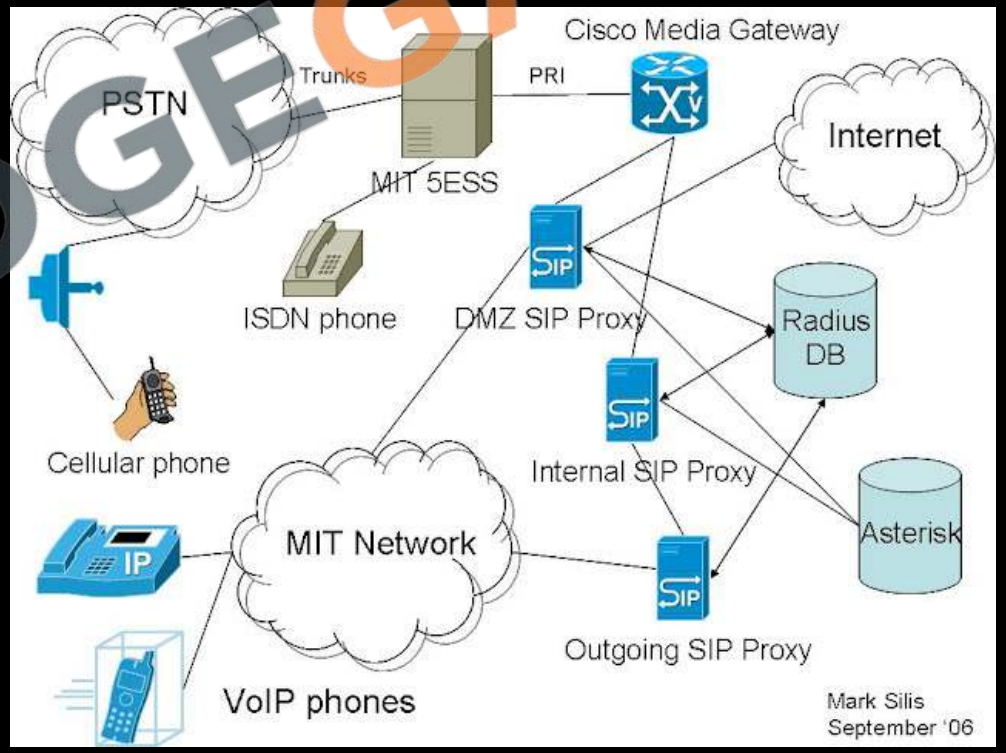
- Graph is a data structure that consists of following two components:
  - A finite set of vertices also called as nodes.
  - A finite set of ordered pair of the form (u, v) called as edge. The pair is ordered because (u, v) is not same as (v, u) in case of a directed graph(di-graph).
  - The pair of the form (u, v) indicates that there is an edge from vertex u to vertex v. The edges may contain weight/value/cost.





KNOWLEDGEGATE

- Graphs are used to represent many real-life applications: Graphs are used to represent networks. The networks may include paths in a city or telephone network or circuit network.
- Graphs are also used in social networks like LinkedIn, Facebook. For example, in Facebook, each person is represented with a vertex (or node). Each node is a structure and contains information like person id, name, gender and locale.



KNOWLEDGE GATE

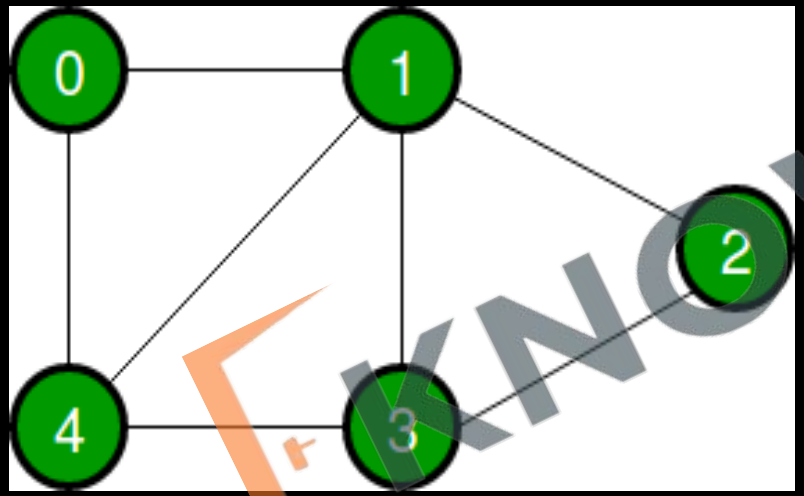
Representation of Graph in Memory

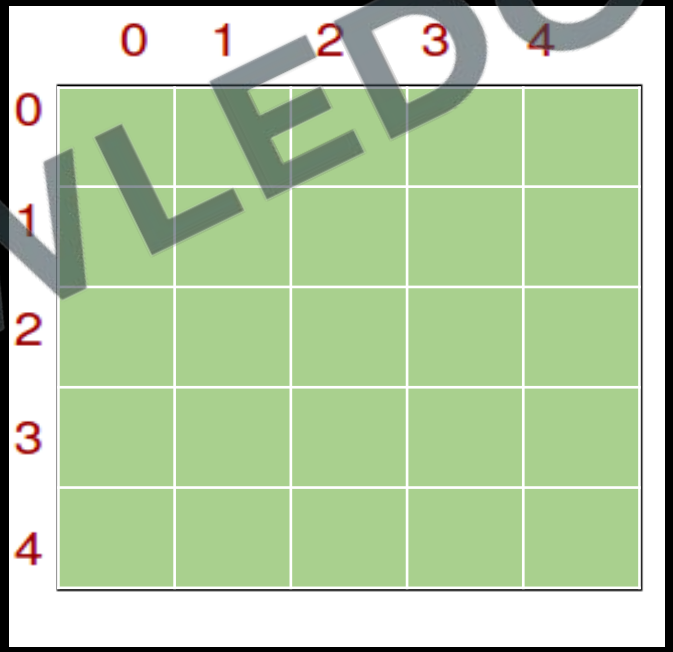
- Following two are the most commonly used representations of a graph.
  - Adjacency Matrix
  - Adjacency List

- There are other representations also like, Incidence Matrix and Incidence List. The choice of the graph representation is situation specific. It totally depends on the type of operations to be performed and ease of use.

KNOWLEDGEGATE

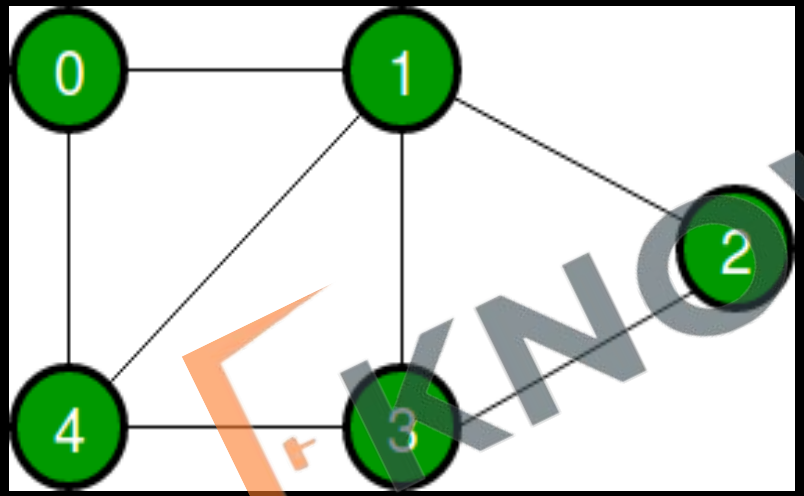
- **Adjacency Matrix:** Adjacency Matrix is a 2D array of size V x V where V is the number of vertices in a graph. Let the 2D array be adj[][], a slot adj[i][j] = 1 indicates that there is an edge from vertex i to vertex j.
- Adjacency matrix for undirected graph is always symmetric.
- Adjacency Matrix is also used to represent weighted graphs. If adj[i][j] = w, then there is an edge from vertex i to vertex j with weight w.

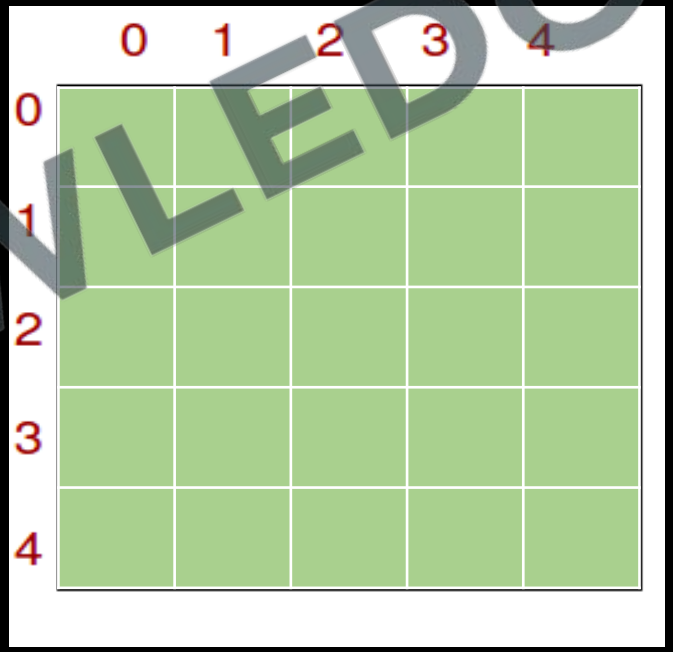




KNOWLEDGEGATE

- For directed graph





KNOWLEDGEGATE

Incidence Matrix

- **Representation of undirected graph** : Consider a undirected graph G = (V, E) which has n vertices and m edges all labelled. The incidence matrix I(G) = [bij], is then n x m matrix,
  - where bi,j=1 when edge ej is incident with vi
  - = 0 otherwise

- **Representation of directed graph** : The incidence matrix I(D) = [bij] of digraph D with n vertices and m edges is the n x m matrix in which.
  - Bi,j = 1 if arc j is directed away from vertex vi
  - =—1 if arc j is directed towards vertex vi
  - =0 otherwise.

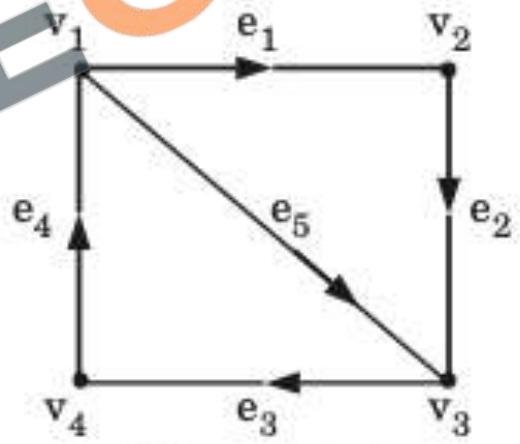


Fig. 4.3.3.

The incidence matrix of the digraph of Fig. 4.3.3 is

$$I(D) = \begin{bmatrix} 1 & 0 & 0 & -1 & 1 \\ -1 & 1 & 0 & 0 & 0 \\ 0 & -1 & 1 & 0 & -1 \\ 0 & 0 & -1 & 1 & 0 \end{bmatrix}$$

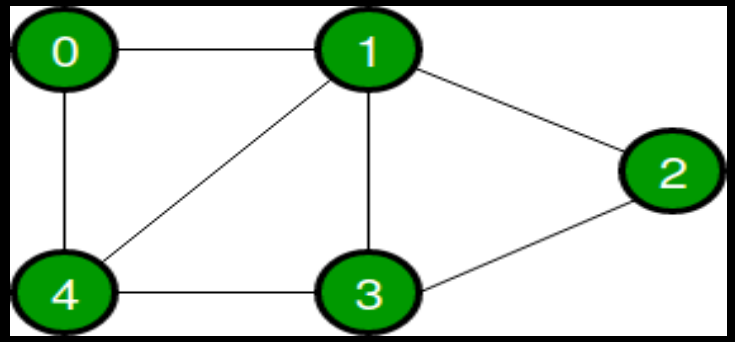
KNOWLEDGE GATE
http://www.know

- **Pros:** Representation is easier to implement and follow. Removing an edge takes O(1) time. Queries like whether there is an edge from vertex ‘u’ to vertex ‘v’ are efficient and can be done O(1).

- **Cons:** Consumes more space $O(V^2)$. Even if the graph is sparse(contains less number of edges), it consumes the same space. Adding a vertex is $O(V^2)$ time.

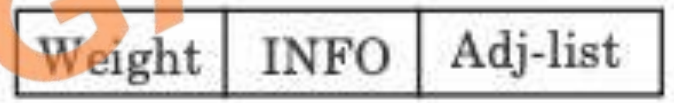
KNOWLEDGEGATE

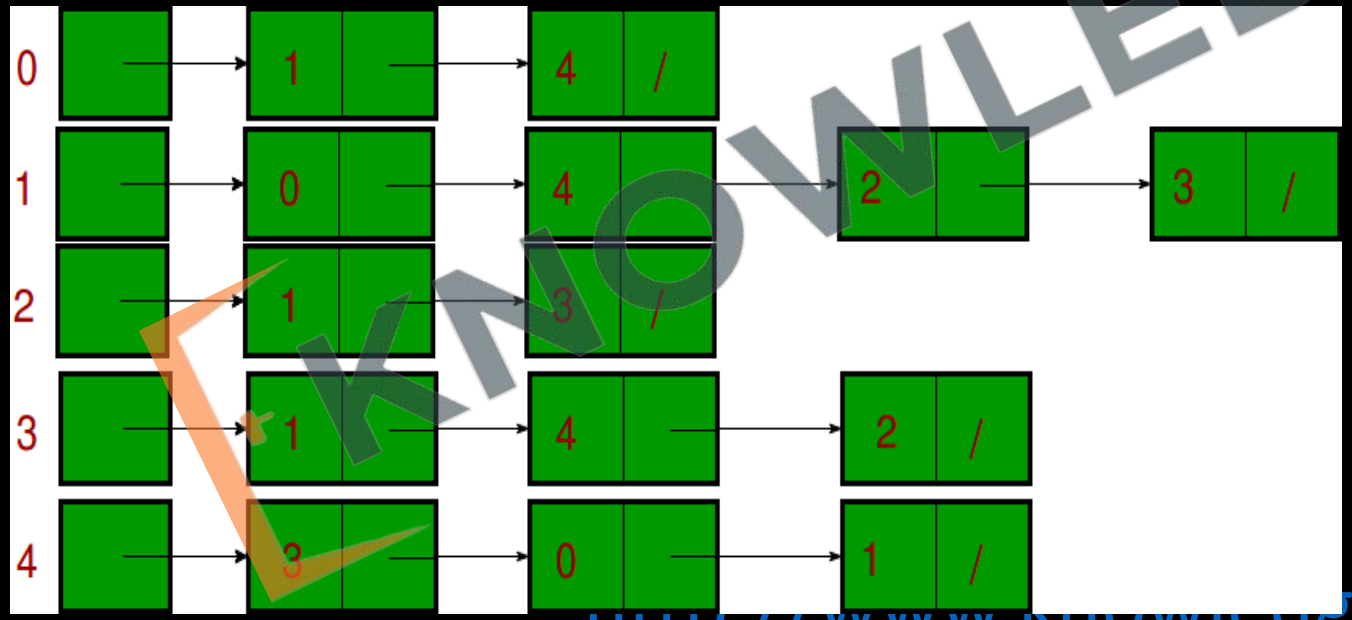
- **Adjacency List:** An array of lists is used. Size of the array is equal to the number of vertices. Let the array be array[]. An entry array[i] represents the list of vertices adjacent to the *i*th vertex. This representation can also be used to represent a weighted graph. The weights of edges can be represented as lists of pairs.



a. For non-weighted graph :

b. For weighted graph :

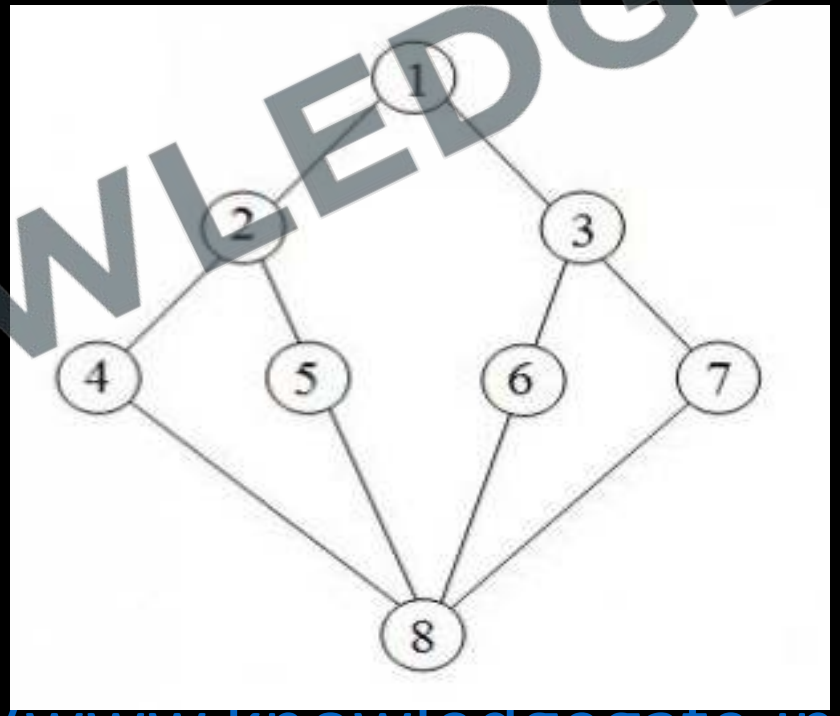




KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Graph Traversal

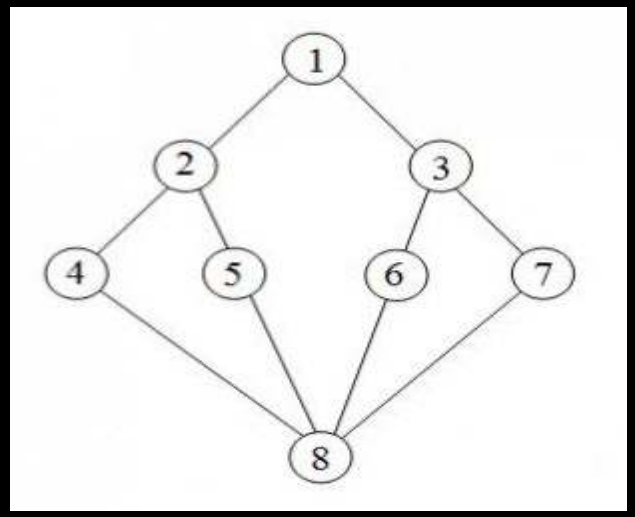
- Traversal means visiting all the nodes of a graph.
- Depth First Traversal (or Search) for a graph is similar to Depth First Traversal of a tree.
- The only catch here is, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a Boolean visited array.



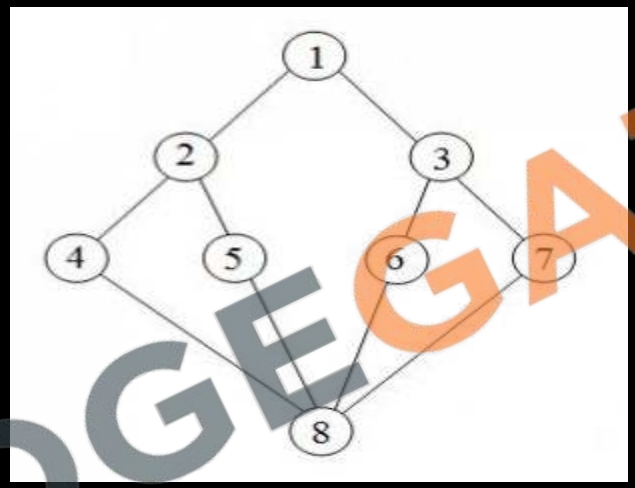
KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Q Which of the following are valid and invalid DFS traversal sequence

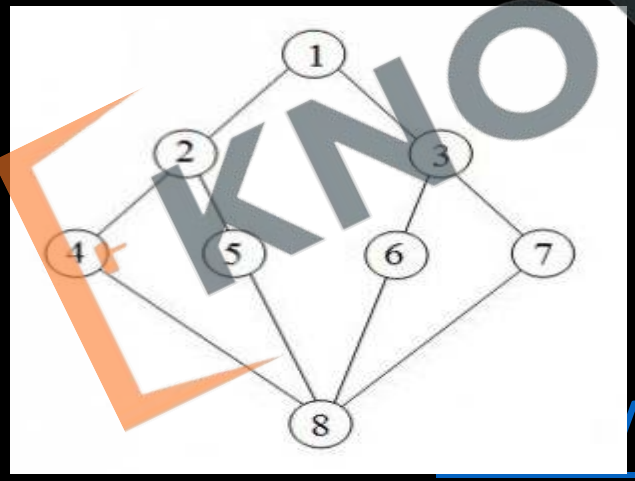
a) 1, 3, 7, 8, 5, 2, 4, 6



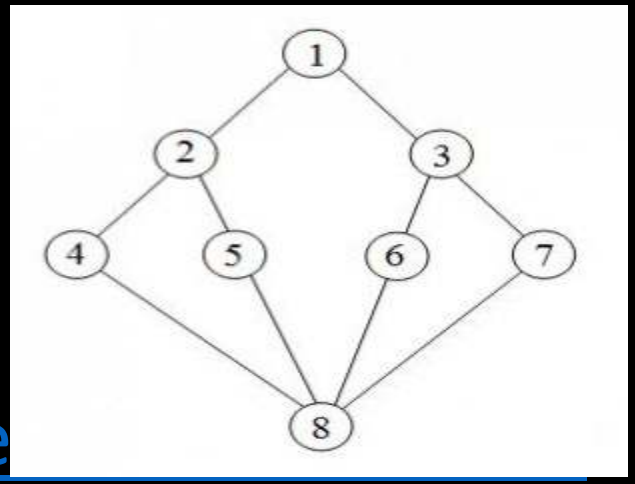
b) 1, 2, 5, 8, 6, 3, 7, 4



c) 1, 3, 6, 7, 8, 5, 2, 4



d) 1, 2, 4, 5, 8, 6, 7, 3



KNOWLEDGEGATE
//www.knowle

- A standard DFS implementation puts each vertex of the graph into one of two categories:
  - Visited
  - Not Visited

- The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

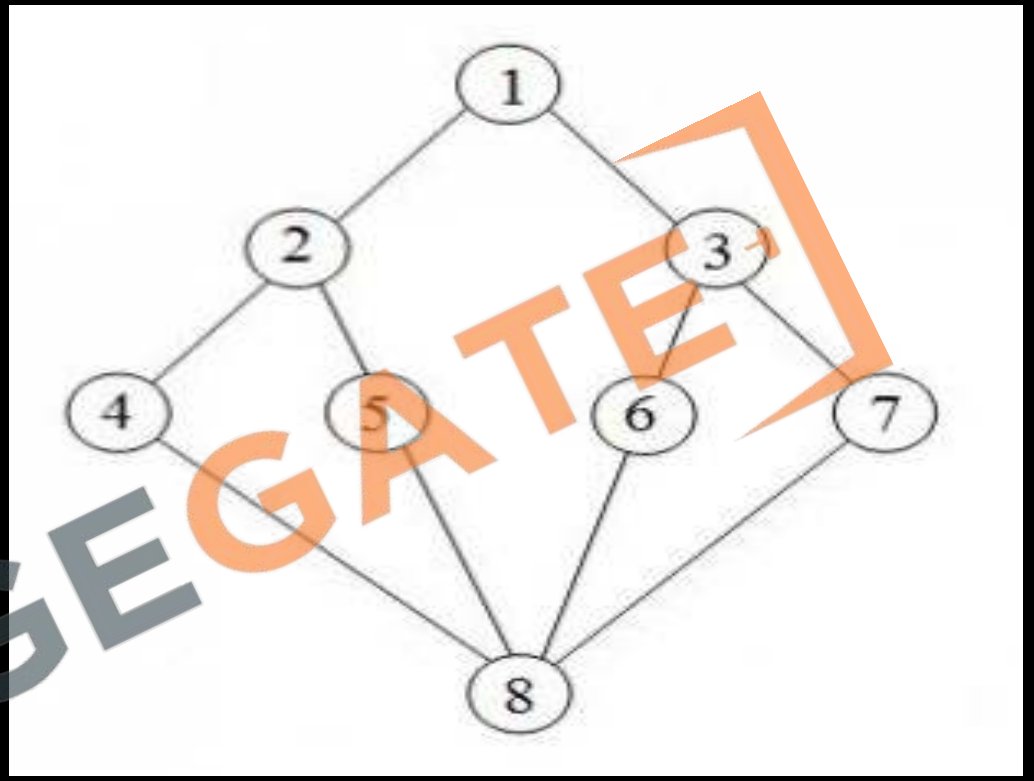
KNOWLEDGEGATE

- The DFS algorithm works as follows:
  - Start by putting any one of the graph's vertices on top of a stack.
  - Take the top item of the stack and add it to the visited list.
  - Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of stack.
  - Keep repeating steps 2 and 3 until the stack is empty.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
DFS(v)
{
    visited(v) = 1
    For all x adjacent to v
    {
        if (x is not visited)
            DFS(x)
    }
}
```



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Numerical : Adjacency list of the given graph :

$1 \rightarrow 2, 7$
$2 \rightarrow 3$
$3 \rightarrow 5, 4, 1$
$4 \rightarrow 6$
$5 \rightarrow 4$
$6 \rightarrow 2, 5, 1$
$7 \rightarrow 3, 6$

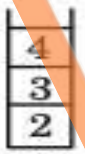
1. Initially set STATUS = 1 for all vertex
2. Push 1 onto stack and set their STATUS = 2

3. Pop 1 from stack, change its STATUS = 1 and
   Push 2, 7 onto stack and change their STATUS = 2; DFS = 1

4. Pop 7 from stack, Push 3, 6; DFS = 1, 7

5. Pop 6 from stack, Push 5; DFS = 1, 7, 6

6. Pop 5 from stack, Push 4; DFS = 1, 7, 6, 5



7. Pop 4 from stack; DFS = 1, 7, 6, 5, 4

KNOWLEDGEGATE
in/GATE

- The time and space analysis of DFS differs according to its application area. In theoretical computer science, DFS is typically used to traverse an entire graph, and takes time O(|V|+|E|), where |V|is the number of vertices and |E| the number of edges.

KNOWLEDGEGATE

```
DFS(G)
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)
```

```
DFS-VISIT(G, u)
 1  time = time + 1          // white vertex u has just been discovered
 2  u.d = time
 3  u.color = GRAY
 4  for each v ∈ G.Adj[u]    // explore edge (u, v)
 5      if v.color == WHITE
 6          v.π = u
 7          DFS-VISIT(G, v)
 8  u.color = BLACK          // blacken u; it is finished
 9  time = time + 1
10  u.f = time
```

What is the running time of DFS? The loops on lines 1–3 and lines 5–7 of DFS take time $\Theta(V)$, exclusive of the time to execute the calls to DFS-VISIT. As we did for breadth-first search, we use aggregate analysis. The procedure DFS-VISIT is called exactly once for each vertex $v \in V$, since the vertex u on which DFS-VISIT is invoked must be white and the first thing DFS-VISIT does is paint vertex u gray. During an execution of DFS-VISIT(G, v), the loop on lines 4–7 executes $|Adj[v]|$ times. Since

$$\sum_{v \in V} |Adj[v]| = \Theta(E),$$

the total cost of executing lines 4–7 of DFS-VISIT is $\Theta(E)$. The running time of DFS is therefore $\Theta(V + E)$.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
DFS-iterative (G, s)
{
    let S be stack
    Push( s )
    while ( S is not empty)
    {
        v = pop(S)
        if v is not marked as visited
        {
            mark v as visited
            for all neighbors w of v in Graph G:
            {
                if w is not marked as visited:
                    push( w )
            }
        }
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Importance of DFS : DFS is very important algorithm as based upon DFS :

- Testing whether graph is connected.
- Computing a spanning forest of *G*.
- Computing the connected components of *G*.
- Computing a path between two vertices of *G* or reporting that no such
- path exists.
- Computing a cycle in *G* or reporting that no such cycle exists.

KNOWLEDGEGATE

Application of DFS : Algorithms that use depth first search as a building block include :

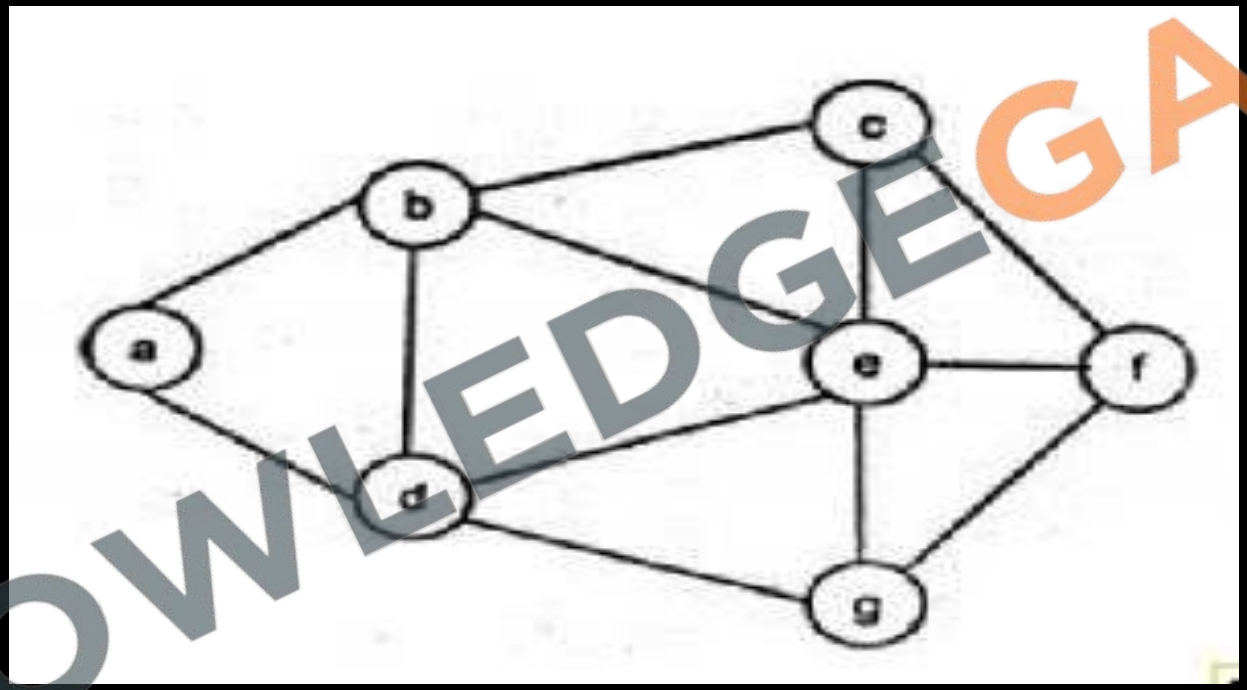
- Finding connected components.
- Topological sorting.
- Finding 2-(edge or vertex)-connected components.
- Finding 3-(edge or vertex)-connected components.
- Finding the bridges of a graph.
- Generating words in order to plot the limit set of a group.
- Finding strongly connected components.

KNOWLEDGEGATE

Q Consider the following sequence of nodes for the undirected graph given below.

1) a b e f d g c **2)** a b e f c g d **3)** a d g e b c f **4)** a d b c g e f

A Depth First Search (DFS) is started at node a. The nodes are listed in the order they are first visited. Which all of the above is (are) possible output(s)?



(A) 1 and 3 only
(B) 2 and 3 only
(C) 2, 3 and 4 only
(D) 1, 2, and 3

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Q Consider the following graph

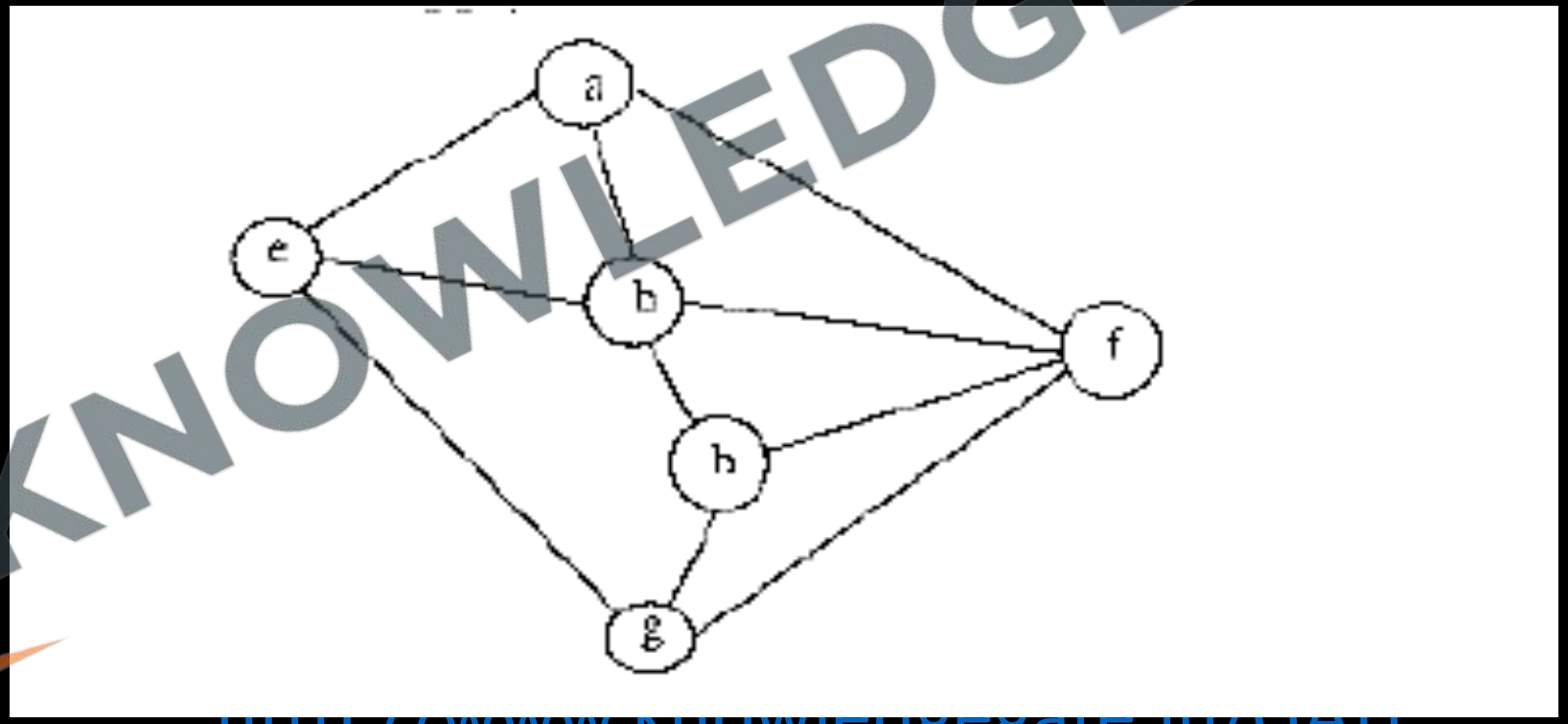
Among the following sequences

I) a b e g h f **II)** a b f e h g **III)** a b f h g e **IV)** a f g h b e

Which are depth first traversals of the above graph?

(A) I, II and IV only **(B)** I and IV only

(C) II, III and IV only **(D)** I, III and IV only



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Breadth First Traversal (or Search)

- Breadth First Traversal (or Search) for a graph is similar to Breadth First Traversal of a tree. The only catch here is, unlike trees, graphs may contain cycles, so we may come to the same node again.
- To avoid processing a node more than once, we use a Boolean visited array. For simplicity, it is assumed that all vertices are reachable from the starting vertex, i.e. the graph is connected

KNOWLEDGEGATE

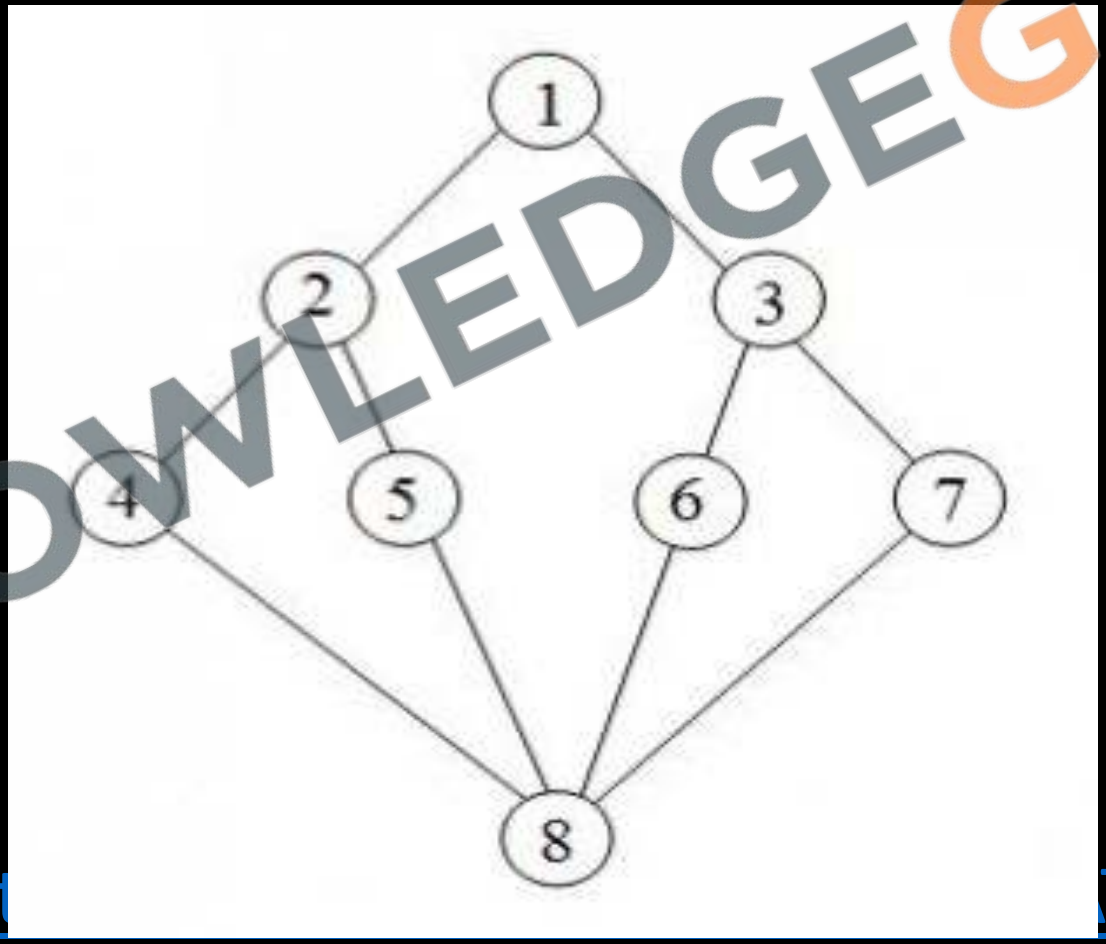
Q Which of the following are valid and invalid BFS traversal sequence

a) 1, 3, 2, 5, 4, 7, 6, 8

b) 1, 3, 2, 7, 6, 4, 5, 8

c) 1, 2, 3, 5, 4, 7, 6, 8

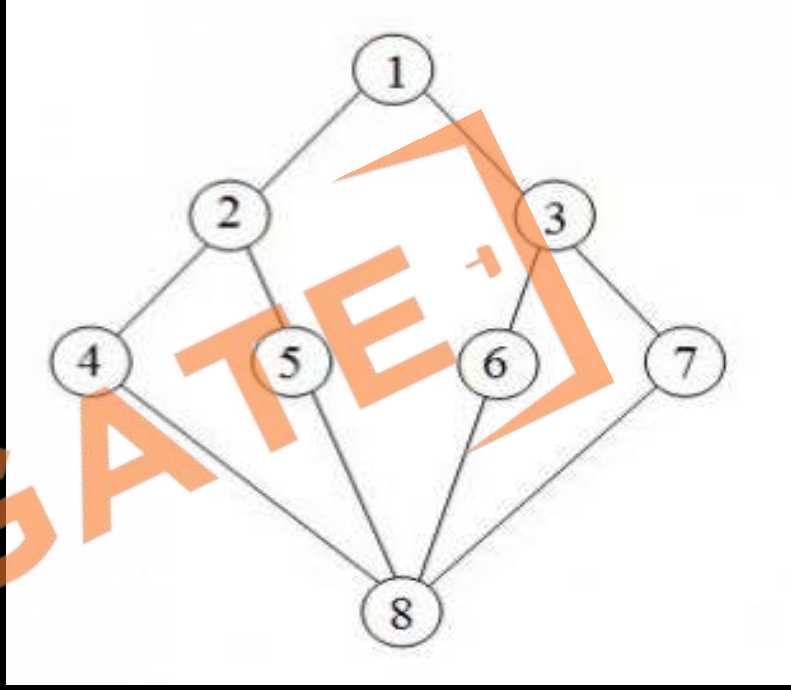
d) 1, 2, 3, 7, 5, 6, 4, 8



KNOWLEDGEGATE

htt ATE

```
BFS(v)
{
    visited(v) = 1
    insert[V,Q]
    While(Q != Phi)
    {
        u = Delete(Q);
        for all x adjacent to u
        {
            if (x is not visited)
                {
                    visited(x) = 1
                    insert(x,Q)
                }
        }
    }
}
```



KNOWLEDGEGATE

- The time complexity can be expressed as O(|V|+|E|), since every vertex and every edge will be explored in the worst case. |V| is the number of vertices and |E| is the number of edges in the graph. Note that O(|E|)may vary between O(1) and O($|V|^2$).

KNOWLEDGEGATE

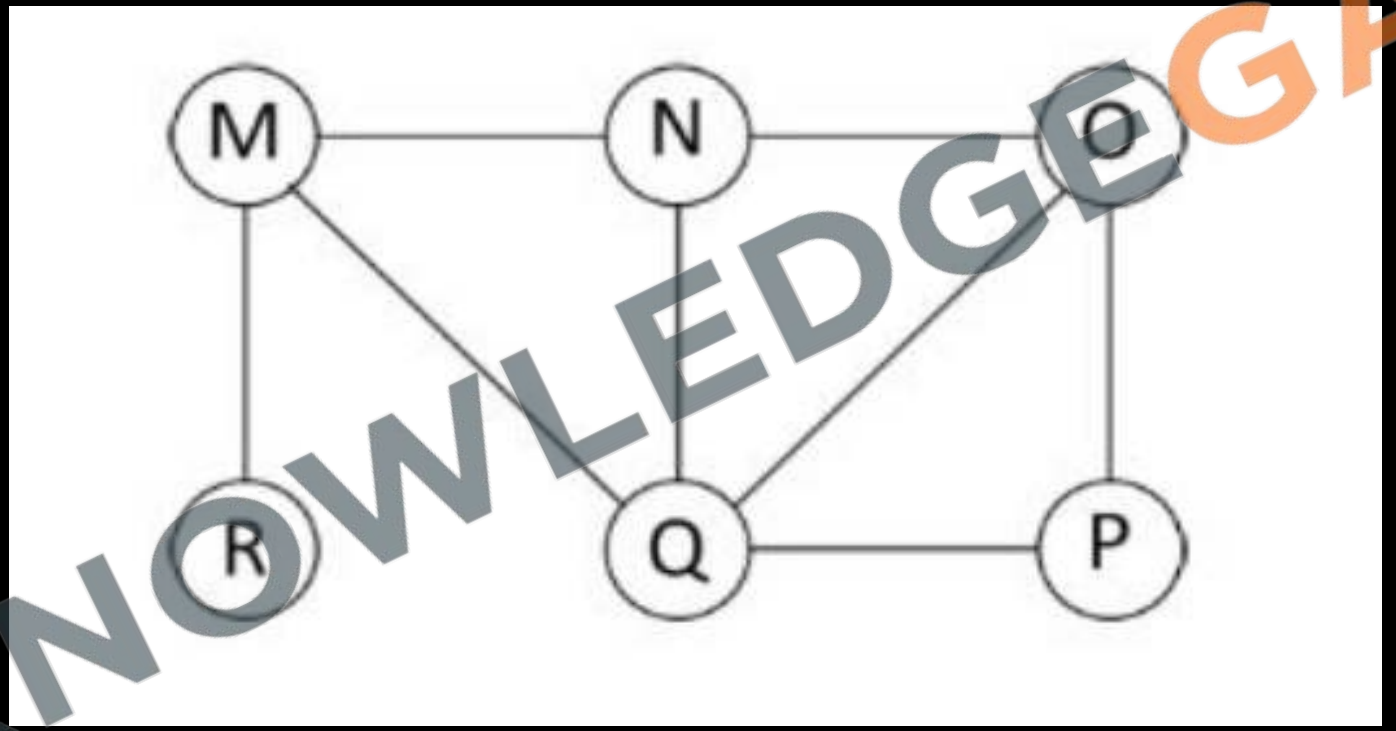
```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2      u.color = WHITE
 3      u.d = ∞
 4      u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11      u = DEQUEUE(Q)
12      for each v ∈ G.Adj[u]
13          if v.color == WHITE
14              v.color = GRAY
15              v.d = u.d + 1
16              v.π = u
17              ENQUEUE(Q, v)
18      u.color = BLACK
```

Before proving the various properties of breadth-first search, we take on the somewhat easier job of analyzing its running time on an input graph $G = (V, E)$. We use aggregate analysis, as we saw in Section 17.1. After initialization, breadth-first search never whitens a vertex, and thus the test in line 13 ensures that each vertex is enqueued at most once, and hence dequeued at most once. The operations of enqueuing and dequeuing take $O(1)$ time, and so the total time devoted to queue operations is $O(V)$. Because the procedure scans the adjacency list of each vertex only when the vertex is dequeued, it scans each adjacency list at most once. Since the sum of the lengths of all the adjacency lists is $\Theta(E)$, the total time spent in scanning adjacency lists is $O(E)$. The overhead for initialization is $O(V)$, and thus the total running time of the BFS procedure is $O(V + E)$. Thus, breadth-first search runs in time linear in the size of the adjacency-list representation of G.

KNOWLEDGE GATE

Q Breath First Search (BFS) has been implemented using queue data structure. Which one of the following is a possible order of visiting the nodes in the graph above?

a) MNOPQR **b)** NQMPOR **c)** QMNROP **d)** POQNMR

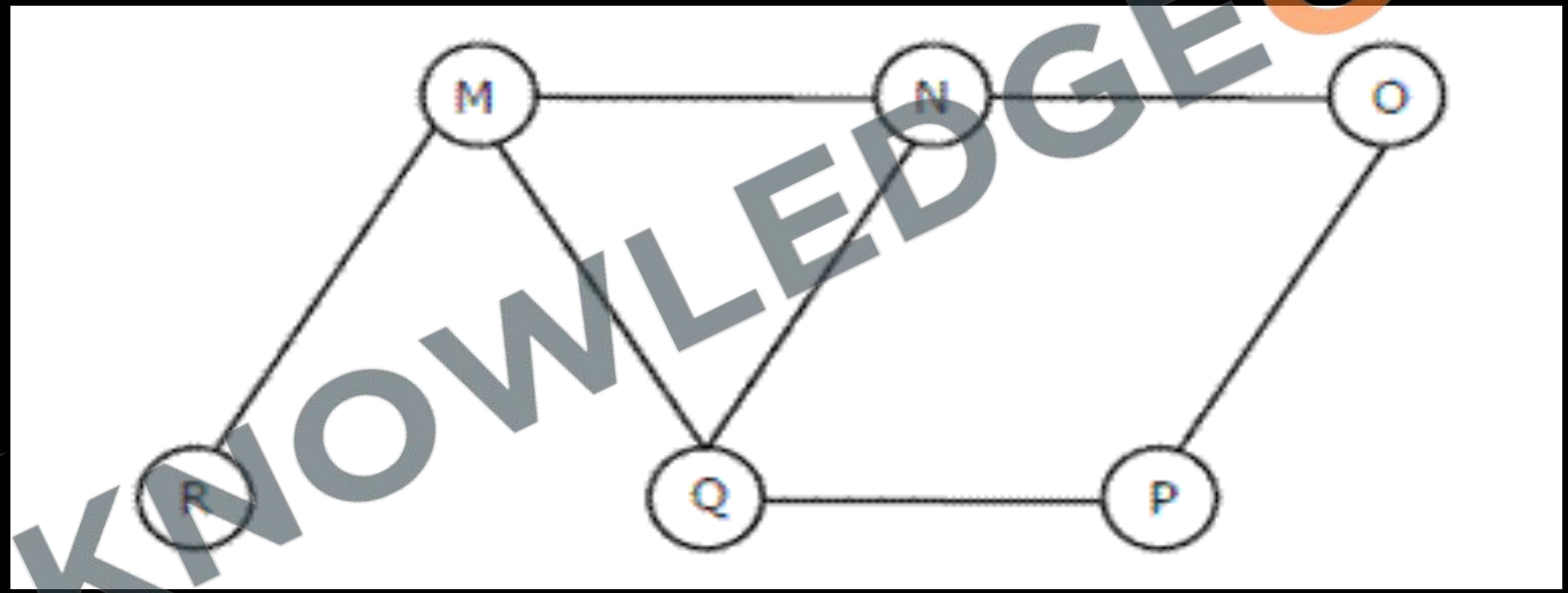


KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

Q The Breadth First Search algorithm has been implemented using the queue data structure. One possible order of visiting the nodes of the following graph is

(A) MNOPQR **(B)** NQMPOR **(C)** QMNPRO **(D)** QMNPOR



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Importance of BFS :

- It is one of the single source shortest path algorithms, so it is used to
- compute the shortest path.
- It is also used to solve puzzles such as the Rubik's Cube.
- BFS is not only the quickest way of solving the Rubik's Cube, but also
- the most optimal way of solving it.

KNOWLEDGEGATE

Application of BFS : Breadth first search can be used to solve many problems in graph theory, for example

- Copying garbage collection.
- Finding the shortest path between two nodes *u* and *v*, with path length measured by number of edges (an advantage over depth first search).
- Ford-Fulkerson method for computing the maximum flow in a flow network.
- Serialization/Deserialization of a binary tree vs serialization in sorted order, allows the tree to be re-constructed in an efficient manner.
- Construction of the failure function of the Aho-Corasick pattern matcher.
- Testing bipartiteness of a graph.

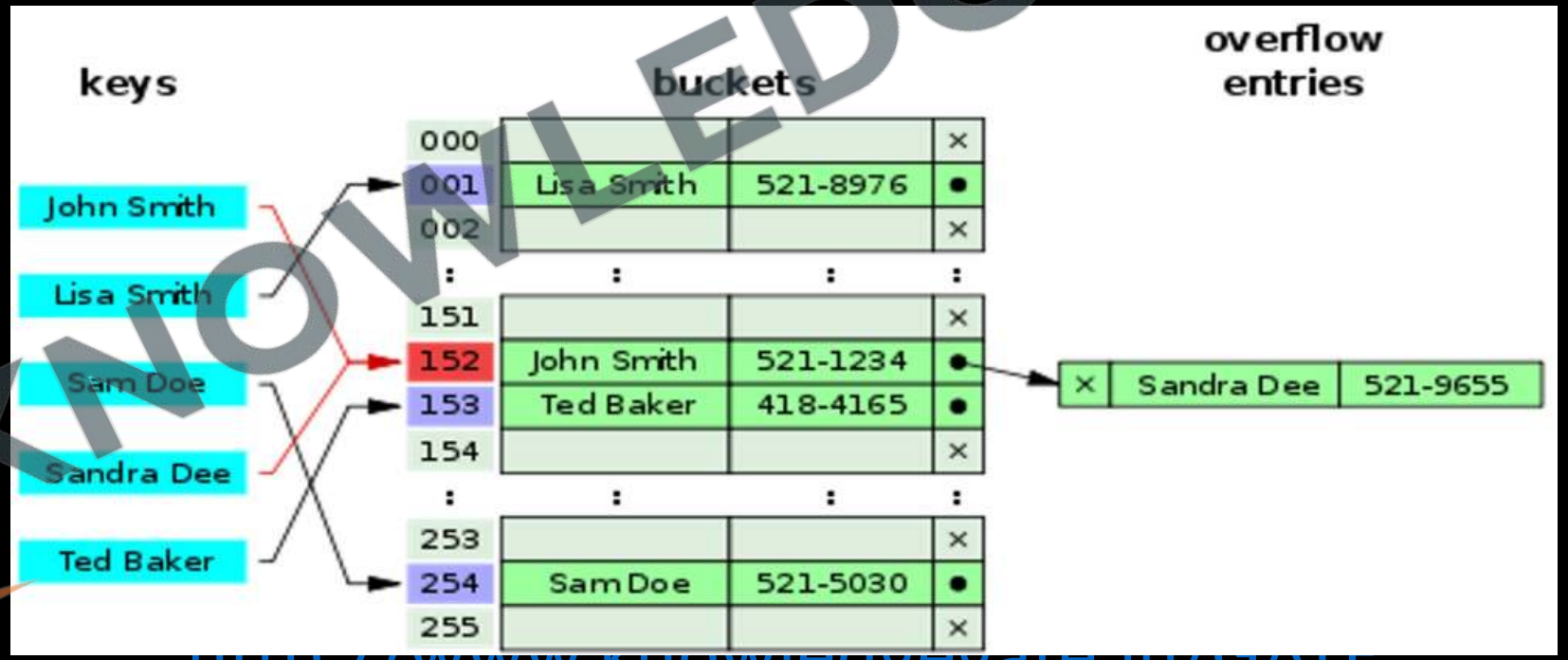
KNOWLEDGEGATE

Introduction to hashing

- Main idea of data structure is to help us store the data. But Most common operation on any data structure is not insert or delete but actually search, as even for insertion and deletion search is also required.
- In any of the data structure the search time first depends on the number of elements which data structure contains and then on type of structure. for e.g.
  - Unsorted array – O(n)
  - sorted array – O(logn)
  - link list – O(n)
  - BT – O(n)
  - BST – O(n)
  - AVL – O(logn)

KNOWLEDGEGATE

- So hashing is a technique where search time is independent of the number of items in which we are searching a data value.
- The basic idea is to use the key itself to find the address in the memory to make searching easy. For e.g. *to use phone number, roll no, Aadhar card, voter id or any other key and convert it into a smaller* practical *number (*but it must be modified so a great deal of space is not wasted*) and uses the small number as index in a table called hash table.*
- The values are then stored in **hash table**, By using that key you can access the element in **O(1)** time.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- This conversion called hash function which is from the set of K keys into the set of memory location L.
  - H: K→L
- In simple terms, a hash function maps a big number or string to a small integer that can be used as index in hash table. An array that stores pointers to records corresponding to our search key. The remaining entries can be nil.

KNOWLEDGEGATE

Q Given the following input (4322, 1334, 1471, 9679, 1989, 6171, 6173, 4199) and the hash function x mod 10, which of the following statements are true?

i. 9679, 1989, 4199 hash to the same value
ii. 1471, 6171 has to the same value
iii. All elements hash to the same value
iv. Each element hashes to a different value

(A) i only
(B) ii only
(C) i and ii only
(D) iii or iv

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Collision**: - It is possible that two different set of keys K_1 and K_2 will yield the same hash address. This situation is called collision. The technique to resolve collision is called collision resolution.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- Characteristics of good hash function
  - Easy to compute and understand
  - Efficiently computable- It must take less time to compute
  - Should uniformly distribute the keys (Each table position equally likely for each key) and should not result in clustering.
  - Must have low collision rate

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Most popular hash function

- **Division-remainder method**: The size of the number of items in the table is estimated. That number is then used as a divisor into each original value or key to extract a quotient and a remainder.
- The remainder is the hashed value. (Since this method is liable to produce a number of collisions, any search mechanism would have to be able to recognize a collision and offer an alternate search mechanism.)
  - $H(K) = K(mod\ m)$
  - $H(K) = K(mod\ m) + 1$

KNOWLEDGEGATE

- **Note**: Irrespective of how good a hash function is, collisions are bound to occur. Therefore, to maintain the performance of a hash table, it is important to manage collisions through various collision resolution techniques.

KNOWLEDGEGATE

Q Which of the following statement(s) is TRUE?

I. A hash function takes a message of arbitrary length and generates a fixed length code.

II. A hash function takes a message of fixed length and generates a code of variable length.

III. A hash function may give the same hash value for distinct messages.

(a) I only **(b)** II and III only **(c)** I and III only **(d)** I only

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Mid-Square Method

- The mid-square method is a technique used to generate hash codes by squaring the key and then extracting a portion of the resulting number. This method was popular for hash function design in early hashing techniques but has been superseded by more robust methods in modern systems.
- **Square the Key**: Take the key, square it (e.g., key 123 gives 15129).
- **Middle Extraction**: Extract middle digits from the squared result (e.g., from 15129, take 512).
- **Fit to Table**: Optionally, use modulus to fit the hash within table size.

KNOWLEDGEGATE

- **Pros**:
  - Simple to use.
  - Effective for random, uniform keys.

- **Cons**:
  - Potential for collisions.
  - Hash quality varies with key distribution.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Folding Method

- The folding method is a technique used in hashing to partition the key into several parts, then combine these parts to determine the hash code.
- Here's how the folding method works:
  - **Partition the Key**: Divide the key into equal-sized parts. For example, for a key 123456789 and partition size of 3, you'd have 123, 456, and 789.
  - **Add the Partitions**: Sum these parts together. Continuing the example, 123 + 456 + 789 = 1368.
  - **Modulus Operation**: If the resulting sum is larger than the hash table size, a modulus operation will bring it within range. For instance, if the hash table has 1000 slots, 1368 % 1000 = 368 would be the final hash code.

KNOWLEDGEGATE

- **Advantages**:
  - It distributes keys that are close in value across the hash table.
  - Simple and intuitive.

- **Disadvantages**:
  - Not as efficient for keys with certain patterns.
  - Might still lead to collisions if the table size isn't chosen wisely.
  - Like other simple hashing techniques, the folding method's usage has been largely superseded by more advanced hash functions in modern systems. However, it remains a basic technique useful for understanding foundational hashing concepts.

KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

Collision Resolution Technique

- **Open Addressing/closed hashing** - In Open Addressing, all elements are stored in the hash table itself. i.e. collision is resolved by **probing** or searching through alternate locations in the Hash table itself in a particular *sequence.*
- When searching for an element, we one by one examine table slots until the desired element is found or it is clear that the element is not in the table. So, at any point, size of table must be greater than or equal to total number of keys.
- It is of three types linear probing, quadratic probing, double hashing

KNOWLEDGE GATE

Q Consider a hash table of size seven, with starting index zero, and a hash function (7x+3) mod 4. Assuming the hash table is initially empty, which of the following is the contents of the table when the sequence 1, 3, 8, 10 is inserted into the table using closed hashing? Here "__" denotes an empty location in the table.

(a) 3, 10, 1, 8, __ , __ , __

(b) 1, 3, 8, 10, __ , __ , __

(c) 1, __ , 3, __ , 8, __ , 10

(d) 3, 10, __ , __ , 8, __ , _

KNOWLEDGEGATE

Linear probing

- Linear probing is a method used in open addressing hashing. When a collision occurs, it searches the table sequentially from the hashed position to find an empty or matching slot.

- **Key Points**:
  - Uses a random hash function, ensuring constant expected time for operations.
  - Achieves O(1) time for insert, remove, and search if the load factor is kept below one.

- **Insert(k)**: Probe until an empty slot is found, then insert k.
- **Search(k)**: Probe until a matching key or empty slot is found.
- **Delete(k)**: Mark slots of deleted keys as “deleted”. Inserts can use these slots, but searches don’t stop at them.

KNOWLEDGEGATE

Linear Probing

- In linear probing method, in case of a collision we find out the next free space and store the key that is causing collision in it.
- The method of *linear probing* uses the hash function

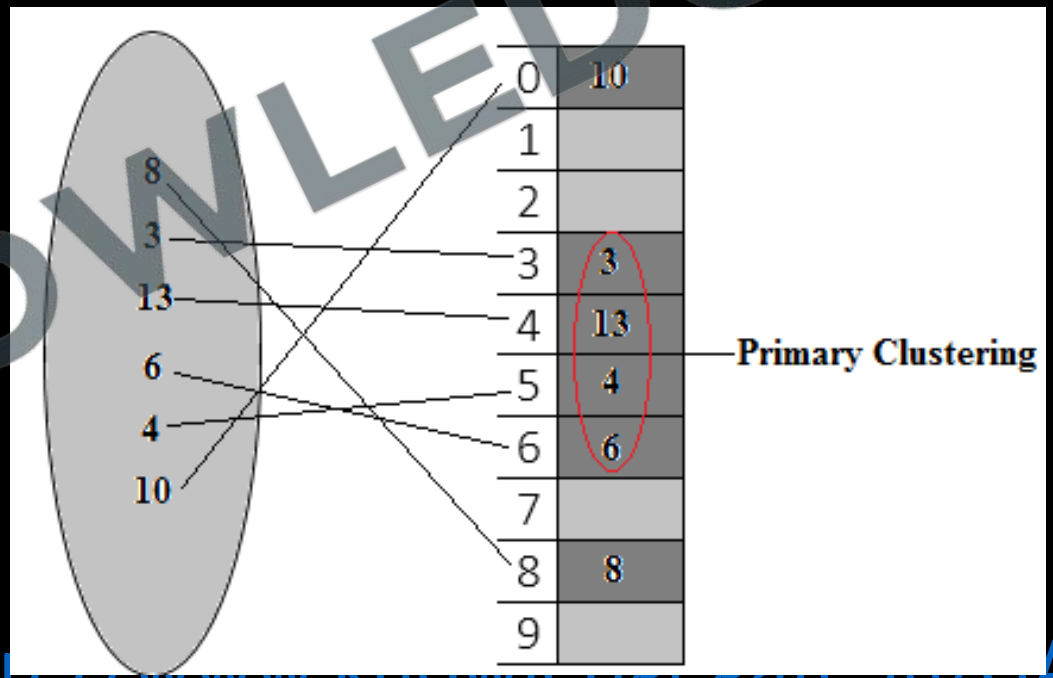
  h(k, i) = (h'(k) + i) mod m;

  for i = 0, 1, … ,m - 1.

KNOWLEDGEGATE

Example: Let us take the previous example, where the key value 13 was causing the collision at location 3.

- h (13) = (h(13) + 0) mod 10 = 3, since it is causing collision we consider the next value of i, i.e. i =1.
- h (13) = (h(13) + 1) mod 10 = 4, now at this location there is no collision so we place the value 13 at location 4.



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q A hash table contains 10 buckets and uses linear probing to resolve collision. The key values are integers and the hash function used is key%10, if the values 43 165 62 123 142 are inserted in the table, in what location would the key value 142 be inserted?

A) 2 **b)** 3 **c)** 4 **d)** 6

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider a hash table of size seven, with starting index zero, and a hash function (3x + 4)mod7. Assuming the hash table is initially empty, which of the following is the contents of the table when the sequence 1, 3, 8, 10 is inserted into the table using closed hashing? Note that '_' denotes an empty location in the table.

(A) 8, _, _, _, _, _, 10

(B) 1, 8, 10, _, _, _, 3

(C) 1, _, _, _, _, _,3

(D) 1, 10, 8, _, _, _, 3

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Advantage**:
  - Linear probing is fast, simple, and easy to implement, making it a popular choice on standard hardware.
  - It offers high performance due to its excellent locality of reference.

- **Disadvantage**:
  - It's sensitive to the quality of its hash function compared to other schemes.
  - Performance drops faster at high load factors due to primary clustering, leading to more nearby collisions and longer operation times.
  - Requires a superior hash function for optimal performance than some other methods.

[KNOWLEDGEGATE]

- **Primary Clustering**: In open-addressing hash tables, especially with linear probing, collisions result in records being placed in the next available hash table cell. This creates a contiguous cluster of occupied cells. When another record hashes to any part of this cluster, the cluster size increases by one.

- **Secondary Clustering**: This occurs in open addressing modes, including linear and quadratic probing, where the probe sequence doesn't depend on the key. A subpar hash function can make many keys hash to the same spot. Consequently, these keys either follow the same probe sequence or land in the same hash chain, leading to slower access times.

KNOWLEDGEGATE

Quadratic Probing

- Quadratic probing operates by taking the original hash index and adding successive values of an arbitrary quadratic polynomial until an open slot is found.

- ***Quadratic probing*** uses a hash function of the form

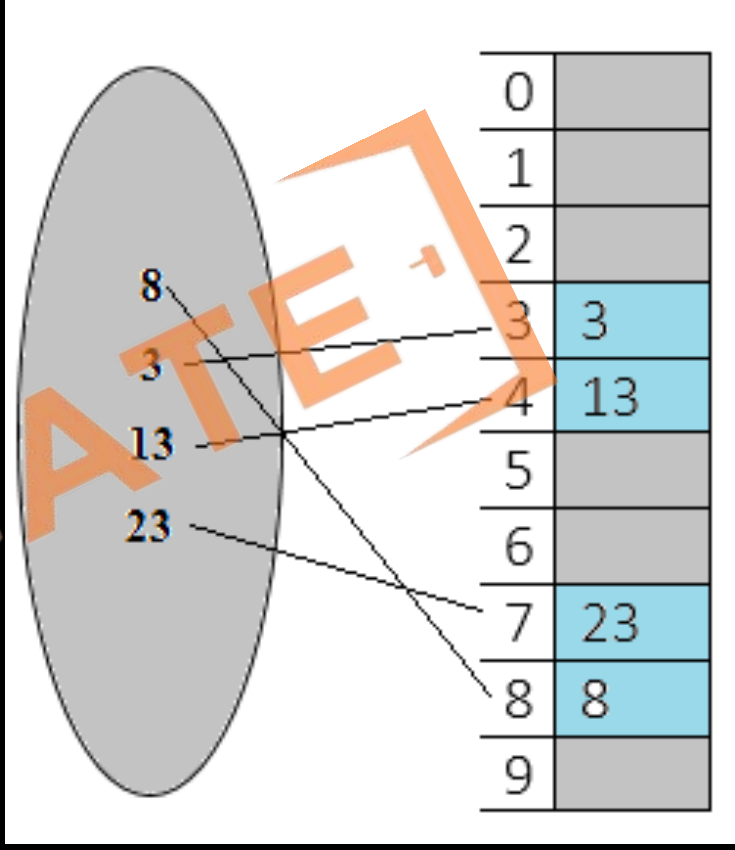
$$h(k, i) = (h'(k) + f(i^2)) \bmod m$$

Where, h' is an auxiliary hash function and i = 0, 1, …, m - 1.

KNOWLEDGEGATE

Example: Consider the key values 8, 3, 13, 23 and the hash table size is 10.

- 8 will be placed at: h (8) = [h (8) + f (0^2)] mod 10 = 8, so it gets placed at location 8.
- 3 will be placed at: h (3) = [h (3) + f (0^2)] mod 10 = 3, no collision, so it gets placed at location 3
- 13 will be placed at: h (13) = [h (13) + f (0^2)] mod 10 = 3, collision occurred, so we increase the value of i.
  - h (13) = [h (13) + f (1^2)] mod 10 = 4, no collision, so it gets placed at location 4.
- 23 will be placed at: h (23) = [h (23) + f (0^2)] mod 10 = 3, collision occurred, so we increase the value of i.
  - h (23) = [h (23) + f (1^2)] mod 10 = 4, again collision occurred, so we increase the value of i.
  - h (23) = [h (23) + f (2^2)] mod 10 = 3 + 4 = 7, no collision occurred, so it gets placed at location 7.
- Quadratic probing avoids clustering of elements and thus improves the searching time.



KNOWLEDGEGATE

- **Advantage**
  - Quadratic probing can be a more efficient algorithm in a closed hashing table, since it better avoids the clustering problem that can occur with linear probing, although it is not immune.
  - It also provides good memory caching because it preserves some locality of reference; however, linear probing has greater locality and, thus, better cache performance.

- **Disadvantage**
  - Quadratic probing lies between the two in terms of cache performance and clustering.

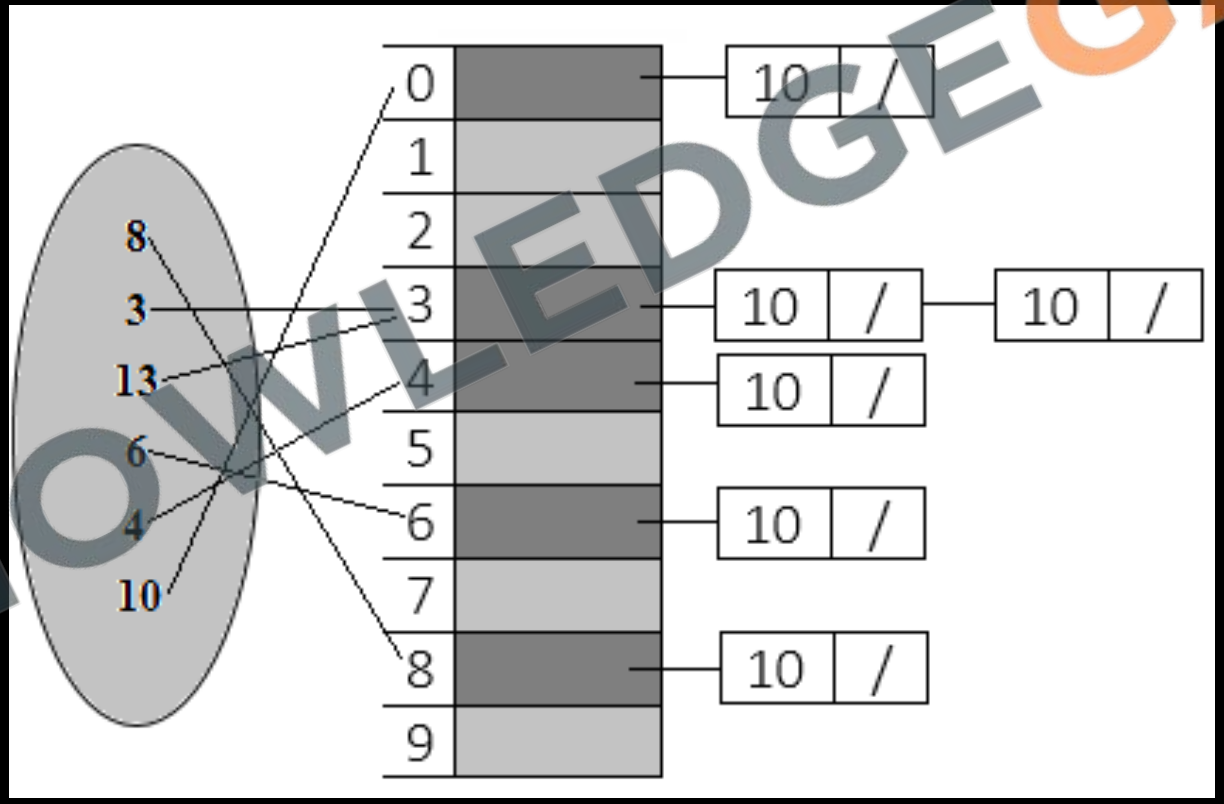
KNOWLEDGEGATE

- **Performance of Open Addressing**: Like Chaining, performance of hashing can be evaluated under the assumption that each key is equally likely to be hashed to any slot of table (simple uniform hashing)
  - m = Number of slots in hash table
  - n = Number of keys to be inserted in hash table
  - Load factor $\alpha = n/m$ (< 1)
  - Expected time to search/insert/delete $< 1/(1 - \alpha)$
  - So Search, Insert and Delete take $(1/(1 - \alpha))$ time

KNOWLEDGEGATE

Chaining

- The idea is to make each cell of hash table point to a linked list of records that have same hash function value. In *chaining*, we place all the elements that hash to the same slot into the same linked list.



KNOWLEDGEGATE

- **Advantage: -** Chaining is simple
- **Disadvantage: -**but requires additional memory outside the table.

KNOWLEDGEGATE

S.No.	Separate Chaining	Open Addressing
1.	Chaining is Simpler to implement.	Open Addressing requires more computation.
2.	In chaining, Hash table never fills up, we can always add more elements to chain.	In open addressing, table may become full.
3.	Chaining is Less sensitive to the hash function or load factors.	Open addressing requires extra care for to avoid clustering and load factor.
4.	Chaining is mostly used when it is unknown how many and how frequently keys may be inserted or deleted.	Open addressing is used when the frequency and number of keys is known.
5.	Cache performance of chaining is not good as keys are stored using linked list.	Open addressing provides better cache performance as everything is stored in the same table.
6.	Wastage of Space (Some Parts of hash table in chaining are never used).	In Open addressing, a slot can be used even if an input doesn't map to it.
7.	Chaining uses extra space for links.	No links in Open addressing

[KNOWLEDGEGATE]
http://www.knowledgegate.in/GATE

Q Consider a hash table with 9 slots. The hash function is $h(k) = k \bmod 9$. The collisions are resolved by chaining. The following 9 keys are inserted in the order: 5, 28, 19, 15, 20, 33, 12, 17, 10. The maximum, minimum, and average chain lengths in the hash table, respectively, are

(A) 3, 0, and 1 **(B)** 3, 3, and 3 **(C)** 4, 0, and 1 **(D)** 3, 0, and 2

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Double Hashing

- Double hashing is used in hash tables to handle hash collisions using open addressing. It uses two hash values: the primary for table indexing and the secondary to set an interval for searching. This method differs from linear and quadratic probing. With double hashing, data mapped to the same location has varied bucket sequences, reducing repeated collisions.
- Given two random, uniform, and independent hash functions h_1 and h_2, the i^{th} location in the bucket sequence for value k in a hash table of |T| buckets is: $h(i, k) = (h_1(k) + i \cdot h_2(k)) \bmod |T|$. Generally, h_1 and h_2 are selected from a set of universal hash functions; h_1 is selected to have a range of $\{0, |T| - 1\}$ and h_2 to have a range of $\{1, |T| - 1\}$. Double hashing approximates a random distribution; more precisely, pair-wise independent hash functions yield a probability of $(n/|T|)^2$ that any pair of keys will follow the same bucket sequence

KNOWLEDGEGATE

Q Consider a double hashing scheme in which the primary hash function is $h_1(k) = k \bmod 23$, and the secondary hash function is $h_2(k) = 1+(k \bmod 19)$. Assume that the table size is 23. Then the address returned by probe 1 in the probe sequence (assume that the probe sequence begins at probe 0) for key value k = 90 is _____ .?

In double hashing scheme, the probe sequence is determined by $(h1(k) + ih2(k)) \mod m$, where i denotes the index in probe sequence and m denotes the hash table size.

Given $h1(k)$ and $h2(k)$, we have to determine the second element of the probe sequence (i.e. $i = 1$) for the key $k = 90$.

$$(h1(90) + 1 * h2(90)) \mod 23 = (21 + 15) \mod 23 = 36 \mod 23 = 13$$

KNOWLEDGEGATE

Module

Free Semester One Shot Exam Course

Topic

Algorithms

Subtopic

Algo One Shot

Lesson

Algorithms_Notes

SEMESTER EXAM SERIES

ALGORITHM

Syllabus

- **Chapter-1 (Introduction)**: Algorithms, Analysing Algorithms, Efficiency of an Algorithm, Time and Space Complexity, Asymptotic notations: Big-Oh, Time-Space trade-off Complexity of Algorithms, Growth of Functions, Performance Measurements.
- **Chapter-2 (**Sorting and Order Statistics**)**: Concept of Searching, Sequential search, Index Sequential Search, Binary Search Shell Sort, Quick Sort, Merge Sort, Heap Sort, Comparison of Sorting Algorithms, Sorting in Linear Time. Sequential search, Binary Search, Comparison and Analysis Internal Sorting: Insertion Sort, Selection, Bubble Sort, Quick Sort, Two Way Merge Sort, Heap Sort, Radix Sort, Practical consideration for Internal Sorting.
- **Chapter-3 (Divide and Conquer)**: with Examples Such as Sorting, Matrix Multiplication, Convex Hull and Searching.
- **Chapter-4 (Greedy Methods):** with Examples Such as Optimal Reliability Allocation, Knapsack, Huffman algorithm
- **Chapter-5 (Minimum Spanning Trees)** – Prim's and Kruskal's Algorithms,
- **Chapter-6 (Single Source Shortest Paths):** - Dijkstra's and Bellman Ford Algorithms.
- **Chapter-7 (Dynamic Programming)** with Examples Such as Knapsack. All Pair Shortest Paths – Warshal's and Floyd's Algorithms, Resource Allocation Problem. Backtracking, Branch and Bound with Examples Such as Travelling Salesman Problem, Graph Coloring, n-Queen Problem, Hamiltonian Cycles and Sum of Subsets.
- **Chapter-8 (Advanced Data Structures)**: Red-Black Trees, B – Trees, Binomial Heaps, Fibonacci Heaps, Tries, Skip List, Introduction to Activity Networks Connected Component.
- **Chapter-9 (Selected Topics)**: Algebraic Computation, Fast Fourier Transform, String Matching, Theory of NPCompleteness, Approximation Algorithms and Randomized Algorithms .

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Chapter-1 (Introduction)

Algorithms, Analysing Algorithms, Efficiency of an Algorithm, Time and Space Complexity, Asymptotic notations: Big-Oh, Time-Space trade-off Complexity of Algorithms, Growth of Functions, Performance Measurements.

KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

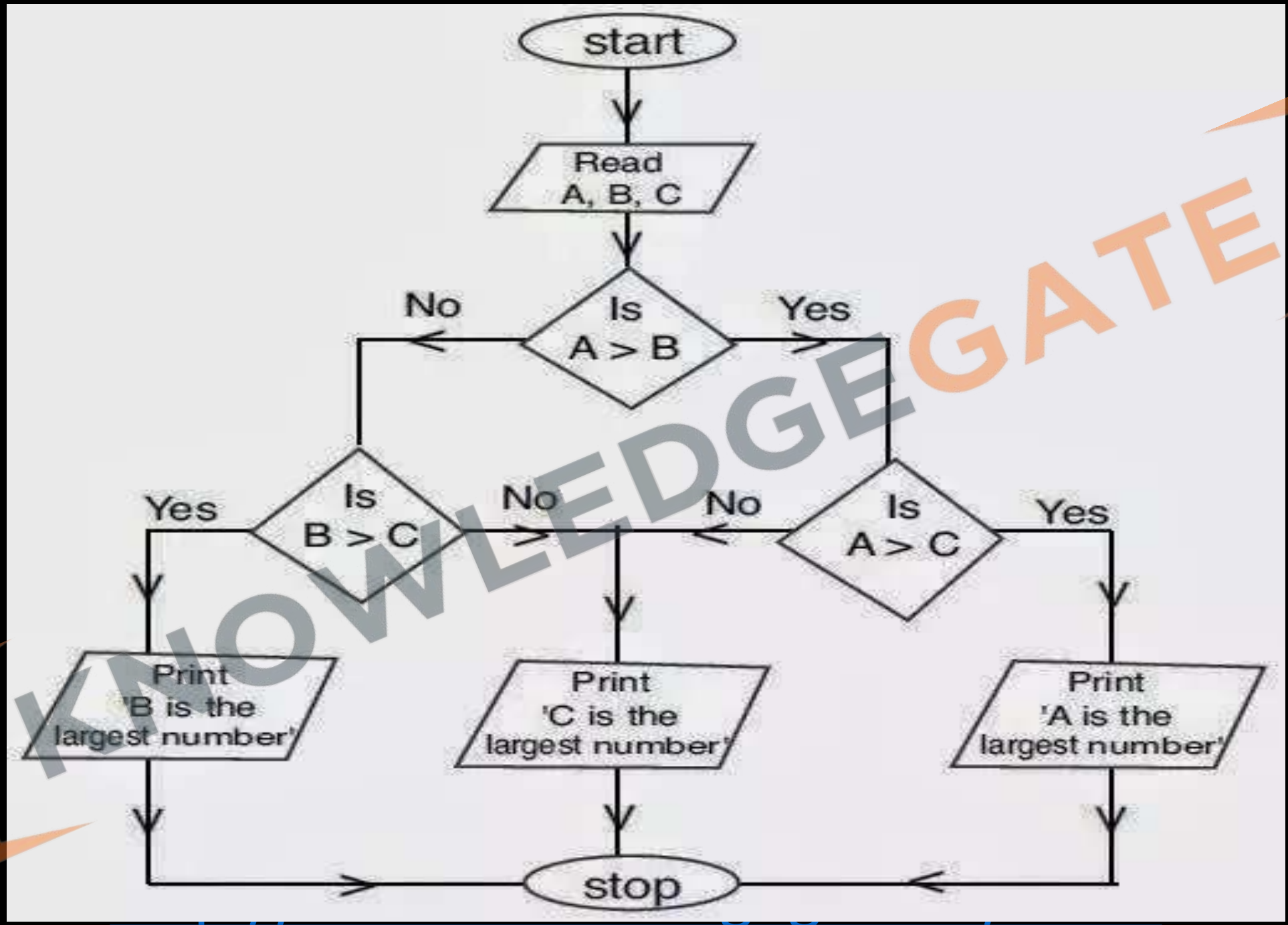
Q Find the Largest Number Among Three Numbers ?

KNOWLEDGEGATE

1. Start
2. Read the three numbers to be compared, as A, B and C.
3. Check if A is greater than B.
    3.1 If true, then check if A is greater than C.
        3.1.1 If true, print 'A' as the greatest number.
        3.1.2 If false, print 'C' as the greatest number.
    3.2 If false, then check if B is greater than C.
        3.2.1 If true, print 'B' as the greatest number.
        3.2.2 If false, print 'C' as the greatest number.
4. End

[KNOWLEDGEGATE]

http://www.knowledgegate.in/GATE



KNOWLEDGEGATE

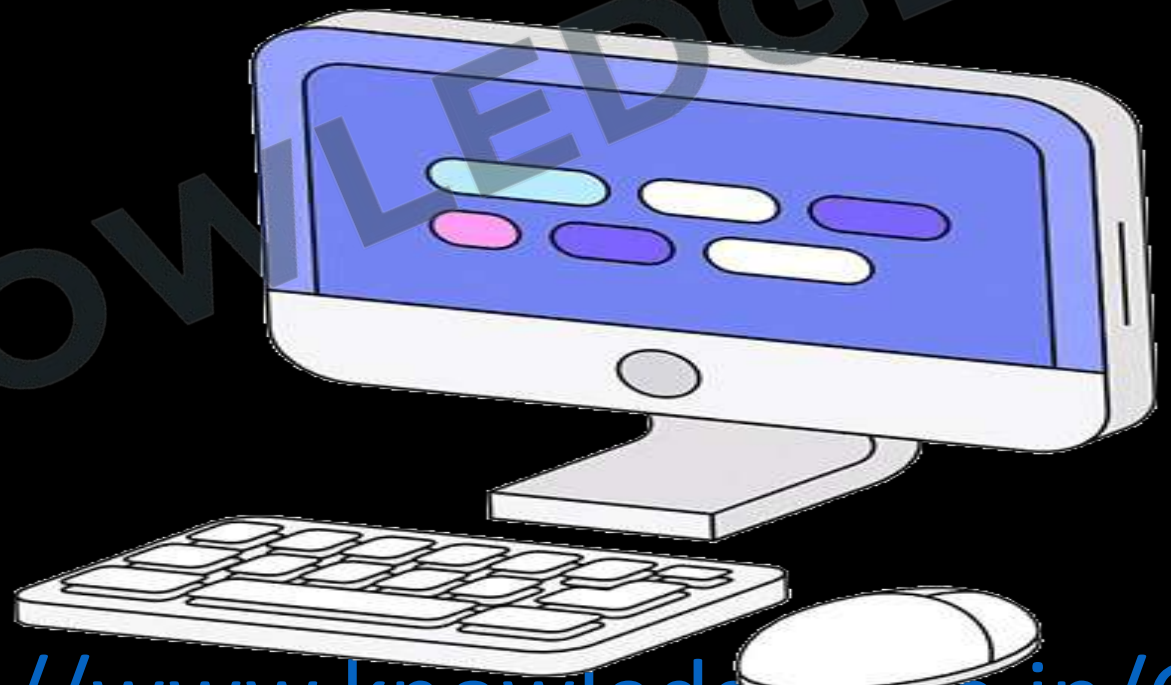
```c
#include <stdio.h>
int main()
{
    int A, B, C;

    printf("Enter the numbers A, B and C: ");
    scanf("%d %d %d", &A, &B, &C);

    if (A >= B && A >= C)
        printf("%d is the largest number.", A);

    if (B >= A && B >= C)
        printf("%d is the largest number.", B);

    if (C >= A && C >= B)
        printf("%d is the largest number.", C);

    return 0;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Introduction to Algorithm

- In mathematics and computer science, an algorithm is a finite sequence of well-defined, computer-implementable instructions, typically to solve a class of problems or to perform a computation. A stem by step Procedure.



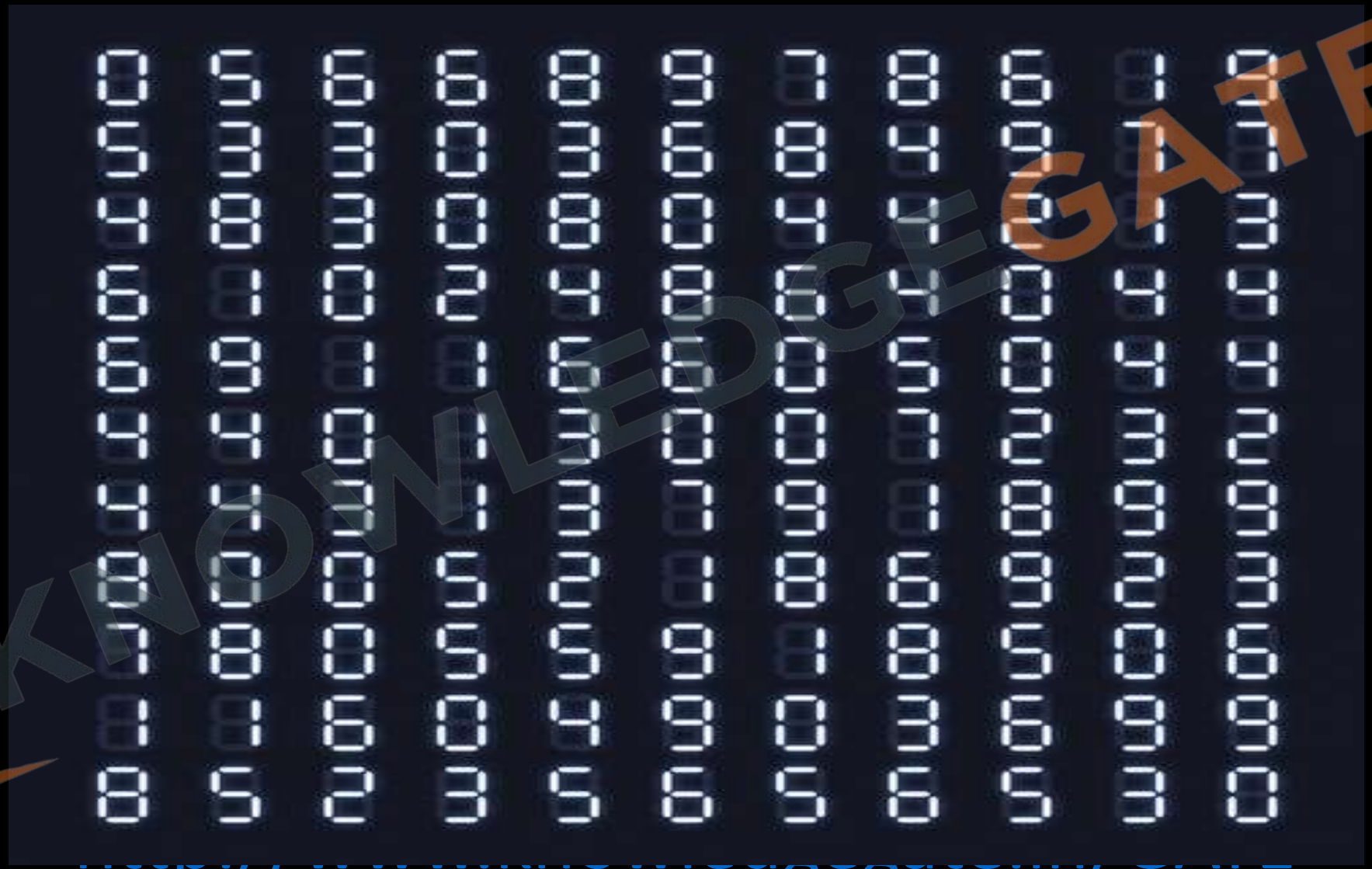
KNOWLEDGEGATE

- Algorithms are unambiguous specifications for performing calculation, data processing, automated reasoning, and other tasks.

बेटा किताबों को आग लगा दो

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- Will accept Zero or more input, but generate at least one output.



KNOWLEDGE GATE

Properties of Algorithm

- Should terminate in finite time
- Unambiguous
- Input Zero or more output at least one
- Every instruction in algo should be effective
- It should be deterministic

KNOWLEDGEGATE

Algorithm	Program
Written in design Phase	Written in Implementation Phase
Needs Domain Knowledge	Needs Programmer Knowledge
Can be Written in Any Language	Can be Written in Programming Language
Independent of H/w & S/w	Dependent of H/w & S/w
We analysis Algorithm for time & space	We test programs for faults

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Problem Solving Cycle

- Problem Definition: Understand Problem
- Constraints & Conditions: Understand constraints if any
- Design Strategies (Algorithmic Strategy)
- Express & Develop the algo
- Validation (Dry run)
- Analysis (Space and Time analysis)
- Coding
- Testing & Debugging
- Installation
- Maintenance

KNOWLEDGEGATE

Need for Analysis

- We do analysis of algorithm to do a performance comparison between different algorithm to figure out which one is best possible option.

KNOWLEDGEGATE

- What parameters can be considered for comparison between cars?

For Algo?

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- Following are the parameters which can be considered while analysis of an algorithm
  - Time
  - Space
  - Bandwidth
  - Register
  - Battery power

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- Out of all **time** is the most important Criteria for analysis of algorithm

KNOWLEDGEGATE

- How to analysis time?

KNOWLEDGEGATE

Types of Analysis

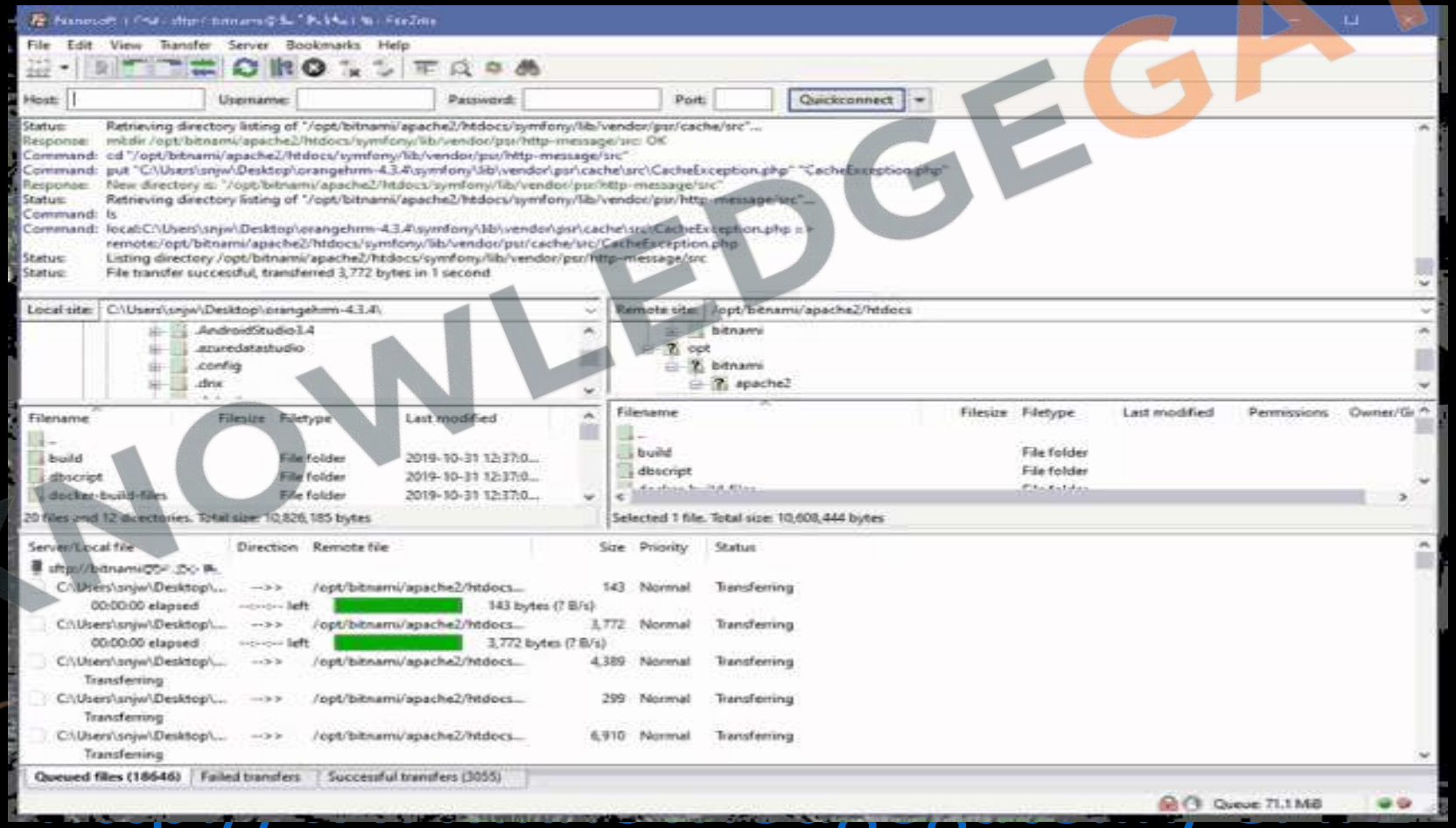
- Experimental or
- Apostrium or
- Relative analysis

- Apriori Analysis or
- Independent analysis or
- Absolute analysis

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Experimental or Apostrium or relative analysis** : Means analysis of algorithm after it is converted to code. Implement both the algorithms and run the two programs on your computer for different inputs and see which one takes less time.



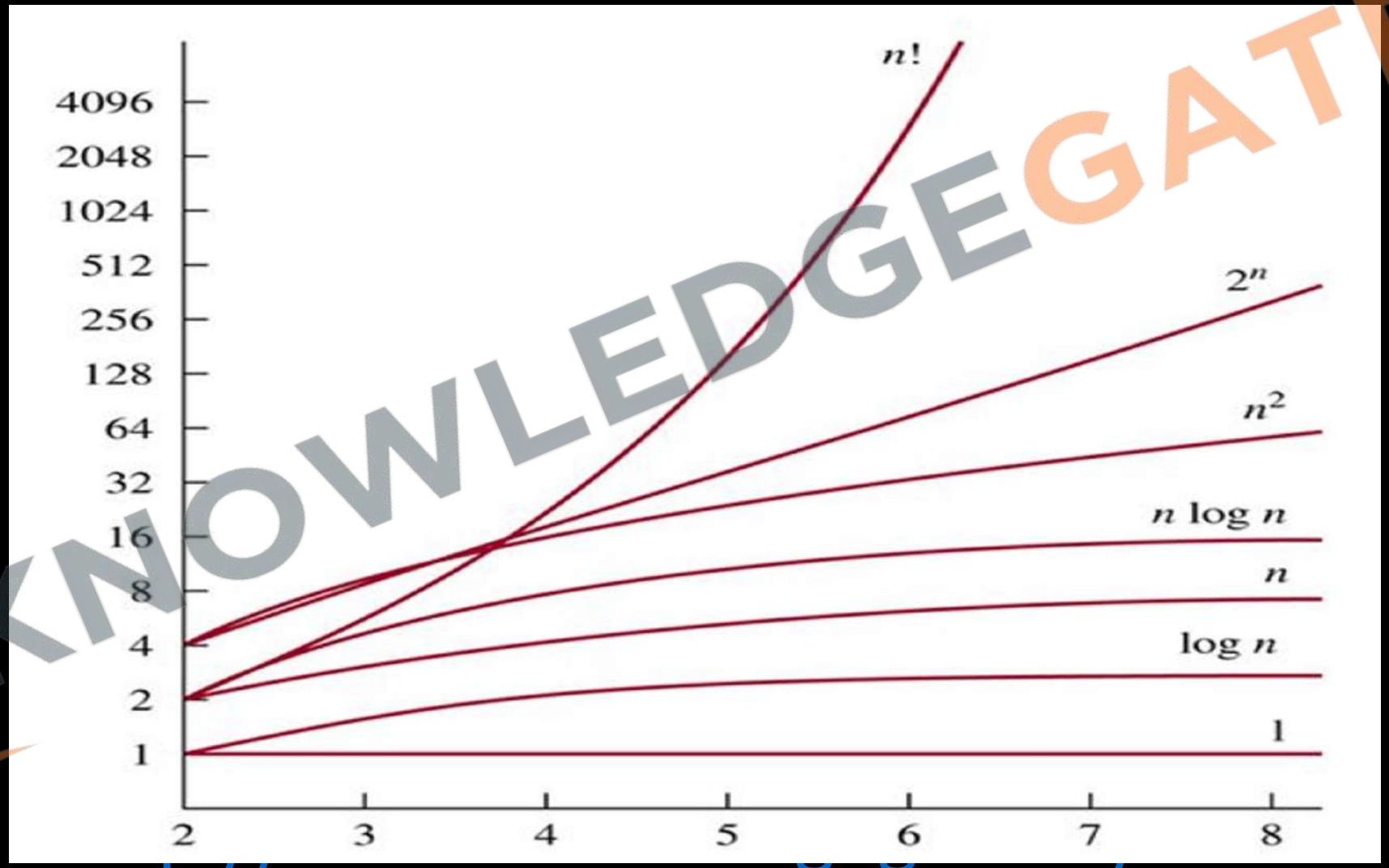
- **Advantage:** Exact values no rough
- **Disadvantage**: final result instead of depending only algorithm depends on many other factors like background software & hardware, programming language, even the temperature of the room.

KNOWLEDGEGATE

- **Apriori Analysis or Independent analysis or Absolute analysis**: we do analysis using asymptomatic notations and mathematical tools of only algorithm, i.e. before converting it into program of a particular programming language.
- It is a determination of order of magnitude of a statement.

KNOWLEDGEGATE

- In Asymptotic Analysis, we evaluate the performance of an algorithm in terms of input size (we don't measure the actual running time). We calculate, how does the time (or space) taken by an algorithm increases with the input size.



KNOWLEDGEGATE

- Asymptotic Analysis is not perfect, but that's the best way available for analyzing algorithms.

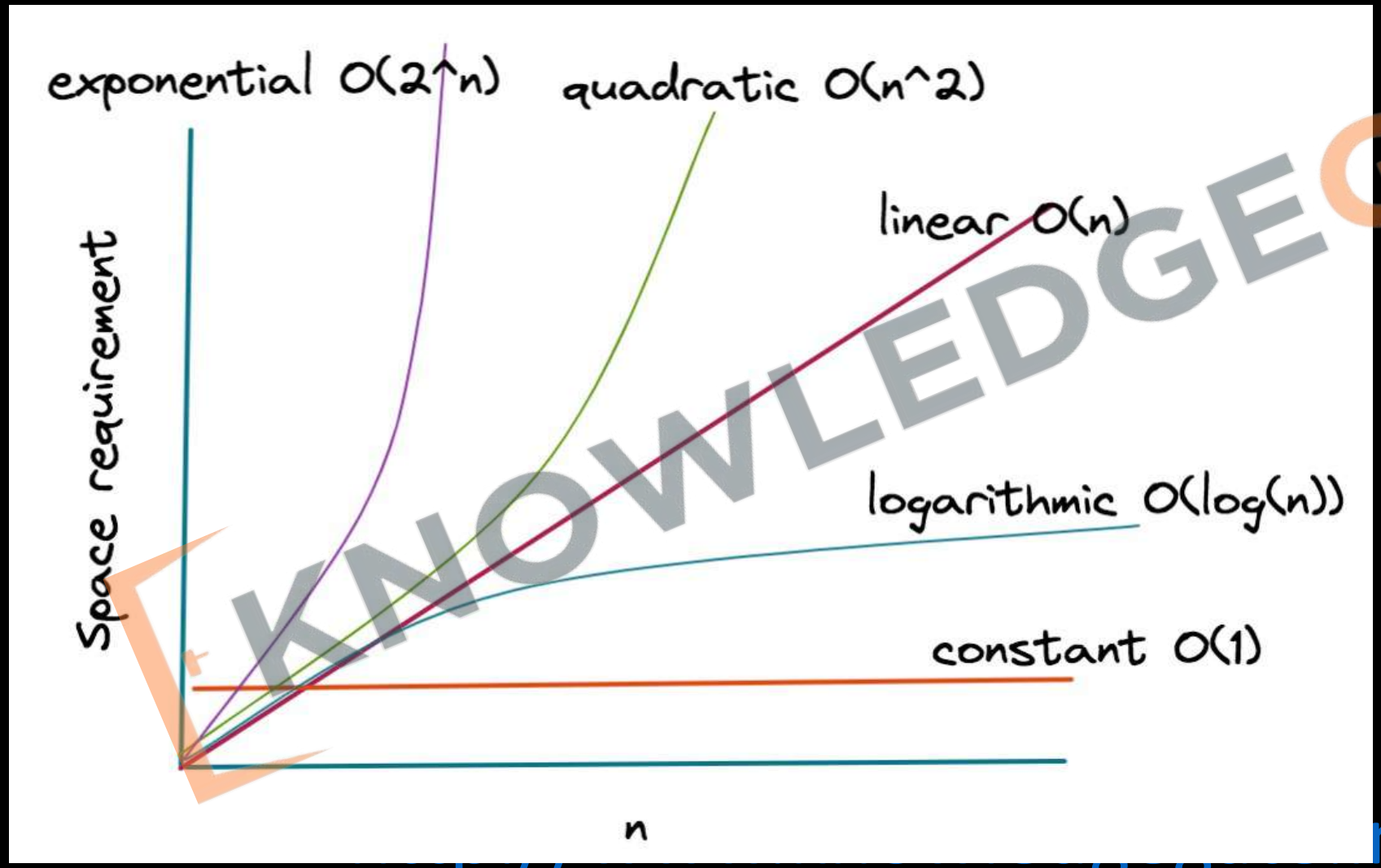
- It might be possible that those large inputs are never given to your software and an algorithm which is asymptotically slower, always performs better for your particular situation. So, you may end up choosing an algorithm that is Asymptotically slower but faster for your software.

  - **Advantage**: Uniform result depends only on algorithm.
  - **Disadvantage**: Estimated or approximate value no accurate and precise value.

KNOWLEDGEGATE

Asymptotic Notations

- Asymptotic notations are Abstract notation for describing the behavior of algorithm and determine the rate of growth of a function.



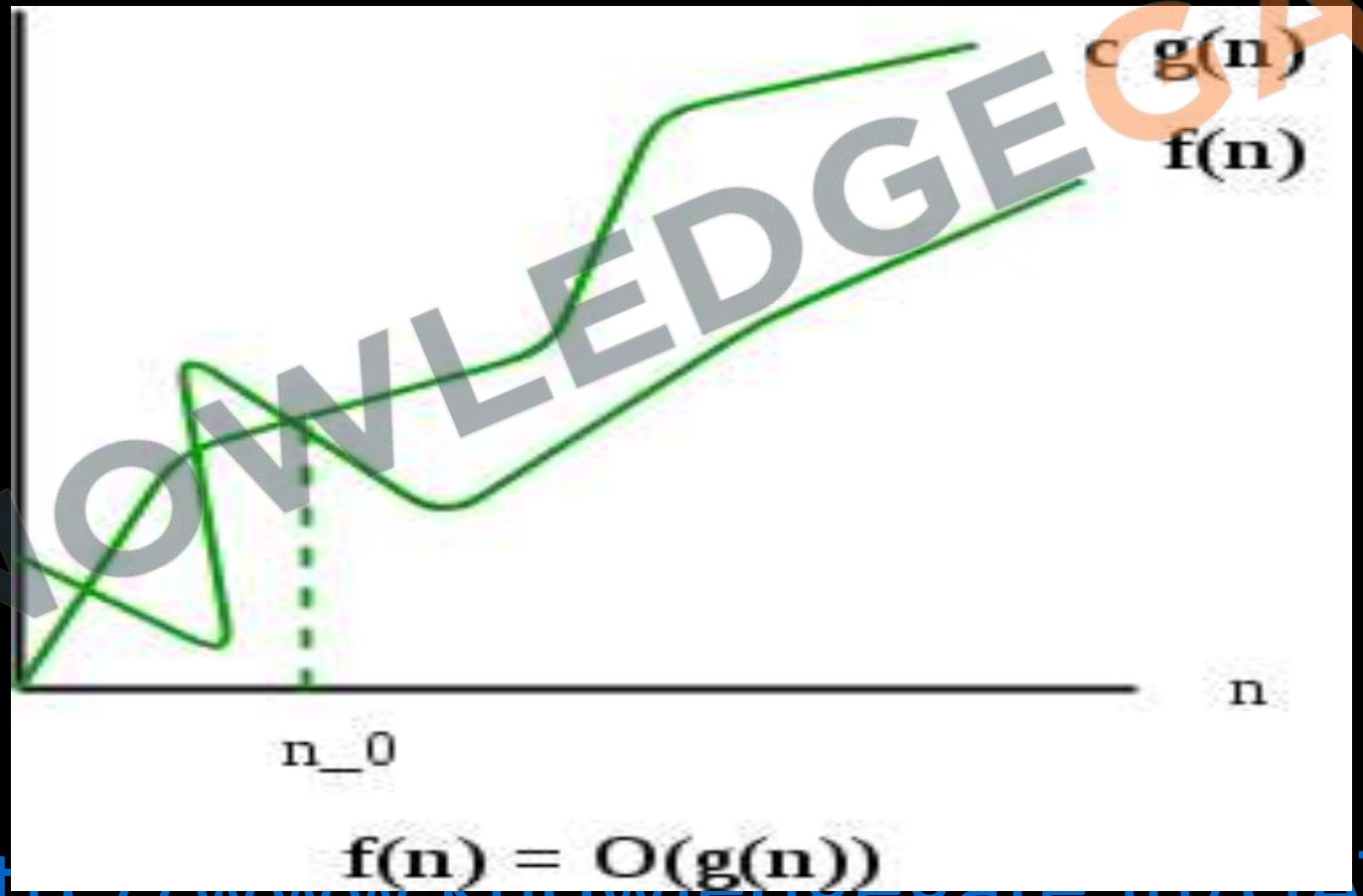
KNOWLEDGEGATE

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Big O Notation

- The Big O notation defines an upper bound of an algorithm, it bounds a function only from above.
- The Big O notation is useful when we only have upper bound on time complexity of an algorithm.
- Many times, we easily find an upper bound by simply looking at the algorithm.
- O(g(n)) = {f(n): there exist positive constants C and N_0 such that 0 <= f(n) <= C*G(n) for all N >= N_0}

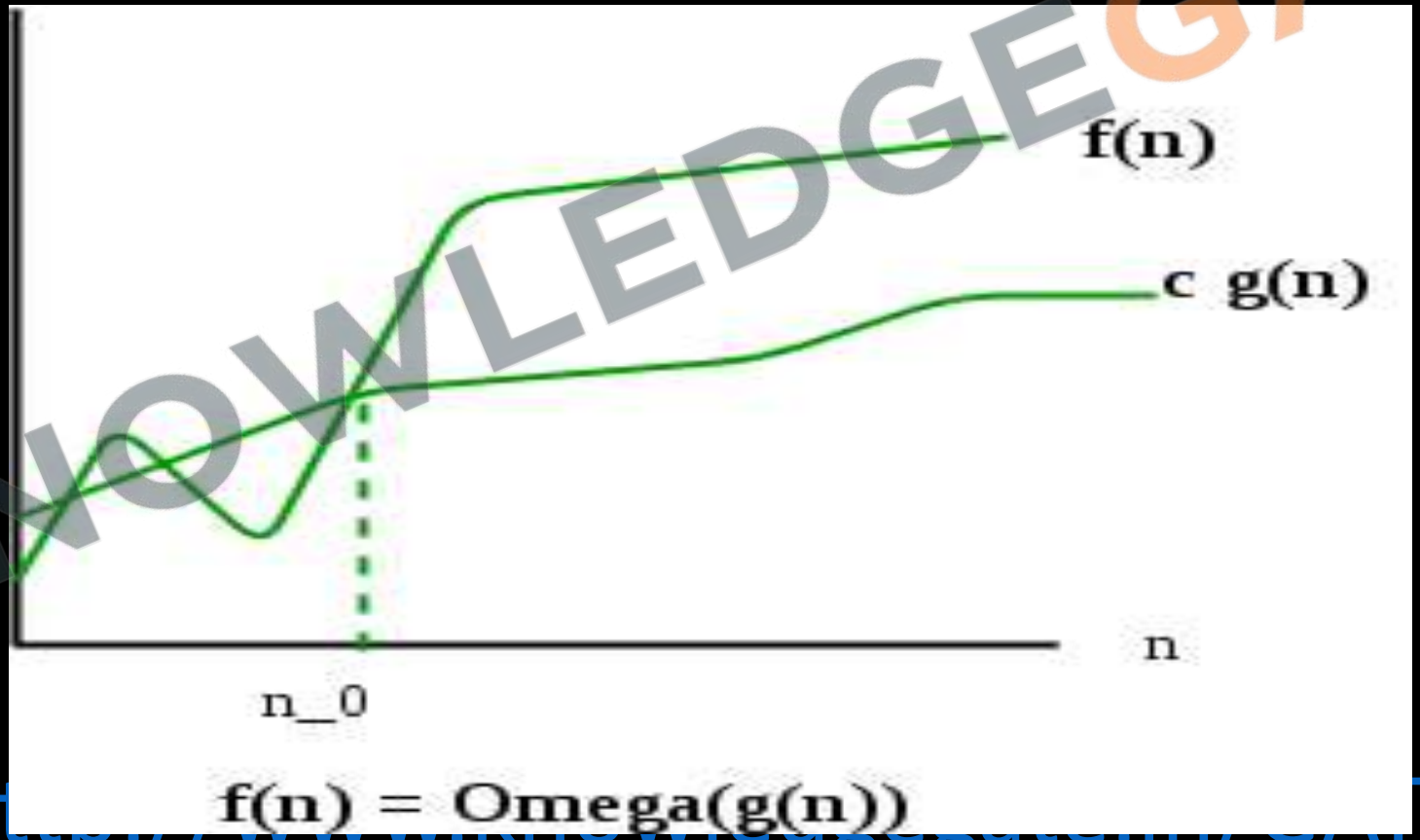


KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Ω Notation

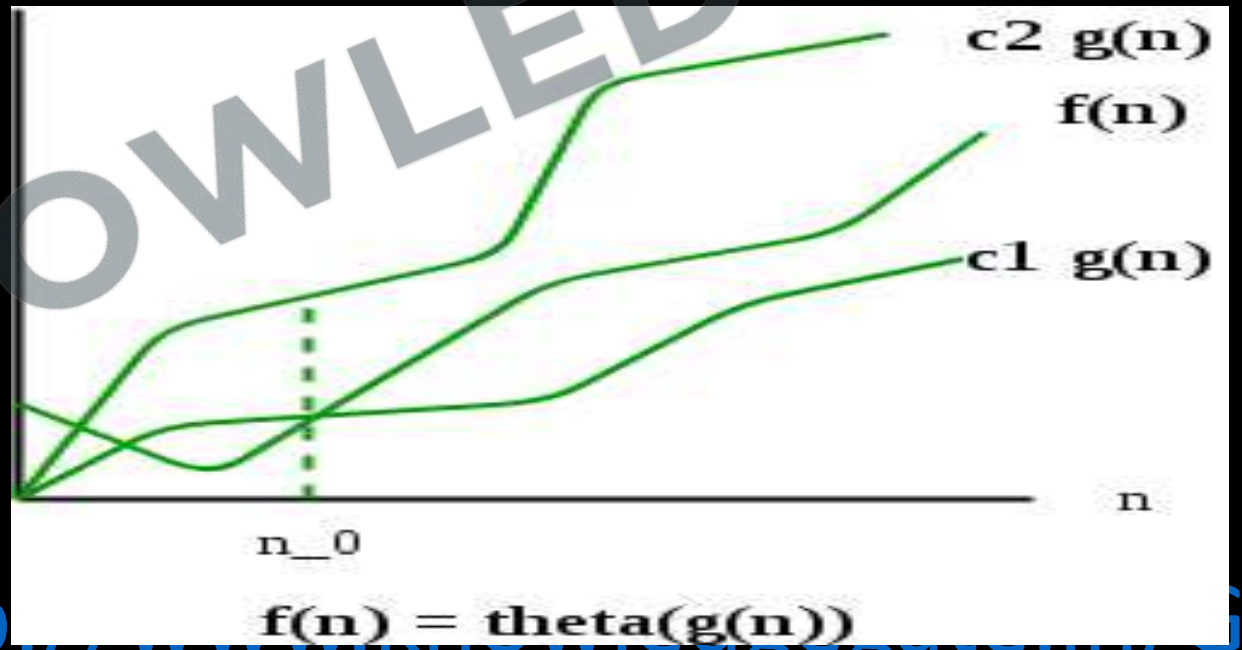
- Just as Big O notation provides an asymptotic upper bound on a function, Ω notation provides an asymptotic lower bound.
- Ω Notation can be useful when we have lower bound on time complexity of an algorithm.
- For a given function g(n), we denote by Ω(g(n)) the set of functions.
- Ω (g(n)) = {f(n): there exist positive constants c and n_0 such that 0 <= c*g(n) <= f(n) for all n >= n_0}.



KNOWLEDGEGATE

Theta Notation

- **Θ Notation:** The theta notation bounds a function from above and below, so it defines exact asymptotic behaviour.
- For a given function g(n), we denote Θ(g(n)) is following set of functions.
- Θ(g(n)) = {f(n): there exist positive constants C_1, C_2 and n_0 such that 0 <= C_1*g(n) <= f(n) <= C_2*g(n) for all n >= n_0}
- The above definition means, if f(n) is theta of g(n), then the value f(n) is always between C_1*g(n) and C_2*g(n) for large values of n (n >= n_0).
- The definition of theta also requires that f(n) must be non-negative for values of n greater than n_0.



KNOWLEDGEGATE

Small notations

- Every thing is same as big notations just, just we take strictly increasing or monotonically increasing case and equal case is not allowed.

- Analogy of asymptomatic notation with real numbers

f(n) is O((g(n))	a <= b
f(n) is Ω (g(n))	a >= b
f(n) is Θ (g(n))	a = b
f(n) is o(g(n))	a < b
f(n) is ω(g(n))	a > b

KNOWLEDGEGATE

Conclusion

- Most useful notation is Theta, followed by Big O

- the Omega notation is the least used notation among all three

KNOWLEDGEGATE

$f(n) = 100 * 2^n + n^5 + n$ then show $f(n) = O(2^n)$

$f(n) = 100 * 2^n + n^5$

$n^5 >> n$

$f(n) = 100 * 2^n$

$2^n >> n^5$

$f(n) = 2^n$

$2^n \cong 100 \times 2^n$

$f(n) = O(2^n)$

KNOWLEDGEGATE

Master Theorem

- In the analysis of algorithms, the master theorem for divide-and-conquer recurrences provides an asymptotic analysis (using Big O notation) for recurrence relations of types that occur in the analysis of many divide and conquer algorithms.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- The approach was first presented by Jon Bentley, Dorothea Haken, and James B. Saxe in 1980, where it was described as a "unifying method" for solving such recurrences. The name "master theorem" was popularized by the widely used algorithms textbook Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein.

KNOWLEDGEGATE

http://w

- The above equation divides the problem into ‘a’ number of subproblems recursively, a >= 1
- Each subproblem being of size n/b. the subproblems (of size less than k) that do not recurse. (b > 1)
- where f(n) is the time to create the subproblems and combine their results in the above procedure.

$$T(n) = a\,T(n/b) + f(n)$$

KNOWLEDGEGATE

Case 1

- If $f(n) = O(n^{\log_b a} - \epsilon)$ for some constant $\epsilon > 0$,
  - then $T(n) = \Theta(n^{\log_b a})$

KNOWLEDGEGATE

- Case1
  - If $f(n) = O(n^{\log_b a} - \epsilon)$ for some constant $\epsilon > 0$,
  - then $T(n) = \Theta(n^{\log_b a})$

- **Q** $T(n) = 4T(n/2) + n$?

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- Case1
  - If $f(n) = O(n^{\log_b a} - \epsilon)$ for some constant $\epsilon > 0$,
  - then $T(n) = \Theta(n^{\log_b a})$

- **Q** $T(n) = 9T(n/3) + n$?

KNOWLEDGEGATE

- Case1
  - If $f(n) = O(n^{\log_b a} - \epsilon)$ for some constant $\epsilon > 0$,
  - then $T(n) = \Theta(n^{\log_b a})$

- **Q** $T(n) = 9T(n/3) + n^2$?

KNOWLEDGEGATE

- Case1
  - If $f(n) = O(n^{\log_b a} - \epsilon)$ for some constant $\epsilon > 0$,
  - then $T(n) = \Theta(n^{\log_b a})$

- **Q** $T(n) = 8T(n/2) + n^2$?

KNOWLEDGEGATE

- **Q** $T(n) = 7T(n/2) + n^2$?

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Case 2

- If $f(n) = \Theta(n^{\log_b a})$,
  - then $T(n) = \Theta(n^{\log_b a} \lg n)$

KNOWLEDGEGATE

- Case1
  - If $f(n) = O(n^{\log_b a} - \epsilon)$ for some constant $\epsilon > 0$,
  - then $T(n) = \Theta(n^{\log_b a})$
- Case2
  - If $f(n) = \Theta(n^{\log_b a})$,
  - then $T(n) = \Theta(n^{\log_b a} \lg n)$

- Q $T(n) = 2T(n/2) + n$?

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- Case1
  - If $f(n) = O(n^{\log_b a} - \epsilon)$ for some constant $\epsilon > 0$,
  - then $T(n) = \Theta(n^{\log_b a})$
- Case2
  - If $f(n) = \Theta(n^{\log_b a})$,
  - then $T(n) = \Theta(n^{\log_b a} \lg n)$

- Q $T(n) = T(2n/3) + 1$?

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Case 3

- If $f(n) = \Omega (n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$,
- and if $a\, f(n/b) <= c\, f(n)$ for some constant $c < 1$ and all sufficiently large n,
  - then $T(n) = \Theta (f(n))$

KNOWLEDGEGATE

- Case1
  - If $f(n) = O(n^{\log_b a} - \epsilon)$ for some constant $\epsilon > 0$,
  - then $T(n) = \Theta(n^{\log_b a})$
- Case2
  - If $f(n) = \Theta(n^{\log_b a})$,
  - then $T(n) = \Theta(n^{\log_b a} \lg n)$
- Case3
  - $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$,
  - and if $a f(n/b) <= c f(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$
- $T(n) = T(n/3) + n$?

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- Case1
  - If $f(n) = O(n^{\log_b a} - \epsilon)$ for some constant $\epsilon > 0$,
  - then $T(n) = \Theta(n^{\log_b a})$
- Case2
  - If $f(n) = \Theta(n^{\log_b a})$,
  - then $T(n) = \Theta(n^{\log_b a} \lg n)$
- Case3
  - $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$,
  - and if $a f(n/b) <= c f(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$
- $T(n) = 3T(n/4) + n \lg n$?

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

$T(n) = 2T(n/2) + n^2 + 2n + 1$

$T(n) = 2T(n/2) + n^2$

$a = 2 \quad b = 2 \quad f(n) = n^2$

Case3: $f(n) = \Omega(n^{\log_b a + \epsilon})$

$= \Omega(n^{1+\epsilon})$

$n^2 = \Omega(n^2)$

$2f(\frac{n^2}{2}) \leq c \cdot f(n^2)$

$c = 2$

$T(n) = \Theta(f(n))$

$T(n) = \Theta(n^2)$

[KNOWLEDGEGATE]

http://www.knowledgegate.in/GATE

$$T(n) = T(\sqrt{n}) + O(\log n)$$

Let $n = \log n$

$2^n = n$

$2^{n/2} = n^{1/2}$

$$T(2^n) = T(2^{n/2}) + O(\log 2^n)$$

$x(n) = T(2^n)$

$$x(n) = x(n/2) + O(n)$$

$a = 1 \quad b = 2 \quad f(n) = O(n)$

$n^{\log_b a} = n^{\log_2 1 + \epsilon}$

$x(n) = \Omega(\log n) \rightarrow T(n) = \Omega(\log \log n)$

KNOWLEDGEGATE

http://www.knowledgegate.co.in/GATE

$T(n) = T(n-1) + n^4$

$$T(n) \leq \begin{cases} c, & \text{if } n \leq 1, \\ aT(n-b) + f(n), & n > 1, \end{cases}$$

$T(n) = 2T(n-1) + 1$

KNOWLEDGEGATE

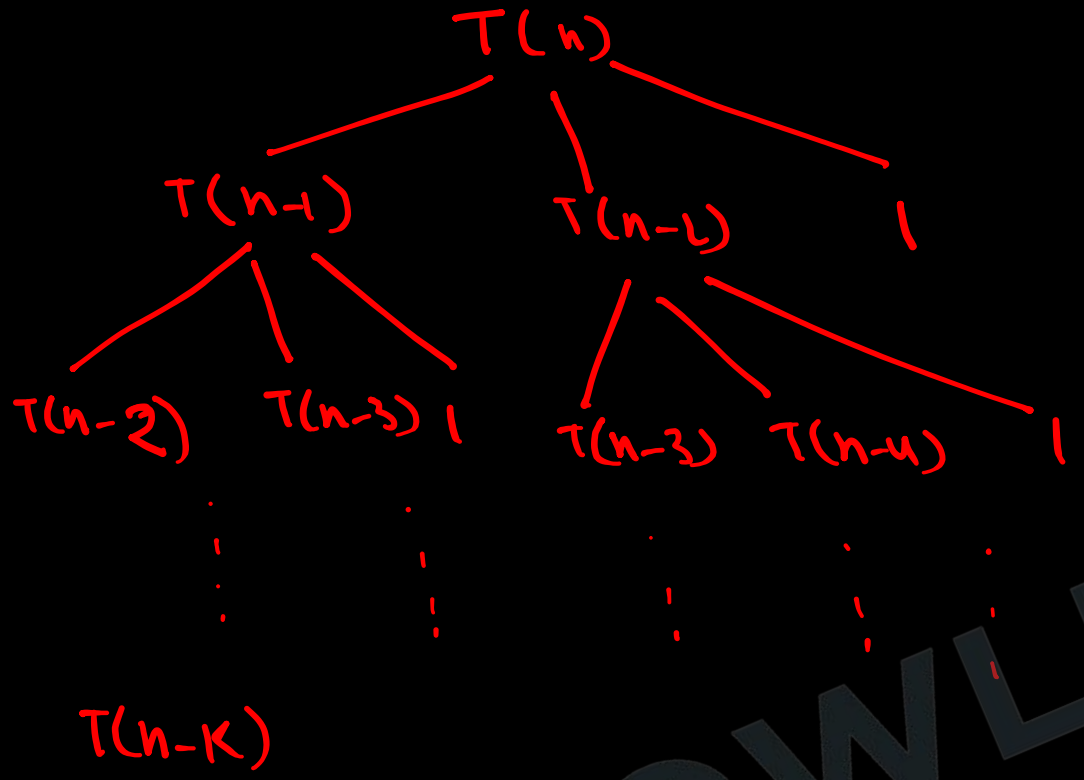
If f(n) is $O(n^k)$, then

1. If a<1 then $T(n) = O(n^k)$
2. If a=1 then $T(n) = O(n^{k+1})$
3. if a>1 then $T(n) = O(n^k a^{n/b})$

$T(n) = T(n-1) + T(n-2) + 1$

$T(0) = 0$

$T(1) = 1$



$2^0 = 1$

$2^1 = 2$

$2^2 = 4$

2^K

$n-K = 1$

$K = n-1$

$= 2^0 + 2^1 + 2^2 + \cdots + 2^{n-1}$

Sum of n terms of geometric progression

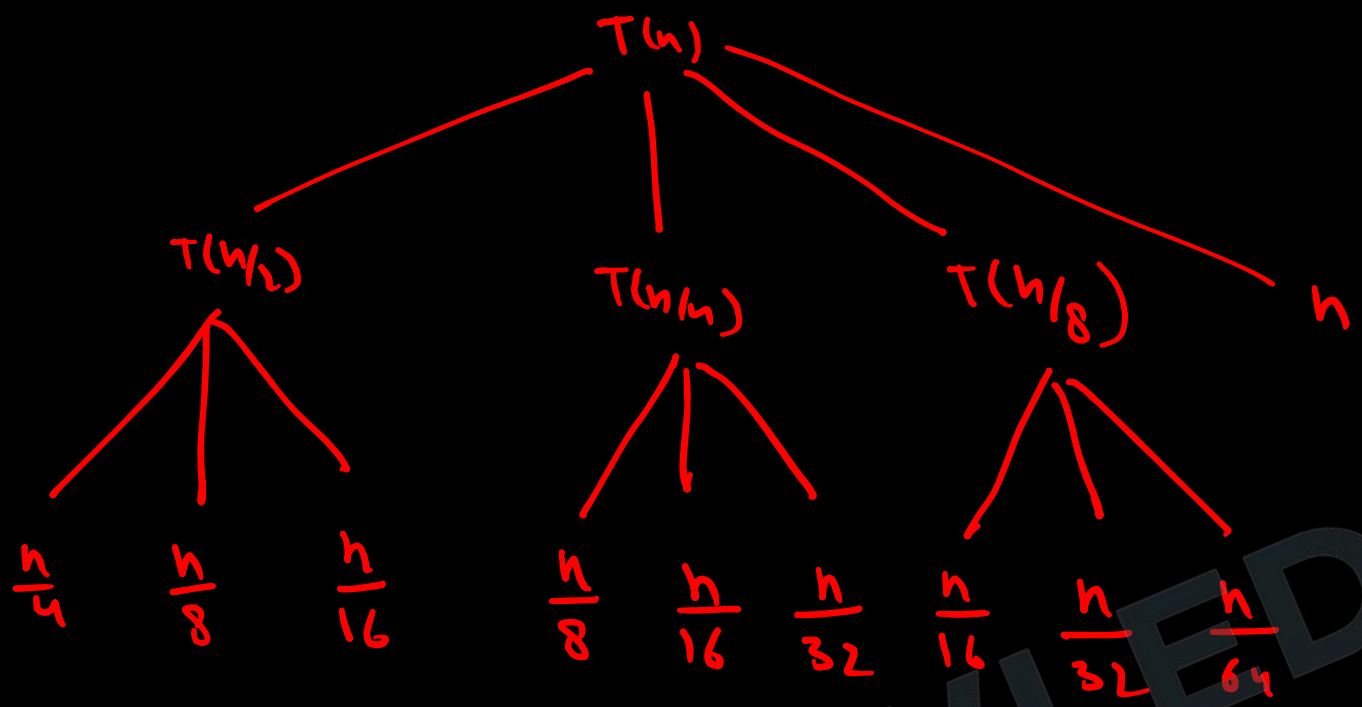
$\frac{a(r^n - 1)}{r-1} = \frac{2^0(2^{n-1} - 1)}{2-1} = 2^{n-1} - 1$

$= O(2^n)$

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

$T(n) = T(n/2) + T(n/4) + T(n/8) + n$



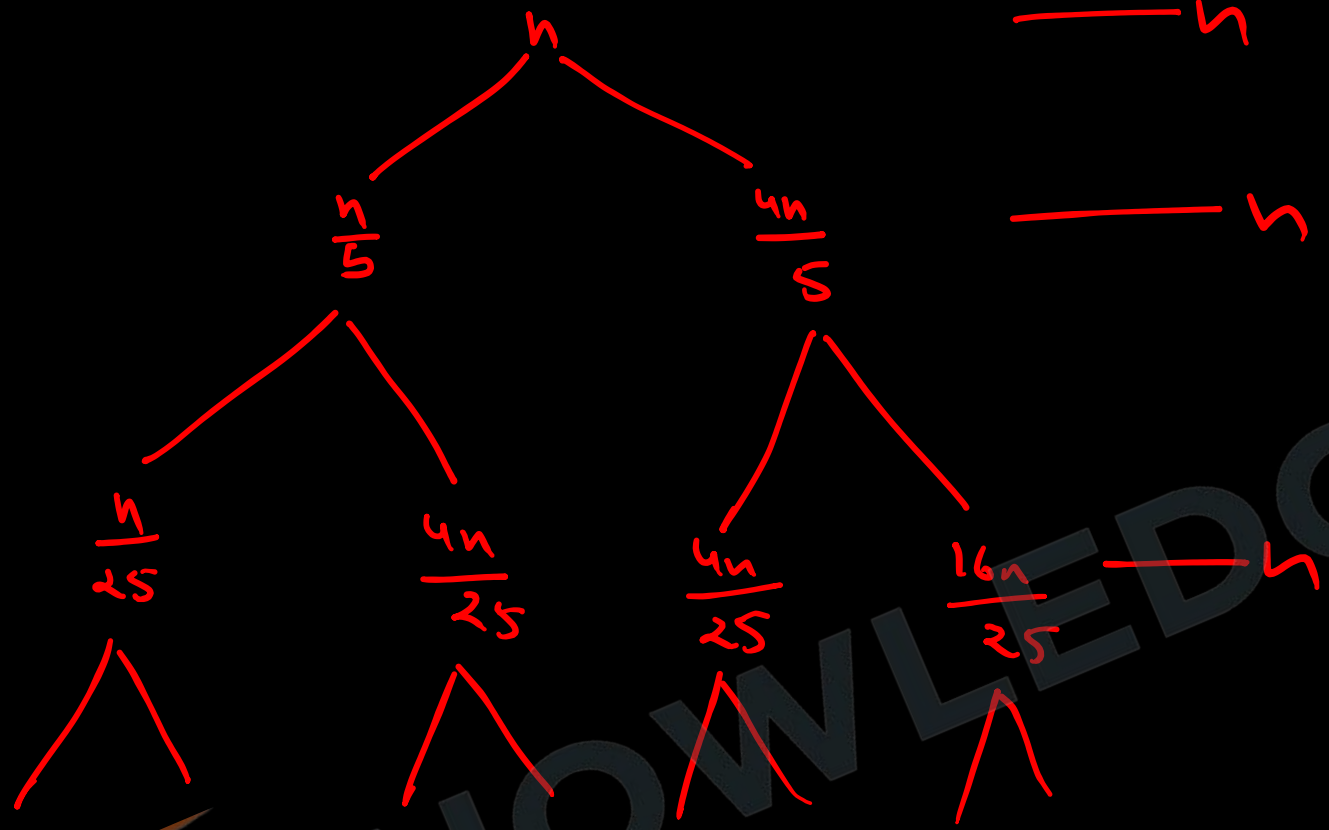
$\frac{7n}{8}$

$\frac{7^2 n}{8^2}$

$T(n) = n + \frac{7n}{8} + \frac{7^2 n}{8^2} + \cdots + \log n = O(n \log n)$

KNOWLEDGEGATE

$T(n) = n + T(n/6) + T(4n/5)$



$T(n) = n + n + n \quad \cdots\cdots \quad n \quad = \Omega(n \log n)$

$\log_{5/4} n$

KNOWLEDGEGATE

Iteration method

$T(n) = 2T(n/2) + n$

$= 2\left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2}\right] + n = 2^2 T\left(\frac{n}{2^2}\right) + n + n$

$= 2^2\left[2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}\right] + n + n = 2^3 T\left(\frac{n}{2^3}\right) + n + n + n$

$= 2^2\left[2T\left(\frac{n}{2^4}\right) + \frac{n}{2^2}\right] + n + n + n = 2^4 T\left(\frac{n}{2^4}\right) + n + n + n + n$

$\vdots$

$T(n) = 2^k T\left(\frac{n}{2^k}\right) + nk$

$T\left(\frac{n}{2^n}\right) = T(1)$

$T(n) = 2^k T(1) + kn$

$= n \cdot 1 + n \log n$

$T(n) = n \log n + n$

$= O(n \log n)$

KNOWLEDGE GATE

Q Consider the following three functions.

$f_1 = 10^n$

$f_2 = n^{\log n}$

$f_3 = n^{\sqrt{n}}$

Which one of the following options arranges the functions in the increasing order of asymptotic growth rate?

$f_2 < f_3 < f_4$

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider the following functions from positives integers to real numbers 10, $\sqrt{n}$, n, $\log_2 n$, 100/n. The CORRECT arrangement of the above functions in increasing order of asymptotic complexity

KNOWLEDGEGATE

$$100/n < 10 < \log_2 n < \sqrt{n} < n$$

http://www.knowledgegate.in/GATE

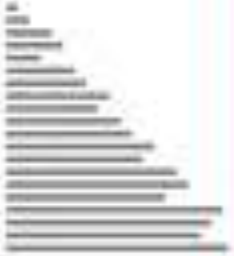
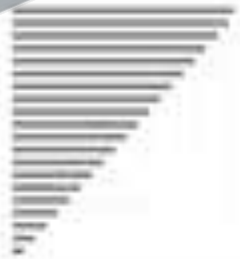
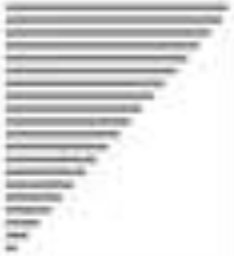
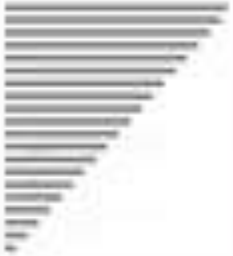
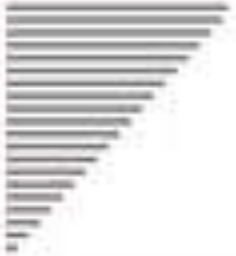
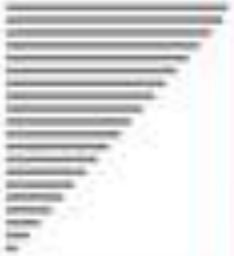
Ch-2

Sorting and Order Statistics

Concept of Searching, Sequential search, Index Sequential Search, Binary Search Shell Sort, Quick Sort, Merge Sort, Heap Sort, Comparison of Sorting Algorithms, Sorting in Linear Time. Sequential search, Binary Search, Comparison and Analysis Internal Sorting: Insertion Sort, Selection, Bubble Sort, Quick Sort, Two Way Merge Sort, Heap Sort, Radix Sort, Practical consideration for Internal Sorting.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

	Insertion	Selection	Bubble	Shell	Merge	Heap	Quick	Quick3
Random								
Nearly Sorted								
Reversed								
Few Unique								

KNOWLEDGEGATE

School Morning Assembly

KNOWLEDGE GATE

Sorting

- The process of arranging data items (numeric or char) in a specific order either increasing or decreasing order is called sorting. It is an important process which is used in a number of applications as many times we require sorted data.

- There are number of approaches available for sorting and some parameter based on which we judge the performance of these algorithm.

KNOWLEDGEGATE

Sorting Algorithm	Best Case	Worst Case
Selection	$O(n^2)$	$O(n^2)$
Bubble	$O(n^2)$ / $O(n)$	$O(n^2)$
Insertion	$O(n)$	$O(n^2)$
Merge	$O(n\log n)$	$O(n\log n)$
Heap	$O(n\log n)$	$O(n\log n)$
Quick	$O(n\log n)$	$O(n^2)$

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Space complexity**: - Internal sorting and External sorting
- **Internal/Inplace sorting** means when a sorting algorithm does not require any additional memory apart from the space acquired by the problem. Eg Heap Sort
- While on the other and some sorting algo requires additional space called **external sorting** algorithm. Eg Merge Sort

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Stable or Unstable**: - When the input sequence contains copies then we check whether the output sequence preserve the order of the respective copies or not?
  - Stable(Bubble Sort), Unstable(Insertion Sort)
- Even studying all the criterion, we understand that the different sorting algo have different advantages in different scenarios therefor we must understand the actual logic of the algorithm and then can solve problems based on them.

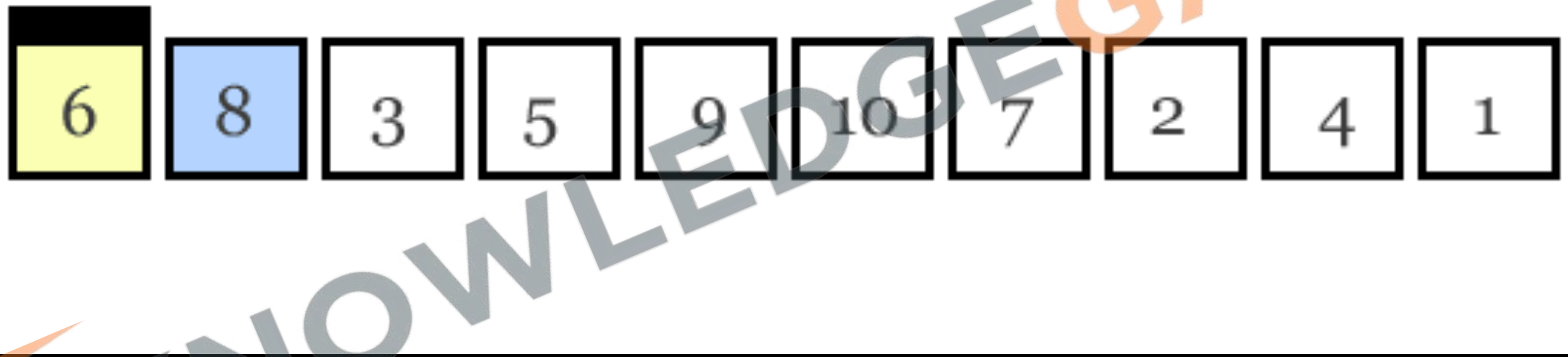
KNOWLEDGEGATE

Selection Sort

- The algorithm divides the input list into two parts: a sorted sublist of items which is built up from left to right at the front (left) of the list and a sublist of the remaining unsorted items that occupy the rest of the list.

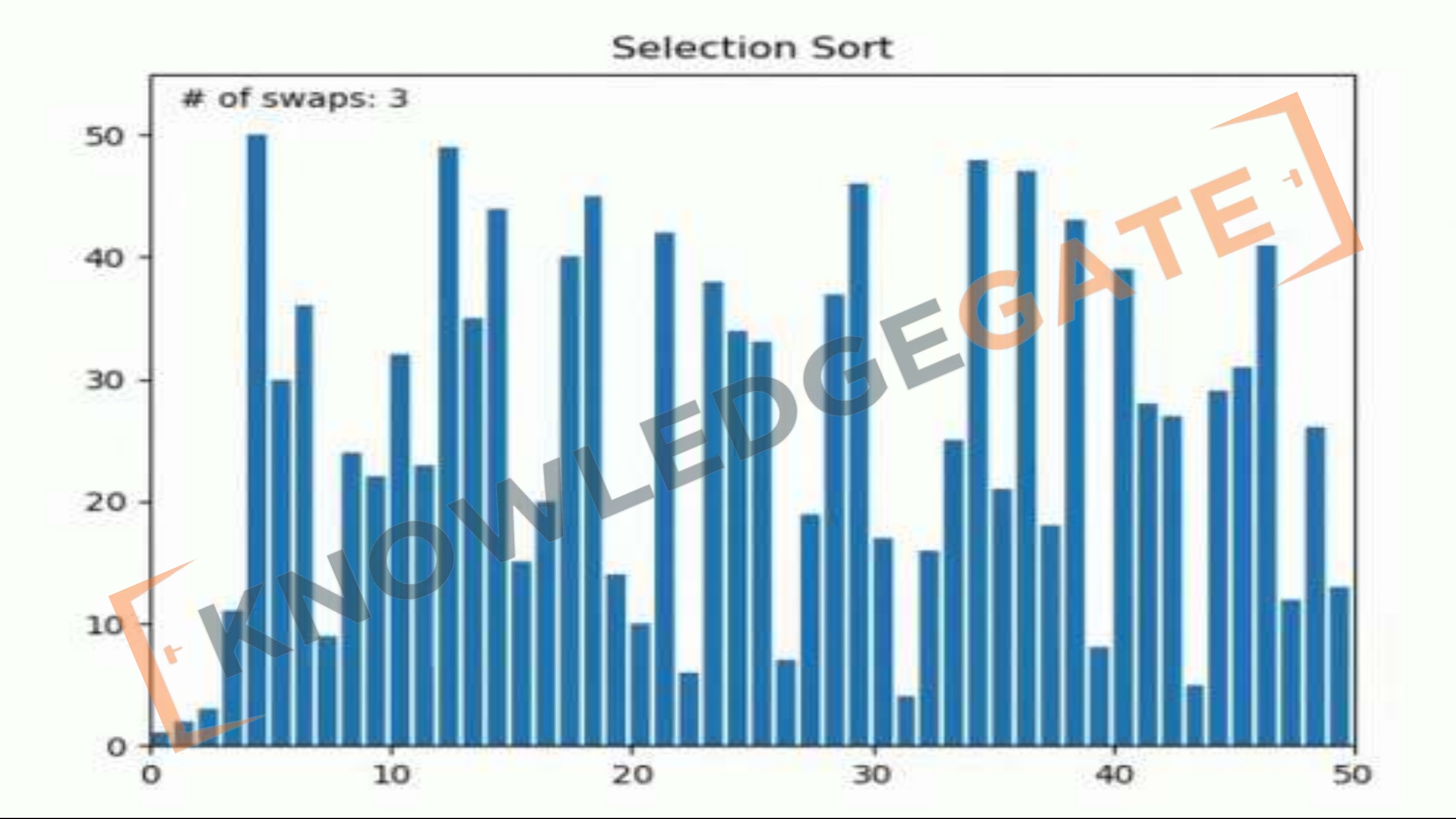
KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- Initially, the sorted sublist is empty and the unsorted sublist is the entire input list. The algorithm proceeds by finding the smallest (or largest, depending on sorting order) element in the unsorted sublist, exchanging (swapping) it with the leftmost unsorted element (putting it in sorted order), and moving the sublist boundaries one element to the right.



KNOWLEDGEGATE

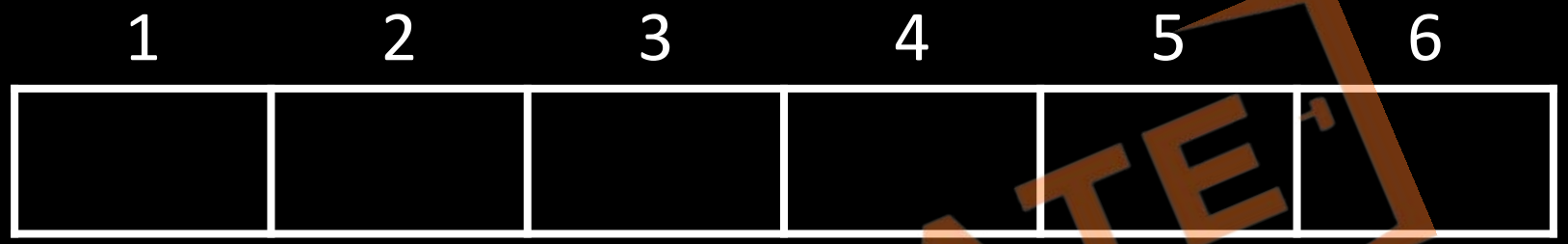
http://www.knowledgegate.in/GATE



KNOWLEDGEGATE

Selection Sort(Algo)

```
Selection sort (A, n)
{
    for k ←1 to n-1
    {
        min = A[k]
        Loc = k
        for j ←k+1 to n
        {
            if(min > A[j])
            {
                min = A[j]
                Loc = j
            }
        }
        swap(A[k],A[Loc])
    }
}
```



KNOWLEDGEGATE

Selection Sort(Analysis)

```
Selection sort (A, n)
{
    for k ←1 to n-1
    {
        min = A[k]
        Loc = k
        for j ←k+1 to n
        {
            if(min > A[j])
            {
                min = A[j]
                Loc = j
            }
        }
        swap(A[k],A[Loc])
    }
}
```

- Depends on structure or content ?
  - Structure
- Internal/External sort algorithm ?
  - Internal
- Stable/Unstable sort algorithm ?
  - Unstable
- Best case time complexity ?
  - $O(n^2)$
- Worst case time complexity ?
  - $O(n^2)$
- Algorithmic Approach?
  - Subtract and Conquer

KNOWLEDGE GATE

Selection Sort (Conclusion)

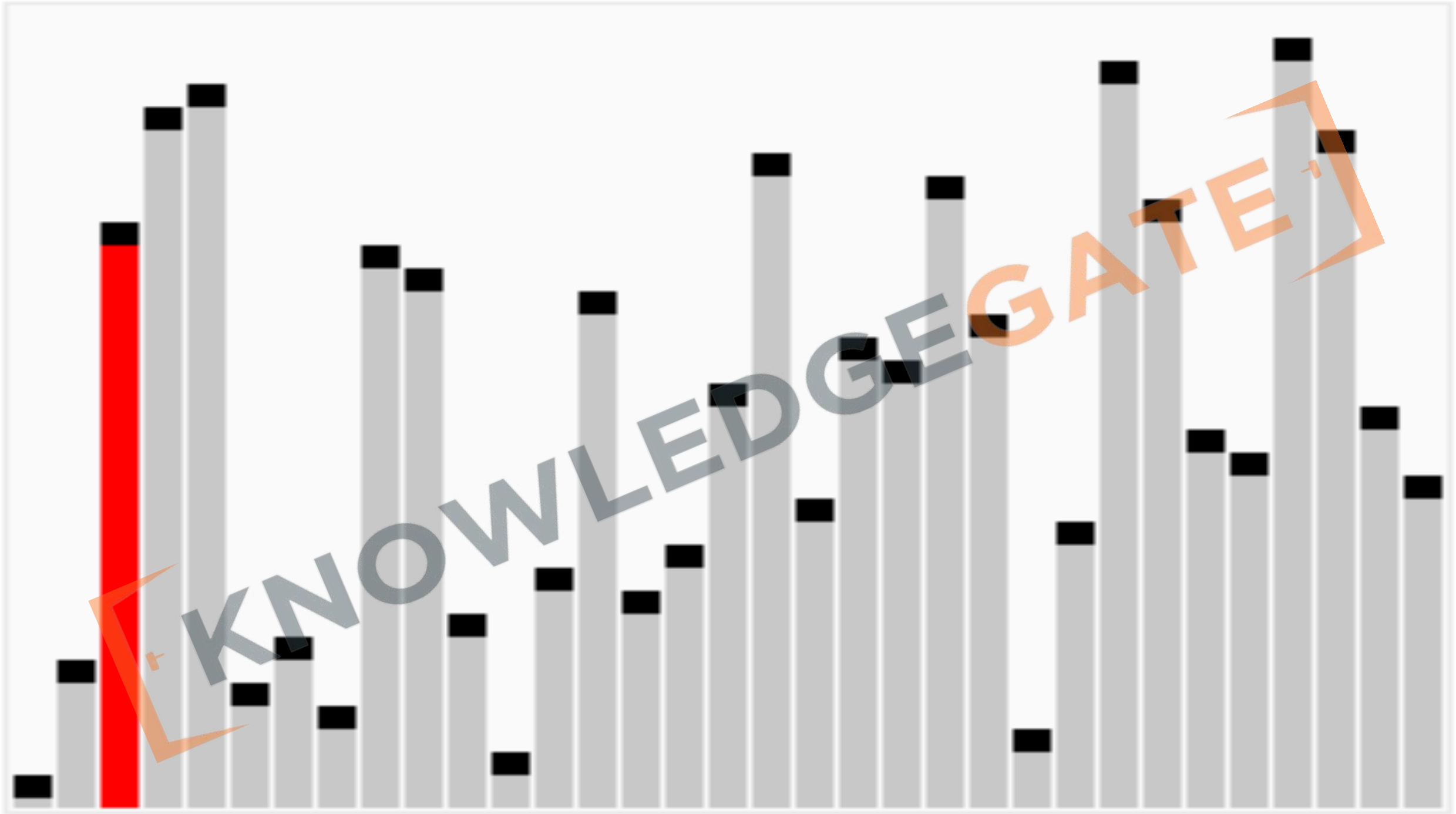
- Selection sort is noted for its simplicity and has performance advantages over more complicated algorithms in certain situations(number of swaps, which is $n - 1$ in the worst case).
- It has an $O(n^2)$ time complexity, which makes it inefficient on large lists.

KNOWLEDGEGATE

Bubble / Shell / Sinking Sort

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

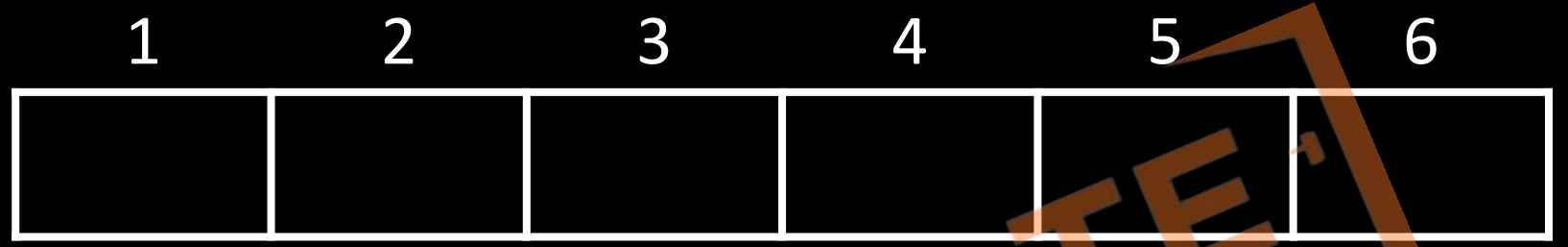

KNOWLEDGEGATE

Bubble / Shell / Sinking Sort

- Bubble sort, sometimes referred to as sinking sort, is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted. The algorithm, which is a comparison sort, is named for the way smaller or larger elements "bubble" to the top of the list.

KNOWLEDGEGATE

Bubble / Shell / Sinking Sort(Algo without flag)



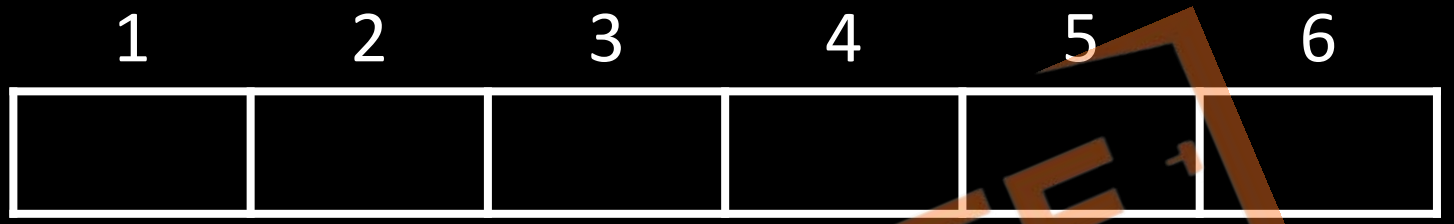

```
Bubble sort (A, n)
{
    for k ←1 to n-1
    {
        ptr = 1
        while(ptr <= n-k)
        {
            if(A[ptr] > A[ptr+1])
            {
                exchange(A[ptr],A[ptr+1])
            }
            ptr = ptr+1
        }
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Bubble / Shell / Sinking Sort (Algo with flag)

```
Bubble sort (A, n)
{
        for k ←1 to n-1
        {
                ptr = 1
                while(ptr <= n-k)
                {
                        if(A[ptr] > A[ptr+1])
                        {
                                exchange(A[ptr],A[ptr+1])
                        }
                        ptr = ptr+1
                }
        }
}
```




```
Bubble sort (A, n)
{
        for k ←1 to n-1
        {
                ptr = 1
                while(ptr <= n-k)
                {
                        if(A[ptr] > A[ptr+1])
                        {
                                exchange(A[ptr],A[ptr+1])
                                flag = 1
                        }
                        ptr = ptr+1
                }
                        if(!flag)
                        {
                        break;
                        }
        }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Bubble / Shell / Sinking Sort (Analysis with flag)

```
Bubble sort (A, n)
{
    for k ←1 to n-1
    {
        ptr = 1
        while(ptr <= n-k)
        {
            if(A[ptr] > A[ptr+1])
            {
                exchange(A[ptr],A[ptr+1])
                flag = 1
            }
            ptr = ptr+1
        }
        if(!flag)
        {
            break;
        }

    }
}
```

- Depends on structure or content ?
  - Both
- Internal/External sort algorithm ?
  - Internal
- Stable/Unstable sort algorithm ?
  - Stable
- Best case time complexity ?
  - O(n)
- Worst case time complexity ?
  - $O(n^2)$
- Algorithmic Approach?
  - Subtract and Conquer

[KNOWLEDGEGATE]

Bubble / Shell / Sinking Sort (Conclusion)

- Even other $O(n^2)$ sorting algorithms, such as insertion sort selection sort, generally run faster than bubble sort, and are no more complex. Therefore, bubble sort is not a practical sorting algorithm. This simple algorithm performs poorly in real world use and is used primarily as an educational tool. More efficient algorithms such as heap sort, or merge sort are used by the sorting libraries built into popular programming languages such as Python and Java.

- When the list is already sorted (best-case), the complexity of bubble sort is only $O(n)$. By contrast, most other algorithms, even those with better average-case complexity, perform their entire sorting process on the set and thus are more complex. However, not only does insertion sort share this advantage, but it also performs better on a list that is substantially sorted (having a small number of inversions).

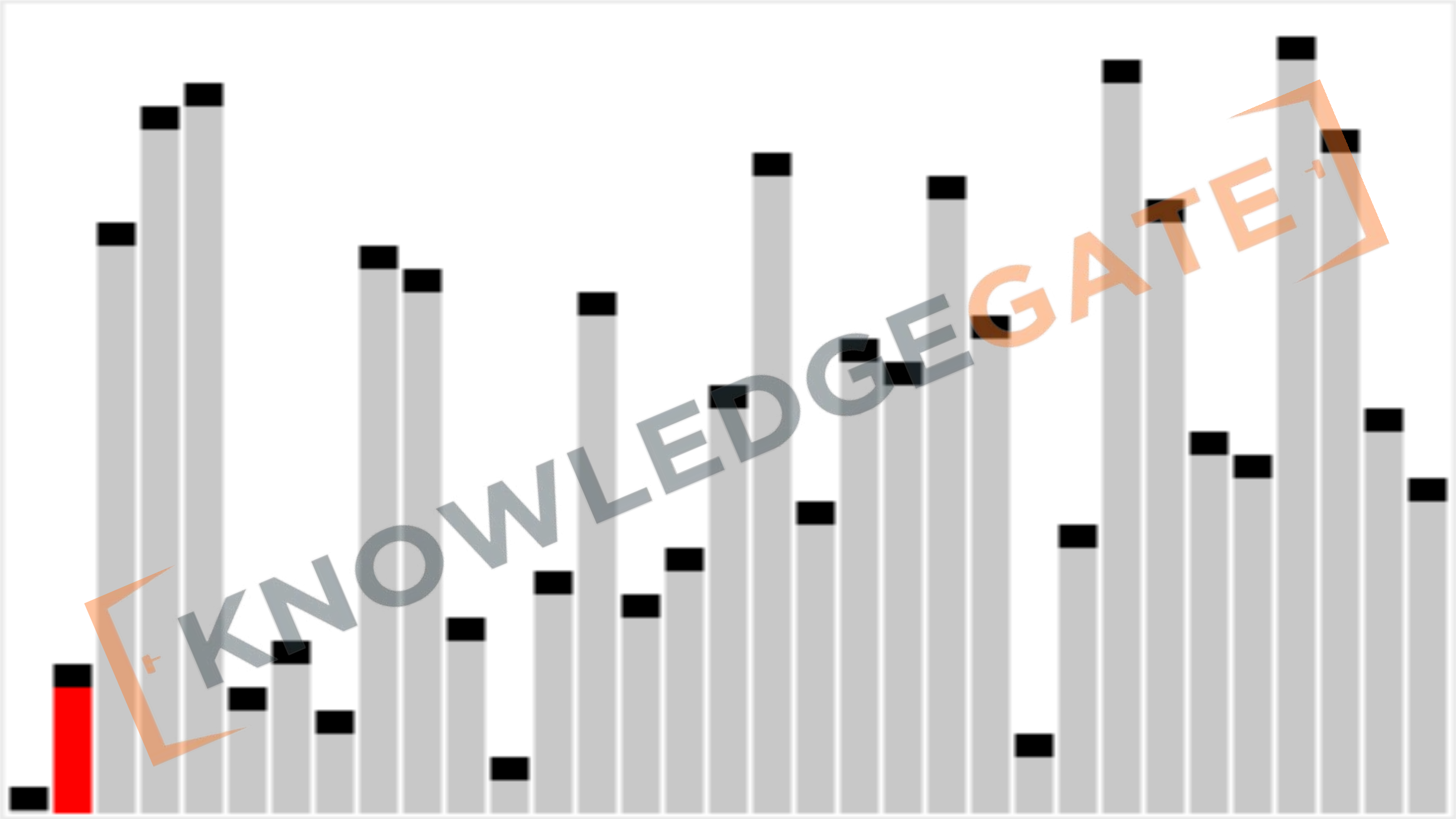
KNOWLEDGEGATE

KNOWLEDGEGATE
http://www.KnowledgeGate.in/GATE

Insertion Sort

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

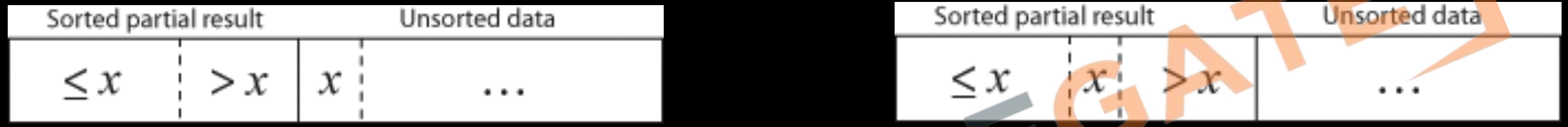

KNOWLEDGEGATE

Insertion Sort

- At each iteration, insertion sort removes one element from the input data, finds the location it belongs within the sorted list, and inserts it there. It repeats until no input elements remain.
- At each array-position, it checks the value there against the largest value in the sorted list (which happens to be next to it, in the previous array-position checked).
- If larger, it leaves the element in place and moves to the next. If smaller, it finds the correct position within the sorted list, shifts all the larger values up to make a space, and inserts into that correct position.

KNOWLEDGE GATE

- The resulting array after *k* iterations has the property where the first *k* + 1 entries are sorted ("+1" because the first entry is skipped). In each iteration the first remaining entry of the input is removed, and inserted into the result at the correct position, thus extending the result:

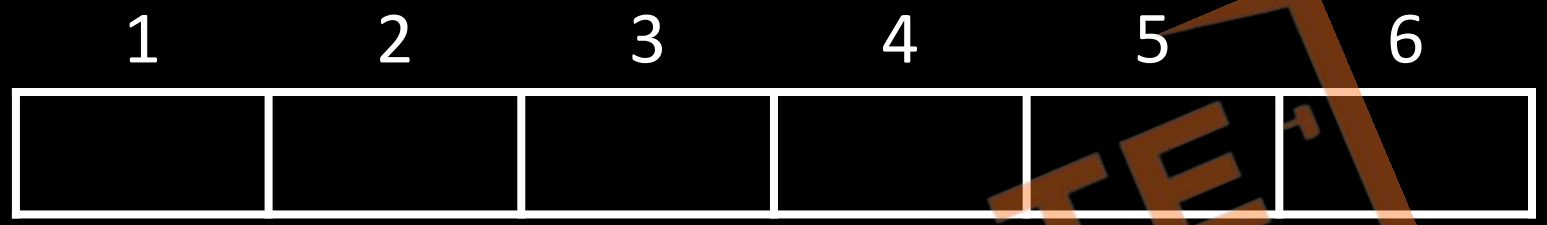


KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Insertion Sort (Algo)

```
Insertion sort (A, n)
{
    for j ←2 to n
    {
        key = A[j]
        i = j - 1
        while(i>0 and A[i] > key)
        {
            A[i+1] = A[i]
            i = i-1
        }
        A[i+1]=key
    }
}
```



KNOWLEDGEGATE

Insertion Sort (Analysis)

```
Insertion sort (A, n)
{
    for j ←2 to n
    {
        key = A[j]
        i = j - 1
        while(i>0 and A[i] > key)
        {
            A[i+1] = A[i]
            i = i-1
        }
        A[i+1]=key
    }
}
```

- Depends on structure or content ?
  - Both
- Internal/External sort algorithm ?
  - Internal
- Stable/Unstable sort algorithm ?
  - Stable
- Best case time complexity ?
  - O(n)
- Worst case time complexity ?
  - $O(n^2)$
- Algorithmic Approach?
  - Subtract and Conquer

KNOWLEDGEGATE

Insertion Sort (Conclusion)

- **Insertion sort** is much less efficient on large lists than more advanced algorithms such as heapsort(O(nlogn)), or merge sort (O(nlogn)). However, insertion sort provides several advantages:
  - Efficient for (quite) small data sets, much better other quadratic sorting algorithms such as selection and bubble sort.

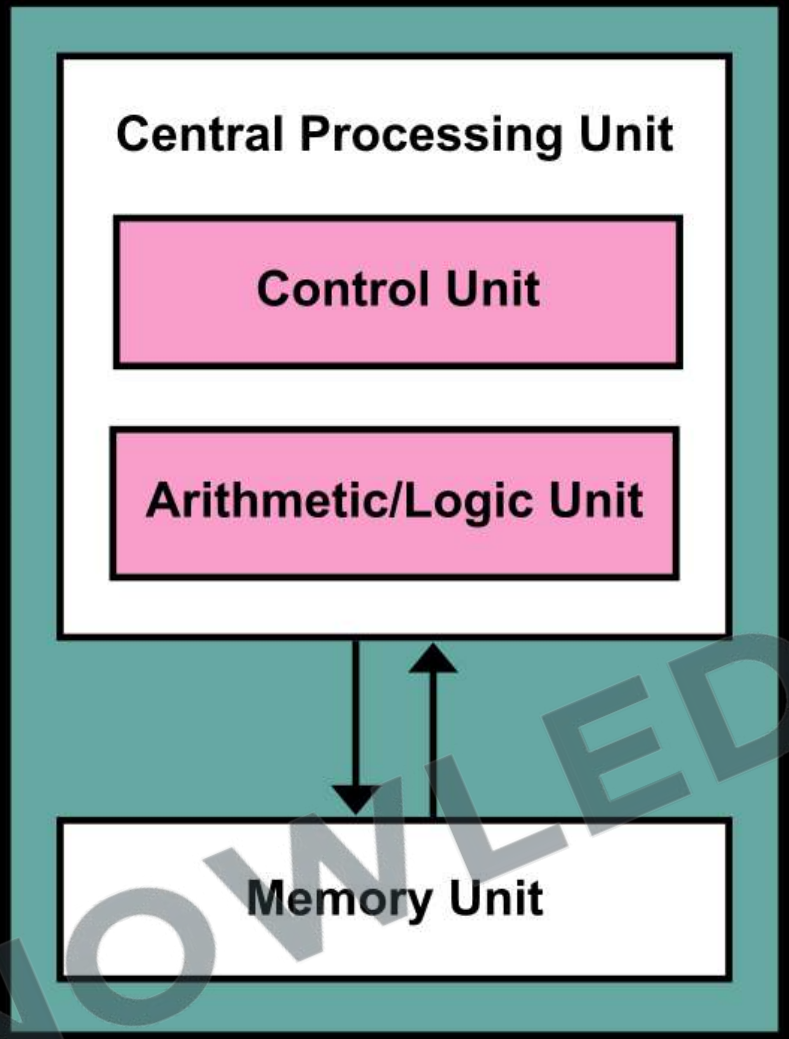
KNOWLEDGEGATE

Merge Sort

- In computer science, merge sort is an efficient, general-purpose, comparison-based sorting algorithm.
- Merge sort is a divide and conquer algorithm that was invented by John von Neumann in 1945.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE



KNOWLEDGEGATE

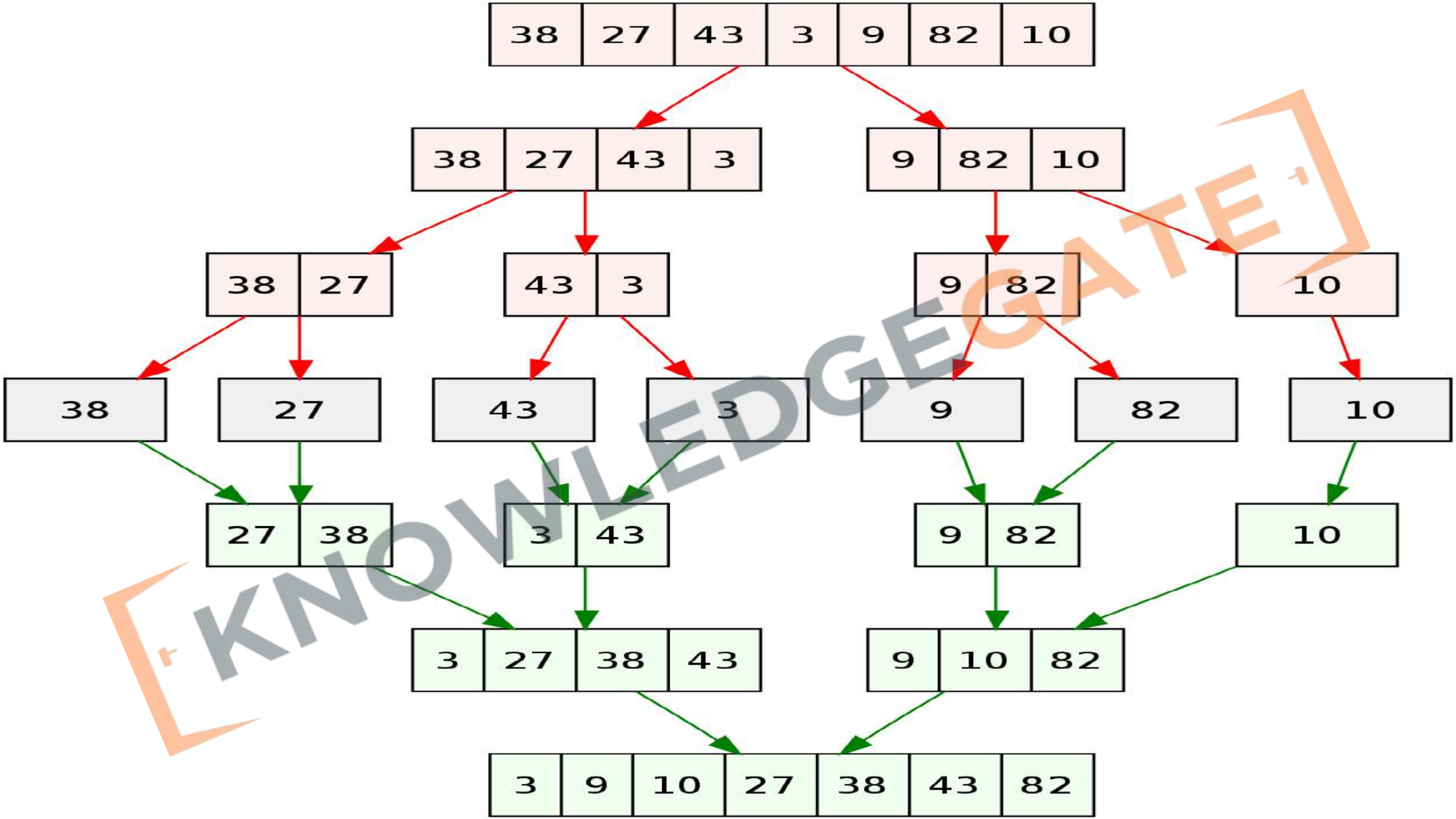
Von Neumann Architecture

http://www.knowledgegate.in/GATE

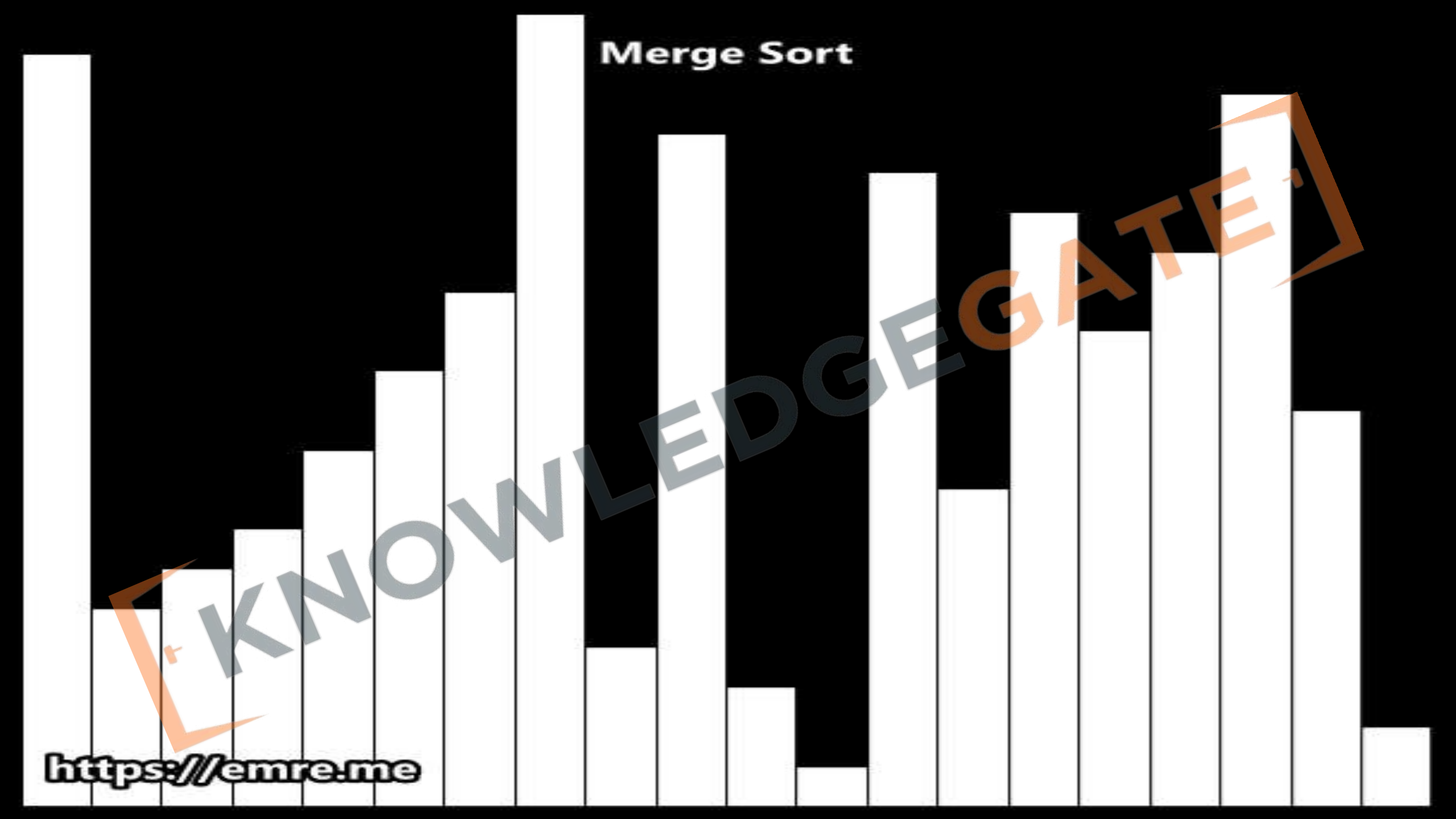
Merge Sort

- Conceptually, a merge sort works as follows:
  - Divide the unsorted list into *n* sublists, each containing one element (a list of one element is considered sorted).
  - Repeatedly merge sublists to produce new sorted sublists until there is only one sublist remaining. This will be the sorted list.

KNOWLEDGEGATE


38 27 43 3 9 82 10
38 27 43 3
9 82 10
38 27
43 3
9 82
10
38
27
43
3
9
82
10
27 38
3 43
9 82
10
3 27 38 43
9 10 82
3 9 10 27 38 43 82
KNOWLEDGEGATE

KNOWLEDGEGATE

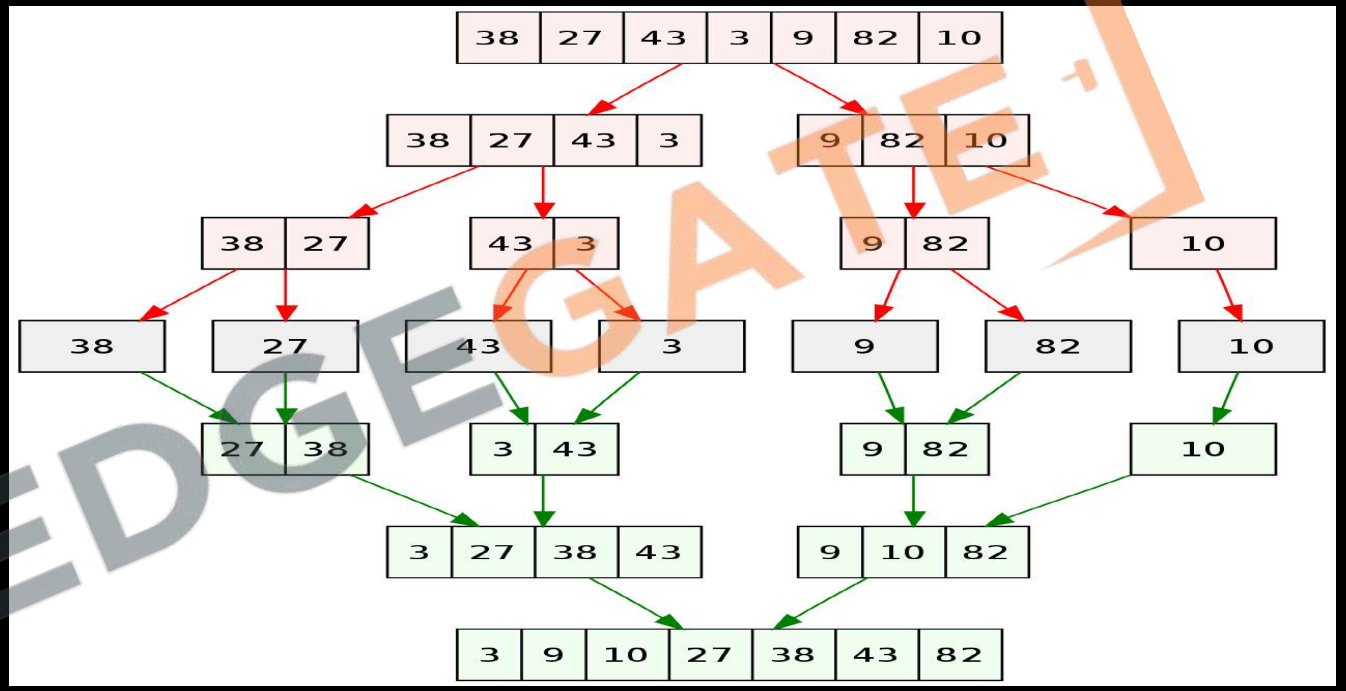


KNOWLEDGEGATE

KNOWLEDGEGATE

Merge Sort(Algo)

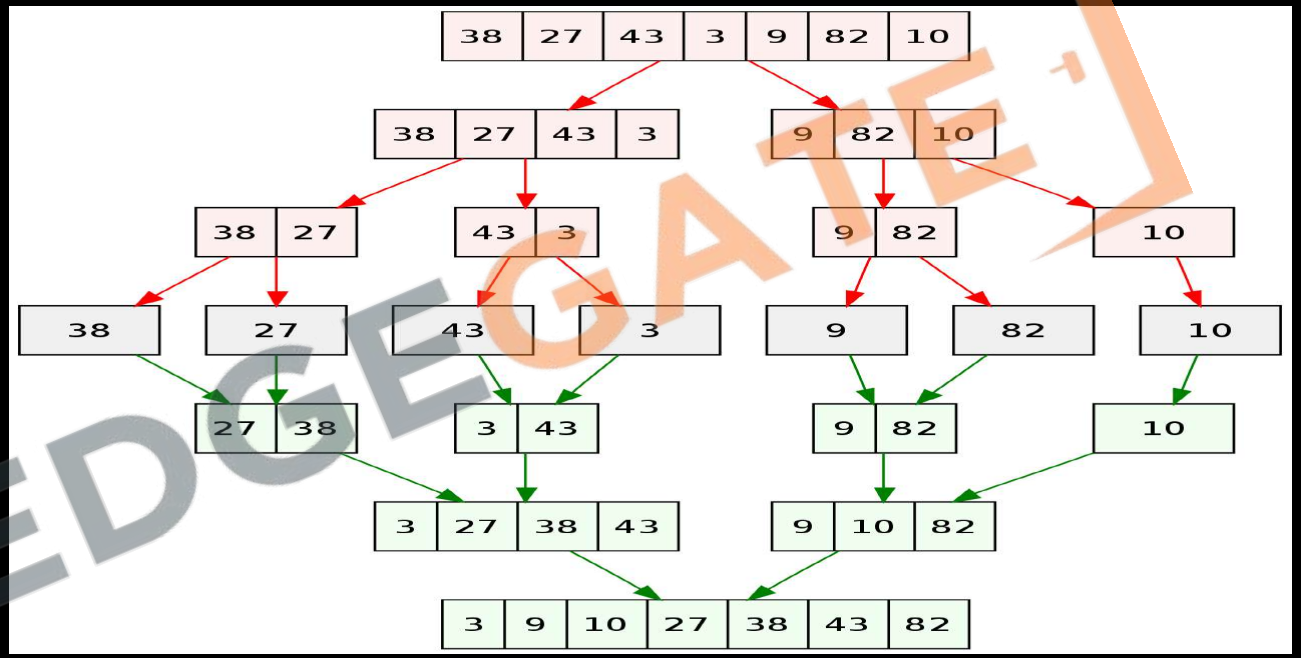
```
Merge_Sort(A, p, r)
{
    if(p < r)
    {
        q ← ⌊ (p + r)/2 ⌋
        Merge_Sort (A, p, q)
        Merge_Sort (A, q + 1, r)
        Merge (A, p, q, r)
    }
}
```



KNOWLEDGEGATE

Merge Sort(Algo)

```
Merge (A, p, q, r)
{
    n1 ← q – p + 1
    n2 ← r – q
    Create array L [1…… n1+1] and R [1…… n2+1]
    for i ← 1 to n1
        do L[i] = A [p + i - 1]
    for j ← 1 to n2
        do R[j] = A [j + q]
    L [n1+1] ← ∞
    R [n2+1] ← ∞
    i ← 1
    j ← 1
    for k ← p to r
    {
        if(L[i] <= R[j])
        {
            A[k] = L[i]
            i = i + 1
        }
        Else
        {
            A[k] = R[j]
            j = j + 1
        }
    }
}
```



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Merge Sort(Analysis)

- Depends on structure or content ?
  - Structure
- Internal/External sort algorithm ?
  - External
- Stable/Unstable sort algorithm ?
  - Stable
- Best case time complexity ?
  - O(nlogn)
- Worst case time complexity ?
  - O(nlogn)
- Algorithmic Approach?
  - Divide and Conquer

KNOWLEDGEGATE

Merge Sort(Conclusion)

- If the running time of merge sort for a list of length n is $T(n)$, then the recurrence
  - $T(n) = 2T(n/2) + n$

- In the *worst* case, merge sort does about 39% fewer comparisons than quicksort does in the *average* case.

- Merge sort is more efficient than quicksort for some types of lists if the data to be sorted can only be efficiently accessed sequentially, and is thus popular in languages such as Lisp, where sequentially accessed data structures are very common.

- Merge sort's requires O(n) space complexity.

KNOWLEDGEGATE

Heap Sort

- Heapsort was invented by J. W. J. Williams in 1964. This was also the birth of the heap, presented already by Williams as a useful data structure in its own right.

KNOWLEDGEGATE

http: GATE

Heap Sort

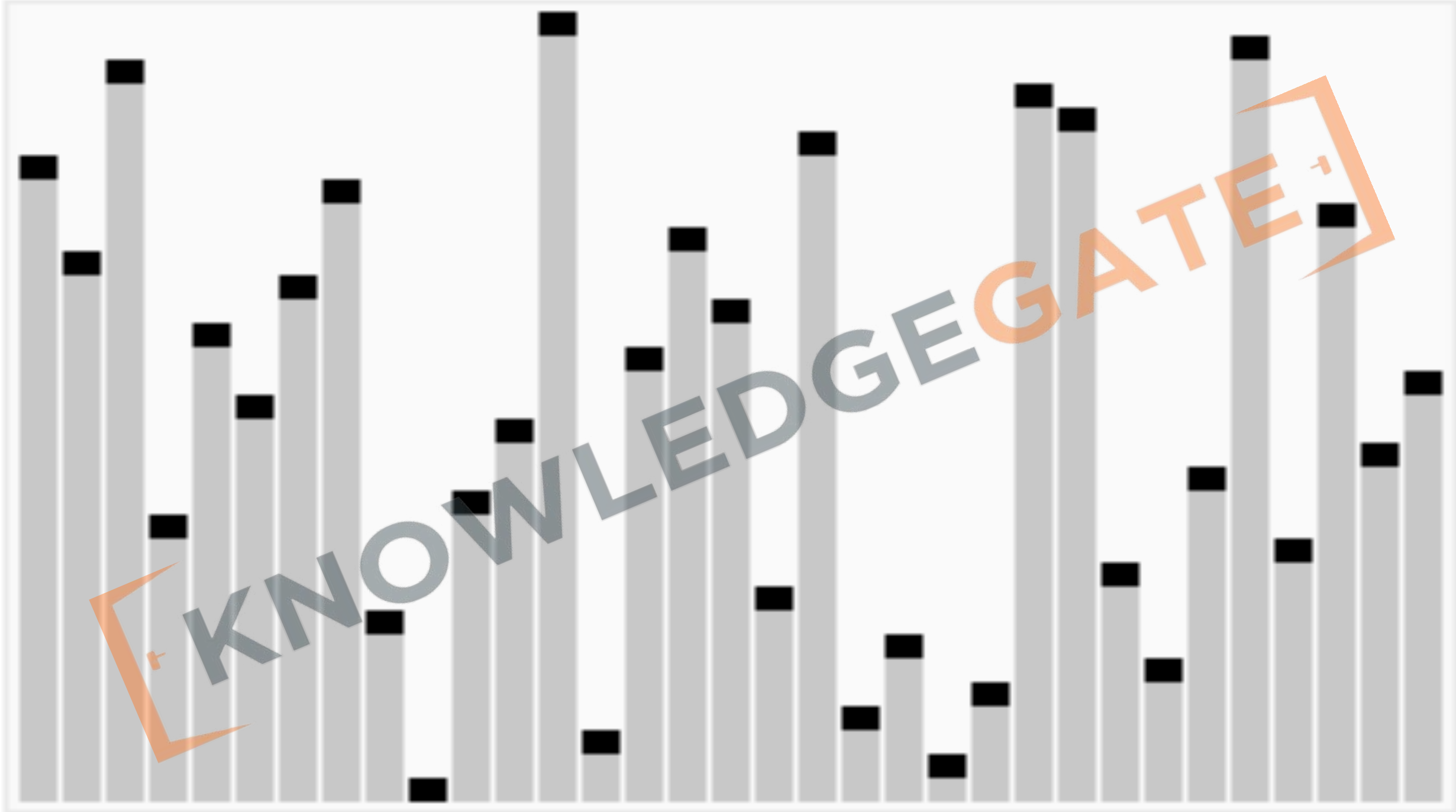
- In computer science, heapsort is a comparison-based sorting algorithm.
- Heapsort divides its input into a sorted and an unsorted region, and it iteratively shrinks the unsorted region by extracting the largest element from it and inserting it into the sorted region.
- Heapsort does not waste time with a linear-time scan of the unsorted region; rather, heap sort maintains the unsorted region in a heap data structure to more quickly find the largest element in each step.

KNOWLEDGEGATE

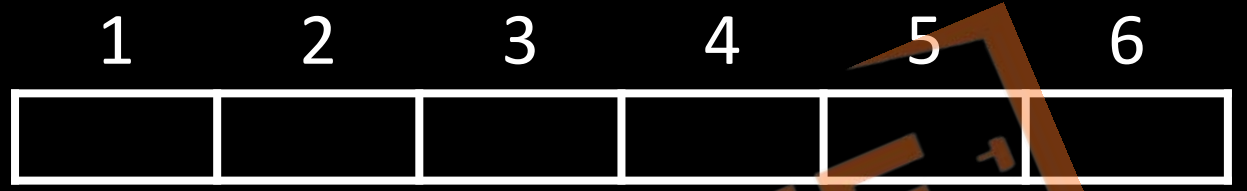
- The heapsort algorithm can be divided into two parts.
  - In the first step, a heap is built out of the data. The heap is often placed in an array with the layout of a complete binary tree.
  - In the second step, a sorted array is created by repeatedly removing the largest element from the heap (the root of the heap), and inserting it into the array. The heap is updated after each removal to maintain the heap property. Once all objects have been removed from the heap, the result is a sorted array.

KNOWLEDGE GATE

http://www.knowledgegate.in/GATE


KNOWLEDGEGATE

Heap Sort(Algo)

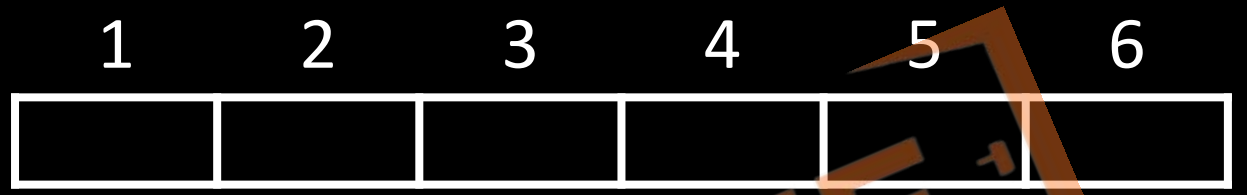



```
Heap_Sort(A)
{
    Build_Max_heap(A)
    for i ← length[A] down to 2
    {
        do exchange (A[1] ←→A[i])
        Heap-size[A] ← Heap-size[A] – 1
        Max-Heapify(A,1)
    }
}
```

KNOWLEDGEGATE

Heap Sort(Algo)

```
Build_Max_Heap(A)
{
    Heap-size[A] ← length[A]
    for i ← ⌊ length[A]/2 ⌋ down to 1
    {
        do Max-Heapify (A, i)
    }
}
```

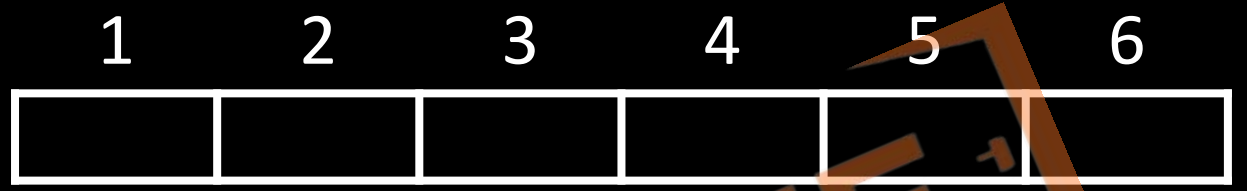


KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Heap Sort(Algo)

```
Max-Heapify(A, i)
{
    L ← Left[i]
    R ← Right[i]
    if( L <= Heap_size[A] and A[L] > A[i])
        Largest ← L
    Else
        Largest ← i
    if(R <= Heap_size[A] and A[r] > A[Largest])
        Largest ← R
    if(Largest != i)
    {
        Exchange( A[i] ←→A[Largest])
        Max-Heapify(A, Largest)
    }
}
```



KNOWLEDGEGATE

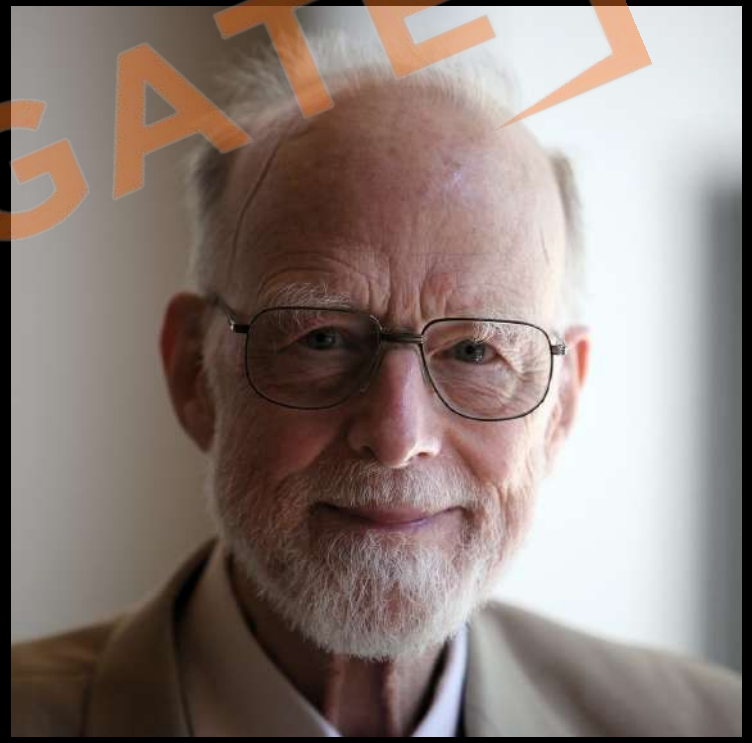
Heap Sort(Analysis)

- Depends on structure or content ?
  - Both
- Internal/External sort algorithm ?
  - Internal
- Stable/Unstable sort algorithm ?
  - Unstable
- Best case time complexity ?
  - O(nlogn)
- Worst case time complexity ?
  - O(nlogn)
- Algorithmic Approach?
  - Mixed Approach

KNOWLEDGEGATE

Quick Sort

- **Quicksort** is an developed by British computer scientist Tony Hoare and published in 1961, it is still a commonly used algorithm for sorting. When implemented well, it can be somewhat faster than merge sort and about two or three times faster than heapsort.
- His work earned him the Turing Award, usually regarded as the highest distinction in computer science, in 1980.
- Along with Edsger Dijkstra, formulated the dining philosophers problem. He is also credited with development of the null pointer.



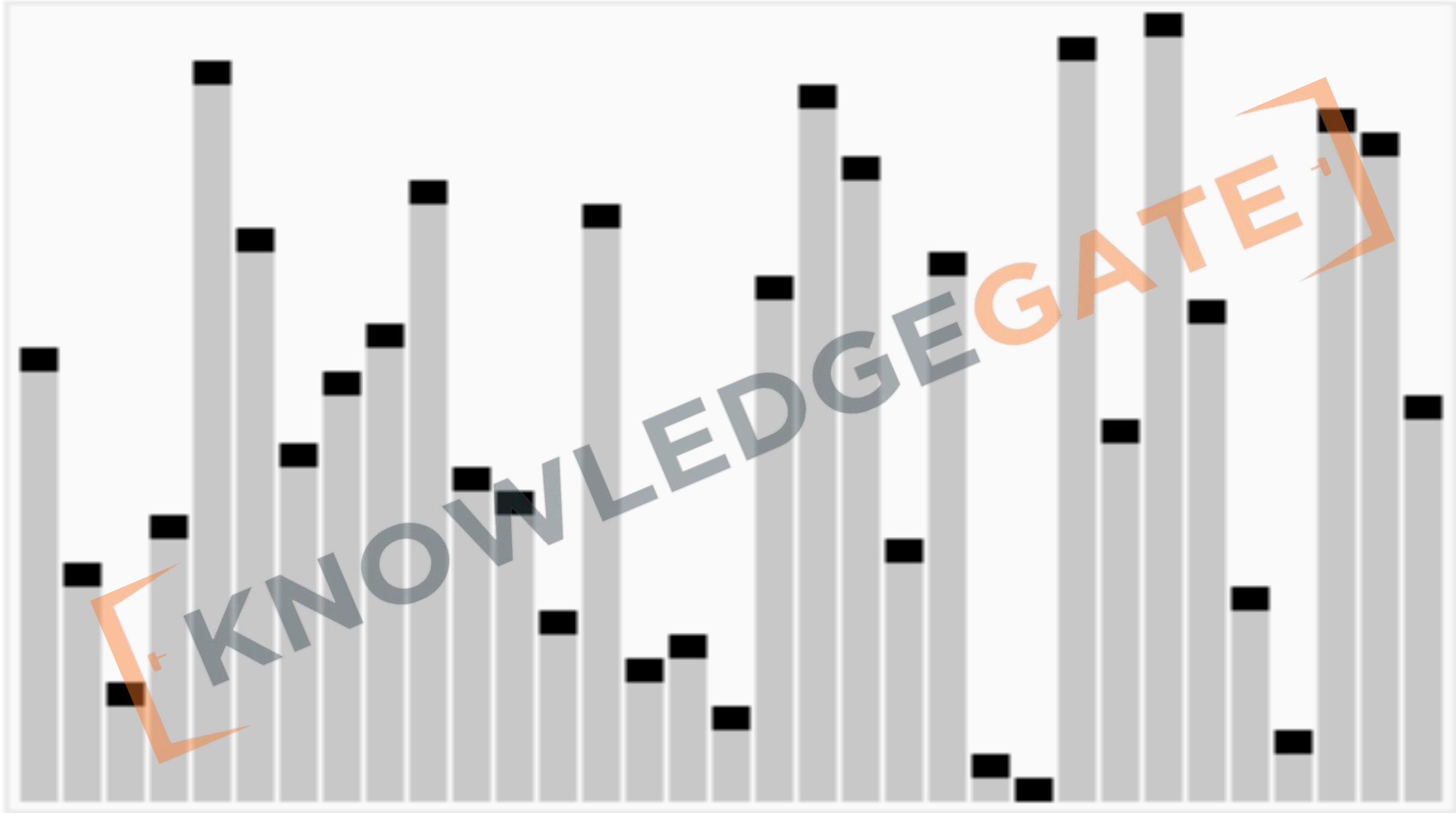
KNOWLEDGEGATE

- Quicksort is a divide-and-conquer algorithm. It works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot.

- The sub-arrays are then sorted recursively. This can be done in-place, requiring small additional amounts of memory to perform the sorting.

[KNOWLEDGEGATE]

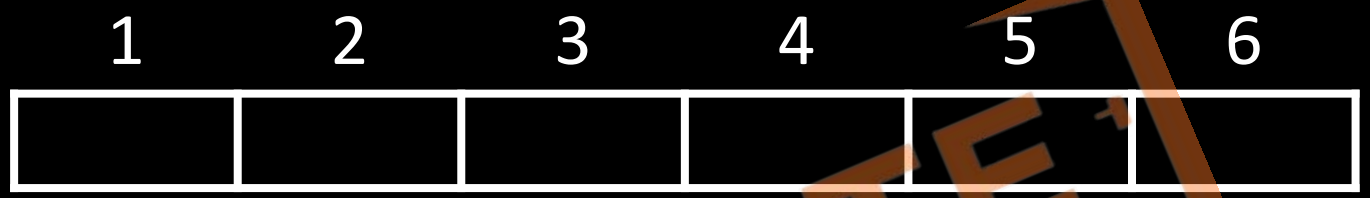
http://www.knowledgegate.in/GATE


KNOWLEDGEGATE

KNOWLEDGEGATE

Quick Sort(Algo)

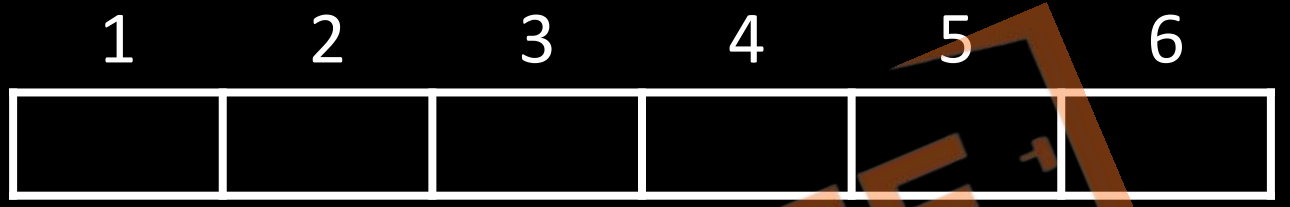
```
Quick_Sort(A, p, r)
{
    if(p < r)
    {
        q ← partition (A, p, r)
        quick_Sort(A, p, q - 1)
        quick_Sort(A, q + 1, r)
    }
}
```



KNOWLEDGEGATE

Quick Sort(Algo)

```
Partition (A, p, r)
{
    x ← A[r]
    i ← p – 1
    for j ← p to r – 1
    {
        if(A[j] <= x)
        {
            i ← i + 1
            Exchange( A[i] ←→A[j])
        }
    }
    Exchange( A[i + 1] ←→A[r])
    return i+1
}
```



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Quick Sort(Analysis)

- Depends on structure or content ?
  - Both
- Internal/External sort algorithm ?
  - Internal
- Stable/Unstable sort algorithm ?
  - Unstable
- Best case time complexity ?
  - O(nlogn)
- Worst case time complexity ?
  - $O(n^2)$
- Algorithmic Approach?
  - Divide and Conquer

KNOWLEDGEGATE

Counting Sort

- Although radix sorting itself dates back far longer, counting sort, and its application to radix sorting, were both invented by Harold H. Seward in 1954.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Counting Sort

KNOWLEDGEGATE

Counting Sort

- Counting sort is a non-comparative stable sorting algorithm suitable for sorting elements within a specific range. It counts the number of objects that have distinct key values, and then does some arithmetic to calculate the position of each object in the output sequence.
- Counting sort is efficient if the range of the input data is not significantly greater than the number of objects to be sorted. It's not a comparison-based sort, and its time complexity is O(n+k), where n is the number of elements in the input array and k is the range of the input. Space complexity is O(k).

KNOWLEDGEGATE

Counting Sort

```
Counting_Sort(A, B, k)
{
    let C[0..k] be a new array
    For I ← 0 to k
        do C[i] ← 0
    for j ← 1 to length[A]
        do C[A[j]] ← C[A[j]] + 1
    for i ← 1tok
        do C[i] ← C[i] + C[i–1]
    for j ← length[A] down to 1
        do B[C[A[j]]] ← A[j]
        C[A[j]] ← C[A[j]] – 1
}
```

1	2	3	4	5	6	7	8
7	4	6	1	3	1	3	6

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Given array : Time complexity of counting sort is O(n).

	1	2	3	4	5	6	7	8	9	10
A	1	6	3	3	4	5	6	3	4	5

Step 1 : $i = 0$ to 6 $k = 6$ (largest element in array A)

$C[i] \leftarrow 0$

	0	1	2	3	4	5	6
C	0	0	0	0	0	0	0

Step 2 : $j = 1$ to 10 ($\because$ length [A] = 10)

$C[A[j]] \leftarrow C[A[j]] + 1$

For $j = 1$

$C[A[1]] \leftarrow C[1] + 1 = 0 + 1 = 1$

	0	1	2	3	4	5	6
C	0	1	0	0	0	0	0

$C[1] \leftarrow 1$

For $j = 2$

$C[A[2]] \leftarrow C[6] + 1 = 0 + 1 = 1$

	0	1	2	3	4	5	6
C	0	1	0	0	0	0	1

$C[6] \leftarrow 1$

Similarly for $j = 5, 6, 7, 8, 9, 10$

	0	1	2	3	4	5	6
C	0	1	0	3	2	2	2

Step 3 :

For $i = 1$ to 6

$C[i] \leftarrow C[i] + C[i-1]$

For $i = 1$

$C[1] \leftarrow C[1] + C[0]$

	0	1	2	3	4	5	6
C	0	1	0	3	2	2	2

$C[1] \leftarrow 1 + 0 = 1$

For $i = 2$

$C[2] \leftarrow C[2] + C[1]$

	0	1	2	3	4	5	6
C	0	1	1	3	2	2	2

$C[1] \leftarrow 1 + 0 = 1$

Similarly for $i = 4, 5, 6$

	0	1	2	3	4	5	6
C	0	1	1	4	6	8	10

Step 4 :

For $j = 10$ to 1

$B[C[A[j]]] \leftarrow A[j]$

$C[A[j]] \leftarrow C[A[j]] - 1$

j	$A[j]$	$C[A[j]]$	$B[C[A[j]]] \leftarrow A[j]$	$C[A[j]] \leftarrow C[A[j]] - 1$
10	5	8	$B[8] \leftarrow 5$	$C[5] \leftarrow 7$
9	4	6	$B[6] \leftarrow 4$	$C[4] \leftarrow 5$
8	3	4	$B[4] \leftarrow 3$	$C[3] \leftarrow 3$
7	6	10	$B[10] \leftarrow 6$	$C[6] \leftarrow 9$
6	5	7	$B[7] \leftarrow 5$	$C[5] \leftarrow 6$
5	4	5	$B[5] \leftarrow 4$	$C[4] \leftarrow 4$
4	3	3	$B[3] \leftarrow 3$	$C[3] \leftarrow 2$
3	3	2	$B[2] \leftarrow 3$	$C[3] \leftarrow 1$
2	6	9	$B[9] \leftarrow 6$	$C[6] \leftarrow 8$
1	1	1	$B[1] \leftarrow 1$	$C[1] \leftarrow 0$

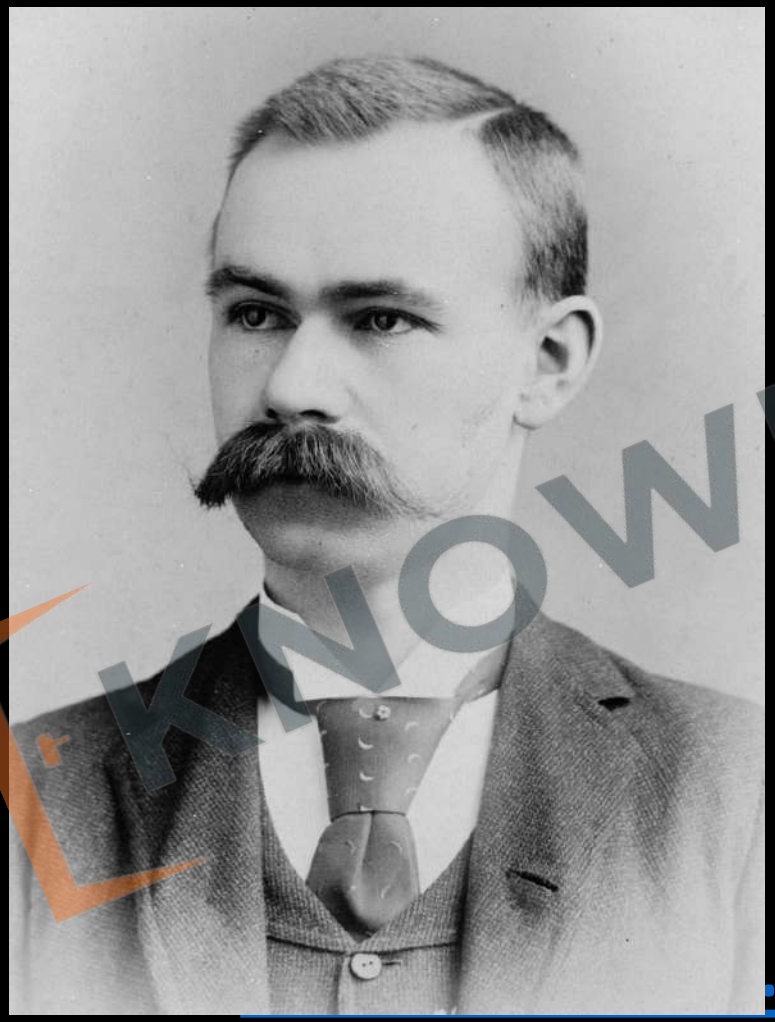
	1	2	3	4	5	6	7	8	9	10
B	1	3	3	3	4	4	5	5	6	6

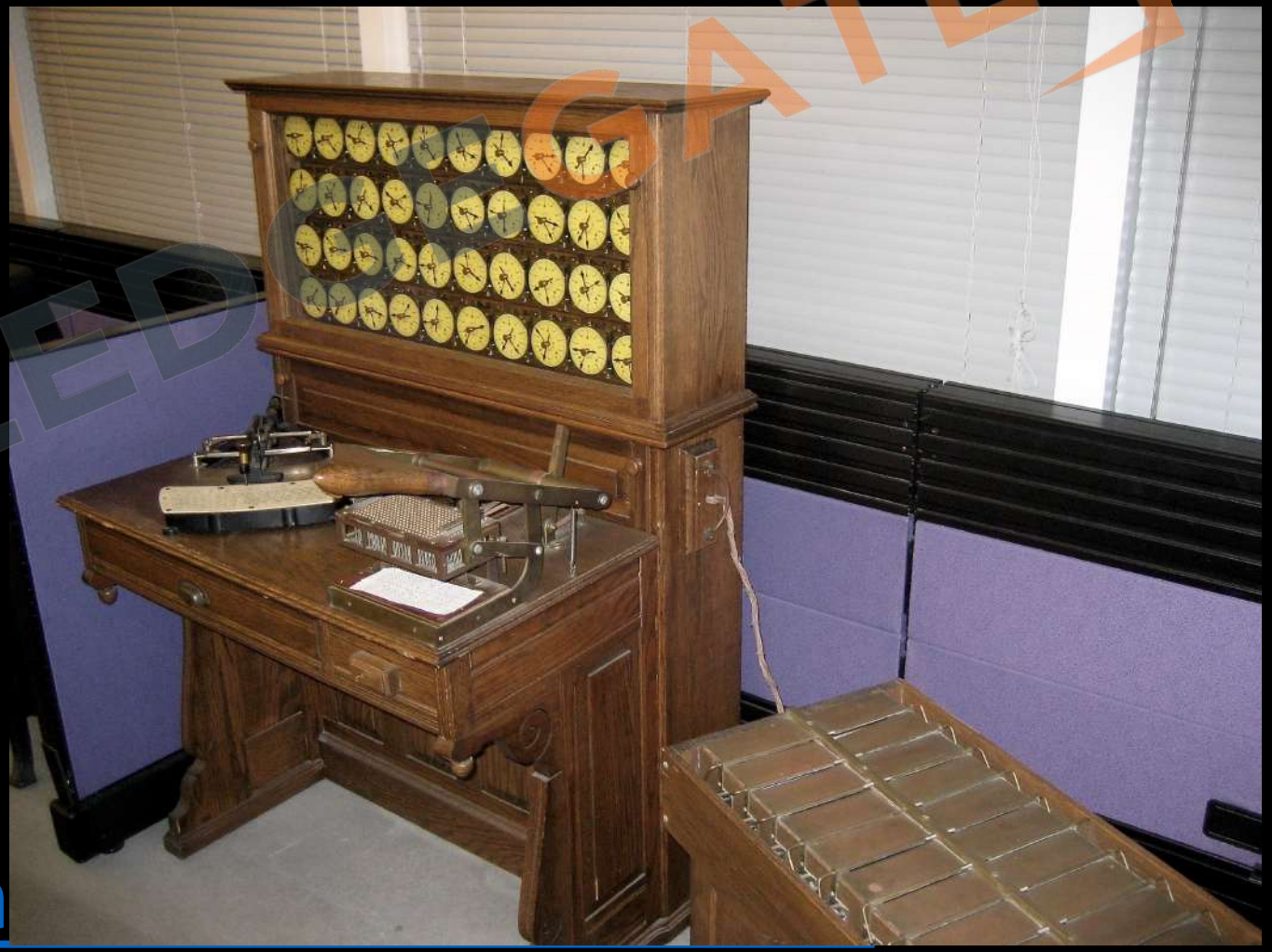
KNOWLEDGEGATE

ledgegate.in/GATE

Radix Sort

- Radix sort dates back as far as 1887 to the work of Herman Hollerith on tabulating machines. Radix sorting algorithms came into common use as a way to sort punched cards as early as 1923.





KNOWLEDGE GATE
.kn

170
45
75
90
802
24
2
66

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Radix Sort

- An IBM card sorter performing a radix sort on a large set of punched cards. Cards are fed into a hopper below the operator's chin and are sorted into one of the machine's 13 output baskets, based on the data punched into one column on the cards. The crank near the input hopper is used to move the read head to the next column as the sort progresses. The rack in back holds cards from the previous sorting pass.

KNOWLEDGE GATE

Radix Sort

- Radix Sort is an integer sorting algorithm that organizes data by individual digits which have the same position and value. It starts by sorting integers based on their least significant digit using a stable sorting method like counting sort to keep the same relative order for similar key values. After sorting by the least significant digit, it progresses to the next digit to the left, continuing this process until it has sorted by the most significant digit.
- The time complexity is generally $O(nk)$, where n is the number of elements and k is the number of passes needed, which depends on the number of digits in the largest number.
- Radix Sort excels at sorting fixed-length number sequences like phone numbers or dates and may outperform comparison-based sorts such as quicksort or mergesort if the numbers aren't much longer than the array size. It's particularly adept at handling large data sets because its speed depends more on digit count rather than the actual size of the numbers being sorted.

KNOWLEDGEGATE

Radix Sort

```
radixSort(arr)
{
    max = largest element in the given array
    d = number of digits in the largest element (or, max)
    Now, create d buckets of size 0 - 9
    for i -> 0 to d
    sort the array elements using counting sort (or any stable sort) according to the
    digits at the ith place
}
```

KNOWLEDGEGATE

Bucket Sort

- Bucket sort, also known as bin sort, is an effective sorting algorithm ideal for uniformly distributed data. Here are the condensed key points:
  - **Element Distribution**: Elements are distributed across several buckets based on their value.
  - **Sorting Buckets**: Each bucket is sorted individually, either using another sorting algorithm or by applying bucket sort recursively.
  - **Combining Buckets**: Sorted buckets are merged back into a single array for the final sorted order.
  - **Performance**: Best for data evenly distributed over a range. Average and best-case complexity is $O(n+k)$, with n being the number of elements and k the number of buckets.
  - **Space Requirement**: Occupies $O(n \cdot k)$ space due to the buckets.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Stability**: Maintains the relative order of equal elements.
- **Ideal Scenarios**: Most efficient for large sets of floating-point numbers or uniformly spread data.
- **Drawbacks**: Performance drops for non-uniformly distributed data and depends on the input distribution and bucket count.
- In brief, bucket sort is fast and stable, best for evenly distributed datasets. It segregates elements into buckets, sorts these, and then merges them into a sorted array.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Bucket Sort

.79	.43	.60	.11	.32	.29	.57	.82	.94	.07

0
1
2
3
4
5
6
7
8
9

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
BUCKET-SORT (A)
{
    let B[0 . . n - 1] be a new array
    n = A.length
    for i = 0 to n - 1
        make B[i] an empty list
    for i = 1 to n
        Insert A[i] into list B[nA[i]]
    for i = 0 to n - 1
        sort list B[i] with insertion sort
    concatenate the lists together
    B[0], B[1],...B[n - 1] (in order)
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Ch-3

Divide and Conquer

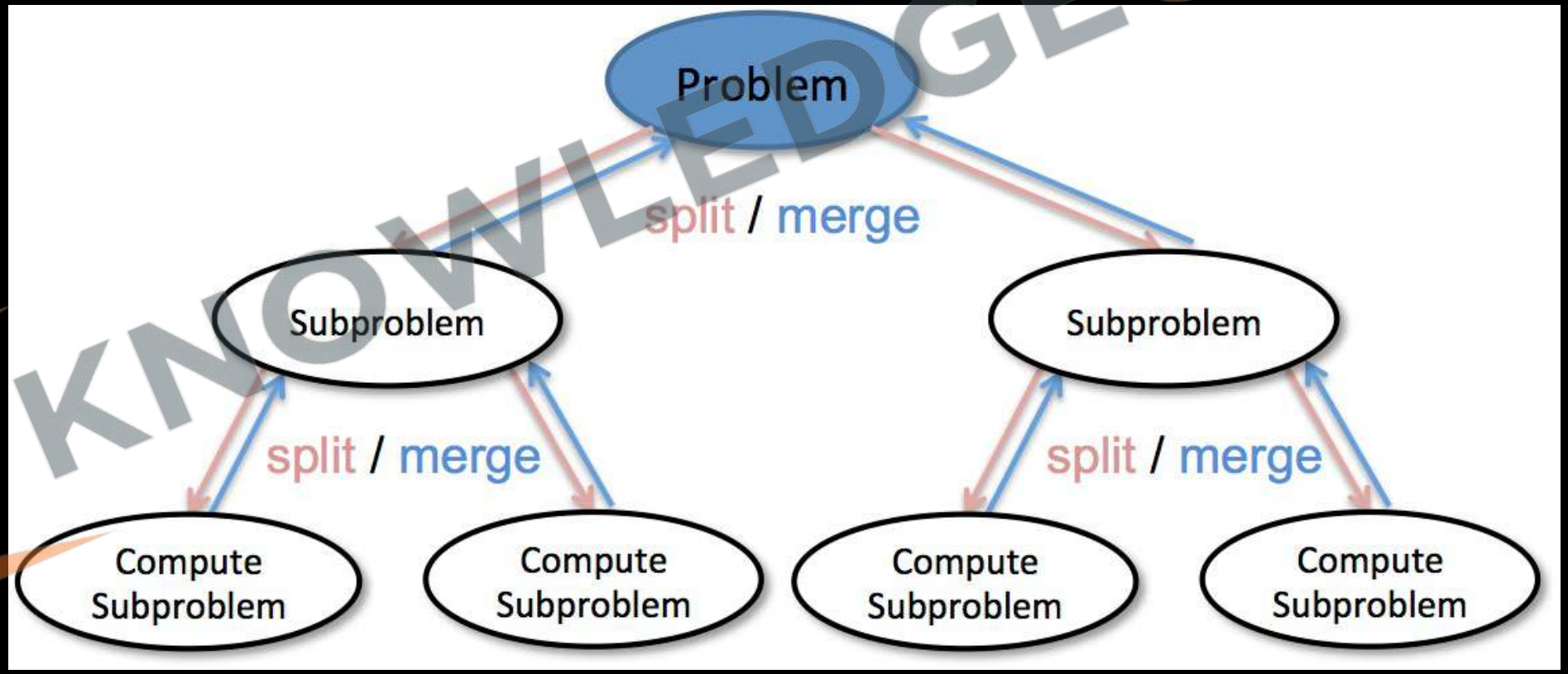
with Examples Such as Sorting, Matrix Multiplication, Convex Hull and Searching.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Divide and conquer

- Divide and conquer is a fundamental algorithm design paradigm in computer science. It works by recursively breaking down a problem into two or more sub-problems of the same or related type, until these become simple enough to be solved directly. The solutions to the sub-problems are then combined to give a solution to the original problem.



KNOWLEDGEGATE

- Here are the main steps involved in a divide and conquer algorithm:
  - **Divide**: Split the problem into several sub-problems that are smaller instances of the same problem.
  - **Conquer**: Solve the sub-problems recursively. If the sub-problems are small enough, solve them in a straightforward manner.
  - **Combine**: Combine the solutions of the sub-problems into the solution for the original problem.
- The efficiency of a divide and conquer algorithm depends on the size reduction at each division and the efficiency of the combine step. When properly designed, such algorithms can lead to significant reductions in time complexity, often achieving logarithmic growth in computational cost.

[KNOWLEDGEGATE]

Merge Sort

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Quick Sort

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

What is Matrix Multiplication

3 × 3 Matrix Multiplication

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} j & k & l \\ m & n & o \\ p & q & r \end{bmatrix} = \begin{bmatrix} (aj+bm+cp) & (ak+bn+cq) & (al+bo+cr) \\ (dj+em+fp) & (dk+en+fq) & (dl+eo+fr) \\ (gj+hm+ip) & (gk+hn+iq) & (gl+ho+ir) \end{bmatrix}$$

SQUARE-MATRIX-MULTIPLY(A, B)

```
1  n = A.rows
2  let C be a new n × n matrix
3  for i = 1 to n
4      for j = 1 to n
5          c_ij = 0
6          for k = 1 to n
7              c_ij = c_ij + a_ik · b_kj
8  return C
```

KNOWLEDGEGATE

Matrix Multiplication with Divide and Conquer

$A = [a_{11}]$ $B = [b_{11}]$ $A \times B = [a_{11} * b_{11}]$

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

$c_{11} = a_{11} * b_{11} + a_{12} * b_{21}$

$c_{12} = a_{11} * b_{12} + a_{12} * b_{22}$

$c_{21} = a_{21} * b_{11} + a_{22} * b_{21}$

$c_{22} = a_{21} * b_{12} + a_{22} * b_{22}$

KNOWLEDGEGATE

Matrix Multiplication with Divide and Conquer

$$A = \left[\begin{array}{cc|cc} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ \hline a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{array}\right] \quad B = \left[\begin{array}{cc|cc} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ \hline b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{43} & b_{43} & b_{44} \end{array}\right]$$

A_{11} A_{12} A_{21} A_{22} — B_{11} B_{12} B_{21} B_{22}

4×4 4×4

$C_{11} = A_{11} * B_{11} + A_{12} \times B_{21}$

$C_{12} = A_{11} * B_{12} + A_{12} * B_{22}$

$C_{21} = A_{21} * B_{11} + A_{22} * B_{21}$

$C_{22} = A_{21} * B_{12} + A_{22} * B_{22}$

$$f(n) = \begin{cases} 1 & n \le 2 \\ 8T(n/2) + n^2 & n > 2 \end{cases}$$

```
MM(A, B, n)
{
  if (n ≤ 2)
  {
    c11 = a11 * b11 + a12 * b21
    c12 = a11 * b12 + a12 * b22
    c21 = a21 * b11 + a22 * b21
    c22 = a21 * b12 + a22 * b22
  }
  else
  {
    Divide A & B of nxn into 4 submatrix
    each of order n/2 * n/2
    MM(A11, B11, n/2) + MM(A12, B21, n/2)
    MM(A11, B12, n/2) + MM(A12, B22, n/2)
    MM(A21, B11, n/2) + MM(A22, B21, n/2)
    MM(A21, B12, n/2) + MM(A22, B22, n/2)
  }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Matrix Multiplication with Divide and Conquer

- Volker Strassen first published this algorithm in 1969 and thereby proved that the n^3 general matrix multiplication algorithm was not optimal. The Strassen algorithm's publication resulted in more research about matrix multiplication that led to both asymptotically lower bounds and improved computational upper bounds.

KNOWLEDGE GATE

Strassens Matrix Multiplication

Input: $A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$ and $B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \in \mathbb{R}^{n \times n}$

1: **if** $n = 1$ **then**
2: $C = A \cdot B$
3: **else**
4: $M_1 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$
5: $M_2 = (A_{21} + A_{22}) \cdot B_{11}$
6: $M_3 = A_{11} \cdot (B_{12} - B_{22})$
7: $M_4 = A_{22} \cdot (B_{21} - B_{11})$
8: $M_5 = (A_{11} + A_{12}) \cdot B_{22}$
9: $M_6 = (A_{21} - A_{11}) \cdot (B_{11} + B_{12})$
10: $M_7 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$
11: $C_{11} = M_1 + M_4 - M_5 + M_7$
12: $C_{12} = M_3 + M_5$
13: $C_{21} = M_2 + M_4$
14: $C_{22} = M_1 - M_2 + M_3 + M_6$

Output: $A \cdot B = C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} \in \mathbb{R}^{n \times n}$

$$T(n) = \begin{cases} 1 & n \leq 2 \\ 7T(n/2) + n^2 & n > 2 \end{cases}$$

$$O(n^{\log_b a}) = O(n^{\log_2 7})$$

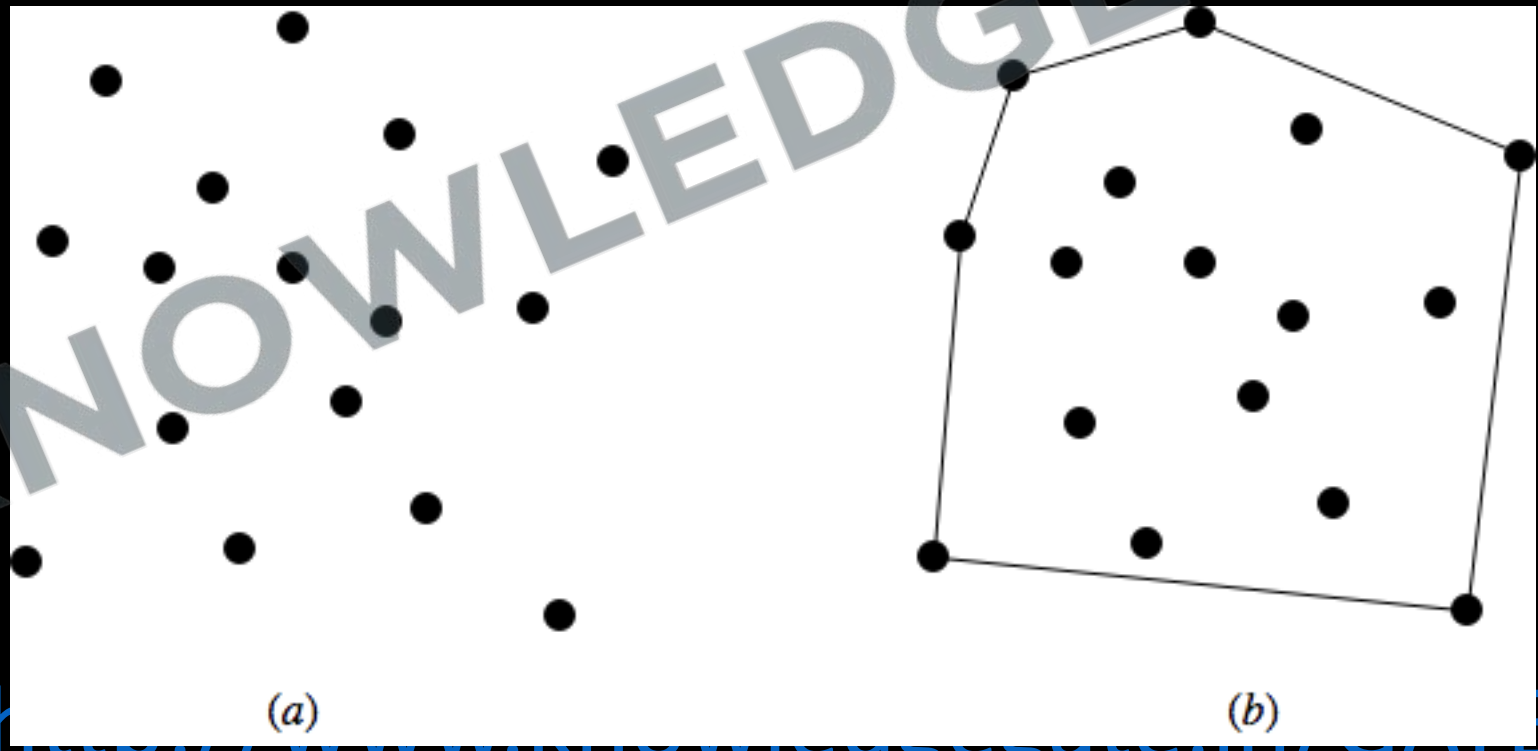
$$= O(n^{2.81})$$

KNOWLEDGEGATE

owledgegate.in/GATE

Convex Hull

- The convex hull problem is a classic algorithmic problem in the field of computational geometry. The goal is to find the smallest convex polygon that encloses a set of points in a plane. In simple terms, if you imagine a set of nails hammered into a board and you wrap a rubber band around all the nails, the rubber band would outline the convex hull.
- **Convex Hull**: The smallest convex polygon formed by a set of points such that no point from the set lies outside the polygon.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- The convex hull is a fundamental structure in computational geometry, with applications in pattern recognition, image processing, GIS (Geographic Information Systems), and in solving other geometric problems.

Applications:

- Pathfinding and motion planning problems.
- Collision detection in physical simulations and computer games.
- Determining the boundary of an object in machine learning and computer vision.
- Clustering analysis in data mining.
- Supporting GIS operations like creating the boundary for geographical datasets.

KNOWLEDGE GATE

```
GRAHAM-SCAN(Q)
{
    Let p0 be the point in Q with the minimum y-coordinate, or the leftmost such point in
case of a   tie.
    Let <p1, p2, ..., pm> be the remaining points in Q, sorted by polar angle in counter
clockwise order around p0.
    Top[S] ← 0
    PUSH(p0, S)
    PUSH(p1, S)
    PUSH(p2, S)
    for i ← 3 to m
        do while the angle formed by points NEXT-TO-TOP(S), Top(S), and pi makes a non-
left turn
            POP(S)
        PUSH(pi, S)
    Return S
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Binary Search

- Binary search is a classic searching algorithm used to find the position of a target value within a sorted array. It is much more efficient than a linear search, offering $O(\log n)$ time complexity, where n is the number of element in the array.

KNOWLEDGEGATE

- Binary search locates a target value within a sorted array using a divide-and-conquer approach:
  - **Initialization**: Set low and high pointers at the array's start and end.
  - **Middle Index**: Calculate the middle of low and high. Use low + (high - low) / 2 to prevent overflow.
  - **Comparison**: Check if the middle element is the target. If yes, return its index.
  - **Direction**: If the target is smaller, search the left side (high becomes middle - 1); if larger, search the right (low becomes middle + 1).
  - **Iteration**: Repeat until low exceeds high.
  - **Outcome**: Return the target's index or indicate it's not found.

[KNOWLEDGEGATE]

http://www.knowledgegate.in/GATE

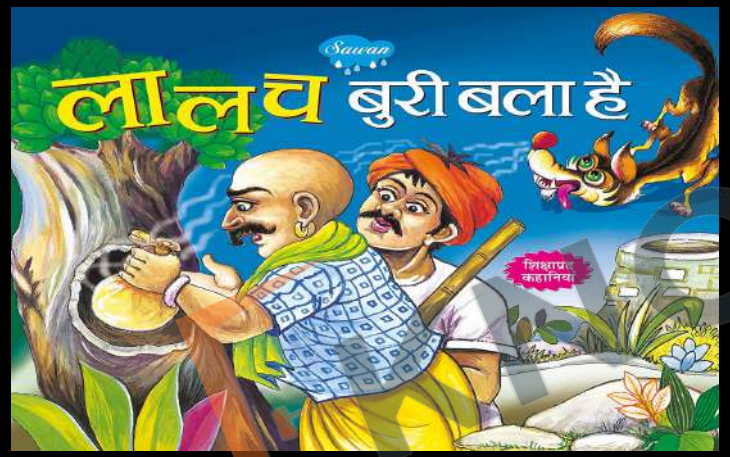
Ch-4

Greedy Methods

with Examples Such as Optimal Reliability Allocation, Knapsack, Huffman algorithm

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE



लालच बुरी बला है,
अगर बुरे काम के लिए किया गया हो तो

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Greedy Algorithm

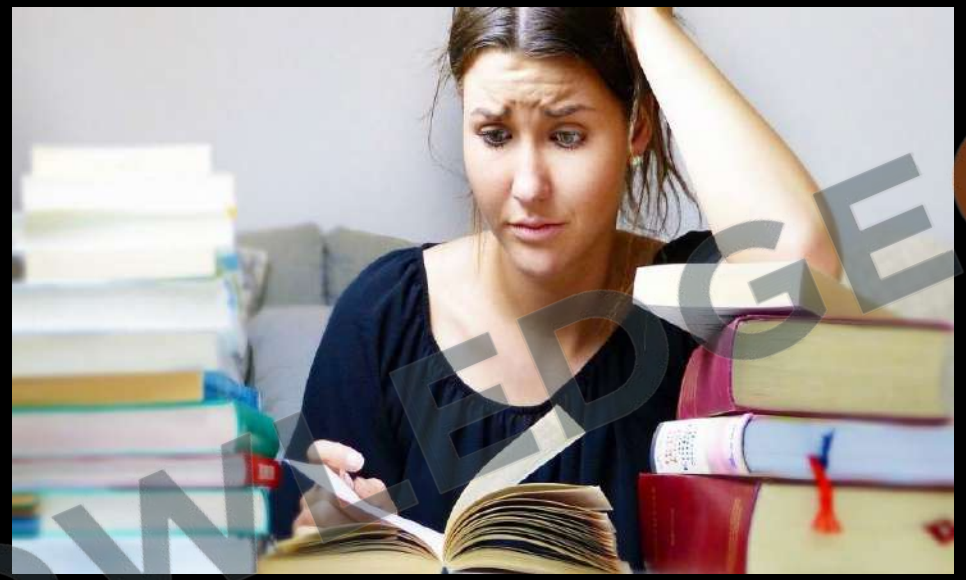
- A **greedy algorithm** is a problem solving approach like Subtract and conquer, divide and conquer and dynamic programming, which is used for solving optimality problem(one Solution), out of all feasible solution.
- Minimum Spanning Tree
- Single source shortest path
- Huffman Coding
- Knapsack Problem
- Optimal Merge Pattern

KNOWLEDGEGATE

Greedy Algorithm

- A **greedy algorithm** is any algorithm that follows the problem-solving heuristic of making the locally optimal choice at each stage with the intent of finding a global optimum.

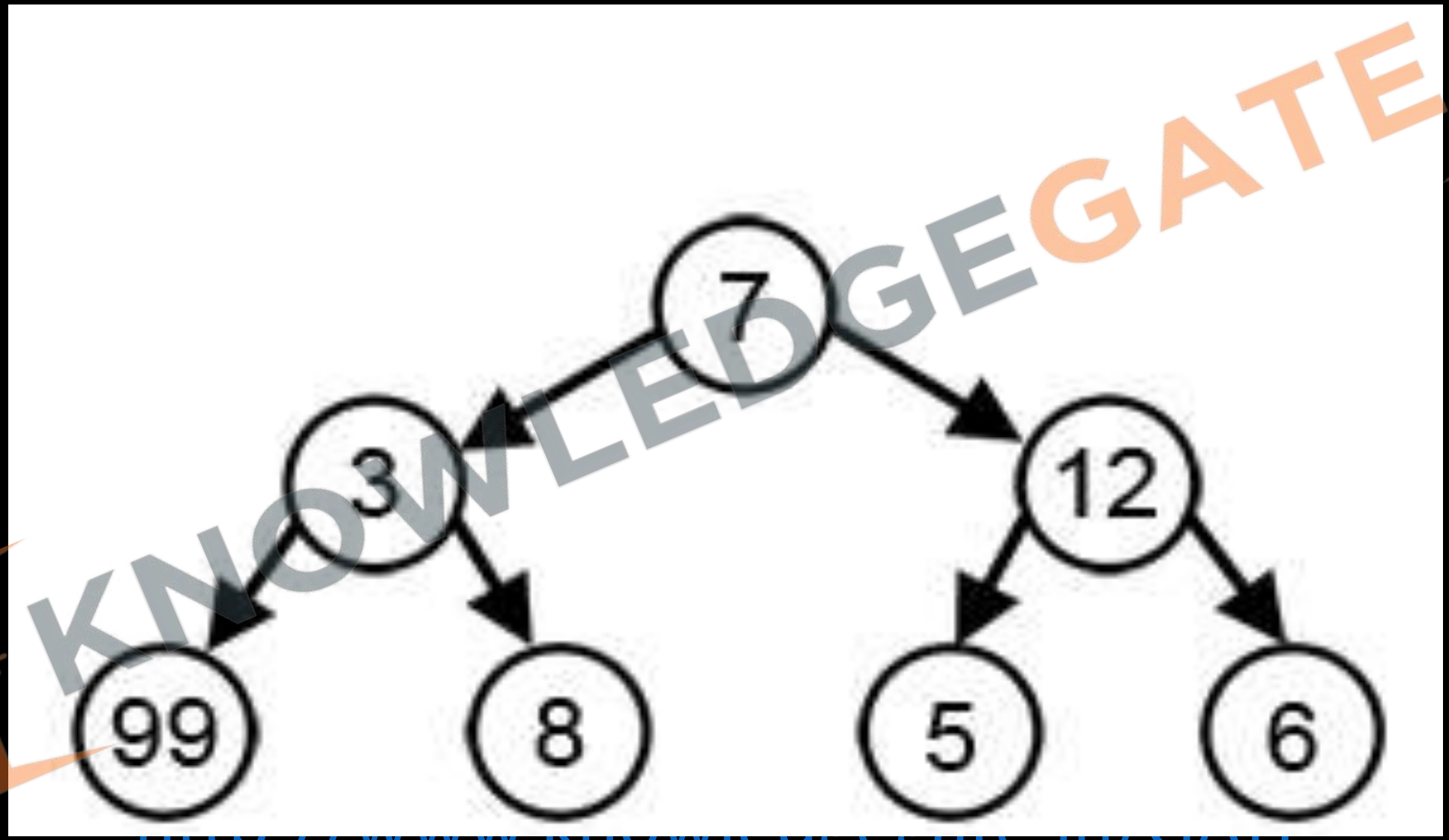
Bank Officer



IAS Officer

KNOWLEDGEGATE

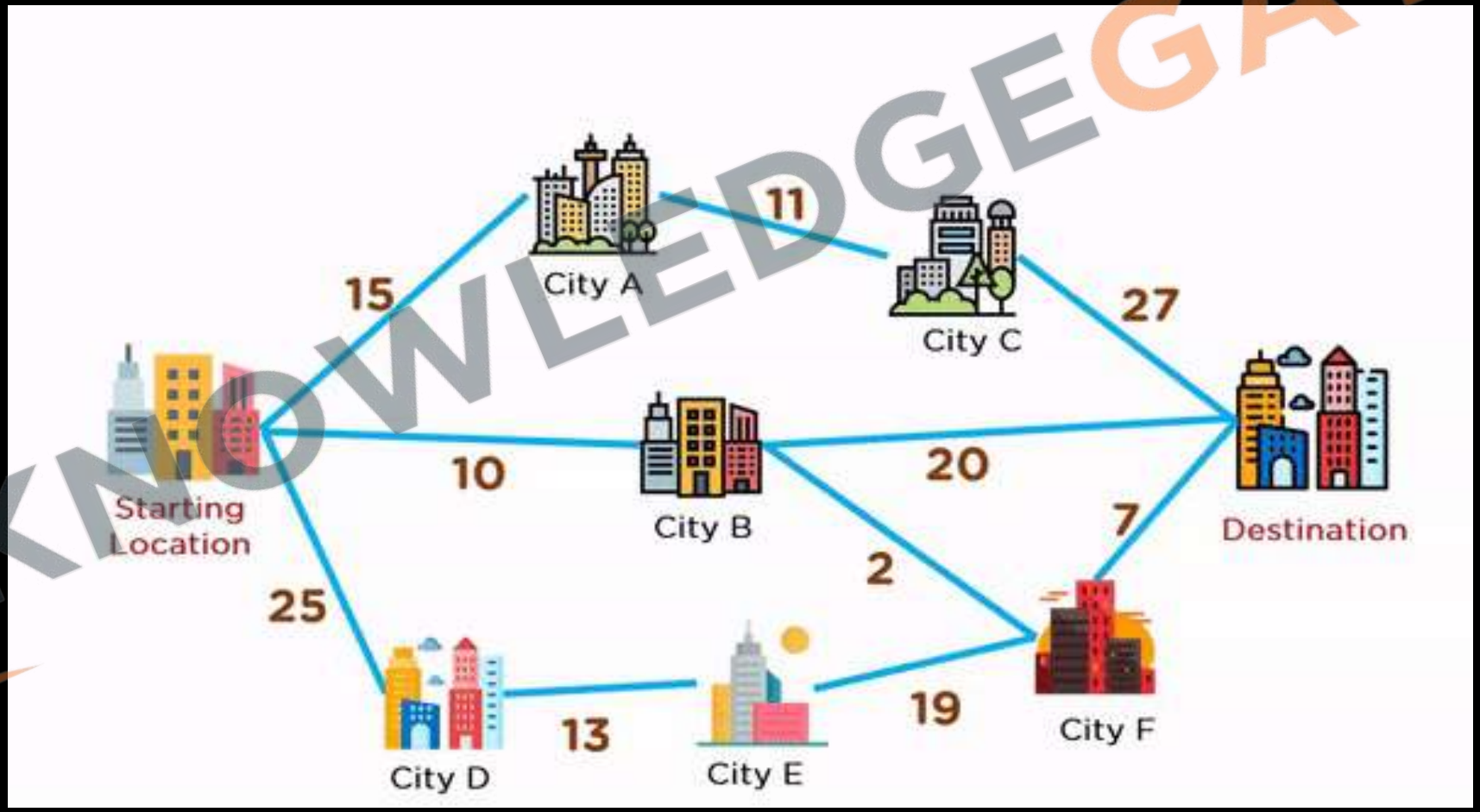
- **Will greedy always work?** - In many problems, a greedy strategy does not usually produce an optimal solution, but nonetheless a greedy heuristic may yield locally optimal solutions that approximate a globally optimal solution in a reasonable amount of time.



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- For example, a greedy strategy for the traveling salesman problem (which is of a high computational complexity) is the following heuristic:
- "At each step of the journey, visit the nearest unvisited city." This heuristic does not intend to find a best solution, but it terminates in a reasonable number of steps; finding an optimal solution to such a complex problem typically requires unreasonably many steps.



KNOWLEDGEGATE

- We can make whatever choice seems best at the moment and then solve the subproblems that arise later.

KNOWLEDGEGATE

- It iteratively makes one greedy choice after another, reducing each given problem into a smaller one. In other words, a greedy algorithm never reconsiders its choices.

इस मेट्रो में जाऊँ या अगली में?

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- This is the main difference from dynamic programming, which is exhaustive and is guaranteed to find the solution. After every stage, dynamic programming makes decisions based on all the decisions made in the previous stage, and may reconsider the previous stage's algorithmic path to solution.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Optimization Problem

- An optimization problem is a type of problem that seeks to find the best solution from all feasible solutions. Here's a simple breakdown:

- **Optimization Problems:**
  - **Objective:** Minimize or maximize some quantity (like cost, profit, distance).
  - **Constraints:** Set of restrictions or conditions that the solution must satisfy.
  - **Feasible Solution:** A solution that meets all constraints.
  - **Optimal Solution:** A feasible solution that yields the best value of the objective function.

KNOWLEDGE GATE

Activity Selection Problem

- The Activity Selection Problem is a classic problem and often used to illustrate the concept of greedy algorithms. Here's a simplified explanation:
- You have a set of activities, each with a start and an end time. Each activity has s_i a start time, and f_i a finish time. If activity i is selected, the resource is occupied in the intervals (s_i, f_i).We say i and j are compatible activities if their start and finish time does not overlap i.e., i and j compatible if $s_i >= f_j$ and $s_j >= f_i$
- You need to select the maximum number of activities that don't overlap in time. The **Goal is to** Maximize the number of activities selected.

KNOWLEDGE GATE

- **Approach:**
  - **Sort Activities:** First, sort all activities by their finish time.
  - **Select First Activity:** Choose the activity that finishes first.
  - **Iterate and Select:** For each subsequent activity, if its start time is after or at the finish time of the previously selected activity, select it.

- Activities (start time, end time): (1, 3), (2, 4), (3, 5), (5, 7)
- Sorted by end time: (1, 3), (3, 5), (2, 4), (5, 7)
- Selected Activities: (1, 3), (3, 5), (5, 7)

KNOWLEDGEGATE

- At each step, you make the choice that seems best at the moment (choosing the activity that ends earliest). This local optimization leads to a globally optimal solution.

- **Use Cases:**
  - Scheduling tasks in a single resource environment (like a single meeting room).
  - Allocating time slots for interviews or exams where no overlap is allowed.

- **Complexity:**
  - If the activities are not sorted, the main complexity is in the sorting step, which is typically O(n log n) for n activities.
  - The selection process is O(n), as it involves iterating through the list once.
  - This problem is an excellent example to teach students about greedy algorithms, showing how a locally optimal choice can lead to a globally optimal solution in certain scenarios.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

ii. Pseudo code for iterative greedy algorithm :

G_A_S (s, f)

1. $n \leftarrow$ length $[s]$
2. $A \leftarrow [a_1]$
3. $i \leftarrow 1$
4. $m \leftarrow 2$ to n
5. do if $s_m \geq f_i$
6. then $A \leftarrow A \cup \{a_m\}$
7. $i \leftarrow m$
8. return A

KNOWLEDGEGATE

Sorted activities	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}
Starting time	2	1	3	0	5	3	5	6	8	8	12
Finish time	3	4	5	6	7	8	9	10	11	12	14

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Knap Sack Problem

- The **knapsack problem** or **rucksack problem** is a problem in combinatorial optimization: Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

KNOWLEDGEGATE
www.knowledgegate.in/GATE

- Knap Sack problem can be studied in two versions fractional Knap Sack and 0/1 Knap Sack, here we will be discussing Fractional Knap Sack and then 0/1 Knap Sack will be solved in Dynamic Programming.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider the weights and values of items listed below. Note that there is only one unit of each item.

Object	O_1	O_2	O_3
Profit	25	24	15
Weight	18	15	10

Greedy by Profit

Object	O_1	O_2	O_3
Profit	25	24	15
Weight	18	15	10
Solution			

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider the weights and values of items listed below. Note that there is only one unit of each item.

Object	O_1	O_2	O_3
Profit	25	24	15
Weight	18	15	10

Greedy by Weight

Object	O_1	O_2	O_3
Profit	25	24	15
Weight	18	15	10
Solution			

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider the weights and values of items listed below. Note that there is only one unit of each item.

Object	O_1	O_2	O_3
Profit	25	24	15
Weight	18	15	10

Greedy by Profit/Weight

Object	O_1	O_2	O_3
Profit	25	24	15
Weight	18	15	10
Profit/Weight	1.38	1.6	1.5
Solution			

KNOWLEDGEGATE

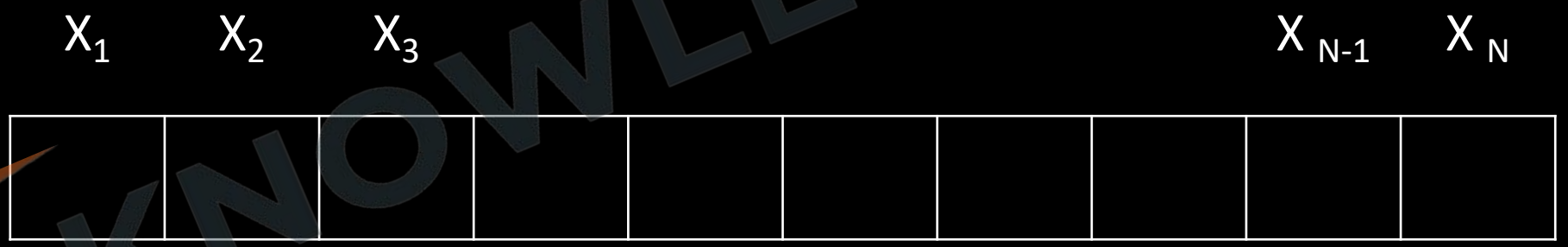
http://www.knowledgegate.in/GATE

Problem Definition

- More formally there are n number of objects O_1, O_2, O_3…O_n, each object has a weight associated with its W_i, and a profit associated with it P_i, we can take x_i fraction of the object ranging from $0 <= x_i <= 1$

- Weight Condition $\sum_{i=1}^{n} Wi \,.\, Xi <= M$

- Profit $\sum_{i=1}^{n} Pi \,.\, Xi$



KNOWLEDGEGATE

- The knapsack problem has been studied for more than a century, with early works dating as far back as 1897. The name "knapsack problem" dates back to the early works of mathematician Tobias Dantzig (1884–1956).

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Conclusion

- It derives its name from the problem faced by someone who is constrained by a fixed-size knapsack and must fill it with the most valuable items.

- The problem often arises in resource allocation where there are financial constraints and is studied in fields such as
  - Combinatorics
  - Computer science
  - Complexity theory
  - Cryptography
  - Applied mathematics.

KNOWLEDGEGATE

Job sequencing with Deadline

- We are given n-jobs, where each job is associated with a deadline D_i and a profit P_i if the job if finished before the deadline.
- We have single CPU with Non-Primitive Scheduling.
- With each job we assume arrival time is 0, burst time of each job requirement is 1.
- Select a Subset of ‘n’ jobs, such that, the jobs in the subset can be completed with in the deadline and generate Max profit.

[KNOWLEDGEGATE]

Q if we have for task T_1, T_2, T_3, T_4, having Deadline $D_1 = 2$, $D_2 = 1$, $D_3 = 2$, $D_4 = 1$, and profit $P_1 = 100$, $P_2 = 10$, $P_3 = 27$, $P_4 = 15$, find the maximum profit possible?

Task	T_1	T_2	T_3	T_4
Profit	100	10	27	15
Deadline	2	1	2	1

KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

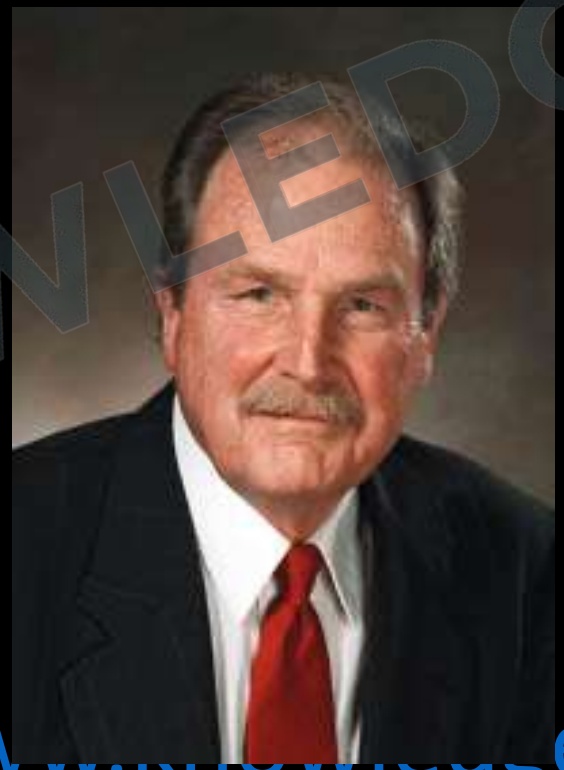
Task	T_1	T_2	T_3	T_4	T_5	T_6	T_7
Profit	35	30	25	20	15	12	5
Deadline	3	4	4	2	3	1	2

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Huffman coding

- In computer science and information theory, a **Huffman code** is a particular type of optimal prefix code that is commonly used for lossless data compression.
- The process of finding or using such a code proceeds by means of **Huffman coding**, an algorithm developed by David A. Huffman while he was a Sc.D. student at MIT, and published in the 1952 paper "A Method for the Construction of Minimum-Redundancy Codes".



KNOWLEDGEGATE

- In 1951, David A. Huffman and his MIT information theory classmates were given the choice of a term paper or a final exam. The professor, Robert M. Fano, assigned a term paper on the problem of finding the most efficient binary code.

- Huffman, unable to prove any codes were the most efficient, was about to give up and start studying for the final when he hit upon the idea of using a frequency-sorted binary tree and quickly proved this method the most efficient.

- In doing so, Huffman outdid Fano, who had worked with Claude Shannon to develop a similar code. Building the tree from the bottom up guaranteed optimality, unlike the top-down approach of Shannon–Fano coding.

KNOWLEDGEGATE

- The output from Huffman's algorithm can be viewed as a variable-length code table for encoding a source symbol (such as a character in a file).

- It always satisfy prefix-code property.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- The algorithm derives this table from the estimated probability or frequency of occurrence (*weight*) for each possible value of the source symbol.

- Huffman's method can be efficiently implemented, finding a code in time linear to the number of input weights if these weights are sorted.

- However, although optimal among methods encoding symbols separately, Huffman coding is not always optimal among all compression methods.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

In alphabetical order			
A	8.15 %	N	7.10 %
B	1.44 %	O	8.00 %
C	2.76 %	P	1.98 %
D	3.79 %	Q	0.12 %
E	13.11 %	R	6.83 %
F	2.92 %	S	6.10 %
G	1.99 %	T	10.47 %
H	5.26 %	U	2.46 %
I	6.35 %	V	0.92 %
J	0.13 %	W	1.54 %
K	0.42 %	X	0.17 %
L	3.39 %	Y	1.98 %
M	2.54 %	Z	0.08 %

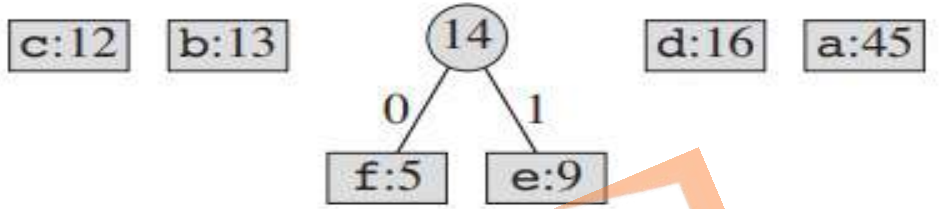
KNOWLEDGEGATE

```
HUFFMAN(C)
1  n = |C|
2  Q = C
3  for i = 1 to n − 1
4      allocate a new node z
5      z.left = x = EXTRACT-MIN(Q)
6      z.right = y = EXTRACT-MIN(Q)
7      z.freq = x.freq + y.freq
8      INSERT(Q, z)
9  return EXTRACT-MIN(Q)      // return the root of the tree
```

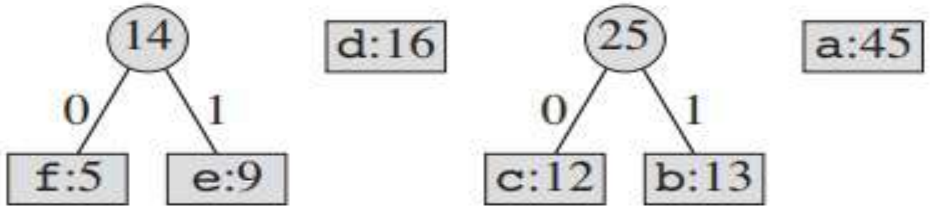
KNOWLEDGEGATE

(a)

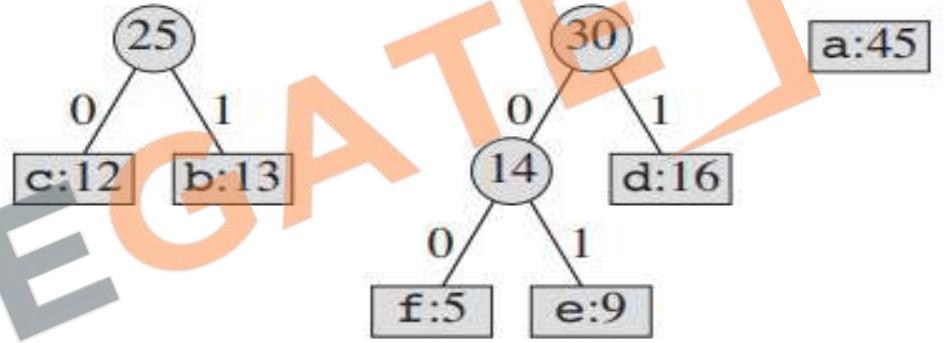
(b)



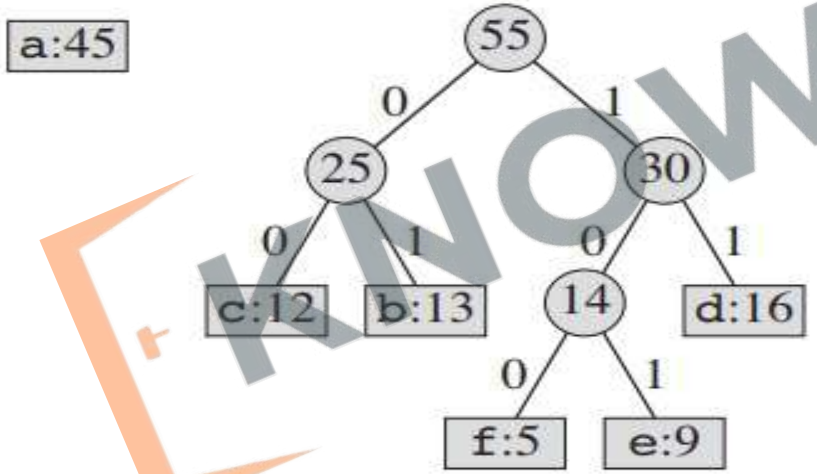
(c)



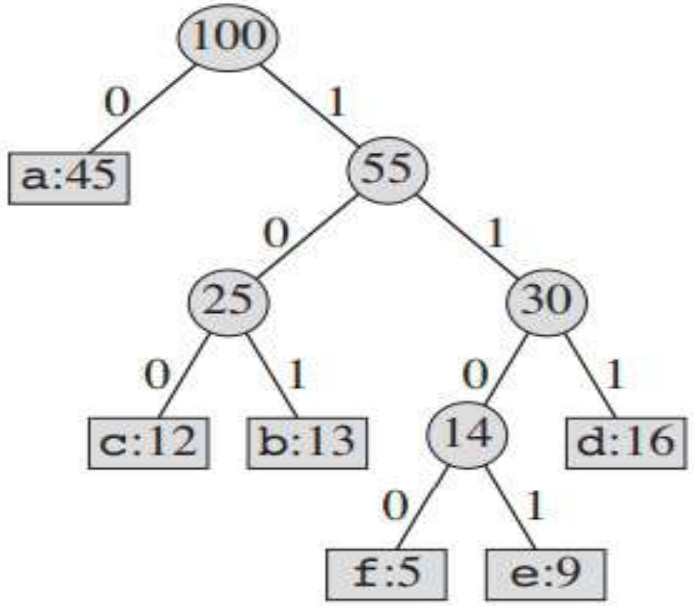
(d)



(e)



(f)



KNOWLEDGEGATE

Q Consider the following character with frequency and generate Huffman tree, find Huffman code for each character, find the number of bits required for a message of 100 characters?

Character	Frequency
M_1	12
M_2	4
M_3	45
M_4	17
M_5	23

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Ch-5

Minimum Spanning Trees

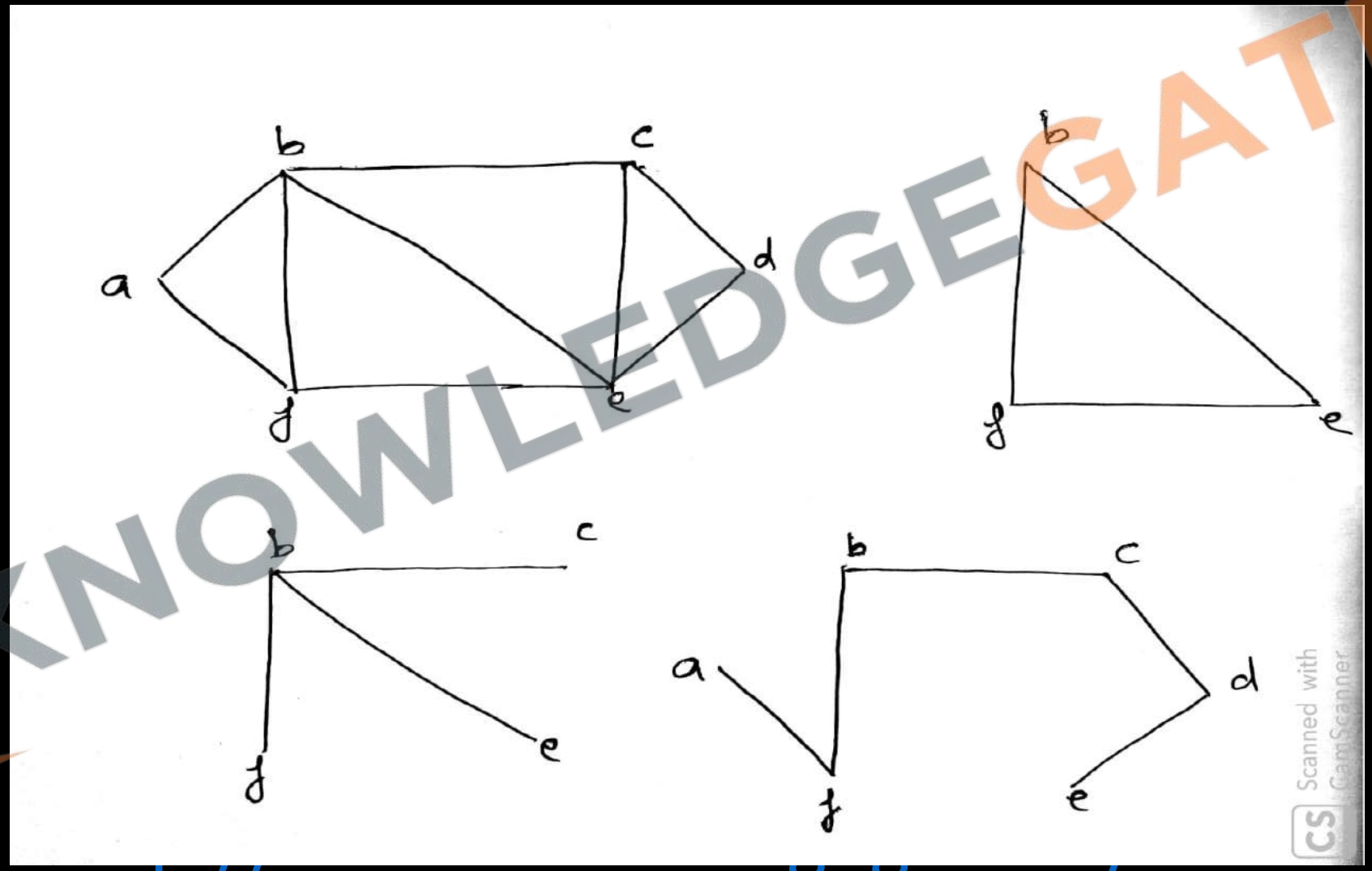
Prim's and Kruskal's Algorithms

KNOWLEDGEGATE

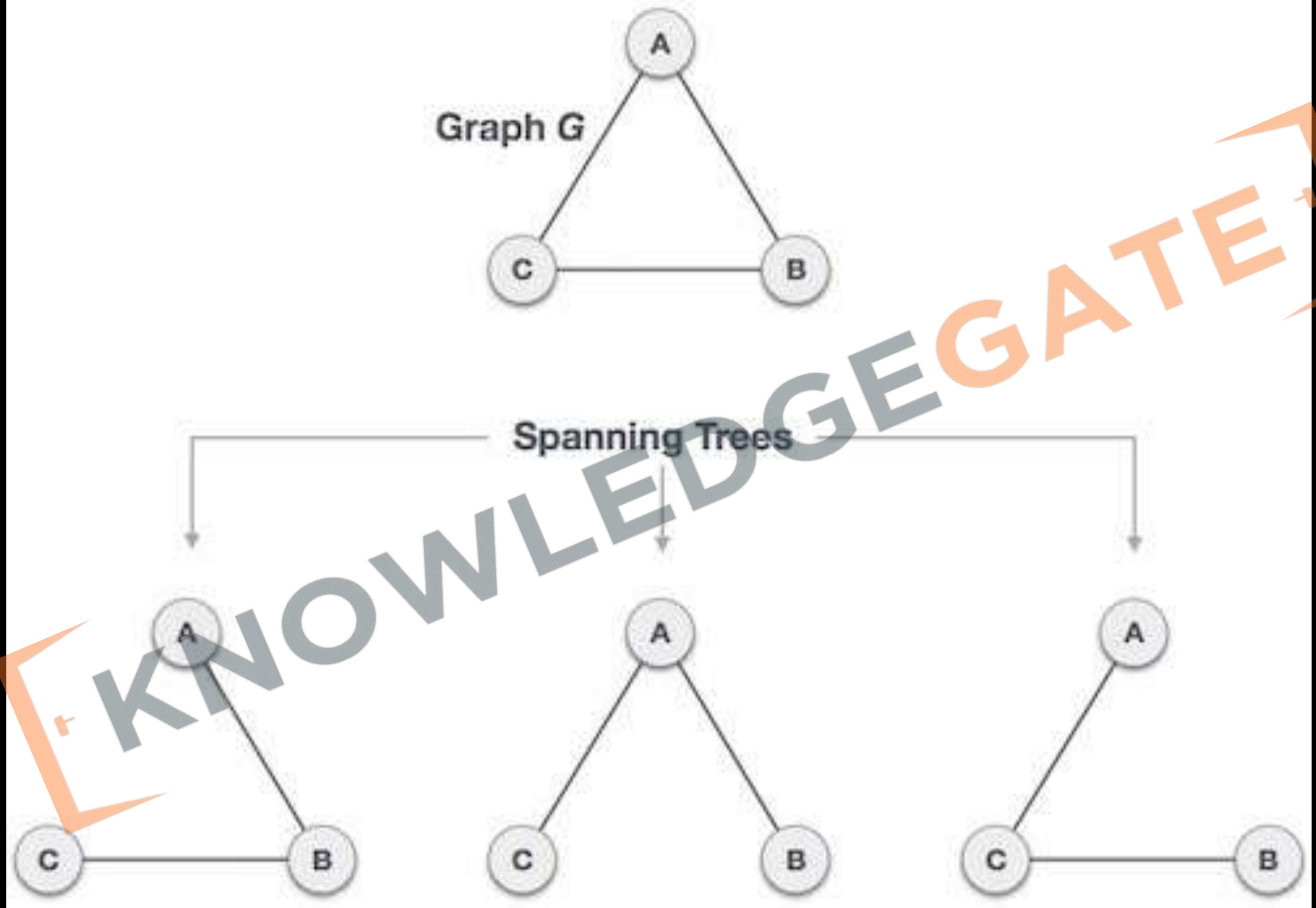
http://www.knowledgegate.in/GATE

Spanning tree

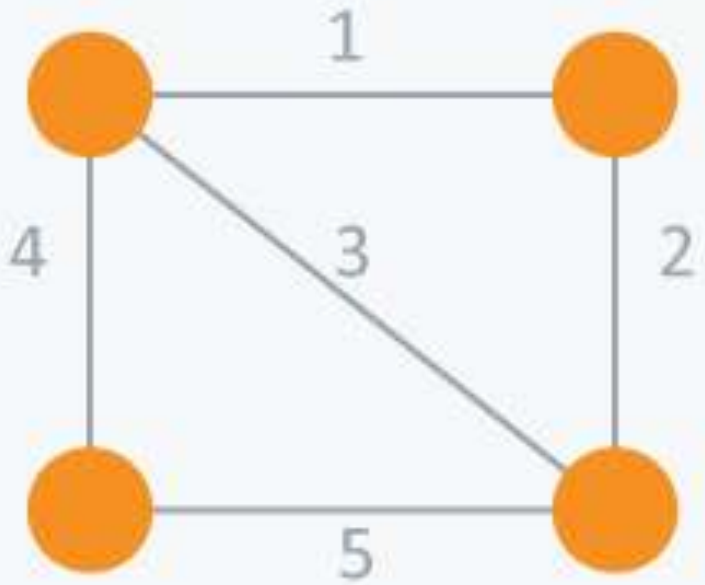
- A tree T is said to be spanning tree of a connected graph G, if T is a subgraph of G and T contains all vertices of G.



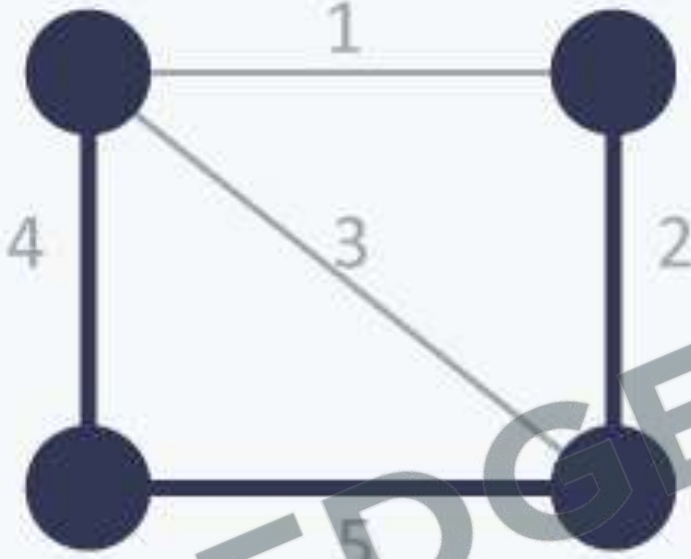
KNOWLEDGEGATE
Scanned with CamScanner



KNOWLEDGEGATE

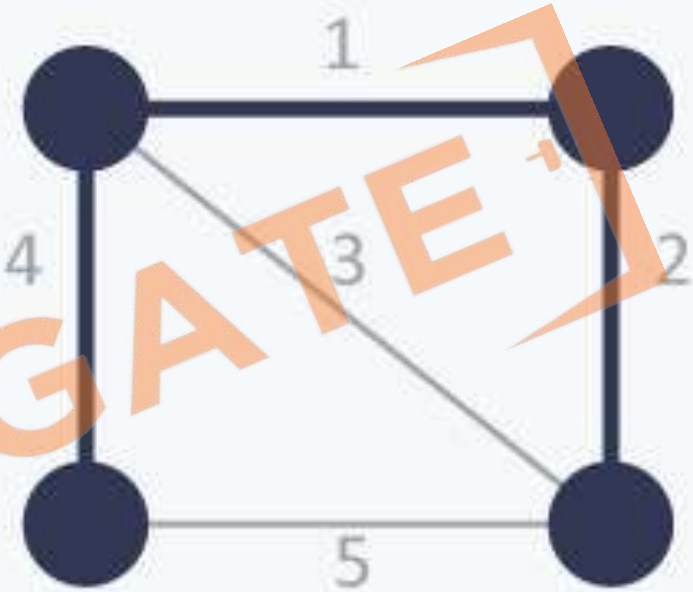


Undirected Graph



Spanning Tree

Cost = 11(=4+5+2)



Minimum Spanning Tree

Cost = 7(=4+1+2)

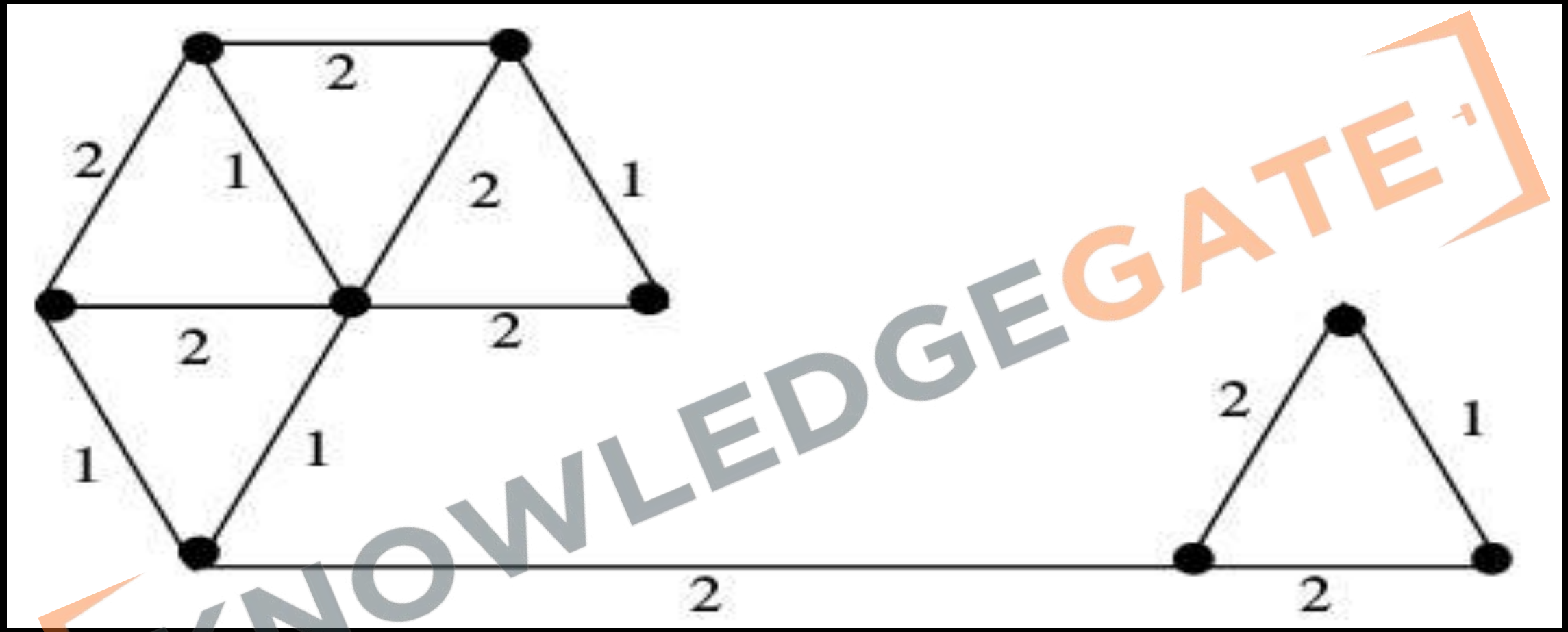
KNOWLEDGEGATE

- A **minimum spanning tree** (**MST**) or **minimum weight spanning tree** is a subset of the edges of a connected, edge-weighted undirected graph that connects all the vertices together, without any cycles and with the **minimum possible total edge weight**.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

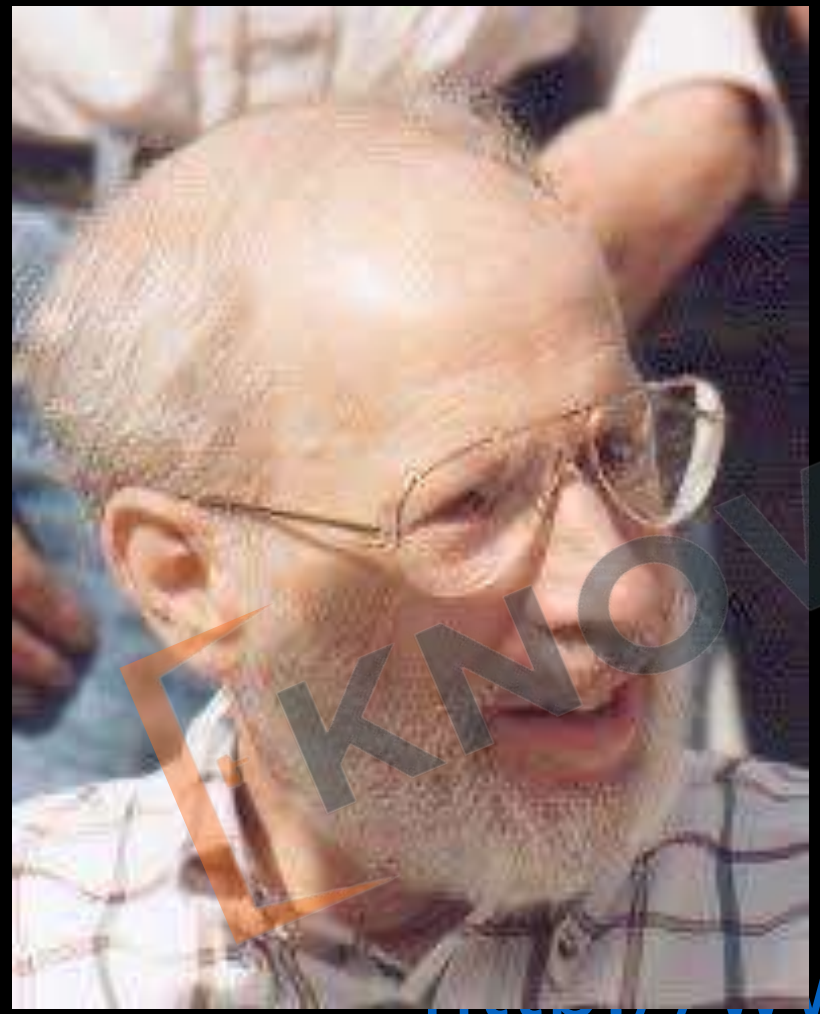
- **Minimum spanning tree** (**MST**) can be more than one



KNOWLEDGEGATE

Kruskal Algorithm

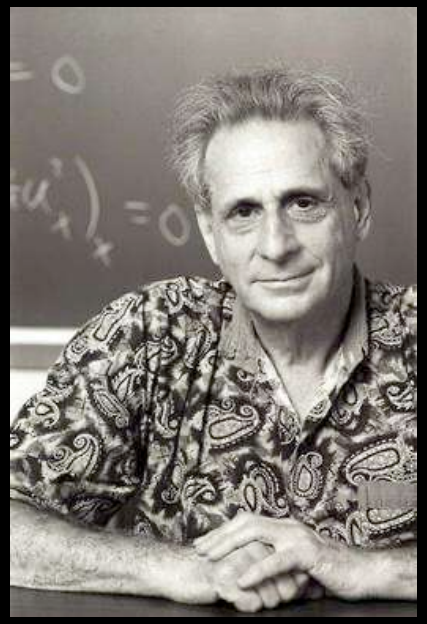
- **Joseph Bernard Kruskal, Jr.** was an American mathematician, statistician, computer scientist and psychometrician.



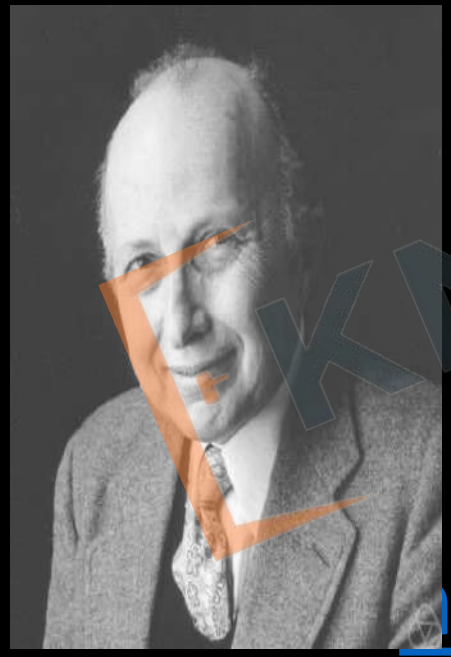
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- Kruskal had two notable brothers



Martin David Kruskal (September 28, 1925 – December 26, 2006) was an American mathematician and physicist. He made fundamental contributions in many areas of mathematics and science, ranging from plasma physics to general relativity and from nonlinear analysis to asymptotic analysis. His most celebrated contribution was in the theory of solitons.

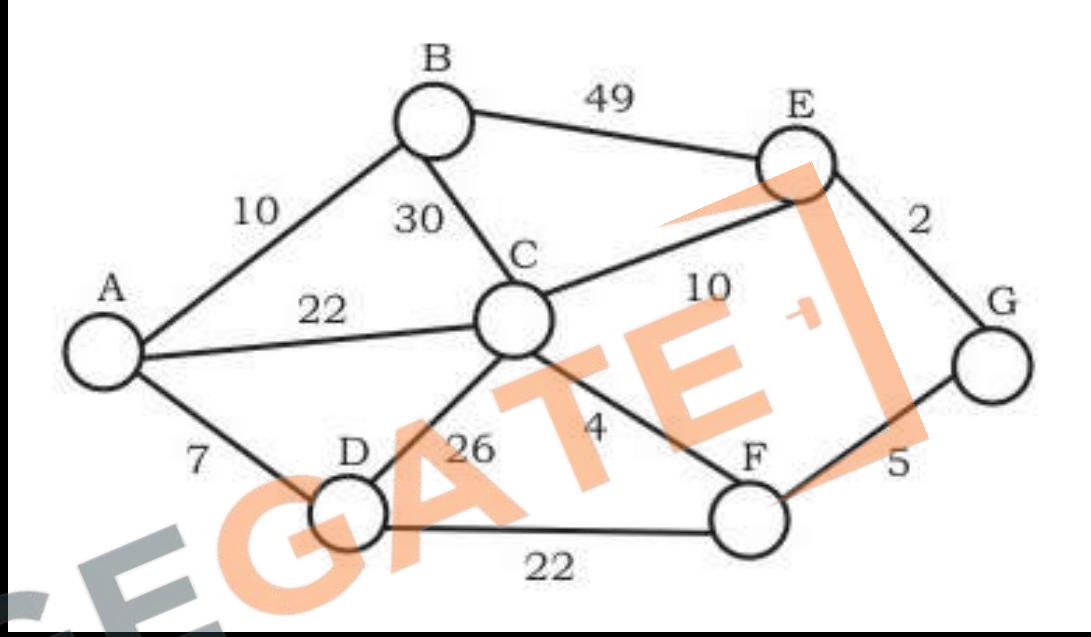


William Henry Kruskal (October 10, 1919 – April 21, 2005) was an American mathematician and statistician. He is best known for having formulated the Kruskal–Wallis one-way analysis of variance (together with W. Allen Wallis), a widely used nonparametric statistical method.

KNOWLEDGEGATE

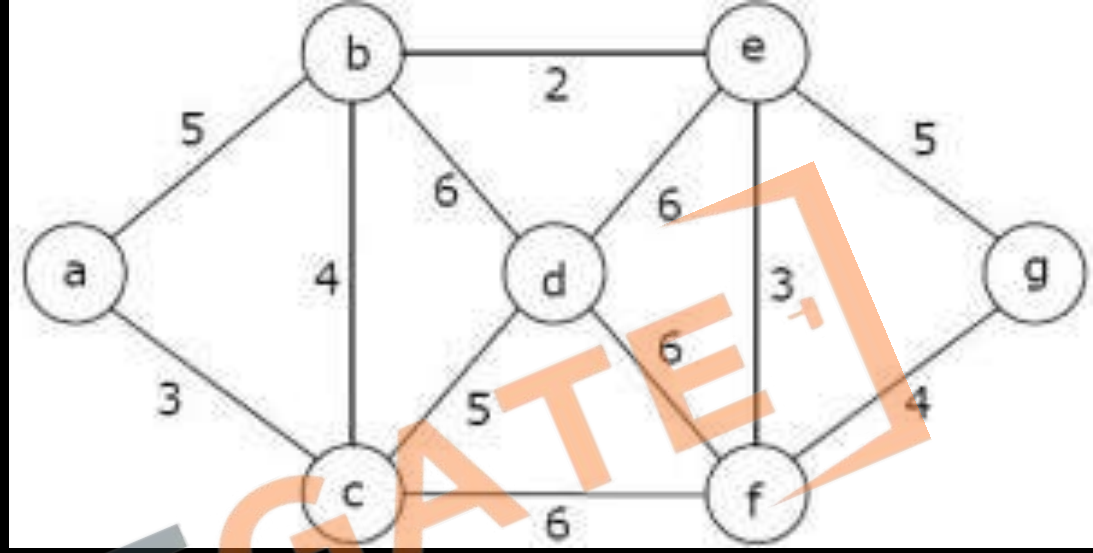
http://www.knowledgegate.in/GATE

- Kruskal Algorithm Actual idea
  - सबसे छोटा Edge उठाने का
  - Cycle बने तो भगाने का
  - जब तक graph connect ना हो चलते जाने का



KNOWLEDGEGATE

Kruskal




```
Minimum_Spanning_Tree (G, w)
{
    A ← ϕ
    For each vertex v ϵ V(G)
    {
        do Make_Set(v)
    }
    Sort the edges of E into non-decreasing order by weight w
    for each edge (u, v) ϵ E, then in non-decreasing order by weights
    {
        if (Find_Set(u) != Find_Set(v))
        {
            A ← A U {(u, v)}
            UNION (u, v)
        }
    }
    Return A
}
```

KNOWLEDGEGATE

```
MST-KRUSKAL(G, w)
1  A = ∅
2  for each vertex v ∈ G.V
3      MAKE-SET(v)
4  sort the edges of G.E into nondecreasing order by weight w
5  for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
6      if FIND-SET(u) ≠ FIND-SET(v)
7          A = A ∪ {(u, v)}
8          UNION(u, v)
9  return A
```

The running time of Kruskal's algorithm for a graph $G = (V, E)$ depends on how we implement the disjoint-set data structure. We assume that we use the disjoint-set-forest implementation of Section 21.3 with the union-by-rank and path-compression heuristics, since it is the asymptotically fastest implementation known. Initializing the set A in line 1 takes $O(1)$ time, and the time to sort the edges in line 4 is $O(E \lg E)$. (We will account for the cost of the $|V|$ MAKE-SET operations in the **for** loop of lines 2–3 in a moment.) The **for** loop of lines 5–8 performs $O(E)$ FIND-SET and UNION operations on the disjoint-set forest. Along with the $|V|$ MAKE-SET operations, these take a total of $O((V + E)\,\alpha(V))$ time, where α is the very slowly growing function defined in Section 21.4. Because we assume that G is connected, we have $|E| \geq |V| - 1$, and so the disjoint-set operations take $O(E\alpha(V))$ time. Moreover, since $\alpha(|V|) = O(\lg V) = O(\lg E)$, the total running time of Kruskal's algorithm is $O(E \lg E)$. Observing that $|E| < |V|^2$, we have $\lg |E| = O(\lg V)$, and so we can restate the running time of Kruskal's algorithm as $O(E \lg V)$.

KNOWLEDGEGATE
ate.in/GATE

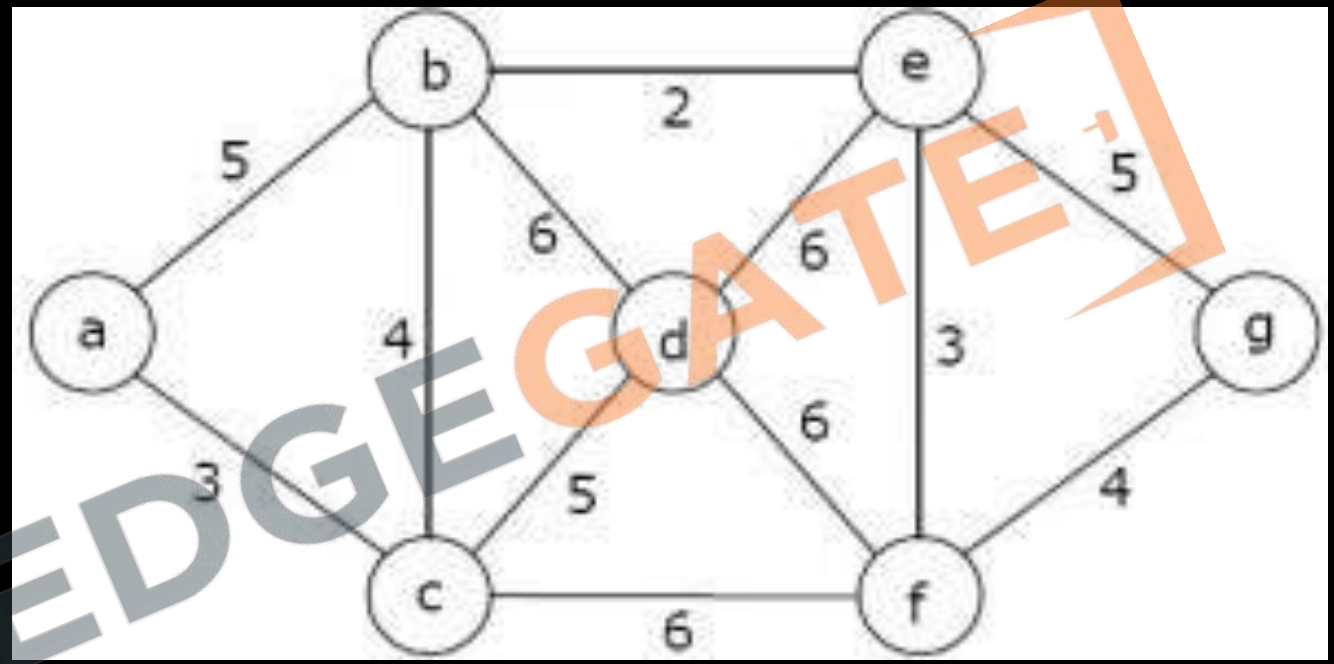
Q Consider the following graph: Which one of the following is NOT the sequence of edges added to the minimum spanning tree using Kruskal's algorithm?

(A) (b, e),(e, f),(a, c),(b, c),(f, g),(c, d)

(B) (b, e),(e, f),(a, c),(f, g),(b, c),(c, d)

(C) (b, e),(a, c),(e, f),(b, c),(f, g),(c, d)

(D) (b, e),(e, f),(b, c),(a, c),(f, g),(c, d)



KNOWLEDGEGATE

Prim's Algorithm

- The algorithm was developed in 1930 by Czech mathematician Vojtěch Jarník
- And later rediscovered and republished by computer scientists Robert C. Prim in 1957
- And Edsger W. Dijkstra in 1959.
- Therefore, it is also sometimes called the **Jarník's algorithm**, **Prim–Jarník algorithm**, **Prim–Dijkstra algorithm** or the **DJP algorithm**.

KNOWLEDGE GATE
://www ate.in/

- Vojtěch Jarník (1897–1970) was a Czech mathematician who worked for many years as a professor and administrator at Charles University, and helped found the Czechoslovak Academy of Sciences. He is the namesake of Jarník's algorithm for minimum spanning trees.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Robert Clay Prim** (born September 25, 1921 in Sweetwater, Texas) is an American mathematician and computer scientist.

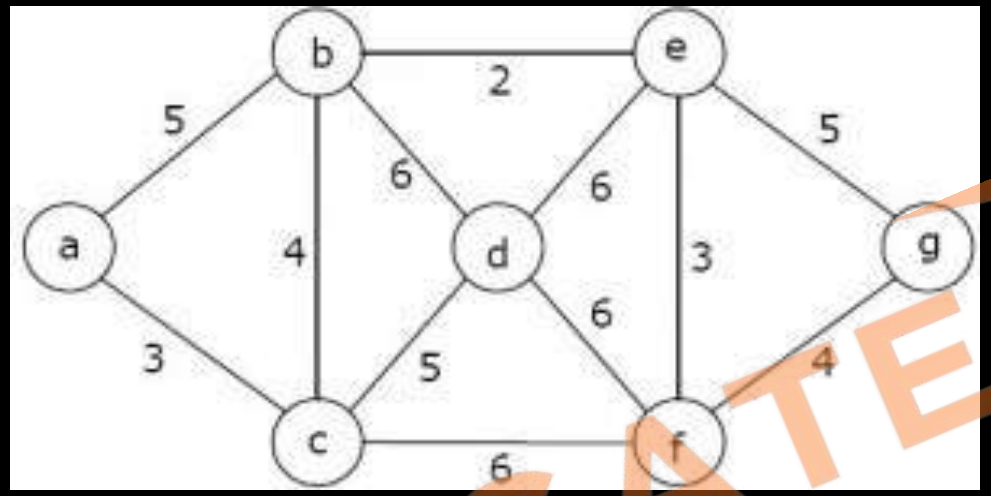
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Edsger Wybe Dijkstra** (11 May 1930 – 6 August 2002) was a Dutch computer scientist, programmer, software engineer, systems scientist, and science essayist. He received the 1972 Turing Award for fundamental contributions to developing programming languages.

KNOWLEDGEGATE

```
Minimum_Spanning_Tree (G, W, R)
{
        for each u ϵ V(G)
        {
                key[u] ← ∞
                ∏[u] ← NIL
        }
        Key[r] ← 0
        Q ←V[G]
        While (Q != ϕ)
        {
                u ← Extract-Min(Q)
                For each v ϵ adj[u]
                {
                        if (v ϵ Q and w(u, v) < key[v])
                        {
                                ∏[v] ← u
                                key[u] ←  w(u, v)
                        }
                }
        }
}
```



a	b	c	d	e	f	g

KNOWLEDGE GATE

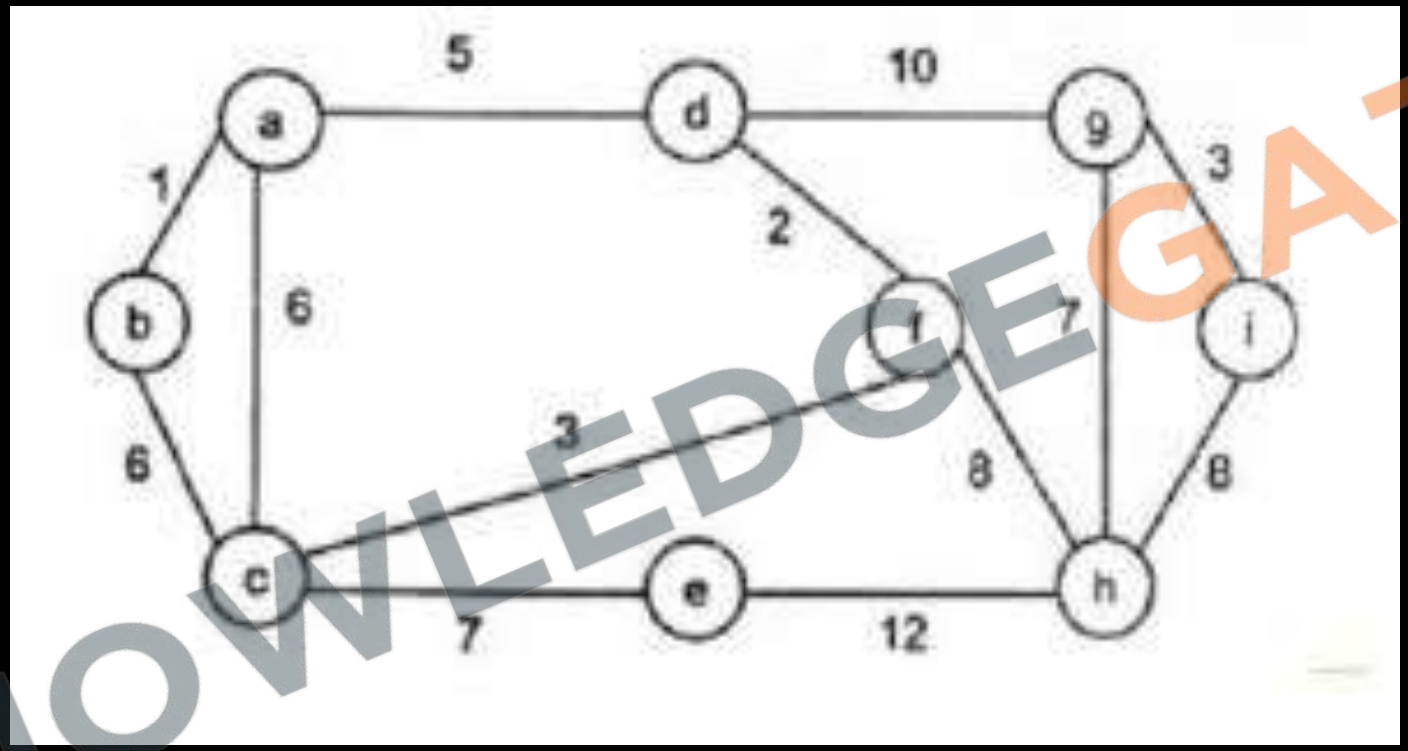
http://www.knowledgegate.in/GATE

```
MST-PRIM(G, w, r)
 1  for each u ∈ G.V
 2      u.key = ∞
 3      u.π = NIL
 4  r.key = 0
 5  Q = G.V
 6  while Q ≠ ∅
 7      u = EXTRACT-MIN(Q)
 8      for each v ∈ G.Adj[u]
 9          if v ∈ Q and w(u, v) < v.key
10              v.π = u
11              v.key = w(u, v)
```

The running time of Prim's algorithm depends on how we implement the min-priority queue Q. If we implement Q as a binary min-heap (see Chapter 6), we can use the BUILD-MIN-HEAP procedure to perform lines 1–5 in $O(V)$ time. The body of the **while** loop executes $|V|$ times, and since each EXTRACT-MIN operation takes $O(\lg V)$ time, the total time for all calls to EXTRACT-MIN is $O(V \lg V)$. The **for** loop in lines 8–11 executes $O(E)$ times altogether, since the sum of the lengths of all adjacency lists is $2|E|$. Within the **for** loop, we can implement the test for membership in Q in line 9 in constant time by keeping a bit for each vertex that tells whether or not it is in Q, and updating the bit when the vertex is removed from Q. The assignment in line 11 involves an implicit DECREASE-KEY operation on the min-heap, which a binary min-heap supports in $O(\lg V)$ time. Thus, the total time for Prim's algorithm is $O(V \lg V + E \lg V) = O(E \lg V)$, which is asymptotically the same as for our implementation of Kruskal's algorithm.

KNOWLEDGE GATE

Q For the undirected, weighted graph given below, which of the following sequences of edges represents a correct execution of Prim's algorithm to construct a Minimum Spanning Tree?



(A) (a, b), (d, f), (f, c), (g, i), (d, a), (g, h), (c, e), (f, h)
(B) (c, e), (c, f), (f, d), (d, a), (a, b), (g, h), (h, f), (g, i)
(C) (d, f), (f, c), (d, a), (a, b), (c, e), (f, h), (g, h), (g, i)
(D) (h, g), (g, i), (h, f), (f, c), (f, d), (d, a), (a, b), (c, e)

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Ch-6

Single Source Shortest Paths

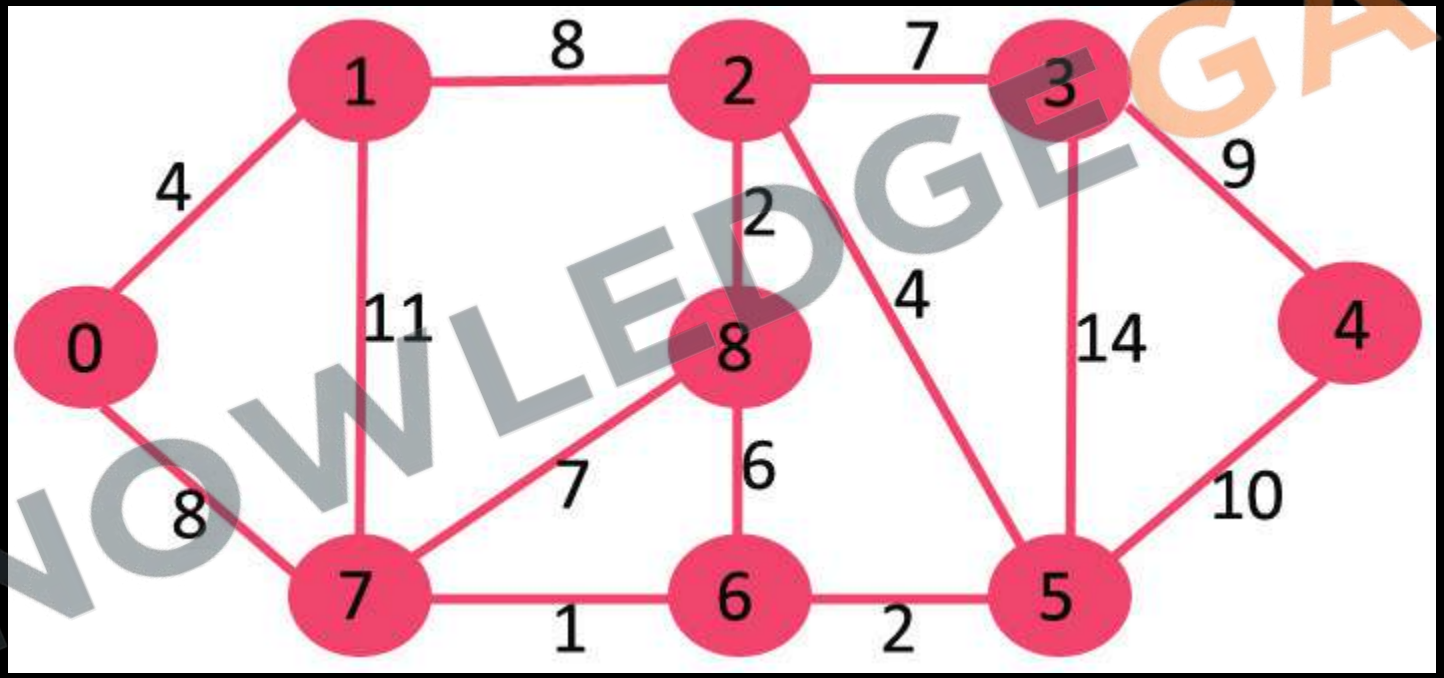
Dijkstra's and Bellman Ford Algorithms

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Single Source Shortest Path

- In graph theory, the **shortest path problem** is the problem of finding a path between two vertices (or nodes) in a graph such that the sum of the weights of its constituent edges is minimized.



KNOWLEDGEGATE

- **Edsger Wybe Dijkstra** (11 May 1930 – 6 August 2002) was a Dutch computer scientist, programmer, software engineer, systems scientist, and science essayist. He received the 1972 Turing Award for fundamental contributions to developing programming languages.

KNOWLEDGEGATE

- One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path.

- As I said, it was a twenty-minute invention. In fact, it was published in ’59, three years later. The publication is still readable, it is, in fact, quite nice.

KNOWLEDGEGATE

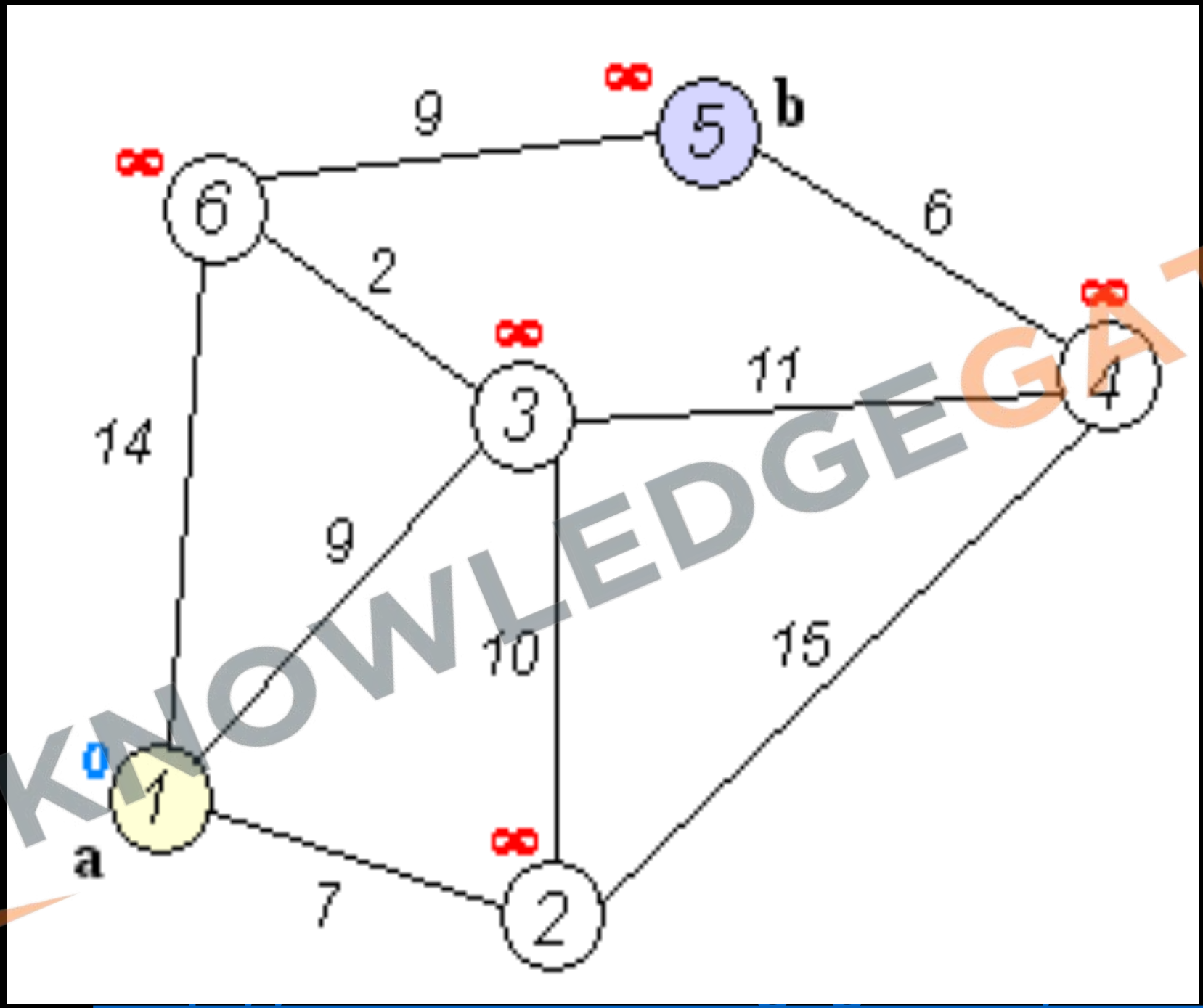
- One of the reasons that it is so nice was that I designed it without pencil and paper. I learned later that one of the advantages of designing without pencil and paper is that you are almost forced to avoid all avoidable complexities.

- Eventually that algorithm became, to my great amazement, one of the cornerstones of my fame.

- A widely used application of shortest path algorithm is network routing protocols, most notably IS-IS (Intermediate System to Intermediate System) and Open Shortest Path First (OSPF).

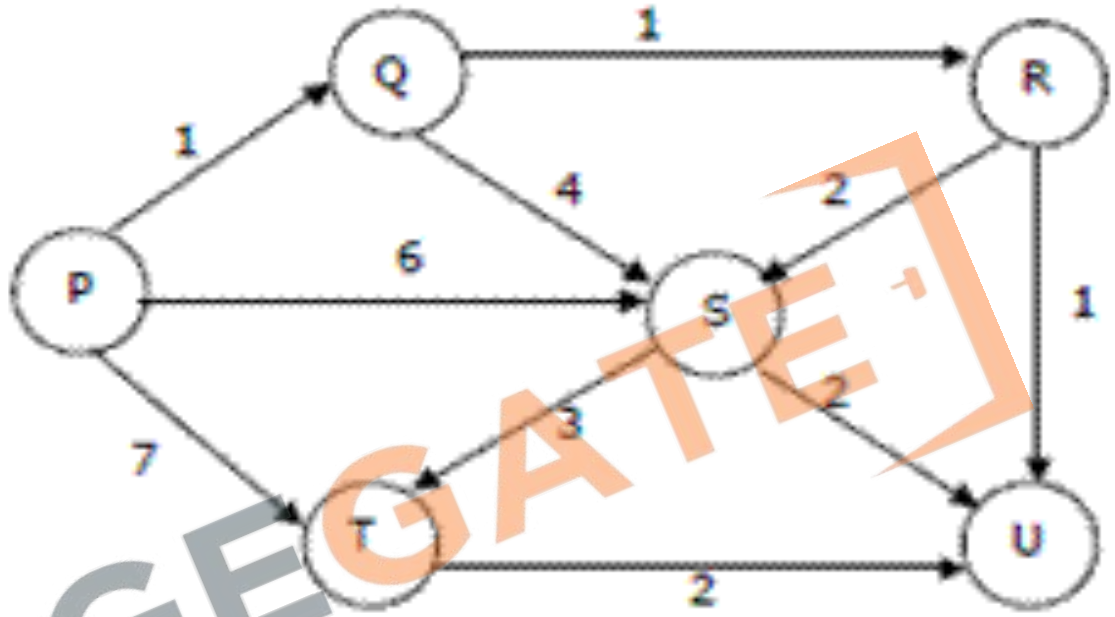
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE



KNOWLEDGEGATE

```
Dijkstra algorithm (G, W, S)
{
    initialize-Single-source (G, S)
    S ← ϕ
    Q ← V[G]
    While (Q != ϕ)
    {
        u ← extract-min (Q)
        S ← S U {u}
        for each vertex v ϵ adj(u)
        {
            relax (u, v, w)
        }
    }
}
```



P	Q	R	S	T	U

KNOWLEDGEGATE

```
Initialize_Single_Source (G, S)
{
    for each vertex v ϵ V[G]
    {
        d[v] ← ∞
        ∏[v] ← NIL
    }
    d[S] ← 0
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
Relax (u, v, w)
{
    if(d[v] > d[u] + w (u, v))
    {
        d[v] ← d[u] + w (u, v)
        ∏[v] ← u
    }
}
```

KNOWLEDGEGATE

- Guarantee to find optimal solution in a connected graph with positive weights.
- Can fail on graph with negative weights.

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each vertex v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

How fast is Dijkstra's algorithm? It maintains the min-priority queue Q by calling three priority-queue operations: INSERT (implicit in line 3), EXTRACT-MIN (line 5), and DECREASE-KEY (implicit in RELAX, which is called in line 8). The algorithm calls both INSERT and EXTRACT-MIN once per vertex. Because each vertex $u \in V$ is added to set S exactly once, each edge in the adjacency list $Adj[u]$ is examined in the **for** loop of lines 7–8 exactly once during the course of the algorithm. Since the total number of edges in all the adjacency lists is $|E|$, this **for** loop iterates a total of $|E|$ times, and thus the algorithm calls DECREASE-KEY at most $|E|$ times overall. (Observe once again that we are using aggregate analysis.)

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each vertex v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

The running time of Dijkstra's algorithm depends on how we implement the min-priority queue. Consider first the case in which we maintain the min-priority queue by taking advantage of the vertices being numbered 1 to $|V|$. We simply store $v.d$ in the vth entry of an array. Each INSERT and DECREASE-KEY operation takes $O(1)$ time, and each EXTRACT-MIN operation takes $O(V)$ time (since we have to search through the entire array), for a total time of $O(V^2 + E) = O(V^2)$.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each vertex v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

If the graph is sufficiently sparse—in particular, $E = o(V^2/\lg V)$—we can improve the algorithm by implementing the min-priority queue with a binary min-heap. (As discussed in Section 6.5, the implementation should make sure that vertices and corresponding heap elements maintain handles to each other.) Each EXTRACT-MIN operation then takes time $O(\lg V)$. As before, there are $|V|$ such operations. The time to build the binary min-heap is $O(V)$. Each DECREASE-KEY operation takes time $O(\lg V)$, and there are still at most $|E|$ such operations. The total running time is therefore $O((V + E) \lg V)$, which is $O(E \lg V)$ if all vertices are reachable from the source. This running time improves upon the straightforward $O(V^2)$-time implementation if $E = o(V^2/\lg V)$.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each vertex v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

We can in fact achieve a running time of $O(V \lg V + E)$ by implementing the min-priority queue with a Fibonacci heap (see Chapter 19). The amortized cost of each of the $|V|$ EXTRACT-MIN operations is $O(\lg V)$, and each DECREASE-KEY call, of which there are at most $|E|$, takes only $O(1)$ amortized time. Historically, the development of Fibonacci heaps was motivated by the observation that Dijkstra's algorithm typically makes many more DECREASE-KEY calls than EXTRACT-MIN calls, so that any method of reducing the amortized time of each DECREASE-KEY operation to $o(\lg V)$ without increasing the amortized time of EXTRACT-MIN would yield an asymptotically faster implementation than with binary heaps.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Problem with Dijkstra's algorithm

- Negative Weight Edge

KNOWLEDGEGATE

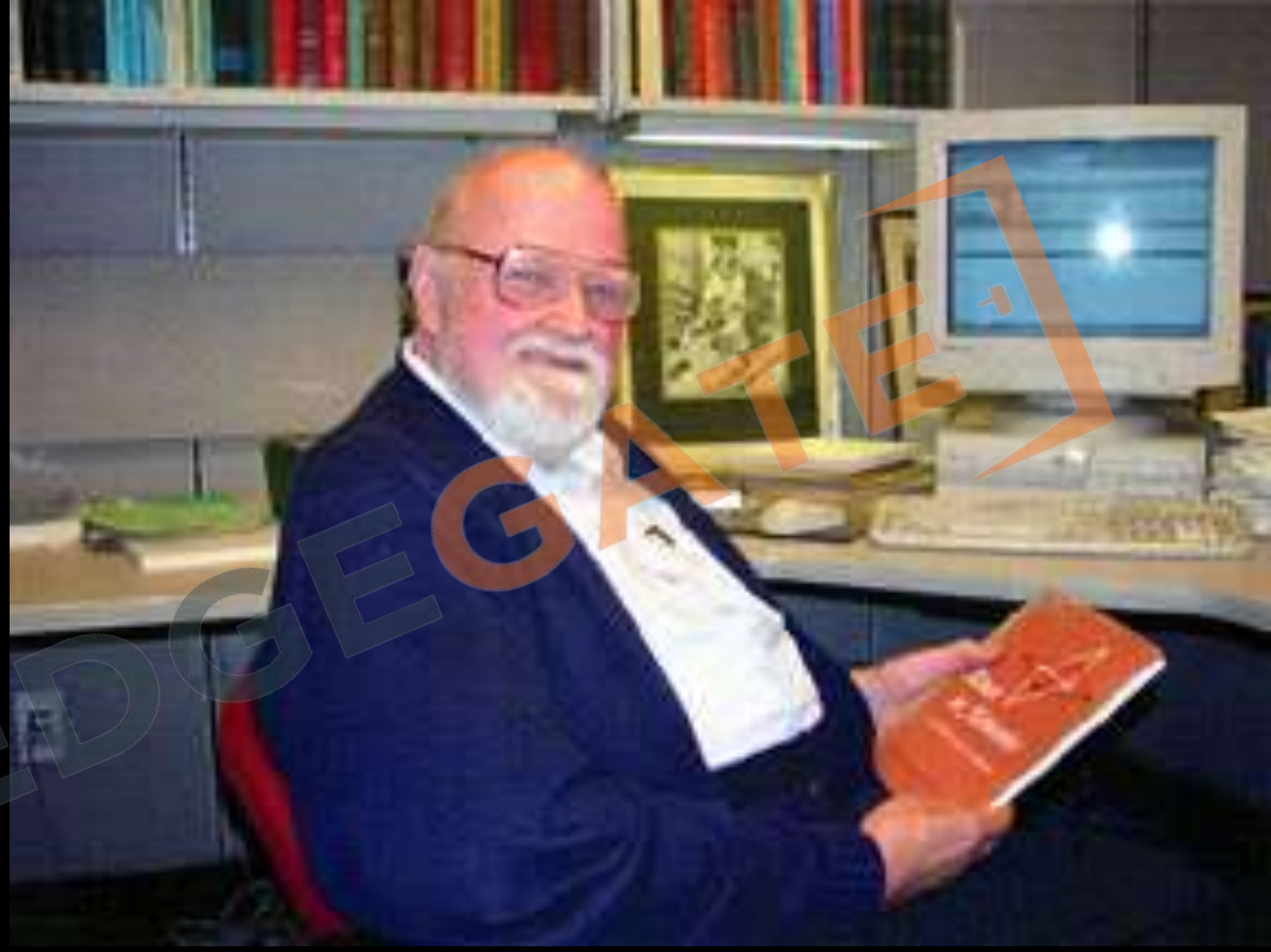
http://www.knowledgegate.in/GATE

Bellman–Ford Algorithm

- The **Bellman–Ford algorithm** is an algorithm that computes shortest paths from a single source vertex to all of the other vertices in a weighted digraph.
- It is slower than Dijkstra's algorithm for the same problem, but more versatile, as it is capable of handling graphs in which some of the edge weights are negative numbers.
- The algorithm was first proposed by Alfonso Shimbel (1955), but is instead named after Richard Bellman and Lester Ford Jr., who published it in 1958 and 1956, respectively.

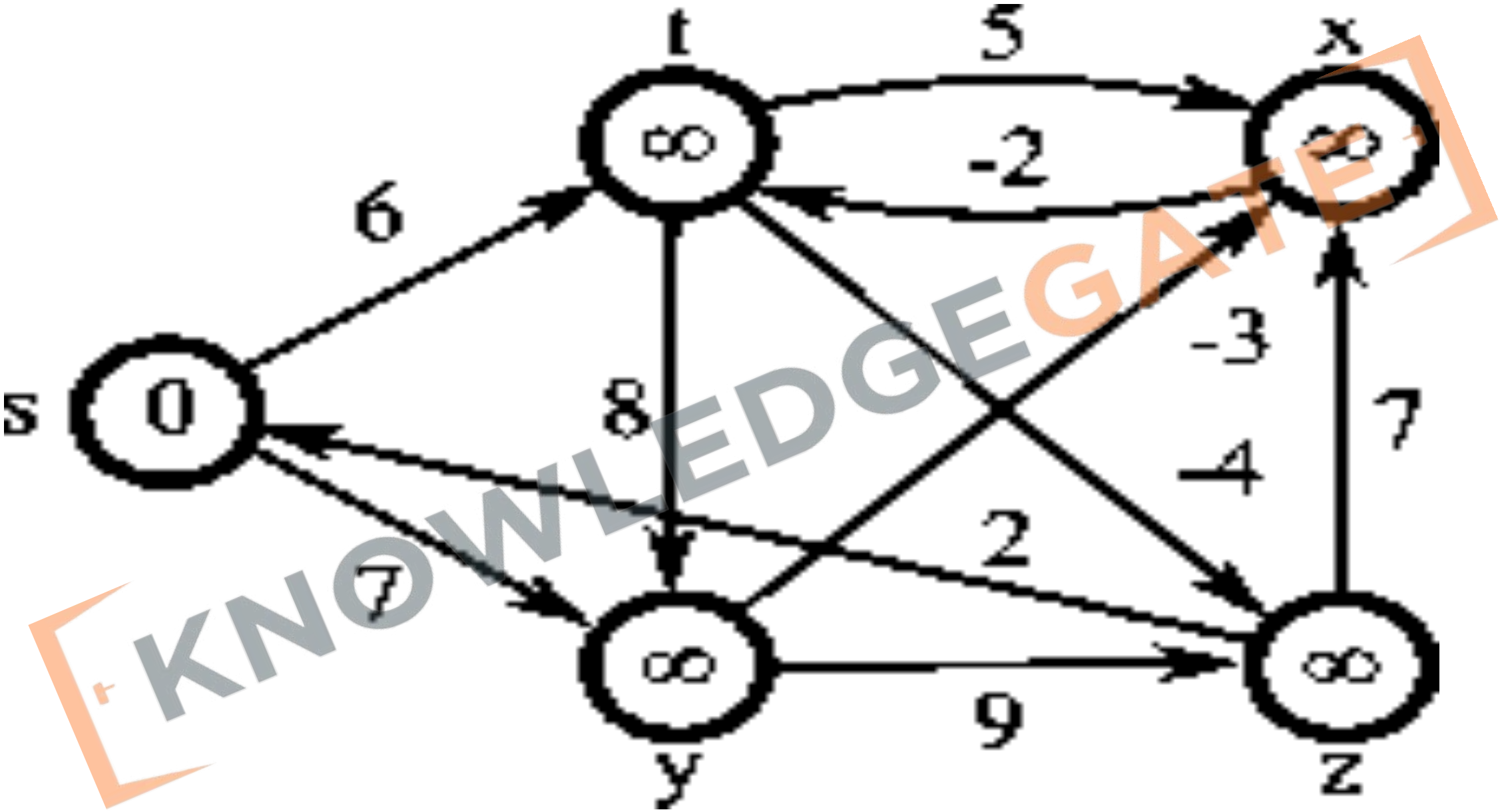
KNOWLEDGEGATE

Richard E. Bellman



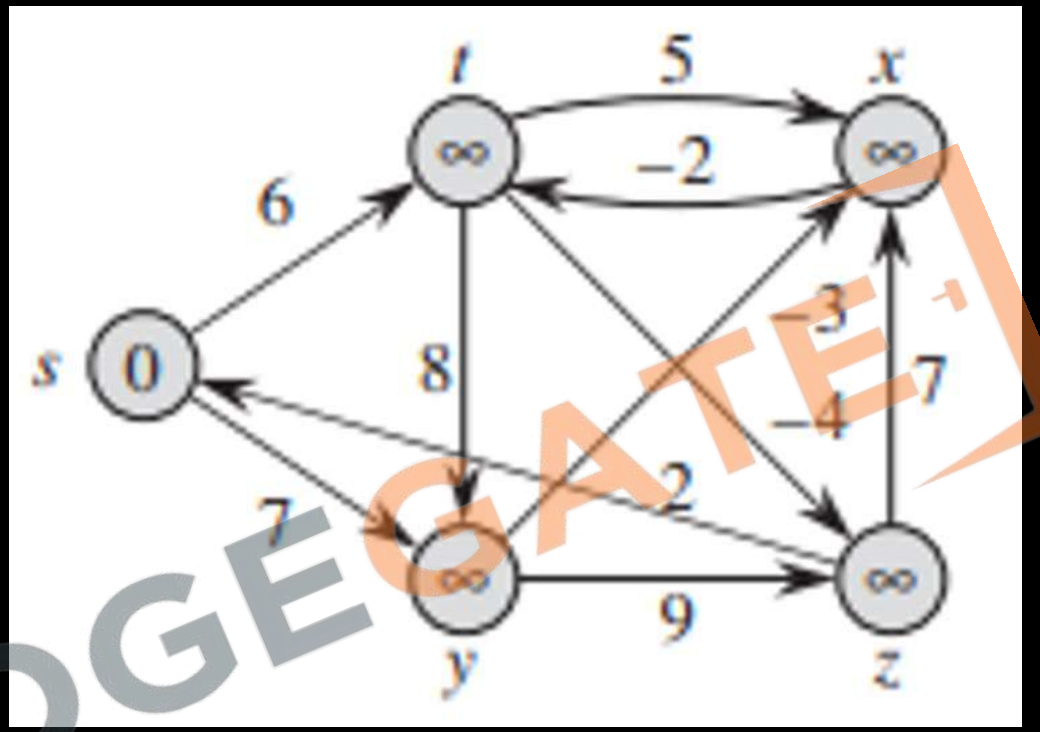
L. R. Ford Jr

KNOWLEDGEGATE



KNOWLEDGEGATE

```
Bellman_ford (G, W, S)
{
    initialize-Single-Source (G, S)
    for i ← 1 to |V(G)| - 1
    {
        for each edge (u, v) ϵ E(G)
        {
            Relax(u, v, w)
        }
    }
    for each edge (u, v) ϵ E(G)
    {
        if(d[v] > d[u] + w (u, v))
        {
            Return false
        }
    }
}
```



S	T	X	Y	Z

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
Initialize_Single_Source (G, S)
{
    for each vertex v ϵ V[G]
    {
        d[v] ← ∞
        ∏[v] ← NIL
    }
    d[S] ← 0
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
Relax (u, v, w)
{
    if(d[v] ← d[u] + w (u, v))
    {
        d[v] ← d[u] + w (u, v)
        ∏[v] ← u
    }
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

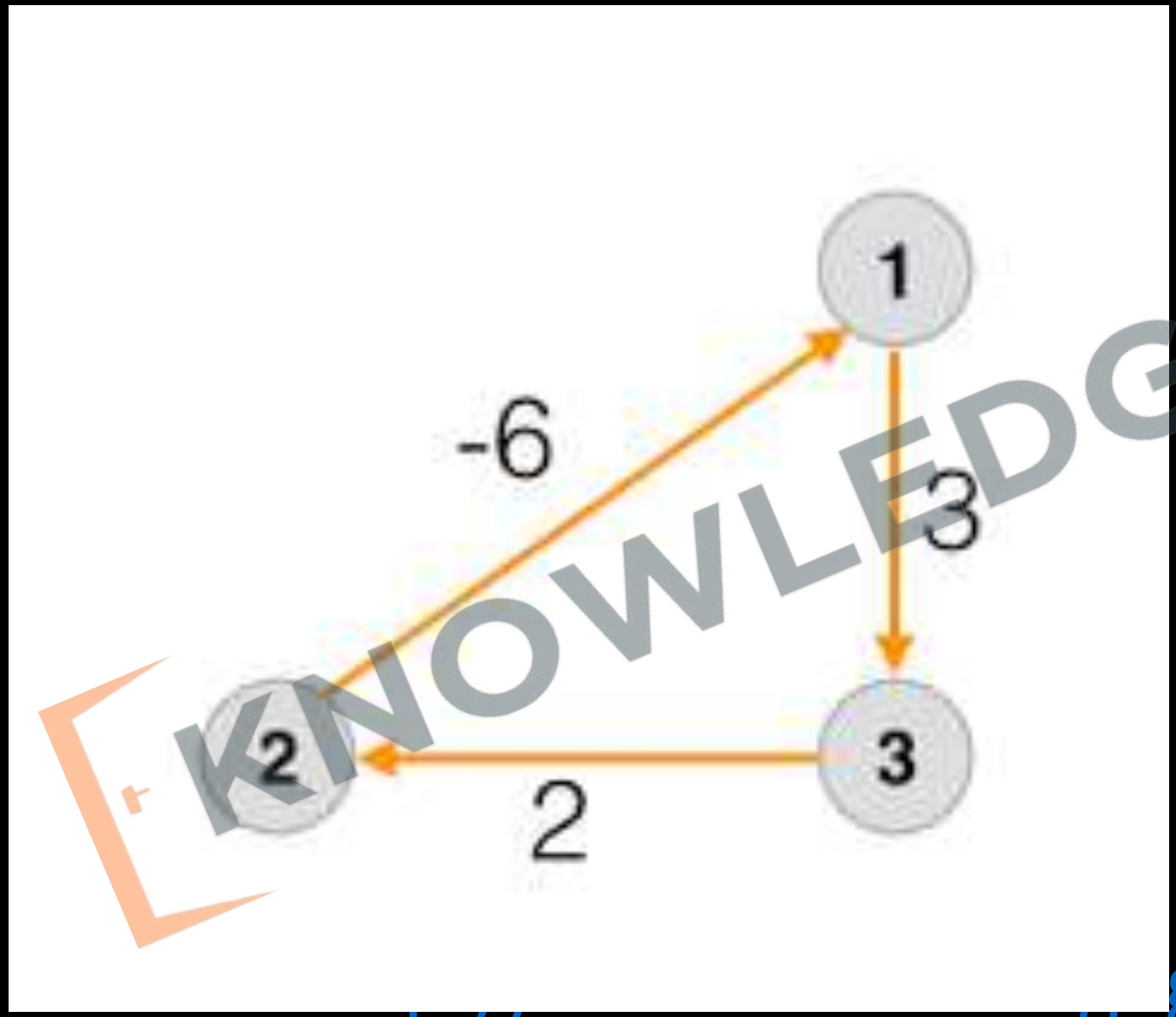
```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i = 1 to |G.V| − 1
3       for each edge (u, v) ∈ G.E
4            RELAX(u, v, w)
5  for each edge (u, v) ∈ G.E
6       if v.d > u.d + w(u, v)
7            return FALSE
8  return TRUE
```

The Bellman-Ford algorithm runs in time $O(VE)$, since the initialization in line 1 takes $\Theta(V)$ time, each of the $|V| - 1$ passes over the edges in lines 2–4 takes $\Theta(E)$ time, and the **for** loop of lines 5–7 takes $O(E)$ time.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Bellman–Ford algorithm with negative weight cycle



KNOWLEDGEGATE

gate.in/GATE

Ch-7

Dynamic Programming

with Examples Such as Knapsack. All Pair Shortest Paths – Warshal's and Floyd's Algorithms, Resource Allocation Problem. Backtracking, Branch and Bound with Examples Such as Travelling Salesman Problem, Graph Coloring, n-Queen Problem, Hamiltonian Cycles and Sum of Subsets.

KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

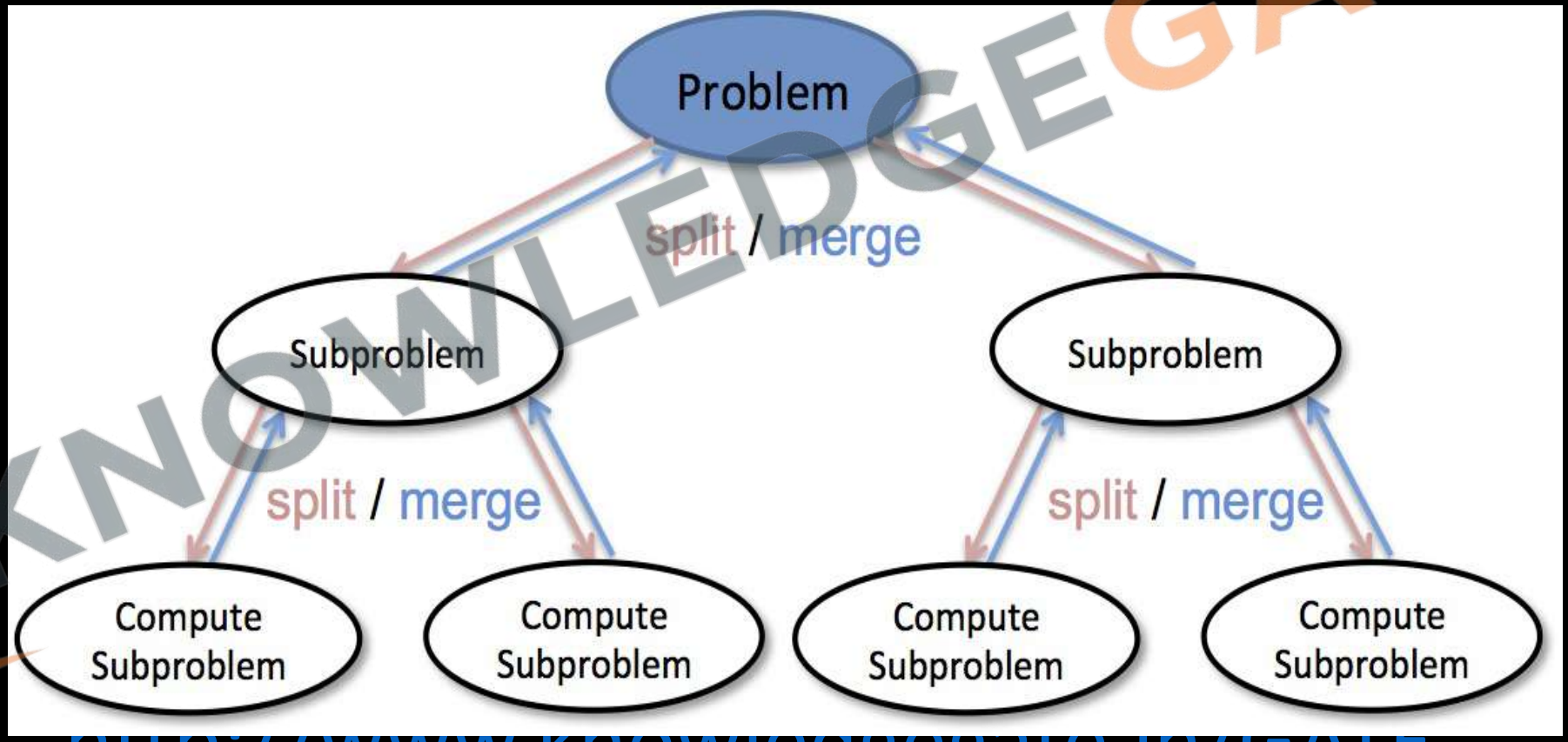
अगर आप अपने Past से कुछ सीख नहीं सकते तो जीवनभर छोटे काम ही करते रहेंगे।

-Dynamic Programming

KNOWLEDGEGATE

Dynamic Programming

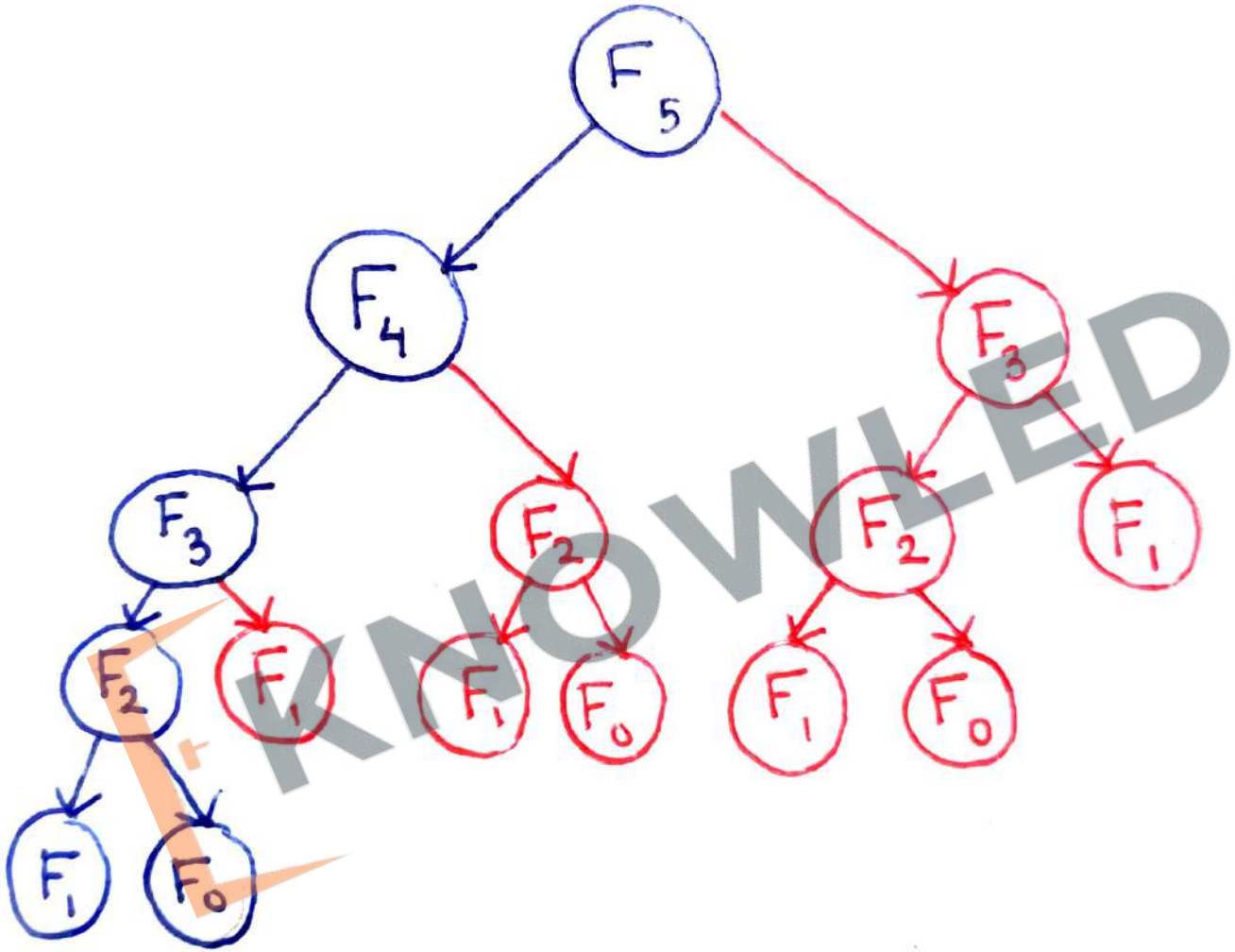
- Divide and conquer partition the problem into independent subproblem, solve the subproblems recursively and then combine their solutions to solve the original problems.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

$$F(0) = 0$$
$$F(1) = 1$$
$$F(n) = F(n-1) + F(n-2)$$

n	0	1	2	3	4	5
F(n)						



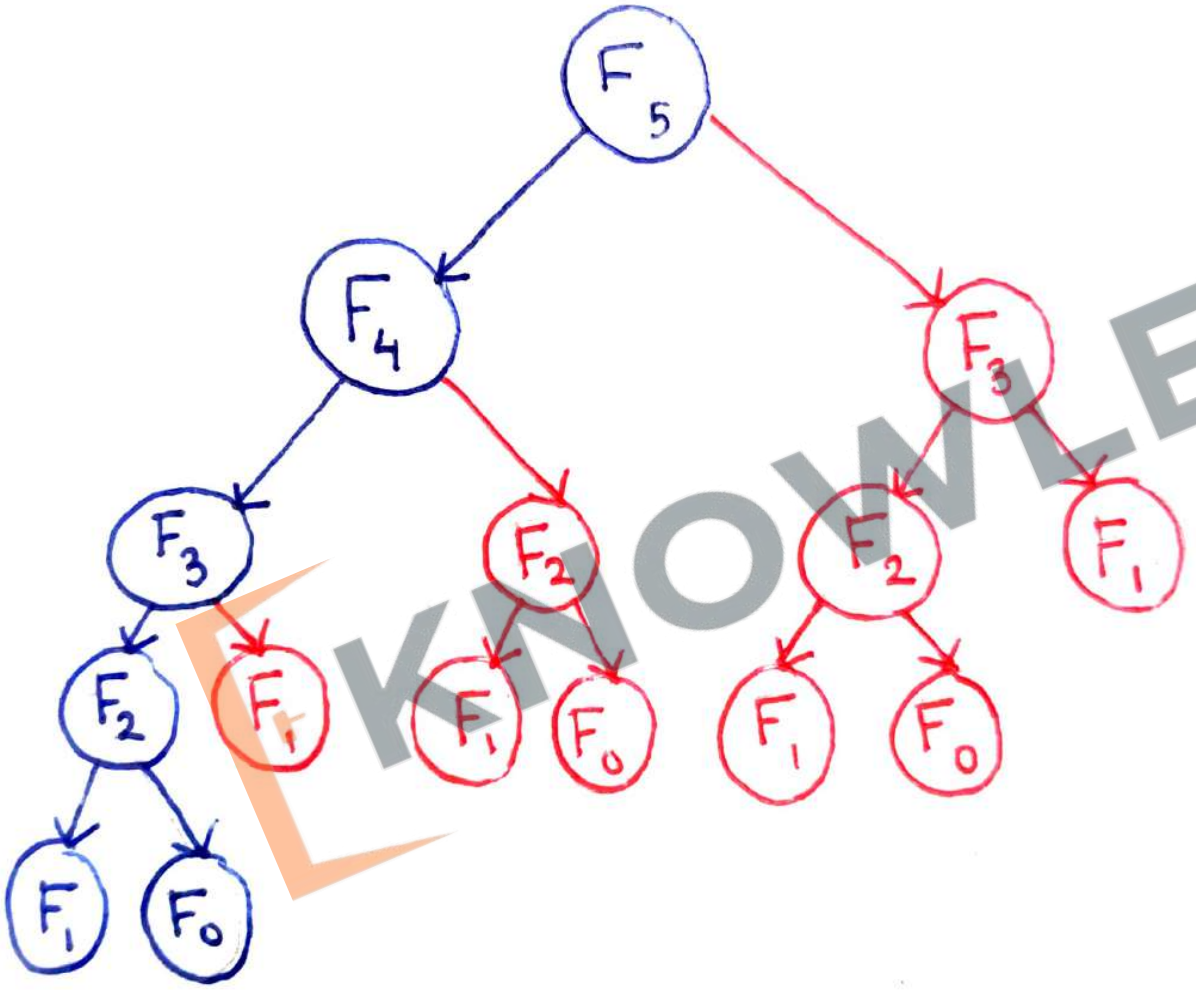
KNOWLEDGEGATE
egate.in/GATE

- Dynamic programming is like the divide and conquer method, solve problems by combining the solutions to the subproblems.
- In contrast, dynamic programming is applicable when the subproblems are not independent, i.e. when subproblems share subsubproblems.

KNOWLEDGEGATE

- A dynamic-programming algorithm solves every subsubproblems just one and then saves its answer in a table there by avoiding the work of recomputing the answer every time the subproblem is encountered.

O(N)



n	f(n)
0	1
1	1
2	2
3	3
4	5
5	8
6	13
7	21
8	34
9	55
10	89
11	144
12	233
13	377
14	610
15	987
16	1597
17	2584
18	4181

KNOWLEDGEGATE

programming

dynamic
programming

KNOWLEDGEGATE

n/GATE

- Dynamic programming is typically applied to optimization problems. In such case there can be many possible solutions. Each solution has a value, and we wish to find a solution with the optimal value (minimum, maximum).

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- There are four steps of dynamic programming
  - Characterize the solution of an optimal solution.
  - Recursively define the value of an optimal solution.
  - Compute the value of an optimal solution in a bottom-up-fashion.
  - Construct an optimal solution from computed information.

भाई असली बात तो example से समझ आएगी

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

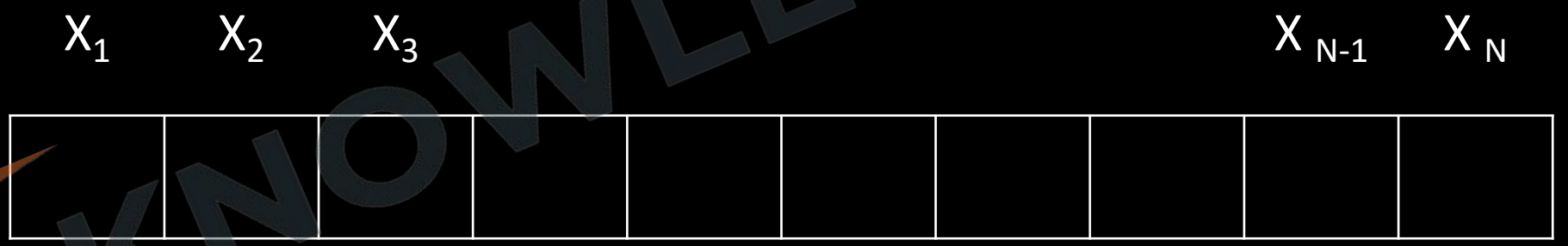
0/1 Knap Sack Problem

- The **knapsack problem** or **rucksack problem** is a problem in combinatorial optimization: Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

KNOWLEDGEGATE
www.knowledgegate.in/GATE

Problem Definition

- More formally there are n number of objects O_1, O_2, O_3…O_n, each object has a weight associated with its W_i, and a profit associated with it P_i, we can take x_i either 0 or 1.
- Weight Condition $\sum_{i=1}^{n} Wi \,.\, Xi$ <= M
- Profit $\sum_{i=1}^{n} Pi \,.\, Xi$



KNOWLEDGEGATE

Object	O_1	O_2	O_3	O_4
Profit	1	2	5	6
Weight	2	3	4	5

$KS(n,w) = 0$ if $n=0$ or $w=0$

$KS(n,w) = KS(n-1,w)$ if $wt[n] > W$

$KS(n,w) = \max\{KS(n-1, w - wt[n]) + P[n],\ KS(n-1,w)\}$

$n = 4 \quad w = 8$

wt	p	W=0	W=1	W=2	W=3	W=4	W=5	W=6	W=7	W=8
0	0	0	0	0	0	0	0	0	0	0
1	2	0	0	1	1	1	1	1	1	1
2	3	0	0	1	2	2	3	3	3	3
5	4	0	0	1	2	5	5	6	7	7
6	5	0	0	1	2	5	6	6	7	8

KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

Floyd warshall problem

- The Floyd–Warshall algorithm was published in its currently recognized form by Robert Floyd in 1962. However, it is essentially the same as algorithms previously published by Stephen Warshall in 1962.

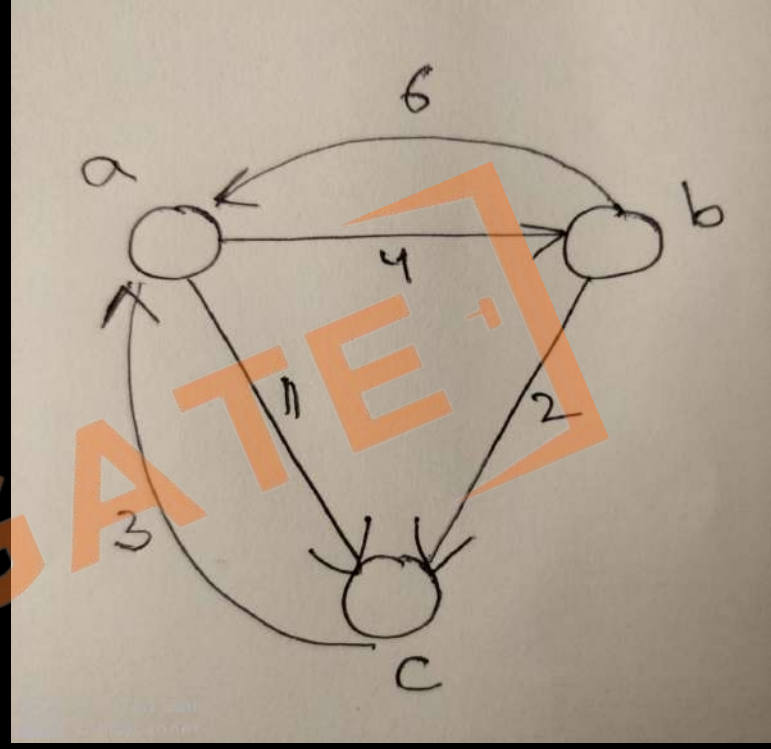
KNOWLEDGE GATE
http://www.knowledgegate.in/GATE

Floyd warshall problem

- The Floyd-Warshall algorithm is used to find the shortest paths between all pairs of vertices in a weighted graph.
- It uses a dynamic programming approach to incrementally improve the solution by considering all possible paths.

KNOWLEDGEGATE

Floyd warshall problem



$D^0 = \begin{matrix} & a & b & c \\ a & & & \\ b & & & \\ c & & & \end{matrix}$ $\Pi^0 = \begin{matrix} & a & b & c \\ a & & & \\ b & & & \\ c & & & \end{matrix}$

$D^a = \begin{matrix} & a & b & c \\ a & & & \\ b & & & \\ c & & & \end{matrix}$ $\Pi^a = \begin{matrix} & a & b & c \\ a & & & \\ b & & & \\ c & & & \end{matrix}$

$D^b = \begin{matrix} & a & b & c \\ a & & & \\ b & & & \\ c & & & \end{matrix}$ $\Pi^b = \begin{matrix} & a & b & c \\ a & & & \\ b & & & \\ c & & & \end{matrix}$

$D^c = \begin{matrix} & a & b & c \\ a & & & \\ b & & & \\ c & & & \end{matrix}$ $\Pi^c = \begin{matrix} & a & b & c \\ a & & & \\ b & & & \\ c & & & \end{matrix}$

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Algorithm Steps**:
  - Initially, the distance between all pairs of vertices is assumed to be infinity, except for the distance from a vertex to itself, which is zero.
  - The algorithm then iteratively updates the distance matrix to include the shortest path using at most k vertices, where k goes from 1 to the number of vertices in the graph.
  - If a shorter path is found, the distance matrix is updated accordingly.

- **Time Complexity**: It has a time complexity of O(n^3), where n is the number of vertices in the graph.

KNOWLEDGEGATE

Sum of subset problem

- Given a set of non-negative integers, and a value *sum*, determine if there is a subset of the given set with sum equal to given *sum*.

Q Consider a set of non-negative integer S = {2, 3, 7, 8, 10}, find if there is a sub set of S with sum equal to 14?

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
2															
3															
7															
8															
10															

KNOWLEDGE GATE

- The subset-sum problem is defined as follows. Given a set of n positive integers, $S = \{a_1, a_2, a_3, \ldots, a_n\}$ and positive integer W, is there a subset of S whose elements sum to W
- A dynamic program for solving this problem uses a 2-dimensional Boolean array X, with n rows and W+1 column. X[i, j],1 <= i <= n, 0 <= j <= W, is TRUE if and only if there is a subset of $\{a_1, a_2, \ldots, a_i\}$ whose elements sum to j.

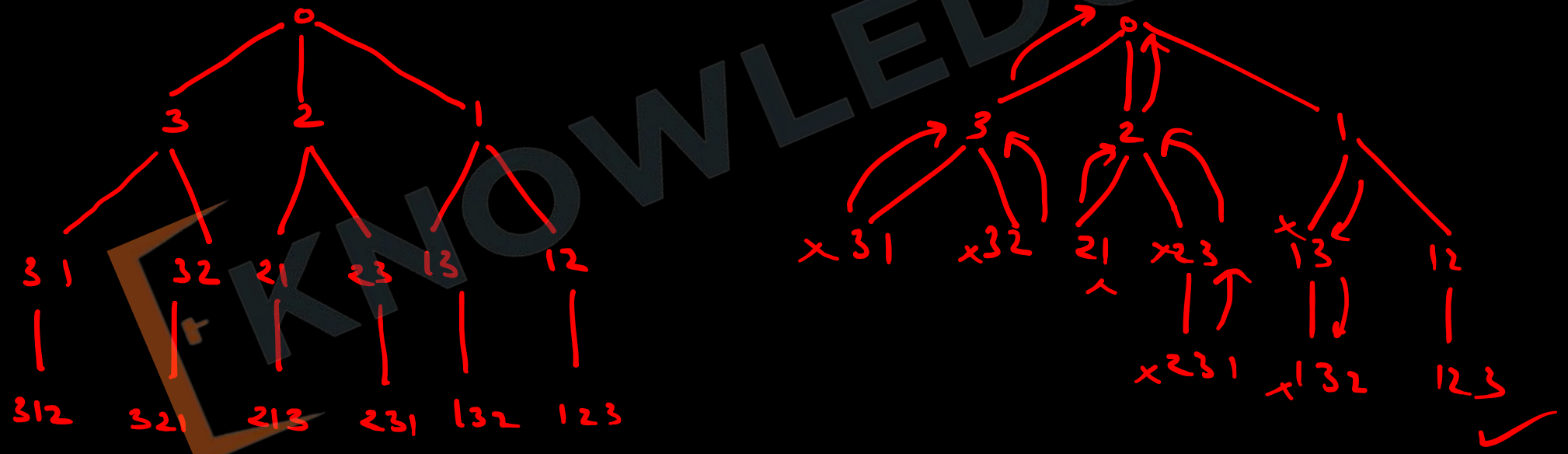
- the following is valid for 2 <= i <= n and ai <= j <= W

  - $X[i, j] = X[i - 1, j] \vee X[i - 1, j - a_i]$

KNOWLEDGEGATE

Backtracking

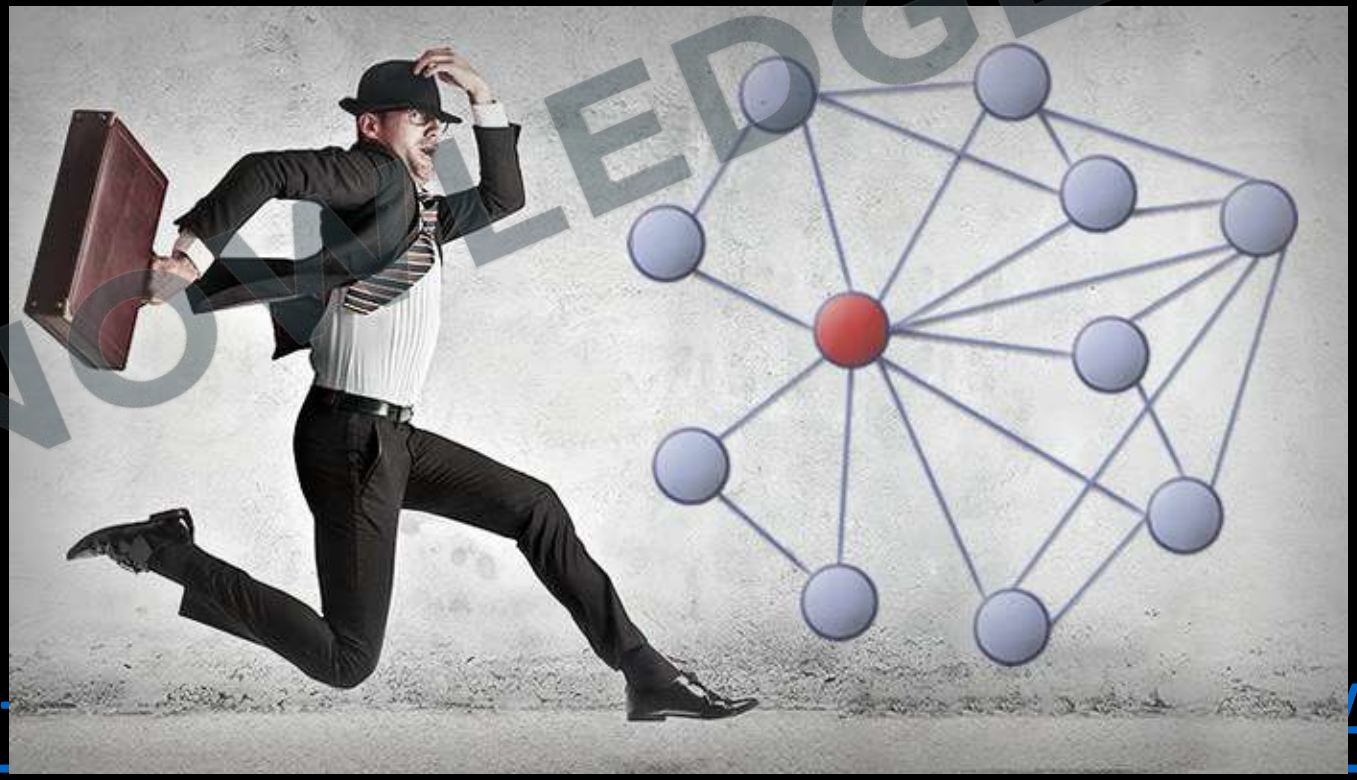
- Backtracking is a programming method used to solve problems recursively by trying to build a solution incrementally, one piece at a time, removing those solutions that fail to satisfy the constraints (Bounding function)of the problem at any point of time.
- Backtracking uses brute force approach generating a State apace tree, following depth first search.
- Backtracking is not for optimality problems; it is use in general where we have multiple solution, and we want all those solutions.



KNOWLEDGE GATE

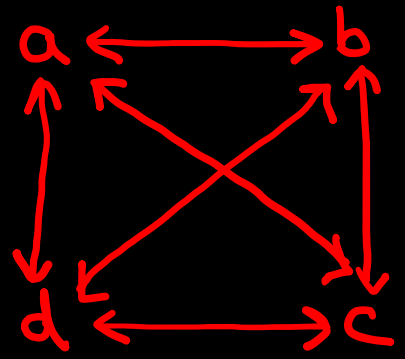
Travelling Salesman Problem

- The Travelling Salesman Problem (TSP) is a classic problem in computer science and operations research.
- **Problem Definition**: The TSP involves finding the shortest possible route that visits a set of cities and returns to the origin city. It's an NP-hard problem in computational complexity theory. As the number of cities increases, the number of possible routes increases factorially, making the problem computationally intensive.

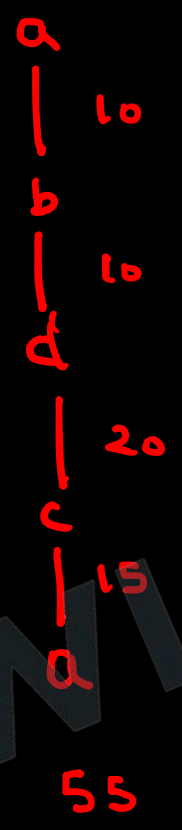


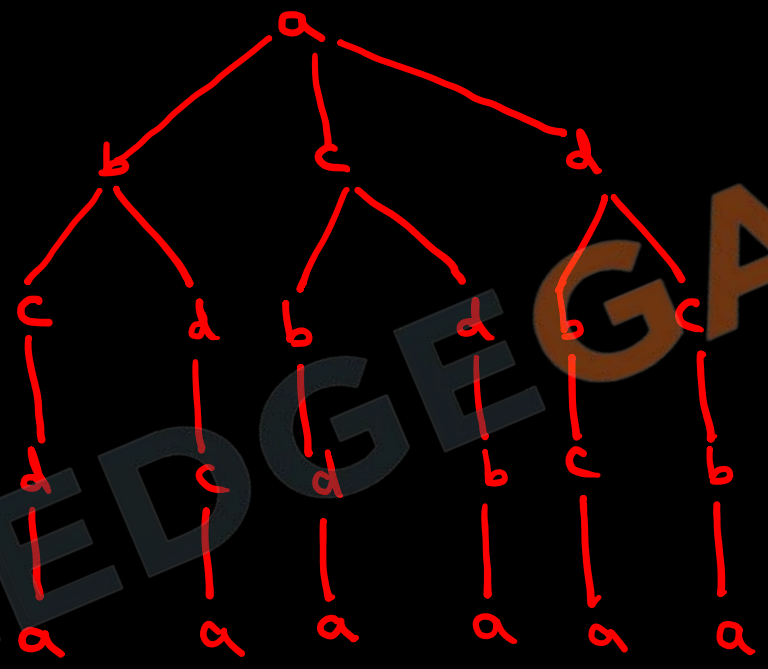
KNOWLEDGEGATE

Travelling Salesman Problem



Greedy





	a	b	c	d
a	0	10	15	20
b	5	0	25	10
c	15	30	0	5
d	15	10	20	0

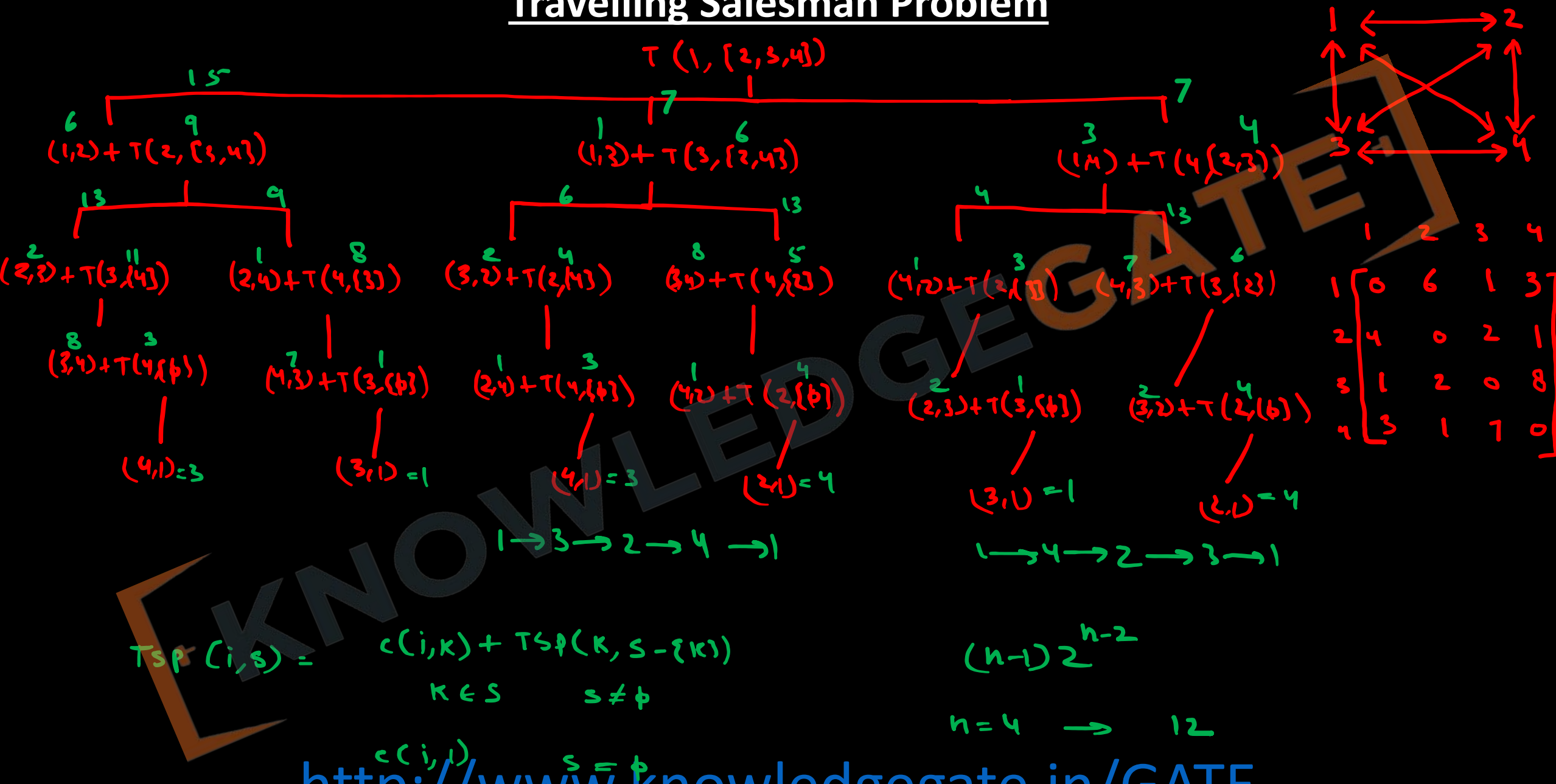
$O(n!)$

↓

$O(n^n)$

KNOWLEDGEGATE

Travelling Salesman Problem



$$Tsp(i,S) = \min_{k \in S,\ S \neq \phi} \{ c(i,k) + Tsp(k, S-\{k\}) \}$$

$$c(i,1), \quad S = \phi$$

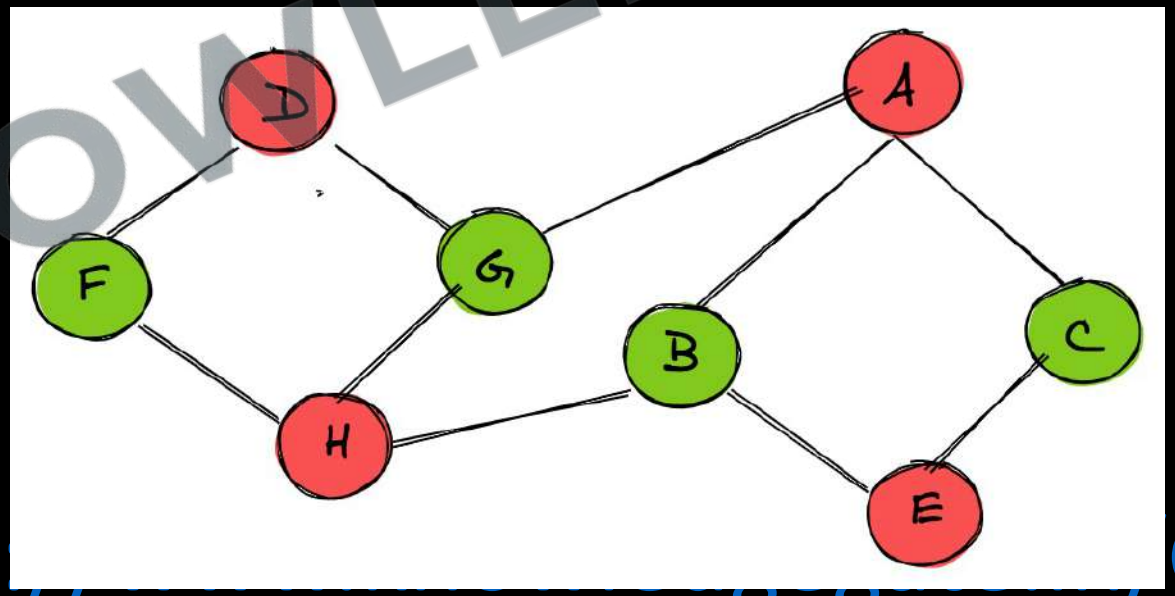
$$(n-1)\,2^{n-2}$$

$n = 4 \rightarrow 12$

KNOWLEDGEGATE

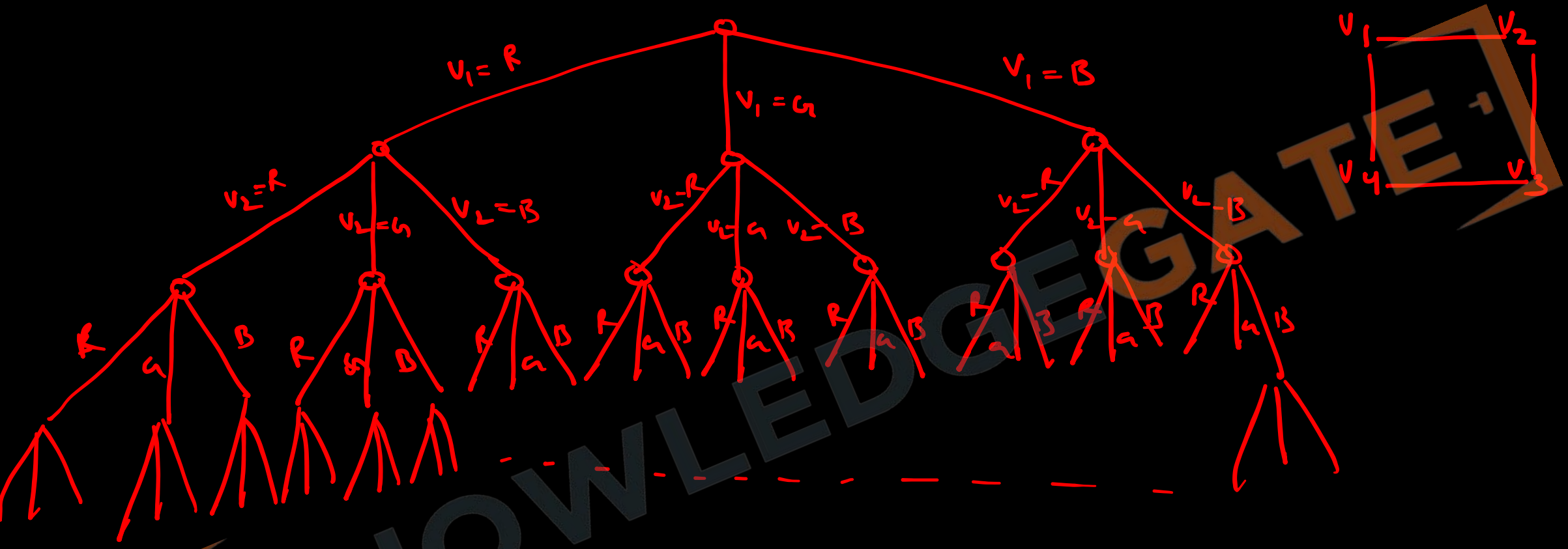
Graph colouring

- **Problem Definition**: It involves assigning colors to the vertices of a graph such that no two adjacent vertices share the same color. The goal is to minimize the number of colors used.
- **Applications**: It's used in scheduling, assigning frequencies in mobile networks, map coloring, and solving Sudoku puzzles.
- **Chromatic Number**: This is the minimum number of colors needed to color a graph. Determining this number is a central challenge of the problem.
- **NP-Hard Problem**: Like the Travelling Salesman Problem, graph coloring is NP-Hard for general graphs. This means finding the optimal solution is computationally challenging.
- **Backtracking Algorithms**: These are more exhaustive methods that try different color assignments and backtrack when a conflict is found.



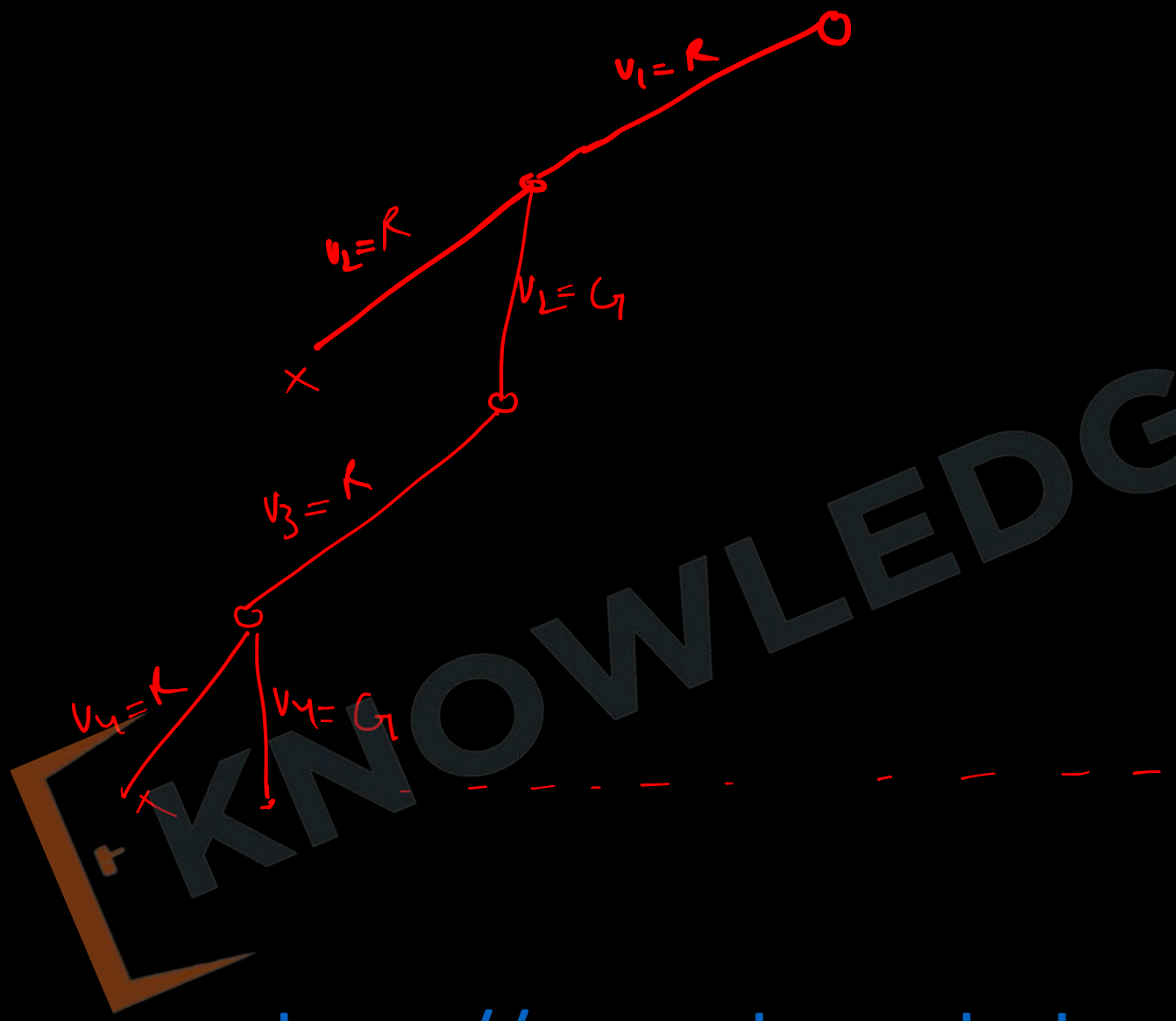
KNOWLEDGEGATE

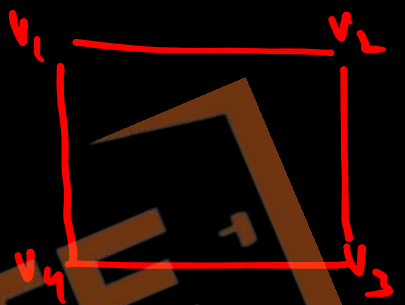
Graph colouring



KNOWLEDGEGATE

Graph colouring

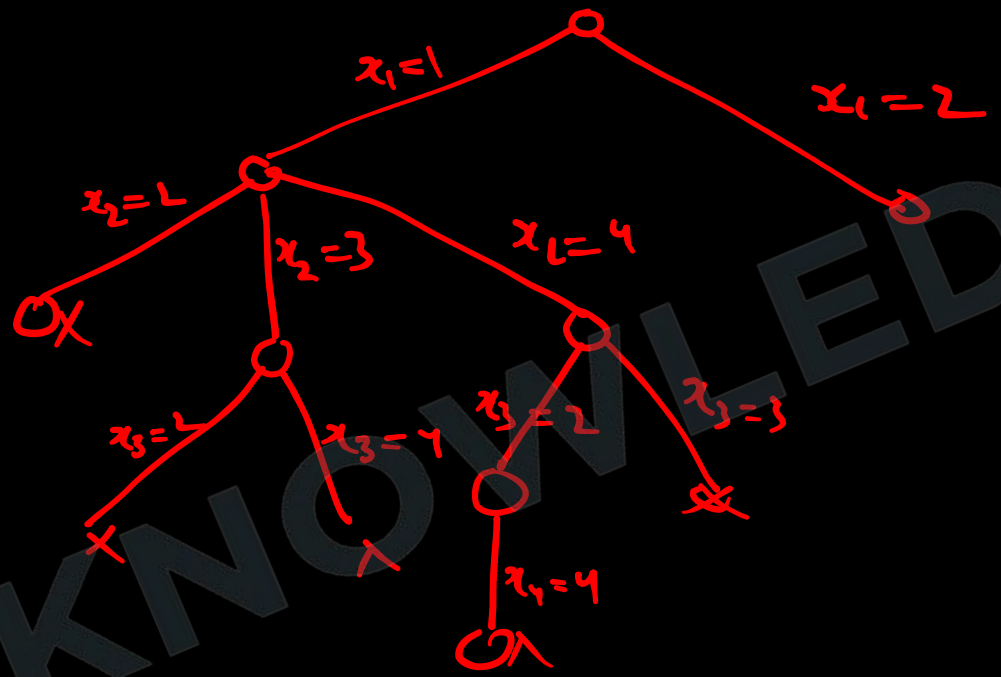


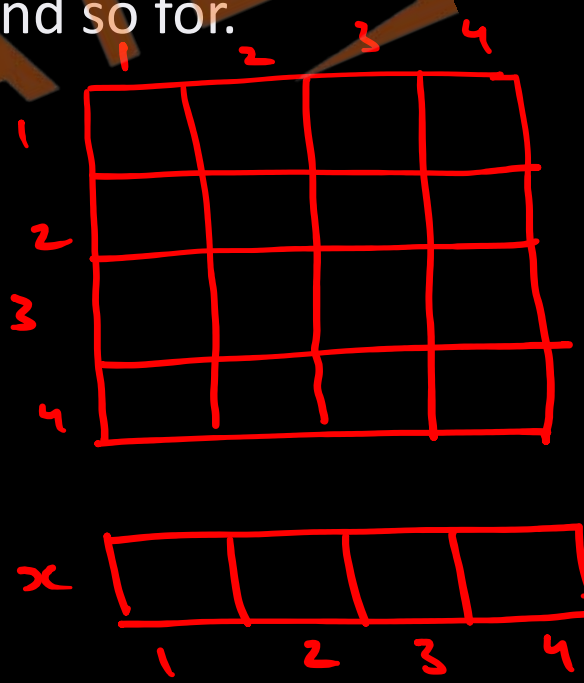


KNOWLEDGEGATE

N-Queens problem

- The N-Queens problem is a well-known puzzle in computer science where the aim is to place N chess queens on an N×N chessboard without any queen threatening another. This means no two queens can be in the same row, column, or diagonal.
  - First queen will in first row, second queen in second row and so on and so for.





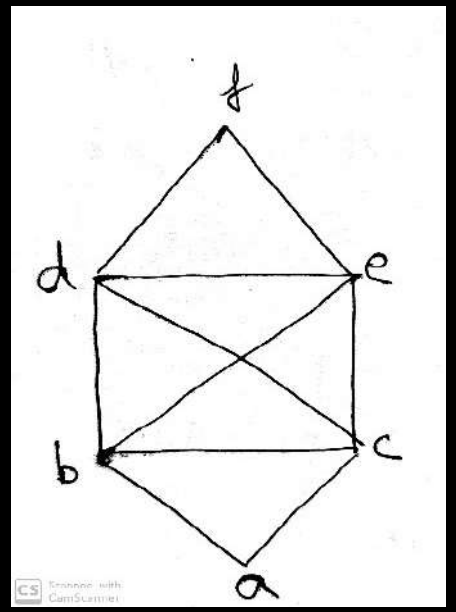
[KNOWLEDGEGATE]

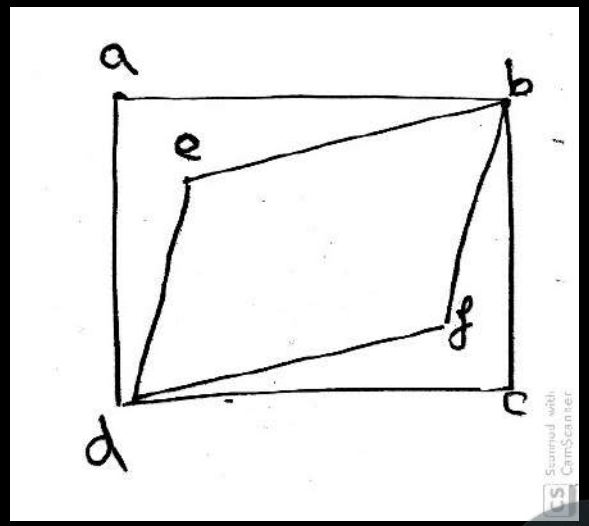
Hamiltonian

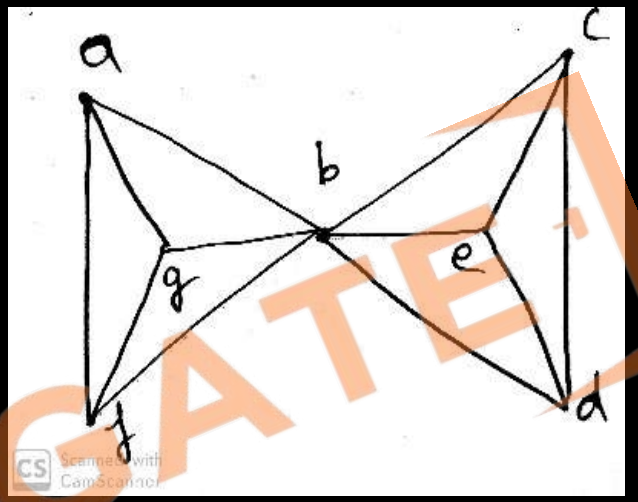
1. **Hamiltonian Graph:** - A Hamiltonian circuit in a connected graph is defined as a closed walk that traverses every vertex of G exactly once, except of course the starting vertex, at which the walk also terminates. A graph containing Hamiltonian circuit is called Hamiltonian graph.

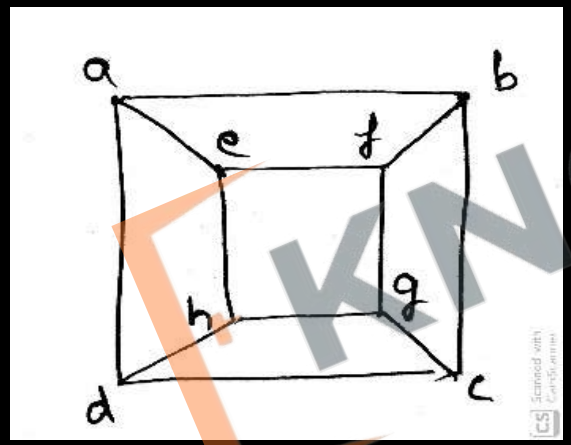
2. Finding weather a graph is Hamiltonian or not is a NPC problem.

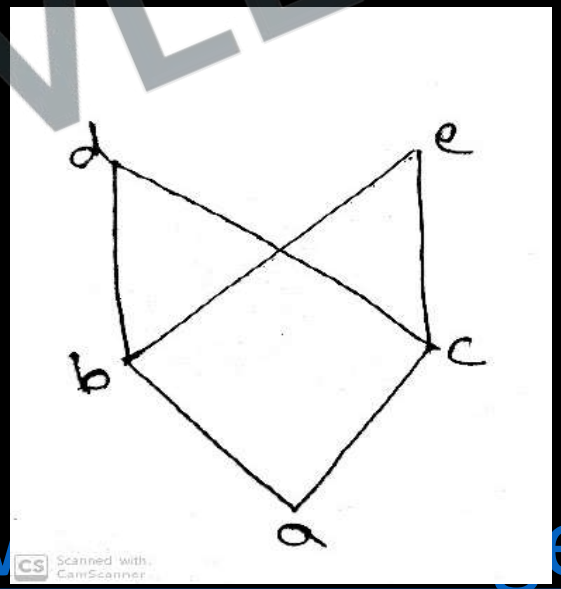
KNOWLEDGEGATE

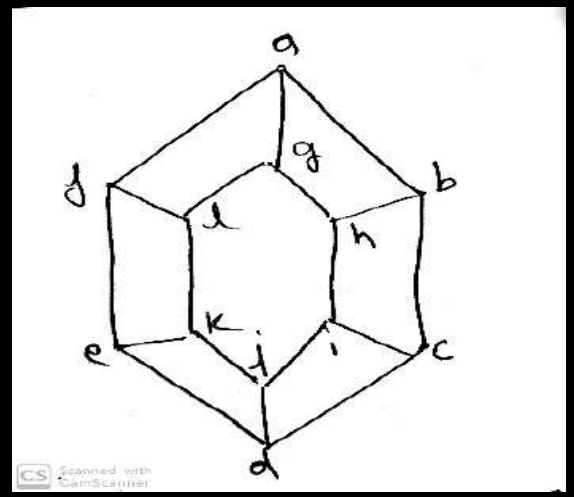








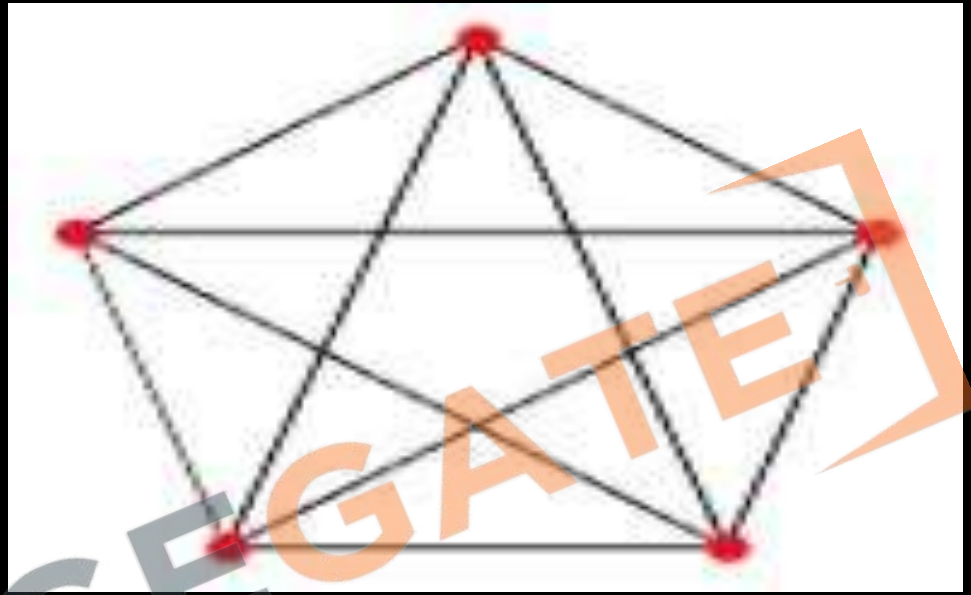




KNOWLEDGEGATE

http://ww egate.in/GATE

- **Start at a Vertex**: Begin at any vertex in the graph.
- **Explore Paths**: Choose a neighbor that has not been visited and add it to the current path.
- **Check for Cycle**: If all vertices are included in the path and there is an edge from the last vertex to the first, a Hamiltonian cycle has been found.
- **Backtrack if Stuck**: If you reach a vertex that has no unvisited neighbouring vertices, or you can't form a cycle, backtrack to the previous vertex and try a different path.
- **Repeat**: Continue this process, exploring edges, backtracking when necessary, and checking for cycles until all possibilities are exhausted.



KNOWLEDGEGATE

Aspect	Backtracking	Branch and Bound
Approach	Systematic search for solution by trying and eliminating possibilities.	Systematic search for solution using optimization and partitioning.
Goal	To find a feasible solution that satisfies all constraints.	To find the optimal solution with respect to a given objective.
Technique	Removes candidates that fail to satisfy the constraints of the problem.	Uses bounds to estimate the optimality of partial solutions.
State Space Tree	Pruning is used to chop off branches that cannot possibly lead to a solution.	Branches are pruned based on bounds of the cost function (lower and upper bounds).
Application	Suitable for decision problems like puzzles.	Used for optimization problems like integer programming.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Ch-8

Advanced Data Structures

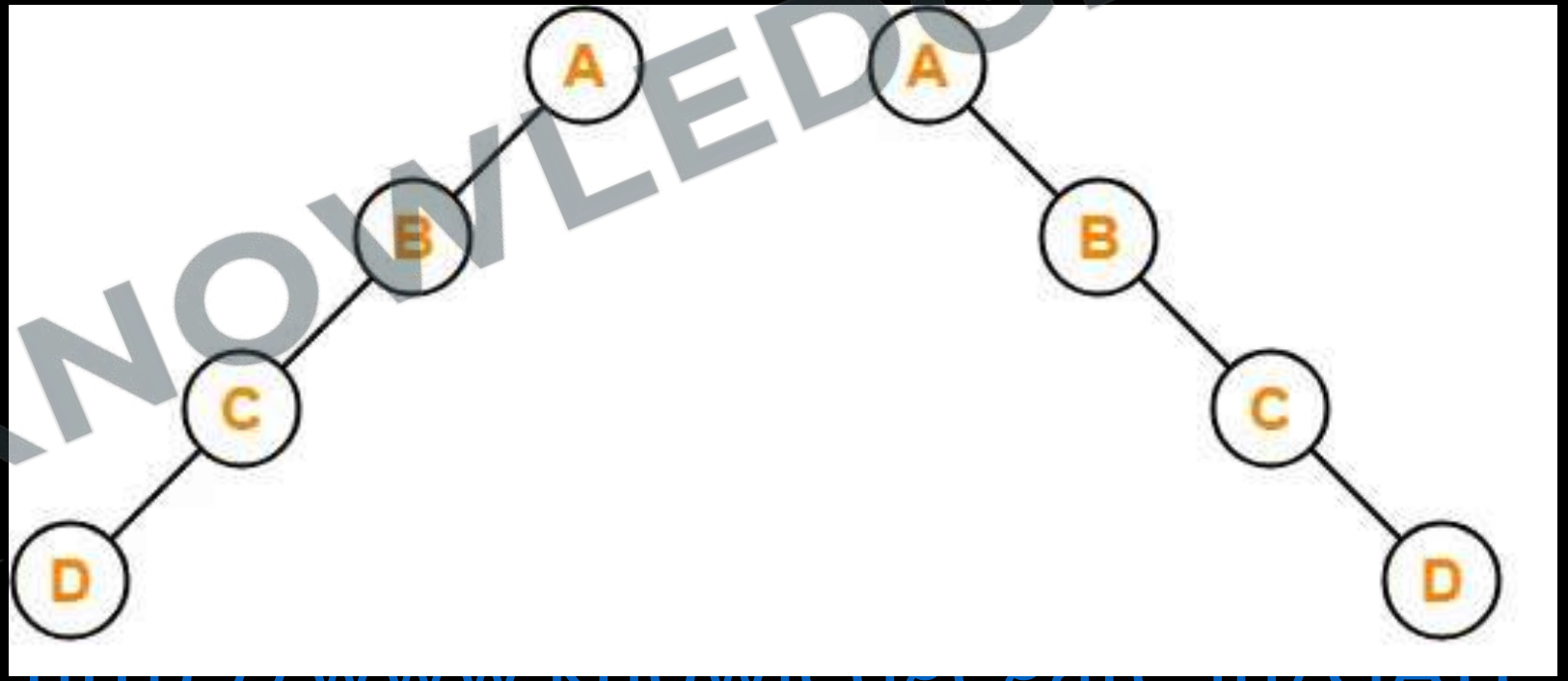
Red-Black Trees, B – Trees, Binomial Heaps, Fibonacci Heaps, Tries, Skip List, Introduction to Activity Networks Connected Component.

KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

Red-Black Trees

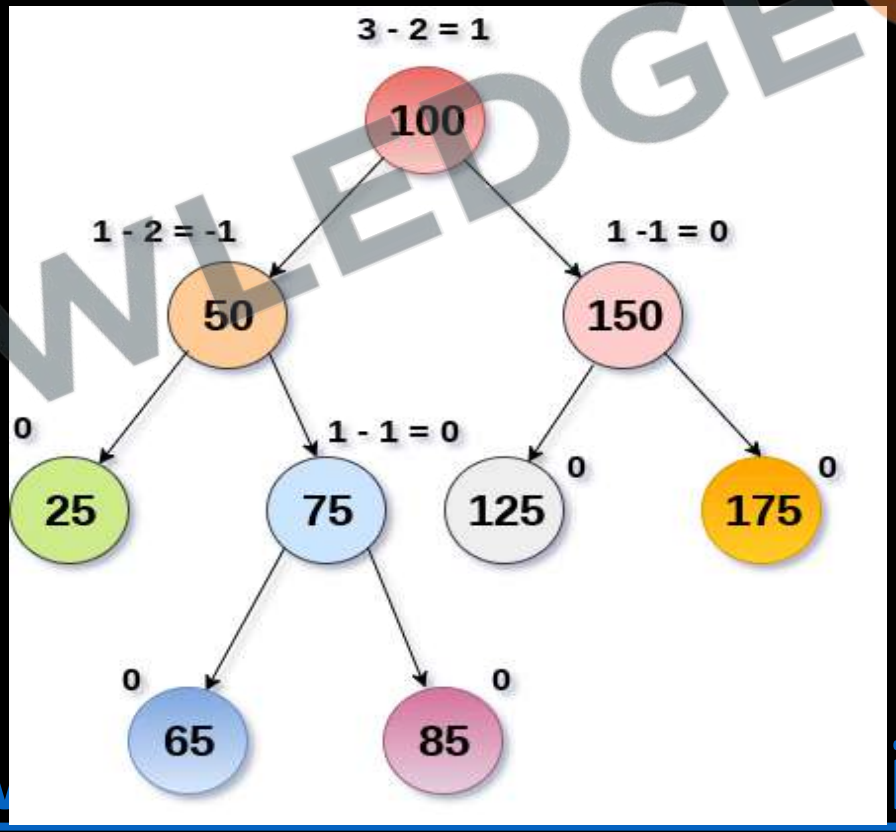
- **Binary Search Trees (BSTs)**: The concept of binary search trees has been around informally for a long time, but the binary tree data structure was first structured and named in computer science literature by P.F. Windley, A.D. Booth, A.J.T. Colin, and T.N. Hibbard between 1955 and 1962.
- May be Left-skewed or Right-skewed leading to O(n) worst case time complexity.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Red-Black Trees

- **AVL Trees**: AVL Trees, named after their inventors Georgy Adelson-Velsky and Evgenii Landis, were introduced in 1962. They were the first dynamically balanced trees to be proposed.
- No possibility of being Left-skewed or Right-skewed leading to $O(\log_2 n)$ worst case time complexity.



KNOWLEDGEGATE

Red-Black Trees

- AVL trees and Red-Black Trees are both self-balancing binary search trees, but there are certain issues with AVL trees that Red-Black Trees address more effectively:
- **Rotations for Balancing**:
  - **AVL Trees**: AVL trees maintain a stricter balance, requiring the heights of two child subtrees of any node to differ by at most one. This can lead to more frequent and potentially complex rotations upon insertions and deletions to maintain the strict height balance.
  - **Red-Black Trees**: Red-Black Trees are more permissive with balance, allowing a greater difference in black heights, which typically leads to fewer rotations needed after insertions and deletions.
- **Use Cases**:
  - **AVL Trees**: AVL trees are more suitable for lookup-intensive applications because they are more strictly balanced and therefore have better search times.
  - **Red-Black Trees**: Red-Black Trees, being less strictly balanced, offer better performance for insertions and deletions and are therefore more suitable for applications where the tree is frequently modified.
- **Algorithmic Complexity**:
  - Both AVL and Red-Black Trees offer $O(\log n)$ search, insertion, and deletion complexities. However, the constants involved in these operations for AVL trees may be higher due to the need for more rotations.

KNOWLEDGE GATE
http://www.knowledgegate.in/GATE

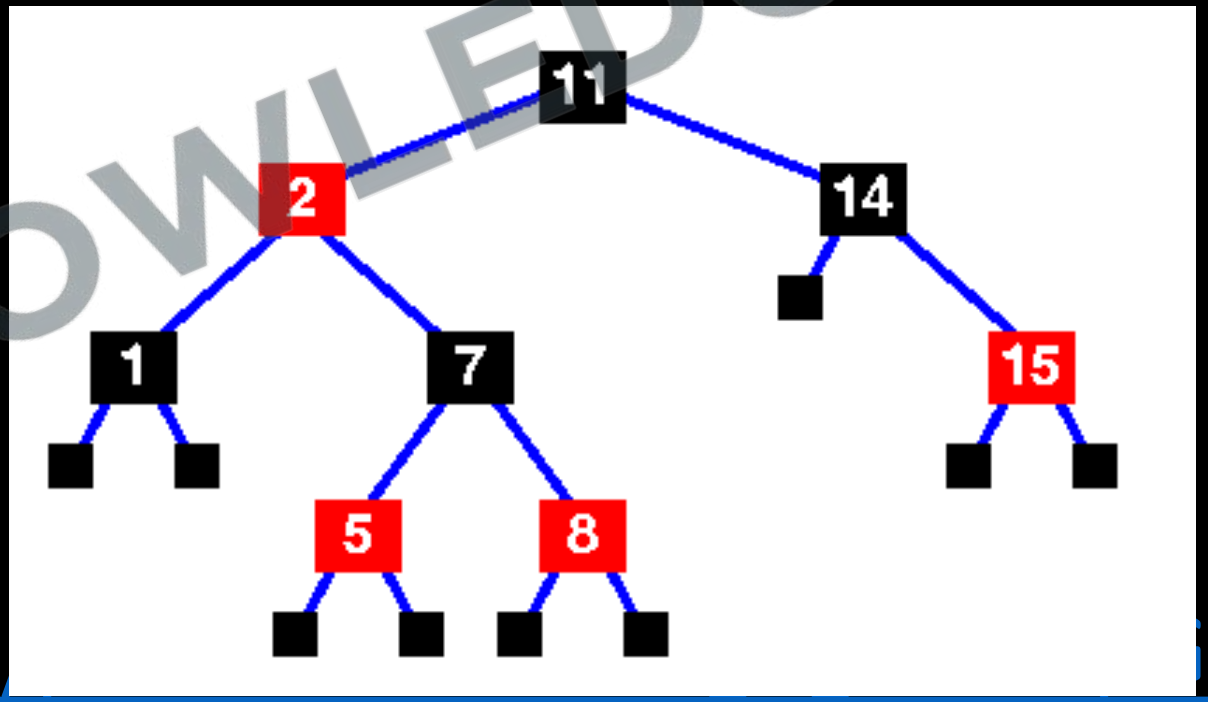
Red-Black Trees

- **AVL Trees Example:**
  - **Database Indexing**: Used in databases for efficient search operations, especially when reads are more frequent than writes. For instance, in a library's user database, AVL trees can optimize user data retrieval based on sorted user IDs.
- **Red-Black Trees Example:**
  - **Programming Libraries**: Common in standard libraries like C++ STL for implementing std::map and std::set. Suitable for scenarios with frequent additions and deletions, such as maintaining a sorted list of users in real-time applications.

KNOWLEDGE GATE

Red-Black Trees

- A Red-Black Tree is a type of self-balancing binary search tree, where each node has an extra attribute: the color, which is either red or black. Here are the defining properties that every Red-Black Tree adheres to:
  - **Node Color**: Every node is colored, either red or black.
  - **Root Property**: The root of the tree is always black.
  - **Leaf Nodes**: Every leaf (NIL node, an empty/black node) is black.
  - **Red Node Rule**: No two red nodes may be adjacent; a red node cannot have a red parent or red child.
  - **Black Height Consistency**: Every path from a given node to any of its descendant NIL nodes must have the same number of black nodes. This is known as the "black-height".



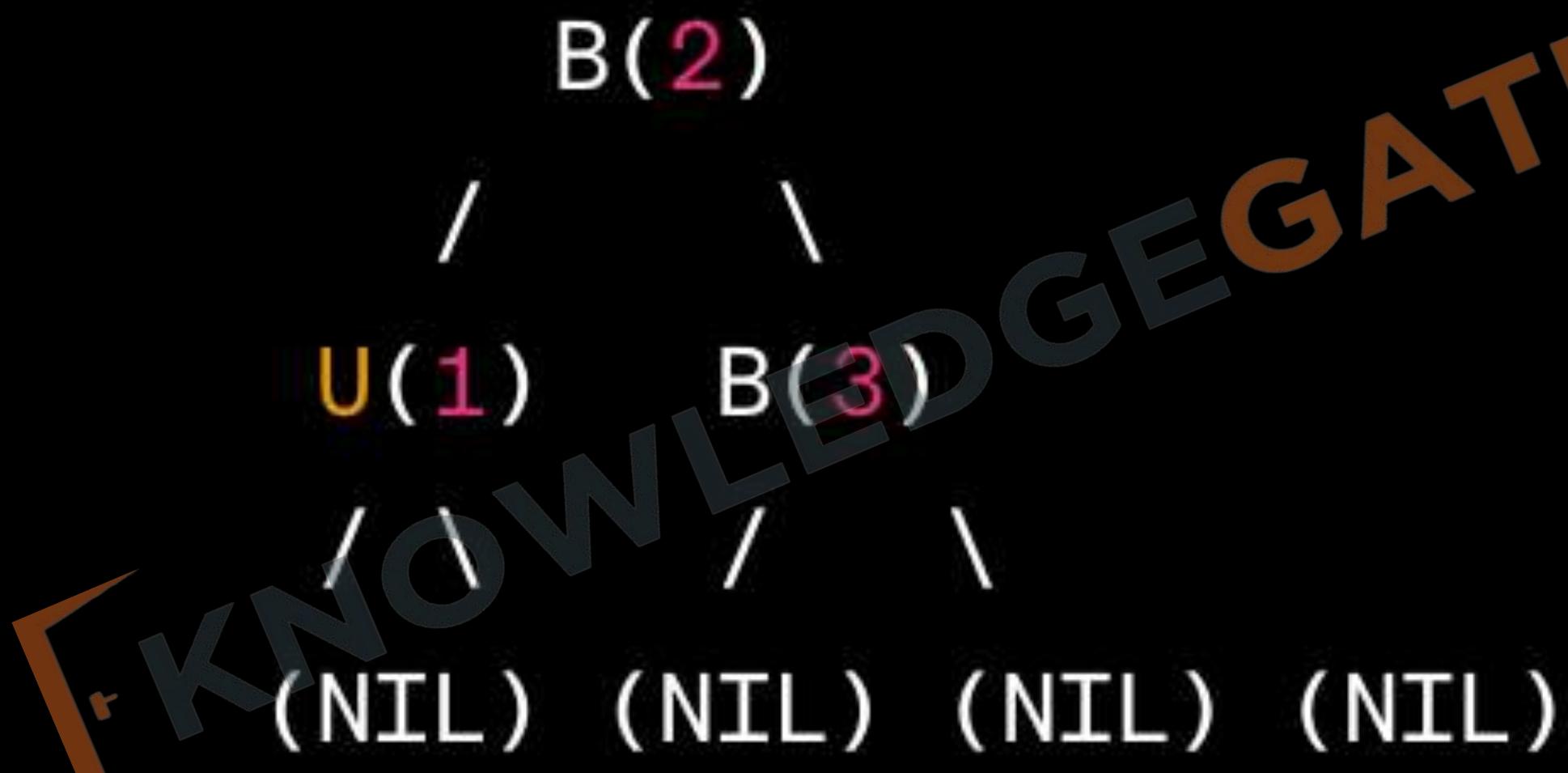
KNOWLEDGE GATE

- These properties ensure that the tree remains balanced, with the longest path from the root to the farthest leaf not more than twice as long as the shortest path from the root to the nearest leaf.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Unspecified Node Color (Violation: Node Color)



KNOWLEDGEGATE

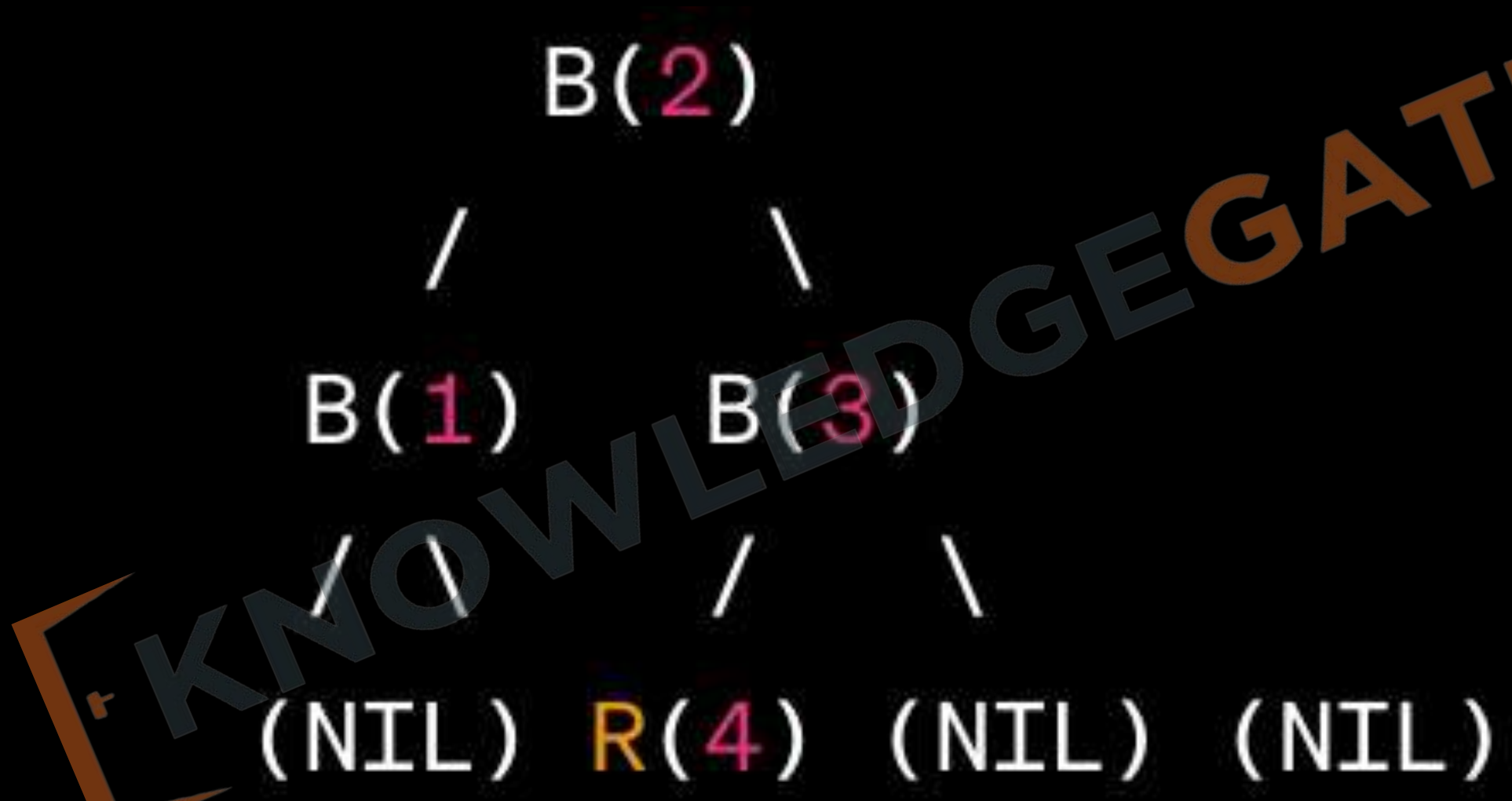
http://www.knowledgegate.in/GATE

Red Root Node (Violation: Root Property)

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

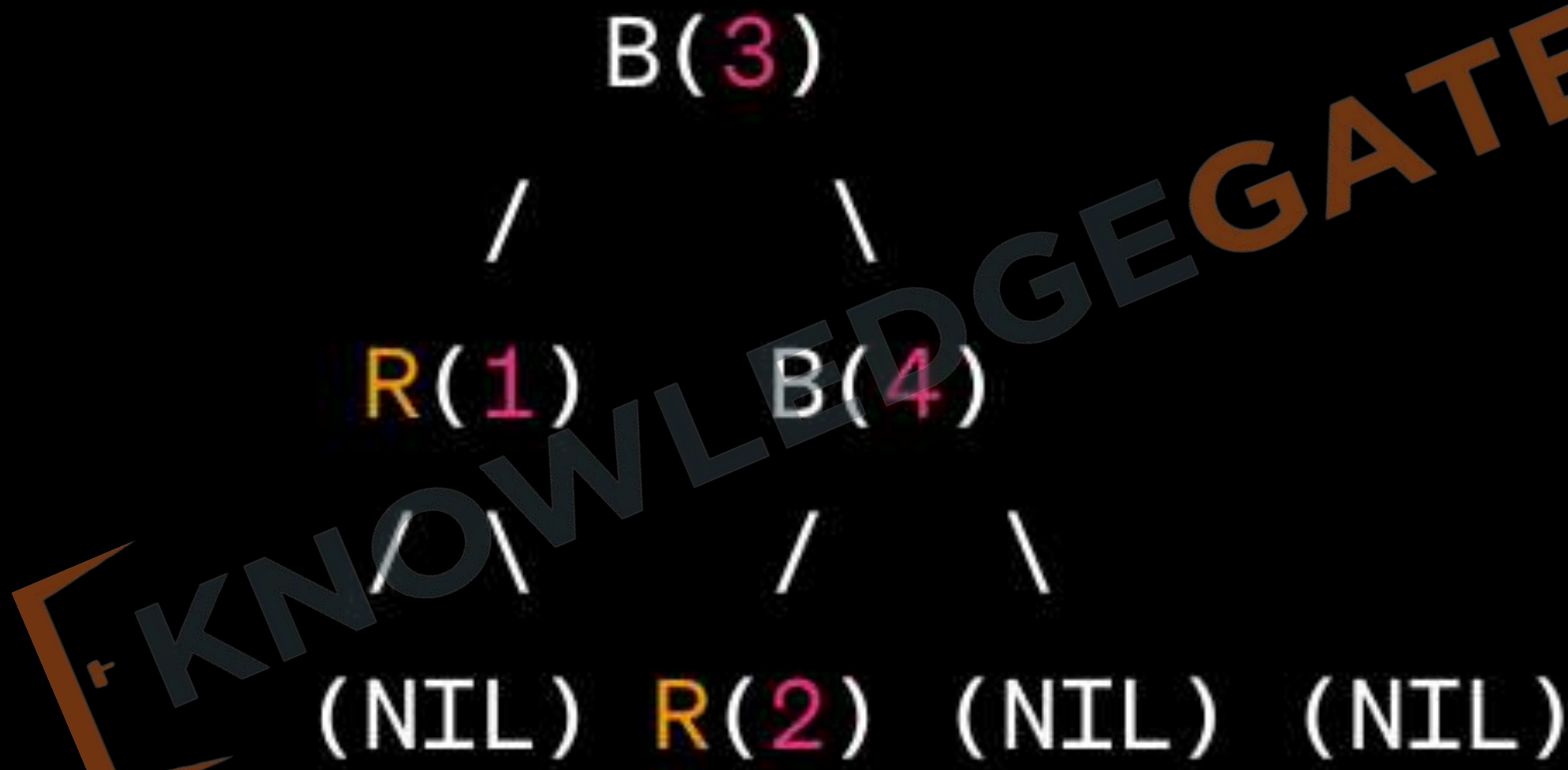
Non-Black Leaf Node (Violation: Leaf Nodes)



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

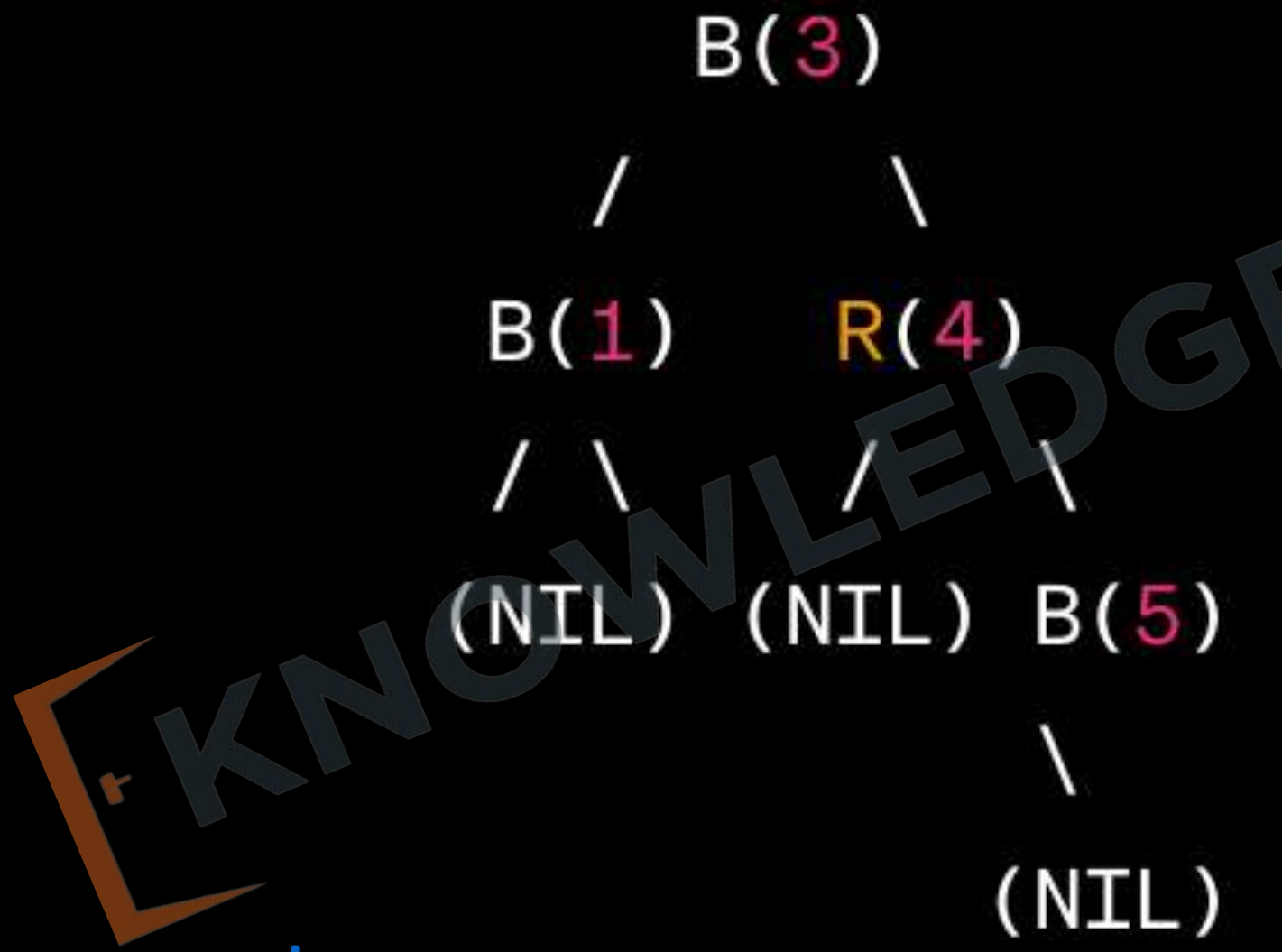
Consecutive Red Nodes (Violation: Red Node Rule)



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Unspecified Node Color (Violation: Node Color)



KNOWLEDGEGATE

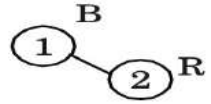
- If tree is empty then create newnode as root node with color black.
- If three is not empty then insert newnode with color red.
- If the parent of the new node is black, the properties are preserved.
- If the parent of newnode is red, then check the color of the sibling of the new node's parent
  - **Case 1: Uncle is Red (Recoloring needed)**:
    - Recolor the parent and uncle to black.
    - Recolor the grandparent to red.
    - Set the grandparent as the new node to be examined and repeat the validation process.
  - **Case 2: Uncle is Black, or null**
    - Perform a rotation on the grandparent and Recolor .
      - former parent to black and the grandparent to red(in LL or RR rotations).
      - In LR or RL rotation re color accordingly
- **Adjust the Root**:
  - If rotations have been performed, the root node might have changed. Ensure the root is still black.
- **Continue Until the Tree is Fixed**:
  - Continue the validation and re-balancing process until all Red-Black Tree properties are satisfied.

KNOWLEDGE GATE
http://www.knowledgegate.in/GATE

Q Insert the following sequence of information in an empty red-black tree **1, 2, 3, 4, 5, 6, 7, 8 and 9** ?

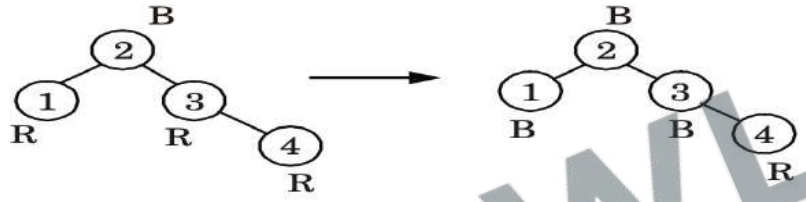
Insert 1 :

Insert 2 :

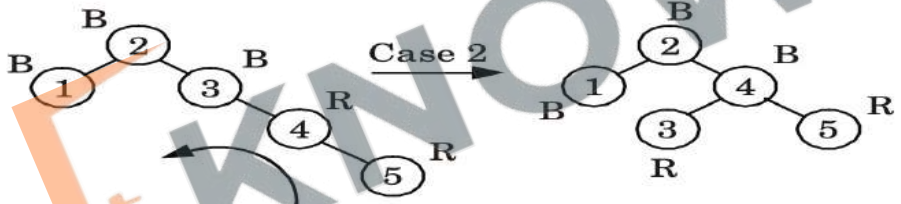


Insert 3 :

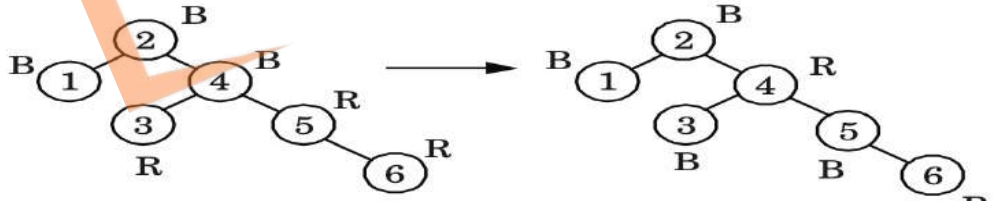
Insert 4 :



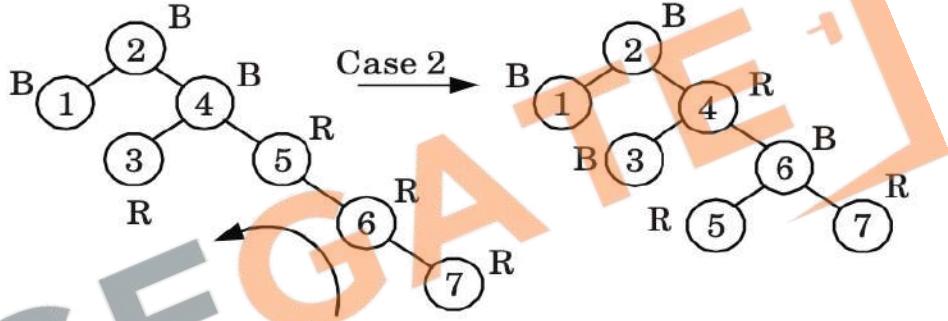
Insert 5 :



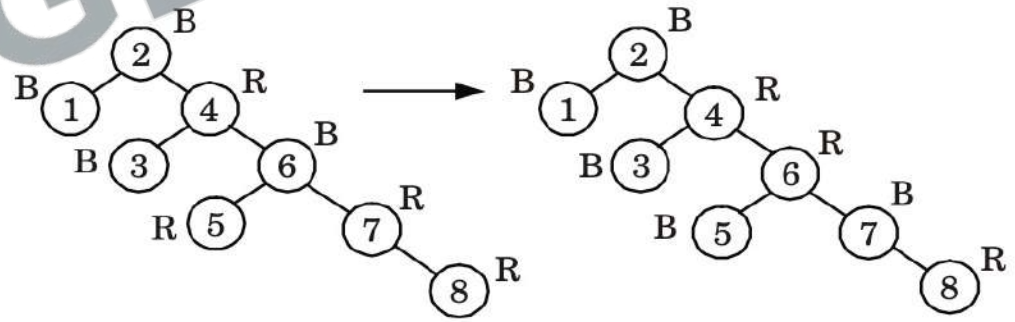
Insert 6 :



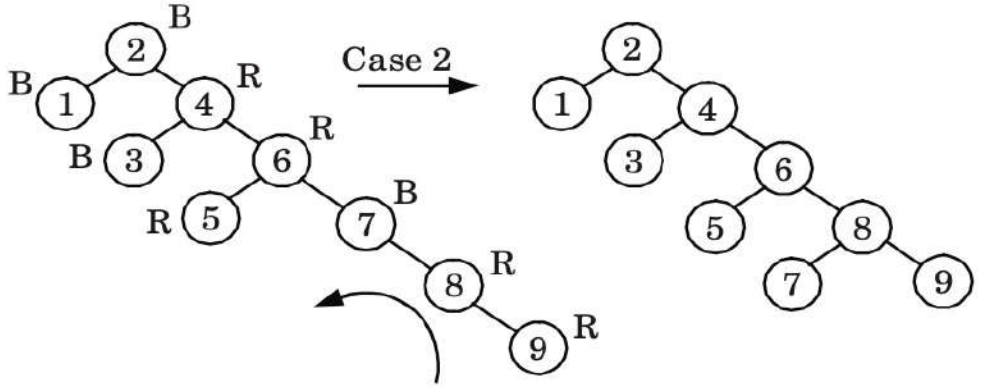
Insert 7 :



Insert 8 :



Insert 9 :

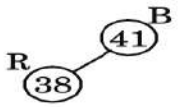


KNOWLEDGEGATE

Q Insert the following sequence of information in an empty red-black tree **41, 38, 31, 12, 19, 8** ?

Insert 41 :

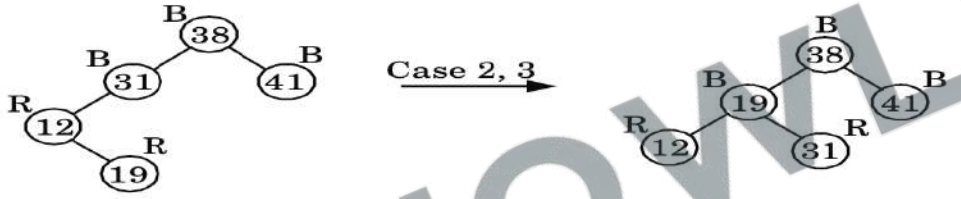
Insert 38 :



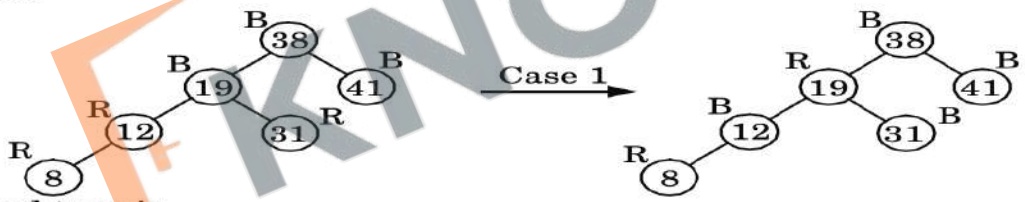
Insert 31 :

Insert 12 :

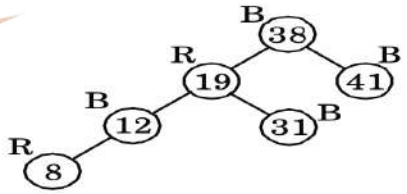
Insert 19 :



Insert 8 :

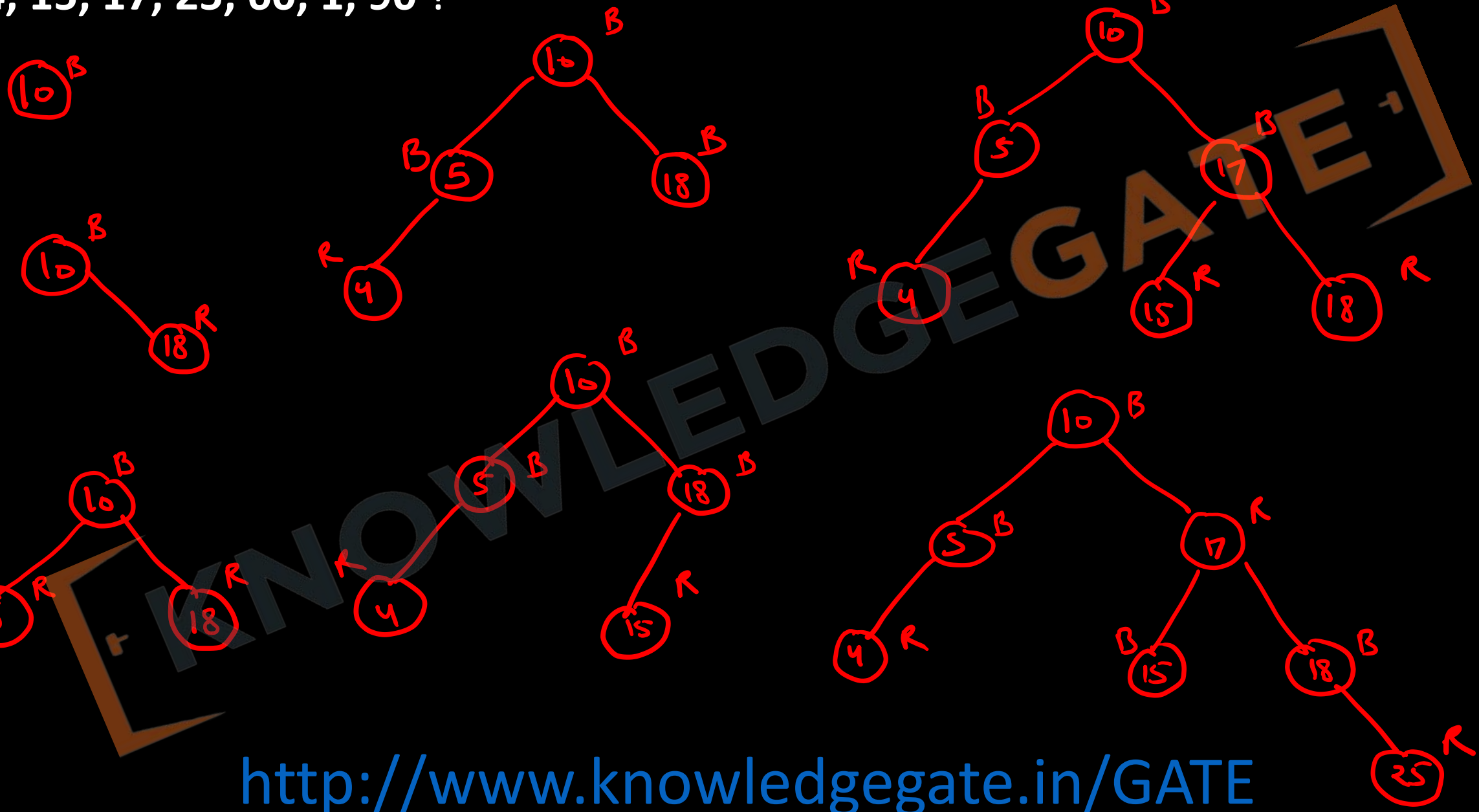


Thus final tree is



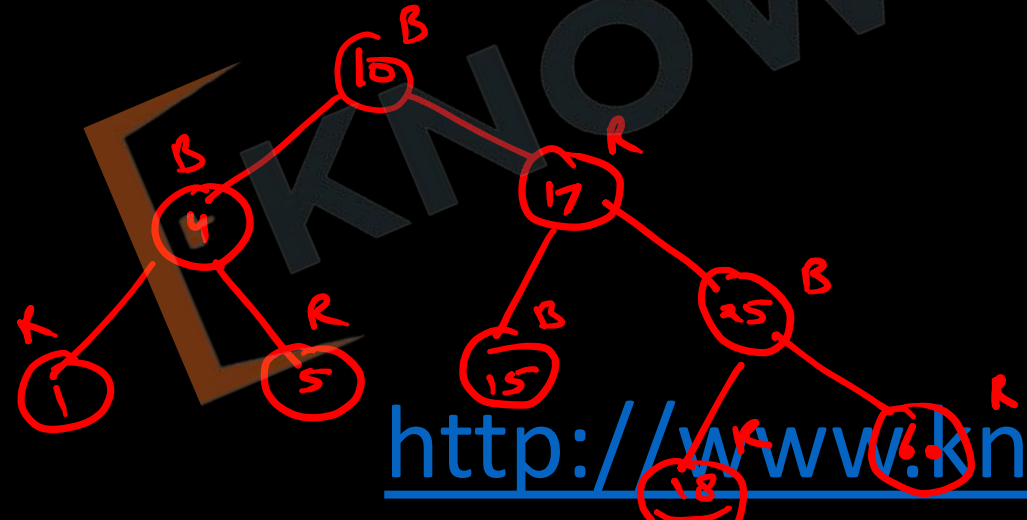
KNOWLEDGEGATE
owledgegate.in/GATE

Q Insert the following sequence of information in an empty red-black tree **10, 18, 5, 4, 15, 17, 25, 60, 1, 90** ?



KNOWLEDGEGATE

Q Insert the following sequence of information in an empty red-black tree **10, 18, 5, 4, 15, 17, 25, 60, 1, 90** ?



[KNOWLEDGEGATE]

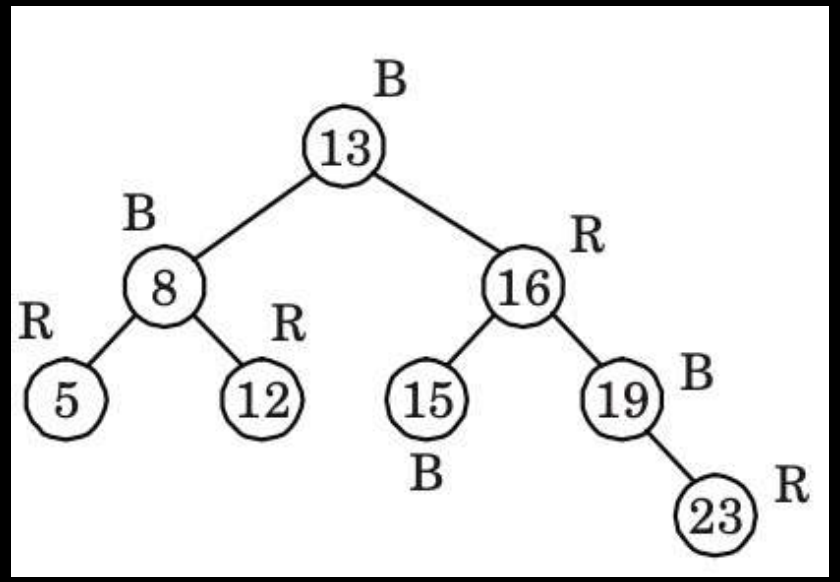
http://www.knowledgegate.in/GATE

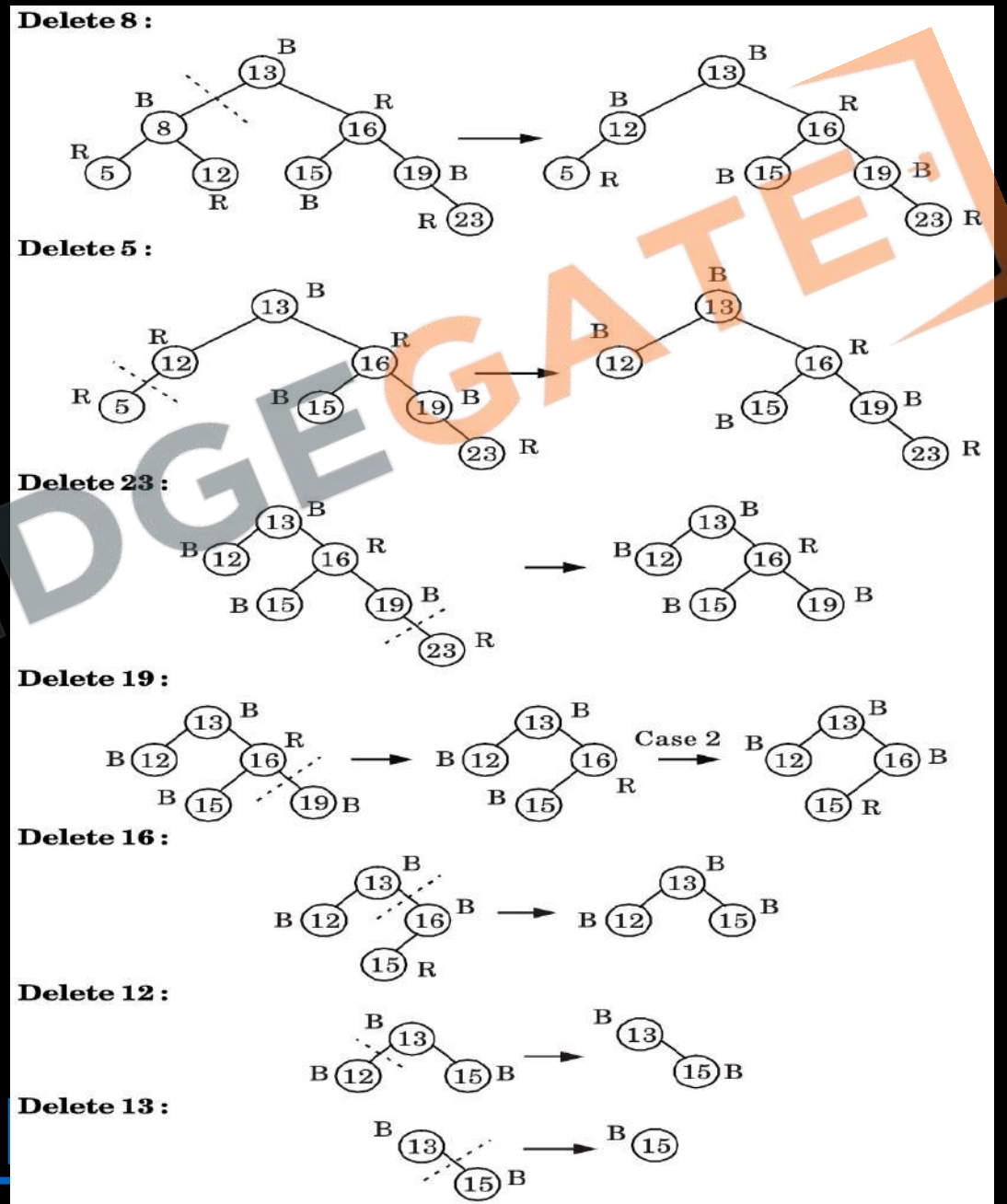
- Deletion in a Red-Black Tree involves several steps and can be more complex than insertion due to the need to maintain the red-black properties post-deletion. Here's a simplified step-by-step process for deleting a node from a Red-Black Tree:

- **Find the Node**: Locate the node you wish to delete using standard binary search tree deletion steps. If the node has two children, find the in-order successor (or predecessor) and swap its value with the node to delete.
- **Remove the Node**: Delete the node from the tree. There are three possible scenarios:
  - Deleting a node with no children: simply remove the node.
  - Deleting a node with one child: replace the node with its child.
  - Deleting a node with two children: replace the node with its in-order successor (or predecessor), which will have at most one child, and then delete the successor.
- **Fixing Double Black Issue**:
  - If you removed a red node, the properties still hold. If a black node was removed, this creates a "double black" issue, where one path has one fewer black node.
  - The node that replaces the deleted node (if any) is marked as "double black," which means it either has an extra black than it should, or it's black and has taken the place of a removed black node.
- **Rebalance the Tree**: To resolve the "double black" issue, several cases need to be handled until the extra blackness is moved up to the root (which can then be discarded) or until re-balancing is done:
  - **Case 1: Sibling is red** - Perform rotations to change the structure such that the sibling becomes black, and then proceed to other cases.
  - **Case 2: Sibling is black with two black children** - Repaint the sibling red and move the double black up to the parent.
  - **Case 3: Sibling is black with at least one red child** - Perform rotations and recoloring so that the red child of the sibling becomes the sibling's parent. This corrects the double black without introducing new problems.
- **Termination**: The process terminates when:
  - The double black node becomes the root (simply remove the extra blackness).
  - The double black node is red (simply repaint it to black).
  - The tree has been adjusted to redistribute the black heights and remove the double blackness.
- **Update Root**: After rotations and recoloring, the root might change, so ensure the root of the tree is still black.

KNOWLEDGEGATE

Q Insert the nodes 15, 13, 12, 16, 19, 23, 5, 8 in empty red-black tree and delete in the reverse order of insertion?

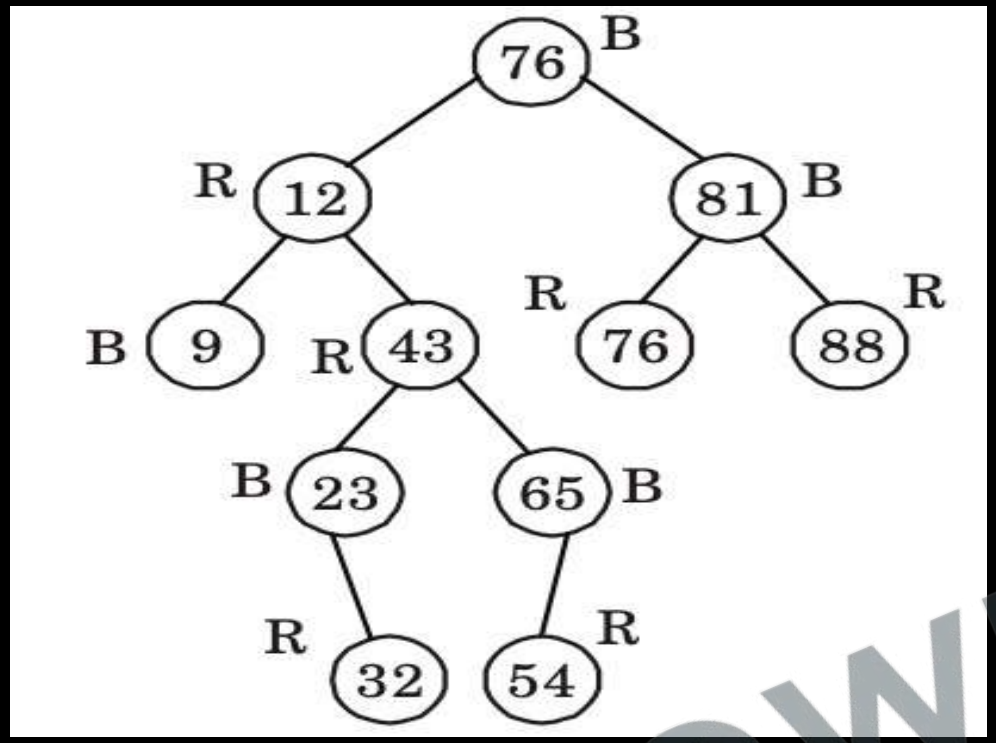


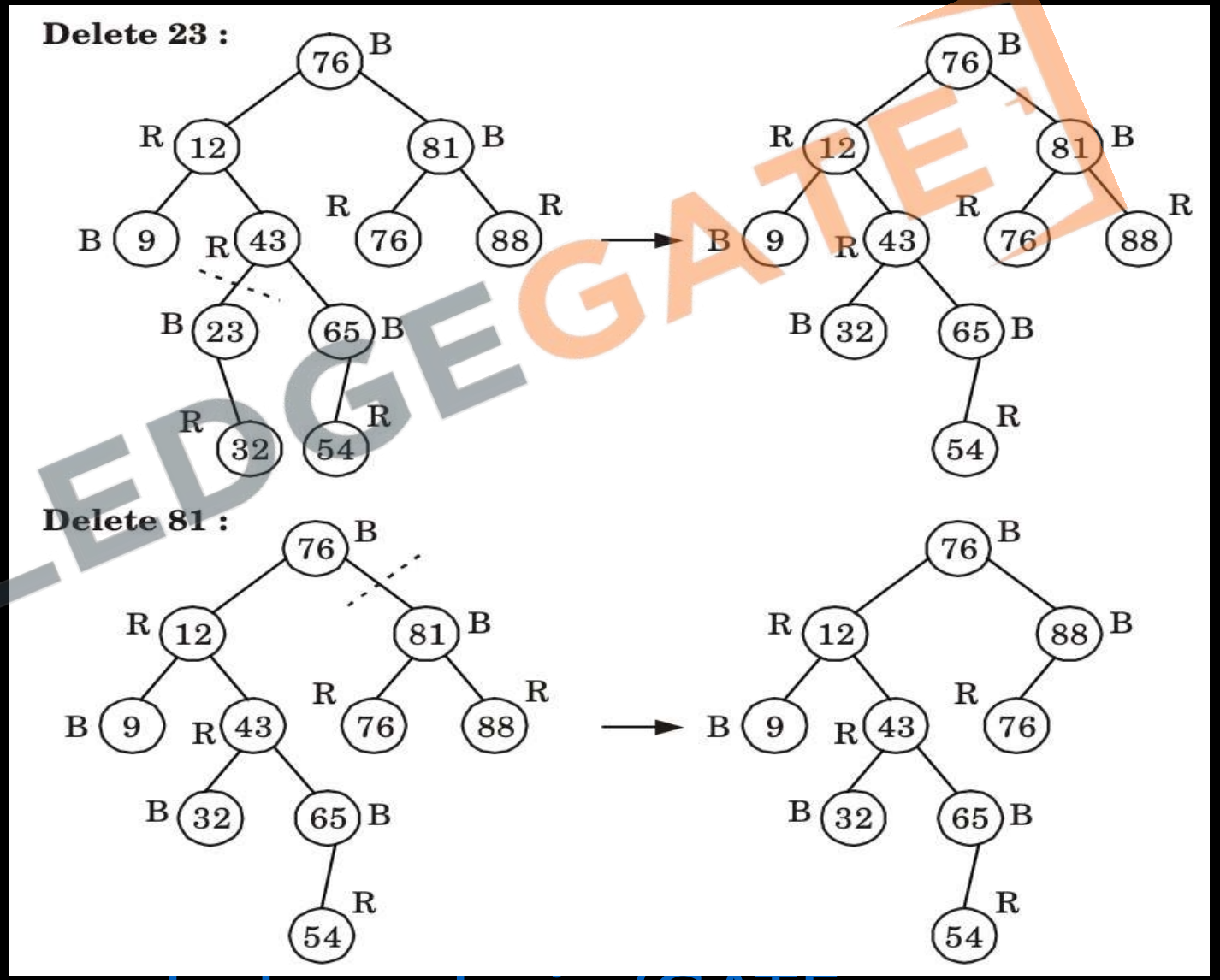


KNOWLEDGEGATE

http://www.knowl

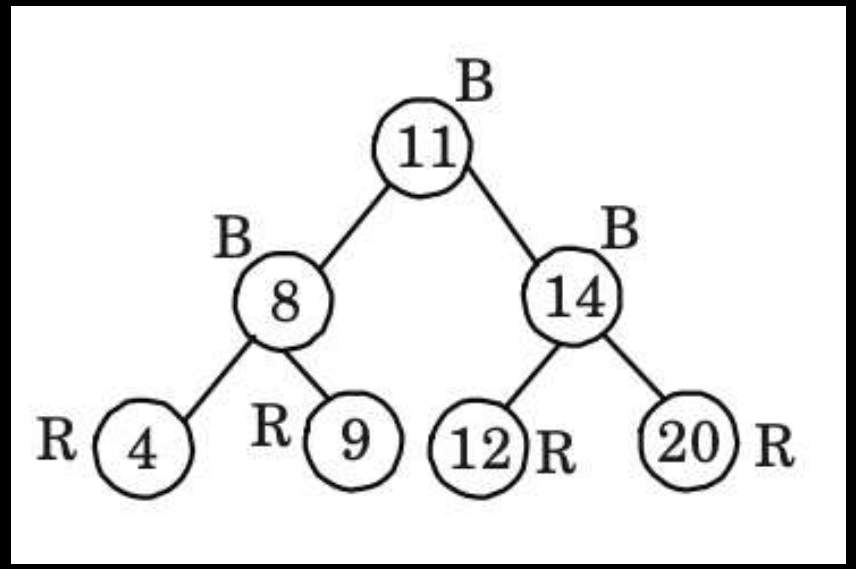
Q Insert the following element in an initially empty RB-Tree 12, 9, 81, 76, 23, 43, 65, 88, 76, 32, 54. Now delete 23 and 81 ?



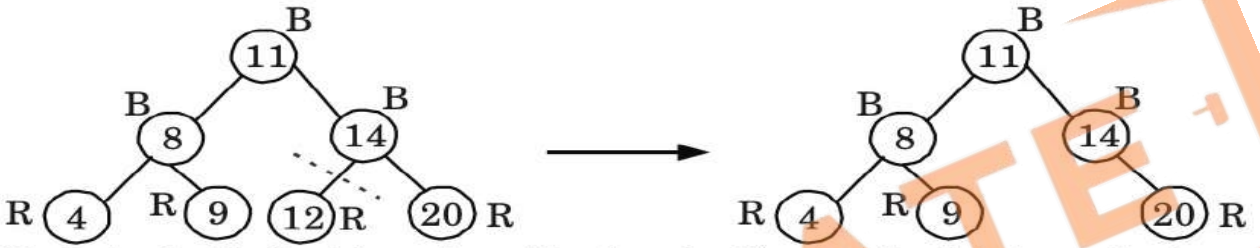


KNOWLEDGEGATE

Q Insert the elements 8, 20, 11, 14, 9, 4, 12 in a Red-Black tree and delete 12, 4, 9, 14 respectively ?

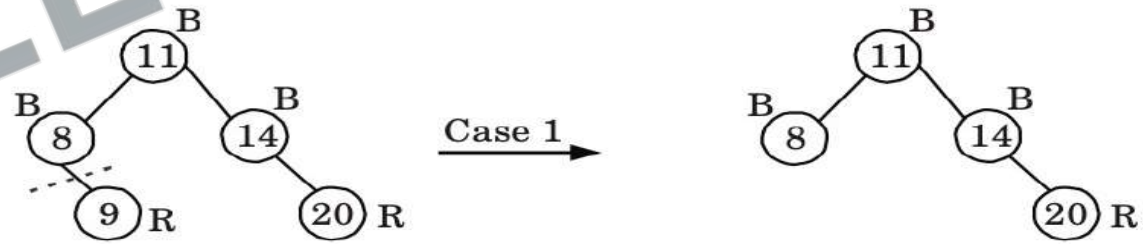


Delete 12 : Node 12 is red and leaf node. So simply delete node 12.

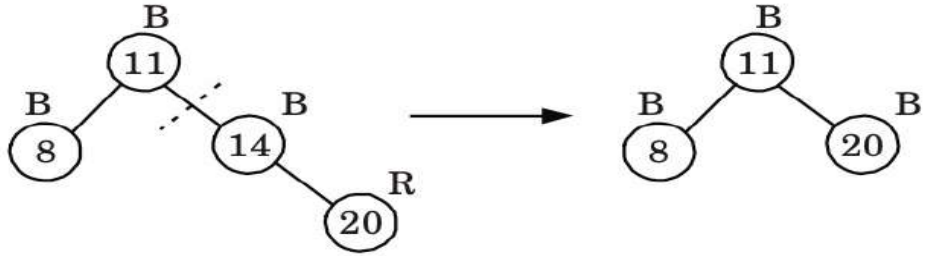


Delete 4 : Node 4 is red and leaf node. So simply delete node 4.

Delete 9 : Node 9 is red and leaf node. So simply delete node 9.



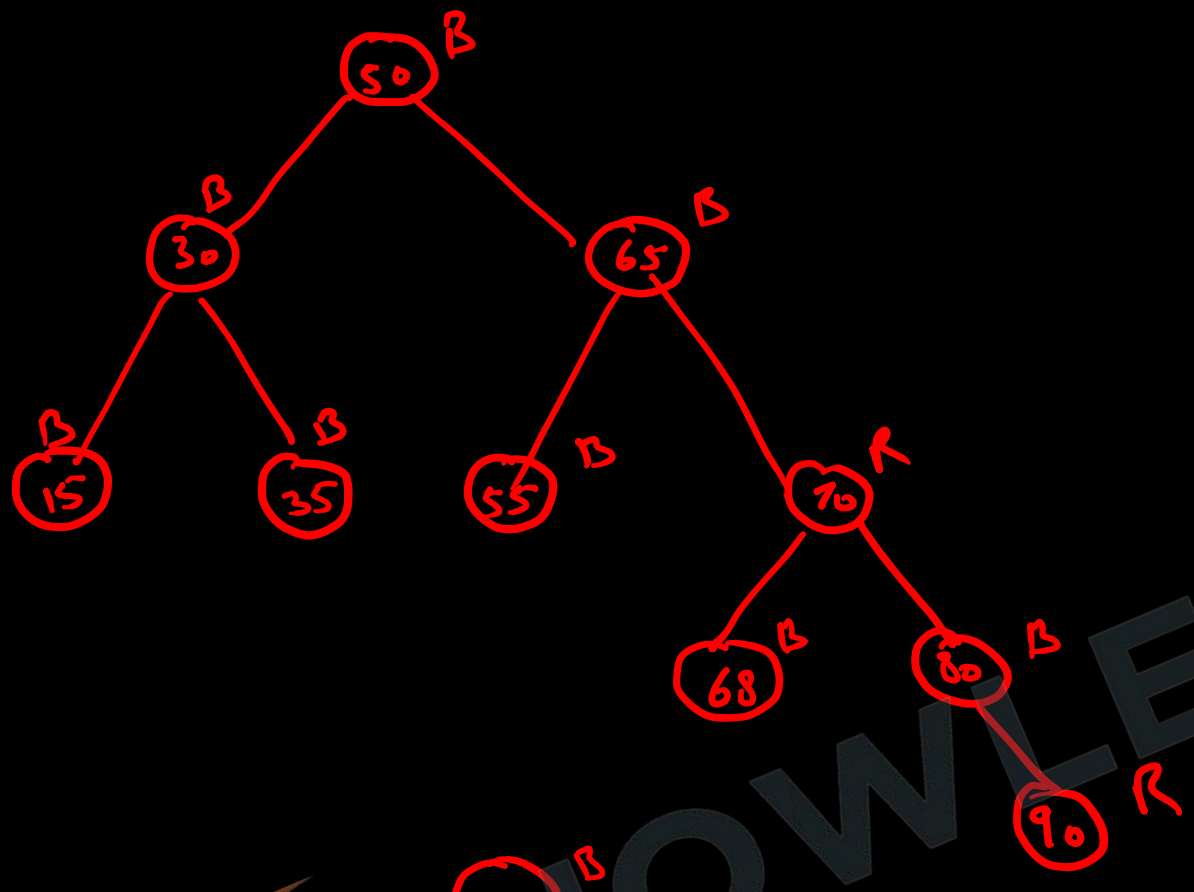
Delete 14 : Node 14 is internal node replace node 14 with node 20 and do not change the colour.

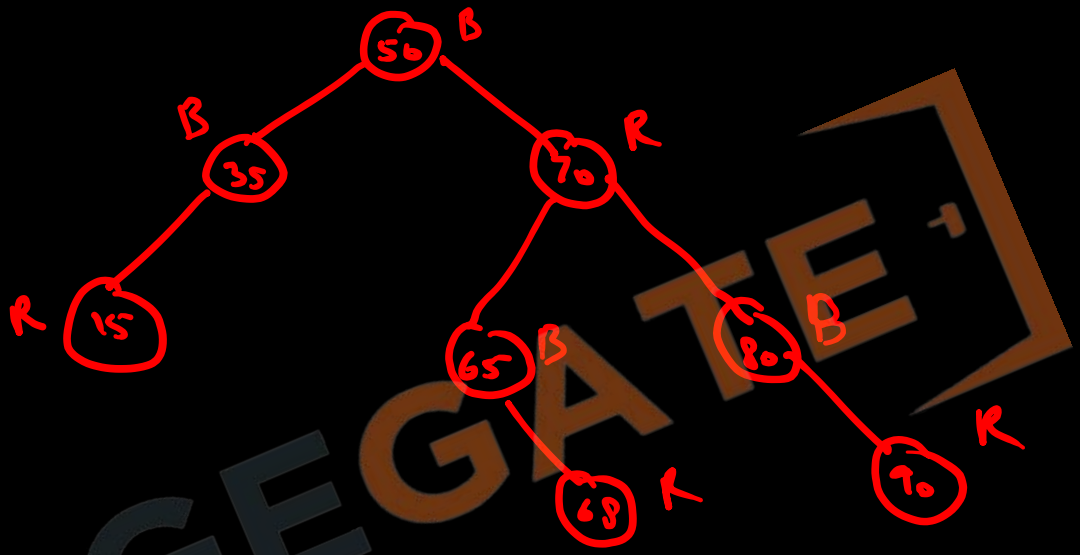


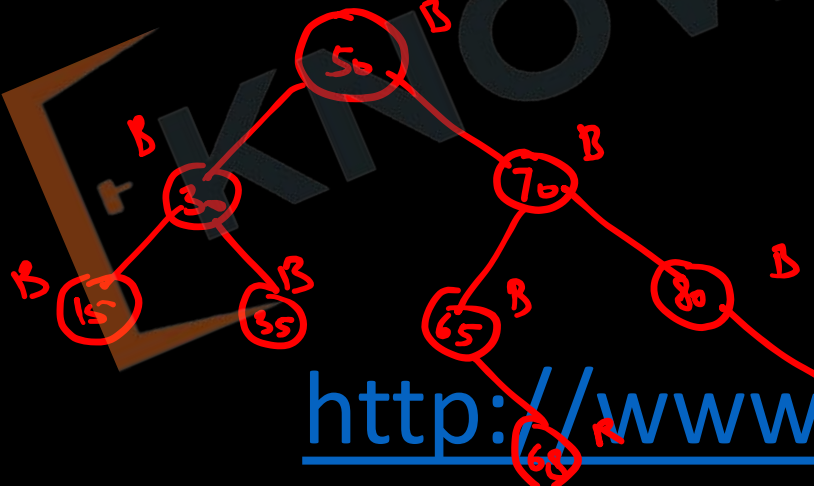
KNOWLEDGEGATE

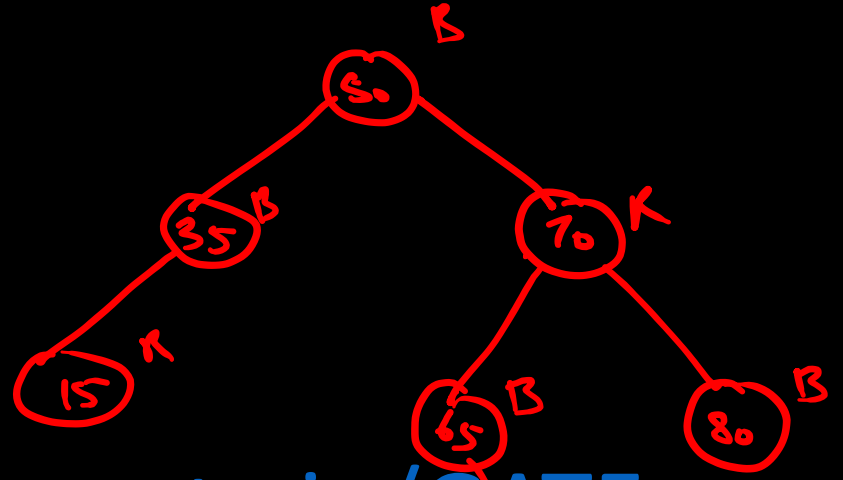
http://www.k

Q Consider the following RB Tree and delete 55, 30, 90, 80, 50, 35, 15 respectively ?





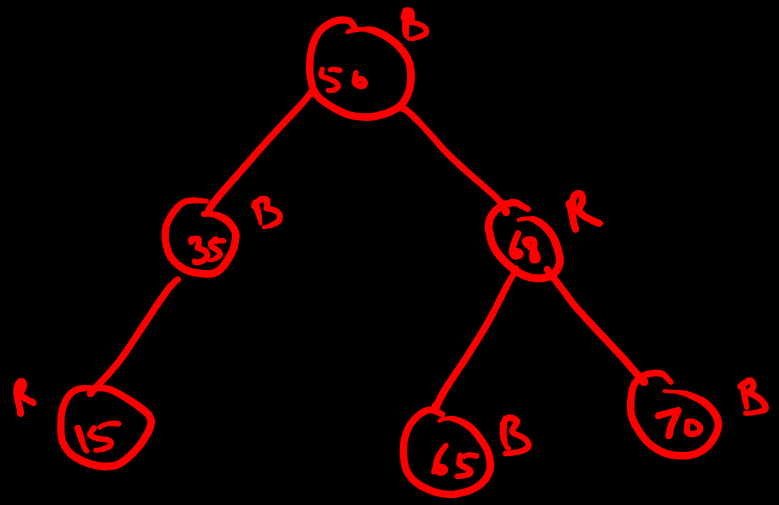


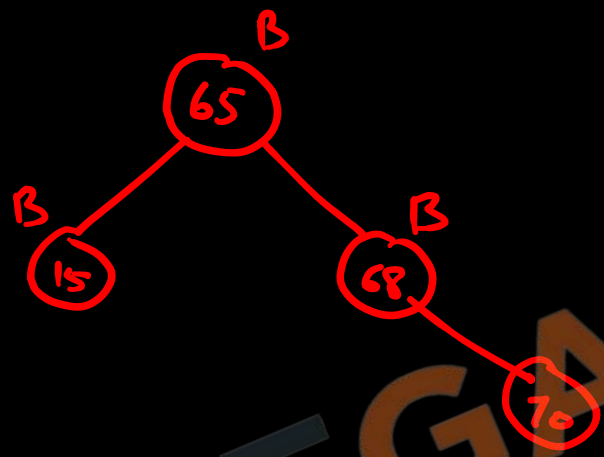


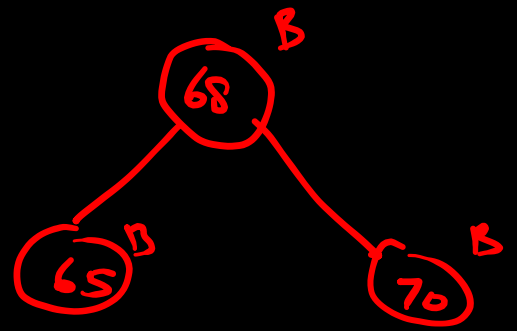
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider the following RB Tree and delete 55, 30, 90, 80, 50, 35, 15 respectively ?







KNOWLEDGEGATE

Binomial Tree

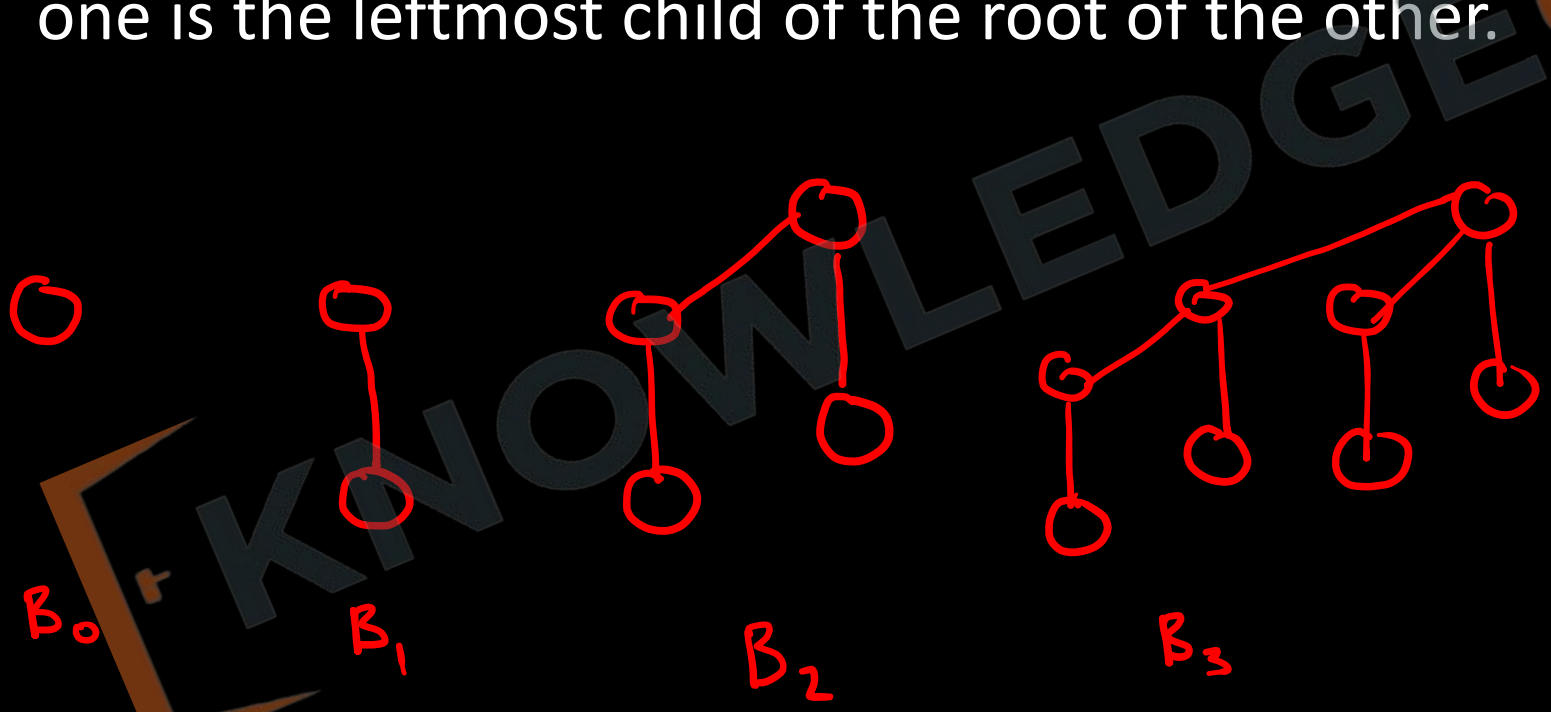
- **Development and Context**: Binomial trees as a data structure were developed as part of the broader exploration of efficient data structures for priority queue operations. They were introduced as a key component of binomial heaps.
- **Key Contributors**: The concept of the binomial heap, which uses binomial trees, was introduced by J. W. J. Williams in 1964. Later, it was further developed and popularized by Jean Vuillemin in 1978.
- **Evolution of Heaps**: Prior to binomial heaps, binary heaps were commonly used for priority queues. Binomial trees and heaps were an evolution in this area, offering more efficient solutions for certain operations.

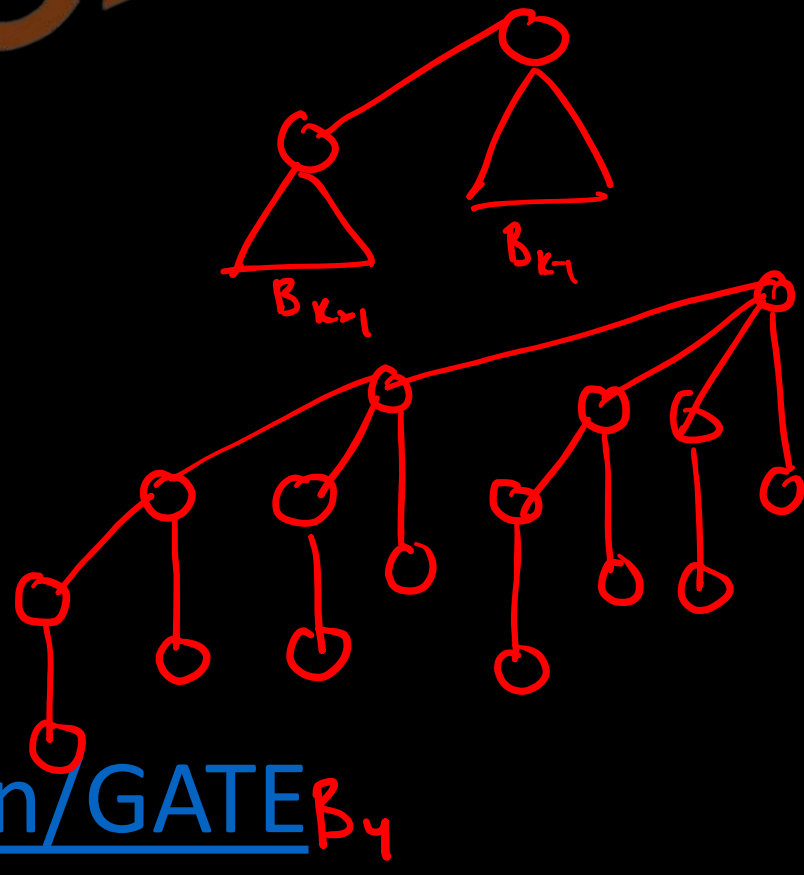
KNOWLEDGE GATE

.knowledgegate.in/GATE

Binomial Tree

- A binomial tree is a specific kind of tree used in certain data structures such as binomial heaps.
  - **Recursive Definition**: A binomial tree of order 0 is a single node. A binomial tree of order *k* consists of two binomial trees of order *k*–1 linked together: one is the leftmost child of the root of the other.

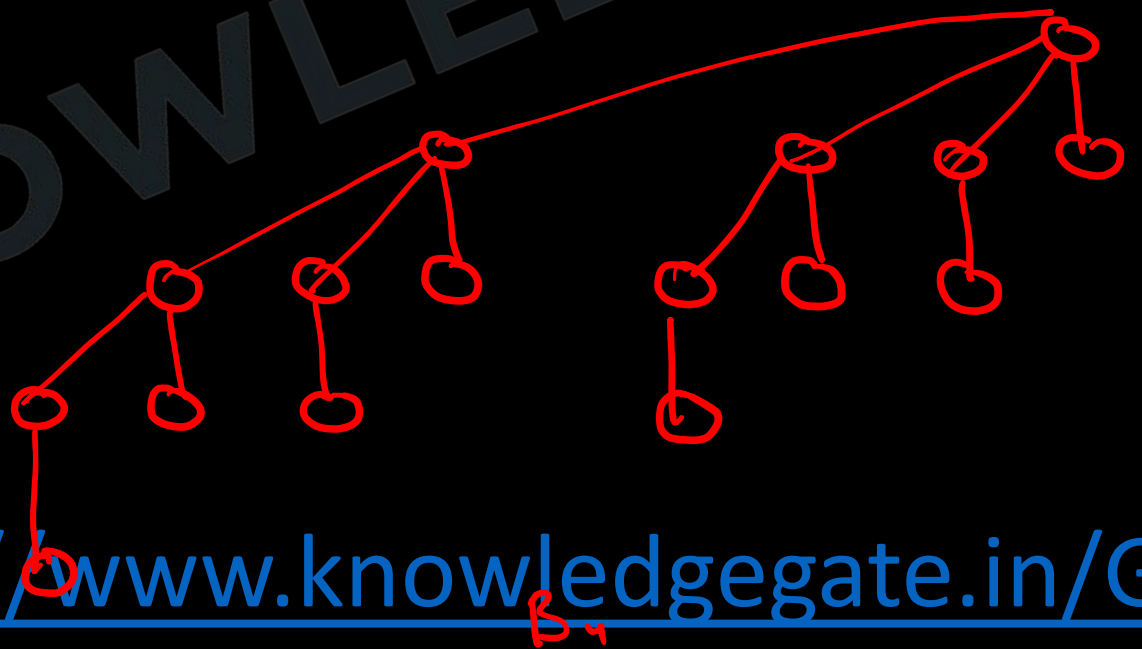




KNOWLEDGEGATE

Binomial Tree Properties

- A binomial tree of order k has exactly 2^k nodes. The height of a binomial tree of order *k* is *k*
- The height of a binomial tree B_k is k. That is, the tree has k+1 levels, indexed from 0 to k.
- The number of nodes at depth i in a binomial tree of order k is given by the binomial coefficient ${}^kC_i$ i=0,1,…,k.
- **T**he root of a binomial tree B_k will have a degree k, which will be greater than any other nodes.
- The root node of a binomial tree of order *k* has a degree of *k*, and it has *k* children. The children are themselves root nodes of binomial trees of orders *k*−1,*k*−2,…,0, respectively.



$i=0 \quad {}^4C_0 = 1$

$i=1 \quad {}^4C_1 = 4$

$i=2 \quad {}^4C_2 = 6$

$i=3 \quad {}^4C_3 = 4$

$i=4 \quad {}^4C_4 = 1$

[KNOWLEDGEGATE]

http://www.knowledgegate.in/GATE

Binomial Heap

- A binomial heap is implemented as a collection of binomial tree. It keeps data sorted and allows insertion and deletion in amortized time.
- **Properties:**
  - Unlike a binary heap, which is a single tree, a binomial heap is a collection of trees. which are ordered and comply with the min-heap (or max-heap) property.
  - Each binomial tree in a binomial heap has a different order
  - The heap with *n* elements will have at most $\lfloor \log n \rfloor + 1$ binomial trees. Because binary representation of n has logn +1 bits.

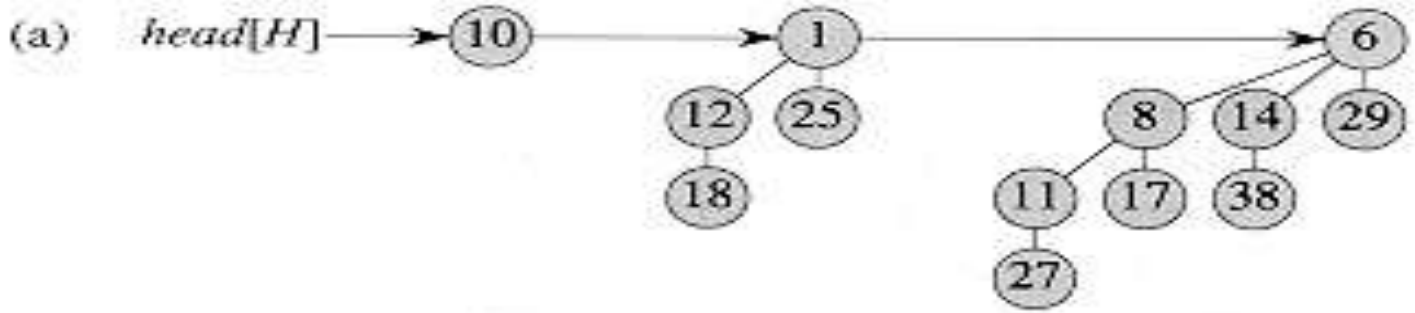
n = 15 1 1 1 1

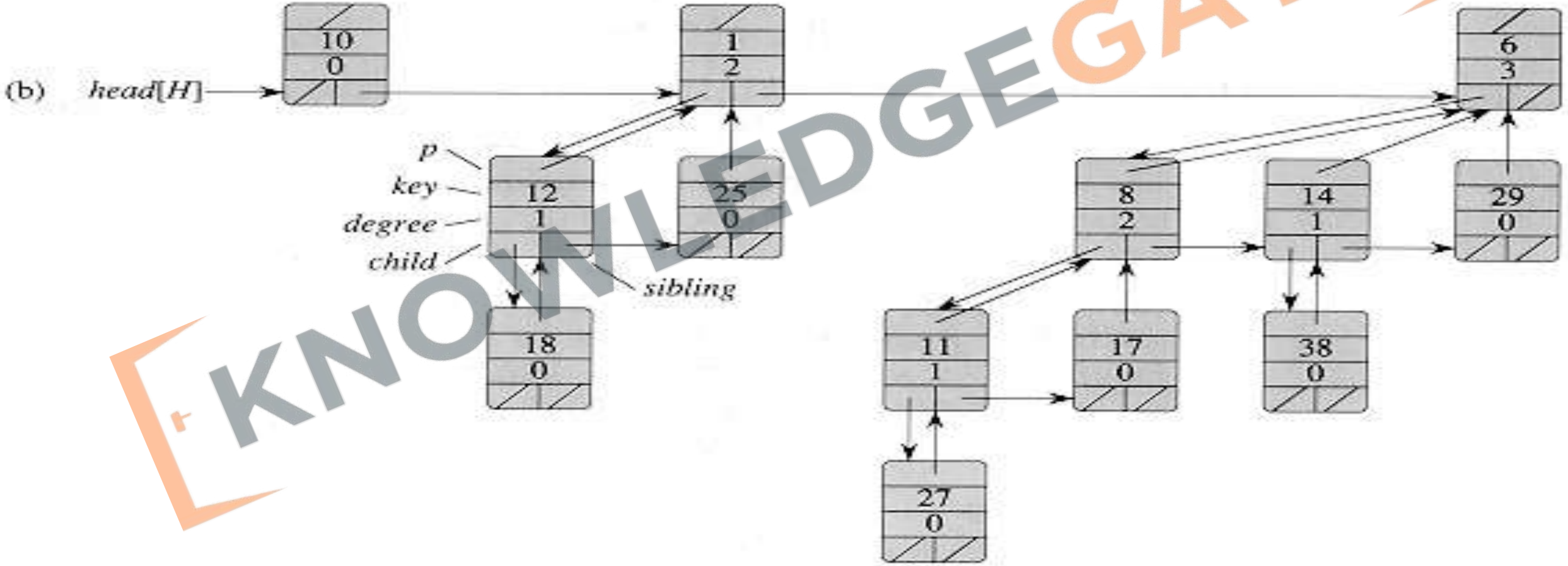
B_3 B_2 B_1 B_0

n = 9 1 0 0 1

B_3 x x B_0

KNOWLEDGEGATE





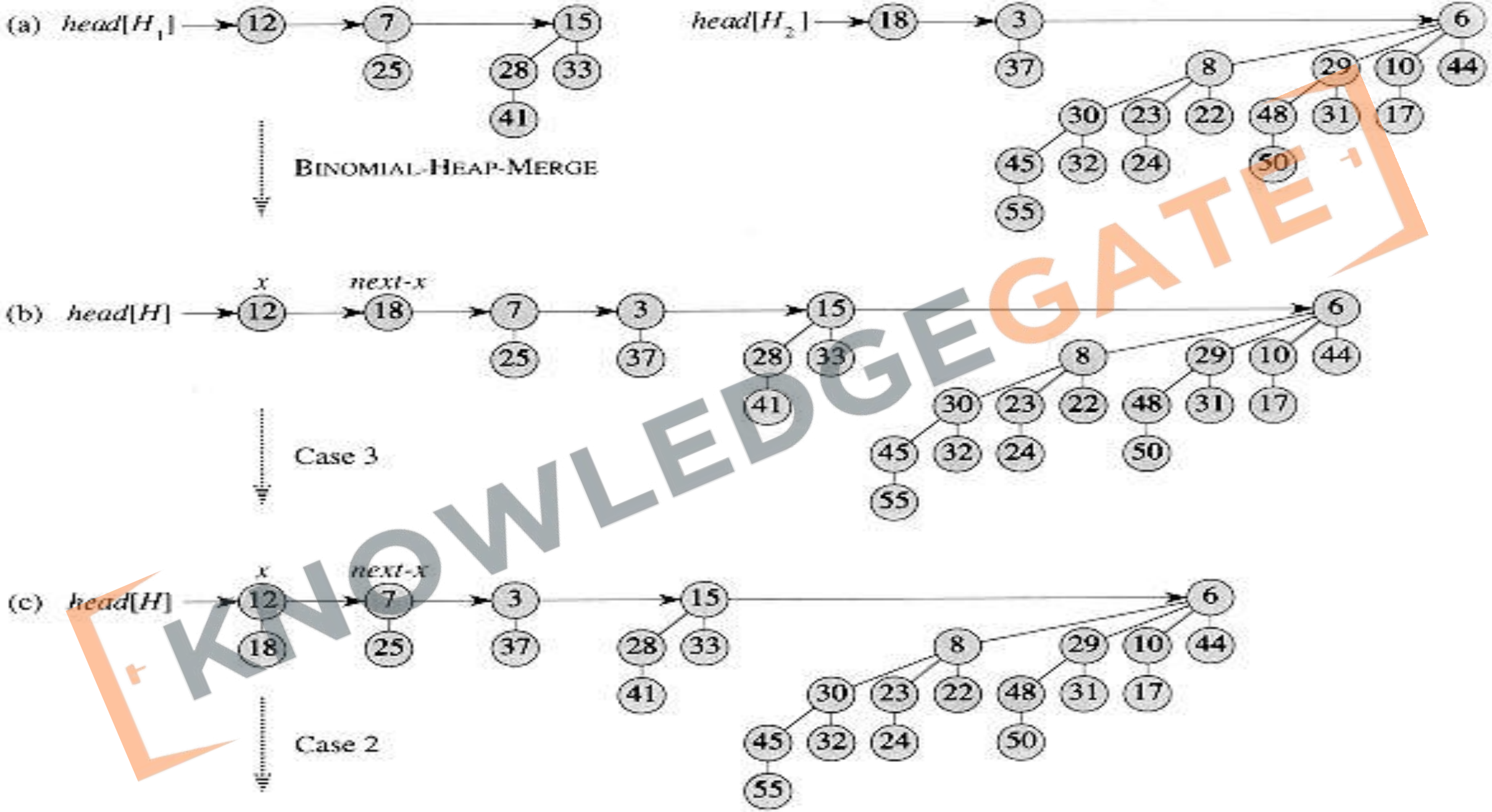
KNOWLEDGEGATE

```
BINOMIAL-HEAP-UNION(H1, H2)
 1  H ← MAKE-BINOMIAL-HEAP()
 2  head[H] ← BINOMIAL-HEAP-MERGE(H1, H2)
 3  free the objects H1 and H2 but not the lists they point to
 4  if head[H] = NIL
 5     then return H
 6  prev-x ← NIL
 7  x ← head[H]
 8  next-x ← sibling[x]
 9  while next-x ≠ NIL
10        do if (degree[x] ≠ degree[next-x]) or
                      (sibling[next-x] ≠ NIL
                      and degree[sibling[next-x]] = degree[x])
11              then prev-x ← x                          ▷ Cases 1 and 2
12                   x ← next-x                          ▷ Cases 1 and 2
13              else if key[x] ≤ key[next-x]
14                      then sibling[x] ← sibling[next-x]      ▷ Case 3
15                      BINOMIAL-LINK(next-x, x)               ▷ Case 3
16                   else if prev-x = NIL                      ▷ Case 4
17                           then head[H] ← next-x             ▷ Case 4
18                           else sibling[prev-x] ← next-x     ▷ Case 4
19                      BINOMIAL-LINK(x, next-x)               ▷ Case 4
20                      x ← next-x                             ▷ Case 4
21           next-x ← sibling[x]
22  return H
```

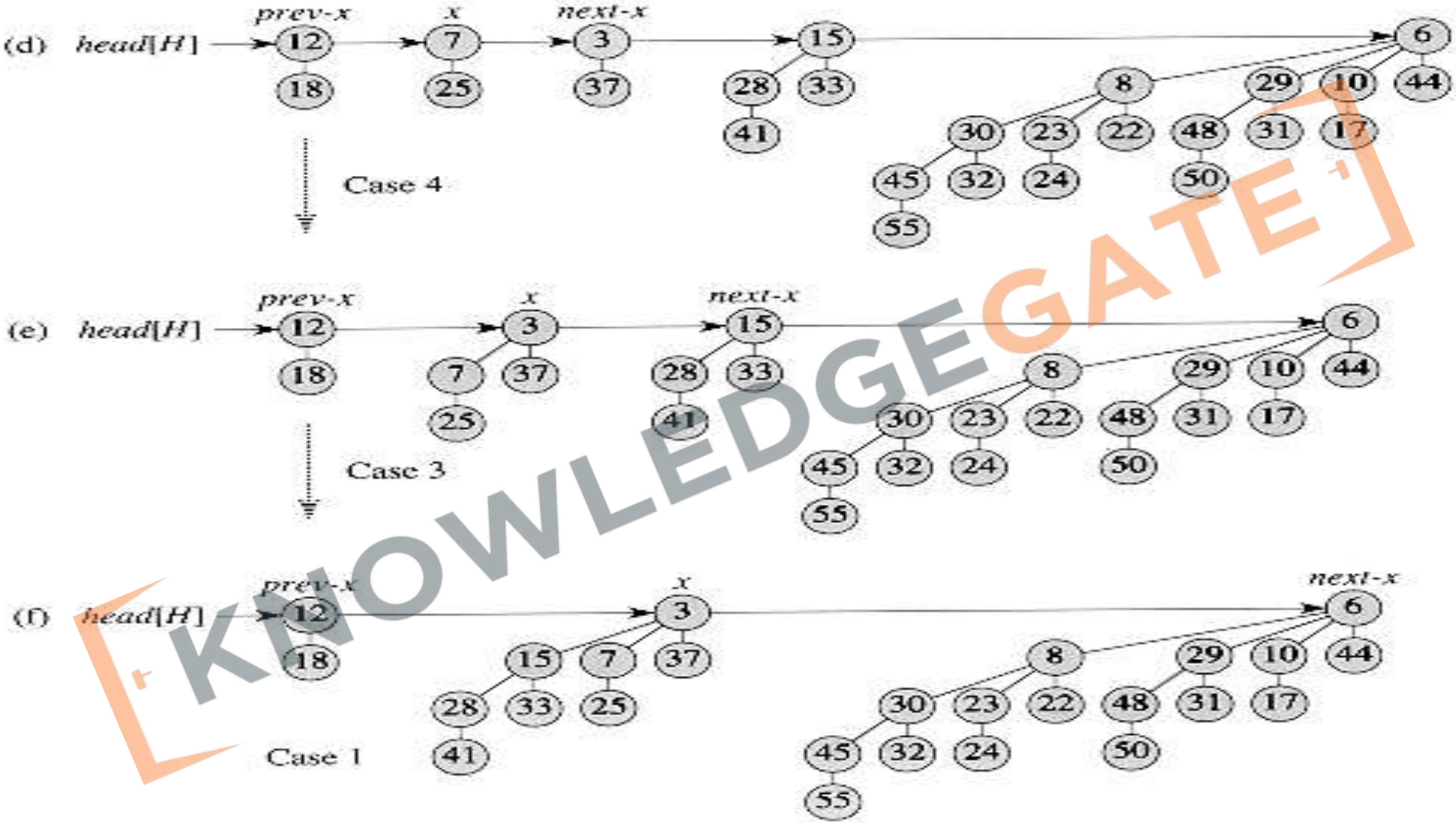
KNOWLEDGEGATE

logn

e.in/GATE



KNOWLEDGEGATE



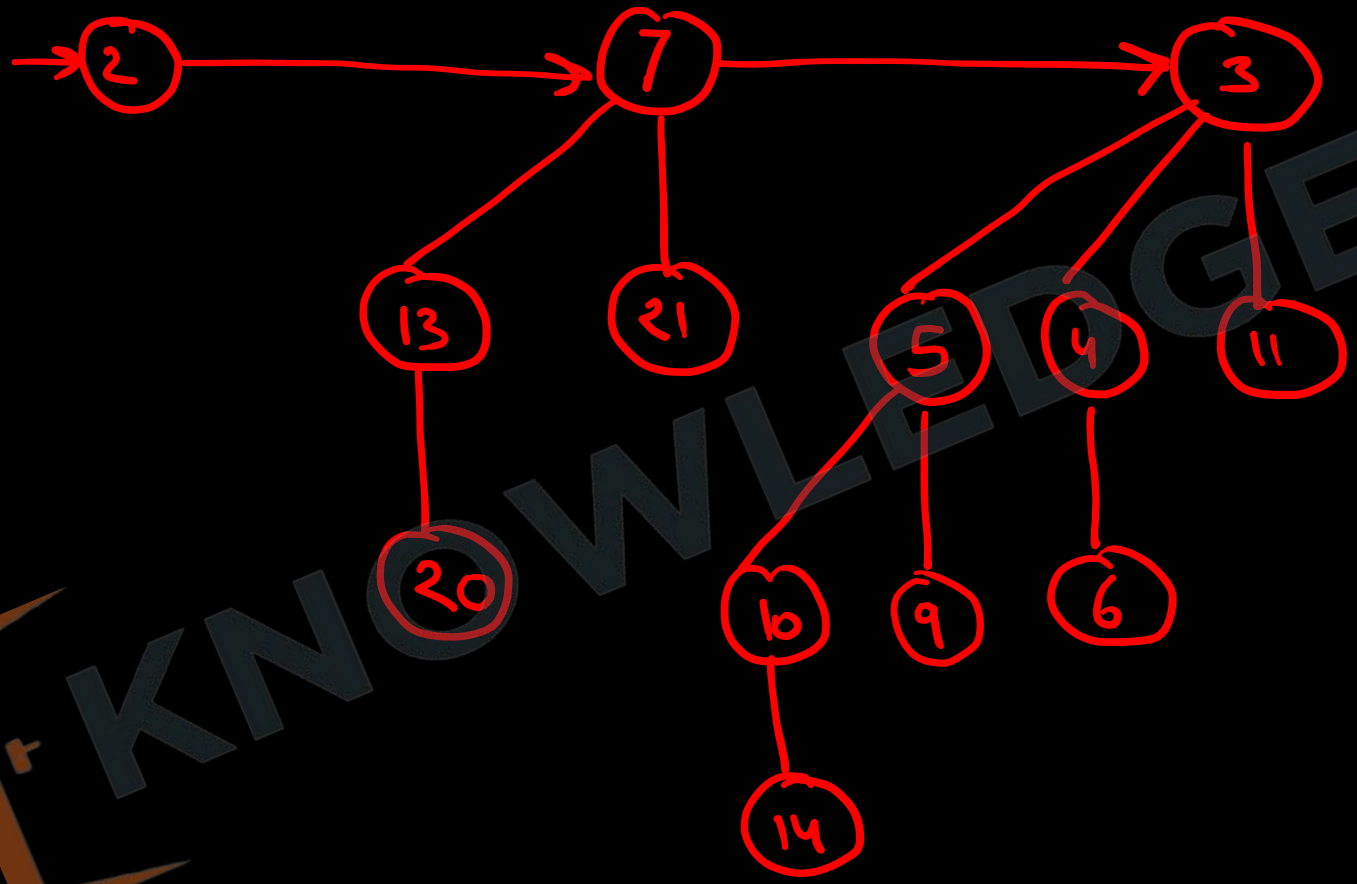
KNOWLEDGEGATE

Q Construct the binomial heap for the following sequence of number 4, 6, 3, 11, 9, 5, 14, 10, 21, 7, 13, 20, 2 ?

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Construct the binomial heap for the following sequence of number 4, 6, 3, 11, 9, 5, 14, 10, 21, 7, 13, 20, 2 ?



KNOWLEDGEGATE

Insertion in Binomial Heap Algorithm

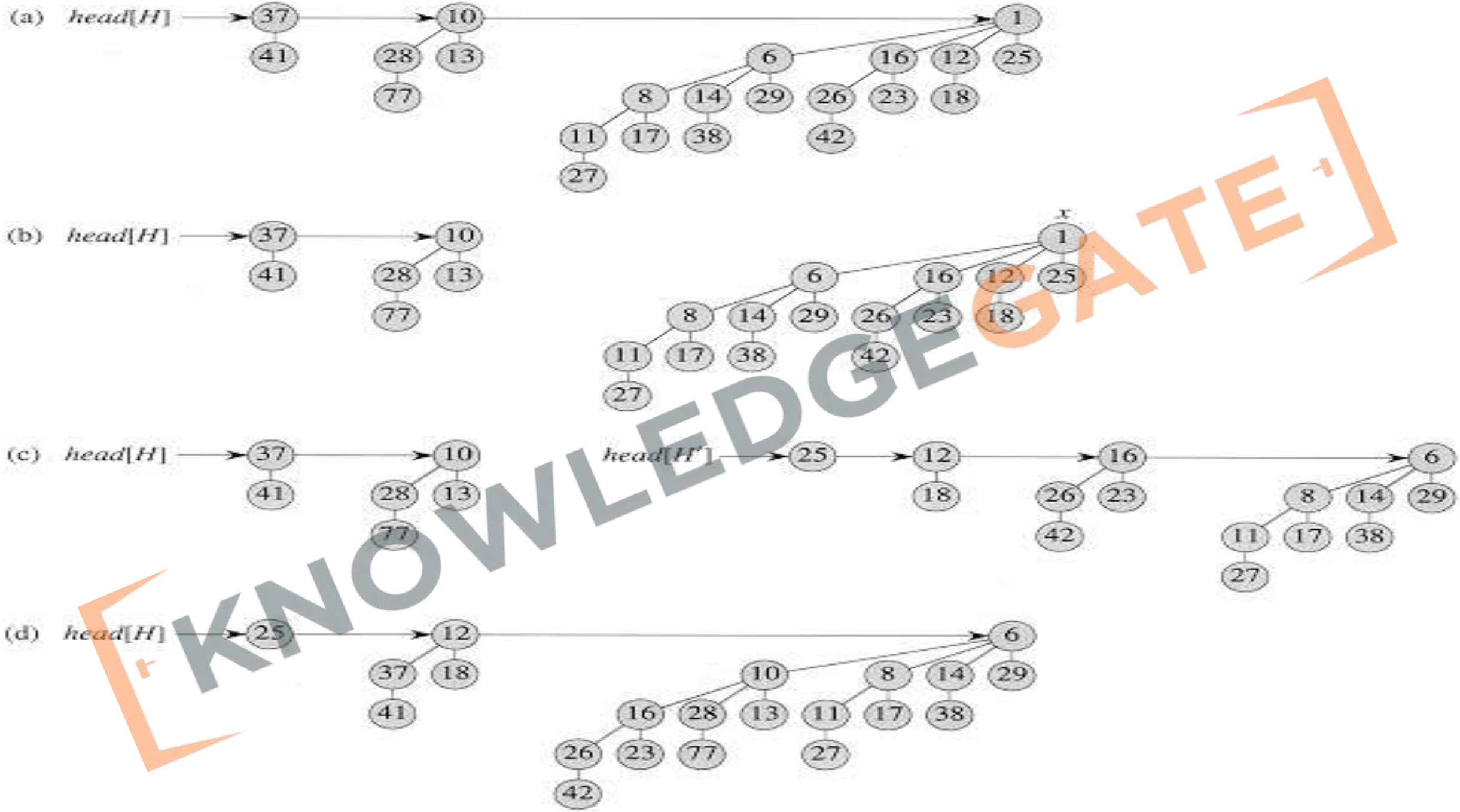
- **Create New Tree**: Form a new single-node binomial tree with the insertion value.
- **Union Heaps**: Merge this new tree with the existing binomial heap.
  - Combine trees of the same degree as necessary.
- **Update Heap**: Set the merged heap as the main heap.

KNOWLEDGEGATE

Extract Min in Binomial Heap Algorithm

- **Find Minimum**: Traverse the root list to find the binomial tree with the minimum root key. Let's call this tree B_min.
- **Remove B_min**: Remove B_min from the root list of the heap.
- **Reverse Children of B_min**: Reverse the order of the children of B_min, which are now roots of binomial trees, and create a new binomial heap H' with these trees.
- **Union Heaps**: Perform a union operation between the original heap (without B_min) and H'.
- **Update Pointers**: Update the root list and the minimum element pointer of the heap.

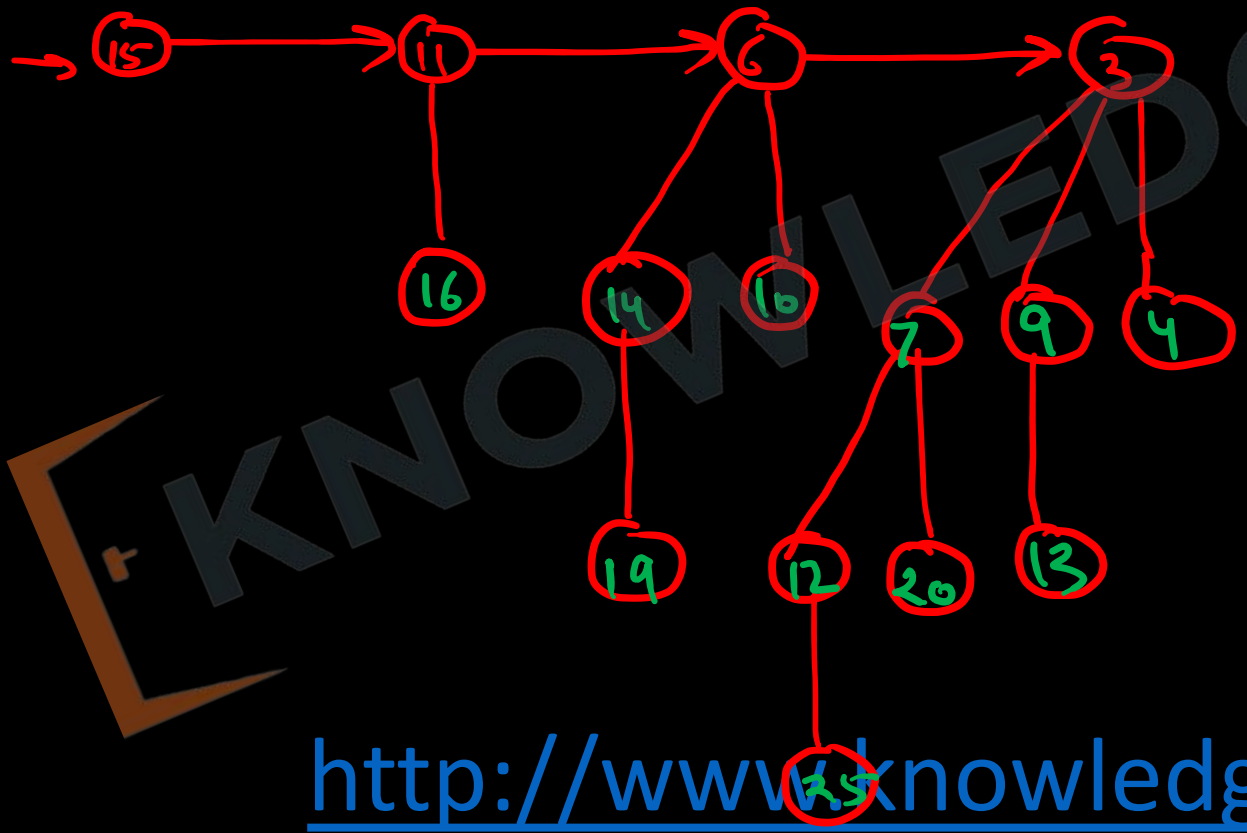
[KNOWLEDGEGATE]



KNOWLEDGEGATE

Decrease Key in Binomial Heap Algorithm

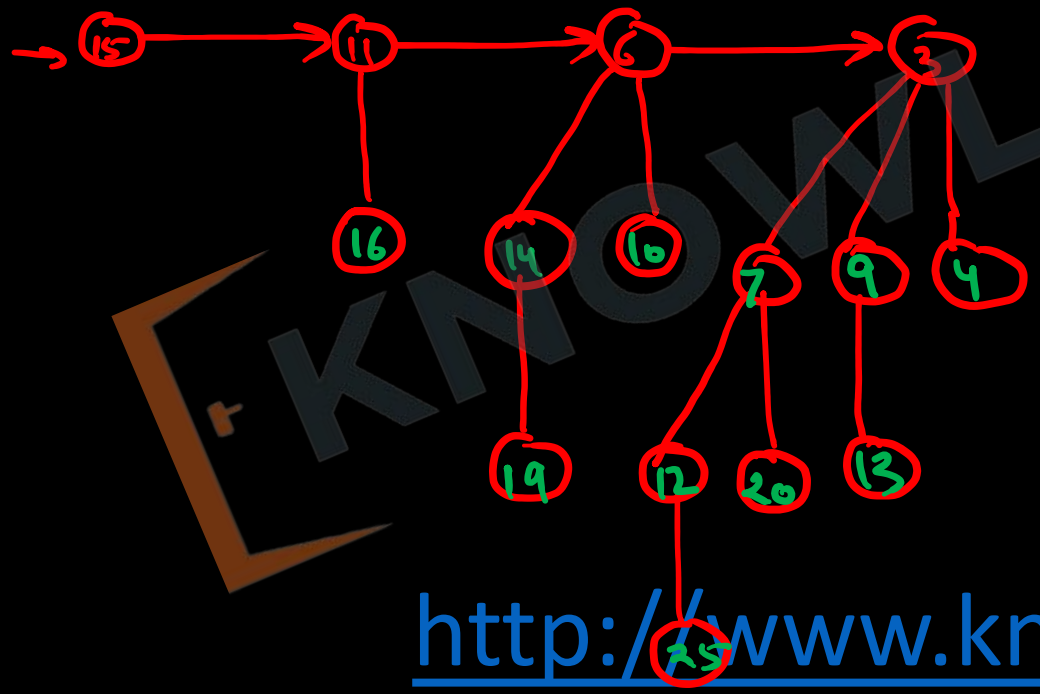
- **Update Key**: Assign new, smaller value to node x.
- **Bubble Up**: If ‘x’ key is less than its parent's, swap x with its parent. Repeat until heap order is restored.
- **Update Min**: Optionally, update the heap's minimum pointer.



KNOWLEDGEGATE

Deletion in Binomial Heap Algorithm

- **Decrease Key**: Decrease the key of the node to be deleted to a value less than or equal to the minimum value in the heap (typically this could be set to negative infinity or a value you know is the smallest possible in your system).
- **Extract Min**: Perform the 'Extract Min' operation to remove the node with the minimum key, which is now the node you wanted to delete.
- **Maintain Heap**: The 'Extract Min' operation will also take care of maintaining the heap properties after deletion.



KNOWLEDGEGATE

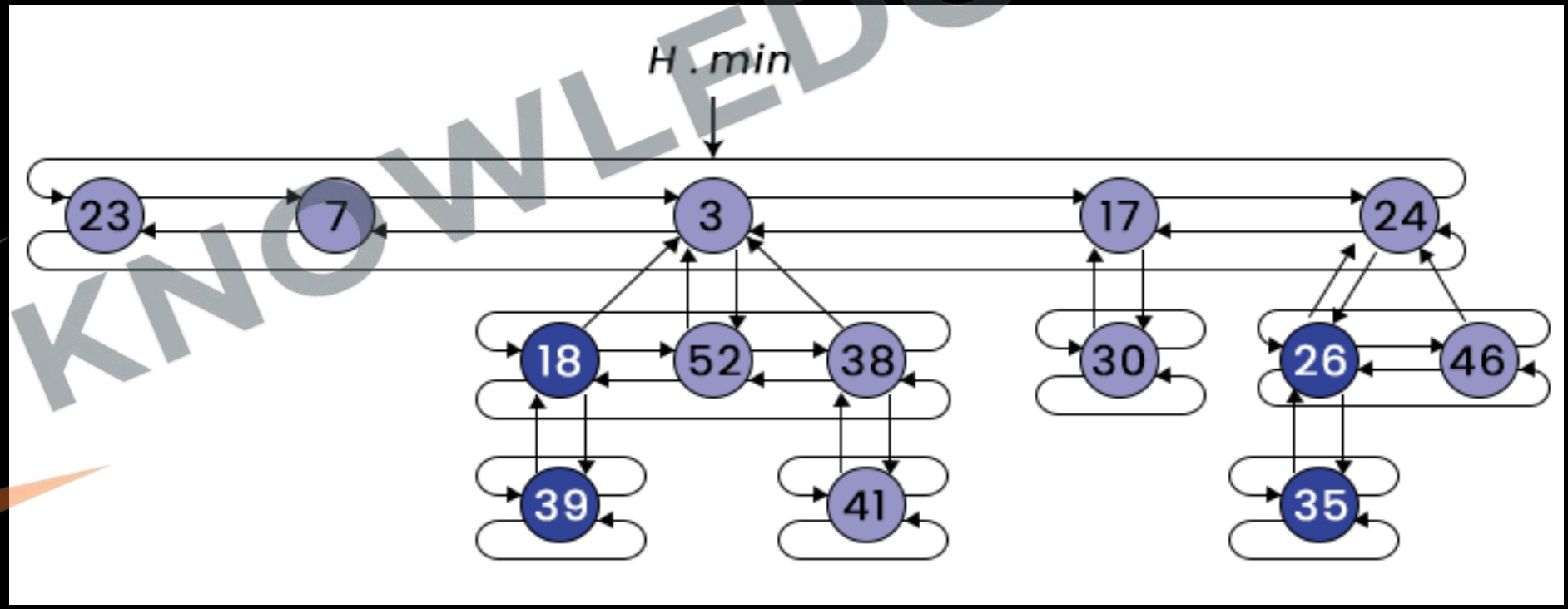
Fibonacci Heaps

- Michael L. Fredman and Robert E. Tarjan developed Fibonacci heaps in 1984 and published them in a scientific journal in 1987. Fibonacci heaps are named after the Fibonacci numbers, which are used in their running time analysis.

KNOWLEDGE GATE

Fibonacci Heaps

- Fibonacci Heaps, introduced in 1984, enhance priority queue operations for algorithms like Dijkstra's and Prim's. They feature lazy consolidation and efficient decrease-key operations with $O(1)$ amortized time, and $O(\log n)$ for deletions. Ideal for sparse graphs and network optimization, their complex structure is best for large-scale applications where their amortized time benefits are most impactful.



KNOWLEDGEGATE

- A Fibonacci Heap is a collection of unordered trees which satisfies the min-heap (or max-heap) property.

- **Structure**: It consists of a set of marked trees. Each tree is a rooted but unordered tree that obeys the min-heap property.

- **Lazy Organization**: Trees within a Fibonacci Heap are not necessarily binomial trees, and the heap does not enforce strict structure. Trees are linked together only as necessary, which usually occurs during extract-min operations.

- **Node Marking**: Nodes in a Fibonacci Heap can be marked, which indicates that a child has been lost since this node was added to its current parent. This is part of the mechanism to limit the degree (number of children) of nodes.
- **Mechanism**: When a child node is removed from a parent (which can happen during certain heap operations), the parent node gets marked. If a marked node loses another child, it's removed from its parent, potentially causing a cascade of changes up the heap.

- **Consolidation**: After the removal of the minimum node, the remaining trees are consolidated into a more structured heap. This operation is "lazy" and is done only when necessary, such as during extract-min operations, to keep the number of trees small.

- **Degree and Number of Children**: The size of a tree rooted at a node is at least F_{k+2}, where k is the degree of the node, and F is the Fibonacci sequence. This ensures that the heap has a low maximum degree.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Amortized Costs**:
  - **Insert**: $O(1)$ amortized
  - **Decrease Key**: $O(1)$ amortized
  - **Merge/Union**: $O(1)$ amortized
  - **Extract Min**: $O(\log n)$ amortized
  - **Delete**: $O(\log n)$ amortized
- The Fibonacci Heap is ideal for applications where the number of insertions and decrease-key operations vastly outnumber the number of extract-min and delete operations, thus taking advantage of the low amortized costs for the former operations. Its complexity comes into play in theory more than in practical applications due to constants and lower-order terms involved in its operations.

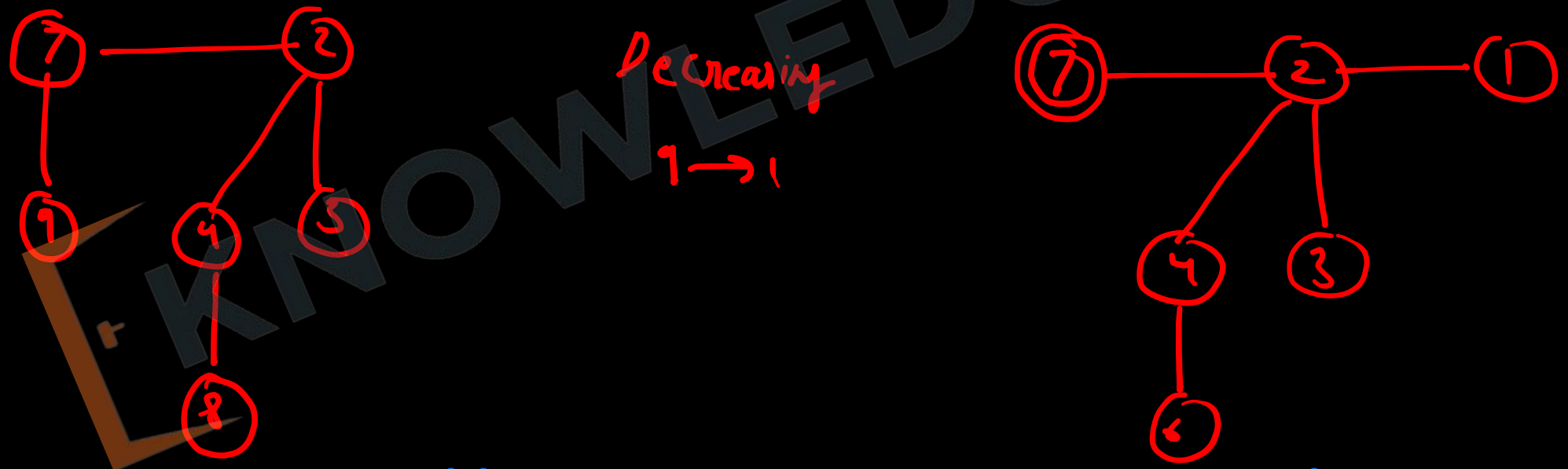
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- Insertion in a Fibonacci heap is a simple and efficient operation. Here's how it is typically implemented:
  - **Create a New Node**: A new node containing the key to be inserted is created.
  - **Add to Root List**: This new node is added to the list of roots in the Fibonacci heap.
  - **Update Minimum**: If the new key is smaller than the current minimum key in the heap, the minimum pointer is updated to this new node.
  - **Amortized Cost**: The operation runs in $O(1)$ amortized time, making it very efficient.
- The simplicity of the insertion operation is due to the lazy approach of Fibonacci heaps, where no immediate reorganization or consolidation of the heap is performed at the time of insertion.

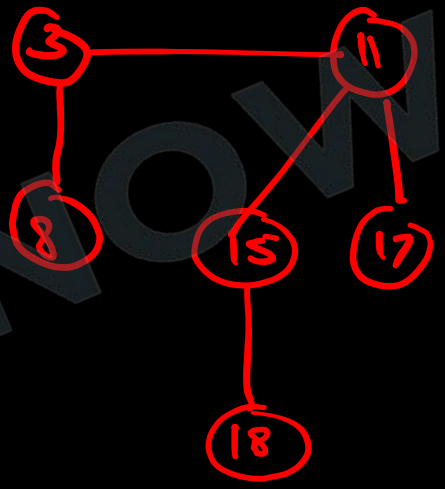
KNOWLEDGE GATE
http://www.knowledgegate.in/GATE

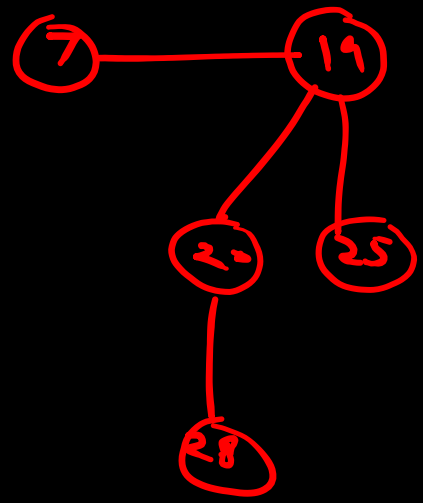
- Decrease key in a Fibonacci heap is an operation to reduce the value of a given node and then adjust the heap to maintain the heap property. Here's how it works:
  - **Decrease the Key**: Update the value of the node to the new lower key.
  - **Check Heap Property**: If the new key is still greater than or equal to the parent's key, the heap property is maintained, and no further action is required.
  - **Cut and Add to Root List**: If the new key violates the heap property (it's now less than the parent's key), cut this node from its parent and add it to the root list.
  - **Mark the Parent**: If the parent node was unmarked, mark it. If it was already marked, cut it as well and add it to the root list, then continue this process recursively up the tree.
  - **Update Minimum**: If the new key is less than the current heap's minimum key, update the minimum pointer.



KNOWLEDGEGATE

- Merging in a Fibonacci heap, also known as the union operation, is the process of combining two Fibonacci heaps into a single heap. This operation is efficient and is performed in the following way:
  - **Concatenate Root Lists**: Combine the root lists of the two heaps into a single root list, which can be done in $O(1)$ time since the root lists are typically circular doubly linked lists.
  - **Update Minimum**: Check the minimum nodes of both heaps and update the pointer to the minimum node if necessary.
  - **No Immediate Consolidation**: Unlike other heap structures, the trees are not consolidated right away. This is part of the lazy strategy of Fibonacci heaps, which delays work until it's needed (e.g., during an extract-min operation).
  - **Amortized Cost**: The amortized cost of the merge operation is $O(1)$.
- The merge operation in Fibonacci heaps is a foundational operation that supports the efficiency of other heap operations, like insertions and decrease-key, because it enables the heap to maintain a more flexible structure.

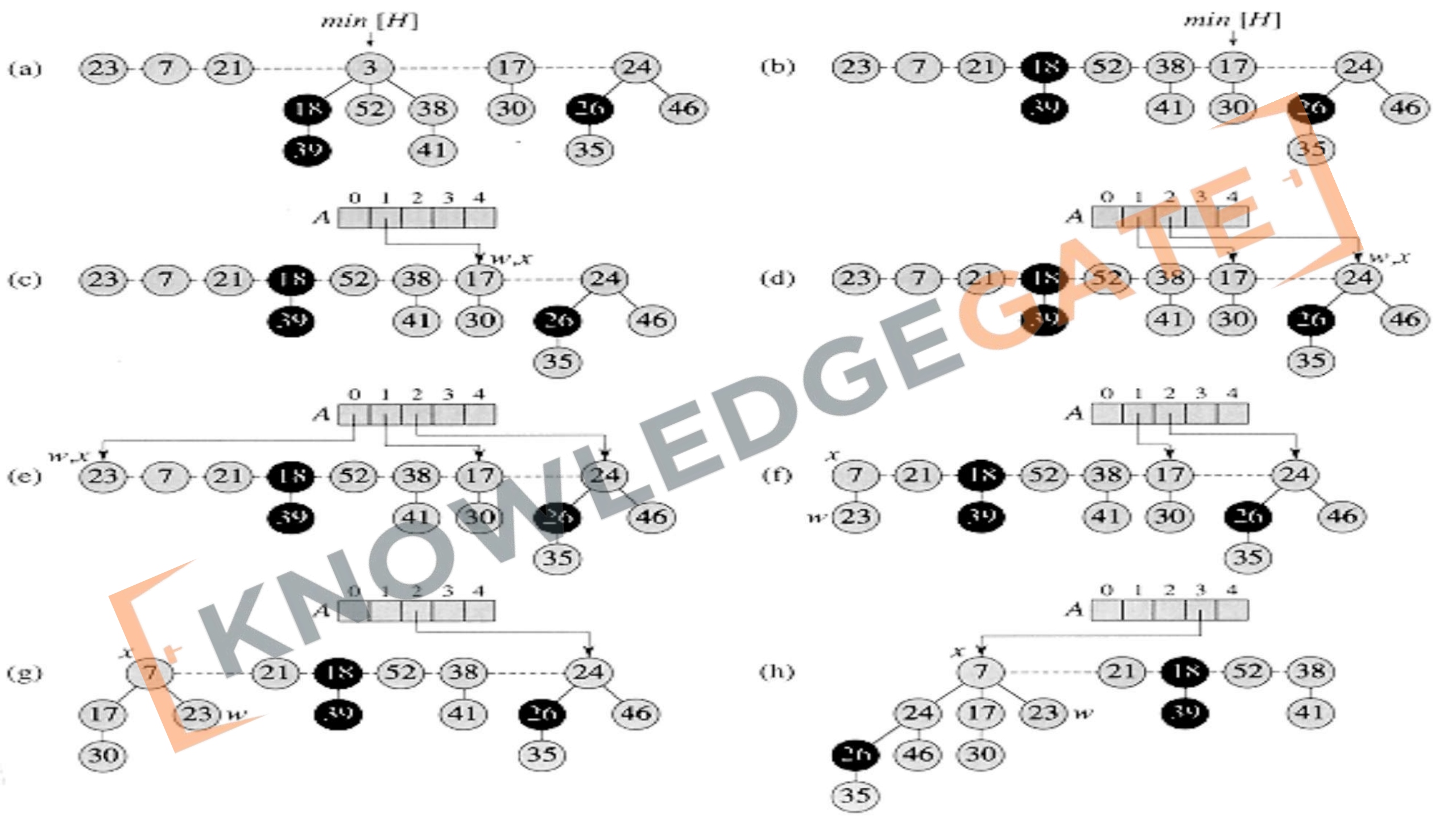




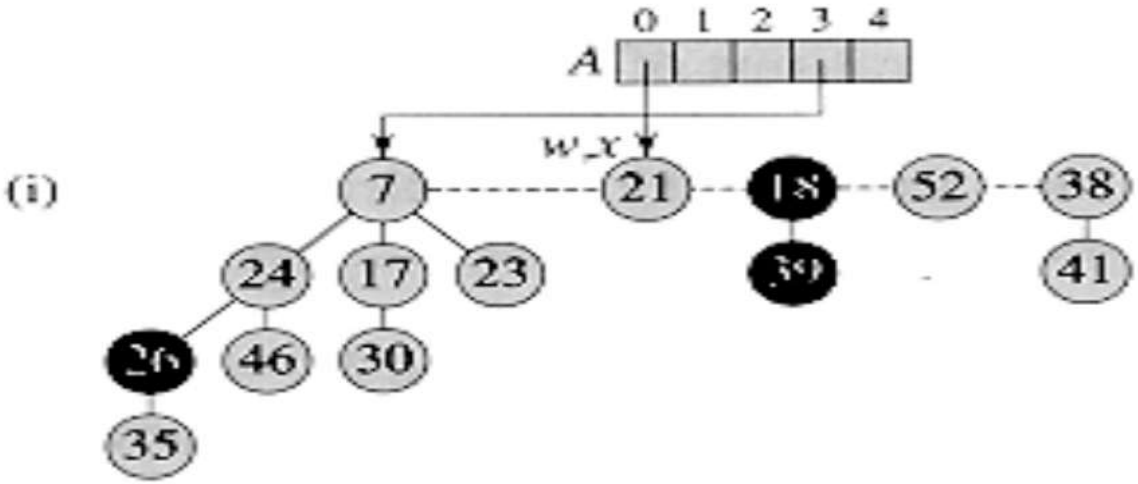
KNOWLEDGEGATE

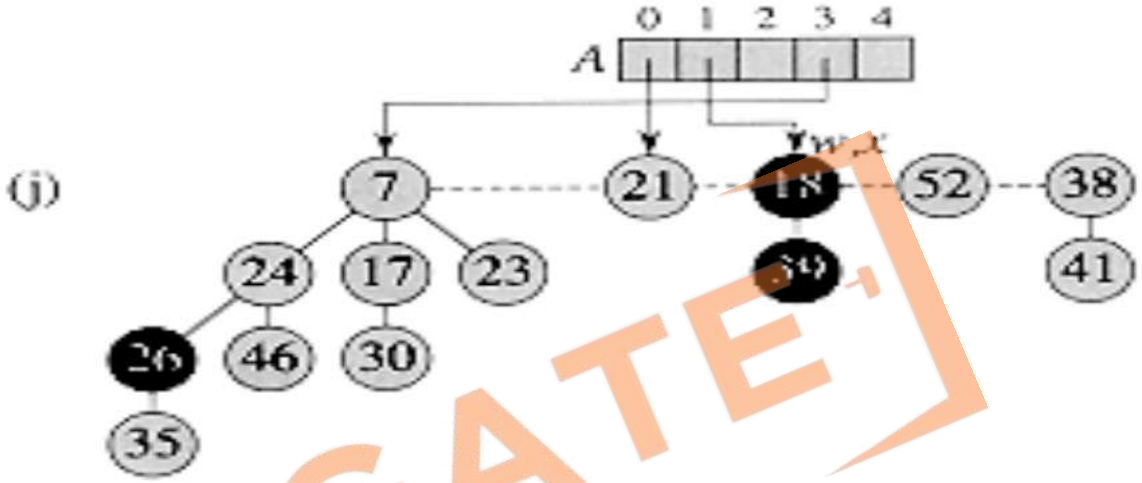
- The "Extract Min" operation in a Fibonacci heap is used to remove and return the smallest key in the heap. It's a key operation that involves more restructuring of the heap compared to other operations. Here's a step-by-step description:
- **Remove the Minimum Node**: Take out the node containing the minimum key from the root list. This node will be returned at the end of the operation.
- **Add Children to Root List**: The children of the minimum node are added to the root list of the heap.
- **Consolidate the Heap**: This step restructures the heap to ensure that no two trees have the same degree in the root list:
  - Pairwise combine trees in the root list that have the same degree until every tree has a unique degree.
  - During this process, link trees by making the one with the larger root a child of the one with the smaller root.
- **Find New Minimum**: Traverse the root list to find the new minimum node and update the heap's minimum pointer.
- **Time Complexity**: The worst-case time complexity for this operation is $O(\log n)$, but this is the amortized complexity because the actual work is done during the consolidation step, which doesn't happen on every operation.
- **Amortized Cost**: The amortized cost is $O(\log n)$ due to the potential increase from the previous operations which pays for the consolidation.

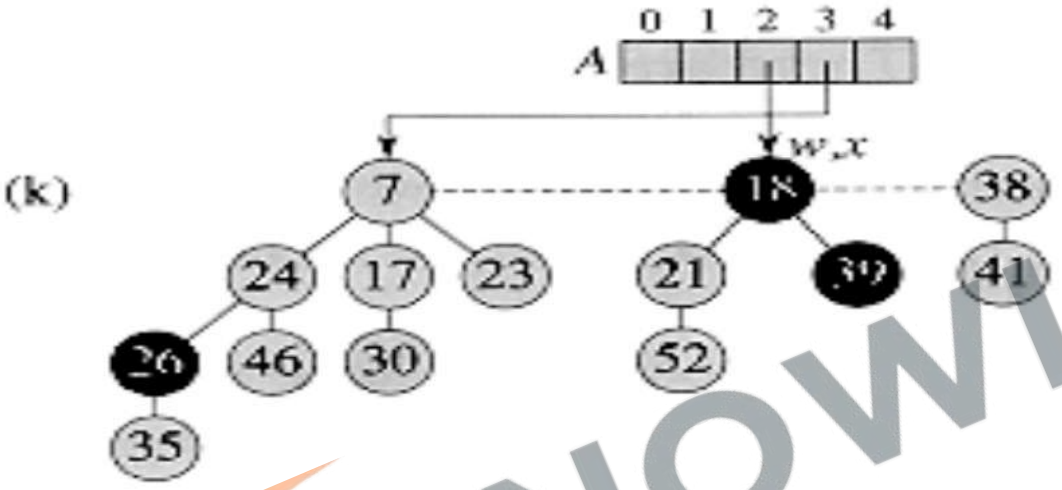
KNOWLEDGE GATE

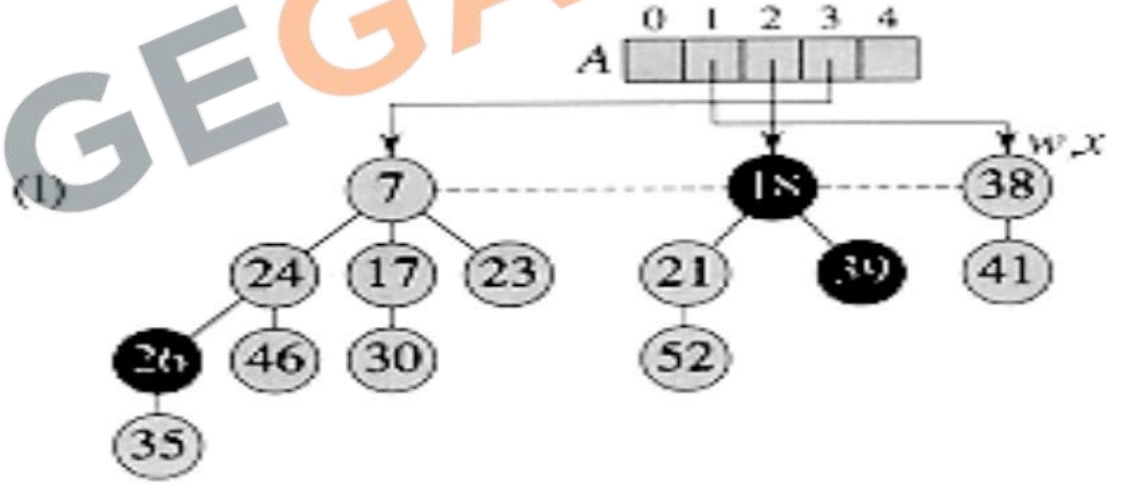


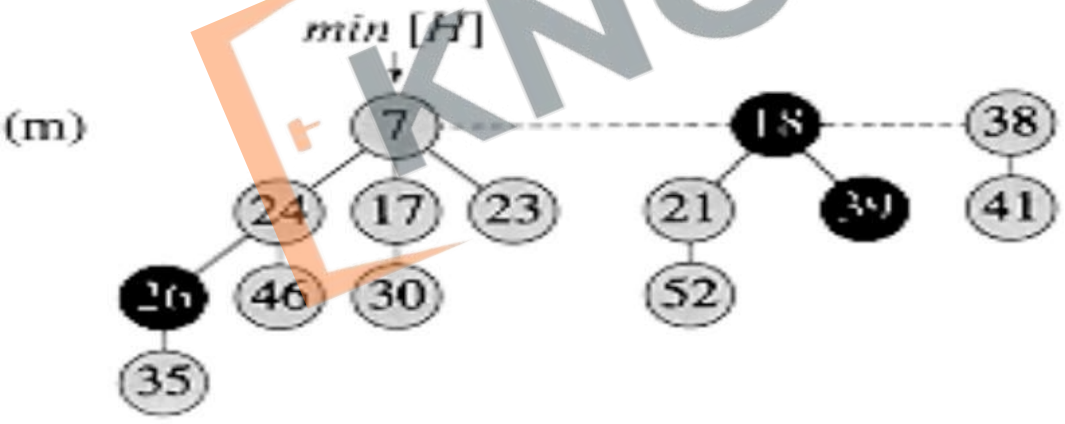
KNOWLEDGEGATE







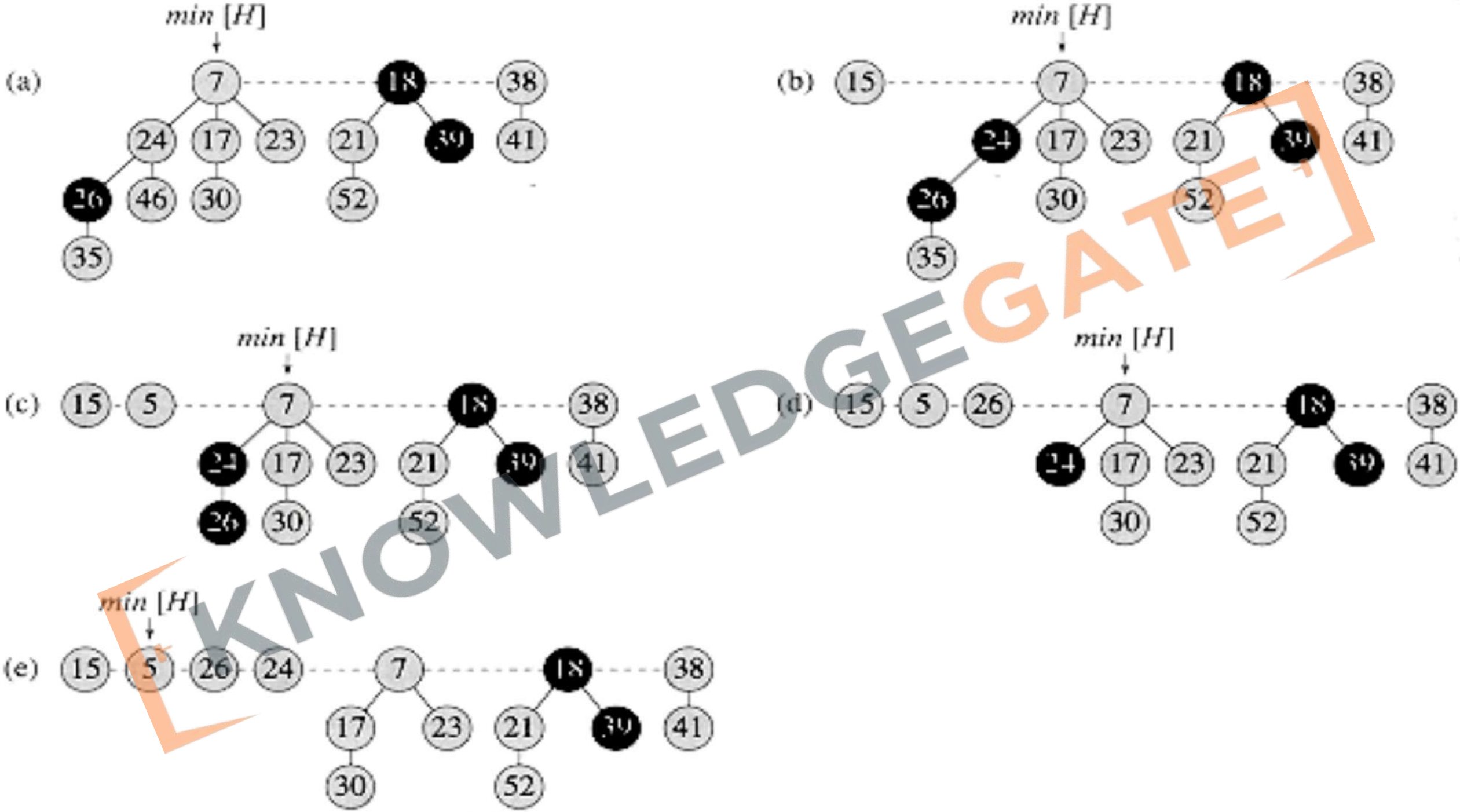




KNOWLEDGEGATE

- The delete operation in a Fibonacci heap removes a specific node from the heap. It's more complex than the extract-min operation because it requires locating the node to be deleted, regardless of its position in the heap. Here's how the delete operation works:
  - **Decrease Key to Minus Infinity**: Decrease the key of the node to be deleted to the smallest possible value. This is often done by setting it to negative infinity or a value lower than any other in the heap.
  - **Extract Min**: Perform the extract-min operation, which will now remove the node with the decreased key since it's the smallest in the heap.
  - **Consolidate the Heap**: If necessary, during the extract-min operation, the heap is consolidated to ensure that it maintains the correct structure, merging trees where necessary.
  - **Time Complexity**: The time complexity for the delete operation is $O(\log n)$ amortized, due to the extract-min operation that is invoked.
- By combining the decrease-key and extract-min operations, the delete operation ensures that the heap remains properly structured and that the min-heap property is maintained throughout.

KNOWLEDGE GATE



KNOWLEDGEGATE

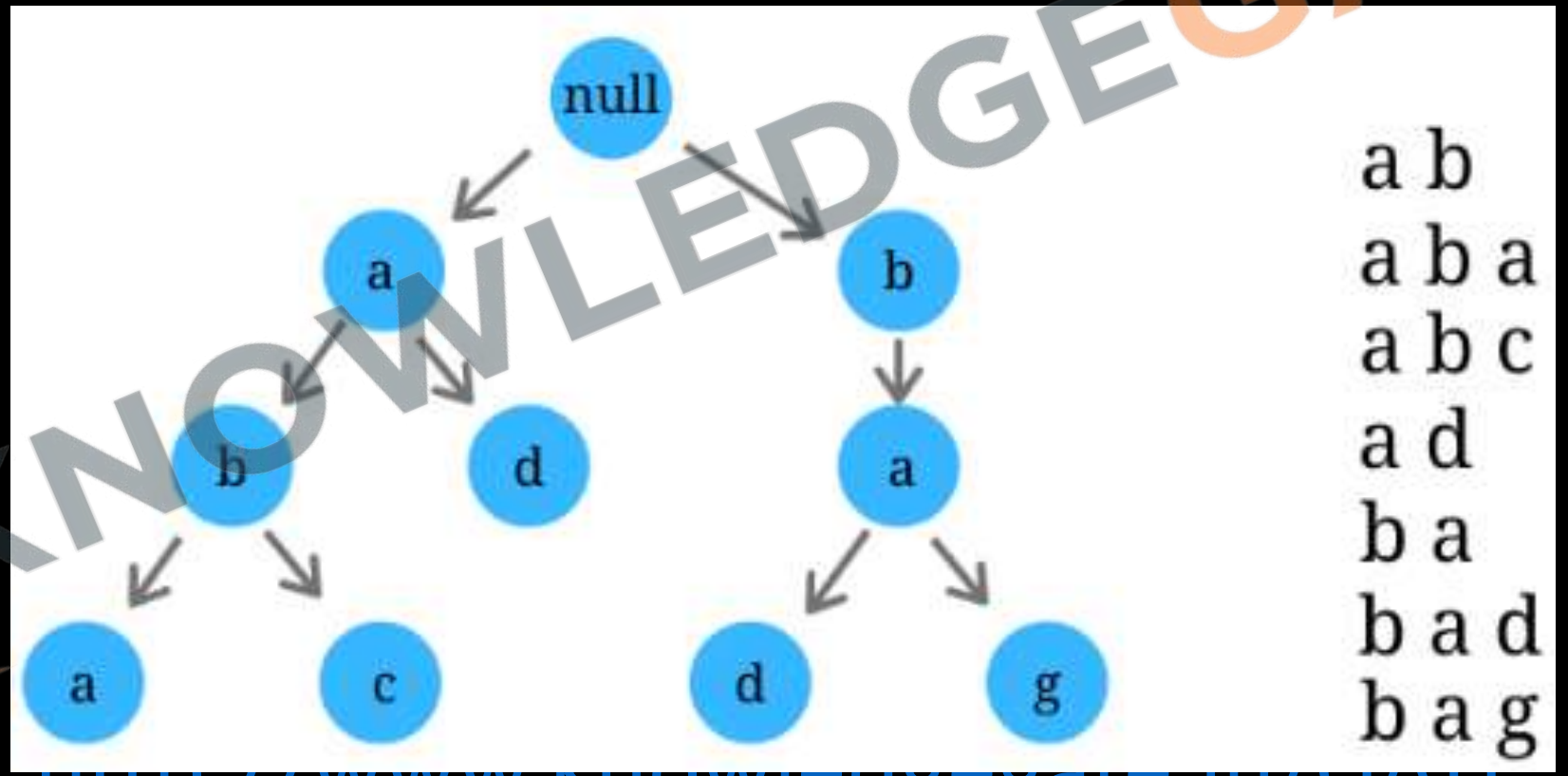
Procedure	Binary Heap	Binomial Heap	Fibonoci Heap
Make Heap	O(1)	O(1)	O(1)
Insert	O(logn)	O(logn)	O(1)
Min	O(1)	O(logn)	O(1)
Extract min	O(logn)	O(logn)	O(logn)
Union	O(n)	O(logn)	O(1)
Decrease Key	O(logn)	O(logn)	O(1)
Delete	O(logn)	O(logn)	O(logn)

KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

Tries

- A Trie, also known as a prefix tree, is a tree-like data structure that stores a dynamic set of strings, usually for retrieval of keys by prefix. It is a type of search tree—an ordered tree data structure used for locating specific keys from within a set.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Requirements:

- **Efficient Storage**: Tries are used to store strings efficiently for retrieval by prefix, which can reduce space requirements compared to naive storage methods.
- **Prefix Lookup**: Tries must support fast lookup to check whether a string is in a set, which is particularly efficient when dealing with prefixes.
- **Dynamic**: They must allow insertion and deletion of keys.
- **Definition:**
  - A Trie is a hierarchical data structure consisting of nodes with pointers to their children along with associated characters. Each node represents a common prefix shared by some keys, and each path down the tree represents a key (usually a string).

KNOWLEDGEGATE

- **Properties:**
  - **Edges**: Each edge in a Trie is labelled with a character or a substring.
  - **Root Node**: The root node is usually an empty string and does not hold any character.
  - **Nodes**: Internal nodes, which can have multiple children, store the common prefixes of the strings. Leaf nodes represent the end of a particular key.
  - **Path**: A path from the root to a leaf or a node with a non-null value represents a word or a prefix to a word.
  - **Time Complexity**: The time complexity for inserting, searching, and deleting from a Trie is O(m), where m is the length of the key.

KNOWLEDGEGATE

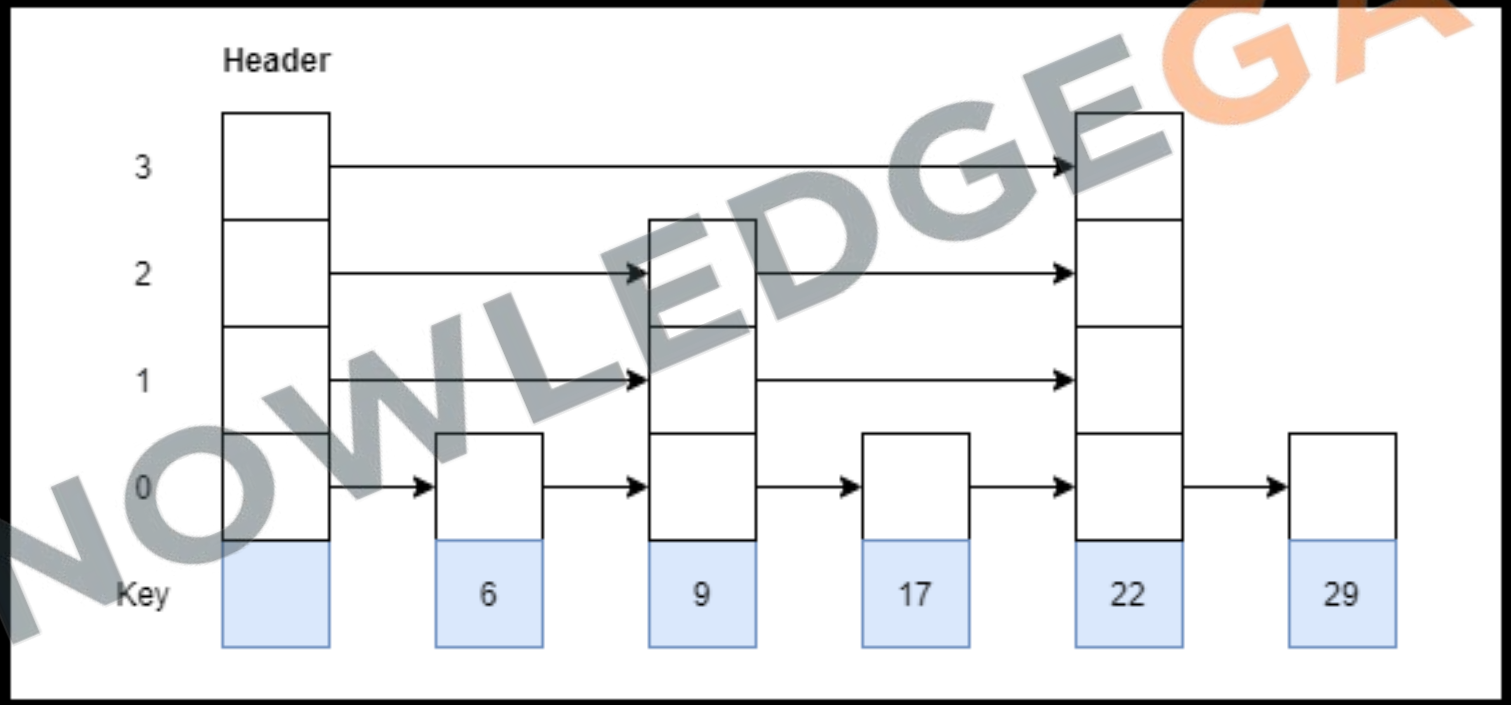
http://www.knowledgegate.in/GATE

- **Autocomplete**: Tries are well-suited for implementing autocomplete features in search engines and text editors due to their efficient prefix searching capabilities.
- **Spell Checking**: They can be used for spell checking as one can quickly check the existence of a word or prefix.
- **IP Routing**: Tries are used in IP routing to store and search IP addresses.
- **Bioinformatics**: In bioinformatics, tries are used for tasks such as genome analysis where sequences of DNA are stored and searched.
- **Text Games**: Games like Scrabble use tries to validate words and search for possible words with given letters.

KNOWLEDGEGATE

Skip List

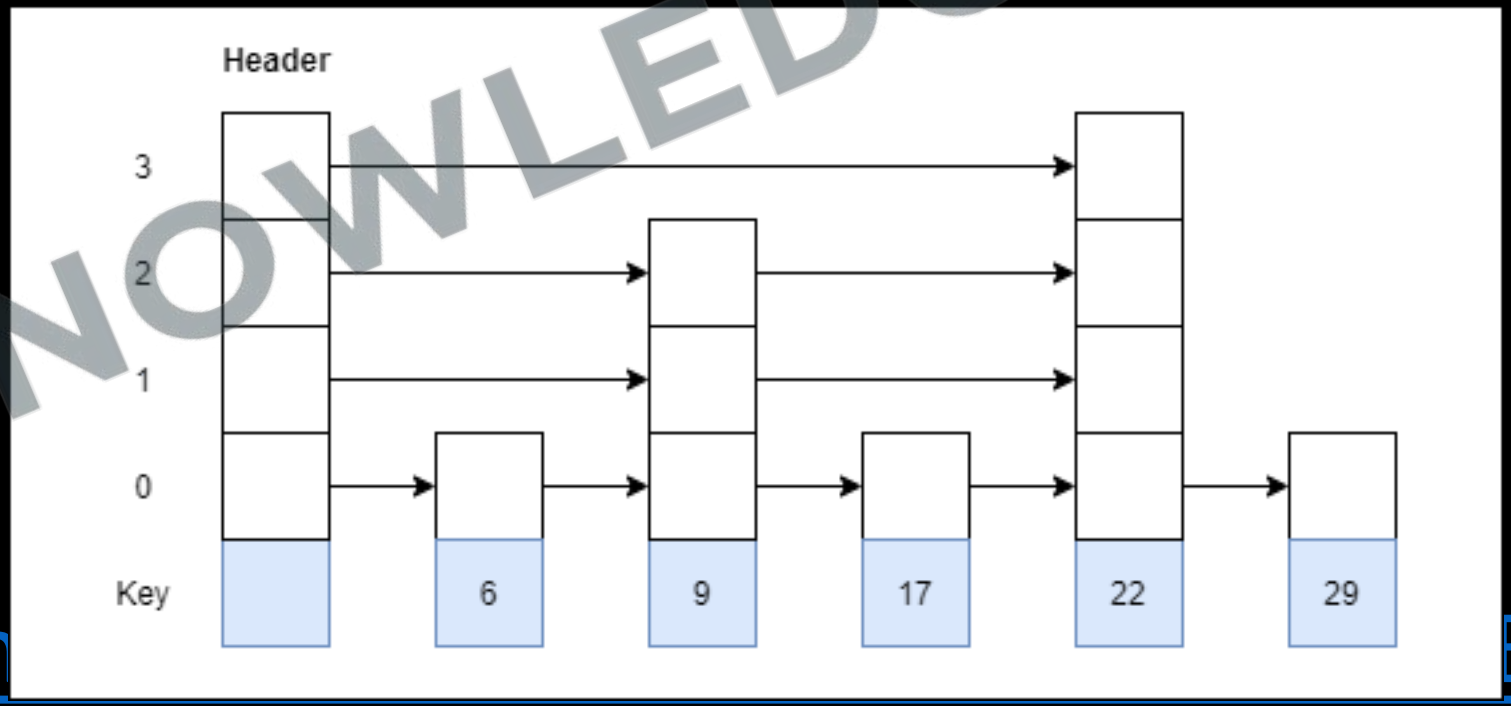
- A Skip List is a data structure that allows fast search within an ordered sequence of elements. It does this by maintaining a hierarchy of linked lists that skip over a subset of the elements.



KNOWLEDGEGATE

Requirements:

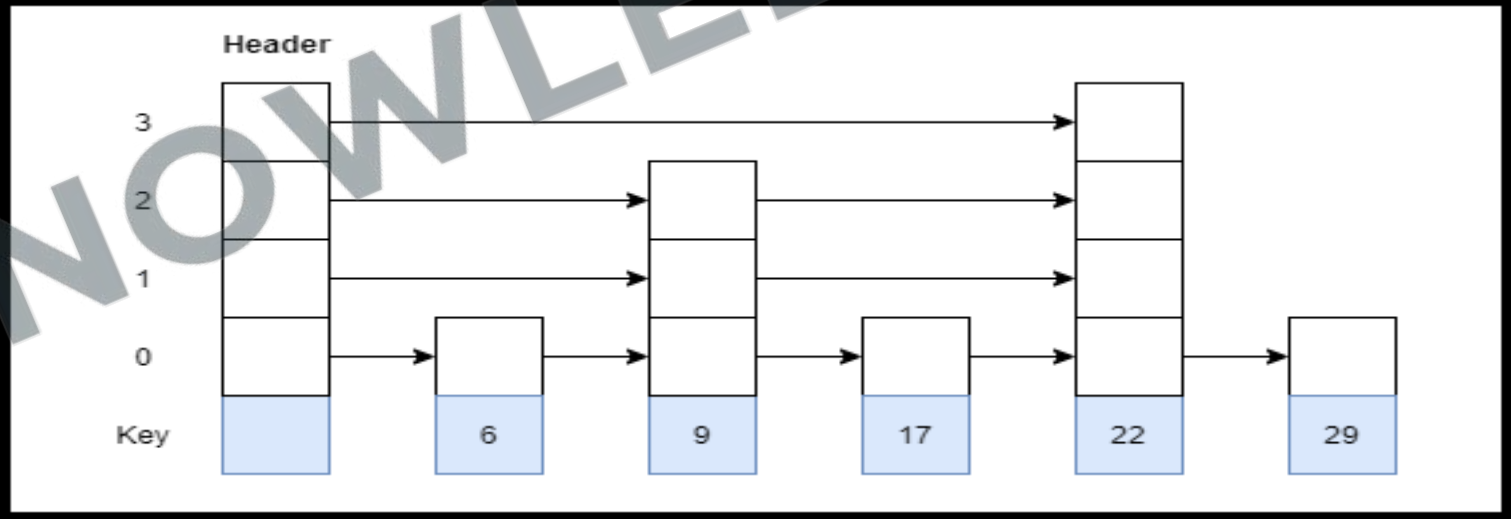
- **Ordered Elements**: Elements must be maintained in a sorted order.
- **Multiple Levels**: Skip lists consist of several layers of linked lists, where the bottom layer is the ordinary list of all elements.
- **Randomization**: The structure of skip lists is probabilistic, typically decided by coin flips or a random number generator to determine the height of each element's "tower."



KNOWLEDGEGATE

Properties:

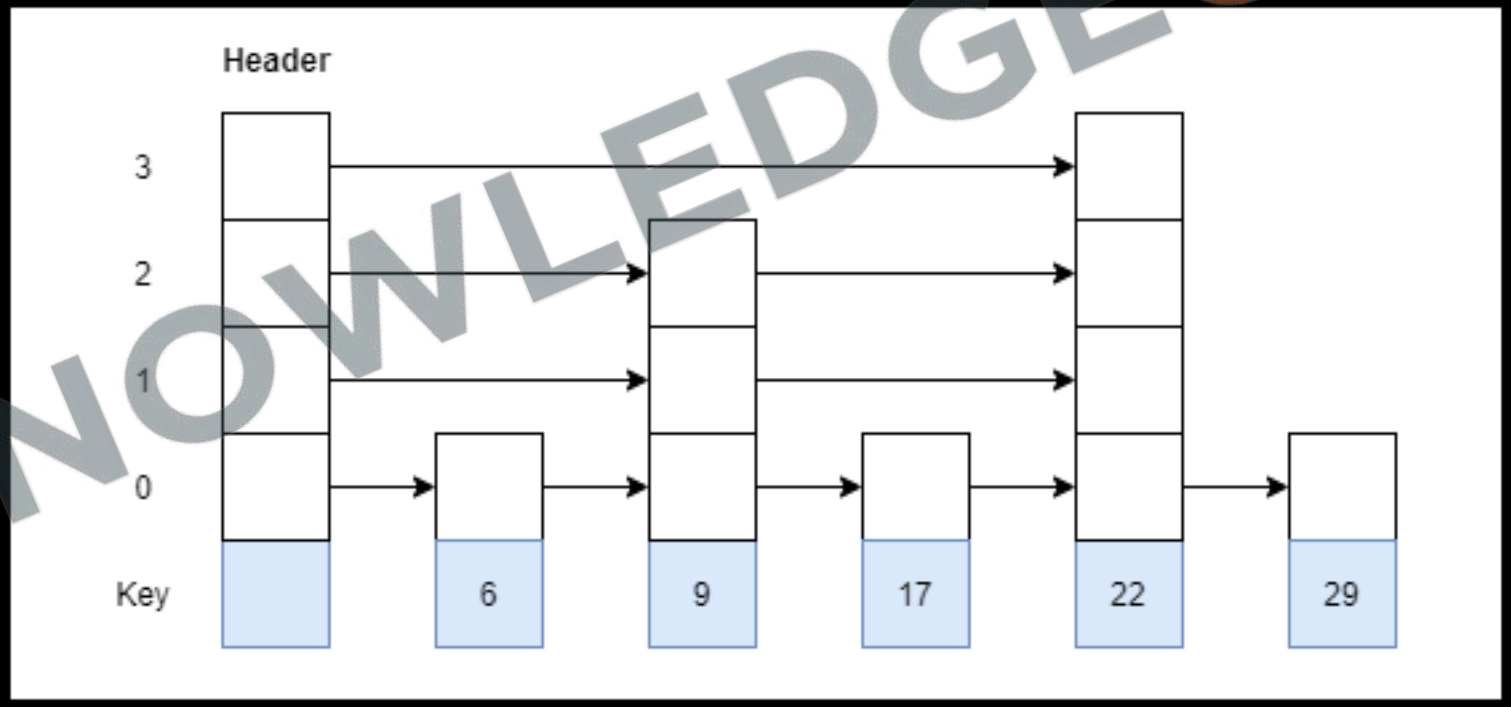
- **Efficient Search**: Provides logarithmic search time similar to binary search trees.
- **Probabilistic Balancing**: Unlike trees, skip lists do not require re-balancing operations.
- **Ease of Implementation**: Simpler to implement than balanced trees.
- **Memory Overhead**: May require additional space compared to a standard linked list due to multiple levels.



KNOWLEDGEGATE

Applications:

- **Databases**: Skip lists are used in database indices where quick search, insertion, and deletion are necessary.
- **Probabilistic Data Structures**: Can be part of larger data structures that deal with approximations or probabilistic results.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Insertion Algorithm:

- Search for the position where the new element should be inserted, starting from the top level and moving down.
- Insert the element in the lowest level linked list.
- Randomly decide the number of levels the new element should participate in.
- Add the element to each of the higher levels according to the decision in step 3.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Deletion Algorithm:

- Locate the element in the bottom list and in every list where it appears.
- Remove the element from all the levels of the list where it is found.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Searching Algorithm:

- Start from the head element of the top list.
- Move forward until the next element is greater than the search key.
- Move down one level and continue the search.
- If the bottom level is reached, the search either finds the element or determines it's not in the list.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Complexity:

- **Search**: Average $O(\log n)$, Worst-case $O(n)$ (if unlucky with randomization).
- **Insertion**: Average $O(\log n)$, Worst-case $O(n)$.
- **Deletion**: Average $O(\log n)$, Worst-case $O(n)$.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Ch-9

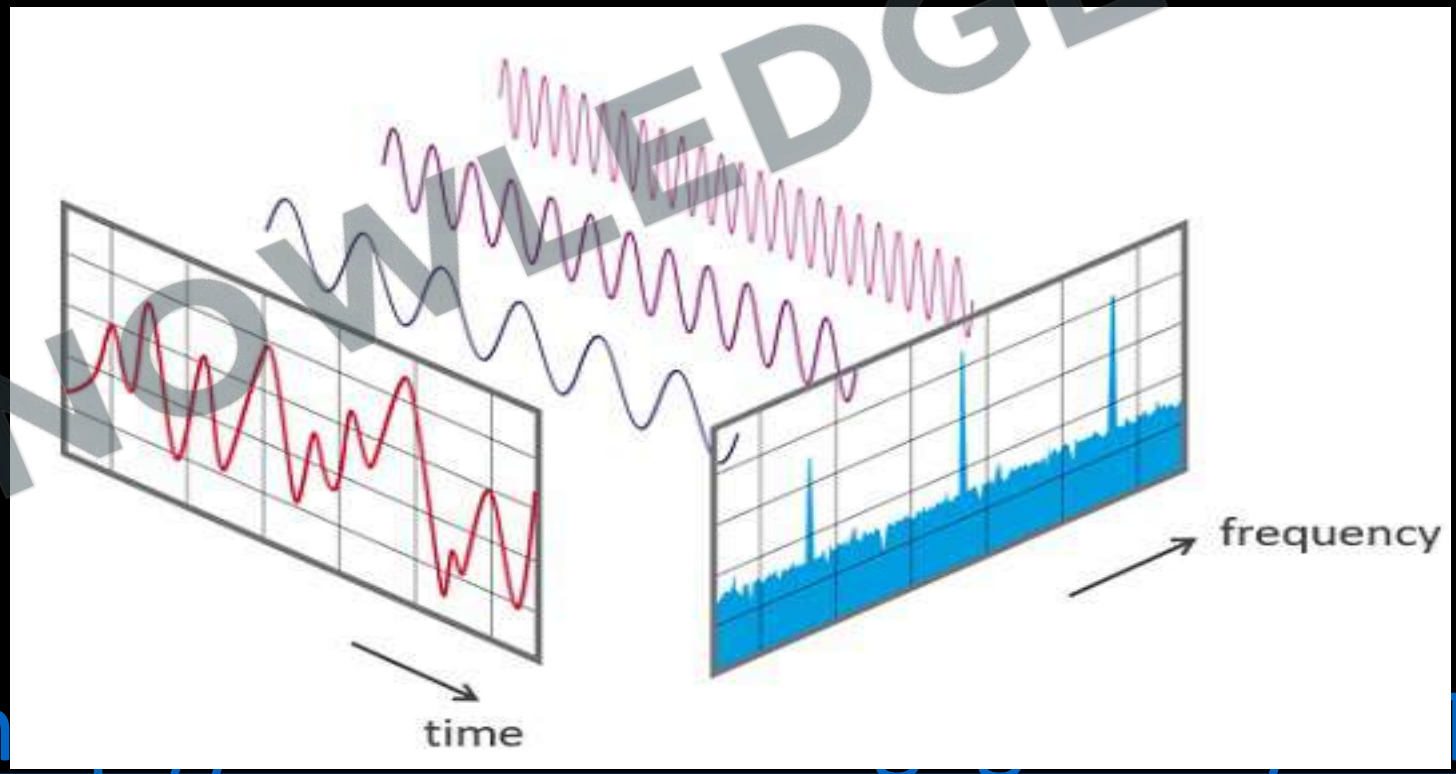
Selected Topics

Algebraic Computation, Fast Fourier Transform, String Matching, Theory of NPCompleteness, Approximation Algorithms and Randomized Algorithms .

KNOWLEDGEGATE

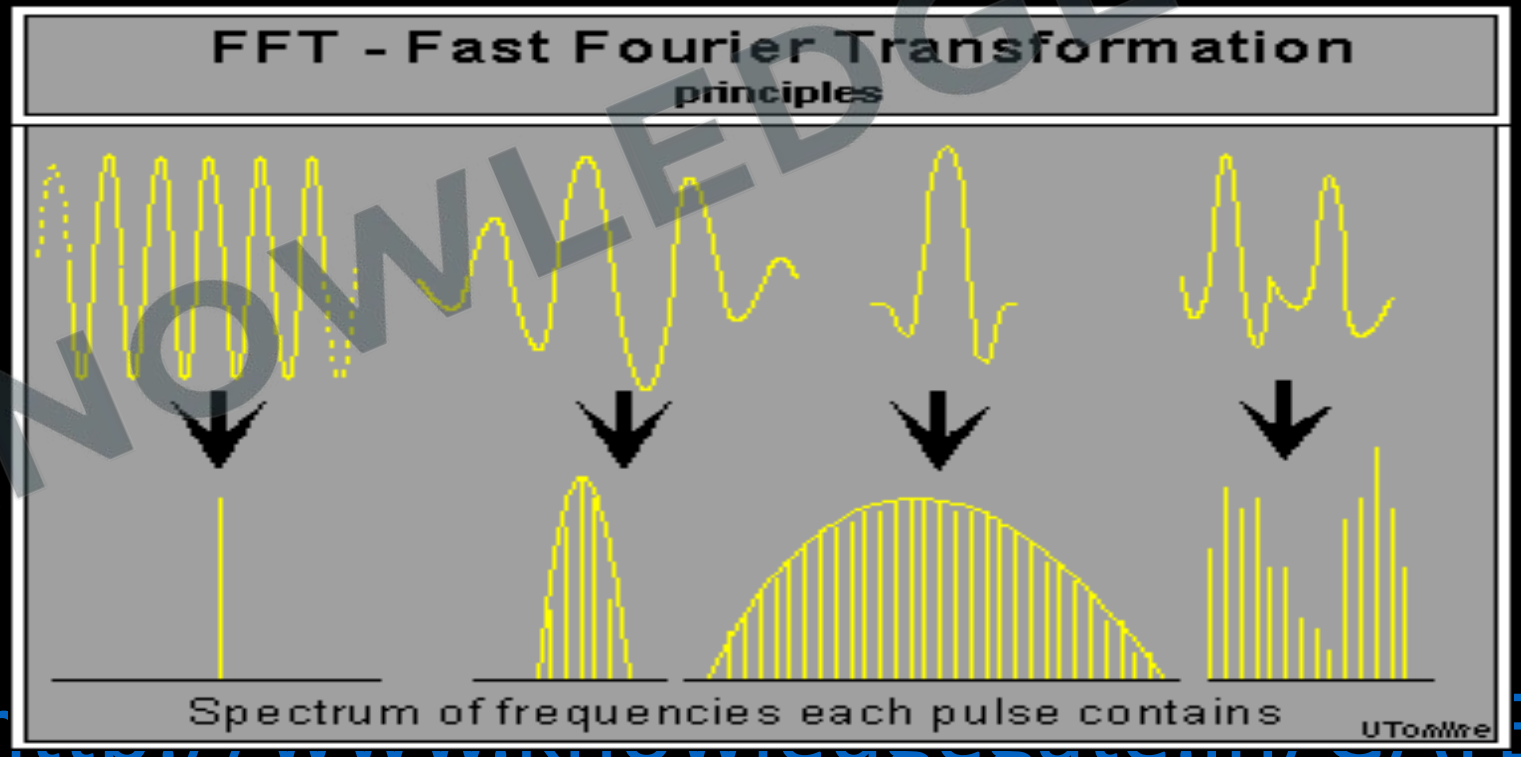
Fast Fourier Transformation

- **What it is**: FFT is a quick way for computers to change how they look at a list of numbers. Instead of seeing them just as a sequence over time, it helps the computer see what cycles or patterns (frequencies) are in that list.
- **Why it's special**: It does its job really fast compared to older methods. Imagine having to check every book in a library to find one book, versus having a quick way to go right to the shelf where the book is.



KNOWLEDGEGATE

- **Where it's used**: It's like a tool in a toolbox that can be used for many things, like making music sound better, getting clear pictures for doctors to see inside the body, and even for phones to send messages without wires.
- **Types**: There's a common kind of FFT that works best when the list of numbers has a total that's a "power of two" (like 2, 4, 8, 16, and so on).
- **What it does**: It helps us understand different pitches in music or bright and dark spots in images by breaking them down into simpler parts that are easier to work with or understand.



KNOWLEDGE GATE

String matching

- String matching, often referred to as "pattern matching," is the process of finding one or more instances of a string (pattern) within another string (text). The goal is to find all the positions at which the pattern appears in the text.
- **String Matching Problem:** The string-matching problem is a classic problem in computer science where you want to find all occurrences of a shorter string (a "pattern") within a longer string (a "text"). It's a foundational problem with applications in text editing, DNA sequence analysis, and information retrieval, among others.
- For example, in the string "abracadabra," you might want to find where the substring "abra" appears. The answer would be at positions 1 and 8.
-

KNOWLEDGE GATE

- **String**: A string is a sequence of characters. The characters can be letters, numbers, or symbols. For example, "hello" is a string made of 5 characters.
- **Substring**: A substring is any continuous sequence of characters within a string. For instance, "hello" contains the substrings "he," "ell," "llo," "hello," etc. A substring can be as short as one character or as long as the entire string itself.
- **Proper Substring**: A proper substring is a substring that is strictly contained within a string and is not equal to the string itself. So, for "hello," "hell" or "ello" would be proper substrings, but "hello" would not be a proper substring of itself.

KNOWLEDGEGATE

Naive Algorithm

- The Naive algorithm is the simplest method where you slide the pattern over the text one character at a time and check for a match.
- Imagine you lost a small toy in a long hallway. The naive way to find it would be to walk along the hallway and check every single spot on the floor, one by one, until you find the toy.

- **Pros**:
  - Very simple to understand and implement.
  - Works well when the patterns are short and the text is not too large.
- **Cons**:
  - Can be very slow, especially if the pattern occurs frequently but with mismatches.

KNOWLEDGEGATE

String : a b c d e f g h

Pattern : d e f

String: a b c d a b c a b c d f — O(m)

Pattern: a b c d f — O(n)

O(m·n)

KNOWLEDGEGATE

```
NAIVE-STRING-MATCHER (T, P)
{
    n = T.length
    m = P.length
    for s = 0 to n- m
        if P[1..m] == T[s + 1 ..s + m]
            print "Pattern occurs with shift" s
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Rabin-Karp Algorithm

- In computer science, the **Rabin–Karp algorithm** or **Karp–Rabin algorithm** is a string-searching algorithm created by Richard M. Karp and Michael O. Rabin (1987) that uses hashing to find an exact match of a pattern string in a text.

KNOWLEDGEGATE

Rabin-Karp Algorithm

- The Rabin-Karp algorithm uses hashing to find any set of pattern occurrences. Instead of checking all characters of the pattern at every position (like the naive algorithm), it checks a hash value.
- Think of it like looking for a specific page in a book by its unique code instead of by reading every word. If the page number (hash) matches, then you check to make sure it's really the page you're looking for.
- **Pros**:
  - Faster than the naive approach on average.
  - Very efficient for multiple pattern searches at once.
- **Cons**:
  - Requires a good hash function to avoid frequent spurious hits.
  - The worst-case time complexity can be as bad as the Naive algorithm if many hash collisions occur.

KNOWLEDGEGATE

Text: a a a a a b

Pattern: a a b

1 + 1 + 2 = 4

↑ h(p)

↑ hash code

a-1
b-2
c-3
d-4
e-5
f-6
g-7
h-8
i-9
j-10

KNOWLEDGEGATE

Text: a b c d a b c e

n

Pattern: b c e

m

a-1
b-2
c-3
d-4
e-5
f-6
g-7
h-8
i-9
j-10

$O(n-m+1)$

KNOWLEDGEGATE

Text: c c a c c a a e d b a

Pattern: d b a

Spurious Hits O(mn)

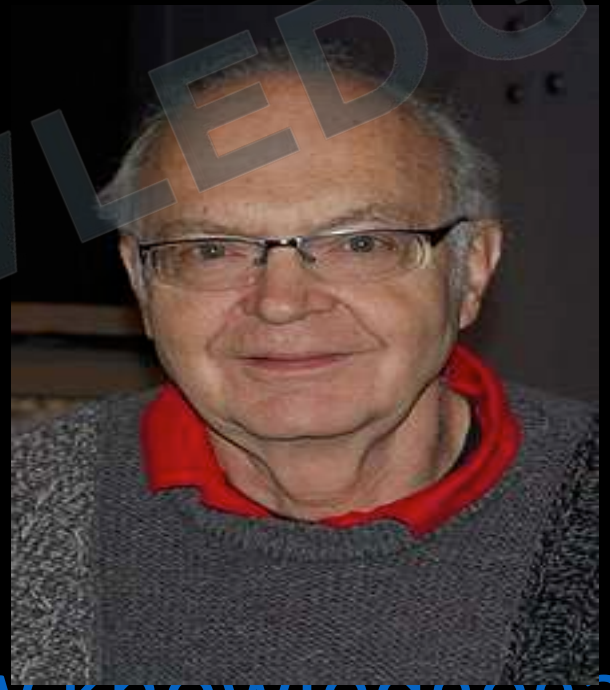
a-1
b-2
c-3
d-4
e-5
f-6
g-7
h-8
i-9
j-10

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Knuth-Morris-Pratt (KMP) Algorithm

- The algorithm was conceived by James H. Morris and independently discovered by Donald Knuth "a few weeks later" from automata theory.[1][2] Morris and Vaughan Pratt published a technical report in 1970.[3] The three also published the algorithm jointly in 1977.[1]



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Knuth-Morris-Pratt (KMP) Algorithm

- The KMP algorithm is smarter. It pre-processes the pattern to understand its structure and eliminates unnecessary comparisons when a mismatch occurs.
- **Pros**:
  - The algorithm ensures that the characters of the text are never compared more than once, which makes it very efficient.
  - No backtracking is needed, so it's more efficient than the naive approach.
- **Cons**:
  - The pre-processing step requires additional time and memory.
  - The algorithm is more complex to understand and implement.

KNOWLEDGEGATE

1 2 3 4

P_1: a b a b

1 2 3 4 5 6 7 8

P_2: a b c d a b c y

1 2 3 4 5 6 7 8 9 10

P_3: a b c d a b e a b f

1 2 3 4 5 6 7 8 9 10 11

P_4: a b c d e a b f a b c

KNOWLEDGEGATE

String: a b a b c a b c a b a b a b d

i 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

j 0 1 2 3 4 5

Pattern: a b a b d

KNOWLEDGEGATE

O(m+n) search

```
KMP-MATCHER (T, P)
{
    n = T.length
    m = P.length
    pie = COMPUTE-PREFIX-FUNCTION(P)
    pie = 0
    for i = 1 to n
        while q > 0 and P[q + 1] ‡ T[i]
            q = pie[q]
        if P[q + 1] == T[i]
            q = q + 1
        if q == m
            print "Pattern occurs with shift i-m
            q = pie[q]
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

```
COMPUTE-PREFIX-FUNCTION( P)
{
    M = P.LENGTH
    Let pie[1 . . .   m] be a new array
    pie[1] = 0
    k = 0
    for q = 2 to m
        whilek>o and p[k+1] ‡ p[q]
            k=pie[k]
        if p[k+1] == p[q]
            k=k+1
        pie[q] = k
    return pie
    }
```

KNOWLEDGEGATE

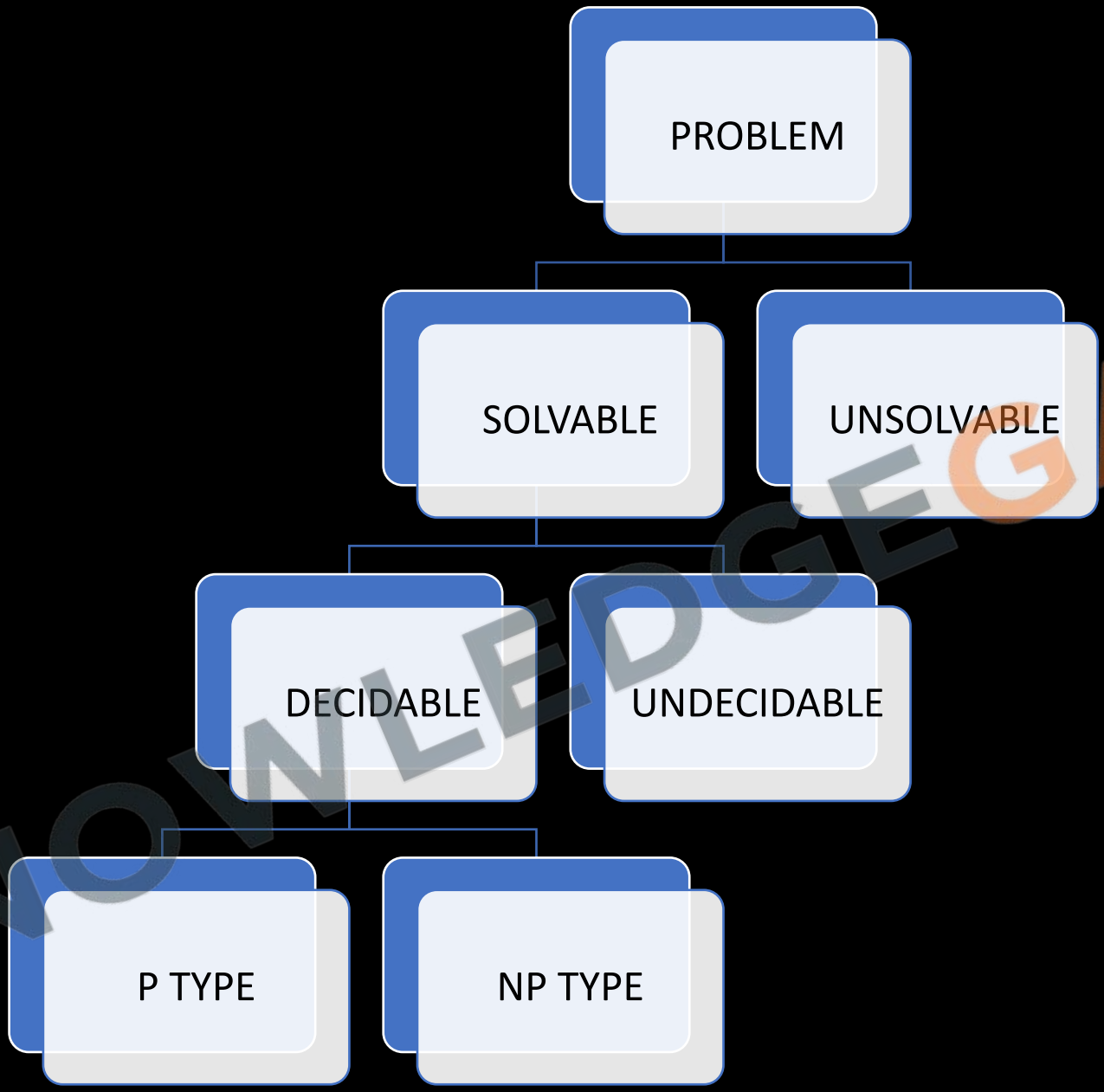
http://www.knowledgegate.in/GATE

What computer science deals with?

- Don’t
  - So we do not study how to design a computer
  - We do not study how to run a computer

- Do
  - We deal with problem solving, according to computer science a problem can be divided as follows

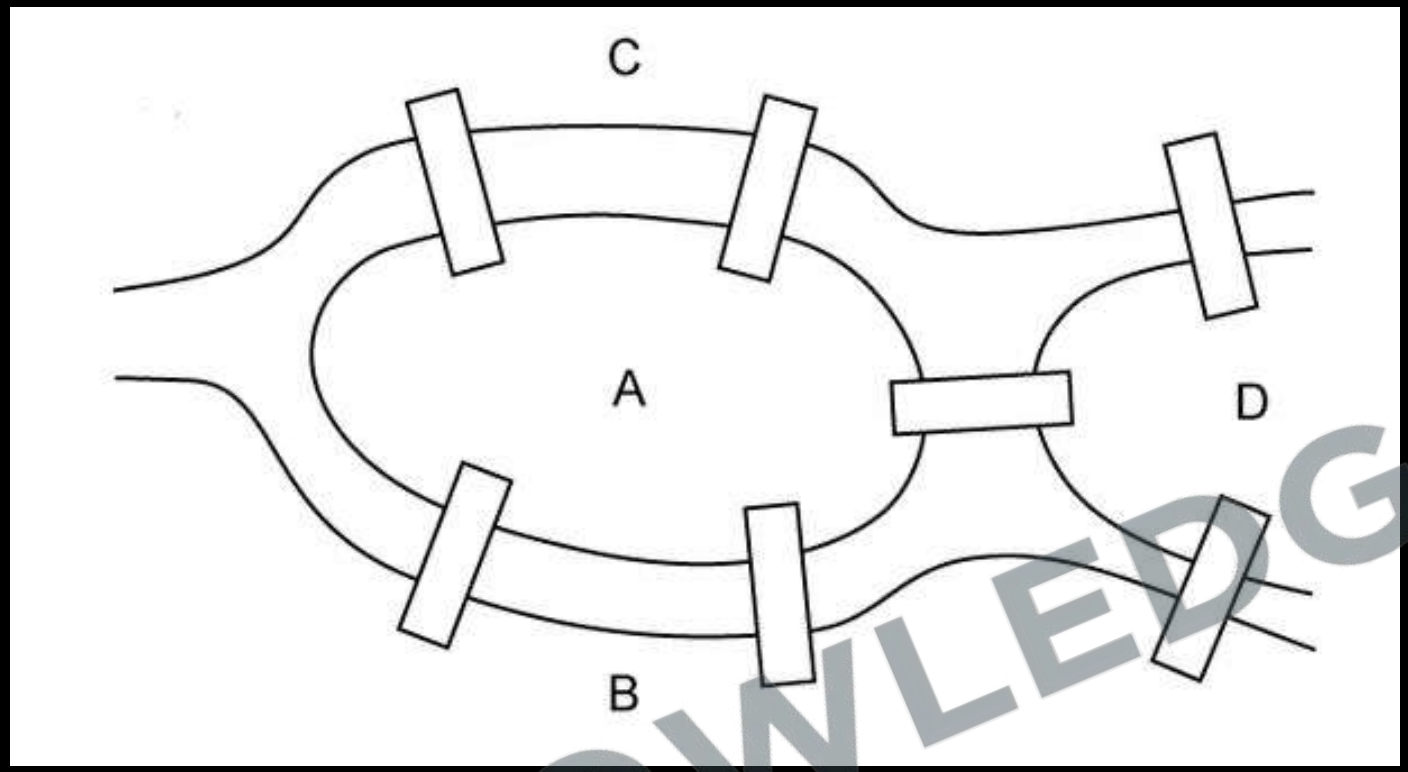
KNOWLEDGEGATE



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Konigsberg Bridge Problem



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **SOLVABLE** - A problem is said to be solvable if either we can solve it or if we can prove that the problem cannot be solved

- **UNSOLVABLE** - A problem is said to be unsolvable if neither we can solve it, nor we can proof that the problem can not be solved

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Decidable**- if there exist a polynomial time algorithm to solve a problem then problem is said to be decidable.

- **Undecidable**- if there exist a non- polynomial time algo to solve a problem.

KNOWLEDGEGATE

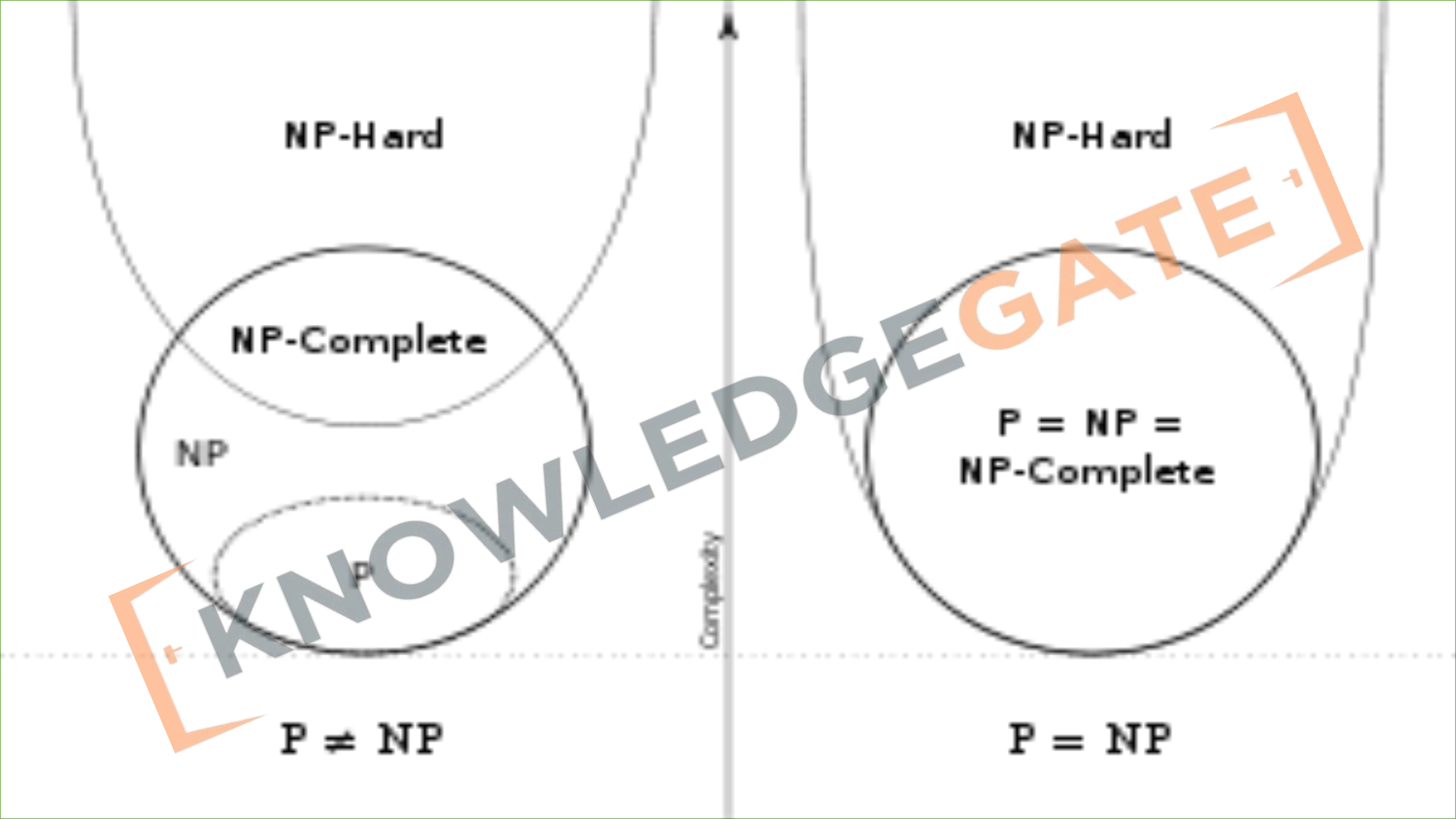
http://www.knowledgegate.in/GATE

- **P Type** - a problem is said to be P type is there exist a polynomial time algo to solve a problem

- **NP Type** – a problem is said to be NP type if there exist a np time algo to solve a problem on a deterministic machine or there exist a polynomial time algo to verify a problem

KNOWLEDGEGATE

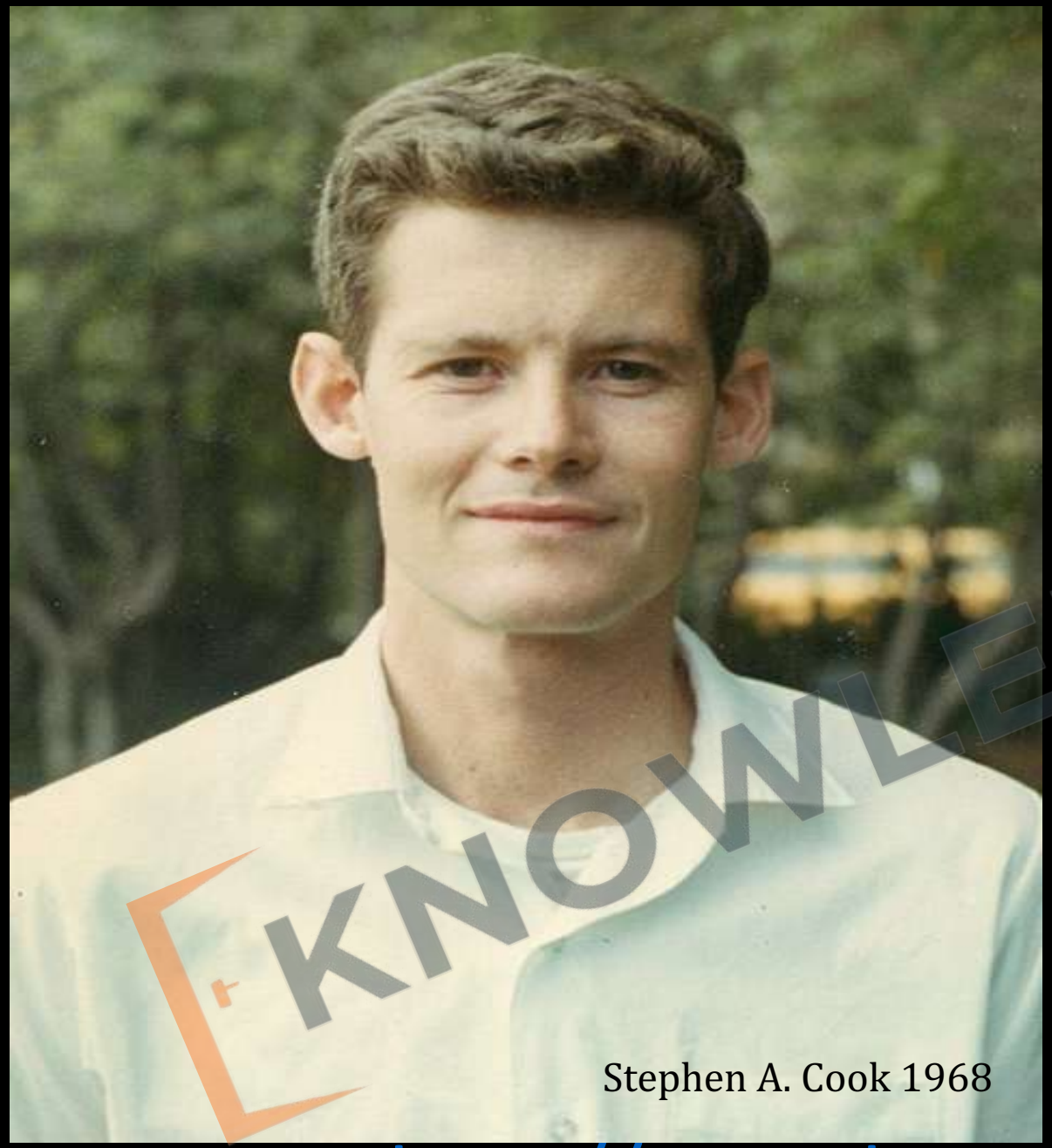
http://www.knowledgegate.in/GATE



KNOWLEDGEGATE

- In computational complexity theory, the **Cook–Levin theorem**, also known as **Cook's theorem**, states that the Boolean satisfiability problem is NP-complete. That is, it is in NP, and any problem in NP can be reduced in polynomial time by a deterministic Turing machine to the Boolean satisfiability problem.
- An important consequence of this theorem is that if there exists a deterministic polynomial time algorithm for solving Boolean satisfiability, then every NP problem can be solved by a deterministic polynomial time algorithm. The question of whether such an algorithm for Boolean satisfiability exists is thus equivalent to the P versus NP problem, which is widely considered the most important unsolved problem in theoretical computer science.

KNOWLEDGEGATE



Stephen A. Cook 1968

Stephen A. Cook 2008

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- During his PhD, Cook worked on complexity of functions, mainly on multiplication. In his seminal 1971 paper "The Complexity of Theorem Proving Procedures", Cook formalized the notions of polynomial-time reduction (a.k.a. Cook reduction) and NP-completeness, and proved the existence of an NP-complete problem by showing that the Boolean satisfiability problem (usually known as SAT) is NP-complete.

- This theorem question asks whether every optimization problem whose answers can be efficiently verified for correctness/optimality can be solved optimally with an efficient algorithm.

- Cook conjectures that there are optimization problems (with easily checkable solutions) which cannot be solved by efficient algorithms, i.e., P is not equal to NP. This conjecture has generated a great deal of research in computational complexity theory, which has considerably improved our understanding of the inherent difficulty of computational problems and what can be computed efficiently. Yet, the conjecture remains open.

- In 1982, Cook received the Turing award for his contributions to complexity theory.

[KNOWLEDGEGATE]

http://www.knowledgegate.in/GATE

- Graph coloring
- 3-Sat
- Cliuqe
- Hamiltonian circuit
- Knapsack problem
- Vertex cover
- Set cover
- Partition problem
- Independent set
- Travelling salesman problem
- Job scheduling

KNOWLEDGEGATE

Approximation Algorithm

- An approximation algorithm is a type of algorithm used for solving optimization problems. The key characteristics of an approximation algorithm are:
  - **Purpose**: It is used when finding the exact solution is too time-consuming, complex, or when an exact solution may not even be necessary.
  - **Performance**: It quickly finds a solution that is close to the best possible answer, or "optimal solution."
  - **Guarantee**: It provides a provable guarantee on how close the output is to the optimal solution, usually expressed as a factor of the optimal value.
  - **Complexity**: Often, the problems tackled by approximation algorithms are NP-hard, meaning that no known polynomial-time algorithms can solve these problems to optimality.
  - **Use Cases**: They are commonly used in fields such as operations research, computer science, and engineering for problems like scheduling, packing, and routing where exact solutions are less feasible as the problem size grows.
  - The concept of approximation is fundamental in scenarios where a perfectly precise answer is either impossible or impractical to obtain, and thus a solution that is "good enough" is acceptable given the constraints of time and resources.

KNOWLEDGEGATE

Feature	Approximation Algorithm	Deterministic Algorithm
Outcome	Produces a solution close to the optimal, with some error margin.	Produces a consistent and exact output for a given input.
Predictability	The solution quality is predictable, but the exact output may vary.	The output is entirely predictable and does not vary.
Optimization Problems	Often used for optimization problems where exact solutions are hard.	Used for problems where an exact solution is required and feasible.
Nature	Can be either deterministic or non-deterministic.	Always deterministic; follows a single path to a solution.
Resource Usage	Designed to find a good solution within reasonable time and resources.	Time and resources depend on algorithm complexity and input size.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- The approximation ratio of an algorithm for an optimization problem is a measure of how close the algorithm's solution is to the optimal solution.
- Showing that a particular algorithm for TSP is a 2-approximation means proving that the length of the tour produced by the algorithm is no more than twice the length of the optimal tour.
- Vertex cover problem is also 2-approximate.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Randomized algorithms

- Randomized algorithms are algorithms that make random choices during their process to simplify the solution of complex problems. They are particularly useful in situations where the input is large or the algorithm needs to be immune to certain worst-case scenarios that can cause deterministic algorithms to perform poorly.
  - **Simplification of Algorithms:** They can simplify complex algorithms by replacing a deterministic step with a random one.
  - **Performance:** On average, they often perform better than their deterministic counterparts. Their performance is typically analyzed in terms of expected running time or probability of correctness.

KNOWLEDGEGATE

- **Las Vegas algorithms:** They always produce the correct result, but the amount of time they take to complete may vary. For example, the Quickselect algorithm for finding the k-th smallest element in an array.
- **Monte Carlo algorithms:** They have a chance of producing an incorrect result but run in guaranteed polynomial time. An example is the Monte Carlo algorithm for primality testing.
- **Applications:** They are used in various fields, including cryptography, numerical analysis, data structures, optimization, and more.
- **Advantages:** They can avoid bad worst-case scenarios by randomizing choices, making it unlikely to encounter the worst case. They are often easier to implement and understand than their deterministic counterparts.

KNOWLEDGEGATE

Aspect	Las Vegas Algorithm	Monte Carlo Algorithm
Result Accuracy	Always accurate; results are either correct or the algorithm does not provide an answer.	Results are probabilistically accurate; there's a non-zero chance of error.
Execution Time	Variable; can take a long time to find a solution, but the solution is correct.	Typically faster; provides approximate solutions in a reasonable time frame.
Use Cases	Used in scenarios where correctness is crucial, like cryptographic algorithms.	Suitable for problems where approximate solutions are acceptable, like in simulations and optimizations.
Termination	Guaranteed to terminate with a correct solution or no solution at all.	Usually terminates in a fixed amount of time, but may not always provide a precise solution.
Examples	Algorithms for exact string matching, like the Rabin-Karp algorithm.	Algorithms for numerical integration, simulations in physics, and optimization problems.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Module

Free Semester One Shot Exam Course

Topic

Theory of Computation

Subtopic

TOC One Shot

Lesson

TOC_Notes

Video chapters

- **Chapter-1 (Basic Concepts and Automata Theory):** Introduction to Theory of Computation- Automata, Computability and Complexity, Alphabet, Symbol, String, Formal Languages, Deterministic Finite Automaton (DFA)- Definition, Representation, Acceptability of a String and Language, Non Deterministic Finite Automaton (NFA), Equivalence of DFA and NFA, NFA with ε-Transition, Equivalence of NFA's with and without ε-Transition, Finite Automata with output- Moore Machine, Mealy Machine, Equivalence of Moore and Mealy Machine, Minimization of Finite Automata.
- **Chapter-2 (Regular Expressions and Languages):** Regular Expressions, Transition Graph, Kleen's Theorem, Finite Automata and Regular Expression- Arden's theorem, Algebraic Method Using Arden's Theorem, Regular and Non-Regular Languages- Closure properties of Regular Languages, Pigeonhole Principle, Pumping Lemma, Application of Pumping Lemma, Decidability- Decision properties, Finite Automata and Regular Languages
- **Chapter-3 (Regular and Non-Regular Grammars):** Context Free Grammar(CFG)-Definition, Derivations, Languages, Derivation Trees and Ambiguity, Regular Grammars-Right Linear and Left Linear grammars, Conversion of FA into CFG and Regular grammar into FA, Simplification of CFG, Normal Forms- Chomsky Normal Form(CNF), Greibach Normal Form (GNF), Chomsky Hierarchy, Programming problems based on the properties of CFGs.
- **Chapter-4 (Push Down Automata and Properties of Context Free Languages):** Nondeterministic Pushdown Automata (NPDA)- Definition, Moves, A Language Accepted by NPDA, Deterministic Pushdown Automata(DPDA) and Deterministic Context free Languages(DCFL), Pushdown Automata for Context Free Languages, Context Free grammars for Pushdown Automata, Two stack Pushdown Automata, Pumping Lemma for CFL, Closure properties of CFL, Decision Problems of CFL, Programming problems based on the properties of CFLs.
- **Chapter-5 (Turing Machines and Recursive Function Theory):** Basic Turing Machine Model, Representation of Turing Machines, Language Acceptability of Turing Machines, Techniques for Turing Machine Construction, Modifications of Turing Machine, Turing Machine as Computer of Integer Functions, Universal Turing machine, Linear Bounded Automata, Church's Thesis, Recursive and Recursively Enumerable language, Halting Problem, Post's Correspondance Problem, Introduction to Recursive Function Theory.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **<u>Chapter-1 (Basic Concepts and Automata Theory)</u>:** Introduction to Theory of Computation- Automata, Computability and Complexity, Alphabet, Symbol, String, Formal Languages, Deterministic Finite Automaton (DFA)- Definition, Representation, Acceptability of a String and Language, Non Deterministic Finite Automaton (NFA), Equivalence of DFA and NFA, NFA with ε-Transition, Equivalence of NFA's with and without ε-Transition, Finite Automata with output- Moore Machine, Mealy Machine, Equivalence of Moore and Mealy Machine, Minimization of Finite Automata.

[KNOWLEDGEGATE]

INTRODUCTION TO THEORY OF COMPUTATIONS

- The theory of computation is a branch of theoretical computer science that deals with the study of algorithms and computational complexity. It aims to answer fundamental questions about
  - what can be computed
  - how efficiently it can be done
  - and what limitations exist in terms of computational power.

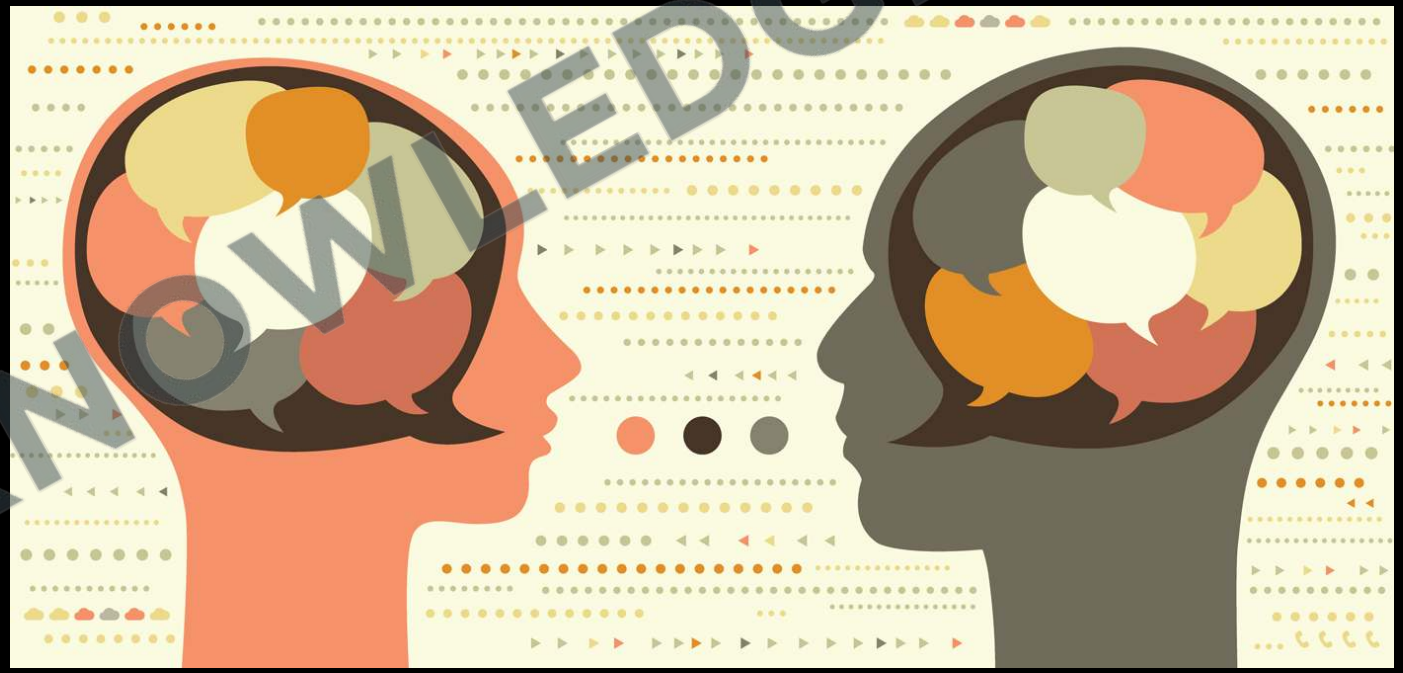
KNOWLEDGE GATE

- As word suggests ‘TOC’ is the study of 'mathematical' machines or systems called automata.
- Theory of computation can be considered as the study of all kinds of computational model in the field of computer science and it also considers how efficiently the problem can be solved (but not is depth).

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

PROBLEM

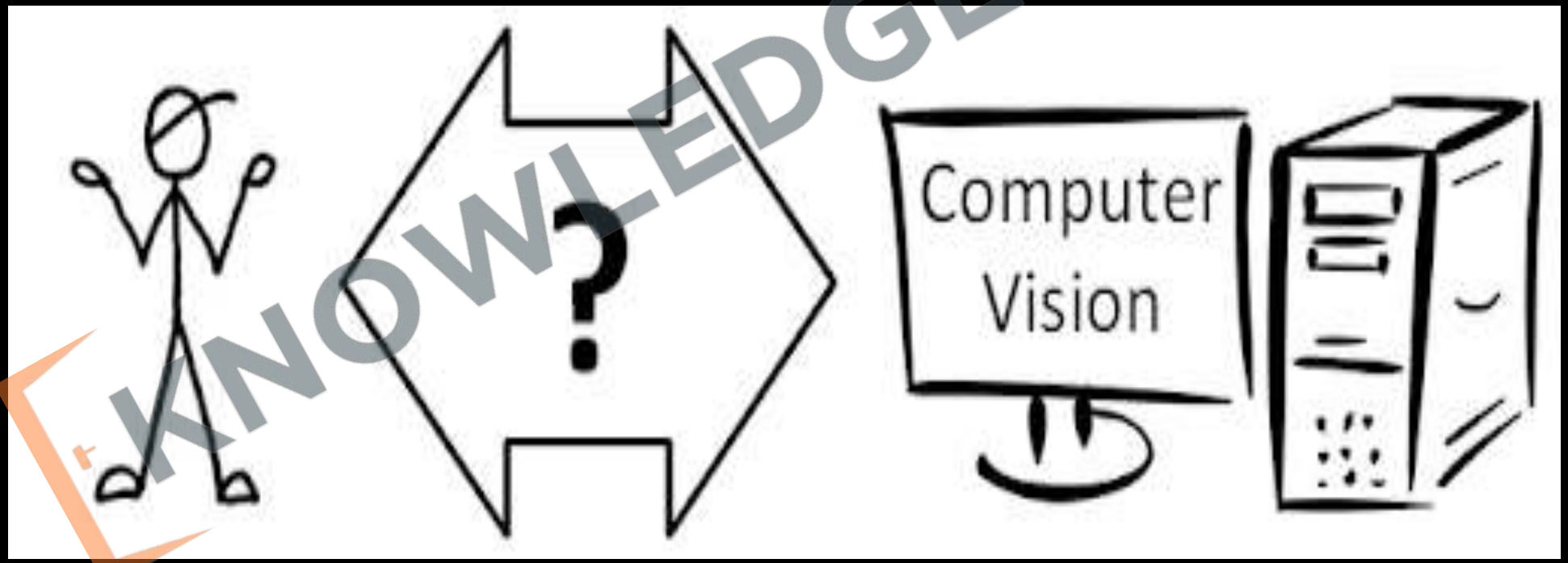
- Now a day's machines (digital, analog, mechanical) play a very important role in the development of human, we need some mechanism (language) to communicate with the machines.



KNOWLEDGEGATE

SOLUTION

- We need a language for communication with machines. But we do not require natural languages to communicate with the machines, as natural languages are very complex and machine interaction require very fewer complex languages compare to natural languages.



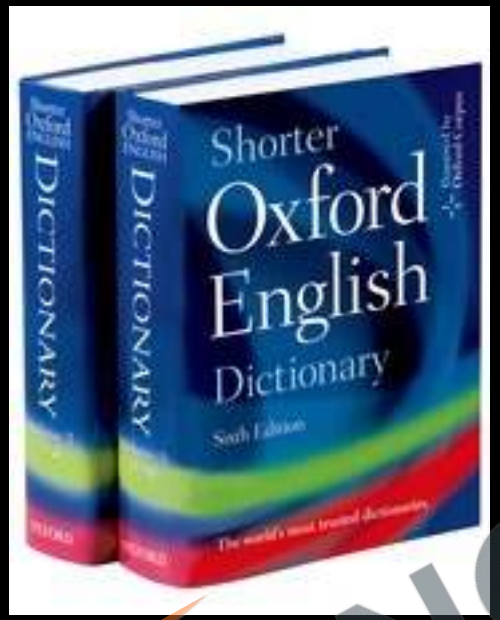
KNOWLEDGEGATE

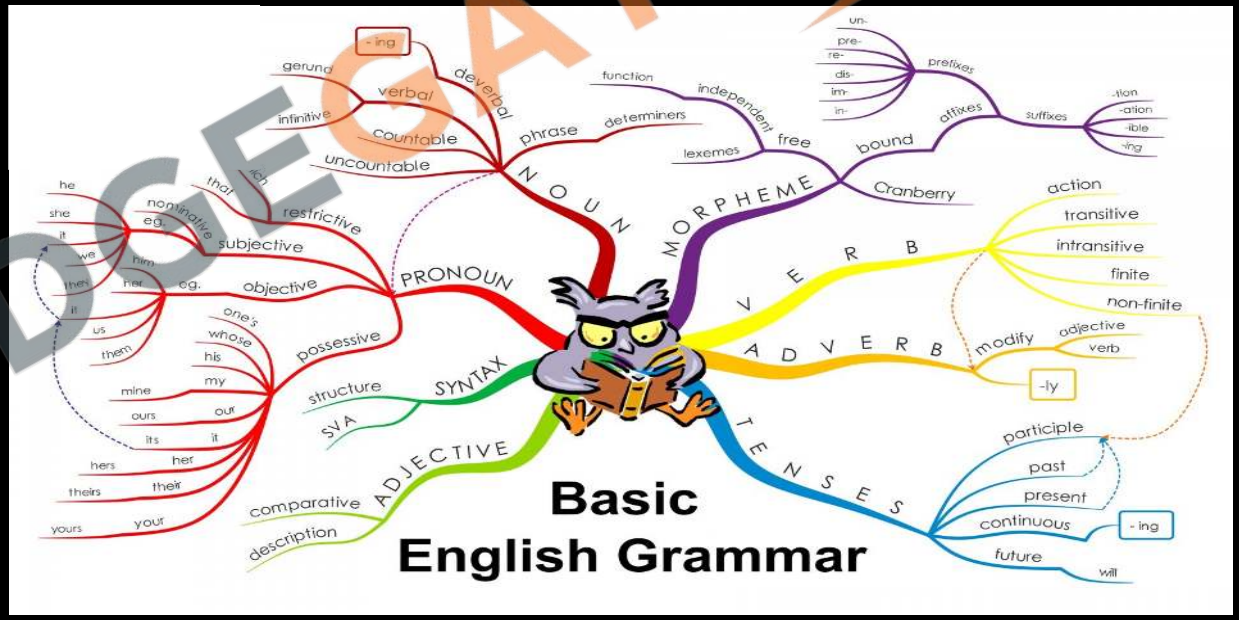
- Languages can be of two types formal languages and informal languages, here in this subject we will only discuss formal languages.
- Dictionary defines the term informally as ‘a system suitable for the expression of certain ideas, facts or concepts including a set of symbols for their manipulation’.

KNOWLEDGEGATE

METHODS TO DEFINE LANGUAGE

- In natural language we define the list of words in a dictionary because they are finite and predefined, but we cannot list all the sentence which can be formed using these words as they are infinite.





- So, we have a mechanism called grammar/rules using which we can decide which sentence is valid and which is invalid.

KNOWLEDGE GATE

MATHEMATICL DEFINATION OF LANGUAGE

- **SYMBOL**- Symbols are the basic building blocks, which can be any character/token. (cow, sheep, , white flag, , , etc.) (in English we called them as letters).

KNOWLEDGE GATE

- **ALPHABET**- An alphabet is a finite non empty set of symbols, (every language has its own alphabet). here in toc, we use symbol Σ for depicting alphabet. e.g. Σ = {0,1}. for English Σ = {a, b, c, …., z} (in English also alphabet is a set of letters, thought in general we called them as alphabet).

KNOWLEDGEGATE

- **STRING** - It is a finite sequence of symbols (which are the member of set alphabet). E.g. Σ = {a, b} String- aabb, aa, b, so on. (in English we called them as words).

KNOWLEDGEGATE

- **LANGUAGE** - A language is defined as a set of strings. (in natural language (set of words(predefined) and grammar) we apply this model from words to sentence).
- In the next level we consider programs as a string and programming constructs/tokens like int, floats as letters/symbols.

KNOWLEDGE GATE

- Similarly, in our system we have finite number of symbols/letters but using those letters we can generate infinite strings/words.
- So, we may have languages that have infinite number of words, so it is not possible for us to list them, we have to use some framework, which can somehow represent the same language. There are mainly two methods to represent a language
  - by a grammar that generates a language [RG generate RL]
  - by a machine that accepts a language [FA accept RL language]

KNOWLEDGEGATE

Machine Grammar

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- The Theory of Formal Languages is a branch of theoretical computer science that deals with the study of formal languages and their properties.
- The theory of formal languages includes the study of formal grammars, which are used to define the syntax of formal languages, and automata theory, which deals with the study of abstract machines used to recognize, generate, or process formal languages. Some of the key concepts studied in the theory of formal languages include regular languages, context-free languages, context-sensitive languages, recursively enumerable languages, and the Chomsky hierarchy.
- The theory of formal languages has important applications in areas such as compilers, parsers, and other software engineering tools, as well as in the design of programming languages and the study of natural language processing.

[KNOWLEDGEGATE]

Q if $\sum = \{a, b\}$ then, find the following?

$\sum^0 =$

$\sum^1 =$

$\sum^2 =$

$\sum^3 =$

KNOWLEDGEGATE

- $\sum^K$ is the set of all the strings from the alphabet $\sum$ of length exactly K.

- $\sum^k = \{W \mid |W| = K\}$ (using the symbols from the alphabet $\sum$)

KNOWLEDGEGATE

Kleene closure- If ∑ is a set of symbols, then we use ∑* to denote the set of strings obtained by concatenating zero or more symbols from ∑ of any length, in general any string of any length which can have only symbols specified in ∑.

- $\sum^* = \bigcup_{i=0}^{i=\infty}\{w \mid |w| = i\}$ (using the symbols from the alphabet ∑)

KNOWLEDGEGATE

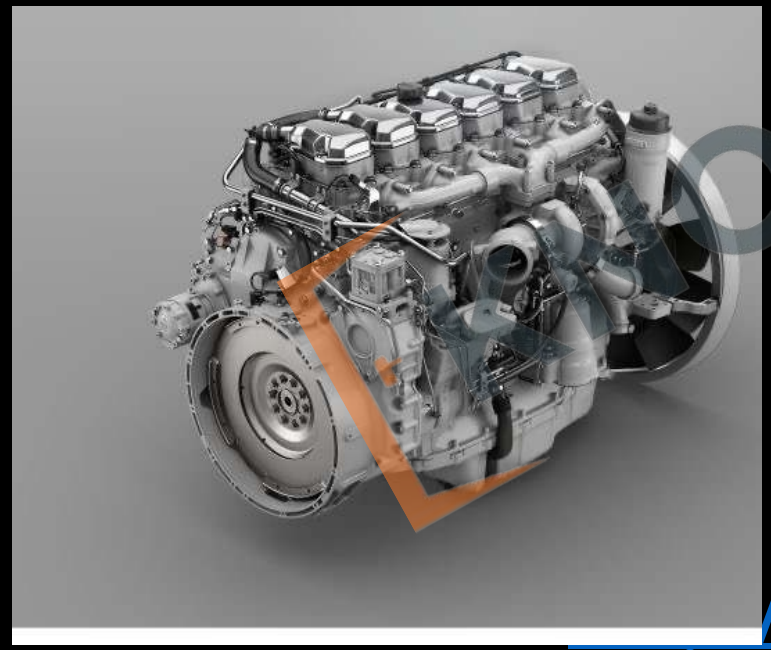
Positive closure – If ∑ is a set of symbols, then we use $\sum^+$ to denote the set of strings obtained by concatenating one or more symbols from ∑ of any length, in general any string of any length which can have only symbols specified in ∑ (except ∈).

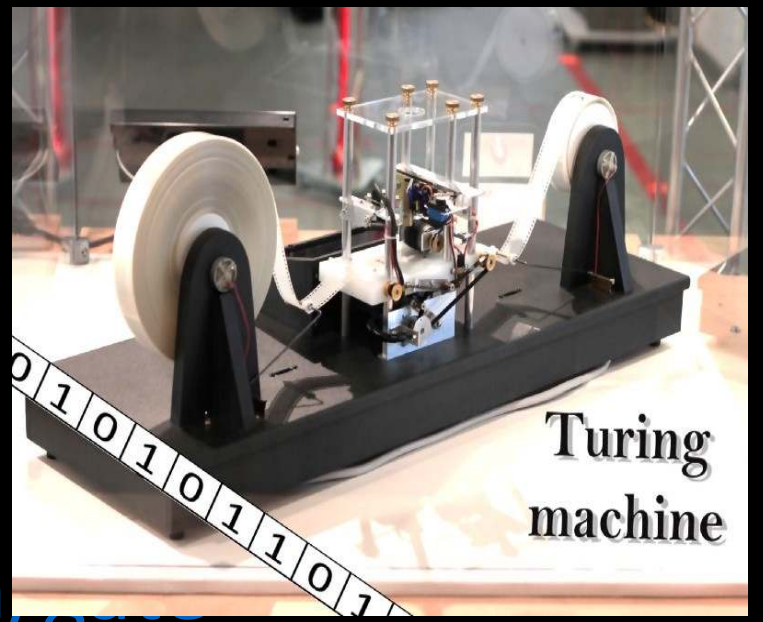
- $\sum^+ = \bigcup_{i=1}^{i=\infty}\{w \mid |w| = i\}$ (using the symbols from the alphabet ∑)

KNOWLEDGEGATE

Automaton

- An automaton is defined as a self-operating system where energy, materials and information are transformed, transmitted and used for performing some functions without direct participation of man.
- The term is often used to describe a theoretical machine that operates according to a set of rules and is capable of carrying out complex operations without human intervention.
- Example: automatic machine tools, automatic packing machines, and automatic photo printing machines.



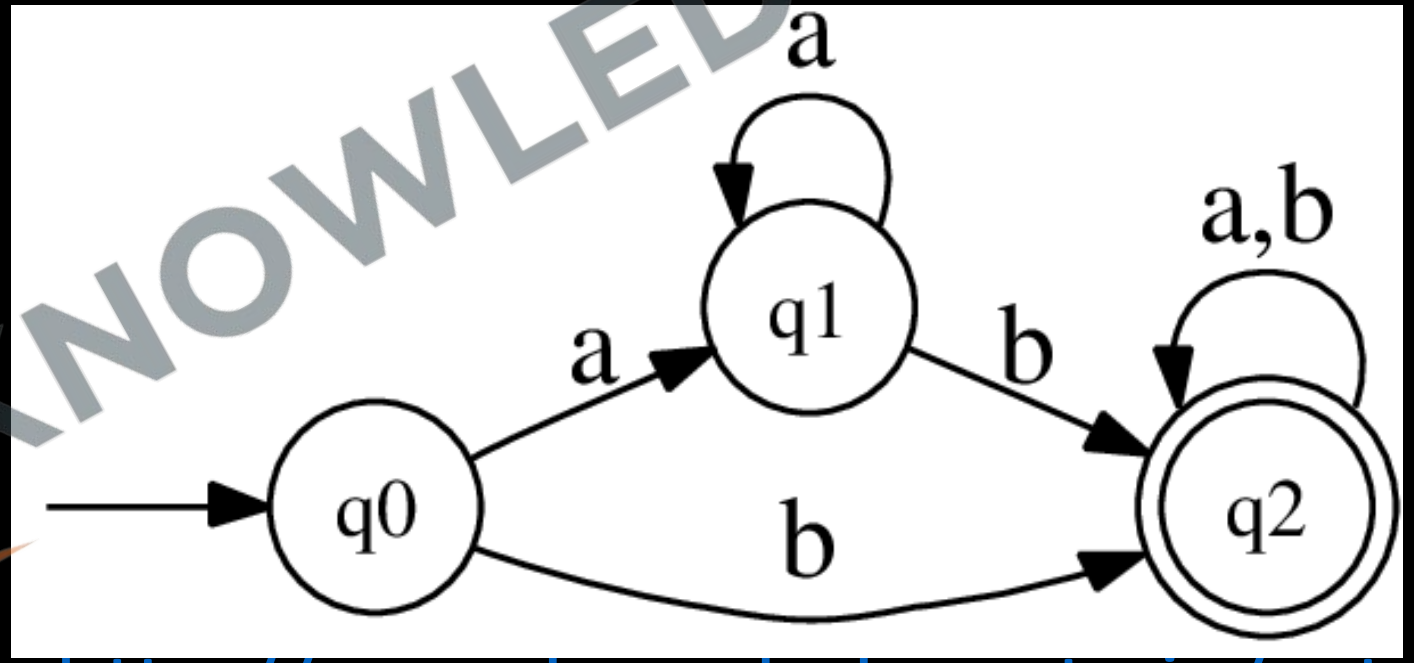


KNOWLEDGEGATE

FINITE AUTOMATA

- A **Finite automaton** is a model that has a finite set of states (represented in the figure by circles) and its control moves from one state to another state in response to external inputs (represented by arrows).

- It is an abstract machine that is used to recognize patterns in strings of symbols. Finite automata are widely used in computer science for pattern matching, lexical analysis, and parsing, among other applications.



KNOWLEDGEGATE

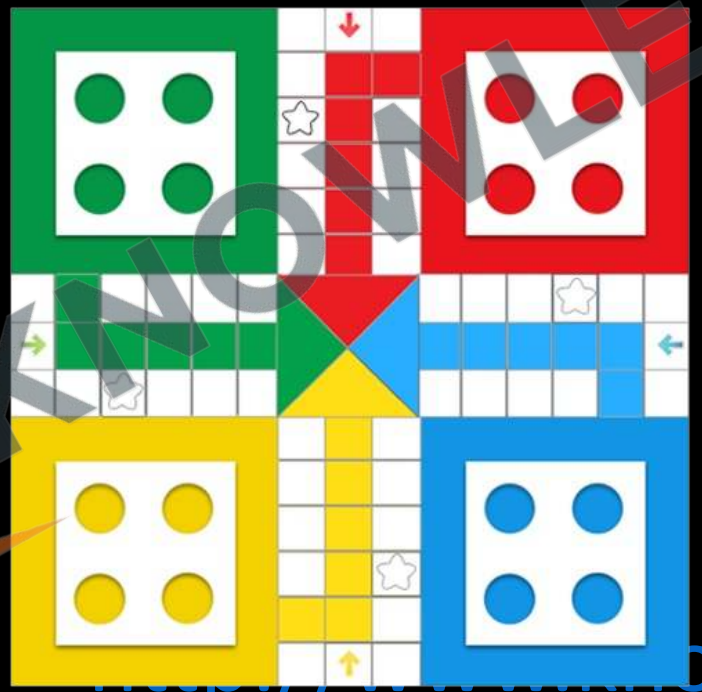
- Finite automata can be broadly classified into two types-

1. **Finite automata without output**
    1. Deterministic finite automata.
    2. Non deterministic finite automata.
    3. Non deterministic finite automata with $\in$

2. **Finite automata with output**
    1. Moore machine
    2. Mealy machine

KNOWLEDGEGATE

- In general, this type of automata is characterized by machine having no temporary storage, as it is severely limited in its capacity to remember things during the computation.
- Only finite amount of information can be in the control unit by placing the unit into a specific state but since the number of states is finite, a finite automaton can only deal with situation in which the information to be stored at any time is strictly bounded.



KNOWLEDGE GATE

DETERMINISTIC FINITE AUTOMATA

A **deterministic finite automaton (DFA)** is defined by 5-tuple (Q,Σ,δ,S,F) where:

- Q is a finite and non-empty set of states
- Σ is a finite non-empty set of finite input alphabet
- δ is a transition function, ($\delta: Q \times \Sigma \rightarrow Q$)
- S is **initial state** (always one) ($S \in Q$)
- F is a set of final states ($F \subseteq Q$) ($0<=|F|<=N$, where n is the number of states)

[KNOWLEDGEGATE]

DFA Designing

- A language a said to be regular language if it can be accepted by a DFA.
- The best knowledge about DFA can be only understood by designing a number of DFA, by doing so we will first understand the process of DFA designing and secondly, we will understand Regular Language.

KNOWLEDGE GATE

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b}, where every accepted string ‘w’ starts with substring s = ‘abb’

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b}. where every accepted string ‘w’ ends with substring ‘s = bab’ ?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b}. where every accepted string ‘w’ contains sub string s= ‘aba’?

KNOWLEDGEGATE

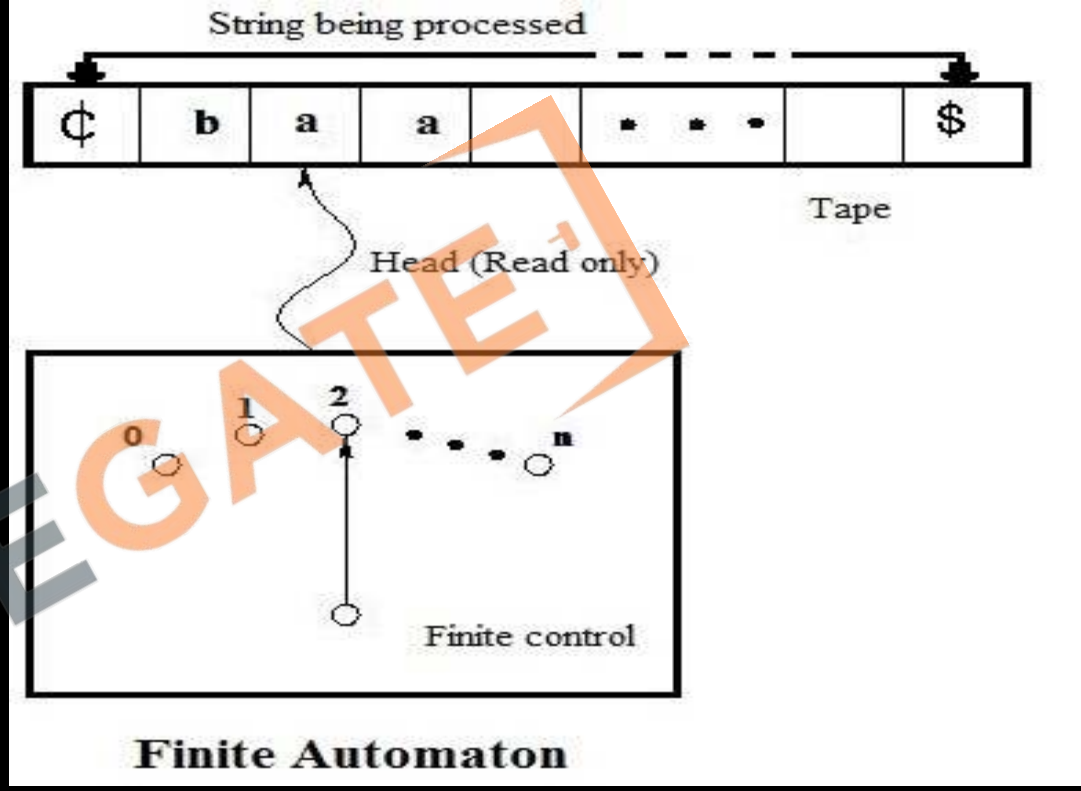
http://www.knowledgegate.in/gate

- The various components of the block diagram are explained as follows:

- **Input tape**: The input tape is divided into squares, each square containing a single symbol from the input alphabet ∑. The end squares of the tape contain the end marker ¢ at the left end and the end marker $ at the right end. The absence of end markers indicates that the tape is of infinite length. The left-to-right sequence of symbols between the two end markers is the input string to be processed.

- **Reading head (R-head)**: The head examines only one square at a time and will move one square to the right.

- **Finite control:** is the inference engine take care of transition



Finite Automaton

KNOWLEDGEGATE

NOTE-

- Produces a unique computation (or run) of the automaton for each input string.
- Deterministic refers to the uniqueness of the computation.
- DFAs are useful for doing lexical analysis (Spell check) in compiler design.

KNOWLEDGEGATE

Representation

TRANSITION STATE DIAGRAM- (Graphical, easy to understand, easy to design) A transition graph or a transition system is a finite directed labelled graph in which each circle represents a state and the directed edges indicate the transition of one state to another state. The initial state is represented by a circle with an arrow pointing towards it, the final state by two concentric circles.

δ	Σ	
	a	b
Q	q_0	
	q_1	

TRANSITION TABLE- it is a two-dimensional table where number of columns is equal to number of input alphabets and number of rows is equal to number of states.

TRANSITION ID- $\delta \{q_i, a) = q_j$, this means that on a state q_i take an input symbol a machine will make a transition q_j

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

ACCEPTANCE BY DFA

- Let ‘w’ be any string designed from the alphabet Σ, corresponding to w, if there exist a transition for which it starts at the initial state and ends in any One of the final states, then the string ‘w’ is said to be accepted by the finite automata. $\delta^*(q_0, w) = q_f$ for some $q_f \in F$.

- Mathematically, it can be represented as: - $L(M) = \{w \in \Sigma^* \mid \delta^*(S, w) \in F\}$

KNOWLEDGEGATE

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b} such that every accepted string start and end with ‘a’?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b} such that every accepted string start and end with same symbol?

KNOWLEDGEGATE

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b} such that every accepted string start and end with different symbol?

KNOWLEDGEGATE

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b} such that every accepted string w, is like w=SX. S=aaa/bbb?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b} such that every accepted string w, is like w=XS. S=aaa/bbb?

KNOWLEDGEGATE

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b} such that every accepted string w, is like w=XSX. S=aaa/bbb?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b}, such that every string ‘w’ accepted must be like

i) |w| = 3

ii) |w|<=3

iii) |w|>=3

KNOWLEDGEGATE

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b}, such that every string accepted must contain exactly two a's, $|w|_a=2$?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b}, such that every string accepted must contain at least two a's, $|w|_a >= 2$?

KNOWLEDGEGATE

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b}, such that every string accepted must contain at most two a's, $|w|_a <= 2$?

KNOWLEDGEGATE

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b}, such that every string ‘w’ accepted must be like

i) |w| = 0(mod 3)

ii) |w|=1(mod 4)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b}, such that every string accepted must contain

i) number of a is, $|w|_a = 0 \pmod 2$

iii) number of a is $|w|_b = 2 \pmod 3$

KNOWLEDGEGATE

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b} such that for every accepted string 2^{th} from left end is always a ?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b} such that for every accepted string 2nd from right end is always b ?

KNOWLEDGEGATE

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b}, such that every string accepted must contain be like,
no of a = 0(mod 2) || no of b = 0(mod 2) ?

KNOWLEDGEGATE

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {0,1}, such that every string ‘w’ which is accepted has a decimal equivalent 0(mod 3) ?

KNOWLEDGEGATE

COMPLIMENT OF DFA

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b}, such that every string accepted must not contain a substring 'aaa'?

KNOWLEDGEGATE

NON DETEMINISTIC FINITE AUTOMATA

- Non determinism means choice of move for an automaton. So rather than prescribing a unique move in each situation, we allow a set of possible moves.
- Non deterministic machine are only theoretical machine i.e. in the first place they are not implementable and neither we want to implement them, the only reason we study non-determinism is because they are easy to design and easily be converted into deterministic machine.

KNOWLEDGEGATE

FORMAL DESCRIPTION OF NDFA

- A Non-Deterministic finite automaton (NDFA) is a 5-tuple (Q,Σ,δ,S,F) where:
  - Q is a finite and non-empty set of states
  - Σ is a finite non-empty set of finite input alphabet
  - δ is a transition function $\delta: Q \times \Sigma \rightarrow 2^Q$
  - q_0 is **initial state** (always one) ($q_0 \in Q$)
  - F is a set of final states ($F \subseteq Q$) ($0<=|F|<=N$), where n is the number of states

KNOWLEDGEGATE

Some points to remember

- Every DFA is also an NFA.
- Every NFA can be translated to an equivalent DFA, so their language accepting capability is same.
- NFAs like DFA's only recognize ***regular languages***.
- It need not to be a complete system. There can be a state that doesn't have any transition on some input symbol.
- It is possible that a single state led to multiple transition on same input to different states.
- NOTE- A null transition is also possible for NFA, such special NFA are called Null-NFA. We will discuss it later.

KNOWLEDGEGATE

PROPERTIES OF NFA

- i) Accepting power of NDFA= Accepting power of DFA.
- ii) NDFA is a theoretical engine and is not implementable, but it is very easy to design compare to DFA.
- iii) No concept of dead state, therefore complementation of DFA is also not possible.
- iv) NDFA will respond for only valid strings and no need to respond for invalid strings. (it is a Incomplete system)

KNOWLEDGEGATE

ACCEPTANCE BY NDFA

- Let ‘w’ be any string defined over the alphabet Σ, corresponding to w, there can be multiple transitions for NFA starting from initial state, if there exist at least one transition for which we start at the initial state and ends in any One of the final state, then the string ‘w’ is said to be accepted by the non-deterministic finite automata, otherwise not.

- Mathematically, it can be represented as, $L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}$

KNOWLEDGEGATE

Q Design a NDFA that accepts all strings over the alphabet ∑ = {a, b}, where every accepted string ‘w’ starts with substring s, Where s = ‘aba’ ?

KNOWLEDGEGATE

Q Design a NDFA that accepts all strings over the alphabet ∑ = {a, b}. where every accepted string ‘w’ ends with substring ‘s’, Where s = ‘bab’?

KNOWLEDGEGATE

Q Design a minimal DFA that accepts all strings over the alphabet ∑ = {a, b}. where every accepted string ‘w’ contains sub string s, Where s = ’aba’ ?

[KNOWLEDGEGATE]

http://www.knowledgegate.in/gate

Q Design a NDFA that accepts all strings over the alphabet ∑ = {a, b} such that every accepted string start and end with same symbol ?

KNOWLEDGEGATE

Q Design a NDFA that accepts all strings over the alphabet ∑ = {a, b} such that every accepted string start and end with different symbol ?

KNOWLEDGEGATE

Q Design a minimal NDFA that accepts all strings over the alphabet ∑ = {a, b} such that every accepted string w, is like w=SX , Where s = aaa/bbb

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a minimal NDFA that accepts all strings over the alphabet ∑ = {a, b} such that every accepted string w, is like w=XS , Where s = aaa/bbb

KNOWLEDGEGATE

Q Design a minimal NDFA that accepts all strings over the alphabet ∑ = {a, b} such that every accepted string w, is like w=XSX , Where s = aaa/bbb ?

[KNOWLEDGEGATE]

http://www.knowledgegate.in/gate

Q Design a NDFA that accepts all strings over the alphabet ∑ = {a, b} such that for every accepted string 3rd from right end is always a ?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

NFA and DFA Equivalence

- In this topic we will be learning about the equivalence of NFA and DFA and how an NFA can be converted to equivalent DFA. Let us take an example and understand the conversion.
- Since every NFA and DFA has equal power that means, for every language if a NFA is possible, then DFA is also possible.
- So, every NFA can be converted to DFA.
- The process of conversion of an NFA into a DFA is called Subset Construction.

KNOWLEDGEGATE

- If NFA have ‘n’ states which is converted into DFA which ‘m’ states than the relationship between n and m will be

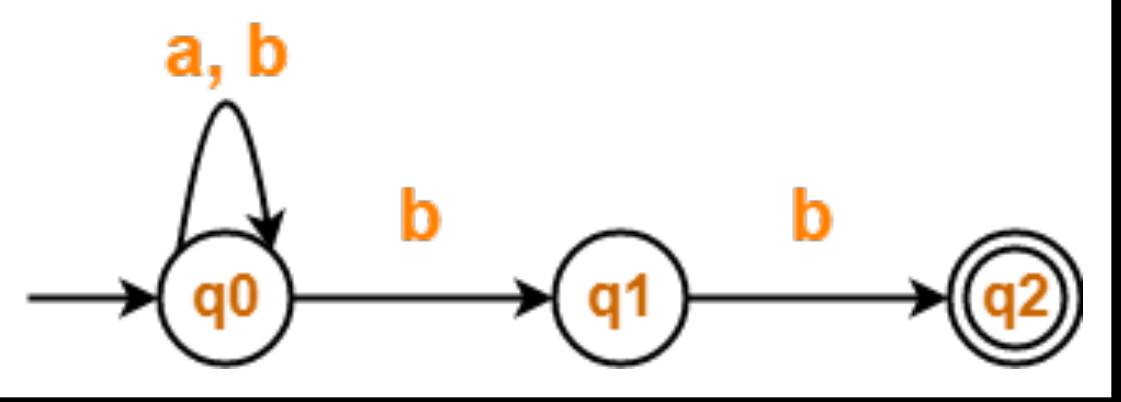
- $1 <= m <= 2^n$

KNOWLEDGEGATE

Procedure for Conversion

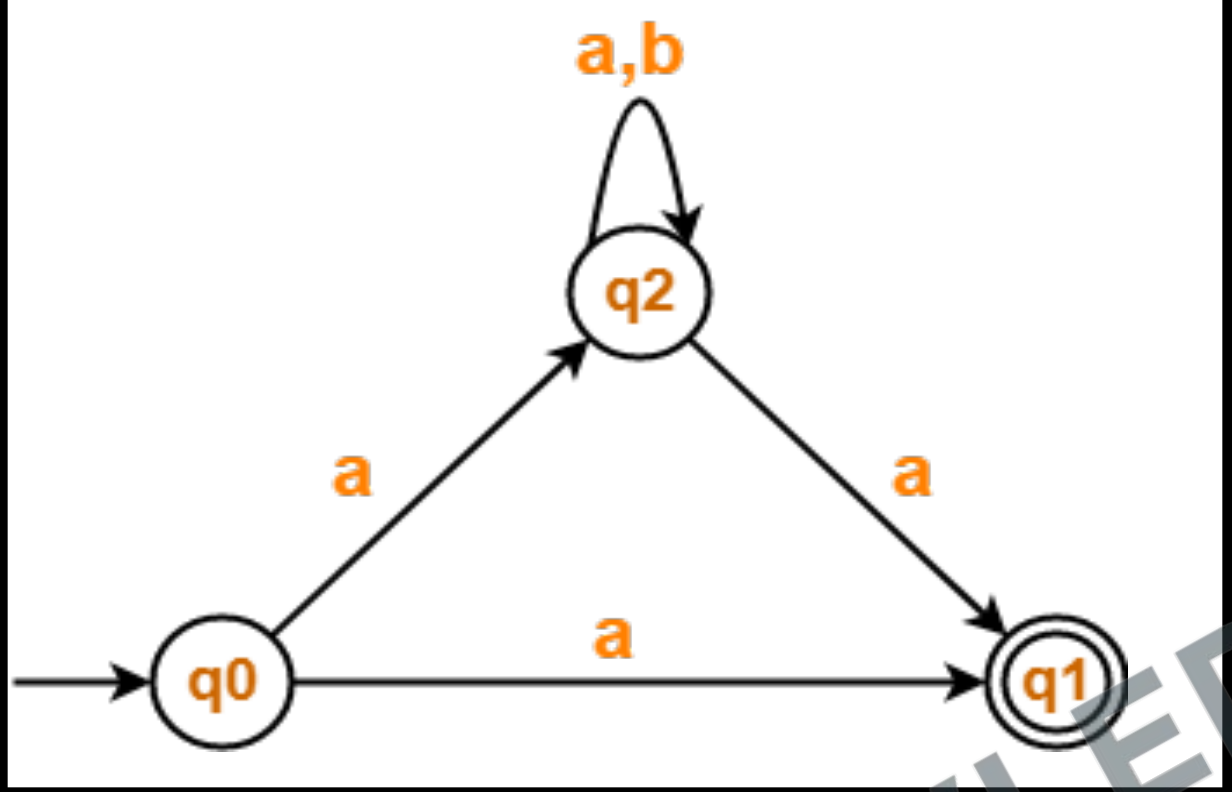
- There lies a fixed algorithm for the NFA and DFA conversion. Following things must be considered
  - Initial state will always remain same.
  - Start the construction of δ' with the initial state & continue for every new state that comes under the input column and terminate the process whenever no new state appears under the input column.
  - Every subset of states that contain the final state of the NFA is a final state in the resulting DFA.
  - $\delta'(q_0, q_1, q_2, q_{3\text{--------}}, q_{n-1}, a) = \bigcup_{i=0}^{i=n-1} \delta(q_i, a)$

KNOWLEDGEGATE



KNOWLEDGEGATE

http://www.knowledgegate.in/gate



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Aspect	DFA (Deterministic Finite Automata)	NDFA (Nondeterministic Finite Automata)
State Transition	On each input symbol, transitions to exactly one state.	Can transition to multiple states or none on the same input symbol.
Determinism and Uniqueness	Each state has a unique transition for each input symbol.	A state can have multiple transitions for the same input symbol.
Computation Path	Always has a single, unique computation path for any input string.	May have multiple computation paths for the same input string.
Ease of Construction	Generally simpler and more straightforward to construct.	Can be more complex to construct due to non-determinism.
Acceptance of Input	Accepts an input if it reaches a final state after processing all input symbols.	Accepts an input if at least one computation path reaches a final state.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

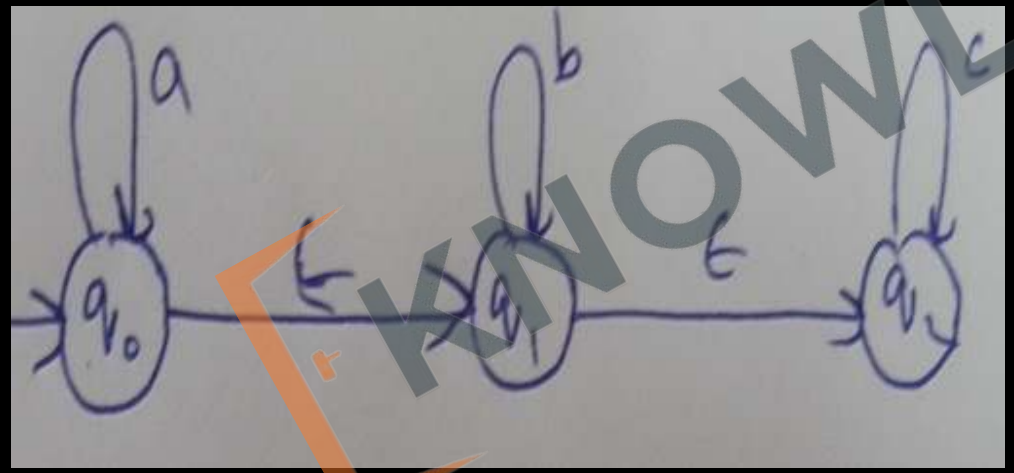
NFA WITH EPSILON MOVES (€-NFA)

- An automaton that consist of null transitions is called a Null- NFA i.e. we allow a transition on null means empty string.
- €-NFA is a 5-tuple (Q,Σ,δ,S,F) where:
  - Q is a finite and non-empty set of states
  - Σ is a finite non-empty set of finite input alphabet
  - δ is a transition function $\delta: (Q \times \{\Sigma \cup \varepsilon\}) \rightarrow 2^Q$
  - S is **initial state** (always one) $(S \in Q)$
  - F is a set of final states $(F \subseteq Q)$ $(0<=|F|<=N$, where n is the number of states

[KNOWLEDGEGATE]

NULL-CLOSURE

- Null closure of a set Q is defined as a set of all the states, which are at zero distance from the state Q. A set of all the states, that can be reached from the state and along a null- transition.
- **€-Closure(q_i)-** The set of all the states which are at zero distance from the state q_i is called €-closure(q_i). Or the set of all the states that can be reached from the state q_i along € labelled transition path, is known as €-closure(q_i).



KNOWLEDGEGATE

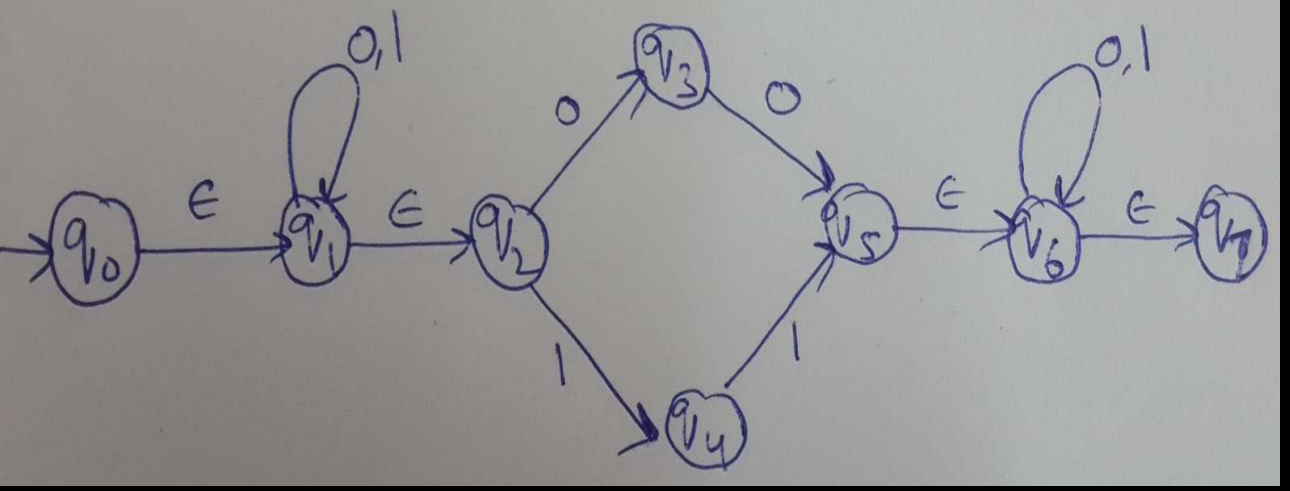
- Every state is at zero distance to itself.
- In NFA or DFA, distance between two states is always 1, because there could be no null transitions.
- The Null closure Q is always a non-empty and finite state, because every state's null closure is that state only.
- €-closure(Φ) = Φ
- €-closure (q_0 , q_1, q_2 , q_3--------, q_n) =$U_{i=0}^{i=n}$ δ(€-closure(q_i))

KNOWLEDGEGATE

EQUIVALENCE BETWEEN NULL NFA TO NFA

- There will be no change in the initial state.
- No change in the total no. of states
- May be change in the number of final states.
- All the states will get the status of the final state in the resulting NFA, whose €-closure contains at least one final state in the initial €-NFA.

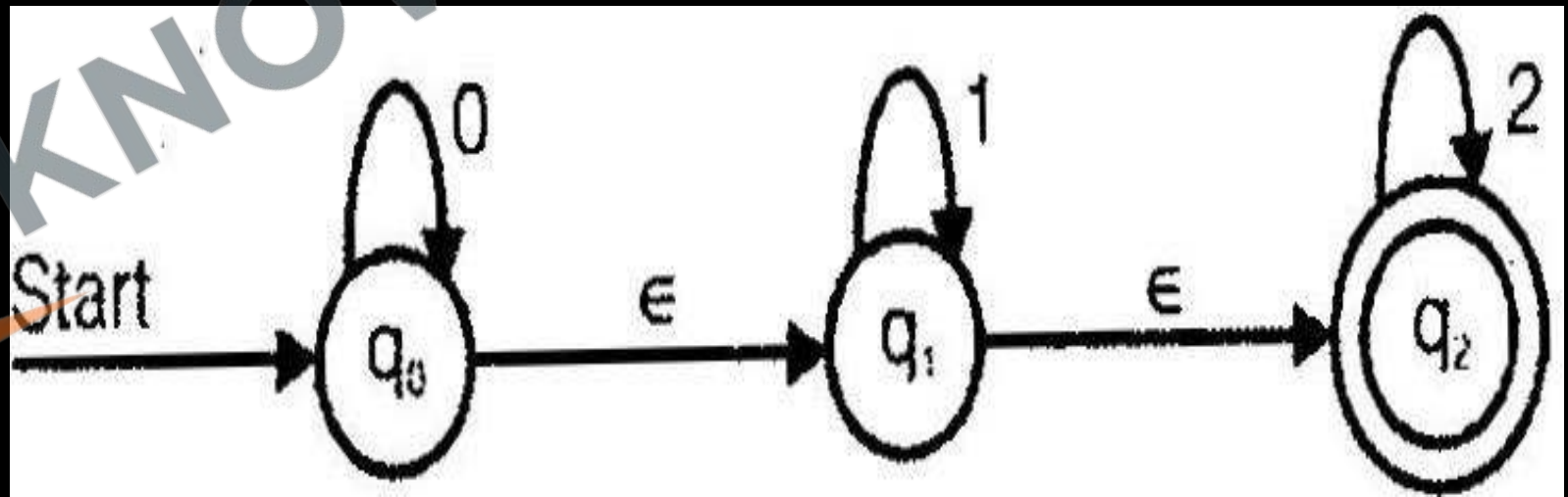
KNOWLEDGEGATE



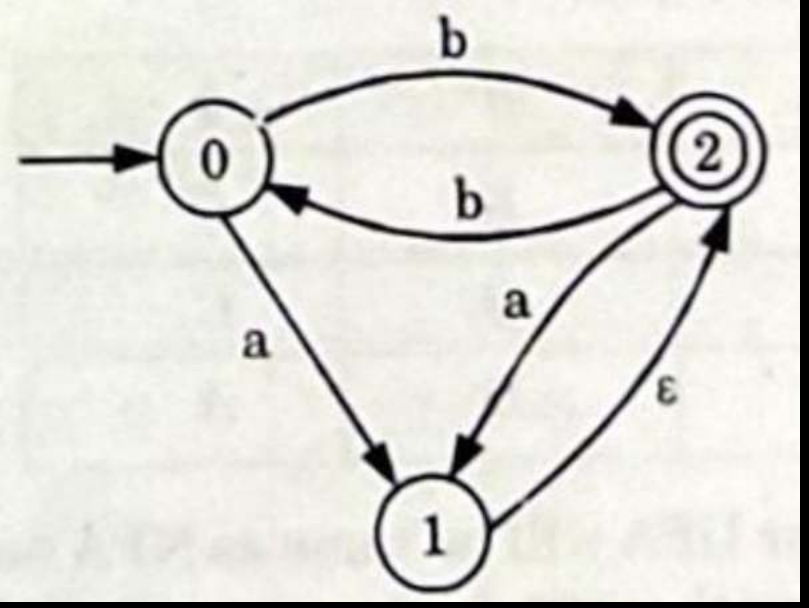
KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- Let us check for $\delta(q_0, a)$ = **ε-Closure [δ[ε-Closure(q_0), a]]**
- first, we find **ε-Closure** of q_0: {q_0, q_1, q_2}
- Now check transition of all **three** states on input symbol a we get: q_0
- After that we again calculate the **ε-Closure** of the above result q_0, we get the result as: **{q_0, q_1, q_2}**
- Similarly, we will check it for every state.
- All those states will be considered as a final state in which we have q_2(final state in initial **ε-NFA**) in there ε-Closure, i.e. all states will be final in this case.



KNOWLEDGEGATE



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Moore and Mealy Machine

- Both moore and mealy machine are special case of DFA
- Both acts like o/p producers rather than language acceptors
- In moore and mealy machine no need to define the final states
- No concepts of dead states and no concepts of final states
- Mealy and Moore Machines are equivalent in power.

George H Mealy

Edward F Moore

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Moore Machine

A Moore machine is a six-tuple ($Q, \sum, \Delta, \delta, \lambda, q_0$), where

- Q is a finite set of states:
- ∑ is the input alphabet:
- Δ is the output alphabet.
- δ is the transition function Q x ∑ into Q
- λ is the output function mapping Q into Δ and
- q_0 is the initial state.

KNOWLEDGEGATE

Examples: The below table shows the transition table of a Moore Machine.

Present state	Next state δ		Output
	$a = 0$	$a = 1$	λ
$\rightarrow q_0$	q_3	q_1	0
q_1	q_1	q_2	1
q_2	q_2	q_3	0
q_3	q_3	q_0	0

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- In moore machine for every state output is associated.
- If the length of i/p string is n, then length of o/p string will be n+1
- Moore machine response for empty string $\in$

KNOWLEDGEGATE

Q construct a Moore machine take all the string of a's and b's as i/p and counts the no of a's in the i/p string in terms of 1, ∑ = {a, b}, Δ = {0, 1}?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q construct a Moore machine take all the string of a's and b's as i/p and counts the no of occurrence of sub-string 'ab' in terms of 1, ∑ = {a, b}, Δ = {0, 1}?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q construct a Moore machine where ∑ = {0, 1}, Δ = {a, b, c}, machine should give o/p a, if the i/p string ends with 10, b if i/p string ends with 11, c otherwise?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Mealy Machine

- Mealy machine is a six-tuple **$(Q, \sum, \Delta, \delta, \lambda, q_0)$**, where all the symbols except λ have the same meaning as in the Moore machine. λ is the output function mapping $Q \times \sum$ into Δ.

- In case of mealy machine, the output symbol depends on the transition.

KNOWLEDGEGATE

Example: The below table shows the transition table of a Mealy Machine.

Present state	Next state			
	$a = 0$		$a = 1$	
	state	output	state	output
$\rightarrow q_1$	q_3	0	q_2	0
q_2	q_4	1	q_4	0
q_3	q_2	1	q_1	1
q_4	q_4	1	q_3	0

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- If the length of i/p string is n, then length of o/p string will be n
- Mealy machine do not response for empty string $\in$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q construct a Mealy machine take all the string of a's and b's as i/p and counts the no of a's in the i/p string in terms of 1, ∑ = {a, b}, Δ = {0, 1}?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q construct a Mealy machine take all the string of a's and b's as i/p and counts the no of occurrence of sub-string 'ab' in terms of 1, $\sum = \{a, b\}$, $\Delta = \{0, 1\}$?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

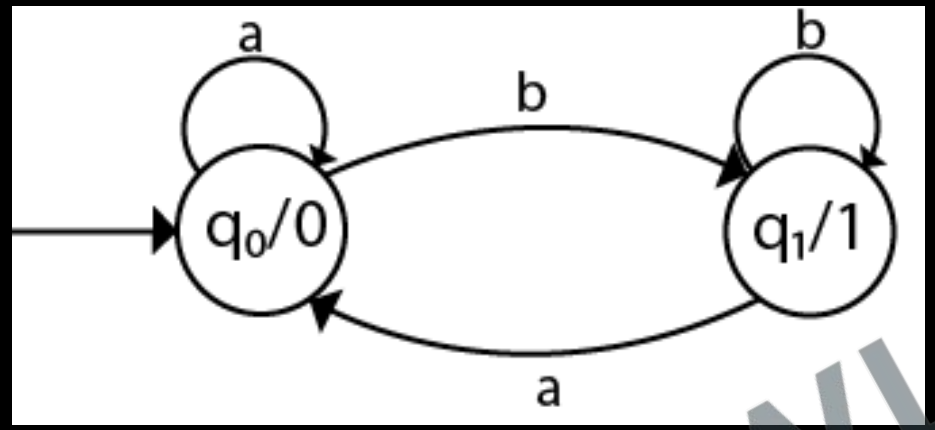
Q construct a Mealy machine where ∑ = {0, 1}, Δ = {a, b, c}, machine should give o/p a, if the i/p string ends with 10, b if i/p string ends with 11, c otherwise?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

CONVERSION OF MOORE TO MEALY MACHINE

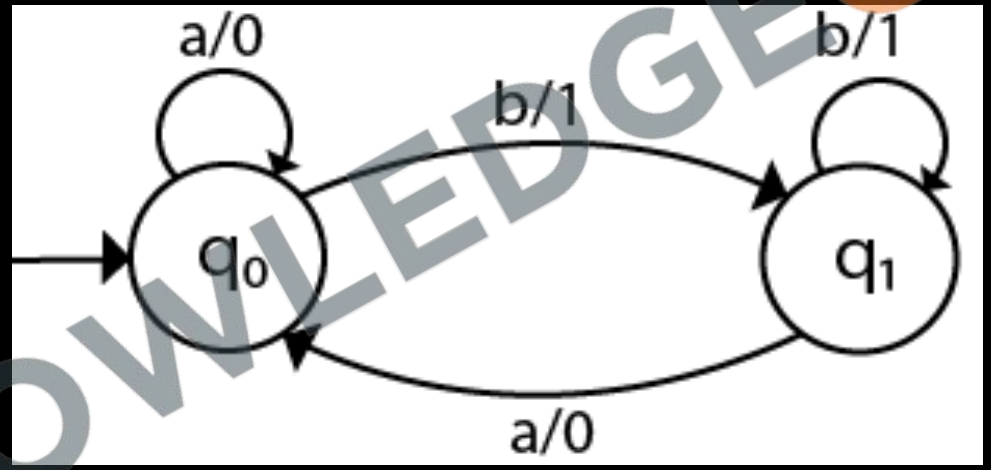
- Let us take an example to understand the conversion:
- Convert the following Moore machine into its equivalent Mealy machine.



Q	a	b	Output(λ)
q0	q0	q1	0
q1	q0	q1	1

KNOWLEDGEGATE

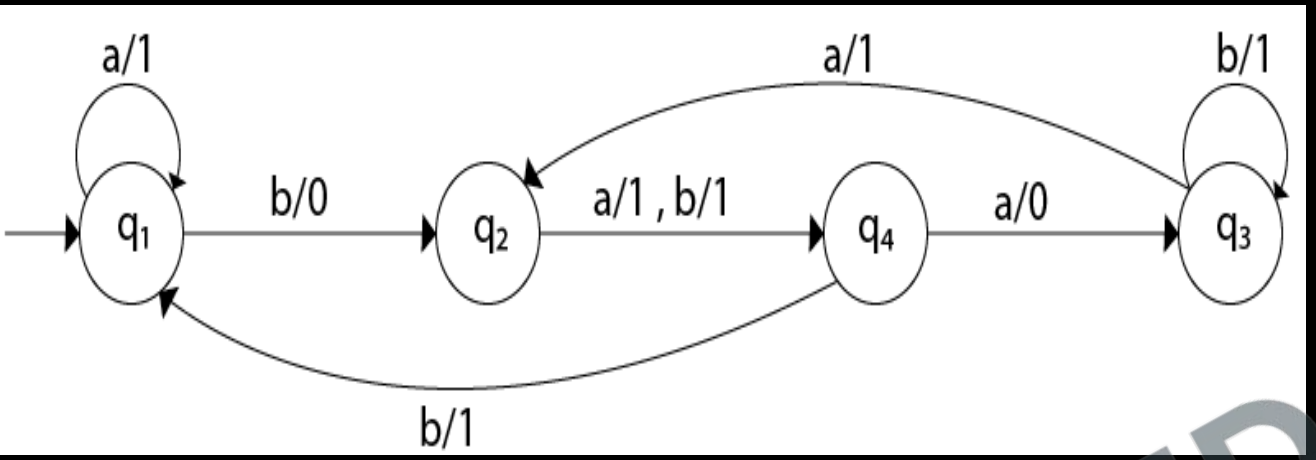
- To convert a mealy machine to moore machine all you need to do is just push out the outputs of states onto to the incoming transitions.
- While conversion from moore to mealy machine, the number of states will we same and there will be no extra states.
- The equivalent Moore Machine will be:



KNOWLEDGEGATE

PROCEDURE FOR TRANSFORMING A MEALY MACHINE INTO A MOORE MACHINE

Consider the Mealy Machine:



Present State	Next State			
	a		b	
	State	O/P	State	O/P
q_1	q_1	1	q_2	0
q_2	q_4	1	q_4	1
q_3	q_2	1	q_3	1
q_4	q_3	0	q_1	1

KNOWLEDGEGATE

Aspect	Moore Machine	Mealy Machine
Response to Null Input	Produces an output corresponding to the initial state.	Does not produce any output.
Output Length for Input of Length n	Generates an output string of length n+1.	Generates an output string of length n.
Initial Output	The initial output is determined by the initial state.	No initial output until an input is provided.
Output Synchronization with Input	Output is one step behind the input.	Output is synchronized with the input; changes immediately with input.
Typical Use Cases	Suitable for applications where stable output is necessary.	Ideal for applications requiring immediate response to input changes.

KNOWLEDGE GATE
https://www.knowledgegate.in/gate

MINIMIZATION OF FINITE AUTOMATA

- The process of elimination of states whose presence or absence doesn't affect the language accepting capability of deterministic Finite Automata is called minimization of automata and the result is minimal deterministic finite automata or commonly called as minimal finite automata as MFA.

- **NOTE- MFA is always unique for a language.**

KNOWLEDGEGATE

- It is sometimes difficult to design a minimal DFA directly so, a better approach is to first design the DFA and then minimize it.
- Based on **productivity**, the states of DFA can be mainly classified in two types-
- **PRODUCTIVE STATES**- State is said to be productive, if it adds any accepting power to the machine that is its presence and absence effect the language accepting capability of the machine.
- **NON- PRODUCTIVE STATES**- These states don't any add anything to the language accepting power to the machine.

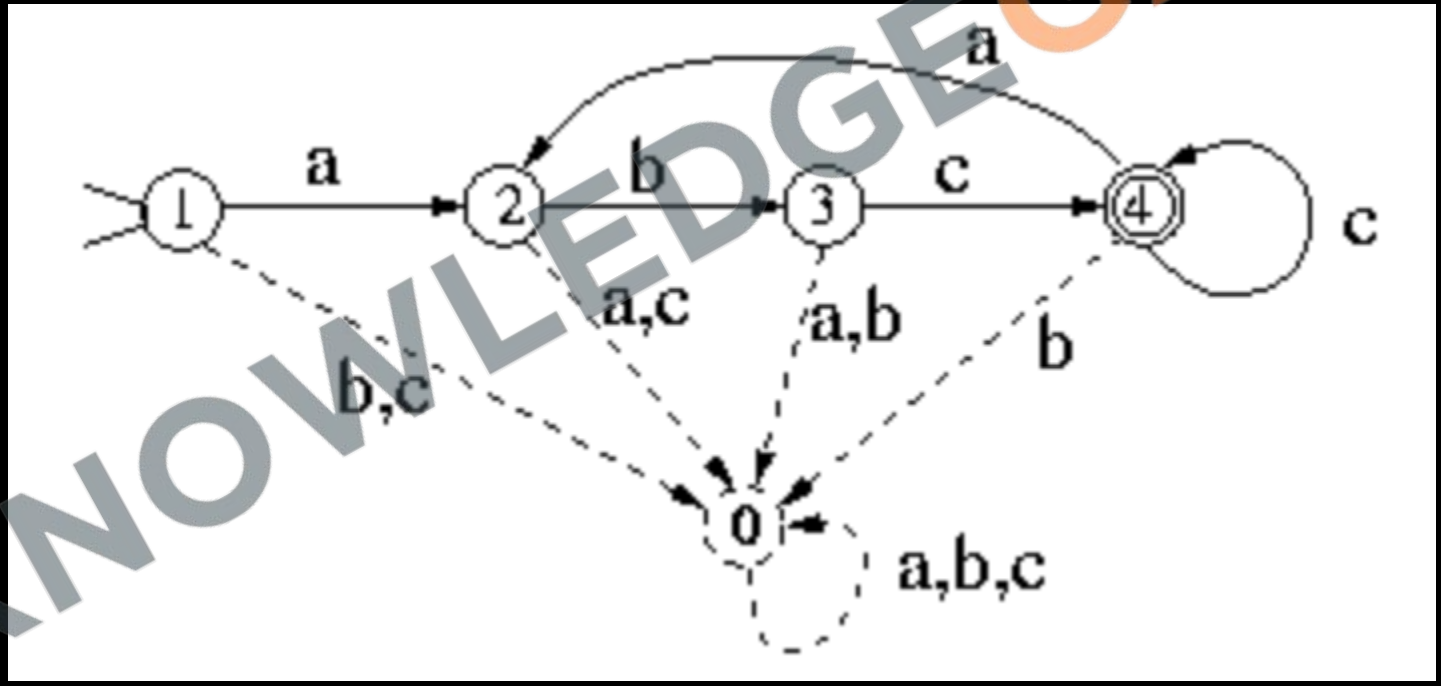
KNOWLEDGEGATE

NON- PRODUCTIVE STATES- These states don't add anything to the language accepting power to the machine. They can further be divided into three types-

- Dead State
- Unreachable Sate
- Equal State

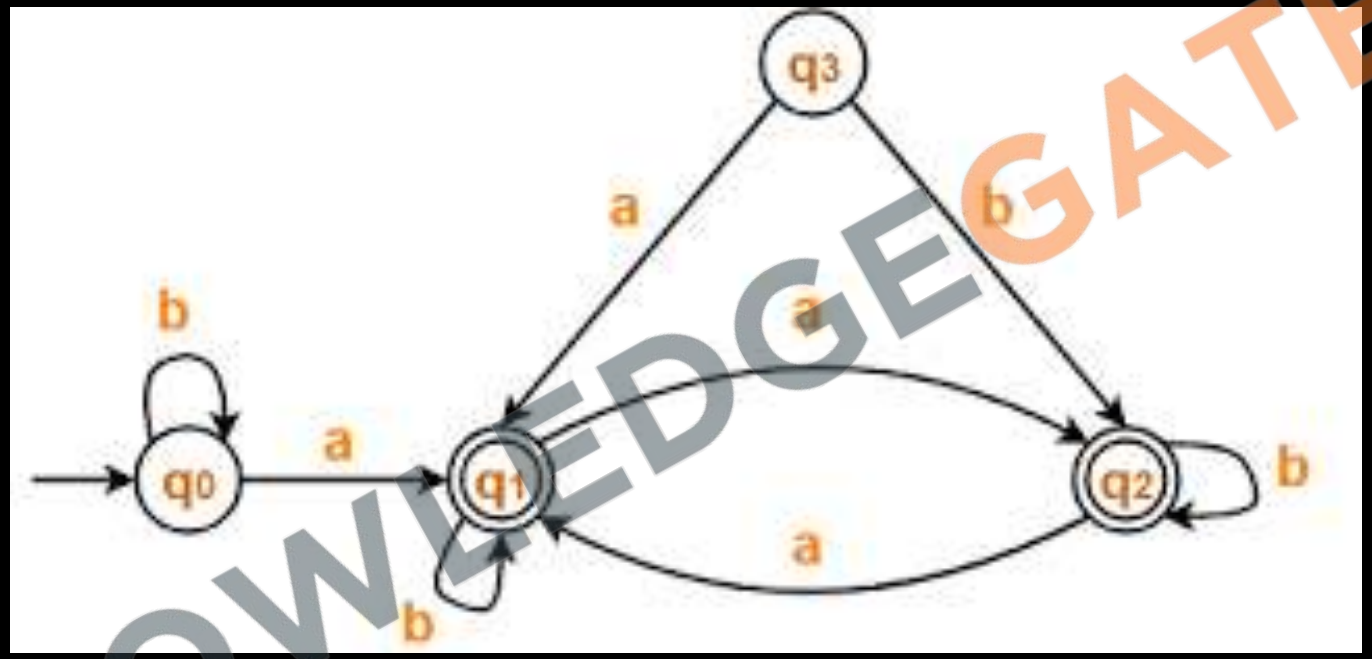
KNOWLEDGEGATE

- **Dead State**- It is basically created to make the system complete, can be defined as a state from which there is no transition possible to the final state.
- In a DFA there can be more than one dead state but logically always one dead state is sufficient to complete the functionality.



KNOWLEDGEGATE

- **Unreachable Sate**- It is that state which cannot be reached starting from initial state by parsing any input string.



KNOWLEDGEGATE

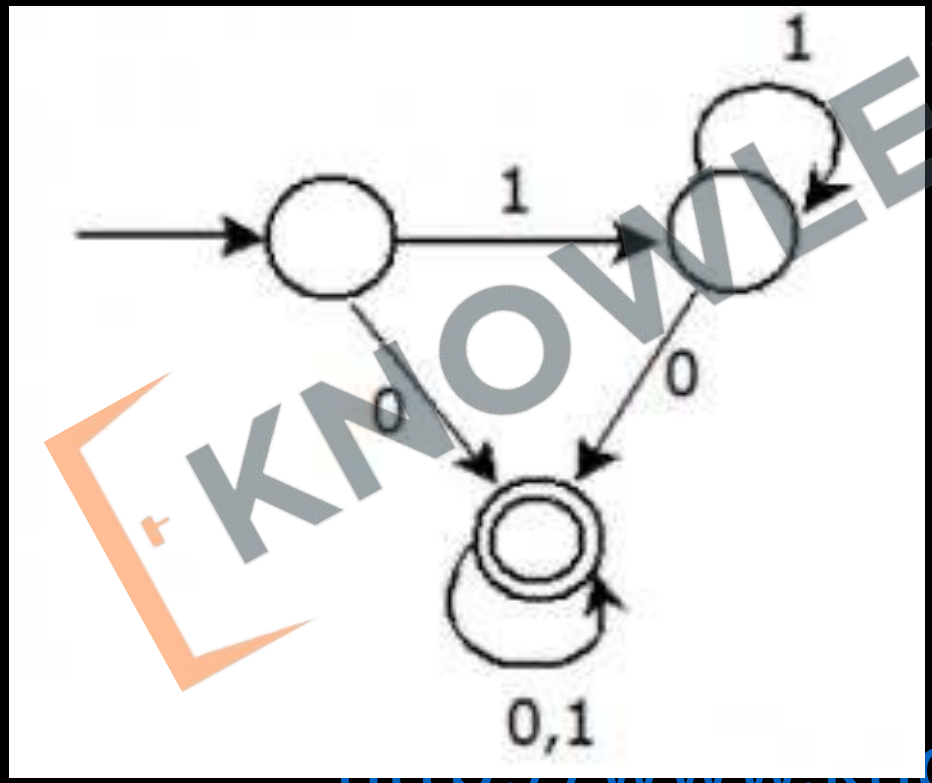
- **Equal State**-These are those states that behave in same manner on each and every input string. That is for any string w where $w \in \Sigma^*$ either both of the states will go to final state or both will go to non-final state. (remember the example of an equal state DFA).

- More formally, two states q_l and q_2 are equivalent (denoted by $q_1 \cong q_2$) if both $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states or both of them are non-final states for all $x \in \sum^*$. If q_1 and q_2 are k-equivalent for all $k \geq 0$, then they are *k*-equivalent.

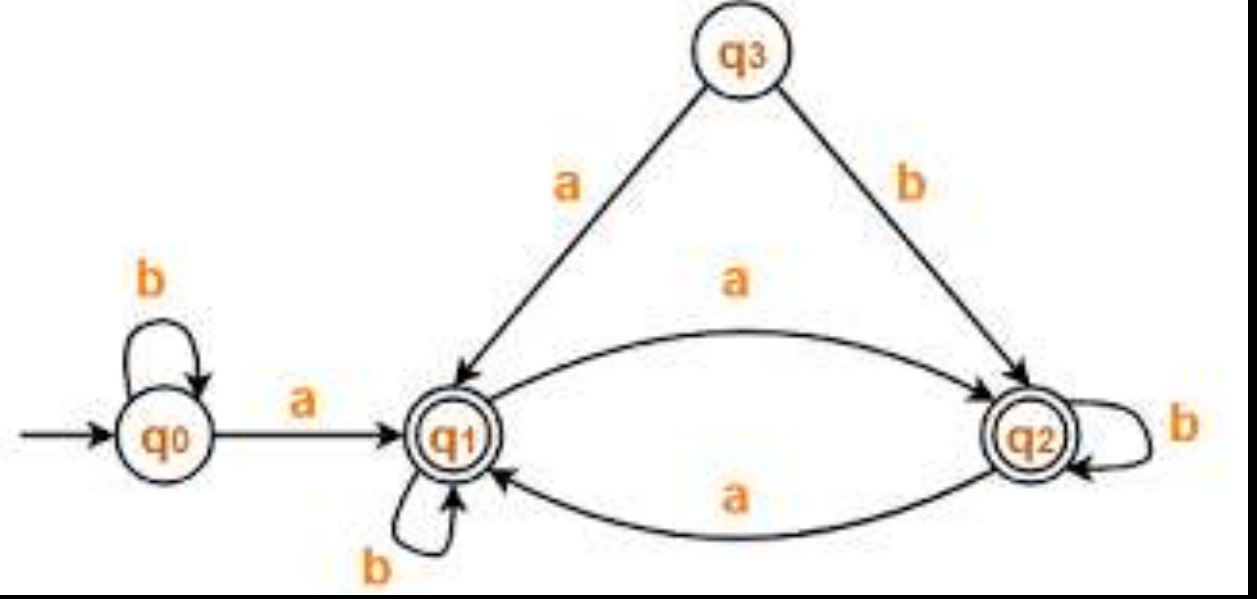
KNOWLEDGEGATE

Procedure of Minimization

- For this first of all, group all the non-final states in one set and all final states in another set.
- Now, on both the sets, individually check, whether any of the underlying elements (states) of that particular set are behaving in the same way, that is are they having same transition(to same set) on each input alphabet.
- if the answer is yes, then these two states are equal, otherwise not.

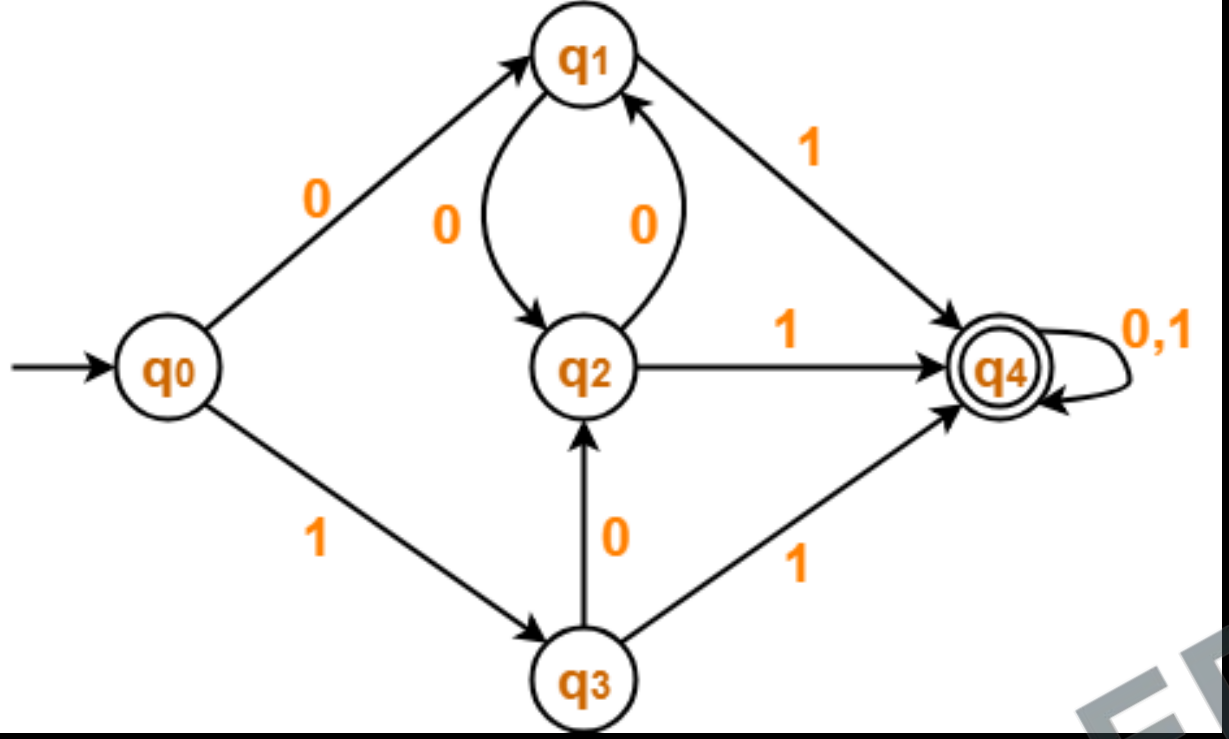


KNOWLEDGEGATE



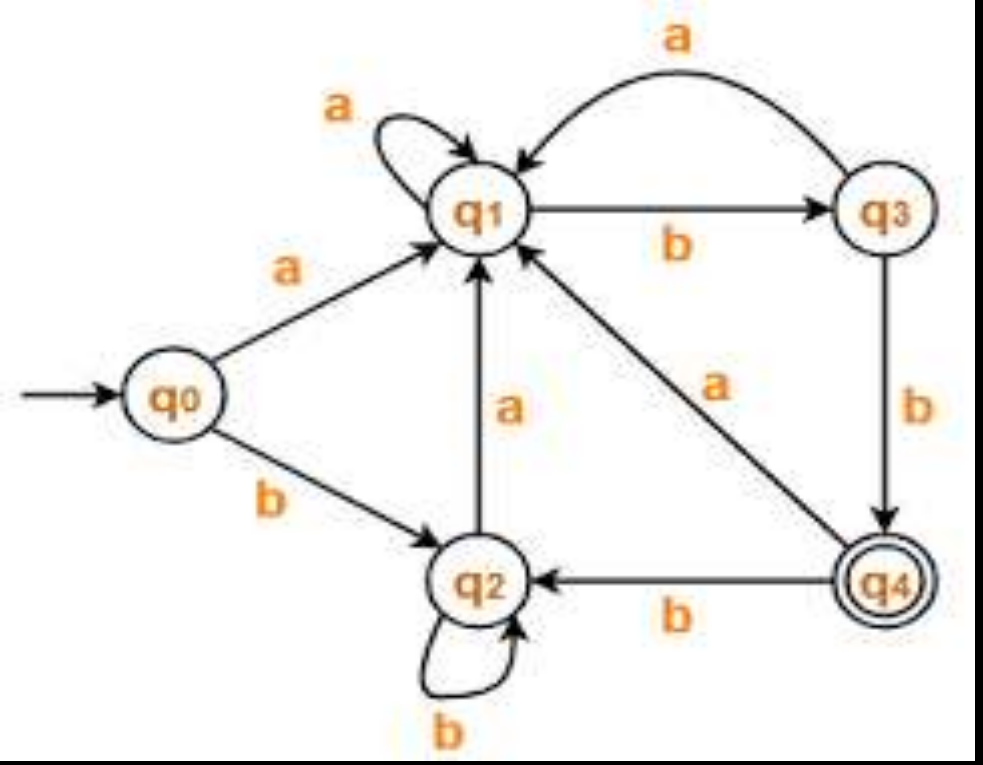
KNOWLEDGEGATE

http://www.knowledgegate.in/gate



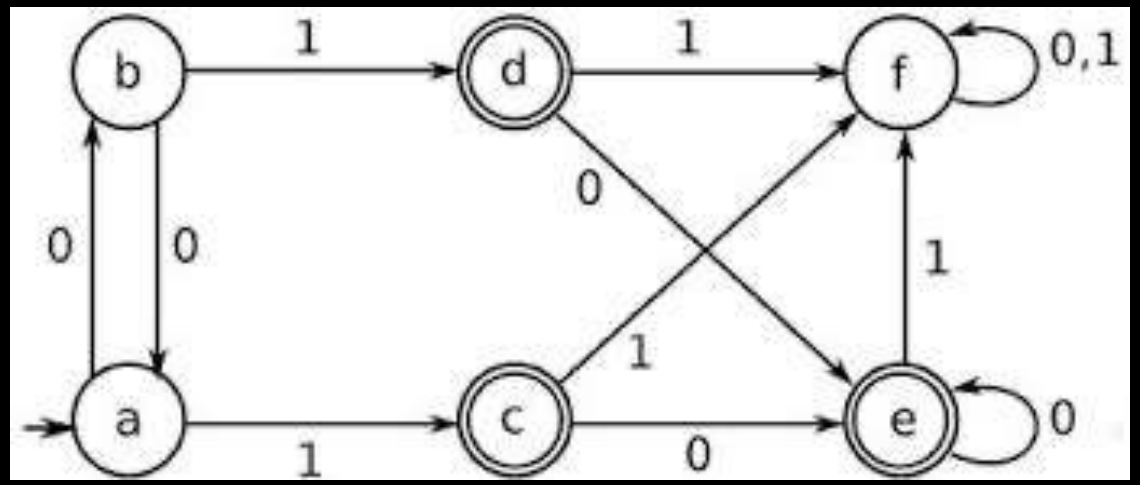
KNOWLEDGEGATE

http://www.knowledgegate.in/gate



KNOWLEDGEGATE

http://www.knowledgegate.in/gate



KNOWLEDGEGATE

- **Chapter-2 (Regular Expressions and Languages):** Regular Expressions, Transition Graph, Kleen's Theorem, Finite Automata and Regular Expression- Arden's theorem, Algebraic Method Using Arden's Theorem, Regular and Non-Regular Languages- Closure properties of Regular Languages, Pigeonhole Principle, Pumping Lemma, Application of Pumping Lemma, Decidability- Decision properties, Finite Automata and Regular Languages

KNOWLEDGEGATE

Regular Expressions

- One way of describing regular language is via the notation of regular expression. An expression of strings which represents regular language is called regular expression. The regular expressions are useful for representing certain sets of strings(Language) in an algebraic fashion.
- We give a formal recursive definition of regular expressions over ∑ as follows:
  - Any terminal symbol (i.e. an element of ∑), ∈ and Φ are regular expressions **(Primitive regular expressions)**.
  - A regular expression is valid iff it can be derived from a primitive regular expression by a finite number of applications of operators.

KNOWLEDGEGATE

Regular Language:- Any set(language) represented by a regular expression is called a **Regular language**. If for example, a, b ∈ ∑, then

- **R =** a denotes the L = {a}
- **R =** a.b denotes L = {ab} concatenation
- **R =** a + b denotes L = {a, b} Union
- **R =** a* denotes the set {∈, a, aa, aaa, ...} known as Kleene closure.
- **R =** a^+ Positive closure {a, aa, aaa…}
- **R =**(a + b)* denotes {a, b}*

KNOWLEDGEGATE

Operators

- When we view *a* in ∑ as a regular expression, we denote it by **a**.
  - If R is a regular expression, then **(R)** is also a regular expression.
  - The iteration (or closure) of a regular expression **R** written as **R***, is also a regular expression.
  - The iteration (or closure) of a regular expression **R** written as $\mathbf{R^+}$, is also a regular expression.
  - The concatenation of two regular expressions $\mathbf{R_1}$ and $\mathbf{R_2}$, written as $\mathbf{R_1\ R_2}$, is also a regular expression.
  - The union of two regular expressions $\mathbf{R_1}$ and $\mathbf{R_2}$, written as $\mathbf{R_1 + R_2}$, is also a regular expression.

KNOWLEDGEGATE

Operator Precedence

- The precedence order to solve is
  - ()Bracket
  - * (Kleene Closure)
  - + Positive Closure
  - Concatenation
  - Union

KNOWLEDGEGATE

IDENTITIES FOR Regular Expression

- Two regular expressions P and Q are equivalent (we write P = Q)
  - if P and Q represent the same set of strings.

KNOWLEDGEGATE

http egate.in/gate

- Every regular expression can generate only one regular language but, a regular language can be generated by more than one regular expression I.e. means two different regular expression can generate same language.

- Two regular expression are said to be equal if they generate same language.

- $r_1 = a^*$

- $r_2 = a^* + (aa)^*$

KNOWLEDGEGATE

हमारी language बताओ

1. R={a}
2. R={a + b}
3. R={a + b + c}

KNOWLEDGEGATE

1. R={a.b}

2. R={a.b + a}b

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a regular expression that represent a language ‘L’, where L={a} over the alphabet ∑={a}.

KNOWLEDGEGATE

Q design a regular expression that represent all strings over the alphabet ∑ = {a, b}, where every accepted string ‘w’ starts with substring s = ‘abb’

KNOWLEDGEGATE

Q design a regular expression that represent all strings over the alphabet ∑ = {a, b}. where every accepted string ‘w’ ends with substring s = ‘bab’?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a regular expression that represent all strings over the alphabet ∑ = {a, b}. where every accepted string ‘w’ contains sub string s = ‘aba’ ?

KNOWLEDGEGATE

Q Design a regular expression that represent all strings over the alphabet ∑ = {a, b} such that every accepted string start and end with a.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a regular expression that represent all strings over the alphabet ∑ = {a, b} such that every accepted string start and end with same symbol?

KNOWLEDGEGATE

Q Design a regular expression that represent all strings over the alphabet ∑ = {a, b} such that every accepted string start and end with different symbol ?

KNOWLEDGEGATE

Q Design a regular expression that represent all strings over the alphabet ∑ = {a, b} such that every accepted string w, is like w=SX S = ‘aaa/bbb’ ?

KNOWLEDGEGATE

Q Design a regular expression that represent all strings over the alphabet ∑ = {a, b} such that every accepted string w, is like w=XS. S = ‘aaa/bbb’ ?

[KNOWLEDGEGATE]

Q Design a regular expression that represent all strings over the alphabet ∑ = {a, b} such that every accepted string w, is like w=XSX. S = aaa/bbb ?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a regular expression that represent all strings over the alphabet ∑ = {a, b}, such that every string ‘w’ accepted must be like

i) |w| = 3 **ii)** |w|<=3 **iii)** |w|>=3

KNOWLEDGEGATE

Q Design a regular expression that represent all strings over the alphabet ∑ = {a, b} such that for every accepted string 2nd from left end is always b.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a regular expression that represent all strings over the alphabet ∑ = {a, b} such that for every accepted string 4^{th} from right end is always a.

[KNOWLEDGEGATE]

http://www.knowledgegate.in/gate

Q Design a regular expression that represent all strings over the alphabet ∑= {a, b}, such that every string ‘w’ where |W| = 0(mod 3)?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a regular expression that represent all strings over the alphabet ∑= {a, b}, such that every string ‘w’ where |W| = 3(mod 4)?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a regular expression that represent all strings over the alphabet ∑= {a, b}, such that every string ‘w’ where $|W|_a = 0(\text{mod } 3)$?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a regular expression that represent all strings over the alphabet ∑= {a, b}, such that every string ‘w’ where $|W|_b = 2(\text{mod } 3)$?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Algebraic Properties of regular expression

- **Closure Property** - Regular expressions satisfy closure property with respect to Union, Concatenation and kleene closure. If R_1 and R_2 are regular expression then the following will also be regular expression.

$r_1 + r_2$

$r_1.r_2$

r_1^*

r_1^+

KNOWLEDGEGATE

- **Associative Property**- Regular expression satisfy associative property with respect to union and intersection

$$(r_1 + r_2) + r_3 \square r_1 + (r_2 + r_3)$$

$$(r_1 . r_2) . r_3 \square r_1 . (r_2 . r_3)$$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Identity Property**- The identity property is satisfied as follows-

$r.\square = r$

$r+\square = r$

KNOWLEDGEGATE

- **Commutative Property**- Regular expressions are commutative with respect to union but not with respect to concatenation.

$r_1 + r_2 \square r_2 + r_1$

$r_1 . r_2 \square r_2 . r_1$

KNOWLEDGEGATE

- **Distributive Property**-Regular expression satisfy this property as follows-

$r_1(r_2+r_3) = r_1r_2+r_1r_3$

$(r_1+r_2)\, r_3 = r_1r_3+r_2r_3$

$r_1+(r_2.\, r_3) = (r_1+r_2)(r_1+r_3)$

$(r_1.r_2) + r_3 = (r_1 + r_3).\, (r_2 + r_3)$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Idempotent Property**-regular expressions satisfies idempotent property with respect to union but not with respect to concatenation.

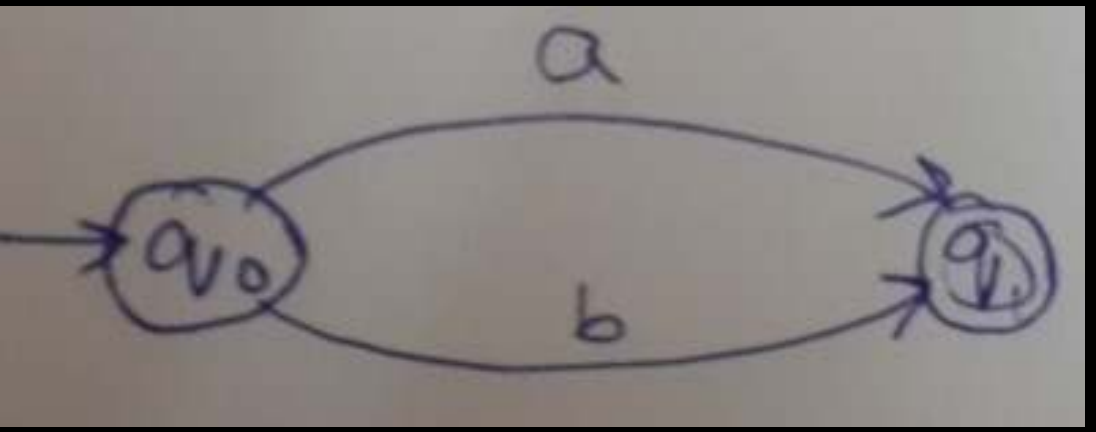
$r_1 + r_1 \square r_1$

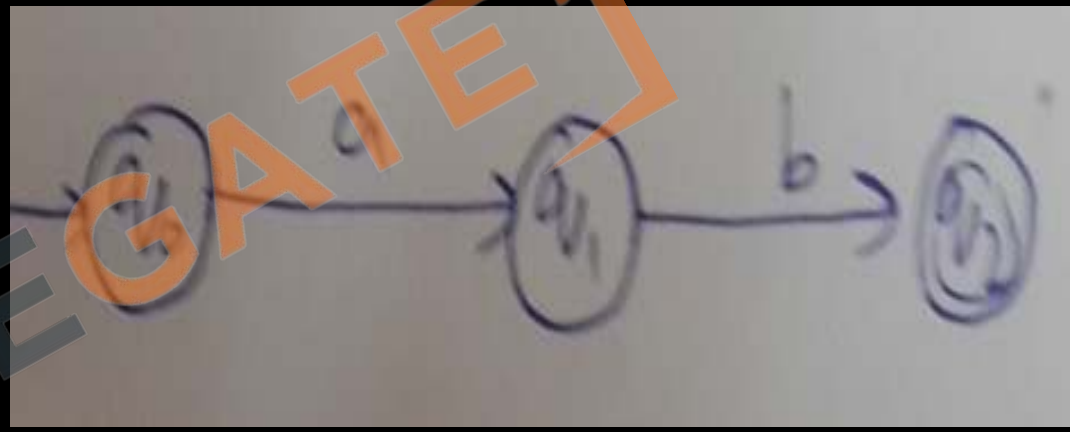
$r_1 . r_1 \square r_1$

KNOWLEDGEGATE

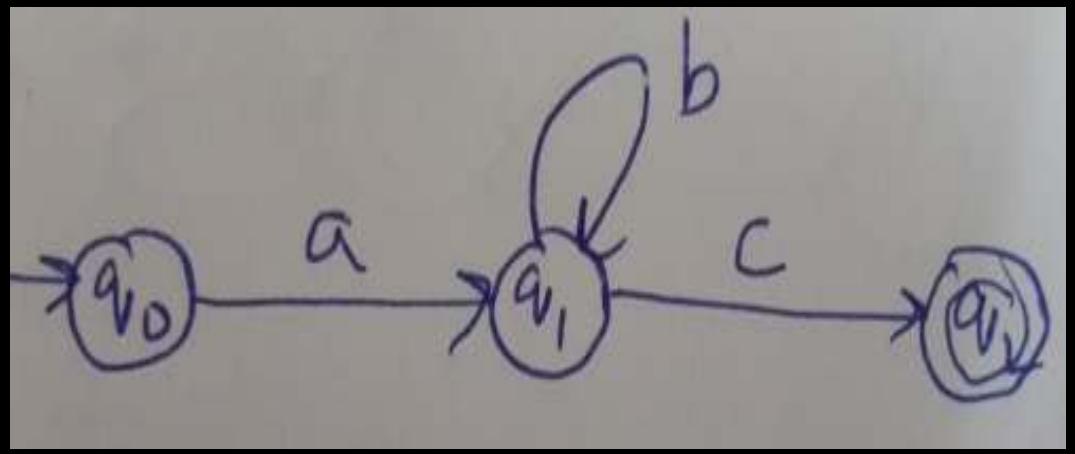
Conversion from Finite Automata to regular expression

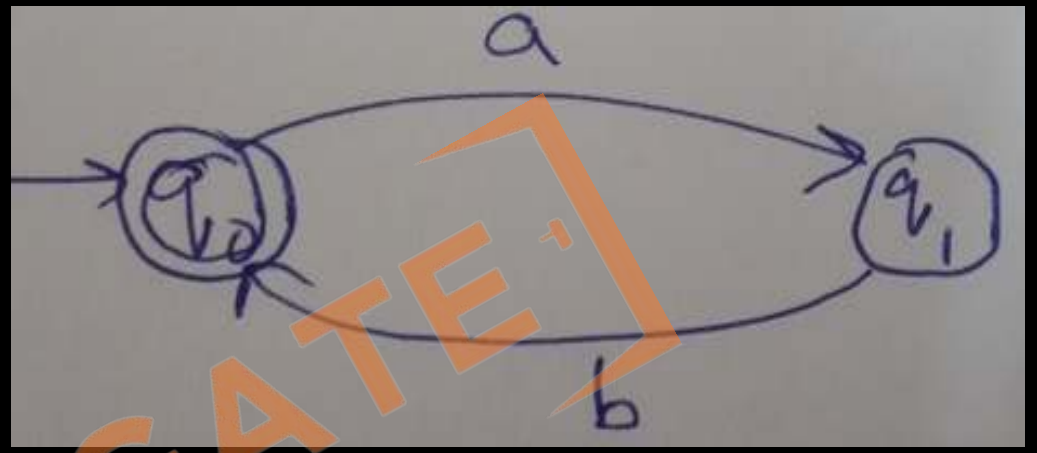
Q Write regular expressions for the following machines





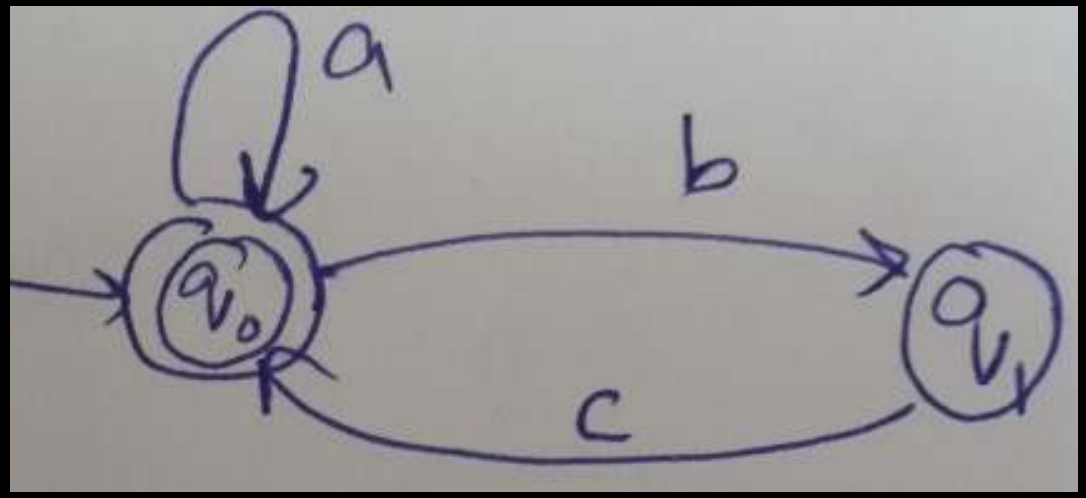
KNOWLEDGE GATE

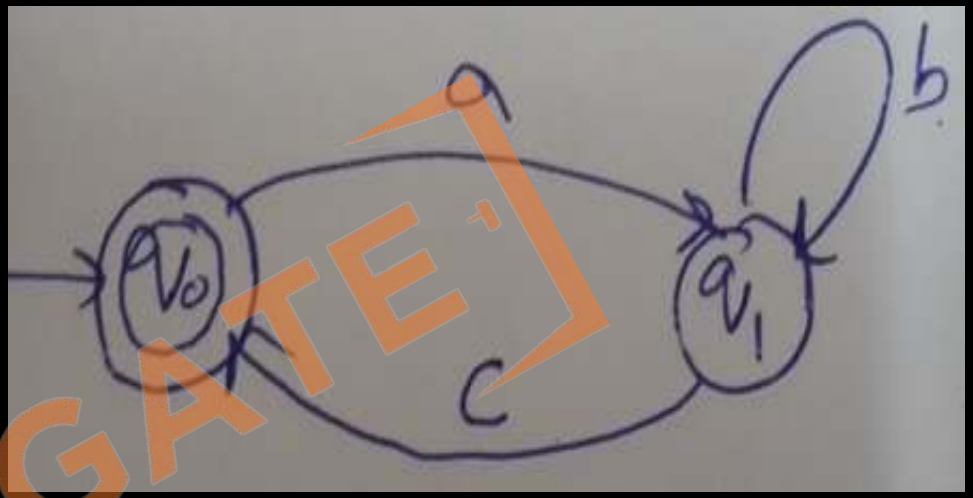




KNOWLEDGEGATE

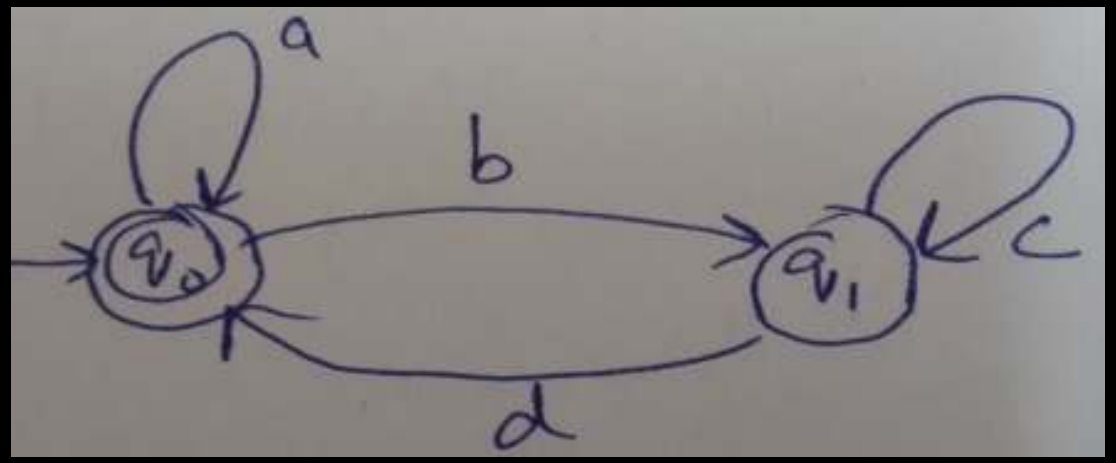
http://www.knowledgegate.in/gate

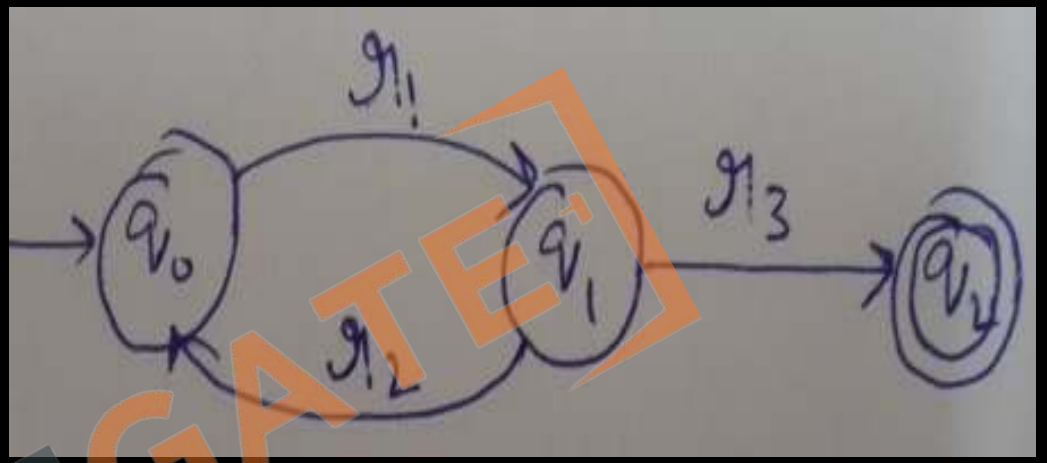




KNOWLEDGEGATE

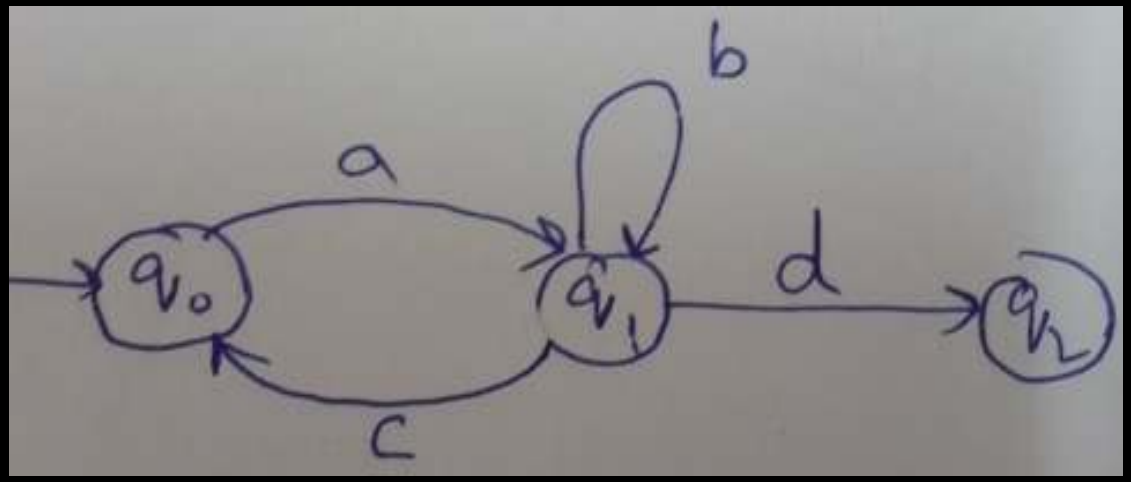
http://www.knowledgegate.in/gate

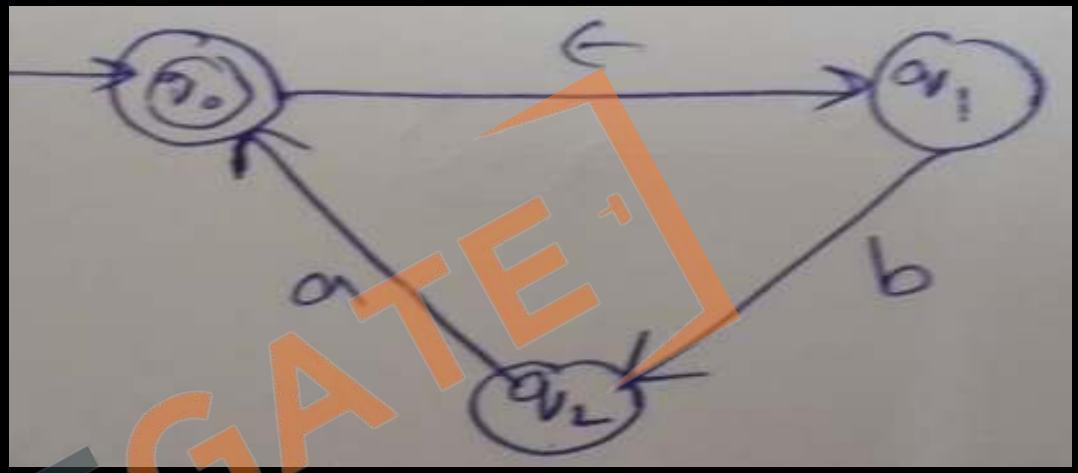




KNOWLEDGEGATE

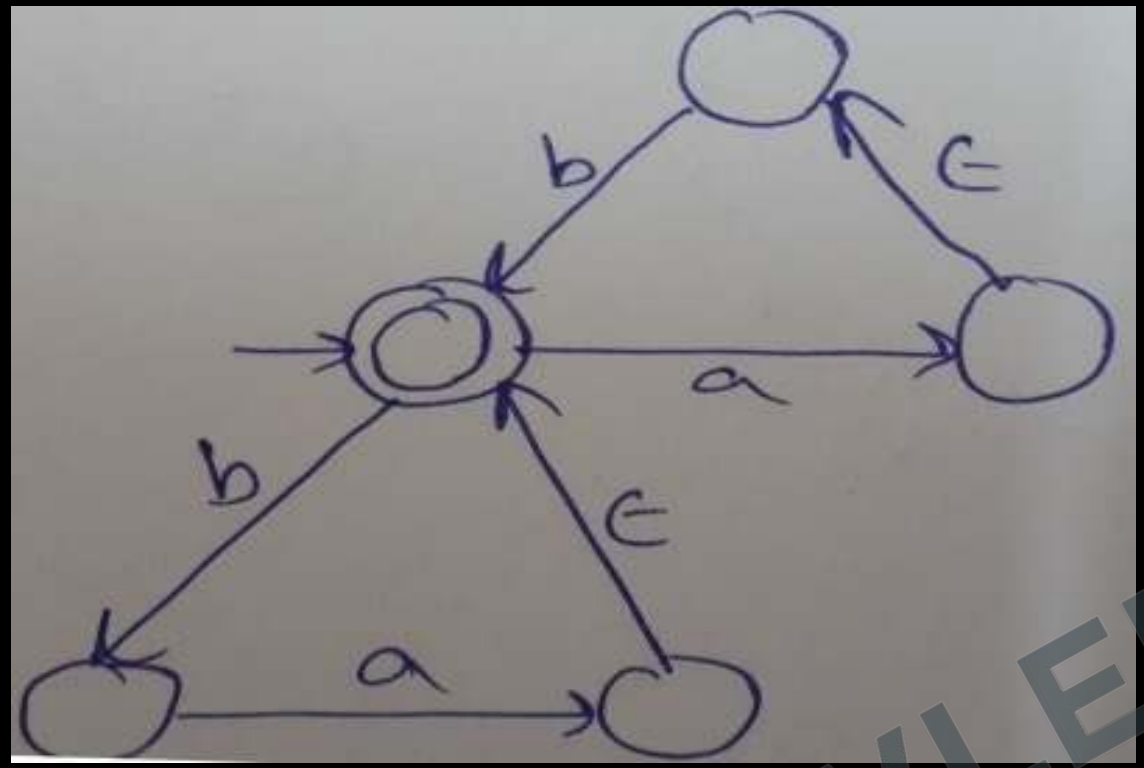
http://www.knowledgegate.in/gate

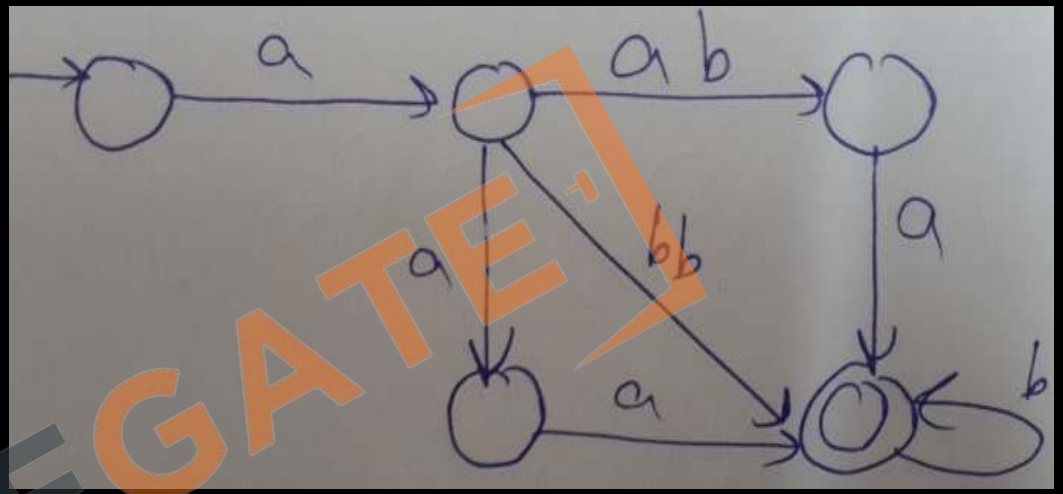




KNOWLEDGEGATE

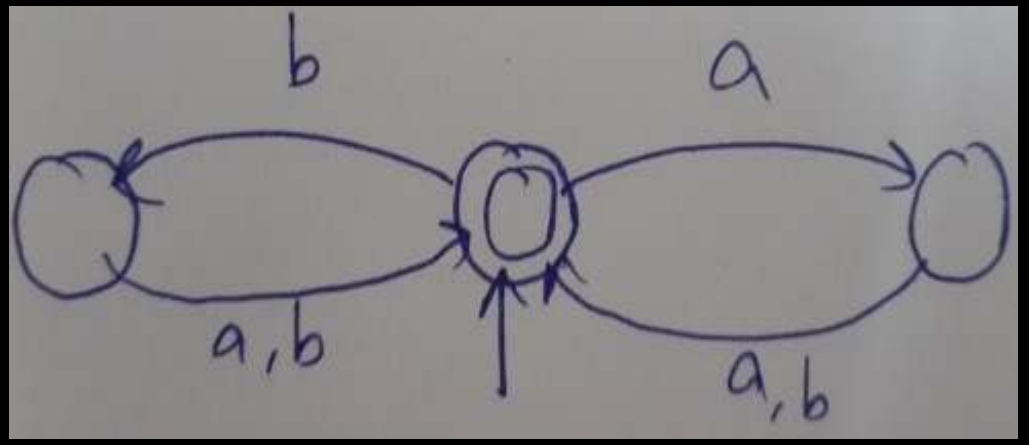
http://www.knowledgegate.in/gate

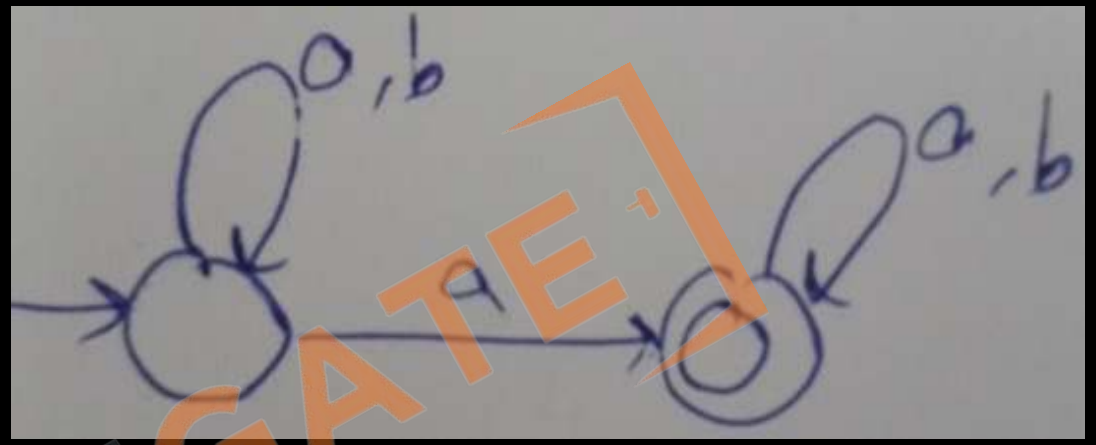




KNOWLEDGEGATE

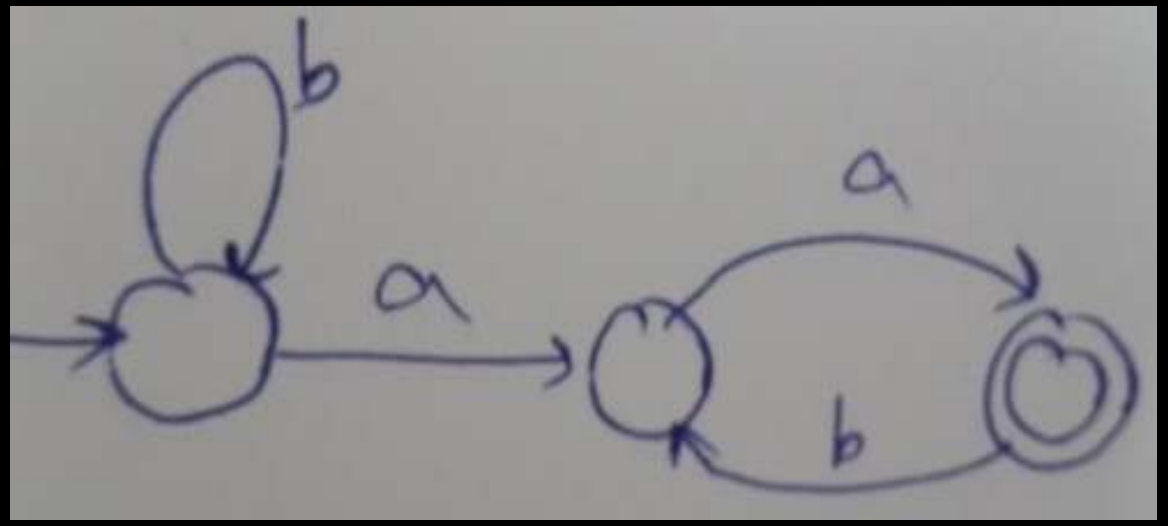
http://www.knowledgegate.in/gate

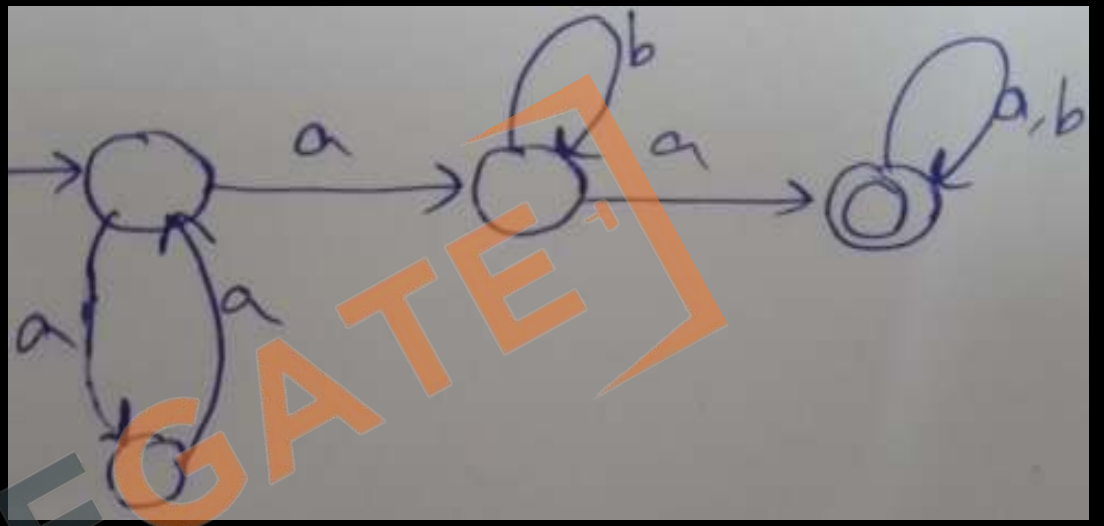




KNOWLEDGEGATE

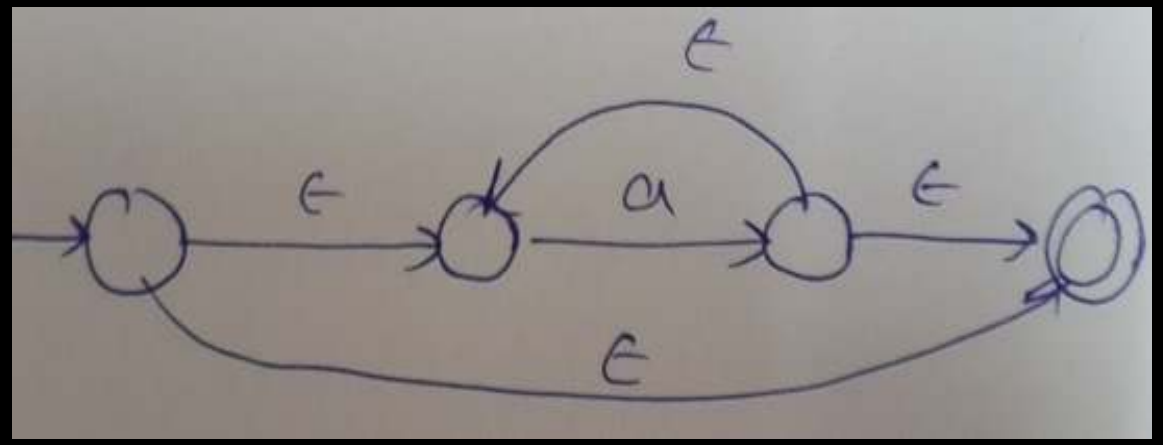
http://www.knowledgegate.in/gate

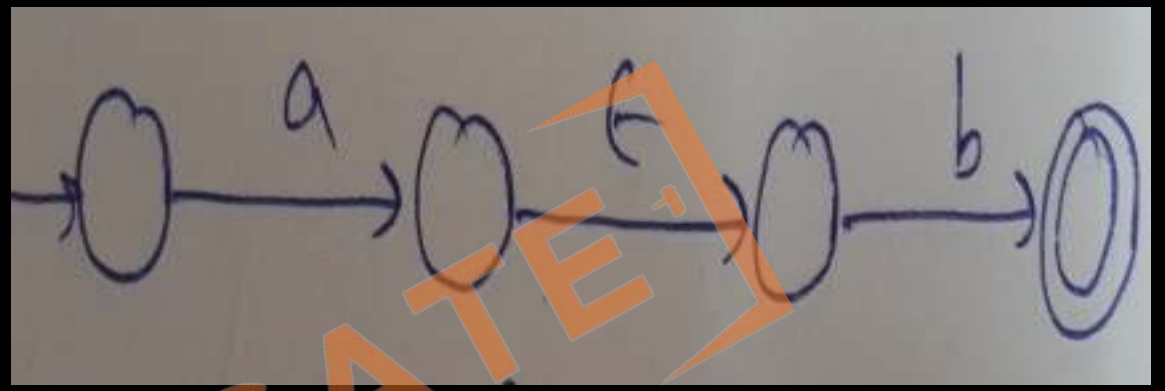




KNOWLEDGEGATE

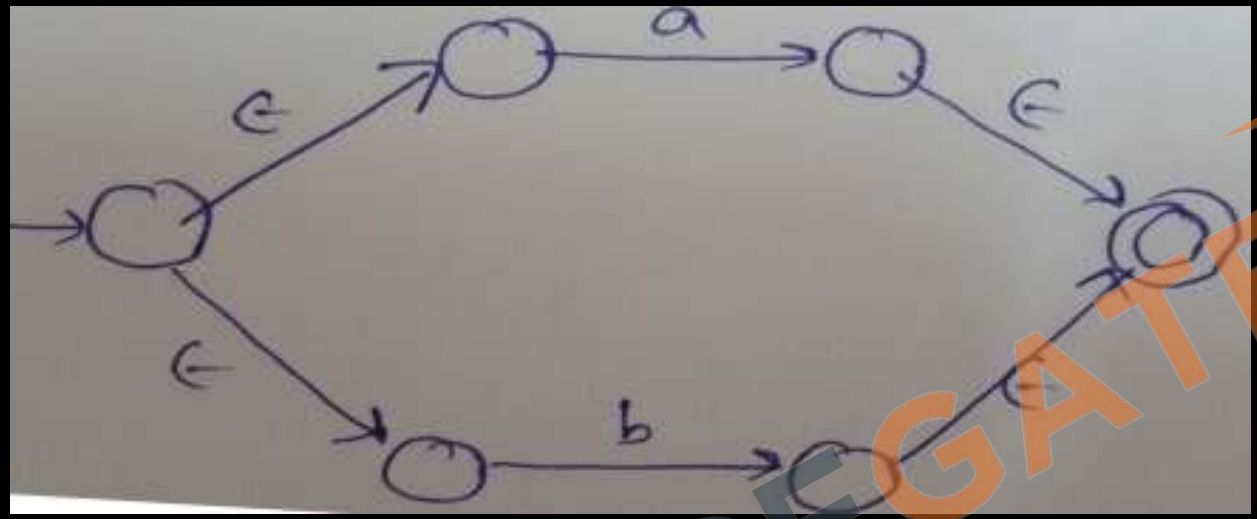
http://www.knowledgegate.in/gate





KNOWLEDGEGATE

http://www.knowledgegate.in/gate



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Conversion from regular expression in Finite Automata

i) R^*

ii) $(R_1.R_2)^*$

iii) $(R_1+R_2)^*$

KNOWLEDGEGATE

iv) $(R_1 * R_2 . R_3 *)$

v) R = a*b(ab)*

vi) R = (a + ba)*ab*

KNOWLEDGEGATE

vii) R = (aa + aaa)*

viii) R = (a + aaaaa)*

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

1. 0 * 10 * 10 *

2. ((ab)* + c*)*

3. (∈ + a + aa + aaa)b* + (a + b)* ba(a + b)*

4. 0*(10 * 1*)*0*

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

1. a*b(b * + aa*b)*

2. ((11*0+0)(0 + 1)*0*1*)

3. (ab+ bc+ acc).(a+ bc)*.(∈+ bc*a)*

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

EQUIVALENCE BETWEEN Regular Expression AND FINITE AUTOMATA

- **ARDEN'S THEOREM** is the mechanism for the construction of a regular expression from a finite automaton.

KNOWLEDGEGATE

Q Consider a DFA and convert it into regular expression using Arden's theorem?

$\delta(A, a) = A$

$\delta(A, b) = B$

$\delta(B, a) = B$

$\delta(B, b) = B$

A is the initial state and B Is the final state

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- Steps used-
  - For, every individual state of the DFA, write an expression for every incoming and outgoing input alphabet.
  - Apply Arden's theorem as follows-
    - If P is free from NULL, then equation R=Q+RP has unique solution, R=QP*
    - If P contains NULL, then equation R=Q+RP has infinitely many solutions.

[KNOWLEDGEGATE]

Q Consider a DFA and convert it into regular expression using Arden's theorem?

$\delta(A, b) = A$

$\delta(A, b) = B$

$\delta(B, a) = C$

$\delta(C, b) = C$

A is the initial state and B,C Is the final state

KNOWLEDGEGATE

Transition Graph

- Same as $\in$-nfa Can have more than one initial state .
- Non-Deterministic in nature (so no dead state is required, can take multiple moves).
- Can have string as a input to transition from one state to another.

KNOWLEDGEGATE

Kleene's Theorem

- The theorem has two main parts:
  - **For any given regular expression, there is an equivalent finite automaton (either deterministic or non-deterministic) that recognizes the same language.**
  - **For any finite automaton, there is an equivalent regular expression that generates the same language.**

KNOWLEDGEGATE

Closure Properties of Regular Languages

- Regular languages are closed under following operations
  - Kleen Closure
  - Positive closure
  - Complement
  - Reverse Operator
  - Prefix Operator
  - Suffix operator
  - Concatenation
  - Union
  - Intersection
  - Set Difference operator
  - Symmetric Difference

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

	RL	DCFL	CFL	CSL	RS	RES
Union	Y	N	Y	Y	Y	Y
Intersection	Y	N	N	Y	Y	Y
Complement	Y	Y	N	Y	Y	N
Set Difference	Y	N	N	Y	Y	N
Kleene Closure	Y	N	Y	Y	Y	Y
Positive Closure	Y	N	Y	Y	Y	Y
Concatenation	Y	N	Y	Y	Y	Y
Intersection with regular set	Y	Y	Y	Y	Y	Y
Reverse	Y	Y	Y	Y	Y	Y
Subset	N	N	N	N	N	N

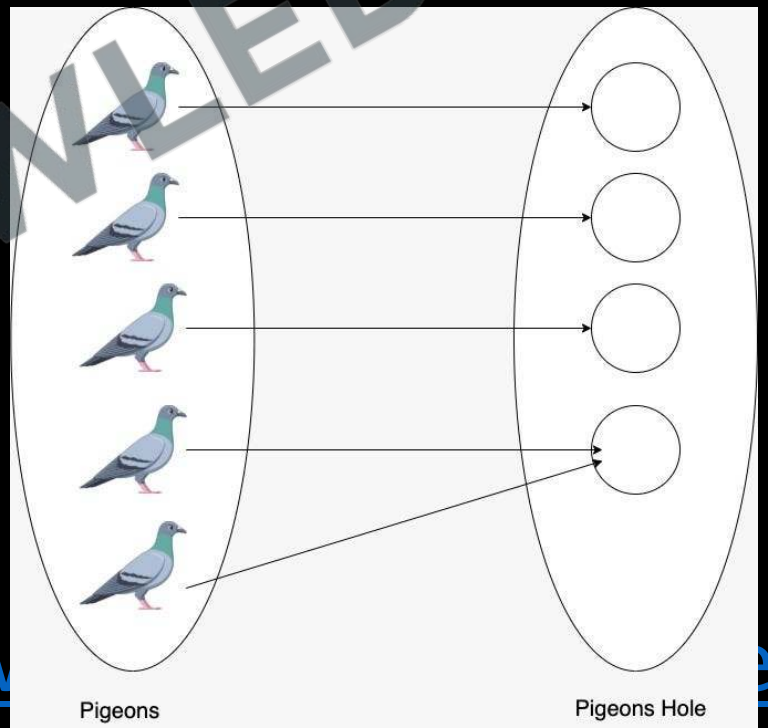
KNOWLEDGEGATE

	RL	DCFL	CFL	CSL	RS	RES
Homomorphism	Y	N	Y	N	N	Y
∈ Free Homomorphism	Y	N	Y	Y	Y	Y
Inverse Homomorphism	Y	Y	Y	Y	Y	Y
Substitution	Y	N	Y	N	N	Y
∈ Free Substitution	Y	N	Y	Y	Y	Y
Quotient with regular set	Y	Y	Y	N	Y	Y

KNOWLEDGEGATE
nowledg

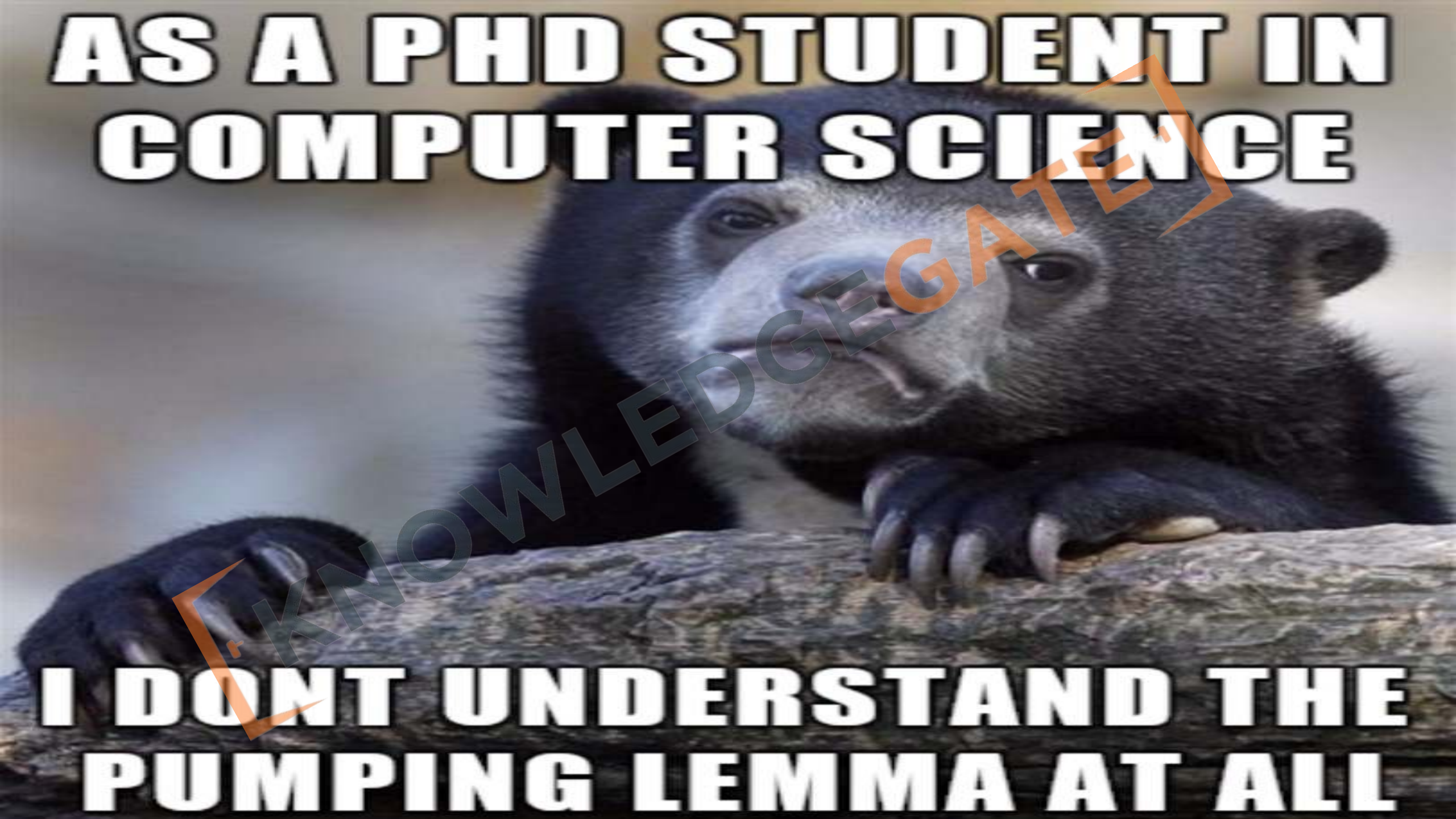
Pigeonhole Principle

- The Pigeonhole Principle is a fundamental principle of combinatorial mathematics that states:
- If *n* items are put into *m* containers, with *n*>*m*, then at least one container must contain more than one item.
- This principle seems intuitive, but it has powerful implications and can be used to prove a variety of seemingly unrelated results across mathematics and computer science.



KNOWLEDGEGATE

http://w e.in/gate



KNOWLEDGEGATE

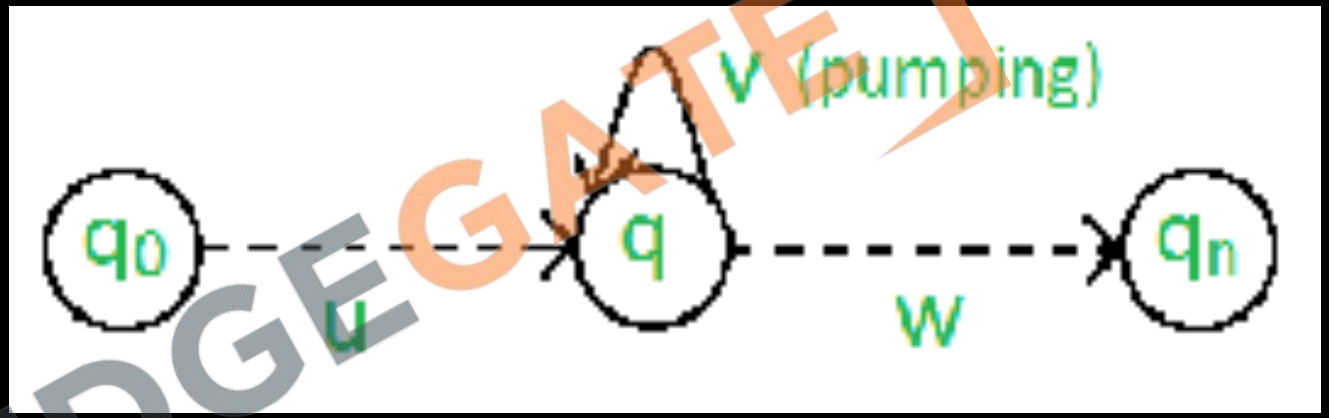
Pumping Lemma For Regular Languages

- Pumping Lemma is used as a proof for irregularity of a language. Thus, if a language is regular, it always satisfies pumping lemma. If there exists at least one string made from pumping which is not in L, then L is surely not regular.
- The opposite of this may not always be true. That is, if Pumping Lemma holds, it does not mean that the language is regular.
- Pumping Lemma is used to prove that some of the language is non-regular.
- For the pumping lemma, i/p is NRL & o/p is also NRL.

KNOWLEDGEGATE

Pumping Lemma For Regular Languages

- For any regular language L, there exists an integer n, such that for all z ∈ L with |z| ≥ n, there exists u, v, w ∈ Σ^*, such that z = uvw, and
  - |uv| ≤ n
  - |v| ≥ 1
  - for all i ≥ 0: $uv^iw \in L$



- In simple terms, this means that if a string v is 'pumped', i.e., if v is inserted any number of times, the resultant string still remains in L.

KNOWLEDGEGATE

Process of pumping Lemma

- Suppose that we need to prove that language L is a non-regular.
- Assume that L is a regular language, then L must satisfy pumping lemma property.
- Choose z ∈ L such that |z| <= n, split z into 3 parts.
  - |uv| ≤ n
  - |v| ≥ 1

- If there exist at least one variable for i such that uv^iw L, then L does not satisfy pumping lemma property so it is a contradiction, therefore language L is non-regular.

KNOWLEDGEGATE

Q Proof that language L = $\{a^m b^n \mid m=n\}$ is non-regular?

L = {ab, aabb, aaabbb, aaaabbbb, ---}

$z \in L$

$Z = a^k b^k$

For i=1 → uv^iw → $a^k b^k$

For i=2 → uv^iw → $a^{k+1} b^k$

KNOWLEDGEGATE

Q Proof that language L = $\{a^m b^n \mid m<n\}$ is non-regular?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Proof that language L = {$a^n b^n$ | n is a prime number} is non-regular?

KNOWLEDGEGATE

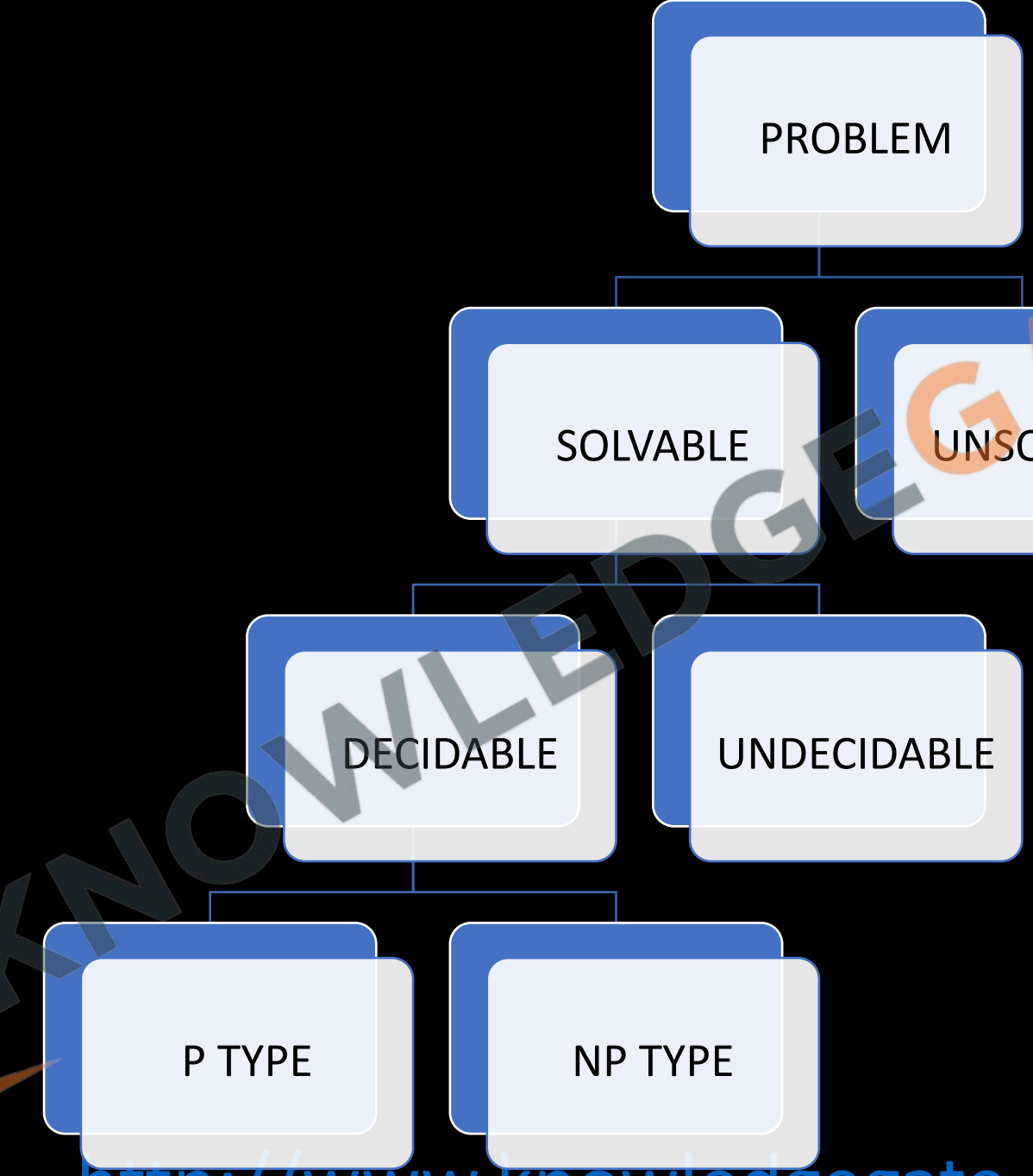
http://www.knowledgegate.in/gate

Q Proof that language L = {$a^{n\hat{}2}$ | n>= 0} is non-regular?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

KNOWLEDGEGATE



KNOWLEDGE GATE

http://www.knowledgegate.in/gate

- **SOLVABLE** - A problem is said to be solvable if either we can solve it or if we can prove that the problem cannot be solved

- **UNSOLVABLE** - A problem is said to be unsolvable if neither we can solve it, nor we can proof that the problem can not be solved

KNOWLEDGEGATE

- P- if there exist a polynomial time algorithm to solve a problem then problem is said to be decidable.

- NP- if there exist a non- polynomial time algo to solve a problem.

KNOWLEDGEGATE

Decision properties

- Approximately all the properties are decidable in case a finite automaton. Here we will use machine model to proof decision properties.

i) Emptiness

ii) Non-emptiness

iii) Finiteness

iv) Infiniteness

v) Membership

vi) Equality

KNOWLEDGEGATE

Emptiness & Non-emptiness

- Step 1: - select the state that cannot be reached from the initial states & delete them (remove unreachable states)
- Step 2: - if the resulting machine contains at least one final states, so then the finite automata accepts the non-empty language.
- Step 3: - if the resulting machine is free from final state, then finite automata accepts empty language.

KNOWLEDGEGATE

Finiteness & Infiniteness

- Step 1: - Select the state that cannot be reached from the initial state & delete them (remove unreachable states)
- Step 2: - Select the state from which we cannot reach the final state & delete them (remove dead states)
- Step 3: - If the resulting machine contains loops or cycles then the finite automata accepts infinite language
- Step 4: - If the resulting machine do not contain loops or cycles then the finite automata accepts finite language.

KNOWLEDGEGATE

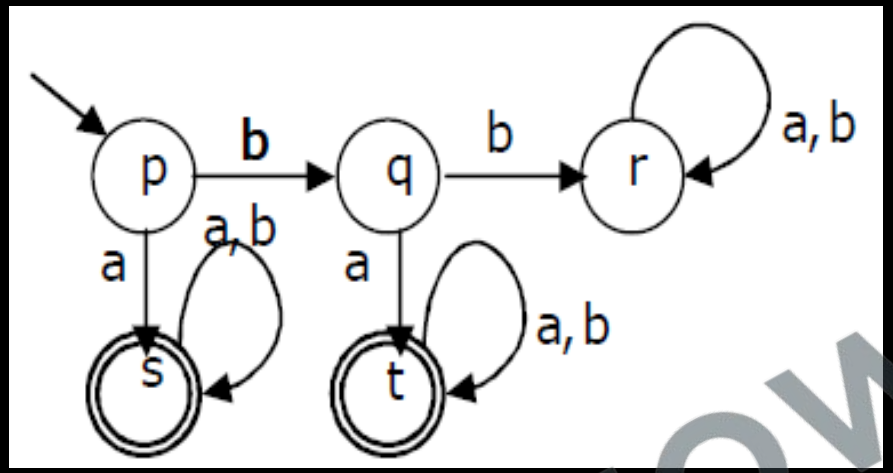
Membership

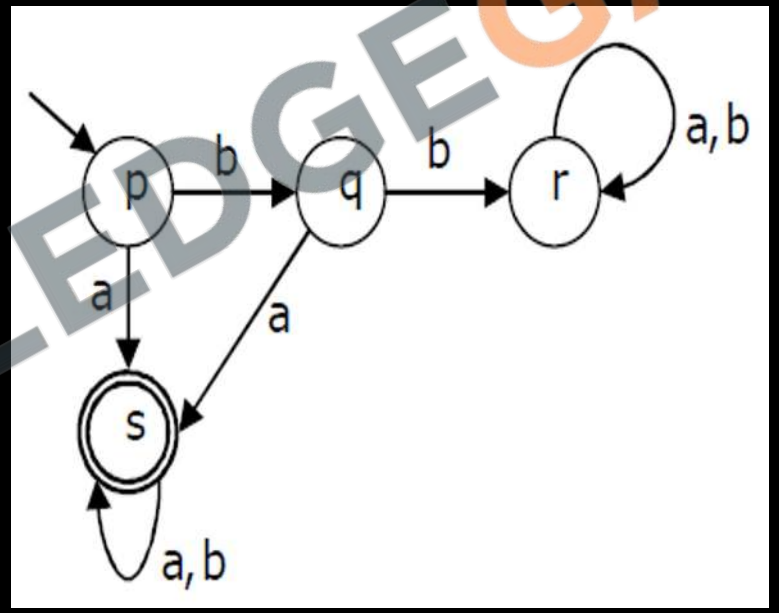
- Membership is a property to verify an arbitrary string is accepted by a finite automaton or not i.e. it is a member of the language or not.
- Let M is a finite automata that accepts some strings over an alphabet, and let ‘w’ be any string defined over the alphabet, if there exist a transition path in M, which starts at initial state & ends in anyone of the final state, then string ‘w’ is a member of M, otherwise ‘w’ is not a member of M.

KNOWLEDGEGATE

Equality

- Two finite state automata M_1 & M_2 is said to be equal if and only if, they accept the same language.
- Minimise the finite state automata and and the minimal DFA will be unique.





KNOWLEDGEGATE

	RL	DCFL	CFL	CSL	RS	RES
Emptiness	Y	Y	Y	X	N	N
Non-Emptiness	Y	Y	Y	X	N	N
Finiteness	Y	Y	Y	X	N	N
Infiniteness	Y	Y	Y	X	N	N
Membership	Y	Y	Y	X	Y	N
Equality	Y	N	N	X	N	N
Ambiguity	Y	N	N	X	N	N
Σ^*	Y	N	N	X	N	N
Halting	Y	Y	Y	X	Y	N

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Chapter-3 (Regular and Non-Regular Grammars):** Context Free Grammar(CFG)-Definition, Derivations, Languages, Derivation Trees and Ambiguity, Regular Grammars-Right Linear and Left Linear grammars, Conversion of FA into CFG and Regular grammar into FA, Simplification of CFG, Normal Forms- Chomsky Normal Form(CNF), Greibach Normal Form (GNF), Chomsky Hierarchy, Programming problems based on the properties of CFGs.

KNOWLEDGE GATE

http://www.knowledgegate.in/gate

Introduction

- Language usually contains infinite number of strings.
- We cannot tabulate each and every string to represent the language, therefore like automata, grammar is also a mathematical model of representing a language, using which we can generate the entire language.
- Therefore, a grammar is usually thought of as a language generator.

KNOWLEDGEGATE

Formal Grammar

A phrase-structure grammar (or simply a grammar) is a 4-tuple (V_N, ∑, P, S), where

- V_N is a finite nonempty set whose elements are called variables,
- ∑ is a finite nonempty set whose elements are called terminals, $V_N \cap \sum = \Phi$.
- S is a special variable (i.e., an element of V_N ($S \in V_n$)) called the start symbol. Like every automaton has exactly one initial state, similarly every grammar has exactly one start symbol.
- P is a finite set whose elements are $\alpha \rightarrow \beta$. where α and β are strings on $V_N \cup \sum$. α has at least one symbol from V_N, the element of P are called productions or production rules or rewriting rules. $\{\Sigma \cup V_n\}^*$ some writer refers it as total alphabet

KNOWLEDGEGATE

For a formal valid production

$\alpha \rightarrow \beta$

$\alpha \in \{\Sigma \cup V_n\}^* V_n \{\Sigma \cup V_n\}^*$

$\beta \in \{\Sigma \cup V_n\}^*$

KNOWLEDGEGATE

If $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1, S \rightarrow \Lambda\}, S)$, find $L(G)$.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Some points to note about productions

- Reverse substitution is not permitted. For example, if S → AB is a production, then we can replace S by AB but we cannot replace AB by S.
- No inversion operation is permitted. For example, if S → AB is a production, it is not necessary that AB →S is a production.

KNOWLEDGEGATE

Defining a language by grammar

- The concept of defining a language using grammar is, starting from a start symbol using the production rules of the grammar any time, deriving the string. Here every time during derivation a production is used as its LHS is replaced by its RHS, all the intermediate stages(strings) are called sentential forms. The language formed by the grammar consists of all distinct strings that can be generated in this manner.

$$L(G) = \{w \mid w \in \sum^*, S \rightarrow^* W\}$$

- $\rightarrow^*$(reflexive, transitive closure) means from s we can derive w in zero or more steps

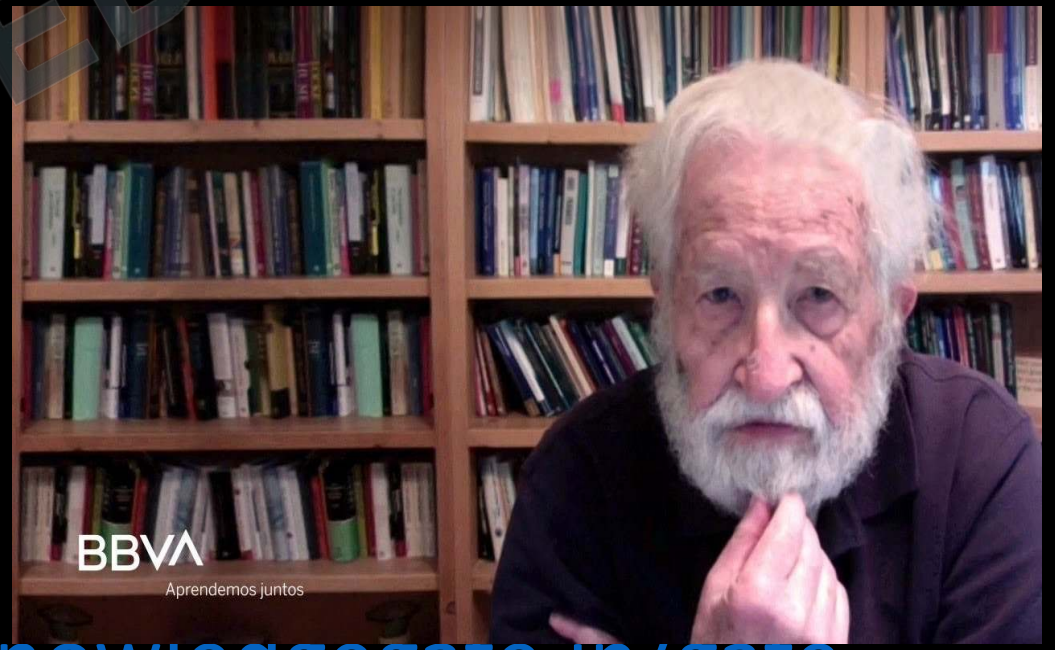
KNOWLEDGEGATE

- Using same idea, we do processing of natural languages in computers, Actively used in compliers
- L (G) is the set of all terminal strings derived from the start symbol S.
- G_1 and G_2 are equivalent if L (G_1) = L (G_2).

KNOWLEDGEGATE

Chomsky Classification of Languages

- Chomsky classified the grammars into four types in terms of productions (types 0-3).
- This hierarchy of grammars was described by Noam Chomsky in 1956. From type 0 to type 3, we will be putting more and more restrictions.
- We will see that more restrictive is grammar easy will be the language and more liberal is the grammar difficult will be the language. Based on the production rules of the grammar, we can classify the formal grammar into four types, based on which we generate different languages.



KNOWLEDGEGATE

- **Avram Noam Chomsky** (born December 7, 1928) is an American linguist, philosopher, cognitive scientist, historian, social critic, and political activist. Sometimes called "the father of modern linguistics",

- Chomsky is also a major figure in analytic philosophy and one of the founders of the field of cognitive science. He is Laureate Professor of Linguistics at the University of Arizona and Institute Professor Emeritus at the Massachusetts Institute of Technology (MIT), and is the author of more than 150 books on topics such as linguistics, war, politics, and mass media. Ideologically, he aligns with anarcho-syndicalism and libertarian socialism.

- Born to Jewish immigrants in Philadelphia, Chomsky developed an early interest in anarchism from alternative bookstores in New York City. He studied at the University of Pennsylvania. During his postgraduate work in the Harvard Society of Fellows, Chomsky developed the theory of transformational grammar for which he earned his doctorate in 1955. That year he began teaching at MIT, and in 1957 emerged as a significant figure in linguistics with his landmark work *Syntactic Structures*, which played a major role in remodeling the study of language.

- From 1958 to 1959 Chomsky was a National Science Foundation fellow at the Institute for Advanced Study. He created or co-created the universal grammar theory, the generative grammar theory, the Chomsky hierarchy, and the minimalist program. Chomsky also played a pivotal role in the decline of linguistic behaviorism, and was particularly critical of the work of B. F. Skinner.

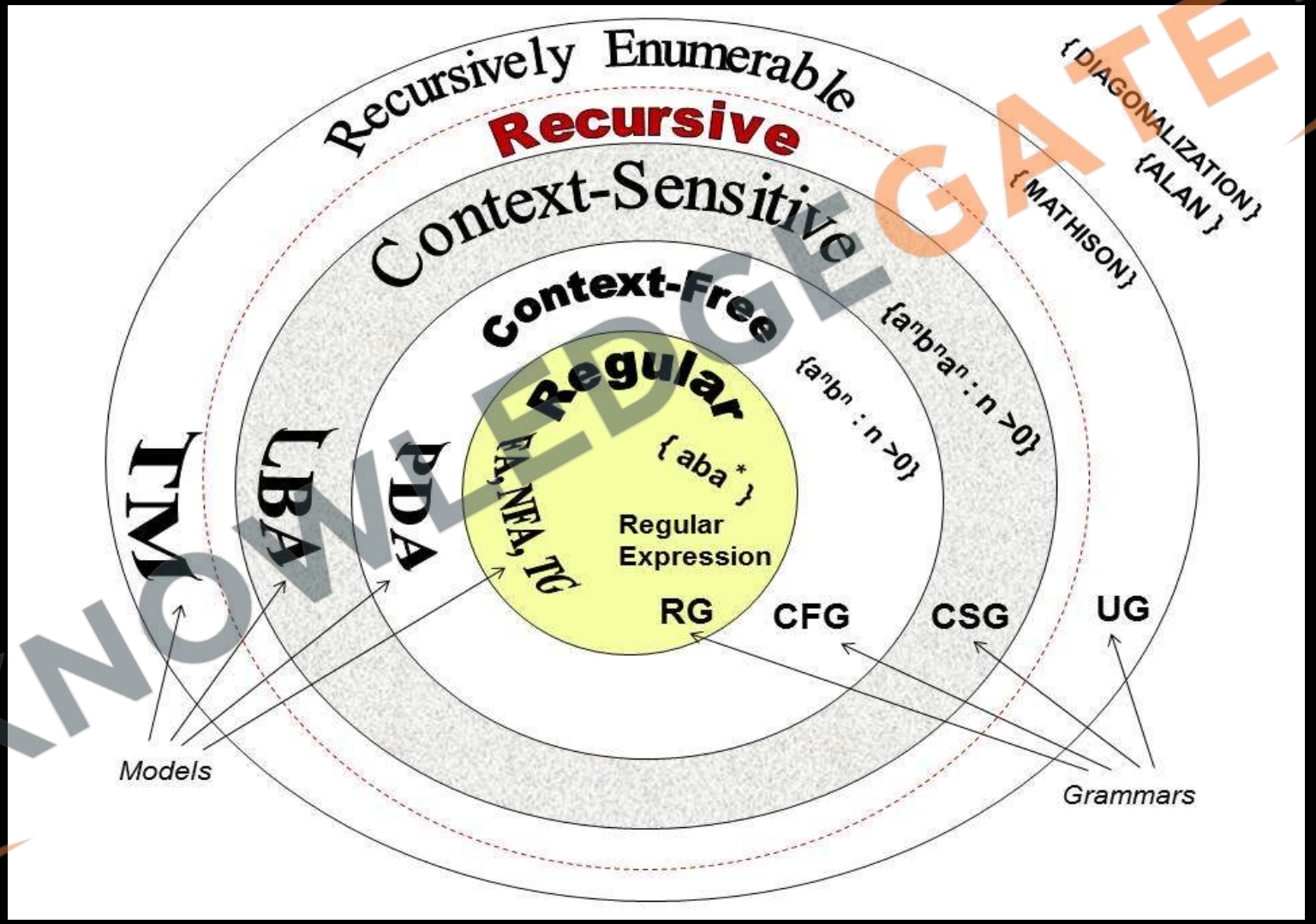
KNOWLEDGEGATE

- An outspoken opponent of U.S. involvement in the Vietnam War, which he saw as an act of American imperialism, in 1967 Chomsky rose to national attention for his anti-war essay "The Responsibility of Intellectuals".
- Associated with the New Left, he was arrested multiple times for his activism and placed on President Richard Nixon's Enemies List. While expanding his work in linguistics over subsequent decades, he also became involved in the linguistics wars. In collaboration with Edward S. Herman, Chomsky later articulated the propaganda model of media criticism in *Manufacturing Consent* and worked to expose the Indonesian occupation of East Timor.
- His defense of freedom of speech, including Holocaust denial, generated significant controversy in the Faurisson affair of the 1980s. Since retiring from MIT, he has continued his vocal political activism, including opposing the 2003 invasion of Iraq and supporting the Occupy movement. Chomsky began teaching at the University of Arizona in 2017.
- One of the most cited scholars alive, Chomsky has influenced a broad array of academic fields. He is widely recognized as having helped to spark the cognitive revolution in the human sciences, contributing to the development of a new cognitivistic framework for the study of language and the mind.
- In addition to his continued scholarship, he remains a leading critic of U.S. foreign policy, neoliberalism and contemporary state capitalism, the Israeli–Palestinian conflict, and mainstream news media. Chomsky and his ideas are highly influential in the anti-capitalist and anti-imperialist movements.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

LANGUAGES AND AUTOMATA

- Following are the machines that accepts the following grammars.



KNOWLEDGE GATE

Type 0 Grammar

- Also known as Unrestricted Grammar, phase structured grammar, recursively enumerable grammar used to generate recursive enumerable language which is accepted by a Turing machine.
- A type 0 grammar is without any restrictions.
- No restriction on the production rule, that is if there is a production from

$$\alpha \rightarrow \beta$$

$$\alpha \in \{\Sigma \cup V_n\}^* V_n \{\Sigma \cup V_n\}^*$$

$$\beta \in \{\Sigma \cup V_n\}^*$$

KNOWLEDGEGATE

Type 1 Grammar

- Also known as case sensitive Grammar, length increasing grammar, non-contracting grammar, used to generate context sensitive language which is accepted by a linear bounded automaton.

$\alpha A \beta \rightarrow \alpha \delta \beta$

$\alpha , \beta \in \{\Sigma \cup V_n\}^*$ $A \in V_n$ $\delta \in \{\Sigma \cup V_n\}^+$

or

$\alpha \rightarrow \beta$

$\alpha \in \{\Sigma \cup V_n\}^* Vn \{\Sigma \cup V_n\}^*$

$\beta \in \{\Sigma \cup V_n\}^+$

$|\alpha| <= |\beta|$

[KNOWLEDGEGATE]

- As from the rule we can understand that we cannot have null production, in order to solve that problem, Production S→∈, is allowed if S do not appear on the right-hand side of the production.

- A grammar is called type 1 or context-sensitive or context dependent if all its productions are type 1 productions.

- Very difficult to have a parse tree

- FORTRAN, PL1 with CGF we use STD

KNOWLEDGEGATE

Type 2 Grammar

- Also known as Context Free Grammar, which will generate context free language that will be accepted by push down automata. (NPDA default case)

- if there is a production, from

$$\alpha \rightarrow \beta$$

$$\alpha \in V_n \qquad |\alpha| = 1$$

$$\beta \in \{\Sigma \cup V_n\}^*$$

- In other words, the L.H.S. has no left context or right context.

- A grammar is called a type 2 grammar if it contains only type 2 productions.

- Eg ALGOL 60, PASCAL

[KNOWLEDGEGATE]

Type 3 Grammar

- Used to generate Regular Grammar which will generate regular language which will be accepted by finite machine.
- Regular grammar can be of two types either left linear or right linear.
- Left regular grammar, support two types of production

$A \rightarrow a / Ba$

$A, B \in V_n \quad |A| = |B| = 1$

$a \in \sum^*$

- Right regular grammar

$A \rightarrow a / aB$

$A, B \in V_n \quad |A| = |B| = 1$

$a \in \sum^*$

- however, if left-linear rules and right-linear rules are combined, the language need no longer be regular.

KNOWLEDGEGATE

Q Consider the following grammar and identify it's language?

S → aAb

A → aB / b

B → c

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the following grammar and identify it's language?

S → AB / Bb

A → b / c

B → d

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the following grammar and identify it's language?

$S \rightarrow aSb / \in$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the following grammar and identify it's language?

S → aA / abS

A → bS / b

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the following grammar and identify it's language?

$S \rightarrow aAB$

$A \rightarrow aA \mid \in$

$B \rightarrow b$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the following grammar and identify it's language?

$S \rightarrow AB$

$A \rightarrow aA \mid \in$

$B \rightarrow bB \mid b$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the following grammar and identify it's language?

$S \rightarrow aSa / bSb / \in$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a grammar that generates all strings over the alphabet ∑ = {a, b}, where every accepted string ‘w’ starts with substring s = abb

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a grammar that generates all strings over the alphabet ∑ = {a, b}. where every accepted string ‘w’ ends with substring s = bab

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a grammar that generates all strings over the alphabet ∑ = {a, b}. where every accepted string ‘w’ contains sub string s = aba

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a grammar that generates all strings over the alphabet ∑ = {a, b} such that every accepted string start and end with a.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a grammar that generates all strings over the alphabet ∑ = {a, b}, such that every string ‘w’ accepted must be like

i) |w| = 3 **ii)** |w|<=3 **iii)** |w|>=3

KNOWLEDGEGATE

Q Design a grammar that generates all strings over the alphabet ∑= {a, b}, such that every string ‘w’ where |W| = 0(mod 3)?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a grammar that generates all strings over the alphabet ∑= {a, b}, such that every string ‘w’ where |W| = 3(mod 4)?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Regular Grammar
to
Regular Expression

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → 01S / 01

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → 0011S / 01 / 10

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → 01A / B11

A →011A / 01

B → 101B / 11

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → 011A / 101B

A →110A / 00

B → 11B / S

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Regular Grammar to (Finite Automata)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → 01S / 1

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → 011S / 01

KNOWLEDGEGATE

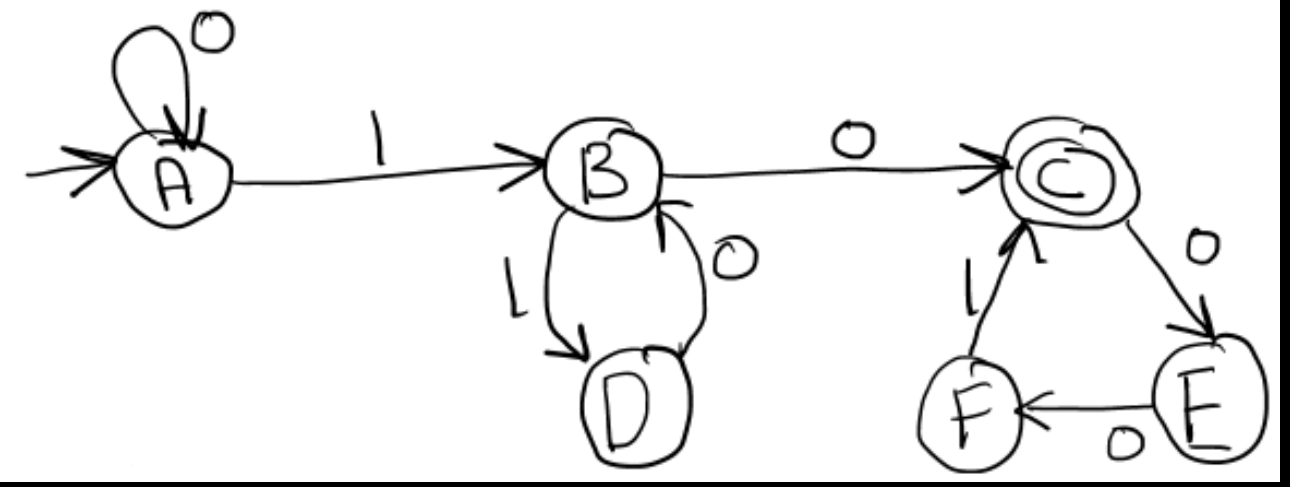
http://www.knowledgegate.in/gate

S → 001S / 10A

A → 101A / 0 / 1

KNOWLEDGEGATE

http://www.knowledgegate.in/gate



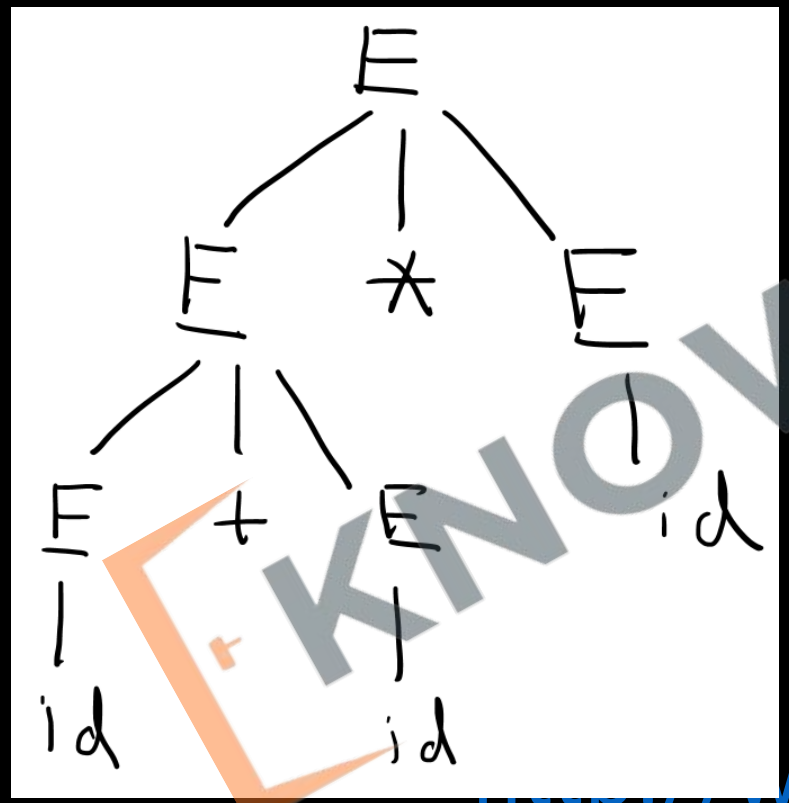
KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Derivation:** - The process of deriving a string is known as derivation.

- **Derivation/ Syntax/ Parse Tree**: - The graphical representation of derivation is known as derivation tree.

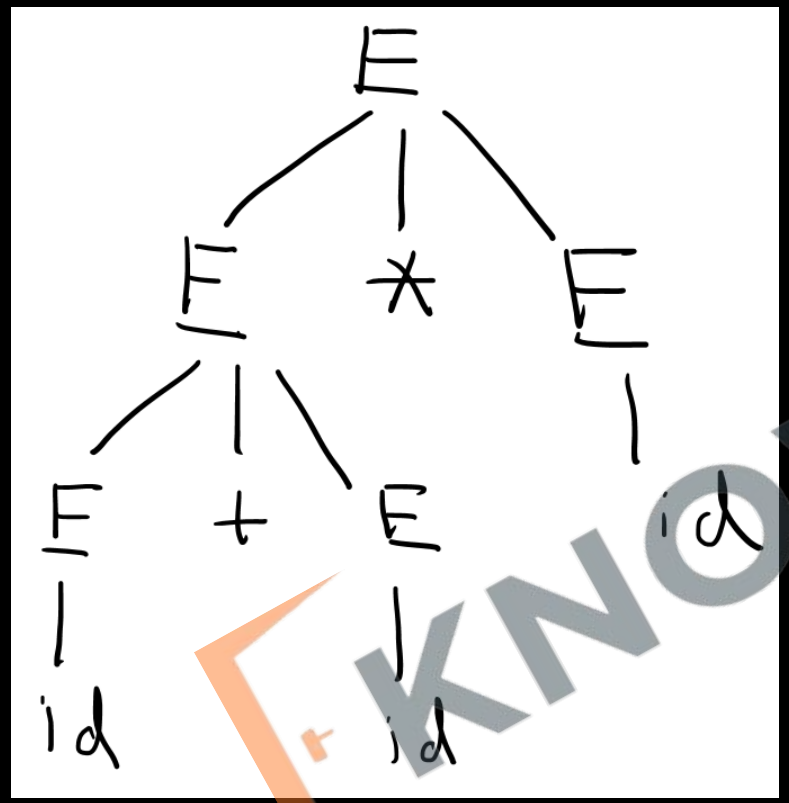
E → E + E / E * E / E = E / id



KNOWLEDGEGATE

- **Sentential form:** - Intermediate step involve in the derivation is known as sentential form.

E → E + E / E * E / E = E / id

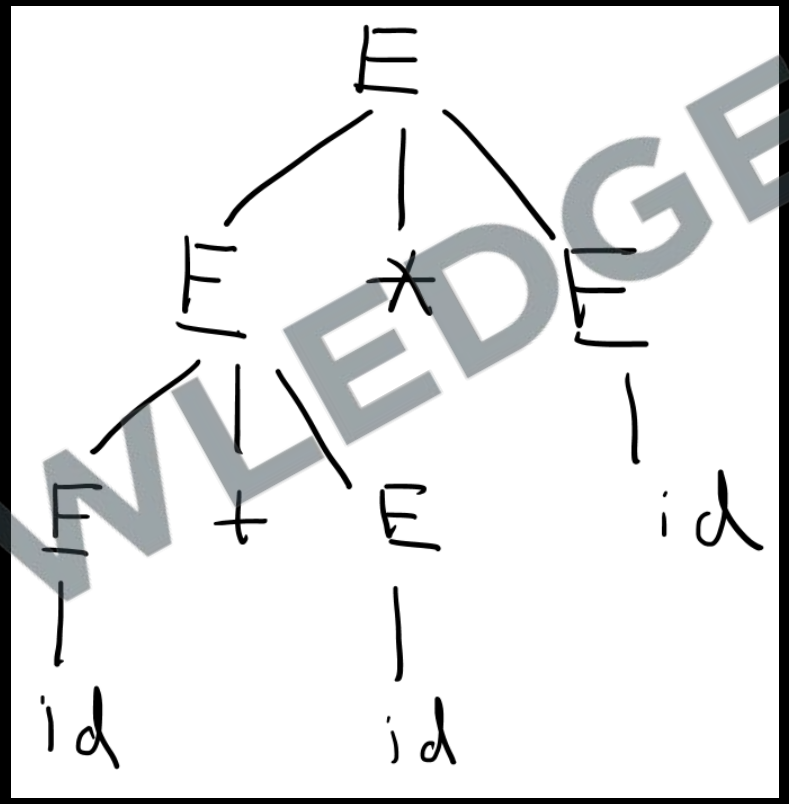


Sentential Form
E
E*E
E+E*E
ID+E*E
ID+ID*E
ID+ID*ID

KNOWLEDGEGATE

- **Left most derivation**: - the process of construction of parse tree by expanding the left most non terminal is known as LMD and the graphical representation of LMD is known as LMDT (left most derivation tree)

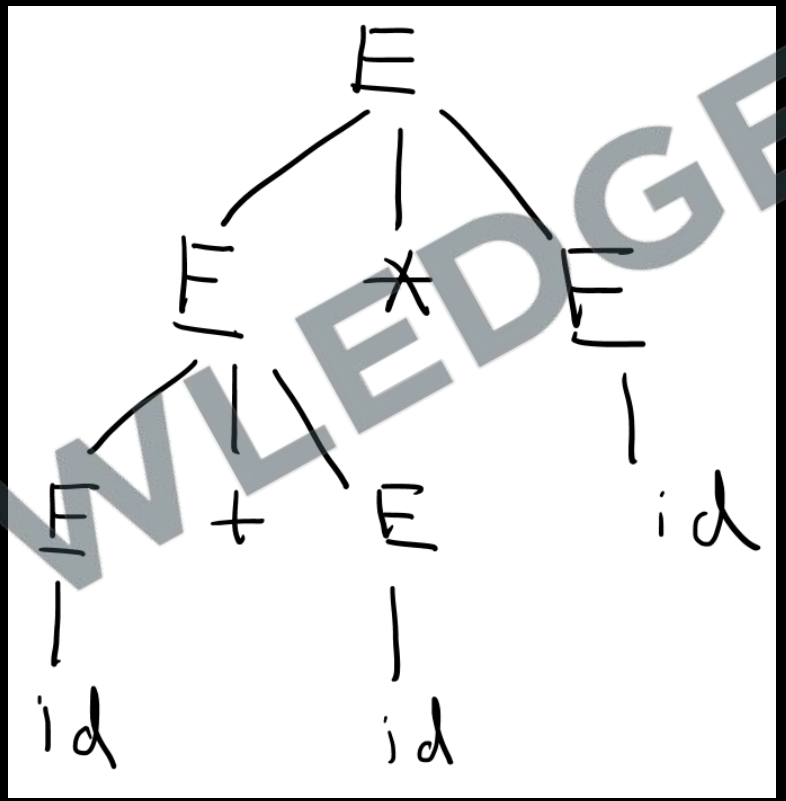
	LMD
E → E + E	E
E→ E * E	E*E
E→ E = E	E+E*E
E→ id	ID+E*E
E→ id	ID+ID*E
E→ id	ID+ID*ID



KNOWLEDGEGATE

- **Right most derivation**: - the process of construction of parse tree by expanding the right most non terminal is known as RMD and the graphical representation of RMD is known as RMDT (right most derivation tree)

	RMD
E → E + E	E
E→ E * E	E+E
E→ E = E	E+E*E
E→ id	E+E*ID
E→ id	E+ID*ID
E→ id	ID+ID*ID



KNOWLEDGEGATE

Ambiguous grammar: - The grammar CFG is said to be ambiguous if there are more than one derivation tree for any string i.e. if there exist more than one derivation tree (LMDT or RMDT), the grammar is said to be ambiguous.

S → aS/Sa/a

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Unambiguous grammar: - The CFG is said to be unambiguous if there exist only one parse tree for every string i.e. if there exist only one LMDT or RMDT, then the grammar is unambiguous e.g.

S → aSb/ab

KNOWLEDGEGATE

- Some CFL are called inherently ambiguous means there exist no unambiguous CFG to generate the corresponding CFL, proved by Rohit Parikh 1961,

- $\{a^n b^m c^m d^n\} \cup \{a^n b^n c^m d^m\}$

KNOWLEDGEGATE

- Grammar which is both left and right recursive is always ambiguous, but the ambiguous grammar need not be both left and right recursive.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Simplification or Minimization of CFG

- The reason we simplify CFG to make it more efficient and compiler friendly.
- The process of deleting and eliminating of useless symbols, unit production and null production is known as simplification of CFG.

KNOWLEDGEGATE

Removal of Null or Empty productions

- The production of the form A → ∈ is known as null production of empty production, here we try to remove them by replacing equivalent derivation.

S → AbB

A → a / ∈

B → b / ∈

KNOWLEDGEGATE

S → AB

A → a / ∈

B → b / ∈

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

$S \rightarrow aSb / \in$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Removal of unit productions

- The production of the form A → B where A, B ∈ Vn, |A| = |B| = 1, is known as unit production.

S → Aa

A → a / B

B → d

KNOWLEDGEGATE

S → aAb

A → B / a

B → C / b

C → D / c

D → d

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

$S \rightarrow aSb / \in$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Removal of useless symbols

- The variables which are not involved in the derivation of any string is known as useless symbol.
- Select the Variable that cannot be reached from the start symbol of the grammar and remove them along with their all production.

S → aAB
A → a
B → b
C → d

- Select variable that are reachable from the start symbol but which does not derive any terminal, remove them along with their productions

S → aA / aB
A → b

KNOWLEDGEGATE

S → aAB / bA / aC

A → aB / b

B → aC / d

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Chomsky Normal Form

- The Grammar G is said to be in Chomsky Normal Form, if every production is in the form

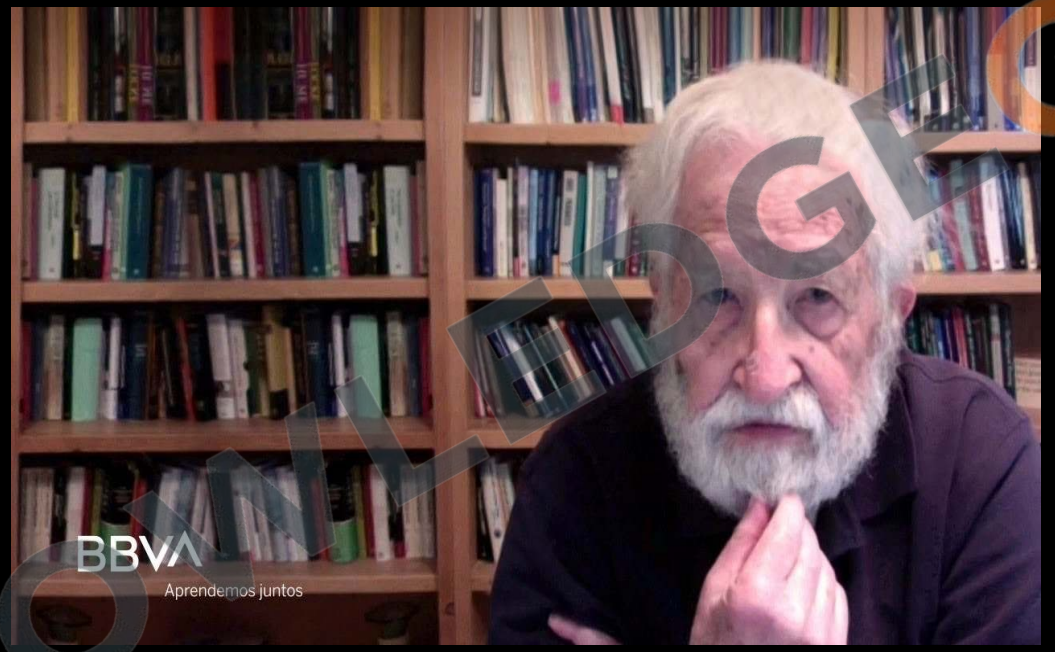
$A \rightarrow BC / a$

$B, C \in V_n$

$a \in \Sigma$

KNOWLEDGEGATE

- **Avram Noam Chomsky** (born December 7, 1928) is an American linguist, philosopher, cognitive scientist, historian, social critic, and political activist. Sometimes called "the father of modern linguistics",



KNOWLEDGEGATE

$S \rightarrow aSb / ab$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → aAb / bB

A → a / b

B → b

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- if CFG is in CNF, then for a derivation of string w, with length we need exactly 2n -1 production. |w| = n, number of sentential forms will be 2n -1

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Greiback Normal Form

- The Grammar G is said to be in Greiback Normal Form, if every production is in the form

$$A \rightarrow a\alpha$$

$A \in V_n$

$a \in \Sigma$

$\alpha \in V_n^*$

KNOWLEDGEGATE

- **Sheila Adele Greibach** (born 6 October 1939 in New York City) is a researcher in formal languages in computing, automata, compiler theory and computer science.

KNOWLEDGEGATE

S → aSb / ab

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → aAb / bB

A → a / b

B → b

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- if CFG is in GNF, then for a derivation of string w, with length we need exactly n production. |w| = n, number of sentential forms will be n

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Chapter-4 (Push Down Automata and Properties of Context Free Languages)**: Nondeterministic Pushdown Automata (NPDA)- Definition, Moves, A Language Accepted by NPDA, Deterministic Pushdown Automata(DPDA) and Deterministic Context free Languages(DCFL), Pushdown Automata for Context Free Languages, Context Free grammars for Pushdown Automata, Two stack Pushdown Automata, Pumping Lemma for CFL, Closure properties of CFL, Decision Problems of CFL, Programming problems based on the properties of CFLs.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

CONTEXT-FREE LANGUAGES AND PUSH DOWN AUTOMATA

- Context-free languages are applied in parser design.
- They are also useful for describing block structures in programming languages.

KNOWLEDGEGATE

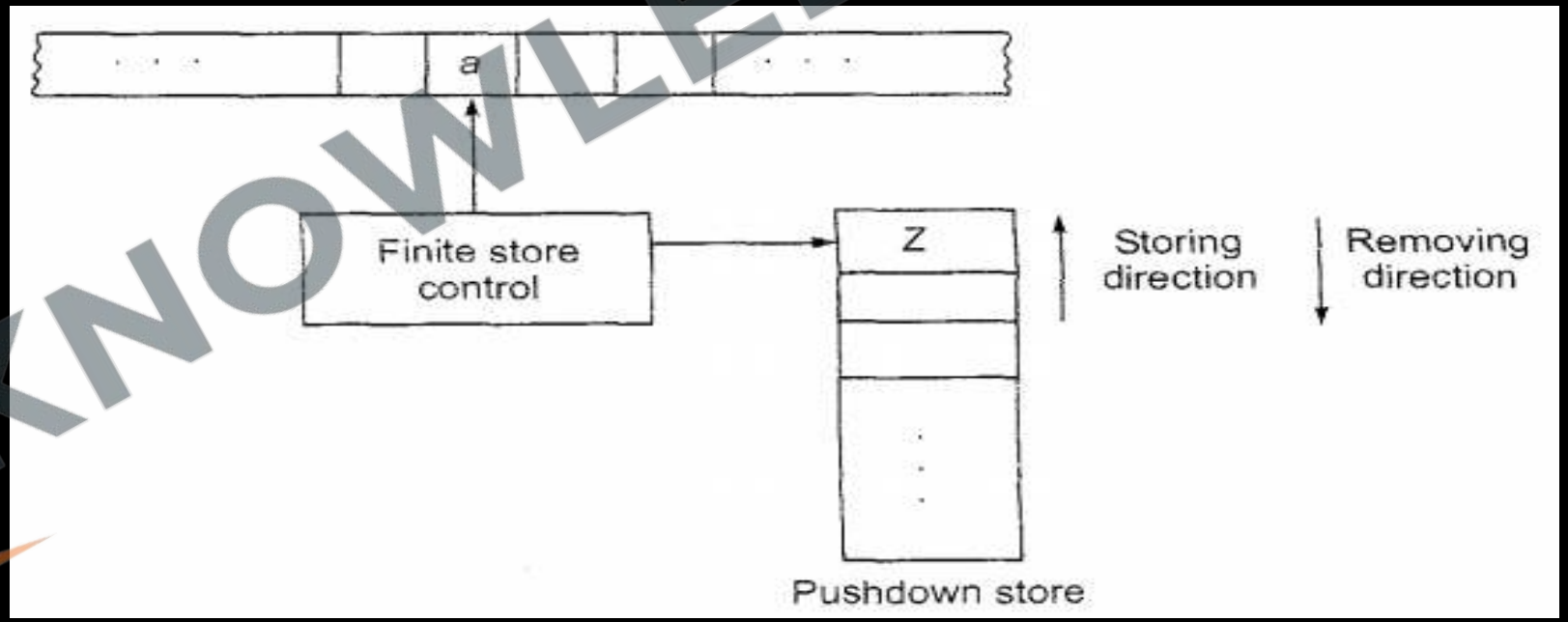
Push Down Automata

- We Already understand the limitation of finite automata, that it cannot do the infinite comparison between the symbols.
- Let us consider L = {a^n b^n | n >= 1}. This is not regular, as it has to remember the number of a's in a string and so it will require an infinite number of states, which is logically not possible.
- This difficulty can be avoided by adding an auxiliary memory in the form of a 'stack'. The reason we choose stack because it is the simplest memory possible.
- This type of arrangement where ***a finite automaton has a stack leads to the generation of a pushdown automaton.***

KNOWLEDGE GATE

BLOCK DIAGRAM OF PDA

- Finite control unit is also called as memory unit it is static and limited. So to process the i/p string if the static memory is not sufficient then we can use the stack.
- i/p tape is divided into cells where is cell is capable of holding one symbol at a time. At stack of infinite size, which support three operations push, pop and skip.
- The accepting power of a pda is more than that of finite automata and less than that of linear bounder automata
- The power of non-deterministic pda is more than the power of deterministic pda.



KNOWLEDGEGATE

Formal Definition of DPDA

A DPDA is a 7-tuple, namely $(Q, \sum, \Gamma, \delta, q_0, Z_0, F)$, where

(i) Q – is a finite nonempty set of states,

(ii) $\sum$ – is a finite nonempty set of input symbols,

(iii) Γ – is a finite nonempty set of pushdown symbols,

(iv) q_0 – is a special state called the initial state,

(v) Z_0 – is a special pushdown symbol called the initial symbol on the pushdown store.

(vi) F – is a set of final states, a subset of Q and

(vii) δ – is a transition function from $Q \times (\sum \cup \{\in\}) \times \Gamma$ to the set of finite subsets of $Q \times \Gamma^*$.

KNOWLEDGEGATE

Representation of States

(1) PUSH – one symbol can be inserted into the stack at one time.

$\delta(q_i, a, z_0) = (q_j, az_0)$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Representation of States

(2) POP – one symbol can be deleted from the stack at one time.

$\delta(q_i, a, z_0) = (q_j, \varepsilon)$

KNOWLEDGEGATE

Representation of States

(3) SKIP – IT means no stack operation, status of the stack will remain same, before a after the operation

$$\delta(q_i, a, z_0) = (q_j, z_0)$$

note- if pda perform a push or a pop operation at least one's during processing of string than we say that pda is using the stack.

KNOWLEDGEGATE

Instantaneous Description (ID)

- An **instantaneous description (ID)** is (q, *x*, α), where q ∈ Q, *x* ∈ ∑* and α ∈ Γ*.
- An initial ID is **(q_0, *x*, Z_0)**, this means that initially the pda is in the initial state q_0, the input string to be processed is *x* and the PDS has only one symbol, namely Z_0.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

ACCEPTANCE BY PDA

- There is no change in the language acceptance capability of the pda either we accept by final state of empty stack.

KNOWLEDGEGATE

Q Design a Deterministic Push Down Automata for $\{a^n b^n \mid n >= 1\}$?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a Deterministic Push Down Automata for $\{a^n b^{2n} \mid n >= 1\}$?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a PDA for $\{w\ c\ w^r \mid w \in (a, b)^*\}$?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Construct PDA that accepts $L = \{|w|_{a=b} \mid w \in (a, b)^*\}$?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Non- Deterministic PDA

- Non- Deterministic PDA can also be defined using 7 tuples.
- $\delta: Q \times \{\Sigma \cup \in\} \times \Gamma \rightarrow 2^{(Q \times \Gamma^*)}$
- i.e. on a given input symbol and stack symbol a NPDA can move to more than one state.
- Rest all other tuples are same as DPDA.

KNOWLEDGEGATE

Q Construct pda that accepts a language L = {w w^r | w € $(a, b)^*$}?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Decision properties

Following properties are decidable in case a CFL. Here we will use Grammar model to proof decision properties.

i) Emptiness

ii) Non-emptiness

iii) Finiteness

iv) Infiniteness

v) Membership

KNOWLEDGEGATE

Q consider the following CFG and identify which of the following CFG generate Empty language?

S → aAB / Aa
A → a

S → aAB
A → a / b

S → aAB / aB
A → aBb
B → aA

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q consider the following CFG and identify which of the following CFG generate Finite language?

S → SS / AB
A → BC / a
B → CC / b

S → AB
A → B / a

S → AB
A → BC / a
B → CC / b
C → AB

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q consider the following CFG and check out the membership properties?

S → AB / BB

A → BA / AS / b

B → AA / SB / a

w_1 = aba

w_2 = abaab

w_3 = abababba

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

CYK algorithm

- In computer science, the **Cocke–Younger–Kasami algorithm** (alternatively called **CYK**, or **CKY**) is a parsing algorithm for context-free grammars, named after its inventors, John Cocke, Daniel Younger and Tadao Kasami. It employs bottom-up parsing and dynamic programming.

- The standard version of CYK operates only on context-free grammars given in Chomsky normal form (CNF). However any context-free grammar may be transformed to a CNF grammar expressing the same language (Sipser 1997).

- The importance of the CYK algorithm stems from its high efficiency in certain situations. Using Big O notation, the worst case running time of CYK is $O(n^3 .|G|)$, Where n is the length of the parsed string and |G| is the size of the CNF grammar G.

- This makes it one of the most efficient parsing algorithms in terms of worst-case asymptotic complexity, although other algorithms exist with better average running time in many practical scenarios.

[KNOWLEDGEGATE]

Following properties are Undecidable in case a CFL

i) Equality

ii) Ambiguity

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

	RL	DCFL	CFL	CSL	RS	RES
Emptiness	Y	Y	Y	X	N	N
Non-Emptiness	Y	Y	Y	X	N	N
Finiteness	Y	Y	Y	X	N	N
Infiniteness	Y	Y	Y	X	N	N
Membership	Y	Y	Y	X	Y	N
Equality	Y	N	N	X	N	N
Ambiguity	Y	N	N	X	N	N
Σ*	Y	N	N	X	N	N
Halting	Y	Y	Y	X	Y	N

KNOWLEDGEGATE

Closure Properties of Deterministic Context Free Languages

- **Deterministic Context Free Languages** are closed under following operations
  - Complement
  - Intersection with regular set
  - Inverse Homeomorphism

- **Deterministic Context Free Languages** are not closed under following operations
  - Union
  - Concatenation
  - Kleen closure
  - homomorphism
  - Substitution
  - Reverse operator
  - Intersection

KNOWLEDGEGATE

Closure Properties of Context Free Languages

- **Context Free Languages** are closed under following operations
  - Union
  - Concatenation
  - Kleen Closure
  - Substitution
  - Homomorphism
  - Inverse Homomorphism
  - Reverse Operator
  - Intersection with regular set

- **Context Free Languages** are not closed under following operations
  - Intersection
  - Complement
  - Symmetric Difference

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

	RL	DCFL	CFL	CSL	RS	RES
Union	Y	N	Y	Y	Y	Y
Intersection	Y	N	N	Y	Y	Y
Complement	Y	Y	N	Y	Y	N
Set Difference	Y	N	N	Y	Y	N
Kleene Closure	Y	N	Y	Y	Y	Y
Positive Closure	Y	N	Y	Y	Y	Y
Concatenation	Y	N	Y	Y	Y	Y
Intersection with regular set	Y	Y	Y	Y	Y	Y
Reverse	Y	Y	Y	Y	Y	Y
Subset	N	N	N	N	N	N

KNOWLEDGEGATE

	RL	DCFL	CFL	CSL	RS	RES
Homomorphism	Y	N	Y	N	N	Y
∈ Free Homomorphism	Y	N	Y	Y	Y	Y
Inverse Homomorphism	Y	Y	Y	Y	Y	Y
Substitution	Y	N	Y	N	N	Y
∈ Free Substitution	Y	N	Y	Y	Y	Y
Quotient with regular set	Y	Y	Y	N	Y	Y

KNOWLEDGEGATE

- **Chapter-5 (Turing Machines and Recursive Function Theory):** Basic Turing Machine Model, Representation of Turing Machines, Language Acceptability of Turing Machines, Techniques for Turing Machine Construction, Modifications of Turing Machine, Turing Machine as Computer of Integer Functions, Universal Turing machine, Linear Bounded Automata, Church's Thesis, Recursive and Recursively Enumerable language, Halting Problem, Post's Correspondance Problem, Introduction to Recursive Function Theory.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Turing Machine

- *The Church-Turing thesis states that any algorithmic procedure that can be carried out by human beings/computer can be carried out by a Turing machine.*

- It has been universally accepted by computer scientists that the Turing machine provides an ideal theoretical model of a computer.

KNOWLEDGEGATE

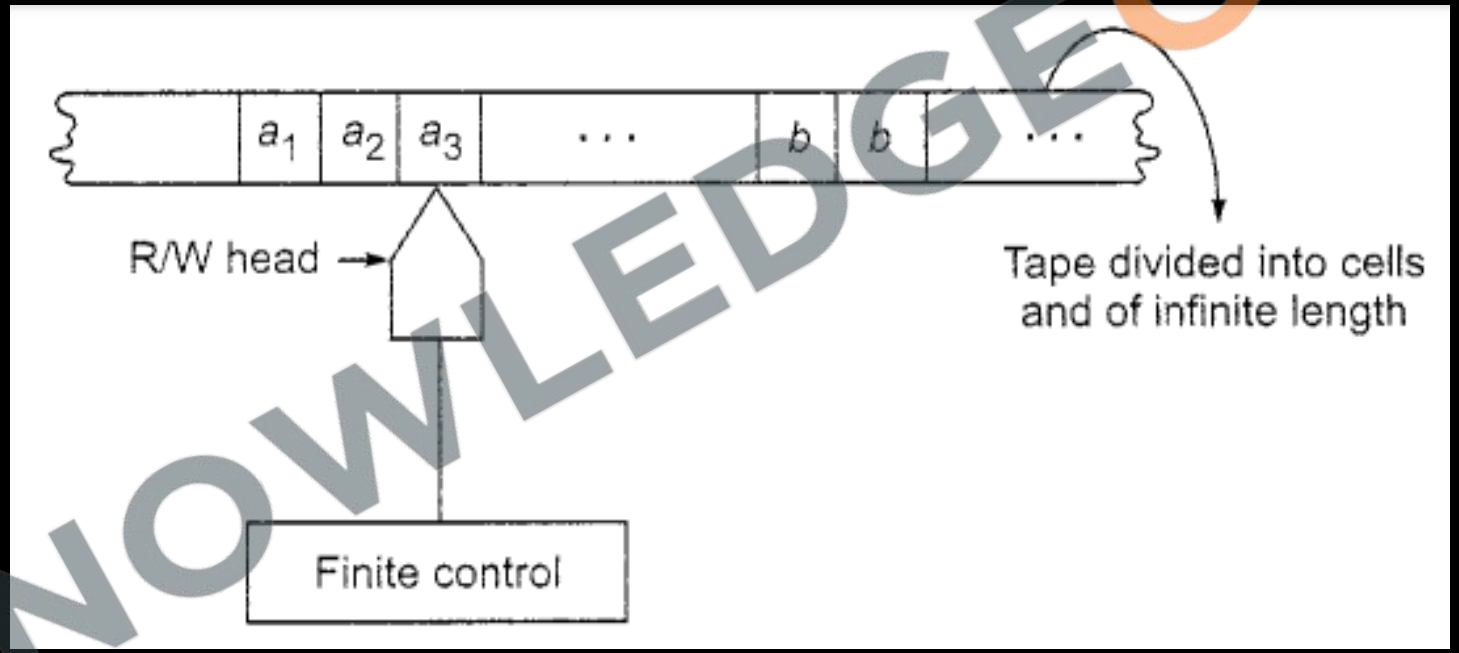
- Turing machines are useful in several ways:
- As an automaton, the Turing machine is the most general model.
- It accepts type-0 languages.
- It can also be used for computing functions.
- Turing machines are also used for determining the undecidability of certain languages and measuring the space and time complexity of problems.

KNOWLEDGEGATE

Components of Turing Machine

Infinite Tape

- An input infinite tape can be, used as an input buffer and also as a member element.
- It is random accessible. It is of infinite length divided into cells and each cell is capable of holding only one tape symbol.
- It is also called 2-way infinite tape.

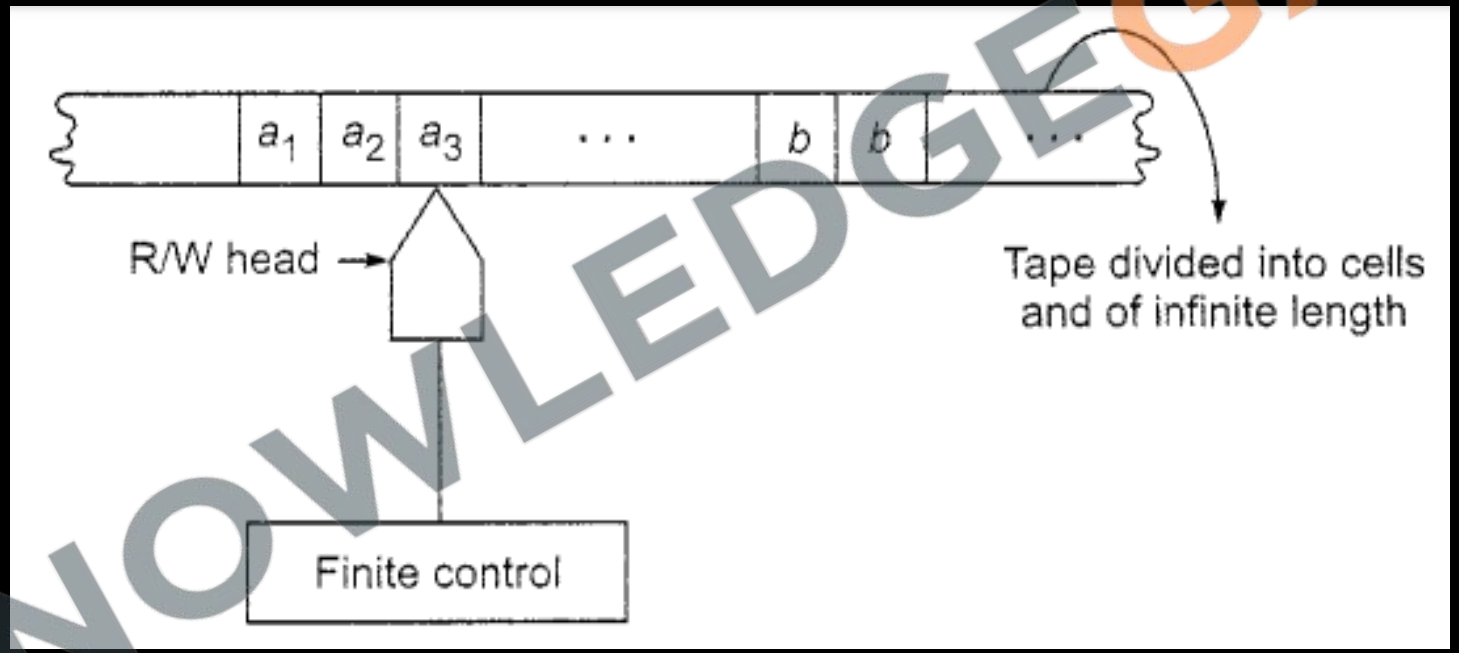


KNOWLEDGEGATE

Components of Turing Machine

Read-Write Buffer

- Read the data from the tape and can also write the data over the tape. After reading the symbol from the tape, the read/write header moves to exactly one cell either in left or right direction.

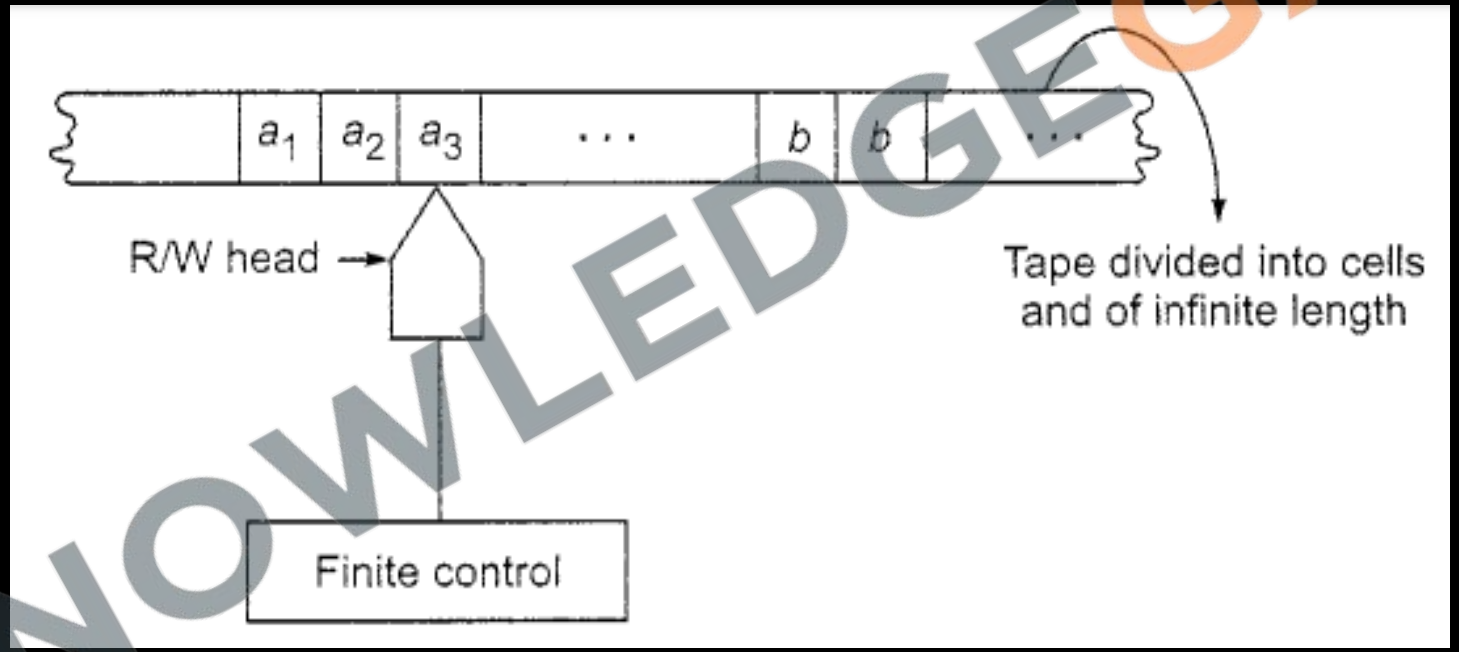


KNOWLEDGEGATE

Components of Turing Machine

Finite Control Unit

- As per the control unit, the transitions will take place, and will finally lead to some output.



KNOWLEDGEGATE

FORMAL DEFINITION

A Turing machine *M* is a 7-tuple, namely (Q, ∑, Γ, δ, q_0, b, F), where

- Q is a finite nonempty set of states.
- Γ is a finite nonempty set of tape symbols,
- b ∈ Γ is the blank.
- ∑ is a nonempty set of input symbols and is a subset of Γ and b ∉∑.
- δ is the transition function mapping δ: Q ×𝕃 -> Q× 𝕃 × (L/R). It says that, on providing a tape symbol, from a particular state there will be a transition to another state, with a different or same tape symbol with defining whether next the machine needs to move in left/ right.
- q_0 ∈ Q is the initial state, and
- F ⊆ Q is the set of final states.

KNOWLEDGEGATE

Q Design a Turing machine for $L = \{a^n b^n \mid n >= 1\}$?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

REPRESENTATION OF TURING MACHINES

Three representations:

- Instantaneous descriptions using move-relations.
- Transition table, and
- Transition diagram (transition graph).

KNOWLEDGEGATE

LANGUAGE ACCEPTABILITY BY TURING MACHINES

- A string *w* in ∑* is said to be accepted by a TM(*M*) if after parsing the string w Turing machine must halts on final state.
- $q_0 w \vdash^* \alpha_1 p \alpha_2$ for some p ∈ F and $\alpha_1, \alpha_2 \in \Gamma^*$.
- M does not accept *w* if the machine *M* either halts in a non-accepting state or does not halt and goes into a loop.

KNOWLEDGEGATE

Q Design a Turing machine for $L = \{a^n b^n c^n \mid n >= 0\}$?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a Turing machine for L = {w c w | w ∈ {0, 1}* }?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a Turing machine for addition of two number in unary?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Design a Turing machine for converting unary number into binary number?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Versions of Turing Machine

KNOWLEDGEGATE

- On the basis of determinism of the next transition to a state on a particular input, Turing Machine are divided in two types-
  - Determinism Turing Machine.
  - Non-Deterministic Turing Machine.

- In non-deterministic Turing machine, there can be more than one possible move for a given state and tape symbol, but non-deterministic TM does not add any power.
- ***Every non-deterministic TM can be converted into deterministic TM.***

KNOWLEDGEGATE

Versions of Turing Machine

- There are multiple versions of Turing Machine which are as follows-
- Multi-tape Turing Machine.
  - In multi-tape Turing machine, there can be more than one tape and corresponding head pointers, but it does not add any power to Turing machine.
  - Every multi-tape TM can be converted into single tape TM.
  - Every language accepted by a multi-tape TM is acceptable by some single-tape TM (that is, the standard TM).
- Multi Read/Write head points.
- Multi-dimensional Turing Machine.
- TM with stay option δ: $Q \times \Gamma \rightarrow Q \times \Gamma \times (L/R/S)$.
- TM with one way infinite tape (Semi infinite tape)

KNOWLEDGEGATE

- Offline TM, this TM have a restriction that input can not be changed
- Jumping TM, where it is allowed to take more that one moves in one transaction δ: Q $\times \mathbb{\Gamma}$ -> Q$\times \mathbb{\Gamma} \times$ (L/R) $\times\{n\}$.
- Non erasing TM, (where input can not be converted to blank)
- Always writing TM, (after reading a symbol from tape it must be replaced with other symbol)
- Multidimensional TM δ: Q $\times \mathbb{\Gamma}$ -> Q$\times \mathbb{\Gamma} \times$ (L/R/U/D)
- Multi-head TM
- FA with a Queue
- TM with only 3 states
- Multi-tape Turing Machine with stay option and at most 2 states.
- Non-Deterministic TM δ: Q $\times \mathbb{\Gamma}$ -> $2^{Q\times \mathbb{\Gamma} \times (L/R)}$
- A NPDA with two independent stacks δ: Q $\times(\Sigma \cup \in) \times \mathbb{\Gamma} \times \mathbb{\Gamma}$ -> $2^{Q\times \mathbb{\Gamma}^* \times \mathbb{\Gamma}^*}$

KNOWLEDGEGATE

Q Consider the Turing machine M defined below
Choose the false statement

a) The machine loops on 01

b) The machine does not accept 0000

c) The machine accepts strings ending with a 1

d) The machine accepts strings ending with 10

	0	1	B
$\rightarrow Q_0$	$(Q_0, 0, R)$	$(Q_2, 1, L)$	$(Q_f, -, -)$
Q_1	$(Q_2, 1, L)$	$(Q_1, 1, R)$	$(Q_f, -, -)$
Q_2	$(Q_2, 1, L)$	$(Q_2, 0, L)$	$(Q_f, -, -)$
$*Q_f$	---	---	---

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Halt

- The state of Turing machine, where no transition is defined or required is known as Halt. There are two types of Halt-
  - Final Halt- Machine has been halted on final state, then it is known as Final halt and hence it depicts that machine is accepted.
  - Non-Final Halt-Machine has been halted on non- final state, then it is known as non- final halt and hence the string is rejected.

- After taking an input string, there are three possibilities for Turing Machine
  - It may go to Final halt.
  - It may go to Non- Final halt.
  - It may go into Infinite loop.

- Final Halt and Non- Final Halt have been described above. After taking an input string, if the machine goes to infinite loop, then we can't say whether the string is Accepted/Rejected.

[KNOWLEDGEGATE]

- So, broadly Recursively Enumerable Languages are categorized as-
  - **Recursive Set**- The Language L, which is accepted by Turing Machine, is said to be recursive set, where for all, 'w' that belongs to L, machine will go to final halt, and for all 'w' that does not belongs to L, machine will go to non-final halt. Hence, membership property is defined here.
  - **Recursively Enumerable Set**- The language L, which is accepted by Turing Machine is said to be REL, where for all, 'w' that belongs to L, machine will go to final halt, and for all 'w' that does not belongs to L, machine will either go to non-final halt or infinite loop. Hence, membership property is not defined here.

- If a set and its complement both are recognizable, then the set is decidable.
- There are some sets which are not recognizable (even members can't be identified) because there are more languages than programs.

KNOWLEDGEGATE

Universal Turing Machine

KNOWLEDGEGATE

Universal Turing Machine

- In computer science, a **universal Turing machine** (**UTM**) is a Turing machine that simulates an arbitrary Turing machine on arbitrary input.

- The universal machine essentially achieves this by reading both the description of the machine to be simulated as well as the input to that machine from its own tape.

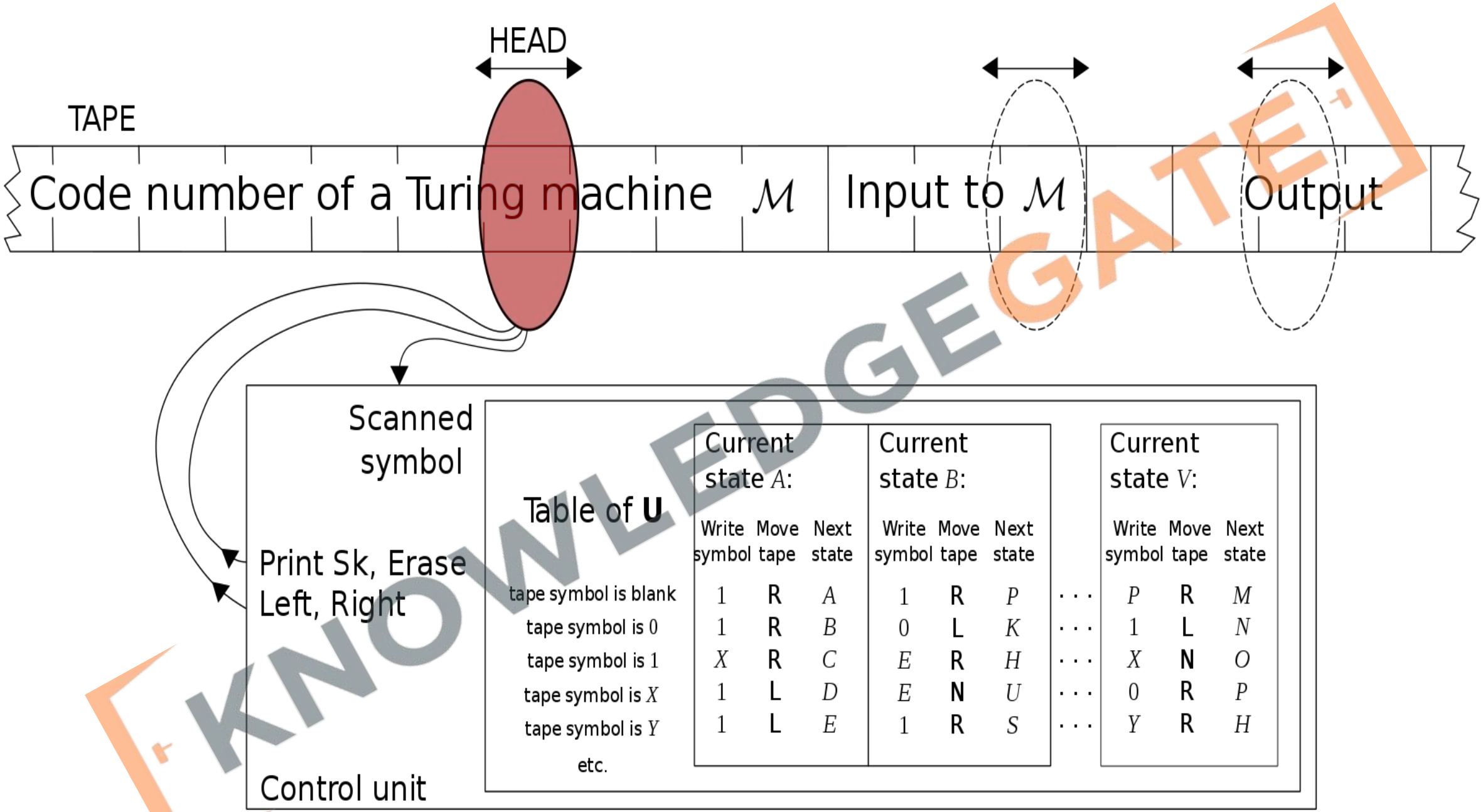
KNOWLEDGEGATE

- Every Turing machine computes a certain fixed partial computable function from the input strings over its alphabet. In that sense it behaves like a computer with a fixed program.

- However, we can encode the action table of any Turing machine in a string. Thus we can construct a Turing machine that expects on its tape a string describing an action table followed by a string describing the input tape, and computes the tape that the encoded Turing machine would have computed. Turing described such a construction in complete detail in his 1936 paper:

- "It is possible to invent a single machine which can be used to compute any computable sequence. If this machine **U** is supplied with a tape on the beginning of which is written the S.D ["standard description" of an action table] of some computing machine **M**, then **U** will compute the same sequence as **M**.

[KNOWLEDGEGATE]



KNOWLEDGEGATE

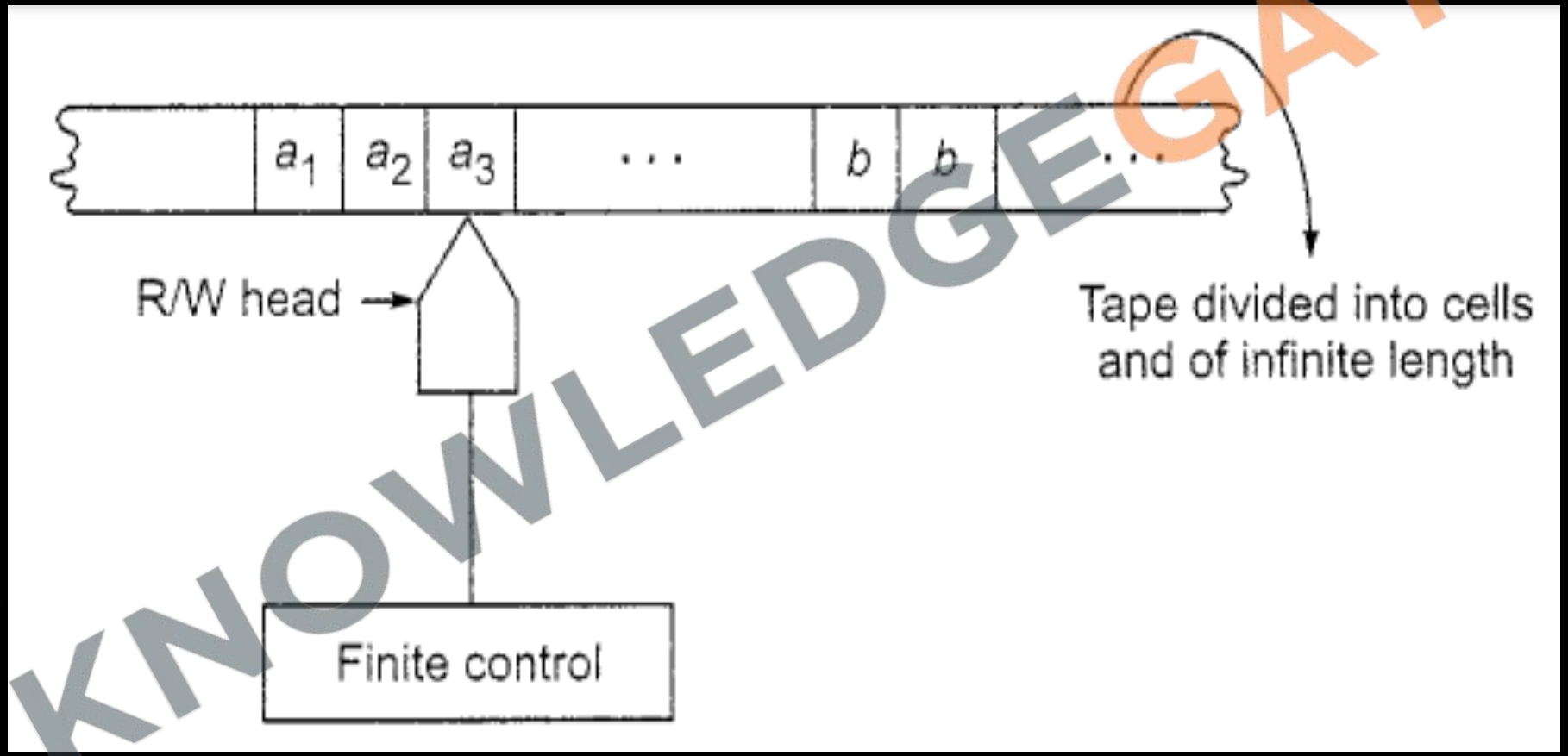
Linear Bounded Automata

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

A Linear Bounded Automaton (LBA) is similar to Turing Machine with property stated below:

- Turing Machine with a bounded finite length of the tape.



KNOWLEDGEGATE

Turing-Church Thesis

- **Concept Origin**: Independently developed by Alan Turing and Alonzo Church in the 1930s, establishing a fundamental principle in computer science about computable functions.

KNOWLEDGE GATE

1.Turing Machines: Turing proposed the concept of a 'Turing machine', a theoretical computing machine that can simulate any algorithm's logic.

2.Church's Lambda Calculus: Church introduced lambda calculus, a formal system for expressing computation based on function abstraction and application.

3.Equivalent Models: The thesis states that what can be computed on a Turing machine can also be computed in Church's lambda calculus, implying both models are equivalent in their computational power.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

The Post Correspondence Problem (PCP)

- is a significant problem in the field of theoretical computer science. It was introduced by Emil Post in 1946 and is known for its undecidability. Here are the main points:
  - **Basic Concept**: The problem involves two lists of words (strings of symbols) over a common alphabet. The challenge is to find a sequence of indices where the concatenation of the words in the first list, in that sequence, is equal to the concatenation of the words in the second list in the same sequence.

- A = {1, 110, 0111}
- B = {111, 001, 11}

[KNOWLEDGEGATE]

- **Undecidability**: The Post Correspondence Problem is known to be undecidable, meaning there is no algorithm that can determine for every instance of the problem whether or not a solution exists. This undecidability is a crucial aspect in the theory of computation, particularly in understanding the limits of algorithmic solvability.

- A = {b, babbb, ba}
- B = {bbb, ba, a}

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Decision properties

- Following properties are decidable in case a RS.
  - Membership

- All properties are undecidable in case of a REL.

KNOWLEDGEGATE

	RL	DCFL	CFL	CSL	RS	RES
Emptiness	Y	Y	Y	X	N	N
Non-Emptiness	Y	Y	Y	X	N	N
Finiteness	Y	Y	Y	X	N	N
Infiniteness	Y	Y	Y	X	N	N
Membership	Y	Y	Y	X	Y	N
Equality	Y	N	N	X	N	N
Ambiguity	Y	N	N	X	N	N
∑*	Y	N	N	X	N	N
Halting	Y	Y	Y	X	Y	N

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Closure Properties of Recursive Set

- Recursive languages are closed under following operations
  - Union
  - Concatenation
  - Intersection
  - Reverse
  - Complement
  - Inverse homomorphism
  - Intersection with regular set
  - Set Difference

- Recursive languages are not closed under following operations
  - Kleen closure
  - Homomorphism
  - Substitution

KNOWLEDGEGATE

Closure Properties of Recursive Enumerable Set

- Recursive Enumerable are closed under following operations
  - Union
  - Concatenation
  - Kleen Closure
  - Intersection
  - Substitution
  - Homomorphism
  - Inverse Homomorphism
  - Intersection with regular set
  - Reverse operation

- Recursive Enumerable are not closed under following operations
  - Compliment
  - Set Difference

KNOWLEDGEGATE

	RL	DCFL	CFL	CSL	RS	RES
Union	Y	N	Y	Y	Y	Y
Intersection	Y	N	N	Y	Y	Y
Complement	Y	Y	N	Y	Y	N
Set Difference	Y	N	N	Y	Y	N
Kleene Closure	Y	N	Y	Y	Y	Y
Positive Closure	Y	N	Y	Y	Y	Y
Concatenation	Y	N	Y	Y	Y	Y
Intersection with regular set	Y	Y	Y	Y	Y	Y
Reverse	Y	Y	Y	Y	Y	Y
Subset	N	N	N	N	N	N

KNOWLEDGEGATE

	RL	DCFL	CFL	CSL	RS	RES
Homomorphism	Y	N	Y	N	N	Y
∈ Free Homomorphism	Y	N	Y	Y	Y	Y
Inverse Homomorphism	Y	Y	Y	Y	Y	Y
Substitution	Y	N	Y	N	N	Y
∈ Free Substitution	Y	N	Y	Y	Y	Y
Quotient with regular set	Y	Y	Y	N	Y	Y

KNOWLEDGEGATE
nowledg

Module

Free Semester One Shot Exam Course

Topic

Compiler Design

Subtopic

CD One Shot

Lesson

Compiler_Notes

SEMESTER EXAM SERIES

COMPILER

IN 7 HOURS + NOTES

KNOWLEDGE GATE

Video chapters

Chapter-1 (INTRODUCTION TO COMPILER): Phases and passes, Bootstrapping, Finite state machines and regular expressions and their applications to lexical analysis, Optimization of DFA-Based Pattern Matchers implementation of lexical analyzers, lexical-analyzer generator, LEX compiler, Formal grammars and their application to syntax analysis, BNF notation, ambiguity, YACC. The syntactic specification of programming languages: Context free grammars, derivation and parse trees, capabilities of CFG.

Chapter-2 (BASIC PARSING TECHNIQUES): Parsers, Shift reduce parsing, operator precedence parsing, top down parsing, predictive parsers Automatic Construction of efficient Parsers: LR parsers, the canonical Collection of LR(0) items, constructing SLR parsing tables, constructing Canonical LR parsing tables, Constructing LALR parsing tables, using ambiguous grammars, an automatic parser generator, implementation of LR parsing tables.

Chapter-3 (SYNTAX-DIRECTED TRANSLATION): Syntax-directed Translation schemes, Implementation of Syntax- directed Translators, Intermediate code, postfix notation, Parse trees & syntax trees, three address code, quadruple & triples, translation of assignment statements, Boolean expressions, statements that alter the flow of control, postfix translation, translation with a top down parser. More about translation: Array references in arithmetic expressions, procedures call, declarations and case statements.

Chapter-4 (SYMBOL TABLES): Data structure for symbols tables, representing scope information. Run-Time Administration: Implementation of simple stack allocation scheme, storage allocation in block structured language. Error Detection & Recovery: Lexical Phase errors, syntactic phase errors semantic errors.

Chapter-5 (CODE GENERATION): Design Issues, the Target Language. Addresses in the Target Code, Basic Blocks and Flow Graphs, Optimization of Basic Blocks, Code Generator. Code optimization: Machine-Independent Optimizations, Loop optimization, DAG representation of basic blocks, value numbers and algebraic laws, Global Data-Flow analysis.

KNOWLEDGEGATE

Chapter-1

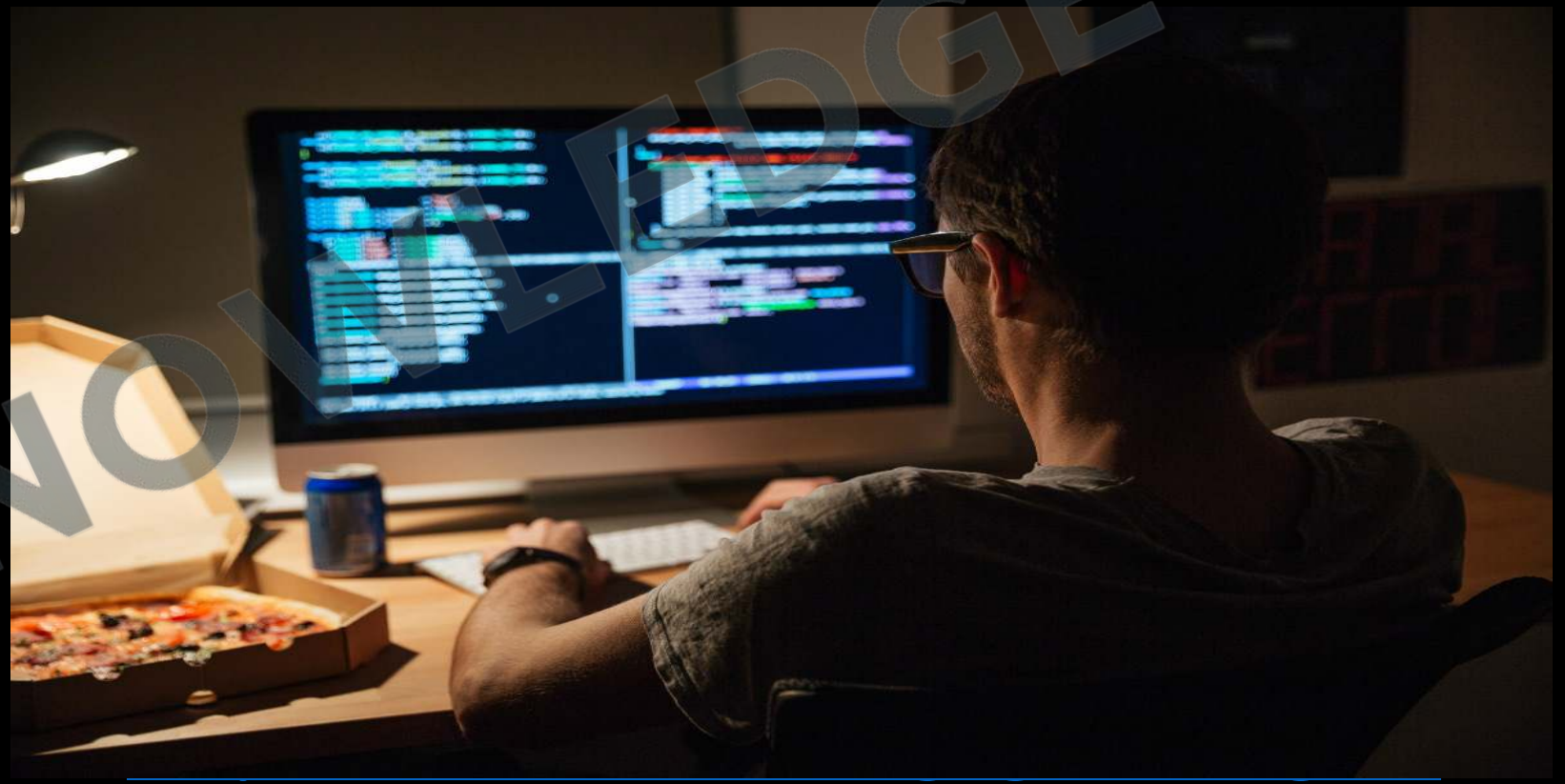
(INTRODUCTION TO COMPILER): Phases and passes, Bootstrapping, Finite state machines and regular expressions and their applications to lexical analysis, Optimization of DFA-Based Pattern Matchers implementation of lexical analyzers, lexical-analyzer generator, LEX compiler, Formal grammars and their application to syntax analysis, BNF notation, ambiguity, YACC. The syntactic specification of programming languages: Context free grammars, derivation and parse trees, capabilities of CFG.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Language Processing System

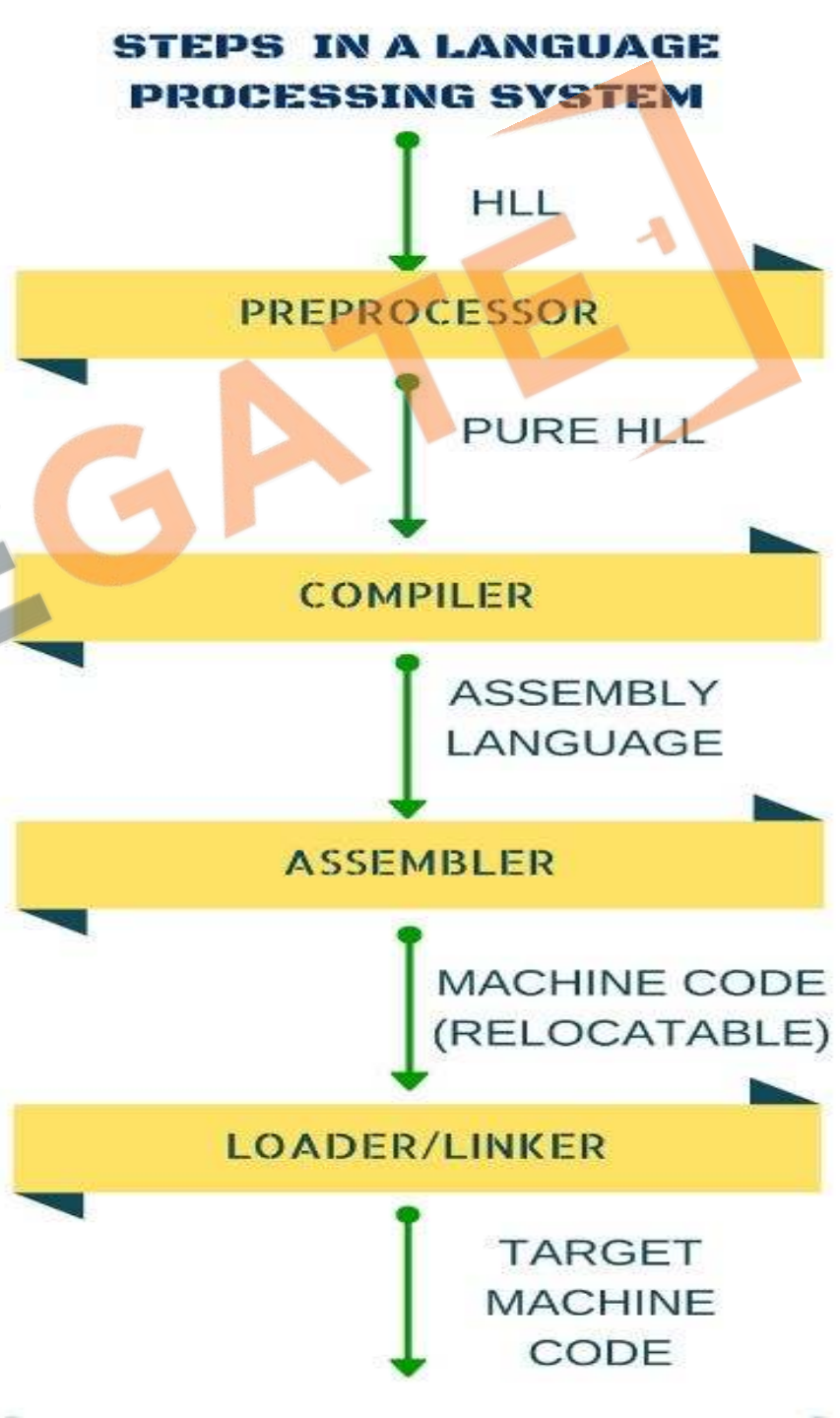
- We tend to write programs in high-level language, that is much less complicated for us to comprehend and maintain in thoughts. These programs go through a series of transformation so that they can readily be used in machines. This is where language processing systems come handy.



KNOWLEDGE GATE

High Level Language

- If a program contains #define or #include directives such as #include or #define it is called HLL.
- They are closer to humans but far from machines.
- These (#) tags are called pre-processor directives. They direct the pre-processor about what to do.



KNOWLEDGEGATE
http://www.knowledgega

```c
#include <stdio.h>
#include <unistd.h>

int main ()
{
        printf("Running 'net join' with the following parameters: \n");
        char *domain="mydomain";
        char *user="domainjoinuser";
        char *pass="mypassword";
        char *vastool="/opt/quest/bin/vastool";
        char *ou="OU=test,DC=mtdomian,DC=local";
        char unjoin[512];

        sprintf(unjoin,"/opt/quest/bin/vastool -u %s -w '%s' unjoin -f",user,pass);

        printf("Domain: %s\n",domain);
        printf("User: %s\n",user);
        printf("-----------------\n");

        printf("\nUnjoin.....\n");
        system(unjoin);

        printf("\nJoin.....\n");
        execl("/opt/quest/bin/vastool", "vastool", "-u", user, "-w", pass, "join", "-c", ou,
"-f", domain, (char*)0);

}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Macros may be nested

- in definitions, e.g.:

```
#define  Pi          3.1416
#define  Twice_Pi    2*Pi
```

- in uses, e.g.:

```
#define  double(x)   x+x
#define  Pi  3.1416
...
if ( x > double(Pi) ) ...
```

KNOWLEDGEGATE
egate.in/gate

Simple assignment operator	Shorthand operator
a = a+1	a + =1
a = a-1	a - =1
a = a* (m+n)	a * = m+n
a = a / (m+n)	a / = m+n
a = a %b	a %=b

KNOWLEDGEGATE

Pre-Processor

- The pre-processor removes all the #include directives by including the files called file inclusion.
- All the #define directives using macro expansion. It performs file inclusion, macro-processing, short hand operators etc.

KNOWLEDGEGATE

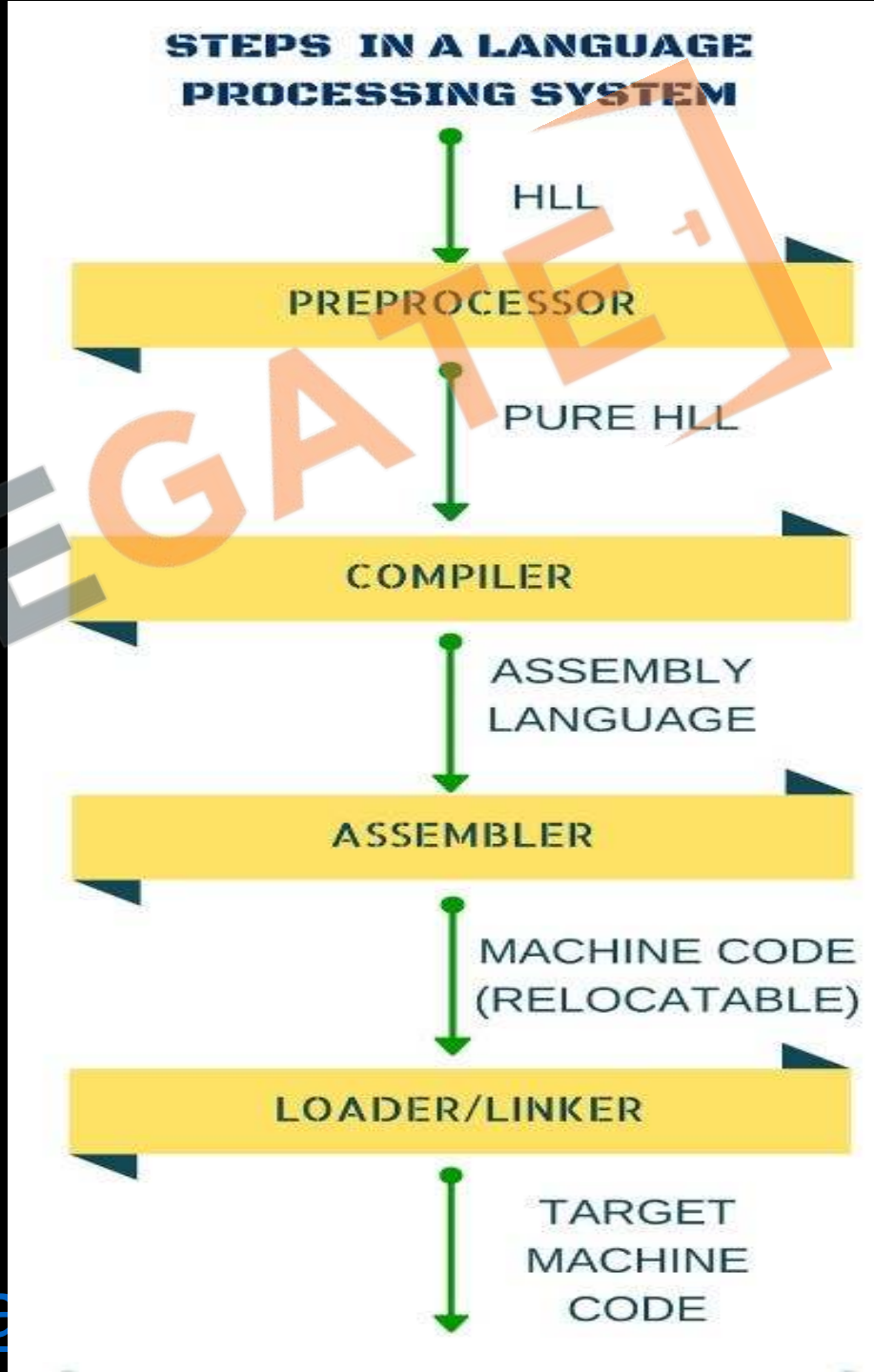
Pure High-Level Language

- That HLL which can be directly understood by the compiler.

डालडा

देसी

http://www.knowledgega



KNOWLEDGEGATE

STEPS IN A LANGUAGE PROCESSING SYSTEM

HLL

PREPROCESSOR

PURE HLL

COMPILER

ASSEMBLY LANGUAGE

ASSEMBLER

MACHINE CODE (RELOCATABLE)

LOADER/LINKER

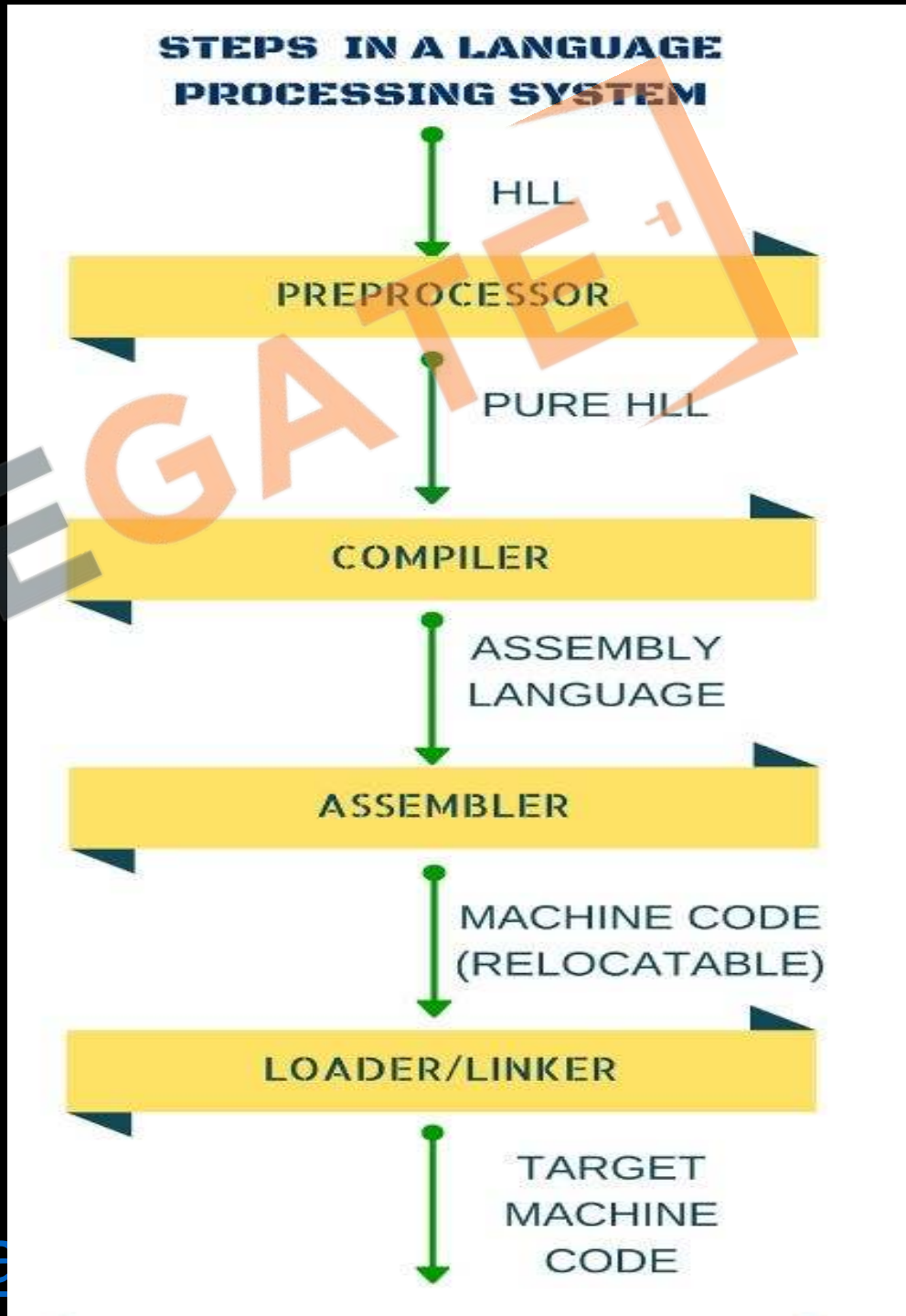
TARGET MACHINE CODE

KNOWLEDGEGATE

http://www.knowledgega

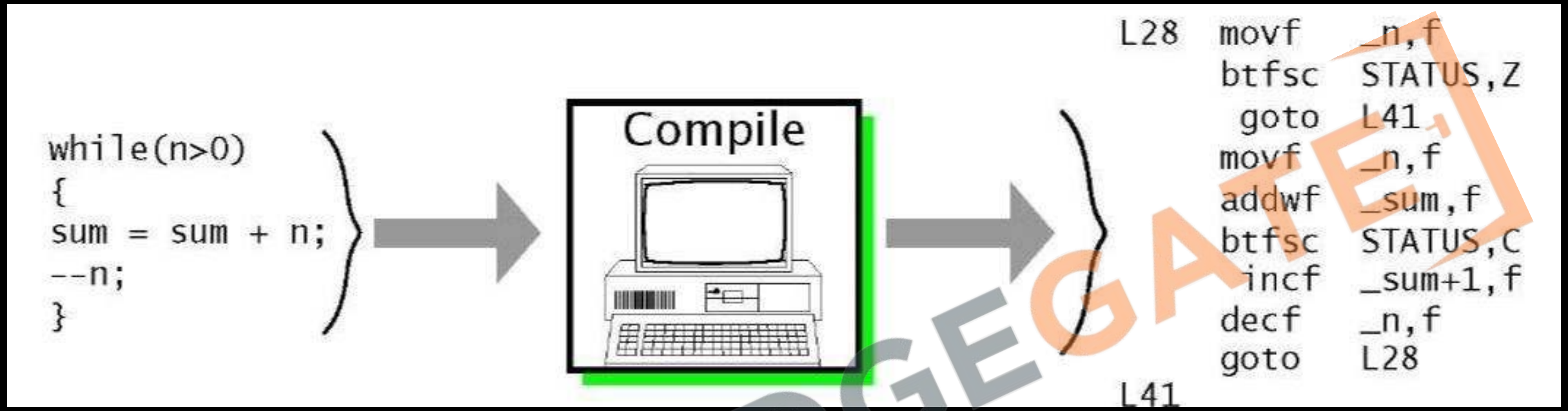
Compiler

- A compiler is a computer program that translates computer code written in one programming language (the source language) into another language (the target language).
- The name compiler is primarily used for programs that translate source code from a high-level programming language to a lower level language (e.g., assembly language) to create an executable program.
- The reason is we are not comfortable in writing a low-level language there for we write a code which is easy and then convert it into low level language.



KNOWLEDGEGATE

http://www.knowledgega

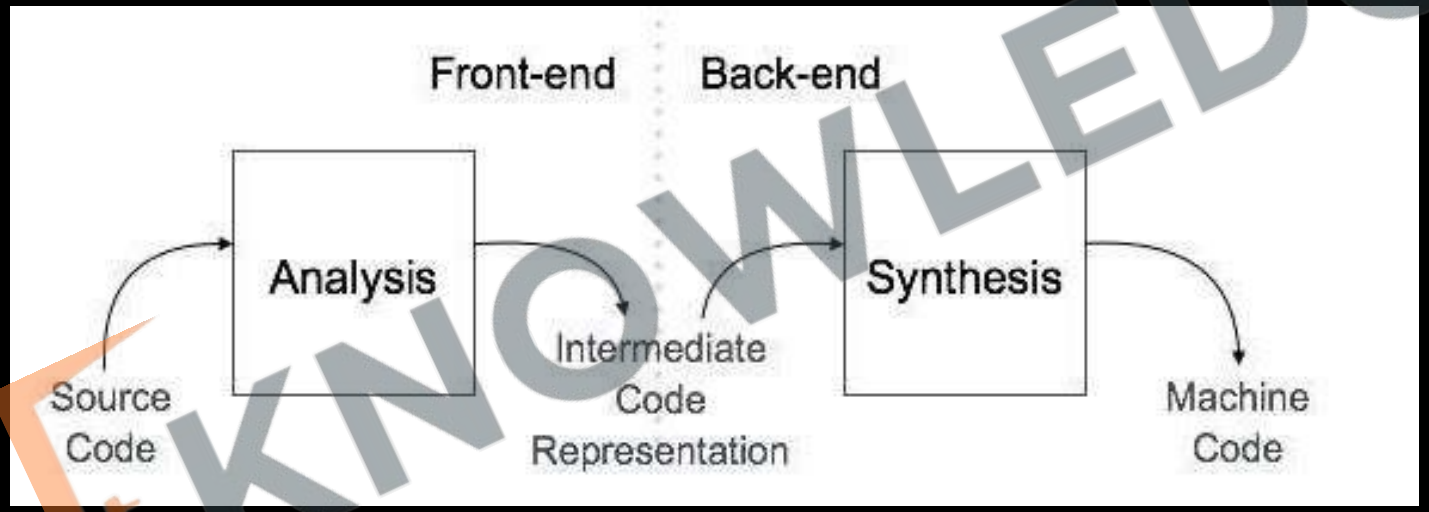


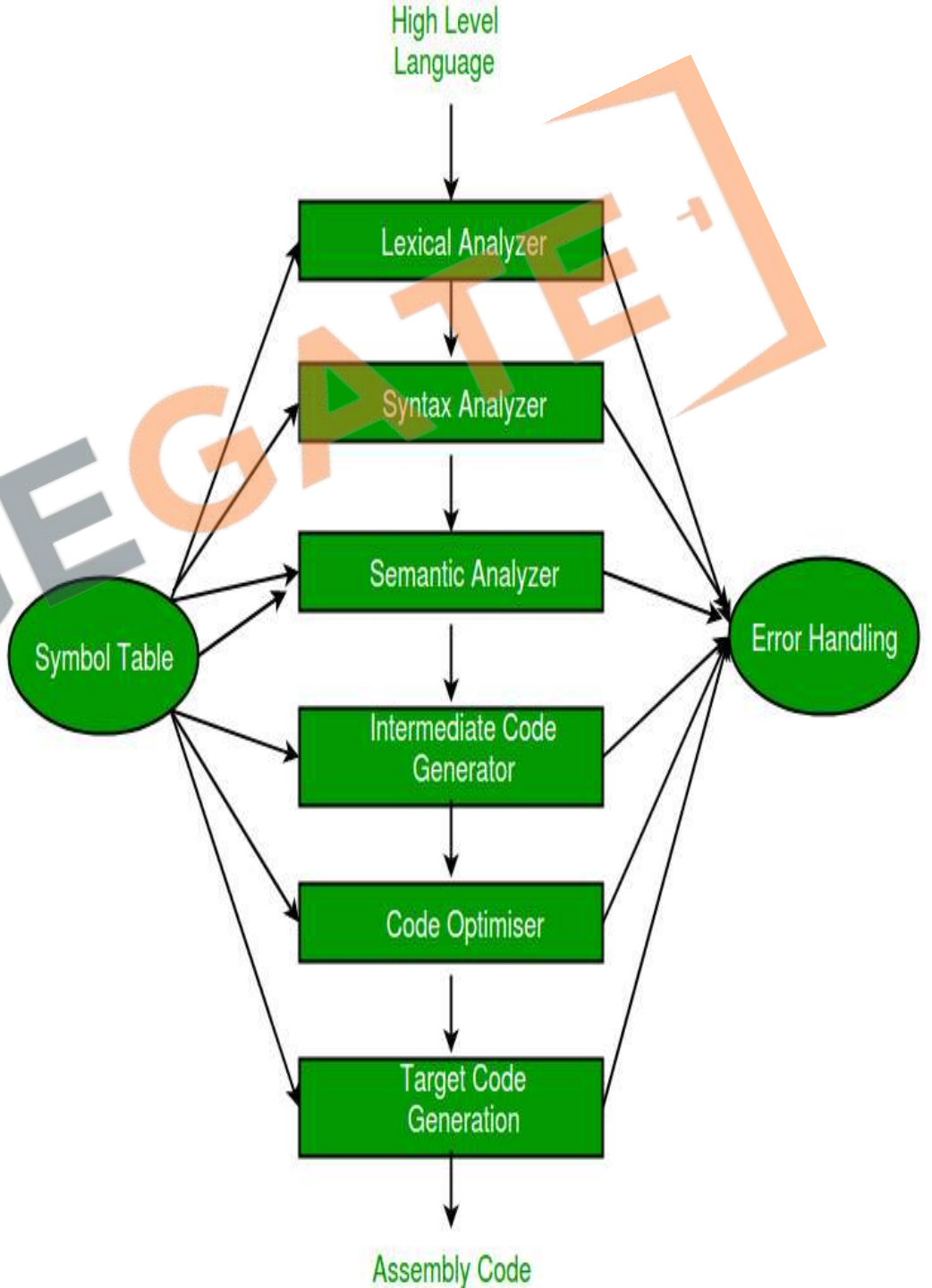
KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Compiler Design

- We basically have two phases of compilers, namely Analysis phase and Synthesis phase.
  - **Analysis phase** creates an intermediate representation from the given source code.
  - **Synthesis phase** creates an equivalent target program from the intermediate representation.



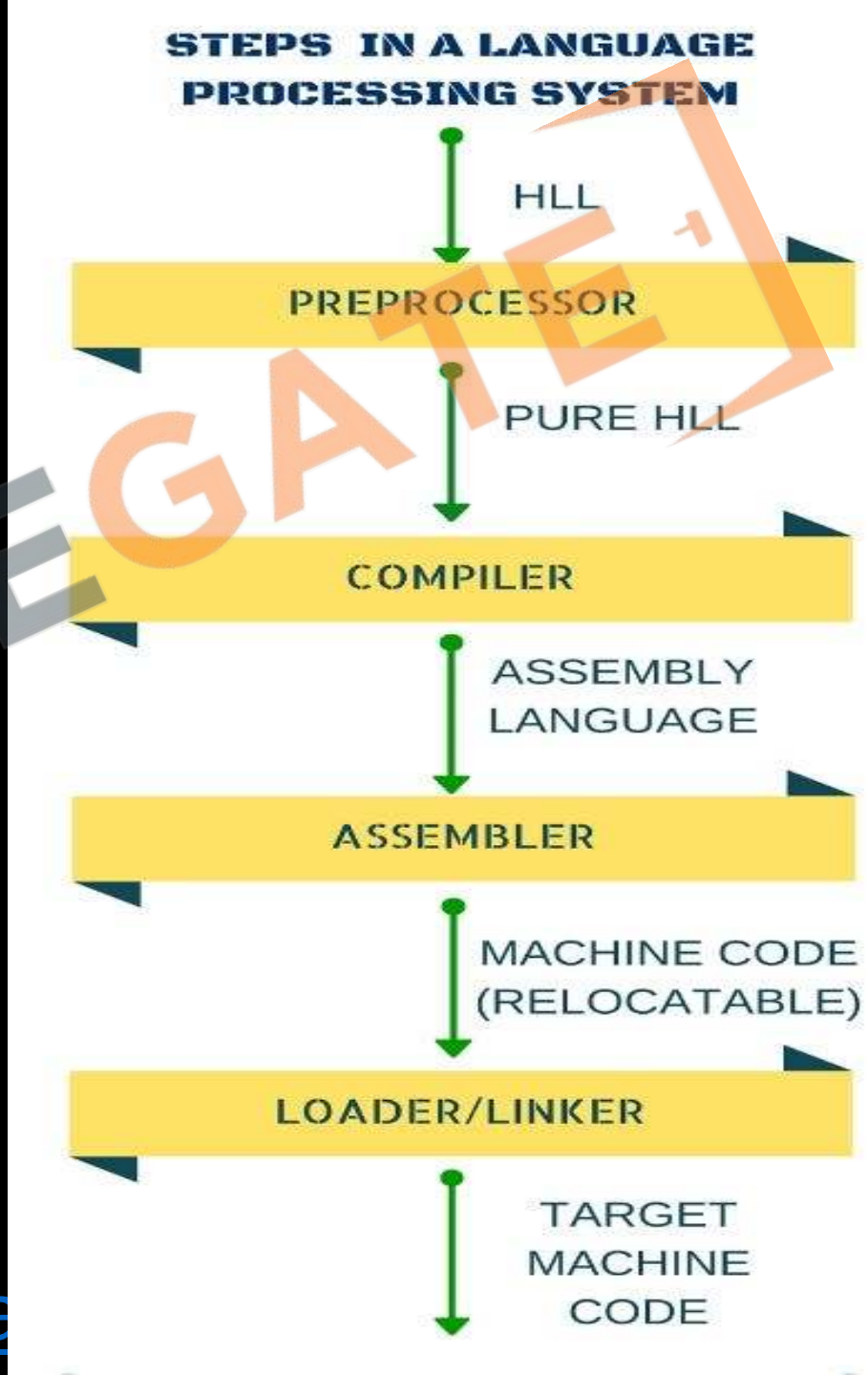


KNOWLEDGEGATE

http://www.knowledge

Assembly Language

- Its neither in binary form nor high level.
- It is an intermediate state that is a combination of machine instructions and some other useful data needed for execution.

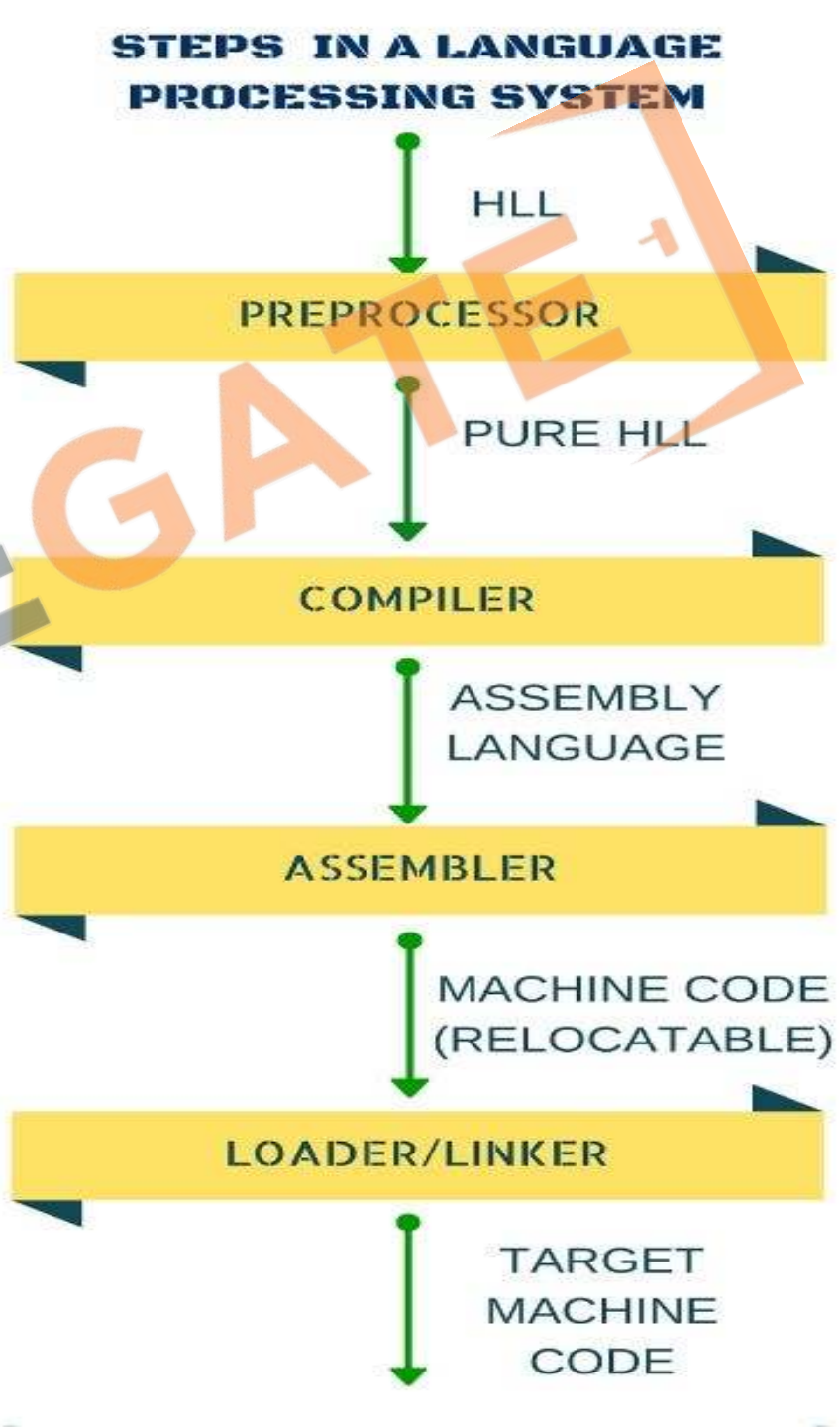


KNOWLEDGEGATE

http://www.knowledgega

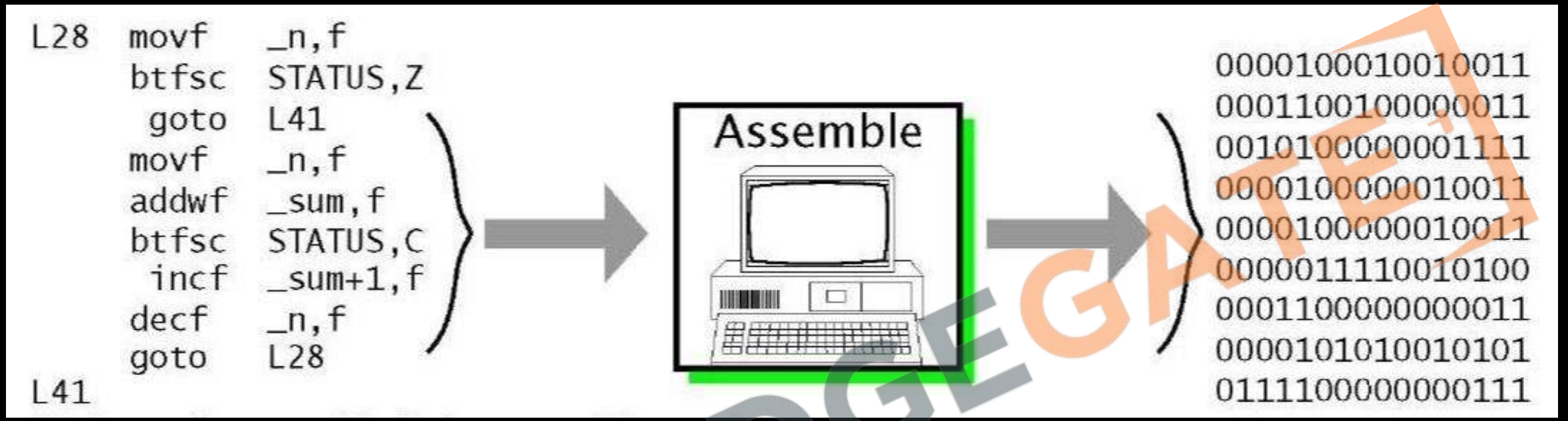
Assembler

- For every platform (Hardware + OS) we will have an assembler. They are not universal since for each platform we have one.
- The output of assembler is called object file. It translates assembly language to machine code.



KNOWLEDGEGATE

http://www.knowledgega



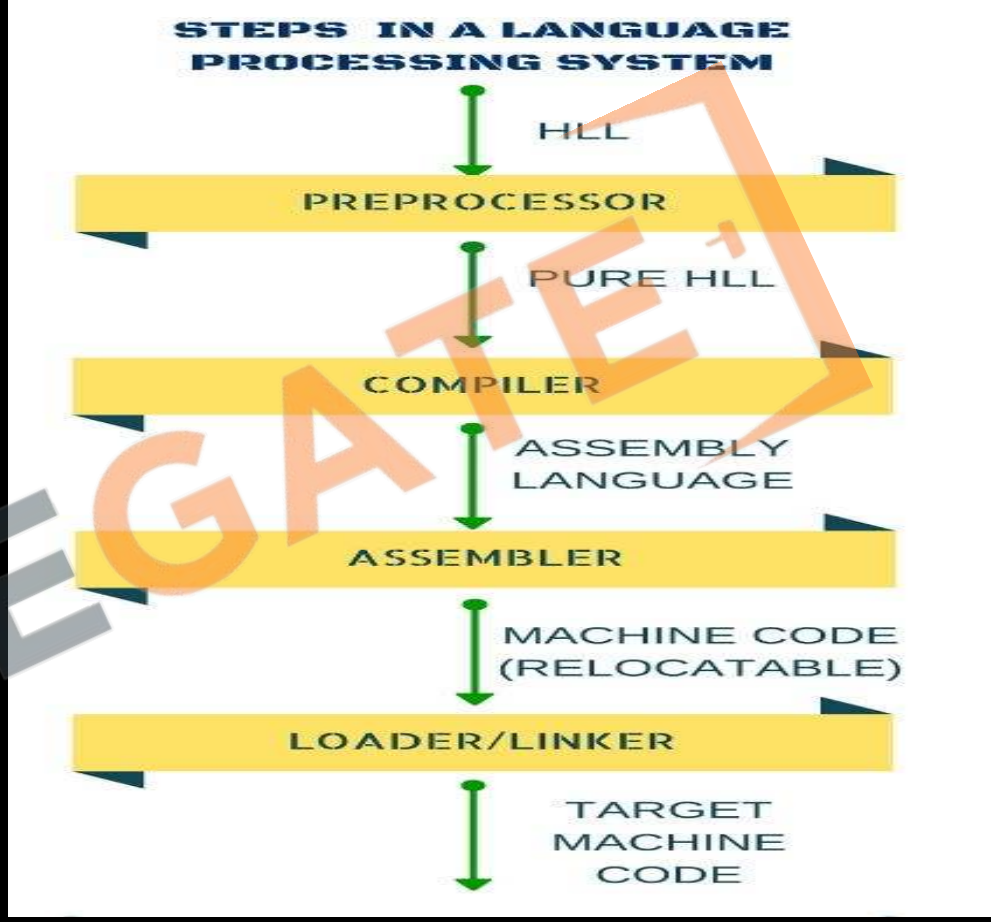
KNOWLEDGEGATE

8085 Microprocessor

http://www.knowledgegate.in/gate

Relocatable Machine Code

- It can be loaded at any point and can be run.
- The address within the program will be in such a way that it will cooperate for the program movement.



KNOWLEDGEGATE

if(a<b) then t=1
else t = 0

i) if (a<b) goto (i+3)
i+1) t=0
i+2) goto (i+4)
i+3) t=1
i+4) exit

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

while(C) do S

i) if (E) goto i+2
i+1) goto i+4
i+2) S
i+3) goto i
i+4) exit

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

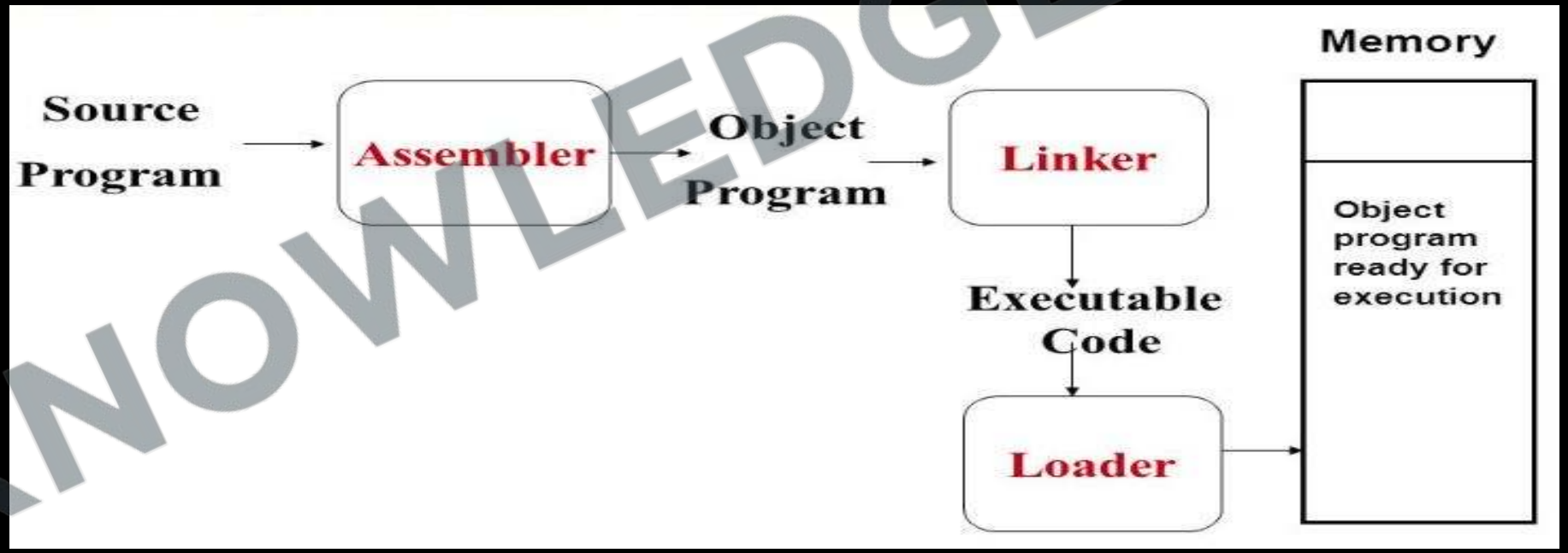
```
for(1=0; i<10 ; i++)
    s
```

i) i = 0
i+1) if(i<10) goto i+3
i+2) goto i+6
i+3) S
i+4) i = i + 1
i+5) goto i+1
i+6) exit

KNOWLEDGEGATE

Linker → Loader

- It converts the relocatable code into absolute code and tries to run the program resulting in a running program or an error message.
- Linker loads a variety of object files into a single file to make it executable. Then loader loads it in memory and executes it.



KNOWLEDGEGATE

- The first practical compiler was written by Corrado Böhm, in 1951, for his PhD thesis.

- The first implemented compiler was written by Grace Hopper, who also coined the term "compiler", referring to her A-0 system which functioned as a loader or linker, not the modern notion of a compiler.

KNOWLEDGEGATE

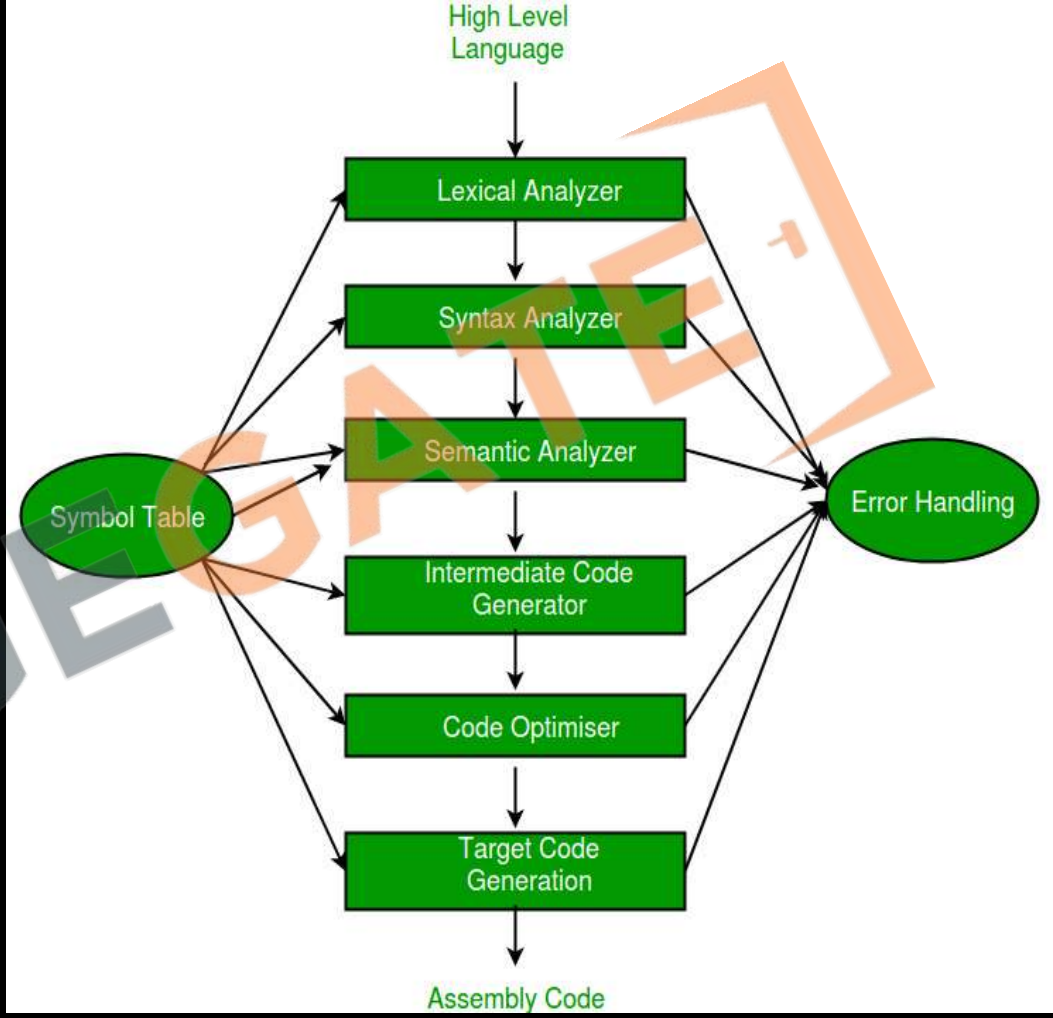
- The first Autocode and compiler in the modern sense were developed by Alick Glennie in 1952 at the University of Manchester for the Mark 1 computer.

- The FORTRAN team led by John W. Backus at IBM introduced the first commercially available compiler, in 1957, which took 18 person-years to create.

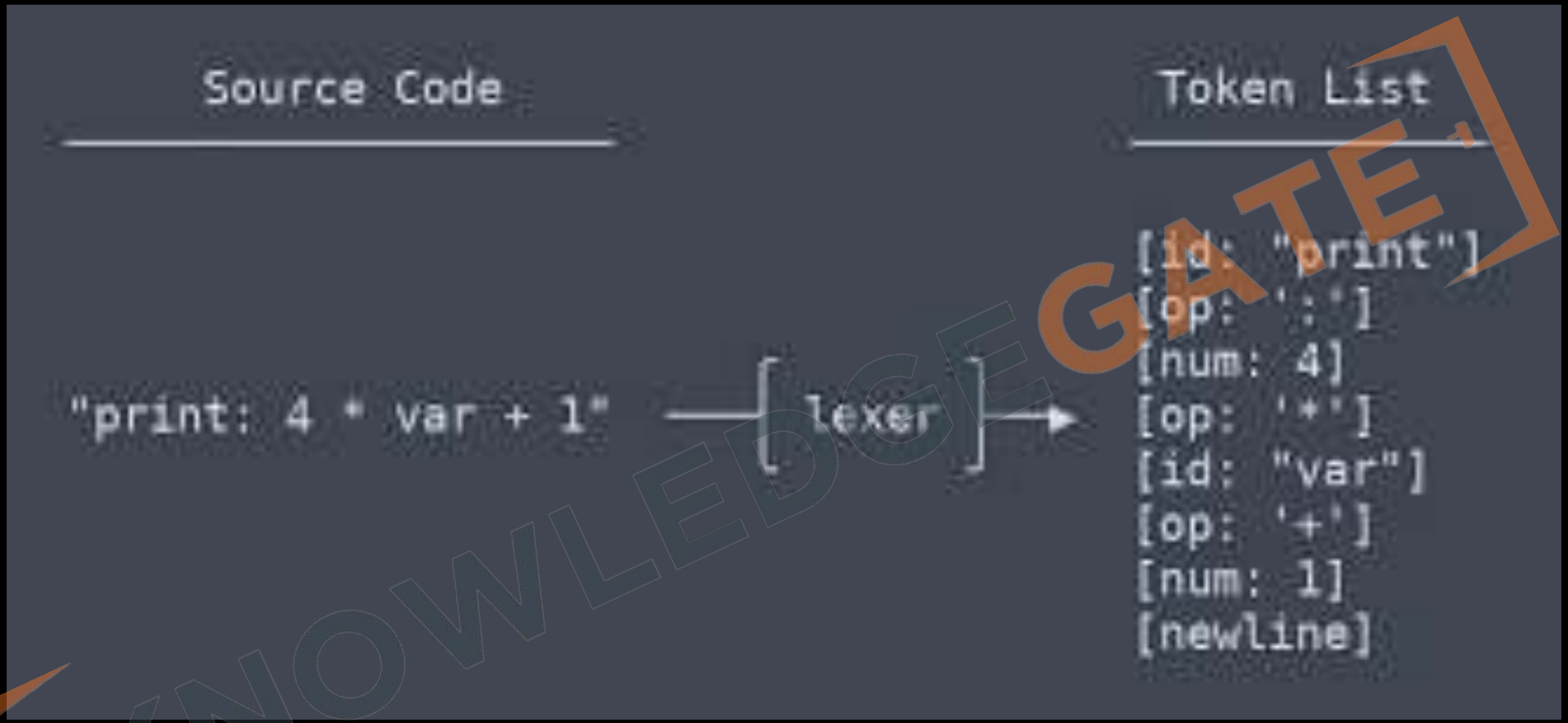
KNOWLEDGEGATE

Lexical Analyzer

- It reads the program and converts it into Lexemes.
- A stream of lexemes into a stream of tokens.
- Tokens are defined by regular expressions which are understood by the lexical analyser. It also removes white-spaces and comments.



KNOWLEDGEGATE



KNOWLEDGE GATE

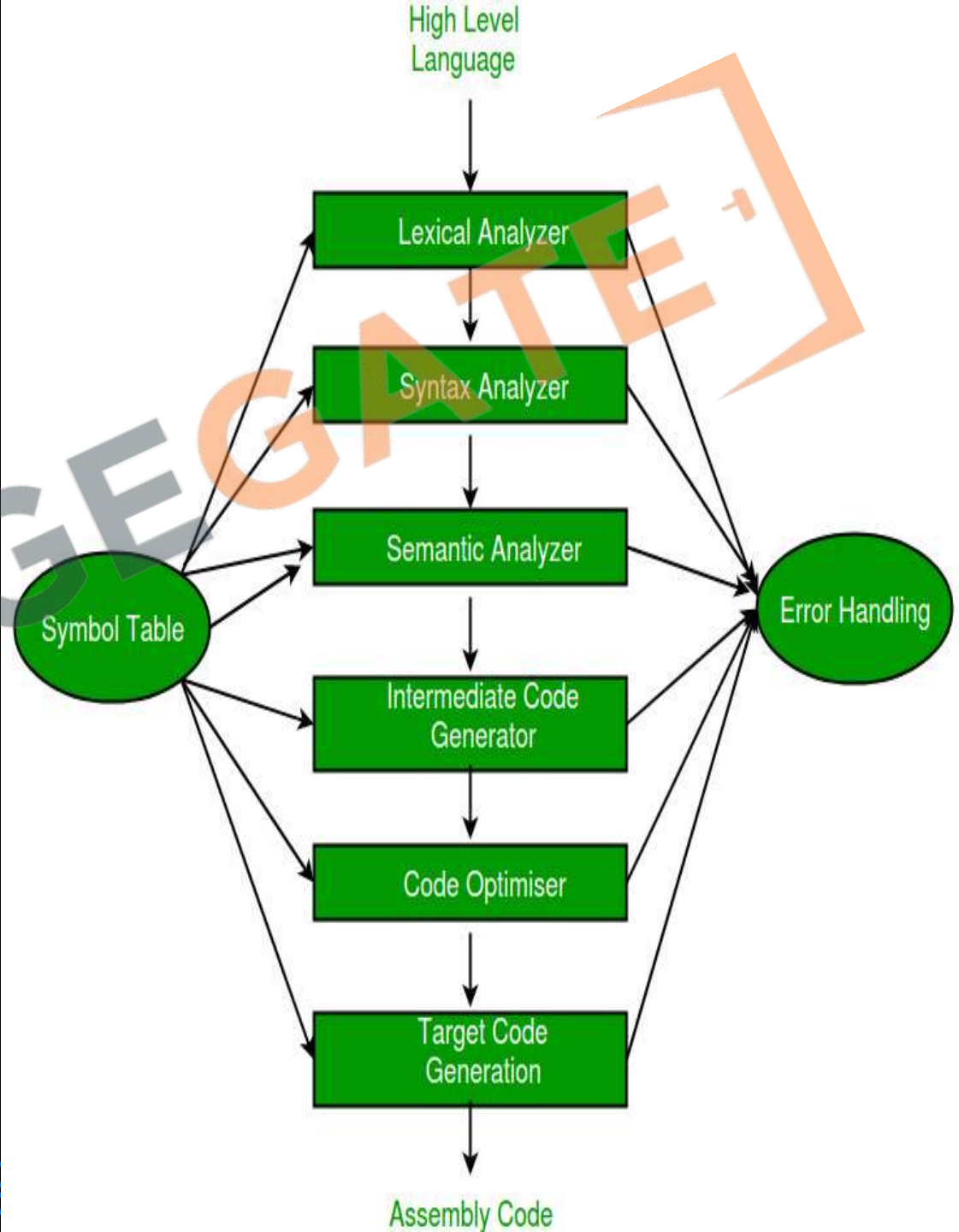
http://www.knowledgegate.in/gate

Float x, y, z;
x = y+ z*60

X → token → identifier
= → token → operator
Y → token → identifier
+ → token → operator
Z → token → identifier
* → token → operator
60 → token → constant

$id_1 = id_2 + id_3 * 60$

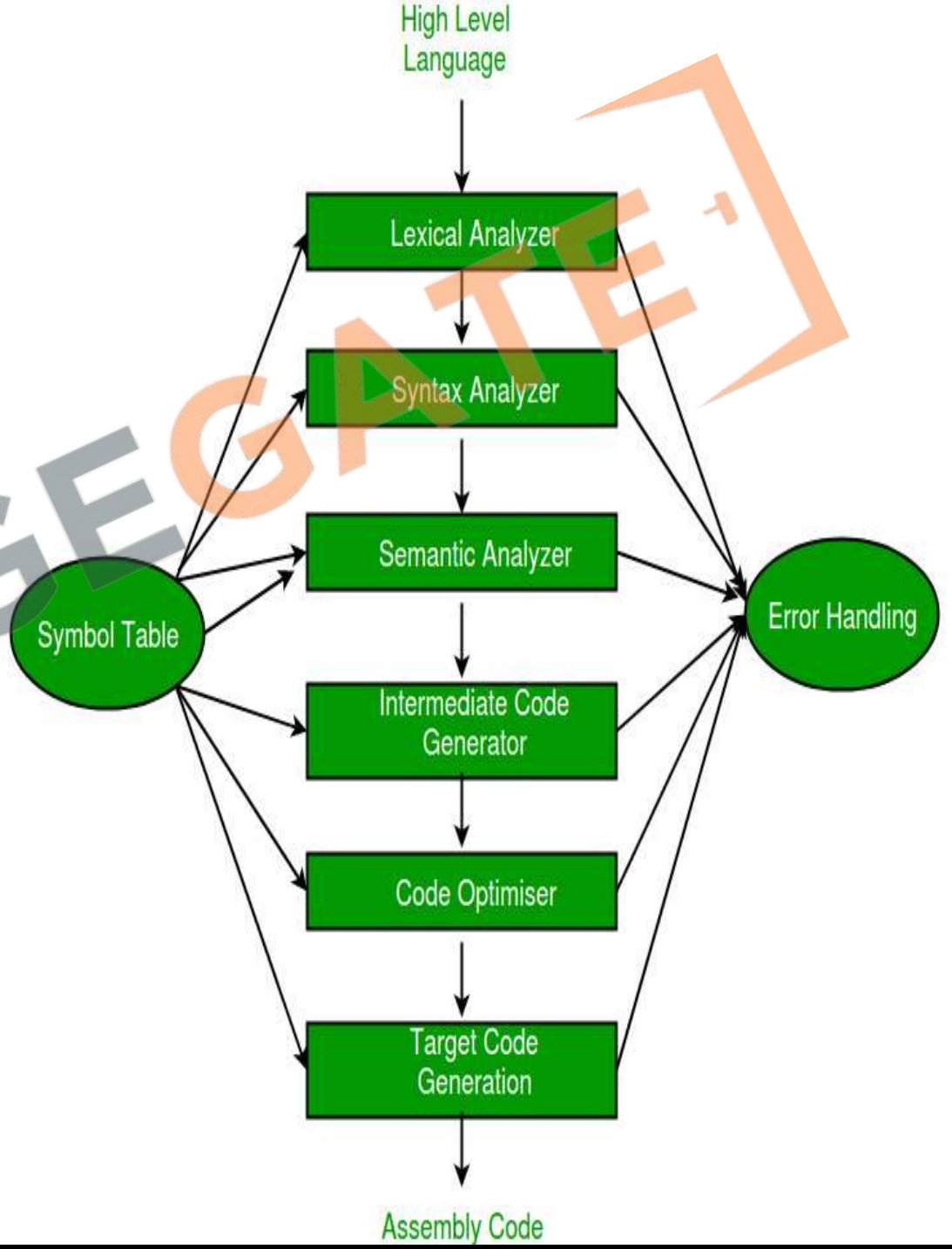
http://www.knowledg



KNOWLEDGEGATE

Syntax Analyzer

- It is sometimes called as parser.
- It constructs the Parse/Syntax tree.
- Uses productions of Context Free Grammar to construct the parse tree.
- It reads all the tokens one by one.



KNOWLEDGEGATE

http://www.knowledg

S → id = E

E → E + T / T

T → T * F / F

F → id / num

$id_1 = id_2 + id_3 * 60$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

1. *program* → *declarationList*
2. *declarationList* → *declarationList declaration* | *declaration*
3. *declaration* → *varDeclaration* | *funDeclaration*

---

4. *varDeclaration* → *typeSpecifier varDeclList* **;**
5. *scopedVarDeclaration* → *scopedTypeSpecifier varDeclList* **;**
6. *varDeclList* → *varDeclList* **,** *varDeclInitialize* | *varDeclInitialize*
7. *varDeclInitialize* → *varDeclId* | *varDeclId* **:** *simpleExpression*
8. *varDeclId* → **ID** | **ID [NUMCONST]**
9. *scopedTypeSpecifier* → **static** *typeSpecifier* | *typeSpecifier*
10. *typeSpecifier* → **int** | **bool** | **char**

---

11. *funDeclaration* → *typeSpecifier* **ID** **(** *params* **)** *statement* | **ID** **(** *params* **)** *statement*
12. *params* → *paramList* | ϵ
13. *paramList* → *paramList* **;** *paramTypeList* | *paramTypeList*
14. *paramTypeList* → *typeSpecifier paramIdList*

KNOWLEDGEGATE

15. *paramIdList* → *paramIdList* , *paramId* | *paramId*

16. *paramId* → **ID** | **ID** [ ]

---

17. *statement* → *expressionStmt* | *compoundStmt* | *selectionStmt* | *iterationStmt* | *returnStmt* | *breakStmt*

18. *expressionStmt* → *expression* ; | ;

19. *compoundStmt* → { *localDeclarations statementList* }

20. *localDeclarations* → *localDeclarations scopedVarDeclaration* | **ϵ**

21. *statementList* → *statementList statement* | **ϵ**

22. *elsifList* → *elsifList* **elsif** *simpleExpression* **then** *statement* | **ϵ**

23. *selectionStmt* → **if** *simpleExpression* **then** *statement elsifList* | **if** *simpleExpression* **then** *statement elsifList* **else** *statement*

24. *iterationRange* → **ID** = *simpleExpression* .. *simpleExpression* | **ID** = *simpleExpression* .. *simpleExpression* : *simpleExpression*

25. *iterationStmt* → **while** *simpleExpression* **do** *statement* | **loop forever** *statement* | **loop** *iterationRange* **do** *statement*

26. *returnStmt* → **return** ; | **return** *expression* ;

27. *breakStmt* → **break** ;

KNOWLEDGEGATE

28. *expression* → *mutable* = *expression* | *mutable* += *expression* | *mutable* −= *expression* | *mutable* *= *expression* | *mutable* /= *expression* | *mutable* ++ | *mutable* −− | *simpleExpression*

29. *simpleExpression* → *simpleExpression* **or** *andExpression* | *andExpression*

30. *andExpression* → *andExpression* **and** *unaryRelExpression* | *unaryRelExpression*

31. *unaryRelExpression* → **not** *unaryRelExpression* | *relExpression*

32. *relExpression* → *sumExpression relop sumExpression* | *sumExpression*

33. *relop* → <= | < | > | >= | == | !=

34. *sumExpression* → *sumExpression sumop mulExpression* | *mulExpression*

35. *sumop* → + | −

KNOWLEDGEGATE

36. *mulExpression* → *mulExpression mulop unaryExpression* | *unaryExpression*

37. *mulop* → * | / | %

38. *unaryExpression* → *unaryop unaryExpression* | *factor*

39. *unaryop* → − | * | ?

40. *factor* → *immutable* | *mutable*

41. *mutable* → **ID** | *mutable* [*expression*]

42. *immutable* → (*expression*) | *call* | *constant*

43. *call* → **ID** (*args*)

44. *args* → *argList* | **ϵ**

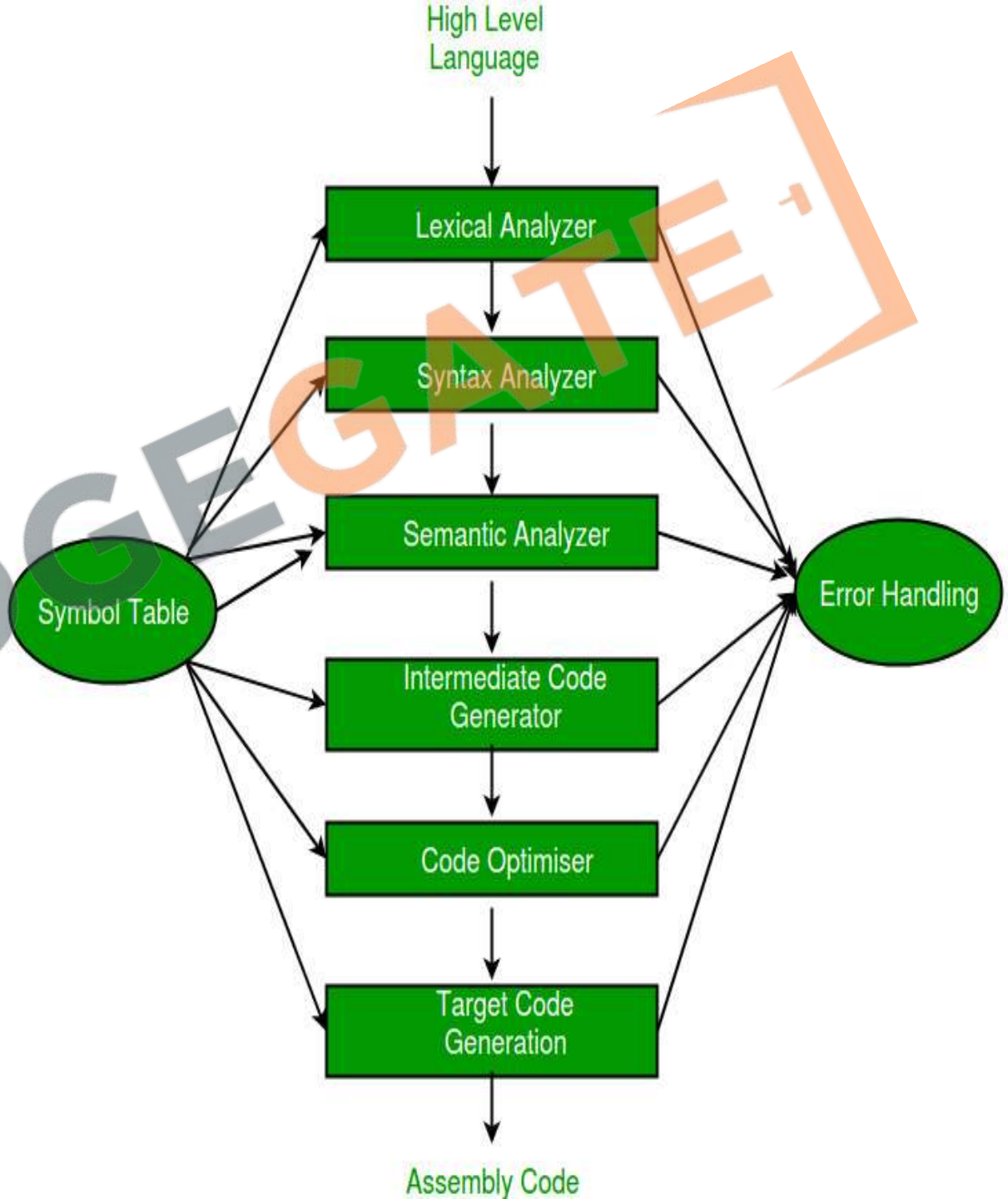
45. *argList* → *argList* , *expression* | *expression*

46. *constant* → **NUMCONST** | **CHARCONST** | **STRINGCONST** | **true** | **false**

KNOWLEDGEGATE

Semantic Analyzer

- It verifies the parse tree, whether it's meaningful or not.
- It furthermore produces a verified / Annotated parse tree.

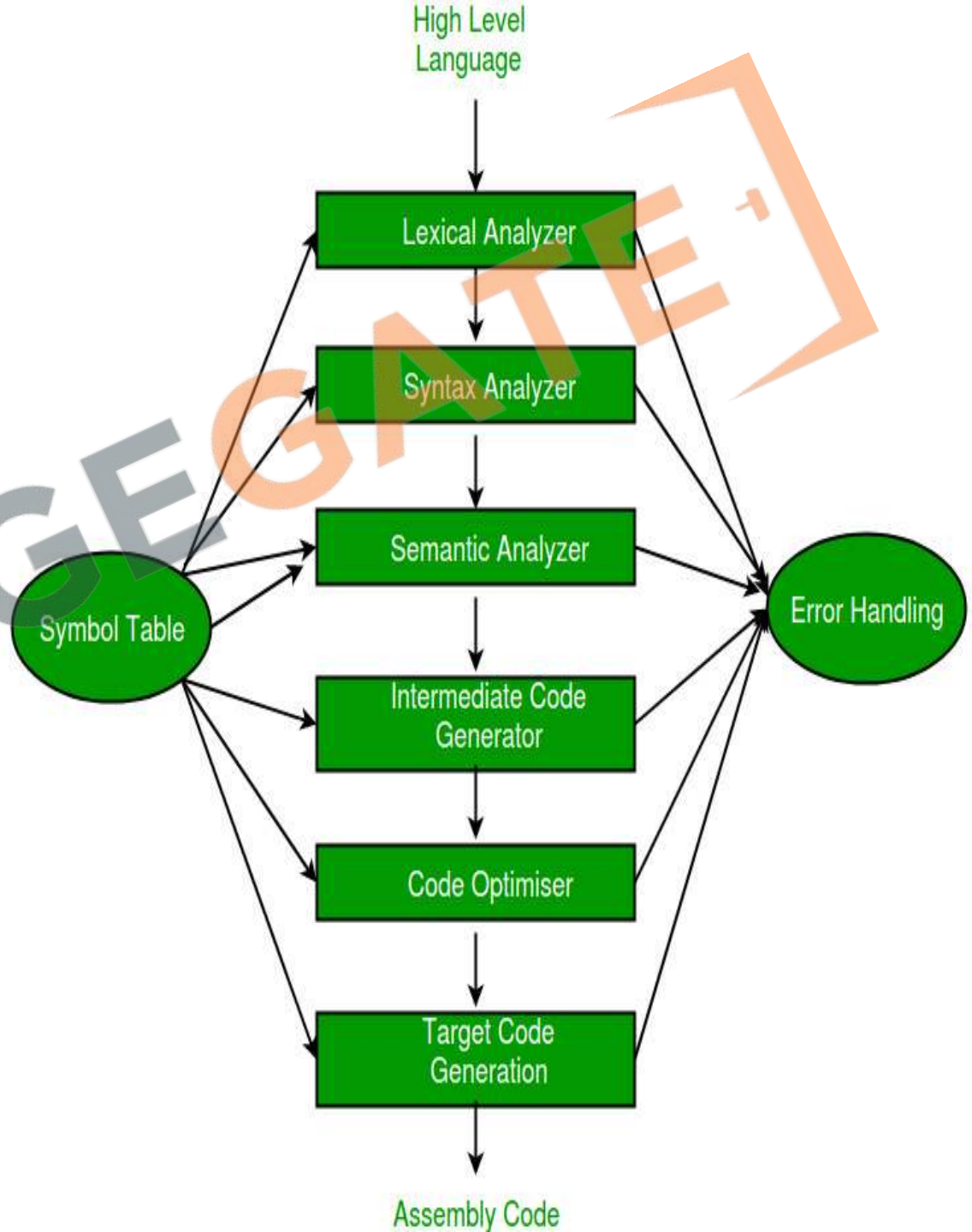


KNOWLEDGEGATE
http://www.knowle

Intermediate Code Generator

- It generates intermediate code, that is a form which can be readily understood by machine.
- We have many popular intermediate codes. Example – Three address code etc.
- Intermediate code is converted to machine language using the last two phases which are platform dependent.
- Till intermediate code, it is same for every compiler out there, but after that, it depends on the platform.
- To build a new compiler we don't need to build it from scratch. We can take the intermediate code from the already existing compiler and build the last two parts.

http://www.knowled



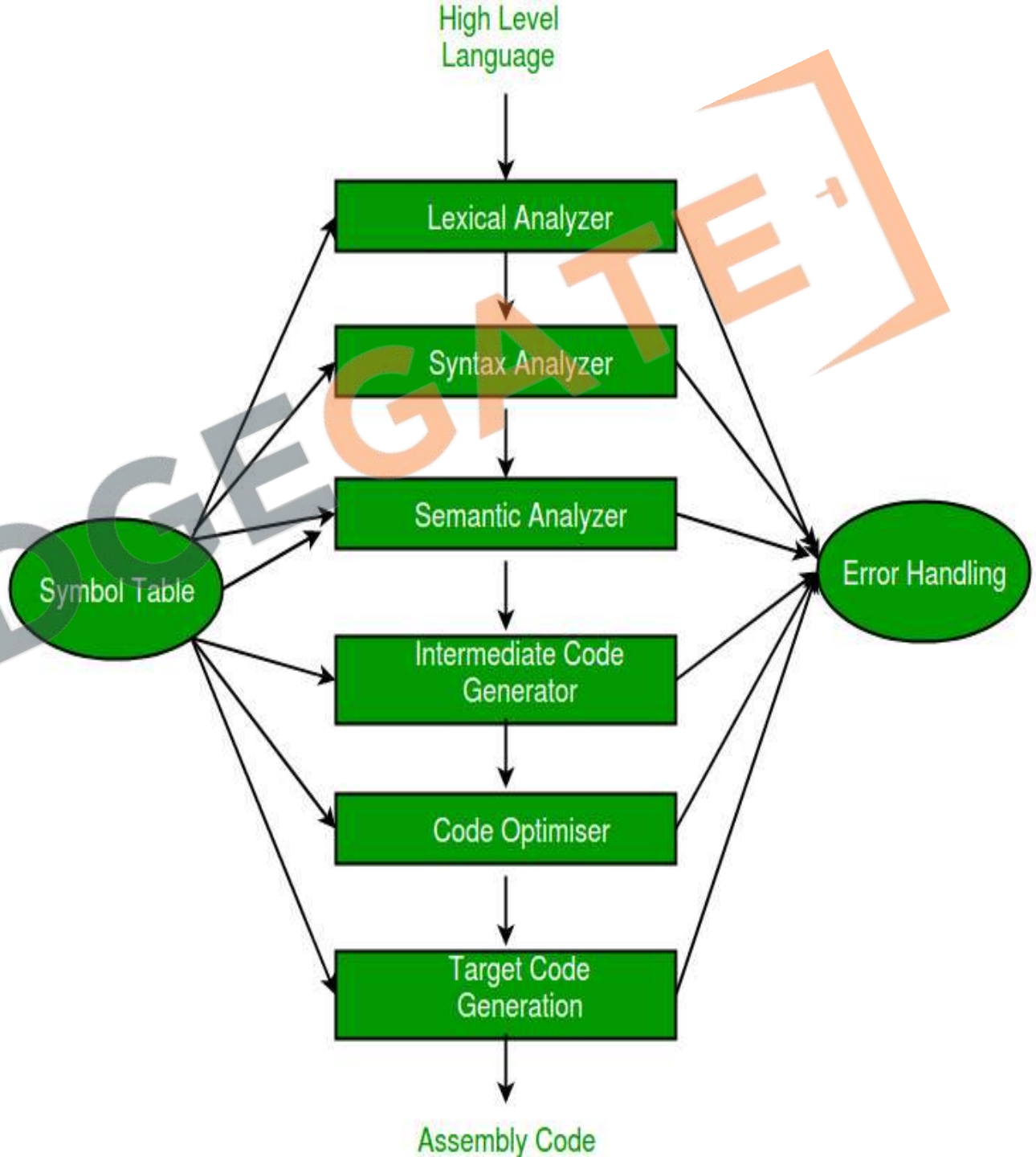
KNOWLEDGEGATE

- $x = y + z*60$

  $t_1 = z * 60$

  $t_2 = y + t_1$

  $x = t_2$



KNOWLEDGEGATE
http://www.knov

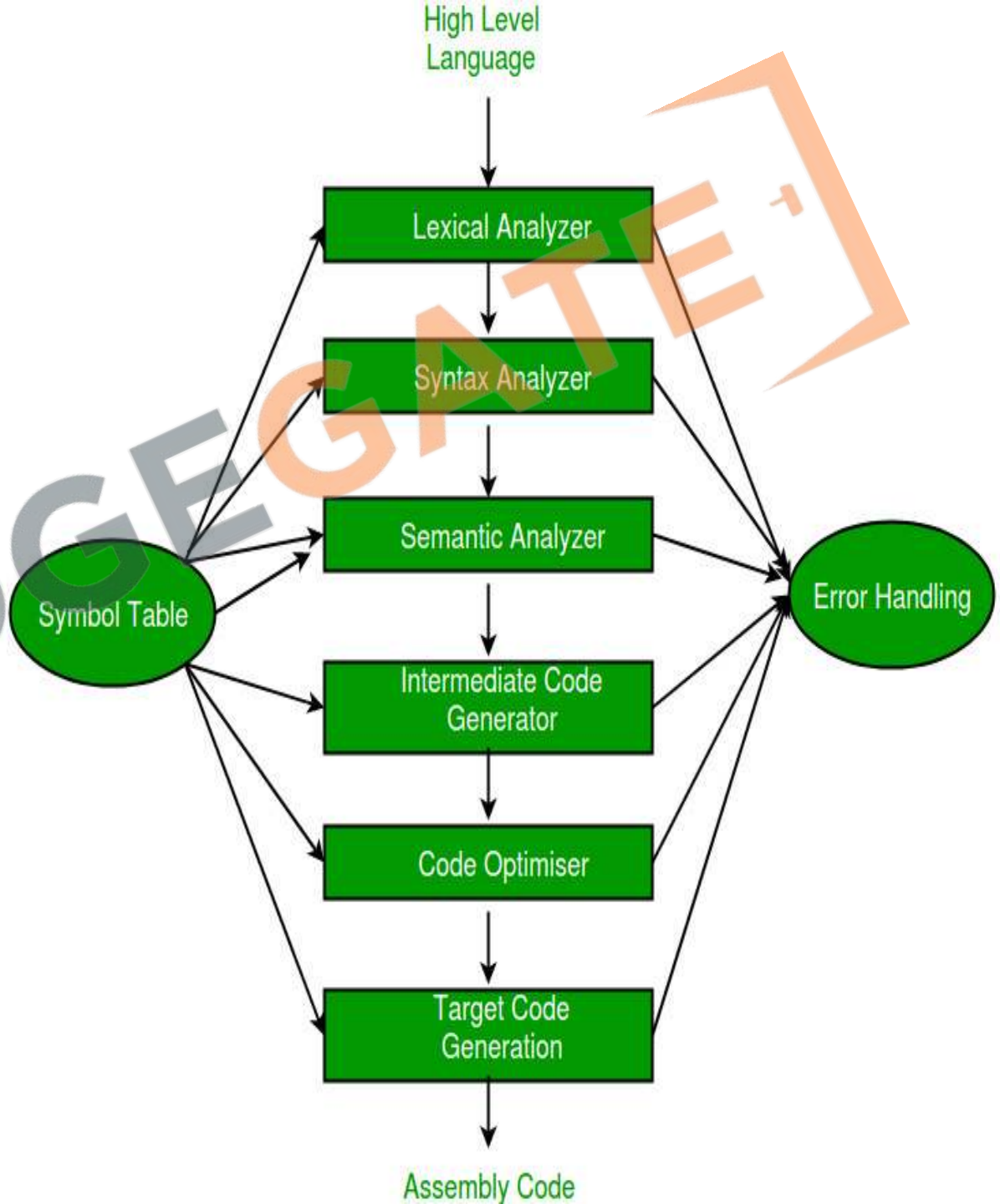
Code Optimizer

- It transforms the code so that it consumes fewer resources and produces more speed.
- The meaning of the code optimizer is code being transformed is not altered.
- Optimisation can be categorized into two types: machine dependent and machine independent.

$t_1 = z * 60$

$x = y + t_1$

http://www.knowle

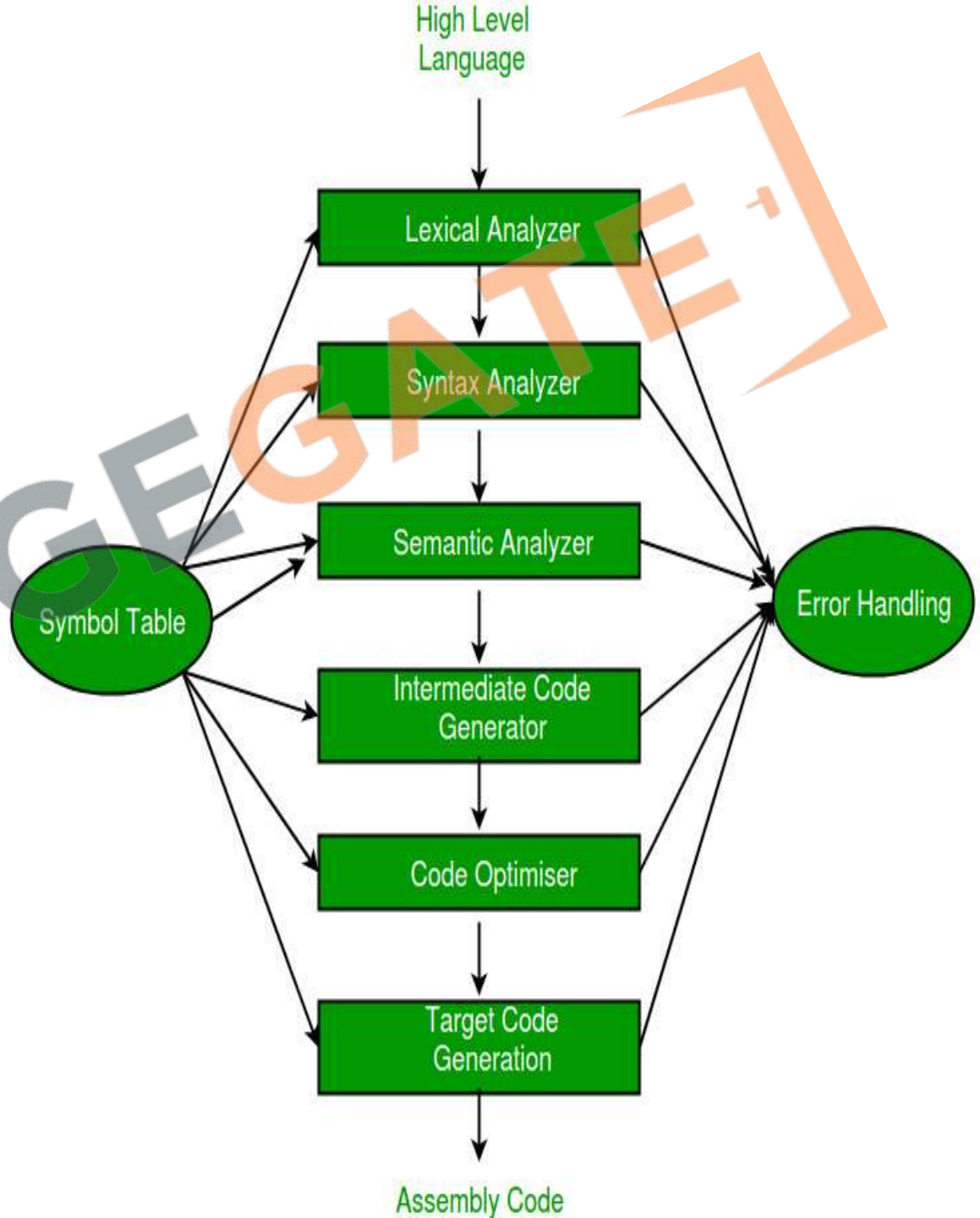


KNOWLEDGEGATE

Target Code Generator

- The main purpose of Target Code generator is to write a code that the machine can understand and also register allocation, instruction selection etc.
- The output is dependent on the type of assembler. This is the final stage of compilation.

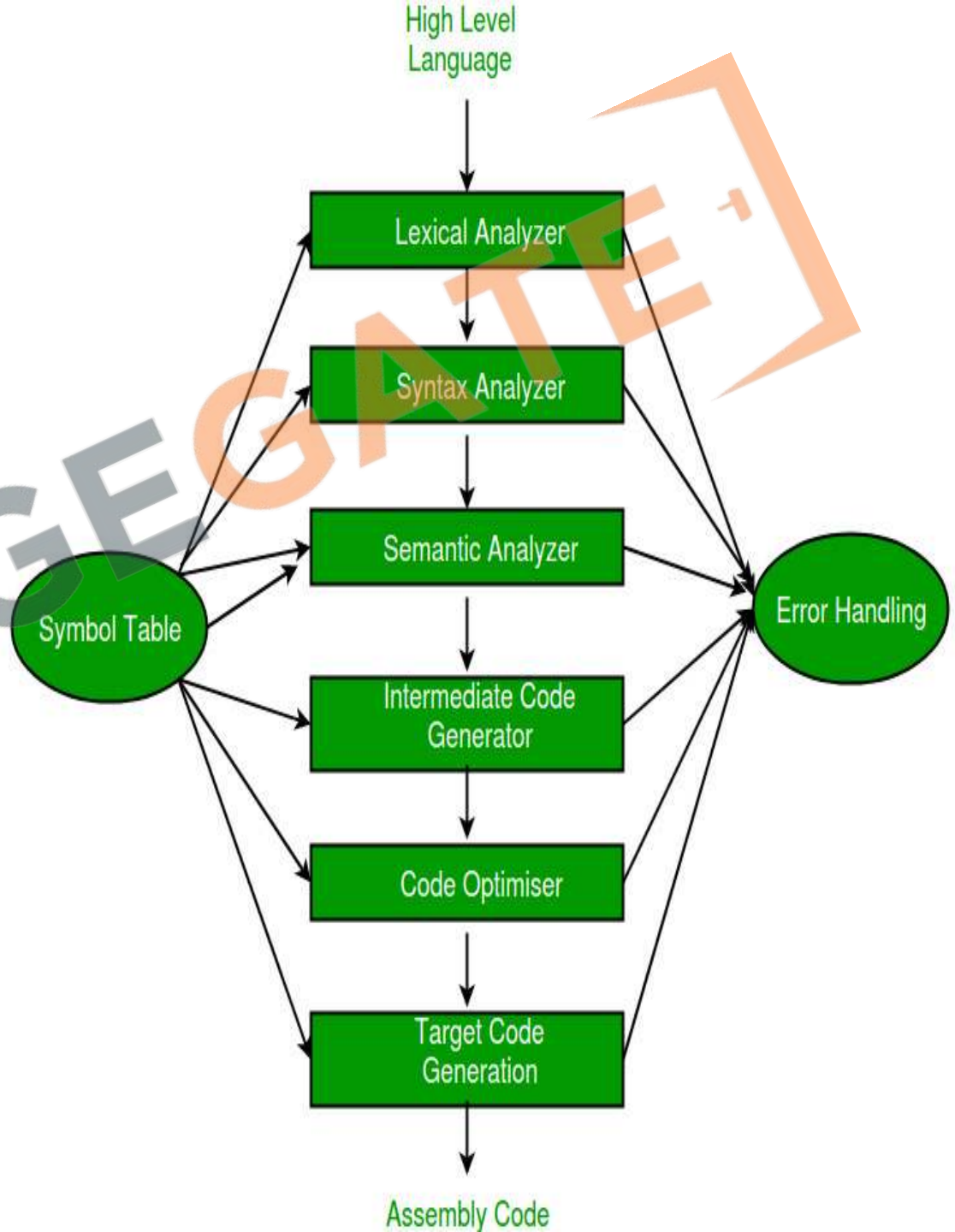
MOV R_1, Z
MUL R_1, 60
ADD R_1, Y
STORE X, R_1



KNOWLEDGEGATE
http://www.knowled

Symbol Table

- It is a data structure being used and maintained by the compiler, consists all the identifier's name along with their types.
- It helps the compiler to function smoothly by finding the identifiers quickly.



KNOWLEDGEGATE
http://www.knowledg

- The compiler has two modules namely front end and back end. Front-end constitutes of the Lexical analyser, semantic analyser, syntax analyser and intermediate code generator. And the rest are assembled to form the back end.

- In general Front end fill symbol table and back end uses it.

KNOWLEDGEGATE

(1) int x = 10;

Line No	Keyword	identifier	Constant	Operator
1	int	x	10	;

- Lexical analysis in the first phase to communicate with the symbol and the compiler generate the symbol table during the lexical analysis phase.
- Compiler is responsible to provide the memory for symbol table. at every phase if any new variable occurs, then they will be stored in the symbol table.
- Every phase of the compiler will be interacting with the symbol table.
- In general, during the first two phases, we store the information in the symbol table and in the memory and in the later phases, we make use of the information available in symbol table.

KNOWLEDGEGATE

- Information stored in the symbol table about identifier
  - name
  - type
  - scope
  - size
  - offset

- Other information in case of array, records and procedures etc.
  - array → size
  - records → column names
  - procedure → i/p parameter
  - functions → i/p, o/p parameter, actual, formal parameter

KNOWLEDGEGATE

- Operation on the symbol table: - there are 4 operation function that can be performed on symbol table
  - insert
  - lookup/search
  - Modify
  - delete

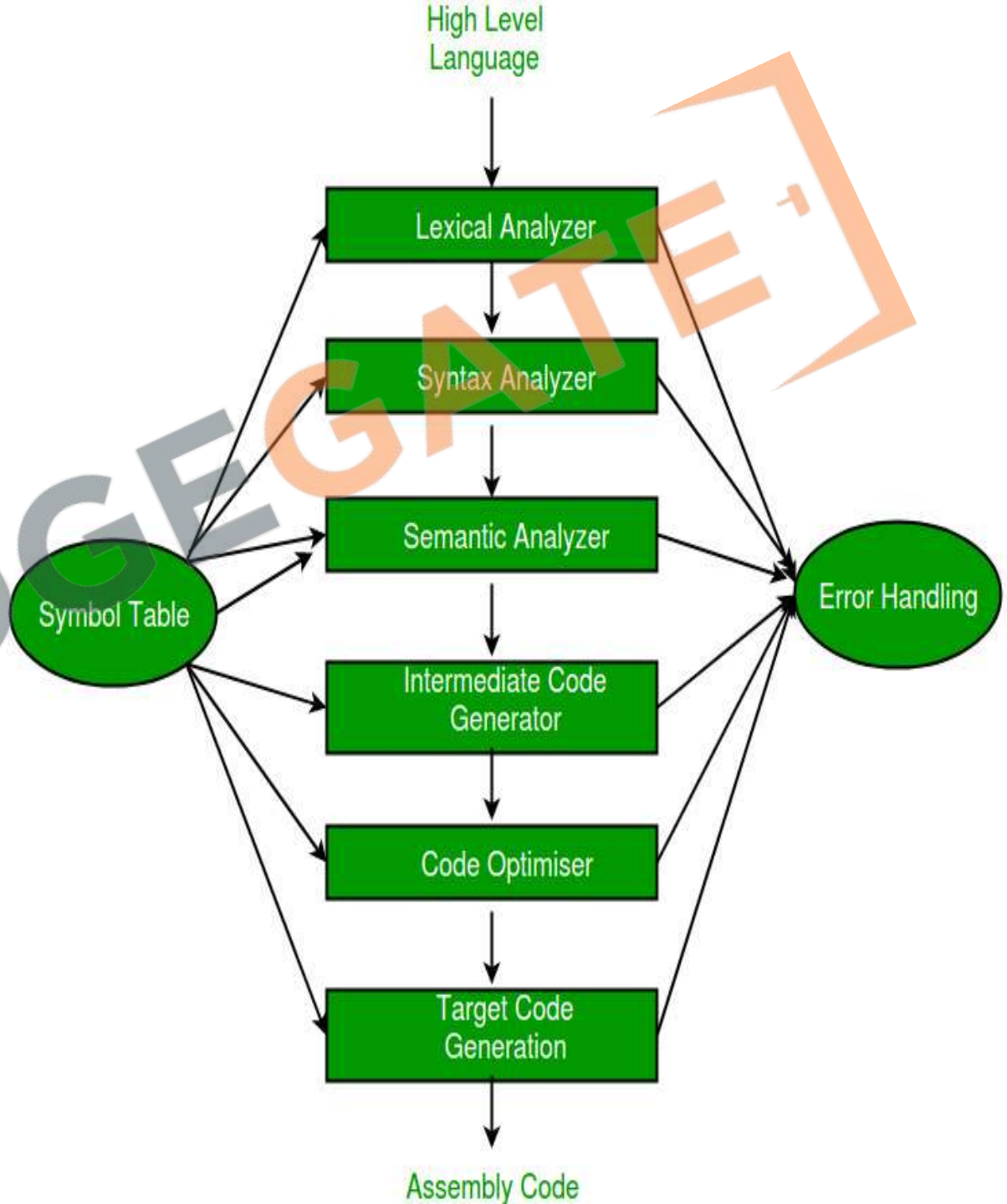
- Implementation of symbol tables can be done using anyone of the data structure
  - liner table
  - Binary Search Tree
  - Linked List
  - Hash Table (most popular)

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Error handler

- It is a sub routine to care take care of the continuation of compilation, even any error at any phase, i.e. error handler is responsible to continue the compilation process even any error occurs at phase 1 or phase 2 or phase 3.

- After phase 3, if the error handler object is empty, then the source code is free error and it can be converted into target code. if the error handler object is not empty after phase 3 then there will be some error at phase 1,2 or 3 then the error will be displayed.



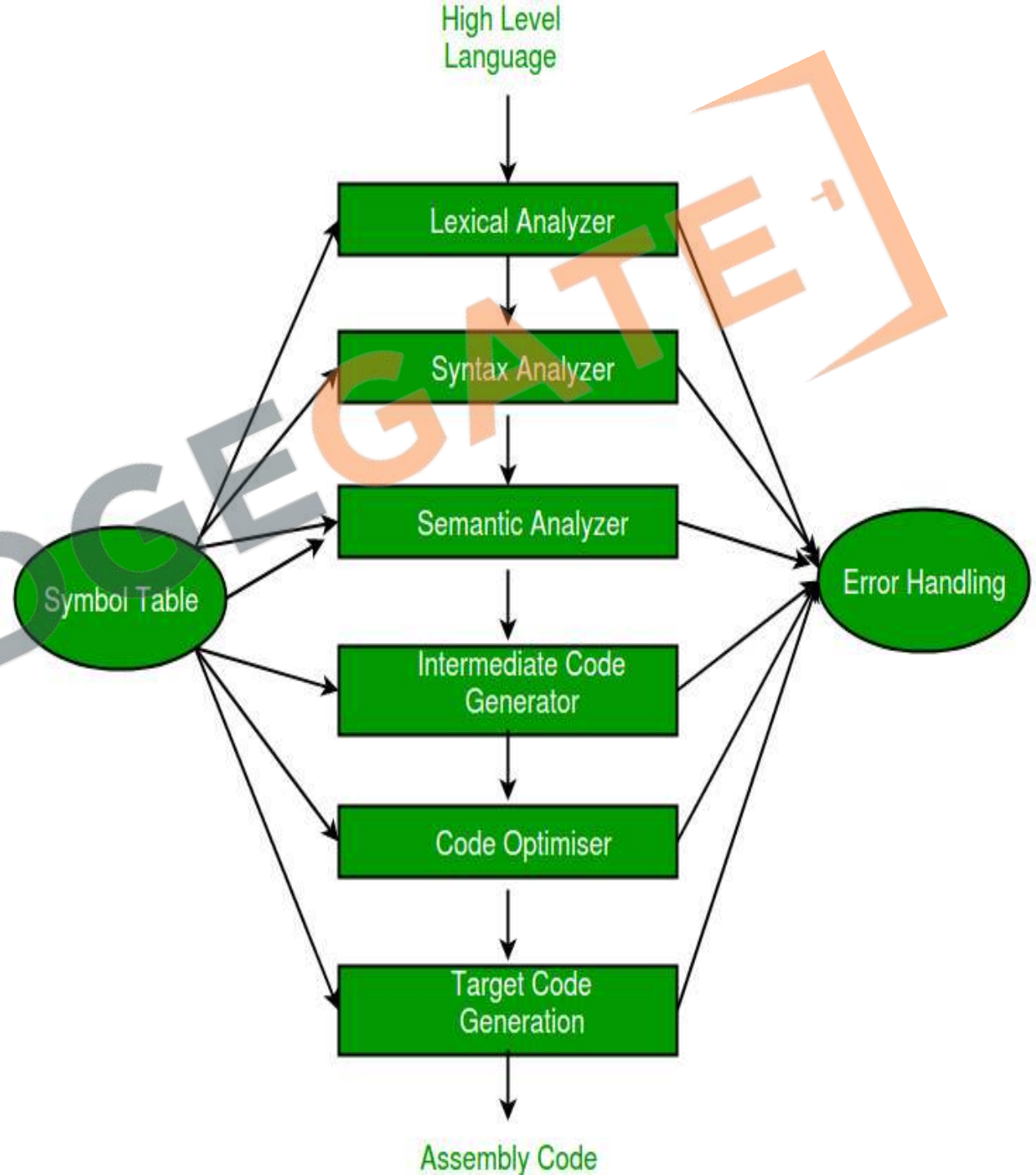
KNOWLEDGEGATE
http://www.knowle

- Compiler can handle, there types of errors: -
  - lexical error
  - syntax error
  - semantic error

- The error handles by compiler are known as exception and programmer is responsible to handle the exception.

- At the time of execution also, we can get some error they are called fatal error and system admin is responsible to handle fatal error.

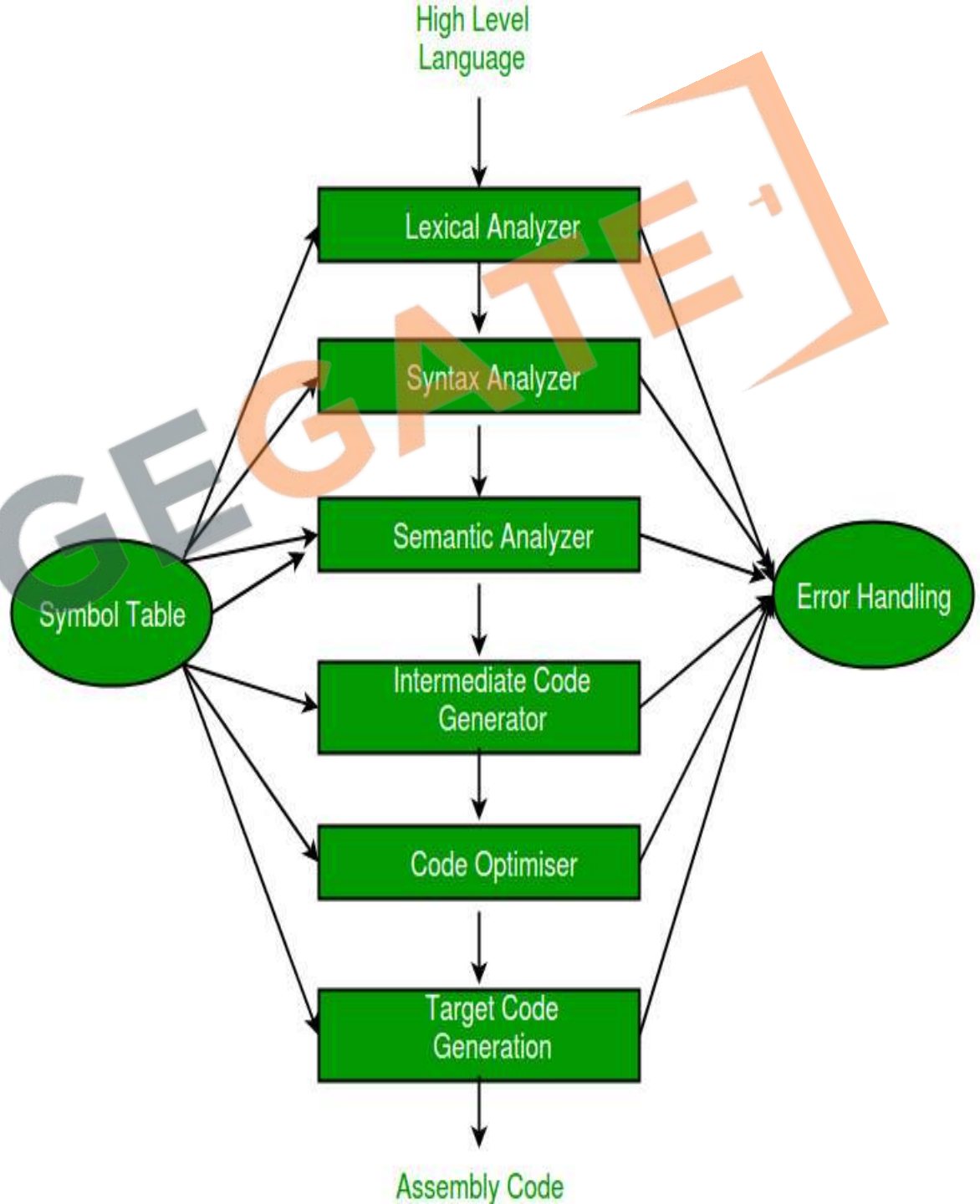
http://www.know



KNOWLEDGEGATE

Phases and Passes

- A compiler can have many phases and passes.
- **Pass**: A pass refers to the traversal of a compiler through the entire program.
- **Phase**: A phase of a compiler is a distinguishable stage, which takes input from the previous stage, processes and yields output that can be used as input for the next stage. A pass can have more than one phase.



KNOWLEDGEGATE
http://www.knowle

Single Pass Compiler

- **One-Time Processing**: Reads and processes the source code in a single pass, translating it directly into machine code.
- **Speed Advantage**: Faster than multi-pass compilers due to the single traversal of the source code.
- **Memory Efficient**: Uses less memory as it doesn't store extensive intermediate state information.
- **Optimization Limitations**: Offers limited optimization capabilities due to less context understanding of the entire program.
- **Best for Simple Languages**: Ideal for simpler programming languages where complex analysis and deep optimization are not required.
- **Early FORTRAN Compilers, Pascal (Original Version), Tiny C Compiler (TCC)**

KNOWLEDGEGATE

Multi-Pass Compiler

- A multi-pass compiler is a type of compiler that processes the source code multiple times before producing the final output. Here's a quick rundown in bullet points:
  - **Multiple Scans of Source Code**: It processes the source code in several passes, each for a specific analysis or transformation task.
  - **Advanced Optimization**: Multi-pass compilers perform complex optimizations by analyzing and re-analyzing the code, improving performance or minimizing resource usage.
  - **Context Awareness**: They have a deeper understanding of the program's context, which allows for more sophisticated error checking and optimization.
  - **Memory Usage**: They typically require more memory than single-pass compilers, as they need to maintain intermediate representations of the source code between passes.
  - **Examples**: Modern compilers like GCC (GNU Compiler Collection) and LLVM are multi-pass compilers that provide extensive optimization and support for complex language features.

KNOWLEDGE GATE

Aspect	Single-Pass Compiler	Multi-Pass Compiler
Compilation Speed	Generally faster as it compiles the source code in one pass.	Slower, as it goes through the source code multiple times.
Memory Usage	Less memory-intensive because it doesn't store much intermediate data.	More memory-intensive due to the need to store intermediate representations.
Error Detection	May not detect all errors in the first pass.	Better at error detection, as it analyzes the code in more depth over multiple passes.
Optimization	Limited optimization opportunities due to the single pass nature.	Better optimization, as multiple passes allow for more analysis and refinement.
Language Complexity	Suited for simpler languages with less complex syntax and semantics.	Better for complex languages, as it can handle intricate syntax and semantics more effectively.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

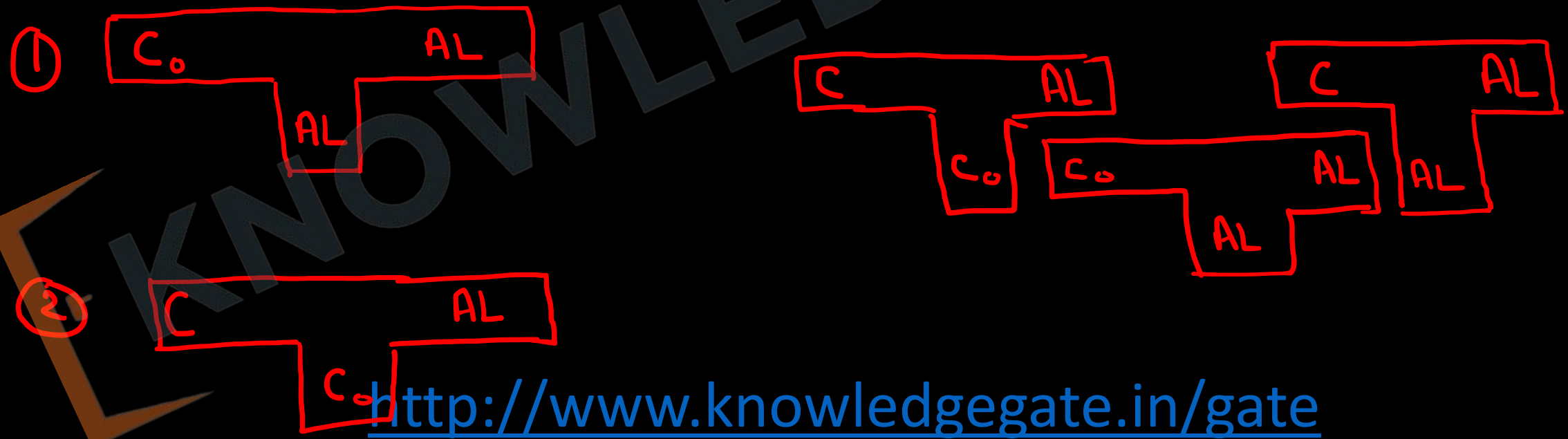
Bootstrapping

- Bootstrapping in compiler design is a fascinating and critical concept.
- **Historical Context**
  - **Early Stages**: Initially, compilers were written in assembly language or a low-level language specific to the hardware.
  - **Evolution**: As programming languages evolved, the need for writing compilers in a higher-level language became evident.
  - **Bootstrapping Emergence**: Bootstrapping was introduced as a solution to this need. It refers to writing a compiler in the same language it intends to compile.

KNOWLEDGEGATE

Concept

- **Initial Step**: A simple compiler is first written in a low-level language. This is often termed as the "bootstrap compiler."
- **Self-Compiling**: The compiler is then rewritten in its own higher-level language and compiled using the bootstrap compiler.
- **Iteration**: This process can be iterated, with each new version of the compiler used to compile its next version.



KNOWLEDGEGATE

- Bootstrapping is the process of writing a compiler for a programming language using the language itself. In other words, it is the process of using a compiler written in a particular programming language to compile a new version of the compiler written in the same language.

- The process of bootstrapping typically involves several stages. In the first stage, a minimal version of the compiler is written in a different language, such as assembly language or C. This minimal version of the compiler is then used to compile a slightly more complex version of the compiler written in the target language. This process is repeated until a fully functional version of the compiler is written in the target language.

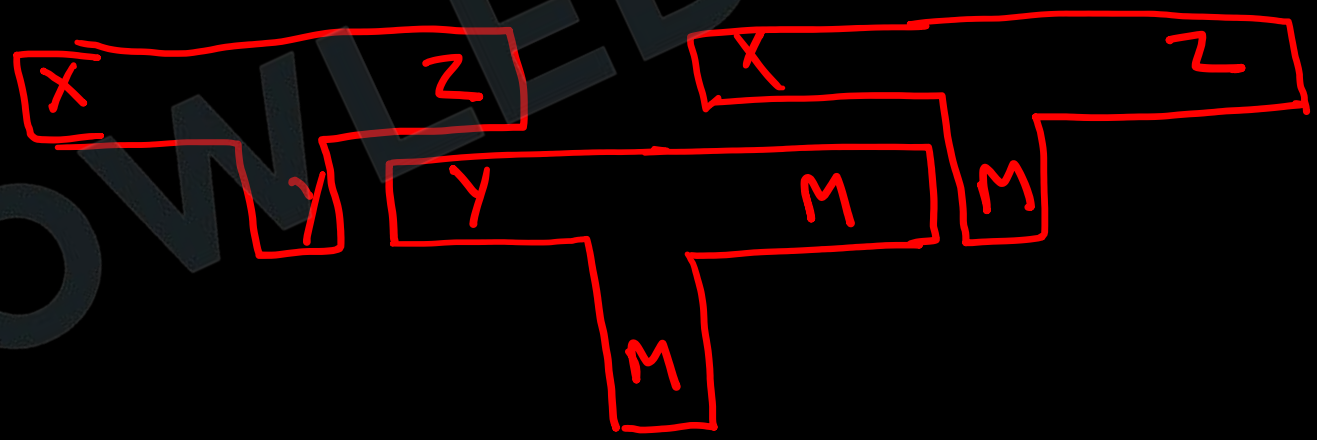
KNOWLEDGE GATE

- One advantage is that it ensures that the compiler is compatible with the language it is designed to compile. so it is better able to understand and interpret the syntax and semantics of the language.
- Another advantage is that it allows for greater control over the optimization and code generation process.
- One disadvantage is that it can be a time-consuming process, especially for complex languages or compilers. It can also be more difficult to debug a bootstrapped compiler, since any errors or bugs in the compiler will affect the subsequent versions of the compiler.

KNOWLEDGEGATE

Cross Compiler

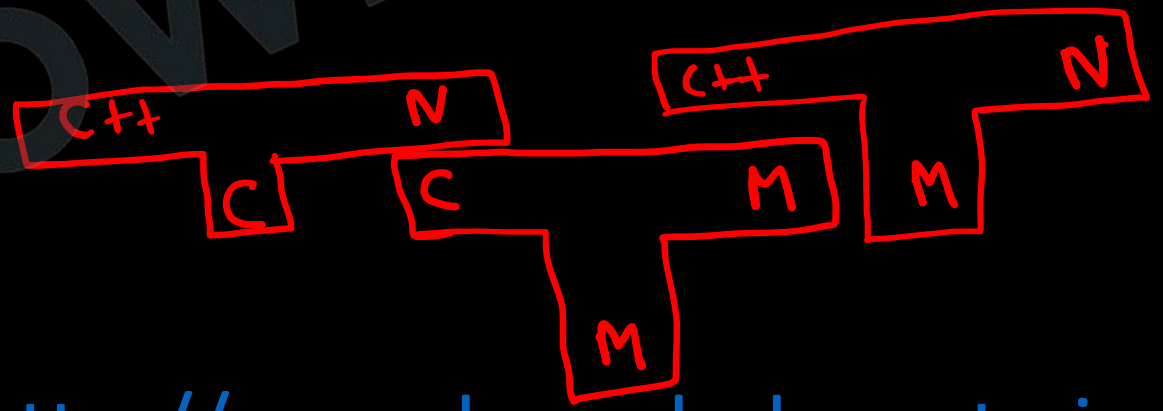
- Suppose we want to write a cross compiler for new language X. The implementation language of this compiler is say Y and the target code being generated is in language Z. That is, we create XYZ. Now if existing compiler Y runs on machine M and generates code for M then it is denoted as YMM. Now if we run XYZ using YMM then we get a compiler XMZ. That means a compiler for source language X that generates a target code in language Z and which runs on machine M.



KNOWLEDGE GATE

Cross Compiler

- **Early Computing Era**: Initially, software was often developed and executed on the same type of machine. This limited software portability across different hardware.
- **Growth of Diverse Hardware**: With the proliferation of various computing systems, including microprocessors and embedded systems, the need for software compatibility across different platforms increased.
- **Development of Cross Compilers**: Cross compilers were developed to address this need. They allowed developers to write code on one machine (the host) and execute it on another (the target), which could have a completely different architecture.



KNOWLEDGEGATE

DETERMINISTIC FINITE AUTOMATA

A **deterministic finite automaton (DFA)** is defined by 5-tuple (Q,Σ,δ,S,F) where:

- Q is a finite and non-empty set of states
- Σ is a finite non-empty set of finite input alphabet
- δ is a transition function, ($\delta: Q \times \Sigma \rightarrow Q$)
- S is **initial state** (always one) ($S \in Q$)
- F is a set of final states ($F \subseteq Q$) ($0 <= |F| <= N$, where n is the number of states)

KNOWLEDGEGATE

Q Construct the NFA for the regular expression $a|abb|a^*b^+$ by using Thompson's construction methodology**?**

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Draw NFA for the regular expression $ab^*|ab$ **?**

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Construct the minimized DFA for the regular expression (0 + 1)*(0 + 1) 10 ?

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

NFA and DFA Equivalence

- In this topic we will be learning about the equivalence of NFA and DFA and how an NFA can be converted to equivalent DFA. Let us take an example and understand the conversion.
- Since every NFA and DFA has equal power that means, for every language if a NFA is possible, then DFA is also possible.
- So, every NFA can be converted to DFA.
- The process of conversion of an NFA into a DFA is called Subset Construction.
- If NFA have ‘n’ states which is converted into DFA which ‘m’ states than the relationship between n and m will be
- $1 <= m <= 2^n$

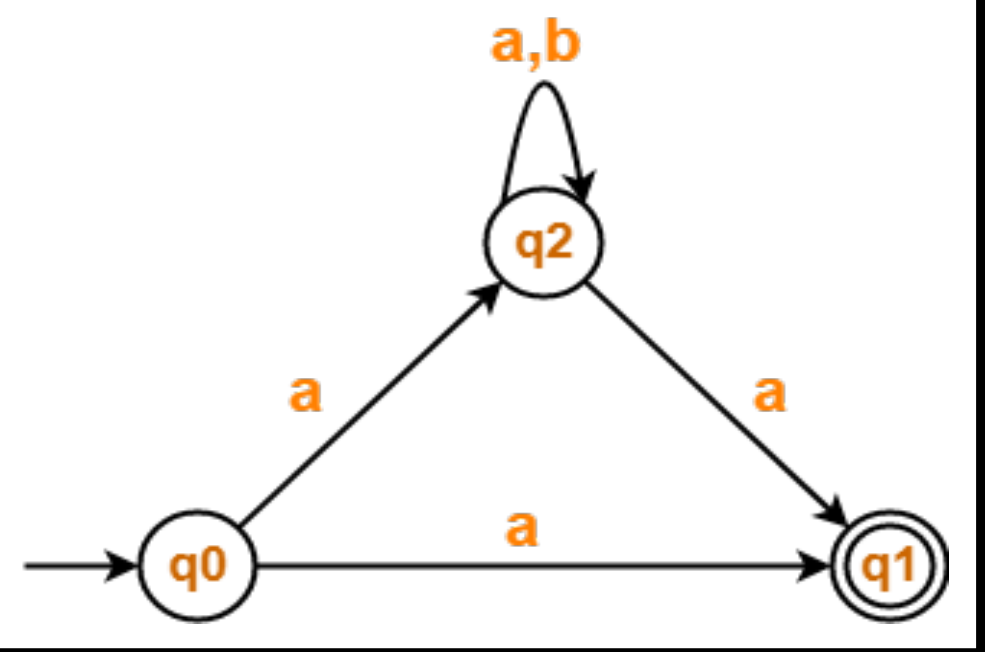
KNOWLEDGEGATE

Procedure for Conversion

- There lies a fixed algorithm for the NFA and DFA conversion. Following things must be considered
  - Initial state will always remain same.
  - Start the construction of δ' with the initial state & continue for every new state that comes under the input column and terminate the process whenever no new state appears under the input column.
  - Every subset of states that contain the final state of the NFA is a final state in the resulting DFA.
  - $\delta'(q_0, q_1, q_2, q_{3\text{--------}}, q_{n-1}, a) = \bigcup_{i=0}^{i=n-1} \delta(q_i, a)$

KNOWLEDGE GATE

http://www.knowledgegate.in/gate



KNOWLEDGEGATE

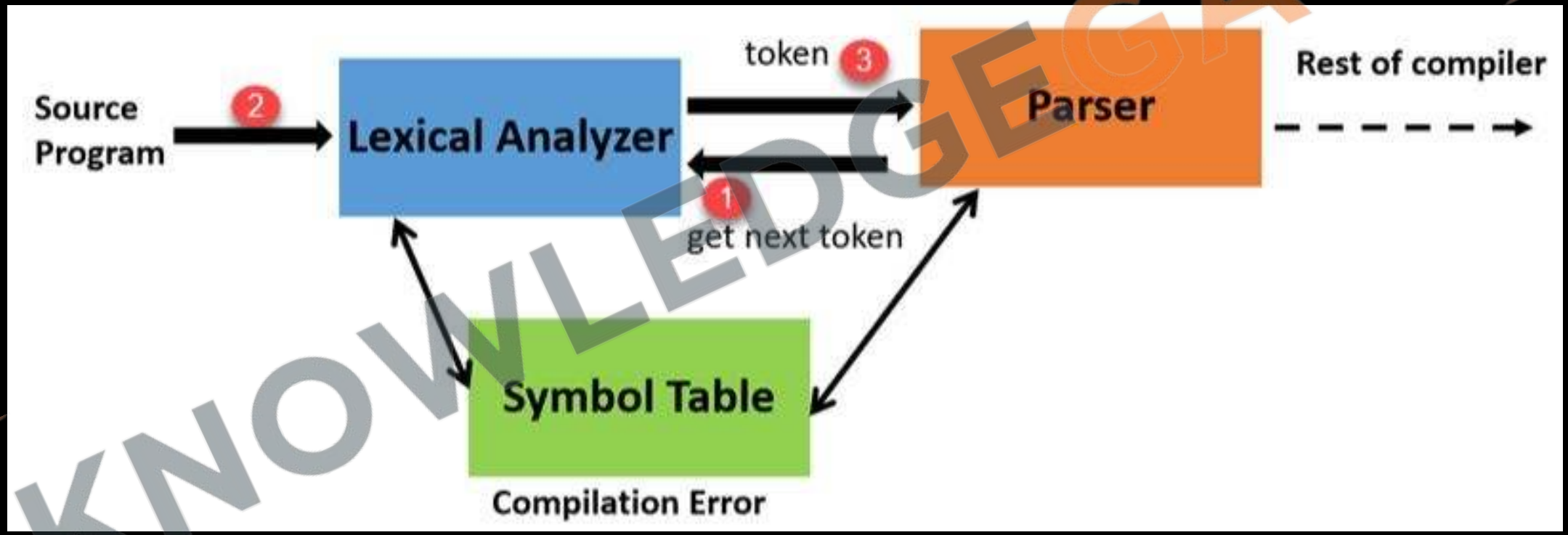
http://www.knowledgegate.in/gate

Aspect	DFA (Deterministic Finite Automata)	NDFA (Nondeterministic Finite Automata)
State Transition	On each input symbol, transitions to exactly one state.	Can transition to multiple states or none on the same input symbol.
Determinism and Uniqueness	Each state has a unique transition for each input symbol.	A state can have multiple transitions for the same input symbol.
Computation Path	Always has a single, unique computation path for any input string.	May have multiple computation paths for the same input string.
Ease of Construction	Generally simpler and more straightforward to construct.	Can be more complex to construct due to non-determinism.
Acceptance of Input	Accepts an input if it reaches a final state after processing all input symbols.	Accepts an input if at least one computation path reaches a final state.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Lexical Analyzer

- **lexical analysis**, **lexing** or **tokenization** is the process of converting a sequence of characters (such as in a computer program or web page) into a sequence of tokens.



KNOWLEDGEGATE

Lexical Analyzer

- A *lexeme* is a sequence of characters in the source program that matches the pattern for a token and is identified by the lexical analyzer as an instance of that token. Where a token is a strings with an assigned and thus identified meaning. Actual representation or stream of characters is called as Lexemes; Logical meaning of Lexemes is known as Tokens.

- Lexing can be divided into two stages: the *scanning*, which segments the input string into syntactic units called *lexemes* and categorizes these into token classes; and the *evaluating*, which converts lexemes into processed values.

Lexeme	token
while	while
(	lparen
y	identifier
>=	Comparison
t	identifier
)	Rparen
y	identifier
=	Assignment
y	identifier
-	Arithmetic
3	integer
;	Finish of a statement

KNOWLEDGEGATE
http://www.know

- x = a + b * 2;

- [(identifier, x), (operator, =), (identifier, a), (operator, +), (identifier, b), (operator, *), (literal, 2), (separator, ;)]

KNOWLEDGEGATE

Q Identify the meaning of the rule implemented by Lexer using regular expression for identifying tokens of a password?

Regular Expression → $A(A+S+D)^3\,(A+S+D+\in)^4$

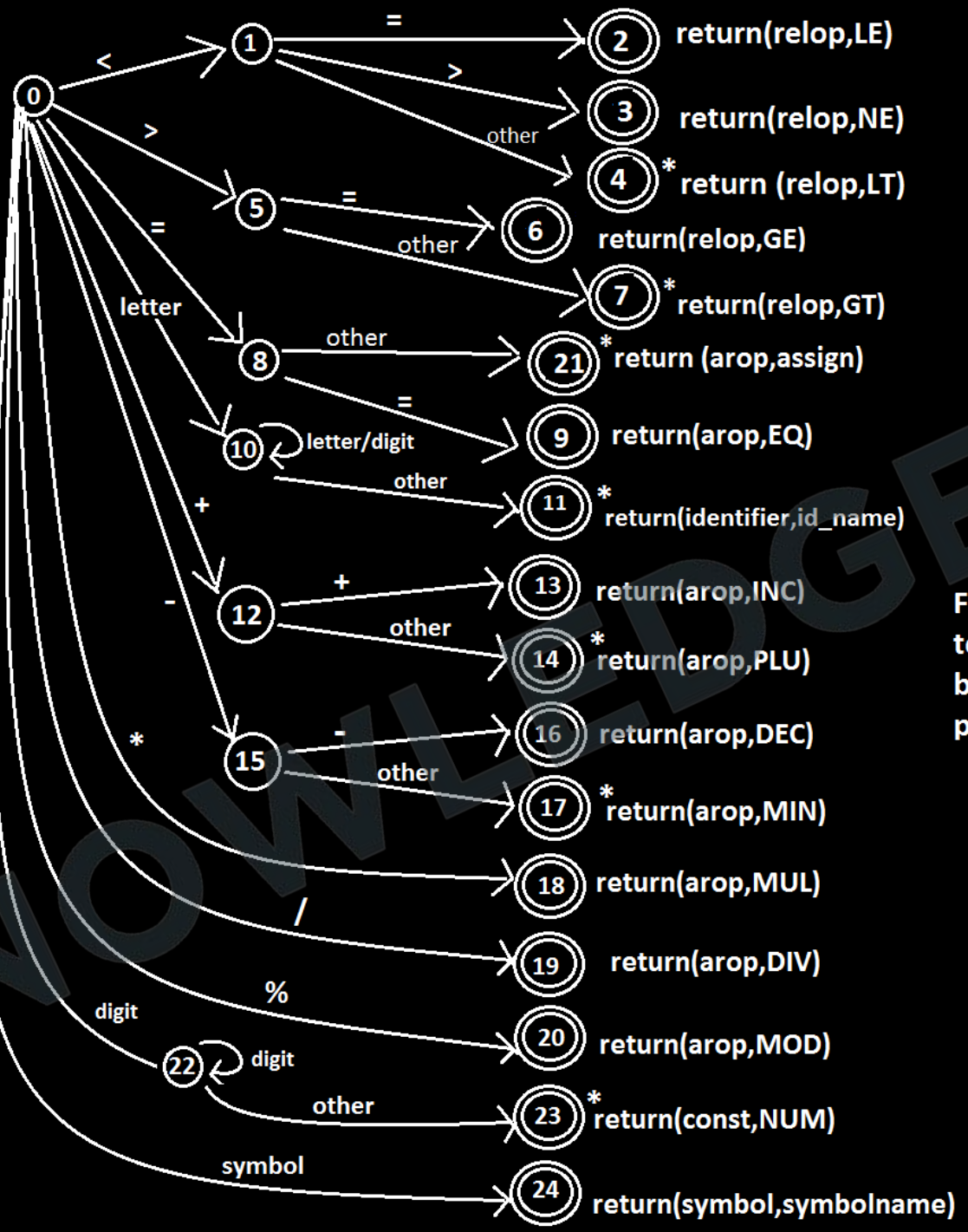
KNOWLEDGEGATE

Implementation of Lexical analyzer

Lexical analyzer can be implemented in following step :

- Input to the lexical analyzer is a source program.
- By using input buffering scheme, it scans the source program.
- Regular expressions are used to represent the input patterns.
- Now this input pattern is converted into NFA by using finite automation machine.
- This NFA are then converted into DFA and DFA are minimized by using different method of minimization.
- The minimized DFA are used to recognize the pattern and broken into lexemes.
- Each minimized DFA is associated with a phase in a programming language which will evaluate the lexemes that match the regular expression.
- The tool then constructs a state table for the appropriate finite state machine and creates program code which contains the table, the evaluation phases, and a routine which uses them appropriately.

KNOWLEDGE GATE



Final states marks with * requires to the forward pointer (fwdptr) to be retracted by one character position

KNOWLEDGEGATE

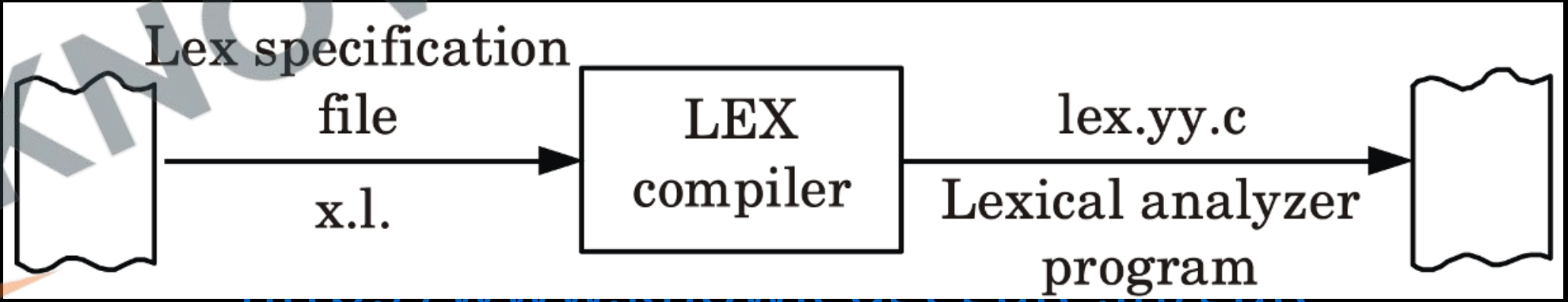
lexical Analyzer Generator

- For efficient design of compiler, various tools are used to automate the phases of compiler. The lexical analysis phase can be automated using a tool called LEX.
- LEX is a Unix utility which generates lexical analyzer.
- The lexical analyzer is generated with the help of regular expressions.
- LEX lexer is very fast in finding the tokens as compared to handwritten LEX program in C.
- LEX scans the source program in order to get the stream of tokens and these tokens can be related together so that various programming structure such as expression, block statement, control structures, procedures can be recognized.

KNOWLEDGEGATE

LEX compiler

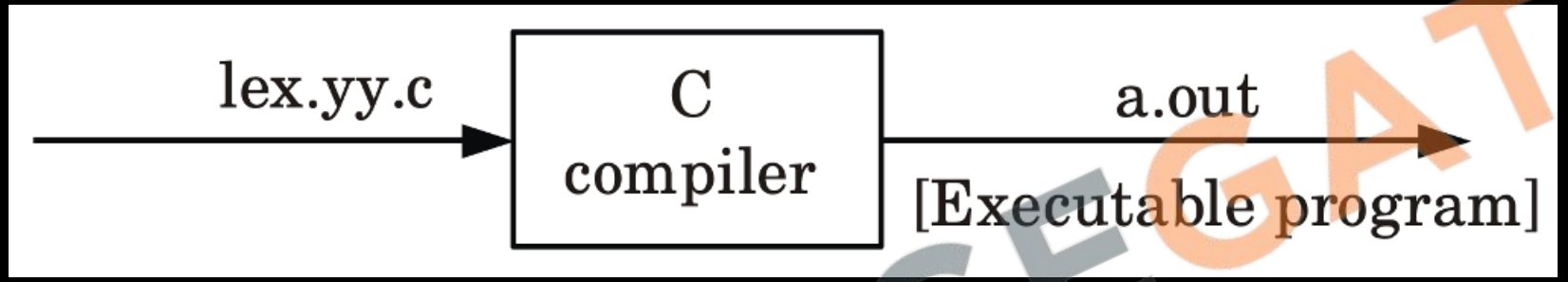
- Automatic generation of lexical analyzer is done using LEX programming language.
- The LEX specification file can be denoted using the extension .l (often pronounced as dot L).
- For example, let us consider specification file as x.l.
- This x.l file is then given to LEX compiler to produce lex.yy.c . This lex.yy.c is a C program which is actually a lexical analyzer program.
- The LEX specification file stores the regular expressions for the token and the lex.yy.c file consists of the tabular representation of the transition diagrams constructed for the regular expression.
- In specification file, LEX actions are associated with every regular expression.
- These actions are simply the pieces of C code that are directly carried over to the lex.yy.c.



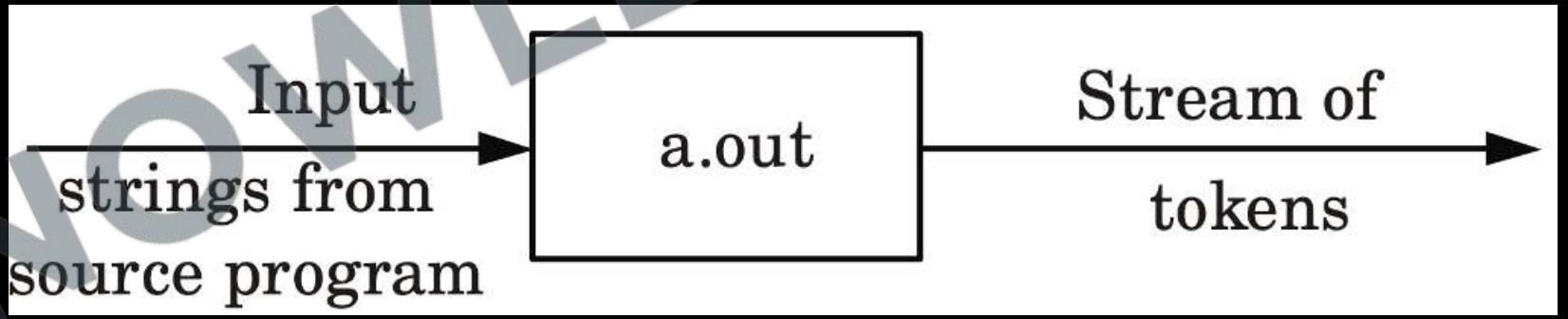
KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Generation of lexical analyzer using LEX

- Finally, the C compiler compiles this generated lex.yy.c and produces an object program a.out.



- When some input stream is given to a.out then sequence of tokens gets generated.



KNOWLEDGEGATE

Components of LEX program

The LEX program consists of three parts :

Declaration section :

- In the declaration section, declaration of variable constants can be done.
- Some regular definitions can also be written in this section.
- The regular definitions are basically components of regular expressions.

KNOWLEDGEGATE

Components of LEX program

- **Rule section :**
  - The rule section consists of regular expressions with associated actions. These translation rules can be given in the form as :
  - R_1 {action$_1$}
  - R_2 {action$_2$}
  - .
  - .
  - R_n {action$_n$}
  - Where each R_i is a regular expression and each action$_i$ is a program fragment describing what action is to be taken for corresponding regular expression.
  - These actions can be specified by piece of C code.

KNOWLEDGEGATE

Components of LEX program

- **Auxiliary procedure section :**
- In this section, all the procedures are defined which are required by the actions in the rule section.
- This section consists of two functions :
  - main() function
  - yywrap() function

KNOWLEDGEGATE

- Secondary Function of Lexical Analyser are: -
  - Removal of Comments lines

Start here | *main.c

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a; // Declare two variables
    printf("Enter any number\n\n");
    scanf("%d",&a);
/* After declaration, we print our message on the screen*/
/* and in the 3rd step, save the address of a */
    return 0;
}
```

KNOWLEDGEGATE

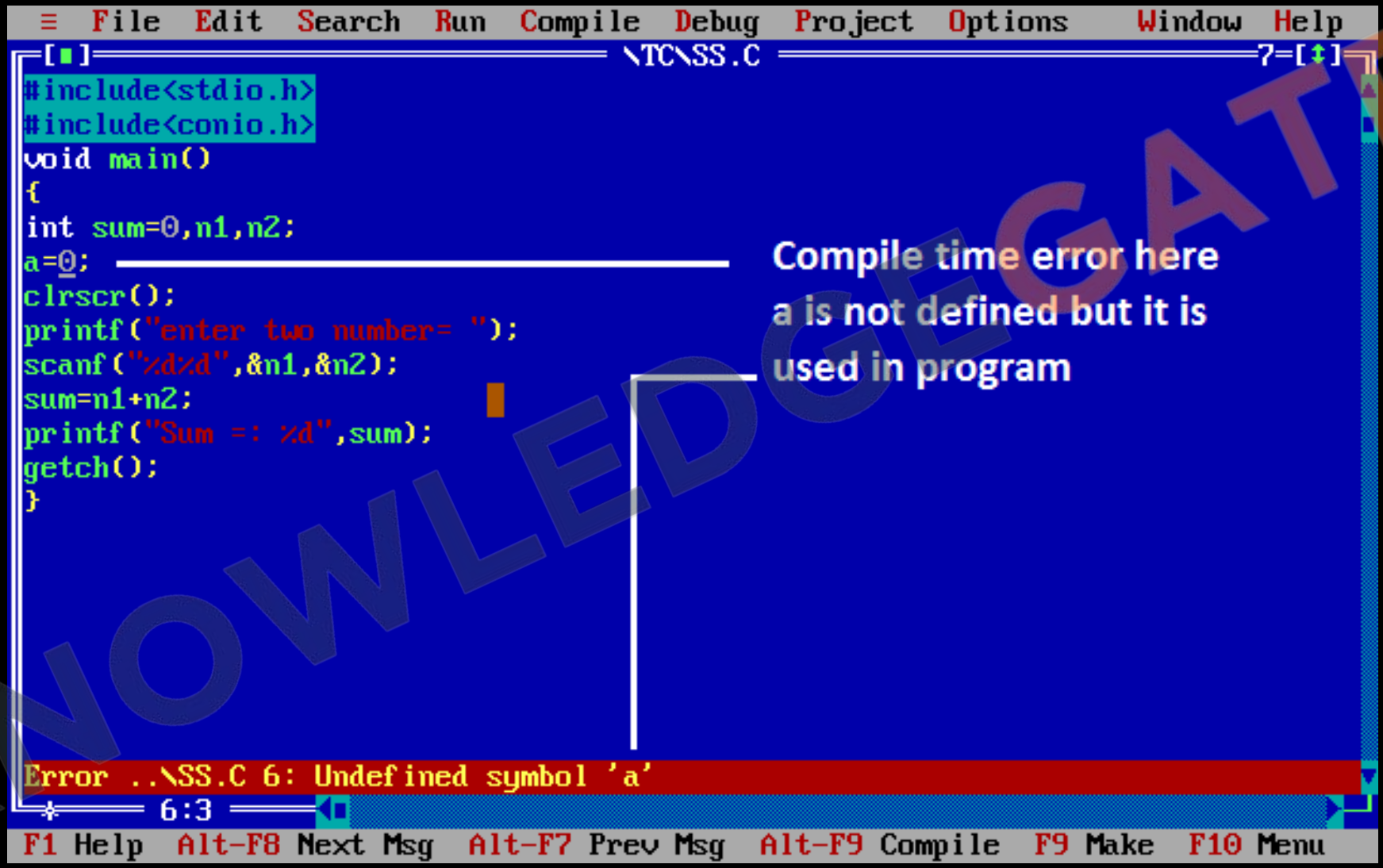
- Removal of White space characters

WHITE SPACE CHARACTERS

\b	blank space	\t	horizontal tab	\v	vertical tab
\\	Back slash	\'	Single quote	\"	Double quote
\r	carriage return	\f	form feed	\n	new line
\?	Question mark	\0	Null	\a	Alarm (bell)

KNOWLEDGEGATE

- Co-relating with Error Messages along with line number.



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Formal Grammar

A phrase-structure grammar (or simply a grammar) is a 4-tuple (V_N, ∑, P, S), where

- V_N is a finite nonempty set whose elements are called variables,
- ∑ is a finite nonempty set whose elements are called terminals, $V_N \cap \sum = \Phi$.
- S is a special variable (i.e., an element of V_N ($S \in V_n$)) called the start symbol. Like every automaton has exactly one initial state, similarly every grammar has exactly one start symbol.
- P is a finite set whose elements are $\alpha \rightarrow \beta$. where α and β are strings on $V_N \cup \sum$. α has at least one symbol from V_N, the element of P are called productions or production rules or rewriting rules. $\{\Sigma \cup V_n\}^*$ some writer refers it as total alphabet

KNOWLEDGE GATE

For a formal valid production

$\alpha \rightarrow \beta$

$\alpha \in \{\Sigma \cup V_n\}^* V_n \{\Sigma \cup V_n\}^*$

$\beta \in \{\Sigma \cup V_n\}^*$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

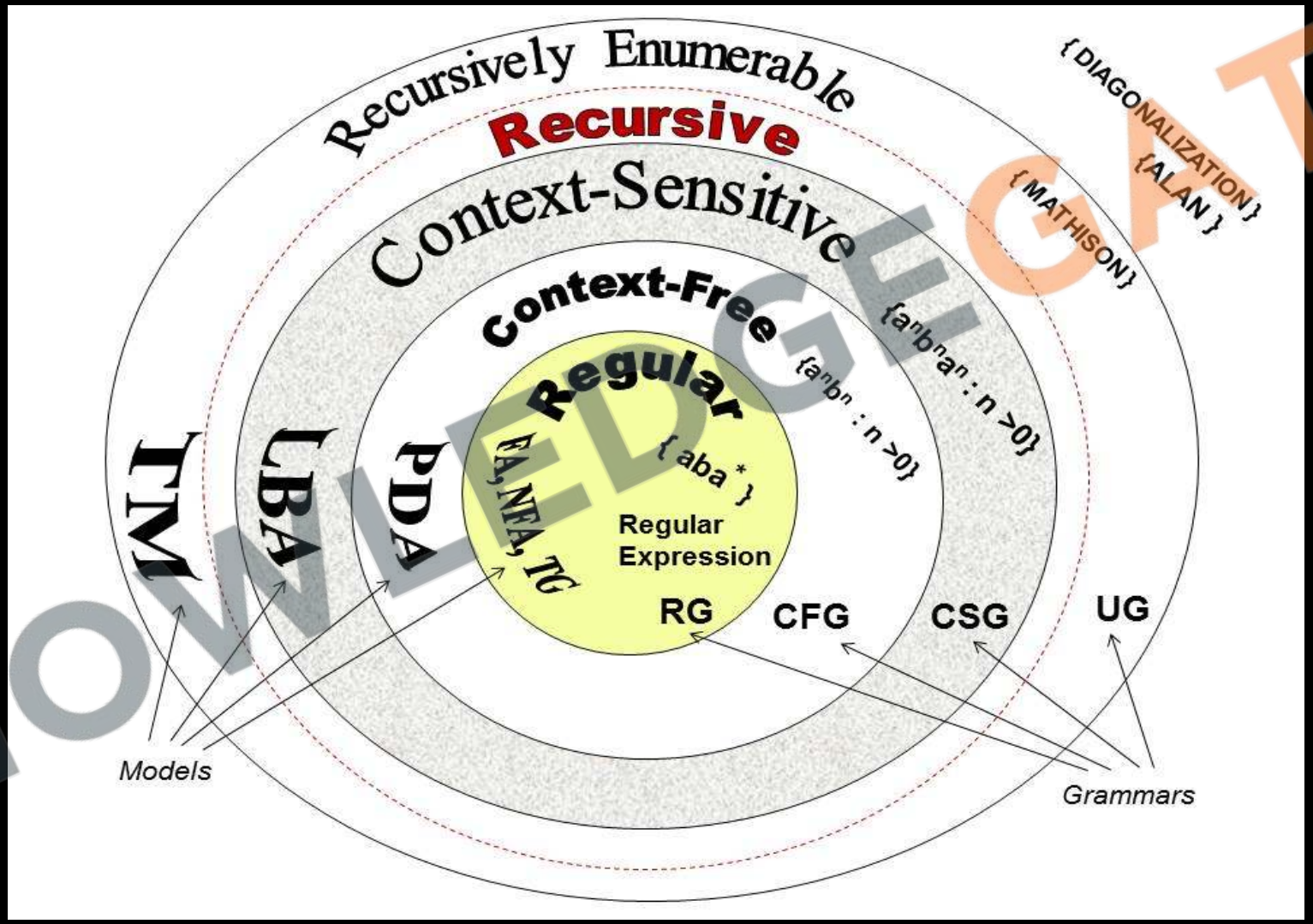
If $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1, S \rightarrow \Lambda\}, S)$, find $L(G)$.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

LANGUAGES AND AUTOMATA

- Following are the machines that accepts the following grammars.



KNOWLEDGEGATE

Type 2 Grammar

- Also known as Context Free Grammar, which will generate context free language that will be accepted by push down automata. (NPDA default case)

- if there is a production, from

$$\alpha \rightarrow \beta$$

$$\alpha \in V_n \qquad |\alpha| = 1$$

$$\beta \in \{\Sigma \cup V_n\}^*$$

- In other words, the L.H.S. has no left context or right context.

- A grammar is called a type 2 grammar if it contains only type 2 productions.

- Eg ALGOL 60, PASCAL

KNOWLEDGEGATE

Formal grammar and its application to syntax analysis

- Formal grammar represents the specification of programming language with the use of production rules.
- The syntax analyzer basically checks the syntax of the language.
- A syntax analyzer takes the tokens from the lexical analyzer and groups them in such a way that some programming structure can be recognized.
- After grouping the tokens if at all any syntax cannot be recognized then syntactic error will be generated.
- This overall process is called syntax checking of the language.
- This syntax can be checked in the compiler by writing the specifications.
- Specification tells the compiler how the syntax of the programming language should be.

KNOWLEDGEGATE

BNF notation

- Backus-Naur Form (BNF) is a notation technique used to express the grammar of a computer language. It's a formal mathematical way to describe a language, which is particularly useful in the context of programming languages. Here's a simple explanation:

- **Symbols**: BNF uses two types of symbols:
  - **Terminal Symbols**: These are the basic characters or strings in the language, like numbers, letters, or specific words.
  - **Non-terminal Symbols**: These represent a set of strings, and they're defined by writing rules in BNF.

- **Production Rules**: A language in BNF is defined by production rules. Each rule has a left-hand side (a non-terminal symbol) and a right-hand side, which is a sequence of terminal and/or non-terminal symbols. The rule shows how you can replace the non-terminal with that sequence.

KNOWLEDGEGATE

- **Example**: Let's consider a simple arithmetic expression grammar:
  - Expression → Expression + Term | Term
  - Term → Term * Factor | Factor
  - Factor → (Expression) | Number
  - Number → Digit | Number Digit
  - Digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
- In this example, "Expression", "Term", "Factor", "Number", and "Digit" are non-terminal symbols, while '+', '*', '(', ')', and the digits are terminal symbols.

KNOWLEDGEGATE

1. *program* → *declarationList*
2. *declarationList* → *declarationList declaration* | *declaration*
3. *declaration* → *varDeclaration* | *funDeclaration*

---

4. *varDeclaration* → *typeSpecifier varDeclList* **;**
5. *scopedVarDeclaration* → *scopedTypeSpecifier varDeclList* **;**
6. *varDeclList* → *varDeclList* **,** *varDeclInitialize* | *varDeclInitialize*
7. *varDeclInitialize* → *varDeclId* | *varDeclId* **:** *simpleExpression*
8. *varDeclId* → **ID** | **ID [NUMCONST]**
9. *scopedTypeSpecifier* → **static** *typeSpecifier* | *typeSpecifier*
10. *typeSpecifier* → **int** | **bool** | **char**

---

11. *funDeclaration* → *typeSpecifier* **ID** **(** *params* **)** *statement* | **ID** **(** *params* **)** *statement*
12. *params* → *paramList* | ϵ
13. *paramList* → *paramList* **;** *paramTypeList* | *paramTypeList*
14. *paramTypeList* → *typeSpecifier paramIdList*

KNOWLEDGEGATE

15. *paramIdList* → *paramIdList* , *paramId* | *paramId*

16. *paramId* → **ID** | **ID** [ ]

---

17. *statement* → *expressionStmt* | *compoundStmt* | *selectionStmt* | *iterationStmt* | *returnStmt* | *breakStmt*

18. *expressionStmt* → *expression* ; | ;

19. *compoundStmt* → { *localDeclarations statementList* }

20. *localDeclarations* → *localDeclarations scopedVarDeclaration* | **ϵ**

21. *statementList* → *statementList statement* | **ϵ**

22. *elsifList* → *elsifList* **elsif** *simpleExpression* **then** *statement* | **ϵ**

23. *selectionStmt* → **if** *simpleExpression* **then** *statement elsifList* | **if** *simpleExpression* **then** *statement elsifList* **else** *statement*

24. *iterationRange* → **ID** = *simpleExpression* .. *simpleExpression* | **ID** = *simpleExpression* .. *simpleExpression* : *simpleExpression*

25. *iterationStmt* → **while** *simpleExpression* **do** *statement* | **loop forever** *statement* | **loop** *iterationRange* **do** *statement*

26. *returnStmt* → **return** ; | **return** *expression* ;

27. *breakStmt* → **break** ;

KNOWLEDGEGATE

28. *expression* → *mutable* = *expression* | *mutable* += *expression* | *mutable* −= *expression* | *mutable* *= *expression* | *mutable* /= *expression* | *mutable* ++ | *mutable* −− | *simpleExpression*

29. *simpleExpression* → *simpleExpression* **or** *andExpression* | *andExpression*

30. *andExpression* → *andExpression* **and** *unaryRelExpression* | *unaryRelExpression*

31. *unaryRelExpression* → **not** *unaryRelExpression* | *relExpression*

32. *relExpression* → *sumExpression relop sumExpression* | *sumExpression*

33. *relop* → <= | < | > | >= | == | !=

34. *sumExpression* → *sumExpression sumop mulExpression* | *mulExpression*

35. *sumop* → + | −

KNOWLEDGEGATE

36. *mulExpression* → *mulExpression mulop unaryExpression* | *unaryExpression*

37. *mulop* → * | / | %

38. *unaryExpression* → *unaryop unaryExpression* | *factor*

39. *unaryop* → − | * | ?

40. *factor* → *immutable* | *mutable*

41. *mutable* → **ID** | *mutable* [*expression*]

42. *immutable* → (*expression*) | *call* | *constant*

43. *call* → **ID** (*args*)

44. *args* → *argList* | **ϵ**

45. *argList* → *argList* , *expression* | *expression*

46. *constant* → **NUMCONST** | **CHARCONST** | **STRINGCONST** | **true** | **false**

KNOWLEDGEGATE

- **Use in Parsing**: BNF is widely used in the design and implementation of compilers and interpreters for programming languages. It helps in building the syntax tree of a program by parsing the source code according to the grammar rules defined in BNF.
- **Readability**: BNF provides a concise and readable way to specify the syntax of a language. It's easier to understand and modify compared to directly writing parser code.
- **Extensions**: There are also extensions to BNF, like Extended BNF (EBNF), which provides more expressive power by adding more constructs like optional elements, repetitions, and choices.

KNOWLEDGE GATE

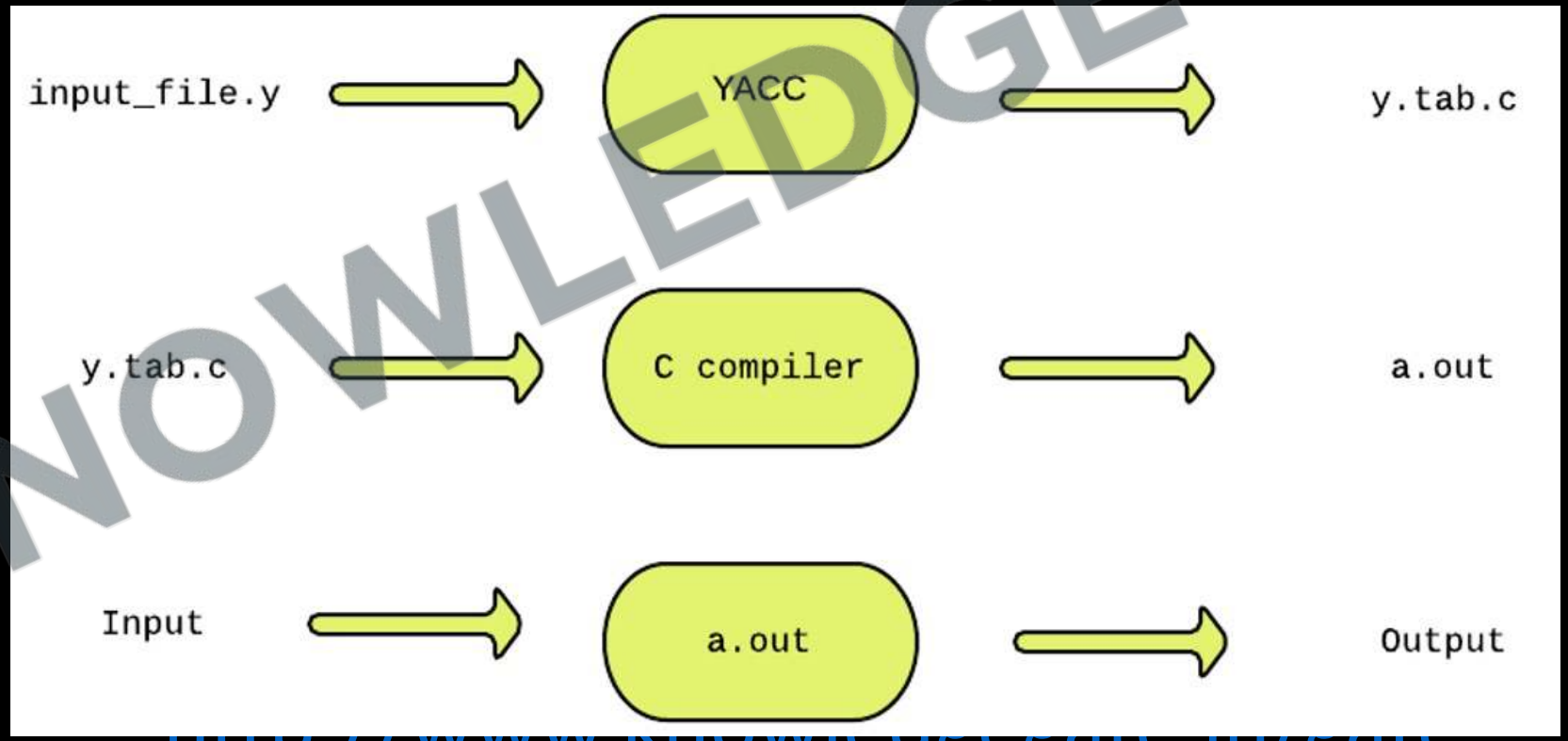
Ambiguous grammar: - The grammar CFG is said to be ambiguous if there are more than one derivation tree for any string i.e. if there exist more than one derivation tree (LMDT or RMDT), the grammar is said to be ambiguous.

S → aS/Sa/a

KNOWLEDGEGATE

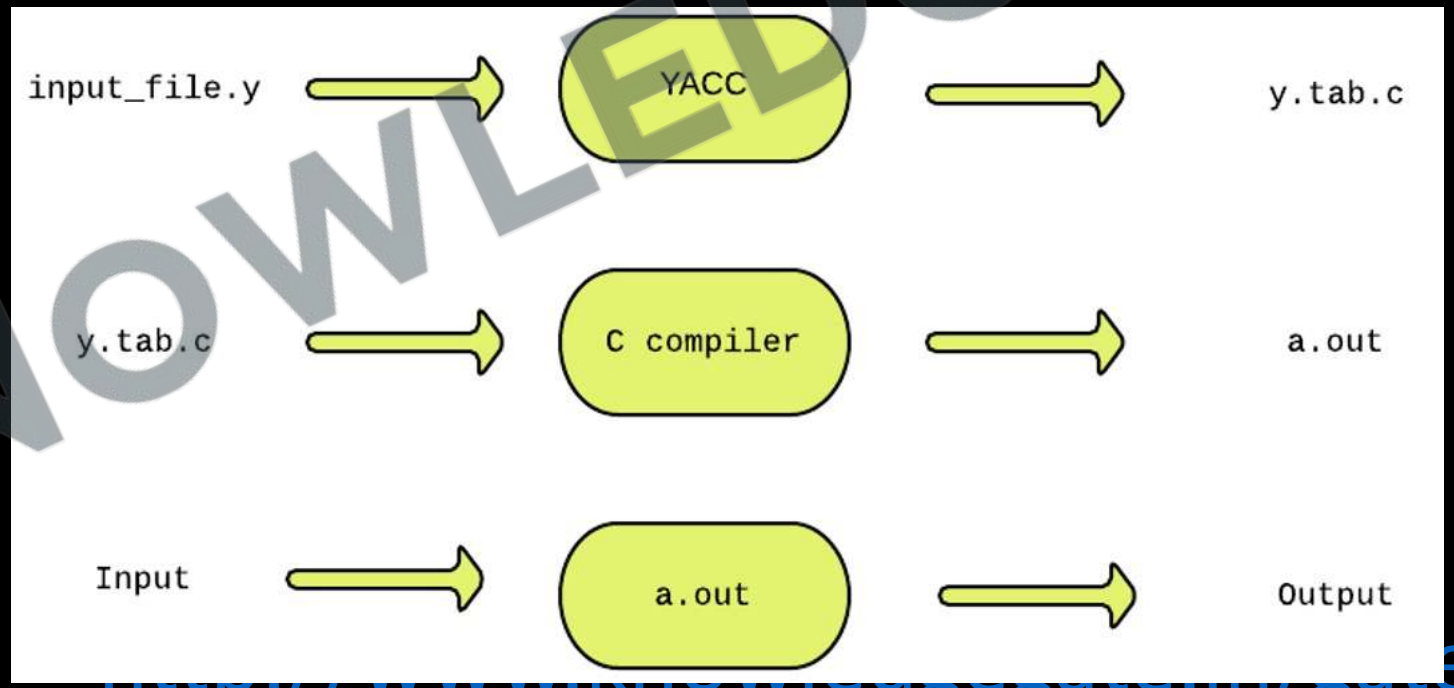
YACC (Yet Another Compiler Compiler)

- YACC (Yet Another Compiler Compiler) is a tool used in compiler construction to generate a parser from a specified grammar. Here are the key aspects:
- **Function**: YACC is utilized for creating compilers or interpreters. It takes a language's grammar (in Backus-Naur Form) and produces C code for a parser.



KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Structure of YACC File**:
  - **Declarations**: Defines tokens and includes necessary C code.
  - **Rules**: Contains grammar rules with associated C actions.
  - **User Subroutines**: Additional C functions for the parser.
- **Integration with Lex**: YACC is commonly used with Lex, a lexical analyzer, for tokenization.
- **Parser Type**: The parser generated is a LALR(1) parser, a type of efficient bottom-up parser.



KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Error Handling**: YACC includes syntax error detection and recovery mechanisms.
- **Advantages**:
  - Automates parser creation.
  - Simplifies grammar modification.
  - Separates syntax rules from actions.
- **Applications and Alternatives**: It's widely used in both academia and industry for compiler and interpreter development. Tools like Bison offer similar functionality with additional features.
- Understanding YACC is important for computer science students, particularly for insights into compiler design and parsing techniques.

KNOWLEDGEGATE

Defining a language by grammar

- The concept of defining a language using grammar is, starting from a start symbol using the production rules of the grammar any time, deriving the string. Here every time during derivation a production is used as its LHS is replaced by its RHS, all the intermediate stages(strings) are called sentential forms. The language formed by the grammar consists of all distinct strings that can be generated in this manner.

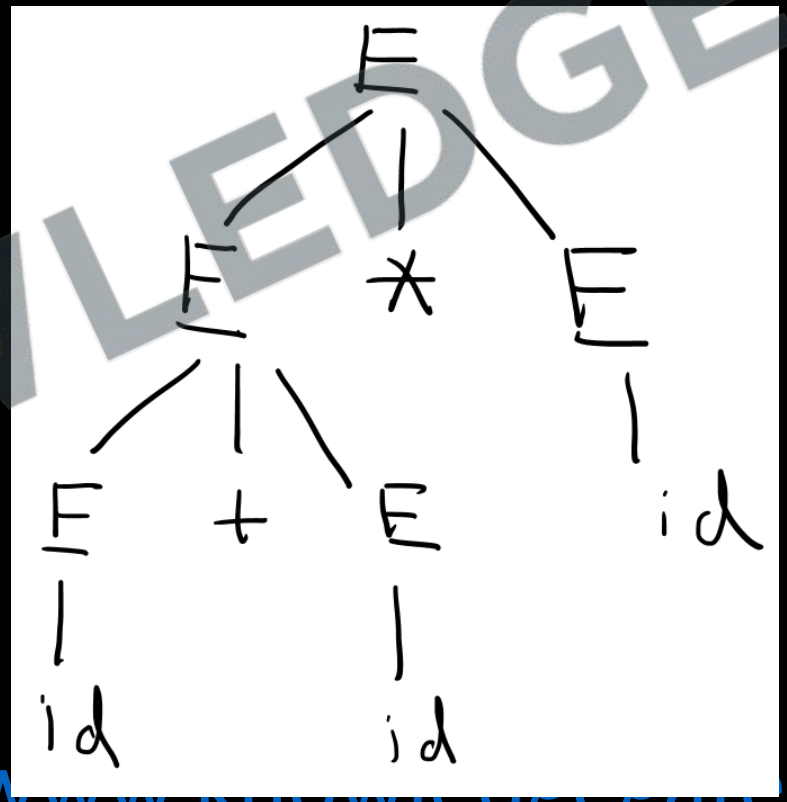
$$L(G) = \{w \mid w \in \sum^*, S \rightarrow^* W\}$$

- $\rightarrow^*$(reflexive, transitive closure) means from s we can derive w in zero or more steps

KNOWLEDGEGATE

- **Derivation:** - The process of deriving a string is known as derivation.
- **Derivation/ Syntax/ Parse Tree**: - The graphical representation of derivation is known as derivation tree.

E → E + E / E * E / E = E / id

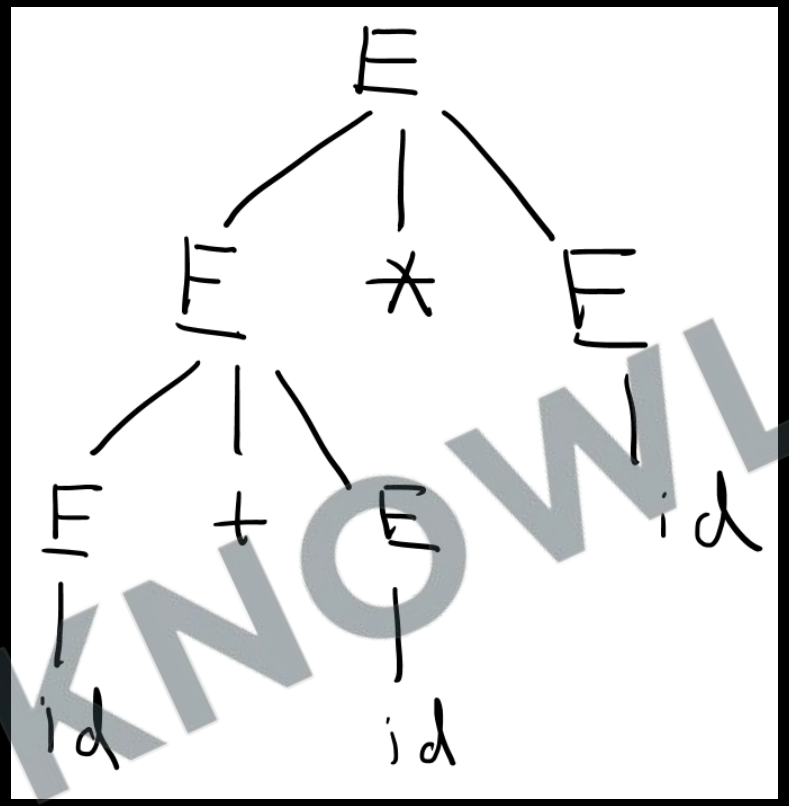


KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Sentential form:** - Intermediate step involve in the derivation is known as sentential form.

E → E + E / E * E / E = E / id

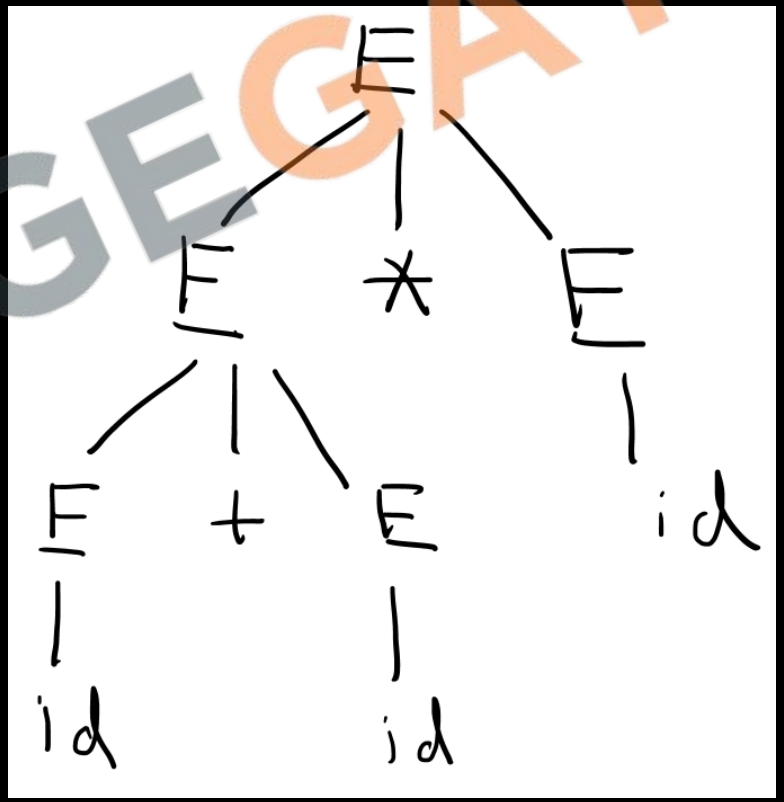


Sentential Form
E
E*E
E+E*E
ID+E*E
ID+ID*E
ID+ID*ID

KNOWLEDGEGATE

- **Left most derivation**: - the process of construction of parse tree by expanding the left most non terminal is known as LMD and the graphical representation of LMD is known as LMDT (left most derivation tree)

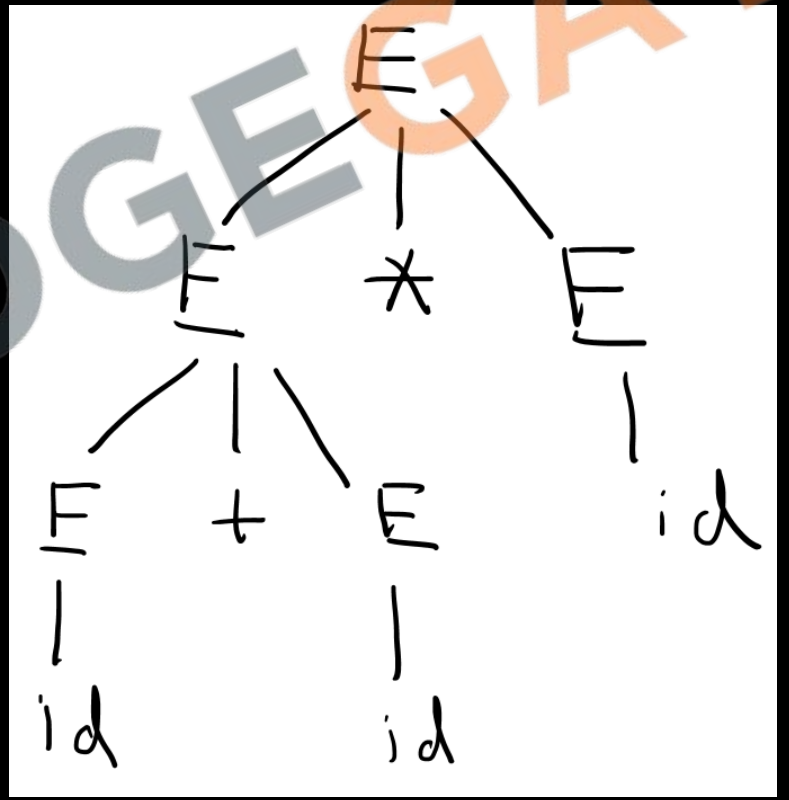
	LMD
E → E + E	E
E→ E * E	E*E
E→ E = E	E+E*E
E→ id	ID+E*E
E→ id	ID+ID*E
E→ id	ID+ID*ID



KNOWLEDGEGATE

- **Right most derivation**: - the process of construction of parse tree by expanding the right most non terminal is known as RMD and the graphical representation of RMD is known as RMDT (right most derivation tree)

	RMD
E → E + E	E
E→ E * E	E+E
E→ E = E	E+E*E
E→ id	E+E*ID
E→ id	E+ID*ID
E→ id	ID+ID*ID



KNOWLEDGEGATE

Capabilities of CFG

Context-Free Grammar (CFG) plays a vital role in computer science for describing programming language syntax and creating efficient parsers:

- **Programming Languages**: CFGs effectively describe the syntax of most programming languages, organizing elements like statements, expressions, and declarations.
- **Efficient Parser Construction**: A well-designed CFG enables the automatic generation of efficient parsers, essential for code analysis and interpretation.
- **Associativity and Precedence**: CFGs handle associativity and precedence in expressions, ensuring correct interpretation of operations and expression hierarchies.
- **Nested Structures**: CFGs excel in depicting nested structures common in programming languages, like balanced parentheses, matching begin-end blocks, and nested if-then-else statements.

KNOWLEDGE GATE

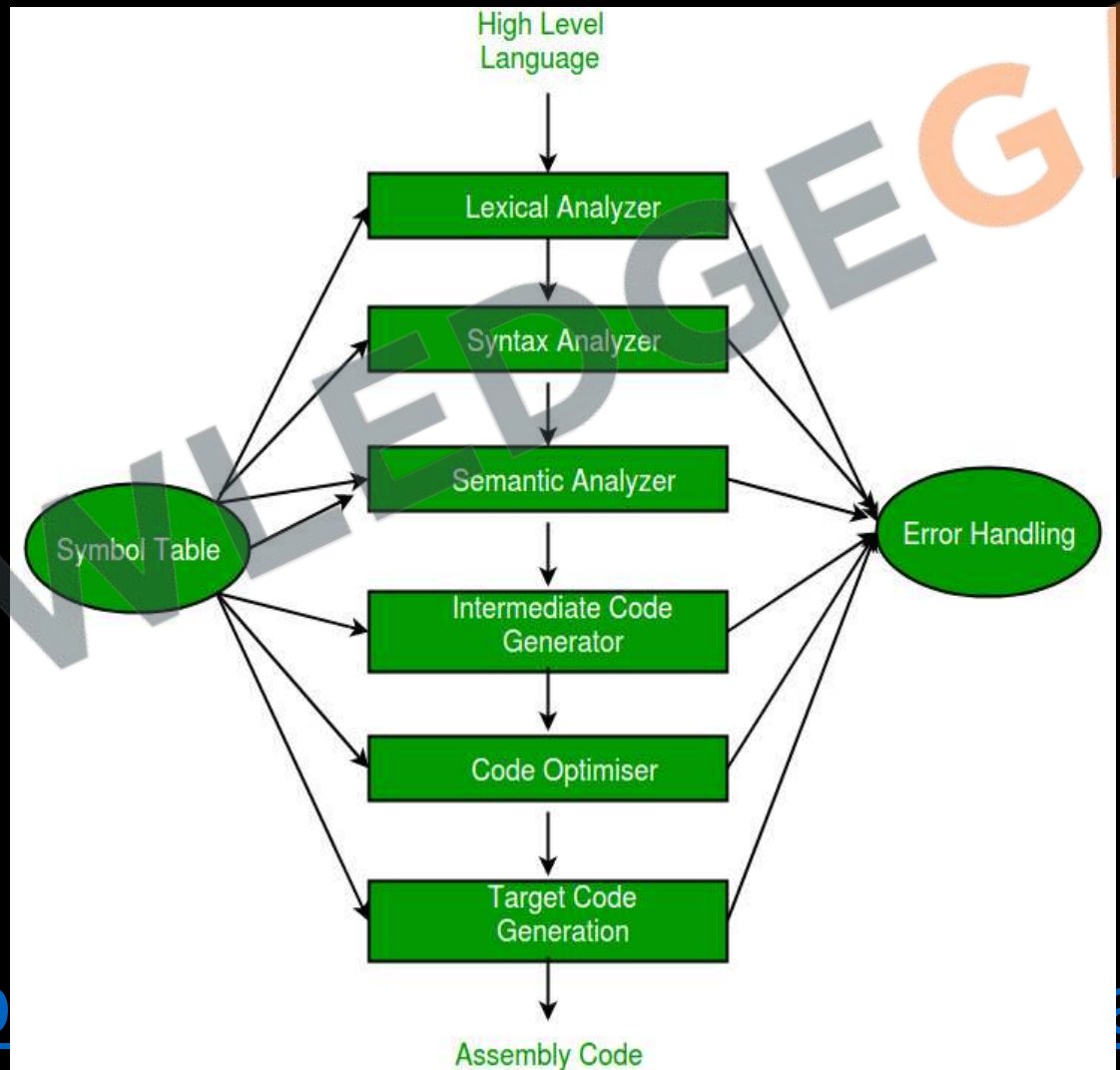
Chapter-2

(BASIC PARSING TECHNIQUES): Parsers, Shift reduce parsing, operator precedence parsing, top down parsing, predictive parsers Automatic Construction of efficient Parsers: LR parsers, the canonical Collection of LR(0) items, constructing SLR parsing tables, constructing Canonical LR parsing tables, Constructing LALR parsing tables, using ambiguous grammars, an automatic parser generator, implementation of LR parsing tables.

KNOWLEDGE GATE
http://www.knowledgegate.in/gate

Syntax analysis

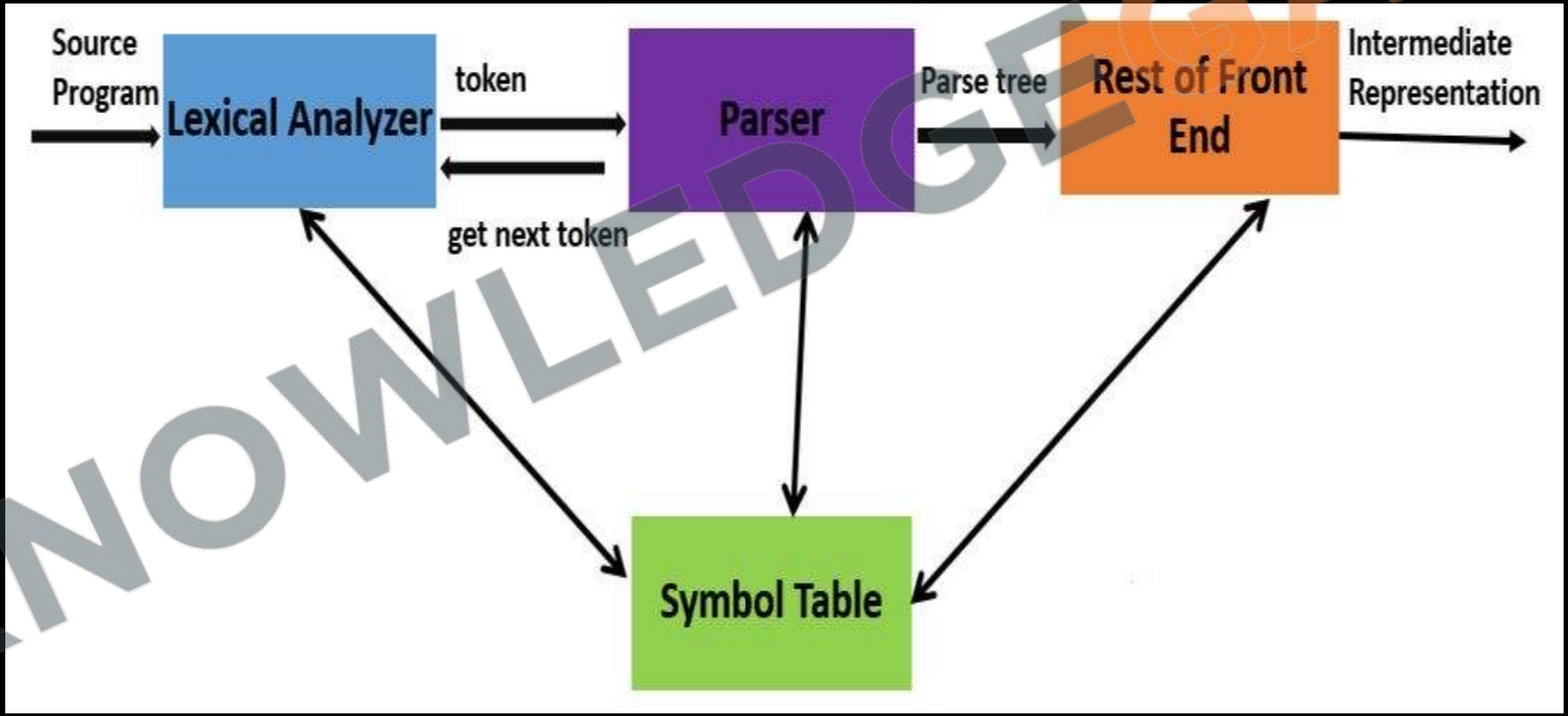
- In Syntax analysis input is stream of tokens and output is syntax tree. The process of construction of the parse tree/ syntax tree/ derivation tree is called as parsing.



KNOWLEDGEGATE

http ate

- For any input string which is given in the form of stream of tokens for the parser, if the derivation tree exists, then the input string is syntactically or grammatically correct.
- If the parser cannot generate the derivation tree from the i/p string, then there must be some grammatical mistakes in the string.



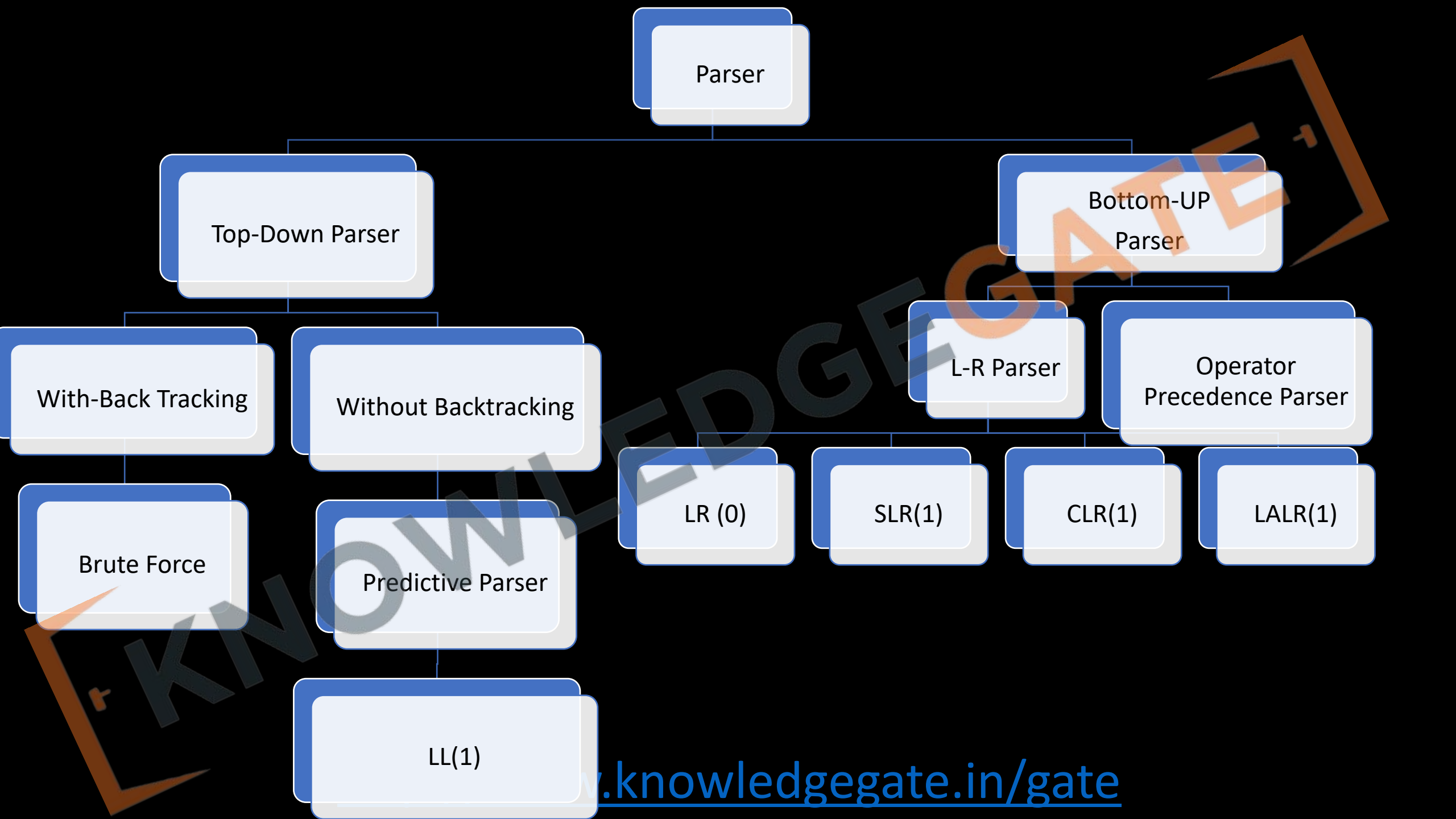
KNOWLEDGEGATE

Classification of parser

- The program which perform parsing is known as parser or syntax analyzer.

- There are two types of parser
  - Top-Down Parser
  - Bottom Up Parser

KNOWLEDGEGATE



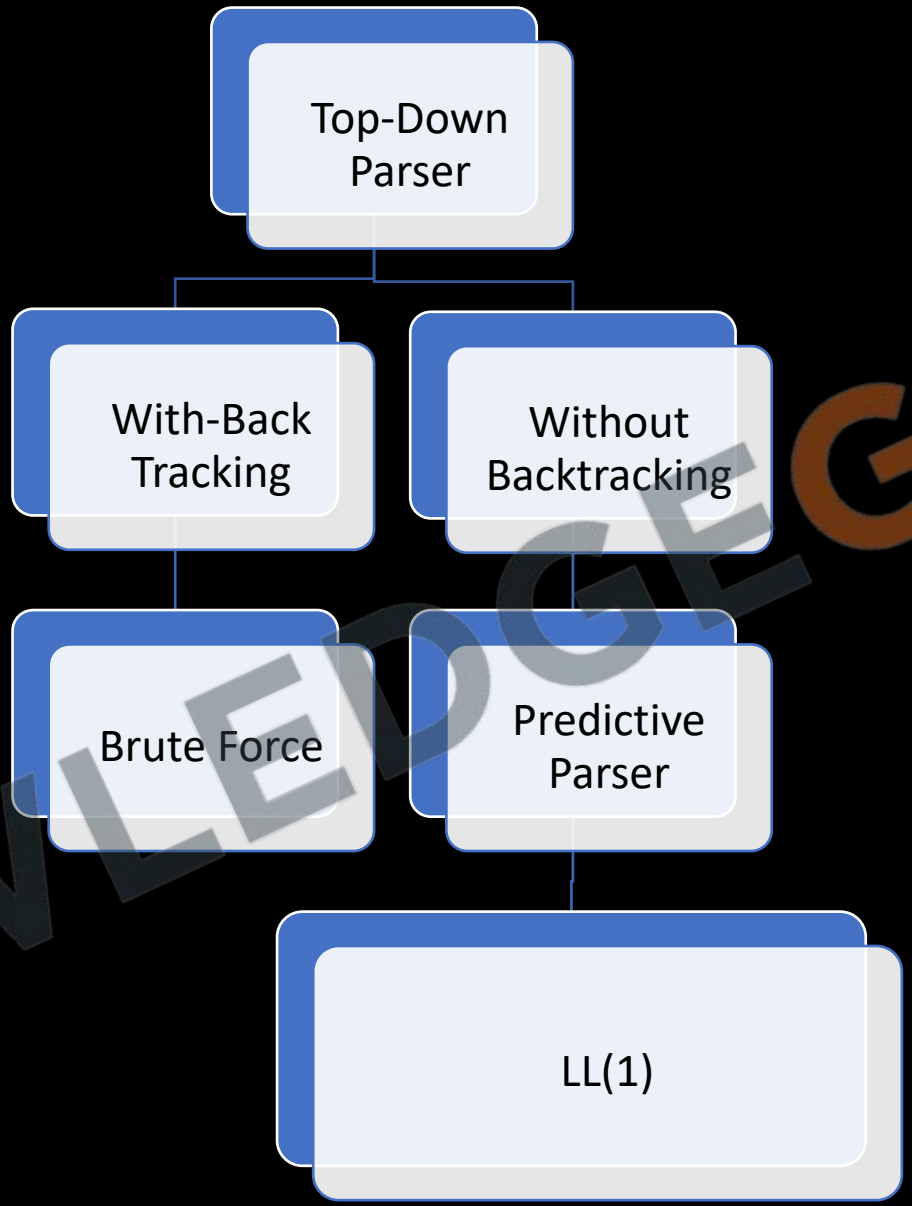
KNOWLEDGE GATE
www.knowledgegate.in/gate

Top down parsing

- The process of construction of parse tree, starting from root and process to children, is known as TOP down parsing, i.e. getting the i/p string by starting with a start symbol of the grammar is top down parsing.

$A \rightarrow aA / \in$

KNOWLEDGEGATE



KNOWLEDGEGATE

Non-Deterministic Grammar

- The grammar with common prefix is known as Non-Deterministic Grammar.
  - $A \rightarrow \alpha\beta_1 / \alpha\beta_2$

KNOWLEDGEGATE

- The grammar with common prefixes requires a lot of Back-tracking, back-tracking is very time consuming.

- To avoid the back-tracking, we need to remove the common prefixes, i.e. we need to convert the non-deterministic Grammar into Deterministic.

KNOWLEDGEGATE

- **<u>Left Factoring</u>**: - The process of conversion of Non-Deterministic grammar into deterministic grammar is known as Left-Factoring.

- $A \rightarrow \alpha\beta_1 / \alpha\beta_2$
  - $A \rightarrow \alpha\beta$
  - $A \rightarrow \beta_1 / \beta_2$

KNOWLEDGEGATE

Recursive production: - the production which has same variable both at left- and right-hand side of production is known as recursive production.

S→aSb

S→aS

S→Sa

Recursive grammar: - the grammar which contains at least one recursive production is known as recursive grammar.

S→aS / a

S→Sa / a

S→aSb / ab

KNOWLEDGEGATE

Left Recursive Grammar: - The grammar G is said to be left recursive, if the Left most variable of RHS is same as the variable at LHS.

Right Recursive Grammar: - The grammar G is said to be right recursive, if the right most variable of RHS is same as the variable at LHS.

KNOWLEDGEGATE

General recursion: - the recursion which is neither left nor right is called as general recursion. If a CFG generates infinite number of string then it must be a recursive grammar.

Non recursive grammar: - the grammar which is free from recursive production is called as non-recursive grammar.

S→AaB

A→a

B→b

KNOWLEDGEGATE

- If a CFG contains left recursion then the compiler may go to infinite loop, hence to avoid the looping of the compiler, we need to convert the left recursive grammar into its equivalent right recursive production.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

$A \rightarrow A\alpha / \beta_1 / \beta_2 \text{-------} / \beta_n$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

$A \rightarrow A\alpha_1 / A\alpha_2 /----------A\alpha_n / \beta$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

$A \rightarrow A\alpha_1 / A\alpha_2 /----------A\alpha_n / \beta_1 / \beta_2 -------/ \beta_m$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Ambiguous grammar: - The grammar CFG is said to be ambiguous if there are more than one derivation tree for any string i.e. if there exist more than one derivation tree (LMDT or RMDT), the grammar is said to be ambiguous.

S → aS/Sa/a

KNOWLEDGEGATE

- Grammar which is both left and right recursive is always ambiguous, but the ambiguous grammar need not be both left and right recursive.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- Top down parser uses, left most derivation.
- **Left most derivation**: - The process of construction of parse tree by expanding the left most non terminal is known as LMD and the graphical representation of LMD is known as LMDT (left most derivation tree).

$A \rightarrow AaA / \in$

KNOWLEDGEGATE

- The TDP is constructed for the grammar, if it is free from left recursion.

- **Left Recursive Grammar**: - The grammar G is said to be left recursive, if the Left most variable of RHS is same as the variable at LHS.

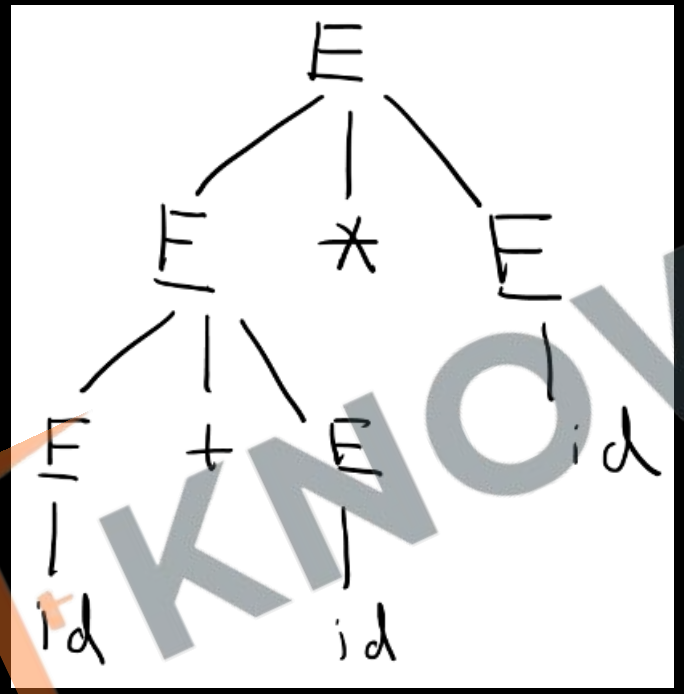
$$A \rightarrow Aa \,/\, \in$$

KNOWLEDGEGATE

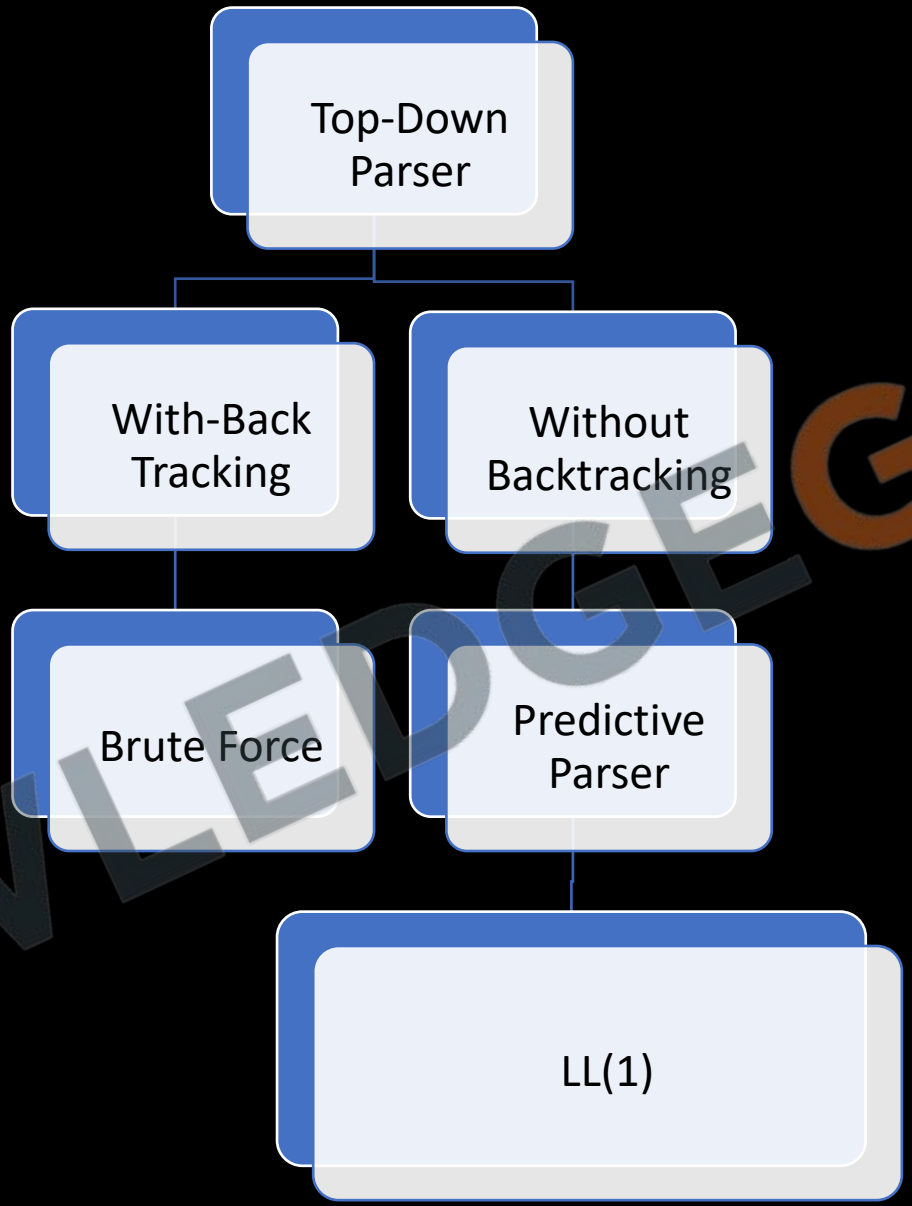
- The TDP is constructed for the grammar, if it is free from ambiguity.

- **Ambiguous grammar**: - The grammar CFG is said to be ambiguous if there are more than one derivation tree for any string i.e. if there exist more than one derivation tree (LMDT or RMDT), the grammar is said to be ambiguous.

E → E + E / E * E / E = E / id



KNOWLEDGEGATE



KNOWLEDGEGATE

Brute force technique

- Whenever a non- terminal is expanding first time, then go with the first alternative and compare with the i/p string. if does not matches, go for the second alternative and compare with i/p string, if does not matches go with the 3rd alternative and continue with each and every alternative.

- if the matching occurs for at least one alternative, then the parsing is successful, otherwise parsing fail.

KNOWLEDGEGATE

S → cAd

A → ab / a

w_1 = cad

w_2 = cada

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → aAc / aB

A → b / c

B → ccd / ddc

w = addc

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- TDP may be constructed for both left factor and non-left factor grammar.
- If the grammar is non- deterministic, then we use brute technique and if the grammar is deterministic, then we go with the predictive parser.
- Brute force requires lot of back-tracking, takes $O(2^n)$
- Back Tracking is very costly & reduces the performance of parser
- Debugging is very difficult.

KNOWLEDGEGATE

- The TDP is constructed for a grammar, if there is less complexity, i.e. if the complexity of grammar is more than the parsing mechanism is very slow and performance is low.

- Worst case time complexity may go up to $O(n^4)$ in TDP except (Brute force).

KNOWLEDGEGATE

- First(α) is a set of all terminals that may be in beginning in any sentential form, derived from α

- FIRST(α) is calculated for both Terminals and Non-Terminals

- **if** α is a terminal, then
  - **First(α) = {α}**

- **if** α is a string of terminal e.g. abc
  - **First(abc) = {a}**

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **if** α is a non - terminal, defined by α → ε, then
  - **First(α) = {ε}**

- **if** α is a non - terminal, defined by α → β, β ∈ T
  - **First(α) = {β}**

KNOWLEDGEGATE

- **if** α is a non - terminal, defined by α → X_1 X_2 X_3, then
  - **First(α) = First(X_1)** **iff** X_1 →! ∈
  - **First(α) = [First(X_1) U First(X_2)] -** ∈ **iff** X_1 → ∈ && **iff** X_2 →! ∈
  - **First(α) = [First(X_1) U First(X_2) U First(X_3)] -** ∈ **iff** X_1 → ∈ && X_2 → ∈ && **iff** X_3 →! ∈
  - **First(α) = First(X_1) U First(X_2) U First(X_2)** **iff** X_1 → ∈ && X_2 → ∈ && **iff** X_3 → ∈

KNOWLEDGEGATE

Q Consider the following Grammar find the First for each of them?

S → a / b / ε

First(S) =

Follow(S) =

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → aA / bB

A → ε

B → ε

First(S) =

First(A) =

First(B) =

Follow(S) =

Follow(A) =

Follow(B) =

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → AaB / BA

A → a / b

B → d / e

First(S) =

First(A) =

First(B) =

Follow(S) =

Follow(A) =

Follow(B) =

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → AaB

A → b / ε

B → c

First(S) =

First(A) =

First(B) =

Follow(S) =

Follow(A) =

Follow(B) =

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → AB

A → a / ε

B → b / ε

First(S) =

First(A) =

First(B) =

Follow(S) =

Follow(A) =

Follow(B) =

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

S → ABCDE

A → a / ε

B → b / ε

C → c / ε

D → d

E → e / ε

First(S) =

First(A) =

First(B) =

First(C) =

First(D) =

First(E) =

Follow(S) =

Follow(A) =

Follow(B) =

Follow(C) =

Follow(D) =

Follow(E) =

KNOWLEDGEGATE

E → TE’

E’ → +TE’ / ε

T → FT’

T’ → *FT’ / ε

F → (E) / id

First(E) =

First(E’) =

First(T) =

First(T’) =

First(F) =

Follow(E) =

Follow(E’) =

Follow(T) =

Follow(T’) =

Follow(F) =

KNOWLEDGEGATE

S → aBDh

B → cC

C → bC / ε

D → EF

E → g / ε

F → f / ε

First(S) =

First(B) =

First(C) =

First(D) =

First(E) =

First(F) =

Follow(S) =

Follow(B) =

Follow(C) =

Follow(D) =

Follow(E) =

Follow(F) =

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Follow

Follow(A) is the set of all terminals that may follow to the right of (A) in any form of sentential Grammar.

Rules:

1) if A is the start symbol then Follow(A) = {$}

2) if A → αAβ, β →! ∈
   Follow(A) = First(β)

3) if S → αA
   Follow(A) = Follow(S)

4) S → αAβ, where β → ∈
   Follow(A) = First(β) U Follow(S) - ∈

KNOWLEDGEGATE

KNOWLEDGEGATE

$S \rightarrow aA$

$A \rightarrow bA / \varepsilon$

KNOWLEDGEGATE

S → AaBb / BbAa

A → ε

B → ε

KNOWLEDGEGATE

E → E+T / T

T → T*F / F

F → (E) / id

KNOWLEDGEGATE

S → aAb

A → Ba / b

B → d

KNOWLEDGEGATE

$S \rightarrow S0S1 / \varepsilon$

KNOWLEDGEGATE

$S \rightarrow AaAb / BaBb$

$A \rightarrow \varepsilon$

$B \rightarrow \varepsilon$

KNOWLEDGEGATE

Follow

Follow(A) is the set of all terminals that may follow to the right of (A) in any form of sentential Grammar.

Rules:

1) if A is the start symbol then Follow(A) = {$}

2) if A → αAβ, β →! ∈
   Follow(A) = First(β)

3) if S → αA
   Follow(A) = Follow(S)

4) S → αAβ, where β → ∈
   Follow(A) = First(β) U Follow(S) - ∈

KNOWLEDGEGATE

E → TE’

E’ → +TE’ / ∈

T → FT’

T’ → *FT’ / ∈

F → (E) / id

id	+	id	*	id	$

	+	*	(	)	id	$
E						
E’						
T						
T’						
F						

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider a given Grammar LL(1) grammar design Parsing table and perform complete parsing table?

$E \rightarrow TE'$

$E' \rightarrow +TE' / \in$

$T \rightarrow FT'$

$T' \rightarrow *FT' / \in$

$F \rightarrow (E) / id$

$w = id+id*id$

Stack	i/p	Action
$	id+id*id$	Push E
$ E	id+id*id$	Use Production E→TE' POP E and Push E'T
$ E'T	id+id*id$	Use Production T→FT' POP T and Push T'F
$ E'T'F	id+id*id$	Use Production F→id POP F and Push id
$ E'T'id	id+id*id$	Match Pop id and Increment Look Ahead Pointer
-	-	-
-	-	-
-	-	-
-	-	-
$	$	Accepted

KNOWLEDGEGATE

- LL(1)
  - First L means Left to right scanning
  - Second L means Left most derivation
  - 1 means no of look ahead symbol

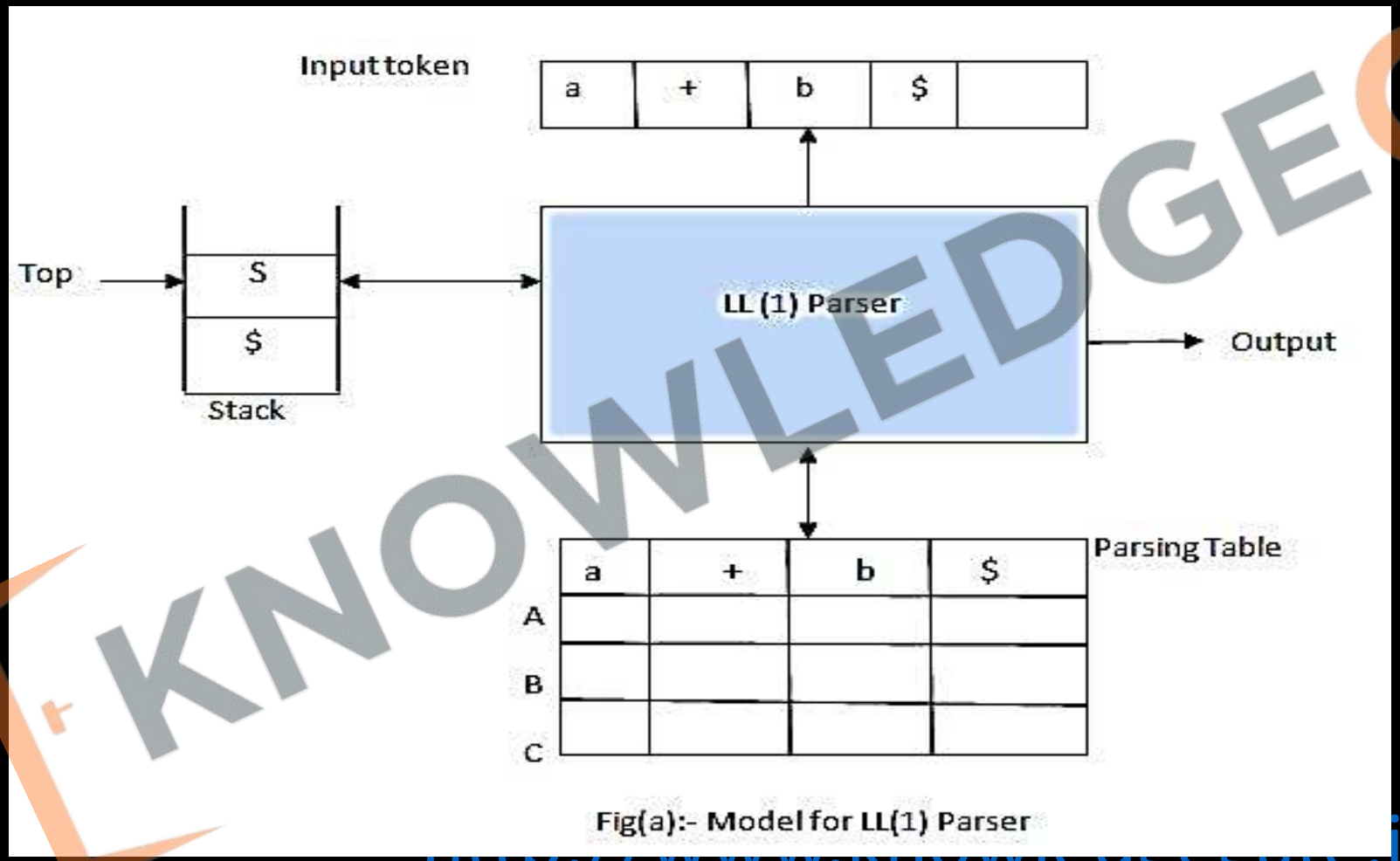
- To predict the required production, to extend the parse tree, LL(1) parser depends on current processing symbol.

- The current processing symbol is called as Look-ahead-symbol.

KNOWLEDGEGATE

Block-Diagram of LL(1) Parser

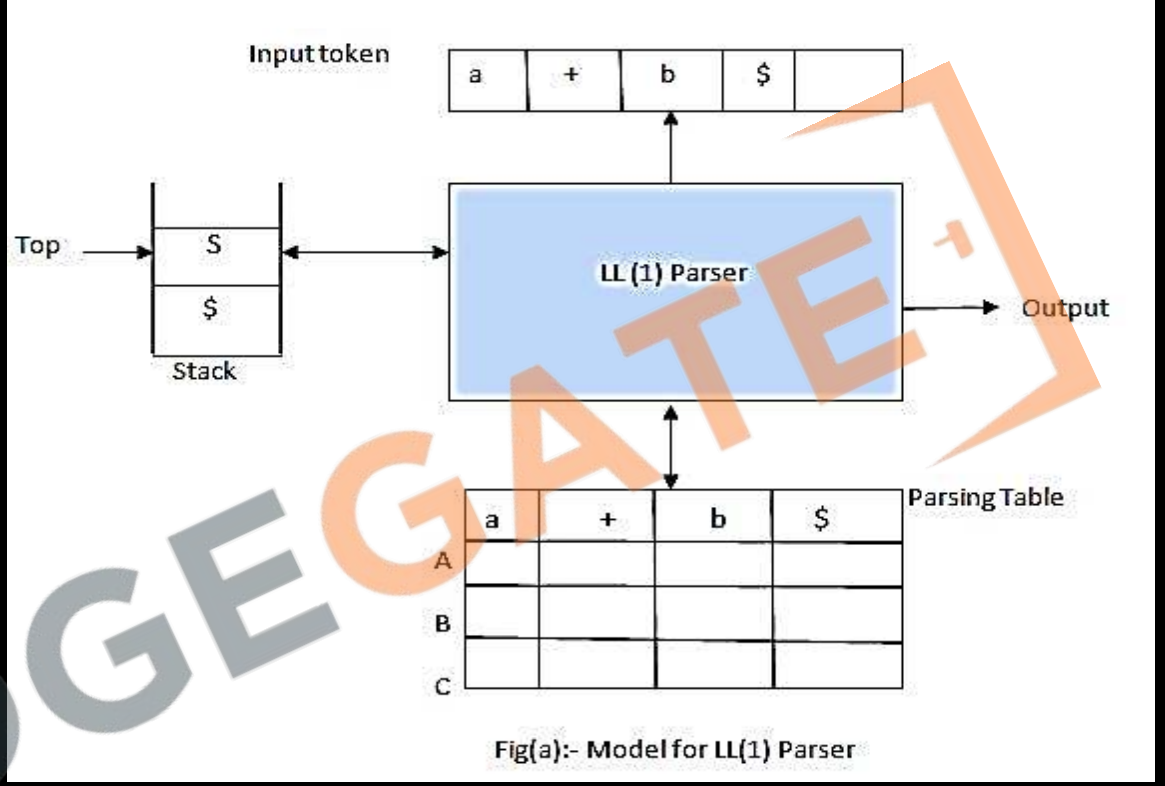
- LL(1) parser consist of 3 components
  - Input Buffer
  - Parse Stack
  - Parse Table



Fig(a):- Model for LL(1) Parser

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

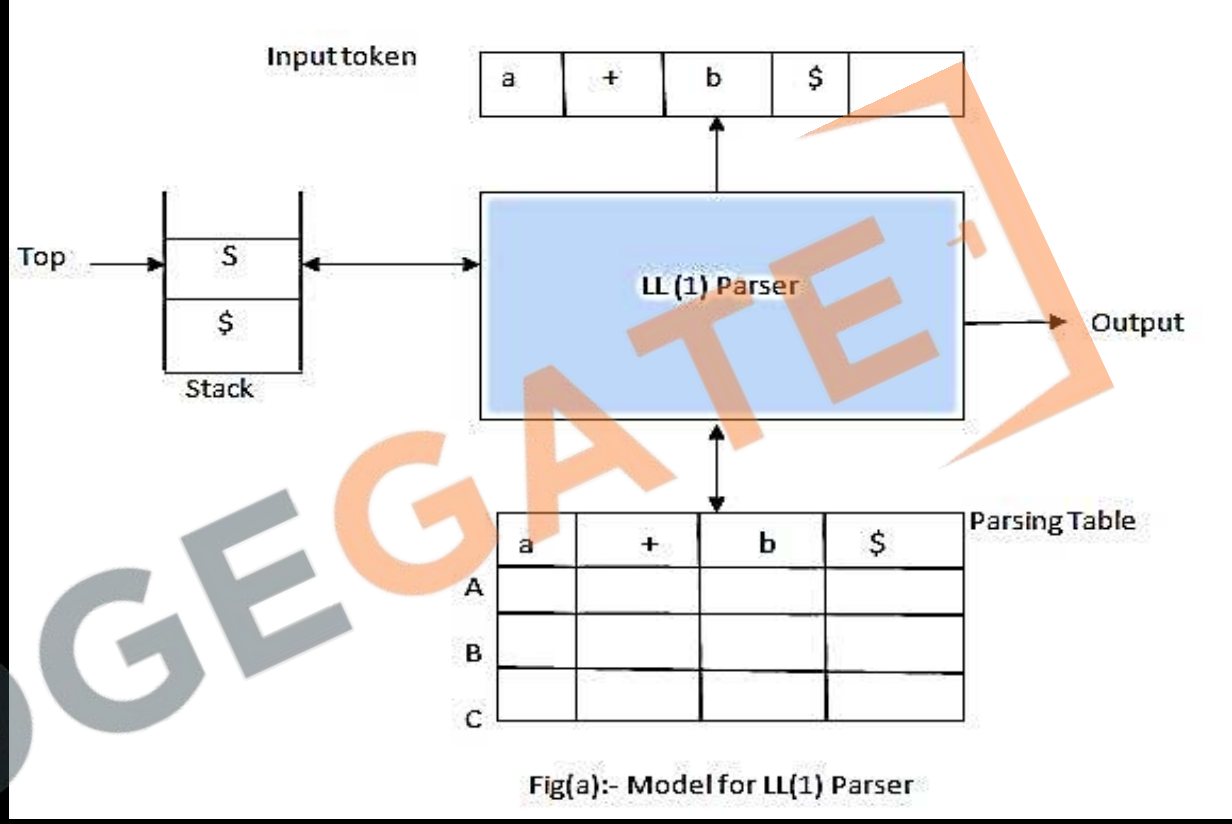
- **Input Buffer**
  - it is divided into finite no of cells and each cell is capable of holding only one i/p symbol.
  - input buffer contains only i/p string at any point of time.
  - the tape header is always pointing only one look ahead symbol and after parsing the current look ahead symbol, the header moves to next cell towards right side.
  - End of the string is recognized by $



Fig(a):- Model for LL(1) Parser

KNOWLEDGEGATE

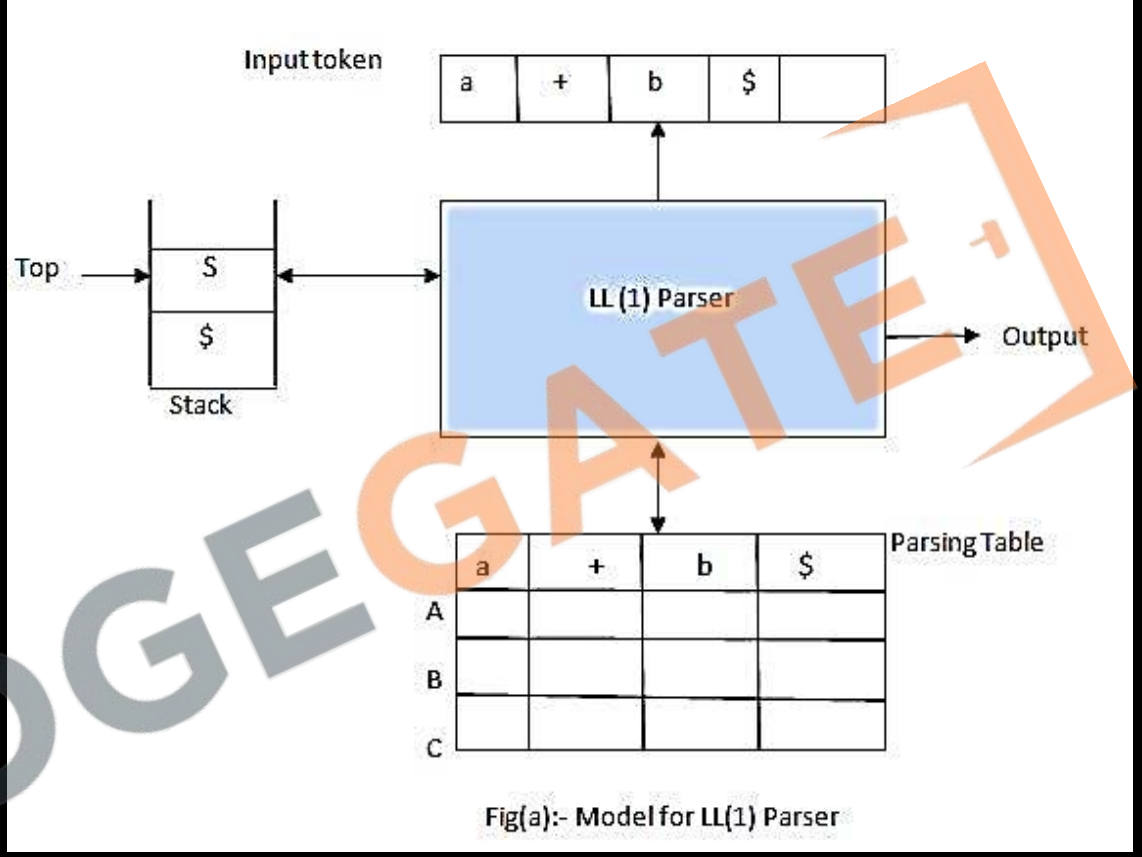
- **Parse Stack**
  - it contains the grammar symbol, the grammar symbol is pushed into stack or POP from the stack based on the occurrence of matching.
  - if the topmost symbol of the stack is matching with look ahead symbol, then the grammar symbol is POP out from the stack.
  - if the TOP most symbol of the stack is not matching with the look ahead symbol, then the grammar symbol is Pushed into stack.



Fig(a):- Model for LL(1) Parser

KNOWLEDGEGATE

- **Parse table**
  - it is a two-dimensional array of order m*n where m = no of non-terminal and n = no of terminals + 1
  - parse table contains all the production which are used to contain the parse tree for that i/p string.



Fig(a):- Model for LL(1) Parser

KNOWLEDGEGATE

- Procedure to constructed LL(1) parser Table:
  - for every production A → α, repeat the following steps
  - add A → α in M[A, α] for every terminal ‘α’ in first(A).
  - if First(α) contains ε, then add A → ε in M[A, b] for every symbol, b in Follow[A]

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Parsing Process**
  - Push the start symbol into stack
  - Compare the top most symbol of the stack with the look ahead symbol
  - If matching occurs, then pop of the grammar symbol from the stack and increment the i/p pointer
  - O/p the production which is used for expanding a non- terminal, i.e. the result is a production which is used for push operation.

KNOWLEDGEGATE

- **LL(1) parsing algorithm**
  - let x is a grammar symbol (stack symbol) and a is the look ahead symbol
  - if x == a == $, then the parsing is successful
  - if x==a! = $, then pop of and increment the i/p pointer
  - if x!= a!=$ and m[x, a] contain the production, x → abc, then replace x by abc in the reverse order and continue the process.
  - out of the production which is used for expanding the non-terminal, i.e. the production which is used for PUSH operation

KNOWLEDGEGATE

- LL(1) Grammar: Grammar for which LL(1) parser can be constructed is known as LL(1) Grammar

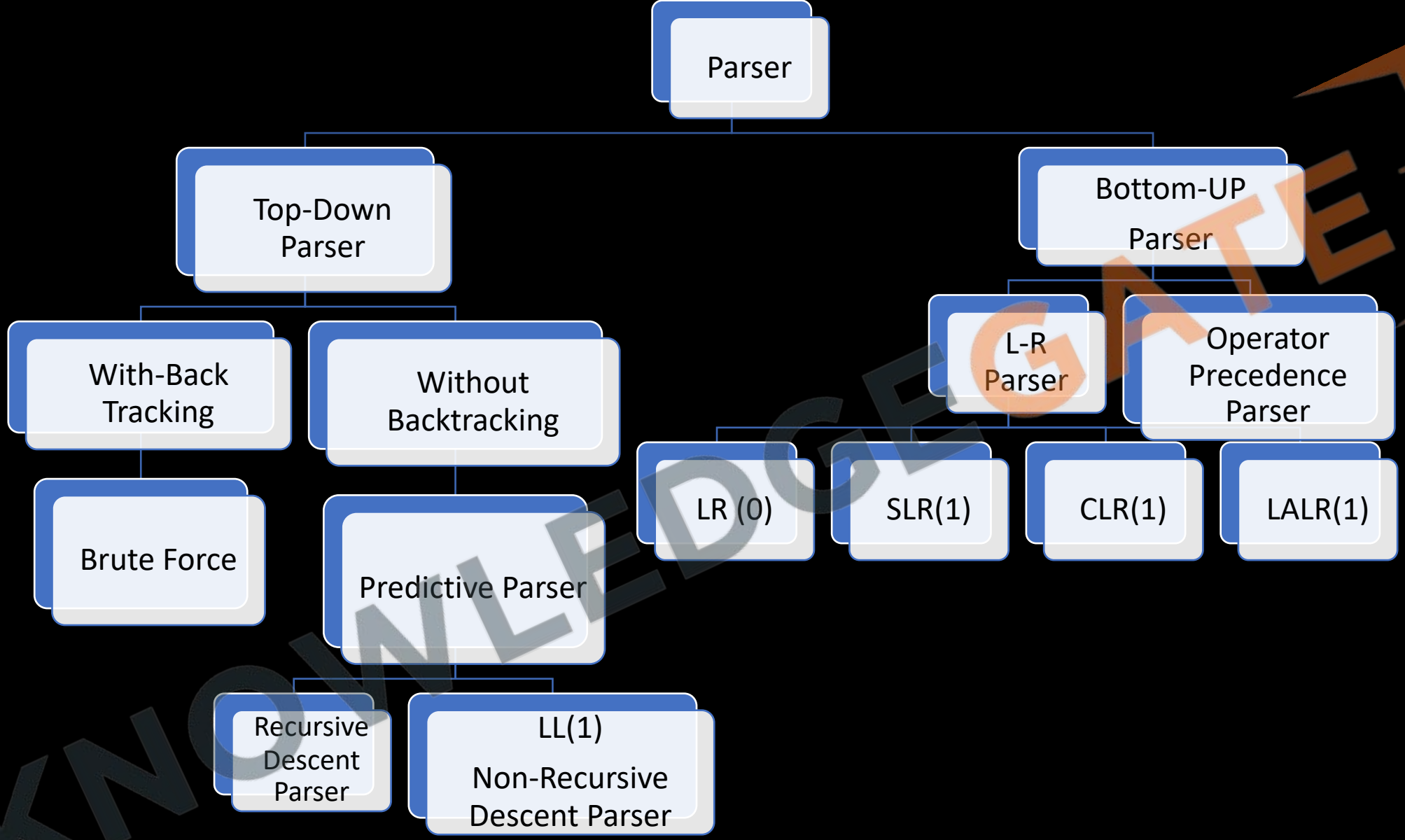
- Or the Grammar whose LL(1) parse table does not contains multiple entries in the same cell, then the grammar is LL(1)

KNOWLEDGEGATE

LL(1) or Table Driven Parser

- LL(1) is constructed for the Grammar, if
  - free from left recursion
  - free from ambiguity
  - left factored

KNOWLEDGEGATE



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Bottom Up parser

- The process of constructing the parse tree in the Bottom-Up manner, i.e. starting from the children & proceeding towards root.

- S → aABc
  A → b / bc
  B → d

- w = abcdc

KNOWLEDGEGATE

- LR parsers were invented by Donald Knuth in 1965 as an efficient generalization of precedence parsers. Knuth proved that LR parsers were the most general-purpose parsers possible.

- **Donald Ervin Knuth** (born January 10, 1938) is an American computer scientist, mathematician, and professor emeritus at Stanford University. He is the 1974 recipient of the ACM Turing Award, informally considered the Nobel Prize of computer science. Knuth has been called the "father of the analysis of algorithms".

KNOWLEDGEGATE

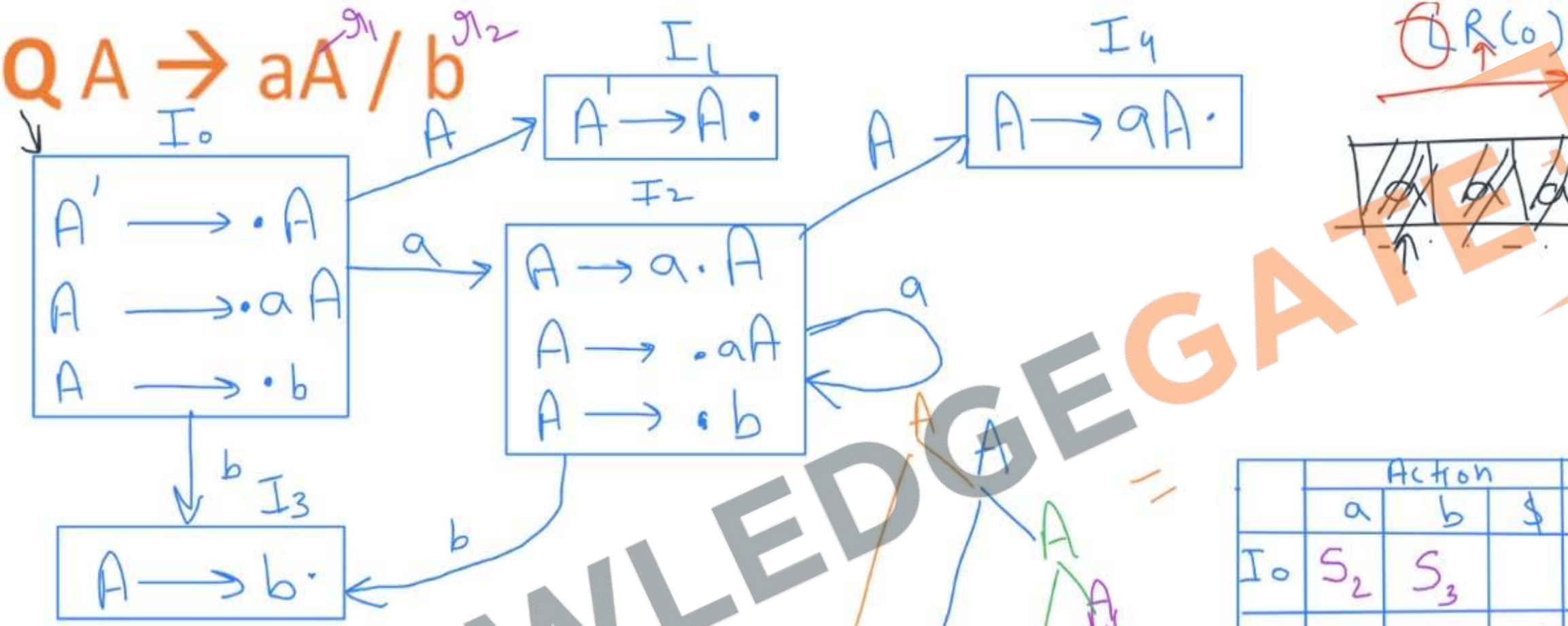
Q A → aA / b

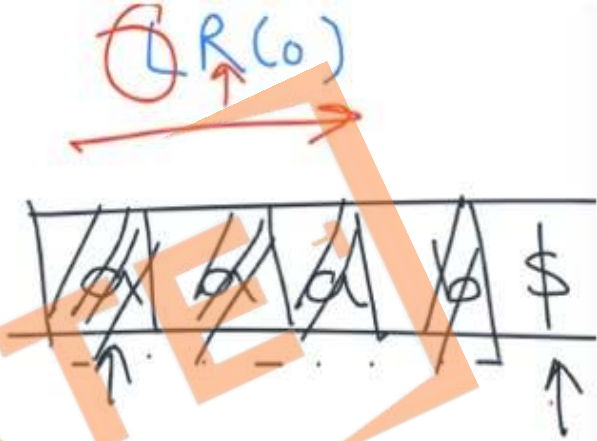
a	a	a	b	$

	Action			Goto

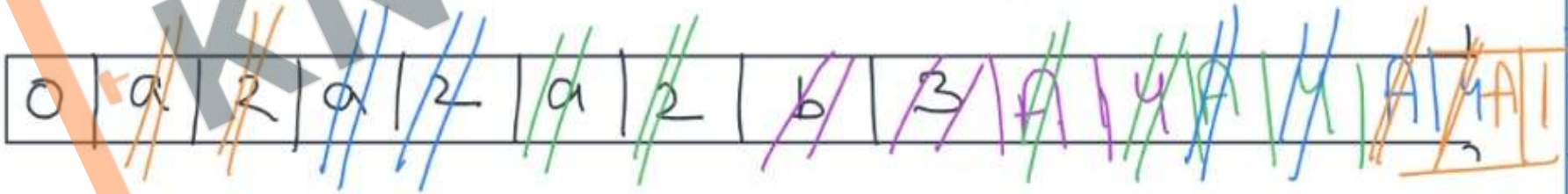
KNOWLEDGEGATE

Q $A \rightarrow aA \;(r_1) \;/\; b \;(r_2)$





	Action			Goto
	a	b	$	A
I_0	S_2	S_3		S_1
I_1			accept	
I_2	S_2	S_3		S_4
I_3	r_2	r_2	r_2	
I_4	r_1	r_1	r_1	



KNOWLEDGEGATE

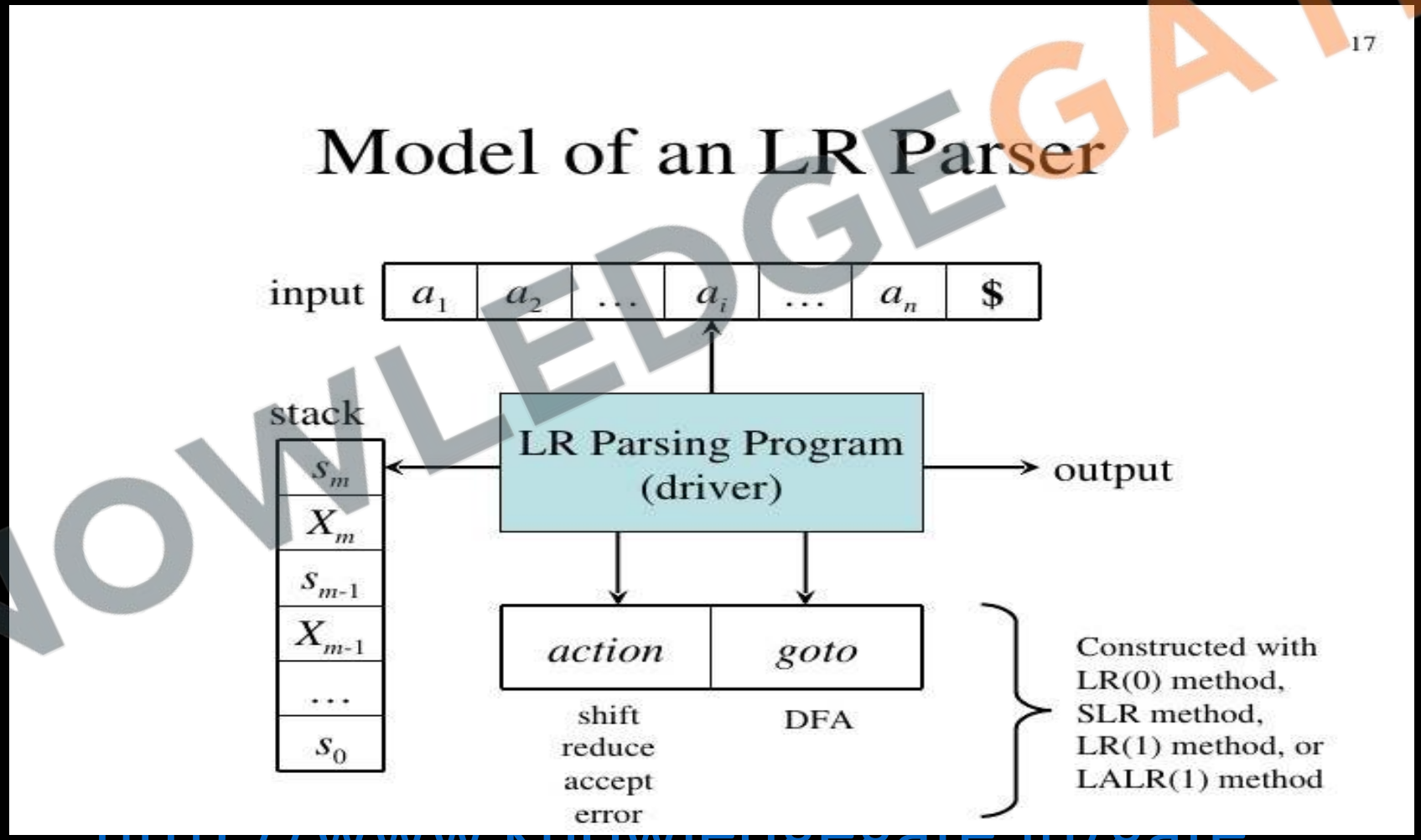
- **Handle**: - Substring of the i/p string that matches with RHS of any production, is called as Handle.
- The process of finding the handle & replacing that handle by it's LHS variable is called Handle Pruning.
- Bottom-Up-Parser is also known as Shit-Reduced Parser.
- BUP can be constructed for both Ambiguous & Unambiguous grammar
  - Ambiguous →OPP
  - Unambiguous → LR(k)

KNOWLEDGEGATE

- LR(K) parser can be constructed for Unambiguous grammar.
- BUP Simulates the Reverse of Right Most Derivation.
- BUP can be constructed for the grammar which has more complexity.
- Bottom-up parsing is faster than Top-Up-Parsing, such that BUP is more efficient than TDP.

KNOWLEDGEGATE

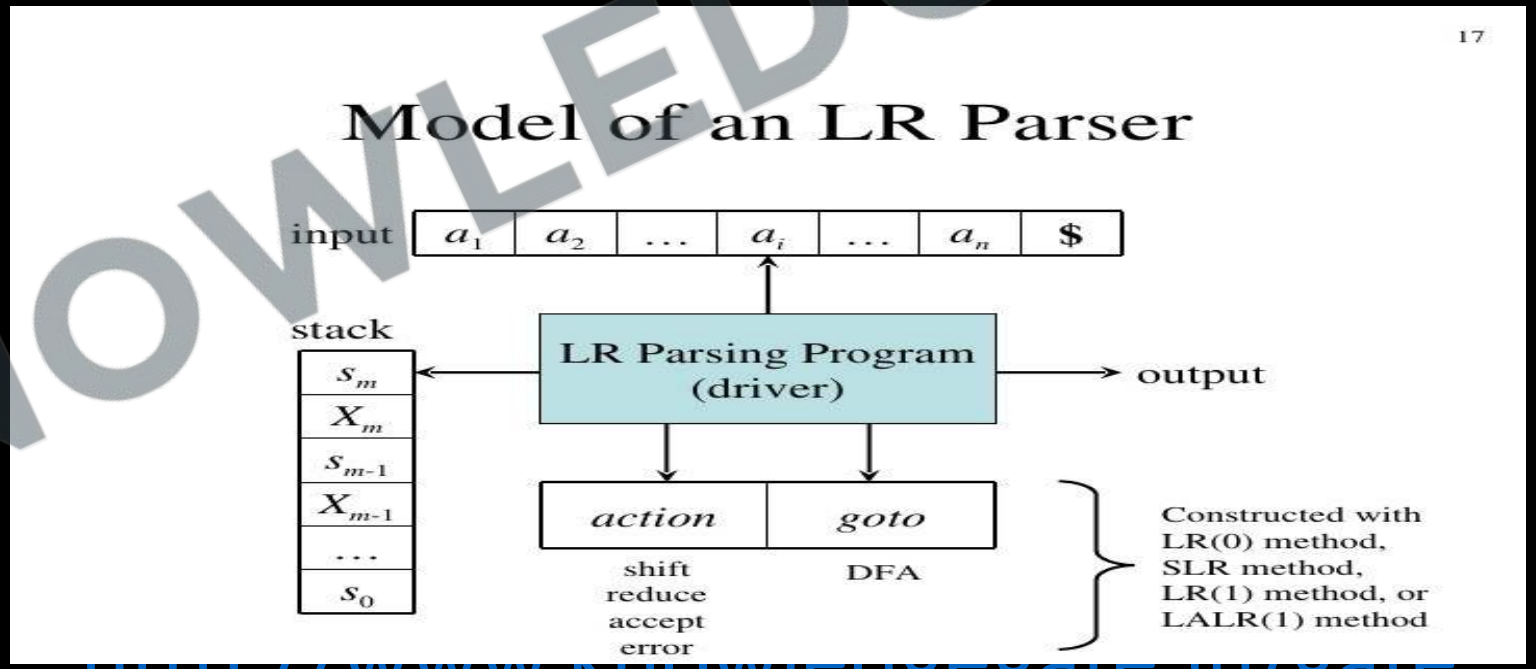
- Bottom Up Parser consist of three components
  - i/p buffer
  - Parse stack
  - Parse table



KNOWLEDGE GATE
http://www.knowledgegate.in/gate

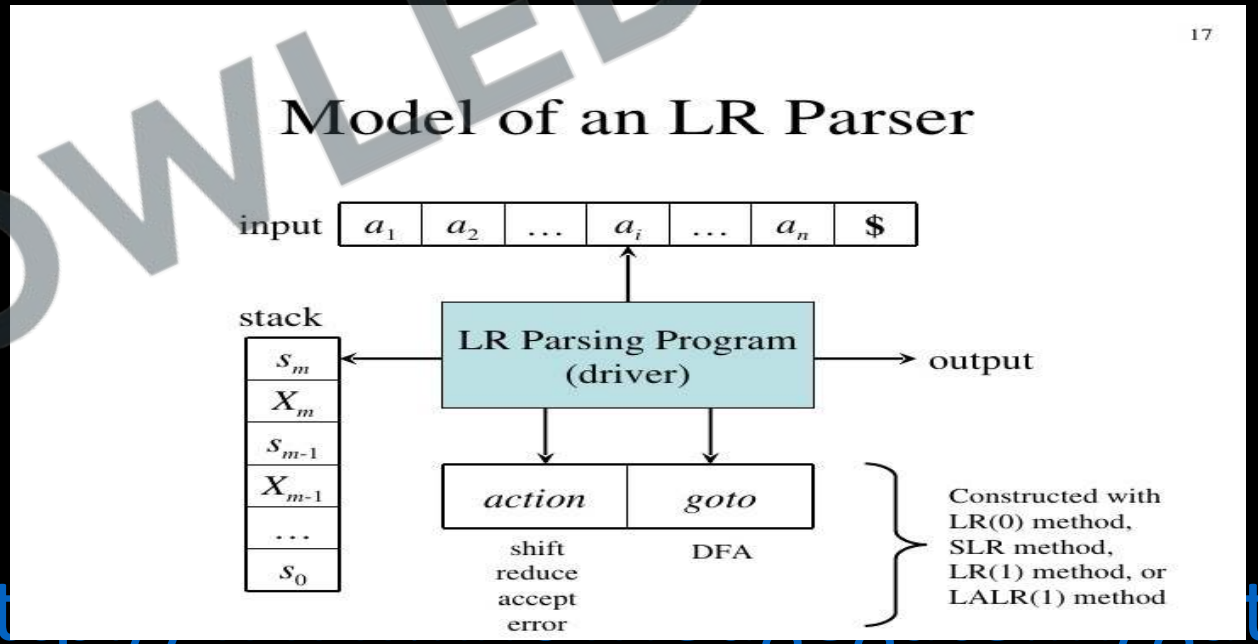
- **Input buffer**: divide into cells & each cell contains only one i/p symbol.

- **Stack**: stack contains the grammar symbol, the grammar symbol are push into stack or pop from the stack, using shift & reduced operation. if habdle occurs from the topmost symbol of the stack, then apply the reduced operation & if handle does not occurs in the topmost symbol of the stack, then apply the shift operation.



KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Parse table**: parse table is constructed using terminals, non-terminals & LR(0) items. this parse table consist of two parts:
  - Action
  - Goto
- Action part contains shift & reduced operation over the terminals
- Goto part consists of only Shift operation over the Non-terminals.



KNOWLEDGEGATE

- Operation in shift/reduced passer
  - shift
  - reduced
  - accept
  - error

	Action	Goto
I_0	Terminals	Non-Terminals
I_{n-1}	Shift / Reduce	Shift

KNOWLEDGEGATE

- **Shift**: shift operation can be used when handle does not occurs from the topmost symbol of the stack. using shift operation, will moving a look ahead symbol in stack.

- **Reduce**: reduce operation can be whenever handle occurs from the Topmost symbol from the stack. using reduced operation, we rename the topmost symbol of the stack that matches with look-ahead symbol.

- **Accept**: after scanning the complete i/p string from the i/p buffer, if the stack contains only the start symbol of the grammar as topmost symbol, then the i/p string is accepted and the parsing is successful.

- **Error**: after the complete i/p string, if the attack contains any symbol which is different from start symbol as a topmost symbol, then the parsing is unsuccessful and hence error.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- LR(K)
  - First L stand for left to right scanning
  - Second R stand for reverse of right most derivation
  - K is Look-Ahead symbol

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- Procedure for the construction of LR Parser table:
  1. Obtain the augmented grammar for the given grammar
  2. Create the canonical collection of LR items or compiler items.
  3. Draw the DFA & prepare the table based on LR items.

KNOWLEDGEGATE

- **Augmented grammar**
  - The grammar which is obtained by addition one more production that generate the start symbol of the grammar, is known as Augmented grammar.
  - S → AB A → a B→ b
  - S' → S S → AB A → a B→ b

KNOWLEDGEGATE

- **LR(0) or Compiler item**
  - The production, which has dot(.) anywhere on RHS is known as LR(0) items.

  - A → abc

  - LR(0) items:
    - A → .abc
    - A → a.bc
    - A → ab.c
    - A → abc. Final / Completed items

KNOWLEDGEGATE

- **Canonical Collection:**
  - The set $C = \{I_0, I_1, I_2, I_3, \ldots\ldots\ldots I_N\}$ is known as canonical collection of LR(0) items.

KNOWLEDGEGATE

- Function used to generate LR(0) item's:
  - Closure: i/p set of items & o/p also set of items
  - GOTO

- Add everything from i/p to o/p
- If A → α. βB is in closure(I) & β → б is in the grammar G
  - Then add β → .б to the closure(I)
  - A → α. βB
  - β → .б
- Repeat the previous step for every newly added item

- Goto(I, X)
  - Goto(I, X)
  - Goto(A → α.Xβ, X) = A → αX.β

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Procedure to construct LR parse Table

- LR parse table consist of two parts
  - Action
  - Goto

- **Action**
  - Action part consists of both shift & reduced operation that are performed on terminals.

- **Goto**
  - Goto parts consists of shift operation performed on Non-terminals

KNOWLEDGEGATE

Goto(I_i, X) = I_j (X is terminal)

	X
I_i	S_j

Goto(I_i, X) = I_j (X is non-terminal)

	X
I_i	j

If I_i is any final item & represent the production R_i, then place R_i under all the terminal symbols in the action part of the table.

	t_1	t_2	t_3					t_n	$
I_i	r_i	r_i	r_i					r_i	r_i

KNOWLEDGE GATE

S → AA

A → aA / b

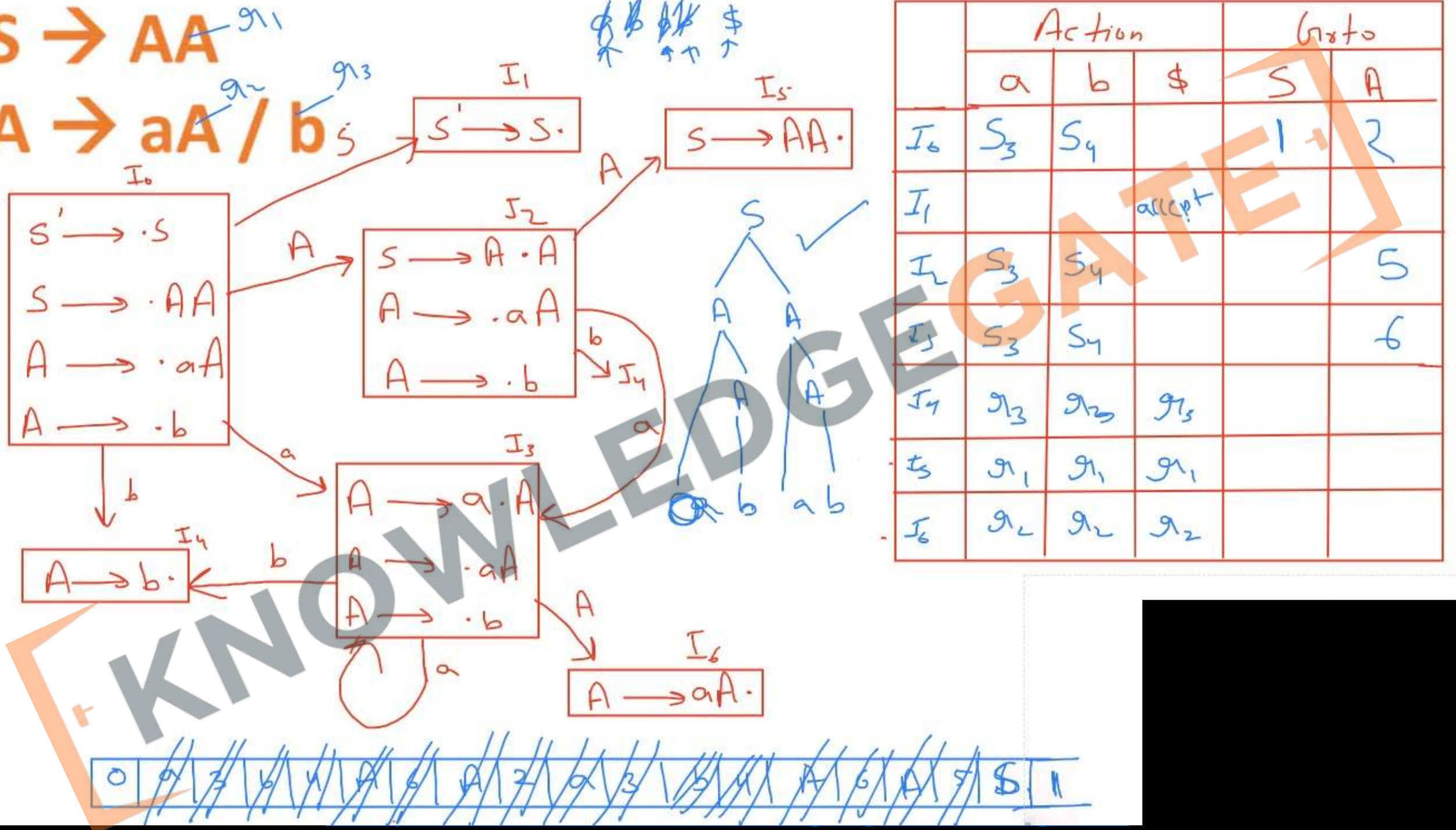
	Action			Goto	

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

$S \rightarrow AA$ g_1

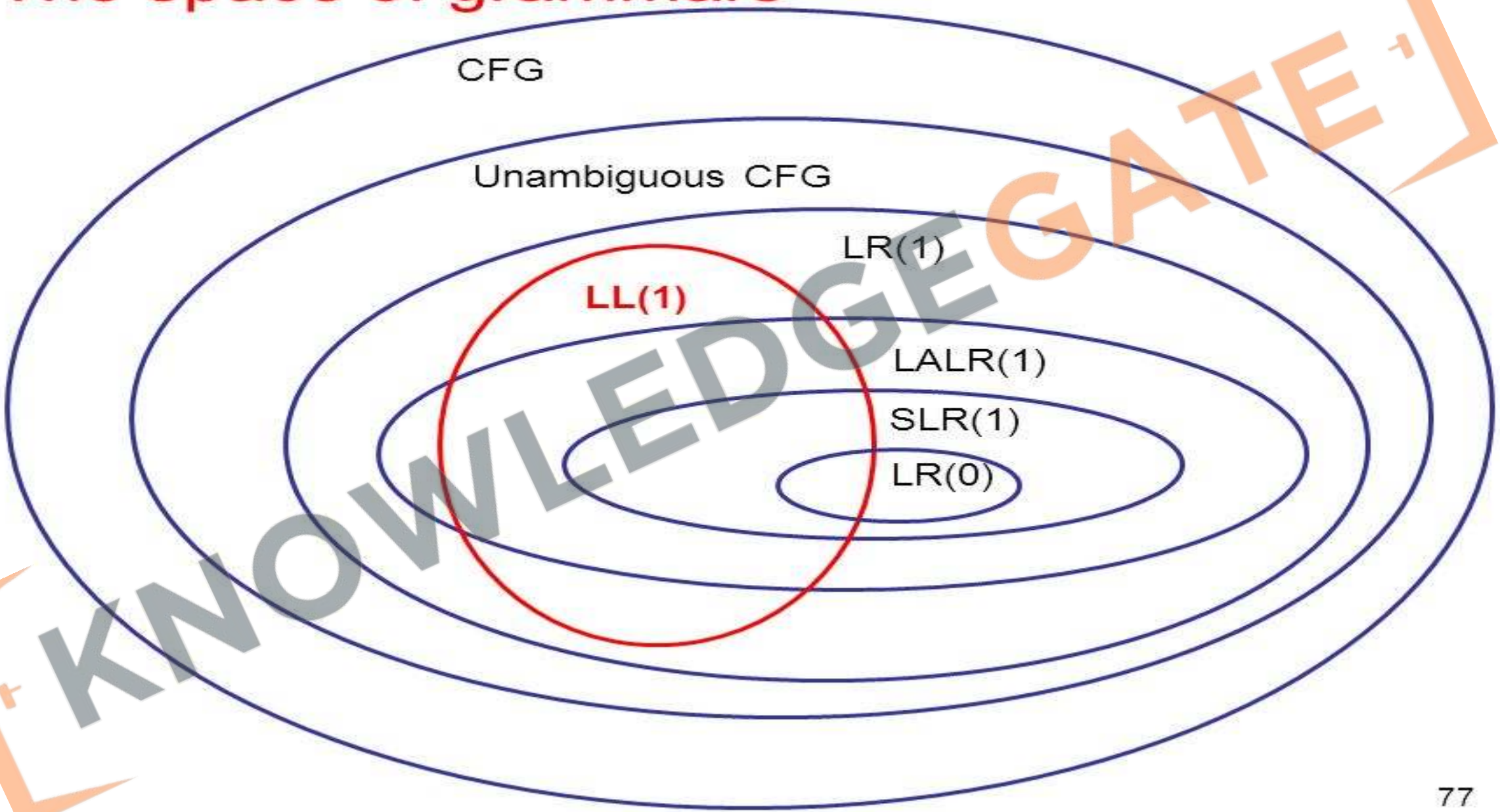
$A \rightarrow aA$ g_2 / b g_3



	Action			Goto	
	a	b	$	S	A
I_0	S_3	S_4		1	2
I_1			accept		
I_2	S_3	S_4			5
I_3	S_3	S_4			6
I_4	g_3	g_3	g_3		
I_5	g_1	g_1	g_1		
I_6	g_2	g_2	g_2		

KNOWLEDGEGATE

The space of grammars



KNOWLEDGEGATE

$E \rightarrow T + E \mid T$

$T \rightarrow id$

	Action			Goto	

KNOWLEDGEGATE

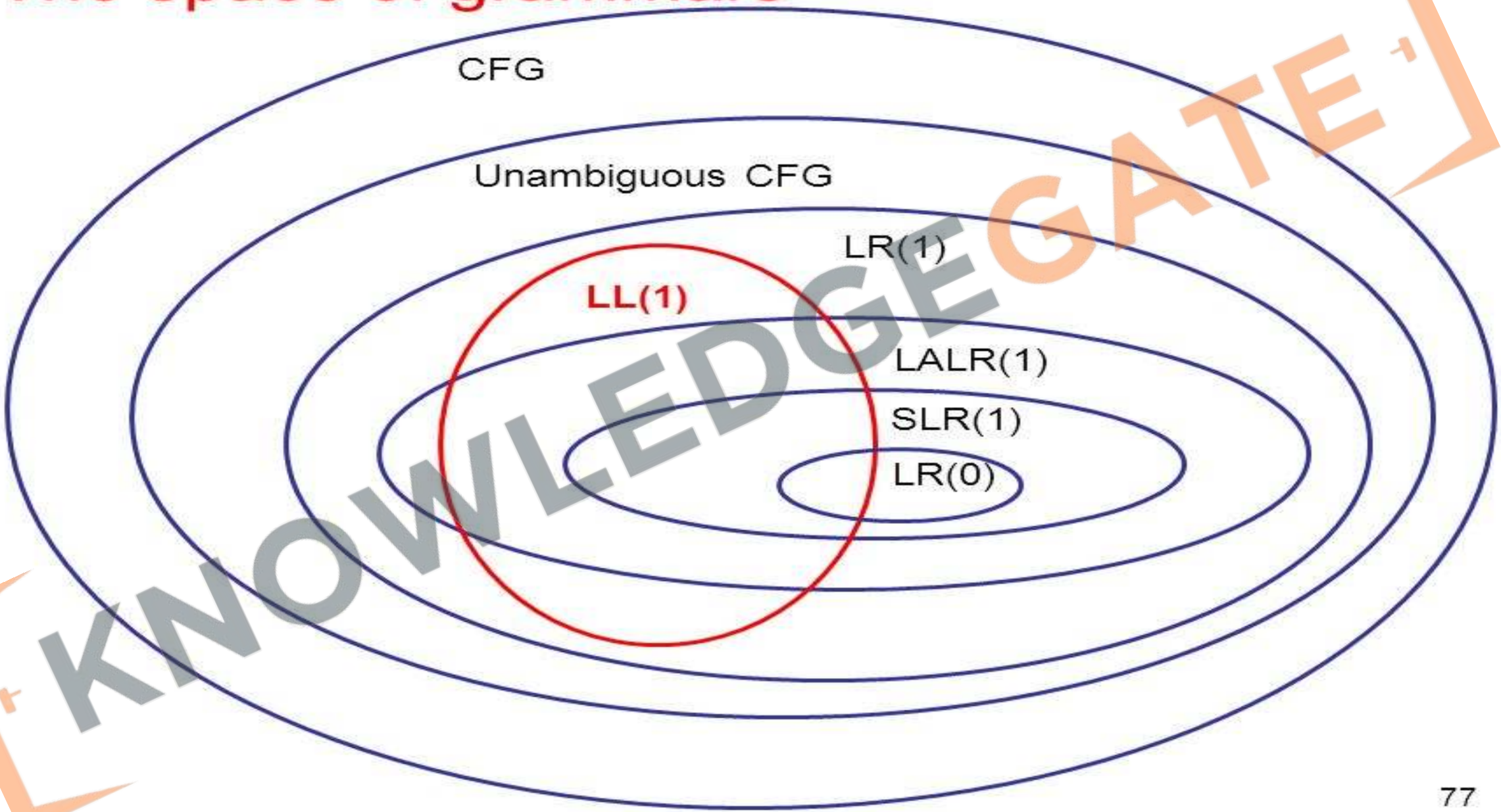
http://www.knowledgegate.in/gate

SLR(1)

- The procedure for constructing the parse table is similar to LR(0) parse, but there is a restriction in the reducing entries.
- Whenever there is a final item, then placed the reduced entries under the follow symbol of LHS symbol.
- If the SLR(1) parse table is free from multiple entries than the grammar is SLR(1) grammar.
- Every LR(0) grammar is SLR(1), but every SLR(1) grammar need not be LR(0)
- SLR(1) parser is more powerful then LR(0) parser

KNOWLEDGEGATE

The space of grammars



KNOWLEDGE GATE

S → CC

C → cC

C → d

CLR(1)

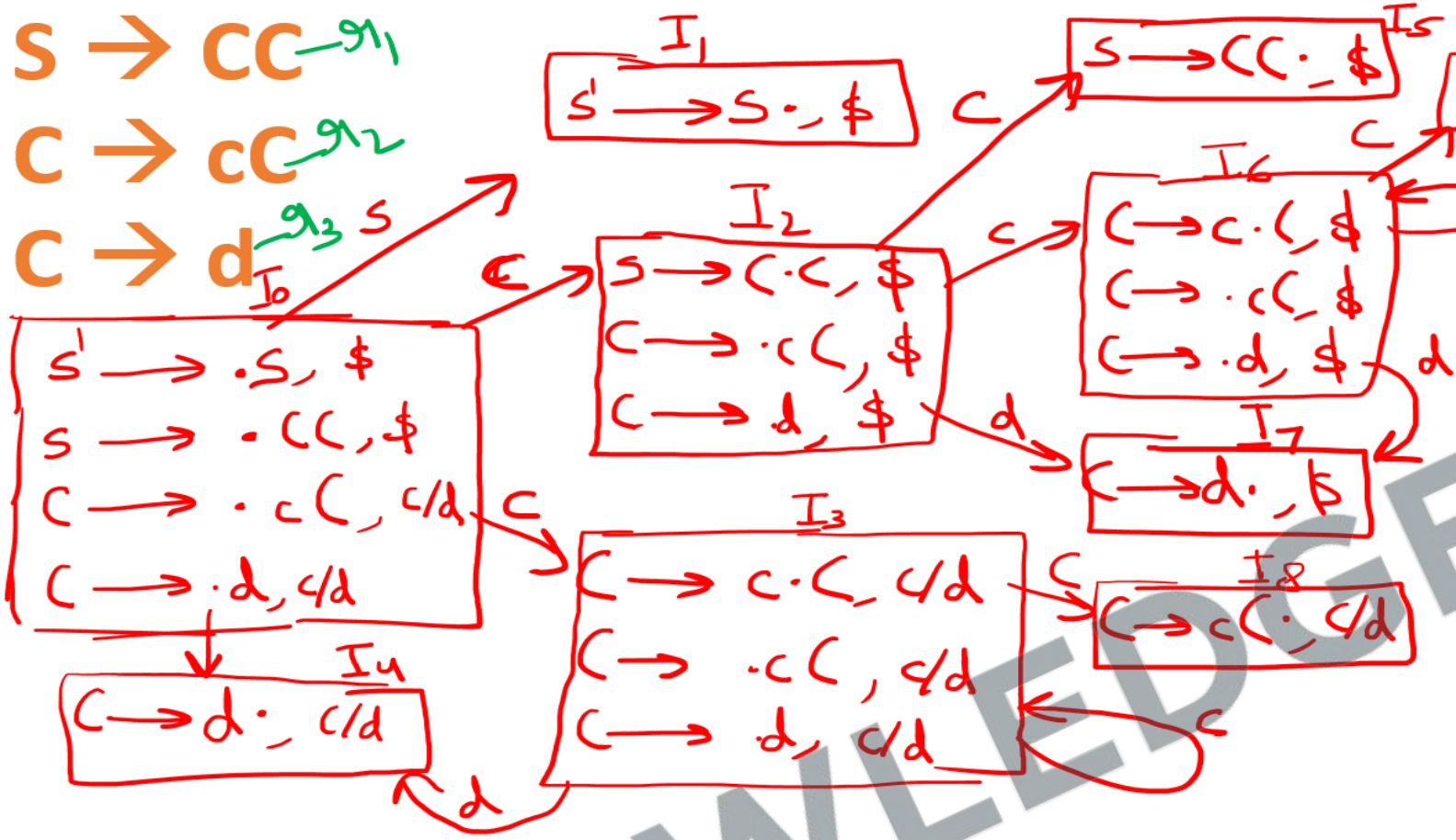
	Action			Goto	

KNOWLEDGEGATE

$S \rightarrow CC$ — r_1

$C \rightarrow cC$ — r_2

$C \rightarrow d$ — r_3



	action			goto	
	c	d	$	S	C
I_0	S_3	S_4		1	2
I_1			accept		
I_2	S_6	S_7			5
I_3	S_3	S_4			8
I_4	r_3	r_3			
I_5			r_1		
I_6	S_6	S_7			9
I_7			r_3		
I_8	r_2	r_2			
I_9			r_2		

KNOWLEDGEGATE

CLR(1)

- LR(1) depends on one look Ahead symbol

- Closure(I):
  - Add everything from i/p to o/p
  - A → α.Bβ, $ is in closure I and β → б is in the grammar G, then add β → .б, first(β, $), to the closure I
  - repeat previous step for every newly added items

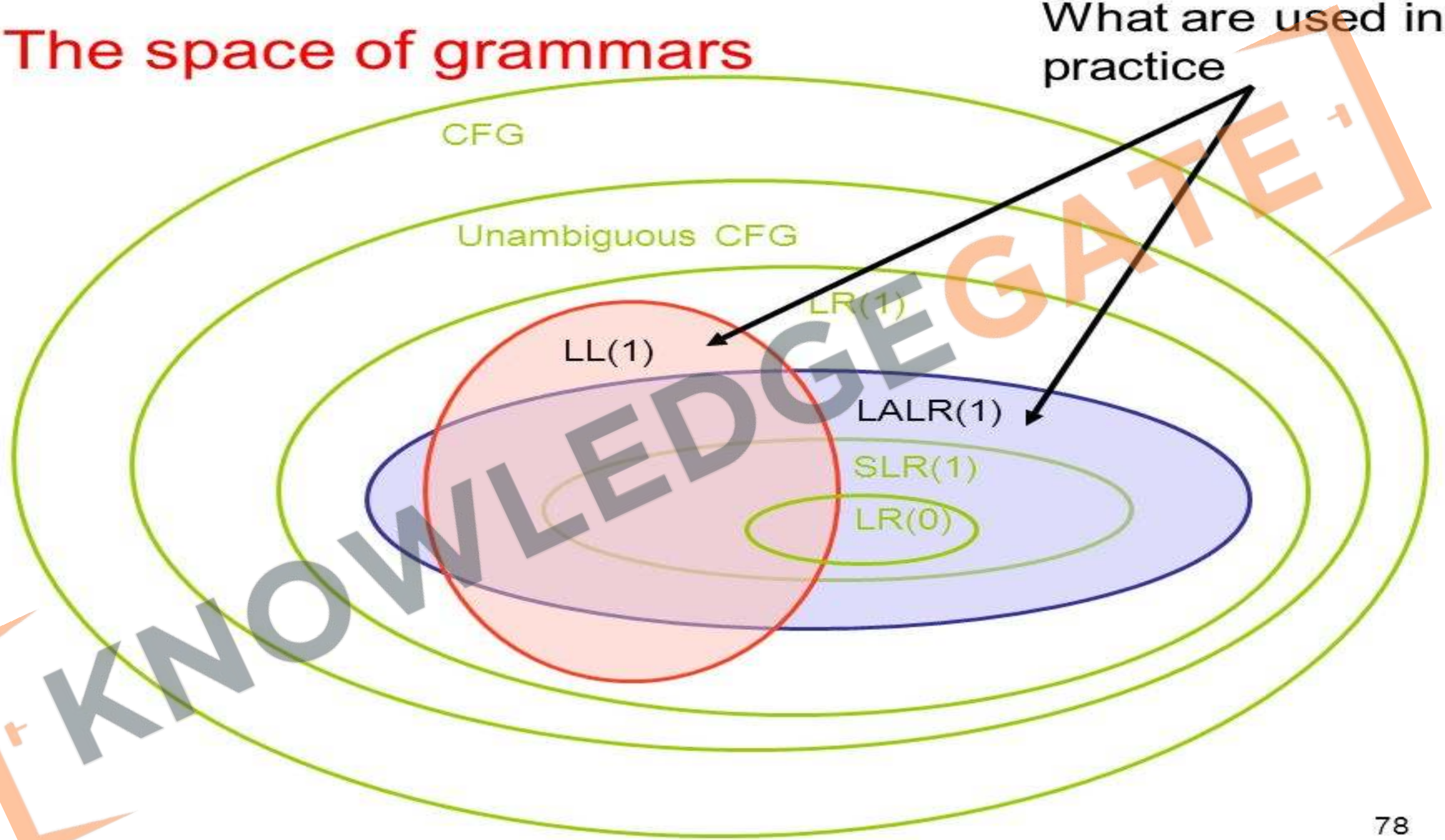
- Goto(I, x):
  - there will not be any change in the goto part while finding the transition.
  - these many be change in the follow or look Ahead part while finding the closure.

KNOWLEDGEGATE

- LR(1) grammar: the grammar for, which LR(1) is constructed is known as LR(1) or CLR(1).
- the grammar whose LR(1) parse is free from multiple entries or conflicts, then it is LR(1) grammar.
- Every SLR(1) grammar is CLR(1). but every CLR(1) grammar need not be SLR(1)
- CLR(1) is more powerful than SLR(1)

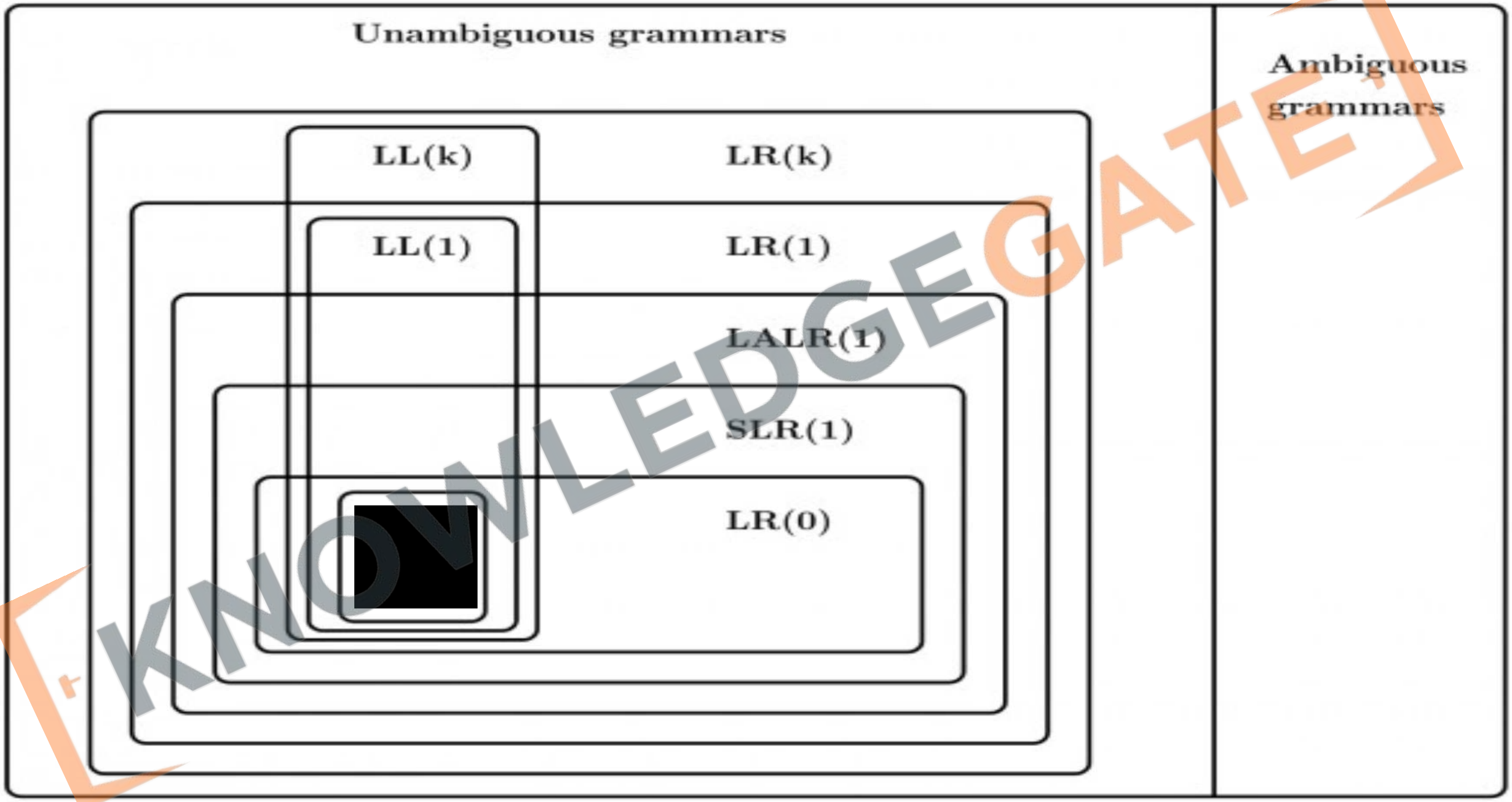
KNOWLEDGEGATE

The space of grammars



KNOWLEDGEGATE

LL(1) versus LR(k)



KNOWLEDGEGATE

- In CLR(1) parser, we can find some of the state contains more than one production whose production part is same but follow part is different.

- because of this reason, the CLR(1) parse contains more no of entries & hence CLR(1) parser become most costly.

KNOWLEDGEGATE

LALR(1)

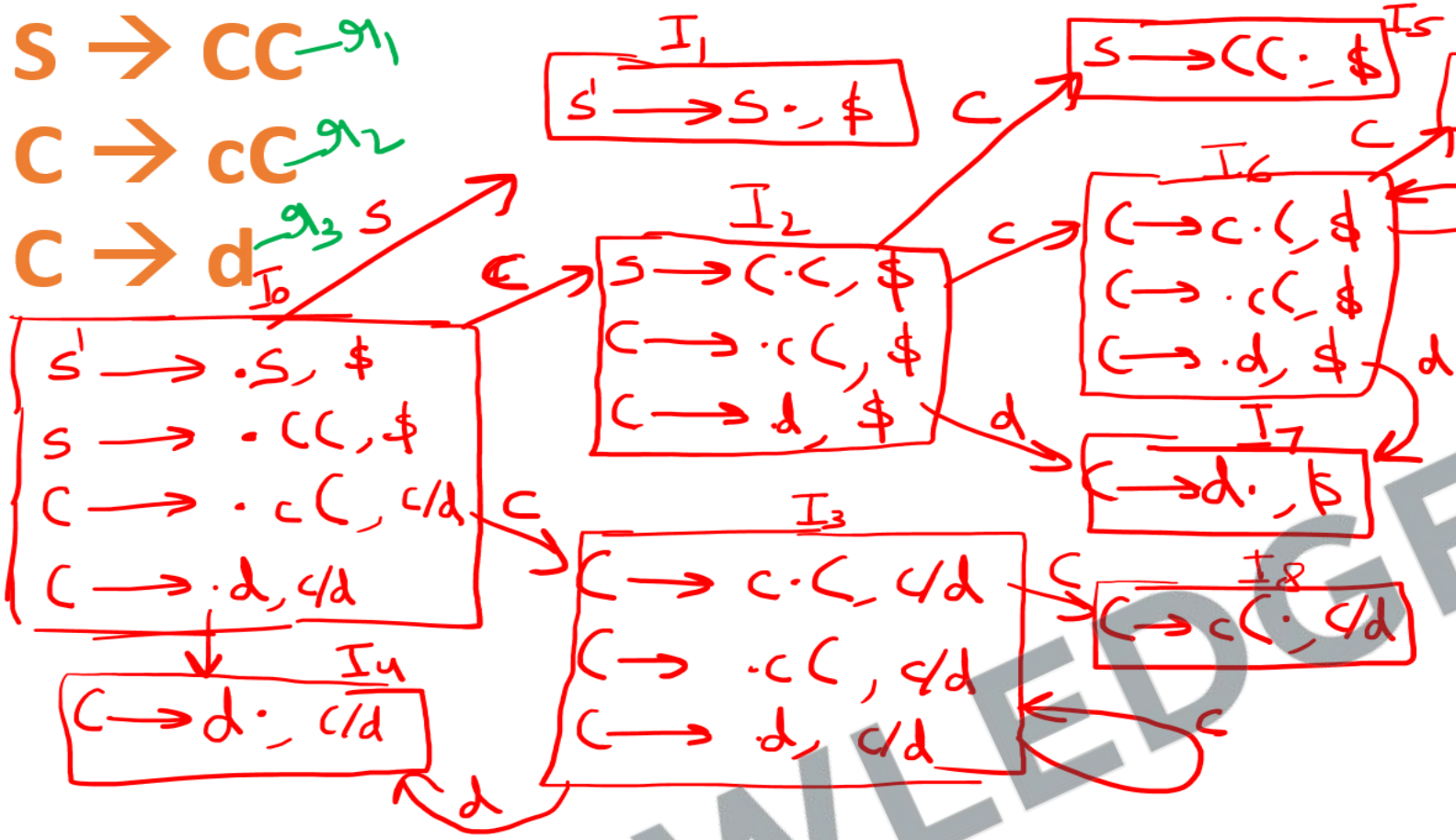
- In CLR(1) parser, there can ne more than one state, having same production part and different follow part. Now combine those state whose production part is common and follow part is different, it is a single state and then construct the parse table, if the parse table is free from multiple entries, then the grammar is LALR(1).

KNOWLEDGEGATE

$S \rightarrow CC$ — r_1

$C \rightarrow cC$ — r_2

$C \rightarrow d$ — r_3



	action			goto	
	c	d	$	S	C
I_0	S_3	S_4		1	2
I_1			accept		
I_2	S_6	S_7			5
I_3	S_3	S_4			8
I_4	r_3	r_3			
I_5			r_1		
I_6	S_6	S_7			9
I_7			r_3		
I_8	r_2	r_2			
I_9			r_2		

KNOWLEDGEGATE

S → Aa / bAc /dc /bda

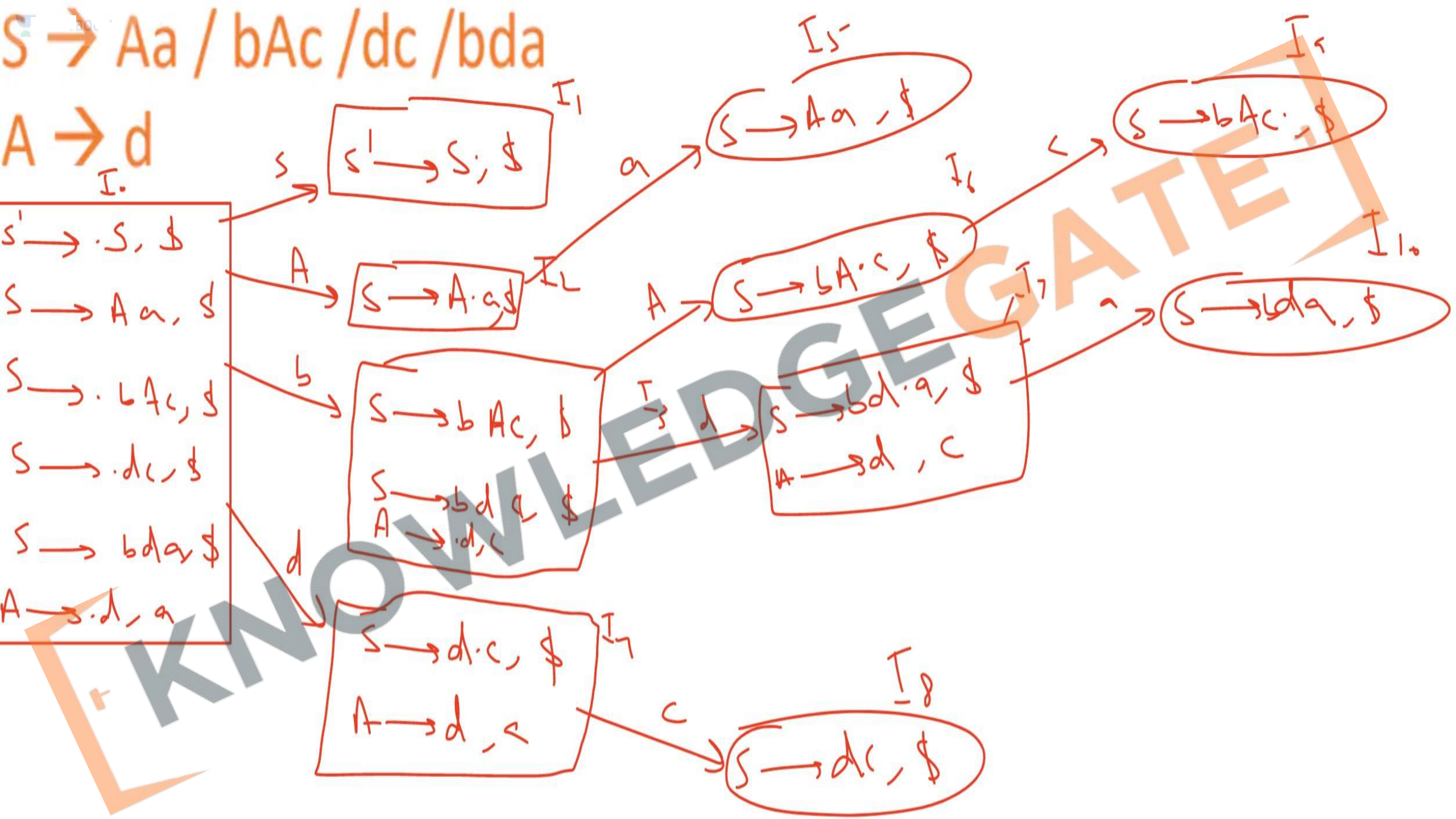
A → d

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

$S \rightarrow Aa \mid bAc \mid dc \mid bda$

$A \rightarrow d$



KNOWLEDGEGATE

- if in CLR(1), if there are no states having some production, but different follow part, the grammar is CLR(1) and LALR(1)

KNOWLEDGEGATE

S → Aa

S → bAc

S → Bc

S → bBa

A → d

B → d

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

$S \to Aa$

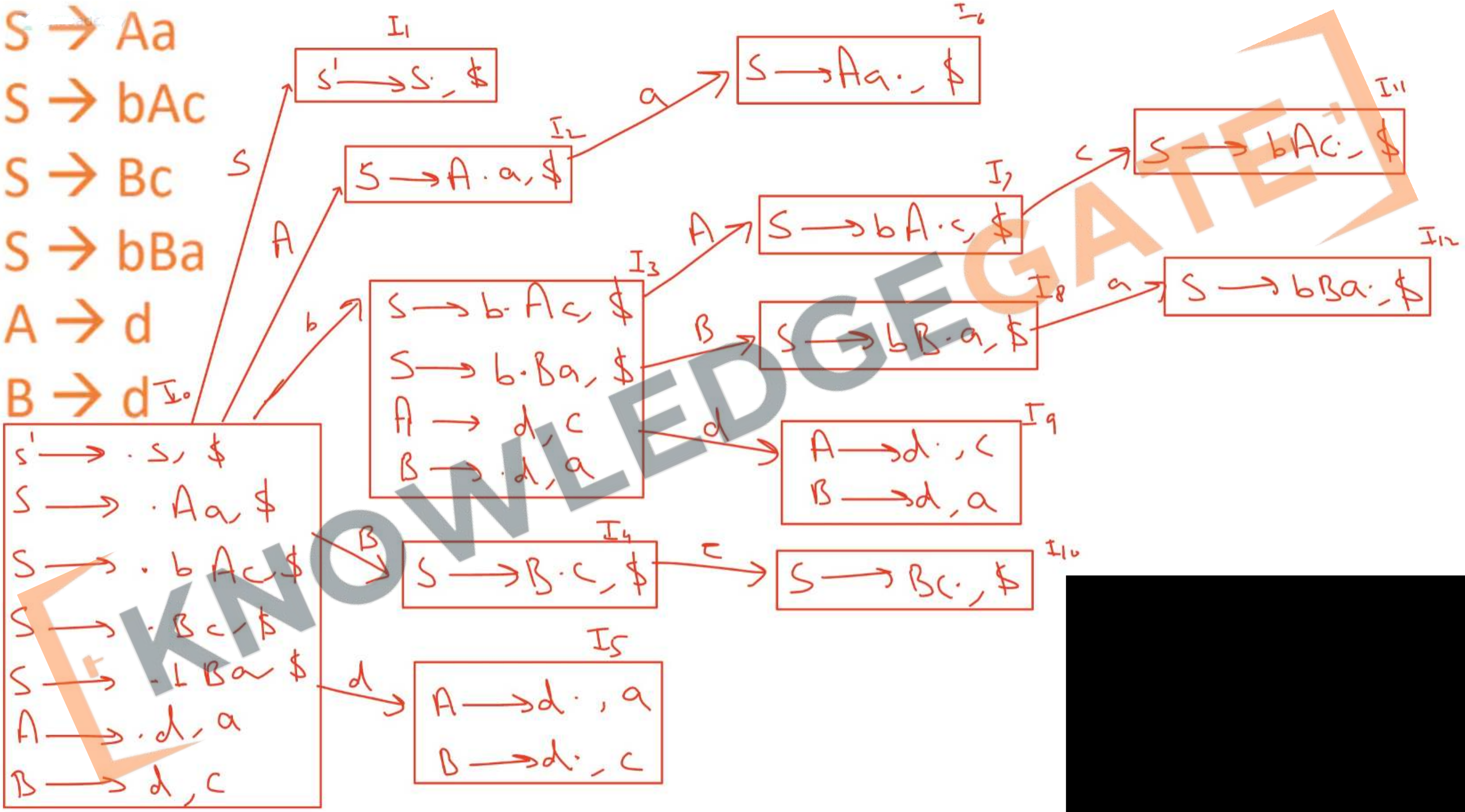
$S \to bAc$

$S \to Bc$

$S \to bBa$

$A \to d$

$B \to d$



KNOWLEDGE GATE

Operator Precedence Grammar

- Operator precedence parser can be constructed for both ambiguous and unambiguous grammar.
- In general operator precedence grammar have less complexity
- Every CFG is not operator precedence grammar
- Generally used for languages which are useful in scientific application.

KNOWLEDGE GATE

- Operator Grammar is a context free grammar that has following properties
  - Does not contain ε production
  - No adjacent non-terminals on RHS of any production.

S → AB
A → a
B → b

S → AaB
B → aA / b
A → b
B → a

S → AaB
A → a / ε
B → b

S → AOB / int
O → + / * / -
A → b
B → a

KNOWLEDGEGATE

Algorithm for computing precedence function

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$

1. $a :> b$
2. $a <\cdot b$
3. $a \doteq b$

KNOWLEDGEGATE

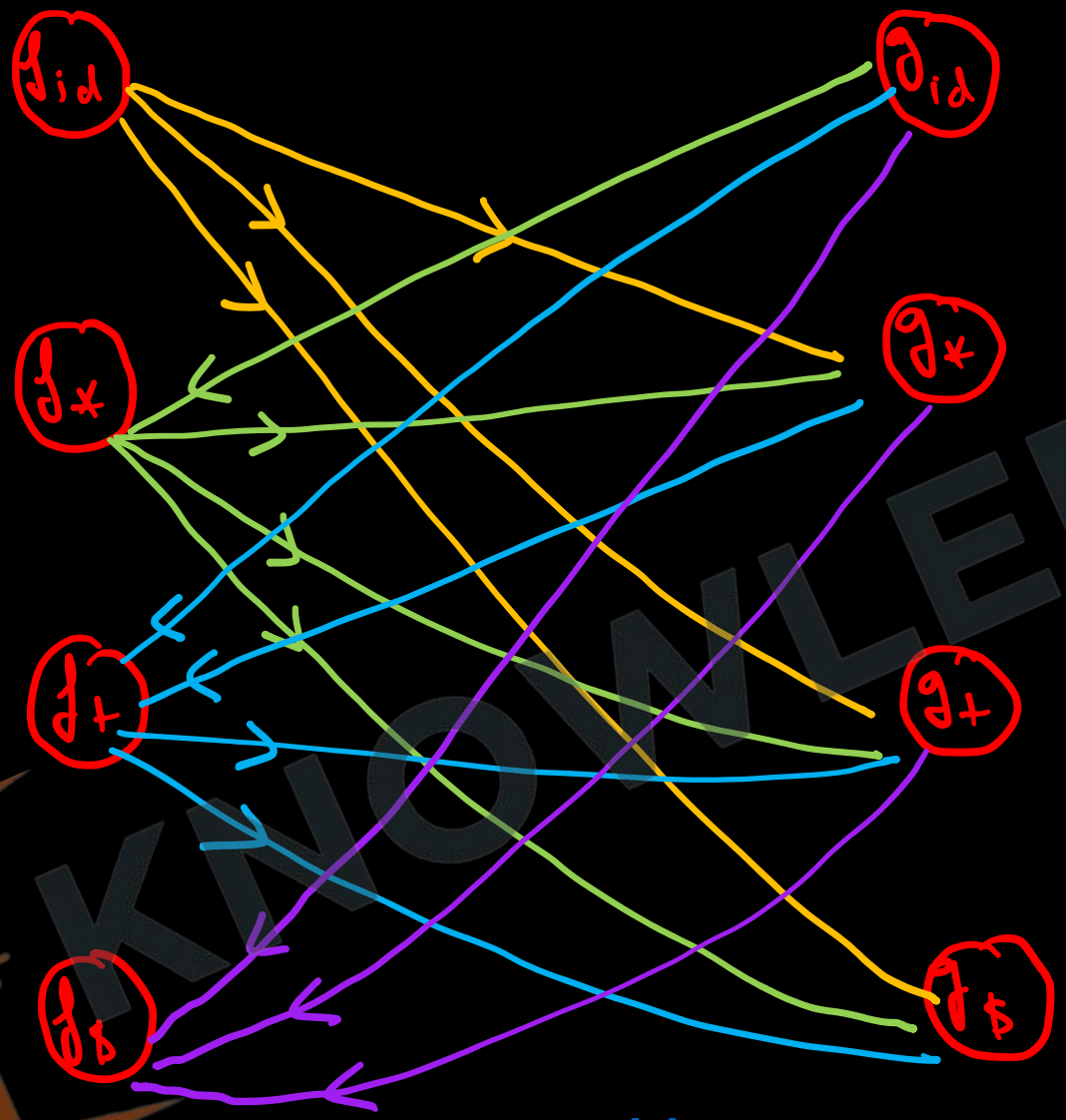
- Now consider the string *id + id * id*
- We will insert $ symbols at the start and end of the input string. We will also insert precedence operator by referring the precedence relation table.
- $ < · *id* · > + < · *id* · > * < *id* · > $
- We will follow following steps to parse the given string :
  - Scan the input from left to right until first · > is encountered.
  - Scan backwards over = until < · is encountered.
  - The handle is a string between < · and · >.

$ < · id · > + < · id · > * < · id · > $	Handle id is obtained between <··>. Reduce this by E id.
E + < · id · > * < · id · > $	Handle id is obtained between <··>. Reduce this by E id.
E + E * < · id · > $	Handle id is obtained between <··>. Reduce this by E id.
E+E*E	Remove all the non-terminals.
+*	Insert $ at the beginning and at the end. Also insert the precedence operators.
$<·+<·*·>$	The * operator is surrounded by <··>. This indicates that * becomes handle. That means, we have to reduce E * E operation first.
$<·+·>$	Now + becomes handle. Hence, we evaluate E + E.
$$	Parsing is done.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Algorithm for computing precedence function



f_{id} → g_* → f_* → g_+ → f_+ → $g_\$$

g_{id} → f_* → g_* → f_+ → g_+ → $f_\$$

	id	*	+	$
id	-	>	>	>
*	<	>	>	>
+	<	<	>	>
$	<	<	<	-

	id	*	+	$
f	4	4	2	0
g	5	3	1	0

KNOWLEDGEGATE

Aspect	Top-Down Parsing	Bottom-Up Parsing
Direction of Analysis	Begins from the start symbol and works towards the leaves of the syntax tree.	Starts from the leaves (input symbols) and works towards the root of the syntax tree.
Parse Tree Construction	Constructs the parse tree from the top (root) to the bottom (leaves).	Constructs the parse tree from the bottom (leaves) to the top (root).
Type of Grammar Used	Generally uses non-left-recursive grammars.	Can handle a more extensive range of grammars, including left-recursive grammars.
Example Methods	Recursive Descent Parsing, LL(k) Parsing (where k is the number of lookahead tokens).	LR(k) Parsing (including SLR, LALR, CLR), Operator-precedence parsing.
Handling Ambiguity	Less efficient in handling ambiguity and requires backtracking in some cases.	More efficient in handling ambiguous grammars and reduces the need for backtracking.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Chapter-3

(SYNTAX-DIRECTED TRANSLATION): Syntax-directed Translation schemes, Implementation of Syntax- directed Translators, Intermediate code, postfix notation, Parse trees & syntax trees, three address code, quadruple & triples, translation of assignment statements, Boolean expressions, statements that alter the flow of control, postfix translation, translation with a top down parser. More about translation: Array references in arithmetic expressions, procedures call, declarations and case statements.

http://www.knowledgegate.in/gate

Semantic Analysis

Input and output: Semantic analysis takes an Abstract Syntax Tree (AST) generated by the syntax analysis phase as its inputs.

Process: It performs type checking, scope resolution, and validates semantic consistency, ensuring that the operations and expressions in the source code are according to the language's rules and semantics.

KNOWLEDGEGATE

Semantic Analysis

- Grammar + Semantic Rule + Semantic Actions = Syntax Directed Translation. SDT is the generalization of CFG.
- With grammar we give meaningful rules, and apart from semantic analysis SDT can also be used to perform things like
  - Code generation
  - Intermediate code generation
  - Value in the symbol table
  - Expression evaluation
  - Converting infix to post fix
- Things can be done in parallel to parsing…so with semantic action and rule parsers become much powerful

KNOWLEDGEGATE

Q Consider the following grammar along with translation rules. Here # and % are operators and id is a token that represents an integer and $id._{val}$ represents the corresponding integer value. The set of non-terminals is {S,T, R, P} and a subscripted non-terminal indicates an instance of the non-terminal.Using this translation scheme, the computed value of $S._{val}$ for root of the parse tree for the expression 20 #10%5 #8%2%2 is _____.

20 #10%5 #8%2%2

$S \rightarrow S_1 \# T$	$\{S._{val} = S_{1}._{val} * T._{val}\}$
$S \rightarrow T$	$\{S._{val} = T._{val}\}$
$T \rightarrow T_1 \% R$	$\{T._{val} = T_{1}._{val} \div R._{val}\}$
$T \rightarrow R$	$\{T._{val} = R._{val}\}$
$R \rightarrow id$	$\{R._{val} = id._{val}\}$

KNOWLEDGEGATE

Q Consider the grammar with the following translation rules and E as the start symbol.

$E \rightarrow E_1$ # T { E.value = E_1.value * T.value }

$E \rightarrow T$ { E.value = T.value }

$T \rightarrow T_1$ & F { T.value = T_1.value + F.value }

$T \rightarrow F$ { T.value = F.value }

$F \rightarrow$ num { F.value = num.value }

Compute E.value for the root of the parse tree for the expression: 2 # 3 & 5 # 6 & 4.

2 # 3 & 5 # 6 & 4

KNOWLEDGEGATE

Q Consider the grammar with the following translation rules and E as the start symbol.

$E \rightarrow E_1 + T$ {print ('+');}

$E \rightarrow T$

$T \rightarrow T_1 * F$ {print ('*');}

$T \rightarrow F$

$F \rightarrow$ num {print ('num.val');}

Construct the parse tree for the string 2 + 3 * 4, and find what will be printed.

KNOWLEDGEGATE

Q Consider the translation scheme shown below S → T R

R → + T {print ('+');} R / ε

T → num {print (num.val);}

Here num is a token that represents an integer and num.val represents the corresponding integer value. For an input string '9 + 5 + 2', this translation scheme will print

KNOWLEDGEGATE

Q Consider the following translation scheme.

S → ER

R → *E{print(“*”);}R | ε

E → F + E {print(“+”);} | F

F → (S) | id {print(id.value);}

Here id is a token that represents an integer and id.value represents the corresponding integer value. For an input ‘2 * 3 + 4’, this translation scheme prints

KNOWLEDGE GATE

Q Consider the following Syntax Directed Translation Scheme (SDTS), with non-terminals {S, A} and terminals {a, b}}. Using the above SDTS, the output printed by a bottom-up parser, for the input 'aab' is

$$
\begin{aligned}
S &\longrightarrow aA \quad \{ \text{print } 1 \} \\
S &\longrightarrow a \quad \{ \text{print } 2 \} \\
A &\longrightarrow Sb \quad \{ \text{print } 3 \}
\end{aligned}
$$

KNOWLEDGEGATE

Attributes

- Attributes attach relevant information like strings, numbers, types, memory locations, or code fragments to grammar symbols of a language, which are used as labels for nodes in a parse tree.
- The value of each attribute at a parse tree node is determined by semantic rules associated with the production applied at that node, defining the context-specific information for the language construct.

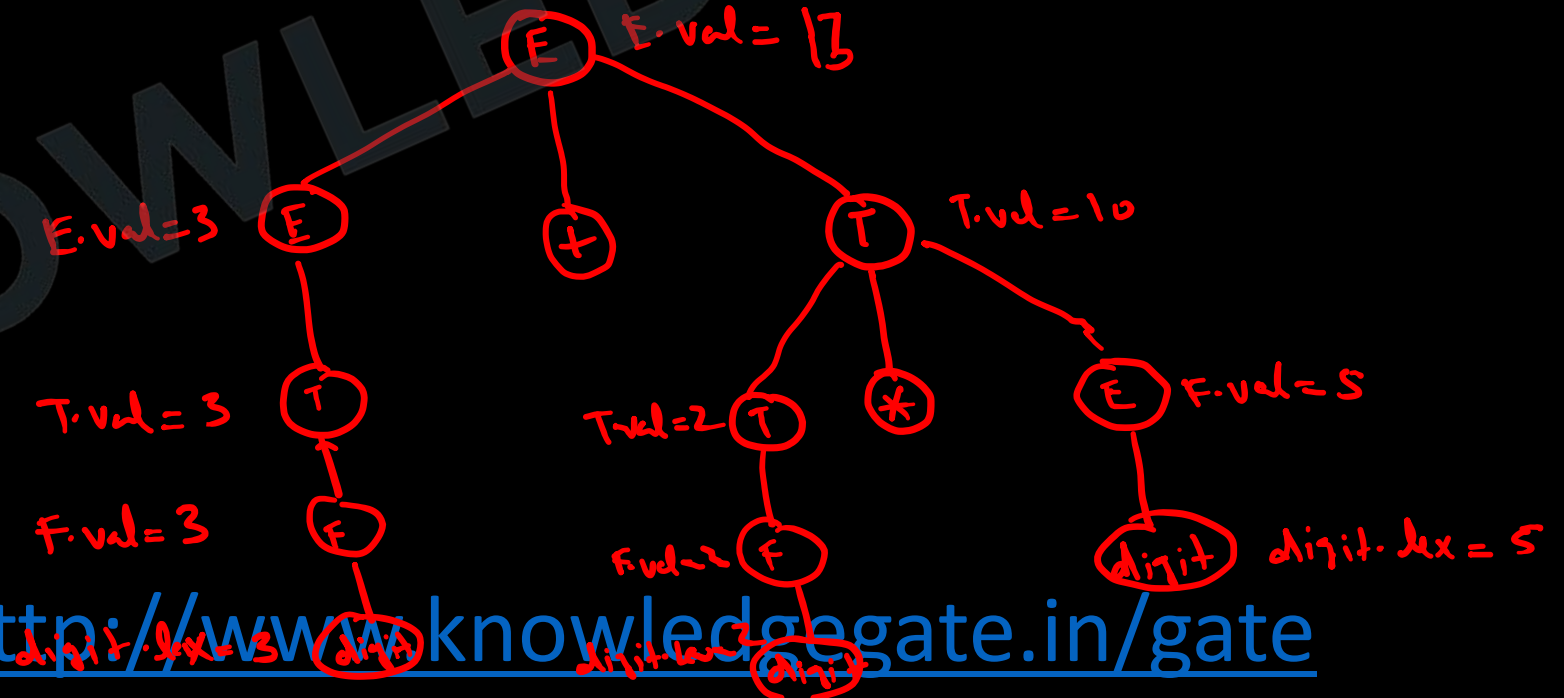
KNOWLEDGEGATE

Classification of Attributes

- Based on the process of Evaluation of the values, attributes are classified into two types:
  - Synthesised Attributes
  - Inherited Attributes

KNOWLEDGEGATE

- Synthesized attributes are derived from a node's children within a parse tree, and a syntax-directed definition relying solely on these attributes is termed as S-attributed.
- S-attributed definitions allow parse trees to be annotated from the leaves up to the root, enabling parsers to directly evaluate semantic rules during the parsing process.
- A → XYZ {A.S = f(X.S / Y.S / Z.S)}

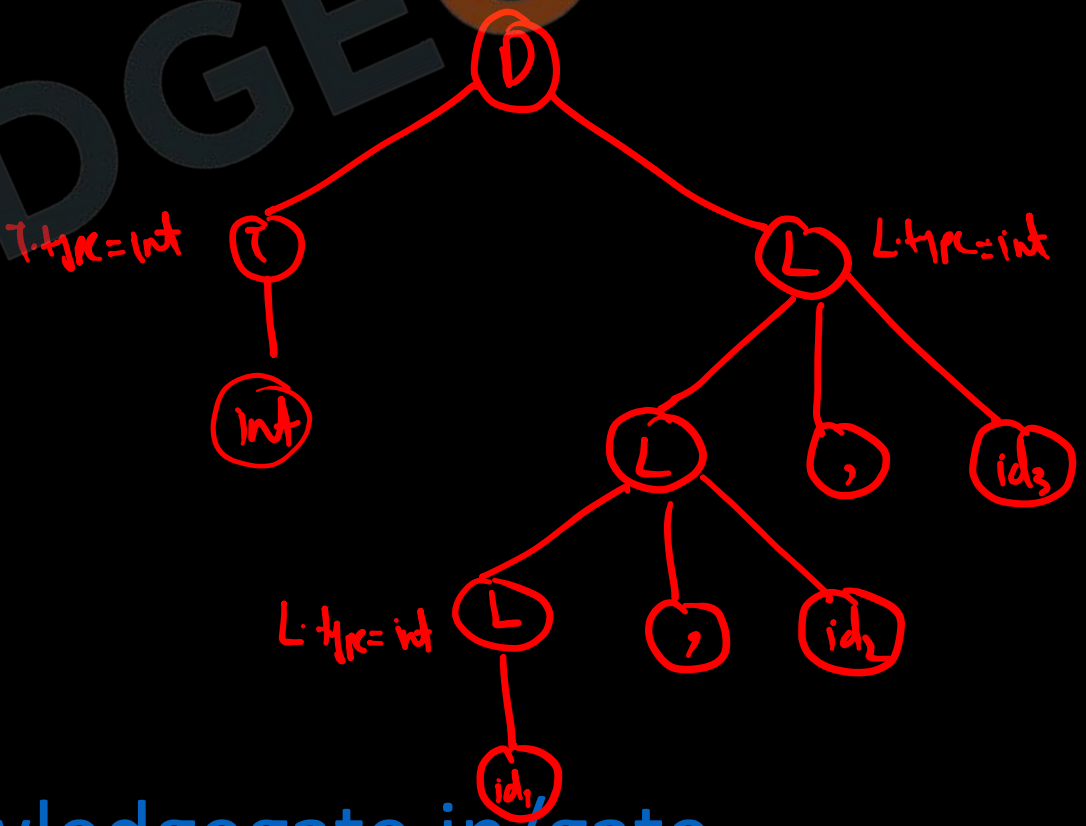


KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Inherited Attributes**: The attribute whose values are evaluated in terms of attribute value of parents & Left siblings is known as inherited attributes.
  - Inherited attributes are convenient for expressing dependence of a programming language construct on the context in which it appears.

$D \rightarrow T\ L$	$L.type = T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.type = L.type$
$L \rightarrow id$	enter(id.part, L.type)
	enter(id.part, L.type)



KNOWLEDGEGATE

Synthesized Attributes	Inherited Attributes
Computed from the attribute values of a node's children in the parse tree.	Computed from the attribute values of a node's siblings and parent.
Used to pass information up the parse tree.	Used to pass information down or across the parse tree.
Examples include the evaluated value of an expression or the size of a data type.	Examples include the data type expected for a child node or the environment in which a node should be evaluated.
Often associated with bottom-up parsing techniques like LALR or SLR.	Often associated with top-down parsing techniques such as LL parsers.
They do not need context from parent nodes, only from children and the node itself.	They require context from parent or surrounding nodes to be computed.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

S-Attributed SDT	L-Attributes SDT
Uses only Synthesized attributes	Uses both inherited and synthesised attributes. Each inherited attribute is restricted to inherit either form parent or left sibling only.
Semantic actions are placed at extreme right on right end of production	Semantic actions are placed anywhere on right hand side of the production.
Attributes are evaluated during BUP	Attributes are evaluated by traversing parse tree depth first left to right.

KNOWLEDGE GATE

Intermediate Code Generation

- Intermediate code generation in compilers creates a machine-independent, low-level representation of source code, facilitating optimization and making the compiler design more modular. This abstraction layer allows for:
  - **Portability**: Easier adaptation of the compiler to different machine architectures, as only the code generation phase needs to be machine-specific.
  - **Optimization Opportunities**: More efficient target code through optimizations performed on the intermediate form rather than on high-level source or machine code.
  - **Ease of Compiler Construction**: Simplifies the development and maintenance of the compiler by decoupling the source language from the machine code generation.

- $x = y + z * 60$

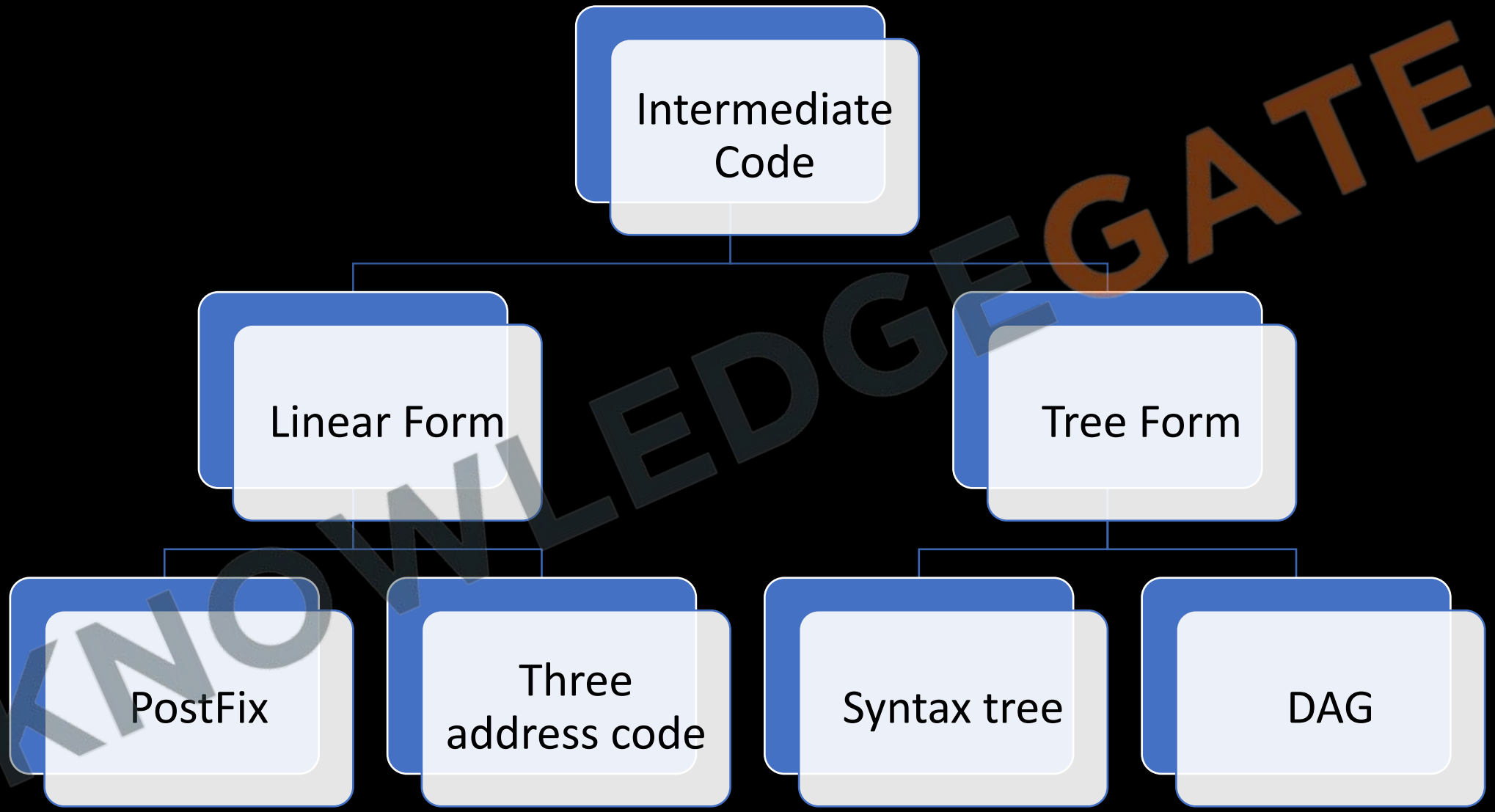
  $t_1 = z * 60$

  $t_2 = y + t_1$

  $x = t_2$

KNOWLEDGEGATE

Intermediate Code Generation



KNOWLEDGE GATE

Q Draw syntax tree for the arithmetic expressions : $a * (b + c) - d/2$. Also write the given expression in postfix notation **?**

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

① if e
then x
else
y

e ? x y → e x y ?

② if a
then if c - d
then a + c
else
a * c
else
a + b

a ? c - d ? a + c a * c a + b

a ? c d - ? a c + a c * a b +

a ? c d - a c + a c * ? a b +

a c d - a c + a c * ? a b + ?

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

(a+b) * (a + b + c)

Post fix

ab+ab+c+*

Three address code

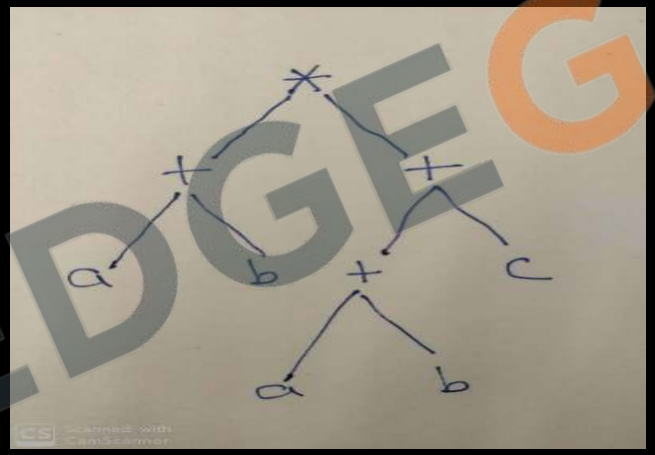
$t_1 = a+b$

$t_2 = a+b$

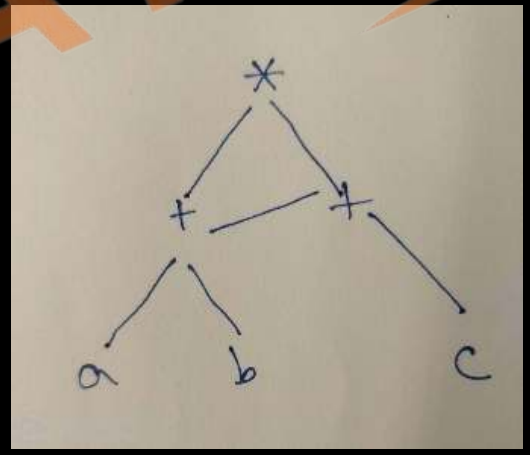
$t_3 = t_2 + c$

$t_4 = t_1 * t_3$

Syntax Tree



Direct Acyclic Graph



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

3 Address Code

- Three-address code is a type of intermediate code where each instruction can have at most three operands and one operator, like a := b op c. It simplifies complex operations into a sequence of simple statements, supporting various operators for arithmetic, logic, or boolean operations.

KNOWLEDGEGATE

3 Address Code

Types of 3 address codes

1) x = y operator z
2) x = operator z
3) x = y
4) goto L
5) A[i] = x
   y = A[i]
6) x = *p
   y = &x

KNOWLEDGEGATE

- 3 address codes can be implemented in a number of ways

-(a + b) * (c + d) + (a + b + c)

1) $t_1 = a+b$
2) $t_2 = -t_1$
3) $t_3 = c+d$
4) $t_4 = t_2 * t_3$
5) $t_5 = a+b$
6) $t_6 = t_5 + c$
7) $t_7 = t_4 + t_6$

KNOWLEDGEGATE

Quadruples

	Operator	$Operand_1$	$Operand_2$	Result
1)	+	a	b	t_1
2)	-	t_1		t_2
3)	+	c	d	t_3
4)	*	t_2	t_3	t_4
5)	+	a	b	t_5
6)	+	t_5	c	t_6
7)	+	t_4	t_6	t_7

1) $t_1 = a+b$
2) $t_2 = -t_1$
3) $t_3 = c+d$
4) $t_4 = t_2 * t_3$
5) $t_5 = a+b$
6) $t_6 = t_5 + c$
7) $t_7 = t_4 + t_6$

KNOWLEDGEGATE

Triplet

	Operator	$Operand_1$	$Operand_2$
1)	+	a	b
2)	-	1	
3)	+	c	d
4)	*	2	3
5)	+	a	b
6)	+	5	c
7)	+	4	6

1) $t_1 = a+b$
2) $t_2 = -t_1$
3) $t_3 = c+d$
4) $t_4 = t_2 * t_3$
5) $t_5 = a+b$
6) $t_6 = t_5 + c$
7) $t_7 = t_4 + t_6$

- **Advantage**
  - Space is not wasted
- **Disadvantage**
  - Statement cannot be moved

KNOWLEDGEGATE

Indirect Triplet

Triple can be separated by order of execution and uses the pointers concepts

- **Advantage**
  - Statement can be moved
- **Disadvantage**
  - two memory access

KNOWLEDGEGATE

Q Write the quadruples, triple and indirect triple for the following expression : **(*x*+*y*)* (*y*+*z*)+(*x*+*y*+*z*) ?**

$t_1 = x + y$

$t_2 = y + z$

$t_3 = t_1 + t_2$

$t_4 = t_1 + z$

$t_5 = t_3 + t_4$

Index	Operator	Arg1	Arg2	Result
1	+	x	y	t1
2	+	y	z	t2
3	*	t1	t2	t3
4	+	x	y	t4
5	+	t4	z	t5
6	+	t3	t5	t6

KNOWLEDGEGATE

Q Write the quadruples, triple and indirect triple for the following expression : **(*x*+*y*)* (*y*+*z*)+(*x*+*y*+*z*) ?**

$t_1 = x + y$

$t_2 = y + z$

$t_3 = t_1 + t_2$

$t_4 = t_1 + z$

$t_5 = t_3 + t_4$

Index	Operator	Arg1	Arg2
0	+	x	y
1	+	y	z
2	*	0	1
3	+	x	y
4	+	3	z
5	+	2	4

KNOWLEDGEGATE

Q Write the quadruples, triple and indirect triple for the following expression : **(*x*+*y*)* (*y*+*z*)+(*x*+*y*+*z*) ?**

$t_1 = x + y$

$t_2 = y + z$

$t_3 = t_1 + t_2$

$t_4 = t_1 + z$

$t_5 = t_3 + t_4$

Index	Operator	Arg1	Arg2
0	+	x	y
1	+	y	z
2	*	0	1
3	+	x	y
4	+	3	z
5	+	2	4

Pointer	Index
p0	0
p1	1
p2	2
p3	3
p4	4
p5	5

KNOWLEDGEGATE

One Dimensional array

- Address of the element at k^{th} index
  - a[k] = B + W*k
  - a[k] = B + W*(k – Lower bound)

- B is the base address of the array
- W is the size of each element
- K is the index of the element
- Lower bound index of the first element of the array
- Upper bound index of the last element of the array

$a[k] = Addr_A + w \times k - w \times LB$

$a[k] = \underbrace{Addr_A - w}_{t_2} + \underbrace{w * k}_{t_1}$

$A[i] = t_2[t_1]$

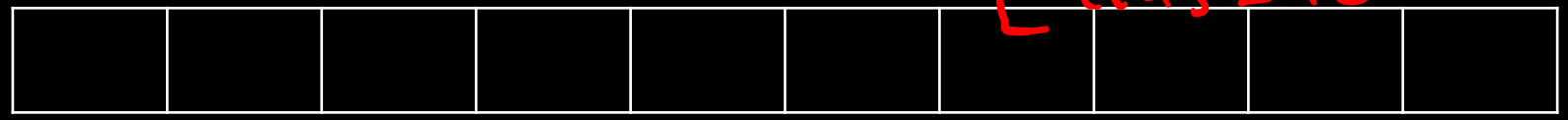
i] [LB = 1]

e.g. $a[i] = 10$ $w = 4$

↳ $t_1 = 4 \times i$

$t_2 = Addr_A - 4$

$t_2[t_1] = 10$



KNOWLEDGEGATE

if(a, b) then t=1
else t = 0

i) if (a<b) goto (i+3)
i+1) t=0
i+2) goto (i+4)
i+3) t=1
i+4) exit

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

while(C) do S

i) if (E) goto i+2
i+1) goto i+4
i+2) S
i+3) goto i
i+4) exit

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

```
for(1=0; i<10 ; i++)
    s
```

i) i = 0
i+1) if(i<10) goto i+3
i+2) goto i+6
i+3) S
i+4) i = i + 1
i+5) goto i+1
i+6) exit

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Generate three address code for the following code :

```
switch a + b
{
    case 1 : x = x + 1
    case 2 : y = y + 2
    case 3 : z = z + 3
    default : c = c – 1
}
```

101: $t_1 = a + b$ goto 103
102 : goto 115
103 : $t = 1$ goto 105
104 : goto 107
105: $t_2 = x + 1$
106 : $x = t_2$
107 : if $t = 2$ goto 109
108 : goto 111
109: $t_3 = y + 2$
110 : $y = t_3$
111 : if $t = 3$ goto 113
112 : goto 115 113: $t_4 = z + 3$
114 : $z = t_4$
115: $t_5 = c - 1$
116 : $c = t_5$
117 : Next statement

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the grammar with the following translation rules and E as the start symbol.

$E \rightarrow E_1 + T$ { E.nptr = mknode(E_1.nptr, +, T.ptr);}

$E \rightarrow T${ E.nptr = T.nptr }

$T \rightarrow T_1 * F$ { T.nptr = mknode(T_1.nptr, *, F.ptr);}

$T \rightarrow F${ T.nptr = F.nptr }

$F \rightarrow$ id { F.nptr = mknode(null, id name, null);}

Construct the parse tree for the expression: 2+3*4

KNOWLEDGEGATE

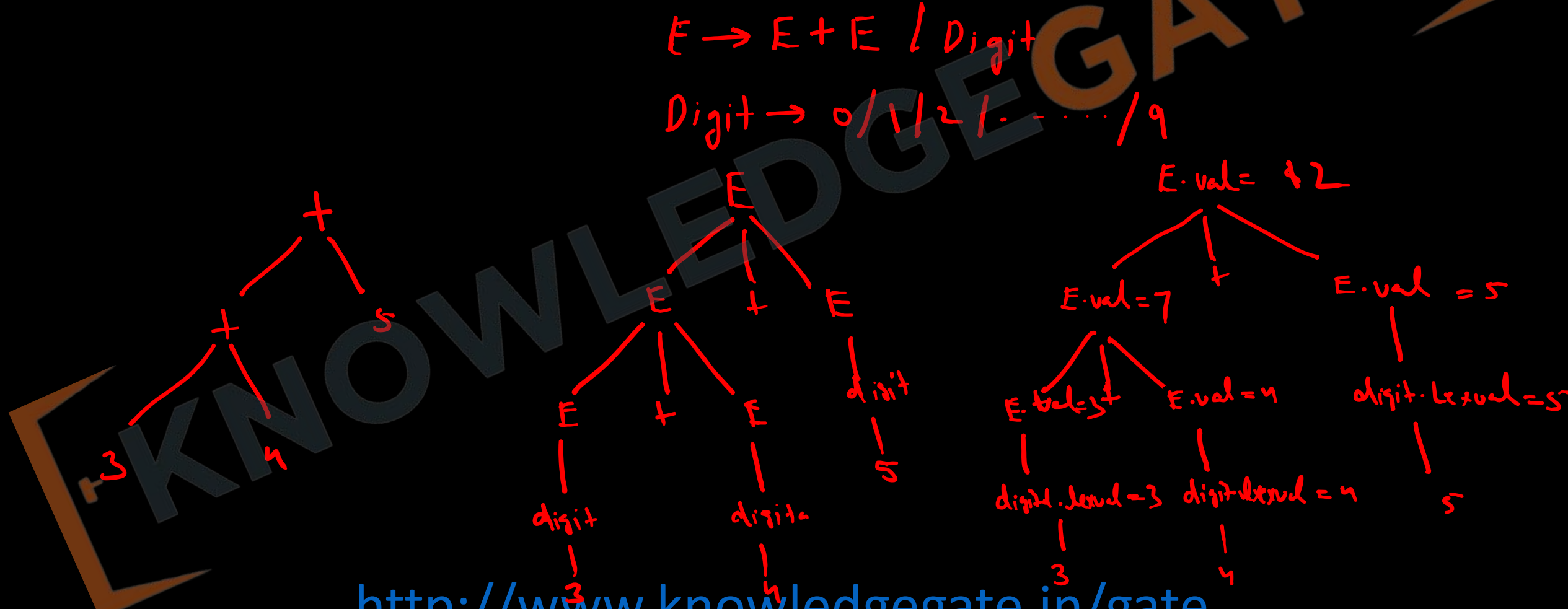
Types of 3-address Statements in intermediate Code

Assignment Statement	X = y op z
Assignment Instruction	X = op y
Copy Statement	X = y
Unconditional Jump	Goto L
Conditional Jump	If x relop y goto L
Procedure Call	Parm x1 Parm x2 . Parmxn Call p,n Return y
Array Statement	X = y[i] X[i] = y
Address and Pointer Assignment	X=&y X=*y *x=y

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Syntax Tree**: An abstract representation of the syntactic structure of a program, omitting syntactical elements like brackets and punctuation.
- **Parse Tree**: A detailed tree diagram showing all the syntactical elements of a program, including brackets, punctuation, and keywords.
- **Annotated Parse Tree**: A parse tree enhanced with additional information like values, types, or variable bindings, useful for semantic analysis and code generation.



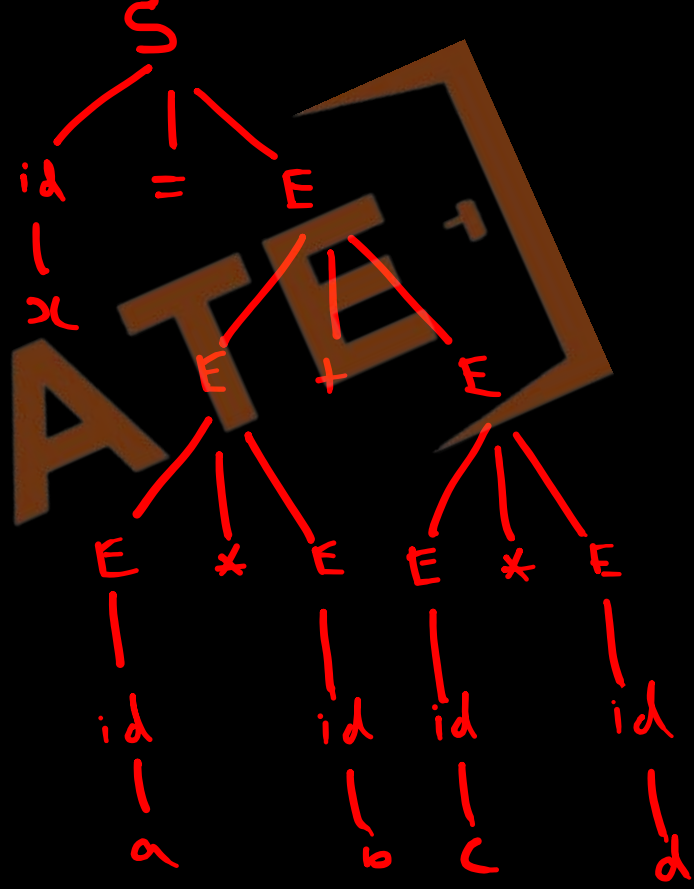
KNOWLEDGEGATE

SDT of Boolean Expression

$E \rightarrow E_1$ OR E_2	E.place=newtemp(); Emit(E.place=E_1.place 'or' E_2.place);
$E \rightarrow E_1$ AND E_2	E.place=newtemp(); Emit(E.place=E_1.place 'and' E_2.place);
$E \rightarrow$ NOT E_1	E.place=newtemp(); Emit(E.place= 'not' E_1.place);
$E \rightarrow (E_1)$	E.place=E_1.place;
$E \rightarrow$ TRUE	E.place=newtemp(); Emit(E.place='1');
$E \rightarrow$ FALSE	E.place=newtemp(); Emit(E.place='0');

KNOWLEDGE GATE
http://www.knowledgegate.in/gate

Production rule	Semantic actions
$S \rightarrow id := E$	{ id_entry := look_up(id.name); if id_entry != nil then append (id_entry ':=' E.place) else error; /* id not declared*/ }
$E \rightarrow E_1 + E_2$	{ E.place := newtemp(); append (E.place ':=' E_1.place '+' E_2.place) }
$E \rightarrow E_1 * E_2$	{ E.place := newtemp(); append (E.place ':=' E_1.place '*' E_2.place) }
$E \rightarrow - E_1$	{ E.place := newtemp(); append (E.place ':=' 'minus' E_1.place) }
$E \rightarrow (E_1)$	{ E. place:= E_1.place }
$E \rightarrow id$	{ id_entry: = look_up(id.name); if id_entry != nil then append (id_entry ':=' E.place) else error; /* id not declared*/ }



$t_1 = a + b$

$t_2 = c + d$

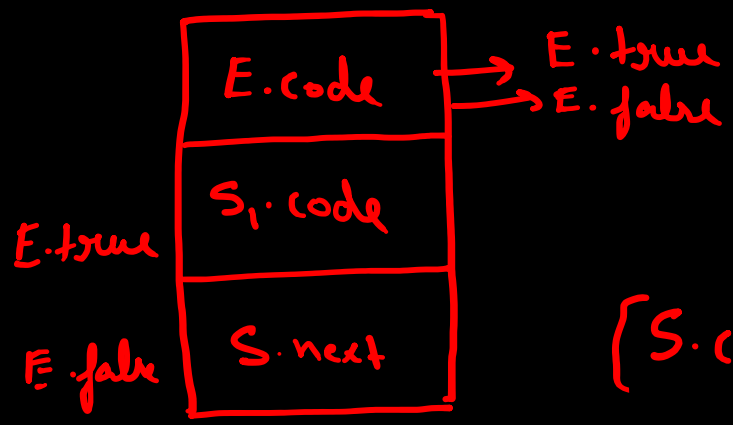
$t_3 = t_1 + t_2$

$x = t_3$

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

SDT for Flow Control

$S \rightarrow$ if E then S_1



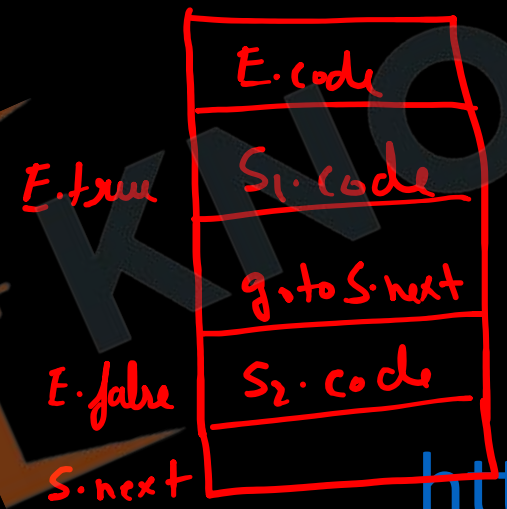
$E.true = newLabel$

$E.false = S.next$

$S_1.next = S.next$

$\{S.code = E.code \,||\, gen(E.true':') \,||\, S_1.code\}$

$S \rightarrow$ if E then S_1 else S_2



$E.true = newlabel$

$E.false = new\ label$

$S_1.next = S.next$

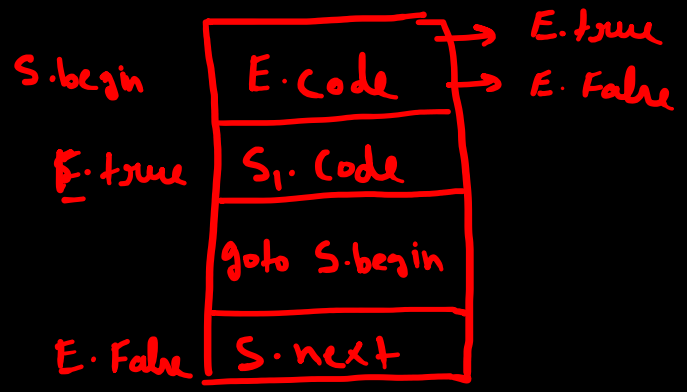
$S_2.next = S.next$

$S.code = E.code \,||\, gen(E.true':') \,||\, S_1.code \,||\, gen(S_2.next) \,||\, gen(E.false':') \,||\, S_2.code$

KNOWLEDGE GATE
http://www.knowledgegate.in/gate

SDT for Flow Control

$S \rightarrow$ while E S_1



S.begin = newlabel()

E.true = newlabel

E.False = S.next

S_1.next = S.begin

S.code = gen(S.begin) || E.code || gen(E.true ':') || S_1.code || gen('goto' S.begin)

KNOWLEDGEGATE

Production rule	Semantic action

Switch E
{
case v_1 : s_1
case v_2 : s_2 ...
 	
case v_{n-1} : s_{n-1}
default : s_n
}
	Evaluate E into t such that
	t = E goto check
	L_1 : code for s_1 goto last
	
	L_2 :code for s_2 goto last
	
	L_n :code for s_n goto last
	
	check:if t=v_1 goto L_1
	if t = v_2 goto L_2
	...
	if t = v_{n-1} goto L_{n-1}
	goto L_n
	last

```
switch(ch)
{
    case 1 : c = a + b;
    break;
    case 2 : c = a – b;
    break;
}
```

if *ch* = 1 goto L_1
if *ch* = 2 goto L_2
L_1: t_1:=*a*+*b*
c := t_1
goto last
L_2 : t_2 := *a* – *b*
c := t_2
goto last

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Backpatching

- Backpatching is an essential technique used during the generation of three-address code in compiler design, especially in single-pass compilers.

- **Backpatching**: It's a method used in single-pass generation of three-address code where the address for jump statements is temporarily left unspecified due to unknown labels; backpatching later fills these addresses with correct values once the target labels are determined during the code generation process.

KNOWLEDGEGATE

if(a, b) then t=1
else t = 0

i) if (a<b) goto (i+3)
i+1) t=0
i+2) goto (i+4)
i+3) t=1
i+4) exit

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

while(C) do S

i) if (E) goto i+2
i+1) goto i+4
i+2) S
i+3) goto i
i+4) exit

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

```
for(1=0; i<10 ; i++)
    s
```

i) i = 0
i+1) if(i<10) goto i+3
i+2) goto i+6
i+3) S
i+4) i = i + 1
i+5) goto i+1
i+6) exit

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

A<B OR C<D AND P<Q

1. if A<B goto ____
2. goto ____
3. if C<D goto ____
4. goto ____
5. if P<Q goto ____
6. goto ____
7. next statement

KNOWLEDGEGATE

Chapter-4

(SYMBOL TABLES): Data structure for symbols tables, representing scope information. Run-Time Administration: Implementation of simple stack allocation scheme, storage allocation in block structured language. Error Detection & Recovery: Lexical Phase errors, syntactic phase errors semantic errors.

KNOWLEDGE GATE

- **Definition**:- The symbol table is a data structure used by a compiler to store information about the source program's variables, functions, constants, user-defined types, and other identifiers.
- **Need**:- It helps the compiler track the scope, life, and attributes (like type, size, value) of each identifier. It is essential for semantic analysis, type checking, and code generation.

KNOWLEDGEGATE

- The information is collected by the analysis phases of the compiler and used by the synthesis phases of the compiler.
  - Lexical Analysis:- Creates new table entries
  - Syntax Analysis:- add information about attributes types, scope and use in the table.
  - Semantic Analysis:- to check expression are semantically correct and type checking.
  - Intermediate Code Generation:-symbol table helps in adding temporary variable information in code.
  - Code Optimization:- use symbol table in machine dependent optimization.
  - Code Generation:- uses address information of identifier present in the table for code generation.

KNOWLEDGEGATE

- **Information used by compiler from symbol table**
  - Data type and name.
  - Declaring procedure.
  - Pointer to structure table or record.
  - Parameter passing by value or by reference.
  - No and types of argument passed to function.
  - Base address.

KNOWLEDGEGATE

A symbol table should possess these essential functions:

- **Lookup**: This function checks if a specific name exists in the table.
- **Insert**: This allows the addition of a new name (or a new entry) into the table.
- **Access**: It enables retrieval of information associated with a specific name.
- **Modify**: This function is used to update or add additional information about an already existing name.
- **Delete**: This capability is for removing a name or a set of names from the table.

KNOWLEDGEGATE

Important symbol table requirements

- **Adaptive Structure**: Entries must be comprehensive, reflecting each identifier's specific use.
- **Quick Lookup/Search**: High-speed search functionality is essential, dependent on the symbol table's design.
- **Space-Efficient**: The table should dynamically adjust in size for optimal space use.
- **Language Feature Accommodation**: It needs to support language-specific aspects like scoping and implicit declarations.

KNOWLEDGEGATE

Different data structures used in implementing a symbol table

- **Unordered List**:
  - Easy to implement using arrays or linked lists.
  - Linked lists allow dynamic growth, avoiding fixed size constraints.
  - Quick insertion $O(1)$ time, but slower lookup $O(n)$ for larger tables.

Student Name	Age
John	20
Maria	19
Alex	21
Sarah	18
Brian	22

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Different data structures used in implementing a symbol table

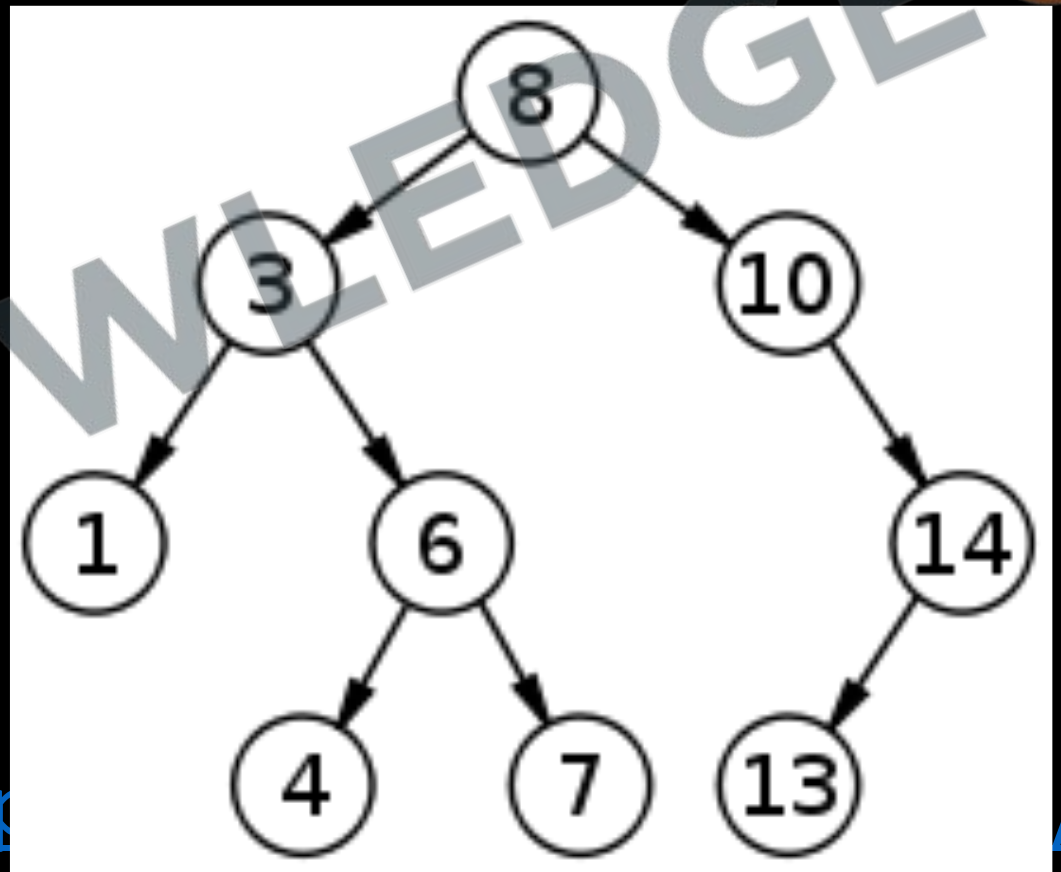
- **Ordered List**:
  - Binary search on a sorted array enables faster search $O(\log_2 n)$.
  - Insertion is slower $O(n)$ due to sorting.
  - Beneficial for fixed sets of names, like reserved words.

Variable Name	Data Type
age	int
height	float
name	string
score	double
width	float

KNOWLEDGEGATE

Different data structures used in implementing a symbol table

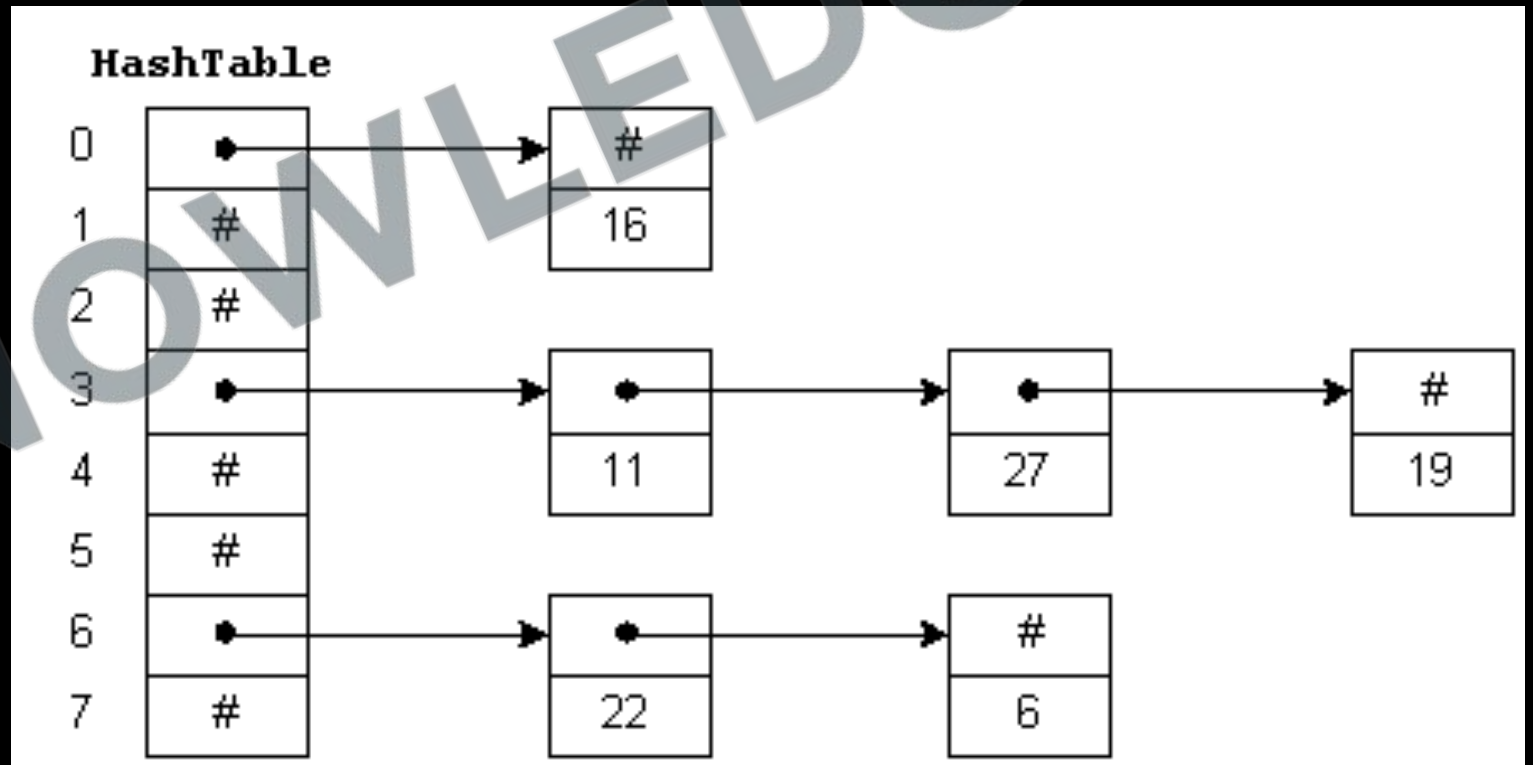
- **Search Tree**:
  - Offers logarithmic time for operations and lookup.
  - Balancing is achieved through AVL or Red-black tree algorithms.



KNOWLEDGEGATE

Different data structures used in implementing a symbol table

- **Hash Tables and Hash Functions:**
  - Hash tables map elements to a fixed range of values (hash values) using hash functions.
  - They minimize element movement within the symbol table.
  - Hash functions ensure a uniform distribution of names.



KNOWLEDGEGATE

Symbol table and its entries

- **Variables**:
  - Identifiers with changeable values during and between program executions.
  - Represent memory location contents.
  - Symbol table tracks their names and runtime storage allocation.
- **Constants**:
  - Identifiers for immutable values.
  - Runtime storage allocation is unnecessary.
  - Compiler embeds them directly into the code during compilation.
- **User-Defined Types**:
  - Combinations of existing types defined by the user.
  - Accessed by name, referring to a specific type structure.
- **Classes**:
  - Abstract data types offering controlled access and language-level polymorphism.
  - Include information about constructors, destructors, and virtual function tables.
- **Records**:
  - Collections of named, possibly heterogeneous members.
  - Symbol table likely records each member within a record.

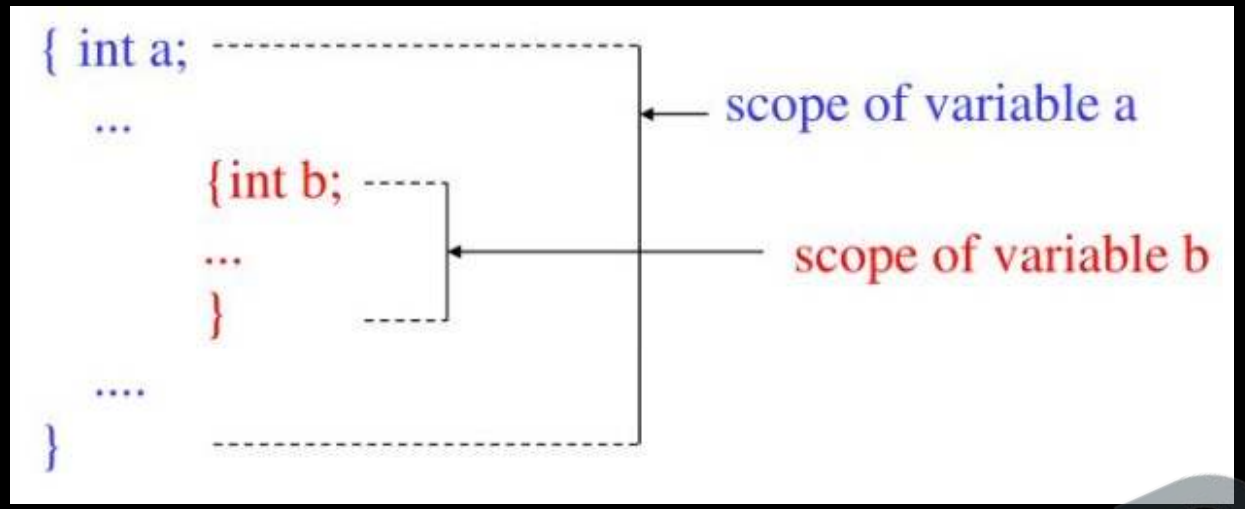
KNOWLEDGEGATE

Representing Scope Information

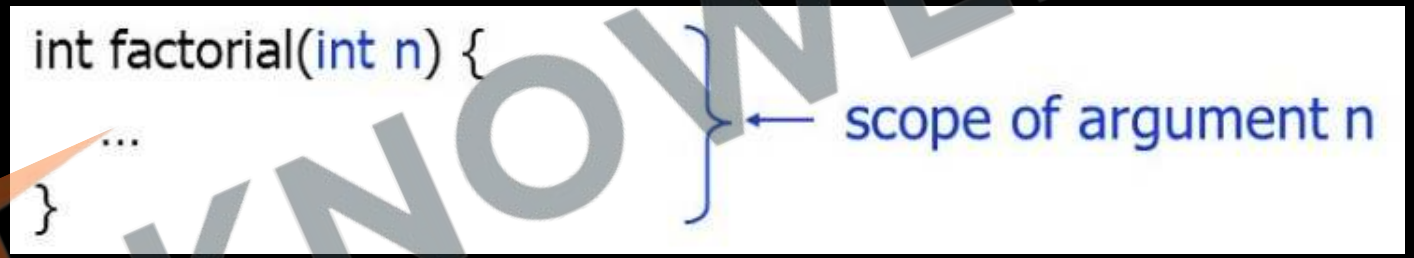
- **Scope of Identifiers**: Scope defines where in a program an identifier is valid and accessible.
- **Language-Dependent Scope**: Different programming languages have varying scopes. For instance, in FORTRAN, a name's scope is limited to a single subroutine, while in ALGOL, it's confined to the section or procedure where it's declared.
- **Multiple Declarations**: An identifier can be declared multiple times in different scopes as distinct entities, each with unique attributes and storage locations.
- **Symbol Table's Role**: The symbol table maintains the uniqueness of each identifier's declarations.
- **Unique Identification**: Each program element is assigned a unique number, which helps in differentiating local data in different scopes.
- **Scope Determination**: Semantic rules from the program's structure are utilized to identify the active scope, especially in subprograms.
- **Scope-Related Semantic Rules**: a. An identifier's usage is confined to its defined scope. b. Identifiers with the same name and type cannot coexist within the same lexical scope.

KNOWLEDGE GATE

- Scope of variables in statement blocks



- Scope of formal arguments of functions



KNOWLEDGEGATE

Scoping in Programming Languages

- Scoping refers to the rules that govern the visibility and lifespan of variables and functions within different parts of a program. It defines the context in which an identifier, such as a variable or function name, is valid and accessible.

KNOWLEDGEGATE

Aspect	Lexical Scoping	Dynamic Scoping
Determination Time	At compile time.	At runtime.
Scope Basis	Location in source code.	Calling sequence of functions.
Predictability	High; scope is clear from code structure.	Low; depends on program's execution path.
Variable Accessibility	Determined by code blocks and functions.	Influenced by function call order.
Common Usage	Preferred in most modern languages.	Less common; found in older languages.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Run-Time Administration: Implementation of simple stack allocation scheme

- The activation record is crucial for handling data necessary for a procedure's single execution. When a procedure is called, this record is pushed onto the stack, and it's removed once control returns to the calling function.

Return value
Actual parameters
Control link
Access link
Saved machine status
Local data
Temporaries

KNOWLEDGEGATE

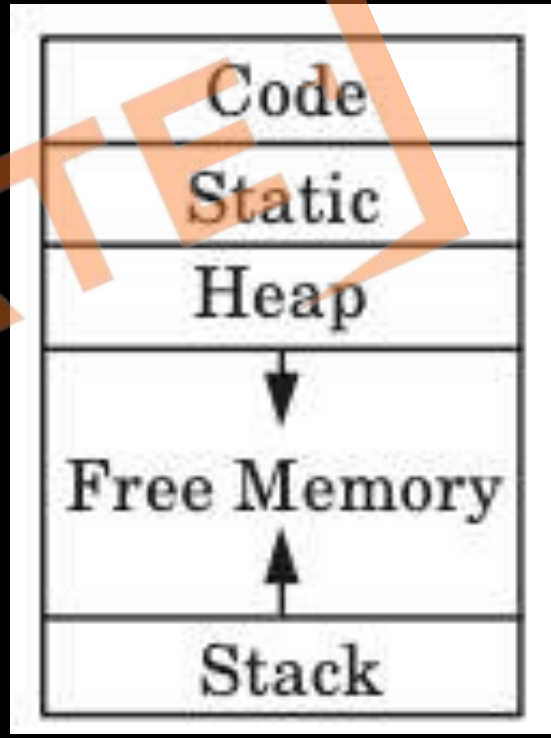
Activation record fields include:

- **Return Value**: Allows the called procedure to return a value to the caller.
- **Actual Parameters**: Used by the caller to provide parameters to the called procedure.
- **Control Link**: Connects to the caller's activation record.
- **Access Link**: References non-local data in other activation records.
- **Saved Machine Status**: Preserves machine state prior to the procedure call.
- **Local Data**: Contains data specific to the procedure's execution.
- **Temporaries**: Stores interim values during expression evaluation.

KNOWLEDGEGATE

Overview of Memory Allocation Methods in Compilation

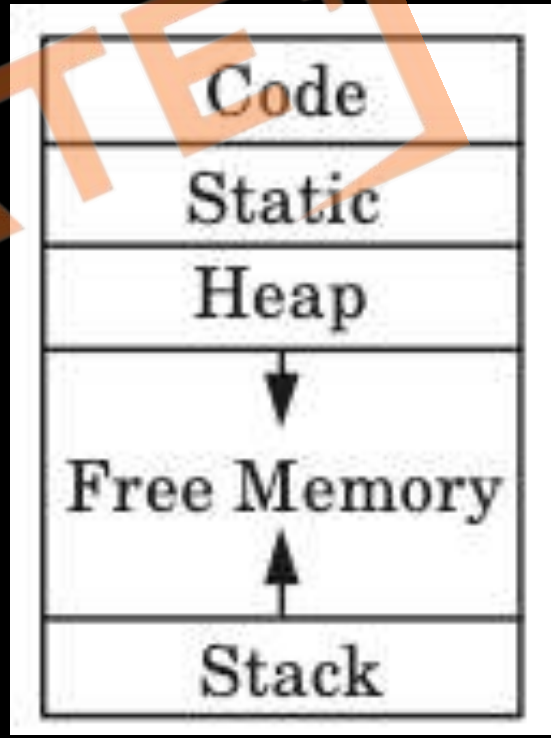
- **Code Storage**:
  - Contains fixed-size, unchanging executable target code during compilation.
- **Static Allocation**:
  - Allocates storage for all data objects at compile time.
  - Object sizes are known at compile time.
  - Object names are bound to storage locations during compilation.
  - The compiler calculates the required storage for each object, simplifying address determination in the activation record.
  - Compiler sets up addresses for the target code to access data at compile time.



KNOWLEDGEGATE

Overview of Memory Allocation Methods in Compilation

- **Heap Allocation Methods**:
  - **Garbage Collection**:
    - Handles objects that persist after losing all access paths.
    - Reclaims object space for reuse.
    - Garbage objects are identified by a 'garbage collection bit' and returned to free space.
  - **Reference Counting**:
    - Reclaims heap storage elements as soon as they become inaccessible.
    - Each heap cell has a counter tracking the number of references to it.
    - The counter is adjusted as references are made or removed.

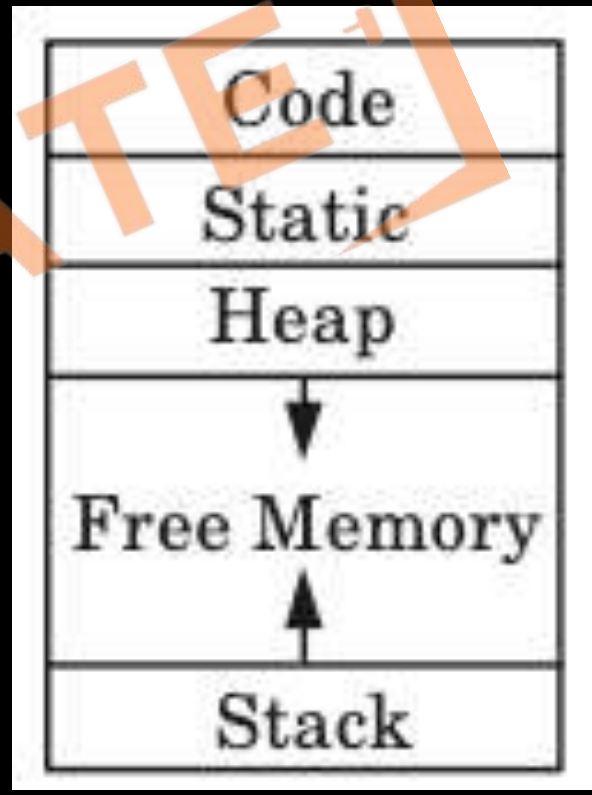


KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Overview of Memory Allocation Methods in Compilation

- **Stack Allocation**:
  - Manages data structures known as activation records.
  - Activation records are pushed onto the stack at call time and popped when the call ends.
  - Local variables for each procedure call are stored in the respective activation record, ensuring fresh storage for each call.
  - Local values are removed when the procedure call completes.



KNOWLEDGEGATE

Overview of Error Recovery in Compilers

Error Recovery Significance:

- Essential for compilers to process and execute programs even with errors.

Error Message Characteristics:

- Messages should relate to the original source code, not its internal representation.
- Simplicity is key in error messaging.
- Precision in pinpointing and fixing errors is crucial.
- Avoid repetition of the same error message.

KNOWLEDGEGATE

Objectives of Error Handling:

- Identify errors and provide clear, useful diagnostics.
- Rapid recovery to identify further errors in the code.
- Ensuring error handling doesn't substantially delay the compilation of correct programs.

KNOWLEDGEGATE

Lexical Phase Errors in Compiling

- **Definition of Lexical Phase Error**:
  - Occurs when a sequence of characters doesn't form a recognizable token during the source program scanning, preventing valid token generation.
- **Common Causes of Lexical Errors**:
  - Adding an unnecessary character.
  - Omitting a required character.
  - Substituting a character incorrectly.
  - Swapping two characters.
- **Examples of Lexical Errors**:
  - In Fortran, identifiers exceeding 7 characters are considered lexical errors.
  - The presence of characters like ~, &, and @ in a Pascal program constitutes a lexical error.

KNOWLEDGEGATE

Syntactic Phase Errors (Syntax Errors)

- **Overview of Syntax Errors**:
  - Syntax errors occur due to coding mistakes made by programmers.

- **Common Sources of Syntax Errors**:
  - Omitting semicolons.
  - Imbalanced parentheses and incorrect punctuation usage.

- **Example**:
  - Consider the code snippet: int x; int y //Syntax error
  - The error arises from the missing semicolon after int y.

KNOWLEDGEGATE

Semantic Phase Errors

- **Definition of Semantic Errors**:
  - Semantic errors relate to incorrect use of program statements, impacting the program's meaning or logic.
- **Typical Reasons for Semantic Errors**:
  - Using undeclared names.
  - Type mismatches.
  - Inconsistencies between actual and formal arguments in function calls.
- **Example**:
  - In the code scanf(“%f%f”, a, b);, the error lies in not using the addresses of variables a and b. The correct usage should be scanf(“%f%f”, &a, &b); to provide the address locations.

KNOWLEDGEGATE

Logical Errors(Run Time Error) in Programming

- **Nature of Logical Errors**:
  - Logical errors are mistakes in a program's logic that the compiler does not catch.
  - These errors occur in programs that are syntactically correct but do not function as intended.
- **Example of a Logical Error**:
  - Consider the code snippet:

```
x = 4;
y = 5;
average = x + y / 2;
```

KNOWLEDGEGATE

Basics of Panic Mode Recovery

- A straightforward and commonly used method in various parsing techniques.
- Upon detecting an error, the parser discards input symbols until it encounters a synchronizing token from a predefined set.

- **Characteristics**:
  - Panic mode may bypass large sections of input without further error checking.
  - This approach ensures that the parser does not enter an infinite loop.
  - a = b + c;
  - d = e + f;
  - In panic mode, the parser might skip over the entire line a = b + c; without identifying specific errors in it.

KNOWLEDGEGATE

Phrase-Level Recovery

- This method involves making localized corrections to the input when an error is detected by the parser.
- It involves substituting a part of the input with a string that allows the parser to proceed.
- **Correction Mechanism**:
  - Common corrections include replacing a comma with a semicolon, removing an unnecessary semicolon, or inserting a missing one.
  - while(x>0)
    - y=a+b;
  - Phrase-level recovery might correct this by adding 'do' to form while(x>0) do y=a+b;, enabling the parsing process to continue smoothly.

KNOWLEDGEGATE

Error Production

- **Overview of Error Production**:
  - This technique allows a parser to generate relevant error messages while continuing the parsing process.
- **Functionality**:
  - When the parser encounters an incorrect production, it issues an error message and then resumes parsing.
  - E → + E | - E | * A | / A
  - A → E
  - If the parser comes across the production '* A' and deems it incorrect, it can alert the user with a message, perhaps querying whether '*' is intended as a unary operator, before continuing with the parsing.

KNOWLEDGE GATE

Global Correction in Parsing

- **Nature of Global Correction**:
  - Primarily a theoretical concept in the realm of parsing.

- **Implications on Parsing Process**:
  - Utilizing global correction significantly increases both the time and space resources required during the parsing stage.

KNOWLEDGEGATE

Error Recovery in Operator Precedence Parsing

- **Detection Points in Operator Precedence Parsing**:
  - Errors are identifiable at two key stages:
    - When there's no precedence relation between the terminal at the top of the stack and the current input symbol.
    - When a handle is identified, but no corresponding production exists.

KNOWLEDGEGATE

Error Recovery in Operator Precedence Parsing

- **Error Types and Diagnostics**: The parser can detect several types of errors:
  - **Errors Detected During Reduction**:
    - Missing operand.
    - Missing operator.
    - Absence of expression within parentheses.
  - **Errors Detected During Shift/Reduce Actions**:
    - Missing operand.
    - Unbalanced or missing right parenthesis.
    - Missing operators.

KNOWLEDGEGATE

Chapter-5

(CODE GENERATION): Design Issues, the Target Language. Addresses in the Target Code, Basic Blocks and Flow Graphs, Optimization of Basic Blocks, Code Generator. Code optimization: Machine-Independent Optimizations, Loop optimization, DAG representation of basic blocks, value numbers and algebraic laws, Global Data-Flow analysis.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Code Generation

- Code generation is the process of converting the intermediate representation (IR) of source code into a target code(assembly level). Which is also optimized.
- It involves translating the syntax and semantics of the high-level language into assembly level code, typically after the source code has passed through lexical analysis, syntax analysis, semantic analysis, and intermediate code generation.

```
int main() {
    int a = 5;
    int b = 3;
    int sum = a + b;
}
```

```
t1 = 5
t2 = 3
t3 = t1 + t2
```

```
MOV R1, 5
MOV R2, 3
ADD R3, R1, R2;
```

KNOWLEDGEGATE

- **Code Generator Input**:
  - Uses the source program's intermediate representation (IR) and symbol table data.
- **Intermediate Representation (IR)**:
  - Consists of three-address and graphical forms.
- **Target Program**:
  - Influenced by the target machine's instruction set architecture (ISA).
  - Common ISAs: RISC, CISC, and stack-based.
- **Instruction Selection**:
  - Converts IR to executable code for the target machine.
  - High-level IR may use code templates for translation.
- **Register Allocation**:
  - Critical to decide which values to store in registers.
  - Non-registered values stay in memory.
  - Register use leads to shorter, faster instructions.
  - Involves two steps: i. Choosing variables for register storage. ii. Assigning specific registers to these variables.
- **Evaluation Order**:
  - The sequence of computations impacts code efficiency.
  - Some sequences minimize register usage for intermediate results.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Optimization**: During code generation, various optimization techniques are applied to improve efficiency and performance. This could include optimizing for speed, memory usage, or even power consumption.

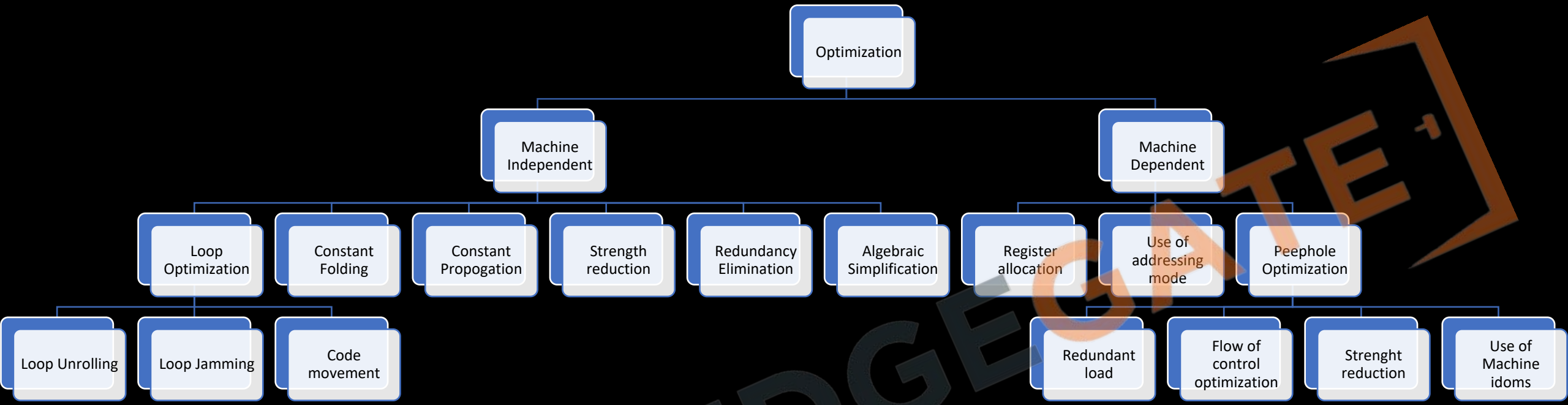
- **Target Platform**: The generated code is specific to the target platform's architecture, such as x86, ARM, etc. This means the same high-level code will have different generated machine code for different platforms.

KNOWLEDGEGATE

Optimization

- Process of reducing the execution time of a code without effecting the outcome of the source program, is called as optimization.

KNOWLEDGEGATE



KNOWLEDGEGATE

Aspect	Machine-Independent Optimization	Machine-Dependent Optimization
Definition	Optimizations that are not specific to any processor or machine architecture.	Optimizations tailored to the specifics of a particular machine or processor architecture.
Focus	Concentrates on improving the logic and efficiency of the code at a high level, often in the intermediate code.	Focuses on enhancing performance by taking advantage of the unique features and instructions of the target machine.
Examples	Common subexpression elimination, Code motion, Dead code elimination, Loop unrolling, Loop fusion	Instruction scheduling, Register allocation, Pipeline optimization, Use of machine-specific instructions, Cache optimization
Portability	Highly portable across different architectures as they do not rely on machine-specific features.	Not portable; optimizations must be re-applied or altered for different architectures.
Stage of Application	Applied before the code is mapped to the target machine's instruction set, often during the intermediate code generation phase.	Applied during or after the generation of the target machine code, tailoring the optimizations to the specifics of the machine's hardware.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Algorithm to partition a sequence of three address statements into basic blocks

- Loop Optimization
  - To apply loop optimization, we must first detect loops.
  - For detecting loops, we use control flow analysis (CFA) using program flow graph (PFG)
  - To find PFG, we need to find basic blocks.
  - A Basic block is a sequence of 3-adress statements where control enters at the beginning and leaves only at the end without any jumps or halts

KNOWLEDGEGATE

- The block can be identified with the help of leader
  - Finding the leader
  - Finding the bocks
  - Construct PFG

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- In order to find the basic blocks, we need to finds the leader in the program then a basic block will start from one leader to the next leader but not including next leader.
- identifying leaders in a basic block
  - First statement is a leader
  - Statement that is the target of conditional or unconditional statement is a leader
  - Statement that follow immediately a conditional or unconditional statement is a leader

KNOWLEDGEGATE

```
Fact(x)
{
    int f=1
    for(i=2 ; i<=x ; i++)
        f = f*i;
    return f;
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

1) f=1;
2) i=2
3) if(i>x), goto 9
4) t1=f*i;
5) f=t1;
6) t2=i+1;
7) i=t2;
8) goto(3)
9) goto calling program

KNOWLEDGEGATE

1) f=1;
2) i=2

3) if(i>x), goto 9

4) t1=f*i;
5) f=t1;
6) t2=i+1;
7) i=t2;
8) goto(3)

9) goto calling program

KNOWLEDGEGATE

Q Consider the intermediate code given below: The number of nodes and edges in the control-flow-graph constructed for the above code, respectively, are

1. i = 1
2. j = 1
3. $t_1 = 5 * i$
4. $t_2 = t_1 + j$
5. $t_3 = 4 * t_2$
6. $t_4 = t_3$
7. $a[t_4] = -1$
8. j = j + 1
9. if j <= 5 goto(3)
10. i = i + 1
11. if i < 5 goto(2)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the following sequence of three address codes :

1. Prod:=0
2. *I*:=1
3. *T*1:=4***I*
4. *T*2:=addr(*A*)–4
5. *T*3:=*T*2[*T*1]
6. *T*4:=addr(*B*)–4
7. *T*5:=*T*4[*T*1]
8. *T*6:=*T*3***T*5
9. Prod:=Prod+*T*6
10. *I*=*I*+1
11. If *I* <= 20 goto (3)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Loop Jamming: combining the bodies of two loops, whenever they share the same index and same no of variables

```
for (int i=0; i<=10; i++)
      for (int j=0; j<=10; j++)
            x[i, j]=”TOC”
for (int j=0; j<=10; j++)
            y[i]=”CD”
```

```
for (int i=0; i<=10; i++)
{
      for (int j=0; j<=10; j++)
      {
            x[i, j]=”TOC”
      }
      y[i]=”CD”
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Loop Unrolling: getting the same output with less no of iteration is called loop unrolling

```
int i=1;
while(i<=100)
{
    print(i)
    i++
}
```

```
int i=1;
while(i<=100)
{
    print(i)
    i++
    print(i)
    i++
}
```

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Code movement(Loop invariant computation): removing those code out from the loop which is not related to loop.

```
int i=1;
while(i<=100)
{
    a =b+c
    print(i)
    i++
}
```

KNOWLEDGEGATE

Optimization of basic blocks

- Algebraic Simplification
- Redundant code Elimination / Common subexpression elimination
- Strength reduction
- Constant Propagation
- Constant Folding

KNOWLEDGEGATE

Constant Folding: Replacing the value of expression before compilation is called as constant folding

x = a + b + 2 * 3 + 4

x = a + b + 10

KNOWLEDGEGATE

Constant Propagation: replacing the value of constant before compile time, is called as constant propagation.

pi = 3.1415

x = 360 / pi

x = 360/3.1415

KNOWLEDGEGATE

Strength reduction: replacing the costly operator by cheaper operator, this process is called strength reduction.

y = 2 * x

y = x + x

KNOWLEDGEGATE

Redundant code Elimination / Common subexpression elimination:

Avoiding the evaluation of any expression more than once is redundant code elimination.

```
x = a + b
y = b + a

x = a + b
y = x
```

KNOWLEDGEGATE

Algebraic Simplification: Basic laws of math's which can be solved directly.

a = b*1

a=b

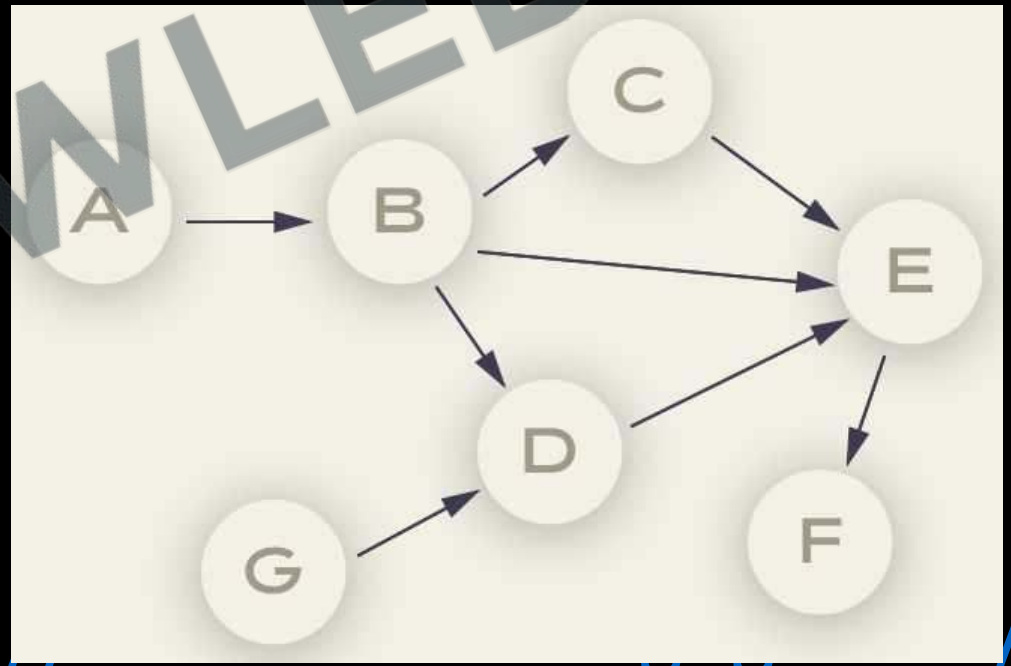
a=b+0

a=b

KNOWLEDGEGATE

Direct Acyclic Graph (DAG)

- A Direct Acyclic Graph is a graph that is directed and contains no cycles. So it is impossible to start at any vertex v and follow a sequence of edges that eventually loops back to v again.
- By representing expressions and operations in a DAG, compilers can easily identify and eliminate redundant calculations, thus optimizing the code.
  - We automatically detect common sub-expressions with the help of DAG algorithm.
  - We can determine which identifiers have their values used in the block.
  - We can determine which statements compute values which could be used outside the block.



KNOWLEDGE GATE

http:/ /gate

((a+a) + (a+a)) + ((a+a) + (a+a))

KNOWLEDGEGATE

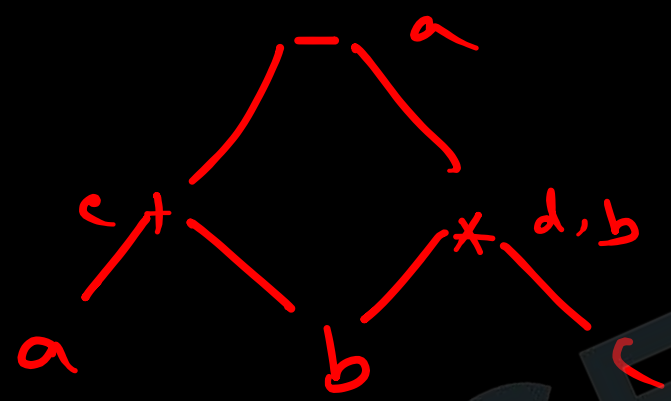
http://www.knowledgegate.in/gate

d = b * c

c = a + b

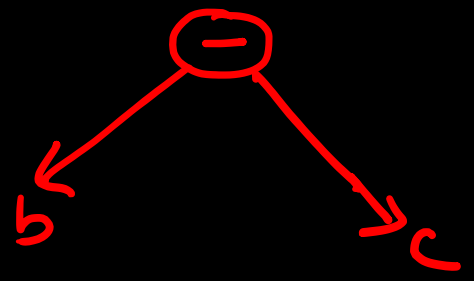
b = b * c

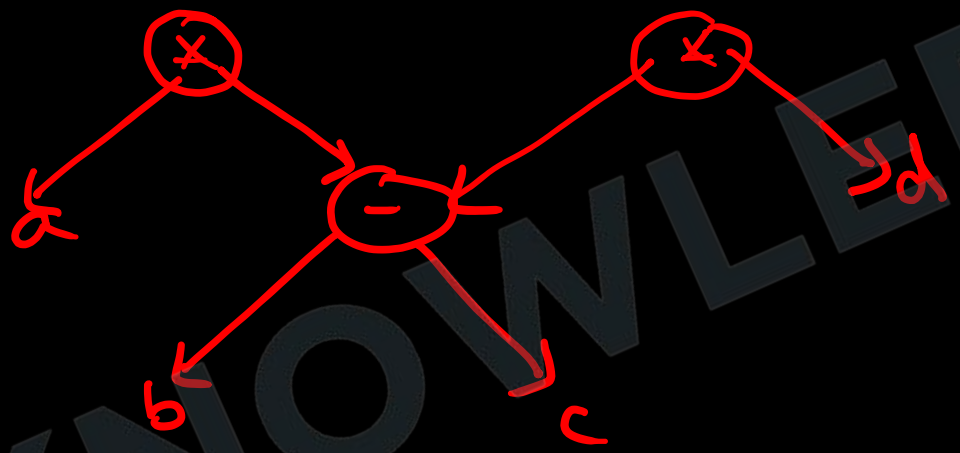
a = c - d

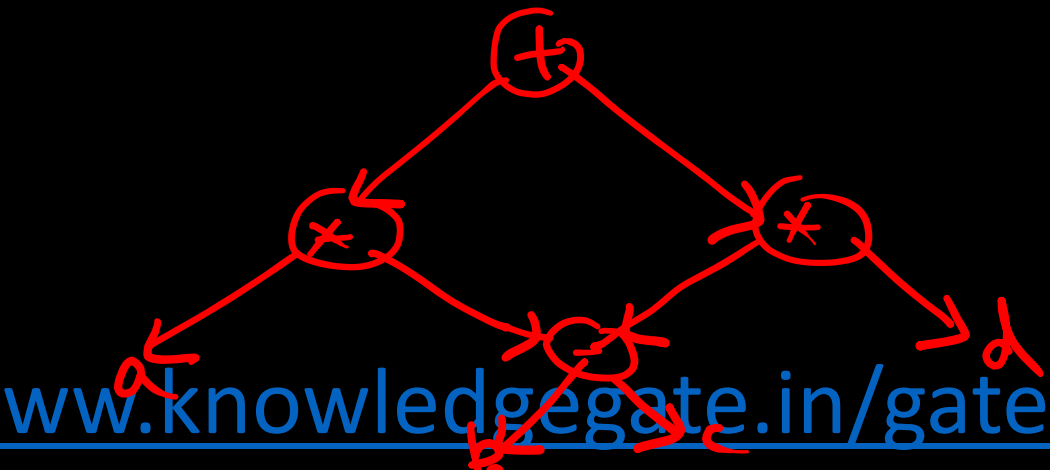


KNOWLEDGEGATE

a * (b − c) + (b − c) * d







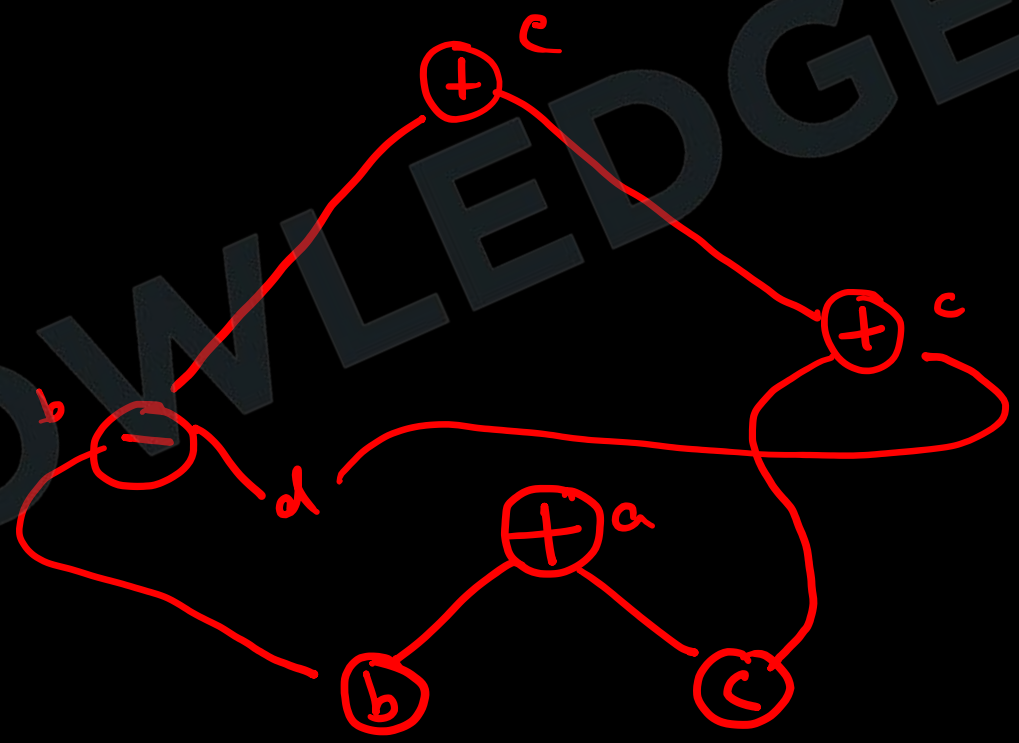
KNOWLEDGEGATE

a = b + c

b = b - d

c = c + d

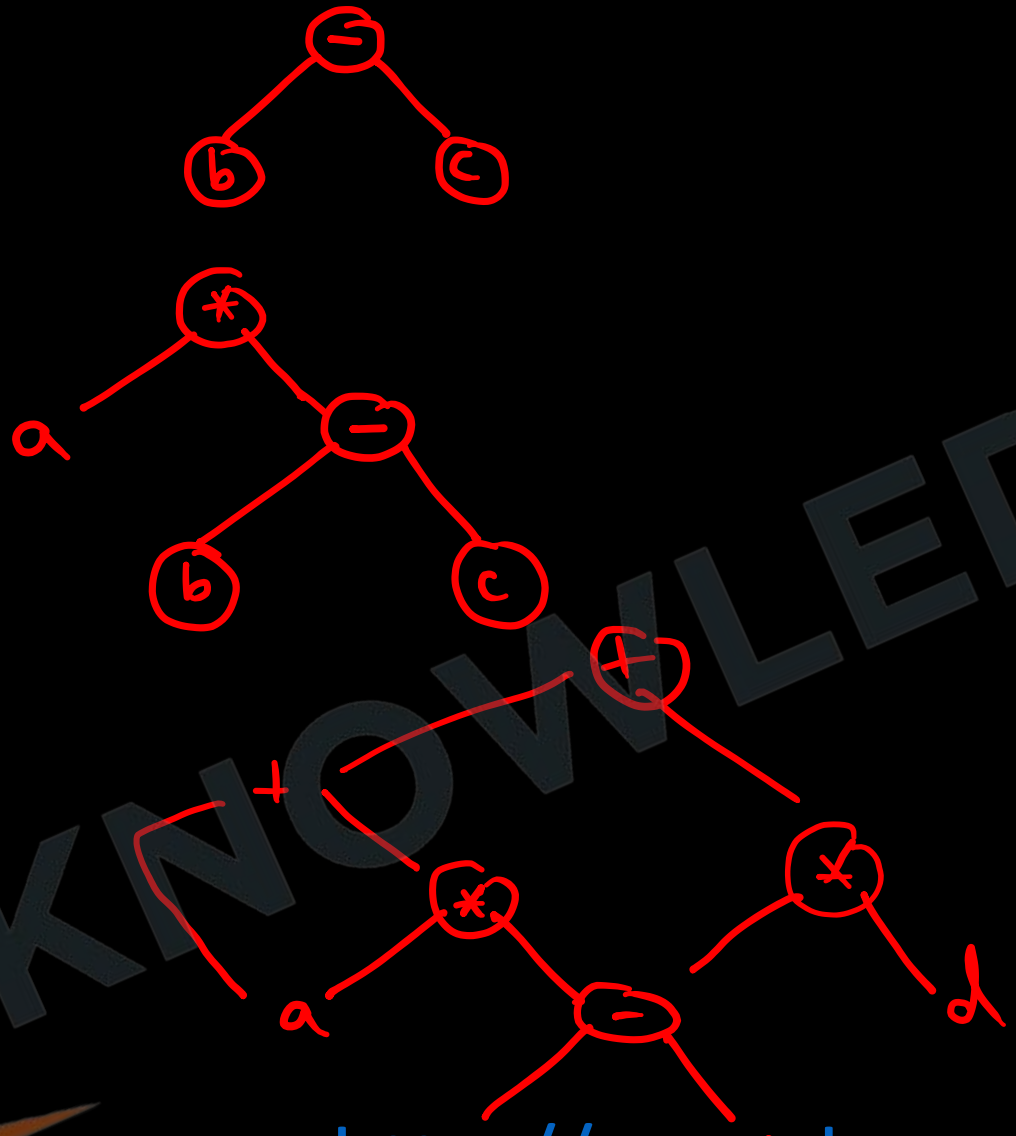
e = b + c



KNOWLEDGEGATE

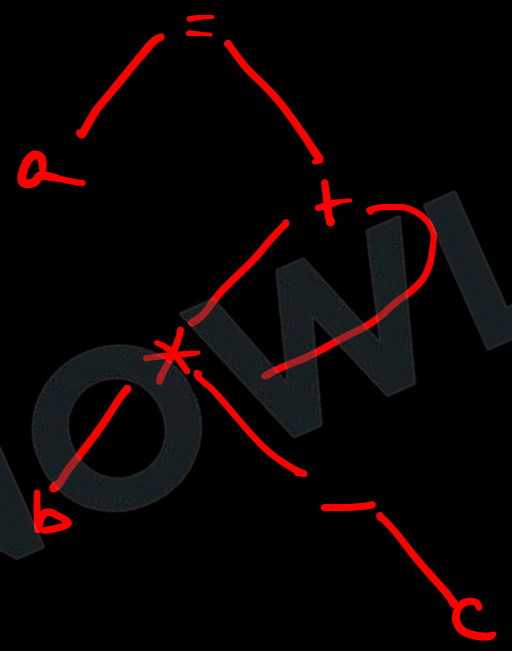
http://www.knowledgegate.in/gate

a + a * (b - c) + (b - c) * d



KNOWLEDGEGATE

a = b * -c + b * -c



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Module

Free Semester One Shot Exam Course

Topic

Software Engineering

Subtopic

SE One Shot

Lesson

Software_Engineering_Notes

SEMESTER EXAM SERIES

SOFTWARE ENGINEERING

IN 6 HOURS

KNOWLEDGE GATE

Unit-1(Introduction)

- Introduction to Software Engineering,
- Software Components,
- Software Characteristics,
- Software Crisis,
- Software Engineering Processes,
- Similarity and Differences from Conventional Engineering Processes, Software Quality Attributes,
- Software Development Life Cycle (SDLC)
- Models:
  - Water Fall Model,
  - Prototype Model,
  - Spiral Model,
  - Evolutionary Development Models,
  - Iterative Enhancement Models.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Unit-2(Software Requirement Specifications (SRS))

- Requirement Engineering Process:
  - Elicitation,
  - Analysis,
  - Documentation,
  - Review and Management of User Needs,
- Feasibility Study,
- Information Modeling,
- Data Flow Diagrams,
- Entity Relationship Diagrams,
- Decision Tables,
- SRS Document,
- IEEE Standards for SRS.
- Software Quality Assurance (SQA): Verification and Validation,
- SQA Plans, Software Quality Frameworks, ISO 9000 Models, SEI-CMM Model

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Unit-3(Software Design)

- **Basic Concept of Software Design**,
  - Architectural Design,
  - Low Level Design: Modularization, Design Structure Charts, Pseudo Codes, Flow Charts,
  - Coupling and Cohesion Measures,
- **Design Strategies**
  - Function Oriented Design,
  - Object Oriented Design,
  - Top-Down and Bottom-Up Design.
- Software Measurement and Metrics:
  - Various Size Oriented Measures: Halestead's Software Science,
  - Function Point (FP) Based Measures,
  - Cyclomatic Complexity Measures: Control Flow Graphs.

KNOWLEDGEGATE

Unit-4(Software Testing)

- Testing Objectives,
  - Unit Testing,
  - Integration Testing,
  - Acceptance Testing,
  - Regression Testing,
- Testing for Functionality and Testing for Performance,
- Top-Down and Bottom-Up
- Testing Strategies:
  - Test Drivers and Test Stubs,
  - Structural Testing (White Box Testing),
  - Functional Testing (Black Box Testing),
  - Test Data Suit Preparation,
  - Alpha and Beta Testing of Products.
- Static Testing Strategies: Formal Technical Reviews (Peer Reviews), Walk Through, Code Inspection, Compliance with Design and Coding Standards.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Unit-5(Software Maintenance)

- Software as an Evolutionary Entity,
- Need for Maintenance,
- Categories of Maintenance: Preventive, Corrective and Perfective Maintenance,
- Cost of Maintenance,
- Software Re-Engineering,
- Reverse Engineering.
- Software Configuration Management Activities,
- Change Control Process,
- Software Version Control,
- Resource Allocation Models,
- Software Risk Analysis and Management.

KNOWLEDGEGATE

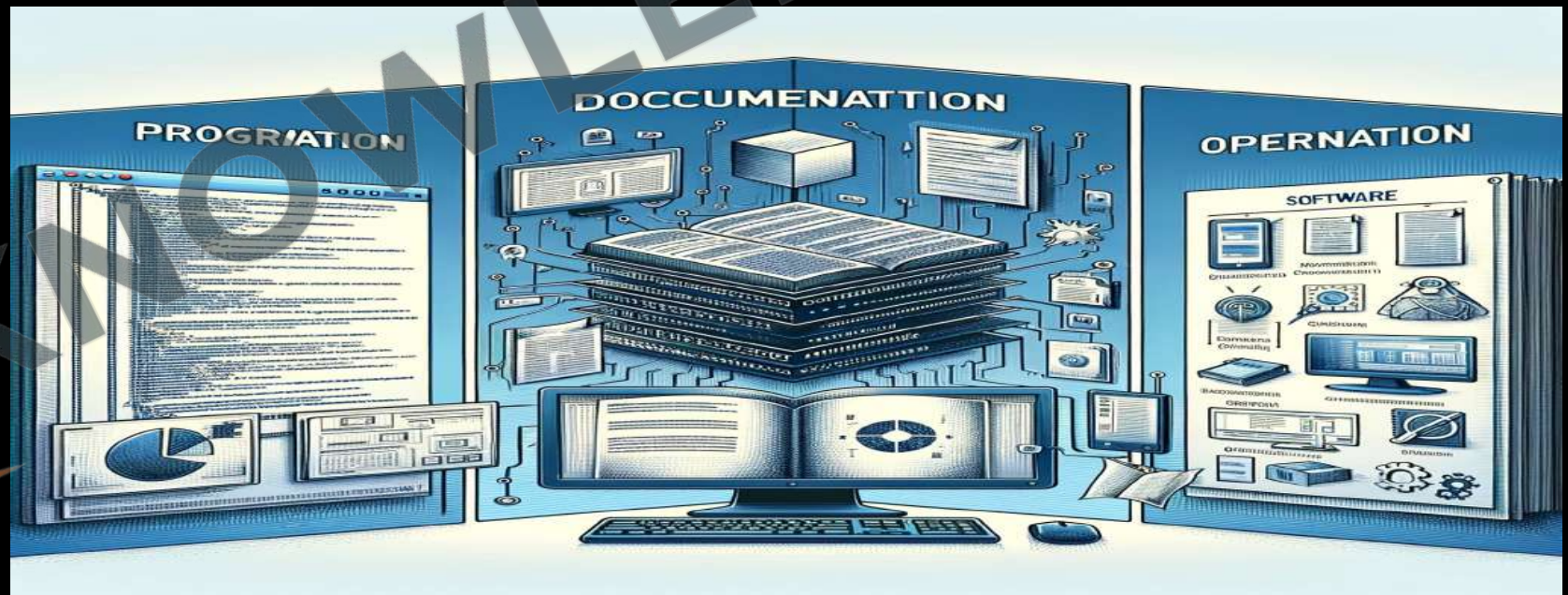
What is Software

- Program: A step by step instructions to perform a specific task on a computer in a programming language is called program, i.e. set of instructions.
- Software: Is a program along with proper documentation (requirement analysis, design, coding, testing) and user manuals which mainly includes installation guide and other manuals.
- Software = Program + documentation

KNOWLEDGEGATE

Software Components

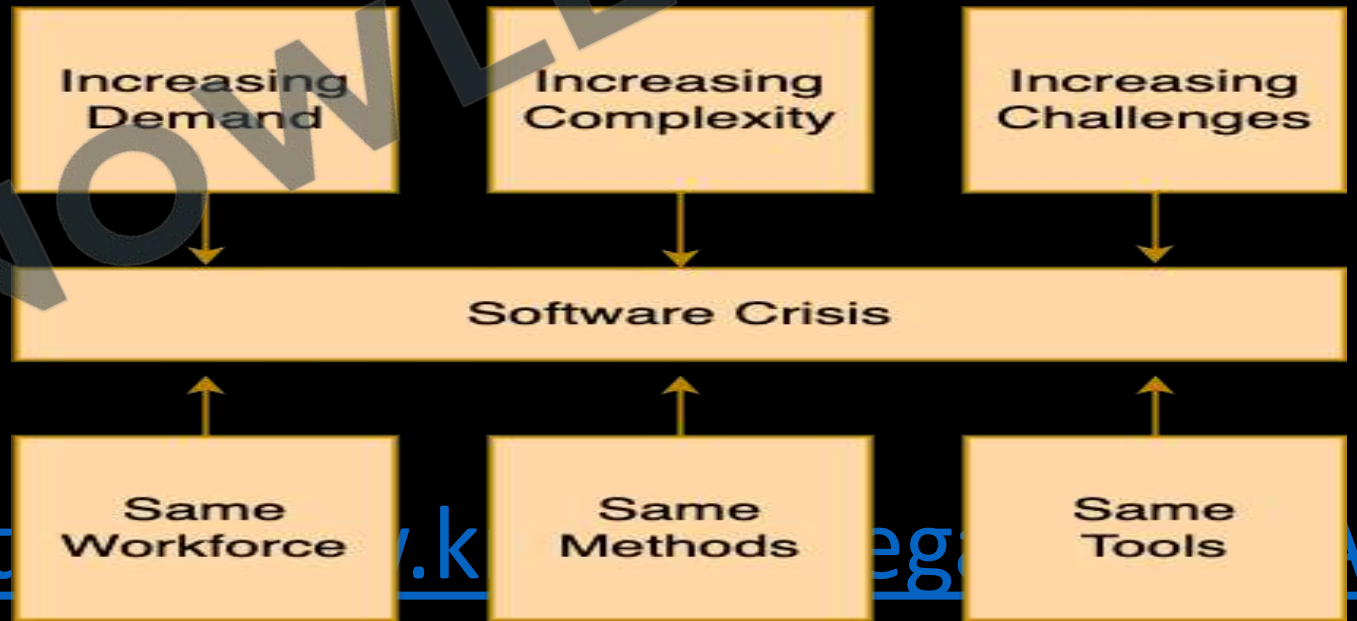
- A program is a subset of software and it becomes software only if documentation and an operating procedure manual are prepared.
  - **Program**
  - **Documents**
    - Software documentation consist all the description, programs, graphics and instructions pertaining to design, coding, testing and preparation of software.
  - **Operating Procedure(User Manual & Operational Manual)**
    - Provides information about what software is how to work with it how to install it on your system and how to control all the activities of the software.



KNOWLEDGEGATE

Software Crisis

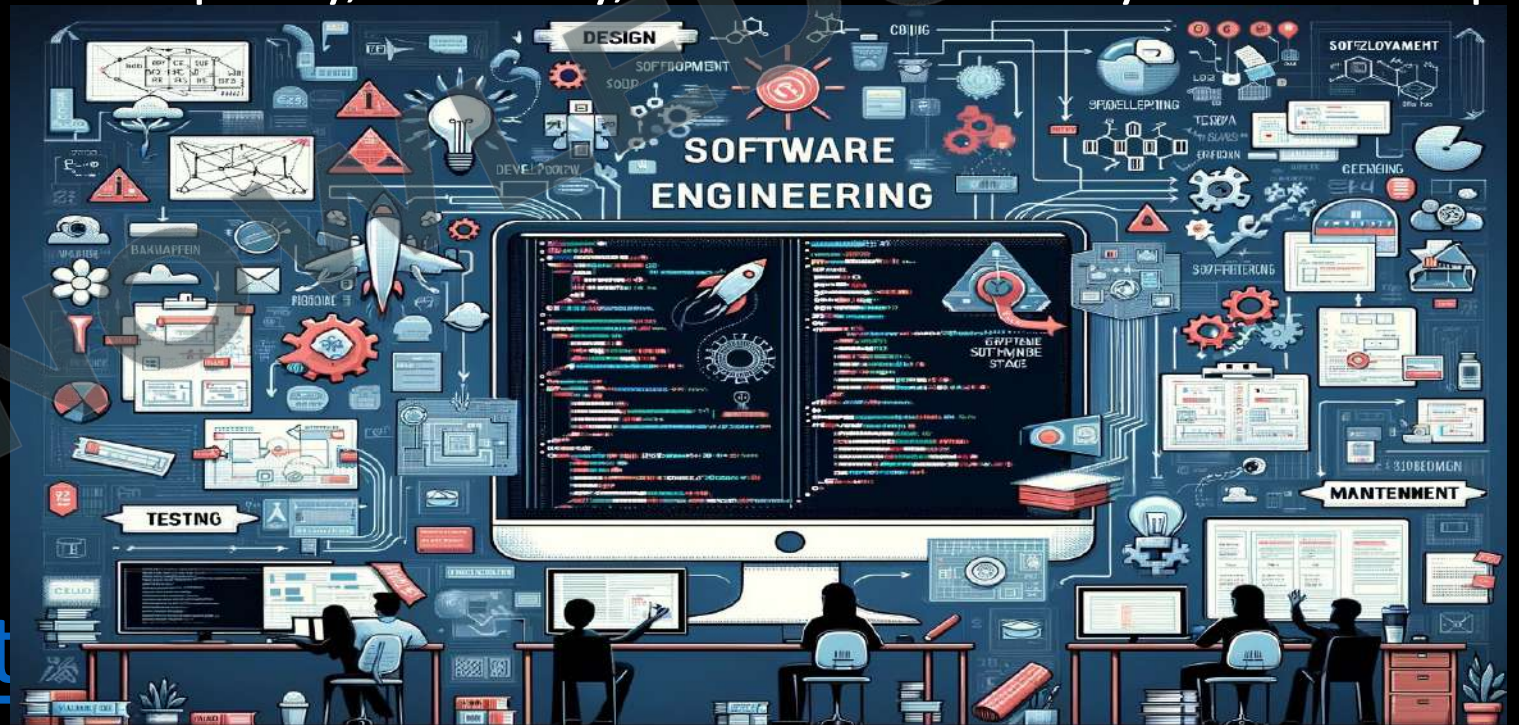
- Problems with software?
  - The major problem with current scenario in Software industry is, software usually overrun their development cost, they exceed their development duration limits, are usually of poor quality.
  - As long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem, and now we have gigantic computers, programming has become an equally gigantic problem.— *Edsger Dijkstra, The Humble Programmer (EWD340), Communications of the ACM*



KNOWLEDGE GATE

What is Software Engineering

- In software industry we understand that there is an urgent need to use a proper strategies, process and development cycles so that we can produce, or we can design quality products that are within budget, with in time and must satisfy the requirement of their users.
- Software engineering is the systematic application of engineering principles and methods to the design, development, testing, and maintenance of software products. It involves the use of various tools, techniques, and methodologies to manage the software development process and ensure the quality, reliability, and maintainability of software products.



KNOWLEDGEGATE

Characteristics of software

1. Software Development vs. Manufacturing:

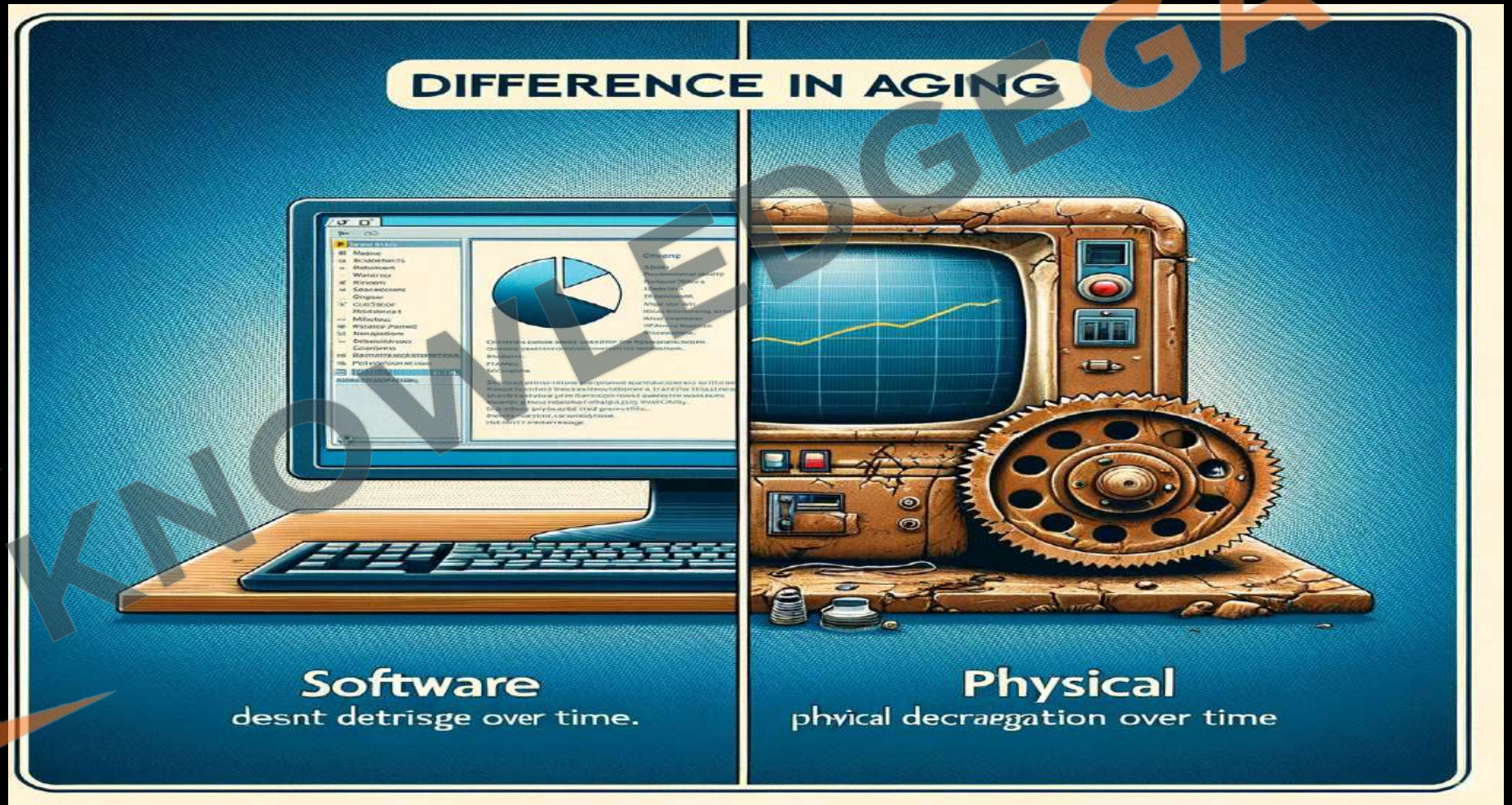
1. Software is crafted through a process of coding and development.
2. Unlike physical products, it isn't produced in factories through a manufacturing process.

KNOWLEDGE GATE

Characteristics of software

2. No Physical Wear and Tear:

1. Software doesn't deteriorate over time like physical objects.
2. Its performance doesn't degrade due to physical use.



KNOWLEDGEGATE

Characteristics of software

3. Custom-Built Nature:

1. Most software is specially designed to meet specific needs.
2. It's often not just a combination of pre-existing components but requires unique coding and design.

KNOWLEDGE GATE

Characteristics of software

4. Intangibility of Software:

1. Software can't be touched or handled physically.
2. It exists as code and functions within computers or digital devices.

KNOWLEDGE GATE

Major Problems in Software Development

- **Inadequate Requirements Gathering**:
  - Ambiguous or incomplete requirements
  - Lack of communication between stakeholders

KNOWLEDGE GATE

Major Problems in Software Development

- **Poor Project Management**:
  - Inadequate planning, monitoring, and control
  - Lack of risk assessment and mitigation
  - Multiplicity of software development life cycle
  - Selection of wrong technology or tool for development

KNOWLEDGE GATE

Major Problems in Software Development

- **Insufficient Time and Budget**:
  - Unrealistic deadlines and resource constraints
  - Inefficient resource allocation and prioritization

KNOWLEDGE GATE

Major Problems in Software Development

- **Lack of Skilled Personnel**:
  - Inadequate expertise in the development team
  - High turnover rates and loss of experienced staff

Major Problems in Software Development

- **Resistance to Change**:
  - Difficulty in adapting to new technologies or processes
  - Reluctance to modify established practices or mindsets
  - Rapid technology advancement

KNOWLEDGEGATE

Similarity and Differences from conventional Engineering Process

- **Nature of the Product**:
  - **Similarity**: Both processes aim to create high-quality, reliable products
  - **Difference**: Conventional engineering focuses on physical systems, while software engineering deals with intangible software systems

KNOWLEDGE GATE

Similarity and Differences from conventional Engineering Process

- **Design Flexibility and Iteration**:
  - **Similarity**: Both processes involve iterative design and prototyping
  - **Difference**: Software engineering allows for greater flexibility and ease of modification due to the non-physical nature of software

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Similarity and Differences from conventional Engineering Process

- **Quality Assurance and Testing**:
  - **Similarity**: Both processes emphasize the importance of testing and quality assurance to ensure product performance and reliability
  - **Difference**: Conventional engineering often involves physical testing of prototypes, while software engineering relies on various types of software testing, such as unit, integration, and system testing

KNOWLEDGE GATE

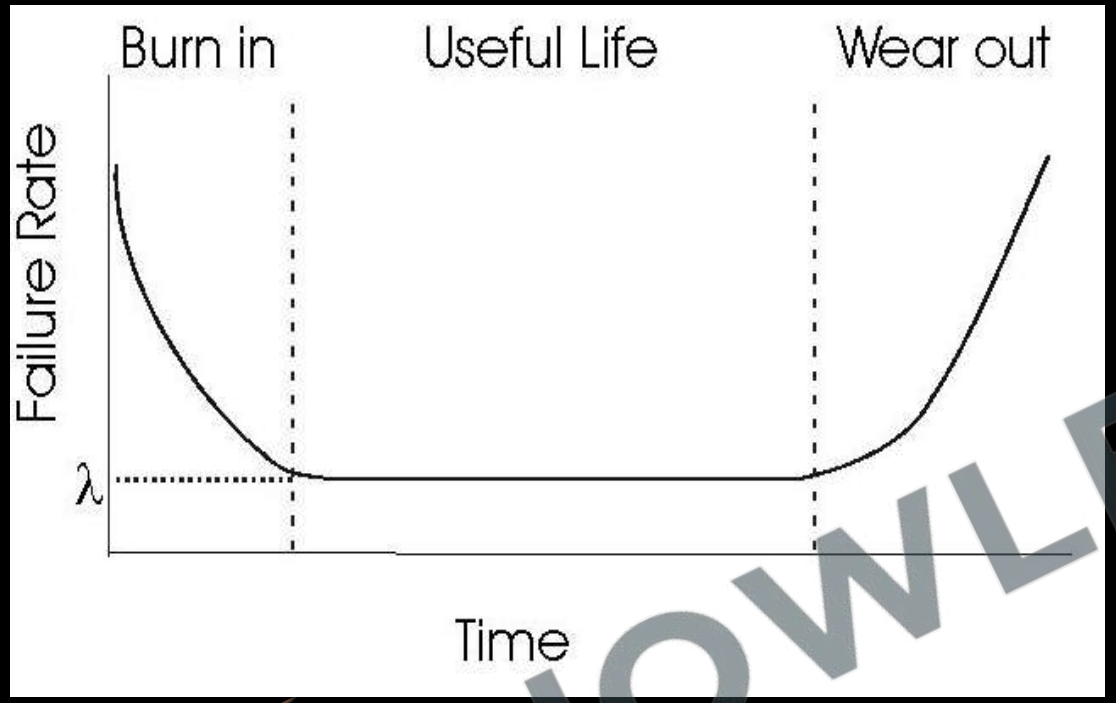
- **Project Management and Collaboration**:
  - **Similarity**: Both processes require effective project management, team collaboration, and communication among stakeholders
  - **Difference**: Software engineering projects may involve distributed teams and rely more heavily on digital communication and collaboration tools

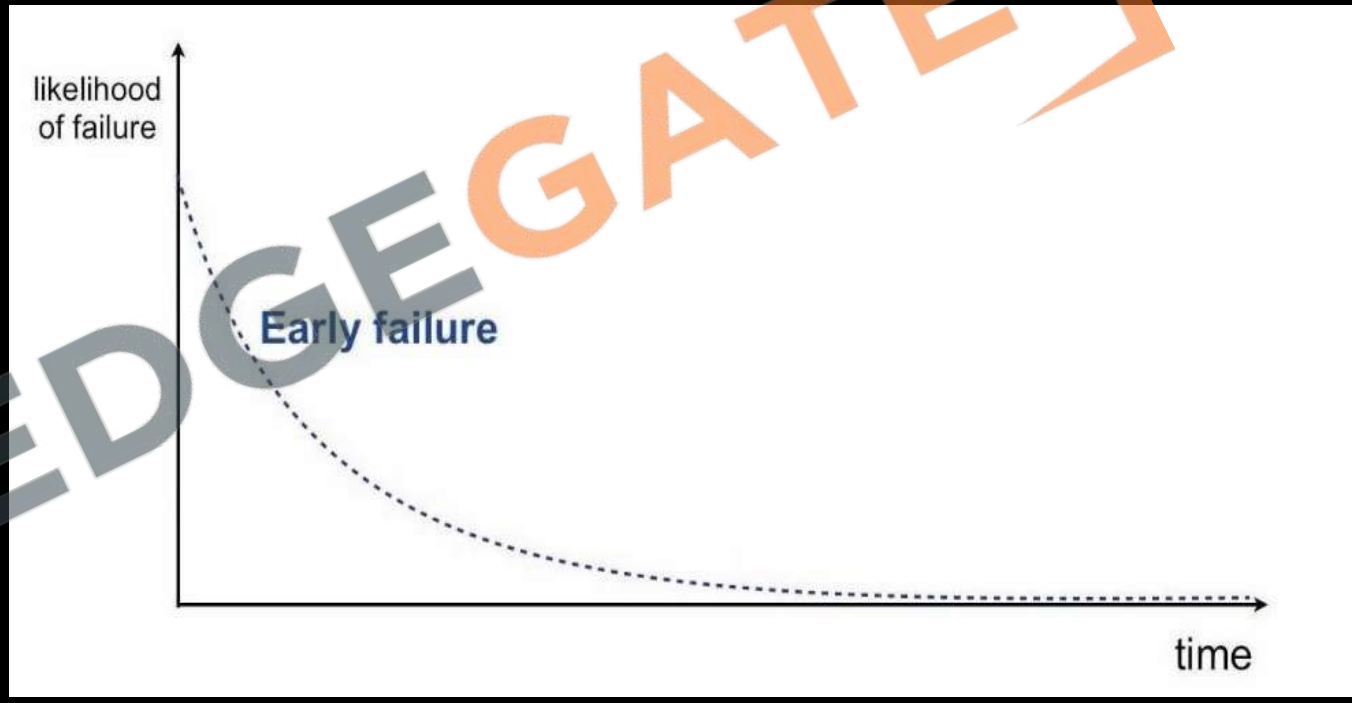
KNOWLEDGE GATE

- **Maintenance and Evolution**:
  - **Similarity**: Both processes involve maintenance and support activities to ensure the ongoing performance and reliability of the product
  - **Difference**: Software engineering typically demands more frequent updates, patches, and evolution due to the rapidly changing nature of technology and user requirements

KNOWLEDGEGATE

- For hardware Same production cost every time, for software production cost only for the first time and then only maintenance cost. This life cycle of the hardware follows the bath tub curve, while in the life cycle of software failure intensity goes down with time





KNOWLEDGEGATE

Software Quality Attributes

KNOWLEDGE

Software Quality Attributes

- **Correctness**: The ability of the software to perform its intended tasks effectively and meet user requirements.
- **Usability**: The ease with which users can learn, operate, and navigate the software.
- **Reliability**: The software's consistency in producing accurate results and maintaining performance over time.
- **Efficiency**: The optimal use of system resources, such as memory and processing power, to achieve desired outcomes.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Maintainability**: The ease of updating, modifying, and fixing the software to accommodate changing requirements or fix issues.
- **Portability**: The ability of the software to operate on different platforms or environments without significant modifications.
- **Scalability**: The software's capacity to handle increased workloads or user demands without compromising performance.
- **Security**: The software's ability to protect against unauthorized access, data breaches, and other potential threats.

KNOWLEDGEGATE

- **Modularity**: The degree to which the software's components are organized into separate, manageable units that can be independently developed or updated.

- **Reusability**: The potential for the software's components to be used in other applications or contexts, reducing development time and costs.

- **Testability**: The ease with which the software can be tested to ensure it meets its requirements and performs as expected.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

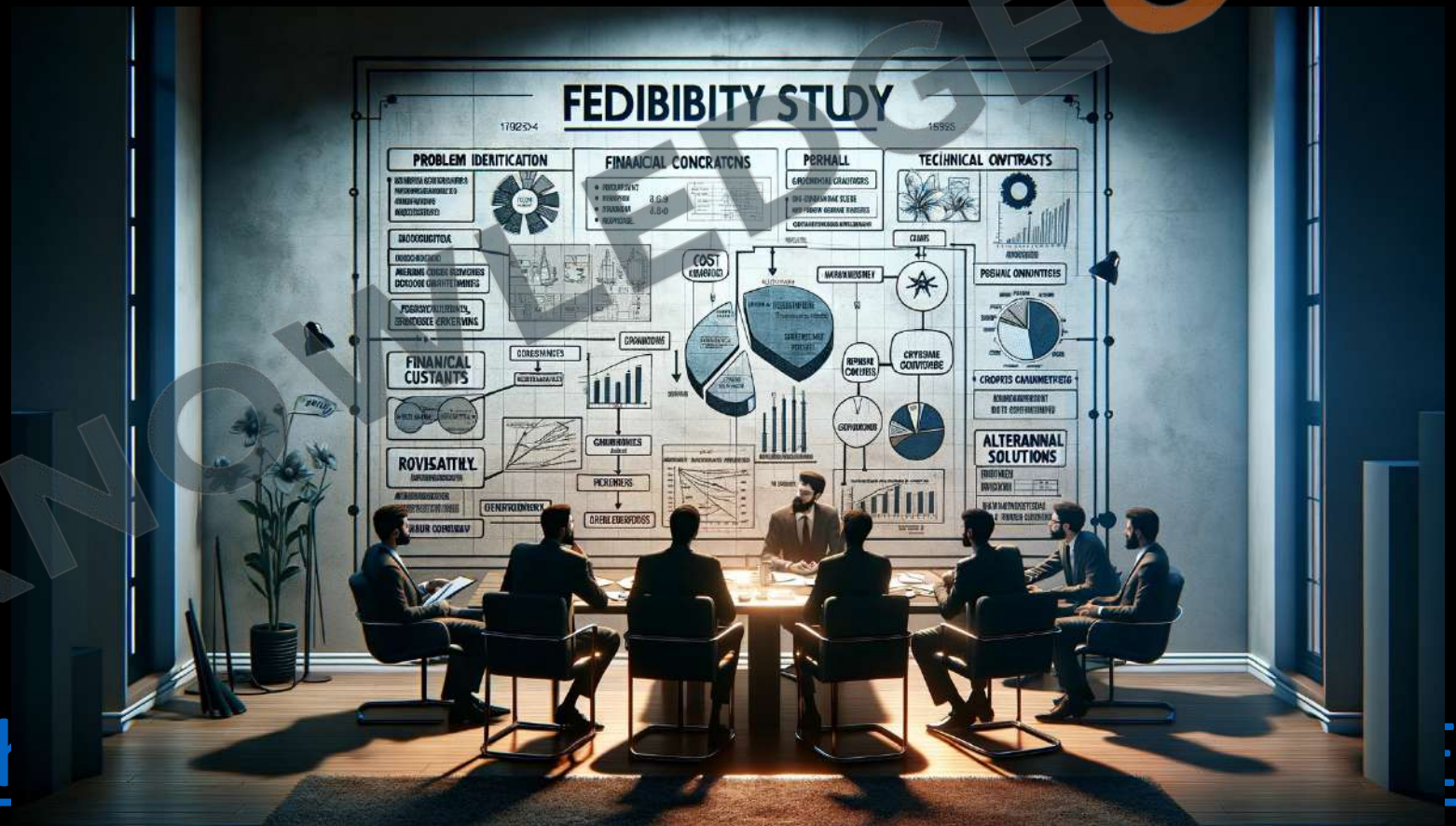
Software Process

- A software process (also known as software methodology) is a set of related activities that leads to the production of the software. These activities may involve the development of the software from the scratch, or, modifying an existing system.

KNOWLEDGEGATE

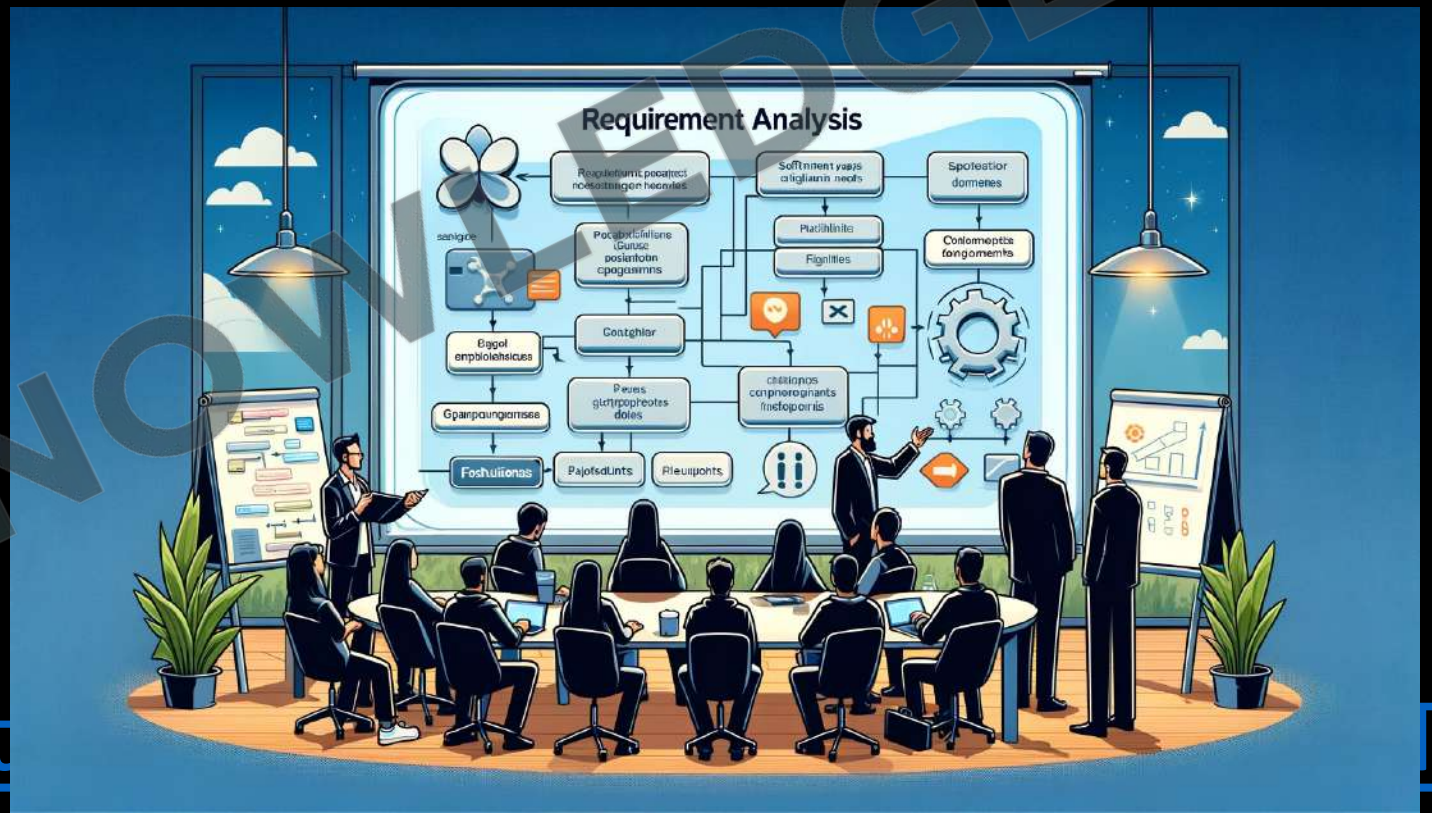
http://www.knowledgegate.in/GATE

- **Feasibility study**
  - Find abstract definition of the problem
  - Majorly checking the financial and technical feasibility
  - Analysis of cost and befit ratio
  - Checking availability of infrastructure and human resource
  - Examination of alternative solution strategies



KNOWLEDGE GATE

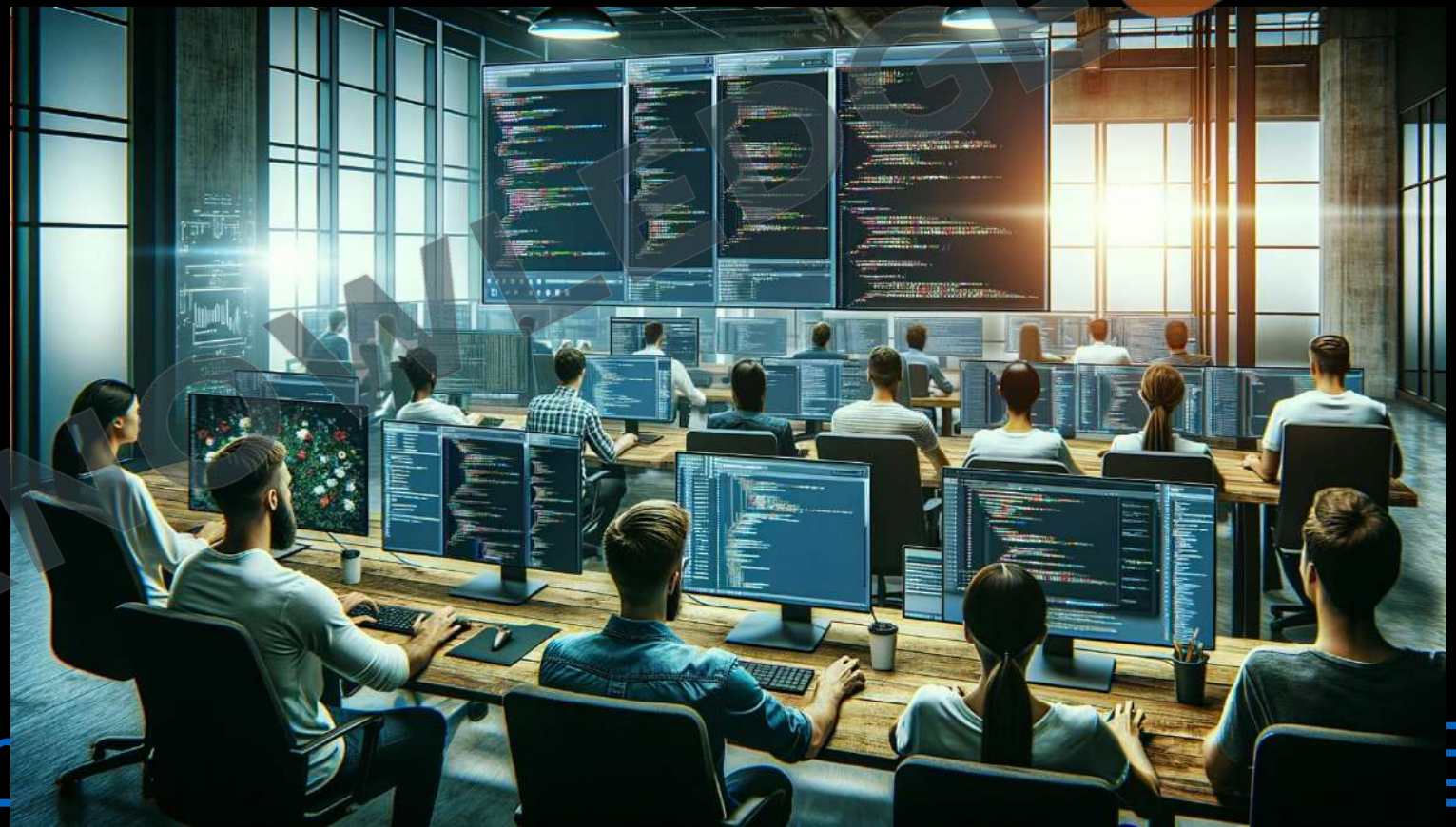
- **Requirement analysis and specification**
  - Try to understood the exact and complete requirement of the customer and document them properly.
  - Try to collect and analysis all data related to the project.
  - In last, a large document will be written in the natural language which will describe what the system will do without describing it how, called SRS Software Requirement Specification.
  - Very critical phase because, a small error hear can result of sever problem in later stages



KNOWLEDGEGATE

- **Designing**
  - We transform the requirements into a structure that is suitable for implementation of the code in a specific programming language.
  - Overall architecture and the algorithmic strategy are chosen (Coupling and cohesion).
  - Lastly will prepare a document called SDD (software design description), which will describe how the system will perform functionality.

- **Coding**
  - Goal of coding is to translate the design of the system into a code of programming language.
  - It affects both testing and maintenance, so also critical feature.
  - We will be discussing some guidelines for how to write maintainable and readable code.



KNOWLEDGE GATE

- **Testing**
  - Because of human errors there will be a bug or fault in the code and if that bug/fault is executed it become a failure.
  - Software testing is a process of executing a program with the intention of finding bugs or fault in the code.

- **Implementation**
  - Software is installed on the user site and training of the user and h/w requirement check is done.

KNOWLEDGEGATE

- **Maintenance**
  - Any change made in the software after its official release is called maintenance. It could be because of various reasons.
    - Adaptive
    - Corrective
    - Perfective

KNOWLEDGE GATE

SOFTWARE DEVELOPMENT LIFE CYCLE

- Software development organization follows some process when developing a s/w product, in mature organization this is well defined and managed.

- In SDLC we development s/w in a systematic and disciplined manner. SDLC will define entry and exit for every stage. It makes assessment possible, time prediction, cost prediction, scheduling, to identify faults early possible.

- Selection of a correct development model play an important role in cost, quality, overall success of the project.

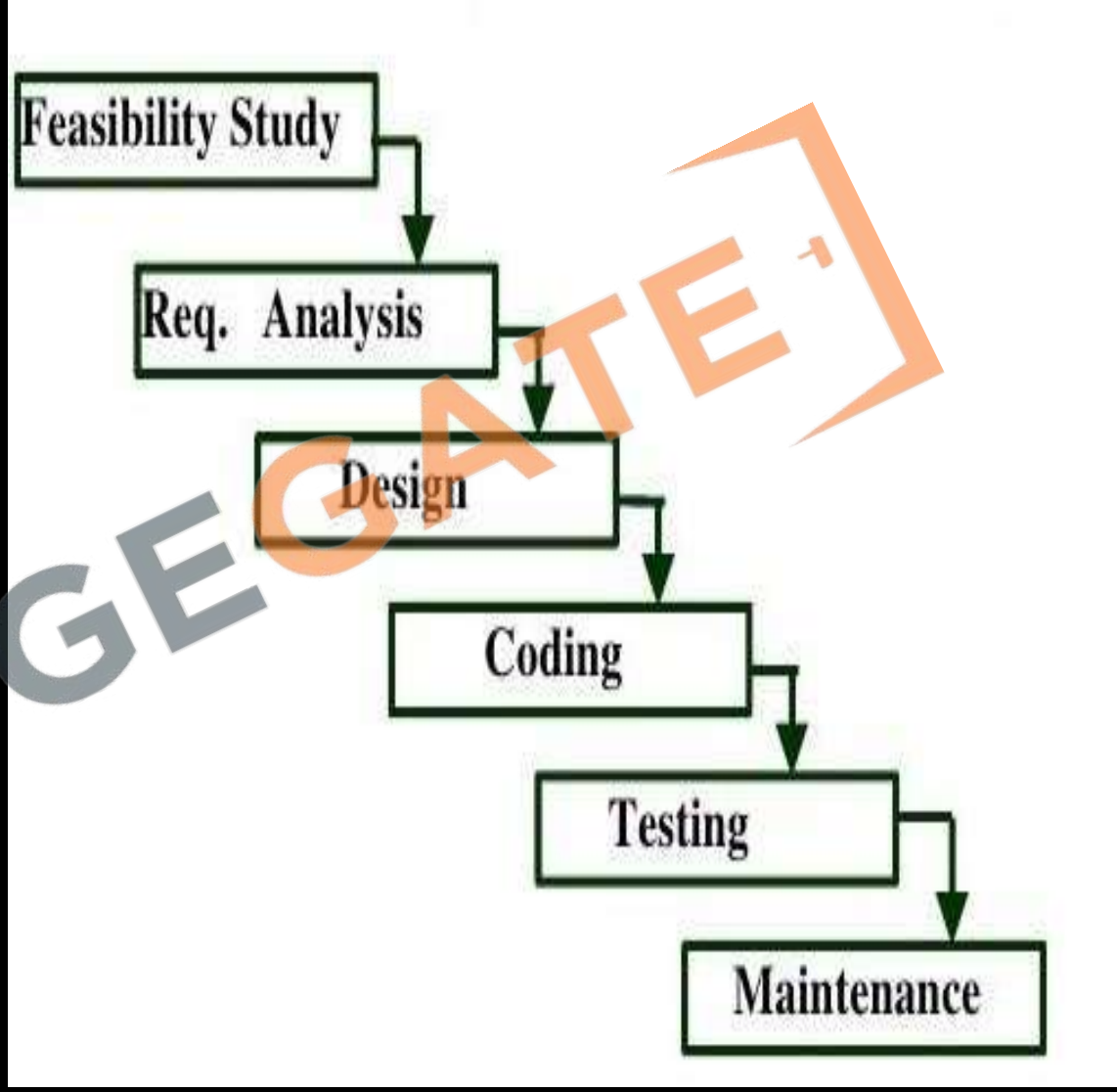
KNOWLEDGEGATE

Waterfall Model

- Developed in the 1970s by Winston W. Royce.
- Inspired by manufacturing and construction processes, where each step relies on the completion of the previous one.

KNOWLEDGEGATE
gegate.in/GATE

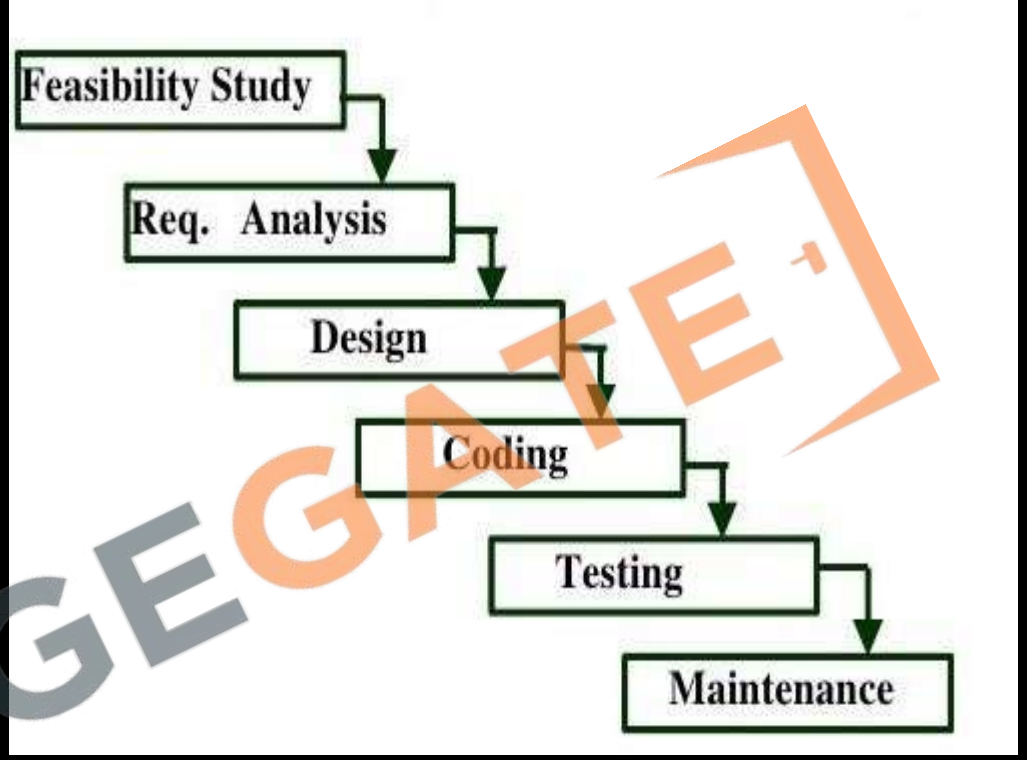
- It is a simplest SDLC in which phases are organised in a linear and sequential order.
- It is called waterfall model as its diagrammatic representation resembles as like a waterfall, also known as classical life cycle model.
- This type of model is basically used for the Small to medium-sized projects with clear, well-defined requirements. When the technology and tools to be used are well-known and stable.
- Projects where minimal changes are expected during the development process, and predictability is prioritized over adaptability.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Advantage

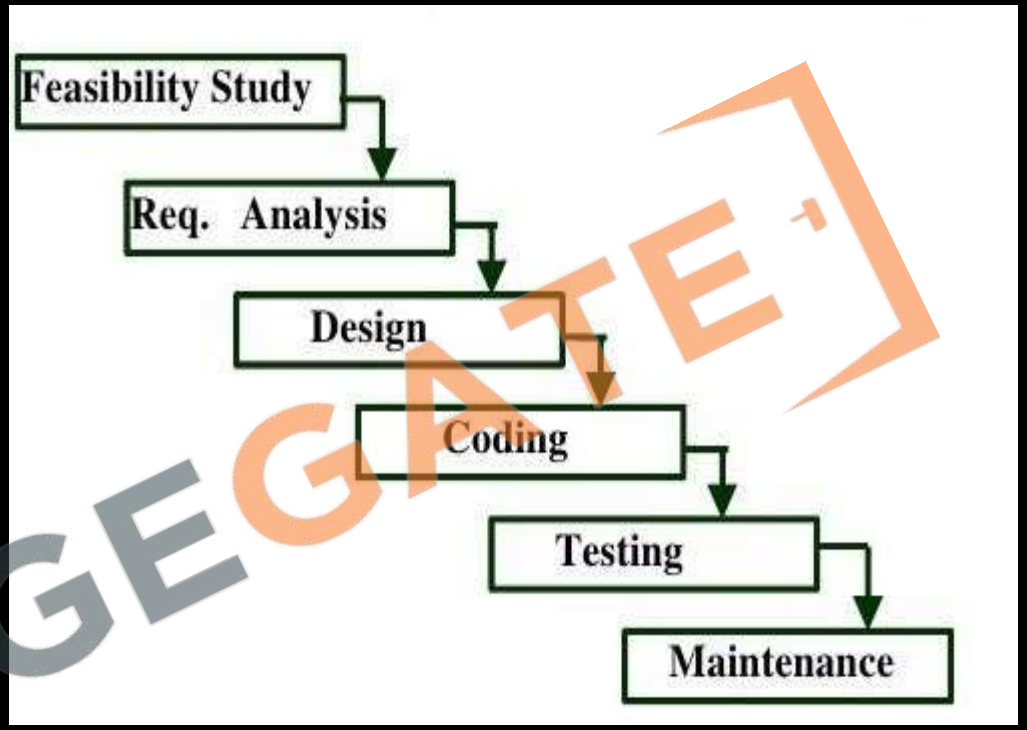
- Easy to understand and implement, with well-defined stages and clear milestones.
- Each phase has well defined input and output, phases are processed and completed one at a time. Phases do not overlap.
- Low cost and easy to schedule, as all staff do not work concurrently on the same project, so can work on different projects.



KNOWLEDGEGATE

Disadvantage

- Not suitable to accommodate any change or iteration once development has begun. It is always difficult to acquire of the requirement in the starting.
- Working version is produced in the last level, so not good for large size sophisticated projects.
- High amounts of risk and uncertainty.



KNOWLEDGEGATE

PROTOTYPE MODEL

- key design details to consider when designing a 3bhk flat in brief:
- **Living Room**: Sofa set, coffee table, entertainment center, lighting, natural light, TV unit, indoor plant/art piece, coordinated color scheme.
- **Dining Area**: Space for 4-6 seater dining table, lighting fixture, storage space, mirror/artwork.
- **Kitchen**: Storage cabinets, refrigerator, stove, oven, ventilation, lighting, durable countertops and backsplash.
- **Master Bedroom**: Space for queen/king-sized bed, wardrobe, dressing table, natural light, coordinated color scheme.
- **Other Bedrooms**: Space for single/double bed, wardrobe, natural light, coordinated color scheme.
- **Bathrooms**: Space for shower, sink, toilet, ventilation, storage space, water-resistant materials.
- **Balcony**: Space for seating/table, natural light, outdoor view, coordinated color scheme.

KNOWLEDGEGATE

PROTOTYPE MODEL

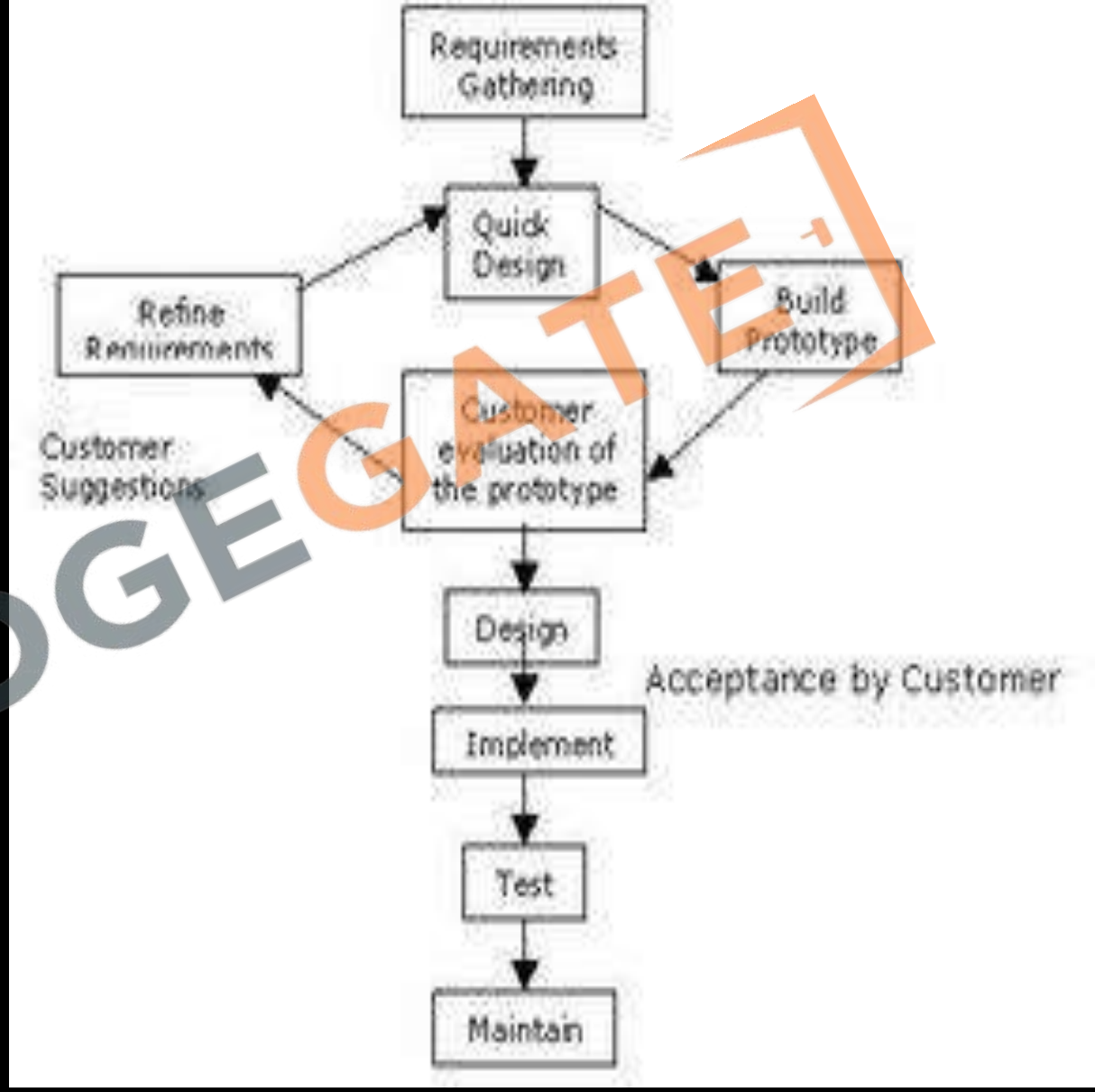
KNOWLEDGE GATE

PROTOTYPE MODEL

KNOWLEDGEGATE

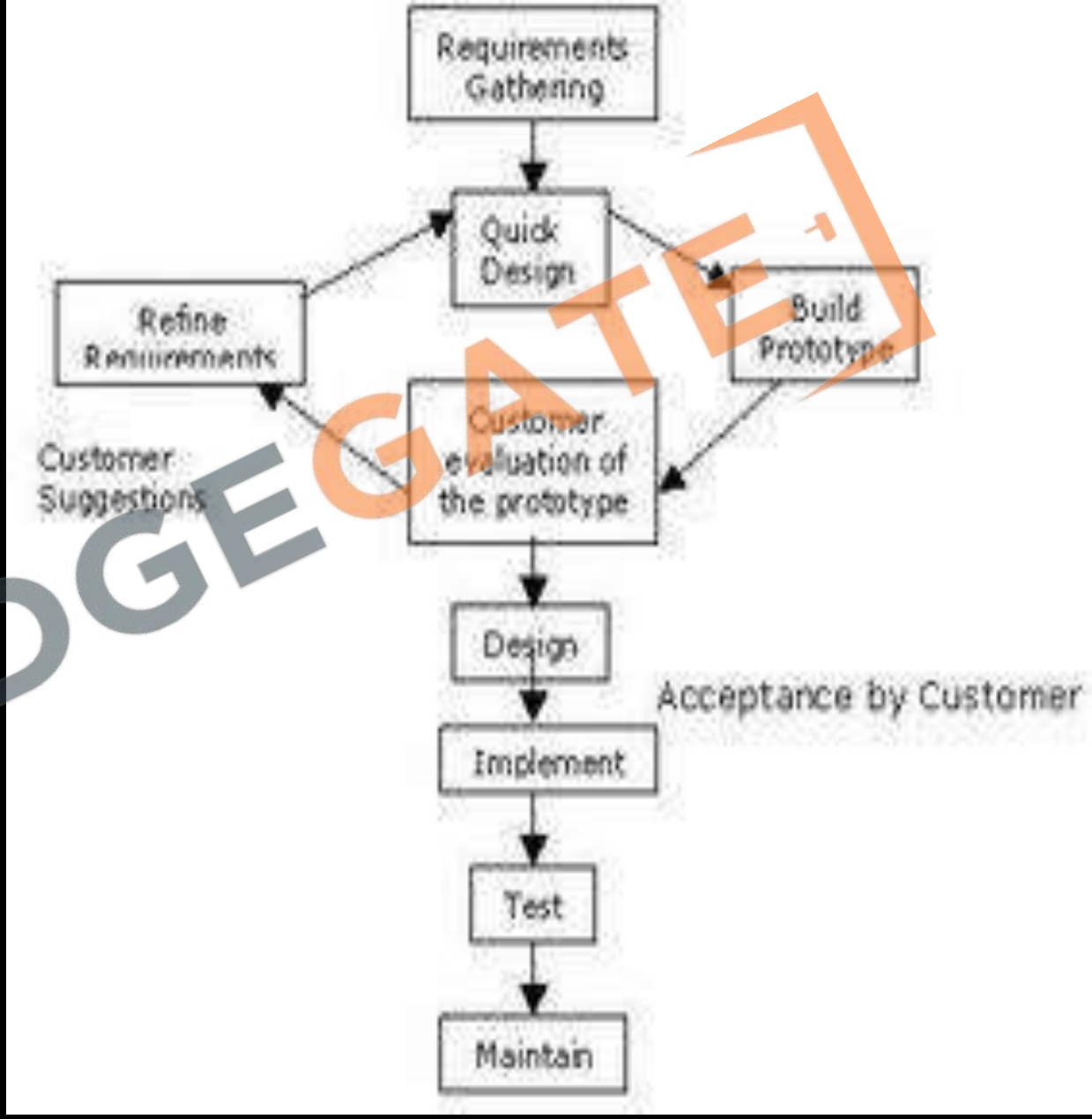
PROTOTYPE MODEL

- Most of the customer are not sure about the functionality they require from the software, as a result the final s/w is not according to exact demand.
- It is an iterative approach, which involves developing an early working model of the software based on the currently known requirements with limited functionalities, low reliability and untrusted performance.
- Refining it through user feedback, and repeating the process until a satisfactory solution is achieved.



KNOWLEDGEGATE

- Then shown to the user, as per the feedback of the user prototype is rebuilt and modified and again shown to the user, the process continue till the customer is not satisfied.
- After this process the final SRS document is developed.
- Developing the prototype help in building the actual design.
- Prototype model is of two types
  - Evolutionary prototype
  - Throwaway prototype



KNOWLEDGEGATE

Evolutionary prototype

- Also known as incremental or iterative prototyping, develops a working prototype, gradually improving and refining it based on user feedback.
- Prototype evolves into the final product over time with feature additions and modifications. Accommodates changing requirements during the development process.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Throwaway prototype

- Throwaway prototyping, also known as rapid or exploratory prototyping, involves creating a temporary, simplified version of the software to validate the feasibility of specific features, test ideas, or gain user feedback on the initial design. The throwaway prototype is not intended to be used in the final product; instead, it is discarded once it has served its purpose.
- Once the developers have gathered the necessary information, they can then start developing the actual software product from scratch, using the lessons learned from the prototype. This approach is particularly useful for testing complex or risky features, as it helps to minimize the potential impact of these features on the overall project.

KNOWLEDGEGATE

- **Advantage**
  - Customer get a chance to see the product early in the life cycle, and give important feedback.
  - There is a scope to accommodate new requirements.
  - Developer and more confident, and hence risk is reduced.

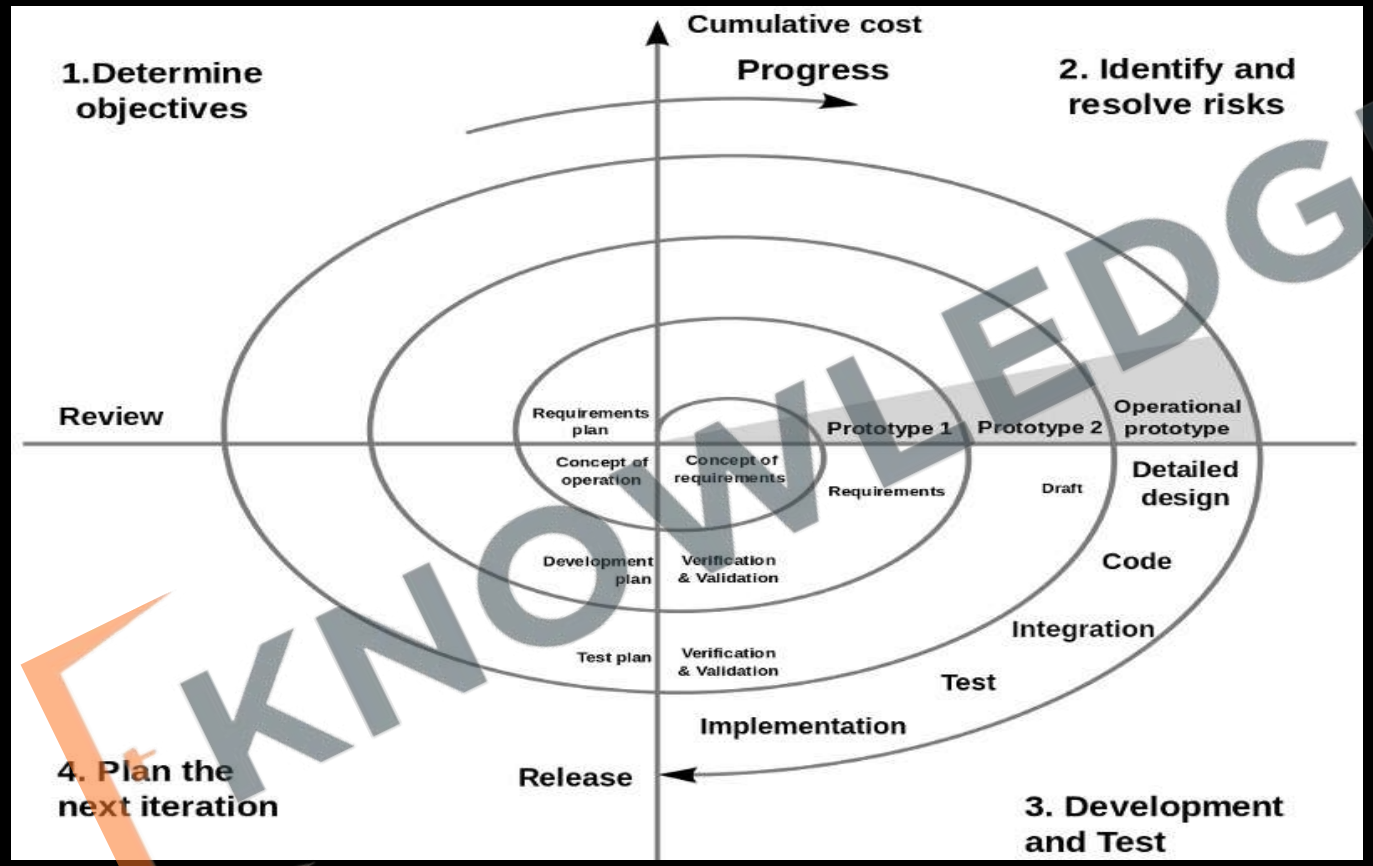
- **Disadvantage**
  - After seeing the early prototype use demand the actual system soon.
  - If not managed properly the iterative process can run for a long time.
  - If user is not satisfied, he may lose its interest in the project.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Spiral Model

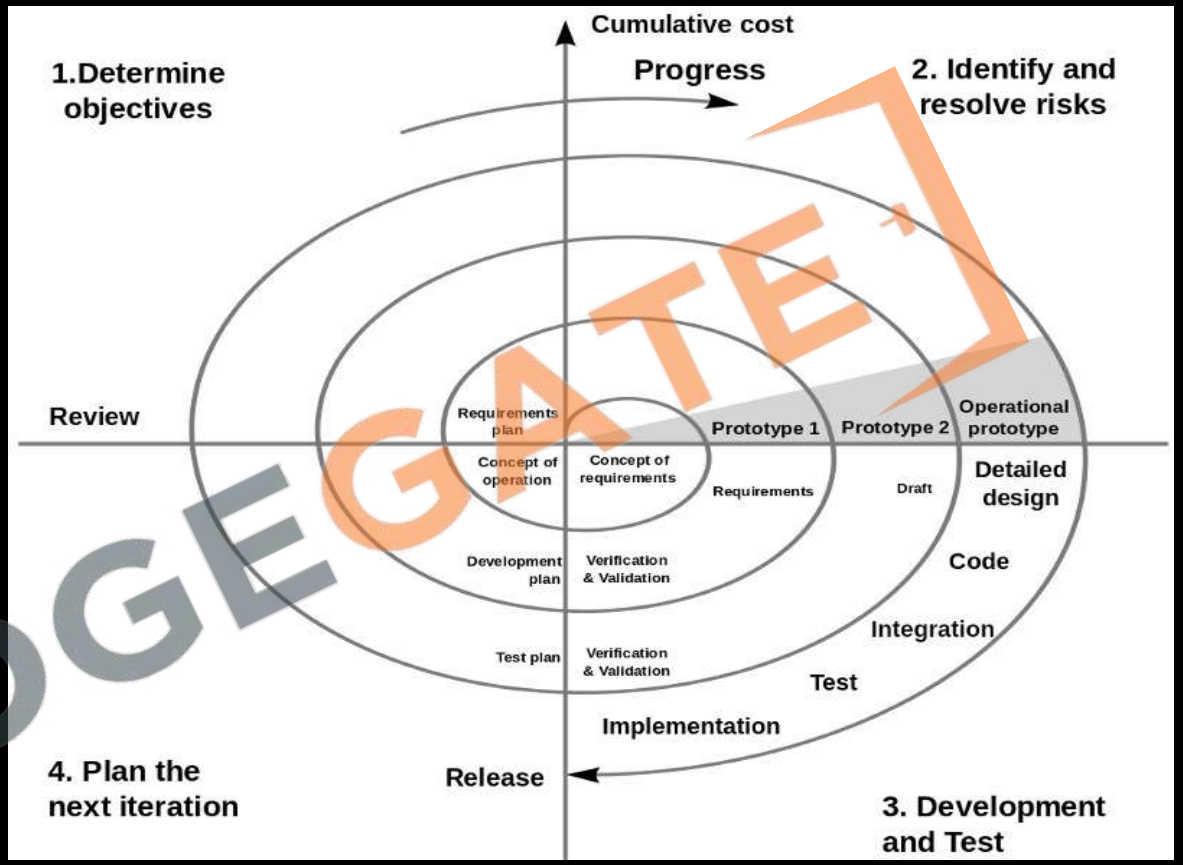
- Barry boehm recognised the project risk factor into a life cycle model, the result was the spiral model, which was presented in 1986.



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- The radial dimensions of the model represent the cumulative costs.
- Each path around the spiral is the indicative of the increased cost.
- The angular dimensions represent the progress made in the completing each cycle, each loop of the spiral from X-axis clockwise through 360 represent one phase, which is divided into four sectors
  - Determining objectives and planning the next phase
  - Risk analysis
  - Engineering the product, develop and test the product
  - Customer evaluation



KNOWLEDGEGATE

- **Advantages**
  - Provide early and frequent feedback of the customer, Additional Functionality can be added at a later date.
  - Management control of quality, correctness, cost, schedule is improved through review at the conclusion of each iteration.
  - Resolve all the possible risk involved in the product early in the life cycle.
  - Allows for incremental releases and testing.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

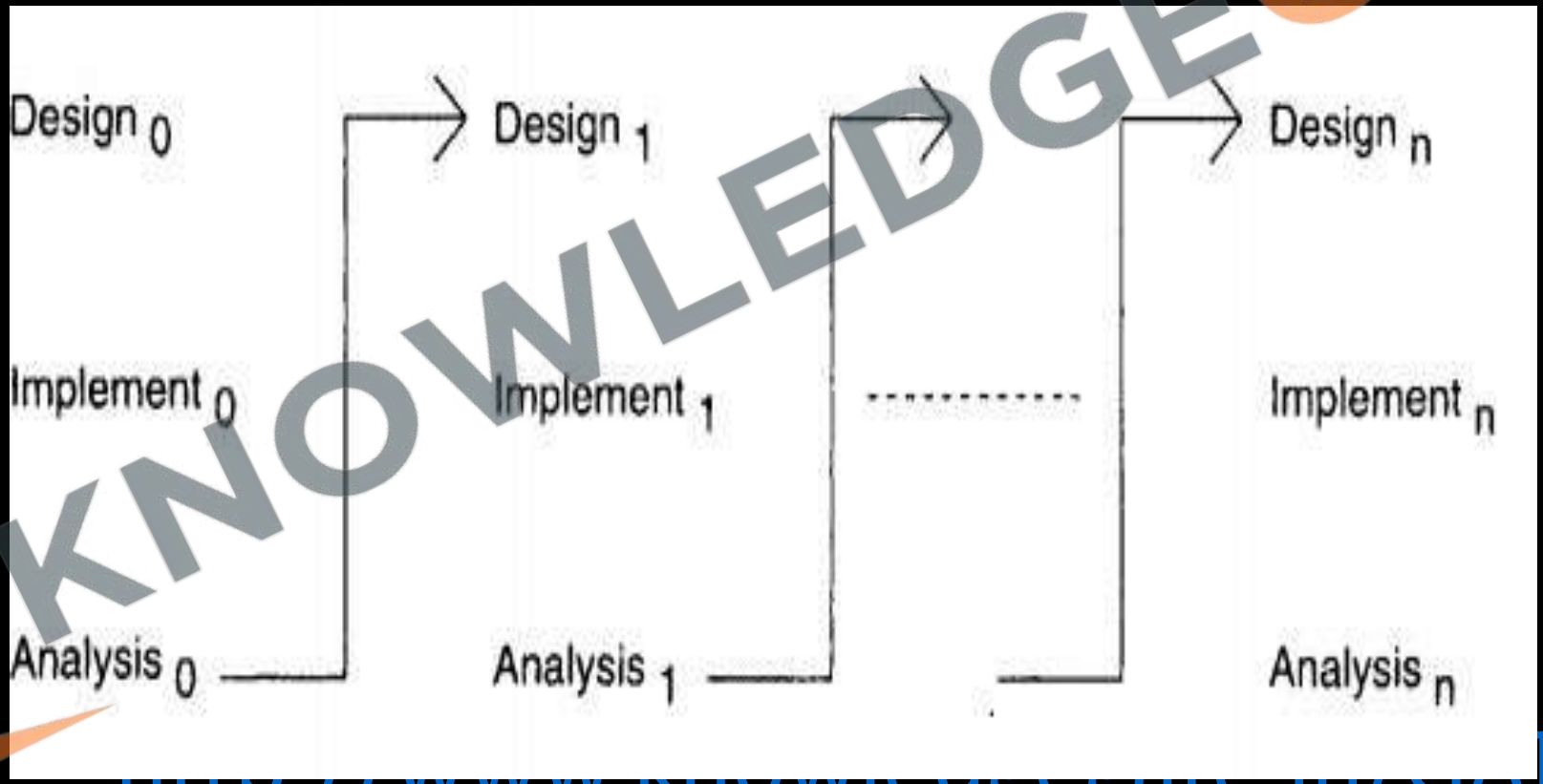
- **Disadvantages**
  - Not suitable for small size project as the cost of risk analysis may exceed the actual cost of the project, High administrative overhead.
  - It is complex and time-consuming to use.
  - Risk analysis requires highly specific expertise.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

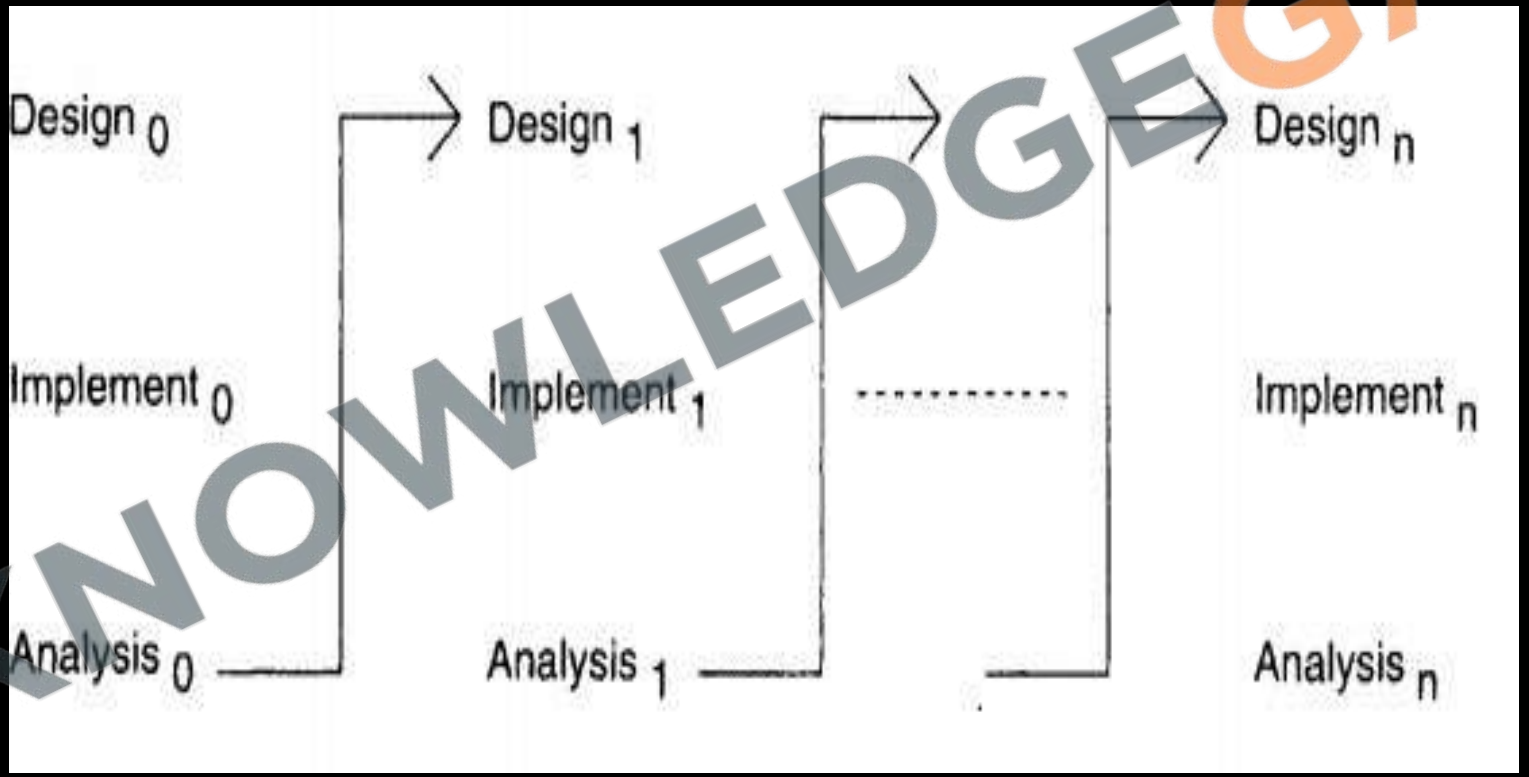
Incremental Process / Incremental Development Model

- Incremental development is based on the idea of developing an initial implementation, exposing this to user feedback, and evolving it through several versions until an acceptable system has been developed.
- In incremental model the whole requirement is divided into various builds. Multiple



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

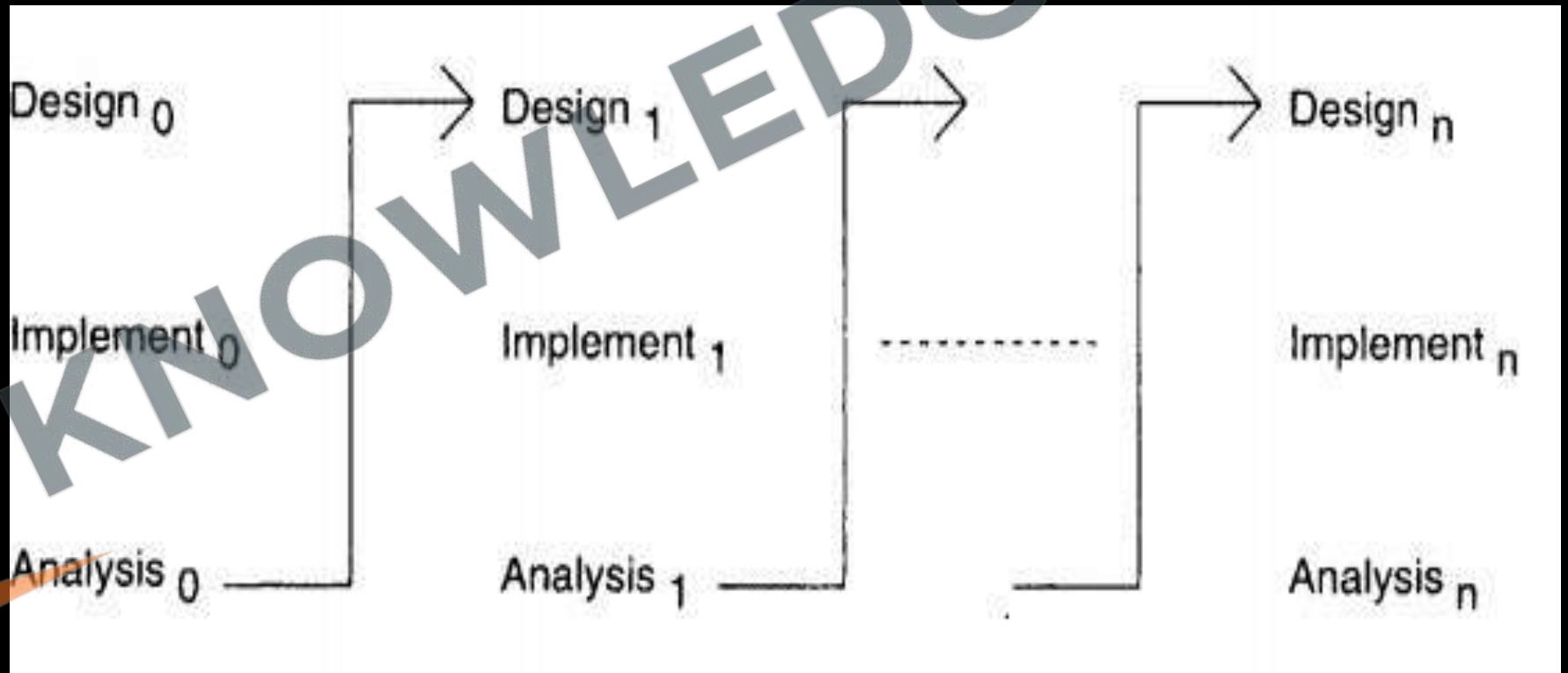
- Development cycles take place here, making the life cycle a “multi-waterfall” cycle.
- Cycles are divided up into smaller, more easily managed modules. Each module passes through the requirements, design, implementation and testing phases.



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- A working version of software is produced during the first module, so you have working software early on during the software life cycle.
- Each subsequent release of the module adds function to the previous release. The process continues till the complete system is achieved.
- Generally, the early increments of the system should include the most important or most urgently required functionality.
- tries to combine the benefits of both prototyping and the waterfall model.



KNOWLEDGE GATE

Advantages of Incremental model:

- Generates working software quickly and early during the software life cycle.
- This model is more flexible – less costly to change scope and requirements.
- It is easier to test and debug during a smaller iteration.
- In this model customer can respond to each built.
- Easier to manage risk because risky pieces are identified and handled during it'd iteration.

KNOWLEDGEGATE

Disadvantages of Incremental model

- Needs good planning and design.
- Needs a clear and complete definition of the whole system before it can be broken down and built incrementally.
- Total cost is higher than waterfall.

KNOWLEDGEGATE

Aspect	Evolutionary Development Models	Iterative Enhancement Models
Focus	Adaptation and evolution of software in response to changing requirements and user feedback.	Continuous improvement and refinement of a software system in iterations.
Initial Release	May start with a basic prototype or a partially functional system to gather feedback.	Begins with a basic version of the software, usually with minimal functionality.
Development Approach	Develops through repeated cycles (e.g., prototypes or increments) with each cycle refining and expanding the software.	Focuses on enhancing and refining an existing system through multiple iterations.
Feedback Integration	Heavily relies on user feedback to guide each evolutionary stage, often with significant changes between stages.	Uses feedback to make improvements in each iteration, usually with smaller, incremental changes.
Examples	Prototyping Model, Spiral Model, Agile Development, RAD.	Agile Development (as a specific approach focusing on enhancement and refinement), Iterative Development Model.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Unit-2(Software Requirement Specifications (SRS))

- Requirement Engineering Process:
  - Elicitation,
  - Analysis,
  - Documentation,
  - Review and Management of User Needs,
- Feasibility Study,
- Information Modeling,
- Data Flow Diagrams,
- Entity Relationship Diagrams,
- Decision Tables,
- SRS Document,
- IEEE Standards for SRS.
- Software Quality Assurance (SQA): Verification and Validation,
- SQA Plans, Software Quality Frameworks, ISO 9000 Models, SEI-CMM Model

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Requirement Analysis

- The hardest part of building a software is deciding precisely what is to be built. Requirement engineering is the disciplined application of proven principle, methods, tools and notations to describe a proposed system's intended behaviour and its associated constraints.

KNOWLEDGEGATE

How the customer explained it

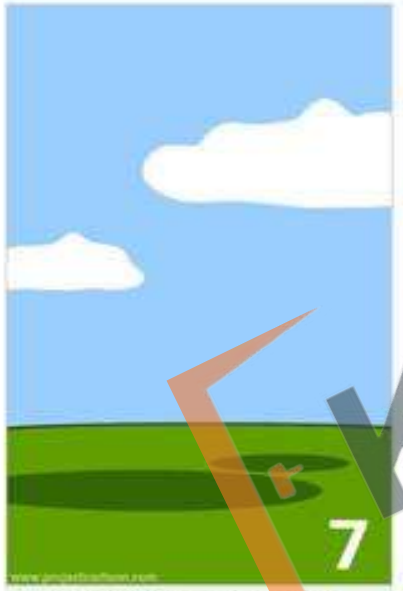
How the project leader understood it

How the analyst designed it

How the programmer wrote it

What the beta testers received

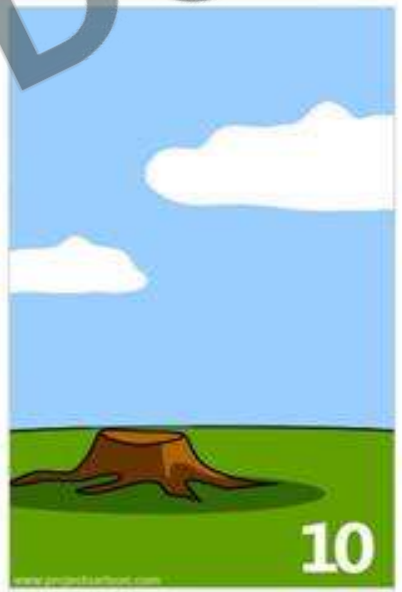
How the business consultant described it



How the project was documented

What operations installed

How the customer was billed



How it was supported

What marketing advertised

What the customer really needed

KNOWLEDGEGATE

Difficulties of Requirement Engineering

- **Requirement are difficult to uncover**: no one give the complete requirement in the first time, if someone does then also requirement are incomplete

KNOWLEDGEGATE

- **Requirement Changes**: with the development process requirements get added and changed as the user begins to understand the system and his or her real need

KNOWLEDGE GATE

- **Tight project schedule**: have insufficient time to do a decent job

KNOWLEDGEGATE

- **<u>Communication Barrier</u>**: user and developer have different technical background and tastes

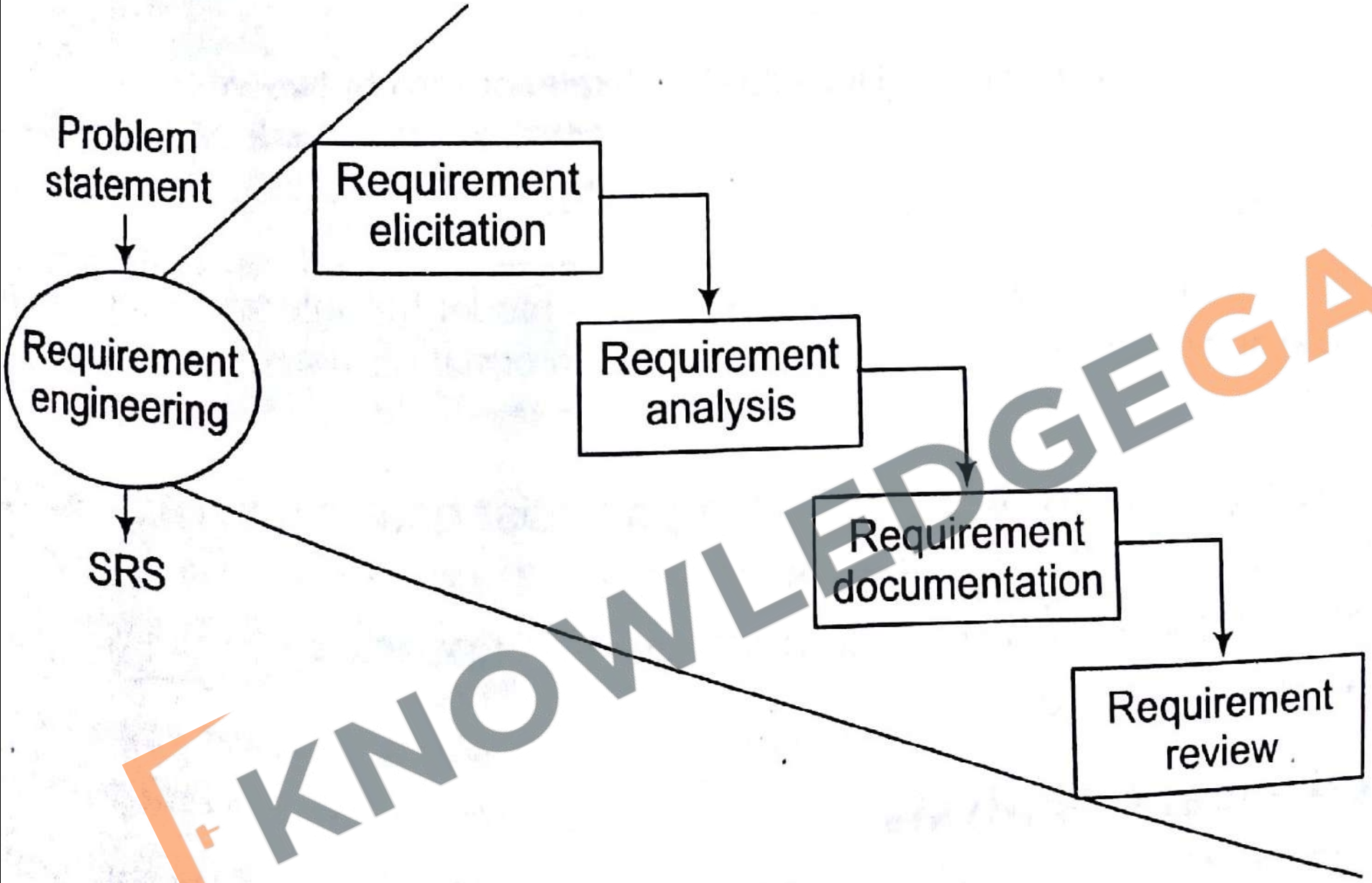
KNOWLEDGE GATE

- **Lack of resources**: there may not be enough resources to build software that can do everything the customer wants

- Types of requirement
  - **Known requirement**: Which is already known to the stake holder
  - **Unknown Requirement**: Forgotten by stake holder because that are not needed right now
  - **Undreamt Requirement**: Stakeholder is unable to think of new requirement due to limited domain knowledge

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE



Various steps of analysis and specifications

KNOWLEDGEGATE

- There are majorly four steps in requirement analysis
  - **Requirement Elicitation**: Gathering of Requirement
  - **Requirement Analysis**: Requirements are analysed to identify inconsistences, defects etc and to resolve conflicts
  - **Requirement documentation**: Here we document all the refined requirement properly
  - **Requirement Review**: Review is carried out to improve the quality of the SRS

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Methods of requirement elicitation

- Is the most difficult, most critical, most error-prone and communication intensive aspect of the s/w development. It can only succeed only through an effective customer developer partnership.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Interview**
  - Both parties would like to understand each other
  - Interview can be of two types open-ended or structured
  - In open ended, there is no present agenda, context free questions can be asked to understand the problem, to have an over view over the situation
  - In structured interview, agenda is pre-set.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Brain Storming**
  - A kind of group discussion, which lead to ideas very quickly and help to promote creative thinking
  - Very popular now a days and is being used in most of the organizations
  - All participants are encouraged to say whatever idea come to their mind and no one will be criticized for any idea no matter how goofy it seems

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Delphi technique**
  - Here participants are made to write the requirement on a piece of paper, then these requirements are exchanged among participants who gave their comments to get a revised set of requirements. This process is repeated till the final consensus is reached

KNOWLEDGEGATE

http:/ e.in/GATE

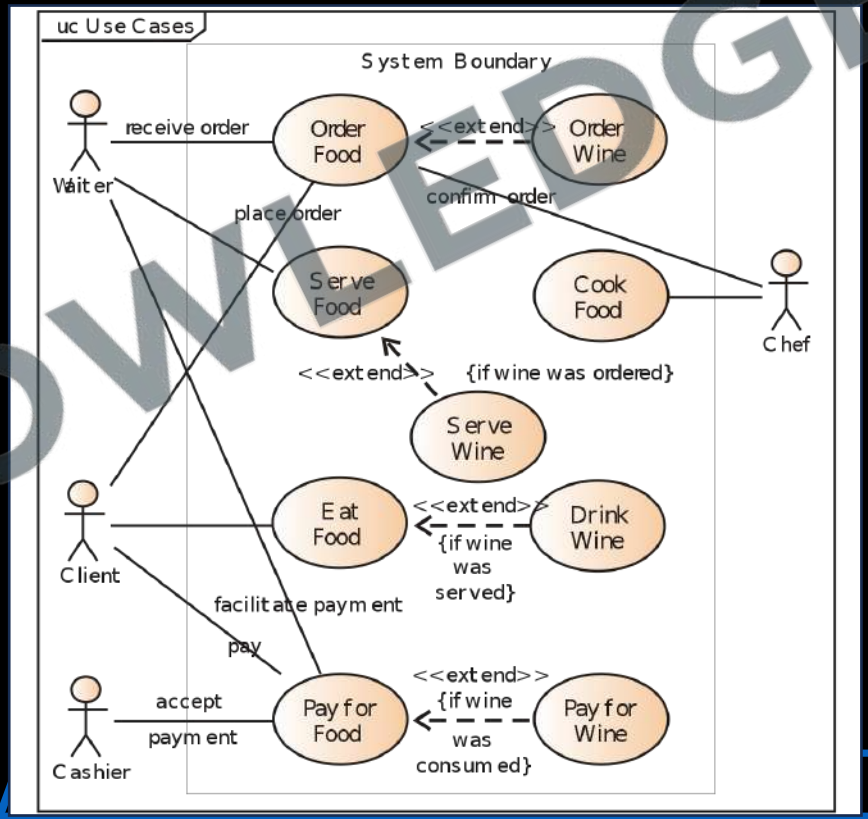
- **FAST (facilitated application specification technique)**
  - This approach encourages the creation of joint team of customer and developer who works together to understand correct set of requirements
  - Everyone is asked to prepare a list of
    - What surrounds the system
    - Produced by the system
    - Used by the system
    - List of service, constraints, and performance criterion
    - Then we divide these lists into smaller list to work in smaller teams

KNOWLEDGE
http://www.knowle

- **QFD (quality functional deployment)**
  - Its emphases to incorporate the voice of the customer with importance
  - Then according to customer, a value indicating a degree of importance, is assigned to each requirement. Thus, the customer, determine the importance of each requirement on a scale of 1 to 5 as:
    - **5 points**: very important
    - **4 points**: important
    - **3 points**: not important but nice to have
    - **2 point**: not important
    - **1 point**: unrealistic, requires further explanation

KNOWLEDGEGATE
http://www.knowled

- **Use case approach**
  - These are structured description of the user requirement. It is a narrative which describe the sequence of events from user's perspective
  - Use case diagrams are graphical representation to show the system at different levels
  - They are some times supportive my the activity diagrams, to understand the work flow



KNOWLEDGEGATE

http://...te.in/GATE

Requirement analysis

- In this phase we analysis all the set of requirements to find any inconsistency or conflicts.
- In requirement gathering phase our all concentration was on getting all the set of requirements but now, we see how many requirements are contradictory to each other or requires further exploration to be considered further.
- Different tools can be used Data flow diagram, Control flow diagram, ER diagram etc

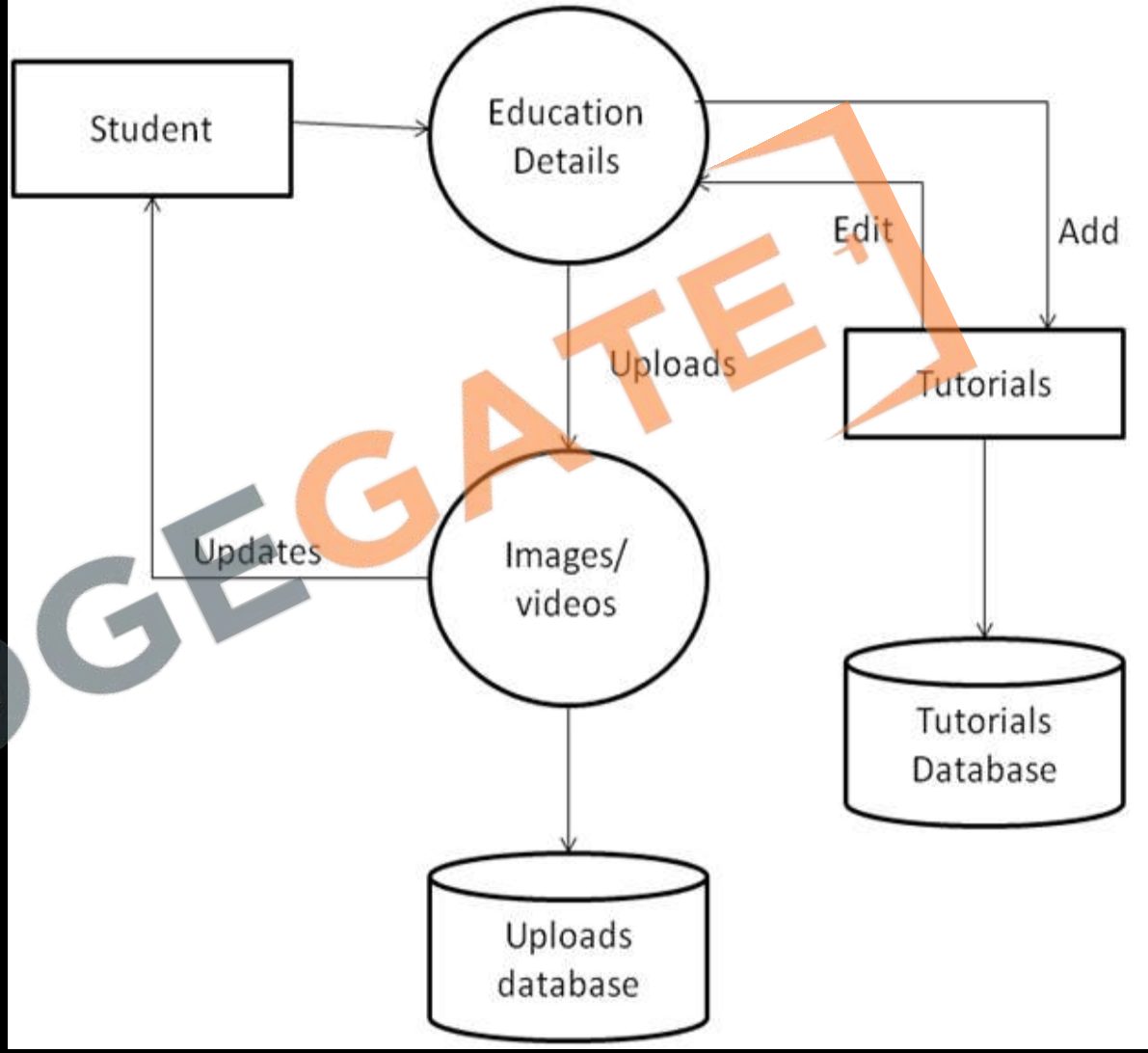
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Data Flow Diagram

- A data flow diagram or bubble chart is a graphical representation of the flow of data through a system. It clarifies systems requirements and identify major transformations.

- DFD represent a system at different level of abstraction. DFD may be Partitioned into levels that represent increasing information flow and functional details



KNOWLEDGEGATE

- Components of DFD
  - Function/Process
  - Data Store
  - External Entity
  - Data Flow

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

DFD uses hierarchy to maintain transparency thus multilevel DFD's can be created. Levels of DFD are as follows:

- **0-level DFD**: It represents the entire system as a single bubble and provides an overall picture of the system.

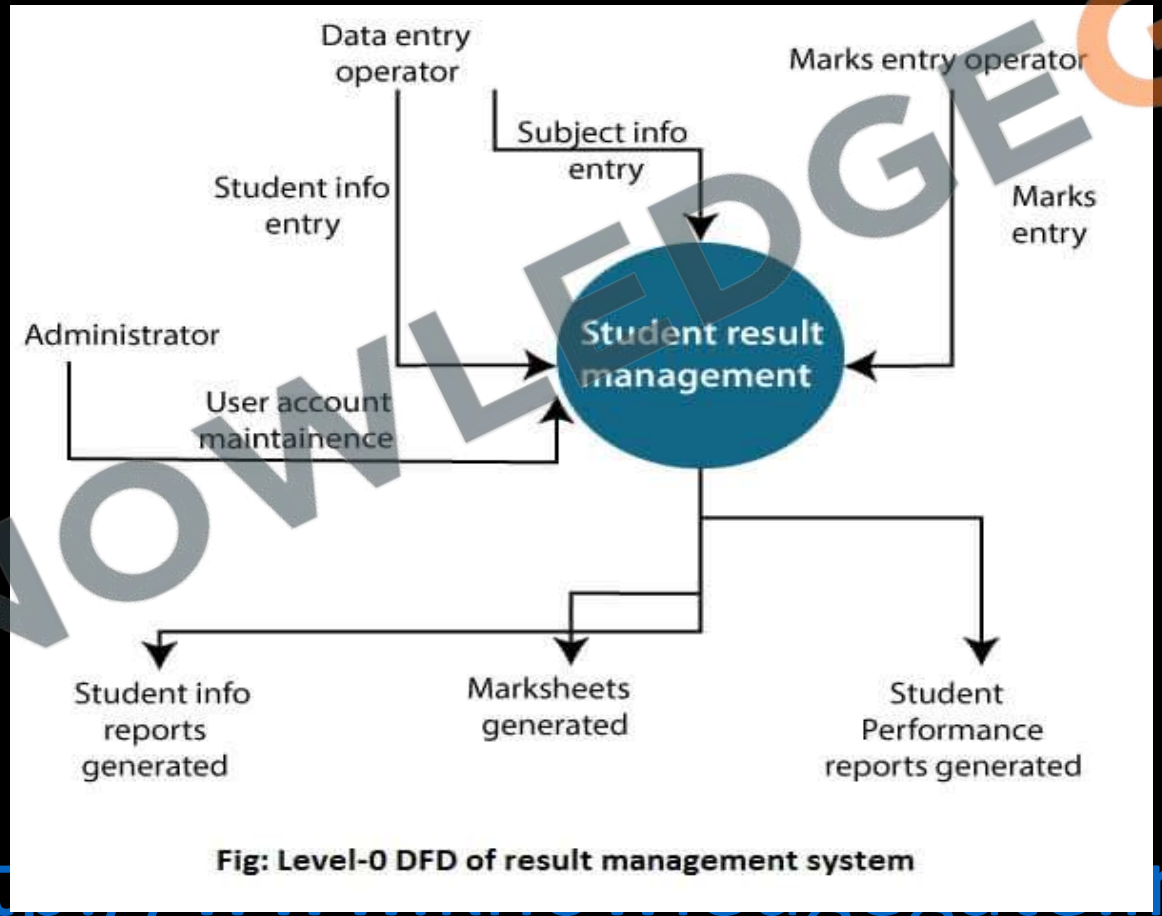


Fig: Level-0 DFD of result management system

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **1-level DFD**: It represents the main functions of the system and how they interact with each other.

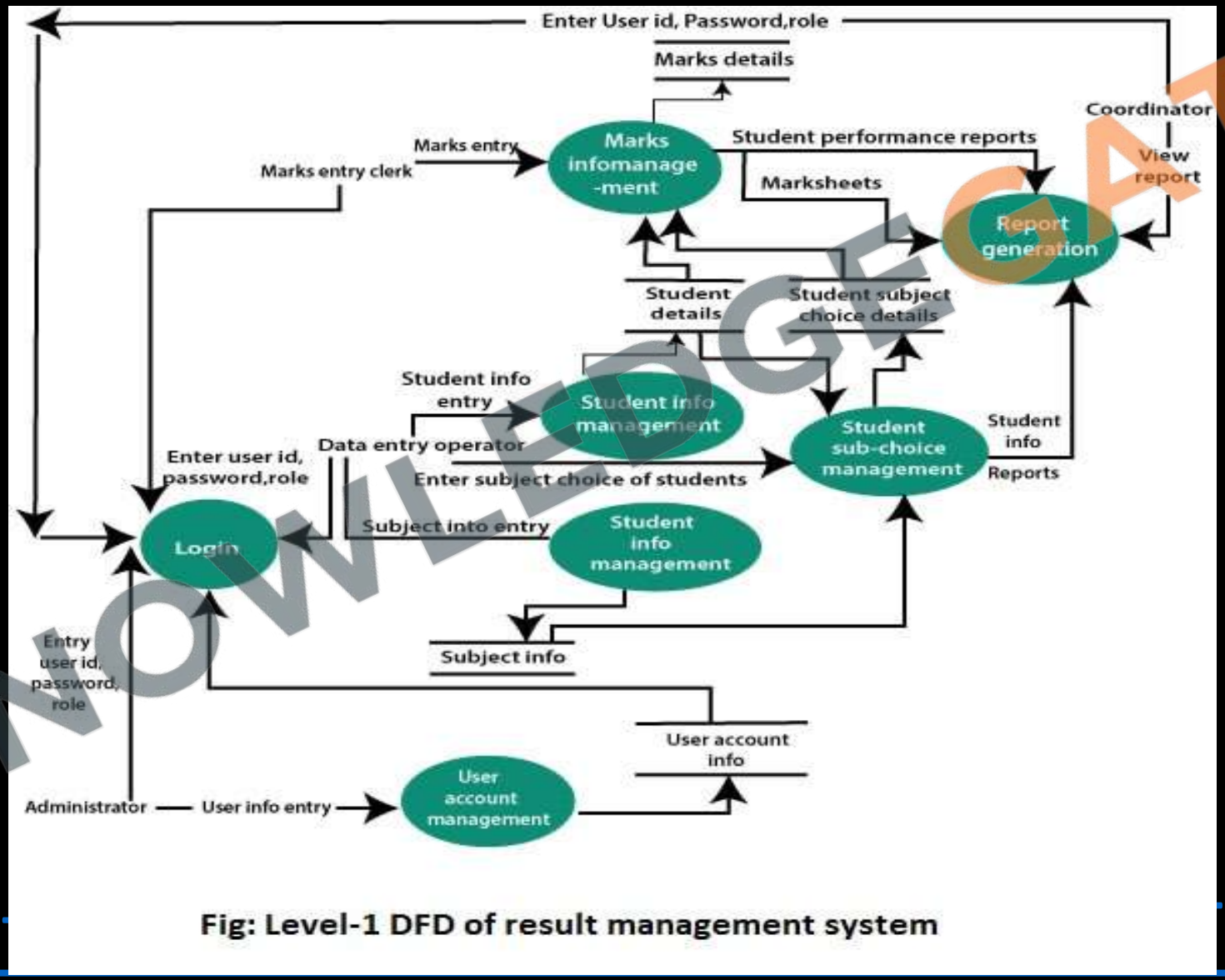


Fig: Level-1 DFD of result management system

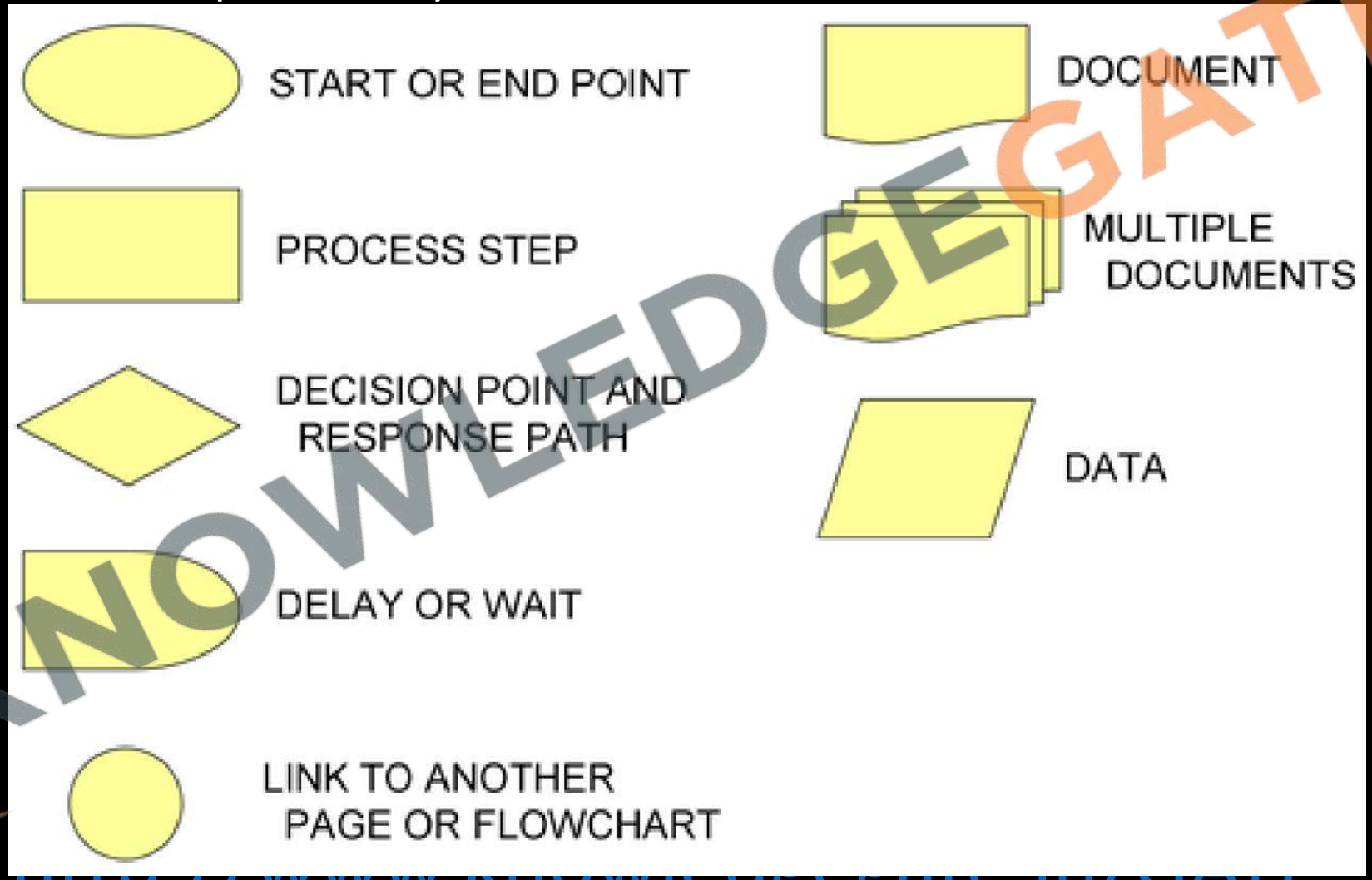
KNOWLEDGE GATE

- **2-level DFD**: It represents the processes within each function of the system and how they interact with each other.

KNOWLEDGEGATE

Control Flow Diagram/Control flow chart/Flow Chart

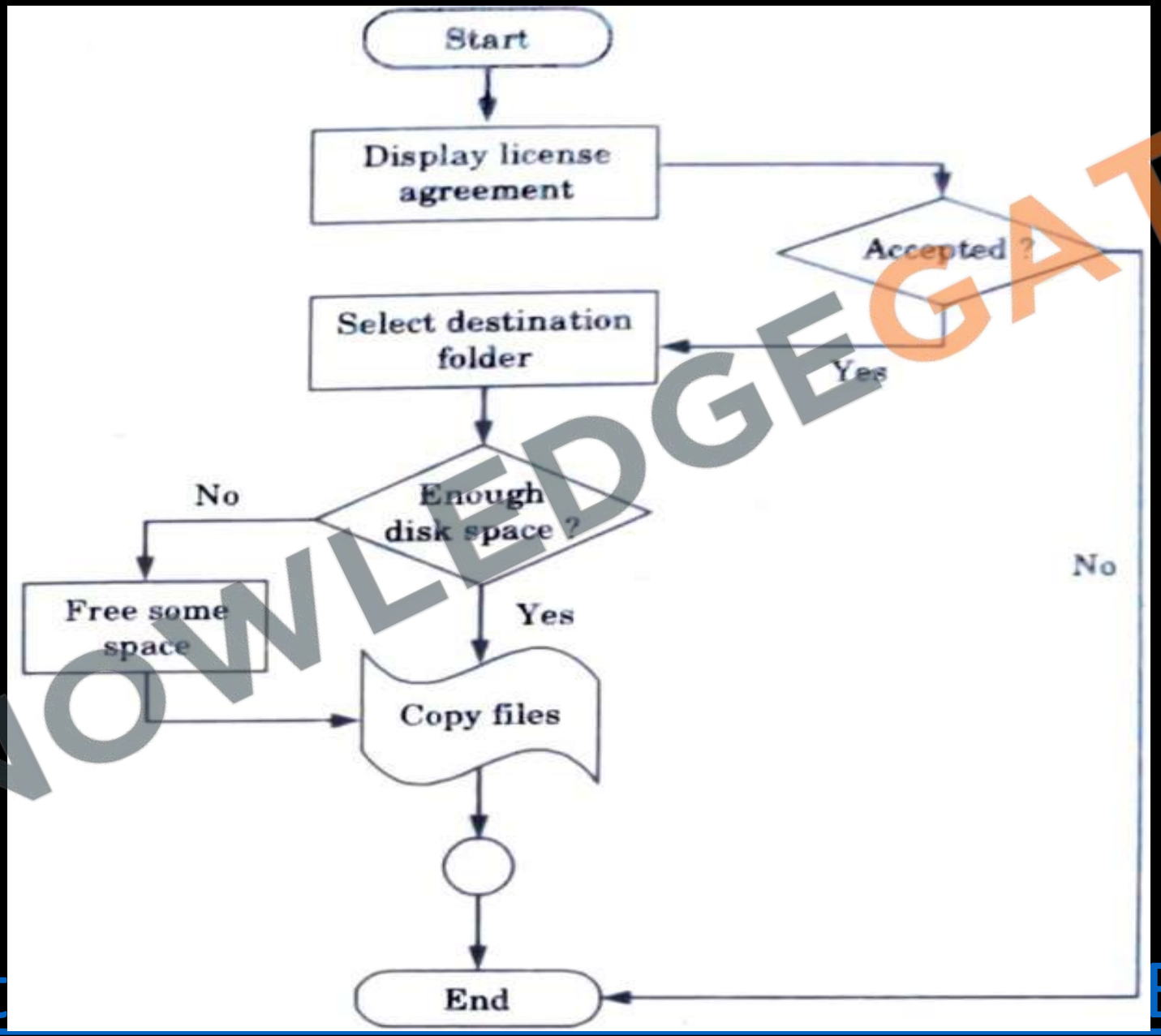
- A flow chart is a graphical representation of how control flow during the execution of a program. It use the following symbols to represent a system's control flow.



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- Flow Chart of software installation



KNOWLEDGEGATE

Data Dictionary

- **Purpose**: Serves as a repository for data item details in Data Flow Diagrams, ensuring consistent definitions between customers and developers.
- **Content**: Includes data item name, aliases, and purpose, promoting clear understanding.
- **Relationships & Range**: Records data item relationships and value ranges (e.g., marks between 0-100).
- **Data Flows & Structures**: Tracks process interactions with data items and documents their structure or composition.
- **Role in Requirements**: Essential in initial stages for defining customer data items, aligning developer and customer understanding.

KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

Data Dictionary

Data Dictionary outlining a Online Gaming Service

Data Item	Data Type	Data Format	Number of Bytes for Storage	Size for Display	Description	Example	Validation
MemberID	String	XNNNNNN	7	7	Unique Identifier For Member	M123456	
First Name	String		25	25	First Name of Member	Scott	
Surname	String		25	25	Last Name of Member	Daniels	
D.O.B	Floating Point (Date Format)	DD/MM/YYYY	4	10	Birth Date of Member	02/04/1990	Date < Today - 15 years
Platinum Membership?	Boolean	X	1	1	True (T) or False (F)	T	
Subscription Cost	Floating Point (Currency Format)	$NN.NN	4	6	Cost of Members Subscription	$27.50	Cost > 0 Cost < $50.00
Available Game Packs	Array (String)		25 * Number of Games	25 * Number of Games	Names of Game Packs	Open World Game Pack	

KNOWLEDGEGATE

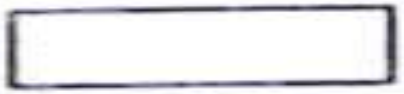
Entity Relationship Diagram

- ER Diagram a non-technical design method works on conceptual level based on the perception of the real world.

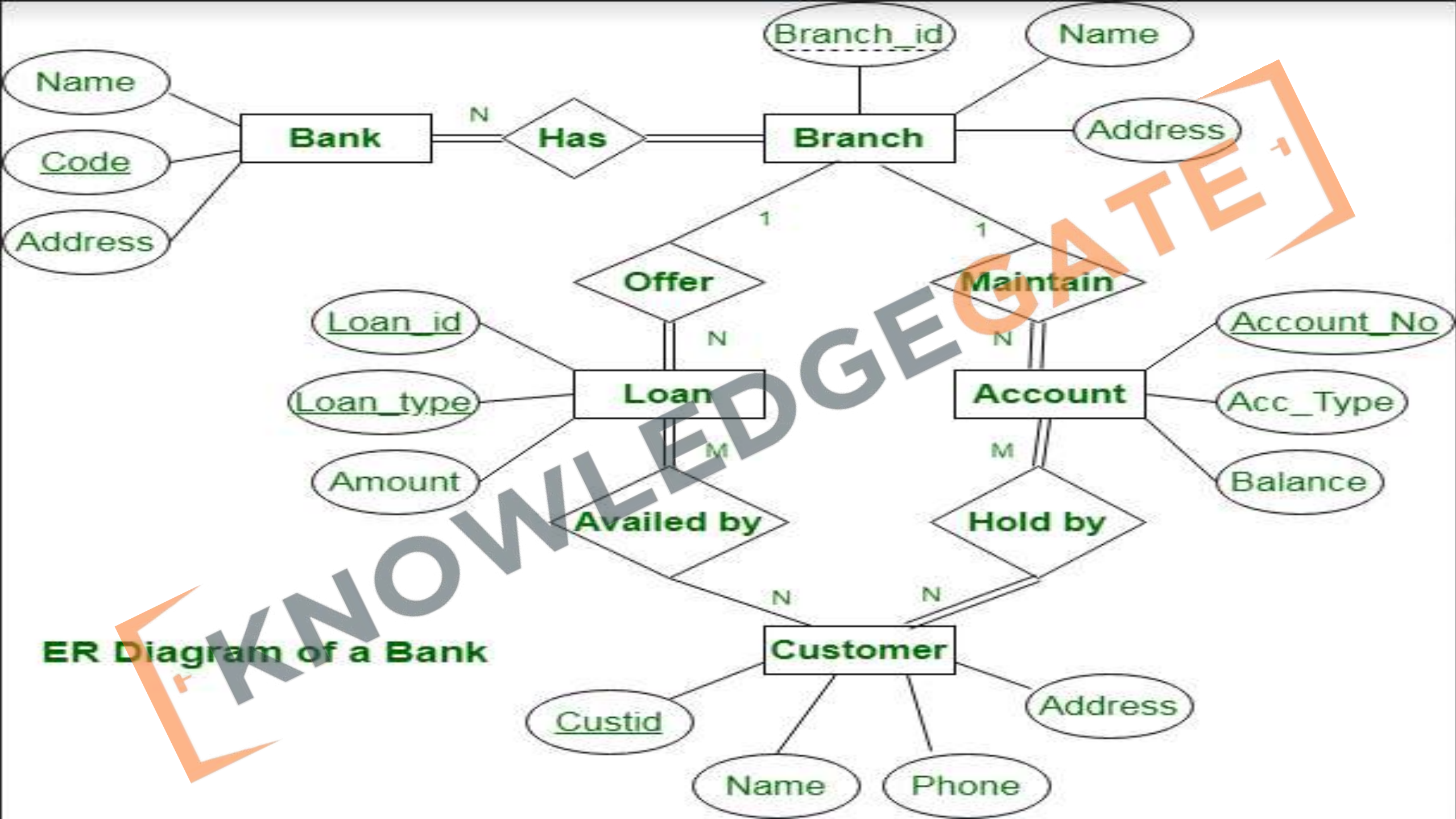
- E-R data model was developed to facilitate software designers by allowing specification of an enterprise schema that represents *the overall logical structure of a database*.

- Three main constructs are data entities, their relationships and their associated attributes

KNOWLEDGE GATE

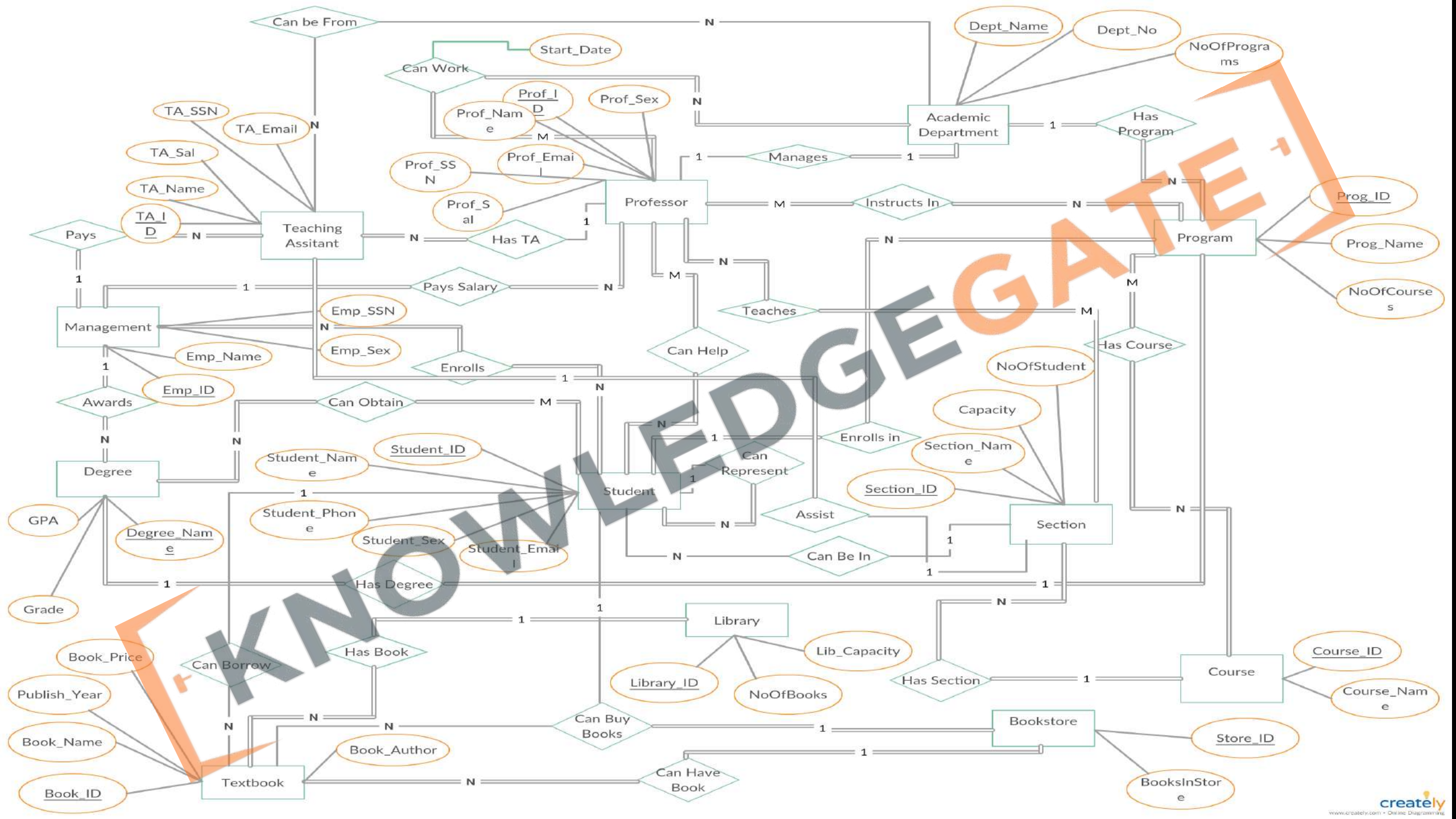
S. No.	Component name	Symbol	Description
1.	Rectangles		Represents entity sets
2.	Ellipses		Represents attributes
3.	Diamonds		Represents relationship sets
4.	Lines		Links attributes to entity sets and entity sets to relationship sets
5.	Double ellipses		Represents multivalued attributes
6.	Dashed ellipses		Represents derived attributes
7.	Double rectangles		Represents weak entity sets
8.	Double lines		Represents total participation of an entity in a relationship set

KNOWLEDGEGATE



ER Diagram of a Bank

KNOWLEDGEGATE


Can be From
N
Dept_Name
Dept_No
NoOfPrograms
Start_Date
Can Work
Prof_ID
Prof_Name
Prof_Sex
TA_SSN
TA_Email
TA_Sal
TA_Name
TA_ID
Prof_SSN
Prof_Email
Prof_Sal
Academic Department
Has Program
Manages
Professor
Instructs In
Prog_ID
Prog_Name
NoOfCourses
Program
Pays
Teaching Assitant
Has TA
Pays Salary
Management
Emp_SSN
Emp_Name
Emp_Sex
Emp_ID
Enrolls
Teaches
Can Help
Has Course
NoOfStudent
Capacity
Awards
Can Obtain
Degree
Student_ID
Student_Name
Student_Phone
Student_Sex
Student_Email
Student
Can Represent
Enrolls in
Section_Name
Section_ID
Section
GPA
Degree_Name
Grade
Assist
Can Be In
Has Degree
Library
Lib_Capacity
Library_ID
NoOfBooks
Has Section
Course
Course_ID
Course_Name
Book_Price
Publish_Year
Book_Name
Book_ID
Can Borrow
Has Book
Can Buy Books
Bookstore
Store_ID
BooksInStore
Book_Author
Textbook
Can Have Book
KNOWLEDGEGATE

Decision Tables

- A decision table is a brief visual representation for specifying which actions to perform depending on given conditions.
- A decision table is a good way to settle with different combination inputs with their corresponding outputs.
- Decision tables are very much helpful in requirements management and test design techniques. It provides a regular way of starting complex business rules, that is helpful for developers as well as for testers.

	Conditions/ Courses of Action	Rules					
		1	2	3	4	5	6
Condition Stubs	Employee type	S	H	S	H	S	H
	Hours worked	<40	<40	40	40	>40	>40
Action Stubs	Pay base salary	X		X		X	
	Calculate hourly wage		X		X		X
	Calculate overtime						X
	Produce Absence Report		X				

KNOWLEDGEGATE

Aspect	Decision Table	Decision Tree
Structure	Tabular format, lists conditions, actions, and rules.	Tree-like diagram, shows decision paths with branches.
Decision Making	Based on various combinations of conditions leading to specific actions.	Based on sequential decisions leading to outcomes.
Ease of Use	Simple for representing complex logical relationships, can become cumbersome with many variables.	More intuitive for visualizing decision paths, can become complex with many branches.
Application	Preferred in scenarios where all possible combinations of conditions and outcomes need clear representation.	Ideal for situations where decision-making process is more sequential and hierarchical.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Requirement Documentation

- After have final set of requirements it is also necessary to document them properly in a standard format, so that can be understood easily even by non-technical person.
- Here IEEE provides standard for SRS documents in IEEE830, using which we can make SRS document more readable, modifiable, and a format which can be followed through the world.

KNOWLEDGE TE

http://www.knowle

Preparation of Papers for IEEE TRANSACTIONS and JOURNALS (December 2013)

First A. Author, *Fellow, IEEE*, Second B. Author, and Third C. Author, Jr., *Member, IEEE*

***Abstract*—These instructions give you guidelines for preparing papers for IEEE Transactions and Journals. Use this document as a template if you are using Microsoft *Word* 6.0 or later. Otherwise, use this document as an instruction set. The electronic file of your paper will be formatted further at IEEE. Paper titles should be written in uppercase and lowercase letters, not all uppercase. Avoid writing long formulas with subscripts in the title; short formulas that identify the elements are fine (e.g., "Nd–Fe–B"). Do not write "(Invited)" in the title. Full names of authors are preferred in the author field, but are not required. Put a space between authors' initials. Define all symbols used in the abstract. Do not cite references in the abstract. Do not delete the blank line immediately above the abstract; it sets the footnote at the bottom of this column.**

***Index Terms*—Enter key words or phrases in alphabetical order, separated by commas. For a list of suggested keywords, send a blank e-mail to keywords@ieee.org or visit** http://www.ieee.org/organizations/pubs/ani_prod/keywrd98.txt

I. INTRODUCTION

THIS document is a template for Microsoft *Word* versions 6.0 or later. If you are reading a paper or PDF version of this document, please download the electronic file, TRANS-JOUR.DOC, from the IEEE Web site at http://www.ieee.org/web/publications/authors/transjnl/index.html so you can use it to prepare your manuscript. If you would prefer to use LATEX, download IEEE's LATEX style and sample files from the same Web page. Use these LATEX files for formatting, but please follow the instructions in TRANS-JOUR.DOC or TRANS-JOUR.PDF.

If your paper is intended for a conference, please contact your conference editor concerning acceptable word processor formats for your particular conference.

This paragraph of the first footnote will contain the date on which you submitted your paper for review. It will also contain support information, including sponsor and financial support acknowledgment. For example, "This work was supported in part by the U.S. Department of Commerce under Grant BS123456".

The next few paragraphs should contain the authors' current affiliations, including current address and e-mail. For example, F. A. Author is with the National Institute of Standards and Technology, Boulder, CO 80305 USA (e-mail: author@ boulder.nist.gov).

S. B. Author, Jr., was with Rice University, Houston, TX 77005 USA. He is now with the Department of Physics, Colorado State University, Fort Collins, CO 80523 USA (e-mail: author@lamar.colostate.edu).

T. C. Author is with the Electrical Engineering Department, University of Colorado, Boulder, CO 80309 USA, on leave from the National Research Institute for Metals, Tsukuba, Japan (e-mail: author@nrim.go.jp).

II. GUIDELINES FOR MANUSCRIPT PREPARATION

When you open TRANS-JOUR.DOC, select "Page Layout" from the "View" menu in the menu bar (View | Page Layout), (these instructions assume MS 6.0. Some versions may have alternate ways to access the same functionalities noted here). Then, type over sections of TRANS-JOUR.DOC or cut and paste from another document and use markup styles. The pull-down style menu is at the left of the Formatting Toolbar at the top of your *Word* window (for example, the style at this point in the document is "Text"). Highlight a section that you want to designate with a certain style, then select the appropriate name on the style menu. The style will adjust your fonts and line spacing. Do not change the font sizes or line spacing to squeeze more text into a limited number of pages. Use italics for emphasis; do not underline.

To insert images in *Word*, position the cursor at the insertion point and either use Insert | Picture | From File or copy the image to the Windows clipboard and then Edit | Paste Special | Picture (with "float over text" unchecked).

IEEE will do the final formatting of your paper. If your paper is intended for a conference, please observe the conference page limits.

A. Abbreviations and Acronyms

Define abbreviations and acronyms the first time they are used in the text, even after they have already been defined in the abstract. Abbreviations such as IEEE, SI, ac, and dc do not have to be defined. Abbreviations that incorporate periods should not have spaces: write "C.N.R.S.," not "C. N. R. S." Do not use abbreviations in the title unless they are unavoidable (for example, "IEEE" in the title of this article).

B. Other Recommendations

Use one space after periods and colons. Hyphenate complex modifiers: "zero-field-cooled magnetization." Avoid dangling participles, such as, "Using (1), the potential was calculated." [It is not clear who or what used (1).] Write instead, "The potential was calculated by using (1)," or "Using (1), we calculated the potential."

Use a zero before decimal points: "0.25," not ".25." Use "cm^3," not "cc." Indicate sample dimensions as "0.1 cm × 0.2 cm," not "0.1 × 0.2 cm^2." The abbreviation for "seconds" is "s," not "sec." Use "Wb/m^2" or "webers per square meter," not "webers/m^2." When expressing a range of values, write "7 to 9" or "7-9," not "7~9."

IEEE Standard for SRS

1-**Introduction**

- 1.1-Purpose
- 1.2-Scope/Intended Audience
- 1.3-Definition, Acronyms and Abbreviation
- 1.4-References / Contact Information / SRS team member
- 1.5-Overview

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

2-**Overall Description**

- 2.1-Product Perspective
- 2.2-Product Functions
- 2.3-User Characteristics
- 2.4-General Constraints
- 2.5-Assumptions and dependencies

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

3-**Specific Requirement**

- 3.1-External interface requirements(user/hardware/software interface)
- 3.2-Functional requirements
- 3.3-Performance requirements
- 3.4-Design constraints(Standards compliance/hardware limitations)
- 3.5-Logical database requirements
- 3.6-Software system attributes(Reliability, availability, Security, Maintainability)

KNOWLEDGEGATE

4-**Change Management process**

5-**Document approval**

- 5.1-Tables, diagrams, and flowcharts
- 5.2-Appendices
- 5.3-Index

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Requirement Review

- Before finalising the requirement one more review specially by a third party, who a master in the industry is advisable, to have a fresh look over the system, and can mentions any points if missed by team.

KNOWLEDGEGATE

Software Quality Assurance (SQA)

- **Definition**: SQA is a process ensuring software products meet quality standards and requirements.
- **Objective**: Prevent defects, improve quality, and ensure customer satisfaction.
- **Process**: Implements quality control and management activities throughout development.
- **Techniques**: Uses code reviews, inspections, audits, and testing to evaluate software quality.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Standards**: Adheres to international standards like ISO 9001, IEEE 730, and CMMI.
- **Quality Attributes**: Focuses on reliability, maintainability, usability, efficiency, and functionality.
- **Continuous Improvement**: Monitors processes and products to identify and implement improvements.
- **Metrics**: Employs metrics like defect density and code coverage to evaluate quality.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Training**: Emphasizes skill development for developers and testers to meet quality standards.

- **Documentation**: Requires proper documentation for transparency and traceability.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Verification/White Box

- Verification ensures that the software product is designed and developed according to the specified requirements and standards. “Are we building the product right”

- Process of Ensuring that the product meets its design specifications

- Focuses on static analysis techniques. Checks for consistency, completeness, and correctness
  - Code reviews
  - Static analysis tools
  - Inspection of requirements, design, and code documentation

KNOWLEDGEGATE

Validation/Black Box

- Validation ensures that the software product meets the end-user requirements and is fit for its intended purpose, “Are we Building the right product”. Ensures that the product meets its design specifications

- Focuses on static analysis techniques. Checks for consistency, completeness, and correctness
  - Code reviews
  - Static analysis tools
  - Inspection of requirements, design, and code documentation

KNOWLEDGEGATE

Software Quality Factors

- The various factors, which influence the software, are termed as software factors. They can be broadly divided into two categories.
  - The **first category** of the factors is of those that can be measured directly such as the number of logical errors.
  - **Second category** clubs those factors which can be measured only indirectly. For example, maintainability but each of the factors is to be measured to check for the content and the quality control.

KNOWLEDGEGATE

- Several models of software quality factors and their categorization have been suggested over the years. The classic model of software quality factors, suggested by McCall in 1977). The 11 factors are grouped into three categories .

  - **Product operation factors** – Correctness, Reliability, Efficiency, Integrity, Usability.

  - **Product revision factors** – Maintainability, Flexibility, Testability.

  - **Product transition factors** – Portability, Reusability, Interoperability.

[KNOWLEDGEGATE]

http://www.knowledgegate.in/GATE

ISO 9000 Models in Software Engineering

- ISO 9000 is a series of international standards related to quality management and assurance. In the context of software engineering, these standards provide guidelines for implementing effective processes and ensuring high-quality software products.

KNOWLEDGEGATE

- **Key Components of ISO 9000**
  - **Quality Management System (QMS):**
    - Defines policies and objectives for quality, Documentation of procedures and processes, Monitoring, measurement, and analysis of processes, Improvement opportunities and corrective actions.
  - **Management Responsibility:**
    - Top management commitment to quality Establishment of a quality policy Ensuring adequate resources for QMS implementation Reviewing QMS performance regularly
  - **Resource Management:**
    - Provision of necessary resources (human, infrastructure, work environment) Competence and training of personnel Infrastructure maintenance and improvement
  - **Product Realization:**
    - Requirements determination and communication Product design and development Verification, validation, and testing Release, delivery, and post-delivery support
  - **Measurement, Analysis, and Improvement:**
    - Monitoring and measurement of processes and products Internal audits to ensure compliance Corrective and preventive actions Continual improvement of the QMS

[KNOWLEDGEGATE]

1. **ISO 9000 Principles**
    1. Customer focus
    2. Leadership
    3. Process approach
    4. Continual improvement

2. **ISO 9001:2015 - Quality Management Systems**
    1. Applicable to any organization size and industry
    2. Focus on customer satisfaction, regulatory compliance, and improvement
    3. Requirements cover context, leadership, planning, support, operation, evaluation, and improvement

3. **ISO/IEC/IEEE 90003:2018 - Software Engineering**
    1. Guidance for applying ISO 9001:2015 to software engineering
    2. Covers software development, supply, acquisition, operation, and maintenance
    3. Addresses both product and process quality aspects
    4. Applicable to various software development methodologies

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Benefits of ISO 9000 in Software Engineering

- Improved customer satisfaction
- Enhanced process efficiency and effectiveness
- Reduced risk of software failures and defects
- Increased marketability and competitiveness

KNOWLEDGEGATE

Implementation Steps for ISO 9000 in Software Engineering

- Obtain top management commitment
- Establish a quality management system
- Train and educate employees
- Document processes, procedures, and policies
- Monitor, measure, and analyze processes
- Implement improvements and corrective actions
- Conduct internal audits
- Seek third-party certification (ISO 9001)

KNOWLEDGEGATE

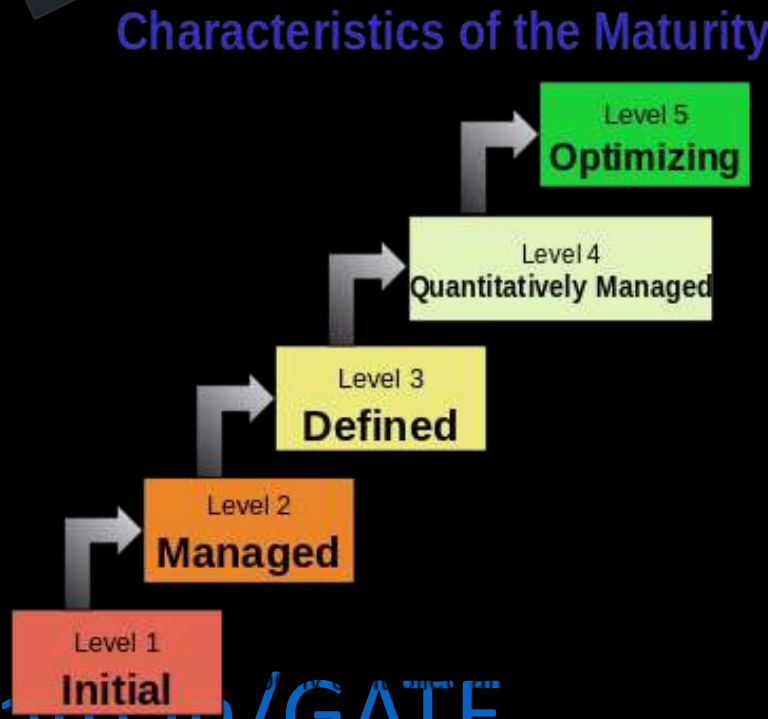
http://www.knowledgegate.in/GATE

CAPABILITY MATURATY MODEL

- CMM is a strategy for improving the software process, to generate quality software.
- The Capability Maturity Model (CMM) is a development model created after a study of data collected from organizations that contracted with the U.S. Department of Defence, who funded the research. The term "maturity" relates to the degree of formality and optimization of processes, from ad hoc practices, to formally defined steps, to managed result metrics, to active optimization of the processes.
- It is use to judge the maturity of s/w process an organization and to identify the key practise that are required to increase the maturity of theses process. There are five levels of the CMM

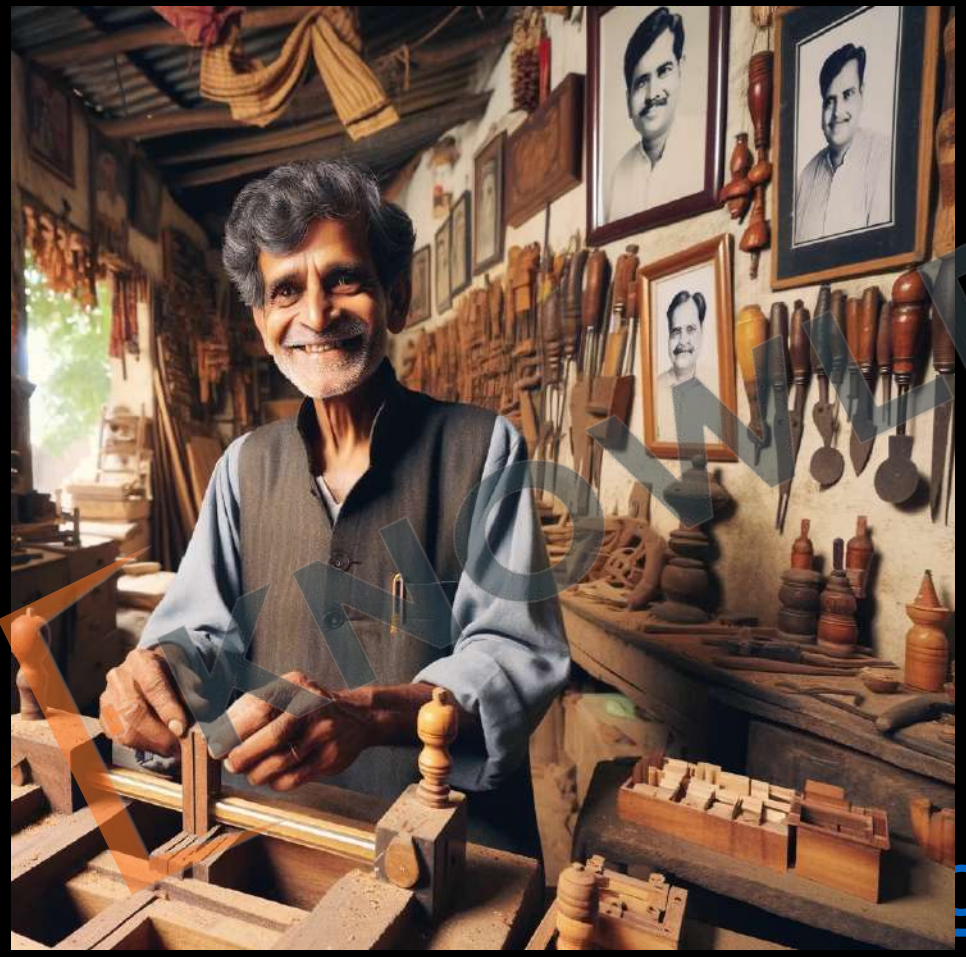
KNOWLEDGEGATE

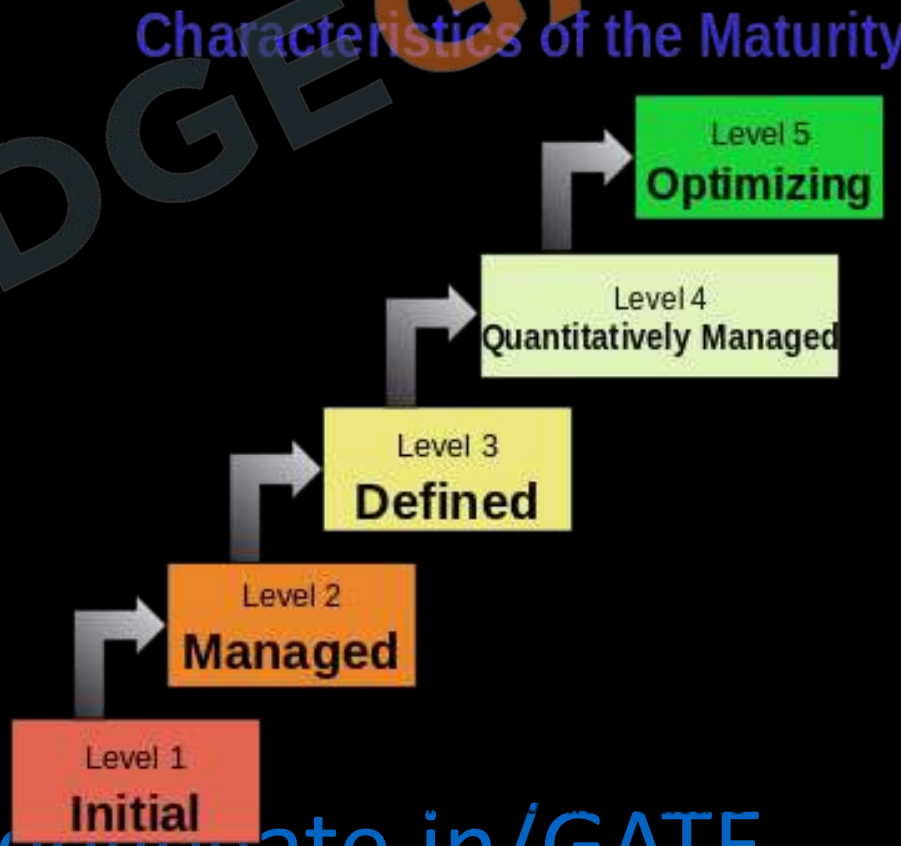
- **Initial (process unpredictable and poorly controlled)**
  - No engineering management, everything done on ad hoc basis
  - Software process is unpredictable with respect to time and cost
  - It depends on current staff, as staff change so does the process



KNOWLEDGEGATE
egate.in/GATE

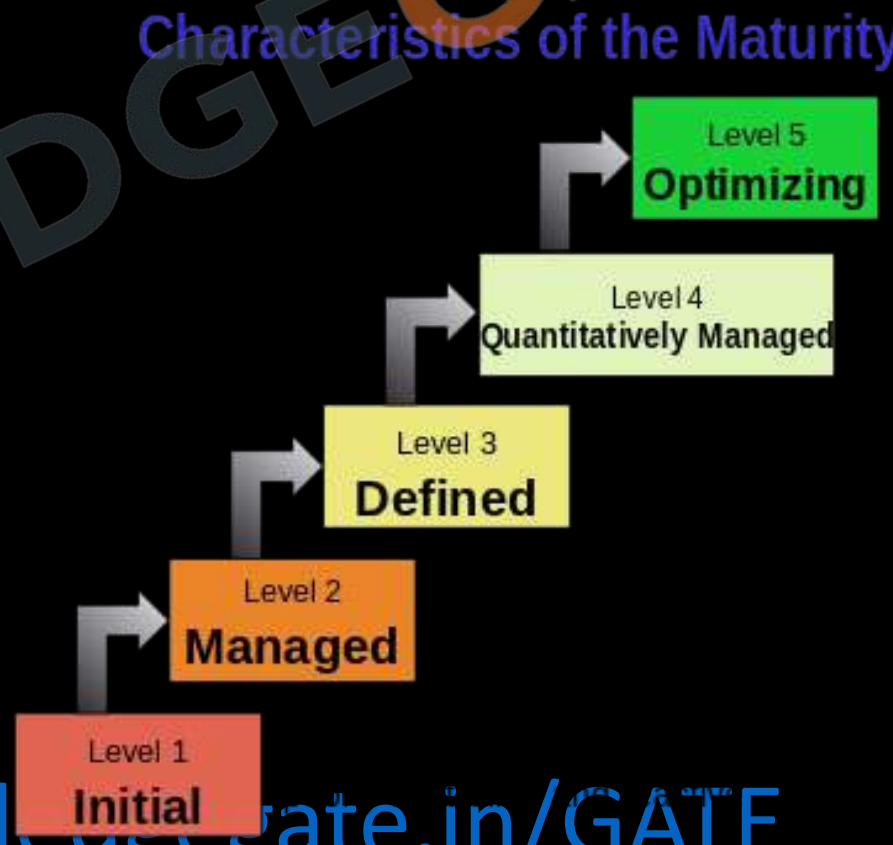
- **Repeatable (basic project management)**
  - Planning and managing of new projects are based on the experience with the similar projects
  - Realistic plans based on the performance based on the previous projects





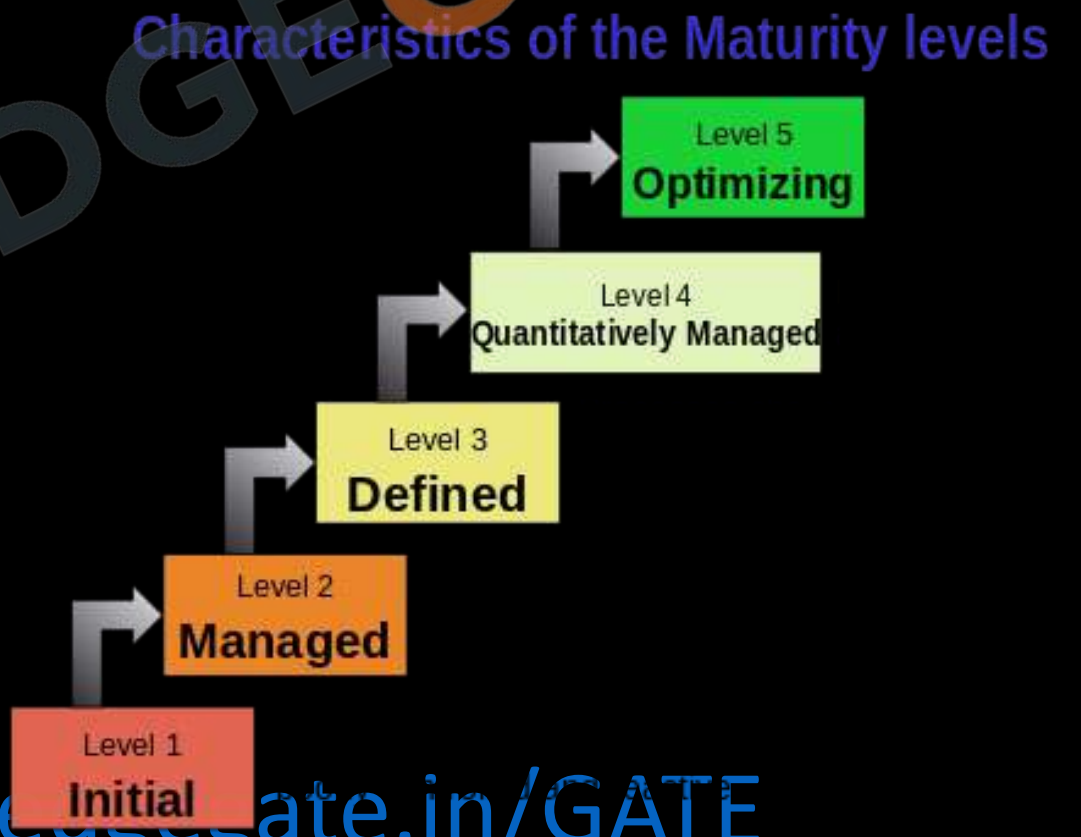
KNOWLEDGEGATE
owledgegate.in/GATE

- **Defined (process standardization)**
  - Process of developing and maintaining s/w across the organization is documented including engineering and management.
  - Training programs are implemented to ensure that the staff have skills and knowledge required
  - Risk management



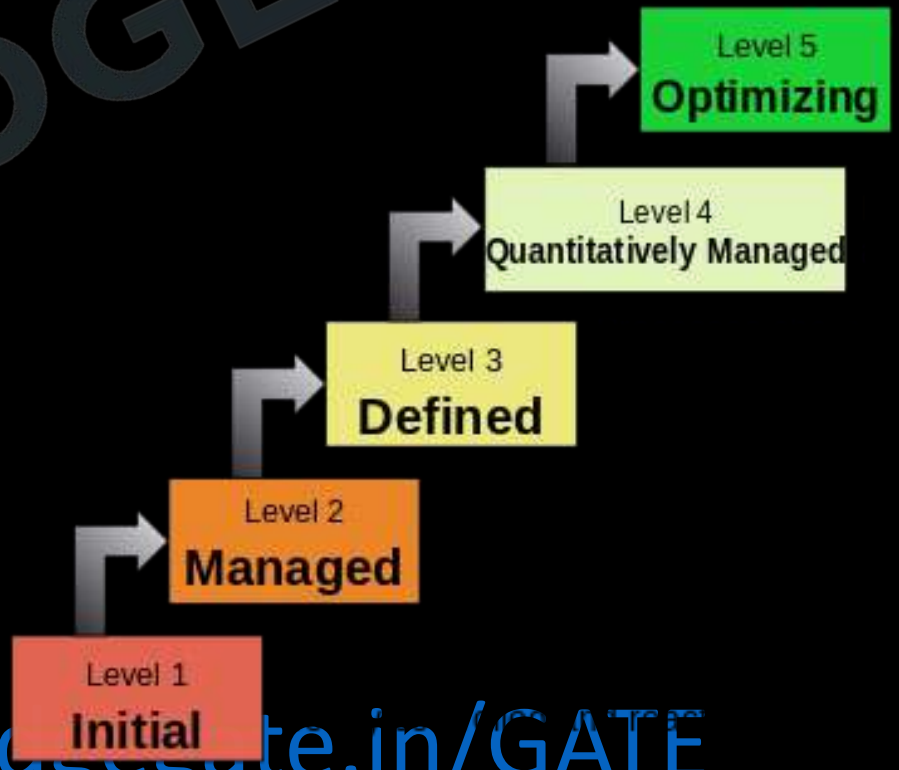
KNOWLEDGEGATE
knowledgegate.in/GATE

- **Managed (Quantitative measurement)**
  - Organization set quantitative goals for both product and process.
  - Here process is predictable both with respect to time and cost



owledgegate.in/GATE

- **Optimized (continuous process improvement)**
  - Here organization analysis defects to determine their causes and goals is to preventing the occurrence of defects
  - Here company continuously improve the process performance of their projects



KNOWLEDGE GATE
owledgegate.in/GATE

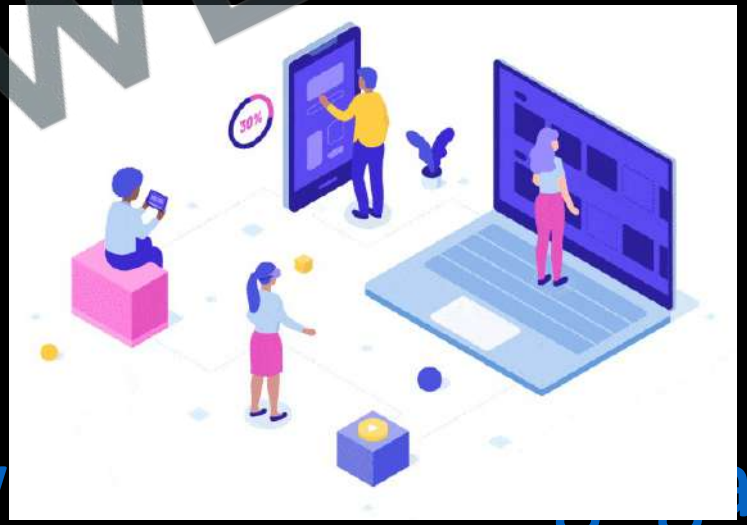
Unit-3(Software Design)

- **Basic Concept of Software Design**,
  - Architectural Design,
  - Low Level Design: Modularization, Design Structure Charts, Pseudo Codes, Flow Charts,
  - Coupling and Cohesion Measures,
- **Design Strategies**
  - Function Oriented Design,
  - Object Oriented Design,
  - Top-Down and Bottom-Up Design.
- Software Measurement and Metrics:
  - Various Size Oriented Measures: Halestead's Software Science,
  - Function Point (FP) Based Measures,
  - Cyclomatic Complexity Measures: Control Flow Graphs.

KNOWLEDGEGATE

Design

- In software design phase input is SRS document and output is SDD.
- Software designing is most creative process, where we actually decide how a problem will be solved.



KNOWLEDGEGATE

Characteristics of a Good SDD

- The SDD must contain all the requirement which were given in SRS.
- The design document must be readable, easy to understand and maintainable.
- It must describe a complete picture of data, functional and behavioural domain.

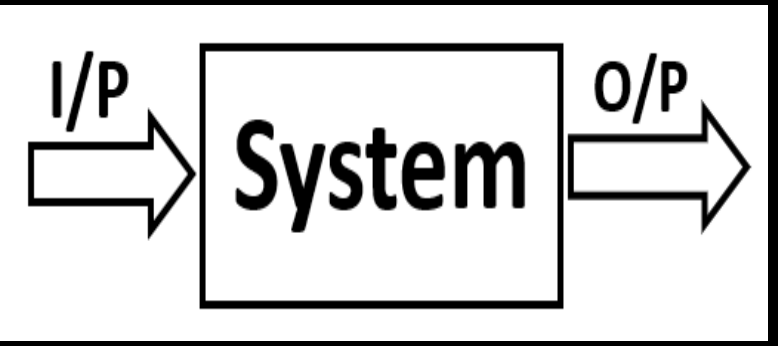
KNOWLEDGEGATE

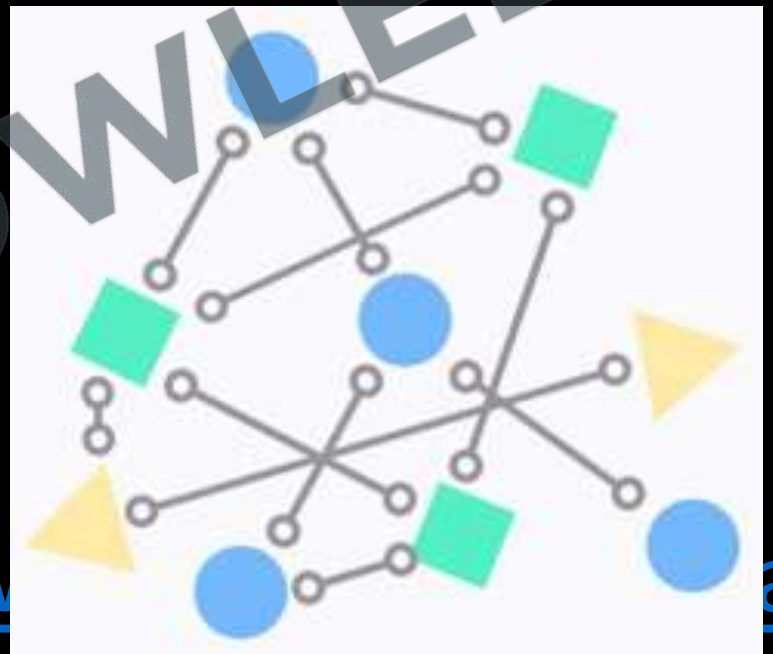
http://www.knowledgegate.in/GATE

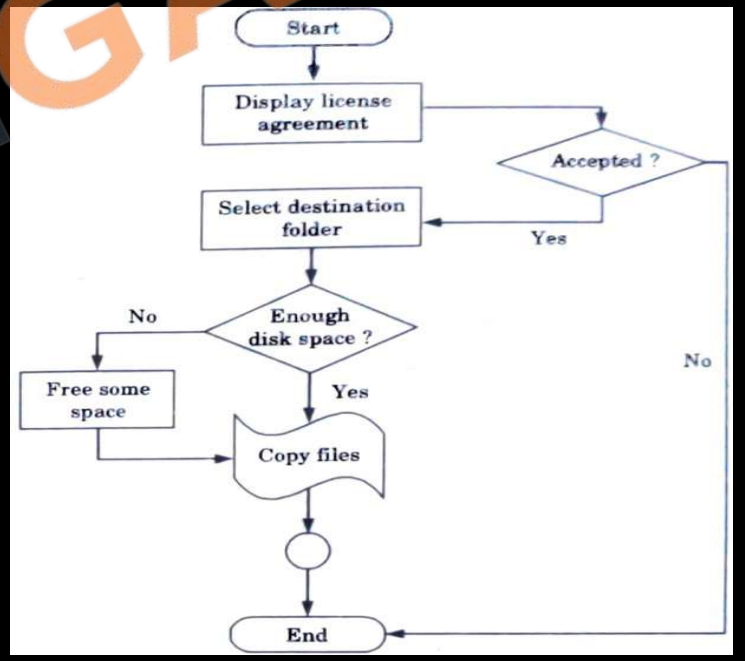
Steps of design

- Software designing is a 3-step process

Interface Design Architectural Design Details Design







KNOWLEDGEGATE

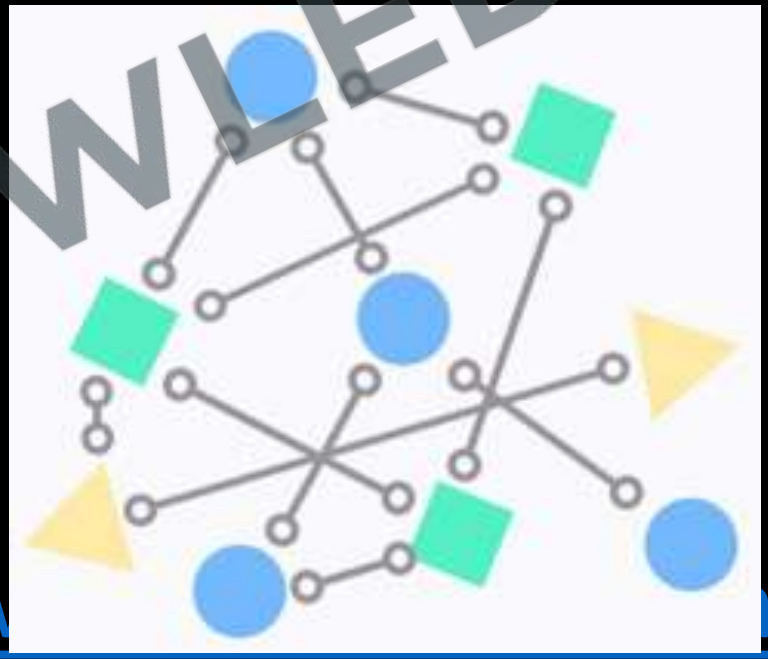
Interface design

- In this, we treat system as a whole and understand the relationship between the system & environment.
- Here we treat system as a black box and do not concentrate how a function will be implemented but we decide what is input & what should be the output according to user requirement or SRS

KNOWLEDGEGATE

Architectural design

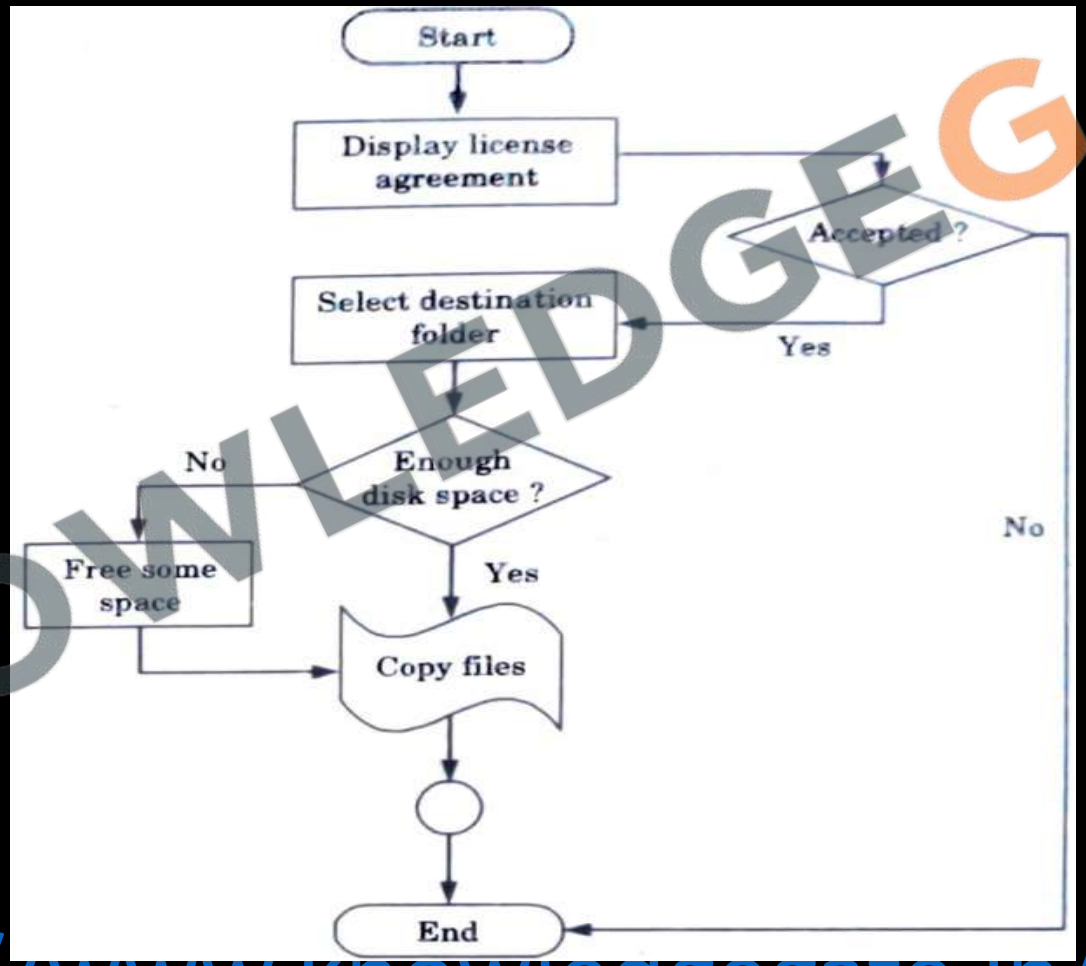
- In this we understand what are the major modules that must be implemented in the system and what are their responsibility and how they will communicate with each other.
- We do not give stress on individual modules but concentrate coupling and cohesion between the modules. Here we treat modules as black box.



KNOWLEDGEGATE

Detailed design/Low level design

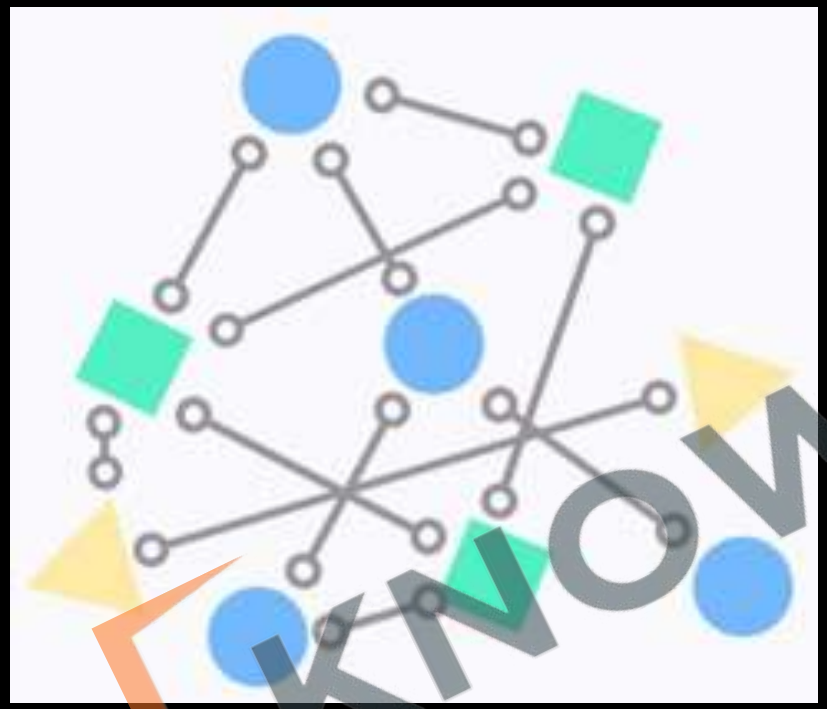
- In this, specification of internal elements of all modules their functions, their processing methods, data structure, algorithms, everything is defined properly.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Modularity

- In modular architecture, we understand that a system is composed of well defined conceptually simple and independent units interacting through a well-defined interface.



KNOWLEDGE GATE

http://www.kno

- Advantage of having modular architecture
  - Easy to understand and explain
  - It is easy to design and document
  - It is easy to code and test.
  - It is easy to maintain.

KNOWLEDGEGATE

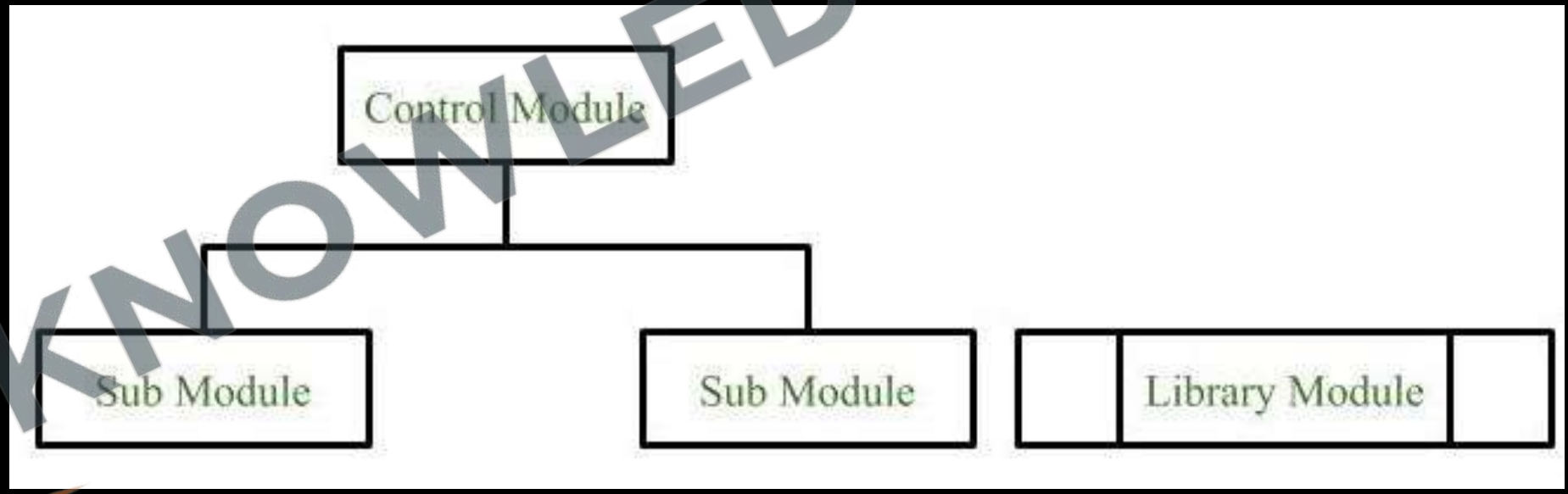
Design Structure Charts

- **Structure Chart** represent hierarchical structure of modules. It breaks down the entire system into lowest functional modules, describe functions and sub-functions of each module of a system to a greater detail.
- Structure Chart partitions the system into black boxes (functionality of the system is known to the users but inner details are unknown). Inputs are given to the black boxes and appropriate outputs are generated.

KNOWLEDGEGATE

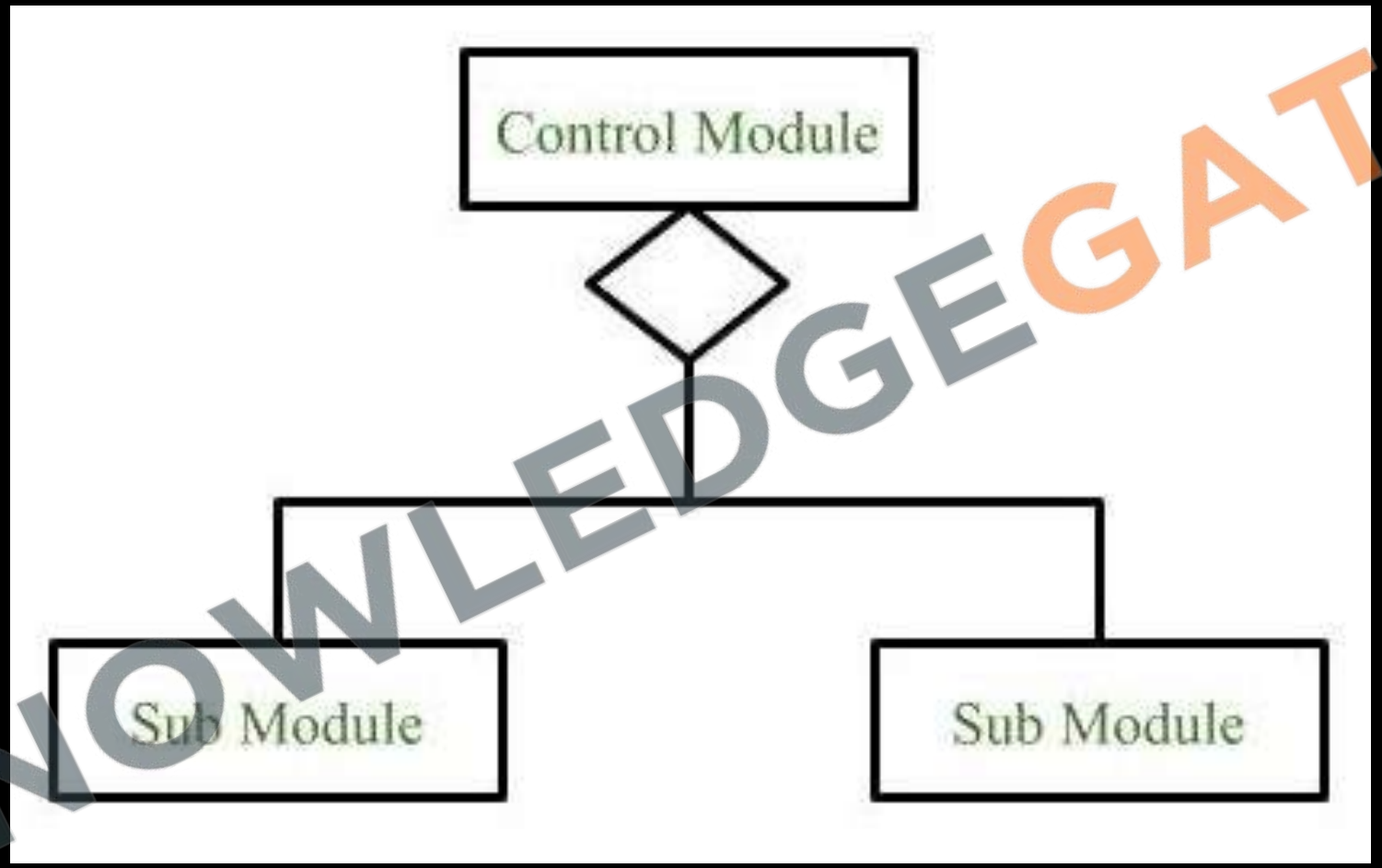
Symbols used in construction of structured chart

- **Module**: It represents the process or task of the system. It is of three types.
  - **Control Module**: A control module branches to more than one sub module.
  - **Sub Module**: Sub Module is a module which is the part (Child) of another module.
  - **Library Module**: Library Module are reusable and invokable from any module.



KNOWLEDGEGATE

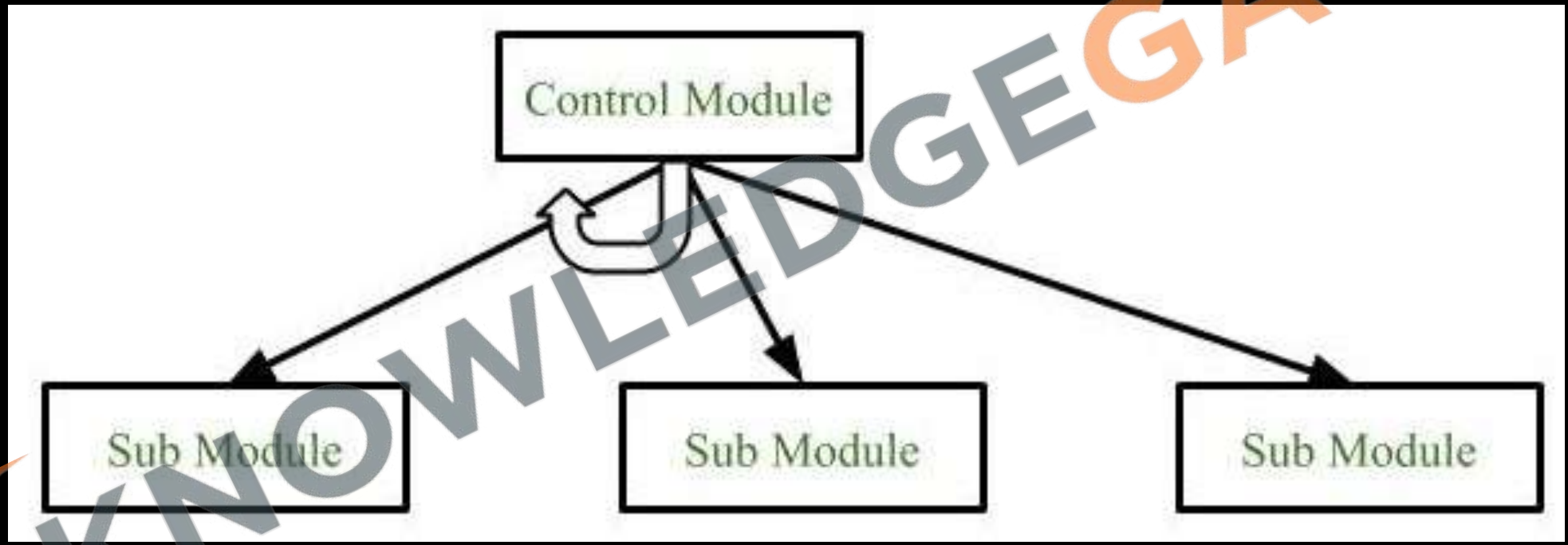
- **Conditional Call:** It represents that control module can select any of the sub module on the basis of some condition.



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

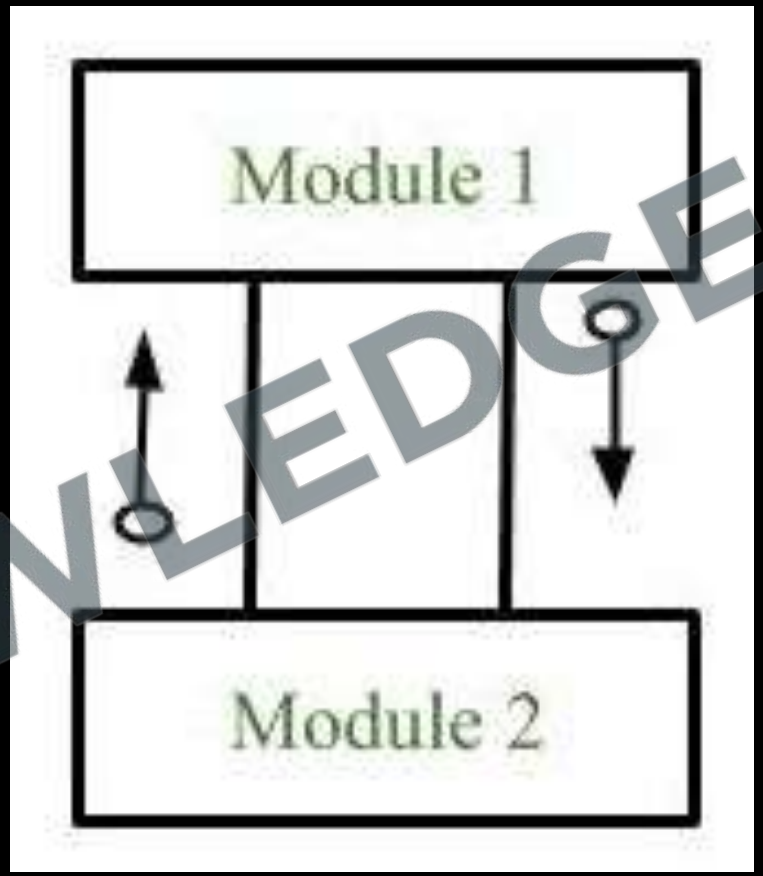
- **<u>Loop (Repetitive call of module)</u>:** It represents the repetitive execution of module by the sub module. A curved arrow represents loop in the module. All the sub modules cover by the loop repeat execution of module.



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

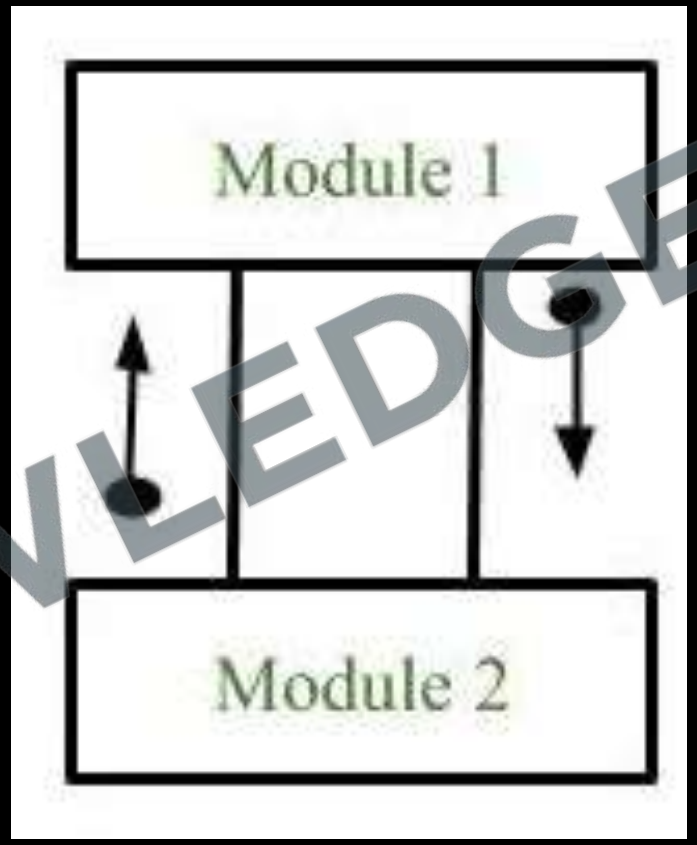
- **Data Flow:** It represents the flow of data between the modules. It is represented by directed arrow with empty circle at the end.



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Control Flow:** It represents the flow of control between the modules. It is represented by directed arrow with filled circle at the end.



KNOWLEDGEGATE

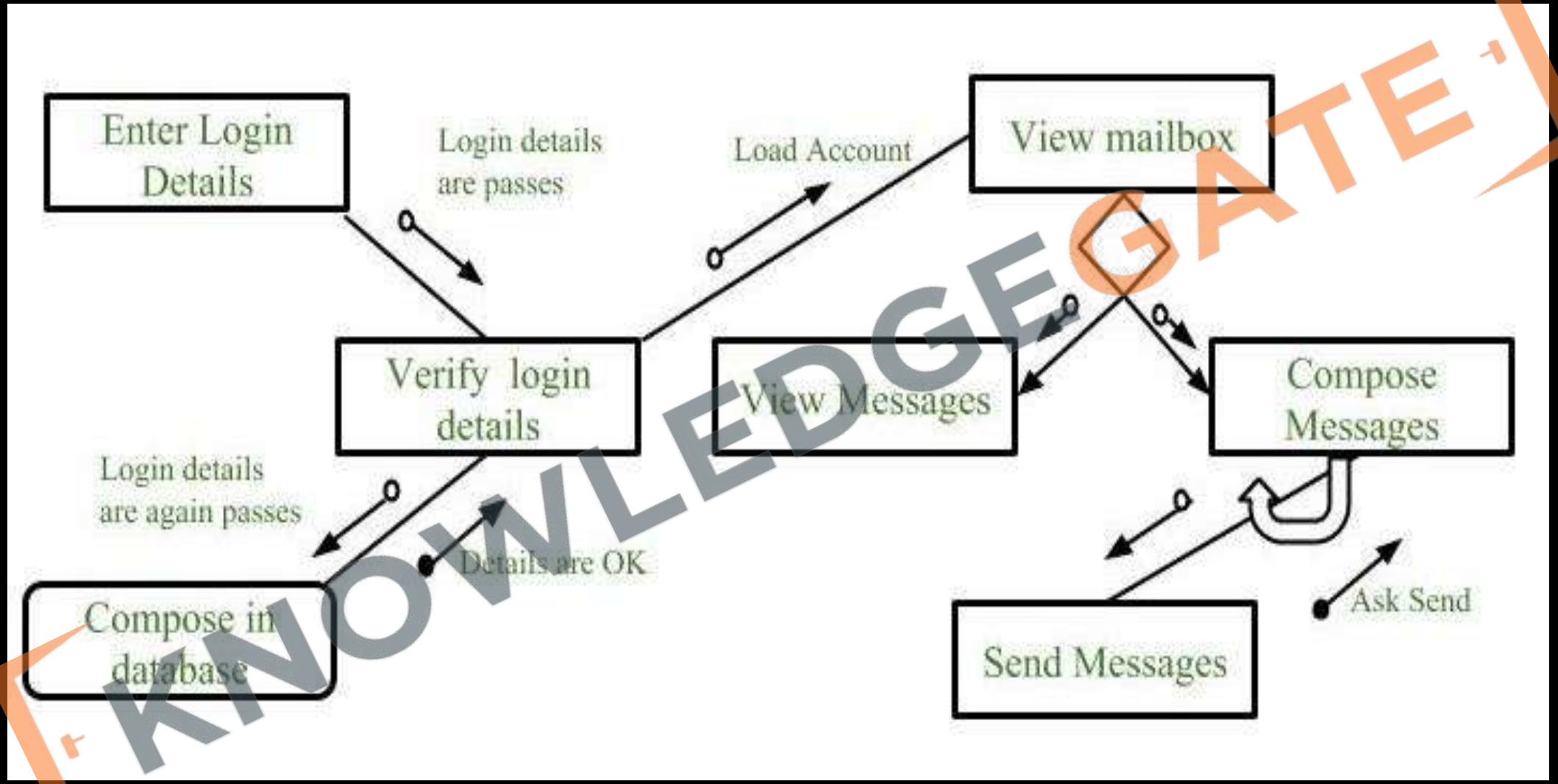
http://www.knowledgegate.in/GATE

- **Physical Storage:** Physical Storage is that where all the information are to be stored.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Structure chart for an Email server**



KNOWLEDGEGATE

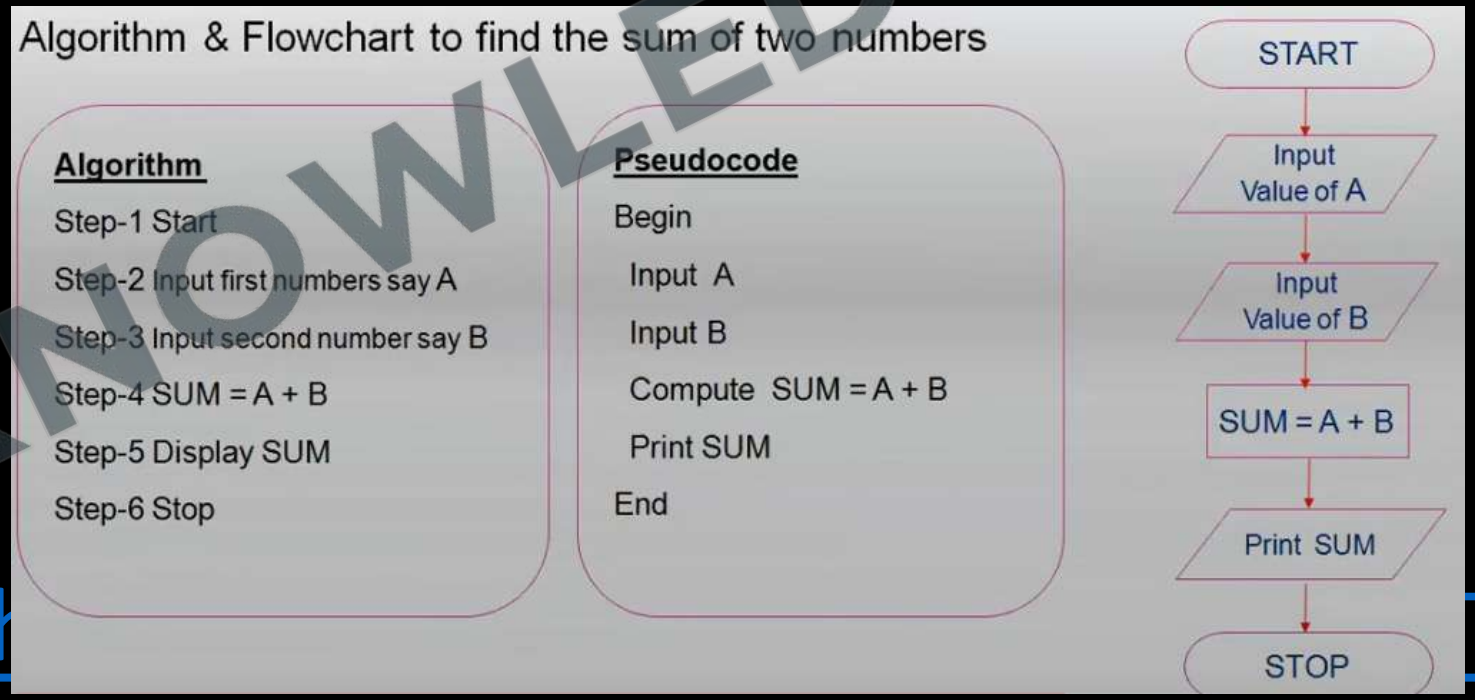
Benefits of Design Structure Charts in Software Engineering

- Improved understanding of system architecture
- Simplified communication between team members
- Easier identification of potential issues and dependencies
- Support for modular and maintainable software design

KNOWLEDGEGATE

Pseudocode

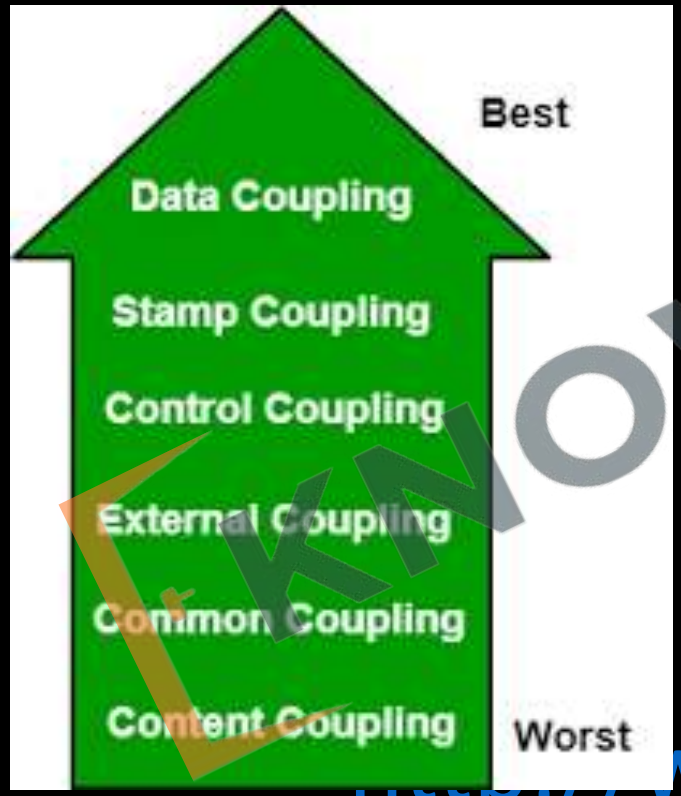
- Pseudocode is a simplified, informal representation of an algorithm or a program that uses a mix of natural language and programming constructs.
- It is used to illustrate the high-level structure and logic of an algorithm without the syntactic details of a specific programming language.
- Pseudocode helps developers plan and discuss algorithms, making it easier to understand and translate into actual code later in the software development process.

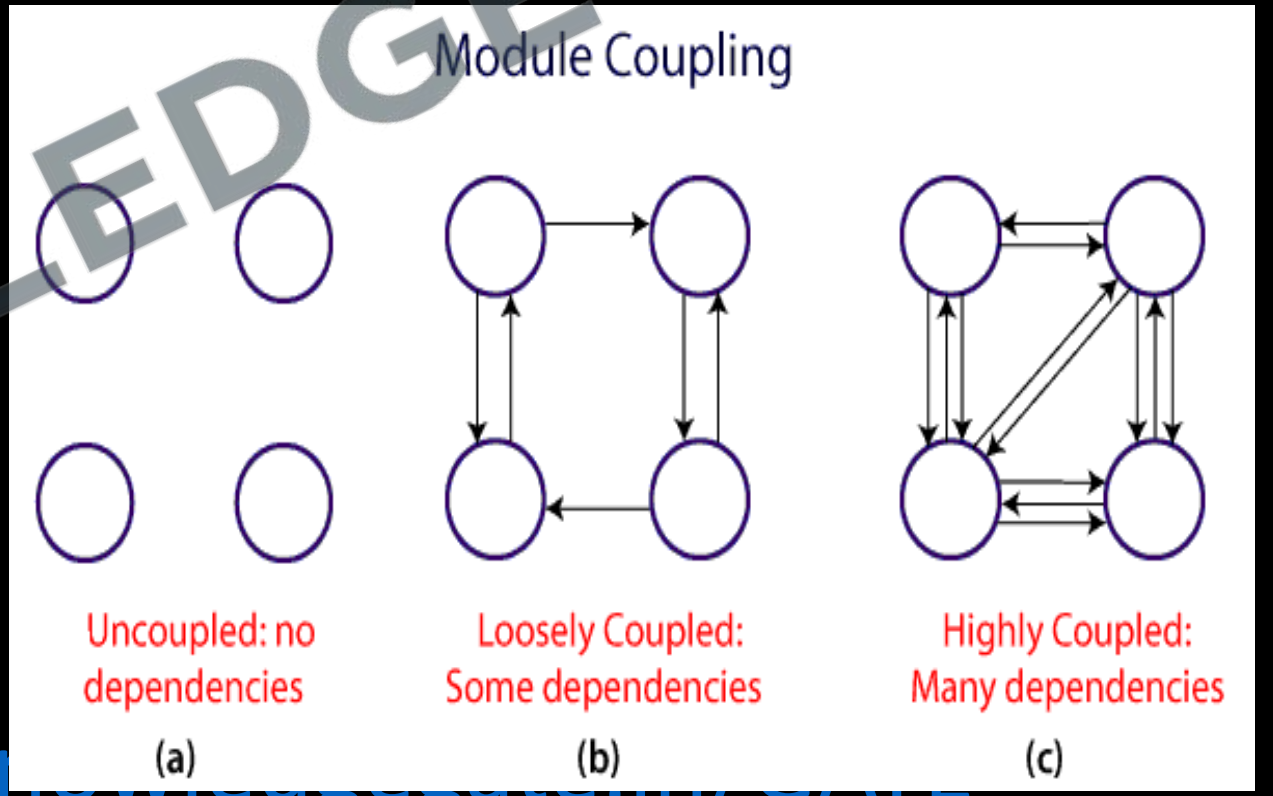


KNOWLEDGEGATE

Coupling and cohesion

- Coupling and cohesion are two parameters on which we can understand the quality of modularity in the design of a software
- **Coupling**
  - The measure of interdependence of one module over another module





KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Types of coupling**
  - **Data Coupling**: Here two modules communicate with each other only by-passing data. it is most desired type of coupling. e.g. addition by using call by value is an example of data coupling.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Stamp Coupling**: When two modules communicate with each other by passing of data structure is called stamp coupling. e.g. addition of two value but by call by reference.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Control Coupling**: The module is set to be controlled coupled if they communicate using control information with each other. e.g. dependence of two modules on each other because of flag.

KNOWLEDGEGATE

- **External Coupling**: in case of external coupling two modules on each other external to the logic of software (mostly because of hardware issue). e.g. data bus, CPU, main memory etc.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Common Coupling**: Two modules are set to be common coupled if they share some global data. e.g. synchronization issue between the processes.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

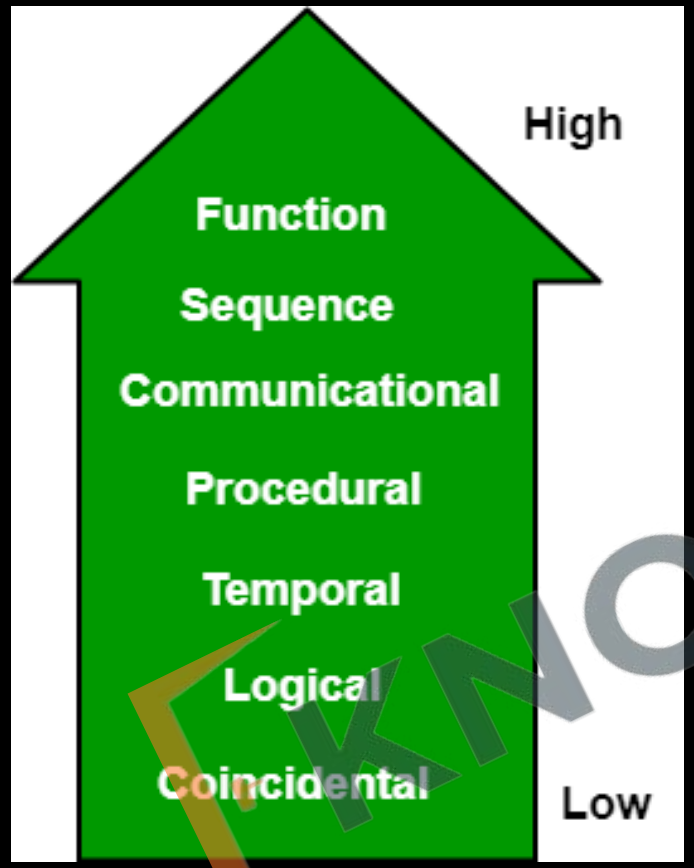
- **Content Coupling**: when one module is a part or context of another module then it is called content coupling it is worst type of coupling.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Cohesion

- We mean the measure of functions strength of the module.



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Types of cohesion**
  - **Coincidental Cohesion**: as the name suggest is the worst kind of cohesion where the only relationship between the functions which are present in a module is random or coincidental.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Logical Cohesion**: All the elements of a module perform similar or slightly similar operations, for e.g. if mouse, printer, scanner functions are written in the same module.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Temporal Cohesion**: In case of temporal cohesion functions are related by a fact that all the task must be executed in the same time span. so, there is a flow control between the functions. for e.g. two functions first which catch the exception and second which flash an error message.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Procedural Cohesion**: Here functions of a module are related to each other through a flow control i.e. they are part of an algorithm or procedure. for e.g. two functional first which check the file access permission and second which opens the file.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Communicational Cohesion**: in case of communicational cohesion different functions are operating on the same data structure. for e.g. push and pop.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Sequential Cohesion**: here sequence is very important and the functions are the part of sequence along with data dependency for e.g. function1 which provides the data and function2 which process the data.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Functional Cohesion**: in functional cohesion, different functions of a module cooperate with each other. perform a single function. for e.g. a module which control mouse must have different functions like accessing the location of the track ball, then decoding it on the screen and give an appropriate cursor movement

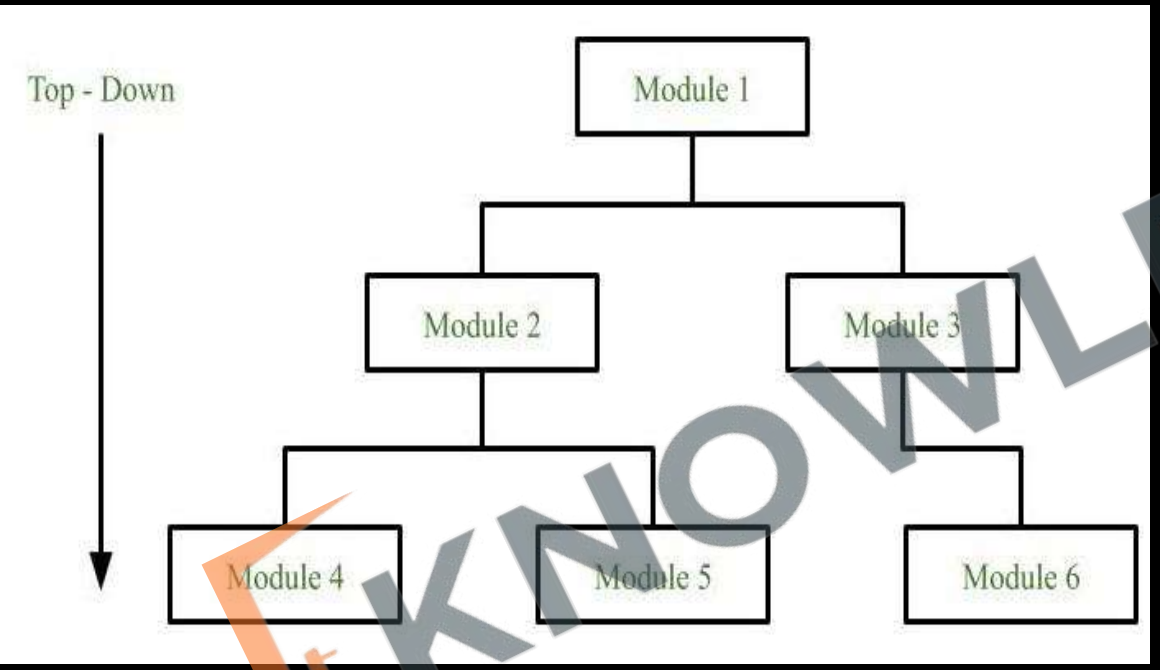
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

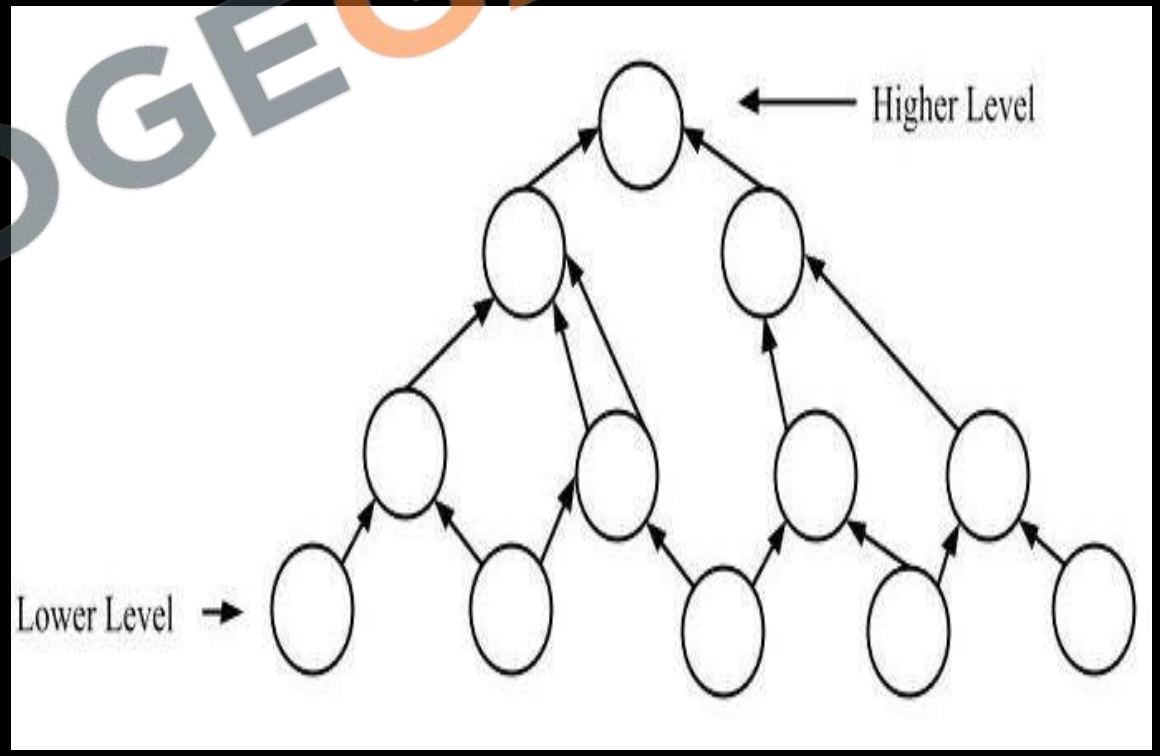
Design Approach

- There are two popular approach using which design is done

Top down approach

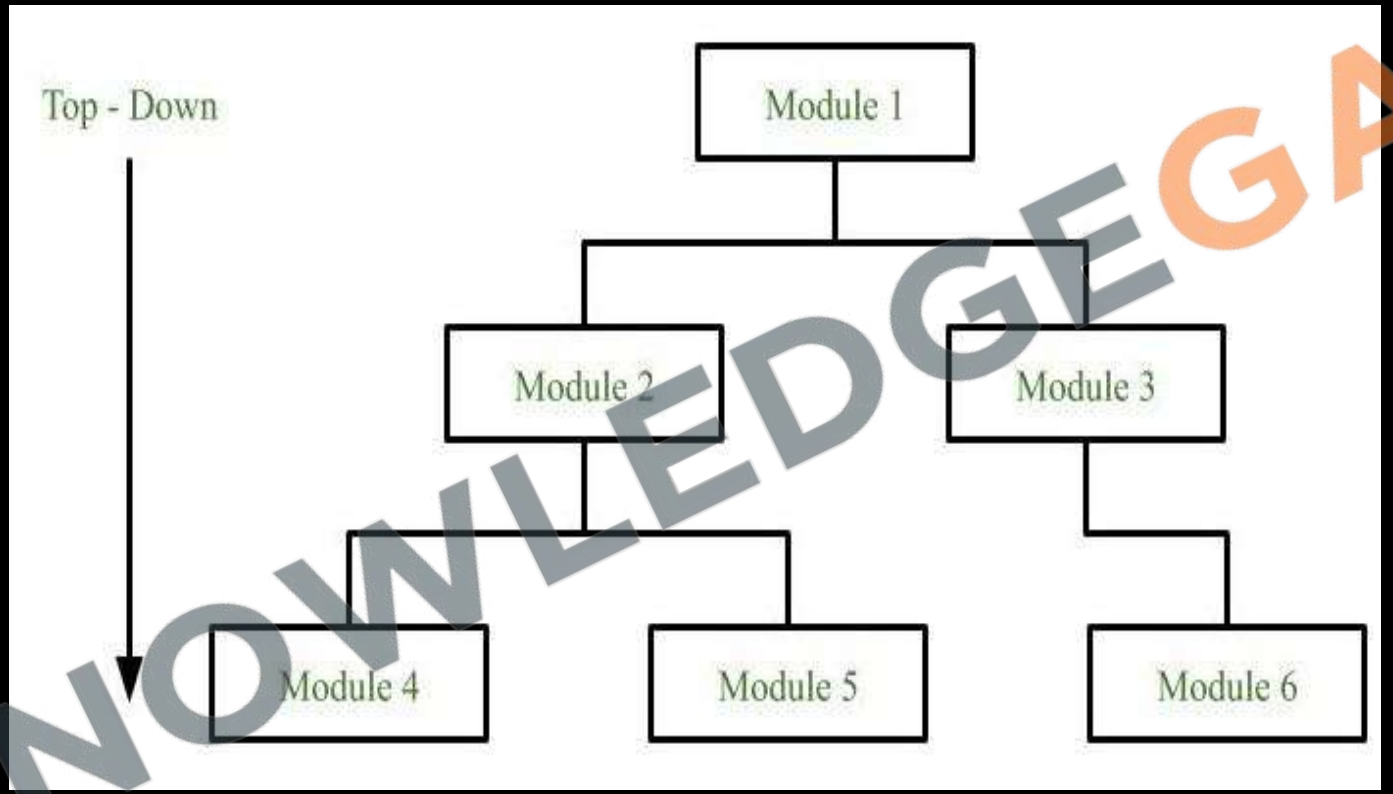


Bottom Up approach



KNOWLEDGEGATE

- In top down approach, the problem is divided into small no of programs according to user requirements and we repeat the process until the problem become so easy till we can solve it directly.



KNOWLEDGEGATE

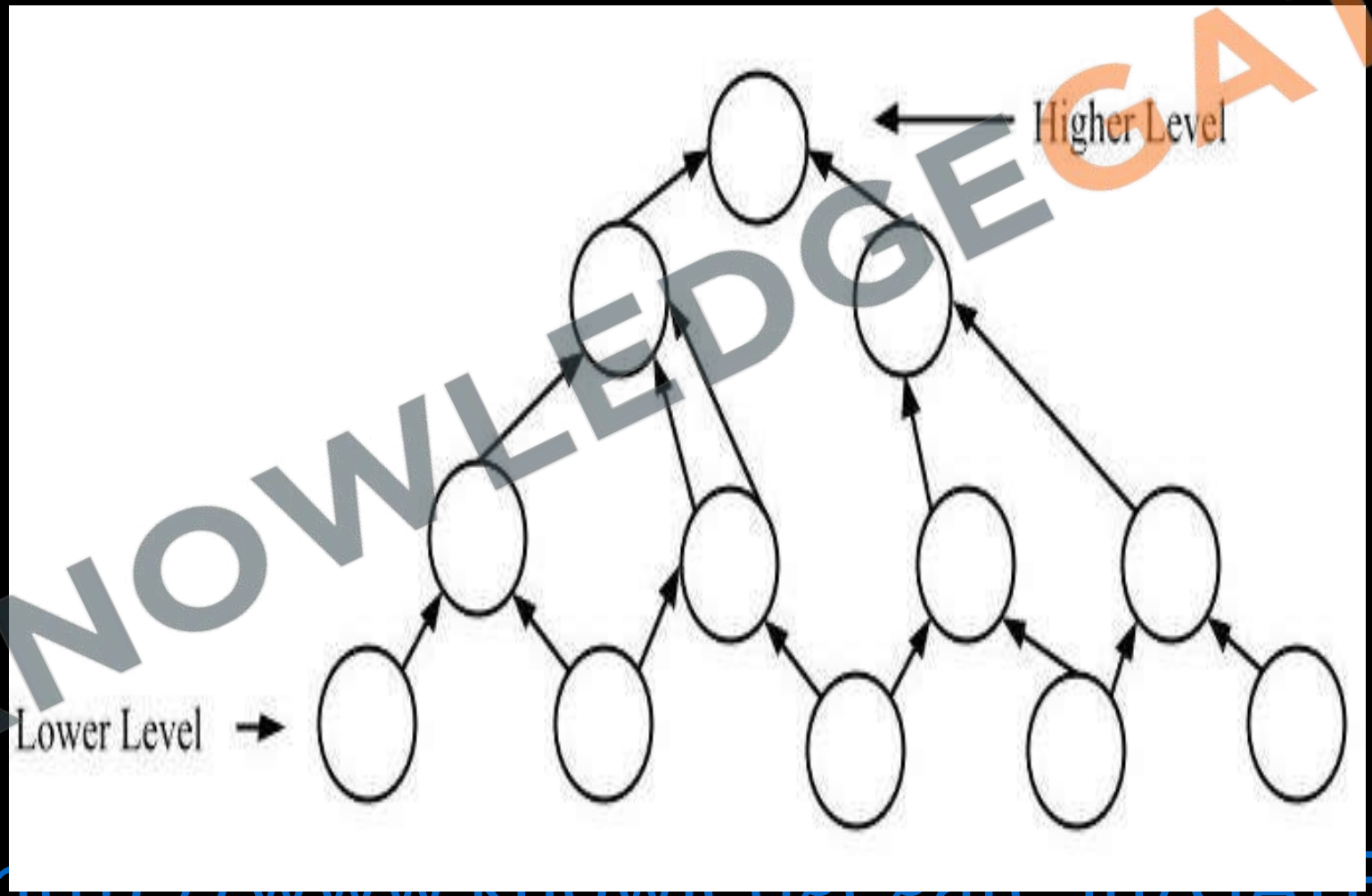
- **Advantage**
  - More Systematic.
  - Easy to understand and provide a modular architecture.
  - In correspondence with user requirement cost and time prediction is possible.
  - Should be used with small and medium size products.
  - Used mostly.

- **Disadvantage**
  - When a problem is very complex and very large then we can not understand the entire problem as a whole.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Bottom Up**: This approach is important as the s/w grows in complexity in a systematic fashion. if the problem is very difficult to understand instead of understanding the entire problem, we must solve some sub-problem and then should keep repeating the process until the entire problem is solved.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Advantage**
  - Should be used on large size projects.
  - Easy to use by designers as we work in incremental fashion.

- **Disadvantage**
  - it has complex architecture very difficult to understand and manage.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Estimation model

- Here we try to estimate about two things time and cost of development

- Majorly estimation can be divided into four types
  - Post estimation
  - Base estimation
  - Decomposition
  - Empirical model

KNOWLEDGEGATE

Post / Delayed estimation

- In case of a friendly partly and known technology, we do not estimate either the time or the cost, because we know that cost will support as in every situation.

KNOWLEDGE GATE

Base estimation

- In base estimation we predict the cost and time of the entire project based on the experience which we have gained from the previous projects.

KNOWLEDGEGATE

Decomposition based estimation

- It is used for large projects where decomposition of the problem into smaller problems is done, usually it is done on two bases.
- **Direct Estimation(White Box):** Size oriented metrices (KLOC)
- **Indirect Estimation(Black Box):** Function oriented metrices (FP)

KNOWLEDGEGATE

- Effort = Size / Productivity
- Productivity = Size / Effort
- Size = Effort X Productivity
- Cost = Effort X Pay
- Duration = Effort / Team Size
- Team Size = Effort / Duration
- Effort = Duration X Team Size

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider a banking application which required 4 modules as follows:
User Interface = 5400 LOC
Database = 8500 LOC
Server = 7300 LOC
Client = 5500 LOC
Find the total expenditure of company if productivity of the person is 870 Loc per month.
Consider the salary of each developer is 1500 US dollar per month.

KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

Q Consider a application which contain 3 modules M_1, M_2, M_3 having following size

M_1 – 14.4 KLOC

M_2 – 21.5 KLOC

M_3 – 8.4 KLOC

if the cost of writing 1KLOC is 50 dollars. Find the cost of the application. if the productivity of the developer is 4 KLOC per month. Find the effort, if the project is required to be completed in month then predict the team size.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Q Consider a application which if required to be developed by drdo contain 3 modules M_1, M_2, M_3 having following size

M_1 – 4600 LOC optimistic developer

M_2 – 6900 LOC most likely developer

M_3 – 8600 LOC pessimistic developer

What is the expected size of application

In general if the software development effort is 10 person-month, find the productivity.

if the development is required to be done in 2 months then find the team size?

$$\text{Size} = (S_{optimistic} + 4 \text{ X } S_{likely} + S_{pessimistic})/6$$

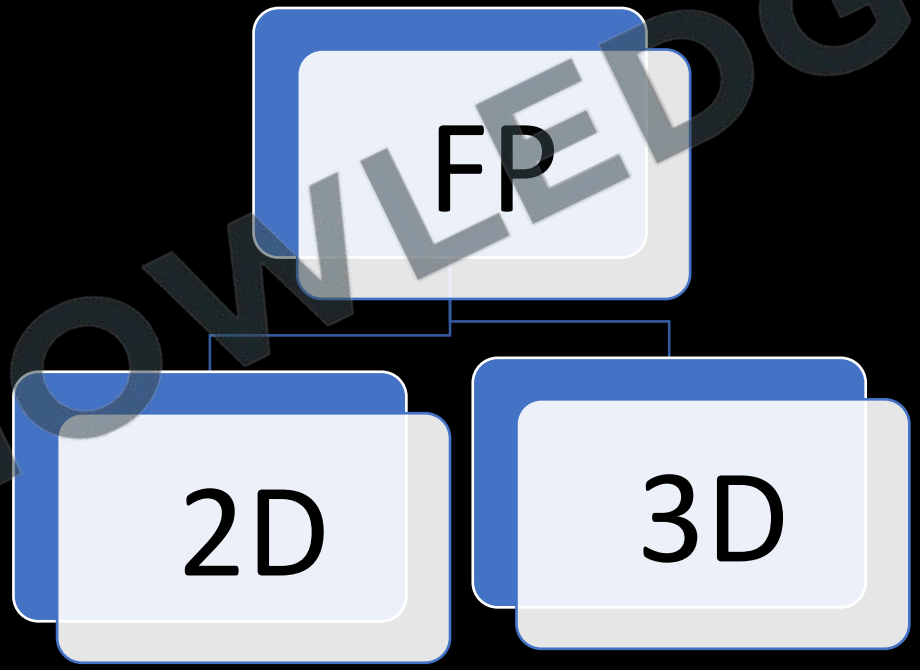
[KNOWLEDGEGATE]

http://www.knowledgegate.in/GATE

Indirect Estimation

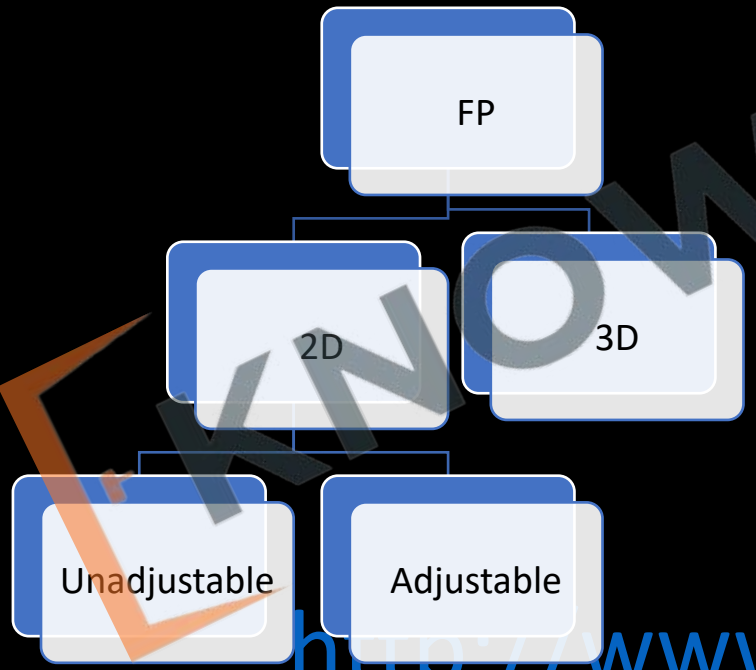
- Here we use functional points to predict the cost of development, it is a better technique in general compare to kloc because it considers the logical complexity of the product, as in general it is not necessary that the larger the project the more complex it will be code.

- (size oriented, function oriented, extended functions point matrices) here we say that the size of the software is directly dependent on the number and type of different functions it performs.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- 2D- in 2D FP we consider only information domain where we consider mainly five factors, as follows:
  - **No of inputs**: Each user data input is counted
  - **No of outputs**: Output refers to reports, screen and error messages
  - **No of inquiries**: The no of distinct interactive queries made by user which requires specific actions by the system
  - **No of files**: Each logical file. So either can be data structure or physical files
  - **No of external interfaces**: Data files on tapes, disk etc. and other interfaces that are used to transmit information to other systems are counted



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Function point calculation

Measurement parameters	Simple	Average	Complex
i/p	3	4	6
o/p	4	5	7
Inquires	3	4	6
Files	7	10	15
External interface	5	7	10

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

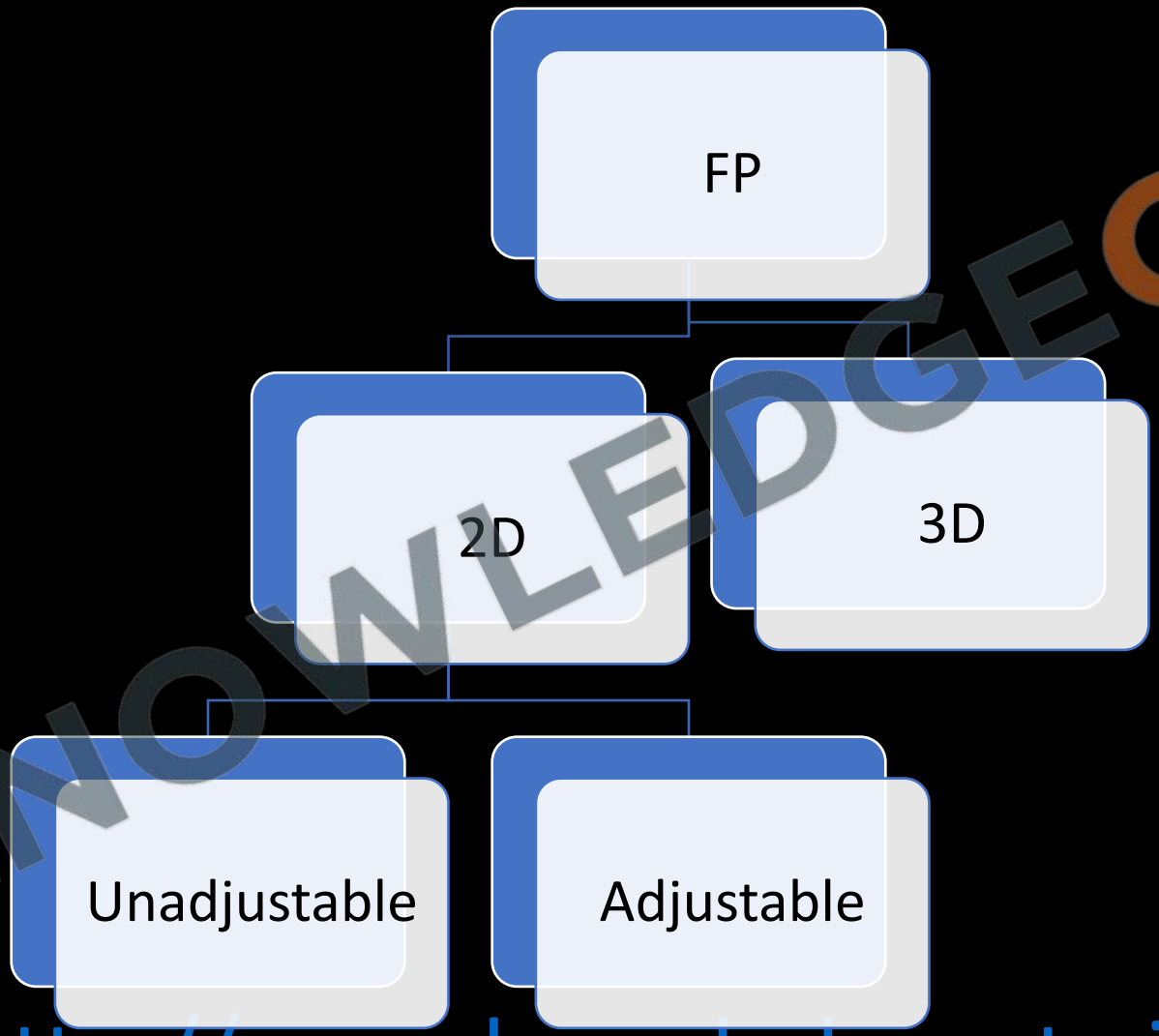
- In Unadjustable case
  - FP= total_count

- In Adjustable case
  - FP= total_count * EAF(error adjustable factor/Cost adjustment Factor)
  - EAF = 0.65 + 0.01*$\sum_{i=1}^{i=14} f_i$
  - In FP analysis 14 factors are indirectly involved to estimate the size of the software.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- 3D- In 3D FP we consider only information domain where we consider mainly five factors, and along with Function domain and behaviour domain.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Empirical models

- Empirical models are estimation models which uses empirically derived formulas for predicting based on LOC or FP.
- Different authors provides different mathematically derived formula based on either LOC or FP.
- Out of all COCOMO models in most versatile and universally accepted

KNOWLEDGEGATE

COCOMO(1981)

- COCOMO model (construction cost model) is the most widely used estimating technique. It is a regression-based model developed by barry boehm, he postulated that there are essentially three important classes of the software.

  - **Organic(application)**
  - **Semidetached(utility)**
  - **Embedded(system)**

KNOWLEDGEGATE

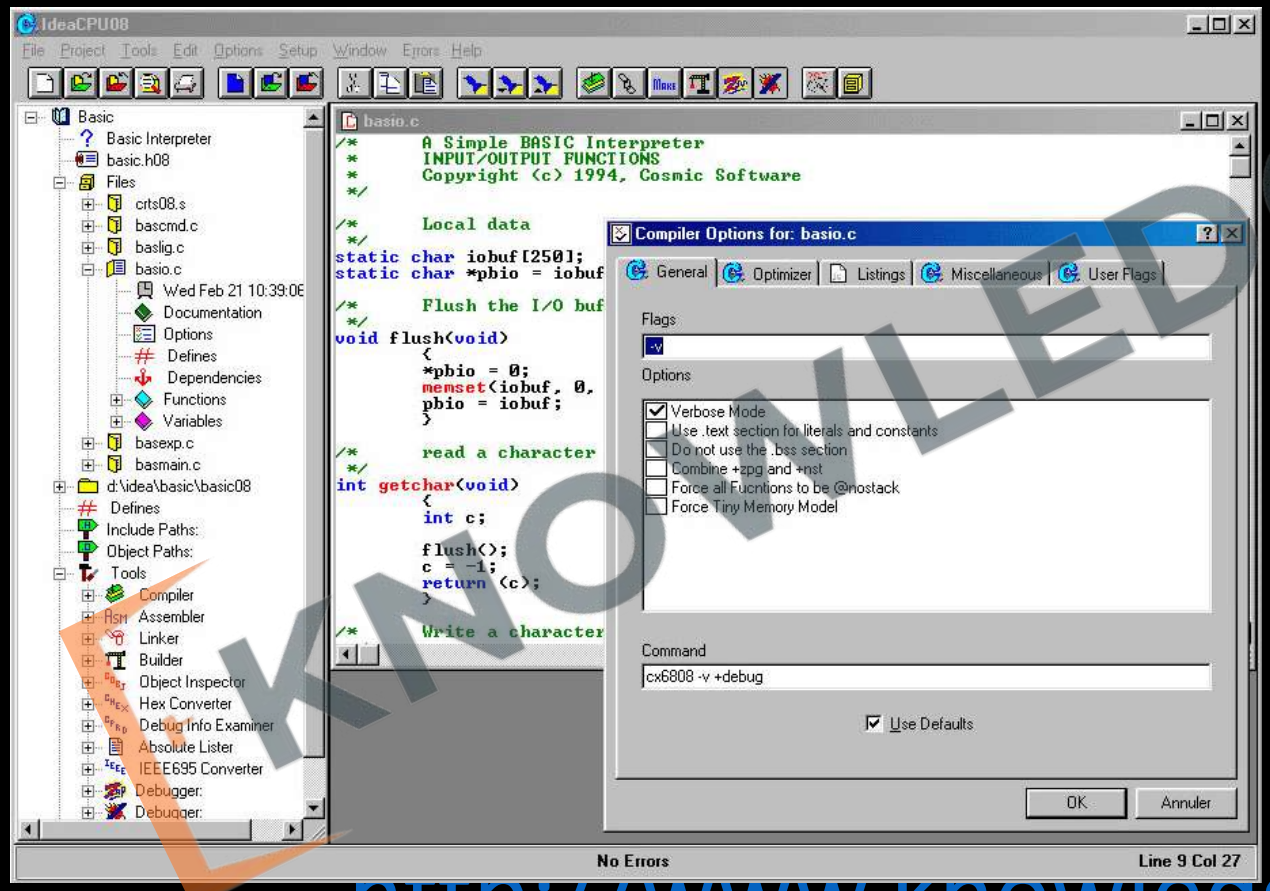
- In order to classify a product into any of the three proposed classes, we take into consideration the characteristics of the product as well as those of the development team.
- Data processing and scientific program are considered to be application programs. Complier, linkers etc are utility programs. Operating system and real time system programs are system programs.
- According to the brooks the relative level of the product complexity for the three categories of product are in the ratio 1:3:9 for application, utility and system programs relatively

KNOWLEDGEGATE

- **Organic**: Relatively small group work in a familiar environment to develop well understood application program, here little innovation is required, constraints and deadlines are few and the development environment is stable.

KNOWLEDGEGATE

- **Semidetached**: Project teams consist of a mixture of experienced and inexperienced staff. It is of medium size, some innovation is required, constraints and deadlines are moderate and the development environment is somewhat fluid.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Embedded**: the s/w is strongly coupled to complex h/w, such as air traffic control, ATM's or weapon systems. The project team is large, a great deal of innovation is required, constraints and deadlines are tight and the development environment consist of many complex interfaces, including those with h/w and with customer.

KNOWLEDGEGATE

Basic COCOMO

- A simple and static cost estimation model that calculates project effort and duration based on the size of the software, measured in thousands of lines of code (KLOC)
- Provides a rough order-of-magnitude estimate of project effort and duration
- Uses three modes: Organic, Semi-Detached, and Embedded

KNOWLEDGEGATE

Basic COCOMO

- $D_E = a_b (KLOC)^{bb}$
- $D_D = c_b (KLOC)^{db}$
- Team Size = D_E / D_D

Software Project	a_b	b_b	c_b	d_b
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

KNOWLEDGEGATE

Software Project	a_b	b_b	c_b	d_b
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

20KLOC	Organic	Semidetached	Embedded
Effort	55.75 PM	85.95 PM	131 PM
Duration	11.52 M	11.8 M	11.89 M
Team Size	5	7	11

- $D_E = a_b (KLOC)^{bb}$
- $D_D = c_b (KLOC)^{db}$
- Team Size = D_E / D_D

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Intermediate COCOMO

- **Definition**: An extension of the Basic COCOMO model that refines effort and duration estimates by considering additional project factors, such as product attributes, hardware constraints, and personnel/team attributes.
- Introduces 15 cost drivers to adjust the estimation, these drivers plays a major role in computation of effort estimation.
- Effort Adjustment Factor (EAF) accounts for the influence of cost drivers.

KNOWLEDGEGATE

Mode	a	b
Organic	3.2	1.05
Semi-detached	3.0	1.12
Embedded	2.8	1.20

20 KLOC / EAF=1.25	Organic	Semidetached	Embedded
Effort	92.9 PM	107.4 PM	127.4 PM

- D_E = a (KLOC)^b X EAF(Error Adjustment Factor)

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- D_E = a $(KLOC)^b$ X EAF(Error Adjustment Factor)

Mode	a	b
Organic	3.2	1.05
Semi-detached	3.0	1.12
Embedded	2.8	1.20

20 KLOC / EAF=1.25	Organic	Semidetached	Embedded
Effort	92.9 PM	107.4 PM	127.4 PM

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- D_E = a $(KLOC)^b$ X EAF(Error Adjustment Factor)

Mode	a	b
Organic	3.2	1.05
Semi-detached	3.0	1.12
Embedded	2.8	1.20

20 KLOC / EAF=1..675	Organic	Semidetached	Embedded
Effort	124 PM	116.7 PM	170.76 PM

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Detailed COCOMO

- **Definition**: The most comprehensive and accurate COCOMO model that divides a software project into multiple components or modules, and accounts for interactions between cost drivers and project phases.
- Estimates effort and duration for each component using the Intermediate COCOMO model, then sums them up.
- Considers software reuse, hardware constraints, and personnel/team attributes in estimation.

KNOWLEDGEGATE

Other Empirical models

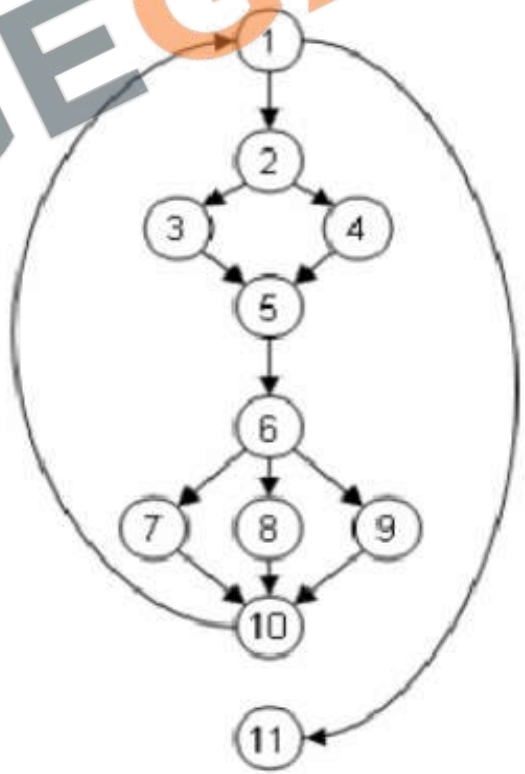
- $E = 5.2 \times (KLOC)^{0.91}$ Waltson & Felix Model
- $E = 5.2 + 0.73X(KLOC)^{1.16}$ Bailey-Basili Model
- $E = 3.2 \times (KLOC)^{1.05}$ Simple Bohem Model
- $E = 5.288 \times (KLOC)^{1.047}$ Doty Model

KNOWLEDGEGATE

Cyclomatic Complexity

- Cyclomatic complexity is a software metric that measures the complexity of a program's control flow by counting the number of linearly independent paths through the source code.

Node	Statement
(1)	while(x<100){
(2)	if (a[x] % 2 == 0) {
(3)	parity = 0;
	}
	else {
(4)	parity = 1;
(5)	}
(6)	switch(parity){
	case 0:
(7)	println("a[" + i + "] is even");
	case 1:
(8)	println("a[" + i + "] is odd");
	default:
(9)	println("Unexpected error");
	}
(10)	x++;
	}
(11)	p = true;



KNOWLEDGEGATE

Need:

- Assess code maintainability and readability
- Identify potential errors and areas of high risk
- Determine testing effort required
- Aid in refactoring efforts to simplify code

KNOWLEDGEGATE

Objective:

- Quantify the complexity of a program or function
- Facilitate better decision-making in software development
- Improve overall code quality and understandability

KNOWLEDGEGATE

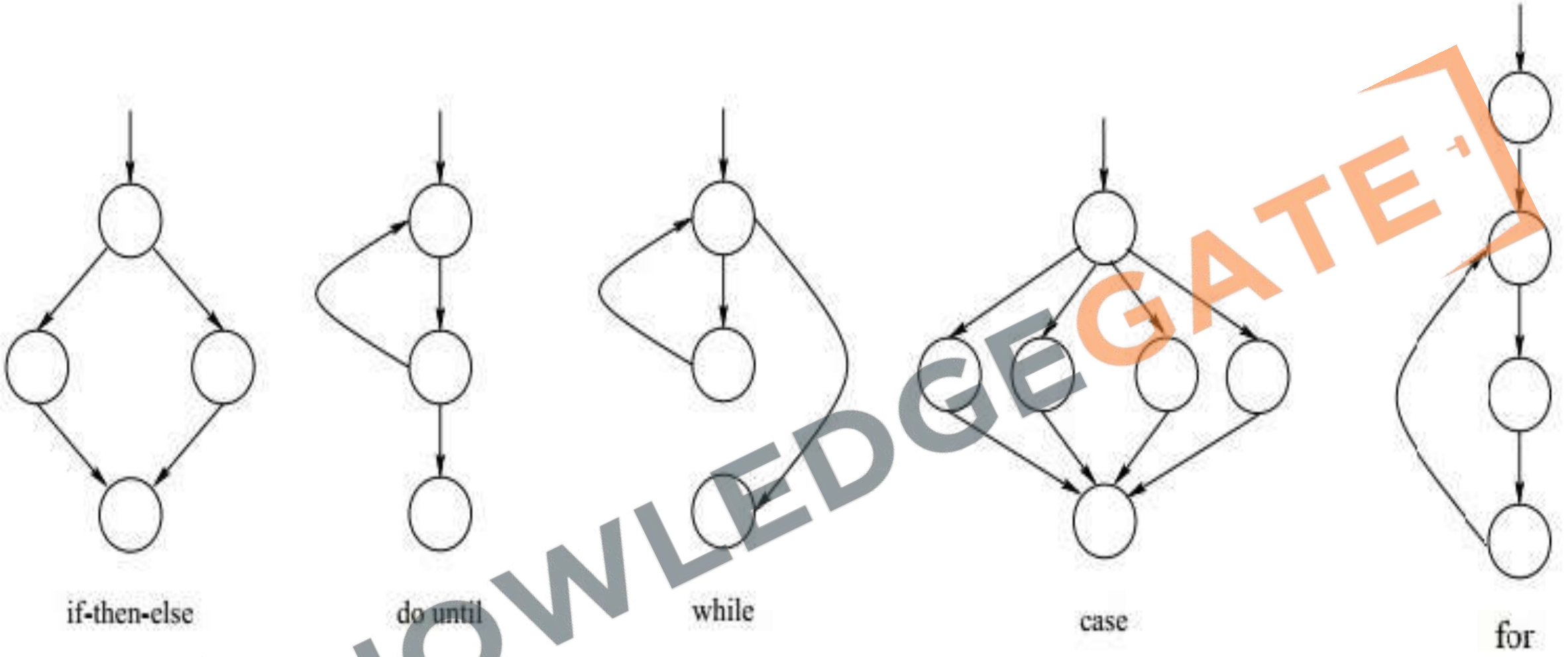
http://www.knowledgegate.in/GATE

Process:

- Identify the program's decision points (such as if, while, and for statements)
- Calculate the number of linearly independent paths by counting the decision points and adding one
- Interpret the resulting value:
  - Low value: lower complexity, easier to maintain and test
  - High value: higher complexity, harder to maintain and test, may require refactoring

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE



KNOWLEDGEGATE

- **Given a control flow graph G of a program, the cyclomatic complexity V(G) can be computed as:**

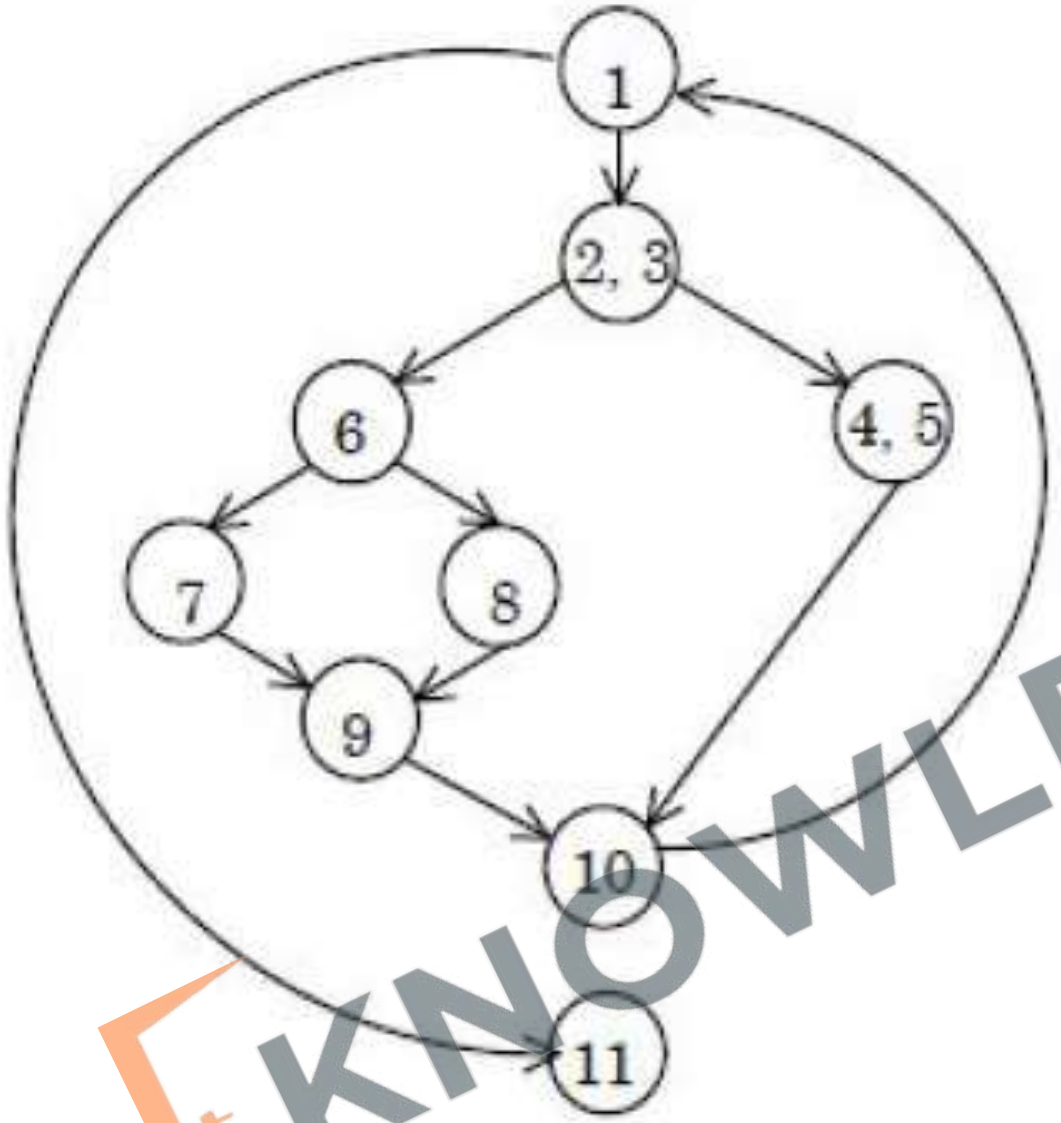
- **V(G) = E – N + 2**, where E is the number of edges and N is the total number of nodes

- **V(G) = V(G) = Total number of bounded areas + 1**
- Any region enclosed by nodes and edges can be called as a bounded area.

- The cyclomatic complexity of a program can also be easily computed by computing the number of decision statements of the program.

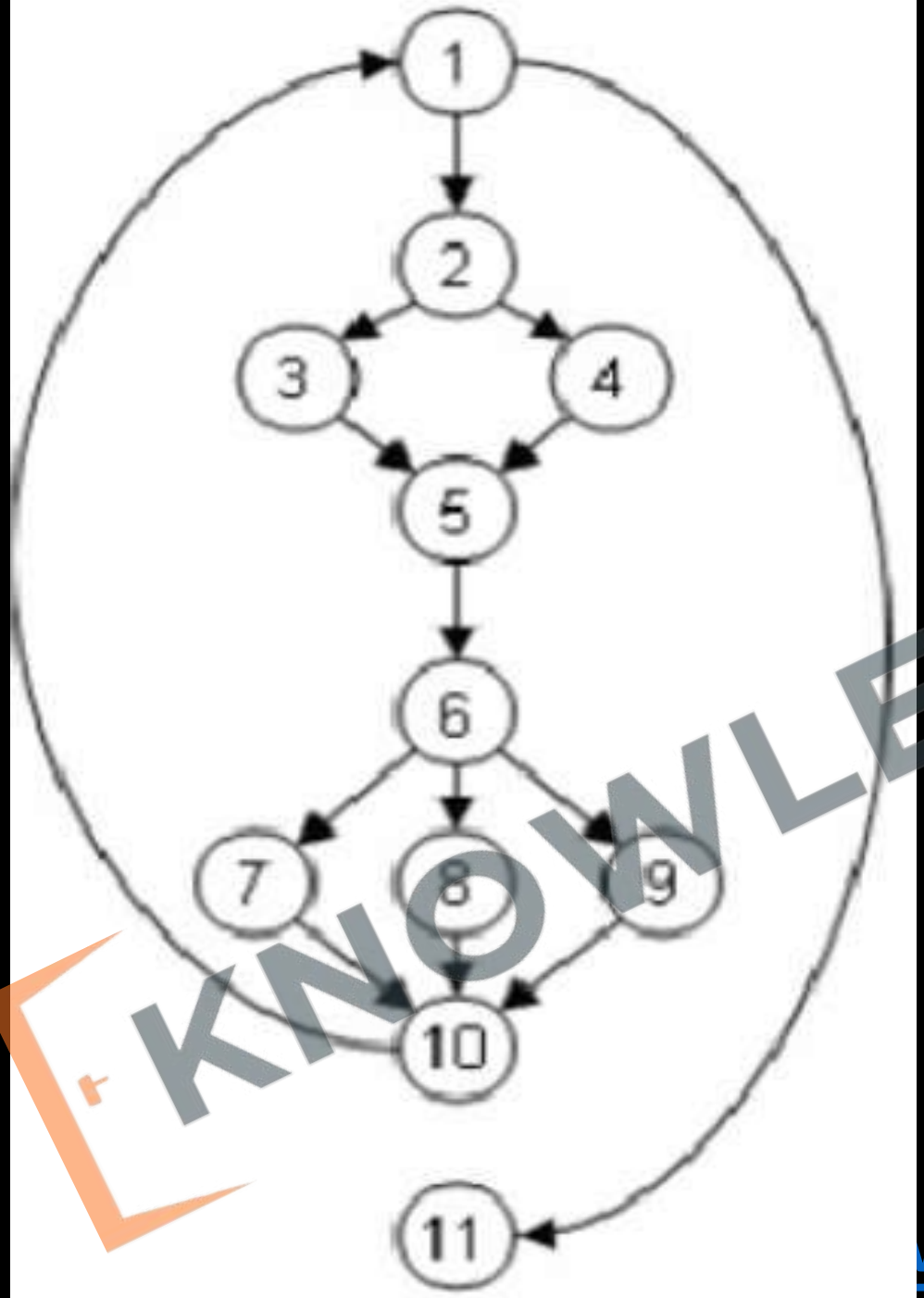
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE



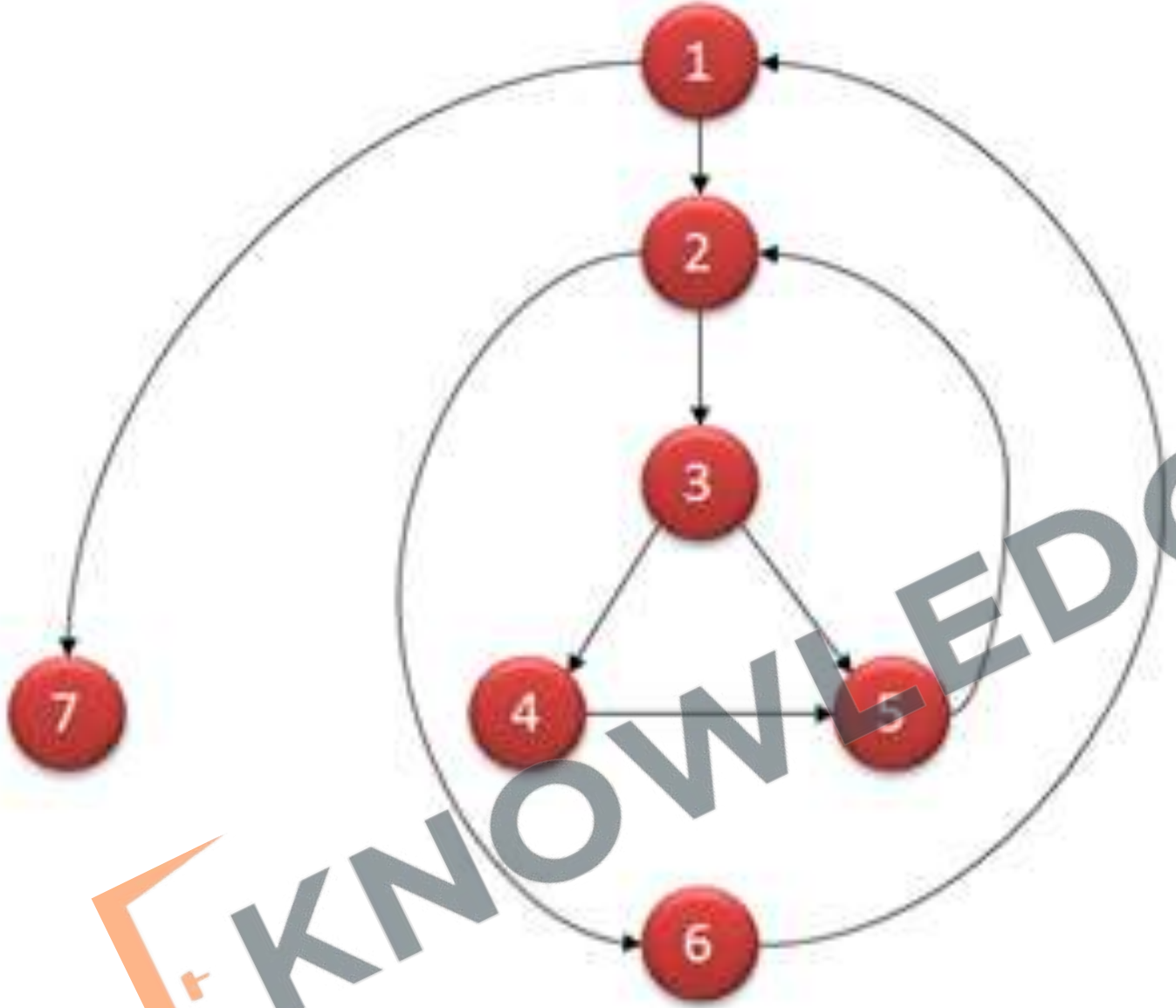
Flowgraph F

KNOWLEDGEGATE
edgegate.in/GATE



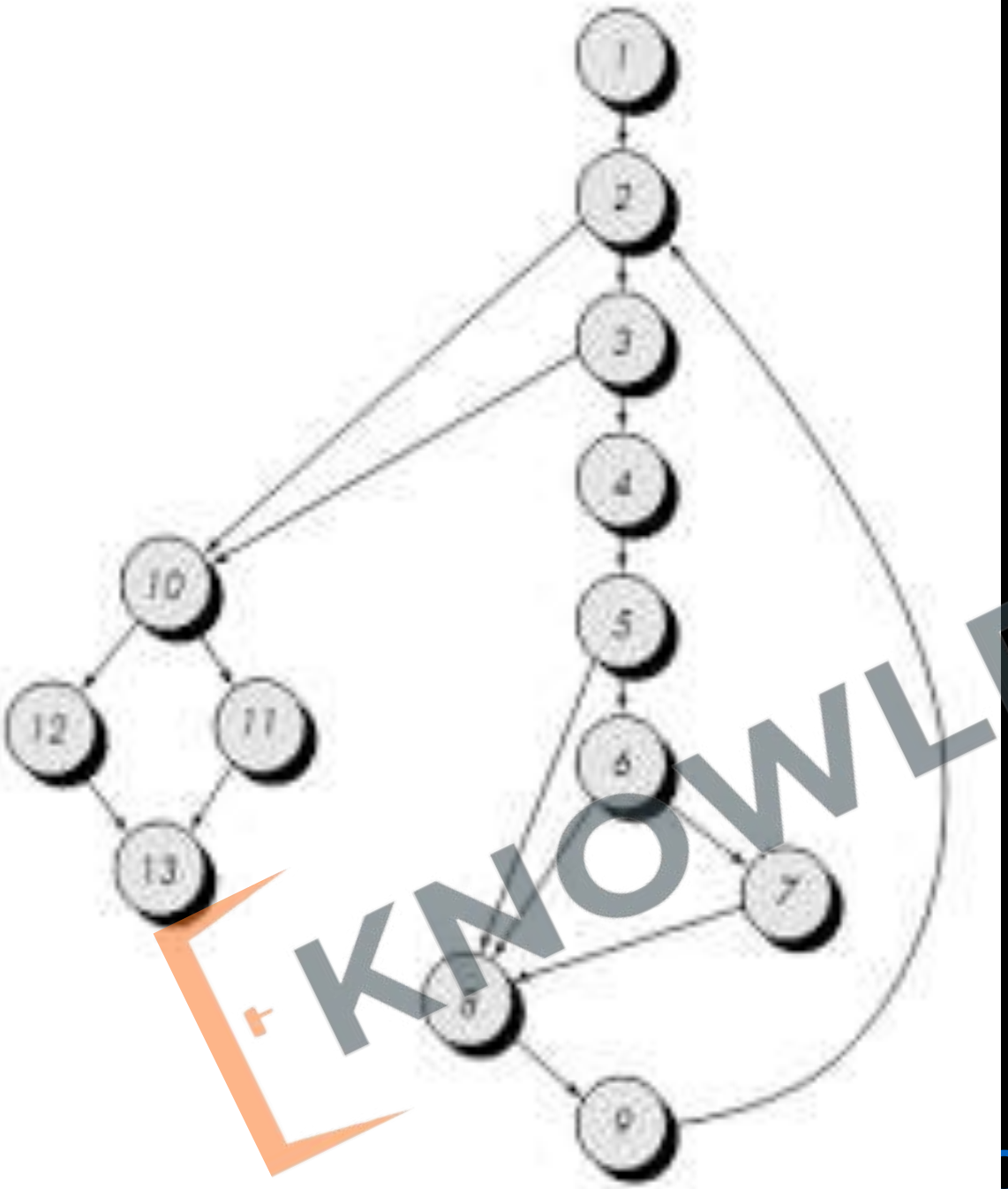
KNOWLEDGEGATE

wledgegate.in/GATE



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE



KNOWLEDGEGATE

owledgegate.in/GATE

Unit-4(Software Testing)

- Testing Objectives,
  - Unit Testing,
  - Integration Testing,
  - Acceptance Testing,
  - Regression Testing,
- Testing for Functionality and Testing for Performance,
- Top-Down and Bottom-Up
- Testing Strategies:
  - Test Drivers and Test Stubs,
  - Structural Testing (White Box Testing),
  - Functional Testing (Black Box Testing),
  - Test Data Suit Preparation,
  - Alpha and Beta Testing of Products.
- Static Testing Strategies: Formal Technical Reviews (Peer Reviews), Walk Through, Code Inspection, Compliance with Design and Coding Standards.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Coding

- It is a phase where we translate a design or algorithm into a code of a particular programming language.
- It is a technical phase so there is not much in software engineering to given guideline in this phase, still we can give characteristics of good code.
- **Characteristics of Good Coding**
  - It must be simple, easy to understand (unconditional jumps must be avoided)
  - It must be readable, i.e. we must use proper space and comments lines and write code in hierarchical fashion
  - Usability try to code in modular fashion so that we can reuse the previous code.

KNOWLEDGE GATE

Compliance with Design and Coding Standards

- **Definition**:
  - Compliance with design and coding standards refers to adhering to a set of rules, guidelines, and best practices that govern the process of designing and writing software code, ensuring consistent, high-quality, and maintainable software products.

- **Importance**:
  - Improves code readability and maintainability
  - Facilitates collaboration and communication among team members
  - Minimizes the introduction of defects and vulnerabilities
  - Reduces development time and costs

KNOWLEDGEGATE

- **Design Standards**:
  - **Architectural consistency**: Ensuring that the overall structure of the software adheres to established patterns and principles
  - **Component modularity**: Encouraging a modular design that promotes separation of concerns and code reusability
  - **Interface design**: Defining clear, consistent, and easy-to-understand interfaces for components or modules
  - **Scalability and performance**: Designing software to handle future growth and changing requirements without negatively impacting performance

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Coding Standards**:
  - **Naming conventions**: Establishing consistent rules for naming variables, functions, classes, and other code elements
  - **Formatting**: Defining guidelines for indentation, whitespace, and code layout to improve readability
  - **Comments and documentation**: Providing clear, concise, and accurate inline comments and external documentation
  - **Error handling**: Implementing proper error handling and reporting mechanisms to improve software robustness and reliability

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Compliance Enforcement**:
  - **Code reviews**: Conducting regular peer reviews to ensure adherence to design and coding standards
  - **Automated tools**: Using static code analysis and linter tools to identify deviations from established standards
  - **Continuous integration**: Integrating and testing code changes frequently to catch issues early in the development process
  - **Training and education**: Providing team members with training and resources to stay up-to-date on best practices and standards

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Challenges**:
  - Balancing strict adherence to standards with development speed and flexibility
  - Ensuring that standards evolve as new technologies and best practices emerge
  - Obtaining buy-in from all team members and fostering a culture of compliance

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Testing

- Because of human error there will be a bug or fault in the code and if that bug/fault is executed it become a failure.
- Software testing is a process of executing a program with the intention of finding bugs or fault in the code.
- It is generally very difficult to test a software exhaustibility or completely because the i/p space is very large. So, we have to write test cases wisely so that in minimum test we can provide the maximum reliability.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Preparation for Testing

- Before we start test, we must have written, complete & approved copy of SRS.
- The budget, time & schedule for the testing must be written and document with proper timeliness & milestones which are to be achieved.
- We must have a proper assembled team with we understood responsibilities.
- We must have a written document of limitation property and scope of testing.

KNOWLEDGE GATE

- There are two methods of arranging testing
  - **Skilled based approach**: In this, a person works on a specific technology for a long time. here the person has a chance to become a true specialist in that particular area or technology
  - **Project based approach**: Here we assign test team to a project so that they can have a much deeper and better understand of that particular project which increases the chance of success.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Objective of testing

- To check weather software is build accordance with the requirement or not.
- It guarantees to give a more reliable or better-quality product to the customer which will ensure its satisfaction.
- It also helps in software quality assurance where we understood what are the frequent or normal mistakes we do.

KNOWLEDGEGATE

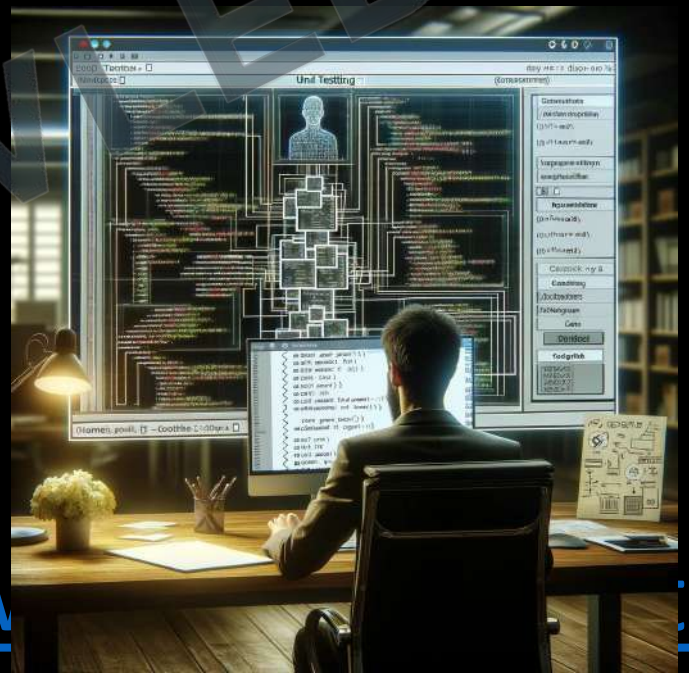
Principle of software testing/guidelines

1. Testing must be based on user requirement
2. Test time, resource cost is limited.
3. It is impossible to check entire i/p space.
4. Testing must start after a proper test plan.
5. The possibility of existence of more error in a module id directly proportional to error already being found.
6. Testing must be done by a third party (at least not developer)
7. Assign best person of the company during testing.
8. Tester must have a destructive attitude towards the code.
9. We must perform both functional and non-functional testing
10. We must give emphasize on automated test tools but the final testing must be done by the humans

KNOWLEDGEGATE

Unit Testing

- **Definition**: It is the first level of testing that involves testing individual components or units of code to ensure they work correctly in isolation. It is concerned for the testing only a specific module.
- **Purpose**: To verify that each unit of code performs its intended function and to catch bugs early in the development process. Will check internal logic of module and Functionality and interface with the other module
- **Test Cases**: Writing small, specific test cases that cover different input scenarios and edge cases for each unit of code.



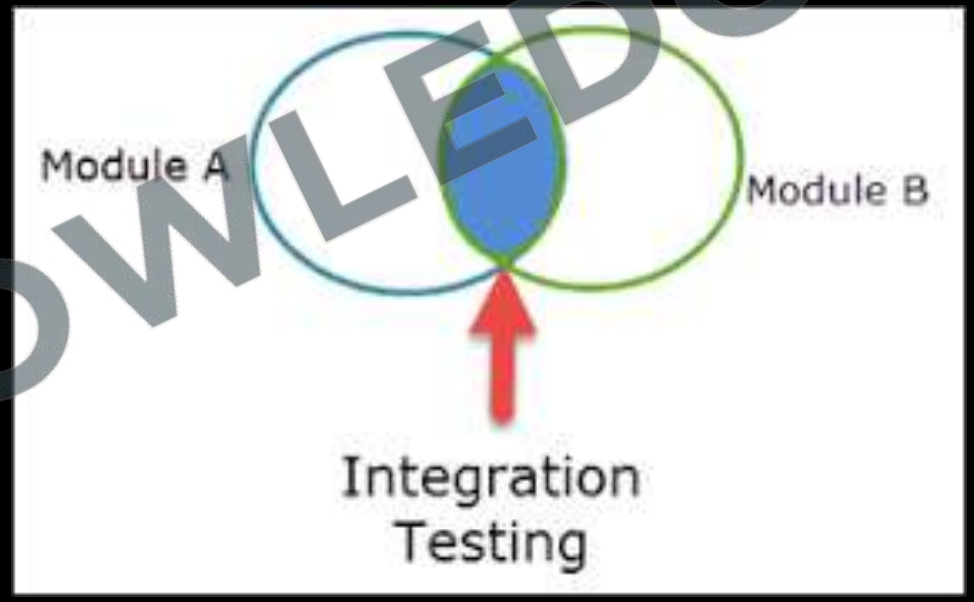
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Integration Testing

- Integration testing is a phase in software testing in which individual software modules are combined and tested as a group.

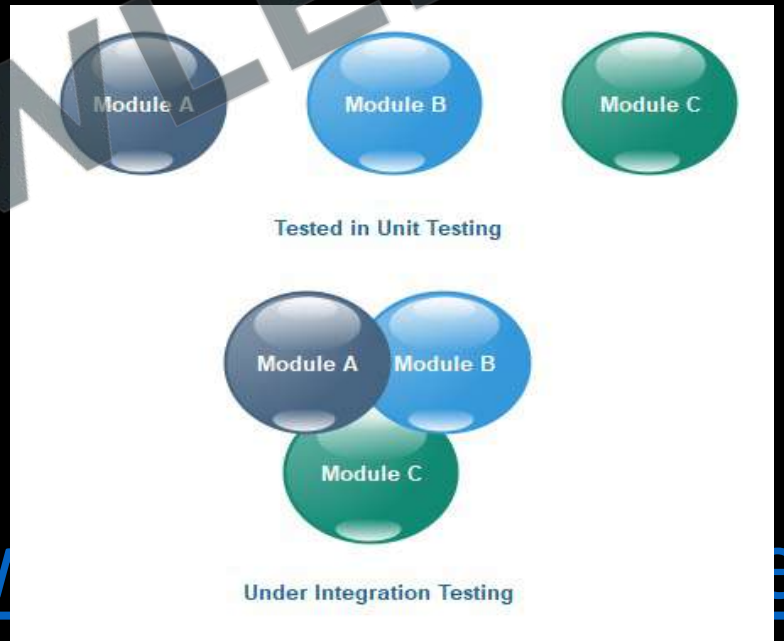
- The primary objective of integration testing is to test the module interfaces, i.e. there are no errors in the parameter passing, when one module invokes another module. To verify the functional, performance, and reliability between the modules that interact with each other.



[KNOWLEDGEGATE]

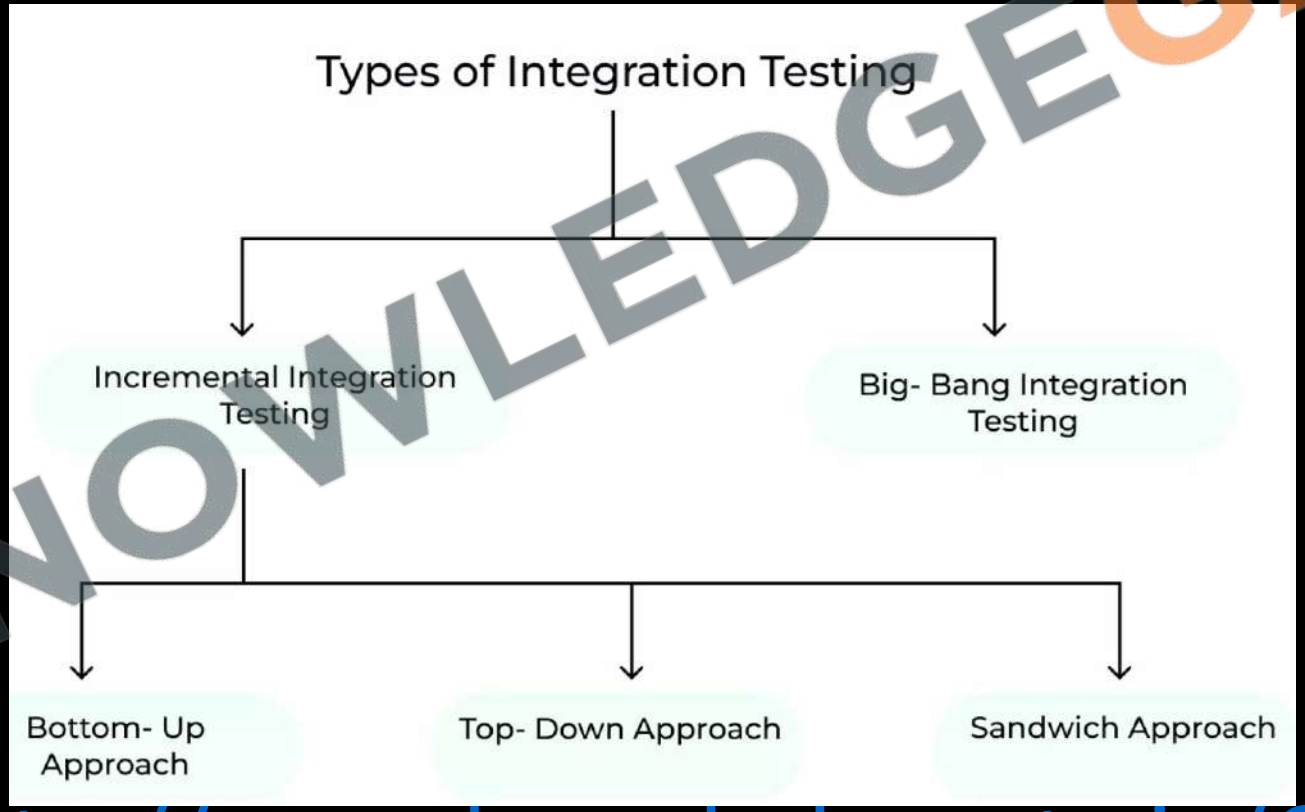
Integration Testing

- Integration testing is the phase in software testing in which individual software modules are combined and tested as a group. It occurs after unit testing and before validation testing.
- Purpose is to expose faults in the interaction between integrated units.
- It can be time-consuming and costly due to the complexity of inter-module dependencies. It requires a lot of coordination between different teams. The sequence of tasks is crucial for integration testing which may cause delays if not planned properly.



KNOWLEDGE GATE

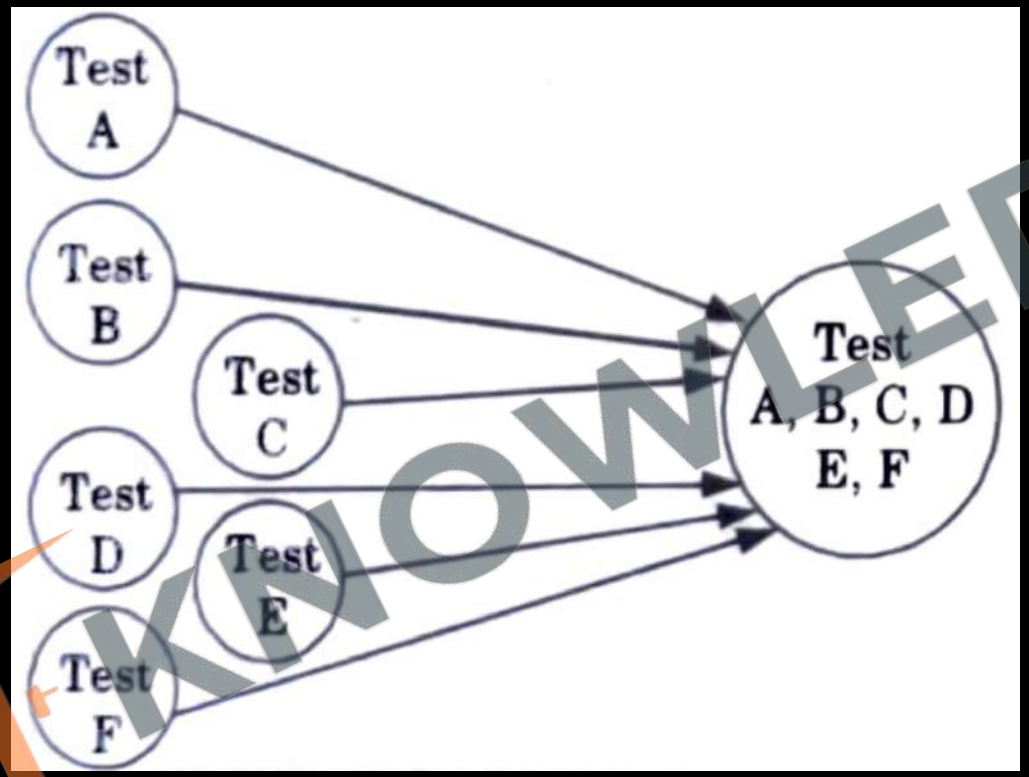
- Types of Integration Testing:
  - Big Bang Integration Testing
  - Top-Down Integration Testing
  - Bottom-Up Integration Testing
  - Sandwich/Hybrid Integration Testing



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Big-Bang

- Is a type of integration testing where all the modules are integrated simultaneously and then tested as a complete system,



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Simplicity:** It is easier to set up because testing starts only after all the modules have been integrated.

- **Suitability:** It is better suited for smaller systems where the modules are heavily interlinked.

- **Efficiency:** It can potentially save time, as testing is conducted after the entire software has been developed and integrated.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

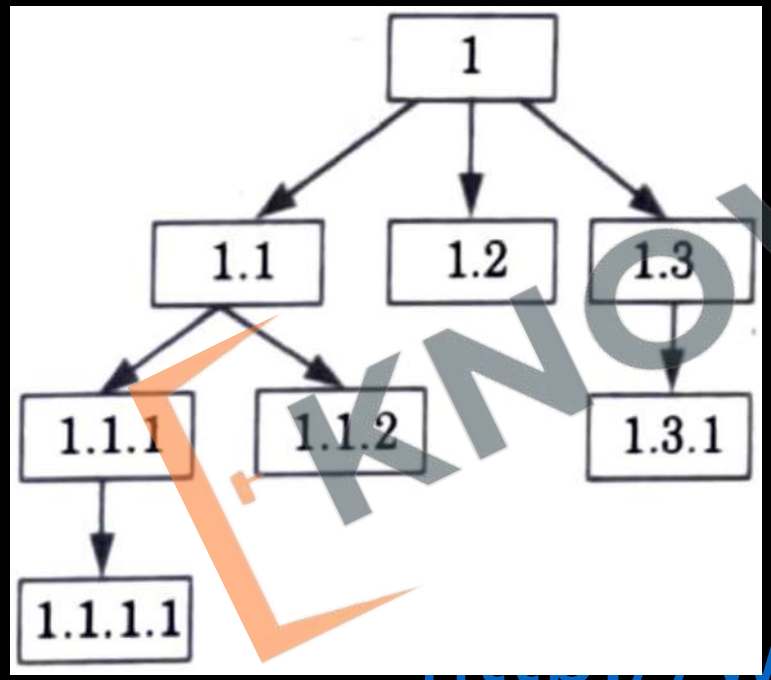
- **Issue Detection and Resolution:** It can be challenging to isolate and fix bugs because of the high level of integration.
- **High Risk:** There's a high risk involved as any significant issues are only found late in the development process, which can lead to project delays.
- **Resource Consumption:** It can be resource-intensive, requiring a significant amount of time and effort to find and fix bugs.
- **Inefficiency in Large Systems:** It's inefficient for larger systems where problems can become increasingly complex and hard to identify when all modules are integrated at once.

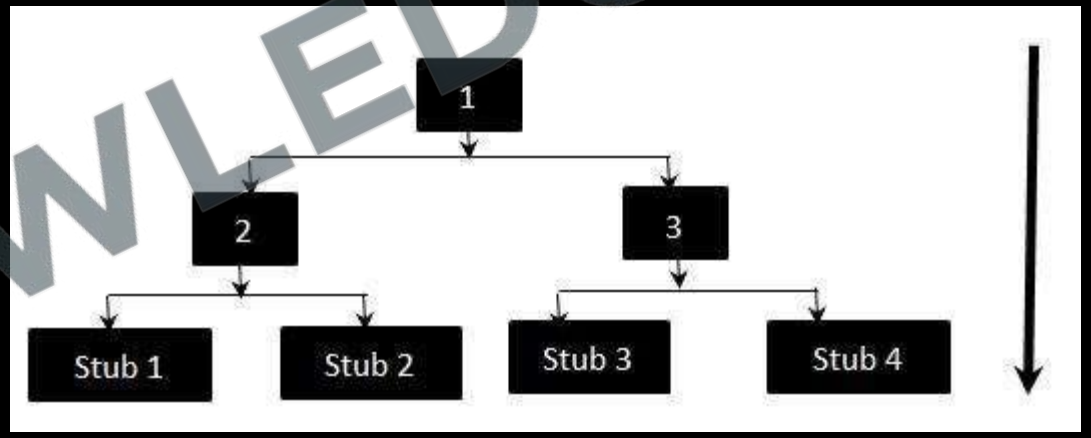
KNOWLEDGEGATE

Top to Bottom

- Top-Down Integration Testing is a method of software integration testing where the top-level modules are tested first and the lower-level modules are tested step by step after that. This process continue until all components are integrated and then whole system has been completely tested.

- Stub modules may be used to simulate the effect of lower-level modules that have not yet been integrated and are called by the routines under test.





KNOWLEDGEGATE

Advantages of Top-Down Integration Testing

- **Early Defect Identification:** Critical high-level design and control flow issues can be detected at an early stage.
- **Facilitates Progressive Testing:** Testing is easier and more systematic, progressing from top-level modules to lower-level modules.
- **Supports Early Demonstration:** The basic functionality of the system can be demonstrated early in the testing process, even if lower-level modules are not yet developed or tested.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

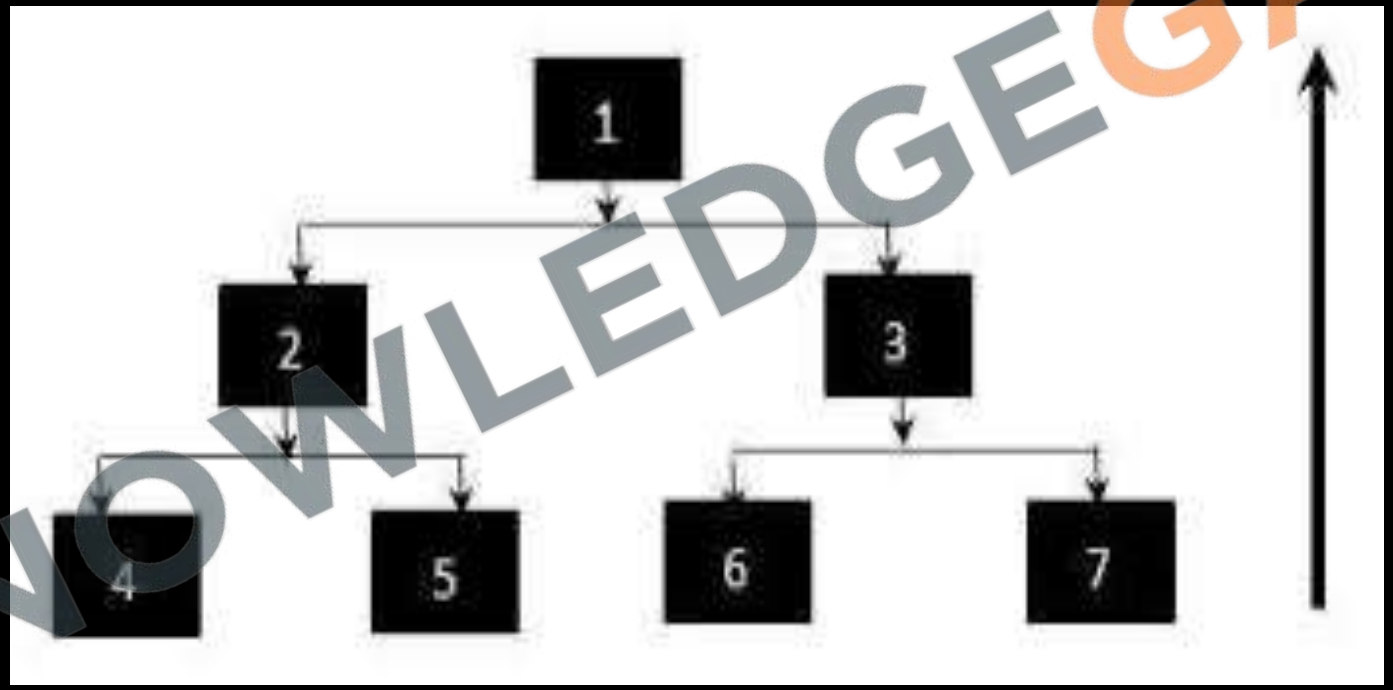
Disadvantages of Top-Down Integration Testing

- **Stub Development:** Stubs need to be created for simulating lower-level modules, which may require additional time and resources.
- **Late Detection of Lower-Level Bugs:** Bugs in lower-level modules may not be found until the later stages of testing, which can lead to delays.
- **Incomplete Testing:** Due to reliance on stubs, some types of errors can be difficult to detect until full functionality is integrated.
- **Difficulty in Test Management:** The complexity and dependencies between different modules can make test management challenging.

KNOWLEDGEGATE

Bottom to Top

- Bottom-Up Integration Testing is a strategy in software testing where the lower-level modules are tested first and then integrated and tested with the higher-level modules. This approach often uses "driver" modules for testing and simulation.



KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Advantages of Bottom-Up Integration Testing

- **Early Problem Detection:** It allows early detection of faults and failures in the lower-level modules of the software.
- **No Need for Stubs:** Unlike top-down testing, bottom-up testing doesn't require the use of stubs as testing begins from the lower-level modules.
- **Simultaneous Development and Testing:** Different modules can be tested in parallel, potentially speeding up the testing process.

KNOWLEDGEGATE

Disadvantages of Bottom-Up Integration Testing

- **Delay in Higher-Level Module Testing:** Testing of higher-level modules is delayed as lower-level modules need to be tested first.
- **Need for Drivers:** Drivers need to be created to simulate higher-level modules, which can require additional time and resources.
- **Late Detection of Higher-Level Bugs:** Issues in the integration of high-level modules may not become apparent until late in the testing process.
- **Incomplete System Overview:** Early stages of testing do not provide a complete view of the system, which may make it harder to assess overall functionality and performance.

KNOWLEDGEGATE

Sandwich/Hybrid integration testing

- The combination of top down and bottom integration testing is called sandwich integration.
- This system is viewed as a three layer just like a sandwich, the upper layer of sandwich use top down integration the lower layer of the sandwich integration use bottom up integration and Stubs and drivers are used to replace the missing modules in the middle layers.

KNOWLEDGEGATE

http://v e.in/GATE

- **Combines Strengths:** It combines the advantages of both Top-Down and Bottom-Up testing approaches to achieve more comprehensive testing coverage.

- **Time-Efficiency:** The simultaneous testing at both ends can help to reduce the overall testing time.

- **Variety of Scenarios:** This approach allows for a wide range of testing scenarios and can lead to more thorough verification of the system's functionality.

- **Flexibility:** It offers flexibility in the testing process, as it can be adjusted according to the nature of the software and the resources available.

KNOWLEDGEGATE

System Testing

- System testing is a level of software testing where the complete and integrated software system as a whole is tested to evaluate its compliance with specified requirements in SRS.
- It is a crucial step before the software gets deployed to the user, aiming to catch any defects that might have slipped through the earlier stages of testing.
- System testing is performed in an environment that closely resembles the real-world or production environment.
- Generally, it is performed by independent testers who haven't been involved in the development phase to ensure unbiased testing.
- It may include functional testing, usability testing, performance testing, security testing, and compatibility testing.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

User Acceptance Testing

- **Definition and Purpose:** User Acceptance Testing (UAT) is the final testing phase before software deployment, aiming to validate if the system meets the business requirements and is fit for use.
- **Participants:** Usually performed by clients or end-users, UAT evaluates the software's functionality in a real-world scenario.

KNOWLEDGE GATE
http://ww in/GATE

- **Focus:** Emphasizing the software's user-friendliness, efficiency, and effectiveness, UAT goes beyond purely technical aspects to assess overall user experience.

- **Documentation:** During UAT, all scenarios, outcomes, and user feedback are recorded to inform potential changes and improvements.

- **Outcome:** Successful UAT culminates in user sign-off, signifying the system meets the set acceptance criteria and is ready for release.

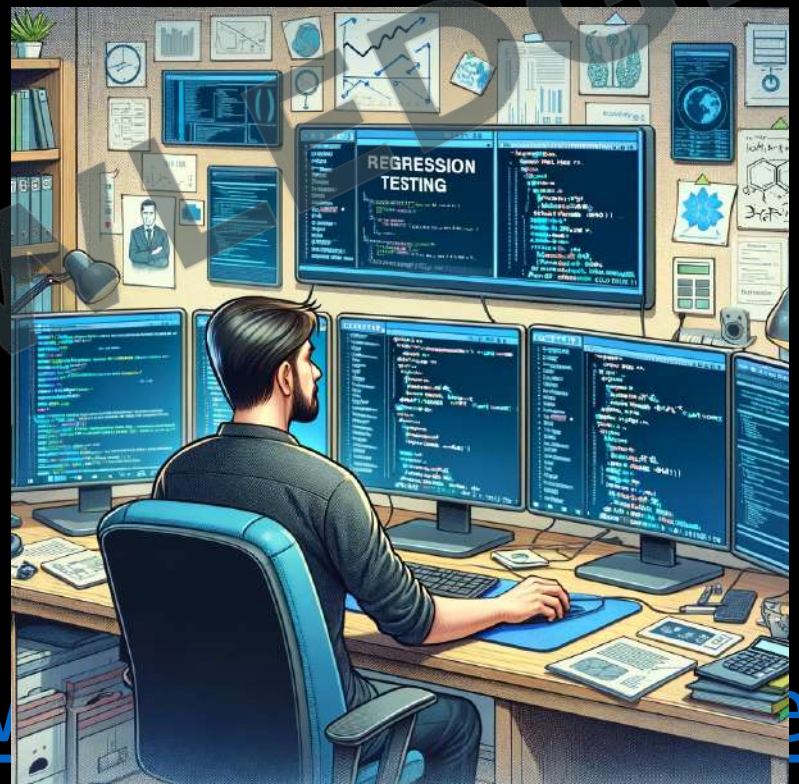
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Regression Testing

- **Definition:** Regression testing is a type of software testing carried out to ensure that previously developed and tested software still functions as expected after making changes, such as updates or bug fixes.

- **Purpose:** The main goal is to identify any issues or defects introduced by changes in the software, and to ensure that the changes have not disrupted any existing functionality.



KNOWLEDGE GATE

- **Types:** Types of regression testing include unit regression, partial regression, and complete regression testing.
- **Test Cases:** Regression testing generally involves re-running previously completed tests and verifying that program behavior has not changed as a result of the newly introduced changes.
- **Automation:** Due to the repetitive nature of these tests, regression testing is often automated to improve efficiency and accuracy.

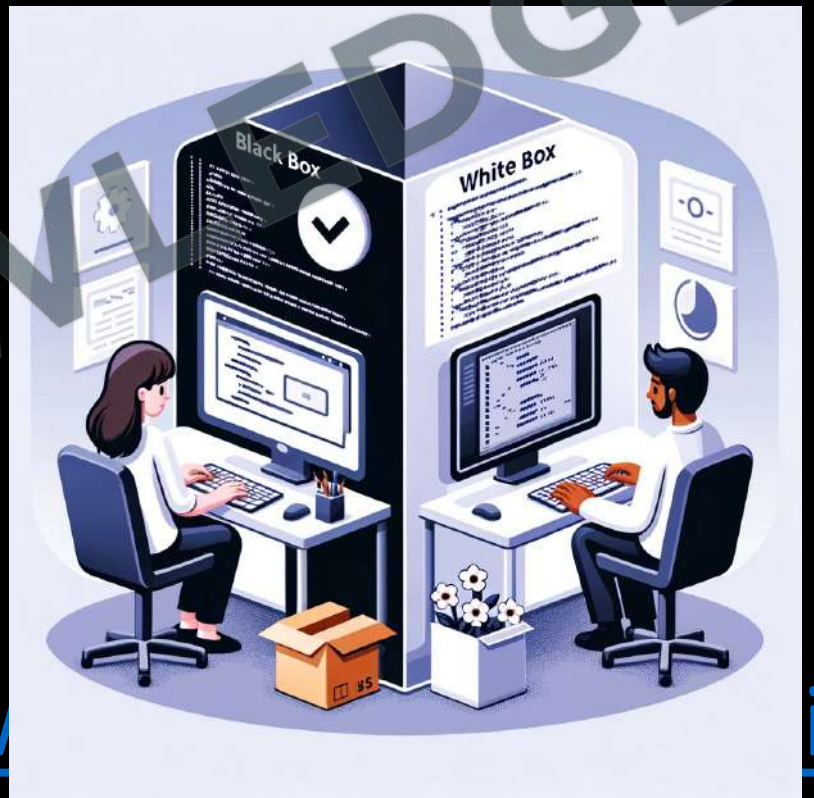
KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Black box Testing/ White box Testing

- **Black Box Testing(Validation)** – Where we treat system as a whole, and check system according to user requirement (are we making the right product), i.e. we check o/p for every i/p

- **White Box Testing(Verification)** – Here we go inside a system and check how actual functionality is performed(are we making the product right)



KNOWLEDGEGATE

http://ww in/GATE

Alpha Testing/ Beta Testing

- Any type of testing which is done on developer side is called alpha testing, usually performed with artificial test cases.

- Any type of testing which is done at customer side is called beta testing, usually performed with real time data.

KNOWLEDGE GATE

http://w in/GATE

Stress Testing

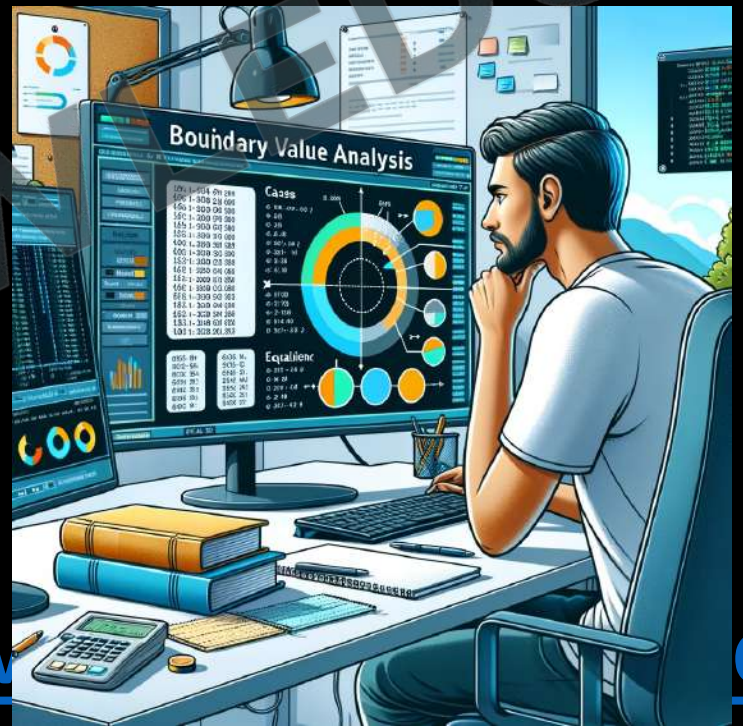
- Stress testing is also known as endurance testing.
- Stress testing evaluates system performance when it is stressed for short periods of time.
- Stress tests are black box tests which are designed to impose a range of abnormal and even illegal input conditions so as to stress the capabilities of the software.

KNOWLEDGEGATE

http://w .in/GATE

Boundary value analysis

- Boundary Value Analysis is a software testing technique that focuses on the values at the boundaries of the input domain.
- The theory behind BVA is that errors are more likely to occur at the extremes of an input domain rather than in the center. Hence, it's generally more useful to focus on testing the boundary values.
- BVA is used for testing ranges and data array elements in a software application.



KNOWLEDGEGATE

- In practice, BVA can be applied by identifying all the boundaries and then creating test cases for the boundary values and just above and below the boundary values.
- Example: Let's consider a simple application that accepts an integer input from 1 to 100.
- The boundary values here would be 0 (just below the valid range), 1 (lower limit), 100 (upper limit), and 101 (just above the valid range).
- You would then create test cases to input these values and verify the system's behavior.

KNOWLEDGEGATE

Equivalence partitioning

- Equivalence Partitioning is a black box testing technique that divides the input data of a software unit into partitions of equivalent data.
- The logic behind EP is that the system should handle all the equivalent data in the same way, thus you can save testing effort by testing only one value from each partition.
- It helps to reduce the total number of test cases from an infinite pool to a more manageable number.
- This technique can be used for both valid and invalid data input.

KNOWLEDGE GATE

- Example: Let's consider the same application that accepts an integer input from 1 to 100.
- The equivalence classes here would be: less than 1 (invalid), between 1 and 100 (valid), and greater than 100 (invalid).
- You would then create test cases to input a value from each of these classes (for example, 0, 50, and 101) and verify the system's behavior.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Graph-Based Testing Methods

- Uses graphical representation for software testing.
- Nodes represent states; edges represent transitions.
- Example: For a web app with Login, Dashboard, and Logout screens, draw nodes for each and edges for transitions. Test paths like Login -> Dashboard -> Logout.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Formal Technical Review (Peer Reviews)

- A Formal Technical Review (FTR) is a structured, systematic, and disciplined approach to examining and evaluating software artifacts, such as code, design documents, or requirements specifications, with the primary goal of identifying and addressing defects, inconsistencies, or areas for improvement.
- FTRs are conducted by a team of peers, consisting of the artifact's author and other software professionals, who analyze the work product and provide constructive feedback to ensure high-quality software development.
- Formal Technical Reviews play a crucial role in improving software quality, detecting issues early in the development lifecycle, promoting knowledge sharing, and fostering collaboration within software engineering teams.

KNOWLEDGE GATE

- Types of Formal Technical Reviews
  - Code reviews
  - Walkthroughs
  - Inspections
  - Pair programming

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- The Formal Technical Review Process
  - A. **Planning**
    - 1. Establish objectives
    - 2. Select participants
    - 3. Set schedule
  - B. **Preparation**
    - 1. Distribute materials
    - 2. Review guidelines
    - 3. Allocate time for individual review
  - C. **Conducting the review**
    - 1. Discuss objectives
    - 2. Review findings
    - 3. Make decisions on action items
  - D. **Post-review activities**
    - 1. Document review results
    - 2. Implement action items
    - 3. Monitor follow-up actions

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Roles in Formal Technical Reviews**
  - A. Review leader
  - B. Author
  - C. Reviewers
  - D. Recorder

- **Benefits of Formal Technical Reviews**
  - A. Improved code quality
  - B. Early defect detection
  - C. Knowledge sharing
  - D. Increased team collaboration

- **Challenges and Best Practices in Formal Technical Reviews**
  - A. Time management
  - B. Maintaining a constructive environment
  - C. Addressing bias
  - D. Ensuring consistency in review standards

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Walk Through

- In software engineering, a walkthrough is a type of informal review process in which the author of a software artifact, such as code, design documents, or requirements specifications, presents their work to a group of peers.
- The main purpose of a walkthrough is to identify defects, inconsistencies, or areas for improvement through a collaborative discussion.
- Walkthroughs are typically less formal than other review methods, such as inspections, and focus on knowledge sharing, collaboration, and training among team members.

KNOWLEDGEGATE

Walkthrough Process

- A. Planning
  - 1. Define objectives
  - 2. Select participants
  - 3. Schedule the walkthrough
- B. Preparation
  - 1. Distribute materials
  - 2. Review guidelines
  - 3. Allocate time for individual review
- C. Conducting the walkthrough
  - 1. Present the work product
  - 2. Discuss findings
  - 3. Record issues and suggestions
- D. Post-walkthrough activities
  - 1. Summarize findings
  - 2. Assign action items
  - 3. Monitor follow-up actions

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- Roles in Walkthroughs
  - A. Presenter
  - B. Reviewers
  - C. Recorder

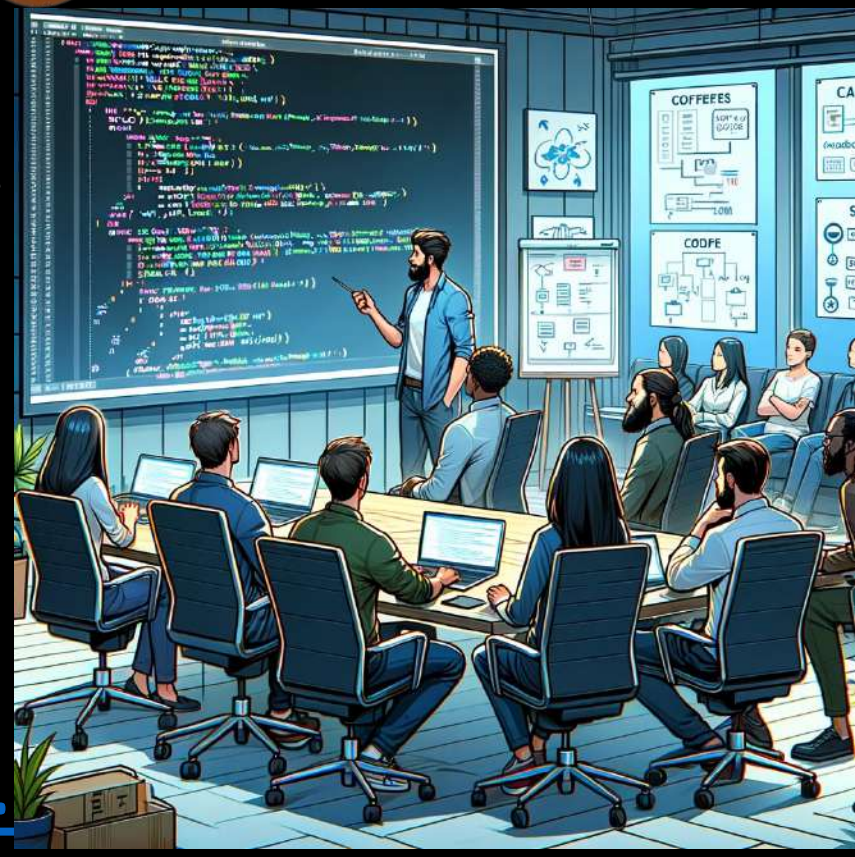
- Benefits of Walkthroughs
  - A. Early defect detection
  - B. Knowledge sharing
  - C. Team collaboration
  - D. Training and mentoring

KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

Code Inspection

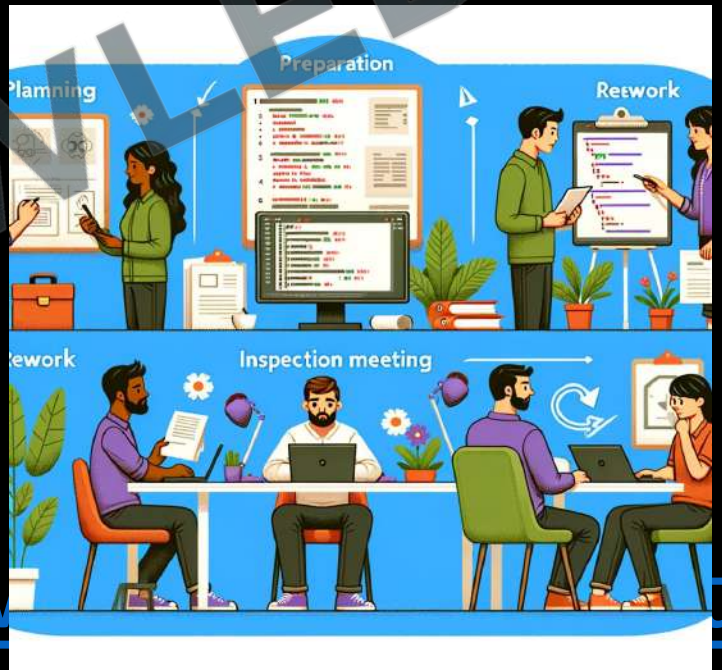
- **Definition**:
  - Code inspection is a systematic review process in which a team of developers evaluates a software product's source code for potential issues, such as errors, vulnerabilities, and deviations from coding standards.
- **Objectives**:
  - Improve code quality
  - Detect and fix defects early in the development cycle
  - Share knowledge and best practices among team members
  - Enforce coding standards and guidelines



KNOWLEDGEGATE

http://www.knowledgegate.

- Process:
  - **Planning**: Select the code to be inspected, define goals, and assemble the inspection team
  - **Preparation**: Team members review the code individually to identify potential issues
  - **Inspection Meeting**: The team discusses the identified issues, and the moderator notes down agreed-upon action items
  - **Rework**: The original developer addresses the identified issues and submits the revised code
  - **Follow-up**: The moderator verifies that all action items have been addressed and closes the inspection



KNOWLEDGEGATE

http://ww te.in/GATE

- Inspection Team Roles:
  - **Author**: The developer who wrote the code being inspected
  - **Moderator**: The person who leads the inspection process and ensures it runs smoothly
  - **Reviewers**: Other developers who provide insights and suggestions for improvements
  - **Recorder**: The person responsible for documenting the issues found and decisions made during the inspection

KNOWLEDGE GATE

http://w .in/GATE

- **Benefits**:
  - Enhanced code quality and maintainability
  - Reduced development costs and project risks
  - Faster time-to-market due to early detection of defects
  - Improved team collaboration and learning

- **Limitations**:
  - Time-consuming process
  - Possibility of human errors or oversights
  - Potential for conflict among team members
  - May not catch all types of defects, such as performance or concurrency issues

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Unit-5(Software Maintenance)

- Software as an Evolutionary Entity,
- Need for Maintenance,
- Categories of Maintenance: Preventive, Corrective and Perfective Maintenance,
- Cost of Maintenance,
- Software Re-Engineering,
- Reverse Engineering.
- Software Configuration Management Activities,
- Change Control Process,
- Software Version Control,
- Resource Allocation Models,
- Software Risk Analysis and Management.

KNOWLEDGEGATE

Why Software Maintenance

- **Always Changing**: Software keeps changing to work better for people who use it. This is like how animals and plants change to survive in the world.
- **Getting More Complex**: Software used to be simple, but now it can do many things and connects people all over the world. This is like how nature has many different animals and plants living together.
- **Survival of the Fittest**: Good software will keep being used, while bad software will go away. This is like how strong animals and plants in nature can live longer and make more babies.

KNOWLEDGEGATE

- **Categories of Software Maintenance –** Maintenance can be divided into the following:
  - **Corrective maintenance:** Corrective maintenance of a software product may be essential either to rectify some bugs observed while the system is in use, or to enhance the performance of the system.
  - **Adaptive maintenance:** This includes modifications and updation when the customers need the product to run on new platforms, on new operating systems, or when they need the product to interface with new hardware and software.
  - **Perfective maintenance:** A software product needs maintenance to support the new features that the users want or to change different types of functionalities of the system according to the customer demands.

KNOWLEDGEGATE

- The key software maintenance issues are both managerial and technical. e.g. alignment with customer priorities, staffing, which organization does maintenance, estimating costs.

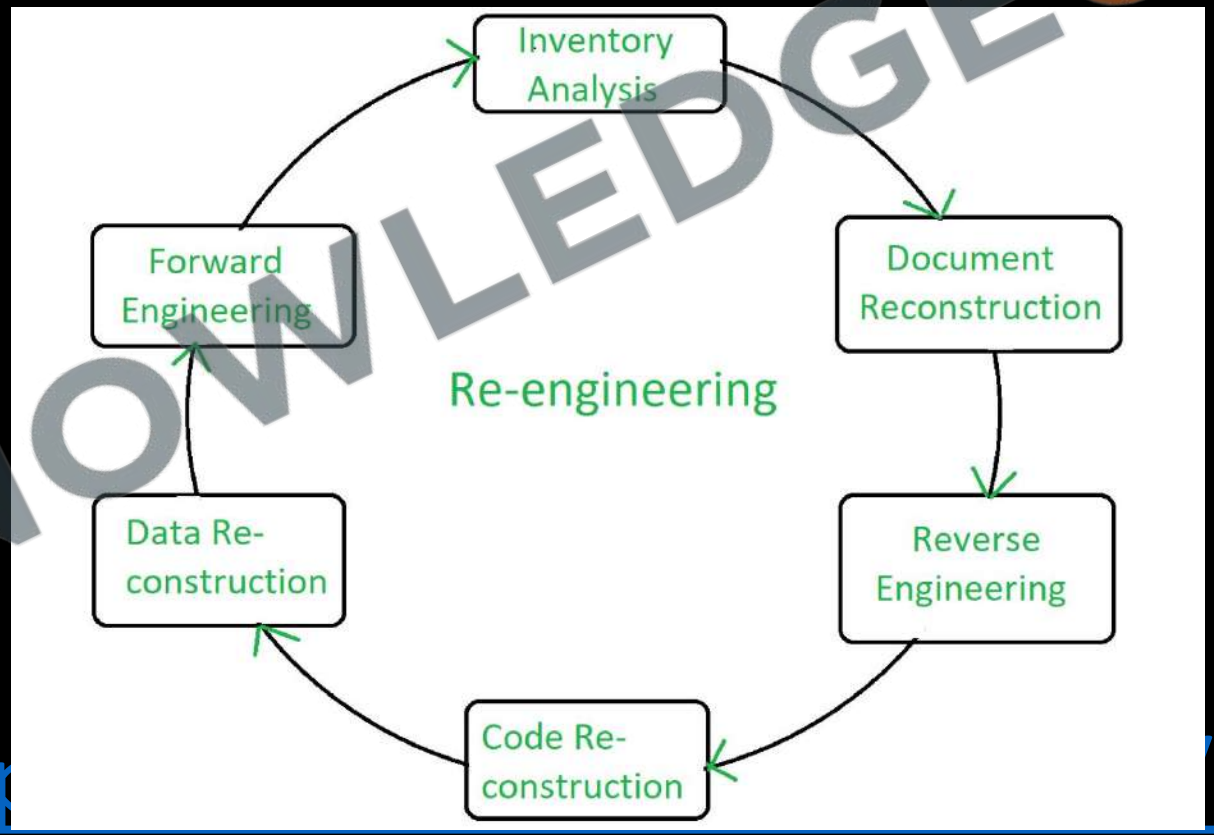
- Key technical issues are: limited understanding, impact analysis, testing, maintainability measurement.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Software Re-Engineering

- Software re-engineering is the process of examining and modifying an existing software system to improve its functionality, performance, or adaptability without changing its core function.
- It's often undertaken to restructure a legacy system to enhance understandability, maintainability, and to upgrade to newer technologies or software practices.



KNOWLEDGEGATE

http GATE

Software Re-Engineering Activities

- **Reverse Engineering**: This is the process of deconstructing a system to understand its components and their relationships, often to create documentation for a system where none exists or is out of date.

KNOWLEDGEGATE

- **Restructuring**: This involves transforming the existing source code into a more maintainable form, often while preserving its functionality. This can include activities such as code refactoring, where the code is reorganized to be more understandable or efficient.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Forward Engineering**: After understanding and possibly restructuring the system, changes are made to the system to improve it or adapt it to new requirements or technologies. This can include implementing new features, improving performance, or changing the system to use a new technology stack.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Re-documentation**: This involves updating or creating new documentation for the system. This can be especially important if the original documentation was lost, out of date, or if the system has been significantly changed during the re-engineering process.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

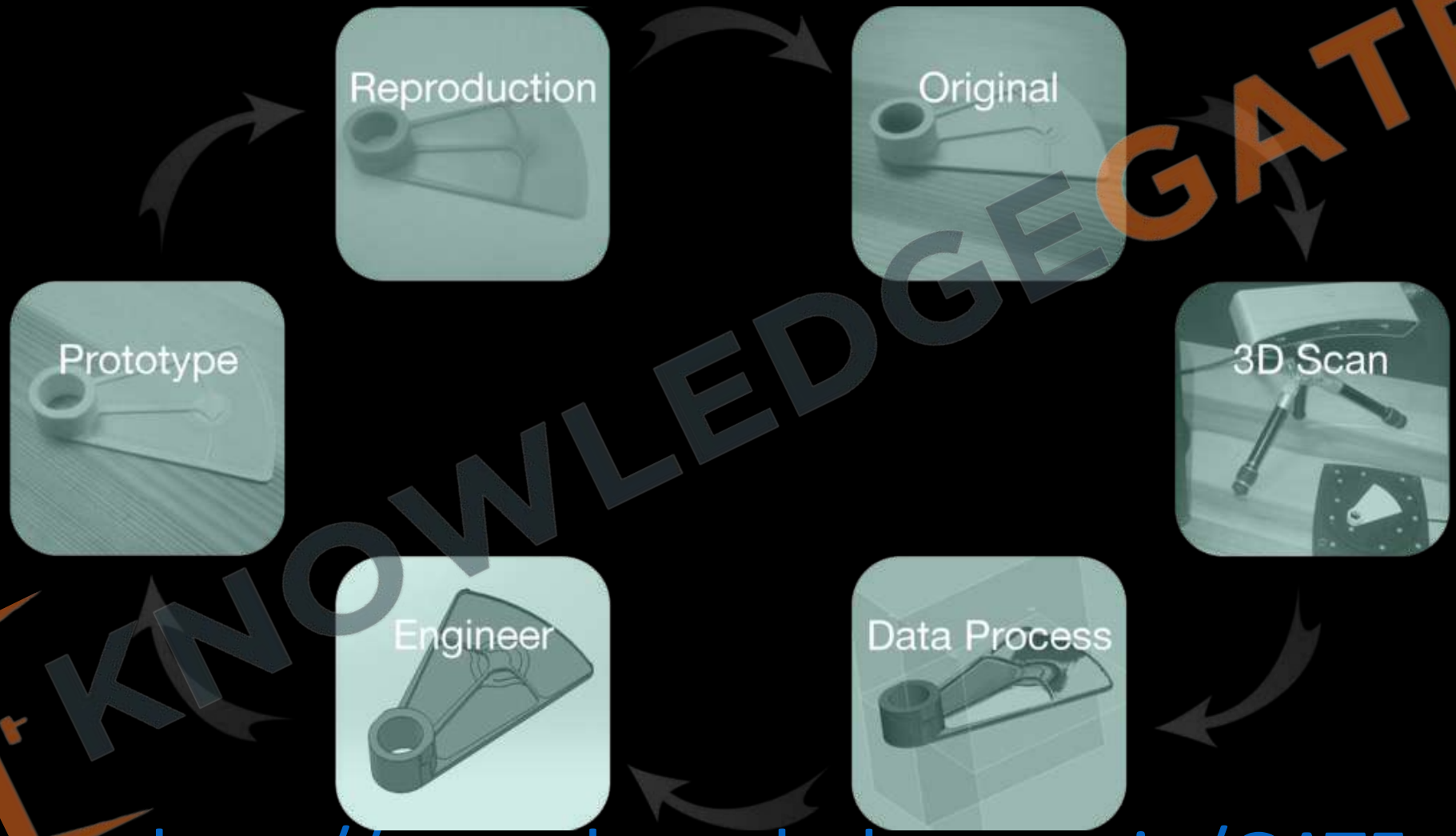
- **Benefits of re-engineering phases**:

  - **Cost**: Re-engineering of an existing software system costs significantly less than new system development.
  - **Lower Risks**: Software re-engineering is based on incremental improvement of systems, rather than racial system replacement. The risk of losing critical software knowledge is drastically reduced.
  - **Better use of existing Staff**: The skill of existing staff can be better utilized by re-engineering.
  - **Incremental Development**: The development is not carried out as a whole. It is in stages.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

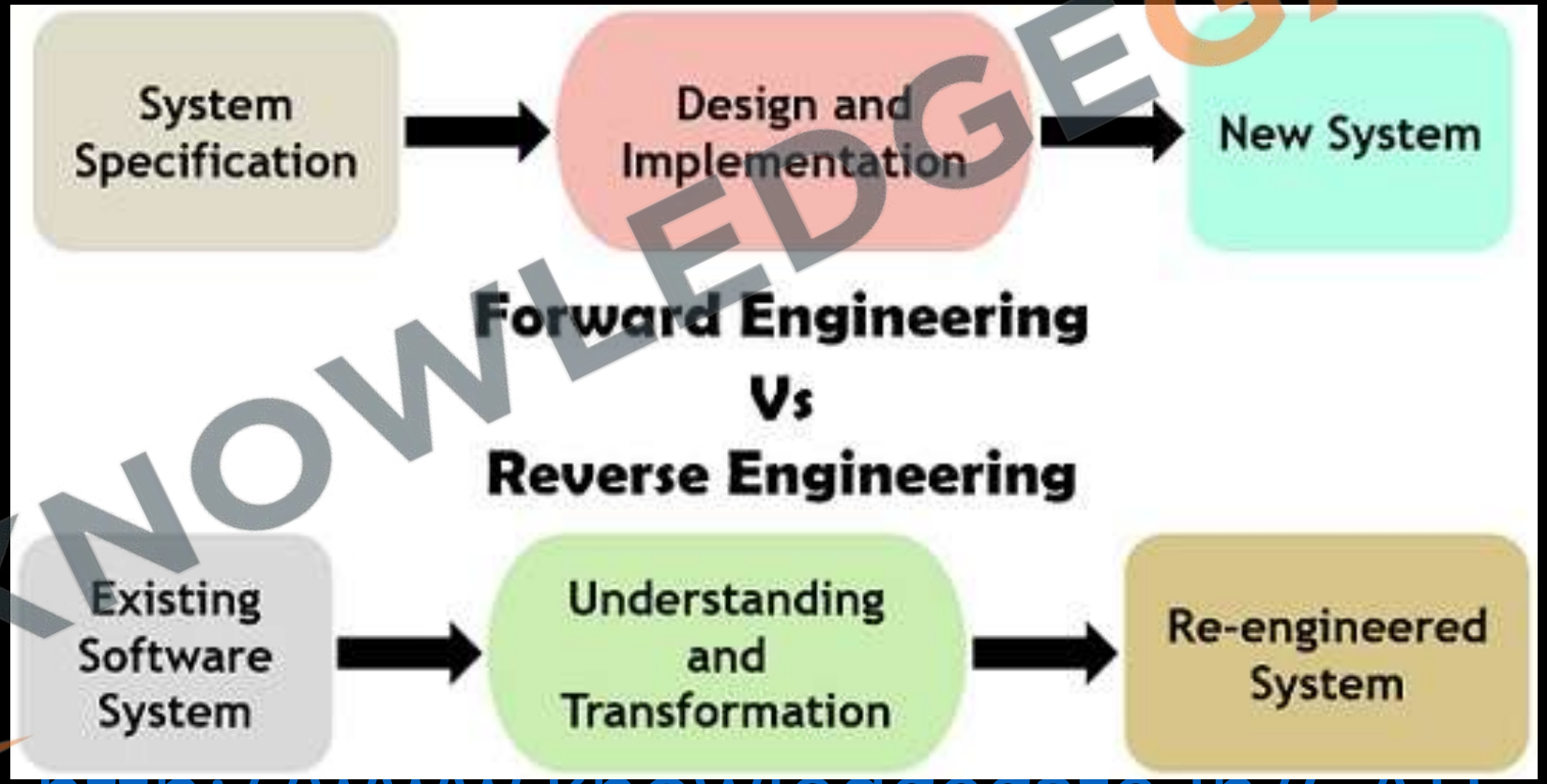
- **Reverse Engineering** – Reverse Engineering is processes of extracting knowledge or design information from anything man-made and reproducing it based on extracted information. It is also called back Engineering.



KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

- **Software Reverse Engineering –** Software Reverse Engineering is the process of recovering the design and the requirements specification of a product from an analysis of it's code. Reverse Engineering is becoming important, since several existing software products, lack proper documentation, are highly unstructured, or their structure has degraded through a series of maintenance efforts.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Steps of Software Reverse Engineering**

1. Collecting Information
2. Examining the Information
3. Extracting the Structure
4. Recording the Functionality
5. Recording Data Flow
6. Recording Control Flow
7. Review extracted Design
8. Generate Documentation

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Why Reverse Engineering?**

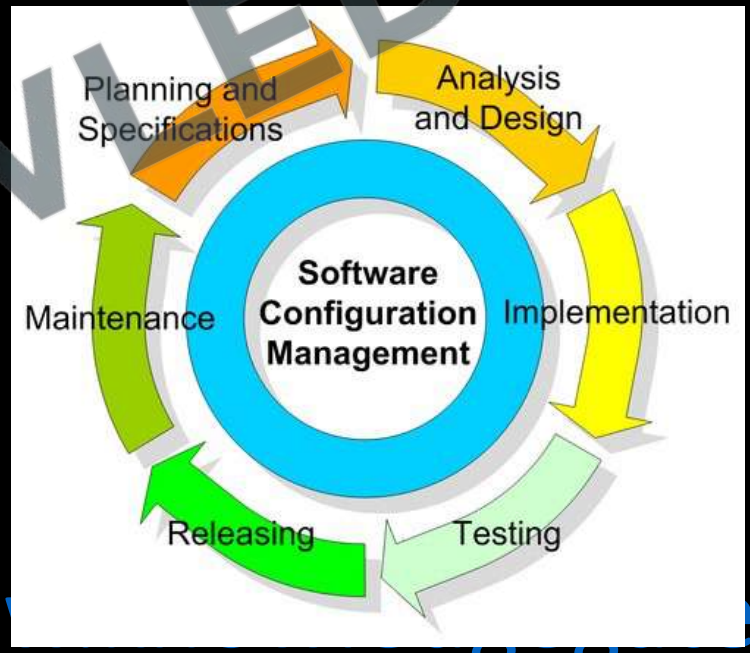
1. Providing proper system documentation.
2. Recovery of lost information.
3. Assisting with maintenance.
4. Facility of software reuse.
5. Discovering unexpected flaws or faults.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Software Configuration Management Activities

- The primary reasons for Implementing Software Configuration Management System are:
  - There are multiple people working on software which is continually updating.
  - It may be a case where multiple version, branches, authors are involved in a software project, and the team is geographically distributed and works concurrently.
  - Changes in user requirement, policy, budget, schedule need to be accommodated.
  - Software should able to run on various machines and Operating Systems.
  - Helps to develop coordination among stakeholders.
  - SCM process is also beneficial to control the costs involved in making changes to a system.



KNOWLEDGEGATE

Software Configuration Management

- Can be defined as a process of defining and implementing a standard configuration, which results into the primary benefits such as easier setup and maintenance, less down-time, better integration with enterprise management, and more efficient and reliable backups and also maximize productivity by minimizing mistakes.
- SCM is used to track and manage the emerging product and its versions.
- SCM ensures that all people involved in the software process know what is being designed, developed, built, tested and delivered.
- Through SCM, the design requirements can be traced to the final software product.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

Objectives Software Configuration Management

- Remote system administration
- Reduced user down-time
- Reliable data backups
- Easy workstation setup
- Multi-User support

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Tasks in SCM process**
  - **Configuration Identification:**
    - Configuration identification is a method of determining the scope of the software system.
    - With the help of this step, you can manage or control something even if you don't know what it is.

  - **Baseline:**
    - A baseline is a formally accepted version of a software configuration item.
    - It is designated and fixed at a specific time while conducting the SCM process. It can only be changed through formal change control procedures.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Change Control:**
  - Change control is a procedural method which ensures quality and consistency when changes are made in the configuration object.
  - In this step, the change request is submitted to software configuration manager.

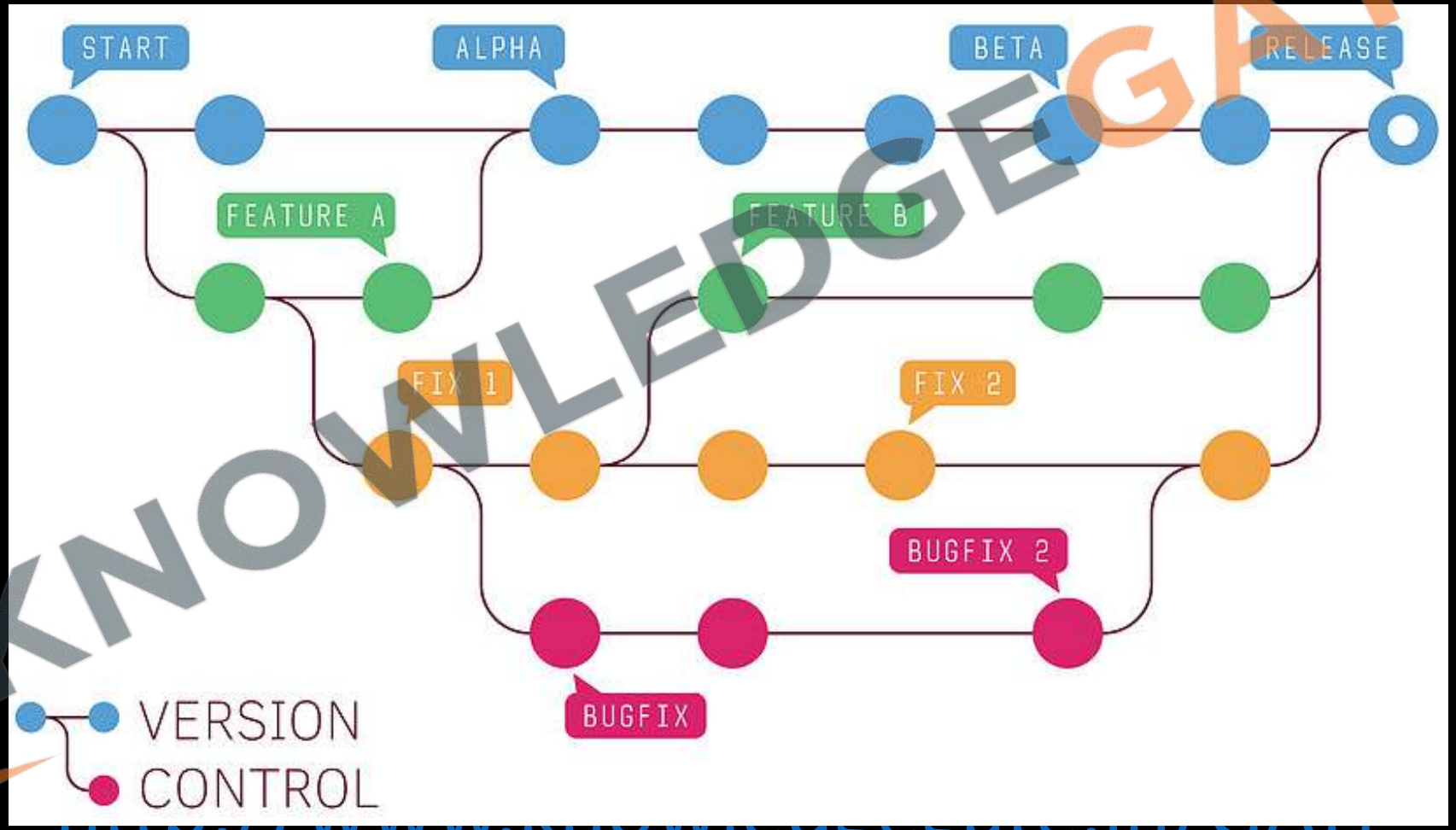
- **Configuration Status Accounting:**
  - Configuration status accounting tracks each release during the SCM process.
  - This stage involves tracking what each version has and the changes that lead to this version.

- **Configuration Audits and Reviews:**
  - Software Configuration audits verify that all the software product satisfies the baseline needs.
  - It ensures that what is built is what is delivered.

[KNOWLEDGEGATE]

Software version control

- Version control is a way to keep track of changes in computer programs. It helps people work together on a project without causing problems. They can see what changes were made, when, and by whom. It also lets them go back to an older version if something goes wrong.



KNOWLEDGEGATE

- **Identifying New Versions**: Each time changes are made and saved, the version control system creates a new, unique version of the software. This helps developers track the software's evolution and easily manage multiple versions.

- **Numbering Scheme**: Version control systems assign a specific numbering or naming scheme to each version (e.g., 1.0, 1.1, 1.2, or 2.0). This helps developers and users identify the software's progression and understand the scope of changes between versions.

- **Visibility**: Version control makes the entire history of the software project visible to all team members. This enhances collaboration and helps developers understand the context and impact of past changes on the current codebase.

- **Tracking**: Version control systems keep detailed records of who made each change, when it was made, and the specific modifications. This level of tracking helps maintain accountability and facilitates communication among team members.

KNOWLEDGEGATE

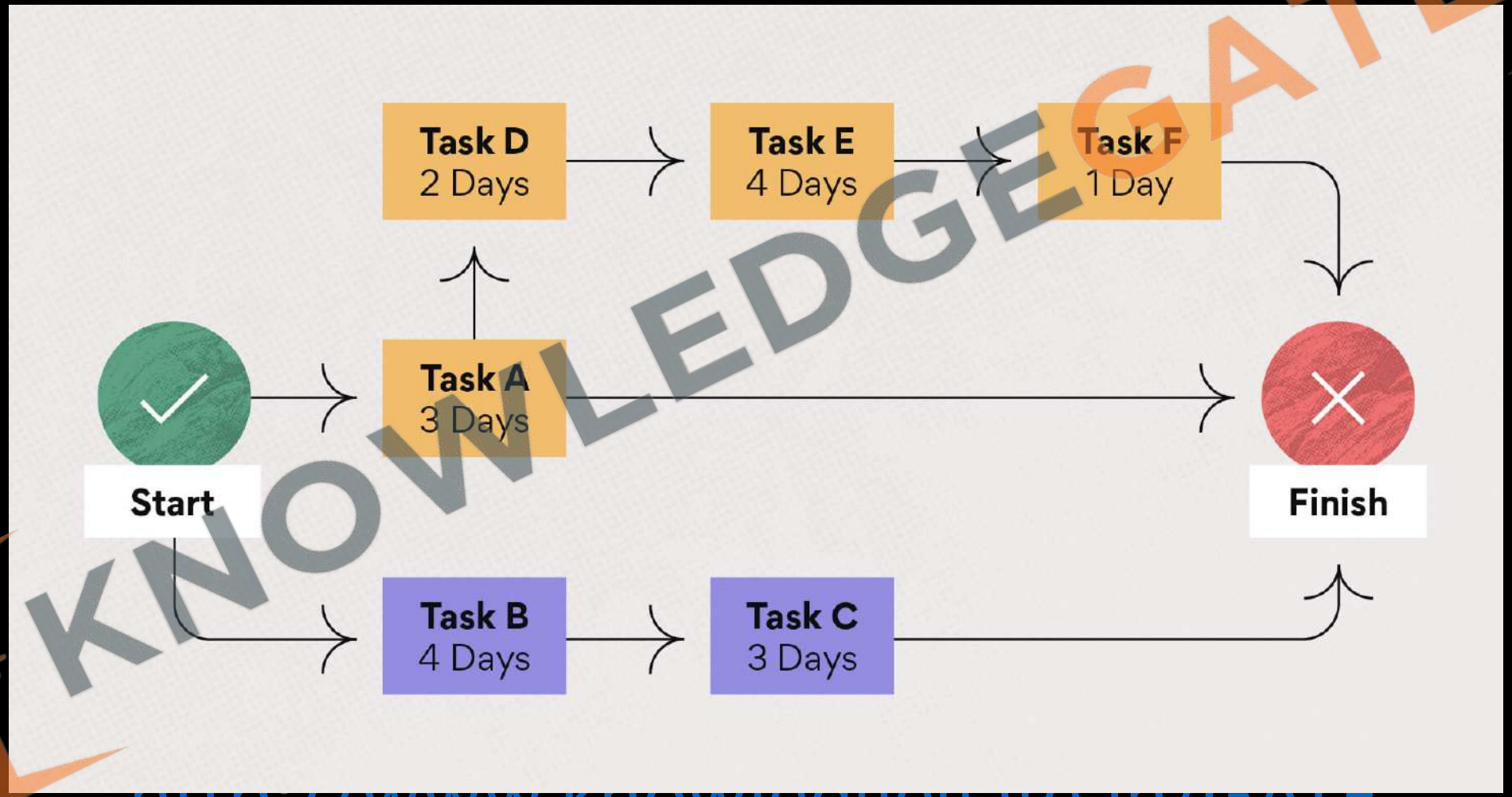
http://www.knowledgegate.in/GATE

Resource Allocation Models

- Resource allocation models in software engineering refer to strategies or methods used to distribute various types of resources effectively across different aspects of a software project. These resources could include things like personnel, time, budget, computing resources, etc.
- These models help project managers in decision-making processes to ensure the efficient use of resources, meet project deadlines, maintain the quality of the software product, and stay within the budget.

KNOWLEDGEGATE

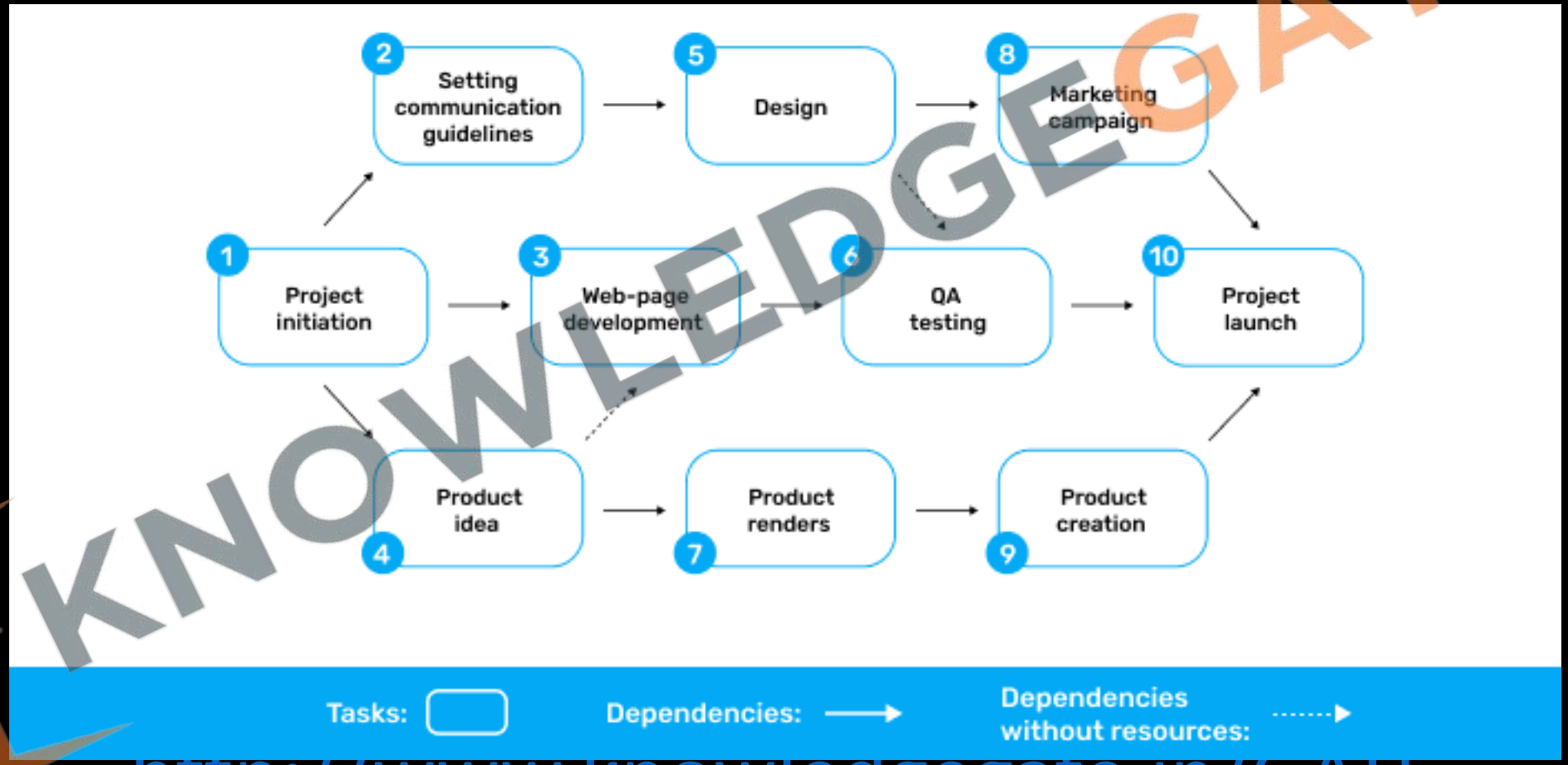
- **Critical Path Method (CPM)**: This is a step-by-step technique used in project management for scheduling project activities. It identifies critical and non-critical tasks to prevent conflicts and bottlenecks.



KNOWLEDGEGATE

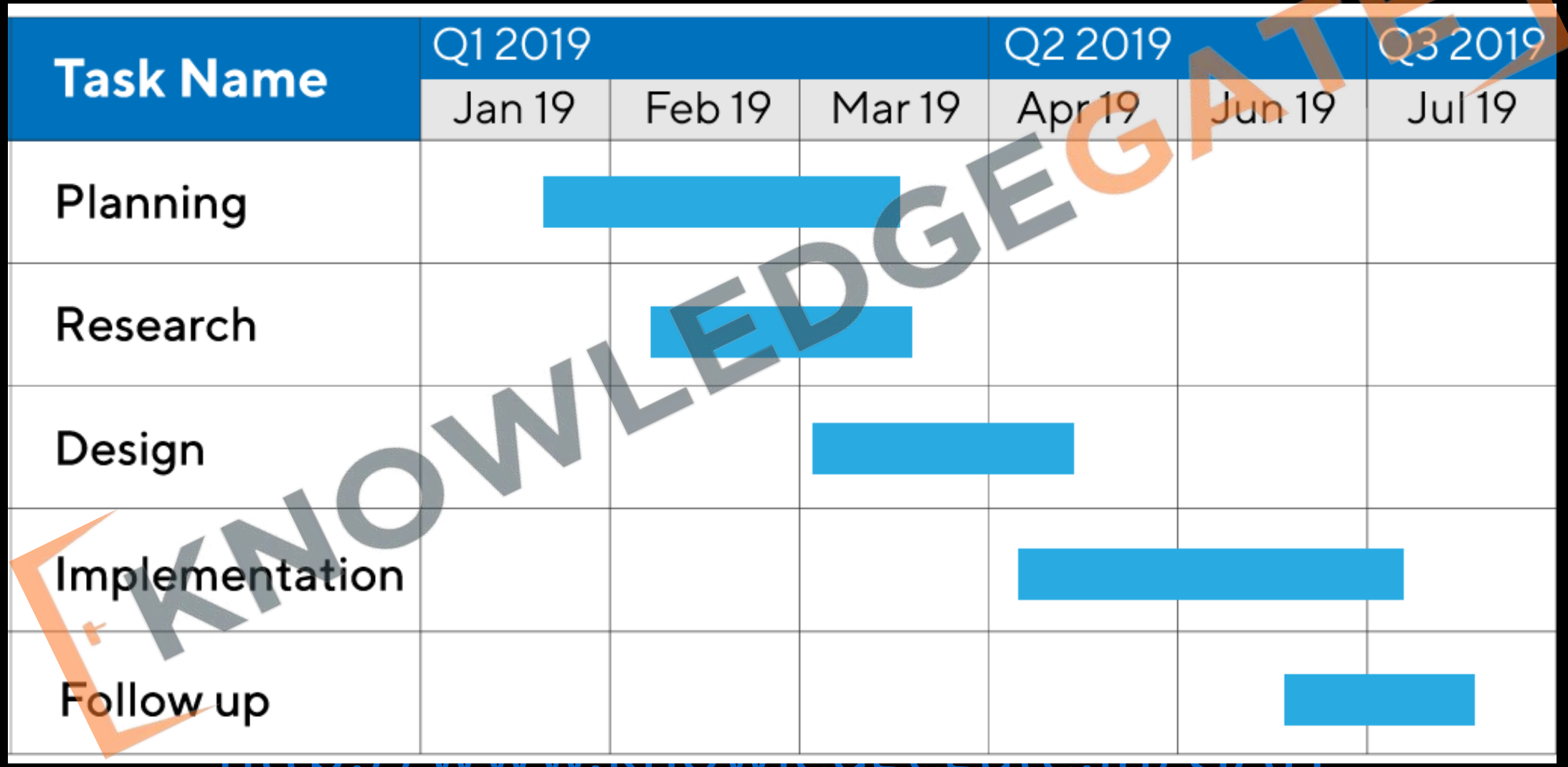
http://www.knowledgegate.in/GATE

- **Program Evaluation and Review Technique (PERT)**: This is a statistical tool used in project management, designed to analyze and represent the tasks involved in completing a given project. It helps to estimate the minimum time needed to complete the project.



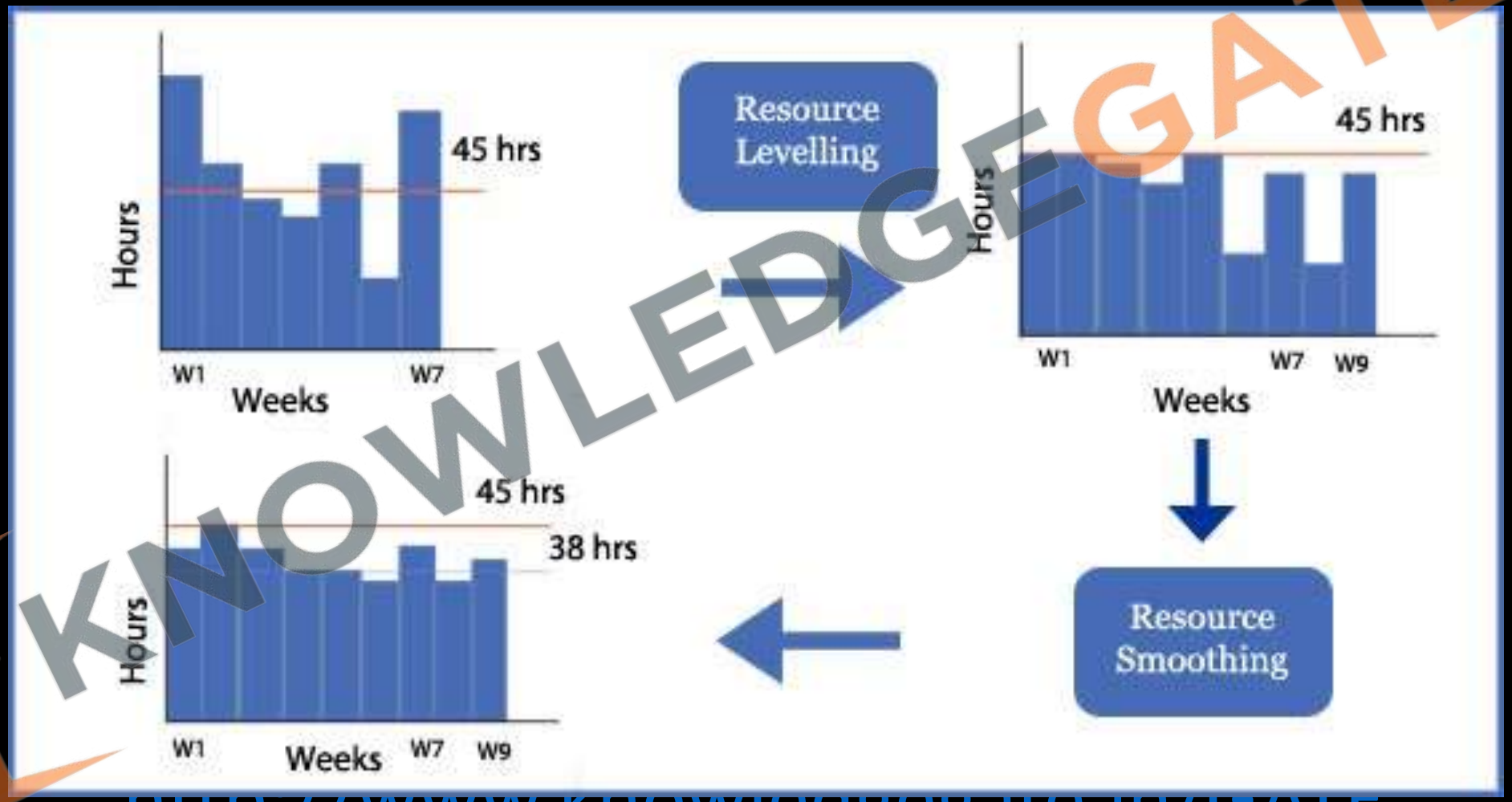
KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Gantt Chart**: It's a type of bar chart that illustrates a project schedule. This chart lists the tasks to be performed on the vertical axis, and time intervals on the horizontal axis.



KNOWLEDGEGATE

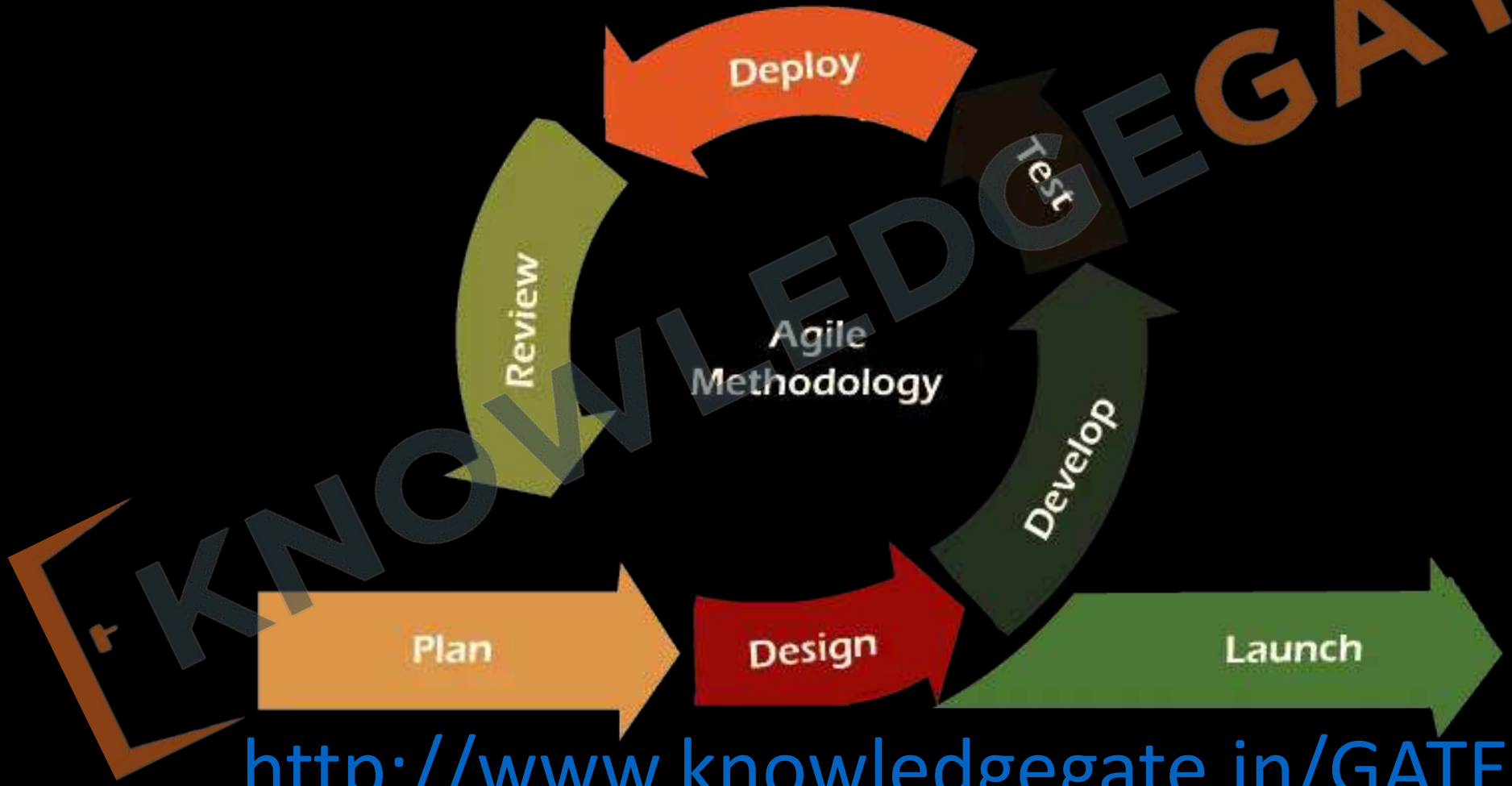
- **Resource Leveling**: Also known as resource smoothing, it's a technique in project management that involves adjusting the project schedule to balance the demand for project resources with the available supply.



KNOWLEDGE GATE

http://www.knowledgegate.in/GATE

- **Allocation Models in Agile Methodology**: In agile teams, resources might be allocated based on different factors such as team velocity, priority of user stories, and size of tasks.



KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

Software Risk Analysis and Management

- Risk management is the process of identifying risk, assessing risk hat could negatively impact a software project's success and taking steps to reduce this to acceptable level.

KNOWLEDGEGATE

- Here are the key components:
  - **Risk Identification**: This is the initial process of determining risks that could potentially prevent the program, enterprise, or investment from achieving its objectives. It includes documenting and communicating the concern.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Risk Analysis**: Once risks are identified, they are analyzed to identify the qualities of the risk such as its likelihood of occurrence, severity of impact, what precipitates the risk, and other characteristics. This can be done quantitatively or qualitatively.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Risk Assessment/Evaluation**: This involves comparing the identified and analyzed risk against predefined criteria such as acceptable risk levels and priorities. It's used to make decisions about the impact of the risk and whether it's acceptable or if it needs to be treated.

KNOWLEDGEGATE
http://www.knowledgegate.in/GATE

- **Risk Mitigation**: This is the process of prioritizing, evaluating, and implementing actions that reduce or control the risks. It can involve risk avoidance, risk transfer, risk acceptance, or risk limitation.

KNOWLEDGEGATE

http://www.knowledgegate.in/GATE

- **Risk Monitoring**: This is the process of monitoring identified and mitigated risks, as well as identifying new risks. This is an ongoing process throughout the lifecycle of the project.

- In a nutshell, Software Risk Analysis and Management aims to anticipate what might not go as planned and prepare a response to those scenarios, so that action can be taken to resolve risk events that do occur or to minimize their impact.

[KNOWLEDGEGATE]

http://www.knowledgegate.in/GATE

Module

Free Semester One Shot Exam Course

Topic

Artificial Intelligence

Subtopic

AI One Shot

Lesson

AI_Notes

ARTIFICIAL INTELLIGENCE

SEMESTER EXAM SERIES

IN 6 HOURS + NOTES

KNOWLEDGE GATE

Artificial Intelligence

- **Chapter-1** (INTRODUCTION)
- **Chapter-2** (PROBLEM SOLVING METHODS)
- **Chapter-3** (KNOWLEDGE REPRESENTATION)
- **Chapter-4** (SOFTWARE AGENTS)
- **Chapter-5** (APPLICATIONS)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Video chapters

- **Chapter-1 (**INTRODUCTION**):** Introduction-Definition - Future of Artificial Intelligence - Characteristics of Intelligent Agents - Typical Intelligent Agents - Problem Solving Approach to Typical Al problems.
- **Chapter-2 (**PROBLEM SOLVING METHODS**):** Problem solving Methods - Search Strategies- Uninformed - Informed - Heuristics - Local Search Algorithms and Optimization Problems - Searching with Partial Observations - Constraint Satisfaction Problems - Constraint Propagation - Backtracking Search - Game Playing - Optimal Decisions in Games - Alpha - Beta Pruning - Stochastic Games.
- **Chapter-3 (**KNOWLEDGE REPRESENTATION**):** First Order Predicate Logic - Prolog Programming - Unification - Forward Chaining - Backward Chaining - Resolution - Knowledge Representation - Ontological Engineering-Categories and Objects - Events - Mental Events and Mental Objects - Reasoning Systems for Categories - Reasoning with Default Information.
- **Chapter-4 (**SOFTWARE AGENTS**):** Architecture for Intelligent Agents - Agent communication - Negotiation and Bargaining - Argumentation among Agents - Trust and Reputation in Multi-agent systems.
- **Chapter-5 (**APPLICATIONS**):** Al applications - Language Models - Information Retrieval - Information Extraction - Natural Language Processing - Machine Translation - Speech Recognition - Robot -Hardware – Perception – Planning – Moving.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

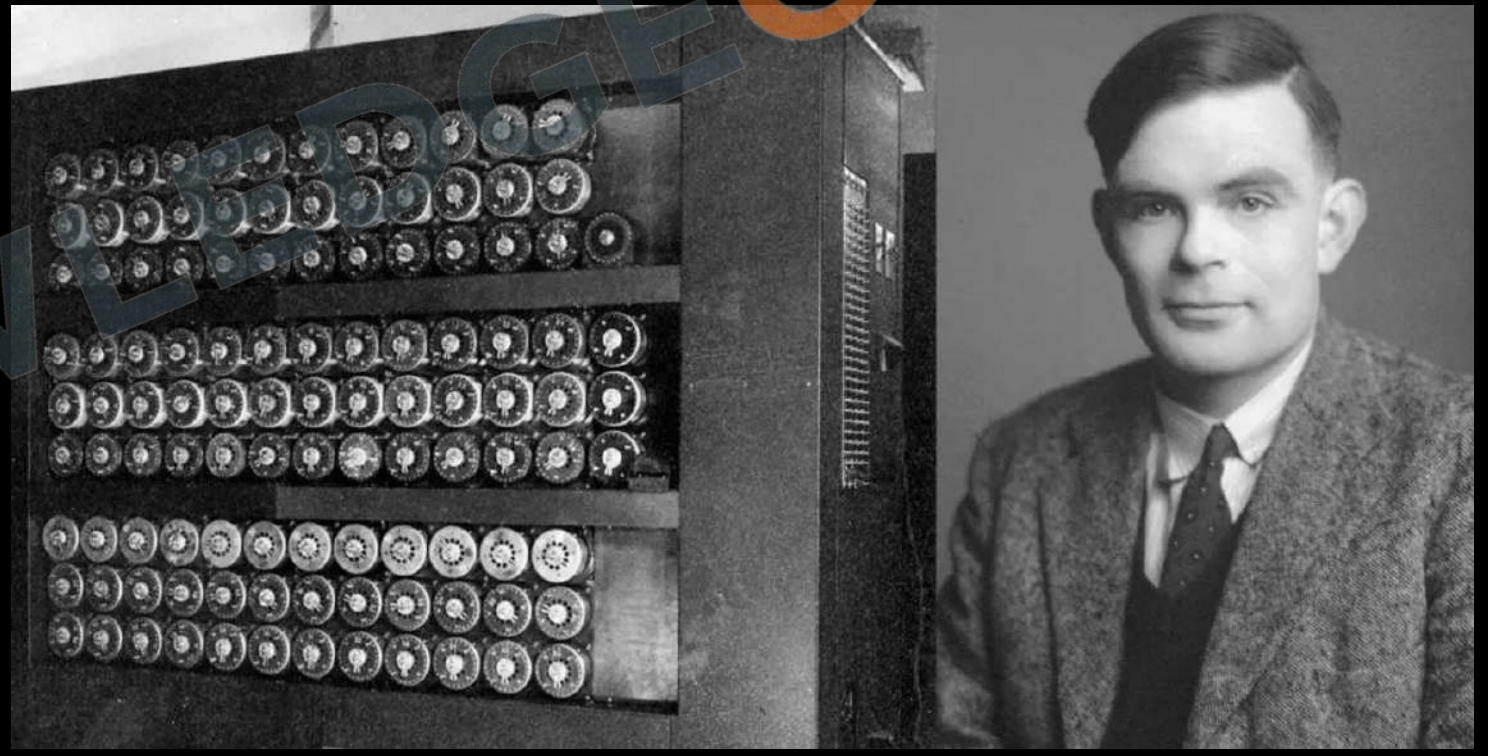
- **Chapter-1 (**INTRODUCTION**):** Introduction-Definition - Future of Artificial Intelligence - Characteristics of Intelligent Agents - Typical Intelligent Agents - Problem Solving Approach to Typical AI problems.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

History of AI

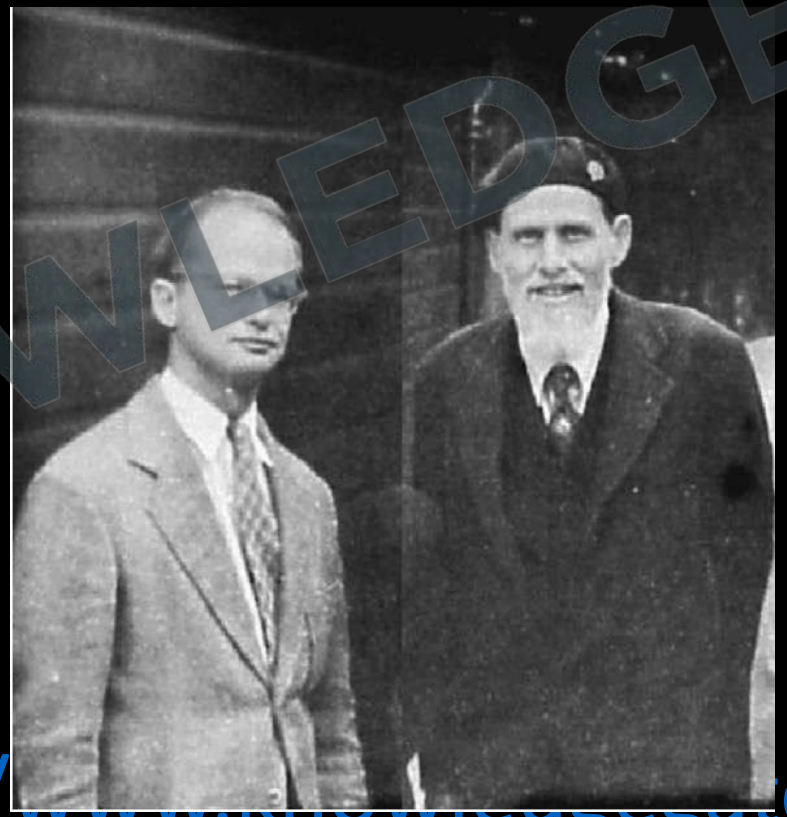
- During the Second World War, British computer scientist **Alan Turing** worked to crack the ‘Enigma’ code which was used by German forces to send messages securely. Alan Turing and his team created the Bombe machine that was used to decipher Enigma’s messages.
- The Enigma and Bombe Machines laid the foundations for Machine Learning.



KNOWLEDGE GATE

First work in AI

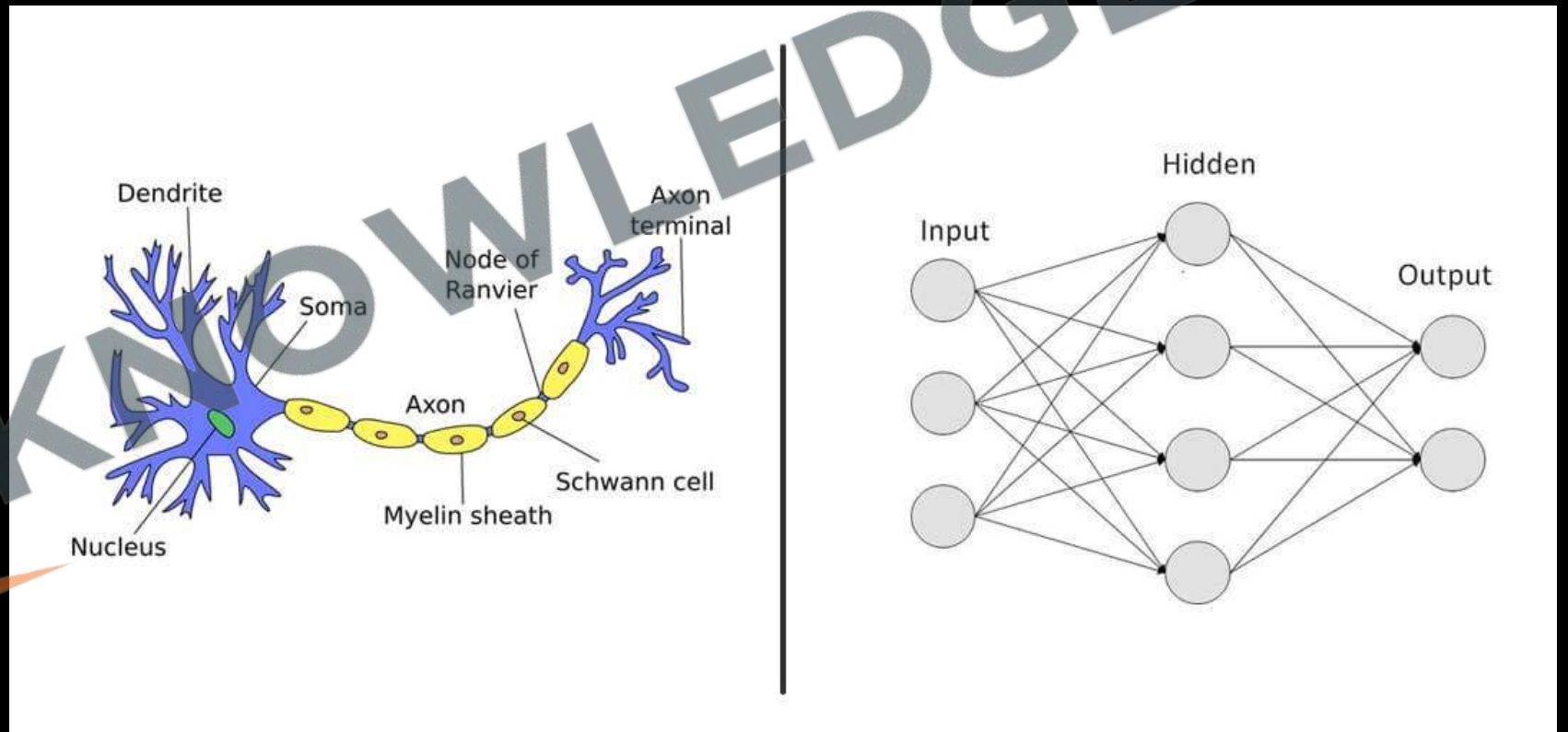
- The first work that is now generally recognized as AI was done by Warren McCulloh and Walter Pits in 1943. They drew on three sources:
  - **Knowledge of basic Physiology and Functions of Neurons in brain.**
  - **A formal analysis of propositional logic**
  - **Turing's Theory of Computation**



KNOWLEDGEGATE

Key Findings

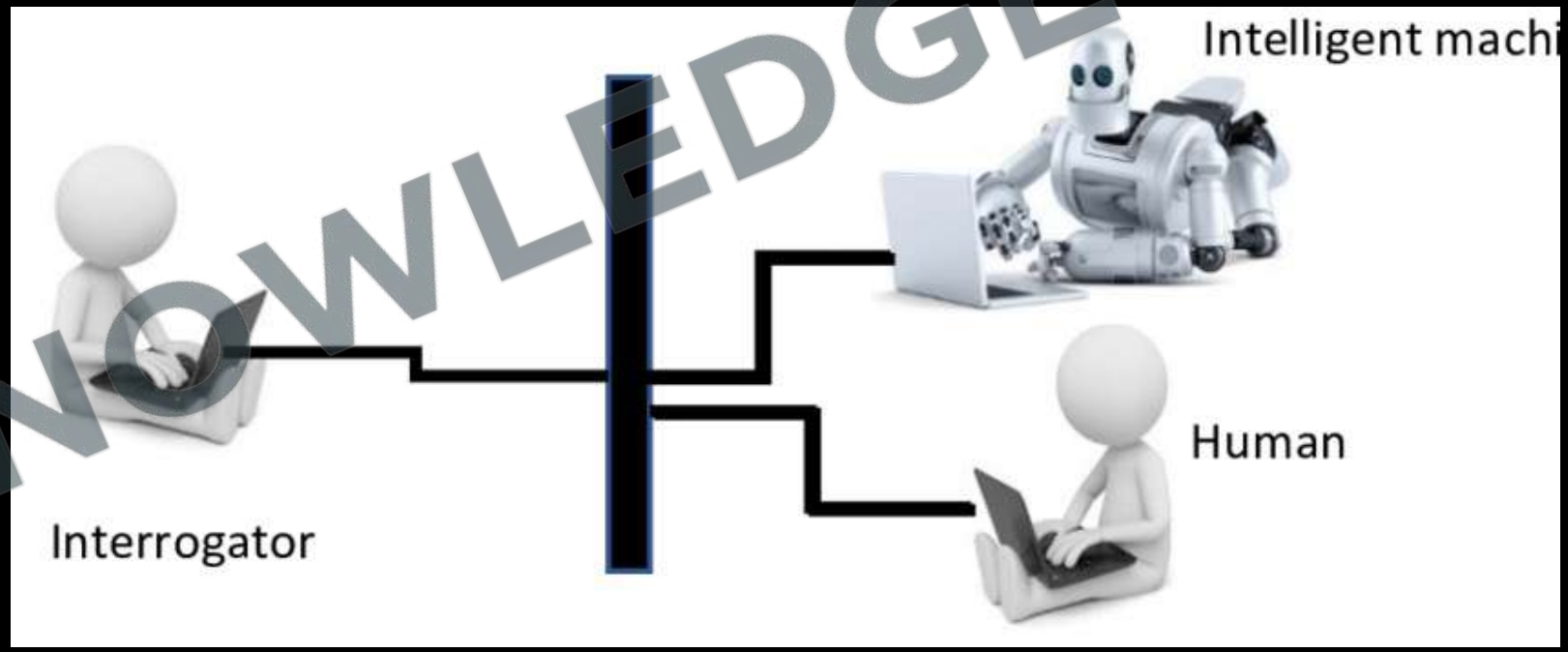
- They proposed a model of artificial neurons in which each neuron can be characterized as being **on** and **off**.
- They showed that any computable function could be computed by some network of connected neurons.
- They also suggested that suitably defined networks can also learn.



KNOWLEDGEGATE

Acting humanly: The Turing Test approach

- The Turing Test, proposed by Alan Turing (1950), was designed to provide a satisfactory operational definition of intelligence.
- A computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or from a computer.



KNOWLEDGEGATE

- Marven Minsky and Dean Edmonds from Harvard built the first neural network computer in 1950, it was called “***The SNARC***”.

KNOWLEDGEGATE

The Birth of AI

- The term AI was coined in 1956 by a computer scientist “John McCarthy” in Dartmouth Conference organized by him and three other scientists. He is often known as “***Father of AI”.*** McCarthy in 1958 developed LISP. There were so many discoveries going on in the field of AI from the year 1952 to 1969.

This picture was originally downloaded from exhibits.stanford.edu. The author is unknown. This picture was altered to enhance the clarity of its content and to add comments. This picture is used for research, teaching and education and it is a transformative use. We consider that the use of this picture in this context represents fair use. Please, do not try to take down this picture before considering whether our conduct constitutes fair use!

http://www.knowledgegate.in/gate

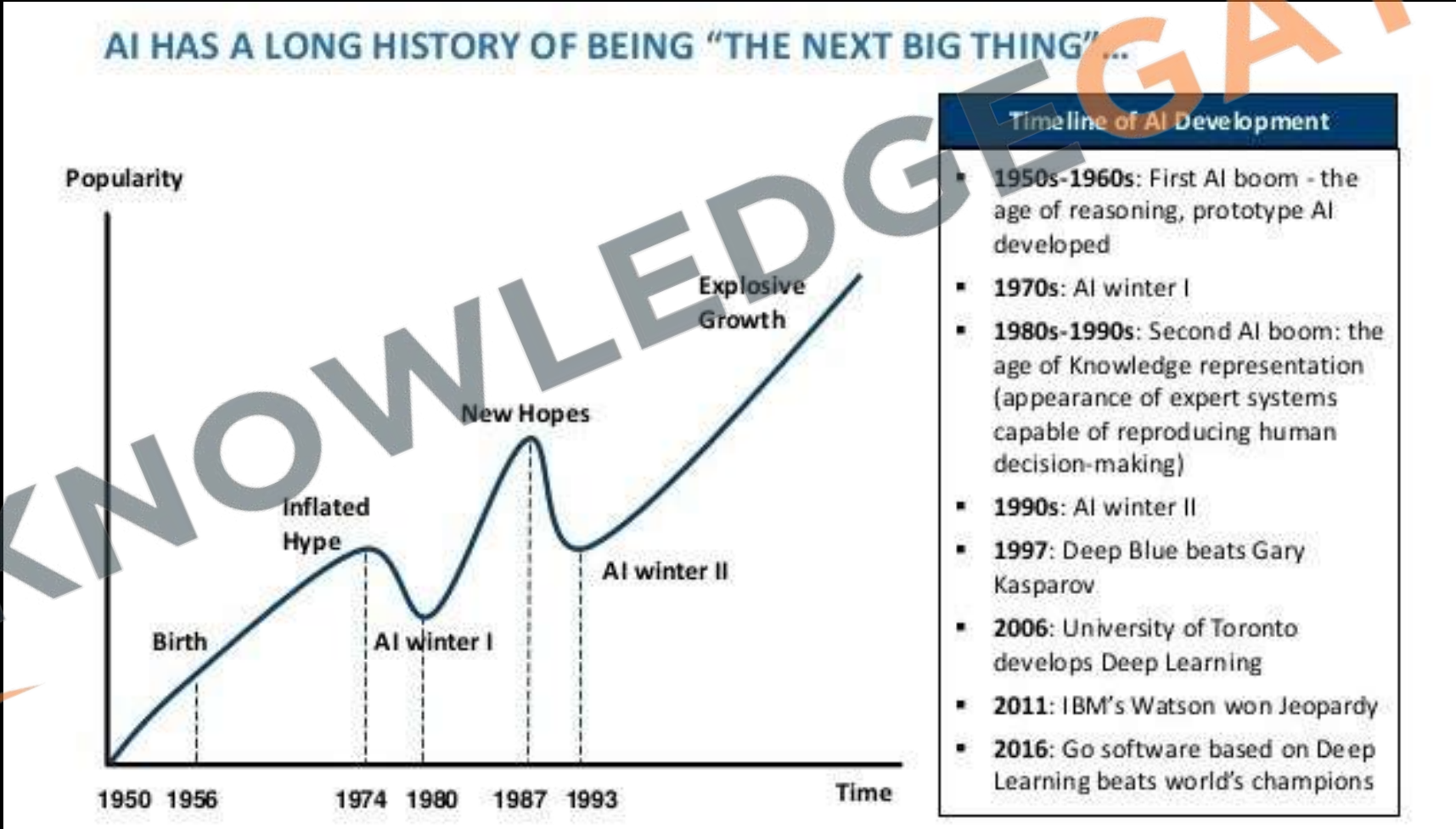
AI Winter (1974-1980)

- In 1970's AI was subjected to criticism and financial setbacks. The expectations from AI was quite high due to heavy optimism but the researchers had failed to materialize the promised results.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Boom in AI after 1980 (1980-87)

- In 1980's "Expert Systems" an AI program was adopted by many corporations. **The Expert system was a program that answers questions or solves problems from a particular domain using logical rules derived from the knowledge of experts.**



KNOWLEDGEGATE

- **1997:** IBM's Deep Blue beats the world chess champion, Garry Kasparov, a significant milestone for AI in games.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

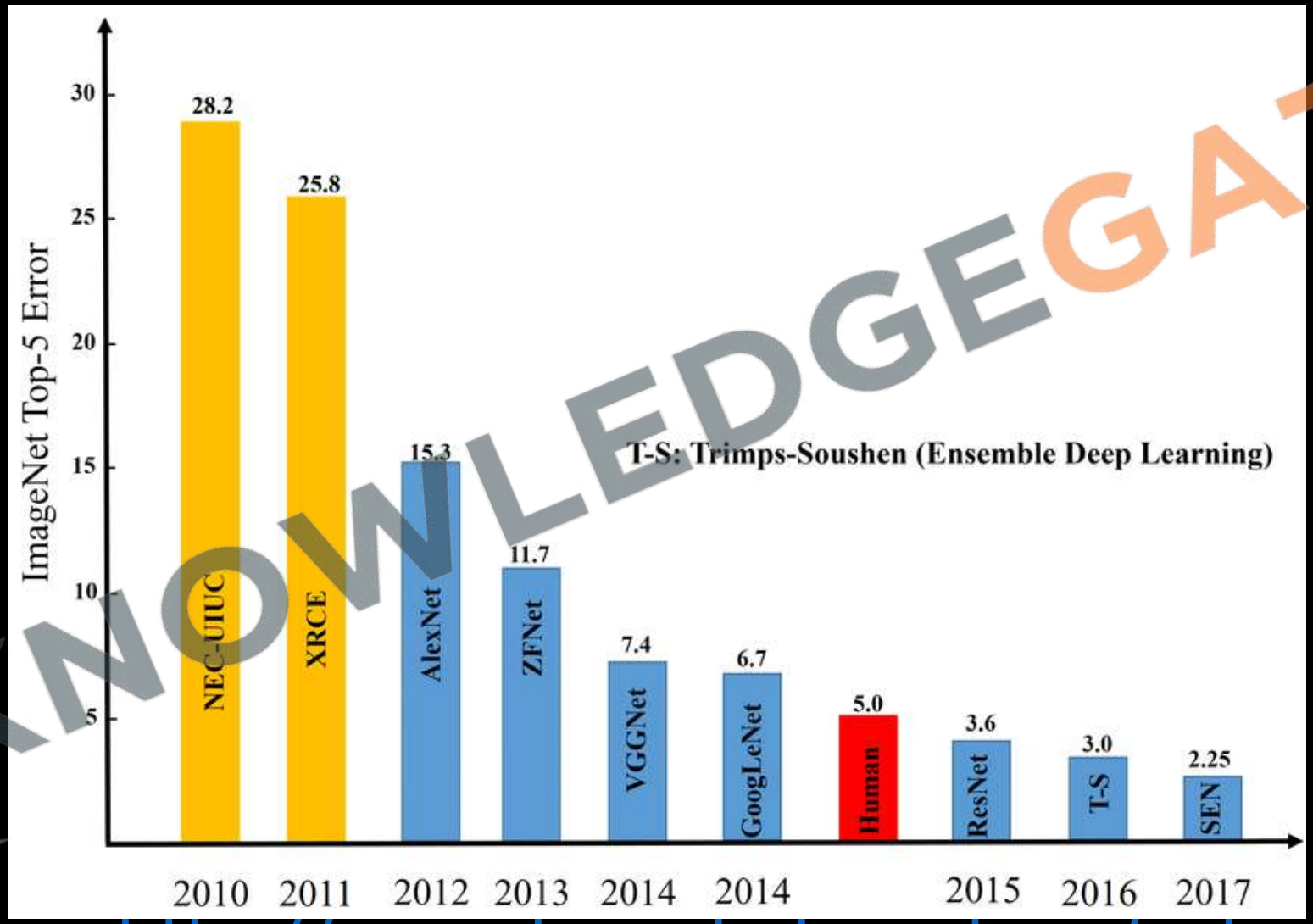
- **2005:** Stanley, the self-driving car developed by Stanford University, wins the DARPA Grand Challenge.

KNOWLEDGEGATE

- **2011:** IBM's Watson wins Jeopardy! against former winners, showing the capabilities of AI in natural language processing.

KNOWLEDGEGATE

- **2015:** A breakthrough in deep learning occurs when a neural network wins the ImageNet competition, significantly improving the state of image recognition.



KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **2016:** Google's AlphaGo defeats world champion Go player Lee Sedol, a significant milestone due to the complexity of the game.

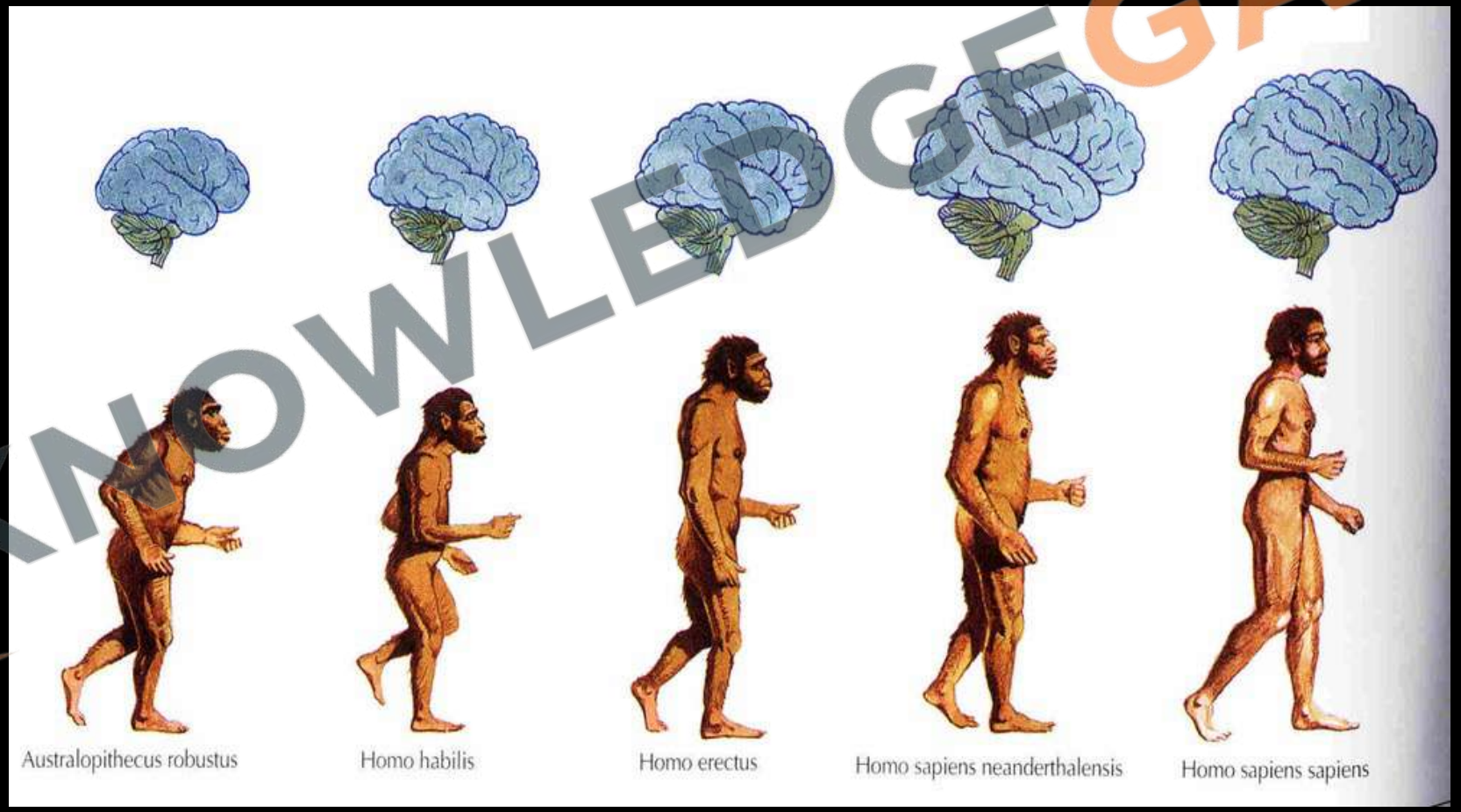
KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **2018:** OpenAI's GPT showcases a leap in natural language processing capabilities.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Intelligence and AI:** Intelligence in general is the "**Ability to acquire and apply knowledge and skills".** We are known as Homo-Sapiens "Homo- ***Man*** and Sapiens - ***Wise***" it is because our intelligence is so important to us.
- For thousands of years we have tried to understand how our brain functions. The field of AI goes beyond this: It not only attempts to understand it but also tries to build intelligent entities.



KNOWLEDGEGATE

What is AI

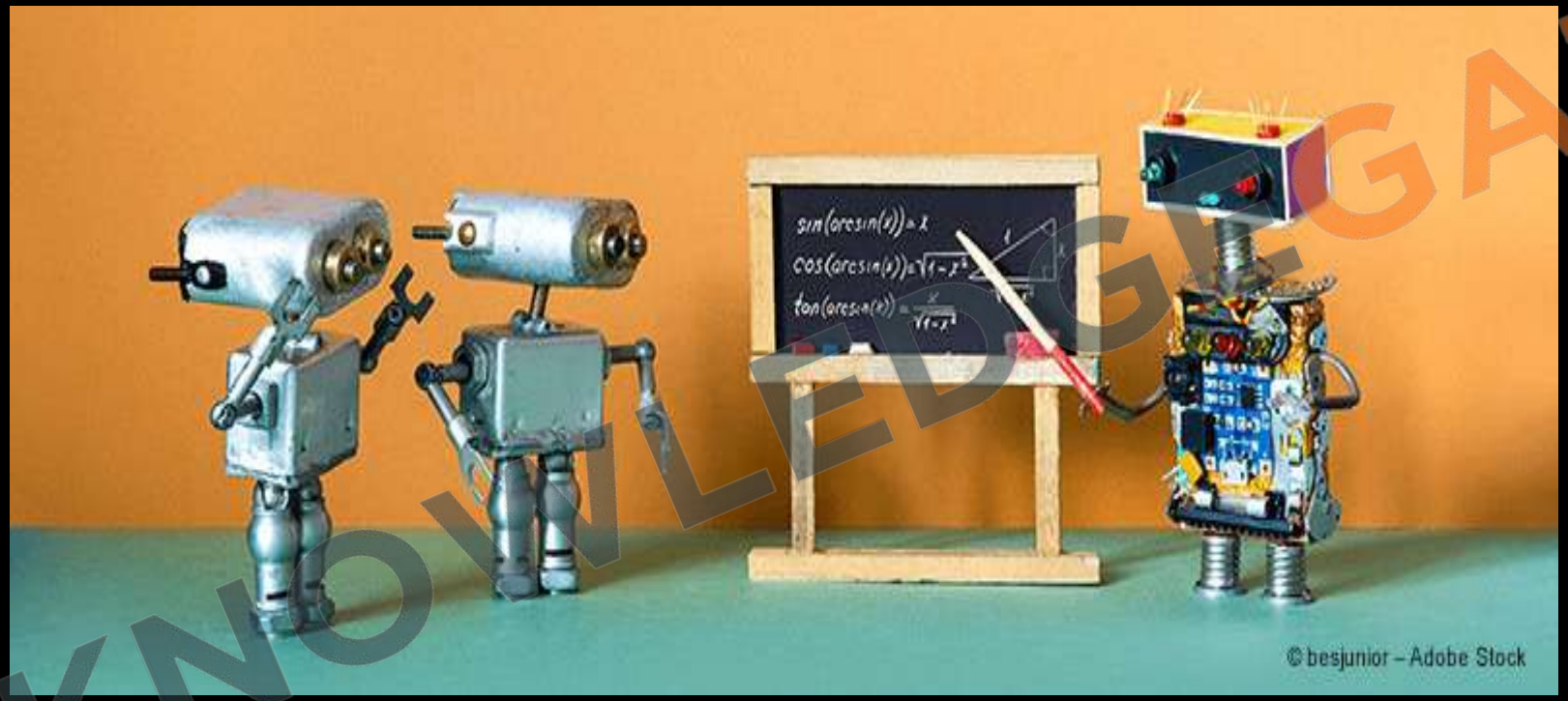
- There is no universally accepted definition of AI in general refers to the simulation of human intelligence processes by machines, especially computer systems.
- **Sub-fields:** It includes various sub-fields like machine learning, where machines learn from experience, natural language processing, which is about interaction with human language, and computer vision, where AI interprets and understands visual data.

KNOWLEDGE GATE

- **<u>Problem-solving</u>:** AI is used to solve complex problems and perform tasks that would usually require human intelligence. This could be anything from **recognizing speech or images, making decisions, or translating languages**.

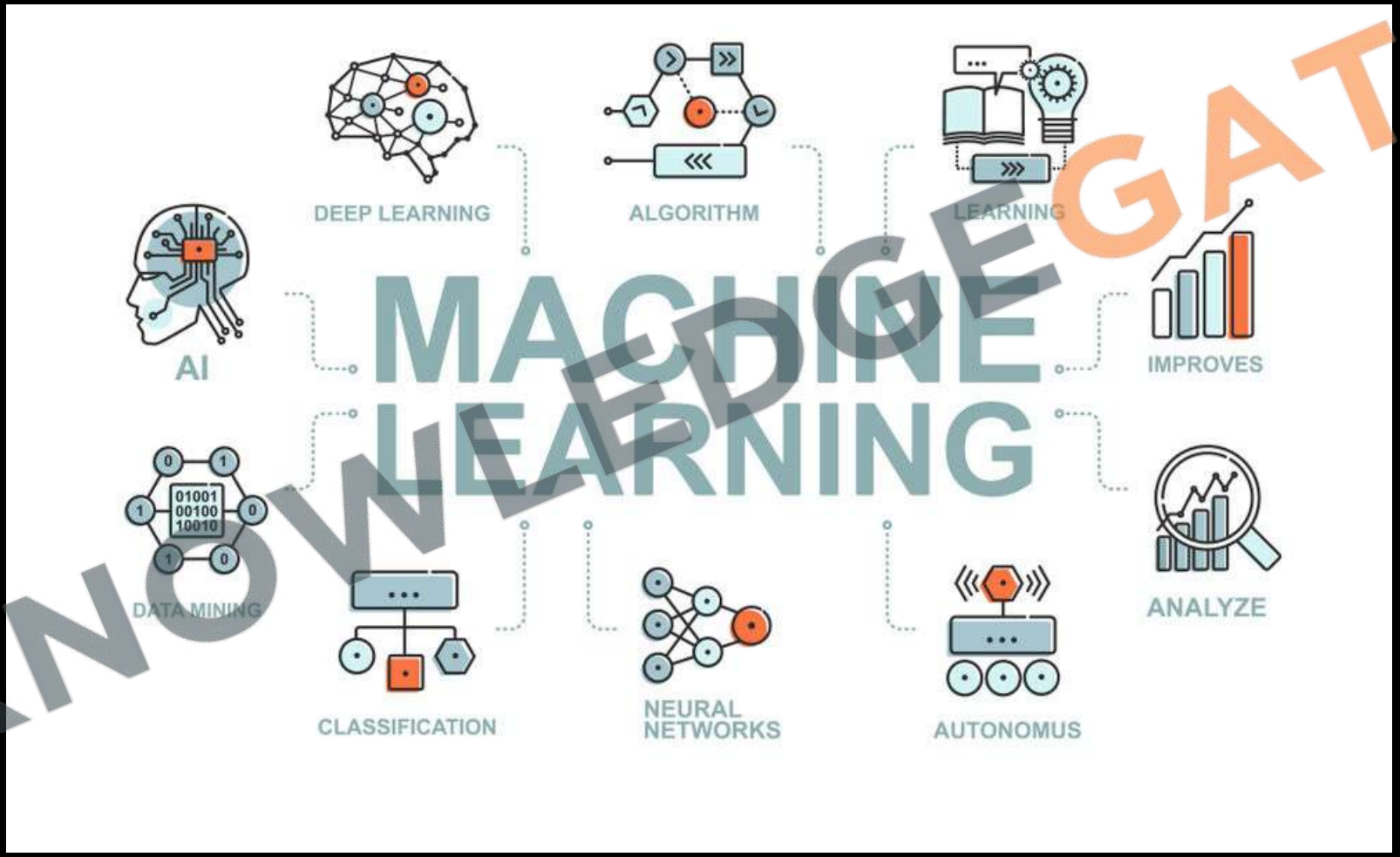
KNOWLEDGEGATE

- **Training:** AI systems are trained using large amounts of data. For instance, an AI used for recognizing speech would be trained on a large database of spoken language.



KNOWLEDGEGATE

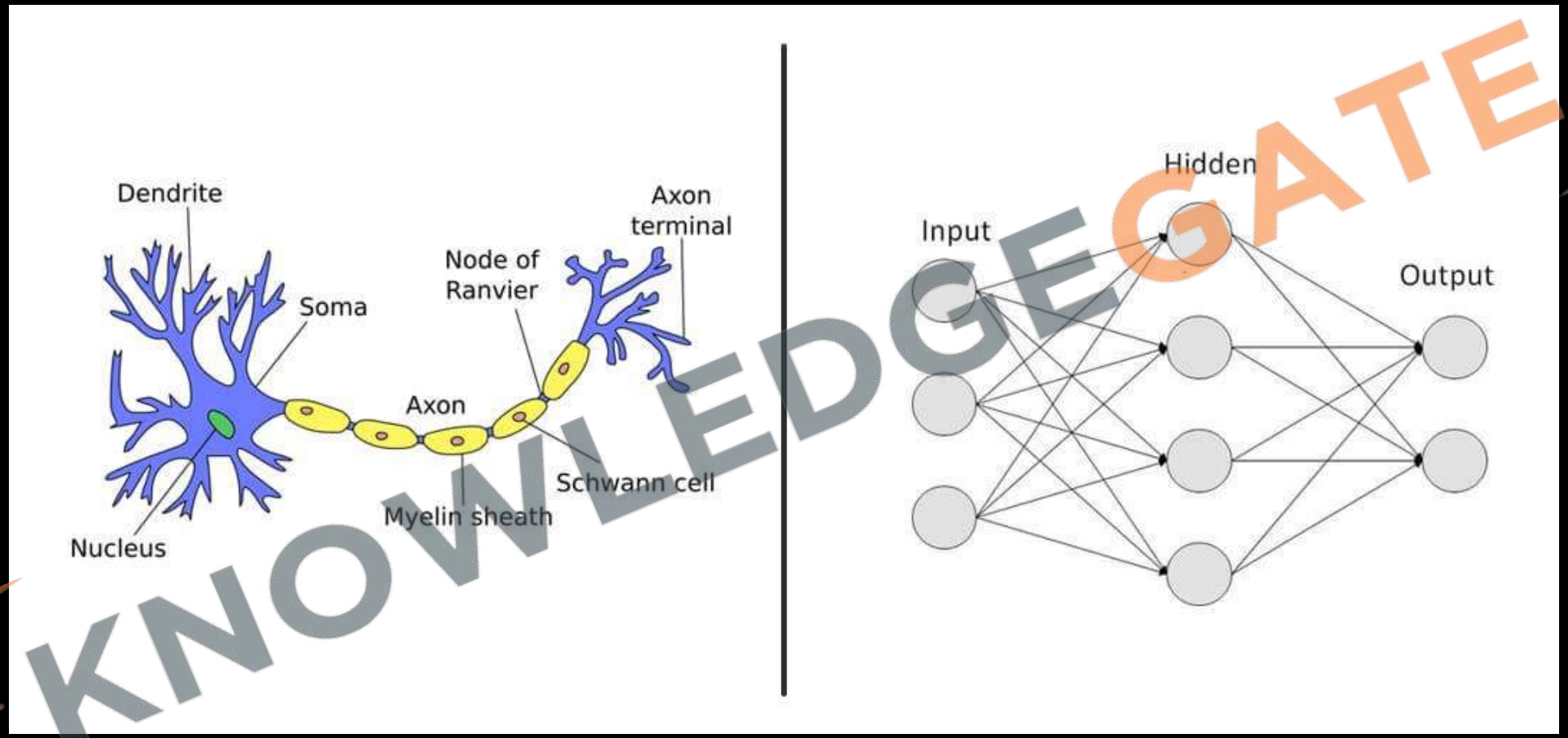
- **<u>Machine Learning</u>:** One of the most common types of AI in use today is based on machine learning, where AI gets smarter the more data it is given.



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Neural Networks:** AI systems often use neural networks, which are designed to mimic the human brain's ability to recognize patterns and make decisions.



KNOWLEDGEGATE

- **Benefits:** AI has many potential benefits, such as improving efficiency and productivity, making faster and more informed decisions, and performing tasks that are dangerous for humans. The future potential of AI is vast, with ongoing research in fields like self-driving cars, voice assistants, predictive healthcare, and many more.

KNOWLEDGEGATE

Artificial Intelligence (AI) can be classified into four categories based on capabilities and functionalities:

- **Reactive Machines:**
  - These are the most basic type of AI.
  - They can react to specific situations or inputs but don't have memory or past experience to influence their decisions.
  - Example: IBM's Deep Blue, the chess-playing AI.

KNOWLEDGE GATE

http gate

- **Limited Memory:**
  - These AI systems can use past experiences or historical data to make current decisions.
  - They have a temporary or limited memory to store past information and predictions.
  - Example: Self-driving cars.

KNOWLEDGEGATE

- **Theory of Mind:**
  - This is a more advanced type of AI that researchers are still working to develop.
  - These systems would understand emotions, people, beliefs, and be able to interact socially.
  - Example: More advanced personal assistants that can understand human emotions and react accordingly.

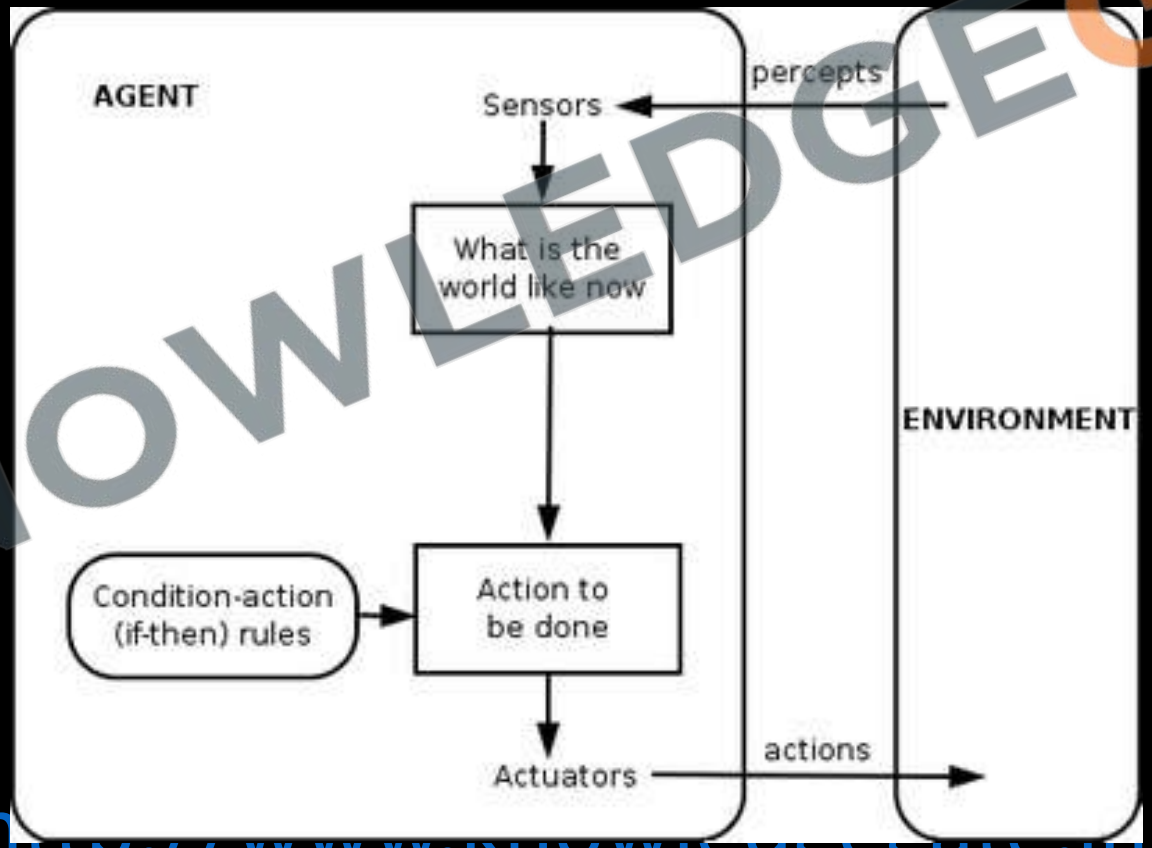
KNOWLEDGEGATE

- **Self-aware AI:**
  - This is the most advanced form of AI, which is still theoretical and not yet realized.
  - These systems would have their own consciousness, self-awareness, and emotions.
  - They would be smarter and more capable than the human mind.

KNOWLEDGE GATE

AGENTS AND ENVIRONMENTS

- An agent is anything that can be viewed as perceiving its **environment** through sensors and acting upon that environment through **actuators.**
- A human agent has eyes, ears, and other organs for sensors are hands, legs, vocal tract, and so on for actuators.
- A robotic agent might have cameras and infrared range finders for sensors and various motors for actuators.



KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Percept:** This refers to what an AI agent (like a robot or a computer program) senses or perceives at a specific moment.
- **Percept Sequence:** This is the total history or record of what the AI agent has sensed or perceived since it started operating. An AI agent's decisions or actions at any moment can be influenced by the entire percept sequence it has experienced so far. However, it can't base decisions on information it hasn't perceived or sensed.
- **Agent Function:** This is a rule or set of rules that determines what action an AI agent takes. based on its percept sequence.
- **Agent Program:** This is the practical, real-world implementation of the agent function.

KNOWLEDGEGATE

Concept of Rationality in AI

- **Rational Agent:** An AI entity that always chooses the best action.
- **Action Selection:** Based on its percepts, a rational agent selects an action that's expected to maximize its performance measure, using its percept history and innate knowledge.
- **Performance Measure:** Not a fixed measure for all tasks and agents, it varies based on the specific task at hand.
- **Designing Performance Measures:** Ideally, these are designed based on desired outcomes in the environment, not preconceived ideas about the agent's behavior.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Understanding Task Environments and Rational Agents

- **Task Environment:** a task environment is essentially the setting or context in which an AI agent operates. This could be a physical environment, like a warehouse where a robot is sorting boxes, or a virtual environment, like a chess game where an AI is playing against a human opponent. each task environment presents a unique set of challenges or problems that the AI agent needs to navigate.
- the task environment of a chess game includes the rules of the game, the current state of the board, and the goal of checkmating the opponent's king. The "problem" in this case is determining the best move to make at each step to win the game. In the chess example, a rational agent would be an AI system that can analyze the state of the game, consider all possible moves, and choose the one that maximizes its chances of winning.
- A rational agent, then, is the "solution" to this problem: it's an AI system designed to make the best possible decisions given the circumstances of the task environment.

KNOWLEDGEGATE

- Acronymically we call it **PEAS (P**erformance, **E**nvironment, **A**ctuators, **S**ensors**)** description. **PEAS for agent Taxi Driver:**

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

KNOWLEDGEGATE

Properties of task environments

- **Fully Observable vs. Partially Observable:**
  - **Fully Observable:** Every relevant aspect in the environment is visible to the agent.
  - **Partially Observable:** Not all aspects are visible. The agent doesn't have complete information about the environment's state because of noisy and inaccurate sensors or because parts of the state are simply missing from the sensor data.
  - **Unobservable:** If the agent has no sensors at all then the environment is **unobservable**.

KNOWLEDGEGATE

Single Agent vs. Multi-Agent:

- **Single Agent:** Only one agent is acting in the environment, like playing solitaire.
- **Multi-Agent:** Multiple agents are interacting with the environment and potentially with each other, like playing chess.
- Driving taxi requires avoiding collisions maximizes the performance measure of all agents, so it is a partially **cooperative** multiagent environment. It is also partially competitive because only one car can occupy a parking space.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Deterministic vs. Stochastic:

- **Deterministic:** The next state of the environment is entirely determined by the current state and the action executed by the agent.
- **Stochastic:** The next state cannot be entirely predicted and might depend on probability, like playing poker.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Episodic vs. Sequential:

- **Episodic:** The agent's experience is divided into episodes. Each episode is independent of others, like in image recognition tasks.
- **Sequential:** The current decision could influence all future decisions, like in a game of chess.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Static vs. Dynamic:

- **Static:** The environment doesn't change while the agent is deciding, like a game of tic-tac-toe.
- **Dynamic:** The environment can change while the agent is making a decision, like in stock trading.

KNOWLEDGEGATE

Discrete vs. Continuous:

- **Discrete:** There are a finite number of actions the agent can take, like moves in a chess game.
- **Continuous:** The actions or states can vary in a continuous way, like a self-driving car controlling its speed.

KNOWLEDGEGATE

Known vs. Unknown:

- **Known:** The agent knows the rules of the environment, like in a game of checkers.
- **Unknown:** The agent doesn't fully understand the rules of the environment and must learn them, like navigating a new city.

KNOWLEDGEGATE

- Four basic kinds of agent programs that embody the principles underlying almost all intelligent systems are:
  - **Simple reflex agents;**
  - **Model-based reflex agents;**
  - **Goal-based agents; and**
  - **Utility-based agents.**

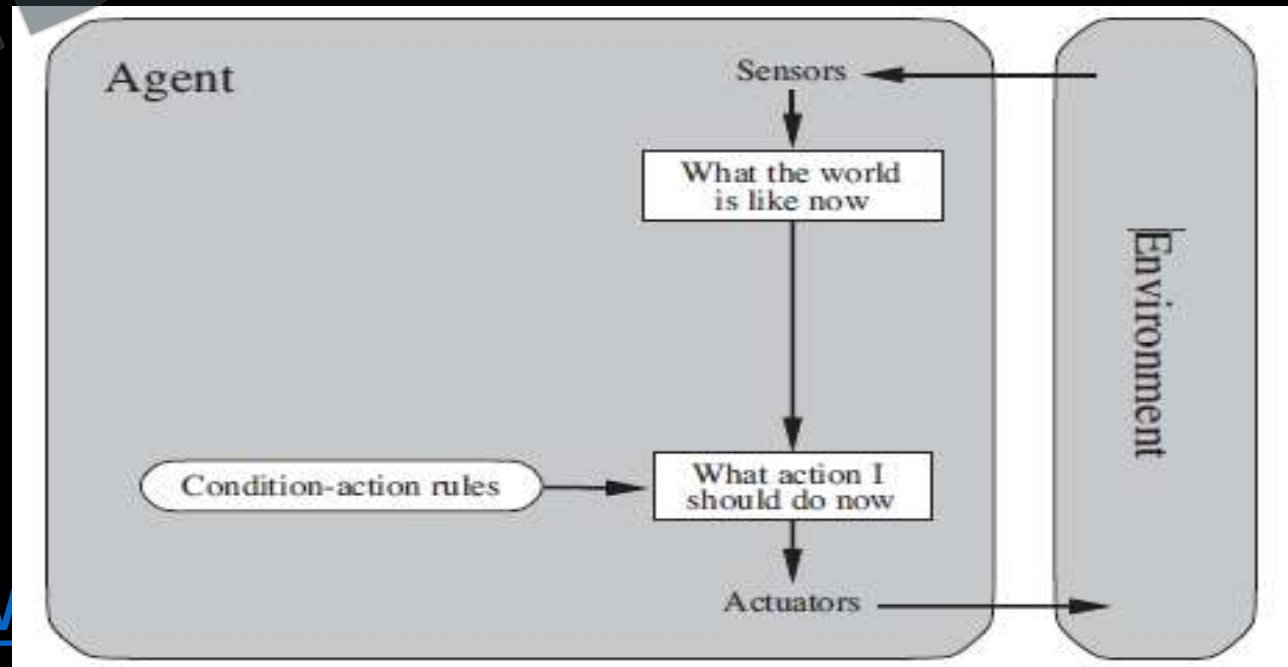
KNOWLEDGEGATE

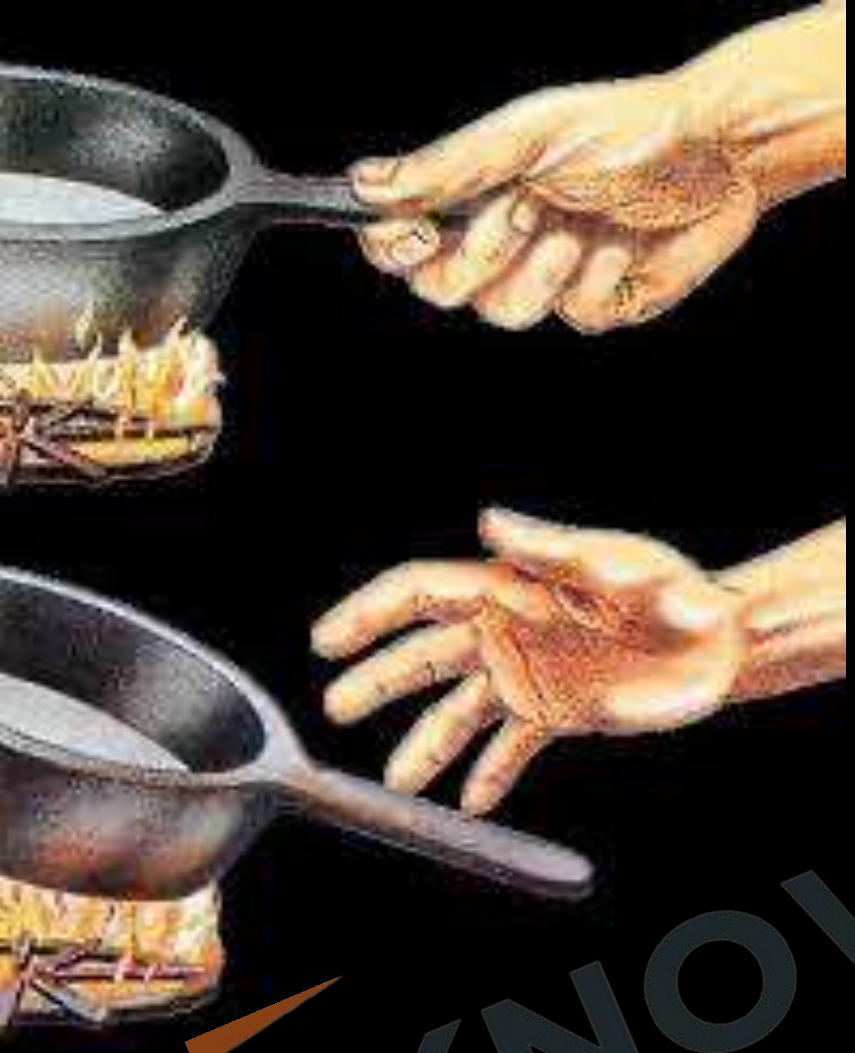
http://www.knowledgegate.in/gate

Simple reflex agents

- **Basic Functioning:** Simple reflex agents act based on current percepts, ignoring historical percepts.
- **Condition-Action Rule:** These agents follow "if-then" rules, like "if someone is in front of the car then initiate-braking."
- **Simplicity and Limitations:** While such agents are simple to design do not perform complex calculation, their intelligence and memory is limited.
- **Problems with Unobservability:** Issues arise if all necessary information isn't observable, like different configurations of car lights.

KNOWLEDGEGATE



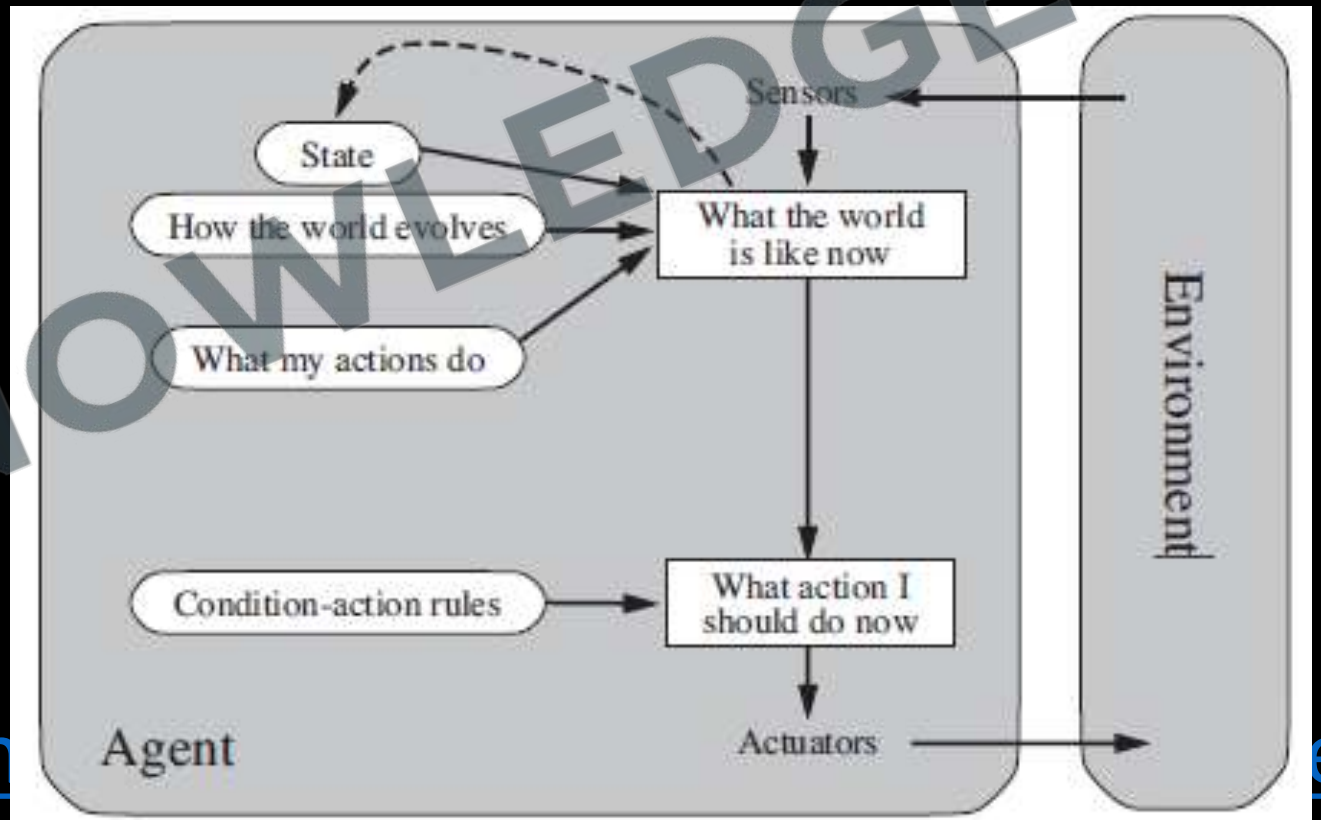


KNOWLEDGEGATE

http://www.knowledgegate.in/gate

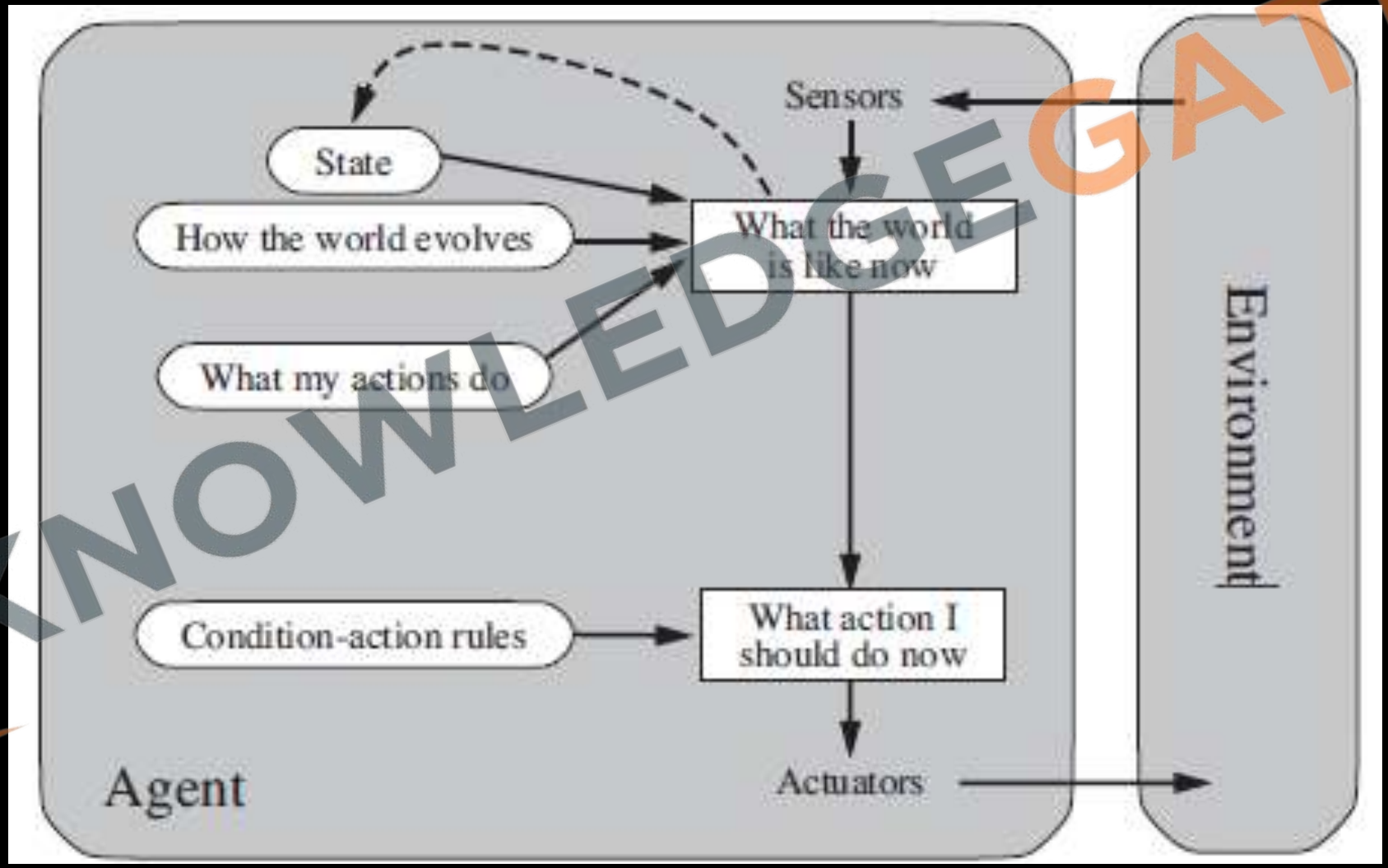
Model-based reflex agents

- **Handling Partial Observability:** Model-based agents is designed to work in partially observable environment.
- **Internal State:** These agents maintain an internal state that reflects unobserved aspects of the environment, based on percept history.
- **Updating Internal State:** This process requires knowledge of how the world evolves independently of the agent and how the agent's actions affect the world.



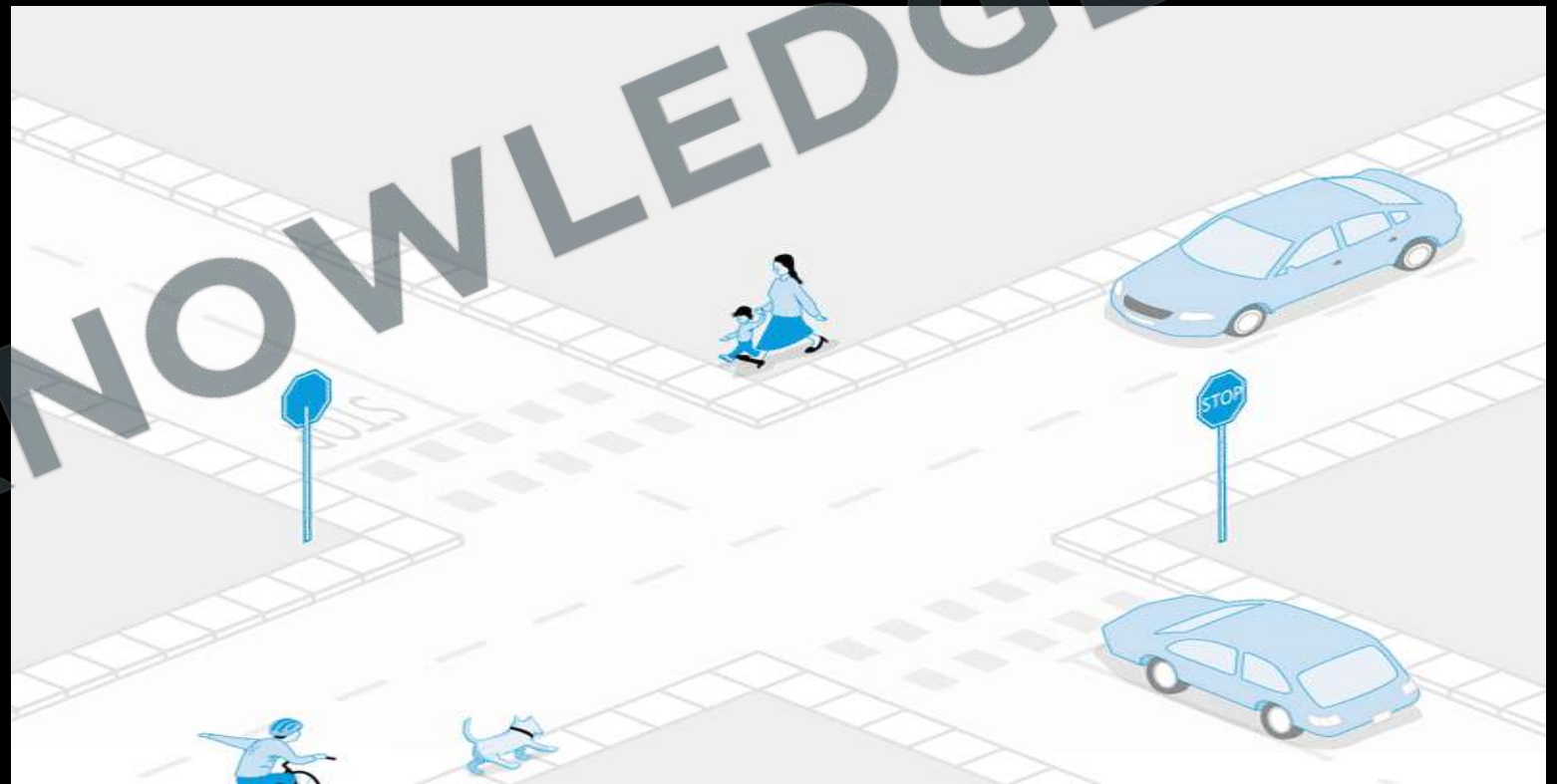
KNOWLEDGEGATE

- **Model of the World:** Agents use this knowledge, whether simple or complex, to form a model of how the world works, and then take actions accordingly.



KNOWLEDGEGATE

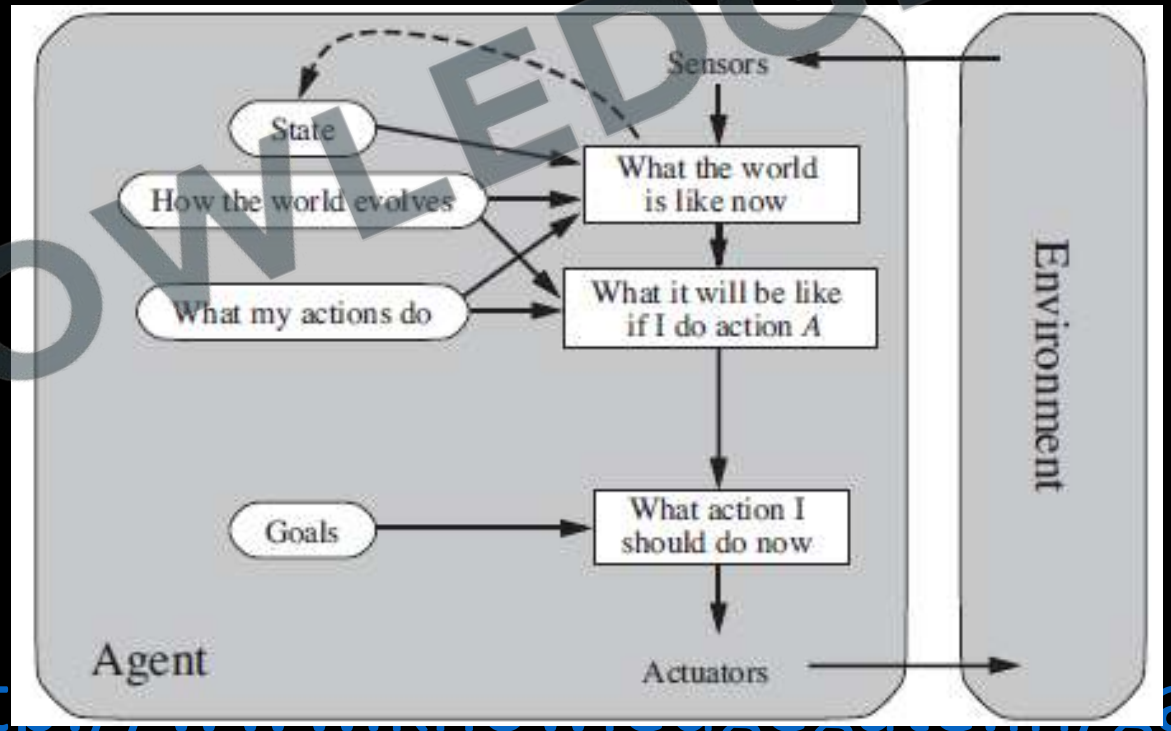
•**Internal Model**: The car has a map of the roads, traffic rules, and typical traffic patterns.
•**Sensors**: Uses cameras and sensors to perceive traffic lights, road signs, other vehicles, pedestrians, and road conditions.
•**Updating Knowledge**: It updates its model in real-time as it drives, learning current traffic conditions, construction work, or detours.
•**Decision Making**: If the car approaches a red light, it stops. When the light turns green and the path is clear, it proceeds.
•**Learning Over Time**: The car refines its internal model to improve predictions, like estimating the time it will take to reach destinations considering the current traffic.



KNOWLEDGEGATE

Goal-based agents

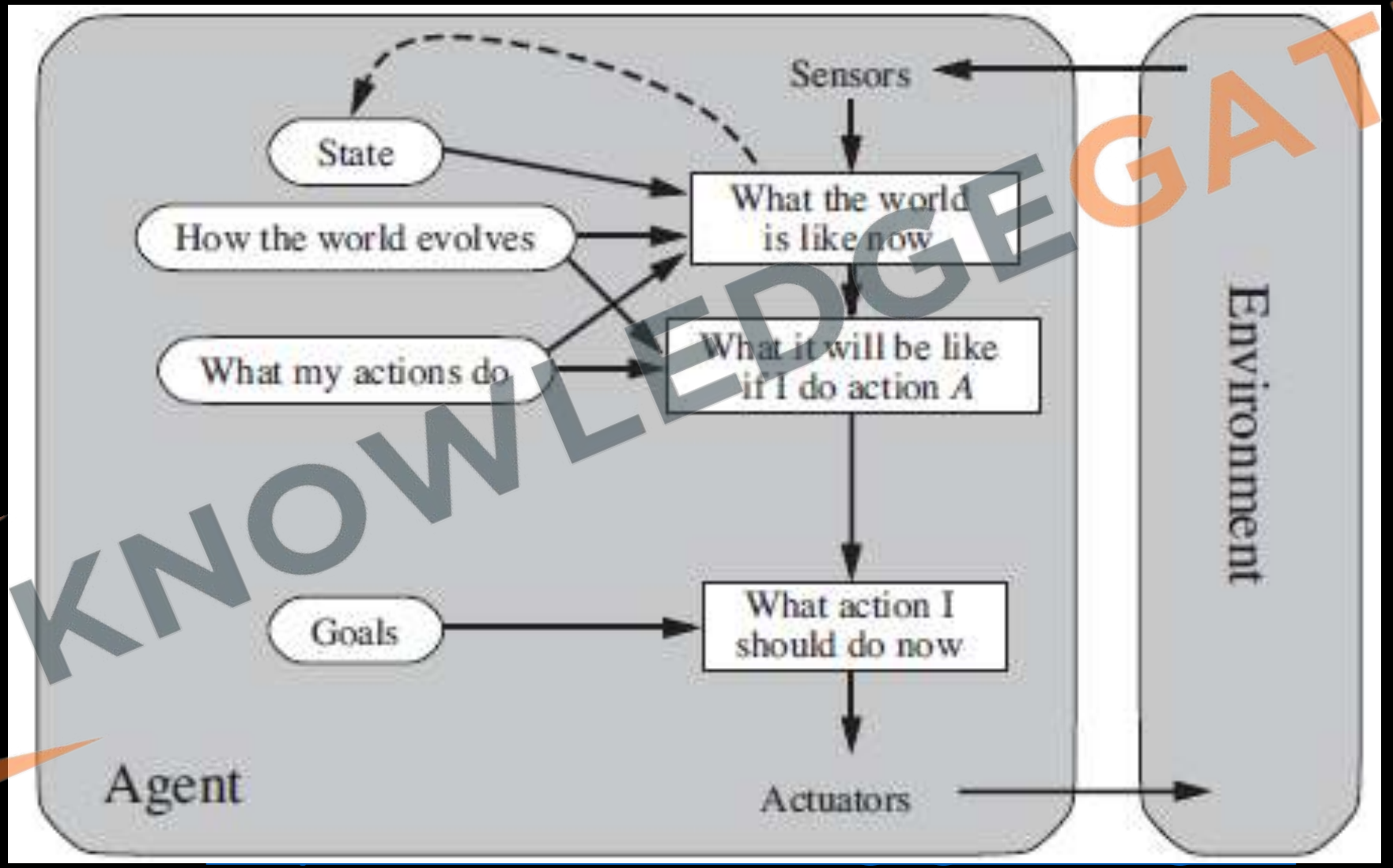
- **Goal Information:** It is an extension of model based reflex agent, here we have a goal information that describes desirable situations, guiding their actions.
- **Action Decision:** These agents use their internal model and goal information to decide actions. For example, a taxi at a junction chooses a direction based on its destination.
- **Action Selection:** Goal-based agents may need to consider sequences of actions to achieve the goal. This can be straightforward or complex depending on the goal.



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Example:** If it starts to rain, a goal-based agent can update its knowledge about how the brakes will operate, adjusting behavior accordingly. This is more flexible than a reflex agent that requires explicit rules for each new situation.



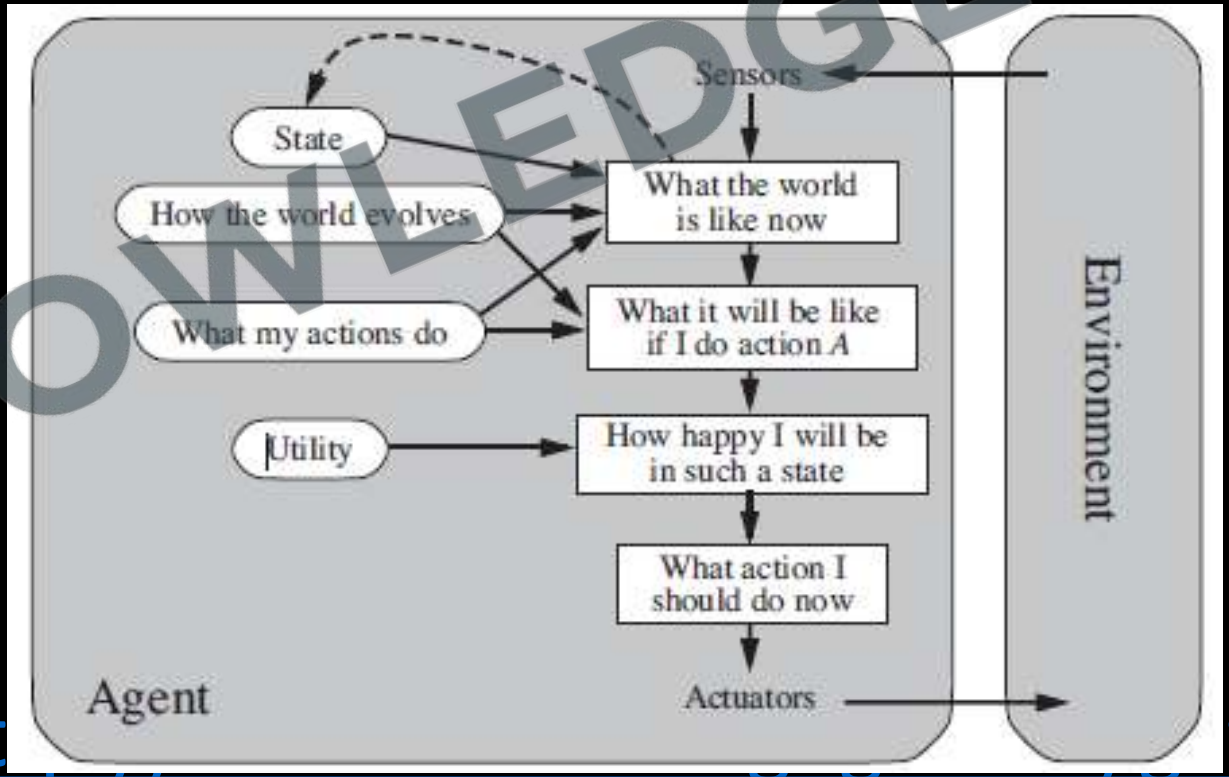
KNOWLEDGE GATE

Example: Automated Vacuum Cleaner
Goal:- Clean the entire floor area.
Perception:- Detects location and whether a spot is clean or dirty.
Actions:
- **Move:** Changes position to cover the floor.
- **Clean:** Activates vacuum on dirty spots.
Decision-making:- Chooses actions based on its goal and what it perceives (e.g., cleans dirty spots, moves to new areas if current is clean).
Goal Achievement:- Stops operating once no dirty spots are detected, indicating the floor is clean.

KNOWLEDGE GATE

Utility-based agents

- **Performance Measure:** Beyond having a goal, utility-based agents have a performance measure that compares different world states based on how "desirable" they are.
- **Utility:** To sound more scientific, the term "utility" is used to denote how happy or satisfied an agent is with a state.
- **Example:** Many routes can get a taxi to its destination, but some are quicker, safer, or cheaper. Utility allows the agent to distinguish between these routes.



KNOWLEDGEGATE

Example: Smart Thermostat

Goal:

- Maintain comfortable home temperature.

Perception:

- Senses current temperature and occupant preferences.

Actions:

- **Heat**, **Cool**, or **Maintain** temperature.

Utility Calculation:

- Balances occupant comfort with energy efficiency.

Decision-making:

- Chooses action to maximize comfort while minimizing energy use.

Utility Achievement:

- Adjusts temperature to keep high utility, considering both comfort and efficiency.

KNOWLEDGE GATE

Feature	Simple Reflex Agents	Model-Based Reflex Agents	Goal-Based Agents	Utility-Based Agents
Basis of Operation	Operate on the current percept only.	Use an internal model to keep track of the world state.	Act to achieve defined goals.	Act to maximize a utility function.
Environment Interaction	React to stimuli with conditioned actions.	Update internal state from current state and action history.	Consider future consequences of actions.	Evaluate success based on a utility function.
Flexibility	Very limited; cannot handle new situations well.	More flexible, can handle unseen scenarios to an extent.	Flexible; can adapt actions to achieve goals in changing environments.	Highly flexible; aims for optimal performance.
Learning Ability	None; they do not learn from past actions.	Limited; can improve the model with new experiences.	Can learn better strategies to achieve goals.	Can adjust utility function based on outcomes.
Example	Thermostat controlling a heating system.	A car with anti-lock braking system.	Chess-playing AI considering future moves.	Investment AI maximizing portfolio return.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Key characteristics of a learning agent

- Adaptability
- Memory
- Feedback Sensitivity
- Decision-Making Ability
- Exploration and Exploitation
- Generalization
- Performance Improvement Over Time
- Problem-Solving Skills

KNOWLEDGEGATE

- **<u>Chapter-2</u> (**PROBLEM SOLVING METHODS**):** Problem solving Methods - Search Strategies- Uninformed - Informed - Heuristics - Local Search Algorithms and Optimization Problems - Searching with Partial Observations - Constraint Satisfaction Problems - Constraint Propagation - Backtracking Search - Game Playing - Optimal Decisions in Games - Alpha - Beta Pruning - Stochastic Games.

KNOWLEDGEGATE

Problem Spaces States Space Representation

The concept of "Problem Spaces States Space Representation" is a method used in artificial intelligence to formulate and solve problems. We use this representation for several reasons:

- **Structure:** It provides a structured way of defining all possible configurations that can occur while solving a problem.
- **Search:** It allows algorithms to search through possible states.
- **Optimization and Decision Making:** Helps in finding the most efficient path or sequence of actions to reach a desired goal state from an initial state.
- **Clarity:** Visually illustrates the complexity of a problem, making it easier to understand and explain.
- **Scalability:** It can be applied to simple as well as very complex problems, scaling from basic puzzles to real-world issues in robotics or planning.

[KNOWLEDGEGATE]

Problem Spaces States Space Representation

Defining a problem as a state space search

- A problem can be defined formally by five components:
  - {Initial State, Action(s), Result(s, a), Goal Test, Path Cost Function}

- **Initial State**:
  - The agent is at the starting point, for example, 'Home'.
- **Action(s)**:
  - The possible actions include choosing a road to travel from one intersection to another.
- **Result(s, a)**:
  - For a chosen action 'a' (road taken), the result would be a new state representing the agent's new location (another intersection).
- **Goal Test**:
  - A function to determine if the agent has reached the destination, 'Work'.
- **Path Cost Function**:
  - A function that adds up the distance (or time) to travel from the initial state to the current state via the chosen paths. The objective is to minimize this cost.

[KNOWLEDGEGATE]

http://www.knowledgegate.in/gate

Search Strategies

- Search strategies refer to systematic methods and approaches used to explore and find relevant information or solutions within a given search space or dataset. Parameters for Evaluating Search Technique Performance:
  - **Completeness**: Determines if the search technique guarantees finding a solution if one exists within the search space.
  - **Time and Space Complexity**: Evaluates the efficiency of the search technique in terms of the time and space required to find a solution.
  - **Optimality**: Determines whether the search technique can find the best or optimal solution among all possible solutions.

KNOWLEDGEGATE

Uninformed Search(Brute Force Search/Blind Search/Exhaustive Search)

- Uninformed search strategies explore the search space without any specific information or heuristics about the problem. Here we proceed in a systematic way by exploring nodes in some predetermined order or simply by selecting nodes at random.
  - **Advantage**:
    - **Simplicity**: Uninformed search strategies are generally easy to implement and understand.
  - **Disadvantage**:
    - **Inefficiency**: Without additional information, uninformed search strategies may require an extensive search, leading to inefficiency in terms of time and space.
  - **Examples**:
    - **Breadth First Search**
    - **Depth First Search**
    - **Uniform Cost Search**

KNOWLEDGEGATE

Informed Search(Heuristic Search)

- Informed search strategies utilize domain-specific information or heuristics to guide the search towards more promising paths. Here we have knowledge such as how far we are from the goal, path cost, how to reach to goal node etc. This knowledge helps agents to explore less to the search space and find more efficiently the goal node.
- **Advantage**:
  - **Efficiency**: By leveraging domain knowledge, informed search strategies can make informed decisions and focus the search on more relevant areas, leading to faster convergence to a solution.
- **Disadvantage**:
  - **Heuristic Accuracy**: The effectiveness of informed search strategies heavily relies on the quality and accuracy of the chosen heuristic function. An inaccurate or misleading heuristic can lead to suboptimal or incorrect solutions.
- **Example**:
  - **Hill Climbing**
  - **Best First Search**
  - **A* Algorithm**

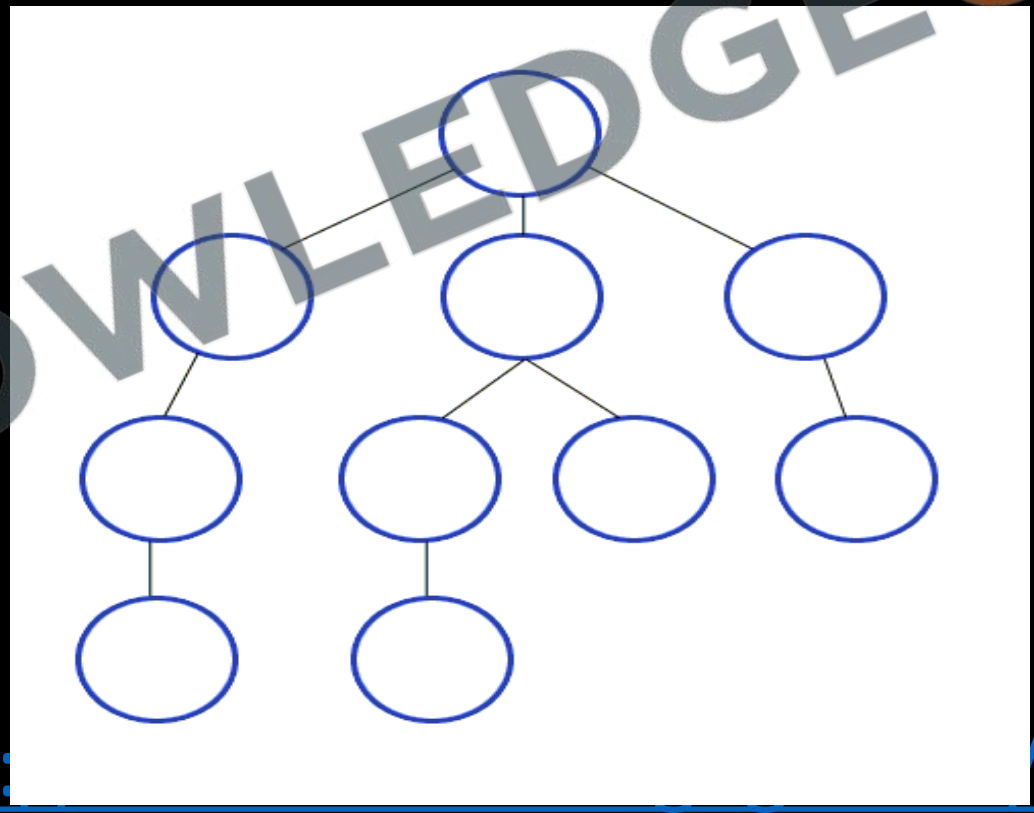
KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Aspect	Uninformed Search	Informed Search
Knowledge	Searches without any prior knowledge.	Uses knowledge, such as heuristics, to guide the search.
Time Efficiency	Generally more time-consuming.	Finds solutions quicker by prioritizing certain paths.
Complexity	Higher complexity due to lack of information, affecting time and space complexity.	Reduced complexity and typically more efficient in both time and space due to informed decisions.
Example Techniques	Depth-First Search (DFS), Breadth-First Search (BFS).	A* Search, Heuristic Depth-First Search, Best-First Search.
Solution Approach	Explores paths blindly.	Explores paths strategically towards the goal.

KNOWLEDGEGATE

BFS

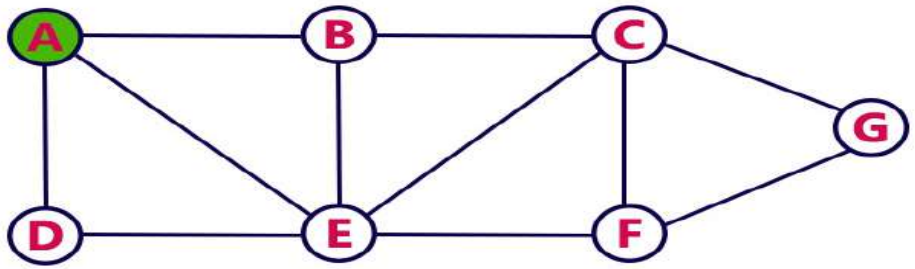
- Breadth-First Search is an uninformed search strategy that explores all neighboring nodes at the current level before moving to the next level starting from the root.
- This is achieved very simply by using a FIFO queue. New nodes (which are always deeper than their parents) go to the back of the queue, and old nodes, which are shallower than the new nodes, get expanded first.



KNOWLEDGEGATE

Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Insert **A** into the Queue.

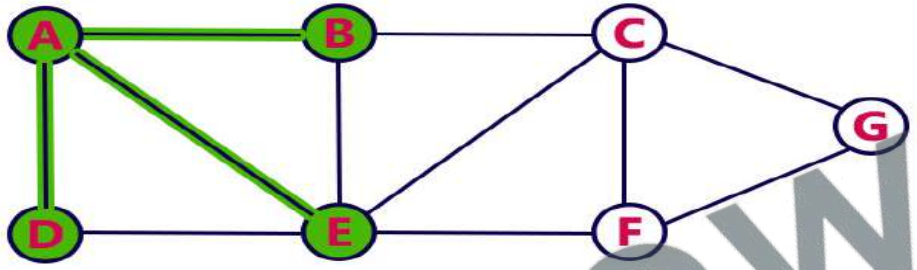


Queue

A						

Step 2:

- Visit all adjacent vertices of **A** which are not visited (**D**, **E**, **B**).
- Insert newly visited vertices into the Queue and delete A from the Queue..

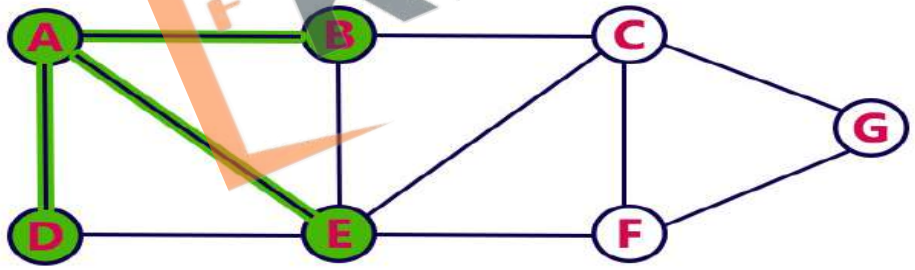


Queue

	D	E	B			

Step 3:

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.



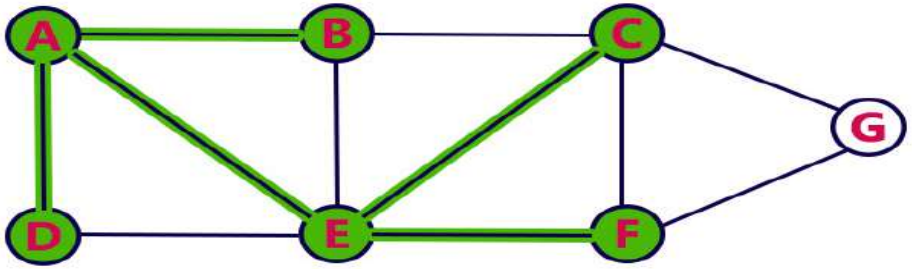
Queue

		E	B			

KNOWLEDGEGATE
gate

Step 4:

- Visit all adjacent vertices of **E** which are not visited (**C**, **F**).
- Insert newly visited vertices into the Queue and delete E from the Queue.

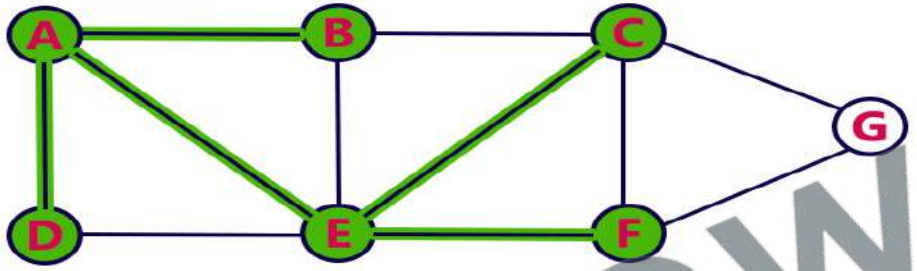


Queue

			B	C	F	

Step 5:

- Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
- Delete **B** from the Queue.

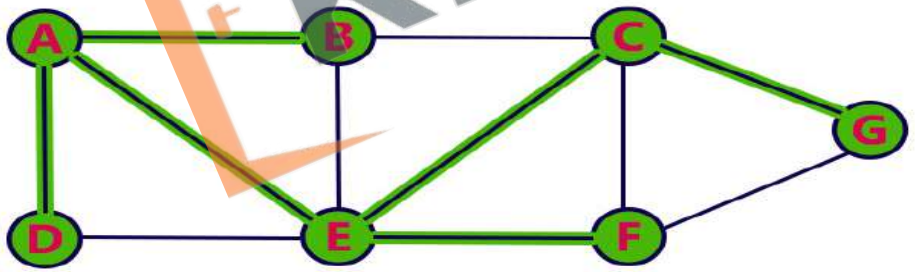


Queue

				C	F	

Step 6:

- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.



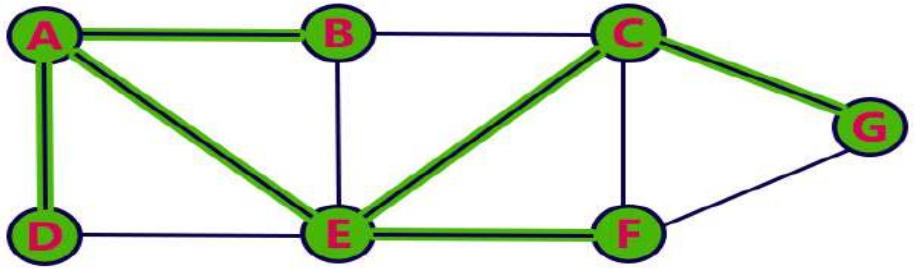
Queue

					F	G

KNOWLEDGEGATE
gate

Step 7:

- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.

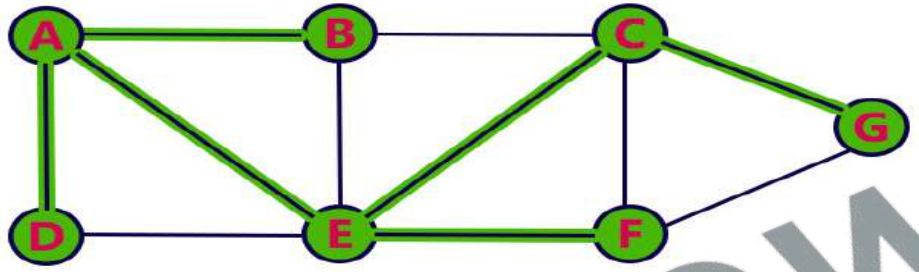


Queue

						G

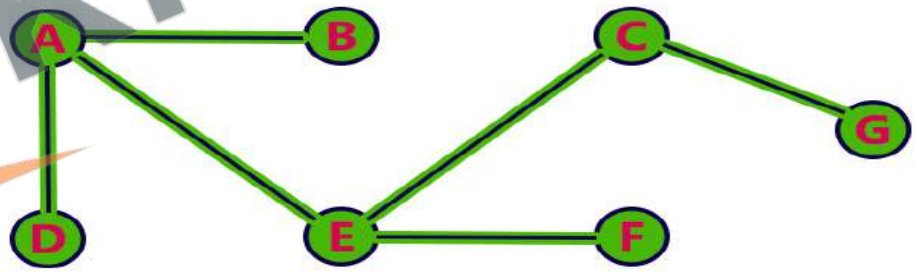
Step 8:

- Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
- Delete **G** from the Queue.



Queue

- Queue became Empty. So, stop the BFS process.
- Final result of BFS is a Spanning Tree as shown below...



KNOWLEDGEGATE
/gate

Algorithm:

1. Start with the initial state as the root node.
2. Enqueue the root node into a queue.
3. While the queue is not empty, do the following:
    1. Dequeue a node from the front of the queue.
    2. Expand the node and enqueue its unvisited neighboring nodes.
    3. Mark the dequeued node as visited.
4. Repeat steps 3 until the queue is empty or a goal state is found.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Advantages**:
  - Guarantees finding the shortest path if one exists in an unweighted graph.
  - Closer nodes are discovered before moving to farther ones.

- **Disadvantages**:
  - Inefficient in terms of time and space for large search spaces.
  - Not suitable for scenarios where the cost or weight of edges is not uniform.

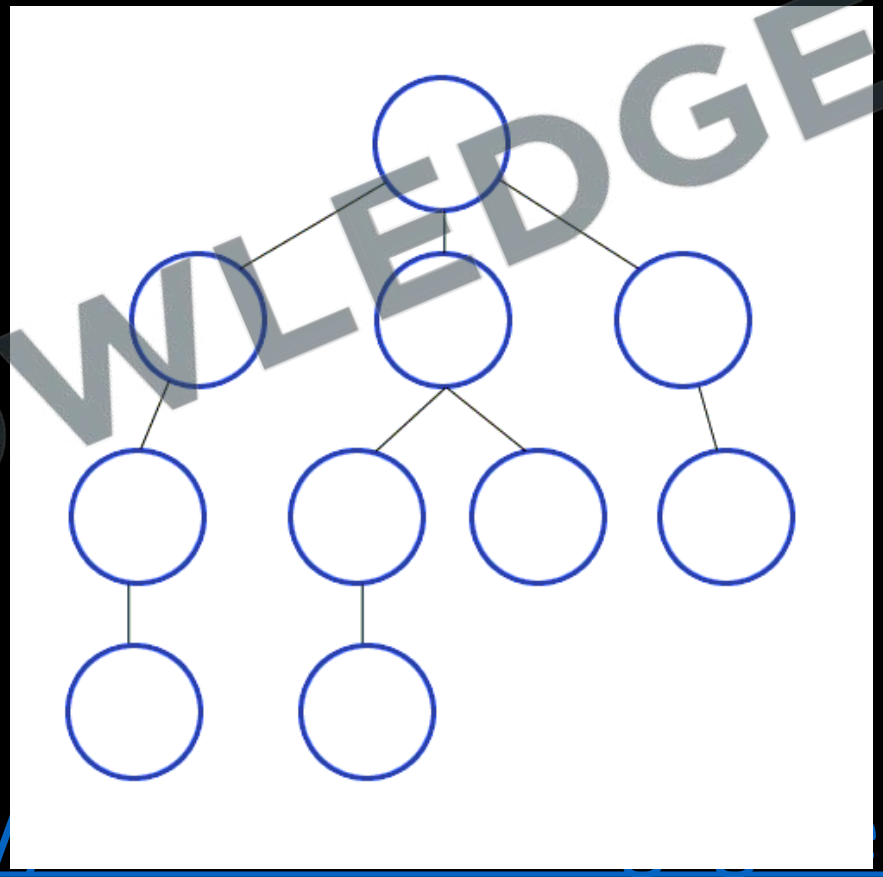
KNOWLEDGEGATE

- **Completeness**:
  - Breadth-First Search is complete if the search space is finite or if there is a solution within a finite depth.
- **Optimality**:
  - Breadth-First Search is optimal in terms of finding the shortest path in an unweighted or uniformly weighted graph.
- **Time Complexity**:
  - The time complexity of Breadth-First Search is $O(b^d)$, where b is the branching factor and d is the depth of the shallowest goal node.
- **Space Complexity**:
  - The space complexity of Breadth-First Search is $O(b^d)$, as it stores all the nodes at each level in the search tree in memory.

KNOWLEDGEGATE

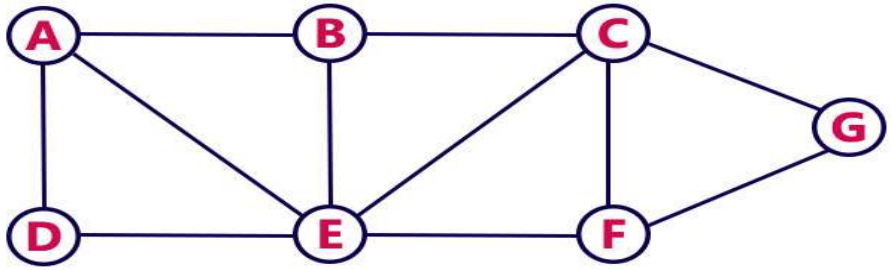
Depth-first search

- Depth-First Search is an uninformed search strategy that explores as far as possible along each branch before backtracking. It traverses the depth of a search tree or graph before exploring the neighboring nodes.



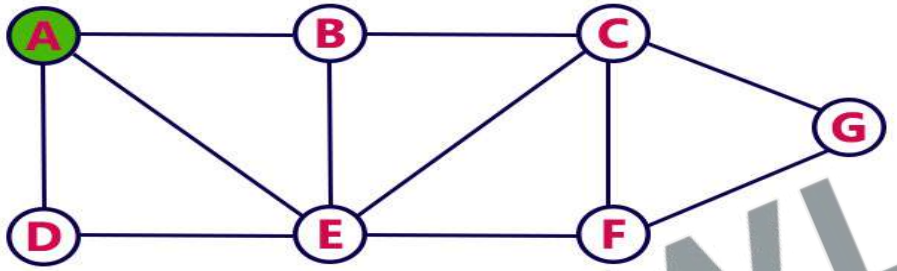
KNOWLEDGEGATE

Consider the following example graph to perform DFS traversal



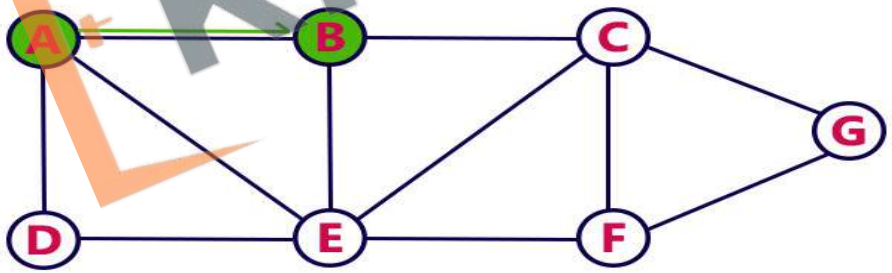
Step 1:

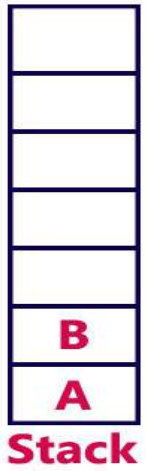
- Select the vertex **A** as starting point (visit **A**).
- Push **A** on to the Stack.



Step 2:

- Visit any adjacent vertex of **A** which is not visited (**B**).
- Push newly visited vertex B on to the Stack.



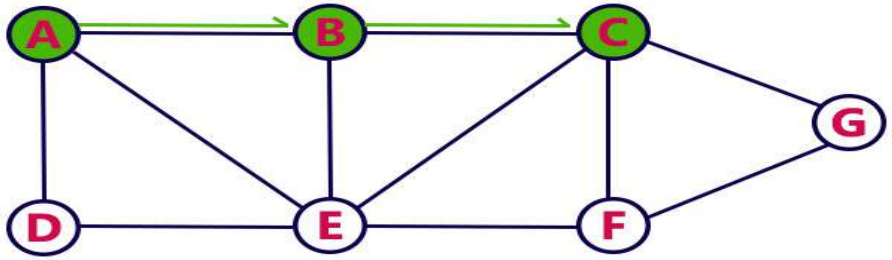


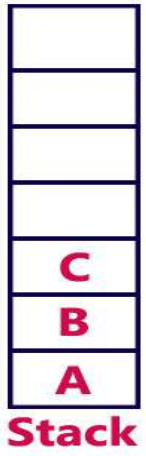
KNOWLEDGEGATE

gate

Step 3:

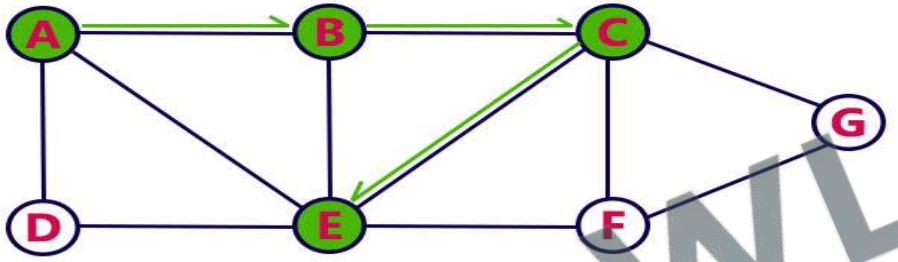
- Visit any adjacent vertext of **B** which is not visited (**C**).
- Push C on to the Stack.





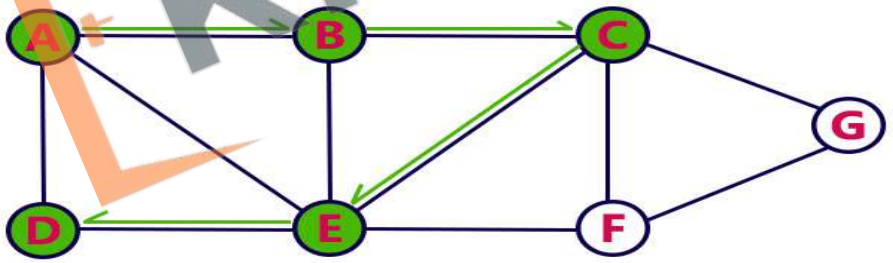
Step 4:

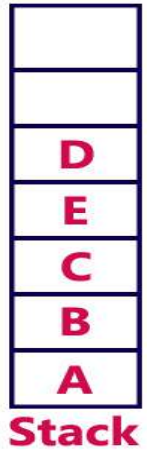
- Visit any adjacent vertext of **C** which is not visited (**E**).
- Push E on to the Stack



Step 5:

- Visit any adjacent vertext of **E** which is not visited (**D**).
- Push D on to the Stack



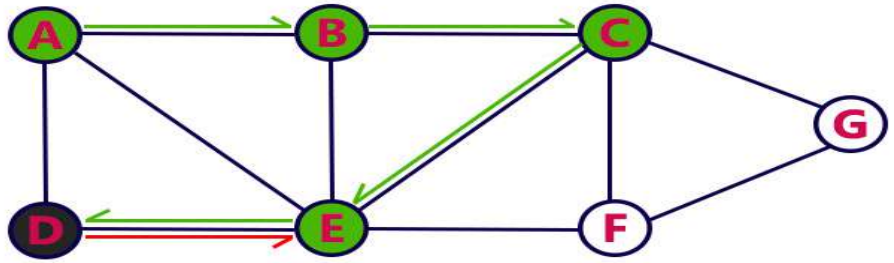


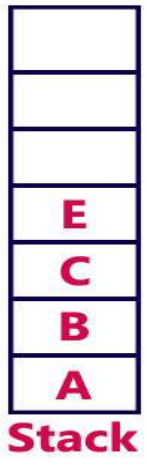
KNOWLEDGEGATE

/gate

Step 6:

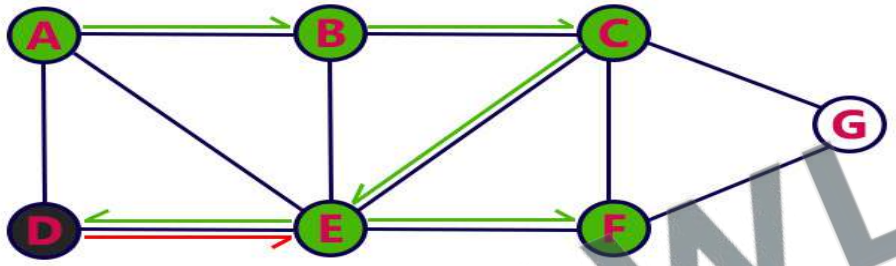
- There is no new vertiex to be visited from D. So use back track.
- Pop D from the Stack.





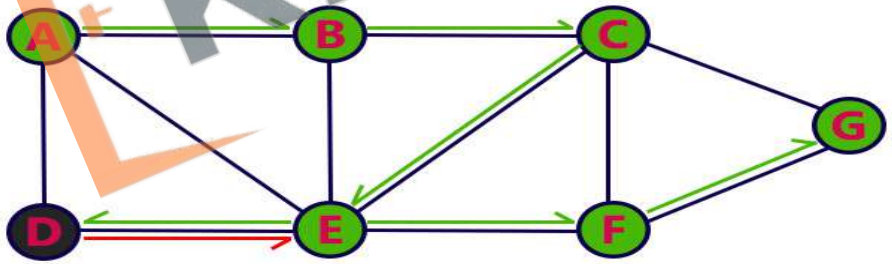
Step 7:

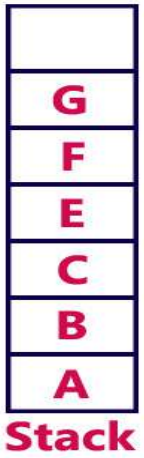
- Visit any adjacent vertex of **E** which is not visited (**F**).
- Push **F** on to the Stack.



Step 8:

- Visit any adjacent vertex of **F** which is not visited (**G**).
- Push **G** on to the Stack.

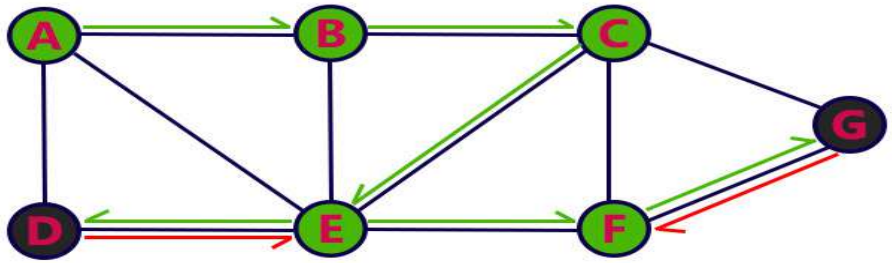


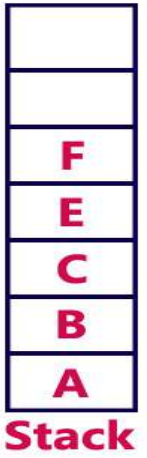


KNOWLEDGEGATE

Step 9:

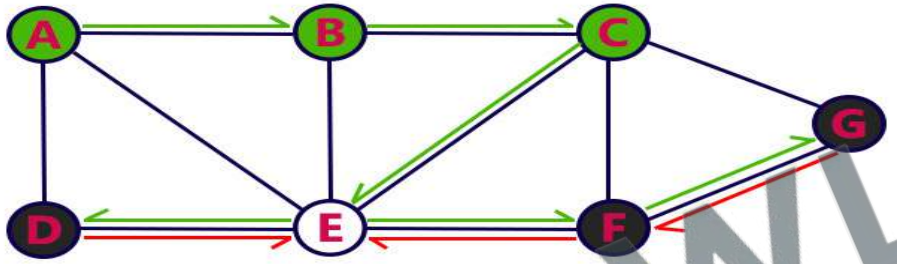
- There is no new vertiex to be visited from G. So use back track.
- Pop G from the Stack.





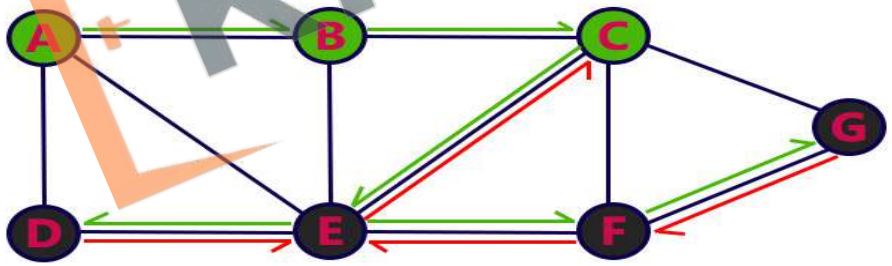
Step 10:

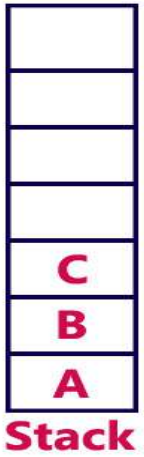
- There is no new vertiex to be visited from F. So use back track.
- Pop F from the Stack.



Step 11:

- There is no new vertiex to be visited from E. So use back track.
- Pop E from the Stack.

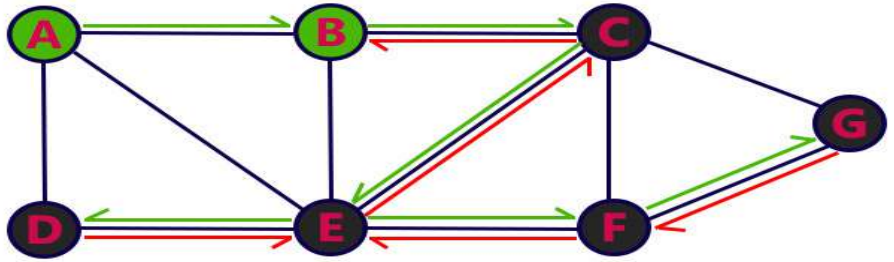




KNOWLEDGEGATE

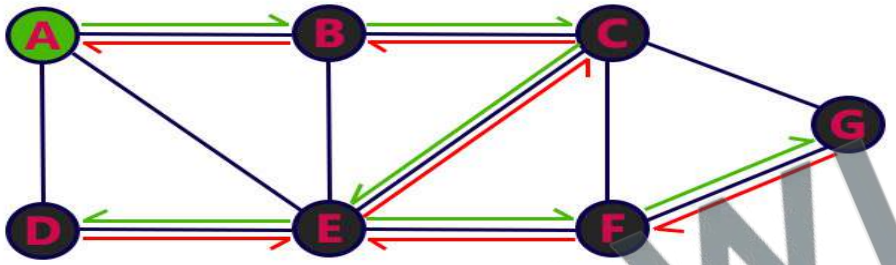
Step 12:

- There is no new vertiex to be visited from C. So use back track.
- Pop C from the Stack.



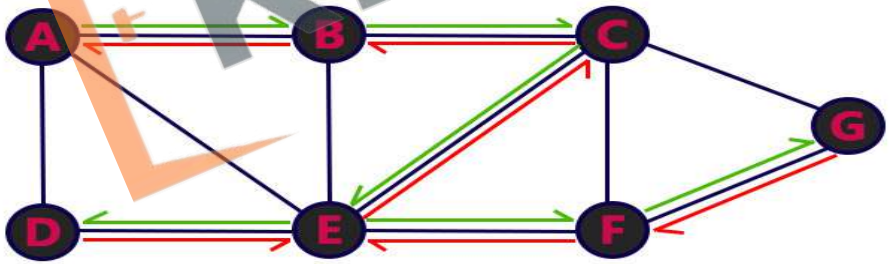
Step 13:

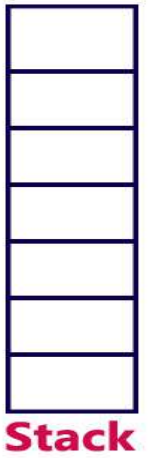
- There is no new vertiex to be visited from B. So use back track.
- Pop B from the Stack.



Step 14:

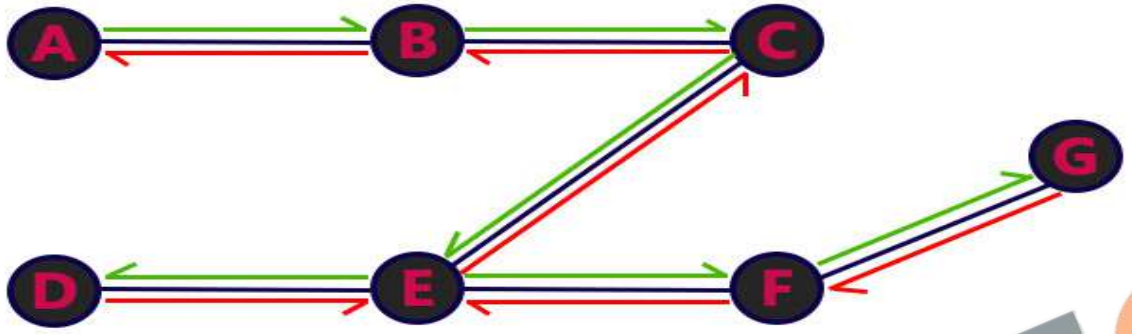
- There is no new vertiex to be visited from A. So use back track.
- Pop A from the Stack.





KNOWLEDGEGATE
/gate

- Stack became Empty. So stop DFS Treversal.
- Final result of DFS traversal is following spanning tree.



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Algorithm:

1. Start with the initial state as the root node.
2. Explore a node and recursively explore its unvisited neighboring nodes.
3. Backtrack if all neighboring nodes have been visited or there are no unvisited nodes left.
4. Repeat steps 2 and 3 until a goal state is found or all nodes have been visited.

KNOWLEDGEGATE

Advantages:

- **Memory Efficiency**: Depth-First Search stores fewer nodes in memory compared to breadth-first search, as it explores deeper paths first.

Disadvantages:

- **Completeness Not Guaranteed**: DFS may get trapped in infinite loops or infinite branches without reaching the goal state if there are cycles in the search space.
- Non-Optimal Solution: DFS does not guarantee finding the optimal solution, as it may find a solution at a greater depth before finding a shorter path.

KNOWLEDGEGATE

Completeness:

- DFS is not complete if the search space is infinite or contains cycles. If depth is finite than it is complete.

Optimality:

- DFS does not guarantee finding the optimal solution, as it may find a solution at a greater depth before finding a shorter path.

Time Complexity:

- The time complexity of DFS can vary depending on the search space structure. In the worst case, it can be $O(b^m)$, where b is the branching factor and m is the maximum depth of the search tree.

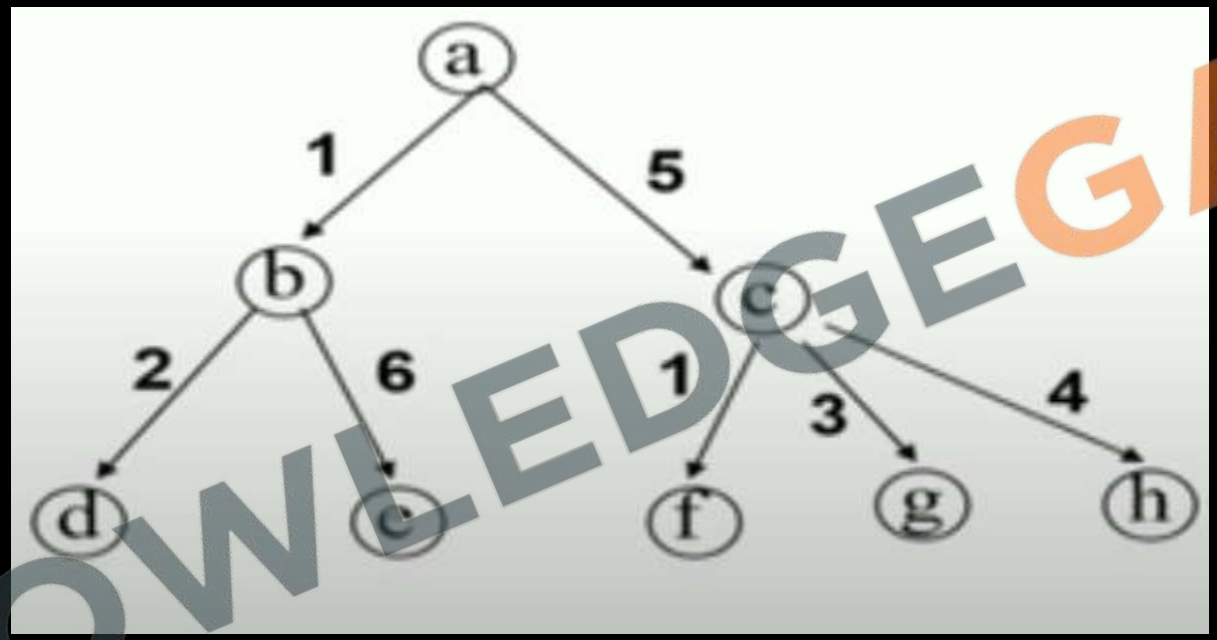
Space Complexity:

- The space complexity of DFS is O(bm), where b is the branching factor and m is the maximum depth of the search tree. It stores nodes along the current path in memory.

KNOWLEDGEGATE

Depth-limited search

- It is an extension of Breadth-First Search (BFS) that takes into account the cost of reaching each node to find the lowest-cost path to the goal. **Example:** Consider the graph below.



- **Based on the cost we will expand the graph in order: a→ b → d → c → f → e → g → h**

[KNOWLEDGEGATE]

Algorithm:

1. Start with the initial state as the root node.
2. Maintain a priority queue or a priority-based data structure to store nodes based on their path cost.
3. Enqueue the root node with a path cost of zero.
4. While the priority queue is not empty, do the following:
    1. Dequeue the node with the lowest path cost from the priority queue.
    2. If the dequeued node is the goal state, terminate the search and return the solution.
    3. Otherwise, expand the node and enqueue its unvisited neighboring nodes with their updated path costs.
5. Repeat steps 4 until the goal state is found or the priority queue is empty.

[KNOWLEDGEGATE]

Advantages:

- **Optimal Solution**: Uniform Cost Search guarantees finding the lowest-cost path to the goal.

Disadvantages:

- **Time and Space Complexity**: The time and space complexity can be high, especially in large search spaces or when there are many nodes with different path costs.

KNOWLEDGEGATE

Completeness:

- Uniform Cost Search is complete if the search space is finite, and all path costs are greater than some threshold.

Optimality:

- Uniform Cost Search is optimal as it guarantees finding the lowest-cost path to the goal.

Time Complexity:

- The time complexity of Uniform Cost Search depends on the number of nodes and the cost of the lowest-cost path to the goal. In the worst case, it can be exponential, i.e., $O(b^d)$, where b is the branching factor and d is the depth of the shallowest goal node.

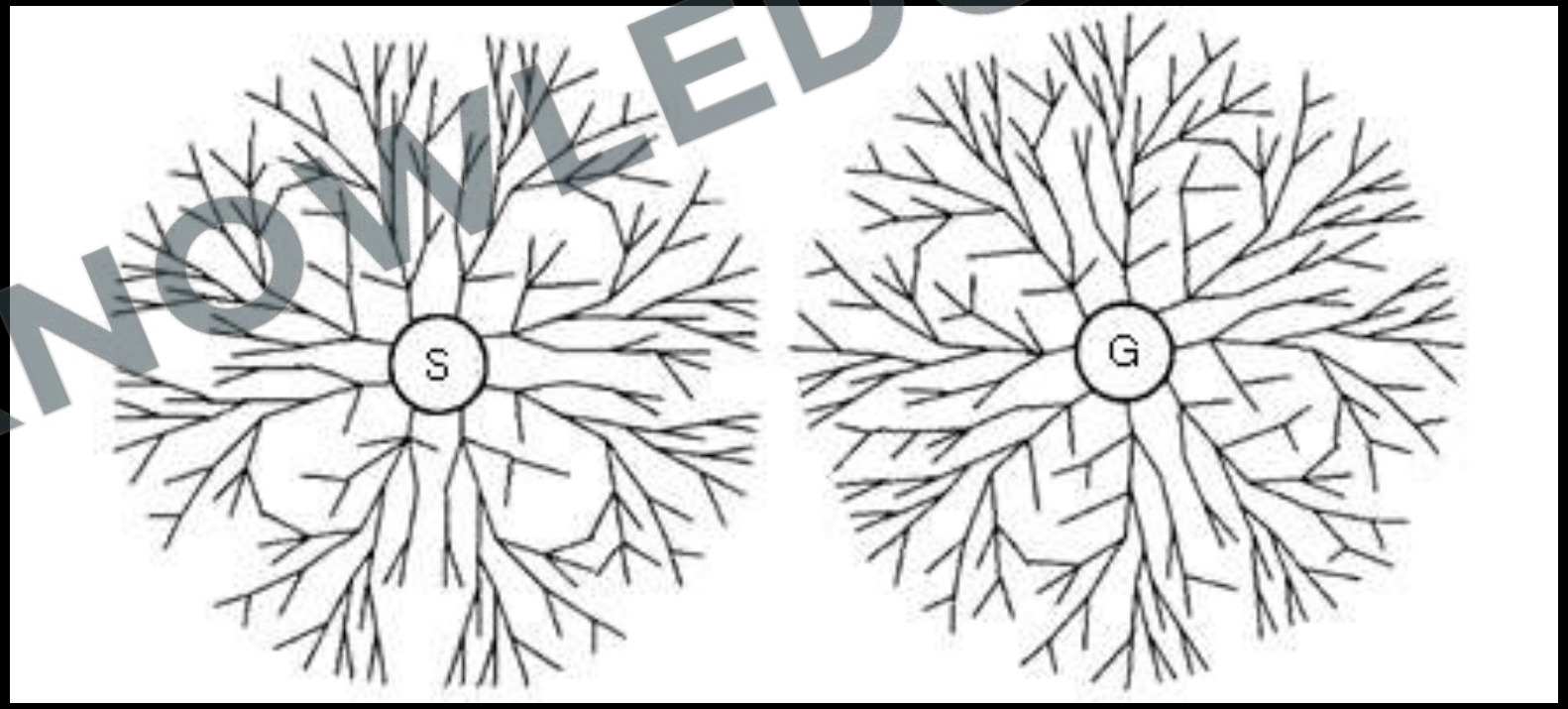
Space Complexity:

- The space complexity of Uniform Cost Search can also be exponential in the worst case, i.e., $O(b^d)$, as it may need to store all the nodes along the lowest-cost path in memory.

KNOWLEDGEGATE

Bidirectional Search

- Bidirectional Search is a search algorithm that simultaneously performs two separate searches, one forward from the initial state and one backward from the goal state. It aims to meet in the middle by searching for a common node reached from both directions.
- The motivation is that $b^{d/2} + b^{d/2}$ is much less than b^d. Bidirectional search is implemented by replacing the goal test with a check to see whether the frontiers of the two searches intersect; if they do, a solution has been found.



KNOWLEDGEGATE

Advantages:

- **Faster Exploration**: Bidirectional Search can be faster than traditional searches by exploring the search space simultaneously from both ends, potentially reducing the search effort.
- **Reduces Effective Branching Factor**: As the search progresses from both directions, the effective branching factor is reduced, leading to a more efficient search.

Disadvantages:

- **Increased Memory Requirement**: Bidirectional Search requires storing visited nodes from both directions, leading to increased memory consumption compared to unidirectional searches.
- **Additional Overhead**: The coordination and synchronization between the two searches introduce additional overhead in terms of implementation complexity.

KNOWLEDGE GATE

Completeness:

- Bidirectional Search is complete if both the forward and backward searches are complete in a finite search space.

Optimality:

- Bidirectional Search is optimal if both the forward and backward searches are optimal.

Time Complexity:

- The time complexity of Bidirectional Search depends on the branching factor, the depth of the shallowest goal state, and the meeting point of the two searches. In the best case, it can be $O(b^{d/2})$, where b is the branching factor and d is the depth of the goal state.

Space Complexity:

- The space complexity of Bidirectional Search depends on the memory required to store visited nodes from both directions. In the best case, it can be $O(b^{d/2})$, where b is the branching factor and d is the depth of the goal state.

KNOWLEDGEGATE

Comparison of Uniformed Searches

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[a]	Yes[a,b]	No	No	Yes[a]	Yes[a,d]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes[c]	Yes	No	No	Yes[c]	Yes[c,d]

Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: [a] complete if b is finite; [b] complete if step costs $\geq \epsilon$ for positive ϵ; [c] optimal if step costs are all identical; [d] if both directions use breadth-first search.

KNOWLEDGEGATE

Problems in Uninformed Search

1. **Blind Exploration**: Uninformed search strategies, such as Breadth-First Search or Depth-First Search, lack domain-specific knowledge or heuristics. They explore the search space without any information about the problem structure or the goal state

2. **Inefficient in Complex Spaces**: Uninformed search strategies can be inefficient in large or complex search spaces, as they may require extensive exploration of irrelevant or unpromising paths.

KNOWLEDGEGATE

INFORMED (HEURISTIC) SEARCH STRATEGIES

- Heuristic search strategies are like smart search methods that use special knowledge or hints to find solutions more efficiently. It's similar to when we use our intuition or common sense to solve a problem.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

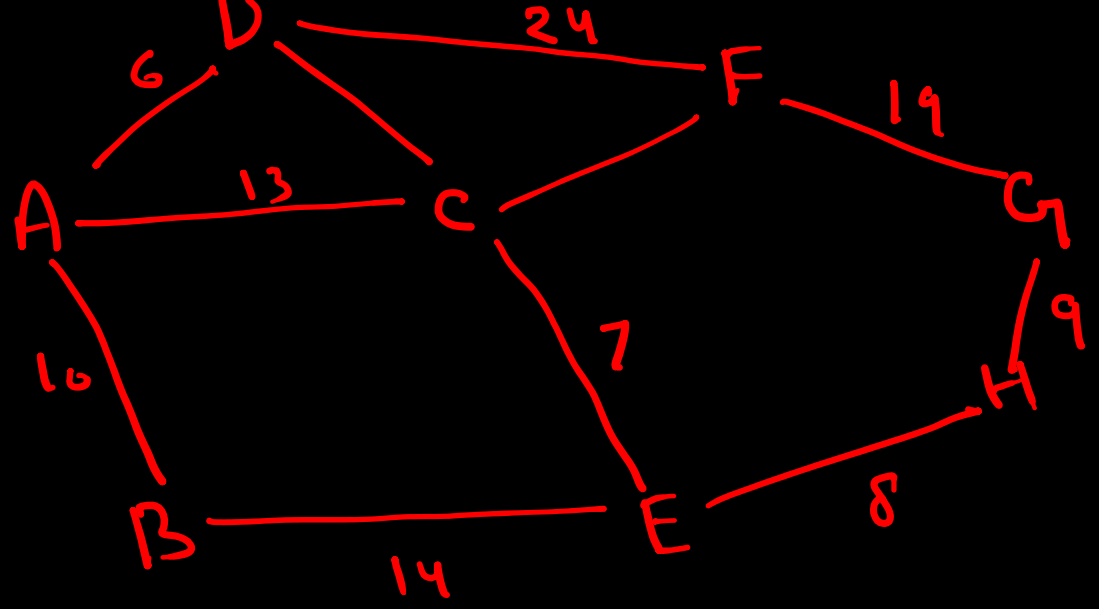
Best-first Search

- Best-first search is a search algorithm which explores a graph by expanding the most promising node chosen according to a specified rule. It's called "best-first" because it greedily chooses the node that seems most promising according to the heuristic.

- Best-first search takes the advantage of both BFS and DFS. The evaluation function is construed as a cost estimate, so the node with the *lowest* evaluation is expanded first.

- ***h(n) = estimated cost of the cheapest path from the state at node n to a goal state.***

KNOWLEDGEGATE



node	h(n)
A =	39
B =	31
C =	24
D =	34
E =	18
F =	16
H =	9
G =	0

KNOWLEDGEGATE

Algorithm:

1. Define a ‘priority queue’ of nodes, where each node has a priority equal to its heuristic value ‘h(n)’.
2. Start with the ‘root node’
3. Dequeue the node with the highest priority.
4. If the node contains the goal state, return it and exit. Else, enqueue all the node's successors.
5. If the queue is empty, return failure. Else, go to step 3.

KNOWLEDGEGATE

Advantages of Best-First Search

- It can find a solution without exploring much of the state space.
- It uses less memory than other informed search methods like A* as it does not store all the generated nodes.

Disadvantages of Best-First Search

- It is not complete. In some cases, it may get stuck in an infinite loop.
- It is not optimal. It does not guarantee the shortest possible path will be found.
- It heavily depends on the accuracy of the heuristic.

KNOWLEDGE GATE

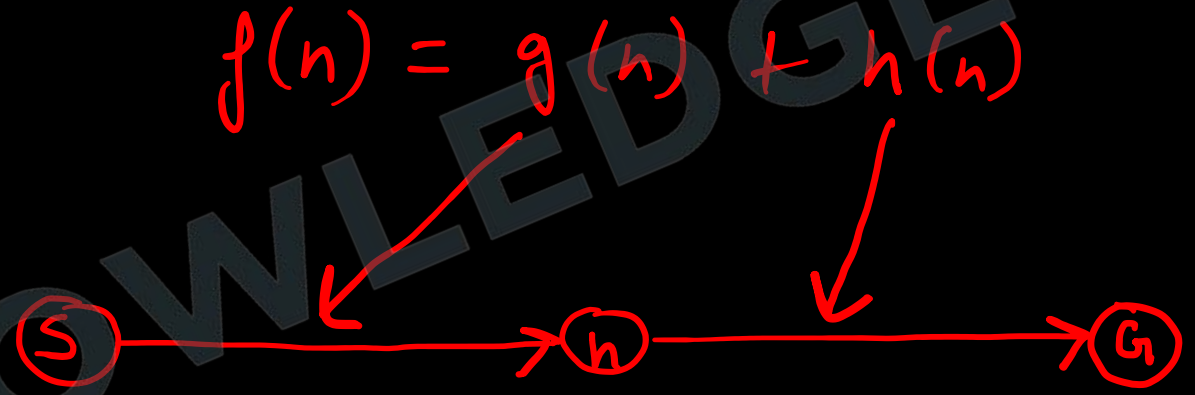
- **Time and Space Complexity**
  - The time and space complexity of Best-First Search are both $O(b^d)$, where b is the branching factor (number of successors per state), and d is the depth of the shallowest solution.

  - These complexities can be very high in problems with a large number of states and actions. In the worst case, the algorithm will need to visit all nodes, and since each node is stored, the space complexity is also proportional to the number of nodes.

KNOWLEDGEGATE

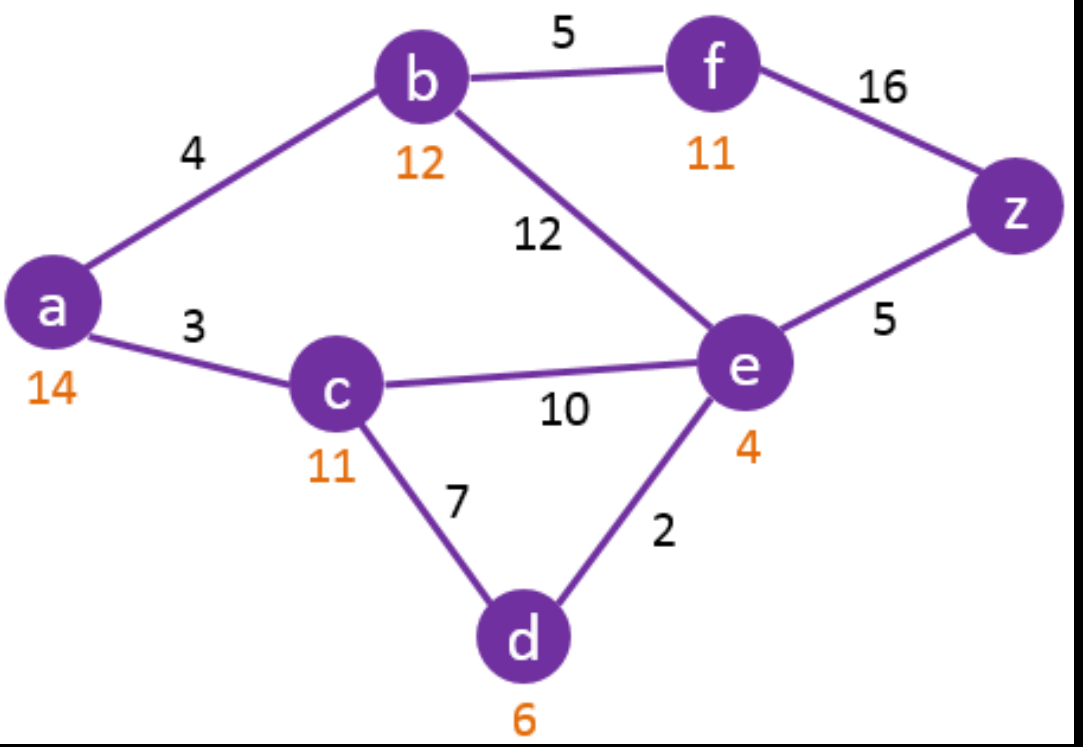
A* search: Minimizing the total estimated solution cost

- A* Search is an informed search algorithm that combines the advantages of both Uniform Cost Search and Greedy Best-First Search. It evaluates a node based on a combination of the cost of the path from the start node to that node and an estimated heuristic function that estimates the cost to reach the goal form the current node.



- A∗ search evaluates nodes by combining g(n), the cost to reach the node, and h(n), the estimated cost to get from the node to the goal: **f(n) = g(n) + h(n)**

KNOWLEDGEGATE



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Algorithm:

1. Start with the initial state as the root node.
2. Create an evaluation function that combines the cost of the path and a heuristic estimate.
3. Initialize an empty priority queue or priority-based data structure.
4. Enqueue the initial state into the priority queue based on the evaluation function.
5. While the priority queue is not empty, do the following:
    1. Dequeue the node with the highest priority from the priority queue.
    2. If the dequeued node is the goal state, terminate the search and return the solution.
    3. Otherwise, expand the node and enqueue its unvisited neighboring nodes into the priority queue based on the evaluation function.
6. Repeat steps 5 until a solution is found or the priority queue is empty.

[KNOWLEDGEGATE]

http://www.knowledgegate.in/gate

Completeness:

- A* Search is complete if the search space is finite and the heuristic function is admissible (never overestimates the actual cost).

Optimality:

- A* Search is optimal if the heuristic function is admissible and consistent (also known as monotonic).

Time Complexity:

- The time complexity of A* Search depends on the heuristic function, the branching factor, and the structure of the search space. In the worst case, it can be exponential.

Space Complexity:

- The space complexity of A* Search depends on the size of the priority queue and the number of nodes stored in memory. In the worst case, it can be exponential.

KNOWLEDGEGATE

Conditions for optimality: Admissibility and consistency

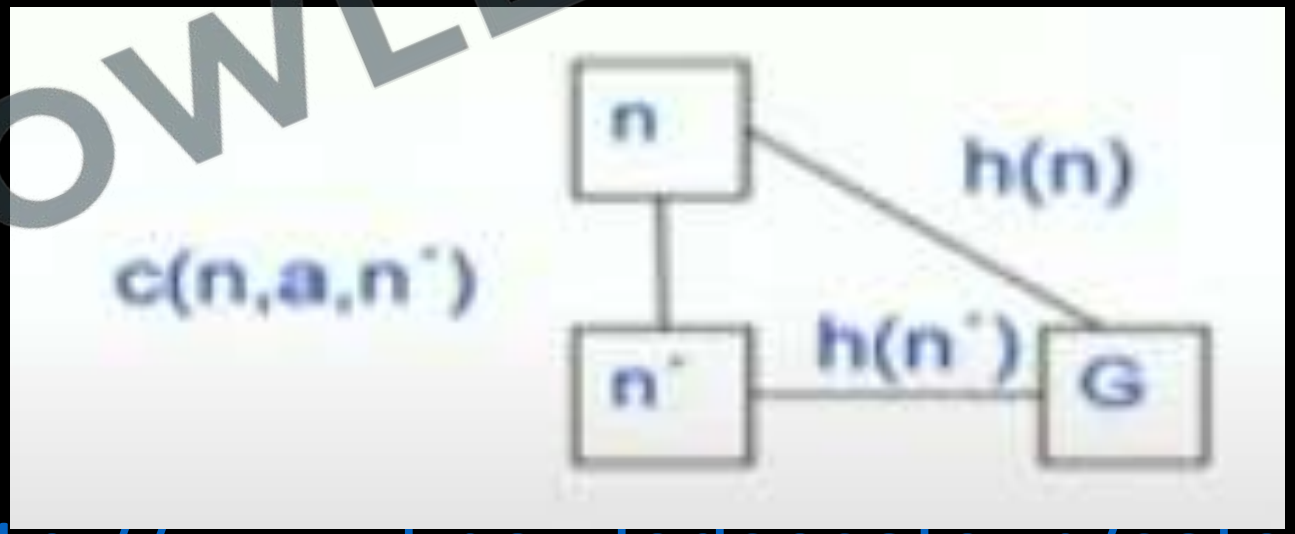
- The first condition we require for optimality is that h(n) be an **admissible heuristic.**
- An admissible heuristic is one that ***never overestimates*** the cost to reach the goal. Admissible heuristics are by nature **optimistic** because they think the cost of solving the problem is less than it actually is.
- For example, Straight-line distance is admissible because the shortest path between any two points is a straight line, so the straight line cannot be an overestimate.

KNOWLEDGEGATE

- A second, slightly stronger condition called **consistency** (or **monotonicity**) is required only for applications of A∗ to graph search.

- A heuristic h(n) is **consistent** if, for every node n and every successor n' of n generated by any action a, the estimated cost of reaching the goal from n is no greater than the step cost of getting to n' plus the estimated cost of reaching the goal from n':

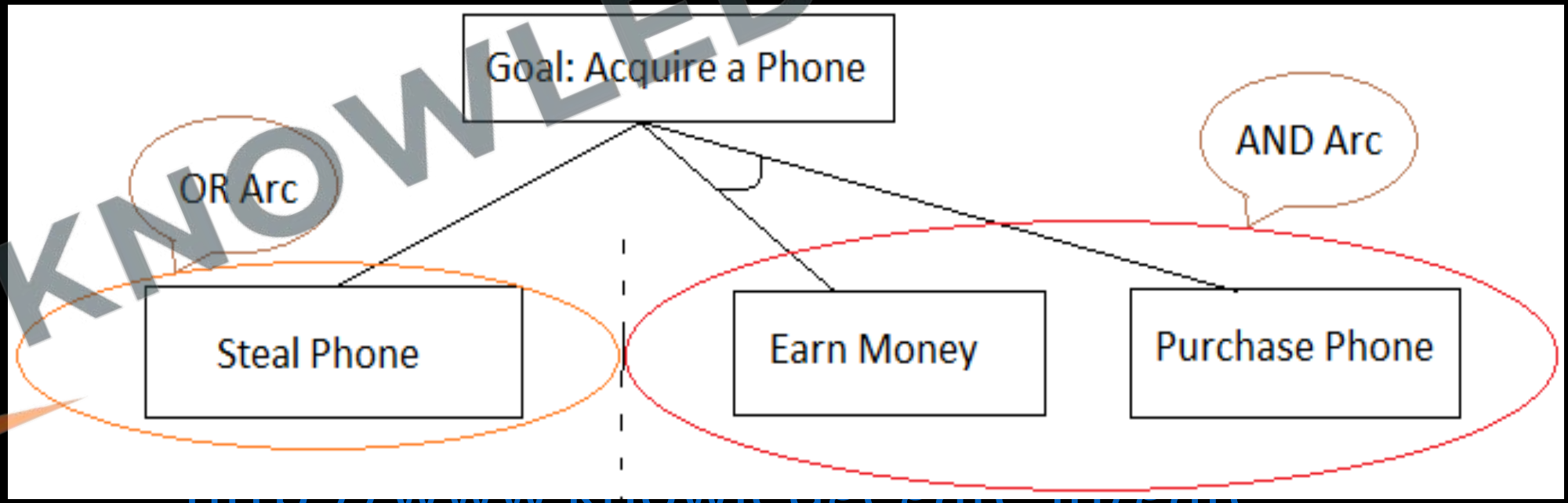
$$h(n) \leq c(n, a, n') + h(n')$$



KNOWLEDGEGATE

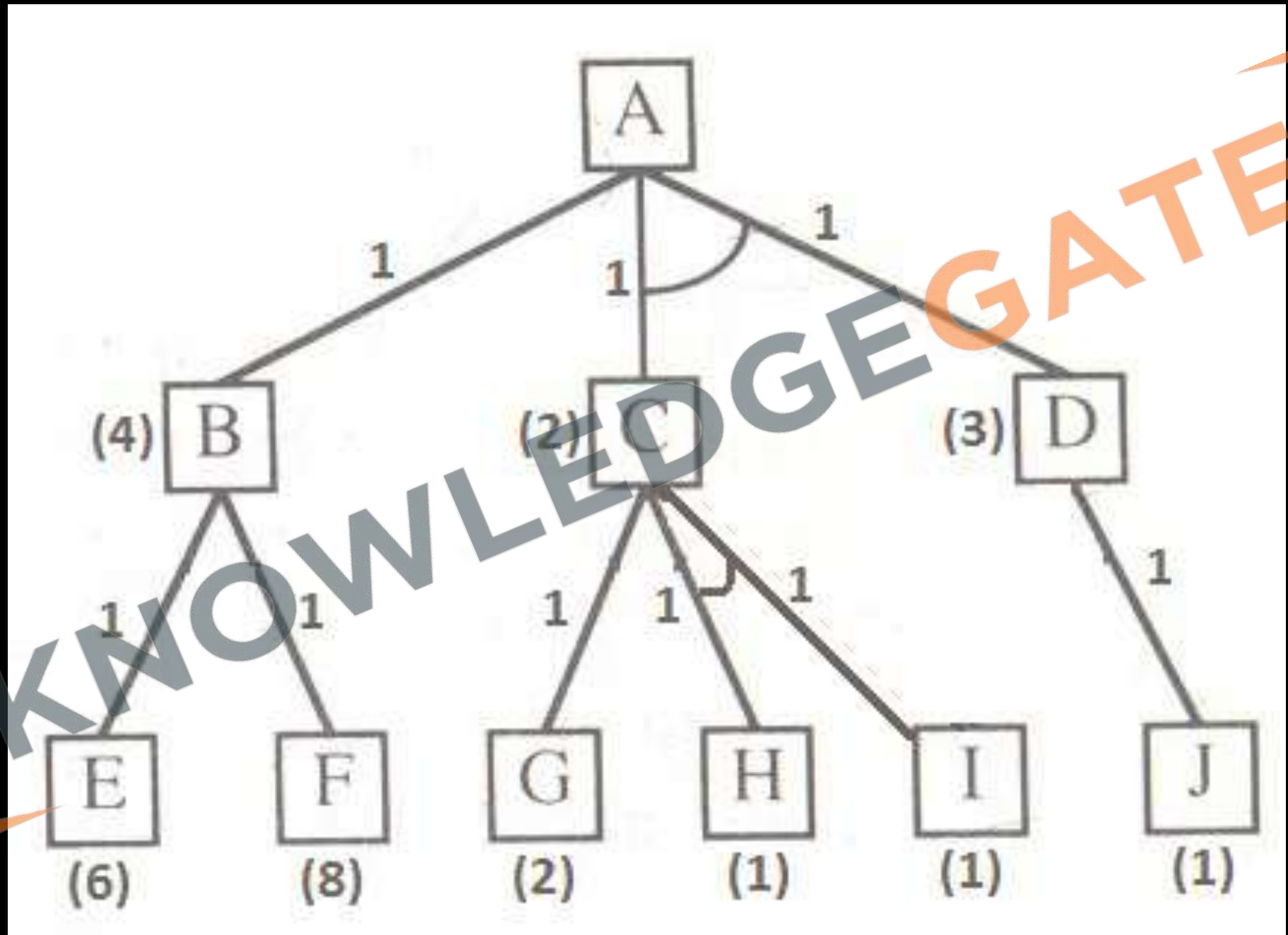
Problem with A* Algorithm

- So far we have considered search strategies for OR graphs through which we want to find a single path to a goal.
- The AND-OR GRAPH (or tree) is useful for representing the solution of problems that can solved by decomposing them into a set of smaller problems, all of which must then be solved. This decomposition, or reduction, generates arcs that we call AND arcs. One AND arc may point to any number of successor nodes, all of which must be solved in order for the arc to point to a solution.

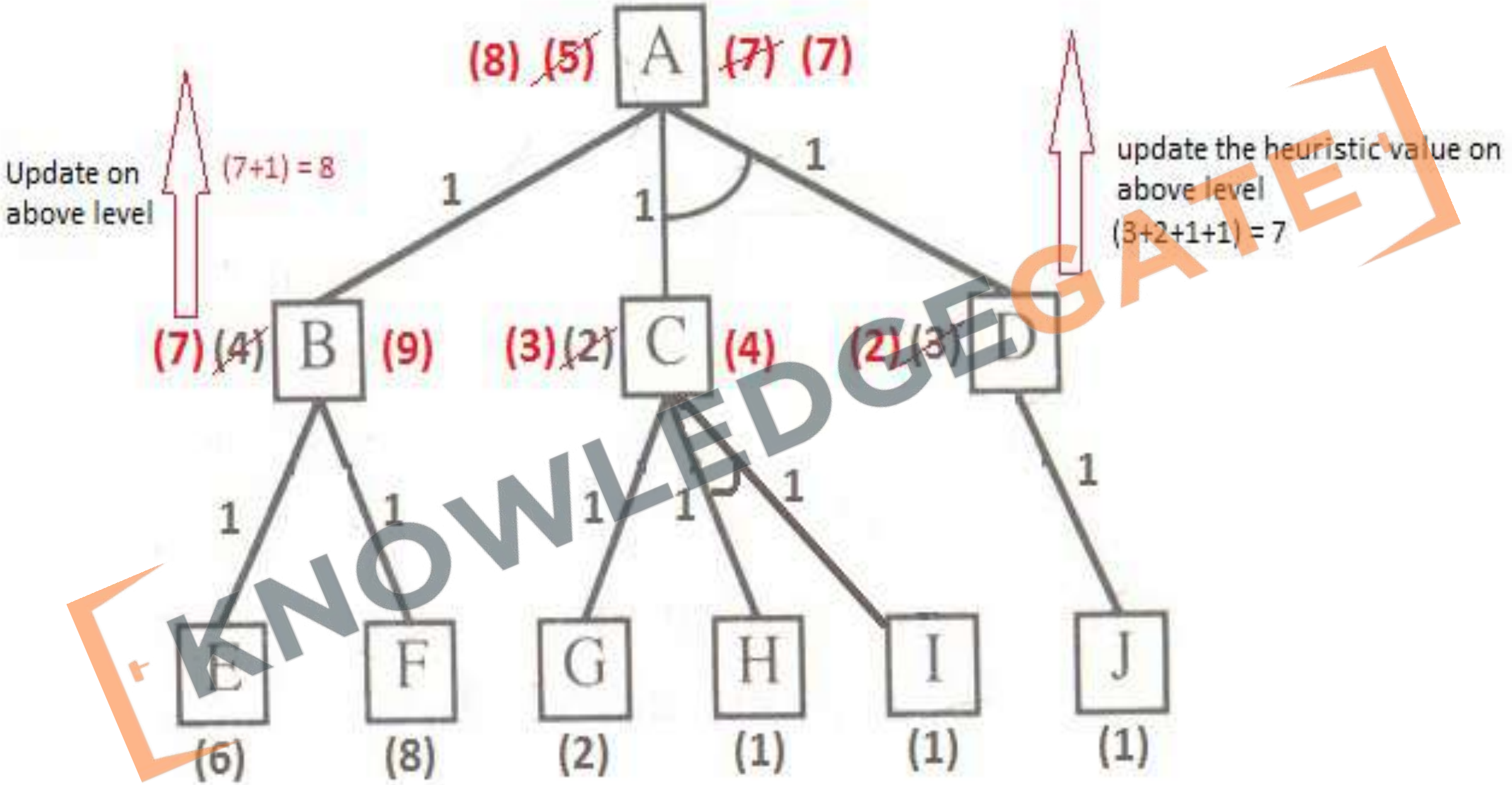


KNOWLEDGEGATE
http://www.knowledgegate.in/gate

AO Search Algorithm*



KNOWLEDGEGATE



KNOWLEDGEGATE

The pseudocode of AO* is as follows:

1. Initialize the graph with a single node (the start node).
2. While the solution graph contains non-terminal nodes (nodes that have successors not in the graph):
    1. Choose a non-terminal node for expansion based on a given strategy.
    2. Expand the node (add successors to the graph and update the costs of nodes).
3. The process continues until the start node is labeled as a terminal node.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Advantages of AO Algorithm*

- It can efficiently solve problems with multiple paths due to its use of heuristics.
- It is optimal when the heuristic function is admissible (never overestimates the true cost).

Disadvantages of AO Algorithm*

- It can consume a large amount of memory, similar to the A* algorithm.
- The performance of AO* is heavily dependent on the accuracy of the heuristic function. If the heuristic function is not well-chosen, AO* could perform poorly.

KNOWLEDGEGATE

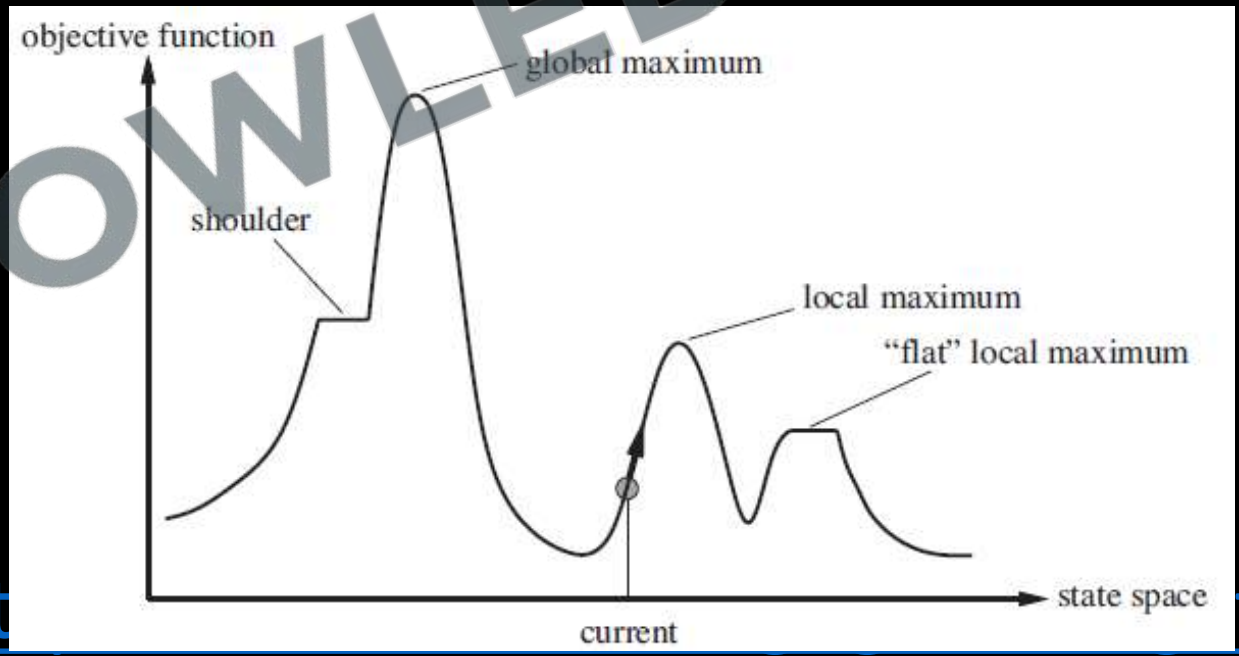
- **Completeness and Optimality**
  - AO* is complete, meaning it is guaranteed to find a solution if one exists. It is also optimal, meaning it will find the best solution, provided that the heuristic function is admissible (never overestimates the true cost) and consistent (satisfies the triangle inequality).

- **Time and Space Complexity**
  - The time and space complexity of the AO* algorithm is highly dependent on the problem at hand, particularly the size of the state space, the number of goals, and the quality of the heuristic function. In the worst case, the algorithm will need to visit all nodes, and since each node is stored, the space complexity is also proportional to the number of nodes.

KNOWLEDGEGATE

Hill Climbing Algorithm

- It is an iterative algorithm that starts with an arbitrary solution to a problem, then attempts to find a better solution by making a change, If the change produces a better solution, another incremental change is made to the new solution, and so on until no further improvements can be found.
- Hill climbing is sometimes called **greedy local search with no backtracking** because it grabs a good neighbor state without thinking ahead about where to go next.



KNOWLEDGE GATE

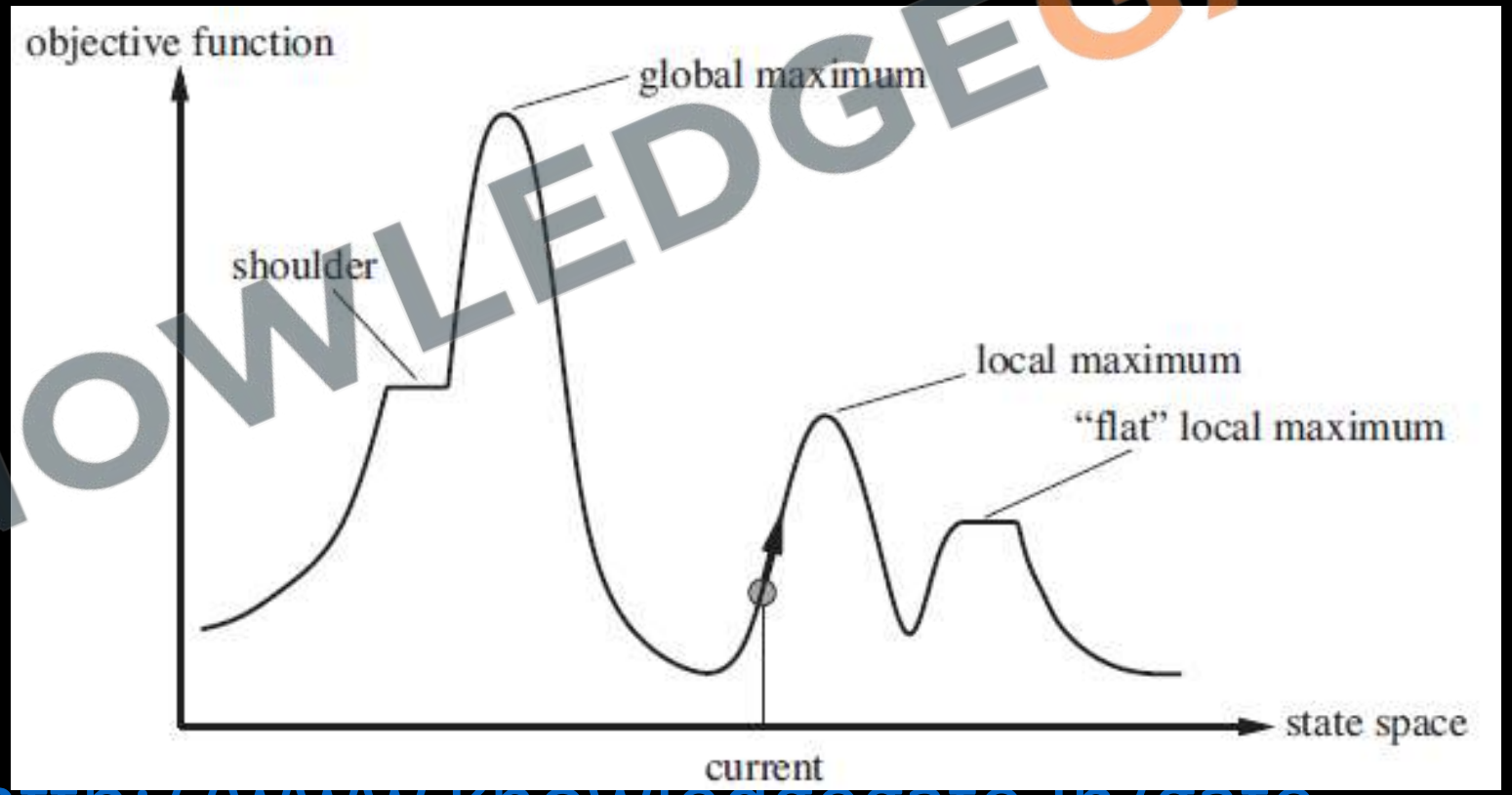
Algorithm for Simple Hill climbing

- **Start**: Begin at a random point on the problem landscape.
- **Evaluate**: Assess the current position.
- **Look Around**: Check neighboring points and evaluate them.
- **Can Improve**: Determine if any neighbors are better (higher value) than the current position.
- **Move**: If a better neighbor exists, move to that point.
- **Repeat**: Continue the process from the new point.
- **No Improvement**: If no better neighbors are found, you've hit a peak.
- **End**: The process stops when no further improvement can be made.

KNOWLEDGE GATE

Problems with Hill Climbing

- **Local maxima**: a local maximum is a peak that is higher than each of its neighboring states but lower than the global maximum. Hill-climbing algorithms that reach the vicinity of a local maximum will be drawn upward toward the peak but will then be stuck with nowhere else to go.



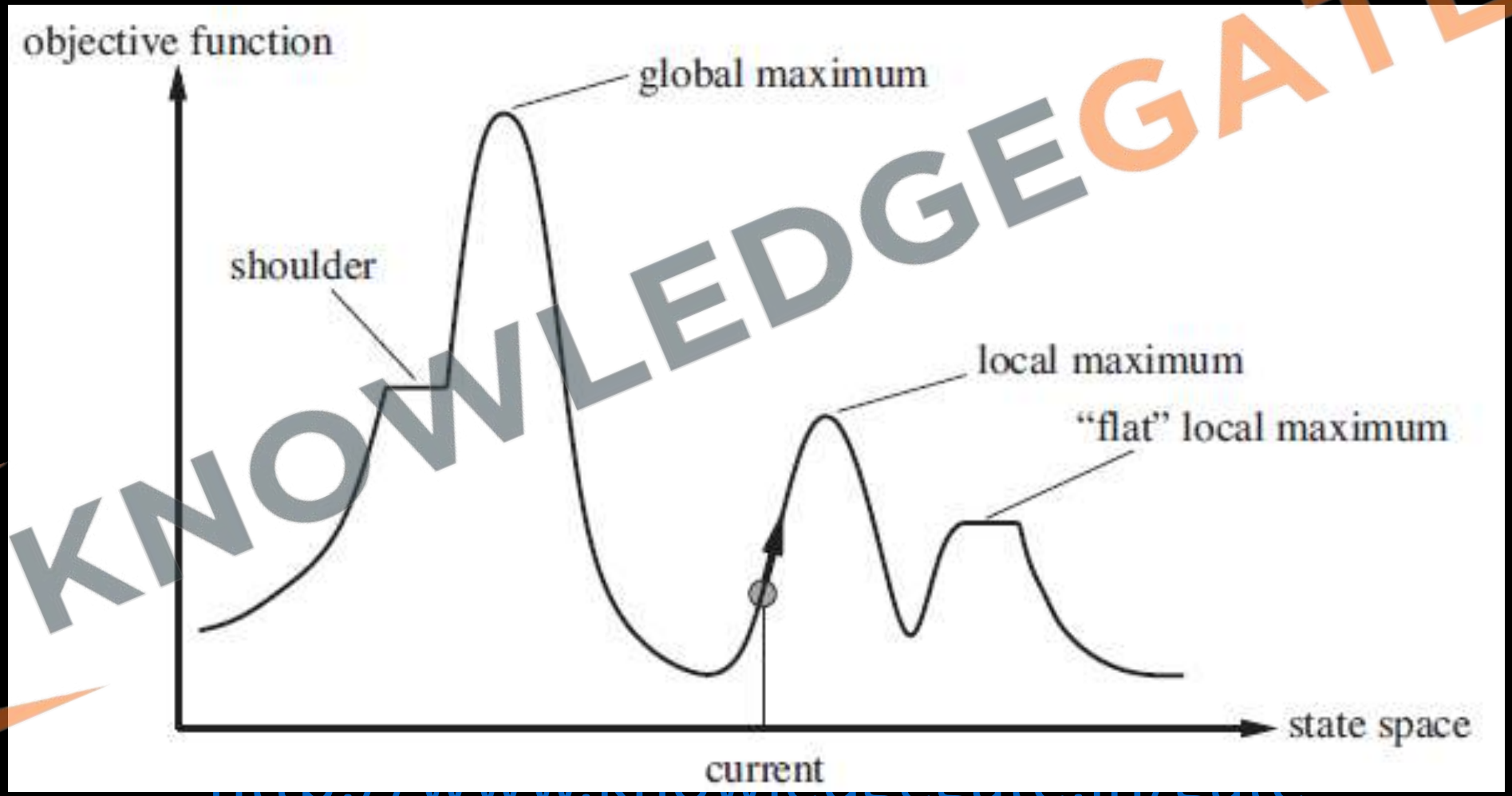
KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Ridges:** Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Plateau:** A plateau is a flat area of the state-space landscape. It can be a flat local maximum, from which no uphill exit exists, or a shoulder, from which progress is possible. A hill-climbing search might get lost on the plateau. In each case, the algorithm reaches a point at which no progress is being made.



KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Advantages of Hill Climbing:

- It's simple to understand and easy to implement.
- It requires less computational power compared to other search algorithms.
- If the heuristic is well chosen, it can find a solution in a reasonable time.

Disadvantages of Hill Climbing:

- It's not guaranteed to find the optimal solution.
- It's highly sensitive to the initial state and can get stuck in local optima.
- It does not maintain a search history, which can cause the algorithm to cycle or loop.
- It can't deal effectively with flat regions of the search space (plateau) or regions that form a ridge.

KNOWLEDGEGATE

Random-restart hill climbing

- To avoid local optima, you can perform multiple runs of the algorithm from different random initial states. This is called random-restart hill climbing and increases the chance of finding a global optimum.
- **Tabu Search:** To prevent the algorithm from getting stuck in a loop or revisiting states, a list of previously visited states can be maintained, which are then avoided in future steps.
- **Local Beam Search:** To tackle plateau and local optima, this variation of hill climbing keeps track of k states rather than just one. It begins with k randomly generated states. At each step, all the successors of all k states are generated, and if any one is a goal, it halts. Else, it selects the best k successors from the complete list and repeats.

[KNOWLEDGEGATE]

Means-Ends Analysis

- **What it is**: A strategic approach combining forward and backward reasoning to tackle complex and large problems in AI.
- **Why use it**: It helps in focusing the problem-solving effort on significant discrepancies between the current state and the goal, reducing unnecessary exploration.

KNOWLEDGEGATE

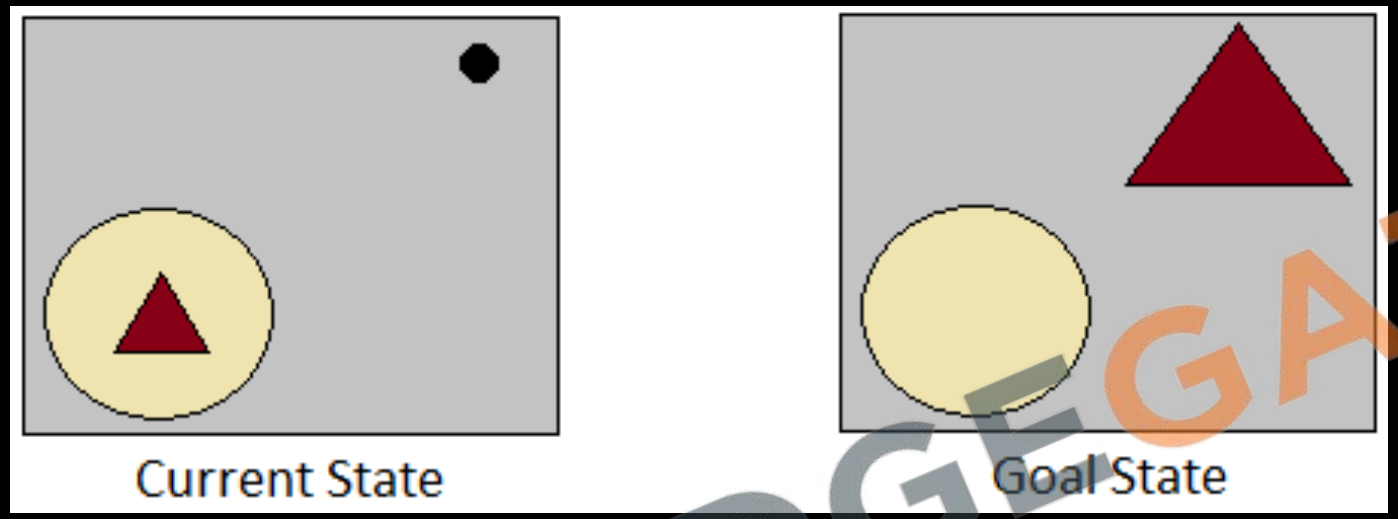
http://www.knowledgegate.in/gate

Process:

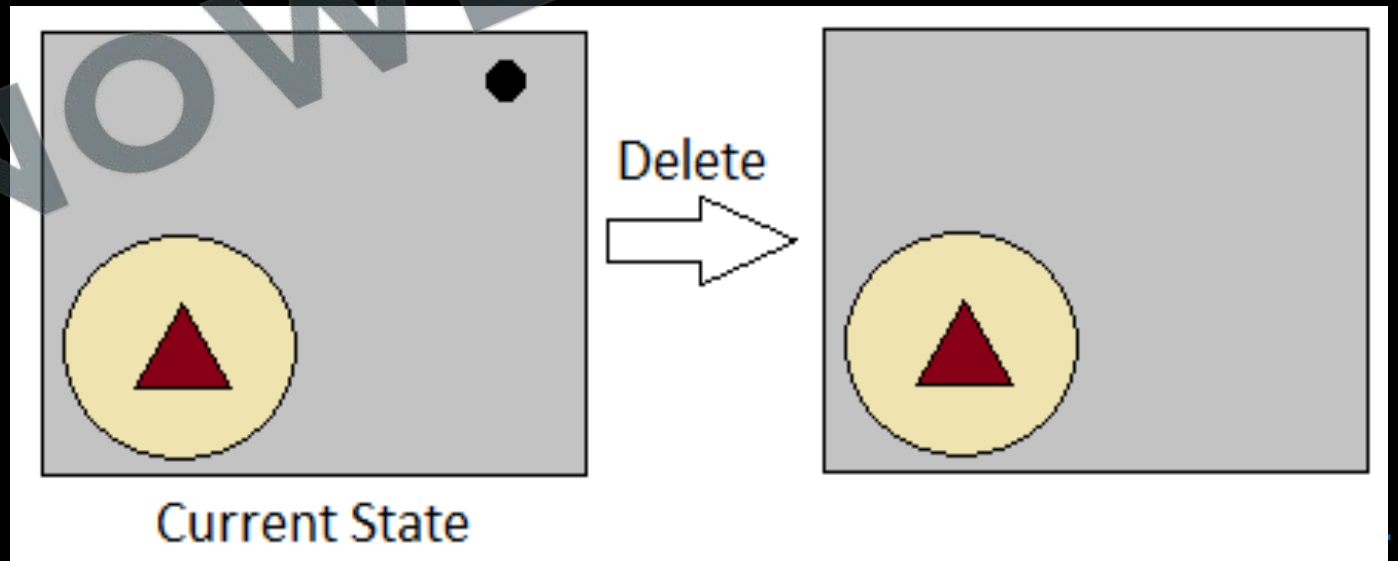
- **Break down the problem**: Split a large problem into major components and address these first.
- **Address sub-problems**: Once the major components are aligned towards the goal, solve any smaller issues that emerge.
- **Integrate strategies**: Use a blend of forward search (from the starting point) and backward search (from the goal) to guide the problem-solving process effectively.

KNOWLEDGEGATE

- We first evaluate the difference between the current and the goal state. For each difference, we will generate a new state and will apply the operators.

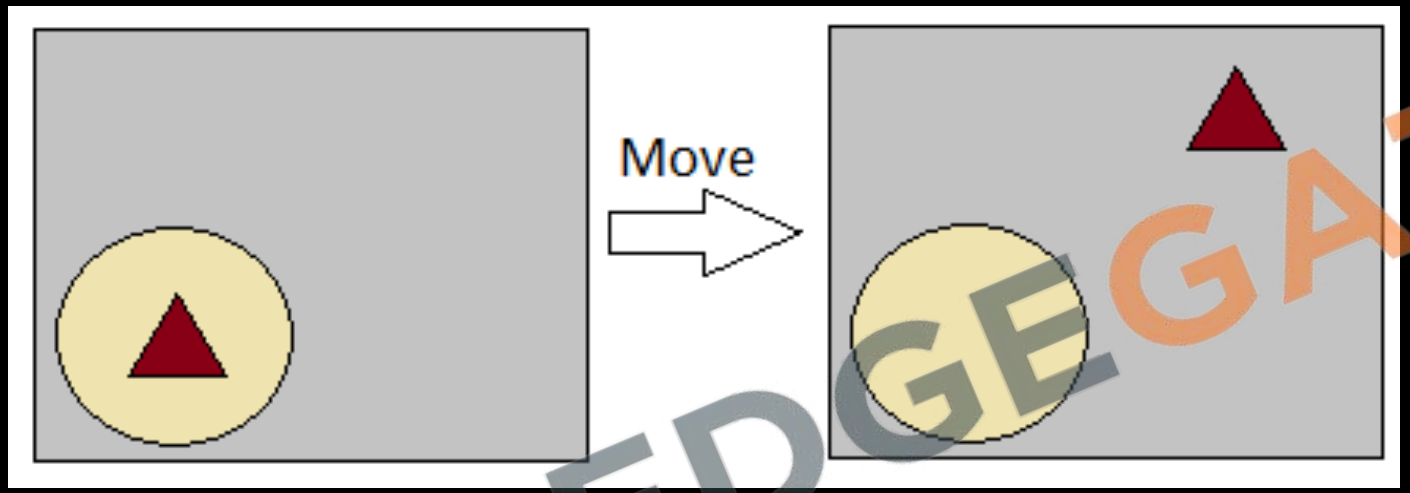


- **<u>Applying Delete operator</u>:** The first difference that we find is that in goal state there is no dot symbol which is present in the initial state, so, first we will apply the **Delete operator** to remove this dot.



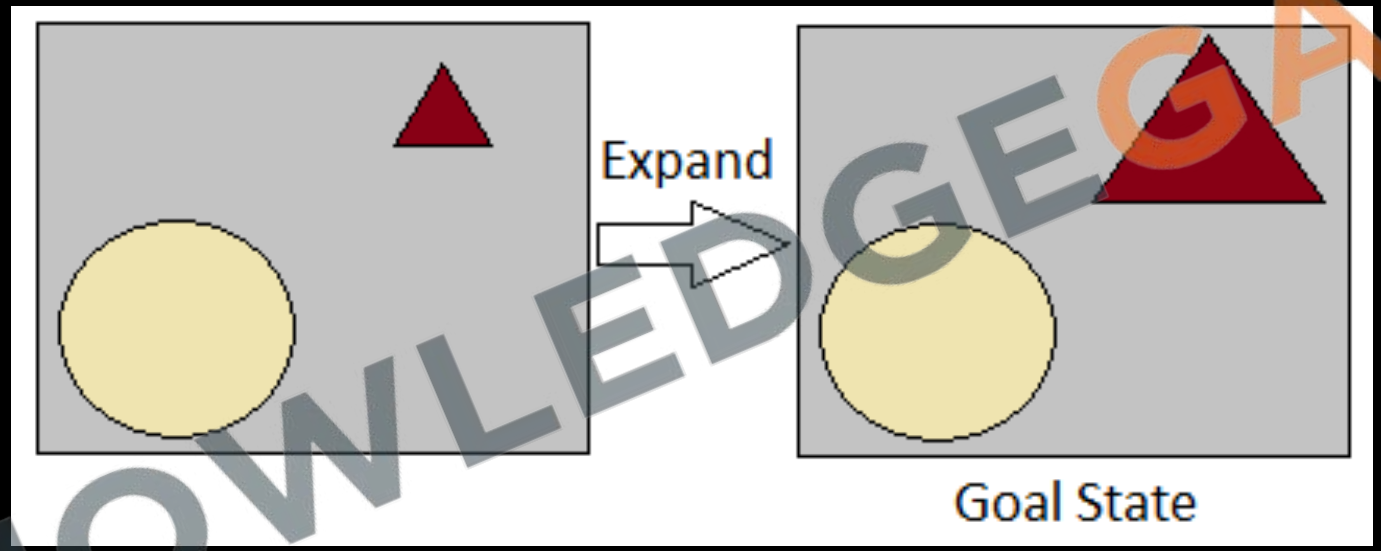
KNOWLEDGE GATE

- **Applying Move Operator:** After applying the Delete operator, the new state occurs which we will again compare with goal state. After comparing these states, there is another difference that is the triangle is outside the circle, so, we will apply the Move Operator.



KNOWLEDGEGATE

- **Applying Expand Operator:** Now a new state is generated in the third step, and we will compare this state with the goal state. After comparing the states there is still one difference which is the size of the triangle, so, we will apply Expand operator, and finally, it will generate the goal state.



KNOWLEDGEGATE

Operator Subgoaling:

- **What it is**: A process within Means-Ends Analysis (MEA) for handling situations where a direct action (operator) cannot be applied to the current state.
- **Purpose**: To establish the necessary conditions (preconditions) for applying an operator by setting up sub-goals that modify the current state.

KNOWLEDGE GATE

- **Means-Ends Analysis Steps**:
  - **Identify Differences**: Note the discrepancies between the current and goal states.
  - **Set Sub-goals**: Create intermediate goals aimed at reducing identified differences.
  - **Find Operators**: Look for actions that can achieve these sub-goals.
  - **Apply Operators**: Execute the most promising actions. If constraints prevent direct action, set new sub-goals to remove these constraints.
  - **Iterate**: Repeat these steps until the goal is achieved or no further progress can be made.

KNOWLEDGE GATE

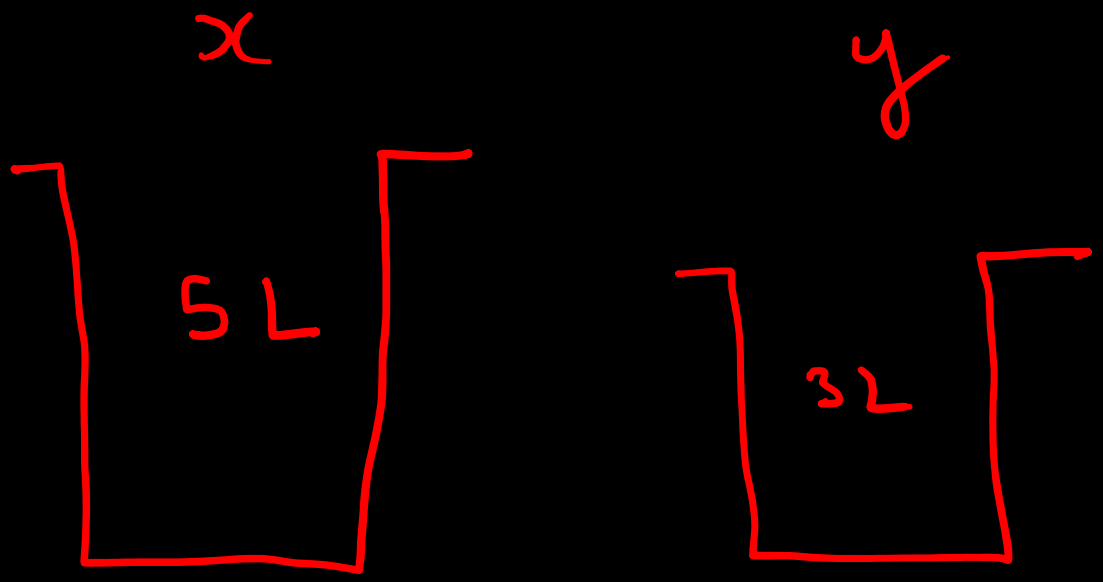
http://www.knowledgegate.in/gate

Water Jug Problem

- The Water Jug Problem is a classic example often used in computer science and artificial intelligence to demonstrate problem-solving methods such as Means-Ends Analysis. Here's a brief overview:

- **Problem Statement**: You have two jugs with different capacities (for example, one jug holds 3 liters and the other holds 5 liters) and a water source. The goal is to measure out a specific amount of water (say 4 liters), you can fill, empty, or transfer water between the jugs and no measurements marking on either jug.

KNOWLEDGEGATE



→ Jug x can contain 5L

→ Jug y can contain 3L

Goal is to have 4L in Jug x

5 3

(x, y)

<0,0>

<5,0>

<2,3>

<2,0>

<0,2>

<5,2>

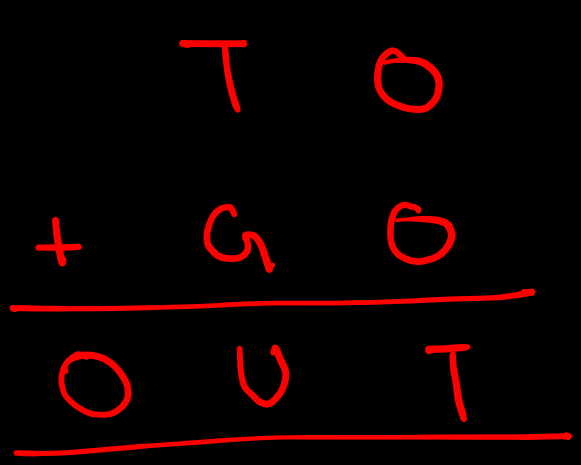
<4,3>

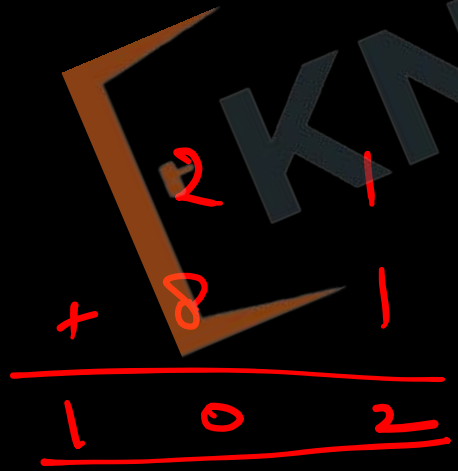
KNOWLEDGEGATE

Constraints Satisfaction Problem

- CSPs are a key topic in computer science, especially in artificial intelligence (AI).
- **Definition**: A CSP consists of a set of variables, a domain for each variable, and a set of constraints that specify allowable combinations of values.
- **Variables**: Elements that have to be assigned values from their domains. For example, in a scheduling problem, variables could be different time slots.
- **Domains**: The range of possible values that each variable can take. For example, in a coloring problem, the domain might be a set of colors.
- **Constraints**: Rules that define which combinations of values are valid or invalid. They restrict the ways in which variables can be assigned values.
- **Solutions**: Assignments of values to all variables that do not violate any constraints. A problem can have zero, one, or multiple solutions.
- **Solving Methods**: There are various strategies for solving CSPs, including backtracking, forward checking, and constraint propagation.
- **Applications**: CSPs are widely used in many fields such as scheduling, planning, resource allocation, timetabling, and in games and puzzles like Sudoku.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate





KNOWLEDGEGATE

http://www.knowledgegate.in/gate

C_4 C_3 C_2 C_1

```
  S E N D
+ M O R E
---------
M O N E Y
```

$M = 1$

$S + M \geq 10$

$\therefore S = 9$

$E + O = N$ if $C_2 = 0$ $[E = N]$

$\therefore C_2 = 1$

$O = 0$

let $E = 5$

then $N = 6$

$D + E = Y$

$D + 5 = Y$

if $C_1 = 1$

then $D \geq 5$ ×

6 ×

7 ✓

8 ×

9 ×

```
  9 5 6 7
+ 1 0 8 5
---------
1 0 6 5 2
```

if $C_1 = 1$

$R = 8$

$D = 7$, $Y = 2$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

```
  EAT
 THAT
------
 APPLE
```

```
  SOME
+ TIME
------
 SPENT
```

```
  BASE
+ BALL
------
 GAMES
```

```
  819
 9219
------
10038
```

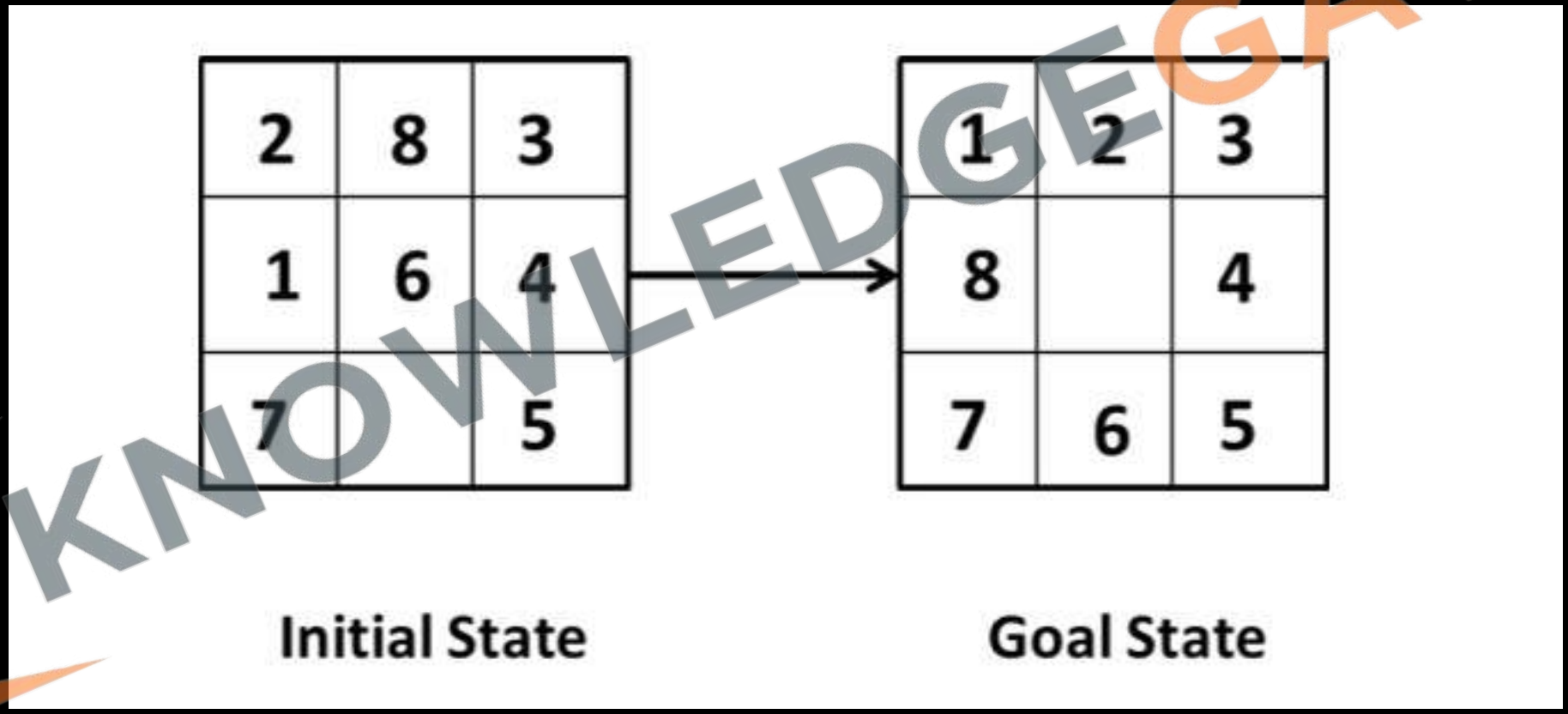
```
  1934
+ 8534
------
 10468
```

```
  7483
+ 7455
------
 14938
```

KNOWLEDGEGATE

The 8-puzzle Problem

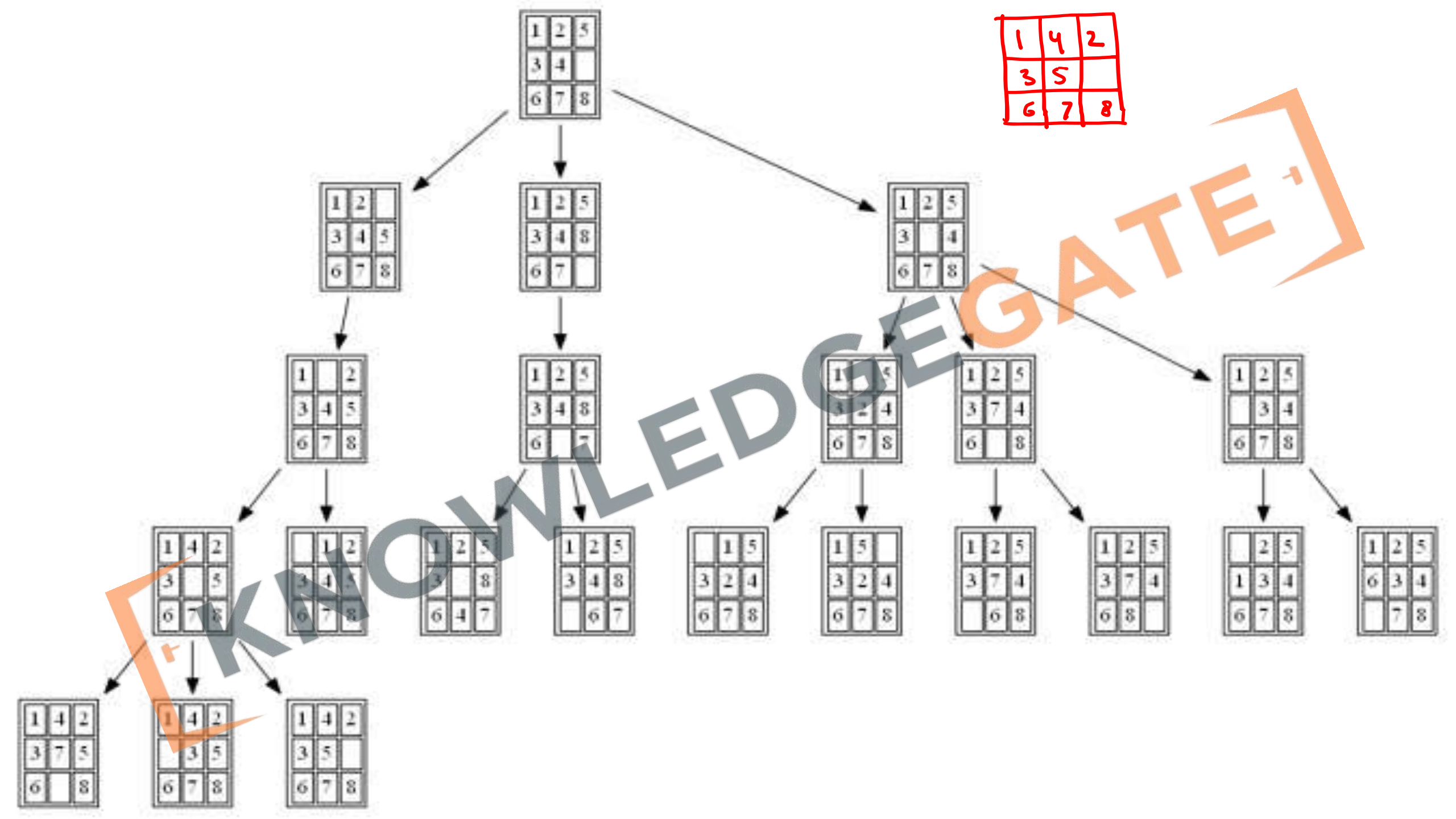
- The 8-puzzle problem is a classic search problem in the field of Artificial Intelligence (AI) and computer science. It involves a 3x3 grid with 8 numbered tiles and one empty space. The goal is to move the tiles around until they are in a specific goal state.

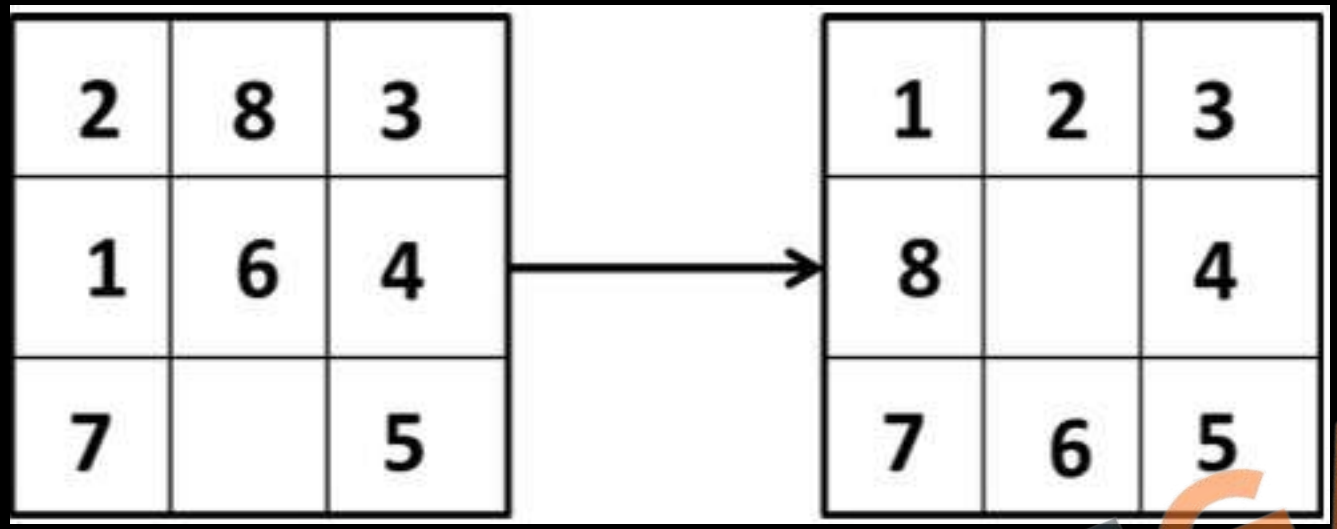


KNOWLEDGEGATE

- **Grid Structure**: The puzzle consists of a 3x3 grid, making 9 spaces. There are 8 numbered tiles and one empty space (often represented by 0 or a blank).
- **Initial State**: The puzzle starts in a random configuration with the tiles not in order.
- **Goal State**: The objective is to arrange the tiles in numerical order.
- **Moves**: You can move a tile into the empty space if it is adjacent to it (up, down, left, or right). Diagonal moves are not allowed.
- **Difficulty Levels**: Not all initial states are solvable.
- **Search Algorithms**: To find a solution, various search algorithms can be used, such as Breadth-First Search (BFS), Depth-First Search (DFS), A* Search, and others. These algorithms explore possible moves from the initial state to reach the goal state.
- **Heuristics**: For more efficient search, heuristics such as the Manhattan distance or the number of misplaced tiles can be used, especially in algorithms like A* to estimate the cost to reach the goal from a given state.
- **Applications**: Solving the 8-puzzle problem helps in understanding important concepts in AI and algorithms, such as state space, search strategies, and the use of heuristics.

KNOWLEDGEGATE


1 4 2
3 5
6 7 8
KNOWLEDGEGATE



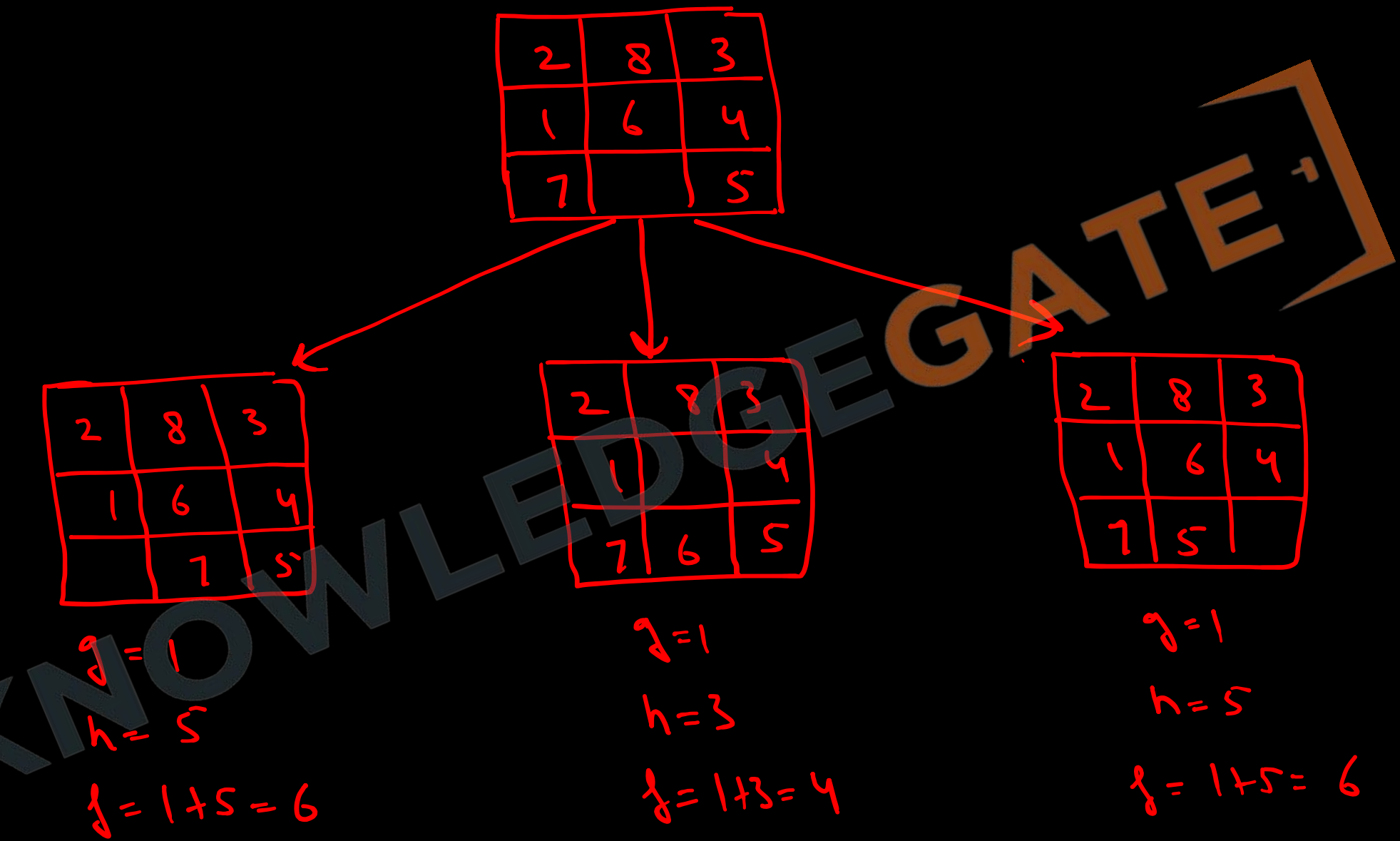
$g = 0$, $h = 4$

KNOWLEDGEGATE

- Find the most cost-effective path to reach the final state from initial state using A* Algorithm.
- $f(n) = g(n) + h(n)$
- Consider $g(n)$ = Depth of node and $h(n)$ = Number of misplaced tiles.

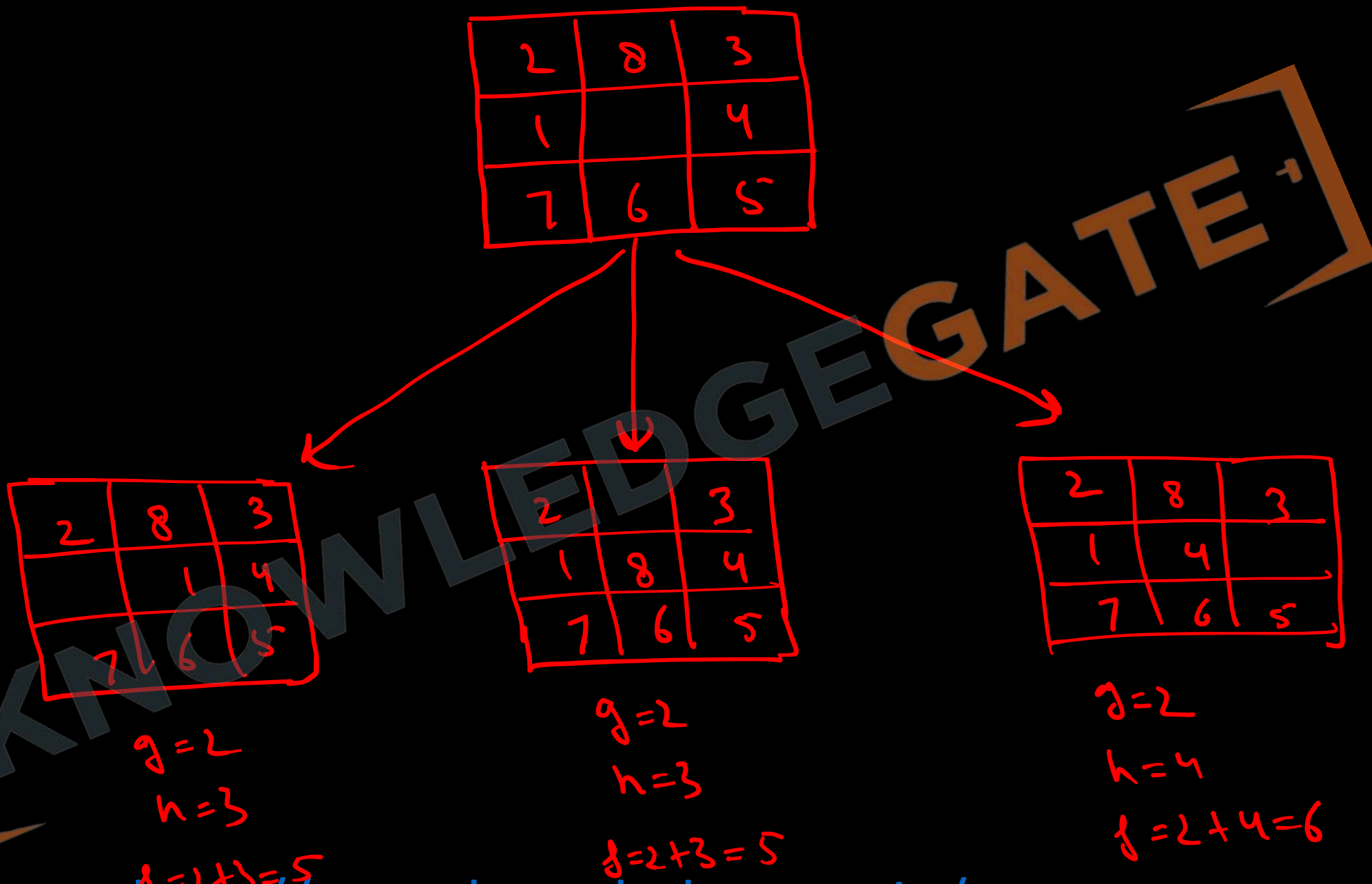
http://www.knowledgegate.in/gate

1	2	3
8		4
7	6	5

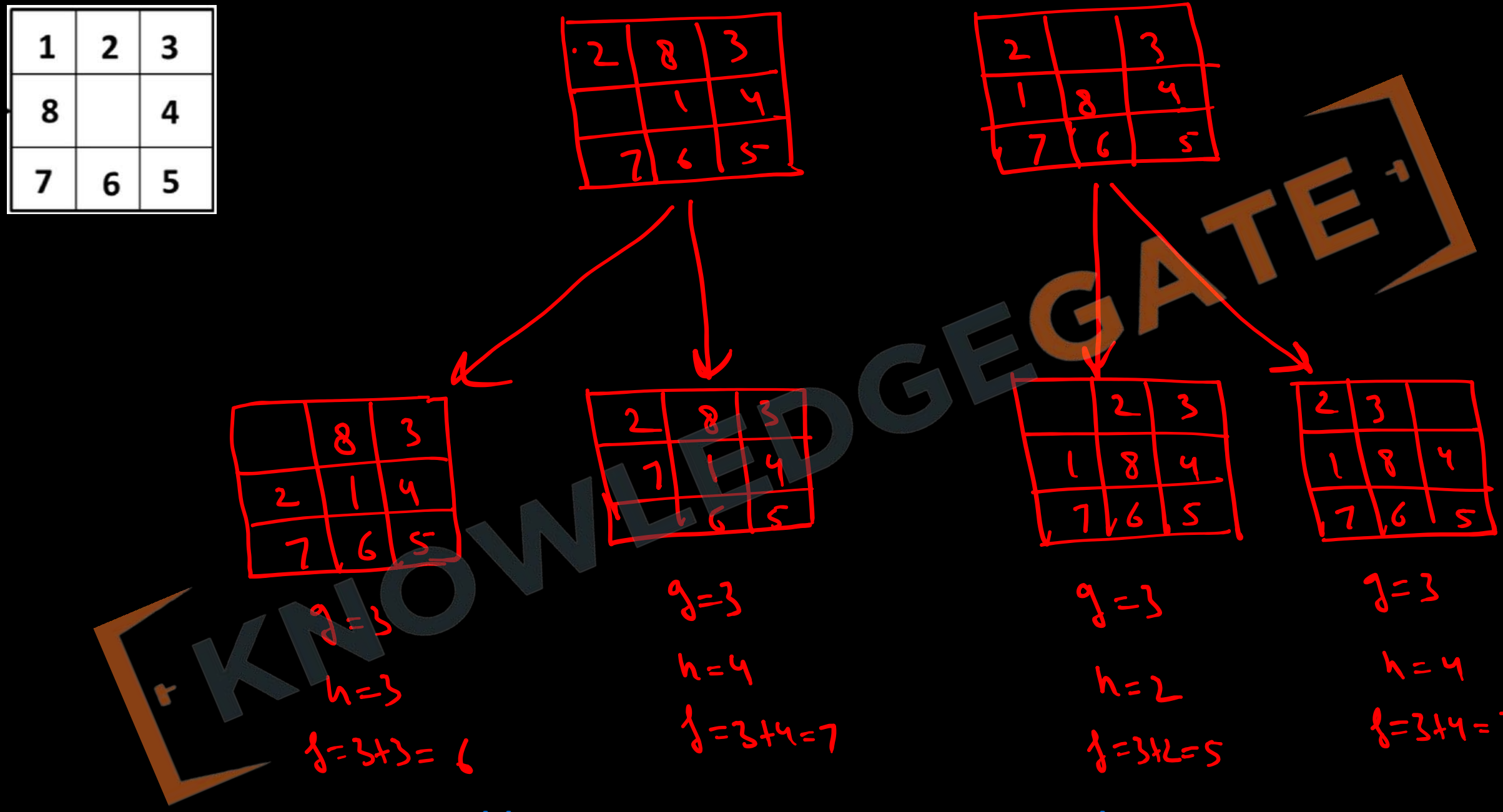


KNOWLEDGEGATE

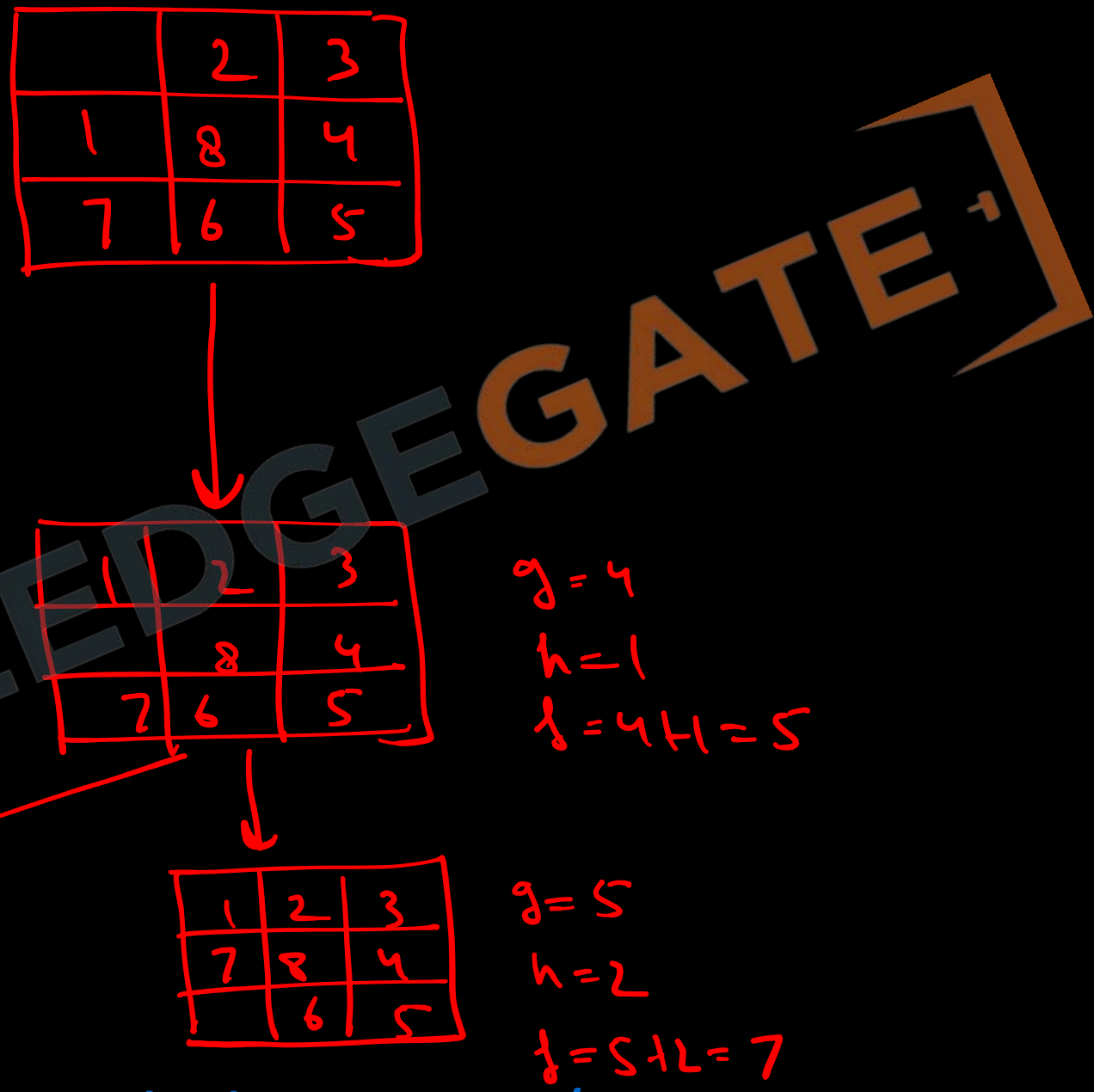
1	2	3
8		4
7	6	5



KNOWLEDGEGATE


1 2 3
8 4
7 6 5
2 8 3
1 4
7 6 5
2 3
1 8 4
7 6 5
8 3
2 1 4
7 6 5
g=3
h=3
f=3+3=6
2 8 3
7 1 4
6 5
g=3
h=4
f=3+4=7
2 3
1 8 4
7 6 5
g=3
h=2
f=3+2=5
2 3
1 8 4
7 6 5
g=3
h=4
f=3+4=7
KNOWLEDGEGATE

1	2	3
8		4
7	6	5

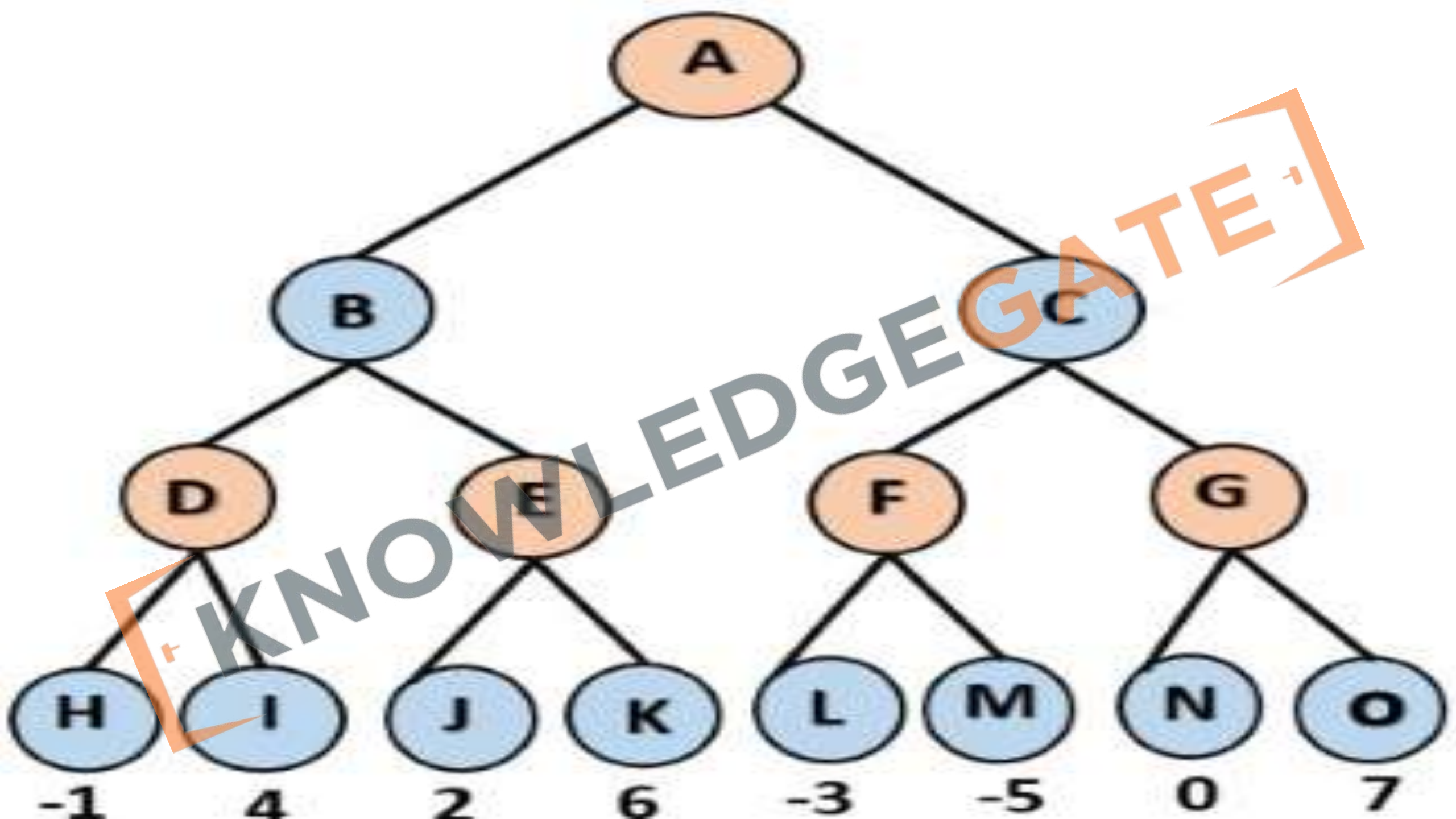


KNOWLEDGEGATE

The minimax algorithm

- **Definition**: Minimax is a specialized search algorithm used in game playing to determine the optimal sequence of moves for a player in a zero-sum game involving two players.
- **Mechanism**: It operates by recursively using backtracking to simulate all possible moves in the game, effectively searching through a game tree.
- **Decision Making**: The algorithm computes the minimax decision for the current state and uses a depth-first search algorithm for the exploration of the complete game tree.
- **Players' Roles**: In the game tree, there are two types of nodes: MAX nodes, where the algorithm selects the move with the maximum value, and MIN nodes, where the algorithm selects the move with the minimum value.
- **Games**: It's typically applied to perfect information games like chess, checkers, and tic-tac-toe.
- **Initial Values**: In the algorithm, the MAX player is often represented with negative infinity (-∞) as the initial worst case, while MIN is represented with positive infinity (∞).

KNOWLEDGEGATE



KNOWLEDGEGATE

- **Properties**:
  - **Completeness**: The algorithm is complete and will definitely find a solution if one exists.
  - **Optimality**: It guarantees to find the optimal strategy for both players.
  - **Time Complexity**: The time complexity is O(b^m), where b is the branching factor (the average number of child nodes for each node in the tree) and m is the maximum depth of the tree.
  - **Space Complexity**: The space complexity is also O(b^m), because of the need to store all the nodes in memory.

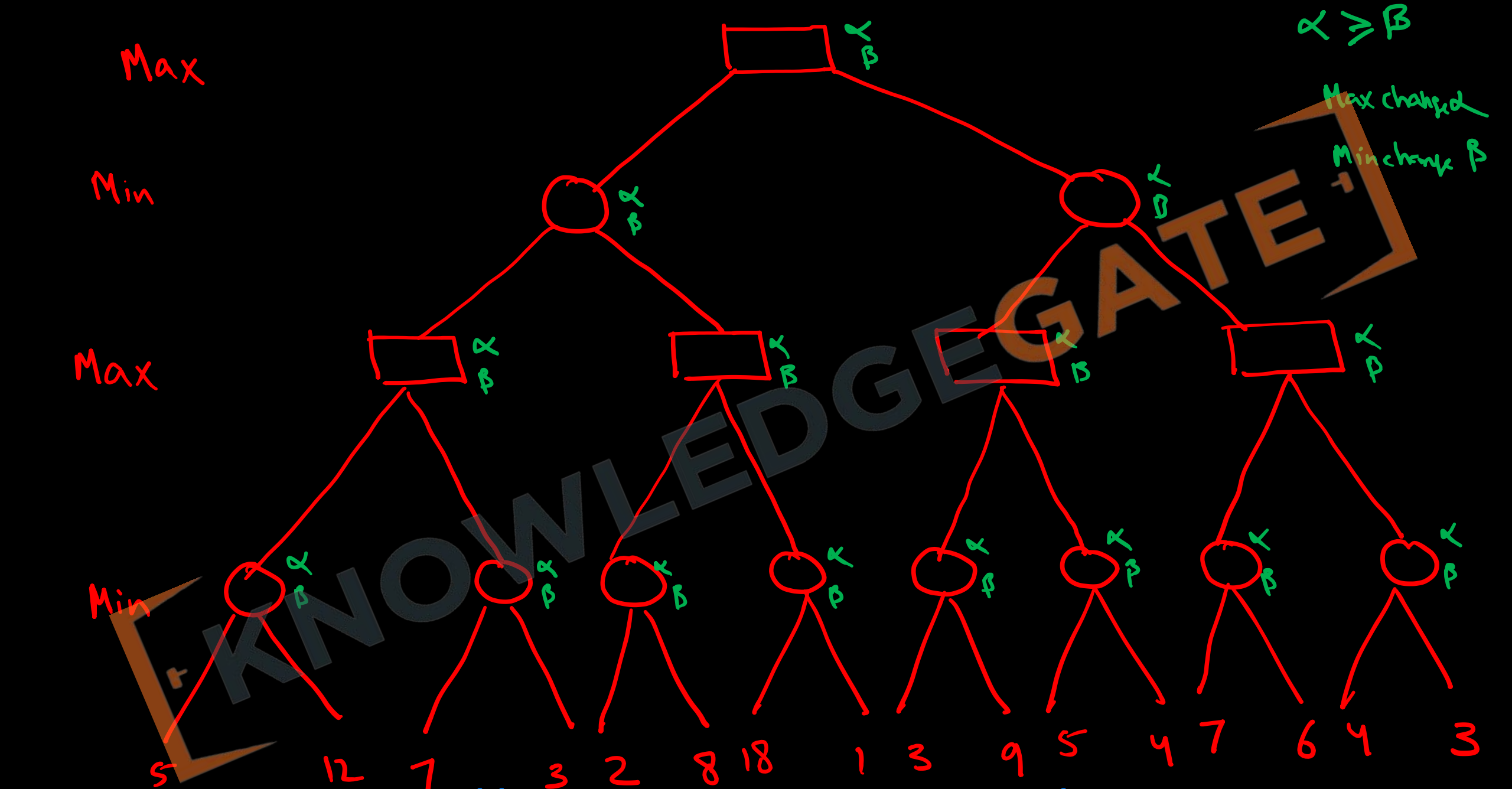
- **Limitations**:
  - The algorithm can be slow for complex games with many possible moves (like chess).
  - A game like chess can have around 35 possible moves at any point, leading to a vast number of possible game states to evaluate.

KNOWLEDGEGATE

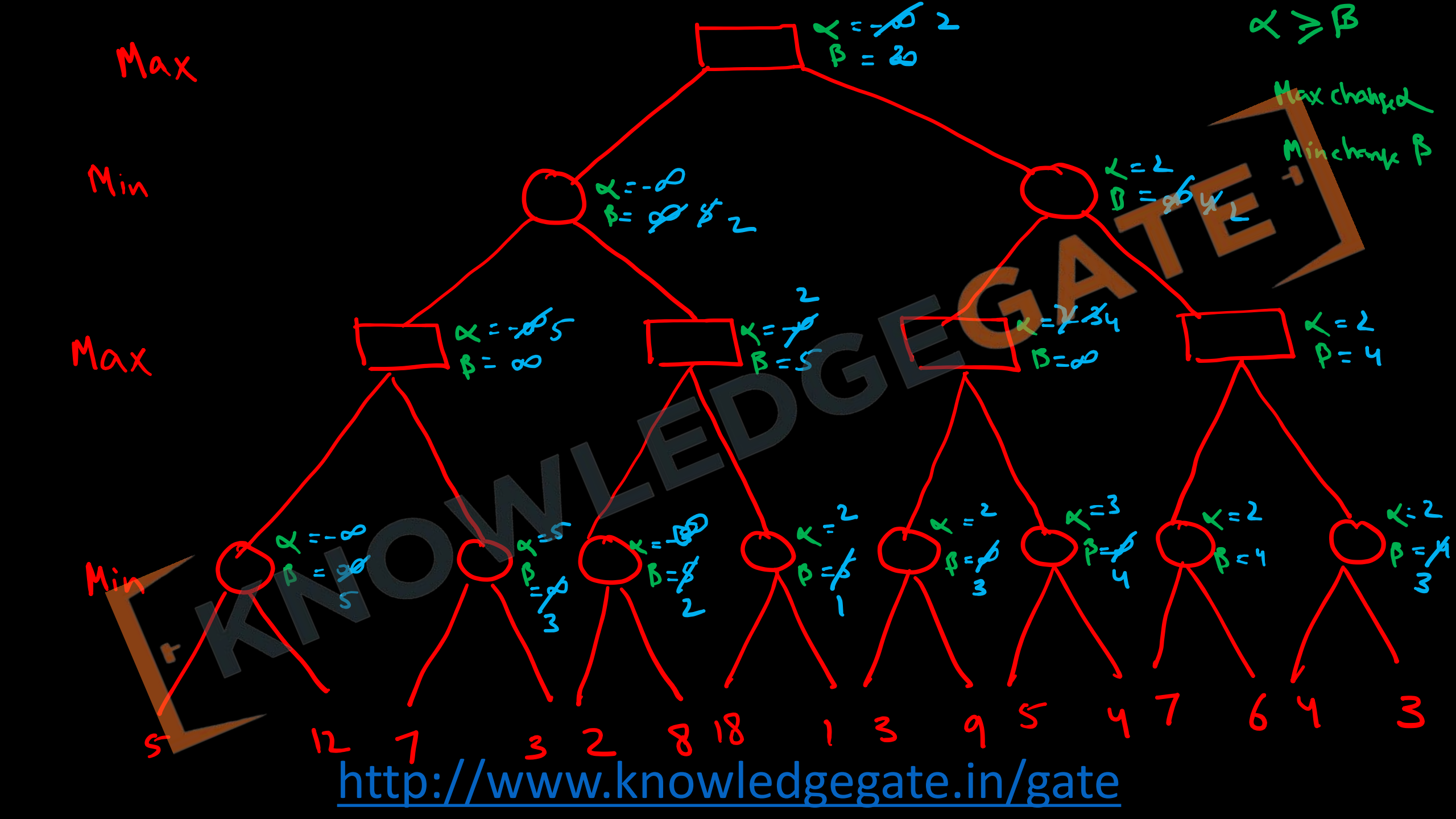
ALPHA–BETA PRUNING

- Alpha-beta pruning is an optimization technique for the minimax algorithm. It significantly reduces the number of nodes that need to be evaluated in the game tree without affecting the final result. Alpha-beta pruning skips the evaluation of certain branches in the game tree by identifying moves that are evidently worse than previously examined moves.
- **Alpha Value**: The best value that the maximizer currently can guarantee at that level or above.
- **Beta Value**: The best value that the minimizer currently can guarantee at that level or above.
- **Initial Values**: Alpha starts at negative infinity (-∞), and Beta starts at positive infinity (∞).
- **When to Prune**: A branch is pruned when the minimizer's best option (Beta) is less than the maximizer's best option (Alpha) since the maximizer will never allow the game to go down a path that could lead to a worse outcome than it has already found.

KNOWLEDGE GATE



http://www.knowledgegate.in/gate


Max
Min
Max
Min
α ≥ β
Max changes α
Min change β
α = -∞ 2
β = 20
α = -∞
β = ∞ 2
α = 2
β = ∞ 2
α = -∞ 5
β = ∞
α = 2
β = 5
α = 2 3 4
β = ∞
α = 2
β = 4
α = -∞
β = 5
α = 5
β = ∞ 3
α = -∞
β = 2
α = 2
β = 1
α = 2
β = 3
α = 3
β = 4
α = 2
β = 4
α = 2
β = ∞ 3
5
12
7
3
2
8
18
1
3
9
5
4
7
6
4
3
KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Benefits:

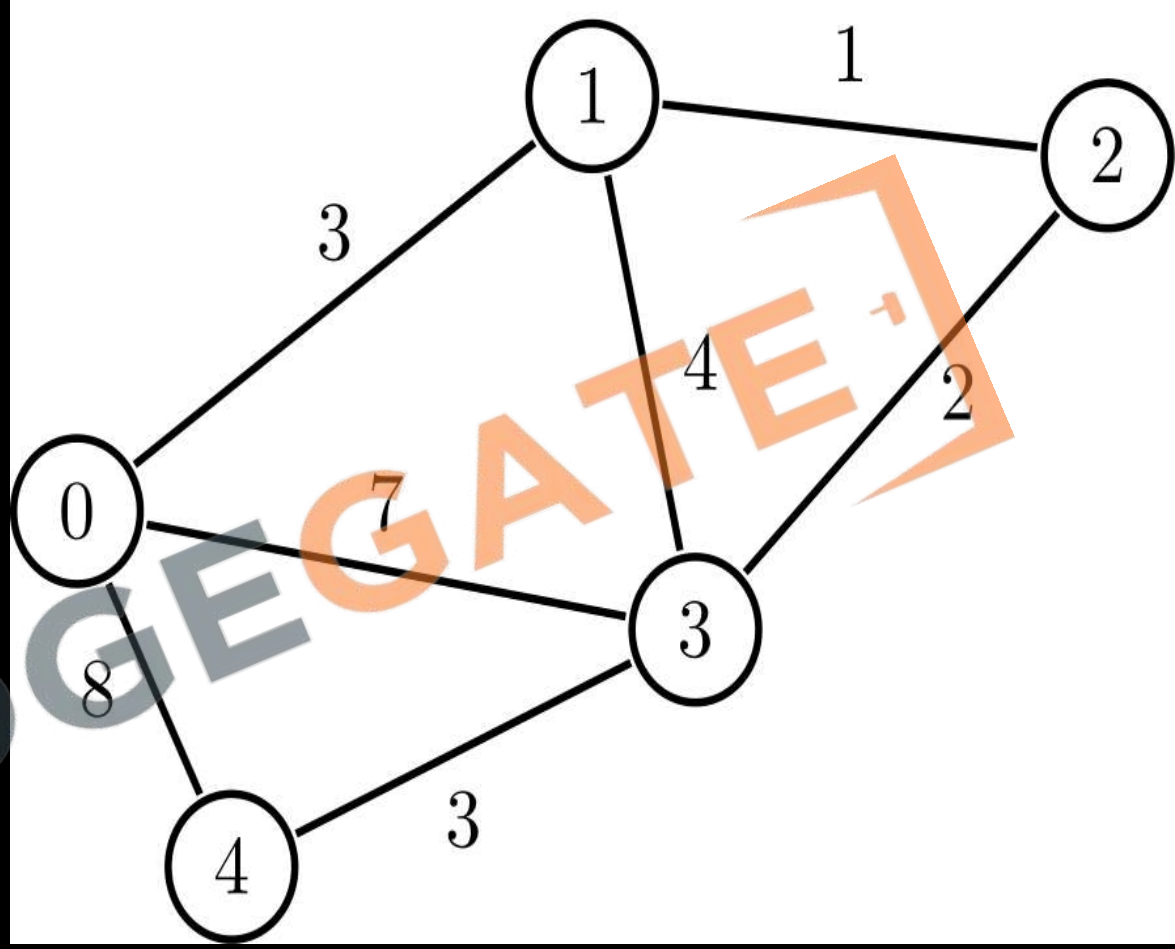
- **Efficiency**: Alpha-beta pruning can greatly reduce the number of nodes that are explored in the game tree, leading to faster decision-making.
- **Optimality**: The final decision made by the alpha-beta pruning is the same as the decision that would be made by the full minimax algorithm, ensuring optimality is not compromised.
- **Widely Applicable**: It can be applied to any game tree, not just two-player games, though the algorithm assumes perfect play on both sides.

KNOWLEDGEGATE

Branch and Bound

- **Objective**: To find the optimal solution in problems like the traveling salesman, knapsack, and job scheduling.
- **Mechanism**: Searches a state space tree of potential solutions by branching and bounding.
- **Branching**: Expands nodes, creating children that represent partial solutions.
- **Bounding**: Uses bounds to estimate the minimum or maximum value of regions in the space; prunes if the bound is worse than the best solution so far.
- **Strategies**: Can use best-first, breadth-first, or depth-first search to explore nodes.
- **Pruning**: Skips branches that can't improve the best current solution.
- **Efficiency**: Reduces the search space, making some intractable problems solvable.
- **Result**: Finds the global optimum if the bounding function is accurate.

KNOWLEDGEGATE



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

0/1 Knapsack using Branch and Bound

i	1	2	3	4
P	31	29	21	24
W	6	8	5	3

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Chapter-3

(KNOWLEDGE REPRESENTATION)

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

LOGICAL AGENTS

- Intelligence comes from Knowledge and Knowledge comes from the ability to reason. **Knowledge-based agents**: The intelligence of humans is achieved—not by purely reflex mechanisms but by processes of **reasoning** that operate on internal **representations** of knowledge.

- In AI, this approach to intelligence is embodied in **knowledge-based agents**. **Logical Agents:** Agents that can form representations of a complex world, use a process of inference to derive new representations about the world, and use these new representations to deduce what to do.

KNOWLEDGE GATE

KNOWLEDGE-BASED AGENTS

- The central component of a knowledge-based agent is its **knowledge base**, or KB which may initially contain some **background knowledge**. A knowledge base is a set of **sentences**. Each sentence is expressed in a language called a **knowledge representation language** and represents some assertion about the world.

- Sometimes we dignify a sentence with the name **axiom.** We add new sentences to the knowledge base and query what is known through **Inferences** (deriving new sentences from old).

KNOWLEDGEGATE

- A **proposition** is a declarative sentence (that is, a sentence that declares a fact) that is either true or false, but not both.
  1. Delhi is the capital of USA
  2. How are you doing
  3. 5<= 11
  4. Temperature is less than 10 C
  5. It is cold today
  6. Read this carefully
  7. X + y = z

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- Premises is a statement that provides reason or support for the conclusion(proposition). Premises(proposition) is always considered to be true.
- If a set of Premises(P) yield another proposition Q(Conclusion), then it is called an Argument.
- An argument is said to be valid if the conclusion Q can be derived from the premises by applying the rules of inference.

$\{P_1, P_2, P_3..., P_N\} \vdash Q$	P_1 P_2 P_3 . . . P_N ..... Q	$\{P_1 \wedge P_2 \wedge P_3 \wedge .... \wedge P_N\} \vdash Q$

KNOWLEDGEGATE

Types of proposition

- We use letters to denote **propositional variables** to represent propositions, just as letters are used to denote numerical variables. The conventional letters used for propositional variables are *p, q, r, s.*
- Many mathematical statements are constructed by combining one or more propositions. New propositions, called **compound propositions**, are formed from existing propositions using logical operators.

KNOWLEDGEGATE

Operators / Connectives

1. **Negation: -** let p be a proposition, then negation of p new proposition, denoted by ¬*p*, is the statement "it is not the case that p".

Negation	
P	¬P
F	
T	

KNOWLEDGEGATE

Conjunction

- Let *p* and *q* be propositions. The *conjunction* of *p* and *q*, denoted by *p* ∧ *q*, is the proposition "*p* and *q*." The conjunction *p* ∧ *q* is true when both *p* and *q* are true and is false otherwise.

Conjunction		
p	q	p ∧ q
F	F	
F	T	
T	F	
T	T	

KNOWLEDGEGATE

Disjunction

- Let *p* and *q* be propositions. The *disjunction* of *p* and *q*, denoted by *p* ∨ *q*, is the proposition. "*p* or *q*." The disjunction *p* ∨ *q* is false when both *p* and *q* are false and is true otherwise.

Disjunction		
p	q	p ∨ q
F	F	
F	T	
T	F	
T	T	

KNOWLEDGE GATE

http://www.knowledgegate.in/gate

Implication

1. Let *p* and *q* be propositions. The *conditional statement* $p \rightarrow q$ is the proposition "if *p*, then q". The conditional statement $p \rightarrow q$ is false when *p* is true and *q* is false, and true otherwise.
2. In conditional statement $p \rightarrow q$, *p* is called the hypothesis (or antecedent or premise) and q is called the conclusion.

Implication		
p	q	$p \rightarrow q$
F	F	
F	T	
T	F	
T	T	

KNOWLEDGEGATE

p	q	P → q	¬p	¬q	¬P → ¬q	q → p	¬q → ¬p	¬p V q
F	F							
F	T							
T	F							
T	T							

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- $p \rightarrow q$ *implication*
- $q \rightarrow p$ *converse*
- $\neg p \rightarrow \neg q$ *inverse*
- $\neg q \rightarrow \neg p$ *contra positive*
- $p \rightarrow q = \neg q \rightarrow \neg p$
- $p \rightarrow q$ *will be true if either p is false or q is true,* $p \rightarrow q = \neg p \vee q$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Bi-conditional

- Let *p* and *q* be propositions. The bi*conditional statement* $p \leftrightarrow q$ is the proposition.
  - " *p if and only* q".
  - "*p* is necessary and sufficient for *q*"
  - "if *p* then *q*, and conversely"
  - "*p* iff *q*."
  - $p \leftrightarrow q = (p \rightarrow q) \wedge (q \rightarrow p)$
- The biconditional statement $p \leftrightarrow q$ *is true when p and q have the same values, and false otherwise.*

Bi-conditional		
p	q	P ↔ q
F	F	
F	T	
T	F	
T	T	

KNOWLEDGEGATE

Type of cases

- **Tautology/valid: -** A propositional function which is always having truth in the last column, is called tautology. E.g. $p \vee \neg p$

p	¬ p	p V ¬ p
F	T	
T	F	

KNOWLEDGE GATE

- **<u>Contradiction/ Unsatisfiable</u>: -** A propositional function which is always having false in the last column, is called Contradiction. E.g. $p \wedge \neg p$

p	¬ p	p ∧ ¬ p
F	T	
T	F	

KNOWLEDGEGATE

- **Contingency:** - A propositional function which is neither a tautology nor a contradiction, is called Contingency. E.g. ***p*** ∨ ***q***

p	q	p ∨ q
F	F	
F	T	
T	F	
T	T	

KNOWLEDGEGATE

- **Satisfiable: -** A propositional function which is not contradiction is satisfiable. i.e. it must have at least one truth value in the final column e.g. $p \vee q$

KNOWLEDGEGATE

De Morgan's Laws

- $\neg(P \wedge Q) \equiv (\neg P \vee \neg Q)$

P	Q	$\neg(P \wedge Q)$	$\neg P \vee \neg Q$
T	T	F	F
T	F	T	T
F	T	T	T
F	F	T	T

- $\neg(P \vee Q) \equiv (\neg P \wedge \neg Q)$

P	Q	$\neg(P \vee Q)$	$\neg P \wedge \neg Q$
T	T	F	F
T	F	F	F
F	T	F	F
F	F	T	T

KNOWLEDGEGATE

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$
$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$
$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$
$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$
$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$
$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$
$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{De Morgan}$$
$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{De Morgan}$$
$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$
$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

KNOWLEDGE GATE

Well Formed Formulas

- A sentence S is also called a well formed formula is defined as follows:
- A symbol S is a sentence.
- If S is a sentence then ¬S is also a sentence
- If S is a sentence then (S) is also a sentence.
- If S and T are sentences then:
  - S ∧ T
  - S ∨ T
  - S ⇒ T
  - S ⇔ T

  Are also a sentence.
- A sentence resulted from finite number of applications of the above rules.

KNOWLEDGEGATE

TABLE 1 Rules of Inference.

Rule of Inference	*Name*
p $p \rightarrow q$ $\therefore q$	Modus ponens
$\neg q$ $p \rightarrow q$ $\therefore \neg p$	Modus tollens
$p \rightarrow q$ $q \rightarrow r$ $\therefore p \rightarrow r$	Hypothetical syllogism
$p \vee q$ $\neg p$ $\therefore q$	Disjunctive syllogism
p $\therefore p \vee q$	Addition
$p \wedge q$ $\therefore p$	Simplification
p q $\therefore p \wedge q$	Conjunction
$p \vee q$ $\neg p \vee r$ $\therefore q \vee r$	Resolution

KNOWLEDGEGATE
nowledgegate.in/gate

Disadvantages of Propositional Logic

- Propositional logic is too puny a language to represent knowledge of complex environments in a concise way. Thus we introduce first order logic which is sufficiently expressive to represent a good deal of our commonsense knowledge.

KNOWLEDGEGATE

First order Predicate Logic

- Sometime propositional logic cannot derive any meaningful information even though, we as human can understand that argument is meaningful or not.

- P_1: Every Indian like cricket
- P_2: Sunny is an Indian
- Q: Sunny Likes cricket

- The reason propositional logic fails here because using only inference system we can not conclude Q from P_1 and P_2.

KNOWLEDGEGATE

- In first order logic we understand, a new approach of subject and predicate to extract more information from a statement
  - 1 is a natural number (1 is subject, natural number is predicate)
  - we can write FOPL (short hand notation) for this as NatNo(1) = 1 is natural number
  - Similarly, we can understand the meaning of NatNo(2) as 2 is a natural number
  - NatNo(x): x is a natural number

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- If i say four Indian are there I_1, I_2, I_3, I_4
- I_1 likes cricket $\wedge$ I_2 likes cricket $\wedge$ I_3 likes cricket $\wedge$ I_4 likes cricket
- Cricket(I_1) $\wedge$ Cricket(I_2) $\wedge$ Cricket(I_3) $\wedge$ Cricket(I_4)
- But problem with this notation is as there is 130+ corers Indian this formula will become very long and in some case we actually do not know how many subjects are there in the universe of discourse. so, we again need a short hand formula.
- $\forall_x$ Cricket(x), if we confine x to be Indian then it means every x like cricket.

KNOWLEDGEGATE

- **<u>Universal quantifiers</u>**: - The universal quantification of a propositional function is the proposition that asserts

- P(x) is true for all values of x in the universe of discourse.

- The universe of discourse specifies the possible value of x.

- $\forall_x$ P(x), i.e. for all value of a P(x) is true

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- Let try some other statement ‘Some Indian like samosa’
  - if i say four Indian are there I_1, I_2, I_3, I_4
  - I_1 like samosa $\vee$ I_2 like samosa $\vee$ I_3 like samosa $\vee$ I_4 like samosa
  - Samosa(I_1) $\vee$ Samosa(I_2) $\vee$ Samosa(I_3) $\vee$ Samosa(I_4)
  - $\exists_x$ Samosa(x), if we confine x to be Indian then it means some x likes samosa.

KNOWLEDGEGATE

- **Existential quantifiers**: - with existential quantifier of a propositional that is true if and only if P(x) is true for at least one value of x in the universe of discourse.

- There exists an element x is the universe of discourse such that P(x) is true.

- $\exists_x P(x)$, i.e. for at least one value of a P(x) is true

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- let’s change the universe of discourse from Indian to human
  - if human is Indian then it likes cricket
  - Indian(x): x is an Indian
  - Cricket(x): x likes Cricket

  - if I_1 is Indian then likes cricket $\wedge$ if I_2 is Indian then likes cricket $\wedge$ if I_3 is Indian then likes cricket $\wedge$ if I_4 is Indian then likes cricket

  - $[Indian(I_1) \rightarrow cricket(I_1)] \wedge [Indian(I_2) \rightarrow cricket(I_2)] \wedge [Indian(I_3) \rightarrow cricket(I_3)] \wedge [Indian(I_4) \rightarrow cricket(I_4)]$

  - $\forall_x [Indian(x) \rightarrow cricket(x)]$

KNOWLEDGEGATE

- let’s change the universe of discourse from Indian to human
  - if human is Indian then it likes samosa
  - Indian(x): x is an Indian
  - Samosa(x): x likes Samosa

  - if I_1 is Indian then likes samosa $\vee$ if I_2 is Indian then likes samosa $\vee$ if I_3 is Indian then likes samosa $\vee$ if I_4 is Indian then likes samosa

  - $[Indian(I_1) \wedge samosa(I_1)] \vee [Indian(I_2) \wedge samosa(I_2)] \vee [Indian(I_3) \wedge samosa(I_3)] \vee [Indian(I_4) \wedge samosa(I_4)]$

  - $\exists_x [Indian(x) \wedge samosa(x)]$

KNOWLEDGEGATE

Negation

- $\neg [\forall_x P(x)] = \exists_x \neg P(x)$
- $\neg [\exists_x P(x)] = \forall_x \neg P(x)$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q consider the statement $\exists_x[P(x) \wedge \neg Q(x)]$, Which of the following is equivalent?

a) $\forall_x [P(x) \rightarrow Q(x)]$

b) $\forall_x [\neg P(x) \rightarrow Q(x)]$

c) $\neg \{\forall_x [P(x) \rightarrow Q(x)]\}$

d) $\neg \{\forall_x [\neg P(x) \rightarrow Q(x)]\}$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q let in a set of all integers

G (x, y): x is greater than y

“for any given positive integer, there is a greater positive integer”

a) $\forall_x \exists_y G(x, y)$

b) $\exists_y \forall_x G(x, y)$

c) $\forall_y \exists_x G(x, y)$

d) $\exists_x \forall_y G(x, y)$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q let in a set of all humans

L (x, y): x likes y

“there is someone, whom no one like”

a) $\forall_x \exists_y \{\neg L(x, y)\}$

b) $\{\neg \forall_x \exists_y L(x, y)\}$

c) $\neg \{\forall_y \exists_x L(x, y)\}$

d) $\neg \{\exists_y \forall_x L(x, y)\}$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q The CORRECT formula for the sentence, **"not all rainy days are cold"** is

a) $\forall_d$ (Rainy(d) $\wedge$ ~Cold(d))

b) $\forall_d$ (~Rainy(d) $\rightarrow$ Cold(d))

c) $\exists_d$ (~Rainy(d) $\rightarrow$ Cold(d))

d) $\exists_d$ (Rainy(d) $\wedge$ ~Cold(d))

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Consider the statement

"Not all that glitters is gold"

Predicate glitters(x) is true if x glitters and predicate gold(x) is true if x is gold. Which one of the following logical formulae represents the above statement?

a) $\forall_x$: glitters(x) $\Rightarrow$ $\neg$gold(x)

b) $\forall_x$: gold(x) $\Rightarrow$ glitters(x)

c) $\exists_x$: gold(x) $\wedge$ $\neg$glitters(x)

d) $\exists_x$: glitters(x) $\wedge$ $\neg$gold(x)

KNOWLEDGEGATE

Q What is the logical translation of the following lowing statement?

"None of my friends are perfect."

A) $\exists_x(F(x) \land \neg P(x))$

B) $\exists_x(\neg F(x) \land P(x))$

C) $\exists_x(\neg F(x) \land \neg P(x))$

D) $\neg\exists_x(F(x) \land P(x))$

KNOWLEDGEGATE

Q Consider the following well-formed formulae:

1) $\neg\forall x(P(x))$ **2)** $\neg\exists x(P(x))$ **3)** $\neg\exists x(\neg P(x))$ **4)** $\exists x(\neg P(x))$

Which of the above are equivalent?

a) I and III

b) I and IV

c) II and III

d) II and IV

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Q Which one of the following is the most appropriate logical formula to represent the statement? **"Gold and silver ornaments are precious".** The following notations are used:

G(x): x is a gold ornament

S(x): x is a silver ornament

P(x): x is precious

(A) $\forall_x (P(x) \rightarrow (G(x) \wedge S(x)))$

(B) $\forall_x ((G(x) \wedge S(x)) \rightarrow P(x))$

(C) $\exists_x ((G(x) \wedge S(x)) \rightarrow P(x)$

(D) $\forall_x ((G(x) \vee S(x)) \rightarrow P(x))$

KNOWLEDGEGATE

Q Which one of the first order predicate calculus statements given below correctly express the following English statement? **"Tigers and lions attack if they are hungry or threatened"**

a) $\forall_x[(tiger(x) \wedge lion(x)) \rightarrow (hungry(x) \vee threatened(x)) \rightarrow attacks(x)]$

b) $\forall_x[(tiger(x) \vee lion(x)) \rightarrow (hungry(x) \vee threatened(x)) \wedge attacks(x)]$

c) $\forall_x[(tiger(x) \vee lion(x)) \rightarrow attacks(x) \rightarrow (hungry(x) \vee threatened(x))]$

d) $\forall_x[(tiger(x) \vee lion(x)) \rightarrow (hungry(x) \vee threatened(x)) \rightarrow attacks(x)]$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Conjunctive Normal Form

- A formula is in conjunctive normal form (CNF) or clausal normal form if it is a conjunction of one or more clauses, where a clause is a disjunction of literals; otherwise put, it is an AND of ORs.

- **Procedure for converting Propositional logic to CNF**:

- **Example:** $A \Leftrightarrow (B \vee C)$ into CNF.

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.
   $(A \Rightarrow (B \vee C)) \wedge ((B \vee C) \Rightarrow A)$

2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$:
   $(\neg A \vee B \vee C) \wedge (\neg(B \vee C) \vee A)$.

KNOWLEDGEGATE

3. CNF requires ¬ to appear only in literals, so we “move ¬ inwards” by repeated application of the following equivalences

In the example, we require just one application of the last rule:

- (¬A ∨ B ∨ C) ∧ ((¬B ∧ ¬C) ∨ A).

4. Now we have a sentence containing nested ∧ and ∨ operators applied to literals. We apply the distributivity law, distributing ∨ over ∧ wherever possible.

- (¬A ∨ B ∨ C) ∧ (¬B ∨ A) ∧ (¬C ∨ A).

- The original sentence is now in CNF, as a conjunction of three clauses.

KNOWLEDGEGATE

Rules of Inference

- Rules of inference that have been introduced in propositional logic as I told you can also be used in predicate logic. The only thing that makes the difference is the quantifiers.

- **Four different methods to deal with quantified sentences in first order logic:**

  - **Universal specialization / Universal instantiation.**
  - **Existential instantiation**
  - **Existential generalization**
  - **Universal generalization / universal introduction.**

KNOWLEDGEGATE

Universal specialization / Universal instantiation

- Universal instantiation is also called as universal elimination or UI is a valid inference rule. It can be applied multiple times to add new sentences. As per UI, we can infer any sentence obtained by substituting a ground term for the variable.

- The UI rule state that we can infer any sentence P(c) by substituting a ground term c (a constant within domain x) from ∀ x P(x) for any object in the universe of discourse.

- It can be represented as: $\frac{\forall x\, P(x)}{P(c)}$

Example: "Every person like ice-cream”:

∀x P(x)

So we can infer that "John likes ice-cream“: P(c)

[KNOWLEDGEGATE]

Existential Instantiation

- Existential instantiation is also called as Existential Elimination, which is a valid inference rule in first-order logic. It can be applied only once to replace the existential sentence.

- This rule states that one can infer P(c) from the formula given in the form of ∃x P(x) for a new constant symbol c.The restriction with this rule is that c used in the rule must be a new term for which P(c) is true.

- It can be represented as:

$$\frac{\exists x\, P(x)}{P(c)}$$

KNOWLEDGEGATE

Existential introduction

- An existential introduction is also known as an existential generalization, which is a valid inference rule in first-order logic. This rule states that if there is some element c in the universe of discourse which has a property P, then we can infer that there exists something in the universe which has the property P.
- It can be represented as: $\frac{P(c)}{\exists x P(x)}$
- **Example:** "Priyanka got good marks in English."
- "Therefore, someone got good marks in English."

KNOWLEDGEGATE

Universal Generalization

- Universal generalization is a valid inference rule which states that if premise P(c) is true for any arbitrary element c in the universe of discourse, then we can have a conclusion as ∀ x P(x).
- It can be represented as: $\frac{P(c)}{\forall x\, P(x)}$
- This rule can be used if we want to show that every element has a similar property. In this rule, x must not appear as a free variable.
- **Example:** Let's represent, P(c): "A byte contains 8 bits", so for ∀ x P(x) "All bytes contain 8 bits.", it will also be true.

KNOWLEDGEGATE

Forward Chaining

- **Begins with what is known**: starts with basic facts or data.
- **Moves forward**: applies rules to these facts to derive new information.
- **Data-driven**: new conclusions are made as data is processed.
- **Adds knowledge**: builds upon existing information incrementally.
- Stops when no further deductions can be made or a goal is reached.

KNOWLEDGE GATE

Example: A Rice Crop Management System

- **Initial Observation**: A farmer in Odisha notices that some rice plants are shorter than usual and have yellow leaves.
- **Rule 1**: If rice plants are short and have yellow leaves, they might lack nitrogen.
- **Rule 2**: If rice plants lack nitrogen, then using nitrogen-rich fertilizer may help.
- **Rule 3**: If the season is monsoon and the problem is nitrogen deficiency, then use urea fertilizer, which is suitable for wet conditions.

Forward Chaining Process:

1. **Observing Symptoms**: The farmer inputs into the system that the rice plants are short with yellow leaves.
2. **Applying Knowledge**: The system uses Rule 1 to deduce that the plants might be suffering from nitrogen deficiency.
3. **Taking Action**: Based on Rule 2, the system suggests that applying a nitrogen-rich fertilizer could be beneficial.
4. **Considering Conditions**: Seeing that it is the monsoon season, Rule 3 advises the farmer to use urea fertilizer specifically.
5. **Outcome**: The farmer follows the advice, applies urea, and over time the rice plants become healthier and grow taller.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

A

B

$A \wedge B \longrightarrow L$

$A \wedge P \longrightarrow L$

$B \wedge L \longrightarrow M$

$L \wedge M \longrightarrow P$

$P \longrightarrow Q$

KNOWLEDGEGATE

Backward Chaining

- **Starts with a goal**: looks at the desired outcome first.
- **Works backward**: checks if current knowledge supports the goal.
- **Goal-driven**: each step is aimed at proving the end objective.
- **Deduces necessities**: determines what conditions must be met for the goal.
- Stops when it reaches the beginning facts or rules, or the goal is proven or disproven.

KNOWLEDGEGATE

Example: Deciding on a Postgraduate Course

- **Goal**: A student wants to determine the best postgraduate course to enroll in.
- **Question**: What course should the student choose for a successful career?
- **Check Desired Outcome**: The desired outcome is a fulfilling and prosperous career in a field the student is passionate about.
- **Compare Interests**: The student is interested in technology and innovation.
- **Search for Career Paths**: The system considers various career paths that align with technology and innovation.

- **Hypothesis 1**: Is a Master's in Computer Science suitable?
- **Check for Evidence**: The system checks the student's academic background, skills, and job market trends for computer science graduates.

- **Hypothesis 2**: Could an MBA in Technology Management be an alternative?

- **Check for Evidence**: The system evaluates the student's leadership potential, market demand for technology managers, and the student's long-term career goals.

- **Solution if Matched**: If the evidence strongly supports one path over the other based on the student's profile and job market trends, the system recommends that course.

- **Outcome**: The student chooses the recommended course which aligns with their interests, background, and market demand, leading to a more focused educational path towards a desired career.

KNOWLEDGE GATE
n/gate

A

B

$A \wedge B \longrightarrow L$

$A \wedge P \longrightarrow L$

$B \wedge L \longrightarrow M$

$L \wedge M \longrightarrow P$

$P \longrightarrow Q$

KNOWLEDGEGATE

Forward Chaining	Backward Chaining
1. Data-driven approach: begins with known facts.	1. Goal-driven approach: starts with the desired conclusion.
2. Bottom-up reasoning: builds up from facts to a conclusion.	2. Top-down reasoning: works backwards from goal to facts.
3. Uses Modus Ponens: applies rules to derive new information.	3. Uses Modus Tollens: checks if conclusions can lead to the goal.
4. Continues until no new information can be derived.	4. Continues until it reaches the starting facts or rules.
5. Common in real-time systems and production rule systems.	5. Used in diagnostic systems where the end goal is known.

KNOWLEDGEGATE

Resolution

- Resolution in AI is a logical inference rule used to deduce new information or solve problems by finding contradictions.
- It involves converting logical statements into a standardized form called clausal form, which is a set of normalized logical expressions.
- Contradictory pairs of clauses are identified, and through a process of unification and combination, new clauses are generated.
- This process is repeated iteratively until either a contradiction is found (represented by an empty clause) or no new information can be deduced.
- Resolution is particularly powerful in automated theorem proving and logic programming, such as in Prolog, where it's used to infer conclusions from given facts and rules.

KNOWLEDGEGATE

- All birds fly: $\forall x$ (Bird(x) $\rightarrow$ Fly(x))
- Tweety is a bird: Bird(Tweety)

We want to prove that Tweety can fly. Using resolution:

1. We negate the query ¬Fly(Tweety) and add it to our knowledge base.
2. Our clauses in clausal form are:
   - ¬Bird(x) $\vee$ Fly(x) (from $\forall x$ (Bird(x) $\rightarrow$ Fly(x)))
   - Bird(Tweety)
   - ¬Fly(Tweety) (negated query)
3. Now we apply resolution:
   - We resolve ¬Fly(Tweety) with ¬Bird(x) $\vee$ Fly(x) using unification by substituting x with Tweety.
   - This gives us ¬Bird(Tweety) $\vee$ Fly(Tweety).
4. Finally, we resolve Bird(Tweety) with ¬Bird(Tweety) $\vee$ Fly(Tweety), which gives us Fly(Tweety), the desired fact.

Since we started with ¬Fly(Tweety) and ended up with Fly(Tweety), we have derived a contradiction, meaning our original negation of the query must be false, thus proving that, according to our knowledge base, Tweety can fly.

KNOWLEDGEGATE

Knowledge Representation

- **Definition**: It's the study of how knowledge can be represented in a computer system to mimic human thought and reasoning.
- **Components**:
  - Facts: True information relevant to a specific domain.
  - Representation: How facts are formally expressed, often using logical structures.
- **Example**: For the fact "Charlie is a dog," a logical representation could be Dog(Charlie).
- **Characteristics**:
  - **Syntax and semantics**: Clearly defined rules for structure and meaning.
  - **Expressive capacity**: Ability to convey all necessary domain knowledge.
  - **Efficiency**: Supports effective computing and reasoning.

KNOWLEDGEGATE

Example Scenario: Knowledge Representation of a Library System

Fact:

"Book X is available in the library."

Representation in Logical Structure:

Using a logical statement, this fact can be represented as:

`Available(Book X)`

Characteristics Applied:

- **Syntax and Semantics**: The function `Available()` clearly defines that it checks for the availability of a specific book.
- **Expressive Capacity**: This structure allows the system to represent the availability of any book in the library by substituting "Book X" with any book's name.
- **Efficiency**: Such representation supports quick queries about book availability, facilitating efficient information retrieval and decision-making processes in the library's computer system.

KNOWLEDGEGATE

Knowledge Acquisition

- **Definition**:
  - Involves gathering expertise for use in expert systems, organizing it into structured formats like IF-THEN rules.
  - Also refers to the process of absorbing information into memory, focusing on effective retrieval.

- **Procedure**:
  - **Identification**: Break down complex issues into parts.
  - **Conceptualization**: Define key concepts.
  - **Formalization**: Organize knowledge formally for programmatic use.
  - **Implementation**: Code the structured knowledge.
  - **Testing**: Check and validate the system's functionality.

KNOWLEDGEGATE

To create an expert system that helps diagnose skin diseases based on symptoms and patient history.

Knowledge Acquisition Process:

1. **Identification**:
   - Break down the domain of skin diseases into manageable categories, such as infectious, non-infectious, inflammatory, and allergic skin conditions.
2. **Conceptualization**:
   - Define key concepts and terms, such as "eczema," "psoriasis," and "dermatitis." Understand the common symptoms associated with each category, like redness, itching, or peeling.
3. **Formalization**:
   - Organize the knowledge into a structured format:
     - IF the patient exhibits symptoms X, Y, and Z, AND has a history of A,
     - THEN the likely diagnosis is B.
   - Example rule: IF the patient has red, itchy patches, AND has a family history of allergies, THEN consider "eczema" as a diagnosis.
4. **Implementation**:
   - Code the knowledge into the expert system using a suitable programming language, integrating decision-making logic based on the IF-THEN rules.
5. **Testing**:
   - Validate the system's functionality by inputting test cases (e.g., symptoms of known diseases) to see if the expert system correctly diagnoses them based on the programmed knowledge.

KNOWLEDGEGATE
gate

- In artificial intelligence (AI), knowledge representation is a critical area that enables systems to simulate human-like reasoning and decision-making. Various techniques are employed to represent knowledge in AI systems, each with specific applications and advantages. Here are some key techniques:

- **Logic-Based Representation**:
  - **Propositional Logic**: Uses simple statements that are either true or false.
  - **Predicate Logic**: Extends propositional logic with predicates that can express relations among objects and quantifiers to handle multiple entities.
  - **Example**: Representing the relationship, "All humans are mortal," can be written in predicate logic as $\forall x$ (Human(x) $\rightarrow$ Mortal(x)).

[KNOWLEDGEGATE]

http://www.knowledgegate.in/gate

- **Semantic Networks**:
  - Graph structures used to represent semantic relations between concepts. Nodes represent objects or concepts, and edges represent the relations between them.
  - Example: A node for "Socrates" linked by an "is a" edge to a "Human" node, and "Human" linked to "Mortal".

- **Frame-Based Representation**:
  - Uses data structures called frames, which are similar to objects in object-oriented languages. Frames allow the grouping of related properties and actions.
  - Example: A frame for "Bird" might include properties like "has feathers," "can fly," and actions like "lay eggs."

KNOWLEDGEGATE

- **Production Rules**:
  - Consist of sets of rules in the form of IF-THEN constructs that are used to derive conclusions from given data.
  - Example: IF the patient has a fever and rash, THEN consider a diagnosis of measles.

- **Ontologies**:
  - Formal representations of a set of concepts within a domain and the relationships between those concepts. They are used to reason about the entities within that domain and are often employed in semantic web applications.
  - Example: In a medical ontology, concepts like "symptom," "disease," and "treatment" might be related in ways that define what symptoms are commonly associated with a disease.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Bayesian Networks**:
  - Probabilistic models that represent a set of variables and their conditional dependencies via a directed acyclic graph (DAG). These are useful for handling uncertainty in AI applications.
  - Example: A network might represent the probabilistic relationships between diseases and symptoms, allowing the system to infer disease probabilities given observed symptoms.

- **Neural Networks**:
  - Although typically classified under machine learning, neural networks can represent complex relationships between inputs and outputs through learned weights and can effectively capture and model knowledge after training.
  - Example: A neural network trained on historical weather data might predict future weather conditions.

KNOWLEDGEGATE

- **Chapter-4** (SOFTWARE AGENTS): Architecture for Intelligent Agents - Agent communication - Negotiation and Bargaining - Argumentation among Agents - Trust and Reputation in Multi-agent systems.

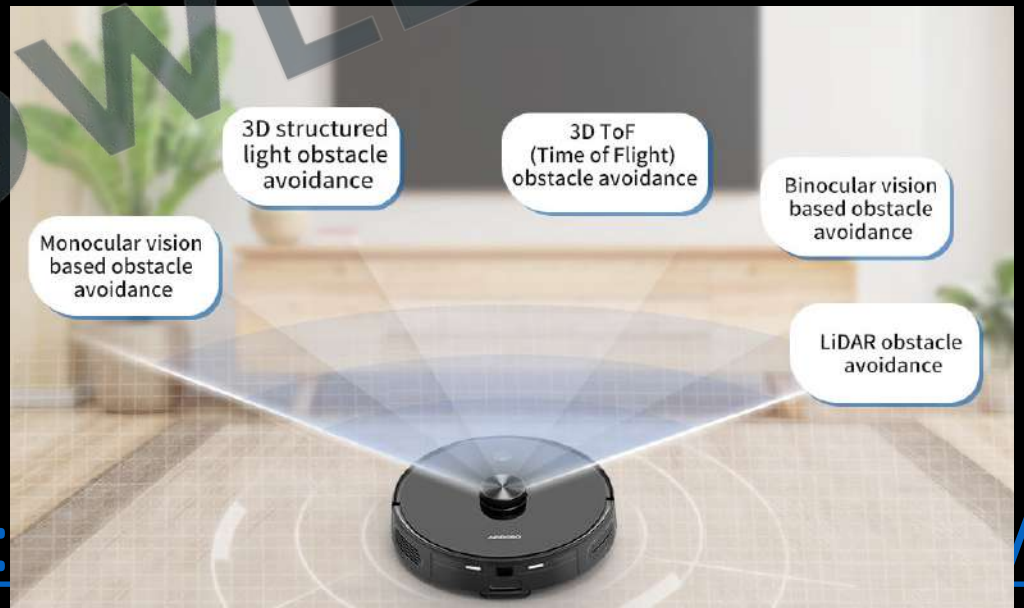
KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Intelligent agents

Intelligent agents in the context of artificial intelligence (AI) refer to autonomous entities that observe through sensors and act upon an environment using actuators (i.e., they perform actions) to achieve specified goals autonomously. Here's a more detailed breakdown:

- **Autonomy**: Intelligent agents operate without the direct intervention of humans or others, and have some kind of control over their actions and internal state.
  - **Example**: A Roomba vacuum cleaner navigates and cleans a room by itself, adjusting its path when encountering obstacles.



KNOWLEDGEGATE

http: /gate

- **Sensory Capability**: They perceive their environment through sensory organs or data input mechanisms.
  - **Example**: A smart thermostat like Nest uses sensors to detect the temperature in a room and the presence of people to adjust heating and cooling settings.

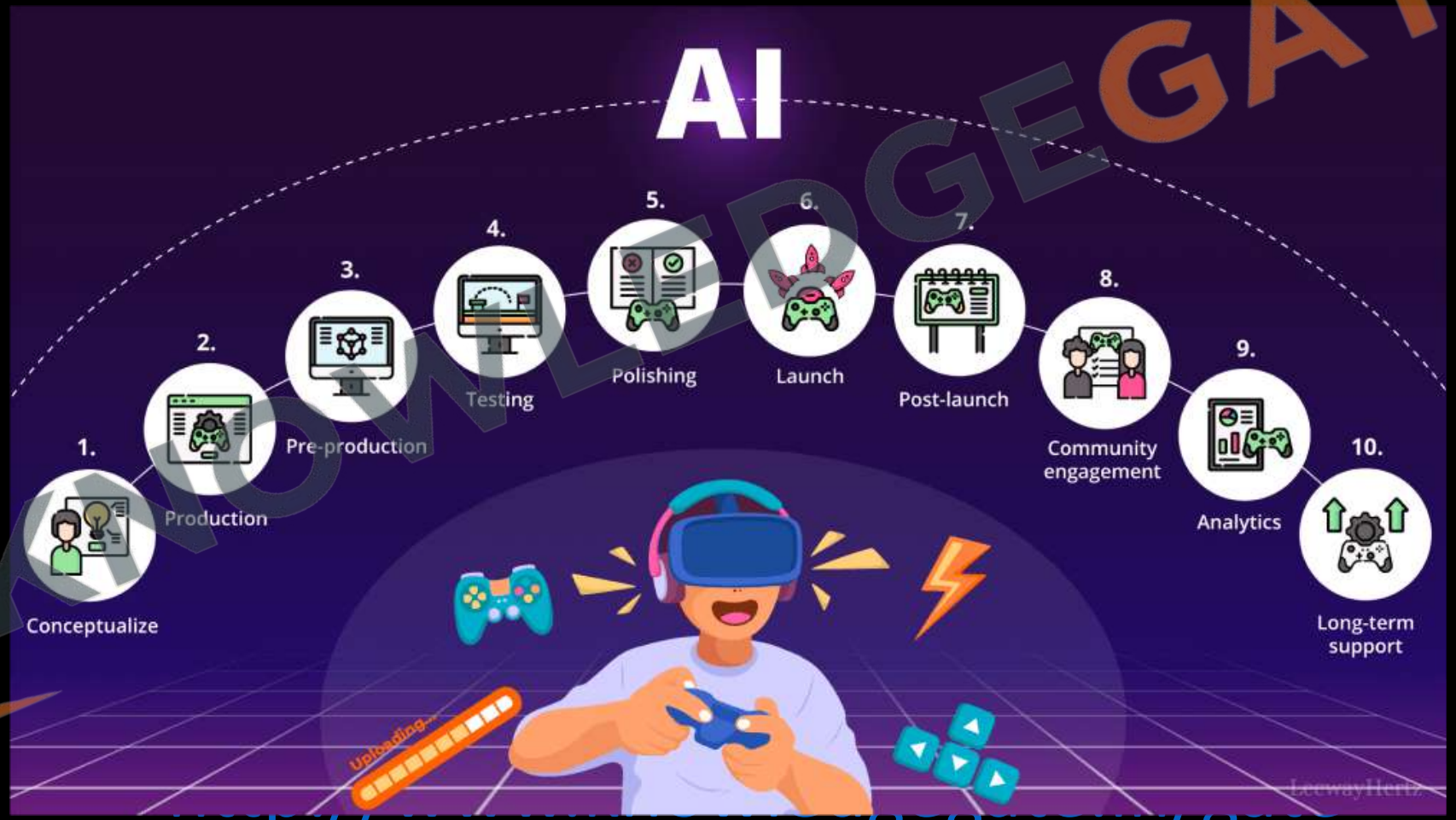
KNOWLEDGEGATE

- **Actuation**: Intelligent agents can act upon the environment using effectors (physical or virtual actuators) to change its state or the agent's own state to achieve goals.
  - **Example**: A chatbot on a customer service website interacts with users, providing answers to questions and guiding them through troubleshooting steps.

KNOWLEDGEGATE

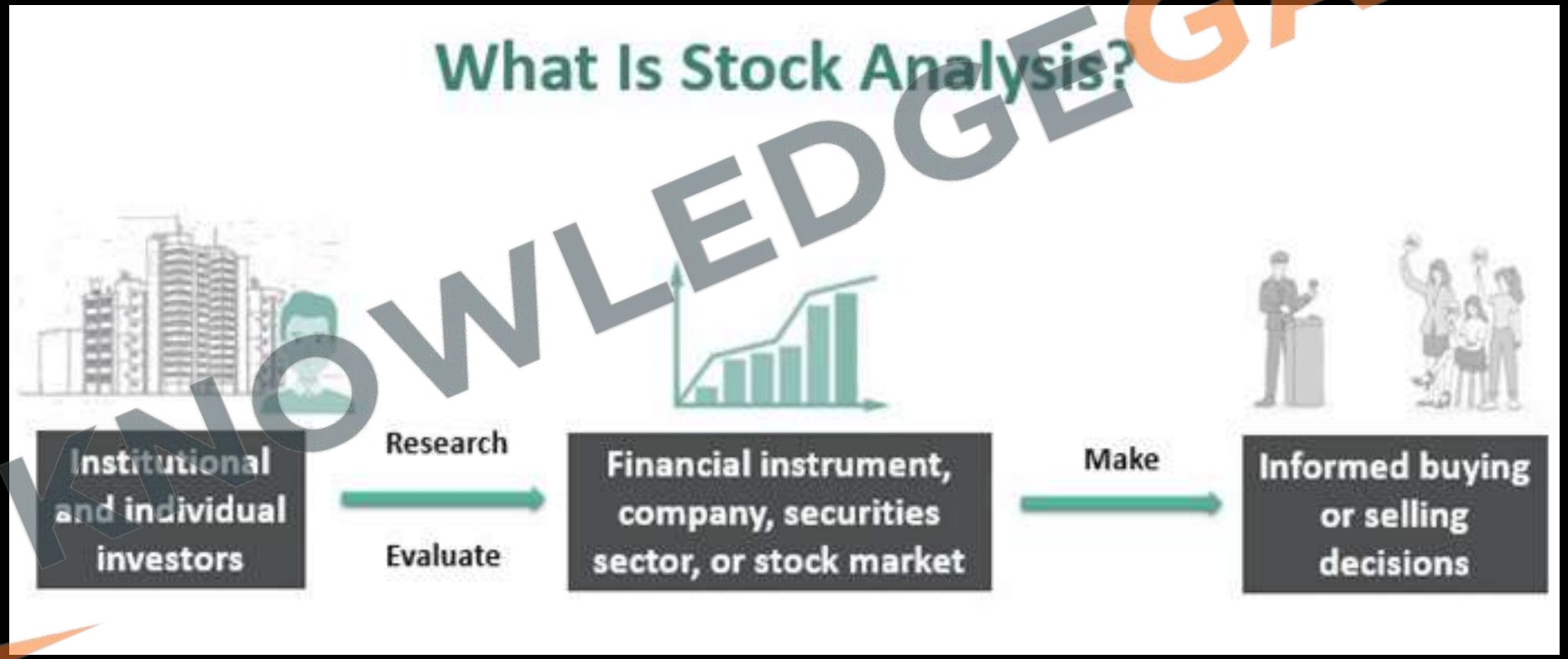
htt

- **Goal-Oriented Behavior**: They are designed with specific objectives in mind and can make decisions to fulfill those objectives.
  - **Example**: An AI in a strategy video game seeks to win against human players by managing resources and deploying units strategically.



KNOWLEDGEGATE

- **Rationality**: An intelligent agent acts to achieve the best outcome, or when there is uncertainty, the best expected outcome.
  - **Example**: An investment bot analyses stock market trends and makes buying or selling decisions to maximize investor returns.



KNOWLEDGEGATE

- **Learning and Adaptation**: Many intelligent agents have the ability to learn from their environment and experiences to improve their performance over time.
  - **Example**: Spotify's recommendation system learns from the user's listening habits to suggest new songs and playlists that match their tastes.

KNOWLEDGE GATE

- **Interaction**: They may also be capable of interacting with other agents, including both intelligent agents and humans, in order to complete tasks or to simulate human interaction.
  - **Example**: Siri, Apple's virtual assistant, interacts with users to understand and execute a variety of tasks such as setting reminders or sending messages.

KNOWLEDGEGATE

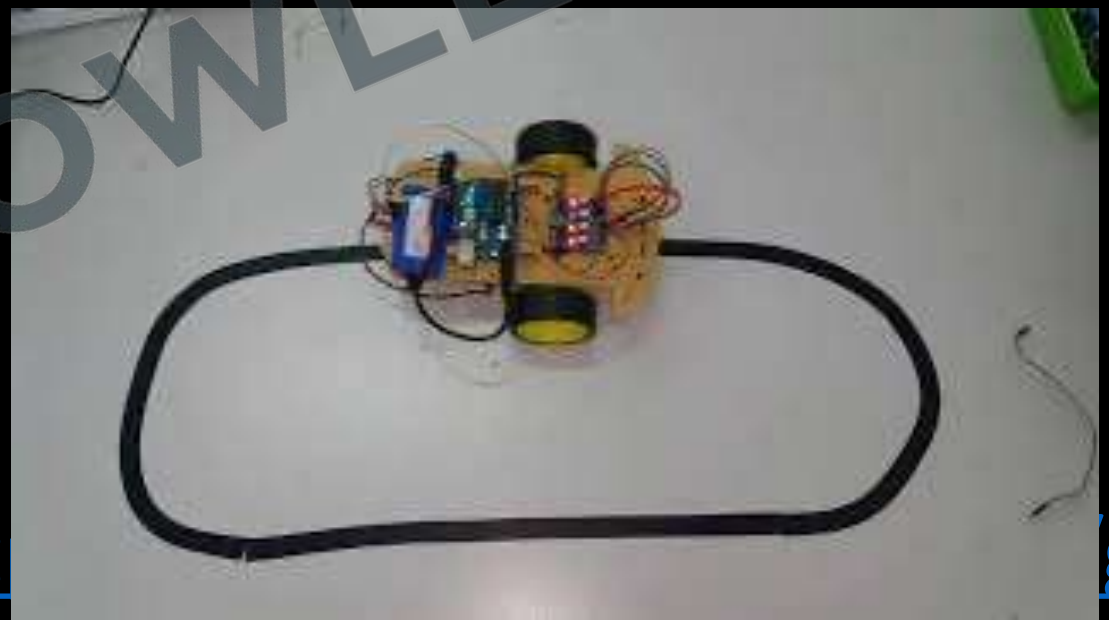
- Intelligent agents can range from simple, rule-based systems that perform automated tasks, to complex, learning and adapting systems with advanced features like natural language processing, problem-solving, and even forms of consciousness or self-awareness. They are applied in various domains such as virtual personal assistants, robots in manufacturing, autonomous vehicles, and software agents in complex simulation systems.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Architecture for intelligent agents

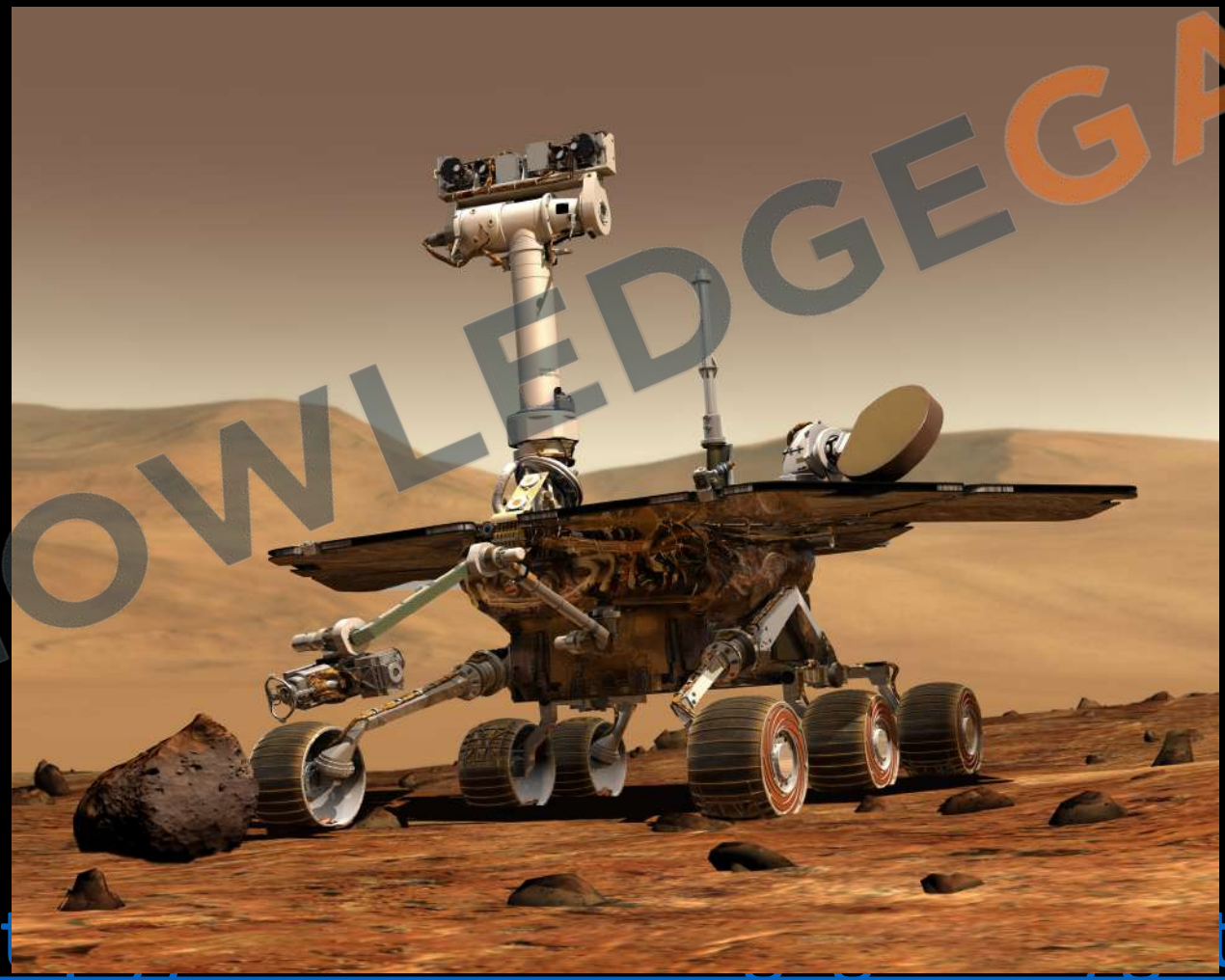
- The architecture for intelligent agents broadly refers to the underlying structure or model that defines how the agents perceive their environment, make decisions, and take actions. There are several types of architectures, each with its own approach to processing and responding to stimuli:
- **Reactive Architecture**:
  - Example: A line-following robot that adjusts its path based on real-time sensor input without an internal representation of the environment.



KNOWLEDGEGATE

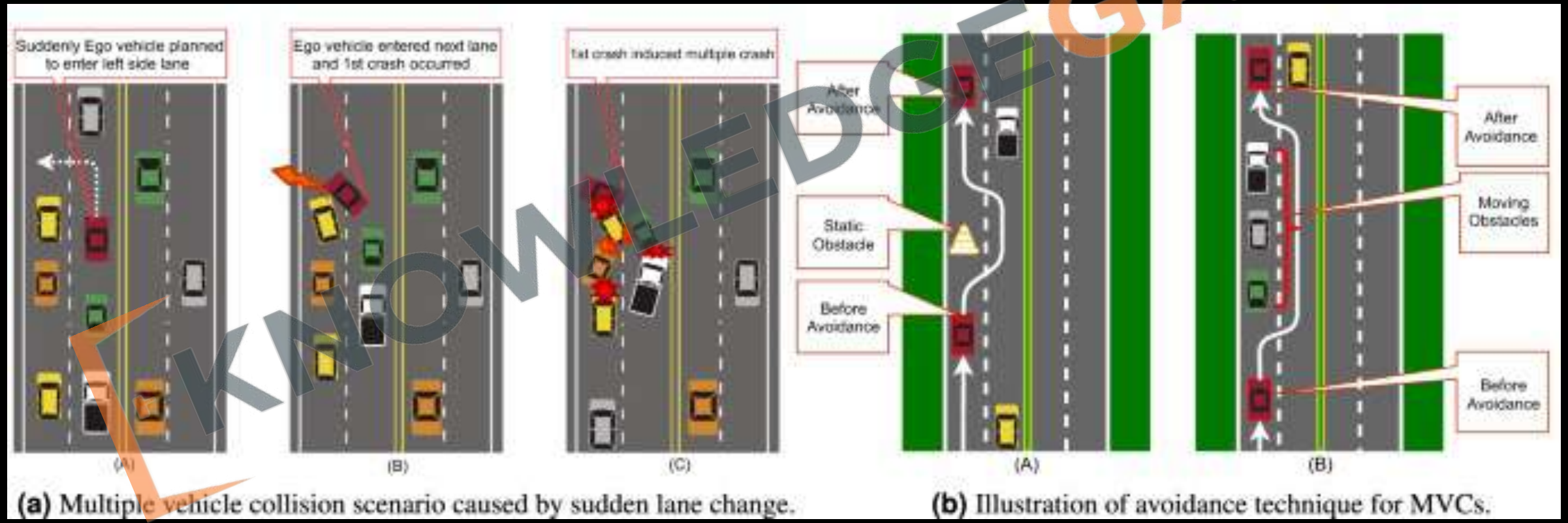
htt gate

- Deliberative/Reasoning Architecture:
  - **Example**: NASA's Mars rovers, which contain a world model and use planning and reasoning to decide on actions like navigation and experimentation.



KNOWLEDGEGATE

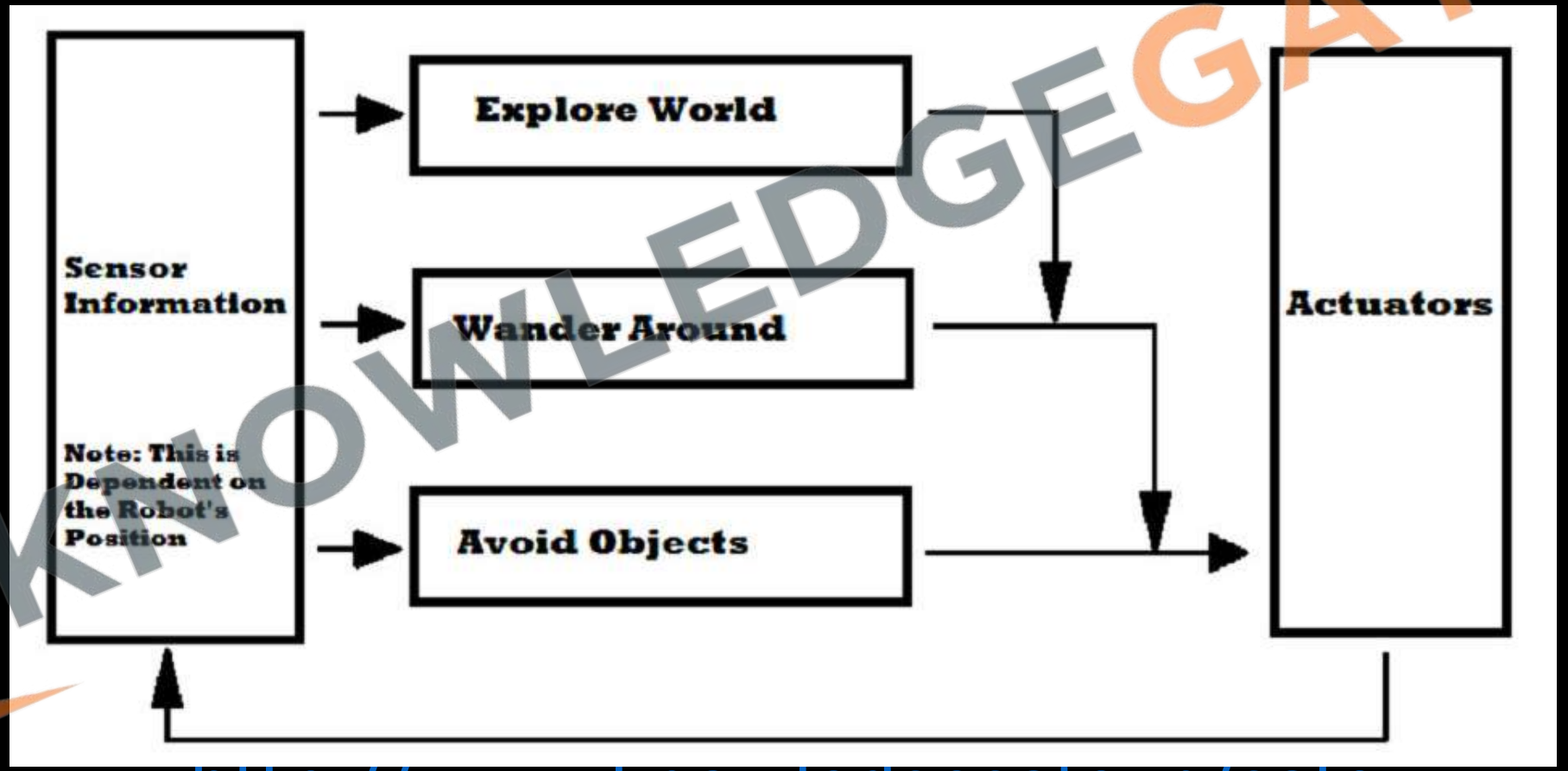
- Hybrid Architecture:
  - **Example**: Autonomous vehicles that combine reactive systems for collision avoidance with deliberative systems for route planning and navigation.



(a) Multiple vehicle collision scenario caused by sudden lane change. (b) Illustration of avoidance technique for MVCs.

KNOWLEDGEGATE

- Layered Architecture:
  - **Example**: The subsumption architecture developed by Rodney Brooks, where a robot's behavior is controlled by a set of hierarchical layers that build upon reflexive actions up to complex behaviors.

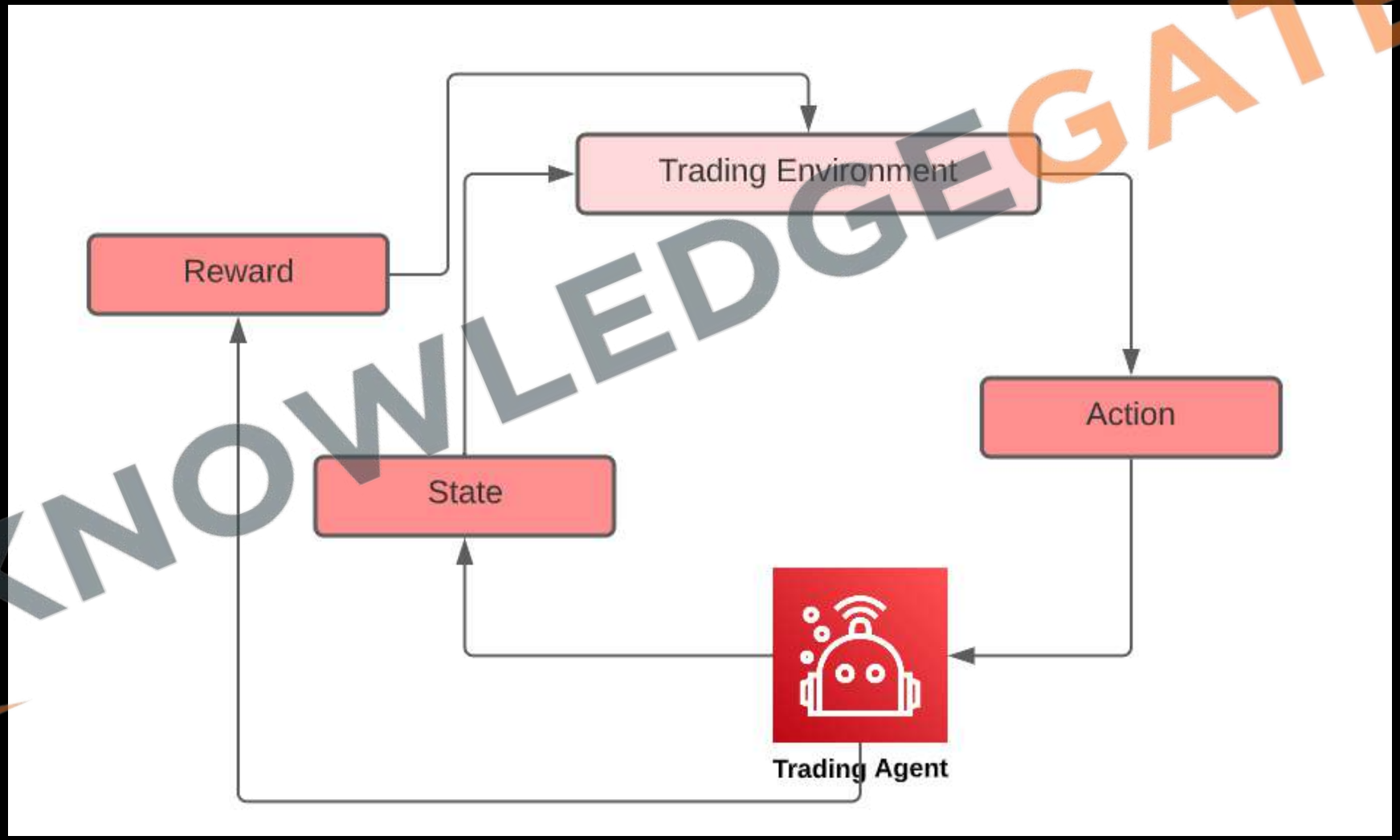


KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- Behavior-Based Architecture:
  - **Example**: Sony's AIBO, a robotic pet dog that exhibits behaviors based on a combination of internal states and sensory input.

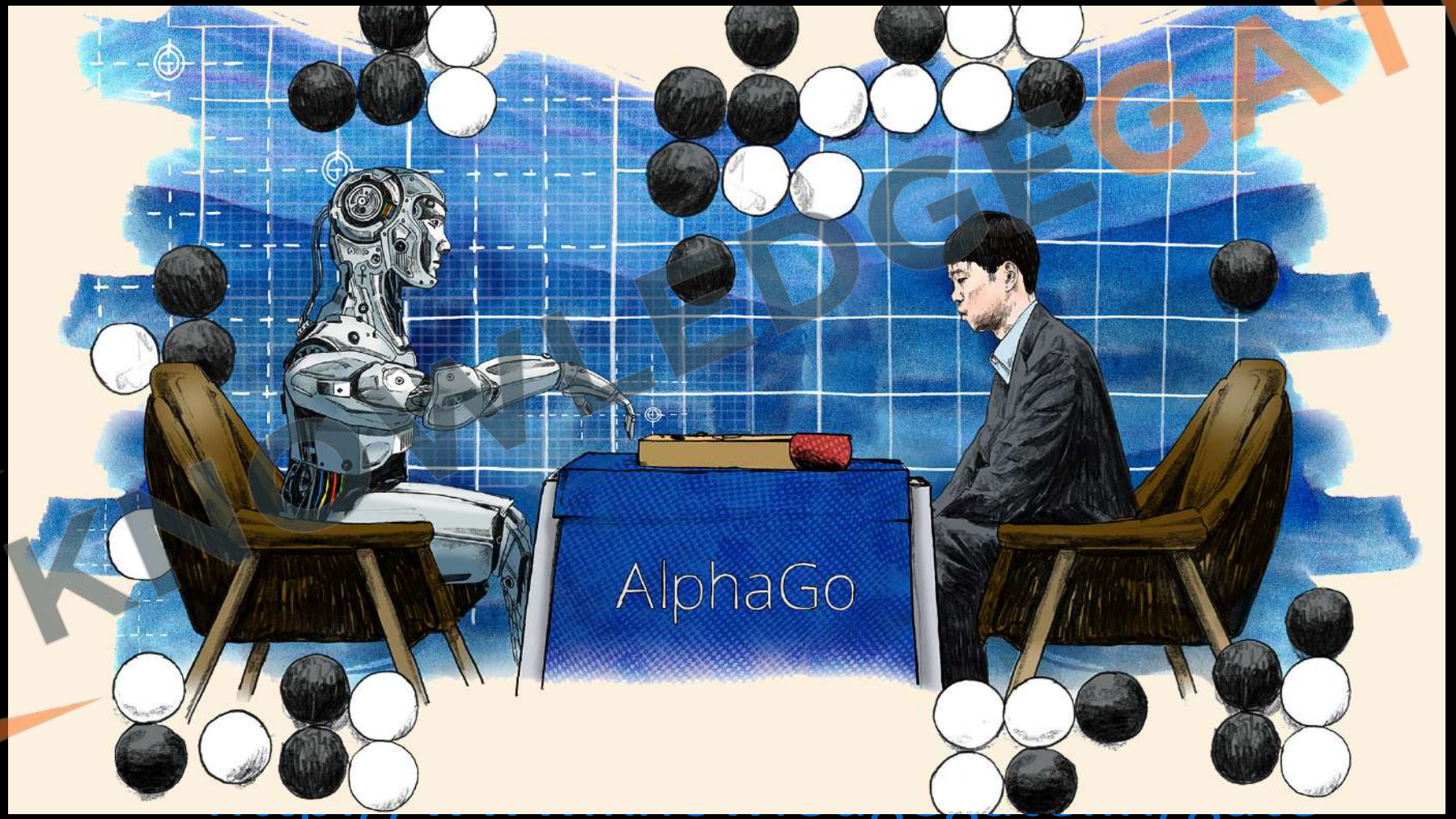
KNOWLEDGEGATE

- Agent-Oriented Programming:
  - **Example**: Multi-agent systems used for automated trading in stock markets where each agent represents a buyer or seller with specific strategies and goals.



KNOWLEDGEGATE

- Learning Architecture:
  - **Example**: DeepMind's AlphaGo, which uses deep neural networks and reinforcement learning to improve its play in the game of Go.



KNOWLEDGEGATE

Agent communication - Negotiation and Bargaining

- **Multi-Agent Interaction**:
  - **Theory**: In multi-agent systems, negotiation is a type of interaction among agents who aim to reconcile their different goals.
  - **Example**: Consider an e-commerce website where buyer and seller agents negotiate the price of goods. Each agent has the goal to buy low and sell high, respectively.

KNOWLEDGEGATE

- **Resource Allocation Challenge**:
  - **Theory**: Negotiation in multi-agent systems often involves the allocation of scarce resources which can't be simultaneously satisfied for all competing agents.
  - **Example**: In network bandwidth allocation, multiple streaming services negotiate for bandwidth to provide a smooth service to users during peak hours.

KNOWLEDGEGATE

- **Joint Decision Process**:
  - **Theory**: Negotiation and bargaining allow agents to reach a joint decision that meets each agent's objectives to some extent.
  - **Example**: In automated traffic management, different car agents negotiate the right of way at an intersection to minimize overall waiting time and avoid collisions.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- Main Features of Negotiation:
  - i. **Communication Language**: Agents use a predefined language for negotiations.
    - Example: Automated stock trading bots use Financial Information eXchange (FIX) language to place orders.
  - ii. **Negotiation Protocol**: Agents follow a set of rules during negotiation.
    - Example: Real estate agents use a standardized offer/counter-offer process when negotiating property sales.
  - iii. **Decision-Making**: Agents employ strategies to decide their stance in negotiations.
    - Example: A hotel booking agent uses past pricing data and current demand to decide on discounts for room rates.

[KNOWLEDGEGATE]

- Attributes of Effective Negotiation:
  - i. **Efficiency**: Agents should not expend unnecessary resources in negotiations.
    - Example: Energy management systems in smart grids where agents representing households and utility companies negotiate power distribution efficiently.
  - ii. **Stability**: Agents have no reason to deviate from an agreed-upon deal.
    - Example: In international trade agreements, countries adhere to tariffs and quotas they've negotiated as deviation can lead to sanctions or trade wars.
  - iii. **Simplicity**: The negotiation mechanism should be straightforward and require minimal resources.
    - Example: Contactless payment systems negotiate transaction details with a simple tap, minimizing time and processing requirements.
  - iv. **Distribution**: The system functions without central control.
    - Example: Cryptocurrency trades where the negotiation mechanism is embedded in the decentralized blockchain protocol.
  - v. **Symmetry**: The mechanism does not favor any particular agent.
    - Example: Peer-to-peer file-sharing networks where each user's data is treated equally without any user having privileged access.

KNOWLEDGEGATE

Arguments in Multi-Agent Systems

- **Argumentation as Communication**:
  - Just like people debate, AI agents discuss to convince others of their ideas.
  - **Example**: Consider two AI systems managing Delhi traffic. One argues for increasing green light time on a busy road to reduce jams, while the other argues against it to prevent congestion on cross streets.

KNOWLEDGE GATE

- **Factors in Argumentation**:
  - Arguments have a structure like in debates, and they use logic. Agents also try to persuade others, like a friend convincing you to try a new restaurant.
  - Example: An AI system in a shopping app may argue for cheaper product options based on your purchase history, trying to persuade you that you'll like them.

KNOWLEDGEGATE

- **Reasoning and Justification**:
  - Agents give reasons to back up what they say, like when you explain why you think it's going to rain by pointing to the dark clouds.
  - Example: A weather forecasting system in Mumbai uses data (reasons) to justify its prediction (conclusion) of a heavy monsoon.

KNOWLEDGE GATE

- Types of Arguments:
  - a. **Informational Arguments**: Using facts to come to a new belief.
  - Example: An app says, "If the air quality index is high (fact), then it's not a good day for outdoor sports (new belief)."

  - b. **Motivational Arguments**: Using beliefs and wants to form a new want.
  - Example: If you believe it's hot outside and you want to stay cool, an AI assistant might suggest, "You might want to drink cold nimbu pani."

  - c. **Practical Arguments**: Using facts to decide what to do.
  - Example: "If the local cricket match is today (fact) and you want to watch it (subgoal), then you should finish work early (goal)."

  - d. **Social Arguments**: Considering social rules to decide what to do.
  - Example: "I can't play loud music late at night because my neighbors will be disturbed (social commitment)."

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Interactions Between Arguments**:
  - Sometimes arguments go against each other, like when siblings argue over who gets to use the computer.
  - Example: In a village council meeting, one member's argument to use funds for a new well (support1 → conclusion1) might be opposed by another member who argues for new farm equipment (support2 → not conclusion1).

- By using these types of arguments, AI agents can discuss and make decisions just like we do in our daily lives, from choosing the quickest route to work to deciding the best way to spend a budget.

KNOWLEDGEGATE

Trust and reputation

- Trust and reputation, along with belief maintenance, play critical roles in the dynamics of multi-agent systems, ensuring reliable interactions and stability within agent societies. Here's how they are intertwined:

- **Truth Maintenance System (TMS)**:
  - A TMS is used to maintain the integrity and consistency of the agents' beliefs. It helps in keeping the knowledge base stable and logically coherent.
  - Example: An AI system managing traffic signals in New Delhi uses a TMS to update its beliefs about traffic flow. If new data contradicts previous beliefs (like a sudden road closure), the system adjusts its knowledge base to maintain consistency.

KNOWLEDGE GATE

- **Stable Knowledge Base**:
  - A stable knowledge base is one where all accepted data are backed by valid justifications, and any data without support are rejected.
  - Example: An e-commerce recommendation system believes a user likes a product based on their browsing history (valid justification) and disregards random items that the user never interacts with (lack of justification).

KNOWLEDGEGATE

- **Well-Founded and Logical Consistency**:
  - The system avoids circular dependencies among beliefs and ensures that no contradictions exist within the knowledge base.
  - Example: A weather forecasting AI for a city like Chennai won't simultaneously hold the belief that it is both the dry season and the monsoon season.

KNOWLEDGEGATE

- **Consistency and Completeness**:
  - The knowledge base should not contain contradictory beliefs, and it should be complete, concise, accurate, and efficient.
  - Example: A customer support AI contains complete information about product issues but doesn't hold conflicting information, such as a product being both available and discontinued.

KNOWLEDGEGATE

- **<u>Chapter-5</u> (**APPLICATIONS**):** AI applications - Language Models - Information Retrieval - Information Extraction - Natural Language Processing - Machine Translation - Speech Recognition - Robot - Hardware – Perception - Planning – Moving.

KNOWLEDGEGATE

AI applications

- **Gaming**: In India, AI is being utilized to develop sophisticated mobile and online games, with companies like MPL (Mobile Premier League) using AI for enhancing user engagement and improving gaming strategies.
- **Natural Language Processing (NLP)**: Startups like Haptik and Vernacular.ai are harnessing NLP to create chatbots and virtual assistants that understand and respond in multiple Indian languages, improving customer service experiences.
- **Expert Systems**: Agriculture is a major sector in India, and AI-based expert systems are being developed to help farmers with crop selection, pest control, and disease diagnosis, improving yields and sustainability.
- **Vision Systems**: India's traffic and public safety departments are using AI-driven vision systems for surveillance, crowd management, and traffic control, particularly in smart city projects.
- **Speech Recognition**: Organizations like the Indian government's e-governance services are incorporating speech recognition to help citizens interact with services in a variety of Indian languages and dialects.
- **Handwriting Recognition**: Banks and financial institutions in India are employing handwriting recognition technologies to process cheques and forms faster and more accurately.
- **Intelligent Robots**: Healthcare in India is seeing the introduction of AI robots that assist in surgeries and patient care, while industries are using robots for tasks like assembly and warehouse management.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Language Models

- **State Machines**:
  - Think of these like a game of stepping stones where each stone is a state. You can move from one to another (transition) based on certain rules (inputs). Examples include:
    - Finite-State Automata (FSA): These are simple state machines that follow strict rules to move from state to state.
    - Finite-State Transducers (FST): These are like FSAs but can produce outputs, not just state transitions.

KNOWLEDGEGATE

- **Rule Systems**:
  - These are sets of "if-then" rules. In linguistics:
    - Regular Grammars: Used for simple structures like validating email formats.
    - Context-Free Grammars: Good for describing the structure of sentences where context isn't needed.
    - Feature-Augmented Grammars: These add extra information to grammatical rules, like gender or number in languages.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Logic**:
  - Logic models are like precise language for computers. They help computers understand meanings and implications. Types include:
    - First-Order Logic: Deals with objects and their relationships.
    - Lambda Calculus: A way to express computations, used in functional programming languages and semantics.
    - Feature Structures: Like filling out a form, each feature (like 'color') has a value (like 'red').
    - Semantic Primitives: These are basic concepts used to build more complex meanings.

KNOWLEDGEGATE

- **Probabilistic Models**:
  - These models are like weather forecasts for language, assigning probabilities to different linguistic events. Examples are:
    - Weighted Automata: State machines where each transition has a weight, like a cost or likelihood.
    - Markov Models: Predicts a sequence of events, where each event only depends on the previous one.
    - Hidden Markov Models (HMMs): Like Markov models, but the states are hidden and only the outcomes are seen.

KNOWLEDGEGATE

- **Vector-Space Models**:
  - Imagine a giant space where each word is a point, and similar words are closer together. This is useful for tasks like finding synonyms or grouping similar words.

KNOWLEDGEGATE

Information Retrievel

- Certainly, information retrieval (IR) is a vital component across various domains in computer science, particularly in areas where large datasets are involved. Here's a simplified explanation of your points:

- **Huge Data/Document Collection**:
  - Think of IR systems as a vast library. Just like in a physical library where books are categorized and indexed for easy retrieval, IR systems must manage and sift through vast amounts of digital information to find what the user is looking for.

- **Format of Query with Standard Query Language**:
  - When you want to find information, you ask a question or type a search. In some systems, especially databases, you use a specific language with a particular syntax, like SQL (Structured Query Language), to tell the system exactly what you're looking for.

KNOWLEDGEGATE

- **The Generated Result Model**:
  - This is about how the system finds and picks the documents that answer your query. It's like having a smart assistant in the library who understands your question and knows exactly where the answers are in the countless shelves of books.

- **Displaying Results Model**:
  - Once your assistant finds the books, they can show them to you in different ways. Some systems just show you the titles or a brief summary; others give you more detailed previews. And then there are those that might use fancy visual aids, like maps or 3-D models, to present the information in an easy-to-understand manner.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Definitions and Scope of NLP:

- **Natural Language Processing (NLP)**: This field studies the challenges in processing and manipulating natural language, aiming to make computers understand human language.
- **Automatic Processing**: NLP enables the automatic processing of human languages, making it a subset of AI focused on human-computer communication in natural languages like English.

KNOWLEDGEGATE

Purpose and Need for NLP:

- **Analyzing Text**: NLP is used to analyze and represent natural texts at various linguistic levels to achieve human-like language processing for diverse tasks.
- **Bridging Communication Gaps**: It serves as a bridge between human communication and machine understanding, facilitating more intuitive interactions.

KNOWLEDGE GATE

Key Areas in Language Generation:

- **Content Determination**: Deciding what information should be conveyed in the response.
- **Text Planning**: Organizing the content into a coherent structure before generating the actual text.
- **Realization**: Transforming the structured content into natural language text.

KNOWLEDGEGATE

Capabilities of a Full NLU System:

- **Paraphrasing**: Rewording the input text while retaining the original meaning.
- **Translation**: Converting text into another language.
- **Question Answering**: Providing answers based on the content of the text.
- **Inference**: Making logical deductions from the text.

KNOWLEDGEGATE

Applications of Natural Language Processing

1. Machine Translation:

- **Purpose**: Helps translate texts like manuals, support content, or catalogs at reduced costs, enabling access to multilingual information.
- **Challenge**: The main difficulty lies not in translating words but in understanding and conveying the intended meanings of sentences.

KNOWLEDGEGATE

2. Automatic Summarization:

- **Need**: Useful in managing information overload by condensing large volumes of data into concise summaries.
- **Applications**: Provides quick overviews of long articles, such as news stories or blog posts, and is useful in extracting sentiment and key points from social media.

KNOWLEDGEGATE

- 3. Sentiment Analysis:
  - **Goal**: To detect and understand the sentiments expressed in text, even when these emotions are not explicitly stated.
  - **Use Cases**: Companies utilize this to gauge public opinion on products and services, helping them to tailor marketing strategies and improve customer experiences.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

4. Text Classification:

- **Functionality**: Involves categorizing documents into predefined groups to streamline information retrieval and organization.
- **Examples**: Common applications include email spam filtering and sorting content into thematic folders.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

5. Question Answering:

- **Overview**: Develops systems that can answer questions posed by humans in a clear and accurate manner.
- Technologies: This includes interfaces like chatbots and virtual assistants, which use NLP to provide timely and contextually relevant responses.

KNOWLEDGEGATE

fundamental steps involved in Natural Language Processing (NLP)

1. Morphological Analysis

- **Purpose**: To break down words into their smallest meaningful units, known as morphemes (e.g., "comes" breaks down into "come" + "s").
- **Process**: Words are analyzed to identify their root forms and affixes. This step also involves distinguishing tokens, like punctuation, which are not part of any words.

KNOWLEDGEGATE

2. Syntactic Analysis

- **Purpose**: To arrange words into a grammatical structure that makes sense according to the rules of the language.
- **Process**: This involves parsing sentences to identify grammatical structures, determining the role of each word (such as subject, verb, object), and how these words interrelate to form coherent sentences.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

3. Semantic Analysis

- **Purpose**: To understand the meanings of individual words and how these meanings combine in sentences to produce overall sentence meanings.
- **Process**: Semantic analysis takes the structures formed during syntactic analysis and assigns meanings to them, ensuring the entire sentence makes sense semantically.

KNOWLEDGEGATE

4. Pragmatic Analysis

- **Purpose**: To interpret sentences based on their usage in specific contexts, considering how meaning changes with different situations.
- **Process**: It goes beyond the literal meaning of sentences to consider factors like the speaker’s intent, the listener's perception, and the context in which the conversation occurs. This helps in determining what is meant from what is simply said.

KNOWLEDGEGATE

5. Discourse Integration

- **Purpose**: To consider the influence of preceding sentences on the interpretation of the current sentence, and how the current sentence can affect subsequent sentences.
- **Process**: This step looks at the text as a whole, rather than in isolated sentences, ensuring that the interpretation of one sentence is consistent with and influenced by others around it.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

AI significantly enhances NLP by allowing machines to understand and interact with human language in a natural and intuitive way. This involves several key capabilities and approaches:

- **Speech Recognition**: AI enables machines to process spoken words, turning speech into text, which is crucial for applications like voice-activated assistants.
- **Natural Communication**: AI programs communicate with humans using natural language, making interactions with machines more user-friendly and accessible.
- **Understanding Language**: AI must understand the structure of language, including vocabulary, syntax (arrangement of words), and semantics (meaning), to effectively process human language.

KNOWLEDGEGATE

Approaches to NLU (Natural Language Understanding)

- **Keyword and Pattern Matching**: Simple methods for identifying essential words or patterns to interpret language.
- **Syntactic and Semantic Analysis**: More complex analyses that consider both the structure and meaning of language to generate responses.
- **Real-World Situation Matching**: AI compares inputs to known real-world scenarios to enhance understanding and response accuracy.

KNOWLEDGEGATE

Machine Translation (MT)

- Refers to the automated process of translating text from one language (source language) to another (target language) using computer algorithms.
- **Definition**: Machine translation automates the translation of text from one language to another, converting sequences of symbols (words, phrases) from the source language to equivalent sequences in the target language.
- **Purpose**: The primary goal is to enable communication across language barriers without human translators, enhancing accessibility and efficiency in global interactions.

KNOWLEDGE GATE

Types of Machine Translation Systems

- **Bilingual MT System**:
  - These systems focus on translating between two specific languages. They are typically unidirectional, meaning they translate from one language to another but not vice versa.
- **Multilingual MT System**:
  - Capable of translating between multiple language pairs. These systems can be either unidirectional or bidirectional, facilitating translations across several languages.

KNOWLEDGEGATE

Major Approaches to Machine Translation

- **Direct MT Approach**:
  - **Description**: Direct translation from the source to the target language without intermediate steps.
  - **Example**: Translating "Buenos días" (Spanish) directly to "Good morning" (English) without analyzing or restructuring the sentence's grammatical components.

KNOWLEDGEGATE

- **Interlingua Approach**:
  - **Description**: Translation involves converting the source language into an artificial intermediate language (Interlingua) and then into the target language.
  - **Example**: For translating "Bonjour" (French) to "Guten Morgen" (German), the system first translates French into an Interlingua representation like "Greeting(morning)" and then from Interlingua to the German equivalent.

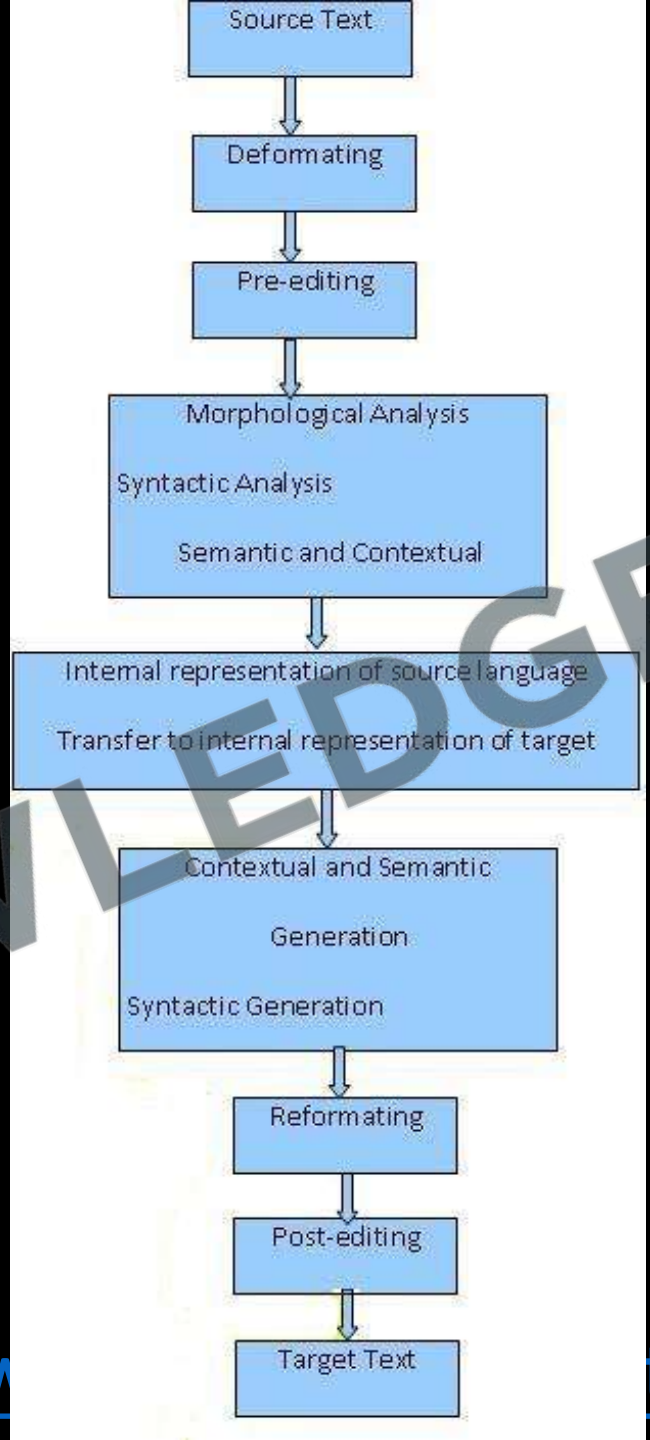
KNOWLEDGE GATE

- **Transfer Approach**:
  - **Description**: Involves three steps: analysis of the source text, conversion to a target-language representation, and generation of the target text.
  - **Example**: To translate "Estoy cansado" (Spanish, meaning "I am tired") to English, the approach would first analyze and understand the structure ("[subject] [verb] [adjective]"), map it to an equivalent English structure ("[subject] [verb] [adjective]"), and generate the target text ("I am tired").

KNOWLEDGEGATE

- **Empirical (Statistical or Neural) MT Approach**:
  - **Description**: Uses statistical models or neural networks trained on large datasets of bilingual texts to predict translations.
  - **Example**: Google Translate uses neural machine translation (NMT). When translating "Je t'aime" from French to English, it analyzes vast amounts of French-English text data to learn that the most probable translation of "Je t'aime" is "I love you" based on statistical likelihood and context learned from data.

KNOWLEDGEGATE



KNOWLEDGEGATE

http://ww te.in/gate

Overview of Speech Recognition

- Speech recognition technology allows computers to recognize spoken language and convert it into a digital format that machines can process. This capability can transform spoken commands or dictations into text or other forms of actionable data, depending on the application.

KNOWLEDGEGATE

Commonly Used Algorithms for Speech Recognition:

- **Hidden Markov Models (HMM)**:
  - HMMs are statistical models that assume the probability of each state depends only on the previous state. In speech recognition, they are used to model sequences of speech units (like phonemes or words) and predict the most likely sequence from the audio input.
- **Neural Networks**:
  - Deep learning approaches use neural networks to model complex patterns in data. These networks consist of layers of interconnected nodes that mimic the human brain's structure and function. They process training data, learn to recognize speech patterns, and improve accuracy over time through methods like gradient descent.

KNOWLEDGEGATE

Applications of Speech Recognition

- **Automotive**: Integration in vehicles for hands-free navigation, controlling media playback, and other voice-activated features enhance driver safety.
- **Technology**: Virtual assistants (like Siri, Google Assistant, Alexa, Cortana) embedded in smartphones and smart speakers help with tasks such as setting reminders, playing music, and controlling smart home devices through voice commands.
- **Healthcare**: Doctors and nurses use speech-to-text applications for efficient documentation of patient care, improving accuracy and saving time.
- **Sales and Customer Service**: Call centers use speech recognition to transcribe and analyze calls, helping identify trends and common issues. Cognitive bots engage in voice-based customer interactions, improving response times and service quality.
- **Security**: Voice authentication systems provide an additional layer of security, using unique vocal characteristics to verify a user's identity.

KNOWLEDGEGATE

Techniques for Speech Recognition

- **Speech Analysis**: This involves analyzing the physical and behavioral characteristics of speech, such as pitch and tone, which can help differentiate speakers and understand spoken commands.
- **Feature Extraction**: Key to improving performance, this technique involves condensing speech into a set of meaningful data points that represent the essential characteristics of the speech signal without the redundancy.
- **Modeling**: Techniques like speaker recognition and identification are used to create models that can recognize and differentiate between different speakers based on their unique vocal features.

KNOWLEDGEGATE

Robots, robotics, and how they intersect with artificial intelligence (AI)

- What are Robots?
  - Definition: Robots are artificial agents designed to perform tasks in the real world. They can manipulate objects through actions like picking, moving, modifying, or even destroying them to achieve specific goals, often in repetitive roles to free up human labor.
- What is Robotics?
  - Field Description: Robotics is an interdisciplinary branch that combines electrical engineering, mechanical engineering, and computer science to design, construct, and operate robots. It involves developing systems that can perform complex actions autonomously or semi-autonomously.

KNOWLEDGEGATE

Differences Between AI Programs and Robot Systems

- **Environment**: AI programs typically operate in simulated virtual environments, while robots operate in the real, physical world.
- **Inputs**: AI programs often receive inputs as symbols and rules, whereas robots deal with analog signals such as sounds and images.
- **Hardware Needs**: AI programs run on general-purpose computers, while robots require specialized hardware like sensors and actuators to interact with their environment.

KNOWLEDGEGATE

Integration of Robots and AI

- **Enhanced Capabilities**: AI equips robots with skills like computer vision, allowing them to navigate and interact dynamically with their environment. AI also enables robots to learn from experiences and make autonomous decisions, using various forms of machine learning.

KNOWLEDGEGATE

Integration of Robots and AI

Components of a Robot

- **Power Supply**: Includes batteries, solar power, hydraulic, or pneumatic sources.
- **Actuators**: Convert energy into mechanical motion.
- **Sensors**: Equip robots with the ability to gather real-time data about their surroundings, crucial for tasks requiring environmental interaction.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Robot Locomotion

- **Legged Locomotion**: Uses multiple legs, suitable for varied terrain but power-intensive and complex due to the need for coordination and balance.
- **Wheeled Locomotion**: Involves fewer motors, generally easier to implement, and more stable and power-efficient compared to legged options.
- **Slip/Skid Locomotion**: Similar to a tank, using tracks that allow stability and maneuverability on challenging surfaces.

KNOWLEDGEGATE

Mobile Robot Hardware Overview

- A mobile robot integrates hardware and computational components to operate autonomously or semi-autonomously in various environments. Key hardware subsystems of a mobile robot include:
- **Motion**: This subsystem allows the robot to change its position relative to its environment. It includes mechanisms like wheels, legs, or tracks that enable movement.
- **Sensing**: Sensing technologies enable a robot to gather information about its own state and its surroundings. This can include a range of sensors such as cameras, infrared sensors, ultrasonic sensors, and lidar.

KNOWLEDGEGATE

- **Reasoning**: This involves processing sensor data to make decisions. The reasoning subsystem often includes onboard computers that execute algorithms for navigation, obstacle avoidance, and task execution.

- **Communication**: Robots often need to communicate with human operators or other machines. This subsystem can include wireless communication technologies like Wi-Fi, Bluetooth, or even satellite communications in more advanced robots.

KNOWLEDGE GATE

Categories of Motion of Mobile Robots

- Mobile robots are designed to operate in various environments, each requiring specialized modes of locomotion:
- **Terrestrial**:
  - **Environment**: These robots navigate ground surfaces, leveraging the stability of solid support.
  - **Locomotion**: Terrestrial robots predominantly use wheels for simplicity and efficiency. Other forms include legged robots for walking or climbing, tracked robots for navigating rough terrains, and robots that slither, adapting to diverse ground conditions.

KNOWLEDGEGATE

- **Airborne**:
  - **Environment**: Airborne robots are crafted to fly, often emulating the designs of conventional aircraft or avian species.
  - **Locomotion**: This category includes robotic helicopters, fixed-wing aircraft, and inventive forms like automated parachutes and dirigibles. Challenges for these robots include maintaining position and navigating through varying air conditions.

KNOWLEDGEGATE

- **Aquatic**:
  - **Environment**: Aquatic robots function in water, suitable for both surface and underwater operations.
  - **Locomotion**: They mainly utilize propellers or jets for movement, essential for conducting tasks in locations that are otherwise inaccessible to humans, such as deep-sea exploration or maintaining underwater structures.

KNOWLEDGEGATE

- **Space**:
  - **Environment**: Operating in the extreme conditions of outer space, including microgravity and intense radiation.
  - **Locomotion**: Space robots may operate independently as free-fliers or move across larger vehicles like space stations. Their movement typically relies on thrusters suited for the vacuum of space.

KNOWLEDGEGATE

Types of AI in Robotics

- **Weak AI**: Often used for simulated human thought processes in controlled environments, such as virtual assistants (e.g., Siri, Alexa).
- **Strong AI**: Employs sophisticated decision-making capabilities without human oversight, used in autonomous vehicles and advanced robotics.
- **Specialized AI**: Tailored for specific tasks, commonly found in industrial robots performing repetitive actions like painting or assembly.

KNOWLEDGEGATE

Applications of Speech Recognition in Robotics

- Used extensively in fields like automotive for voice-activated systems, healthcare for documentation, and customer service for interactive bots and security systems, enhancing both efficiency and safety.

KNOWLEDGEGATE

Module

Free Semester One Shot Exam Course

Topic

Machine Learning

Subtopic

ML One Shot

Lesson

Machine_Learning_Notes

SEMESTER EXAM SERIES

MACHINE LEARNING

IN 7 HOURS + FREE NOTES

KNOWLEDGE GATE

- **(UNIT-1 : INTRODUCTION)** Learning, Types of Learning, Well defined learning problems, Designing a Learning System, History of ML, Introduction of Machine Learning Approaches - (Artificial Neural Network, Clustering, Reinforcement Learning, Decision Tree Learning, Bayesian networks, Support Vector Machine, Genetic Algorithm), Issues in Machine Learning and Data Science Vs Machine Learning.
- **(UNIT-2: REGRESSION & BAYESIAN LEARNING)** REGRESSION: Linear Regression and Logistic Regression. BAYESIAN LEARNING - Bayes theorem, Concept learning, Bayes Optimal Classifier, Naïve Bayes classifier, Bayesian belief networks, EM algorithm. SUPPORT VECTOR MACHINE: Introduction, Types of support vector kernel - (Linear kernel, polynomial kernel,and Gaussiankernel), Hyperplane - (Decision surface), Properties of SVM, and Issues in SVM.
- **(UNIT-3: DECISION TREE LEARNING)** DECISION TREE LEARNING - Decision tree learning algorithm, Inductive bias, Inductive inference with decision trees, Entropy and information theory, Information gain, ID-3 Algorithm, Issues in Decision tree learning. INSTANCE-BASED LEARNING - k-Nearest Neighbour Learning, Locally Weighted Regression, Radial basis function networks, Case-based learning.
- **(UNIT-4: ARTIFICIAL NEURAL NETWORKS)** ARTIFICIAL NEURAL NETWORKS - Perceptron's, Multilayer perceptron, Gradient descent & the Delta rule, Multilayer networks, Derivation of Backpropagation Algorithm, Generalization, Unsupervised Learning - SOM Algorithm and its variant; DEEP LEARNING - Introduction, concept of convolutional neural network, Types of layers - (Convolutional Layers, Activation function, pooling, fully connected), Concept of Convolution (1D and 2D) layers, Training of network, Case study of CNN for eg on Diabetic Retinopathy, Building a smart speaker, Self-deriving car etc.
- **(UNIT-5: REINFORCEMENT LEARNING)** REINFORCEMENT LEARNING-Introduction to Reinforcement Learning, Learning Task,Example of Reinforcement Learning in Practice, Learning Models for Reinforcement - (Markov Decision process, Q Learning - Q Learning function, @ Learning Algorithm), Application of Reinforcement Learning,Introduction to Deep Q Learning. GENETIC ALGORITHMS: Introduction, Components, GA cycle of reproduction, Crossover, Mutation, Genetic Programming, Models of Evolution and Learning, Applications.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

(UNIT-1 : INTRODUCTION)

- Learning, Types of Learning
- Well defined learning problems, Designing a Learning System.
- History of ML
- Introduction of Machine Learning Approaches –
  - Artificial Neural Network
  - Clustering
  - Reinforcement Learning
  - Decision Tree Learning
  - Bayesian networks
  - Support Vector Machine
  - Genetic Algorithm),
- Issues in Machine Learning and
- Data Science Vs Machine Learning.

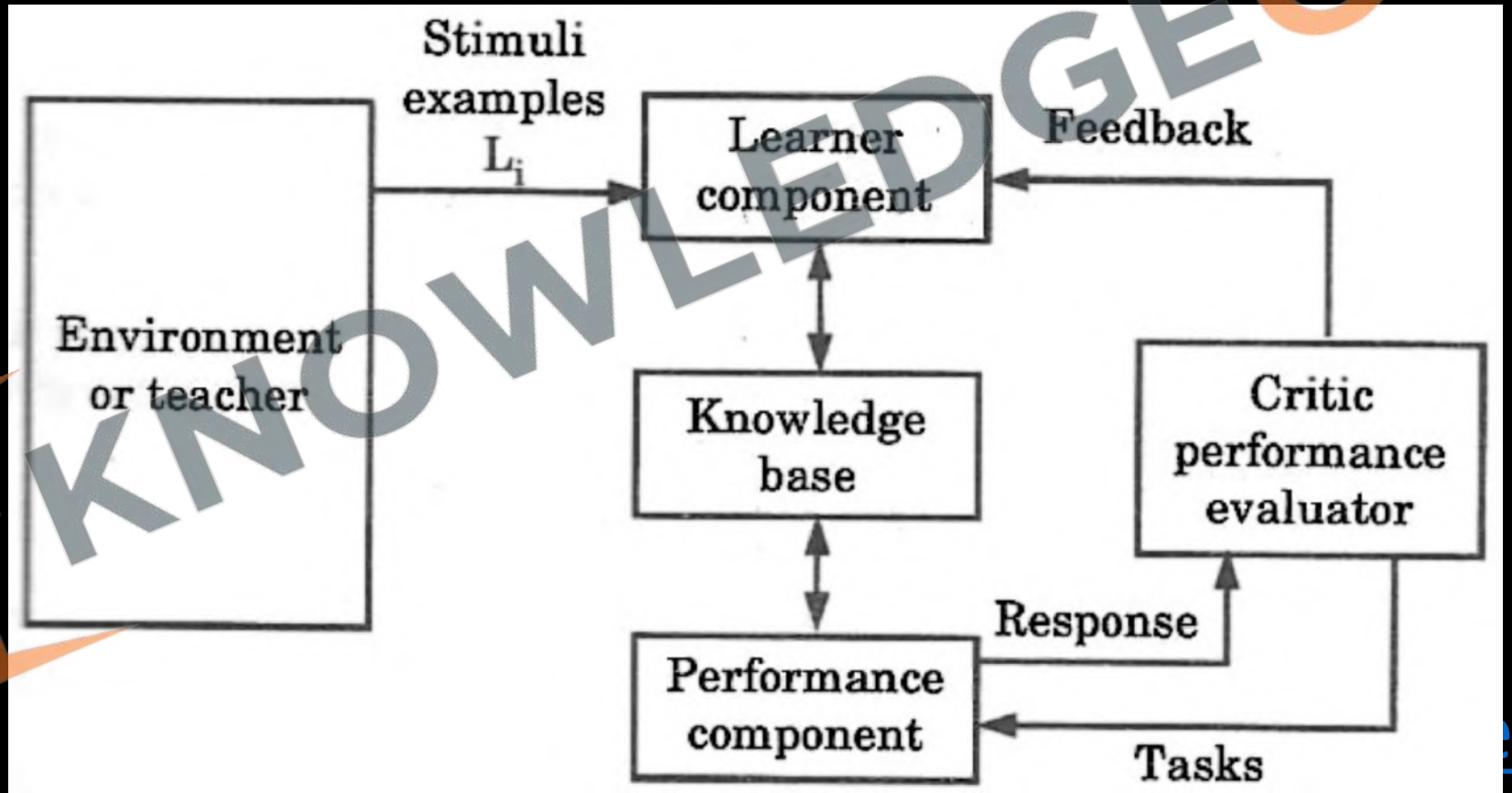
KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Definition of Learning

- Learning involves changes in behaviour due to experiences, focusing on adapting rather than relying on instinct or temporary states.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Components of a Learning System**:
  - **Performance Element**: Determines actions based on existing strategies.
  - **Learning Element**: Improves the performance element by analyzing past outcomes. Influences include:
    - **Components of Performance**: Understanding existing capabilities.
    - **Feedback Mechanism**: Using feedback to enhance performance.
    - **Knowledge Representation**: How information is organized and accessed.



KNOWLEDGEGATE

- **Acquisition of New Knowledge**:
  - Essential to learning; involves understanding new information, similar to how students learn new mathematical techniques.

- **Problem Solving**:
  - Integrates new knowledge and deduces solutions when not all data is available, akin to a doctor diagnosing illnesses with limited information.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Performance measures for learning

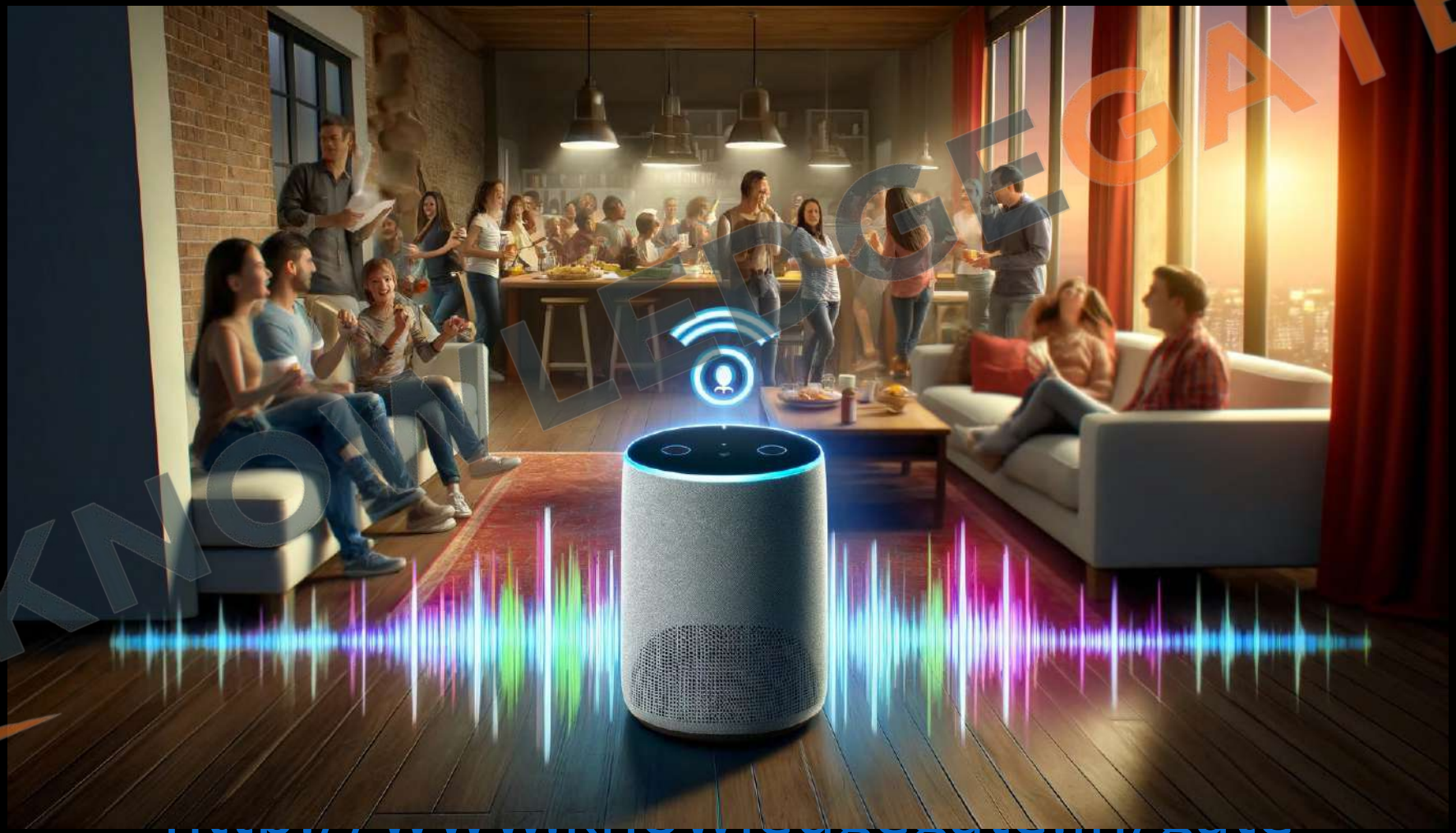
- **Generality**
  - Generality refers to a machine learning model's ability to perform well across various datasets and environments, not just the one it was trained on. For instance, a facial recognition system that can accurately identify faces in diverse lighting conditions and angles demonstrates good generality.

KNOWLEDGE GATE

- **Efficiency**
  - Efficiency in machine learning measures how quickly a model can learn from data. A spam detection algorithm that quickly adapts to new types of spam emails with minimal training data exhibits high efficiency.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Robustness**
  - Robustness is the ability of a model to handle errors, noise, and unexpected data without failing. A voice recognition system that can understand commands in a noisy room shows robustness.



KNOWLEDGE GATE

- **Efficacy**
  - Efficacy is the overall effectiveness of a machine learning model in performing its intended tasks. An autonomous driving system that safely navigates city traffic and avoids accidents under various conditions demonstrates high efficacy.

- **Ease of Implementation**
  - This measures how straightforward it is to develop and deploy a machine learning model. A recommendation system that can be integrated into an existing e-commerce platform using standard algorithms and software libraries highlights ease of implementation.

Supervised Learning

- Supervised learning involves training a machine learning model using labeled data, which means the data is already associated with the correct answer.
- **Example**: Consider teaching a child to identify fruits. You show them pictures of various fruits, like apples and bananas, while telling them, "This is an apple," and "This is a banana." Over time, the child learns to identify fruits correctly based on the examples given.

KNOWLEDGEGATE

- Key Steps in Supervised Learning:
  - **Input and Output Pairing**: Each input (e.g., a fruit picture) is paired with its correct label (e.g., "apple").
  - **Training**: The model learns by comparing its prediction with the actual label and adjusting itself to improve accuracy.
  - **Error Correction**: If the model predicts incorrectly (e.g., calls an apple a banana), it adjusts its internal parameters to reduce the error.
  - **Outcome**: The model eventually learns to map inputs (fruit images) to the correct outputs (fruit names).

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Unsupervised learning

- Unsupervised learning involves training a model without any labels, which means the model tries to identify patterns and data groupings on its own.
- **Example**: Imagine placing a mix of different coins on a table and asking a child to sort them. Without explaining any criteria, the child might start grouping the coins by size, color, or denomination on their own.

KNOWLEDGE GATE

- Key Steps in Unsupervised Learning:
  - **Input Without Labels**: The model receives data without any explicit instructions on what to do with it.
  - **Pattern Recognition**: The model analyzes the data and tries to find any natural groupings or patterns (e.g., clustering coins based on size or color).
  - **Self-Organization**: The model organizes data into different categories based on the patterns it perceives.
  - **Outcome**: The model creates its own system of categorization without external guidance.

KNOWLEDGEGATE

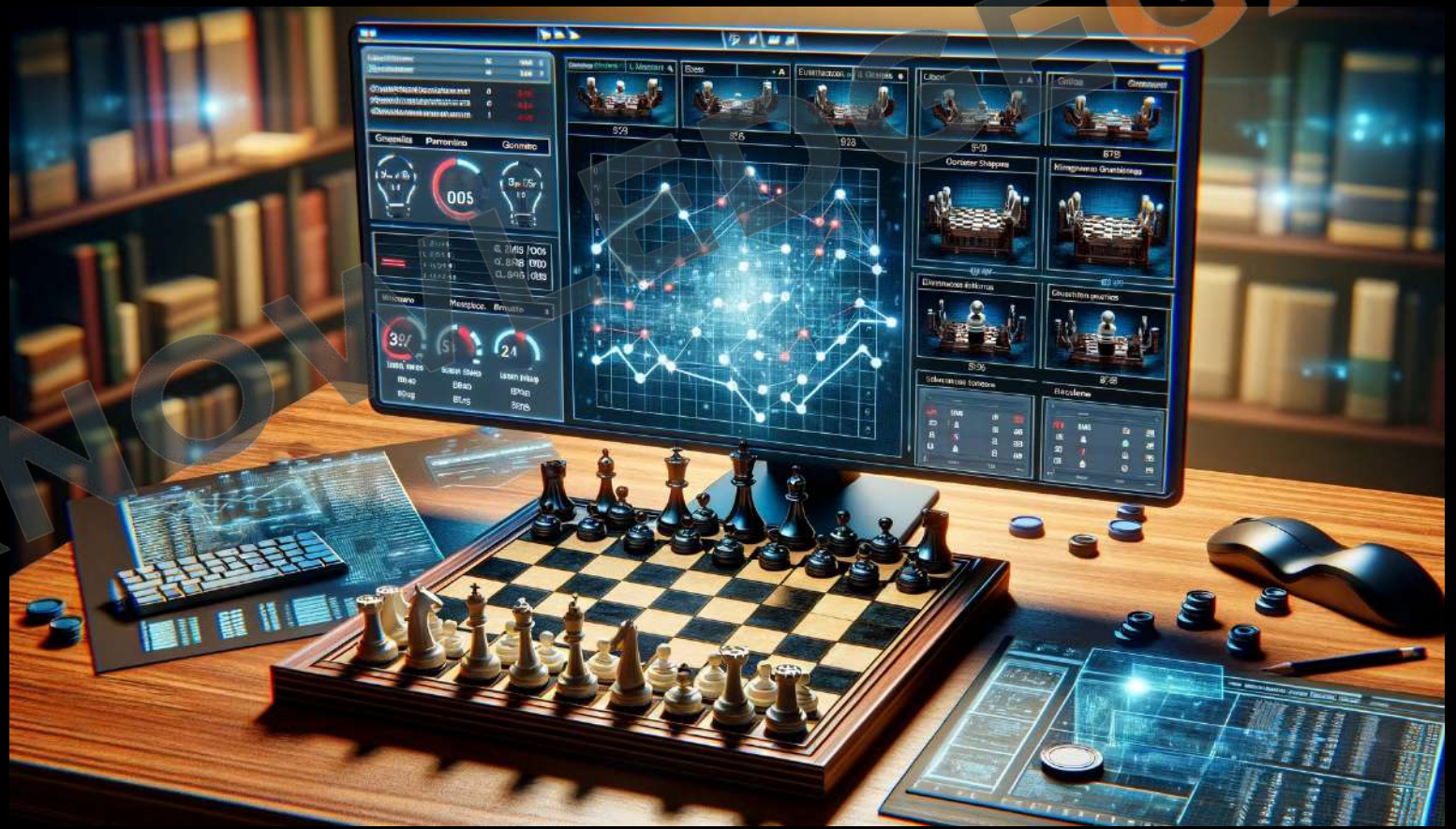
http://www.knowledgegate.in/gate

Well-defined learning problems

- A well-defined learning problem allows a computer program to improve at a specific task through experience. This is characterized by three key elements:
  - **Task (T)**: The specific activity or challenge the program is expected to perform.
  - **Performance Measure (P)**: The criteria used to gauge the program's effectiveness at the task.
  - **Experience (E)**: The data or interactions from which the program learns.

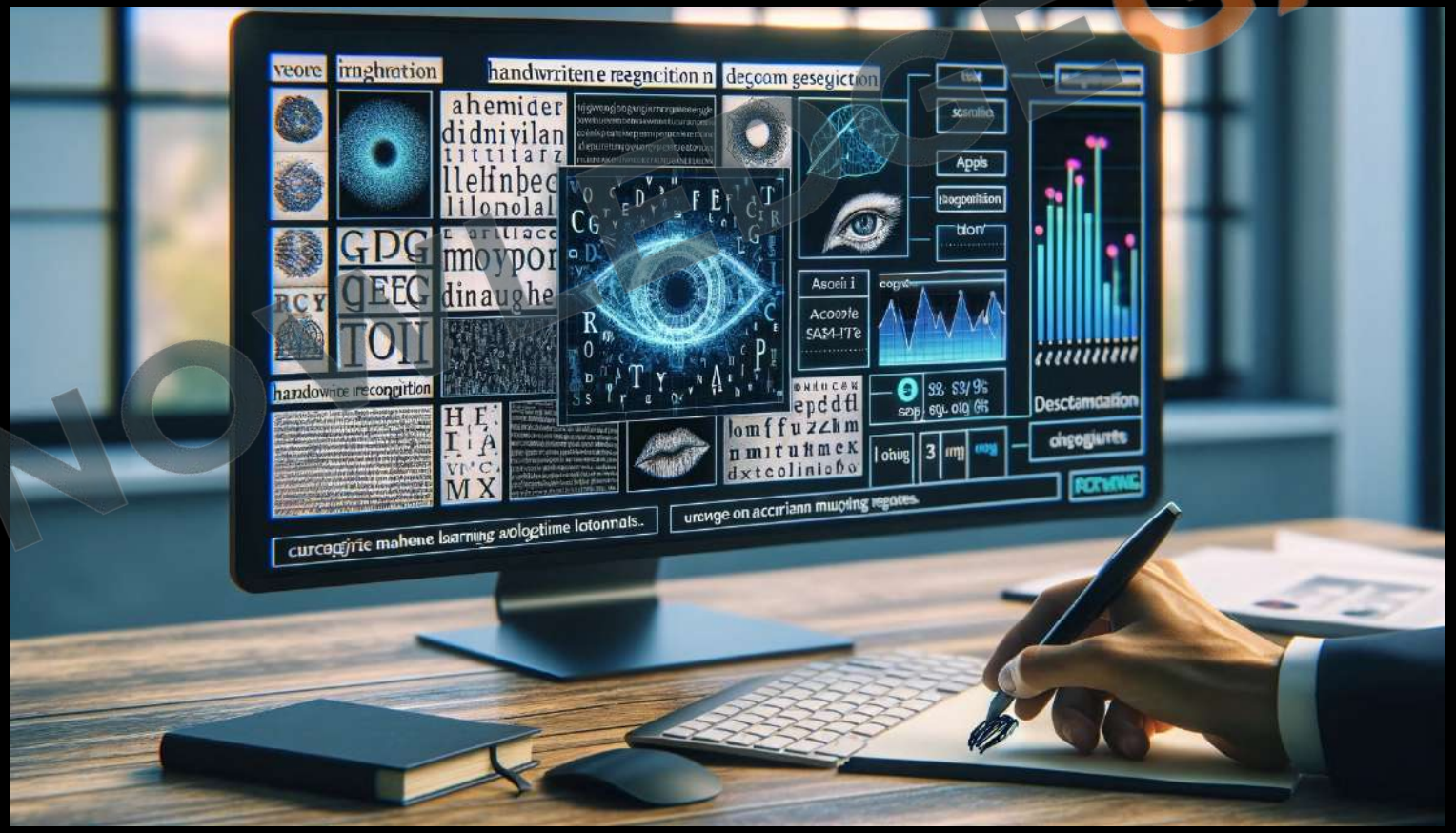
KNOWLEDGEGATE

- **Checkers Game**:
  - **Task (T)**: Playing the game of checkers.
  - **Performance Measure (P)**: The percentage of games won against various opponents.
  - **Experience (E)**: Engaging in numerous practice games, possibly including self-play.



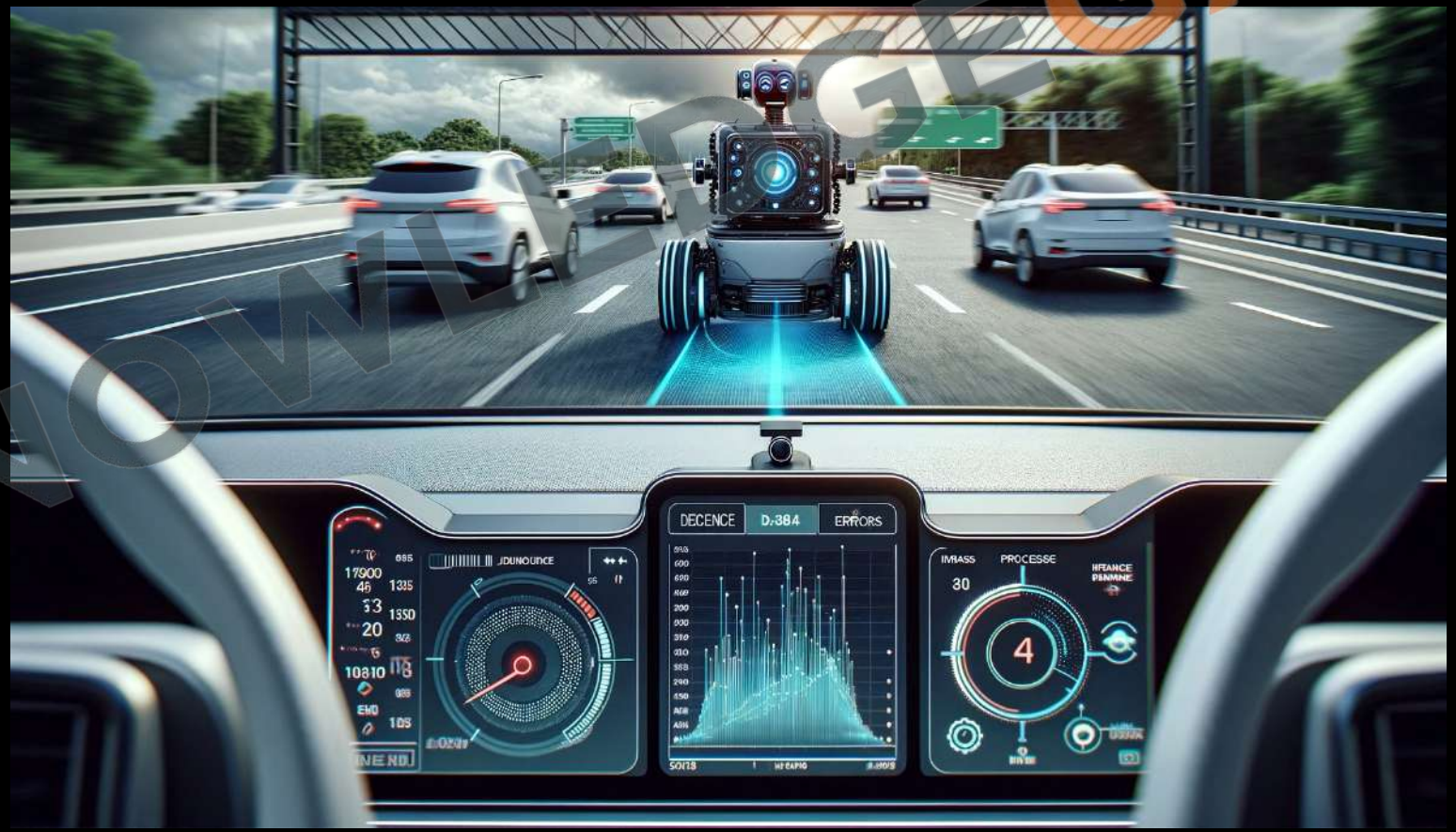
KNOWLEDGE GATE

- **Handwriting Recognition**:
  - **Task (T)**: Identifying and categorizing handwritten words in images.
  - **Performance Measure (P)**: The accuracy rate, measured as the percentage of words correctly recognized.
  - **Experience (E)**: Analysis of a large dataset of labeled handwritten word images.



KNOWLEDGE GATE

- **Autonomous Driving Robot**:
  - **Task (T)**: Navigating public four-lane highways using vision-based sensors.
  - **Performance Measure (P)**: The average distance the robot travels without making a mistake, as determined by a human supervisor.
  - **Experience (E)**: Processing sequences of images and corresponding steering commands previously collected from human drivers.

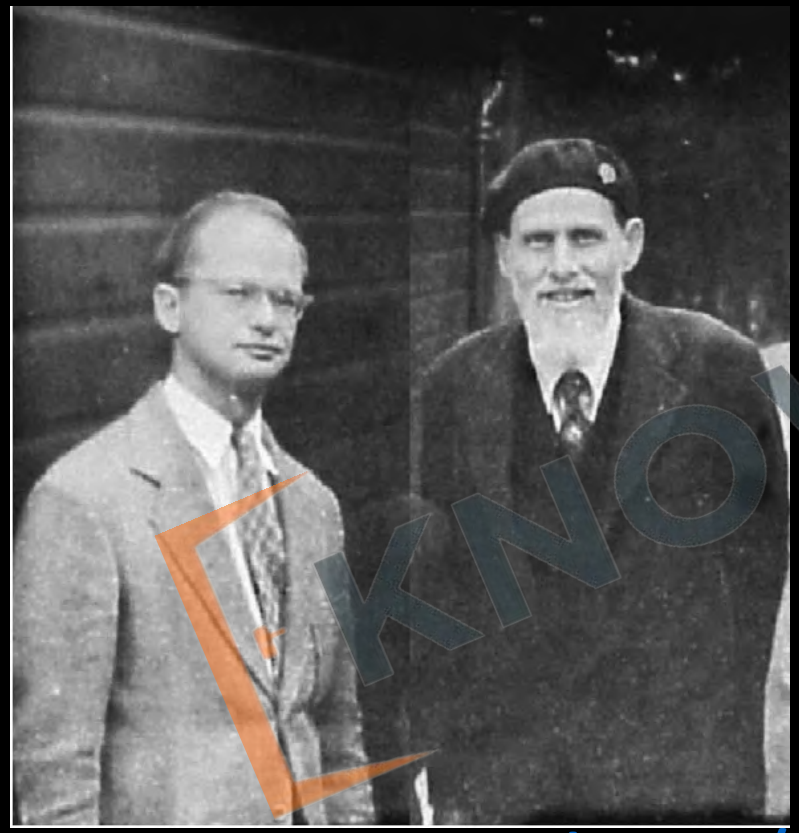


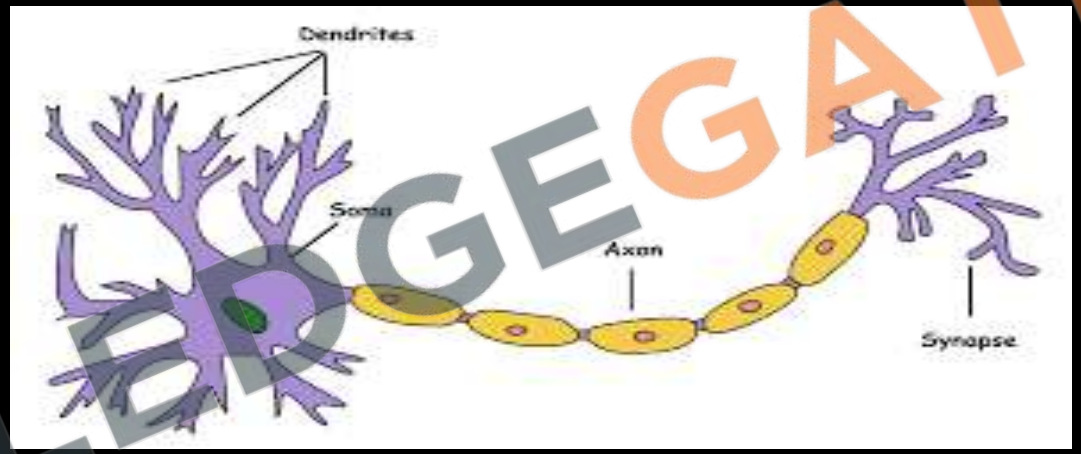
KNOWLEDGE GATE

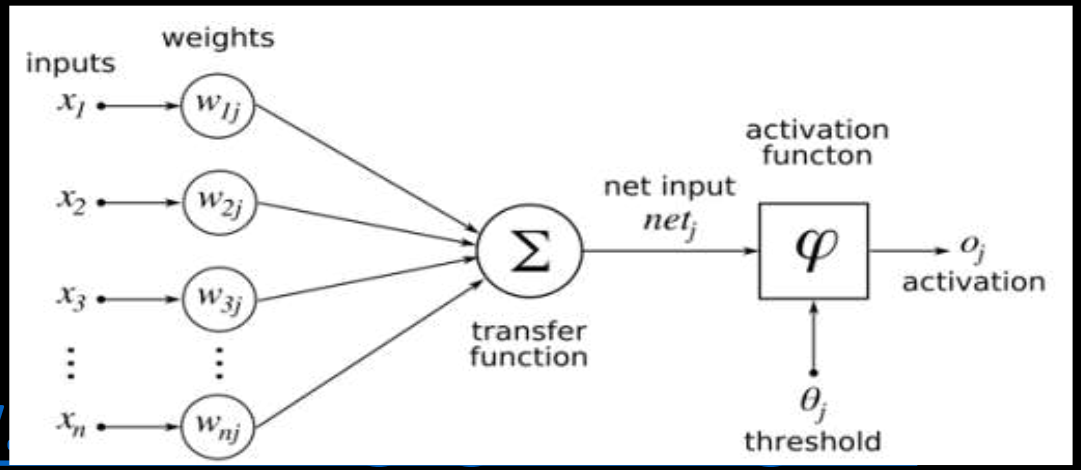
Overview of the history of Machine Learning

Early Developments:

- **1943**: Neurophysiologist Warren McCulloch and mathematician Walter Pitts introduced the concept of a neural network by modeling neurons with electrical circuits.







KNOWLEDGEGATE

http://www

Overview of the history of Machine Learning

Early Developments:

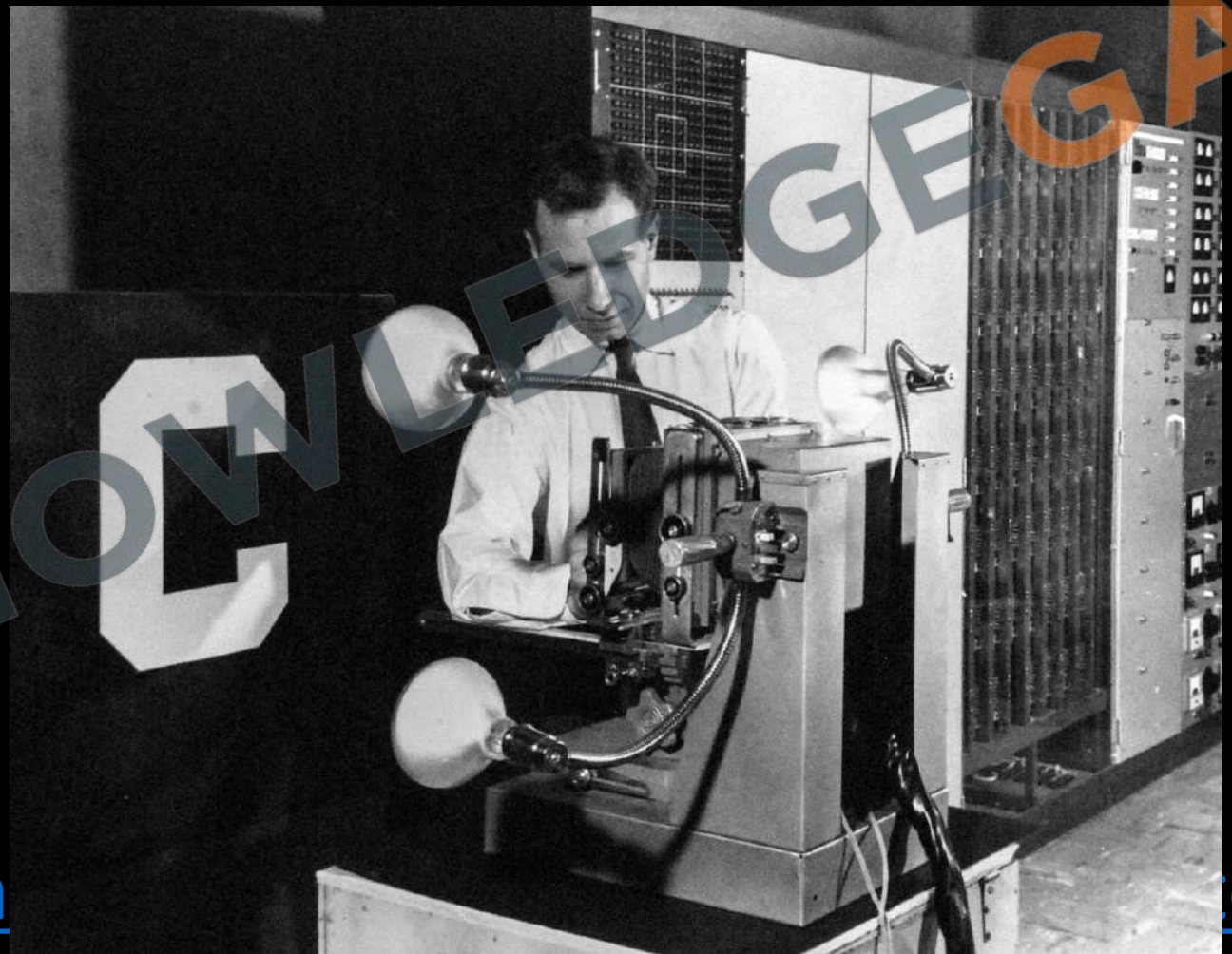
- **1952**: Arthur Samuel developed the first computer program capable of learning from its activities.

KNOWLEDGE GATE

Overview of the history of Machine Learning

Early Developments:

- **1958**: Frank Rosenblatt created the Perceptron, the first artificial neural network, which was designed for pattern and shape recognition.

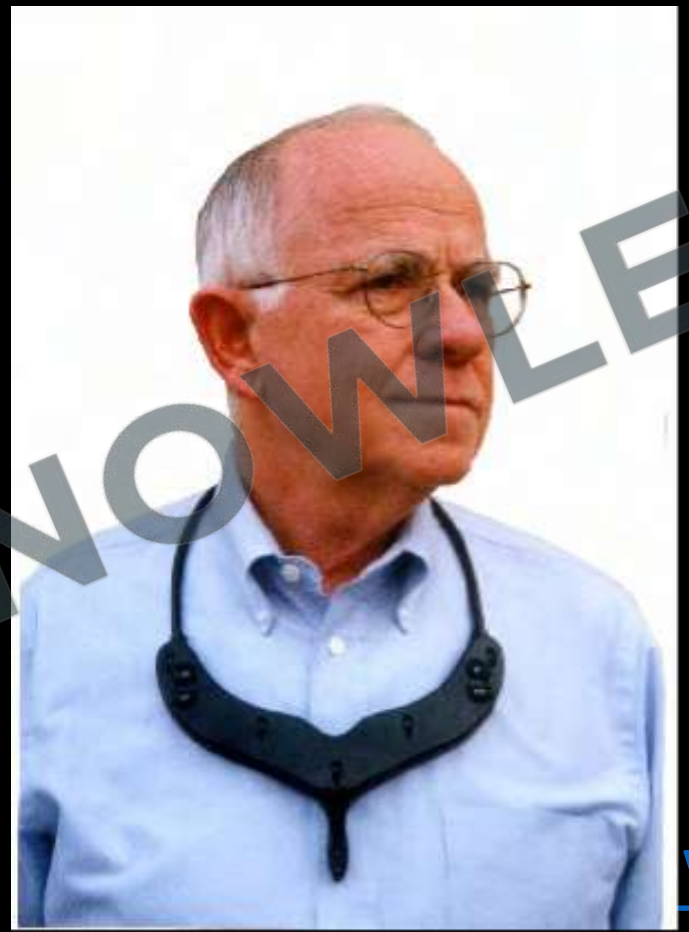


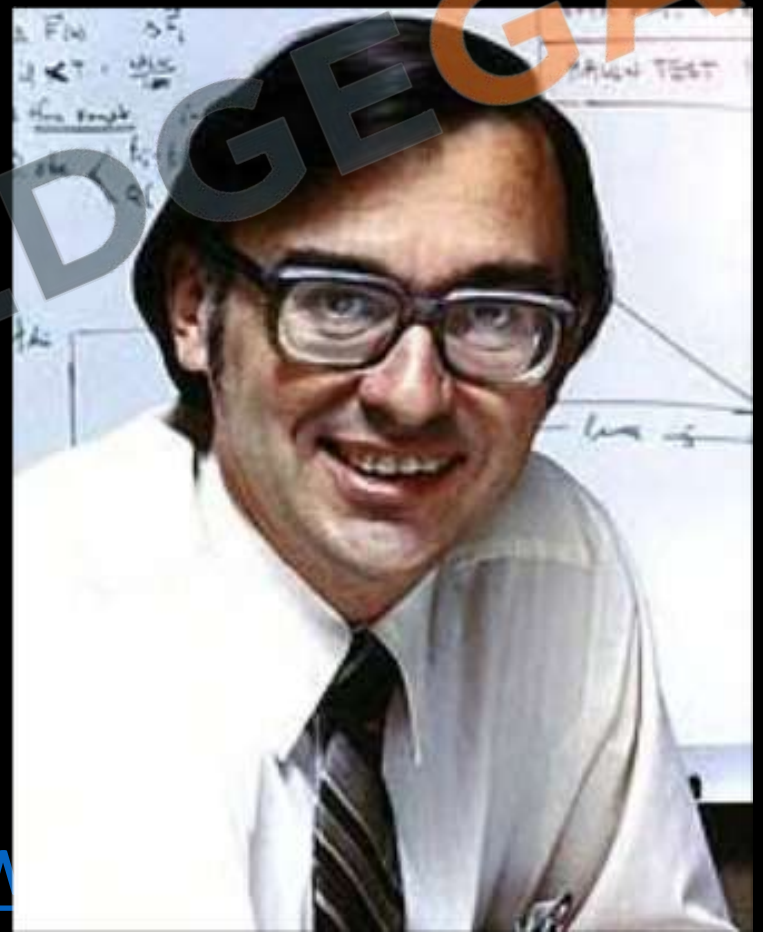
KNOWLEDGEGATE

Overview of the history of Machine Learning

Early Developments:

- **1959**: Bernard Widrow and Marcian Hoff developed two neural network models: ADELINE, which could detect binary patterns, and MADELINE, which was used to reduce echo on phone lines.

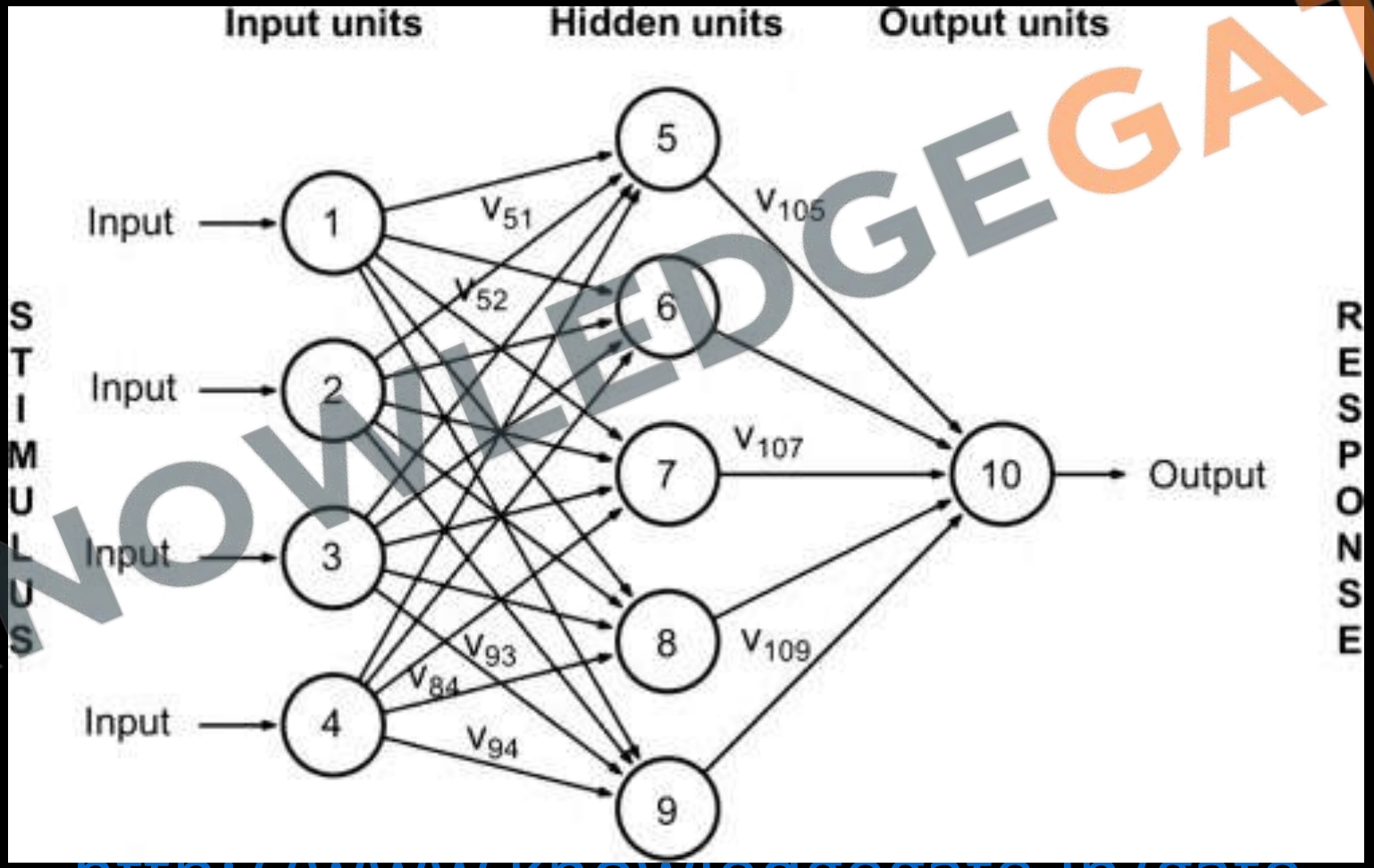




KNOWLEDGEGATE

Advancements in the 1980s and 1990s:

- **1982**: John Hopfield proposed a network with bidirectional lines that mimicked actual neuronal structures.

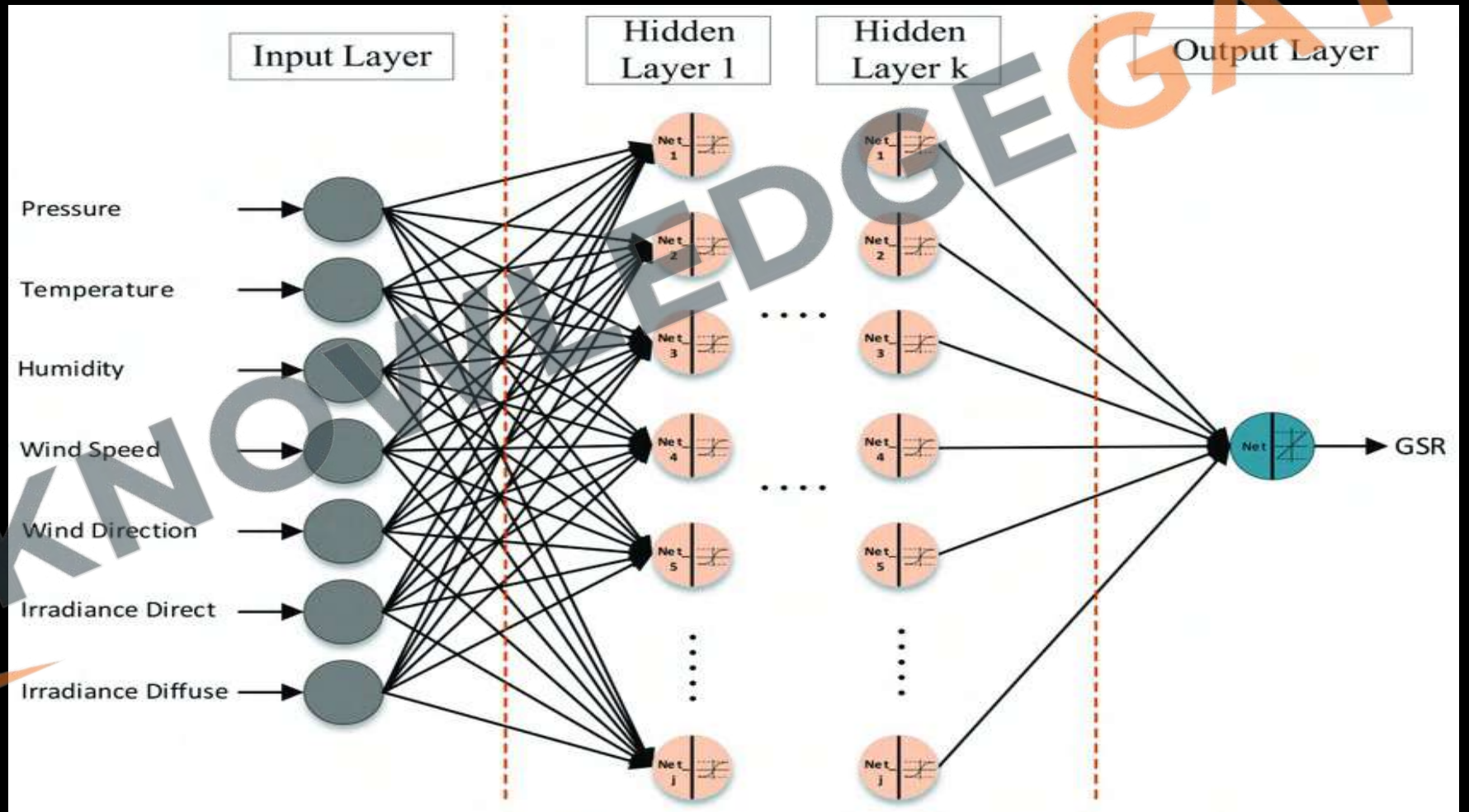


KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Advancements in the 1980s and 1990s:

- **1986**: The backpropagation algorithm was popularized, allowing the use of multiple layers in neural networks, enhancing their learning capabilities.



KNOWLEDGEGATE

Advancements in the 1980s and 1990s:

- **1997**: IBM's Deep Blue, a chess-playing computer, famously beat the reigning world chess champion.

KNOWLEDGEGATE

Advancements in the 1980s and 1990s:

- **1998**: AT&T Bell Laboratories achieved significant progress in digit recognition, notably enhancing the ability to recognize handwritten postcodes for the US Postal Service.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

21st Century Innovations:

- The 21st century has seen a significant surge in machine learning, driven by both industry and academia, to boost computational capabilities and innovation.

KNOWLEDGEGATE

- Notable projects include:
  - **GoogleBrain (2012)**: A deep learning project.
  - **AlexNet (2012)**: A deep convolutional neural network.
  - **DeepFace (2014)** and DeepMind (2014): Projects that advanced facial recognition and AI decision-making.
  - **OpenAI (2015), ResNet (2015), and U-net (2015)**: Each contributed to advancements in AI capabilities, from gameplay to medical imaging.

KNOWLEDGEGATE

Machine learning

- Machine learning is a subset of artificial intelligence (AI) that enables computers to learn from and make decisions based on data, without being explicitly programmed.
- **Definition**: Machine learning involves developing algorithms that allow computers to process and learn from data automatically.
- **Purpose**: The aim is to enable computers to learn from their experiences and improve their performance over time without human intervention.
- **Functionality**: Machine learning algorithms analyze vast amounts of data, enabling them to perform tasks more efficiently and accurately. This could be anything from predicting consumer behavior to detecting fraudulent transactions.
- **Integration**: Combining machine learning with AI and cognitive technologies enhances its ability to process and interpret large volumes of complex data.

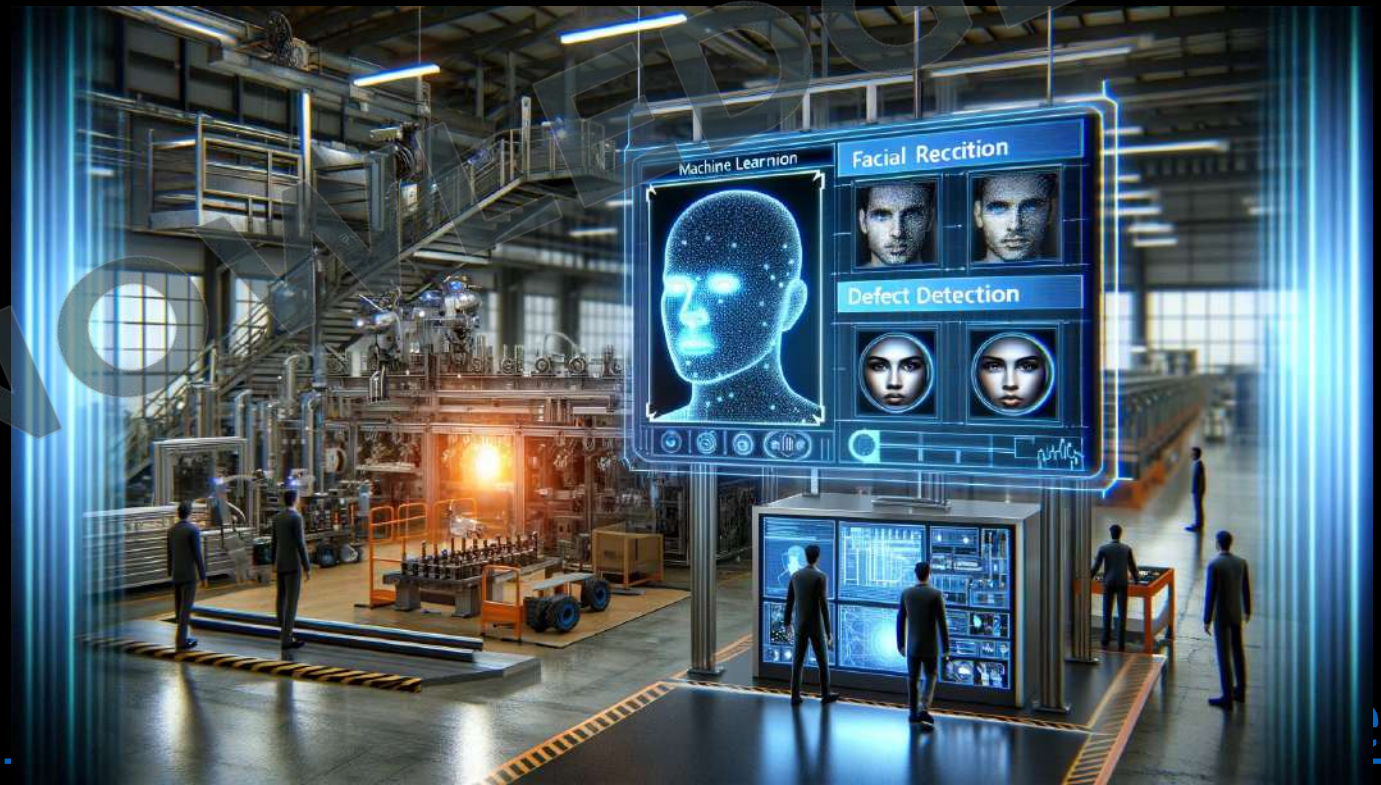
KNOWLEDGE GATE

Example: Consider a streaming service like Netflix. Machine learning is used to analyze your viewing habits and the habits of others with similar tastes. Based on this data, the system recommends movies and shows that you might like. Here, the algorithm learns from the accumulated data to make increasingly accurate predictions over time, thereby enhancing user experience without manual intervention. This demonstrates machine learning's capability to adapt and improve autonomously, making it a powerful tool in many tech-driven applications.

KNOWLEDGEGATE

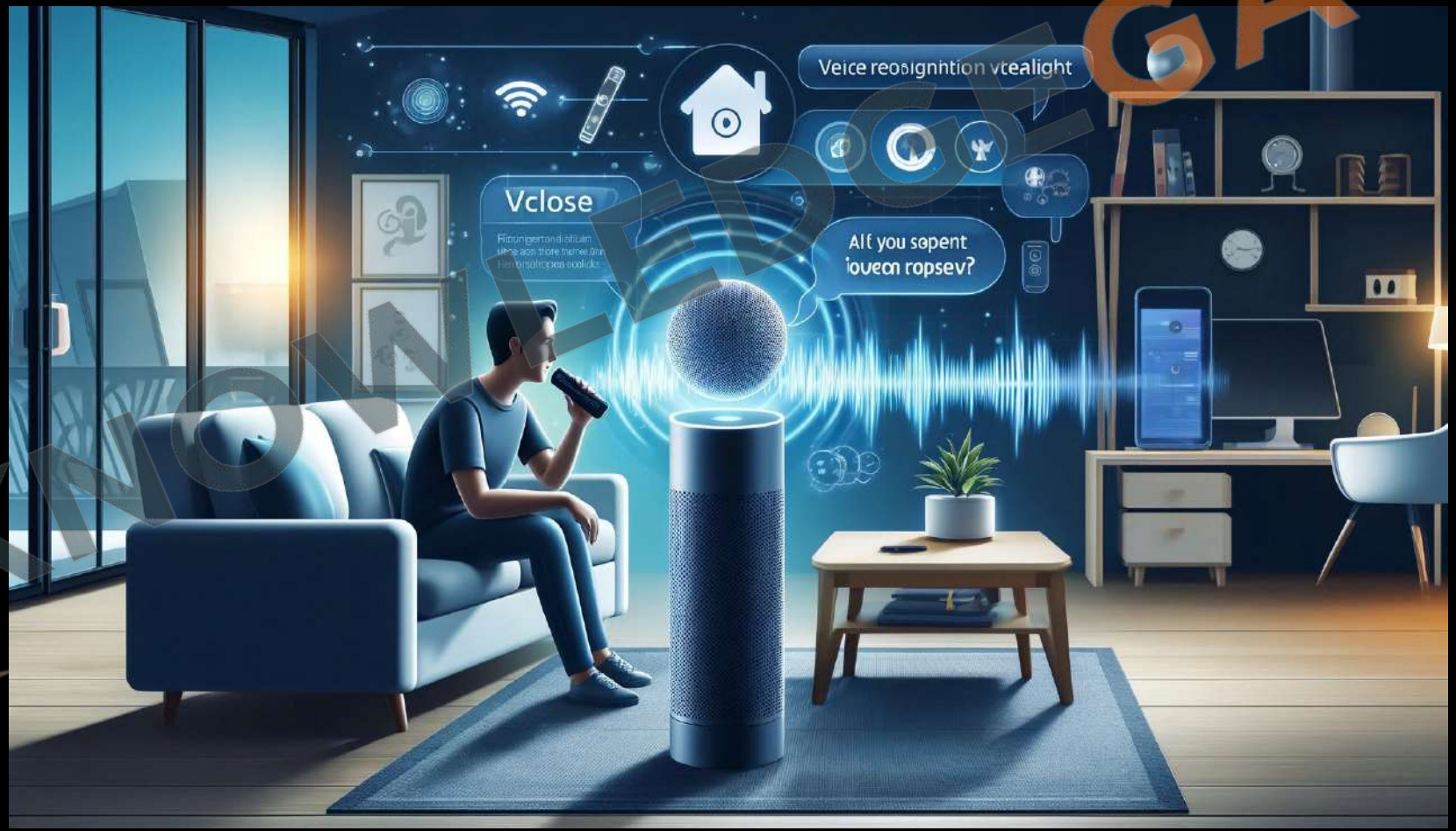
Machine learning has a wide range of applications across different fields, Here are some key applications along with examples:

- **Image Recognition**:
  - **Application**: Image recognition involves identifying objects, features, or patterns within digital images or videos.
  - **Example**: Used in facial recognition systems for security purposes or to detect defective products on assembly lines in manufacturing.



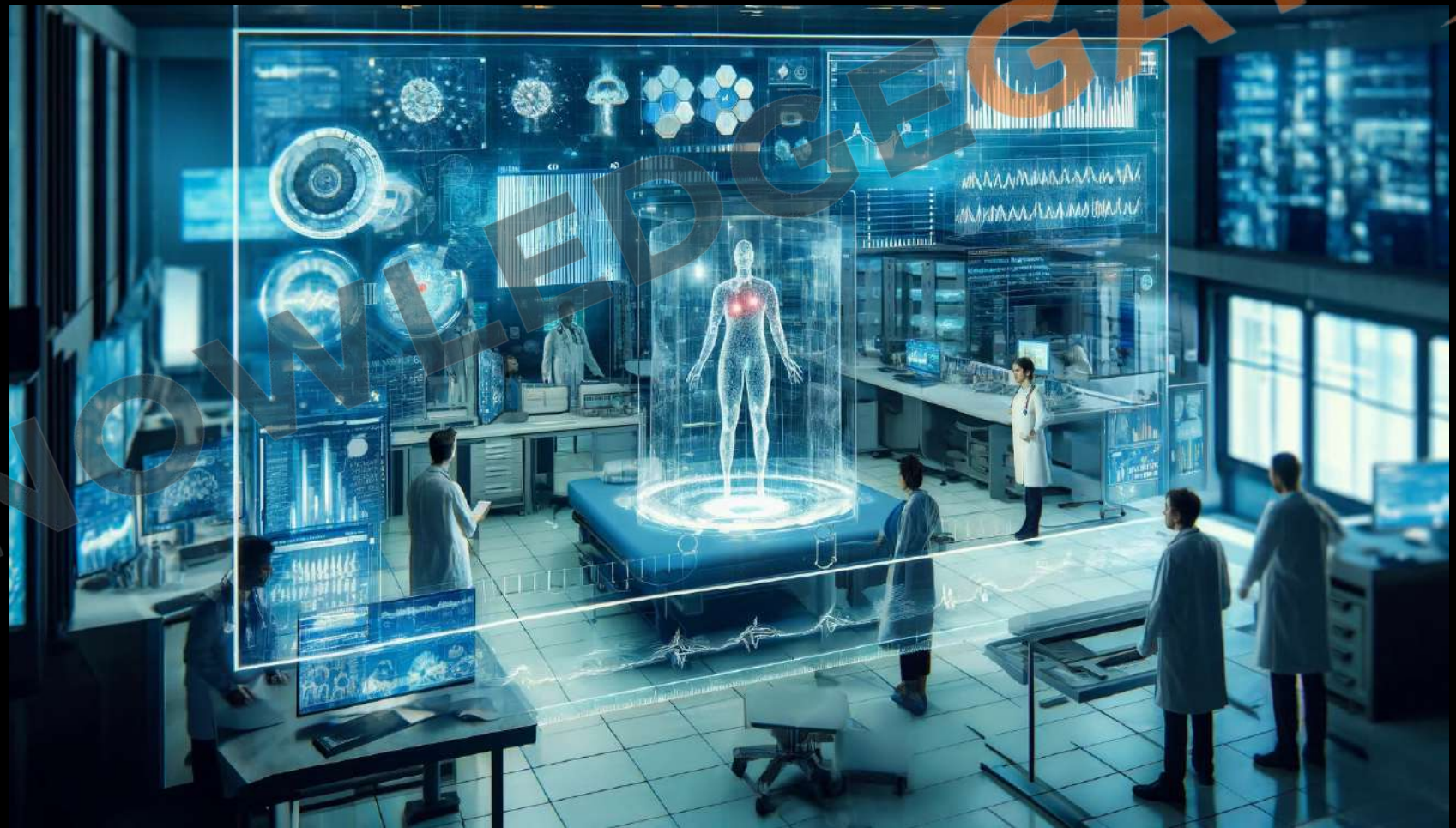
KNOWLEDGE GATE

- **Speech Recognition**:
  - **Application**: Speech recognition technology converts spoken words into text, facilitating user interaction with devices and applications.
  - **Example**: Virtual assistants like Siri and Alexa use speech recognition to understand user commands and provide appropriate responses.



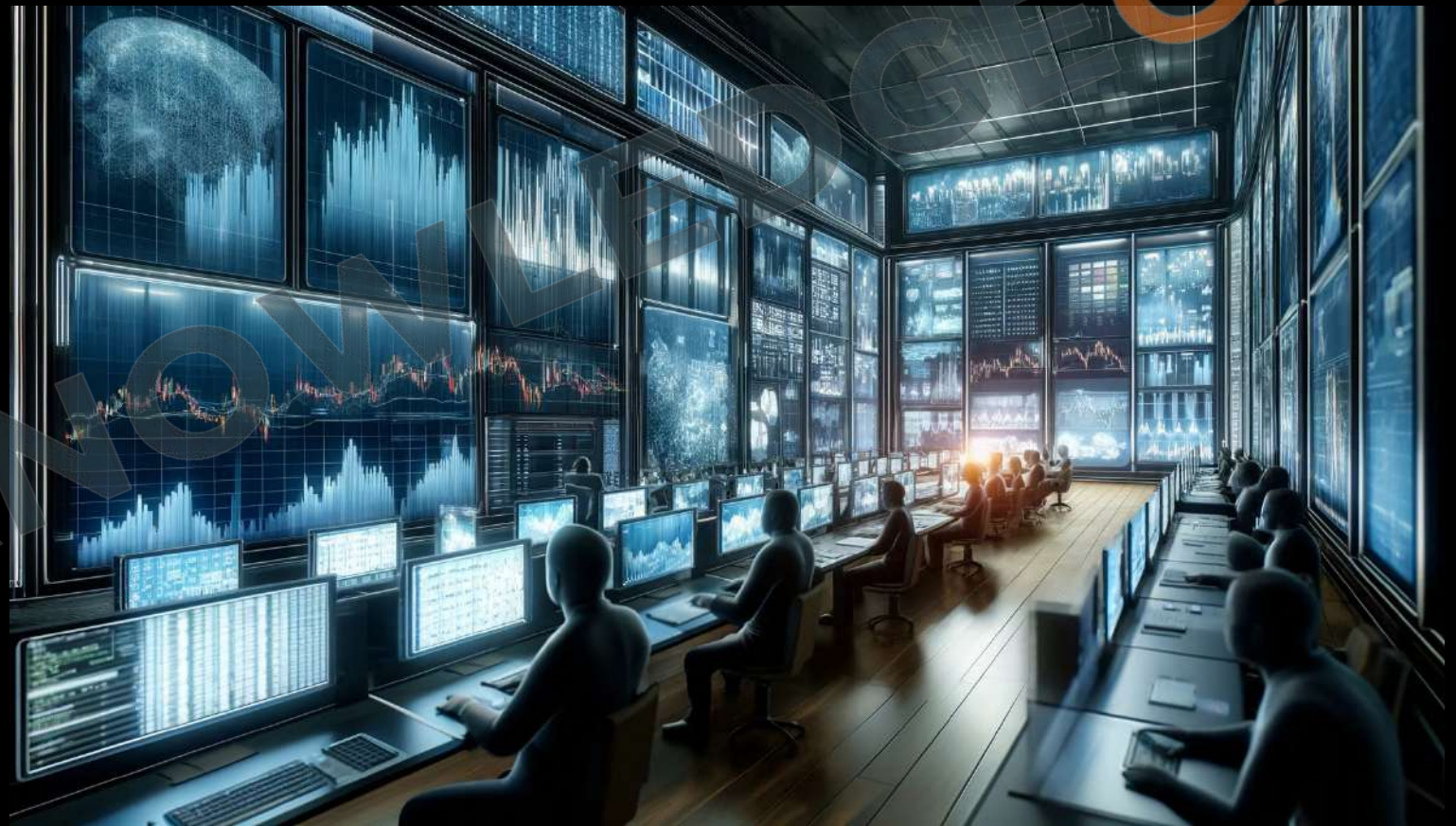
KNOWLEDGE GATE

- **Medical Diagnosis**:
  - **Application**: Machine learning assists in diagnosing diseases by analyzing clinical parameters and their combinations.
  - **Example**: Predicting diseases such as diabetes or cancer by examining patient data and previous case histories to identify patterns that precede diagnoses.



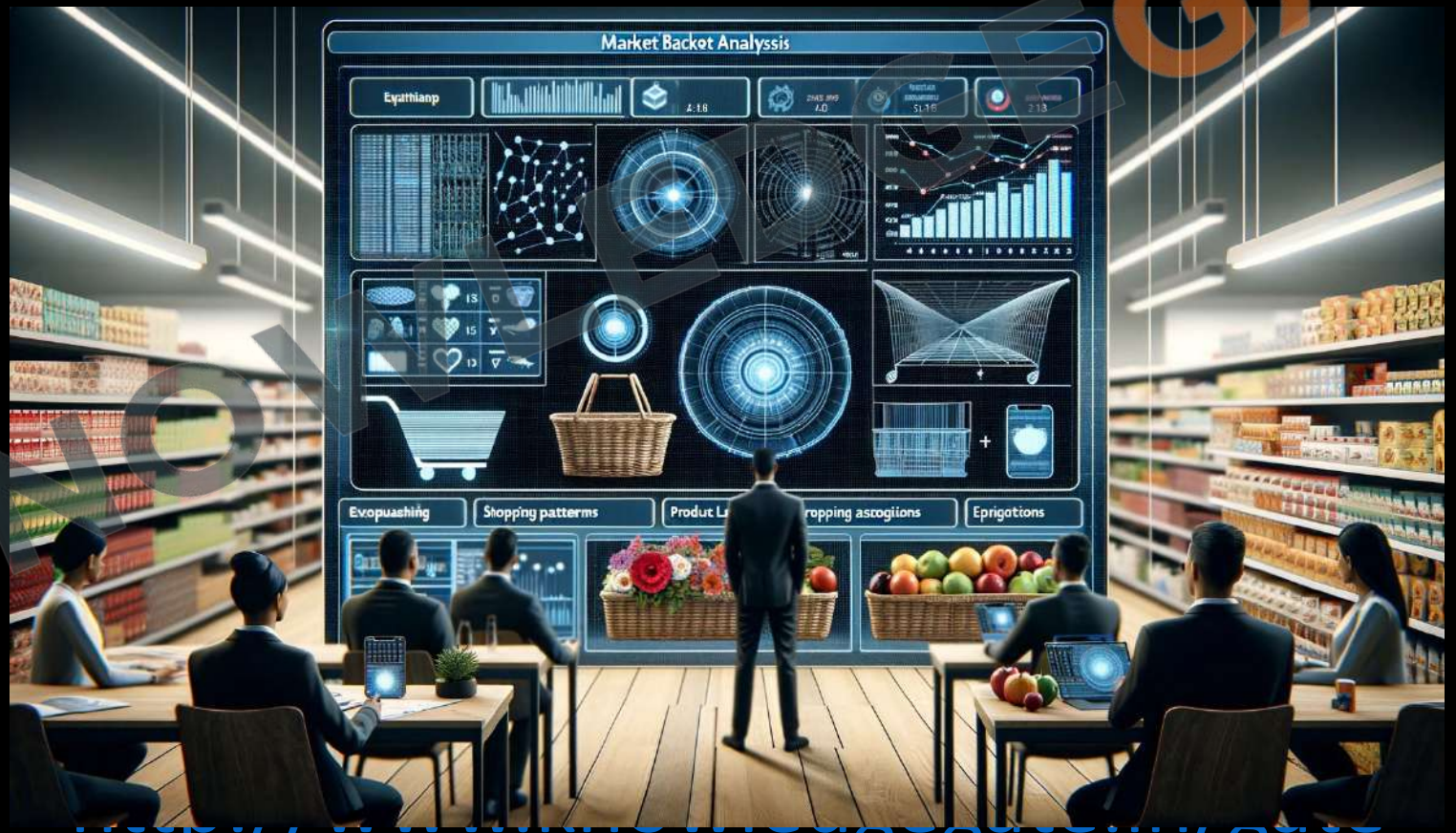
KNOWLEDGE GATE

- **Statistical Arbitrage**:
  - **Application**: In finance, statistical arbitrage involves automated trading strategies that capitalize on patterns identified in trading data.
  - **Example**: Algorithmic trading platforms that analyze historical stock data to make buy or sell decisions in milliseconds to capitalize on market inefficiencies.



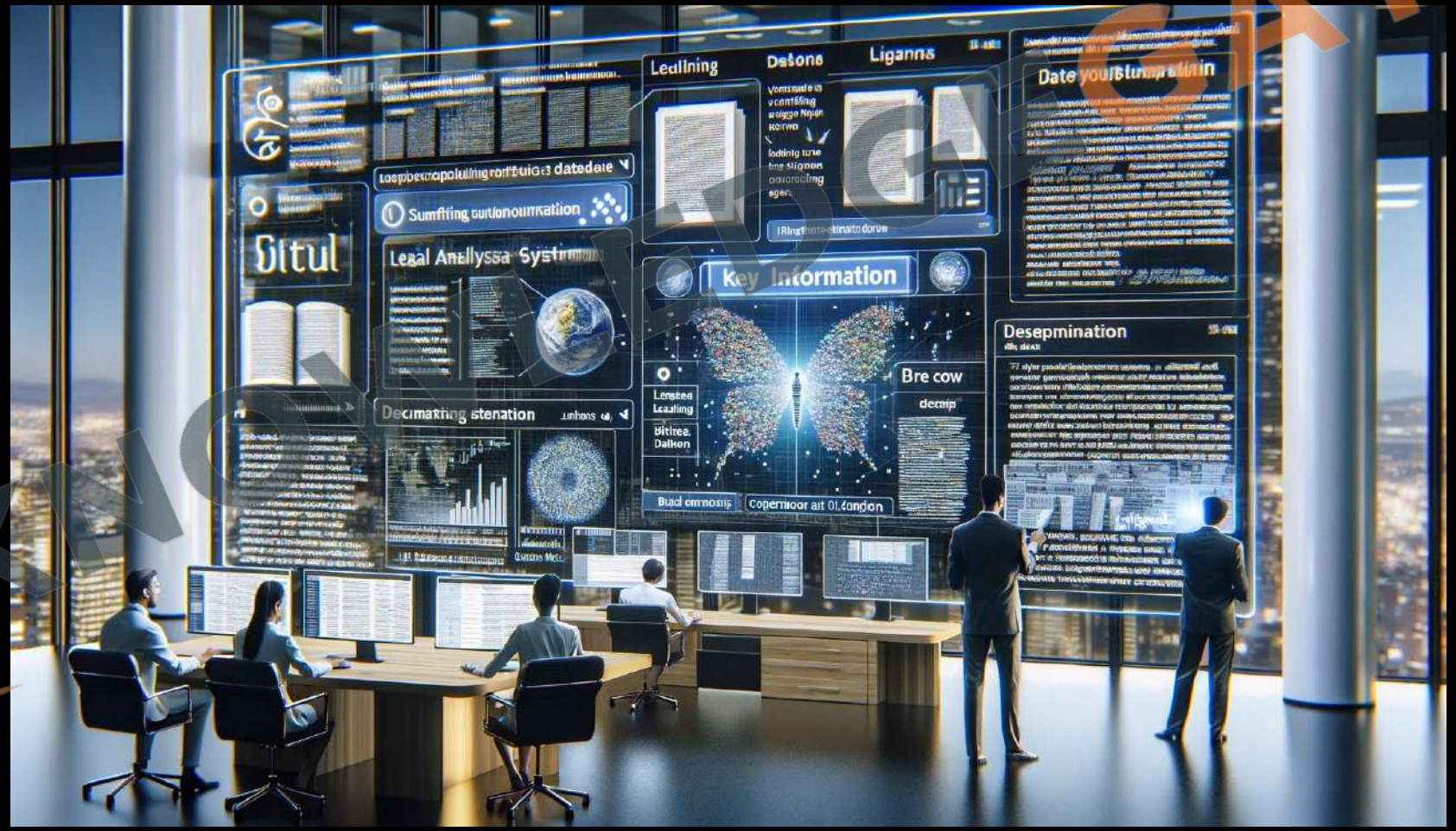
KNOWLEDGE GATE

- **Learning Associations**:
  - **Application**: This process uncovers relationships between variables in large databases, often revealing hidden patterns.
  - **Example**: Market basket analysis in retail, which analyzes purchasing patterns to understand product associations and optimize store layouts.



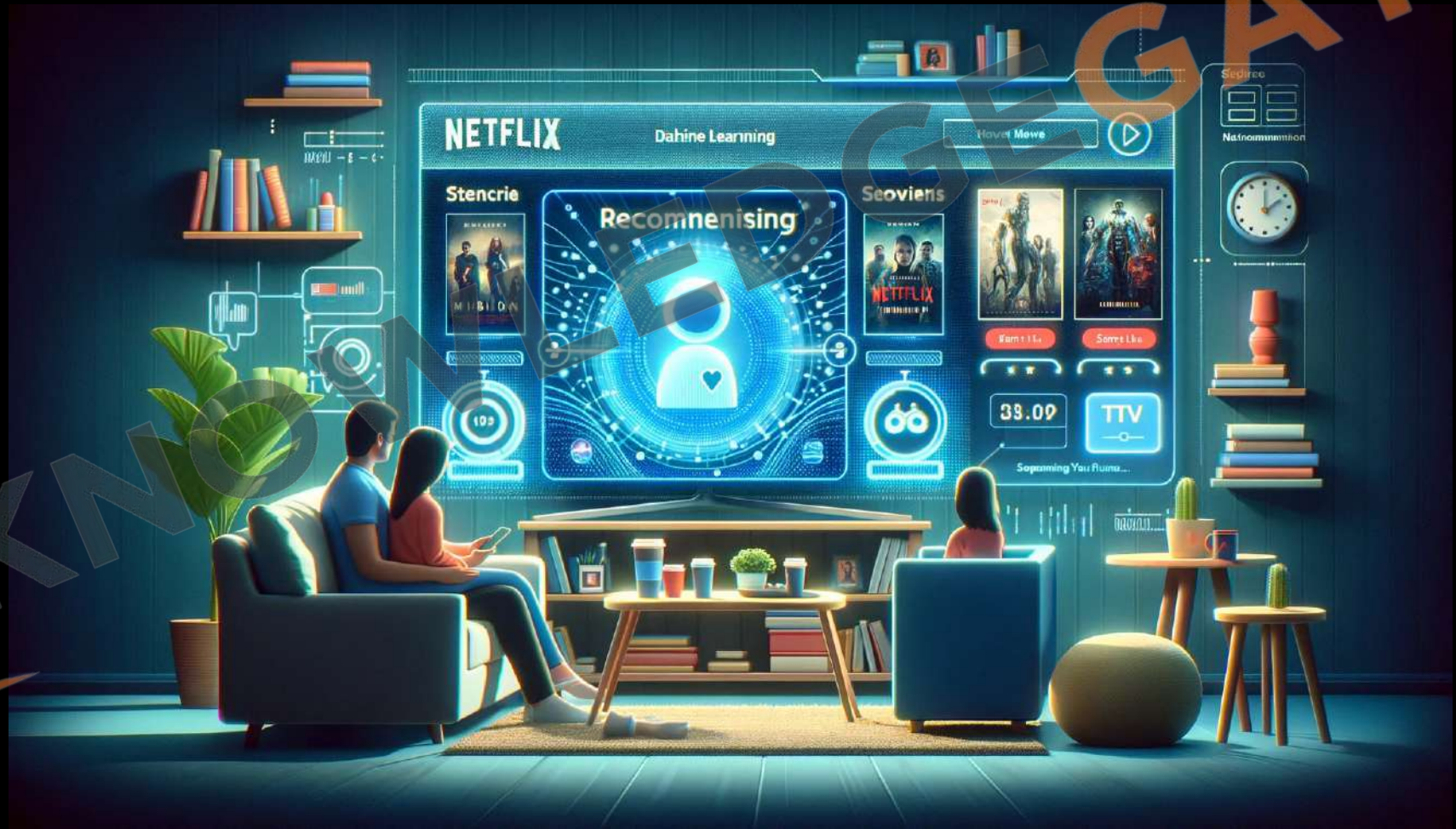
KNOWLEDGE GATE

- **Information Extraction**:
  - **Application**: Information extraction involves pulling structured information from unstructured data, like text.
  - **Example**: Extracting key pieces of information from legal documents or news articles to summarize content or populate databases automatically.



Advantages of Machine Learning:

- **Identifies Trends and Patterns**:
  - **Example**: Streaming services like Netflix analyze viewer data to identify viewing patterns and recommend shows and movies that individual users are likely to enjoy.



KNOWLEDGE GATE

Advantages of Machine Learning:

- **Automation**:
  - **Example**: Autonomous vehicles use machine learning to interpret sensory data and make driving decisions without human input, improving transportation efficiency and safety.

KNOWLEDGE GATE

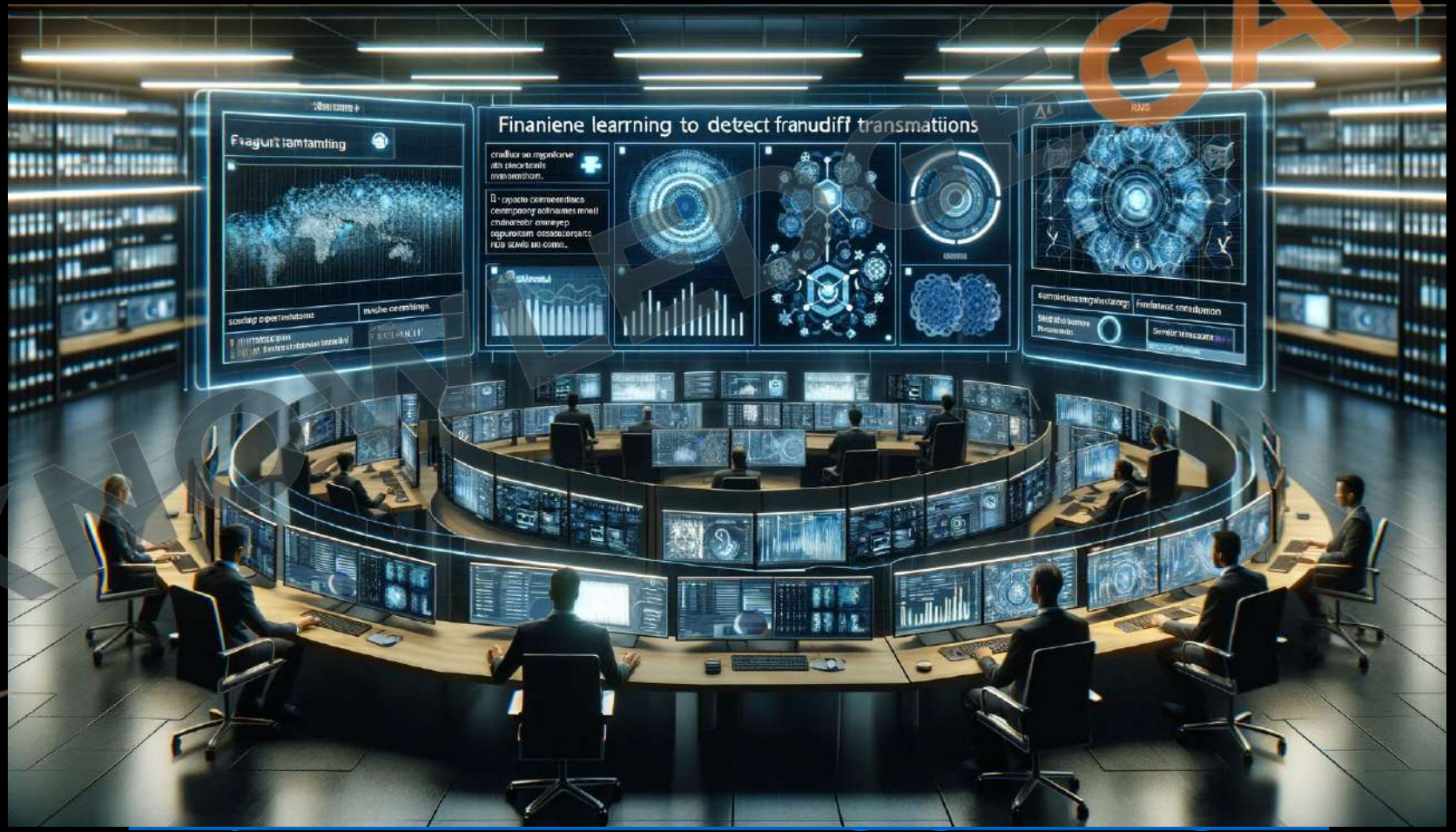
Advantages of Machine Learning:

- **Continuous Improvement**:
  - **Example**: Credit scoring systems evolve by learning from new customer data, becoming more accurate in predicting creditworthiness over time.

KNOWLEDGE GATE

Advantages of Machine Learning:

- **Handling Complex Data**:
  - **Example**: Financial institutions use machine learning algorithms to detect fraudulent transactions by analyzing complex patterns of customer behavior that would be difficult for humans to process.



KNOWLEDGE GATE

Disadvantages of Machine Learning:

- **Data Acquisition**:
  - **Example**: In healthcare, acquiring large datasets of patient medical records that are comprehensive and privacy-compliant is challenging and expensive.

KNOWLEDGEGATE

Disadvantages of Machine Learning:

- **Time and Resources**:
  - **Example**: Developing a machine learning model for predicting stock market trends requires extensive computational resources and time to analyze years of market data before it can be deployed.

KNOWLEDGEGATE

Disadvantages of Machine Learning:

- **Interpretation of Results**:
  - **Example**: In genomics research, interpreting the vast amounts of data produced by machine learning algorithms requires highly specialized knowledge to ensure findings are accurate and meaningful.

KNOWLEDGEGATE

Disadvantages of Machine Learning:

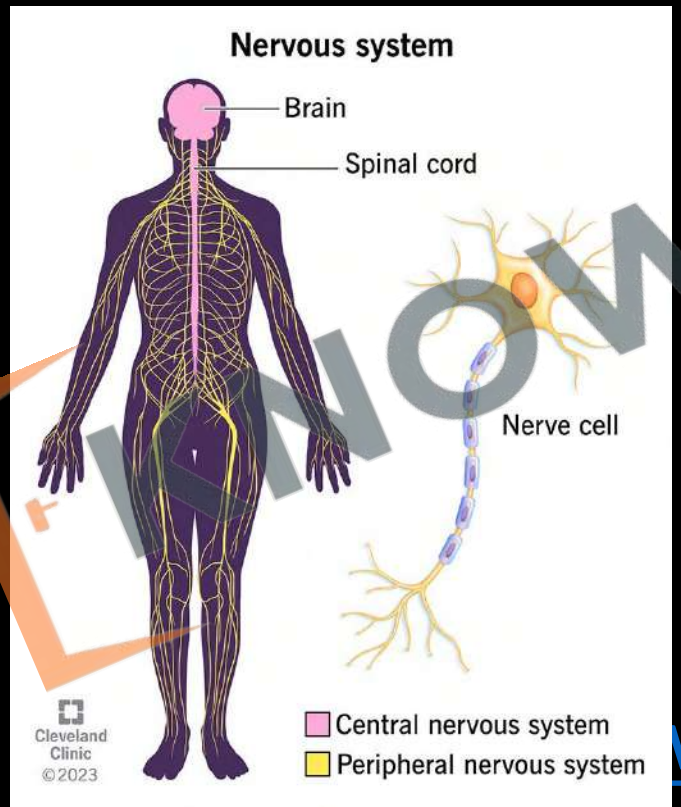
- **High Error-Susceptibility**:
  - **Example**: Early stages of facial recognition technology showed high error rates, particularly in accurately identifying individuals from minority groups, leading to potential biases and inaccuracies.

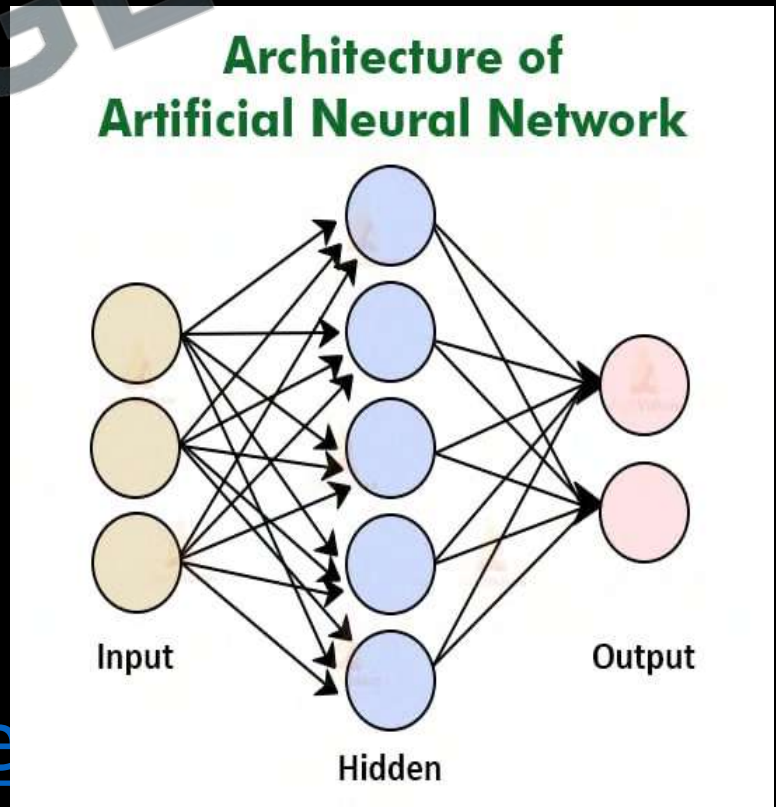
KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Machine Learning Approaches
Artificial Neural Network

- **Overview of ANNs**:
  - **Inspiration**: ANNs mimic the structure and function of the nervous systems in animals, particularly how neurons transmit signals.
  - **Functionality**: These networks are used for machine learning and pattern recognition, handling complex data inputs effectively.

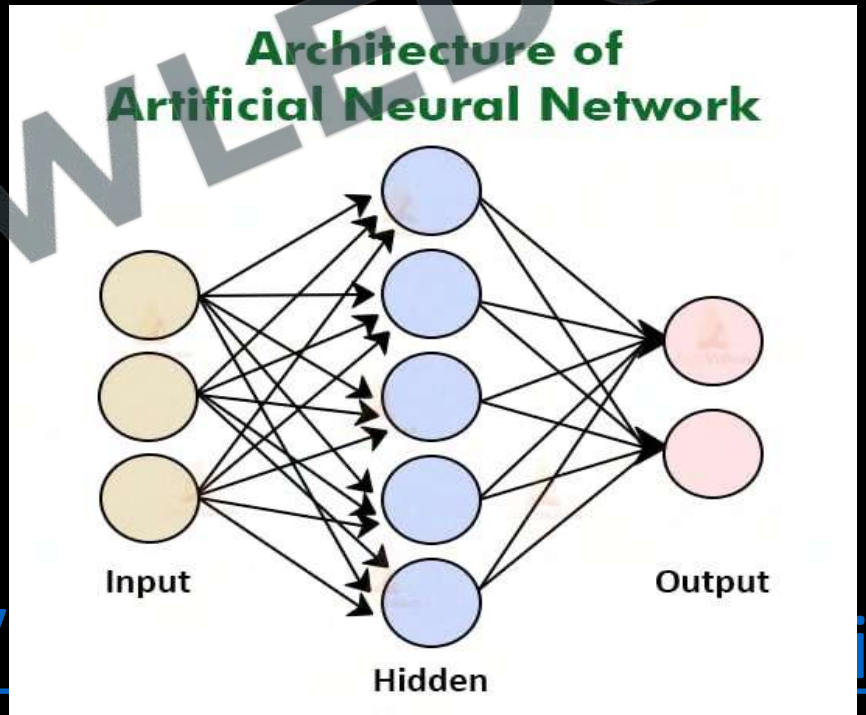




KNOWLEDGEGATE
www.knowledge

Artificial Neural Network

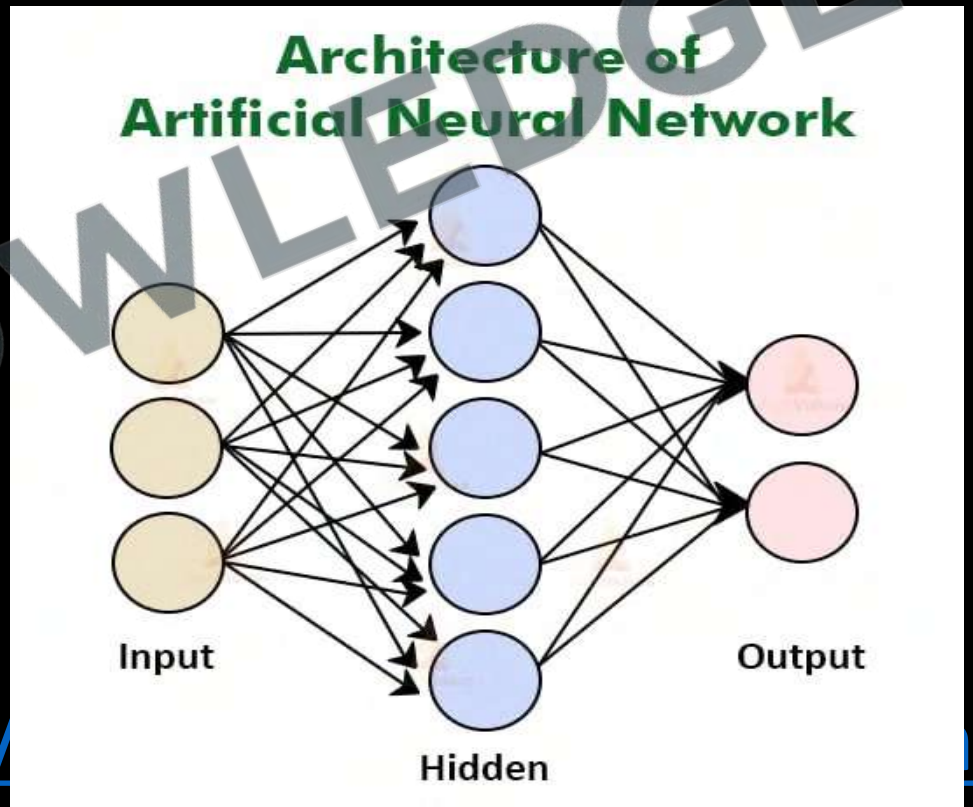
- Components of ANNs:
  - **Neurons**: Modeled as nodes within a network.
  - **Connections**: Nodes are linked by arcs that represent synapses, with weights that signify the strength of each connection.
  - **Processing**: The network processes signals in a way analogous to neural activity in biological brains.



KNOWLEDGEGATE

Artificial Neural Network

- **Operation**:
  - **Signal Transmission**: Connections in the network facilitate the propagation of data, similar to synaptic transmission in biology.
  - **Information Processing**: ANNs adjust the weights of connections to learn from data and make informed decisions.

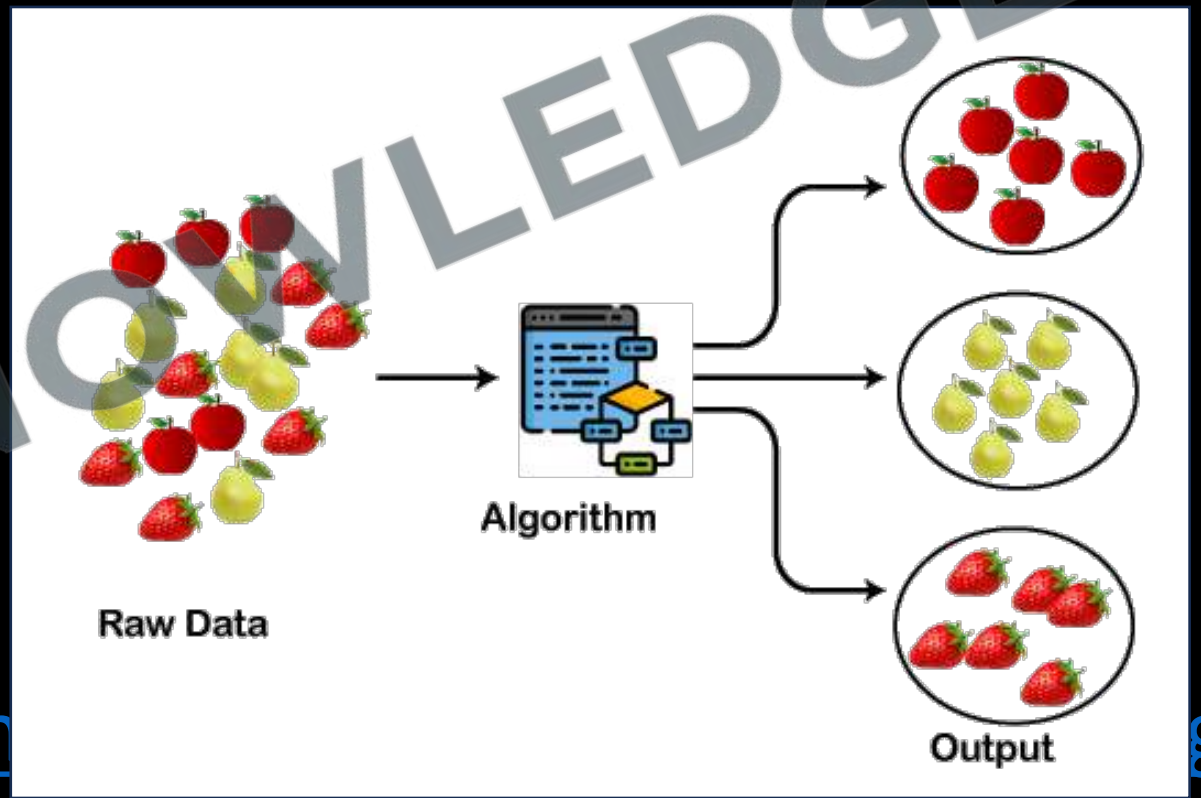


KNOWLEDGEGATE

http:/ /gate

Clustering

- **Definition**: Clustering is the process of sorting items into groups based on their similarities, forming distinct clusters where items within each cluster are more alike to each other than to those in other clusters.
- **Visual Representation**: Imagine organizing fruits into groups by type, such as grouping apples together, oranges in another group, and bananas in a separate one, visually representing how clusters segregate similar items.



KNOWLEDGEGATE

- **Characteristics**: Clusters act like exclusive clubs, where members share common traits but differ significantly from members of other clusters, illustrating the distinctiveness of each group.
- **Multidimensional Space**: Clusters are akin to islands in an expansive ocean, with dense population points representing similar items within each cluster, and low-density water symbolizing dissimilar items separating clusters.

KNOWLEDGEGATE

- **Machine Learning Perspective**: Clustering entails discovering patterns without explicit guidance, akin to exploring a forest without a map, where similarities guide the grouping process. It's a form of unsupervised learning, akin to solving a puzzle without knowledge of the final solution.

- **Unsupervised Learning**: Clustering is learning through observation, not instruction. It's like solving a puzzle without knowing what the final picture looks like.

KNOWLEDGE GATE

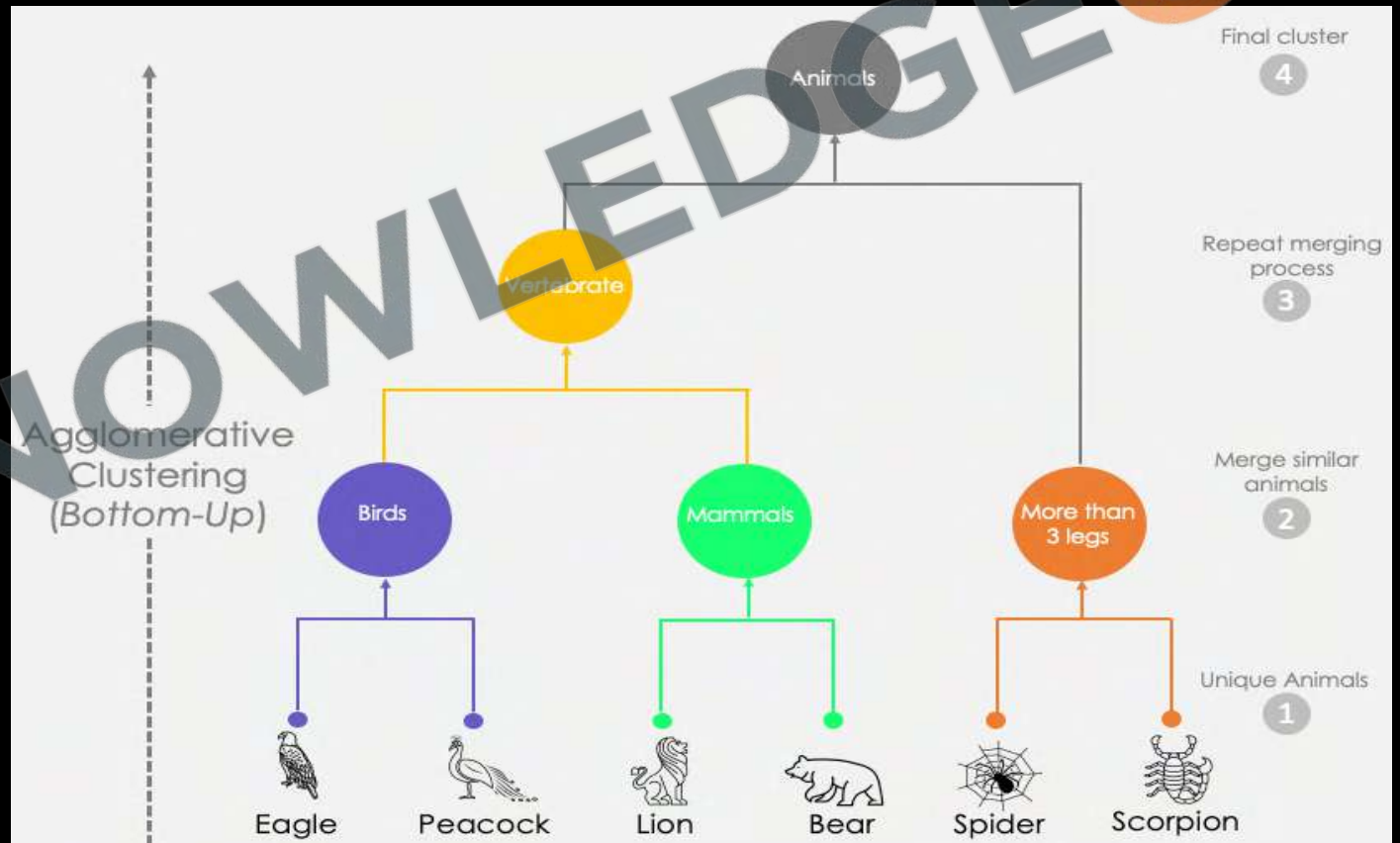
- **Data Reduction**:
  - **Example**: Imagine sorting a massive collection of books into genres (fiction, non-fiction, sci-fi, etc.). Clustering reduces the data into manageable chunks for easier processing.
- **Hypothesis Generation**:
  - **Example**: Grouping customer purchase data to generate hypotheses about shopping preferences, which can then be tested with additional research.
- **Hypothesis Testing**:
  - **Example**: Using clustering to verify if certain customer segments show different purchasing behaviors, confirming or disproving existing hypotheses.
- **Prediction Based on Groups**:
  - **Example**: Suppose we have a dataset of customer demographics and spending habits. By clustering similar customers, we can predict the behavior of new customers based on their group's characteristics. For instance, if a new customer shares similarities with the "budget-conscious" cluster, we can predict their spending patterns accordingly.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Differentiating Clustering and Classification	Clustering	Classification
1.	Clustering analyzes data objects without known class label.	In classification, data are grouped by analyzing the data objects whose class label is known.
2.	There is no prior knowledge of the attributes of the data to form clusters.	There is some prior knowledge of the attributes of each classification.
3.	It is done by grouping only the input data because output is not predefined.	It is done by classifying output based on the values of the input data.
4.	The number of clusters is not known before clustering. These are identified after the completion of clustering.	The number of classes is known before classification as there is predefined output based on input data.
5.	Unknown class label	Known class label
6.	It is considered as unsupervised learning because there is no prior knowledge of the class labels	It is considered as the supervised learning because class labels are known before

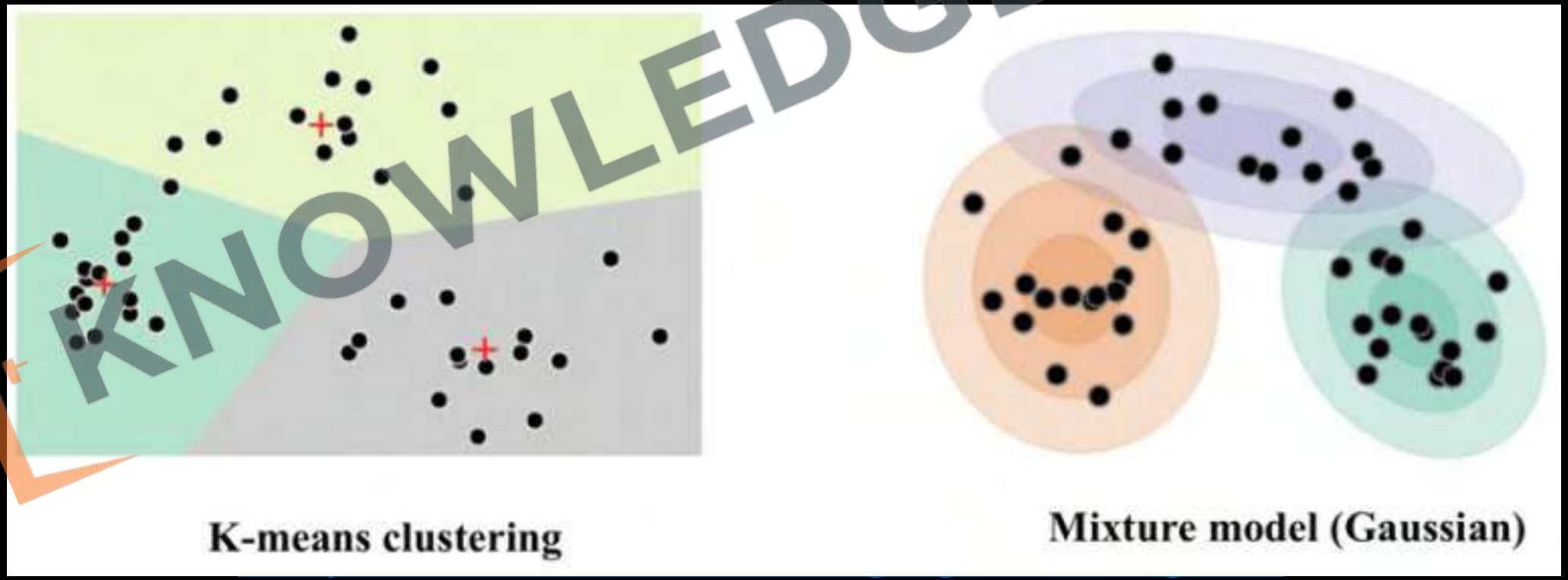
KNOWLEDGEGATE

- **Hierarchical Clustering**:
  - **Agglomerative Hierarchical Clustering**: Treats each data point as its own cluster, then merges clusters into larger ones. For example, a dataset of academic papers starts with each paper as its own cluster, then papers on similar topics merge into bigger clusters.
  - **Divisive Hierarchical Clustering**: Starts with all data points in one cluster and splits them into smaller clusters. For instance, starting with one cluster of all store customers, the cluster is split based on purchasing behavior until each customer forms their own cluster.



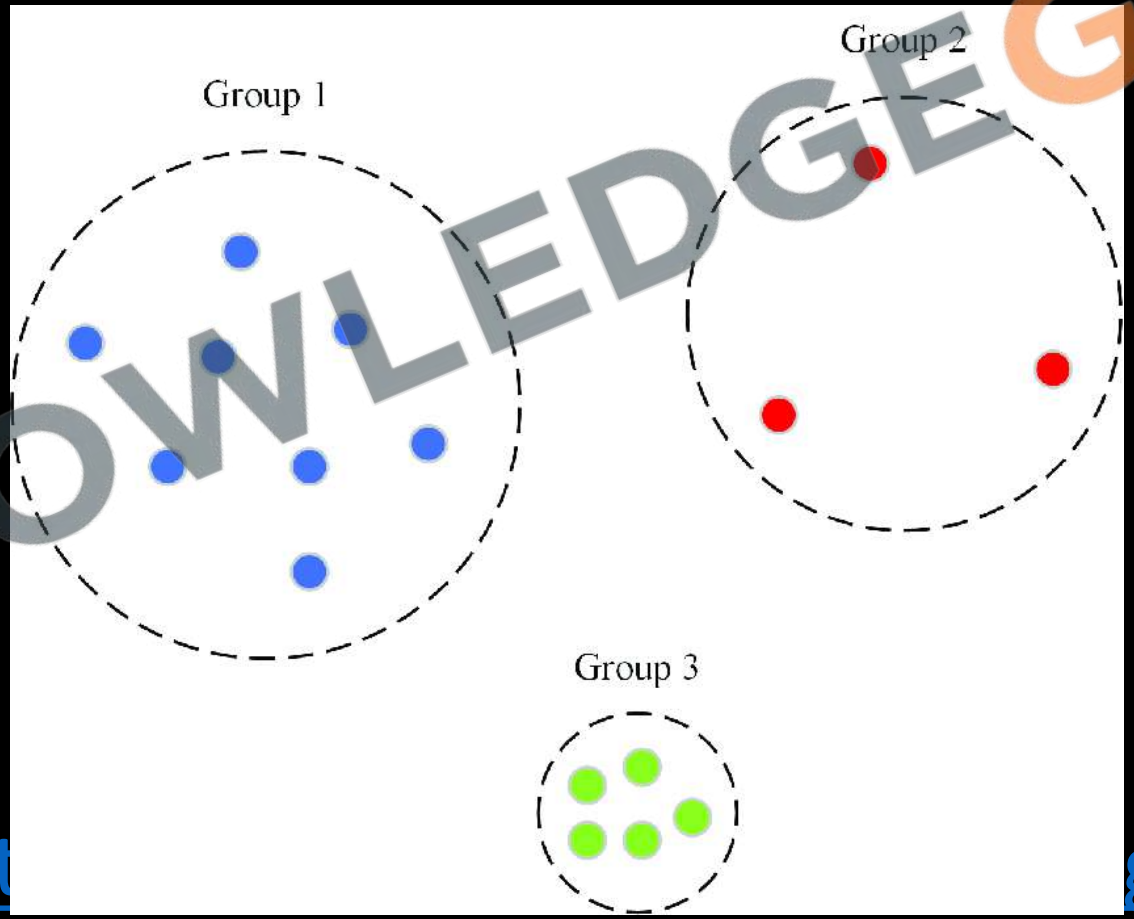
KNOWLEDGEGATE

- **Partitional Clustering**:
  - **Centroid-based Clustering (e.g., K-means)**: Partitions data into clusters, each represented by a centroid. Clusters minimize distance between data points and centroid, optimizing intra-cluster similarity and inter-cluster dissimilarity. For example, retail customers can be clustered by buying patterns, with each cluster's centroid reflecting average behavior.
  - **Model-based Clustering**: Uses a statistical model for each cluster, finding the best data fit. For instance, Gaussian mixture models assume data points in each cluster are Gaussian distributed. This method is used in image processing to model different textures as coming from different Gaussian distributions.



KNOWLEDGEGATE

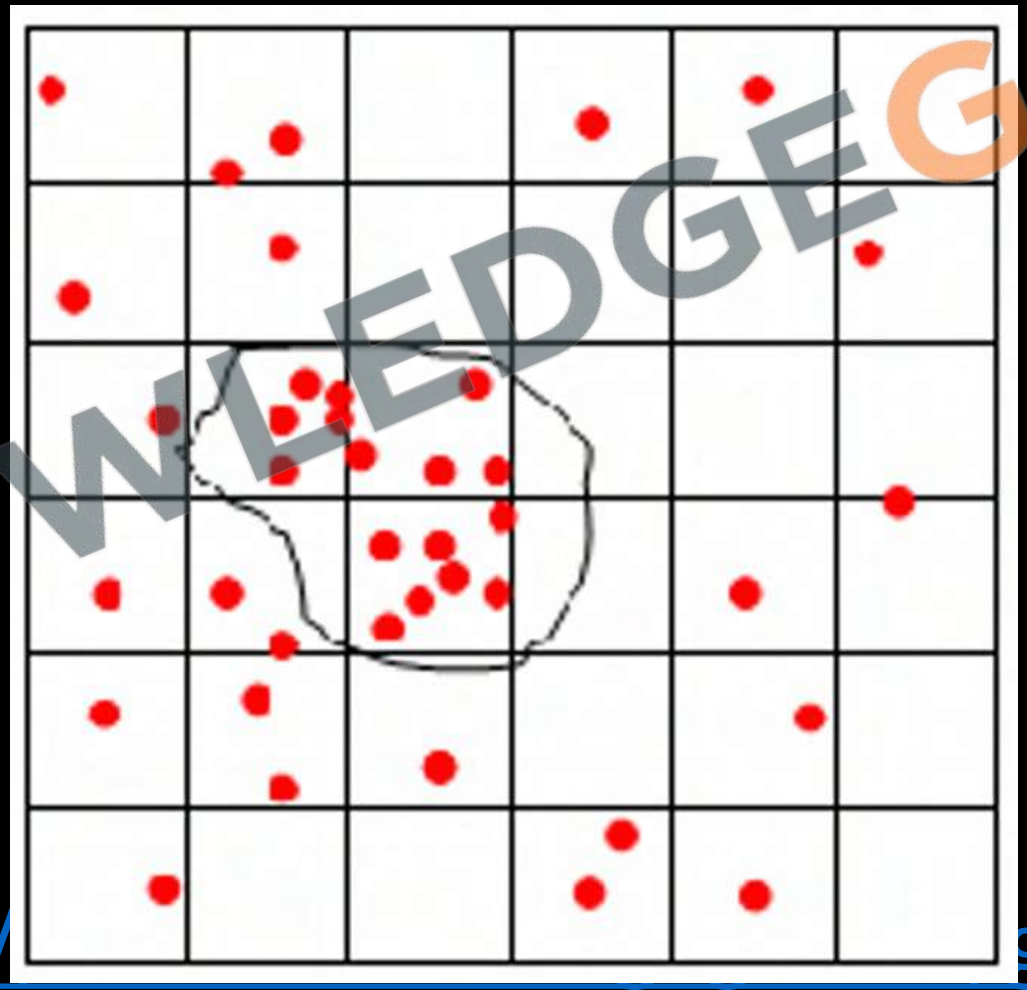
- **Density-based Clustering (e.g., DBSCAN)**:
  - This method clusters points that are closely packed together, marking as outliers points that lie alone in low-density regions. This is useful in geographical data analysis where, for example, identifying regions of high economic activity based on point density of businesses can be achieved.



KNOWLEDGEGATE

htt gate

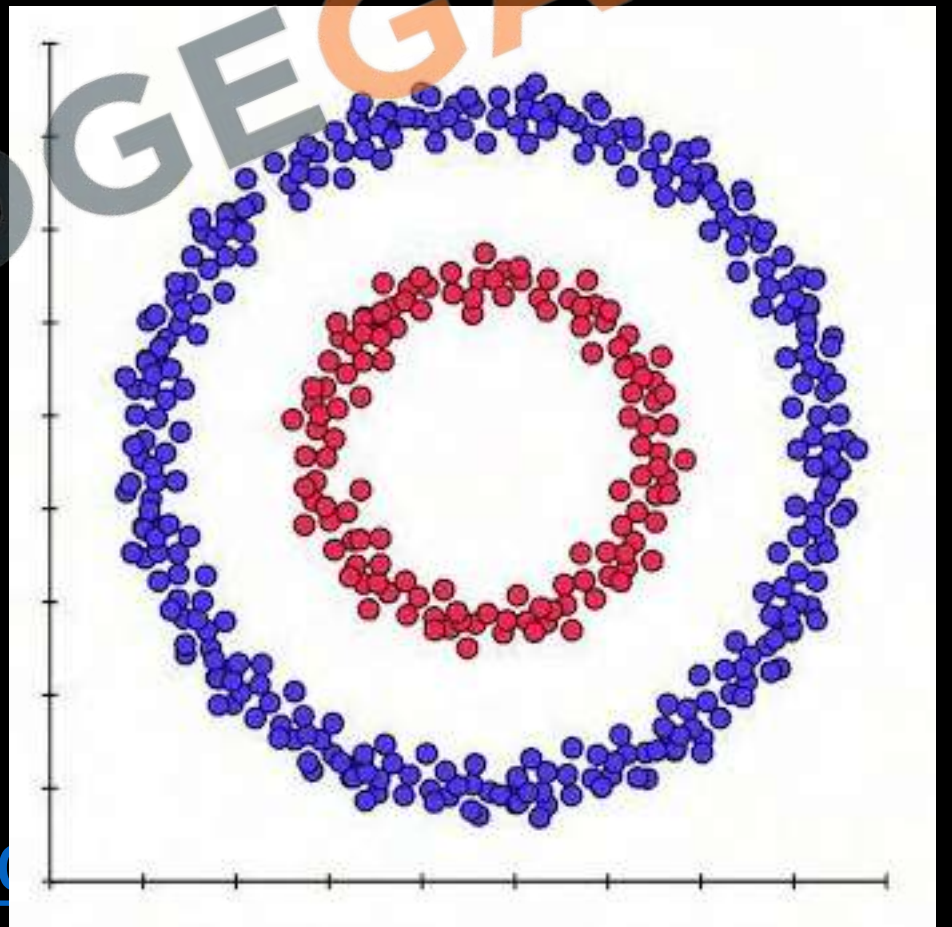
- **Grid-based Clustering**:
  - This method quantizes the space into a finite number of cells that form a grid structure and then performs clustering on the grid structure. This is effective for large spatial data sets, as it speeds up the clustering process. For example, in meteorological data, clustering can be applied to grid squares to categorize regional weather patterns.



KNOWLEDGEGATE

http:/ gate

- **Spectral Clustering**:
  - Uses the eigenvalues of a similarity matrix to reduce dimensionality before clustering in fewer dimensions. This technique is particularly useful when the clusters have a complex shape, unlike centroid-based clustering which assumes spherical clusters. For example, in social network analysis, spectral clustering can help identify communities based on the patterns of relationships between members.

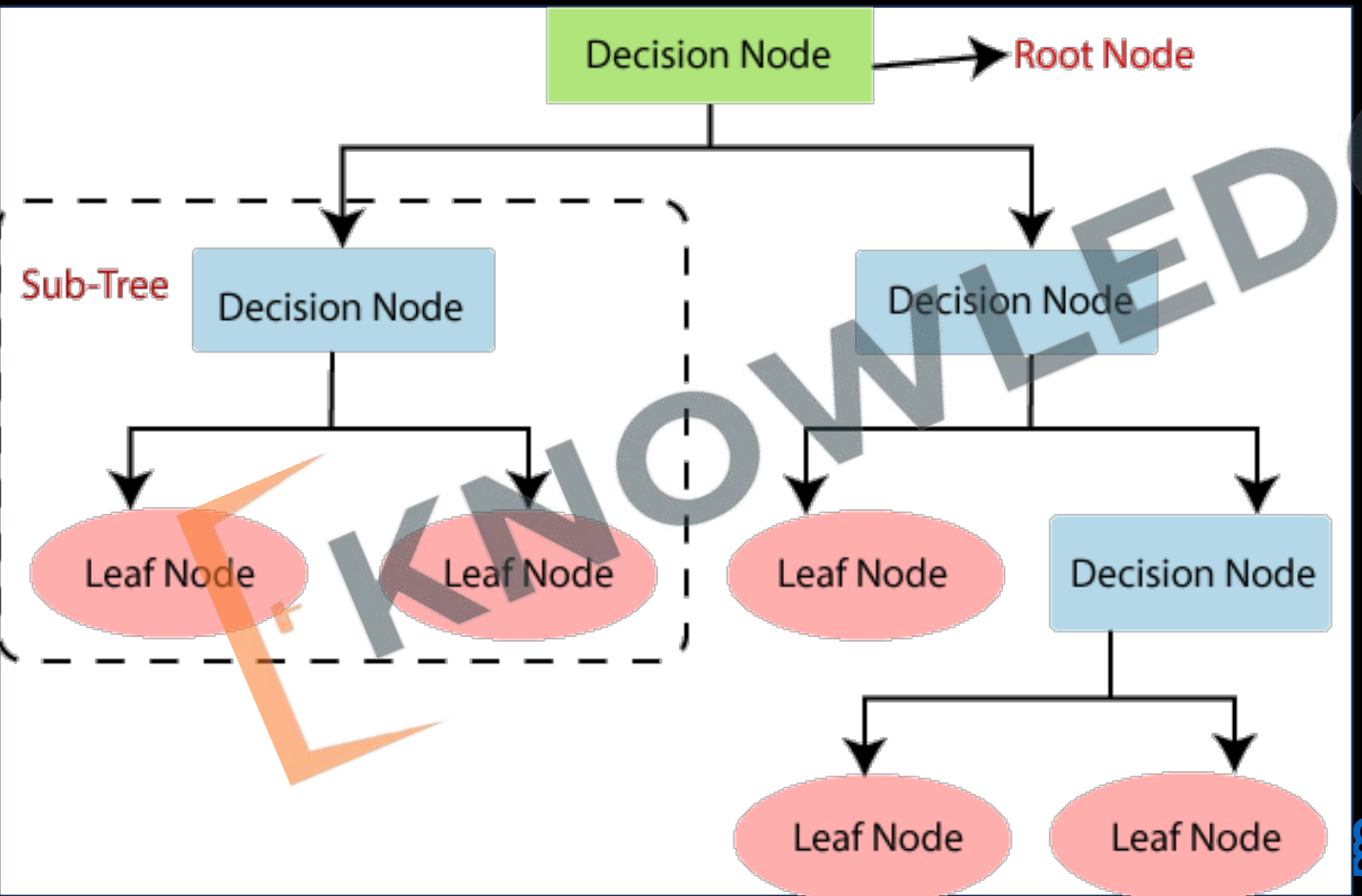


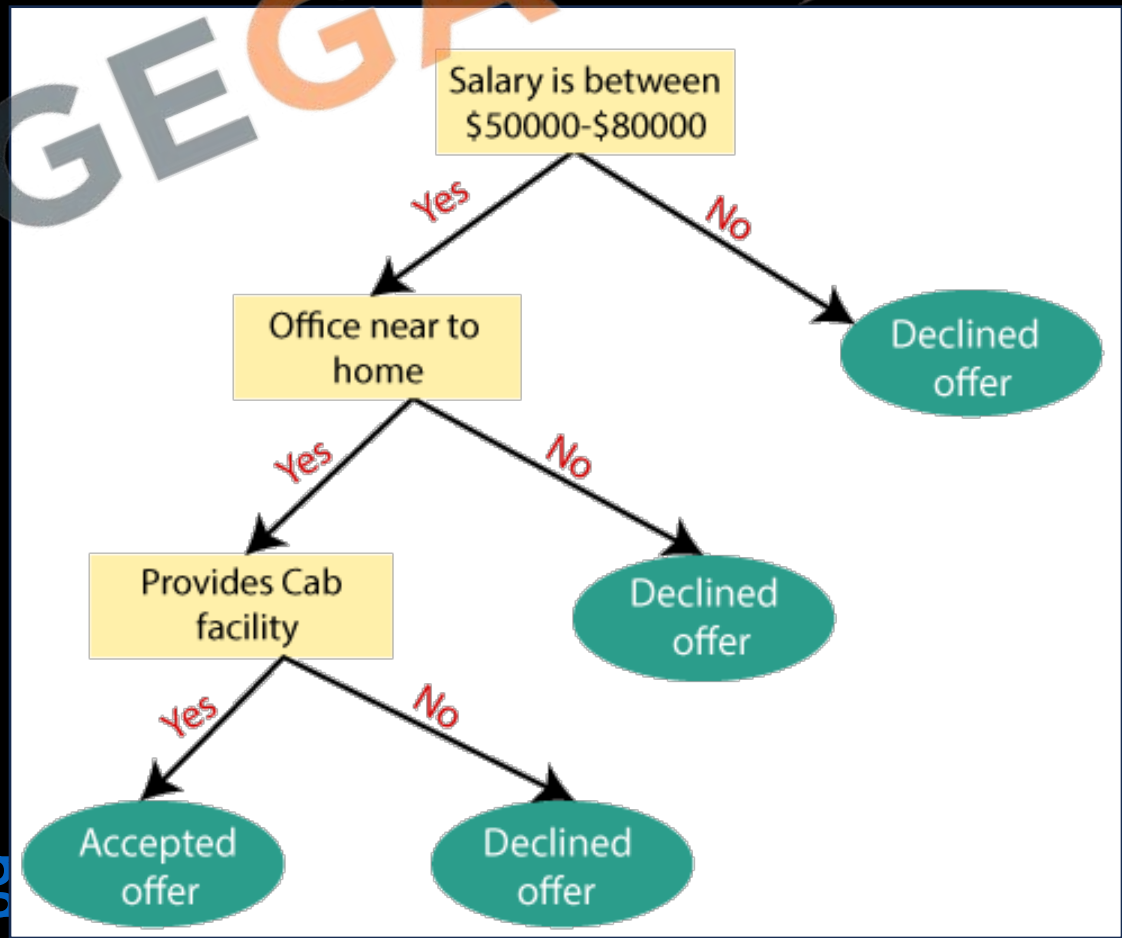
KNOWLEDGEGATE

http://www.knowle

Decision Tree

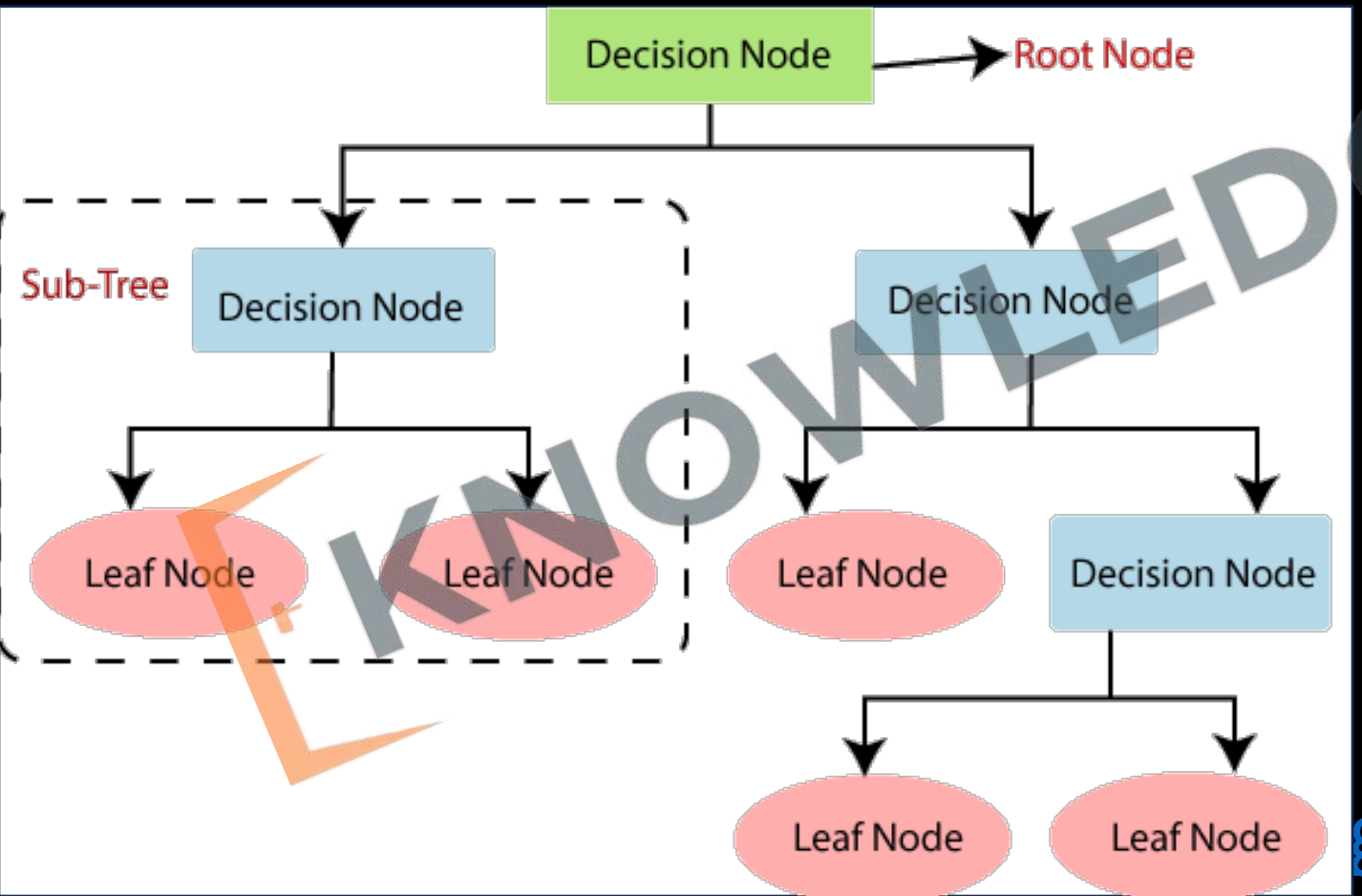
- A decision tree is a model used in data mining, statistics, and machine learning to predict an outcome based on input variables. It resembles a tree structure with branches and leaves, where each internal node represents a "decision" based on a feature, each branch represents the outcome of that decision, and each leaf node represents the final outcome or class label.

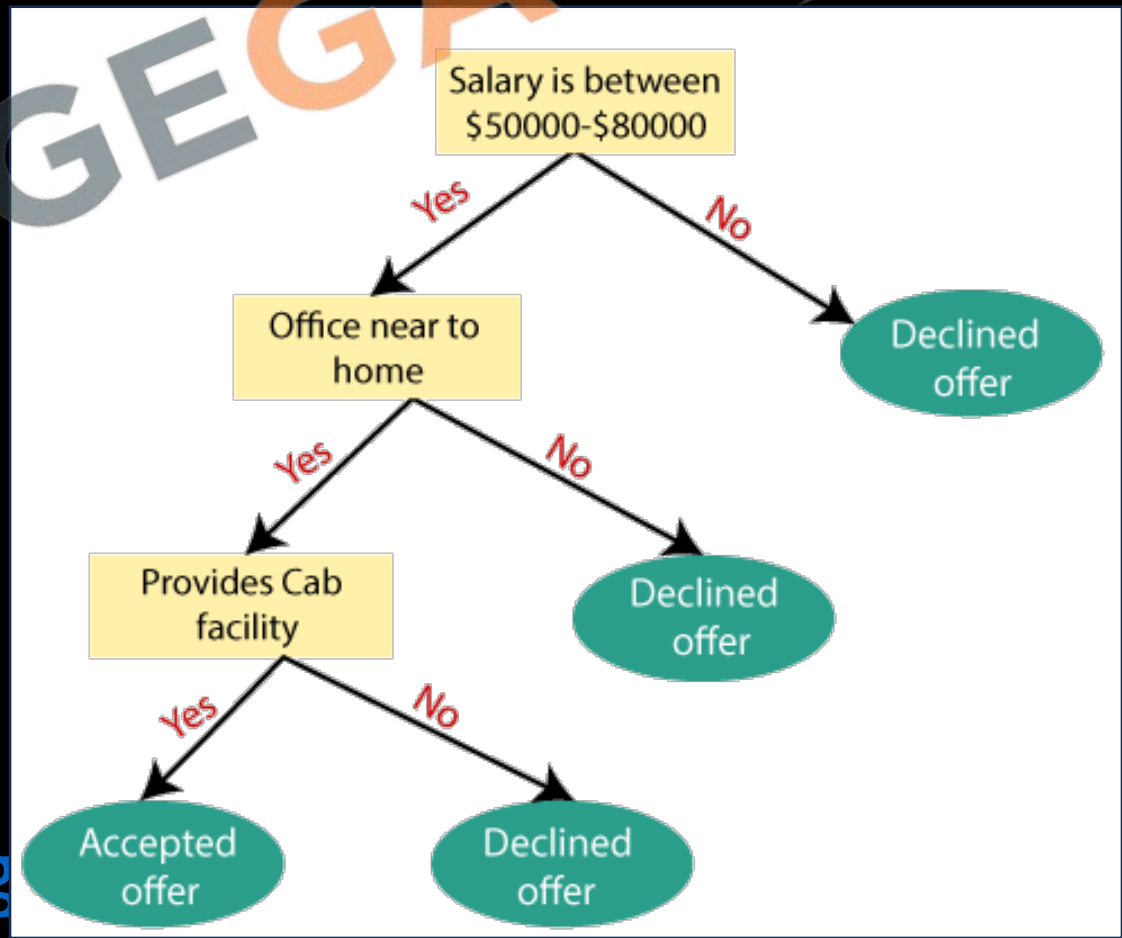




KNOWLEDGEGATE

- **Advantages and Limitations**:
  - **Advantages**:
    - Easy to interpret and visualize.
    - Requires little data preparation compared to other algorithms.
    - Can handle both numerical and categorical data.
  - **Limitations**:
    - Prone to overfitting, especially with many branches.
    - Can be biased towards features with more levels.
    - Decisions are based on heuristics, hence might not provide the best split in some cases.

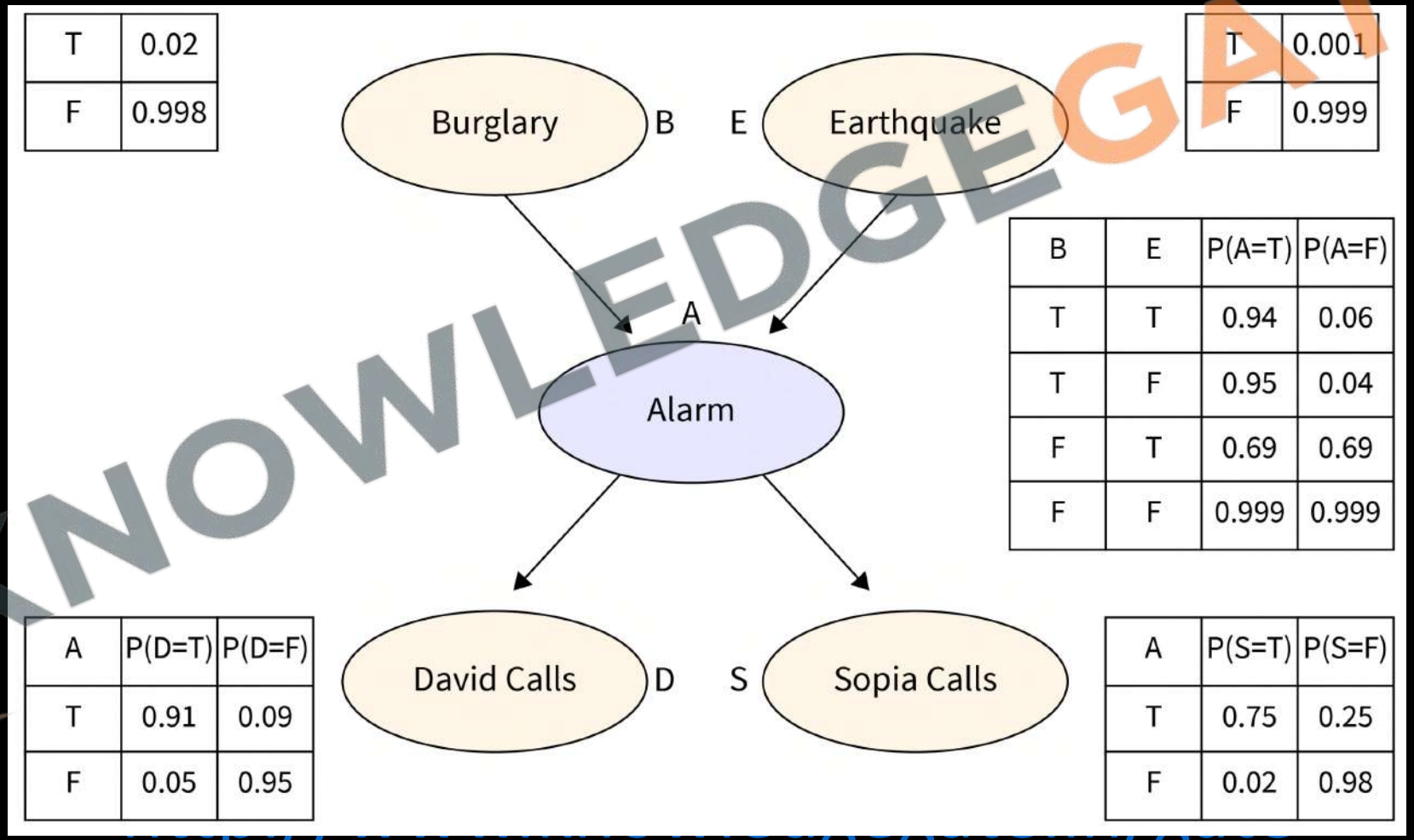




KNOWLEDGEGATE

Bayesian belief networks

- Are tools for representing and reasoning under conditions of uncertainty. They capture the probabilistic relationships among a set of variables and allow for the inference of probabilities even with partial information.



T	0.02
F	0.998

T	0.001
F	0.999

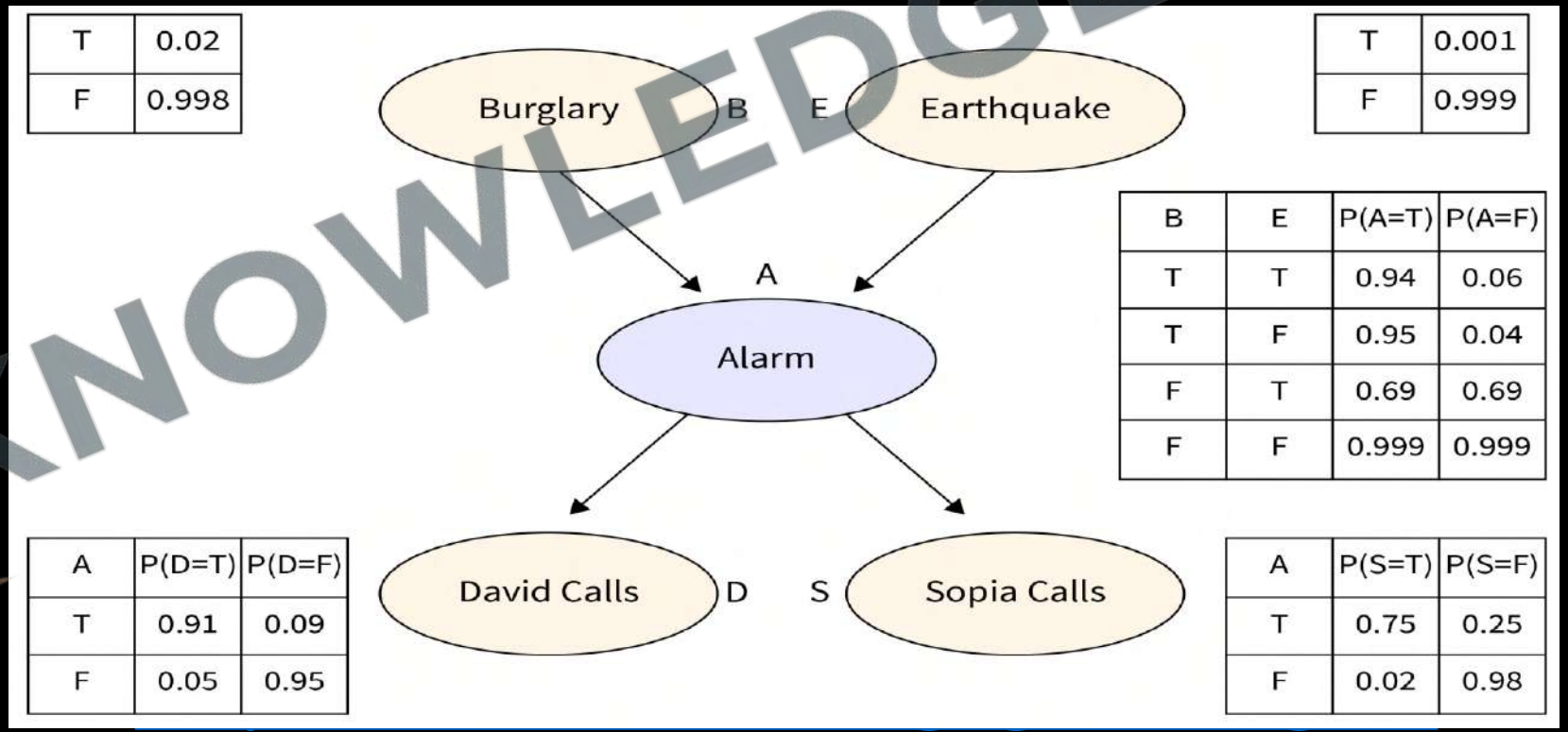
B	E	P(A=T)	P(A=F)
T	T	0.94	0.06
T	F	0.95	0.04
F	T	0.69	0.69
F	F	0.999	0.999

A	P(D=T)	P(D=F)
T	0.91	0.09
F	0.05	0.95

A	P(S=T)	P(S=F)
T	0.75	0.25
F	0.02	0.98

KNOWLEDGEGATE

- **Structure**: The core components of a Bayesian belief network include:
  - **Directed Acyclic Graph (DAG)**: Each node in the graph represents a random variable, which can be either discrete or continuous. These variables often correspond to attributes in data. Arrows or arcs between nodes represent causal influences.
  - **Conditional Probability Tables (CPTs)**: Each node has an associated table that quantifies the effect of the parents on the node.



T	0.02
F	0.998

T	0.001
F	0.999

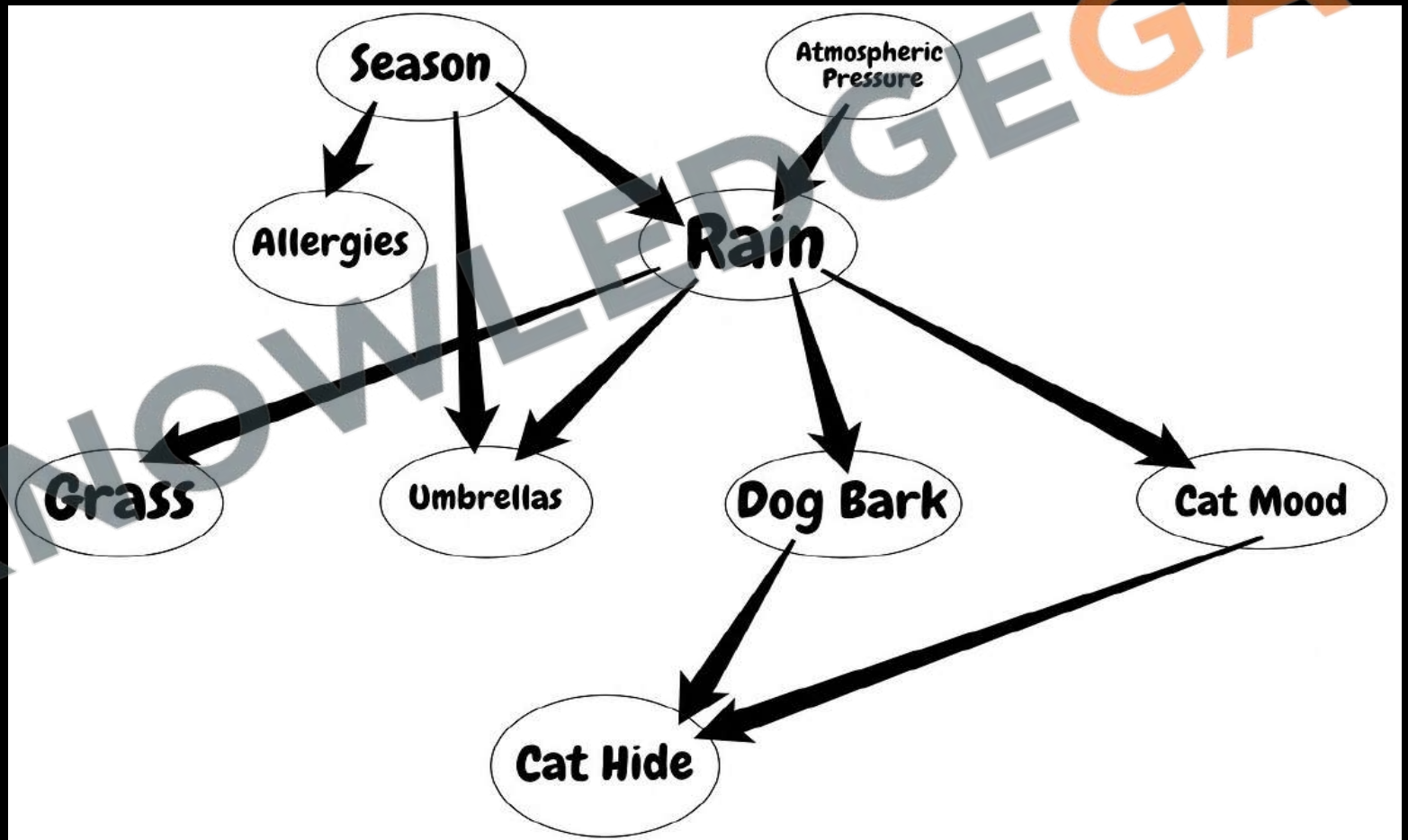
B	E	P(A=T)	P(A=F)
T	T	0.94	0.06
T	F	0.95	0.04
F	T	0.69	0.69
F	F	0.999	0.999

A	P(D=T)	P(D=F)
T	0.91	0.09
F	0.05	0.95

A	P(S=T)	P(S=F)
T	0.75	0.25
F	0.02	0.98

KNOWLEDGEGATE

- **Usage**:
  - **Learning**: Bayesian networks can be trained using data to learn the conditional dependencies.
  - **Inference**: Once trained, the network can be used for inference, such as predicting the likelihood of lung cancer given that a patient is a smoker with no family history.
  - **Classification**: Bayesian networks can classify new cases based on learned probabilities.



KNOWLEDGEGATE

Reinforcement learning

- Reinforcement learning is a type of machine learning where an agent learns to make decisions by performing actions and receiving feedback in the form of rewards or penalties. This method is similar to how individuals learn from the consequences of their actions in real life.
- Key Concepts in Reinforcement Learning:
  - Environment: The world in which the agent operates.
  - **State**: The current situation of the agent.
  - **Actions**: What the agent can do.
  - **Rewards**: Feedback from the environment which can be positive (reinforcements) or negative (punishments).

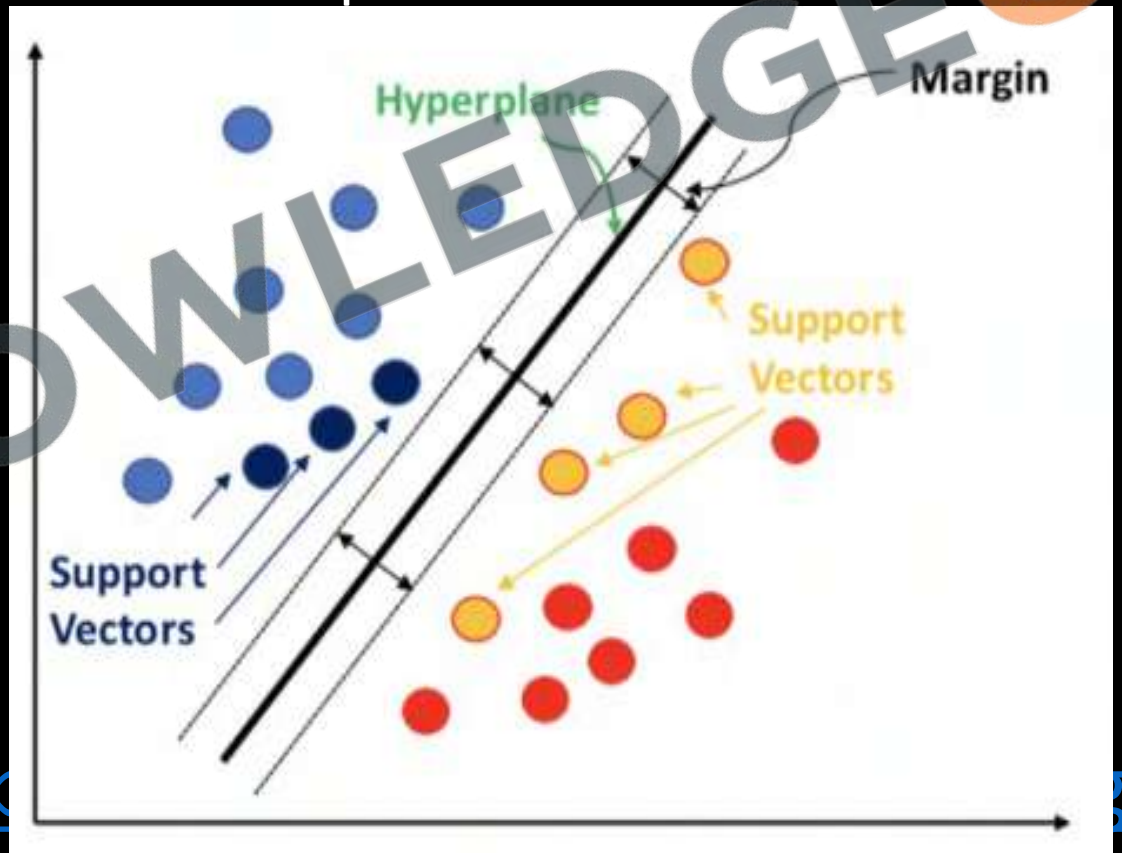
KNOWLEDGEGATE

- Imagine a robot navigating a maze. The robot has to find the shortest path to a destination without prior knowledge of the layout. Each step it takes provides new information:
- If it moves closer to the destination, it receives a positive reward.
- If it hits a wall or moves away from the goal, it receives a negative reward. Through trial and error, the robot learns the optimal path by maximizing its cumulative rewards.

KNOWLEDGE GATE

Support Vector Machine

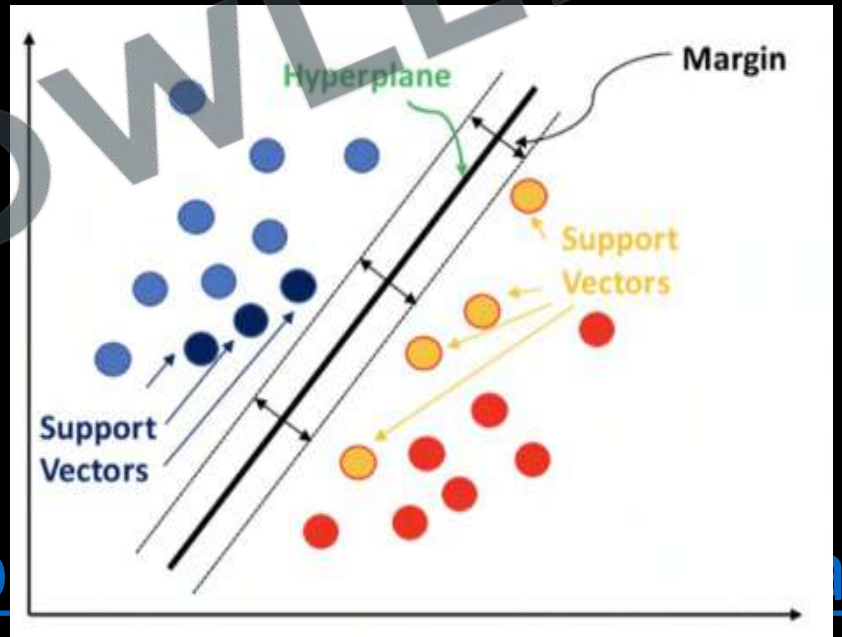
- A Support Vector Machine (SVM) is a powerful machine most commonly used in classification problems.
- SVM constructs a hyperplane or set of hyperplanes in a high-dimensional space, which can be used for classification. The goal is to find the best hyperplane that has the largest distance to the nearest training data points of any class (functional margin), in order to improve the classification performance on unseen data.



KNOWLEDGEGATE

http gate

- **Applications of SVM**:
  - **Text and Hypertext Classification**: For filtering spam and categorizing text based content for news articles.
  - **Image Classification**: Useful in categorizing images into different groups (e.g., animals, cars, fruits).
  - **Handwritten Character Recognition**: Used to recognize letters and digits from handwritten documents.
  - **Biological Sciences**: Applied in protein classification and cancer classification based on gene expression data.



KNOWLEDGEGATE

http ate.in/gate

Genetic Algorithm

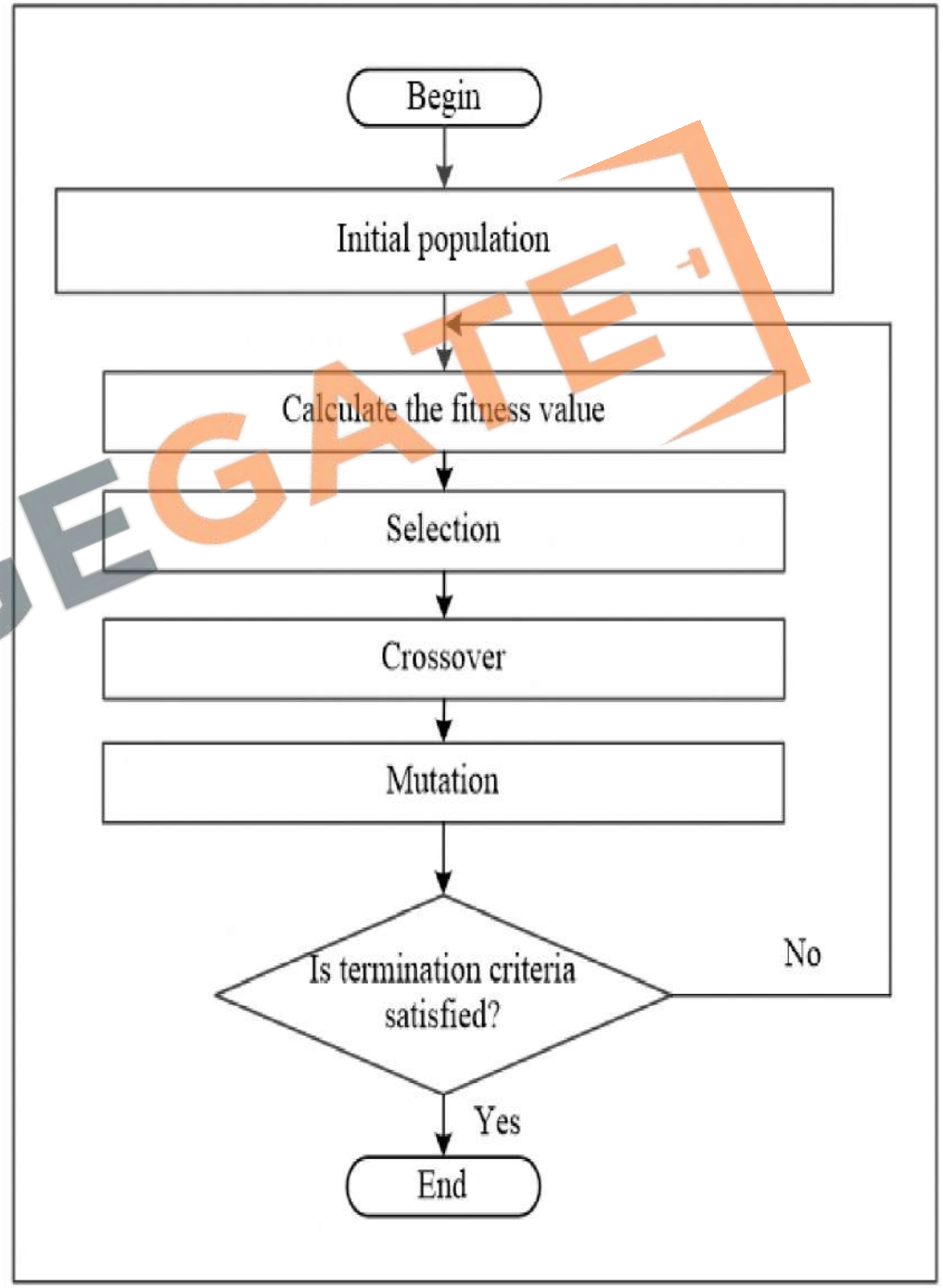
- A genetic algorithm (GA) is a search heuristic inspired by Charles Darwin's theory of natural selection. It is used to find optimal or near-optimal solutions to complex problems which might otherwise take a long time to solve.

- **Overview of Genetic Algorithm**:
  - **Purpose**: Genetic algorithms are used to solve optimization and search problems by mimicking the process of natural selection.
  - **Process**: This involves a population of individuals which evolve towards a better solution by combining the characteristics of high-quality individuals.

KNOWLEDGEGATE

Flowchart of Genetic Algorithm Process:

1. **Initialize Population**: Start with a randomly generated population of n individuals.
2. **Fitness Evaluation**: Evaluate the fitness of each individual in the population. The fitness score determines how good an individual solution is at solving the problem.
3. **Selection**: Select pairs of individuals (parents) based on their fitness scores. Higher fitness scores generally mean a higher chance of selection.
4. **Crossover (Recombination)**: Combine the features of selected parents to create offspring. This simulates sexual reproduction.
5. **Mutation**: Introduce random changes to individual offspring to maintain genetic diversity within the population.
6. **Replacement**: Replace the older generation with the new generation of offspring.
7. Termination: Repeat the process until a maximum number of generations is reached or a satisfactory fitness level is achieved.



KNOWLEDGEGATE
http://www.knowledge

Example of Genetic Algorithm: Imagine we want to optimize the design of an aerodynamic car. The objective is to minimize air resistance, which directly impacts fuel efficiency.

- **Encoding**: Each car design is encoded as a string of numbers (genes), representing different design parameters like shape, size, and materials.
- **Initial Population**: Generate a random set of car designs.
- **Fitness Evaluation**: Use a simulation to calculate the air resistance of each design.
- **Selection**: Choose designs with the lowest air resistance.
- **Crossover**: Create new designs by mixing the features of selected designs.
- **Mutation**: Slightly alter the designs to explore a variety of design possibilities.
- **Repeat**: Continue the process to evolve increasingly efficient designs over multiple generations.

KNOWLEDGEGATE

Issues in Machine Learning

- **Data Quality**:
  - **Importance of Quality**: High-quality data is crucial for developing effective ML models. Poor data can lead to inaccurate predictions and unreliable outcomes.
  - **Challenges**:
    - **Data Evaluation and Integration**: Ensuring data is clean, well-integrated, and representative. For example, a model trained to recognize faces needs a diverse dataset that reflects various ethnicities, ages, and lighting conditions.
    - **Data Exploration and Governance**: Implementing robust data governance to maintain the integrity and usability of data over time.

KNOWLEDGEGATE

- **Transparency**:
  - **Model Explainability**: ML models, especially complex ones like deep neural networks, can act as "black boxes," where it's unclear how decisions are made.
  - **Example**: In a credit scoring model, it's crucial for regulatory and fairness reasons to explain why a loan application was denied, which can be challenging with highly complex ML models.

KNOWLEDGEGATE

- **Manpower**:
  - **Skill Requirement**: Effective use of ML requires a combination of skills in data science, software development, and domain expertise.
  - **Bias Avoidance**: Having diverse teams is important to prevent biases in model development.
  - **Example**: An organization implementing an ML solution for customer service might need experts in natural language processing, software engineering, and customer interaction to develop a comprehensive tool.

KNOWLEDGEGATE

- **Other Issues**:
  - **Misapplication of Technology**: ML is not suitable for every problem, and its misuse can lead to wasted resources or poor decisions.
    - **Example**: Employing deep learning for a simple data analysis task, where traditional statistical methods would be more appropriate and less costly.
  - **Innovation Misuse**: The hype around new ML techniques can lead to premature adoption without proper understanding or necessity.
    - **Example**: The early overuse of deep learning in situations where simpler models could suffice, like predicting straightforward outcomes from small datasets.
  - **Traceability and Reproducibility**: Ensuring that ML experiments are reproducible and that results can be traced back to specific data and configuration settings.
    - **Example**: A research team must be able to replicate an ML experiment's results using the same datasets and parameters to verify findings and ensure reliability.

KNOWLEDGE GATE

S. No.	Data Science	Machine Learning
1	Involves data cleansing, preparation, and analysis.	Practice of using algorithms to learn from and make predictions based on data.
2	Deals with a variety of data operations.	A subset of Artificial Intelligence focused on statistical models and algorithms.
3	Focuses on sourcing, cleaning, and processing data to extract meaningful insights.	Programs learn from data and improve autonomously without explicit instructions.
4	Tools include SAS, Tableau, Apache Spark, MATLAB.	Tools include Amazon Lex, IBM Watson Studio, Microsoft Azure ML Studio.
5	Applied in fraud detection, healthcare analysis, and business optimization.	Used in recommendation systems like Spotify, facial recognition technologies.

KNOWLEDGEGATE

(UNIT-2: REGRESSION & BAYESIAN LEARNING)

- **REGRESSION**: Linear Regression and Logistic Regression.
- **BAYESIAN LEARNING** - Bayes theorem, Concept learning, Bayes Optimal Classifier, Naïve Bayes classifier, Bayesian belief networks, EM algorithm.
- **SUPPORT VECTOR MACHINE**: Introduction, Types of support vector kernel - (Linear kernel, polynomial kernel,and Gaussiankernel), Hyperplane - (Decision surface), Properties of SVM, and Issues in SVM.

KNOWLEDGEGATE

Regression

- Regression is a statistical technique used to analyze the relationship between a dependent variable and one or more independent variables. It is widely used in areas like finance, economics, and more, to predict outcomes and understand variable interactions.

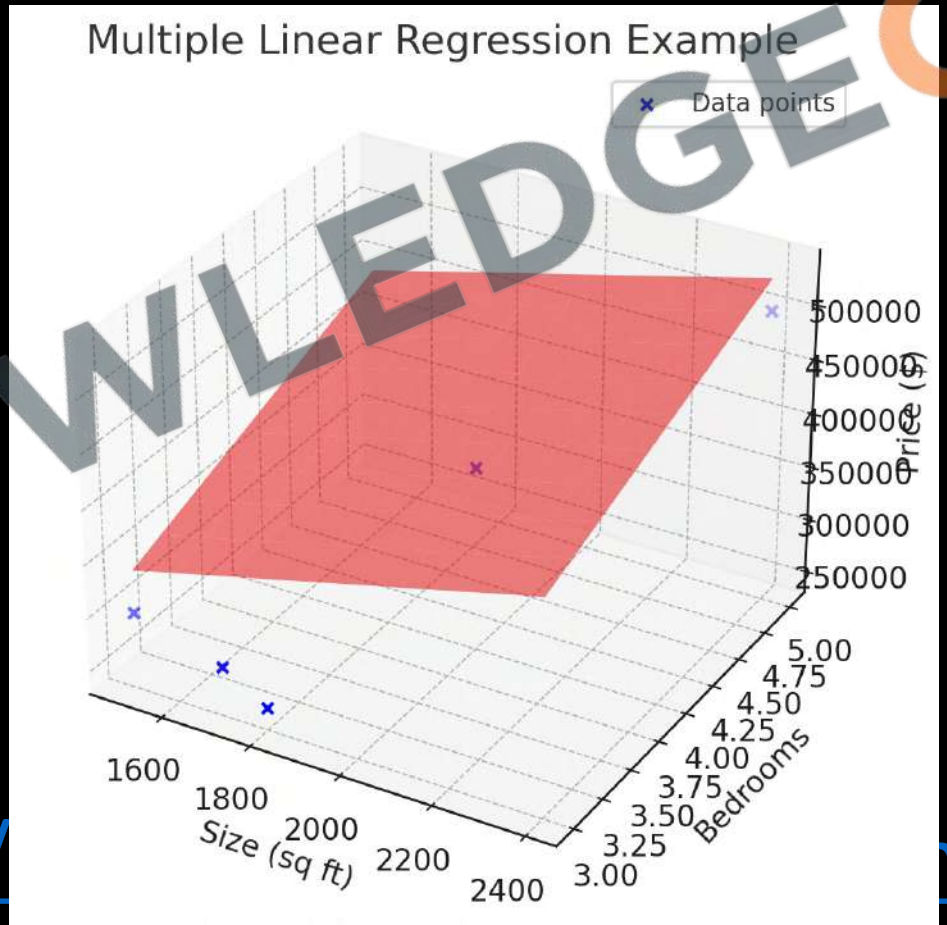
KNOWLEDGE GATE

Types of Regression

- **Simple Linear Regression**:
  - This method involves one independent variable used to predict the outcome of a dependent variable. The formula is Y=a+bX+u, where:
    - Y is the dependent variable we want to predict.
    - X is the independent variable used for prediction.
    - a is the intercept of the regression line (value of Y when X is 0).
    - b is the slope of the regression line, representing the change in Y for a one-unit change in X.
    - u is the regression residual, which is the error in the prediction.
  - Example: Predicting house prices (Y) based on house size (X). A larger house size generally increases the house price.

KNOWLEDGEGATE

- **Multiple Linear Regression**:
  - Involves two or more independent variables to predict the outcome. The formula is $Y=a+b_1X_1+b_2X_2+...+b_nXn+u$, where:
    - Each X_i represents a different independent variable.
    - Each b_i is the coefficient for the corresponding independent variable, showing how much Y changes when that variable changes by one unit, holding other variables constant.



KNOWLEDGEGATE

http:/ /gate

Logistic Regression

Definition of Logistic Regression:

- Logistic regression is a statistical method and a type of supervised machine learning algorithm. It is used to estimate the probability that a given input belongs to a certain category (typically a binary outcome).

- **Characteristics of the Dependent Variable**:
  - The target variable in logistic regression is binary, meaning it has two possible outcomes. These outcomes are usually coded as 1 (indicating success or the presence of a feature, like "yes") and 0 (indicating failure or the absence of a feature, like "no").

KNOWLEDGEGATE

- Applications:
  - Logistic regression is widely applied in fields such as medicine, finance, and marketing. It helps in binary classification tasks such as detecting whether an email is spam or not, predicting whether a patient has a disease like diabetes, or determining if a transaction might be fraudulent.
- **Example**:
  - **Predicting Disease Occurrence**: Suppose a medical researcher wants to predict the likelihood that individuals have diabetes based on their age and BMI. Here, the outcome variable Y is whether the person has diabetes (1) or not (0), and the predictors X_1 and X_2 are age and BMI, respectively. The logistic regression model would help estimate the probability of diabetes for different age groups and BMI levels, using historical data to determine the coefficients b_1 and b_2 for age and BMI.

KNOWLEDGE GATE

Aspect	Linear Regression	Logistic Regression
Type of Model	Supervised regression model	Supervised classification model
Prediction Outcome	Predicts continuous values	Predicts binary outcomes (0 or 1)
Mathematical Model	Uses linear functions	Uses logistic functions with an activation function
Purpose	Estimates values of a dependent variable	Estimates probability of an event
Example	Predicting house prices based on size	Predicting whether a patient has a disease or not

KNOWLEDGEGATE

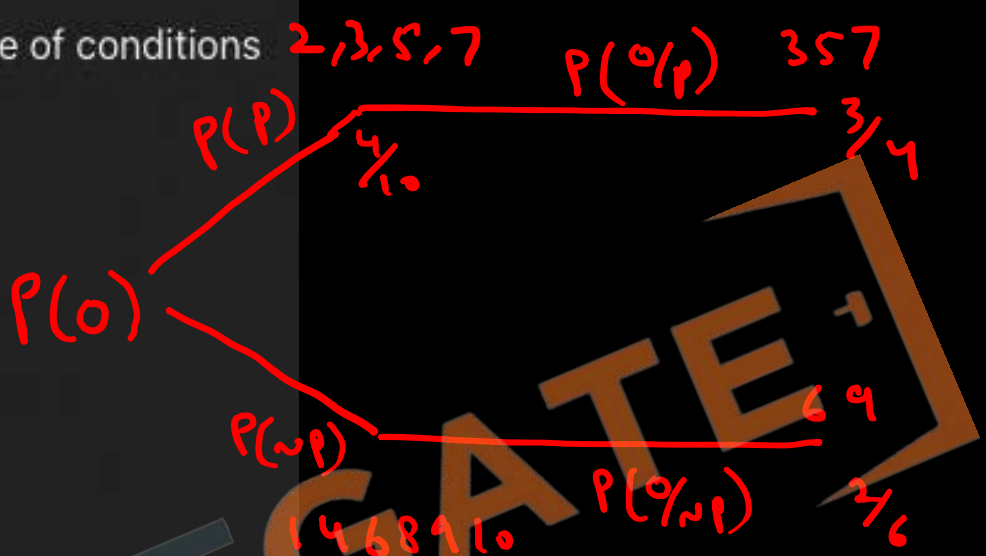
Bayes' Theorem describes the probability of an event based on prior knowledge of conditions related to the event. It is expressed mathematically as:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

1 2 3 4 5 6 7 8 9 10

Where:

- $P(A|B)$ is the probability of event A occurring given that B is true.
- $P(B|A)$ is the probability of event B occurring given that A is true.
- $P(A)$ is the probability of event A occurring.
- $P(B)$ is the probability of event B occurring.



$P(O) = P(P) \cdot P(O/P) + P(\sim P)(P O/\sim P)$

$= \frac{4}{10} \times \frac{3}{4} + \frac{6}{10} \times \frac{3}{6}$

$= \frac{5}{10} = .5$

$P(P/O) = \frac{P(P) \cdot P(O/P)}{P(O)} = \frac{\frac{4}{10} \times \frac{3}{4}}{.5} = .6$

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Concept learning

- **Concept learning** is the process of inferring a function from labeled training data in supervised learning. It involves identifying patterns or rules that correctly classify instances into predefined categories, using methods like decision trees or neural networks to search through possible hypotheses and select the best one.

KNOWLEDGEGATE

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

- The given table represents a dataset where Tom's enjoyment of his favorite water sports is recorded based on various weather conditions. The goal is to determine under what conditions Tom enjoys water sport

Attributes and Their Values

- **Sky**: Sunny, Rainy
- **AirTemp**: Warm, Cold
- **Humidity**: Normal, High
- **Wind**: Strong
- **Water**: Warm, Cool
- **Forecast**: Same, Change
- **EnjoySport**: Indicates whether Tom enjoys water sports under these conditions. Possible values: Yes, No

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Learning Process

1. **Initialization**:
   - Start with the most specific hypothesis, which initially does not match any instances.
   - Example: $h = (\text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same})$
2. **Generalization**:
   - Update the hypothesis to generalize it as more examples are considered.
   - If the example is positive (EnjoySport = Yes), generalize the hypothesis to match the example.
3. **Updating Hypothesis**:
   - For each positive example, update the hypothesis to the least general generalization that still matches the example.
   - For each negative example, ensure the hypothesis does not match the negative example.

KNOWLEDGEGATE

Step-by-Step Hypothesis Update

1. **Initial Hypothesis:**
   - $h = (\text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same})$
2. **First Example (Sunny, Warm, Normal, Strong, Warm, Same, Yes):**
   - Hypothesis remains the same because it matches the example.
   - $h = (\text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same})$
3. **Second Example (Sunny, Warm, High, Strong, Warm, Same, Yes):**
   - Generalize the hypothesis to match both examples.
   - $h = (\text{Sunny}, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same})$
   - '?' means any value is acceptable for Humidity.
4. **Third Example (Rainy, Cold, High, Strong, Warm, Change, No):**
   - Negative example, so no change to the hypothesis.
5. **Fourth Example (Sunny, Warm, High, Strong, Cool, Change, Yes):**
   - Generalize the hypothesis further to include this example.
   - $h = (\text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ?)$

KNOWLEDGEGATE

Final Hypothesis

The final hypothesis represents the conditions under which Tom enjoys water sports:

- $(\text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ?)$

This means Tom enjoys water sports when:

- The sky is Sunny,
- The air temperature is Warm,
- The wind is Strong,
- Humidity, water temperature, and forecast can be any value.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Advantages**:
  - Generalization: Learns broad rules from specific examples.
  - Interpretability: Produces human-readable rules or models.

- **Disadvantages**:
  - Overfitting: Risk of overly complex models.
  - Requires Labeled Data: Needs a lot of labeled examples.

- **Applications**:
  - Email Spam Detection: Classifies emails as spam or not.
  - Medical Diagnosis: Predicts diseases from patient data.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Bayes Optimal Classifier

- The Bayes Optimal Classifier is a theoretical model in machine learning that makes predictions based on the highest posterior probability. It uses Bayes' theorem to combine prior knowledge with observed data to make the most accurate possible predictions.

KNOWLEDGEGATE

Example: Spam Email Classification

Suppose we want to classify an email as "spam" or "not spam" based on certain features. Let's consider the features "contains the word 'offer'" and "contains the word 'win'".

Given Data:

- **Prior Probabilities:**
  - $P(\text{spam}) = 0.4$
  - $P(\text{not spam}) = 0.6$
- **Likelihoods:**
  - $P(\text{contains 'offer'}|\text{spam}) = 0.8$
  - $P(\text{contains 'win'}|\text{spam}) = 0.7$
  - $P(\text{contains 'offer'}|\text{not spam}) = 0.2$
  - $P(\text{contains 'win'}|\text{not spam}) = 0.1$
- **Feature Observation:**
  - The email contains both the words "offer" and "win".

KNOWLEDGEGATE

Calculate Posterior Probabilities

1. **Calculate the joint likelihoods:**
   - For spam: $P(x|\text{spam}) = P(\text{contains 'offer'}|\text{spam}) \cdot P(\text{contains 'win'}|\text{spam})$

$$P(x|\text{spam}) = 0.8 \cdot 0.7 = 0.56$$

   - For not spam: $P(x|\text{not spam}) = P(\text{contains 'offer'}|\text{not spam}) \cdot P(\text{contains 'win'}|\text{not spam})$

$$P(x|\text{not spam}) = 0.2 \cdot 0.1 = 0.02$$

2. **Calculate the evidence:**
   - $P(x) = P(x|\text{spam}) \cdot P(\text{spam}) + P(x|\text{not spam}) \cdot P(\text{not spam})$

$$P(x) = (0.56 \cdot 0.4) + (0.02 \cdot 0.6) = 0.224 + 0.012 = 0.236$$

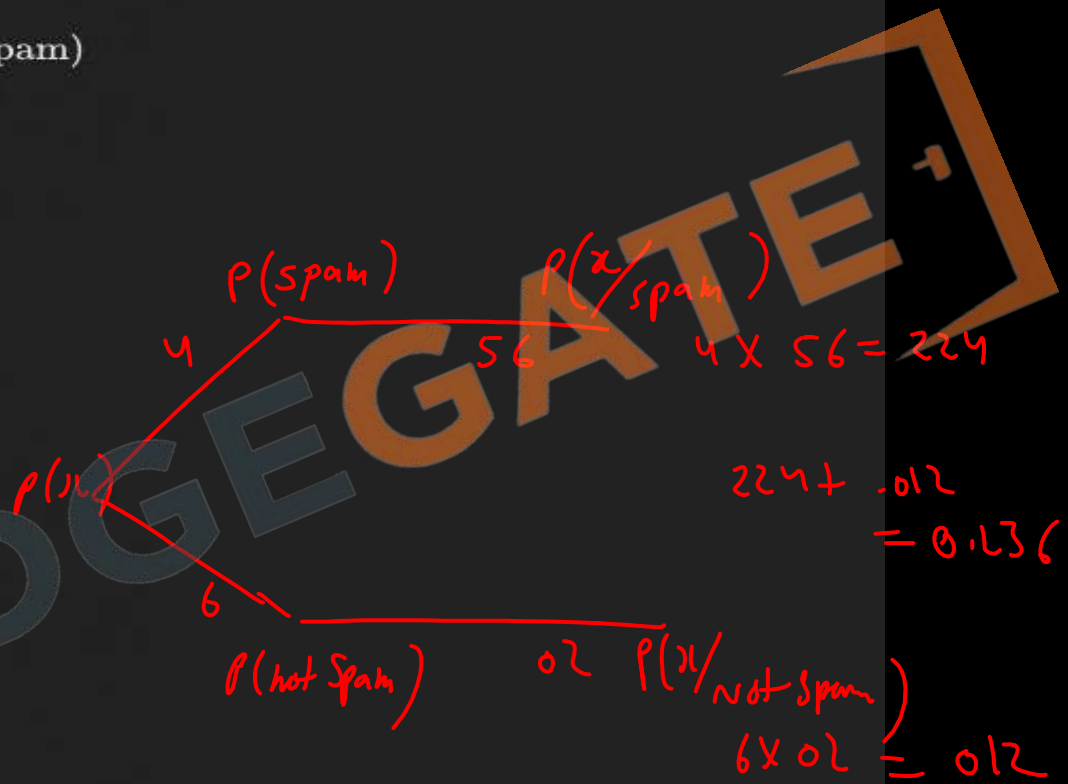
3. **Calculate the posterior probabilities using Bayes' theorem:**
   - For spam:

$$P(\text{spam}|x) = \frac{P(x|\text{spam}) \cdot P(\text{spam})}{P(x)} = \frac{0.56 \cdot 0.4}{0.236} \approx 0.949$$

   - For not spam:

$$P(\text{not spam}|x) = \frac{P(x|\text{not spam}) \cdot P(\text{not spam})}{P(x)} = \frac{0.02 \cdot 0.6}{0.236} \approx 0.051$$

Since $P(\text{spam}|x) \approx 0.949$ and $P(\text{not spam}|x) \approx 0.051$, the Bayes Optimal Classifier will classify this email as "spam" because the posterior probability for "spam" is higher.



KNOWLEDGEGATE

Posterior Probability Calculation:

- Uses Bayes' theorem to calculate the posterior probability of each class given the data.
- Formula: $P(C_i|x) = \frac{P(x|C_i) \cdot P(C_i)}{P(x)}$
  - $P(C_i|x)$: Posterior probability of class C_i given instance x
  - $P(x|C_i)$: Likelihood of instance x given class C_i
  - $P(C_i)$: Prior probability of class C_i
  - $P(x)$: Probability of instance x

Classification:

- Assigns the class label with the highest posterior probability to the instance.
- $C^* = \arg\max_{C_i} P(C_i|x)$

KNOWLEDGEGATE

- **Advantages**
  - **Optimal Predictions**: Provides the most accurate predictions theoretically possible by minimizing the probability of misclassification.
  - **Incorporates All Information**: Uses all available data and prior knowledge.

- **Disadvantages**
  - **Computationally Intensive**: Often impractical to compute for real-world applications due to the need for exact probability distributions.
  - **Requires Accurate Probabilities**: Performance depends on the accuracy of the estimated probabilities.

- **Applications**
  - **Theoretical Benchmark**: Serves as an ideal performance benchmark for other classifiers.
  - **Ensemble Methods**: Used in ensemble learning techniques like Bayesian averaging to improve classification performance.

KNOWLEDGEGATE

Naive Bayes Classifier

- The Naive Bayes classifier is a simple and effective probabilistic classifier based on Bayes' Theorem. It assumes that the features are conditionally independent given the class label, which is often not true in practice but simplifies the computation significantly.

KNOWLEDGEGATE

Bayes' Theorem for Class "Yes" and "No" with Single and Multiple Features

1. **Bayes' Theorem for Single Feature X and Class "Yes" (Y):**

$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

2. **Bayes' Theorem for Multiple Features $X_1, X_2, \ldots, X_n$ and Class "Yes" (Y):**

$$P(Y|X_1, X_2, \ldots, X_n) = \frac{P(X_1|Y) \cdot P(X_2|Y) \cdot P(X_3|Y) \cdots P(X_n|Y) \cdot P(Y)}{P(X_1) \cdot P(X_2) \cdot P(X_3) \cdots P(X_n)}$$

3. **Bayes' Theorem for Single Feature X and Class "No" (N):**

$$P(N|X) = \frac{P(X|N) \cdot P(N)}{P(X)}$$

4. **Bayes' Theorem for Multiple Features $X_1, X_2, \ldots, X_n$ and Class "No" (N):**

$$P(N|X_1, X_2, \ldots, X_n) = \frac{P(X_1|N) \cdot P(X_2|N) \cdot P(X_3|N) \cdots P(X_n|N) \cdot P(N)}{P(X_1) \cdot P(X_2) \cdot P(X_3) \cdots P(X_n)}$$

KNOWLEDGEGATE

- Overweight Due to Eating Habits or Medical Issues

Person	Eating Habits (Good/Bad)	Medical Issues (Yes/No)	Overweight (Yes/No)
1	Bad	No	Yes
2	Bad	No	No
3	Bad	Yes	Yes
4	Good	No	No
5	Good	Yes	Yes
6	Good	No	No
7	Bad	No	Yes
8	Good	Yes	Yes
9	Bad	No	No
10	Good	Yes	No

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- Overweight Due to Eating Habits or Medical Issues

Person	Eating Habits (Good/Bad)	Medical Issues (Yes/No)	Overweight (Yes/No)
1	Bad	No	Yes
2	Bad	No	No
3	Bad	Yes	Yes
4	Good	No	No
5	Good	Yes	Yes
6	Good	No	No
7	Bad	No	Yes
8	Good	Yes	Yes
9	Bad	No	No
10	Good	Yes	No

Step 1: Prior Probability

- $P(\text{overweight} = \text{yes}) = \frac{5}{10} = 0.5$
- $P(\text{overweight} = \text{no}) = \frac{5}{10} = 0.5$

Step 2: Conditional Probability

	Overweight = Yes	Overweight = No
Eating Habits (Bad)	$\frac{3}{5}$	$\frac{2}{5}$
Eating Habits (Good)	$\frac{2}{5}$	$\frac{3}{5}$
Medical Issues (Yes)	$\frac{2}{5}$	$\frac{1}{5}$
Medical Issues (No)	$\frac{3}{5}$	$\frac{4}{5}$

KNOWLEDGE GATE

Bayes' Theorem for Class "Yes" and "No" with Single and Multiple Features

1. **Bayes' Theorem for Single Feature X and Class "Yes" (Y):**

$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

2. **Bayes' Theorem for Multiple Features $X_1, X_2, \ldots, X_n$ and Class "Yes" (Y):**

$$P(Y|X_1, X_2, \ldots, X_n) = \frac{P(X_1|Y) \cdot P(X_2|Y) \cdot P(X_3|Y) \cdots P(X_n|Y) \cdot P(Y)}{P(X_1) \cdot P(X_2) \cdot P(X_3) \cdots P(X_n)}$$

3. **Bayes' Theorem for Single Feature X and Class "No" (N):**

$$P(N|X) = \frac{P(X|N) \cdot P(N)}{P(X)}$$

4. **Bayes' Theorem for Multiple Features $X_1, X_2, \ldots, X_n$ and Class "No" (N):**

$$P(N|X_1, X_2, \ldots, X_n) = \frac{P(X_1|N) \cdot P(X_2|N) \cdot P(X_3|N) \cdots P(X_n|N) \cdot P(N)}{P(X_1) \cdot P(X_2) \cdot P(X_3) \cdots P(X_n)}$$

$P(OW/BE, ME) = \frac{P(BE/OW)\ P(ME/OW)\ P(OW)}{P(BE)\ P(ME)}$

$P(\sim OW/BE, ME) = \frac{P(BE/\sim OW)\ P(ME/\sim OW)\ P(\sim OW)}{P(BE)\ P(ME)}$

KNOWLEDGE GATE

Step 3: Posterior Probability Calculation

Given a person with bad eating habits and medical issues, we want to calculate:

1. **Probability of being overweight given bad eating habits and medical issues:**

$$P(\text{overweight} = \text{yes} | \text{bad eating habits, medical issues}) = P(\text{bad eating habits} | \text{yes}) \cdot P(\text{medical issues} | \text{yes}) \cdot P(\text{yes})$$

$$P(\text{overweight} = \text{yes} | \text{bad eating habits, medical issues}) = \frac{3}{5} \cdot \frac{2}{5} \cdot 0.5 = 0.3 \cdot 0.4 = 0.12$$

2. **Probability of not being overweight given bad eating habits and medical issues:**

$$P(\text{overweight} = \text{no} | \text{bad eating habits, medical issues}) = P(\text{bad eating habits} | \text{no}) \cdot P(\text{medical issues} | \text{no}) \cdot P(\text{no})$$

$$P(\text{overweight} = \text{no} | \text{bad eating habits, medical issues}) = \frac{2}{5} \cdot \frac{1}{5} \cdot 0.5 = 0.2 \cdot 0.2 = 0.02$$

KNOWLEDGEGATE

Conclusion

Given the person with bad eating habits and medical issues, we compare the posterior probabilities:

- $P(\text{overweight} = \text{yes} | \text{bad eating habits, medical issues}) = 0.12$
- $P(\text{overweight} = \text{no} | \text{bad eating habits, medical issues}) = 0.02$

Since $0.12 > 0.02$, the person is more likely to be overweight given bad eating habits and medical issues.

Bayes' Theorem

Bayes' Theorem for a given class C_i and feature vector x is:

$$P(C_i|x) = \frac{P(x|C_i) \cdot P(C_i)}{P(x)}$$

Naive Bayes Assumption

The naive assumption of feature independence means that the joint probability $P(x|C_i)$ can be decomposed into the product of individual probabilities:

$$P(x|C_i) = \prod_{j=1}^{n} P(x_j|C_i)$$

Thus, the classifier can be written as:

$$P(C_i|x) \propto P(C_i) \cdot \prod_{j=1}^{n} P(x_j|C_i)$$

Classification Rule

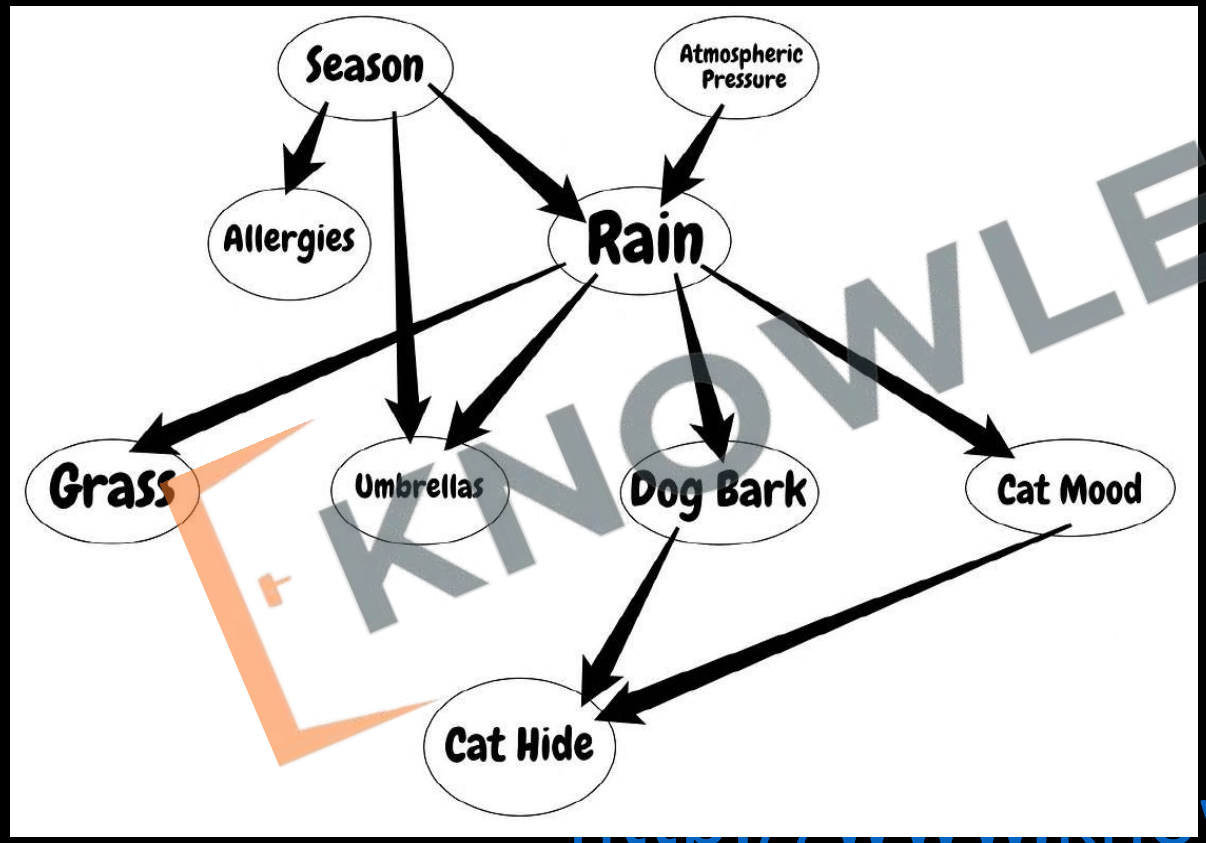
The Naive Bayes classifier assigns a class label $\hat{C}$ to a given instance x by choosing the class with the highest posterior probability:

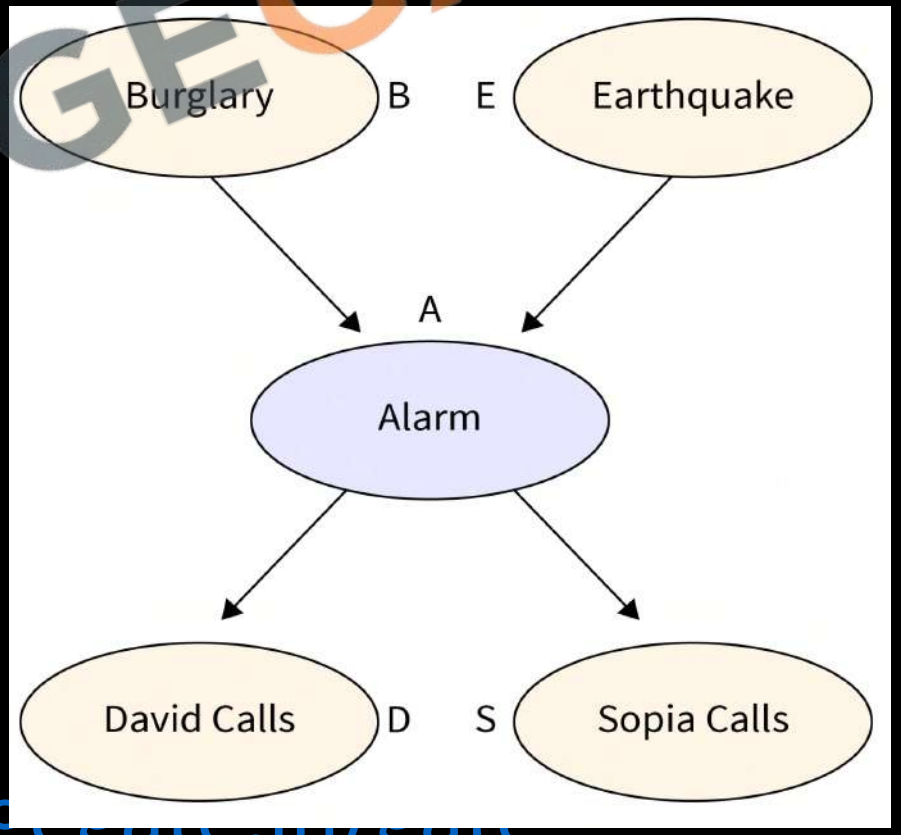
$$\hat{C} = \arg\max_{C_i} P(C_i) \cdot \prod_{j=1}^{n} P(x_j|C_i)$$

KNOWLEDGEGATE

Bayesian Belief Network (BBN)

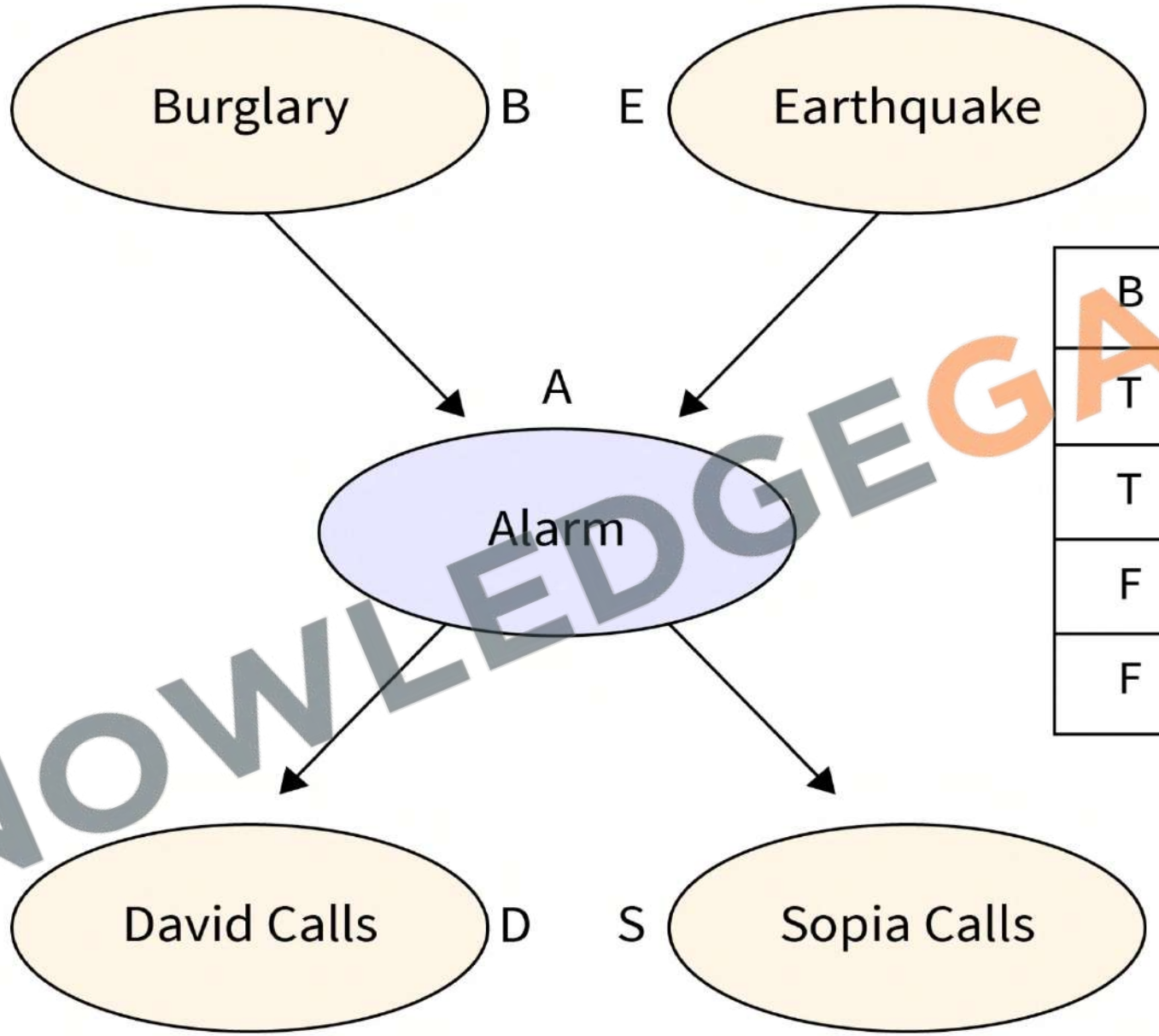
- A Bayesian Belief Network (BBN), also known as a Bayesian Network or Belief Network, is a graphical model that represents a set of variables and their conditional dependencies using a directed acyclic graph (DAG). Each node in the network represents a variable, and each edge represents a conditional dependency between variables.





KNOWLEDGEGATE
http://www.knowledgegate.in/gate

T	0.02
F	0.998



T	0.001
F	0.999

B	E	P(A=T)	P(A=F)
T	T	0.94	0.06
T	F	0.95	0.04
F	T	0.29	0.71
F	F	0.001	0.999

A	P(D=T)	P(D=F)
T	0.91	0.09
F	0.05	0.95

A	P(S=T)	P(S=F)
T	0.75	0.25
F	0.02	0.98

KNOWLEDGEGATE

Key Components:

1. **Nodes:**
   - Represent random variables.
   - Example: Variables like "Smoking", "Cancer", "Coughing".
2. **Edges:**
   - Directed edges represent conditional dependencies.
   - Example: An edge from "Smoking" to "Cancer" indicates that smoking affects the probability of cancer.
3. **Conditional Probability Tables (CPTs):**
   - Each node has a CPT that quantifies the effect of the parent nodes.
   - Example: The CPT for "Cancer" might show the probability of having cancer given whether or not a person smokes.

KNOWLEDGEGATE

- **Advantages**:
  - **Modular Representation**: Each variable is conditionally independent of its non-descendants given its parents.
  - **Efficient Computation**: Supports efficient inference algorithms for calculating probabilities.
  - **Visualization**: Provides a clear visual representation of dependencies among variables.

KNOWLEDGEGATE

- **Disadvantages:**
  - **Complexity**: Can become computationally intensive for large networks.
  - **Data Requirements**: Requires sufficient data to accurately estimate the CPTs.
  - **Design Effort**: Constructing a good network requires expert knowledge and effort.

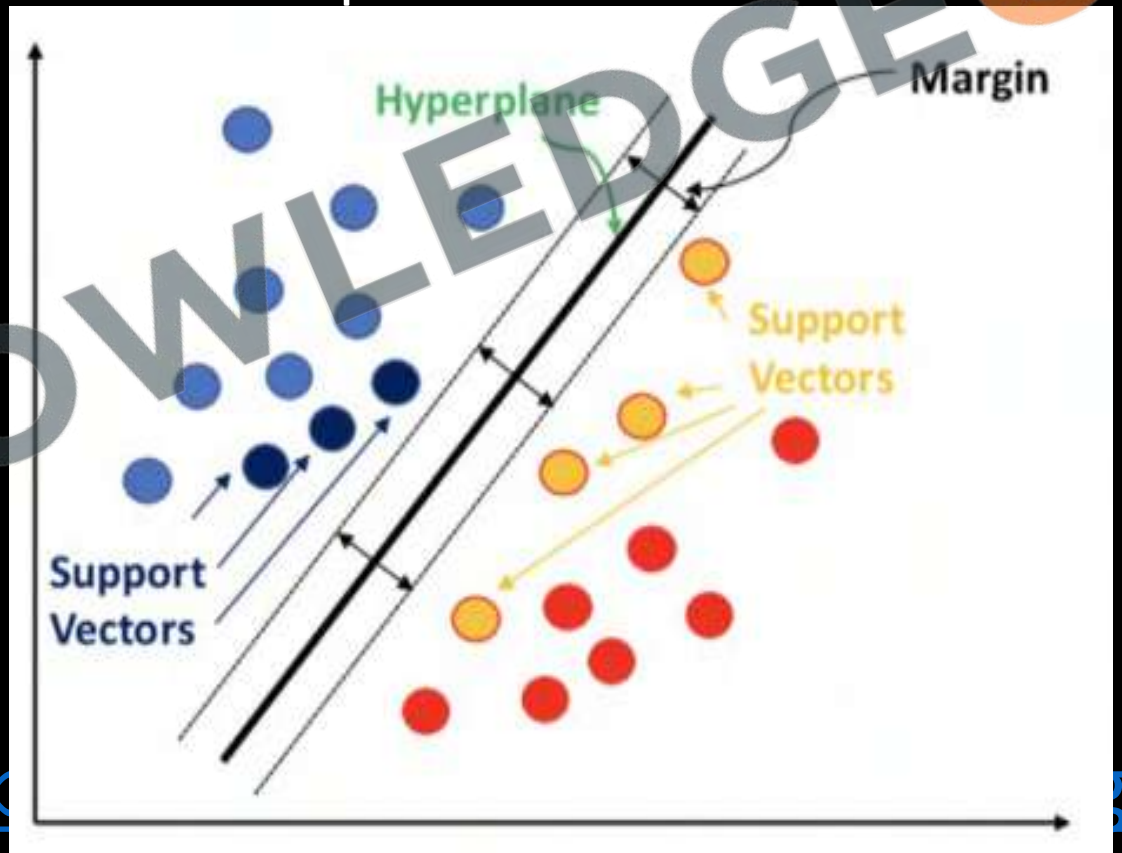
KNOWLEDGEGATE

- **Applications**:
  - **Medical Diagnosis**: Modeling diseases and symptoms to assist in diagnosis.
  - **Risk Assessment**: Evaluating risks in engineering and finance.
  - **Natural Language Processing**: Understanding dependencies between words and sentences.
  - **Decision Support Systems**: Providing recommendations based on probabilistic reasoning.

KNOWLEDGEGATE

Support Vector Machine

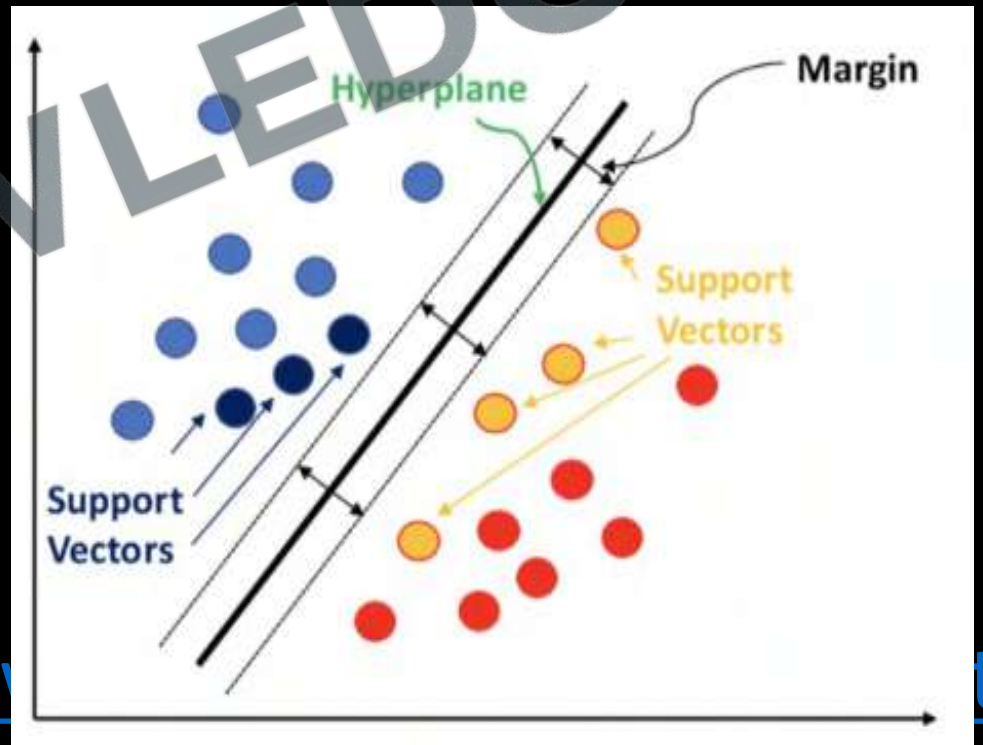
- A Support Vector Machine (SVM) is a powerful machine most commonly used in classification problems.
- SVM constructs a hyperplane or set of hyperplanes in a high-dimensional space, which can be used for classification. The goal is to find the best hyperplane that has the largest distance to the nearest training data points of any class (functional margin), in order to improve the classification performance on unseen data.



KNOWLEDGEGATE

http gate

- **Applications of SVM**:
  - **Text and Hypertext Classification**: For filtering spam and categorizing text based content for news articles.
  - **Image Classification**: Useful in categorizing images into different groups (e.g., animals, cars, fruits).
  - **Handwritten Character Recognition**: Used to recognize letters and digits from handwritten documents.
  - **Biological Sciences**: Applied in protein classification and cancer classification based on gene expression data.



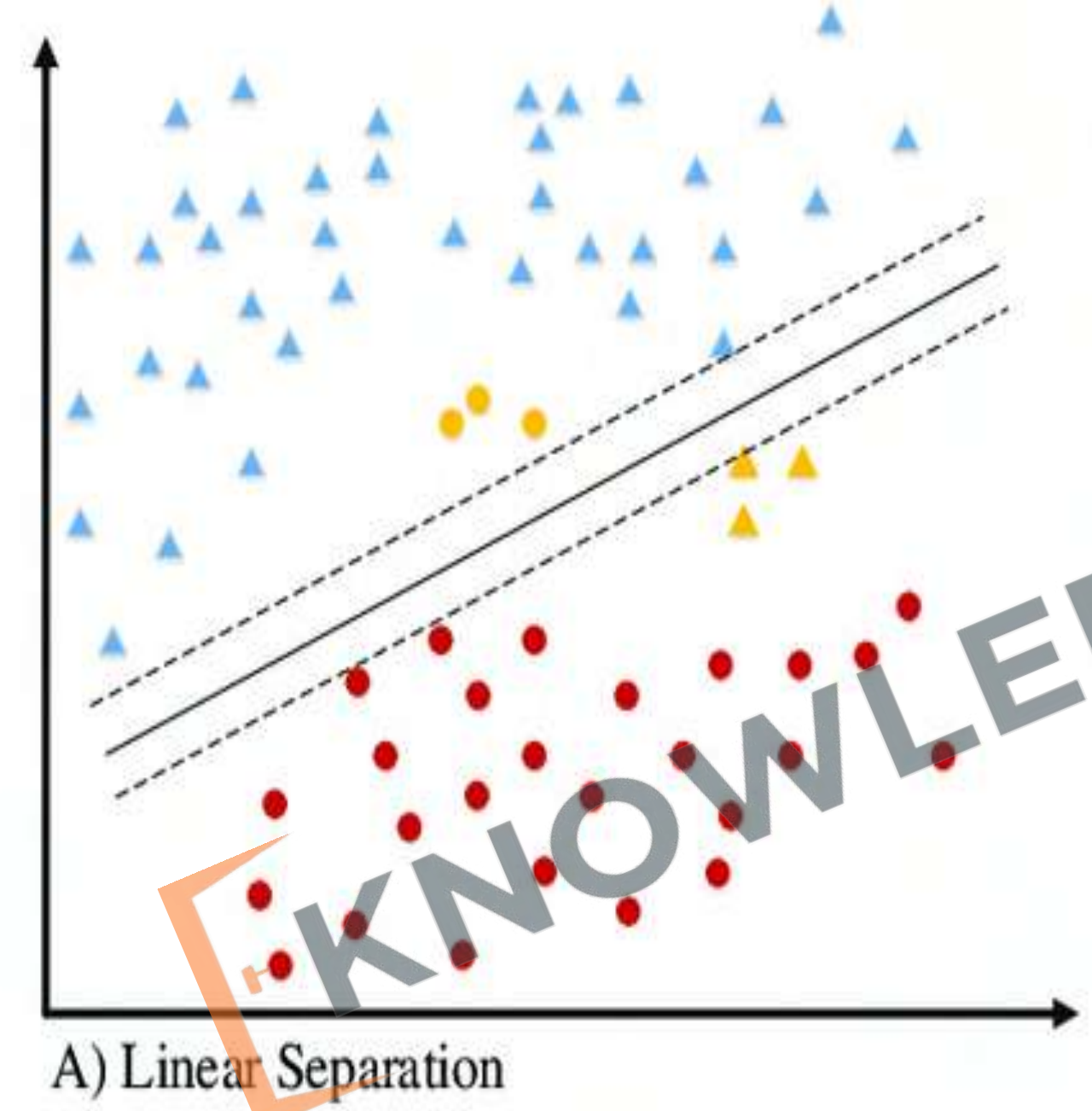
KNOWLEDGEGATE

http://w te

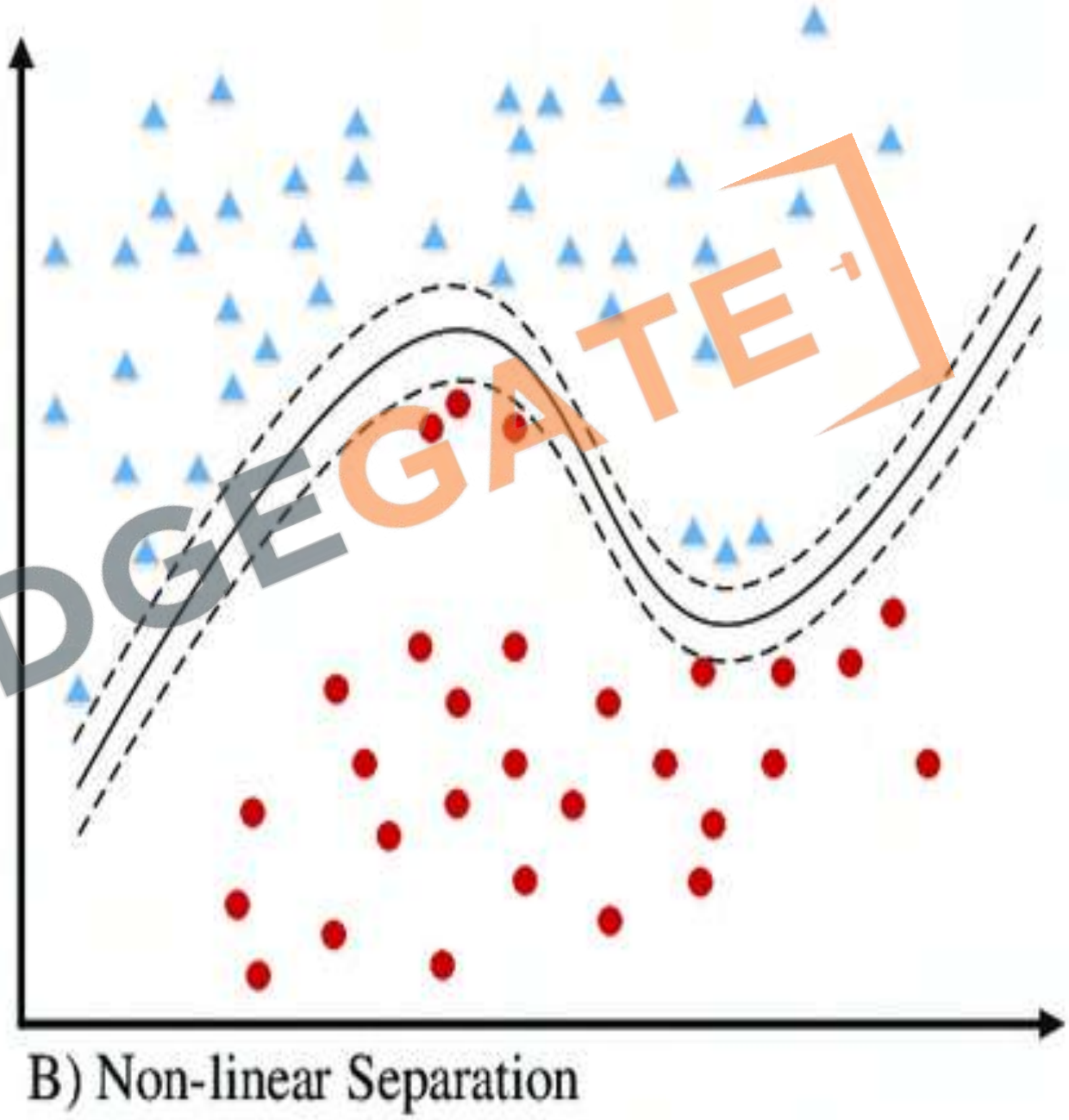
Linear SVM Vs Non-Liner SVM

Conclusion Point	Linear SVM	Non-Linear SVM
Data Suitability	Best for linearly separable data	Suitable for complex, non-linearly separable data
Decision Boundary	Straight line or hyperplane	Non-linear boundary using kernel functions
Training Complexity	Simpler and faster	More computationally intensive
Kernel Usage	Not required	Requires kernel functions (e.g., RBF, polynomial)
Flexibility and Applications	Less flexible, used for simpler problems	Highly flexible, used for complex data patterns

KNOWLEDGE GATE



A) Linear Separation



B) Non-linear Separation

KNOWLEDGEGATE

Polynomial Kernel in SVM

- **Historical Context:** The polynomial kernel has been a fundamental part of kernel methods in machine learning since the 1990s. Introduced as a way to handle non-linearly separable data, it has helped extend the applicability of Support Vector Machines (SVMs) to more complex problems. Initially popularized by researchers like Vladimir Vapnik, the polynomial kernel has been used in various domains including image recognition, bioinformatics, and text classification.

KNOWLEDGEGATE

Mathematical Representation:

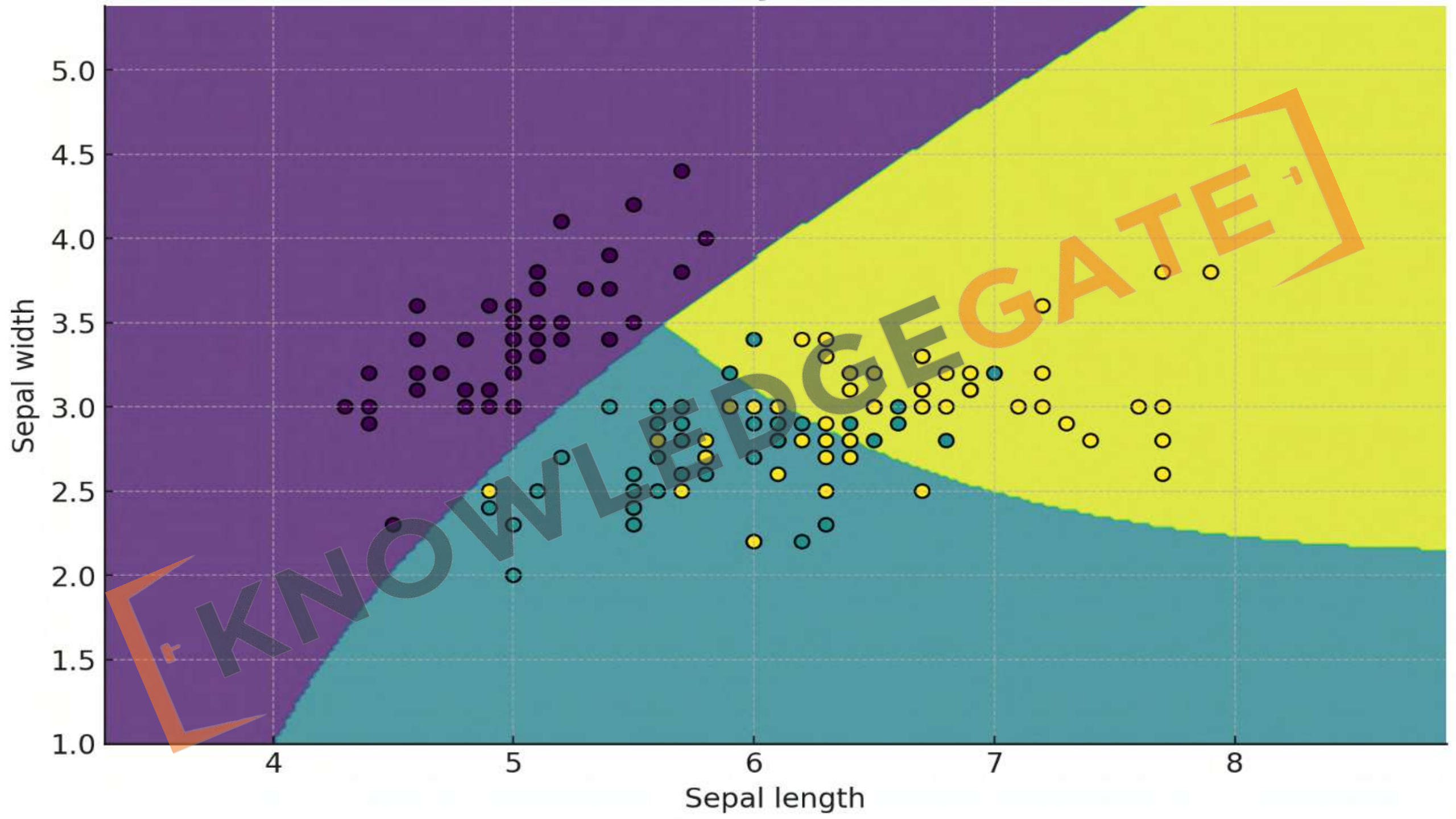
$$K(x, y) = (\gamma x^T y + r)^d$$

- x and y are input vectors.
- γ (gamma) is a scale factor, typically a positive value.
- r is a free parameter, generally a constant term.
- d is the degree of the polynomial.

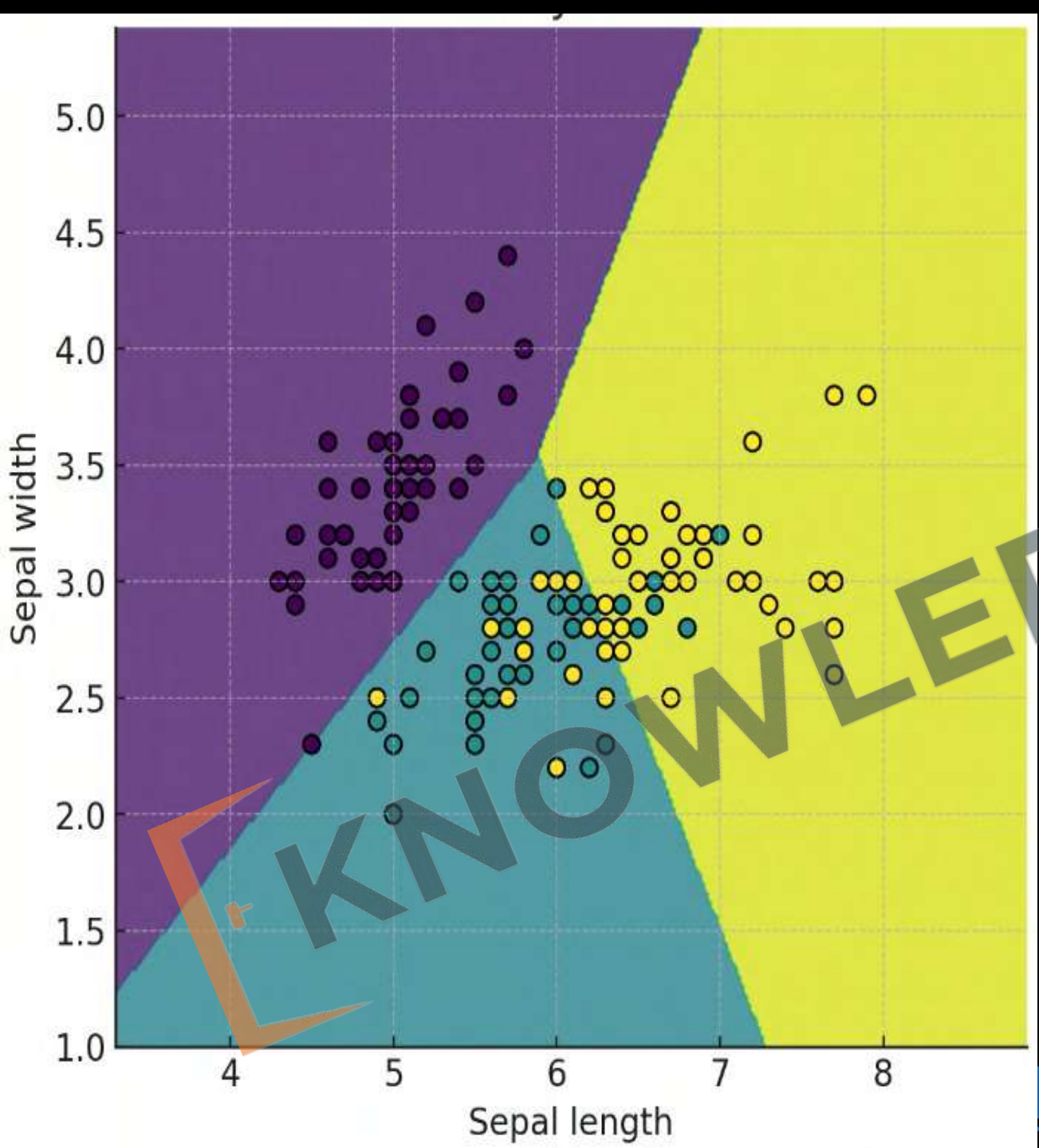
Parameters:

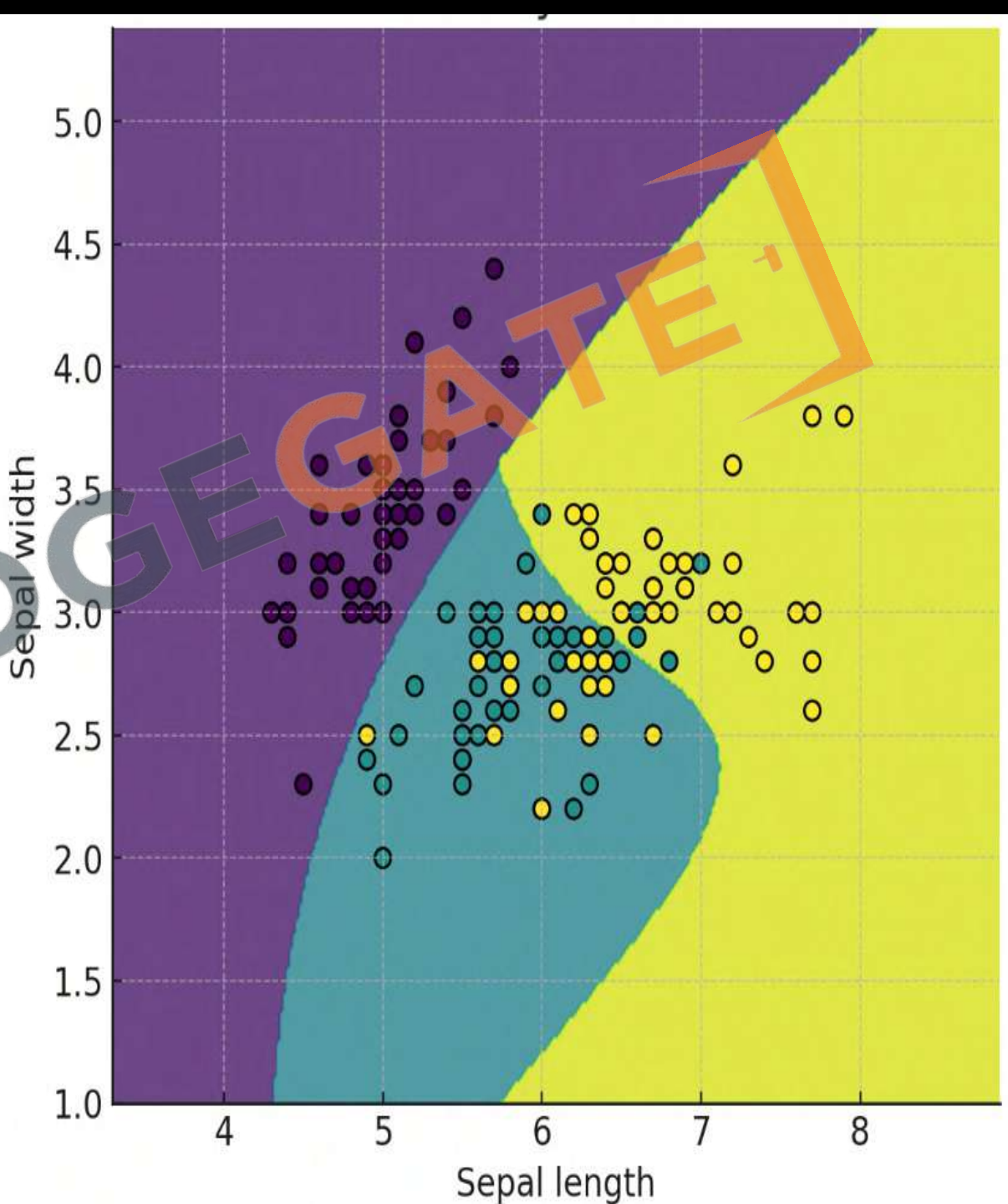
- **Degree (d)**: Determines the flexibility of the decision boundary. Higher degrees lead to more complex boundaries.
- **Gamma (γ)**: Controls the influence of a single training example. Lower values mean 'far' and higher values mean 'close'.
- **Coefficient (r)**: Shifts the kernel value by adding a constant.

KNOWLEDGEGATE



KNOWLEDGEGATE





KNOWLEDGEGATE

Gaussian (RBF) Kernel in SVM

- The Gaussian kernel, also known as the Radial Basis Function (RBF) kernel, is widely used in SVM due to its ability to handle non-linear relationships between features. It maps input features into an infinite-dimensional space where a linear separator can be found.

KNOWLEDGEGATE

The Gaussian kernel function is defined as:

$$K(x, y) = \exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right)$$

Where:

- x and y are input vectors.
- $\|x - y\|$ is the Euclidean distance between the two vectors.
- σ (sigma) is a parameter that defines the spread of the kernel (controls the width of the Gaussian).

In practice, the kernel is often written with a parameter γ (gamma) which is inversely proportional to σ^2:

$$K(x, y) = \exp(-\gamma\|x - y\|^2)$$

KNOWLEDGEGATE

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Applications**
  - **Image and text classification**: Effective for data with complex structures.
  - **Anomaly detection**: Widely used in one-class SVM for identifying outliers.

- **Advantages**
  - Capable of modelling complex decision boundaries.
  - Effective in high-dimensional spaces.

- **Disadvantages**
  - Requires careful tuning of the parameter γ\gammaγ.
  - Computationally intensive with large datasets.

KNOWLEDGEGATE

- **(UNIT-3: DECISION TREE LEARNING)** DECISION TREE LEARNING - Decision tree learning algorithm, Inductive bias, Inductive inference with decision trees, Entropy and information theory, Information gain, ID-3 Algorithm, Issues in Decision tree learning. INSTANCE-BASED LEARNING - k-Nearest Neighbour Learning, Locally Weighted Regression, Radial basis function networks, Case-based learning.

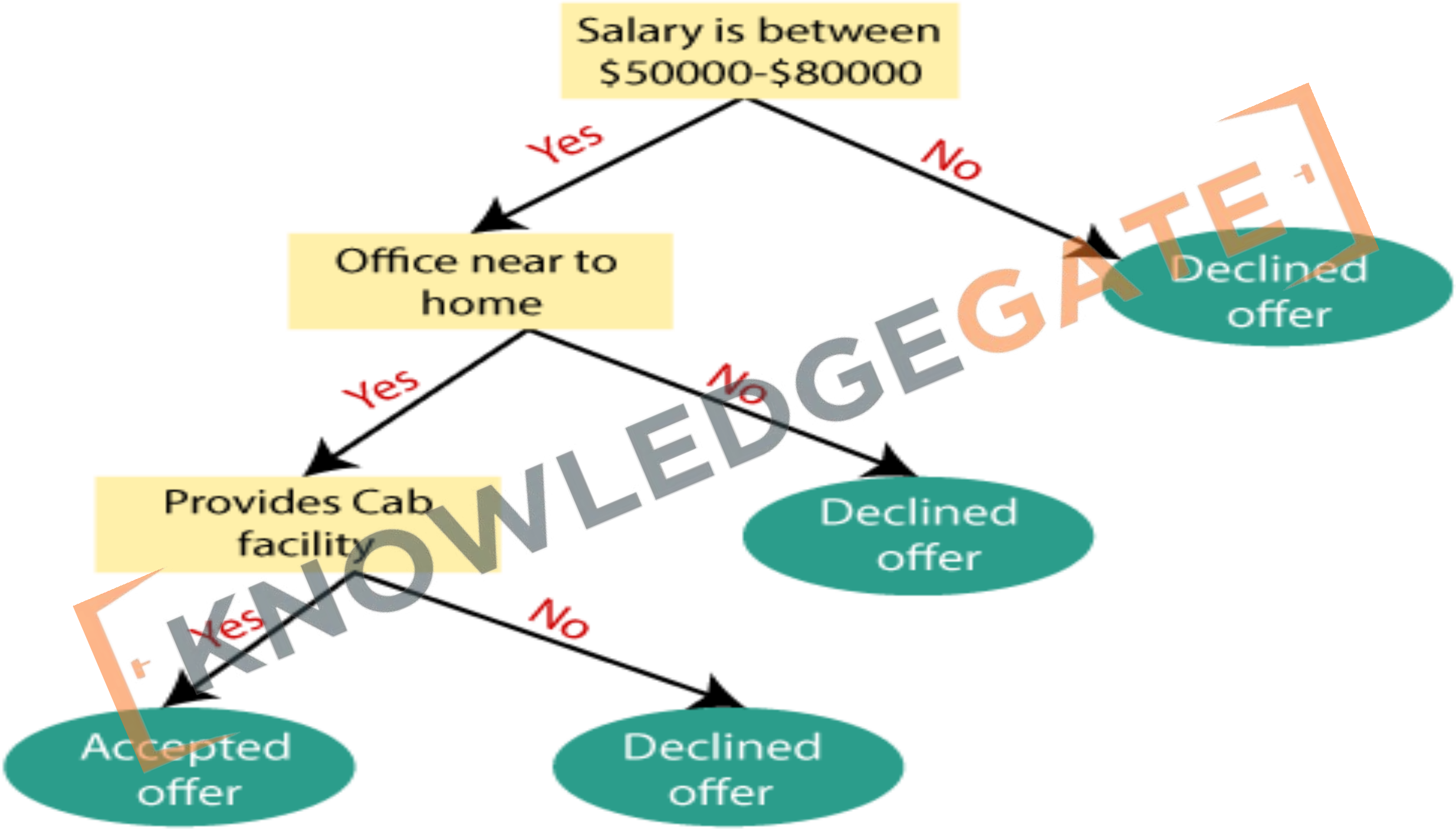
KNOWLEDGEGATE

http://www.knowledgegate.in/gate

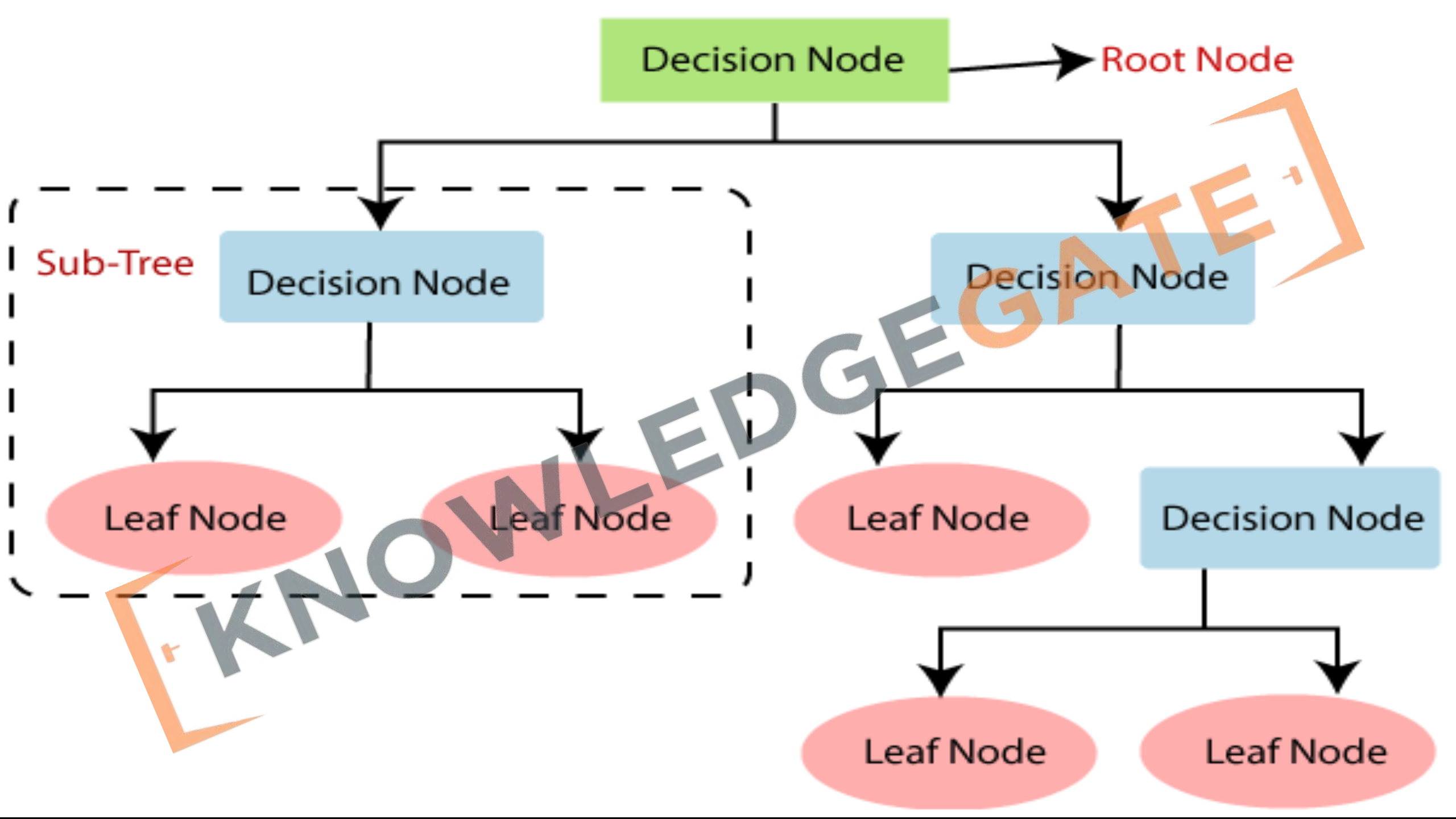
Basic terminology used in Decision Trees

- **Root Node**: This is the starting point of the decision tree. It represents the entire dataset, which is then divided into smaller, more homogeneous groups.
- **Splitting**: This process involves dividing a node into two or more sub-nodes based on a certain condition that maximizes the separation of the classes.
- **Decision Node**: These nodes are where the splits happen. A decision node can lead to further decision nodes or to a leaf node.
- **Leaf (Terminal) Node**: These are the final nodes that do not split further. Each leaf node represents a classification or decision outcome.
- **Pruning**: This technique is used to reduce the size of a decision tree. It involves removing nodes that have little impact on the decision process, to prevent overfitting.
- **Branch/Sub-tree**: A section of the decision tree that represents a part of the whole decision process.
- **Parent and Child Nodes**: In any split, the original node is called the parent node, and the resulting sub-nodes are called child nodes.

KNOWLEDGEGATE



KNOWLEDGEGATE


Decision Node
Root Node
Sub-Tree
Decision Node
Decision Node
Leaf Node
Leaf Node
Leaf Node
Decision Node
Leaf Node
Leaf Node
KNOWLEDGEGATE

Decision trees are a popular tool for decision-making and prediction for several reasons

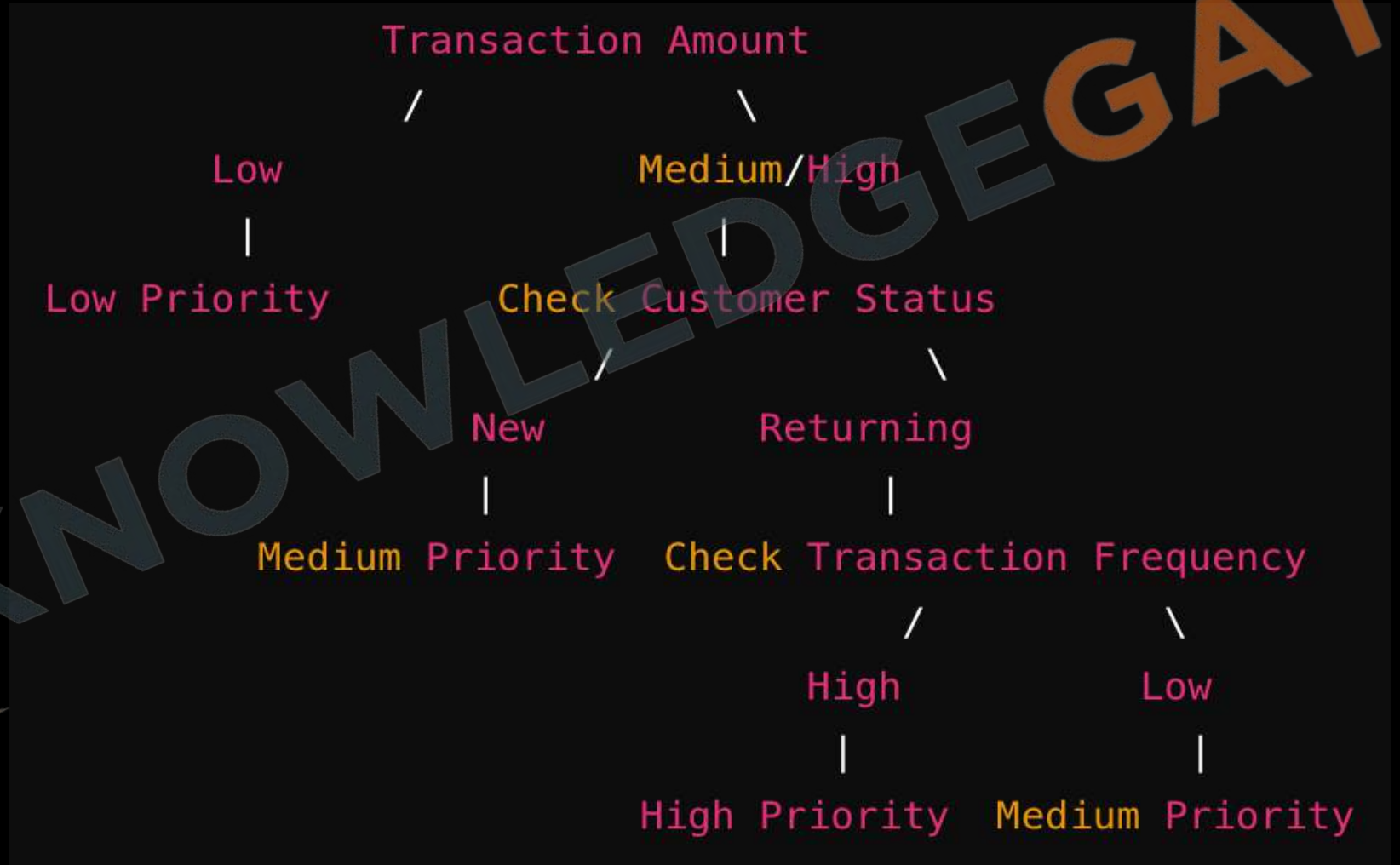
- **Clear Visualization**: Decision trees are easily visualized, allowing users to follow decision-making steps clearly. For example, a bank might use a decision tree to determine creditworthiness based on criteria like income, debt, and credit history.

KNOWLEDGEGATE

- **Low Data Preparation**: They require minimal data preprocessing, such as not needing to normalize data. An example is a marketing team using customer demographic data directly to predict purchasing behaviors.

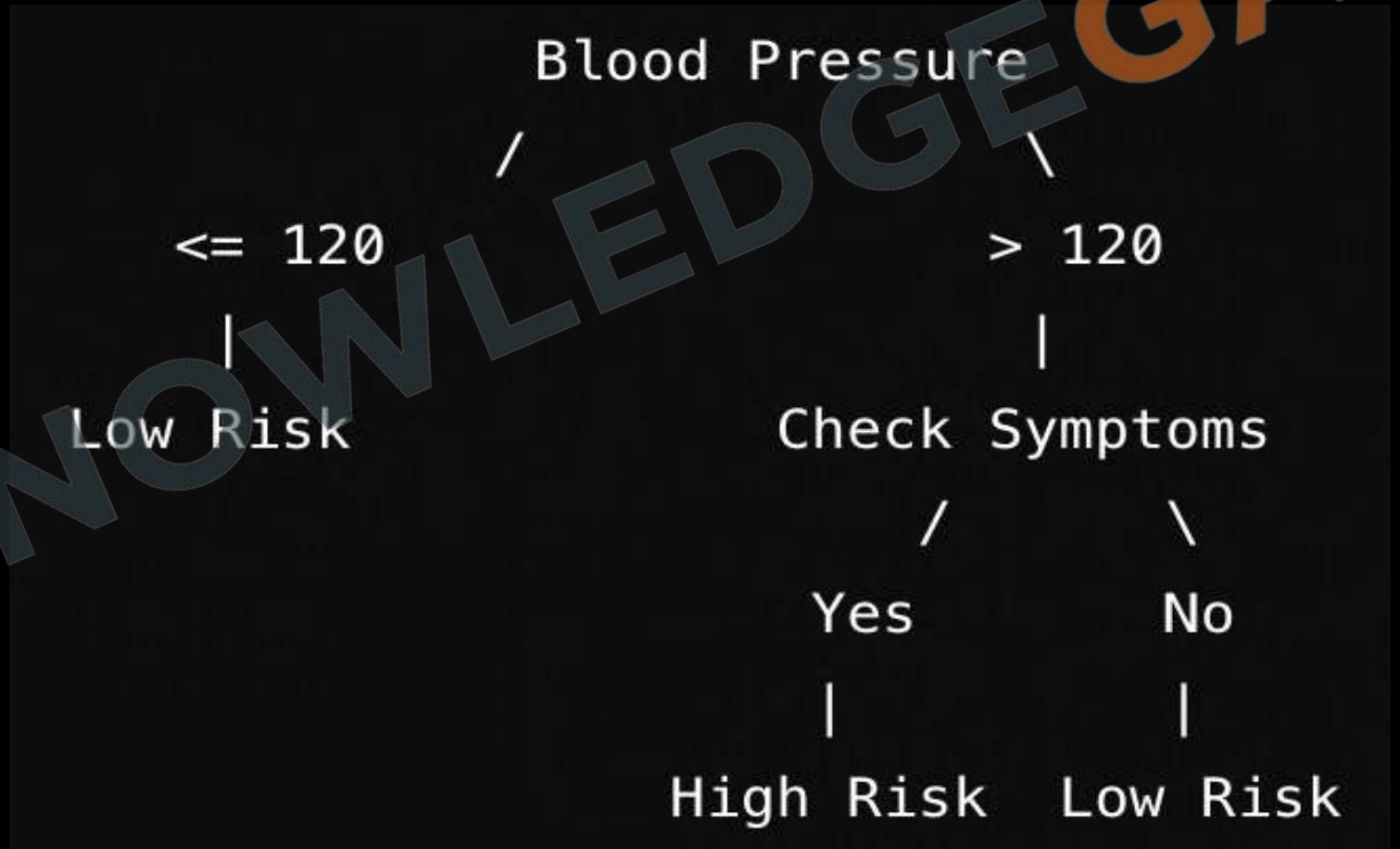
KNOWLEDGEGATE

- **Efficient and Scalable**: The model is efficient, with prediction costs growing logarithmically with data size. For instance, an online retailer could use a decision tree to handle millions of customer transactions efficiently.



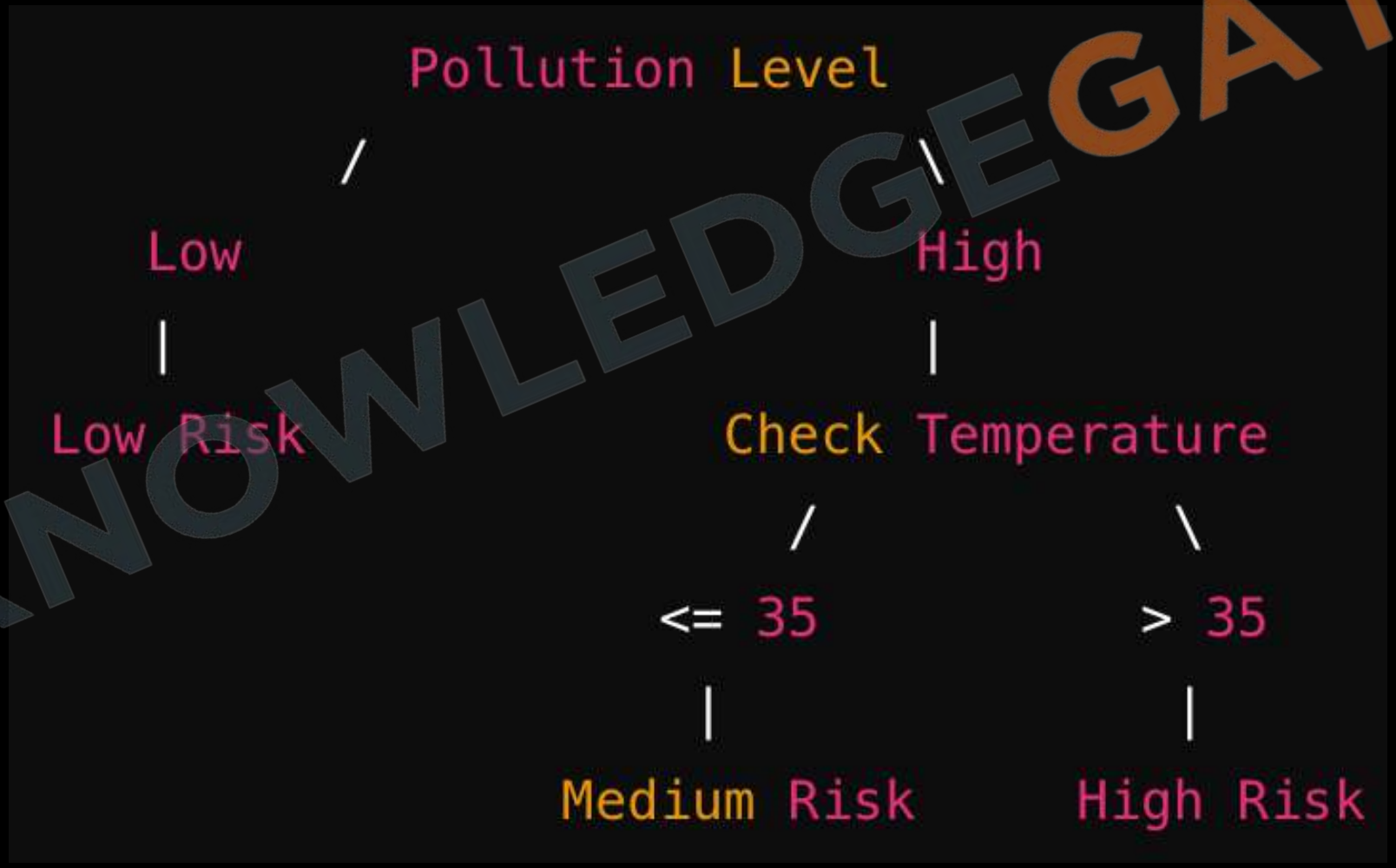
KNOWLEDGEGATE

- **Flexible Data Handling**: Capable of processing both numerical and categorical data, making them suitable for diverse scenarios. For example, a healthcare provider could use decision trees to analyze patient data that includes both numerical (e.g., blood pressure readings) and categorical (e.g., symptom presence) data.



KNOWLEDGE GATE

- **Robust to Assumptions**: Decision trees work well even when data assumptions are not met, such as non-linearity or non-normal distributions. An example is their use in environmental studies, where data often defies typical statistical assumptions.



KNOWLEDGEGATE

Entropy

- Entropy is a measure of the uncertainty or impurity in a dataset. In the context of machine learning, particularly in decision trees, entropy quantifies the amount of randomness or disorder in the target variable. It helps in determining the best way to split the data to reduce this uncertainty and create pure subsets.

KNOWLEDGEGATE

Interpretation:

- **Entropy = 0:** All instances in the dataset belong to a single class (pure set).
- **Entropy = 1:** The instances are equally distributed among all classes (maximum uncertainty).
- **0 < Entropy < 1:** The dataset contains a mix of classes, with varying degrees of uncertainty.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Formula:

For a binary classification problem, entropy $H(S)$ is calculated as:

$$H(S) = -\sum_{i=1}^{c} P_i \log_2 P_i$$

where:

- S is the set of examples (dataset).
- c is the number of classes.
- P_i is the probability of class i in the set S.

For a dataset with classes "Yes" and "No":

- $P(Yes)$ is the proportion of "Yes" instances in the dataset.
- $P(No)$ is the proportion of "No" instances in the dataset.

KNOWLEDGEGATE

Calculate the probabilities:

- $P(Yes) = \frac{9}{14} \approx 0.643$
- $P(No) = \frac{5}{14} \approx 0.357$

Calculate the entropy using the formula:

$$\text{Entropy} = -\left(P(Yes)\log_2 P(Yes) + P(No)\log_2 P(No)\right)$$

Substitute the probabilities into the entropy formula:

$$\text{Entropy} = -\left(0.643\log_2 0.643 + 0.357\log_2 0.357\right)$$

Calculate each term using a calculator:

$$0.643\log_2 0.643 \approx -0.409$$

$$0.357\log_2 0.357 \approx -0.530$$

Sum the values and take the negative:

$$\text{Entropy} = -(-0.409 + -0.530) \approx 0.939$$

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE

1. Calculate Entropy for Each Subset:

We need to calculate the entropy for each subset of the "Outlook" attribute (Sunny, Overcast, Rain).

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Subset: Sunny

- Count: 5
- Yes: 2
- No: 3

Entropy for Sunny:

$$H(\text{Sunny}) = -\left(\frac{2}{5}\log_2\frac{2}{5} + \frac{3}{5}\log_2\frac{3}{5}\right)$$

$$H(\text{Sunny}) = -\left(0.4\log_2 0.4 + 0.6\log_2 0.6\right)$$

$$H(\text{Sunny}) = -(0.4 \times -1.322 + 0.6 \times -0.737)$$

$$H(\text{Sunny}) \approx 0.971$$

KNOWLEDGEGATE

Subset: Overcast

- Count: 4
- Yes: 4
- No: 0

Entropy for Overcast:

$$H(\text{Overcast}) = -\left(\frac{4}{4}\log_2\frac{4}{4} + \frac{0}{4}\log_2\frac{0}{4}\right)$$

$$H(\text{Overcast}) = -(1 \times 0 + 0 \times \log_2 0)$$

$$H(\text{Overcast}) = 0$$

Subset: Rain

- Count: 5
- Yes: 3
- No: 2

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE

Entropy for Rain:

$$H(\text{Rain}) = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right)$$

$$H(\text{Rain}) = -(0.6\log_2 0.6 + 0.4\log_2 0.4)$$

$$H(\text{Rain}) = -(0.6 \times -0.737 + 0.4 \times -1.322)$$

$$H(\text{Rain}) \approx 0.971$$

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE

http://www.knowlec

1. **Outlook**

Outlook	Count	Yes	No
Sunny	5	2	3
Overcast	4	4	0
Rain	5	3	2

$$H(Sunny) = -\left(\frac{2}{5}\log_2\frac{2}{5} + \frac{3}{5}\log_2\frac{3}{5}\right) \approx 0.971$$
$$H(Overcast) = -\left(\frac{4}{4}\log_2\frac{4}{4} + \frac{0}{4}\log_2\frac{0}{4}\right) = 0$$
$$H(Rain) = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right) \approx 0.971$$

$$H(Outlook) = \frac{5}{14}H(Sunny) + \frac{4}{14}H(Overcast) + \frac{5}{14}H(Rain)$$
$$H(Outlook) = \frac{5}{14} \times 0.971 + \frac{4}{14} \times 0 + \frac{5}{14} \times 0.971$$
$$H(Outlook) \approx 0.693$$

$$\text{Gain}(S, \text{Outlook}) = H(S) - H(Outlook)$$
$$\text{Gain}(S, \text{Outlook}) \approx 0.939 - 0.693 = 0.246$$

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE

Information Gain:

Entropy is used to calculate information gain, which measures the reduction in uncertainty about the target variable after splitting the data based on an attribute. The attribute with the highest information gain is chosen to split the dataset at each node in the tree.

$$\text{Information Gain}(S, \text{Attribute}) = H(S) - \sum_{v \in \text{Values}} \frac{|S_v|}{|S|} H(S_v)$$

where:

- S is the original dataset.
- S_v is the subset of S for which the attribute has value v.
- $H(S_v)$ is the entropy of the subset S_v.
- $\frac{|S_v|}{|S|}$ is the proportion of S represented by S_v.

By repeatedly choosing the attribute with the highest information gain, decision trees iteratively partition the data, reducing entropy and increasing purity in the resulting subsets, leading to the final decision tree.

KNOWLEDGEGATE

2. Temperature

Temp	Count	Yes	No
Hot	4	2	2
Mild	6	4	2
Cool	4	3	1

$$H(Hot) = -\left(\frac{2}{4}\log_2\frac{2}{4} + \frac{2}{4}\log_2\frac{2}{4}\right) = 1$$
$$H(Mild) = -\left(\frac{4}{6}\log_2\frac{4}{6} + \frac{2}{6}\log_2\frac{2}{6}\right) \approx 0.918$$
$$H(Cool) = -\left(\frac{3}{4}\log_2\frac{3}{4} + \frac{1}{4}\log_2\frac{1}{4}\right) \approx 0.811$$

$$H(Temp) = \frac{4}{14}H(Hot) + \frac{6}{14}H(Mild) + \frac{4}{14}H(Cool)$$
$$H(Temp) = \frac{4}{14} \times 1 + \frac{6}{14} \times 0.918 + \frac{4}{14} \times 0.811$$
$$H(Temp) \approx 0.911$$

$$\text{Gain}(S, \text{Temp}) = H(S) - H(Temp)$$
$$\text{Gain}(S, \text{Temp}) \approx 0.939 - 0.911 = 0.028$$

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE

3. Humidity

Humidity	Count	Yes	No
High	7	3	4
Normal	7	6	1

$$H(High) = -\left(\frac{3}{7}\log_2\frac{3}{7} + \frac{4}{7}\log_2\frac{4}{7}\right) \approx 0.985$$
$$H(Normal) = -\left(\frac{6}{7}\log_2\frac{6}{7} + \frac{1}{7}\log_2\frac{1}{7}\right) \approx 0.592$$

$$H(Humidity) = \frac{7}{14}H(High) + \frac{7}{14}H(Normal)$$
$$H(Humidity) = \frac{7}{14} \times 0.985 + \frac{7}{14} \times 0.592$$
$$H(Humidity) \approx 0.789$$

$$\text{Gain}(S, \text{Humidity}) = H(S) - H(Humidity)$$
$$\text{Gain}(S, \text{Humidity}) \approx 0.939 - 0.789 = 0.150$$

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE

4. Wind

Wind	Count	Yes	No
Weak	8	6	2
Strong	6	3	3

$$H(Weak) = -\left(\frac{6}{8}\log_2\frac{6}{8} + \frac{2}{8}\log_2\frac{2}{8}\right) \approx 0.811$$
$$H(Strong) = -\left(\frac{3}{6}\log_2\frac{3}{6} + \frac{3}{6}\log_2\frac{3}{6}\right) = 1$$

$$H(Wind) = \frac{8}{14}H(Weak) + \frac{6}{14}H(Strong)$$
$$H(Wind) = \frac{8}{14} \times 0.811 + \frac{6}{14} \times 1$$
$$H(Wind) \approx 0.892$$

$$\text{Gain}(S, \text{Wind}) = H(S) - H(Wind)$$
$$\text{Gain}(S, \text{Wind}) \approx 0.939 - 0.892 = 0.047$$

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE

Select the Attribute with the Highest Information Gain as the Root Node:

- In this case, "Outlook" has the highest information gain (0.246), so it becomes the root node.

Split the Dataset Based on the Root Node (Outlook):

- Create branches for each value of "Outlook": Sunny, Overcast, and Rain.

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE

http://www.knowledgega

Step 2: Subset Splits

- Sunny:
  - Instances: D1, D2, D8, D9, D11
  - Calculate entropy and information gain for attributes: Temp, Humidity, Wind
  - Highest Gain: Humidity (e.g., $\text{Gain}(Sunny, Humidity) = 0.97$)

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE
http://www.knowledgega

Entropy of "PlayTennis" in Sunny Subset:

- Yes: 2
- No: 3

$$P(Yes) = \frac{2}{5} = 0.4$$
$$P(No) = \frac{3}{5} = 0.6$$

$$H(Sunny) = -(0.4 \log_2 0.4 + 0.6 \log_2 0.6)$$
$$H(Sunny) \approx -(0.4 \times -1.322 + 0.6 \times -0.737)$$
$$H(Sunny) \approx 0.971$$

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE
http://www.knowledgega

Attribute: Temperature

- Hot: 2 instances (No, No)
- Mild: 2 instances (No, Yes)
- Cool: 1 instance (Yes)

Entropy for each subset:

$$H(Hot) = -(1\log_2 1 + 0\log_2 0) = 0$$
$$H(Mild) = -(0.5\log_2 0.5 + 0.5\log_2 0.5) = 1$$
$$H(Cool) = -(1\log_2 1 + 0\log_2 0) = 0$$

Weighted Entropy:

$$H(Temp) = \frac{2}{5} \times 0 + \frac{2}{5} \times 1 + \frac{1}{5} \times 0$$
$$H(Temp) = 0 + 0.4 + 0$$
$$H(Temp) = 0.4$$

Information Gain:

$$\text{Gain}(Sunny, Temp) = H(Sunny) - H(Temp)$$
$$\text{Gain}(Sunny, Temp) = 0.971 - 0.4$$
$$\text{Gain}(Sunny, Temp) = 0.571$$

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE

Attribute: Humidity

- High: 3 instances (No, No, No)
- Normal: 2 instances (Yes, Yes)

Entropy for each subset:

$$H(High) = -(0 \log_2 0 + 1 \log_2 1) = 0$$
$$H(Normal) = -(1 \log_2 1 + 0 \log_2 0) = 0$$

Weighted Entropy:

$$H(Humidity) = \frac{3}{5} \times 0 + \frac{2}{5} \times 0$$
$$H(Humidity) = 0$$

Information Gain:

$$\text{Gain}(Sunny, Humidity) = H(Sunny) - H(Humidity)$$
$$\text{Gain}(Sunny, Humidity) = 0.971 - 0$$
$$\text{Gain}(Sunny, Humidity) = 0.971$$

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGEGATE

Attribute: Wind

- Weak: 3 instances (No, No, Yes)
- Strong: 2 instances (No, Yes)

Entropy for each subset:

$$H(Weak) = -(0.333 \log_2 0.333 + 0.666 \log_2 0.666) \approx 0.918$$

$$H(Strong) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$$

Weighted Entropy:

$$H(Wind) = \frac{3}{5} \times 0.918 + \frac{2}{5} \times 1$$

$$H(Wind) \approx 0.950$$

Information Gain:

$$\text{Gain}(Sunny, Wind) = H(Sunny) - H(Wind)$$

$$\text{Gain}(Sunny, Wind) = 0.971 - 0.950$$

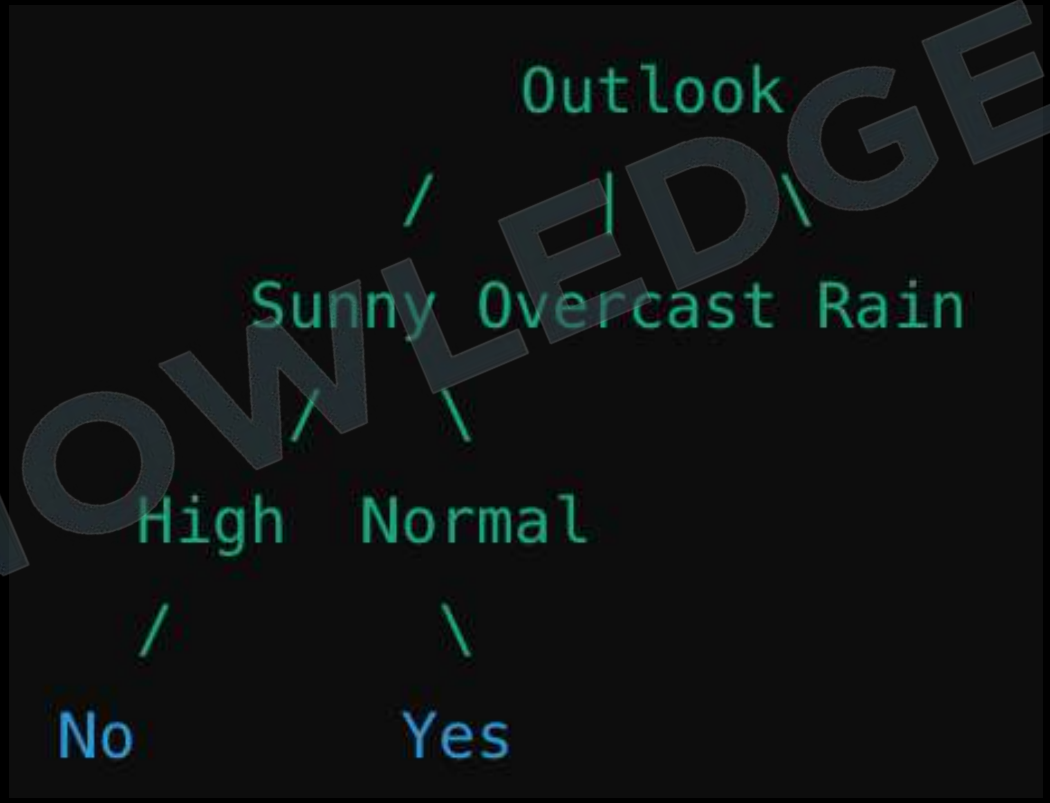
$$\text{Gain}(Sunny, Wind) = 0.021$$

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE

4. Choose the Attribute with the Highest Information Gain

From the calculations, "Humidity" has the highest information gain (0.971). Therefore, we choose "Humidity" to split the "Sunny" subset.



KNOWLEDGEGATE
http://www.knowledgegate.in/gate

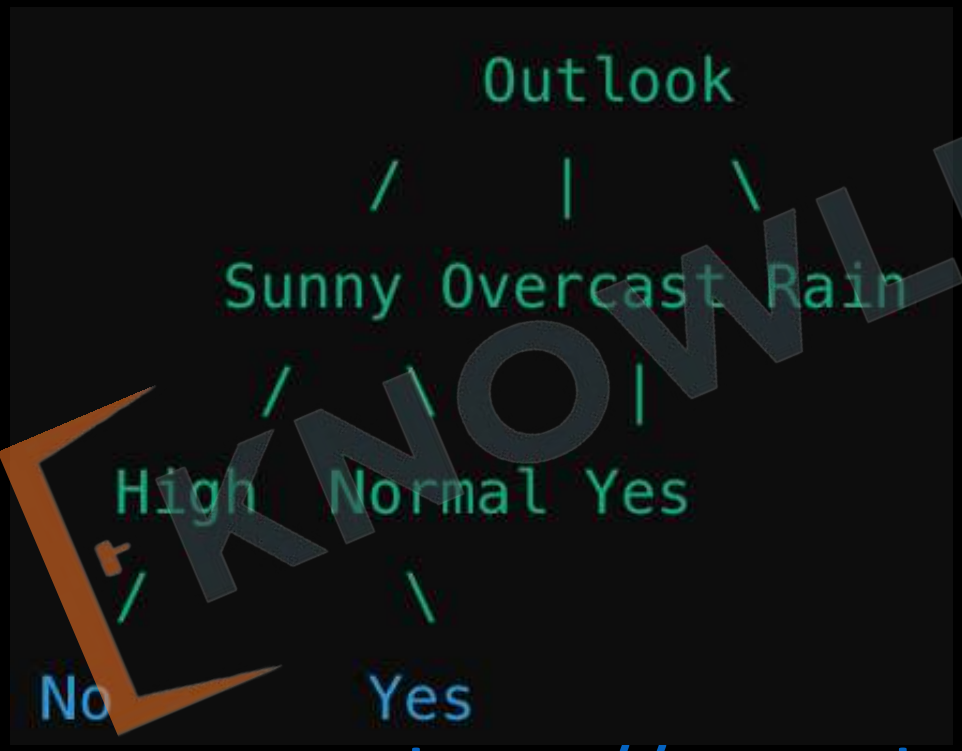
Step 2: Subset Splits

- Sunny:
  - Instances: D1, D2, D8, D9, D11
  - Calculate entropy and information gain for attributes: Temp, Humidity, Wind
  - Highest Gain: Humidity (e.g., $\text{Gain}(Sunny, Humidity) = 0.97$)

Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE
http://www.knowledgega

- **Overcast:**
  - Instances: D3, D7, D12, D13
  - All instances are "Yes", so it becomes a leaf node.

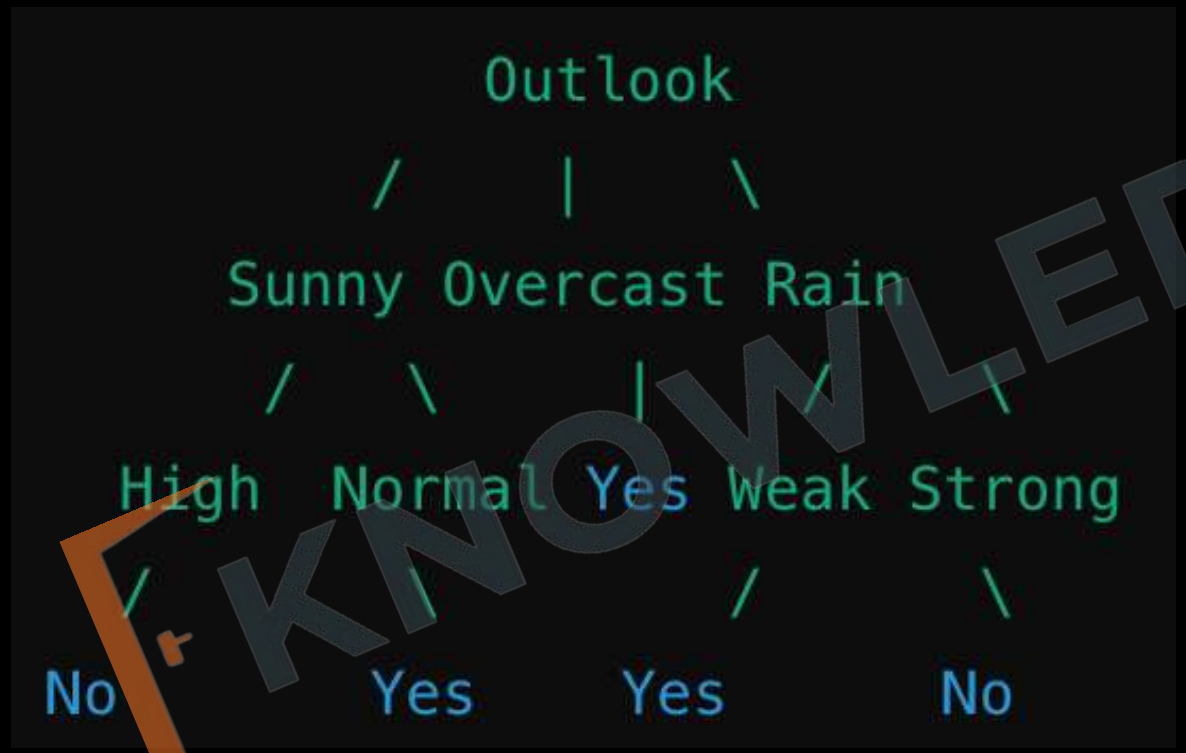


Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE
http://www.knowle

Rain:

- Instances: D4, D5, D6, D10, D14
- Calculate entropy and information gain for attributes: Temp, Humidity, Wind
- Highest Gain: Wind (e.g., $\text{Gain}(Rain, Wind) = 0.97$)



Day	Outlook	Temp	Humidity	Wind	Play Cricket
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

KNOWLEDGE GATE

http://www.knowlec

ID3 (Iterative Dichotomiser 3)

- **Historical Context**: Developed by Ross Quinlan in 1986. One of the earliest algorithms used for decision tree construction.
- **Algorithm**: ID3 uses a top-down, greedy approach to build a decision tree. Splits the dataset on the attribute with the highest information gain (based on entropy).
- **Usage Today**: Primarily of historical interest and educational purposes. Basis for more advanced algorithms like C4.5.
- **Advantages**: Simple and easy to understand. Good for small datasets.
- **Disadvantages**: Prone to overfitting. Handles only categorical attributes. Doesn't handle missing values well.

KNOWLEDGEGATE

C4.5

- **Historical Context**: Developed by Ross Quinlan as an extension of ID3 in 1993. Became one of the most widely used decision tree algorithms.
- **Algorithm**: Uses gain ratio (normalized information gain) to select the best attribute for splitting. Can handle both categorical and continuous attributes. Prunes the tree after it is created to avoid overfitting. Handles missing values by assigning probabilities to each potential value.
- **Usage Today**: Used in various machine learning applications and often cited in academic research. Basis for the popular data mining tool, See5/C5.0.
- **Advantages**: More robust than ID3, handling both categorical and continuous data. Includes tree pruning to reduce overfitting. Handles missing values better than ID3.
- **Disadvantages**: More complex than ID3. Computationally intensive due to gain ratio calculations.

KNOWLEDGEGATE

CART (Classification and Regression Trees)

- **Historical Context**: Developed by Breiman, Friedman, Olshen, and Stone in 1984. Introduced to provide a method for both classification and regression tasks.
- **Algorithm**: Uses Gini impurity (for classification) or variance reduction (for regression) to split nodes. Can handle both categorical and continuous attributes. Performs binary splits, which simplifies the tree but may increase its depth. Includes pruning to avoid overfitting.
- **Usage Today**: Widely used in various machine learning applications, especially in ensemble methods like Random Forests and Gradient Boosting Machines.
- **Advantages**: Versatile, supporting both classification and regression tasks. Handles both categorical and continuous data. Simplifies splits to binary decisions, making it easy to implement. Robust and forms the basis of many ensemble learning techniques.
- **Disadvantages**: Prone to overfitting if not pruned properly. Can create deep trees, which may be difficult to interpret.

KNOWLEDGEGATE

Each algorithm has contributed significantly to the development of decision tree methods in machine learning:

- ID3 laid the foundation for decision tree algorithms but is less commonly used today due to its limitations.
- C4.5 improved on ID3 by handling a wider variety of data types and implementing pruning techniques, making it more robust and versatile.
- CART further expanded the capabilities of decision trees by supporting both classification and regression tasks, simplifying tree splits, and becoming integral to advanced ensemble methods.

KNOWLEDGEGATE

Inductive Bias in Machine Learning

- **Definition**: Inductive bias refers to the set of assumptions a learning algorithm makes to generalize from specific training data to unseen data. It guides the learning process and helps the algorithm to make predictions.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Learning Algorithms with Inductive Bias

- **Decision Trees (e.g., ID3, C4.5, CART)**: Preference for shorter trees (Occam's Razor), preference for attributes with higher information gain or lower Gini impurity. Classification and regression tasks in various domains like finance, healthcare, and marketing.
- **Linear Regression**: Assumes a linear relationship between input features and the target variable. Predictive modeling in economics, biology, and engineering.
- **Neural Networks**: Assumes that complex patterns can be learned through multiple layers of non-linear transformations. Image and speech recognition, natural language processing, and game playing.
- **k-Nearest Neighbors (k-NN)**: Assumes that similar instances have similar outputs (locality-based bias). Recommendation systems, pattern recognition, and anomaly detection.
- **Support Vector Machines (SVM)** : Assumes a maximum margin hyperplane separates different classes. Text classification, image recognition, and bioinformatics.

KNOWLEDGEGATE

Advantages of Inductive Bias

- Guides Generalization: Helps algorithms generalize from limited training data to unseen data.
- Reduces Overfitting: By incorporating domain-specific knowledge, it helps in reducing overfitting.
- Speeds Up Learning: Provides a framework that can speed up the learning process by narrowing the hypothesis space.

Disadvantages of Inductive Bias

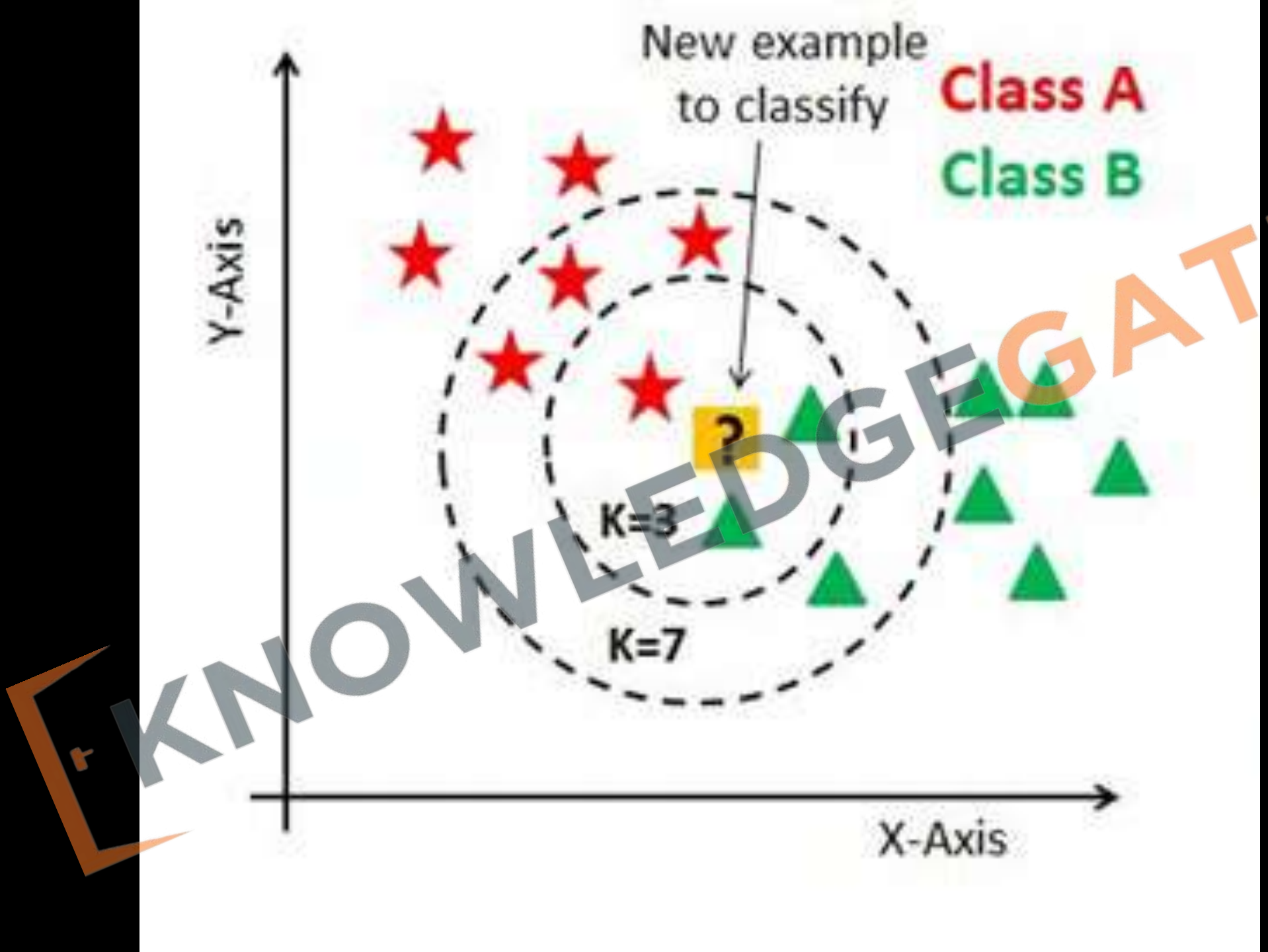
- May Introduce Bias: If the inductive bias is incorrect, it can lead to poor generalization and biased predictions.
- Limited Flexibility: Strong inductive biases can limit the flexibility of the model, making it less adaptable to diverse data.
- Over-reliance on Assumptions: Incorrect assumptions can degrade the performance of the learning algorithm.

KNOWLEDGEGATE

Instance-Based Learning in Machine Learning

- **Historical Context**: Instance-based learning has been foundational in pattern recognition and classification tasks since the early days of AI. The k-NN algorithm, one of the simplest and most effective instance-based methods, was developed in the 1950s.
- **Definition**: Instance-based learning, also known as memory-based learning or lazy learning, is a type of machine learning where the algorithm stores instances of training data and uses them to make predictions on new data. Instead of explicitly creating a model during the training phase, instance-based learning methods delay the generalization process until a query is made during the prediction phase.

KNOWLEDGEGATE



KNOWLEDGEGATE

Key Characteristics:

- **Lazy Learning**: No explicit model is built during training. The algorithm simply stores the training instances. Generalization occurs at the time of prediction.

- **Similarity-Based**: Predictions are made based on the similarity between the new data instance and stored training instances. Common similarity measures include Euclidean distance, Manhattan distance, and cosine similarity.

- **Local Decision Making**: Decisions are made based on the local neighbourhood of the input instance. The algorithm considers only a subset of the training data (e.g., the k-nearest neighbours) to make a prediction.

KNOWLEDGEGATE

Example Algorithms:

- **k-Nearest Neighbors (k-NN)**:
  - **Algorithm**: For a given input instance, the algorithm finds the k training instances closest to the input and assigns the most common label (for classification) or averages the labels (for regression).
  - **Application**: Used in recommendation systems, image recognition, and anomaly detection.

KNOWLEDGEGATE

Advantages of Instance-Based Learning:

- **Simplicity**:
  - Easy to implement and understand.
  - No need for a complex model training phase.
- **Adaptability**:
  - Can adapt quickly to new data without retraining the entire model.
  - Useful for applications where the data distribution changes over time.
- **Performance**:
  - Effective for problems where the decision boundary is irregular and cannot be easily captured by a global model.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Disadvantages of Instance-Based Learning:

- **Computational Cost**:
  - High storage requirements since all training instances are retained.
  - Prediction can be slow for large datasets due to the need to compute distances to all stored instances.
- **Sensitivity to Noise**:
  - Can be sensitive to noisy data, as outliers can affect the prediction.
- **Feature Scaling**:
  - Performance depends on the proper scaling of features, as different scales can distort distance calculations.

KNOWLEDGEGATE

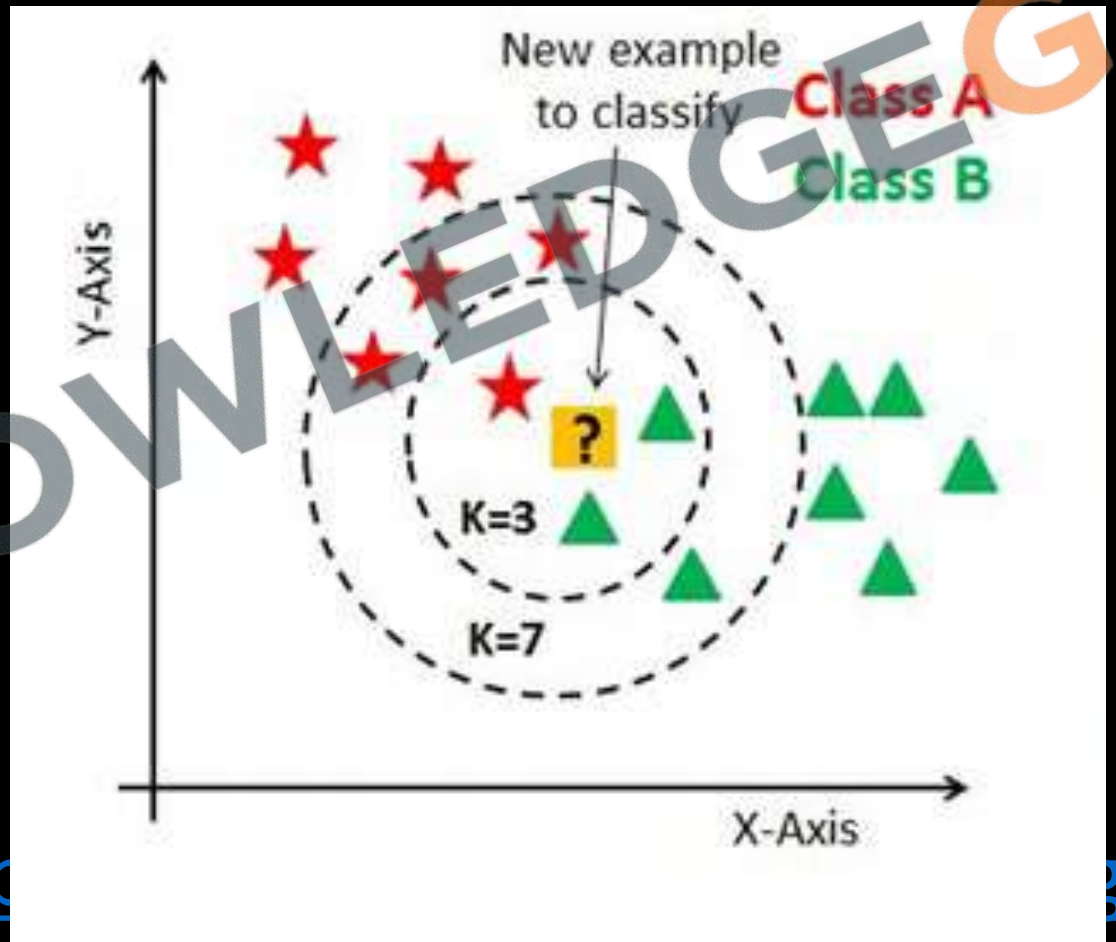
K-Nearest Neighbors (KNN)

- **Historical Context**: Introduced in the 1950s by Fix and Hodges, formalized in the 1960s by Cover and Hart, KNN was initially popular for pattern recognition and statistical estimation due to its simplicity and intuitive approach.

KNOWLEDGEGATE

K-Nearest Neighbors (KNN)

- K-Nearest Neighbors (KNN) is a simple, non-parametric, and lazy learning algorithm used for classification and regression tasks. KNN is an instance-based learning algorithm that classifies a data point based on how its neighbors are classified.



KNOWLEDGEGATE

http gate

- Algorithm Steps:
  - **Select K**: Choose the number of neighbors (K).
  - **Calculate Distance**: Compute the distance between the new data point and all the training points (commonly used distance metrics are Euclidean, Manhattan).
  - **Find Nearest Neighbors**: Identify the K nearest neighbors to the new data point.
  - **Vote (for classification)**: Each neighbor votes for their class, and the class with the most votes is assigned to the data point.
  - **Average (for regression)**: The average value of the K nearest neighbors is taken as the prediction.

Euclidean Distance: $\sqrt{\sum (x_i - y_i)^2}$

Manhattan Distance: $\sum |x_i - y_i|$

KNOWLEDGEGATE

- To classify the new data point (170, 57) using the K-Nearest Neighbors (KNN) algorithm and Euclidean distance, follow these steps

Calculate the Euclidean Distance:

- The Euclidean distance between two points (x_1, y_1) and (x_2, y_2) is given by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Compute Distances for All Points:

- Calculate the distance between the new point (170, 57) and each point in the dataset.

Select the Nearest Neighbors:

- Choose the $K = 3$ nearest neighbors.

Height (cm)	Weight (kg)	Classification
168	50	Underweight
181	63	Normal
175	68	Normal
172	63	Normal
173	64	Normal
173	55	Underweight
170	59	Normal
174	56	Normal
170	57	?

KNOWLEDGEGATE

Distance to (168, 50):

$$d = \sqrt{(170-168)^2 + (57-50)^2} = \sqrt{2^2 + 7^2} = \sqrt{4+49} = \sqrt{53} \approx 7.28$$

Distance to (181, 63):

$$d = \sqrt{(170-181)^2 + (57-63)^2} = \sqrt{(-11)^2 + (-6)^2} = \sqrt{121+36} = \sqrt{157} \approx 12.53$$

Distance to (175, 68):

$$d = \sqrt{(170-175)^2 + (57-68)^2} = \sqrt{(-5)^2 + (-11)^2} = \sqrt{25+121} = \sqrt{146} \approx 12.08$$

Distance to (172, 63):

$$d = \sqrt{(170-172)^2 + (57-63)^2} = \sqrt{(-2)^2 + (-6)^2} = \sqrt{4+36} = \sqrt{40} \approx 6.32$$

Distance to (173, 64):

$$d = \sqrt{(170-173)^2 + (57-64)^2} = \sqrt{(-3)^2 + (-7)^2} = \sqrt{9+49} = \sqrt{58} \approx 7.62$$

Distance to (173, 55):

$$d = \sqrt{(170-173)^2 + (57-55)^2} = \sqrt{(-3)^2 + 2^2} = \sqrt{9+4} = \sqrt{13} \approx 3.61$$

Distance to (170, 59):

$$d = \sqrt{(170-170)^2 + (57-59)^2} = \sqrt{0^2 + (-2)^2} = \sqrt{4} = 2$$

Distance to (174, 56):

$$d = \sqrt{(170-174)^2 + (57-56)^2} = \sqrt{(-4)^2 + 1^2} = \sqrt{16+1} = \sqrt{17} \approx 4.12$$

Height (cm)	Weight (kg)	Classification
168	50	Underweight
181	63	Normal
175	68	Normal
172	63	Normal
173	64	Normal
173	55	Underweight
170	59	Normal
174	56	Normal
170	57	?

KNOWLEDGE GATE
gate.in/gate

Nearest Neighbors:

- The distances are:
  - (170, 59): 2 (Normal)
  - (173, 55): 3.61 (Underweight)
  - (174, 56): 4.12 (Normal)

- Classification:
  - The majority class among the nearest neighbors is "Normal".
  - Thus, the new data point (170, 57) is classified as "Normal".

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Advantages**:
  - Simple and easy to implement.
  - No training phase (lazy learning), which makes it fast for small datasets.

- **Disadvantages**:
  - Computationally expensive for large datasets.
  - Sensitive to the choice of K and the distance metric.
  - Poor performance with imbalanced data.

- **Applications**:
  - Pattern recognition
  - Data mining
  - Image classification
  - Recommendation systems

KNOWLEDGEGATE

- **Modern-Day Use**: KNN is currently applied in image recognition, recommendation systems, anomaly detection, and genomics. It remains favoured for its ease of implementation and versatility, despite computational challenges, which are mitigated by techniques like KD-Trees, PCA, and distributed computing.

- **Current Research and Trends**: Ongoing research focuses on optimizing KNN for specific applications and enhancing efficiency through approximate nearest neighbour searches. Trends include integrating KNN with deep learning models, using it in edge computing and IoT, and improving performance with parallel processing and GPU acceleration.

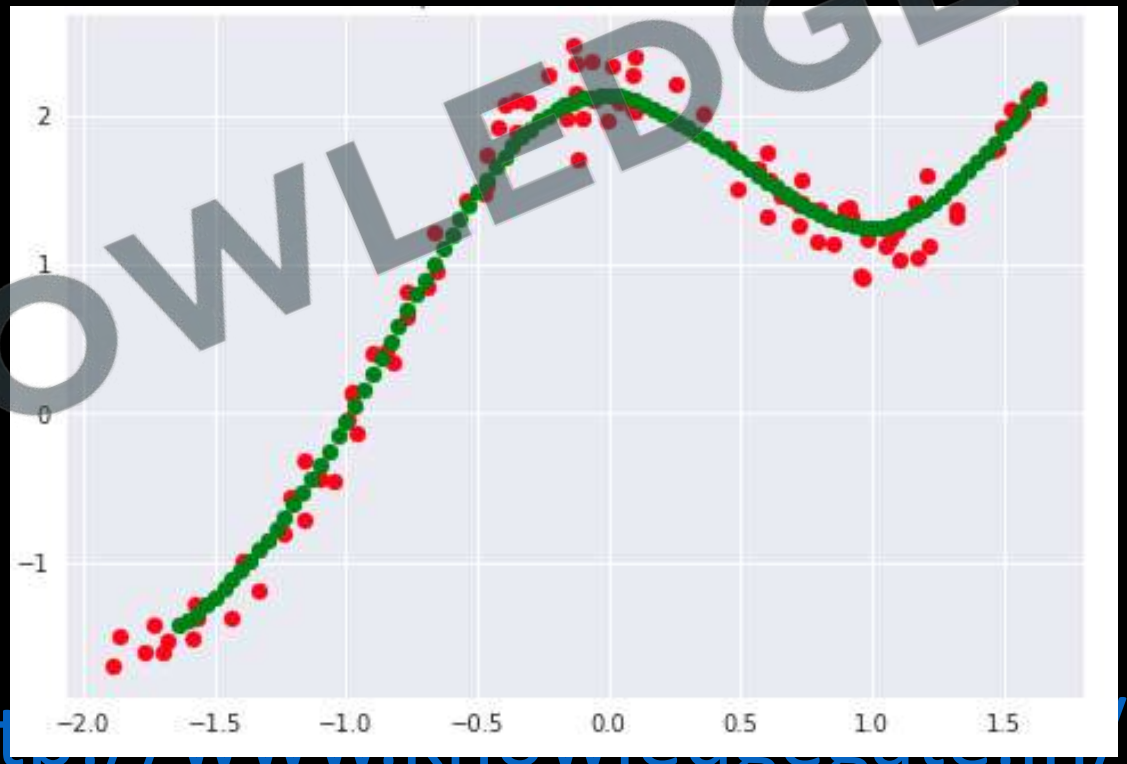
KNOWLEDGEGATE

Locally Weighted Regression (LWR)

- Locally Weighted Regression (LWR), also known as Locally Weighted Scatterplot Smoothing (LOWESS or LOESS), is a non-parametric regression method that fits multiple regressions in localized subsets of the data to produce a smooth curve through points in a scatter plot.

KNOWLEDGEGATE

- **Non-Parametric**: LWR does not assume a fixed form (parametric form) for the relationship between predictors and the response variable. It adapts to the data locally.
- **Localized Fitting**: Regression is performed at each point using a subset of data that is close to the point in question. This local fitting is weighted by the distance of each data point to the point being estimated.
- **Weighting**: Points closer to the point of interest have higher weights. Commonly used weighting functions include the Gaussian kernel and the tricube kernel.



KNOWLEDGE GATE
http://www.knowledgegate.in/gate

- Imagine you are a botanist studying the growth of trees in a forest. You have a small dataset of tree heights at various ages, and you want to predict the height of a tree that is 3 years old.

Age (years)	Height (meters)
1	2
2	3
3	2
4	5
5	4

KNOWLEDGEGATE

Select a Query Point:

- Our query point is $x_q = 3$.

Compute Weights:

- We use a weighting function to assign weights to each data point based on their distance to x_q.
- For simplicity, let's use a Gaussian kernel with bandwidth parameter $\tau = 1$:

$$w_i = \exp\left(-\frac{(x_i - x_q)^2}{2\tau^2}\right)$$

- Compute the weights for each data point:
  - For $x_1 = 1$:

$$w_1 = \exp\left(-\frac{(1-3)^2}{2 \times 1^2}\right) = \exp\left(-\frac{4}{2}\right) = \exp(-2) \approx 0.1353$$

  - For $x_2 = 2$:

$$w_2 = \exp\left(-\frac{(2-3)^2}{2 \times 1^2}\right) = \exp\left(-\frac{1}{2}\right) = \exp(-0.5) \approx 0.6065$$

  - For $x_3 = 3$:

$$w_3 = \exp\left(-\frac{(3-3)^2}{2 \times 1^2}\right) = \exp(0) = 1$$

  - For $x_4 = 4$:

$$w_4 = \exp\left(-\frac{(4-3)^2}{2 \times 1^2}\right) = \exp\left(-\frac{1}{2}\right) = \exp(-0.5) \approx 0.6065$$

  - For $x_5 = 5$:

$$w_5 = \exp\left(-\frac{(5-3)^2}{2 \times 1^2}\right) = \exp\left(-\frac{4}{2}\right) = \exp(-2) \approx 0.1353$$

Age (years)	Height (meters)
1	2
2	3
3	2
4	5
5	4

KNOWLEDGEGATE
ledgegate.in/gate

- The goal of Locally Weighted Regression (LWR) is to fit a linear model to the data points in a localized region around a query point. To achieve this, we assign weights to the data points based on their distance from the query point, and then perform a weighted least squares regression. This ensures that points closer to the query point have more influence on the model than points farther away.

Formulate the Weighted Linear Regression Problem

We need to minimize the weighted sum of squared errors:

$$\sum_i w_i(y_i - (\theta_0 + \theta_1 x_i))^2$$

Using matrix notation:

- W is a diagonal matrix of weights.
- X is the matrix of input features, including a column of ones for the intercept.
- y is the vector of target values.

Matrices and Vectors

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix}, \quad W = \begin{bmatrix} 0.1353 & 0 & 0 & 0 & 0 \\ 0 & 0.6065 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0.6065 & 0 \\ 0 & 0 & 0 & 0 & 0.1353 \end{bmatrix}, \quad y = \begin{bmatrix} 2 \\ 3 \\ 2 \\ 5 \\ 4 \end{bmatrix}$$

Normal Equation for Weighted Linear Regression

$$\theta = (X^T W X)^{-1} X^T W y$$

KNOWLEDGEGATE
/gate

Compute θ

1. Compute $X^T W X$:

$$X^T W X = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} \begin{bmatrix} 0.1353 & 0 & 0 & 0 & 0 \\ 0 & 0.6065 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0.6065 & 0 \\ 0 & 0 & 0 & 0 & 0.1353 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix}$$

2. Compute $X^T W y$:

$$X^T W y = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} \begin{bmatrix} 0.1353 & 0 & 0 & 0 & 0 \\ 0 & 0.6065 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0.6065 & 0 \\ 0 & 0 & 0 & 0 & 0.1353 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 2 \\ 5 \\ 4 \end{bmatrix}$$

3. Solve for θ:

$$\theta = (X^T W X)^{-1} (X^T W y)$$

After computing the above, we get:

$$X^T W X = \begin{bmatrix} 2.0836 & 7.5149 \\ 7.5149 & 29.1717 \end{bmatrix}$$

$$X^T W y = \begin{bmatrix} 12.0591 \\ 46.2269 \end{bmatrix}$$

$$\theta = \begin{bmatrix} 1.5 \\ 0.5 \end{bmatrix}$$

Thus, the fitted local regression line is:

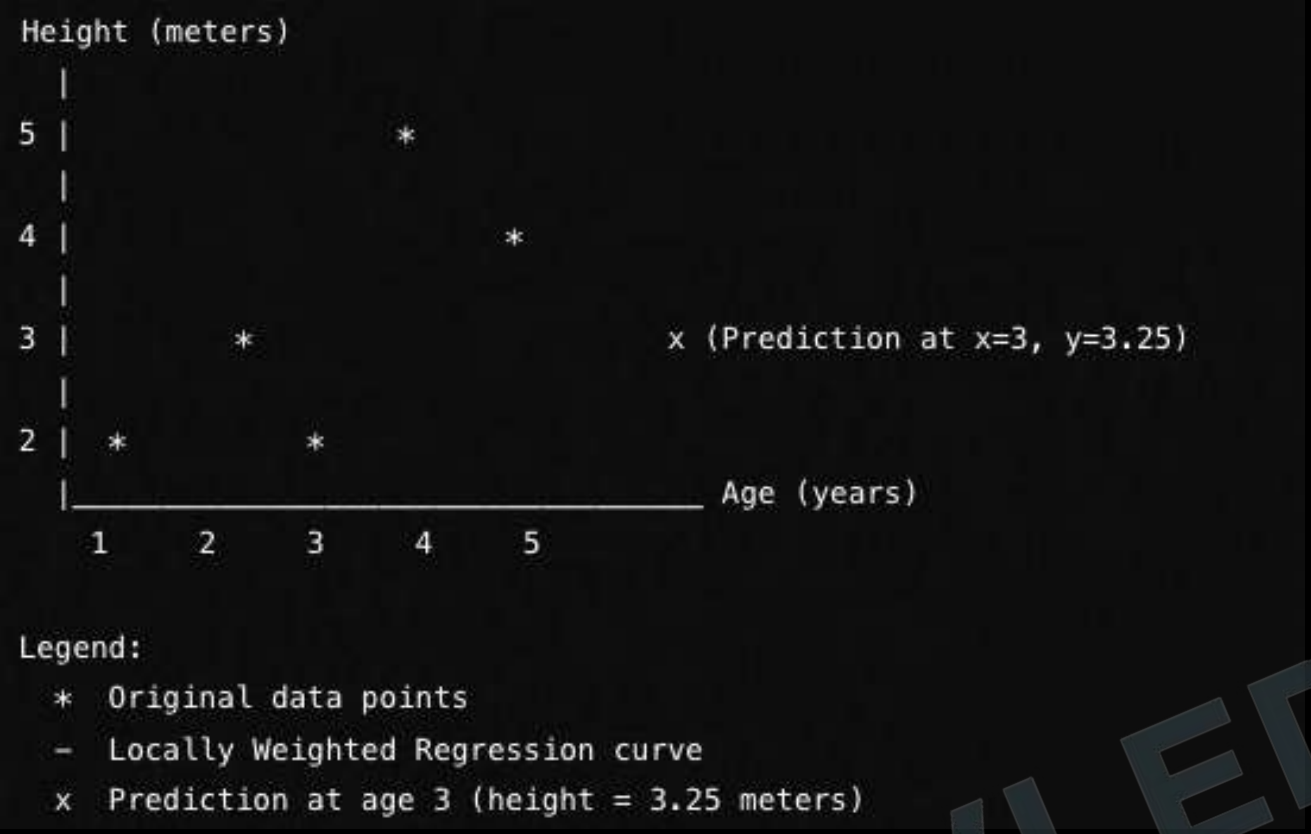
$y = 0.5x + 1.5$

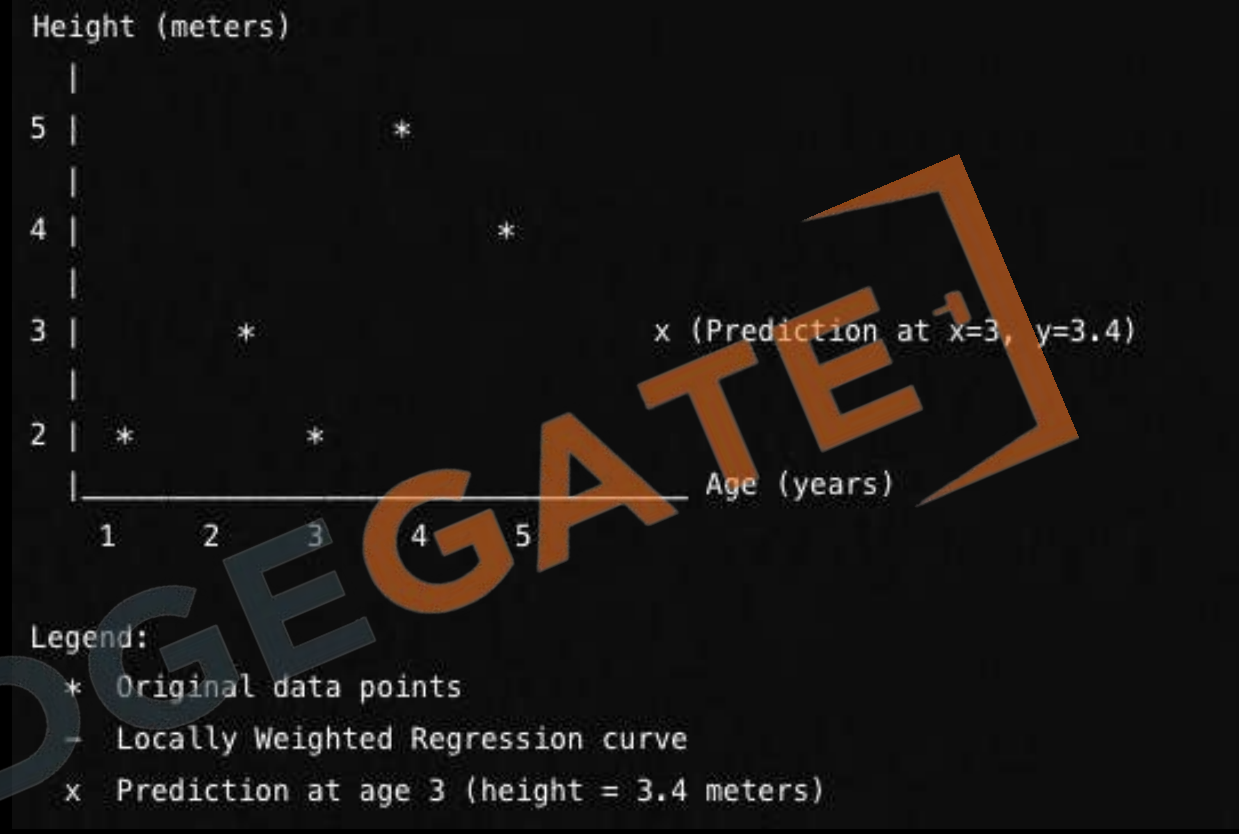
KNOWLEDGEGATE

Legend:

- * Original data points
- - Locally Weighted Regression curve
- x Prediction at age 3 (height = 3 meters)

KNOWLEDGEGATE
Z=1

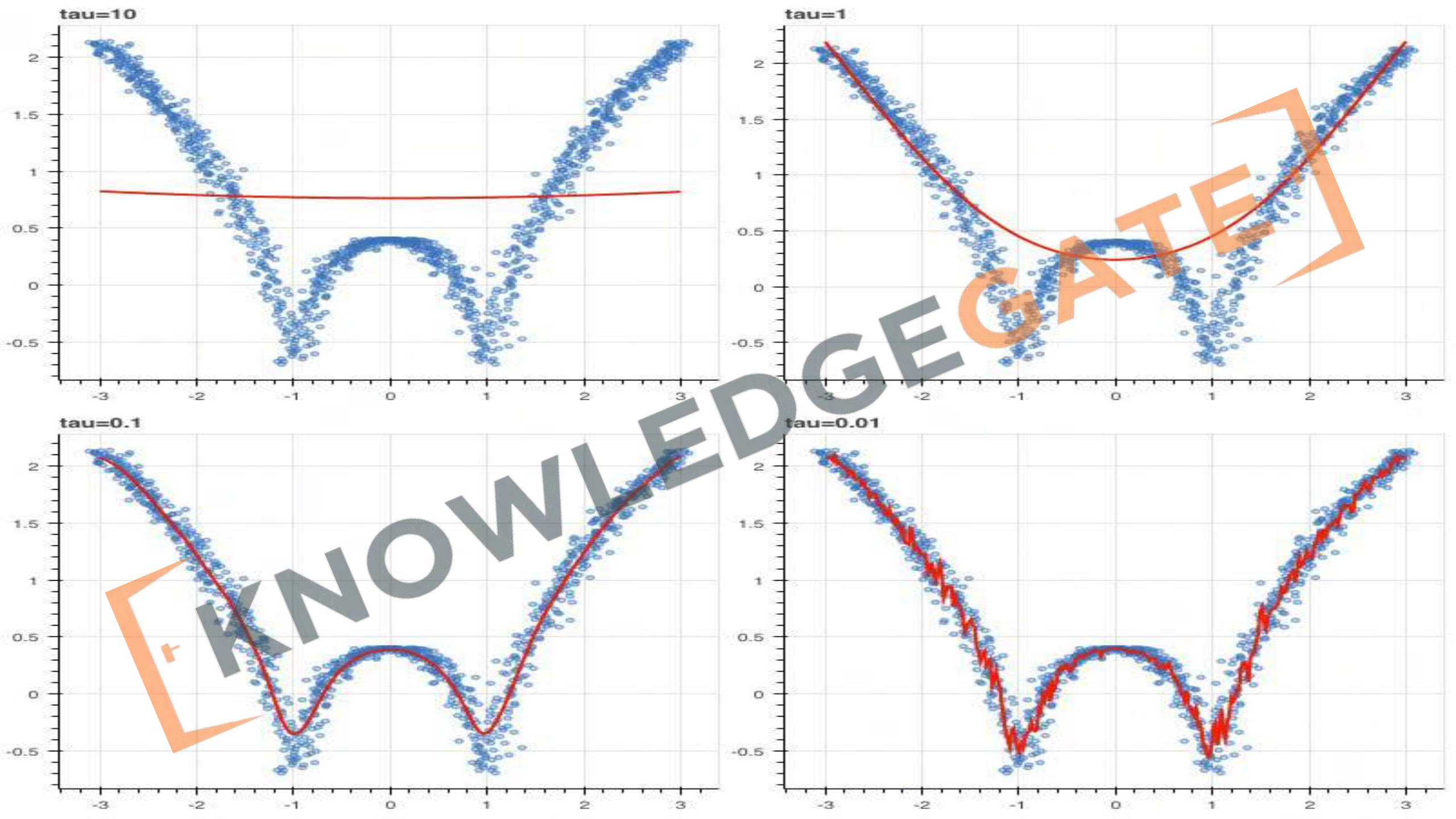




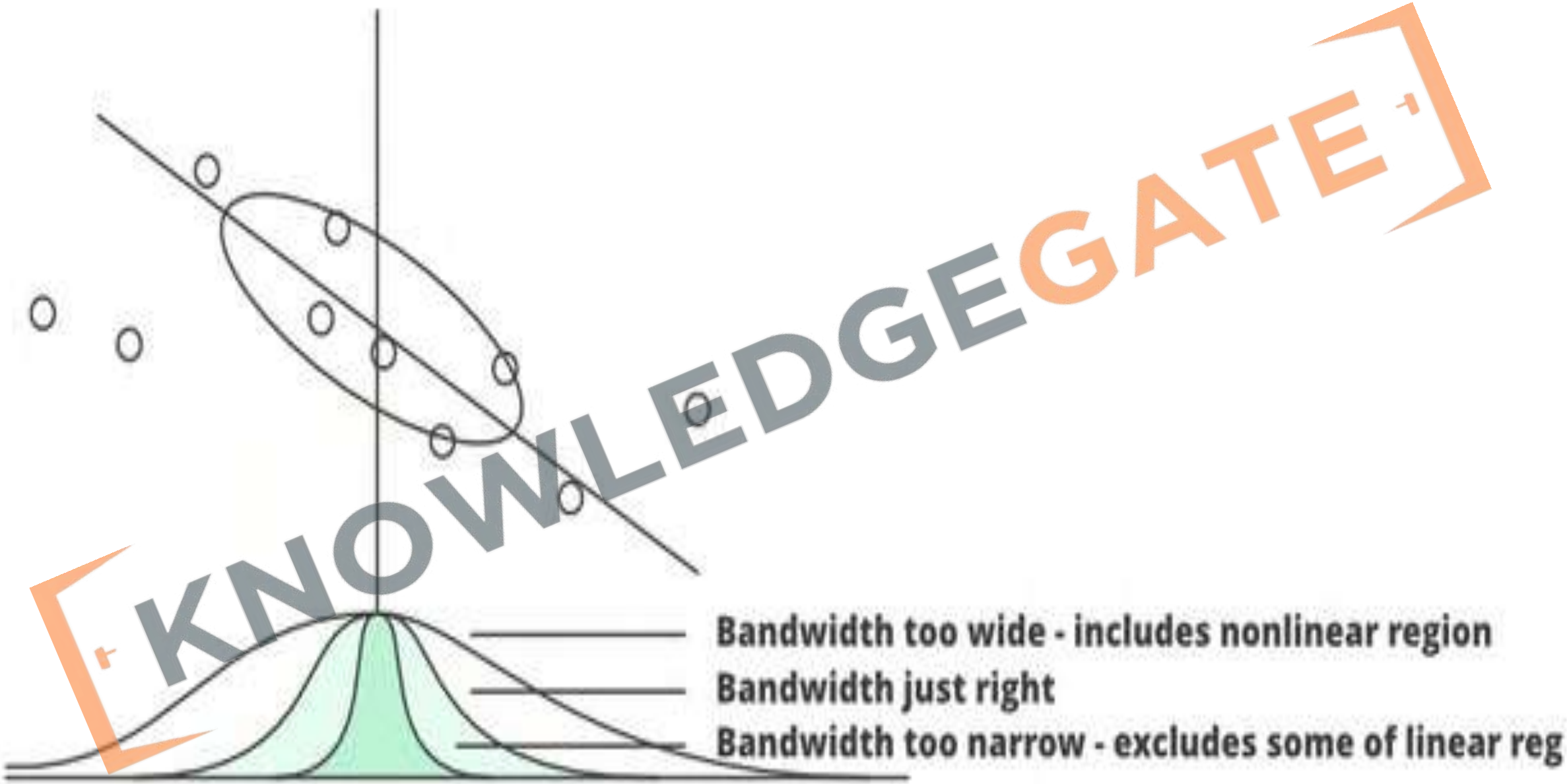
Z = 2

Z = 3

KNOWLEDGEGATE



KNOWLEDGEGATE



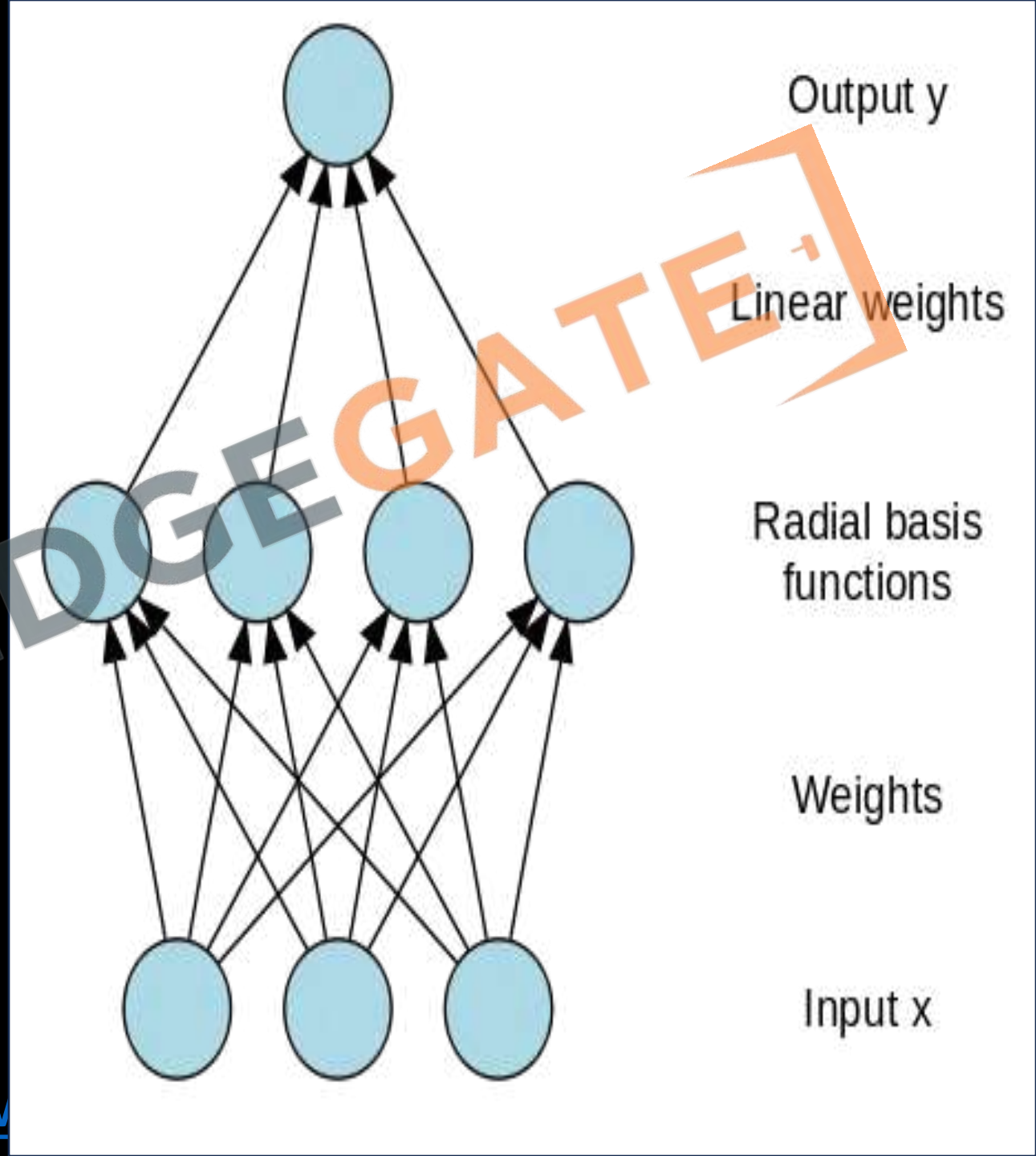
KNOWLEDGEGATE

- **History and Development**:
  - Emerged from mid-20th-century non-parametric regression techniques to handle nonlinear relationships without assuming a global model.
  - Formalized in the 1970s and 1980s by statisticians like Cleveland and Devlin through methods like LOWESS and LOESS, introducing key innovations such as kernel functions and bandwidth selection.
- **Modern-Day Applications**:
  - Used in diverse fields like economics, bioinformatics, robotics, and environmental science for forecasting, gene expression analysis, trajectory planning, and modeling environmental variables.
  - Advantages include flexibility in modeling complex, nonlinear relationships and local adaptation to data variations.
- **Current Trends and Research**:
  - Ongoing research focuses on improving efficiency, especially for high-dimensional data, and integrating LWR with deep learning.
  - Trends include real-time applications, big data, and edge computing, with modern toolkits like Python's statsmodels and scikit-learn and R's loess and gam facilitating implementation.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Radial Basis Functions

- Radial Basis Functions are a powerful tool for modeling complex, nonlinear relationships in data. They originated from mathematical interpolation methods and have evolved to become a key component in various modern applications, particularly in machine learning and data science. RBFs offer flexibility, smoothness, and local adaptation, making them suitable for a wide range of problems where traditional linear models fall short.



KNOWLEDGEGATE
http://www.know

Problem They Solve:

- Function Approximation, Interpolation, and Machine Learning: RBFs approximate complex functions and model intricate patterns in data, create smooth surfaces for precise interpolation (crucial in geostatistics for predicting unsampled values), and are used in machine learning for classification and regression, effectively handling nonlinear relationships between inputs and outputs.

KNOWLEDGEGATE

History:

- **Origins and Development**: Rooted in mid-20th century interpolation and approximation theory, RBFs were formalized in the 1970s and 1980s by researchers like Hardy. They became popular for smooth interpolations and handling scattered data points, with RBF networks extending their applications to machine learning and artificial intelligence.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Applications**:
  - **Machine Learning**: RBFs are used in Radial Basis Function Networks (RBFNs) for classification, regression, and time-series prediction.
  - **Image and Signal Processing**: RBFs help in tasks like image reconstruction, noise reduction, and signal interpolation.
  - **Geostatistics**: Used for spatial interpolation and modeling, such as predicting pollution levels at unmonitored locations.
  - **Robotics**: Applied in path planning and control systems, where smooth and adaptive control strategies are required.
  - **Finance**: Used for modeling financial data and predicting market trends.

KNOWLEDGEGATE

- **Problem and Significance**:
  - RBFs are used to predict air pollution levels across a city by interpolating data from a limited number of monitoring stations, providing smooth and accurate estimates even in unsampled locations.

- **Why Use RBFs**:
  - RBFs excel in smooth interpolation, adapting to local variations due to pollution sources, and handling nonlinear relationships between pollution levels and factors like traffic and industrial activity.

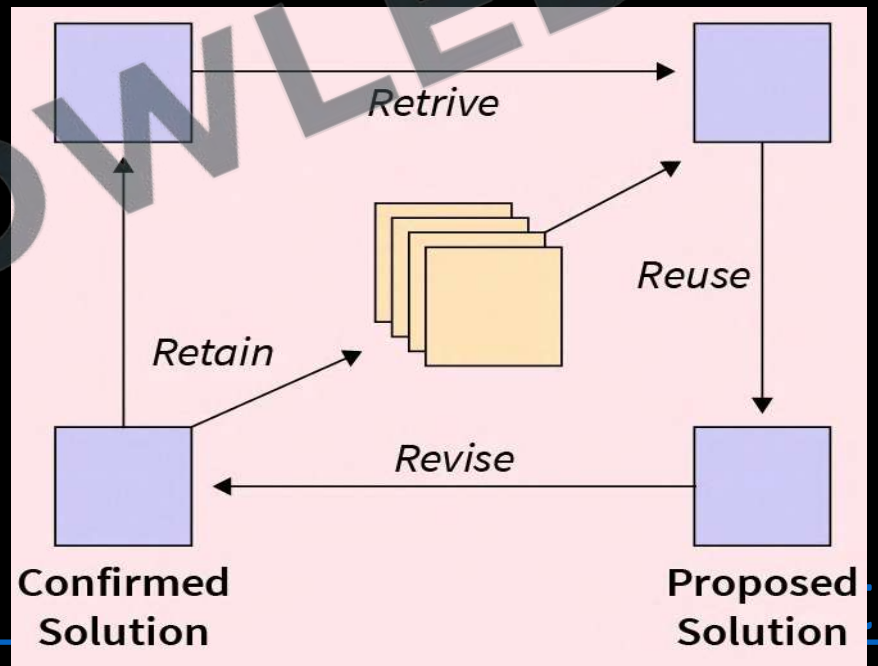
- **Outcome and Benefits**:
  - The resulting pollution maps help city planners and environmental agencies make informed decisions on pollution control, and increase public awareness, encouraging actions to reduce pollution.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Case-Based Learning in Machine Learning

- **History**: Originating from cognitive science in the 1980s, Case-Based Learning (CBL) in machine learning was inspired by the way humans use past experiences to solve new problems, formalized through early AI research on analogical reasoning.
- **Function or Use**: CBL solves new problems by retrieving and adapting solutions from similar past cases, making it useful for domains where similar problems recur and historical solutions are available.
- **Advantage**: CBL can handle complex and unstructured problems without needing explicit programming for every scenario, making it flexible and adaptable to a wide range of applications.



KNOWLEDGEGATE

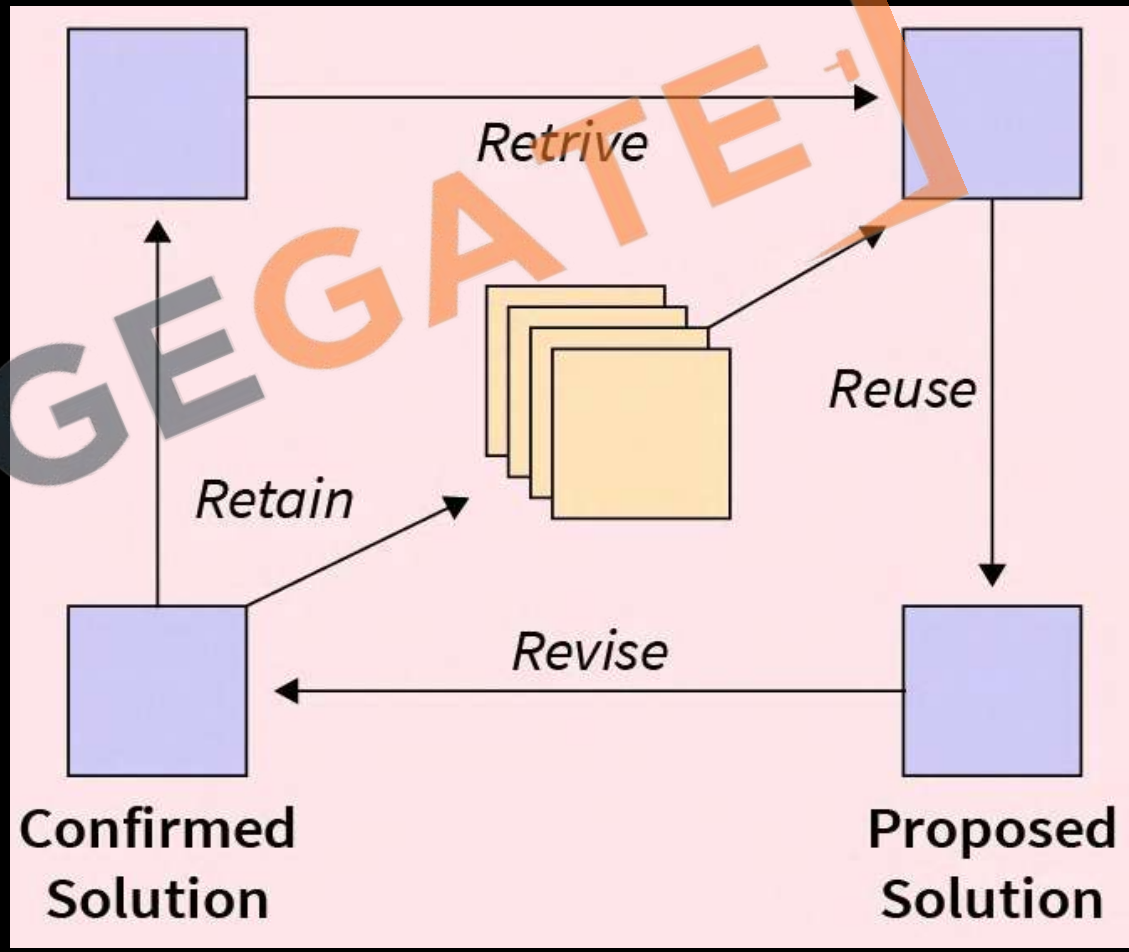
http e.in/gate

- **Advantage of CBL as a Lazy Problem Solving**: As a lazy learning method, CBL does not require a model to be built or trained beforehand, allowing it to use the most recent data for problem-solving, which can be particularly useful in dynamic environments.
- **Disadvantage or Limitation**: CBL can be computationally expensive and slow for large datasets due to the need to search through a potentially vast case library to find similar past cases.
- **Applications**: CBL is used in various fields such as medical diagnosis, legal reasoning, customer support, and maintenance troubleshooting, where leveraging past cases can significantly improve decision-making and problem-solving.
- **Case-Based Learning Cycle with Different Schemes**: The CBL cycle typically involves retrieving the most relevant cases, reusing the information to solve the current problem, revising the proposed solution if necessary, and retaining the new solution for future use, often implemented through schemes like the nearest neighbor, template-based, and knowledge-based approaches.

KNOWLEDGEGATE

Medical Diagnosis Support System

- **Scenario**: A hospital uses a Case-Based Learning system to assist doctors in diagnosing rare diseases. The system contains a large database of past medical cases, including patient symptoms, diagnostic tests, treatments, and outcomes.
- **How It Works**:
  - **Retrieve**: When a new patient presents symptoms that are difficult to diagnose, the doctor inputs these symptoms into the system.
  - **Reuse**: The CBL system searches its database for past cases with similar symptom profiles.
  - **Revise**: It retrieves the most relevant cases and suggests potential diagnoses and treatment plans that were successful in those past cases.
  - **Retain**: After the doctor reviews and possibly adjusts the suggested diagnosis and treatment, the final decision is added to the system's database as a new case.



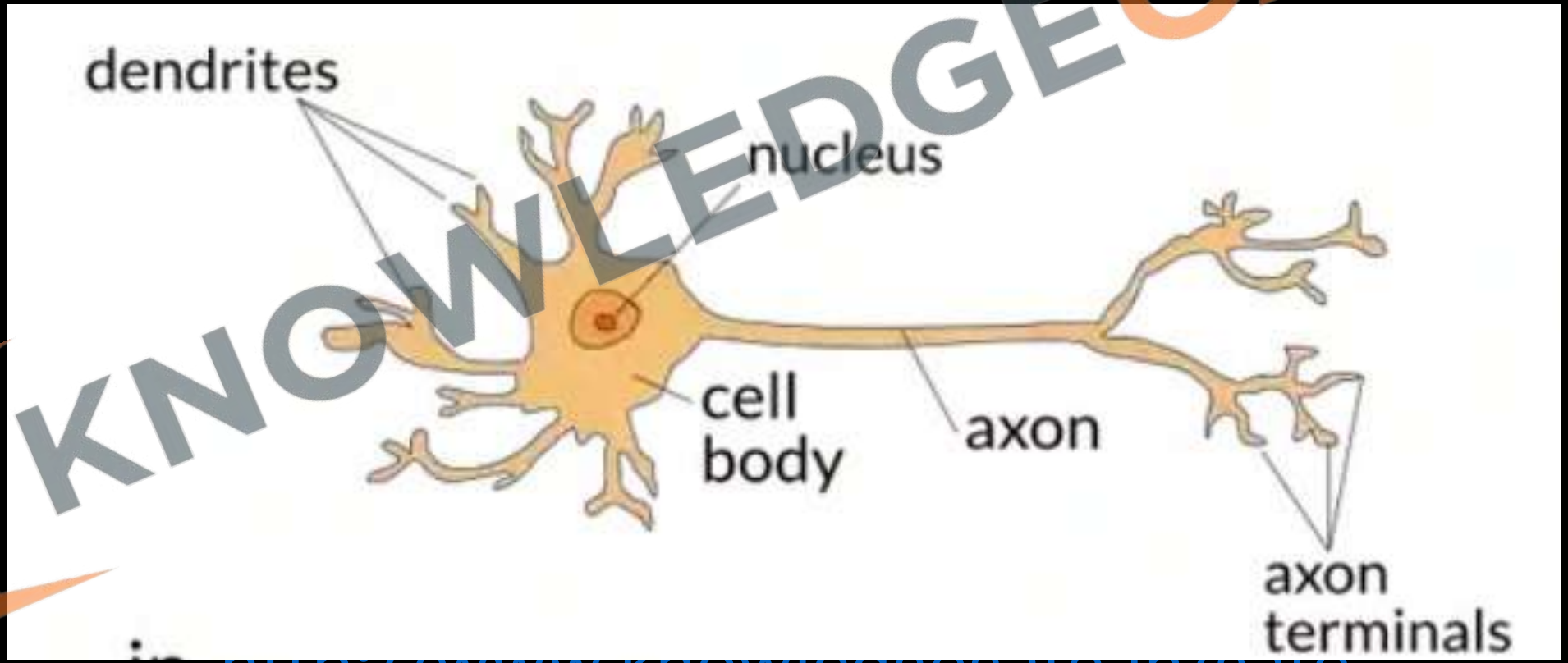
KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **(UNIT-4: ARTIFICIAL NEURAL NETWORKS)** ARTIFICIAL NEURAL NETWORKS - Perceptron's, Multilayer perceptron, Gradient descent & the Delta rule, Multilayer networks, Derivation of Backpropagation Algorithm, Generalization, Unsupervised Learning - SOM Algorithm and its variant; DEEP LEARNING - Introduction, concept of convolutional neural network, Types of layers - (Convolutional Layers, Activation function, pooling, fully connected), Concept of Convolution (1D and 2D) layers, Training of network, Case study of CNN for eg on Diabetic Retinopathy, Building a smart speaker, Self-deriving car etc.

KNOWLEDGE GATE

Biological Neurons and Their Structure

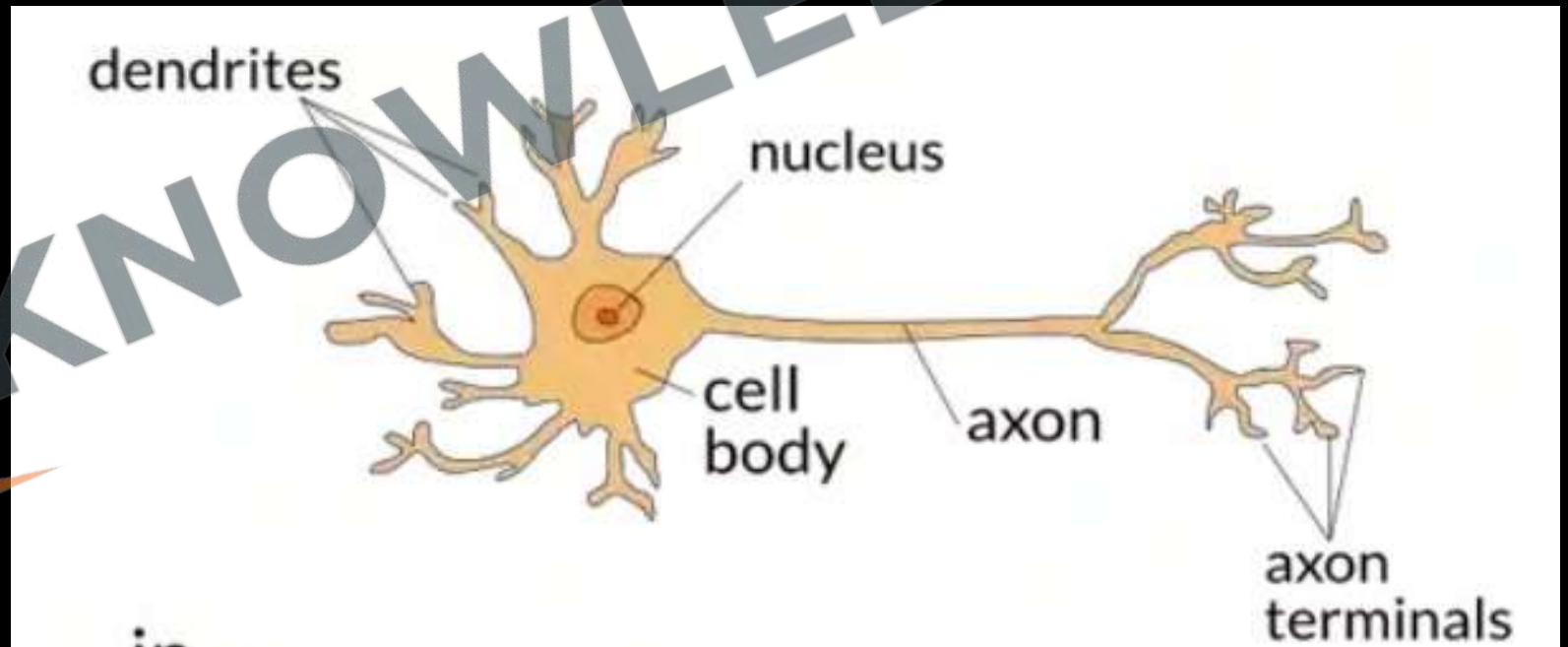
- Neuron, Also Called Nerve Cell:
  - Basic working unit of the brain.
  - Specialized cells responsible for sending and receiving signals from the brain.



KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Structure of a Biological Neuron:

- **Dendrite (Input)**:
  - Receives signals from other neurons.
- **Cell Body (Summing Unit)**:
  - Sums up all the incoming input signals.
  - Contains the threshold unit.
- **Axon (Output)**:
  - When the sum reaches a threshold level, the neuron generates an output signal which travels down to other neurons.



KNOWLEDGE GATE

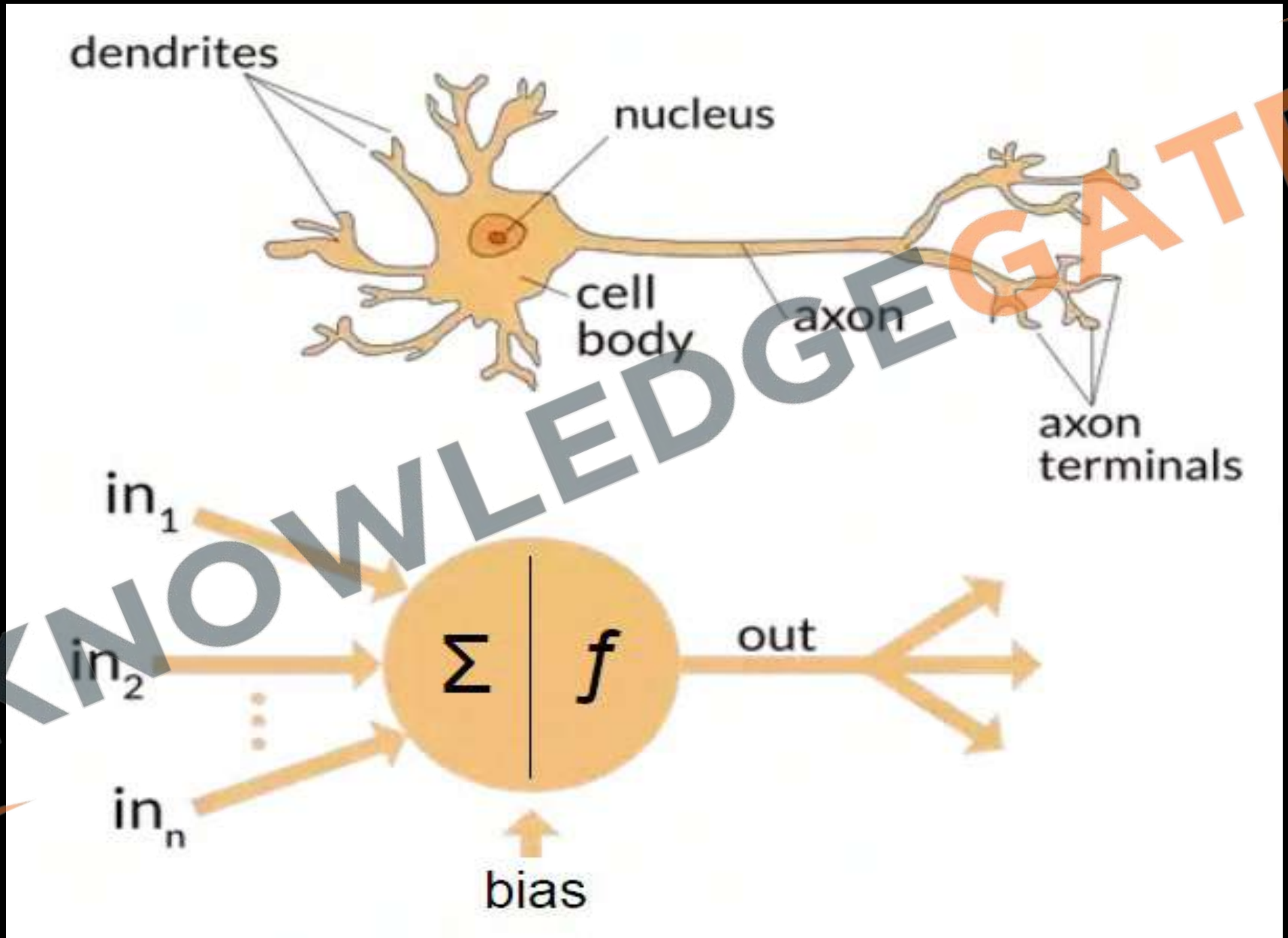
Types of Biological Neurons:

- **Sensory Neurons**:
  - Handle taste, smell, hearing, seeing, and feeling.
- **Motor Neurons**:
  - Control voluntary and involuntary movements of muscles and organs.
- **Interneurons**:
  - Intermediate neurons found in the spinal cord and brain.
  - Pass signals from sensory neurons to motor neurons.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

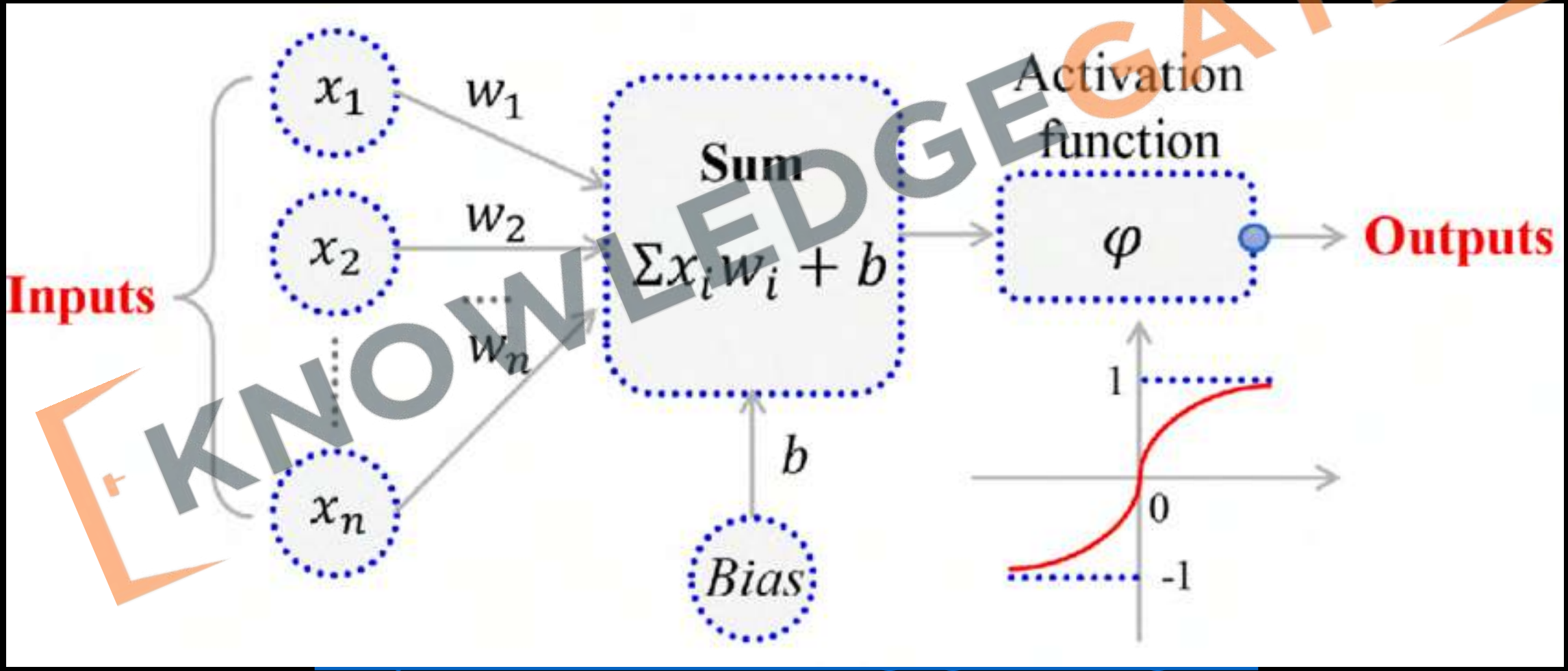
Artificial Neuron



KNOWLEDGEGATE

Artificial Neuron

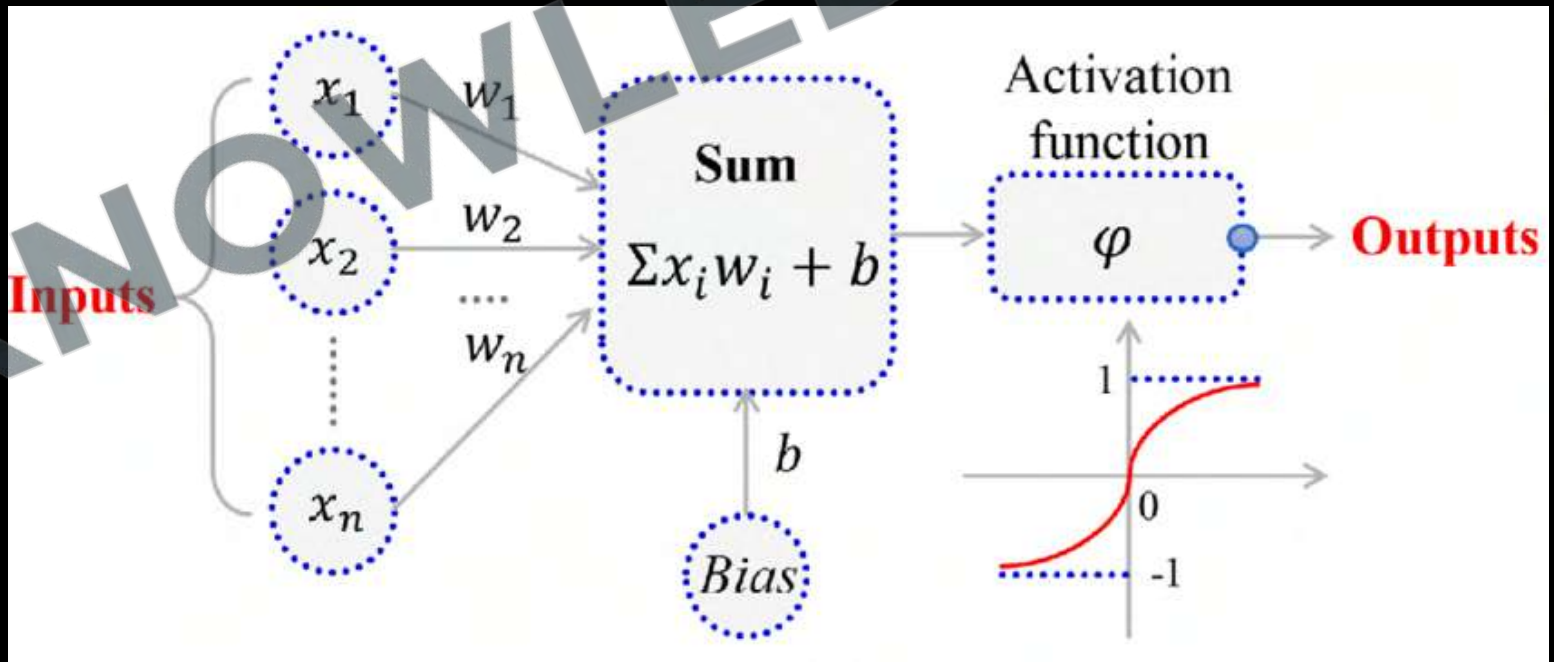
- An artificial neuron is a mathematical function based on the model of biological neurons.



KNOWLEDGE GATE

The model of an artificial neuron has five components:

- **Input Values/Input Layer**:
  - Input values are passed to a neuron using this layer. Similar to dendrites in biological neurons.
- **Synapses/Weighted Connection Links**:
  - Each connection link is characterized by a weight or strength (w_i). Each input (x_i) is multiplied by its weight to get the synaptic strength.
- **Summing Function**:
  - The summing function takes the sum of all weighted connection links: $\Sigma w_i x_i$. A bias value (b) is also added to this weighted sum.
- **Activation Function**:
  - The activation function decides whether or not a neuron is fired based on the output value produced.
- **Output/Output Layer**:
  - The output layer gives the final output of a neuron which can then be passed to other neurons in the network.



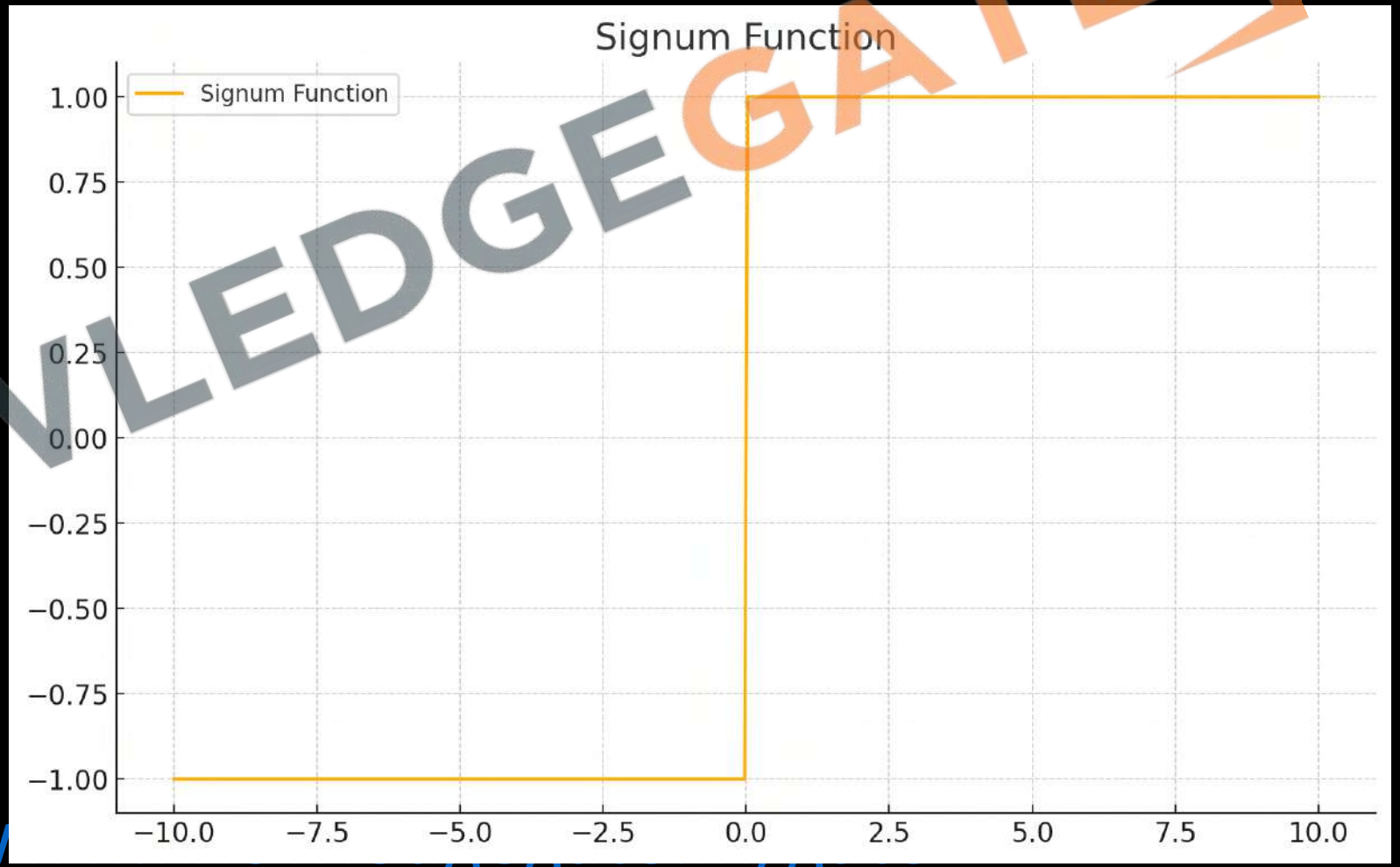
KNOWLEDGEGATE

Activation Functions in Neural Networks

- Activation functions are mathematical functions that determine the output of a neural network. They decide whether a neuron should be activated ("fired") or not, introducing non-linearity into the model, which is crucial for handling complex data.

Step Function:

- $z = f(y) = \begin{cases} 0, & y \leq 0 \\ 1, & y > 0 \end{cases}$

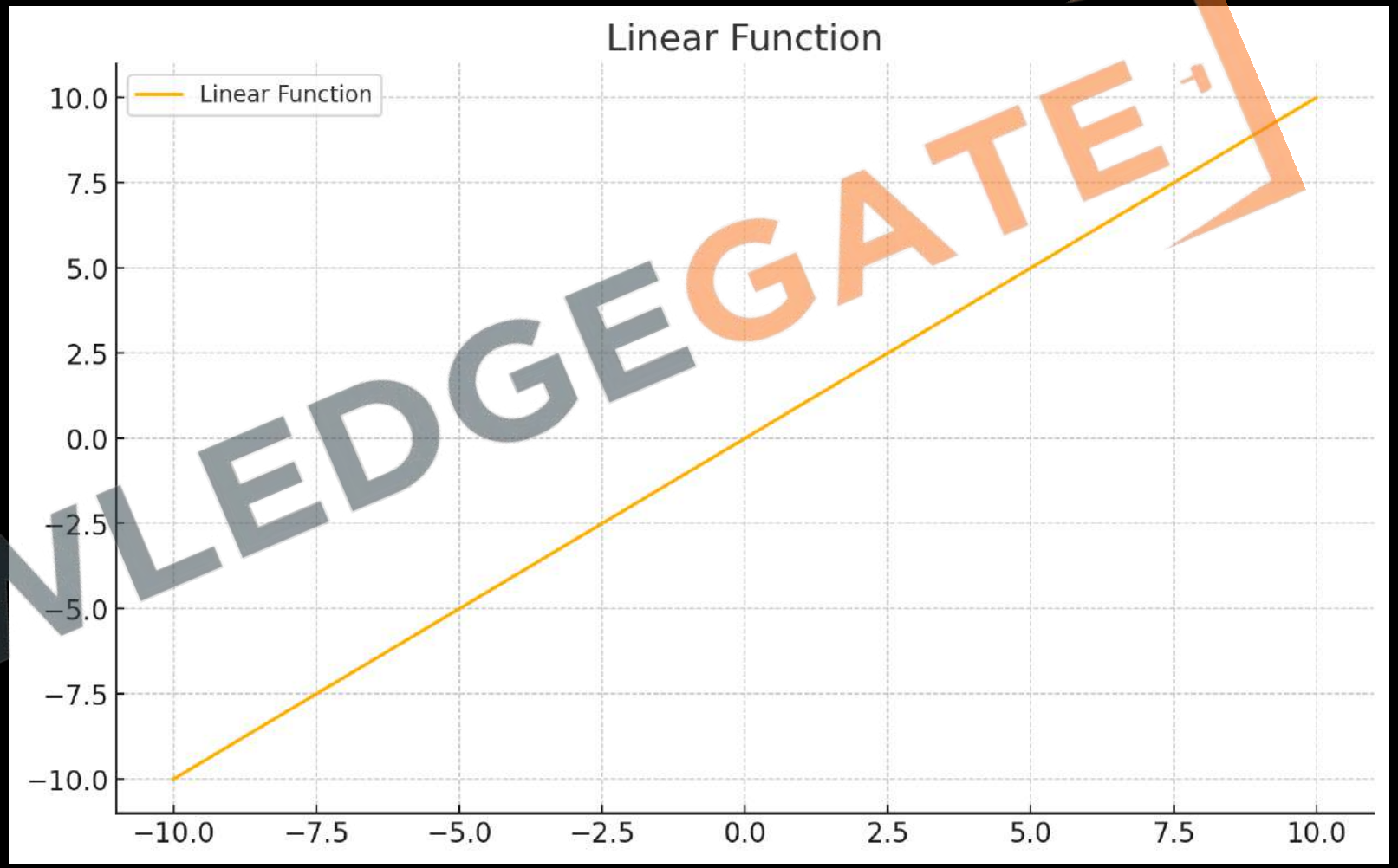


KNOWLEDGEGATE

http://w

Linear Function:

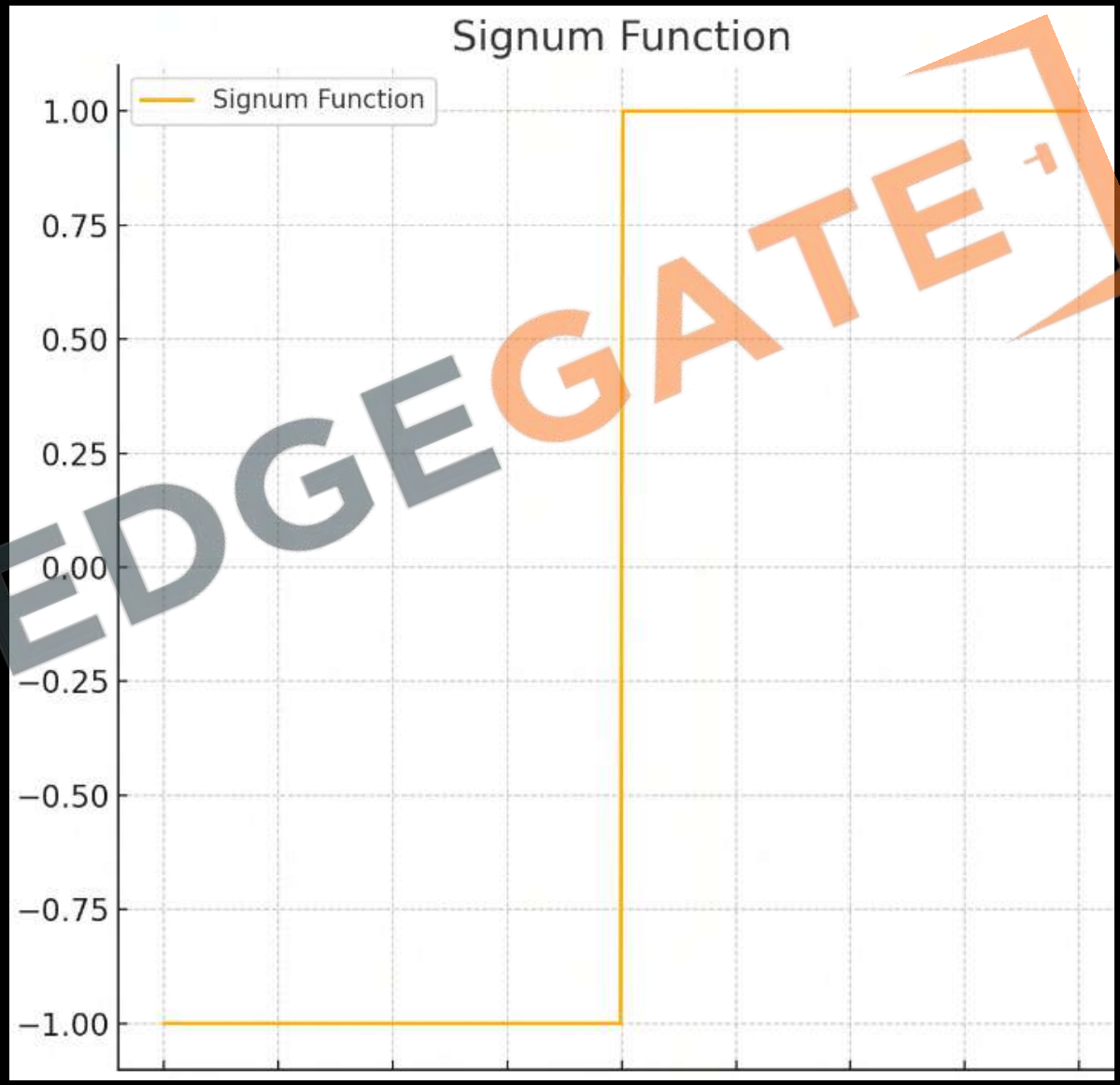
- $z = f(y) = y$



KNOWLEDGEGATE

Signum Function:

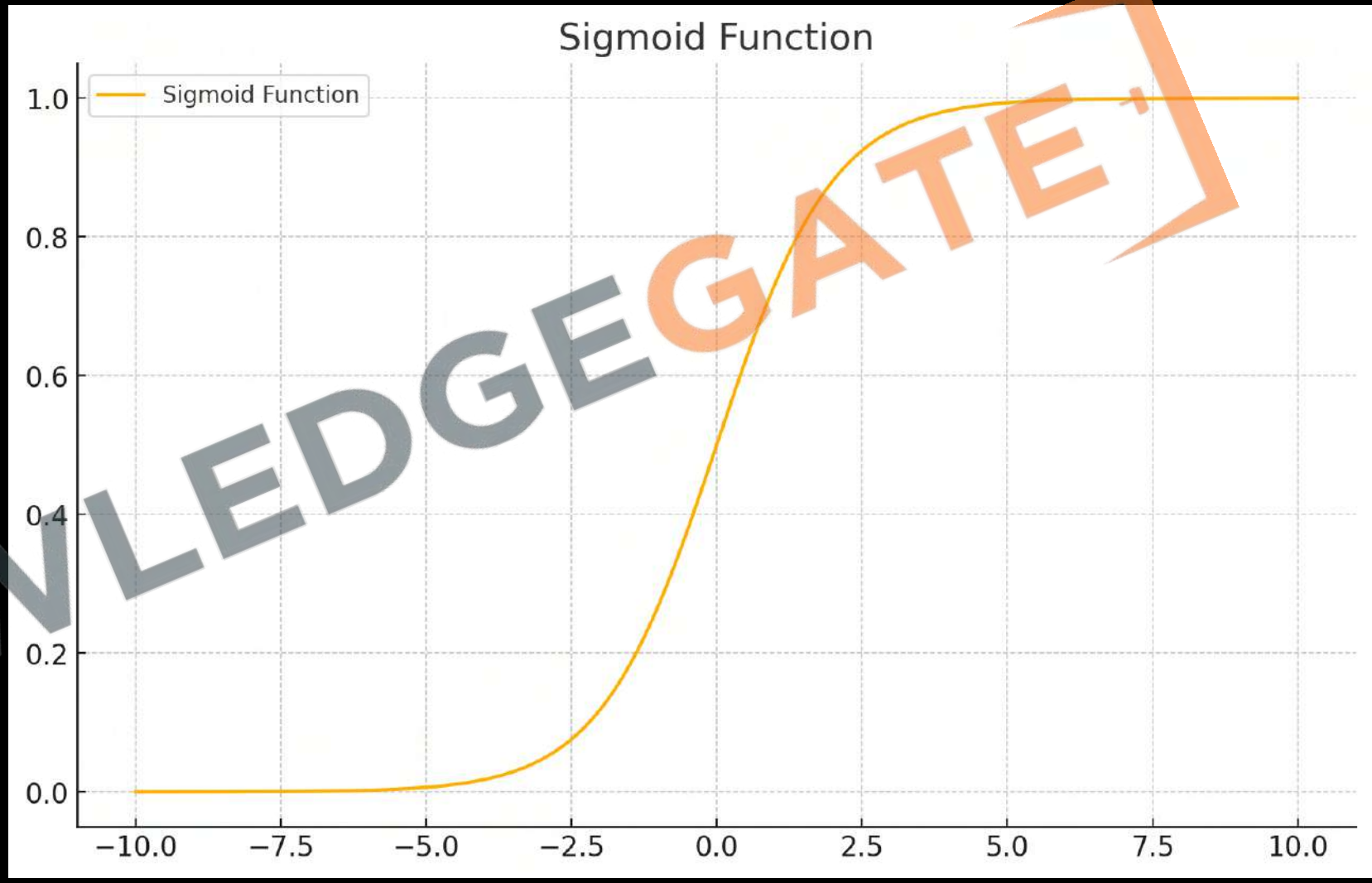
- $$z = f(y) = \begin{cases} -1, & y < 0 \\ 0, & y = 0 \\ 1, & y > 0 \end{cases}$$



KNOWLEDGEGATE

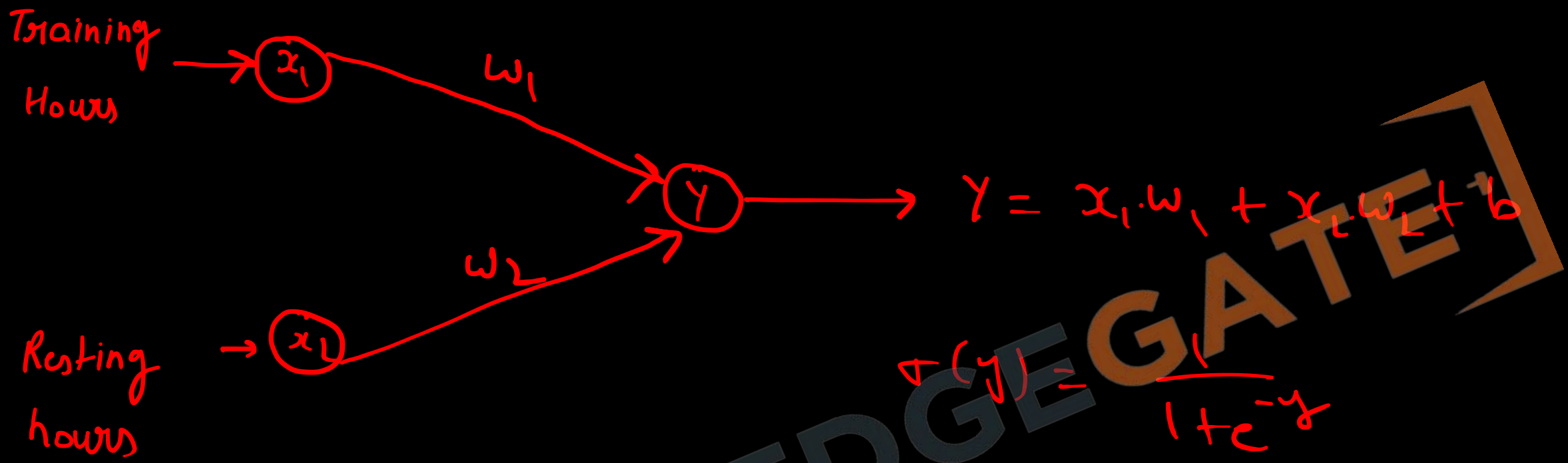
Sigmoid (Logistic) Function:

- $z = f(y) = \frac{1}{1+e^{-y}}$



KNOWLEDGEGATE

http://www.knowledgegate.in/gate



$w_1 =$

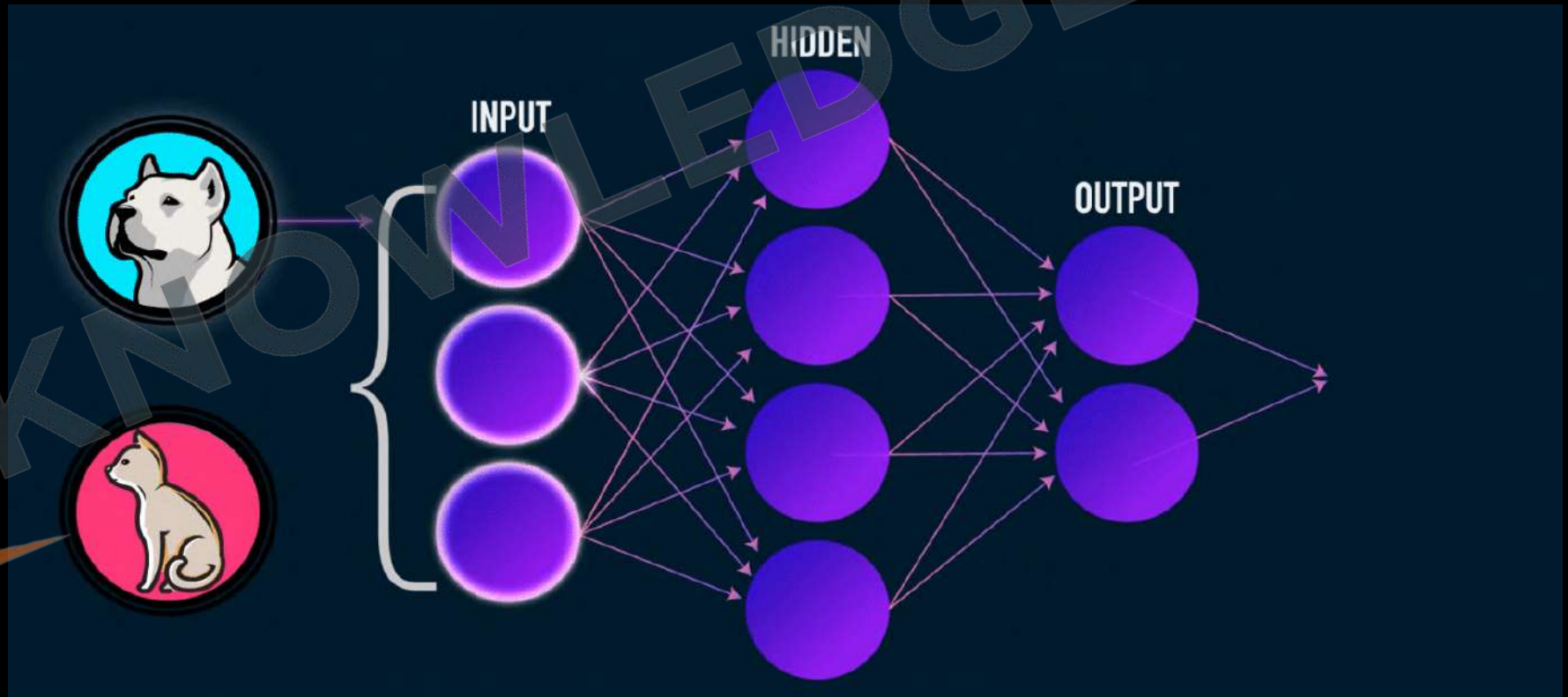
$w_2 =$

$b =$

KNOWLEDGEGATE

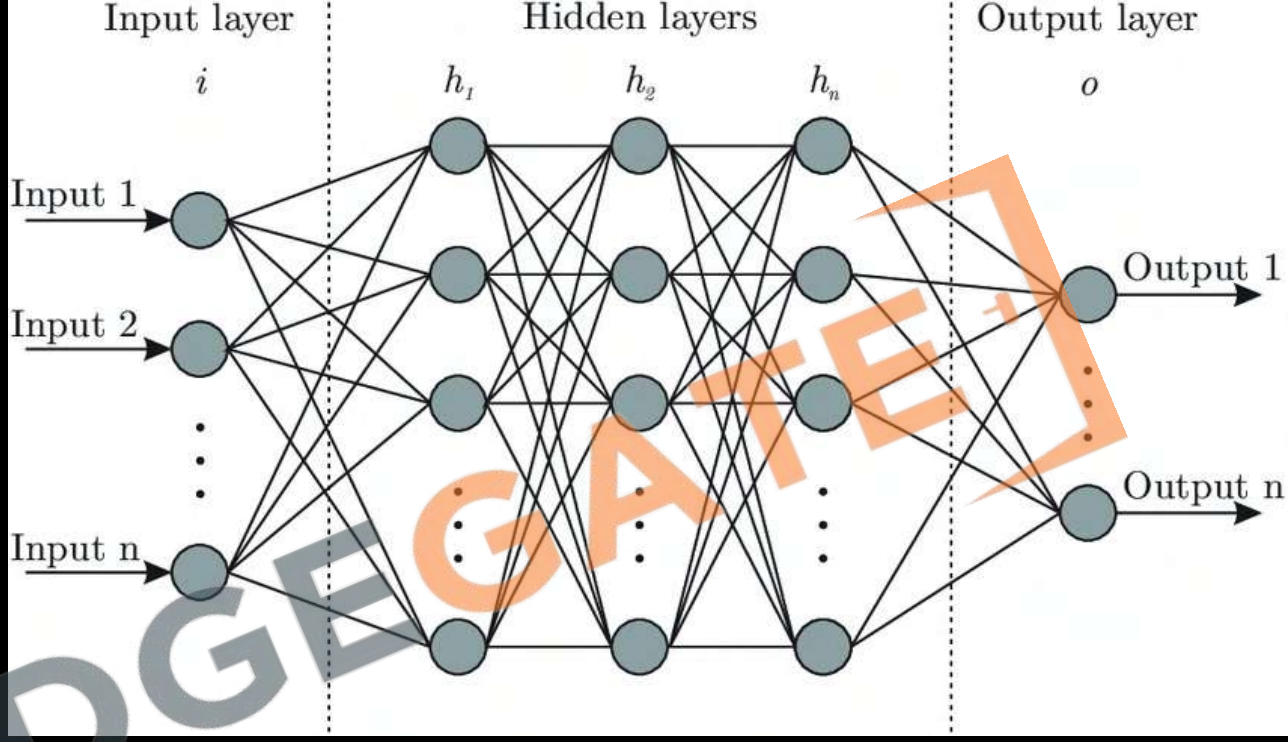
Artificial Neural Network (ANN)

- An Artificial Neural Network (ANN) is a computing system designed to simulate the way the human brain analyzes and processes information.
- It is a data processing system consisting of a large number of interconnected processing elements called artificial neurons (nodes). These neurons are connected through connection links.
- ANNs have self-learning capabilities, allowing them to improve as more data becomes available.



KNOWLEDGEGATE

Architecture of ANN:



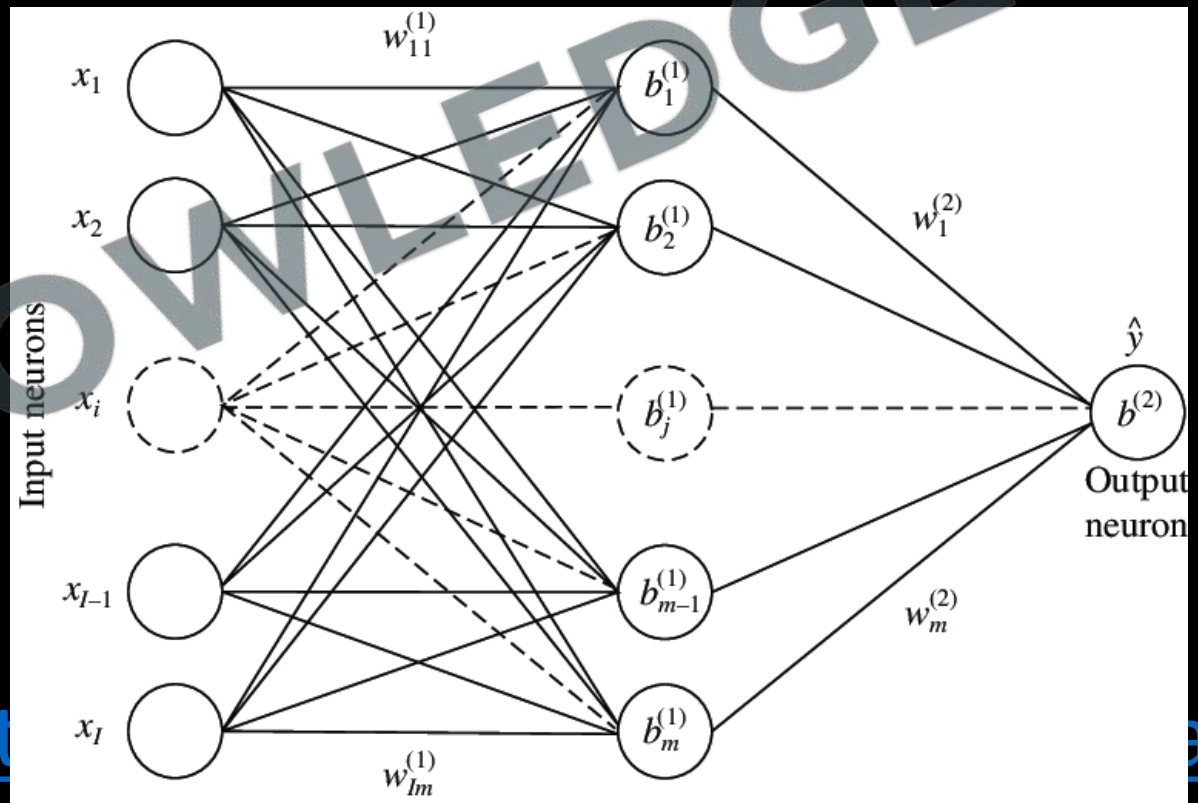
- **Input Layer**:
  - Contains nodes (artificial neurons) that receive input from the outside world.
- **Output Layer**:
  - Contains nodes that provide the output of the network.

- **Hidden Layers**:
  - Located between the input and output layers.
  - Comprise hidden nodes or neurons not directly visible to the external system.
  - Hidden layers are responsible for intermediate computations and feature extraction.
  - There can be zero or more hidden layers depending on the complexity of the problem.
  - One hidden layer is sufficient for many problems, but deeper networks (with more hidden layers) can model more complex patterns.

KNOWLEDGEGATE

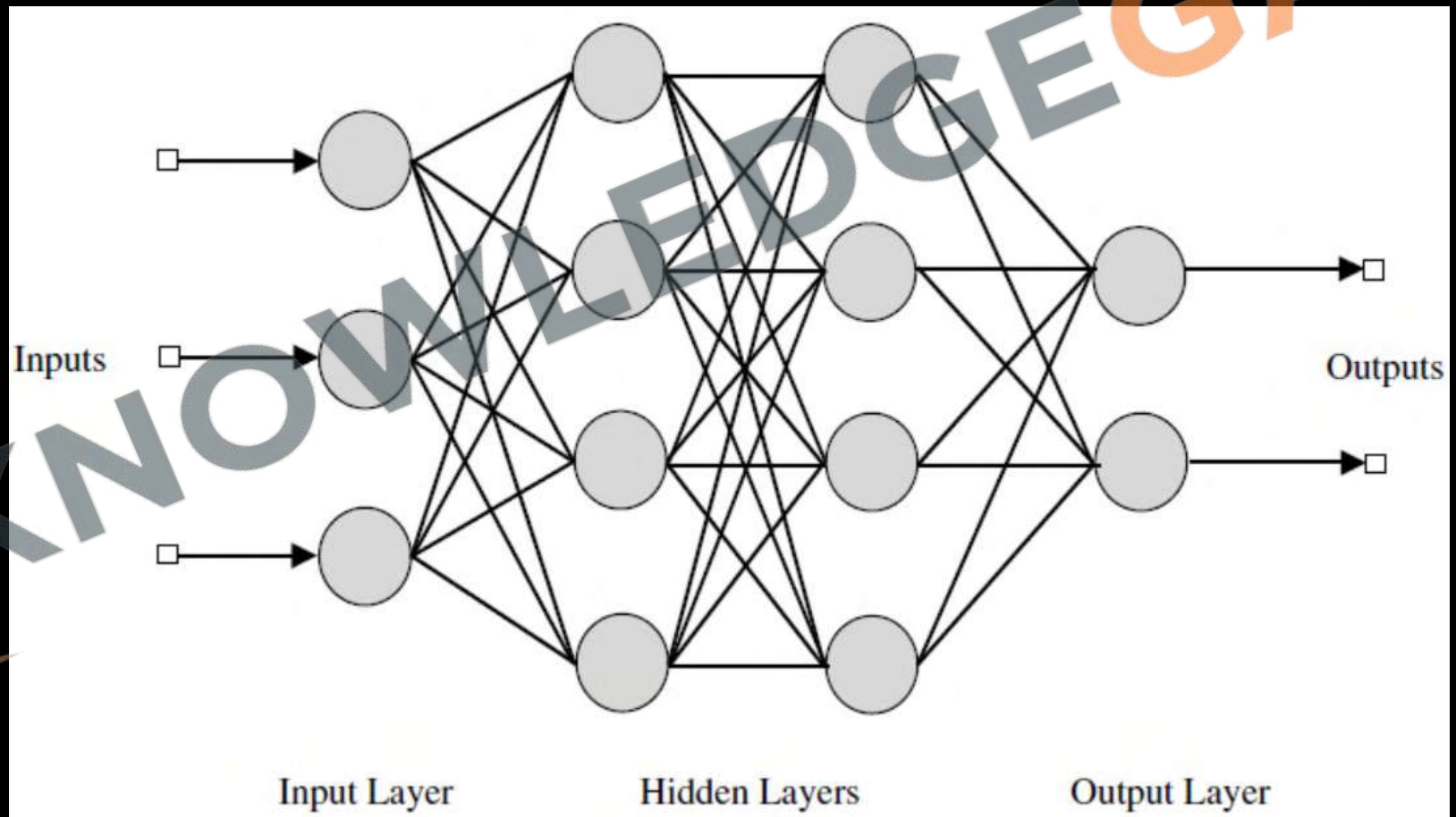
Classification of Artificial Neural Networks (ANN)

- **Single Layer Feed-forward Network**:
  - **Single Layer**: Only one computational layer (output layer), making it a single-layer ANN, but it consists of two layers in total (input and output layers).
  - **Feed-forward Network**: Information flows from the input layer to the output layer without any feedback loops.



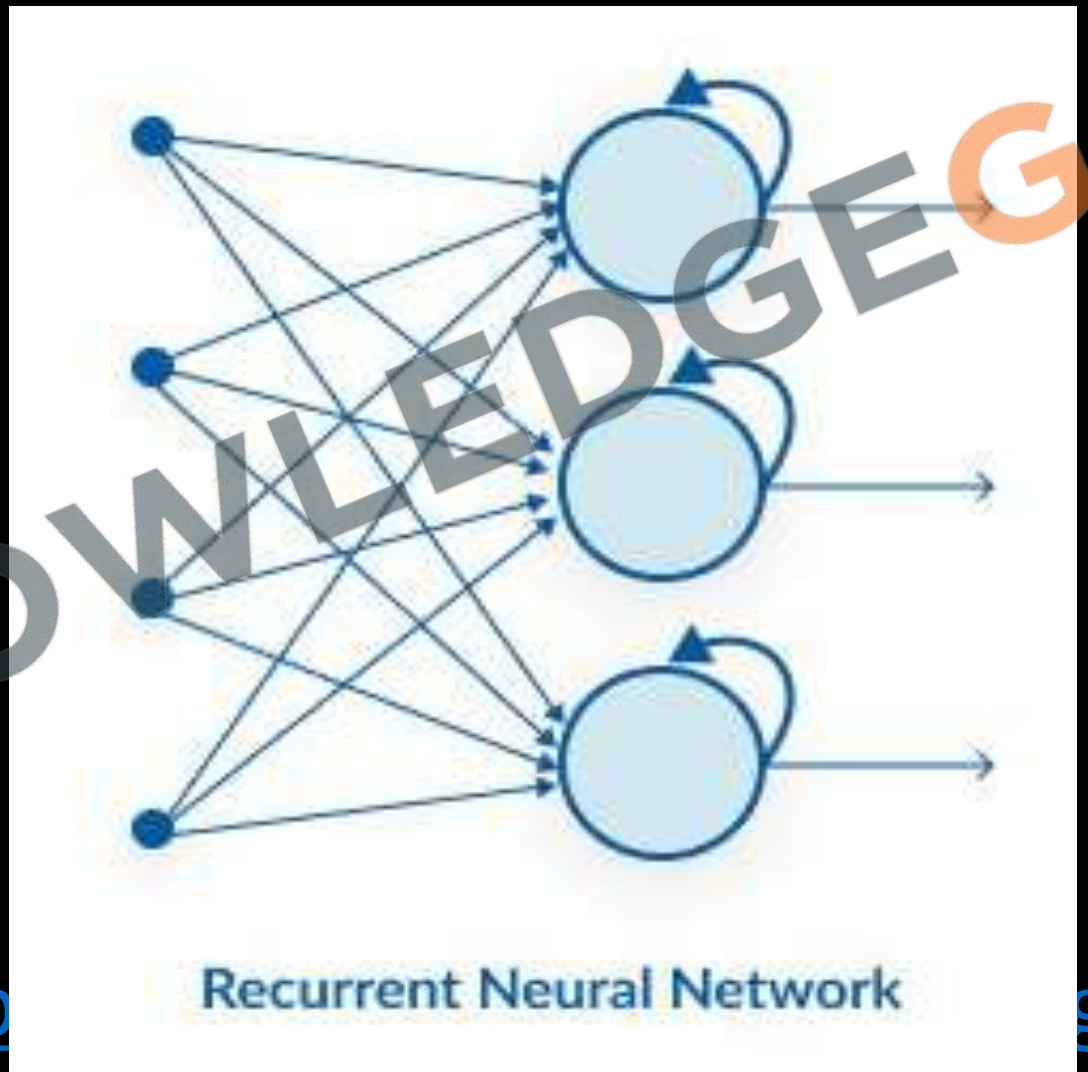
KNOWLEDGEGATE

- **Multilayer Feed-forward Network:**
  - **Multilayer**: Includes an input layer, one or more intermediate (hidden) layers, and an output layer.
  - Hidden layers perform intermediate computations before passing the information to the output layer.



KNOWLEDGEGATE

- **Recurrent Network**:
  - These networks include at least one feedback loop, differing from feed-forward architectures. Can be single-layer or multi-layer recurrent networks.

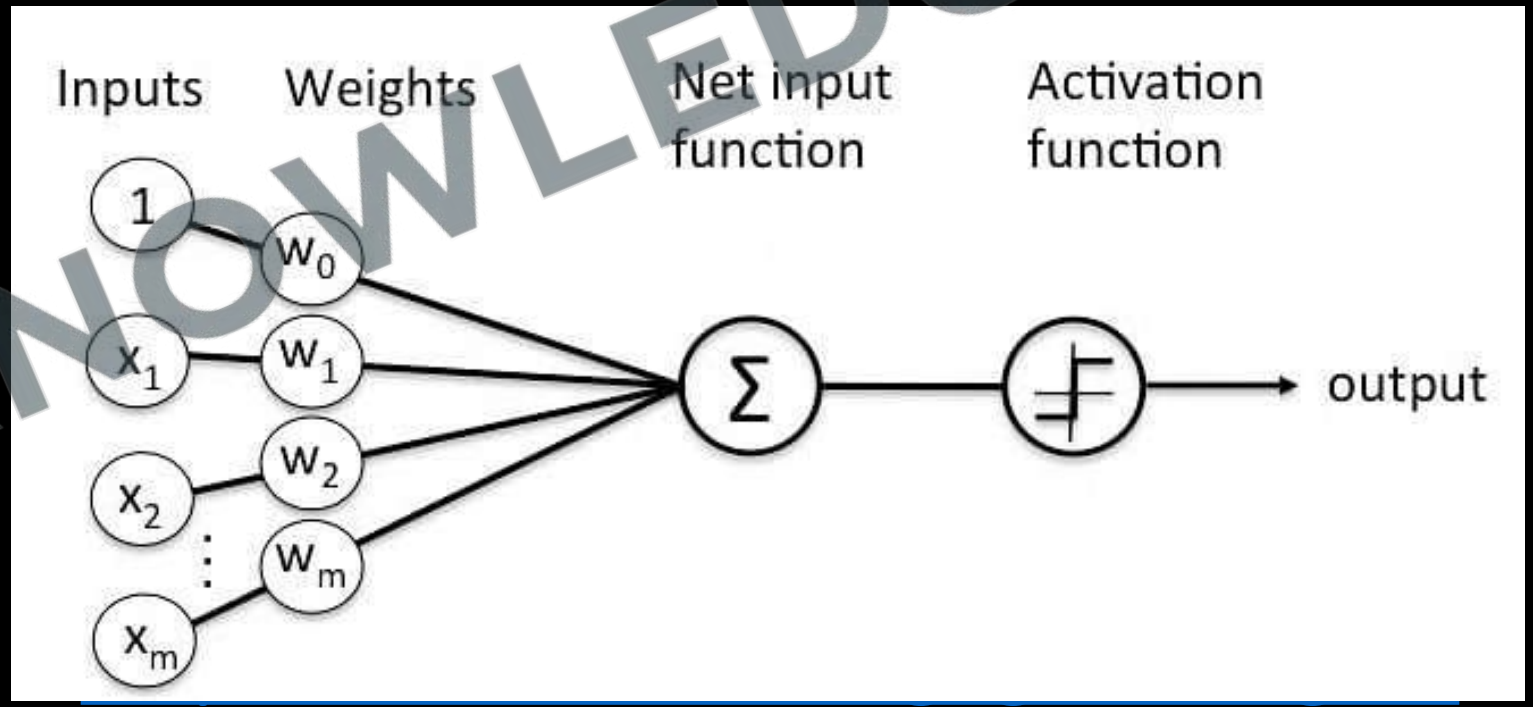


KNOWLEDGEGATE

http gate

(Artificial Neuron) Perceptron

- **Definition**: The simplest form of a neural network. Consists of a single neuron with adjustable synaptic weights and bias.
- **History**: Introduced by Frank Rosenblatt in 1957.
- **Components**: One or more inputs, a processing unit, and a single output. Used for supervised learning of binary classifiers. Enables neurons to learn and process elements in the training set one at a time.



KNOWLEDGE GATE

Perceptron Function

- **Definition**: Maps input x, multiplied with the learned weight coefficient, to generate an output value $f(x)$.
- Formula:

$$f(x) = \begin{cases} 1, & \text{if } wx + b > 0 \\ 0, & \text{if } wx + b \leq 0 \end{cases}$$

- Where:
  - w: Values of weights.
  - b: Bias.
  - x: Values of inputs.
  - n: Number of inputs to the perceptron.

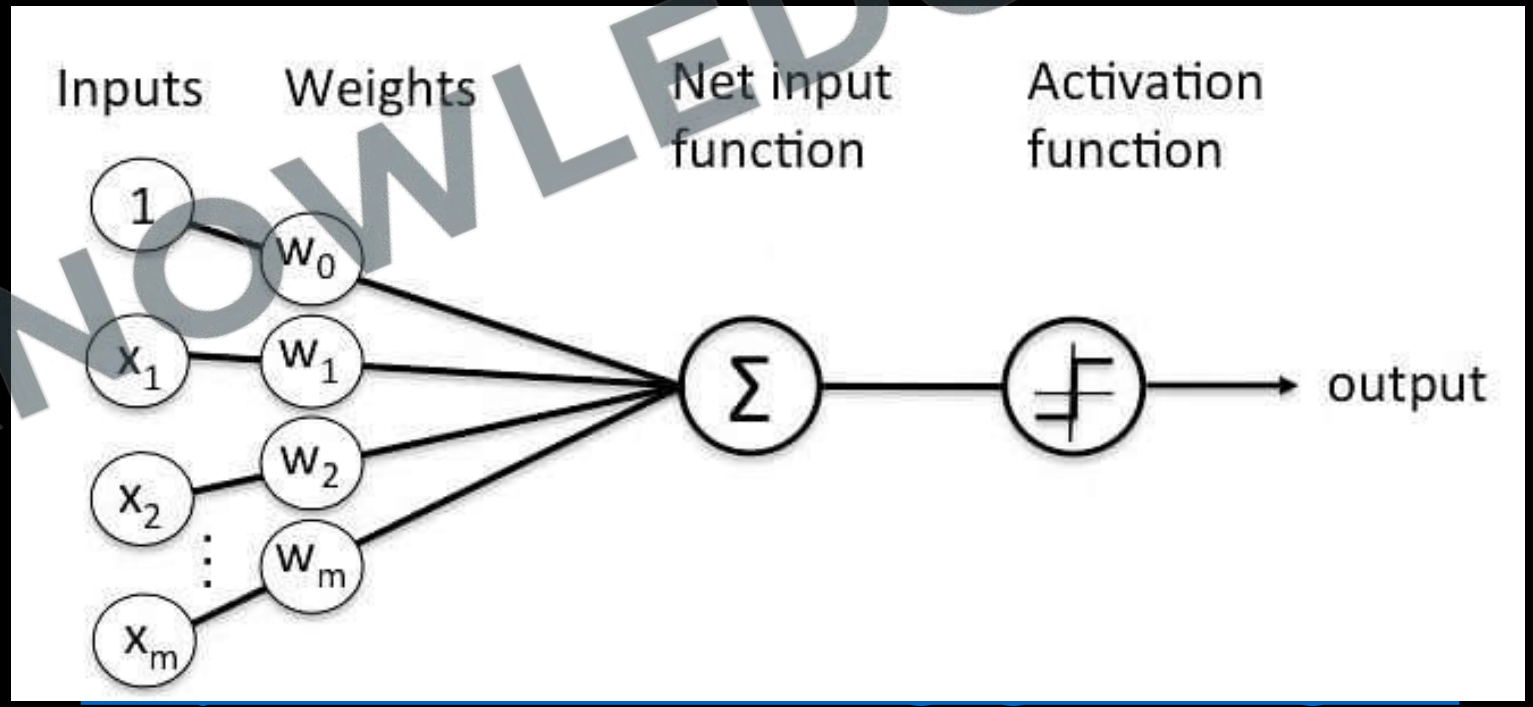
[KNOWLEDGEGATE]

Output:

- The output can be represented as 0 or 1, or -1 and 1, depending on the activation function used. For e.g.

$$f(x_1, x_2, \ldots, x_n) = 1 \text{ if } w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n > 0$$

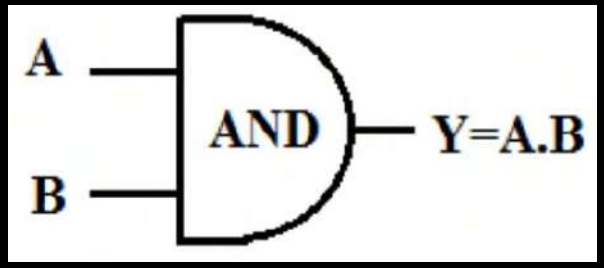
$$f(x_1, x_2, \ldots, x_n) = 0 \text{ if } w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n \leq 0$$



KNOWLEDGEGATE

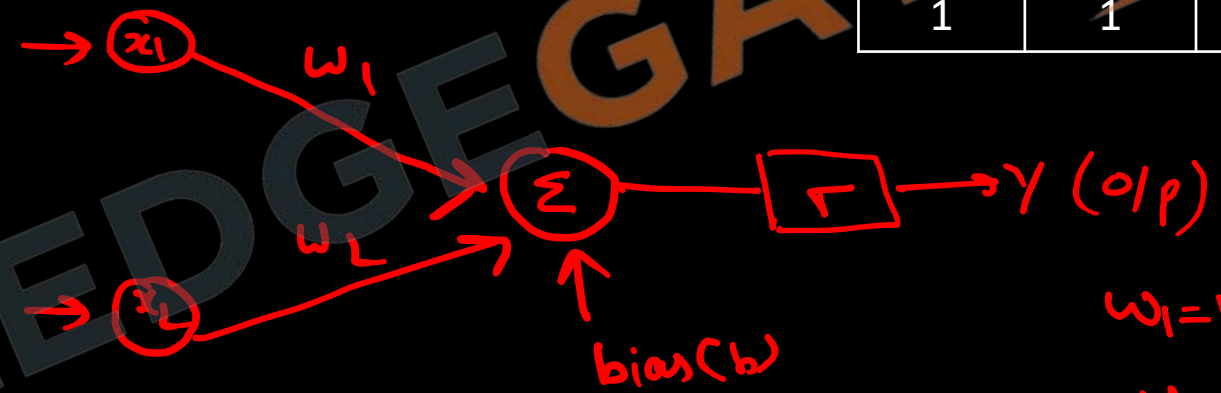
AND gate using neural networks

- Steps in Perceptron Algorithm:
  - Initialize weight values and bias.
  - Forward propagate the inputs.
  - Check the error.
  - Backpropagate and adjust weights and bias.
  - Repeat for all training examples.



Truth Table		
Input		Output
A	B	Y = A . B
0	0	0
0	1	0
1	0	0
1	1	1

$$Y = \begin{cases} 1, & \text{if } wx + b > 0 \\ 0, & \text{if } wx + b \leq 0 \end{cases}$$



$w_1 = 1$

$w_2 = 1$

$b = -1$

$Y = w_1 x_1 + w_2 x_2 + b$

Row1 → $1 \cdot 0 + 1 \cdot 0 + (-1) = -1 \rightarrow 0$ ✓

Row2 → $1 \cdot 0 + 1 \cdot 1 + (-1) = 0 \rightarrow 0$ ✓

Row3 → $1 \cdot 1 + 1 \cdot 0 + (-1) = 0 \rightarrow 0$ ✓

Row4 → $1 \cdot 1 + 1 \cdot 1 + (-1) = 1 \rightarrow 1$ ✓

$w_1 \cdot x_1 + w_2 x_2 + b$

↓

$1 \cdot x_1 + 1 \cdot x_2 - 1$

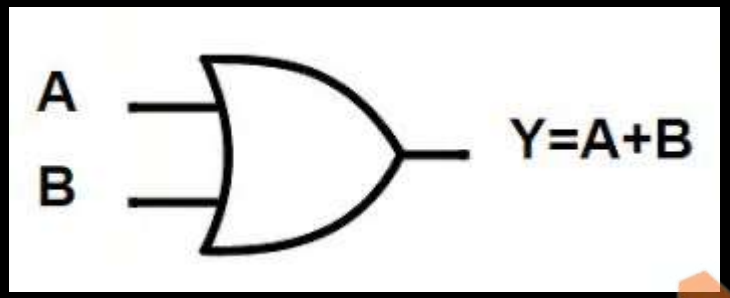
$x_1 + x_2 - 1$

KNOWLEDGEGATE

OR gate using neural networks

Truth Table		
Input		Output
A	B	Y = A + B
0	0	0
0	1	1
1	0	1
1	1	1

- Steps in Perceptron Algorithm:
  - Initialize weight values and bias.
  - Forward propagate the inputs.
  - Check the error.
  - Backpropagate and adjust weights and bias.
  - Repeat for all training examples.



$$Y = \begin{cases} 1, & \text{if } wx + b > 0 \\ 0, & \text{if } wx + b \leq 0 \end{cases}$$

$w_1 = 1$
$w_2 = 1$
$b = -1$

$w_1 = 1$, $w_2 = 1$

$w_1 x_1 + w_2 x_2 + b$

$1 \cdot 0 + 1 \cdot 0 + (-1) = -1 \rightarrow 0$

$1 \cdot 0 + 1 \cdot 1 + (-1) = 0 \rightarrow 0$

$1 \cdot 1 + 1 \cdot 0 + (-1) = 0 \rightarrow 0$

$1 \cdot 1 + 1 \cdot 1 + (-1) = 1 \rightarrow 1$

$w_1 = 2$, $w_2 = 1$

$w_1 x_1 + w_2 x_2 + b$

$2 \cdot 0 + 1 \cdot 0 + (-1) = -1 \rightarrow 0$

$2 \cdot 0 + 1 \cdot 1 + (-1) = 0 \rightarrow 0$

$2 \cdot 1 + 1 \cdot 0 + (-1) = 1 \rightarrow 1$

$2 \cdot 1 + 1 \cdot 1 + (-1) = 2 \rightarrow 1$

$w_1 = 2$, $w_2 = 2$

$w_1 x_1 + w_2 x_2 + b$

$2 \cdot 0 + 2 \cdot 0 + (-1) = -1 \rightarrow 0$

$2 \cdot 0 + 2 \cdot 1 + (-1) = 1 \rightarrow 1$

$2 \cdot 1 + 2 \cdot 0 + (-1) = 1 \rightarrow 1$

$2 \cdot 1 + 2 \cdot 1 + (-1) = 3 \rightarrow 1$

KNOWLEDGEGATE

formula to change weights

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta (t - o) \cdot x_i$$

η = learning rate ($0.1 \leftrightarrow 1$)

t = target o/p

o = actual o/p

x_i = i/p associated with w_i

$$w_1 \leftarrow w_1 + \Delta w_1$$

Row 2 ($x_1 = 0$, $x_2 = 1$)

$$\Delta w_1 = \eta \cdot (t - o) \cdot x_1$$

$$\Delta w_1 = \eta \cdot e \cdot x_1$$

$$= .1 \times (1 - 0) \cdot 0$$

$$= .1 \times 1 \times 0$$

$$= 0$$

$$w_1 = 1 + 0 = 1$$

$$w_2 \leftarrow w_2 + \Delta w_2$$

Row 2 ($x_1 = 0$, $x_2 = 1$)

$$\Delta w_2 = \eta \cdot (t - o) \cdot x_2$$

$$\Delta w_2 = \eta \cdot e \cdot x_2$$

$$= .1 \times 1 \times 1$$

$$= .1$$

$$w_2 = 1 + .1 = 1.1$$

$$\Delta b = \eta \cdot e = .1 \times 1 = 0.1$$

$$b = b + \Delta b = -1 + .1 = -0.9$$

$\rightarrow$ $w_1 = 1$, $w_2 = 1.1$, $b = -0.9$

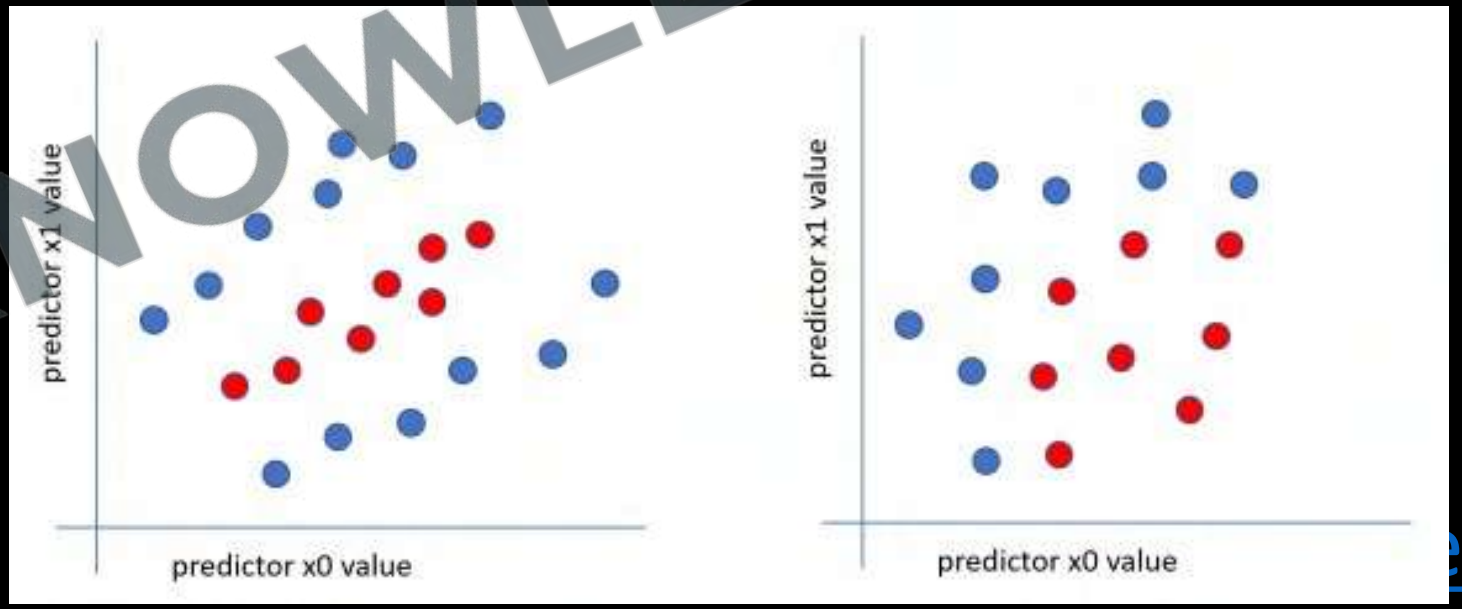
KNOWLEDGEGATE

Linearly Separable Data

- **Definition**: Data that can be perfectly separated by a straight line.
- **Algorithm**: Perceptron adjusts weights and bias to minimize classification error.
- **Outcome**: Converges to find an optimal decision boundary.

Non-Linearly Separable Data

- **Definition**: Data that cannot be perfectly separated by a single straight line.
- **Algorithm Limitation**: Perceptron may not converge, continuously adjusting weights and bias without finding a perfect solution.



KNOWLEDGEGATE

Key Point

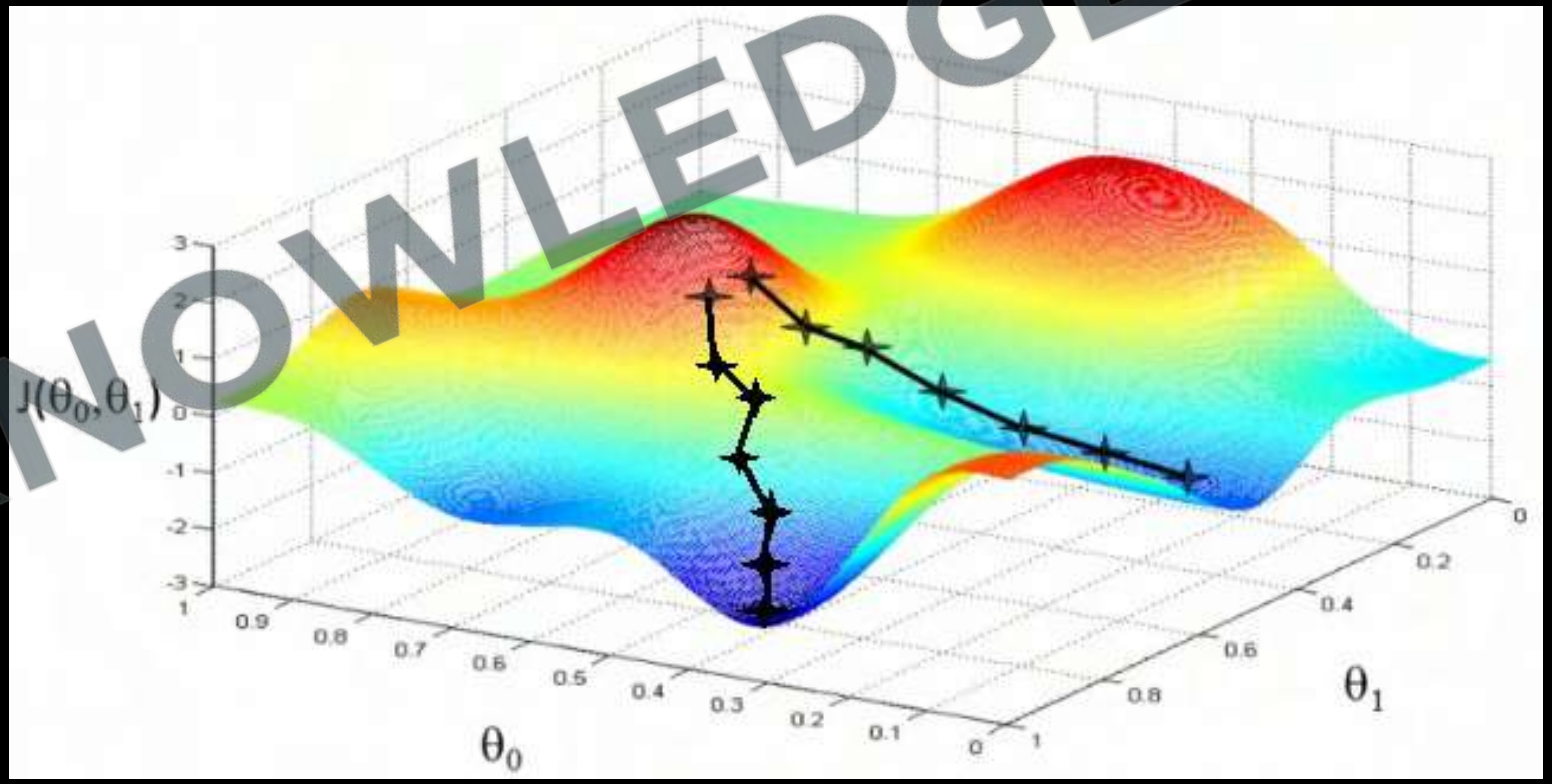
- The Perceptron rule is effective for linearly separable data.
- For non-linearly separable data, more advanced methods like the delta rule (gradient descent) or multi-layer neural networks are required.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Gradient Descent Learning Algorithm

- It is the most used algorithm to train neural networks.
- An optimization algorithm to find model parameters (coefficients, weights) that minimize the error on the training dataset.
- It achieves this by making changes to the model that move it along a gradient/slope of errors towards a minimum error value.



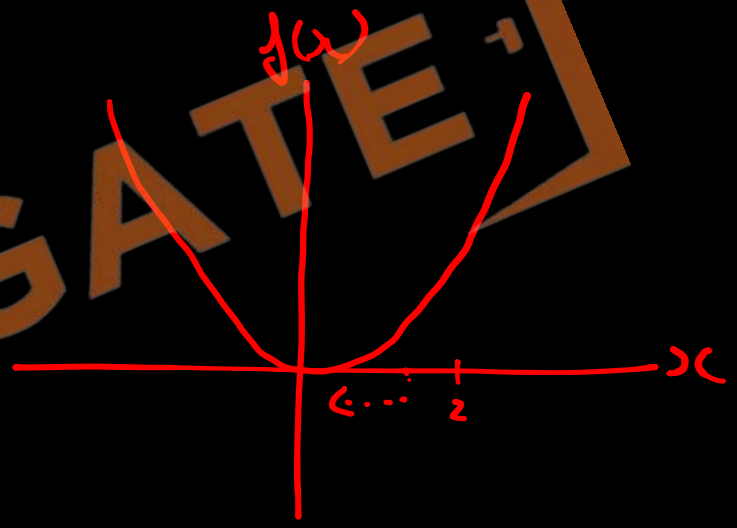
KNOWLEDGE GATE

- **Working**:
  - Adjusts model parameters iteratively to minimize the error.
  - Gradient descent can vary in terms of the training patterns used to calculate errors and update the model.

$f(x) = x^2$ $\quad L.R = .1$

$\downarrow$

$\frac{d(x^2)}{dx} = 2x = \text{slope}$



New value = old value − slope * Learning Rate

$= 2 - 2(2) \times .1$

$= 2 - 4 * .1$

$= 2 - .4 = 1.6$

KNOWLEDGEGATE

$w_i \leftarrow w_i + \Delta w_i$

$\Delta w_i = -\eta \nabla E(w)$ — (1) ← gradient of E

$o_d = \sum w_i x_i$

$= w_0 x_0 + w_1 x_1 + w_2 x_2 \cdots = w_n x_n$

$\sum_{i=1}^{n}$

$\nabla E(w)$ → derivative of error wrt weights

also called as gradient

D is the set of training example, t_d is target o/p for example d training

$$\nabla E(w) = \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots, \frac{\partial E}{w_n} \right]$$

$$E = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

$$\frac{\partial E}{\partial w_i} = \frac{\partial}{\partial w_i} \left(\frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \right)$$

$$= \frac{1}{2} \frac{\partial}{\partial w_i} \sum (t_d - o_d)^2$$

$$= \frac{1}{2} \times 2 \sum (t_d - o_d)^{1} \frac{\partial}{\partial w_i} (t_d - o_d)$$

$$= \sum (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - w_d \cdot x_d)$$

$$= \sum (t_d - o_d)(-x_d)$$

putting it in eqn (1)

$$\Delta w_i = -\eta \times \sum (t_d - o_d)(-x_d)$$

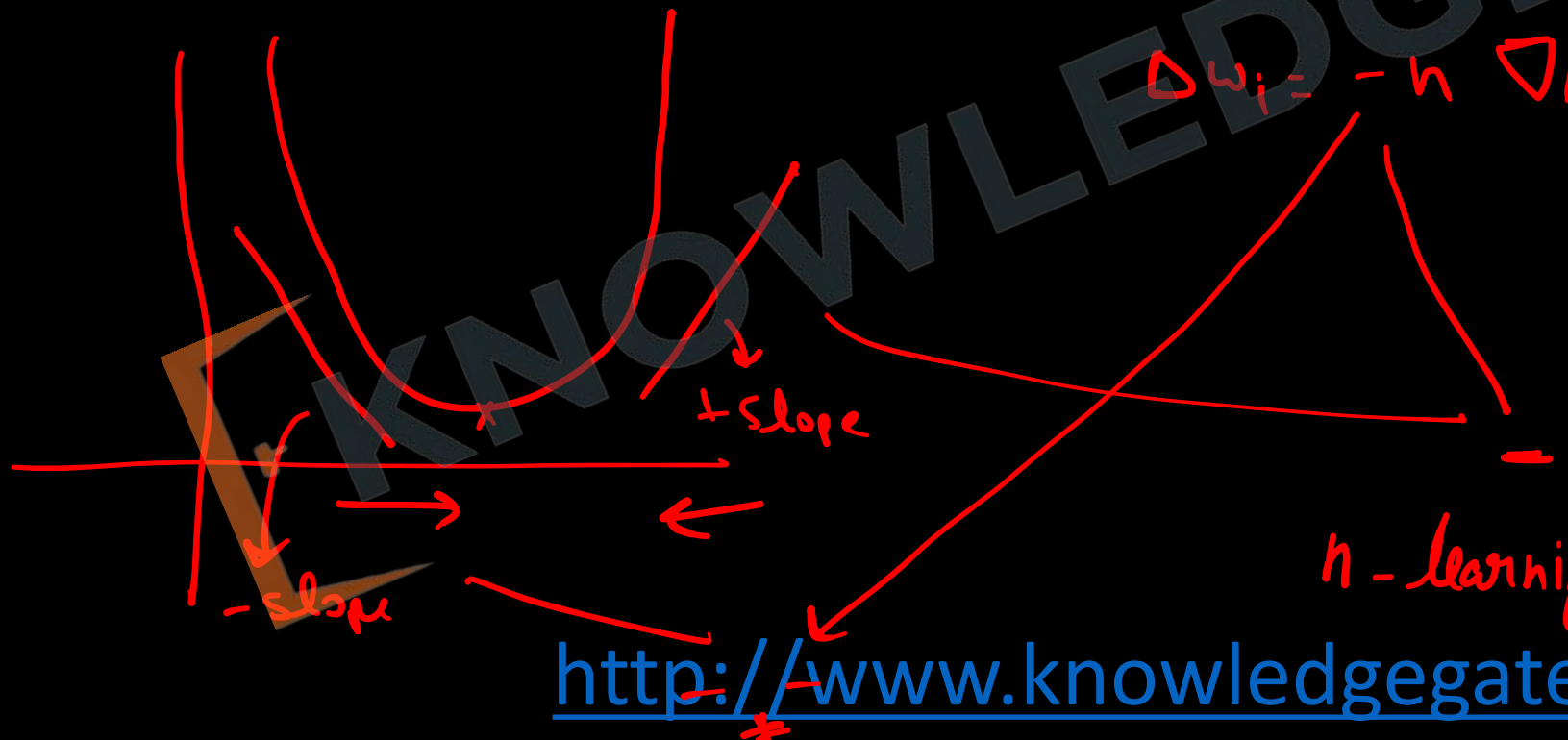
KNOWLEDGEGATE
http://www.knowledgegate.in/gate

$\Delta w_i = \eta \sum (t_d - O_d) x_{id}$

- η : learning rate
- t_d : target o/p
- O_d : actual o/p
- x_{id} : i/p for respective weight
- $d \in D$

This rule also provide the basis for Back Propagation alg.

$$\Delta w_i = -\eta \nabla E(w)$$



η - learning rate is also very imp

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

The main practical challenges in applying gradient descent are:

- Converging to a local minimum can be quite slow, often requiring many thousands of gradient descent steps.
- With multiple local minima in the error surface, there is no guarantee that the procedure will find the global minimum.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Types of Gradient Descent Algorithms

- **Batch Gradient Descent**:
  - Calculates the error for each example in the training dataset but updates the model after all training examples are evaluated.
  - Decreased update frequency results in more stable error gradients and computational efficiency but can be slow and expensive for large datasets.

KNOWLEDGE GATE

- **Stochastic Gradient Descent (SGD)**:
  - Calculates the error for random examples in the training dataset instead of all examples.
  - Faster than batch gradient descent as it updates the model parameters more frequently.

$$w_i = w_i + \eta(t - o)x_i$$ ← based on current example

KNOWLEDGE GATE

- **Mini Batch Gradient Descent**:
  - Splits the training dataset into smaller batches to calculate model error and update parameters.
  - The update frequency is higher than batch gradient descent but less than stochastic gradient descent.
  - Commonly used in deep learning due to a balance of efficiency and performance.

KNOWLEDGEGATE

Delta Learning Rule

- **Definition**:
  - Developed by Widrow & Hoff. States that the modification in the weight of a node is equal to the multiplication of the error and the input.

- **Key Concepts**:
  - **Error**: The difference between the target and the actual output value.
  - Also Known As: LMS (Least Mean Square) Learning Rule.

KNOWLEDGEGATE

Formula:

- $\Delta w = \eta \times (\text{error}) \times (\text{input})$
- $\Delta w = \eta \times (\text{target} - \text{actual}) \times \text{input}$
- New weight: $w_{\text{new}} = w_{\text{old}} + \Delta w$
- η: Learning rate.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

- **Characteristics**:
  - Used in supervised training models.
  - Independent of the activation function.
  - Update rule for a single layer perceptron.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

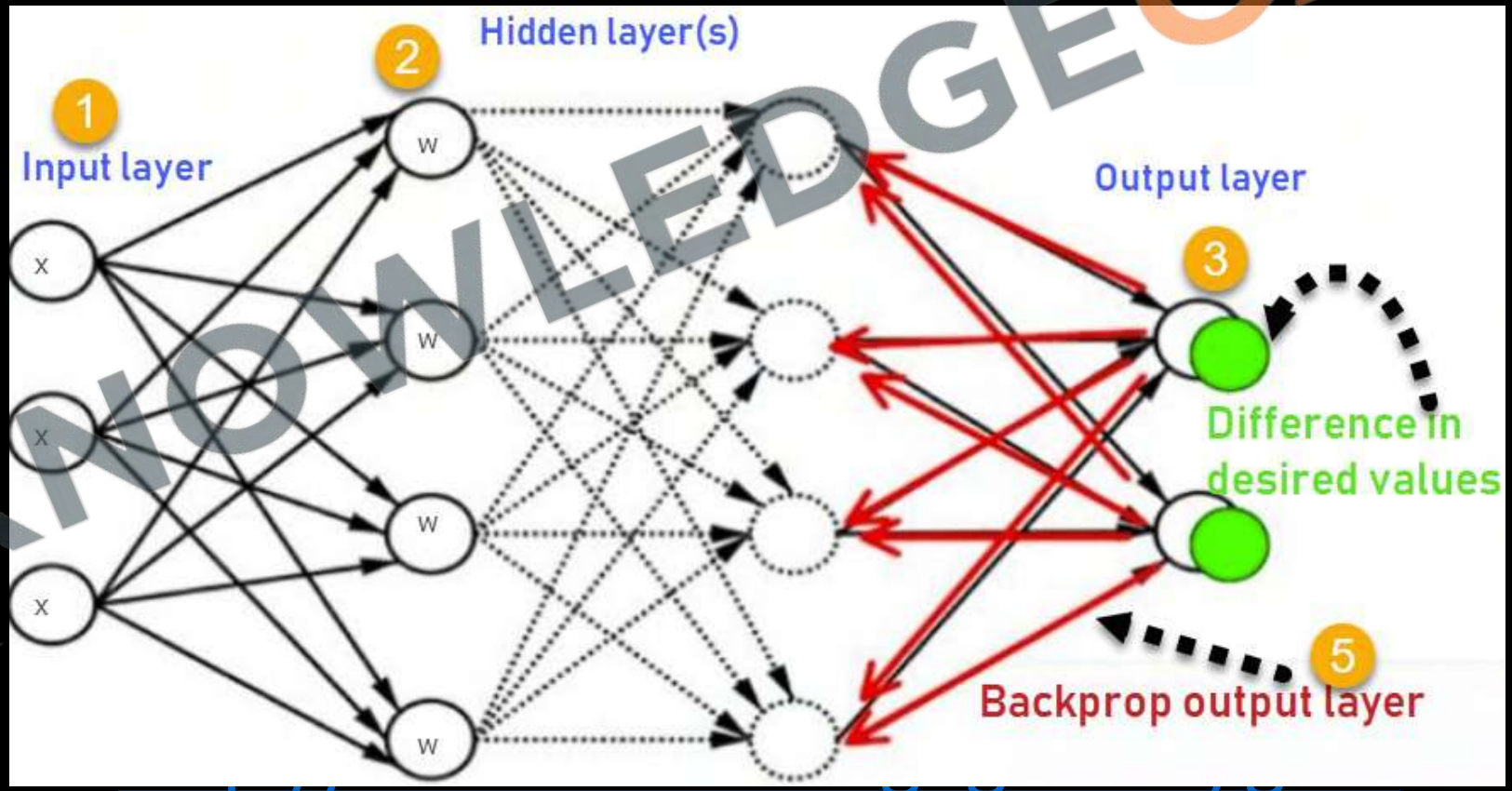
Steps in Delta Learning Rule

- Initialize weights with random values.
- Apply perceptron to each training sample i:
  - If sample i is misclassified, modify all weights:
    - $w_j = w_i + \eta(t_i - y_i)x_{ij}$
- Continue until all samples are correctly classified or the process converges.

KNOWLEDGEGATE

Back Propagation

- **Definition**:
  - A standard method for training artificial neural networks.
  - Repeatedly adjusts the weights of the connections in the network to minimize the difference between actual output and desired output.



KNOWLEDGE GATE

- **Steps in Back Propagation Algorithm**:
  - Inputs X arrive through the precomputed path.
  - Neurons are modelled using randomly assigned weights W.
  - Calculate the output for each neuron from the input layer, through the hidden layers, to the output layer.
  - Calculate the error in the outputs:Error=Actual Output−Desired
  - Travel back from the output layer to the hidden layer to adjust the weights such that the error is decreased.
  - Repeat the process until the desired output is achieved.

KNOWLEDGEGATE

Advantages of Back Propagation

- **Speed**: Fast, simple, and easy to program.
- **Flexibility**: Does not require prior knowledge about the network.
- **Utility**: Mainly useful for deep neural networks working on error-prone projects such as image or speech recognition.

KNOWLEDGEGATE

Step 1: Forward Pass

Input to Hidden Layer

- Inputs: $x_1 = 0.3, x_2 = 0.8$
- Weights:
  - $W_{13} = 0.1$
  - $W_{14} = 0.5$
  - $W_{23} = 0.9$
  - $W_{24} = 0.5$

Calculate the input to hidden neurons H_3 and H_4:

$$H_3 = \sigma(x_1 \cdot W_{13} + x_2 \cdot W_{23})$$
$$H_4 = \sigma(x_1 \cdot W_{14} + x_2 \cdot W_{24})$$

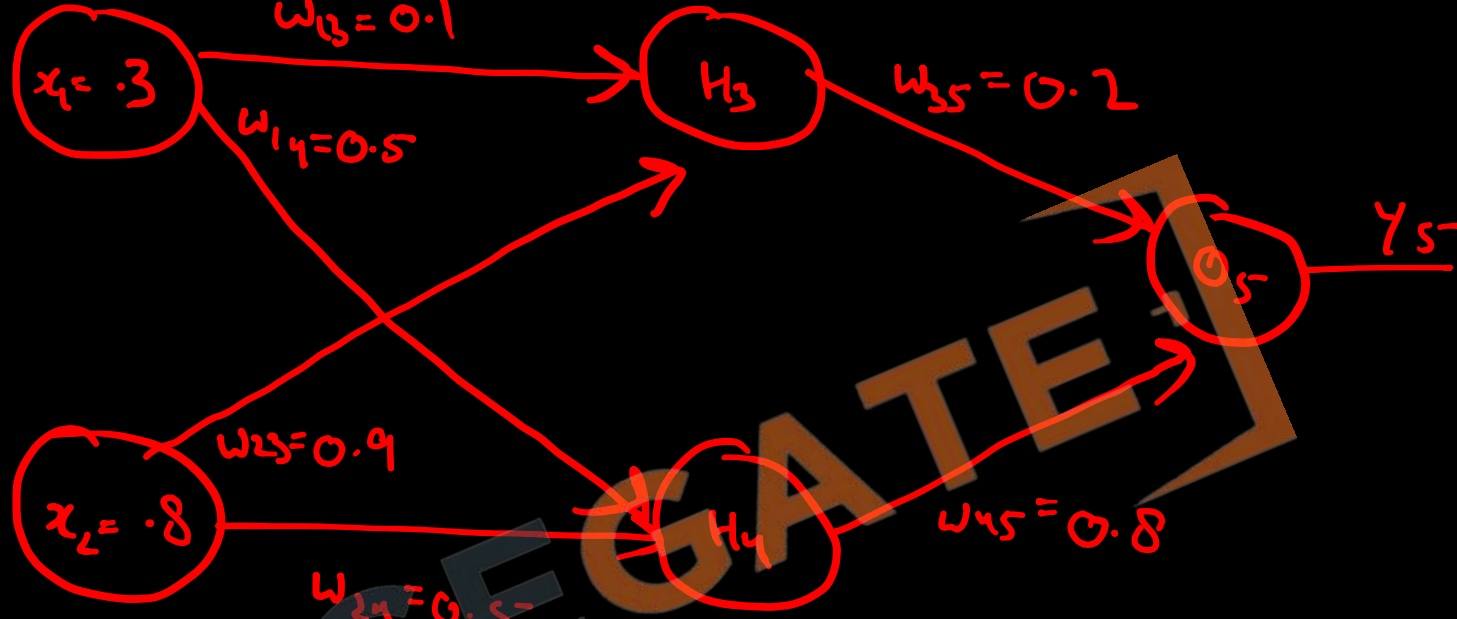
Where σ is the sigmoid function:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Calculations:

$$H_3 = \sigma(0.3 \cdot 0.1 + 0.8 \cdot 0.9)$$
$$H_3 = \sigma(0.03 + 0.72)$$
$$H_3 = \sigma(0.75)$$
$$H_3 = \frac{1}{1+e^{-0.75}} \approx 0.679$$

$$H_4 = \sigma(0.3 \cdot 0.5 + 0.8 \cdot 0.5)$$
$$H_4 = \sigma(0.15 + 0.4)$$
$$H_4 = \sigma(0.55)$$
$$H_4 = \frac{1}{1+e^{-0.55}} \approx 0.634$$



KNOWLEDGEGATE

knowledgegate.in/gate

Hidden to Output Layer

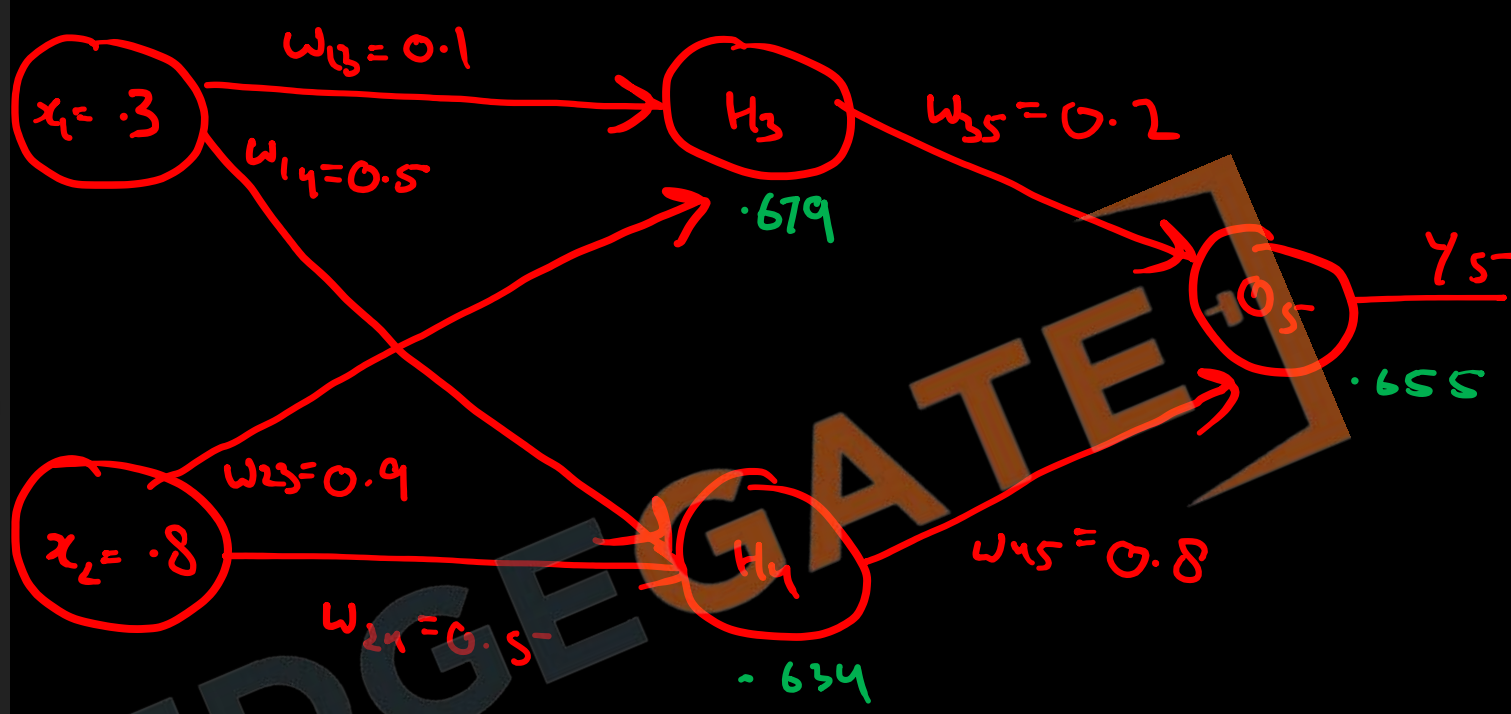
- Weights:
  - $W_{35} = 0.2$
  - $W_{45} = 0.8$

Calculate the input to output neuron O_5:

$$O_5 = \sigma(H_3 \cdot W_{35} + H_4 \cdot W_{45})$$

Calculations:

$$O_5 = \sigma(0.679 \cdot 0.2 + 0.634 \cdot 0.8)$$
$$O_5 = \sigma(0.1358 + 0.5072)$$
$$O_5 = \sigma(0.643)$$
$$O_5 = \frac{1}{1+e^{-0.643}} \approx 0.655$$



KNOWLEDGEGATE

Step 2: Backward Pass

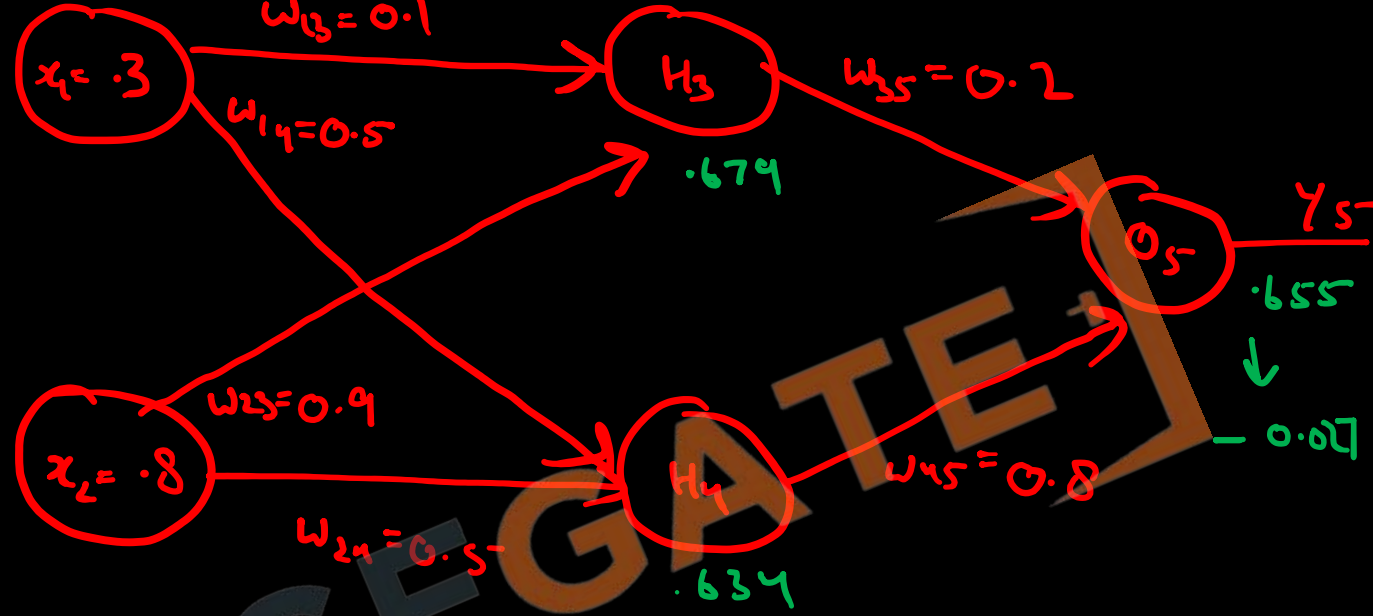
Given:

- Actual output $y = 0.5$
- Learning rate $\eta = 1$

Calculate the error and update weights:

Error at Output Neuron

$$\delta_5 = (y - O_5) \cdot O_5 \cdot (1 - O_5)$$
$$\delta_5 = (0.5 - 0.655) \cdot 0.655 \cdot (1 - 0.655)$$
$$\delta_5 = -0.027$$



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Update Weights from Hidden to Output Layer

$$\Delta W_{35} = \eta \cdot \delta_5 \cdot H_3$$
$$\Delta W_{35} = 1 \cdot -0.027 \cdot 0.679$$
$$\Delta W_{35} = -0.018$$
$$W_{35} = 0.2 - 0.018 = 0.182$$

$$\Delta W_{45} = \eta \cdot \delta_5 \cdot H_4$$
$$\Delta W_{45} = 1 \cdot -0.027 \cdot 0.634$$
$$\Delta W_{45} = -0.017$$
$$W_{45} = 0.8 - 0.017 = 0.783$$

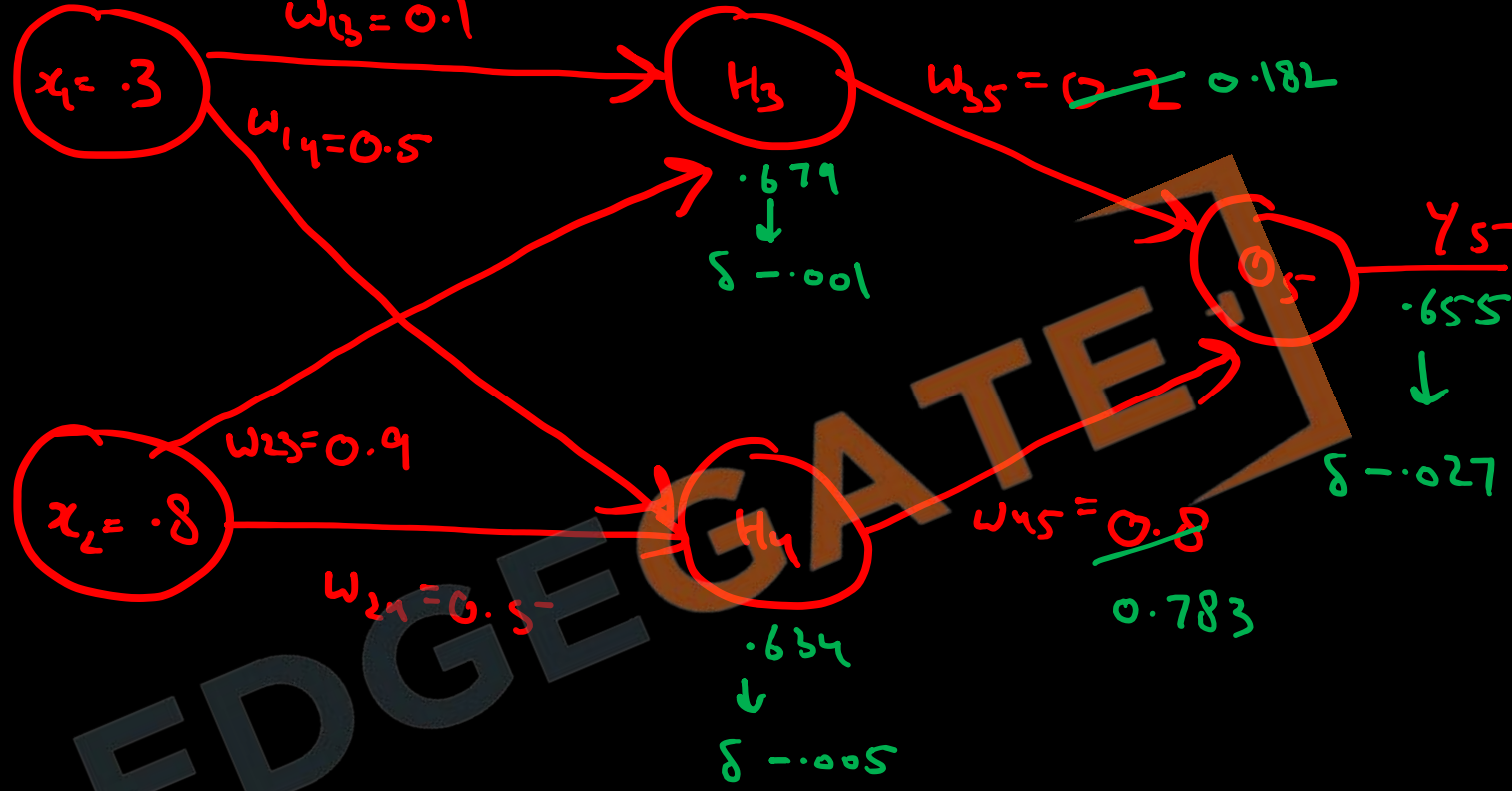
Error at Hidden Neurons

$$\delta_3 = \delta_5 \cdot W_{35} \cdot H_3 \cdot (1 - H_3)$$
$$\delta_4 = \delta_5 \cdot W_{45} \cdot H_4 \cdot (1 - H_4)$$

Calculations:

$$\delta_3 = -0.027 \cdot 0.2 \cdot 0.679 \cdot (1 - 0.679)$$
$$\delta_3 = -0.001$$

$$\delta_4 = -0.027 \cdot 0.8 \cdot 0.634 \cdot (1 - 0.634)$$
$$\delta_4 = -0.005$$



KNOWLEDGEGATE

www.knowledgegate.in/gate

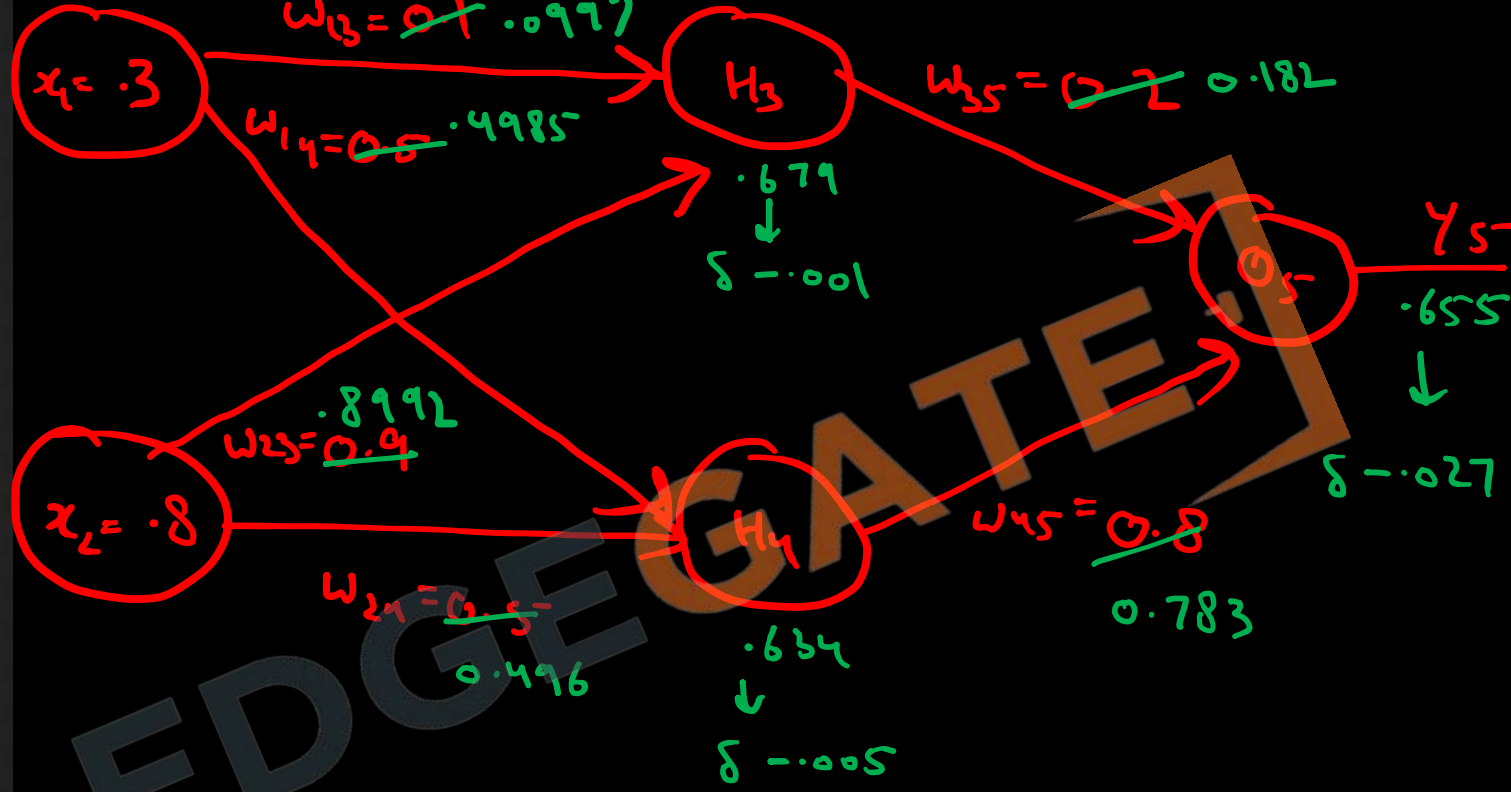
Update Weights from Input to Hidden Layer

$$\Delta W_{13} = \eta \cdot \delta_3 \cdot x_1$$
$$\Delta W_{13} = 1 \cdot -0.001 \cdot 0.3$$
$$\Delta W_{13} = -0.0003$$
$$W_{13} = 0.1 - 0.0003 = 0.0997$$

$$\Delta W_{23} = \eta \cdot \delta_3 \cdot x_2$$
$$\Delta W_{23} = 1 \cdot -0.001 \cdot 0.8$$
$$\Delta W_{23} = -0.0008$$
$$W_{23} = 0.9 - 0.0008 = 0.8992$$

$$\Delta W_{14} = \eta \cdot \delta_4 \cdot x_1$$
$$\Delta W_{14} = 1 \cdot -0.005 \cdot 0.3$$
$$\Delta W_{14} = -0.0015$$
$$W_{14} = 0.5 - 0.0015 = 0.4985$$

$$\Delta W_{24} = \eta \cdot \delta_4 \cdot x_2$$
$$\Delta W_{24} = 1 \cdot -0.005 \cdot 0.8$$
$$\Delta W_{24} = -0.004$$
$$W_{24} = 0.5 - 0.004 = 0.496$$



KNOWLEDGEGATE

.knowledgegate.in/gate

Step 3: Perform Another Forward Pass

Using the updated weights:

New Weights:

- $W_{13} = 0.0997$
- $W_{14} = 0.4985$
- $W_{23} = 0.8992$
- $W_{24} = 0.496$
- $W_{35} = 0.182$
- $W_{45} = 0.783$

Forward Pass with Updated Weights

Calculate new hidden layer activations:

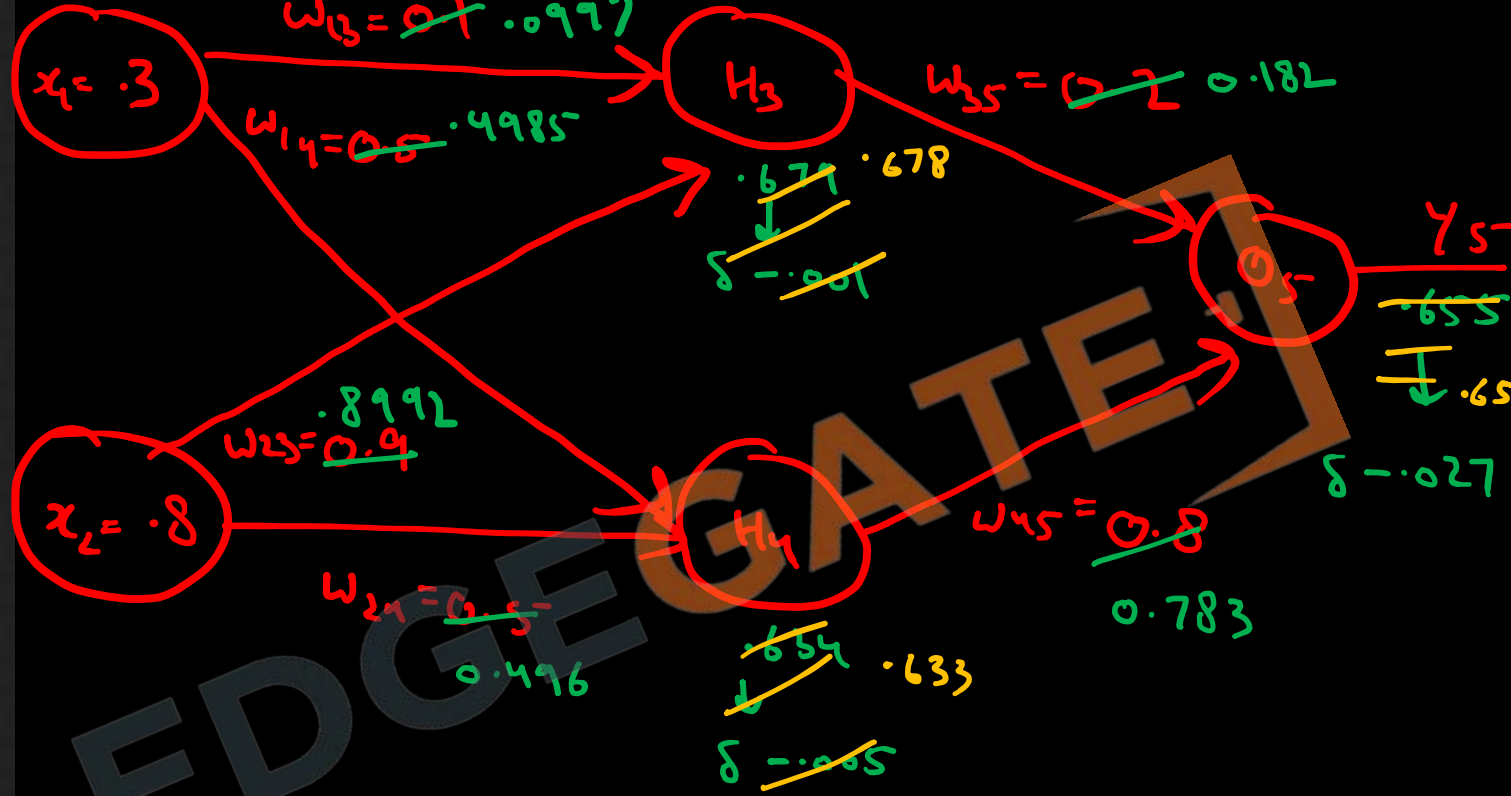
$$H_3 = \sigma(0.3 \cdot 0.0997 + 0.8 \cdot 0.8992)$$
$$H_3 = \sigma(0.02991 + 0.71936)$$
$$H_3 = \sigma(0.74927)$$
$$H_3 \approx 0.678$$

$$H_4 = \sigma(0.3 \cdot 0.4985 + 0.8 \cdot 0.496)$$
$$H_4 = \sigma(0.14955 + 0.3968)$$
$$H_4 = \sigma(0.54635)$$
$$H_4 \approx 0.633$$

Calculate new output layer activation:

$$O_5 = \sigma(H_3 \cdot W_{35} + H_4 \cdot W_{45})$$
$$O_5 = \sigma(0.678 \cdot 0.182 + 0.633 \cdot 0.783)$$
$$O_5 = \sigma(0.123396 + 0.495189)$$
$$O_5 = \sigma(0.618585)$$
$$O_5 \approx 0.650$$

Thus, the output y after performing another forward pass with the updated weights is approximately 0.650.

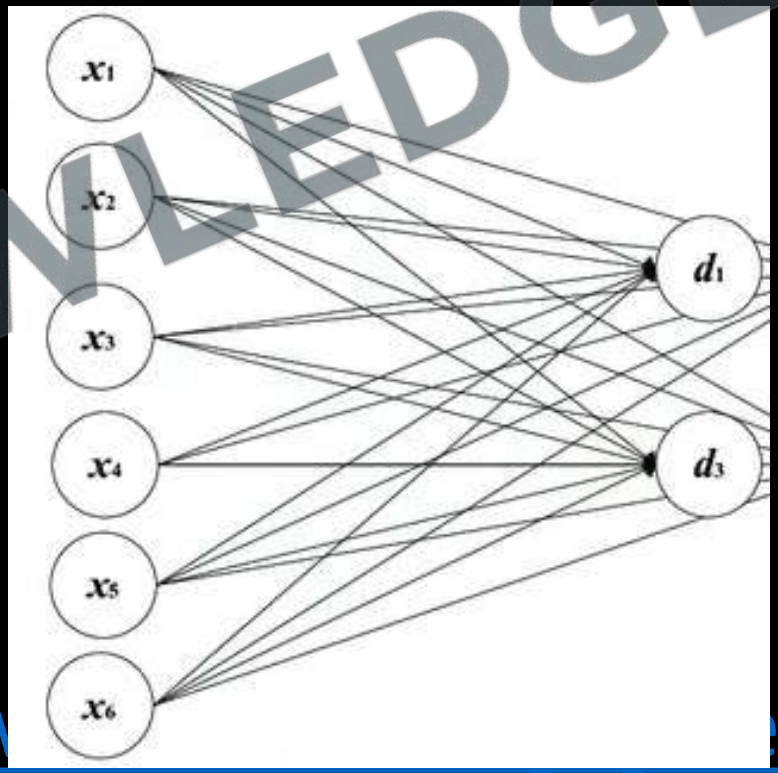


KNOWLEDGEGATE

.knowledgegate.in/gate

Self-Organizing Maps (SOM) / Kohonene Self Organizing Maps

- Developed by T. Kohonen in 1982. Named "Self-organizing" because no supervision is required. Learn through unsupervised competitive learning.
- A Kind of neural network which uses Competitive learning algorithm to maps multidimensional data into lower dimensions for easier interpretation specially for complex problems. It will contain only two layers
  - Input layer (n units)
  - Output layer (m units)



KNOWLEDGEGATE

http://w .in/gate

Example of Dimensionality Reduction Using SOMs

Scenario: Visualizing customer data with features like age, income, spending score, and product preferences.

Goal: Reduce dimensionality to visualize patterns in customer behavior.

1. **High-Dimensional Data:**
   Each customer is represented as a vector:

$$\mathbf{c_1} = [35, 50000, 85, 1, 0, 0, 1, ...]$$

   (age, income, spending score, product preferences)

2. **Using SOM for Dimensionality Reduction**:
   - Train a SOM with the customer data.
   - Project the high-dimensional data onto a 2D grid, preserving relationships.

3. **2D Grid Visualization:**
   The SOM organizes customers into clusters:

diff Copy code

```
+-----------+-----------+-----------+
| Cluster 1 | Cluster 2 | Cluster 3 |
+-----------+-----------+-----------+
| Cluster 4 | Cluster 5 | Cluster 6 |
+-----------+-----------+-----------+
```

KNOWLEDGEGATE
egate.in/gate

4. **Interpreting Clusters:**

   - **Cluster 1**: Young, high income, high spending.
   - **Cluster 2**: Middle-aged, moderate income and spending.
   - **Cluster 3**: Older, low income, high spending on specific products.

5. **Applications:**

   - **Marketing**: Target personalized campaigns.
   - **Product Development**: Develop products for different customer segments.
   - **Customer Support**: Tailor strategies for different clusters.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

SOM Training Algorithm:

1. **Initialization**: Choose random values for the initial weights w_{ij}.
2. **Sampling**: Take a sample training input vector $x(t)$ from the input layer.
3. **Matching**: Find the winning neuron from the output layer that has the weight vector closest to the input vector. It can be calculated by taking the square of the Euclidean distance for each output unit and finding the output unit that has the minimum Euclidean distance from the input vector.

$$D_j = \sum_{i=1}^{n} (x_i - w_{ij})^2 \quad \text{for each } j = 1 \text{ to } m$$

4. **New Weight Calculation**: Find the new weights between the input vector sample and the winning output unit (neuron).

$$w_{ij}(new) = w_{ij}(old) + \eta(t) \cdot (x_i - w_{ij}(old))$$

where $\eta(t)$ is the learning rate.

5. **Continuation**: Repeat steps 2 to 4 until weight updation is negligible (i.e., new weights are similar to old weights or feature map stops changing).

KNOWLEDGEGATE

Training Samples:

x_1 (0, 1, 0, 1)

x_2 (0, 1, 1, 1)

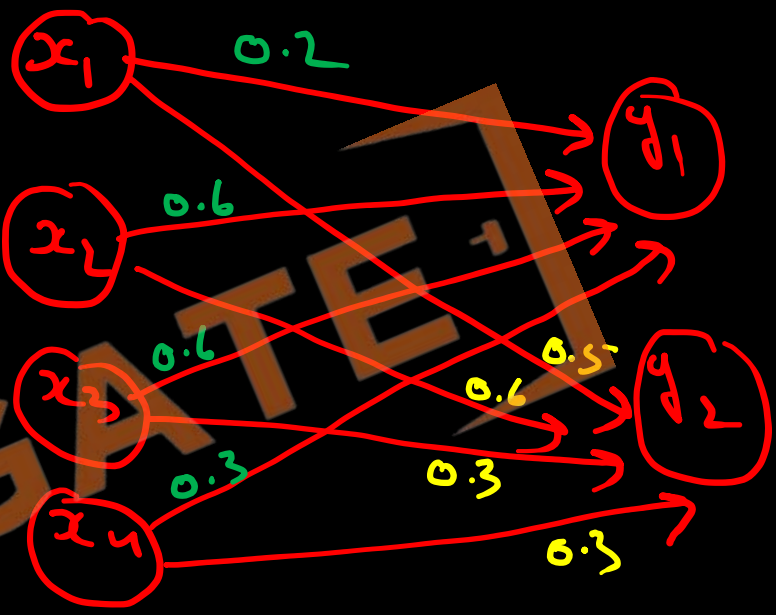
x_3 (0, 0, 0, 0)

x_4 (1, 0, 0, 1)

Learning rate = 0.5

initial weight matrix

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} = \begin{bmatrix} 0.2 & 0.6 & 0.6 & 0.3 \\ 0.5 & 0.6 & 0.3 & 0.3 \end{bmatrix}$$



KNOWLEDGEGATE

To compute the Euclidean distance between the input vector $\mathbf{x_1} = [0, 1, 0, 1]$ and the weight vector of Unit 1, we use the formula for Euclidean distance:

$$D = \sqrt{\sum_{i=1}^{n} (x_i - w_i)^2}$$

Given:

- $\mathbf{x_1} = [0, 1, 0, 1]$
- Weight vector for Unit 1: $\mathbf{w_{unit1}} = [0.2, 0.6, 0.6, 0.3]$

Let's compute the Euclidean distance step-by-step:

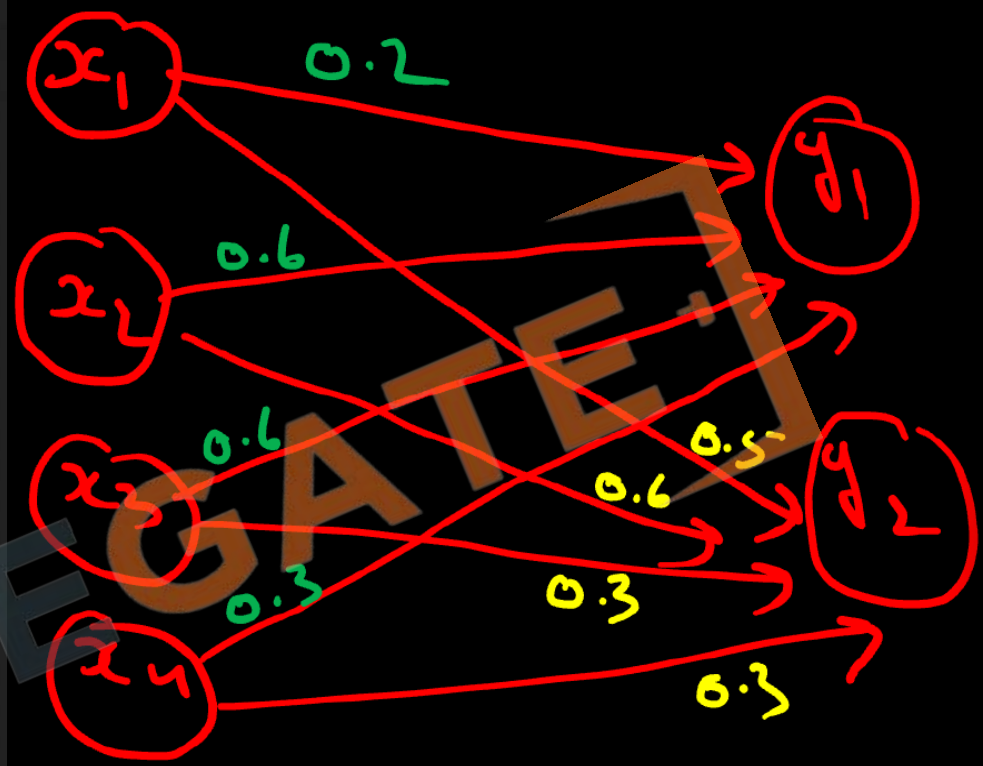
1. Compute the difference for each component:
   - $(0 - 0.2)^2 = 0.04$
   - $(1 - 0.6)^2 = 0.16$
   - $(0 - 0.6)^2 = 0.36$
   - $(1 - 0.3)^2 = 0.49$
2. Sum the squared differences:

$$0.04 + 0.16 + 0.36 + 0.49 = 1.05$$

3. Take the square root of the sum:

$$D = \sqrt{1.05} \approx 1.0247$$

So, the Euclidean distance between $\mathbf{x_1}$ and the weight vector of Unit 1 is approximately 1.0247.



KNOWLEDGE GATE

e.in/gate

To compute the Euclidean distance between the input vector $\mathbf{x_1} = [0, 1, 0, 1]$ and the weight vector of Unit 2, we will use the same formula:

$$D = \sqrt{\sum_{i=1}^{n} (x_i - w_i)^2}$$

Given:

- $\mathbf{x_1} = [0, 1, 0, 1]$
- Weight vector for Unit 2: $\mathbf{w_{unit2}} = [0.5, 0.6, 0.3, 0.3]$

Let's compute the Euclidean distance step-by-step:

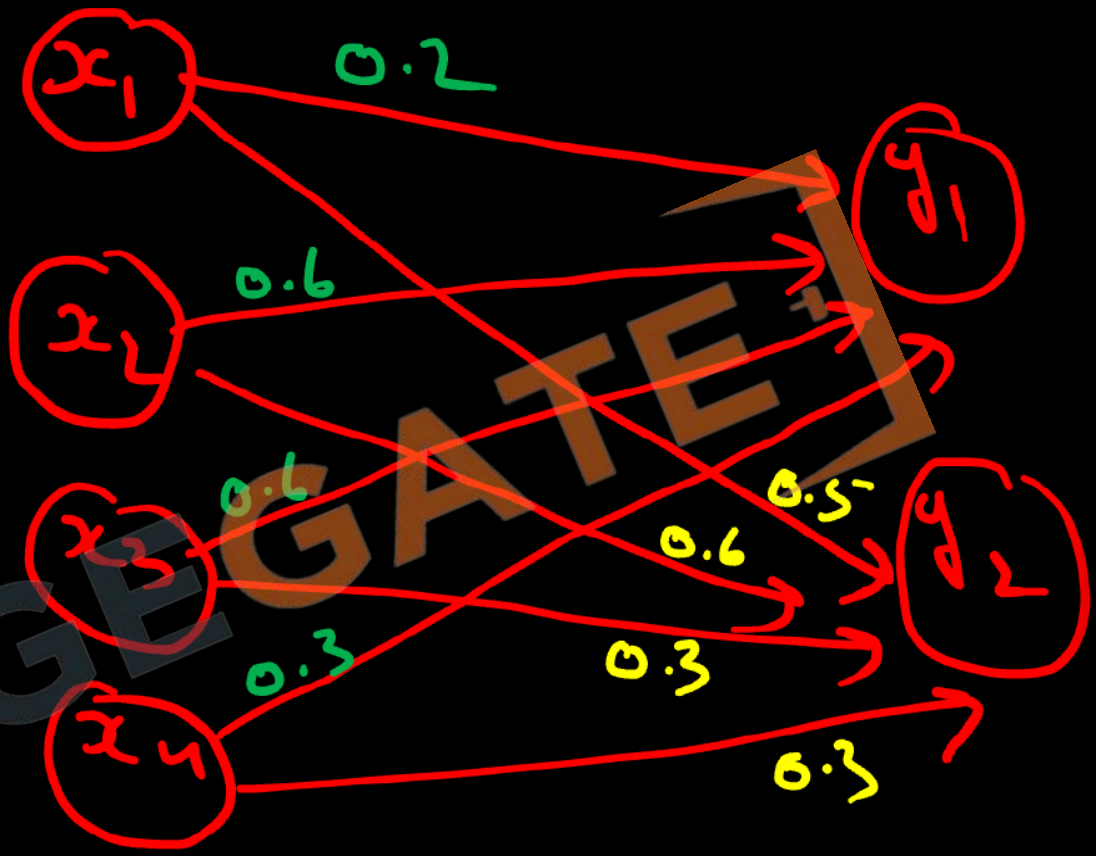
1. Compute the difference for each component:
   - $(0 - 0.5)^2 = 0.25$
   - $(1 - 0.6)^2 = 0.16$
   - $(0 - 0.3)^2 = 0.09$
   - $(1 - 0.3)^2 = 0.49$
2. Sum the squared differences:

$$0.25 + 0.16 + 0.09 + 0.49 = 0.99$$

3. Take the square root of the sum:

$$D = \sqrt{0.99} \approx 0.995$$

So, the Euclidean distance between $\mathbf{x_1}$ and the weight vector of Unit 2 is approximately 0.995.



KNOWLEDGEGATE
egate.in/gate

To update the weights of Unit 2 after determining that it is the winning unit, we use the formula:

$$w_{ij}(new) = w_{ij}(old) + \eta \cdot (x_i - w_{ij}(old))$$

Given:

- $\mathbf{x_1} = [0, 1, 0, 1]$
- Initial weights for Unit 2: $\mathbf{w_{unit2}} = [0.5, 0.6, 0.3, 0.3]$
- Learning rate (η) = 0.5

Let's update each weight step-by-step:

1. For the first weight w_{21}:

$$w_{21}(new) = 0.5 + 0.5 \cdot (0 - 0.5) = 0.5 + 0.5 \cdot (-0.5) = 0.5 - 0.25 = 0.25$$

2. For the second weight w_{22}:

$$w_{22}(new) = 0.6 + 0.5 \cdot (1 - 0.6) = 0.6 + 0.5 \cdot 0.4 = 0.6 + 0.2 = 0.8$$

3. For the third weight w_{23}:

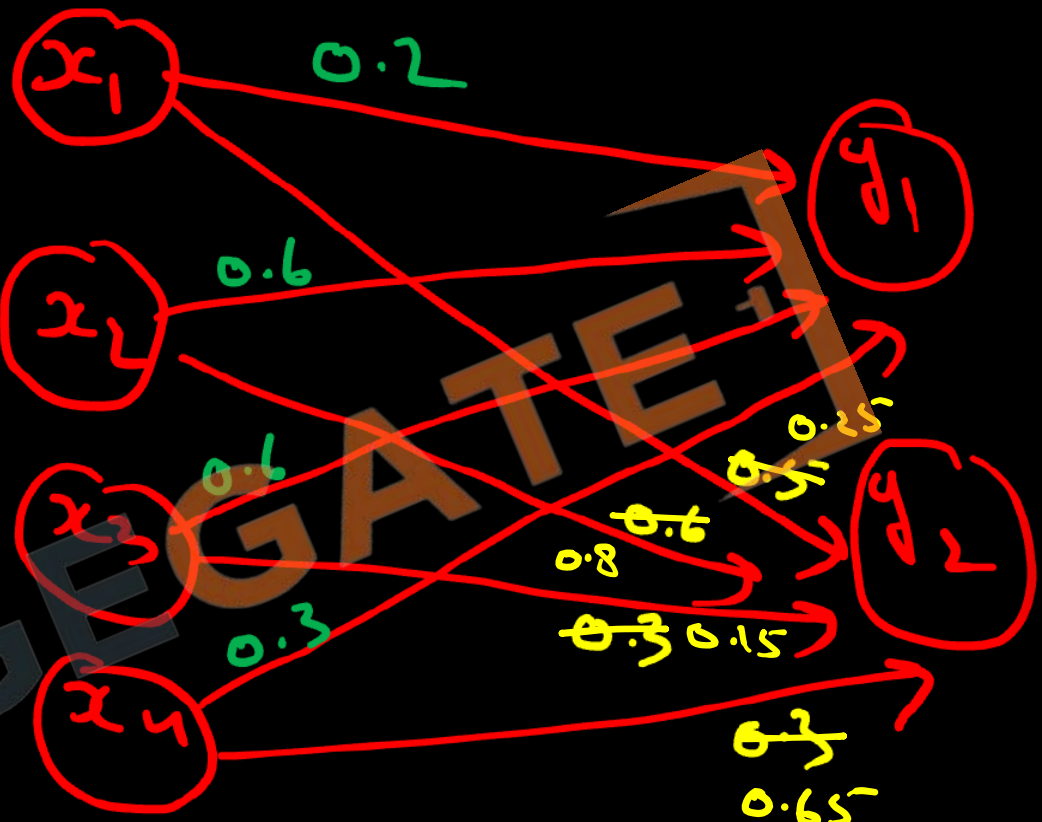
$$w_{23}(new) = 0.3 + 0.5 \cdot (0 - 0.3) = 0.3 + 0.5 \cdot (-0.3) = 0.3 - 0.15 = 0.15$$

4. For the fourth weight w_{24}:

$$w_{24}(new) = 0.3 + 0.5 \cdot (1 - 0.3) = 0.3 + 0.5 \cdot 0.7 = 0.3 + 0.35 = 0.65$$

So, the updated weights for Unit 2 are:

$$\mathbf{w_{unit2}(new)} = [0.25, 0.8, 0.15, 0.65]$$



KNOWLEDGE GATE
gate.in/gate

To continue training with $\mathbf{x_2} = [0, 1, 1, 1]$, we first need to compute the Euclidean distances between $\mathbf{x_2}$ and the updated weight vectors for both Unit 1 and Unit 2.

Euclidean Distance Calculation

1. **Distance between $\mathbf{x_2}$ and Unit 1:**
   - Weight vector for Unit 1: $\mathbf{w_{unit1}} = [0.2, 0.6, 0.6, 0.3]$
   - $\mathbf{x_2} = [0, 1, 1, 1]$

$$D_1 = \sqrt{(0 - 0.2)^2 + (1 - 0.6)^2 + (1 - 0.6)^2 + (1 - 0.3)^2}$$

$$D_1 = \sqrt{0.04 + 0.16 + 0.16 + 0.49} = \sqrt{0.85} \approx 0.922$$

2. **Distance between $\mathbf{x_2}$ and Unit 2:**
   - Updated weight vector for Unit 2: $\mathbf{w_{unit2}} = [0.25, 0.8, 0.15, 0.65]$
   - $\mathbf{x_2} = [0, 1, 1, 1]$

$$D_2 = \sqrt{(0 - 0.25)^2 + (1 - 0.8)^2 + (1 - 0.15)^2 + (1 - 0.65)^2}$$

$$D_2 = \sqrt{0.0625 + 0.04 + 0.7225 + 0.1225} = \sqrt{0.9475} \approx 0.973$$

Determine the Winning Unit

- The Euclidean distance for Unit 1 is approximately 0.922.
- The Euclidean distance for Unit 2 is approximately 0.973.

Since Unit 1 has the smaller distance, Unit 1 is the winning unit this time.

KNOWLEDGE GATE
egate.in/gate

Update Weights for Unit 1

Using the same update formula:

$$w_{ij}(new) = w_{ij}(old) + \eta \cdot (x_i - w_{ij}(old))$$

Given:

- $\mathbf{x_2} = [0, 1, 1, 1]$
- Initial weights for Unit 1: $\mathbf{w_{unit1}} = [0.2, 0.6, 0.6, 0.3]$
- Learning rate (η) = 0.5

1. For the first weight w_{11}:

$$w_{11}(new) = 0.2 + 0.5 \cdot (0 - 0.2) = 0.2 + 0.5 \cdot (-0.2) = 0.2 - 0.1 = 0.1$$

2. For the second weight w_{12}:

$$w_{12}(new) = 0.6 + 0.5 \cdot (1 - 0.6) = 0.6 + 0.5 \cdot 0.4 = 0.6 + 0.2 = 0.8$$

3. For the third weight w_{13}:

$$w_{13}(new) = 0.6 + 0.5 \cdot (1 - 0.6) = 0.6 + 0.5 \cdot 0.4 = 0.6 + 0.2 = 0.8$$

4. For the fourth weight w_{14}:

$$w_{14}(new) = 0.3 + 0.5 \cdot (1 - 0.3) = 0.3 + 0.5 \cdot 0.7 = 0.3 + 0.35 = 0.65$$

So, the updated weights for Unit 1 are:

$$\mathbf{w_{unit1}(new)} = [0.1, 0.8, 0.8, 0.65]$$

KNOWLEDGEGATE
lgegate.in/gate

To continue training with $\mathbf{x_3} = [0, 0, 0, 0]$, we first need to compute the Euclidean distances between $\mathbf{x_3}$ and the updated weight vectors for both Unit 1 and Unit 2.

Euclidean Distance Calculation

1. **Distance between $\mathbf{x_3}$ and Unit 1:**
   - Updated weight vector for Unit 1: $\mathbf{w_{unit1}} = [0.1, 0.8, 0.8, 0.65]$
   - $\mathbf{x_3} = [0, 0, 0, 0]$

$$D_1 = \sqrt{(0 - 0.1)^2 + (0 - 0.8)^2 + (0 - 0.8)^2 + (0 - 0.65)^2}$$

$$D_1 = \sqrt{0.01 + 0.64 + 0.64 + 0.4225} = \sqrt{1.7125} \approx 1.309$$

2. **Distance between $\mathbf{x_3}$ and Unit 2:**
   - Updated weight vector for Unit 2: $\mathbf{w_{unit2}} = [0.25, 0.8, 0.15, 0.65]$
   - $\mathbf{x_3} = [0, 0, 0, 0]$

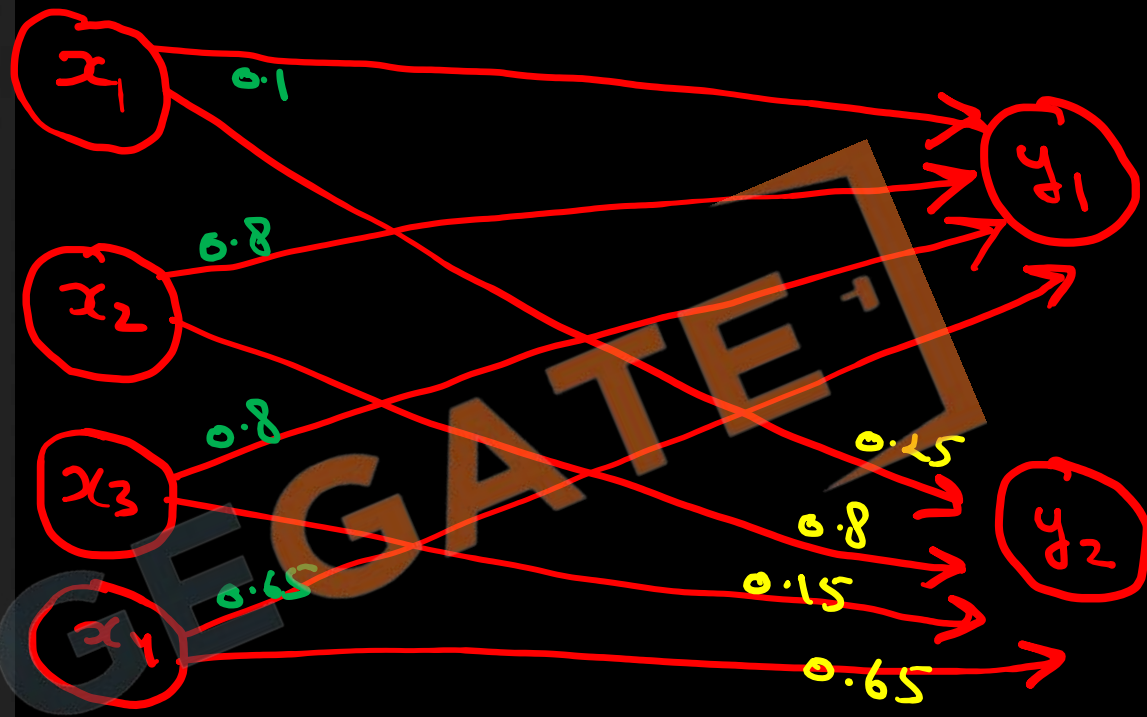
$$D_2 = \sqrt{(0 - 0.25)^2 + (0 - 0.8)^2 + (0 - 0.15)^2 + (0 - 0.65)^2}$$

$$D_2 = \sqrt{0.0625 + 0.64 + 0.0225 + 0.4225} = \sqrt{1.1475} \approx 1.071$$

Determine the Winning Unit

- The Euclidean distance for Unit 1 is approximately 1.309.
- The Euclidean distance for Unit 2 is approximately 1.071.

Since Unit 2 has the smaller distance, Unit 2 is the winning unit this time.



KNOWLEDGE GATE

gegate.in/gate

Update Weights for Unit 2

Using the same update formula:

$$w_{ij}(new) = w_{ij}(old) + \eta \cdot (x_i - w_{ij}(old))$$

Given:

- $\mathbf{x_3} = [0, 0, 0, 0]$
- Updated weights for Unit 2: $\mathbf{w_{unit2}} = [0.25, 0.8, 0.15, 0.65]$
- Learning rate (η) = 0.5

1. For the first weight w_{21}:

$$w_{21}(new) = 0.25 + 0.5 \cdot (0 - 0.25) = 0.25 + 0.5 \cdot (-0.25) = 0.25 - 0.125 = 0.125$$

2. For the second weight w_{22}:

$$w_{22}(new) = 0.8 + 0.5 \cdot (0 - 0.8) = 0.8 + 0.5 \cdot (-0.8) = 0.8 - 0.4 = 0.4$$

3. For the third weight w_{23}:

$$w_{23}(new) = 0.15 + 0.5 \cdot (0 - 0.15) = 0.15 + 0.5 \cdot (-0.15) = 0.15 - 0.075 = 0.075$$

4. For the fourth weight w_{24}:

$$w_{24}(new) = 0.65 + 0.5 \cdot (0 - 0.65) = 0.65 + 0.5 \cdot (-0.65) = 0.65 - 0.325 = 0.325$$

So, the updated weights for Unit 2 are:

$$\mathbf{w_{unit2}(new)} = [0.125, 0.4, 0.075, 0.325]$$

KNOWLEDGEGATE
dgegate.in/gate

To continue training with $\mathbf{x}_4 = [1, 0, 0, 1]$, we first need to compute the Euclidean distances between $\mathbf{x}_4$ and the updated weight vectors for both Unit 1 and Unit 2.

Euclidean Distance Calculation

1. **Distance between $\mathbf{x}_4$ and Unit 1:**
   - Updated weight vector for Unit 1: $\mathbf{w}_{\mathbf{unit1}} = [0.1, 0.8, 0.8, 0.65]$
   - $\mathbf{x}_4 = [1, 0, 0, 1]$

$$D_1 = \sqrt{(1 - 0.1)^2 + (0 - 0.8)^2 + (0 - 0.8)^2 + (1 - 0.65)^2}$$

$$D_1 = \sqrt{(0.9)^2 + (-0.8)^2 + (-0.8)^2 + (0.35)^2}$$

$$D_1 = \sqrt{0.81 + 0.64 + 0.64 + 0.1225} = \sqrt{2.2125} \approx 1.487$$

2. **Distance between $\mathbf{x}_4$ and Unit 2:**
   - Updated weight vector for Unit 2: $\mathbf{w}_{\mathbf{unit2}} = [0.125, 0.4, 0.075, 0.325]$
   - $\mathbf{x}_4 = [1, 0, 0, 1]$

$$D_2 = \sqrt{(1 - 0.125)^2 + (0 - 0.4)^2 + (0 - 0.075)^2 + (1 - 0.325)^2}$$

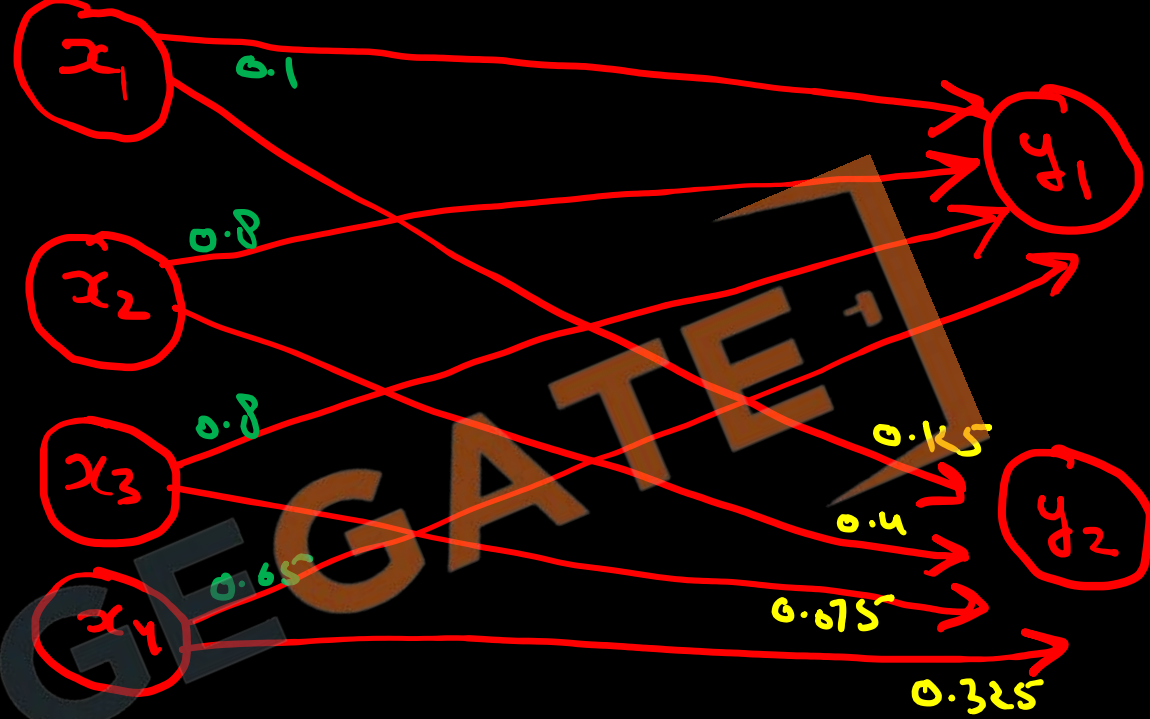
$$D_2 = \sqrt{(0.875)^2 + (-0.4)^2 + (-0.075)^2 + (0.675)^2}$$

$$D_2 = \sqrt{0.765625 + 0.16 + 0.005625 + 0.455625} = \sqrt{1.386875} \approx 1.178$$

Determine the Winning Unit

- The Euclidean distance for Unit 1 is approximately 1.487.
- The Euclidean distance for Unit 2 is approximately 1.178.

Since Unit 2 has the smaller distance, Unit 2 is the winning unit again.



KNOWLEDGE GATE
dgegate.in/gate

Update Weights for Unit 2

Using the same update formula:

$$w_{ij}(new) = w_{ij}(old) + \eta \cdot (x_i - w_{ij}(old))$$

Given:

- $\mathbf{x_4} = [1, 0, 0, 1]$
- Updated weights for Unit 2: $\mathbf{w_{unit2}} = [0.125, 0.4, 0.075, 0.325]$
- Learning rate (η) = 0.5

1. For the first weight w_{21}:

$$w_{21}(new) = 0.125 + 0.5 \cdot (1 - 0.125) = 0.125 + 0.5 \cdot 0.875 = 0.125 + 0.4375 = 0.5$$

2. For the second weight w_{22}:

$$w_{22}(new) = 0.4 + 0.5 \cdot (0 - 0.4) = 0.4 + 0.5 \cdot (-0.4) = 0.4 - 0.2 = 0.2$$

3. For the third weight w_{23}:

$$w_{23}(new) = 0.075 + 0.5 \cdot (0 - 0.075) = 0.075 + 0.5 \cdot (-0.075) = 0.075 - 0.0375 =$$

4. For the fourth weight w_{24}:

$$w_{24}(new) = 0.325 + 0.5 \cdot (1 - 0.325) = 0.325 + 0.5 \cdot 0.675 = 0.325 + 0.3375 = 0.6$$

So, the updated weights for Unit 2 are:

$$\mathbf{w_{unit2}(new)} = [0.5625, 0.2, 0.0375, 0.6625]$$

KNOWLEDGEGATE

dgegate.in/gate

After the training process with the given samples $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4$, the final weights for Unit 1 and Unit 2 are:

Final Weights

Unit 1

Updated after training with $\mathbf{x}_2$:

$$\mathbf{w}_{\text{unit1}}(\text{new}) = [0.1, 0.8, 0.8, 0.65]$$

Unit 2

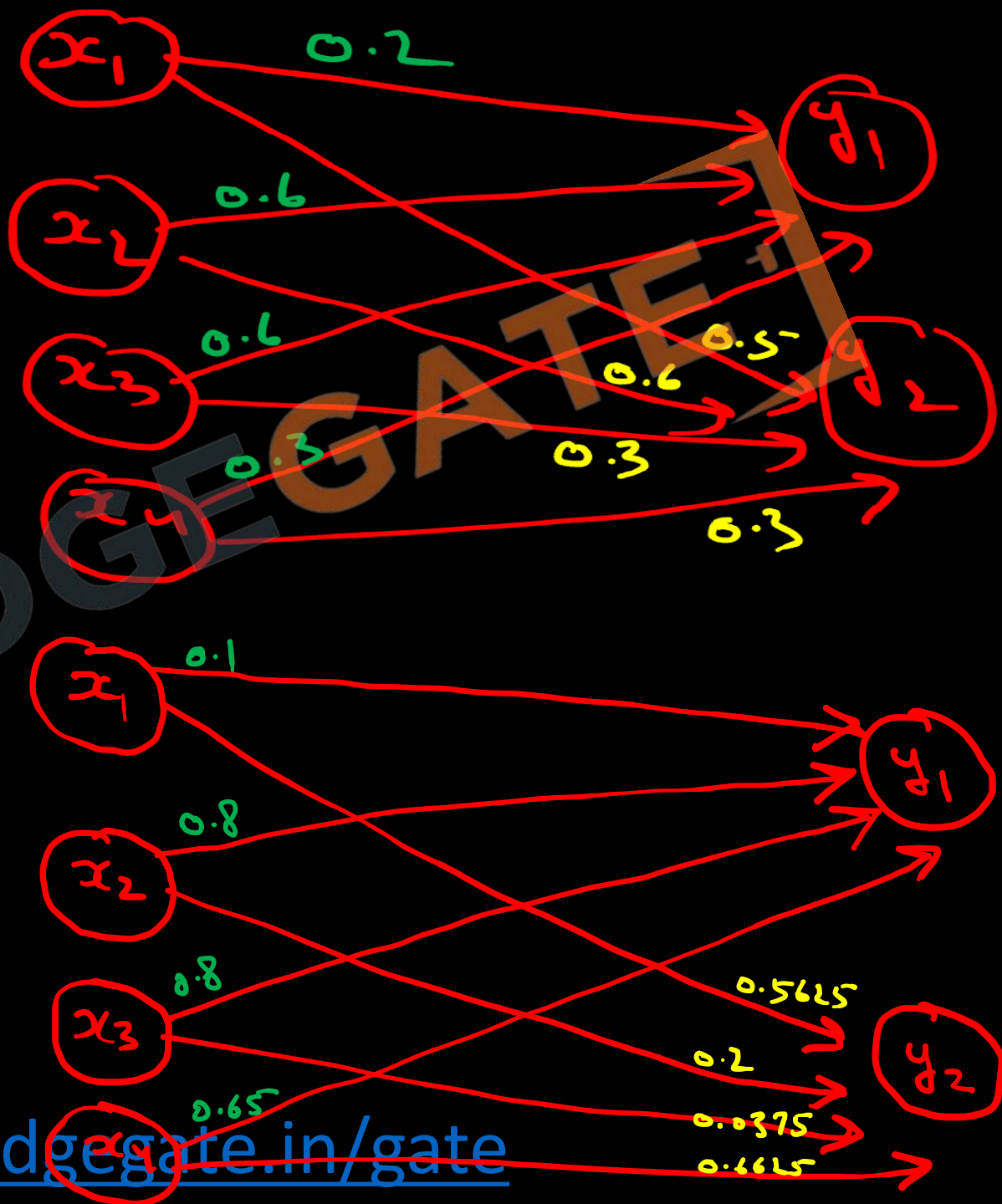
Updated after training with $\mathbf{x}_1, \mathbf{x}_3, \mathbf{x}_4$:

$$\mathbf{w}_{\text{unit2}}(\text{new}) = [0.5625, 0.2, 0.0375, 0.6625]$$

Summary

- **Unit 1 Weights**: $[0.1, 0.8, 0.8, 0.65]$
- **Unit 2 Weights**: $[0.5625, 0.2, 0.0375, 0.6625]$

These are the final weights for both units after the complete training process.



KNOWLEDGE GATE
dgegate.in/gate

Deep Learning Overview

- Deep learning is a subset of machine learning that involves neural networks with many layers (hence "deep"). These models are capable of learning from large amounts of data and can perform complex tasks by automatically learning features and representations.

KNOWLEDGEGATE

Historical Context

- **1940s-1950s**: Early neural network models like the Perceptron were developed.
- **1980s**: Backpropagation algorithm was introduced, enabling the training of multi-layer neural networks.
- **2000s**: Deep learning gained popularity due to increased computational power (GPUs), large datasets, and algorithmic advancements.
- **2012**: Breakthrough in image recognition with AlexNet winning the ImageNet competition, showcasing the power of deep neural networks.

KNOWLEDGEGATE

Current Use Deep learning is widely used across various domains:

- **Computer Vision**: Image classification, object detection, facial recognition (e.g., self-driving cars use deep learning for environment perception).
- **Natural Language Processing (NLP)**: Language translation, sentiment analysis, chatbots (e.g., Google Translate, GPT-3).
- **Speech Recognition**: Transcribing spoken words into text (e.g., virtual assistants like Siri, Alexa).
- **Healthcare**: Medical image analysis, drug discovery (e.g., detecting anomalies in X-rays, predicting patient outcomes).
- **Finance**: Fraud detection, algorithmic trading (e.g., identifying fraudulent transactions, stock price prediction).

KNOWLEDGE GATE

Advantages

- **High Performance**: Exceptional accuracy and performance on complex tasks.
- **Automatic Feature Extraction**: Automatically learns relevant features from raw data.
- Versatility: Applicable to a wide range of problems (vision, speech, text, etc.).

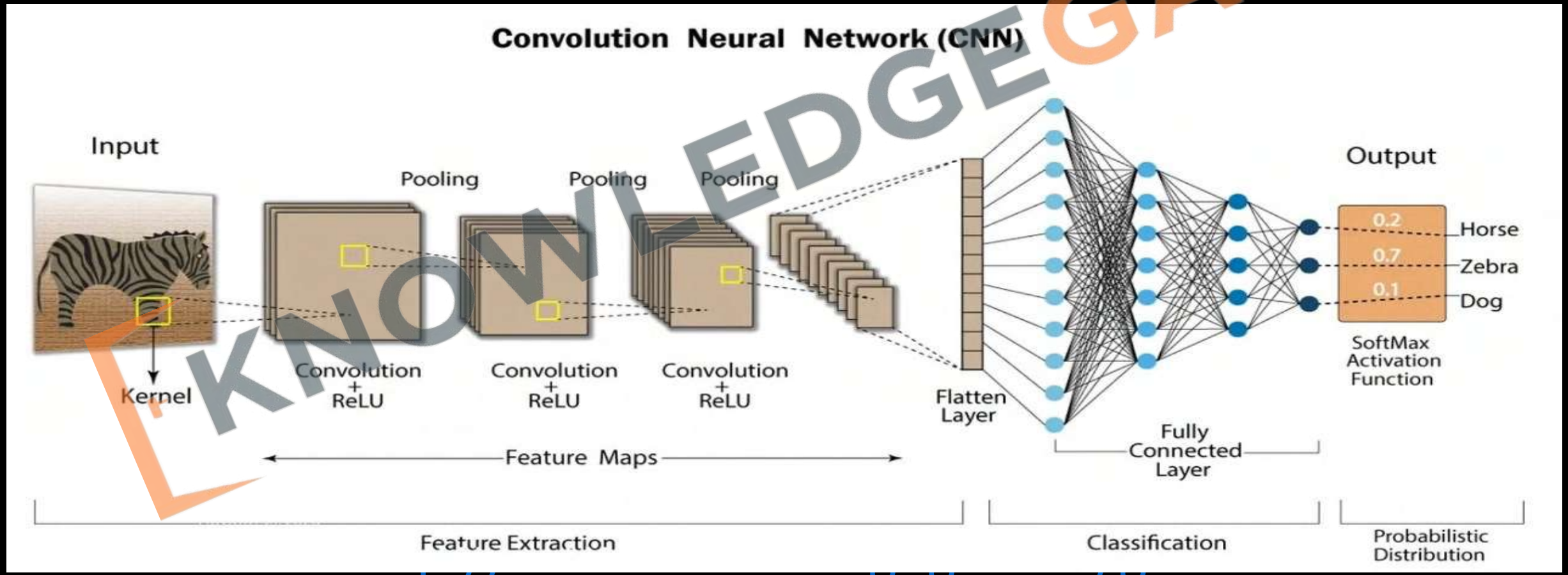
Disadvantages

- **Data-Hungry**: Requires large amounts of labeled data for training.
- **Computationally Intensive**: Demands significant computational resources (GPUs, TPUs).
- **Black Box**: Lack of interpretability; difficult to understand how decisions are made.
- **Overfitting**: Prone to overfitting if not properly regularized or if insufficient data is available.

KNOWLEDGE GATE

Convolutional Neural Networks (CNNs)

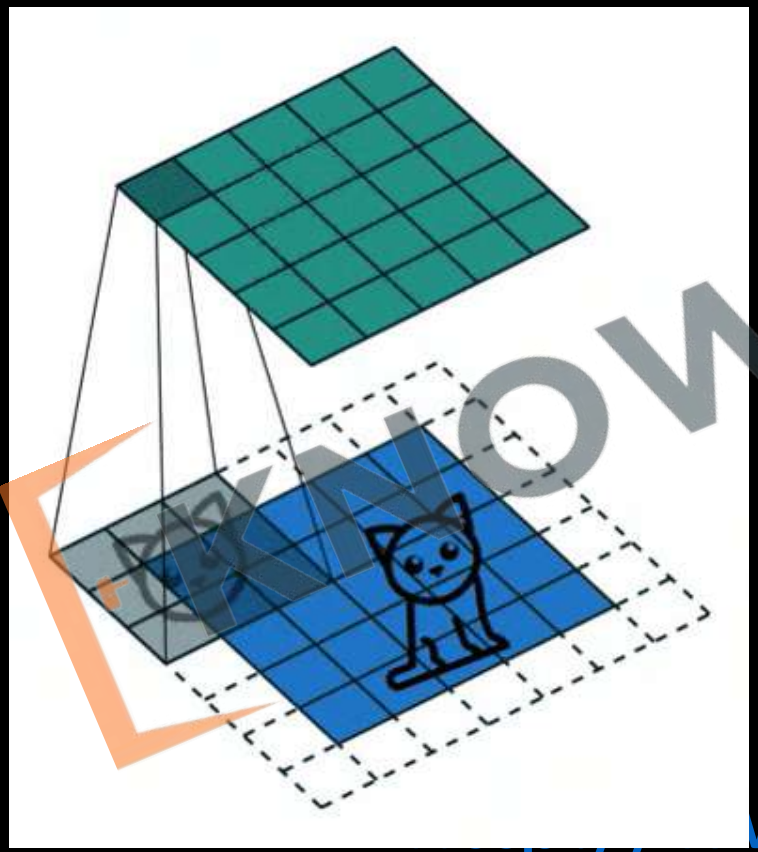
- Convolutional Neural Networks (CNNs) are a class of deep learning models designed specifically for processing data with a grid-like topology, such as images. They are composed of multiple layers that automatically and adaptively learn spatial hierarchies of features from input images.

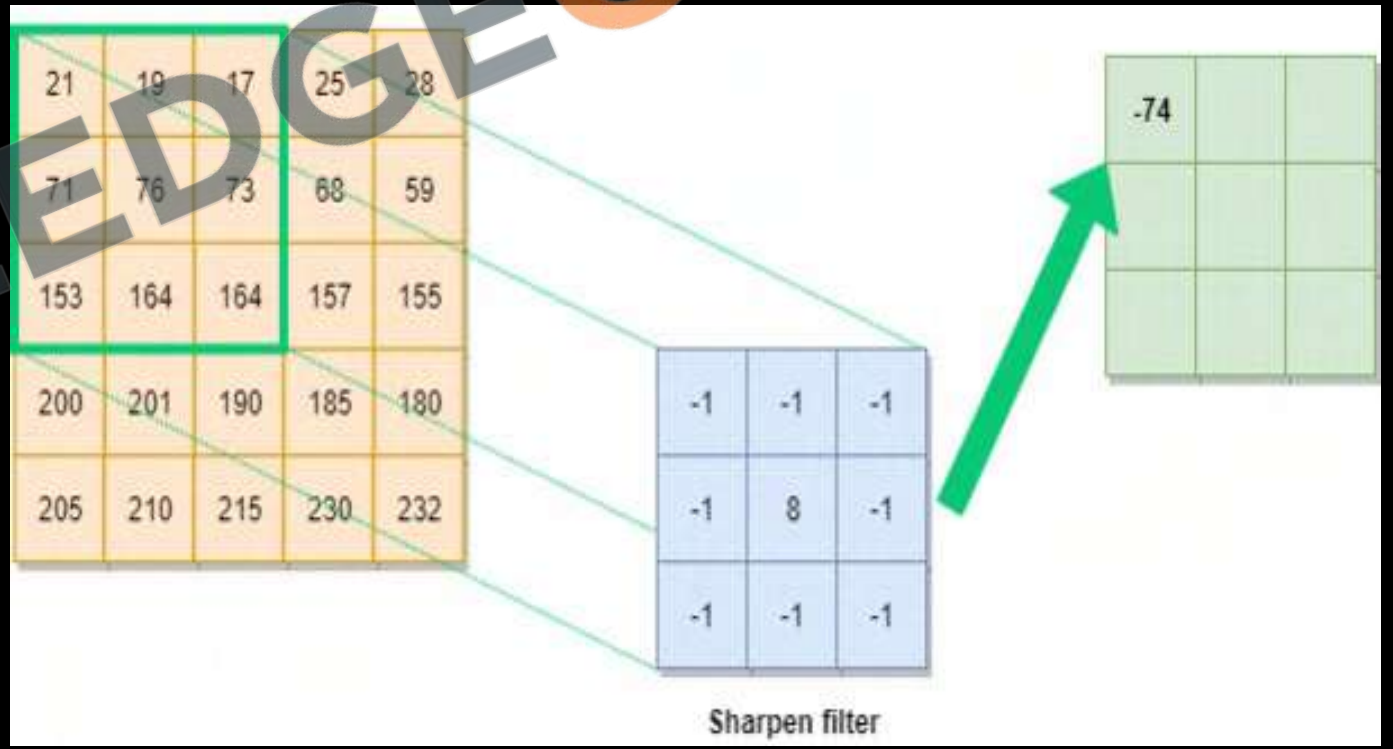


KNOWLEDGEGATE

Convolutional Layers

- **Function**: Extract features from the input image by applying convolution operations using filters (kernels).
- **Mechanism**:
  - **Filters/Kernels**: Small matrices (e.g., 3x3, 5x5) that slide over the input image.
  - **Convolution Operation**: Each filter convolves (slides) over the input image, computing the dot product between the filter and the input patch. This generates a feature map.

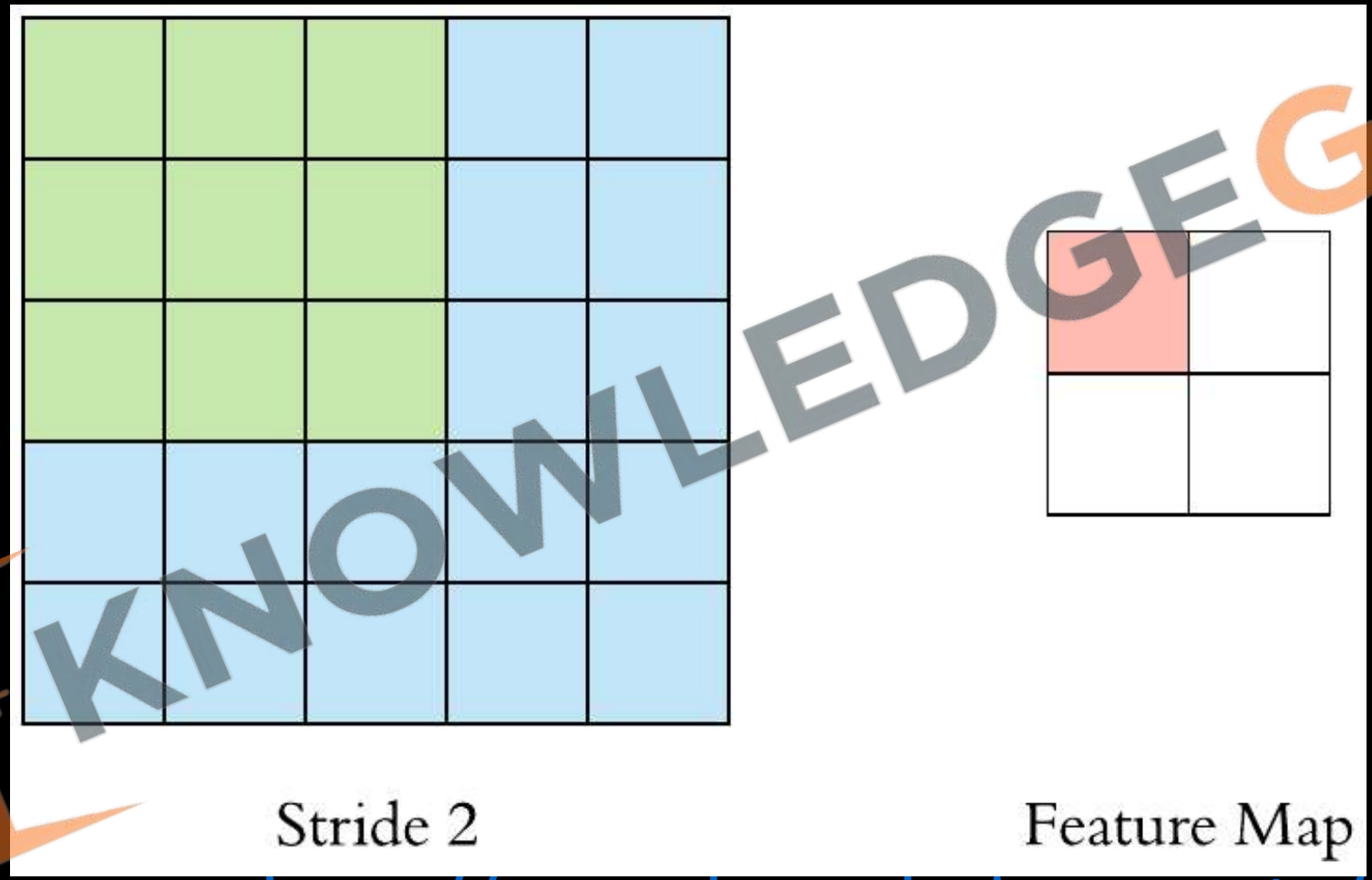




KNOWLEDGEGATE

http://www.knowledgegate.in/gate

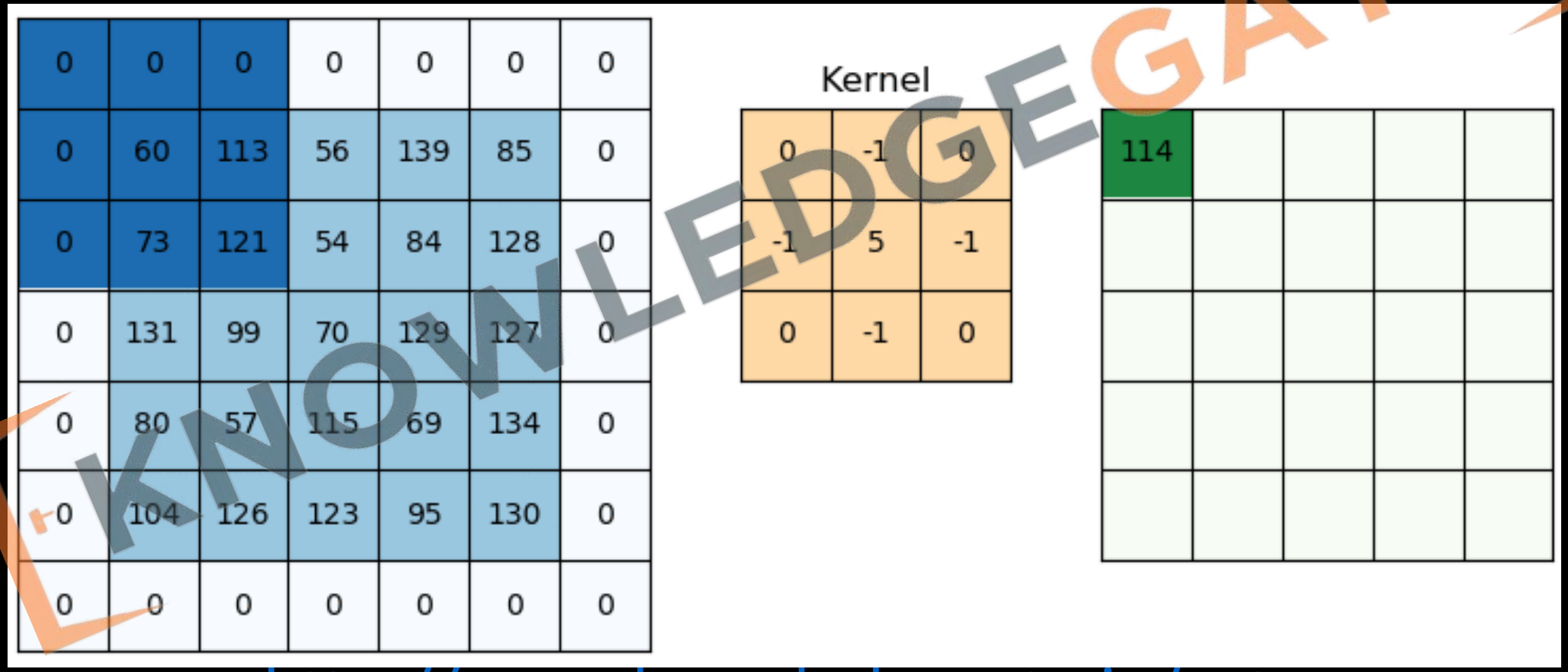
- **Stride**: The step size by which the filter moves. Larger strides result in smaller feature maps.



KNOWLEDGEGATE

http://www.knowledgegate.in/gate

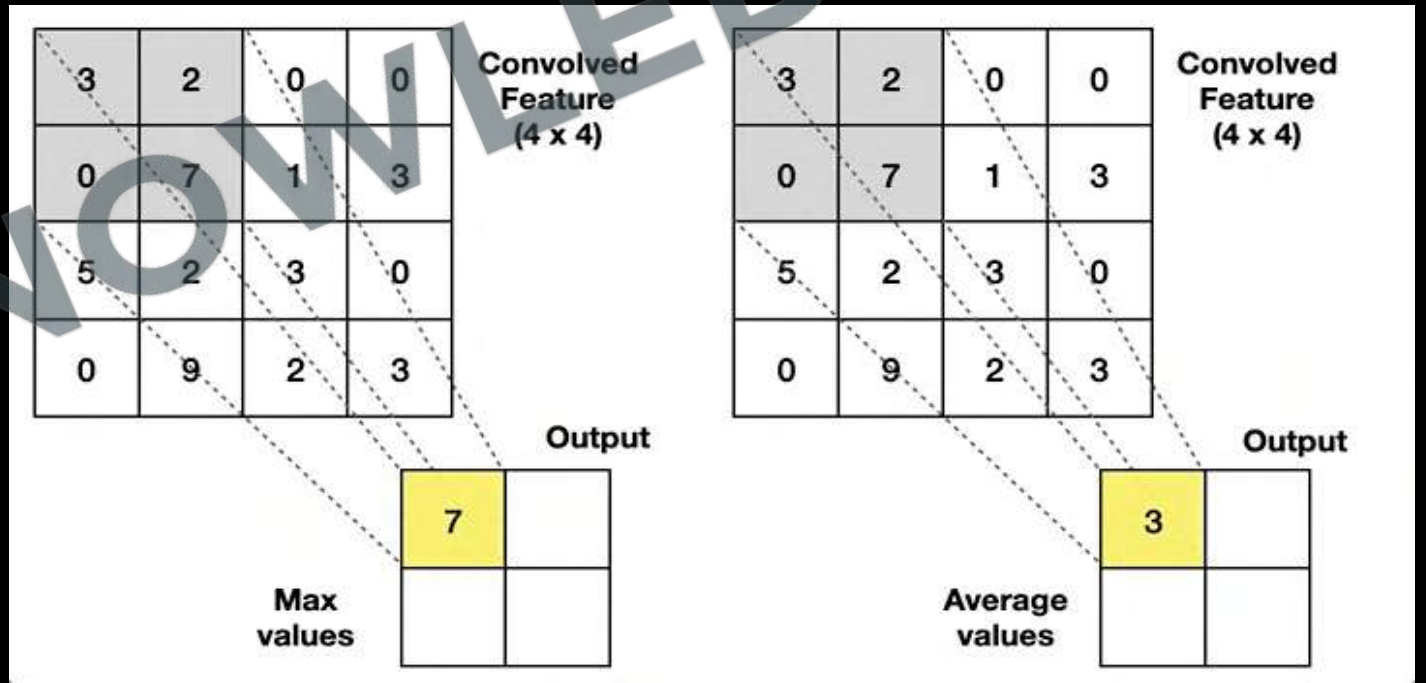
- **Padding**: Adding zeros around the input to preserve the spatial dimensions. 'Same' padding keeps the output size the same as the input, while 'valid' padding reduces it.



KNOWLEDGEGATE

Pooling Layers

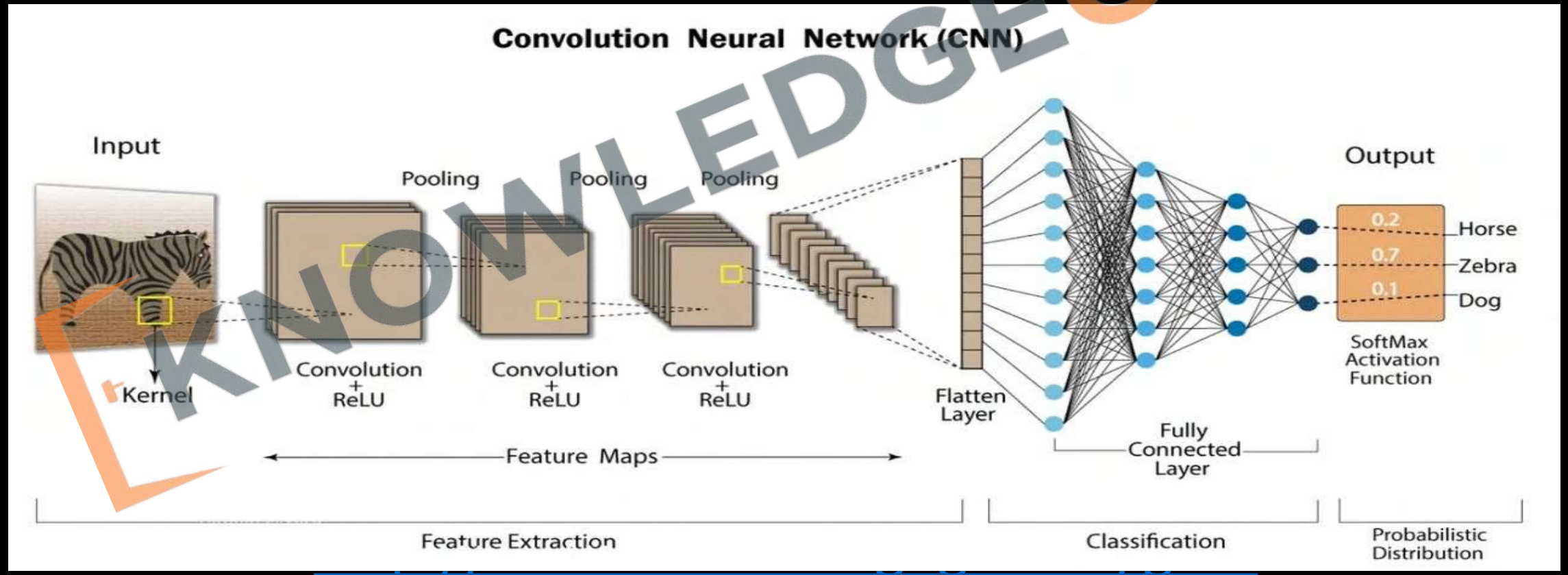
- **Function**: Reduce the spatial dimensions of the feature maps, keeping the most important information.
- **Types**:
  - **Max Pooling**: Takes the maximum value from each patch of the feature map.
  - **Average Pooling**: Takes the average value from each patch of the feature map.
- **Mechanism**:
  - A pooling layer slides a window (e.g., 2x2) over the input feature map and downsamples it by taking the maximum or average value within the window.



KNOWLEDGEGATE

Fully Connected Layers

- **Function**: Perform classification based on the features extracted by the convolutional and pooling layers.
- **Mechanism**:
  - Flatten the output from the last convolutional or pooling layer into a one-dimensional vector.
  - Connect each neuron in the fully connected layer to every neuron in the previous layer.



KNOWLEDGEGATE

Activation Functions

- **Function**: Introduce non-linearity into the network, allowing it to learn complex patterns.
- **Common Activation Functions**:
  - **ReLU (Rectified Linear Unit)**: $\mathrm{ReLU}(x) = \max(0, x)$
  - **Sigmoid**: $\sigma(x) = \frac{1}{1+e^{-x}}$
  - **Tanh**: $\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$

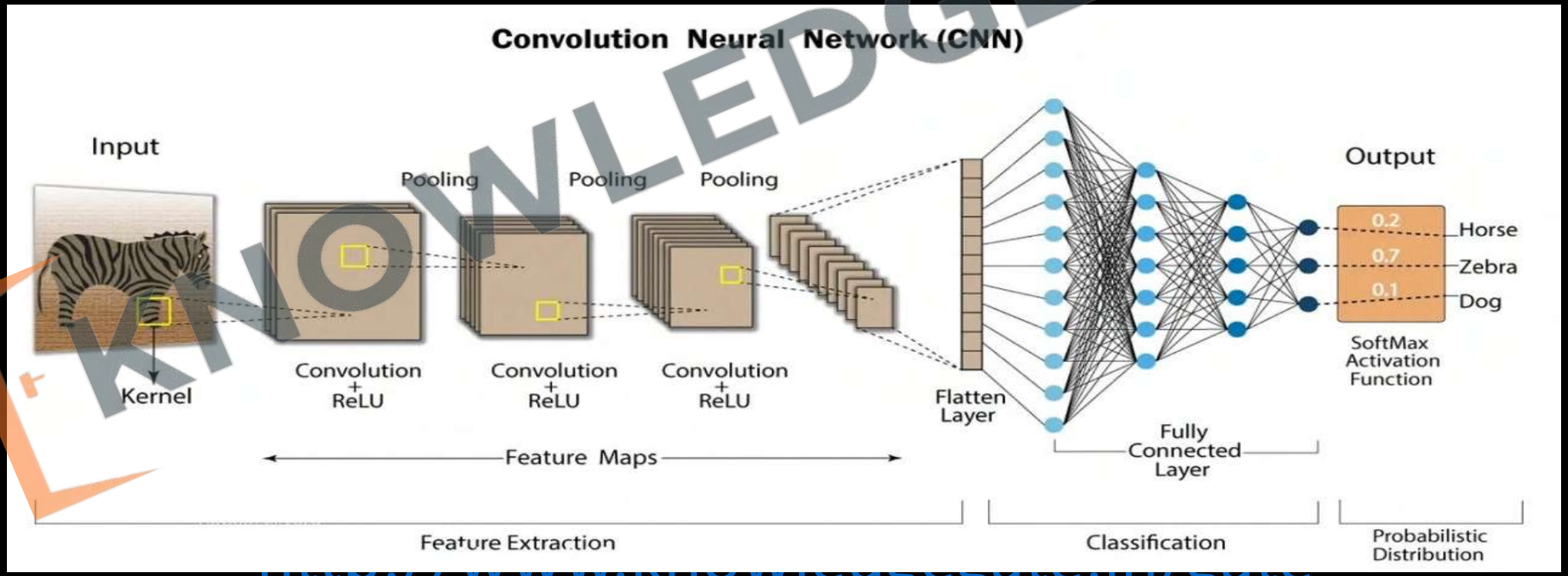
KNOWLEDGEGATE

http://www.knowledgegate.in/gate

KNOWLEDGEGATE

Dropout Layers

- **Function**: Prevent overfitting by randomly setting a fraction of input units to zero at each update during training.
- **Mechanism**:
  - During training, randomly drop neurons along with their connections.
  - During testing, use all neurons but scale the output to account for dropped units during training.



KNOWLEDGEGATE

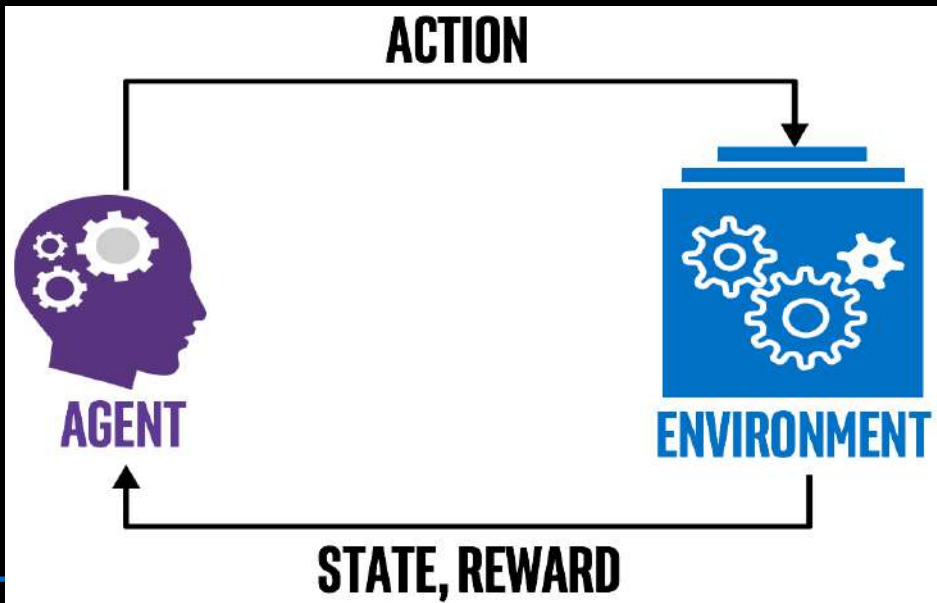
(UNIT-5: REINFORCEMENT LEARNING)

- -Introduction to Reinforcement Learning, Learning Task, Example of Reinforcement Learning in Practice, Learning Models for Reinforcement - (Markov Decision process, Q Learning - Q Learning function, @ Learning Algorithm), Application of Reinforcement Learning, Introduction to Deep Q Learning. GENETIC ALGORITHMS: Introduction, Components, GA cycle of reproduction, Crossover, Mutation, Genetic Programming, Models of Evolution and Learning, Applications.

KNOWLEDGEGATE

Reinforcement Learning

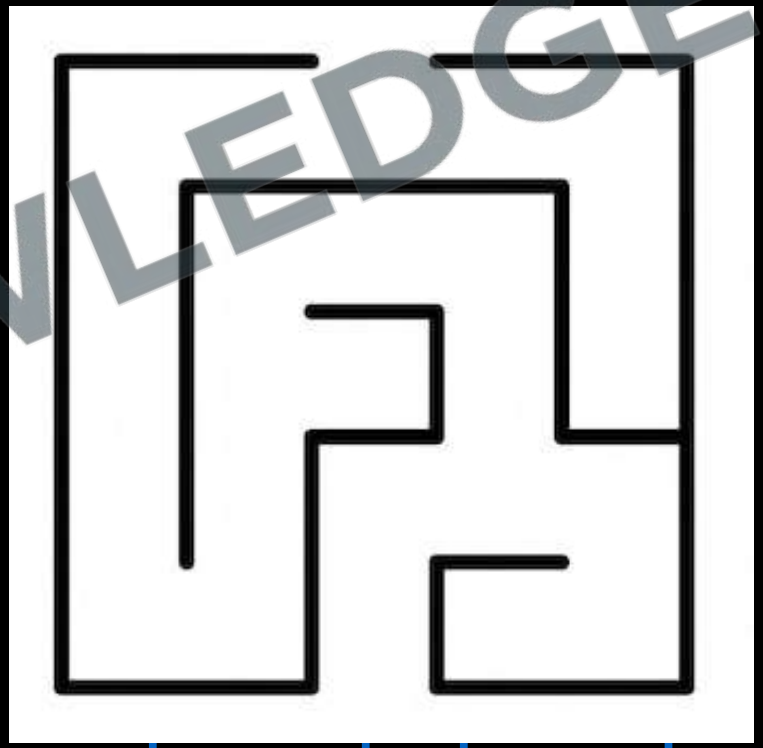
- Reinforcement Learning (RL) is a feedback-based machine learning approach. An agent learns which actions to perform by observing the environment and the results of its actions.
- **Feedback**:
  - Correct actions receive positive feedback.
  - Incorrect actions receive negative feedback or penalties.
- **Key Components**:
  - **Agent**: Learns and makes decisions.
  - **Environment**: The external system the agent interacts with.
  - **Action**: What the agent does.
  - **State**: The current situation of the agent.
  - **Reward**: Feedback from the environment.



KNOWLEDGEGATE

http://www.knowledgegat

- **Process**:
  - The agent interacts with the environment.
  - It identifies possible actions.
  - Performs actions based on observations.
  - Receives rewards or penalties.



KNOWLEDGEGATE

- **Goals and Learning**:
  - **Primary Goal**: Perform actions that maximize positive rewards.

- **Learning Method**:
  - The agent learns automatically from feedback without labelled data.
  - Unlike supervised learning, RL relies on experience.

- **Application**:
  - **Problem Solving**: Suitable for problems requiring sequential decision-making like game-playing, robotics, etc.

- **Learning Types**:
  - **Positive Reinforcement**: Encourages behaviour by providing positive rewards.
  - **Negative Reinforcement**: Discourages behaviour by providing negative rewards.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Examples**:
  - **Maze Game**: Avoiding danger spots to minimize loss.
  - **Grid Games**: Finding the shortest path or avoiding blocks and danger.

- **Limitations**:
  - **Inapplicability**: Not suitable for environments with complete information. For example, object detection and face recognition are better solved using classifiers rather than RL.

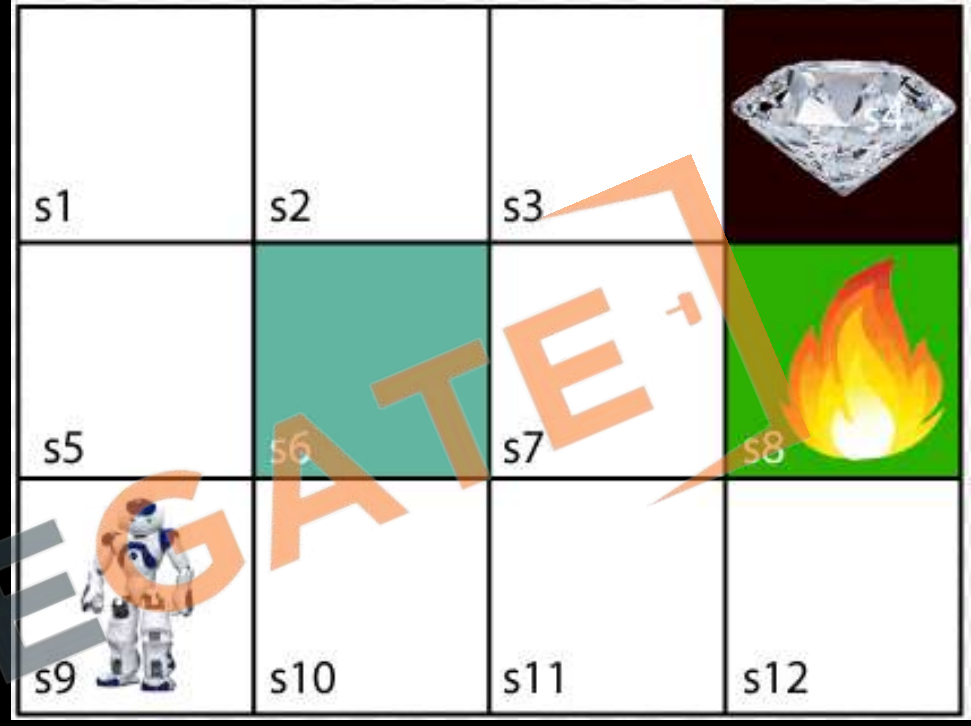
KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Criteria	Supervised Learning	Reinforcement Learning	Unsupervised Learning
Mapping	Present	Present	Not present
Feedback	Instantaneous feedback once the model is created	Constant feedback from environment	No feedback from environment
Supervisor	Presence of supervisor and labeled data	No supervisor and labeled dataset is not available	No supervisor
Decisions	Independent and based on training input	Dependent and made sequentially	Independent
Examples	Classifiers (e.g., image classification)	Chess, Go Games, Robotics	Clustering (e.g., customer segmentation)

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Characteristics of Reinforcement Learning

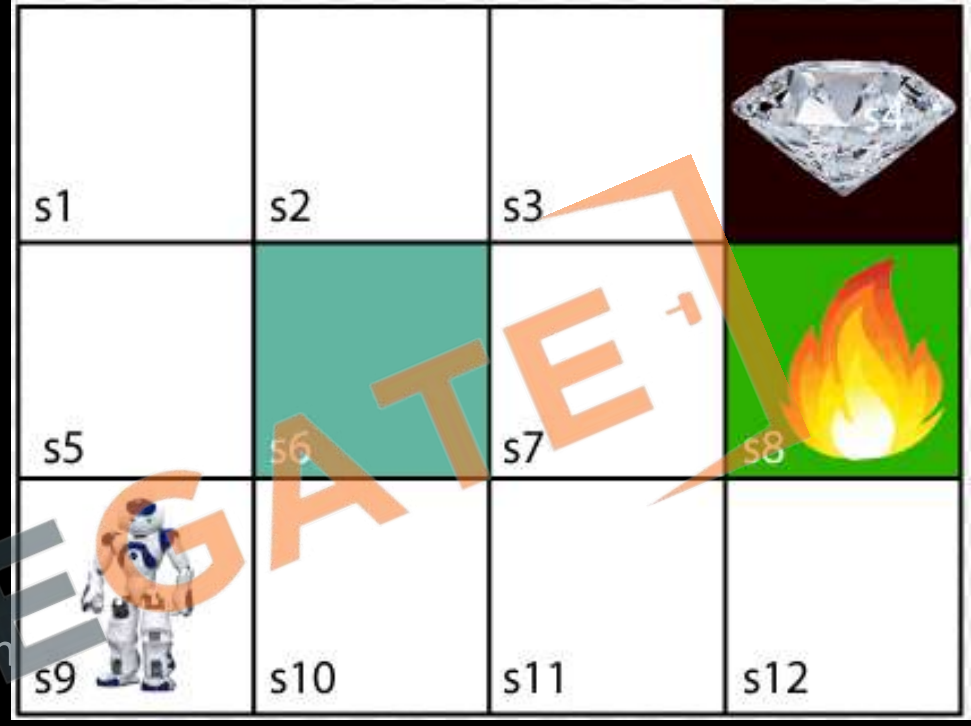
- **Sequential Decision Making**:
  - Decisions are made in sequence, each action affecting the next. Example: In a maze game, the path from start to goal involves a series of decisions.
- **Actions are Interdependent**:
  - Each action's outcome affects subsequent actions. Example: The agent's next move depends on the reward received from the previous action.
- **Delayed Feedback**:
  - Rewards or penalties are not immediate and can take time to be realized. Example: A robot may need several steps to receive feedback on whether it reached a target or avoided an obstacle.
- **Time-Related**:
  - All actions are associated with time stamps and are ordered in sequence. Example: In a grid game, actions are performed in a specific order over time.
- **Time-Consuming Operations**:
  - The learning process can be time-consuming due to the complexity and number of possible actions and states. Example: Training a model to play a game requires extensive computation due to the large state space.



KNOWLEDGEGATE

Challenges of Reinforcement Learning

- **Reward Design**:
  - Designing appropriate rewards and determining their value can be complex. Example: In many games, setting suitable rewards is a significant challenge.
- **Absence of a Model**:
  - Some environments lack a fixed structure or rules, requiring simulations to gather experience. Example: Unlike chess, many games need simulations because they have no underlying model.
- **Partial Observability of States**:
  - Not all states are fully observable, leading to uncertainty. Example: In weather forecasting, complete information about the state is often unavailable.
- **Complexity**:
  - Large board configurations and numerous possible actions increase complexity. Example: Games like Go have vast possibilities, making labeled data unavailable and increasing algorithmic complexity.
- **Time-Consuming Operations**:
  - The need to explore extensive state spaces and possible actions increases the time required for learning. Example: Complex scenarios result in longer computational times, making the training process slow.



KNOWLEDGEGATE

Applications of Reinforcement Learning

- **Industrial Automation**:
  - Used to automate and optimize manufacturing processes.
- **Resource Management Applications**:
  - Allocates resources efficiently in various domains.
- **Traffic Light Control**:
  - Reduces congestion by optimizing traffic light timings.
- **Personalized Recommendation Systems**:
  - Provides personalized content recommendations, such as news articles.
- **Bidding for Advertisement**:
  - Optimizes bidding strategies in digital advertising.
- **Driverless Cars**:
  - Used in autonomous vehicle navigation and decision-making.

KNOWLEDGEGATE

Markov Decision Process (MDP)

- A Markov Decision Process (MDP) is a mathematical framework used to describe an environment in reinforcement learning. It provides a formal model for decision-making in situations where outcomes are partly random and partly under the control of a decision-maker (agent).

- **Characteristics of MDP**
- **Markov Property**:
  - The future state depends only on the current state and action, not on the sequence of events that preceded it.
  - This is known as the "memoryless" property.
- **Policy (π)**:
  - A strategy or rule that defines the action to be taken in each state.
  - Example: A policy that always chooses the action that leads to the highest reward.

KNOWLEDGEGATE

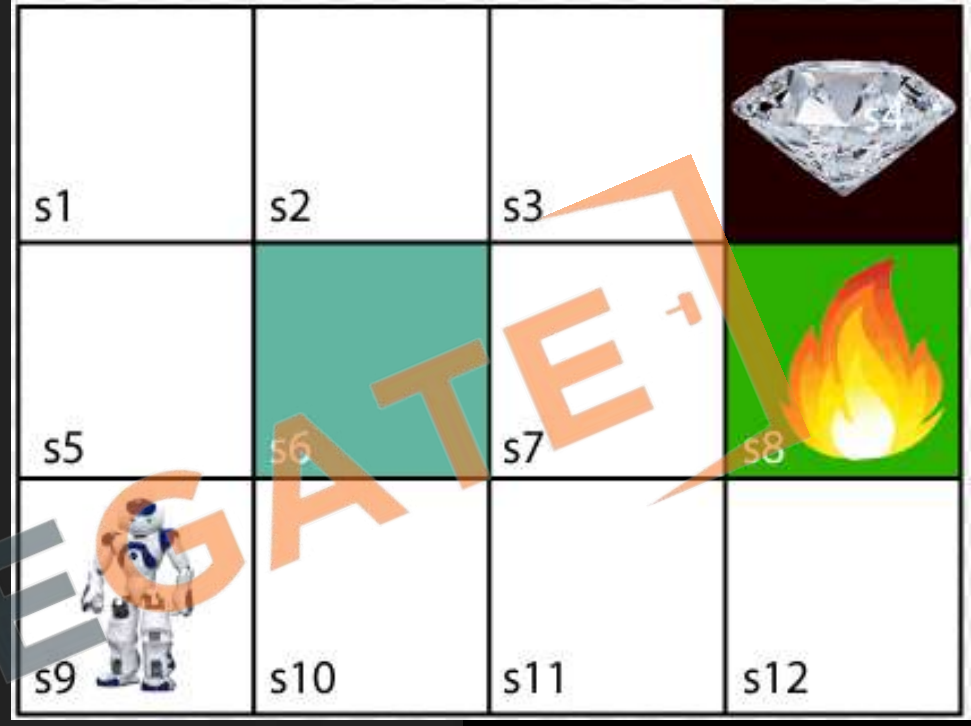
1. **States (S):**
   - The set of all possible states in which an agent can exist.
   - Example: Different rooms in a building.
2. **Actions (A):**
   - The set of all possible actions an agent can take.
   - Example: Moving from one room to another.
3. **Transition Function (P):**
   - A probability function $P(s'|s, a)$ that defines the probability of transitioning to state s' from state s when action a is taken.
   - Example: The probability of successfully moving from one room to another.
4. **Reward Function (R):**
   - A function $R(s, a, s')$ that provides the immediate reward received after transitioning from state s to state s' due to action a.
   - Example: Receiving a reward for successfully reaching a target room.
5. **Discount Factor (γ):**
   - A factor $0 \leq \gamma \leq 1$ that represents the difference in importance between future rewards and present rewards.
   - Example: Immediate rewards might be more valuable than future rewards.

KNOWLEDGEGATE

Example of MDP

Consider a robot navigating a grid environment:

- **States (S):**
  - The cells in the grid (e.g., S = {s1, s2, s3, ..., sn}).
- **Actions (A):**
  - The possible moves (e.g., A = {up, down, left, right}).
- **Transition Function (P):**
  - The probability of moving to a new cell based on the chosen action (e.g., $P(s'|s, a) = 0.8$ for successful moves and $P(s'|s, a) = 0.2$ for slipping to an adjacent cell).
- **Reward Function (R):**
  - The reward received for reaching a specific cell (e.g., R = +10 for reaching the goal cell, R = -1 for falling into a trap).



KNOWLEDGE GATE

Q-Learning

- Q-learning is a model-free reinforcement learning algorithm used to find the optimal action-selection policy for any given finite Markov decision process (MDP). It is a value-based method, meaning it focuses on finding the optimal value of the action-state pairs.

KNOWLEDGEGATE

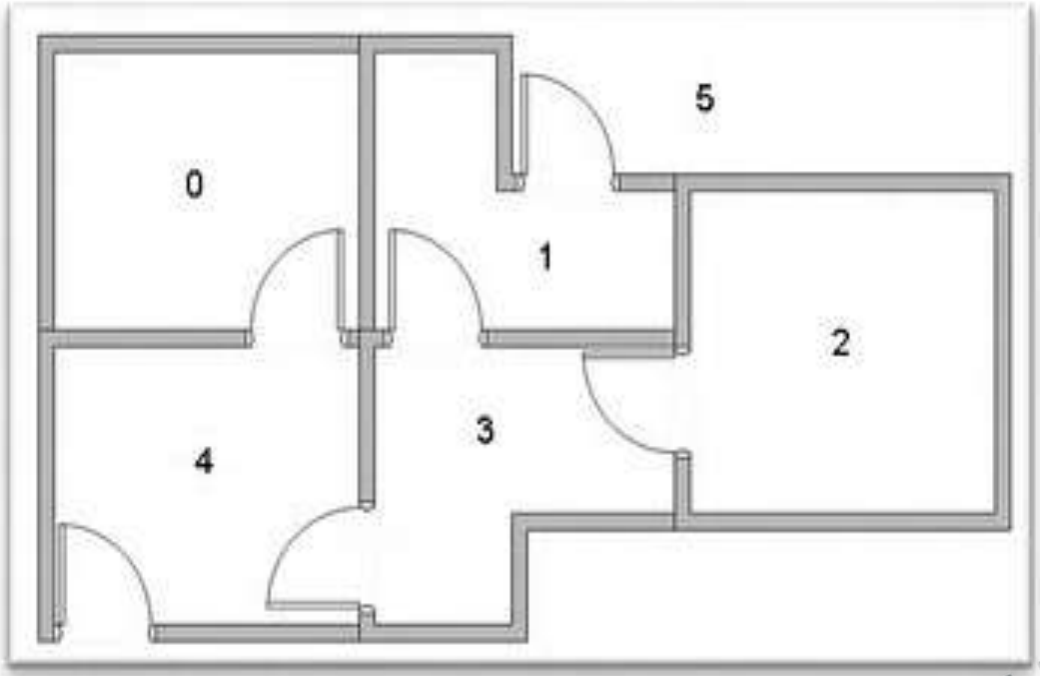
Q-Learning Algorithm

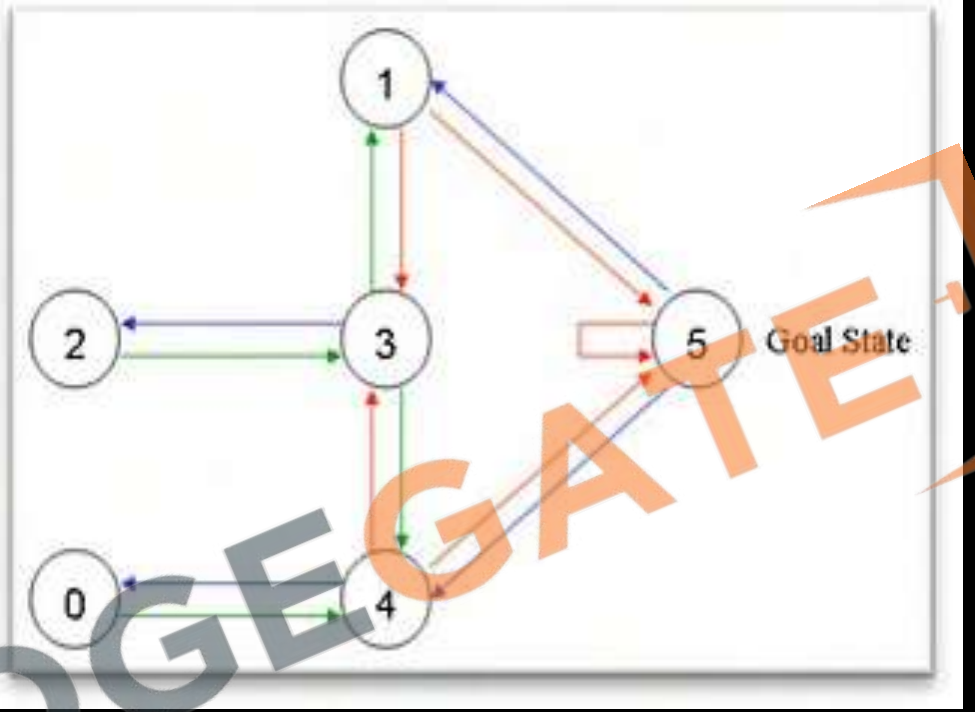
1. **Initialization**:
   - For each state s and action a, initialize the table entry $\hat{Q}(s, a)$ to zero.
2. **Observation**:
   - Observe the current state s.
3. **Repeat Forever**:
   - **Select Action**: Choose an action a and execute it.
   - **Receive Reward**: Get the immediate reward r.
   - **Observe New State**: Note the new state s'.
   - **Update Q-Value**: Update the table entry for $\hat{Q}(s, a)$ using the formula:

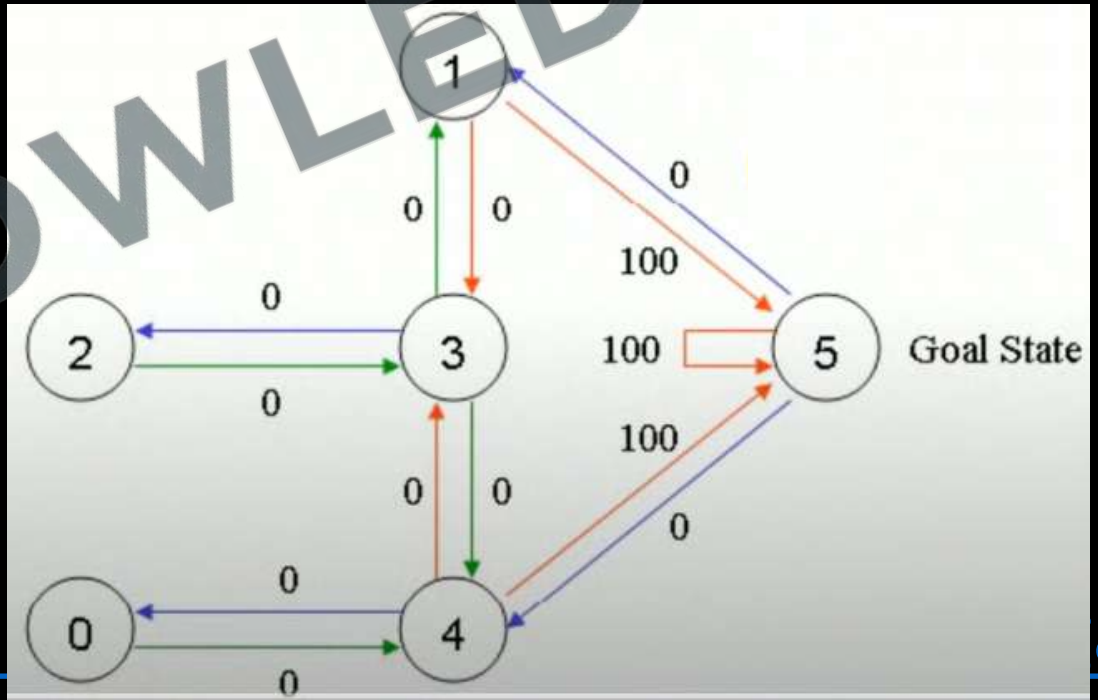
$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

   - **Transition**: Set $s \leftarrow s'$.

KNOWLEDGEGATE

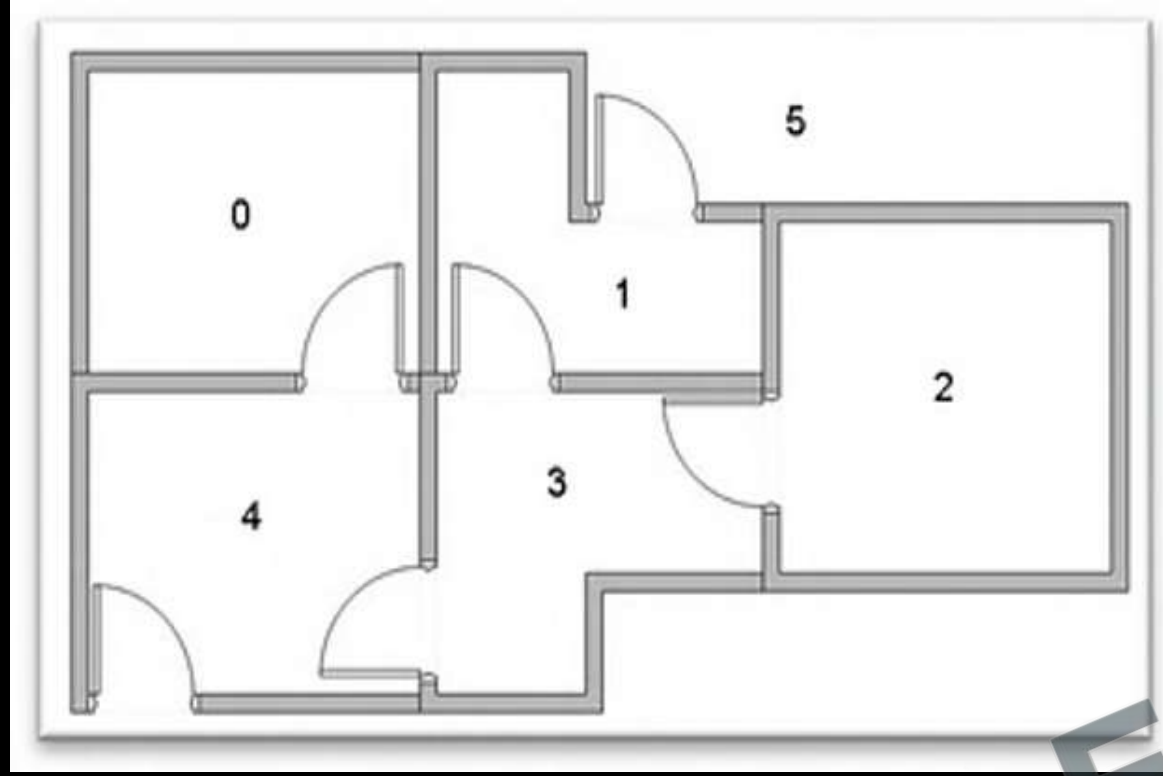


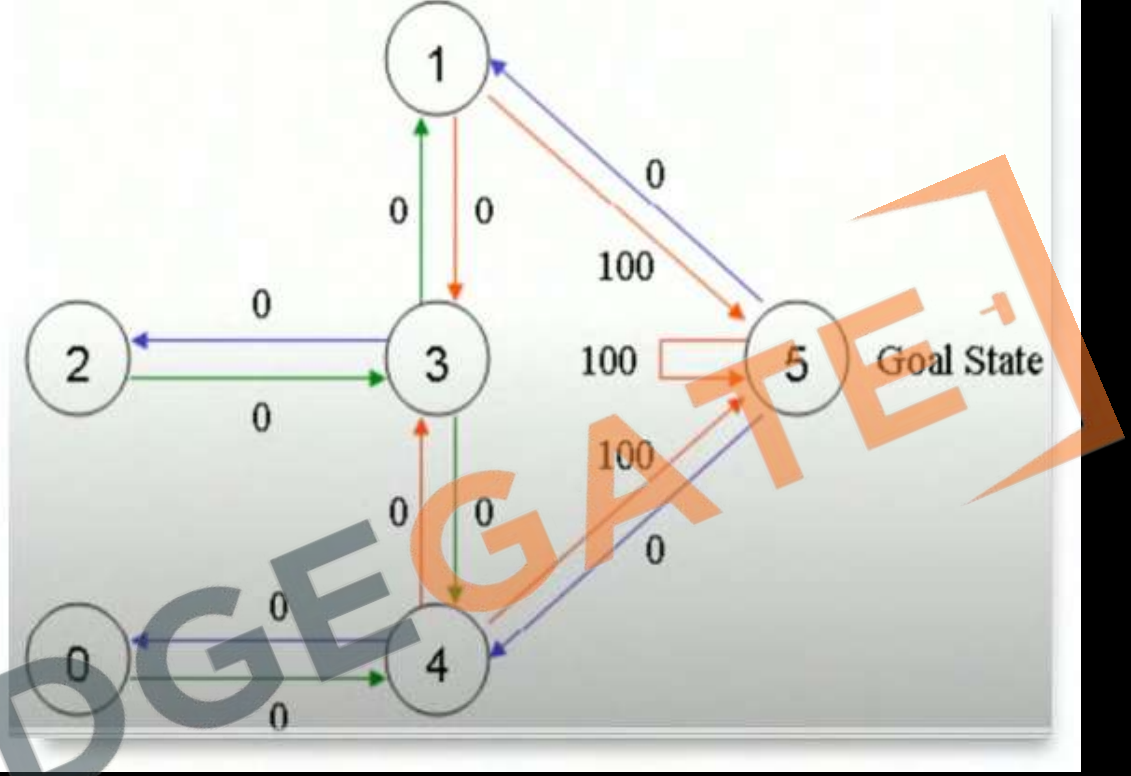




KNOWLEDGEGATE

http ate



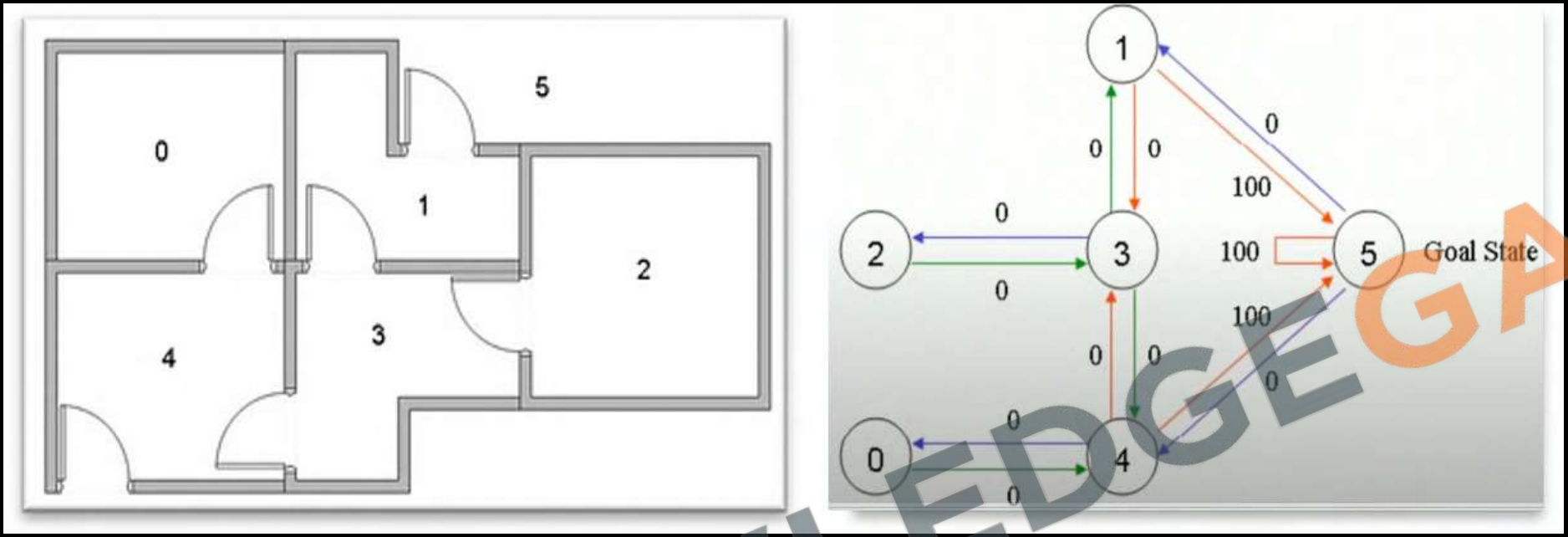


R	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

KNOWLEDGEGATE

ht gate

- Let learning rate =0.8 and the initial state as room 1



R	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

Q	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	0	0	0	0
3	0	0	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

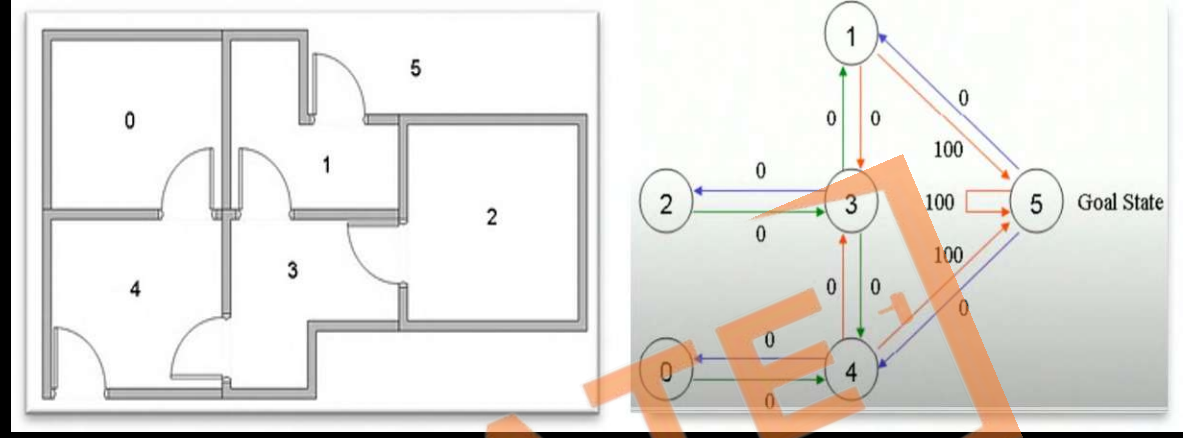
KNOWLEDGEGATE
v.knowl

Consider the second row (state 1) of matrix R:

•There are two possible actions for the current state 1:

- Go to state 3 (Action 3)
- Go to state 5 (Action 5)

•By random selection, we choose to go to state 5 as our action.



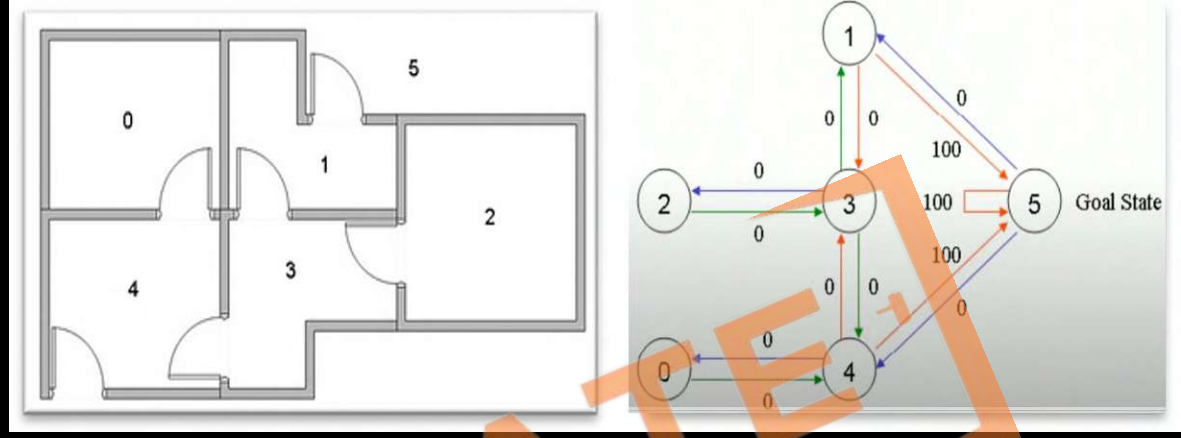
- if our agent were in state 5 (next state).
- Look at the reward matrix R (i.e. state 5).
- It has 3 possible actions: go to state 1, 4 or 5.
- Q(state, action) = R(state, action) + Gamma * Max[Q(next state, all actions)]
- Q(1, 5) = R(1, 5) + 0,8 * Max[Q(5, 1), Q(5, 4), Q(5, 5)] = 100 + 0.8 * 0 = 100

R	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

Q	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	0	0	0	0
3	0	0	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

KNOWLEDGEGATE
v.knowl

Now lets consider next episode, suppose initial state is 3
We can go to 1 or 2 or 4. and suppose we choose 1.



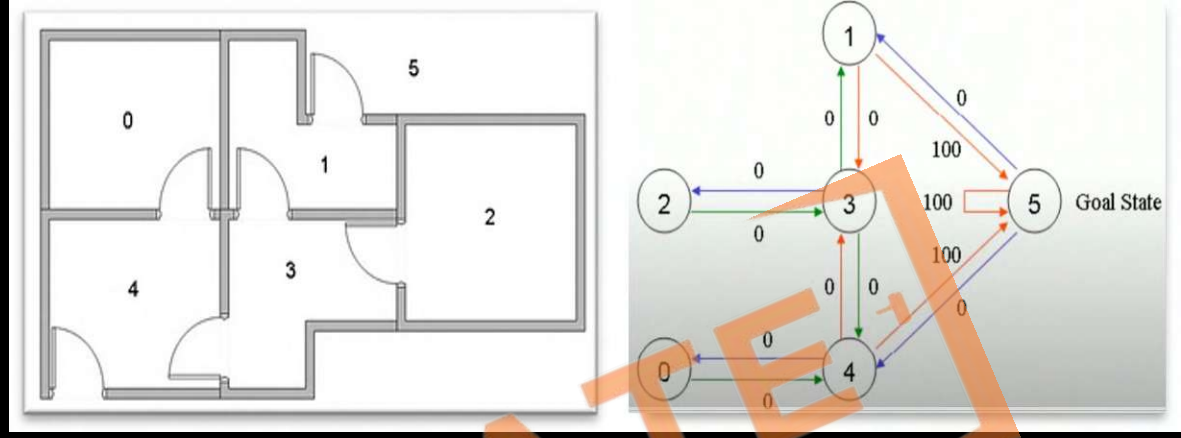
- if our agent were in state 1 (next state).
- Look at the reward matrix R (i.e. state 1).
- It has 3 possible actions: go to state 3 or 5.
- Q(state, action) = R(state, action) + Gamma * Max[Q(next state, all actions)]
- Q(3,1) = R(3, 1) + 0.8 * Max[Q(1, 3), Q(1, 5)] = 0 + 0.8 * Max(0, 100) = 80

R	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

Q	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	100
2	0	0	0	0	0	0
3	0	0	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

KNOWLEDGE GATE
v.knowl

Now lets consider next episode, suppose initial state is 3
We can go to 1 or 2 or 4. and suppose we choose 1.



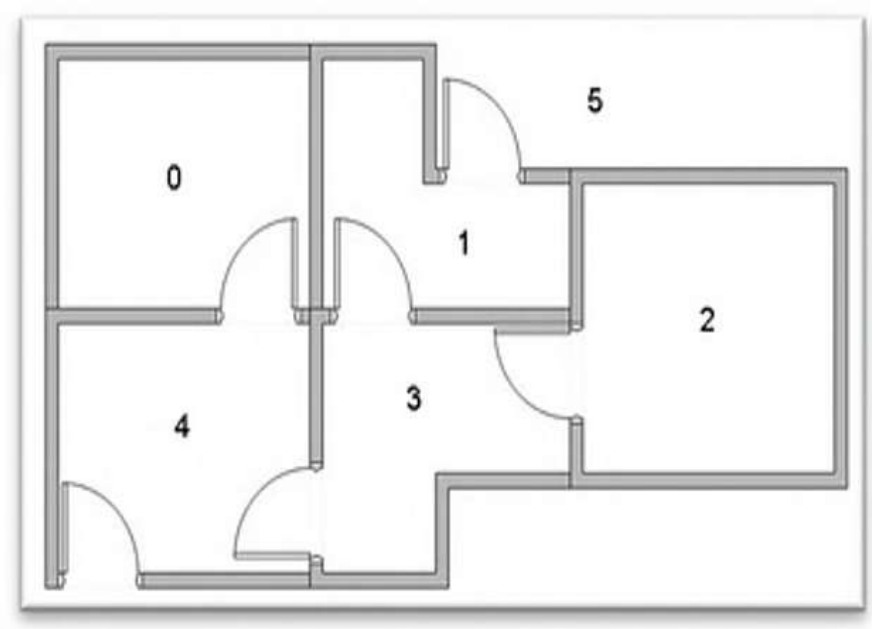
- if our agent were in state 1 (next state).
- Look at the reward matrix R (i.e. state 1).
- It has 3 possible actions: go to state 3 or 5.
- Q(state, action) = R(state, action) + Gamma * Max[Q(next state, all actions)]
- Q(3,1) = R(3, 1) + 0.8 * Max[Q(1, 3), Q(1, 5)] = 0 + 0.8 * Max(0, 100) = 80

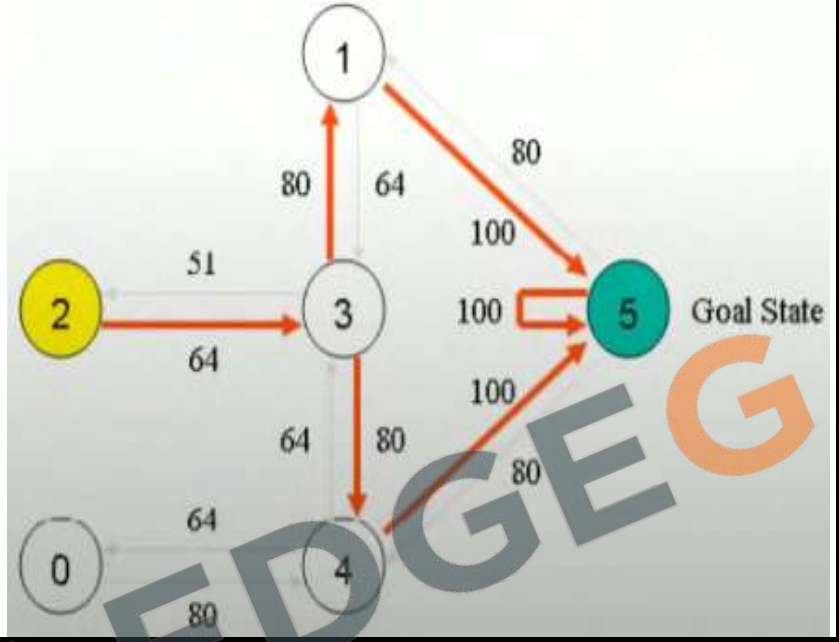
R	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

Q	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	100
2	0	0	0	0	0	0
3	0	80	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

KNOWLEDGE GATE
v.knowl

After repeating these episode multiple times





R	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

Q	0	1	2	3	4	5
0	0	0	0	0	80	0
1	0	0	0	64	0	100
2	0	0	0	64	0	0
3	0	80	51	0	80	0
4	64	0	0	64	0	100
5	0	80	0	0	80	100

KNOWLEDGEGATE
v.knowl

Steps of the Q-Learning Algorithm

1. **Initialize** the Q-values for all state-action pairs to zero (or random values).
2. **Observe** the current state s.
3. **Select** an action a based on the current policy (e.g., ε-greedy policy, where ε is a small probability of choosing a random action).
4. **Execute** the action a and observe the reward r and new state s'.
5. **Update** the Q-value using the Q-learning update rule.
6. **Set** the new state s' as the current state.
7. **Repeat** steps 3-6 until the convergence of Q-values or for a predetermined number of episodes.

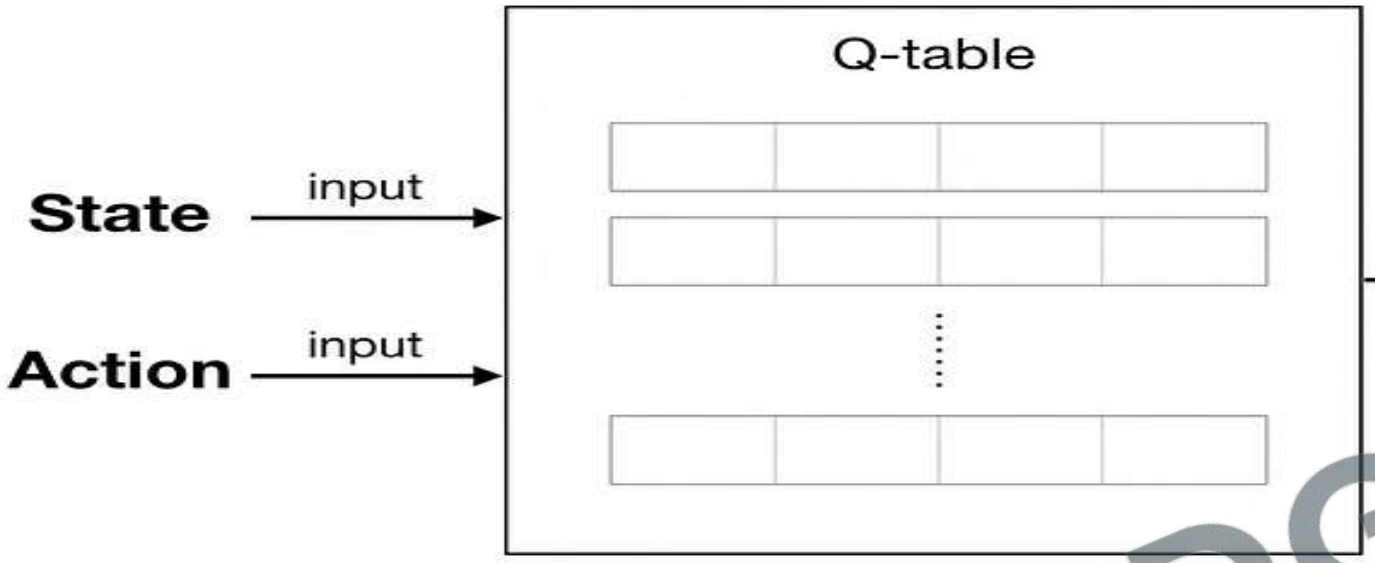
KNOWLEDGE GATE

Deep Q-Learning

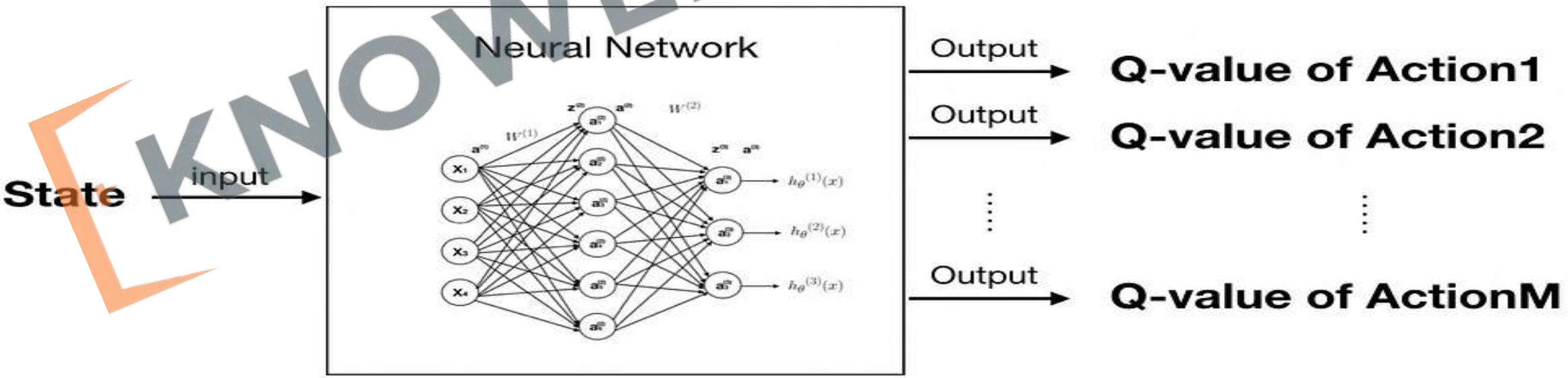
- Deep Q-Learning (DQL) is an extension of Q-Learning that uses deep neural networks to approximate the Q-values. This method is particularly useful for handling high-dimensional state spaces, where traditional Q-Learning becomes impractical due to the exponential growth in the number of state-action pairs.

KNOWLEDGEGATE

Q-Learning



Deep Q-Learning



KNOWLEDGEGATE

Benefits:

- Can handle high-dimensional state spaces.
- Generalizes across similar states, reducing the need for exhaustive state-action pair enumeration.
- Experience replay improves sample efficiency and stability.

Challenges:

- Training can be unstable due to the non-stationary target values.
- Requires significant computational resources.
- Hyperparameter tuning (e.g., learning rate, batch size, replay buffer size) can be complex.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

GENETIC ALGORITHMS: Introduction

- Evolutionary Algorithms (EAs) are optimization algorithms inspired by the principles of natural selection and genetics. They are used to find approximate solutions to complex problems. Types of Evolutionary Algorithms:

- **Genetic Algorithms** (GAs): Use binary strings to represent solutions.
    - Solving the Traveling Salesman Problem (TSP)

- **Genetic Programming** (GP): Evolve computer programs.
    - Symbolic Regression

KNOWLEDGE GATE

Genetic Algorithms

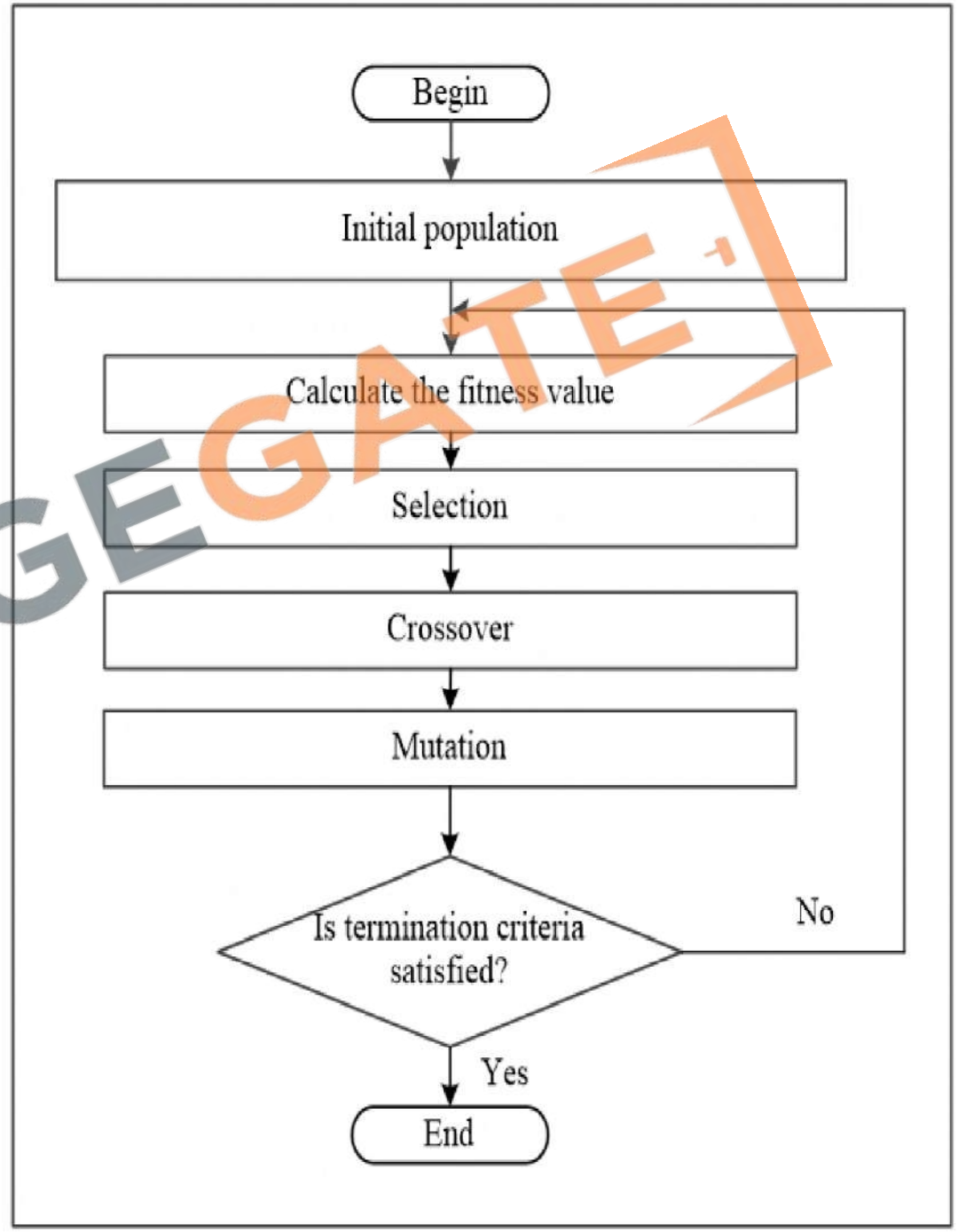
- Genetic Algorithms (GAs) are a subset of evolutionary algorithms that model biological processes such as genetics and evolution to optimize highly complex functions. They are particularly useful for solving problems that are difficult to model mathematically or are NP-hard, which are computationally very expensive to solve or involve a large number of parameters.

KNOWLEDGEGATE

Steps of a Genetic Algorithm:

- **Initialization**: Generate an initial population of random chromosomes.
- **Evaluation**: Calculate the fitness of each chromosome using the fitness function.
- **Selection**: Select pairs of chromosomes to reproduce based on their fitness (e.g., roulette wheel selection).
- **Crossover**: Combine selected pairs of chromosomes to produce offspring (e.g., single-point crossover).
- **Mutation**: Apply random changes to the offspring to maintain diversity (e.g., bit-flip mutation).
- **Termination**: Repeat the evaluation, selection, crossover, mutation, and replacement steps until a stopping criterion is met (e.g., a maximum number of generations or satisfactory fitness level).

http://www.knowledg



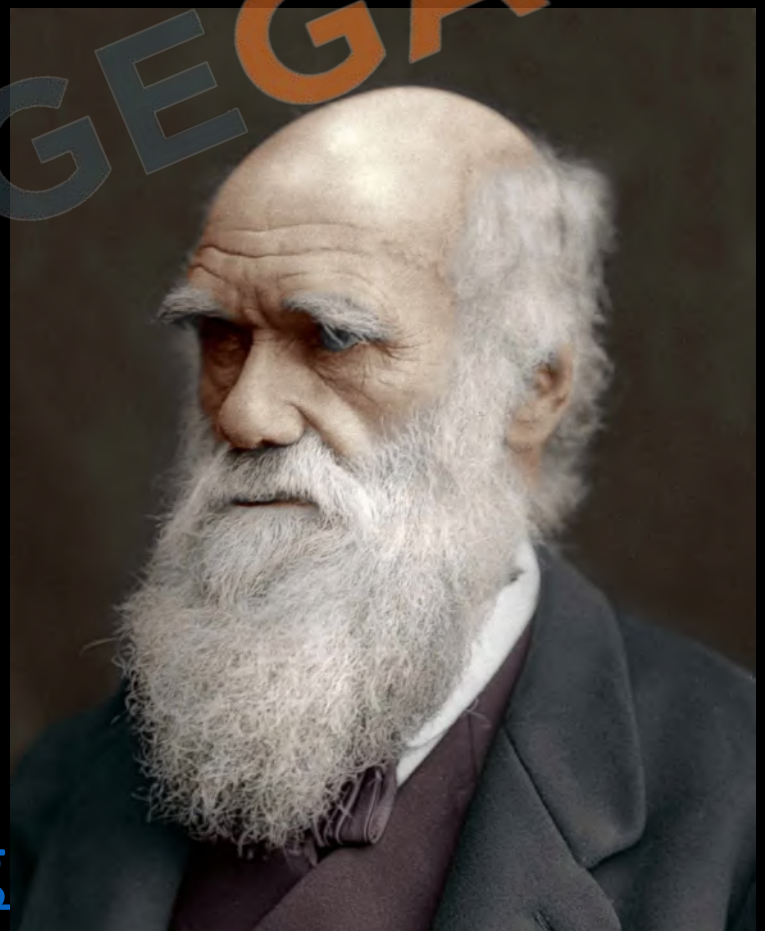
KNOWLEDGEGATE

Background of Genetic Algorithm

- Introduction:
  - Introduced by Prof. John Holland at the University of Michigan, USA in 1965.
  - The first article on Genetic Algorithms (GAs) was published in 1975.

KNOWLEDGEGATE

- Fundamental Biological Processes:
  - **Genetics**: Based on the principles of genetics as discovered by Gregor Johann Mendel in 1865.
  - **Evolution**: Inspired by the theory of evolution by Charles Darwin published in 1875.



KNOWLEDGEGATE
/www.knowledg

Genetic Algorithm

Based on Two Main Concepts:

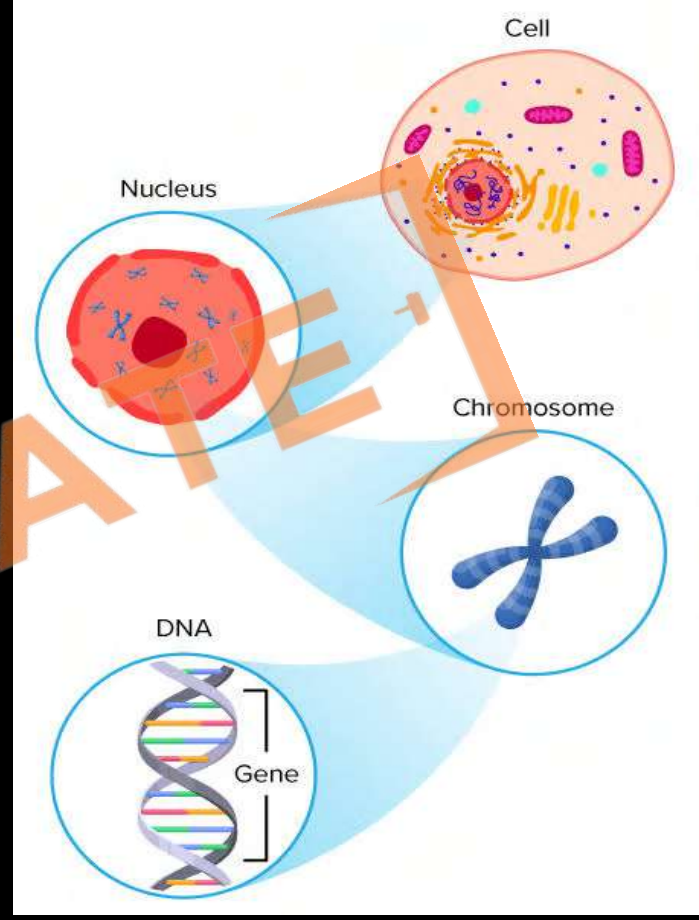
- **Genetics**: Involves cells, chromosomes, genes, and DNA.
- **Evolution**: Involves heredity, diversity, selection, and ranking.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Genetics

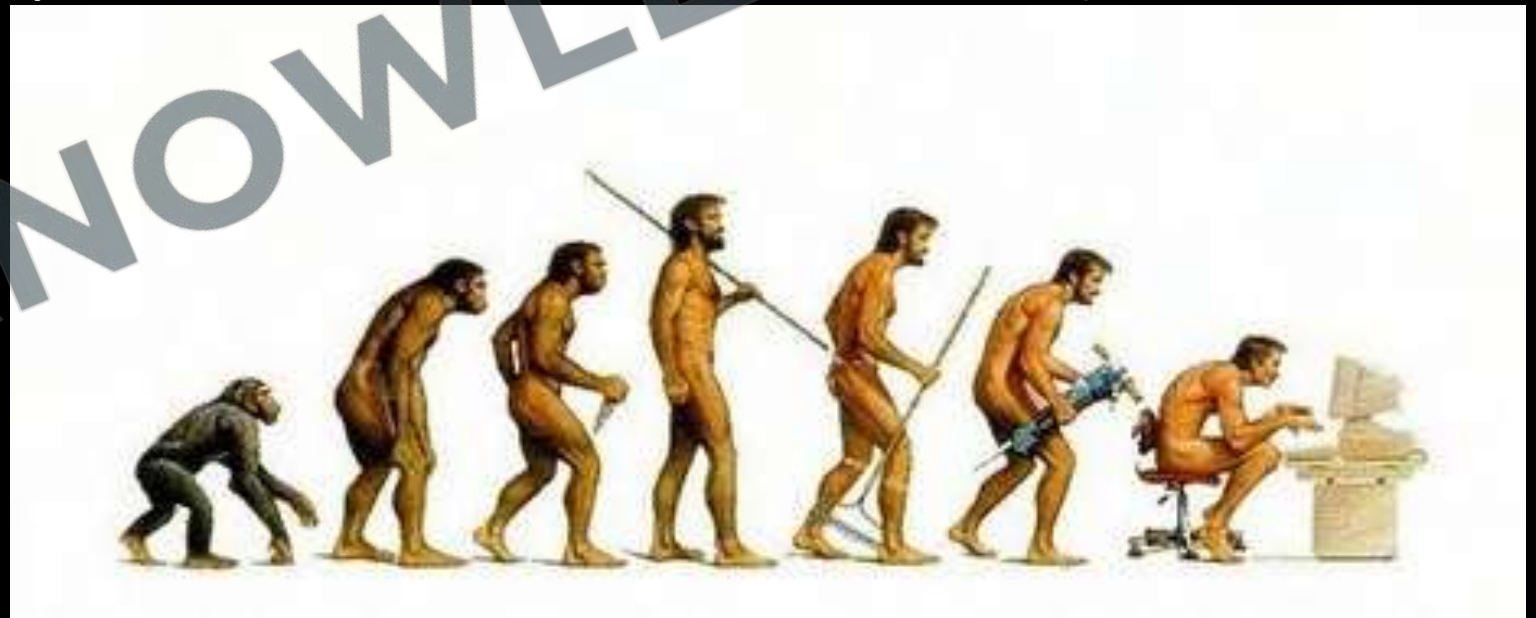
- **Cells**:
  - Basic building blocks of living bodies.
  - Each cell has the same set of chromosomes.
- **Chromosomes**:
  - Humans: 46 chromosomes.
  - Frogs: 26 chromosomes.
  - Maize: 6 chromosomes.
- **DNA**:
  - Chromosomes are made up of DNA (sequences of 0's and 1's).
  - Genes are segments of DNA.
  - Alleles are different forms of a gene.
  - Example: A gene for eye color might have alleles for blue, brown, etc.



KNOWLEDGEGATE

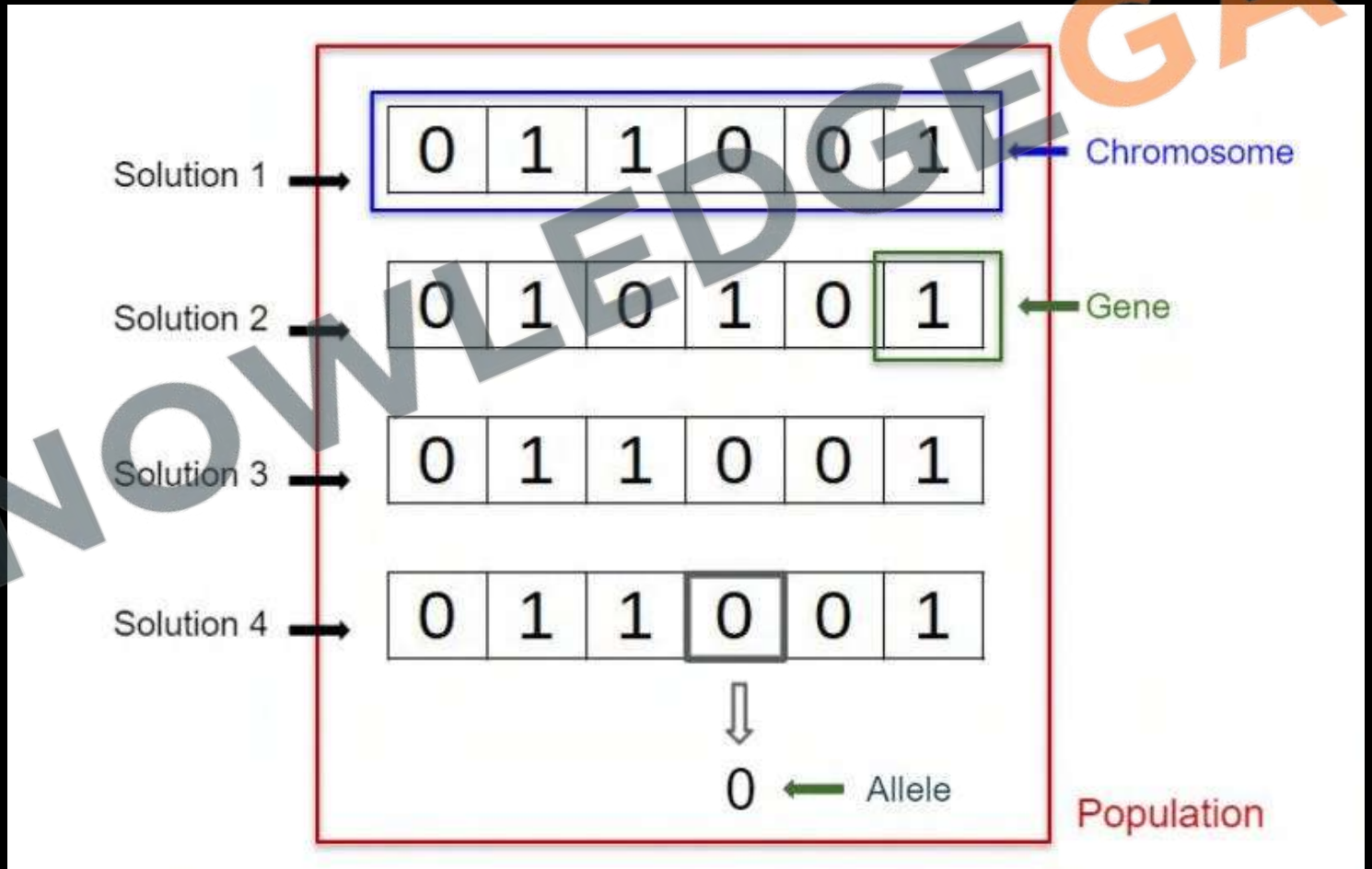
Evolution

- **Heredity**:
  - Offspring inherit many characteristics from their parents.
- **Diversity**:
  - Variation in characteristics within the next generation.
- **Selection**:
  - Only a small percentage of offspring survive to adulthood.
- **Ranking**:
  - Survival depends on inherited characteristics (natural selection).



KNOWLEDGEGATE

- **Definition**: A subset of all possible solutions to a given problem.
- **Chromosome**: One possible solution to the problem.
- **Gene**: An element position of a chromosome.
- **Population**: A set of chromosomes.



KNOWLEDGEGATE

Encoding

Encoding is a process of representing individual genes. The encoding depends mainly on solving the problem. Types of Encoding:

- **Binary Encoding**:
  - Description: The most common way of encoding is a binary string.
  - Example:
    - Chromosome 1: 10101011
    - Chromosome 2: 11011001
  - Details:
    - Each chromosome consists of a binary (bit) string.
    - Each bit in the string can represent some characteristic of the solution.
    - Every string is a solution, but not necessarily the best solution.
- **Octal Encoding**:
  - Description: In this encoding, each string uses a set of octal numbers (0 to 7).
  - Example:
    - Chromosome 1: 1534620
    - Chromosome 2: 0772014

KNOWLEDGEGATE

- **Hexadecimal Encoding**:
  - Description: This encoding uses strings made up of hexadecimal numbers (0-9, A-F).
  - Example:
    - Chromosome 1: ZAD4F
    - Chromosome 2: A05CE
- **Permutation Encoding**:
  - Description: In permutation encoding, every chromosome is a string of integers/real numbers.
  - Example:
    - Chromosome 1: 2199456476.1
    - Chromosome 2: 21752437976
- **Value Encoding**:
  - Description: In value encoding, every chromosome is a string of some values.
  - Details:
    - Values can be anything connected to the problem (integers, real numbers, characters, combination of words, etc.).
  - Example:
    - Chromosome 1: 2156784
    - Chromosome 2: 356259327891
    - Chromosome 3: AB1DFG3

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Genetic Operators

- Genetic operators are used in genetic algorithms to guide the algorithm towards a solution for a given problem. These operators alter (change) the gene composition of the offspring (children). They work in conjunction with one another for the algorithm to be successful.
- Types of Genetic Operators:
  - Selection
  - Crossover
  - Mutation

KNOWLEDGEGATE

Selection

- **Objective**: Emphasize good solutions and eliminate bad solutions in a population while keeping the population size constant.
- **Methods**: Different solutions for choosing the best solution exist (e.g., fitness function, tournament selection).
- **Goal**: Ensure survival of the fittest, keeping the best-performing solutions for reproduction.

KNOWLEDGEGATE

Different Techniques for Parent Selection:

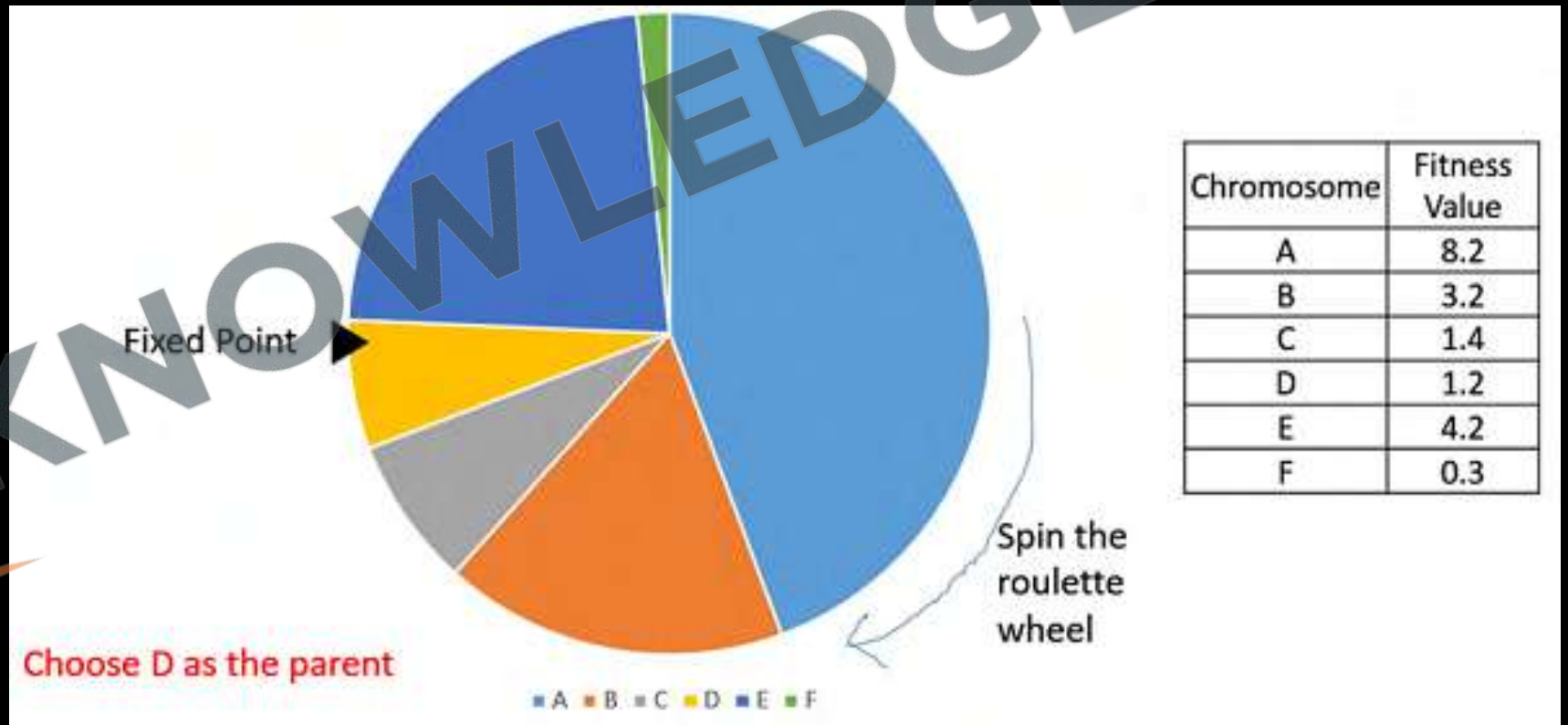
- Fitness Proportionate Selection
- Tournament Selection
- Rank Selection
- Random Selection

- Fitness Proportionate Selection:
  - **Description**: Every individual can become a parent with a probability proportional to its fitness.
  - **Detail**: Fitter individuals have a higher chance of mating and propagating their features to the next generation.
  - **Example Method**: Roulette Wheel Selection (Stochastic Universal Sampling).

KNOWLEDGEGATE

Roulette Wheel Selection

- Process:
  - The circular wheel is divided into parts, where each part is proportional to the fitness value of an individual.
  - Each individual gets a portion of the circle corresponding to its fitness value.
  - A fixed point on the wheel is chosen and the wheel is spun. The region which comes in front of the fixed point is chosen as the parent.
  - The process is repeated to select the second parent.



Chromosome	Fitness Value
A	8.2
B	3.2
C	1.4
D	1.2
E	4.2
F	0.3

KNOWLEDGEGATE

- Example:
  - Chromosomes: 1, 2, 3, 4, 5
  - Fitness values: 29, 18, 16, 6, 26
  - Sum of fitness: 95
  - Probabilities: 0.305, 0.189, 0.168, 0.063, 0.274
  - Cumulative Probabilities: 0.305, 0.494, 0.662, 0.726, 1.000
  - If r=0.65, select chromosome 3 since $0.494 < 0.65 \leq 0.662$.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Crossover

- **Process**: Taking more than one parent solution (chromosomes) and producing a child solution from them.
- **Method**: Re-combine portions of good solutions to create better solutions.
- Example:
  - Parent 1: ABCDE|FGH
  - Parent 2: EBCDA|HAF
  - Offspring: ABCDEHAF
- **Goal**: Reproduction using recombination to combine the best features of parent solutions.

KNOWLEDGEGATE

- Types of Crossover:
  - **<u>One-Point Crossover</u>**
    - Process: A single crossover point is selected, and the segments to the right of this point are swapped between the two parents.
    - Example:
      - Parent 1: 101|1011
      - Parent 2: 010|1001
      - Offspring 1: 101|1001
      - Offspring 2: 010|1011

KNOWLEDGEGATE

- **Two-Point Crossover**
  - **Process**: Two crossover points are selected, and the segments between these points are swapped between the two parents.
  - **Example**:
    - Parent 1: 10|110|11
    - Parent 2: 01|011|01
    - Offspring 1: 10|011|11
    - Offspring 2: 01|110|01

- **Uniform Crossover**
  - **Process**: Each gene is treated separately, and we decide for each gene whether to swap it based on a predefined probability or using a binary mask.
  - Example:
    - Parent 1: 00101100
    - Parent 2: 11010011
    - Binary Mask: 11010110
    - Child 1: 01101100 (biased to Parent 1)
    - Child 2: 10010011 (biased to Parent 2)

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Half-Uniform Crossover**
  - Method:
    - Calculate the Hamming distance (number of different bits) between Parent 1 and Parent 2.
    - Swap the bits in Offspring 1 and 2 if the bits are different; otherwise, they remain the same.
  - Example:
    - Parent 1: 11100010
    - Parent 2: 01011011
    - Hamming Distance = 4
    - Offspring 1: 01101010
    - Offspring 2: 11010001

- **Three-Parent Crossover**
  - Method:
    - Three parents are taken.
    - Each bit of Parent 1 is compared with each bit of Parent 2.
    - If the bit is the same in both parents, it is taken in the offspring. If different, the bit from Parent 3 is taken for the offspring.
  - Example:
    - Parent 1: A B C A D B E A
    - Parent 2: B D E A C D B E
    - Parent 3: D A B C D B E A
    - Offspring: D B C C D B B A

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

- **Shuffle Crossover**
  - Method:
    - A single crossover point is selected, dividing the chromosome into two parts.
    - Shuffle bits (genes) in each parent using any logic.
    - After shuffling, mix modified parents as in the single crossover method.
  - Example:
    - Parent 1: 1 2 3 4 | 5 6 7 8
    - Parent 2: 8 1 7 6 | 2 5 3 4
    - Shuffled Parent 1: 3 5 1 4 2 7 6 8
    - Shuffled Parent 2: 5 7 8 6 1 2 3 4
    - Offspring 1: 3 5 1 4 5 7 8 6
    - Offspring 2: 5 7 8 6 2 7 6 8

KNOWLEDGEGATE

Mutation

- **Objective**: Encourage genetic diversity among solutions and attempt to provide genetic algorithms with solutions that are slightly different from the previous generation. Mutation alters one or more gene values in a chromosome from its initial state. With the new gene values, the genetic algorithm may be able to arrive at better solutions.

- **Process**: A small change in the DNA segment of a chromosome to produce an improved solution.
- **Example**:
  - Before Mutation: ABCDEHAF
  - After Mutation: ABCDFHAF
- **Goal**: Introduce small changes to maintain diversity and avoid local optima.

KNOWLEDGEGATE

- Types of Mutation:
  - **Flip Bit Method (Flipping)**
    - **Description**:
      - For a binary encoded chromosome, the corresponding bit is flipped (0 to 1 or 1 to 0).
    - **Example**:
      - Offspring Chromosome: 00100100
      - Mutation: Flip the second bit.
      - Mutated Chromosome: 01100100
  - **Boundary Method**
    - **Description**:
      - The mutation operator replaces the value of the chosen gene with either the upper or lower bound for that gene (chosen randomly).
    - Example:
      - Offspring Chromosome: 12543173
      - Mutation: Replace the first gene with its upper bound.
      - Mutated Chromosome: 17543173

KNOWLEDGEGATE

- **Interchange Method**
  - Description:
    - Two positions of the child's chromosome are chosen randomly, and the bits corresponding to these positions are interchanged.
  - Example:
    - Offspring Chromosome: 10100100
    - Mutation: Interchange the second and sixth bits.
    - Mutated Chromosome: 11100000
- **Reversing Method**
  - Description:
    - A position is chosen at random, and the bit next to that position is reversed to produce a mutated child.
  - Example:
    - Offspring Chromosome: 10101100
    - Mutation: Reverse the bit next to the third position.
    - Mutated Chromosome: 10001100

KNOWLEDGEGATE

Convergence & Convergence Criteria in Genetic Algorithms

- **Convergence in GA**:
  - For any problem, if a Genetic Algorithm (GA) is correctly implemented, the population (set of solutions) evolves over successive generations with fitness values increasing toward the global optimum.

- **Convergence**:
  - The progression toward increasing uniformity. A population is said to have converged when 95% of the individuals (solutions) constituting the population have the same fitness value.

- Convergence (Termination) Criteria:
  - Each iteration should be tested with some convergence test.

KNOWLEDGEGATE

- Various Convergence (Termination) Criteria:
  - **Objective Criteria**: A solution is found that satisfies the objective criteria.
  - **Fixed Number of Generations**: A predetermined number of generations is executed.
  - **Allocated Budget**: The allocated computational time or resources are exhausted.
  - **Fitness Threshold**: The highest-ranking solution's fitness is reaching or has reached a point such that successive iterations no longer produce better results.
  - **Manual Inspection**: Convergence is determined through manual inspection.
  - **Combination**: A combination of the above criteria is used.

KNOWLEDGE GATE

Applications of Genetic Algorithms

- **Optimization**- GAs are most commonly used for optimization problems where we have to maximize or minimize a given objective function under a set of constraints.
- **Neural Networks**- GAs are used to train neural networks, especially recurrent neural networks.
- **Image Processing**- GAs are used for various digital image processing tasks.
- **DNA Analysis**- GAs have been used to determine the structure of DNA.
- **Traveling Salesman Problem**- GAs are used to solve the Traveling Salesman Problem (TSP).

KNOWLEDGEGATE

- **Scheduling Applications**- GAs are used in solving various scheduling problems.
- **Machine Learning**- GAs are used in GBML (Genetic-Based Machine Learning).
- **Parametric Design of Aircraft**- GAs are used to design aircraft by varying parameters and evolving better solutions.
- **Robotics**- GAs are used to create learning robots which behave like humans.
- **Economics**- GAs are used to characterize various economic models like Cobweb models, game theory, equilibrium resolution, and asset pricing, etc.

KNOWLEDGEGATE

GA Approach:

1. **Chromosome Representation**: A permutation of city indices.
   - Example: For 5 cities, a chromosome could be `[1, 3, 5, 2, 4]`.
2. **Fitness Function**: The total distance of the route represented by the chromosome.
   - Calculate the distance for each route permutation and use the inverse of this distance as the fitness value (shorter distances have higher fitness).
3. **Selection**: Roulette wheel selection or tournament selection to choose parent chromosomes based on fitness.
4. **Crossover**: Order crossover (OX) where segments of parents are exchanged while maintaining city sequence.
   - Example: Parent1 = `[1, 3, 5, 2, 4]`, Parent2 = `[2, 5, 1, 4, 3]`
   - Offspring could be `[1, 3, 5, 4, 2]` and `[2, 5, 1, 3, 4]`.
5. **Mutation**: Swap mutation where two cities in the chromosome are swapped to introduce variation.
   - Example: Mutate `[1, 3, 5, 2, 4]` to `[1, 4, 5, 2, 3]`.
6. **Replacement**: Replace the least fit chromosomes in the population with the new offspring.
7. **Termination**: Continue for a set number of generations or until no significant improvement is observed.

KNOWLEDGE GATE
e.in/gate

Step 1: Initialization

- **Cities**: Assume 5 cities with the following distances between them:

	1	2	3	4	5
1	0	10	8	9	7
2	10	0	6	4	5
3	8	6	0	7	3
4	9	4	7	0	6
5	7	5	3	6	0

- **Initial Population**: Let's assume an initial population of 4 chromosomes:

$$\begin{aligned} &\text{Chromosome 1} && [1, 3, 5, 2, 4] \\ &\text{Chromosome 2} && [2, 5, 1, 4, 3] \\ &\text{Chromosome 3} && [3, 4, 2, 1, 5] \\ &\text{Chromosome 4} && [5, 2, 4, 3, 1] \end{aligned}$$

KNOWLEDGEGATE

Step 2: Calculate Fitness

- **Chromosome 1**: $[1, 3, 5, 2, 4]$
  - Distance: $1 \to 3 \to 5 \to 2 \to 4 \to 1 = 8 + 3 + 5 + 4 + 9 = 29$
  - Fitness: $\frac{1}{29} \approx 0.0345$
- **Chromosome 2**: $[2, 5, 1, 4, 3]$
  - Distance: $2 \to 5 \to 1 \to 4 \to 3 \to 2 = 5 + 7 + 9 + 7 + 6 = 34$
  - Fitness: $\frac{1}{34} \approx 0.0294$
- **Chromosome 3**: $[3, 4, 2, 1, 5]$
  - Distance: $3 \to 4 \to 2 \to 1 \to 5 \to 3 = 7 + 4 + 10 + 7 + 3 = 31$
  - Fitness: $\frac{1}{31} \approx 0.0323$
- **Chromosome 4**: $[5, 2, 4, 3, 1]$
  - Distance: $5 \to 2 \to 4 \to 3 \to 1 \to 5 = 5 + 4 + 7 + 8 + 7 = 31$
  - Fitness: $\frac{1}{31} \approx 0.0323$

KNOWLEDGE GATE
n/gate

Step 3: Selection

- Using roulette wheel selection, we select parents based on their fitness.

Step 4: Crossover

- **Parents**: Let's select Chromosome 1 and Chromosome 2 for crossover.
- **Order Crossover (OX)**:
  - Parent 1: $[1, 3, 5, 2, 4]$
  - Parent 2: $[2, 5, 1, 4, 3]$
  - Offspring 1: $[1, 3, 5, 4, 2]$
  - Offspring 2: $[2, 5, 1, 3, 4]$

Step 5: Mutation

- **Offspring 1 before mutation**: $[1, 3, 5, 4, 2]$
  - Swap mutation: Swap city 3 and city 5.
  - **Offspring 1 after mutation**: $[1, 5, 3, 4, 2]$
- **Offspring 2 before mutation**: $[2, 5, 1, 3, 4]$
  - Swap mutation: Swap city 1 and city 4.
  - **Offspring 2 after mutation**: $[2, 5, 4, 3, 1]$

KNOWLEDGEGATE
n/gate

Step 6: Replacement

- Replace the least fit chromosomes in the population with the new offspring.
- **New Population:**

Chromosome 1	$[1, 3, 5, 2, 4]$
Chromosome 2	$[3, 4, 2, 1, 5]$
Offspring 1	$[1, 5, 3, 4, 2]$
Offspring 2	$[2, 5, 4, 3, 1]$

Step 7: Termination

- Continue the process for a set number of generations or until no significant improvement is observed.

KNOWLEDGEGATE

http://www.knowledgegate.in/gate

Genetic Algorithm: Advantages

- **Global Search Capability**:
  - **Example**: In optimizing the design of an aircraft wing, genetic algorithms can explore a wide range of shapes and configurations to find the most aerodynamic design.
- **No Need for Derivative Information**:
  - **Example**: For complex financial modeling where the relationship between variables is not clearly defined, genetic algorithms can optimize investment strategies without needing gradient information.
- **Adaptability**:
  - **Example**: In dynamic routing for delivery trucks, genetic algorithms can adapt to changes in traffic conditions or delivery requirements, continually finding optimal routes.
- **Versatility**:
  - **Example**: Genetic algorithms can be used for a variety of problems, such as scheduling tasks in a manufacturing plant, optimizing investment portfolios, or designing efficient network topologies.

KNOWLEDGEGATE

Genetic Algorithm: Disadvantages

- **High Computational Cost**:
  - **Example**: In a complex scheduling problem for a large manufacturing plant, the computational resources required to evaluate many potential schedules can be substantial.
- **Parameter Sensitivity**:
  - **Example**: When optimizing a neural network's architecture, the performance of the genetic algorithm can vary significantly based on the chosen mutation and crossover rates, requiring careful tuning.
- **Premature Convergence**:
  - **Example**: In a genetic algorithm optimizing a chemical process, if the population quickly converges to a local optimum, it may miss better solutions, leading to suboptimal process parameters.
- **Complexity in Implementation**:
  - **Example**: Implementing a genetic algorithm for optimizing the layout of electronic circuits can be complex, requiring detailed knowledge of genetic operations and careful parameter tuning.

KNOWLEDGEGATE
http://www.knowledgegate.in/gate

Genetic Programming (GP)

- Genetic Programming (GP) is an evolutionary algorithm-based methodology inspired by biological evolution to find computer programs that perform a user-defined task. It is a specialized form of genetic algorithms (GAs) where the individuals in the population are computer programs.

- Example Workflow:
  - **Initialization**: Generate an initial population of random programs.
  - **Evaluation**: Assess the fitness of each program using the fitness function.
  - **Selection**: Select the fittest programs to act as parents for the next generation.
  - **Crossover**: Combine parts of two parent programs to create offspring programs.
  - **Mutation**: Apply random changes to some programs to introduce variability.
  - **Iteration**: Repeat the evaluation, selection, crossover, and mutation steps for many generations.

KNOWLEDGEGATE

- **Example Application**:
  - **Symbolic Regression**: GP can be used to find a mathematical expression that best fits a set of data points. For instance, given a dataset of input-output pairs, GP can evolve an equation that models the relationship between inputs and outputs.

KNOWLEDGEGATE